RapidResponse Analytic and Data Model Guide

1908

kinaxis®
RapidResponse®

All Specifications, claims, features, representations, and/or comparisons provided are correct to the best of our knowledge as of the date of publication, but are subject to change without notice. While we will always strive to ensure our documentation is accurate and complete, this document may also contain errors and omissions of which we are not aware.

THIS INFORMATION IS PROVIDED BY KINAXIS ON AN "AS IS" BASIS, WITHOUT ANY OTHER WARRANTIES OR CONDITIONS, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABLE QUALITY, SATISFACTORY QUALITY, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, OR THOSE ARISING BY LAW, STATUTE, USAGE OF TRADE, COURSE OF DEALING OR OTHERWISE. YOU ASSUME THE ENTIRE RISK AS TO THE RESULTS OF THE INFORMATION PROVIDED. WE SHALL HAVE NO LIABILITY TO YOU OR ANY OTHER PERSON OR ENTITY FOR ANY INDIRECT, INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER, INCLUDING, BUT NOT LIMITED TO, LOSS OF REVENUE OR PROFIT, LOST OR DAMAGED DATA OR OTHER COMMERCIAL OR ECONOMIC LOSS, EVEN IF WE HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES, OR THEY ARE FORESEEABLE. WE ARE ALSO NOT RESPONSIBLE FOR CLAIMS BY A THIRD PARTY. OUR MAXIMUM AGGREGATE LIABILITY TO YOU AND THAT OF OUR DEALERS AND SUPPLIERS SHALL NOT EXCEED THE COSTS PAID BY YOU TO PURCHASE THIS DOCUMENT. SOME STATES/COUNTRIES DO NOT ALLOW THE EXCLUSION OR LIMITATION OF LIABILITY FOR CONSEQUENTIAL OR INCIDENTAL DAMAGES, SO THE ABOVE LIMITATIONS MAY NOT APPLY TO YOU. All product, font and company names are trademarks or registered trademarks of their respective owners, and use of them does not imply any affiliation with or endorsement by them.

Copyright © 1999 -2019 Kinaxis Inc. All rights reserved. This document may not be reproduced or transmitted in any form (whether now known or hereinafter discovered or developed), in whole or in part, without the express prior written consent of Kinaxis.

Kinaxis RapidResponse contains technology which is protected under US Patents #7,610,212, #7,698,348, #7,945,466, #8,015,044, #9,292,573, #9,710,501, and #10,467,337 by the United States Patent and Trademark Office. It is also protected under the Japan Patent Office, Japanese patent #4393993 and the Intellectual Property India Office, patents #255768 and #279101. Other patents are pending.

This document may include examples of fictitious companies, products, Web addresses, email addresses, and people. No association with any real company, product, Web address, email address, and person is intended or should be inferred.

RapidResponse Analytic and Data Model Guide (E76301-20162)

Date of Publication: Thursday, December 12, 2019

Published in Canada.

Support: support@kinaxis.com

Web site: http://www.kinaxis.com

Contents

Table index **25**
A 25
B 25
C 26
D 27
E 27
F 27
H 28
I 28
J 29
K 29
L 29
M 29
N 29
O 29
P 30
R 31
S 31
T 33
U 33
V 33
W 33
Field index **35**
A 35
B 39
C 41
D 48
E 55
F 59
G 62
H 63
I 64
K 69
L 69
M 72
N 75
O 77
P 80

Q 89
R 91
S 93
T 103
U 107
V 108
W 110
Y 111
Z 111
CHAPTER 1: Welcome 113
What's in this Guide 113
About Kinaxis 114
CHAPTER 2: Accessing help and documentation 115
Determining which help system or guide to use 116
Documentation conventions 118
Finding the content you need in HTML help 118
Your feedback 119
Help embedded in resources 120
Other ways to learn about RapidResponse 125
Customer Support 126

Part 1: Table and field reference 129

CHAPTER 3: RapidResponse database 131
Namespaces 131
Table types 133
Historical tables 134
Vector data 138
Field types 140
Data types 143
Invalid characters in data 145
Accuracy of calculated results with long numbers 146
Data Model dialog box 147
CHAPTER 4: Introduction to the RapidResponse data model 149
RapidResponse data 150
Products and structure 151
Activity 152
Controls 159
CHAPTER 5: Optional applications and tables 165
Optional analytics and the data model 165
Optional RapidResponse capabilities 166
Optional RapidResponse applications 170
CHAPTER 6: Input tables 171
ABCCode table 176
Activity table 176
AggregatePartCustomer table 179
Allocation table 182
AllotmentOverride table 187
AllotmentOverrideDetail table 189
AlternatePart table 190

AlternateRouting table ... 193
Assignment table ... 194
Assumption table ... 195
AssumptionByConstraint table ... 197
AssumptionByPart table ... 197
AssumptionByPartCustomer table ... 198
AssumptionByPartSource table ... 198
AssumptionStatus table ... 199
AssumptionType table ... 199
AttributeMap table ... 200
BaselineConflict table ... 204
BaselinePlannedOrder table ... 206
BaselineTaskLink table ... 207
BestFitProfile table ... 209
BestFitProfileDetail table ... 210
BillOfMaterial table ... 217
BOMAlternate table ... 229
BonusSchedule table ... 230
BonusScheduleByDate table ... 231
BonusScheduleByInterval table ... 232
BuyerCode table ... 233
Capacity table ... 234
CapacityOverride table ... 236
Carrier table ... 238
CausalFactor table ... 239
CausalFactorCategory table ... 240
CausalFactorDetail table ... 241
Class table ... 242
Cluster table ... 243
ClusterProperty table ... 244
CollaborativeForecast table ... 245
ConsensusForecastRollingHorizon table ... 246
Constraint table ... 247
ConstraintAvailable table ... 249
ConstraintGroup table ... 251
ConstraintMetric table ... 252
ConstraintMetricValues table ... 253
ControlSet table ... 253
CRPOperation table ... 256
CRPOperationState table ... 260
CurrencyConversionForecast table ... 263
CurveParameters table ... 263
CurvePoint table ... 265
Customer table ... 266
CustomerDestination table ... 269
CustomerGroup table ... 269
CustomerPrice table ... 270
DeliveryRoute table ... 271
DemandOrder table ... 272
Department table ... 274

EngineeringChange table ... 274
Event table ... 276
EventPhase table ... 276
EventPhaseHeader table ... 279
EventType table ... 280
Feature table ... 281
FeatureMap table ... 281
ForecastDetail table ... 283
ForecastDisaggregationParameters table ... 287
ForecastDisaggregationParametersByCategory table ... 288
ForecastDisaggregationOverride table ... 290
ForecastItem table ... 292
ForecastItemFitOutput table ... 294
ForecastItemParameters table ... 296
ForecastItemParametersFitOutput table ... 313
ForecastItemParametersMap table ... 321
ForecastItemParametersOutlier table ... 322
ForecastItemParametersRegressionCoefficient table ... 322
HistoricalCurrencyConversion table ... 324
HistoricalCurrencyHeader table ... 325
HistoricalDemandActual table ... 325
HistoricalDemandCategory table ... 329
HistoricalDemandCategoryRollingWeight table ... 333
HistoricalDemandHeader table ... 334
HistoricalDemandHeaderRollingWeight table ... 336
HistoricalSupplyOrder table ... 337
HistoricalDemandHeaderTimePhasedAttributes table ... 338
HistoricalDemandOrder table ... 339
HistoricalDemandSeries table ... 340
HistoricalDemandSeriesDetail table ... 341
HistoricalPartKPI table ... 343
HistoricalReceipt table ... 345
HistoricalReceiptHeader table ... 345
HistoricalShipment table ... 346
HistoricalSupplyActual table ... 347
HistoricalSupplyCategory table ... 350
HistoricalSupplyHeader table ... 351
HistoricalSupplySeries table ... 353
HistoricalSupplySeriesDetail table ... 354
HoldCode table ... 355
ImportedStatisticalForecast table ... 356
IndependentDemand table ... 358
IndependentDemandFeatureList table ... 369
IndependentDemandSolution table ... 370
InventoryTransfer table ... 374
ItemSource table ... 379
ItemSourceFitOutput table ... 386
ItemSourceFitOutputClass table ... 387
ItemSourceFitOutputForecastModel table ... 388
ItemSourceFitOutputValueName table ... 388

ItemSourceHistoricalSupplyCategory table ... 389
Job table ... 389
KPI table ... 390
Location table ... 391
LogisticsLocation table ... 392
MEIOLeadTime table ... 393
MEIOLeadTimeDistributionProfile table ... 396
MEIOLeadTimeDistributionProfilePoint table ... 397
MEIOLeadTimeOutlier table ... 398
Metric table ... 399
MetricAsOf table ... 400
Model table ... 401
Notification table ... 402
OnHand table ... 404
Operation table ... 407
OperationSequence table ... 410
OriginatingFile table ... 413
OriginatingSource table ... 413
OverrideMajorChangeover table ... 414
Part table ... 415
PartCustomer table ... 444
PartCustomerTimePhasedAttributes table ... 449
PartMetric table ... 451
PartMetricValues table ... 452
PartSequence table ... 452
PartSolution table ... 453
PartSource table ... 459
PartSourceTimePhasedAttributes table ... 476
PartSupplier table ... 477
PartUOMConversion table ... 478
PartValidation table ... 479
PenaltySchedule table ... 480
PenaltyScheduleByDate table ... 481
PenaltyScheduleByInterval table ... 482
PlannerCode table ... 483
PointsCurve table ... 484
PointsProfile table ... 485
Pool table ... 486
PoolMap table ... 487
PoolMapOverride table ... 487
Predecessor table ... 488
ProcessInstance table ... 491
ProductFamily table ... 493
ProductionGroup table ... 493
ProductionRecurrence table ... 495
ProductionWheel table ... 496
ProductionWheelConstraint table ... 498
ProfilePoint table ... 499
Project table ... 500
ProjectedLeadTime table ... 505

ProjectedYield table 506
ProjectGroup table 507
ProjectManager table 508
ProjectStatus table 508
RangeOfCoveragePartOverride table 509
ReferenceForecastDimension table 511
ReferenceForecastTarget table 512
ReferencePart table 513
Region table 514
RegionGroup table 514
RegressionModel table 515
RegressionModelTerm table 516
Regressor table 517
RegressorCategory table 518
RegressorValue table 518
Resource table 519
ResourceAssignment table 522
ResourceAssignmentAvailable table 525
ResourceAvailable table 526
ResourceCalendarException table 527
ResourceCostActual table 528
ResourceCosts table 530
ResourceGroup table 531
ResourceGroupProjectOverride table 532
ResourceGroupTaskOverride table 533
ResourceProjectOverride table 534
Routing table 534
RParameter table 536
RParameterSet table 537
SafetyLeadTimeProfile table 538
SafetyLeadTimeProfileDetail table 539
SafetyStockAverageDemandProfile table 540
SafetyStockItem table 541
SafetyStockItemMapping table 554
SafetyStockItemOutlier table 556
SafetyStockOverride table 557
SafetyStockPart table 560
SafetyStockRangeOfCoverage table 565
SafetyStockTimePhasedBounds table 565
SAP_MD04 table 567
ScheduledReceipt table 569
Segment table 589
SequenceFlow table 590
Shipment table 590
ShipSet table 592
Shop table 593
ShopKanban table 594
Site table 596
SiteManager table 600
SiteManagerType table 601

SiteMetric table ... 602
SiteMetricValues table ... 602
Source table ... 603
SourceConstraint table ... 605
SourceKanban table ... 607
StorageType table ... 610
SubProcess table ... 610
SubstituteGroup table ... 611
Supplier table ... 612
SupplierGroup table ... 614
SupplierMetric table ... 615
SupplierMetricValues table ... 616
SupplyAllocation table ... 616
SupplyOrder table ... 618
Task table ... 624
TaskDemandLink table ... 646
TaskHyperlink table ... 648
TaskPartLink table ... 648
TaskSupplyLink table ... 650
TimePhasedDemandParameterSet table ... 653
TimePhasedLocationCapacity table ... 653
TimePhasedMaximumInventory table ... 654
TimePhasedPartSourceLeadTime table ... 654
TimePhasedSafety table ... 655
TimePhasedSourceLeadTime table ... 660
TimePhasedYield table ... 661
TransferDestination table ... 662
TransportationMode table ... 663
UnitOfMeasureCategory table ... 664
Warehouse table ... 665
WBS table ... 666
WorkCenter table ... 666
CHAPTER 7: Control tables ... 669
AggregatePartCustomerType table ... 670
AllotmentOverrideType table ... 671
AlternatePartType table ... 672
AnalyticsConfiguration table ... 674
AttributeMapType table ... 676
AutomatedLeadTimeType table ... 678
AutomatedYieldType table ... 683
AvailableRule table ... 686
BOMType table ... 687
Calendar table ... 695
CalendarDate table ... 700
ConstraintType table ... 700
ConstraintUnitOfMeasure table ... 703
CRPUnitOfMeasure table ... 704
DaysSupplyPolicy table ... 705
DemandStatus table ... 713
DemandType table ... 715

DDMRPRule table ... 721
DistributionPlanningRule table ... 722
DRPConfiguration table ... 724
ExpiryType table ... 726
ForecastItemParametersType table ... 727
HistoricalDemandCategoryType table ... 736
IMConfiguration table ... 739
IncrementalRule table ... 741
InventoryTransferType table ... 741
IPOConfiguration table ... 743
MEIOLeadTimeType table ... 744
MPSConfiguration table ... 746
MUEPoolNettingType table ... 749
MultiEchelonSafetyStockRule table ... 756
MultiLevelSearchRule table ... 758
OFConfiguration table ... 767
OnHandType table ... 768
OperationSequenceType table ... 768
OperationType table ... 770
OrderPolicy table ... 773
OrderPriority table ... 782
OutlierType table ... 792
PartSourceType table ... 797
PartType table ... 809
PlanningCalendars table ... 843
ProcessInstanceType table ... 848
ProductionWheelType table ... 849
ProjectType table ... 850
RangeOfCoverage table ... 855
ResourceType table ... 860
SafetyStockItemType table ... 861
SafetyStockPolicy table ... 874
ScenarioSetting table ... 887
ShipmentPolicy table ... 887
ShipSetType table ... 888
SHSCConfiguration table ... 889
SOPAnalyticsConfiguration table ... 892
SourceRule table ... 895
SpreadProfile table ... 899
SubstituteGroupType table ... 901
SupplyAllocationType table ... 903
SupplyStatus table ... 904
SupplyType table ... 919
TaskType table ... 925
ToleranceProfile table ... 930
ToleranceProfileZone table ... 930
UnitOfMeasure table ... 931
WorkCenterType table ... 933
CHAPTER 8: Calculated tables ... 937
Activity table ... 940

Tables referenced by the Activity table 951
AvailableToPromise table 956
BillOfMaterialFeatureRule table 957
BlowThroughNewAllocation table 958
BlowThroughPlannedAllocation table 959
BOM_MUECUMLead table 960
CoByProductSupply table 961
Conflict table 963
ConsensusForecast table 966
ConsensusForecastDetail table 970
ConsensusForecastRollingHorizonWeight table 971
ConsensusForecastWeightByHeader table 972
ConstraintLoad table 973
ConstraintSpreadLoad table 974
ConstraintUsed table 975
CostOfGoodsSold table 977
CostRollUp table 982
CriticalPath table 984
CTPActivity table 985
CTPCoByProductSupply table 996
CTPForecast table 998
CTPForecastCost table 999
CTPPlannedOrder table 1000
CTPSafetyStockHoldings table 1002
CTPSubstituteSupply table 1003
DataChange table 1005
DataChange_Historical table 1008
DataChange_Historical_Split table 1011
DataChange_Local table 1014
DataChange_Local_Split table 1016
DataChange_PendingUpdate table 1019
DataChange_PendingUpdate_Split table 1022
Demand table 1025
DemandException table 1028
DependentConsensusForecast table 1032
DependentDemand table 1033
DirectAssembly table 1036
DirectComponent table 1036
DisaggregationRateByPartCustomer table 1037
EventConsensusForecastDetail table 1039
EventDisaggregationRate table 1041
EventForecastDetailAdjustment table 1041
EventStatisticalDisaggregationRate table 1043
EventStatisticalForecastDetail table 1044
EventStatisticalForecastDetailAdjustment table 1044
FeatureBOMForIndependentDemand table 1045
FlatAssembly table 1046
FlatBillDown table 1046
FlatBillUp table 1051
FlatComponent table 1054

FlatDemand table ... 1055
FlatConstraint table ... 1058
FLPConstraint table ... 1058
Forecast table ... 1059
ForecastConsumption table ... 1063
ForecastItemParametersActual table ... 1065
ForecastItemParametersRegressionWeight table ... 1066
ForecastItemParametersRegressor table ... 1067
HistoricalDemandWaterfall table ... 1068
HistoricalSupplyWaterfall table ... 1070
IndependentDemandByFeature table ... 1072
IndependentDemandCost table ... 1073
IndependentDemandFeature table ... 1074
IndependentDemandFeatureSummary table ... 1075
InProcess table ... 1075
InventoryAnalysis table ... 1078
InventoryCTPAnalysis table ... 1081
InventorySummary table ... 1084
ItemSourceCandidate table ... 1087
ItemSourceIntervalEffectivePartSource table ... 1088
ItemSourcePartSourceEffectivityInterval table ... 1089
ItemSourcePartSourceHistoricalLeadTime table ... 1089
ItemSourcePartSourceHistoricalReceipt table ... 1091
LateSupply table ... 1092
LoadDailyCurrent table ... 1097
LoadDailyPlanned table ... 1098
LoadDetailCurrent table ... 1100
LoadDetailPlanned table ... 1102
LoadFLPPlanned table ... 1104
LoadOperationNewOrderPlanned table ... 1106
LoadOperationReceiptsCurrent table ... 1108
LoadOperationReceiptsPlanned table ... 1110
LoadWeeklyCurrent table ... 1112
LoadWeeklyPlanned table ... 1114
MaximumInventoryResult table ... 1115
MEIODriverPart table ... 1116
MEIOFamilyResult table ... 1117
MEIOLeadTimeOutlierResult table ... 1117
MEIOLeadTimeOutlierResultSummary table ... 1119
MEIOHistoricalSupply table ... 1121
MEIOParentPart table ... 1123
MEIOStage table ... 1123
MEIOStageHistoricalDemand table ... 1127
MEIOStageHistoricalForecast table ... 1127
MEIOStageLink table ... 1128
MEIOStageResult table ... 1128
MEIOStageTimePhasedResult table ... 1130
MLSActivity table ... 1131
NewAllocation table ... 1137
NewTransferAllocation table ... 1138

Numbers table 1138
Part_MUECumLead table 1139
Part_MUEPoolValidation table 1140
PartSourceCTPPlannedOrder table 1141
PartSourcePlannedOrder table 1141
PartSourceScheduledReceipt table 1142
PeriodConstraintLoad table 1142
PeriodsOfCoverage table 1144
PeriodsOfCTPCoverage table 1145
PlannedAllocation table 1147
PlannedOrder table 1148
PlannedTransferAllocation table 1154
ProductionCycle table 1155
ResourceDataByDate table 1156
ResourceLoad table 1157
SafetyStockHistoricalDemand table 1161
SafetyStockHistoricalForecast table 1162
SafetyStockHistoricalSupply table 1163
SafetyStockItemDemandParameterSet table 1164
SafetyStockItemFutureDemand table 1165
SafetyStockItemOutlierResult table 1166
SafetyStockItemOutlierResultSummary table 1167
SafetyStockResult table 1169
SafetyStockTimePhasedResult table 1174
ScheduledReceiptCost table 1178
StatisticalForecast table 1179
StatisticalForecastDetail table 1181
StatisticalForecastDisaggregationRate table 1181
StatisticalForecastFitOutput table 1182
StatisticalForecastOutlier table 1192
StatisticalForecastOutlierSummary table 1194
StatisticalForecastPredictActual table 1196
StatisticalForecastRegressionCoefficient table 1197
StatisticalForecastResidualAutocorrelation table 1198
StatisticalForecastResidualCount table 1199
SubstituteSupply table 1200
Supply table 1202
SupplyDemand table 1205
TaskDemandLinkCost table 1215
TaskPartLinkCost table 1215
TaskSupplyLinkCost table 1216
TrialCoByProductSupply table 1216
TrialConstraintShortage table 1218
TrialConsumption table 1219
TrialPlannedOrder table 1220
TrialSubstituteSupply table 1222
ValidatePlannedOrder table 1224
WhereConsumed table 1225
WhereConsumedForDemand table 1234
WhereConsumedForForecast table 1243

WhereConsumedForSupply table ... 1252
WhereConsumedToHighVolumeOrder table ... 1262
WorkCenterCapacity table ... 1265
CHAPTER 9: Notes tables ... 1269
EngineeringChange_Notes table ... 1270
ForecastItemParametersOutlier_Notes table ... 1270
IndependentDemand_Notes table ... 1270
IndependentDemandSolution_CustomerNotes table ... 1271
IndependentDemandSolution_SupplyNotes table ... 1271
Mfg_SupplyOrder_TextLine table ... 1272
ScheduledReceipt_Notes table ... 1272
SupplyAllocation_Notes table ... 1273
Task_Notes table ... 1273
CHAPTER 10: Referenced system tables ... 1275
Currency table ... 1275
CurrencyConversionActual table ... 1276
RParameterName table ... 1277
TimeZone table ... 1278

Part 2: Analytics ... 1283

CHAPTER 11: Overview of RapidResponse analytics ... 1285
Core analytics ... 1285
Analytic modifiers ... 1287
Analytic compatibility and limitations ... 1291
CHAPTER 12: Calendars and date calculations ... 1295
Calendar tables ... 1295
Dates and time spans ... 1296
RapidResponse date values ... 1297
Date constants ... 1303
RunDate and DataDate ... 1303
Date calculation rules ... 1304
Calendar date display values ... 1306
CHAPTER 13: Lead time calculations ... 1307
Lead time dates ... 1307
Lead time elements ... 1308
Lead time calendars ... 1317
Inclusive and cumulative lead time ... 1323
Lead time rounding ... 1327
Lead time overrides from scheduled receipts ... 1328
CHAPTER 14: Material Requirements Planning calculations ... 1331
Netting basics ... 1331
Rescheduling supplies ... 1335
Explosion ... 1337
Explosion and order BOMs ... 1342
Planned orders ... 1343
Planning time fence ... 1344
Planned orders with MRP rules ... 1351
Planned orders with order point ... 1351
Inventory levels and planned orders ... 1358

Order Policy and planned orders ... 1358
Planned order lot sizing ... 1358
Days of Supply calculations ... 1362
Shortage calculations ... 1375
Limitations on the number of planned orders ... 1376
Start date calculation for planned orders ... 1376
Spreading planned orders with Takt Time ... 1376
Part source selection ... 1377
Examples of part source selection ... 1382
Part sources for scheduled receipts ... 1389
Phantom processing ... 1390
Yield ... 1391
Supply ... 1396
Demand ... 1397
Allocation data ... 1399
Safety stock ... 1400
Safety stock policy configuration ... 1401
Basic safety stock rules ... 1402
Safety stock calculations ... 1402
Analytic limitations with safety stock ... 1418
Ignoring unsatisfied demands ... 1420
Recursive data ... 1423
CHAPTER 15: Master Production Scheduling calculations ... 1425
Demand types ... 1425
Forecast spreading ... 1426
Forecast consumption ... 1431
MPS configuration ... 1442
Dependent forecast on scheduled receipts ... 1446
Forecast consumption by pool/customer ... 1447
Available to promise ... 1449
CHAPTER 16: Inventory turns calculations ... 1451
CHAPTER 17: Capable-to-Promise calculations ... 1453
Capable-to-promise tables ... 1454
Sample data used in CTP examples ... 1454
Available date calculations ... 1456
Gating part calculations ... 1457
Gating constraint calculations ... 1458
Late supply calculations ... 1458
VMI parts and supply demand allocation records ... 1458
Allocation of supply to demand ... 1458
Allocating limited supplies ... 1464
Improve on-time quantity by reallocating late supply ... 1471
Incremental availability calculations ... 1476
Incremental availability basics ... 1476
Constraints and incremental availability ... 1478
Controlling incremental availability calculations ... 1479
Best practices for incremental availability results ... 1484
Analytic limitations with incremental availability ... 1487
CTP data examples ... 1488

CHAPTER 18: Full-level pegging calculations **1493**
Full-level pegging tables 1493
Full-level pegging data example 1496
Full-level pegging through allocations 1499
Full-level pegging reporting horizon 1501
CHAPTER 19: Cost of goods sold calculations **1503**
Allocated scheduled receipts 1503
Unallocated scheduled receipts 1505
Calculating PCOGS material costs 1505
Calculating PCOGS labor costs 1508
Calculating PCOGS overhead costs 1509
CHAPTER 20: Cost roll-up calculations **1511**
Calculating CR material costs 1511
Calculating CR labor costs 1512
Calculating CR overhead costs 1513
CHAPTER 21: Capacity Requirements Planning (CRP) calculations **1515**
Capacity Requirements Planning data model 1515
About Capacity Requirements Planning calculations 1519
Scheduled receipts and planned orders 1520
Routings 1521
Operations and sequences 1522
Operation phases and durations 1523
Sequencing and parallel operations 1524
Operation lot splitting 1526
Ongoing operations 1526
Overlapping operations 1527
Aligning dependent requirements to operations 1531
Operations and supply lead time 1532
Work centers and capacity 1534
Operation loads 1536
WorkCenter loads 1538
Limitations on other analytics with CRP 1540
CHAPTER 22: Kanban calculations **1541**
Enabling kanban calculations 1541
Impacts on netting calculations 1542
Impacts on capable-to-promise calculations 1542
Limitations on other analytics with kanban 1543
CHAPTER 23: Interchangeable parts **1545**
Setting up interchangeable parts 1545
Analytics with interchangeable parts 1549
CHAPTER 24: Global part substitution **1551**
Setting up part substitution 1551
Limit substitute usage to excess supply 1558
Enable customer-specific part substitution 1560
Part substitution calculations 1562
Limitations on other analytics 1562
CHAPTER 25: BOM level part substitution **1565**
Substitute BOM configuration 1566
Define substitutable components 1567
Specify usage of substitute supply 1569

Define planned order generation ... 1571
Specify date effectivity ... 1574
Substituting groups of components ... 1574
Substitutable allocations ... 1576
Limitations on other analytics ... 1578
CHAPTER 26: Co-products and by-products ... 1581
Co-product and by-product basics ... 1581
Co-product and by-product data model ... 1582
Configuring co-products and by-products ... 1583
Co-product and by-product calculations ... 1596
Limitations on other analytics ... 1598
CHAPTER 27: Material expiration ... 1601
Setting up expiry data ... 1601
Expiry calculations ... 1609
Calculating the earliest expiry date on a demand ... 1611
Calculating the expiry date on supplies ... 1616
Calculating the expiring quantity on a supply ... 1624
Unsourceable demands due to material expiry ... 1625
Limitations on other analytics ... 1626
CHAPTER 28: Constraint Analysis calculations ... 1629
Understanding constraint ... 1629
Constraint Analysis data model tables ... 1630
Constraints on part sources ... 1631
Families and multi-level constraints ... 1633
Constraint Analysis process overview ... 1635
Planning with fixed and variable constraints ... 1642
Planning with multiple constraints ... 1643
Planning with multi-level constraints ... 1644
Reporting constraint loads ... 1645
Constraint Analysis control settings ... 1651
ProcessingRule ... 1652
Rescheduling rules ... 1653
ConstraintDateRule ... 1654
OrderPriority rules ... 1657
SchedulePriority ... 1659
SplitLateSupply ... 1662
AllowLatePlan ... 1663
FillSchedule ... 1665
AvailableDateLimit ... 1666
SchedulingRule ... 1667
PrePlanRule ... 1676
FairShareAllocation ... 1679
Respecting order policy rules ... 1684
Control settings and calculated fields in Netting ... 1685
CHAPTER 29: Production Planning calculations ... 1691
Production Planning data model ... 1691
Managing production wheel constraint ... 1693
Handling unused constraints and unsatisfied supplies ... 1694
Changeover in production ... 1697
Part sources in production wheels ... 1698

Scheduled receipts and planned orders ... 1699
Production wheel durations and cycle lengths ... 1699
Production recurrence ... 1702
Limitations with other analytics ... 1703
CHAPTER 30: Campaign planning ... 1707
Campaign planning data model ... 1707
Configuring campaigns ... 1708
Campaign planning calculations ... 1710
Limitations on other analytics ... 1715
CHAPTER 31: Distribution planning ... 1717
Configuring distribution planning for pushing ... 1717
Distribution planning calculations ... 1718
Forward-bucketed fair share ... 1726
Limitations on other analytics ... 1730
CHAPTER 32: Multi-site calculations ... 1731
Sites ... 1731
Transfer part sources ... 1731
Transfer scheduled receipts ... 1734
Using address data to support maps ... 1735
CHAPTER 33: Model and Unit Effectivity calculations ... 1737
Model effectivity ... 1737
Unit effectivity ... 1738
Explosion based on effectivity ... 1739
Cumulative lead time ... 1741
Capacity load calculations ... 1742
CHAPTER 34: Inventory Pooling calculations ... 1745
Understanding pools ... 1745
Pool netting ... 1745
Pool, model and unit netting factors ... 1746
Pool, model and unit netting ... 1748
Pools and part sources ... 1750
CHAPTER 35: Alternate BOM calculations ... 1759
Alternate BOM basics ... 1759
Effects on Netting calculations ... 1761
Best practices for using alternate BOMs ... 1762
Default BOMAlternate record ... 1764
CHAPTER 36: Order priority calculations ... 1765
Order priority tables and fields ... 1765
Priority calculations ... 1767
Effective priority ... 1767
Effective transaction sequence ... 1769
Priorities on dependent demands ... 1770
Prioritized demand order ... 1771
CTP supply allotments ... 1771
Safety stock and priority adjustments ... 1773
Priority and safety stock holds in CTP ... 1780
Priorities on scheduled receipts ... 1788
Priorities on planned orders ... 1790
Constraint planning and usage ... 1794
Prioritized demand examples ... 1795

Prioritized supply examples ... 1799
CHAPTER 37: Two-date planning ... 1803
Two-date planning configuration ... 1803
Provide request date on independent demands ... 1804
Enable two-date planning by priority level ... 1805
Ensure supply sources can plan to request dates ... 1806
Apply safety lead time at request date (optional) ... 1807
Two-date planning calculations ... 1810
Limitations on other analytics ... 1814
CHAPTER 38: Attribute-based planning ... 1817
Supported analytics ... 1817
Configuring attributes ... 1818
Configure control table settings ... 1818
Create attributes ... 1820
Create and define attribute fields ... 1821
Define attribute rules ... 1825
Attribute calculations ... 1827
Supply allotment by attribute ... 1827
Forecast consumption by attribute ... 1830
Forecast adjustment by attribute ... 1831
Attribute-based planning examples ... 1832
CHAPTER 39: Feature BOMs ... 1845
Setting up Feature BOMs ... 1845
Feature BOM calculations ... 1848
Limitations on other analytics ... 1850
CHAPTER 40: Allotment overrides ... 1853
Setting up allotment overrides ... 1853
Allotment override calculations ... 1856
Allotment override examples ... 1857
CHAPTER 41: Multi-level search logic ... 1863
Multi-level search configuration ... 1863
Enabling multi-level search logic ... 1864
Multi-level part source effectivity ... 1865
Partial planning ... 1866
Fairshare of material and constraints ... 1868
Including material cost as a selection criteria ... 1869
Reducing short-term excess inventory ... 1870
Limiting the number of part sources evaluated ... 1870
Other control settings to consider ... 1871
Pre-defined resources supporting MLS ... 1873
Multi-sourcing ... 1876
Multi-level part source selection criteria ... 1877
Substitution ... 1909
Non-blocking allocations ... 1916
Limitations on other analytics ... 1926
MLS infrastructure changes (optional and limited release) ... 1929
CHAPTER 42: Inventory planning and optimization ... 1935
Single and multi-echelon basics ... 1936
Safety stock item data model ... 1937
Single-echelon data model ... 1938

Multi-echelon data model ... 1942
Safety stock item configuration ... 1946
Define IP&O parameters ... 1947
Provide historical data for safety stock calculations ... 1947
Configure control settings for safety stock calculations ... 1948
Set safety stock bounds (optional) ... 1959
Configure multi-echelon lead time parameters (optional) ... 1960
Define safety stock item mappings (optional) ... 1962
Define single-echelon average demand profiles (optional) ... 1962
Use forecast error pooling for safety stock (optional) ... 1963
Single-echelon safety stock calculations ... 1968
Safety stock item outlier processing ... 1983
Multi-echelon safety stock calculations ... 1989
Create multi-echelon families ... 1990
Calculate inventory holding costs for MEIO ... 1996
Determine demand and supply parameters for MEIO ... 1999
Recommend safety stock ... 2005
Apply safety stock bounds in MEIO ... 2010
Automated safety stock updates ... 2012
Comparing single-echelon settings and multi-echelon settings ... 2014
CHAPTER 43: Statistical forecasting ... 2019
Statistical forecasting tables ... 2019
Statistical forecast configuration ... 2020
Statistical forecasting operations ... 2030
Statistical forecasting models ... 2031
Univariate time series forecasting models ... 2032
Multiple Linear Regression forecasting models ... 2042
Multiple linear regression tables ... 2044
Multiple linear regression calculations ... 2047
Additional forecasting options ... 2052
Data characteristics ... 2059
Error measures ... 2060
CHAPTER 43: Consensus forecast ... 2069
Consensus forecast data model ... 2070
Consensus forecast calculations ... 2072
Consensus forecast category weights ... 2073
Consensus forecast adjustments and overrides ... 2079
CHAPTER 44: Forecast disaggregation ... 2083
General disaggregation data model ... 2083
General disaggregation rates ... 2087
Configuring general forecast disaggregation ... 2088
Statistical disaggregation data model ... 2098
Statistical disaggregation rates ... 2102
Configuring statistical forecast disaggregation ... 2103
CHAPTER 45: Event Management ... 2109
Action types for event phases ... 2110
Event Management and the RapidResponse data model — Input tables ... 2112
A process for adjusting forecast values using Event Management ... 2114
Using curves to adjust the forecast ... 2116
Event Management and the RapidResponse data model — calculated tables ... 2120

Date bucketing and disaggregation of event phase forecast adjustments 2124
Adjusting the forecast by negative amounts 2133
CHAPTER 46: Self-Healing Supply Chain 2135
Self-Healing Supply Chain model 2136
Lead time data model 2137
Yield data model 2139
Self-Healing Supply Chain data flow 2141
Self-Healing Supply Chain calculations 2146
Self-Healing Supply Chain configuration 2153
CHAPTER 47: Project management calculations 2161
Integrated project management data model 2161
Task scheduling 2167
Task durations 2176
Resource efficiency 2181
Resource availability and exceptions 2183
Leveling task resources 2186
Project costs 2188
Penalties 2191
Bonuses 2196
CHAPTER 48: Configuring analytic calculations using global settings 2203
Calculating TimeUnit records 2203
Limiting the number of planned orders 2204
Configuring buckets for Inventory Analysis tables 2205
Defining a periods of coverage calendar 2206
Configuring AbsolutePrePlanLimit usage 2206
Enabling automatic maintenance of TransactionSequence values 2207
Configuring task finish date calculations 2208
Defining the And operator for feature rules 2208
CHAPTER 49: Configuring analytic calculations using workbook parameters 2209
Include inactive IndependentDemands in the Activity and CTPActivity tables 2210
Include inactive ScheduledReceipts in the Activity and CTPActivity tables 2211
Include inactive OnHand in the Activity and CTPActivity tables 2212
Duplicate supplies and demands in the Activity and CTPActivity tables 2212
Merge PartSources and BOMs in FlatBillDown and FlatBillUp calculations 2215
Include non-primary part sources in FlatBillDown and FlatBillUp calculations 2217
Include substitute BOM components in FlatBillDown and FlatBillUp calculations 2218
Include global substitutes and their components in FlatBillDown calculations 2219
Explode BOMs for "NoOrders" parts in FlatBillDown calculations 2221
Apply a parameter date to FlatBillDown calculations 2222
Limiting FlatBillDown and FlatBillUp calculations by model 2224
Specify workbook-specific Inventory Analysis buckets 2225
Specify a reporting horizon for WhereConsumed records 2227
Specify a reporting horizon for WhereConsumedForForecast records 2229
Specify a reporting horizon for WhereConsumedForSupply records 2230
Specify a reporting horizon for WhereConsumedToHighVolumeOrder records 2232
Enabling feature-based queries on IndependentDemandByFeature records 2234
Specifying the number of levels used in WhereConsumedForSupply calculations 2235
Calculate general disaggregation rates by forecast category 2236

Part 3: Configuring resource calculations **2237**

CHAPTER 50: Kanban Manager calculations **2239**
Kanban Manager tables 2239
ShopKanban calculations 2240
SourceKanban calculations 2242
Configuring Kanban Manager calculations 2244
CHAPTER 51: Multi-enterprise resources **2249**
Configuring the Multi-Enterprise Inventory workbook 2249
Configuring the Multi-Enterprise Management workbook 2250
Configuring multi-enterprise data 2251
CHAPTER 52: Data validation resources **2253**
Configuring outsourced manufacturing validation 2253
Configuring demand management validation 2254
CHAPTER 53: Clear to Build resources **2257**
Configuring Clear to Build calculations 2257
Configuring orders in the Clear to Build workbook 2258

Part 4: Analytic and data model change summary **2259**

CHAPTER 54: Data model calculations after upgrading **2261**
CHAPTER 55: Analytic and data model changes in 2019 ..... **2263**
Service Update 1908 2264
Service Update 1907 2266
Service Update 1906 2267
Service Update 1905 2269
Service Update 1904 2271
Service Update 1903 2272
Service Update 1902 2277
Service Update 1901 2281
Version 2016.2 changes Service Update 30 2282
Version 2016.2 changes Service Update 29 2284
Version 2016.2 changes Service Update 28 2287
Version 2016.2 changes Service Update 27 2288
CHAPTER 56: Analytic and data model changes in 2018 ..... **2295**
Version 2016.2 changes Service Update 26 2296
Version 2016.2 changes Service Update 25 2299
Version 2016.2 changes Service Update 24 2302
Version 2016.2 changes Service Update 23 2303
Version 2016.2 changes Service Update 22 2309
Version 2016.2 changes Service Update 21 2313
Version 2016.2 changes Service Update 20 2318
Version 2016.2 changes Service Update 19 2321
Version 2016.2 changes Service Update 18 2325
Version 2016.2 changes Service Update 17 2331
Version 2016.2 changes Service Update 16 2335
Version 2016.2 changes Service Update 15 2339
CHAPTER 57: Analytic and data model changes in 2017 ..... **2343**
Version 2016.2 changes Service Update 14 2344
Version 2016.2 changes Service Update 13 2347

Version 2016.2 changes Service Update 12 2350
Version 2016.2 changes Service Update 11 2355
Version 2016.2 changes Service Update 10 2359
Version 2016.2 changes Service Update 9 2363
Version 2016.2 changes Service Update 8 2366
Version 2016.2 changes Service Update 7 2370
Version 2016.2 changes Service Update 6 2373
Version 2016.2 changes Service Update 5 2380
CHAPTER 58: Analytic and data model changes in 2016 2389
Version 2016.2 changes Service Update 4 2390
Version 2016.2 changes Service Update 3 2394
Version 2016.2 changes — Initial release 2398
CHAPTER 59: Legacy upgrade: SafetyStockPolicy table 2409
CHAPTER 60: Legacy upgrade: Statistical forecasting 2413

Index 2417

Table index

A

ABCCode table

ABCCode table — calculated and set fields

Activity table (Mfg)

Activity table (Process Orchestration)

AggregatePartCustomer table

AggregatePartCustomerType table

Allocation table

Allocation table — calculated and set fields

AllotmentOverride table

AllotmentOverride table — calculated and set fields

AllotmentOverrideDetail table

AllotmentOverrideType table

AlternatePart table

AlternatePart table — calculated and set fields

AlternatePartType table

AlternateRouting table

AnalyticsConfiguration table

Assignment table

Assignment table — calculated and set fields

Assumption table

Assumption table — calculated and set fields

AssumptionByConstraint table

AssumptionByPart table

AssumptionByPartCustomer table

AssumptionByPartSource table

AssumptionStatus table

AssumptionStatus table — calculated and set fields

AssumptionType table

AssumptionType table — calculated and set fields

AttributeMap table

AttributeMap table — calculated and set fields

AttributeMapType table

AutomatedLeadTimeType table

AutomatedYieldType table

AvailableRule table

AvailableToPromise table

B

BaselineConflict table

BaselinePlannedOrder table

BaselineTaskLink table

BestFitProfile table

BestFitProfileDetail table

BillOfMaterial table

BillOfMaterial table — calculated and set fields

BillOfMaterialFeatureRule table

BlowThroughNewAllocation table

BlowThroughPlannedAllocation table

BOM_MUECUMLead table

BOMAlternate table

BOMType table

BonusSchedule table

BonusSchedule table — calculated and set fields

BonusScheduleByDate table
BonusScheduleByInterval table
BuyerCode table
BuyerCode table — calculated and set fields

C

Calendar table
CalendarDate table
Capacity table
CapacityOverride table
Carrier table
Carrier table — calculated and set fields
CausalFactor table
CausalFactor table — calculated and set fields
CausalFactorCategory table
CausalFactorCategory table — calculated and set fields
CausalFactorDetail table
Class table
Cluster table
ClusterProperty table
CoByProductSupply table
CollaborativeForecast table
Conflict table
ConsensusForecast table
ConsensusForecastDetail table
ConsensusForecastRollingHorizon table
ConsensusForecastRollingHorizonWeight table
ConsensusForecastWeightByHeader table
Constraint table
Constraint table — calculated and set fields
ConstraintAvailable table
ConstraintAvailable table — calculated and set fields
ConstraintGroup table
ConstraintLoad table
ConstraintMetric table
ConstraintMetric table — calculated and set fields
ConstraintMetricValues table
ConstraintSpreadLoad table
ConstraintType table
ConstraintUnitOfMeasure table
ConstraintUsed table
ControlSet table
CostOfGoodsSold table
CostRollUp table
CriticalPath table
CRPOperation table
CRPOperation table — calculated and set fields
CRPOperationState table
CRPUnitOfMeasure table
CTPActivity table
CTPCoByProductSupply table
CTPForecast table
CTPForecastCost table
CTPPlannedOrder table
CTPSafetyStockHoldings table
CTPSubstituteSupply table
Currency table
CurrencyConversionActual table
CurrencyConversionForecast table
CurveParameters table
CurvePoint table
Customer table
Customer table — calculated and fields
CustomerDestination table
CustomerGroup table
CustomerGroup table — calculated and set fields
CustomerPrice table

D

DataChange table

DataChange_Historical table

DataChange_Local table

DataChange_PendingUpdate table

DaysSupplyPolicy table

DDMRPRule table

DeliveryRoute table

Demand table

DemandException table

DemandOrder table

DemandOrder table — calculated and set fields

DemandStatus table

DemandType table

Department table

Department table—calculated and set fields

DependentConsensusForecast table

DependentDemand table

DirectAssembly table

DirectComponent table

DisaggregationRateByPartCustomer table

DistributionPlanningRule table

DRPConfiguration table

E

EngineeringChange table

EngineeringChange table—calculated and set fields

EngineeringChange_Notes table

Event table

EventConsensusForecastDetail table

EventDisaggregationRate table

EventForecastDetailAdjustment table

EventPhase table

EventPhaseHeader table

EventStatisticalDisaggregationRate table

EventStatisticalForecastDetail table

EventStatisticalForecastDetailAdjustment table

EventType table

ExpiryType table

F

Feature table

Feature table — calculated and fields

FeatureBOMForIndependentDemand table

FeatureMap table

FlatAssembly table

FlatBillDown table

FlatBillUp table

FlatComponent table

FlatDemand table

FlatConstraint table

FLPConstraint table

Forecast table

ForecastConsumption table

ForecastDetail table

ForecastDisaggregationOverride table

ForecastDisaggregationOverride calculated fields

ForecastDisaggregationParameters table

ForecastDisaggregationParametersByCategory table

ForecastItem table

ForecastItem table — calculated and set fields

ForecastItemFitOutput table

ForecastItemParametersMap table

ForecastItemParameters table

ForecastItemParameters table — calculated and set fields
ForecastItemParametersActual table
ForecastItemParametersFitOutput table
ForecastItemParametersOutlier table
ForecastItemParametersOutlier_Notes table
ForecastItemParametersRegressionCoefficient table
ForecastItemParametersRegressionWeight table
ForecastItemParametersRegressor table
ForecastItemParametersType table

H

HistoricalCurrencyConversion table
HistoricalCurrencyHeader table
HistoricalDemandActual table
HistoricalDemandCategory table
HistoricalDemandCategoryRollingWeight table
HistoricalDemandCategoryType table
HistoricalDemandHeader table
HistoricalDemandHeaderRollingWeight table
HistoricalDemandHeaderTimePhasedAttributes table
HistoricalDemandOrder table
HistoricalDemandSeries table
HistoricalDemandSeriesDetail table
HistoricalDemandWaterfall table
HistoricalPartKPI table
HistoricalReceipt table
HistoricalReceiptHeader table
HistoricalShipment table
HistoricalSupplyActual table
HistoricalSupplyCategory table
HistoricalSupplyHeader table
HistoricalSupplyOrder table
HistoricalSupplySeries table
HistoricalSupplySeriesDetail table
HistoricalSupplyWaterfall table
HoldCode table

I

IMConfiguration table
ImportedStatisticalForecast table
IncrementalRule table
IndependentDemand table
IndependentDemand table — calculated and set fields
IndependentDemand_Notes table
IndependentDemandByFeature table
IndependentDemandCost table
IndependentDemandFeature table
IndependentDemandFeatureList table
IndependentDemandFeatureSummary table
IndependentDemandSolution table
IndependentDemandSolution table — calculated and set fields
IndependentDemandSolution_CustomerNotes table
IndependentDemandSolution_SupplyNotes table
InProcess table
InventoryAnalysis table
InventoryCTPAnalysis table
InventorySummary table
InventoryTransfer table
InventoryTransfer table — calculated and set fields
InventoryTransferType table
ItemSource table
ItemSource table — calculated and set fields
ItemSourceCandidate table
ItemSourceHistoricalSupplyCategory table

ItemSourceIntervalEffectivePartSource table
ItemSourcePartSourceEffectivityInterval table
ItemSourcePartSourceHistoricalLeadTime table
ItemSourcePartSourceHistoricalReceipt table

J

Job table

K

KPI table

L

LateSupply table
LoadDailyCurrent table
LoadDailyPlanned table
LoadDetailCurrent table
LoadDetailPlanned table
LoadFLPPlanned table
LoadOperationNewOrderPlanned table
LoadOperationReceiptsCurrent table
LoadOperationReceiptsPlanned table
LoadWeeklyCurrent table
LoadWeeklyPlanned table
Location table
Location table — calculated and set fields
LogisticsLocation table
LogisticsLocation table — calculated and set fields

M

MaximumInventoryResult table
MEIODriverPart table
MEIOFamilyResult table
MEIOHistoricalSupply table
MEIOLeadTime table
MEIOLeadTimeDistributionProfile table
MEIOLeadTimeDistributionProfilePoint table
MEIOLeadTimeType table
MEIOParentPart table
MEIOStage table
MEIOStageHistoricalDemand table
MEIOStageHistoricalForecast table
MEIOStageLink table
MEIOStageResult table
MEIOStageTimePhasedResult table
Metric table
Metric table — calculated and set fields
MetricAsOf table
MetricAsOf table — calculated and set fields
Mfg_SupplyOrder_TextLine table
MLSActivity table
Model table
Model table — calculated and set fields
MPSConfiguration table
MUEPoolNettingType table
MultiEchelonSafetyStockRule table
MultiLevelSearchRule table

N

NewAllocation table
NewTransferAllocation table
Notification table
Numbers table

O

OFConfiguration table

OnHand table
OnHand table — calculated and set fields
OnHandType table
Operation table
Operation table — calculated and set fields
OperationSequence table
OperationSequence table — calculated and set fields
OperationSequenceType table
OperationType table
OrderPolicy table
OrderPriority table
OriginatingFile table
OriginatingSource table
OutlierType table
OverrideMajorChangeover table

P

Part table
Part table — calculated and set fields
Part_MUECumLead table
Part_MUEPoolValidation table
PartCustomer table
PartCustomerTimePhasedAttributes table
PartMetric table
PartMetric table — calculated and set fields
PartMetricValues table
PartSequence table
PartSolution table
PartSource table
PartSource table — calculated and set fields
PartSourceCTPPlannedOrder table
PartSourcePlannedOrder table
PartSourceScheduledReceipt table
PartSourceTimePhasedAttributes table
PartSourceType table
PartSupplier table
PartType table
PartUOMConversion table
PartValidation table
PenaltySchedule table
PenaltySchedule table — calculated and set fields
PenaltyScheduleByDate table
PenaltyScheduleByInterval table
PeriodConstraintLoad table
PeriodsOfCoverage table
PeriodsOfCTPCoverage table
PlannedAllocation table
PlannedOrder table
PlannedTransferAllocation table
PlannerCode table
PlannerCode table — calculated and set fields
PlanningCalendars table
PointsCurve table
PointsCurve table — calculated and set fields
PointsProfile table
PointsProfile table — calculated and set fields
Pool table
PoolMap table
PoolMapOverride table
Predecessor table
Predecessor table — calculated and set fields
ProcessInstance table
ProcessInstance — table calculated and set fields
ProcessInstanceType table
ProductFamily table
ProductFamily table — calculated and set fields
ProductionCycle table

ProductionGroup table
ProductionGroup table — calculated and set fields
ProductionRecurrence table
ProductionRecurrence table — calculated and set fields
ProductionWheel table
ProductionWheel table — calculated and set fields
ProductionWheelConstraint table
ProductionWheelType table
ProfilePoint table
Project table
Project table — calculated and set fields
ProjectedLeadTime table
ProjectedYield table
ProjectGroup table
ProjectGroup table — calculated and set fields
ProjectManager table
ProjectManager table — calculated and set fields
ProjectStatus table
ProjectStatus table — calculated and set fields
ProjectType table

R

RangeOfCoverage table
RangeOfCoveragePartOverride table
ReferenceForecastDimension table
ReferenceForecastTarget table
ReferenceForecastTarget table — calculated and set fields
ReferencePart table
Region table
RegionGroup table
RegressionModel table
RegressionModelTerm table
Regressor table
RegressorCategory table
RegressorValue table
Resource table
Resource table — calculated and set fields
ResourceAssignment table
ResourceAssignment table — calculated and set fields
ResourceAssignmentAvailable table
ResourceAvailable table
ResourceCalendarException table
ResourceCostActual table
ResourceCostActual table — calculated and set fields
ResourceCosts table
ResourceDataByDate table
ResourceGroup table
ResourceGroup table — calculated and set fields
ResourceGroupProjectOverride table
ResourceGroupTaskOverride table
ResourceLoad table
ResourceProjectOverride table
ResourceType table
Routing table
Routing table — calculated and set fields
RParameter table
RParameterName table
RParameterSet table

S

SafetyLeadTimeProfile table
SafetyLeadTimeProfileDetail table
SafetyStockAverageDemandProfile table
SafetyStockHistoricalDemand table
SafetyStockHistoricalForecast table
SafetyStockHistoricalSupply table

SafetyStockItem table
SafetyStockItem table — calculated and set fields
SafetyStockItemDemandParameterSet table
SafetyStockItemFutureDemand table
SafetyStockItemMapping table
SafetyStockItemOutlierResult table
SafetyStockItemOutlierResultSummary table
SafetyStockItemType table
SafetyStockOverride table
SafetyStockPart table
SafetyStockPolicy table
SafetyStockRangeOfCoverage table
SafetyStockResult table
SafetyStockTimePhasedBounds table
SafetyStockTimePhasedResult table
SAP_MD04 table
ScenarioSetting table
ScheduledReceipt table
ScheduledReceipt table — calculated and set fields
ScheduledReceipt_Notes table
ScheduledReceiptCost table
Segment table
Segment table — calculated and set fields
SequenceFlow table
SequenceFlow table —calculated and set fields
Shipment table
Shipment table — calculated and set fields
ShipmentPolicy table
ShipSet table
ShipSetType table
Shop table
Shop table — calculated and set fields
ShopKanban table
SHSCConfiguration table
Site table
SiteManager table
SiteManagerType table
SiteMetric table
SiteMetric table — calculated and set fields
SiteMetricValues table
SOPAnalyticsConfiguration table
Source table
Source table — calculated and set fields
SourceConstraint table
SourceKanban table
SourceRule table
SpreadProfile table
StatisticalForecast table
StatisticalForecastDetail table
StatisticalForecastDisaggregationRate table
StatisticalForecastFitOutput table
StatisticalForecastOutlier table
StatisticalForecastOutlierSummary table
StatisticalForecastPredictActual table
StatisticalForecastRegressionCoefficient table
StatisticalForecastResidualAutocorrelation table
StatisticalForecastResidualCount table
StorageType table
SubProcess table
SubstituteGroup table
SubstituteGroupType table
SubstituteSupply table
Supplier table
Supplier table — calculated and set fields
SupplierGroup table
SupplierGroup table — calculated and set fields
SupplierMetric table
SupplierMetric table — calculated and set fields

SupplierMetricValues table
Supply table
SupplyAllocation table
SupplyAllocation_Notes table
SupplyAllocationType table
SupplyDemand table
SupplyOrder table
SupplyOrder table — calculated and set fields
SupplyStatus table
SupplyType table

T

Task table
Task table — calculated and set fields
Task_Notes table
TaskSupplyLink table
TaskSupplyLink table — calculated and set fields
TaskDemandLinkCost table
TaskHyperlink table
TaskPartLink table
TaskPartLink table — calculated and set fields
TaskSupplyLink table
TaskSupplyLink table — calculated and set fields
TaskSupplyLinkCost table
TaskType table
TimePhasedDemandParameterSet table
TimePhasedLocationCapacity table
TimePhasedMaximumInventory table
TimePhasedPartSourceLeadTime table
TimePhasedSafety table
TimePhasedSourceLeadTime table
TimePhasedYield table
TimeZone table
ToleranceProfile table
ToleranceProfileZone table
TransferDestination table
TransportationMode table
TrialCoByProductSupply table
TrialConstraintShortage table
TrialConsumption table
TrialPlannedOrder table
TrialSubstituteSupply table

U

UnitOfMeasure table
UnitOfMeasureCategory table
UnitOfMeasureCategory table — calculated and set fields

V

ValidatePlannedOrder table

W

Warehouse table
Warehouse table — calculated and set fields
WBS table
WBS table — calculated and set fields
WhereConsumed table
WhereConsumedForDemand table
WhereConsumedForForecast table
WhereConsumedForSupply table
WhereConsumedToHighVolumeOrder table
WorkCenter table
WorkCenter table — calculated and set fields
WorkCenterCapacity table
WorkCenterType table

Field index

A

ABCCode (Part)

ABCCodes (Site)

AboveThresholdRule (OutlierType)

AbsolutePrePlanLimit (PartSource)

AcceptLeadTimeChanges (ItemSource)

AcceptYieldChanges (ItemSource)

ActionCode (ScheduledReceipt)

ActionPartsCount (IndependentDemand)

ActionSupply (LateSupply)

ActionSupply (WhereConsumed)

ActionSupply (WhereConsumedForDemand)

ActionSupply (WhereConsumedForForecast)

ActionSupply (WhereConsumedForSupply)

ActionType (EventPhase)

Active (Notification)

Activities (ProcessInstance)

Activity (Assignment)

Activity (Notification)

Activity (Part)

ActualCategory (HistoricalDemandWaterfall)

ActualCategory (HistoricalSupplyWaterfall)

ActualCost (Task)

ActualDate (ItemSourcePartSourceHistoricalLeadTime)

ActualDockDate (HistoricalSupplyActual)

ActualDockDate (InventoryTransfer)

ActualDockDate (ScheduledReceipt)

ActualDueDate (ForecastConsumption)

ActualDuration (Task)

ActualFinishDate (Activity)

ActualFinishDate (Assignment)

ActualFinishDate (CRPOperationState)

ActualFinishDate (ProcessInstance)

ActualFinishDate (Task)

ActualFinishOffset (CRPOperationState)

ActualHeader (ItemSourcePartSourceHistoricalLeadTime)

ActualLeadTime (ItemSourcePartSourceHistoricalLeadTime)

ActualLeadTimeDays (ItemSourcePartSourceHistoricalLeadTime)

ActualMaterialCost (Project)

ActualMaterialCost (Task)

ActualProcessStartDate (CRPOperationState)

ActualProcessStartOffset (CRPOperationState)

ActualQuantity (ForecastConsumption)

ActualQuantity (ItemSourcePartSourceHistoricalLeadTime)

ActualQuantity (StatisticalForecastPredictActual)

ActualReceiptDate (HistoricalDemandActual)

ActualReceiptDate (IndependentDemandSolution)

Actuals (HistoricalDemandHeader)

Actuals (HistoricalSupplyHeader)

ActualsCategory (ForecastDisaggregationParameters)

ActualsCategory (ForecastDisaggregationParametersByCategory)

ActualsCategory (ForecastItemParameters)

ActualsCategory (OFConfiguration)

ActualSetupStartDate (CRPOperationState)

ActualSetupStartOffset (CRPOperationState)

ActualShipDate (HistoricalDemandActual)

ActualShipDate (HistoricalSupplyActual)

ActualShipDate (IndependentDemandSolution)

ActualShipDate (ScheduledReceipt)

ActualStartDate (Activity)

ActualStartDate (Assignment)

ActualStartDate (CRPOperationState)

ActualStartDate (ProcessInstance)

ActualStartDate (Task)

ActualStartOffset (CRPOperationState)

ActualTaskCost (Project)

ActualTearDownStartDate (CRPOperationState)

ActualTearDownStartOffset (CRPOperationState)

ActualUnitPrice (ItemSourcePartSourceHistoricalLeadTime)

ActualWork (ResourceAssignment)

ActualWork (Task)

AddActualDurationToStartDate (ProjectType)

Address (CustomerDestination)

Address (LogisticsLocation)

Address (Shop)

Address (Site)

Address (Supplier)

AdjustedDemandDate (Activity)

AdjustedDemandDate (CTPActivity)

AdjustedNeedDate (MLSActivity)

AdjustedPartSourceYield (ItemSourcePartSourceHistoricalReceipt)

Adjustment (MEIOLeadTimeOutlier)

Adjustment (EventStatisticalForecastDetailAdjustment)

Adjustment (SafetyStockItemOutlier)

AdjustementType (EventPhase)

AfterForecastInterval (Part)

Aggregate (AggregatePartCustomer)

AggregatePartCustomers (AggregatePartCustomerType)

Aggregates (PartCustomer)

AggregationRule (HistoricalDemandCategoryType)

Airport (Shipment)

Airports (LogisticsLocation)

AlignmentRule (OperationSequenceType)

AllocatedQty (SupplyDemand)

Allocation (Activity)

Allocation (CTPActivity)

Allocation (DependentDemand)

Allocation (Forecast)

Allocation (ForecastConsumption)

Allocations (Model)

Allocation (SupplyDemand)

AllocationCalendar (PlanningCalendars)

AllocationDemandType (SupplyType)

AllocationForecastRule (SupplyType table)

AllocationMultiple (Part)

AllocationQuantityRule (PartType)

AllocationRule (OrderPriority)

Allocations (CRPOperation)

Allocations (Part)

Allocations (ScheduledReceipt)

Allocations (Shop)

Allocations (SubstitueGroup)

AllocUsed (FLPConstraint)

AllotmentOrder (DemandType)

AllotmentOverride (AllotmentOverrideDetail)

AllotmentOverride (IndependentDemand)

AllotmentOverride (PartCustomerTimePhasedAttributes)

AllotmentOverride (PartSource)

AllotmentOverride (ScheduledReceipt)

AllotmentOverride (SupplyDemand)

AllotmentOverrideDetails (AllotmentOverride)

AllotmentOverrideDetails (Part)

AllotmentOverrides (AllotmentOverrideType)

AllotmentOverrideTypes (ControlSet)

AllotmentRule (DistributionPlanningRule)

AllotmentRule (SourceRule)

AllowLatePlan (SupplyStatus)

AllowNonZeroMilestones (ProjectType)

AllowScheduling (HoldCode)

AllowShip (HoldCode)

AllowTaskOnNonWorkingDay (ProjectType)

Alternate (BillOfMaterial)

AlternateDemandPart (SupplyDemand)

AlternatePart (Activity)

AlternatePart (CostOfGoodsSold)

AlternatePart (CTPActivity)

AlternatePart (Demand)

AlternatePart (DependentDemand)

AlternatePart (InProcess)

AlternatePart (SubstituteSupply)

AlternatePart (Supply)

AlternatePart (TrialSubstituteSupply)

AlternatePart (WhereConsumed)

AlternatePart (WhereConsumedForDemand)

AlternatePart (WhereConsumedForForecast)

AlternatePart (WhereConsumedForSupply)

AlternateParts (AlternatePartType)

AlternateParts (Part)

AlternatePartTypes (ControlSet)

AlternatePrimaryPart (Part)

AlternateRoutings (Part)

AlternateRoutings (Routing)

AlternateSupplyPart (SupplyDemand)

AnalysisParameters (EngineeringChange)

AnchorDate (ScheduledReceipt)

ApplyConstraint (BaselineTaskLink)

ApplyConstraint (TaskDemandLink)

ApplyConstraint (TaskSupplyLink)

ApplyRateAdjustment (ResourceAssignment)

ApplyRateAdjustment (Task)

ASAPBackwardTotalSlackRule (ProjectType)

AsOf (ConstraintMetric)

AsOf (PartMetric)

AsOf (SiteMetric)

AsOf (SupplierMetric)

AsOfDate (HistoricalCurrencyHeader)

AsOfDate (HistoricalDemandSeries)

AsOfDate (HistoricalSupplySeries)

AsOfDate (ItemSourceFitOutput)

AsOfDateCalendar (SafetyStockItemType)

AsOfDateCalendarSafetyStockItemTypes (Calendar)

AsOfDateCalendarSafetyStockItemTypes (Calendar)

Assembly (BillOfMaterial)

Assembly (DirectAssembly)

Assembly (FlatAssembly)

AssemblyLLC (BillOfMaterial)

AssignedQuantity (BaselineTaskLink)

AssignedQuantity (TaskDemandLink)

AssignedQuantity (TaskSupplyLink)

Assignments (Activity)

Assignments (Resource)

Assignments (Task)

AssignTo (Assumption)

AssociatedDate (Activity)

AssociatedDate (CTPActivity)
AssociatedQty (Activity)
Assumption (AssumptionByConstraint)
Assumption (AssumptionByPart)
Assumption (AssumptionByPartCustomer)
Assumption (AssumptionByPartSource)
AssumptionByConstraints (Assumption)
AssumptionByConstraints (Constraint)
AssumptionByPartCustomers (Assumption)
AssumptionByPartCustomers (PartCustomer)
AssumptionByParts (Assumption)
AssumptionByParts (Part)
AssumptionByPartSources (Assumption)
Assumptions (AssumptionStatus)
Assumptions (AssumptionType)
Assumptions (ReferenceForecastTarget)
ATP (WhereConsumed)
ATPCTPRule (PartType)
ATPQuantity (CTPActivity)
AttachToForecastItemParameters (Regressor)
AttributeExcessRule (MUEPoolNettingType)
AttributeFlowToCommon (MUEPoolNettingType)
AttributeMaps (AttributeMapType)
AttributeRule (MUEPoolNettingType)
AutoCommitHorizon (PartCustomer)
AutoCommitHorizon (PartSolution)
AutoCommitHorizonDate (IndependentDemandSolution)
AutoCommitHorizonDate (HistoricalDemandActual)
AutoFirmWindow (MPSConfiguration)
AutomatedLeadTimeType (ItemSource)
AutomatedYieldType (ItemSource)
AutoRegressiveTerms (BestFitProfileDetail)
AutoRegressiveTerms (ForecastItemParameters)
Available (PeriodConstraintLoad)
AvailableDate (BaselineTaskLink)
AvailableDate (CTPActivity)
AvailableDate (CTPCoByProductSupply)
AvailableDate (CTPForecast)
AvailableDate (CTPPlannedOrder)
AvailableDate (CTPSubstituteSupply)
AvailableDate (IndependentDemand)
AvailableDate (InProcess)
AvailableDate (LateSupply)
AvailableDate (LoadFLPPlanned)
AvailableDate (ScheduledReceipt)
AvailableDateLimit (SupplyStatus)
AvailableReceiptDate (IndependentDemand)
AvailableRule (Part)
AvailableRule (PartType)
AvailableRules (ControlSet)
Availables (Resource)
Availables (ResourceAssignments)
AvailableShipDate (IndependentDemand)
AvailableStartDate (CTPActivity)
AvailableStartDate (CTPPlannedOrder)
AvailableStartDate (InProcess)
AvailableStartDate (MLSActivity)
AvailableStartDate (ScheduledReceipt)
AvailableStartDate (SupplyDemand)
AvailableToPromise (Part)
Average (MEIOLeadTime)
Average (SafetyStockItemDemandParameterSet)
Average (TimePhasedDemandParameterSet)
AverageDailyDemand (ShopKanban)
AverageDailyDemand (SourceKanban)
AverageDemand (MEIOStageResult)
AverageDemand (MEIOStageTimePhasedResult)

AverageDemand (SafetyStockItem)

AverageDemand (SafetyStockOverride)

AverageDemand (SafetyStockPart)

AverageDemand (SafetyStockResult)

AverageDemand (SafetyStockTimePhasedResult)

AverageDemandRule (SafetyStockItemType)

AverageFirstYearInProcess (Part)

AverageFirstYearInventory (Part)

AverageFutureDemand (SafetyStockPart)

AverageFutureDemand (SafetyStockResult)

AverageQty (Part)

AverageRequirementInterval (RangeOfCoverage)

AverageRequirementIntervalCount (RangeOfCoverage)

AverageRequirementIntervalCount (RangeOfCoveragePartOverride)

AverageRequirementIntervalDurationType (RangeOfCoverage)

AverageRequirementIntervalLeadTimeCount (RangeOfCoverage)

AverageRequirementIntervalLeadTimeCount (RangeOfCoveragePartOverride)

AverageRequirementIntervalOffset (RangeOfCoverage)

AverageRequirementIntervalOffset (RangeOfCoveragePartOverride)

AverageSellingPrice (Part)

AverageUnitPrice (EventConsensusForecastDetail)

AverageUnitPrice (EventForecastDetailAdjustment)

AverageUnitPriceAdjustment (EventConsensusForecastDetail)

AverageUnitPriceAdjustment (EventForecastDetailAdjustment)

B

BalanceDelta (Activity)

BalanceDelta (CTPActivity)

BaseCalendarDate (StatisticalForecastDisaggregationRate)

BaseConversion (UnitOfMeasure)

BaseKey (Allocation)

BaseKey (AlternatePart)

BaseKey (AttributeMap)

BaseKey (BillOfMaterial)

BaseKey (FeatureMap)

BaseKey (HistoricalCurrencyHeader)

BaseKey (OnHand)

BaseKey (Operation)

BaseKey (PartSource)

BaseKey (ResourceAssignment)

Baseline (Project)

BaselineCalendar (ToleranceProfile)

Allocations (Model)

BaselinePlannedOrders (Part)

BaselineQuantity (ValidatePlannedOrder)

BaselineTaskLinks (Task)

BaseQuantity (ForecastItemParameters)

BatchIndex (PlannedOrder)

BatchQuantity (CRPOperation)

BatchQuantity (Operation)

BatchQuantityRule (OperationType)

BAUGR_PartNumberHigherLevelAssembly (SAP_MD04)

BccRecipients (Notification)

BeforeForecastInterval (Part)

BeginDate (HistoricalDemandSeriesDetail)

BeginDate (HistoricalSupplySeriesDetail)

BelowPTFViolation (MLSActivity)

BelowThresholdRule (OutlierType)

BestFitForecastLag (ForecastItemParameters)

BestFitHoldoutPeriodIntervalCount (BestFitProfileDetail)

BestFitHoldoutPeriodIntervalCount (ForecastItemParameters)

BestFitProfile (ForecastItemParameters)

BestFitProfileDetail (ForecastItemParameters)

BestFitProfileDetail (ForecastItemParametersFitOutput)

BestFitProfileDetails (BestFitProfile)

BestFitProfileDetail (StatisticalForecastFitOutput)

BillOfMaterial (Activity)

BillOfMaterial (BillOfMaterialFeatureRule)

BillOfMaterial (BOM_MUECUMLead)

BillOfMaterial (DependentDemand)

BillOfMaterial (FlatDemand)

BillOfMaterial (MLSActivity)

BillOfMaterial (NewAllocation)

BillOfMaterial (PlannedAllocation)

BillOfMaterial (WhereConsumed)

BillOfMaterial (WhereConsumedForDemand)

BillOfMaterial (WhereConsumedForForecast)

BillOfMaterial (WhereConsumedForSupply)

BillOfMaterials (BOMAlternate table)

BillOfMaterials (BOMType)

BillOfMaterials (Model)

BillOfMaterials (SubstituteGroup)

Bin (StatisticalForecastResidualCount)

BinLocation (ShopKanban)

BinLocation (SourceKanban)

BinQty (ShopKanban)

BinQty (SourceKanban)

Bins (ShopKanban)

Bins (SourceKanban)

BinType (ShopKanban)

BinType (SourceKanban)

BlowThrough (BOMType)

BlowThroughNewAllocations (Part)

BlowThroughPlannedAllocations (Part)

BOM (CoByProductSupply)

BOM (DependentConsensusForecast)

BOM (FeatureBOMForIndependentDemand)

BOM (FlatBillDown)

BOM (FlatBillUp)

BOM (SupplyDemand)

BOMAlternate (HistoricalReceiptHeader)

BOMAlternate (ItemSource)

BOMAlternate (PartSource)

BOMAlternate (ScheduledReceipt)

BOMIn (EngineeringChange)

BOMOut (EngineeringChange)

BOMs (CRPOperation)

BOMs (Shop)

BOMTypes (ControlSet)

BOMTypesFlowedFrom (Model)

Bonus (Project)

Bonus (Task)

BonusAccumulationRule (ProjectType)

BonusAccumulationRule (TaskType)

BonusByDate (BonusSchedule table)

BonusByInterval (BonusSchedule table)

BonusCalendar (ProjectType)

BonusCalendar (TaskType)

BonusDate (Project)

BonusDate (Task)

BonusRule (ProjectType)

BonusRule (TaskType)

BonusSchedule (Task)

BonusSchedules (Site)

BoundsRule (SafetyStockItemType)

BranchOperation (OperationSequence)

BucketDate (CollaborativeForecast)

BucketingName (Calendar)

BuildStatus (ScheduledReceipt)

BuyerCode (Part)

BuyerCodes (Site)

BuyerCommit (ScheduledReceipt)

BuyInProcessRule (SupplyType)

C

CalcActualDuration (Task)

CalcActualWork (Task)

CalcAverage (MEIOLeadTime)

CalcAverageLeadTime (SafetyStockItem)

CalcBestFitProfileDetail (ForecastItemParameters)

CalcDuration (Activity)

CalcDuration (Task)

CalcEndDate (MPSConfiguration)

CalcEffectiveLowerBound (ProjectedYield)

CalcEffectiveUpperBound (ProjectedYield)

CalcEffectiveYield (ProjectedYield)

CalcExpiryDate (ScheduledReceipt)

CalcExpiryStartDate (ScheduledReceipt)

CalcFinishDate (Project)

CalcFinishDate (Task)

CalcForecastEndDate (SOPAnalyticsConfiguration)

CalcForecastStartDate (SOPAnalyticsConfiguration)

CalcFutureIntervalsOfSupply (SafetyStockPart)

CalcHistoricalEndDate (SOPAnalyticsConfiguration)

CalcHistoricalLeadTimeError (ItemSource)

CalcHistoricalYieldError (ItemSource)

CalcLeadTimeZone (ItemSource)

CalcPercentComplete (Task)

CalcPercentReceived (Task)

CalcPercentReceived (TaskDemandLink)

CalcPercentReceived (TaskPartLink)

CalcPercentReceived (TaskSupplyLink)

CalcPercentWorkComplete (Task)

CalcProjectedLeadTimeCOV (ItemSource)

CalcProjectedYieldCOV (ItemSource)

CalcRemainingDuration (Task)

CalcRequestStartDate (PlannedOrder)

CalcRequestStartDate (ScheduledReceipt)

CalcRequestStartDate (SubstituteSupply)

CalcStandardDeviation (MEIOLeadTime)

CalcStartDate (InProcess)

CalcStartDate (MPSConfiguration)

CalcStartDate (Project)

CalcStartDate (ScheduledReceipt)

CalcStartDate (Supply)

CalcStartDate (Task)

CalcStartDay (Activity)

CalcState (Project)

CalcState (Task)

CalcUseLeadTimeChanges (ItemSource)

CalcUseYieldChanges (ItemSource)

CalculatedQuantity (ConsensusForecast)

CalculatedQuantity (ValidatePlannedOrder)

CalculatedServiceLevel (SafetyStockOverride)

CalculatedServiceLevel (SafetyStockResult)

CalculatedServiceLevel (SafetyStockTimePhasedResult)

CalcWork (Task)

CalcYieldRatio (HistoricalReceipt)

CalcYieldZone (ItemSource)

Calendar (CalendarDate)

Calendar (Constraint)

Calendar (DRPConfiguration)

Calendar (EventPhase)

Calendar (MPSConfiguration)

Calendar (OFConfiguration)

Calendar (ProcessInstanceType)

Calendar (ProductionWheel)

Calendar (ProjectType)

Calendar (ResourceType)

Calendar (TaskType)

CalendarConversionRate (SafetyStockResult)

CalendarExceptions (Resource)

CalendarInterval (ProductionWheel)

CampaignFixedConstraintFactor (SourceConstraint)

CampaignIndex (PlannedOrder)

CampaignIndex (SupplyOrder)

Capacity (WorkCenter)

Capacity (WorkCenterCapacity)

CapacityMethod (WorkCenterType)

CapacityOverride (WorkCenterCapacity)

CapacityProcessingRule (SupplyType)

CapacityReportLimit (WorkCenterType)

Carrier (DeliveryRoute)

Carrier (Shipment)

Carrier (Source)

Carriers (TransportationMode)

CarryingCost (Part)

Category (CausalFactor)

Category (DisaggregationRateByPartCustomer)

Category (EventPhase)

Category (ForecastDisaggregationParametersByCategory)

Category (HistoricalDemandCategoryRollingWeight)

Category (HistoricalDemandHeader)

Category (HistoricalReceiptHeader)

Category (HistoricalSupplyHeader)

Category (MEIOLeadTimeType)

Category (Regressor)

Category (UnitOfMeasure)

CategoryWeights (ConsensusForecastRollingHorizon)

CausalFactors (CausalFactorCategory)

CcRecipients (Notification)

ChangeDateTime (DataChange_Historical)

ChangeDateTime (DataChange_Local)

ChangeDateTime (DataChange_PendingUpdate)

ChangeDateTime (DataChange)

ChangedFieldNames (DataChange_Historical)

ChangedFieldNames (DataChange_Local)

ChangedFieldNames (DataChange_PendingUpdate)

ChangedFieldNames (DataChange)

ChangedFieldSchema (DataChange_Historical)

ChangedFieldSchema (DataChange_Local)

ChangedFieldSchema (DataChange_PendingUpdate)

ChangedFieldSchema (DataChange)

ChangedFieldValues (DataChange_Historical)

ChangedFieldValues (DataChange_Local)

ChangedFieldValues (DataChange_PendingUpdate)

ChangedFieldValues (DataChange)

ChangeRecordHandle (DataChange_Historical)

ChangeRecordHandle (DataChange_Local)

ChangeRecordHandle (DataChange_PendingUpdate)

ChangeRecordHandle (DataChange)

Child (SubProcess)

Child (WBS)

ChildPartSource (FlatBillUp)

Children (Activity)

Children (Task)

Class (ForecastItemFitOutput)

Class (ForecastItemParametersFitOutput)

Class (ItemSourceFitOutput)

Class (MEIOLeadTimeOutlierSummary)

Class (SafetyStockItemOutlierResultSummary)

Class (StatisticalForecastFitOutput)

Class (StatisticalForecastOutlierSummary)

Cleanup (OverrideMajorChangeover)

Cluster (ClusterProperty)

Cluster (ItemSource)

ClusterProperties (Cluster)

CoByProductPlanningInterval (PlanningCalendars)

CoByProductSupplies (Part)

CoByProductSupply (Activity)

CoByProductSupply (CTPCoByProductSupply)

CoByProductSupply (Supply)

CoByProductUsage (SupplyStatus)

CollaborativeForecasts (Part)

CollaborativeForecasts (Source)

Comment (DataChange_Local)

Comment (DataChange_PendingUpdate)

Comment (DataChange)

CommentsByPart (PartValidation)

CommentsForDemand (PartValidation)

CommentsForOnHand (PartValidation)

CommentsForSupply (PartValidation)

Comments (SAP_MD04)

CommitDate (HistoricalDemandActual)

CommitLevel (PartType)

CommitQty (CollaborativeForecast)

CommitToParentScenario (SHSCConfiguration)

CommonPool (PlanningCalendars)

CompleteQty (CRPOperationState)

Component (AggregatePartCustomer)

Component (AllotmentOverrideDetail)

Component (BillOfMaterial)

Component (DirectComponent)

Component (FeatureBOMForIndependentDemand)

Component (FlatComponent)

ComponentRule (SubstituteGroupType)

Components (Part)

Components (PartCustomer)

ComponentSourceRule (SubstituteGroupType)

CompoundingCalendar (EventPhase)

CompoundingIntervalCount (EventPhase)

ConfidenceIntervalLower (StatisticalForecastResidualAutocorrelation)

ConfidenceIntervalUpper (StatisticalForecastResidualAutocorrelation)

ConfidenceLevel (ForecastItemParameters)

ConfirmationDateTime (ScheduledReceipt)

ConfirmByDate (IndependentDemand)

ConfirmDate (ScheduledReceipt)

ConfirmQty (ScheduledReceipt)

Conflict (Task)

ConsensusForecast (ConsensusForecastDetail)

ConsensusForecast (EventConsensusForecastDetail)

ConsensusForecast (Part)

ConsensusForecastDetails (Part)

ConsensusForecastWeight (HistoricalDemandCategory)

ConsensusForecastWeight (HistoricalDemandCategoryRollingWeight)

ConsensusForecastWeight (HistoricalDemandHeader)

ConsensusForecastWeight (HistoricalDemandHeaderRollingWeight)

ConsensusForecastWeight (HistoricalDemandHeaderTimePhasedAttributes)

ConsensusForecastWeightByHeader (HistoricalDemandHeader)

Constant (BestFitProfileDetail)

Constant (ForecastItemParameters)

ConstrainedDemand (Part)
ConstrainedExpiryDate (DemandException)
ConstrainedExternalDemand (Part)
ConstrainedInProcess (Part)
ConstrainedIntersiteDemand (Part)
ConstrainedInventory (Part)
Constraint (AssumptionByConstraint)
Constraint (BaselineConflict)
Constraint (Conflict)
Constraint (ConstraintAvailable)
Constraint (ConstraintLoad)
Constraint (ConstraintMetric)
Constraint (ConstraintSpreadLoad)
Constraint (ConstraintUsed)
Constraint (CTPActivity)
Constraint (CTPPlannedOrder)
Constraint (FLPConstraint)
Constraint (LateSupply)
Constraint (PeriodConstraintLoad)
Constraint (ProductionCycle)
Constraint (ProductionWheelConstraint)
Constraint (ScheduledReceipt)
Constraint (SourceConstraint)
Constraint (SupplyDemand)
ConstraintAllocationCalendar (OrderPriority)
ConstraintAllocationRule (OrderPriority)
ConstraintAvailable (ProductionCycle)
ConstraintAvailables (Constraint)
ConstraintCalendar (Calendar)
ConstraintDataDate (Calendar)
ConstraintDate (Task)
ConstraintDateRule (PartSourceType)
ConstraintDueDate (MLSActivity)
ConstraintFactor (SourceConstraint)
ConstraintGroup (Constraint)
ConstraintLoad (ConstraintUsed)
ConstraintLoads (Constraint)
ConstraintMetric (ConstraintMetricValues)
ConstraintMetrics (Metric)
ConstraintMetrics (MetricAsOf)
ConstraintMetricValues (ConstraintMetric)
ConstraintMinimumRule (PartSourceType table)
ConstraintMultipleRule (PartSourceType)
ConstraintNeedDate (WhereConsumed)
ConstraintNeedDate (WhereConsumedToHighVolumeOrder)
ConstraintRule (SupplyStatus)
Constraints (ConstraintGroup)
Constraints (ConstraintUnitOfMeasure)
Constraints (Site)
ConstraintShareFence (Part)
ConstraintShareRule (PartType)
ConstraintSpreadLoads (Constraint)
ConstraintType (Task)
ConstraintTypes (ControlSet)
ConstraintUnitOfMeasures (ControlSet)
ConstraintUnitOfMeasures (UnitOfMeasure)
ConstraintUsed (ProductionCycle)
ConstraintUsed (FLPConstraint)
ConstraintUseds (Constraint)
ConsumedQuantity(Forecast)
ConsumedQuantity (ForecastConsumption)
ConsumptionSequence (ConstraintUsed)
ContactName (CustomerDestination)
ContactName (Source)
ContactName (Supplier)
Contacts (Supplier)
ControlSet (AllotmentOverrideType)
ControlSet (AlternatePartType)

ControlSet (AvailableRule)
ControlSet (BOMType)
ControlSet (ConstraintType)
ControlSet (ConstraintUnitOfMeasure)
ControlSet (CRPUnitOfMeasure)
ControlSet (DaysSupplyPolicy)
ControlSet (DemandStatus)
ControlSet (DemandType)
ControlSet (DistributionPlanningRule)
ControlSet (ExpiryType)
ControlSet (ForecastItemParametersType)
ControlSet (IncrementalRule)
ControlSet (InventoryTransferType)
ControlSet (MEIOLeadTimeType)
ControlSet (MUEPoolNettingType)
ControlSet (MultiEchelonSafetyStockRule)
ControlSet (MultiLevelSearchRule)
ControlSet (OnHandType)
ControlSet (OperationSequenceType)
ControlSet (OperationType)
ControlSet (OrderPolicy)
ControlSet (OrderPriority)
ControlSet (PartSourceType)
ControlSet (PartType)
ControlSet (PlanningCalendars)
ControlSet (ProcessInstanceType)
ControlSet (ProductionWheelType)
ControlSet (ProjectType)
ControlSet (RangeOfCoverage)
ControlSet (ResourceType)
ControlSet (SafetyStockItemType)
ControlSet (ShipmentPolicy)
ControlSet (ShipSetType)
ControlSet (Site)
ControlSet (SourceRule)
ControlSet (SpreadProfile)
ControlSet (SubstituteGroupType)
ControlSet (SupplyAllocationType)
ControlSet (SupplyStatus)
ControlSet (SupplyType)
ControlSet (TaskType)
ControlSet (ToleranceProfile)
ControlSet (ToleranceProfileZone)
ControlSet (UnitOfMeasure)
ControlSet (WorkCenterType)
ConversionDate (CTPForecastCost)
ConversionDate (IndependentDemandCost)
ConversionDate (ScheduledReceiptCost)
CoProductYield (TimePhasedYield)
CoProductYield (PartSource)
CopySafetyStock (SafetyStockPart)
Cost (Task)
CostAccrualRule (ResourceType)
CostInfo (ScheduledReceipt)
CostOfGoods (ScheduledReceiptCost)
CostOfGoodsSold (CTPForecastCost)
CostOfGoodsSold (IndependentDemand)
CostOfGoodsSold (IndependentDemandCost)
CostOfGoodsSold (ScheduledReceipt)
CostOfGoodsSolds (Part)
CostRollUps (Part)
CostRule (MultiEchelonSafetyStockRule)
CostRule (PartSourceType)
Costs (Resource)
Count (IndependentDemandFeatureSummary)
Count (SafetyStockRangeOfCoverage)
Count (StatisticalForecastResidualCount)
Coverage (PeriodsOfCoverage)

Created (Assumption)

CreatedDate (HistoricalDemandOrder)

CreatedDate (HistoricalShipment)

CreatedDate (HistoricalSupplyOrder)

CreatedDateTime (ProcessInstance)

CreationDate (Event)

CreationDate (EventPhase)

CreationTime (OriginatingFile)

Creator (Assumption)

Critical (BaselineTaskLink)

Critical (CriticalPath)

Critical (Task)

Critical (TaskDemandLink)

Critical (TaskSupplyLink)

CriticalLimit (HistoricalDemandCategory)

CriticalPath (Project)

CrostonsPredictRule (ForecastItemParametersType)

CRPFloatAfter (PartSource)

CRPFloatBefore (PartSource)

CRPOperation (CRPOperationState)

CRPOperation (LoadDetailCurrent)

CRPOperation (LoadDetailPlanned)

CRPOperation (LoadFLPPlanned)

CRPOperation (LoadOperationNewOrderPlanned)

CRPOperation (LoadOperationReceiptsCurrent)

CRPOperation (LoadOperationReceiptsPlanned)

CRPOperations (Model)

CRPOperations (OperationSequence)

CRPOperationStates (CRPOperation)

CRPOperationStates (ScheduledReceipt)

CRPUnitOfMeasures (ControlSet)

CRPUnitOfMeasures (UnitOfMeasure)

CTPActivity (Part)

CTPCoByProductSupplies (Part)

CTPCoByProductSupply (CostOfGoodsSold)

CTPCoByProductSupply (SupplyDemand)

CTPCoByProductSupply (WhereConsumed)

CTPCoByProductSupply (WhereConsumedForDemand)

CTPCoByProductSupply (WhereConsumedForForecast)

CTPCoByProductSupply (WhereConsumedForSupply)

CTPCoProductSupply (LateSupply)

CTPCoverage (PeriodsOfCTPCoverage)

CTPForecast (CTPActivity)

CTPForecast (CTPForecastCost)

CTPForecast (SuppyDemand)

CTPForecast (WhereConsumedForForecast)

CTPForecast (WhereConsumedToHighVolumeOrder)

CTPForecast (CostOfGoodsSold)

CTPForecast (WhereConsumed)

CTPForecastCostInfo (Part)

CTPForecasts (Part)

CTPHoldRule (SafetyStockPolicy)

CTPIntervalsToExpiry (PartSource)

CTPPlannedOrder (ConstraintLoad)

CTPPlannedOrder (ConstraintSpreadLoad)

CTPPlannedOrder (CostOfGoodsSold)

CTPPlannedOrder (CTPActivity)

CTPPlannedOrder (InProcess)

CTPPlannedOrder (LateSupply)

CTPPlannedOrder (PartSourceCTPPlannedOrder)

CTPPlannedOrder (SuppyDemand)

CTPPlannedOrder (WhereConsumed)

CTPPlannedOrder (WhereConsumedForDemand)

CTPPlannedOrder (WhereConsumedForForecast)

CTPPlannedOrder (WhereConsumedForSupply)

CTPPlannedOrders (Part)

CTPPlannedOrders (PartSource)

CTPSubstituteSupplies (Part)

CTPSubstituteSupply (CostOfGoodsSold)

CTPSubstituteSupply (CTPActivity)

CTPSubstituteSupply (InProcess)

CTPSubstituteSupply (LateSupply)

CTPSubstituteSupply (SupplyDemand)

CTPSubstituteSupply (WhereConsumed)

CTPSubstituteSupply (WhereConsumedForDemand)

CTPSubstituteSupply (WhereConsumedForForecast)

CTPSubstituteSupply (WhereConsumedForSupply)

CumDelayed (InventoryCTPAnalysis)

CumDemand (InventoryAnalysis)

CumDemand (InventoryCTPAnalysis)

CumLead (BOM_MUECUMLead)

CumLead (Part_MUECumLead)

CumLeadTime (BillOfMaterial)

CumLeadTime (Part)

CumPlannedDelayed (InventoryCTPAnalysis)

CumPlannedDemand (InventoryAnalysis)

CumPlannedDemand (InventoryCTPAnalysis)

CumPlannedSupply (InventoryAnalysis)

CumPlannedSupply (InventoryCTPAnalysis)

CumSupply (InventoryAnalysis)

CumSupply (InventoryCTPAnalysis)

CumSynchronizedDemand (InventoryCTPAnalysis)

CumSynchronizedPlannedDemand (InventoryCTPAnalysis)

CumulativeActualCost (Task)

CumulativeActualMaterialCost (Task)

CumulativeBonus (Project)

CumulativeBonus (Task)

CumulativeCost (Task)

CumulativeLeadTime (MEIOStage)

CumulativeLeadTime (SafetyStockPart)

CumulativeMaterialCost (Task)

CumulativeMaterialRevenue (Task)

CumulativeMax (Constraint)

CumulativePenaltyCost (Project)

CumulativePenaltyCost (Task)

CumulativeRevenue (Task)

Currency (CurrencyConversionActual)

Currency (CurrencyConversionForecast)

Currency (HistoricalCurrencyConversion)

Currency (SupplyOrder)

CurrencyAsOfDate (ScenarioSetting)

CurrencyConversionActuals (Currency)

CurrencyConversionForecasts (Currency)

CurrentLoad (PeriodConstraintLoad)

CurrentOperation (ScheduledReceipt)

CurrentOperation (Supply)

CurrentOperationCompleteQuantity (ScheduledReceipt)

CurrentQuantity (SafetyStockOverride)

CurrentSafetyStock (SafetyStockTimePhasedResult)

CurrentSpreadLoad (PeriodConstraintLoad)

CurrentUsedLoad (PeriodConstraintLoad)

Curve (CurvePoint)

CurveParameters (EventPhase)

CurveParameters (UnitOfMeasure)

CurvePoints (PointsCurve)

Customer (CustomerDestination)

Customer (CustomerPrice)

Customer (DemandOrder)

Customer (PartCustomer)

Customer (Project)

Customers (Site)

CustomerChange (IndependentDemandSolution)

CustomerChanges (HistoricalDemandActual)

CustomerGroup (Customer)

CustomerNotes (IndependentDemandSolution)

CustomerOrders (Department)

CustomerPartPrices (Part)

CustomerRequestDate (HistoricalDemandActual)

CustomerRequestDate (IndependentDemand)

Customers (CustomerGroup)

Customers (HoldCode)

Customers (Region)

CustomerServiceRepresentative (Customer)

CustomerServiceRepresentatives (PlannerCode)

Customs (Shipment)

Customss (LogisticsLocation)

CycleCalendar (SafetyStockItemType)

CycleCalendar (SOPAnalyticsConfiguration)

CycleCalendarSafetyStockItemTypes (Calendar)

CycleEndDate (ProductionCycle)

CycleNumber (ProductionCycle)

CycleStartDate (ProductionCycle)

D

DAT00_ReceiptDate (SAP_MD04)

DAT01_DeliveryDate (SAP_MD04)

DataByDate (Resource)

DataChangeSource (DataChange_Historical)

DataChangeSource (DataChange_Local)

DataChangeSource (DataChange_PendingUpdate)

DataChangeSource (DataChange)

DataDate (Constraint)

DataDate (PlanningCalendars)

DataDate (Resource)

DataRule (OutlierType)

Date (AllotmentOverrideDetail)

Date (BaselineConflict)

Date (CausalFactorDetail)

Date (Conflict)

Date (ConsensusForecast)

Date (ConsensusForecastDetail)

Date (ConsensusForecastWeightByHeader)

Date (ConstraintLoad)

Date (ConstraintMetricValues)

Date (ConstraintSpreadLoad)

Date (ConstraintUsed)

Date (CTPSafetyStockHoldings)

Date (CurrencyConversionActual)

Date (CurrencyConversionForecast)

Date (DemandException)

Date (DependentConsensusForecast)

Date (DisaggregationRateByPartCustomer)

Date (EventConsensusForecastDetail)

Date (EventDisaggregationRate)

Date (EventForecastDetailAdjustment)

Date (EventStatisticalDisaggregationRate)

Date (EventStatisticalForecastDetail)

Date (EventStatisticalForecastDetailAdjustment)

Date (Forecast)

Date (ForecastDetail)

Date (ForecastDisaggregationOverride)

Date (ForecastItemParametersActual)

Date (ForecastItemParametersOutlier)

Date (ForecastItemParametersRegressionWeight)

Date (ForecastItemParametersRegressor)

Date (HistoricalCurrencyConversion)

Date (HistoricalDemandActual)

Date (HistoricalDemandWaterfall)

Date (HistoricalPartKPI)

Date (HistoricalSupplyActual)

Date (HistoricalSupplyWaterfall)

Date (ImportedStatisticalForecast)
Date (LoadDailyCurrent)
Date (LoadDailyPlanned)
Date (LoadDetailCurrent)
Date (LoadDetailPlanned)
Date (MaximumInventoryResult)
Date (MEIOHistoricalSupply)
Date (MEIOLeadTimeOutlier)
Date (MEIOStageHistoricalDemand)
Date (MEIOStageHistoricalForecast)
Date (MEIOStageTimePhasedResult)
Date (MetricAsOf)
Date (OnHand)
Date (PartMetricValues)
Date (PeriodsOfCoverage)
Date (PeriodsOfCTPCoverage)
Date (RegressorValue)
Date (ResourceCostActual)
Date (ResourceDataByDate)
Date (ResourceLoad)
Date (SafetyStockItemOutlier)
Date (SafetyStockItemOutlierResult)
Date (SafetyStockHistoricalDemand)
Date (SafetyStockHistoricalForecast)
Date (SafetyStockHistoricalSupply)
Date (SafetyStockItemFutureDemand)
Date (SafetyStockOverride)
Date (SafetyStockTimePhasedResult)
Date (SiteMetricValues)
Date (StatisticalForecast)
Date (StatisticalForecastDetail)
Date (StatisticalForecastDisaggregationRate)
Date (StatisticalForecastOutlier)
Date (StatisticalForecastPredictActual)
Date (SupplierMetricValues)
Date (SupplyDemand)
Date (TaskPartLinkCost)
Date (TaskSupplyLinkCost)
Date (TimePhasedLocationCapacity)
Date (TimePhasedMaximumInventory)
Date (TimePhasedSafety)
Date (WorkCenterCapacity)
DateEffectivityRule (SourceRule)
DateQualifier (SupplyOrder)
DateRule (BOMType)
DateRule (OperationType)
Dates (Calendar)
DayOfWeek (ResourceCalendarException)
Days (TimePhasedSafety)
DaysLate (CTPForecast)
DaysLate (IndependentDemand)
DaysLate (MLSActivity)
DaysLate (Task)
DayOfWeek (CapacityOverride)
DaysSupplyFenceDate (Part)
DaysSupplyPolicy (Part)
DaysSupplyPrioritySource (PartType)
DaysSupplyRule (DaysSupplyPolicy)
DaysSupplyRule (PartType)
DDMRPRule (Part)
DecoupledLeadTime (Part)
Default (TransferDestination)
DefaultCustomerServiceLevelTarget (IMConfiguration)
DefaultHoursPerDay (WorkCenterType)
DefaultInventoryQualityRatioTarget (IMConfiguration)
DefaultLaborRunRatio (WorkCenterType)
DefaultLaborSetupRatio (WorkCenterType)

DefaultMachineRunRatio (WorkCenterType)
DefaultMachineSetupRatio (WorkCenterType)
DefaultMEIONoInventoryRule (IPOConfiguration)
DefaultMEIOUseRule (IPOConfiguration)
DefaultNumberOfResources (WorkCenterType)
DefaultOutlierType (IPOConfiguration)
DefaultPickPackTime (DeliveryRoute)
DefaultPriority (PartType)
DefaultQueueTime (WorkCenterType)
DefaultRoute (CustomerDestination)
DefaultShipment (Supplier)
DefaultSurplusTransition (IMConfiguration)
DefaultTransitTime (Carrier)
DefaultTransportationMode (Carrier)
DefaultWaitTime (WorkCenterType)
DEL12_MRPElementNumber (SAP_MD04)
DELET_SchedulingNumber (SAP_MD04)
DeliveryDateTime (Shipment)
DeliveryRoute (HistoricalDemandActual)
DeliveryRoute (IndependentDemand)
DeliveryRoutes (Carrier)
DeliveryRoutes (CustomerDestination)
DeliveryRoutes (Site)
DeliveryRoutes (TransportationMode)
DELKZ_MRPElementType (SAP_MD04)
DELNR_MRPElementData (SAP_MD04)
DELPS_MRPElementItem (SAP_MD04)
Demand (Conflict)
DemandDate (Activity)
DemandDate (CoByProductSupply)
DemandDate (CTPActivity)
DemandDate (PlannedOrder)
DemandDate (ScheduledReceipt)
DemandDate (Supply)
DemandDate (TrialPlannedOrder)
DemandDueDate (IndependentDemandFeatureSummary)
DemandDueDate (SupplyDemand)
DemandEarliestExpiryDate (SupplyDemand)
DemandException (Activity)
DemandException (CTPActivity)
DemandExceptions (Part)
DemandFraction (SupplyDemand)
DemandLinks (TaskPartLink)
DemandOrder (Activity)
DemandOrderPriority (SupplyDemand)
DemandOrders (DemandType)
DemandOrders (Site)
DemandOutlierMovingAverageWindow (SafetyStockItem)
DemandOutlierProcessingRule (SafetyStockItemType)
DemandOutlierThreshold (SafetyStockItem)
DemandOutlierType (SafetyStockItem)
DemandPeriod (PeriodsOfCoverage)
DemandPeriod (PeriodsOfCTPCoverage)
DemandPlannerCode (PartSolution)
DemandPlannerCodes (PlannerCode)
DemandPool (PoolMap)
DemandPool (PoolOverride)
DemandQty (Activity)
DemandQty (CTPActivity)
Demands (Part)
DemandShipDate (SupplyDemand)
DemandLinks (Task)
DemandSmoothingAfterIntervalCount (SafetyStockItem)
DemandSmoothingBeforeIntervalCount (SafetyStockItem)
DemandSource (DemandException)

DemandSource (SupplyDemand)

DemandStatuses (ControlSet)

DemandTimeFence (Part)

DemandTransformationRule (SafetyStockPolicy)

DemandType (Activity)

DemandType (CTPActivity)

DemandType (PartCustomer)

DemandTypes (ControlSet)

DemandTypes (OrderPriority)

DemandTypes (SpreadProfile)

DemonstratedLaborCapacity (Capacity)

DemonstratedLaborCapacity (LoadDailyCurrent)

DemonstratedLaborCapacity (LoadDailyPlanned)

DemonstratedMachineCapacity (Capacity)

DemonstratedMachineCapacity (LoadDailyCurrent)

DemonstratedMachineCapacity (LoadDailyPlanned)

Department (WorkCenter)

Departments (Site)

DependentConsensusForecasts (Part)

DependentDemandForecastConsumption (PartType)

DependentDemandModel (BOMType)

DependentDemands (Part)

DependentDemandUsage (PartType)

DependentSafetyLeadTimeRule (PartType)

DependentUnitRule (BOMType)

DependentUnitRule (OperationType)

Description (ABCCode)

Description (AggregatePartCustomerType)

Description (AllotmentOverride)

Description (AllotmentOverrideDetail)

Description (AllotmentOverrideType)

Description (AlternatePartType)

Description (AssumptionStatus)

Description (AssumptionType)

Description (AttributeMapType)

Description (AvailableRule)

Description (BestFitProfile)

Description (BOMAlternate table)

Description (BOMType)

Description (BonusSchedule)

Description (BuyerCode)

Description (Calendar)

Description (CapacityOverride)

Description (Carrier)

Description (CausalFactor)

Description (CausalFactorCategory)

Description (Class)

Description (Constraint)

Description (ConstraintGroup)

Description (ConstraintType)

Description (ConstraintUnitOfMeasure)

Description (ControlSet)

Description (CRPOperation)

Description (CRPUnitOfMeasure)

Description (Currency)

Description (CustomerDestination)

Description (CustomerGroup)

Description (DaysSupplyPolicy)

Description (DeliveryRoute)

Description (DemandStatus)

Description (DemandType)

Description (DDMRPRule)

Description (DistributionPlanningRule)

Description (EngineeringChange)

Description (Event)

Description (EventPhase)

Description (EventType)

Description (ExpiryType)

Description (Feature)

Description (ForecastItemParametersType)

Description (HistoricalDemandCategory)

Description (HistoricalDemandCategoryType)

Description (HistoricalSupplyCategory)

Description (HoldCode)

Description (IncrementalRule)

Description (InventoryTransferType)

Description (ItemSourceFitOutputClass)

Description (ItemSourceFitOutputForecastModel)

Description (ItemSourceFitOutputValueName)

Description (Job)

Description (KPI)

Description (LogisticsLocation)

Description (MEIOLeadTimeDistributionProfile)

Description (Model)

Description (MUEPoolNettingType)

Description (MultiEchelonSafetyStockRule)

Description (MultiLevelSearchRule)

Description (OnHandType)

Description (Operation)

Description (OperationSequence)

Description (OperationSequenceType)

Description (OperationType)

Description (OrderPolicy)

Description (OrderPriority)

Description (OutlierType)

Description (Part)

Description (PartSourceType)

Description (PartType)

Description (PenaltySchedule)

Description (PlannerCode)

Description (PlanningCalendars)

Description (PointsCurve)

Description (PointsProfile)

Description (Pool)

Description (Predecessor)

Description (ProcessInstanceType)

Description (ProductFamily)

Description (ProductionGroup)

Description (ProductionRecurrence)

Description (ProductionWheel)

"Description" on page 849 (ProductionWheelType)

Description (Project)

Description (ProjectGroup)

Description (ProjectStatus)

Description (ProjectType)

Description (RangeOfCoverage)

Description (ReferenceForecastDimension)

Description (RegressionModel)

Description (Regressor)

Description (RegressorCategory)

Description (ResourceGroup)

Description (RParameter)

Description (RParameterName)

Description (RParameterSet)

Description (SafetyLeadTimeProfile)

Description (SafetyStockAverageDemandProfile)

Description (SafetyStockItem)

Description (SafetyStockItemMapping)

Description (SafetyStockItemType)

Description (SafetyStockPolicy)

Description (Segment)

Description (ShipmentPolicy)

Description (ShipSet)

Description (ShipSetType)

Description (Shop)

Description (Site)
Description (SiteManager)
Description (SiteManagerType)
Description (SourceRule)
Description (SpreadProfile)
Description (StorageType)
Description (SubstituteGroup)
Description (SubstituteGroupType)
Description (SupplierGroup)
Description (SupplyAllocationType)
Description (SupplyStatus)
Description (SupplyType)
Description (Task)
Description (TaskDemandLink)
Description (TaskHyperlink)
Description (TaskPartLink)
Description (TaskSupplyLink)
Description (TaskType)
Description (ToleranceProfile)
Description (TransportationMode)
Description (UnitOfMeasure)
Description (UnitOfMeasureCategory)
Description (WorkCenter)
Description (WorkCenterType)
DesiredResults (HistoricalDemandCategory)
Destination (DeliveryRoute)
DestinationDate (SupplyAllocation)
DestinationPart (SupplyAllocation)
DestinationSite (Source)
Details (CausalFactor)
Details (HistoricalDemandSeries)
Details (HistoricalSupplySeries)
Details (RParameterSet)
DetectionRule (OutlierType)
DisaggregationActualsCategory (SOPAnalyticsConfiguration)
DifferenceLevel (BestFitProfileDetail)
DifferenceLevel (ForecastItemParameters)
DifferencesThisLevel (Part_MUEPoolValidation)
DifferencesRollUp (Part_MUEPoolValidation)
Dimension (ReferenceForecastTarget)
DirectComponents (Part)
DisaggregationActualsCategoryRule (ForecastItemParametersType)
DisaggregationCalendar (PartCustomer)
DisaggregationCalendar (SOPAnalyticsConfiguration)
DisaggregationHistoricalIntervalCount (SOPAnalyticsConfiguration)
DisaggregationInnerCalendar (SOPAnalyticsConfiguration)
DisaggregationOuterCalendar (SOPAnalyticsConfiguration)
DisaggregationQuantityRule (HistoricalDemandCategoryType)
DisaggregationRates (ForecastItemParameters)
DisaggregationRates (PartCustomer)
DisplayValue (CalendarDate)
DispositionCode (OnHand)
DistributionPlannerCode (PartSolution)
DistributionPlanningRule (Part)
DistributionProfile (MEIOLeadTime)
DistributionProfileRule (MEIOLeadTimeType)
DockDate (Activity)
DockDate (CTPActivity)
DockDate (InventoryTransfer)
DockDate (PlannedOrder)
DockDate (ScheduledReceipt)
DockDateFromStart (PlannedOrder)
DockDateFromStart (ScheduledReceipt)
DockToStockLeadTime (InventoryTransfer)

DockToStockLeadTime (PartSource)

DockToStockLeadTime (ScheduledReceipt)

DockToStockLeadTime (TimePhasedPartSourceLeadTime)

DockToStockLeadTime (TransferDestination)

DockToStockLeadTimeRule (PartSourceType)

DocQualifier (SupplyOrder)

DownsidePercentage (ToleranceProfileZone)

DrawingRevision (ScheduledReceipt)

DriverAvailableDate (CostOfGoodsSold)

DriverAvailableDate (LoadFLPPlanned)

DriverAvailableDate (WhereConsumed)

DriverAvailableDate (WhereConsumedForDemand)

DriverAvailableDate (WhereConsumedForForecast)

DriverAvailableDate (WhereConsumedForSupply)

DriverAvailableDate (WhereConsumedToHighVolumeOrder)

DriverCTPCoByProductSupply (WhereConsumed)

DriverCTPCoByProductSupply (WhereConsumedForSupply)

DriverCTPPlannedOrder (WhereConsumed)

DriverCTPPlannedOrder (WhereConsumedForSupply)

DriverCTPSubstituteSupply (WhereConsumedForSupply)

DriverDaysLate (WhereConsumed)

DriverDueDate (CostOfGoodsSold)

DriverDueDate (LateSupply)

DriverDueDate (LoadFLPPlanned)

DriverDueDate (WhereConsumed)

DriverDueDate (WhereConsumedForDemand)

DriverDueDate (WhereConsumedForForecast)

DriverDueDate (WhereConsumedForSupply)

DriverForecast (MLSActivity)

DriverIndependentDemand (MLSActivity)

DriverInventoryTransfer (WhereConsumed)

DriverNeedQuantity (LAteSupply)

DriverNeedQuantity (WhereConsumed)

DriverNeedQuantity (WhereConsumedForDemand)

DriverNeedQuantity (WhereConsumedForForecast)

DriverNeedQuantity (WhereConsumedForSupply)

DriverOrderID (WhereConsumed)

DriverOrderId (WhereConsumedToHighVolumeOrder)

DriverOrderType (WhereConsumed)

DriverPart (CostOfGoodsSold)

DriverPart (LoadFLPPlanned)

DriverPart (MEIODriverPart)

DriverPart (MLSActivity)

DriverPart (WhereConsumed)

DriverPart (WhereConsumedForForecast)

DriverPart (WhereConsumedForSupply)

DriverPart (WhereConsumedToHighVolumeOrder)

DriverPartCustomer (WhereConsumedToHighVolumeOrder)

DriverQuantity (CostOfGoodsSold)

DriverQuantity (LateSupply)

DriverQuantity (LoadFLPPlanned)

DriverQuantity (MLSActivity)

DriverQuantity (WhereConsumed)

DriverQuantity (WhereConsumedForDemand)

DriverQuantity (WhereConsumedForForecast)

DriverQuantity (WhereConsumedForSupply)

DriverScheduledReceipt (CostOfGoodsSold)

DriverScheduledReceipt (WhereConsumed)

DriverScheduledReceipt (WhereConsumedForSupply)

DriverSupplyDemand (WhereConsumed)

DriverSupplyDemand (WhereConsumedForDemand)

DriverSupplyDemand (WhereConsumedForForecast)
DriverSupplyDemand (WhereConsumedForSupply)
DriverSupplyDemand (WhereConsumedToHighVolumeOrder)
DriverType (CostOfGoodsSold)
DriverType (MLSActivity)
DriverType (WhereConsumed)
DriverType (WhereConsumedForSupply)
DriverType (WhereConsumedToHighVolumeOrder)
DRPCalendar (PartSolution)
DRPFrozenWindow (DRPConfiguration)
DRPFrozenWindow (PartSolution)
DRPHorizon (DRPConfiguration)
DRPHorizon (PartSolution)
DRPHorizonCalendars (Calendar)
DRPPartSiteHorizonCalendars (Calendar)
DRPPlanningHorizon (PartSolution)
DSTName (TimeZone)
DSTOffset (TimeZone)
DSTStartDN (TimeZone)
DSTStartDOW (TimeZone)
DSTStartMonth (TimeZone)
DSTStartTime (TimeZone)
DTFDate (Part)
DTFRule (PartType)
DueDate (Activity)
DueDate (Allocation)
DueDate (Assumption)
DueDate (AvailableToPromise)
DueDate (BaselinePlannedOrder)
DueDate (BlowThroughNewAllocation)
DueDate (BlowThroughPlannedAllocation)
DueDate (CoByProductSupply)
DueDate (CTPActivity)
DueDate (Demand)
DueDate (DependentDemand)
DueDate (IndependentDemand)
DueDate (InProcess)
DueDate (LateSupply)
DueDate (NewAllocation)
DueDate (NewTransferAllocation)
DueDate (PlannedAllocation)
DueDate (PlannedOrder)
DueDate (PlannedTransferAllocation)
DueDate (ScheduledReceipt)
DueDate (SubstituteSupply)
DueDate (Supply)
DueDate (TrialCoByProductSupply)
DueDate (TrialPlannedOrder)
DueDate (TrialSubstituteSupply)
DueDate (ValidatePlannedOrder)
DueDateFromStart (PlannedOrder)
DueDateFromStart (ScheduledReceipt)
Duration (Activity)
Duration (ConsensusForecastRollingHorizon)
Duration (Task)
DurationType (Task)
DwellTime (HistoricalDemandActual)
DwellTime (IndependentDemandSolution)

E

EarliestExpiryDate (BlowThroughPlannedAllocation)
EarliestExpiryDate (Activity)
EarliestExpiryDate (Allocation)
EarliestExpiryDate (BlowThroughNewAllocation)
EarliestExpiryDate (CoByProductSupply)
EarliestExpiryDate (ConsensusForecast)
EarliestExpiryDate (CTPActivity)

EarliestExpiryDate (Demand)

EarliestExpiryDate (DependentDemand)

EarliestExpiryDate (Forecast)

EarliestExpiryDate (IndependentDemand)

EarliestExpiryDate (InventoryTransfer)

EarliestExpiryDate (NewAllocation)

EarliestExpiryDate (NewTransferAllocation)

EarliestExpiryDate (PlannedAllocation)

EarliestExpiryDate (PlannedOrder)

EarliestExpiryDate (PlannedTransferAllocation)

EarliestExpiryDate (ScheduledReceipt)

EarliestExpiryDate (SubstituteSupply)

EarliestExpiryDate (Supply)

EarliestFinishDate (Task)

EarliestStartDate (Task)

EarlyArrivalStock (MEIOStageTimePhasedResult)

EarlyArrivalStock (SafetyStockResult)

EarlyShipDate (IndependentDemandSolution)

EarlyShipWindow (Customer)

EarlyShipWindow (IndependentDemandSolution)

Echelon (MEIOStage)

Echelon (SafetyStockPart)

EffDockDate (ScheduledReceipt)

EffDueDate (ScheduledReceipt)

EffectiveAdjustment (EventStatisticalForecastDetailAdjustment)

EffectiveAllotmentOverride (ConsensusForecast)

EffectiveBranchOperation (OperationSequence)

EffectiveCalendar (Task)

EffectiveDate (CustomerPrice)

EffectiveDate (PartSourceTimePhasedAttributes)

EffectiveDate (PenaltyScheduleByDate)

EffectiveDate (ResourceAssignmentAvailable)

EffectiveDate (ResourceAvailable)

EffectiveDate (ResourceCalendarException)

EffectiveDate (ResourceCosts)

EffectiveDemand (IndependentDemand)

EffectiveDemandQuantity (Forecast)

EffectiveDemandQuantity (InventoryTransfer)

EffectiveDisaggregationCalendar (ForecastItem)

EffectiveDisaggregationCalendar (PartCustomer)

EffectiveDRPFrozenWindow (PartSolution)

EffectiveDRPHorizon (PartSolution)

EffectiveDRPPlanningHorizon (PartSolution)

EffectiveExpiryStartDate (InventoryTransfer)

EffectiveForecastModel (ForecastItemParameters)

EffectiveHistoricalYield (ItemSourcePartSourceHistoricalReceipt)

EffectiveHistoryStartDate (ForecastItemParameters)

EffectiveHoldCode (IndependentDemandSolution)

EffectiveIn (Assumption)

EffectiveInDate (AggregatePartCustomer)

EffectiveInDate (AlternatePart)

EffectiveInDate (BillOfMaterial)

EffectiveInDate (Capacity)

EffectiveInDate (CapacityOverride)

EffectiveInDate (ConstraintAvailable)

EffectiveInDate (CRPOperation)

EffectiveInDate (FeatureMap)

EffectiveInDate (HistoricalDemandHeaderTimePhasedAttributes)

EffectiveInDate (Operation)

EffectiveInDate (PartCustomerTimePhasedAttributes)

EffectiveInDate (PartSource)

EffectiveInDate (ProductionWheel)

EffectiveInDate (ProjectedLeadTime)

EffectiveInDate (ProjectedYield)

EffectiveInDate (SafetyLeadTimeProfileDetail)

EffectiveInDate (SourceConstraint)

EffectiveInDate (TimePhasedPartSourceLeadTime)

EffectiveInDate (TimePhasedSourceLeadTime)

EffectiveInDate (TimePhasedYield)

EffectiveInEC (BillOfMaterial)

EffectiveMinimumShelfLife (ConsensusForecast)

EffectiveMPSAutoFirmWindow (PartSolution)

EffectiveMPSFrozenWindow (PartSolution)

EffectiveMPSHorizon (PartSolution)

EffectiveOrderPointDateRule (Part)

EffectiveOrderPointIncrease (Part)

EffectiveOut (Assumption)

EffectiveOutDate (AggregatePartCustomer)

EffectiveOutDate (AlternatePart)

EffectiveOutDate (BillOfMaterial)

EffectiveOutDate (BonusScheduleByDate table)

EffectiveOutDate (ConstraintAvailable)

EffectiveOutDate (CRPOperation)

EffectiveOutDate (FeatureMap)

EffectiveOutDate (HistoricalDemandHeaderTimePhasedAttributes)

EffectiveOutDate (Operation)

EffectiveOutDate (PartCustomeTimePhasedAttributes)

EffectiveOutDate (PartSource)

EffectiveOutDate (ProductionWheel)

EffectiveOutEC (BillOfMaterial)

EffectiveOvertimeRate (ResourceLoad)

EffectivePercentSafetyInterval (Part)

EffectivePriority (ConsensusForecast)

EffectiveQuantity (EventConsensusForecastDetail)

EffectiveQuantityAdjustment (EventConsensusForecastDetail)

EffectiveQuantityAdjustment (EventForecastDetailAdjustment)

EffectiveRangeOfCoverage (Part)

EffectiveRateAdjustment (ResourceAssignment)

EffectiveRateAdjustmentSource (ResourceAssignment)

EffectiveReturnOperation (OperationSequence)

EffectiveSafetyStockDateRule (Part)

EffectiveSafetyStockQuantityRule (Part)

EffectiveSafetyStockRule (Part)

EffectiveStandardRate (ResourceLoad)

EffectiveSupplyQuantity (InventoryTransfer)

EffectiveUnitPrice (CausalFactorDetail)

EffectiveUnitPrice (ConsensusForecast)

EffectiveUnitPrice (DisaggregationRateByPartCustomer)

EffectiveUnitPrice (Forecast)

EffectiveUnitPrice (ForecastDetail)

EffectiveUnitPrice (ForecastDisaggregationOverride)

EffectiveUnitPrice (IndependentDemand)

EffectiveUnitPrice (ScheduledReceipt)

EffectiveUnitTransferCost (InventoryTransfer)

EffectivityRule (AlternatePartType)

EffectivityRule (BOMType)

EffectivityRule (ConstraintType)

EffectivityRule (OperationType)

EffectivityRule (PartSourceType)

EffFixedLeadTime (PartSource)

Efficiency (Capacity)

Efficiency (CapacityOverride)

Efficiency (WorkCenterCapacity)

EfficiencyCurveRule (TaskType)

EfficiencyFactor (ResourceLoad)

EffLeadTime (FlatBillDown)

EffLeadTime (FlatBillUp)

EffLeadTime (PartSource)

EffLeadTimeAdjust (Part)

EffOffSet (BillOfMaterial)

EffOptionRatio (BillOfMaterial)

EffOrderPriority (ScheduledReceipt)

EffQuantity (Allocation)

EffQuantity (PlannedOrder)

EffQuantity (ScheduledReceipt)

EffQuantity (Supply)

EffQuantity (SupplyAllocation)

EffQuantity (TrialPlannedOrder)

EffQuantityPer (BillOfMaterial)

EffSafetyLeadTime (Part)

EffShipDate (ScheduledReceipt)

EffStartDate (ScheduledReceipt)

EffUnitCost (PartSource)

EffVarLeadTime (PartSource)

EffYield (PartSource)

ElasticNetParameter (BestFitProfileDetail)

ElasticNetParameter (ForecastItemParameters)

Email (CustomerDestination)

Email (LogisticsLocation)

Email (Source)

Email (Supplier)

Enabled (AttributeMap)

EnableUOM (KPI)

EndDate (EventPhase)

EndDate (HistoricalDemandSeriesDetail)

EndDate (HistoricalSupplySeriesDetail)

EndDate (LoadOperationNewOrderPlanned)

EndDate (LoadOperationReceiptsCurrent)

EndDate (LoadOperationReceiptsPlanned)

EndingSafetyStock (Part)

EndOffset (LoadOperationNewOrderPlanned)

EndOffset (LoadOperationReceiptsCurrent)

EndOffset (LoadOperationReceiptsPlanned)

EndUnit (BillOfMaterial)

EndUnit (BOM_MUECUMLead)

EndUnit (CRPOperation)

EndUnit (Operation)

EndUnit (Part_MUECumLead)

EngineeringChange (EngineeringChange_Notes)

ErrorComponent (StatisticalForecastOutlier)

ErrorComponent (SafetyStockItemOutlierResult)

ErrorMessage (AttributeMap)

Estimate (ForecastItemParametersRegressionCoefficient)

Estimate (StatisticalForecastRegressionCoefficient)

EstimatedCost (DeliveryRoute)

EstimatedCost (Source)

EstimatedExpiryDate (Activity)

EstimatedExpiryDate (CoByProductSupply)

EstimatedExpiryDate (CTPActivity)

EstimatedExpiryDate (PlannedOrder)

EstimatedExpiryDate (ScheduledReceipt)

EstimatedExpiryDate (SubstituteSupply)

EstimatedExpiryDate (Supply)

Event (EventPhase)

Events (EventType)

EventAllowNegative (HistoricalDemandCategoryType)

EventDate (MLSActivity)

EventDisaggregationRates (EventPhase)

EventForecastDetailAdjustment (EventConsensusForecastDetail)

EventPhase (EventDisaggregationRate)

EventPhase (EventPhaseHeader)

EventPhase (EventStatisticalDisaggregationRate)

EventPhases (CurveParameters)

EventPhases (Event)

EventPhaseHeader (EventDisaggregationRate)

EventPhaseHeader (EventStatisticalDisaggregationRate)

EventPhaseHeaders (EventPhase)

EventStatisticalForecastDetails (ForecastItemParameters)

EventStatisticalDisaggregationRates (EventPhase)

ExcessFence (Part)

ExcessFenceDate (Part)

ExcessRule (PartType)

ExcessSupply (PeriodsOfCoverage)

ExcessSupply (PeriodsOfCTPCoverage)

Excluded (SAP_MD04)

ExcludePastDueDemands (SafetyStockItemType)

ExistingOutlierAdjustment (SafetyStockItemOutlierResult)

ExistingOutlierQuantity (StatisticalForecastOutlier)

ExpectedFinishDate (Activity)

ExpectedFinishDate (ProcessInstance)

ExpectedFinishDate (Task)

ExpectedStartDate (Activity)

ExpectedStartDate (ProcessInstance)

ExpediteLimit (SupplyStatus)

ExpiringQuantity (CTPCoByProductSupply)

ExpiringQuantity (CTPPlannedOrder)

ExpiringQuantity (CTPSubstituteSupply)

ExpiringQuantity (InventoryTransfer)

ExpiringQuantity (OnHand)

ExpiringQuantity (ScheduledReceipt)

ExpiryCalendar (PlanningCalendars)

ExpiryDate (CTPActivity)

ExpiryDate (CTPCoByProductSupply)

ExpiryDate (CTPPlannedOrder)

ExpiryDate (CTPSubstituteSupply)

ExpiryDate (InventoryTransfer)

ExpiryDate (OnHand)

ExpiryDate (ScheduledReceipt)

ExpiryDate (SupplyDemand)

ExpiryDateRule (PartSourceType)

ExpiryOffset (IndependentDemand)

ExpiryOffset (PartCustomer)

ExpiryOffset (PartCustomerTimePhasedAttributes)

ExpiryPastDueRule (PartType)

ExpiryRule (PartSourceType)

ExpiryRule (PartType)

ExpiryRule (SafetyStockPolicy)

ExpiryStartDate (CTPActivity)

ExpiryStartDate (CTPCoByProductSupply)

ExpiryStartDate (CTPPlannedOrder)

ExpiryStartDate (CTPSubstituteSupply)

ExpiryStartDate (InventoryTransfer)

ExpiryStartDate (OnHand)

ExpiryStartDate (ScheduledReceipt)

ExpiryType (Part)

ExplodeNegatives (BOMType)

ExponentialRate (CurveParameters)

Extend (SafetyStockAverageDemandProfile)

ExtendedType (Activity)

ExtendedType (CTPActivity)

F

Factor (CausalFactorDetail)

Factor (PartUOMConversion)

Family (MEIOFamilyResult)

Family (SafetyStockPart)

FamilyMEIOParts (Part)

Feature (BillOfMaterialFeatureRule)

Feature (IndependentDemandFeature)

Feature (IndependentDemandFeatureList)

Feature (IndependentDemandFeatureSummary)

FeatureBOMs (IndependentDemand)

FeatureBOMRule (PartType)

FeatureGroupIndex (BillOfMaterialFeatureRule)

FeatureList (IndependentDemand)

FeatureMaps (Part)

FeatureRule (BillOfMaterial)

FeatureRulesList (BillOfMaterial)

FeaturesList (IndependentDemand)

Fence (DaysSupplyPolicy)

FenceCalendar (DaysSupplyPolicy)

FillSchedule (ConstraintType)

FilterExpression (ForecastItem)

FinishDate (Project)

FinishDate (Task)

FirstAvailableDate (CTPCoByProductSupply)

FirstAvailableDate (CTPPlannedOrder)

FirstAvailableDate (CTPSubstituteSupply)

FirstAvailableDate (ScheduledReceipt)

FirstDate (Calendar)

FirstDate (CostRollUp)

FirstDate (FlatBillDown)

FirstDate (FlatBillUp)

FirstEffectiveDate (AlternatePart)

FirstEffectiveDate (BillOfMaterial)

FirstEffectiveDate (ConstraintAvailable)

FirstEffectiveDate (EventPhase)

FirstEffectiveDate (ItemSourcePartSourceEffectivityInterval)

FirstEffectiveDate (Operation)

FirstEffectiveDate (PartSource)

FirstEffectiveDate (ProductionWheel)

FirstEffectiveUnit (BillOfMaterial)

FirstEffectiveUnit (Operation)

FirstNeededSupplyDate (InventorySummary)

FirstNeededSupplyDate (Part)

FirstOnAsOfDate (HistoricalDemandSeries)

FirstOnAsOfDate (HistoricalSupplySeries)

FirstShortageDate (InventorySummary)

FirstShortageDate (Part)

FirstUnit (CostRollUp)

FirstUnit (FlatBillDown)

FirstUnit (FlatBillUp)

FirstYearExternalDemand (Part)

FirstYearIntersiteDemand (Part)

FirstZoneIntervalCount (RangeOfCoverage)

FirstZoneTarget (RangeOfCoverage)

FitMeasure (ForecastItemParametersType)

FitOutput (ForecastItemParameters)

FixedConstraintFactor (SourceConstraint)

FixedCost (Resource)

FixedCost (ResourceCostActual)

FixedCost (ResourceCosts)

FixedCost (ResourceDataByDate)

FixedCost (ResourceLoad)

FixedCostAccrualRule (TaskType)

FixedLeadTime (PartSource)

FixedLeadTime (ProjectedLeadTime)

FixedLeadTime (ScheduledReceipt)

FixedLeadTime (TimePhasedPartSourceLeadTime)

Flag (SupplyOrder)

Flagged (PartValidation)

FlatAssembly (FlatBillUp)

FlatBillDowns (Part)

FlatBillUps (Part)

FlatComponent (FlatBillDown)

FlatComponents (Part)

FlatConstraints (Part)

FlatIndex (FlatBillDown)

FlatIndex (FlatBillUp)

FlatLeadTimeDue (FlatBillDown)

FlatLeadTimeDue (FlatBillUp)

FlatLeadTimeStart (FlatBillDown)

FlatLeadTimeStart (FlatBillUp)

FlatQuantityPer (FlatBillDown)

FlatQuantityPer (FlatBillUp)

FlowToCommonForecastQuantity (Activity)

FlowToCommonForecastQuantity (Allocation)

FlowToCommonForecastQuantity (IndependentDemand)

FlowToCommonForecastQuantity (NewAllocation)

FlowToCommonForecastQuantity (NewTransferAllocation)

FlowToCommonForecastQuantity (PlannedAllocation)

FlowToCommonForecastQuantity (PlannedTransferAllocation)

FLPConstraints (Constraint)

FocusIndicator (ScheduledReceipt)

Forecast (Activity)

Forecast (CTPActivity)

Forecast (CTPForecast)

Forecast (Demand)

Forecast (ForecastConsumption)

Forecast (Part)

Forecast (StatisticalForecastOutlier)

Forecast (SupplyDemand)

ForecastAdjustment (AttributeMapType)

ForecastBiasAdjustmentRule (SafetyStockItemType)

ForecastCalendar (PlanningCalendars)

ForecastCategory (ForecastItemParameters)

ForecastConsumption (AttributeMapType)

ForecastConsumption (Part)

ForecastConsumptionCalendar (PartType)

ForecastConsumptionDate (IndependentDemand)

ForecastConsumptionDateRule (DemandStatus)

ForecastConsumptionDateRule (PartType)

ForecastConsumptionOrder (PartType)

ForecastConsumptionOverride (DemandStatus)

ForecastDetail (ConsensusForecastDetail)

ForecastDetails (HistoricalDemandHeader)

ForecastDisaggregationOverrideCategory (SOPAnalyticsConfiguration)

ForecastDisaggregationParametersActualCategories (HistoricalDemandCategory)

ForecastDisaggregationParametersByActualCategory (HistoricalDemandCategory)

ForecastDisaggregationParametersByCategory (HistoricalDemandCategory)

ForecastDisaggregationParametersInnerCalendars (Calendar)

ForecastDisaggregationParametersOuterCalendars (Calendar)

ForecastEndOffset (SOPAnalyticsConfiguration)

ForecastErrorOutlierThreshold (SafetyStockItem)

ForecastErrorOutlierProcessingRule (SafetyStockItemType)

ForecastIntervalCount (ForecastItemParameters)

ForecastItem (PartCustomer)

ForecastItemParameterUOMs (UnitOfMeasure)

ForecastItemParameters (BestFitProfile)

ForecastItemParameters (BestFitProfileDetail)

ForecastItemParameters (ForecastItemParametersActual)

ForecastItemParameters (ForecastItemParametersOutlier)

ForecastItemParameters (ForecastItemParametersRegressionWeight)

ForecastItemParameters (ForecastItemParametersRegressor)

ForecastItemParameters (ForecastItemParametersType)

ForecastItemParameters (OutlierType)

ForecastItemParameters (RParameterSet)

ForecastItemParametersActuals (ForecastItemParameters)

ForecastItemParametersActualsCategories (HistoricalDemandCategory)

ForecastItemParametersFitOutputs (BestFitProfileDetail)

ForecastItemParametersFitOutputs (ForecastItemParameters)

ForecastItemParametersForecastCategories (HistoricalDemandCategory)

ForecastItemParametersForecastProfiles (PointsProfile)

ForecastItemParametersMaps (ForecastItemParameters)

ForecastItemParametersOutlier (ForecastItemParametersOutlier_Notes)

ForecastItemParametersOutlierCalcs (ForecastItemParameters)

ForecastItemParametersOutlierFitOutputCalcs (ForecastItemParameters)

ForecastItemParametersReferenceMaps (ForecastItemParameters)

ForecastItemParametersRegressionWeights (ForecastItemParameters)

ForecastItemParametersRegressors (ForecastItemParameters)

ForecastItemParametersTypes (ControlSet)

ForecastItemParametersTypes (Calendar)

ForecastLag (SafetyStockItem)

ForecastModel (BestFitProfileDetail)

ForecastModel (ForecastItemFitOutput)

ForecastModel (ForecastItemParametersFitOutput)

ForecastModel (ForecastItemParametersType)

ForecastModel (ItemSourceFitOutput)

ForecastModel (StatisticalForecastFitOutput)

ForecastProfile (ForecastItemParameters)

ForecastQuantity (Allocation)

ForecastQuantity (IndependentDemand)

ForecastQuantity (NewAllocation)

ForecastQuantity (NewTransferAllocation)

ForecastQuantity (PlannedAllocation)

ForecastQuantity (PlannedTransferAllocation)

ForecastSafetyStockItemMappings (HistoricalDemandCategory)

ForecastSafetyStockItems (HistoricalDemandCategory)

ForecastSource (Forecast)

ForecastStartOffset (SOPAnalyticsConfiguration)

FreeSlack (Task)

FreeSlackCalendar (ProjectType)

FromAttributeName (AttributeMap)

FromAttributeValue (AttributeMap)

FromProductionGroup (OverrideMajorChangeover)

FromRule (FeatureMap)

FromStage (MEIOStageLink)

FrozenHorizon (PartCustomer)

FrozenHorizon (PartSolution)

FrozenHorizonDate (HistoricalDemandActual)

FrozenHorizonDate (IndependentDemandSolution)

FrozenWindow (MPSConfiguration)

Function (RParameterName)

FutureAvailability (MultiLevelSearchRule)

FutureAverageDemandProfile (SafetyStockItem)

FutureDemand (SafetyStockOverride)

FutureDemand (SafetyStockTimePhasedResult)

FutureIntervalCount (SafetyStockItem)

FutureReorderPoint (SafetyStockTimePhasedResult)

FutureYieldCalendar (SHSCConfiguration)

FutureYieldHorizon (SHSCConfiguration)

G

GatingConstraint (IndependentDemand)

GatingDemand (CrticalPath)

GatingPart (IndependentDemand)

GatingSupply (CrticalPath)

GatingTask (CrticalPath)

Group (Project)

Group (Resource)

H

HandlingCost (HistoricalSupplyActual)

HasConstraintShortageByNeedDate (MLSActivity)

Header (CausalFactor)

Header (ConsensusForecastRollingHorizonWeight)

Header (ConsensusForecastWeightByHeader)

Header (DisaggregationRateByPartCustomer)

Header (EventConsensusForecastDetail)

Header (EventForecastDetailAdjustment)

Header (EventPhaseHeader)

Header (EventStatisticalForecastDetail)

Header (EventStatisticalForecastDetailAdjustment)

Header (ForecastDetail)

Header (ForecastDisaggregationOverride)

Header (ForecastDisaggregationParameters)

Header (HistoricalCurrencyConversion)

Header (HistoricalDemandActual)

Header (HistoricalDemandHeaderRollingWeight)

Header (HistoricalDemandHeaderTimePhasedAttributes)

Header (HistoricalDemandSeries)

Header (HistoricalDemandWaterfall)

Header (HistoricalReceipt)

Header (HistoricalSupplyActual)

Header (HistoricalSupplySeries)

Header (HistoricalSupplyWaterfall)

Header (StatisticalForecastDetail)

Header (StatisticalForecastDisaggregationRate)

HeaderWeights (ConsensusForecastRollingHorizon)

Historical (ProcessInstance)

HistoricalAverageDemandProfile (SafetyStockItem)

HistoricalDemandActuals (HistoricalDemandOrder)

HistoricalDemandCategories (HistoricalDemandCategoryType)

HistoricalDemandCategory (SafetyStockItem)

HistoricalDemandCategory (SafetyStockItemMapping)

HistoricalDemandHeaders (HistoricalDemandCategory)

HistoricalDemandHeaders (PartCustomer)

HistoricalDemandOrders (Site)

HistoricalEndDate (SafetyStockItem)

HistoricalEndDate (SafetyStockItemMapping)

HistoricalForecastCategory (SafetyStockItem)

HistoricalForecastCategory (SafetyStockItemMapping)

HistoricalHorizon (SHSCCConfiguration)

HistoricalIntervalCount (BestFitProfileDetail)

HistoricalIntervalCount (ForecastDisaggregationParameters)

HistoricalIntervalCount (ForecastDisaggregationParametersByCategory)

HistoricalIntervalCount (ForecastItemParameters)

HistoricalIntervalCount (SafetyStockItem)

HistoricalLeadTimeErrorMeasure (AutomatedLeadTimeType)

HistoricalPartKPIs (Part)

HistoricalReceipt (ItemSourcePartSourceHistoricalReceipt)

HistoricalReorderPoint (SafetyStockTimePhasedResult)

HistoricalShipments (Site)

HistoricalSkipLeadingZeros (SafetyStockItem)

HistoricalStartDate (SafetyStockItem)

HistoricalStartDate (SafetyStockItemMapping)

HistoricalSupplies (MEIOLeadTime)

HistoricalSupplyActuals (Source)

HistoricalSupplyCategory (ItemSourceCandidate)

(ItemSourceHistoricalSupplyCategory)

HistoricalSupplyCategory (SafetyStockItem)

HistoricalSupplyCategory (SafetyStockItemMapping)

HistoricalSupplyHeaders (Calendar)

HistoricalSupplyHeaders (PartSupplier)

HistoricalSupplyOrders (Site)

HistoricalYieldErrorMeasure (AutomatedYieldType)

HistoricalYieldHorizon (SHSCConfiguration)

HistorySkipLeadingZeros (ForecastItemPareameters)

HistoryStartDate (ForecastItemParameters)

HoldCode (Customer)

HoldCode (IndependentDemandSolution)

HoldEndDate (CTPActivity)

HoldEndDate (SupplyDemand)

HoldoutPeriodUsageRule (ForecastItemParametersType)

HoldStartDate (CTPActivity)

HoldStartDate (SupplyDemand)

Horizon (ConsensusForecastRollingHorizonWeight)

Horizon (HistoricalDemandCategoryRollingWeight)

Horizon (HistoricalDemandHeaderRollingWeight)

Horizon (MPSConfiguration)

HorizonCalendar (IMConfiguration)

HorizonCalendar (SHSCCConfiguration)

HoursPerDay (Capacity)

HoursPerDay (Project)

HoursPerDay (Resource)

HoursPerDay (ResourceAvailable)

HoursPerDay (ResourceCalendarException)

HoursPerDay (ResourceDataByDate)

I

Id (Activity)

Id (AggregatePartCustomer)

Id (AllotmentOverride)

Id (AllotmentOverrideDetail)

Id (Assignment)

Id (Assumption)

Id (BaselineConflict)

Id (BestFitProfileDetail)

Id (BonusScheduleByDate table)

Id (BonusScheduleByInterval table)

Id (CapacityOverride)

Id (CausalFactorDetail)

Id (ConstraintGroup)

Id (CurveParameters)

Id (CurvePoint)

Id (Customer)

Id (CustomerDestination)

Id (CustomerGroup)

Id (CustomerPrice)

Id (DeliveryRoute)

Id (DemandOrder)

Id (Department)

Id (EngineeringChange)

Id (EventPhaseHeader)

ID (ForecastDetail)

Id (ForecastDisaggregationOverride)

Id (ForecastDisaggregationParameters)

Id (ForecastItemParametersOutlier)

Id (HistoricalDemandHeaderTimePhasedAttributes)

Id (HistoricalDemandOrder)

Id (HistoricalDemandSeries)

Id (HistoricalReceipt)

Id (HistoricalSupplyOrder)

Id (HistoricalSupplySeries)

Id (HoldCode)

Id (InventoryTransfer)
Id (Job)
Id (Location)
Id (LogisticsLocation)
Id (MEIOLeadTimeDistributionProfilePoint)
Id (OperationSequence)
Id (PartCustomerTimePhasedAttributes)
Id (PartSourceTimePhasedAttributes)
Id (PenaltyScheduleByDate)
Id (PenaltyScheduleByInterval)
Id (ProfilePoint)
Id (ReferenceForecastDimension)
Id (ReferenceForecastTarget)
Id (Region)
Id (RegionGroup)
Id (ResourceAssignmentAvailable)
Id (ResourceAvailable)
Id (ResourceCalendarException)
Id (ResourceCostActual)
Id (ResourceCosts)
Id (Routing)
Id (RParameter)
Id (SafetyStockItemMapping)
Id (SafetyStockOverride)
Id (SafetyStockPart)
Id (Shipment)
Id (Source)
Id (Supplier)
Id (SupplierGroup)
Id (SupplyAllocation)
Id (SupplyOrder)
Id (Task)
Id (TimePhasedDemandParameterSet)
Id (TransferDestination)
Id (UnitOfMeasureCategory)
Id (Warehouse)
ID (WhereConsumed)
IDByFeatures (Part)
IDByFeatureSummary (Part)
IdentificationFieldNames (DataChange_Historical)
IdentificationFieldNames (DataChange_Local)
IdentificationFieldNames (DataChange_PendingUpdate)
IdentificationFieldNames (DataChange)
IdentificationFieldValues (DataChange_Historical)
IdentificationFieldValues (DataChange_Local)
IdentificationFieldValues (DataChange_PendingUpdate)
IdentificationFieldValues (DataChange)
Idle (LoadFLPPlanned)
Idle (LoadOperationNewOrderPlanned)
Idle (LoadOperationReceiptsCurrent)
Idle (LoadOperationReceiptsPlanned)
IDOCDateTime (SupplyOrder)
IgnoreAssemblyYield (BOMType)
IgnoreBestFitProfileDetailParameters (ForecastItemParameters)
IgnoreStartDate (SupplyStatus)
IgnoreSupply (SupplyStatus)
IMFutureHorizon (IMConfiguration)
ImitationRate (CurveParameters)
IMPastHorizon (IMConfiguration)
ImportedStatisticalForecasts (ForecastItemParameters)
IncludeFutureDemand (SafetyStockItemMapping)
IncomingFlowList (Activity)
IncomingFlows (Activity)
IncomingServiceTime (MEIOStageResult)
IncomingServiceTime (SafetyStockPart)
IncomingServiceTime (SafetyStockResult)

IncrementalRule (Part)

IncrementalRules (ControlSet)

IncrementalSplit (SupplyStatus)

IndependentDemand (Activity)

IndependentDemand (CostOfGoodsSold)

IndependentDemand (CTPActivity)

IndependentDemand (Demand)

IndependentDemand (FeatureBOMForIndependentDemand)

IndependentDemand (FlatDemand)

IndependentDemand (Forecast)

IndependentDemand (ForecastConsumption)

IndependentDemand (IndependentDemand_Notes)

IndependentDemand (IndependentDemandByFeature)

IndependentDemand (IndependentDemandCost)

IndependentDemand (IndependentDemandFeature)

IndependentDemand (IndependentDemandFeatureList)

IndependentDemand (IndependentDemandSolution)

IndependentDemand (LateSupply)

IndependentDemand (SupplyDemand)

IndependentDemand (TaskDemandLink)

IndependentDemand (WhereConsumed)

IndependentDemand (WhereConsumedForDemand)

IndependentDemand (WhereConsumedToHighVolumeOrder)

IndependentDemands (AllotmentOverride)

IndependentDemands (DeliveryRoute)

IndependentDemands (DemandStatus)

IndependentDemand (Model)

IndependentDemands (Part)

IndependentDemands (ShipmentPolicy)

IndependentDemands (ShipSet)

IndependentDemandSolution (IndependentDemandSolution_CustomerNotes)

IndependentDemandSolution (IndependentDemandSolution_SupplyNotes)

IndependentDemandSolutions (HoldCode)

IndependentDemandSolutions (IndependentDemand)

IndependentDemandsUnderOrderPriority (OrderPriority)

IndependentDemandsUnderSavedPriority (OrderPriority)

IndependentSafetyLeadTimeRule (PartType)

Index (CapacityOverride)

Index (FlatDemand)

Index (ForecastItemParametersRegressionWeight)

Index (CurvePoint)

Index (MLSActivity)

Index (ProfilePoint)

Index (SafetyStockItemDemandParameterSet)

Index (StatisticalForecastFitOutput)

Index (Task)

Index (TaskHyperlink)

Index (TimePhasedDemandParameterSet)

Index (TrialCoByProductSupply)

Index (TrialPlannedOrder)

Index (TrialSubstituteSupply)

Index (WhereConsumedForDemand)

Index (WhereConsumedForForecast)

Index (WhereConsumedForSupply)

Initials (Resource)

InitialValue (CurveParameters)

InKit (Allocation)

InnerCalendar (ForecastDisaggregationParameters)

InnerCalendar (ForecastDisaggregationParametersByCategory)

InnovationRate (CurveParameters)

InProcess (InventoryAnalysis)

InProcess (InventoryCTPAnalysis)

InProcess (Part)

InputModel (ScheduledReceipt)

InputPool (IndependentDemand)

InputPool (ScheduledReceipt)

InsideLeadTime (LateSupply)

InsideLeadTime (WhereConsumed)

InsideLeadTime (WhereConsumedForDemand)

InsideLeadTime (WhereConsumedForForecast)

InsideLeadTime (WhereConsumedForSupply)

InsideToleranceAction (AutomatedLeadTimeType)

InsideToleranceAction (AutomatedYieldType)

ItemParameters (ForecastItemParametersRegressionCoefficient)

IntendedDriverType (OrderPriority)

InterSiteLevel (FlatBillDown)

InterSiteLevel (FlatBillUp)

Interval (ItemSourceIntervalEffectivePartSource)

Interval (PenaltyScheduleByInterval)

IntervalsCalendar (ForecastItemParametersType)

IntervalsCalendar (SafetyStockItemType)

IntervalsOfSupply (SafetyStockResult)

IntervalsPerPeriod (SafetyStockItemType)

IntervalsToExpiry (PartSource)

IntervalToExpiryOffset (BillOfMaterial)

InTransitConsumedQuantity (ScheduledReceipt)

Inventory (InventoryAnalysis)

Inventory (InventoryCTPAnalysis)

InventoryAnalysis (Part)

InventoryCost (MEIOFamilyResult)

InventoryCTPAnalysis (Part)

InventoryHoldingRate (Part)

InventoryPlannerCode (PartSolution)

InventoryPlannerCodes (PlannerCode)

InventoryQualityRatioCalendar (IMConfiguration)

InventorySourcingTransfers (Part)

InventorySummarys (Part)

InventoryTransfer (Activity)

InventoryTransfer (CostOfGoodsSold)

InventoryTransfer (CTPActivity)

InventoryTransfer (Demand)

InventoryTransfer (DependentDemand)

InventoryTransfer (InProcess)

InventoryTransfer (LateSupply)

InventoryTransfer (Supply)

InventoryTransfer (SupplyDemand)

InventoryTransfer (WhereConsumed)

InventoryTransfer (WhereConsumedForDemand)

InventoryTransfer (WhereConsumedForForecast)

InventoryTransfer (WhereConsumedForSupply)

InventoryTransferAllocation (SupplyDemand)

InventoryTransfers (InventoryTransferType)

InventoryTransfers (Model)

InventoryTransfers (Part)

InventoryTransferModels (Model)

InventoryTransferTypes (ControlSet)

IsActual (HistoricalDemandWaterfall)

IsActual (HistoricalSupplyWaterfall)

IsBase (Currency)

IsBaseline (HistoricalDemandSeries)

IsBaseline (HistoricalDemandWaterfall)

IsBaseline (HistoricalSupplySeries)

IsBaseline (HistoricalSupplyWaterfall)

IsDependent (Demand)

IsForecast (DependentDemand)

IsInsideCapacityReportLimit (LoadDailyCurrent)

IsInsideCapacityReportLimit (LoadDailyPlanned)

IsInsideCapacityReportLimit (LoadWeeklyCurrent)

IsInsideCapacityReportLimit (LoadWeeklyPlanned)

IsLeadTimeOutlier (MEIOHistoricalSupply)

IsLeadTimeOutlier (SafetyStockHistoricalSupply)

IsLeaf (FlatDemand)

IsLeaf (MLSActivity)

IsModelEffective (FlatBillDown)

IsModelEffective (FlatBillUp)

IsNested (AggregatePartCustomer)

IsOutlier (SafetyStockHistoricalDemand)

IsOutlier (SafetyStockItemOutlierResult)

IsOutlier (StatisticalForecastOutlier)

IsPhantom (Part)

IsPlanned (ConstraintLoad)

IsPlanned (ConstraintSpreadLoad)

IsPlanned (InProcess)

IsPlanned (Supply)

IsPlanned (TrialConsumption)

IsRecursive (Allocation)

IsRecursive (AlternatePart)

IsRecursive (BillOfMaterial)

IsRecursive (Constraint)

IsRecursive (PartSource)

IsRecursive (Predecessor)

IsRecursive (SequenceFlow)

IsRecursive (SubProcess)

IsRecursive (WBS)

IsServerTimezone (TimeZone)

IsSubstitute (SubstituteSupply)

IsSubstitute (TrialSubstituteSupply)

IsSummary (Task)

IsTransfer (FlatBillDown)

IsTransfer (FlatBillUp)

Item (ForecastItemParameters)

Item (ForecastItemParametersMap)

Item (MEIOHistoricalSupply)

Item (SafetyStockHistoricalDemand)

Item (SafetyStockHistoricalForecast)

Item (SafetyStockHistoricalSupply)

Item (SafetyStockItemDemandParameterSet)

Item (SafetyStockItemFutureDemand)

Item (SafetyStockItemOutlierResult)

Item (SafetyStockItemOutlierResultSummary)

ItemParameters (EventStatisticalForecastDetail)

ItemParameters (ForecastItemFitOutput)

ItemParameters (ForecastItemParametersFitOutput)

ItemParameters (ImportedStatisticalForecast)

ItemParameters (StatisticalForecast)

ItemParameters (StatisticalForecastDetail)

ItemParameters (StatisticalForecastDisaggregationRate)

ItemParameters (StatisticalForecastFitOutput)

ItemParameters (StatisticalForecastOutlier)

ItemParameters (StatisticalForecastOutlierSummary)

ItemParameters (StatisticalForecastPredictActual)

ItemParameters (StatisticalForecastRegressionCoefficient)

ItemParameters (StatisticalForecastResidualAutocorrelation)

ItemParameters (StatisticalForecastResidualCount)

Items (MEIOLeadTimeType)

ItemSource (ItemSourceFitOutput)

ItemSource (ItemSourcePartSourceHistoricalReceipt)

ItemSource (ItemSourceIntervalEffectivePartSource)

ItemSource (ItemSourcePartSourceEffectivityInterval)

ItemSource (ItemSourcePartSourceHistoricalLeadTime)
ItemSource (ProjectedLeadTime)
ItemSource (ProjectedYield)
ItemSourceEffectivePartSources (ItemSource)
ItemSourceHistoricalSupplyCategorys (HistoricalSupplyCategory)
ItemSourcePartSourceEffectivityIntervals (ItemSource)
ItemSourcePartSourceHistoricalLeadTimes (ItemSource)
ItemSources (Cluster)
ItemSources (ProjectedLeadTime)
IterationsPerformed (MEIOFamilyResult)

K

KanbanHorizon (Shop)
KanbanHorizon (SourceKanban)
Kanbans (Shop)
KeyConstraint (Constraint)
KeyCustomer (Customer)
KeyPart (PartSolution)
KindPriority (DemandType)
KnownIssueByPart (PartValidation)
KnownIssueForDemand(PartValidation)
KnownIssueForOnHand (PartValidation)
KnownIssueForSupply (PartValidation)
KPI (HistoricalPartKPI)

L

Label (TaskHyperlink)
LaborCapacity (LoadDailyCurrent)
LaborCapacity (LoadDailyPlanned)
LaborCapacity (LoadWeeklyCurrent)
LaborCapacity (LoadWeeklyPlanned)
LaborCost (CostOfGoodsSold)
LaborCost (CostRollUp)
LaborCost (PartSource)
LaborCost (PartSourceTimePhasedAttributes)
LaborCost (WhereConsumedForDemand)
LaborCost (WhereConsumedForForecast)
LaborCost (WhereConsumedForSupply)
LaborIdle (LoadDailyCurrent)
LaborIdle (LoadDailyPlanned)
LaborIdle (LoadDetailCurrent)
LaborIdle (LoadDetailPlanned)
LaborIdle (LoadFLPPlanned)
LaborIdle (LoadWeeklyCurrent)
LaborIdle (LoadWeeklyPlanned)
LaborRun (LoadDailyCurrent)
LaborRun (LoadDailyPlanned)
LaborRun (LoadDetailCurrent)
LaborRun (LoadDetailPlanned)
LaborRun (LoadWeeklyCurrent)
LaborRun (LoadWeeklyPlanned)
LaborRunRatio (Capacity)
LaborSetup (LoadDailyCurrent)
LaborSetup (LoadDailyPlanned)
LaborSetup (LoadDetailCurrent)
LaborSetup (LoadDetailPlanned)
LaborSetup (LoadFLPPlanned)
LaborSetup (LoadWeeklyCurrent)
LaborSetup (LoadWeeklyPlanned)
LaborSetupRatio (Capacity)
LaborRun (LoadFLPPlanned)
Lag (ForecastItemParametersMap)
Lag (RegressionModelTerm)
Lag (StatisticalForecastResidualAutocorrelation)
LastCommitted (CollaborativeForecast)
LastDataUpdate (Site)
LastDate (Calendar)

LastDate (CostRollUp)

LastDate (FlatBillDown)

LastDate (FlatBillUp)

LastEffectiveDate (AlternatePart)

LastEffectiveDate (BillOfMaterial)

LastEffectiveDate (ConstraintAvailable)

LastEffectiveDate (EventPhase)

LastEffectiveDate (ItemSourcePartSourceEffectivityInterval)

LastEffectiveDate (Operation)

LastEffectiveDate (PartSource)

LastEffectiveDate (ProductionWheel)

LastEffectiveUnit (BillOfMaterial)

LastEffectiveUnit (Operation)

LastEvaluationDate (ShopKanban)

LastEvaluationDate (SourceKanban)

LastModified (Assumption)

LastModified (ScheduledReceipt)

LastOnAsOfDate (HistoricalDemandSeries)

LastOnAsOfDate (HistoricalSupplySeries)

LastSeenTime (OriginatingFile)

LastUnit (CostRollUp)

LastUnit (FlatBillDown)

LastUnit (FlatBillUp)

LastUpdated (CollaborativeForecast)

Late (LateSupply)

Late (WhereConsumed)

Late (WhereConsumedForDemand)

Late (WhereConsumedForForecast)

Late (WhereConsumedForSupply)

LateAtHere (LateSupply)

LateAtHere (WhereConsumed)

LateAtHere (WhereConsumedForDemand)

LateAtHere (WhereConsumedForForecast)

LateAtHere (WhereConsumedForSupply)

LateBeyondToleranceQuantity(CTPActivity)

LatePart (LateSupply)

LateQuantity (LateSupply)

LatestDueDate (IndependentDemandSolution)

LatestFinishDate (Task)

LatestStartDate (Task)

LateSupplies (IndependentDemand)

LeadTime (HistoricalSupplyActual)

LeadTime (MEIOHistoricalSupply)

LeadTime (MEIOStage)

LeadTime (SafetyStockHistoricalSupply)

LeadTime (SafetyStockItem)

LeadTime (SafetyStockPart)

LeadTime (SafetyStockResult)

LeadTimeAdjust (Part)

LeadTimeCalendar (HistoricalSupplyHeader)

LeadTimeCalendar (SafetyStockItemType)

LeadTimeCalendarSafetyStockItemTypes (Calendar)

LeadTimeCorrectionStatus (ItemSource)

LeadTimeDate (PartSource)

LeadTimeOffset (BillOfMaterial)

LeadTimeOutlierMovingAverageWindow (MEIOLeadTime)

LeadTimeOutlierMovingAverageWindow (SafetyStockItem)

LeadTimeOutlierThreshold (MEIOLeadTime)

LeadTimeOutlierThreshold (SafetyStockItem)

LeadTimeOutlierType (MEIOLeadTime)

LeadTimeOutlierType (SafetyStockItem)

LeadTimeOverride (SupplyStatus)

LeadTimePerInterval (SafetyStockItemType)

LeadTimeRule (SupplyType)

LeadTimeSmoothingAfterIntervalCount (SafetyStockItem)

LeadTimeSmoothingBeforeIntervalCount (SafetyStockItem)
LeadTimeType (SHSCConfiguration)
LeadTimeUnit (Source)
Level (FeatureBOMForIndependentDemand)
Level (FlatBillDown)
Level (FlatBillUp)
Level (FlatDemand)
Level (MLSActivity)
Level (WhereConsumedForDemand)
Level (WhereConsumedForForecast)
Level (WhereConsumedForSupply)
LevelResources (TaskType)
LicenseType (Site table)
LIFNR_Vendor (SAP_MD04)
LimitEffectiveDate (BOMType)
Line (BaselineTaskLink)
Line (HistoricalDemandActual)
Line (IndependentDemand)
Line (PlannedOrder)
Line (ScheduledReceipt)
Line (SubstituteSupply)
Line (Supply)
LinearRate (CurveParameters)
LineCreatedDate (HistoricalDemandActual)
LineQuantity (HistoricalDemandActual)
LineReceivedDate (HistoricalReceipt)
Lines (DemandOrder)
Lines (SupplyOrder)
LineValue (ScheduledReceipt)
Links (Task)
Load (ConstraintLoad)
Load (ConstraintSpreadLoad)
Load (ConstraintUsed)
Load (ResourceDataByDate)
Load (ResourceLoad)
LoadsDailyPlanned (WorkCenter)
LoadDate (TrialConstraintShortage)
LoadDate (TrialConsumption)
LoadFLPPlanneds (Part)
LoadsReceiptsPlanned (Routing)
Loads (Resource)
LoadsDailyCurrent (WorkCenter)
LoadsDetailCurrent (WorkCenter)
LoadsDetailPlanned (WorkCenter)
LoadsNewOrderPlanned (Routing)
LoadsReceiptsCurrent (Routing)
LoadWeeklyCurrent (WorkCenter)
LoadWeeklyPlanned (WorkCenter)
Location (OnHand)
Location (TimePhasedLocationCapacity)
LocationCapacities (UnitOfMeasure)
Locations (Warehouse)
LoggingRule (MultiLevelSearchRule)
LogisticsLocation (Carrier)
LogisticsLocations (LogisticsLocation)
LongTermDaysSupplyRule (DaysSupplyPolicy)
LongTermNumberOfDaysSupply (DaysSupplyPolicy)
LongTermPeriodSupplyDateRule (DaysSupplyPolicy)
LongTermPeriodSupplyDueCalendar (DaysSupplyPolicy)
LongTermPeriodSupplyInterval (DaysSupplyPolicy)
LotSizeLastOrder (OrderPolicy)
LowerBound (ProjectedLeadTime)
LowerBound (ProjectedYield)
LowerThreshold (SafetyStockItemOutlierResult)
LowerThreshold (StatisticalForecastOutlier)
LowLevelCode (Part)

M

MachineCapacity (LoadDailyCurrent)

MachineCapacity (LoadDailyPlanned)

MachineCapacity (LoadWeeklyCurrent)

MachineCapacity (LoadWeeklyPlanned)

MachineIdle (LoadDailyCurrent)

MachineIdle (LoadDailyPlanned)

MachineIdle (LoadDetailCurrent)

MachineIdle (LoadDetailPlanned)

MachineIdle (LoadFLPPlanned)

MachineIdle (LoadWeeklyCurrent)

MachineIdle (LoadWeeklyPlanned)

MachineRun (LoadDailyCurrent)

MachineRun (LoadDailyPlanned)

MachineRun (LoadDetailCurrent)

MachineRun (LoadDetailPlanned)

MachineRun (LoadFLPPlanned)

MachineRun (LoadWeeklyCurrent)

MachineRun (LoadWeeklyPlanned)

MachineRunRatio (Capacity)

MachineSetup (LoadDailyCurrent)

MachineSetup (LoadDetailCurrent)

MachineSetup (LoadDetailPlanned)

MachineSetup (LoadFLPPlanned)

MachineSetup (LoadWeeklyCurrent)

MachineSetupRatio (Capacity)

MaintainCommitments (OrderPriority)

MajorCleanup (ProductionGroup)

MajorSetup (ProductionGroup)

Manager (Project)

MappedPool (Allocation)

MappedPool (IndependentDemand)

MappedPool (NewAllocation)

MappedPool (NewTransferAllocation)

MappedPool (PlannedAllocation)

MappedPool (PlannedTransferAllocation)

MasterSchedulerCode (PartSolution)

MasterSchedulerCodes (PlannerCode)

MaterialAvailableDate (CTPPlannedOrder)

MaterialAvailableDate (ScheduledReceipt)

MaterialCost (BaselineTaskLink)

MaterialCost (CostOfGoodsSold)

MaterialCost (Part)

MaterialCost (PartSource)

MaterialCost (PartSourceTimePhasedAttributes)

MaterialCost (Project)

MaterialCost (Task)

MaterialCost (TaskDemandLink)

MaterialCost (TaskPartLink)

MaterialCost (TaskSupplyLink)

MaterialCost (WhereConsumedForDemand)

MaterialCost (WhereConsumedForForecast)

MaterialCost (WhereConsumedForSupply)

MaterialCostOnly (CostRollUp)

MaterialRevenue (Project)

MaterialRevenue (Task)

MaterialSpend (CTPForecastCost)

MaterialSpend (IndependentDemand)

MaterialSpend (IndependentDemandCost)

MaterialStartDate (CTPPlannedOrder)

MaterialStartDate (MLSActivity)

MaterialStartDate (ScheduledReceipt)

MaterialStartDate (SupplyDemand)

MATNR_Material (SAP_MD04)

MaxCriticalShortage (InventorySummary)

MaximumBinQty (ShopKanban)

MaximumBinQty (SourceKanban)

MaximumConstraintConsumption (ProductionGroup)

MaximumDaysOfSupply (SafetyStockOverride)

MaximumDaysOfSupply (SafetyStockResult)

MaximumDaysOfSupply (SafetyStockTimePhasedResult)

MaximumInventoryResults (Part)

MaximumLaborCapacity (Capacity)

MaximumLaborCapacity (LoadDailyCurrent)

MaximumLaborCapacity (LoadDailyPlanned)

MaximumLaborCapacity (LoadWeeklyCurrent)

MaximumLaborCapacity (LoadWeeklyPlanned)

MaximumMachineCapacity (Capacity)

MaximumMachineCapacity (LoadDailyCurrent)

MaximumMachineCapacity (LoadDailyPlanned)

MaximumMachineCapacity (LoadWeeklyCurrent)

MaximumMachineCapacity (LoadWeeklyPlanned)

MaximumNumberOfCustomerChanges (OFConfiguration)

MaximumNumberOfCustomerChanges (PartCustomer)

MaximumNumberOfSupplyChanges (OFConfiguration)

MaximumNumberOfSupplyChanges (PartCustomer)

MaximumQty (PartSource)

MaximumQuantity (SafetyStockOverride)

MaximumQuantity (SafetyStockTimePhasedBounds)

MaximumResources (CRPOperation)

MaximumSafetyStock (SafetyStockResult)

MaximumSafetyStock (SafetyStockTimePhasedResult)

MaximumUnits (Resource)

MaximumUnits (ResourceAvailable)

MaximumUsage (OrderPolicy)

MaximumValue (CurveParameters)

MEIOAllowSafetyStockOnArtificialStages (AnalyticsConfiguration)

MEIODemandHistoricalsPropagationRule (SafetyStockItemType)

MEIODriverParts (Part)

MEIOFamilies (Part)

MEIOFamily (Part)

MEIOFamily (SafetyStockResult)

MEIOFutureDemandIntervalCount (AnalyticsConfiguration)

MEIOLeadTime (MEIOLeadTimeOutlier)

MEIOLeadTime (MEIOLeadTimeOutlierSummary)

MEIOLeadTimes (MEIOLeadTimeDistributionProfile)

MEIOLeadTimes (PartSource)

MEIOLeadTimeTypes (ControlSet)

MEIOMaximumIterations (AnalyticsConfiguration)

MEIOMaximumTime (AnalyticsConfiguration)

MEIOParentParts (Part)

MEIOParts (PartSource)

MEIOReportCalendar (AnalyticsConfiguration)

MEIOReportIntervalCount (AnalyticsConfiguration)

MEIOStage (SafetyStockResult)

MEIOStageLinks (Part)

MEIOStageResults (Part)

MEIOStages (Part)

MEIOStageTimePhasedResults (Part)

Metric (ConstraintMetric)

Metric (Metric)

Metric (PartMetric)

Metric (SiteMetric)

Metric (SupplierMetric)

Milestone (Task)

MinimumConstraintConsumption (ProductionWheelConstraint)

MinimumDaysOfSupply (SafetyStockOverride)

MinimumDaysOfSupply (SafetyStockResult)

MinimumDaysOfSupply (SafetyStockTimePhasedBounds)

MinimumDaysOfSupply (SafetyStockTimePhasedResult)

MinimumQty (PartSource)

MinimumQuantity (SafetyStockOverride)

MinimumQuantity (SafetyStockTimePhasedBounds)

MinimumSafetyStock (SafetyStockResult)

MinimumSafetyStock (SafetyStockTimePhasedResult)

MinimumShelfLife (IndependentDemand)

MinimumShelfLife (Part)

MinimumShelfLife (PartCustomer)

MinimumShelfLife (PartCustomerTimePhasedAttributes)

MinimumShelfLifeThreshold (PartSource)

MinimumUsage (OrderPolicy)

MinorChangeOver (ProductionGroup)

MissingFeatures (BillOfMaterialFeatureRule)

MissingFeatures (IndependentDemandFeature)

MLSActivity (TrialConstraintShortage)

MLSAllocationRule (OrderPriority)

MLSFairshareCalendar (OrderPriority)

MLSFairshareSliceFactor (OrderPriority)

MLSPreTargetSequence (PartSource)

MLSTrialLimitPerDemand (AnalyticsConfiguration)

MNG01_RecordedRequiredQuantity (SAP_MD04)

Model (Activity)

Model (Allocation)

Model (AvailableToPromise)

Model (BaselinePlannedOrder)

Model (BillOfMaterial)

Model (BlowThroughNewAllocation)

Model (BlowThroughPlannedAllocation)

Model (BOM_MUECUMLead)

Model (CoByProductSupply)

Model (ConstraintLoad)

Model (ConstraintSpreadLoad)

Model (CostOfGoodsSold)

Model (CostRollUp)

Model (CRPOperation)

Model (CTPActivity)

Model (Demand)

Model (DemandException)

Model (DependentDemand)

Model (FlatBillDown)

Model (FlatBillUp)

Model (Forecast)

Model (IndependentDemand)

Model (InProcess)

Model (InventoryAnalysis)

Model (InventoryCTPAnalysis)

Model (InventorySummary)

Model (InventoryTransfer)

Model (MLSActivity)

Model (OnHand)

Model (Operation)

Model (Part_MUECumLead)

Model (Part_MUEPoolValidation)

Model (PeriodsOfCoverage)

Model (PeriodsOfCTPCoverage)

Model (PlannedAllocation)

Model (PlannedOrder)

Model (RegressionModelTerm)

Model (ScheduledReceipt)

Model (SubstituteSupply)

Model (Supply)

Model (SupplyAllocation)

Model (SupplyDemand)
Model (TrialCoByProductSupply)
Model (TrialPlannedOrder)
Model (TrialSubstituteSupply)
Model (ValidatePlannedOrder)
Model (WhereConsumed)
Model (WhereConsumedForDemand)
Model (WhereConsumedForForecast)
Model (WhereConsumedForSupply)
ModelConstantUsage (ForecastItemParametersType)
ModelRule (MUEPoolNettingType)
ModelUsage (DemandStatus)
ModelUsage (SupplyStatus)
MovingAverageIntervalCount (BestFitProfileDetail)
MovingAverageIntervalCount (ForecastItemParameters)
MovingAverageTerms (BestFitProfileDetail)
MovingAverageTerms (ForecastItemParameters)
MPSAutoFirmWindow (PartSolution)
MPSCalendars (Calendar)
MPSConfigDemandSource (BomType)
MPSFrozenWindow (PartSolution)
MPSHorizon (PartSolution)
MPSRunDateCalendars (Calendar)
MPSToleranceCalendars (Calendar)
MUECumLeads (Part)
MUEPoolNettingType (Part)
MUEPoolNettingTypes (ControlSet)
MUEPoolValidations (Part)
MUERule (BOMType)
MUERule (OperationType)
MultiEchelonSafetyStockRule (Part)
MultiLevelSearchRule (Part)
MultiLevelSearchRule (ShipSetType)
MultipleQty (PartSource)
MultipleUsage (OrderPolicy)
Multiplier (ForecastItemParametersMap)
Multiplier (SafetyStockAverageDemandProfile)
Multiplier (SafetyStockItemMapping)
MultiSiteAssemblyLLC (BillOfMaterial)
MultiSiteCumLeadTime (BillOfMaterial)
MultiSiteCumLeadTime (BOM_MUECUMLead)
MultiSiteCumLeadTime (Part)
MultiSiteLowLevelCode (Part)

N

Name (Activity)
Name (BestFitProfile)
Name (BonusSchedule table)
Name (Cluster)
Name (Constraint)
Name (Currency)
Name (Customer)
Name (Event)
Name (EventPhase)
Name (ForecastItemParametersRegressionCoefficient)
Name (Job)
Name (Location)
Name (MEIOLeadTimeDistributionProfile)
Name (OriginatingFile)
Name (OriginatingSource)
Name (Part)
Name (PenaltySchedule)
Name (PointsCurve)
Name (PointsProfile)
Name (ProcessInstance)
Name (ProductionGroup)
Name (ProductionWheel)

Name (Project)
Name (ProjectGroup)
Name (ProjectManager)
Name (Region)
Name (RegionGroup)
Name (RegressionModel)
Name (RegressionModelTerm)
Name (Regressor)
Name (RegressorCategory)
Name (Resource)
Name (ResourceGroup)
Name (RParameter)
Name (RParameterName)
Name (RParameterSet)
Name (SafetyStockAverageDemandProfile)
Name (StatisticalForecastRegressionCoefficient)
Name (Supplier)
Name (Warehouse)
Name (WorkCenter)
NearToleranceAction (AutomatedLeadTimeType)
NearToleranceAction (AutomatedYieldType)
NearToleranceThreshold (AutomatedLeadTimeType)
NearToleranceThreshold (AutomatedYieldType)
NeedDate (CostOfGoodsSold)
NeedDate (FlatDemand)
NeedDate (LateSupply)
NeedDate (MLSActivity)
NeedDate (WhereConsumed)
NeedDate (WhereConsumedForDemand)
NeedDate (WhereConsumedForForecast)
NeedDate (WhereConsumedForSupply)
NeedDate (WhereConsumedToHighVolumeOrder)
NeedQty (PlannedOrder)
NeedQty (ScheduledReceipt)
NeedQty (Supply)
NeedQuantity (CostOfGoodsSold)
NeedQuantity (LateSupply)
NeedQuantity (FLPConstraint)
NeedQuantity (MLSActivity)
NeedQuantity (WhereConsumed)
NeedQuantity (WhereConsumedForDemand)
NeedQuantity (WhereConsumedForForecast)
NeedQuantity (WhereConsumedForSupply)
NeedQuantity (WhereConsumedToHighVolumeOrder)
NewAllocation (Forecast)
NewAllocation (ForecastConsumption)
NewAllocations (BillOfMaterial)
NewProcessCost (CTPActivity)
NewProcessCost (CTPCoByProductSupply)
NewProcessCost (CTPPlannedOrder)
NewProcessCost (CTPSubstituteSupply)
NewProcessCost (IndependentDemand)
NewProcessCost (ScheduledReceipt)
NewProcessCost (SupplyDemand)
NewProcessCost (WhereConsumed)
NewProcessCost (WhereConsumedForDemand)
NewProcessCost (WhereConsumedForForecast)
NewProcessCost (WhereConsumedForSupply)
NewPurchaseCost (CTPCoByProductSupply)
NewPurchaseCost (CTPSubstituteSupply)
NewPurchasesCost (CTPActivity)
NewPurchasesCost (CTPPlannedOrder)
NewPurchasesCost (IndependentDemand)
"NewPurchasesCost" on page 760 (MultiLevelSearchRule)
NewPurchasesCost (ScheduledReceipt)
NewPurchasesCost (SupplyDemand)
NewPurchasesCost (WhereConsumed)

NewPurchasesCost (WhereConsumedForDemand)
NewPurchasesCost (WhereConsumedForForecast)
NewPurchasesCost (WhereConsumedForSupply)
NewSupplyStatus (SHSCConfiguration)
NewTransferAllocation (Forecast)
NewTransferAllocation (ForecastConsumption)
NewTransferAllocations (PartSource)
NewTransferType (DRPConfiguration)
NextUnit (Part)
NextWorkCenter (LoadDetailCurrent)
NextWorkCenter (LoadDetailPlanned)
NextWorkCenter (LoadOperationNewOrderPlanned)
NextWorkCenter (LoadOperationReceiptsCurrent)
NextWorkCenter (LoadOperationReceiptsPlanned)
NonStationaryDemandRule (SafetyStockItemType)
Notes (EngineeringChange)
Notes (ForecastItemParametersOutlier)
Notes (IndependentDemand)
Notes (InventoryTransfer)
Notes (MEIOLeadTimeOutlier)
Notes (PartValidation)
Notes (SafetyStockItemOutlier)
Notes (ScheduledReceipt)
Notes (SupplyAllocation)
Notes (Task)
Notifications (Activity)
NotifyOwner (Notification)
NotifyPerformers (Notification)
Number (CRPOperation)
Number (HistoricalDemandSeries)
Number (HistoricalSupplySeries)
Number (Operation)
NumberOfCustomerChanges (HistoricalDemandActual)
NumberOfDaysSupply (DaysSupplyPolicy)
NumberOfDaysSupply (Part)
NumberOfMachines (LoadDailyCurrent)
NumberOfMachines (LoadDailyPlanned)
NumberOfPoints (SpreadProfile)
NumberOfResources (Capacity)
NumberOfResources (CapacityOverride)
NumberOfResources (WorkCenterCapacity)
NumberOfSupplyChanges (HistoricalDemandActual)
NumberOfWorkers (LoadDailyCurrent)
NumberOfWorkers (LoadDailyPlanned)

O

ObsoleteTransition (IMConfiguration)
Offset (BaselineTaskLink)
Offset (Predecessor)
Offset (ProductionRecurrence)
Offset (SafetyStockAverageDemandProfile)
Offset (SafetyStockRangeOfCoverage)
Offset (TaskDemandLink)
Offset (TaskSupplyLink)
OneTimeBonus (BonusScheduleByDate table)
OneTimeBonus (BonusScheduleByInterval table)
OneTimeCost (PenaltyScheduleByDate)
OneTimeCost (PenaltyScheduleByInterval)
OnHand (Activity)
OnHand (CostOfGoodsSold)
OnHand (CTPActivity)
OnHand (SupplyDemand)
OnHand (WhereConsumed)
OnHand (WhereConsumedForDemand)
OnHand (WhereConsumedForForecast)
OnHand (WhereConsumedForSupply)
OnHandDemand (SupplyDemand)

OnHands (Location)
OnHands (Model)
OnHands (OnHandType)
OnHands (Part)
OnHandTypes (ControlSet)
OnTimeAllocationRule (PartType)
OnTimeQuantity (CTPActivity)
OnTimeQuantity (CTPCoByProductSupply)
OnTimeQuantity (CTPForecast)
OnTimeQuantity (CTPPlannedOrder)
OnTimeQuantity (CTPSubstituteSupply)
OnTimeQuantity (IndependentDemand)
OnTimeQuantity (ScheduledReceipt)
Operation (Allocation)
Operation (BillOfMaterial)
Operation (LoadDetailCurrent)
Operation (LoadDetailPlanned)
Operation (LoadFLPPlanned)
Operation (LoadOperationNewOrderPlanned)
Operation (LoadOperationReceiptsCurrent)
Operation (LoadOperationReceiptsPlanned)
OperationEndDate (LoadDetailCurrent)
OperationEndDate (LoadDetailPlanned)
OperationNumber (BillOfMaterial)
Operations (CRPUnitOfMeasure)
Operations (Model)
Operations (OperationType)
Operations (Routing)
Operations (WorkCenter)
OperationSequences (OperationSeqeunceType)
OperationStartDate (LoadDetailCurrent)
OperationStartDate (LoadDetailPlanned)
OperationTypes (ControlSet)
OptimizationPriority (OrderPriority)
OptionRatio (BillOfMaterial)
Order (Demand)
Order (HistoricalDemandActual)
Order (HistoricalSupplyActual)
Order (IndependentDemand)
Order (ScheduledReceipt)
Order (Supply)
OrderCreation (HistoricalDemandActual)
OrderCreation (IndependentDemandSolution)
OrderDate (HistoricalSupplyActual)
OrderDueDate (HistoricalSupplyActual)
OrderGenerationRule (OrderPolicy)
OrderId (BaselineTaskLink)
OrderPointDateRule (PartType)
OrderPointDateRule (SafetyStockPolicy)
OrderPointIncrease (PartType)
OrderPointIncrease (PlanningCalendars)
OrderPointIncrease (SafetyStockPolicy)
OrderPolicies (ControlSet)
OrderPolicy (PartSource)
OrderPolicy (SHSCConfiguration)
OrderPriorities (Calendar)
OrderPriorities (ControlSet)
OrderPriority (Activity)
OrderPriority (BlowThroughPlannedAllocation)
OrderPriority (CTPActivity)
OrderPriority (CTPSafetyStockHoldings)
OrderPriority (DemandException)
OrderPriority (Forecast)
OrderPriority (IndependentDemand)
OrderPriority (LateSupply)
OrderPriority (MLSActivity)
OrderPriority (PartCustomer)
OrderPriority (PartCustomerTimePhasedAttributes)

OrderPriority (PlannedOrder)

OrderPriority (ScheduledReceipt)

OrderPriority (SubstituteSupply)

OrderPriority (SupplyAllocation)

OrderPriority (SupplyDemand)

OrderPriority (TimePhasedSafety)

OrderPriority (TrialPlannedOrder)

OrderPriority (TrialSubstituteSupply)

OrderPromised (IndependentDemandSolution)

OrderPromisedDateTime (IndependentDemandSolution)

OrderQuantity (SafetyStockItem)

Orders (Customer)

Orders (Supplier)

OrderSite (BaselineTaskLink)

OrderType (BaselineTaskLink)

Organization (SupplyOrder)

OrgQualifier (SupplyOrder)

Origin (DataChange_Historical)

Origin (DataChange_Local)

Origin (DataChange_PendingUpdate)

Origin (DataChange)

Origin (PlannedOrder)

OriginalDueDate (ScheduledReceipt)

OriginalFieldValues (DataChange_Historical)

OriginalFieldValues (DataChange_Local)

OriginalFieldValues (DataChange_PendingUpdate)

OriginalFieldValues (DataChange)

OriginalFinishDate (Task)

OriginalLeadTime (MEIOHistoricalSupply)

OriginalLeadTime (SafetyStockHistoricalSupply)

OriginalParts (Part)

OriginalQuantity (EventForecastDetailAdjustment)

OriginalQuantity (SafetyStockHistoricalDemand)

OriginalQuantity (SafetyStockOverride)

OriginalQuantity (ScheduledReceipt)

OriginalRecordId (IndependentDemand)

OriginalRecordId (InventoryTransfer)

OriginalRecordId (ScheduledReceipt)

OriginalStartDate (Task)

OriginatingFiles (OriginatingSource)

Originator (EngineeringChange)

OuterCalendar (ForecastDisaggregationParameters)

OuterCalendar (ForecastDisaggregationParametersByCategory)

OutgoingFlowList (Activity)

OutgoingFlows (Activity)

OutgoingServiceTime (MEIOStageResult)

OutgoingServiceTime (SafetyStockPart)

OutgoingServiceTime (SafetyStockResult)

OutlierAdjustment (StatisticalForecastOutlier)

OutlierAdjustment (SafetyStockItemOutlierResult)

OutlierMovingAverageWindow (ForecastItemParameters)

OutlierQuantity (ForecastItemParametersActual)

Outliers (ForecastItemParameters)

OutlierSmoothingAfterIntervalCount (ForecastItemParameters)

OutlierSmoothingBeforeIntervalCount (ForecastItemParameters)

OutlierThreshold (ForecastItemParameters)

OutlierType (ForecastItemParameters)

OutlineLevel (Activity)

OutlineLevel (Task)

OutlineNumber (Activity)

OutlineNumber (Task)

OutsideToleranceAction (AutomatedLeadTimeType)

OutsideToleranceAction (AutomatedYieldType)

OutsideToleranceThreshold (AutomatedLeadTimeType)

OutsideToleranceThreshold (AutomatedYieldType)

OverheadCost (CostOfGoodsSold)

OverheadCost (CostRollUp)

OverheadCost (PartSource)

OverheadCost (PartSourceTimePhasedAttributes)

OverheadCost (WhereConsumedForDemand)

OverheadCost (WhereConsumedForForecast)

OverheadCost (WhereConsumedForSupply)

OverheadCost2 (CostOfGoodsSold)

OverheadCost2 (CostRollUp)

OverheadCost2 (PartSource)

OverheadCost2 (PartSourceTimePhasedAttributes)

OverheadCost2 (WhereConsumedForDemand)

OverheadCost2 (WhereConsumedForForecast)

OverheadCost2 (WhereConsumedForSupply)

Overlap (ConstraintAvailable)

Overloaded (ConstraintSpreadLoad)

Overloaded (ConstraintUsed)

Overloaded (PeriodConstraintLoad)

Overloaded (Resource)

OverloadedDate (Resource)

OverrideNotifications (Activity)

OverrideQuantity (ConsensusForecast)

OvertimeCost (ResourceCostActual)

OvertimeCost (ResourceLoad)

OvertimeHours (ResourceCostActual)

OvertimeHours (ResourceLoad)

OvertimeRate (Resource)

OvertimeRate (ResourceCosts)

OvertimeRate (ResourceDataByDate)

OwnerId (ProcessInstance)

P

ParameterDate (PlannedOrder)

ParameterDate (ScheduledReceipt)

ParameterName (ForecastItemFitOutput)

Parameters (ForecastItem)

ParameterValue (ForecastItemFitOutput)

Parent (Activity)

Parent (SubProcess)

Parent (Task)

Parent (WBS)

ParentCTPCoByProductSupply (SupplyDemand)

ParentConsensusForecast (DependentConsensusForecast)

ParentCTPPlannedOrder (SupplyDemand)

ParentCTPPlannedOrder (WhereConsumedForSupply)

ParentCTPPlannedOrder (WhereConsumedForForecast)

ParentCTPSubstituteSupply (SupplyDemand)

ParentCTPSubstituteSupply (WhereConsumedForForecast)

ParentCTPSubstituteSupply (WhereConsumedForSupply)

ParentInventoryTransfer (WhereConsumedForForecast)

ParentInventoryTransfer (WhereConsumedForSupply)

ParentPart (MEIOParentPart)

ParentPart (MLSActivity)

ParentPart (WhereConsumedForForecast)

ParentPart (WhereConsumedForSupply)

ParentPartSource (DependentConsensusForecast)

ParentPartSource (FlatDemand)

ParentPartSource (MLSActivity)

Parents (Activity)

Parents (Task)

ParentScheduledReceipt (SupplyDemand)

ParentScheduledReceipt (WhereConsumedForForecast)

ParentScheduledReceipt (WhereConsumedForSupply)

ParentPartSource (MLSActivity)

ParentSupplyDemand (WhereConsumedForForecast)

ParentSupplyDemand (WhereConsumedForSupply)

ParentType (WhereConsumed)

ParentType (WhereConsumedForDemand)

ParentType (WhereConsumedForForecast)

ParentType (WhereConsumedForSupply)

Part (Activity)

Part (Allocation)

Part (AlternateRouting)

Part (AssumptionByPart)

Part (AvailableToPromise)

Part (BaselinePlannedOrder)

Part (BlowThroughNewAllocation)

Part (BlowThroughPlannedAllocation)

Part (CoByProductSupply)

Part (CollaborativeForecast)

Part (ConsensusForecast)

Part (ConsensusForecastDetail)

Part (CostOfGoodsSold)

Part (CostRollUp)

Part (CTPActivity)

Part (CTPCoByProductSupply)

Part (CTPForecast)

Part (CTPForecastCost)

Part (CTPPlannedOrder)

Part (CTPSafetyStockHoldings)

Part (CTPSubstituteSupply)

Part (CustomerPrice)

Part (Demand)

Part (DemandException)

Part (DependentConsensusForecast)

Part (DependentDemand)

Part (DirectAssembly)

Part (DirectComponent)

Part (EventConsensusForecastDetail)

Part (FeatureMap)

Part (FlatAssembly)

Part (FlatBillDown)

Part (FlatBillUp)

Part (FlatComponent)

Part (FlatConstraint)

Part (FlatDemand)

Part (Forecast)

Part (ForecastConsumption)

Part (HistoricalPartKPI)

Part (IndependentDemand)

Part (IndependentDemandByFeature)

Part (IndependentDemandFeatureSummary)

Part (InProcess)

Part (InventoryAnalysis)

Part (InventoryCTPAnalysis)

Part (InventorySummary)

Part (InventoryTransfer

Part (ItemSource)

Part (ItemSourceCandidate)

Part (LoadDetailPlanned)

Part (LoadFLPPlanned)

Part (LoadOperationNewOrderPlanned)

Part (MaximumInventoryResult)

Part (MEIODriverPart)

Part (MEIOParentPart)

Part (MEIOStage)

Part (MEIOStageHistoricalDemand)

Part (MEIOStageHistoricalForecast)

Part (MEIOStageLink)

Part (MEIOStageResult)

Part (MEIOStageTimePhasedResult)

Part (MLSActivity)

Part (OnHand)

Part (Part_MUECumLead)

Part (Part_MUEPoolValidation)

Part (PartCustomer)

Part (PartMetric)

Part (PartSequence)

Part (PartSolution)

Part (PartSource)

Part (PartSupplier)

Part (PartUOMConversion)

Part (PartValidation)

Part (PeriodsOfCoverage)

Part (PeriodsOfCTPCoverage)

Part (PoolMapOverride)

Part (RangeOfCoveragePartOverride)

Part (SafetyStockItem)

Part (SafetyStockOverride)

Part (SafetyStockPart)

Part (SafetyStockRangeOfCoverage)

Part (SafetyStockResult)

Part (SafetyStockTimePhasedResult)

Part (SAP_MD04)

Part (ScheduledReceipt)

Part (ShopKanban)

Part (SubstituteSupply)

Part (Supply)

Part (SupplyDemand)

Part (TaskPartLink)

Part (TimePhasedMaximumInventory)

Part (TimePhasedSafety)

Part (TransferDestination)

Part (TrialCoByProductSupply)

Part (TrialConsumption)

Part (TrialPlannedOrder)

Part (TrialSubstituteSupply)

Part (ValidatePlannedOrder)

Part (WhereConsumed)

Part (WhereConsumedForDemand)

Part (WhereConsumedForForecast)

Part (WhereConsumedForSupply)

Part (WhereConsumedToHighVolumeOrder)

PartClass (Part)

PartCustomer (AssumptionByPartCustomer)

PartCustomer (ConsensusForecast)

PartCustomer (DependentConsensusForecast)

PartCustomer (DisaggregationRateByPartCustomer)

PartCustomer (Forecast)

PartCustomer (HistoricalDemandHeader)

PartCustomer (IndependentDemand)

PartCustomer (PartCustomerTimePhasedAttributes)

PartCustomer (StatisticalForecastDetail)

PartCustomer (StatisticalForecastDisaggregationRate)

PartCustomers (Calendar)

PartCustomers (ForecastItem)

PartCustomers (Part)

PartCustomers (ToleranceProfile)

PartCustomerTimePhasedAttributes (AllotmentOverride)

PartCustomerTimePhasedAttributes (OrderPriority)

PartCustomerTypes (DemandType)

PartCustomerTypes (OrderPriority)

PartialExplode (SupplyStatus)

PartialRule (ShipmentPolicy)

PartLink (TaskDemandLink)

PartLink (TaskSupplyLink)

PartLinks (Task)

PartMetric (PartMetricValues)

PartMetricValues (PartMetric)

PartMetrics (Metric)

PartMetrics (MetricAsOf)

PartName (BaselineTaskLink)

PartnerFunc (SupplyOrder)

Parts (ABCCode)

Parts (AvailableRule)

Parts (BuyerCode)

Parts (Class)

Parts (DaysSupplyPolicy)

Parts (DistributionPlanningRule)

Parts (ExpiryType)

Parts (IncrementalRule)

Parts (MUEPoolNettingType)

Parts (MultiEchelonSafetyStockRule)

Parts (MultiLevelSearchRule)

Parts (PartType)

Parts (PlannerCode)

Parts (PlanningCalendars)

Parts (ProductFamily)

Parts (ReferencePart)

Parts (SafetyLeadTimeProfile)

Parts (SafetyStockPolicy)

Parts (Site)

Parts (SourceRule)

Parts (UnitOfMeasure)

PartSequences (ProductionWheel)

PartSite (BaselineTaskLink)

PartSolutions (Part)

PartSolutions (Segment)

PartSource (Activity)

PartSource (AssumptionByPartSource)

PartSource (CostOfGoodsSold)

PartSource (CTPActivity)

PartSource (FlatBillDown)

PartSource (FlatBillUp)

PartSource (FlatDemand)

PartSource (InProcess)

PartSource (ItemSourcePartSourceHistoricalLeadTime)

PartSource (ItemSourcePartSourceHistoricalReceipt)

PartSource (ItemSourceIntervalEffectivePartSource)

PartSource (LateSupply)

PartSource (MEIOLeadTime)

PartSource (MEIOStage)

PartSource (NewTransferAllocation)

PartSource (PartSourceCTPPlannedOrder)

PartSource (PartSourcePlannedOrder)

PartSource (PartSourceScheduledReceipt)

PartSource (PlannedOrder)

PartSource (PlannedTransferAllocation)

PartSource (SafetyStockPart)

PartSource (SafetyStockResult)

PartSource (ScheduledReceipt)

PartSource (SourceConstraint)

PartSource (Supply)

PartSource (SupplyAllocation)

PartSource (TimePhasedPartSourceLeadTIme)

PartSource (TimePhasedYield)

PartSource (TrialPlannedOrder)

PartSource (WhereConsumed)

PartSource (WhereConsumedForDemand)

PartSource (WhereConsumedForForecast)

PartSource (WhereConsumedForSupply)

PartSourceLeadTime (ItemSourcePartSourceHistoricalLeadTime)

PartSourceLimit (MultiLevelSearchRule)

PartSources (AllotmentOverride)

PartSources (BOMAlternate table)

PartSources (OrderPolicy)

PartSources (Part)

PartSources (PartSourceType)

PartSources (ProductionGroup)

PartSources (ProductionRecurrence)

PartSources (ProductionWheel)

PartSources (Routing)

PartSources (UnitOfMeasure)

PartSourceTypes (ControlSet)

PartSourceTypes_PlannedOrder (SupplyStatus)

PartSourceTypes_PlannedOrder (SupplyType)

PartSourceTypes_PlannedOrderToSR (SupplyStatus)

PartSourceTypes_PlannedOrderToSR (SupplyType)

PartSourceTypes_ScheduledReceipts (SupplyStatus)

PartSourceTypes_SupplyOrders (SupplyType)

PartSourceYield (ItemSourcePartSourceHistoricalReceipt)

PartSupplier (HistoricalReceiptHeader)

PartSupplier (HistoricalSupplyHeader)

PartSupplier (ScheduledReceipt)

PartSupplier (SupplyAllocation)

PartSuppliers (Part)

PartSuppliers (Supplier)

PartSuppliers (ToleranceProfile)

PartSuppliers (UnitOfMeasure)

PartType (PoolMap)

PartTypes (ControlSet)

PartTypes (OrderPriority)

Path (ForecastItem)

PathExpression (ForecastItem)

PathLevels (ForecastItem)

PaymentTerms (SupplyOrder)

PegCoByProductPart (SupplyDemand)

PegCTPPlannedOrder (WhereConsumed)

PegCTPPlannedOrder (WhereConsumedForDemand)

PegCTPSubstituteSupply (WhereConsumed)

PegCTPSubstituteSupply (WhereConsumedForDemand)

PegDueDate (BlowThroughPlannedAllocation)

PegInventoryTransfer (WhereConsumed)

PegInventoryTransfer (WhereConsumedForDemand)

PegModel (Activity)

PegModel (CTPActivity)

PegModel (Demand)

PegModel (DependentDemand)

PegModel (LoadDetailPlanned)

PegModel (LoadOperationNewOrderPlanned)

PegPart (Activity)

PegPart (BlowThroughPlannedAllocation)

PegPart (CTPActivity)

PegPart (Demand)

PegPart (DependentDemand)

PegPart (LateSupply)

PegPart (SupplyDemand)

PegPart (WhereConsumed)

PegPart (WhereConsumedForDemand)

PegPartSource (BlowThroughPlannedAllocation)

PegPartSource (FlatBillDown)

PegPlannedOrder (PlannedAllocation)

PegPool (Activity)

PegPool (CTPActivity)

PegPool (Demand)

PegPool (DependentDemand)

PegPool (LoadDetailPlanned)

PegPool (LoadOperationNewOrderPlanned)

PegPool (SupplyDemand)

PegScheduledReceipt (WhereConsumed)

PegScheduledReceipt (WhereConsumedForDemand)

PegSource (DependentDemand)

PegSR (Demand)

PegSR (DependentDemand)

PegSupplyDemand (WhereConsumed)

PegSupplyDemand (WhereConsumedForDemand)

PenaltiesByDate (PenaltySchedule)

PenaltiesByInterval (PenaltySchedule)

PenaltyAccumulationRule (ProjectType)

PenaltyAccumulationRule (TaskType)

PenaltyCalendar (ProjectType)

PenaltyCalendar (TaskType)

PenaltyCost (Project)

PenaltyCost (Task)

PenaltyDate (Project)

PenaltyDate (Task)

PenaltyRule (ProjectType)

PenaltyRule (TaskType)

PenaltySchedule (Project)

PenaltySchedule (Task)

PenaltySchedules (Site)

Percent (EventPhase)

PercentCompleteRule (ProjectType)

PercentSafetyInterval (PlanningCalendars)

PercentSafetyInterval (SafetyStockPolicy)

PercentSafetyIntervalCount (Part)

PercentSafetyPercent (Part)

PerIntervalBonus (BonusScheduleByDate table)

PerIntervalBonus (BonusScheduleByInterval table)

PerIntervalCost (PenaltyScheduleByDate)

PerIntervalCost (PenaltyScheduleByInterval)

Period (InventoryAnalysis)

Period (InventoryCTPAnalysis)

Period (PeriodConstraintLoad)

PeriodCalendar (SafetyStockItemType)

PeriodCalendarSafetyStockItemTypes (Calendar)

PeriodDemand (InventoryAnalysis)

PeriodDemand (InventoryCTPAnalysis)

PeriodEffective (OrderPriority)

PeriodLoads (Constraint)

PeriodPlannedDemand (InventoryAnalysis)

PeriodPlannedDemand (InventoryCTPAnalysis)

PeriodPlannedSupply (InventoryAnalysis)

PeriodPlannedSupply (InventoryCTPAnalysis)

PeriodPlannedSupplyExpiry (InventoryCTPAnalysis)

PeriodsOfCoverage (Part)

PeriodsOfCTPCoverage (Part)

PeriodsOfSupplyTargetCalendar (IMConfiguration)

PeriodsPerCycle (SafetyStockItemType)

PeriodSupply (InventoryAnalysis)

PeriodSupply (InventoryCTPAnalysis)

PeriodSupplyDateRule (DaysSupplyPolicy)

PeriodSupplyDueCalendar (DaysSupplyPolicy)

PeriodSupplyDueCalendar (PlanningCalendars)

PeriodSupplyExpiry (InventoryCTPAnalysis)

PeriodSupplyInterval (DaysSupplyPolicy)

PeriodSupplyInterval (PlanningCalendars)

PeriodSynchronizedDemand (InventoryCTPAnalysis)

PeriodSynchronizedPlannedDemand (InventoryCTPAnalysis)

Phone (CustomerDestination)

Phone (LogisticsLocation)

Phone (Source)

Phone (Supplier)

PickPackTime (Part)

PickupCalendar (DeliveryRoute)

PlannedAllocation (Forecast)

PlannedAllocation (ForecastConsumption)

PlannedAllocations (BillOfMaterial)

PlannedInProcess (InventoryAnalysis)

PlannedInProcess (InventoryCTPAnalysis)

PlannedOrder (Activity)

PlannedOrder (CTPPlannedOrder)

PlannedOrder (DependentDemand)

PlannedOrder (LoadDetailPlanned)

PlannedOrder (LoadOperationNewOrderPlanned)

PlannedOrder (PartSourcePlannedOrder)

PlannedOrder (PlannedTransferAllocation)

PlannedOrder (ValidatePlannedOrder)

PlannedOrders (Part)

PlannedOrders (PartSource)

PlannedOrderSupplyStatus (PartSourceType)

PlannedOrderSupplyType (PartSourceType)

PlannedOrderToSRSupplyStatus (PartSourceType)

PlannedOrderToSRSupplyType (PartSourceType)

PlannedReceiptDate (IndependentDemand)

PlannedShipDate (IndependentDemand)

PlannedTransferAllocation (Forecast)

PlannedTransferAllocation (ForecastConsumption)

PlannedTransferAllocations (PartSource)

PlannerCode (Constraint)

PlannerCode (Event)

PlannerCode (Part)

PlannerCodes (Site)

PlanningCalendars (Part)

PlanningCalendars (RangeOfCoverage)

PlanningCalendars (WorkCenter)

PlanningCalendarsAllocationCalendar (Calendar)

PlanningCalendarsCoByProductPlanningInterval (Calendar)

PlanningCalendarsDataDate (Calendar)

PlanningCalendarsExpiryCalendars (Calendar)

PlanningCalendarsForecast (Calendar)

PlanningCalendarsOrderPointIncreases (Calendar)

PlanningCalendarsPercentSafetyInterval (Calendar)

PlanningCalendarsPeriodSupplyDue (Calendar)

PlanningCalendarsPeriodSupplyInterval (Calendar)

PlanningCalendarsRunDate (Calendar)

PlanningCalendarsSecondCalendar (Calendar)

PlanningCalendarsSubstitutionToleranceCalendar (Calendar)

PlanningCalendarsTimeFence (Calendar)

PlanningCalendarsTimeUnits (Calendar)

PlanningCalendarsWeekly (Calendar)

PlanningHorizon (DRPConfiguration)

PlanningOnly (Calendar)

PlanningPriority (OrderPriority)

PlanningRule (DemandType)

PlanningRule (ProjectType)

PlanningTimeFence (PartSource)

Point1 (SpreadProfile)

Point10 (SpreadProfile)

Point11 (SpreadProfile)

Point12 (SpreadProfile)

Point13 (SpreadProfile)

Point2 (SpreadProfile)

Point3 (SpreadProfile)

Point4 (SpreadProfile)

Point5 (SpreadProfile)

Point6 (SpreadProfile)

Point7 (SpreadProfile)

Point8 (SpreadProfile)

Point9 (SpreadProfile)

Points (MEIOLeadTimeDistributionProfile)

PointsCurve (Task)

Pool (Activity)

Pool (AlternatePart)

Pool (AvailableToPromise)

Pool (BaselinePlannedOrder)

Pool (BlowThroughPlannedAllocation)

Pool (CoByProductSupply)

Pool (ConstraintLoad)

Pool (ConstraintSpreadLoad)

Pool (CostOfGoodsSold)

Pool (CTPActivity)

Pool (Demand)

Pool (DemandException)

Pool (DemandOrder)

Pool (DependentDemand)

Pool (Forecast)

Pool (InProcess)

Pool (InventoryAnalysis)

Pool (InventoryCTPAnalysis)

Pool (InventorySummary)

Pool (InventoryTransfer)

Pool (LateSupply)

Pool (MaximumInventoryResult)

Pool (OnHand)

Pool (Part_MUEPoolValidation)

Pool (PartCustomer)

Pool (PartSource)

Pool (PeriodsOfCoverage)

Pool (PeriodsOfCTPCoverage)

Pool (PlannedOrder)

Pool (ScheduledReceipt)

Pool (SubstituteSupply)

Pool (Supply)

Pool (SupplyAllocation)

Pool (SupplyDemand)

Pool (SupplyOrder)

Pool (TimePhasedMaximumInventory)

Pool (TimePhasedSafety)

Pool (TrialCoByProductSupply)

Pool (TrialPlannedOrder)

Pool (TrialSubstituteSupply)

Pool (ValidatePlannedOrder)

Pool (WhereConsumed)

Pool (WhereConsumedForDemand)

Pool (WhereConsumedForForecast)

Pool (WhereConsumedForSupply)

PoolFlowToCommon (MUEPoolNettingType)

PoolMapOverrides (Part)

PoolMaps (PartType)

PoolRule (AlternatePartType)

PoolRule (MUEPoolNettingType)

Port (Shipment)

Ports (LogisticsLocation)

Predecessor (Predecessor)

PredecessorList (Task)

Predecessors (Task)

PredictionIntervalLower (StatisticalForecast)

PredictionIntervalUpper (StatisticalForecast)

PrePlanLimit (PartSource)

PrePlanLimitDate (MLSActivity)

PrePlanRule (PartSourceType)

PreShipLeadTimeRule (PartSourceType)

PreShipLT (Source)

PreShipLT (TimePhasedSourceLeadTime)

PreviousQty (ScheduledReceipt)

PrevWorkCenter (LoadDetailCurrent)

PrevWorkCenter (LoadDetailPlanned)

PrevWorkCenter (LoadOperationNewOrderPlanned)

PrevWorkCenter (LoadOperationReceiptsCurrent)

PrevWorkCenter (LoadOperationReceiptsPlanned)

PrimaryCTPPlannedOrder (CTPCoByProductSupply)

PrimaryPart (AlternatePart)

PrimaryPart (CoByProductSupply)

PrimaryPart (CTPCoByProductSupply)

PrimaryPart (TrialCoByProductSupply)

PrimaryPartSource (Part)

PrimaryPlannedOrder (CoByProductSupply)

PrimaryScheduledReceipt (CoByProductSupply)

PrimaryScheduledReceipt (CTPCoByProductSupply)

PrimaryScheduledReceipt (TrialCoByProductSupply)

PrimarySubstitutionSequence (Part)

PrimarySupplyType (CoByProductSupply)

PrimarySupplyType (Part)

PrimarySupplyType (TrialCoByProductSupply)

PrimaryTrialPlannedOrder (TrialCoByProductSupply)

Priority (PartSource)

Priority (ProductionGroup)

Priority (ProductionWheel)

Priority (ScheduledReceipt)

Priority (TransferDestination)

PriorityRule (SourceRule)

PrioritySource (SupplyStatus)

ProbabilityDistribution (MEIOLeadTimeType)

Processed (IndependentDemandSolution)

Processed (ScheduledReceipt)

ProcessingRule (AutomatedLeadTimeType)

ProcessingRule (AutomatedYieldType)

ProcessingRule (AggregatePartCustomerType)

ProcessingRule (AllotmentOverrideType)

ProcessingRule (AlternatePartType)

ProcessingRule (AvailableRule)

ProcessingRule (CausalFactorCategory)

ProcessingRule (ConstraintType)

ProcessingRule (CRPUnitOfMeasure)

ProcessingRule (DDMRPRule)

ProcessingRule (DemandStatus)

ProcessingRule (DemandType)

ProcessingRule (DistributionPlanningRule)

ProcessingRule (ExpiryType)

ProcessingRule (ForecastItemParametersOutlier)

ProcessingRule (HistoricalDemandCategoryType)

ProcessingRule (IncrementalRule)

ProcessingRule (InventoryTransferType)

ProcessingRule (MultiEchelonSafetyStockRule)

ProcessingRule (MultiLevelSearchRule)

ProcessingRule (OnHandType)

ProcessingRule (OperationSequenceType)

ProcessingRule (PartType)

ProcessingRule (RegressionModelTerm)

ProcessingRule (SafetyStockItem)

ProcessingRule (SafetyStockItemOutlier)

ProcessingRule (SafetyStockItemOutlierResult)

ProcessingRule (SafetyStockItemOutlierResultSummary)

ProcessingRule (SubstituteGroupType)

ProcessingRule (SupplyAllocationType)

ProcessingRule (SupplyType)

ProcessingRule (WorkCenterType)

ProcessingTime (LogisticsLocation)

ProcessInstance (Activity)

ProcessInstance (Notification)

ProcessInstanceOnly (Notification)

ProcessInstances (ProcessInstanceType)

ProductFamily (Part)

ProductGroup1 (Part)

ProductGroup2 (Part)

ProductionCycleNumber (ConstraintUsed)

ProductionGroup (PartSource)

ProductionGroups (Site)

ProductionRecurrence (PartSource)

ProductionWheel (ConstraintUsed)

ProductionWheel (PartSource)

ProductionWheel (ProductionWheelConstraint)

ProductionWheel (ProductionCycle)

ProductionWheel (PartSequence)

ProductionWheels (ProductionWheelType)

ProductionWheels (Site)

ProductionWheelConstraints (ProductionWheel)

ProductType (BOMType)

Profile (BestFitProfileDetail)

Profile (MEIOLeadTimeDistributionProfilePoint)

Profile (ProfilePoint)

ProfilePoints (PointsProfile)

ProfileStartDate (ForecastItemParameters)

Project (Conflict)

Project (CrticalPath)

Project (ResourceGroupProjectOverride)

Project (ResourceLoad)

Project (ResourceProjectOverride)

Project (Task)

ProjectedYields (ItemSource)

ProjectRunDate (Calendar)

Projects (BonusSchedule)

Projects (Customer)

Projects (PenaltySchedule)

Projects (ProjectGroup)

Projects (ProjectManager)

ProjectStatus table (ProjectStatus)

Projects (ProjectType)

Projects (Site)

ProjectedLeadTimes (ItemSource)

ProjectTypeCalendars (Calendar)

ProjectTypePenaltyCalendars (Calendar)

ProjectTypes (ControlSet)

PromisedDate (Demand)

PromisedDate (IndependentDemand)

Property (ClusterProperty)

ProtectQuantity (ForecastDetail)

ProtectQuantity (IndependentDemand)

ProtectQuantity (ScheduledReceipt)

PTFDate (PartSource)

PTFLastDueRule (SupplyAllocationType)

PTFLastDueRule (SupplyStatus)

PTFLastDueRule (SupplyType)

PTFRule (OrderPolicy)

PTFUnit (OrderPolicy)

PurchaseGroup (SupplyOrder)

PurchaseOrganization (SupplyOrder)

PushDate (SupplyDemand)

PushTransferID (CTPActivity)

pValue (ForecastItemParametersRegressionCoefficient)

pValue (StatisticalForecastRegressionCoefficient)

Q

QtyRemaining (LoadDetailCurrent)

QtyRemaining (LoadDetailPlanned)

QtyThisDate (LoadDetailCurrent)

QtyThisDate (LoadDetailPlanned)

QtyToDate (LoadDetailCurrent)

QtyToDate (LoadDetailPlanned)

Quantity (AllotmentOverrideDetail)

Quantity (AvailableToPromise)

Quantity (BaselinePlannedOrder)

Quantity (BaselineTaskLink)

Quantity (BlowThroughNewAllocation)

Quantity (BlowThroughPlannedAllocation)

Quantity (CausalFactorDetail)

Quantity (CoByProductSupply)

Quantity (ConsensusForecast)

Quantity (ConstraintMetricValues)

Quantity (CurvePoint)

Quantity (DemandException)

Quantity (DependentConsensusForecast)

Quantity (DisaggregationRateByPartCustomer)

Quantity (EventDisaggregationRate)

Quantity (EventPhase)

Quantity (EventStatisticalDisaggregationRate)

Quantity (EventStatisticalForecastDetail)

Quantity (FeatureBOMForIndependentDemand)

Quantity (Forecast)

Quantity (ForecastDetail)

Quantity (ForecastDisaggregationOverride)

Quantity (ForecastItemParametersActual)

Quantity (ForecastItemParametersOutlier)

Quantity (ForecastItemParametersRegressionWeight)

Quantity (ForecastItemParametersRegressor)

Quantity (HistoricalDemandActual)

Quantity (HistoricalDemandSeriesDetail)

Quantity (HistoricalDemandWaterfall)

Quantity (HistoricalPartKPI)

Quantity (HistoricalSupplyActual)

Quantity (HistoricalSupplySeriesDetail)

Quantity (HistoricalSupplyWaterfall)

Quantity (ImportedStatisticalForecast)

Quantity (IndependentDemand)

Quantity (InProcess)

Quantity (InventoryTransfer)

Quantity (LateSupply)

Quantity (MaximumInventoryResult)

Quantity (MEIOHistoricalSupply)

Quantity (MEIOStageHistoricalDemand)

Quantity (MEIOStageHistoricalForecast)

Quantity (NewAllocation)

Quantity (NewTransferAllocation)

Quantity (OnHand)

Quantity (PartMetricValues)

Quantity (PlannedAllocation)

Quantity (PlannedOrder)

Quantity (PlannedTransferAllocation)

Quantity (ProfilePoint)

Quantity (RegressorValue)

Quantity (SafetyStockHistoricalDemand)

Quantity (SafetyStockHistoricalForecast)

Quantity (SafetyStockHistoricalSupply)

Quantity (SafetyStockItemFutureDemand)

Quantity (SafetyStockItemOutlierResult)

Quantity (SafetyStockOverride)

Quantity (ScheduledReceipt)

Quantity (SiteMetricValues)

Quantity (StatisticalForecast)

Quantity (StatisticalForecastDetail)

Quantity (StatisticalForecastDisaggregationRate)

Quantity (StatisticalForecastOutlier)

Quantity (StatisticalForecastPredictActual)

Quantity (StatisticalForecastResidualAutocorrelation)

Quantity (SubstituteSupply)

Quantity (SupplierMetricValues)

Quantity (Supply)

Quantity (SupplyAllocation)

Quantity (TaskPartLink)

Quantity (TimePhasedMaximumInventory)

Quantity (TimePhasedSafety)

Quantity (TrialCoByProductSupply)

Quantity (TrialPlannedOrder)

Quantity (TrialSubstituteSupply)

QuantityAdjustment (EventForecastDetailAdjustment)

QuantityDelta (HistoricalDemandWaterfall)

QuantityDelta (HistoricalSupplyWaterfall)

QuantityPer (Allocation)

QuantityPer (BillOfMaterial)

QuantityPer (MEIOStageLink)

QuantityUnitOfMeasure (InProcess)

QuantityUnitOfMeasure (LateSupply)

QuantityUnitOfMeasure (Supply)

QuantityWithoutOverride (DisaggregationRateByPartCustomer)

QueueDate (LoadOperationNewOrderPlanned)

QueueDate (LoadOperationReceiptsCurrent)

QueueDate (LoadOperationReceiptsPlanned)

QueueOffset (LoadOperationNewOrderPlanned)

QueueOffset (LoadOperationReceiptsCurrent)

QueueOffset (LoadOperationReceiptsPlanned)

QueueTime (Capacity)

QueueTimeOverride (CRPOperation)

QueueTimeOverride (Operation)

R

Ramp (Task)

RangeOfCoverage (PlanningCalendars)

RangeOfCoverage (SafetyStockPolicy)

RangeOfCoverageAverageRequirementIntervals (Calendar)

RangeOfCoverageBuffer (Part)

RangeOfCoverageInterval (RangeOfCoverage)

RangeOfCoverageIntervals (Calendar)

RangeOfCoveragePartOverrides (Part)

RangeOfCoverages (ControlSet)

Rate (ConstraintAvailable)

Rate (CurrencyConversionActual)

Rate (CurrencyConversionForecast)

Rate (HistoricalCurrencyConversion)

Rate (ResourceAssignment)

Rate (Task)

RateAdjustment (Project)

RateAdjustment (ResourceAssignment)

RateAdjustment (ResourceGroupProjectOverride)

RateAdjustment (ResourceGroupTaskOverride)

RateAdjustment (ResourceProjectOverride)

RateAdjustment (Task)

RatioRule (BOMType)

Real (MEIOStage)

Reason (EngineeringChange)

RebalancingAdjustmentQuantity (ConsensusForecast)

RebalancingOverrideQuantity (ConsensusForecast)

ReceiptQuantity (HistoricalSupplyActual)

ReceivedQuantity (HistoricalReceipt)

ReceivedQuantity (TaskDemandLink)

ReceivedQuantity (TaskSupplyLink)

ReceivedQuantity (TaskPartLink)

RecommendedBinQty (ShopKanban)

RecommendedBinQty (SourceKanban)

RecommendedBins (ShopKanban)

RecommendedBins (SourceKanban)

RecommendedDate (Activity)

RecommendedDate (CTPActivity)

RecommendedDate (ScheduledReceipt)

RecommendedDate (Supply)

RecommendedDockDate (ScheduledReceipt)

Recurrence (ProductionRecurrence)

Recurrence (Segment)

ReferenceItem (ForecastItemParametersMap)

ReferencePart (Part)

Region (Customer)

Region (RegionGroup)

Region (Site)

Region (Supplier)

RegionGroup (Region)

RegressionCoefficients (ForecastItemParameters)

RegressionModel (ForecastItemParameters)

RegressionModelTerm (ForecastItemParametersRegressor)

RegressionModelTerms (Regressor)

Regressor (RegressionModelTerm)

Regressor (RegressorValue)

Regressors (RegressorCategory)

RegressorValues (Regressor)

RegularizationParameter (BestFitProfileDetail)

RegularizationParameter (ForecastItemParameters)

RemainingDuration (Task)

RemainingQuantity (Allocation)

RemainingQuantity (Demand)

RemainingQuantity (DependentDemand)

RemainingWork (ResourceAssignment)

RemovalTime (OriginatingFile)

RemoveDemandOutliers (SafetyStockItem)

ReplenishmentLeadTime (ShopKanban)

ReplenishmentPeriod (ShopKanban)

ReplenishmentPeriod (SourceKanban)

ReportCalendar (SafetyStockResult)

ReportLimit (ConstraintType)

ReportLimit (ResourceType)

RequestDate (HistoricalDemandActual)

RequestDate (IndependentDemand)

RequestedStatus (OFConfiguration)

RequestQty (CollaborativeForecast)

ReschedDate (Activity)

ReschedDate (Allocation)

ReschedDate (CTPActivity)

ReschedDate (Demand)

ReschedDate (DependentDemand)

ReschedDate (InProcess)

ReschedDate (ScheduledReceipt)

ReschedDate (Supply)

ReschedDockDate (ScheduledReceipt)

RescheduleRule (SupplyStatus)

ResidualAutocorrelationConfidenceLevel (ForecastItemParameters)

Resource (ResourceAssignment)

Resource (ResourceAvailable)

Resource (ResourceCalendarException)

Resource (ResourceCosts)

Resource (ResourceDataByDate)

Resource (ResourceLoad)

Resource (ResourceProjectOverride)

ResourceActualFixedCost (Project)

ResourceActualFixedCost (Task)

ResourceActualOvertimeCost (Project)

ResourceActualOvertimeCost (Task)

ResourceActualStandardCost (Project)

ResourceActualStandardCost (Task)

ResourceAssignment (ResourceAssignmentAvailable)

ResourceAssignment (ResourceCostActual)

ResourceFixedCost (Project)

ResourceFixedCost (Task)

ResourceGroup (ResourceGroupProjectOverride)

ResourceGroup (ResourceGroupTaskOverride)

ResourceOvertimeCost (Project)

ResourceOvertimeCost (Task)
Resources (ResourceGroup)
Resources (ResourceType)
ResourceStandardCost (Project)
ResourceStandardCost (Task)
ResourceTypes (Calendar)
ResourceTypes (ControlSet)
ResponseProvided (CollaborativeForecast)
ResponseRequired (CollaborativeForecast)
ReturnOperation (OperationSequence)
Revenue (Task)
Reviewed (ForecastItemParameters)
ReviewedDateTime (ScheduledReceipt)
RevisedLeadTime (ProjectedLeadTime)
RevisedYield (ProjectedLeadTime)
RollingErrorIntervalCount (ForecastItemParameters)
RollingLeadTimeDemandLastDate (SafetyStockItem)
RollingLeadTimeQuantity (SafetyStockHistoricalDemand)
RollingWeight (HistoricalDemandCategory)
RoundingPrecision (UnitOfMeasure)
RoundingRule (BOMType)
Route (HistoricalDemandActual)
Routing (AlternateRouting)
Routing (HistoricalReceiptHeader)
Routing (ItemSource)
Routing (LoadOperationNewOrderPlanned)
Routing (LoadOperationReceiptsCurrent)
Routing (LoadOperationReceiptsPlanned)
Routing (Operation)
Routing (OperationSequence)
Routing (PartSource)
Routing (ScheduledReceipt)
Routings (Site)
RParameters (RParameterSet)
RParameterSet (ForecastItemParameters)
RServerAvailable (SOPAnalyticsConfiguration)
Rule (BaselineTaskLink)
Rule (Predecessor)
Rule (TaskDemandLink)
Rule (TaskSupplyLink)
RunTimeUOM (CRPOperation)
RunTimeUOM (Operation)
Run (LoadFLPPlanned)
Run (LoadOperationNewOrderPlanned)
Run (LoadOperationReceiptsCurrent)
Run (LoadOperationReceiptsPlanned)
RunDate (IMConfiguration)
RunDate (LoadOperationNewOrderPlanned)
RunDate (LoadOperationReceiptsCurrent)
RunDate (LoadOperationReceiptsPlanned)
RunDate (MPSConfiguration)
RunDate (PlanningCalendars)
RunDate (ProcessInstanceType)
RunDate (Project)
RunDate (SOPAnalyticsConfiguration)
RunDateTimeOfDay (Project)
RunOffset (LoadOperationNewOrderPlanned)
RunOffset (LoadOperationReceiptsCurrent)
RunOffset (LoadOperationReceiptsPlanned)
RunTime (CRPOperation)
RunTime (MEIOFamilyResult)
RunTime (Operation)

S

SafetyDaysCalendar (SafetyStockPolicy)
SafetyDaysDependentDemandUsage (SafetyStockPolicy)
SafetyFactor (ShopKanban)

SafetyFactor (SourceKanban)

SafetyLeadTime (Part)

SafetyLeadTime (PartSource)

SafetyLeadTime (SafetyLeadTimeProfileDetail)

SafetyLeadTimeCalendarRule (PartType)

SafetyLeadTimeProfile (Part)

SafetyLeadTimeProfile (SafetyLeadTimeProfileDetail)

SafetyLeadTimeProfiles (Site)

SafetyLeadTimeProfileDetails (SafetyLeadTimeProfile)

SafetyLeadTimeRule (DemandType)

SafetyStock (MEIOStageTimePhasedResult)

SafetyStock (SafetyStockResult)

SafetyStock (SafetyStockTimePhasedResult)

SafetyStockBounds (Part)

SafetyStockBoundsViolation (MEIOFamilyResult)

SafetyStockDateRule (PartType)

SafetyStockDateRule (SafetyStockPolicy)

SafetyStockDemandUsage (DemandType)

SafetyStockHistoricalForecasts (SafetyStockItem)

SafetyStockItem (MEIOStage)

SafetyStockItem (SafetyStockItemOutlier)

SafetyStockItem (SafetyStockPart)

SafetyStockItem (SafetyStockResult)

SafetyStockItem (TimePhasedDemandParameterSet)

SafetyStockItemCreate (PartSolution)

SafetyStockItemMappings (HistoricalDemandCategory)

SafetyStockItemMappings (Part)

SafetyStockItems (HistoricalDemandCategory)

SafetyStockItems (Part)

SafetyStockItemTypes (Calendar)

SafetyStockOverrides (SafetyStockPart)

SafetyStockParts (Part)

SafetyStockPolicies (Calendar)

SafetyStockPolicies (RangeOfCoverage)

SafetyStockPolicy (Part)

SafetyStockQty (Part)

SafetyStockQuantityRule (PartType)

SafetyStockQuantityRule (SafetyStockPolicy)

SafetyStockResult (SafetyStockTimePhasedResult)

SafetyStockResults (Part)

SafetyStockRule (MUEPoolNettingType)

SafetyStockRule (PartType)

SafetyStockRule (SafetyStockPolicy)

SafetyStockTimePhasedResults (Part)

SalesActualConsumptionOrder (PartType)

SavedPriority (IndependentDemand)

SavedPriority (ScheduledReceipt)

ScaleFactor (CRPUnitOfMeasure)

ScalingFactor (ForecastItemParameters)

ScenarioPath (DataChange_Historical)

ScenarioPath (DataChange_Local)

ScenarioPath (DataChange_PendingUpdate)

ScenarioPath (DataChange)

Schedule (BonusScheduleByDate table)

Schedule (BonusScheduleByInterval table)

Schedule (PenaltyScheduleByDate)

Schedule (PenaltyScheduleByInterval)

ScheduledReceipt (Activity)

ScheduledReceipt (Allocation)

ScheduledReceipt (BlowThroughNewAllocation)

ScheduledReceipt (ConstraintLoad)

ScheduledReceipt (ConstraintSpreadLoad)

ScheduledReceipt (CostOfGoodsSold)

ScheduledReceipt (CRPOperationState)

ScheduledReceipt (CTPActivity)

ScheduledReceipt (InProcess)

ScheduledReceipt (LateSupply)

ScheduledReceipt (LoadDetailCurrent)

ScheduledReceipt (LoadFLPPlanned)

ScheduledReceipt (LoadOperationReceiptsCurrent)

ScheduledReceipt (LoadOperationReceiptsPlanned)

ScheduledReceipt (MLSActivity)

ScheduledReceipt (NewAllocation)

ScheduledReceipt (NewTransferAllocation)

ScheduledReceipt (PartSourceScheduledReceipt)

ScheduledReceipt (ScheduledReceipt_Notes)

ScheduledReceipt (ScheduledReceiptCost)

ScheduledReceipt (Supply)

ScheduledReceipt (SupplyDemand)

ScheduledReceipt (TaskSupplyLink)

ScheduledReceipt (TrialConsumption)

ScheduledReceipt (WhereConsumed)

ScheduledReceipt (WhereConsumedForDemand)

ScheduledReceipt (WhereConsumedForForecast)

ScheduledReceipt (WhereConsumedForSupply)

ScheduledReceipts (AllotmentOverride)

ScheduledReceipts (BOMAlternate table)

ScheduledReceipts (Model)

ScheduledReceipts (Part)

ScheduledReceipts (PartSource)

ScheduledReceipts (Routing)

ScheduledReceipts (Shipment)

ScheduledReceiptsUnderOrderPriority (OrderPriority)

ScheduledReceiptsUnderSavedPriority (OrderPriority)

ScheduledSupplyRule (MultiLevelSearchRule)

SchedulePriority (OrderPriority)

SchedulePriority (ScheduledReceipt)

ScheduleRule (ProjectType)

SchedulingRule (ConstraintType)

SchedulingRule (Task)

Scrap (BillOfMaterial)

ScrapRule (BOMType)

SeasonalComponent (SafetyStockItemOutlierResult)

SeasonalComponent (StatisticalForecastOutlier)

SeasonalIntervalCount (BestFitProfileDetail)

SeasonalIntervalCount (ForecastItemParameters)

SecondaryPriority (PartSource)

SecondCalendar (PlanningCalendars)

SecondZoneIntervalCount (RangeOfCoverage)

SecondZoneTarget (RangeOfCoverage)

SegmentQualifier (ScheduledReceipt)

Segments (ProductionRecurrence)

Segments (Site)

SelfHealingParameter (ItemSourceFitOutput)

SelfHealingParameter (Cluster)

SelfOutlierQuantity (ForecastItemParametersActual)

SelfQuantity (ForecastItemParametersActual)

Sequence (AllotmentOverride)

Sequence (AlternatePart)

Sequence (CRPOperation)

Sequence (DataChange_Historical)

Sequence (DataChange_Local)

Sequence (DataChange_PendingUpdate)

Sequence (DataChange)

Sequence (FeatureMap)

Sequence (HistoricalDemandSeries)

Sequence (HistoricalSupplySeries)

Sequence (PartSequence)

SequenceId (ConsensusForecastRollingHorizon)

Sequences (Routing)

Series (HistoricalDemandSeriesDetail)

Series (HistoricalDemandWaterfall)

Series (HistoricalSupplySeriesDetail)

Series (HistoricalSupplyWaterfall)

ServiceLevel (MEIOStageResult)

ServiceLevel (MEIOStageTimePhasedResult)

ServiceLevel (SafetyStockItem)

ServiceLevel (SafetyStockOverride)

ServiceLevel (SafetyStockPart)

ServiceLevel (SafetyStockResult)

ServiceLevel (SafetyStockTimePhasedResult)

ServiceLevelRule (SafetyStockItemType)

ServiceTime (SafetyStockItem)

Set (RParameter)

Setup (LoadDailyPlanned)

Setup (LoadFLPPlanned)

Setup (LoadOperationNewOrderPlanned)

Setup (LoadOperationReceiptsCurrent)

Setup (LoadOperationReceiptsPlanned)

Setup (LoadWeeklyPlanned)

Setup (OverrideMajorChangeover)

SetupDate (LoadOperationNewOrderPlanned)

SetupDate (LoadOperationReceiptsCurrent)

SetupDate (LoadOperationReceiptsPlanned)

SetupHours (CRPOperation)

SetupHours (Operation)

SetupOffset (LoadOperationNewOrderPlanned)

SetupOffset (LoadOperationReceiptsCurrent)

SetupOffset (LoadOperationReceiptsPlanned)

ShipCalendar (Source)

ShipCalendarSources (Calendar)

ShipDate (Activity)

ShipDate (CTPActivity)

ShipDate (InventoryTransfer)

ShipDate (PlannedOrder)

ShipDate (ScheduledReceipt)

ShipDate (Supply)

ShipDate (TrialPlannedOrder)

ShipDateFromStart (PlannedOrder)

ShipDateFromStart (ScheduledReceipt)

ShipDateTime (Shipment)

ShipFrom (DeliveryRoute)

Shipment (HistoricalSupplyActual)

Shipment (ScheduledReceipt)

ShipmentPolicies (ControlSet)

ShipmentPolicy (IndependentDemand)

Shipments (Carrier)

Shipments (Supplier)

ShippedQty (IndependentDemand)

ShippingCost (HistoricalSupplyActual)

ShippingCost (PartSource)

ShipSet (HistoricalDemandActual)

ShipSet (IndependentDemand)

ShipSetAvailableDate (IndependentDemand)

ShipSetHasHoldCode (IndependentDemandSolution)

ShipSets (ShipSetType)

ShipTo (ScheduledReceipt)

Shop (Allocation)

Shop (BillOfMaterial)

Shop (ShopKanban)

ShopKanbans (Part)

Shops (Site)

ShortQuantity (TrialConstraintShortage)

ShortTermExcess (MultiLevelSearchRule)

SiblingIndex (Activity)

SiblingIndex (Task)

Site (ABCCode)

Site (BonusSchedule table)

Site (BuyerCode)

Site (Constraint)

Site (Customer)

Site (DataChange_Historical)

Site (DataChange_Local)

Site (DataChange_PendingUpdate)

Site (DataChange)

Site (DemandOrder)

Site (Department)

Site (EngineeringChange)

Site (HistoricalDemandOrder)

Site (HistoricalShipment)

Site (HistoricalSupplyOrder)

Site (Part)

Site (PenaltySchedule table)

Site (PlannerCode)

Site (ProductionGroup)

Site (ProductionWheel)

Site (Project)

Site (Resource)

Site (Routing)

Site (SafetyLeadTimeProfile)

Site (SAP_MD04)

Site (Segment)

Site (Shop)

Site (SiteManager)

Site (SiteMetric)

Site (SubstituteGroup)

Site (Supplier)

Site (SupplyOrder)

Site (Warehouse)

Site (WorkCenter)

SiteManagers (Site)

SiteManagerTypes (SiteManagerType)

SiteMetric (SiteMetricValues)

SiteMetricValues (SiteMetric)

SiteMetrics (Metric)

SiteMetrics (MetricAsOf)

Sites (ControlSet)

Sites (Currency)

Sites (Region)

SlowMovingTransition (IMConfiguration)

SmoothingIntervalsAfter (IPOConfiguration)

SmoothingIntervalsBefore (IPOConfiguration)

SOPActualsCategory (HistoricalDemandCategory)

SOPBaseCalendars (Calendar)

SOPCycleCalendars (Calendar)

SOPForecastDisaggregationOverrideCategories (HistoricalDemandCategory)

SOPInnerCalendars (Calendar)

SOPOuterCalendars (Calendar)

SOPRunDateCalendars (Calendar)

SortBeforeDate (SupplyType)

SortBeforeStart (SupplyStatus)

SortKey (Activity)

SortKey (Segment)

SortKey (Task)

SortSameDate (SupplyType)

SortSameStart (SupplyStatus)

Source (BaselineConflict)

Source (CollaborativeForecast)

Source (Conflict)

Source (ConsensusForecastRollingHorizonWeight)

Source (CriticalPath)

Source (ForecastConsumption)

Source (HistoricalReceiptHeader)

Source (HistoricalSupplyActual)

Source (ItemSource)

Source (ItemSourceCandidate)

Source (InventoryTransfer)

Source (LoadDetailCurrent)
Source (LoadDetailPlanned)
Source (MaximumInventoryResult)
Source (OriginatingFile)
Source (PartSource)
Source (SequenceFlow)
Source (StatisticalForecastRegressionCoefficient)
Source (SupplyType)
Source (TimePhasedSourceLeadTime)
Source (TransferDestination)
Source (WorkCenterCapacity)
SourceConstraint (ConstraintLoad)
SourceConstraint (ConstraintSpreadLoad)
SourceConstraint (TrialConsumption)
SourceConstraints (Constraint)
SourceEffectivityRule (MultiLevelSearchRule)
SourceKanban (PartSource)
SourceKanban (PlannedOrder)
SourceKanbans (PartSource)
SourceLeadTimeUnits (Calendar)
SourcePart (SafetyStockItemMapping)
SourceRecordHandle (DataChange_Historical)
SourceRecordHandle (DataChange_Local)
SourceRecordHandle (DataChange_PendingUpdate)
SourceRecordHandle (DataChange)
SourceRule (Part)
SourceRules (ControlSet)
Sources (Carrier)
Sources (Site)
SplitGroupSequence (DataChange_Historical_Split)
SplitLateSupply (SupplyStatus)
SplitQuantity (SupplyDemand)
SplitQuantityUnitOfMeasure (SupplyDemand)
SplitSupplyAvailable (CostOfGoodsSold)
SplitSupplyAvailable (LateSupply)
SplitSupplyAvailable (WhereConsumed)
SplitSupplyAvailable (WhereConsumedForDemand)
SplitSupplyAvailable (WhereConsumedForForecast)
SplitSupplyAvailable (WhereConsumedForSupply)
SpreadForecastInterval (Part)
SpreadProfile (DemandType)
SpreadProfiles (ControlSet)
SpreadRule (DemandType)
SRIntransitStatus (DRPConfiguration)
SRLaborIdle (LoadDailyPlanned)
SRLaborRun (LoadDailyPlanned)
SRLaborSetup (LoadDailyPlanned)
SRMachineIdle (LoadDailyPlanned)
SRMachineRun (LoadDailyPlanned)
SRMachineSetup (LoadDailyPlanned)
SRQuantity (AvailableToPromise)
Stage (MEIOStageHistoricalDemand)
Stage (MEIOStageHistoricalForecast)
Stage (MEIOStageLink)
Stage (MEIOStageResult)
Stage (MEIOStageTimePhasedResult)
StandardCost (ResourceCostActual)
StandardCost (ResourceLoad)
StandardDaysPerInterval (RangeOfCoverage)
StandardDeviation (MEIOLeadTime)
StandardDeviation (SafetyStockItemDemandParameterSet)
StandardDeviation (TimePhasedDemandParameterSet)
StandardDeviationDemand (MEIOStageResult)
StandardDeviationDemand (MEIOStageTimePhasedResult)
StandardDeviationDemand (SafetyStockItem)

StandardDeviationDemand (SafetyStockOverride)

StandardDeviationDemand (SafetyStockPart)

StandardDeviationDemand (SafetyStockResult)

StandardDeviationDemand (SafetyStockTimePhasedResult)

StandardDeviationDemandRule (SafetyStockItemType)

StandardDeviationLeadTime (SafetyStockItem)

StandardDeviationLeadTime (SafetyStockPart)

StandardDeviationLeadTime (SafetyStockResult)

StandardDeviationRule (MEIOLeadTimeType)

StandardError (ForecastItemParametersRegressionCoefficient)

StandardError (StatisticalForecastRegressionCoefficient)

StandardHours (ResourceCostActual)

StandardHours (ResourceLoad)

StandardRate (Resource)

StandardRate (ResourceCosts)

StandardRate (ResourceDataByDate)

StartDate (BaselinePlannedOrder)

StartDate (CoByProductSupply)

StartDate (EventPhase)

StartDate (ForecastItemParameters)

StartDate (LoadOperationNewOrderPlanned)

StartDate (LoadOperationReceiptsCurrent)

StartDate (LoadOperationReceiptsPlanned)

StartDate (PlannedOrder)

StartDate (ProcessInstance)

StartDate (Project)

StartDate (ScheduledReceipt)

StartDate (Supply)

StartDate (Task)

StartDate (TrialCoByProductSupply)

StartDate (TrialPlannedOrder)

StartDate (ValidatePlannedOrder)

StartDay (Activity)

StartingFixedCost (ResourceAssignment)

StartingUnadjustedOvertimeRate (ResourceAssignment)

StartingUnadjustedStandardRate (ResourceAssignment)

StartOffset (LoadOperationNewOrderPlanned)

StartOffset (LoadOperationReceiptsCurrent)

StartOffset (LoadOperationReceiptsPlanned)

StartOffset (MPSConfiguration)

StartQuantity (HistoricalReceipt)

StartUnit (Activity)

StartUnit (Allocation)

StartUnit (AvailableToPromise)

StartUnit (BaselinePlannedOrder)

StartUnit (BillOfMaterial)

StartUnit (BlowThroughNewAllocation)

StartUnit (BlowThroughPlannedAllocation)

StartUnit (BOM_MUECUMLead)

StartUnit (CoByProductSupply)

StartUnit (CRPOperation)

StartUnit (CTPActivity)

StartUnit (Demand)

StartUnit (DependentDemand)

StartUnit (Forecast)

StartUnit (IndependentDemand)

StartUnit (InProcess)

StartUnit (InventoryAnalysis)

StartUnit (InventoryCTPAnalysis)

StartUnit (InventorySummary)

StartUnit (InventoryTransfer)

StartUnit (LateSupply)

StartUnit (LoadDetailCurrent)

StartUnit (LoadDetailPlanned)

StartUnit (LoadOperationNewOrderPlanned)

StartUnit (LoadOperationReceiptsCurrent)

StartUnit (LoadOperationReceiptsPlanned)

StartUnit (MLSActivity)

StartUnit (NewAllocation)

StartUnit (OnHand)

StartUnit (Operation)

StartUnit (Part_MUECumLead)

StartUnit (Part_MUEPoolValidation)

StartUnit (PeriodsOfCoverage)

StartUnit (PeriodsOfCTPCoverage)

StartUnit (PlannedAllocation)

StartUnit (PlannedOrder)

StartUnit (ScheduledReceipt)

StartUnit (SubstituteSupply)

StartUnit (Supply)

StartUnit (SupplyAllocation)

StartUnit (SupplyDemand)

StartUnit (TrialAvailableDate)

StartUnit (TrialCoByProductSupply)

StartUnit (TrialPlannedOrder)

StartUnit (TrialSubstituteSupply)

StartUnit (ValidatePlannedOrder)

State (CollaborativeForecast)

State (SupplyStatus)

StatisticalDate (HistoricalSupplyActual)

StatisticalForecastDetails (ForecastItemParameters)

StatisticalForecastFitOutputs (ForecastItemParameters)

StatisticalForecastOverrides (PointsProfile)

StatisticalForecastPredictActuals (ForecastItemParameters)

StatisticalForecasts (ForecastItemParameters)

Status (Activity)

Status (Assumption)

Status (IndependentDemand)

Status (InventoryTransferType)

Status (Job)

Status (ProcessInstance)

Status (Project)

Status (ScheduledReceipt)

StdLaborRunCost (WorkCenter)

StdLaborSetupCost (WorkCenter)

StdMachineRunCost (WorkCenter)

StdMachineSetupCost (WorkCenter)

StdUnitCost (Part)

STLKN_ItemNodeNumber (SAP_MD04)

STLNR_BillOfMaterial (SAP_MD04)

STLTY_BOMCategory (SAP_MD04)

STName (TimeZone)

StockDate (InventoryTransfer)

StockUnitQuantity (Activity)

StockUnitQuantity (CTPActivity)

StockUnitQuantity (InProcess)

StockUnitQuantity (PlannedOrder)

StockUnitQuantity (ScheduledReceipt)

StockUnitQuantity (Supply)

STOffset (TimeZone)

StopDate (ForecastItemParameters)

StorageCapacity (TimePhasedLocationCapacity)

StorageCapacityUOM (TimePhasedLocationCapacity)

StorageType (Location)

StorageType (PartSolution)

STPOZ_Counter (SAP_MD04)

Strategy (PartSolution)

Strategy (Segment)

STStartDN (TimeZone)

STStartDOW (TimeZone)

STStartMonth (TimeZone)

STStartTime (TimeZone)

SubstituteGroup (WhereConsumed)

SubstituteGroup (Allocation)

SubstituteGroup (BillOfMaterial)

SubstituteGroup (CostOfGoodsSold)

SubstituteGroup (SupplyDemand)

SubstituteGroup (WhereConsumedForDemand)

SubstituteGroup (WhereConsumedForForecast)

SubstituteGroup (WhereConsumedForSupply)

SubstituteGroups (SubstituteGroupType)

SubstituteGroupTypes (ControlSet)

SubstitutePart (AlternatePart)

SubstitutePool (SubstituteSupply)

SubstituteSupplies (Part)

SubstituteSupply (Activity)

SubstituteSupply (CTPSubstituteSupply)

SubstituteSupply (Demand)

SubstituteSupply (DependentDemand)

SubstituteSupply (Supply)

SubstitutionToDateQty (BillOfMaterial)

SubstitutionPreference (PartType)

SubstitutionSequence (BillOfMaterial)

SubstitutionTolerance (Part)

SubstitutionToleranceCalendar (PlanningCalendars)

SuccessorList (Task)

Successors (Task)

SuggestedQuantity (SafetyStockItemOutlierResult)

SuggestedQuantity (StatisticalForecastOutlier)

Supplier (Activity)

Supplier (CTPActivity)

Supplier (PartSupplier)

Supplier (Shipment)

Supplier (Source)

Supplier (SupplierMetric)

Supplier (Supply)

Supplier (SupplyOrder)

SupplierCommit (ScheduledReceipt)

SupplierGroup (Supplier)

SupplierMetric (SupplierMetricValues)

SupplierMetricValues (SupplierMetric)

SupplierPart (PartSource)

SupplierResponse (ScheduledReceipt)

Suppliers (Region)

Suppliers (Site)

Suppliers (SupplierGroup)

SupplierUOM (PartSource)

SupplierUOM (PartSupplier)

Supplies (Part)

Supply (Conflict)

SupplyAllocation (SupplyAllocation_Notes)

SupplyAllocationRule (PartType)

SupplyAllocations (Model)

SupplyAllocations (OrderPriority)

SupplyAllocations (Part)

SupplyAllocations (PartSource)

SupplyAllocations (SupplyAllocationType)

SupplyAllocationTypes (ControlSet)

SupplyAvailableDate (CostOfGoodsSold)

SupplyAvailableDate (SupplyDemand)

SupplyAvailableDate (WhereConsumed)

SupplyAvailableDate (WhereConsumedForDemand)

SupplyAvailableDate (WhereConsumedForForecast)

SupplyAvailableDate (WhereConsumedForSupply)

SupplyDate (SupplyAllocation)

SupplyDemand (CTPSafetyStockHoldings)

SupplyDemand (CostOfGoodsSold)

SupplyDemand (WhereConsumed)

SupplyDemand (WhereConsumedForDemand)

SupplyDemand (WhereConsumedForForecast)

SupplyDemand (WhereConsumedForSupply)

SupplyDemands (Part)

SupplyDueDate (CostOfGoodsSold)

SupplyDueDate (WhereConsumed)

SupplyDueDate (WhereConsumedForDemand)

SupplyDueDate (WhereConsumedForForecast)

SupplyDueDate (WhereConsumedForSupply)

SupplyEstimatedExpiryDate (SupplyDemand)

SupplyFraction (SupplyDemand)

SupplyIsPlanned (LateSupply)

SupplyIsPlanned (LoadFLPPlanned)

SupplyLinks (Task)

SupplyLinks (TaskPartLink)

SupplyNotes (HistoricalDemandActual)

SupplyNotes (IndependentDemandSolution)

SupplyOrder (Activity)

SupplyOrder (Mfg__SupplyOrder_TextLine)

SupplyOrders (Site)

SupplyPart (ConstraintLoad)

SupplyPart (ConstraintSpreadLoad)

SupplyPlannedOrder (LoadFLPPlanned)

SupplyPool (PoolMap)

SupplyPool (PoolMapOverride)

SupplyPreference (SubstituteGroupType)

"SupplyPushRule" on page 724
(DistributionPlanningRule)

SupplyQty (Activity)

SupplyQty (CTPActivity)

SupplyQtyUnitOfMeasure (Activity)

SupplyQuantity (CostOfGoodsSold)

SupplyQuantity (WhereConsumed)

SupplyQuantity (WhereConsumedForDemand)

SupplyQuantity (WhereConsumedForForecast)

SupplyQuantity (WhereConsumedForSupply)

SupplyQuantityUnitOfMeasure (CostOfGoodsSold)

SupplyQuantityUnitOfMeasure
(WhereConsumedForDemand)

SupplyQuantityUnitOfMeasure
(WhereConsumedForForecast)

SupplyQuantityUnitOfMeasure
(WhereConsumedForSupply)

SupplyReschedDate (LoadFLPPlanned)

SupplyReschedDate (SupplyDemand)

SupplySelection (AttributeMapType)

SupplyShareFence (Part)

SupplyShareRule (PartType)

SupplySource (CostOfGoodsSold)

SupplySource (InProcess)

SupplySource (LateSupply)

SupplySource (MLSActivity)

SupplySource (Supply)

SupplySource (SupplyDemand)

SupplySource (WhereConsumed)

SupplySource (WhereConsumedForDemand)

SupplySource (WhereConsumedForForecast)

SupplySource (WhereConsumedForSupply)

SupplyStartDate (LoadFLPPlanned)

SupplyStartDate (WhereConsumed)

SupplyStartDate (WhereConsumedForDemand)

SupplyStartDate (WhereConsumedForForecast)

SupplyStartDate (WhereConsumedForSupply)

SupplyStartUnit (CostOfGoodsSold)

SupplyStartUnit (WhereConsumed)

SupplyStartUnit (WhereConsumedForDemand)

SupplyStartUnit (WhereConsumedForForecast)

SupplyStartUnit (WhereConsumedForSupply)

SupplyStatuses (ControlSet)

SupplyType (Activity - Mfg)

SupplyType (CTPActivity)

SupplyType (LateSupply)

SupplyType (WhereConsumed)
SupplyTypes (ControlSet)
SupplyTypes (DemandType)
SupplyTypeString (WhereConsumed)
SupplyTypeString (WhereConsumedForDemand)
SupplyTypeString (WhereConsumedForForecast)
SupplyTypeString (WhereConsumedForSupply)
SupplyVariabilityRule (SafetyStockItemType)
Symbol (Currency)
SynchronizedDate (CTPActivity)
SynchronizedDate (WhereConsumed)
SynchronizedDate (WhereConsumedForDemand)
SynchronizedDate (WhereConsumedForForecast)
SynchronizedDate (WhereConsumedForSupply)

T

Table (DataChange_Historical)
Table (DataChange_Local)
Table (DataChange_PendingUpdate)
Table (DataChange)
TaktTime (PartSource)
Target (Assumption)
Target (BillOfMaterial)
Target (PartSource)
Target (SequenceFlow)
TargetAccum (SupplyStatus)
TargetAccumUsage (SupplyStatus)
TargetPercentComplete (Task)
TargetPercentWorkComplete (Task)
TargetWorkComplete (Task)
Task (BaselineConflict)
Task (BaselineTaskLink)
Task (Conflict)
Task (CrticalPath)
Task (Predecessor)
Task (ResourceAssignment)
Task (ResourceGroupTaskOverride)
Task (ResourceLoad)
Task (Task_Notes)
Task (TaskDemandLink)
Task (TaskHyperlink)
Task (TaskPartLink)
Task (TaskSupplyLink)
TaskCost (Project)
TaskDemandLink (TaskDemandLinkCost)
TaskDemandLinks (IndependentDemand)
TaskLink (BaselineConflict)
TaskNeedDate (IndependentDemand)
TaskNeedDate (ScheduledReceipt)
TaskPartLink (TaskPartLinkCost)
TaskLinks (Part)
TaskRevenue (Project)
Tasks (PenaltySchedule)
Tasks (PointsCurve)
Tasks (Project)
Tasks (TaskType)
TaskSupplyLink (TaskSupplyLinkCost)
TaskSupplyLinks (ScheduledReceipt)
TaskTypeCalendars (Calendar)
TaskTypePenaltyCalendars (Calendar)
TaskTypes (ControlSet)
TearDownTime (CRPOperation)
TemplateName (ProcessInstance)
Text (Assumption)
TextId (SupplyOrder)
TextLine (SupplyOrder)
ThirdZoneTarget (RangeOfCoverage)
ThresholdFactorLower (RangeOfCoverage)

ThresholdFactorLower (RangeOfCoveragePartOverride)

ThresholdFactorUpper (RangeOfCoverage)

ThresholdFactorUpper (RangeOfCoveragePartOverride)

TimeFenceCalendar (PlanningCalendars)

TimePhasedAttributes (PartCustomer)

TimePhasedAttributes (PartSource)

TimePhasedLeadTimes (PartSource)

TimePhasedLeadTimes (Source)

AutomatedLeadTimeType table (AutomatedLeadTimeType)

TimePhasedMinCOV (AutomatedYieldType)

TimePhasedProcessingRule (SafetyStockItemType)

TimePhasedReportCount (SafetyStockItem)

TimePhasedSafetys (OrderPriority)

TimePhasedSafetys (Part)

TimePhasedYields (PartSource)

TimeStamp (EngineeringChange_Notes)

TimeStamp (ForecastItemParametersOutlier_Notes)

TimeStamp (IndependentDemand_Notes)

TimeStamp (IndependentDemandSolution_CustomerNotes)

TimeStamp (IndependentDemandSolution_SupplyNotes)

Timestamp (Job)

TimeStamp (Mfg__SupplyOrder_TextLine)

TimeStamp (ScheduledReceipt_Notes)

TimeStamp (SupplyAllocation_Notes)

TimeStamp (Task_Notes)

TimeUnits (PlanningCalendars)

TimeZone (Site)

ToAttributeName (AttributeMap)

ToDateQty (PartSource)

ToFeature (FeatureMap)

ToFeatures (Feature)

Tolerance (MPSConfiguration)

ToleranceCalendar (MPSConfiguration)

ToleranceCalendar (ToleranceProfile)

ToleranceInterval (MPSConfiguration)

ToleranceProfile (PartCustomer)

ToleranceProfile (PartSupplier)

ToleranceProfile (ToleranceProfileZone)

ToleranceProfileBaseline (Calendar)

ToleranceProfiles (ControlSet)

ToleranceProfileTolerance (Calendar)

ToleranceProfileZone (ForecastDetail)

ToleranceProfileZone (HistoricalDemandWaterfall)

ToleranceProfileZone (HistoricalSupplyWaterfall)

ToleranceProfileZone (IndependentDemand)

ToleranceProfileZone (ScheduledReceipt)

ToleranceProfileZone (SupplyAllocation)

ToleranceProfileZones (ControlSet)

ToProductionGroup (OverrideMajorChangeover)

ToRecipients (Notification)

TotalAllocations (InventorySummary)

TotalAllocations (Part)

TotalByProductSupply (InventorySummary)

TotalByProductSupply (Part)

TotalCoProductSupply (InventorySummary)

TotalCoProductSupply (Part)

TotalDemand (InventoryAnalysis)

TotalDemand (InventoryCTPAnalysis)

TotalDemand (InventorySummary)

TotalDemand (Part)

TotalEffPlannedOrder (Part)

TotalEffPlannedOrders (InventorySummary)

TotalEffScheduledReceipts (InventorySummary)

TotalEffScheduledReceipt (Part)

TotalFirstYearDemand (Part)

TotalForecast (InventorySummary)
TotalForecast (Part)
TotalIndependentDemand (InventorySummary)
TotalIndependentDemand (Part)
TotalIndependentDemandCount (InventorySummary)
TotalIndependentDemandCount (Part)
TotalInventoryTransfer (InventorySummary)
TotalInventoryTransferAllocation (InventorySummary)
TotalLoad (PeriodConstraintLoad)
TotalNeededSupply (InventorySummary)
TotalNeededSupply (Part)
TotalNettableOnHand (InventorySummary)
TotalNettableOnHand (Part)
TotalNewAllocations (InventorySummary)
TotalNewAllocations (Part)
TotalNonNettableOnHand (InventorySummary)
TotalNonNettableOnHand (Part)
TotalPlannedAllocations (InventorySummary)
TotalPlannedAllocations (Part)
TotalPlannedOrder (Part)
TotalPlannedOrderCount (Part)
TotalPlannedOrders (InventorySummary)
TotalQuantity (Allocation)
TotalQuantity (Demand)
TotalQuantity (DependentDemand)
TotalQuantity (NewAllocation)
TotalQuantity (NewTransferAllocation)
TotalQuantity (ScheduledReceipt)
TotalSlack (Task)
TotalSlackRule (ProjectType)
TotalSpreadLoad (PeriodConstraintLoad)
TotalSubstituteAllocation (InventorySummary)
TotalSubstituteSupply (InventorySummary)
TotalSupply (InventorySummary)
TotalSupply (Part)
TotalSupplyCount (Part)
TotalUnconsumedForecast (InventorySummary)
TotalUnconsumedForecast (Part)
TotalUsedLoad (PeriodConstraintLoad)
ToAttributeValue (AttributeMap)
TransactionSequence (Activity)
TransactionSequence (BlowThroughPlannedAllocation)
TransactionSequence (CTPActivity)
TransactionSequence (IndependentDemand)
TransactionSequence (LateSupply)
TransactionSequence (MLSActivity)
TransactionSequence (PlannedOrder)
TransactionSequence (ScheduledReceipt)
TransactionSequence (TrialPlannedOrder)
TransferCost (InventoryTransfer)
TransferCost (TransferDestination)
TransferModel (InventoryTransfer)
TransferPart (InventoryTransfer)
TransferPart (PartSource)
TransferPart (TransferDestination)
TransferPartSources (Part)
TransferPool (InventoryTransfer)
TransferStartUnit (InventoryTransfer)
TransitCalendar (Carrier)
TransitCalendar (InventoryTransfer)
TransitCalendar (Source)
TransitRule (PartSourceType)
TransitTime (CRPOperation)
TransitTime (DeliveryRoute)
TransitTime (InventoryTransfer)
TransitTime (Operation)
TransitTime (ScheduledReceipt)

TransitTime (Source)

TransitTime (TimePhasedPartSourceLeadTime)

TransitTimeProcessingRule (OperationType)

TransitTimeRule (PartSourceType)

TransitTimeSequenceRule (OperationType)

TransportationModes (TransportationMode)

TransportationMode (DeliveryRoute)

TransportationMode (Source)

TrendComponent (SafetyStockItemOutlierResult)

TrendComponent (StatisticalForecastOutlier)

TrendDecayFactor (BestFitProfileDetail)

TrendDecayFactor (ForecastItemParameters)

TrendDecayFactorRule (ForecastItemParametersType)

TrialAvailableDate (MLSActivity)

TrialAvailableDate (TrialPlannedOrder)

TrialAvailableDate (TrialSubstituteSupply)

TrialCoByProductSupply (MLSActivity)

TrialPlannedOrder (MLSActivity)

TrialSubstituteSupply (MLSActivity)

TriggerEvent (Notification)

TriggerEventOffset (Notification)

tStatistic (ForecastItemParametersRegressionCoefficient)

tStatistic (StatisticalForecastRegressionCoefficient)

TwoDatePlanningRule (OrderPriority)

TwoDatePriorityLimit (OrderPriority)

Type (Activity)

Type (AggregatePartCustomer)

Type (AllotmentOverride)

Type (AlternatePart)

Type (Assumption)

Type (AttributeMap)

Type (BaselineConflict)

Type (BaselineTaskLink)

Type (BillOfMaterial)

Type (BlowThroughPlannedAllocation)

Type (Constraint)

Type (ConstraintUsed)

Type (CRPOperation)

Type (CTPActivity)

Type (CurveParameters)

Type (DataChange_Historical)

Type (DataChange_Local)

Type (DataChange_PendingUpdate)

Type (DataChange)

Type (Demand)

Type (DemandException)

Type (DemandOrder)

Type (DependentDemand)

Type (EngineeringChange)

Type (Event)

Type (ForecastItemParameters)

Type (HistoricalDemandCategory)

Type (HistoricalSupplyOrder)

Type (InventoryTransfer)

Type (Job)

Type (LogisticsLocation)

Type (MEIOLeadTime)

Type (MEIOStage)

Type (OnHand)

Type (Operation)

Type (OperationSequence)

Type (Part)

Type (PartSource)

Type (PlannedAllocation)

Type (ProcessInstance)

Type (ProductionWheel)

Type (Project)

Type (Resource)

Type (SafetyStockItem)

Type (SafetyStockPart)

Type (ShipSet)

Type (Site)

Type (SiteManager)

Type (SubstituteGroup)

Type (Supply)

Type (SupplyAllocation)

Type (SupplyOrder)

Type (Task)

Type (TaskPartLink)

Type (WorkCenter)

U

UnassignedQuantity (TaskPartLink)

UnboundedQuantity (SafetyStockOverride)

UnboundedSafetyStock (SafetyStockResult)

UnconstrainedQuantity (ScheduledReceipt)

Unit (DemandException)

UnitCost (HistoricalDemandActual)

UnitCost (OnHand)

UnitCost (PartSource)

UnitHoldingCost (MEIOStage)

UnitHoldingCost (SafetyStockPart)

UnitOfMeasure (Constraint)

UnitOfMeasure (ConstraintUnitOfMeasure)

UnitOfMeasure (CRPUnitOfMeasure)

UnitOfMeasure (CurveParameters)

UnitOfMeasure (Part)

UnitOfMeasure (PartUOMConversion)

UnitOfMeasures (ControlSet)

UnitOfMeasures (UnitOfMeasureCategory)

UnitPrice (CausalFactorDetail)

UnitPrice (CustomerPrice)

UnitPrice (ForecastDetail)

UnitPrice (HistoricalDemandActual)

UnitPrice (HistoricalDemandSeriesDetail)

UnitPrice (HistoricalSupplyActual)

UnitPrice (HistoricalSupplySeriesDetail)

UnitPrice (ScheduledReceipt)

UnitRule (MUEPoolNettingType)

Units (ResourceAssignmentAvailable)

Units (ResourceDataByDate)

UnitSellingPrice (Demand)

UnitSellingPrice (IndependentDemand)

UnitType (HistoricalDemandCategoryType)

UnsatisfiedDemandLoggedQuantityRule (MultiLevelSearchRule)

UnsatisfiedDemandTolerance (Part)

UnsatisfiedQuantity (Allocation)

UnsatisfiedQuantity (Demand)

UnsatisfiedQuantity (DependentDemand)

UnsatisfiedQuantity (Forecast)

UnsatisfiedQuantity (IndependentDemand)

UnsatisfiedQuantity (NewTransferAllocation)

UnsatisfiedQuantity (PlannedTransferAllocation)

UnsatisfiedSupplyRule (ProductionWheelType)

UnusedConstraintRule (ProductionWheelType)

UOMConversions (Part)

UOMConversions (UnitOfMeasure)

UpperBound (ProjectedLeadTime)

UpperBound (ProjectedYield)

UpperThreshold (SafetyStockItemOutlierResult)

UpperThreshold (StatisticalForecastOutlier)

UpsidePercentage (ToleranceProfileZone)

URL (TaskHyperlink)

UsageRule (EventPhase)

UsageRule (ForecastItem)

UseCase (Job)

UseDate (TrialConsumption)

UseDaysSupply (PartType)

UsedThisPeriod (Constraint)

UseFixedLeadTime (OrderPolicy)

UseLeadTimeAdjust (PartType)

UseLeadTimeOffset (BOMType)

UseLTForFlatBill (PartType)

UseModel (BOM_MUECUMLead)

UseModel (Part_MUECumLead)

UseNegativeOnHand (PartType)

UseNegativeOnHand (PartType)

UseNegativeOnHand (PartType)

UseObsoleteTransition (IMConfiguration)

UseOwnDemand (ForecastItemParameters)

User (DataChange_Historical)

User (DataChange_Local)

User (DataChange_PendingUpdate)

User (DataChange)

UserId (BuyerCode)

UserID (EngineeringChange_Notes)

UserId (ForecastItemParametersOutlier_Notes)

UserId (IndependentDemand_Notes)

UserId (IndependentDemandSolution_CustomerNotes)

UserId (IndependentDemandSolution_SupplyNotes)

UserId (Mfg__SupplyOrder_TextLine)

UserId (PlannerCode)

UserId (ScheduledReceipt_Notes)

UserId (SupplyAllocation_Notes)

UserId (Task_Notes)

UseRollingLeadTimeDemand (SafetyStockItemType)

UserRecord (DataChange_Historical)

UserRecord (DataChange_Local)

UserRecord (DataChange_PendingUpdate)

UserRecord (DataChange)

UseSafetyLeadTime (PartType)

UseTimePhasedPartSourceLeadTime (PartSourceType)

UseTimePhasedSourceLeadTime (PartSourceType)

UseVarLeadTime (OrderPolicy)

UseWhenDetectingOutliers (MEIOLeadTimeOutlier)

UseWhenDetectingOutliers (SafetyStockItemOutlier)

Utilization (Capacity)

Utilization (CapacityOverride)

Utilization (WorkCenterCapacity)

V

ValidatePlannedOrders (Part)

ValidationDifferencesRollUpCount (Part)

ValidationDifferencesRollUpSum (Part)

ValidationDifferencesThisLevel (Part)

Value (ABCCode)

Value (AggregatePartCustomerType)

Value (AllotmentOverrideType)

Value (AlternatePartType)

Value (AssumptionStatus)

Value (AssumptionType)

Value (AttributeMapType)

Value (AvailableRule)

Value (AutomatedLeadTimeType)

Value (AutomatedYieldType)

Value (BOMAlternate table)

Value (BOMType)

Value (BuyerCode)

Value (Calendar)

Value (CalendarDate)

Value (Carrier)

Value (CausalFactorCategory)

Value (Class)

Value (ConstraintType)

Value (ConstraintMetricValues)

Value (ConstraintUnitOfMeasure)

Value (ControlSet)

Value (CRPUnitOfMeasure)

Value (Currency)

Value (DaysSupplyPolicy)

Value (DDMRPRule)

Value (DemandStatus)

Value (DemandType)

Value (DistributionPlanningRule)

Value (EngineeringChange_Notes)

Value (EventType)

Value (ExpiryType)

Value (Feature)

Value (ForecastDetail)

Value (ForecastItemParametersOutlier_Notes)

Value (ForecastItemParametersType)

Value (HistoricalDemandCategory)

Value (HistoricalDemandCategoryType)

Value (HistoricalDemandSeriesDetail)

Value (HistoricalSupplyCategory)

Value (IncrementalRule)

Value (IndependentDemand_Notes)

Value (IndependentDemandSolution_CustomerNotes)

Value (IndependentDemandSolution_SupplyNotes)

Value (InventoryTransferType)

Value (ItemSourceFitOutputClass)

Value (ItemSourceFitOutputForecastModel)

Value (ItemSourceFitOutputValueName)

Value (KPI)

Value (MEIOLeadTimeDistributionProfilePoint)

Value (MEIOLeadTimeType)

Value (Mfg__SupplyOrder_TextLine)

Value (Model)

Value (MUEPoolNettingType)

Value (MultiEchelonSafetyStockRule)

Value (MultiLevelSearchRule)

Value (Numbers)

Value (OnHandType)

Value (OperationSequenceType)

Value (OperationType)

Value (OrderPolicy)

Value (OrderPriority)

Value (OutlierType)

Value (PartMetricValues)

Value (PartSourceType)

Value (PartType)

Value (PlannerCode)

Value (PlanningCalendars)

Value (Pool)

Value (ProcessInstanceType)

Value (ProductFamily)

Value (ProductionRecurrence)

Value (ProductionWheelType)

Value (ProjectStatus)

Value (ProjectType)

Value (RangeOfCoverage)

Value (ReferencePart)

Value (ResourceType)

Value (RParameter)

Value (SafetyLeadTimeProfile)

Value (SafetyStockItemType)

Value (SafetyStockPolicy)

Value (ScheduledReceipt_Notes)

Value (Segment)

Value (ShipmentPolicy)
Value (ShipSet)
Value (ShipSetType)
Value (Site)
Value (SiteManager)
Value (SiteManagerType)
Value (SiteMetricValues)
Value (SourceRule)
Value (SpreadProfile)
Value (StorageType)
Value (SubstituteGroup)
Value (SubstituteGroupType)
Value (SupplierMetricValues)
Value (SupplyAllocation_Notes)
Value (SupplyAllocationType)
Value (SupplyStatus)
Value (SupplyType)
Value (Task_Notes)
Value (TaskDemandLinkCost)
Value (TaskPartLinkCost)
Value (TaskSupplyLinkCost)
Value (TaskType)
Value (TimeZone)
Value (ToleranceProfile)
Value (TransportationMode)
Value (UnitOfMeasure)
Value (WorkCenterType)
ValueBoolean (ClusterProperty)
ValueDate (ForecastItemParametersFitOutput)
ValueDate (ItemSourceFitOutput)
ValueDate (MEIOLeadTimeOutlierSummary)
ValueDate (SafetyStockItemOutlierResultSummary)
ValueDate (StatisticalForecastFitOutput)
ValueDate (StatisticalForecastOutlierSummary)
ValueName (ForecastItemParametersFitOutput)
ValueName (ItemSourceFitOutput)
ValueName (MEIOLeadTimeOutlierSummary)
ValueName (SafetyStockItemOutlierResultSummary)
ValueName (StatisticalForecastFitOutput)
ValueName (StatisticalForecastOutlierSummary)
ValueQuantity (ClusterProperty)
ValueQuantity (ForecastItemParametersFitOutput)
ValueQuantity (ItemSourceFitOutput)
ValueQuantity (MEIOLeadTimeOutlierSummary)
ValueQuantity (SafetyStockItemOutlierResultSummary)
ValueQuantity (StatisticalForecastFitOutput)
ValueQuantity (StatisticalForecastOutlierSummary)
ValueText (ClusterProperty)
ValueText (ForecastItemParametersFitOutput)
ValueText (ItemSourceFitOutput)
ValueText (MEIOLeadTimeOutlierSummary)
ValueText (SafetyStockItemOutlierResultSummary)
ValueText (StatisticalForecastFitOutput)
ValueText (StatisticalForecastOutlierSummary)
ValueType (ClusterProperty)
ValueType (ForecastItemParametersFitOutput)
ValueType (MEIOLeadTimeOutlierSummary)
ValueType (Metric)
ValueType (SafetyStockItemOutlierResultSummary)
ValueType (StatisticalForecastFitOutput)
VarLeadTime (PartSource)
VarLeadTime (ScheduledReceipt)
VisibleFuturePeriods (IMConfiguration)
VisibleHistoricalPeriods (IMConfiguration)

W

WaitDate (LoadOperationNewOrderPlanned)
WaitDate (LoadOperationReceiptsCurrent)

WaitDate (LoadOperationReceiptsPlanned)
WaitOffset (LoadOperationNewOrderPlanned)
WaitOffset (LoadOperationReceiptsCurrent)
WaitOffset (LoadOperationReceiptsPlanned)
WaitTime (Capacity)
WaitTimeOverride (CRPOperation)
WaitTimeOverride (Operation)
Warehouse (Location)
Warehouses (Site)
WarningLimit (HistoricalDemandCategory)
Waybill (Shipment)
Weekly (PlanningCalendars)
WeekStarting (LoadWeeklyCurrent)
WeekStarting (LoadWeeklyPlanned)
Weight (AttributeMap)
Weight (ConsensusForecastRollingHorizonWeight)
Weight (ConsensusForecastWeightByHeader)
Weight (EventDisaggregationRate)
Weight (EventStatisticalDisaggregationRate)
Weight (MEIOLeadTimeDistributionProfilePoint)
WeightsDecayFactor (ForecastItemParameters)
WERKS_Plant (SAP_MD04)
WhereConsumed (CostOfGoodsSold)
WhereConsumed (FLPConstraint)
WhereConsumed (LoadFLPPlanned)
WhereConsumed (WhereConsumedToHighVolumeOrder)
WhereConsumedForDemands (IndependentDemand)
WhereConsumedForForecasts (Part)
WhereConsumedForSupplies (Part)
WhereConsumeds (Part)
WhereConsumedToHighVolumeOrders (Part)
WhereUsed (Part)
WIPStart (InProcess)
Work (Task)
WorkCenter (Capacity)
WorkCenter (CapacityOverride)
WorkCenter (CRPOperation)
WorkCenter (LoadDailyCurrent)
WorkCenter (LoadDailyPlanned)
WorkCenter (LoadDetailCurrent)
WorkCenter (LoadDetailPlanned)
WorkCenter (LoadWeeklyCurrent)
WorkCenter (LoadWeeklyPlanned)
WorkCenter (Operation)
WorkCenter (Site)
WorkCenter (WorkCenterCapacity)
WorkCenters (PlanningCalendars)
WorkCenters (WorkCenterType)
WorkCenterTypes (ControlSet)
WorkingHour (CapacityOverride)
WorkingHour (WorkCenterCapacity)

Y

Yield (PartSource)
Yield (ProjectedYield)
Yield (TimePhasedYield)
YieldCorrectionStatus (ItemSource)
YieldRoundingUsage (OrderPolicy)
YieldRule (SupplyStatus)
YieldType (SHSCConfiguration)
YieldUsage (OrderPolicy)

Z

Zone (Cluster)
ZoneEndInterval (ToleranceProfile)
Zones (ToleranceProfile)

CHAPTER 1: Welcome

This guide provides reference information about the RapidResponse data model. This guide can be used by users involved with integrating data into RapidResponse and workbook authors (report writers).

All tables and fields are described in this guide. Information is also provided about RapidResponse analytics and calculations.

In this chapter, you'll learn more about:

What's in this Guide **113**
About Kinaxis **114**

Note: For information about changes made to recent versions of the RapidResponse data model, see "Part 4: Analytic and data model change summary" on page 2259.

What's in this Guide

This guide is divided into four parts each of which contains numerous chapters. The following is a brief overview of the information contained in each part:

- **Part I: Table and field reference**—provides and overview of the RapidResponse data model, and contains reference for each table and field it contains. Individual chapters describe the input, calculated, and control tables in the data model.
- **Part II: Analytics and calculations**—provides an overview of the standard and optional analytics in RapidResponse, as well as detailed information on configuring and understanding the calculations produced by each analytic. Individual chapters exists for most of the main analytics, or calculation types, in RapidResponse.
- **Part III: Configuring resource calculations**—provides an overview of and describes how to configure the calculations performed by certain RapidResponse resources.
- **Part IV: Analytic and data Model Change Summary**—provides an overview of changes made to the data model in recent versions of RapidResponse. For each new version, all new tables and fields are listed, along with descriptions of new analytics and potential changes in calculated output.

What you need to know

This guide assumes that you are familiar with RapidResponse. This guide also assumes that you are familiar with manufacturing concepts such as Material Requirements Planning (MRP) and your enterprise data sources.

About Kinaxis

Kinaxis® delivers cloud-based S&OP and supply chain applications for discrete manufacturers and brand owners with complex supply chain networks and volatile business environments.

Leaders across multiple industry verticals, including A&D, Automotive, High Tech, Industrial and Life Sciences rely on RapidResponse® applications to create a foundation for concurrent planning, continuous performance monitoring, and coordinated responses to plan variances across multiple areas of the business. All founded on a single product, RapidResponse's configurable applications encompass a full spectrum of supply chain processes, including such functions as S&OP, supply planning, capacity planning, demand planning, inventory management, MPS, and order fulfillment.

As a result, Kinaxis customers have replaced disparate planning and performance management tools and are realizing significant operations performance breakthroughs in planning cycles, supply chain response times and decision accuracy. From a single product, customers are able to easily model varying supply chain conditions to make both long-term and real-time demand and supply balancing decisions quickly, collaboratively, and in line with the shared business objectives of multiple stakeholders.

For more information, visit www.kinaxis.com or the Kinaxis Knowledge Network at http://knowledge.kinaxis.com/.

CHAPTER 2: Accessing help and documentation

Determining which help system or guide to use **116**
Documentation conventions **118**
Finding the content you need in HTML help **118**
Your feedback **119**
Help embedded in resources **120**
Other ways to learn about RapidResponse **125**
Customer Support **126**

The RapidResponse documentation set includes several different help systems and guides, each dealing with a different area of knowledge. Help for individual resources, such as workbooks is also available.

In addition to the RapidResponse documentation, you can access a variety of other resources to help you learn about the capabilities and features in RapidResponse. These include an online community where you can view how-to videos and find advice, a mailing list that you can use to keep up to date on supply chain news and upcoming events, and instructor-led training courses.

Accessing RapidResponse help systems and guides

You can access RapidResponse help systems and guides two ways:

- Access documentation directly from the Help menu in RapidResponse.
- For links to all of the available help systems and guides, see "Determining which help system or guide to use" on page 116.

Account permissions and the contents of your Help menu

The list of options you see when you click the Help menu in RapidResponse depends on the permissions associated with your RapidResponse account. The more permissions you have in RapidResponse, the more options you see on the menu. For example, resource authors have more permission than general users.

Links to documentation that you are less likely to need, based on your permissions, are not shown on the Help menu. For example, users without administrator permissions do not see Administration Help.

Documentation formats

Most RapidResponse documentation resources are available as both HTML help systems and PDF documents. Some documentation is only available in PDF format.

HTML help systems are optimized for viewing in a web browser. While it is possible to print individual topics from HTML help, you will get better results if you print the topic or topics from the equivalent PDF guide.

PDF guides are optimized for printing. They can be viewed and printed using Adobe Reader®. You can download Adobe Reader from the Adobe site at www.adobe.com.

Accessing help for RapidResponse resources

Help is often available in the RapidResponse application window for individual resources of the following types:

- Dashboards and widgets
- Worksheets and workbooks
- Scorecards
- Forms
- Scripts
- Responsibility definitions

For more information, see "Help embedded in resources" on page 120.

Determining which help system or guide to use

The RapidResponse documentation set includes several help systems and guides, each dealing with a different area of knowledge. The following table summarizes the available documentation, some of which you can access from the Help menu in RapidResponse.

Title	Description
RapidResponse User Guide	This guide provides RapidResponse users with basic reference and procedural information. It covers topics such as viewing data, modifying data as part of simulation, solving business problems, and customizing the user interface.
RapidResponse User Guide (Mobile Client)	This guide provides information on how to view and edit data in the Mobile Client.
RapidResponse Applications Guide	Practical ways to use RapidResponse and the applications that can be deployed to support business processes and make decisions. It also includes instructions aimed at RapidResponse administrators that outline how to install and configure applications.

Title	Description
RapidResponse Fundamental Concepts Guide	RapidResponse product overview, which includes key capabilities and the deployment methodology.
RapidResponse Resource Authoring Guide	This guide provides information on creating and managing resources, such as dashboards, workbooks, and filters. Detailed information about RapidResponse Query language is also included.
RapidResponse Analytic and Data Model Guide	Description of the RapidResponse data model and associated analytics. All tables and fields in the data model are listed and described. This guide also notes changes to the data model corresponding to the RapidResponse release.
Data model posters	A series of posters illustrating the structure of tables and fields in RapidResponse, and the relationships between fields. For more information, see "Data Model Posters" on page 117.
RapidResponse Administration Guide	Information for administrators, covering installation, upgrades, maintenance, system settings, and user administration.
RapidResponse Scripting Guide	Information about building custom applications using scripting language objects, functions, and methods to automate some RapidResponse processes.
RapidResponse Web Services Guide	Information on using RapidResponse Web services to create users, resources, and Web service client programs.
RapidResponse Data Integration Guide	Information about integrating enterprise data sources with RapidResponse, including mapping data from source tables and fields, customizing the RapidResponse database, extracting data from enterprise sources, performing data updates to bring new and updated data into RapidResponse, moving data between RapidResponse instances, and moving data changes between RapidResponse and business partners in real-time.
Global Help	If you are not sure which help system contains the information you need, you can search for it in this help system, which includes information from all of the other help systems.
Release summary	The latest release summary outlines all of the changes that have been made in RapidResponse 1908, including new features and defect resolutions in service updates. Release summaries can be found in the Kinaxis Knowledge Network in the Documentation Center.
Custom Help	Your company might have also created custom help specific to your RapidResponse implementation.

The permissions associated with your RapidResponse account affect the list of guides that you can access from the Help menu. For more information, see "Account permissions and the contents of your Help menu" on page 115.

Data Model Posters

Users who have permission to create resources such as workbooks and dashboards have access to the following posters on their RapidResponse Help menus:

- *RapidResponse Data Model for Import poster*—displays the relationship between the tables and fields used in the RapidResponse data import process.

- *RapidResponse Calculated Data Model poster*—displays the relationship between the main RapidResponse database calculated tables and fields. Calculated fields in the RapidResponse data model input tables are also displayed.
- *RapidResponse Sales & Operations Planning poster* —displays the relationship between the tables used to support the Sales & Operations Planning application.
- *RapidResponse Inventory Planning and Optimization poster* —displays the relationship between the tables used to support Inventory Planning and Optimization application.
- *RapidResponse Integrated Project Management poster* —displays the relationship between the tables used to support Integrated Project Management application.
- *RapidResponse Historical Supply poster* and *RapidResponse Historical Demand poster*—displays the relationship between all tables (input, control, and calculated) that are used to store historical supply data and historical demand data, respectively.

Documentation conventions

To help you quickly locate and interpret information, Kinaxis guides use conventions noted in the following table.

	Convention	Description
	bold	Bold is used for user-interface elements in procedures. For example: On the **File** menu, click **New**.
	italic	Italic is used when citing other documents. For example: For more information, see the *RapidResponse Administration Guide*.
	`Courier New`	Courier New is used for programming examples and text that is entered in Microsoft Windows Command Prompt window or command lines.
	Caution:	Identifies a caution message (critical information).
	Note:	Identifies a note.
	Tip:	Identifies a tip.

If you are using a Mac OS, procedures that use the Windows CTRL key should be replaced with the Mac OS Command key.

Finding the content you need in HTML help

You can find information in any RapidResponse HTML help system by searching or by browsing in the contents or the index.

Searching the help

You can find the information you require by entering a term in the Search box. You can also refine your search by using AND or OR to search for a combination of words, and you can put quotation marks around your search word or phrase to search for that exact phrasing only. For example, "sort" would return any topics with the word "sort", but not the words "sorted" or "sorting".

Search for content

- Type a search term in the search bar and then click .

Note: If you do not want the words you searched for highlighted in the help topic, click **Remove Highlights** .

Tip: If the browser window is small, the search bar might not be displayed. To see a search bar click or enlarge the window.

Browsing the contents and the index

You can also find information by browsing in either of these panes:

- **Contents**—Browse through the topics in the table of contents.
- **Index**—Find a topic through the index. Scroll through the index or type a word or phrase in the Search Index box to find an index entry.

Tip: If the browser window is small, the Contents and Index panes might not be displayed. To see them, click or enlarge the window.

Your feedback

Kinaxis Inc. takes great pride in developing user-friendly applications and we hope that our documentation resources ensure a high level of usability. We welcome your feedback about the help topics you access.

If you have comments or suggestions about Kinaxis documentation or training materials, you can email them to learning-techsupport@kinaxis.com.

Help embedded in resources

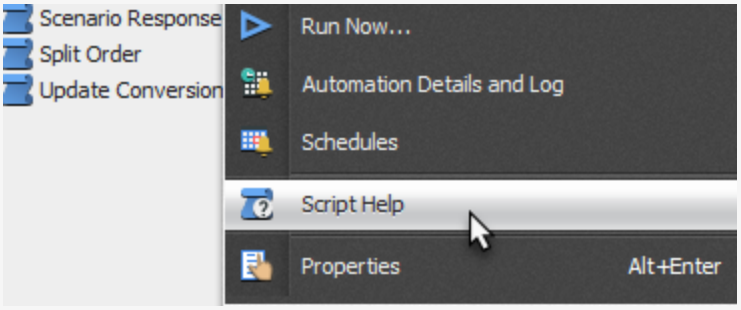
Many openable RapidResponse resources have help embedded within the resource. The form the help takes depends on the type of resource.

When help is available for a resource, you can view it from the open resource. You can also view the help for some types of resources from the Explorer, without having to open the resource first. These resources include workbooks, scorecards, widgets, forms, and scripts. Viewing resource help from the Explorer can help you to determine whether a resource is the right one to use, without having to wait for it to load.

View resource help without opening the resource

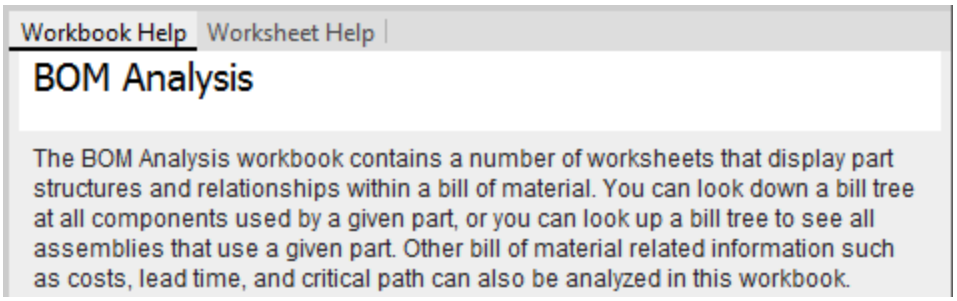
- In the **Explorer**, select a resource and then click ⓘ beside the name of the resource.

Tip: You can also view resource help from the Explorer by right-clicking the name of the resource and then selecting the Help option for the resource from the menu. For example, to view help for a script, click Script Help.



View and print worksheet and workbook help

RapidResponse worksheets can include a help page that is displayed in a pane on the right-hand side of the worksheet. Workbooks might also include a help page, in which case, the help pane includes two tabs: Workbook Help and Worksheet Help.



Show or hide the help pane

- On the **Help** menu, click **Show Workbook/Worksheet Help**.

Note: This command is not available for workbooks that do not have workbook help or help for at least one worksheet defined.

Tip: You can also show or hide the help pane by clicking **Show Workbook/Worksheet Help** on the toolbar.

Print the worksheet or workbook help

You can print the worksheet or workbook help when the help pane is displayed.

1. On the **File** menu, click **Print**.
2. Click **Worksheet help** or **Workbook Help**, and then click **OK**.

Note: The worksheet and workbook help are printed as they appear in the help pane. Widen the help pane before printing to maximize the amount of information printed on a page.

Tip: You can also print the worksheet help by right-clicking in the help pane, and then clicking **Print**.

View and print scorecard help

Each scorecard in RapidResponse includes a help page that is displayed in a pane on the right-hand side of the scorecard. You can show and hide the scorecard help pane.

- On the **Help** menu, click **Show Scorecard Help**.

Note: This command is available only for scorecards that have help defined.

Tip: You can also show or hide the scorecard help pane by clicking **Show Scorecard Help** on the toolbar.

Print the scorecard help

You can print the scorecard help when it is displayed.

1. On the **File** menu, click **Print**.
2. Click **Scorecard help**, and then click **OK**.

Note: The scorecard help is printed as it appears in the help pane. Widen the worksheet help pane before printing to maximize the amount of information printed on a page.

Tip: You can also print the scorecard help by right-clicking in the help pane, and then clicking **Print**.

View dashboard and widget help

Dashboard help is often available for a dashboard as a whole, and you might also be able to view help for individual widgets.

Dashboard help

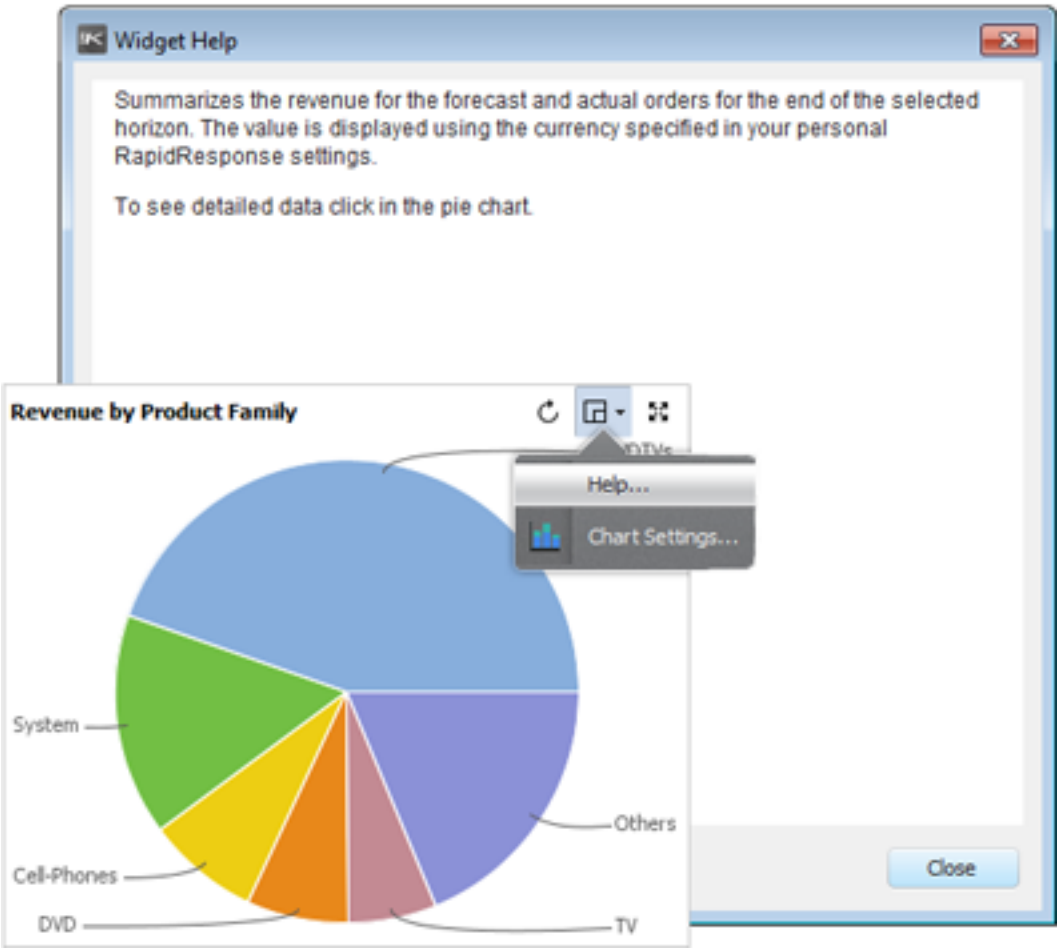
Some dashboards might have an Information tab with a text widget that shows dashboard help. Dashboard help contains information about how the dashboard is intended to be used, and can include links to related resources, websites, or help videos.

Widget help

Most widgets on dashboards that are supplied with RapidResponse have widget help, and some custom widgets might also include help. Widget help typically describes the metrics displayed in a widget, and might include links to other resources such as workbooks and websites. You can access help for widgets directly from the widget controls on a widget on your dashboard.

View widget help

- On a widget title bar, click **Actions**, and then click **Help**.
  The widget help displays in a dialog box.

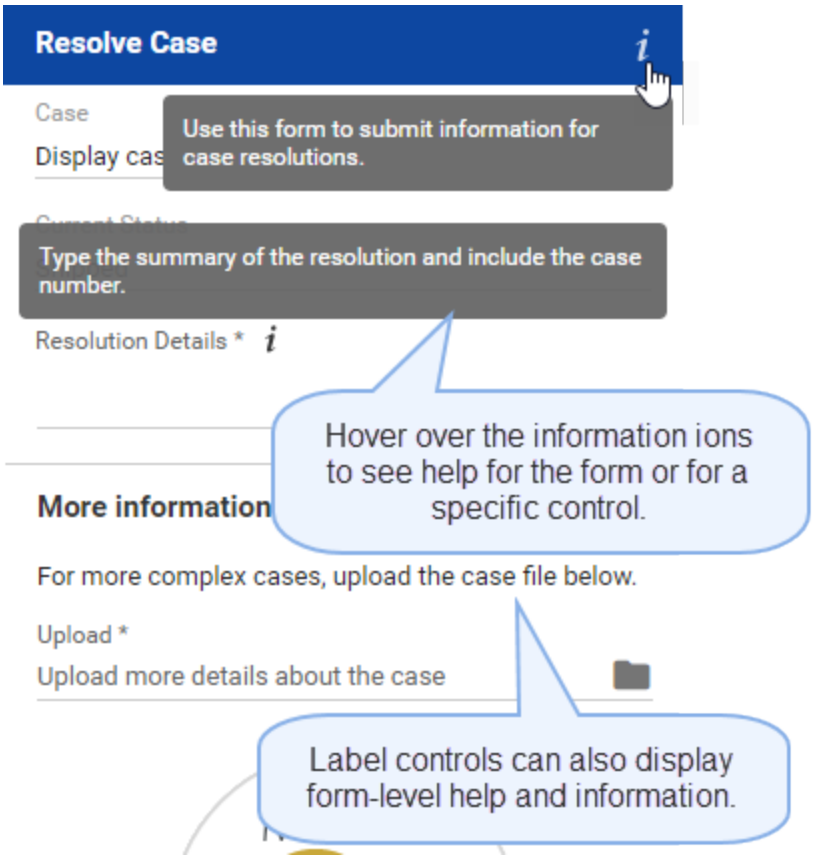


View form help

Forms can display three types of help: contextual help directly on the form, form pop-up help, and field pop-up help.

Both contextual help and form pop-up help might describe the purpose of the form, how to use it, or what happens when the form is run. Contextual help displays right on the form. Form pop-up displays when you hover over the information icon at the top right corner of the form.

Individual fields on a form might also display help specific to the field, such as what types of values it accepts.



View pop-up help for a form

1. On the form, point to the information icon in the upper right corner of the form
   Form help displays below the form title bar.
2. Point away from the information icon to close the help.

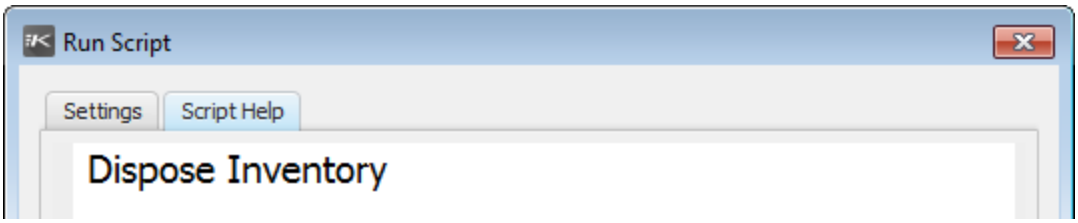
Note: Only forms with help display the information icon in the upper corner of the form.

View pop-up help for a field

1. Point to the field you want to view help for.
   Only fields with help display next to the field title
2. Point to the information icon on the field.
   Help for the field displays above or below the field.
3. Point away from information icon to close the help.

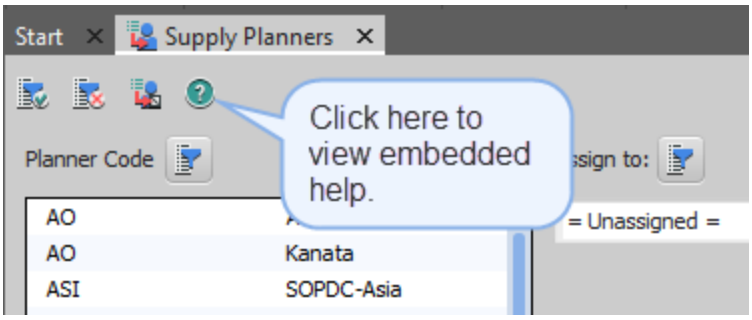
View script help

Some scripts might have embedded help. If a script has embedded help, a Script Help tab is shown when you run the script.



View help for a responsibility definition

You might be able to access embedded help for an open responsibility definition. If help is available, a Responsibility Help button is available near the top left of the tab.



Resources without embedded help

Some types of resources, such as filters, cannot have embedded help, and not every resource that could have embedded help does have embedded help. Resource authors decide whether or not to include help when they create new resources that can have embedded help.

Sometimes, the information needed to use that resource might be available elsewhere. For example, if you access a dashboard by opening it from a task flow, the information you need to understand the purpose of the dashboard might be included in the task flow. Another example is a custom resource created for your company, which might be documented in company procedures that are maintained outside of RapidResponse.

Other ways to learn about RapidResponse

In addition to the RapidResponse documentation set, there are several other ways to learn about RapidResponse.

Kinaxis Knowledge Network

Visit the Kinaxis Knowledge Network at knowledge.kinaxis.com to access features including:

- Upgrade Center—Learn what's new in each version of RapidResponse.
- Training videos—Familiarize yourself with the basic capabilities of RapidResponse by taking an introductory course, delivered as a series of short videos. You can also view videos on some more advanced topics.
- Discussion Forums—Discuss RapidResponse with other users.
- Blogs—Learn how others are using RapidResponse and solving supply chain problems.

Some content is only available after you create a Kinaxis Knowledge Network user account and sign in.

News mailing list

Subscribe to the mailing list to stay up-to-date on news and upcoming supply chain events, such as Kinexions, the annual Kinaxis training and user conference. You can find the subscription form at http://info.kinaxis.com/customer-subscription-page.

Training courses

Kinaxis offers instructor-led courses and self-paced online training courses to help you develop in-depth RapidResponse knowledge. Instructor-led courses are usually delivered online in a virtual classroom, but on-site instruction can also be arranged. To see a list of available courses, visit https://www2.kinaxis.com/rapidresponse-training/courses.cfm.

Customer Support

The Kinaxis Customer Support team understands demand and supply chain planning, monitoring and response, and is experienced in applying RapidResponse to real-world business scenarios. Areas of expertise include:

- RapidResponse functionality
- Data extraction and mapping issues
- Technical software compatibility
- Coordination of integration
- Product implementation

If you are a key support contact, you can submit support cases. To submit a support case, log into the Kinaxis Knowledge Network. You can also contact Kinaxis Customer Support by phoning 1-866-463-7877 or by sending a message to support@kinaxis.com.

Kinaxis Knowledge Network

The Kinaxis Knowledge Network is the place to find knowledge about RapidResponse. In one site, you can:

- Search all knowledge about RapidResponse, including product documentation, knowledge base articles, and threaded discussions about common questions.
- Review your company's Customer Support requests (and, if you are a key support contact, create new Support requests).
- Engage with a community of RapidResponse experts in the Discussion Groups.

Go to https://knowledge.kinaxis.com and log in today. You can also access the Kinaxis Knowledge Network from RapidResponse by clicking **Kinaxis Knowledge Network** on the **Help** menu.

Part 1: Table and field reference

- RapidResponse database
- Introduction to the RapidResponse data model
- Optional applications and tables
- Input tables
- Control tables
- Calculated tables
- Notes tables
- Referenced system tables

CHAPTER 3: RapidResponse database

The following chapters in this guide provide reference information about all tables in the RapidResponse data model. Each of these tables is associated with a particular namespace and classified as belonging to one of several distinct types. Each field contained in a table is also classified as belonging to a specific field type. As well, the data contained in each field belongs to a specific type. This chapter introduces the concept of namespaces as well as each of the table, field, and data types described in this guide.

In this chapter, you'll learn about:

Namespaces **131**
Table types **133**
Historical tables **134**
Vector data **138**
Field types **140**
Data types **143**
Invalid characters in data **145**
Accuracy of calculated results with long numbers **146**
Data Model dialog box **147**

Namespaces

The RapidResponse database includes namespace support. Namespaces are abstract containers used to identify and give context to groupings of related data model tables. This allows for the modeling of multiple data model schemas within RapidResponse, and provides a means to identify and distinguish between tables that have the same name but different purposes.

By default, the RapidResponse data model tables described in this guide belong to one of the following namespaces:

- **Core**—holds a small group of foundational tables that are used to support calculations in both standard RapidResponse tables and any custom tables added for your organization. This namespace includes tables that hold site, calendar, and currency information.
- **Mfg**—holds tables containing demand, supply, and part/item information used to support Supply Chain Management (SCM) and Sales and Operations Planning (S&OP) resources in RapidResponse.
- **MLService**—holds tables and fields used to support Machine Learning services in RapidResponse.
- **ProcOrch**—holds tables containing information about process instances and supports process orchestration in RapidResponse.

- **ProjMgmt**—holds tables containing project, tasks, and resource information used to support Integrated Project Management resources in RapidResponse.
- **Solutions**—holds tables and fields used to support RapidResponse applications such as Master Production Scheduling and Order Fulfillment.
- **SupCollab**—holds fields used to support data automation for supplier collaboration processes and to communicate supplier and buyer commitments.
- **Validation**—holds tables and fields used to perform input validation from an SAP data source.

If you are using a data integration server with the Integration Toolkit, additional namespaces are available. A namespace is created for each source system that data is brought in for.

Each namespace created specifically for the data integration server has an origin of 'Data Integration', and should not be migrated to the planning server. Data in these namespaces is automatically managed by the Integration Toolkit and cannot be modified. For more information, see the *RapidResponse Data Integration Guide*.

Because all of the namespaces included with RapidResponse are read-only, new tables and fields cannot be defined within them. However, your organization can create new namespaces for this purpose.

For example, you might create multiple namespaces to hold and identify groups of custom tables being added in different functional areas of your business, or you might create a single generic namespace to hold all of your company's custom tables.

Similarly, namespaces can also be used to hold custom fields added to tables in other namespaces (typically, tables in standard, non-editable RapidResponse namespaces). For example, if you wanted to add a field named Margin to the standard IndependentDemand table which is contained in the Mfg namespace, that field would need to be defined in a custom namespace. This ensures that if a field named Margin were added to the standard IndependentDemand table in a subsequent RapidResponse release, it would be distinguishable from your Margin field by namespace.

When working with and selecting tables in the RapidResponse interface, the table name is sometimes displayed along with its namespace. This typically occurs when the namespace is required to distinguish between identically named tables. In such cases, you might see table names presented in the format *TableName (Namespace)*; for example, *Customer(Mfg)* and *Customer(Xyz)*.

Similarly, in the reference section of this guide, the description of each table and its fields identifies the table's namespace in the format `TableName(namespace)`; for example `Task(ProjMgmt) fields`.

Also when defining RapidResponse resources such as workbooks, fields included in expressions sometimes need to be specified along with their namespace in the format `Namespace::Fieldname` (for example, `Mfg::Suppliers)`. This is required when using fields defined in a namespace other than the one the table itself belongs to, and when there are multiple fields of the same name defined on a given table.

Similarly, in the reference section of this guide, fields on a table that reference tables in namespaces other than the one the table itself is contained in are noted in the format `Namespace::TableName` (for example, `Core::Site`).

Notes:

- If your RapidResponse installation has ever upgraded from a Version earlier than 11.0, a User namespace is also available and contains all custom tables and fields added prior to upgrading to Version 11.0. These custom tables and fields all begin with a prefix of "U_".

- For information about working with namespaces when authoring RapidResponse resources, see the *RapidResponse Resource Authoring Guide*. For information about creating and maintaining namespaces, see the *RapidResponse Data Integration Guide*.
- If you integrate data using the Integration Toolkit, namespaces specific to the data integration server are also available. These include namespaces that are typically named after your enterprise data sources and contain tables that represent data you are mapping from the enterprise data source. These namespaces exist only on the data integration server and should not be migrated to the planning server.

Table types

The following discusses the table types in the RapidResponse data model.

Table 3.1: RapidResponse table types

Type	Description
Input	Contain the fields that hold data imported from your enterprise data source. These fields are used as input by RapidResponse when performing its algorithmic calculations, and are editable. However, some input tables also contain calculated fields which are part of the record structure, but are generated by RapidResponse calculations and are not editable. An example of an input table is the **ScheduledReceipt** table. This table contains any supply for a single part that has an assigned due date (for example, work orders and purchased orders). For more information, see "Input tables" on page 171.
Calculated	Store data produced by RapidResponse calculations. Calculated tables can be classified as either Planning or Consolidated. A planning calculated table is populated with the result of calculations performed by RapidResponse. For example, the PlannedOrder table stores recommended supply orders generated by RapidResponse calculations. A consolidated planning table is made up of a union of multiple tables or the presentation of a multi-stage summarization and filtering process. For example, the Activity table consolidates data from supply, demand, and on hand tables into a single place. For more information, see "Calculated tables" on page 937.
Control	Control tables contain the rules and values that determines how RapidResponse interprets the data stored in input tables, and hence the results shown in calculated tables. The data in control tables is set up during a RapidResponse implementation to represent the types of data, codes, and processing rules of the enterprise data source. These rules are modifiable on a per scenario basis which allows for scenarios that analyze the impact of a global policy change (for example, a change in safety stock rules). For more information, see "Control tables" on page 669.
Notes	Used to store the data entered in Notes fields associated with the EngineeringChange, IndependentDemand, ScheduledReceipt, SupplyAllocation, SupplyOrder, and Task tables. Notes tables are automatically created when a custom Notes field is added to a table. For more information, see "Notes tables" on page 1269.
System	System tables hold information used internally by RapidResponse, such as user information, system settings, or any other information that is not directly used in calculations or imported from an enterprise system. Most system workbooks are based on system tables. You cannot create or modify worksheets based on system tables except for presentation attributes such as column headers, sorting, grouping, worksheet help, and fonts. Only system tables and fields that are referenced from other tables are described in this document. For more information, see "Referenced system tables" on page 1275.

Historical tables

The RapidResponse data model includes tables for storing and analyzing historical data. Historical tables in RapidResponse are available for both historical demand data and historical supply data, as well as to archive key performance indicator data values and currency conversion rates.

Editing data in historical input tables

For all general RapidResponse users, historical input tables are read-only and cannot be edited. This is because historical tables are intended strictly for analysis purposes and not for simulations.

However, as a RapidResponse administrator, you have the ability to edit historical data to reflect other data changes in RapidResponse. You might need to move data from input demand or supply tables to historical demand or supply tables. For example, certain forecasts imported into the IndependentDemand table might be edited by RapidResponse users and, when these edited forecast values are accepted, the new values should be copied to the historical demand tables. Also, bucketed demands for components may need to be copied from the Demand tables to the historical demand tables in order to be used in waterfall reports. Similarly, bucketed open and planned supply needs to be copied from the Supply table to the historical supply tables in order to be used in waterfall reports.

Data updates and historical data

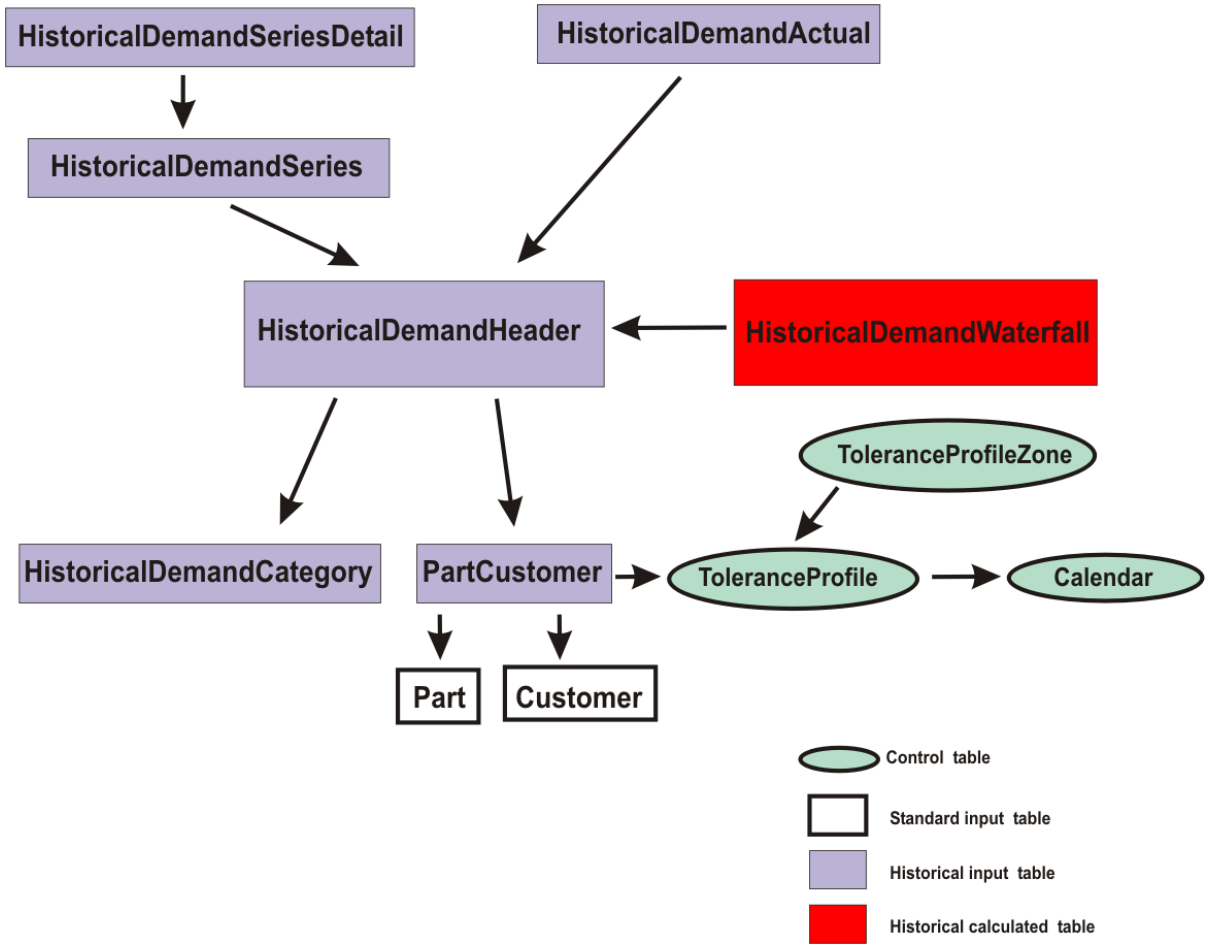
Historical data supplied in data updates can insert records into the historical tables. However, depending on your company's historical data requirements, you can configure the historical tables to also modify or delete historical records, similarly to other RapidResponse tables. To modify these tables, you must be a data or system administrator with permission to modify data integrations. For more information, see the *RapidResponse Data Integration Guide*.

During a data update, only new data should be provided for historical tables as the existing data is not deleted during the update. Instead, a RapidResponse command can be used to delete data from the historical tables as required. As well, the initial bulk import of historical data must be performed as part of a data update operation using the DataUpdate command. For detailed information about managing data in historical tables, see "Managing historical data" in the *RapidResponse Data Integration Guide*.

Historical demand tables

Historical demand data includes details of historical forecasts as well as actual shipments to customers or distribution centers. As shown in the following illustration, data must be provided for two main input tables; **HistoricalDemandSeriesDetail** and **HistoricalDemandActual**.

Figure 3.1 Historical demand tables

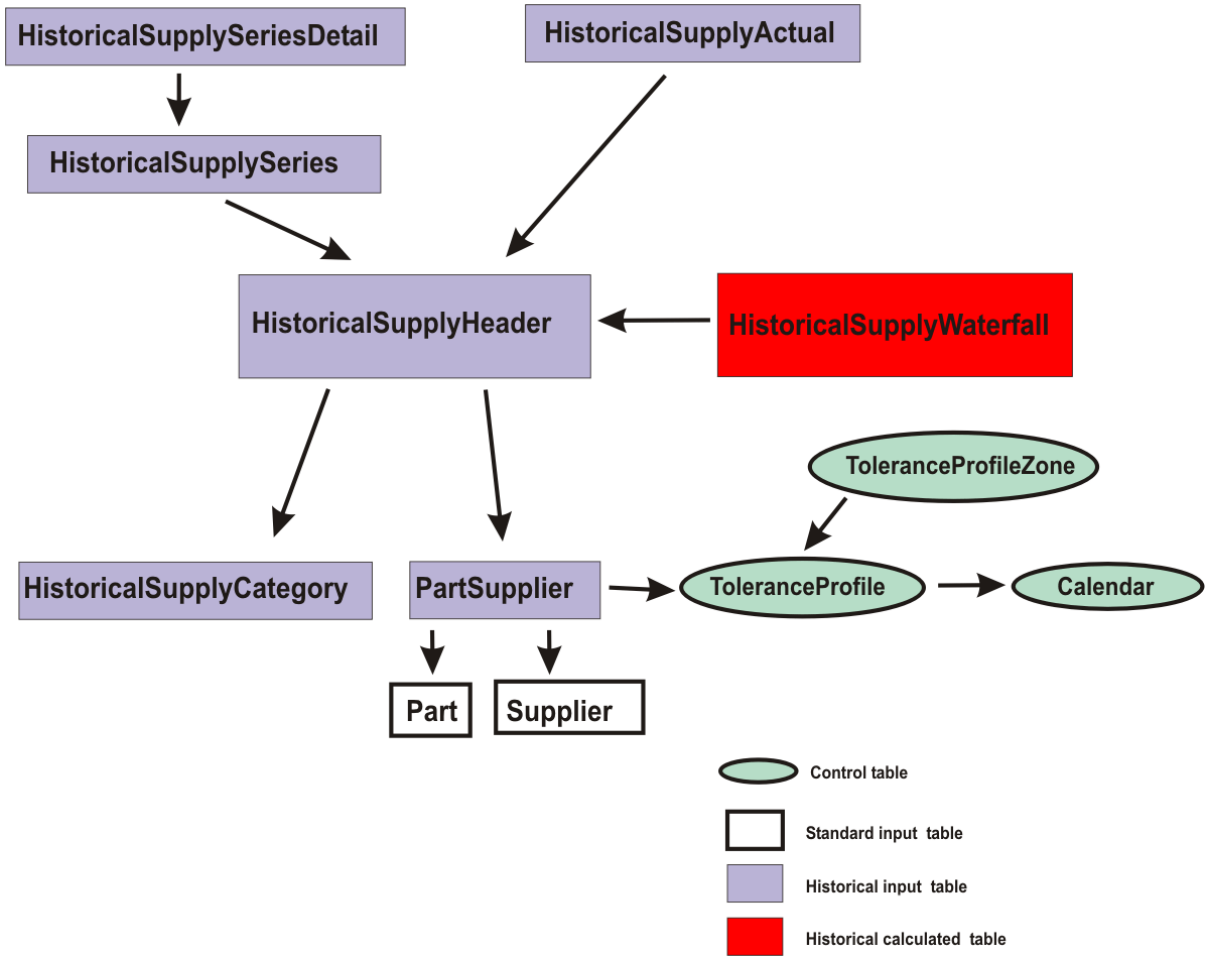


The **HistoricalDemandSeriesDetail** table holds the details of each historical demand series (forecast data), and **HistoricalDemandActual** holds details of actual shipments of parts (from fulfillment sites to customer, manufacturing sites to distribution centers, and so on). RapidResponse can then auto-create the necessary references to other related input tables. As well, a RapidResponse analytic populates the calculated **HistoricalDemandWaterfall** table. This table consolidates the historical forecast and actual demand data into a single table useful for analyzing forecast accuracy and creating waterfall reports.

Historical supply tables

Historical supply data includes details of historical supply series as well as actual orders placed with suppliers. As shown in the following illustration, data must be provided for two main input tables; **HistoricalSupplySeriesDetail** and **HistoricalSupplyActual**.

Figure 3.2 Historical supply tables



The **HistoricalSupplySeriesDetail** table holds the details of each historical supply series, and **HistoricalSupplyActual** holds details of actual orders placed with suppliers. RapidResponse can then auto-create the necessary references to other related input tables. As well, a RapidResponse analytic populates the calculated **HistoricalSupplyWaterfall** table. This table consolidates the historical supply series and actual supply data into a single table useful for creating waterfall reports.

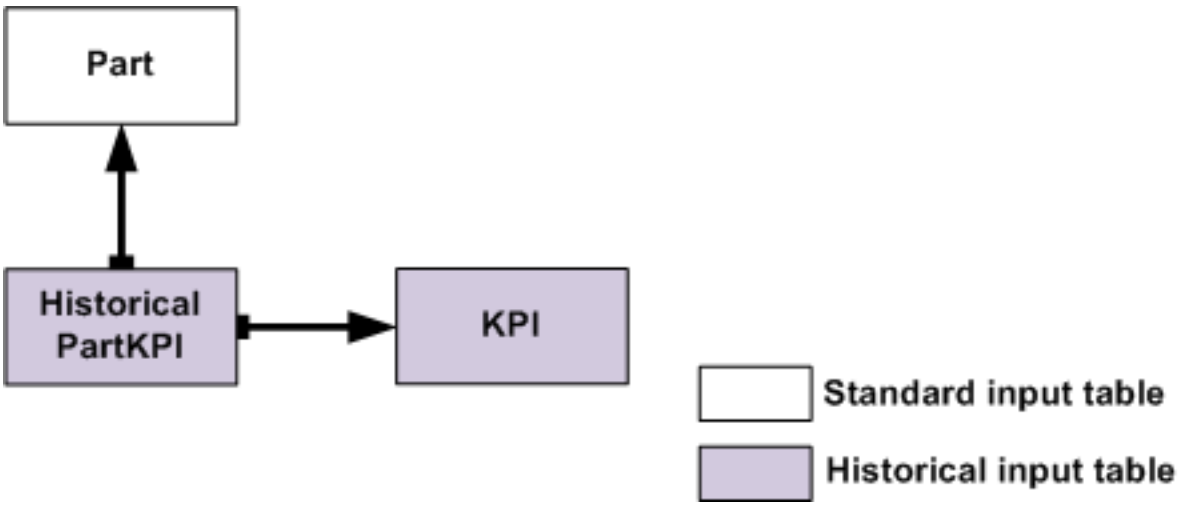
Historical KPI tables

Key performance indicators (KPI), such as orders at risk and revenue opportunities, are calculated and can be reported.

In RapidResponse, a series of automatic data modifications are configured during integration to periodically create data in the HistoricalPartKPI and KPI tables.

The **HistoricalPartKPI** table contains part-based historical key performance indicator data. Each record in this table identifies a part, key performance indicator, and the quantity value associated with that part and KPI combination on a specific date. The **KPI** table identifies each key performance indicator, and has its records automatically created based on values added to the HistoricalPartKPI table.

Figure 3.3 Historial KPI tables

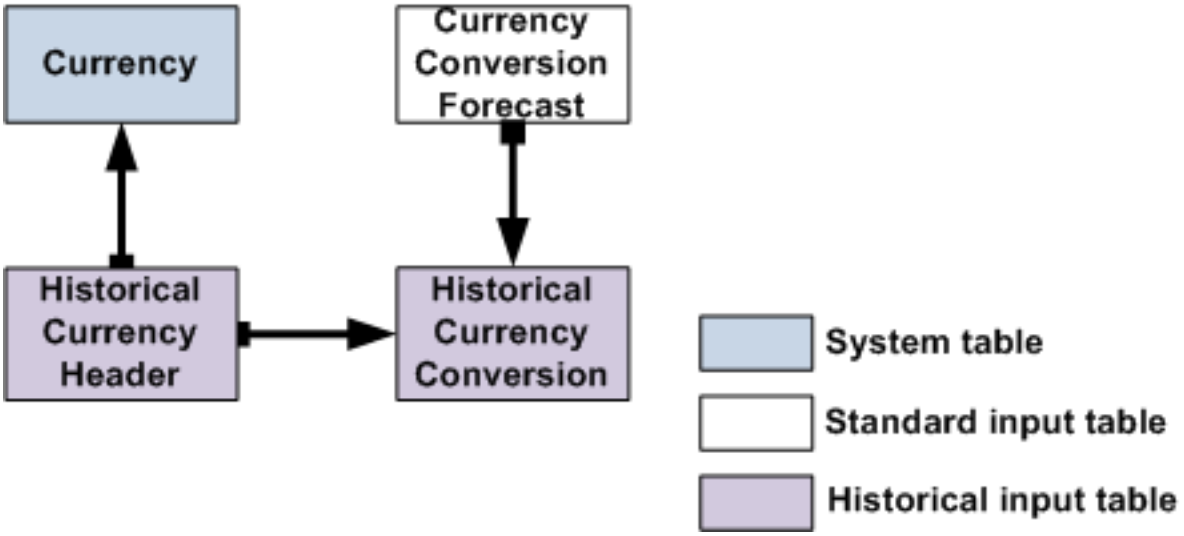


Historical currency tables

Currency conversions are represented using a conversion rate, which is multiplied or divided by a value to convert between the currency and the system's base currency. Conversion rates change over time, and in some cases you might want to use a rate calculated from a previous period. For example, you might want to report currency values for orders over the past six months.

The **HistoricalCurrencyHeader** table contains date-based currency data. Each record in this table identifies a date that historical conversion rates are available for. The **HistoricalCurrencyConversion** table contains the conversion rates for each currency as of the specified historical date. The conversion rates are taken from the **CurrencyConversionForecast** table.

Figure 3.4 Historical currency tables diagram



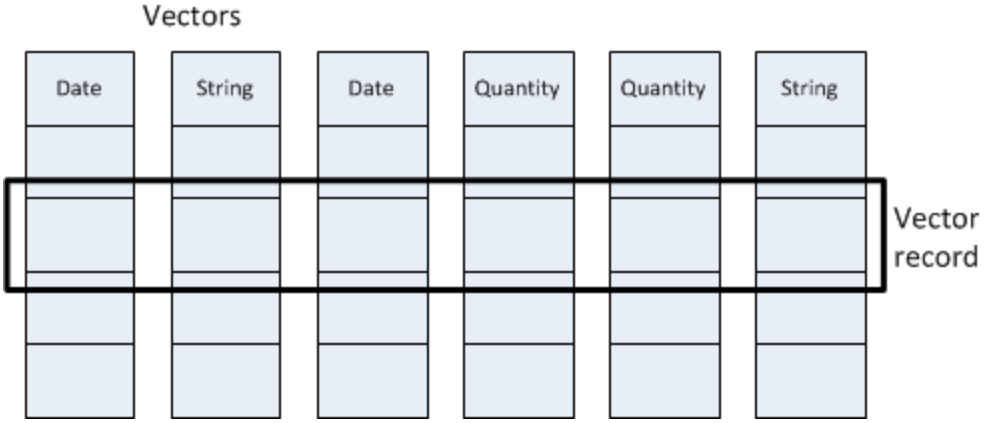
Vector data

Some input tables contain vector data, which consists of arrays of values. Each vector can contain any number of values, each with the same data type. For example, a vector of Date values can contain any number of calendar dates. Vectors are stored in memory and can be reused multiple times. If similar values are needed in multiple tables, using a vector can reduce memory consumption by storing the data one time in vectors and accessing the vectors from each table that requires them.

Vectors are assembled into a set of records under a shared owning record. These records can contain up to 96 fields. Any operation performed on the records within the vector set are performed as a single operation, regardless of how many records are in the set. For example, if an operation modifies 50 records in the vector set, all 50 modifications are performed at one time. A similar operation on a standard table would perform all 50 modifications separately.

The vectors in a set are always the same length. The set of vector values at the same position in each vector in the set represent a vector record.

Figure 3.5 Vector records



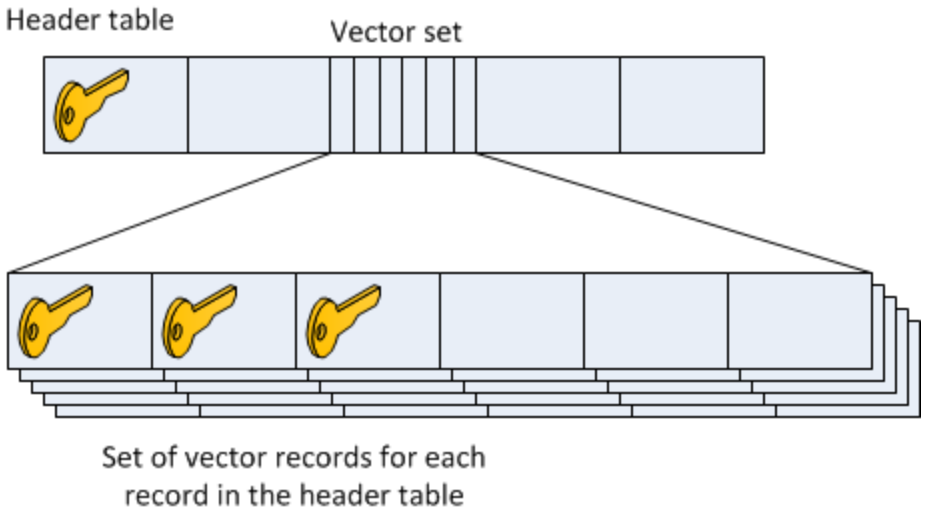
Vector records exist only in the context of the vector set field, which is represented as a field on the table that owns the vector set. Each record in the owning table has its own set of vector records, which can be constructed from the same individual vectors as required.

Vector sets are represented as tables that contain the individual vectors that comprise the required data as fields. These fields must uniquely identify values within the vector. Vectors have the same key field requirements as input tables, however, the vector set contains a reference to the owning table, called the owning reference, which is also used as a key field. For more information about key fields, see "Key fields" on page 141.

On the owning table, the vector set field contains an identifier that defines the vectors that are used in the set. This identifier consumes the same memory as a standard field, and is the only value stored in the table for the entire vector set, regardless of how many values are present in the vectors. For more information about the memory requirements for fields, see "Data types" on page 143.

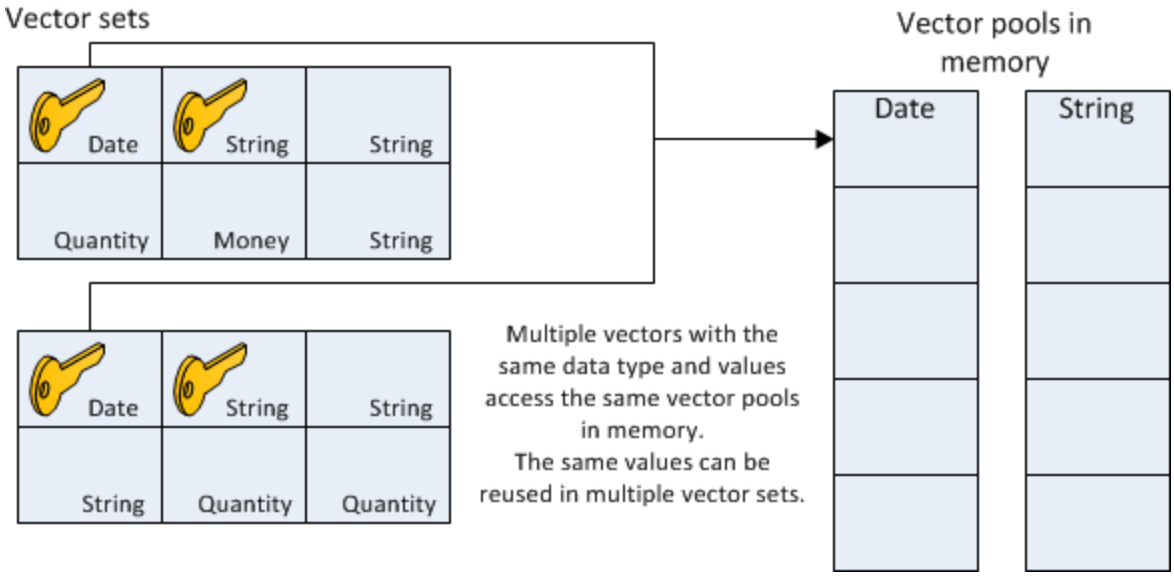
A vector set is typically used to optimize or replace a header and detail table structure where many detail records are created for each header record. The vector set contains the detailed records for each record in the header table.

Figure 3.6 Relationship between a vector set and its owning table



The groupings of vector key fields are used to sort the records in the vector set, and affect how values are read from the vector pools. For example, for a vector set that contains a Date field and a Type field as its keys, the vector set is sorted by date and then by type. The values that line up with those date and type values in each vector are retrieved from the vector pools, and can be reused for other vector sets that use the same key fields.

Figure 3.7 Reusing vector pool values



Because the vector set's key values are stored in memory, they can be accessed by any vector set with the same groupings of key fields. This improves performance and reduces memory consumption because the key values are not stored as records in the vector table. This requires less memory than would be needed to store the equivalent number of detailed records in each individual vector set.

Vector data is brought in with other data during a data update, and each vector that is loaded is available to be reused by another vector set. For example, a Date vector can be populated with monthly values, and then that vector can used in any other vector set that requires the monthly values. If data in a vector set is modified, the vector is replaced with the new values. If values are inserted into the vector set, they are inserted where required in the sort order of the vector sets.

Field types

Each field in a RapidResponse table belongs to one of the following field types:

Table 3.2: RapidResponse field types

Type	Description
Base field	A base field represents an individual piece of information contained in a table. For example, in the ScheduledReceipt table, Due Date is a base field as it represents the specific due date associated with the order. Each base field contains either data imported from an enterprise data source or data generated by RapidResponse analytic calculation.
Reference field	A reference field is a link to another field in a table with a one-to-one relationship. For example, on the ScheduledReceipt table, Part is a reference field as each scheduled receipt is for exactly one part. A reference field can typically only contain values stored in the referenced table. For example, each scheduled receipt must be for a part already defined in the Part table. However, a small number of reference fields in the standard RapidResponse data are nullable, and all custom reference fields (unless designated as part of their table's key fields) can be configured to be nullable. For more information, see "Nullable references" on page 142.

Table 3.2: RapidResponse field types

Type	Description
Set field	A set field is a link between two tables where there is a one-to-many relationship. It represents a set of summary records on the other table. For example, on the Part table, ScheduledReceipts is a set field as each part can have more than one scheduled receipt. As a result, each part has a set of records containing summarized information about all its receipts.
Vector set field	A vector set field represents a vector table, which contains the series of detailed data for each record in a header table. Vector fields are accessed similarly to set fields.

Key fields

Some RapidResponse fields also act as key fields. The key fields for a table consist of one or more fields that form a unique combination of values in a table. Any combination of base or reference fields can be used as keys, and a table can contain up to eight key fields. Fields of the following data types can be used as key fields:

- String
- Reference
- Date
- Time
- DateTime
- Boolean
- Integer

For more information about these data types, see "Data types" on page 143.

The key fields for vector sets also include a reference to the table that owns the vector, which is one of the table's key fields. This field cannot be removed and must be a key field.

Only tables where record uniqueness is required have key fields. For example, the Name and Site fields make up the keys of the Part table as each part name and site combination in the RapidResponse data model must be unique. Conversely, the Capacity table does not have key fields, and records added to this table are not required to be unique.

A small number of tables in the RapidResponse data model that do not contain key fields by default can be configured with key fields. These tables are:

- Allocation
- BillOfMaterial
- OnHand
- Operation

These tables, and the standard tables that contain vector data, are the only standard RapidResponse tables you can add or modify key fields in.

Nullable references

Reference fields are used to point to actual values stored in other tables. Many reference fields on input tables are mandatory and must point to a valid value in the referenced table. Reference fields that must point to a valid value on another table include all reference fields that make up part of the table's key, most control table references, and a small number of other input table references that point to required data.

For example, on the **ScheduledReceipt** table, the reference to the **SupplyOrder** is required as it is part of the table's key, the reference to the **SupplyStatus** table is required as it contains settings used in determining how each receipt is processed by RapidResponse analytics, and the reference to the **Part** table is required to identify the actual part the receipt is for.

However, many other input reference fields that do not contain mandatory data can be left empty or "Null". For example, on the **ScheduledReceipt** table the reference to the Shipment table can be left Null as not all receipts will be part of a shipment, and the AllotmentOverride reference can be left null as allotment overrides are generally only used on an exception basis.

When a reference field is set as nullable, that field value can be left empty (Null) and no defaults are applied if a reference value is provided. Nullable reference fields can be utilized in cases where a given reference field is not always expected to be populated. If a nullable reference field is followed from a record where no reference value was provided, only an empty reference is returned and any base fields on the referenced record return their data type's default value (for example, 0.0 for Quantity, Undefined for Date, and so on).

Nullable references also differ from standard reference fields in how they handle record deletions in a referenced table. If a reference field is not nullable and a given record is deleted from the referenced table, then all records that reference that deleted value are also deleted. This known as a cascading deletion. Nullable reference fields can, optionally, be configured so that instead of their records being deleted when a referenced value is deleted, the reference is set to Null instead.

Notes:

- All nullable reference fields in standard input tables are identified as such in the Input tables chapter of this guide. This is noted after the name of the referenced table in the field description. For example, **Referenced table**: BuyerCode (Nullable).
- For information about designating custom reference fields as nullable, or to find out which of your company's custom reference fields are nullable, see the *RapidResponse Data Integration Guide* or talk to your RapidResponse administrator.
- If writing query expressions that use nullable reference fields, you might need to adjust your logic to account for the possibility of a null reference. For this purpose, the IsNull function can be used to detect records on which a reference is actually null as described in the *RapidResponse Resource Authoring Guide*.
- Reference fields in a vector table or reference fields that are used as key fields cannot be made nullable.

Nullable references in versions prior to 10.1

The concept of a nullable reference was introduced in RapidResponse version 10.1. In subsequent releases, many new non-key reference fields have been designated as nullable.

Additionally in Version 10.1., a number of existing reference fields were set as Nullable. However, for users upgrading from versions prior to 10.1., they were designated as non-nullable during the upgrade process.

Thus, the reference fields listed in the following table are generally nullable in all new installations of RapidResponse from version 10.1 and later. However, for installations that have ever been upgraded from a version earlier than 10.1, these references are not nullable.

Table 3.3: Nullable references in new RapidResponse installations 10.1 and later (only)

Table	Namespace	Reference Field	Referenced Table
Allocation	Mfg	Shop	Shop
BillOfMaterial	Mfg	EffectiveInEC	EngineeringChange
BillOfMaterial	Mfg	EffectiveOutEC	EngineeringChange
BillOfMaterial	Mfg	Shop	Shop
BillOfMaterial	Mfg	SubstituteGroup	SubstituteGroup
Customer	Mfg	CustomerGroup	CustomerGroup
IndependentDemand	Mfg	ShipSet	ShipSet
PartCustomer	Mfg	ToleranceProfile	ToleranceProfile
PartSupplier	Mfg	ToleranceProfile	ToleranceProfile
ScheduledReceipt	Mfg	Shipment	Shipment
Shipment	Mfg	Carrier	Carrier
Supplier	Mfg	SupplierGroup	SupplierGroup
WorkCenter	Mfg	Department	Department

Data types

Each field in a RapidResponse table supports a particular data type. The data type determines the type of data a field can contain, and subsequently the types of queries that can be performed on it. The following table outlines the data types supported in the RapidResponse data model.

Table 3.4: Data types

Type	Description	Memory allocation
Boolean	Either a **Y** (Yes or True) or **N** (No or False) value. This data type is treated similarly to the String data type in RapidResponse, except that it accepts only two possible values and consumes less memory per record.	2 bytes
Date	A date value. The valid date range is from January 2, 1970 to December 31, 2037. Undefined, Future, and Past values are also permitted, and represent dates outside the valid range.	4 bytes
DateTime	A date and time value with the same range as the Date type. A time zone value can be added; if not, RapidResponse uses the client's local time zone. The value can be Past, Future, or Undefined.	4 bytes

Table 3.4: Data types

Type	Description	Memory allocation
Integer	Whole number with an optional preceding minus sign. Valid range is -2,147,483,648 to 2,147,483,647.	4 bytes
Money	A real number (whole and decimal fraction numbers) representing a monetary value. All Money values have an associated currency, and can be converted to other currencies or represented as a raw (unconverted) value. Any money value, consisting of a number and an associated currency. The valid range of values is -1.79769313486231**E**308 to 1.79769313486231**E**308. Values with more than 308 decimal places (**E**-308) are interpreted as zero values. This data type uses a double-precision floating number. You can also use exponential notation to express this number.	8 bytes
Note	Stores notes data. If you add a Note field to a custom table, RapidResponse creates a corresponding Notes table (the Notes table takes the name of the custom table and adds the suffix _**Notes**). Each time a note is added to the Note field in the custom table, a notes record is created and stored in the corresponding Notes table. For more information, see "Notes tables" on page 1269.	4 bytes for each record in a RapidResponse table that has the Note field. Per note instance: 20 bytes + size of UserId who created the note + size of Note text.
Quantity	A real number (whole and decimal fraction numbers) with an optional preceding minus sign. The valid range of values is -1.79769313486231**E**308 to 1.79769313486231**E**308. Values with more than 308 decimal places (**E**-308) are interpreted as zero values. This data type is also known as a double-precision floating number. You can also use exponential notation to express this number.	8 bytes
QuantitySingle	A real number (whole and decimal fraction numbers) with an optional preceding minus sign. The valid range of values is -3.402823**E**38 to 3.402823**E**38. Values with more than 45 decimal places (**E**-45) are interpreted as zero values. This data type is also known as a single-precision floating number. You can also use exponential notation to express this number. **Note:** This data type can only be used by custom fields.	4 bytes
Reference	A reference to custom or standard RapidResponse table. The reference typically must contain a value for each of the referenced table's key fields.	8 bytes
String	A series of alphanumeric characters.	8 bytes or greater depending on the length of the string.
Time	A time value entered in the format shown above. It can be an explicit time using the 24-hour format, hh:mm:ss.	4 bytes
Vector Set	A vector table that contains the date-phased detail records for a header record. Each unique value in the vector's key fields is stored once within the vector records and referenced from all other records where that value is used.	8 bytes

Notes:

- Non-printing control characters, such as line breaks or null characters, are not supported in RapidResponse. For specific examples of data types that are not supported, see the *RapidResponse Administration Guide*.
- For information about the rules and syntax required when including the different data types in queries, see the *RapidResponse Resource Authoring Guide*..
- Some string fields described in this guide are also marked as Enum. These are fields which contain or return values from a list of predefined strings. For example, many of the fields containing processing options in control tables are identified as Enum.

Invalid characters in data

The table below lists the control and non-printable characters that RapidResponse reads as invalid. Avoid the use of these characters to prevent data errors.

Character	ASCII Character	Unicode Value	Description
0, Nul, \0, ^@	U+0000	U+2400	Null
SOH, ^A	U+0001	U+2401	Start of Header
STX, ^B	U+0002	U+2402	Start of text
ETX, ^C	U+0003	U+2403	End of text
EOT, ^D	U+0004	U+2404	End of transmission
ENQ, ^E	U+0005	U+2405	Enquiry
ACK, ^F	U+0006	U+2406	Acknowledge
BEL, \a, ^G	U+0007	U+2407	Bell
BS, \b, ^H	U+0008	U+2408	Backspace
FF, \f, ^L	U+000C	U+240C	Form feed
SO, ^N	U+000E	U+240E	Shift out
SI, ^O	U+000F	U+2401	Shift in
SUB, EOF, ^Z	U+001A	U+241A	Control-Z
ESC, \e, GCC only, ^[	U+001B	U+241B	ESC/Escape
SP, SUB	U+0020	U+2420	Space
ƀ	U+180B	U+2422	Blank
NL, \n	U+0085	U+2424	Next line

Accuracy of calculated results with long numbers

RapidResponse stores and computes Quantity data (whole and decimal fraction numbers) as floating-point numbers. Simply put, "floating point" refers to the number of digits before and after the decimal point as not being fixed; that is, the decimal point "floats". Some software applications fix the number of digits before and after the decimal point. However, most software applications use floating-point numbers because they can represent a larger range of numbers.

Floating-point numbers use two main standards for accuracy: single-precision and double-precision. With single precision numbers, the first seven significant digits are considered accurate. The first non-zero digit is considered the first significant digit. In the following examples, the bolded digits are considered significant.

- **123.4567**89
- **1,234.567**89
- **1,234,567**,890
- 0.**1234567**89
- 0.000**1234567**89

Double-precision numbers are similar to single-precision numbers except the first 15 significant digits are considered accurate.

Software applications, including RapidResponse, internally store floating-point numbers as binary numbers. This means they might not have a precise representation. For example, take the number 1.23456789. If this value was stored as a single-precision number, it would be 1.2345679. If the single-precision value was then stored as, or converted to, a double-precision number, it would be 1.2345678806304932.

When working with floating-point numbers, especially single-precision numbers that use many significant numbers, you might encounter some unexpected results from certain calculations. For example, if you are summing many numbers which are stored as single precision and use more than seven significant digits, the result might include unexpected values in the position of the sixth, seventh, and eighth significant digits.

RapidResponse can store Quantity input data as single-precision numbers using the QuantitySingle data type, or as double-precision numbers using the Quantity data type. However, for single-precision numbers, the RapidResponse analytics convert the data into double-precision numbers. In some cases this can produce unexpected results if using data that includes more than seven significant digits. For example, a series of numbers such as 1,234.56789 used in analytic calculations might produce unexpected results in the last three decimal places, especially as numbers are multiplied and compounded.

Storing Quantity input data as double-precision numbers means analytic calculations of data with more than seven significant digits will be more accurate. However, the Quantity data type uses 8 bytes of memory for each field and the QuantitySingle uses 4 bytes. Storing Quantity values as double-precision numbers might result in increased memory consumption. When creating custom quantity fields, you can choose whether to use the Quantity or the QuantitySingle data type.

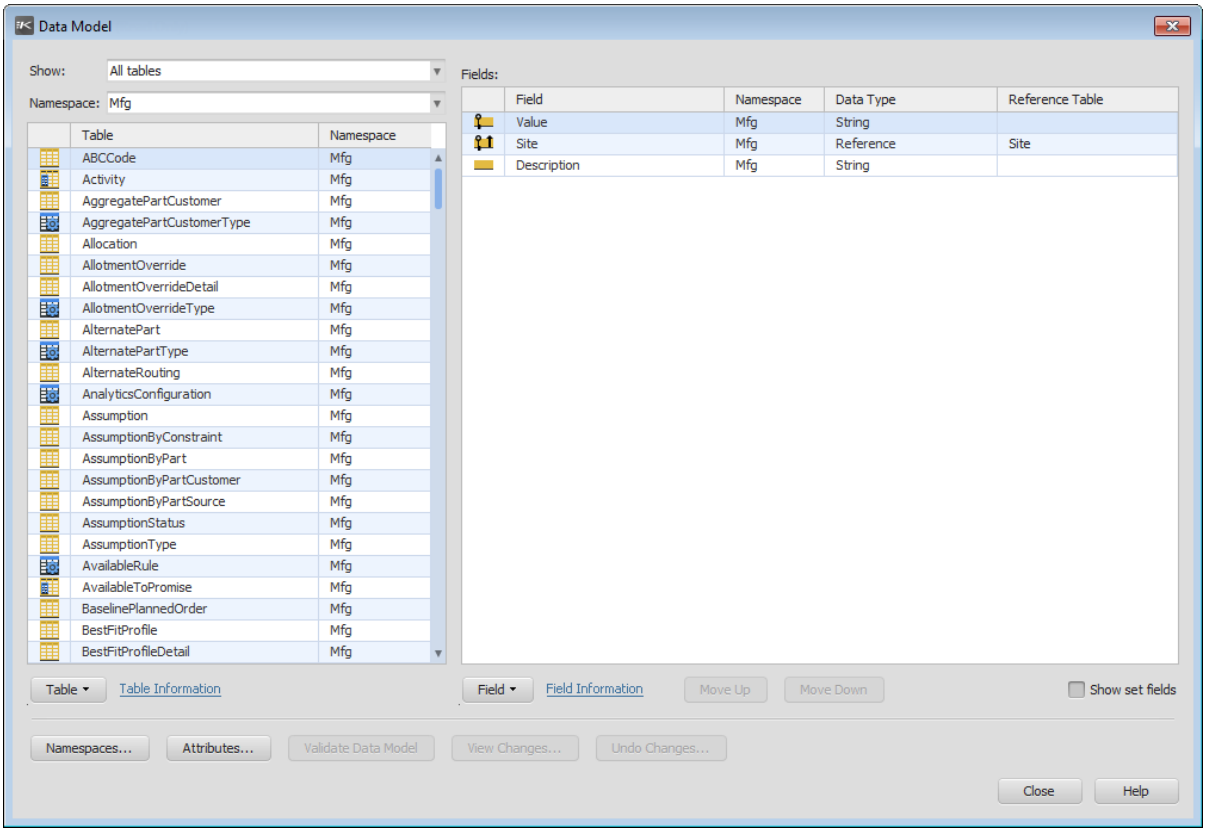
For more information about floating-point numbers and IEEE (Institute of Electrical and Electronics Engineers, Inc.) standards, see https://en.wikipedia.org/wiki/IEEE_754.

Data Model dialog box

System and data administrators can customize the RapidResponse data model using the Data Model dialog box. These administrators need to be granted permission to modify data integration settings.

This dialog box, as shown in the following illustration, provides information about all the tables and fields in the RapidResponse data model. It also provides data model reference information.

General users and authors can be given permission to open the Data Model dialog box. Access to the dialog box might help general users troubleshoot issues. It can also provide authors a better understanding of the data model to improve the resource authoring process.



To open the Data Model dialog box

1. Do one of the following:
   - **System and data administrators**—In the **Administration** pane, under **Data**, click **Data Model**.
   - **All other users**—On the **View** menu, click **Data Model**.
2. From the **Show** list, select one of the following:
   - **All tables**—displays all standard and custom RapidResponse input and control tables, as well as all standard RapidResponse calculated tables.
   - **Calculated tables—**displays all standard RapidResponse calculated tables.
   - **Control tables**—displays all standard and custom RapidResponse control tables.

- **Frequently used tables**—displays all tables that have been designated as frequently used. These allow you to define a group of tables you expect to frequently work with or modify.
- **Input tables**—displays all standard and custom RapidResponse input tables.
- **Tables with fields in namespace**—displays only tables with fields that belong to the namespace selected from the **Namespace** list. For example, for a namespace created by your company, this might include custom input tables you've created as well as standard RapidResponse input tables in the Mfg namespace that you've added fields to.

3. Optionally, from the **Namespace** list, select the namespace whose tables you want to view. The list of tables shown is then narrowed to those matching the selection in the **Show** list which also belong to the selected namespace.
4. If you want to display the set fields on each table, select the **Show set fields** check box.

To view table and field reference information

- Do one of the following:
  - To view help for a given table, select it from the **Table** box, and then click **Table Information**.
  - To view help for a given field, select it from the **Fields** box, and then click **Field Information**.

  For standard RapidResponse tables and field, the *RapidResponse Analytic and Data Model Guide* opens and displays information about the selected table or field. For custom tables and fields, a dialog box opens displaying properties and information provided by the user who added the table or field.

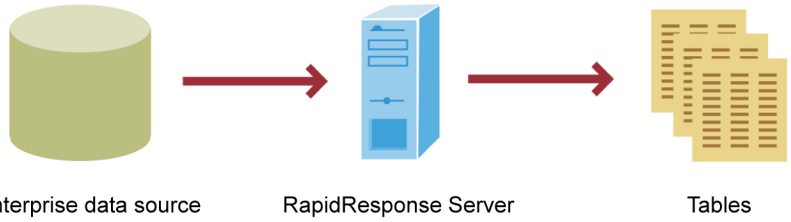
Tips:
- For more information about the Data Model dialog box, click the **Help** button.
- You can also access reference information for a given table by right-clicking it in the **Table** box and then clicking **Table Information.** Similarly, you can access reference information for a given table by right-clicking it in the **Field** box and then clicking **Field Information**.

CHAPTER 4: Introduction to the RapidResponse data model

A typical manufacturing system consists of items that need to be built or manufactured. Whether it be an aircraft or a bicycle, companies need to keep track of all the parts needed to build a product, who wants to buy the part, and how to replenish the supply once demand for the part increases. In order to manage the data required to keep track of a product and what's required to make, buy, sell, or ship it, the RapidResponse data model can be used.

With RapidResponse, data is extracted from your enterprise system and imported into the tables of the data model as shown in the following illustration. Each of these table stores information related to a specific area of your demand management or manufacturing operation such as SKUs, parts, bill of materials, shortages, customer orders, and work orders. These tables are editable which enables simulations to be performed on your data. Besides the tables which hold this imported data, the data model holds other tables which are populated by calculations performed by RapidResponse (for example, RapidResponse generates planned order recommendations). These tables are not editable, and their calculations are based on your imported data, and are configurable through processing rules.

Figure 4.1 Data imported into tables



Each table in the data model is made up of fields. Each field holds a single piece of data such as a part's number, low level code, or standard unit cost. A table also consists of records. Each record is a collection of fields that make up a single entry in your table. For example, the Part table holds information about each unique part in the data model.

In order to leverage the data in the RapidResponse database, it is important to understand how data is stored and organized. This chapter introduces you to the RapidResponse data model, how it is organized, and the basic data groupings.

In this chapter, you'll learn about:

RapidResponse data **150**
Products and structure **151**
Activity **152**

Controls 159

Note: For information about the RapidResponse calculations used to populate various tables in the data model, see "Part 2: Analytics" on page 1283.

RapidResponse data

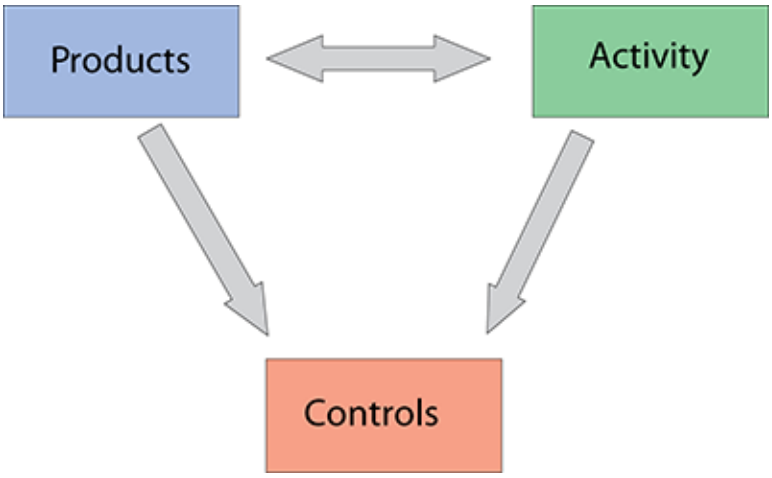
In an enterprise data system, all of the pieces of data required to manage the supply and demand for a product are linked together. The same is true for the RapidResponse Data Model. Although many different tables are used to store specific supply or demand related information, these tables are linked together where logically appropriate. For example, tables containing demand information for your parts link to the table where part information is stored.

All links between tables in the data model are bi-directional. This means that if Table X points to Table Y, then Table Y must point back to Table X. Each link from one table to another in the data model is of one of two types:

- **One-to-Many** – one record in the first table can link to many records in the second table. For example, a single part may have many customer orders.
- **Many-to-One** – many records in the first table link to one record in the second table. For example, there can be many customer orders for a single part.

Regardless of the links between them, the tables in the RapidResponse data model contain data belonging to one of three general groups as shown in the following diagram.

Figure 4.2 RapidResponse data



The following list briefly describes the three groupings of RapidResponse data discussed in this section.

- **Products and Structure**—Detailed information about all individual parts or SKUs, and shows the parts, raw materials, subassemblies, and intermediates that go into a parent assembly, as well as the quantity of each required to make an assembly. For more information, see "Products and structure" on page 151.

- **Activity**—Time-phased events that increase or decrease the inventory for a part. Typically, this involves a date and quantity associated with some activity that either consumes parts (demand) or provides parts (supply). Some activity data is imported directly from your enterprise data source and represents decisions that have already been made. (for example, open work orders). Other activity data is calculated by RapidResponse (for example, planned orders). RapidResponse also includes consolidation and pegging tables that combine or associate different levels of supply and demand activity for reporting. For more information, see "Activity" on page 152.
- **Controls—**Processing rules typically set up during the implementation of RapidResponse that determine how the different RapidResponse calculations are carried out. These are used to ensure that RapidResponse processing behavior matches that of your enterprise data source. For more information, see "Controls" on page 159.

Products and structure

At the heart of the RapidResponse data model is the **Part** table. This table lists each unique part (or SKU) that has been imported into the data model. It also holds information on each of these parts such as a part's name (or number), description, material cost, and standard unit cost.

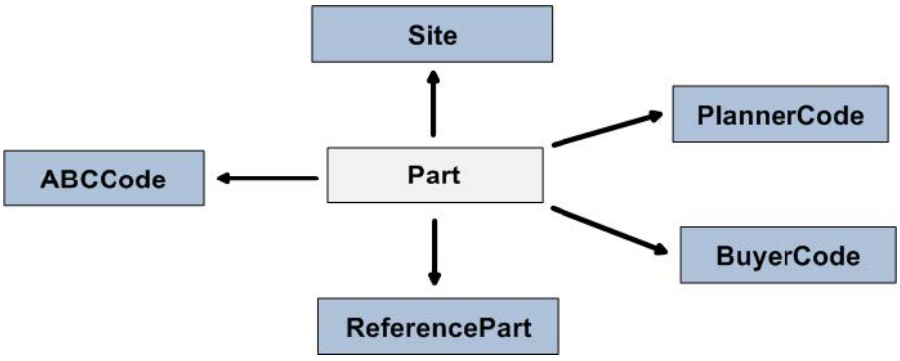
Besides the data contained directly on the Part table, the Part table links to other tables that contain part related data as shown in the following illustration. These tables allow for common part information typically associated with many different parts (such as abc codes, buyer codes, and planner codes), to be stored in just one place and then referenced from the **Part** table.

The **Part** table also has a reference to the **Site** and **ReferencePart** tables.

The **Site** table contains an entry for each unique site in the data model (for example, a particular factory or distribution center). Together a part's name and site make up its primary key and guarantee its uniqueness in the RapidResponse data model.

The **ReferencePart** table supports cross-referencing of a part's name with the name it is more commonly known by elsewhere. For example, a contract manufacturer can use the **ReferencePart** table to store the part names used and understood by a brand owner. The **ReferencePart** table might also be used by any manufacturer who wants to cross-reference two or more parts to indicate they should be treated as the same part (for example, if a new part numbering scheme is being implemented).

Figure 4.3 Part table and referenced

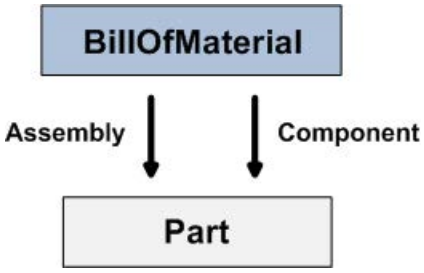


Besides a list of all parts and their attributes, RapidResponse needs to keep track of all the components required to build each part. This information is held in the **BillOfMaterial** table which references the **Part** table.

The **BillOfMaterial** table identifies parent /child relationships between assembly parts (products, assemblies and sub-assemblies), and their component parts (assemblies, sub-assemblies, and raw materials). For this reason, the **BillOfMaterial** table has 2 separate references to the **Part** table, one to define the assembly and another to define a component under that assembly. Each record in this table also contains details of the relationship between a given assembly and component. For example, the **QuantityPer** of the component required to make one unit of the assembly.

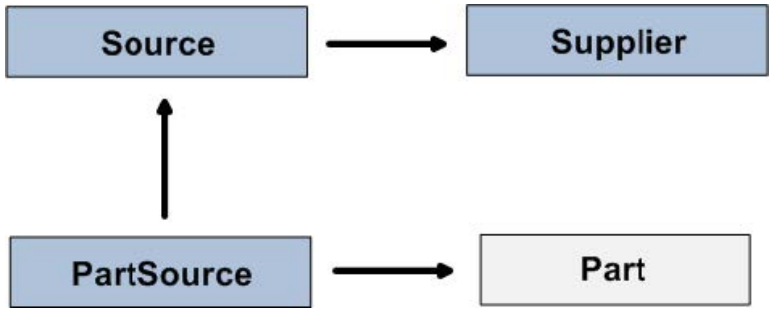
The relationship between the **Part** and **BillOfMaterial** tables is shown in the following illustration.

Figure 4.4 BillOfMaterial table defines how products are built



Another important table to understand when dealing with parts is the **PartSource** table. This table can be used to define multiple sources of supply for a single part as well as the planning attributes for that part and source relationship. Examples of multiple sources of supply for a single part includes parts supplied by more than one vendor, parts that can be made or bought, and parts that are transferred in from another internal plant. The **PartSource** table references the **Part** table and it contains information such as the lead times and costs associated with the part source. As shown in the following illustration, the **PartSource** table also references the Source table which contains delivery information and provides a reference to the **Supplier** table which identifies the part's supplier (could be either a vendor, work center, plant, and so on).

Figure 4.5 PartSource table defines multiple sources of supply for parts



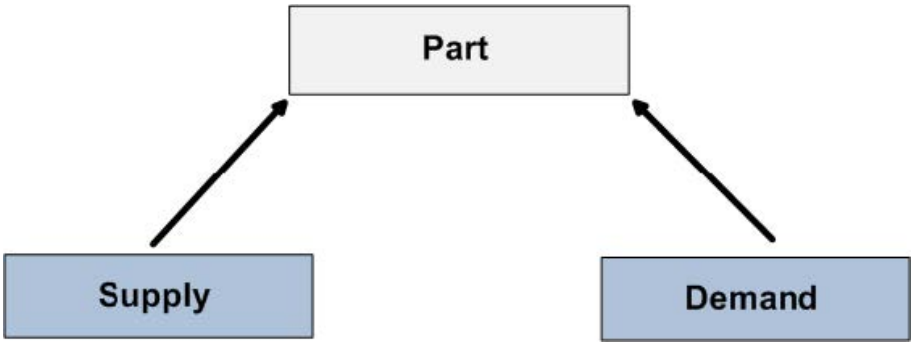
Activity

Once you have a part and all of the pieces required to build it, there has to be some activity in the system that drives demand for the part. Similarly, there must be some activity that generates or replenishes supply of the part as necessary.

Input activity

The RapidResponse data model contains tables into which you can import this supply and demand (or "activity") information.

Figure 4.6 Parts can have imported supply and demand

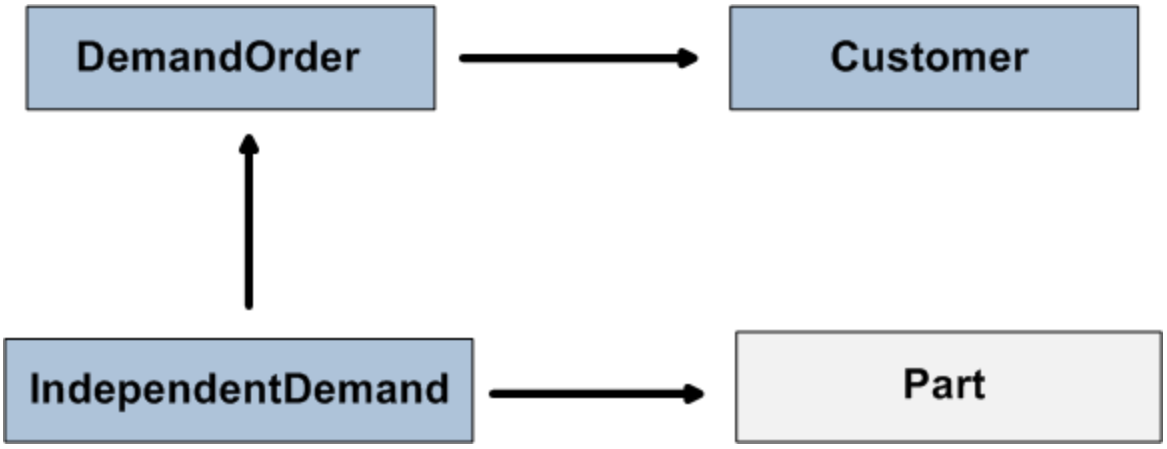


Independent demand is imported from your enterprise data source. It refers to demand not directly generated from the demand for other parts. For example, forecasted or actual customer demand for finished goods.

IndependentDemand table

The **IndependentDemand** table contains a record for each line item associated with an actual customer order or forecast. These records contain details such as due date, request date, and quantity associated with the line item. However, since a customer can place a single order that has many line items for each part, common header information for each independent demand (such as its order number or customer) is stored in the **DemandOrder** table. As shown in the following illustration, the **DemandOrder** table then links to the **Customer** table to identify the customer associated with the order.

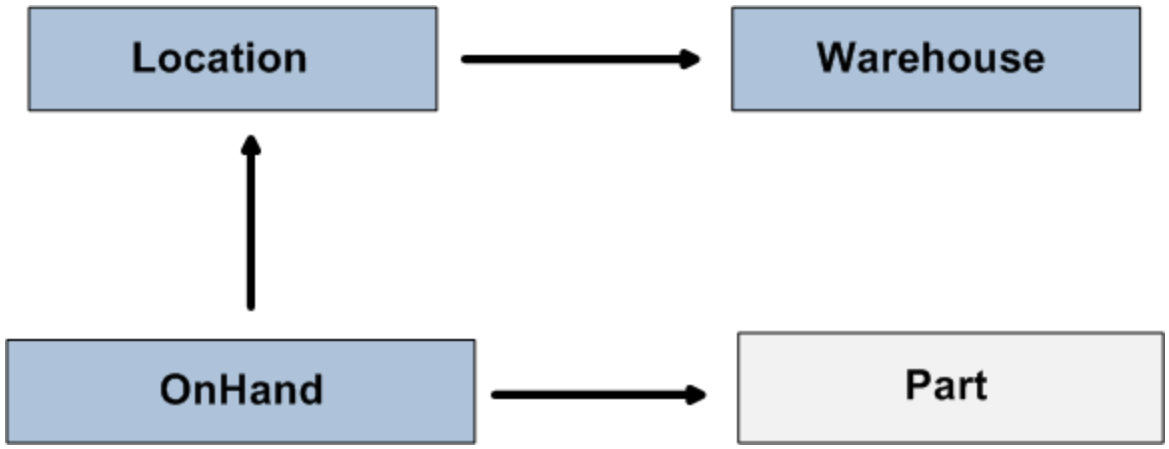
Figure 4.7 IndependentDemand table references



OnHand table

Another important table to understand when working with activity data is the **OnHand** table. This table holds the quantity of inventory for each part in a particular storage location within a warehouse. As shown in the following illustration, this table links to the **Location** table and subsequently to the **Warehouse** table which store the actual location and warehouse values respectively.

Figure 4.8 OnHand table references

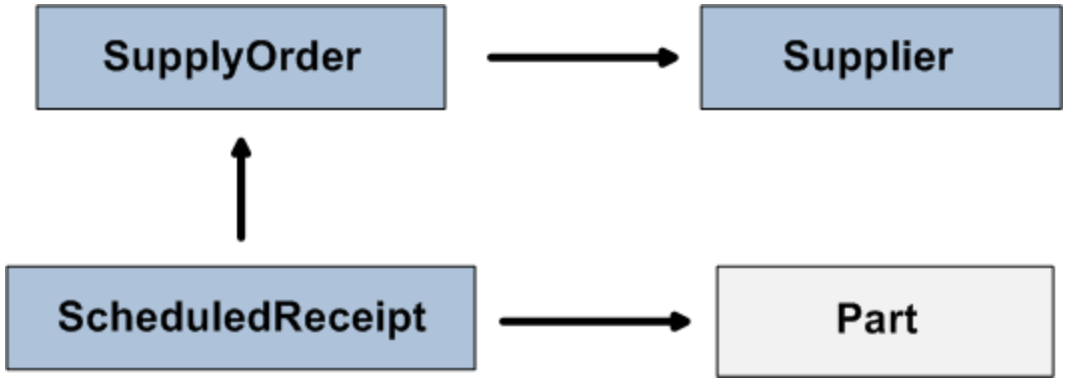


ScheduledReceipt table

Each record in the **ScheduledReceipt** table represents a single line item on a supply order. It stores firm orders with an assigned due date and quantity, such as work orders or purchase orders. These orders are imported from the enterprise data source, although **ScheduledReceipt** table also contains some fields populated by Rapid Response calculations (for example, RapidResponse calculates a rescheduled due date for each order).

As shown in the following illustration, the common header information about each order (such as the order Id and supplier) is stored in the **SupplyOrder** table which is referenced from the ScheduledReceipt table. The **SupplyOrder** table then references to the **Supplier** table which holds all suppliers defined in the system. A supplier may be a vendor for a purchase order, a work center for work orders, or a plant identifier for inter-plant orders.

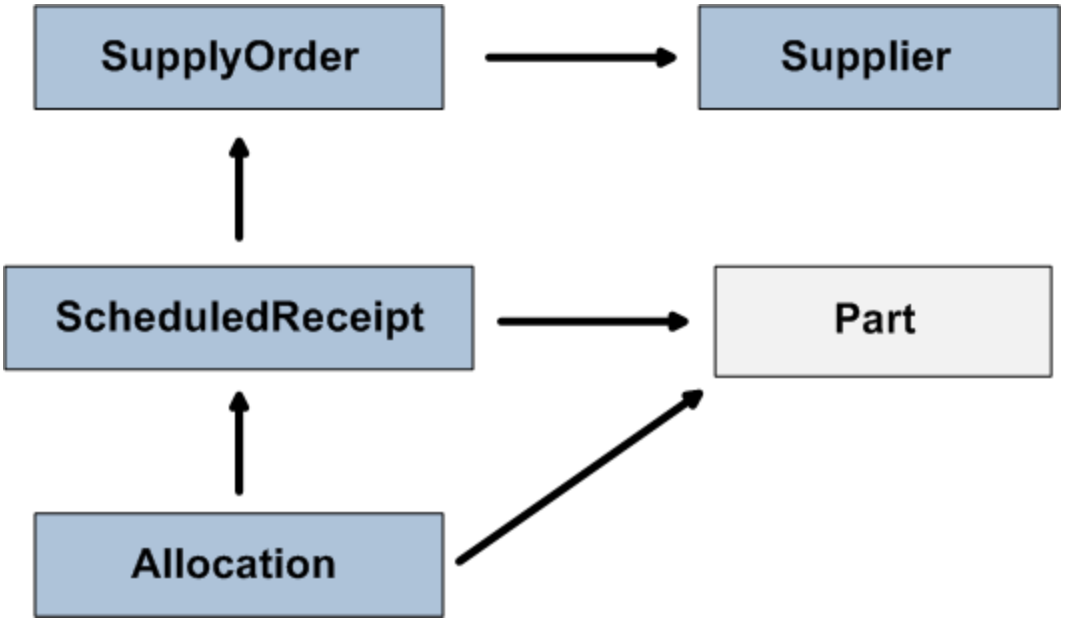
Figure 4.9 ScheduledReceipt table references



As shown in the following illustration, the **ScheduledReceipt** table is referenced by the **Allocation** table which is used to store allocations (dependent demands) generated from a work order for a make item. If

you do not import allocations from your enterprise data source, RapidResponse can calculate them for you as discussed later in this chapter. That is, the **Allocation** table is used only for those allocations imported from your enterprise data source (typically, this is done at a time close to when the order is being released to the shop floor, or when you have deviated from the standard bill of materials).

Figure 4.10 Allocation table for importing dependent demands



Calculated activity

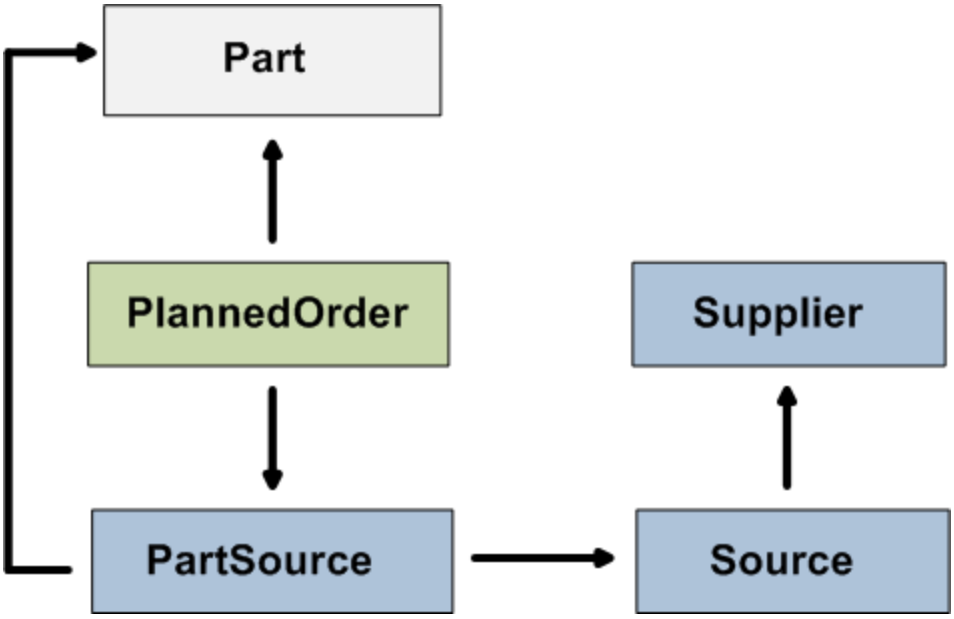
The RapidResponse data model also contains calculated tables that hold activity data not input.

PlannedOrder table

The **PlannedOrder** table is a calculated table providing recommendations for new supply orders to be generated.When there are not sufficient supply quantities in **ScheduledReceipt** or **OnHand** records to satisfy demands, RapidResponse attempts to create planned orders to satisfy demand. Unlike the **ScheduledReceipt** table, the **PlannedOrder** table does not contain data imported from the enterprise data source. Instead it contains new recommended planned orders generated by RapidResponse analytic calculations. Parameters like days supply and lot sizing can affect the size and frequency of planned orders generated as discussed later in this guide.

Each **PlannedOrder** record contains details of the planned supply such as due date, order quantity, and a reference to the part the order is for. The **PlannedOrder** table references not only the part associated with the supply, but also a part source. This is made through a link to the **PartSource** table which is a calculated reference made by RapidResponse when the order is generated. This is shown in the following illustration.

Figure 4.11 PlannedOrder and associated tables



Note: For each record generated in the **PlannedOrder** table, a corresponding record is generated in the **CTPPlannedOrder** table. This table references the PlannedOrder table and contains capable-to-promise details relating to the planned order (for example, availability details).

NewAllocation and PlannedAllocation tables

The RapidResponse data model also includes the **NewAllocation** and **PlannedAllocation** tables each of which hold calculated allocations (dependent demands) generated by RapidResponse.

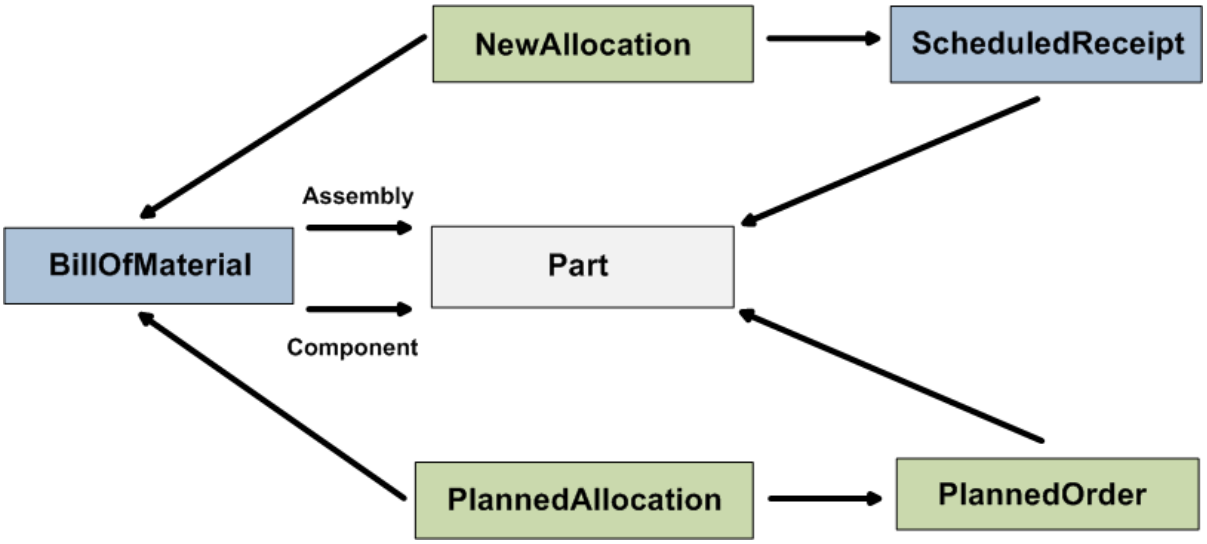
As discussed earlier in this chapter, the input **Allocation** table is used to store dependent demands imported from your enterprise data source and tied to a particular scheduled receipt. This might be done, for example, when the order is close to being released or when the standard bill of materials has been deviated from. Typically, though, it is not necessary to import dependent demands on a scheduled receipt.

For those scheduled receipts which do not have records imported into the **Allocation** table, the **NewAllocation** table is then used to store calculated allocations generated by RapidResponse calculations.

Similarly, the **PlannedAllocation** table contains calculated dependent demands generated because RapidResponse generated a planned order for a parent part.

Records in the **NewAllocation** and **PlannedAlllocation** tables are calculated by the explode calculation for each BillOfMaterial record. As such, these tables both reference the BillOfMaterial table as well as the corresponding **ScheduledReceipt** or **PlannedOrder** record that drove the dependent requirement.

Figure 4.12 NewAllocation and PlannedAllocation table hold calculated dependent demands



Consolidation tables

RapidResponse also includes several calculated "consolidation" tables. These tables combine multiple sources of supply and/or demand records into a single table. For example, as discussed previously, RapidResponse stores calculated dependent demands in both the **NewAllocation** and **PlannedAllocation** tables and also allows dependent demands to be imported into the **Allocation** table. For a single view of dependent demands, records from all of these tables are consolidated into the calculated **DependentDemand** table.

The following outlines some key consolidation table in RapidResponse:

Table 4.1: Consolidation tables

Table	Description
DependentDemand	Combines input allocations, new allocations, and planned allocations together to show all dependent demands.
Demand	Combines dependent demand and independent demand together to give the total demand picture. Includes only those demands processed by Netting calculations
Supply	Combines multiple sources of supply such as scheduled receipts, planned orders, and inventory transfers to give the total supply picture. Includes only those demands processed by Netting calculations.
Activity / CTPActivity	Combines all supply and demand information together in a single place. Activity reports details from Netting calculations and CTPActivity reports details from Capable-to-Promise calculations.

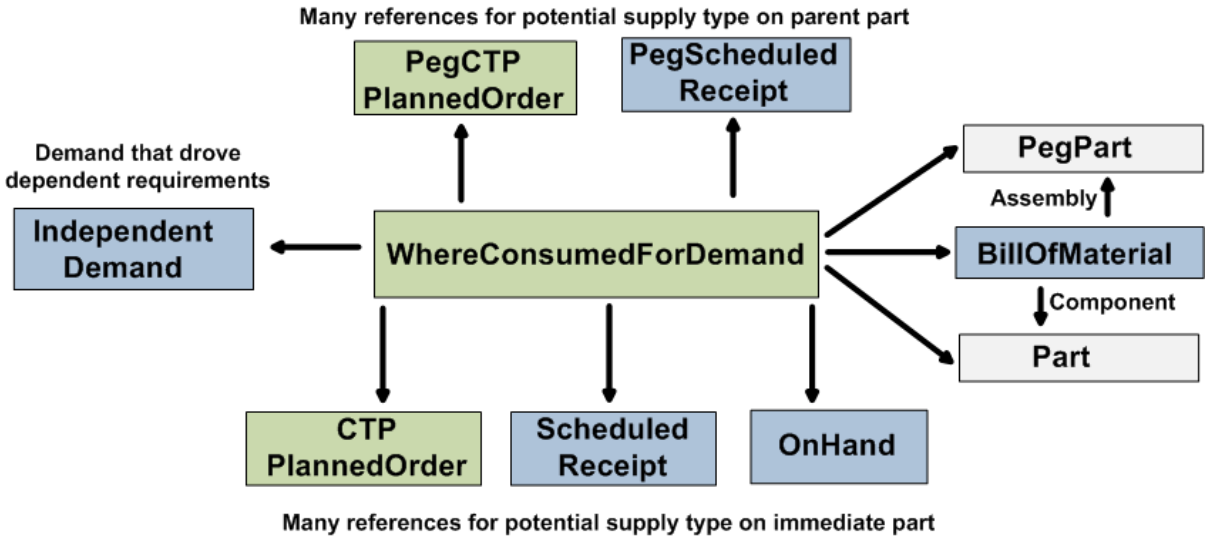
Pegging tables

RapidResponse includes many other calculated full-level pegging tables you might find useful for reporting and analysis purposes. This includes tables that trace allotments of supply to the higher level demands (or supplies) that consume them.

WhereConsumedForDemand table

For example, the WhereConsumedForDemand table can be used to peg independent demands to the set of supplies they consume. Each record in this table reports details of a given supply (for example, an onhand quantity, scheduled receipt or planned order), the immediate parent supply that it satisfies, and the driver independent demand ultimately responsible for driving demand down the structure. This make it possible to show all supplies that ultimately satisfy a given independent demand by reporting all of the individual supply allotments between the top-level demand and its lowest level component(s).

Figure 4.13 References from the **WhereConsumedForDemand** table.



The following outlines some the main tables used for reporting supply allotment details in RapidResponse.

Table 4.2: Tables that contain pegging information

Table	Description
SupplyDemand	Populated by capable-to-promise calculations, but contains records linking the supply of components to the next level demands that consume them (single-level only). Useful when you want to match demands to the allocated supply at a single part level only.
WhereConsumed	Pegs supply records to the end item demands which consume them. Useful when you want to show pegging information across all levels in the structure with a focus on the supply that is being consumed.

Table 4.2: Tables that contain pegging information

Table	Description
WhereConsumedForDemand	Pegs independent demand records to the set of supplies they consume. Useful when you want to show pegging information across levels in the structure with a focus on the demand that is driving the activity.
WhereConsumedForSupply	Pegs supply records to the higher level production orders that consume them. Useful when you want to see all material required for any order in the supply structure or that are ultimately used in satisfying a top-level supply.
WhereConsumedForForecast	Pegs consensus forecast records to the set of supplies they consume. Useful when you want pegging information across levels in the structure with a focus on demands generated from the sales and operations planning process in RapidResponse.

Controls

The RapidResponse data model contains control data. This data is found in control tables each of which contains one or more configurable rules or values. The configuration of these rules determines the processing applied to your imported data and the results of the calculations seen in the RapidResponse data model.

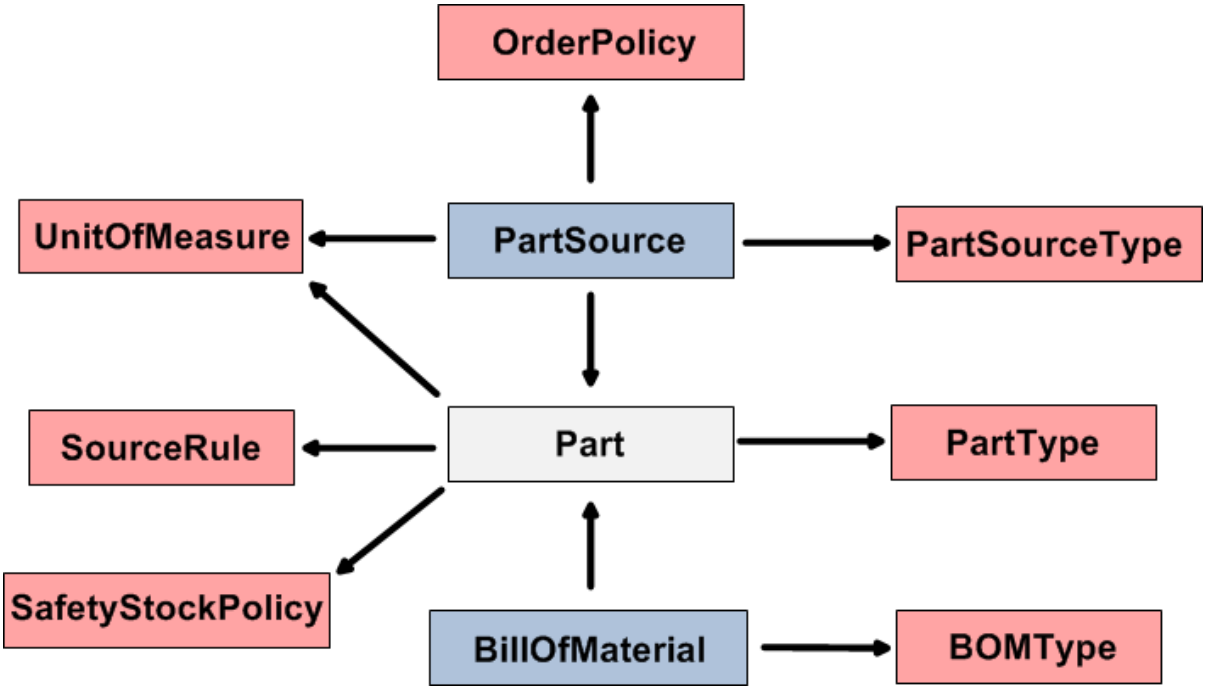
Typically, these control tables are set up once during a RapidResponse implementation and are intended to correspond to the types of data, codes, and associated processing rules found in the enterprise data source. This ensures that the data generated by RapidResponse is familiar to what you have been accustomed to seeing in the enterprise data source. It is also possible to change these controls if, for example, you wanted to simulate the impact of a global policy change.

Control tables are referenced from many input tables in the data model.

Control tables (on products and structure)

As shown in the following illustration, control tables are referenced from many tables containing imported enterprise data about products and structure.

Figure 4.14 Examples of control tables referenced by products and structure tables



The following list describes some of the main control tables referenced by product and structure tables in RapidResponse.

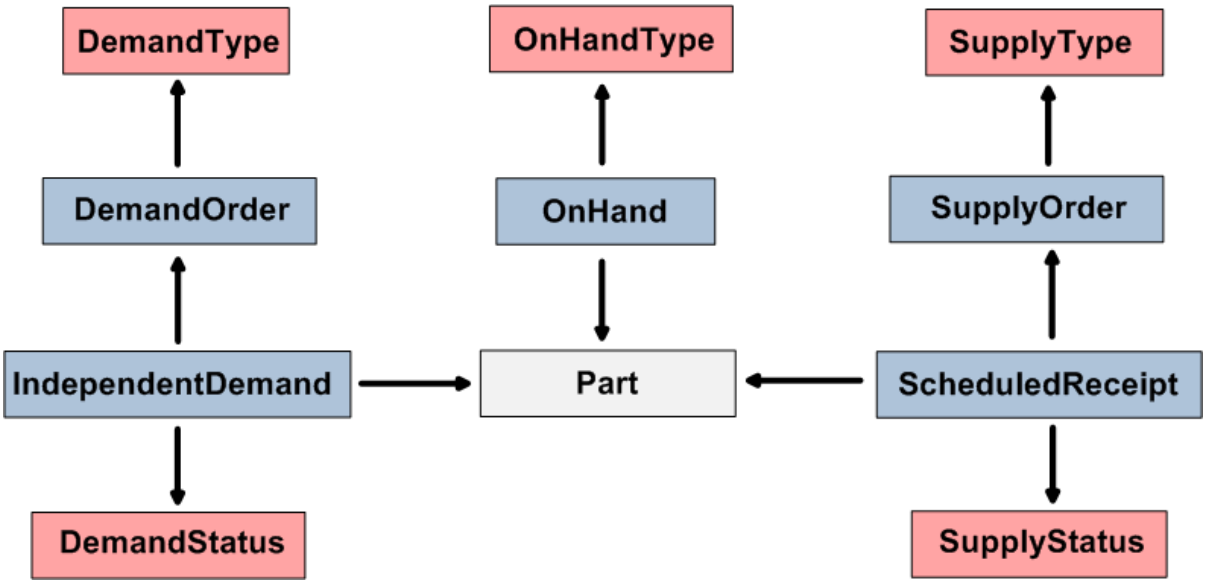
Table 4.3: Descriptions of control tables referenced by products and structure table

Table	Description
PartType	Sets how the part is processed by RapidResponse calculations. For example, can planned orders be created for the part.
BOMType	Identifies different types of bill of material records and the rules for processing them. This includes things such as blowthrough logic (phantom parts), scrap rules and effectivity rules.
PartSourceType	Sets the characteristics of each potential supply source. For example, it controls the effectivity of sourcing features and determines the type and status of planned order's when supplies are required using this source.
OrderPolicy	Controls details of planned order generation from this source. For example, lot sizing policies and lead time rules for a part source on defined on this table.
SourceRule	Defines how part sources are handled when a part has more than one active source.
UnitOfMeasure	Defines, and allows conversion between, different unit of measure codes. For example, supplier units to inventory units.

Control tables (on activity)

As shown in the following illustration, control tables are referenced from tables containing imported enterprise activity data.

Figure 4.15 Examples of control tables referenced by activity tables



Examples of supply and demand control tables

The following list describes some of the main control tables referenced by input activity tables in RapidResponse.

Table 4.4: Descriptions of control tables referenced by supply and demand tables

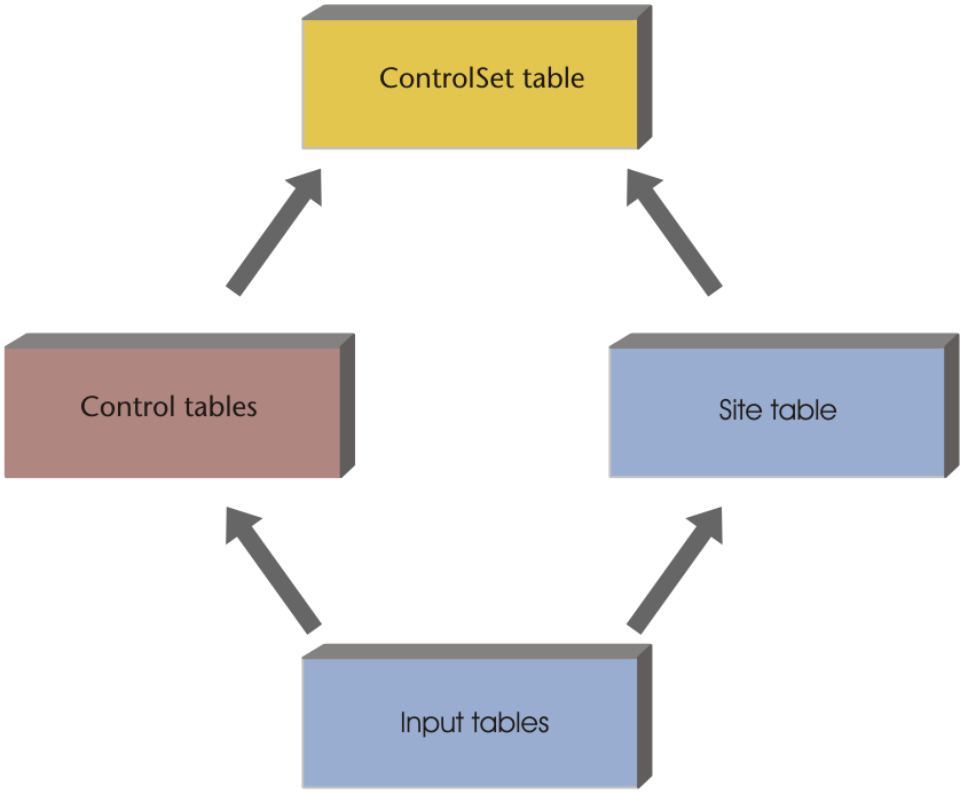
Table	Description
OnHandType	Sets whether a given stock of inventory is included in Netting calculations.
SupplyType	Defines processing rules for handling a particular supply order and its associated line items. For example, can allocations be generated for the line items. Also used by the **PartSourceType** control table to define characteristics for processing planned orders using this part source.
SupplyStatus	Defines processing rules for handling a particular line item (scheduled receipt). For example, a line item can be reschedulable. Also used by the **PartSourceType** control table to define characteristics for processing planned orders using this part source.
DemandType	Defines processing rules for handling a particular demand order and its associated line items. For example, should it be treated as actual or forecasted demand.
DemandStatus	Sets the processing rule for a line item (independent demand). For example, you can specify that the item be ignored by RapidResponse calculations.
ShipmentPolicy	Sets whether a given independent demand can be broken into partial shipments.

Control sets

RapidResponse allows for named groupings of control table settings known as control sets.

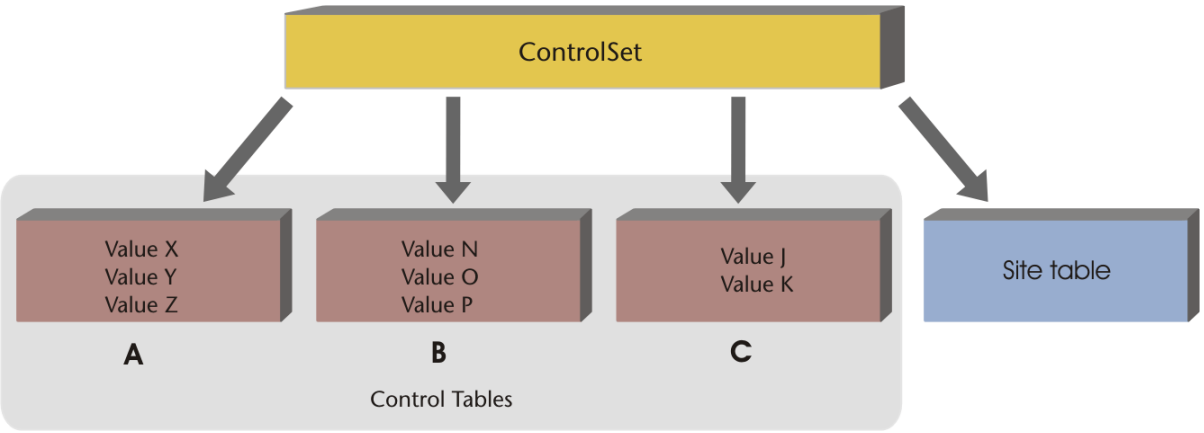
All control set values are stored in the ControlSet table. This table is referenced by all control tables other than the HistoricalDemandCategoryType table. As shown in the following illustration, the ControlSet table is also referenced by the Site table. This reference indicates the control set that input data and its controls at a given site are typically expected to use.

Figure 4.16 References to ControlSet table



As shown in the following illustration, each given control set is made up of all Values on any of the tables listed above that reference that control set. Using control sets you can then easily show only those relevant control tables values for a particular type of RapidResponse site or configuration. Control sets also allow a defined group of control settings to be copied, modified, and re-used at other installations with similar data configurations.

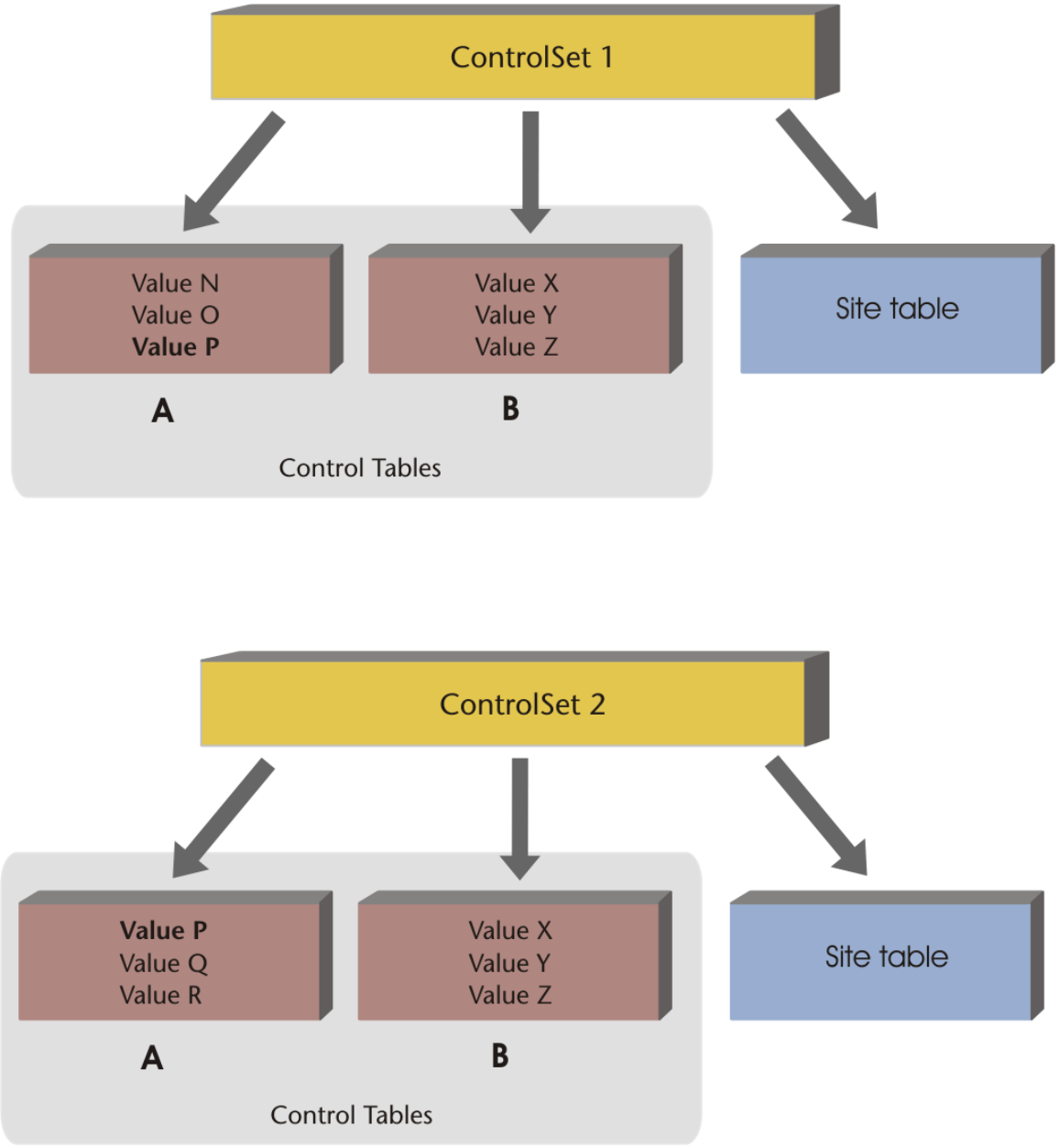
Figure 4.17 Values belonging to a single control set



Control sets also allow for the possibility of duplicate control values existing across sites and each being associated with different behaviors or properties. For example, in the following illustration a value of "P" exists on Control table A in both Control Set 1 and Control Set 2. If Control table A is assumed to

represent the PartType table, then "P" might indicate a part type of "purchased" in one control set, but a part type of "phantom" in another control set. By allowing duplicate control values that potentially represent different behaviors, control sets protect against data collisions in cases where multiple data sets are merged together in RapidResponse.

Figure 4.18 Values belonging to different control sets



Notes:

- For more information about the ControlSet table, see "ControlSet table" on page 253.
- For more information about managing control sets, see the *RapidResponse Administration Guide*.

CHAPTER 5: Optional applications and tables

This guide describes all the standard and optional tables, fields, and analytics available with RapidResponse.

RapidResponse offers optional application options to help manage many supply chain planning processes in different functional areas. RapidResponse also includes optional functional capabilities that your company might enable to support business processes.

The deployment of optional applications and capabilities impacts the RapidResponse client and the analytic calculations.

Optional analytics and the data model 165
Optional RapidResponse capabilities 166
Optional RapidResponse applications 170

Optional analytics and the data model

Optional RapidResponse applications and capabilities and are enabled through license keys. License keys are provided by Kinaxis Customer Services in a file that is imported into RapidResponse by a system administrator. If you are using the On-Demand version of RapidResponse, contact your Master Administrator at Kinaxis for more information about enabling optional applications and capabilities. If you are using the On-Premises version of RapidResponse, review the *RapidResponse Administration Guide* for more information.

The RapidResponse client might hide tables and fields associated with optional applications and capabilities that have not been enabled by your company. As such, resource authors, or users who create reports, might not have access to tables and fields that have not been enabled. Resource authors cannot create a report based on a table that has not been enabled or use a field that references a table that has not been enabled. If you import data into tables that have not been enabled, you will not be able to view or use this data.

If a table contains a field that references a table that has not been enabled, then that field is not available to resource authors. For example, the Operation table includes a field that references the Model table. If the Model-Unit Effectivity capability is not enabled then the Model table is not available and the reference from the Operation table is also not available.

In addition, analytic calculations associated with optional applications and capabilities might not be calculated. As such, calculated fields associated with these optional applications and capabilities remain empty.

The tables, fields, and analytics associated with optional applications and capabilities are not explicitly noted in this document. Contact your Kinaxis Customer Service for more detailed information.

Optional RapidResponse capabilities

This section describes the following optional capabilities available with RapidResponse:

- "Campaign Planning" on page 166
- "Constraint Manager" on page 166
- "Model Effectivity and Unit Effectivity " on page 167
- "R integration" on page 168
- "Self-Healing Supply Chain" on page 169

Campaign Planning

Campaign planning is typically used in process industries where supplies are produced in batches and efficiencies can be gained from sequencing and processing those batches as a contiguous group. When campaign planning logic is enabled, supplies for a part are planned in defined batch sizes and grouped together into production campaigns.

For more information, see "Campaign planning" on page 1707.

Fields and options available with Campaign Planning

- “CampaignPlanning” option in OrderPolicy.MaximumUsage
- PlannedOrder.CampaignIndex
- SourceConstraint.CampaignFixedConstraintFactor
- SupplyOrder.CampaignIndex

Constraint Manager

Constrained planning logic determines the date when supplies and demands are available with respect to one or more constraints associated with the source of supply. Constraints can be fixed or variable factors representing the capacity of a process or supplier (for example, the number of units per day a production line can produce). A single source of supply can be constrained (restricted) by multiple constraints.

Tables available with Constraint Manager

- Constraint
- ConstraintAvailable
- ConstraintLoad
- ConstraintSpreadLoad
- ConstraintType

- ConstraintUnitOfMeasure
- ConstraintUsed
- FLPConstraint
- PeriodConstraintLoad
- SourceConstraint
- TrialConstraintShortage
- TrialConsumption

Constrained planning with Constraint Manager

The **ConstaintType** table holds control settings used in constrained planning. The **ProcessingType** field on this table determines how constraints are applied and impacts load, available date, and other calculations.

If a full Constraint Manager license is applied, then all **ConstraintType.ProcessingRule** settings are available and constrained supply plans can be generated. That is, any of the "Constrained", "CumMax", "LoadOnly", "PlanningOnly", "Unconstrained", and "UnconstrainedPlan" options can be applied to a constraint.

However, if a limited load-only Constraint Manager license is applied, then only the "LoadOnly" and "Unconstrained" options are available and constrained supply plans cannot be generated (though all tables listed above are still visible).

Note: In the absence of any Constraint Manager license, supply planning is always unconstrained and the tables listed above are not visible.

Model Effectivity and Unit Effectivity

Model and Unit Effectivity are grouped as a single capability (Model-Unit Effectivity). Effectivity is a method for determining if a bill or operation record is used to generate dependent demand. Enabling this capability also enables the use of the Model control in worksheets. For more information about the analytic calculation, see "Model and Unit Effectivity calculations" on page 1737.

Tables available with Model-Unit Effectivity

- BOM_MUECumLead
- Model
- MUEPoolNetting
- Part_MUECumLead
- Part_MUEPoolValidation

The following fields are only available if the Model-Unit Effectivity is enabled.

Activity.StartUnit	FlatBillUp.FirstUnit	Operation.LastEffectiveUnit
Allocation.StartUnit	FlatBillUp.IsModelEffective	Operation.StartUnit
AvailableToPromise.StartUnit	FlatBillUp.LastUnit	Part.NextUnit

BaselinePlannedOrder.StartUnit	Forecast.StartUnit	Part_MUEPoolValidation.StartUnit
BillOfMaterial.EndUnit	IndependentDemand.StartUnit	PlannedAllocation.StartUnit
BillOfMaterial.FirstEffectiveUnit	InProcess.StartUnit	PlannedOrder.StartUnit
BillOfMaterial.LastEffectiveUnit	InventoryAnalysis.StartUnit	ScheduledReceipt.StartUnit
BillOfMaterial.StartUnit	InventoryCTPAnalysis.StartUnit	Supply.StartUnit
BlowThroughNewAllocation.New Unit	InventorySummary.StartUnit	SupplyAllocation.StartUnit
BlowThroughPlannedAllocation.StartUnit	LateSupply.StartUnit	SupplyDemand.StartUnit
CostOfGoodsSold.SupplyStartUnit	LoadDetailCurrent.StartUnit	ValidPlannedOrder.StartUnit
CostRollUp.FirstUnit	LoadDetailPlanned.StartUnit	WhereConsumed.SupplyStartUnit
CostRollUp.LastUnit	LoadOperationNewOrderPlanned. StartUnit	WhereConsumedForDemand.StartUnit
CTPActivity.StartUnit	LoadOperationReceiptsCurrent. StartUnit	
Demand.StartUnit	LoadOperationReceiptsPlanned. StartUnit	
DependentDemand.StartUnit	NewAllocation.StartUnit	
FlatBillDown.FirstUnit	OnHand.StartUnit	
FlatBillDown.IsModelEffective	Operation.EndUnit	
FlatBillDown.LastUnit	Operation.FirstEffectiveUnit	

R integration

RapidResponse provides optional integration with the R programming language. R is integrated with RapidReponse to perform advanced statistical calculations for the following features:

- Statistical forecasting using the Rforecast model. For information, see "Rforecast" on page 2039.
- Advanced methods of outlier detection (stl, tsclean methods) in historical data in demand planning (data cleansing) and inventory planning and optimization (safety stock).
- Advanced statistical forecasting capabilities using multiple linear regression models. For more information, see "Multiple Linear Regression forecasting models" on page 2042.

Integration with R in RapidResponse is only available if the optional Rforecast capability has been enabled, and is available only with the On-Demand RapidResponse Service. In these cases, Kinaxis will deploy a distribution of specific standard 'R' package and ensure proper configuration with your company's instance of RapidResponse. Every effort will be made to keep these as up to date as possible. However, Kinaxis does not offer any services to deploy packages other than the specific set, nor to alter the content of the specific standard 'R' packages.

To learn more about the R, see www.r-project.org.

Tables available with Rforecast capability

- RParameter
- RParameterName
- RParameterSet

Fields and settings available with Rforecast capability

- BestFitProfileDetail.RParameterSet
- ForecastItemParameters.RParameterSet
- "Rforecast" option in ForecastItemParametersType.ForecastModel
- "RtslError" option in OutlierType.DataRule
- "Rtclean" option in OutlierType.DetectionRule

Note: For information about using R to generate the statistical forecast, see "Rforecast" on page 2039.

Self-Healing Supply Chain

The Self-Healing Supply Chain is only available if it has been enabled, and is only available with the On-Demand RapidResponse service. In these cases, the Self-Healing Supply Chain data model and calculations will be available. For more information, see "Self-Healing Supply Chain" on page 2135.

RapidResponse provides the Self-Healing Supply Chain. The Self-Healing Supply Chain uses machine learning to automatically detect and correct deviations in parameters used in supply planning data.

Tables available with Self-Healing Supply Chain

- AutomatedLeadTimeType
- AutomatedYieldType
- Cluster
- ClusterProperty
- HistoricalReceipt
- HistoricalReceiptHeader
- HistoricalSupplyActual
- ItemSource
- ItemSourceCandidate
- ItemSourceFitOutput
- ItemSourceFitOutputClass
- ItemSourceFitOutputForecastModel
- ItemSourceFitOutputValueName
- ItemSourceHistoricalSupplyCategory
- ItemSourceIntervalEffectivePartSource
- ItemSourcePartSourceEffectivityInterval

- ItemSourcePartSourceHistoricalLeadTime
- ItemSourcePartSourceHistoricalReceipt
- Job
- ProjectedLeadTime
- ProjectedYield

Optional RapidResponse applications

RapidResponse supports the following optional applications:

- Order Fulfillment
- Sales and Operation Planning
- Inventory Management
- Inventory Planning and Optimization
- Master Production Scheduling
- Supply Action Management
- Demand Planning
- Aggregate Supply Planning
- Capacity Planning - Constraints
- Capacity Requirements Planning
- Distribution Requirements Planning
- Engineering Change Management
- Supplier Collaboration
- Integrated Project Management
- Self-Healing Supply Chain

Deploying optional applications controls the visibility of tables and fields. It also controls whether some analytics are calculated.

CHAPTER 6: Input tables

This chapter describes each input table in the RapidResponse data model, along with all the input fields it contains. For those tables that contain either calculated or set fields, the details of these fields are described separately, immediately after the input fields.

RapidResponse contains input tables belonging to the Mfg namespace that store data imported from your enterprise data source. This includes data such as part and bill of material information, customer orders, and work orders. For the most part, this data is editable. However, some tables contain calculated fields. This data is not imported from your enterprise data source, but is instead generated by RapidResponse calculations. For example, in the ScheduledReceipt table, RapidResponse calculates fields such as the AvailableDate and RecommendedDate associated with an order.

RapidResponse also contains input tables belonging to the ProjMgmt namespace that store data imported from your project management application. This includes data such as project details, the work tasks that comprise each project, and the resources assigned to complete project tasks. Many of the tables in this namespace contain both input and calculated fields. However, when using the predefined project management resources included with RapidResponse, many input fields are populated by values inserted into other tables, and many calculated fields are editable. For example, records are added to the WBS table when inserting new Task table values.

In this chapter, you'll learn about the following:

ABCCode table 176
Activity table 176
AggregatePartCustomer table 179
Allocation table 182
AllotmentOverride table 187
AllotmentOverrideDetail table 189
AlternatePart table 190
AlternateRouting table 193
Assignment table 194
Assumption table 195
AssumptionByConstraint table 197
AssumptionByPart table 197
AssumptionByPartCustomer table 198
AssumptionByPartSource table 198
AssumptionStatus table 199
AssumptionType table 199
AttributeMap table 200
BaselineConflict table 204
BaselinePlannedOrder table 206

BaselineTaskLink table 207
BestFitProfile table 209
BestFitProfileDetail table 210
BillOfMaterial table 217
BOMAlternate table 229
BonusSchedule table 230
BonusScheduleByDate table 231
BonusScheduleByInterval table 232
BuyerCode table 233
Capacity table 234
CapacityOverride table 236
Carrier table 238
CausalFactor table 239
CausalFactorCategory table 240
CausalFactorDetail table 241
Class table 242
Cluster table 243
ClusterProperty table 244
CollaborativeForecast table 245
ConsensusForecastRollingHorizon table 246
Constraint table 247
ConstraintAvailable table 249
ConstraintGroup table 251
ConstraintMetric table 252
ConstraintMetricValues table 253
ControlSet table 253
CRPOperation table 256
CRPOperationState table 260
CurrencyConversionForecast table 263
CurveParameters table 263
CurvePoint table 265
Customer table 266
CustomerDestination table 269
CustomerGroup table 269
CustomerPrice table 270
DeliveryRoute table 271
DemandOrder table 272
Department table 274
EngineeringChange table 274
Event table 276
EventPhase table 276
EventPhaseHeader table 279
EventType table 280
Feature table 281
FeatureMap table 281
ForecastDetail table 283
ForecastDisaggregationParameters table 287
ForecastDisaggregationParametersByCategory table 288
ForecastDisaggregationOverride table 290
ForecastItem table 292
ForecastItemFitOutput table 294
ForecastItemParameters table 296
ForecastItemParametersFitOutput table 313
ForecastItemParametersMap table 321

ForecastItemParametersOutlier table 322
ForecastItemParametersRegressionCoefficient table 322
HistoricalCurrencyConversion table 324
HistoricalCurrencyHeader table 325
HistoricalDemandActual table 325
HistoricalDemandCategory table 329
HistoricalDemandCategoryRollingWeight table 333
HistoricalDemandHeader table 334
HistoricalDemandHeaderRollingWeight table 336
HistoricalSupplyOrder table 337
HistoricalDemandHeaderTimePhasedAttributes table 338
HistoricalDemandOrder table 339
HistoricalDemandSeries table 340
HistoricalDemandSeriesDetail table 341
HistoricalPartKPI table 343
HistoricalReceipt table 345
HistoricalReceiptHeader table 345
HistoricalShipment table 346
HistoricalSupplyActual table 347
HistoricalSupplyCategory table 350
HistoricalSupplyHeader table 351
HistoricalSupplySeries table 353
HistoricalSupplySeriesDetail table 354
HoldCode table 355
ImportedStatisticalForecast table 356
IndependentDemand table 358
IndependentDemandFeatureList table 369
IndependentDemandSolution table 370
InventoryTransfer table 374
ItemSource table 379
ItemSourceFitOutput table 386
ItemSourceFitOutputClass table 387
ItemSourceFitOutputForecastModel table 388
ItemSourceFitOutputValueName table 388
ItemSourceHistoricalSupplyCategory table 389
Job table 389
KPI table 390
Location table 391
LogisticsLocation table 392
MEIOLeadTime table 393
MEIOLeadTimeDistributionProfile table 396
MEIOLeadTimeDistributionProfilePoint table 397
MEIOLeadTimeOutlier table 398
Metric table 399
MetricAsOf table 400
Model table 401
Notification table 402
OnHand table 404
Operation table 407
OperationSequence table 410
OriginatingFile table 413
OriginatingSource table 413
OverrideMajorChangeover table 414
Part table 415

PartCustomer table 444
PartCustomerTimePhasedAttributes table 449
PartMetric table 451
PartMetricValues table 452
PartSequence table 452
PartSolution table 453
PartSource table 459
PartSourceTimePhasedAttributes table 476
PartSupplier table 477
PartUOMConversion table 478
PartValidation table 479
PenaltySchedule table 480
PenaltyScheduleByDate table 481
PenaltyScheduleByInterval table 482
PlannerCode table 483
PointsCurve table 484
PointsProfile table 485
Pool table 486
PoolMap table 487
PoolMapOverride table 487
Predecessor table 488
ProcessInstance table 491
ProductFamily table 493
ProductionGroup table 493
ProductionRecurrence table 495
ProductionWheel table 496
ProductionWheelConstraint table 498
ProfilePoint table 499
Project table 500
ProjectedLeadTime table 505
ProjectedYield table 506
ProjectGroup table 507
ProjectManager table 508
ProjectStatus table 508
RangeOfCoveragePartOverride table 509
ReferenceForecastDimension table 511
ReferenceForecastTarget table 512
ReferencePart table 513
Region table 514
RegionGroup table 514
RegressionModel table 515
RegressionModelTerm table 516
Regressor table 517
RegressorCategory table 518
RegressorValue table 518
Resource table 519
ResourceAssignment table 522
ResourceAssignmentAvailable table 525
ResourceAvailable table 526
ResourceCalendarException table 527
ResourceCostActual table 528
ResourceCosts table 530
ResourceGroup table 531
ResourceGroupProjectOverride table 532

ResourceGroupTaskOverride table 533
ResourceProjectOverride table 534
Routing table 534
RParameter table 536
RParameterSet table 537
SafetyLeadTimeProfile table 538
SafetyLeadTimeProfileDetail table 539
SafetyStockAverageDemandProfile table 540
SafetyStockItem table 541
SafetyStockItemMapping table 554
SafetyStockItemOutlier table 556
SafetyStockOverride table 557
SafetyStockPart table 560
SafetyStockRangeOfCoverage table 565
SafetyStockTimePhasedBounds table 565
SAP_MD04 table 567
ScheduledReceipt table 569
Segment table 589
SequenceFlow table 590
Shipment table 590
ShipSet table 592
Shop table 593
ShopKanban table 594
Site table 596
SiteManager table 600
SiteManagerType table 601
SiteMetric table 602
SiteMetricValues table 602
Source table 603
SourceConstraint table 605
SourceKanban table 607
StorageType table 610
SubProcess table 610
SubstituteGroup table 611
Supplier table 612
SupplierGroup table 614
SupplierMetric table 615
SupplierMetricValues table 616
SupplyAllocation table 616
SupplyOrder table 618
Task table 624
TaskDemandLink table 646
TaskHyperlink table 648
TaskPartLink table 648
TaskSupplyLink table 650
TimePhasedDemandParameterSet table 653
TimePhasedLocationCapacity table 653
TimePhasedMaximumInventory table 654
TimePhasedPartSourceLeadTime table 654
TimePhasedSafety table 655
TimePhasedSourceLeadTime table 660
TimePhasedYield table 661
TransferDestination table 662
TransportationMode table 663

UnitOfMeasureCategory table **664**
Warehouse table **665**
WBS table **666**
WorkCenter table **666**

ABCCode table

This table lists valid ABC codes which organize parts into groups based on annual dollar volume or other criteria. These are used to identify those parts which have the most impact and should be closely controlled.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.1: ABCCode (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this ABC code.	String	
Site	The site associated with this ABCCode. This field is optional. **Referenced table**: Site	Reference	Yes
Value	A unique identifier associated with this ABC code. For example, "A".	String	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

ABCCode table – calculated and set fields

The following describes the calculated and set fields in the ABCCode table. For an introduction to this table, and descriptions of its input fields, see "ABCCode table" on page 176.

Table 6.2: Calculated ABCCode (Mfg) fields

Field name	Description	Data type	Key
Parts	Set of parts associated with this ABC code.	Set	

Activity table

The **Activity** table defines all the individual work items that together form a process. In addition to a reference to the applicable process instance, each record in this table contains details such as the

activity name, duration, and start day. It also reports calculated details such as activity start and finish dates.

Table 6.3: Activity (ProcOrch) fields

Field name	Description	Data type	Key
Duration	An input value that defines the duration of the activity.	Integer	
Id	A unique string identifier for this task.	String	Yes
Name	A name for the activity. This value should make the activity easy to recognize	String	
Override Notifications	Specifies whether the global notifications for the activity are overridden or not.	Boolean	
ProcessInstance	The name of the process instance that this activity belongs to. **Referenced table:** ProcessInstance	Reference	
SortKey	Defines the sequence of this activity.	String	
StartDay	The day in the process instance that this activity is planned to start in relation to the first day of the process instance.	Integer	

Activity table – calculated and set fields

The following describes the calculated and set fields in the Activity table. For an introduction to this table, and descriptions of its input fields, see "Activity table" on page 176.

Table 6.4: Activity (ProcOrch) fields

Field name	Description	Data type	Key
ActualFinishDate	If the activity has no assignment, the date is the expected finish date, if it falls after **RunDate.FirstDate**. If the activity has assignments, the date is the date of the last assignment to be completed.	Date	
ActualStartDate	If the activity has no assignment, the date is the expected start date, if it falls after **RunDate.FirstDate**. If the activity has assignments, the date is the date of the first assignment to begin work on the activity.	Date	

Table 6.4: Activity (ProcOrch) fields

Field name	Description	Data type	Key
CalcDuration	The calculated duration of an activity. For activities that are not sub-processes, the input duration is used. For the sub-processes, the **CalcDuration** is the summary of the durations of the child activities.	Integer	
CalcStartDay	For activities that are not sub-processes, the input StartDay is used. For activities that are sub-processes, the earliest **Child.CalcStartDate** is used.	Integer	
ExpectedFinish Date	The date the activity is expected to finish. For activities that are not sub-processes, the date equals to **ExpectedStartDate + CalcDuration 'ProcessInstance.Type.Calendar'** For activities that are sub-processes, the latest **Child.ExpectedFinishDate** is used.	Date	
ExpectedStart Date	The date the activity is expected to start. For activities that are not subprocesses, the date equals to **ProcessInstance.EpxectedStartDate + CalcStartDay 'ProcessInstance.Type.Calendar'**. For activities that are sub-processes, the earliest **Child.ExpectedStartDate** is used.	Date	
IncomingFlow List	Used to display a concatenated list of the prerequisites of this activity.	String	
OutlineLevel	The level of the activity, in terms of its relationship with other activities, in the process tree.	Quantity	
OutlineNumber	A string value used to indicate the activity's numerical position in the process instance hierarchy. For example, the first top-level activity has a value of 1, and activities underneath it have values such as 1.1., 1.2, and so on.	String	
OutgoingFlow List	Used to display a concatenated list of the subsequent activities of this activity.	String	
Parent	Parent sub-process of the activity. **Referenced table:** Activity	Reference	
SiblingIndex	An index number that is used to sort activities within a sub-process. Starts at 1 for each grouping of sibling activities.	Quantity	

Table 6.4: Activity (ProcOrch) fields

Field name	Description	Data type	Key
Status	The rolled up status of the activity. The status can be one of the following: • **NotStarted** • **InProgress** • **Finished** If the activity is not a sub-process, the status of the activity is based on the status of the users assigned to perform the activity: • **NotStarted**—the activity is NotStarted until at least one performer changes their status to InProgress or Finished. • **InProgress**—the activity is InProgress when at least one performer changes their status to InProgress or Finished. • **Finished**—the activity is Finished when all of the performers set their status to Finished. If the activity is not a sub-process, and has no assignments, the status is calculated in the following manner: • **NotStarted**—RunDate is less than the ExpectedStartDate • **InProgress**—The ExpectedStartDate is less than or equal to the RunDate, which is less than or equal to the ExpectedFinishDate • **Finished**—RunDate is greater than or equal to the ExpectedFinishDate If the activity is a sub-process, the status is based on the status of the child activities.	Enum	
Assignments	The set of users assigned to this activity.	Set	
Children	Set of sub-process records that reference this activity as a parent process.	Set	
IncomingFlows	Set of prerequisites for this activity.	Set	
Notifications	Set of notifications defined for this activity.	Set	
OutgoingFlows	Set of subsequent activities after this activity.	Set	
Parents	Set of sub-process records that reference this activity as a sub-process.	Set	

AggregatePartCustomer table

The **AggregatePartCustomer** table can be used to group part customers together to define the level at which forecast disaggregation occurs. In cases where it is sufficient to forecast at an aggregate level,

rather than down to a detailed part and customer combination, this can result in improved database performance.

This table ties unique part and customer combinations in the **PartCustomer** table with other **PartCustomer** records representing aggregate part or customer-based groupings. Therefore, each record in this table has two references to the **PartCustomer** table; the **Component** field which references the actual part and customer combination being aggregated, and the **Aggregate** field which references an aggregate part customer value.

For example, multiple customers for the same part might be associated with an aggregate part customer value representing the sales region in which all those customers are located. Thus, historical demands used as input in generating the statistical forecast can define the actual part and customer combination a demand was for, but it might only be necessary to disaggregate the part's statistical forecast to the regional level.

For more information about using this table, see "Define aggregate part customer relationships" on page 2091.

Note: In order for aggregate part-customer disaggregation to work, ensure the **PartCustomer.ForecastItem.Path** value for both the **Aggregate** and **Component** part customers are set to the same value.

Table 6.5: AggregatePartCustomer (Mfg) fields

Field name	Description	Data type	Key
Aggregate	A reference to an aggregate **PartCustomer** record. Aggregate part customers are used to group defined part and customer combinations together during forecast disaggregation. For example, this might reference a **PartCustomer** value added to aggregate all of a part's customers in a particular region. **Referenced table:** PartCustomer	Reference	
Component	A reference to a part and customer combination being aggregated under the **PartCustomer** value referenced in the **Aggregate** field. If this references the same value as the **Aggregate** field, then the component is assumed not to be aggregated (as of the **EffectiveInDate**). **Referenced table:** PartCustomer	Reference	Yes
EffectiveInDate	Date when the part customer aggregation defined on this record first becomes effective. For a given effective date, there should be at most one record for a particular Component (on a given date, a PartCustomer can only have one aggregate).	Date	

Table 6.5: AggregatePartCustomer (Mfg) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this record. It is recommended to set this to a string returning the EffectiveInDate field value (in other words, `text (EffectiveInDate, yyyymmdd)`. This ensures that additional records having the same Component and EffectiveInDate value are flagged and rejected as duplicates.	String	Yes
Type	A reference to a control setting used to include or exclude aggregate part customers from use in disaggregation calculations. **Referenced table:** AggregatePartCustomerType	Reference	

AggregatePartCustomer table – calculated and set fields

The following describes the calculated and set fields in the **AggregatePartCustomer** table. For an introduction to this table, and descriptions of its input fields, see "AggregatePartCustomer table" on page 179.

Table 6.6: Calculated AggregatePartCustomer (Mfg) fields

Field name	Description	Data type	Key
EffectiveOut Date	Date when the part customer aggregation defined on this record stops being effective. Each record is considered effective until the next **EffectiveInDate** on a record with matching **Aggregate** and **Component** values (a value of "Future" is reported if no such record exists).	Date	
IsNested	Indicates whether this is a nested record. A nested record refers to situations where the value referenced in the **Aggregate** field is also referenced in the **Component** field of another **AggregatePartCustomer** record. Valid values are: • **N**—a nested record that is ignored during forecast disaggregation calculations. • **Y**—a valid record that is used during forecast disaggregation calculations.	Boolean	

Allocation table

This table identifies allocations; that is, dependent demand items generated because a work order (scheduled receipt) was created for a make item. It only contains those allocations which were contained in your enterprise data source, and imported into the RapidResponse data model. Typically, this is done at a time close to when the order is being released to the shop floor, or when you have deviated from the standard bill of materials. For any scheduled receipts you are providing allocations to RapidResponse for, you need to ensure it does not generate allocations for them. This is done by setting an appropriate value for the Status field on the ScheduledReceipt table (this is a reference to the **SupplyStatus** table).

If you do not provide allocations, RapidResponse will generate them and store them in the NewAllocation table. For more information about this table, see "NewAllocation table" on page 1137.

Table 6.7: Allocation (Mfg) fields

Field name	Description	Data type	Key
BaseKey	Can optionally be used to uniquely identify each Allocation record. **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	String	
DueDate	The date the part is required during the assembly process (not the supply's DueDate). **Default value**: Future	Date	
Model	The assembly model the allocation is associated with. For example, this might be different than the scheduled receipt model in cases where Order BOM logic is used. If left Null, then depending on configuration either the model specified in the **InputModel** field on the **ScheduledReceipt** record or the default model is used. **Referenced table:** Model (Nullable) **Note:** This field was added in Version 2016.2.	Reference	
Operation	The specific operation in the scheduled receipt's routing where this component part is used. The referenced operation can be in the routing's standard sequence or one of its parallel sequences. Note that this reference is only considered if the parent supply uses capacity-based variable lead time (**OrderPolicy.UseVarLeadTime** set to "CRP"). **Referenced table:** CRPOperation (Nullable) **Reference Syntax**: Operation.Id, Operation.Sequence.Id, Operation.Sequence.Routing.Id, Operation.Sequence.Routing.Site.	Reference	

Table 6.7: Allocation (Mfg) fields

Field name	Description	Data type	Key
Part	The part required for the manufacturing order. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	
QuantityPer	The flat quantity per between the scheduled receipt part and the allocation part. This should include any consideration for BOM scrap. For example, if the QuantityPer on the BOM record is 2 and Scrap is 20%, this field should be set to 2.4. This field should be populated for all substitutable allocations underneath a given scheduled receipt, and indicates the number of units of the allocation part required to build one unit of the scheduled receipt part. QuantityPer is used in calculating the InKit value, as well as in rolling up availability of the allocation for determining incremental availability of the scheduled receipt. For records that do not define substitutable allocations, this field can be ignored as RapidResponse calculates an effective quantity per based on the ratio between the allocation and the scheduled receipt. **Default value**: 0 For more information, see "Substitutable allocations" on page 1576.	Quantity	
Remaining Quantity	The amount that is still needed from inventory. **Default value**: 0	Quantity	
Scheduled Receipt	The scheduled receipt for which this allocation is reserved. **Referenced table:** ScheduledReceipt **Reference Syntax:** ScheduledReceipt.Line, ScheduledReceipt.Order.Type, ScheduledReceipt.Order.ID, ScheduledReceipt.Order.Site **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	
Shop	The shop floor location associated with this Allocation record. **Referenced table**: Shop (this reference is nullable in new installations of RapidResponse Version 10.1 and later).	Reference	

Table 6.7: Allocation (Mfg) fields

Field name	Description	Data type	Key
StartUnit	The unit number for the first item being made from this allocated material. **Default value**: -1	Integer	
SubstituteGroup	A reference to the **SubstituteGroup** table that is used to define groups of substitutable allocations underneath a scheduled receipt. All allocations that are assumed to be substitutable for one another underneath a given scheduled receipt must point to the same SubstituteGroup record. If the referenced SubstituteGroup record has a **SubstituteGroupType.ProcessingRule** value of "Substitute", then the availability of each substitutable allocation can be individually rolled up to the scheduled receipt. This allows for incremental availability on the scheduled receipt to be calculated based on when each substitute component supply is available. InKit calculations are also affected by substitutable allocations. **Referenced table**: SubstituteGroup For more information about using this field, see "Substitutable allocations" on page 1576.	Reference	
TotalQuantity	The amount that was needed. **Default value**: 0	Quantity	

Allocation table – calculated and set fields

The following describes the calculated and set fields in the Allocation table. For an introduction to this table, and descriptions of its input fields, see "Allocation table" on page 182.

Table 6.8: Calculated Allocation(Mfg) fields

Field name	Description	Data type	Key
EarliestExpiry Date	The earliest date that supply can expire if it is to be used in satisfying this demand. The value in this field depends on the expiry related settings for both the assembly on the referenced **ScheduledReceipt** record and the component being reserved by this allocation. It is calculated as follows: • If the assembly on the scheduled receipt has a **Type.ExpiryRule** of "Dependent", then this field inherits the **EarliestExpiryDate** from the scheduled receipt. • If the assembly on the scheduled receipt has a **Type.ExpiryRule** of "Self", then this field is calculated starting from the **DueDate** on this allocation and adding the assembly's **Part.MinimumShelfLife** value (in units of the assembly's **ExpiryCalendar**). • If the assembly on the scheduled receipt has a **Type.ExpiryRule** of "SelfReset", then this field is calculated starting from the **DueDate** on this allocation and adding the component's **Part.MinimumShelfLife** value (in units of the component's **ExpiryCalendar**).	Date	
EffQuantity	The effective quantity of this dependent demand. It equals the RemainingQuantity unless ScheduledReceipt.OriginalQuantity > 0. The EffQuantity is calculated to account for the new ScheduledReceipt.Quantity. Typically, assuming valid data, the value in this field should be greater than zero. A negative quantity in this field might be indicative of a bad data environment. For example, if using RapidResponse's method of automatically updating the allocation every time there is a change in the scheduled receipt, this might happen if the Quantity on the scheduled receipt was changed without also changing the OriginalQuantity on the scheduled receipt. **Note:** For information about how this field is calculated, see "Automatic allocation adjustment" on page 1399.	Quantity	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
ForecastQuantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	

Table 6.8: Calculated Allocation(Mfg) fields

Field name	Description	Data type	Key
InKit	The quantity of this allocation that remains in the allocation kit, but not yet allocated to completed supply units. This field is calculated as follows: `MAX (0, Allocation.TotalQuantity - Allocation.RemainingQuantity - ((ScheduledReceipt.TotalQuantity - ScheduledReceipt.Quantity) * EffQtyPer)` where `EffQtyPer` is calculated based on the ratio between the allocation and the scheduled receipt, unless the record represents a substitutable allocation in which case it is set to the input QuantityPer value on the Allocation record. If the record represents a substitutable allocation, the formula shown above is individually applied to each substitutable allocation under a scheduled receipt, starting with the allocation with the lowest RemainingQuantity, until the finished amount on the receipt has been fully distributed and consumed.	Quantity	
IsRecursive	Indicates whether recursive data has been found during processing. Bill records that have a BOMType.EffectivityRule value of 'Never' are ignored. Valid values are: • **Y**—recursive data has been found. The calculation skips the parts involved in the recursive relationship. • **N**—recursive data has not been found. **Note:** A self-allocation occurs when the part on the **Allocation** record is the same as the part on the referenced **ScheduledReceipt** record. This is an allowable form of recursion (returns "N") and represents a rework order where some assembly is being returned for rework.	Boolean	
MappedPool	The pool used in netting this demand. **Referenced table:** Pool	Reference	
ReschedDate	The date RapidResponse recommends the allocation should be due in order to satisfy the parent scheduled receipt. For example, this might differ from the provided DueDate based on rescheduling of the receipt. If the allocation is for a backflush scheduled receipt (for example, where SupplyStatus.State = 'BackflushAndAllocated'), this value is always set to 'Past'. If the allocation is for a scheduled receipt not processed by Netting, this is value is always set to 'Undefined'.	Date	
Unsatisfied Quantity	The portion of this demand that was left unsatisfied. This field is only applicable to allocations on scheduled receipts using transfer part sources.	Quantity	

AllotmentOverride table

This table holds allotment overrides which are used to manually reserve limited component supply to the higher level demands or supplies that require them. Typically, when faced with limited component supply, RapidResponse performs allotments based on things like due date, demand priority and fair or equal share logic. However, in some cases the desired allotment of limited supply cannot easily be achieved through RapidResponse processing rules and settings. In these cases, allotment overrides can be used to override the supply allotments RapidResponse would otherwise make and manually share or reserve components to help achieve the desired availability of top-level demands.

The **AllotmentOverride** table identifies and controls processing of each unique allotment override in RapidResponse. The specifics of each allotment override are then specified in the **AllotmentOverrideDetail** table which references the **AllotmentOverride** table and contains details such as the component the allotment override pertains to, as well as the amount of that component's supply to be reserved. Multiple AllotmentOverrideDetail records can point to a single AllotmentOverride values in cases where multiple components are being reserved together.

The supply reserved through a given AllotmentOverride and its associated AllotmentOverrideDetail record(s) is then ultimately allocated to those higher level demands or supplies which reference that AllotmentOverride value. The following tables have a reference to the AllotmentOverride table for this purposes:

- **IndependentDemand**—for reserving component supply to specific top-level demands.
- **ScheduledReceipt**—for reserving component supply to specific work orders.
- **PartCustomerTimePhasedAttributes**—for reserving component supply to S&OP consensus forecast demands for a given part and customer combination.
- **PartSource**—for reserving component supply to dependent demands with supply generated from a given part source.

Note: This table is primarily intended to be populated using the Allotment Overrides workbook. If necessary, however, values can be imported into this table. For more information about allotment overrides, see "Allotment overrides" on page 1853.

Table 6.9: AllotmentOverride (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this allotment override.	String	
Id	A unique string identifier for this allotment override record. For example, an Id might combine the Customer and Site the component supply is being reserved for. **Note:** If the record is created using the AllotmentOverride workbook, this Id set to a string determined by the selected Allotment Override Id Template.	String	Yes

Table 6.9: AllotmentOverride (Mfg) fields

Field name	Description	Data type	Key
Sequence	A number that can be used to set the processing order when there are multiple allotment overrides reserving supply of the same component part (records with lower values are given priority). For example, if there are multiple AllotmentOverride records reserving a given component's supply to different demands, and there is not actually enough of the reserved component to full satisfy all those demands on time, RapidResponse attempts to satisfy demands referencing AllotmentOverride record with the lowest Sequence values first.	Integer	
Type	A reference that sets processing rules for this allotment override. For example, it determines if this record and its associated details are enabled or disabled in analytic calculations. **Referenced table:** AllotmentOverrideType	Reference	

AllotmentOverride table – calculated and set fields

The following describes the calculated and set fields in the AllotmentOverride table. For an introduction to this table, and descriptions of its input fields, see "AllotmentOverride table" on page 187.

Table 6.10: AllotmentOverride calculated (Mfg) fields

Field name	Description	Data type	Key
Allotment OverrideDetails	Set of AllotmentOverrrideDetails records that contain details of reserved component supply associated with this allotment override.	Set	
Independent Demands	Set of IndependentDemand records that are reserving component supply from this allotment override.	Set	
PartCustomer TimePhased Attributes	Set of PartCustomerCustomerTimePhasedAttributes records that are reserving component supply from this allotment override.	Set	
PartSources	Set of PartSource records that are reserving component supply from this allotment override.	Set	
Scheduled Receipts	Set of ScheduledReceipt records that are reserving component supply from this allotment override.	Set	

AllotmentOverrideDetail table

The **AllotmentOverrideDetail** table contains details of allotment overrides. These are used in cases where there is both limited component supply and a need to override the supply allocations calculated by RapidResponse in order to achieve the desired component distribution and availability on higher level demands or supplies. For example, this table specifies the component whose supply is to be reserved along with the amount of supply to be reserved. Each record in this table also contains a reference to **AllotmentOverride** table which is used to identify each allotment override in RapidResponse and ties the component supply identified in this table to the top-level demands or supplies where they are to be allocated.

Although this tables allows for a specified quantity of a component to be reserved to the demands or supplies associated with the referenced allotment override, it does not provide the ability to choose the specific supply that will provide the reserved quantity.

Note: For more information about allotment overrides, see "Allotment overrides" on page 1853. RapidResponse also includes the **AllotmentOverrides** workbook to simplify the process of creating and maintain allotment overrides. For more information about this workbook see the *RapidResponse Applications Guide*.

Table 6.11: AllotmentOverrideDetail (Mfg) fields

Field name	Description	Data type	Key
Allotment Override	A reference to the AllotmentOverride value the details in this record pertain to. Supply reserved by this detail record can be used by demands or supplies that point to the referenced AllotmentOverride value (for example, from an IndependentDemand record). **Referenced table:** AllotmentOverride	Reference	Yes
Component	A reference to the component part whose supply is being reserved for the allotment override. The component can be multiple levels down the product structure from the assembly part the supply is reserved for. For example, supply of a bottom level component might be reserved to demand from a top level assembly. In these cases, RapidResponse ensures the allotment override details are propagated up the product structure so that the component supply is ultimately used in satisfying the assembly demand. **Referenced table:** Part	Reference	Yes

Table 6.11: AllotmentOverrideDetail (Mfg) fields

Field name	Description	Data type	Key
Date	The allotment override date for the component. In order to find component supply to reserve for the allotment override, RapidResponse starts looking on this date, and then moves back through earlier date buckets as required. If sufficient supply can still not be found, RapidResponse will then look in buckets after this date. If multiple supplies on a given date are available to provide the reservation, RapidResponse will choose which one to use (that is, you cannot specify the specific order to provide the quantity). Also, when **AllotmentOverride.Type** is set to "AfterRunDate", the date provided here sets when this record is effective in RapidResponse analytic calculations (effective as long as this date is on or after RunDate). For more information, see "AllotmentOverrideType table" on page 671.	Date	
Description	A description of this allotment override detail.	String	
Id	An optional string identifier. This field is used to uniquely identify records in cases where a given Component and AllotmentOverride combination have multiple detail records (that is, with different date values). Typically, this might be set to a string displaying the date associated with the record. Otherwise, if there is only one detail record for a given Component and AllotmentOverride combination, this field can be left blank.	String	Yes
Quantity	The amount of the component part supply to be reserved for the allotment override. If the amount specified is more than is actually required by the demands or supplies associated with the allotment override, the difference is subsequently made available to other demands and supplies.	Quantity	

AlternatePart table

This table is used to define interchangeable and substitute part relationships. Each record identifies a PrimaryPart and a SubstitutePart, and the setting in the Type field indicates if the parts are interchangeable or substitutes.

In an interchangeable part relationship, all parts are considered equivalent in form, fit, and function. Demand for one part in an interchangeable part group can be satisfied by supply from any of the other parts in the group, and no preference is given as to which part's supply satisfies the demand. Also, planned orders and calculated results are only generated for the primary part in an interchangeable part group. For more information about interchangeable parts, see "Interchangeable parts" on page 1545.

In a substitute part relationship, a substitute part can be used in a place of the primary part, but the primary part cannot be used in place of a substitute part. Substitute parts are only used when not

enough of the primary part can be found. Also, planned orders and calculated results can be generated for both the primary and substitute part(s) in a substitute part relationship. For more information about substitute parts, see "Global part substitution" on page 1551.

Table 6.12: AlternatePart (Mfg) fields

Field name	Description	Data type	Key
BaseKey	Along with the PrimaryPart and SubstitutePart fields, uniquely identifies the interchangeable or substitute part grouping. This field might contain, for example, a manufacturer name, the name of a group of parts, or a blank value.	String	Yes
EffectiveInDate	Date on which this AlternatePart record becomes effective. Along with EffectiveOutDate and the setting in Type.EffectivityRule, this field defines the period over which this record is effective. Only applicable to substitute part relationships (interchangeable part relationships are always considered effective).	Date	
EffectiveOutDate	Date on which this AlternatePart record is no longer effective. Along with EffectiveInDate and the setting in Type.EffectivityRule, this field defines the period over which this record is effective. Only applicable to substitute part relationships (interchangeable parts relationships are always considered effective).	Date	
Pool	For substitute part relationships, defines the applicable pool. Pools are used to enable customer-specific part substitution. **Referenced table:** Pool	Reference	
PrimaryPart	The part that either interchangeable or substitute parts are defined for. **Referenced table**: Part	Reference	Yes

Table 6.12: AlternatePart (Mfg) fields

Field name	Description	Data type	Key
Sequence	For substitute part relationships, this value is used to define a preference between eligible substitute parts with lower values being used first. This value is also compared against the **PrimarySubstitutionSequence** value on the primary Part record to determine if existing supply on a given substitute or the primary part is preferred. If this value is greater than or equal to the value in **PrimarySubstitutionSequence**, then existing supply on the primary part is preferred. However, if this value is strictly less than the value in **PrimarySubstitutionSequence**, then existing supply of this substitute is used before that of the primary (and in such cases, new planned orders will never be generated on the substitute to satisfy demand from the primary part). If multiple parts have the same Sequence value, then the following sort order is used to determine which substitutes are used first: • Pool specific records. • FirstEffectiveDate (earliest). • SubstitutePart.Name and SubstitutePart.Site (alphabetical) **NOTE**: If using global part substitution along with multi-level search logic, then some multi-level criteria take precedence over the Sequence value when determining which substitute parts are used. For example, if two substitutes can satisfy a demand equally well and the lowest level of component supply used by each comes from input supply, then preference is given to the substitute that provides that supply at a higher level of the product. That is, substitutes with shorter paths to component materials are preferred even if they do not have the lower Sequence value.	Integer	
SubstitutePart	That part that can be used either interchangeably or as a substitute for the primary part. **Referenced table**: Part	Reference	Yes
Type	A reference that determines if this record represents a substitute or interchangeable part relationship. If it is a substitute part relationship, then this reference includes other rules that affect substitution calculations. **Referenced table**: AlternatePartType	Reference	

AlternatePart table – calculated and set fields

The following describes the calculated and set fields in the AlternatePart table. For an introduction to this table, and descriptions of its input fields, see "AlternatePart table" on page 190.

Table 6.13: AlternatePart calculated (Mfg) fields

Field name	Description	Data type	Key
FirstEffectiveDate	The date when this AlternatePart record first becomes effective. Represents an adjusted EffectiveInDate that takes Type.EffectivityRule into account.	Date	
IsRecursive	Indicates whether this pair of parts is recursive, or if the parts create a recursive product structure. A pair is recursive if both parts are primary parts in a group, and both are substitutes of each other. For example, if Part A is a substitute for Part B and Part B is a substitute for Part A, the pair is recursive. A product structure is recursive if a part is a component of itself. For example, assume you have a product structure for Part A, with component Part C, which has component Part B. Part A is interchangeable with Part B, which means Part A is a component of itself, and the product structure is recursive. Recursive pairs and product structures are not valid, and are ignored when interchangeable and substitute part relationships are considered. Valid values are: • **Y**—The pair is recursive. • **N**—The pair is not recursive.	Boolean	
LastEffectiveDate	The final date when this AlternatePart record is effective. Represents an adjusted EffectiveOutDate that takes Type.EffectivityRule into account.	Date	

AlternateRouting table

The **AlternateRouting** table is used as a place holder for alternate routings. It links routings to parts for a list of alternate routings. To use an alternate routing, the Routing field on the Part table must be changed to one of the alternate routings listed here.

Table 6.14: AlternateRouting (Mfg) fields

Field name	Description	Data type	Key
Part	The name of the part. **Referenced table:** Part	Reference	
Routing	The routing Id. **Referenced table:** Routing	Reference	

Assignment table

The assignment table links the RapidResponse users to the activities that they are tasked with performing. It defines information such as the date when the user begins work on an activity and the user's assignment status.

Table 6.15: Assignment (ProcOrch) fields

Field name	Description	Data type	Key
Activity	The activity associated with this assignment. **Referenced table:** Activity	Reference	
FinishDate	The date the assignment is complete. An assignment is complete when a user changes their assignment status to Finished.	Date	
Id	A unique id that identifies this assignment.	String	Yes
StartDate	The date work on the assignment starts. An assignment is started when a user changes their assignment status to InProgress or Finished.	Date	
Status	The status of the assignment. Valid values are: • **NotStarted** • **InProgress** • **Finished**	Enum	
UserId	The RapidResponse user ID of the user linked to this of this assignment.	String	

Assignment table – calculated and set fields

The following describes the calculated and set fields in the Assignment table. For an introduction to this table, and descriptions of its input fields, see "Assignment table" on page 194.

Table 6.16: Assignment (ProcOrch) fields

Field name	Description	Data type	Key
ActualFinishDate	If the **Status** is "Finished" and **FinishDate** is defined, then the later of the **FinishDate** or **ActualStartDate** is used. If the **Status** is "Finished" but **FinishDate** is undefined, then the later of the the **ExpectedFinishDate** on the activity or **ActualStartDate** is used.	Date	
ActualStartDate	If the **Status** is "Started" or "Finished", then the **StartDate** is used. If the **Status** is "Started" or "Finished", but no **StartDate** is defined, then the **ExpectedStartDate** of the activity is used.	Date	

Assumption table

This table contains details of external information or "assumptions" on which your S&OP plans are based. For example, an assumption from the marketing group might note an expected change in market share for a particular product line.

The Assumption table supports Sales and Operations Planning in RapidResponse. Values in this table are added and maintained using the **New Assumption** and **Edit Assumption** dialog boxes in RapidResponse. These dialog boxes can only be used in workbooks compatible with the PartCustomer, Part, PartSource, and Constraint tables. For more information about assumptions, see the *RapidResponse Applications Guide*.

Table 6.17: Assumption (Mfg) fields

Field name	Description	Data type	Key
AssignTo	The User ID of the user the assumption is assigned to in RapidResponse. This is the user who is expected to take some action on this assumption (for example, to review it).	String	
Created	The date and time at which this assumption record was created.	DateTime	
Creator	The User ID of the user who created the assumption in RapidResponse.	String	
DueDate	The date that any tasks associated with this assumption should be completed by.	Date	
EffectiveIn	The date on which this assumption become effective.	Date	
EffectiveOut	The date on which this assumption is assumed to no longer be effective.	Date	

Table 6.17: Assumption (Mfg) fields

Field name	Description	Data type	Key
Id	A unique identifier for this assumption.	String	Yes
LastModified	The date and time at which this assumption record was last modified.	DateTime	
Status	A reference to the current status of this assumption. For example, an assumption status might be "open", "closed" or "invalid". **Referenced table:** AssumptionStatus	Reference	
Target	A reference that identifies the active hierarchy and filter settings in the workbook at the time this assumption record was created. **Referenced table:** ReferenceForecastTarget	Reference	
Text	A description of the assumption.	String	
Type	A reference to the type value used to categorize this assumption. For example, type values might be used to indicate the department in your company which the assumption is associated with. **Referenced table**: AssumptionType	Reference	

Assumption table – calculated and set fields

The following describes the calculated and set fields in the Assumption table. For an introduction to this table, and descriptions of its input fields, see "Assumption table" on page 195.

Table 6.18: Calculated Assumption (Mfg) fields

Field name	Description	Data type	Key
AssumptionBy Constraints	The set of AssumptionByConstraint records associated with this assumption.	Set	
AssumptionByParts	The set of AssumptionByPart records associated with this assumption.	Set	
AssumptionBy PartCustomers	The set of AssumptionByPartCustomer records associated with this assumption.	Set	
AssumptionBy PartSources	The set of AssumptionByPartSource records associated with this assumption.	Set	

AssumptionByConstraint table

This table associates assumptions with the constraints they apply to. A given Assumption record can apply to many constraints, and one Constraint record can have many assumptions associated with it.

The **AssumptionByConstraint** table supports Sales and Operations Planning in RapidResponse. Values in this table are added and maintained using the **New Assumption** and **Edit Assumption** dialog boxes. These dialog boxes might be available from select RapidResponse workbooks that are compatible with Constraint-based filters and include hierarchies. For more information about assumptions, see the *RapidResponse Applications Guide*.

Table 6.19: AssumptionByConstraint (Mfg) fields

Field name	Description	Data type	Key
Assumption	A reference to the assumption belonging to this assumption and constraint combination. **Referenced table:** Assumption	Reference	Yes
Constraint	A reference to the constraint belonging to this assumption and constraint combination. **Referenced table:** Constraint	Reference	Yes

AssumptionByPart table

This table associates assumptions with the parts they apply to. A given Assumption record can apply to many parts, and one Part record can have many assumptions associated with it.

The **AssumptionByPart** table supports Sales and Operations Planning in RapidResponse. Values in this table are added and maintained using the **New Assumption** and **Edit Assumption** dialog boxes. These dialog boxes might be available from select RapidResponse workbooks that are compatible with Part-based filters and include hierarchies. For more information about assumptions, see the *RapidResponse Applications Guide*.

Table 6.20: AssumptionByPart (Mfg) fields

Field name	Description	Data type	Key
Assumption	A reference to the assumption belonging to this assumption and part combination. **Referenced table:** Assumption	Reference	Yes
Part	A reference to the part belonging to this assumption and part combination. **Referenced table:** Part	Reference	Yes

AssumptionByPartCustomer table

This table associates assumptions with the part customers they apply to. A given Assumption record can apply to many part customers, and one PartCustomer record can have many assumptions associated with it.

The **AssumptionByPartCustomer** table supports Sales and Operations Planning in RapidResponse. Values in this table are added and maintained using the **New Assumption** and **Edit Assumption** dialog boxes. These dialog boxes might be available from select RapidResponse workbooks that are compatible with PartCustomer-based filters and include hierarchies. For more information about assumptions, see the *RapidResponse Applications Guide*.

Table 6.21: AssumptionByPartCustomer (Mfg) fields

Field name	Description	Data type	Key
Assumption	A reference to the assumption belonging to this assumption and part customer combination. **Referenced table:** Assumption	Reference	Yes
PartCustomer	A reference to the part customer belonging to this assumption and part customer combination. **Referenced table:** PartCustomer	Reference	Yes

AssumptionByPartSource table

This table associates assumptions with the part sources they apply to. A given Assumption record can apply to many part sources, and one PartSource record can have many assumptions associated with it.

The **AssumptionByPartSource** table supports Sales and Operations Planning in RapidResponse. Values in this table are added and maintained using the **New Assumption** and **Edit Assumption** dialog boxes. These dialog boxes might be available from select RapidResponse workbooks that are compatible with PartSource-based filters and include hierarchies. For more information about assumptions, see the *RapidResponse Applications Guide*.

Table 6.22: AssumptionByPartSource (Mfg) fields

Field name	Description	Data type	Key
Assumption	A reference to the assumption belonging to this assumption and part source combination. **Referenced table:** Assumption	Reference	Yes
PartSource	A reference to the part source belonging to this assumption and part source combination. **Referenced table:** PartSource	Reference	Yes

AssumptionStatus table

This table contains all valid status values defined in your company to indicate the state of an assumption. For example, status values might indicate if an assumption is open or closed.

The AssumptionStatus table supports Sales and Operations Planning in RapidResponse. This table should be configured during initial RapidResponse deployment.

Table 6.23: AssumptionStatus (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this assumption status.	String	
Value	A name for this assumption status. Examples of assumption status value might include “Open”, “Closed”, and “Under Review”. This field is referenced from the Status field on the Assumption table.	String	Yes

AssumptionStatus table – calculated and set fields

The following describes the calculated and set fields in the AssumptionStatus table. For an introduction to this table, and descriptions of its input fields, see "AssumptionStatus table" on page 199. This table should be configured during initial RapidResponse deployment

Table 6.24: Calculated AssumptionStatus (Mfg) fields

Field name	Description	Data type	Key
Assumptions	The set of Assumption records associated with this assumption status.	Set	

AssumptionType table

This table contains all valid type values defined in your company to categorize assumptions. For example, type values might indicate the department in your company that the assumption is associated with.

This AssumptionType table should be configured during initial RapidResponse deployment.

Table 6.25: AssumptionType (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this assumption type.	String	
Value	A unique identifier (name) for this assumption type. Examples of assumption status value might include "Revenue", or "Action". This field is referenced from the Type field on the Assumption table.	String	Yes

AssumptionType table – calculated and set fields

The following describes the calculated and set fields in the AssumptionType table. For an introduction to this table, and descriptions of its input fields, see "AssumptionType table" on page 199.

Table 6.26: Calculated AssumptionType (Mfg) fields

Field name	Description	Data type	Key
Assumptions	The set of Assumption records associated with this assumption type.	Set	

AttributeMap table

The **AttributeMap** table holds attribute rules that pair named attributes and their values together for use with certain analytic calculations. Attributes are custom data model elements that identify specific characteristics of demands and/or supplies and rely on calculated field expressions to identify those data characteristics in a particular table.

Attributes rules can be used to provide finer control over any or all of the following calculations in RapidResponse:

- **Supply Allotment**— used to ensure that demands with given attribute values can only be satisfied by supplies that have attribute values meeting specified criteria. For example, demands from a given part customer might only be able to accept supply of the part if was assembled at a particular location or contains components produced by select suppliers.
- **Forecast consumption**—used to ensure that forecasted demands can only be consumed by actual demands with matching attribute values. For example, a forecast demand associated with a particular customer or region might only be eligible to be consumed by actual demands from that same customer or region.
- **Forecast adjustments**—used to ensure that forecast adjustments and overrides with attributes specified on them are only made against consensus forecast records with matching attributes. For example, demand attributes might be used to adjust consensus forecast values for a particular customer (this adjustment might later influence supply selection if the same attribute is defined for supply allotments).

For each rule in this table, the **FromAttributeName** and **FromAttributeValue** fields must be used to specify a demand-based attribute and a value to associate with it, the **ToAttributeName** and

ToAttributeValue fields can be used specify a supply-based attribute and one or values to associate with it (if applicable).

The **AttributeMap** table is not shown in the *RapidResponse Data Model for Import* poster. Values are typically added to this table using the standard Attribute Rules workbook included with RapidResponse. For example, different attribute values might be specified in different scenarios using this workbook as part of a simulation exercise. By default, the Attribute Rules workbook is available only to data administrators, but it might be shared to other users as required. For more information about this workbook, see the *RapidResponse Applications Guide*.

Table 6.27: AttributeMap (Core) fields

Field name	Description	Data type	Key
BaseKey	A string value used to uniquely identify each AttributeMap record.	String	Yes
Enabled	Specifies if the attribute rule defined by this record is used by RapidResponse analytics: • **N**—this rule is ignored by analytic calculations. • **Y**—this rule is eligible to be used by analytic calculations (usage further depends on the Type setting). **Default value**: N	Boolean	
FromAttribute Name	The name of a demand attribute to use with this rule. For example, this might refer to an attribute that identifies customers or part and customer combinations. It is recommended to specify attribute names in their fully namespace qualified format (that is, namespaceName::attributeName) to avoid potential issues if the same attribute name is ever defined in multiple namespaces.	String	

Table 6.27: AttributeMap (Core) fields

Field name	Description	Data type	Key
FromAttribute Value	A single value to associate with the demand attribute specified for this rule. For example, if the attribute specified in the FromAttributeName field identifies customers, then this field might contain the name of a particular customer that has some attribute-based qualifications or limitations on the supplies it will accept. Note that this is a string field, however the values provided should resolve to the data type of the demand attribute itself. Depending on the data type of the attribute identified in the FromAttributeName field, the following should be kept in mind: • **Boolean**—only Y (yes) and N (no) values are permitted. • **Date**—dates can be specified in any of the date formats supported in RapidResponse (Past, Future, Today and MRPDate keywords can also be used). • **Integer / Quantity**—numeric values can be specified with or without leading plus (+) or minus (-) signs, and optional decimal point (quantities only). • **String**—string values can be specified directly or enclosed in matching single, double, or start/end quotes. To escape a character in a string, use the backslash character (for example, 'AB\'C').	String	
ToAttribute Name	The name of the supply attribute, to use in this rule. For example, this might refer to an attribute used to identify plant locations or component suppliers. It is recommended to specify attribute names in their fully namespace qualified format (that is, namespaceName::attributeName) to avoid potential issues if the same attribute name is defined in multiple namespaces. **Note:** If this rule is only being used for forecast consumption and/or adjustment but not supply allotment (that is, has **Type.SupplySelection** set to "N"), then any valid attribute name can be specified here. For example, the name of the demand attribute might be used. In such cases, the **ToAttributeValue** field can then be left blank.	String	

Table 6.27: AttributeMap (Core) fields

Field name	Description	Data type	Key
ToAttribute Value	One or more values to associate with the supply attribute, if any, specified for this rule. For example, if the attribute specified in the ToAttributeName field identifies component suppliers, then this field might contain a delimited list of all qualified suppliers whose components can be used in end-items provided to a particular part customer. Note that this is a string field, however the values provided should resolve to the data type of the supply attribute itself and can also include certain elements of the search syntax used in RapidResponse worksheets as follows: • **Boolean**—only Y (yes) and N (no) values are permitted. • **Date**—dates can be specified in any of the date formats supported in RapidResponse (Today and MRPDate keywords are not supported). Greater than (>, >=) and less than (<, <=) symbols can be used to indicate values before or after a given date, and two dots can be used to indicate a date range (for example, 20120801..20120831). • **Integer / Quantity**—numeric values can be specified with or without leading plus (+) or minus (-) signs, and can include an optional decimal point (quantities only). Greater than (>, >=) and less than (<, <=) symbols can be used to indicate values above or below a specified value, and two dots can be used to indicate a quantity range (for example, 500..2000). • **String**—string values can be specified directly or enclosed in matching single, double, or start/end quotes. A list of string values can be specified by using a comma separator (for example, 'AB', 'CD', 'XYZ') and any value within the list will then be considered an attribute match. To indicate a match occurs only if the specified attribute values are not found, use <> (for example, <> ABC). To escape a character in a string, use the backslash character (for example, 'XY\'Z').	String	
Type	A reference to the type value that defines how the attribute rule defined on this record is interpreted and used. Based on the settings defined on the referenced type, the attribute rule might be used during supply allotment, forecast consumption, and/or forecast adjustment calculations. **Referenced table:** AttributeMapType	Reference	

Table 6.27: AttributeMap (Core) fields

Field name	Description	Data type	Key
Weight	If this attribute rule is to be used in supply allotment calculations, then this field can be used to specify and assign a weight to demands that have values matching those defined in the FromAttributeValue field. This weight is then used as part of the criteria in determining the order in which demands are processed and hence the order in which they receive potentially scarce supplies. For example, it can be used to ensure the demands associated with specified demand attribute values, such as a customer, are given priority over others. RapidResponse also automatically calculates a weight for each record which is applied to matching demands when this field is set to 0 (zero). This is the default behavior and ensures that a higher weight is assigned to records that are likely to have fewer attribute matches, and a lower weight is assigned to records that are likely to have more attribute matches. For example, an attribute rule that defines a single acceptable supply source would be assigned a higher weight than a rule that defines lists of acceptable suppliers. **Default value**: 0 **Note:** For more information about how weights are calculated and used, see "Attribute weights" on page 1827.	Quantity	

AttributeMap table – calculated and set fields

The following describes the calculated and set fields in the AttributeMap table. For an introduction to this table, and descriptions of its input fields, see "AttributeMap table" on page 200.

Table 6.28: Calculated AttributeMap (Core) fields

Field name	Description	Data type	Key
ErrorMessage	If a record has its **Enabled** field set to "Y", this field returns a string indicating any errors in the map definition. For example, an attribute name might be misspelled or the value provided for an attribute might be incompatible with its data type. An attribute rule will not be used in analytic calculations until all reported errors have been fixed.	String	

BaselineConflict table

The **BaselineConflict** table contains details of task scheduling conflicts that existed at the point when a project was baselined (copied to the baseline scenario). Task conflicts can occur if a constraint or material order link causes a task to move in the opposite direction from which its project is being scheduled in such a manner that not all parameters or limits placed on the task can be simultaneously

respected. Each record in this table identifies the task and baselined project in which the conflict existed along with details of the constraint or material order requirement that caused the conflict.

For details of actual task conflicts in projects, see "Conflict table" on page 963.

Table 6.29: BaselineConflict (ProjMgmt) fields

Field name	Description	Data type	Key
Constraint	If the conflict in the baselined project was caused by a task constraint, this indicates the type of constraint (for example, "FinishNoEarlierThan").	String (Enum)	
Date	The date associated with the source of the task conflict in the baseline project. If the conflict was caused by a task constraint, then the constraint date is reported. Otherwise, if the conflict was caused by a material order requirement, then the supply or demand available date plus any applicable offset is reported.	Date	
Id	A unique string identifier for this record. This value is a concatenation of the baselined project name, project site, task Id that had the conflict, and certain details of the constraint or material order link that caused the conflict.	String	Yes
Task	A reference to the task that had a conflict at the point when the project was baselined. **Referenced table:** Task	Reference	
TaskLink	If the conflict in the baselined project was caused by a material order link, this contains a reference to the BaselineTaskLink record having details of the demand or supply order link that caused the conflict in the baselined project. **Referenced table:** BaselineTaskLink	Reference	Yes
Type	Indicates the phase in task scheduling where the conflict was detected in the baseline project. Because project tasks can be scheduled either from the project start date or the project finish date (based on a **ProjectType** setting), conflicts can exist in either direction. Valid values are: • **Primary**—Indicates that the task conflict exists as the project is currently scheduled. For example, if the project has a **ProjectType.ScheduleRule** value of "FromStart", then this value indicates the conflict occurs when scheduling tasks from the project start date. This type of conflict typically requires investigation. • **Secondary**—Indicates a conflict that does not present itself as the project is currently scheduled. For example, if the project has a **ProjectType.ScheduledRule** of "FromStart", then this value indicates the conflict would occur if scheduling tasks from the project finish date. This type of conflict can typically be ignored.	String (Enum)	

Table 6.29: BaselineConflict (ProjMgmt) fields

Field name	Description	Data type	Key
Source	Indicates whether the scheduling conflict in the baselined project was caused by a task constraint or a material order requirement from a demand or supply order. Valid values are: • Constraint • Demand • Supply	String (Enum)	

BaselinePlannedOrder table

The **BaselinePlannedOrder** input table lists planned orders imported from an ERP system. This table supports data validation and is used for internal purposes only.

Table 6.30: BaselinePlannedOrder (Mfg) fields

Field name	Description	Data type	Key
DueDate	Due date of the imported planned order.	Date	
Model	The model that applies to this imported planned order. Ignored in data validation if either **Part.MUEPoolNettingType.ModelRule** is set to "Ignore" or Model-Unit Effectivity is not enabled. **Referenced table:** Model	Reference	
Part	The part the imported planned order is for. **Referenced table:** Part	Reference	
Pool	The pool that applies to this imported planned order. Ignored in data validation if either **Part.MUEPoolNettingType.ModelRule** is set to "Ignore" or Pool is not enabled. **Referenced table:** Pool	Reference	
Quantity	The amount of the imported planned order.	Quantity	
StartDate	The start date for the imported planned order.	Date	
StartUnit	The start unit that applies to this imported planned order. Ignored in data validation if either **Part.MUEPoolNettingType.UnitRule** is set to "Ignore" or Model-Unit Effectivity is not enabled.	Integer	

BaselineTaskLink table

The **BaselineTaskLink** table contains details of any material supply or demand links defined for a project's tasks at the point when the project was baselined (copied to the baseline scenario), along with the associated part link. For each supply or demand link associated with a task in a baselined project, details of that link are reported such as the order id and line number, the quantity assigned to the task, and whether or not the order's availability impacts the task finish date. These details can be used for comparison against the material supply or demand links defined in the current or working version of the project.

Table 6.31: BaselineTaskLink (ProjMgmt) fields

Field name	Description	Data type	Key
ApplyConstraint	Indicates if the available date of the linked demand or supply can potentially constrain the task start or finish date. Valid values as taken from the **ApplyConstraint** field on either the **TaskDemandLink** or **TaskSupplyLink** tables are. • **N** • **Y**	Boolean	
Assigned Quantity	If this record pertains to an order requirement in the baselined project, then this is the amount of the order assigned to this task. That is, the value from the **AssignedQuantity** field on either the **TaskDemandLink** or **TaskSupplyLink** record.	Quantity	
AvailableDate	If this record pertains to an order requirement in the baselined project, then this is the date the order is expected to be available. That is, the value from the **AvailableDate** field on either the **IndependentDemand** or **ScheduledReceipt** record.	Date	
Critical	If this record pertains to an order requirement in the baselined project, then this indicates if the order's availability impacts the task finish date. That is, the value from the **Critical** field on either the **TaskDemandLink** or **TaskSupplyLink** record.	Boolean	
Id	A unique string identifier for this record. When a project is baselined in RapidResponse, this value is a concatenation of the task's Id, project name, along with details of the part, demand, or supply that were linked to the task in the associated task link record.	String	Yes
Line	If this record pertains to an order requirement in the baselined project, then this indicates the specific line item of that order. That is, the value from the **Line** field on either the **IndependentDemand** or **ScheduledReceipt** record.	String	

Table 6.31: BaselineTaskLink (ProjMgmt) fields

Field name	Description	Data type	Key
MaterialCost	The cost of the parts linked to the task at the point when the project was baselined. Based on the corresponding **MaterialCost** field in either the **TaskPartLink**, **TaskDemandLink**, or **TaskSupplyLink** table.	Money	
Offset	If this record pertains to an order requirement in the baselined project, then this represents the number of days to add to orders available date when determining either the task's start date or finish date (based on the setting in the **Rule** field). Taken from the **Offset** field on the associated **TaskDemandLink** or **TaskSupplyLink** record.	Quantity	
OrderId	If this record pertains to an order requirement in the baselined project, then this indicates the specific line item of that order. That is, the value from the **Order.Id** reference on either the **IndependentDemand** or **ScheduledReceipt** record.	String	
OrderSite	If this record pertains to an order requirement in the baselined project, then this indicates the specific line item of that order. That is, the value from the **Order.Site** reference on either the **IndependentDemand** or **ScheduledReceipt** record.	String	
OrderType	If this record pertains to an order requirement in the baselined project, then this indicates the specific line item of that order. That is, the value from the **Order.Type** reference on either the **IndependentDemand** or **ScheduledReceipt** record.	String	
PartName	The name of the part the task had a material requirement for when the project was baselined. That is, the **Part.Name** value referenced through the **TaskPartLink** record.	String	
PartSite	The site associated with the part that the task had a material requirement for when the project was baselined. That is, the **Part.Site.Value** value referenced through the **TaskPartLink** record.	String	
Quantity	The required quantity of the part associated with the original task link this record pertains to (that is, the **TaskPartLink.Quantity** value).	Quantity	

Table 6.31: BaselineTaskLink (ProjMgmt) fields

Field name	Description	Data type	Key
Rule	If this record pertains to an order requirement in the baselined project, then this defines how, if at all, the order's availability constrains the task completion. Taken from the **Rule** field on the associated **TaskDemandLink** or **TaskSupplyLink** record with one of the following valid values: • FinishDate • None • StartDate	String (Enum)	
Task	A reference to a task that had a material supply or demand requirement defined when the project was baselined. The name of the baselined project this task pertains to can also be accessed through this reference. **Referenced table:** Task	Reference	
Type	Indicates the type of baselined material requirement for the task that this record pertains to. Valid values are: • **Demand**—this record pertains to an independent demand requirement for the task (as set through the **TaskDemandLink** table). • **Part**—this record pertains to a part requirement for the task (as set through the **TaskPartLink** table). • **Supply**—this record pertains to a scheduled receipt requirement for the task (as set through the **TaskSupplyLink** table).	String (Enum)	

BestFitProfile table

The **BestFitProfile** table is used to define groups of statistical models for consideration in best fit statistical forecasting.

Named profiles in this table are referenced by records in the **BestFitProfileDetail** table each of which identifies a particular statistical model and contains parameter values that override corresponding values on the **ForecastItemParameters** table. Thus, each profile is made up of the models and associated parameter values that reference it from **BestFitProfileDetail** table. This can include multiple instances of the same model in a given profile. For example, the "MovingAverage" model might be added to the same profile multiple times with each instance specifying a different range over which to collect data for use in calculating that moving average.

The **BestFitProfile** table is also referenced by the **ForecastItemParameters** table. This allows forecast items using the "BestFit" method to evaluate only select models during the best fit process. For example, if a known subset of models are known to be generally applicable for a given forecast item, then a profile consisting of the appropriate models can be defined and referenced by that item. This can save processing time associated with evaluating models that are known to not fit well with the item's data. Otherwise, if the **ForecastItemParameters.BestFitProfile** reference is Null, then all RapidResponse forecast models (other than the optional "Rforecast" model) are considered during the best fit process and the applicable model parameters are taken from the **ForecastItemParameters** record.

Table 6.32: BestFitProfile (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this profile. For example, this might describe the group of models that belong to the profile.	String	
Name	A unique identifier for this profile. This value is referenced from the **ForecastItemParameters** and **BestFitProfileDetail** tables.	String	Yes

BestFitProfile table – calculated and set fields

The following describes the calculated and set fields in the **BestFitProfile** table. For an introduction to this table, and descriptions of its input fields, see "BestFitProfile table" on page 209.

Table 6.33: Calculated BestFitProfile (Mfg) fields

Field name	Description	Data type	Key
BestFitProfile Details	Set of **BestFitProfileDetail** records that belong to this profile.	Set	
ForecastItem Parameters	Set of **ForecastItemParameters** records that use this profile.	Set	

BestFitProfileDetail table

The **BestFitProfileDetail** table holds the detailed records that make up each best fit profile.

Each record in this table specifies a given statistical forecast model, select model parameters that override corresponding values in the **ForecastItemParameters** table, and contains a reference to the **BestFitProfile** table. Together, all records that reference the same **BestFitProfile** record define the subset of statistical models that belong to the profile. Thus, only that subset of models is considered during best fit forecasting for items referencing the profile through **ForecastItemParameters.BestFitProfile**. Otherwise, if **ForecastItemParameters.BestFitProfile** is Null, then all standard RapidResponse forecast models are evaluated if the item uses the "BestFit" method and the corresponding model parameters in the **ForecastItemParameters** table are used instead. For more information, see ForecastItemParameters table.

Note: There may be instances where you want a forecast item to reference a group of statistical models that are defined in **BestFitProfile**, but you do not want to use the model parameter values that are associated with the profile in the **BestFitProfileDetail** table. To do this, you can set **ForecastItemParameters.IgnoreBestFitProfileDetailParameters** to **"True"** to ignore the model parameter values in the **BestFitProfileDetail** table and apply the values from **ForecastItemParameters** instead. See "IgnoreBestFitProfileDetailParameters" on page 302.

Note: Multiple records for a given statistical model can be defined and associated with the same entry in the **BestFitProfile** table. For example, if the "HoltWintersMethod" is used, there might be a requirement to evaluate the model using different seasonal interval counts. Similarly, a given model might be evaluated multiple times using different amounts of historical data in each instance.

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
AutoRegressive Terms	Specifies the auto-regression terms used in an ARIMA forecast calculation. Values for this field are typically a list of lag numbers. For example, 1,2,4 specifies that the ARIMA model should consider lags 1, 2, and 4 when calculating the statistical forecast. **Default value**: ' '	String	
BestFitHoldout PeriodInterval Count	Defines the duration of the holdout period used for best fit forecasting (expressed in intervals of the **SOPAnalyticsConfiguration.CycleCalendar**) The holdout period is applied immediately before **RunDate** and is made up of the historical periods whose data is held out (excluded) when fitting statistical models to historical data. Instead, this period is used for evaluation purposes as simulated forecasts based on each of the models fitted to earlier history is compared against the actuals in the holdout region. The model that most accurately predicts the values in the holdout region, based on minimizing the chosen error measure, is then used to generate the forecast starting at **RunDate**. **Default value**: 0 **Note:** This field is only applicable to items with **ForecastItemType.HoldoutPeriodUsageRule** set to "SimpleHoldout" or "Simulation". For more information, see "Using best fit with a holdout period" on page 2053.	Integer	
Constant	Indicates whether a constant value, also known as the Y intercept, is included in an ARIMA or ARIMAX forecast calculation. Valid values are: • **Use**— The constant is used in the calculation. • **Ignore**—For ARIMA, The constant is not used in the calculation. For ARIMAX, **Ignore** forces the constant term to be zero. **Default value**: Ignore	String (Enum)	

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
DifferenceLevel	The number of differencing operations to perform when calculating an ARIMA or ARIMAX function. If you enter a negative DifferenceLevel using ARIMA, ARIMA is not used. However, if you enter a negative DifferenceLevel using ARIMAX, the order is automatically chosen. **Default value**: -1 **Note:** It is strongly recommended that you do not enter a value that is greater than 2 in this field.	Integer	
ElasticNetParameter	A parameter, also known as alpha, that can be from 0 to 1 (inclusive) that controls how much penalization of the L1 and L2 norms is applied when using the Elastic Net forecast model. Elastic Net combines both the LASSO and ridge regression techniques to shrink coefficients. The value 1 is equivalent to LASSO and 0 is equivalent to ridge. This parameter works in conjunction with **RegularizationParameter**. **Default value**: 1 **Note:** This field was added in Version 2016.2, Service Update 27.	Quantity	

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
ForecastModel	Indicates the statistical model that will be fitted to the historical data and then evaluated for best fit. Based on the model selected, parameter values might need to be specified in certain other fields. Valid values are: • **AdditiveHoltWintersMethod**—similar to HoltWintersMethod, but forecast points are calculated in an additive rather than multiplicative, manner. Suitable for demand that has constant variation over a period. For more information, see "Holt-Winters multiplicative and additive" on page 2037. • **ARIMA**—calculates forecasts for a time period using a linear combination of actual values and error values in a time series. This model can be used on a stationary time series, or a differencing function can be applied to non-stationary data to make it stationary. For more information, see "Auto-Regressive Integrated Moving Average (ARIMA)" on page 2034. • **ARIMAX**—Is an extension of the ARIMA model, but also includes independent variables (regressors) in the forecast calculation. This makes ARIMAX a multiple linear regression model. To improve forecast accuracy, ARIMAX also takes into consideration autocorrelation that may be found in regressor residuals. For a more detailed description, see "ARIMAX" on page 2042. • **BestFit—**fits each of the other forecast models and selects the one with the least margin of error. Although this is an available option in the **BestFitProfileDetail** table, it should not be selected when defining best fit profiles. If this option is selected on a **BestFitProfileDetail** record, the record will be ignored within the profile. For a more detailed description, see "Best fit" on page 2053. • **CrostonsMethod**— uses exponential smoothing with a single smoothing constant for the forecast values, but calculates two different values; the first being the forecast values at a point and the second being the average length of time between forecasts. These values determine the forecast quantities and the periods that have forecast demands. Suitable for demand that is not constant, such as lumpy demand that contains periods of zero quantities. For more information, see "Croston's method" on page 2034. • **DoubleExponentialSmoothing**—considers the data points and trends within the historical data, and applies a smoothing algorithm to both. The result is then used as the statistical forecast. Suitable for demand that is fairly stable but trending up or down over time. For more information, see "Double	String (Enum)	

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
	exponential smoothing" on page 2036. • **ElasticNet**—A multiple linear regression model that uses regularized, also known as penalized, regression methods. It blends Lasso and ridge regression techniques by penalizing both L1 and L2 norms to reduce the coefficient vector. To use ElasticNet, you must attach a minimum of two regressors to a forecast item or you won't be able to generate a statistical forecast. For a more detailed description, see "ElasticNet" on page 2043. • **ExponentialSmoothing**— considers data points within the historical data, and applies a smoothing algorithm. The result of the final data point is then used as a constant value for the statistical forecast. Suitable for demand that is fairly stable and lacks a trend. For more information, see "Exponential smoothing" on page 2037. • **HoltWintersMethod**—similar to double exponential smoothing in that it considers data points and trends within the historical data, but also accounts for seasonal trends. That is, the values for each point in the forecast and trend are further adjusted to account for seasonal demand changes. This model is the Holt-Winters multiplicative method, so it is suitable for demand that has proportional variation over a period. For more information, see "Holt-Winters multiplicative and additive" on page 2037. • **Linear**—performs a linear regression on a set of historical data points to generate a trend line that determines statistical forecast quantities. Suitable for demand that does not fluctuate. For more information, see "Linear" on page 2039. • **MovingAverage**—calculates the simple moving average of a specified number of previous data points to generate a constant forecast. Suitable for demand that is fairly stable and does not change much over time. For more information, see "Moving average" on page 2039. • **MultipleLinearRegression**—Performs a linear regression using multiple independent variables to predict the outcome of one dependent variable. For a more detailed description, see "Multiple linear regression" on page 2043. • **Rforecast**—uses a statistical model as defined in the R statistical programming language. If selected, the RParameterSet reference can be used to identify parameters associated with the intended model. This option supports the optional R Forecast capability. For more information, see "Rforecast" on page 2039. Note		

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
	also that "Rforecast" is only considered in the best fit process for item's that use a BestFitProfile. That is, if an item's BestFitProfile reference is Null, then all models other than "Rforecast" will be considered during best fit forecasting). • **StepWiseARIMA**—identical to the ARIMA model, but all parameters are calculated automatically. Step-wise ARIMA selects the order of difference through KPSS unit-root tests and selects AR order and MA order by minimizing AIC. For more information, see "Step-wise ARIMA" on page 2039.		
Historical IntervalCount	The number of periods of historical demand to use as input into generating the statistical forecast. Expressed in units of the **IntervalsCalendar** on the **ForecastItemParametersType** table (the value applicable to the given forecast item).	Integer	
Id	A unique string identifier for this record.	String	Yes
MovingAverage IntervalCount	Specifies the number of periods of historical demand to use in calculating statistical forecasts using the "MovingAverage" model. Expressed in units of the **IntervalsCalendar** on the **ForecastItemParametersType** table (the value applicable to the given forecast item). **Note:** If this value is less than or equal to zero (0), then this value is automatically calculated so to minimize the selected error measure.	Integer	
Moving AverageTerms	Specifies the number of moving average terms used with the ARIMA method. Values for this field are typically a list of lag numbers. For example, 2,3 specifies the ARIMA model considers lags 2 and 3 when calculating the statistical forecast. Default value: ' '	String	
Profile	A reference to the best fit profile the statistical forecast model and other parameters on this record belong to. **Referenced table:** BestFitProfile	Reference	Yes

Table 6.34: BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
RegularizationParameter	A parameter, also known as lambda, that controls the strength of the penalty term for coefficients that is used in Elastic Net regression. This parameter is dependent on the ElasticNetParameter to shrink the coefficients of regressors. If you set a negative value in this field, RapidResponse will choose a value for you. **Default value**: -1 **Note:** This field was added in Version 2016.2, Service Update 27.		
RParameterSet	A reference to a grouped set of parameters used to define the function being used from the R programming language. This field is typically populated when the ForecastModel is set to "Rforecast" and is only available if the optional R Forecast capability has been configured. **Referenced table:** RParameterSet (Nullable)	Reference	
Seasonal IntervalCount	Specifies the duration of a season for forecasts calculated using the "BestFit", "Elastic Net", "HoltWintersMethod", "MultipleLinearRegression", or "Rforecast" models. Expressed in units of the **IntervalsCalendar** on the **ForecastItemParametersType** table (the value applicable to the given forecast item). **Default value**: -1	Integer	
TrendDecay Factor	A decay factor that can be applied to the following forecast models: • AdditiveHoltWintersMethod • DoubleExponentialSmoothing • ElasticNet • HoltWintersMethod • MultipleLinearRegression For AdditiveHoltWintersMethod, DoubleExponentialSmoothing, and HoltWintersMethod, if the trend should decay over time, this value must fall between 0 and 1. All values less than 0 result in complete elimination of the trend. If neither decay nor elimination is required, this value should be 1 or greater. For ElasticNet and MultipleLinearRegression, a value that is equal or less than 0 will result in complete elimination of the trend. If neither decay nor elimination is required, this value should be 1 or greater. The use of the decay factor is controlled by the **TrendDecayFactorRule** option on the **ForecastItemParametersType** table.	Quantity	

BestFitProfileDetail table – calculated and set fields

The following describes the calculated and set fields in the **BestFitProfileDetail** table. For an introduction to this table, and descriptions of its input fields, see "BestFitProfileDetail table" on page 210.

Table 6.35: Calculated BestFitProfileDetail (Mfg) fields

Field name	Description	Data type	Key
ForecastItem Parameters	Set of **ForecastItemParameters** records that reference this record, and thereby lock the best fit method to always select the specified model (and applicable parameters) for the item. **Referenced table:** ForecastItemParameters	Set	
ForecastItem ParametersFit Outputs	Set of **ForecastItemParametersFitOutput** records that reference this record. **Referenced table:** ForecastItemParameterFitOutput	Set	

BillOfMaterial table

The **BillOfMaterial** table identifies parent and child relationships between assembly parts (products assemblies and sub-assemblies) and component parts (assemblies, sub-assemblies, and raw materials) used in the production of the assembly.

RapidResponse can explode bill of material records with a negative QuantityPer value. Negative QuantityPer values are ignored unless the bill record has a **Type.ExplodeNegatives** control value of **Y**, or a **Type.ProductType** control value of either "CoProduct" or "ByProduct". For information about negative dependent demands, see "Exploding bill records with a negative QuantityPer value" on page 1338.

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
Alternate	A reference to the alternate BOM to which this component belongs. When Netting logic performs explosion calculations for dependent demands, this value is matched to either **PartSource.BOMAlternate** or **ScheduledReceipt.BOMAlternate**. If this value matches the default record in the **BOMAlternate** table, then the component is used by all 'Make' part sources and scheduled receipts for this assembly. For more information about alternate BOMs, see "Alternate BOM calculations" on page 1759. For information about identifying the "default" **BOMALternate** record, see "Default BOMAlternate record" on page 1764. **Referenced table**: BOMAlternate	Reference	
Assembly	The assembly being produced. **Referenced table:** Part **Reference Syntax:** Assembly.Site, Assembly.Name **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	
BaseKey	Can optionally be used to uniquely identify each BillOfMaterial record. **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	String	
Component	The component used to produce the assembly. **Referenced table:** Part **Reference Syntax:** Component.Site, Component.Name **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	
EffectiveInDate	Date on which this BillOfMaterial record will be effective. For more information, see "BOMType table" on page 687. **Default value**: Past	Date	
EffectiveInEC	The engineering change that introduced this BillOfMaterial record. **Referenced table:** EngineeringChange (this reference is nullable in new installations of RapidResponse 10.1 and later). **Reference Syntax:** EffectiveInEC.Id	Reference	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
EffectiveOutDate	Date on which this BillOfMaterial record is no longer in effect. **Default value**: Undefined	Date	
EffectiveOutEC	The engineering change that makes this BillOfMaterial record obsolete. **Referenced table:** EngineeringChange (this reference is nullable in new installations of RapidResponse 10.1 and later). **Reference Syntax:** EffectiveOutEC.Id	Reference	
EndUnit	The end of the unit range to which this BillOfMaterial record applies. The end unit is either included in the range or excluded depending upon the values of the BOMType.EffectivityRule and DependentUnitRule. This field is ignored unless BOMType.MUERule is 'Unit' or 'ModelUnit'. Every supply will be seen as being before EndUnit if less than zero. **Default value**: -1 **Note:** This field supports the Model-Unit Effectivity analytic and is ignored if this capability is not enabled.	Integer	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
FeatureRule	If this record pertains to a configurable end-item (that is, a made to order assembly where **PartType.FeatureBOMRule** is other than "Ignore"), then this field is used to specify a feature-based rule defining the conditions under which the component is needed in supplies satisfying assembly demands. For example, a component might only be required in production of the assembly if a given demand contains a particular set of features. To determine if the component is required to satisfy a given assembly demand, the specified rule is evaluated against the string of features provided in **IndependentDemand.FeatureList**. This rule can be comprised of feature names as well as any of the following logical operators. • **And**—used to indicate multiple features that must all be present on the assembly demand for the component to be used in satisfying it. By default, an ampersand character (&) is used to represent "and". For example, *a1 & b2*. If necessary, a different "and" symbol can be chosen by an administrator as discussed in "Defining the And operator for feature rules" on page 2208. • **Or**—used to indicate multiple features any one (or more) of which must be present on the assembly demand for the component to be used in satisfying it. A comma (,) is used to represent "or" and the specified features should also be enclosed in parentheses. For example, *(a1, a2, a3)*. • **Not**—used to indicate features that must not be present on the assembly demand for the component to be used in satisfying it. An exclamation (!) is used to represent "not". For example, *! (a1, a3) & b2*. If this field is left blank, then the component is assumed to be required in supplies satisfying any demand for the assembly regardless of features. **Default value**: blank string	String	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
IntervalToExpiry Offset	If the BillOfMaterial record represents a co-product or by-product configuration, this field indicates the number of intervals to add to or subtract from the assembly's **PartSource.IntervalsToExpiry** and **PartSource.CTPIntervalsToExpiry** values when calculating estimated and actual expiry dates on the co-product or by-product. These intervals are expressed in terms of the assembly part's **PlanningCalendars.ExpiryCalendar** value. This setting allows for a co-product or by-product to have a different shelf life than the primary part whose production it is derived from. If the co-product or by-product has a longer shelf life than the primary part, this field should be set to a positive value. If the co-product or by-product has a shorter shelf life than the primary part, this field should be set to a negative value. Otherwise, this value should be left at zero. **Default value**: 0	Integer	
LeadTimeOffset	Specifies the number of units defined by the **Assembly.PlanningCalendars.TimeUnits** calendar (typically, workday) to offset (move later) the required **DueDate** of the components for use in the assembly. Allows material to be staggered in time for long assembly operations. If the BillOfMaterial record defines a co-product or by-product configuration, this field allows for the due and available dates on the co-product or by-product to be either earlier or later then the due and available dates on the primary product by a specified amount of time. **Default value**: 0	Quantity	
Model	The model to which this bill record applies. The Model value is ignored unless the MUERule for the BOMType associated with this record is Model or ModelUnit. **Referenced table:** Model **Note:** This field supports the Model-Unit Effectivity analytic, and is ignored if this capability is not enabled.	Reference	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
Operation	The specific operation in the assembly supply's routing where the component is used. The referenced operation can be in the routing's standard sequence or one of its optional parallel sequences. Use of this field allows dependent demands to be placed at the start of the operation where they are actually needed (which may be after the parent order start date). Note that this reference is only considered if the parent supply uses capacity-based variable lead time (**OrderPolicy.UseVarLeadTime** set to "CRP"). **Referenced table:** CRPOperation (nullable) **Reference Syntax**: Operation.Id, Operation.Sequence.Id, Operation.Sequence.Routing.Id, Operation.Sequence.Routing.Site.	Reference	
OperationNumber	Operation sequence number at which the component is needed. This value is not used in Netting calculations, and refers to operations defined in the **Operation** table only. **Default value**: 0	Integer	
OptionRatio	Probability the assemblies will need these components. The range should be between 0 - 1 (if **BOMType.RatioRule** = 'Fraction') or between 1 - 100 (**BOMType.RatioRule** = 'Percent'). The supply quantity is multiplied by the ratio, then rounded (depending on rounding rules), then multiplied by QuantityPer (and possibly rounded again according to the rounding rules). MRP bills have this set to '1' or '100'. **Default value**: 1 **Note:** This field is ignored on BillOfMaterial records that define co-product or by-product configurations.	Quantity	
QuantityPer	The amount of this component that is required to build each finished assembly. A negative value can be processed to cancel positive demand. If the BillOfMaterial record defines a co-product or by-product configuration, then a negative quantity per should be used to define the amount of the supply generated for the primary product that ends up as a co-product or by-product. This is done using the ratio `-(co or by-product% / primary product%)` For more information about configuring co-products and by-products, see "Co-products and by-products" on page 1581. **Default value**: 1	Quantity	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
Scrap	Specifies the fraction of the components that is lost during the production of the assembly. For example, due to breakage. The interpretation of the value specified in this field is configurable as described in "ScrapRule" on page 694. **Default value**: 1 **Note:** This field is ignored on BillOfMaterial records that define co-product or by-product configurations.	Quantity	
Shop	The shop floor location associated with this BillOfMaterial record. **Referenced table**: Shop (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
StartUnit	The beginning of the unit range to which this bill record applies. The start unit is either included in the range or excluded depending upon the values of the BOMType EffectivityRule and DependentUnitRule. StartUnit is ignored unless MUERule is Unit or ModelUnit. Every supply will be seen as being after StartUnit if StartUnit is less than zero. **Default value**: -1 **Note:** This field supports the Model-Unit Effectivity analytic, and is ignored if this capability is not enabled.	Integer	
SubstituteGroup	A reference to the **SubstituteGroup** table that is used to define groups of substitutable components within an assembly. All components that are assumed to be substitutable for one another within a given assembly must point to the same **SubstituteGroup** record. In cases where an entire group of components can be substituted for another group of components, the **SubstituteGroup** reference should be made on the phantom BOM that defines each group of components. To ensure expected results/performance in bom-level substitution, avoid having multiple assemblies reference the same group for their substitutable components. That is, each assembly should have its own unique **SubstituteGroup** record(s) available for grouping substitutable components. **Referenced table:** SubstituteGroup (this reference is nullable in new installations of RapidResponse 10.1 and later). **Note:** This field is ignored on BillOfMaterial records that define co-product or by-product configurations. However, co-products can still be defined as substitutable components under an assembly (that is, each co-product should be configured as a component with the primary part in the group as the assembly).	Reference	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
SubstitutionTo DateQty	Indicates the quantity to date that has already been allocated to this substitute BOM (for example, it might represent known allotments on input supplies). This value should be expressed in component stock units, and is then used as an initial value for ongoing BOM-level substitution decisions. This field is applicable to substitute BOM records where **SubstituteGroup.Type.SourceRule** is set to "OnGoing", and is used in the logic that determines which substitutable components in a group should have planned orders generated on them to satisfy a dependent demand from the assembly. For more information, see "Define planned order generation" on page 1571.	Quantity	
Substitution Sequence	Defines the preferred sequence in which an assembly should look for scheduled/input supply within a group of substitutable components (components with lower values are used first). A value of 0 should be assigned to the primary component in the group. When substitution logic is turned off for the group (or if there is no **SubstituteGroup** record defined), the primary component is the only one for which dependent demand is generated when supply for the assembly is exploded. Also, ensure that no two of an assembly's components in a given **SubstituteGroup** have the same value in this field (duplicate **SubstitutionSequence** values in a group can result in excess dependent demand being generated). For more information, see "Substitute BOM configuration" on page 1566. In cases where an entire group of components can be substituted for another group of components, the component with the lowest value in this field acts as the indicator component within the group (the component whose supply determines if substitutions can be made). For example, a value of -1 could be assigned to the indicator component, and all other components in the group could be given higher values. If all values are the same, RapidResponse will use the component with the highest **Part.StdUnitCost** as the indicator. For more information, see "Substituting groups of components" on page 1574 **Default value**: 0 **NOTE**: This value defines a preference between substitutable components only when they can provide scheduled supply equally well (on-time or least late). For example, if Component A has scheduled supply that is on-time to the demand and Component B has scheduled supply that is late to the demand, then the supply of A is preferred regardless of the value in this field.	Integer	

Table 6.36: BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
Target	Determines the ratio, or percentage, in which dependent demand from the assembly is exploded down to the components in a substitute group when creating planned orders. This value should be expressed in terms of the assembly's supply unit. The percentage of demand that should be exploded down to a given component is then calculated as the **Target** value for the component divided by the sum of the **Target** values across all components in the group. Note that depending on the setting in the **ComponentSourceRule** field on the **SubstituteGroupType** table, each dependent requirement from the assembly is either split proportionally between substitutable components or fully assigned to the substitutable component which would otherwise be furthest away from its ongoing target. If planned orders should only be created on the primary component in the group, then it can be assigned a positive value in this field and all other components should be assigned a value of 0 (zero). If planned orders should be evenly spread across all components in the group, then each can be assigned the same postive value in this field (for example, a value of 1 could be specified). **Default value:** 0 **Note:** This value defines a preference between substitutable components only when they can plan new supply supply equally well (on-time or least late). For example, if Component A can plan a new order that is one day late to the demand and Component B can plan a new order that is five days late to the demand, then the entire dependent demand is placed on A regardless of the value in this field.	Quantity	
Type	A value that is associated with processing rules that defines the explosion, effectivity, and netting behavior. This field references the BOMType table. **Referenced table:** BOMType	Reference	

BillOfMaterial table – calculated and set fields

The following describes the calculated and set fields in the BillOfMaterial table. For an introduction to this table, and descriptions of its input fields, see "BillOfMaterial table" on page 217.

Table 6.37: Calculated BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
AssemblyLLC	The low-level code of the assembly. This field is intended for internal use in propagating the LLC down from the assembly to the component.	Integer	
CumLeadTime	The component's CumLeadTime adjusted by the **LeadTimeOffset**. Cumulative lead time is calculated using BillOfMaterial records that are effective at the MRP date. If a component is treated as a phantom to a bill, the component's lead-time is subtracted from its cumulative lead time. For example, only its component cumulative lead times are used. If it is not a phantom, or conditionally a phantom, then it is assumed to contribute its lead-time. Parts that are Buy parts will have cumulative lead time equal to its lead-time, (for example, it will not consider the lead-times of the components). This calculation is done only within a part's site. For multi-site cumulative lead time calculations (that is, calculations that consider transfer part sources), use the **MultiSiteCumLeadTime** field. For more information about lead time, see "Lead time calculations" on page 1307. **NOTE**: If using time-phased lead time. then the lead times effective for the component's primary part source at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used in this calculation.	Quantity	
EffOffSet	This value reflects the value of the LeadTimeOffset field but will be zero if the UseLeadTimeOffset field is 'N'. This value is based on the value of: BillOfMaterial.Type.UseLeadTimeOffset (if UseLeadTimeOffset= 'N' then EffOffset=0, otherwise EffOffset= LeadTimeOffset.)	Quantity	
EffOptionRatio	This field reflects the value of the OptionRatio field expressed as a fraction (value between 0 and 1). However, it always returns 1 if Type.RatioRule is set to 'Ignore'.	Quantity	

Table 6.37: Calculated BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
EffQuantityPer	The effective quantity of the component per unit of the assembly (after consideration for **Scrap** value and the setting in **BOMType.ScrapRule**). If **BOMType.RoundingRule** is set to "QtyPerOrder", then this field is always set to the **QuantityPer** value (all other **RoundingRule** settings are ignored by this field). If **BOMType.RoundingRule** is not set to "QtyPerOrder", then this calculation is dependent on ScrapRule as follows: 1. ScrapFraction BillOfMaterial.QtyPer / (1 - BillOfMaterial.Scrap) 2. ScrapPercent BillOfMaterial.QtyPer / (1 - BillMaterial.Scrap * 0.01) 3. YieldFraction BillOfMaterial.QtyPer / BillMaterial.Scrap 4. YieldPercent BillOfMaterial.QtyPer / (BillMaterial.Scrap * 0.01) 5. InflationFraction BillOfMaterial.QtyPer / (1 / (1 + BillOfMaterial.Scrap)) 6. InflationPercent BillOfMaterial.QtyPer / (1 / (1 + BillOfMaterial.Scrap * 0.01))	Quantity	
FirstEffectiveDate	Represents the adjusted value for the EffectiveInDate taking into account Type.EffectivityRule (for example, 'Inclusive'). If the record will never be effective, the dates will be 'Undefined'.	Date	
FirstEffectiveUnit	The first unit to which this BillOfMaterial record applies. **Note:** If not using Unit Effectivity, this field is set to –1.	Integer	
IsRecursive	Indicates whether recursive data has been found during processing (this happens when a part ends up being a component of itself). BillOfMaterial records that have a BOMType.EffectivityRule of 'Never' are ignored. Valid values are: • **Y**—recursive data has been found. The calculation skips the parts involved in the recursive relationship. • **N**—recursive data has not been found	Boolean	
LastEffectiveDate	Represents the adjusted value for the EffectiveOutDate taking into account Type.EffectivityRule (for example, InclusiveExclusive). This field is always inclusive. If the record will never be effective, the dates will be 'Undefined'.	Date	

Table 6.37: Calculated BillOfMaterial (Mfg) fields

Field name	Description	Data type	Key
LastEffectiveUnit	The last unit to which this BillOfMaterial record applies. **Note:** If not using Unit Effectivity, this field is set to 2,147,483,647.	Integer	
MultiSite AssemblyLLC	The multi-site low level of the assembly (any "transfer" part sources and their bill of material structures are considered in this calculation). This field is intended for internal use in propagating the low level code down from the assembly to the component.	Quantity	
MultiSiteCum LeadTime	The component's cumulative lead time, calculated using BillOfMaterial records that are effective at the MRP date, adjusted by **LeadTimeOffset**. This field uses a calculation similar to the **CumLeadTime** field, however this calculation will follow a component's product structure across site boundaries. That is, if a component's product structure contains a part with a primary supply type of "transfer", then cumulative lead time associated with the transfer part is included in the value reported in this field. Because this field adds together different lead time values without specifying the calendar to be used in the calculation, it is possible that the different lead time values reported when a product structure crosses site boundaries may not all use the same calendar. Therefore, depending on the calendars used in different sites, this field may not always return the expected result. For more information about lead time, see "Lead time calculations" on page 1307. **NOTE**: If using time-phased lead time. then the lead times effective for the component's primary part source at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used in this calculation.	Quantity	
FeatureRulesList	Set of BillOfMaterialFeatureRule records that reference this BillOfMaterial record.	Set	
NewAllocations	Set of NewAllocation records that define the dependent demands on the component due to ScheduledReceipts of the assembly that were exploded in RapidResponse.	Set	
Planned Allocations	Set of PlannedAllocation records that define the dependent demands on the component due to PlannedOrders generated for the assembly in RapidResponse.	Set	

Visualizing the bill of material in a BOM chart

Some workbooks can include a chart that shows the bill of material. A BOM chart provides you with an up-to-date view of the product structure, and you can filter the chart to focus only on the components

or sites (or other criteria) that are relevant to you. The BOM Visualization worksheet in the BOM Analysis workbook includes a BOM chart.

In a BOM chart you can:

- Control your view by moving the whole chart, moving individual nodes, or zooming in and out of the chart.
- Focus on a single component and its relationships.
- View substitutes, co-products, and by-products for all components or a specific component.
- Reset the chart to return to its default view.

For more information, see the *RapidResponse User Guide*.

BOMAlternate table

For RapidResponse configurations using alternate BOM logic, this table holds all alternate BOMs (that is, named variations of an assembly's BOM).

One record in this table is the "default" record and contains a special or common alternate code to be used by components that belong to all alternate BOM configurations under a given assembly. Typically, this special **BOMAlternate** record will have an empty string (<blank>) in the **Value** field. For information about finding the default **BOMAlternate** record, see "Default BOMAlternate record" on page 1764.

Table 6.38: BOMAlternate (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this alternate BOM.	String	
Value	The name of this alternate. Each alternate BOM value can be used by many part sources for many different parts. **Default value**: blank	String	Yes

BOMAlternate table – calculated and set fields

The following describes the calculated and set fields in the **BOMAlternate** table. For an introduction to this table, and descriptions of its input fields, see "BOMAlternate table" on page 229.

Table 6.39: Calulcated BOMAlternate (Mfg) fields

Field name	Description	Data type	Key
BillOfMaterials	Set of records in the **BillOfMaterial** table that use this alternate BOM value.	Set	
PartSources	Set of records in the **PartSource** table that use this alternate BOM value.	Set	

Table 6.39: Calulcated BOMAlternate (Mfg) fields

Field name	Description	Data type	Key
Scheduled Receipts	Set of records in the **ScheduledReceipt** table that use this alternate BOM value.	Set	

BonusSchedule table

The **BonusSchedule** table contains string values identifying the different bonus schedules that can be used in RapidResponse. The **Project** and **Task** tables each reference this table to specify the bonus schedule, if any, associated with a given project or task. Bonus schedules are used to determine the bonus revenue applied to tasks or projects that finish in advance of specified bonus dates. That is, bonuses can be earned by projects where **Project.CalcFinishDate** is on or before **Project.BonusDate**, and by tasks where **Task.CalcFinishDate** is on or before **Task.BonusDate**.

Each record in this table is also associated with a set of referencing records in the **BonusScheduleByDate** and/or **BonusScheduleByInterval** tables. These tables define the actual one-time and/or recurring costs that are associated with a given bonus schedule.

This table's records can be edited using the **Bonus Schedules** worksheet in the **Project Master Data** workbook included with RapidResponse. For more information about using this table, see "Bonuses" on page 2196.

Table 6.40: BonusSchedule (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this bonus schedule.	String	
Name	A unique name for this bonus schedule. The value in this field can be referenced from the **Project.BonusSchedule** and **Task.BonusSchedule** fields to set the bonus schedule associated with a given project or task	String	Yes
Site	The site associated with this bonus schedule. **Referenced table**: Site	Reference	Yes

BonusSchedule table – calculated and set fields

The following describes the calculated and set fields in the **BonusSchedule** table. For an introduction to this table, and descriptions of its input fields, see "BonusSchedule table" on page 230.

Table 6.41: BonusSchedule (Mfg) calculated fields

Field name	Description	Data type	Key
BonusByDate	Set of date-based bonus values associated with this bonus schedule.	Set	
BonusByInterval	Set of interval-based bonus values associated with this bonus schedule.	Set	
Projects	Set of projects that use this bonus schedule.	Set	
Tasks	Set of tasks that use this penalty schedule.	Set	

BonusScheduleByDate table

The **BonusScheduleByDate** table holds date effective records used to define the one-time and recurring bonus revenue applicable to a specified bonus schedule. The bonuses on the schedule can then be applied to projects that have a **ProjectType.BonusRule** value of "Date" and/or tasks that have a **TaskType.BonusRule** value of "Date.

For example, this table can be used to set up a bonus schedule that applies a one-time bonus if a project completes before a given date, and also applies a weekly bonus for each week in advance of a given date that the project finishes where the amount of that weekly bonus varies by date.

This table's records can be edited using the **Bonuses by Effective Date** worksheet in the **Project Master Data** workbook included with RapidResponse. For more information about how this table is used, see "Date-based bonus schedules" on page 2197.

Table 6.42: BonusScheduleByDate (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveOutDate	Sets the last date on which bonuses defined on this record are effective. Bonuses are effective between this date and the **EffectiveOutDate** on the immediately preceding record (by date). If there are multiple effective records between a project or task's finish date and bonus date, then their effective bonus values are used as defined by the setting in the **AccumulationRule** field on either the **ProjectType** or **TaskType** table.	Date	
Id	A unique string identifier for this record.	String	Yes
OneTimeBonus	A one-time bonus to be received if a project or task's **CalcFinishDate** is earlier than its **BonusDate**, and the **EffectiveOutDate** on this record is between **CalcFinishDate** and **BonusDate** (inclusive).	Money	

Table 6.42: BonusScheduleByDate (ProjMgmt) fields

Field name	Description	Data type	Key
PerIntervalBonus	A recurring bonus to be received for each bonus calendar interval between a project or task's **CalcFinishDate** and **BonusDate**. Bonus calendars are defined on the project type or task type; for example, bonuses might be earned on a daily or weekly basis.	Money	
Schedule	A reference to the named bonus schedule the bonus values on this record apply to. **Referenced table**: BonusSchedule	Reference	Yes

BonusScheduleByInterval table

The **BonusScheduleByInterval** table holds interval-based records, expressed in terms of bonus calendar interval, used to define the one-time and recurring bonus revenue applicable to a specified bonus schedule. The bonuses on the schedule can then be applied to projects that have a **ProjectType.BonusRule** value of "Interval" and/or tasks that have a **TaskType.BonusRule** value of "Interval".

For example, assuming a weekly bonus calendar, this table can be used to set up a bonus schedule that applies one set of weekly bonuses if a project finishes up to two weeks earlier than its bonus date, and then applies another set of weekly bonuses if a project finishes more than two weeks before its bonus date.

This table's records can be edited using the **Bonuses by Interval** worksheet in the **Project Master Data** workbook included with RapidResponse. For more information about how this table is used, see "Interval-based bonus schedules" on page 2199.

Table 6.43: BonusScheduleByInterval (ProjMgmt) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this record.	String	Yes
Interval	A number of bonus calendar intervals to subtract from a project or task's **BonusDate** to determine the last effective date for the bonus values defined on this record. Bonuses are effective between this date and last effective date on the immediately preceding record (by interval). If intervals define multiple effective records between a project or task's finish date and bonus date, then their effective bonus values are used as defined by the setting in the **AccumulationRule** field on either the **ProjectType** or **TaskType** table.	Integer	

Table 6.43: BonusScheduleByInterval (ProjMgmt) fields

Field name	Description	Data type	Key
OneTimeBonus	A one-time bonus to be earned if a project or task's **CalcFinishDate** is earlier than its **BonusDate**	Money	
PerIntervalBonus	A recurring bonus to be received for each bonus calendar interval between a project or task's **CalcFinishDate** and **BonusDate**. Bonus calendars are defined on the project type or task type; for example, bonuses might be earned on a daily or weekly basis.	Money	
Schedule	A reference to the named bonus schedule the bonus values on this record apply to. **Referenced table**: BonusSchedule	Reference	Yes

BuyerCode table

The **BuyerCode** table contains a list of buyer codes. This table and the BuyerCode field in the **Part** table have no algorithmic use.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.44: BuyerCode (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this buyer code.	String	
Site	The site associated with this buyer code. This field is optional. **Referenced table:** Core::Site	Reference	Yes
UserId	The user responsible for this buyer code. This field is in the **Solutions** namespace.	String	
Value	A unique identifier associated with this buyer code.	String	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

BuyerCode table – calculated and set fields

The following describes the calculated and set fields in the **BuyerCode** table. For an introduction to this table, and descriptions of its input fields, see "BuyerCode table" on page 233.

Table 6.45: BuyerCode (Mfg) calculated fields

Field name	Description	Data type	Key
Parts	Set of parts associated with this buyer.	Set	

Capacity table

The **Capacity** table identifies the available capacity for a work center at a particular point in time used to schedule the duration of an operation.

Table 6.46: Capacity (Mfg) fields

Field name	Description	Data type	Key
Demonstrated LaborCapacity	The average number of standard labor hours that are actually achieved in a day for the work center. **Default value**: 0	Quantity	
Demonstrated MachineCapacity	The average number of standard labor hours that are actually achieved in a day for the work center. **Default value**: 0	Quantity	
EffectiveInDate	Date this record becomes effective. **Default value**: Undefined	Date	
Efficiency	The factor used to scale operation setup and run times for this work center. In other words, a measurement of what you can hypothetically produce versus what you really are producing. When multiplied with the Utilization factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Default value**: 1	Quantity	
HoursPerDay	Total number of working hours in a day at this work center. **Default value**: 8	Quantity	
LaborRunRatio	The number of hours of labor required for one standard hour of run time. This factor is used to convert work center run time load to a labor load. **Default value**: 1	Quantity	

Table 6.46: Capacity (Mfg) fields

Field name	Description	Data type	Key
LaborSetupRatio	The number of hours of labor required for one standard hour of setup. This factor is used to convert work center setup time load to a labor load. **Default value**: 1	Quantity	
MachineRun Ratio	The number of hours of machine time required for one standard hour of run time. This factor is used to convert work center run time load to a machine load. **Default value**: 1	Quantity	
MachineSetup Ratio	The number of hours of machine time required for one standard hour of setup. This factor is used to convert work center setup time load to a machine load.	Quantity	
Maximum Labor Capacity	The maximum number of standard labor hours possible for the work center. **Default value**: 0	Quantity	
Maximum Machine Capacity	The maximum number of standard labor hours possible for the work center. **Default value**: 0	Quantity	
NumberOf Resources	The number of resources available at this work center. Available capacity at the work center is adjusted using this value which must be non-negative. If there are multiple resources available and lot splitting is allowed on a given operation (set by **CRPOperation.MaximumResources**), then the operation will finish quicker than it would if only one resource were used. **Default value**: 0	Quantity	
QueueTime	Pre-operation queue time used for scheduling all operations for this work center unless overridden (see **CRPOperation.QueueTimeOverride**). This value is the number of hours to wait before setup can begin and must be greater than or equal to zero. **Default value**: 0	Quantity	
Utilization	The factor used to scale operation setup and run times for this work center to allow for non-productive periods in the day (for example, breaks). When multiplied with the Efficiency factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Default value**: 1	Quantity	

Table 6.46: Capacity (Mfg) fields

Field name	Description	Data type	Key
WaitTime	The post-operation wait time used for scheduling all operations for this work center unless overridden (see Operation.WaitTimeOverride). The number of hours to wait after run time processing ends and must be greater than or equal to zero. **Default value**: 0	Quantity	
WorkCenter	Name of the work center. **Referenced table:** WorkCenter	Reference	

CapacityOverride table

The **CapacityOverride** table is used to define work center capacity (such as, working hours, efficiency, number of resources) by specific day of the week. If a work center has one or more effective records defined in this table for a given day of the week, then the record(s) for that day of the week override the work center's standard **Capacity** record. For days of the week on which a work center does not have an effective record in this table, capacity is taken from the **Capacity** table (assuming the work center has a record defined there).

Table 6.47: CapacityOverride (Mfg) fields

Field name	Description	Data type	Key
DayOfWeek	The day of the week to the which the work center capacity details in this record apply. Valid values are: • Monday • Tuesday • Wednesday • Thursday • Friday • Saturday • Sunday If necessary, each work center can have multiple effective records for a given day of the week. For example, each record might represent a different shift in the day with its own working hours, resources, efficiency, and utilization. When taken together, these records would give total work center capacity for the day. If setting up capacity in this manner, ensure to provide an **Index** value for each record. **Note:** Capacity overrides only apply to days within the work center's working calendar.	String (Enum)	
Description	An optional description for this capacity record.	String	

Table 6.47: CapacityOverride (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	The date on which the capacity details on this record becomes effective. The record is then effective until the next **EffectiveInDate** found on a record for the same work center, day of the week, and Index.	Date	
Efficiency	The factor used to scale operation setup and run times for this work center. In other words, a measurement of what can hypothetically be produced versus what is actually produced. When multiplied with the **Utilization** factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Default value:** 1	Quantity	
Id	A string identifier for this capacity record. Along with the **WorkCenter** reference, this field uniquely identifies each capacity record in this table.	String	Yes
Index	An index value for this record. If there are multiple records having the same **EffectiveInDate**, **DayOfWeek**, and **WorkCenter**, then a different index value should be assigned to each record. For example, the index might refer to a specific shift in the day. Note if the same index value is assigned to multiple records having the same **EffectiveInDate**, **DayOfWeek**, and **WorkCenter**, only one of those records will contribute to work center capacity (the others will be ignored).	Integer	
NumberOf Resources	The number of resources available at the work center on the specified day of the week. Available capacity at a work center is adjusted using this value which must be a non-negative number. If there are multiple resources available and lot splitting is allowed on a given operation (set by **CRPOperation.MaximumResources**), then the operation will should quicker than it would if only one resource were used. **Default value:** 0	Quantity	
Utilization	The factor used to scale operation setup and run times for this work center to allow for non-productive periods in the day (for example, breaks). When multiplied by the Efficiency factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Default value:** 1	Quantity	

Table 6.47: CapacityOverride (Mfg) fields

Field name	Description	Data type	Key
WorkCenter	The name of the work center whose capacity is defined for the day of the week on this record. **Referenced table:** WorkCenter	Reference	Yes
WorkingHour	Number of working hours for this day of the week and work center. **Default value:** 0	Quantity	

Carrier table

The **Carrier** table contains the names of carriers that are either transporting shipments from suppliers or transporting finished goods to customer receiving locations.

Table 6.48: Carrier (Mfg) fields

Field name	Description	Data type	Key
DefaultTransit Time	Default transit time to be applied to customer delivery routes that use this carrier. If the **TransitTime** field on a given **DeliveryRoute** record is non-negative, then that transit time value is used instead.	Quantity	
Default Transportation Mode	The default mode of transportation to be assumed for customer delivery routes that use this carrier. If the **TransportationMode** reference on a given **DeliveryRoute** record that uses this carrier is populated, then that referenced transportation mode is assumed instead. **Referenced table:** TransportationMode (Nullable)	Reference	
Description	A description of this carrier.	String	
LogisticsLocation	Logistics location used by this carrier. **Referenced table**: LogisticsLocation (Nullable) This field was added in Version 2016.2, Service Update 9.	Reference	
TransitCalendar	A reference to the calendar defining transit dates for delivery routes that use this carrier. This calendar thereby determines how planned receipt dates and available receipt dates on **IndependentDemand** records are calculated from the due and available date, respectively. If this reference is Null, then an Everyday calendar is assumed. **Referenced table:** Calendar (Nullable)	Reference	
Value	A unique identifier associated with this carrier.	String	Yes

Carrier table – calculated and set fields

The following describes the calculated and set fields in the **Carrier** table. For an introduction to this table, and descriptions of its input fields, see "Carrier table" on page 238.

Table 6.49: Calculated Carrier (Mfg) fields

Field name	Description	Data type	Key
DeliveryRoutes	Set of delivery routes, used to fulfill customer orders, that use this carrier.	Set	
Shipments	Set of supply orders, shipped from a supplier, that use this carrier.	Set	
Sources	Set of supply sources associated with this carrier.	Set	

CausalFactor table

This table stores causal factors used to adjust historical data for data errors or unusual demand events before a statistical forecast is generated. It references the part customer and historical actual the causal factor applies to (through the **HistoricalDemandHeader** reference), as well as the category the causal factor is associated with (through the **CausalFactorCategory** reference). Additional details of the causal factor, such as the adjustment quantity and date, are stored in the **CausalFactorDetail** table which references this table.

The **CausalFactor** table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained through the **S&OP Data Cleansing** workbook in RapidResponse. For more information about causal factors, see the *RapidResponse Applications Guide*.

Table 6.50: CausalFactor (Mfg) fields

Field name	Description	Data type	Key
Category	A reference to the category this causal factor applies to. **Referenced table:** CausalFactorCategory	Reference	Yes
Description	A description of this causal factor.	String	
Header	A reference to the part, customer, and historical demand combination this causal factor applies to. **Referenced table:** HistoricalDemandHeader	Reference	Yes

CausalFactor table – calculated and set fields

The following describes the calculated and set fields in the CausalFactor table. For an introduction to this table, and descriptions of its input fields, see "CausalFactor table" on page 239.

Table 6.51: Calculated CausalFactor (Mfg) fields

Field name	Description	Data type	Key
Details	Set of causal factor details associated with this CausalFactor record.	Set	

CausalFactorCategory table

This table stores categories defined in your company for grouping causal factors. For example, causal factor categories might include things such as promotions and weather events.

The CausalFactorCategory table supports Sales and Operations Planning in RapidResponse. Values in this table should be configured during initial RapidResponse deployment.

Table 6.52: CausalFactorCategory (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this causal factor category.	String	
ProcessingRule	Indicates whether the causal factor details associated with the causal factor category should be included in S&OP analytics calculations, inventory planning and optimization (safety stock) calculations, or both. Valid values are: • **SOP**—Indicates that causal factor details should be included in S&OP calculations only. For example, they would be used when calculating the statistical forecast or when disaggregating the forecast. • **SafetyStock**—Indicates that the causal factor details should be included in safety stock calculations only. This value is used when outlier adjustments have been created at the part level. • **All**—Indicates that causal factor details should be included in both S&OP and safety stock calculations. This is the default value.	String (Enum)	
Value	A unique identifier (name) for this causal factor category.	String	Yes

CausalFactorCategory table – calculated and set fields

The following describes the calculated and set fields in the CausalFactorCategory table. For an introduction to this table, and descriptions of its input fields, see "CausalFactorCategory table" on page 240.

Table 6.53: Calculated CausalFactorCategory (Mfg) fields

Field name	Description	Data type	Key
CausalFactors	Set of causal factors associated with this CausalFactorCategory record.	Set	

CausalFactorDetail table

This table includes date and quantity details that belong to a given causal factor through the reference to the **CausalFactor** table. In addition, the quantities reported in this table are reflected in disaggregated forecast details and disaggregation rates. If causal details are reported, they are taken into account in the following tables:

- ForecastDetail
- StatisticalForecastDetail
- DisaggregationRateByPartCustomer
- StatisticalForecastDisaggregationRate

The **CausalFactorDetail** table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained through the **S&OP Data Cleansing** workbook in RapidResponse. For more information about causal factors, see the *RapidResponse Applications Guide*.

Table 6.54: CausalFactorDetail (Mfg) fields

Field name	Description	Data type	Key
Date	The date the causal factor is to be applied.	Date	
Factor	A reference to the causal factor the details in this record apply to. **Referenced table**: CausalFactor	Reference	Yes
Id	A unique identifier for this record.	String	Yes
Quantity	The quantity of the causal factor to be applied.	Quantity	
UnitPrice	An optional field allowing a unit price to be applied to the causal factor. If a positive value is not provided in this field, then the unit price for the causal factor is calculated and reported in the EffectiveUnitPrice field. By default, this field is set to -1.	Money	

CausalFactorDetail table – calculated and set fields

The following describes the calculated and set fields in the CausalFactorDetail table. For an introduction to this table, and descriptions of its input fields, see "CausalFactorDetail table" on page 241.

Table 6.55: CausalFactorDetail (Mfg) fields

Field name	Description	Data type	Key
EffectiveUnit Price	The effective unit price for this causal factor. If a positive value is provided in the UnitPrice field, then that value is provided here. Otherwise, this field is calculated as follows: • RapidResponse checks the **CustomerPrice** table for records with matching Part and Customer values that have effective dates less than or equal to the date of this record. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, the EffectiveUnitPrice calculation uses the record with the larger value in the **CustomerPrice.Id** field. • However, if no matching Part and Customer values are found, RapidResponse next checks the CustomerPrice table for records with a matching Part and blank Customer value (that is, ' ') that have effective dates less than or equal to the record date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above or if the value provided in **CustomerPrice.UnitPrice** is negative, the value in **Part.AverageSellingPrice** is reported here. However, if the value in **Part.AverageSellingPrice** is also negative, then **CausalFactorDetail.EffectiveUnit Price** is set to zero. Currency conversions on this field use the **Date** field's conversion rate.	Money	

Class table

The **Class** table defines part classes. This table is used to define standard part classes (finished goods, work in progress, and raw materials) and custom classes, such as consumables. These part classes are used to assign planning, safety stock, and replenishment strategies. Part classes are also used in some widgets on the Inventory Manager application dashboard and as standard filters in worksheets.

Table 6.56: Class (Solutions) fields

Field name	Description	Data type	Key
Description	A description of this part class.	String	
Value	The name of the part class.	String	Yes
Parts	The set of parts that belong to this class. **Referenced table:** Part	Set	

Cluster table

The **Cluster** table contains the clusters reported by the Machine Learning service for the Self-Healing Supply Chain. A **Cluster** contains a group of **ItemSource** records with similar lead time properties. For more information about the properties associated with a **Cluster**, see the "ClusterProperty table" on page 244.

This table was added in Version 2016.2, Service Update 21.

Table 6.57: Cluster (Mfg) fields

Field name	Description	Data type	Key
Name	The name of the **Cluster**.	String	Yes
SelfHealingParameter	A parameter to support different use cases for healing planning parameters, such as LeadTime or Yield.	String	Yes
Zone	A parameter passed to the Machine Learning server that is used to generate clusters specific to each zone. The Zone value is calculated for historical data based on the settings defined in the AutomatedLeadTimeType and AutomatedYieldType.	String	Yes

Cluster table – calculated and set fields

The following describes the calculated and set fields in the **Cluster** table.

Table 6.58: Clustercalculated (Mfg) fields

Field name	Description	Data type	Key
ClusterProperties	Set of **ClusterProperty** records that reference this **Cluster**. **Referenced table:** "ClusterProperty table" on page 244	Set	
ItemSources	Set of **ItemSource** records that reference this **Cluster**. **Referenced table:** "ItemSource table" on page 379	Set	

ClusterProperty table

The **ClusterProperty** table contains the properties and values for a particular **Cluster** that is reported by the Machine Learning server for the Self-Healing Supply Chain. A **Cluster** contains a group of **ItemSource** records with similar lead time or yield properties.

This table was added in Version 2016.2, Service Update 21.

Table 6.59: ClusterProperty (Mfg) fields

Field name	Description	Data type	Key
Cluster	The **Cluster** of which this is a property. **Referenced table:** Cluster **Reference syntax:** Cluster.Name, Cluster.SelfHealingParameter	Reference	Yes

Property	The property name. Reported values are:
	• **Site** —The part site for the ItemSource.
	• **Supplier** —The supplier for the ItemSource.
	• **To** —The region of the part site (the destination site) for the ItemSource.
	• **From** —The region of the supplier site for the ItemSource.
	• **By** —The transportation mode for the ItemSource.
	• **Trend** —Whether the planning parameters (such as lead time) for the ItemSource are trending upward, downward, or whether they are flat.
	  • If the slope of linear regression is above a threshold, then the Trend is labeled Upward.
	  • If the slope of linear regression is below a threshold, then the Trend is labeled Downward.
	  • Otherwise, the Trend is labeled Flat.
	• **Seasonality** —Whether the planning parameters for the ItemSource are seasonal or not.
		String	Yes

Table 6.59: ClusterProperty (Mfg) fields

Field name	Description	Data type	Key
ValueBoolean	The value for the boolean property. Valid values are: • **Y** • **N** **Default value:** N	Boolean	
ValueQuantity	The value for the quantity property.	Quantity	
ValueText	The value for the text property.	String	
ValueType	The type for the cluster property values. Valid values are: • **Text** • **Boolean** • **Quantity**	Enum	

CollaborativeForecast table

The CollaborativeForecast table stores forecast information including data provided from suppliers.

Table 6.60: CollaborativeForecast (Mfg) fields

Field name	Description	Data type	Key
BucketDate	The bucketed date of the forecast.	Date	
CommitQty	The quantity committed by the subscriber.	Quantity	
LastCommitted	The date/time of last commit made by the subscriber.	DateTime	
LastUpdated	The date/time of last change made by the customer.	DateTime	
Part	This field is a reference to the Part table. **Referenced table:** Part	Reference	
RequestQty	The quantity requested by the customer.	Quantity	
Response Provided	Specifies if a response was provided from the supplier. Valid values are: • Y • N	Boolean	

Table 6.60: CollaborativeForecast (Mfg) fields

Field name	Description	Data type	Key
Response Required	Specifies if a response is required from the supplier. Valid values are: • Y • N	Boolean	
Source	This field is a reference to the Source table. **Referenced table:** Source	Reference	
State	Supports forecast-collaboration. Valid values are: • InfoOnly • NeedCommit • Committed	String (Enum)	

ConsensusForecastRollingHorizon table

The **ConsensusForecastRollingHorizon** table stores records for defining the order and the duration of the horizon used for applying rolling forecast weights to individual forecast categories when creating the consensus demand plan. This table is referenced by the "ConsensusForecastRollingHorizonWeight table" on page 971, which reports the weight that is applied to a forecast category in a given horizon.

The **ConsensusForecastRollingHorizon** table supports the Demand Planning application in RapidResponse. Values in this table are maintained using the both the **Control Tables** and **S&OP Demand Planning Ratios** workbooks in RapidResponse. For more information about the Demand Planning application, see the RapidResponse Applications Guide.

This table was added in Version 2016.2, Service Update 12.

Table 6.61: ConsensusForecastRollingHorizon (Mfg) fields

Field name	Description	Data type	Key
SequenceId	A value that determines the order in which the duration of a **ConsensusForecastWeight** is applied to the forecast category or header when generating the consensus forecast. The sequence begins at the **SOPAnalyticsConfiguration.CalcForecastStartDate**.	Integer	Yes
Duration	A value that specifies the length of the horizon in which to use the **ConsensusForecastWeight** that is applied to the forecast category or header when generating the consensus forecast. The duration is based on the calendar specified by **SOPAnalyticsConfiguration.CycleCalendar**. If a duration value is less or equal to zero (<=0), it is ignored.	Integer	

ConsensusForecastRollingHorizon table – calculated and set fields

The following describes the set fields in the **ConsensusForecastRollingHorizon** table.

Table 6.62: ConsensusForecastRollingHorizon (Mfg) set fields

Field name	Description	Data type	Key
CategoryWeights	Set of **HistoricalDemandCategoryRollingWeight** records associated with a given horizon.	Set	
HeaderWeights	Set of **HistoricalDemandHeaderRollingWeight** records associated with a given horizon.	Set	

Constraint table

The Constraint table lists all available constraints used by RapidResponse.

Table 6.63: Constraint (Mfg) fields

Field name	Description	Data type	Key
Calendar	The calendar to use when allocating constraint to load. **Referenced table:** Calendar	Reference	
ConstraintGroup	A reference to the group, if any, this constraint belongs to. Groups of related constraints are often managed together based on key work function and/or the planner responsible for them. **Referenced table**: ConstraintGroup (nullable)	Reference	
CumulativeMax	The total amount of this constraint that remains available to all supplies (contract maximum). Regardless of effective dates, the constraint is no longer available after the CumulativeMax has been used up. If CumulativeMax <= 0, it is ignored.	Quantity	
DataDate	Defines the current Constraint period. No Constraint is available in periods before the one containing this date. DataDate is also used as MRP Run Date when defining Constraint Manager bucket ranges. **Referenced table:** Calendar **Reference Syntax:** DataDate.Value	Reference	
Description	A description of the constraint.	String	

Table 6.63: Constraint (Mfg) fields

Field name	Description	Data type	Key
KeyConstraint	Indicates if the constraint is a key constraint. To identify the constraints that are important to your organization, you should set them as key constraints. For example, the constraints that are key to your organization's planning process. This field is in the **Solutions** namespace. **Default value:** N	Boolean	
Name	The identifier for the constraint. Typical identifiers include: • Part number • Supplier + Part number • Bottleneck resource name • Assembly line or production area • Supplier + part family	String	Yes
PlannerCode	A reference to the planner responsible for this constraint. **Referenced table:** PlannerCode (Nullable)	Reference	
Site	The site associated with this constraint. Choose one site to own all common external constraints and the None constraint. **Referenced table:** Site	Reference	Yes
Type	A value that is associated with processing rules that defines the constraint's behavior. **Referenced table:** ConstraintType	Reference	
UnitOfMeasure	The units in which this constraint is measured. Not used for any conversion. (Use SourceConstraint.ConstraintFactor and PartSource.SupplierUOM to handle any conversions). **Referenced table:** ConstraintUnitOfMeasure	Reference	
UsedThisPeriod	Adjusts constraint available for the first (current) period. The amount of constraint that has been consumed, in the Constraint.Calendar period containing DataDate, by supplies that are not in the current data set. (Values <=0 are ignored). **Note:** UsedThisPeriod does not affect CumulativeMax calculations.	Quantity	

Constraint table – calculated and set fields

The following describes the calculated and set fields in the Constraint table. For an introduction to this table, and descriptions of its input fields, see "Constraint table" on page 247.

Table 6.64: Calculated Constraint (Mfg) fields

Field name	Description	Data type	Key
IsRecursive	In versions of RapidResponse prior to 8.2.1, this field was used to indicate if a constraint was involved in a recursive relationship (**Y** or **N**). As of Version 8.2.1, this field is obsolete and always reports a value of **N** as multi-level constraints are now supported. For more information, see "Planning with multi-level constraints" on page 1644.	Boolean	
AssumptionBy Constraints	Set of AssumptionbyConstraint records.	Set	
Constraint Availables	Time-phased availability of this Constraint.	Set	
ConstraintLoads	Set of calculated ConstraintLoad records.	Set	
Constraint SpreadLoads	Set of calculated ConstraintSpreadLoad records.	Set	
ConstraintUseds	Set of calculated ConstraintUsed records.	Set	
FLPConstraints	Set of calculated FLPConstraint records.	Set	
PeriodLoads	Set of calculated PeriodConstraintLoad records.	Set	
SourceConstraints	Set of SourceConstraint records.	Set	

ConstraintAvailable table

This table supports the Constraint Manager application, and is used to define the availability (rate) of a constraint over a period of time.

Two calculated fields on this table report the first and last effective dates for a record based on the input dates provided as well as calendar and control settings.

Table 6.65: ConstraintAvailable (Mfg) fields

Field name	Description	Data type	Key
Constraint	The constraint this availability applies to. **Referenced table:** Constraint **Reference Syntax:** Constraint.Name, Constraint.Site	Reference	
EffectiveInDate	The date when this record becomes effective (subject to the **ConstraintType.EffectivityRule**). Use "Past" if the constraint is available before the beginning of the calendar. **NOTE**: If this date is not aligned to a date on the constraint calendar (**Constraint.Calendar**), then calendar rounding occurs and results might not be as expected. For example, assume the default **ConstraintType.EffectvityRule** of "InclusiveExclusive". If the constraint calendar is "Month" and the **EffectiveInDate** is "05/16/2018", the record is effective from the beginning of May.	Date	
EffectiveOutDate	The date when this record is no longer effective (subject to the **ConstraintType.EffectivityRule** setting). Use "Future" if the Constraint is available through the end of the calendar. **NOTE**: If this date is not aligned to a date on the constraint calendar (**Constraint.Calendar**), then calendar rounding occurs and results might not be as expected. For example, assume the default **ConstraintType.EffectvityRule** of "InclusiveExclusive". If the constraint calendar is "Month" and the **EffectiveOutDate** is "07/31/2018", the record stops being effective at the end of June.	Date	
Rate	Amount of constraint available for each time period (Constraint.Calendar) during the effective time of this record. If more than one ConstraintAvailable record is effective for a period, the Rates are added. Rates < 0 are seen as reducing Constraint available over their effective period. A total rate of zero (0) over a period indicates that no Constraint is available during that period (and if the total rate is < 0, then constraint avaialble is reported as 0). Rate is assumed to be defined in terms of the Constraint.UOM, however no conversion is done. Also, any unit of measure can be used for the rate of constraint available. However, all supplies using the constraint should define their constraint usage with the same units.	Quantity	

ConstraintAvailable table – calculated and set fields

The following describes the calculated and set fields in the ConstraintAvailable table. For an introduction to this table, and descriptions of its input fields, see "ConstraintAvailable table" on page 249.

Table 6.66: Calculated ConstraintAvailable (Mfg) fields

Field name	Description	Data type	Key
FirstEffective Date	Date when this ConstraintAvailable record becomes effective. (Undefined if never effective). Calculated as: • Undefined if calculated to be after LastEffectiveDate or if Type.EffectivityRule = 'Never' • Past if Type.EffectivityRule = 'Always' or if EffectiveInDate = 'Undefined' • EffectiveInDate+ 0 Constraint.Calendar If Type.EffectivityRule = 'Inclusive' or 'InclusiveExclusive' • EffectiveInDate + 1 Constraint.Calendar If Type.EffectivityRule = 'Exclusive'	Calculated Date	
LastEffective Date	Last date when this ConstraintAvailable record remains effective. (Undefined if never effective). Calculated as: • Undefined if calculated to be before FirstEffectiveDate or if Type.EffectivityRule=Never • Future if Type.EffectivityRule = 'Always' or if EffectiveOutDate = 'Undefined' • (EffectiveOutDate + 1 Constraint.Calendar)- 1: If Type.EffectivityRule = 'Inclusive' • (EffectiveOutDate + 0 Constraint.Calendar) - 1: If Type.EffectivityRule in (Exclusive, InclusiveExclusive)	Calculated Date	
Overlap	Indicates an anomaly in the ConstraintAvailable records for this Constraint. All ConstraintAvailable records that overlap each other are flagged. Valid values are: • Y • N During the period of overlap, the Rate is seen as the sum of the overlapping Rates.	Boolean	

ConstraintGroup table

The **ConstraintGroup** table is used to group sets of related constraints that should typically be managed together. For example, constraints might be grouped on the basis of key work function or the individual responsible for planning them.

Table 6.67: ConstraintGroup (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this group. Typically, this might indicate the key work function performed by the constraints in this group.	String	
Id	A unique string identifier for this group.	String	Yes

ConstraintGroup table – calculated and set fields

The following describes the calculated and set fields in the ConstraintGroup table. For an introduction to this table, and descriptions of its input fields, see "ConstraintGroup table" on page 251.

Table 6.68: Calculated ConstraintGroup (Mfg) fields

Field name	Description	Data type	Key
Constraints	Set of constraints that belong to this group.	Set	

ConstraintMetric table

The **ConstraintMetric** table identifies the combination for each constraint and metric definition generated for a specific as of date that are used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.69: ConstraintMetric (Solutions) fields

Field name	Description	Data type	Key
AsOf	The date on which the constraint and metric definitions are generated. **Referenced table:** MetricAsOf	Reference	Yes
Constraint	A reference to the constraint associated with this combination of constraint and metric definition data. **Referenced table:** Constraint	Reference	Yes
Metric	A reference to the metric associated with this combination of constraint and metric definition data. **Referenced table:** Metric	Reference	Yes

ConstraintMetric table – calculated and set fields

The following describes the calculated and set fields in the ConstraintMetric table. For an introduction to this table, and descriptions of its input fields, see "ConstraintMetric table" on page 252.

Table 6.70: ConstraintMetric (Solutions) fields

Field name	Description	Data type	Key
ConstraintMetricValues	The set of constraint calculations that belong to this combination of constraint and metric definition data. **Referenced table:** ConstraintMetricValues	Set	

ConstraintMetricValues table

The **ConstraintMetricValues** table stores the constraint calculations for metrics used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.71: ConstraintMetricValues (Solutions) fields

Field name	Description	Data type	Key
Date	The date associated with this calculation.	Date	Yes
ConstraintMetric	A reference to the constraint and metric definition combination associated with this calculation. **Referenced table:** ConstraintMetric	Reference	Yes
Quantity	The quantity for the metric. The value in this field is used if Metric.ValueType= 'Quantity'. For more information, see "Metric table" on page 399.	Quantity	
Value	The money value for the metric. The value in this field is used if Metric.ValueType= 'Money'. For more information, see "Metric table" on page 399.	Value	

ControlSet table

This table holds all control sets defined in your RapidResponse system. Each control set is comprised of all entries in other tables that point to that record's Value field. The tables that reference the ControlSet table include the Site table and a large number of control tables; for a full listing of these tables, see "ControlSet table — calculated and set fields" on page 254.

Control sets allow for the same control table value to potentially have different meanings across RapidResponse sites. For example, a PartType value of "P" may indicate a purchased part in the control set associated with one site, and a phantom part in the control set associated with another site. Control sets can also be used to hold all expected characteristics for a particular type of RapidResponse installation within your demand or supply network. The control set can then be exported, imported, and otherwise manipulated across sites as needed. For complete information on managing controls sets, see the *RapidResponse Administration Guide*.

Table 6.72: ControlSet (Core) fields

Field name	Description	Data type	Key
Value	A unique identifier for this control set.	String	Yes
Description	A description of this control set.	String	

Note: For more information on control sets, see "Control sets" on page 161.

ControlSet table – calculated and set fields

The following describes the calculated and set fields in the Control Set table. Each field represents the set of records on another table that references the Control Set table. For an introduction to this table, and descriptions of its input fields, see "ControlSet table" on page 253.

Table 6.73: Calculated ControlSet (Core) fields

Field name	Description	Data type	Key
Allotment Override Types	The set of values in the AllotmentOverrideType table associated with this control set.	Set	
AlternatePart Types	The set of values in the AlternatePartType table associated with this control set.	Set	
AvailableRules	The set of values in the AvailableRule table associated with this control set.	Set	
BOMTypes	The set of values in the BOMType table associated with this control set.	Set	
ConstraintTypes	The set of values in the ConstraintType table associated with this control set.	Set	
ConstraintUnitOf Measures	The set of values in the ConstraintUnitOfMeasure table associated with this control set.	Set	
CRPUnitOf Measures	The set of values in the CRPUnitOfMeasure table associated with this control set.	Set	
DemandStatuses	The set of values in the DemandStatus table associated with this control set.	Set	
DemandTypes	The set of values in the DemandType table associated with this control set.	Set	
ForecastItem ParametersTypes	The set of values in the ForecastItemParametersType table associated with this control set.	Set	
IncrementalRules	The set of values in the IncrementalRule table associated with this control set.	Set	

Table 6.73: Calculated ControlSet (Core) fields

Field name	Description	Data type	Key
InventoryTransfer Types	The set of values in the IncrementalTransferType table associated with this control set.	Set	
MEIOLeadTime Types	The set of values in the MEIOLeadTimeType table associated with this control set.	Set	
MUEPoolNetting Types	The set of values in the MUEPoolNettingType table associated with this control set.	Set	
OnHandTypes	The set of values in the OnHandType table associated with this control set.	Set	
OperationTypes	The set of values in the OperationType table associated with this control set.	Set	
OrderPolicies	The set of values in the OrderPolicy table associated with this control set.	Set	
OrderPriorities	The set of values in the OrderPriority table associated with this control set.	Set	
PartSourceTypes	The set of values in the PartSourceType table associated with this control set.	Set	
PartTypes	The set of values in the PartType table associated with this control set.	Set	
ProjectTypes	The set of values in the ProjectType table associated with this control set.	Set	
RangeOf Coverages	The set of values in the RangeOfCoverage table associated with this control set.	Set	
ResourceTypes	The set of values in the ResourceType table associated with this control set.	Set	
Shipment Policies	The set of values in the ShipmentPolicy table associated with this control set.	Set	
Sites	The set of values in the Site table associated with this control set.	Set	

Table 6.73: Calculated ControlSet (Core) fields

Field name	Description	Data type	Key
SourceRules	The set of SourceRule records associated with this control set.	Set	
SpreadProfiles	The set of SpreadProfile records associated with this control set.	Set	
SubstituteGroup Types	The set of SubstituteGroupType records associated with this control set.	Set	
SupplyAllocation Types	The set of SupplyAllocationType records associated with this control set.	Set	
SupplyStatuses	The set of SupplyStatus records associated with this control set.	Set	
TaskTypes	The set of TaskType records associated with this control set.	Set	
ToleranceProfiles	The set of ToleranceProfile records associated with this control set.	Set	
ToleranceProfile Zones	The set of ToleranceProfileZone records associated with this control set.	Set	
SupplyTypes	The set of SupplyType records associated with this control set.	Set	
UnitOfMeasures	The set of UnitOfMeasure records associated with this control set.	Set	
WorkCenterTypes	The set of WorkCenterType records associated with this control set.	Set	

CRPOperation table

The **CRPOperation** table identifies operations required in the production of a part. Each operation is identified by an Id along with the supply routing and sequence it belongs to and work center it uses. Additional operation details such as the duration of each phase of an operation and the sequence in which the operations should occur should also be provided.

Prior to Version 2014.4, operations were defined in the **Operation** table. Any resources (such as, workbooks) based on that table will continue to function, however the **CRPOperation** table should be used for new purposes.

Table 6.74: CRPOperation (Mfg) fields

Field name	Description	Data type	Key
BatchQuantity	The batch quantity for this operation. This represents the number of units per supply that the operation must complete before it can be passed on the next operation for processing. If set to 0 (zero), then the entire operation must be completed before the supply can be sent to the next operation in the sequence. When the current operation produces supply in batches, the next operation in the sequence may be able to begin processing the supply once it receives a batch. However, if that next operation's batch size is greater than that of the current operation, it must further wait until it has received enough supply to also satisfy its own batch quantity. **Default value:** 0 **Note:** The **Type.BatchQuantityRule** control setting provides further control over how a value of 0 (zero) is used in determining when an operation can begin processing batched supply from the previous operation in the sequence. For more information about working with batches, see "Overlapping operations" on page 1527.	Quantity	
Description	A description of this operation.	String	
EffectiveInDate	The date this record becomes effective. The effectivity of each operation in a routing is determined based on the start date of the work order (scheduled receipt or planned order). That is, the entire sequence of operations is determined from the start of the order. **Default value:** Undefined	Date	
EffectiveOutDate	The date this record expires. The effectivity of each operation in a routing is determined based on the start date of the work order (scheduled receipt or planned order). That is, the entire sequence of operations is determined from the start of the order. **Default value:** Undefined	Date	
EndUnit	The end of the unit range to which this operation record applies. The end unit is either included in the range or excluded depending upon the values in **OperationType.EffectivityRule** and **OperationType.DependentUnitRule**. This field is ignored unless MUERule is Unit or ModelUnit. Every supply will be seen as being before EndUnit if EndUnit is <0. **Default value:** -1	Integer	

Table 6.74: CRPOperation (Mfg) fields

Field name	Description	Data type	Key
Id	A string identifier for this operation. Together with the Sequence and WorkCenter reference fields, this value uniquely identifies an operation. Typically, it might be set to the operation number.	String	Yes
Maximum Resources	Indicates the maximum number of work center resources that can concurrently work on this operation. If it is possible to split the operation between resources, then the operation should finish sooner. The lesser of this value and the available number of resources at the work center is used to determine the actual number of resources applied to the operation. **Default value:** 0	Quantity	
Model	The model to which this operation applies. The values in this field is ignored unless the referenced **Type.MUERule** value is either "ModelUnit" or "Unit". **Referenced table:** Model (Nullable)	Reference	
Number	The operation number. This value is used for scheduling operations that belong to the same sequence. Operations with lower numbers are scheduled first. If multiple effective operations in a sequence have the same number, the following fields are used to determine scheduling order: • WorkCenter.Name (alphabetical sort) • EffectiveInDate (earliest first) **Default value:** 0	Integer	
QueueTime Override	Optionally, used to specify the number of hours to wait before operation setup can begin at the work center. If this field is left negative, then the **QueueTime** value specified on the **Capacity** record applicable to the work center is used. Otherwise, if this value is greater than or equal to zero, it overrides the **QueueTime** value. **Default value:** -1	Quantity	
RunTime	Specifies the amount of time required for this operation to run through or process one unit of an order. This value is adjusted by **RunTimeUOM** and multiplied by the order quantity to get the total run time in hours for a given order. **Default value:** 0	Quantity	

Table 6.74: CRPOperation (Mfg) fields

Field name	Description	Data type	Key
RunTimeUOM	The run time unit of measure for this operation. Used to convert the specified run time into standard hours per piece. **Referenced table:** CRPUnitOfMeasure	Reference	
Sequence	A reference to the specific sequence within the routing this operation belongs to. An operation can belong to the routing's standard sequence, of which there should be exactly one, or any of its optional parallel sequences. **Referenced table:** OperationSequence **Reference Syntax**: Sequence.Id, Sequence.Routing.Id	Reference	Yes
SetupHours	The number of standard hours required to set up one machine for use in this operation. **Default value:** 0	Quantity	
StartUnit	The beginning of the unit range to which this operation record applies. The start unit is either included or excluded in the range depending on the settings in **Type.EffectivityRule** and **Type.DependentUnitRule**. Every supply will be seen as being after StartUnit if StartUnit is <0. This field is ignored unless **Type.MUERule** is set to either "Unit" or "ModelUnit". **Default value:** -1	Integer	
TearDownTime	Indicates the amount of tear down time (in hours) required after this operation has completed processing. This is the time used to remove parts and restore the work center to its normal state in preparation for the next order. **Default value:** 0	Quantity	
TransitTime	Indicates the amount of time needed to move between operations. If **Type.TransitTimeSequenceRule** is "AfterOperation", this represents the time to move from this operation to the next one in the sequence. If **Type.TransitTimeSequenceRule** is "BeforeOperation", this represents the time to move from the previous operation in the sequence to this one. This value should be expressed in either hours (by default) or days as determined by the setting in **Type.TransitTimeProcessingRule.** **Default value:** 0	Quantity	

Table 6.74: CRPOperation (Mfg) fields

Field name	Description	Data type	Key
Type	A reference to the control value that defines processing rules for this operation such as how effectivity is determined. **Referenced table:** OperationType	Reference	
WaitTimeOverride	Optionally, used to specify the number of hours of waiting time after the operation run has completed at the work center. If this field is left negative, then the **WaitTime** value specified on the **Capacity** record applicable to the work center is used. Otherwise, if this value is greater than or equal to zero, it overrides the **WaitTime** value. **Default value:** -1	Quantity	
WorkCenter	The name of the work center where this operation is performed. **Referenced table:** WorkCenter	Reference	

CRPOperation table – calculated and set fields

The following describes the calculated and set fields in the **CRPOperation** table. For an introduction to this table, and descriptions of its input fields, see "CRPOperation table" on page 256.

Table 6.75: Calculated CRPOperation (Mfg) fields

Field name	Description	Data type	Key
Allocations	Set of Allocation records that reference this record.	Set	
BOMs	Set of BillOfMaterial records that reference this record.	Set	
CRPOperation States	Set of CRPOperationState records that reference this record.	Set	

CRPOperationState table

The **CRPOperationState** table can be used to identify a scheduled receipt that has been partially completed by a given work center operation in its routing. Each record in this table references both a **ScheduledReceipt** record and a **CRPOperation** record, and also contains a **CompleteQty** field used to indicate how much of the receipt has already been processed by the referenced operation. This allows Capacity Requirements Planning calculations to start processing an ongoing scheduled receipt at an operation other than the first one in its routing and/or to only schedule that operation for the portion of the order has yet to be completed.

This table also contains fields used to indicate the date and time at which an ongoing operation starts a particular phase. Typically, the start date for the current phase of the ongoing progress operation should be provided. However, if multiple phase start dates are defined, then the latest of those phases is used to represent the ongoing state of the operation and the earlier phases are assumed to be completed.

The **CRPOperationState** table was added in Version 2014.4

Table 6.76: CRPOperationState (Mfg) fields

Field name	Description	Data type	Key
ActualFinishDate	The date on which the operation actually finished processing this order. This field is optional and should typically only be used if maintaining historical operation data. Providing a date in this field implies that the operation has already finished processing this order (regardless of any values provided in the **CompleteQty** field). Note also that any provided date in this field is assumed to represent the start of the available working hours for the day unless an offset is given in **ActualFinishOffset**. **Default value**: 0	Date	
ActualFinish Offset	The number of hours into the available hours per day at which the operation finished processing this order. **Default value:** 0	Quantity	
ActualProcess StartDate	The date on which the operation starts processing the order (beginning of the run time phase). This is assumed be the start of the available working hours for the day unless an offset is specified in **ActualProcessStartOffset**. **Default value:** Undefined	Date	
ActualProcess StartOffset	The number of hours into the available hours per day at which work center processing starts for the remainder of this operation. **Default value:** 0	Quantity	
ActualSetup StartDate	The date on which the setup phase begins for this operation. This is assumed to be the start of the available working time for the day unless an offset is specified in **ActualSetupStartOffset**. **Default value:** Undefined	Date	
ActualSetup StartOffset	The number of hours into the available hours per day at which work center setup began for this operation. **Default value:** 0	Quantity	

Table 6.76: CRPOperationState (Mfg) fields

Field name	Description	Data type	Key
ActualStartDate	The date on which the overall operation starts. This is assumed to be the start of the available working hours for the day unless an offset is specified in **ActualStartOffset**. **Default value:** Undefined	Date	
ActualStartOffset	The number of hours into the available hours per day at which work center tear down began for this operation. **Default value:** 0	Quantity	
ActualTearDown StartDate	The actual date on which work center tear down starts for this operation. This is assumed to be the start of the available working hours for the day unless an offset is specified in **ActualTearDownStartOffset**. **Default value:** 0	Date	
ActualTearDown StartOffset	The number of hours into the available hours per day at which work center tear down starts for this operation. **Default value:** 0	Quantity	
CompleteQty	For partially completed operations, this indicates the amount of the order that has already been processed. Note that when batch processing is enabled, this should specify the total amount of the order that the operation has processed across all batches. The difference between the value in this field and the order quantity is the remaining amount still be processed by the operation. If there is an amount still to be processed, then load is generated from the operation starting on the date defined in one of the following fields: **Default value:** 0	Integer	
CRPOperation	A reference to the work center operation being run for the supply identified on this record. When parallel operations are enabled, a single scheduled receipt might reference multiple parallel operations. However, if multiple operations within the same sequence are configured as partially complete for a given scheduled receipt, then the latest non-batched operation in the sequence is assumed to be ongoing and all operations before it are assumed to be finished. **Referenced table:** CRPOperation **Reference Syntax**: **Reference Syntax**: CRPOperation.Id, CRPOperation.Sequence.Id, CRPOperation.Sequence.Routing.Id, CRPOperation.Sequence.Routing.Site.	Reference	Yes

Table 6.76: CRPOperationState (Mfg) fields

Field name	Description	Data type	Key
Scheduled Receipt	A reference to the supply being processed by the work center operation identified on this record. **Referenced table:** ScheduledReceipt **Reference Syntax**: ScheduledReceipt.Line, ScheduledReceipt.Order.Type, ScheduledReceipt.Order.ID, ScheduledReceipt.Order.Site	Reference	Yes

CurrencyConversionForecast table

Contains forecasted currency conversion rates. The records in this table represent conversion rates that will be used as of the specified date. The rates in this table are combined with the **CurrencyConversionActual** and **HistoricalCurrencyConversion** tables to determine the rates used to perform currency conversions. The date specified for the conversion determines which conversion rate is used. For example, the rates in this table are used as conversion rates only if the specified conversion date is later than the current date.

This table is in the Core namespace.

Table 6.77: CurrencyConversionForecast (Core) fields

Field name	Description	Data type	Key
Currency	The currency type. **Referenced table**: Currency	Reference	
Date	The date the conversion rate is calculated for.	Date	
Rate	The conversion rate as of the specified date.	Quantity	

For more information, see "HistoricalCurrencyConversion table " on page 324 and "CurrencyConversionActual table" on page 1276.

CurveParameters table

This table is used by the Event Management analytic when calculating the adjustments for an event phase that has its **ActionType** set to "Curve." The type of curve, specified in this table's **Type** field, determines the formula used to define the shape of the curve.

In the context of the following field descriptions, `time` is the number of dates on **EventPhase.Calendar** since the beginning of the event phase.

For more information, see "Using curves to adjust the forecast" on page 2116.

This table was added in Version 2016.2, Service Update 3.

Table 6.78: CurveParameters (Mfg) table

Field name	Description	Type	Key
ExponentialRate	The base of the exponent in the formula: `InitialValue * ExponentialRate`^{time} This value cannot be negative. If a negative value is specified, it is treated as zero. Only used when "ExponentialCurve" is specified in the **Type** field. **Default value:** 0	Quantity	
Id	The unique identifier for the set of curve parameters defined in this record.	String	Yes
ImitationRate	The proportion of imitators in the populations of consumers. This value must fall between zero and 1. Negative numbers are treated as zero and numbers greater than one are treated as one. Only used when "DiffusionCurve" is specified in the **Type** field. **Default value:** 0	Quantity	
InitialValue	The first point on the curve at time t=0 for all types of curves. **Default value:** 0	Quantity	
InnovationRate	The proportion of innovators in the population of consumers. This value must fall between zero and 1. Negative numbers are treated as zero and numbers greater than one are treated as one. If a value greater than zero is not specified, the curve is flat. Only used when "DiffusionCurve" is specified in the **Type** field. **Default value:** 0	Quantity	
LinearRate	The slope of the linear curve, used in the formula: `InitialValue + LinearRate * time` Only used when "LinearCurve" is specified in the **Type** field. **Default value:** 0	Quantity	
MaximumValue	The maximum value for the Bass Diffusion Curve. If this value is equal to or less than the InitialValue, the curve is flat. Only used when "DiffusionCurve" is specified in the **Type** field. **Default value:** equal to InitialValue	Quantity	

Table 6.78: CurveParameters (Mfg) table

Field name	Description	Type	Key
Type	The type of curve defined in this record. Determines how the shape of the curve is calculated. Valid values are: • **LinearCurve** Use a linear curve (straight line with a defined slope). • **ExponentialCurve** Use an exponential curve. • **DiffusionCurve** Use Bass Diffusion Model curve to model behavior of consumers. **Default value:** LinearCurve	String (Enum)	
UnitOfMeasure	The unit of measure that this CurveParameters record is associated with. **Referenced table:** UnitOfMeasure	Reference	

CurveParameters table – calculated and set fields

The following describes the set field in the CurveParameters table. For an introduction to this table, and descriptions of its input fields, see "CurveParameters table" on page 263.

Table 6.79: CurveParameters (Mfg) fields

Field name	Description	Data type	Key
EventPhases	The EventPhase records that use this set of curve parameters.	Set	

CurvePoint table

The **CurvePoint** table contains the individual point values that make up the efficiency curves associated with each named value in the **PointsCurve** table. The resulting curves can be used by tasks that have a **TaskType.EfficiencyCurveRule** of "PointsCurve" in order to define a varying efficiency of resources assigned to those tasks and hence impact the calculated task durations.

Besides a reference to the relevant PointsCurve value, each value in this table also contains Index and Quantity values. The Index defines the particular point on the curve and typically corresponds to a day on the task, and the Quantity defines the efficiency of resources on the task at for that point or day.

For example, assume the following set of index and quantity values referencing a given curve YZ, and further assume a single resource assigned to a task using that curve and on an 8 hour work day.

Curve	Index	Quantity
YZ	1	.25
YZ	2	.25

Curve	Index	Quantity
YZ	3	.5
YZ	4	1

With these curve values, 2 hours would be achieved towards completion of the task on the first day, 2 hours again on the second day, 4 hours on the third day, and 8 hours on the fourth and all subsequent days. Thus, a task that would take a resource 4 days to complete if no efficiency curve were used, would take the same resource 6 days to complete if using the YZ curve.

This table's records can be edited using the **Efficiency Curves** worksheet included with the **Project Master Data** workbook in RapidResponse.

Table 6.80: CurvePoint (ProjMgmt) fields

Field name	Description	Data type	Key
Curve	A reference to the named points curve this point belongs to. Each value in this table that references a given PointsCurve defines the full set of points on that curve. **Referenced table:** PointsCurve	Reference	Yes
Id	A unique string identifier for this point.	String	Yes
Index	The position on the curve that the point value specified in the Quantity field applies to. For example, 1 is the first point on the curve, 2 is the second point on the curve, and so on. Typically, these values align to resource work days on the task.	Integer	
Quantity	The point value (resource efficiency) for the position on the curve specified in the Index field. Typically, values greater than zero (0) and less than or equal to one (1) are specified. For example, a value of .4 implies resource efficiency of 40% at a given point on the curve. Note that any points after the last point on a curve inherit their Quantity value from that last entry. As well, any negative or zero values are interpreted to mean one (1).	Quantity	

Customer table

A customer is a consumer of an independent demand item and may be a consumer, a distributor, service center, or a plant identifier for inter-plant orders.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.81: Customer (Mfg) fields

Field name	Description	Data type	Key
CustomerGroup	The group this customer is in. **Referenced table**: CustomerGroup (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
CustomerService Representative	Identifies the customer service representative responsible for this customer. **Referenced table:** PlannerCode (Nullable) **Note:** This field is in the **Solutions** namespace.	Reference	
EarlyShipWindow	Maximum number of days earlier than needed that orders for the customer are allowed to ship. For example, shipping an order early might allow for early revenue recognition. Shipping their orders any earlier than the amount of time specified here requires explicit customer contact and approval. This field supports the order fulfillment application and should be expressed in terms of the Everyday calendar**.** It acts as a default early ship window for all of the customer's orders, but can be overridden on individual demands by providing a positive **EarlyShipWindow** value on the **IndependentDemandSolution** record. **Note:** This field is in the Solutions namespace.	Integer	
HoldCode	The default hold code, if any, that is currently assigned to the customer's orders. Hold codes can be used to prevent orders from moving past a particular stage of processing. For example, if a customer has exceeded their credit limit, a credit hold might be placed on their orders to prevent shipment. **Referenced table:** HoldCode (Nullable) **Note:** This field is in the Solutions namespace. Note also that hold codes can also be associated with individual line items (on **IndependentDemandSolution** records) which then take precedence over any hold code specified for the customer.	Reference	
Id	A value that identifies the customer. **Default value**: Undefined	String	Yes

Table 6.81: Customer (Mfg) fields

Field name	Description	Data type	Key
KeyCustomer	Indicates whether or not this customer is considered a key (important) customer. This field is used in certain metrics supporting the Order Fulfillment application to enable metric values that are calculated for both all customers as well as key customers only. Valid values are: • **N**—this is not considered a key customer. • **Y**—this is considered a key customer. **Note:** This field is in the Solutions namespace.	Boolean	
Name	The name of the customer. **Default value**: blank string	String	
Region	The name of the region associated with this supplier. For example, this might identify the sales territory associated with this customer. **Referenced table:** Region (Nullable)	Reference	
Site	The site associated with this customer record. This field is optional. **Referenced table:** Site	Reference	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

Customer table – calculated and fields

The following describes the calculated and set fields in the Customer table. For an introduction to this table, and descriptions of its input fields, see "Customer table" on page 266.

Table 6.82: Calculated Customer (Mfg) fields

Field name	Description	Data type	Key
Orders	Set of DemandOrder records for existing orders from this customer.	Set	
Projects	Set of Project records for projects associated with this customer.	Set	

CustomerDestination table

The **CustomerDestination** table identifies the specific locations where customers receive delivery of their orders. For example, a record in this table might identify a given customer warehouse or store along with contact information for that location.

Table 6.83: CustomerDestination (Mfg) fields

Field name	Description	Data type	Key
Address	A street address for this customer destination. If this table is used to support maps, the address in this field determines the geocode that is used to mark the destination's location on the map. To ensure that the correct geocode is used, follow the guidelines outlined in "Using address data to support maps" on page 1735.	String	
ContactName	The name of the key contact at this customer destination.	String	
Customer	A reference to the customer associated with the receiving destination described on this record. **Referenced table:** Customer **Reference Syntax:** Customer.Id, Customer.Site	Reference	Yes
DefaultRoute	The default delivery route that is typically used to send deliveries to this customer destination.	String	
Description	An optional description of the customer destination described on this record.	String	
Email	An email address associated with a contact at this customer destination.	String	
Id	A unique string identifier for this customer destination.	String	Yes
Phone	Telephone number used to make contact with this customer destination.	String	
DeliveryRoutes	Set of delivery routes that serve this customer destination.	Set	

CustomerGroup table

The CustomerGroup table defines groups that different customers can be placed under. If your company's host system uses more than one name to refer to the same company, they can be placed in the same group, and the group name used to refer to the entire company.

Table 6.84: CustomerGroup (Mfg) fields

Field name	Description	Data type	Key
Id	The group name.	String	Yes
Description	A description of this group. This can be the company name or a general classification for several different companies.	String	

CustomerGroup table – calculated and set fields

The following describes the calculated and set fields in the CustomerGroup table.

Table 6.85: Calculated CustomerGroup (Mfg) fields

Field name	Description	Data type	Key
Customers	The customers in this group.	Set	

CustomerPrice table

The CustomerPrice table stores time-phased pricing details. It allows for different unit prices to be specified over time for given part and customer combinations. Various tables that report **EffectiveUnitPrice** fields, such as the **ForecastDetail** table, make use of this table when calculating the price to use for a part and customer combination on a given date.

Table 6.86: CustomerPrice (Mfg) fields

Field name	Description	Data type	Key
Customer	The customer associated with this customer price record. A blank (' ') value indicates this is the "default" customer used when an effective entry for a given part and customer combination cannot be found. **Referenced table**: Customer **Reference Syntax:** Customer.Id, Customer.Site	Reference	Yes
EffectiveDate	The date the unit price takes effect for this customer and part combination.	Date	
Id	A value that uniquely identifies this record. If multiple records exist for a given part and customer combination with the same effective date, the **EffectiveUnitPrice** calculation uses the record with the larger value in this field.	String	Yes

Table 6.86: CustomerPrice (Mfg) fields

Field name	Description	Data type	Key
Part	The part associated with this customer price record. **Referenced table**: Part **Reference Syntax**: Part.Name, Part.Site	Reference	Yes
UnitPrice	The unit price that takes effect for this customer and part combination on the effective date.	Money	

DeliveryRoute table

The **DeliveryRoute** table is used to model the different routes for shipping finished goods to customer destinations. For example, the expected shipping time required for an order to move from the order site to the customer's receiving location is specified here.

Many of the fields in this table are is used in calculations determining the planned ship and customer receipt dates on IndependentDemand records, as well as the available ship and customer receipt dates on those records.

Table 6.87: DeliveryRoute (Mfg) fields

Field name	Description	Data type	Key
Carrier	The carrier being used to transport goods along this route to the customer. **Referenced table:** Carrier	Reference	
DefaultPickPack Time	The amount of time required to pull an order from inventory at the site and prepare it for shipment along the route to the customer. Note that if the **PickPackTime** field on the **Part** record has a positive value, then it is used instead of the value provided here.	Integer	
Description	Description of this delivery route.	String	
Destination	The customer location that is receiving the order. **Referenced table:** CustomerDestination	Reference	Yes
EstimatedCost	An estimate of the cost associated with using this delivery route to ship an order (can be used to express relative costs between delivery routes).	Money	

Table 6.87: DeliveryRoute (Mfg) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this delivery route. For example, this might be a concatenation of the Carrier and TransportationMode values or some other string that uniquely identifies a route between the ShipFrom site and customer destination.	String	Yes
PickupCalendar	A reference to the calendar that defines the dates on which orders can be shipped from this site. **Referenced table:** Calendar	Reference	
ShipFrom	The site from which the order is being shipped to the customer. This should resolve to the same site as referenced from **IndependentDemand.Part.Site**. **Referenced table**: Site	Reference	Yes
TransitTime	The amount of time a shipment using this route is expected to take to reach the customer. This value should be expressed in units of the **Carrier.TransitCalendar**. If that reference is Null, then an Everyday calendar is assumed. **Note:** If this field is set to a negative value, then the value from **Carrier.DefaultTransitTime** is used.	Integer	
Transportation Mode	Indicates the mode of transportation associated with this route. For example, land, air, or sea. **Referenced table**: TransportationMode	Reference	
Independent Demands	Set of independent demands using delivery route to reach the customer receiving location.	Set	

DemandOrder table

The **DemandOrder** table contains all common (header) information for **IndependentDemand** records. Header information is common information about an entire order.

Table 6.88: DemandOrder (Mfg) fields

Field name	Description	Data type	Key
Customer	The customer of the order. **Referenced table:** Customer **Reference Syntax:** Customer.Id, Customer.Site	Reference	
Id	A value that identifies an order number for an IndependentDemand record. **Default value**: blank string	String	Yes
Pool	The pool to which this demand order belongs. For example, this might typically refer to a pool specific to the customer the order is for. This field supports inventory pooling as well as forecast consumption by pool. It is only applicable to parts who use a **MUEPoolNettingType.PoolRule** setting other than "Ignore". If any line items under the order have their **InputPool** reference populated, then that referenced pool is used instead. Typically, this is done when pools do not correspond to items associated with demand orders. For example, if pools are meant to represent specific plants or storage locations, rather than customers, then pool values should typically be specified on **IndependentDemand** records. **Referenced table**: Pool **Reference Syntax:** Pool.Value	Reference	
Site	The site this demand order relates to. **Referenced table:** Site	Reference	Yes
Type	A value that determines the processing policies of demand. **Referenced table:** DemandType	Reference	Yes

DemandOrder table – calculated and set fields

The following describes the calculated and set fields in the DemandOrder table. For an introduction to this table, and descriptions of its input fields, see "DemandOrder table " on page 272.

Table 6.89: Calculated DemandOrder (Mfg) fields

Field name	Description	Data type	Key
Lines	Set of line items (IndependentDemand records) for this order.	Set	

Department table

The Department table identifies the department of a work center. A department is generally a group of related WorkCenters.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.90: Department (Mfg) fields

Field name	Description	Data type	Key
Id	Name for the department. It must be unique. **Default value**: blank string	String	Yes
Site	The site where this department is located. This field is optional. **Referenced table:** Site	Reference	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Administration Guide*

Department table—calculated and set fields

The following describes the calculated and set fields in the Department table. For an introduction to this table, and descriptions of its input fields, see "Department table" on page 274.

Field name	Description	Data Type	Key
CustomerOrders	Set of CustomerOrder records that reference this Department value.	Set	

EngineeringChange table

The EngineeringChange table identifies engineering changes. Engineering changes (EC) are referenced by BillOfMaterial records to identify sets of records that are associated with a common change.

Field name	Description	Data type	Key
Analysis Parameters	Parameters used in the analysis of the engineering change. This field is populated automatically when simulating an engineering change, and should be left blank.	String	
Description	A description of the change. String	String	
Id	Identifier for the change. **Default value**: blank string	String	Yes
Originator	The name of the person that originated the change.	String	
Reason	The motivation for creating the change. For example, reduction in cost, product improvement, or an introduction of a new product.	String	
Site	The RapidResponse site. This is applicable only if the change applies to a single site.	String	
Type	The type of change, such as 'ECO', 'Deviation', or 'Stop Ship'.	String	

EngineeringChange table–calculated and set fields

The following describes the calculated and set fields in the EngineeringChange table. For an introduction to this table, and descriptions of its input fields, see "EngineeringChange table" on page 274.

Field name	Description	Data type	Key
BOMIn	Set of BillOfMaterial records introduced by this EC.	Set	
BOMOut	Set of BillOfMaterial records made obsolete by this EC.	Set	
Notes	Collaborative notes related to the change. This field can used to enter notes about an engineering change. Each time a note value is added to this field through a worksheet, a new record is created in the EngineeringChange_Notes table with a date and timestamp, user ID, note text, and a reference to the EngineeringChange record against which it was created. **Referenced table:** EngineeringChange_Notes	Note	

Event table

The Event table is used to group related event phases, which are used by the Event Management analytic to make forecast adjustments. This table is used for reference, to help organize and keep track of event phases. Two event phases in the EventPhase table can have the same name and adjustment type, as long as they belong to different events.

The data in this table do not affect the analytic calculations, apart from helping to distinguish one event phase from another. For more information, see "Event Management" on page 2109.

This table was added in Version 2016.2, Service Update 3.

Table 6.91: Event (Mfg) fields

Field name	Description	Data type	Key
CreationDate	Date when the event was created.	Date	
Description	A description of the event	String	
Name	The name of the event	String	Key
PlannerCode	The planner code associated with the event **Referenced table:** PlannerCode	Reference	
Type	The event type. **Referenced table:** EventType	Reference	

Event table – calculated and set fields

The following describes the set field in the Event table. For an introduction to this table, and descriptions of its input fields, see "Event table" on page 276.

Table 6.92:

Field name	Description	Data type	Key
EventPhases	The EventPhase records associated with this event	Set	

EventPhase table

This table is used by the Event Management analytic. An **EventPhase** record contains information including when the phase begins and ends, the calendar used, and how the forecast should be adjusted during that phase.

For more information, see "Event Management" on page 2109.

This table was added in Version 2016.2, Service Update 3.

Table 6.93: EventPhase (Mfg) fields

Field name	Description	Data type	Key
Adjustement Type	The aspect of the forecast to adjust. Valid values are: • Quantity • UnitPrice **Default value:** Quantity	Enum	Yes
ActionType	The type of action that will be taken to adjust the forecast. For more information, see "Action types for event phases" on page 2110. Valid values are: • **Quantity** Add the value in the Quantity field to the original forecast. • **CompoundedQuantity** Adjust the forecast by compound increments of the value in the **Quantity** field. • **Percentage** Increase the original forecast by the percentage specified in the **Percent** field. • **Compounded Percentage** Adjust the forecast by compound increments of the percent value in the **Percent** field. • **Constant** Replace original forecast with the value in the **Quantity** field. • **Curve** Use a curve to determine forecast adjustments. The **CurveParameters** field determines which set of curve parameters to use. **Default value:** Quantity	Enum	
Calendar	Calendar to be used when applying the adjustment. Input parameters, such as **Quantity**, **Percent,** and **CurveParameters**, are specified in terms of this calendar. Input data is bucketed using this calendar before the adjustment is applied. **Referenced table:** Calendar	Reference	
Category	The EventPhase applies to HistoricalDemandHeaders whose category matches the category referenced by this field. This field is nullable. If the field is null, the EventPhase applies to all associated HistoricalDemandHeaders **Referenced table:** HistoricalDemandCategory	Reference	
Compounding Calendar	Calendar used to determine compounding intervals within compounding buckets. This field is nullable. If a valid value is not specified, the Everyday calendar is used. Single date calendars, such as RunDate, are not valid in this field. **Referenced table:** Calendar	Reference	

Table 6.93: EventPhase (Mfg) fields

Field name	Description	Data type	Key
CompoundingIntervalCount	The number of intervals in a single compounding bucket. If a negative value is specified, it is treated a 1. **Default value:** 1	Integer	
CreationDate	The date when the **EventPhase** was created. This is an optional value that can be used for reference. It is not used for analytic calculations.	Date	
CurveParameters	The set of curve parameters to use. This value is only relevant if **ActionType** is set to "Curve." This field is nullable. **Referenced table:** CurveParameters **Default value:** null	Reference	
Description	A description of the event phase.	String	
EndDate	First date (after the **StartDate**) to which this **EventPhase** does not apply.	Date	
Event	The **Event** that this **EventPhase** is associated with. **Referenced table:** Event	Reference	Yes
Name	The name of the event phase	String	Yes
Percent	Used to determine the percent to adjust the forecast by when **ActionType** is set to "Percentage" or "CompoundedPercentage." To specify 100%, enter `100` in this field. **Default value:** 0	Quantity	
Quantity	Used to determine the quantity of forecast adjustments when **ActionType** is set to "Quantity," "CompoundedQuantity," or "Constant." **Default value:** 0	Quantity	
StartDate	First date to which this EventPhase applies.	Date	
UsageRule	An EventPhase is only applied if this field is set to "Use." To prevent an EventPhase from affecting the forecast without deleting the EventPhase, set this field to "Ignore." Valid values include: • Use • Ignore **Default value:** Use	String (Enum)	

EventPhase table – calculated and set fields

The following describes the calculated and set fields in the Model table. For an introduction to this table, and descriptions of its input fields, see "EventPhase table" on page 276.

Table 6.94: Calculated EventPhase (Mfg) fields

Field name	Description	Data type	Key
FirstEffective Date	First effective date for this EventPhase. This is a date on the calendar specified in the **Calendar** field. It falls on or after the **StartDate**. For more information, see "Determining the effective dates and date buckets for an event phase" on page 2125.	Date	
LastEffective Date	Last effective date for this EventPhase. This is a date on the calendar specified in the **Calendar** field. It falls before the **EndDate**. For more information, see "Determining the effective dates and date buckets for an event phase" on page 2125.	Date	
Event Disaggregation Rates	The EventDisaggretationRate records associated with this EventPhase.	Set	
EventPhase Headers	The EventPhaseHeader records associated with this EventPhase. EventPhaseHeader records connect EventPhase records with HistoricalDemandHeader records.	Set	
EventStatistical Disaggregation Rates	The EventStatisticalDisaggregationRates records associated with this Eventphase.	Set	

EventPhaseHeader table

This table supports the Event Management analytic. The **EventPhaseHeader** table connects **EventPhase** records with records on the **HistoricalDemandHeader** table, which identifies each unique part, customer, and historical demand category combination. For more information, see "Connecting event phases to the affected forecast" on page 2112.

This table was added in Version 2016.2, Service Update 3.

Table 6.95: EventPhaseHeader (Mfg) fields

Field name	Description	Data type	Key
EventPhase	An EventPhase **Referenced table:** EventPhase	Reference	Yes
Header	The HistoricalDemandHeader record that defines the portion of the forecast the event phase is applied to. **Referenced table:** HistoricalDemandHeader	Reference	Yes
Id	It is recommended that this field be left blank unless you are using attribute-based planning. For more information about attribute-based planning, see "Attribute-based planning" on page 1817. If you are using attribute-based planning you might need to enter a unique identifier in this field to distinguish EventPhaseHeader records applicable to data with different attributes for the same HistoricalDemandHeader and EventPhase combination.	String	Yes

EventType table

The EventType table is used to categorize events. This table is used for reference, to help organize and keep track of events. Values in this table do not affect analytic functionality.

This table was added in Version 2016.2, Service Update 3.

Table 6.96: EventType (Mfg) fields.

Field name	Description	Data type	Key
Description	Description of the event type.	String	
Value	Name of the event type.	String	Yes

EventType table – calculated and set fields

The following describes the set field in the EventType table. For an introduction to this table, and descriptions of its input fields, see "EventType table" on page 280.

Table 6.97: EventType (Mfg) fields

Field name	Description	Data type	Key
Events	The Event records associated with this EventType.	Set	

Feature table

The **Feature** table holds all features that has been defined in RapidResponse. Each feature represents a particular option on a configurable end-item assembly. For example, a feature might refer to a particular engine model or a set of tires that go into an automobile.

If using feature BOM logic in RapidResponse, a list of features can be specified on a demand using the **IndependentDemand.FeatureList** field. This list is then compared against rules defined in the **BillOfMaterial.FeatureRule** field to determine which of an assembly's components are required in a supply being built to satisfy that demand.

Note that feature values can be specified on **IndependentDemand** and **BillOfMaterial** records without having been defined in the this table. However, it is recommended to ensure that all applicable feature values are added to the **Feature** table. This provides a reference for all features, allows for features values to be mapped together in the **FeatureMap** table, and ensures that certain calculated tables are able to reference and report details for all active features.

Table 6.98: Feature (Mfg) fields

Field name	Description	Data type	Key
Description	A description of the feature.	String	
Value	A unique name or identifier for the feature.	String	Yes

Feature table – calculated and fields

The following describes the calculated and set fields in the Feature table. For an introduction to this table, and descriptions of its input fields, see "Feature table" on page 281.

Table 6.99: Calculated Feature (Mfg) fields

Field name	Description	Data type	Key
ToFeatures	Set of FeatureMap records that reference this value.	Set	

FeatureMap table

The **FeatureMap** table is used to setup feature values expected to be inputted on assembly's demands and map or expand them to larger groups of features. When a demand matches a feature value set up in this table, all of its mapped features are then used for comparison against BOM records to determine which of the assembly's components should have dependent demands placed on them.

Each record in this table identifies the assembly part the record pertains to, along with a **FromRule** field used to specify a feature or feature rule to compare against feature lists on independent demands, and a **ToFeature** field that references a feature in the **Feature** table. If the feature list on a demand matches a given **FromRule** value, then all corresponding **ToFeature** references are used in determining effective features for the demand.

Typically, each **FromRule** value associated with an assembly should be associated with multiple **ToFeature** values. For example, suppose the following set of records.

Part	FromRule	ToFeature
Auto1	AA	Y1
Auto1	AA	Y2
Auto1	AA	Y3

This would indicate that if a demand for part "Auto1" contains the feature "AA" in its feature list, then features "Y1", "Y2", and "Y3" apply to the demand and components of "Auto1" corresponding to these features are required in supply built to satisfy the demand.

Table 6.100: FeatureMap (Mfg) fields

Field name	Description	Data type	Key
BaseKey	A unique string identifier for this record.	String	Yes
EffectiveInDate	The date on which this record becomes effective. This allows for feature mappings that change over time.	Date	
EffectiveOut Date	The date on which this record is no longer effective. This allows for feature mappings that change over time.	Date	
FromRule	The feature value being mapped by this record. When a feature specified on an assembly demand in the **IndependentDemand.FeatureList** field matches the value specified here, then that input feature is expanded to mean all features referenced through the **ToFeature** field for this value. This expanded feature list is then compared against values in the **BillOfMaterial.FeatureRule** field in order to determine which of the assembly's components that dependent demands should be placed on. This field can contain either a single feature name or it can contain a more complex feature rule using any of the following logical operators: • **And**—used to indicate multiple features that must all be specified on the demand for this rule to be considered a match. By default, an ampersand symbol (&) is used for "and". For example, *a1 & b2*. • **Or**—used to indicate multiple features any one (or more) of which must be present on the demand for this rule to be considered a match. A comma (,) is used to represent "or" and the specified features should also be enclosed in parentheses. For example, *(a1, a2, a3)*. • **Not**—used to indicate features that must not be present on the assembly demand for the rule to be considered a match. For example, *! b2*.	String	

Table 6.100: FeatureMap (Mfg) fields

Field name	Description	Data type	Key
Part	The assembly part whose demands this feature map is used for. **Referenced table**: Part	Reference	
Sequence	The order in which expansion of mapped features occurs. The order can be important for specifying situations where a given FromRule relies on previously mapped feature expansions in this table. **Default value:** 0	Integer	
ToFeature	Reference to a particular feature that the value or rule provided in the **FromRule** field should map to. All values referenced through this field for a given **FromRule** value together comprise the expanded feature list. **Referenced table:** Feature	Reference	

ForecastDetail table

This table holds current forecast data at the detail level (after any disaggregation calculations have been performed). Each record pertains to a given part, customer, and category combination, and shows details such as the forecast quantity and date. Different types of forecasts can be stored in this table such as the statistical forecast, sales forecast, marketing forecast, and so on.

The **ForecastDetail** table supports Sales and Operations Planning in RapidResponse. The values shown in this table are based on aggregate values entered or maintained through various resources in RapidResponse. For example, details of the statistical forecast are based on values generated by the **Save Forecast** command in the **S&OP Statistical Forecast** workbook. Details of other types of forecast are based on values entered in workbooks belonging to the related constituent group (for example, values might be provided through the **S&OP Marketing Forecast** workbook, **S&OP Sales Forecast** workbook, **S&OP Finance Operating Plan** workbook, and so on). The values in this table also reflect any causal quantities that are reported in the **CausalFactorDetail** table.

This table contains vector set data for records in the **HistoricalDemandHeader** table. For more information about vectors, see "Vector data" on page 138.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Date field is not a key field.
- The Header field is a reference, not an owning reference.

You can determine whether this table is using vector data by opening the Data Model dialog box and reviewing table icons. Tables that contain vector data are displayed with the icon.

Table 6.101: ForecastDetail (Mfg) fields

Field name	Description	Data type	Key
Date	The date of this forecast item.	Date	Yes
ID	An identifier for this forecast record.	String	Yes
Header	A reference to the historical demand header associated with the record. This identifies the part customer and demand category for this forecast. **Referenced table**: HistoricalDemandHeader **Reference Syntax**: Header.PartCustomer.Part.Name, Header.PartCustomer.Part.Site, Header.Category	Reference (Owning)	Yes
ProtectQuantity	Indicates whether the value in the Quantity field is modified when data is edited in a grouped worksheet. This field is used in the Advanced Data Editing dialog box as part of the expression that determines which records are not edited when a grouped worksheet is edited. Valid values are: • **Y**—the Quantity cannot be modified in summarized worksheets. • **N**—the Quantity can be modified. For more information, see "Protect a record" in the *RapidResponse Resource Authoring Guide*. **Default value**: N	Boolean	
Quantity	The amount of this forecast. The value in this field is used if the header's **HistoricalDemandCategoryType.UnitType = 'Quantity'**. For more information, see "HistoricalDemandCategory table" on page 329.	Quantity	
UnitPrice	Allows for a unique unit price to be specified for this forecast order. For example, this might be used to reflect the price effective during a sales promotion. If a non-negative value is provided here, it is always reported in the **EffectiveUnitPrice** field (typically used for revenue calculations). If a negative value is provided here, the unit price is calculated based on matching records in the **CustomerPrice** or **Part** tables. For more information, see EffectiveUnitPrice Default value: -1	Quantity	
Value	The total revenue target for this forecast. The value in this field is used if the header's **HistoricalDemandCategory.Type.UnitType = 'Value'**. For more information, see "HistoricalDemandCategoryType table" on page 736.	Money	

Notes:

- If a historical record of all forecast detail items in this table is required, then a process should be set up to archive data from this table into the **HistoricalDemandSeries** and **HistoricalDemandSeriesDetail** tables.
- Consensus forecast values calculated based on weighted combinations of different streams of forecast data in this table can be seen in the **ConsensusForecast** table, and are utilized elsewhere by RapidResponse analytics. For more information, see "ConsensusForecast table" on page 966.

ForecastDetail table – calculated fields

The following describes the calculated fields in the ForecastDetail table. For an introduction to this table, see "ForecastDetail table" on page 283.

Table 6.102: ForecastDetail calculated (Mfg) fields

Field name	Description	Data type	Key
EffectiveUnit Price	The effective unit price for this forecast order. This value is based on either the input unit price provided on this record or else the unit price effective for the forecast's part and customer (as defined through the Header field reference) on the date of this record. This value is useful when calculating revenue, and is calculated as follows: • If a non-negative value is provided in the **UnitPrice** field, then it is always used. • Otherwise, RapidResponse checks the **CustomerPrice** table for records with matching Part and Customer values that have effective dates less than or equal to the date of this record. If any matching records are found, then the **UnitPrice** from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, record with the larger value in the **CustomerPrice.Id** field is used. • However, if no matching Part and Customer values are found, RapidResponse next checks the **CustomerPrice** table for records with a matching Part and blank Customer value (that is, ' ') that have effective dates less than or equal to the record date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above or if the value provided in **CustomerPrice.UnitPrice** is negative, then the value in **Part.AverageSellingPrice** is reported here. However, if the value in **Part.AverageSellingPrice** is also negative, then **ForecastDetail.EffectiveUnit Price** is set to zero. Currency conversions on this field use the **Date** field's conversion rate.	Money	

Table 6.102: ForecastDetail calculated (Mfg) fields

Field name	Description	Data type	Key
ToleranceProfile Zone	A calculated reference to the tolerance profile zone to be used for evaluating whether changes to forecast details are inside or outside of tolerance. To choose a tolerance profile zone, this field first calculates the number of ToleranceCalendar intervals between the Date on this ForecastDetail record and the part's planning date as follows: `Date - Header.PartCustomer.Part.PlanningCalendars.RunDate.FirstDate Header.PartCustomer.ToleranceProfile.ToleranceCalendar` For the applicable tolerance profile, the ToleranceProfileZone record with the ZoneEndInterval field value that is nearest to and greater than the result of the calculation shown above is chosen. If no ToleranceProfileZone record meets this condition, the reference is Null. **Referenced table:** ToleranceProfileZone	Reference	

ForecastDisaggregationParameters table

This table holds parameter values at the part-customer and forecast category level that are used to override the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates.

The **ForecastDisaggregationParameters** table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained using the **S&OP Forecast Disaggregation** workbook in RapidResponse. For more information about forecast disaggregation, see the *RapidResponse Applications Guide*.

Notes:

- If the same disaggregation parameters should apply to all part-customers for a given forecast category, then override parameters can be specified on a record in the **ForecastDisaggregationParametersByCategory** table instead.
- Calculated disaggregation rates, whether based on the values in this table or other tables, can also be overridden for a given part customer and forecast category combination by providing override date and quantity (rate) values in the **ForecastDisaggregationOverride** table. For more information, see "ForecastDisaggregationOverride table" on page 290.

Table 6.103: ForecastDisaggregationParameters (Mfg) fields

Field name	Description	Data type	Key
ActualsCategory	A reference to the historical demand actual category used to calculate disaggregation rates for the part customer. **Referenced table:** HistoricalDemandCategory	Reference	
Header	A reference to the part customer and category combination this set of disaggregation parameters applies to. The category reference is used as the forecast category. **Referenced table:** HistoricalDemandHeader	Reference	Yes
HistoricalInterval Count	The number of periods of historical data to use when calculating disaggregation rates for this part customer and forecast category combination.	Integer	
Id	A unique identifier for this record.	String	Yes
InnerCalendar	The inner disaggregation calendar used in forecast disaggregation with seasonal trends to define the length of a season. For example, with season month-in-year disaggregation, this would be set to a monthly calendar. Months with higher values in them would then receive more of the data being disaggregated. If forecast disaggregation is not seasonal, this should be set to the same value as the OuterCalendar. **Referenced table:** Calendar	Reference	
OuterCalendar	The outer disaggregation calendar used in forecast disaggregation to define the period over which a forecast is disaggregated. For example, with season month-in-year disaggregation, this would be set to a yearly calendar. The calendar referenced by this field cannot represent smaller intervals than the one referenced by the InnerCalendar field (it can however be the same calendar if disaggregation is not seasonal). The OuterCalendar marker should also always land directly on the InnerCalendar marker. For example, with month-in-year disaggregation, then the Yearly calendar marker should land on a marker that defines the start of a monthly calendar. **Referenced table:** Calendar	Reference	

ForecastDisaggregationParametersByCategory table

The **ForecastDisaggregationParametersByCategory** table holds parameter values at the forecast category level that are used to override the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates. This table is appropriate for use when disaggregation parameters are required that apply to all part customers associated with a particular forecast category.

For example, all part customers associated with a given forecast category might have their disaggregation rates based on the same historical demand category and using the same range of historical data.

The **ForecastDisaggregationParametersByCategory** table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained using the **S&OP Forecast Disaggregation** workbook in RapidResponse. For more information about forecast disaggregation, see the *RapidResponse Applications Guide*.

Notes:

- If parameters need to be overridden for specific part customer and forecast category combinations, the **ForecastDisaggregationParameters** table should be used (values provided in that table take precedence over this one).
- Calculated disaggregation rates, whether based on parameter values in this table or other tables, can also be manually overridden for a given part customer and forecast category combination by providing override date and quantity (rate) values in the **ForecastDisaggregationOverride** table. For more information, see "ForecastDisaggregationOverride table" on page 290.

Table 6.104: ForecastDisaggregationParametersByCategory (Mfg) fields

Field name	Description	Data type	Key
ActualsCategory	A reference to the historical demand actual category used to calculate disaggregation rates for this forecast category. **Referenced table:** HistoricalDemandCategory	Reference	
Category	A reference to the forecast category that the set of disaggregation parameters defined on this record applies to. **Referenced table:** HistoricalDemandCategory	Reference	Yes
HistoricalInterval Count	The number of periods of historical data to use when calculating disaggregation rates for this forecast category.	Integer	
InnerCalendar	The inner disaggregation calendar used in forecast disaggregation with seasonal trends to define the length of a season. For example, with season month-in-year disaggregation, this would be set to a monthly calendar. Months with higher values in them would then receive more of the data being disaggregated. If forecast disaggregation is not seasonal, this should be set to the same value as the OuterCalendar. **Referenced table:** Calendar	Reference	

Table 6.104: ForecastDisaggregationParametersByCategory (Mfg) fields

Field name	Description	Data type	Key
OuterCalendar	The outer disaggregation calendar used in forecast disaggregation to define the period over which a forecast is disaggregated. For example, with season month-in-year disaggregation, this would be set to a yearly calendar. The calendar referenced by this field cannot represent smaller intervals than the one referenced by the InnerCalendar field (it can however be the same calendar if disaggregation is not seasonal). The OuterCalendar marker should also always land directly on the InnerCalendar marker. For example, with month-in-year disaggregation, then the Yearly calendar marker should land on a marker that defines the start of a monthly calendar. **Referenced table:** Calendar	Reference	

ForecastDisaggregationOverride table

This table provides disaggregation rates that override the calculated rates when disaggregating forecasts to the detail level.

If a record exists in this table for a specific part customer and forecast category on a given date, the quantity on that record is used as the effective rate in determining forecast disaggregation for that part customer, category and date. If a record does not exist in this table for a specific part customer and forecast category on a given date, then disaggregation rates are calculated based on the part-customer and forecast category specific parameters provided in the **ForecastDisaggregationParameters** table (if they exist, or else using the default disaggregation parameters for the scenario as specified in the **SOPAnalyticsConfiguration** table).

The **ForecastDisaggregationOverride** table supports Demand Planning in RapidResponse. Values in this table are maintained using the **S&OP Forecast Disaggregation** workbook in RapidResponse. For more information about forecast disaggregation, see the *RapidResponse Applications Guide*.

Notes:

- If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "Actuals", the **ForecastDisaggregationOverride** table can be used to specify rates for a particular part-customer and forecast category on specific dates and these will then be reported in the **DisaggregationRateByPartCustomer** table, rather than calculating them for that part-customer and category on that date.
- If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "ConsensusForecast" for calculating forecast disaggregation rates, the overrides in this table are not used in the **DisaggregationRateByPartCustomer** table.

Table 6.105: ForecastDisaggregationOverride (Mfg) fields

Field name	Description	Data type	Key
Date	The date on which the disaggregation rate becomes effective. The **ForecastDisaggregationOverride.Quantity** value is then considered effective from the specified **Date** until the next **DisaggregationCalendar** marker (at which point another rate override can be provided or else the calculated disaggregation rate will be used). **Note:** For more information about how disaggregation override rates are calculated, see "Override calculated disaggregation rates" on page 2092.	Date	
Header	A reference to the part customer and category combination this set of values applies to. The category reference is used as the forecast category. **Referenced table:** HistoricalDemandHeader	Reference	Yes
Id	A unique identifier for this record.	String	Yes
Quantity	The override quantity for the part customer and forecast category on the specified date. This value represents a rate and not the actual percentage used in disaggregation. For example, suppose the forecast for a given date is being disaggregated between two items X and Y, where X has an override quantity of 600 and Y has an override quantity of 200. The actual percentages used in disaggregation would be 75% on X (600/800) and 25% on Y (200/800).	Quantity	

ForecastDisaggregationOverride calculated fields

The following describes the calculated fields in the ForecastDisaggregationOverride table.

Table 6.106: ForecastDisaggregationOverride calculated (Mfg) fields

Field name	Description	Data type	Key
EffectiveUnit Price	The unit price effective for the forecast's part and customer (as defined through the Header field reference) on the date of this record. This value is useful when calculating revenue, and is calculated as follows: • RapidResponse checks the CustomerPrice table for records with matching Part and Customer values that have effective dates less than or equal to the date of this record. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, the EffectiveUnitPrice calculation uses the record with the larger value in the CustomerPrice.Id field. • However, if no matching Part and Customer values are found, RapidResponse next checks the CustomerPrice table for records with a matching Part and blank Customer value (that is, ' ') that have effective dates less than or equal to the record date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above, the value in Part.AverageSellingPrice is reported here. Currency conversions on this field use the **Date** field's conversion rate.	Money	

ForecastItem table

This table is used for the configuration of forecast items for which statistical forecasts can be generated. A record exists for each unique forecast item that has been calculated or configured in RapidResponse. A forecast item is made up of a combination of levels from different hierarchies that are accessible from the PartCustomer table.

The ForecastItem table supports Demand Planning and Statistical Forecasting in RapidResponse. Typically, the values in this table are automatically created based on attributes that are configured on the **PartCustomer** table during a RapidResponse deployment.

Table 6.107: ForecastItem (Mfg) fields

Field name	Description	Data type	Key
FilterExpression	A query filter expression that can be applied to the **PartCustomer** table to find all part customers belonging to this forecast item. **Note:** This field is obsolete	String	
Path	A unique identifier for this record. Each value in this field is a concatenated string consisting of all values that identify the forecast item. For example, it might consist of a part name and customer group.	String	Yes
PathExpression	The query expression that returned the string of values in the Path field at the time this ForecastItem record was created. RapidResponse uses the result of evaluating this expression to compare against the Path field and ensure that part customer are still assigned to the correct forecast item. **Note:** This field is obsolete	String	
PathLevels	A concatenated string of consisting of each levels of the reference path from the PartCustomer table to the value(s) reported in the Path field. **Note:** This field is obsolete (as of Version 2015.3).	String	
UsageRule	Controls whether the statistical forecast is calculated for this forecast item. Valid values are: • **Use**—calculate the statistical forecast for this item. • **Ignore**—do not calculate the statistical forecast for this item.	String (Enum)	

ForecastItem table – calculated and set fields

The following describes the calculated and set fields in the ForecastItem table. For an introduction to this table, and descriptions of its input fields, see "ForecastItem table" on page 292.

Table 6.108: Calculated ForecastItem (Mfg) fields

Field name	Description	Data type	Key
Effective Disaggregation Calendar	The disaggregation calendar used when disaggregating forecast quantities for this part customer. **PartCustomer.DisaggregationCalendar** is used when defined; otherwise, the disaggregation calendar specified in the **SOPAnalyticsConfiguration** table is used for disaggregation **Referenced table:** Calendar	Reference	
Parameters	Set of forecast item parameters associated with this ForecastItem record.	Set	
PartCustomers	Set of part customers associated with this ForecastItem record.	Set	

ForecastItemFitOutput table

This table shows the result of fitting the statistical model for forecast items. This table contains output parameters associated with a particular forecast model along with details generated from fitting the model such as model constants and error measures.

The ForecastItemFitOutput table supports Demand Planning and Statistical Forecasting in RapidResponse. This table is populated by running the **Generate and Save Forecast** command in the **S&OP Forecast Item Configuration** workbook. This table is considered obsolete and the **ForecastItemParametersFitOutput** table should generally be used instead.

Table 6.109: ForecastItemFitOutput (Mfg) fields

Field name	Description	Data type	Key
Class	The type of output parameter being reported in this record. Valid values are: • **DataCharacteristic**—a characteristic of the input data points characteristic. For example, "MeanQty", or "Count". • **ErrorMeasure**—error measure of fitted values against the input data points. For example, "MeanError". • **Message**—message type generated from the statistical models. For example, "Warning" or "Error" • **ModelConstant**—constants used by the model (alpha, beta, gamma, and so on). • **ModelOutput—**output from the statistical model that will be used to generate forecast.	String (Enum)	
ForecastModel	The forecast model used to generate this fitted output. Valid values are: • ARIMA • ARIMAX • BestFit • CrostonsMethod • DoubleExponentialSmoothing • ExponentialSmoothing • Elastic Net • HoltWintersMethod • Linear • MovingAverage • MultipleLinearRegression • StepWiseARIMA **Note:** For more information about these models, see "ForecastModel" on page 731.	String (Enum)	
ItemParameters	A reference to the forecast item parameters this fitted output applies to. The forecast item can also be found through this reference. **Referenced table:** ForecastItemParameters	Reference	Yes
ParameterName	The name of the output parameter. For example, "Error" or "Warning".	String	Yes
ParameterValue	The value associated with the named parameter. For example, this field might contain the description of an error.	String	

ForecastItemParameters table

This table contains input parameters used in generating a statistical forecast for a given forecast item and/or used in generated safety stock recommendations for a given safety stock item. Note that each forecast item can have several statistical forecasts generated, but each must be associated with a different forecast category.

The **ForecastItemParameters** table supports Demand Planning and Statistical Forecasting and Inventory Planning and Optimization in RapidResponse. Most values in this table are maintained through the **S&OP Forecast Item Management** workbook in RapidResponse, and new records are added to the table whenever new **ForecastItem** records are created.

The values in this table that relate to using multiple linear regression models for statistical forecasting, such as **ElasticNetParameter** and **WeightsDecayFactor**, are maintained in the **S&OP Regressor Management** workbook.

For more information, see "Statistical forecasting models" on page 2031.

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ActualsCategory	A reference to the historical actual demand category used as input in generating the statistical forecast for the forecast item. Actual demands in this category are adjusted for causal factors and then used as the basis for the statistical forecast. **Referenced table:** HistoricalDemandCategory	Reference	
AutoRegressive Terms	Specifies the auto-regression terms used in an ARIMA forecast calculation. Values for this field are typically a list of lag numbers. For example, `1,2,4` specifies the ARIMA model considers lags 1, 2, and 4 when calculating the statistical forecast. **Default value**: ' '	String	
BaseQuantity	Overrides the forecast quantities for an item to simulate a runout. This override replaces the quantity used in the statistical forecast. **Default value**: 0	Quantity	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
BestFitForecastLag	If using best fit forecasting with a holdout period, this indicates the first interval in the holdout period in which simulated forecasts generated from fitted models are compared against historical actuals for the purpose of calculating forecast error. A value of "1" is used by default and indicates to start the evaluation process in the first interval of the holdout period. Alternatively, a larger value can be specified to offset the forecast evaluation later into the holdout period. Typically, this might be used to account for a factor of lead time. This value should be expressed in intervals of the **SOPAnalyticsConfiguration.CycleCalendar**. **Default value**: 1 **Note:** This field is only applicable to forecast items where the **BestFitHoldoutPeriodIntervalCount** field contains a positive value, and where the **Type** reference has **ForecastModel** set to "BestFit" and **HoldoutPeriodUsageRule** set to either "SimpleHoldout" or "Simulation". For more information, see "Using best fit with a holdout period" on page 2053.	Integer	
BestFitHoldout PeriodInterval Count	Defines the duration of the holdout period used for best fit forecasting (expressed in intervals of the **SOPAnalyticsConfiguration.CycleCalendar**) The holdout period is applied immediately before **RunDate** and indicates the historical periods whose data is held out (excluded) when fitting statistical models to historical data. Instead, this period is used for evaluation purposes as simulated forecasts based on each of the fitted models are compared against the actuals in the holdout period. The model that minimizes the chosen error measure is then used to generate the statistical forecast starting at **RunDate**. **Default value**: 0 **Note:** This field is applicable only to those items whose **Type** reference has **ForecastModel** set to "BestFit" and **HoldoutPeriodUsageRule** set to "SimpleHoldout" or "Simulation". For more information, see "Using best fit with a holdout period" on page 2053.	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
BestFitProfile	A reference to a named profile used in best fit statistical forecasting (applicable when the item's **Type.ForecastModel** reference is "BestFit"). Each profile defines a group of statistical models that are considered during best fit forecasting for the item, along with parameter values applicable to certain models (these parameter values override corresponding parameters on this table). You can use a profile to include only the models which are most applicable to a given item. As a result, it can help reduce processing time by excluding other less appropriate models from the fitting process. Otherwise, if this reference is Null then all supported statistical models (other than optional "Rforecast" model) are considered when best fit forecasting is used for the item, and parameter values are as defined on this table. **Referenced table:** BestFitProfile (Nullable)	Reference	
BestFitProfile Detail	Specifies a specific forecast model and associated parameters to be automatically chosen and applied in best fit forecasting (applicable when the item's **ForecastItemType.Model** is "BestFit"). This field is typically populated after best fit forecasting has been run and the recommended model is deemed appropriate for the item. This locks the selected model to the item, and prevents the selection of a different model if statistical forecasting is run in the future. When this reference is Null, then the best fit forecasting process proceeds as normal through all applicable models. **Referenced table:** BestFitProfileDetail (Nullable)	Reference	
Constant	Indicates whether a constant value, also known as the Y intercept, is included in an ARIMA or ARIMAX forecast calculation. Valid values are: • **Use**—The constant is used in the calculation. • **Ignore**—For ARIMA, The constant is not used in the calculation. For ARIMAX, **Ignore** forces the constant term to be zero. **Default value**: Ignore	String	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ConfidenceLevel	A confidence level used to calculate prediction intervals in statistical forecasts that are generated using one of the following statistical forecasting models: • ARIMA • ARIMAX • Double exponential smoothing • Exponential smoothing • Holt-Winters (multiplicative and additive) • Linear • Multiple Linear Regression • Rforecast • Step-wise ARIMA Values between .5 and 0.9999 should be provided. Any value >= 1 is treated as 0.9999 to determine StatisticalForecast.PredictionIntervalLower and StatisticalForecast.PredictionIntervalUpper. Any value <= 0 indicates to ignore the field. **Note:** The value in this field is always ignored for items where a non-null reference is specified in **ForecastProfile** field, or a positive value is provided in either of the **BaseQuantity**, or **ScalingFactor** fields.	Quantity	
DifferenceLevel	The number of differencing operations to perform when calculating an ARIMA or ARIMAX function. If you enter a negative DifferenceLevel using ARIMA, ARIMA is not used. However, if you enter a negative DifferenceLevel using ARIMAX, the order is automatically chosen. **Default value**: -1 **Note:** It is strongly recommended that you do not enter a value that is greater than 2 in this field.	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ElasticNetParameter	A parameter, also known as alpha, that can be from 0 to 1 (inclusive) that controls how much penalization of the L1 and L2 norms is applied when using the Elastic Net forecast model. Elastic Net combines both the LASSO and ridge regression techniques to shrink coefficients. The value 1 is equivalent to LASSO and 0 is equivalent to ridge. This parameter works in conjunction with RegularizationParameter. **Default value**: 1 **Note:** This field was added in Version 2016.2, Service Update 27.	Quantity	
ForecastCategory	A reference to the forecast category the statistical forecast is generated for. Each forecast item can have one statistical forecast generated per forecast category. **Referenced table:** HistoricalDemandCategory	Reference	Yes
ForecastInterval Count	The number of future periods to generate the statistical forecast for. This value is expressed in terms of the **Type.IntervalsCalendar** reference.	Integer	
ForecastProfile	The profile points to apply to a statistical forecast. Points defined in the profile are applied starting at the **ProfileStartDate**. **Referenced table**: PointsProfile (Nullable)	Reference	
HistoricalInterval Count	The number of periods of historical demand to use as input into generating the statistical forecast. This value is expressed in terms of the **Type.IntervalsCalendar** reference.	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
HistorySkip LeadingZeros	Determines whether the first period bucket or buckets in the **ForecastItemParametersActual** table generate records in the **StatisticalForecastPredictActual** if the demand in these first period buckets is a quantity of zero (leading zeros). **Y**—Leading zeros are ignored when calculating historical data and the calculation begins with the first occurrence of a non-zero data point. **N**—Leading zeros are not ignored when calculating historical data. **Default value:** N **Note:** If **HistorySkipLeadingZeros** is set to "Y", the statistical forecast is computed beginning with the first bucket in the historical demand period (based on **ForecastItemParameters.HistoricalIntervalCount** and **ForecastItemParameters.HistoricalStartDate**) for which the demand quantity is not zero, but only if all earlier period buckets in the **ForecastItemParametersActual** table have a demand of zero. If the first period buckets in the table have demands with positive quantities, leading zeros are not ignored. **Note:** This field was added in Version 2016.2 Service Update 10.	Boolean	
HistoryStartDate	The first date when actual demand for the forecast item is available for use in calculating the forecast. This is used to set a start date for collecting historical data when there is less data available than the **HistoricalIntervalCount** setting implies. This avoids including empty historical periods in forecast calculations. For example, if there is 9 months of history available for the item, but **HistoricalIntervalCount** is set to "12", then this field could be set to a date 9 months before the **RunDate**.	Date	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
IgnoreBestFitProfileDetailParameters	When a forecast item is using a **BestFitProfile** to define the list of models to be considered for BestFit, this parameter ignores the model parameter values that are associated with the profile in **BestFitProfileDetails** and applies the model parameter values from the **ForecastItemParameters** record to the models in the profile instead. The following values are valid: • **Y**—Use the model parameters in the **ForecastItemParameters** record instead of those specified in **BestFitProfileDetails**. • **N**—Use the model parameters in the **BestFitProfileDetails** records. **Default value: N** **Note:** For more information, see "Applying best fit profiles" on page 2056.	Boolean	
Item	A reference to the forecast item this set of parameters applies to. **Referenced table:** ForecastItem	Reference	Yes
MovingAverage IntervalCount	Number of previous periods of demand, expressed in terms of the **Type.IntervalsCalendar** reference, to use for statistical forecasts calculated using the MovingAverage or BestFit models as follows: • If < 0 and **Type.ForecastModel** is set to "BestFit", then "MovingAverage" is excluded from consideration. • If = 0 and **Type.ForecastModel** is set to "BestFit", then this value is calculated for the statistical forecast so as to minimize the error measure set by **Type.FitMeasure**. • If <=0 and **Type.ForecastModel** is set to "MovingAverage", then this value is calculated for the statistical forecast so as to minimize the error measure set by **Type.FitMeasure.** • If > 0 and **Type.ForecastModel** is set to either "MovingAverage" or "BestFit", then the value provided here is used**.** Default value: -1	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
MovingAverage Terms	Specifies the number of moving average terms used in an ARIMA forecast calculation. Values for this field are typically a list of lag numbers. For example, `2,3` specifies that the ARIMA model considers lags 2 and 3 when calculating the statistical forecast. **Default value**: ' '	String	
OutlierMoving AverageWindow	Number of historical intervals used in calculating the moving average for use in outlier detection. Only applicable when **OutlierType.DataRule** is set to "MovingAverageError" which uses the difference between historical data points and the calculated moving average to determine when determining which points are outliers. **Default value:** 3	Integer	
OutlierSmoothing AfterIntervalCount	When smoothing demand outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods forward over which to spread the values. This occurs after any backward spreading. **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	
OutlierSmoothing BeforeIntervalCount	When smoothing demand outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods backward over which to spread the values. This occurs after any forward spreading. **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
OutlierThreshold	Sets a threshold for detecting outliers within historical data used in generating the statistical forecast. The value must be > 0. This value is used to determine both a value above which a point is considered an outlier (upper threshold) and a value below which a point is considered an outlier (lower threshold). The actual interpretation of the value in this field is influenced by the **OutlierType.DetectionRule** setting as follows: • **IglewiczHoaglinMethod**—A modified Z-score. The Z-score value of should be based on the Normal Distribution table. A Z-score of less than 2.33 corresponds to a 99% probability that the data point is not an outlier. The following table lists probabilities and their corresponding Z-score: **% Prob** / **Z-score** 90.0 / 1.28 95.0 / 1.64 96.0 / 1.75 97.0 / 1.88 98.0 / 2.05 99.0 / 2.33 99.5 / 2.58 99.6 / 2.65 99.7 / 2.75 99.8 / 2.88 99.9 / 3.09 • **StandardDeviation**—Number of standard deviations from the mean. For example, 3 indicates that points more than three standard deviations from the mean are considered outlier. • **Winsorizing**—A percentile within the data. A value between .01 and .49 should be specified. For example, .05 indicates that points below the 5th percentile or above the 95th percentile are considered outliers. **Default value:** 3	Quantity	
OutlierType	A reference to a set of processing rules that define how outliers in historical data are identified and subsequently processed. **Referenced table:** OutlierType (Nullable)	Reference	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ProfileStartDate	The date to begin applying the **ForecastProfile**, **BaseQuantity**, and **ScalingFactor** for a statistical forecast.	Date	
RegressionModel	A reference to the regression model that is used by the forecast item. This field applies only to forecast items that are using multiple linear regression to generate the statistical forecast. **Referenced table:** RegressionModel (Nullable) **Note:** This field was added in Version 2016.2, Service Update 27.	Reference	
RegularizationParameter	A parameter, also known as lambda, that controls the strength of the penalty term for coefficients that is used in Elastic Net regression. This parameter is dependent on the ElasticNetParameter to shrink the coefficients of regressors. The value that is used for RegularizationParameter is reported in the StatisticalForecastFitOutput table. **Default value**: -1 **Note:** If you set a negative value in this field, the value of the regularization parameter will be chosen by the model. Because the implementation of the underlying model in R involves randomization, you may notice that the value of the regularization parameter and the statistical forecast that is generated may be different from run to run. **Note:** This field was added in Version 2016.2, Service Update 27.	Quantity	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ResidualAutocorrelationConfidenceLevel	A value used to calculate confidence levels for residual autocorrelation when using statistical forecasts that are generated using one of the following statistical forecasting models: • ARIMAX • Elastic Net • Multiple Linear Regression Autocorrelation measures the relationship between the residuals in one time period to the residuals in a previous, or lagged, time period to determine the correlation between the two values. Values must be between 0 and 1. Values that are <0 or >1 result in upper and lower confidence bounds of -1, which indicates that no confidence interval was generated. **Default value**: .95 **Note:** This field was added in Version 2016.2, Service Update 27.		
Reviewed	Whether the newly-entered forecast item has been reviewed for correctness. This field is for reporting only. **Default value**: N	Boolean	
RollingError IntervalCount	Indicates the number of intervals in the holdout period that should be used for measuring forecast error when the "RollingLagPercentageError" fit measure is chosen. This value is specified in **SOPAnalyticsConfiguration.CycleCalendar** intervals. If the value specified is greater than the number of intervals in the holdout period, then forecast error calculations continue until **RunDate**. This field is only applicable for items whose **Type** reference has **ForecastModel** set to "BestFit" and **HoldoutPeriodUsageRule** set to a value other than "Ignore". **Default value:** 3	Integer	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
RParameterSet	A reference to an optional set of parameter values to be used by named forecasting functions in the R statistical programming language. This reference is only applicable when the **ForecastModel** is set to "Rforecast", and is intended for advanced uses. If left Null when using "Rforecast", then the functions in R are run using a standard set of parameters based on field values and data passed from RapidResponse. **Referenced table:** RParameterSet (Nullable) **Note:** This field supports the optional R Forecast capability.	Reference	
ScalingFactor	A multiplier applied to the **BaseQuantity** value for the forecast items. Values less than zero are ignored. **Default value**: 1	Quantity	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
SeasonalInterval Count	This field indicates the number of intervals in a season, in terms of the **Type.IntervalsCalendar** reference, for statistical forecasts calculated using the BestFit, Elastic Net, HoltWintersMethod, MultipleLinearRegression, or Rforecast models as follows: • If < 0 and **Type.ForecastModel** is set to BestFit, then HoltWintersMethod and MultipleLinearRegression are excluded from consideration. Therefore, it does not try to find seasonality. • If < 0 and **Type.ForecastModel** is set to Rforecast, then this value is calculated as 1 and no seasonality is applied. • If < 0 and **Type.ForecastModel** is set to Elastic Net or MultipleLinearRegression, then no seasonality is applied. • If = 0 and **Type.ForecastModel** is set to BestFit, then this value is calculated for the statistical forecast so as to minimize the error measure set by **Type.FitMeasure**. • If = 0 and **Type.ForecastModel** is set to Rforecast, Elastic Net, or MultipleLinearRegression, then R chooses the seasonality value that is calculated for the statistical forecast. • If <= 0 and **Type.ForecastModel** is set to HoltWintersMethod, then this value is calculated for the statistical forecast so as to minimize the error measure set by **Type.FitMeasure**. • If > 0 and **Type.ForecastModel** is set to either BestFit, Elastic Net, HoltWintersMethod, MultipleLinearRegression, or Rforecast, then the value provided here is used. Default value: -1 **Note:** The support for the Rforecast was added in Version 2016.2 Service Update 13. The support for the MultipleLinearRegression was added in Version 2016.2 Service Update 27.	Integer	
StartDate	The first date the forecast for this item is calculated. Values earlier than this date are considered to be zero. If this value is Undefined, Past is used.	Date	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
StopDate	The last date the forecast for this item is calculated. Values later than this date are considered zero. If this value is Undefined, Future is used.	Date	
TrendDecayFactor	A decay factor to dampen trend that can be applied to the following forecast models: • AdditiveHoltWintersMethod • DoubleExponentialSmoothing • ElasticNet • HoltWintersMethod • MultipleLinearRegression • Rforecast For AdditiveHoltWintersMethod, DoubleExponentialSmoothing, and HoltWintersMethod, if the trend should decay over time, this value must fall between 0 and 1. All values less than 0 result in complete elimination of the trend. If neither decay nor elimination is required, this value should be 1 or greater. For ElasticNet and MultipleLinearRegression, the trend damping applies only to the generated statistical forecast, not to past fitted values. In addition, if you enter a TrendDecayFactor that is equal or less than 0, this will result in complete elimination of the trend. If neither decay nor elimination is required, this value should be 1 or greater. For Rforecast, if you enter a TrendDecayFactor that is less than 0 or equal or greater than 1, then trend is dampening is ignored. If you enter a value of 0, then R automatically detects trend and the forecast model can select decay factor as part of the fit. If TrendDecayFactor is greater than 0 and less than 1, then this value is used for trend dampening. The use of the decay factor is controlled by the **TrendDecayFactorRule** option on the **ForecastItemParametersType** table.	Quantity	
Type	A value that determines processing policies for this set of forecast item parameters such as the forecast model and planning calendar to use when generating the statistical forecast. **Referenced table:** ForecastItemParametersType	Reference	

Table 6.110: ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
UnitofMeasure	The unit of measure that is used in the statistical forecast generated for this record. All inputs used in the statistical forecast are converted to this unit of measure. **Referenced table:** UnitOfMeasure table **Note:** To successfully use the **Unit of Measure** control in any of the workbooks, it's necessary to assign a forecast item unit of measure to forecast items even if they are reported in the same unit of measure as their part-customers. **Note:** This field was added in Version 2016.2 Service Update 12.	Reference	
UseOwnDemand	Whether historical demand for this forecast item is included in determining the total historical demand that is used to calculate the item's statistical forecast. The following values are valid: • **Use**—The demand for this forecast item is included in determining the total historical demand. • **Ignore**—The demand for this forecast item is not included in determining the total historical demand.	String	
WeightsDecayFactor	The factor to use to decrease the influence of regressors over time. When WeightDecayFactor is set, the regressor values in more recent time periods are given more importance than the values in earlier time periods in the statistical forecast calculation. It is an optional value that is used to improve model fit. You can apply WeightsDecayFactor to a forecast item that uses the Elastic Net or Multiple Linear Regression statistical forecasting methods. This value must fall be greater than 0 and less than 1. You cannot use a negative value. The formula used to calculate weight decay is as follows: For the t-th observation in the historical set Quantity = WeightsDecayFactor^ (HistoricalIntervalCount - t) **Default value**: 1 **Note:** This field was added in Version 2016.2, Service Update 27.	Quantity	

ForecastItemParameters table – calculated and set fields

The following describes the calculated and set fields in the **ForecastItemParameters** table. For an introduction to this table, and descriptions of its input fields, see "ForecastItemParameters table" on page 296.

Table 6.111: Calculated ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
CalcBestFitProfile Detail	If a profile was used in best fit forecasting for the item, this returns a reference to the actual **BestFitProfileDetail** record associated with the chosen forecast model and parameters. Otherwise, this reference is Null.	Reference	
Disaggregation Rates	Set of statistical forecast disaggregation rates calculated for the forecast item.	Set	
EffectiveForecast Model	The model used to generate the statistical forecast associated with this item. For all forecast models other than "BestFit", this always returns the referenced **ForecastModel** value from the **ForecastItemParametersType** table. For the "BestFit" forecast model, this returns one of the following: • The **ForecastModel** referenced from the **ForecastItemParametersType** table (if the **BestFitProfileDetail** reference is Null). • The **ForecastModel** referenced from the **BestFitProfileDetail** table (if the **BestFitProfileDetail** reference is populated).	String (Enum)	
EffectiveHistory StartDate	The earliest **HistoryStartDate** value for this forecast item or any other item referenced from the **ForecastItemParametersMap** table. Any Undefined dates are ignored. If all dates are Undefined, Past is returned.	Date	
EventStatistical ForecastDetails	Set of statistical forecast quantities for events for the forecast item.	Set	
FitOutput	Set of forecast item outputs associated with this ForecastItemParameters record.	Set	
ForecastItem ParametersActuals	Set of historical demands for the forecast item.	Set	

Table 6.111: Calculated ForecastItemParameters (Mfg) fields

Field name	Description	Data type	Key
ForecastItem Parameters FitOutputs	Set of forecast item parameters fit outputs associated with this **ForecastItemParameters** record.	Set	
ForecastItem ParametersMaps	Set of mappings defined for this ForecastItemParameters record.	Set	
ForecastItemParameters OutlierCalcs	Set of outlier calculations used for a forecast item.	Set	
ForecastItemParameters OutlierFitOutputCalcs	Set of outlier fit output calculations used for a forecast item.	Set	
ForecastItem Parameters ReferenceMaps	Set of referenced mappings defined for the ForecastItemParameters record.	Set	
ForecastItemParameters RegressionWeights	Set of regression weights used for the forecast item.	Set	
ForecastItemParameters Regressors	Set of regressors associated with this forecast item.	Set	
ImportedStatistical Forecasts	Set of imported statistical forecast records for the forecast item. **Note:** This field was added in Version 2016.2, Service Update 5.	VectorSet	
Outliers	Set of outlier adjustments for the forecast item.	Set	
RegressionCoefficients	Set of regression coefficients generated for a forecast item.	Set	
StatisticalForecasts	Set of statistical forecasts calculated for the forecast item.	Set	
StatisticalForecast Details	Set of disaggregated statistical forecast quantities for the forecast item.	Set	
StatisticalForecast FitOutputs	Set of statistical forecast item outputs associated with this **ForecastItemParameters** record.	Set	
StatisticalForecast PredictActuals	Set of forecast parameters generated for the forecast item that have been fitted with historical actuals.	Set	

ForecastItemParametersFitOutput table

The **ForecastItemParametersFitOutput** table saves the fit output generated by **StatisticalForecastFitOutput**, a calculated table. The fit output consists of statistical model parameters used to calculate the statistical forecast and various calculated data characteristics and error measures of the statistical model.

This table is used in RapidResponse 2014.4 (or later) by running the **Generate and Save Forecast** command in the **S&OP Forecast Item Management** workbook and when **ForecastItemParametersType.ModelConstantUsage** is set to "Use". The model constants used in the statistical forecast calculations are provided by this table.

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
BestFitProfile Detail	A reference to the detailed profile, and associated parameters, used in the process of fitting and selecting the best forecast model. This reference is populated on records where **Class** is "BestFit", and a best fit profile was used in selecting the model. **Referenced table:** BestFitProfileDetail	Reference	
Class	The type or category of forecast parameter being reported in the record. The name of the parameter is reported in the **ValueName** field for all classes except Message. Valid values are: • **BestFit**—the statistical model with the least margin of error. This field reports "BestFit" when **ForecastItemParametersType.ForecastModel** is set to "BestFit". The three models with the least margin of error are reported in the **ValueName** field. • **DataCharacteristic**—a characteristic of the input data points. • **ErrorMeasure**—an error measure of fitted values against the input data points. • **Message**—a message type generated from the statistical models. The message is available in the **ValueText** field. • **ModelConstant**—the constant values used by the forecast model. • **ModelOutput**—the output from the statistical model that will be used to generate forecast. • **StepWiseARIMA**—the ARIMA model with the least margin of error. This field reports "StepWiseARIMA" when **ForecastItemParametersType.ForecastModel** is set to "StepWiseARIMA". The three models with the least margin of error are reported in the **ValueName** field.	String (Enum)	

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
ForecastModel	The statistical model that will be used to calculate the statistical forecast. For more detailed information about the forecast models supported by RapidResponse, see "Statistical forecasting models" on page 2031. Valid values are: • **AdditiveHoltWintersMethod**—differs from the Holt-Winters model in that forecast points are calculated in an additive, rather than multiplicative, manner. Suitable for demand that has constant variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **ARIMA**—calculates forecasts for a time period using a linear combination of actual values and error values in a time series. This model can be used on a stationary time series, or a differencing function can be applied to non-stationary data to make it stationary. For a more detailed description, see "Auto-Regressive Integrated Moving Average (ARIMA)" on page 2034. • **ARIMAX**—Is an extension of the ARIMA model, but also includes independent variables (regressors) in the forecast calculation. This makes ARIMAX a multiple linear regression model. To improve forecast accuracy, ARIMAX also takes into consideration autocorrelation that may be found in regressor residuals. For a more detailed description, see "ARIMAX" on page 2042. • **BestFit**—fits each of the other forecast models and selects the one with the least margin of error. For a more detailed description, see "Best fit" on page 2053. • **CrostonsMethod**—uses exponential smoothing with a single smoothing constant for the forecast values, but calculates two different values; the first being the forecast values at a point, and the second being the average length of time between forecasts. These values determine the forecast quantities and the periods that have forecast demands. Suitable for demand that is not constant, such as lumpy demand that contains periods of zero quantities. For a more detailed description, see "Croston's method" on page 2034. • **DoubleExponentialSmoothing**—considers the data points and trends within the historical data, and applies a smoothing algorithm to both. The result is then used as the statistical forecast. Suitable for demand that is fairly stable but trending up or down over time. For a more detailed description, see "Double exponential smoothing" on page 2036. • **ElasticNet**—A multiple linear regression model that uses regularized, also known as penalized, regression methods. It blends Lasso and ridge regression techniques by penalizing both L1 and L2 norms to reduce the coefficient vector. To use ElasticNet, you must attach a minimum of two	String (Enum)	

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
	regressors to a forecast item or you won't be able to generate a statistical forecast. For a more detailed description, see "ElasticNet" on page 2043. • **ExponentialSmoothing**— considers data points within the historical data, and applies a smoothing algorithm. The result of the final data point is then used as a constant value for the statistical forecast. Suitable for demand that is fairly stable and lacks a trend. For a more detailed description, see "Exponential smoothing" on page 2037. • **ForecastImport**—means that the statistical forecast is generated using an external tool. The statistical forecast values that are generated using an external tool are saved in the ImportedStatisticalForecast table. For more information, see "ForecastImport" on page 2058. • **HoltWintersMethod**—similar to double exponential smoothing in that it considers data points and trends within the historical data, but also accounts for seasonal trends. That is, the values for each point in the forecast and trend are further adjusted to account for seasonal demand changes. This model is the Holt-Winters multiplicative method, so it is suitable for demand that has proportional variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **Linear**—performs a linear regression on a set of historical data points to generate a trend line that determines statistical forecast quantities. Suitable for demand that does not fluctuate. For a more detailed description, see "Linear" on page 2039. • **MovingAverage**—calculates the simple moving average of a specified number of previous data points to generate a constant forecast. Suitable for demand that is fairly stable and does not change much over time. For a more detailed description, see "Moving average" on page 2039. • **MultipleLinearRegression**—Performs a linear regression using multiple independent variables to predict the outcome of one dependent variable. For a more detailed description, see "Multiple linear regression" on page 2043. • **StepWiseARIMA**—identical to the ARIMA model, but all parameters are calculated automatically. Step-wise ARIMA selects the order of difference through KPSS unit-root tests and selects AR order and MA order by minimizing AIC. For a more detailed description, see "Step-wise ARIMA" on page 2039. **Note:** This field was modified in Version 2016.2, Service Update 27.		

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters (Owner)	Reference	Yes
ValueDate	The forecast parameter **MeanDate**, a data characteristic, is expressed as a date. This field is populated if the value in the **ValueType** field is "Date".	Date	

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
ValueName	The name of the forecast parameter. The **Class** field provides the category of the forecast parameter, then the **Name** field provides the specific parameter being reported, so various names could be reported for a given class. For example, if "ModelConstant" is the class of the parameter, the name could be "Alpha", indicating that the forecast model uses the alpha constant when performing its calculations (alpha's quantity is then reported in the **ValueQuantity** field). Data characteristics: • **Count**—reports the number of values in your input set. • **MeanDate**—determines the average of all dates included in the statistical forecast, and the approximate midpoint of the dates used in the statistical calculations. • **MeanQuantity**—determines the average of all actual values in the input data. • **StandardErrorOfY**—determines how values differ from the mean. • **TotalSumSquares**—determines the total difference between each data point and the mean. • **Variance**—determines how data values differ from the mean. For more information about data characteristics, see "Statistical forecasting models" on page 2031. Error measures: • **AdjustedRSquared**—determines the ratio between the regression sum of squares and total sum of squares measures, but is influenced by the number of values in the input data set. • **AkaikeInformationCriterion (AIC)**—determines how well a statistical model fits the values it was generated from using the log-likelihood result and double the number of estimated parameters. This measure is reported only if the ARIMA or Step-wise ARIMA models are used. • **ForecastError**—determines forecast accuracy by showing the adjusted error of the predicted actuals compared to the historical actuals. • **Hannan-QuinnCriterion (HQC)**—determines how well a statistical model fits the values it was generated from, using the log-likelihood result and the product of double the number of parameters and the logarithm of the logarithm of the number of observed samples. This measure is reported only if the ARIMA or Step-wise ARIMA models are used. • **LogLikelihood**—determines the maximum likelihood for a statistical function. This measure is reported only if the ARIMA or Step-wise ARIMA models are used. • **MeanAbsoluteError**—determines the absolute value of the variance between the historical actual values and the	String (Enum)	

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
	predicted forecast values. • **MeanAbsoluteErrorByMean**—normalizes the value returned by the mean absolute error measure by dividing it by the mean quantity of the input data. • **MeanAbsolutePercentageError**—determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. • **MeanAbsolutePercentageErrorByForecast**—determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage. • **MeanAbsoluteScaledError (seasonal)**—determines the mean absolute error of the forecast divided by the mean absolute error of the seasonal naïve forecast. • **MeanError**—determines the average variance between the historical actual values and the predicted forecast values. • **MeanPercentageError**—determines the average variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. • **MeanPercentageErrorByForecast**—determines the average variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage. • **MeanSquareErrors**—calculates the average of the residual sum of squares for the forecast. • **RegressionSumSquares**—determines the difference between the total sum of squares of the input data and the residual sum of squares, and shows how the variance of the statistical forecast and input values differs from the variance of the input values and average of input values. • **ResidualSumSquares**—calculates the sum of the square of differences between the historical actual values and the values predicted by the statistical forecast. • **RootMeanSquareErrors**—determines the square root of the value produced by the mean square errors measure, and shows the correlation between the historical actual values and the results of the statistical forecast calculation. • **RSquared**—determines the ratio between the regression sum of squares and total sum of squares measures, and shows how closely the forecast values fit the historical actual values. • **SchwarzBayesianInformationCriterion (SBC)**—determines how well a statistical model fits the values it was generated from using the log-likelihood result and the product of the number of parameters and logarithm of the number of observed samples. This measure is reported only if the		

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
	ARIMA or Step-wise ARIMA models are used. • **VarianceError**—determines if the forecast model produces forecasts with more stable forecast errors or unstable, varying forecast errors by showing how widely the errors are spread around the mean error. For more information about error measures, see "Error measures" on page 2060. Model constants: • **Alpha**—a smoothing constant, required for exponential smoothing, double exponential smoothing, Holt-Winters, additive Holt-Winters, and Croston's method. • **Beta**—a seasonal trend smoothing constant, required for Holt-Winters and additive Holt-Winters. • **Gamma**—a trend smoothing constant, required for double exponential smoothing, Holt-Winters, and additive Holt-Winters. • **Mu**—used only for ARIMA, if a constant is included in the calculation. • **Phi**—used only for ARIMA, and must be provided for each auto-regressive and moving average lag included in the ARIMA calculation. • **Theta**—used only for ARIMA, and must be provided for each auto-regressive and moving average lag included in the ARIMA calculation. Model outputs: • **AutoRegressiveTerms**—the auto-regressive values considered for the ARIMA calculation. • **DifferenceLevel**—determines how many differencing functions are performed on the data when using the ARIMA or ARIMAX model. • **FitError**—any error encountered during the model fit process. Can be either "None", "Error", or "CriticalError". The error message is displayed in the **ValueText** field with "Message" in the **Class** field. • **Forecast**—the calculated forecast quantity reported across all forecast points when using exponential smoothing, moving average, or Croston's method. • **IncludeConstant**—determines whether a constant value is included in the ARIMA and ARIMAX calculations. • **InterarrivalValue**—the final estimated inter-arrival time between forecasts, which is the unrounded SporadicInterval, when using the Croston's method. • **KPSS_UnitRootTest_(i)**—the calculated difference level used in the Step-wise ARIMA calculation. • **MovingAverageIntervalCount**— the specified number of historical periods used to calculate the moving average. • **MovingAverageTerms**—the moving average values		

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
	considered for the ARIMA calculation. • **RegularizationParameter**—A parameter, also known as lambda, that controls the strength of the penalty term for coefficients that is used in Elastic Net regression. • **Root_AR_(i)**—the (i) root of the auto-regressive terms of the ARIMA calculation. • **Root_AR_(i)_Modulus**—the modulus of (i) root of the auto-regressive terms of the ARIMA calculation. • **Root_MA_(i)**—the (i) root of the moving average terms of the ARIMA calculation. • **Root_MA_(i)_Modulus**—the modulus of (i) root of the moving average terms of the ARIMA calculation. • **SeasonalIntervalCount**—the duration of a season, based on the intervals calendar; used for Holt-Winters and additive Holt-Winters. • **SeasonalIntervalCount**—the duration of a season, based on the intervals calendar; used for Elastic Net, Holt-Winters, additive Holt-Winters, and Multiple Linear Regression. • **SeasonalityValue_(i)**—the value calculated for each season; use for Holt-Winters and additive Holt-Winters. • **SmoothedValue**—the value required for statistical models that use smoothing, namely exponential smoothing, double exponential smoothing, Holt-Winters and additive Holt-Winters. For the Croston's method, it is the estimated non-zero demand at every InterarrivalValue. • **TrendValue**—the value used to represent a trend, used for double exponential smoothing, Holt-Winters, and additive Holt-Winters. Best fit: • **ErrorMeasure**—the error measure used to determine the best fit. • The names of the three statistical models with the least margin of error. Step-Wise ARIMA: • The names of the three ARIMA models with the least margin of error. **Note:** The **MeanAbsoluteScaledError** (seasonal) option was added in Version 2016.2, Service Update 13. **Note:** The **Interarrival** model output was added and the **SmoothedValue** was modified in Version 2016.2, Service Update 15. **Note:** The **RegularizationParameter** model output was added in Version 2016.2, Service Update 27. **DifferenceLevel**, **IncludeConstant**, and **SeasonalIntervalCount** were also modified in this update to support multiple linear regression models.		

Table 6.112: ForecastItemParametersFitOutput (Mfg) fields

Field name	Description	Data type	Key
ValueQuantity	Error measures, model constants, model outputs, and most data characteristics are expressed as quantities. This field is populated if the value in the **ValueType** field is "Quantity".	Quantity	
ValueText	Messages or arrays needed for ARIMA are expressed as strings. This field is populated if the value in the **ValueType** field is "String".	String	
ValueType	The data type of the forecast parameter. The data types are: • Text • Quantity • Date	String (Enum)	

ForecastItemParametersMap table

This table defines mappings between forecast items, which allows the historical demand from one item to be used in calculating the statistical forecast of another. For example, for introducing a new product, the history from the item it replaces can be used to calculate the statistical forecast by defining a mapping from the new item to the old.

Table 6.113: ForecastItemParametersMap (Mfg) fields

Field name	Description	Data type	Key
Item	The forecast item parameters the mapping is defined for. **Referenced table**: ForecastItemParameters	Reference	Yes
Multiplier	Scales the historical quantities for the referenced forecast item. **Default value**: 1.0	Quantity	
Lag	The number of **ForecastItemParametersType.IntervalsCalendar** periods after the **HistoryStartDate** at which to begin using the historical demand of the referenced forecast item in statistical forecast calculations. If a negative value is set for Lag, the statistical forecast could begin using the historical demand before the **HistoryStartDate**. **Default value**: 0 **Note:** This field was added in Version 2016.2, Service Update 13.	Integer	
ReferenceItem	The forecast item parameters that are mapped to the **Item**. **Referenced table**: ForecastItemParameters	Reference	Yes

ForecastItemParametersOutlier table

The **ForecastItemParametersOutlier** table is an input table. It stores outlier adjustments at the forecast item level, which means that items that belong to more than one forecast category in **ForecastItemParameters** can use different outlier adjustments for each category.

Table 6.114: ForecastItemParametersOutlier

Field name	Description	Data type	Key
Date	The date of the outlier.	Date	
ForecastItem Parameters	A reference to the forecast item the outlier adjustments should be applied to. **Referenced table:** ForecastItemParameters	Reference	Yes
Id	A unique string identifier for the outlier adjustment.	String	Yes
Notes	Descriptive notes appended to this record This field can used to enter notes about a ForecastItemParametersOutlier. Each time a note value is added to this field through a worksheet, a new record is automatically created in the ForecastItemParametersOutlier_Notes table with a date and timestamp, user ID, note text, and a reference to the ForecastItemParametersOutlier record against which it was created. **Referenced table:** ForecastItemParametersOutlier_Notes	Reference	
ProcessingRule	Specifies when the outlier adjustment defined on this record should be used. Valid values are: • **All**—the quantity on the record is used in both statistical forecasting and the outlier detection process. • **StatisticalForecastOnly**—the quantity on the record is used in statistical forecasting only (ignored during the outlier detection process).	String (Enum)	
Quantity	The quantity of the outlier adjustment. This can be a negative value.	Quantity	

ForecastItemParametersRegressionCoefficient table

This table saves the regression coefficients that are generated by **StatisticalForecastRegressionCoefficient**, which is a calculated table. The coefficients help determine if the regressors play a significant role in predicting demand. The table saves the coefficients the first time you generate a statistical forecast for a forecast item that uses a multiple regression model, such as

ARIMAX or Elastic Net. Once the coefficients are saved to this table, they are used in all subsequent forecasts, instead of being recalculated. However, to save coefficients to and to use the coefficients from the **ForecastItemParametersRegressionCoefficient** table, **ForecastItemParametersType.ModelConstantUsage** must be set to **Use**. For more information, see "ForecastItemParametersType table" on page 727

The reporting of the coefficient results also reflect the following:

- Values in the **ForecastItemParameters.WeightsDecayFactor**.
- Values in the **ForecastItemParameters.RegularizationParameter**.
- Values in the **ForecastItemParameters.ElasticNetParameter**.
- Trend and seasonality as reported by R.

Notes:

- Depending on the forecast model that you choose, not all properties of the regression coefficients are reported. For example, if you are using the Elastic Net model, only the **Estimate** is reported. If you are using ARIMAX, only **Estimate** and **StandardErrors** are reported.
- The **Estimate** field is the only field in this table that is used in calculations that are reported in the **StatisticalForecast** forecast table.

The **ForecastItemParametersRegressionCoefficient** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table is populated by running the **Generate and Save Forecast** command in the **S&OP Forecast Item Configuration** workbook. The values in this table are reported in the **S&OP Regressor Management** workbook. This table was introduced in version 2016.2 Service Update 27.

Table 6.115: ForecastItemParametersRegressionCoefficient (Mfg) fields

Field name	Description	Data type	Key
Estimate	A value that indicates the direction and magnitude of the relationship between the regressors and demand. It represents the mean change in the regressors' response given one unit change in the demand. • A positive estimate indicates that an increase of the regressors results in an increase in demand. • A negative estimate indicates that an increase of the regressors results in a decrease in demand.	Quantity	
ItemParameters	A reference to the forecast item that is associated with these regression coefficients. **Referenced table**: ForecastItemParameters	Reference	Yes
Name	The unique name that is used to identify the model term, or intercept, that is associated with these coefficients.	String	Yes

Table 6.115: ForecastItemParametersRegressionCoefficient (Mfg) fields

Field name	Description	Data type	Key
pValue	A value that indicates the statistical significance of the correlation between demand and the regressor. It reflects the probability that the observed relationship between demand and the regressors in the historical data would hold true were they not correlated. • A low p-value indicates that there is a correlation between the regressors and demand. • A high p-value indicates that there is little correlation between the regressors and demand.	Quantity	
StandardError	A value that represents the standard deviation of the statistical sample mean. By analyzing the spread of the data points in the historical data, the standard error measures how closely the historical data represents the overall data set. The more data points are measured in the calculation of the mean results in a lower standard error. A lower standard error indicates the more representative the sample is of the overall data set. This value is always positive.	Quantity	
tStatistic	A value that represents the estimate divided by the standard error. You can use the tStatistic to determine if you should support or reject that the regression coefficient is equal to 0 (null hypothesis). A high t-statistic indicates a greater statistical significance, therefore, provides evidence that the regression coefficient is not equal to 0.	Quantity	

HistoricalCurrencyConversion table

This table contains historical conversion rates for currencies. The conversion rates from the **CurrencyConversionForecast** table can be periodically written to this table to maintain historical records of rates.

Table 6.116: HistoricalCurrencyConversion (Core) fields

Field name	Description	Data type	Key
Currency	The currency the rate is calculated for. **Referenced table**: Currency	Reference	
Date	The date the historical conversion was calculated on.	Date	
Header	The historical conversion header associated with a historical conversion. **Referenced table**: HistoricalCurrencyHeader	Reference	
Rate	The historical conversion rate.	Quantity	

HistoricalCurrencyHeader table

This table identifies historical currency conversion rate series, which can be used to report currency conversion rates from previous periods.

Table 6.117: HistoricalCurrencyHeader (Core) fields

Field name	Description	Data type	Key
AsOfDate	The date the historical rate series was created.	Date	
BaseKey	A String representation of the AsOfDate.	String	Yes

HistoricalDemandActual table

This table contains demand data related to historical actual customer shipments on a given date. It might also contain other types of historical actual demand data including consumption of inventory by a customer or point of sale data from a customer.

This table supports Demand Planning and Statistical Forecasting in RapidResponse, and is needed for statistical forecasting, forecast, disaggregation, and forecast accuracy. This table should be integrated during the initial RapidResponse deployment.

This table contains vector set data for records in the **HistoricalDemandHeader** table. For more information about vectors, see "Vector data" on page 138.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Date field is not a key field.
- The Header field is a reference, not an owning reference.

You can determine whether this table is using vector data by opening the Data Model dialog box and reviewing table icons. Tables that contain vector data are displayed with the icon.

Table 6.118: HistoricalDemandActual (Mfg) fields

Field name	Description	Data type	Key
ActualReceiptDate	The date on which the customer actually received this historical order. **Note:** This field is in the Solutions namespace.	Date	
ActualShipDate	The date on which this historical order was actually shipped to the customer. **Note:** This field is in the Solutions namespace.	Date	

Table 6.118: HistoricalDemandActual (Mfg) fields

Field name	Description	Data type	Key
AutoCommit HorizonDate	The date from which the Order Fulfillment application would have been able to automatically commit to this historical order. That is, if the historical line item's **RequestDate** had been on or after this date, then a commitment would have been automatically made to the customer request (regardless of supply availability). Otherwise, manual intervention would have been needed in order to commit to the customer request. **Note:** This field is in the Solutions namespace.	Date	
CommitDate	The date the order is committed to ship.	Date	
Customer Changes	This field is used to hold a concatenated string of all **CustomerNotes** values that were added to the **IndependentDemandSolution** record to track customer-initiated changes made against this historical demand. **Note:** This field is in the Solutions namespace.	String	
Customer RequestDate	The date on which the customer originally requested to receive this historical order. **Note:** This field is in the Solutions namespace.	Date	
Date	The date associated with this historical demand. This is the date on which the record would be considered for use in statistical forecasting, forecast disaggregation, and/or inventory optimization analytic calculations. For records associated with categories used in statistical forecasting, this field should be set to the request date. For records belonging to categories not used in statistical forecasting, this date should be set to either the request date or the actual shipment date as appropriate.	Date	Yes
DeliveryRoute	A reference to the delivery route that was used to transport this order to the customer. Related details about the delivery such as pick and pack time, transit time, and the carrier that was used are available from this reference. **Referenced table:** DeliveryRoute (Nullable) **Note:** This field s in the Solutions namespace.	Reference	

Table 6.118: HistoricalDemandActual (Mfg) fields

Field name	Description	Data type	Key
DwellTime	Dwell time for this historical order. This represents extra time, if any, that was built into the customer request and during which no activity took place against the order. Calculated as the number of Everyday intervals between **IndependentDemand.RequestDate** and **AutoCommitHorizonDate** at the time the order was first processed in RapidResponse (and not recalculated if the request date subsequently changed).	Integer	
FrozenHorizon Date	The date before which the Order Fulfillment application would not have been able to automatically commit to the customer request date on this historical customer order. That is, if the order's RequestDate had been or before this date, then the commitment would have been pushed outside of the horizon, and manual intervention required to pull it back in and commit to the customer request date. **Note:** This field is in the Solutions namespace.	Date	
Header	A reference to the historical demand header associated with the record. This identifies the part, customer, and demand category for this actual shipment. **Referenced table**: HistoricalDemandHeader **Reference Syntax**: Header.PartCustomer.Part.Name, Header.PartCustomer.Part.Site, Header.Category	Reference (Owning)	Yes
Line	The order line number associated with this record. This field is in the **Solutions** namespace.	String	
LineCreatedDate	When the line number associated with this record was created. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2014.2.	Date	
LineQuantity	The quantity associated with this record. This field is in the **Solutions** namespace.	Quantity	
NumberOf Customer Changes	The number of customer-initiated changes that were made to this historical order during the order fulfillment process. **Note:** This field is in the Solutions namespace.	Integer	
NumberOf SupplyChanges	The number of supply changes that were made to this historical order during the order fulfillment process. **Note:** This field was is in the Solutions namespace.	Integer	

Table 6.118: HistoricalDemandActual (Mfg) fields

Field name	Description	Data type	Key
Order	A reference to the historical demand order number associated with this record. This field is in the **Solutions** namespace. **Referenced table:** Solutions::HistoricalDemandOrder (Nullable)	Reference	
OrderCreation	The date and time at which the historical line item represented by this record was created within the Order Fulfillment application. **Note:** This field is in the Solutions namespace.	DateTime	
OrderPromised DateTime	The date and time at which the Order Fulfillment application was able to promise the historical order to the customer. **Note:** This field is in the Solutions namespace.	DateTime	
Quantity	The number of parts in this actual shipment.	Quantity	
RequestDate	The date by which this order needed to be completed to stock in order to ensure that the order could subsequently ship in time to meet the customer's requested date.	Date	
Route	The routing associated with this record. This field is in the **Solutions** namespace.	String	
ShipSet	The ship set associated with this record. This field is in the **Solutions** namespace.	String	
SupplyNotes	This field is used to hold a concatenated string of all **SupplyNotes** values that were added to the **IndependentDemandSolution** record to track supply-related changes made against this historical demand. **Note:** This field is in the Solutions namespace.	String	
UnitCost	The unit cost for the historical demand. This value can be used in calculating margins where the margin per unit is calculated as the difference between UnitPrice and UnitCost.	Money	
UnitPrice	The unit price for the historical demand. This value can be used to calculate revenue associated with historical actual demand. The default value for **HistoricalDemandActual.UnitPrice** is 0.	Money	

HistoricalDemandCategory table

This table is used to categorize values in both the HistoricalDemandActual and HistoricalDemandSeries tables. It identifies both different categories of forecasted demand (for example, customer, sales, statistical, and so on) as well as different categories of actual demand (for example, shipments, consumption, point of sale, and so on).

If there is no effective record for a given forecast category and part customer combination in of the **HistoricalDemandHeaderTimePhasedAttributes**, **HistoricalDemandHeaderRollingWeight**, **HistoricalDemandCategoryRollingWeight**, or **HistoricalDemandHeader** tables, then the category weight for the part customer is taken from this table.

This table supports Sales and Operations Planning, Demand Planning, and Statistical Forecasting in RapidResponse, and is configured during the initial RapidResponse deployment. Additional values might be entered in this table to support new streams of forecast data.

Table 6.119: HistoricalDemandCategory (Mfg) fields

Field name	Description	Data type	Key
Consensus Forecast Weight	Used to specify a weight to apply to a given forecast category when generating the consensus forecast (as reported in the **ConsensusForecast** table and used to drive supply planning in RapidResponse). This field is applicable only if **Type.ProcessingRule** is set to "Forecast", and typically contains values between 0 and 1. When generating the consensus forecast for a given part and customer combination, the **ForecastDetail.Quantity** value associated with each forecast category on a given date is multiplied by the value in this field, and the results are summed together. As such, only categories where this field is greater than zero are included in the consensus forecast calculation. For example, if the Statistical and Adjustment forecast categories both had this field set to 1, and all other categories were set to 0, the consensus forecast would be the sum of the Statistical and Adjustment categories. If forecast weights by part customer are required, the ConsensusForecastWeight field on the **HistoricalDemandHeader** table can be used to override this value, and if time-phased forecast weights by part customer are required, the ConsensusForecastWeight field on the **HistoricalDemandHeaderTimePhased Attributes** table can be used to override both of these values. Default value: 0 For more information about how the consensus forecast is calculated, see "ConsensusForecast table" on page 966.	Quantity	
CriticalLimit	Indicates the critical level for metric values in this historical demand category expressed as a percentage of their target values. The interpretation of the value provided here depends on the setting in the **DesiredResults** field. If set to "Higher", then the critical limit is reached when below target by the percentage specified here. But if set to "Lower", then the critical limit is reached when above target by the percentage specified here For example, if a value of "20" is specified and **DesiredResults** is set to "Higher", then the critical limit is reached when the category is below target by 20%. This field is maintained in the **S&OP Annual Plan**, **S&OP Constraint Annual Plan**, **SCM Targets**, and **SCM Constraint Targets** workbooks in RapidResponse. Other workbooks in RapidResponse then use these limits to determine when metric values should display with critical conditional formatting (typically, red). **Note:** This field is applicable when **Type.ProcessingRule** is set to "Target".	Quantity	

Table 6.119: HistoricalDemandCategory (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this historical demand category.	String	
DesiredResults	This setting is used to indicate whether a high or low metric value, relative to the category's target, is considered desirable. It also determines how the **CriticalLimit** and **WarningLimit** values are applied against the target when determining if the critical and warning limits have been reached. Valid values are: • **High**—Metric values above their target are considered desirable (for example, revenue values). Thus critical and warning limits are triggered when metric values are below target. • **Low**—Metric values below their target are considered desirable for this category (for example, inventory levels). Thus critical and warning limits are triggered when metric values are above target. **Default Value:** High **Note:** This field is applicable when **Type.ProcessingRule** is set to "Target".	String (Enum)	
Type	The type of demand data this category contains. **Referenced table:** HistoricalDemandCategoryType	Reference	
Value	The name of this historical demand category.	String	Yes
WarningLimit	Indicates the warning level for metric values in this historical demand category expressed as a percentage of their target values. The interpretation of the value provided here depends on the setting in the **DesiredResults** field. If set to "Higher", then the warning limit is reached when below target by the percentage specified here. But if set to "Lower", then the warning limit is reached when above target by the percentage specified here For example, if a value of "5" is specified and **DesiredResults** is set to "Lower", then the warning limit is reached when the category is above target by 5%. This field is maintained in the **S&OP Annual Plan**, **S&OP Constraint Annual Plan**, **SCM Targets**, and **SCM Constraint Targets** workbooks in RapidResponse. Other workbooks in RapidResponse then use these limits to determine when metric values should display with warning conditional formatting (typically, yellow). **Note:** This field is applicable when **Type.ProcessingRule** is set to "Target".	Quantity	

HistoricalDemandCategory table – calculated and set fields

The following describes the calculated and set fields in the **HistoricalDemandCategory** table. For an introduction to this table, and descriptions of its input fields, see "HistoricalDemandCategory table" on page 329.

Table 6.120: Calculated HistoricalDemandCategory (Mfg) fields

Field name	Description	Data type	Key
Forecast Disaggregation ParametersActual Categories	Set of **ForecastDisaggregationParameter** records that reference this category as the actual category to use when calculating disaggregation parameters.	Set	
Forecast Disaggregation ParametersBy ActualCategory	Set of **ForecastDisaggregationParametersBy Category** records that reference this category as the actual category to use when calculating disaggregation parameters.	Set	
Forecast Disaggregation ParametersBy Category	Set of **ForecastDisaggregationParametersBy Category** records that reference this category as the category for which disaggregation parameters are being defined.	Set	
ForecastItem Parameters Actuals Categories	Set of **ForecastItemParameters** records that reference this category as the historical actual demand category.	Set	
ForecastItem Parameters Forecast Categories	Set of **ForecastItemParameters** records that reference this category as the historical actual demand category.	Set	
ForecastSafety StockItem Mappings	Set of **SafetyStockItemMapping** records that use this record for the historical forecast category.	Set	
ForecastSafety StockItems	Set of **SafetyStockItem** records that use this record for the historical forecast category.	Set	
Historical DemandHeaders	Set of **HistoricalDemandHeader** records that reference this record.	Set	

Table 6.120: Calculated HistoricalDemandCategory (Mfg) fields

Field name	Description	Data type	Key
RollingWeight	Set of **HistoricalDemandCategoryRollingWeight** records associated with this demand category. **Note:** This field was added in Version 2016.2, Service Update 12.	Set	
SafetyStockItem Mappings	Set of **SafetyStockItemMapping** records that use this record for the historical demand category.	Set	
SafetyStockItems	Set of **SafetyStockItemMapping** records that use this record for the historical demand category.	Set	
SOPActuals Category	Set of **SOPAnalyticsConfiguration** records that use this demand category to calculate disaggregation rates.	Set	
SOPForecast Disaggregation OverrideCategories	Set of **SOPAnalyticsConfiguration** records that use this demand category to override the default category used to calculate disaggregation rates.	Set	

HistoricalDemandCategoryRollingWeight table

The **HistoricalDemandCategoryRollingWeight** table stores forecast weight records for forecast categories when using rolling forecast weights to create the consensus demand plan. Each record in this table applies to a forecast category and indicates the weight given to that forecast category for a specific horizon. If a header has existing **HistoricalDemandHeaderTimephasedAttributes**, **HistoricalDemandHeaderRollingWeight**, or **HistoricalDemandHeader** records, this table is ignored. This table is also ignored if **CalcForecastStartDate** or **CycleCalendar** are undefined in the **SOPAnalyticsConfiguration** table.

The **HistoricalDemandCategoryRollingWeight** table supports the Demand Planning application in RapidResponse. Values in this table are maintained using the both the Control Tables and S&OP Demand Planning Ratios workbooks in RapidResponse. For more information about the Demand Planning application, see the RapidResponse Applications Guide.

This table was added in Version 2016.2, Service Update 12.

Table 6.121: HistoricalDemandCategoryRollingWeight (Mfg) fields

Field name	Description	Data type	Key
Category	A reference to the demand category that is associated with this **ConsensusForecastWeight**. **Referenced table**: HistoricalDemandCategory	Reference	Y
Horizon	A reference to the horizon that is set for this **ConsensusForecastWeight** for this demand category. **Referenced table**: ConsensusForecastRollingHorizon	Reference	Y
ConsensusForecast Weight	Specifies a weight as a percentage to apply to a given forecast category when generating the consensus forecast (as reported in the **ConsensusForecast** table and used to drive demand planning in RapidResponse). This field should typically contain values between 0 and 100. When generating the consensus forecast for a given part and customer combination, the **ForecastDetail.Quantity** value associated with each forecast category on a given date is multiplied by the value in this field, and the results are summed together. For example, if the Marketing and Finance forecast categories both had this field set to 20%, and all other categories were set to 0, the consensus forecast would be the sum of twenty percent of each of the marketing and finance forecasts. Within the effective horizon, a value provided in this field will override the values provided for the category in the **ConsensusForecastWeight** field in either of the **HistoricalDemandHeader** or **HistoricalDemandHeaderCategory** tables. **Default value**: -1 **Note:** For more information about how the consensus forecast is calculated, see ConsensusForecast table.	Quantity	

HistoricalDemandHeader table

This table identifies each unique part, customer, and historical demand category combination, and can be used to specify the weights associated with forecast categories when generating the consensus forecast for a particular part customer. Typically, it is referenced by multiple groups of historical demand actuals, forecasts, and other items.

If the **HistoricalDemandHeader** table contains a record for a given forecast category and part customer combination, and there is no effective record for that combination in either the **HistoricalDemandHeaderTimePhasedAttributes** table or the **HistoricalDemandHeaderRollingWeight** table, then the category weight is taken from this table.

This table supports Sales and Operations Planning, Demand Planning, and Statistical Forecasting in RapidResponse.

Table 6.122: HistoricalDemandHeader (Mfg) fields

Field name	Description	Data type	Key
Category	A reference to the demand category associated with this group of historical demands. **Referenced table**: HistoricalDemandCategory **Reference Syntax**: HistoricalDemandCategory	Reference	Yes
Consensus Forecast Weight	Specifies a weight to apply to the referenced forecast category for this part and customer combination when generating the consensus forecast (as reported in the **ConsensusForecast** table and used to drive supply planning in RapidResponse). The weight should be a value between 0 and 1. When generating the consensus forecast for a given part and customer combination, the **ForecastDetail.Quantity** value associated with each forecast category on a given date is multiplied by the value in this field, and the results are summed together. As such, only categories where this field is greater than zero are included in the consensus forecast calculation. For example, if the Marketing and Statistical forecast categories both had this field set to .5, and all other categories were set to 0, the consensus forecast would be the sum of fifty percent of each of the marketing and statistical forecasts. If time-phased weights per forecast category and part customer combination are required, the **HistoricalDemandHeaderTimePhasedAttributes** table can be used instead, and values in its ConsensusForecastWeight field override those in this table (within their effective date ranges). If consensus forecast weights are not provided for a given forecast category and part customer combination in either of these tables, then the ConsensusForecastWeight field value from the **HistoricalDemandHeaderCategory** table is used instead. **Default value:** -1 (indicates to ignore this field and use HistoricalDemandHeader.Category.ConsensusForecastWeight field value instead).	Quantity	
PartCustomer	The part and customer combination associated with this group of historical demands. **Referenced table**: PartCustomer **Reference Syntax**: PartCustomer.Part.Name, PartCustomer.Part.Site, PartCustomer.Customer	Reference	Yes

HistoricalDemandHeader table – calculated and set fields

The following describes the calculated and set fields in the **HistoricalDemandHeader** table. For an introduction to this table, and descriptions of its input fields, see "HistoricalDemandHeader table" on page 334.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Actuals field is a set, not a vector set.
- The ForecastDetails field is a set, not a vector set.

Table 6.123: Calculated HistoricalDemandHeader (Mfg) fields

Field name	Description	Data type	Key
Actuals	The **HistoricalDemandActual** records associated with this historical demand header.	Vector Set	
Consensus Forecast WeightByHeader	The **ConsensusForecastWeightByHeader** records associated with this historical demand header.	Set	
ForecastDetails	The **ForecastDetail** records associated with this historical demand header.	Vector Set	

HistoricalDemandHeaderRollingWeight table

The **HistoricalDemandHeaderRollingWeight** table stores the weight that is applied to a header for a specific duration of the horizon when using rolling forecast weights to create the consensus demand plan. If a header has existing **HistoricalDemandHeaderTimephasedAttributes** records, this table is ignored. This table is also ignored if **CalcForecastStartDate** or **CycleCalendar** are undefined in the **SOPAnalyticsConfiguration** table.

The **HistoricalDemandHeaderRollingWeight** table supports the Demand Planning application in RapidResponse. Values in this table are maintained using the both the **Control Tables** and **S&OP Demand Planning Ratios** workbooks in RapidResponse. In addition, the value of the **ForecastStartOffset** field in the **SOPAnalyticsConfiguration** table impacts how the remainder horizon period is calculated in the **Rolling Weight Summary** worksheet (**S&OP Demand Planning Ratios** workbook). If the value is >0, the remainder period starts at month 0 (and then continues after all other defined rolling horizons have ended). If the value = 0, the remainder period starts only after all other defined horizons have ended. For more information about the Demand Planning application, see the RapidResponse Applications Guide.

This table was added in Version 2016.2, Service Update 12.

Table 6.124: HistoricalDemandHeaderRollingWeight (Mfg) fields

Field name	Description	Data type	Key
Header	A reference to the header to which this rolling weight record applies. **Referenced table**: HistoricalDemandHeader **Referenced syntax**: Header.PartCustomer.Part.Name, Header.PartCustomer.Part.Site, Header.PartCustomer.Customer, HeaderCategory	Reference	Y
Horizon	A reference to the horizon that is set for this **ConsensusForecastWeight** for this demand category. **Referenced table**: ConsensusForecastRollingHorizon	Reference	Y
ConsensusForecastWeight	Specifies a weight as a percentage to apply to a given header when generating the consensus forecast (as reported in the **ConsensusForecast** table and used to drive demand planning in RapidResponse). This field should typically contain values between 0 and 100. Within the effective horizon, a value provided in this field will override the values provided for the category in the **ConsensusForecastWeight** field in either of the **HistoricalDemandHeader** or **HistoricalDemandHeaderCategory** tables. **Default value**: -1 **Note:** For more information about how the consensus forecast is calculated, see ConsensusForecast table.	Quantity	

HistoricalSupplyOrder table

This table stores historical supply orders, and is used for reporting historical supply.

This table was added in the Version 2016.2, Service Update 5, to support the Distribution Requirements Planning application.

Table 6.125: HistoricalSupplyOrder (Solutions) fields

Field name	Description	Data type	Key
CreatedDate	The date the supply order was created.	Date	
Id	A string identifier for the historical supply order.	String	Yes
Site	The site associated with the supply order. **Referenced table**: Site	Reference	Yes
Type	A type used to categorize this supply order.	String	Yes

HistoricalDemandHeaderTimePhased Attributes table

This **HistoricalDemandHeaderTimePhasedAttributes** table can be used if time-phased attributes are to be used when generating the consensus forecast during the sales and operations planning process in RapidResponse. Each record in this table applies to a specific part, customer, and forecast category combination, and indicates the weight given to that forecast category when generating the consensus forecast within the effective date range. If time-phased forecast weights are not required, this table can be ignored and the weight used for a given forecast category will be taken from the value in either the **HistoricalDemandHeader.ConsensusForecastWeight** field (if populated) or else the value in the **HistoricalDemandHeader.Category.ConsensusForecastWeight** field.

Table 6.126: HistoricalDemandHeaderTimePhasedAttributes (Mfg) fields

Field name	Description	Data type	Key
Consensus Forecast Weight	Specifies a time-phased weight to apply to a given forecast category for a part and customer combination when generating the consensus forecast (as reported in the **ConsensusForecast** table and used to drive supply planning in RapidResponse). This field should typically contain values between 0 and 1. When generating the consensus forecast for a given part and customer combination, the **ForecastDetail.Quantity** value associated with each forecast category on a given date is multiplied by the value in this field, and the results are summed together. For example, if the Marketing and Finance forecast categories both had this field set to 0.5, and all other categories were set to 0, the consensus forecast would be the sum of fifty-percent of each of the marketing and finance forecasts. Within the effective date range, a value provided in this field will override the values provided for the category in the **ConsensusForecastWeight** field in either of the **HistoricalDemandHeader** or **HistoricalDemandHeaderCategory** tables. Default value: 0 **Note:** For more information about how the consensus forecast is calculated, see "ConsensusForecast table" on page 966.	Quantity	
EffectiveInDate	The date on which this attribute record becomes effective.	Date	
EffectiveOutDate	The date on which this attribute record is no longer effective.	Date	

Table 6.126: HistoricalDemandHeaderTimePhasedAttributes (Mfg) fields

Field name	Description	Data type	Key
Header	A reference to the part, customer, and category this attribute record applies to. Records in this table are only applicable when **Header.Category.Type.ProcessingRule** is set to "Forecast". **Referenced table:** HistoricalDemandHeader **Reference Syntax:** Header.PartCustomer.Part.Name, Header.PartCustomer.Part.Site, Header.PartCustomer.Customer, Header.Category	Reference	Yes
Id	A unique identifier for this record. Note that if there are multiple records with the same EffectiveInDate for a given part, customer, and category, the one with the largest ID will take precedence.	String	Yes

HistoricalDemandOrder table

This table stores historical demand orders, and is used for reporting historical demands.

Table 6.127: HistoricalDemandOrder (Solutions) fields

Field name	Description	Data type	Key
CreatedDate	The date the demand order was created.	Date	
Id	Identifies the historical demand order.	String	Yes
Site	The site where the demand order was created. **Referenced table:** Site	Reference	Yes

HistoricalDemandOrder table – calculated fields

The following describes the calculated fields in the HistoricalDemandOrder table. For an introduction to this table, see "HistoricalDemandOrder table" on page 339.

Table 6.128: Calculated HistoricalDemandOrder (Solutions) fields

Field name	Description	Data type	Key
HistoricalDemand Actuals	The set of **HistoricalDemandActual** records that refer to this demand order.	Set	

HistoricalDemandSeries table

This table contains one record for each HistoricalDemandHeader and particular AsOfDate (the date when the demand records were generated) and Sequence (in cases where multiple series are generated on the same date). Each historical demand series entry corresponds to a collection of points that make up one set of historical demands (for a given part, customer, category, and as-of date).

This table supports Sales and Operations Planning in RapidResponse. It can be integrated during the initial RapidResponse deployment, and additional records are created as necessary through data change alerts in RapidResponse.

Table 6.129: HistoricalDemandSeries (Mfg) fields

Field name	Description	Data type	Key
AsOfDate	The date on which this historical demand series was generated.	Date	
Id	An identifier used to ensure uniqueness of this Header, AsOfDate, and Sequence combination.	String	Yes
Header	A reference to the part, customer, and demand category combination associated with this record. **Referenced table**: HistoricalDemandHeader **Reference Syntax**: Header.PartCustomer.Part.Name, Header.PartCustomer.Part.Site, Header.PartCustomer.Customer, Header.Category	Reference	Yes
Sequence	In cases where there is more than one series of historical demands for a part, customer, and demand category combination on a given AsOfDate, this field is used to identify each series. The first series on a given AsOfDate should have the lowest Sequence value, and the last series on a given AsOfDate should have the highest Sequence value.	Integer	

HistoricalDemandSeries table – calculated and set fields

The following describes the calculated and set fields in the HistoricalDemandSeries table. For an introduction to this table, see "HistoricalDemandSeries table" on page 340.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Details field is a set, not a vector set.

Table 6.130: HistoricalDemandSeries calculated (Mfg) fields

Field name	Description	Data type	Key
Details	The set of **HistoricalDemandSeriesDetail** records associated with this historical demand series header.	Vector Set	
FirstOnAsOfDate	Indicates if this HistoricalDemandSeries is the first one for a given Header and AsOfDate (has the lowest Sequence value). This field can be used in reports to allow filtering that shows only the first series for each AsOfDate. It is always set to 'Y' (yes) if a given Header and AsOfDate has only one series. Valid values are: • Y • N	Boolean	
IsBaseline	Indicates if this HistoricalDemandSeries is a baseline series for the associated part customer. A baseline series is one where the LastOnAsOfDate = 'Y' and the AsOfDate belongs to Header.PartCustomer.ToleranceProfile.BaselineCalendar. Valid values are: • Y • N	Boolean	
LastOnAsOfDate	Indicates if this HistoricalDemandSeries is the last one for a given Header and AsOfDate (has the highest Sequence value). This field can be used in reports to allow filtering that shows only the last series for each AsOfDate. It is always set to 'Y' (yes) if a given Header and AsOfDate has only one series. Valid values are: • Y • N	Boolean	
Number	Numbers each historical demand series from 0 to N, where the most recent series has a value of 0 and the oldest series has a value of N. This field can be used in reports to compare points in the latest historical demand series to points in the previous historical demand series.	Integer	

HistoricalDemandSeriesDetail table

This table contains an entry for each unique quantity and date interval combination in a historical demand series. It contains one entry for each non-zero point in a historical demand series.

Note: If you are entering past due amounts in the **HistoricalDemandSeriesDetail** table, it is important to insert all past due amounts into the current bucket. This will ensure that the data is correctly calculated by the **HistoricalDemandWaterfall** table.

This table supports Sales and Operations Planning in RapidResponse. It can be integrated during the initial RapidResponse deployment, and additional values from the ForecastDetail table are added to this table by the Insert Demand Planning History alerts in RapidResponse.

This table contains vector set data for records in the **HistoricalDemandSeries** table. For more information about vectors, see "Vector data" on page 138.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The BeginDate field is not a key field.
- The EndDate field is not a key field.
- The Series field is a reference, not an owning reference.

You can determine whether this table is using vector data by opening the Data Model dialog box and reviewing table icons. Tables that contain vector data are displayed with the icon.

Table 6.131: HistoricalDemandSeriesDetail (Mfg) fields

Field name	Description	Data type	Key
BeginDate	The begin date of the interval to which this historical demand applies.	Date	Yes
EndDate	The end date of the interval to which this historical demand applies. This value should be greater than or equal to BeginDate. In delta calculations for the **HistoricalDemandWaterfall** table, EndDate is treated inclusively (that is, the interval covered by this historical demand includes this EndDate.)	Date	Yes
Quantity	The historical demand quantity for this interval. The value in this field is used if **HistoricalDemandCategory.UnitType = 'Quantity'**. For more information, see "HistoricalDemandCategory table" on page 329.	Quantity	
Series	A reference to the historical demand series that a given data point belongs to. **Referenced table:** HistoricalDemandSeries **Reference Syntax:** Series.Header.PartCustomer.Part.Name, Series.Header.PartCustomer.Part.Site, Series.Header.PartCustomer.Customer, Series.Header.Category, Series.Id	Reference (Owning)	Yes
UnitPrice	The unit price associated with this historical demand. This field is useful for calculating the revenue associated with historical demands.	Money	

Table 6.131: HistoricalDemandSeriesDetail (Mfg) fields

Field name	Description	Data type	Key
Value	The monetary value associated with this historical demand. The value in this field is used if **HistoricalDemandCategory.UnitType = 'Value'**. For more information, see "HistoricalDemandCategory table" on page 329.	Money	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalPartKPI table

The **HistoricalPartKPI** table contains part-based historical key performance indicator data. It supports RapidResponse applications that show inventory trends over time.

Data is not directly imported into this table. Instead it is populated by automatic data modifications (workbook commands) that archive results. To ensure unit of measure conversions work properly, when entering values for a key performance indicator that measures a quantity, you should typically ensure that the **KPI.EnableUOM** is set to "Yes" in the automatic data modification.

Notes:

- Data from this table was formerly used by the Risk/Opportunity Assessment workbook to display graphs. This table was populated by automatic data modifications that archived results associated with different key performance indicators in the Risk/Opportunity Assessment workbook. Data was not be directly imported into this table. In RapidResponse 11.0, the Risk/Opportunity Assessment workbook was modified to remove support for historical KPI information. If your company is using the RapidResponse 10.1 (or earlier) version of this workbook then this table continues to use the key performance indicators listed in the following table.
- For more information about historical tables, see "Historical tables" on page 134.

Table 6.132: HistoricalPartKPI (Mfg) fields

Field name	Description	Data type	Key
Date	The date associated with this key performance indicator value.	Date	
KPI	The key performance indicator with a historical data value saved in this record. Each record should reference one of the following KPI values: • OnHandNetttable • OnHandNonNettable • OnTimeHistory • OnTimeHistoryRequest • ServiceLevelTarget • TotalDemand • TotalHistory • TotalSupply **Referenced table:** KPI **Reference Syntax**: KPI.Value	Reference	Yes
Part	The part that this key performance indicator value applies to. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes
Quantity	The value associated with this key performance indicator for the given part and date combination.	Quantity	

The following KPIs were used by the Risk/Opportunity Assessment workbook.

- Excess$
- NumberAllocationIntegrity
- NumberBOMIntegrity
- NumberCancels
- NumberExpedites
- NumberIDIntegrity
- NumberNewPlannedOrders
- NumberPartMasterIntegrity
- NumberPartSourceIntegrity
- NumberPushes
- NumberSRIntegrity
- Obsolete$
- OrdersAtRiskQuantity
- OrdersAtRisk$
- PurchaseAvoidanceQty
- PurchaseAvoidance$

- RevenueOpportunitiesQty
- RevenueOpportunities

HistoricalReceipt table

This table contains historical receipt information relating to yields used by the Self-Healing Supply Chain.

Table 6.133: HistoricalReceipt (Mfg) fields

Field name	Description	Data type	Key
Header	A reference to the "HistoricalReceiptHeader table" on page 345.	Reference	Yes
LineReceivedDate	The actual date of receipt.	Date	Yes
Id	The ID value. This may be a concatenated string based on key fields.	String	Yes
StartQuantity	The supply quantity before yield adjustment. Values less than 0 are considered invalid.	Money	
ReceivedQuantity	The supply quantity after yield adjustment. Values less than 0 are considered invalid.	Quantity	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalReceipt table – calculated and set fields

The following describes the calculated and set fields in the **HistoricalReceipt** table.

Table 6.134: Calculated HistoricalReceipt (Mfg) fields

Field name	Description	Data type	Key
CalcYieldRatio	Calculated field. Reports the ratio: ReceivedQuantity / StartQuantity. If input is invalid, this field reports -1.	Quantity	

HistoricalReceiptHeader table

This table identifies each unique part, supplier, and historical supply category combination. Each of the entries in this table are typically associated with multiple historical supply series or historical receipt records.

Table 6.135: HistoricalReceiptHeader (Mfg) fields

Field name	Description	Data type	Key
BOMAlternate	Reference to BOMAlternate, to support different product versions.	Reference	Yes
Category	A reference to the historical supply category associated with this group of historical supplies. **Referenced table**: HistoricalSupplyCategory **Reference Syntax**: HistoricalSupplyCategory	Reference	Yes
PartSupplier	The part and supplier combination associated with this group of historical supplies. **Referenced table**: PartSupplier **Reference Syntax**: Part.Name, PartSupplier.Part.Site, PartSupplier.Supplier	Reference	Yes
Routing	Reference to Routing, to support different product versions.	Reference	Yes
Source	Nullable reference to Source.	Reference	Yes

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalShipment table

This table stores historical shipments, and is used for reporting historical shipping data for the Distribution Requirements Planning application.

This table was added in the Version 2016.2, Service Update 5, to support the Distribution Requirements Planning application.

Table 6.136: HistoricalShipment (Solutions) fields

Field name	Description	Data type	Key
CreatedDate	The date the shipment was created.	Date	
Id	A string identifier for this historical shipment.	String	Yes
Site	The site associated with the shipment. **Referenced table**: Site	Reference	Yes

HistoricalSupplyActual table

This table contains supply data related to historical actual orders placed against suppliers on a given date.

This table contains vector set data for records in the **HistoricalSupplyHeader** table. For more information about vectors, see "Vector data" on page 138.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Date field is not a key field.
- The Header field is a reference, not a key reference.

You can determine whether this table is using vector data by opening the Data Model dialog box and reviewing table icons. Tables that contain vector data are displayed with the icon.

Table 6.137: HistoricalSupplyActual (Mfg) fields

Field name	Description	Data type	Key
ActualDockDate	The actual date this supply arrived at the receiving dock, which is used in historical reporting. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2016.2, Service Update 5	Date	
ActualShipDate	The actual date this supply was shipped from the supplier, which is used in historical reporting. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2016.2, Service Update 5.	Date	
Date	The date associated with this actual shipment. The date used should be consistent with the historical supply series dates. For example, if the historical supply series dates represent the dates that parts ship from the dock, then this date should be a dock departure date. **Note:** If set to either Undefined, Past, or Future, then this record is ignored by safety stock calculations.	Date	Yes
HandlingCost	The cost associated with handling the shipment. Negative values are treated as zero. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2016.2, Service Update 5.	Money	

Table 6.137: HistoricalSupplyActual (Mfg) fields

Field name	Description	Data type	Key
Header	A reference to the historical supply header associated with the record. This identifies the part, supplier, and supply category for this actual order. **Referenced table**: HistoricalSupplyHeader **Reference Syntax**: Header.PartSupplier.Part.Name, Header.PartSupplier.Part.Site, Header.Category	Reference (Owning)	Yes
LeadTime	The known lead time associated with this historical supply. Typically when both the **OrderDate** and **Date** fields are defined, the historical lead time value used in safety stock calculations where supply variability is assumed is calculated based on those two fields. However, if the **OrderDate** field is not provided, then the historical lead time value used in safety stock calculations is set to the value provided in this field. This value should be expressed in units of the **HistoricalSupplyHeader.LeadTimeCalendar** (otherwise, if that calendar is not defined, an "Everyday" calendar is assumed).	Integer	
LineDueDate	The date that the ScheduledReceipt (or purchase order) is expected from the supplier. For orders with multiple lines, this may be different dates per line. **Note:** This field was added in Version 2016.2, Service Update 26.	Date	
Order	A reference to the historical supply order number associated with this record. This field is in the **Solutions** namespace. **Referenced table:** HistoricalSupplyOrder **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	
OrderDate	The date associated with the ordering of this supply. For parts configured as either single or multi-echelon safety stock items and where supply lead time variability is to be considered, the difference between the Date field and this value is typically used to determine historical supply lead times. If this value is undefined, or set to either 'past' or 'future', then the value in the **LeadTime** field is used during safety stock item calculations.	Date	

Table 6.137: HistoricalSupplyActual (Mfg) fields

Field name	Description	Data type	Key
OrderDueDate	Indicates the DueDate associated with a historical supply order. This field should be populated if using the MPS application, and is required by certain resources that report details such as schedule attainment. **Note:** This field is in the **Solutions** namespace.	Date	
Quantity	The number of units in this actual order.	Quantity	
ReceiptQuantity	The total amount that was received on this shipment record. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2016.2, Service Update 5.	Quantity	
Shipment	A reference to the historical shipment associated with this record. This field is in the **Solutions** namespace. **Referenced table:** HistoricalShipment **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	
ShippingCost	The cost associated with shipping the shipment. Negative values are treated as zero. This field is in the **Solutions** namespace. **Note:** This field was added in Version 2016.2, Service Update 5.	Money	
Source	A reference to the source that provided the part this historical supply record pertains to. This reference should typically be populated for parts belonging to multi-echelon families and for which variable lead time is enabled based on historical supply actuals. If a part has multiple sources of supply, this reference can help ensure the correct one is identified. **Referenced table:** Source (Nullable)	Reference	

Table 6.137: HistoricalSupplyActual (Mfg) fields

Field name	Description	Data type	Key
StatisticalDate	The date that the buyer uses to track the agreed-upon delivery date from the supplier. In the case where the buyer has asked to delay a shipment, then the StatisticalDate will be adjusted to match the updated LineDueDate. In the case where the supplier has identified that they will deliver late, then the StatisticalDate remains unchanged, but the LineDueDate will be adjusted to the new expected delivery date. **Note:** This field was added in Version 2016.2, Service Update 26.	Date	
UnitPrice	The unit price for the historical supply. This value can be used to calculate the cost associated with historical actual supplies. The default value for **HistoricalSupplyActual.UnitPrice** is 0.	Money	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalSupplyCategory table

This table identifies both different categories of historical supply series and historical supply actuals. It is used to categorize values in both the **HistoricalSupplyActual** and **HistoricalSupplySeries** tables.

Table 6.138: HistoricalSupplyCategory (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this historical supply category.	String	
Value	The name of this historical supply category.	String	Yes

HistoricalSupplyCategory table – calculated and set fields

The following describes the calculated and set fields in the **HistoricalSupplyCategory** table.

Table 6.139: HistoricalSupplyCategory calculated (Mfg) fields

Field name	Description	Data type	Key
HistoricalSupplyHeaders	A set of calculated **HistoricalSupplyHeader** records associated with this **HistoricalSupplyCategory**. **Referenced table:** "HistoricalSupplyHeader table" on page 351	Set	
ItemSourceHistoricalSupplyCategorys	A set of calculated **ItemSourceHistoricalSupplyCategory** records associated with this **HistoricalSupplyCategory**. **Referenced table:** "ItemSourceHistoricalSupplyCategory table" on page 389	Set	
MEIOLeadTimeTypes	A set of calculated **MEIOLeadTimeType** records associated with this **HistoricalSupplyCategory**. **Referenced table:** "MEIOLeadTimeType table" on page 744	Set	
SafetyStockItemMappings	A set of calculated **SafetyStockItemMapping** records associated with this **HistoricalSupplyCategory**. **Referenced table:** "SafetyStockItemMapping table" on page 554	Set	
SafetyStockItems	A set of calculated **SafetyStockItem** records associated with this **HistoricalSupplyCategory**. **Referenced table:** "SafetyStockItem table" on page 541	Set	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalSupplyHeader table

This table identifies each unique part, supplier, and historical supply category combination. Each of the entries in this table are typically associated with multiple historical supply series or historical supply actual records.

Table 6.140: HistoricalSupplyHeader (Mfg) fields

Field name	Description	Data type	Key
Category	A reference to the historical supply category associated with this group of historical supplies. **Referenced table**: HistoricalSupplyCategory **Reference Syntax**: HistoricalSupplyCategory	Reference	Yes
LeadTime Calendar	The calendar used to express known lead times associated with historical supply actuals under this header. That is, if **HistoricalSupplyActual.LeadTime** is populated, it is assumed to be in intervals of this calendar. The **LeadTime** field is used in safety stock calculations that assume historical lead time variability, but where the **OrderDate** is not provided and therefore historical lead time cannot be calculated. If this reference is left Null, then an "Everyday" calendar is assumed. **Referenced table**: Calendar (Nullable)	Reference	
PartSupplier	The part and supplier combination associated with this group of historical supplies. **Referenced table**: PartSupplier **Reference Syntax**: Part.Name, PartSupplier.Part.Site, PartSupplier.Supplier	Reference	Yes

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalSupplyHeader table – calculated and set fields

The following describes the calculated and set fields in the **HistoricalSupplyHeader** table. For an introduction to this table, and descriptions of its input fields, see "HistoricalSupplyHeader table" on page 351.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Actuals field is a set, not a vector set.

Table 6.141: Calculated HistoricalSupplyHeader (Mfg) fields

Field name	Description	Data type	Key
Actuals	The **HistoricalSupplyActual** records associated with this historical supply header.	Vector Set	

HistoricalSupplySeries table

This table contains one record for each HistoricalSupplyHeader and particular AsOfDate (the date when the supply records were generated) and Sequence (in cases where multiple series were generated on the same date). Each historical supply series entry corresponds to a collection of points that make up one set of historical supplies (for a given part, supplier, category, and as-of date).

Table 6.142: HistoricalSupplySeries (Mfg) fields

Field name	Description	Data type	Key
AsOfDate	The date on which this historical supplier series was generated.	Date	
Id	An identifier used to ensure uniqueness of this Header, AsOfDate, and Sequence combination.	String	Yes
Header	A reference to the part, supplier, and supply category combination associated with this record. **Referenced table**: HistoricalSupplyHeader **Reference Syntax**: Header.PartSupplier.Part.Name, Header.PartSupplier.Part.Site, Header.PartSupplier.Supplier, Header.Category	Reference	
Sequence	In cases where there is more than one series of historical supplies for a part, supplier, and supply category combination on a given AsOfDate, this field is used to identify each series. The first series on a given AsOfDate should have the lowest Sequence value, and the last series on a given AsOfDate should have the highest Sequence value.	Integer	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalSupplySeries table – calculated and set fields

The following describes the calculated fields in the HistoricalSupplySeries table. For an introduction to this table, see "HistoricalSupplySeries table" on page 353.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The Details field is a set, not a vector set.

Table 6.143: HistoricalSupplySeries (Mfg) calculated fields

Field name	Description	Data type	Key
FirstOnAsOfDate	Indicates if this HistoricalSupplySeries is the first one for a given Header and AsOfDate (has the lowest Sequence value). This field can be used in reports to allow filtering that shows only the first series for each AsOfDate. It is always set to 'Y' (yes) if a given Header and AsOfDate has only one series. Valid values are: • Y • N	Boolean	
IsBaseline	Indicates if this HistoricalSupplySeries is a baseline series for the associated part supplier. A baseline series is one where the LastOnAsOfDate = 'Y' and the AsOfDate belongs to Header.PartSupplier.ToleranceProfile.BaselineCalendar. Valid values are: • Y • N	Boolean	
LastOnAsOfDate	Indicates if this HistoricalSupplySeries is the last one for a given Header and AsOfDate (has the highest Sequence value). This field can be used in reports to allow filtering that shows only the last series for each AsOfDate. It is always set to 'Y' (yes) if a given Header and AsOfDate has only one series. Valid values are: • Y • N	Boolean	
Number	Numbers each historical supply series from 0 to N, where the most recent series has a value of 0 and the oldest series has a value of N. This field can be used in reports to compare points in the latest historical supply series to points in the previous historical supply series.	Integer	
Details	The set of HistoricalSupplySeriesDetail records associated with this historical supply series.	Vector Set	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HistoricalSupplySeriesDetail table

This table contains an entry for each unique quantity and date interval combination in a historical supply series. It contains one entry for each non-zero point in a historical supply series.

This table contains vector set data for records in the **HistoricalSupplySeries** table. For more information about vectors, see "Vector data" on page 138.

If you have upgraded from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later) and your company has not converted the five standard tables to contain vector data, the following differences exist in this table:

- The BeginDate field is not a key field.
- The EndDate field is not a key field.
- The Series field is a reference, not an owning reference.

You can determine whether this table is using vector data by opening the Data Model dialog box and reviewing table icons. Tables that contain vector data are displayed with the icon.

Table 6.144: HistoricalSupplySeriesDetail (Mfg) fields

Field name	Description	Data type	Key
BeginDate	The begin date of the interval to which this historical supply applies.	Date	Yes
EndDate	The end date of the interval to which this historical supply applies. This value should be greater than or equal to BeginDate. In delta calculations for the **HistoricalSupplyWaterfall** table, EndDate is treated inclusively (that is, the interval covered by this historical demand includes this EndDate.)	Date	Yes
Quantity	The historical supply quantity for this interval.	Quantity	
Series	A reference to the historical supply series that a given data point belongs to. **Referenced table:** HistoricalSupplySeries **Reference Syntax:** Series.Header.PartSupplier.Part.Name, Series.Header.PartSupplier.Part.Site, Series.Header.PartSupplier.Supplier, Series.Header.Category, Series.Id	Reference (Owning)	Yes
UnitPrice	The unit price associated with this historical supply. This field is useful for calculating the cost associated with historical supplies.	Money	

Note: For an introduction to historical tables, see "Historical tables" on page 134.

HoldCode table

The **HoldCode** table contains hold codes assigned to customers and/or individual line items for the purpose of preventing orders from moving beyond a particular stage in the order fulfillment process. For example, a hold code might be created for a customer's credit issues and their orders not allowed to ship until the issue is resolved.

This table supports the Order Fulfillment application in RapidResponse.

Note: If a line item (from an **IndependentDemandSolution** record) and the customer on the order it belongs to (**DemandOrder.Customer** reference) both reference a valid **HoldCode** value, the code on the line item takes precedence. For example, if a customer is on credit hold and references a **HoldCode** that does not allow orders to ship, this can be overridden on an individual line item by referencing a **HoldCode** that does allow the order to ship.

Table 6.145: HoldCode (Solutions) fields

Field name	Description	Data type	Key
AllowScheduling	Indicates whether orders associated with this hold code are brought into the order promising process and a commitment made to the customer request. Valid values are: • **N**—orders are not brought into the order promising process. No order commitments are made to these customer requests which are treated as informational only. • **Y**—orders are brought into the order promising process. Order commitments are generated in response to the customer request dates.	Boolean	
AllowShip	Indicates whether an order associated with this hold code can be released for shipment to the customer. Valid values are: • **N**—orders using this hold code are flagged to indicate they should not be shipped. • **Y**—orders using this hold code can be shipped to the customer as normal.	Boolean	
Description	A description or purpose of the hold code. For example, the hold code might have been created to deal with customer credit problems or an issue at the customer's receiving location.	String	
Id	A unique string identifier for this hold code.	String	Key
Customers	Set of customers that are currently assigned this hold code.	Set	
Independent Demand Solutions	Set of independent demands (line items) that are currently assigned this hold code.	Set	

ImportedStatisticalForecast table

The **ImportedStatisticalForecast** table can be used to import statistical forecasts that were generated by an external tool or application (instead of having them calculated by a forecast model in

RapidResponse). This table contains vector set data for records in the **ForecastItemParameters** table, and each record added should specify a forecast date and quantity for the referenced forecast item.

The statistical forecast values imported into this table can then be used by all other subsequent RapidResponse forecast calculations. For example, NPI/EOL profiles can be applied to the imported forecast before corresponding records are added to the **StatisticalForecast** table, and disaggregation logic can be applied to those values before being added to the **StatisticalForecastDetail** table at the detailed part-customer level.

Note that in order for the records in this table to be used by forecast logic, the referenced **ForecastItemParameters** record must have a **Type.ForecastModel** setting of "ForecastImport" (for all other settings, RapidResponse will generate the statistical forecast based on history and specified item-level parameters).

This table was added in Version 2016.2, Service Update 5.

Table 6.146: ImportedStatisticalForecast (Mfg) fields

Field name	Description	Data type	Key
Date	The date this forecast quantity applies to. Forecast values should be specified on dates belonging to the forecast item's **Type.IntervalsCalendar**. If a forecast is specified for a date not belonging to the item's **IntervalsCalendar**, it is automatically rounded back to the previous date that does belong to that calendar. Note also that forecast values should be specified on **IntervalsCalendar** dates starting after **SOPAnalyticsConfiguration.CalcHistoricalEndDate** and continuing for the number of intervals specified in **ForecastItemParameters.ForecastIntervalCount**. Any forecasts values imported outside of this range will be ignored (and intervals missing a forecast value are flagged with a warning).	Date	Yes
ItemParameters	A reference to the parameters associated with the forecast item for which this statistical forecast quantity is being imported. The parameters include those that would otherwise be used in generating the statistical forecast, as well as those applied to all statistical forecasts regardless of whether the forecast is generated or imported. The forecast item itself is also available through this reference. **Referenced table:** ForecastItemParameters **Reference syntax:** ItemParameters.ForecastCategory.Value ItemParameters.Item.Path	Reference (Owning)	Yes
Quantity	The quantity of the statistical forecast for the date specified. Note that any values less than zero, will be interpreted as zero (0).	Quantity	

IndependentDemand table

The **IndependentDemand** table identifies demand not directly related to demand for other parts. Demand for finished goods or services parts are examples of independent demand.

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
Allotment Override	A reference to the allotment override reserving component supply to this demand. The reserved supply is allocated to any demand(s) referencing the allotment override before it can be made available to other demands. **Referenced table:** AllotmentOverride (Nullable)	Reference	
ConfirmByDate	Date by which this order should be confirmed, or its allocations may be released. Is not directly used in calculations, but is used by applications and can be used in reports to show orders to be confirmed.	Date	
Customer RequestDate	The date on which the customer wants to receive delivery of the order at their location. The Order Fulfillment application uses this date along with pick/pack and transit times to set the **RequestDate** (the date by which the order should be completed to stock in order to meet the customer request). However, this field is not currently used in any analytic calculations.	Date	
DeliveryRoute	A reference to the delivery route used to transport this order to the customer. Related details such as transit time and carrier responsible for the delivery are available from this reference. **Referenced table:** DeliveryRoute (Nullable)	Reference	
DueDate	The date the order is scheduled to be completed to stock. **Default value**: Future **Note:** If the date is 'Undefined', the demand is ignored.	Date	

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
ExpiryOffset	Allows for the expiry requirements placed on component ingredients to be adjusted (reduced) on a demand by demand basis. It specifies the number of **Part.PlanningCalendars.ExpiryCalendar** intervals to extend the minimum shelf life, and hence the calculated **EarliestExpiryDate**, before exploding demand down to the item's components. Thus, the expiry date on supplies used to satisfy the demand must then satisfy this adjusted earliest expiry date. When calculated expiry dates are subsequently rolled back up the product structure, the **SupplyDemand**.**ExpiryDate** on the record reporting an allotment of supply to this top level demand is then adjusted (reduced) by the value in this field. This ensures that the **ExpiryDate** reflects the limiting and demand-specific expiry requirements on the rolled up components. Note that this field is only applicable on records for which the **MinimumShelfLife** value is provided (ignored when **MinimumShelfLife** is taken from the **Part** record). **Default value:** -1	Integer	
InputPool	The pool to which this independent demand line belongs. This field supports inventory pooling as well as forecast consumption by pool. It is used only when **Part.MUEPoolNettingType.PoolRule** is set to a value other than "Ignore". If this field is left Null, then the pool from **Order.Pool** is used instead. Typically, this field should be used when the pools applicable to demands do not easily correspond to the order level. For example, it should be used if pools are meant to represent specific areas like a plant or storage location. However, if pools are meant to represent customers, or other order-level items, then this value might be left Null and the pool from the **DemandOrder** record will be used instead. **Referenced table:**Pool (Nullable) **NOTE**: This field was added in Version 2016.2, Service Update 19	Reference	
Line	A unique identifier associated with each IndependentDemand record with the same Order.Id. **Default value**: blank string	String	Yes

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
MinimumShelf Life	This field specifies the number of **Part.PlanningCalendars.ExpiryCalendar** intervals that are added to the demand's **DueDate** when calculating its **EarliestExpiryDate**. The EarliestExpiryDate is the date before which supplies must not expire if they are to be allocated to the demand. For example, if the DueDate is July 1 2009, Part.PlanningCalendars.ExpiryCalendar is set to "Everyday", and this value is set to "21", then the EarliestExpiryDate would be set to July 22, 2009 (that is, any supply expiring before July 22, 2009 cannot be used to satisfy the demand). When this field is undefined (set to a value less than zero), then the **Part.MinimumShelfLife** is used in calculating the EarliestExpiryDate for the demand. **Default value:** -1	Integer	
Model	The model that is associated with this independent demand. Based on the **DemandStatus.ModelUsage** setting, either this model or the default model (the None model) is effective for the demand. The actual usage of the demand's effective model in analytic calculations is determined by the setting in **Part.MUEPoolNettingType.ModelRule.** This further depends on the Model-Unit Effectivity capability having been enabled. **Referenced table:** Model	Reference	
Order	A value that determines the processing policies of IndependentDemand. **Referenced table:** DemandOrder **Reference Syntax:** Order.Type, Order.Id, Order.Site	Reference	Yes
OrderPriority	The priority associated with this demand. **Referenced table:** OrderPriority	Reference	
Original RecordId	Identifies records that have been sent to your enterprise data sources in a closed-loop operation. This is a concatenation of the date the closed-loop operation was performed and the RecordId value for the record. When data is next updated, this value is compared to the new IndependentDemand records and ensures the records are not double-counted. The RecordId value is a number representing the sequence a record was created in. Each additional record added to the table increments the RecordId by one, and RecordId values are not reused when records are deleted. For more information about RecordId, see the *RapidResponse Resource Authoring Guide*. This field is in the **Solutions** namespace.	String	

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
Part	Part for which the IndependentDemand exists. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	
PromisedDate	The date the part was promised for delivery to the customer receiving location. The difference between DueDate and this date represents pick/pack and transit time when reporting projected delivery date from AvailableDate. **Default value**: Undefined	Date	
ProtectQuantity	Indicates whether the value in the Quantity field is modified when data is edited in a grouped worksheet. This field is used in the Advanced Data Editing dialog box as part of the expression that determines which records are not edited when a grouped worksheet is edited. Valid values are: • **Y**—the Quantity cannot be modified in summarized worksheets. • **N**—the Quantity can be modified. For more information, see "Protect a record" in the *RapidResponse Resource Authoring Guide*. **Default value**: N	Boolean	
Quantity	The amount still to be delivered. Demands with negative quantities are ignored. **Default value**: 0	Quantity	
RequestDate	The date by which the order would need to be completed to stock in order to ensure it can be shipped in time to meet the customer's requested date. After all demands are initially planned to their scheduled due date, any demands that have **OrderPriority.TwoDatePlanningRule** set to "Use" are then moved towards their request date as possible. This allows for available dates that are closer to the original customer request date (and therefore potentially earlier than the committed due date). For more information about how this field is used in two-date planning logic, see "Two-date planning" on page 1803. **Default Value:** Undefined	Date	
SavedPriority	Duplicates OrderPriority, load with same data as OrderPriority. Not used by RapidResponse analytics, but may be used by RapidResponse legacy applications. **Referenced table:** OrderPriority	Reference	

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
ShipmentPolicy	The shipment policy associated with this independent demand. It controls whether the quantity can be split into partial shipments, or if it must be shipped in its entirety. **Referenced table:** ShipmentPolicy **Note:** This field has no impact on any independent demands that belong to ship sets.	Reference	
ShippedQty	The quantity that has already been shipped. Negative quantities are ignored (treated as 0) by RapidResponse calculations. Note that if **DemandType.ProcessingRule** is set to "SalesActual", then this quantity can consume forecast (though is otherwise ignored by Netting). **Default value**: 0	Quantity	
ShipSet	A reference to the ship set, if any, this independent demand belongs to. All independent demands with the same Order and ShipSet value should ship together as a whole order. A blank value (that is, ' ') should be specified for independent demands not belonging to a ship set. **Referenced table:** ShipSet (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
StartUnit	The unit number to use for this independent demand. The StartUnit is applied to the entire demand, so the supply for this demand must have the same StartUnit. For example, if the demand has StartUnit=25 and Quantity=3, then all three units of supply will be for StartUnit=25 (and not for StartUnit=25, 26, and 27). Therefore, if using serial number (unit) control of netting or configuration, the Quantity should be 1 so that each unit will be planned and configured independently. This field is ignored unless the Model-Unit Effectivity is enabled, and the value of Part.MUEPoolNettingType.UnitRule is set to either “Net” or “Block”. **Default value**: -1	Integer	
Status	A reference to the DemandStatus table. **Referenced table:** DemandStatus	Reference	

Table 6.147: IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
Transaction Sequence	The sequence of this demand. If demands have the same order priority and **MaintainCommitments** is set to 'Y' for the part, then supplies will be allocated by transaction sequence value (smallest to largest). **Note:** This field has no impact on any independent demands that belong to ship sets. **Note:** On startup, the server checks the maximum value for a transaction sequence number and if the value is greater than 0x30000000 hex (or 805306368 decimal), it will reset the sequence number back to zero and re-sequence all the ScheduledReceipt and IndependentDemand records.	Integer	
UnitSelling Price	The unit price for this line item. A negative value is a flag to calculate the unit selling price using the Quantity and the Part.AverageSellingPrice. **Default value**: 0	Money	

IndependentDemand table – calculated and set fields

The following describes the calculated and set fields in the IndependentDemand table. For an introduction to this table, and descriptions of its input fields, see "IndependentDemand table" on page 358.

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
ActionParts Count	The number of action parts on this demand. An action part refers to any part with a supply pegged to this demand where the availability of that supply is contributing to the demand being late. For example, the supply might have a due date later than needed, or it might have a constraint that makes it unavailable in time.	Integer	
AvailableDate	The date at which the demand will actually be completed to stock in consideration of capacity, constraint, and material availability. If EffectiveDemand = 0, then this field reports a value of "Undefined".	Date	
AvailableReceipt Date	The date on which the order is now expected to be received at the customer destination. This is calculated by adding the applicable pick and pack time to **ShipSetAvailableDate**, offsetting to the next date on the delivery route's pickup calendar (or the Everyday calendar if a pickup calendar is not defined), and then adding transit time for the route.	Date	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
AvailableShip Date	The date on which the order will actually be available to ship to the customer. This is calculated by adding the applicable pick and pack time to **ShipSetAvailableDate** and then returning the next date on the delivery route's pickup calendar (or the Everyday calendar if a pickup calendar is not defined).	Date	
CostOfGoods Sold	The total rolled-up costs of goods sold from all component supplies used in satisfying this independent demand. This field summarizes the date-specific cost values reported in the **IndependentDemandCost** table and described on page "IndependentDemandCost table" on page 1073.	Money	
DaysLate	How late (in working days) this demand is expected to be.	Integer	
EarliestExpiry Date	The date before which a supply must not expire if is to be used in satisfying this demand. Calculated based on the demand **DueDate** and either the **MinimumShelfLife** and **ExpiryOffset** values (if the **MinimumShelfLife** value is greater than or equal to zero) or **Part.MinimumShelfLife**. When the **Part.Type.ExpiryRule** is set to "Ignore", then this field is always set to "Past".	Date	
EffectiveDemand	Effective demand quantity. The quantity of this demand actually used by netting. For MPS or MPSConfig type parts, if the record has **Order.Type.ProcessingRule** = 'SalesForecast', then EffectiveDemand is the portion of the forecast that is past the demand time fence and not consumed (and hence drives demand). This is the sum of UnconsumedQty of Forecast records linked to this IndependentDemand. For other parts, if **Order.Type.ProcessingRule** is 'SalesForecast' or 'Ignore', then EffectiveDemand is 0 (demand is ignored by netting). Similarly if **Part.Type.ProcessingRule** is set "Ignore" or if the part never has any valid part sources, than a value of 0 is also returned. **Note:** For orders left at least partially unsatisfied, this field will have also been reduced by the value reported in the UnsatisfiedQuantity field.	Quantity	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
EffectiveUnit Price	The unit price effective for the part and customer on the date this demand is due. This value is useful when calculating revenue, and is calculated as follows: • If IndependentDemand.UnitSellingPrice is greater than zero, that value is reported here. • Otherwise, RapidResponse checks the CustomerPrice table for records with matching Part and Customer values that have effective dates less than or equal to the demand due date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, the EffectiveUnitPrice calculation uses the record with the larger value in the CustomerPrice.Id field. • However, if no matching Part and Customer values are found, RapidResponse next checks the CustomerPrice table for records with a matching Part and blank Customer value (that is, ' ') that have effective dates less than or equal to the demand due date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above, the value in Part.AverageSellingPrice is reported here. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
FeatureList	Set of **IndependentDemandFeatureList** records that reference this independent demand. Applicable to configurable, made to order end-items (parts where **PartType.FeatureBOMRule** is set to a value other than "Ignore"). The set of string values in this field represents the list of all features required for the final assembly on this order. This feature list is then compared against values in the **BillOfMaterial.FeatureRule** field for each record that defines a component directly used in the end-item assembly that this order is for. If the **FeatureRule** field either matches a value defined in the feature list or is empty, then the component is required for this demand; otherwise, it is not. Thus, this feature ensures that the correct components by feature are selected and dependent demands placed correspondingly throughout the structure. **Note:** This field was modified in Version 2016.2	VectorSet	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption Date	Reports the date from which the forecast consumption window is based for this demand. Considering the applicable settings in the **ForecastConsumptionDateRule** field on the **DemandStatus** and/or **PartType** tables, this returns the demand's **DueDate**, **RequestDate**, or **Part.PlanningCalendars.Date.FirstDate**. This field is populated for all actual demands (**Order.Type.ProcessingRule** = 'SalesActual') on MPS parts (**Part.Type.ProcessingRule** = 'MPS' or 'MPSConfig'). Otherwise, it returns "Undefined".	Date	
ForecastQuantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	
GatingConstraint	The Constraint reported for the latest supply allocated to this independent demand from the gating part. Null if that supply's reported Constraint is Null. **Referenced table:** Constraint	Reference	
GatingPart	A reference to the part that determines the AvailableDate for this demand, if it is late. GatingPart is also calculated for Forecast demands (the GatingPart of the latest supply, including spreading). GatingPart for a demand is the GatingPart of the latest available supply allocated to the demand. It will be Null if the demand is satisfied on time. It will be the part itself if the last supply is gated by itself (which includes Constraint, rescheduling limits, time fence), or is available for its ReschedDate, but that is later than the demand due date. **Referenced table:** Part	Reference	
MappedPool	The pool used in netting this demand. The value returned is affected by the **Part.MUEPoolNettingType.PoolRule** setting as follows: • If **PoolRule** is "Net" and the **InputPool** reference on this line item is populated, then that referenced pool is used. • If **PoolRule** is "Net" and the **InputPool** reference is Null, then the pool referenced by **Order.Pool** is used (from the demand order). • If **PoolRule** is "Ignore", "Forecast", or "ForecastOptional", the part's common pool, from the **Part.PlanningCalendars.CommonPool** reference, is always used. • If **PoolRule** is "Optional" and the demand's pool is listed in the corresponding **PoolMap** table (by PartType) or **PoolMapOverride** table (by Part), then the demand's pool is used. Otherwise, the part's common pool is used. **Referenced table:** Pool	Reference	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
MaterialSpend	The total rolled-up material costs from purchased parts used in satisfying this independent demand. This field summarizes the date-specific cost values reported in the **IndependentDemandCost** table and described on page "IndependentDemandCost table" on page 1073.	Money	
NewProcessCost	Standard labor and overhead cost of planned orders to complete this order. NewProcessCost is calculated as: (UOMConversion (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2) * (inventory) Quantity) from all planned orders pegged back to this IndependentDemand. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
NewPurchases Cost	The cost of planned order purchase cost attributed to this demand. This is calculated using PartSource.EffUnitCost. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
OnTimeQuantity	The amount of this demand that can be delivered on time. If Type.ProcessingRule is 'SalesForecast', then the sum of OnTimeQuantity from every Forecast record on this part derived from this record.	Quantity	
PartCustomer	A reference to the entry in the PartCustomer table, if any, that has the same part and customer as this IndependentDemand record. **Referenced table**: PartCustomer	Reference	
PlannedReceipt Date	The date on which the order is currently scheduled to be received at the customer destination (based on the last commitment made to the customer. This is calculated by adding the applicable pick and pack time to **DueDate**, offsetting to the next date on the delivery route's pickup calendar (or the Everyday calendar if a pickup calendar is not defined), and then adding the transit time for the route.	Date	
PlannedShipDate	The date on which the order is currently scheduled to be shipped to the customer destination (based on the last commitment made to the customer). This is calculated by adding the applicable pick and pack time to the **ShipSetAvailableDate** and then offsets to the next date on the delivery route's pickup calendar (or the Everyday calendar if no pickup calendar is specified).	Date	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
ShipSetAvailable Date	The latest **AvailableDate** reported across all independent demands belonging to the same demand order and ship set. For independent demands not belonging to a ship set, this value is the same as **AvailableDate**.	Date	
TaskNeedDate	The earliest date that this demand is required by any project task associated with it through a **TaskDemandLink** record. If this order is not required by any tasks, then 'undefined' is reported. **Note:** This field belongs to the ProjMgmt namespace.	Date	
ToleranceProfile Zone	A reference to the entry in the ToleranceProfileZone table that applies to this IndependentDemand record. It is used in defining the acceptable levels of change when comparing independent demands against baseline historical demand series. The ToleranceProfile for this record is the value referenced from the PartCustomer field, and the ToleranceProfileZone is set to the entry with the lowest ZoneEndInterval value where `Part.PlanningCalendars.RunDate.FirstDate + ZoneEndInterval ToleranceProfile.ToleranceCalendar > IndependentDemand.DueDate`. **Referenced table**: ToleranceProfileZone	Reference	
Unsatisfied Quantity	The portion of this demand that was left unsatisfied. T This field is intended for use with parts that use partial planning logic. Thus it always returns zero (0) in cases where **Order.Type.PlanningRule** is set to "Full". If **Order.Type.PlanningRule** is set to "Partial", then it represents the portion of the demand that was left unsatisfied because it could not be satisfied within the defined lateness tolerance. Note however that if a demand is left unsatisfied because the part never has any valid part sources or because the part references a **Type.ProcessingRule** of "Ignore", then this field reports a value of zero (0) regardless.	Quantity	
CostInfo	Set of **IndependentDemandCost** records that contain cost values associated with the set of supplies needed to satisfy this demand. Used in calculations that populate the **CostOfGoodsSold** and **MaterialSpend** fields.	Set	
FeatureBOMs	Set of **FeatureBOMForIndependentDemand** records associated with this IndependentDemand.	Set	
FeaturesList	Set of **IndependentDemandFeature** records associated with this IndependentDemand.	Set	
Independent DemandSolutions	Set of **IndependentDemandSolution** records that hold order fulfillment details pertaining to this IndependentDemand.	Set	

Table 6.148: Calculated IndependentDemand (Mfg) fields

Field name	Description	Data type	Key
LateSupplies	Set of **LateSupply** records associated with this IndependentDemand.	Set	
Notes	Descriptive notes appended to this demand. This field can used to enter notes about an independent demand. Each time a note value is added to this field through a worksheet, a new record is created in the IndependentDemand_Notes table with a date and timestamp, user ID, note text, and a reference to the IndependentDemand record against which it was created.	Note	
TaskDemandLinks	Set of TaskDemandLink records associated with this IndependentDemand.	Set	
WhereConsumed ForDemands	Set of WhereConsumedForDemand records associated with this IndependentDemand.	Set	

IndependentDemandFeatureList table

The **IndependentDemandFeatureList** table supports Feature BOM logic and is used to indicate the features required on independent demands for configurable end-items (parts where **PartType.FeatureBOMRule** is set to a value other than "Ignore"). Each record in this table references an **IndependentDemand** record and specifies one feature required by that demand.

Together, all records that reference a given **IndependentDemand** record define the list or set of feature options required by that demand and are represented by the corresponding **IndependentDemand.FeatureList** set field. The set of features are then compared against the **BillOfMaterial.FeatureRule** value for each record that defines a component directly used under the configurable end-item assembly that the demand is for. If the **FeatureRule** value matches a value in the feature list or is empty, then the component is required for the demand (otherwise, it is not).

This table was added in Version 2016.2

Table 6.149: IndependentDemandFeatureList (Mfg) fields

Field name	Description	Data type	Key
Feature	The name of a feature required by the demand referenced on this record. **Note:** Feature names are not case-sensitive, and any leading or trailing spaces in the feature name are ignored. Optionally, the feature name can be enclosed in single or double quotes (but this is not required).	String	Yes
Independent Demand	The end-item demand where the feature specified on this record is required. **Referenced table:** IndependentDemand	Reference (Owning)	Yes

IndependentDemandSolution table

The **IndependentDemandSolution** table supports the Order Fulfillment application available with RapidResponse, and a record is typically added this table by that application when it processes a new **IndependendentDemand** record (customer request).

Each record in this table references a record in the **IndependentDemand** table, and contains additional details for the referenced line item. For example, this table allows for assigning hold codes to demand lines, and also reports the number of customer-requested or supply-driven changes associated with each demand

Note that although use of this table is required by the Order Fulfillment application, none of its fields are used in actual RapidResponse analytic calculations.

Table 6.150: IndependentDemandSolution (Solutions) fields

Field name	Description	Data type	Key
ActualReceipt Date	Used to record the date on which the customer actually received the order (reflects transit time incurred from the point the shipment was made).	Date	
ActualShipDate	Used to record the date on which the order was actually shipped to the customer	Date	
AutoCommit HorizonDate	A date after which the Order Fulfillment application will automatically commit to the customer request. That is, if the line item's **RequestDate** is on or after this date, then the line's **DueDate** is set to the **RequestDate** and the order is automatically promised. Otherwise, manual intervention may be needed in order to commit to the customer request (orders earlier than this date, but after the **FrozenHorizonDate,** will have their **DueDate** set to the **RequestDate** only if there is sufficient unallocated supply). **Note:** This field is calculated when the order is first processed in RapidResponse by adding the **AutoCommitHorizon** field value, from either the **PartCustomer** table (if populated) or else the **PartSolution** table, to the **OrderCreation** date.	Date	
CustomerChange	Indicates whether the order has had a customer requested change (for example, an increase in the order quantity or a change in the requested date). Valid values are: • **N**—the order has not had any customer changes (this is the default). • **Y**—the order has had at least one customer change (this value is automatically set on records when a change is detected).	Boolean	

Table 6.150: IndependentDemandSolution (Solutions) fields

Field name	Description	Data type	Key
DwellTime	Dwell time for this order. This represents extra time, if any, built into the customer request and during which no activity takes place against the order. Calculated as the number of Everyday intervals between **IndependentDemand.RequestDate** and **AutoCommitHorizonDate** at the time the order is first processed in RapidResponse (and not recalculated if the request date subsequently changes). This will be a positive value if the **RequestDate** is later than the **AutoCommitHorizonDate** and 0 (zero) otherwise.	Integer	
EarlyShipWindow	Maximum number of days earlier than the planned ship date that the order is allowed to ship. For example, shipping an order early might allow for early revenue recognition. Shipping the order any earlier than the amount of time specified here requires explicit approval from the customer. This field value should be expressed in terms of the Everyday calendar and is used in calculating the **EarlyShipDate**. If a negative value is specified here, then the default **EarlyShipWindow** from the **Customer** record is used.	Integer	
FrozenHorizon Date	A near-term date before which the Order Fulfillment application will never automatically commit to the customer request. That is, if the order's **RequestDate** is before this date, then the **DueDate** will never be set to that request even if there is supply available to do so. Instead, the **DueDate** is set to the latter of this date or the **AvailableDate**. Manual intervention would then be needed to pull the order back in and commit to a request within the frozen horizon. Note that orders later than this date but before the **AutoCommitHorizonDate** have their **DueDate** set to the request only when there is sufficient unallocated supply available. **Note:** This field is calculated when the order is first processed by adding the **FrozenHorizon** field value, from either the **PartCustomer** table (if populated) or else the **PartSolution** table, to the **OrderCreation** date.	Date	

Table 6.150: IndependentDemandSolution (Solutions) fields

Field name	Description	Data type	Key
HoldCode	The hold code, if any, that has been applied to this line item. Hold codes are used to prevent orders from being processed beyond a particular state (for example, an order might be prevented from shipping due a credit issue or a problem at the customer warehouse). Hold codes can also be specified at the customer-level , and the hold code that applies to a given order line is reported in the **EffectiveHoldCode** field (a hold code at the order line level always takes precedence over one at the customer level). **Referenced table:** HoldCode (Nullable)	Reference	
Independent Demand	The specific line item to which the details on this record apply. **Referenced table:** IndependentDemand	Reference	Yes
LatestDueDate	Used to store the original **DueDate** that the Order Fulfillment application generated on the **IndependentDemand** record. Thus, when exploring options to move demand dates earlier, this date is available for comparison purposes.	Date	
OrderCreation	The date and time at which the order was first imported into RapidResponse.	DateTime	
OrderPromised	Indicates whether a initial date for the order has been committed or promised to the customer. Valid values are: • **N**—a commitment has not yet been made to the customer (default value). • **Y**—a commitment has been made to customer (this value is automatically set when the order is promised in RapidResponse for the first time).	Boolean	
OrderPromised DateTime	The date and time at which a date for the order was committed to the customer.	DateTime	
Processed	Indicates whether the order has been processed. Valid values are: • **N**—Indicates the order is in need of an initial promise date or a re-promising. • **Y**—If this is a new order, indicates an initial commitment has been made to the customer. If this an existing order in need of re-promising, indicates a new commitment has been made to the customer.	Boolean	

IndependentDemandSolution table – calculated and set fields

The following describes the calculated and set fields in the **IndependentDemandSolution** table. For an introduction to this table, and descriptions of its input fields, see "IndependentDemandSolution table" on page 370.

Table 6.151: Calculated IndependentDemandSolution (Solutions) fields

Field name	Description	Data type	Key
CustomerNotes	Notes added to this demand for the purpose of tracking customer-requested changes to orders during the order fulfillment process. Note that this field supports the Order Fulfillment application and should not directly be edited. It is automatically updated whenever a change is made to the request date, quantity, part, or routing on the order. Each time a value is added to this field, a new record is then automatically added to the **IndependentDemandSolution_CustomerNotes** table with a datetime stamp, user ID, note text, and reference to the **IndependentDemandSolution** record against which it was created.	Note	
EarlyShipDate	The earliest date on which this order can ship without requiring explicit approval from the customer. In other words, if the order is ready earlier than the planned ship date, it can potentially ship as early as the date returned by this field. This calculation relies on the **EarlyShipWindow** field (if positive) or else the applicable default **EarlyShipWindow** value from the **Customer** table.	Date	
EffectiveHoldCode	The effective hold code for this line item. If this record has a valid **HoldCode** reference, then it is always reported here. If the **HoldCode** reference is Null, but there is a valid **HoldCode** referenced on the **Customer** record, then that will be reported here. Otherwise, this reference returns Null. **Referenced table:** HoldCode	Reference	

Table 6.151: Calculated IndependentDemandSolution (Solutions) fields

Field name	Description	Data type	Key
ShipSetHasHold Code	Indicates whether a hold code is applied to any line items in the ship set, if any, that this demand belongs to. For example, this can help determine if a hold code on another demand in the same ship set might prevent this one from shipping. Valid values are: • **N**—no demands in the ship set that this one belongs to have hold codes applied. • **Y**—at least one demand in the ship set this one belongs to has a hold code applied.	Boolean	
SupplyNotes	Notes automatically added to this record for the purpose of tracking supply-related changes to demands during the order fulfillment process. This field supports the Order Fulfillment application and should not directly be edited. It is automatically updated whenever a supply-related change is made to the **IndependentDemand** record during the order fulfillment process. Each time a value is added to this field, a new record is automatically added to the **IndependentDemandSolution_ SupplyNotes** table with a datetime stamp, user ID, note text, and reference to the **IndependentDemandSolution** record against which it was created.	Note	

InventoryTransfer table

The **InventoryTransfer** table holds transfers of nettable inventory from one part and site to another. For example, when a customer order is projected to be late due to a supply shortage, one response might be to transfer the required supply from a site with excess on hand inventory to the site that requires it.

Each record in this table contains details of an inventory transfer such as the two parts involved in the transfer (the part with the shortage and the part being transferred), the quantity of inventory being transferred, and the cost and lead time associated with the transfer. Depending on the control settings associated with a given inventory transfer, RapidResponse analytics may decrease the on hand quantity of the transfer part at the sourcing site and create supply for the part at the destination site to reflect the transfer.

Note also that the amount of a given transfer cannot exceed the reported on hand inventory at the site supplying the part. However, care should be taken when using inventory transfers in a backflush environment. For example, when SupplyStatus.State is set to "BackflushButNotAllocated", the dependent requirements of a scheduled receipt are dated at "Past" and the on hand inventory is reduced although the actual OnHand count does not reflect this until the scheduled receipt is built. In this case, an inventory transfer could be created and planned, but the actual supply might not ultimately be available to be moved.

Table 6.152: InventoryTransfer (Mfg) fields

Field name	Description	Data type	Key
ActualDockDate	The actual date the inventory transfer arrived at the receiving dock, which is used in historical reporting. This field is in the **Solutions** namespace. **Note:** This field was added in the Version 2016.2, Service Update 5.	Date	
DockToStock LeadTime	The amount of time required to move supply from the receiving dock to available inventory. It is measured in **Part.PlanningCalendars.TimeUnits**.	Quantity	
ExpiryStartDate	Used to specify the expiry start date for this inventory transfer. If a value is not provided, then the **Date** field from the **OnHand** record used to supply this transfer is used instead (the **EffectiveExpiryStartDate** field reports the actual expiry start date being used for inventory transfers on limiting components). **Default value:** Future	Date	
Id	A unique identifier for this inventory transfer record.	String	Yes
Model	A reference to the model for the **Part**. **Referenced table:** Model	Reference	
Notes	Any notes users have added to this inventory transfer record.	String	
OriginalRecordId	Identifies records that have been sent to your enterprise data sources in a closed-loop operation. This is a concatenation of the date the closed-loop operation was performed and the RecordId value for the record. When data is next updated, this value is compared to the new InventoryTransfer records and ensures the records are not double-counted. The RecordId value is a number representing the sequence a record was created in. Each additional record added to the table increments the RecordId by one, and RecordId values are not reused when records are deleted. For more information about RecordId, see the *RapidResponse Resource Authoring Guide*. This field is in the **Solutions** namespace.	String	
Part	The name of the part (at the destination site) that is receiving the inventory transfer. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes

Table 6.152: InventoryTransfer (Mfg) fields

Field name	Description	Data type	Key
Pool	A reference to the pool for the Part. **Referenced table:** Pool	Reference	
Quantity	The amount of the inventory transfer. This value is reported in the unit of measure used by the Part.	Quantity	
ShipDate	The date the inventory transfer is expected to leave the sourcing site.	Date	
Source	References the **Source** record that applies to the transfer destination record. **Referenced table**: Mfg::Source (This field is in the Solutions namespace.) **Note:** This field was added in the Version 2016.2, Service Update 5.	Reference	
StartUnit	The start unit for the Part.	Integer	
TransferCost	The total cost of transferring supply from the sourcing site to the destination site.	Money	
TransferModel	A reference to the model for the TransferPart. **Referenced table**: Model	Reference	
TransferPart	The name of the part (at the sourcing site) that is providing the inventory transfer. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes
TransferPool	A reference to the Pool for the TransferPart. **Referenced table**: Pool	Reference	
TransferStart Unit	The start unit for the TransferPart.	Integer	

Table 6.152: InventoryTransfer (Mfg) fields

Field name	Description	Data type	Key
TransitCalendar	A reference to the calendar in which the **TransitTime** value should be expressed. For example, if the vehicles used to transport goods from the supplying site to the destination site do not operate on weekends, this might be set to a standard Workday calendar. If this reference is left Null, then an Everyday calendar is assumed. **Referenced table:** Core::Calendar (Nullable) **Note:** This table was added in Version 2016.2.	Reference	
TransitTime	The amount of time required to transfer inventory from the sourcing site to the destination site. This value should be expressed in intervals of the **TransitCalendar** (or the Everyday calendar if the **TransitCalendar** reference is left Null)	Quantity	
Type	A control setting that indicates the state of the inventory transfer (received, intransit, not released, or by date) and hence the processing RapidResponse performs on the transfer. **Referenced table**: InventoryTransferType	Reference	

InventoryTransfer table – calculated and set fields

The following describes the calculated and set fields in the **InventoryTransfer** table. For an introduction to this table, and descriptions of its input fields, see "InventoryTransfer table" on page 374.

Table 6.153: Calculated InventoryTransfer (Mfg) calculated fields

Field name	Description	Data type	Key
DockDate	The date the inventory transfer is expected to arrive at the destination site's receiving dock. Calculated as *ShipDate* + *TransitTime*.	Date	
EarliestExpiry Date	Earliest date that on hand inventory can expire if it is to be allotted to this transfer. Determined from the transfer **StockDate** and then offset by the number of expiry calendar intervals specified in **Part.MinimumShelfLife**.	Date	

Table 6.153: Calculated InventoryTransfer (Mfg) calculated fields

Field name	Description	Data type	Key
EffectiveDemand Quantity	The effective demand quantity for this inventory transfer. This value is reported in the TransferPart's unit of measure, and represents the amount to reduce on hand inventory by at the sourcing site. If Type.Status is set to "Received" or "InTransit", this value is always 0. If Type.Status is set to "NotReleased" this value can be between 0 and Quantity (set to Quantity as long as there is sufficient on hand inventory at the sourcing site to satisfy the transfer).	Quantity	
EffectiveExpiry StartDate	The expiry start date for the inventory transfer. Expiry start dates from components are used to determine when expiry date calculations should begin for assemblies with **PartType.ExpiryRule** set to "DependentStart". This field returns the value in **ExpiryStartDate** (if provided) or else set to the **Date** field from the **OnHand** record used to supply the transfer. Note that if the part is not a limiting component (has **ExpiryType.ProcessingRule** of "Normal"), then this field reports a value of "Future".	Date	
EffectiveSupply Quantity	The effective supply quantity for this inventory transfer. This value is reported in the Part's unit of measure, and represents the amount of supply to create at the destination site. If Type.Status is set to "Received", this value is always 0. If Type.Status is set to either "InTransit" or "NotReleased", this value can be between 0 and Quantity (set to Quantity as long as there is sufficient on hand inventory at the sourcing site to satisfy the transfer).	Quantity	
EffectiveUnit TransferCost	The per unit cost of this inventory transfer. Calculated as *TransferCost /EffectiveSupplyQuantity*. Currency conversions on this field use the **ShipDate** field's conversion rate.	Money	
ExpiryDate	The date on which this inventory transfer expires (is no longer usable). It is set to the expiry date of the OnHand record supplying the transfer.	Date	
ExpiringQuantity	The amount of the inventory transfer that will not be used before its expiry date.	Date	
StockDate	The date the inventory transfer is expected to be received into inventory at the destination site. Calculated as *ShipDate* + *TransitTime* + *DockToStockLeadTime* (adjusted by Part.PlanningCalendars.TimeUnits).	Date	

ItemSource table

The **ItemSource** table defines multiple sources of supply (**ItemSource** records) for parts (**PartSource** records) that have similar deviations in their planning parameters, such as lead time.

The lead time deviation is determined based on the difference between the **PartSource**'s fixed lead time and the **PartSource**'s actual lead time from the **HistoricalSupplyActual** data, as reported by **ItemSourcePartSourceHistoricalLeadTime**.

Yield deviation is determined based on the difference between the **PartSource**'s yield and the actual yield from the **HistoricalReceipt** data, as reported by **ItemSourcePartSourceHistoricalReceipt**.

This table was added in Version 2016.2, Service Update 21.

Table 6.154: ItemSource (Mfg) fields

Field name	Description	Data type	Key
Id	A key to allow item sources with the same Part and Source combination to have various combinations of BOMAlternate and Routing settings.	String	Yes
AcceptLeadTimeChanges	Used to manually accept the projected lead times for the **ItemSource**. Valid values are: • **Y** • **N** **Default value:** N	Boolean	
AcceptYieldChanges	Used to manually accept the projected yields for the **ItemSource**. Valid values are: • **Y** • **N** **Default value:** N	Boolean	
AutomatedLeadTimeType	The **AutomatedLeadTimeType** that defines how the projected lead times are processed for the **ItemSource**. **Referenced table:** AutomatedLeadTimeType (Nullable) **Reference syntax:** AutomatedLeadTimeType.Value	Reference	
AutomatedYieldType	The **AutomatedYieldType** that defines how the projected yields are processed for the **ItemSource**. **Referenced table:**AutomatedYieldType (Nullable) **Reference syntax:** AutomatedYieldType.Value	Reference	
BOMAlternate	A reference to the alternate BOM for the **ItemSource**. **Referenced table:**BOMAlternate (Nullable) **Reference syntax:** BOMAlternate.Value	Reference	

Table 6.154: ItemSource (Mfg) fields

Field name	Description	Data type	Key
Cluster	A reference to the **Cluster** for the **ItemSource**. **Referenced table:** Cluster (Nullable) **Reference syntax:** Cluster.Name	Reference	
HistoricalSupplyCategoryRule	Determines whether the **ItemSourceHistoricalSupplyCategory** is to be interpreted as the set of **HistoricalSupplyCategory** records to include or exclude when matching **HistoricalSupplyActual** records for **ItemSourcePartSourceHistoricalLeadTime**. Valid values are: • **Include** —Any category in **ItemSourceHistoricalSupplyCatergory** will be used by the Machine Learning service. Note that this value is currently unsupported. • **Exclude**— Any category in **ItemSourceHistoricalSupplyCatergory** will be ignored by the Machine Learning service. Note that only this value is currently supported. **Default value:** Exclude **Note:** This field was added in Version 2016.2, Service Update 23 and is hidden.	Enum	
LeadTimeCorrectionStatus	Specifies whether the projected lead times for the **ItemSource** have been reviewed. Valid values are: • **Not Reviewed** —The projected lead times have not been reviewed and not yet copied to the planning data (**TimePhasedPartSourceLeadTime**) for the associated **PartSource** records. • **Reviewed** —The projected lead times have been reviewed and copied to the planning data (**TimePhasedPartSourceLeadTime**) for the associated **PartSource** records. **Default value:** Not Reviewed	Enum	
Part	A reference to the **Part** associated with the **ItemSource**. **Referenced table:** Part **Reference syntax:** Part.Name, Part.Site	Reference	Yes
Routing	A reference to the sequence of operations (**Routing**) associated with the **ItemSource**. **Referenced table:**Routing **Reference syntax:** Routing.Id, Routing.Site	Reference	

Table 6.154: ItemSource (Mfg) fields

Field name	Description	Data type	Key
Source	A reference to the **Source** associated with the **ItemSource**. **Referenced table:** Source **Reference syntax:** Source.DestinationSite, Source.ID, Source.Supplier	Reference	Yes
YieldCluster	A reference to the **Cluster** associated with the **ItemSource** for yields. **Referenced table:**Cluster (Nullable) **Reference syntax:** Cluster.Name, Cluster.SelfHealingParameter	Reference	
YieldCorrectionStatus	Specifies whether or not the given **ItemSource** has been reviewed for yield correction or not. Valid values are: • **Reviewed** • **NotReviewed** **Default value:** NotReviewed	Enum	

ItemSource table – calculated and set fields

The following describes the calculated and set fields in the **ItemSource** table.

Table 6.155: ItemSource calculated (Mfg) fields

Field name	Description	Data type	Key
CalcHistoricalLeadTimeError	A measure (e.g. mean absolute deviation) of the error between the posted lead times (from **PartSource.FixedLeadTime** or **TimePhasedPartSourceLeadTime.FixedLeadTime**) and the actual lead times (from **HistoricalSupplyActual.LeadTime**). This measure determines how accurate the posted lead times have been and is used to determine whether an **ItemSource** is near, inside, or outside tolerance. The particular error measure used to calculate this value is determined by the value of **AutomatedLeadTimeType.HistoricalLeadTimeErrorMeasure**. The input to this calculation may be seen in the **ItemSourcePartSourceHistoricalLeadTime** table. For a given **ItemSource**, the value of this field is the mean absolute deviation (or mean absolute percentage error) between **ItemSourcePartSourceHistoricalLeadTime.PartSourceLeadTime** and **ActualLeadTime** for all records. **Note:** This calculation is not based on a limited time horizon. It uses all matching **HistoricalSupplyActual** records for which there is an effective **PartSource**. **Note:** If no error measure can be calculated (i.e. if there are no matching **HistoricalSupplyActual** records and effective **PartSource** records can be found), then the value of this field is -1.	Quantity	

Table 6.155: ItemSource calculated (Mfg) fields

Field name	Description	Data type	Key
CalcHistoricalYieldError	A measure (e.g. mean absolute deviation) of the error between modeled yields (from **PartSource.Yield** or **TimePhasedYield**) and the actual yield (from **HistoricalReceipt.ReceivedQuantity**). This measure determines how accurate the posted yields have been and is used to determine whether an **ItemSource** is near, inside, or outside tolerance. The particular error measure used to calculate this value is determined by the value of **AutomatedYieldType.HistoricalYieldErrorMeasure**. The input to this calculation may be seen in the **ItemSourcePartSourceHistoricalReceipt** table. For a given **ItemSource**, the value of this field is the error measure between **ItemSourcePartSourceHistoricalReceipt.AdjustedPartSourceYield** and **ItemSourcePartSourceHistoricalReceipt.EffectiveHistoricalYield** for all records. **Note:** This calculation is not based on a limited time horizon. It uses all matching **HistoricalReceipt** records for which there is an effective **PartSource**. **Note:** If no error measure can be calculated (i.e. if there are no matching **HistoricalReceipt** records and effective **PartSource** records can be found), then the value of this field is -1.	Quantity	
CalcLeadTimeZone	The zone into which the lead time deviation for the **ItemSource** falls. For example, if 0 ≤ **NearToleranceThreshold** ≤ **OutsideToleranceThreshold**, then this field's value is equivalent to: • **None** —If **AutomatedLeadTimeType** = Null or **AutomatedLeadTimeType.ProcessingRule** = Ignore. • **Inside** —If **CalcHistoricalLeadTimeError** < **NearToleranceThreshold** . • **Near** —If **NearToleranceThreshold** ≤ **CalcHistoricalLeadTimeError** < **OutsideToleranceThreshold**. • **Outside** —If **OutsideToleranceThreshold** ≤ **CalcHistoricalLeadTimeError**. The value for this field is used in the calculation for **CalcUseLeadTimeChanges**.	Enum	

Table 6.155: ItemSource calculated (Mfg) fields

Field name	Description	Data type	Key
CalcProjectedLeadTimeCOV	The coefficient of variation of **ProjectedLeadTimes.FixedLeadTime** for this **ItemSource**. The coefficient of variation is defined as standard deviation divided by mean. **Note:** If the mean lead time is 0, then this field reports -1.	Quantity	
CalcProjectedYieldCOV	The coefficient of variation of **ProjectedYields.Yield** for this **ItemSource**. **Note:** If the mean projected yield is 0, then this field reports -1.	Quantity	
CalcUseLeadTimeChanges	Whether the values recommended in the **ProjectedLeadTime** table should be applied, either because they should be accepted automatically or because the user has accepted them manually. This field's value is equal to 'Y' when: • **CalcLeadTimeZone** ≠ None and **AcceptLeadTimeChanges** = Y, or • **CalcLeadTimeZone** = Inside and **InsideToleranceAction** = AutoAccept, or • **CalcLeadTimeZone** = Near and **NearToleranceAction** = AutoAccept, or • **CalcLeadTimeZone** = Outside and **NearToleranceAction** = AutoAccept.	Boolean	

Table 6.155: ItemSource calculated (Mfg) fields

Field name	Description	Data type	Key
CalcYieldZone	The zone (inside, near or outside tolerance) into which this ItemSource falls. If **AutomatedYieldType** = Null or **AutomatedYieldType.ProcessingRule**= Ignore or **CalcYieldUsageRule** = Ignore or **AutomatedYieldType.NearToleranceThreshold** > **AutomatedYieldType.OutsideToleranceThreshold**, or the **SelfHealingSupplyChain** license is not enabled, or the **CalcHistoricalYieldError** is < 0, then this field's value is None. Otherwise, assuming that 0 ≤ **AutomatedYieldType.NearToleranceThreshold** ≤ **AutomatedYieldType.OutsideToleranceThreshold**. Then, this field's value is: • Inside, if **CalcHistoricalYieldError** < **AutomatedYieldType.NearToleranceThreshold** • Near, if **AutomatedYieldType.NearToleranceThreshold** ≤ **CalcHistoricalYieldError** < **AutomatedYieldType.OutsideToleranceThreshold** • Outside, if **AutomatedYieldType.OutsideToleranceThreshold** ≤ **CalcHistoricalYieldError**	Boolean	
CalcUseYieldChanges	Whether the values recommended in the **ProjectedYield** table should be applied, either because they should be accepted automatically or because the user has accepted them manually. This field's value is equal to 'Y' when: • **CalcYieldZone** ≠ None and **AcceptYieldChanges** = Y, or • **CalcYieldZone** = Inside and **InsideToleranceAction** = AutoAccept, or • **CalcYieldZone** = Near and **NearToleranceAction** = AutoAccept, or • **CalcYieldZone** = Outside and **OutsideToleranceAction** = AutoAccept.	Boolean	
CalcYieldUsageRule	YieldUsage rule applicable to the **ItemSource**, calculated based on the **OrderPolicy.YieldUsage** value for all relevant **PartSources**. If an **OrderPolicy.YieldUsage** value is Ignore, the PartSource is considered to be invalid.	Boolean	

Table 6.155: ItemSource calculated (Mfg) fields

Field name	Description	Data type	Key
ItemSourceEffectivePartSources	A set of calculated **ItemSourceIntervalEffectivePartSource** records associated with this **ItemSource**. **Referenced table:** "ItemSourceIntervalEffectivePartSource table" on page 1088	Set	
ItemSourcePartSourceEffectivityIntervals	A set of calculated **ItemSourcePartSourceEffectivityInterval** records associated with this **ItemSource**. **Referenced table:** "ItemSourcePartSourceEffectivityInterval table" on page 1089	Set	
ItemSourcePartSourceHistoricalLeadTimes	A set of calculated **ItemSourcePartSourceHistoricalLeadTime** records associated with this **ItemSource**. **Referenced table:** "ItemSourcePartSourceHistoricalLeadTime table" on page 1089	Set	
ProjectedLeadTimes	A set of **ProjectedLeadTime** records associated with this **ItemSource**. **Referenced table:** "ProjectedLeadTime table" on page 505	Set	
ProjectedYields	A set of **ProjectedYield** records associated with this **ItemSource**. **Referenced table:** "ProjectedYield table" on page 506	Set	

ItemSourceFitOutput table

The ItemSourceFitOutput table supports the Self-Healing Supply Chain in RapidResponse.

This table shows the result of fitting the statistical model for forecast items in the Self-Healing Supply Chain process, such as lead time. This table contains output parameters associated with a particular forecast model along with details generated from fitting the model such as model constants and error measures.

Table 6.156: ItemSourceFitOutput (Mfg) fields

Field name	Description	Data type	Key
ItemSource	The ItemSource record associated with the current record.	Reference	Yes
ValueName	The name of the metric or characteristic being reported. For example, MeanAbsolutePercentageError (MAPE), which is the mean average percentage error calculated from the difference between the historically observed lead time or yields and the defined lead time or yields for each series. Reference to the "ItemSourceFitOutputValueName table" on page 388.	Reference	Yes
SelfHealingParameter	Indicates the Self-Healing use case where the information in the record is applicable (for example, LeadTime).	String (Enum)	Yes
AsOfDate	Date when the current record was generated or inserted.	Date	
ForecastModel	The forecast model used to generate this fitted output. Reference to the "ItemSourceFitOutputForecastModel table" on page 388.	Reference	
Class	The type of output parameter being reported in this record. For example, ErrorMeasure, to represent an error measure of fitted values against the input data points. Reference to the "ItemSourceFitOutputClass table" on page 387.	Reference	
ValueQuantity	Quantity associated with the metric. Only applicable if ValueType is Quantity.	Quantity	
ValueText	Text associated with the metric. Only applicable if ValueType is Text.	String	
ValueDate	Date associated with the metric. Only applicable if ValueType is Date.	Date	

ItemSourceFitOutputClass table

This table allows customers to define a custom class to use for forecast reporting by the Self-Healing Supply Chain. The class is defined at deployment time, and is referenced by the "ItemSourceFitOutput table" on page 386.

The ItemSourceFitOutputClass table supports the Self-Healing Supply Chain in RapidResponse.

Table 6.157: ItemSourceFitOutputClass (Mfg) fields

Field name	Description	Data type	Key
Value	The class of the record. For example, ErrorMeasure.	String	Yes
Description	An optional description of the referenced class. This value is not used by the Self-Healing Supply Chain.	String	

ItemSourceFitOutputForecastModel table

This table allows customers to have a custom forecast model used by the Self-Healing Supply Chain. The forecast model is defined at deployment time, and is referenced by the "ItemSourceFitOutput table" on page 386.

The ItemSourceFitOutputForecastModel table supports the Self-Healing Supply Chain in RapidResponse.

Table 6.158: ItemSourceFitOutputForecastModel (Mfg) fields

Field name	Description	Data type	Key
Value	The name of the forecast model to use.	String	Yes
Description	An optional description of the referenced value. This value is not used by the Self-Healing Supply Chain.	String	

ItemSourceFitOutputValueName table

This table allows customers to have a custom metric reported by the Self-Healing Supply Chain. The metric is defined at deployment time, and is referenced by the "ItemSourceFitOutput table" on page 386.

The ItemSourceFitOutputValueName table supports the Self-Healing Supply Chain in RapidResponse.

Table 6.159: ItemSourceFitOutputName (Mfg) fields

Field name	Description	Data type	Key
Value	The name of the metric or characteristic reported.	String	Yes
Description	An optional description of the referenced value. This value is not used by the Self-Healing Supply Chain.	String	

ItemSourceHistoricalSupplyCategory table

The **ItemSourceHistoricalSupplyCategory** table defines a set of **HistoricalSupplyCategory** records that are associated with an **ItemSource**. By default, **HistoricalSupplyActual** records from all **HistoricalSupplyCategory**s will be used by the Machine Learning service except those that are added in this table. The **ItemSourcePartSourceHistoricalLeadTime** calculated table will be filtered to exclude the **HistoricalSupplyActual** data for the **HistoricalSupplyCategory** records that are added in this table.

The **HistoricalSupplyActual** data is required as input to calculate the **ItemSourcePartSourceHistoricalLeadTime**.

This table was added in Version 2016.2, Service Update 23 and replaces the **ItemSource**.**HistoricalSupplyCategory** field.

Table 6.160: ItemSourceHistoricalSupplyCategory (Mfg) fields

Field name	Description	Data type	Key
HistoricalSupplyCategory	Defines the **HistoricalSupplyCategory** records to exclude from the historical supply data that will be sent to the Machine Learning server. Any records that are added will be ignored. If there are no records in the **ItemSourceHistoricalSupplyCategory** table, then all historical supply categories will be included and analyzed by the Machine Learning server. **Referenced table:** "HistoricalSupplyCategory table" on page 350	Reference	Yes

Job table

The **Job** table logs the status of the machine learning jobs used to calculate the **ProjectedLeadTime** and **ProjectedYield**, the **Cluster** and the **ClusterProperty** for the **ItemSource** that is configured for the Self-Healing Supply Chain.

This table was added in Version 2016.2, Service Update 21 and is unversioned.

Notes:

- This table is imported into the MLService namespace during the installation of the Self-Healing Supply Chain.
- The status of the machine learning jobs for the Self-Healing Supply Chain can be monitored with the **Job Status** worksheet in the **Self-Healing Job Status** workbook. The **Job Status** worksheet reports the status of the GetPrepData, Forecast, and Cluster jobs.

Table 6.161: Job (MLService) fields

Field name	Description	Data type	Key
Description	The job description. For example: "Forecasting is in progress".	String	
Id	The job Id created by the Machine Learning service.	String	
Name	The job name. The jobs are: • **GetPrepData** —Retrieves **HistoricalSupplyActual** and **HistoricalReceipt**data from **ItemSourcePartSourceHistoricalLeadTime** and **ItemSourcePartSourceHistoricalReceipt**. • **Forecast** —Performs the calculations for **ProjectedLeadTime** and **ProjectedYield**. • **Cluster** —Performs the calculations for **Cluster** and **ClusterProperty**.	String	
Status	The job status. The job statuses are: • **Started** • **In Progress** • **No Data** • **Completed** • **Failed** **Note:** After data cleansing, if there is no data for Forecast and Cluster, GetPrepData will first send a "No Data" status and then send a "Job Complete" status.	String	
Timestamp	The current timestamp in UTC format.	DateTime	
Type	The job type (User, Operational, etc.).	String	
UseCase	The use case for this job.	String	

KPI table

The KPI table identifies each unique key performance indicator in RapidResponse. Records in this table are referenced on values loaded into the HistoricalPartKPI table as discussed in "HistoricalPartKPI table" on page 343.

Table 6.162: KPI (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this key performance indicator.	String	
EnableUOM	Specifies whether this key performance indicator (KPI) has a unit associated with it and can be converted to other unit of measures. This field is not used by used by RapidResponse analytics and is intended to support writing certain worksheet column expressions. For example, it might be referenced in a conditional expression that determines whether a value in the **HistoricalPartKPI** table should have a unit of measure conversion applied or returned in its raw stored value. Valid values are: • **N**—the KPI does not have a unit/can't be converted. • **Y**—the KPI has a unit/can be converted.. **Default value**: N **Note**: This field was added in Version 2016.2, Service Update 30.	Boolean	
Value	A unique identifier for this key performance indicator.	String	Yes

Location table

The **Location** table identifies an inventory location within a warehouse.

Table 6.163: Location (Mfg) fields

Field name	Description	Data type	Key
Id	An identification code for the location within the warehouse (for example, stock room). **Default value**: blank string	String	Yes
Name	The name of the location. **Default value:** blank string	String	
StorageType	A reference to the storage type that is associated with this location. This field is in the **Solutions** namespace. **Referenced table**: Solutions::StorageType **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	
Warehouse	The warehouse where this location is. **Referenced table:** Warehouse **Reference Syntax:** Warehouse.Id, Warehouse.Site	Reference	Yes

Location table – calculated and set fields

The following describes the calculated and set fields in the Location table. For an introduction to this table, and descriptions of its input fields, see "Location table" on page 391.

Table 6.164: Calculated Location (Mfg) fields

Field name	Description	Data type	Key
OnHands	Set of OnHand records for this location.	Set	

LogisticsLocation table

This table identifies locations used for cross-docking, including airports, customs, ports, and other logistics locations. It supports mapping capabilities in RapidResponse.

This table was added in Version 2016.2, Service Update 9.

Table 6.165: LogisticsLocation (Mfg) fields

Name	Description	Data type	Key
Address	The street address of the logistics location. If this table is used to support maps, the address in this field determines the geocode that is used to mark the location on the map. To ensure that the correct geocode is used, follow the guidelines outlined in "Using address data to support maps" on page 1735.	String	
Description	A description of this logistics location.	String	
Email	Email address of the contact person for this logistics location.	String	
Id	A unique identifier for this logistics location.	String	Yes
Phone	The phone number of the contact person for this logistics location.	String	
ProcessingTime	Processing time at this location. This field is included for informational purposes. The units (eg. hours or days) are not specified in RapidResponse.	Quantity	

Table 6.165: LogisticsLocation (Mfg) fields

Name	Description	Data type	Key
Type	Type of logistics location. Valid values are: • Airport • Customs • Logistics • Port **Default value:** Logistics	String (Enum)	

LogisticsLocation table – calculated and set fields

The following describes the calculated and set fields in the LogisticsLocation table. For an introduction to this table, and descriptions of its input fields, see "LogisticsLocation table" on page 392.

Table 6.166: LogisticsLocation (Mfg) calculated fields

Name	Description	Data Type	Key
Airports	Set of Shipment records for which this LogisticsLocation is referenced in the Airport field.	Set	
Customss	Set of Shipment records for which this LogisticsLocation is referenced in the Customs field.	Set	
LogisticsLocations	Set of Carrier records for which this LogisticsLocation is referenced in the LogisticsLocation field.	Set	
Ports	Set of Shipment records for which this LogisticsLocation is referenced in the Ports field.	Set	

MEIOLeadTime table

The **MEIOLeadTime** table identifies each part for which variable lead time parameters should be generated for use in multi-echelon safety stock calculations (assuming the part belongs to a multi-echelon family).

Each record in this table is identified by its **PartSource** reference and also references a control table record that determines how parameters such as average lead time and standard deviation of lead time are calculated. Additionally, this table contains input fields that can be used to manually provide certain lead time parameters rather than having them calculated based on historical data.

Note: All MEIO calculations are expressed in Everyday calendar.

Table 6.167: MEIOLeadTime (Mfg) fields

Field name	Description	Data type	Key
Average	The known or estimated average of the part's historical lead time values. Typically used when there is insufficient historical supply data available to calculate the parameter. This field is only applicable when **Type.ProbabilityDistribution** is set to "Normal" and **Type.StandardDeviationRule** is set to either "Manual" or "CoefficientOfVariation". **Note:** This field is expressed in Everyday calendar.	Quantity	
Distribution Profile	A reference to a distribution profile containing discrete lead time values along with the probability associated with each. This field is only applicable when **Type.ProbabilityDistribution** is set to "Discrete". **Referenced table:** MEIOLeadTimeDistribution (Nullable)	Reference	
LeadTimeOutlier MovingAverage Window	The length of the moving average window for historical lead time. For more information, see the **SafetyStockItem.LeadTimeOutlierMovingAverageWindow** field. **Default value**: 3 **Note:** This field was added in Version 2016.2, Service Update 5.	Quantity	
LeadTimeOutlier Threshold	The threshold for detecting outliers in historical lead time. For more information, see the **SafetyStockItem.LeadTimeOutlierThreshold** field. **Default value**: 3 **Note:** This field was added in Version 2016.2, Service Update 5.	Quantity	
LeadTimeOutlier Type	A reference to the **OutlierType** that defines how to process outliers in historical lead time. To be reported in **MEIOHistoricalSupply**. **Default value**: Null **Referenced table**: OutlierType (Nullable) **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	

Table 6.167: MEIOLeadTime (Mfg) fields

Field name	Description	Data type	Key
LeadTimeSmoothingAfterIntervalCount	When smoothing historical lead time outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods forward over which to spread the values. This occurs after any backward spreading. For more information, see the **SafetyStockItem.LeadTimeSmoothingAfterIntervalCount** field. **Note:** This field was added in Version 2016.2, Service Update 5.		
LeadTimeSmoothinBeforeIntervalCount	When smoothing historical lead time outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods backward over which to spread the values. This occurs after any forward spreading. For more information, see the **SafetyStockItem.LeadTimeSmoothingBeforeIntervalCount** field. **Note:** This field was added in Version 2016.2, Service Update 5.		
PartSource	A reference to a part and source whose historical supplies should be used in calculating lead time parameters. A given historical supply for a part can be used in determining lead time parameters if the **HistoricalSupplyActual.Source** value matches the source returned by this reference. If a part in a multi-echelon family has multiple sources of supply, then one record should be added to this table for each effective part source that has variable/random lead time. **Referenced table:** PartSource	Reference	Yes
Standard Deviation	The known or estimated standard deviation of the part's historical lead time values. Typically used when there is insufficient historical supply data available to calculate the parameter. This field is only applicable when **Type.ProbabilityDistribution** is set to "Normal" and **Type.StandardDeviationRule** is set to either "Manual" or "CoefficientOfVariation". **Note:** This field is expressed in Everyday calendar.	Quantity	

Table 6.167: MEIOLeadTime (Mfg) fields

Field name	Description	Data type	Key
Type	A reference to the control setting that determines how lead time parameters are calculated and used for this part. For example, do its historical lead times follow a normal or discrete distribution. **Referenced table:** MEIOLeadTimeType	Reference	

MEIOLeadTime table – calculated and set fields

The following describes the calculated and set fields in the **MEIOLeadTime** table. For an introduction to this table, and descriptions of its input fields, see "MEIOLeadTime table" on page 393.

Table 6.168: Calculated MEIOLeadTime (Mfg) fields

Field name	Description	Data type	Key
CalcAverage	Average lead time for this part source as calculated based on all applicable input and control settings.	Quantity	
CalcStandard Deviation	Standard deviation of lead time for this part source as calculated based on all applicable input and control settings.	Quantity	
Historical Supplies	Set of historical supplies reported in the **MEIOHistoricalSupply** table that reference this record through the **Item** reference.	Set	

MEIOLeadTimeDistributionProfile table

The **MEIOLeadTimeDistributionProfile** table contains named lead time distribution profiles that can be used for parts in multi-echelon safety stock families.

These profiles are used in calculating the average and standard deviation of lead time values for items where lead time is modelled as having a discrete distribution (that is, **MEIOLeadTimeType.ProbabilityDistribution** is set to "Discrete"). The points that make up each profile consist of a lead time value and weight and are stored in records in the **MEIOLeadTimeDistributionProfilePoint** table.

Table 6.169: MEIOLeadTimeDistributionProfile (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this lead time distribution profile.	String	
Name	A unique name that identifies this lead time distribution profile.	String	Yes

MEIOLeadTimeDistributionProfile table – calculated and set fields

The following describes the calculated and set fields in the **MEIOLeadTimeDistributionProfile** table. For an introduction to this table, and descriptions of its input fields, see "MEIOLeadTimeDistributionProfile table" on page 396.

Table 6.170: Calculated MEIOLeadTimeDistributionProfile (Mfg) fields

Field name	Description	Data type	Key
MEIOLeadTimes	Set of MEIOLeadTime records containing lead time parameters that use this profile.	Set	
Points	Set of MEIOLeadTimeDistributionProfilePoint records containing the discrete lead time points that belong to this profile.	Set	

MEIOLeadTimeDistributionProfilePoint table

The **MEIOLeadTimeDistributionProfilePoint** table contains the discrete point values that make up a given lead time distribution profile (as defined in the referenced **MEIOLeadTimeDistributionProfile** table). Each record in this table specifies a lead time value along with a weight to assign to that value within its distribution profile.

For example, if the discrete lead times associated with a given profile is expected to be seven days for fifty-percent of the time, ten days for thirty-percent of the time, and fifteen days for twenty-percent of the time, then records similar to the following might be added:

Profile	Value	Weight
Profile1	7	5
Profile1	10	3
Profile1	15	2

Table 6.171: MEIOLeadTimeDistributionProfilePoint (Mfg) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this profile point.	String	Yes
Profile	A reference to the named lead time profile this point belongs to. **Referenced table:** MEIOLeadTimeDistributionProfile	Reference	Yes
Value	The lead time value associated with this profile point. This value should be expressed as either a number of days or as a multiplier used to determine lead time from the calculated average lead time (based on the **DistributionProfilePoint** setting on the applicable **MEIOLeadTimeType** record).	Quantity	
Weight	The weight to assign to this point value in the referenced distribution profile. The probability assigned to this point value is then determined by dividing this weight by the sum of all weights in the profile.	Quantity	

MEIOLeadTimeOutlier table

This table stores outlier adjustments for historical data associated with a MEIO stage. It is similar to **SafetyStockItemOutlier**, except this table references **MEIOLeadTime** instead of **SafetyStockItem**.

- Outliers are calculated based on the **MEIOLeadTime** table and the results are stored in the **MEIOLeadTimeOutlierResult** and the **MEIOLeadTimeOutlierResultSummary** tables.
- The LeadTime in **MEIOHistoricalSupply** will be corrected based on the data in this table.

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Field name	Description	Data type	Key
Adjustment	The quantity of the outlier adjustment. This can be negative value.	Quantity	
Date	The actual date associated with the historical supply record. For example, this might represent the date parts shipped from the dock. **Note:** Corresponds to **MEIOHistoricalSupply.Date**	Date	Yes
MEIOLeadTime	A reference to the **MEIOLeadTime** to which the outlier adjustment applies. **Referenced table**: MEIOLeadTime	Reference	Yes

Field name	Description	Data type	Key
Notes	Descriptive notes appended to this record.	Notes	
UseWhen Detecting Outliers	Specifies whether to apply or ignore the **SafetyStockItem** record before detecting outliers when calculating **MEIOLeadTimeOutlierResult**. Valid values are: • **Y**—The historical data series for the **SafetyStockItem** record is used to detect outliers. • **N**—The historical data series for the **SafetyStockItem** record is not used to detect outliers. **Default value**: Y **Note:** This field has the same functionality as **ForecastItemParametersOutlier.ProcessingRule** = "All".	Boolean	

Metric table

The **Metric** table stores metric definitions for metrics used in live lenses and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.172: Metric (Solutions) fields

Field name	Description	Data type	Key
Metric	The name of the metric.	String	Yes
ValueType	The type of metric: Quantity or Money.	String (Enum)	

Metric table – calculated and set fields

The following describes the calculated and set fields in the Metric table. For an introduction to this table, and descriptions of its input fields, see "Metric table" on page 399.

Table 6.173: Metric (Solutions) fields

Field name	Description	Data type	Key
ConstraintMetrics	The set of constraint and metric definition combinations that belong to this metric definition. **Referenced table:** PartMetric	Set	
PartMetrics	The set of part and metric definition combinations that belong to this metric definition. **Referenced table:** PartMetric	Set	
SiteMetrics	The set of site and metric definition combinations that belong to this metric definition. **Referenced table:** SiteMetric	Set	

MetricAsOf table

The **MetricAsOf** table stores the date that the metric calculations are generated on for the PartMetric, SiteMetric, and ConstraintMetric tables.

This table was added in Version 2016.2, Service Update 19.

Table 6.174: MetricAsOf(Solutions) fields

Field name	Description	Data type	Key
Date	The date that metric calculations are generated.	Date	Yes

MetricAsOf table – calculated and set fields

The following describes the calculated and set fields in the MetricAsOf table. For an introduction to this table, and descriptions of its input fields, see "MetricAsOf table" on page 400.

Table 6.175: MetricAsOf (Solutions) fields

Field name	Description	Data type	Key
ConstraintMetrics	The set of constraint and metric definition combinations that belong to the stored metric calculations. **Referenced table:** ConstraintMetric	Set	
PartMetrics	The set of part and metric definition combinations that belong to the stored metric calculations. **Referenced table:** PartMetric	Set	

Table 6.175: MetricAsOf (Solutions) fields

Field name	Description	Data type	Key
SiteMetrics	The set of site and metric definition combinations that belong to the stored metric calculations. **Referenced table:** SiteMetric	Set	

Model table

A model represents a variant of a part. A model uses the same part number, but is different from other models of the same part. In classic APICS terms, there should be a different part number for each model or part combination. However, primarily to simplify bill of material maintenance, the RapidResponse data model uses a single part number. Supply of one model cannot be used to satisfy demand for a different model.

This table is only available for use if you have enabled Model-Unit Effectivity.

The **Model** table supports the Model and Unit-Effectivity analytics. It contains the names of every model used by any part.

The default value in the **Model** table of 'None' (and the only model entry for RapidResponse if Model-Unit Effectivity has not been enabled) is special and indicates that this is not really a model. For example, when a part is not netted by model (for example, the value of Part.MuePoolNettingType.ModelRule is 'Ignore'), all supplies and demands for the part are set to use the Default value and therefore not really treated as models.

Table 6.176: Model (Mfg) fields

Field name	Description	Data type	Key
Description	Additional information about the model. **Default value**: blank string	String	
Value	The name of a model. Note that the same model name can be applied to different parts, and may have different meanings for each part with which it is used. **Default value**: None **Note:** The first model name loaded into this table is called None. This is the default model.	String	Yes

Model table – calculated and set fields

The following describes the calculated and set fields in the Model table. For an introduction to this table, and descriptions of its input fields, see "Model table " on page 401.

Table 6.177: Calculated Model (Mfg) fields

Field name	Description	Data type	Key
Allocations	Set of Allocation records associated with this model.	Set	
Baseline PlannedOrders	Set of BaselinePlannedOrder records associated with this model.	Set	
BillOfMaterials	Set of BillOfMaterial records associated with this model.	Set	
BOMTypes FlowedFrom	Set of BOMType records associated with this model.	Set	
CRPOperations	Set of CRPOperation records associated with this model.	Set	
Independent Demand	Set of IndependentDemand records associated with this model.	Set	
Inventory Transfers	Set of InventoryTransfer records associated with this model (through the Model reference).	Set	
Inventory TransferModels	Set of InventoryTransfer records associated with this model (through the TransferModel reference).	Set	
OnHands	Set of OnHand records associated with this model.	Set	
Operations	Set of Operation records associated with this model.	Set	
Scheduled Receipts	Set of ScheduledReceipt records associated with this model.	Set	
Supply Allocations	Set of SupplyAllocation records associated with this model.	Set	

Notification table

The Notification table defines the information related to notifications for process instances and activities. Each record in this table contains details about a notification, such as which process instance and activity it is defined for, as well as the event or criteria that triggers the notification.

The information in this table should be modified only in the RapidResponse dialog boxes designated for creating and editing process notifications.

Table 6.178: Notification (ProcOrch) fields

Field name	Description	Data type	Key
Active	Determines whether the notification is active or not	Boolean	
Activity	The ID of the activity that the notification applies to. **Referenced table:** Activity	Reference	
BccRecipients	Semicolon delimited list of additional notification recipients.	String	
CcRecipients	Semicolon delimited list of additional notification recipients.	String	
NotifyOwner	Determines if the process owner is notified when the notification is run.	Boolean	
NotifyPerformers	Determines which activity performers are notified. Valid values are: • **None**—no performers are notified • **OnlyAffected**—only performers matching the notification criteria are notified • **All**—all performers are notified	Enum	
ProcessInstance	The name of the process instance this notification is for. **Referenced table**: ProcessInstance	Reference	
ProcessInstance Only	Determines if the notification is only for the process or for all activities.	Boolean	
ToRecipients	Semicolon delimited list of additional notification recipients.	String	

Table 6.178: Notification (ProcOrch) fields

Field name	Description	Data type	Key
TriggerEvent	The event that triggers the notification. Valid values are (where n is an integer defined in the TriggerEventOffset field): • **AboutToStart**—n days before ExpectedStartDate if the activity status is NotStarted • **AboutToFinish**—n days before ExpectedFinishDate if the activity status is InProgress • **ActuallyStarted**—After ActualStartDate if the activity status is InProgress • **ActuallyFinished**—After ActualFinishDate if the activity status is Finished • **LateStart**—n days after ExpectedFinishDate if the activity status is InProgress • **LateFinish**—n days after ExpectedFinishDate if the activity status is not Finished • **StartedEarly**—n days before ExpectedStartDate if the activity status is InProgress • **FinishedEarly**—n days before ExpectedFinishDate if the activity status is Finished	Enum	
TriggerEvent Offset	The number of days offset before the event will trigger the notification. For example, the notification might be triggered 2 days before an activity is supposed to start.	Integer	

OnHand table

The **OnHand** table stores the amount of inventory and other details for a part in a particular location within a warehouse. When on hand inventory is allocated to demands for pegging, high priority demands are protected. However, if there is no priority, then on hand for a part is allocated by the following fields in this table:

- ExpiryDate (earliest first)
- Date (earliest first)
- Unit Cost (least cost first)
- Quantity (smallest first)
- Location.Id and Location.Warehouse

Table 6.179: OnHand (Mfg) fields

Field name	Description	Data type	Key
BaseKey	Can optionally be used to uniquely identify each OnHand record. **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	String	
Date	Indicates the date at which inventory was received. This field can affect when given on hand inventory is allocated (as described above), and is also used in determining the appropriate currency conversion rate (if any) to apply in cost calculations. **Default value:** Past	Date	
DispositionCode	An inventory code used to determine how the part is handled in an excess or obsolete situation. This field is in the **Solutions** namespace.	String	
ExpiryDate	The date on which this inventory expires (is no longer usable). **Default value:** Undefined	Date	
ExpiryStartDate	The date when this on hand inventory begins to expire. The date provided here is rolled up the product structure through parts that use a "dependent" expiry type. For assembly parts with a **PartType.ExpiryRule** setting of "DependentStart", the earliest rolled up **ExpiryStartDate** from a component supply is then used to start expiry date calculations for the assembly supply. Thus, this field should generally be populated on supply of components that use an **ExpiryType.ProcessingRule** of "Limiting". **Note:** This field was added in Version 2016.2, Service Update 6.	Date	
Location	The stocking or bin location where the inventory is located. **Referenced table:** Location **Reference Syntax:** Location.Warehouse.Id, Location.Id **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	

Table 6.179: OnHand (Mfg) fields

Field name	Description	Data type	Key
Model	The model that corresponds to this material. Use the default model (for example, the None model) if no model is associated with this inventory record. This field is ignored unless Part.MUEPoolNettingType.ModelRule is 'Net'. This field supports Model and Unit- Effectivity analytics. For implementations not using these analytics, all supplies and demands are treated as being the None model. **Referenced table:** Model	Reference	
Part	The part number of the inventory. **Referenced table:** Part **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	
Pool	The pool that owns this material. Use the default pool (Unpooled pool) if no pool is associated with this inventory record. This field is ignored unless is 'Net'. **Referenced table:** Pool **Note:** If the Pool capability is not enabled, all supplies and demands are treated as being in the Unpooled pool.	Reference	
Quantity	The amount in stock. **Default value**: 0	Quantity	
StartUnit	The unit number corresponding to the first item in this inventory. The quantity should be one if Part.MUEPoolNettingType.UnitRule is 'Net' (the entire quantity is treated with the same StartUnit number). **Default value**: -1 **Note:** This field supports the Model-Effectivity, and Unit-Effectivity analytics, and is ignored if Model-Unit Effectivity is not enabled.	Integer	
Type	A value that determines whether the stock or inventory is considered in netting. This field references the **OnHandType** table. For information about this table, see "OnHandType table" on page 768. **Referenced table:** OnHandType	Reference	
UnitCost	Unit cost for parts in this inventory store. If UnitCost is zero, then inventory is valued at Part.StdUnitCost. Negative values are treated as zero. **Default value:** 0	Money	

OnHand table – calculated and set fields

The following describes the calculated and set fields in the OnHand table. For an introduction to this table, and descriptions of its input fields, see "OnHand table" on page 404.

Table 6.180: Calculated OnHand (Mfg) fields

Field name	Description	Data type	Key
ExpiringQuantity	The amount of this nettable inventory that will not be used before the **ExpiryDate**.	Date	

Operation table

The Operation table identifies each of the operations required in the production of a part. These records describe the order of the operations, the duration of the operations and any differences (or overrides) in capacity available at the work center during the processing of this operation.

This table is maintained for backwards compatibility and the **CRPOperation** table should be used for new purposes. The **CRPOperation** table contains most of the fields found on this table as well as additional fields to support functionality such as parallel operations and tear down time.

Table 6.181: Operation (Mfg) fields

Field name	Description	Data type	Key
BatchQuantity	The number of units that must be completed by this operation before the next operation in the sequence can begins processing the order. If set to zero, then the **Type.BatchQuantityRule** setting for the next operation in the sequence determines how that value is interpreted (either the entire operation or just a single unit in the operation must finish before moving to the next operation in the sequence). For more information about batch quantities, see "Overlapping operations" on page 1527. **Default value**: 0	Quantity	
BaseKey	Can optionally be used to uniquely identify each Operation record. **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	String	
Description	The description of the operation. **Default value**: blank string	String	

Table 6.181: Operation (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	Date this record becomes effective. All operations in the routing are selected for effectivity based on the start date of the work order (scheduled receipt or planned order). That is, the entire sequence of operations is determined at the start of the order. **Default value**: Undefined	Date	
EffectiveOutDate	Date this record expires. All operations in the routing are selected for effectivity based on the start date of the work order (scheduled receipt or planned order). That is, the entire sequence of operations is determined at the start of the order. **Default value**: Undefined	Date	
EndUnit	The end of the unit range to which this operation record applies. The end unit is either included in the range or excluded depending upon the values of the OperationType EffectivityRule and DependentUnitRule. EndUnit is ignored unless MUERule is **Unit** or **ModelUnit**. Every supply will be seen as being before EndUnit if EndUnit is <0. **Default value**: -1	Integer	
Model	The model to which the operation record applies. The Model value is ignored unless the MUERule for the OperationType associated with this record is **Model** or **ModelUnit**. **Referenced table:** Model	Reference	
Number	The operation number for the operation. **Default value**: 0	Integer	
QueueTime Override	When scheduling this operation use this value for queue time instead of the queue time specified by the work center. If this field is negative, the work center value will be used. This is the number of hours to wait before the operation setup has been done and is used only if it is greater than or equal to zero. **Default value**: -1	Quantity	
Routing	The routing code for the operation. **Referenced table:** Routing **Reference Syntax**: Routing.Id, Routing.Site **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	

Table 6.181: Operation (Mfg) fields

Field name	Description	Data type	Key
RunTime	The Capacity calculations take the Operation RunTime, adjust it according to the CRP unit of measure, and then multiply by the order quantity to determine the number of hours load from each order. The MachineRunTimeUOM determines the processing of the run time. For example, multiply the operation run time by the order quantity to determine the load on the machine. This value must be greater than or equal to zero. **Default value**: 0	Quantity	
RunTimeUOM	Operation run time unit of measure. The RunTimeUOM determines how to convert the run time into standard hours per piece. **Referenced table:** CRPUnitOfMeasure	Reference	
SetupHours	Work center setup standard hours per lot for each machine. This value must be greater than or equal to zero and is the number of load hours to setup one machine required for this operation. **Default value**: 0	Quantity	
StartUnit	The beginning of the unit range to which this operation record applies. The start unit is either included in the range or excluded, depending upon the values of the OperationType EffectivityRule and DependentUnitRule. StartUnit is ignored unless MUERule is **Unit** or **ModelUnit**. Every supply will be seen as being after StartUnit if StartUnit is <0. **Default value**: -1	Integer	
TransitTime	Transit time for the operation. This value must be greater than or equal to zero and is the number of hours or days to move between operations. If the OperationType.TransitTimeSequenceRule is BeforeOperation this constitutes the time it takes to move from the previous operation to this operation. If the OperationType.TransitTimeSequenceRule is AfterOperation this is the time it takes to move from this operation to the next operation. **Default value**: 0	Quantity	
Type	The operation type. It is a reference to the OperationType table through the OperationType.Value field. **Referenced table:** OperationType	Reference	

Table 6.181: Operation (Mfg) fields

Field name	Description	Data type	Key
WaitTime Override	When scheduling this operation use this value for wait time instead of the wait time specified by the work center. If this field is negative, the effective work center value will be used. This is the number of hours to wait after the operation run has completed. It is used only if it is greater than or equal to zero. **Default value**: -1	Quantity	
WorkCenter	The WorkCenter (resource) for the operation. **Referenced table:** WorkCenter **Note:** If you have upgraded to Version 2015.3 and enabled record uniqueness on this table prior to the upgrade, this field is a key field. For more information, see "Key fields" on page 141.	Reference	

Operation table – calculated and set fields

The following describes the calculated and set fields in the Operation table. For an introduction to this table, and descriptions of its input fields, see "Operation table " on page 407.

Table 6.182: Calculated Operation (Mfg) fields

Field name	Description	Data type	Key
FirstEffectiveDate	Represents the adjusted value for the EffectiveInDate taking into account Type.EffectivityRule (for example, InclusiveExclusive). This field is always inclusive. If the record will never be effective, the dates will be Undefined.	Date	
FirstEffectiveUnit	The first unit to which this operation record applies. If Unit effectivity is not being used, this field is set to –1.	Integer	
LastEffectiveDate	Represents the adjusted value for the EffectiveOutDate taking into account Type.EffectivityRule (for example, InclusiveExclusive). This field is always inclusive. If the record will never be effective, the dates will be Undefined.	Date	
LastEffectiveUnit	The last unit to which this operation record applies. If Unit effectivity is not being used, set this field to –1.	Integer	

OperationSequence table

The **OperationSequence** table is used to identify different sequences of operations within a given routing. Each routing should have exactly one standard sequence of operations and, optionally, one or

more parallel operations that branch off the standard sequence. As such, each record in this table is uniquely identified by the combination of its **Id** and a reference to the **Routing** table. The **OperationSequence** table is then referenced by the **CRPOperation** table and thereby defines the routing and sequence to which each of the operations defined in that table belong.

Note: Prior to Version 2014.4, operations were defined in the **Operation** table which directly referenced the **Routing** table (and did not support parallel sequences). The **Operations** table still exists and any resources (such as, workbooks) based on it will continue to work, however, once a **Routing** record is referenced by a value in the **OperationSequence** table, all Operation records that reference that Routing are ignored.

Table 6.183: OperationSequence (Mfg) fields

Field name	Description	Data type	Key
BranchOperation	Used for parallel sequences only. Specifies the **CRPOperation.Id** value of the operation (in the routing's standard sequence) before which this parallel sequence should branch. If a matching **CRPOperation.Id** value cannot be found, or if this field is left blank, then the parallel sequence branches before the beginning of the standard sequence.	String	
Description	A description of this operation sequence.	String	
Id	An identifier for this operation sequence. Together with the referenced **Routing** record, this uniquely identifies a given sequence of operations.	String	Yes
ReturnOperation	Used for parallel sequences only. Specifies the **CRPOperation.Id** value of the operation (in the routing's standard sequence) after which this parallel sequence should return. If a matching **CRPOperation.Id** value cannot be found, or if this field is left blank, then the parallel sequence returns at the end of the standard sequence.	String	
Routing	The routing to which the sequence defined by this record belongs to. **Referenced table:** Routing **Reference Syntax**: Routing.Id, Routing.Site	Reference	Yes

Table 6.183: OperationSequence (Mfg) fields

Field name	Description	Data type	Key
Type	A reference to the control setting that defines processing rules for this operation sequence. For example, a **Type.ProcessingRule** value of "Standard" indicates the standard sequence of operations for a routing, while a value of "Parallel" indicates a parallel sequence. Typically, a **Routing** record should be referenced by exactly one standard operation sequence and, optionally, one or more parallel sequences. Note that if a routing does not have a standard sequence defined, all of its parallel sequences are ignored. Also, if more than one standard sequence is defined for a routing, RapidResponse designates one of them as the standard sequence and treats the others as parallel sequences. **Referenced table:** OperationSequenceType	Reference	

OperationSequence table – calculated and set fields

The following describes the calculated and set fields in the **OperationSequence** table. For an introduction to this table, and descriptions of its input fields, see "OperationSequence table" on page 410.

Table 6.184: Calculated OperationSequence (Mfg) fields

Field name	Description	Data type	Key
EffectiveBranch Operation	For parallel sequences, this returns the operation from which the parallel sequence branched off the standard sequence. Typically, this equates to the value specified in the **BranchOperation** field. However, in cases where **BranchOperation** is not populated or contains a value that does not exist in **CRPOperation.Id**, this returns the first operation in the standard sequence. Null if this record defines a standard sequence. **Referenced table:** CRPOperation	Reference	
EffectiveReturn Operation	For parallel sequences, this returns the standard sequence operation into which the parallel sequence branch returned. Typically, this is the value specified in the **ReturnOperation** field. However, in cases where **ReturnOperation** is not populated or contains a value that does not exist in **CRPOperation.Id**, this returns the last operation in the standard sequence. Null if this record defines a standard sequence. **Referenced table:** CRPOperation	Reference	
CRPOperations	Set of CRPOperation records associated with this operation sequence.	Set	

OriginatingFile table

The **OriginatingFile** table identifies data files used to provide RapidResponse input tables with data. It is used to help track records in RapidResponse back to their originating file, and supports data update and data persistence capabilities. This table is maintained internally by RapidResponse, and its records cannot be modified by RapidResponse users.

Table 6.185: OriginatingFile (Mfg) fields

Field name	Description	Data type	Key
CreationTime	Date and time when this data file was first encountered during a data import or update.	DateTime	
LastSeenTime	Date and time when this data file was last encountered during a data import or update.	DateTime	
Name	The name of a data file used to provide data to a RapidResponse input table. This table also contains records used to indicate data that was added internally by RapidResponse and now through a dedicated data file. For example, records with values of "User", "Historical", and "Auto-created" might typically be seen.	String	
RemovalTime	Date and time, if any, at which this file was no longer found.	DateTime	
Source	A reference to the OriginatingSource record that identifies the data source this file belongs to. **Referenced table:** OriginatingSource	Reference	

OriginatingSource table

The OriginatingSource table identifies data sources used to provide RapidResponse with input data. It is used to help track records in RapidResponse back to their originating file, and supports data update and data persistence capabilities. This table is maintained internally by RapidResponse, and its records cannot be modified by RapidResponse users.

Table 6.186: OriginatingSource (Mfg) fields

Field name	Description	Data type	Key
Name	The name of a data source that provided RapidResponse with data. This table also contains a record with a value of "RapidResponse Internal" that is used to indicate records added directly by users or through the automatic record creation process.	String	Yes
OriginatingFiles	Set of OriginatingFile records which reference this OriginatingSource value.	Set	

Note: The origins of records in tables that contain vector data are not recorded in this table.

OverrideMajorChangeover table

The OverrideMajorChangeover table describes the amount of constraint required to setup and cleanup when changing between production groups in a production wheel cycle.

Each record describes the changeover from one specific production group to another group. For example, the record below describes the amount of constraint required to changeover from producing Group A to producing Group B.

Field	Value
FromProductionGroup	Group A
ToProductionGroup	Group B
Cleanup	10
Setup	5

Another record is required to describe the amount of constraint required to changeover from producing Group B to producing Group A.

Field	Value
FromProductionGroup	Group B
ToProductionGroup	Group A
Cleanup	7
Setup	20

When the values are defined for a record in this table, these values override the values specified for MajorCleanup and MajorSetup in the ProductionGroup table. For more information about overriding default constraint amounts for changeovers, see "Changeover in production" on page 1697.

This table was added in Service Update 1902.

Table 6.187: OverrideMajorChangeover (Mfg) fields

Field name	Description	Data type	Key
Cleanup	The amount of constraint required to clean up after producing the group specified in FromProductionGroup. Measured in constraint units.	Quantity	
FromProductionGroup	A reference to the production group that production is changing over from. **Referenced table:** ProductionGroup	Reference	Yes
Setup	The amount of constraint required to set up production for the group specified in ToProductionGroup. Measured in constraint units.	Quantity	
ToProductionGroup	A reference to the production group that production is changing over to. **Referenced table:** ProductionGroup	Reference	Yes

Part table

The **Part** table identifies a unique part and site. A record in this table includes data such as: name, description, price, and cost data.

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
ABCCode	Classification based on Activity Based Costing (ABC) analysis. ABC is simply a classification of parts, usually by the extended cost of annual usage (quantity used * unit cost). **Referenced table:** ABCCode	Reference	
AfterForecast Interval	Number of calendar intervals after an actual demand date in which that demand can consume forecast (exclusive). The value specified here is taken in terms of either **PlanningCalendars.ForecastCalendar** or **PlanningCalendars.SecondCalendar** intervals as set by the control setting in the **PartType.ForecastConsumptionCalendar** field (ForecastCalendar is the default) For example, 1 means forecast can only be consumed up to the end of the current forecast or second calendar interval, 2 means forecast can be consumed up to the end of the next forecast or second calendar interval, and so on. For more information, see "Forecast consumption" on page 1431. **Default value**: 0	Quantity	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
Allocation Multiple	If the part is used in fair-share or equal-share allocation, the allocated quantities will be rounded to the nearest multiple of this number. For example, if AllocationMultiple = 1.0, the quantity allocated for the part will always be rounded to the nearest whole number, such as 2.0 or 5.0 If this value is less than or equal to 0.0, the allocated quantity is not rounded. The last allocation of a fractional supply or an allocation to a fractional demand is also not rounded. **Default value**: 1.0	Quantity	
AvailableRule	Control whether normal capable-to-promise rules are used when calculating availability for the part. If this field is left null, then the part's available rule setting comes from **PartType.AvailableRule**. **Referenced table:** AvailableRule (Nullable)	Reference	
AverageQty	Average order quantity used when the value of the SafetyStockPolicy.SafetyStockRule field is OrderPointAverage, OrderPointOver, or OrderPointUnder. If SafetyStockPolicy.SafetyStockRule is OrderPointAverage, this value is the amount ordered to replenish inventory. **Default value**: 0	Quantity	
AverageSelling Price	The average selling price for this part. This can be zero if the part will never have independent demand (for example, not sold). **Default value**: 0	Money	
BeforeForecast Interval	Number of calendar intervals before an actual demand date in which that demand can consume forecast (inclusive). The value specified here is taken in terms of either **PlanningCalendars.ForecastCalendar** or **PlanningCalendars.SecondCalendar** intervals as set by the control setting in the **PartType.ForecastConsumptionCalendar** field (ForecastCalendar is the default) For example, a value of 0 means forecast can be consumed starting from the beginning of the current forecast or second calendar interval containing the demand date, 1 means forecast can be consumed from the beginning of the previous forecast or second calendar interval, 2 means forecast can be consumed from the beginning of the second forecast or second calendar interval before the actual demand, and so on. For more information, see "Forecast consumption" on page 1431. **Default value**: 0	Quantity	
BuyerCode	A text code that identifies the buyer for this part. **Referenced table:** BuyerCode **Reference Syntax:** BuyerCode.Value	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
CarryingCost	The cost of holding inventory. **Default value**: 0	Money	
ConstraintShare Fence	Within each **OrderPriority** level, the number of **ConstraintAllocationCalendar** intervals out from **RunDate** in which to include this part's demands for use in the fair sharing of limited constraint availability amongst parts whose sources use common constraints. For example, if set to "0", then only demands due before **RunDate** (past due) would be eligible for constraint sharing. The window defined by this field is only applicable if the parts' **Type.ConstraintShareRule** setting is "WithinFence" and only for demands where the **OrderPriority.ConstraintAllocationRule** is "FairShare". **Default value**: - 1 (this indicates that constraint sharing logic, if enabled, applies throughout the part's entire planning horizon).	Integer	
DaysSupply Policy	A reference to the days of supply policy for this part. This policy defines time-phased (short-term and long-term) parameters used in days of supply calculations. For example, the number of days or periods of accumulated demand that planned orders should satisfy in both the short-term and long-term horizon. **Referenced table:** DaysSupplyPolicy (Nullable) **Note:** This reference was added in Version 2016.2. If Null and **PartType.UseDaysSupply** is set to "Y", then days of supply logic can still be used based on legacy fields in the **Part**, **PartType**, and **PlanningCalendar**s tables. However, those fields only support a single set of days of supply settings applicable throughout the horizon (are not time-phased). For more information about days of supply, see "Days of Supply calculations" on page 1362.	Reference	
DDMRPRule	A reference field linking to DDMRPRule control table. **Referenced table:**DDMRPRule (Nullable) **Note:** This field was added in Version 2016.2.	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
DemandTime Fence	Number of units of time after the **Part.PlanningCalendar.RunDate.FirstDate** to establish a fence for controlling the use of unconsumed forecast demands. Within the demand time fence, any unconsumed forecast is ignored and not planned for by Netting (does not contribute to the total demand picture). However, any unconsumed forecast outside of the time fence does contribute to total demand and will be planned for by Netting. Values in this field should be expressed in **Part.PlanningCalendar.TimeFenceCalendar** intervals. If this is set to o (zero), then the demand time fence is placed at the RunDate and all unconsumed forecast drives demand (in cases where the RunDate does not fall on a TimeFenceCalendar marker, it set to the previous marker in the calendar). Note also that the demand time fence applies only to SalesForecast, and not to ProductionForecast. **Default value**: 0	Quantity	
Description	A description of the part. **Default value**: blank string	String	
Distribution PlanningRule	Controls whether Distribution Planning within CTP calculations can be used for the part. If the Distribution Planning rule is enabled, when a part exceeds the maximum inventory level at the sourcing site, this triggers a request for the finished goods to be transferred to a destination site before it is required. **Referenced table:** DistributionPlanningRule (Nullable) **Note:** This field was added in Version 2016.2.	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
ExcessFence	The number of **PlanningCalendars.TimeUnit** intervals out from **RunDate** that defines the window for including a substitute part's demands and supplies when determining excess. This is applicable both to parts used as global substitutes (if their **Type.ExcessRule** is 'GlobalFence'), as well as parts used as BOM-level substitutes (if their **Type.ExcessRule** is 'Fence'). Within the excess window set by this field, existing supply (typically, on hand and scheduled receipts) of a substitute component is first used to satisfy any of its own demand. If any supply remains afterwards, then that "excess" can be used to satisfy demand from the parts or components for which it is substitutable. Outside of this window, supply of a substitute part or component is used to satisfy demand on a first-come, first-served basis regardless of whether the demand is its own or that of another part or component for which it is substitutable. Note that setting this field to a negative value (for example, -1) extends the excess window to the end of the planning horizon. This ensures that supplies of a substitute part or component are only ever eligible for use in substitution if they are deemed to be excess. Note also setting this field to 0 (zero) means that only demands and supplies due on **PlanningCalendars.RunDate.FirstDate** are used in determining excess. The end of the excess window is reported in the **ExcessFenceDate** field. Default value: 0	Integer	
ExpiryType	Controls how a component is used in determining the expiry date of any of its assemblies that have a **PartType.ExpiryRule** setting of "DependentStart". **Referenced table:** ExpiryType (Nullable)	Reference	
IncrementalRule	Controls whether incremental availability calculations can be performed for orders for this part. **Referenced table**: IncrementalRule	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
Inventory HoldingRate	The inventory holding rate for this part (applicable to parts used in multi-echelon safety stock calculations). Together with the setting in the **MultiEchelonSafetyStockRule.CostRule** field, this is used to calculate the unit holding cost for the part. Unit holding costs are required as input by multi-echelon calculations that recommend strategic safety stock placement based on minimizing total inventory holding costs across a network of parts. This value should be expressed as a fractional percentage (typically, a value between 0 - 1). For example, if **CostRule** is set to "StdUnitCost", **StdUnitCost** is "200", and this field is ".2", then the unit holding cost for the part is $40 (20% of 200). Note that all negative values are treated the same as 0 (zero) and indicates there is no cost to holding the part in inventory. Also, values greater than 1 are allowed and imply the cost of holding the part in inventory is greater than the cost of the part itself. **Default value**: .2	Quantity	
LeadTimeAdjust	An additional term used to adjust (typically, increase) the total lead-time. Typically, this field is not used and left at zero. However, when dealing with certain enterprise systems where a fixed lead time of zero means there is actually a one-day difference between Due and StartDate, this field can be set to 1 for all affected parts **Default Value:** 0	Quantity	
MaterialCost	The cost of material associated in producing the part. **Default value**: 0 **Note:** This field is obsolete.	Money	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
MinimumShelf Life	This field specifies the number of **PlanningCalendars.ExpiryCalendar** intervals that are added to either an independent demand's or consensus forecast's **DueDate** when calculating its **EarliestExpiryDate**. The EarliestExpiryDate is the date before which supplies must not expire if they are to be allotted to the demand. For example, if PlanningCalendars.ExpiryCalendar is set to "Month" and this value is set to "3", then an independent demand for this part with a DueDate of July 1, 2009 would have its EarliestExpiryDate set to October 1, 2009 (that is, any supply expiring before October 1, 2009 could not be used to satisfy the demand). For independent demands, this can be overridden on a demand basis by providing a value in the **IndependentDemand.MinimumShelfLife.** For consensus forecast demands, this can be overridden on a part customer basis by providing a value in the **MinimumShelfLife** field on either of the **PartCustomerTimePhasedAttributes** or **PartCustomer** tables. **Default value:** 0 **Note:** If a value of 0 is specified, this indicates that the **EarliestExpiryDate** should be set to the demand due date.	Integer	
MUEPool NettingType	The MUEPoolNetting type for this part. The MUEPoolNettingType value controls the model, pool and unit netting for a part. **Referenced table:** MUEPoolNettingType	Reference	
MultiEchelon SafetyStockRule	A value associated with rules that set how this part is used in multi-echelon safety stock calculations. For example, it controls whether a given part in a multi-echelon family is eligible to hold safety stock as inventory. **Referenced table:** MultiEchelonSafetyStockRule (Nullable)	Reference	
MultiLevel SearchRule	A reference to a control table value that is used to enable or disable multi-level search logic, and contains related options determining how multi-level search logic is performed when enabled. Multi-level search logic can be used to modify standard part sourcing and substitution logic to consider component availabilities when making sourcing decisions, and also modifies standard supply allotment logic to consider sibling component allotments when allocating a component part's supplies to the demands that require it. For more information, see "Multi-level search logic" on page 1863. **Referenced table:** MultiLevelSearchRule (Nullable) **Note:** If this reference is left Null, then multi-level search logic is always disabled for the part.	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
Name	Part number or item number.	String	Yes
NextUnit	In conjunction with Part.MUEPoolNettingType.UnitRule field, this field controls how StartUnit is to be propagated from this part. It also determines the unit number to use as the StartUnit for planned orders and forecast to be exploded from this part. The following rules apply: • If NextUnit is < 0 and the part is not netting by unit, then that value is copied into **StartUnit** for all planned supplies. • If UnitRule = **Block**, the StartUnit of the planned supply is set to the larger of NextUnit or the StartUnit field on the demand order. • If UnitRule = **Increment,** and NextUnit >= 0, the next unit will be used to create the StartUnit of the planned order. Note that NextUnit is ignored for phantom parts. • If UnitRule = **Fixed**, every planned supply StartUnit will be set to **NextUnit**. • If UnitRule = **Net**, and Model-Unit Effectivity is enabled, the unit on planned orders is copied from the demand record. Otherwise, StartUnit is set to -1. **Default value**: -1	Integer	
NumberOf DaysSupply	• If **PartType.UseDaysSupply** is set to '**Y**' and the **DaysSupplyPolicy** reference is Null, then when creating a new planned order this field defines the period to cover requirements for as follows: • If **PartType.DaysSupplyRule** = 'FromDemand' or 'FromSupply', indicates the number of units (as defined by **Part.PlanningCalendars.TimeUnits)** to cover requirements for**.** • If **PartType.DaysSupplyRule** = 'ByPeriod' or 'ByPeriodWithPriority', indicates the number of calendar intervals (as defined by **Part.PlanningCalendars. PeriodSupplyInterval**) to cover requirements for. The quantity of the order is reduced if the resulting order exceeds **PartSource.MaximumQty**. **Default value:** 0 **Note:** A value of "0" or less in this field is treated as if it were set to "1"; that is, supply is planned to satisfy one day/interval of demand. **Note:** If the **DaysSupplyPolicy** reference is populated, then the reference record in the **DaysSupplyPolicy** table determine days of supply settings, and this field is ignored. For more information, see "Days of Supply calculations" on page 1362.	Quantity	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
PartClass	The name of the part class, if any, associated with this part. For example, a part class might be finished goods, work in progress, or raw materials. These classes are used by certain application to assign planning, safety stock, and replenishment strategies. **Referenced table:** Class (Nullable) **Note:** This field is in the Solutions namespace.	Reference	
PercentSafety IntervalCount	This value corresponds to the SafetyStockPolicy.PercentSafetyInterval field. If this value is less than one, then one is used.	Integer	
PercentSafety Percent	This is used in the PercentOfDemand (and FractionOfDemand) safety calculation. The safety quantity for a period is determined by multiplying the total gross demand in that period by this percentage value (which may be a fraction or a percent, based on the SafetyStockPolicy.SafetyStockQuantityRule field). If this value is less than 0, then 0 is assumed.	Quantity	
PickPackTime	The amount of time required to pull an order for this part from inventory and prepare it for shipment to the customer. This value is used in calculating planned ship and receipt dates, as well as available ship and receipt dates, on **IndependentDemand** records. Note that if this field is set to a negative value, the **DefaultPickPackTime** field on a given **DeliveryRoute** record is used instead.	Integer	
PlannerCode	A text code that identifies the planner responsible for this part. **Referenced table:** PlannerCode **Reference Syntax:** PlannerCode.Value	Reference	
Planning Calendars	A value associated with processing rules that define the Part's Calendar behavior. This field references the PlanningCalendars table. **Referenced table:** PlanningCalendars **Reference Syntax:** PlanningCalendars.Value	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
Primary Substitution Sequence	For parts configured as the primary part in a global part substitution relationship, this field can be used to define a relative preference between input supplies of that primary part and its substitutes. In global substitution logic, this value is compared against the **AlternatePart.Sequence** value(s) associated with the part's substitute(s). Lower values indicate a higher substitution preference. That is, if this value is less than or equal to the A**lternatePart.Sequence** value for a substitute, then this part's input supplies are preferred. Otherwise, if the **AlternatePart.Sequence** value for a given substitute is less than this value, then input supplies of the substitute are used up first (and planned orders will never be generated on the substitute to satisfy this part's demands). **Default value:** 0 **Note:** The specific usage of this field further depends on the primary part's **PartType.SubstitutionPreference** setting. For more information, see "Define when part substitutes are used" on page 1553.	Integer	
ProductFamily	A reference to the product family (if any) that the part belongs to. Product families can be used to enable aggregate level reporting for groups of related or similar parts. **Referenced table:** ProductFamily (Nullable)	Reference	
ProductGroup1	This field can be used to describe a product grouping with which a part is associated. For example, this could be a product family. **Default value**: blank string	String	
ProductGroup2	This field can be used to describe a different product grouping than ProductGroup1. For example, this could be a product line. **Default value**: blank string	String	
RangeOf CoverageBuffer	This field specifies a quantity to add to calculated range of coverage safety stock levels to account for some amount of expected demand variability. It is also used along with the value in the **RangeOfCoverageThresholdFactor** field to determine if the new calculated safety stock level should replace the value from the previous range of coverage interval. Applicable only to those parts where **SafetyStockPolicy.SafetyStockQuantityRule** is set to either "RangeOfCoverage", "RangeOfCoverageLeadTime" or "RangeOfCoverageTimePhased". **Default value**: 0	Quantity	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
ReferencePart	Depending on how your company is using the reference part functionality, this column displays: • the name of this part as known throughout your company (a common part number or equivalent part). • the part name according to the naming conventions used in a different division. • a name used to group a product line the part belongs to. • the same value as the Name field • blank if this field is not populated in your implementation of RapidResponse **Referenced table**: ReferencePart	Reference	
SafetyLeadTime	Number of days to offset the demand due date when determining when supply should be planned to cover that demand. This is intended to protect against fluctuations in supply and demand and will impact planned order due dates as well as EffLeadTime and CumLeadTime calculations. This value should be expressed in either the part's **PlanningCalendars.TimeUnits** calendar or the Everyday/Gregorian calendar as determined by the part's **PartType.SafetyLeadTimeCalendarRule** setting. Note that a value provided in this field is only used when the **SafetyLeadTimeProfile** reference is Null. Usage of this field also depends on the setting in the **PartType.UseSafetyLeadTime** field, with additional **PartType** settings available to specify whether safety lead time should be used for specific types of demands. Also, if safety lead time should be applied to demand request dates and/or only used in cases where doing so won't make other demands worse off, then use of settings on the **OrderPriority.TwoDatePlanningRule** field should be considered. **Default value**: 0 **Note:** SafetyLeadTime is not included in NeedDate calculations. **Note:** SafetyLeadTime is ignored and not used to offset due dates for safety stock demands.	Quantity	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
SafetyLeadTime Profile	A reference to a safety lead time profile that defines a series of time-phased safety lead time values for the part. If this reference if Null, then the value in the **SafetyLeadTime** field defines a single safety lead time value that is always effective for the part. Note that usage of this field depends on the setting in the **PartType.UseSafetyLeadTime** field, with additional **PartType** settings available to specify whether safety lead time should be used for specific types of demands. Also, if safety lead time should be applied to demand request dates and/or only used in cases where doing so won't make other demands worse off, then use of settings on the **OrderPriority.TwoDatePlanningRule** field should be considered. **Referenced table:** SafetyLeadTimeProfile (Nullable) **Note:** This field was added in Version 2016.2.	Reference	
SafetyStock Policy	A reference to the safety stock policies applicable to this part. For example, how are safety stock levels determined and under what conditions are those levels maintained. If this reference is left Null, then various fields on the **PartType** and **PlanningCalendars** tables (identical to those on **SafetyStockPolicy**) are used instead. **Referenced table:** SafetyStockPolicy (Nullable)	Reference	Yes
SafetyStockQty	A quantity of stock in inventory to protect against fluctuations in demand or supply. **Default value**: 0	Quantity	
Site	The site associated with this part. This field is a reference to the Value field on the Site table. **Referenced table:** Site **Note:** A value is required in this field for both Single-Site and Multi-Site installations.	Reference	Yes
SourceRule	If there are multiple active part sources defined for a part, this reference defines rules outlining how RapidResponse chooses the source to use when satisfying a planned requirement. **Referenced table:** SourceRule	Reference	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
SpreadForecast Interval	Can be used to specify the number of forecast calendar intervals over which forecast spreading logic is applied to those forecast demands where the **DemandType.SpreadRule** value is "Spread". If left at the default value of zero (0) or set to a negative value, then the spreading logic is applied to demands throughout the entire planning horizon. However if set to a positive value, then the forecast spreading logic is only applied to those demands that are due within that number of **Part.PlanningCalendar.ForecastCalendar** intervals out from the RunDate. **Default value**: 0	Integer	
StdUnitCost	The cost to produce this part. Includes all material, labor, and overhead costs. **Default value**: 0	Money	
Substitution Tolerance	When faced with demand for the primary part in a substitute part relationship, this is the number of **PlanningCalendars.SubstitutionTolerance Calendar** intervals out from the demand date to look for existing supply of one part before supply from other parts is considered. For example, if the primary part's **PrimarySubstitutionSequence** value is less than or equal to the **AlternatePart.Sequence** for a given substitute, this defines the number of **SubstitutionToleranceCalendar** intervals out from the demand date to look for existing input supply of the primary part before considering supply from the substitute. However, if the primary part's **PrimarySubstitutionSequence** value is strictly greater than the **AlternatePart.Sequence** for a given substitute, then this is instead the number of **SubstitutionToleranceCalendar** intervals out from the primary part's demand date to look for (use up) existing supply of the substitute before supply from the primary part is considered. **Default value:** 0	Integer	
SupplyShare Fence	Within each **OrderPriority** level, the number of **AllocationCalendar** intervals out from the **RunDate** in which this part's demands are eligible for the fair or equal sharing of limited supplies. For example, if set to "0", then only demands due before **RunDate** (past due) would be eligible for supply sharing. The window defined by this field is only applicable if the parts' **Type**.**SupplyShareRule** setting is "WithinFence", and only for demands where the **OrderPriority.AllocationRule** is set to either "FairShare" or "EqualShare". **Default value**: - 1 (this indicates that supply sharing logic, if enabled, applies throughout the part's entire planning horizon).	Integer	

Table 6.188: Part (Mfg) fields

Field name	Description	Data type	Key
Type	A value that is associated with processing rules that defines the part's behavior. This field references the PartType table. For more information about this table, see "PartType table" on page 809. **Referenced table:** PartType	Reference	
UnitOfMeasure	The stock and purchased units in which quantities of this part are measured. **Referenced table:** UnitOfMeasure	Reference	
Unsatisfied Demand Tolerance	Specifies the part's tolerance level for late or unsatisfied demands. This field should be expressed in **PlanningCalendars.ForecastCalendar** intervals and is used in both partial planning (Netting) and on-time reallocation logic (CTP). If using partial planning logic (**DemandType.PlanningRule** is set to "Partial"), demands or demand portions that cannot be planned within the lateness tolerance are left unsatisfied and not planned. In other words, this field indicates the number of forecast intervals after it's due date upon which a demand should be given up on if supply cannot be planned for it. If using on-time reallocation logic (**PartType.OnTimeAllocationRule** is set to "ImproveWithTolerance"), supply quantities that are allocated to demands late and beyond tolerance can be reallocated. The goal of the reallocation process is improving the on-time quantity on late demands. But these reallocations also have the potential to make the available date on some late demands worse (later) than they otherwise would be. Thus, this field indicates the number of forecast intervals past its due date at which a late demand's overall available date is less important than improving the on-time quantity of late demands. **Default value**: 0 (indicates that the part's lateness tolerance is at the end of the current forecast calendar interval; for example, if the forecast calendar was set to Everyday this would imply any late demand is considered beyond tolerance). **Note:** This field was added in Version 2016.2, and its usage extended in Version 2016.2, Service Update 27.	Integer	

Part table – calculated and set fields

The following describes the calculated and set fields in the Part table. For an introduction to this table, and descriptions of its input fields, see "Part table " on page 415.

Note: When pool-based netting flow-to-common logic is enabled, those calculated fields that report a sum of something (for example, **TotalDemand** or **TotalSupply**) are summed across all pools whereas those fields that report the first of something (for example, **FirstShortageDate** or **FirstNeededSupplyDate**) return the first value across and from just one of those pools.

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
AlternatePrimary Part	The primary part this part can be used interchangeably with. **Referenced table**: Part	Reference	
AverageFirst YearInProcess	Calculated to show work in process (WIP) for supplies starting earlier than RunDate + 365 days. Calculated as 1/365 * the sum over scheduled receipts and planned orders based on SupplyType.Source. If SupplyType.Source is: • **Buy**—If **SupplyType.BuyInProcessRule** is set to **IncludeDockToStock**, this field is calculated as StockUnitQuantity * (ReschedDate - ReschedDockDate). Otherwise, it is set to zero. • **Make**—this field is calculated as: StockUnitQuantity * (ReschedDate - CalcStartDate). • **Transfer**—this field is calculated as: StockUnitQuantity * (ReschedDate - (ShipDate - PreShipLT using source time units)).	Quantity	
AverageFirst YearInventory	Average inventory level at the end of the day over the first 365 days. Useful for inventory turns calculations.	Quantity	
Constrained Demand	Total demand to be satisfied up to RunDate + 365 days. Uses AvailableDate for independent demands and SynchronizedDate for dependent requirements.	Quantity	
Constrained External Demand	The total units of external demand with AvailableDate before RunDate + 365. (excludes intersite demand).	Quantity	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
ConstrainedIn Process	Projects the work in process (units of supply) between the time an order is started (shipped for transfer) and received into inventory. This calculation is based on supplies with AvailableStartDate earlier than RunDate + 365 days, and is influenced by the value of SupplyType.Source as shown in the following list: • **Buy**—If **SupplyType.BuyInProcessRule** is set to IncludeDockToStock, this field is calculated as StockUnitQuantity * (AvailableDate - AvailableDockDate). Otherwise, set to zero. • **Make**—this field is calculated as StockUnitQuantity * (AvailableDate - AvailableStartDate). • **Transfer**—this field is calculated as StockUnitQuantity * (AvailableDate - (AvailableShipDate - PreShipLT using source time units)).	Quantity	
Constrained IntersiteDemand	The total demand which acts like independent demand from another site with AvailableDate before RunDate + 365 days.	Quantity	
Constrained Inventory	Projects average daily inventory (unit) balance based on supply AvailableDate and demand SynchronizedDate (greater of EffDueDate and AvailableDate for independent demands).	Quantity	
CumLeadTime	The time from ordering all the components, to the lowest level of the structure, until the current part can be used to satisfy demand. This calculation is done only within a part's site. For multi-site cum lead time calculations (that is, calculations that consider transfer part sources), use the **MultiSiteCumLeadTime** field. For more information about lead time calculations, see "Lead time calculations" on page 1307. **Note:** Any lead time associated with phantom assemblies will not be included in the calculation of this value. **Note:** If using time-phased lead time, then the lead times effective for the primary part source at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used in this calculation.	Quantity	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
DaysSupply FenceDate	The date fence that indicates the transition point between short-term and long-term days of supply settings. That is, before this date the short-term days of supply settings are used, and on or after this date the long-term days of supply settings are used. This field is only populated if the part has a non-Null reference in the **DaysSupplyPolicy** field. **Note:** This field was added in Version 2016.2.	Date	
DecoupledLeadTime	This calculated field contains the multi-site cumulative lead time between buffer points. The calculation will follow the same logic as multi-site cumulative lead time, but the calculation will stop at any decouple points. **Note:** This field was added in Version 2016.2. **Note:** Only primary part source is considered for this calculation. DecoupledLeadTime calculation for make parts: EffectiveLeadTime + Max (NON-Decoupling-Point component's DLT) DecoupledLeadTime calculation if transfer part is DDMRP part: EffectiveTransferLeadTime DecoupledLeadTime calculation if transfer part is regular part: EffectiveTransferLeadTime + TransferPart's DLT	Integer	
DTFDate	The date of the demand time fence. Calculated from **RunDate** by adding the number of intervals specified in **DemandTimeFence** (based on the **PlanningCalendars.TimeFenceCalendar**). If set to 0, then this demand time fence is placed at **RunDate** (or if the RunDate does not fall on a TimeFenceCalendar marker, than it set to the previous marker).	Date	
EffectiveOrder PointDateRule	Indicates the **OrderPointDateRule** setting that is effective for this part. Valid values are: • Due • Start The effective value comes from the corresponding field on the **SafetyStockPolicy** reference (if not Null) or else the **PartType** reference. Refer to the field description on either of those tables for more information.	String (Enum)	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
EffectiveOrder PointIncrease	Indicates the **OrderPointIncrease** setting that is effective for this part. Valid values are: • Calculate • Ignore • Sum • SumOffset The effective value comes from the corresponding field on the **SafetyStockPolicy** reference (if not Null) or else the **PartType** reference. Refer to the field description on either of those tables for more information.	String (Enum)	
EffectivePercent SafetyInterval	The **PercentSafetyInterval** calendar that is effective for this part. This returns the calendar value referenced through **SafetyStockPolicy.PercentSafetyInterval** (if those references are not Null) or else **PlanningCalendars.PercentSafetyInterval**. Refer to the field description on either of those tables for more information. **Referenced table:** Calendar	Reference	
EffectiveRange OfCoverage	The **RangeOfCoverage** profile that is effective for this part. This returns the range of coverage profile value that is referenced through the **SafetyStockPolicy.RangeOfCoverage** field (if those references are not Null) or else **PlanningCalendars.RangeOfCoverage**. Refer to the field description on either of those tables for more information. **Referenced table:** RangeOfCoverage	Reference	
EffectiveSafety StockDateRule	Indicates the **SafetyStockDateRule** setting that is effective for this part. Valid values are: • FirstDemand • FirstDemandOrLeadTime • FirstDemandOrRunDate • FirstDemandOrPast • LeadTime • Past • RunDate The effective value comes from the corresponding field on the **SafetyStockPolicy** reference (if not Null) or else the **PartType** reference. Refer to the field description on either of those tables for more information.	String (Enum)	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
EffectiveSafety StockQuantityRule	Indicates the **SafetyStockQuantityRule** setting that is effective for this part. Valid values are: • FixedQuantity • FractionOfDemand • PercentOfDemand • RangeOfCoverage • RangeOfCoverageLeadTime • TimePhasedQuantity • TimePhasedQuantityToLatest The effective value comes from the corresponding field on the **SafetyStockPolicy** reference (if not Null) or else the **PartType** reference. Refer to the field description on either of those tables for more information.	String (Enum)	
EffectiveSafety StockRule	Indicates the **SafetyStockRule** setting that is effective for this part. Valid values are: • Always • Ignore • IfDemand • IfActivity • OrderPointAverage • OrderPointOver • OrderPointUnder • Soft The effective value comes from the corresponding field on the **SafetyStockPolicy** reference (if not Null) or else the **PartType** reference. Refer to the field description on either of those tables for more information.	String (Enum)	
EffLeadTime Adjust	Lead time adjustment used based upon the **LeadTimeAdjust** field and the Order Policy values.	Quantity	
EffSafetyLead Time	The effective safety lead-time value for this part. Returns the value in **SafetyLeadTime** unless either **PartType.UseSafetyLeadTime** is set to "N" or **SafetyStockPolicy.SafetyStockRule** is set to one of "OrderPointAverage", "OrderPointUnder", or "OrderPointOver" in which case it returns zero (0).	Quantity	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
EndingSafety Stock	The safety stock level in effect at the end of the planning horizon. This is the latest safety stock level calculated for this part by RapidResponse. For information about how RapidResponse determines safety stock levels, see "Safety stock calculations" on page 1402. For order point parts (that is, parts where **SafetyStockPolicy.SafetyStockRule** is either 'OrderPointUnder', 'OrderPointOver, or 'OrderPointAverage), this field returns the latest order point threshold calculated for the part.	Quantity	
ExcessFenceDate	The date that marks the end of the excess window applicable to either global or BOM-level part substitution. Within this window, a substitute part or component's current supplies will be used to satisfy all of its own demand before that of any part it is a substitute for (thus, only excess supply is available for use in substitution). Beyond this date, a substitute part's current supplies will satisfy the demands that require them first, regardless of whether the demands are its own or come from another part. Calculated based on the **Part.ExcessFence** and **PartType.ExcessRule** fields.	Date	
FirstNeeded SupplyDate	The date when the on hand inventory of the part will run out, that is the day when a supply other than on hand is needed to satisfy demand by the scheduled due date.	Date	
FirstShortage Date	The earliest date that inventory will drop below zero (not safety stock) if no rescheduling or new orders are generated.	Date	
FirstYearExternal Demand	The total units of external demand due before RunDate + 365 (within the next year). Includes independent demand and unconsumed forecast for real parts.	Quantity	
FirstYearIntersite Demand	The total demand which acts like independent demand from another site with DueDate before RunDate + 365.	Quantity	
IsPhantom	A flag to tell if the part will blow through (ignore processing of) all its dependent demand. 1 means all dependent demand is bypassing, 0 means any dependent demand from bills that are not marked as blow through will be processed.	Integer	
LowLevelCode	Lowest level at which this part is used as a component in any BOM structure. 0 indicates a top level part.	Integer	
MEIOFamily	The part that is the head of this part's MEIO family.	Reference	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
MultiSiteCum LeadTime	The amount of time required from ordering all components, down to the lowest level of the product structure, until the part can be used to satisfy demand. This field uses a calculation similar to the **CumLeadTime** field, but it also follows product structures across site boundaries. That is, if a product structure contains a part with a primary supply type of "transfer", then lead time associated with the transfer part and its components is included in the value reported in this field. Because this field sums together different lead time values without specifying the calendar to be used in the calculation, it is possible that the different lead time values summed when a product structure crosses site boundaries may not all use the same calendar. Therefore, depending on the calendars used at different sites, this field may not always return the expected result. For more information about lead time, see "Lead time calculations" on page 1307. **NOTE**: If using time-phased lead time), then the lead times effective for the primary part source at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used in this calculation.	Quantity	
MultiSiteLow LevelCode	The level of the longest path from the part, considering all bill of material structures including transfer parts, to a top level part. Top level parts have a level of 0.	Quantity	
PrimaryPart Source	Reference to the primary PartSource record. The primary record is found as the first PartSource record after sorting by: • If (Part.PlanningCalendars.RunDate.FirstDate >=PartSource.FirstEffectiveDate AND Part.PlanningCalendars.RunDate.FirstDate <= PartSource.LastEffectiveDate) 0 1 • PartSource.Priority (if Ascending) or -1*PartSource.Priority (if Descending) • PartSource.Target (descending) • PartSource.FirstEffectiveDate • PartSource.Source.Supplier.ID PrimaryPartSource = <NULL> if there is no PartSource record for the part. For information about part source selection, see "Planned orders" on page 1343. **Referenced table:** PartSource	Reference	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
PrimarySupply Type	The value of **PrimaryPartSource.Type.PlannedOrderSupplyType**. This is a calculated reference provided to simplify finding whether a part is: • Make • Buy • Transfer **Referenced table:** SupplyType	Reference	
TotalAllocations	Total quantity of allocations used in netting.	Quantity	
TotalByProduct Supply	Total quantity of supply for this part that was generated as a by-product residual to the production of a primary product.	Quantity	
TotalCoProduct Supply	Total quantity of supply for this part that was generated as a co-product alongside the production of a primary product.	Quantity	
TotalDemand	Total quantity of demand processed by netting.	Quantity	
TotalEffPlanned Order	The total of the expected quantity to receive into inventory of all planned orders for the part. This calculation handles unit of measure (UOM) conversion, and scrap/yield.	Quantity	
TotalEff Scheduled Receipt	The total of the expected quantity to receive into inventory of all scheduled receipts for the part. This calculation handles unit of measure (UOM) conversion, and scrap/yield.	Quantity	
TotalFirstYear Demand	Total of demand that must be shipped in the first 365 days. Useful for inventory turns calculations.	Quantity	
TotalForecast	The total of all forecast records.	Quantity	
Total Independent Demand	Total quantity of independent demand used in netting. This does not include forecasted independent demands. That is, IndependentDemand records where **Order.Type.ProcessingRule** is set to "SalesForecast" are excluded from this calculation.	Quantity	
TotalIndependent DemandCount	The number of independent demand records that are reflected in the TotalIndependentDemand field (for example, records that were not ignored in MRP processing).	Quantity	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
TotalNeeded Supply	Total quantity from input supply (including on hand, inventory transfers, and scheduled receipts) that are needed to meet demand. This value is in inventory units, and is not shrunk in consideration of the yield factor. This summarization is faster than doing a summarized query on the Supply table.	Quantity	
TotalNettable OnHand	Total quantity of OnHand for the part that is available to meet demand. This summarization is faster than doing a summarized query on the OnHand table.	Quantity	
TotalNew Allocations	Total quantity of NewAllocations used in netting.	Quantity	
TotalNon Nettable OnHand	Total quantity of OnHand for the part that is not available for satisfying demand.	Quantity	
TotalPlanned Allocations	Total quantity of PlannedAllocations used in netting.	Quantity	
TotalPlanned Order	Total quantity of planned orders being requested. This value is in inventory units, and is not shrunk in consideration of the yield factor. This summarization is faster than doing a summarized query on the planned order table.	Quantity	
TotalPlanned OrderCount	The total number of PlannedOrder records that are reflected in the TotalPlannedOrder field (for example, the number of planned orders requested).	Quantity	
TotalSupply	Total quantity from ScheduledReceipt records that are available to meet demand.This value is in inventory units, and is not shrunk in consideration of the yield factor. This summarization is faster than doing a summarized query on the Supply table.	Quantity	
TotalSupply Count	The total number of ScheduledReceipt records that are reflected in the TotalSupply field (for example, the number of scheduled receipts available to meet demand).	Quantity	
Total Unconsumed Forecast	The amount of forecast that is not covered by actual sales demand.	Quantity	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
Validation Differences RollUpSum	Sum of ValidationDifferencesThisLevel for this part and all of its components, including all part sources. This field supports data validation and is intended for internal use only.	Integer	
Validation Differences RollUpCount	Count of parts below and including this one that have validation differences. This field supports data validation and is intended for internal use only.	Integer	
Validation Differences ThisLevel	Count of all dates on which the sum of PlannedOrder quantities differs from the sum of BaselinePlannedOrder quantities. This field supports data validation and is intended for internal use only.	Integer	
Activity	Set of Activity records.	Set	
Allocations	Set of Allocation records belonging to this part.	Set	
Allotment Override Details	Set of AllotmentOverrideDetail records that reserve supply of this component part.	Set	
AlternateParts	A set of parts from the AlternatePart table that can be substituted for this part in production or planning calculations.	Set	
Alternate Routings	A set of alternate Routing records that are used for informational purposes only at this time.	Set	
AssumptionBy Parts	Set of assumptions associated with this part. **Referenced table:** AssumptionByPart	Set	
AvailableTo Promise	Set of AvailableToPromise (ATP). ATP is the uncommitted portion of Inventory and scheduled receipts (for example, not satisfying actual demand).	Set	
BaselinePlanned Orders	Set of BaselinePlannedOrders records for this part.	Set	
BlowThrough NewAllocations	Set of BlowThroughNewAllocation records bypassing the netting of this part.	Set	
BlowThrough Planned Allocations	Set of BlowThroughPlannedAllocation records bypassing the netting of this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
CoByProduct Supplies	Set of co-product and by-product supplies in the CoByProductSupply table for this part.	Set	
Collaborative Forecasts	Set of CollaborativeForecast records that reference this Part value.	Set	
Components	Set of BillOfMaterial records where this part is the assembly. This represents the set of BOM records defining the components that go into making this part.	Set	
Consensus Forecast	Set of ConsensusForecast records for this part.	Set	
Consensus ForecastDetails	Set of ConsensusForecastDetail records for this part.	Set	
CostOfGoods Solds	Set of CostOfGoodsSold records belonging to this part.	Set	
CostRollUps	Set of CostRollUp records belonging to this part.	Set	
CTPActivity	Set of CTPActivity records associated with this part.	Set	
CTPCoByProduct Supplies	Set of CTPCoByProductSupply records for this part.	Set	
CTPForecast CostInfo	Set of CTPForecastCostInfo records associated with this part.	Set	
CTPForecasts	Set of CTPForecast records associated with this part.	Set	
CTPPlanned Orders	Set of CTPPlannedOrder records associated with this part.	Set	
CTPSubstitute Supplies	Set of CTPPSubstituteSupply records associated with this part.	Set	
CustomerPart Prices	Set of CustomerPrice records containing cost details for this part.	Set	
Demand Exceptions	Set of DemandExceptions (unsatisfied demands due to expiry) for this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
Demands	Set of Demand. Demand is the consolidation of Dependent and Independent Demands.	Set	
Dependent Consensus Forecasts	Set of DependentConsensusForecast records for this part.	Set	
Dependent Demands	Set of DependentDemand. DependentDemand is the consolidation of Allocations and the New and Planned Allocations coming into the part.	Set	
Direct Components	Set of DirectComponent records for this part.	Set	
FamilyMEIO Parts	Set of SafetyStockPart records found under a multi-echelon family with this part as the head.	Set	
FeatureMaps	Set of FeatureMap records configured for this part.	Set	
FlatBillDowns	Set of FlatBillDown records for this part (any components under this part).	Set	
FlatBillUps	Set of FlatBillUp records for this part (any assemblies where this part is used)	Set	
FlatComponents	Set of FlatComponent records for this part.	Set	
FlatConstraints	Set of FlatConstraint records for this part.	Set	
Forecast	Set of Forecast records for this part.	Set	
Forecast Consumption	Set of ForecastConsumption records for this part.	Set	
HistoricalPart KPIs	Set of HistoricalPartKPI records for this part.	Set	
IDByFeatures	Set of IndependentDemandByFeature records associated with this part.	Set	
IDByFeature Summary	Set of IndependentDemandFeatureSummary records associated with this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
Independent Demands	Set of IndependentDemand records belonging to this part.	Set	
InProcess	Set of InProcess records for this part.	Set	
Inventory Analysis	Set of InventoryAnalysis records pertaining to this part.	Set	
InventoryCTP Analysis	Set of InventoryCTPAnalysis records pertaining to this part.	Set	
Inventory Sourcing Transfers	Set of InventoryTransfer records where this part is the transfer part.	Set	
Inventory Summarys	Set of InventorySummary records for this part.	Set	
Inventory Transfers	Set of InventoryTransfer records for this part.	Set	
LoadFLPPlanneds	Set of LoadFLPPlanned records pertaining to this part.	Set	
Maximum Inventory Results	Set of MaximumInventoryResults records pertaining to this part.	Set	
MEIODriverParts	Set of MEIODriverPart records that ultimately brought this part into a multi-echelon family.	Set	
MEIOFamilies	Set of MEIOFamilyResult records for which this part is the head.	Set	
MEIOParent Parts	Set of MEIOParentPart records that brought this part into a multi-echelon family.	Set	
MEIOStageLinks	Set of MEIOStageLink records associated with this part.	Set	
MEIOStageTime PhasedResults	Set of MEIOStageTimePhasedResult records associated with this part.	Set	
MEIOStage Results	Set of MEIOStageResult records associated with this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
MEIOStages	Set of MEIOStage records associated with this part.	Set	
MUECumLeads	Set of MUECumLead records associated with this part.	Set	
MUEPool Validations	Set of Part_MUEPoolValidation records that define all model, unit, and pool combinations for a part with data validation differences. This field supports data validation and is intended for internal use only.	Set	
OnHands	Set of OnHand records belonging to this part.	Set	
OriginalParts	A set of AlternatePart records this part can be substituted for in production or planning calculations.	Set	
PartCustomers	Set of PartCustomer records associating this part with a customer.	Set	
PartSolutions	Set of PartSolution records belonging to this part.	Set	
PartSources	Set of PartSource records belonging to this part.	Set	
PartSuppliers	Set of PartSupplier records associating this part with a supplier.	Set	
PeriodsOf Coverage	Set of PeriodsOfCoverage records belonging to this part.	Set	
PeriodsOfCTP Coverage	Set of PeriodsOfCTPCoverage records belonging to this part.	Set	
PlannedOrders	Set of PlannedOrder recommendations for new supply orders to be generated.	Set	
PoolMap Overrides	Set of PoolMapOverride records for this part.	Set	
RangeOf Coverage PartOverrides	Set of RangeOfCoveragePartOverride records for this part.	Set	
SafetyStock Bounds	Set of SafetyStockTimePhasedBounds records for this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
SafetyStockItem Mappings	Set of SafetyStockItemMapping records for this part.	Set	
SafetyStock Items	Set of SafetyStockItem records configured for this part.	Set	
SafetyStockParts	Set of calculated SafetyStockPart records calculated for this part.	Set	
SafetyStock Results	Set of SafetyStockResult records (recommendations) for this part.	Set	
SafetyStockTime PhasedResults	Set of SafetyStockTimePhasedResult records (recommendations) for this part.	Set	
Scheduled Receipts	Set of ScheduledReceipt records belonging to this part.	Set	
ShopKanbans	Set of ShopKanban records for the part. **Referenced table**:ShopKanban	Set	
Substitute Supplies	Set of SubstituteSupply records where this part is reported as receiving supply from a substitute.	Set	
Supplies	Set of Supply. Supply is the consolidation of ScheduledReceipts and PlannedOrders.	Set	
Supply Allocations	Set of SupplyAllocation records belonging to this part.	Set	
SupplyDemands	Set of SupplyDemand records belonging to this part.	Set	
TaskLinks	Set of TaskPartLink records associating this part with project tasks.	Set	
TimePhased Safetys	Set of TimePhasedSafety records belonging to this part.	Set	
TransferPart Sources	Set of PartSource records where this part is supplied as a transfer.	Set	
UOM Conversions	Set of PartUOMConversion table records associated with this part.	Set	

Table 6.189: Calculated Part (Mfg) fields

Field name	Description	Data type	Key
ValidatePlanned Orders	Set of ValidatePlannedOrder table records associated with this part.	Set	
WhereConsumed ForForecasts	Set of WhereConsumedForForecast records belonging to the part.	Set	
WhereConsumed ForSupplies	Set of WhereConsumedForSupply records belonging to the part.	Set	
Where Consumeds	Set of WhereConsumed table records for the part.	Set	
WhereConsumed ToHighVolume Orders	Set of WhereConsumedToHighVolumeOrder records belonging to this part.	Set	
WhereUsed	Set of BillOfMaterial records where this part is the Component. This represents the set of BOM records defining the Assemblies where this part is used.	Set	

PartCustomer table

The **PartCustomer** table supports Sales and Operations Planning in RapidResponse, and should be integrated during the initial deployment. This table identifies each unique part and customer combination provided in historical data and then used in forecast generation. It also contains certain attributes (such as, priority) to be applied across all Forecast records generated from consensus forecast for a given part and customer combination.

In cases where time-phased part customer attributes are required, the **PartCustomerTimePhasedAttributes** table can be used. Also, if it is sufficient to generate the forecast for an aggregate grouping of a given part's customers, then configuration records can be added to the **AggregatePartCustomer** table.

Table 6.190: PartCustomer (Mfg) fields

Field name	Description	Data type	Key
AutoCommit Horizon	The auto-commit horizon for this part-customer's demands. It is used to determine whether the **CustomerRequestDate** on a given demand from this part customer can be automatically committed to. This value should be expressed in intervals of the **Part.PlanningCalendars.TimeUnits** calendar, and is added to a line item's **OrderCreationDate** to determine the **AutoCommitHorizonDate**. If the line's **RequestDate** is on or after that **AutoCommitHorizonDate**, then the **DueDate** is set to the **RequestDate** and the customer request is automatically promised. Typically, this horizon will be configured to be outside of the part's lead time. If set to a value less than zero(0), then the part's default horizon from **PartSolution.AutoCommitHorizon** is used instead. **Note:** This field is in the Solutions namespace. It supports the Order Fulfillment application, and is not used in any analytic calculations.	Integer	
Customer	The customer associated with the historical demand series and/or forecast. **Referenced table**: Customer **Reference Syntax**: Customer	Reference	Yes
DemandType	The DemandType associated with consensus forecasts for this part and customer combination (for example, this can be used to set how spreading is handled for consensus forecasts. Note that regardless of the DemandType specified, RapidResponse always treats consensus forecasts as if **DemandType.ProcessingRule** is set to 'SalesForecast'. **Referenced table:** DemandType (Nullable)	Reference	
Disaggregation Calendar	A calendar that defines the periods that forecast quantities can be disaggregated into for a part customer. If this field is null, the disaggregation calendar specified in the **SOPAnalyticsConfiguration** table is used for disaggregation. For example, by default a standard monthly calendar might be used, but some customers might use a fiscal month calendar and others a weekly calendar. **Referenced table:** Calendar (Nullable)	Reference	

Table 6.190: PartCustomer (Mfg) fields

Field name	Description	Data type	Key
ExpiryOffset	Allows for the expiry requirements placed on component ingredients to be adjusted (reduced) on a part-customer basis. It specifies the number of **Part.PlanningCalendars.ExpiryCalendar** intervals to extend the minimum shelf life, and hence the calculated **EarliestExpiryDate**, before exploding demands for a given part customer down to the item's components. Thus, the expiry date on supplies used to satisfy demands for the part-customer must then satisfy this adjusted earliest expiry date. When calculated expiry dates are subsequently rolled back up the product structure, the **SupplyDemand**.**ExpiryDate** on the record reporting an allotment of supply to this top level demand is then adjusted (reduced) by the value in this field. This ensures that **ExpiryDate** reflects the limiting and part customer-specific expiry requirements on the rolled up components. Note that this field is only applicable on records for which the **MinimumShelfLife** value is provided (ignored when **MinimumShelfLife** is taken from the **Part** record). **Default value:** -1	Integer	
ForecastItem	A reference to the forecast item configuration that defines statistical forecast generation for the part and customer combination defined by this record. **Referenced table:** ForecastItem	Reference	
ForecastItemLevel	Indicates the forecast level that is defined for generating a statistical forecast for this part-customer. If you choose to forecast by aggregate part-customer, you need to set the forecast item at a level in the hierarchy that is equal to or above the aggregate part-customers. If the forecast items is set at a level that is below the aggregate part customers, you will be able to calculate the statistical forecast, but the forecast will not be recorded and the values are not available in the consensus forecast. **Note:** This field is in the Solutions namespace. It supports the Statistical Forecasting application, and is not used in any analytic calculations.	String	

Table 6.190: PartCustomer (Mfg) fields

Field name	Description	Data type	Key
FrozenHorizon	The frozen horizon for this part-customer's demands. It is used to determine a near-term window in which the **CustomerRequestDate** on a given demand from this part customer will never be automatically committed to. This value should be expressed in intervals of the **Part.PlanningCalendars.TimeUnits** calendar, and is added to a line item's **OrderCreationDate** in order to determine the **FrozenHorizonDate**. If the line's **RequestDate** is before the **FrozenHorizonDate**, then its **DueDate** is set to the later of **FrozenHorizonDate** or the **AvailableDate**, and manual intervention would be needed to pull the commitment within the frozen horizon. If set to a negative value, then the part's default horizon from **PartSolution.Frozen Horizon** is used instead. **Note:** This field is in the Solutions namespace. It supports the Order Fulfillment application, and is not used in any analytic calculations.	Integer	
MaximumNumberOfCustomerChanges	The number of customer-requested changes allowed on line items for this part customer before certain application resources apply formatting to flag the order as having reached the maximum change threshold. This might indicate further investigation of the order is required to understand the cause of all the changes. **Note:** This field is in the Solutions namespace. If a negative value is specified, then the global default from the MaximumNumberOfCustomerChanges field on the OFConfiguration table is used instead.	Integer	
MaximumNumberOfSupplyChanges	The number of supply-related changes allowed on line items for this part customer before certain application resources apply formatting to flag the order as having reached the maximum change threshold. This might indicate further investigation of the order is required to understand the cause of all the changes. **Note:** This field is in the Solutions namespace. If a negative value is specified, then the global default from the **MaximumNumberOfSupplyChanges** field on the **OFConfiguration** table is used instead.	Integer	

Table 6.190: PartCustomer (Mfg) fields

Field name	Description	Data type	Key
MinimumShelfLife	For parts configured for expiry, this is the MinimumShelfLife assigned to consensus forecasts for this part and customer combination. This represents the number of expiry calendar intervals to be added to the forecast date to determine the EarliestExpiryDate (date before which supply must not expire if it is to be used in satisfying this demand). If time-phased shelf-lifes are required, the **PartCustomerTimePhasedAttributes** table can be used (its **MinimumShelfLife** overrides this value within a given date range). Otherwise, if a negative value is provided in both this table and **PartCustomerTimePhasedAttributes**, then **Part.MinimumShelfLife** is used for this part and customer combination.	Integer	
OrderPriority	The priority assigned to consensus forecasts for this part and customer combination. Used when allocating limited supply to demands. If time-phased priorities are required, the **PartCustomerTimePhasedAttributes** table can be used. **Referenced table:** OrderPriority (Nullable)	Reference	
Part	The part associated with the historical demand series and/or forecast. **Referenced table**: Part **Reference Syntax**: Part.Name, Part.Site	Reference	Yes
Pool	The pool, if any, that consensus forecasts for this part and customer combination should belong to. **Referenced table:** Pool (Nullable)	Reference	
ToleranceProfile	The name of the tolerance profile used for this part and customer combination. Tolerance profiles define the calendars used in calculating the baseline historical demand series as well as the calendars used when calculating tolerance zones for measuring change in forecasts. **Referenced table**: ToleranceProfile (this reference is nullable in new installations of RapidResponse Version 10.1 and later).	Reference	

PartCustomer table– calculated and set fields

The following describes the calculated and set fields in the **PartCustomer** table. For an introduction to this table, and descriptions of its input fields, see "PartCustomer table" on page 444.

Table 6.191: Calculated PartCustomer (Mfg) fields

Field name	Description	Data type	Key
Effective Disaggregation Calendar	The disaggregation calendar used when disaggregating forecast quantities for this part customer. **PartCustomer.DisaggregationCalendar** is used when defined; otherwise, the disaggregation calendar specified in the **SOPAnalyticsConfiguration** table is used for disaggregation.	Reference	
Aggregates	The set of **AggregatePartCustomer** records where this part and customer combination is the aggregate part customer.	Set	
AssumptionBy PartCustomers	The set of **AssumptionByPartCustomer** records associated with this part and customer combination.	Set	
Components	The set of **AggregatePartCustomer** records where this part and customer combination is the part customer being aggregated.	Set	
Disaggregation Rates	The set of disaggregation rates calculated for the part and customer combination.	Set	
HistoricalDemand Headers	The set of **HistoricalDemandHeader** records associated with this part and customer combination.	Set	
TimePhased Attributes	Set of **PartCustomerTimePhasedAttributes** records associated with this part and customer combination	Set	

PartCustomerTimePhasedAttributes table

The **PartCustomerTimePhasedAttributes** table contains time-phased attributes (such as, priority) to be applied across all Forecast records generated from consensus forecast for a given part and customer combination. If this table contains an effective record for a part and customer combination on a given consensus forecast date, then the attributes specified in this table are applied to the Forecast record. Otherwise, if no effective record exists for a part and customer combination on a given Forecast date, then the corresponding attribute values from the **PartCustomer** table are applied instead.

This table supports Sales and Operations Planning in RapidResponse.

Table 6.192: PartCustomerTimePhasedAttributes (Mfg) fields

Field name	Description	Data type	Key
Allotment Override	A reference to the allotment override reserving component supply to consensus forecast demands for this part and customer combination. The reserved supply is allocated to consensus forecast demand(s) for part customers that reference the allotment override before it can be made available to other part customers. **Referenced table:** AllotmentOverride **Note:** This reference is nullable.	Reference	
EffectiveInDate	The date on which this set of part customer attributes become effective.	Date	
EffectiveOut Date	The date on which this set of part customer attributes is no longer effective.	Date	
ExpiryOffset	Allows for the expiry requirements placed on component ingredients to be adjusted (reduced) on a demand by demand basis. It specifies the number of **Part.PlanningCalendars.ExpiryCalendar** intervals to extend the minimum shelf life, and hence the calculated **EarliestExpiryDate**, before exploding demand down to the item's components. Thus, the expiry date on supplies used to satisfy demands for the part-customer must then satisfy this adjusted earliest expiry date. When calculated expiry dates are subsequently rolled back up the product structure, the **SupplyDemand.ExpiryDate** on the record reporting an allotment of supply to this top level demand is then adjusted (reduced) by the value in this field. This ensures that the **ExpiryDate** reflects the limiting and demand-specific expiry requirements on the rolled up components. Note that this field is only applicable on records for which the **MinimumShelfLife** value is provided (ignored when **MinimumShelfLife** is taken from the **Part** record). **Default value:** -1	Integer	
Id	A unique identifier for this record.	String	Yes

Table 6.192: PartCustomerTimePhasedAttributes (Mfg) fields

Field name	Description	Data type	Key
Minimum ShelfLife	The minimum shelf life applied to consensus forecasts within the effective date range for this part and customer combination. This field is applicable to parts configured for expiry, and represents the number of expiry calendar intervals to be added to the forecast date to determine the EarliestExpiryDate (date before which supply must not expire if being used to satisfy a demand). If a negative value is provided in this field, then the **PartCustomer.MinimumShelfLife** (if non-negative) or **Part.MinimumShelfLife** value is used instead for this part and customer combination.	Integer	
OrderPriority	The priority applied to consensus forecasts within the effective date range for this part and customer combination. Used when allocating limited supply to demands. **Referenced table:** OrderPriority	Reference	Yes
PartCustomer	A reference to the part and customer combination these attributes apply to. **Referenced table:** PartCustomer	Reference	Yes

PartMetric table

The **PartMetric** table identifies the combination for each part and metric definition generated for a specific as of date that are used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.193: PartMetric (Solutions) fields

Field name	Description	Data type	Key
AsOf	A reference to the date on which the part and metric definitions are generated. **Referenced table:** MetricAsOf	Reference	Yes
Metric	A reference to the metric associated with this combination of part and metric definition data. **Referenced table:** Metric	Reference	Yes
Part	A reference to the part the associated with this combination of part and metric definition data. **Referenced table:** Part	Reference	Yes

PartMetric table – calculated and set fields

The following describes the calculated and set fields in the PartMetric table. For an introduction to this table, and descriptions of its input fields, see "PartMetric table" on page 451.

Table 6.194: PartMetric (Solutions) fields

Field name	Description	Data type	Key
PartMetricValues	The set of part calculations that belong to this combination of part and metric definition data. **Referenced table:** PartMetricValues	Set	

PartMetricValues table

The **PartMetricValues** table stores the part calculations for metrics used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.195: PartMetricValues (Solutions) fields

Field name	Description	Data type	Key
Date	The date associated with this calculation.	Date	Yes
PartMetric	A reference to the part and metric definition combination associated with this calculation. **Referenced table:** PartMetric	Reference	Yes
Quantity	The quantity for the metric. The value in this field is used if Metric.ValueType= 'Quantity'. For more information, see "Metric table" on page 399.	Quantity	
Value	The money value for the metric. The value in this field is used if Metric.ValueType= 'Money'. For more information, see "Metric table" on page 399.	Money	

PartSequence table

The **PartSequence** table identifies the sequence assigned to a part made in a production wheel.

This table was added in Service Update 1902.

Table 6.196: PartSequence(Mfg) fields

Field name	Description	Data type	Key
Part	The part associated with this part sequence. **Referenced table:** Part	Reference	Yes
ProductionWheel	The production wheel associated with this part sequence. **Referenced table:** ProductionWheel	Reference	Yes
Sequence	The order a part is processed in the production wheel. Smaller numbers are processed first.	Integer	

PartSolution table

This table stores application-specific information related to a part. The values in this table are used by resources specific to certain application solutions such as Distribution Requirements Planning, Order Fulfillment, and Master Production Scheduling, and have no impact on analytic results.

For example, several fields in this table hold part-level parameters used by resources belonging to the MPS application such as the duration of the window during which it is recommended not to make changes to MPS orders. However, these fields are not used or required by the Master Production Scheduling analytic.

This table is not shown on the RapidResponse Data Model for Import poster.

Table 6.197: PartSolution (Solutions) fields

Field name	Description	Data type	Key
AutoCommit Horizon	The default auto-commit horizon for this part's demands. It is used to determine whether the **CustomerRequestDate** on a given demand for this part can be automatically committed to. This value should be expressed in intervals of the **Part.PlanningCalendars.TimeUnits** calendar, and is added to a line item's **OrderCreationDate** to determine the **AutoCommitHorizonDate**. If the line's **RequestDate** is on or after that **AutoCommitHorizonDate**, then the **DueDate** is set to the **RequestDate** and the customer request is automatically promised. Typically, this horizon will be configured to be outside of the part's lead time. If an auto-commit horizon is defined for a given part customer in **PartCustomer.AutoCommitHorizon**, then it is used instead. **Note:** This field is in the Solutions namespace. It supports the Order Fulfillment application, and is not used in any analytic calculations.	Integer	
DemandPlanner Code	Identifies the demand planner responsible for this part. **Referenced table**: PlannerCode (Nullable)	Reference	
DistributionPlanner Code	Identifies the distribution requirements planner responsible for this part. **Referenced table**: PlannerCode (Nullable) **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	
DRPCalendar	A reference to the Calendar that is specified for DRP settings for this part and site. If a calendar is specified in this field, it overrides the calendar set by **DRPConfiguration.Calendar**. **Referenced table**: Calendar **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	

Table 6.197: PartSolution (Solutions) fields

Field name	Description	Data type	Key
DRPFrozenWindow	Indicates the length of the DRP frozen window for the part. This value should be expressed in **DRPConfiguration.Calendar** intervals. A value of "0" can be used to indicate that the part does not have a frozen window. A value of "-1" can be used to indicate that the global default from the **DRPConfiguration.DRPFrozenWindow** field should be used. **Default value**: -1 **Note:** This field was added in the Version 2016.2, Service Update 5.	Integer	
DRPHorizon	Indicates the length of the DRP horizon for the part. This value should be expressed in **DRPConfiguration.Calendar** intervals. Within this horizon, the distribution requirements planner is actively executing the distribution requirements plan, such as aligning orders to demand and firming planned orders. A value of "0" can be used to indicate that the part does not have a DRP horizon. A value of "-1" can be used to indicate that the global default from the **DRPConfiguration.DRPHorizon** field should be used. **Default value**: -1 **Note:** This field was added in the Version 2016.2, Service Update 5.	Integer	
DRPPlanning Horizon	Indicates the length of the DRP planning horizon for the part. This value should be expressed in **DRPConfiguration.Calendar** intervals. The DRP planning horizon specifies the length of time that extends into the future for long-term planning. A value of "0" can be used to indicate that the part does not have a DRP planning horizon. A value of "-1" can be used to indicate that the global default from the **DRPConfiguration.PlanningHorizon** field should be used. **Default value**: -1 **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	

Table 6.197: PartSolution (Solutions) fields

Field name	Description	Data type	Key
FrozenHorizon	The default frozen horizon for this part's demands. It is used to determine a near-term window in which the **CustomerRequestDate** on a given demand for this part will never be automatically committed to. This value should be expressed in intervals of the **Part.PlanningCalendars.TimeUnits** calendar, and is added to a line item's **OrderCreationDate** in order to determine the **FrozenHorizonDate**. If the line's **RequestDate** is before the **FrozenHorizonDate**, then its **DueDate** is set to the later of **FrozenHorizonDate** or the **AvailableDate**, and manual intervention would be needed to pull the commitment within the frozen horizon. If a frozen is defined for a given part customer in **PartCustomer.FrozenHorizon** then it is used instead. **Note:** This field is in the Solutions namespace. It supports the Order Fulfillment application, and is not used in any analytic calculations.	Integer	
InventoryPlanner Code	Identifies the inventory planner responsible for this part. **Referenced table**: PlannerCode (Nullable)	Reference	
KeyPart	Indicates whether the part is a key part. Key parts are included in S&OP analysis and can be used to indicate limitations to planning. **Default value:** N	Boolean	
MasterScheduler Code	Identifies the master scheduler responsible for this part. **Referenced table**: PlannerCode (Nullable)	Reference	
MPSAutoFirm Window	Indicates the number of calendar intervals after the end of the MPS planning horizon where a command automatically converts planned orders for the part into firm MPS orders. This value should be expressed in intervals of the **SOPAnalyticsConfiguration.CycleCalendar**. A value of "0" can be used to indicate that the part does not have an auto firm window. A value of "-1" can be used to indicate that the global default from the **MPSConfiguration.AutoFirmWindow** field should be used for that part. **Default value**: -1	Integer	

Table 6.197: PartSolution (Solutions) fields

Field name	Description	Data type	Key
MPSFrozenWindow	Indicates the length of the MPS frozen window for the part. This value should be expressed in **MPSConfiguration.Calendar** intervals. A value of "0" can be used to indicate that the part does not have a frozen window. A value of "-1" can be used to indicate that the global default from the **MPSConfiguration.FrozenWindow** field should be used. **Default value:** -1	Integer	
MPSHorizon	Indicates the length of the MPS planning horizon for the part. This value should be expressed in **MPSConfiguration.Calendar** intervals. Within the horizon defined by this field, MPS orders are assumed to be fully under the authority of the master scheduler. A value of "0" can be used to indicate that the part does not have an MPS planning horizon. A value of "-1" can be used to indicate that the global default from the **MPSConfiguration.Horizon** field should be used. **Default value:** -1	Integer	
Part	The part this set of application solution settings applies to. **Referenced table:** Mfg::Part	Reference	
SafetyStock ItemCreate	Indicates if a part will be used to create a safety stock item. Valid values are: • **Accepted**—Indicates that the part has been reviewed and will be used to create a safety stock item. • **Rejected**—Indicates that the part has been reviewed and will not be used to create a safety stock item. • **Undecided**—indicates that the part has not been reviewed. If no value is specified, the default value for this field is used. **Default value**: Undecided **Note:** This field was added in Version 2016.2, Service Update 10 and is in the Solutions namespace. It supports the Inventory Planning and Optimization application, and is not used in any analytic calculations.	String (Enum)	
StorageType	The storage type that is associated with this part. Storage types define the storage locations that are used in the Distribution Requirements Planning application. For example, a storage type value could be Freezer or Cage. **Referenced table:** StorageType	Reference	

Table 6.197: PartSolution (Solutions) fields

Field name	Description	Data type	Key
Strategy	The manufacturing strategy for the part, such as make to stock, make to order, and so on. The manufacturing strategy can be used to determine whether appropriate planning, inventory, and supply chain strategies are being used for the part.	String	

PartSolution – calculated and set fields

The following describes the calculated and set fields in the **PartSolution** table. For an introduction to this table, and descriptions of its input fields, see "PartSolution table" on page 453.

Table 6.198: Calculated PartSolution (Solutions) fields

Field name	Description	Data type	Key
EffectiveDRP FrozenWindow	The length of the DRP frozen window that is effective for this part. If a non-negative value is provided in **DRPFrozenWindow**, it is used. Otherwise, the global default setting from the **DRPFrozenWindow** field on the **DRPConfiguration** table is used. **Note:** This field was added in Version 2016.2, Service Update 5.	Quantity	
EffectiveDRP Horizon	The length of the DRP horizon that is effective for this part. If a non-negative value is provided in **DRPHorizon**, it is used. Otherwise, the global default setting from the **DRPHorizon** field on the **DRPConfiguration** table is used. **Note:** This field was added in Version 2016.2, Service Update 5.	Quantity	
EffectiveDRP PlanningHorizon	The length of the DRP planning horizon that is effective for this part. If a non-negative value is provided in **DRPPlanningHorizon**, it is used. Otherwise, the global default setting from the **PlanningHorizon** field on the **DRPConfiguration** table is used. **Note:** This field was added in the Version 2016.2 Service Update 5.	Quantity	

Table 6.198: Calculated PartSolution (Solutions) fields

Field name	Description	Data type	Key
EffectiveMPSAuto FirmWindow	The MPS auto firm window value that is effective for this part. If a non-negative value is provided in **MPSAutoFirmWindow**, it is used. Otherwise, the default setting from the **AutoFirmWindow** field on the **MPSConfiguration.AutoFirmWindow** is used.	Quantity	
EffectiveMPS FrozenWindow	The length of the MPS frozen window that is effective for this part. If a non-negative value is provided in **MPSFrozenWindow**, it is used. Otherwise, the global default setting from the **FrozenWindow** field on the **MPSConfiguration** table is used.	Quantity	
EffectiveMPS Horizon	The length of the MPS planning horizon that is effective for this part. If a non-negative value is provided in **MPSHorizon**, it is used. Otherwise, the global default setting from the **Horizon** field on the **MPSConfiguration** table is used.	Quantity	

PartSource table

The **PartSource** table defines multiple sources of supply for a part.

The **BaseKey**, **Part**, and **Source** fields form the **PartSource** table's primary key. In many cases, **BaseKey** can be blank. When only one part source for a part exists, a blank **BaseKey** value is sufficient. Similarly, in cases where there is never more than one **PartSource** record for each supplier of a part, a blank **BaseKey** value is also valid.

Where there is more than one **PartSource** record per part, a BaseKey value must be created to ensure each **PartSource** record is unique.

For information on how RapidResponse handles the selection of multiple part sources, see "Planned orders" on page 1343.

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Absolute PrePlanLimit	Maximum number of calendar days before its normal constraint allocation date that constraint can be planned or allocated to supplies using this part source. It is recommended that this value be set high enough so there is time to build the largest supply within its limit. If this field is set to a negative value then there is assumed to be no limit on how early RapidResponse may try to preplan constraints for supplies using this part source. If this field is set to zero then, depending on your RapidResponse installation, this either means no preplanning is allowed for supplies using this part source or there is no limit on the amount of preplanning allowed for supplies using this part source. However, the interpretation of zero values in this field can be configured as discussed in "Configuring AbsolutePrePlanLimit usage" on page 2206. **Default value:** - 1	Integer	
AllotmentOverride	A reference to the allotment override reserving component supply to dependent demands generated from supply using this part source. The reserved supply is allocated to these dependent demand(s) before it can be made available to dependent demands generated from supplies using a part source that does not reference the allotment override. **Referenced table:** AllotmentOverride (Nullable)	Reference	
BaseKey	The combination of Part (including Site), Source (including Supplier) and this field forms the primary key of this table. This field can remain blank if the Part and Source values are unique. **Default value:** A blank string.	String	Yes
BOMAlternate	A reference to the alternate BOM used by this part source. When Netting logic performs explosion calculations, only components that have **BillOfMaterial.Alternate** matched to either the value in this field or the first value in the **BOMAlternate** table (that is, common components) can potentially create dependent demands for supplies using this part source. For more information about alternate BOMs, see "Alternate BOM calculations" on page 1759. **Referenced table**: BOMAlternate	Reference	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
CoProductYield	Indicates the percentage of a primary product's supply production that actually ends up as the primary product. The remainder is distributed between its co-products and by-products based on the QuantityPer values provided on the relevant BillOfMaterial records. This field respects the setting in the **OrderPolicy.YieldUsage** field, is applied after the Yield field, and is always considered by Netting calculations. If not using co-product and by-product logic in RapidResponse, this field should be set to 0 (indicates the field should be ignored). **Default Value:** 0 **Note**: If this yield is expected to vary over time, then the **TimePhasedYield** table should be used instead.	Quantity	
CRPFloatAfter	If using variable lead time based on work center capacity (**OrderPolicy.UseVarLeadTime** = "CRP"), this optional field indicates the amount of time to leave in the schedule after the final operation in the supply's routing completes. This can serve as a planning buffer to account for unexpected delays or interruptions incurred during work center operations. Expressed in Part.PlanningCalendars.TimeUnits.	Integer	
CRPFloatBefore	If using variable lead time based on work center capacity (**OrderPolicy.UseVarLeadTime** = "CRP"), this optional field indicates the amount of time to leave in the schedule before the first operation in the routing begins. This can serve as a planning buffer to account for unexpected delays in the receipt of component supplies. Expressed in Part.PlanningCalendars.TimeUnits.	Integer	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
CTPIntervalsTo Expiry	This value is used to calculate the actual expiry date on scheduled receipts and planned orders. It should be expressed in intervals of the part's **PlanningCalendars.ExpiryCalendar**, and indicates the number of those intervals it takes supply from the part source to expire. The specific usage of this field in expiry date calculations depends on the part's setting in **PartType.ExpiryRule** as follows: • **Dependent**—Not used (expiry date is entirely determined from the part's component supplies). • **DependentStart**—Number of ExpiryCalendar intervals to add to the earliest expiry start date rolled up from a component supply (by default, that component's available date). • **DependentLimit**—Number of ExpiryCalendar intervals to add to the supply's DueDate (planned orders) or AvailableDate (scheduled receipts), and used only if that date is earlier than the earliest expiry date found on one of the part's component supplies). • **Ignore**—Not used (supply never expires). • **Self/SelfReset**—Number of ExpiryCalendar intervals to add to the supply's DueDate (planned orders) or AvailableDate (scheduled receipts). This value can be overridden at the scheduled receipt level if a date is provided in the **ScheduledReceipt.ExpiryDate** field. If the value in this field is 0 or negative, then the **IntervalsToExpiry** field is used instead. However, because **IntervalsToExpiry** is used in estimated expiry date calculations, which don't necessarily start from the same point as actual expiry date calculations (based on **ExpiryRule**), this should be done with caution. **Note:** This field was added in Version 2016.2, Service Update 5. **Default value**: 0	Integer	
DockToStockLead Time	The time required to move materials from the receiving dock to available inventory. The setting in this field increases the effective lead time for all types of supply; ensure it is set to 0 unless needed. It is measured using Part.PlanningCalendars.TimeUnits. **Default value**: 0 **NOTE**: If this value is expected to change over time, the TimePhasedPartSourcLeadTime table can be used instead.	Quantity	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	Defines the start of the period from which this part source record can be considered valid. This date along with the control setting in **PartSourceType.EffectivityRule** determines the first date from which supply can be planned from the part source (reported in the calculated **FirstEffectiveDate** field).	Date	
EffectiveOutDate	Defines the end of the period in which this part source record is valid. This date along with the control setting in **PartSourceType.EffectivityRule** determines the last date from which supply can be planned from the part source (reported in the calculated **LastEffectiveDate** field).	Date	
FixedLeadTime	Order release offset from due date. This represents a fixed amount of time to produce each order for the part from this source. **NOTE**: If this value is expected to change over time, the TimePhasedPartSourcLeadTime table can be used instead.	Quantity	
IntervalsToExpiry	This field is used internally by the Netting analytic in calculating an estimated expiry date on a potential planned order (this estimation is then used in determining if the part source can be used to supply a given demand). This value should be expressed in intervals of the part's **PlanningCalendars.ExpiryCalendar**, and indicates the number of those intervals it takes supply from the part source to expire. For parts with a **PartType.ExpiryRule** of "Dependent", this field should be set to account for the part's fastest expiring component. A value equal to or less than zero indicates that supplies using this part source should never expire. **Default value:** 0 Note: In previous versions, this field was also used in actual expiry date calculations. However, as of Version 2016.2, Service Update 5, the CTPIntervalsToExpiry field is used in actual expiry date calculations instead. However, if CTPIntervalsToExpiry is set to 0 or negative, then this field is still used in calculating the expiry date (as it was in previous versions).	Integer	
LaborCost	The standard labor cost associated with one unit of this supply (in SupplierUOM). Negative values are treated as zero. **Note:** The **PartSourceTimePhasedAttributes** table can be used to specify time-phased labor cost values. Values provided on that table override those on PartSource.	Money	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
MaterialCost	Material cost associated with one unit of this supply. Used to calculate CostOfGoodsSold.MaterialCost. Also used to calculate CostRollUp.MaterialCost if PartSource.Type.CostRule control value is MaterialLaborOverheadCosts. Negative values are treated as zero. **Default value:** 0 **Note:** The **PartSourceTimePhasedAttributes** table can be used to specify time-phased material cost values. Values provided on that table override those on the **PartSource** table**.**	Money	
MaximumQty	The maximum order quantity (more than one planned order can be placed in a period). If **OrderPolicy.MaximumUsage** is set to either "Use" or "Average", then the quantity on any planned order generated from this part source will not exceed the value in this field (an exception is made on the last planned order in a period if the maximum number of orders per period has been reached). However, in cases where **MaximumUsage** is set to "CampaignPlanning", this field represents the maximum planned order quantity across all supply batches (orders) in a production campaign, and a new campaign must be started once that maximum is reached. In other words, the maximum number of batches allowed in a given production campaign is defined as `MaximumQty/MultipleQty` (and rounded up to the next integer where necessary). Note also that a global maximum on the number of batches in any campaign from any part source can be set as discussed in "Specify global batch limit" on page 1710. For all **MaximumUsage** settings other than those discussed above, the value in this field is ignored when generating planned orders. **Default value:** 0 (no maximum) **NOTE**: **SupplierUOM** applies to **MaximumQty**. CAUTION: Use of the **MaximumQty** setting typically results in more planned orders than would otherwise be generated. If the total demand quantity is large and the **MaximumQty** is set to a small number, then this can result in very large numbers of planned orders being generated which can potentially impact system performance. Note also that a global setting can be used to place a cap on the number of planned orders generated per part each day as discussed in "Limiting the number of planned orders" on page 2204.	Quantity	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
MinimumQty	The minimum order quantity (more than one planned order can be placed in a period). If **OrderPolicy.MinimumUsage** is either "Use" or "RoundMult" then the quantity on any planned order generated from this part source will be greater than or equal to the value in this field (with this field being ignored when **MinimumUsage** is is set to "Ignore"). However, regardless of the **MinimumUsage** value, if the **OrderPolicy.MaximumUsage** option is set to "CampaignPlanning", then this field represents the minimum planned order quantity across all supply batches (orders) in a production campaign. This means a given campaign using this part source must continue planning batches until this minimum supply amount has been built. In other words, the minimum number of batches that make up a valid production campaign is defined as `MinimumQty/MultipleQty` (and rounded up to the next integer where necessary). Additionally, if **OrderPolicy.MaximumUsage** is set to "Kanban", than this field is not used in determining planned order quantities. **Note:** Depending on the setting in **PartSource.Type.ConstraintMinimumRule** this minimum value may also be applied when splitting supplies for constraint allocation or incremental availability. **Note: SupplierUOM** applies to **MinimumQty**.	Quantity	
MinimumShelfLife Threshold	This field can be used to override or extend the minimum shelf life requirements on dependent demands satisfied by orders from the part source. Values provided in this field should be expressed in **Part.PlanningCalendars. ExpiryCalendar** intervals. By default, the earliest expiry date on a planned order is taken from the dependent demand it is used to satisfy. However, if a positive value is provided in this field, then that number of intervals is added to the planned order due date. If the resulting date is later than the earliest expiry date on the dependent demand, it is reported in the **PlannedOrder.EarliestExpiryDate** field (otherwise, the **EarliestExpiryDate** from the **DependentDemand** record is used). **Default value:** -1	Integer	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
MLSPreTarget Sequence	In multi-level search logic, this field can be used to specify an additional selection criterion that gets applied when distinguishing between eligible part sources of the same priority (where those eligible sources can satisfy the provide the supply on-time and the lowest level component supplies being used by each are of the same type). For example, this field might used to specify a preference between secondary part sources. A lower value in this field indicates a higher sort sequence or priority. Thus, if two part sources of the same priority are otherwise equivalent and eligible to provide a given supply, then the one with lower value in the field would be used. If those part sources had the same value in this field, then **PartSource.Target** field is next looked at to determine supply sourcing. **Note:** This field was added in Version 2016.2, Service Update 8.	Integer	
MultipleQty	The target lot size for planned orders. Generally, if **OrderPolicy.MultipleUsage** is set to "Use", then planned orders from the part source should be generated in multiples of this value (for other **MultipleUsage** settings, the value in this field is ignored). However, regardless of the **MultipleUsage** value, if the **OrderPolicy.MaximumUsage** option is set to "CampaignPlanning", then the value in this field represents the intended batch size and all planned orders from the part source are generated for exactly this quantity. Note that if this field is set to "0", then campaign planning is disabled even if **MaximumUsage** is set to "CampaignPlanning". Additionally, if **OrderPolicy.MaximumUsage** is set to "Kanban", than this field is not used in determining planned order quantities. **Note:** Depending on the setting in **PartSource.Type.ConstraintMultipleRule**, this multiple may also be applied when splitting supplies for constraint allocation or incremental availability. **Note: SupplierUOM** applies to **MultipleQty**.	Quantity	
OrderPolicy	Order policies for this part source. **Referenced table:** OrderPolicy	Reference	
OverheadCost	The standard overhead cost associated with one unit of this supply (in SupplierUOM). Negative values are treated as zero. **Note:** The **PartSourceTimePhasedAttributes** table can be used to specify time-phased overhead cost values. Values provided on that table override those on PartSource.	Money	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
OverheadCost2	A second standard overhead cost associated with one unit of this supply. OverheadCost2 is used in the NewProcessCost calculation. Negative values are treated as zero. **Default value:** 0 **Note:** The **PartSourceTimePhasedAttributes** table can be used to specify time-phased overhead cost values. Values provided on that table override those on PartSource.	Money	
Part	The name of the part replenished by this part source. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes
PlanningTimeFence	Used in determining the earliest a new order can be planned using this part source (based on the setting in the **OrderPolicy.PTFRule** field). This field should be expressed in either the part or source's time units (based on the setting in the **OrderPolicy.PTFUnit** field). **Default value**: 0	Quantity	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Pool	The pool associated with this part source. For parts that have inventory pooling logic enabled (the **MUEPoolNettingType.PoolRule** field is set to either "Net" or "Optional"), the referenced pool can influence the part source selection process. This is dependent on the **MUEPoolNettingType.PoolFlowToCommon** setting as follows: • When **PoolFlowToCommon** is either "NetOriginal", "Ignore", or "Forecast", planned orders can only be created on part sources where the **Pool** reference matches the pool being netted or is is Null. • When **PoolFlowCommon** is "NetCommon", planned orders can only be created on part sources where the **Pool** reference matches the "common" pool, or is Null. Note that this means planned orders cannot be created on part sources where the **Pool** reference is a match for the pool being netted. Once the part source is chosen in consideration of pool, new planned orders are created on either the matching supply pool or common pool, and dependent demands generated on components inherit their pools from the parent supplies. Thus, if pooling is enabled on an end-item assembly, part source selection for the assembly can be pool-based and that pool information propagated down to its components. When pooling is disabled on an end-item assembly and this field is populated, supplies are still planned in the common pool (as determined from **PlanningCalendars.CommonPool**) and the part source(s) that can be used for those supplies are not dependent on the referenced **PartSource.Pool** values. However, if pooling is enabled on a lower-level component, then the Pool referenced from that assembly can be injected onto the dependent demands and hence planned supplies for those components. **Referenced table:**Pool (Nullable) **NOTE**: This field was added in Version 2016.2, Service Update 19. For more information, see "Pools and part sources" on page 1750.	Reference	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
PrePlanLimit	The maximum number of Source.LeadTimeUnits to try to plan orders (when the current period has insufficient constraint(s)) before trying to plan orders through lower priority part sources. Other part sources at the same priority will be used in the current period if they are available. Also, if none of the part sources can satisfy the demand for the current period, respecting pre-plan limit, then test periods before the pre-plan limits. This field is dependent upon its value: • **<0**—use this source in earlier periods until it has no more earlier constraint available (PTF, Constraint.DataDate, FirstEffectiveDate) before trying lower priority part sources in the current period. All negative values are treated identically. • **0**—test other sources in the current period before trying this source in earlier periods (default is 0). • **>0**—test part sources at the same priority as this source in the current period, then test this source (then others at the same priority, respecting their PrePlanLimit) in earlier periods, up to and including the period containing the initial calculated ship date less the value of the Source.LeadTimeUnits field.	Integer	
Priority	A priority value to use when selecting a PartSource to supply a given part requirement (used when there are multiple active PartSources can either provide the supply on-time or else equally late). The interpretation of this value is controlled by the **SourceRule.PriorityRule** field (defines whether higher or lower values are assumed to indicate a higher effective priority).	Quantity	
ProductionRecurrence	A reference to the production recurrence associated with this PartSource record. **Referenced table:** ProductionRecurrence	Reference	
ProductionGroup	A reference to the production group associated with this PartSource record. **Referenced table:** ProductionGroup	Reference	
ProductionWheel	A reference to the production wheel associated with this PartSource record. **Referenced table:** ProductionWheel	Reference	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Routing	The standard routing for supplies using this part source. This field is used by the Capacity Requirements Planning analytic. This field must contain a value (it can be set to "None" if it is not being used. **Referenced table:** Routing **Reference Syntax:** Routing.Id	Reference	
SafetyLeadTime	Number of days to offset the demand due date when determining when supply should be planned to cover that demand. This is intended to protect against fluctuations in supply and demand and will impact planned order due dates as well as EffLeadTime and CumLeadTime calculations. This value should be expressed in either the part's **PlanningCalendars.TimeUnits** calendar or the Everyday/Gregorian calendar as determined by the part's **PartType.SafetyLeadTimeCalendarRule** setting. Note that usage of this field depends on the setting in the **PartType.UseSafetyLeadTime** field, with additional **PartType** settings available to specify how safety lead time is applied for specific types of demands. **Default value**: 0 **Note:** SafetyLeadTime is ignored and not used to offset due dates for safety stock demands.	Integer	
SecondaryPriority	If the part's **SourceRule.AllotmentRule** is set to "ProportionalLotSize" or "OngoingToDate", this field is used to chose between eligible sources of the same **Priority** that also have the same **Target** value or the same **Target** value and calculated quota rating. This value is also used in multi-level search logic in cases where two eligible part sources are still considered equal after evaluating their **Target** values. The interpretation of this value is controlled by the **SourceRule.PriorityRule** setting (defines whether higher or lower values are assumed to indicate a higher secondary priority). **Note:** This field was added in Version 2016.2, Service Update 6.	Quantity	
ShippingCost	The standard cost associated with shipping using this part source. Negative values are treated as zero. This field is in the **Solutions** namespace. **Note:** This field was added in the Version 2016.2, Service Update 5.	Money	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Source	Provides information about the source. For example: name, phone, and email. **Referenced table:** Source **Reference Syntax:** Source.Id, Source,Supplier.Id	Reference	Yes
SupplierPart	Supplier's part number (used for reporting if the part source is not a Transfer). If the part has only one part source, leave this field empty. If the part has more than one part source, specify the supplier's part number.	String	
SupplierUOM	The units used for ordering from this source. Dependent demands are created using this unit of measure (UOM). SupplierUOM applies to all unit quantity fields in PartSource. These are: • MaximumQty • MinimumQty • MultipleQty • ToDateQuantity • Yield (only if the OrderPolicy.YieldUsage control value is ScrapPiece)	String (Enum)	
TaktTime	The number of time units (typically, work days) in between consecutive planned orders when the required quantity for the part exceeds the value in **MaximumQty**. The time unit is set by the value in **Part.PlanningCalendars.TimeUnits**. If this field is < 0, then planned orders are spread back in time starting from the supply's original due date. If this field is > 0, then planned orders are spread ahead in time starting from the supply's original due date. If the spread planned orders reach PTFDate before the required quantity is created, then multiple planned orders are created on PTFDate. If this field is set to 0, then multiple planned orders to satisfy a demand for the part can be created on the same date. For more information, see "Spreading planned orders with Takt Time" on page 1376. **Default value**: 0 **Note:** If a given part source uses Takt Time (that is, TaktTime <> 0), then the part source is treated as unconstrained regardless of whether or not any constraints are assigned to the part source.	Integer	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Target	Target allotment value to use for this part source when there are multiple active part sources (of the same priority) eligible to satisfy planned requirements for the part. This value is used to determine the percentage split of planned requirements among the active part sources. Depending on the setting in the **SourceRule.AllotmentRule** field, this split among sources is either made proportionally between sources for each requirement or on ongoing basis with each requirement meant to be satisfied by only one source whenever possible. If all active part sources have their Target set to 0, this indicates demands should be evenly split across the eligible sources. **Default value:** 0	Quantity	
ToDateQty	The quantity that has been allocated to this part source to date. It's used as the initial value for ongoing allotments. **Note: SupplierUOM** applies to **ToDateQty**.	Quantity	
TransferPart	Part to be supplied (only used for Transfer sourcing, includes site.) Typically set to the part itself for Make and Buy part sources. **Referenced table:** Part **Reference Syntax:** TransferPart.Name, TransferPart.Site	Reference	
Type	A value that is associated with processing rules that defines how the sourcing data is interpreted. **Referenced table:** PartSourceType	Reference	
UnitCost	The cost of one unit (in terms of SupplierUOM) of this part from this part source. This cost is assumed to include material, labor, and overhead costs combined. It is therefore similar to Part.StdUnitCost. For a part's primary part source, this field will typically equal Part.StdUnitCost, with appropriate UOM conversion.	Money	
VarLeadTime	The portion of lead time dependent upon order quantity (in units of SupplierUOM). If **OrderPolicy.UseVarLeadTime** is set to "Y", the value in this field is then multiplied by the order quantity (**PlannedOrder.Quantity** or **ScheduledReceipt.Quantity)** to determine the variable portion of lead time. Otherwise, this field is ignored. **Note:** VarLeadTime is also ignored if **PartSourceType.ConstraintDateRule** is set to either "DuringLeadTime", "Start", or "StartExtend".	Quantity	

Table 6.199: PartSource (Mfg) fields

Field name	Description	Data type	Key
Yield	Factor to reduce supply quantities to account for scrap. This value is affected by the **OrderPolicy.YieldUsage** control value. If this factor is expected to vary over time, then the **TimePhasedYield** table should be used to specify yield values for the part source. **Note: SupplierUOM** applies to **Yield** (only if the **OrderPolicy.YieldUsage** control value is ScrapPiece).	Quantity	

PartSource table – calculated and set fields

The following describes the calculated and set fields in the PartSource table. For an introduction to this table, and descriptions of its input fields, see "PartSource table" on page 459.

Table 6.200: Calculated PartSource (Mfg) fields

Field name	Description	Data type	Key
EffFixedLeadTime	Fixed lead time (based on FixedLeadTime and OrderPolicy values.) **NOTE**: If using time-phased fixed lead times, then the lead time effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** is used.	Quantity	
EffLeadTime	The total lead time for this part source (sum of all lead time components defined on the part and part source). This includes effective safety, fixed, and variable lead times as well as lead time adjustments. For part sources where the **Type.TransitRule** is set to "Cumulative", this also includes pre-ship lead time, transit time, and dock-to-stock lead time. **Note:** Fixed and variable lead times are ignored when calculating lead time for transfer sources (lead time calculations start from the built date). **Note:** If using time-phased lead time (for fixed, dock-to-stock, transit, or pre-ship lead times). then the lead times effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used. **Note:** If using time-phased safety lead time for a part, any effective and applicable values defined in the **SafetyLeadTimeProfileDetail** table are ignored when calculating the value returned by this field. If populated, the **Part.SafetyLeadTime** field value is used instead.	Quantity	

Table 6.200: Calculated PartSource (Mfg) fields

Field name	Description	Data type	Key
EffUnitCost	The unit cost of one unit of this part source (reported in terms of stock units). The calculation used to populate this field depends on the setting in the **PartSourceType.CostRule** field as follows: • If set to "MaterialLaborOverheadCosts" then calculated based on the material, labor and overhead values on either the applicable **PartSourceTimePhasedAttributes** record (if one exists) or the **PartSource** record. • Otherwise if set to "UnitCost" then derived from the **UnitCost** value on the **PartSource** record. • And if the applicable calculation returns a value less than or equal to zero (<= o), then derived from the **Part.StdUnitCost** value. Currency conversions on this field use the **FirstEffectiveDate** field's conversion rate.	Money	
EffVarLeadTime	Variable lead time (based on VarLeadTime and OrderPolicy rules).	Quantity	
EffYield	Effective yield on this part source. Calculated by combining (multiplying) the **Yield** and **CoProductYield** values (and respecting the **OrderPolicy.YieldUsage** value). If **OrderPolicy.YieldUsage** is set to 'ScrapPiece' this field reports -Yield, and the net quantity added to inventory is Quantity + EffYield. If **OrderPolicy.YieldUsage** is set to anything other than 'ScrapPiece' this field reports a value between 0 and 1, and the net quantity added to inventory is Quantity * EffYield. **Note**: If time-phased yield logic is applicable, the **Yield** and **CoProductYield** values used are taken from the **TimePhasedYield** table. For a given part source, the **TimePhasedYield** record that is effective on the latter of **PartSource.EffectiveInDate** or **Part.RunDate** is used.	Quantity	
FirstEffectiveDate	Date when this PartSource record becomes effective (undefined if never effective). Planned orders from the part source cannot be earlier than this date (or later than **LastEffectiveDate**). If a demand falls outside of the effective window for all potential part sources, then it can either be shifted into effectivity or left unsatisfied (based on the part's **SourceRule.SourceEffectivityWindow** setting).	Date	

Table 6.200: Calculated PartSource (Mfg) fields

Field name	Description	Data type	Key
IsRecursive	Set to Y when this PartSource record is involved in a recursive data structure. For example, this might occur when a part is a component of itself. Similarly, this might occur if there is a part at two different sites and each has a transfer part source meant to transfer supply from the other part and site. Valid values are: • Y • N **Default value:** N **Note**: For more information about recursive data, see "Recursive data" on page 1423.	Boolean	
LastEffectiveDate	Last date when this PartSource record remains effective (undefined if never effective). Planned orders from the part source cannot be later than this date (or earlier than **FirstEffectiveDate**). If a demand falls outside of the effective window for all potential part sources, then it can either be shifted into effectivity or left unsatisfied (based on the part's **SourceRule.SourceEffectivityWindow** setting).	Date	
LeadTimeDate	The lead time date for this part source. This date is calculated starting from the run date (**Part.PlanningCalendars.RunDate.FirstDate)** and adding all lead time components (either inclusive or cumulative) while respecting the relevant planning calendars. This field is useful when looking for demand and supply due within a part source's lead time. For more information about lead time, see "Lead time calculations" on page 1307.	Date	
PTFDate	The date of the planning time fence calculated based on the run date, **PlanningTimeFence**, and the setting in the **OrderPolicy.PTFRule** field. When **OrderPolicy.OrderGenerationRule** is set to "AfterPTFDate", planned orders from the part source can only be generated after this date. **NOTE**: If time-phased lead times are used, then the lead times effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate** are used.	Date	
SourceKanban	The SourceKanban selected for this part source. If this part source has no SourceKanbans, then this reference can be null. **Referenced table**: SourceKanban	Reference	

Table 6.200: Calculated PartSource (Mfg) fields

Field name	Description	Data type	Key
CTPPlannedOrders	Set of records on calculated table that contain a reference to a CTPPlannedOrder using this PartSource.	Set	
MEIOLeadTimes	Set of MEIOLeadTime records that contain stochastic lead time parameters for this part source.	Set	
MEIOParts	Set of SafetyStockPart records containing safety stock parameters for parts from this part source.	Set	
NewTransfer Allocations	Set of NewTransferAllocation records (dependent demands) arising from transfer Scheduled Receipts through this part source.	Set	
PlannedOrders	Set of records on calculated table that contain a reference to a PlannedOrder using this PartSource.	Set	
PlannedTransfer Allocations	Set of dependent demands arising from PlannedTransferAllocation records through this part source.	Set	
ScheduledReceipts	Set of records that contain a reference to a ScheduledReceipt using this PartSource.	Set	
SourceKanbans	Set of SourceKanban records for this part source.	Set	
SupplyAllocations	Set of manual (user specified) SupplyAllocation records that define allocations of supply to demand made where the supply uses this part source.	Set	
TimePhased Attributes	Set of PartSourceTimePhasedAttributes records for this part source.	Set	
TimePhased LeadTimes	Set of PartSourceTimePhasedLeadTime records for this part source.	Set	
TimePhased Yields	Set of TimePhasedYield records for this part source.	Set	

PartSourceTimePhasedAttributes table

The PartSourceTimePhasedAttributes table is used to define time-phased cost values on part sources. Each record contains a reference to the PartSource record to which the cost values apply, and an effective date indicating the date from which the values are effective. If a given part source has an effective record in this table at a particular date, and the sum of all cost values in that record are greater

than zero, then the cost values from that record override the corresponding values on the part source. Otherwise, the cost values from the PartSource record are used.

This table allows for time-phased costing by part source without the need to create and maintain multiple PartSource records. This can result in improved performance of certain RapidResponse analytics and reduces the effort in trying to match scheduled receipts to the appropriate part source and cost values.

Table 6.201: PartSourceTimePhasedAttributes (Mfg) fields

Field name	Description	Data type	Key
EffectiveDate	The date when this set of attributes becomes effective. Time-phased attributes are considered effective until the next record with a later EffectiveDate is found for a given part source. When determining the effective set of attributes for a given supply, the **DueDate** (planned orders) or **ReschedDate** (scheduled receipts) are compared against this value.	Date	
Id	An identifier for this set of time-phased attributes. The combination of this value and the PartSource reference are used to uniquely identify this record.	String	Yes
LaborCost	The standard labor cost associated with one unit of supply from this source (in SupplierUOM). Negative values are treated as zero.	Money	
MaterialCost	The material cost associated with one unit of supply from this source (in SupplierUOM). Negative values are treated as zero.	Money	
OverheadCost	The standard overhead cost associated with one unit of supply from this source (in SupplierUOM). Negative values are treated as zero.	Money	
OverheadCost2	A second standard overhead cost associated with one unit of supply from this source. This field is used in the NewProcessCost calculation. Negative values are treated as zero.	Money	
PartSource	A reference to the part source this set of time phased attributes applies to. **Referenced table:** PartSource **Reference Syntax:** PartSource.Part.Site, PartSource.Part.Name, PartSource.Source.Id, PartSource.Source.Supplier.Id, PartSource.BaseKey	Reference	Yes

PartSupplier table

This table identifies each unique part and supplier combination used in historical data.

Table 6.202: PartSupplier (Mfg) fields

Field name	Description	Data type	Key
Part	The part associated with the historical supply series. **Referenced table**: Part **Reference Syntax**: Part.Name, Part.Site	Reference	Yes
Supplier	The supplier associated with the historical supply series. **Referenced table**: Supplier **Reference Syntax**: Supplier	Reference	Yes
SupplierUOM	The unit of measure associated with this PartSupplier record. **Referenced table:**UnitOfMeasure **Note**: This field was added in Version 2016.2, Service Update 12.	Reference	
ToleranceProfile	The name of the tolerance profile used for this part and supplier combination. Tolerance profiles define the calendar used in calculating the baseline historical supplier series as well as the calendar used when calculating tolerance zones for measuring acceptable levels of change in forecasts. **Referenced table**: ToleranceProfile (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	

PartSupplier table – calculated and set fields

The following describes the calculated and set fields in the PartSupplier table. For an introduction to this table, and descriptions of its input fields, see "PartSupplier table" on page 477.

Table 6.203: Calculated PartSupplier (Mfg) fields

Field name	Description	Data type	Key
HistoricalSupply Headers	The set of historical supply series' associated with this part and customer combination.	Set	

PartUOMConversion table

The **PartUOMConversion** table is used to store part-specific unit of measure conversion rates. Records stored in this table are used to calculate forecast item quantities when generating a statistical forecast. It allows a part to be calculated using a factor that is specific to that part, instead of the **UnitOfMeasure.BaseConversion** that is specified for a **UnitofMeasure**. Therefore, if a part has a unit of measure conversion record in the **PartUOMConversion** table, the **UnitOfMeasure.BaseConversion** is ignored. The **PartUOMConversion** table should be used if the **Part.UnitOfMeasure** has a different base unit value than the one that is given in **UnitOfMeasure.BaseConversion**. The part-specific unit of

measure conversion factor needs to be entered to ensure that the quantities are correctly converted to a different unit of measure. For more information, see "Define unit of measure" on page 2027.

Quantities in the following tables are calculated using **PartUOMConversion**:

- **ForecastItemParametersActual**
- **StatisticalForecast**
- **StatisticalForecastDetail**
- **StatisticalForecastDisaggregationRate**

Notes:

- To successfully use the **Unit of Measure** control in any of the workbooks, it's necessary to assign a **ForecastItemParameters.UnitofMeasure** as well as **PartUOMConversion** even if the forecast item is reported in the same unit of measure as their part-customers.
- The capability to use the **PartUOMConversion** table to define a part-specific unit of measure conversion rate for calculating of the statistical forecast was added in Version 2016.2, Service Update 12. Prior to this update, the table was intended to store part-specific unit of measure conversion rates only for reporting purposes.

Table 6.204: PartUOMConversion (Mfg) fields

Field name	Description	Data type	Key
Factor	The conversion factor used to convert values from **Part.UnitOfMeasure** to **UnitOfMeasure**. The following formula is used to convert a quantity from **Part.UnitOfMeasure** to **UnitOfMeasure**: Quantity * PartUOMConversion.Factor	Quantity	
Part	A reference to the part and site whose unit of measure this conversion factor applies to. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes
UnitOfMeasure	The unit of measure to which the factor in this table is used in converting to. **Referenced table:** UnitOfMeasure	Reference	Yes

PartValidation table

The **PartValidation** table is used to record notes about parts during input validation.

This table was added in Version 2016.2 Service Update 9, and modified in Version 2016.2 Service Update 11.

Table 6.205: PartValidation (Validation) fields

Field name	Description	Data type	Key
CommentsByPart	Additional information about the part.	String	
CommentsForDemand	Additional information about known demand issues for the part.	String	
CommentsForOnHand	Additional information about known on hand issues for the part.	String	
CommentsForSupply	Additional information about known supply issues for the part.	String	
Flagged	Whether the part has been flagged.	Boolean	
KnownIssueByPart	Whether the part has known issues recorded.	Boolean	
KnownIssueForDemand	Whether the part has known issues for demand recorded.	Boolean	
KnownIssueForOnHand	Whether the part has known issues for on hand recorded.	Boolean	
KnownIssueForSupply	Whether the part has known issues for supply recorded.	Boolean	
Notes	Notes about the part and its validation operations.	Note	
Part	The part being recorded. **Referenced table:** Mfg::Part	Reference	Yes

PenaltySchedule table

The **PenaltySchedule** table contains string values identifying the different penalty schedules that can be used in RapidResponse. Both the **Project** and **Task** tables reference this table to specify the penalty schedule, if any, associated with a given project or task.

Each record in this table is also associated with a set of referencing records in both the **PenaltyScheduleByDate** and **PenaltyScheduleByInterval** tables. These tables define the one-time and recurring costs that are associated with a given penalty schedule.

This table's records can be edited using the **Penalty Schedules** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.206: PenaltySchedule (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this penalty schedule.	String	
Name	A unique name for this penalty schedule. This value can be referenced by both the **Project.PenaltySchedule** and **Task.PenaltySchedule** fields to set the penalty schedule associated with a given project or task.	String	Yes
Site	The site associated with this penalty schedule. Referenced table: Site	Reference	Yes

PenaltySchedule table – calculated and set fields

The following describes the calculated and set fields in the **PenaltySchedule** table. For an introduction to this table, and descriptions of its input fields, see "PenaltySchedule table" on page 480.

Table 6.207: Calculated PenaltySchedule (ProjMgmt) fields

Field name	Description	Data type	Key
PenaltiesByDate	Set of date based penalty costs associated with this penalty schedule.	Set	
PenaltiesBy Interval	Set of interval based penalty costs associated with this penalty schedule.	Set	
Projects	Set of projects that use this penalty schedule.	Set	
Tasks	Set of tasks that use this penalty schedule.	Set	

PenaltyScheduleByDate table

The **PenaltyScheduleByDate** table holds date effective records used to define the one-time and recurring penalty costs for a specified penalty schedule and the projects and/or tasks that use that penalty schedule. The specified penalty costs can then be applied both to projects that have a **ProjectType.PenaltyRule** value of "Date" as well as tasks that have a **TaskType.PenaltyRule** value of "Date".

For example, this table can be used to set up a penalty schedule that applies a one-time penalty cost if a project is not finished by its penalty date and then applies a recurring penalty cost for each workday after that until it finishes.

This table's records can be edited using the **Penalties by Effective Date** worksheet in the **Project Master Data** workbook included with RapidResponse. For more information about how this table is used, see "Date-based penalty schedules" on page 2192.

Table 6.208: PenaltyScheduleByDate (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveDate	Sets the first date on which the penalty costs defined on this record can be applied to referencing projects or tasks. The costs are effective until the next EffectiveDate associated with the referenced penalty schedule. If there are multiple effective records between a project or task's penalty date and finish date, then effective penalty costs are used as defined by the setting in the **AccumulationRule** field on either the **ProjectType** or **TaskType** table.	Date	
Id		String	Yes
OneTimeCost	A one-time penalty cost applied to be applied to projects or task's that have a **CalcFinishDate** later than their **PenaltyDate**.	Money	
PerIntervalCost	A recurring penalty cost. Applied for each effective date between a project or task's **PenaltyDate** and **CalcFinishDate** that belongs to the penalty calendar associated with the project type or task type. For example, penalty costs might be applied on each workday or each weekly interval.	Money	
Schedule	A reference to the named penalty schedule these penalty costs apply to. **Referenced table**: PenaltySchedule	Reference	Yes

PenaltyScheduleByInterval table

The **PenaltyScheduleByInterval** table holds interval-based records, expressed in terms of penalty calendar intervals, used to define the one-time and recurring penalty costs for a specified penalty schedule and the projects and/or tasks that use that penalty schedule. The specified penalty costs can then be applied both to projects that have a **ProjectType.PenaltyRule** value of "Interval" as well as tasks that have a **TaskType.PenaltyRule** value of "Interval".

For example, assuming a weekly calendar, this table can be used to set up a penalty schedule that applies one set of weekly penalty costs for projects that are less than four weeks past their penalty date without finishing, and then applies another set of weekly penalty costs for projects that are more than four weeks past their penalty date without finishing.

This table's records can be edited using the **Penalties by Interval** worksheet in the **Project Master Data** workbook included with RapidResponse. For more information about how this table is used, see "Interval-based penalty schedules" on page 2194.

Table 6.209: PenaltyScheduleByInterval (ProjMgmt) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this record.	String	Yes
Interval	The number of penalty calendar periods after the project or task's **PenaltyDate** at which the costs in this record become effective. For example, if using a weekly penalty calendar, a value of 2 in this field indicates that the costs defined in this record are effective starting from the second week after the penalty date and remain applicable until the next effective record. If there are multiple effective Interval values between a project or task's penalty date and finish date, then effective penalty costs are used as defined by the setting in the **AccumulationRule** field on either the **ProjectType** or **TaskType** table.	Integer	
OneTimeCost	A one- time penalty cost to be applied to projects or task's that have a **CalcFinishDate** later than their **PenaltyDate**.	Money	
PerIntervalCost	A recurring penalty cost. Applied for each effective date between a project or task's **PenaltyDate** and **CalcFinishDate** that belongs to the penalty calendar associated with the project type or task type. For example, penalty costs might be applied on each workday or each weekly interval.	Money	
Schedule	A reference to the named penalty schedule these penalty costs apply to. **Referenced table**: PenaltySchedule	Reference	Yes

PlannerCode table

The **PlannerCode** table contains a list of valid planner codes. The planner code is meant to identify a person or job responsibility for making planning decisions for the part. This table and the PlannerCode field in the **Part** table have no algorithmic use.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.210: PlannerCode (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this planner code.	String	
Site	The site associated with this planner code. This field is optional. **Referenced table:** Core::Site	Reference	Yes
UserId	The user responsible for this buyer code. This field is in the **Solutions** namespace.	String	
Value	A unique identifier associated with this planner code.	String	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

PlannerCode table – calculated and set fields

The following describes the calculated and set fields in the **PlannerCode** table. For an introduction to this table, and descriptions of its input fields, see "PlannerCode table" on page 483.

Table 6.211: Calculated PlannerCode (Mfg) fields

Field name	Description	Data type	Key
CustomerService Representatives	Set of **Customer** records that use this record to identify a customer service rep.	Set	
DemandPlanner Codes	Set of **PartSolution** records that use this record to identify the demand planner responsible for a part.	Set	
InventoryPlanner Codes	Set of **PartSolution** records that use this record to identify the inventory planner responsible for a part.	Set	
MasterScheduler Codes	Set of **PartSolution** records that use this record to identify the master scheduler responsible for a part.	Set	
Parts	Set of parts associated with this planner code.	Set	

PointsCurve table

The **PointsCurve** table holds named efficiency curves which are sets of curve points defining varying rates of resource efficiency during the course of a task. Typically, these might be used to define tasks on which resources ramp up to full efficiency as the task progresses. For example, resources assigned to a

given task might be 20% efficient at one stage of the task, 60% efficient at another stage, and 100% efficient at all subsequent stages. These efficiency rates then impact the tasks expected and calculated duration.

Each curve defined in this table is used by any entries in the Task table that reference it through their **PointsCurve** field and that have a **TaskType.EfficiencyCurveRule** of "PointsCurve". The individual point values that make up each curve are defined in the **CurvePoint** table which also has a reference to this table.

This table's records can be edited using the **Manage Efficiency Curves** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.212: PointsCurve (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this points curve.	String	
Name	A unique name for this efficiency curve. This value is referenced both by all CurvePoint records that define this curve, as well as by all Tasks that use this curve to define resource efficiency.	String	Yes

PointsCurve table – calculated and set fields

The following describes the calculated and set fields in the PointsCurve table. For an introduction to this table, and descriptions of its input fields, see "PointsCurve table" on page 484.

Table 6.213: Calculated PointsCurve (ProjMgmt) fields

Field name	Description	Data type	Key
CurvePoints	Set of CurvePoint records that reference this PointsCurve value.	Set	
Tasks	Set of Tasks records that reference this PointsCurve value.	Set	

PointsProfile table

This table contains the names of all profiles (curves) used in generating statistical forecast for new-product-introduction (NPI) and end-of-life (EOL) items.

The PointsProfile table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained through the **S&OP Forecast Item Management** workbook in RapidResponse. For more information about profiles for NPI and EOL items, see the *RapidResponse Applications Guide*.

Table 6.214: PointsProfile (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this profile.	String	
Name	A unique identifier (name) for this profile.	String	Yes

PointsProfile table – calculated and set fields

The following describes the calculated and set fields in the PointsProfile table. For an introduction to this table, and descriptions of its input fields, see "PointsProfile table" on page 485.

Table 6.215: Calculated PointsProfile (Mfg) fields

Field name	Description	Data type	Key
ForecastItem ParametersForecast Profiles	Set of forecast items this profile is applied to.	Set	
ProfilePoints	Set of forecast item parameters associated with this PointsProfile record.	Set	
Statistical Forecast Overrides	Set of part customers associated with this PointsProfile record.	Set	

Pool table

A pool represents a set of supplies and demands that are segregated from other supplies and demands for the same part. Sometimes supplies for one customer are separately managed from supplies for other customers. In such a situation, inventory pooling is used. For more information about pools, see "Understanding pools" on page 1745.

The **Pool** table supports contains the names of each inventory pool used in netting. Pools are synonymous with a contract, project, contract group or project group.

The **Pool** table must be included in the data extract even if the Pool capability is not enabled, and there must be at least one entry in this table. The first pool name loaded into the **Pool** table (and the only pool entry for implementations where Pool is not enabled) is called Unpooled and is the default pool.

When a part is not netted by pool (the **Part.MuePoolNettingType.PoolRule** control value is **Ignore**), all supplies and demands for the part are treated as being in the **Unpooled** pool.

When the value of the **Part.MuePoolNettingType.PoolRule** control value is **Net**, the **Unpooled** pool might also be used as a distinct pool.

Table 6.216: Pool (Mfg) fields

Field name	Description	Data type	Key
Description	Additional information about the pool. **Default value**: blank string	String	
Value	The name of the pool. The same pool name can be used for different parts. **Note:** The first pool name loaded into this table is called 'Unpooled'. This is the default pool. **Default value**: Unpooled	String	Yes

PoolMap table

The **PoolMap** table specifies rules to allocate demand to supplies on a PartType basis.

Table 6.217: PoolMap (Mfg) fields

Field name	Description	Data type	Key
DemandPool	The DemandPool that is mapped to the SupplyPool for processing. **Referenced table:** Pool **Reference Syntax:** DemandPool.Value	Reference	
PartType	The PartType this relationship relates to. **Referenced table:** PartType	Reference	
SupplyPool	The Supply Pool whose potential customers are being defined. **Referenced table:** Pool **Reference Syntax:** SupplyPool.Value	Reference	

PoolMapOverride table

The **PoolMapOverride** table specifies rules to allocate demand to supplies on a part basis overriding any mapping specified by the PoolMap table.

Table 6.218: PoolMapOverride (Mfg) fields

Field name	Description	Data type	Key
DemandPool	The DemandPool that is mapped to the SupplyPool for processing. **Referenced table:** Pool **Reference Syntax:** DemandPool.Value	Reference	
Part	The Part that this relationship related to. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	
SupplyPool	The Supply Pool whose potential customers are being defined. **Referenced table:** Pool **Reference Syntax:** SupplyPool.Value	Reference	

Predecessor table

The **Predecessor** table identifies dependencies between pairs of project tasks. This table has two references to the **Task** table. The **Predecessor** reference identifies a "predecessor" task that typically must come before another task, and the **Task** reference identifies a "successor" task that typically comes after the predecessor task.

This table also be used to define how these task dependencies affect each other's start or finish dates. In projects where **ProjectType.ScheduleRule** is 'FromStart' this table can be used to specify how a successor task's start or finish date is constrained by the calculated start or finish date on a predecessor task. Similarly, in projects where **ProjectType.ScheduleRule** is 'FromFinish', this table can be used to specify how a predecessor task's start or finish date is constrained by the calculated start or finish on a successor task.

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse, values can be added to this table through the **Predecessors** tab of the **Task Information** dialog box

Table 6.219: Predecessor (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this task dependency.	String	
Offset	Number of days to offset either the predecessor or successor task's calculated start or finish date, in order to calculate when the other can potentially start or finish. The usage of the offset value is determined both by the **Rule** setting on this table, and the project's **Type.ScheduleRule** setting as follows: • If **Project.Type.ScheduleRule** is 'FromStart' and **Rule** is 'FinishToStart' or 'FinishToFinish', this value is added to the predecessor task's calculated finish date. • If **Project.Type.ScheduleRule** is 'FromStart' and **Rule** is 'StartToFinish' or 'StartToStart', the this value is added to the predecessor task's calculated start date. • If **Project.Type.ScheduleRule** is 'FromFinish' and **Rule** is 'FinishToFinish' or 'StartToFinish', this value is subtracted from the successor task's calculated finish date. • If **Project.Type.ScheduleRule** is 'FromFinish' and **Rule** is 'FinishToStart' or 'StartToStart', this value is subtracted from the successor task's calculated start date. **Note:** This value is ignored if **Rule** is set 'None'.	Quantity	
Predecessor	A reference to the predecessor task. This is the task that comes first (before the successor) in the task relationship defined by this record. **Referenced table**: Task	Reference	Yes

Table 6.219: Predecessor (ProjMgmt) fields

Field name	Description	Data type	Key
Rule	Specifies how the task dependency defined by this record is used when calculating task start and finish dates (depends on the projects' **Type.ScheduleRule** setting). For example, should a task be constrained from starting until its predecessor task has finished. Valid values are: • **FinishToFinish**—if **Project.Type.Schedule Rule** is 'FromStart', then the earliest the successor task can finish is the predecessor task's calculated finish date plus any applicable offset. Else, if **Project.Type.Schedule Rule** is 'FromFinish', then the latest the predecessor task can finish is the successor task's calculated finish date less any applicable offset. • **FinishToStart**—If **Project.Type.Schedule Rule** is 'FromStart', then the earliest the successor task can start is the predecessor task's calculated finish date plus any applicable offset. Else, if **Project.Type.ScheduleRule** is 'FromFinish', then the latest the predecessor task can finish is prior to the successor task's calculated start date less any applicable offset. • **None**—the task dependency is ignored when calculating task start and finish dates. • **StartToFinish**—If **Project.Type.Schedule Rule** is 'FromStart', then the earliest the successor task can finish is the predecessor task's calculated start date plus any applicable offset. Else, if **Project.Type.Schedule Rule** is 'FromFinish', then the latest the predecessor task can finish is the successor task's calculated finish date less any applicable offset. • **StartToStart**—If **Project.Type.Schedule Rule** is 'FromStart', then the earliest the successor task can start is the predecessor task's calculated start date plus any applicable offset. Else, if **Project.Type.ScheduleRule** is 'FromFinish', then the latest the predecessor task can start is the successor task's calculated start date less any applicable offset. **Default value:** None For more information about how this field is used, see "Predecessor relationships" on page 2170.	String (Enum)	
Task	A reference to the successor task. This is the task that comes second (after the predecessor) in the task relationship defined by this record. **Referenced table:** Task	Reference	Yes

Predecessor table – calculated and set fields

The following describes the calculated and set fields in the Predecessor table. For an introduction to this table, and descriptions of its input fields, see "Predecessor table" on page 488.

Table 6.220: Calculated Predecessor (ProjMgmt) fields

Field name	Description	Data type	Key
IsRecursive	Indicates if a recursive relationship exists between the predecessor and successor task. Valid values are • **N**—relationship is not recursive. • **Y**—relationship is recursive; the Predecessor task associated with this record is a dependent on completion of the Successor task through some other predecessor/successor relationship(s) defined in this table.	Boolean	

ProcessInstance table

The ProcessInstance table identifies all of the process instances in RapidResponse. A process instance is created from a process defined in RapidResponse, and is comprised of activities that are assigned to performers.

Each record in the ProcessInstance table contains fields that hold process-level details such as name, the start date, and status.

Table 6.221: ProcessInstance (ProcOrch) fields

Field name	Description	Data type	Key
CreatedDate Time	Specifies when the process instance was created.	DateTime	
Historical	Specifies whether the process instance is historical.	Boolean	
Name	The name of this process instance.	String	Yes
OwnerId	RapidResponse user ID of the process instance owner.	String	
StartDate	The date the process instance is supposed to start. This date is determined when the process instance is created.	Date	

Table 6.221: ProcessInstance (ProcOrch) fields

Field name	Description	Data type	Key
Status	Current status of the process instance. Valid values are: • NotStarted • InProgress • Finished	Enum	
TemplateName	The name of the process from which this process instance was created.	String	
Type	The type of this process instance. The type determines the calendar used by the process instance. **Referenced table:** ProcessInstanceType	Reference	

ProcessInstance – table calculated and set fields

The following describes the calculated and set fields in the ProcessInstance table. For an introduction to this table, and descriptions of its input fields, see "ProcessInstance table" on page 491

Table 6.222: ProcessInstance (ProcOrch) fields

Field name	Description	Data type	Key
ActualFinish Date	Based on the actual finish dates of activities in this process instance. If the process instance has activities, then the date of the last activity to be finished is used. If the process instance does not have any activities, then the ExpectedFinishDate is used.	Date	
ActualStartDate	Based on the actual start dates of activities that belong to this process instance. If the process instance has activities, then the date of the first activity to be started is used. If the process instance does not have any activities, then the ExpectedStartDate is used.	Date	
ExpectedFinish Date	Based on the expected start dates of activities that belong to this process instance. If the process instance has activities, the date of the latest activity expected to finish is used. If the process instance does not have any activities, then the ExpectedStartDate is used.	Date	

Table 6.222: ProcessInstance (ProcOrch) fields

Field name	Description	Data type	Key
ExpectedStart Date	The date of the first day of the process instance. If the first day of the process instance does not fall on a date in Type.Calendar, then the next date in Type.Calendar is used.	Date	
Activities	The set of activities that belong to this process instance.	Set	
Notifications	A set of notifications for this process instance.	Set	

ProductFamily table

The **ProductFamily** table can be used to group together parts that have similarities in design, production, or usage. This allows for planning or reporting related parts at an aggregate level.

Table 6.223: ProductFamily (Mfg) fields

Field name	Description	Data type	Key
Description	Descriptive information about this product family.	String	
Value	A unique string identifier for this product family.	String	Yes

ProductFamily table – calculated and set fields

The following describes the calculated and set fields in the **ProductFamily** table. For an introduction to this table, and descriptions of its input fields, see "ProductFamily table" on page 493.

Table 6.224: ProductFamily (Mfg) fields

Field name	Description	Data type	Key
Parts	Set of Part records associated with this ProductFamily.	Set	

ProductionGroup table

The **ProductionGroup** table describes the properties for groups of parts produced using production wheels. Each production group is allocated amounts of constraint for major and minor changeovers and a maximum amount of constraint that can be consumed by the group. In addition, each group is assigned a priority to manage when the group is produced with other groups that have equal part sequences.

This table was added in Service Update 1902.

Table 6.225: ProductionGroup(Mfg) fields

Field name	Description	Data type	Key
Description	A description of the production group.	String	
MajorCleanup	The amount of constraint required to move from producing parts in this production group to producing parts in another group. It models the time required to clean the shared resource/line/machinery after producing the parts in this group. Measured in constraint units.	Quantity	
MajorSetup	The amount of constraint required to move to producing parts in this production group after having produced parts in another group. It models the time required to prepare the shared resource/line/machinery to produce parts in this group. Measured in constraint units.	Quantity	
MaximumConstraintConsumption	The maximum amount of constraint that can be consumed by this production group in a production wheel cycle. This value is expressed as a fraction. For example, if this value is set to 0.5 and there is 500 constraint available in a cycle, this production group can use no more than 250 constraint in that cycle. The maximum constraint consumption includes the constraint used to produce each part in the group, the constraint required to set up and clean up for the group, and the constraint used to changeover between each part in the group. Except for MajorCleanUp, when this limit is reached you cannot plan for parts associated with this production group in this cycle. The amount of constraint allocated to MajorCleanUp for a production group can cause this maximum to exceed it limit and allow planning to continue. Default value: 0 The counter for MaximumConstraintConsumption is reset at the start of each cycle.	Quantity	
MinorChangeOver	The amount of constraint required to change between production each part in this production group.	Quantity	
Name	A unique identifier for this production group record.	String	Yes

Table 6.225: ProductionGroup(Mfg) fields

Field name	Description	Data type	Key
Priority	The priority associated with this production group if PartSequences are equal. Supply for production is sorted using the following criteria: 1. Production wheel priority 2. Production wheel name 3. Supply demand date 4. Part sequence 5. Production group priority	Integer	
Site	The site associated with this production group. **Referenced table:** Site	Reference	Yes

ProductionGroup table – calculated and set fields

The following describes the calculated and set fields in the **ProductionGroup** table. For an introduction to this table, and descriptions of its input fields, see "ProductionGroup table" on page 493

Table 6.226: ProductionGroup(Mfg) fields

Field name	Description	Data type	Key
PartSources	Set of PartSource records associated with this production group.	Set	

ProductionRecurrence table

The **ProductionRecurrence** table defines how often a part can be made in a production wheel. The recurrence can be assigned to a part or to the segment a part is associated with. The calendar used to set the recurrence is the same calendar assigned to the selected production wheel.

You can offset production cycles to stagger when parts are built by adding an offset to the recurrence. For example, if the changeover cost between two parts (Part A and Part B) in a production group is high, you might want to produce them on the same production wheel but not in the same cycles of that wheel. The table below illustrates an example of how you can use an offset to set a production pattern for each part that produces them in separate cycles.

Part	Recurrence	Offset	Production Pattern
A	2	0	Cycles 1, 3, 5, and so forth
B	2	1	Cycles 2, 4, 6, and so forth

This table was added in Service Update 1902.

Table 6.227: ProductionRecurrence (Mfg) fields

Field name	Description	Data type	Key
Description	A description of the production recurrence.	String	
Offset	The number of cycles production of a part is offset by. An offset of 0 starts production of a part in cycle 1. An offset of 1 starts production of a part in cycle 2 and an offset of 2 starts production of a part in cycle 3 and so forth. For example, if Part A has a recurrence of 3 and an offset of 1, Part A can still be produced in every third cycle of a production wheel but it starts 1 cycle later. If the production wheel has 9 cycles, Part A can be built in cycles 2, 5, and 8.	Integer	
Recurrence	Identifies which cycles in the production wheel can produce the selected part. For example, if Part A is assigned a recurrence of 3 it can be produced in every third cycle of a production wheel. Part A can be built in cycle 1, 4, 7, 10, and so forth. If a part is assigned a recurrence of 1 that part can be produced in every cycle of the production wheel. Default value =1 **Note**: Negative values and zero are treated as the default value of 1.	Integer	
Value	A unique identifier for this production recurrence record.	String	Yes

ProductionRecurrence table – calculated and set fields

The following describes the calculated and set fields in the **ProductionRecurrence** table. For an introduction to this table, and descriptions of its input fields, see "ProductionRecurrence table" on page 495

Table 6.228: ProductionRecurrence(Mfg) fields

Field name	Description	Data type	Key
PartSources	Set of PartSource records associated with this production recurrence.	Set	
Segments	Set of Segment records associated with this production recurrence.	Set	

ProductionWheel table

The **ProductionWheel** table defines properties for production wheels such as wheel durations, cycle lengths, and order priority.

This table was added in Service Update 1902.

Table 6.229: ProductionWheel(Mfg) fields

Field name	Description	Data type	Key
Calendar	A reference to the calendar used by the selected production wheel. **Referenced table:** Calendar		
CalendarInterval	The number of calendar intervals in a production cycle. When combined with the interval it gives the size of the cycle. For example, a production wheel with a Calender of week and a CalendarInterval of 2 makes the cycle size 2 weeks. Default value: 1	Integer	
Description	A description of this production wheel.	String	
EffectiveInDate	The date when you want the production wheel cycle to start. This date cannot be Past, Future, or Undefined. Along with the EffectiveOutDate this field defines the period over which this production wheel record is effective.	Date	
EffectiveOutDate	The date when you want the production wheel to stop. If this date falls in the middle of a cycle, the cycle will complete first. This date cannot be Past or Undefined. Along with EffectiveInDate this field defines the period over which this production wheel record is effective.	Date	
FirstEffectiveDate	The specific date when this production wheel can start. This calculated date is calibrated using the calendar for the selected production cycle. For example, if the first effective date is January 11, 2019 and the cycle uses a week calendar (Monday to Sunday), then the first date that the wheel can start is Monday, January 7, 2019.	Date (calculated)	
LastEffectiveDate	The specific date when this production wheel can stop. This calculated date is calibrated using the calendar for the selected production cycle. For example, if the last effective date is January 23, 2019 and the cycle count is set to 2, then the last date the wheel can stop is February 3, 2019.	Date (calculated)	
Name	A unique identifier for this production wheel record.	String	Yes

Table 6.229: ProductionWheel(Mfg) fields

Field name	Description	Data type	Key
Priority	The order this production wheel is processed in when there are multiple production wheels in the same time frame. This priority is also used to select which production wheel to use when a part has multiple effective part sources in different production wheels. When priorities are equal for the production wheels running in the same time frame, then the production wheel name is used to determine the order.	Integer	
Type	A reference to the control table that contains the rules and values used to manage production wheels in RapidResponse. **Referenced table:** ProductionWheelType	Reference	
Site	The site associated with this production wheel. **Referenced table:** Site	Reference	Yes

ProductionWheel table – calculated and set fields

The following describes the calculated and set fields in the **ProductionWheel** table. For an introduction to this table, and descriptions of its input fields, see "ProductionWheel table" on page 496

Table 6.230: ProductionWheel(Mfg) fields

Field name	Description	Data type	Key
PartSequences	Set of PartSequence records associated with this production wheel.	Set	
PartSources	Set of PartSource records associated with thisproduction wheel.	Set	
ProductionWheelConstraints	Set of ProductionWheelConstraint records associated with this production wheel.	Set	

ProductionWheelConstraint table

The **ProductionWheelConstraint** table describes the constraint parameters that affect production wheels.

This table was added in Service Update 1902.

Table 6.231: ProductionWheelConstraint(Mfg) fields

Field name	Description	Data type	Key
Constraint	A reference to constraint associated with this ProductionWheelConstraint record. **Referenced table**: Constraint	Reference	
MinimumConstraintConsumption	The minimum fraction of constraint that can be consumed in a cycle. This value is a target you set. Default value: 0 **Note:** Constraint used in building parts and in major and minor changeovers is counted in the total amount of constraint consumed in the cycle.	Quantity	
ProductionWheel	A reference to the production wheel associated with this ProductionWheelConstraint record. **Referenced table**: ProductionWheel	Reference	Yes

ProfilePoint table

This table contains data points for profiles (curves) stored in PointProfile table and used in generating statistical forecast for new-product-introduction (NPI) and end-of-life (EOL) items. Each point contains an index (for sorting), quantity, and as a reference to the PointProfile record it applies to.

This table supports Sales and Operations Planning in RapidResponse. Values in this table are maintained through the **S&OP Forecast Item** workbook in RapidResponse. For more information about profiles for NPI and EOL items, see the the *RapidResponse Applications Guide*.

Table 6.232: ProfilePoint (Mfg) fields

Field name	Description	Data type	Key
Id	A unique identifier for this point. When profile points are added in RapidResponse, the Index value is converted to a string and stored in this field.	String	Yes
Index	Index of this point on the profile. Used for sorting.	Integer	
Profile	A reference to the profile this point pertains to. **Referenced table:** PointsProfile	Reference	Yes
Quantity	The quantity of this point in the profile.	Quantity	

Project table

The **Project** table identifies all named projects in RapidResponse. A project is typically comprised of many related tasks or work items that together achieve a definable end goal (such as, delivering a particular good or service). The tasks that make up a given project are those that reference it from the **Task** table. Each record in the **Project** table contains input fields holding project-level details such as name, default start or finish date, project manager, status, and so on. In addition, this table reports calculated project-level details such as the start or finish dates determined by RapidResponse, and any resource, material, or penalty costs applied to the project.

This table's records can be edited using the **Project Manager** workbook included with RapidResponse.

Table 6.233: Project (ProjMgmt) fields

Field name	Description	Data type	Key
Baseline	Indicates if this project is a baseline. After a project has been set as a baseline in RapidResponse, certain calculated fields in this table as well as the **Task** table always return the values they held at the point when the project was baselined. Valid values are: • **N**—project is not a baseline. • **Y**—project is a baseline. **Note:** For more information about setting projects as baselines, see the *RapidResponse User Guide*.	Boolean	
BonusDate	The date on or before which bonuses can potentially be applied to this project. Bonuses are calculated based on the referenced **BonusSchedule** and **ProjectType** values, and can be applied to projects that finish on or before this date (bonuses can accumulate between **CalcFinishDate** and **BonusDate**).	Date	
BonusSchedule	The name of the bonuses schedule used for calculating project bonuses. Together with the rules defined on the project type, this determines how bonuses are calculated at the project level. **Referenced table:** BonusSchedule (Nullable)	Reference	
Customer	The customer, if any, associated with the project. **Referenced table:** Customer (this reference is nullable)	Reference	
Description	A description of this project.	String	

Table 6.233: Project (ProjMgmt) fields

Field name	Description	Data type	Key
FinishDate	An input finish date for the project. If this project has a **ProjectType.ScheduleRule** of "FromFinish", this is the latest possible finish date for any task in the project (used as the default finish date for the last task in the project). RapidResponse also calculates a realistic project finish date as reported in the **CalcFinishDate** field.	Date	
Group	The project group, if any, this project belongs to. **Referenced table:** ProjectGroup (this reference is nullable)	Reference	
HoursPerDay	Standard hours per day that can be worked on task's in this project.	Quantity	
Manager	The project manager assigned responsibility for this project. **Referenced table:** ProjectManager (this reference is nullable)	Reference	
Name	A unique name for this project. This is value is referenced from the **Task.Project** field.	String	Yes
PenaltyDate	The date from which penalties can potentially accumulate to this project. Penalty costs are calculated based on the referenced **PenaltySchedule** and **ProjectType** values, and can be applied between this date and **CalcFinishDate**. For more information about how this field is used, see "Penalties" on page 2191.	Date	
PenaltySchedule	The name of the penalty schedule used for calculating project penalties. Together with the rules defined on the project type, this determines how penalties are calculated at the project level. **Referenced table:** PenaltySchedule (this reference is nullable)	Reference	

Table 6.233: Project (ProjMgmt) fields

Field name	Description	Data type	Key
RateAdjustment	An optional percentage adjustment to be applied to the standard and overtime rates charged for any resources assigned to tasks in this project. For example, all resources assigned to a specific customer project might be discounted. If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .05 indicates a 5 percent premium while -.1 indicates a 10 percent discount. Note that if there is an adjustment specified at a more granular level in the project, then that value is used instead of this one. For example, adjustments can also be specified for individual resources and/or tasks in the project. For more information, see "Resource rate adjustments" on page 2189.	Quantity	
RunDate	The project's current working or "as of" date. This date is meant to separate past activities from future activities in the project schedule. Typically, it might be set to today's date or a date in the very recent past. When **ProjectType.PlanningRule** is set to "DateDriven", this value is also used in scheduling tasks and defines the earliest date on which any new work can commence.	Date	
RunDateTimeOfDay	Specifies if the value provided in the **RunDate** field should be interpreted as the start or the end of the given date, and hence specific point from which new task work can be scheduled. Valid values are: • **StartOfDay**—RunDate applies to the start of the given day. • **EndOfDay**—RunDate applies to the end of the given day. • **Default value**: StartOfDay	String (Enum)	
Site	The site associated with this project. **Referenced table:** Core::Site	Reference	Yes
StartDate	An input start date for the project. If this project has a **ProjectType.ScheduleRule** of "FromStart", this is the earliest possible start date for task in the project (used as the default start date for the first task in the project). RapidResponse also calculates a realistic project start date as reported in the **CalcStartDate** field.	Date	

Table 6.233: Project (ProjMgmt) fields

Field name	Description	Data type	Key
Status	The status of the project. Typically, used to indicate the project's level of completion. **Referenced table:** ProjectStatus	Reference	
Type	A value associated with processing rules that control certain calculations at the project level, such as how project dates and penalties are determined. **Referenced table:** ProjectType	Reference	

Project table – calculated and set fields

The following describes the calculated and set fields in the Project table. For an introduction to this table, and descriptions of its input fields, see "Project table" on page 500.

Table 6.234: Calculated Project (ProjMgmt) fields

Field name	Description	Data type	Key
ActualMaterial Cost	The rolled up actual material costs from all tasks in the project that are assigned to specific demand and/or supply orders.	Money	
ActualTaskCost	The rolled up actual costs associated with completing this task that aren't reflected in the actual resource or material costs. Based on values defined in the **Task.ActualCost** field for tasks in this project.	Money	
Bonus	Bonuses earned at the project level. Project bonuses can only be earned on dates between **CalcFinishDate** and **BonusDate**, where the **BonusSchedule** field references a valid bonus schedule, and the referenced **ProjectType.BonusRule** value is other than "None". For more information about how this field is calculated, see "Bonuses" on page 2196. **Note:** Any task-level bonuses are added to this value and together reported in the **Project.CumulativeBonus** field.	Money	
CalcFinishDate	The calculated finish date for the project. Set to the latest finish date of any task within the project.	Date	
CalcStartDate	The calculated start date for the project. Set to the earliest start date of any task within the project.	Date	

Table 6.234: Calculated Project (ProjMgmt) fields

Field name	Description	Data type	Key
CalcState	The calculated state of progress for this project (based on the **CalcState** values found across all of its tasks). Valid values are: • **Completed**—all tasks in the project are finished. • **InProgress**—at least one task in the project has started but not yet finished. • **NotStarted**—no tasks in the project have started.	String (Enum)	
Cumulative Bonus	Total value of all bonus earned by this project. This field sums all bonuses applied to tasks throughout the projects and adds that to the value in the **Bonus** field.	Money	
Cumulative Penalty Cost	Total penalty costs applicable to this project. This field sums all penalty costs incurred by tasks throughout the projects and adds that to the value in the **PenaltyCost** field.	Money	
MaterialCost	The rolled up planned material costs from all tasks in the project that have material part links defined.	Money	
MaterialRevenue	The rolled up material revenue from all tasks in the projects that demand links defined.	Money	
PenaltyCost	Penalty costs incurred at the project level. Penalty costs can only be incurred on project dates between **PenaltyDate** and **CalcFinishDate**, where the **PenaltySchedule** field references a valid penalty schedule, and the referenced **ProjectType.PenaltyRule** value is other than "None". For more information about how this field is calculated, see "Penalties" on page 2191. **Note:** Any task-level penalties are added to this value and together reported in the **Project.CumulativePenaltyCost** field.	Money	
ResourceActual FixedCost	Actual fixed labor costs incurred during project execution from all resource assignments made to tasks in this project.	Money	
ResourceActual OvertimeCost	Actual overtime labor costs incurred during project execution from all resources working on tasks in excess of their maximum hours per day.	Money	
ResourceActual StandardCost	Actual standard labor costs incurred during project execution from all resources working on tasks up to their maximum hours per day.	Money	

Table 6.234: Calculated Project (ProjMgmt) fields

Field name	Description	Data type	Key
ResourceFixed Cost	The rolled up value of all one-time fixed costs incurred from each assignment of a resource to a task in this project.	Money	
Resource Overtime Cost	The rolled up value of all overtime costs incurred by resources assigned to tasks in this project.	Money	
ResourceStandard Cost	The rolled up value of all standard hour working costs incurred by resources assigned to tasks in this project.	Money	
TaskCost	The rolled up planned costs at the task level, that aren't reflected in resource or material costs, from all tasks in this project. Miscellaneous planned cost at the task level is defined in the **Task.Cost** field.	Money	
TaskRevenue	The rolled up revenue associated with all tasks in this project.	Money	
CriticalPath	Set of CriticalPath records that identify all task's and related items on the project's critical path.	Set	
Tasks	Set of Task records that contain all tasks belonging to the project.	Set	

ProjectedLeadTime table

The **ProjectedLeadTime** table contains lead times projected by the Machine Learning server for a set of **ItemSource** records. The projected lead times are processed based on the **AutomatedLeadTimeType** settings for the **ItemSource**. For more information, see "AutomatedLeadTimeType table" on page 678.

The accepted lead time changes are copied to the **TimePhasedPartSourceLeadTime** table, either because they should be accepted automatically or because the user has accepted them manually. For more information, see the TimePhasedPartSourceLeadTime table.

This table was added in Version 2016.2, Service Update 21.

Table 6.235: ProjectedLeadTime (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	The effective date (inclusive) since when the lead time value is taken into consideration.	Date	Yes
FixedLeadTime	The projected fixed lead time.	Quantity	

Table 6.235: ProjectedLeadTime (Mfg) fields

Field name	Description	Data type	Key
ItemSource	The **ItemSource** (Part + Source) associated with the lead times. **Referenced table:** ItemSource **Reference syntax:** ItemSource.Part, ItemSource.Source	Reference	Yes
LowerBound	The lower bound of the confidence interval.	Quantity	
RevisedLeadTime	The revised lead time used to override the projected lead time. **Note:** This value should be non-negative.	Quantity	
UpperBound	The upper bound of the confidence interval.	Quantity	

ProjectedYield table

The **ProjectedYield** table contains yield values projected by the Machine Learning server for a set of **ItemSource** records. The projected yields are processed based on the **AutomatedYieldType** settings for the **ItemSource**. For more information, see "AutomatedYieldType table" on page 683.

The accepted yield changes are copied to the **TimePhasedYield** table, either because they should be accepted automatically or because the user has accepted them manually. For more information, see the TimePhasedYield table.

Table 6.236: ProjectedYield (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	The effective date (inclusive) since when the yield value is taken into consideration.	Date	Yes
Yield	The projected yield as a fraction in the range 0..1.	Quantity	
ItemSource	The **ItemSource** (Part + Source) associated with the yields. **Referenced table:** ItemSource **Reference syntax:** ItemSource.Part, ItemSource.Source	Reference	Yes
LowerBound	The lower bound of the confidence interval.	Quantity	
RevisedYield	The revised yield used to override the projected yield. **Note:** This value should be non-negative.	Quantity	
UpperBound	The upper bound of the confidence interval.	Quantity	

ProjectedYield table – calculated and set fields

The following describes the calculated and set fields in the **ProjectedYield** table.

Table 6.237: Calculated HistoricalReceipt (Mfg) fields

Field name	Description	Data type	Key
CalcEffectiveYield	Calculated field. Reports the projected Yield value in "user-facing" units (as defined by **ItemSource.CalcYieldUsageRule**)	Quantity	
CalcEffectiveUpperBound	Reports the upper bounds of the confidence intervals, converted into "user-facing" units (as defined by **ItemSource.CalcYieldUsageRule**)	Quantity	
CalcEffectiveLowerBound	Reports the lower bounds of the confidence intervals, converted into "user-facing" units (as defined by **ItemSource.CalcYieldUsageRule**)	Quantity	

ProjectGroup table

The **ProjectGroup** table defines group names that can be assigned to projects. These are meant to categorize and group similar types of projects together, and can be used to aid in project-level tracking and filtering.

This table's records can be edited using the **Project Groups** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.238: ProjectGroup (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this project group.	String	
Name	A string value used to identify this project group. This value is referenced from the Group field on the Project table.	String	Yes

ProjectGroup table – calculated and set fields

The following describes the calculated and set fields in the ProjectGroup table. For an introduction to this table, and descriptions of its input fields, see "ProjectGroup table" on page 507.

Table 6.239: Calculated ProjectGroup (ProjMgmt) fields

Field name	Description	Data type	Key
Projects	Set of Project records that reference this ProjectGroup value.	Set	

ProjectManager table

The **ProjectManager** table identifies all projects managers that can be assigned responsibility for projects.

This table's records can be edited using the **Project Managers** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.240: ProjectManager (ProjMgmt) fields

Field name	Description	Data type	Key
Name	The name of the project manager. This value is referenced from the Manager field on the **Project** table.	String	Yes

ProjectManager table – calculated and set fields

The following describes the calculated and set fields in the ProjectManager table. For an introduction to this table, and descriptions of its input fields, see "ProjectManager table" on page 508.

Table 6.241: Calculated ProjectManager (ProjMgmt) fields

Field name	Description	Data type	Key
Projects	Set of Project records that reference this ProjectManager value.	Set	

ProjectStatus table

The **ProjectStatus** table defines status values that can be assigned to projects. These values are meant indicate a project's level of completion and can be used in filtering down to specific projects of interest. For example, projects might be identified as Planned, Open, Delayed, or Completed.

This table's records can be edited using the **Project Status** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.242: ProjectStatus (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this project status type.	String	
Value	A string value used to identify this project status type. This value is referenced from the Status field on the Project table.	String	Yes

ProjectStatus table – calculated and set fields

The following describes the calculated and set fields in the ProjectStatus table. For an introduction to this table, and descriptions of its input fields, see "ProjectStatus table" on page 508.

Table 6.243: Calculated ProjectStatus (ProjMgmt) fields

Field name	Description	Data type	Key
Projects	Set of Project records that reference this ProjectStatus value.	Set	

RangeOfCoveragePartOverride table

This table is applicable only to range of coverage parts; that is, parts where the **SafetyStockQuantityRule** field on either the **SafetyStockPolicy** or **PartType** table (as appropriate) is set to either "RangeOfCoverage", "RangeOfCoverageLeadTime" or "RangeOfCoverageTimePhased". It allows for certain parameters to be specified on a part by part basis overriding the settings in the **RangeOfCoverage** table.

By default, parts take their range of coverage parameters from the RangeOfCoverage table (through the **SafetyStockPolicy.RangeOfCoverage** reference). However, for any part referenced from a record in this table, the **RangeOfCoveragePartOverride** parameters are used in place of the corresponding fields in the **RangeOfCoverage** table.

Note: For more information on the RangeOfCoverage table, see "RangeOfCoverage table" on page 855.

Table 6.244: RangeOfCoveragePartOverride (Mfg) fields

Field name	Description	Data type	Key
Average Requirement IntervalCount	The number of **RangeOfCoverage.Average RequirementIntervals** to consider demand from when calculating average daily demands. **Default value:** 0	Integer	
Average Requirement Interval LeadTimeCount	An optional parameter that is applicable only when **SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime". It represents a factor of lead time that can be used to extend the period over which demands are accumulated when calculating the average daily demand. The value provided in this field is then multiplied by the longest lead time for the part expressed as a number of **RangeOfCoverage.AverageRequirement Intervals** (rounded up, if necessary). The result of that calculation is then added to the value in **AverageRequirementIntervalCount** to determine the total number of calendar units over which demands should be accumulated. **Default value:** 0	Integer	
Average Requirement IntervalOffset	A parameter allowing the average daily demand calculations to be offset a specified number of **RangeOfCoverage.AverageRequirement Intervals** from the start of the **RangeOfCoverageInterval**. For example, if **RangeOfCoverage.AverageRequirement Interval** is set to "Weekly" and this value is set to "3", then the average daily demand requirements will be calculated starting 3 weeks after the start of the **RangeOfCoverageInterval**. **Default value:** 0	Integer	
Part	A reference to the part this set of range of coverage parameters should apply to. If a reference exists to a given part, then all range of coverage parameters in this table override the corresponding fields in the **RangeOfCoverage** table. However, all parts not referenced from this table continue to take all their range of coverage parameters from the **RangeOfCoverage** table. **Referenced table:** Part **Reference Syntax**: Part.Name, Part.Site	Reference	Yes

Table 6.244: RangeOfCoveragePartOverride (Mfg) fields

Field name	Description	Data type	Key
ThresholdFactor Lower	A factor used along with the **Part.RangeOfCoverageBuffer field** to set the lower threshold limit for a given range of coverage interval as follows: • **LowerLimit** = Calculated SafetyStockLevel + (Part.RangeOfCoverageBuffer * ThresholdFactorLower) When the safety stock level is calculated for a range of coverage interval, this limit and an upper limit (using the **ThresholdFactorUpper** value) are also calculated. If the existing safety stock level (from the previous range of coverage interval) falls within these limits, then the new calculated level is discarded and the existing safety stock level remains effective. This is intended to avoid making minor changes to the safety stock level which might have only insignificant benefit. If, however, the existing safety stock level falls outside these limits, then the new calculated safety stock level becomes effective. **Default value:** 0	Quantity	
ThresholdFactor Upper	A factor used along with the **Part.RangeOfCoverageBuffer** field to set the upper threshold limit for a given range of coverage interval as follows: • **UpperLimit** = Calculated SafetyStockLevel + (Part.RangeOfCoverageBuffer * ThresholdFactorUpper) When the safety stock level is calculated for a given range of coverage interval, this limit and a lower limit (using the **ThresholdFactorLower** value) are also calculated. If the existing safety stock level (from the previous range of coverage interval) falls within these limits, then the new calculated level is discarded and the existing safety stock level remains effective. This is intended to avoid making minor changes to the safety stock level which might have only insignificant benefit. If, however, the existing safety stock level falls outside these limits, then the new calculated safety stock level becomes effective. **Default value:** 0	Quantity	

ReferenceForecastDimension table

This table was originally used to identify the dimensions of reference forecasts supporting Sales and Operations Planning in RapidResponse. Note that reference forecasts are no longer used in Sales and Operations Planning, however this table supports assumption functionality (is referenced by the **Assumption** table).

Table 6.245: ReferenceForecastDimension (Mfg) fields

Field name	Description	Data type	Key
Description	A description of the reference forecast dimension	String	
Id	An identifier for the reference forecast dimension. Data in this field should identify the hierarchy and level the dimension is created for.	String	Yes

ReferenceForecastTarget table

This table originally used to identify the hierarchy element that a dimension of a reference forecast is applied to and supported Sales and Operations Planning in RapidResponse. Note that reference forecasts are no longer used in Sales and Operations Planning, however, this table supports assumption functionality (is referenced by the **Assumption** table).

Table 6.246: ReferenceForecastTarget (Mfg) fields

Field name	Description	Data type	Key
Dimension	The dimension the reference forecast target is created under. **Referenced table:** ReferenceForecastDimension **Reference Syntax:** Dimension.Id	Reference	
Id	The hierarchy element the reference forecast is created for. Values in this field can refer to the full path of the hierarchy element (each element at each level between the top level and the target, separated by | characters), or to the value of the target element, depending on how the hierarchy is specified. Typically, the full path should be used if a hierarchy element has more than one parent in the hierarchy. For example, in a hierarchy that has levels for division and product family, if the same product family is carried by two different divisions, the full path should be used in this field.	String	Yes

ReferenceForecastTarget table – calculated and set fields

The following describes the calculated and set fields in the **ReferenceForecastTarget** table. For an introduction to this table, and descriptions of its input fields, see "ReferenceForecastTarget table" on page 512.

Table 6.247: Calculated ReferenceForecastTarget (Mfg) fields

Field name	Description	Data type	Key
Assumptions	Set of assumptions associated with this record.	Set	

ReferencePart table

The **ReferencePart** table lists alternate names for parts. The alternate name could be an old name for the part, the name of a product line, or the name used by a different site or a contract manufacturer. The relationship between this table and the **Part** table varies depending on how it is used. For example, if this table is used to group parts together, many parts will have the same reference part. However, if this table is used to hold old names for parts, each part will have a unique reference part.

This table is primarily used to show relationships between part names for data publishers and subscribers. It holds the name of the part as used by the subscriber and is referenced by the **Part.ReferencePart** field (**Part.Name** holds the name of the part as used by the publisher). This allows a subscriber to see part data in a context that makes sense to them.

This table can also be used in any of the following ways:

- to hold a generic part name to be used by different company divisions that identify the same part using different naming conventions.
- to allow multi-site part analysis by grouping the parts from different locations into a common reference part.
- to group multiple parts into a common product line.
- to maintain and display old part names if a new part naming convention has been adopted.
- left empty if not being used.

Table 6.248: ReferencePart (Mfg) fields

Field name	Description	Data type	Key
Value	The name of the reference part.	Reference	

ReferencePart table – calculated and set fields

The following describes the calculated and set fields in the **ReferencePart** table.

Table 6.249: Calculated ReferencePart (Mfg) fields

Field name	Description	Data type	Key
Parts	The set of parts that refer to this reference part.	Set	

Region table

The **Region** table can be used to identify geographic entities for grouping sites, customers, and suppliers. For example, it might hold values identifying states, countries, or sales territories. A secondary level of regional identifiers is available through the **RegionGroup** reference.

Table 6.250: Region(Mfg) fields

Field name	Description	Data type	Key
Id	A unique id for this region. For example, NE-US.	String	Yes
Name	The full descriptive name for this region. For example, Northeast United States.	String	
RegionGroup	A reference to an additional regional group this region belongs to. For example, if the **Id** field is used to identify countries, this field might indicate the international region each country belongs to. **Referenced table:** RegionGroup (Nullable)	Reference	

Region table – calculated and set fields

The following describes the calculated and set fields in the **Region** table. For an introduction to this table, and descriptions of its input fields, see "Region table" on page 514.

Table 6.251: Calculated Region(Mfg) fields

Field name	Description	Data type	Key
Customers	The set of customers associated with this region.	Set	
Sites	The set of sites associated with this region.	Set	
Suppliers	The set of suppliers associated with this region.	Set	

RegionGroup table

The **RegionGroup** table identifies geographic entities for grouping regions. For example, if the **Region** table identifies individual states or sales territories, then this table might be used to identify the country or international region to which those regions belong.

Table 6.252: RegionGroup (Mfg) fields

Field name	Description	Data type	Key
Id	A unique identifier for this regional group. For example, NA.	String	Yes
Name	The full descriptive name for this regional group. For example, North America.	String	

RegionGroup table – calculated and set fields

For an introduction to this table, and descriptions of its input fields, see "RegionGroup table" on page 514.

Table 6.253: Calculated RegionGroup (Mfg) fields

Field name	Description	Data type	Key
Region	The set of regions	Set	

RegressionModel table

The **RegressionModel** table stores the names of regression models that are defined for use with multiple linear regression in statistical forecasting. The records in this table are referenced by records in the RegressionModelTerm table and the values in both tables represent a regression formula.

The **RegressionModel** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 6.254: RegressionModel (Mfg) fields

Field name	Description	Data type	Key
Name	The unique name for this regression model.	String	Yes
Description	A description of this regression model.	String	

Regression Model table – calculated and set fields

The following describes the calculated and set fields in the **RegressionModel** table.

Table 6.255: Set fields RegressionModel (Mfg) fields

Field name	Description	Data type	Key
ForecastItemParameters	Set of forecast items for this regression model.	Set	
RegressionModelTerms	Set of historical demands for the forecast item.	Set	

RegressionModelTerm table

The records stored in this table contain information about the model terms that are associated with regressors, also known as predictors or independent variables, that are used with multiple linear regression in statistical forecasting. The records in this table reference the RegressionModel table and the values in both tables represent a regression formula.

The **RegressionModelTerm** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 6.256: RegressionModelTerm (Mfg) fields

Field name	Description	Data type	Key
Lag	The number of past **ForecastItemParametersType.IntervalsCalendar** periods that are specified for calculating the lagged regressor values in the statistical forecast. Negative values will be ignored because they violate causality. **Default value**: 0	Quantity	
Model	A reference to the regression model that this model term is used in. **Referenced table**: RegressionModel	Reference	Yes
Name	The unique name used to identify this model term.	String	Yes
ProcessingRule	This value indicates if a regressor is being used in the statistical forecast calculation for a forecast item. Valid values are: **Ignore**–The regressor is not being used to calculate the statistical forecast for a forecast item. **Use**–The regressor is being used to calculate the statistical forecast for a forecast item. **Default value**: Use	String (Enum)	
Regressor	A reference to the regressor that is associated with this model term. **Referenced table**: Regressor	Reference	

Regressor table

The **Regressor** table stores records for regressors that are used with multiple linear regression in statistical forecasting. Multiple linear regression is a statistical forecasting method that uses multiple regressors to predict the future demand of a forecast item. A regressor, which is also known as an independent variable, causal factor, or predictor, is a factor that may drive the demand for a forecast item. You can use the regressors to help determine the future demand for the forecast item in the statistical forecast. For example, you could use product discounts, holidays, or weather values as regressors.

This table is referenced by the RegressionModelTerm and RegressorValue tables.

The **Regressor** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 6.257: Regressor (Mfg) fields

Field name	Description	Data type	Key
AttachToForecastItemParameters	This field is in the Solutions namespace.	Boolean	
Name	The unique name for this regressor.	String	Yes
Description	A description that was provided about this regressor.	String	
Category	A reference to the category to which this regressor belongs. Categories are used to organize regressors in the S&OP Regressor Management workbook. **Referenced table:** RegressorCategory (Nullable)	Reference	

Regressor table – calculated and set fields

The following describes the calculated and set fields in the **Regressor** table.

Table 6.258: Set fields Regressor (Mfg) fields

Field name	Description	Data type	Key
RegressionModelTerms	Set regression model terms for this regressor.	Set	
RegressorValues	Set of regressor values for this regressor.	Set	

RegressorCategory table

This table is used to group regressors into categories. Regressors are used with multiple linear regression in statistical forecasting. You can use categories to organize regressors in the S&OP Regressor Management workbook.

The RegressorCategory table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 6.259: RegressorCategory (Mfg) fields

Field name	Description	Data type	Key
Name	The unique name that identifies a group of regressors.	String	Yes
Description	A description that was provided for the regressor category.	String	

RegressorCategory table – calculated and set fields

The following describes the calculated and set fields in the **RegressorCategory** table.

Table 6.260: Set fields RegressorCategory (Mfg) fields

Field name	Description	Data type	Key
Regressors	Set of regressors that belong to this category.	Set	

RegressorValue table

The **RegressorValue** table stores individual values for a regressor on a given date. Regressors are used with multiple linear regression in statistical forecasting. A regressor, which is also known as an independent variable, causal factor, or predictor, is a factor that may drive the demand for a forecast item. The value of a regressor depends on the type of regressor that you are using in the statistical forecast. For example, you could use values that represent past product discounts, holidays, or temperatures, as long as they are know to drive the demand for a product.

The values in the **RegressorValue** table are used to calculate **ForecastItemParametersRegressor.Quantity**, which is bucketed using **ForecastItemParametersActual.Date**. The **RegressorValue.Date** does not need to match **ForecastItemParametersActual.Date**. You can also have multiple **RegressorValue** records in the interval between two **ForecastItemParametersActual** dates. However, if within this interval there are dates with no **RegressorValue** records, a **RegressorValue.Quantity** remains in effect starting at **RegressorValue.Date** until one day prior to **RegressorValue.Date** of the next available **RegressorValue** record. For more information, about how **ForecastItemParametersRegressor.Quantity** is calculated, see "ForecastItemParametersRegressor table " on page 1067 and "Multiple linear regression calculations" on page 2047.

If there is no **RegressorValue** record associated with a date that is equal to, or earlier than the first **ForecastItemParametersActual.Date** in the historical data series, then a **RegressorValue.Quantity** of zero is applied.

It is also important to note that the values stored in this table are not only historical values. The values within the following range are taken into account in the calculation:

RunDate - **ForecastItemParameters.HistoricalIntervalCount** to **RunDate** + **ForecastItemParameters.ForecastIntervalCount**.

The **RegressorValue** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 6.261: RegressorValue (Mfg) fields

Field name	Description	Data type	Key
Regressor	A reference to the regressor that is associated with this value. **Referenced table:** Regressor (Nullable)	Reference	Yes
Date	The date to start using this regressor value record.	Date	Yes
Quantity	The quantity of this regressor on a given date.	Quantity	

Resource table

The **Resource** table identifies all resources that can be assigned to complete work on project tasks. Each resource's available working time, as well as the fixed and hourly costs associated with its task assignments, are also specified here. However, if time-phased resource costs are required, then the values in this table can be overridden by date by adding records to the **ResourceCosts** table. Similarly, if time-phased resource availabilities are required, then the values in this table can be overridden by date by adding records to the **ResourceAvailable** table.

This table's records can be edited using the **Resources** worksheet in the **Project Resource Master Data** workbook included with RapidResponse.

Table 6.262: Resource (ProjMgmt) fields

Field name	Description	Data type	Key
DataDate	The first date on which this resource's load details can be generated in the **ResourceDataByDate** table. The referenced **ResourceType**.**Calendar** and **ResourceType.ReportLimit** values then together specify how long after this date that resource load details will be generated in the **ResourceDataByDate** table.	Date	
FixedCost	A set cost incurred each time the resource is assigned to a task, and independent of the amount of time actually spent on the task. Based on the **TaskType.FixedCostAccrualRule** setting for a given task, this cost can be incurred either when the task starts, the task finishes, or spread evenly across all days the resource is assigned to the task.	Money	
Group	A reference to the group this resource belongs to. **Referenced table:** ResourceGroup (Nullable)	Reference	
HoursPerDay	Standard number of hours available per working day for one unit of this resource. The number of hours specified here equates to a single unit as specified in the **MaximumUnits** field. For example, if **MaximumUnits** is 1 and this value is 8, then the resource can do a maximum of 8 hours work per day before it is seen as over allocated and overtime costs incurred.	Quantity	
Initials	Initials belonging to this resource. Might be used to simplify or shorten the reporting of multiple resource names assigned to a task.	String	
MaximumUnits	Maximum number of units available per working day for this resource, where each unit corresponds to the value in the **HoursPerDay** field. For example, if **HoursPerDay** is 8 and this value is 1, then the resource can do a maximum of 8 hours work per day before it is seen as over allocated and overtime costs incurred.	Quantity	
Name	A string value used to identify this resource (for example, it is referenced when assigning resources to tasks).	String	Yes
OvertimeRate	Hourly overtime rate for this resource. Any load generated against this resource on a given day, beyond the maximum units specified, is billed at this rate. For example, if **HoursPerDay** is 6 and **MaximumUnits** is 1.5, this rate is applied to all hours worked in excess of nine per day by this resource.	Money	

Table 6.262: Resource (ProjMgmt) fields

Field name	Description	Data type	Key
Site	The site associated with this resource. **Referenced table:** Site	Reference	Yes
StandardRate	Standard hourly rate for this resource. Any load generated against this resource on a given day, up until the maximum units specified, is billed at this rate. For example, if **HoursPerDay** is 8 and **MaximumUnits** is 1.25, this rate is applied to each of the first ten hours worked per day by this resource.	Money	
Type	A value associated with processing rules that control certain calculations at the resource level, such as how costs associated with resource loads are apportioned. **Referenced table:** ResourceType	Reference	

Resource table – calculated and set fields

The following describes the calculated and set fields in the Resource table. For an introduction to this table, and descriptions of its input fields, see "Resource table" on page 519.

Table 6.263: Calculated Resource (ProjMgmt) fields

Field name	Description	Data type	Key
Overloaded	Indicates if the resource is overloaded at any point. A resource is considered overloaded if there is at least one date on which the total load placed on the resource (from assignments across all projects) exceeds the maximum units available for the resource. Valid values are: • Y • N	Boolean	
OverloadedDate	If a resource is overloaded, this field reports the earliest date on which the total load placed against the resource exceeds its maximum units available.	Date	
Assignments	Set of ResourceAssignment records that reference this Resource value.	Set	
Availables	Set of ResourceAvailable records that reference this Resource value.	Set	

Table 6.263: Calculated Resource (ProjMgmt) fields

Field name	Description	Data type	Key
Calendar Exceptions	Set of ResourceCalendarException records that reference this Resource value.	Set	
Costs	Set of ResourceCosts records that reference this Resource value.	Set	
DataByDate	Set of ResourceDataByDate records that reference this Resource value.	Set	
Loads	Set of ResourceLoad records that reference this Resource value.	Set	

ResourceAssignment table

The **ResourceAssignment** table defines the assignments of resources to specific project tasks. Assignments are made in units, based on the effective **HoursPerDay** and **MaximumUnits** values from either the **Resource** or **ResourceAvailable** tables, and are applicable for the duration of the resource's task assignment. If required, the resource units assigned can be time-phased during the duration of a task by adding values to the **ResourceAssignmentAvailable** table.

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse, values can be added to this table through the **Resources** tab of the **Task Information** dialog box.

Table 6.264: ResourceAssignment (ProjMgmt) fields

Field name	Description	Data type	Key
ActualWork	If **Task.DurationType** is "FixedWorkAndRate", then this value is used together with the **RemainingWork** field to determine the work for the task (and hence impacts the calculated task duration). Otherwise, this field is not used in calculating task duration.	Quantity	
ApplyRate Adjustment	Indicates if the resource rate adjustment specified in the **RateAdjustment** field will be applied to for this resource assignment. Valid values are: • **N**—adjustment can be applied. • **Y**—adjustment is ignored.	Boolean	
BaseKey	A unique string identifier for this resource assignment.	String	Yes

Table 6.264: ResourceAssignment (ProjMgmt) fields

Field name	Description	Data type	Key
Rate	If **Task**.**DurationType** is "FixedWorkAndRate", then this value is used to determine the rate at which the resource completes work on the task (and hence impacts the calculated task duration). Otherwise, this field is ignored. For example, assume a single resource assigned to a task with ActualWork of "5"and RemainingWork of "1", along with a Rate of "2". The calculated task duration would then be 3; that is, (5 + 1)/ 2.	Quantity	
RateAdjustment	An optional percentage adjustment to be applied to the standard and overtime rates charged for the resource when working on this s task. This value is considered only if **ApplyRateAdjustment** is set to "Y". If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .05 indicates a 5 percent premium while -.1 indicates a 10 percent discount. Note that this adjustment overrides any other resource rate adjustments specified at the project or task levels. For more information, see "Resource rate adjustments" on page 2189.	Quantity	
RemainingWork	If **Task**.**DurationType** is "FixedWorkAndRate", then this value is used together with the **ActualWork** field to determine the work for the task (and hence impacts the calculated task duration). Otherwise, this field is not used in calculating task duration.	Quantity	
Resource	A reference to the resource that has been assigned to a task. **Referenced table:** Resource	Reference	Yes
Task	A reference to the task that has had a resource assigned to it. **Referenced table:** Task	Reference	Yes
Units	The amount of the resource, expressed in terms of maximum units per day, that should be assigned to the task. For example, if a resource's **HoursPerDay** is set to 8, and its **MaximumUnits** is set to 1, then a value of .75 in this field assigns it for 6 hours a day to the task. Depending on the **Task.DurationType** setting, this value also impacts calculated task durations as discussed in "Task durations" on page 2176.	Quantity	

ResourceAssignment table – calculated and set fields

The following describes the calculated and set fields in the ResourceAssignment table. For an introduction to this table, and descriptions of its input fields, see "ResourceAssignment table" on page 522.

Table 6.265: Calculated ResourceAssignment (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveRate Adjustment	Indicates the rate adjustment, if any, to be applied to the resource's standard and overtime rates when assigned to this task. For example, .05 indicates a 5 percent premium is applied to the resource rate, and -.12 indicates a 12 percent discount is applied to the resource rate.	Quantity	
EffectiveRate Adjustment Source	Indicates the source of the rate adjustment effective for this resource task assignment. Resource rate adjustments can be specified in up to six different tables, but only one adjustment is ever effective for a given resource assignment. The list below shows the order of priority for different rate adjustment sources (items higher in the list always take precedence over lower items). Valid values are: • **ResourceAssignment**—adjustment is defined for a given resource and task assignment. • **ResourceGroupTaskOverride**—adjustment is defined for all members of a resource group assigned to a given task. • **Task**—adjustment is defined for all resources assigned to a given task. • **ResourceProjectOverride**—adjustment is defined for a particular resource assigned to any tasks in a given project. • **ResourceGroupProjectOverride**—adjustment is defined for all members of a resource group assigned to any tasks in a given project. • **Project**—adjustment is defined for all resources assigned to any tasks in a given project. • **None**—no rate adjustment is applicable for the resource assignment.	String (Enum)	
Starting Unadjusted OvertimeRate	The hourly overtime rate that applies to the resource on the task's start date. This value comes from either of the **Resource** or **ResourceCost** tables and does not take into account any rate adjustments.	Money	

Table 6.265: Calculated ResourceAssignment (ProjMgmt) fields

Field name	Description	Data type	Key
Starting Unadjusted StandardRate	The hourly standard rate that applies to the resource on the task's start date. This value comes from either of the **Resource** or **ResourceCost** tables and does not take into account any rate adjustments.	Money	
StartingFixed Cost	The fixed cost that applies to the resource on the task's start date.	Money	
Availables	Set of ResourceAssignmentAvailable records that reference this ResourceAssignment value.	Set	

ResourceAssignmentAvailable table

The **ResourceAssignmentAvailable** table is used to override the units of work specified on the assignment of a resource to a task. For example, this might be done when a resource is not fully available for some portion of the task due to other scheduled task commitments. Each record in this table references a record in the **ResourceAssignment** table and allows for the original Units value specified on the resource assignment to be overridden as of a particular date.

Values in this table are only used if the task whose resource assignment is being overridden has resource leveling enabled (that is, it has a **TaskType.LevelResources** setting of "Y"). Otherwise, if **TaskType.LevelResources** = "N". then any applicable records in this table are ignored and the assignment values specified in the **ResourceAssignment** table are always used by the task.

This table's records can be edited using the **Resource Assignment Available Override** worksheet in the **Project Resource Master Data** included with RapidResponse.

Table 6.266: ResourceAssignmentAvailable (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveDate	Date on which the units specified for this referenced resource assignment take effect. The specified units are then assumed to be effective until the next EffectiveDate specified on a record in this table for the resource assignment. For all dates before the earliest EffectiveDate for a given resource, and in cases of resources that do not have values specified in this table, the units specified on the **ResourceAssignment** table are used by default.	Date	
Id	A unique string identifier for this resource assignment override.	String	Yes
Resource Assignment	A reference to the record defining the task and resource whose assigned units are to be overridden as of the specified effective date. **Referenced table:** ResourceAssignment	Reference	Yes

Table 6.266: ResourceAssignmentAvailable (ProjMgmt) fields

Field name	Description	Data type	Key
Units	Amount of the resource in units that should be assigned to the task as of the effective date specified. For example, if a resource's **HoursPerDay** is set to 8, and its **MaximumUnits** is set to 1, then a value of .5 in this field assigns it to the task for 4 hours a day.	Quantity	

ResourceAvailable table

The **ResourceAvailable** table is used to specify time-phased availability for resources that can be used to complete project tasks. For example, a resource might be on vacation for a period, or have a temporary increase in their working capacity.

When a resource has availabilities defined in this table that are effective on a given date, those values are used instead of the corresponding **HoursPerDay** and **MaximumUnits** fields in the **Resource** table. The resource availabilities defined in this table also respect the **TaskType.LevelResources** setting which can be used to ensure that resources assigned to tasks are not assigned beyond the maximum units specified. For example, this setting can be used to ensure no load is placed against a resource while on vacation.

This table's records can be edited using the **Resource Available Overrides** worksheet in the **Project Resource Master Data** included with RapidResponse.

Table 6.267: ResourceAvailable (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveDate	Date on which the resource availability specified for this resource takes effect. The specified **HoursPerDay** and **MaximumUnits** are then assumed to be effective until the next EffectiveDate specified on a record in this table for the resource. For example, suppose a resource goes on vacation and has 0 units available beginning on the date their vacation begins. At the end of that vacation period, another record should be added to this table specifying the number of units available once the resource returns to work. For all dates before the earliest **EffectiveDate** for a given resource, and in cases of resources that do not have any values specified in this table, the **HoursPerDay** and **MaximumUnits** from the **Resource** table are used by default.	Date	
HoursPerDay	The new standard number of hours available per working day for one unit of this resource. The number of hours specified here equates to a single unit as specified in the **MaximumUnits** field. For example, if **MaximumUnits** is 1 and this value is 8, then the resource can do a maximum of 8 hours work per day before it is seen as over allocated and overtime costs incurred.	Quantity	

Table 6.267: ResourceAvailable (ProjMgmt) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this resource availability override.	String	Yes
MaximumUnits	The new maximum number of units available per working day for this resource, where each unit corresponds to the value in the **HoursPerDay** field. For example, if **HoursPerDay** is 4 and this value is 2, then the resource can do a maximum of 8 hours work per day before it is seen as over allocated and overtime costs incurred. Note also that **TaskType.LevelResources** value for a given task also influences the usage of this field (except in cases of "fixed duration" tasks). If **LevelResources** is set to "N", then resource allocations to a task in excess of the maximum units specified here are seen as over allocations and overtime costs are incurred accordingly. However, if **LevelResources** is set to "Y", then resource allocations to a task are not allowed to exceed the maximum units specified here, and the calculated task duration and finish dates are moved out as required.	Quantity	
Resource	A reference to the resource whose standard availability is being overridden by the values in this record. **Referenced table:** Resource	Reference	Yes

ResourceCalendarException table

The **ResourceCalendarException** table is used to specify exceptions to the hours per day available from resources on given days of the week. For example, a resource might not work on Mondays or might only work half-days on Fridays.

Each record in this table identifies a resource and the hours per day it can work on a specific day of the week. The values provided here override the effective **HoursPerDay** value for the resource in either the **Resource** or **ResourceAvailable** tables (for the specified day of the week only).

This table's records can be edited using the **Resource Calendar Overrides** worksheet in the **Project Master Data** workbook included with RapidResponse.

Table 6.268: ResourceCalendarException (ProjMgmt) fields

Field name	Description	Data type	Key
DayOfWeek	The day of the week for which the standard hours per day specified on either the **Resource** or **ResourceAvailable** table should be overridden. Valid values are: • Monday • Tuesday • Wednesday • Thursday • Friday • Saturday • Sunday **Note:** In order to apply a new **HoursPerDay** value to a given day of the week for the resource, the day specified must exist on the **ResourceType.Calendar** value used by the resource. For example, if the resource's calendar is a standard 5 day workday, then records entered here for Saturday or Sunday would be ignored.	String (Enum)	
EffectiveDate	Date on which the hours per day specified for this resource and day of the week takes effect. The specified hours are then effective until the next **EffectiveDate** in this table for the specified **Resource** and **DayOfWeek** combination.	Date	
HoursPerDay	The new standard hours per day available for this resource on the specified day of the week. The number of hours specified here equates to a single unit as specified in the **MaximumUnits** field on either the **Resource** or **ResourceAvailable** tables. For example, if **DayOfWeek** is 'Friday', **MaximumUnits** is '1', and this value is '4', then four hours per day can be worked on a Friday before the resource is seen as over allocated and any overtime costs incurred.	Quantity	
Id	A unique string identifier for this resource calendar exception.	String	Yes
Resource	A reference to the resource this calendar exception applies to. **Referenced table:** Resource	Reference	Yes

ResourceCostActual table

The **ResourceCostActual** table can be used to input the actual standard and/or overtime hours worked by a resource on a given date, and reports the calculated costs associated with those hours. This data is typically inputted once a project has moved past the planning stage into execution and allows for a comparison of actual and estimated costs. For example, RapidResponse includes standard reports that

show planned and actual project costs in a burndown chart; these reports rely on the actual resource details being provided in this table.

This table's records can be edited using the Project Resource Management workbook included with RapidResponse.

Table 6.269: ResourceCostActual (ProjMgmt) fields

Field name	Description	Data type	Key
Date	The date on which the actual costs accrued to the resource from this task assignment.	Date	
FixedCost	Any fixed (set) cost incurred on this date from assigning the resource to the task.	Money	
Id	A unique string identifier for this record.	String	Yes
OvertimeHours	The number of hours worked by the resource on the task that exceed the resources's maximum hours per day.	Quantity	
Resource Assignment	A reference to the specific resource and task assignment to which the actual hours specified on this record apply. **Referenced table:** ResourceAssignment	Reference	Yes
StandardHours	The number of hours worked by the resource on the task that do not exceed resources's maximum hours per day.	Quantity	

ResourceCostActual table – calculated and set fields

The following describes the calculated and set fields in the **ResourceCostActual** table. For an introduction to this table, and descriptions of its input fields, see "ResourceCostActual table" on page 528.

Table 6.270: ResourceCostActual (ProjMgmt) calculated and fields

Field name	Description	Data type	Key
OvertimeCost	The calculated overtime cost incurred on this date from the resource's work on the task assignment. This value is calculated by multiplying the value given in the **OvertimeHours** field by the resource's hourly overtime rate. The resource's hourly overtime rate comes from the **OvertimeRate** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on this date), and is subject to any applicable project or task-level rate adjustments.	Money	
StandardCost	Reports the standard cost incurred on a this date from the resource's work on the task assignment. This value is calculated by multiplying the value given in the **OvertimeHours** field by the resource's hourly overtime rate. The resource's hourly overtime rate comes from the **StandardRate** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on this date), and is subject to any applicable project or task-level rate adjustments.	Money	

ResourceCosts table

The **ResourceCosts** tables is used to specify time-phased costs, such as standard and overtime rates, associated with using a particular resource to complete tasks. When a resource has cost values in this table that are effective on a given date, those values are used instead of the corresponding values provided in the **Resource** table.

This table's records can be edited using the **Resource Cost Overrides** worksheet in the **Project Resource Master Data** workbook included with RapidResponse.

Table 6.271: ResourceCosts (ProjMgmt) fields

Field name	Description	Data type	Key
EffectiveDate	Date on which the cost values specified for this resource take effect. The specified costs are then assumed to be effective until the next EffectiveDate specified on a record in this table for the resource. For all dates before the earliest EffectiveDate for a given resource, and in cases of resources that do not have values specified in this table, the cost values from the **Resource** table are used by default.	Date	
FixedCost	A new set cost incurred each time the resource is assigned to a task, and independent of the amount of time actually spent on the task. Based on the **TaskType.FixedCostAccrualRule** setting for a given task, this cost can be incurred either when the task starts, the task finishes, or spread evenly across all days the resource is assigned to the task.	Money	
Id	A unique string identifier for this set of resource costs.	String	Yes
OvertimeRate	Overtime rate for this resource. Any load generated against this resource on a given day, beyond the maximum units specified, is billed at this hourly rate. For example, if **Resource.MaximumUnits** is '1' and **Resource.HoursPerDay** is '8', this rate is applied to all hours worked in excess of eight per day by this resource.	Money	
Resource	A reference to the resource these time-phased costs apply to. **Referenced table:** Resource	Reference	Yes
StandardRate	Standard hourly rate for this resource. Any load generated against this resource on a given day, up until the maximum units specified, is billed at this rate. For example, if **Resource.HoursPerDay** is 10 and **Resource.MaximumUnits** is 1., this rate is applied to each of the first ten hours worked per day by this resource.	Money	

ResourceGroup table

The **ResourceGroup** table defines group names that can be assigned to resources. These are meant to categorize and group similar types of resources together. For example, resource groups might be created to identify job titles associated with the resources on a project. These can be used to aid in resource-level tracking and filtering.

This table's records can be edited using the **Resource Groups** worksheet in the **Project Resource Master Data** workbook included with RapidResponse.

Table 6.272: ResourceGroup (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this resource group.	String	
Name	A string value used to identify this resource group. This value is referenced from the Group field on the **Resource** table.	String	Yes

ResourceGroup table – calculated and set fields

The following describes the calculated and set fields in the ResourceGroup table. For an introduction to this table, and descriptions of its input fields, see "ResourceGroup table" on page 531.

Table 6.273: Calculated ResourceGroup (ProjMgmt) fields

Field name	Description	Data type	Key
Resources	Set of Resource records that reference this ResourceGroup value.	Set	

ResourceGroupProjectOverride table

The **ResourceGroupProjectOverride** table is used to apply project-specific adjustments to the standard and overtime rates charged by members of a particular resource group. For example, all engineers assigned to a specific customer project might have their rates discounted.

Adjustments specified in this table override any specified for the project itself. However, if there are adjustments specified at a more granular level, then those values are used instead of the ones in this table. For example, examples can be specified for individual resources and/or tasks in the project. For more information, see "Resource rate adjustments" on page 2189.

Table 6.274: ResourceGroupProjectOverride (ProjMgmt) fields

Field name	Description	Data type	Key
Project	The project in which the rate adjustment will be applied to members of the referenced resource group. **Referenced table:** Project	Reference	Yes
RateAdjustment	The percentage adjustment to be applied to the standard and overtime rates charged by members of the resource group when assigned to tasks in this project. If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .05 indicates a 5 percent premium while -.1 indicates a 10 percent discount.	Quantity	
ResourceGroup	The group whose resources will have a rate adjustment applied when assigned to tasks in the referenced project. **Referenced table:** ResourceGroup	Reference	Yes

ResourceGroupTaskOverride table

The **ResourceGroupTaskOverride** table is used to apply task-specific adjustments to the standard and overtime rates charged by members of a particular resource group. For example, a premium might be applied to any engineers assigned to a particular task.

Adjustments specified in this table override any specified at the project level or on the task itself. However, if a task-based adjustment is specified for an individual member of the resource group, than that value is used instead of the one in this table. For more information, see "Resource rate adjustments" on page 2189.

Table 6.275: ResourceGroupTaskOverride (ProjMgmt) fields

Field name	Description	Data type	Key
RateAdjustment	The percentage adjustment to be applied to the standard and overtime rates charged by members of the resource group when assigned to the referenced task. If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .2 indicates a 20 percent premium while -.12 indicates a 12 percent discount.	Quantity	
ResourceGroup	The resource group whose members will have a rate adjustment applied when assigned to the referenced task. **Referenced table:** ResourceGroup	Reference	Yes

Table 6.275: ResourceGroupTaskOverride (ProjMgmt) fields

Field name	Description	Data type	Key
Task	The task to which a rate adjustment will be applied when any members of the referenced resource group are assigned to it. **Referenced table:** Task	Reference	Yes

ResourceProjectOverride table

The **ResourceGroupProjectOverride** table is used to apply project-specific adjustments to the standard and overtime rates charged by a particular resource. For example, all a specific person working on a customer project might bill at a premium rate.

Adjustments specified in this table override any specified for the project or project and resource group. However, if there are adjustments specified at the task level, then those values are used instead of the ones in this table. For more information, see "Resource rate adjustments" on page 2189.

Table 6.276: ResourceProjectOverride (ProjMgmt) fields

Field name	Description	Data type	Key
Project	The project in which the adjustment will be applied to the rate charged by the referenced resource. **Referenced table:** Project table	Reference	Yes
RateAdjustment	The percentage adjustment to be applied to the standard and overtime rates charged by this resource when assigned to tasks in the project. If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .05 indicates a 5 percent premium while -.1 indicates a 10 percent discount.	Quantity	
Resource	The specific resource that will have a rate adjustment applied when assigned to tasks in the referenced project. **Referenced table:** Resource table	Reference	Yes

Routing table

The **Routing** table contains routing codes that are used to group together the sequence of operations used in the making of a part. It is referenced by the **OperationSequence** table that identifies operation sequences and groups together work center operations from the **CRPOperation**. It is also directly referenced by the **Operation** table to provide compatibility with earlier versions of RapidResponse when operations were stored in that table and directly referenced routings (the **CRPOperation** table is recommended for all new purposes).

This table then serves as the link between work orders and the specific work center operations required in their production. The **Routing** table is referenced by **PartSource** table which uses it to determine all operations required to produce a new planned order from the part source, and also by the **ScheduledReceipt** table which uses it to determine all operations required to produce the order.

The **Routing** table supports the optional capacity calculations.

Table 6.277: Routing (Mfg) fields

Field name	Description	Data type	Key
Id	The unique identifier for the routing. **Default value**: blank string	String	Yes
Site	The site associated with this routing. **Note:** This field is optional. A system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window. **Referenced table:** Site	Reference	Yes

Routing table – calculated and set fields

The following describes the calculated and set fields in the Routing table. For an introduction to this table, and descriptions of its input fields, see "Routing table" on page 534.

Table 6.278: Calculated Routing (Mfg) fields

Field name	Description	Data type	Key
AlternateRoutings	A set of **AlternateRouting** records. These records are used for information purposes only.	Set	
LoadsNewOrder Planned	Set of calculated LoadOperationNewOrderPlanned records reporting load from a planned order through an operation that belongs to this routing.	Set	
LoadsReceipts Current	Set of calculated LoadOperationReceiptsCurrent records reporting load from a scheduled receipt (based on current due dates) through an operation that belongs to this routing.	Set	
LoadsReceipts Planned	Set of calculated LoadOperationReceiptsPlanned records reporting load from a scheduled receipt (based on recommended due dates) through an operation that belongs to this routing.	Set	
Operations	A set of **Operation** records that belong to this routing.	Set	

Table 6.278: Calculated Routing (Mfg) fields

Field name	Description	Data type	Key
PartSources	Set of **PartSource** records using this Routing to determine the operations required when a new planned order is built.	Set	
ScheduledReceipts	Set of **ScheduledReceipt** records that use this routing to determine the operations required in their production.	Set	
Sequences	A set of **OperationSequence** records that reference this routing and identify a named sequence of operations (from the **CRPOperation** table) belonging to the routing.	Set	

RParameter table

The **RParameter** table identifies optional parameters and associated values to be passed to specific functions in the R statistical programming language. Each parameter and value pair defined in this table must then be associated with a set of R parameters that can be used by forecast items.

This table supports the optional Rforecast capability (and is only applicable when forecasting an item that uses the "Rforecast" forecast model).

Table 6.279: RParameter (Mfg) fields

Field name	Description	Data type	Key
Description	A description of the parameter and associated value defined on this record.	String	
Id	A unique string identifier for this record.	String	Yes
Name	A reference to the named parameter. **Referenced table**: RParameterName **Note:** The following values are ignored if defined as R parameters: damped, fan, find.frequency, h, level, lower, object, phi, upper, y. Most of these are passed to R based on data and field values in RapidResponse.	Reference	
Set	A reference to a named set this parameter belongs to. **Referenced table**: RParameterSet **Note:** Values in the **RParameterSet** table can then be referenced by ForecastItemParameters records for items that want to pass a set of parameters to R during statistical forecast generation.	Reference	Yes
Value	A string containing the value to be associated with the named parameter.	String	

RParameterSet table

The **RParameterSet** table holds named groupings of optional parameters and their values for use in statistic functions in the R programming language. This table is referenced by both the **RParameter** and **ForecastItemParameters** tables.

The **RParameter** table is used to specify values for optional parameters and the specific functions in R that those parameters should be passed to. All entries in the **RParameter** table that reference the same **RParameterSet** record are then treated as a group that can be passed collectively to R forecasting functions.

If the **ForecastItemParameters** table used the "Rforecast" model, and also has its **RParameterSet** reference populated, then parameter values defined in the referenced set will be passed to R for use in generating the statistical forecast. Otherwise, if the item's **RParameterSet** is Null, then no optional parameters are assumed and the forecast is generated using a standard set of parameters in R (many of which have their values set to field values from the **ForecastItemParameters** table). Note that the "Rforecast" model is intended to generate reasonable forecast values without the use of any optional parameters which are generally intended for advanced uses.

The **RParameterSet** table supports the optional R Forecast capability.

Table 6.280: RParameterSet (Mfg) fields

Field name	Description	Data type	Key
Name	A unique name for this group of function parameters.	String	Yes
Description	A description of this parameter set.	String	

RParameterSet table–calculated and set fields

The following describes the calculated and set fields in the **RParameterSet** table. For an introduction to this table, and descriptions of its input fields, see "RParameterSet table " on page 537.

Table 6.281: RParameterSet (Mfg) calculated fields

Field name	Description	Data type	Key
Details	Set of **BestFitProfileDetail** records that use this parameter set. **Referenced table:** BestFitProfileDetail	Set	
ForecastItem Parameters	Set of **ForecastItemParameters** records that use this parameter set. **Referenced table:** ForecastItemParameter	Set	
RParameters	Set of **RParameter** records that identify the parameter values that define this parameter set. **Referenced table:** RParameter	Set	

SafetyLeadTimeProfile table

The **SafetyLeadTimeProfile** table holds named profiles that enable time-phased safety lead time. Safety lead time is meant to account for some level of uncertainty in demand and supply, and is used to adjust demand dates and set planned order due dates.

Each profile should be referenced by records in the **SafetyLeadTimeProfileDetail** table that define the safety lead time values that apply within specified date ranges. If a part references a profile through the **Part.SafetyLeadTimeProfile** field, then its safety lead time values come from the set of **SafetyLeadTimeProfileDetail** records associated with that referenced profile (in addition to any safety lead time defined at the part source level).

Otherwise, if a **Part** record has a Null reference to this table, its safety lead time is taken from the non-time phased value in **Part.SafetyLeadTime** (in addition to any safety lead time defined at the part source level).

Note that in order for a part to use a referenced profile and its detail records, the part must use **PartType.SafetyLeadTime** value of either "Y" or "Soft" (otherwise, safety lead time won't be used when planning supply to demand). Also, if safety lead time should be applied to demand request dates and/or only used in cases where doing so won't make other demands worse off, then use of settings on the **OrderPriority.TwoDatePlanningRule** field should be considered.

This table was added in Version 2016.2

Table 6.282: SafetyLeadTimeProfile (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this safety lead time profile. For example, it might provide descriptive details about the set of **SafetyLeadTimeProfileDetail** records associated with this profile.	String	
Site	The site associated with this profile. **Referenced table:** Site	Reference	Yes
Value	The name of this safety lead time profile. Together with the Site value, this uniquely identifies the profile.	String	Yes

SafetyLeadTimeProfile table – calculated and set fields

The following describes the calculated and set fields in the **SafetyLeadTimeProfile** table. For an introduction to this table, and descriptions of its input fields, see "SafetyLeadTimeProfile table" on page 538.

Table 6.283: Calculated SafetyLeadTimeProfile (Mfg) fields

Field name	Description	Data type	Key
Parts	Set of Part records that use this safety lead time profile.	Set	
SafetyLeadTime ProfileDetails	Set of SafetyLeadTimeProfileDetail records that define the effective dates and associated safety lead time values that comprise this profile.	Set	

SafetyLeadTimeProfileDetail table

The **SafetyLeadTimeProfileDetail** table contains the time-phased safety lead time values that define named profiles in the **SafetyLeadTimeProfile** table.

Each record in this table specifies a safety lead time value to associate with a referenced profile, along with the first date on which that safety lead time value is meant to apply (**EffectiveInDate**) and the last date that value is meant to apply (**EffectiveOutDate**).

When defining records in this table, the effective dates should typically be set so that records belonging to the same profile do not have overlapping date ranges. That is, the **EffectiveInDate** on one record belonging to a given profile should typically not be earlier than the **EffectiveOutDate** on the previous record belonging to that same profile. However, in cases where a given demand date falls within the effective range of multiple **SafetyLeadTimeProDetail** records for profile, the detail record with the latter **EffectiveOutDate** is assumed to be effective.

Note also that if a demand date does not fall within the effective date range of any detail record associated with the part's safety lead time profile, then the value in **Part.SafetyLeadTime** is used instead.

This table was added in Version 2016.2.

Table 6.284: SafetyLeadTimeProfileDetail (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	The first date on which the safety lead time defined on this record can be effective (inclusive).	Date	Yes
EffectiveOutDate	The last date on which the safety lead time defined on this record can be effective (inclusive).	Date	

Table 6.284: SafetyLeadTimeProfileDetail (Mfg) fields

Field name	Description	Data type	Key
SafetyLeadTime	Amount of time added to a part's fixed lead time to protect against fluctuations in supply and demand. When planning new supply to satisfy a demand, the demand date is adjusted by this value to determine the planned order due date. Depending on the **PartType.SafetyLeadTimeCalendarRule** setting used the part, this value should be expressed in either the part's **PlanningCalendars.TimeUnits** calendar or the Everyday/Gregorian calendar. **NOTE**: SafetyLeadTime is ignored and not used to offset due dates for safety stock demands.	Quantity	
SafetyLeadTime Profile	A reference to the profile the safety lead time details defined on this record apply to. **Referenced table**: SafetyLeadTimeProfile	Reference	Yes

SafetyStockAverageDemandProfile table

The **SafetyStockAverageDemandProfile** contains optional profiles that can be used to define the horizon over which a safety stock item's historical and/or future demands are collected for use in calculating average demands. These are the average demand calculations that are used as input towards determining the recommended historical and future reorder points.

Each profile in this table is made up an **Offset** value that defines the start or end point for collecting data, a **Multiplier** value that defines a multiple of the item's lead time over which historical data is collected, and an **Extend** value that adds a fixed value to the period over which data is collected. The specific usage of these three fields depends on whether the profile is being used for collecting historical demands or future demands.

When a profile is used to determine a window for collecting historical demands, the values provided in this table are used along with the last date of the historical collection interval defined on the safety stock item and the item's average lead time to define profile start and end dates as follows:

- **Historical start:** `LastDate - Offset - CalcAverageLeadTime * Multiplier - Extend`
- **Historical end:** `LastDate - Offset`

Conversely, when a profile is used to determine a window for collecting future demands, the values provided in this table are used along with the run date and the safety stock item's average lead time to define profile start and end dates as follows:

- **Future start:** `RunDate + Offset`
- **Future end:** `RunDate + Offset + CalcAverageLeadTime * Multiplier + Extend`

The **CalcAverageLeadTime** value used in profile calculations is taken from the calculated field on the **SafetyStockItem** table.

Table 6.285: SafetyStockAverageDemandProfile (Mfg) fields

Field name	Description	Data type	Key
Description	An optional description explaining this average demand profile.	String	
Extend	A fixed number of intervals, expressed in terms of **SafetyStockItem.IntervalsCalendar,** to add to or extend the horizon for calculating average demands (after having accounted for the offset value and applied the lead time multiplier).	Integer	
Multiplier	A multiplier applied to the safety stock item's average lead time, the product of which is then used in determining the size of the window for calculating average demands. **Note:** This field's data type was modified in Version 2016.2, Service Update 4.	Quantity	
Name	A unique name that identifies this average demand profile. Both the **HistoricalAverageDemandProfile** and **FutureAverageDemandProfile** fields on the **SafetyStockItem** table reference this field.	String	Yes
Offset	A fixed number of intervals, expressed in terms of **SafetyStockItem.IntervalsCalendar,** to offset the start/end of the horizon used for collecting demands used in average demand calculations.	Integer	

SafetyStockItem table

The **SafetyStockItem** table is used to configure both single and multi-echelon safety stock items. Each record in this table identifies a specific item (part) for which safety stock recommendations should be made, specifies whether the item is intended for single or multi-echelon safety stock calculations, and contains other parameters applicable to single-echelon, multi-echelon, or both calculations.

Single-echelon safety stock items are used by both the **TimePhasedSafetyStock** and **SafetyStock** functions to generate safety stock and reorder point recommendations for individual parts based on historical data and meant to satisfy a specified service level (optionally, forecast data can also be used to generate future-looking reorder point recommendations). Multi-echelon safety stock items are used by an analytic that generates safety stock recommendations across a network of parts in order to both satisfy service levels defined for customer-facing end items, while also attempting to minimize the total holding cost of that inventory by recommending the strategic placement of that safety stock within the network.

This table was modified in Version 2016.2, Service Update 5. For more information about the calculations it supports, see "Inventory planning and optimization" on page 1935. Note also that the **Inventory Planning** workbook included with RapidResponse can be used to add or modify records in this table, and also reports results of single and multi-echelon safety stock calculations.

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
AverageDemand	A value representing the average historical demand to use for the part in safety stock calculations. Applicable only to non-time phased items and where **Type.StandardDeviationDemandRule** is set to "Manual". Average demand might be manually provided when there is limited historical data available for an item, but its average demand and other parameters are known or estimated from some other means. Note that **AverageDemand** should be expressed in **Type.IntervalsCalendar**.	Quantity	
DemandOutlier MovingAverage Window	The length of the moving average window for historical demand or forecast error. This field is only applicable on items where **OutlierType.DataRule** is set to "MovingAverageError". When using this setting, the threshold value is interpreted depending on the value specified for the **DemandOutlierThreshold** setting. **Default value**: 3 **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	
DemandOutlier Threshold	The (minimum) number of standard deviations away from the mean a demand quantity or forecast error point must be in order to be considered an outlier. Any outliers detected are then removed, smoothed, or ignored, as specified by the setting in the **RemoveDemandOutliers** field. When using this setting, the threshold value is interpreted depending on the specified **OutlierType.DetectionRule**. For example, if the DetectionRule = "Winsorizing", then outliers that are detected are identified as quantities that are outside a specified percentile in the ordered data. For more information, see the **OutlierType.DetectionRule** field. **Default value**: 3 **Note:** This field achieves the same functionality as the **ForecastErrorOutlierThreshold** field and is applicable on items where **Type.StandardDeviationDemandRule** is set to "ForecastError".	Quantity	
DemandOutlier Type	A reference to the **OutlierType** that defines how to process outliers in historical demand or historical forecast error. **Default value**: Null **Referenced table**: OutlierType (Nullable) **Note:** This field was added in Version 2016.2, Service Update 5	Reference	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
DemandSmoothing AfterIntervalCount	When smoothing demand outliers, this indicates the number of **Type.IntervalsCalendar** periods forward over which to spread the values. This occurs after any backward spreading. This field is only applicable on items where **OutlierType.BelowThresholdRule** is set to "SmoothKeepExcess" or "SmoothRemoveExcess".	Integer	
DemandSmoothing BeforeInterval Count	When smoothing demand outliers, this indicates the number of **Type.IntervalsCalendar** periods backward over which to spread the values. This occurs before any forward spreading. This field is only applicable on items where **OutlierType.AboveThresholdRule** is set to "SmoothKeepExcess" or "SmoothRemoveExcess".	Integer	
Description	An optional description for this safety stock item.	String	
ForecastLag	Number of **Type.AsOfDateCalendar** intervals to look back when collecting historical forecast data for use in forecast error calculations. For example, assume this value is set to "1" and the AsOfDateCalendar is set to "Month". In that case, HistoricalDemandActual records from one month are compared against detail forecast records referencing a HistoricalDemandSeries where the AsOfDate is in the previous month. Similarly, a value of "2" would indicate to compare actuals from one month against the forecast made two months earlier. This field is only applicable on items where **Type.StandardDeviationDemandRule** is set to "ForecastError". **Default value**: 1 (zero and all negative values are interpreted as 1). **Note:** For an example of this field's usage, see "Example 3: Fixed lead time (with forecast error)" on page 1972.	Integer	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
ForecastError OutlierThreshold	Number of standard deviations away from the mean that a forecast error point must be in order to be considered an outlier. If a point is determined to be an outlier, it is processed based on the setting in **Type.ForecastOutlierProcessingRule** field (for example, outliers might be removed entirely when calculating the standard deviation of forecast error). This field is only applicable on items where **Type.StandardDeviationDemandRule** is set to "ForecastError". **Note:** This field is obsolete (as of Version 2016.2, Service Update 5). To achieve the same functionality, use the **DemandOutlierThreshold** field.	Quantity	
FutureAverage DemandProfile	A reference to a profile, based on lead time multiples and other settings, that defines the horizon to use when determining the average future demand value for use in reorder point calculations. This reference is nullable and can be left blank if all future data points should be used to calculate average future demand. Note that this field supports reorder point calculations applicable to single-echelon safety stock items only. **Referenced table:** SafetyStockAverageDemandProfile (Nullable)	Reference	
FutureInterval Count	The number of **Type.IntervalsCalendar** periods after the run date over which current/forecast demand data should be collected to determine future average demands. This value is also: • Used to calculate details in the **SafetyStockItemFutureDemand** table. The **SafetyStockItemFutureDemand** table is used to calculate days of supply for single-echelon safety stock items. The table is also used to calculate the average future demand that will help determine **SafetyStockTimePhasedResult.FutureReorderPoint** for multi-echelon safety stock items. • Reflected in the quantity that is reported in **SafetyStockResult.AverageFutureDemand**, which helps determine reorder point calculations for single-echelon safety stock items. If this value is less than or equal to zero, then all future demands are used.	Integer	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
HistoricalAverage DemandProfile	A reference to a profile, based on lead time multiples and other settings, that defines the horizon to use when determining the average historical demand value for use in reorder point calculations. This reference is nullable and can be left blank if all historical data points should be used to calculate average historical demand. **Referenced table:** SafetyStockAverageDemandProfile	Reference	
HistoricalDemand Category	A reference to the historical demand category from which **HistoricalDemandActual** data should be collected for use in safety stock calculations. **Referenced table:** HistoricalDemandCategory	Reference	
HistoricalEndDate	The last date on which to collect historical demand and supply data for use in safety stock calculations (if a date later than RunDate is specified, then `RunDate + 0 Type.IntervalsCalendars` is used instead).	Date	
HistoricalForecast Category	A reference to the historical forecast category. When calculating the standard deviation of forecast error, historical forecast records in the **HistoricalDemandSeriesDetail** are collected for this item only if they match this category through their **Series.Category** reference. This field is only applicable on items where **Type.StandardDeviationDemandRule** is set to "ForecastError" and Type.TimePhasedProcessingRule is either "Ignore" or "DaysOfSupplyBackward" or "DaysOfSupplyForward". **Referenced table:** HistoricalDemandCategory (Nullable)	Reference	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
HistoricalInterval Count	The number of **Type.IntervalsCalendar** periods before the run date over which to collect historical demand and supply data for use in safety stock calculations. HistoricalIntervalCount should be set to the (positive integer) number of IntervalsCalendar intervals of history to consider. Historical data collection starts the specified number of HistoricalIntervalCount intervals before RunDate, and ends the day before RunDate. If a value less than or equal to 0 is provided in this field, then the **HistoricalEndDate** and **HistoricalStartDate** fields are used to define the period instead. If HistoricalIntervalCount is greater than 0, **HistoricalEndDate** is ignored and a non-null **HistoricalStartDate** will still be considered. The effective start date will be min (HistoricalStartDate, RunDate - HistoricalIntervalCount). **Note:** When specifying a **SafetyStockItem.HistoricalIntervalCount** value for a safety stock item, it needs to adequately reflect both the **SafetyStockItemType.IntervalsCalendar** and **SafetyStockItemType.CycleCalendar** that are selected for that item. This is to ensure that an adequate amount of historical demand and supply data is available for use in safety stock calculations. For example, if **IntervalsCalendar** is set to "monthly" and **CycleCalendar** is "yearly", you would need a minimum **SafetyStockItem.HistoricalIntervalCount** of "12" to provide adequate history.	Integer	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
HistoricalSkip LeadingZeros	Determines whether **SafetyStockHistoricalDemand** and **SafetyStockHistoricalForecast** records are created for buckets at the beginning of the historical demand period (based on **HistoricalDemandActuals** table) where the demand and forecast quantities are zero (leading zeros), respectively. **Y**—If **HistoricalSkipLeadingZeros** is set to "Y" and **Type.StandardDeviationDemandRule** is set to "ForecastError," the forecast error is computed beginning with the first bucket in the historical demand period for which both the forecast quantity and the demand quantity are not zero. **N**—Leading zeros are not ignored when calculating historical data. **Default value:** N **Note:** Records with zero quantity in the first bucket or buckets of the historical demand period are skipped if **HistoricalSkipLeadingZeros** is set to "Y," but only if all earlier period buckets in the **HistoricalDemandActuals** table have demand quantities of zero. If the first period buckets in the table have demand with positive quantities, leading zeros are not ignored. **Note:** This field was added in Version 2016.2 Service Update 6.	Boolean	
HistoricalStartDate	The first date on which to collect historical demand and supply data for use in safety stock calculations.	Date	
HistoricalSupply Category	A reference to the historical supply category from which **HistoricalSupplyActual** data should be collected for use in determining average lead times (used only when **Type.SupplyVariabilityRule** is set to "Use"). This reference is nullable and can be left empty in cases where lead time variability considerations are not required for safety stock calculations. Note that this field and historical supply calculations apply to single-echelon safety stock calculations only. **Referenced table:** HistoricalSupplyCategory	Reference	
LeadTime	Standard average lead time value to use for the part in single-echelon safety stock calculations. Applicable if **Type.SupplyVariabilityRule** is set to either "Ignore" or "Manual", or if no historical supply data is available (otherwise, lead time is calculated from historical supply data). This value should be expressed primarily in terms of **Type.LeadTimeCalendar** units, only being expressed as **Type.IntervalsCalendar** units when not defined.	Quantity	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
LeadTimeOutlier MovingAverage Window	The length of the moving average window for historical lead time. This field is only applicable on items where **OutlierType.DataRule** is set to "MovingAverageError". When using this setting, the threshold value is interpreted depending on the value specified for the **LeadTimeOutlierThreshold** setting. **Default value**: 3 **Note:** This field was added in Version 2016.2, Service Update 5 and is used only for SEIP. For MEIO, use **MEIOLeadTime.LeadTimeOutlierMovingAverageWindow**.	Integer	
LeadTime OutlierThreshold	The (minimum) number of standard deviations away from the mean a lead time quantity must be in order to be considered an outlier. Any outliers detected are then removed, smoothed, or ignored, as specified by the setting in the **RemoveDemandOutliers** field. When using this setting, the threshold value is interpreted depending on the specified **OutlierType.DetectionRule**. For example, if the DetectionRule = "Winsorizing", then outliers that are detected are identified as quantities that are outside a specified percentile in the ordered data. For more information, see the **OutlierType.DetectionRule** field. **Default value**: 3 **Note:** This field was added in Version 2016.2, Service Update 5 and is used only for SEIP. For MEIO, use **MEIOLeadTime.LeadTimeOutlierThreshold**.	Quantity	
LeadTime OutlierType	A reference to the **OutlierType** that defines how to process outliers in historical lead time. **Default value**: Null **Referenced table**: OutlierType (Nullable) **Note:** This field was added in Version 2016.2, Service Update 5 and is used only for SEIP. For MEIO, use **MEIOLeadTime.LeadTimeOutlierType**.	Reference	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
LeadTimeSmoothing AfterIntervalCount	When smoothing historical lead time outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods forward over which to spread the values. This occurs after any backward spreading. This field is only applicable on items where **OutlierType.BelowThresholdRule** is set to "SmoothKeepExcess" or "SmoothRemoveExcess". **Note:** This field was added in Version 2016.2, Service Update 5 and is used only for SEIP. For MEIO, use **MEIOLeadTime.LeadTimeSmoothingAfterIntervalCount**.	Integer	
LeadTimeSmoothing BeforeIntervalCount	When smoothing historical lead time outliers for safety stock items, this indicates the default number of **Type.IntervalsCalendar** periods backward over which to spread the values. This occurs after any forward spreading. This field is only applicable on items where **OutlierType.AboveThresholdRule** is set to "SmoothKeepExcess" or "SmoothRemoveExcess". **Note:** This field was added in Version 2016.2, Service Update 5 and is used only for SEIP. For MEIO, use **MEIOLeadTime.LeadTimeSmoothingBeforeIntervalCount**.	Integer	
OrderQuantity	If **Type.ServiceLevelRule** is set to "FillRate", this specifies the minimum order quantity value to be used as input when determining the service factor to use in safety stock calculations.	Quantity	
Part	A reference to the part for which safety stock and/or reorder point recommendations should be calculated. **Referenced table:** Part	Reference	Yes

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Indicates the type of safety stock calculations that are applicable to this safety stock item. Valid values are: • **MultiEchelon**—Recommended safety stock levels are calculated for the safety stock item itself as well as other eligible parts in its family (for example, parts in its BillOfMaterial structure). Variability of the safety stock item's historical demands is calculated, and the demand values are subsequently propagated to the lower-level items in its family. Either fixed or variable lead times are used for each item in the family. From this, a level of safety stock and strategic placement of the safety stock across parts in the item's network is recommended. The recommendations are meant to minimize the total cost of holding inventory in the network while satisfying the item's specified service level and service time. If an item is configured for multi-echelon safety stock calculations, ensure that related parts that should be brought into its family have their **MultiEchelonSafetyStockRule** reference set appropriately (this includes the part configured as a SafetyStockItem itself). • **SingleEchelon**—A recommended safety stock level is calculated just for the safety stock item (part) defined on this record. Based on variability of the item's historical demands and (optionally) supply lead times, safety stock is calculated to satisfy the item's specified service level. A recommended reorder point, based on either history or forecast, is also determined. **Default value:** SingleEchelon	String (Enum)	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
RemoveDemand Outliers	Determines whether outliers in historical data should be removed from the data series (reducing the length of the series, as opposed to adjusting outlier values without changing the length of the series). The **DemandOutlierThreshold** setting defines the number of standard deviations away from the mean an item quantity must be before it is considered an outlier. Valid values are: • **No**—Does not remove outliers. Instead, outliers are adjusted according to the threshold rules on **OutlierType**. • **Yes**—Removes both high and low outliers. • **LowOnly**—Removes only low outliers. • **HighOnly**—Removes only high outliers. **Default value:** No **Note:** This field was added in Version 2016.2, Service Update 5, and achieves the same functionality as **Type.ForecastErrorOutlierProcessingRule** = "Remove" and **Type.DemandOutlierProcessingRule** = "RemoveExcess".	String(Enum)	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
ServiceLevel	The service level percentage that calculated safety stock recommendations for the part are meant to satisfy. For multi-echelon items this always specifies the probability of an order being fully satisfied (not stocking out), and for single-echelon items the interpretation of this field is configurable depending upon the **Type.ServiceLevelRule** setting as follows: • If set to "Cycle", this specifies the probability of an order being fully satisfied (not stocking out). • If set to "FillRate", this specifies the percentage of demand that should be satisfied on time. The service level specified is then converted to a service factor or multiplier for use in calculating safety stock. For example, the following shows estimated z-value scores that correspond to given service levels (assuming normally distributed demands and use of the "Cycle" rule) calculated using a standard normal table: **Service Level** / **Service Factor** 90% 1.282 91% 1.341 92% 1.405 93% 1.476 94% 1.555 95% 1.645 96% 1.751 97% 1.881 98% 2.054 99% 2.326 **Note:** To ensure meaningful results, a value between .50 and .99 should be provided (higher values results in higher safety stock levels being maintained but results in a decreased chance of a stock out). For example, in a typical configuration a value of .95 might be specified.	Quantity	

Table 6.286: SafetyStockItem (Mfg) fields

Field name	Description	Data type	Key
ServiceTime	The service or delivery time that this item should always be able to satisfy. This field is only applicable to multi-echelon safety stock items and defines the allowable time between when a customer orders an item and when the item is shipped. The value in this field should be expressed in terms of the Gregorian (everyday) calendar. Thus, calculated safety stock levels will ensure orders for the item can be satisfied within the number of calendar days specified here. For example, this might typically be set to 0 for parts held at customer-facing locations.	Quantity	
StandardDeviation Demand	A value representing the standard deviation of historical demands to use for the part in safety stock calculations. Applicable only to non-time phased items where **Type.StandardDeviationDemandRule** is set to "Manual" (used when there is limited historical data available but the demand parameters are known from some other means).	Quantity	
StandardDeviation LeadTime	A value representing the standard deviation of supply lead time to use for the part in single-echelon safety stock calculations. Applicable only when **Type.SupplyVariabilityRule** is set to "Manual" (otherwise, standard deviation of lead time is calculated from historical supply data).	Quantity	
TimePhased ReportCount	A value used to define the window of time over which time-phased safety stock calculations can be reported for this item. Expressed in units of **Type.IntervalsCalendar** and applied from the **RunDate.** Note that time-phased safety stock recommendations are made on **RunDate** as well as each subsequent date within the window in which there is a calculated change in the safety stock level. **Default value**: 1 (safety stock level is reported on **RunDate** only)	Integer	
Type	A reference to the control value that defines processing rules used for this safety stock item. For example, this reference sets the calendar used for safety stock calculations, specifies how standard deviation should be calculated, and defines how demand outliers should be processed. **Referenced table:** SafetyStockItemType	Reference	

SafetyStockItem table – calculated and set fields

The following describes the calculated and set fields in the **SafetyStockItem** table. For an introduction to this table, and descriptions of its input fields, see "SafetyStockItem table" on page 541.

Table 6.287: SafetyStockItem (Mfg) calculated fields

Field name	Description	Data type	Key
CalcAverageLead Time	Calculated historical average lead time for this item. The calendar this is reported in depends on the item's **Type.UseRollingLeadTimeDemand** setting as follows: • If **Y**, then expressed in **IntervalsCalendar** units. • If **N**, then expressed in **LeadTimeCalendar** units (but converted to IntervalsCalendar units when used in average demand profile calculations). **Note:** This field was added in Version 2016.2, Service Update 4.	Quantity	
RollingLeadTime DemandLastDate	Last date on which rolling lead time calculations for this item are valid. This is the latest date within the item's historical interval range for which there is a sufficient data range to determine demands within lead time. If the item has any records later than this date in the **SafetyStockHistoricalDemand** table, their **RollingLeadTimeQuantity** value set to -1.	Date	
SafetyStock HistoricalForecasts	Set of **SafetyStockHistoricalForecast** records that reference this item.	Set	

SafetyStockItemMapping table

The **SafetyStockItemMapping** table is used to map a safety stock item to one or more other "source" parts whose historical demand, historical supply, and/or future demand data should be included in calculations that determine the recommended safety stock level and reorder point for the item. The **Item** reference on this table identifies the **SafetyStockItem** that should have safety stock levels calculated, and the **SourcePart** reference identifies the part whose historical supply, historical demand, and historical forecast data is collected and used in generating those safety stock levels.

For example, source parts mapped to a safety stock item might be different revisions of the part defined on the **SafetyStockItem** record. Based on the mapping details, specific data from the source part is then combined with the records generated for the safety stock item in the **SafetyStockHistoricalDemand**, **SafetyStockHistoricalSupply**, and **SafetyStockItemFutureDemand** tables.

Table 6.288: SafetyStockItemMapping (Mfg) fields

Field name	Description	Data type	Key
Description	An optional description explaining the purpose of this item mapping.	String	
HistoricalDemand Category	The historical demand category for which the source part's historical demand data should be collected from the **HistoricalDemandActual** table. This reference is nullable, and can be left empty if historical demand data from the source part is not required. **Referenced table:** HistoricalDemandCategory	Reference	
HistoricalEndDate	The last date on which historical demand and/or supply data should be collected for the source part. This date should not be set later than the end of the historical collection interval for the safety stock item (can only be used to set an interval narrower than the one defined on the item itself). If left undefined, then the end of the safety stock item's historical collection interval is used instead.	Date	
HistoricalForecast Category	The historical category for which the source part's historical forecast data should be collected. When calculating the standard deviation of forecast error, historical forecast records for the source part in the **HistoricalDemandSeriesDetail** are collected only if they match this category through their **Series.Category** reference. This field is only applicable on items where Type.StandardDeviationDemandRule is set to "ForecastError". **Referenced table:** HistoricalDemandCategory (Nullable) **Note:** This field was added in Version 2014.4	Reference	
HistoricalStartDate	The first date on which historical demand and/or supply data should be collected for the source part. This date should not be set earlier than the start of the historical collection interval for the safety stock item (can only be used to set an interval narrower than the one defined on the item itself). If left undefined, then the start of the safety stock item's historical collection interval is used instead.	Date	
HistoricalSupply Category	The historical supply category for which the source part's historical supply data should be collected from the **HistoricalSupplyActual** table. This reference is nullable, and can be left empty if historical supply data from the source part is not required for the safety stock item. **Referenced table:** HistoricalSupplyCategory	Reference	

Table 6.288: SafetyStockItemMapping (Mfg) fields

Field name	Description	Data type	Key
Id	A unique string identifier for this record.	String	Yes
Item	A reference to the safety stock item. This identifies the specific part, and other details, associated with an item for which safety stock recommendations are being calculated. **Referenced table:** SafetyStockItem	Reference	Yes
IncludeFuture Demand	Indicates if future demand data from the source part should be used. Valid values are: • **N**—the source part's future demand data is not collected or used. • **Y**—the source part's future demand data is collected from the Demand table and included in bucketed totals for the item reported in the SafetyStockItemFutureDemand table.	Boolean	
Multiplier	A multiplier to be applied to the source part's historical and future demands. This value is used to increase or decrease the source part's demand quantities when reported for the safety stock item in the **SafetyStockHistoricalDemand** and **SafetyStockItemFutureDemand** tables. By default, this value is set to 1 (no multiplier is applied).	Quantity	
SourcePart	A reference to the source part whose historical demand, historical supply, or future demand data should be collected and used towards determining safety stock and reorder point for the safety stock item. If multiple categories of historical demand or supply data are required for a given safety stock item, then this can reference the same part as Item.Part (and different historical demand or supply categories chosen from what are set on the SafetyStockItem record) **Referenced table**: Part	Reference	

SafetyStockItemOutlier table

This table reports details related to outliers in historical data associated with a **SafetyStockItem**, analogous to **ForecastItemParametersOutliers**.

- Outliers are calculated based on the **SafetyStockItem** table and the results are stored in the **SafetyStockItemOutlierResult** and the **SafetyStockItemOutlierResultSummary** tables.
- The LeadTime in **SafetyStockHistoricalSupply** and the Quantity in **SafetyStockHistoricalDemand** will be corrected based on the data in this table.

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Table 6.289: SafetyStockItemOutlier fields

Field name	Description	Data type	Key
Adjustment	The quantity of the outlier adjustment. This can be negative value.	Quantity	
Date	The historical date to which the outlier data reported on this record pertains. Each date reported here belongs to the item's **Type.IntervalsCalendar**. For example, dates might mark the beginning of a weekly or monthly period.	Date	Yes
Notes	Descriptive notes appended to this record.	Notes	
ProcessingRule	Specifies the historical data series to which the outlier belongs. Valid values are: • **Demand**—Indicates that the outlier belongs to the historical demand data series. • **Lead time**—Indicates that the outlier belongs to the lead time variability data series. This value is used for single-echelon only.	String(Enum)	Yes
SafetyStockItem	A reference to the **SafetyStockItem** record to which the outlier adjustments should apply. **Referenced table**: SafetyStockItem	Reference	Yes
UseWhen DetectingOutliers	Specifies whether to apply or ignore the **SafetyStockItem** record before detecting outliers when calculating **SafetyStockOutlierResult**. Valid values are: • **Y**—The historical data series for the **SafetyStockItem** record is used to detect outliers. • **N**—The historical data series for the **SafetyStockItem** record is not used to detect outliers. **Default value**: Y **Note:** This field has the same functionality as **ForecastItemParametersOutlier.ProcessingRule** = "All".	Boolean	

SafetyStockOverride table

The **SafetyStockOverride** table supports the Inventory Planning workbook. This table holds the results of both single-echelon safety stock calculations (recommendations for a single part) and multi-echelon safety stock calculations (recommendations for an end item and its related parts within a family).

Each record in this table contains a calculated safety stock quantity for a given part and date as taken from the SafetyStock worksheet in the Inventory Planning workbook. These results are copied using a worksheet copy command and are based on either an underlying transformation function (single-echelon parts) or an analytic as reported in the **MEIOStageResults** table (multi-echelon parts). Once

copied to this table, the calculated results are preserved and can be subsequently accessed again without the need to rerun the underlying analytic or function. As well, this table allows for overriding both single and multi-echelon safety stock recommendations as required. For example, if time phased safety stock calculations are produced, there might be a need to adjust the safety stock calculated in a particular month but accept all other recommendations.

The final results in the **SafetyStockOverride** table, including any specified overrides, can subsequently be used to overwrite or update the part's input safety stock values as provided in either the **TimePhasedSafety** table (for time-phased items) or **Part** table (for non time-phased items). This is done using a worksheet command in the Inventory Planning workbook. Note that a part's input values will only be only overwritten if their **SafetyStockPart.CopySafetyStock** value is set to "Y".

Table 6.290: SafetyStockOverride (Mfg) fields

Field name	Description	Data type	Key
AverageDemand	Average demand for the part. Corresponds to the AverageDemand field in the SafetyStockTimePhasedResult table. This field was added in Version 2016.2.	Quantity	
CalculatedService Level	The service level calculated using bounded safety stock. Corresponds to the CalculatedServiceLevel field in the SafetyStockTimePhasedResult table. This field was added in Version 2016.2.	Quantity	
CurrentQuantity	The current safety stock quantity, as determined and used by Netting calculations, that is effective for the part on the Date of this record.	Quantity	
Date	The date this safety stock recommendation applies to. For time-phased calculations, this reports the appropriate bucket date. For non time-phased calculations this always reports the RunDate.	Integer	
FutureDemand	Future bucketed demands for this part as determined by Netting calculations.	Quantity	
Id	A unique string identifier for this record (typically, generated based on workbook commands in the Inventory Planning workbook).	String	Yes
MaximumDays OfSupply	Maximum days of supply for the part. Corresponds to the **MaximumDaysOfSupply** field in the **SafetyStockTimePhasedResult** table. **Note:** This field was added in Version 2016.2.	Quantity	
MaximumQuantity	Maximum quantity for the part. Corresponds to the **MaximumQuantity** field in the **SafetyStockTimePhasedResult** table. **Note:** This field was added in Version 2016.2.	Quantity	

Table 6.290: SafetyStockOverride (Mfg) fields

Field name	Description	Data type	Key
MinimumDays OfSupply	Minimum days of supply for the part. Corresponds to the **MinimumDaysOfSupply** field in the **SafetyStockTimePhasedResult** table. **Note:** This field was added in Version 2016.2.	Quantity	
MinimumQuantity	Minimum quantity for the part. Corresponds to the **MinimumQuantity** field in the **SafetyStockTimePhasedResult** table. This field was added in Version 2016.2.	Quantity	
OriginalQuantity	Calculated safety stock quantity recommended for the part and effective date identified on this record. **Note:** If a part is configured as a single-echelon safety stock item, but it is subsequently brought into a multi-echelon family under another item, then its safety stock value as recommended by multi-echelon calculations is the one reported here.	Quantity	
Part	A reference identifying the part that the safety stock recommendations record pertains to, and containing the setting used to enable or disable automatic safety stock updates for the part. If the part is used in multi-echelon calculations, this reference also reports details of parameters used in recommending the safety stock level. Other details that are mostly relevant to multi-echelon safety stock calculations can also be accessed via this reference. **Referenced table:** SafetyStockPart	Reference	Yes
Quantity	This field can, optionally, be used to manually override the calculated safety stock recommendation. When values are copied to this table, this field is initially set to the same safety stock value as is reported in the **OriginalQuantity** field. However, when values in this table are used by the worksheet copy command to update safety stock values in either the **TimePhasedSafety** or **Part** tables, the value in this field is used. Therefore, any required adjustments or overrides to the safety stock level should be made in this field. The original recommendations are preserved in the **OriginalQuantity** field.	Quantity	
ServiceLevel	The service level calculated using unbounded safety stock. Corresponds to the **ServiceLevel** field in the **SafetyStockTimePhasedResult** table. **Note:** This field was added in Version 2016.2.	Quantity	

Table 6.290: SafetyStockOverride (Mfg) fields

Field name	Description	Data type	Key
StandardDeviation Demand	Calculated standard deviation of demand for the part. Corresponds to the S**tandardDeviationDemand** field in the **SafetyStockTimePhasedResult** table. **Note: StandardDeviationDemand** should be expressed in **Type.IntervalsCalendar**. **Note:** This field was added in Version 2016.2.	Quantity	
Unbounded Quantity	For single-echelon parts, this is the safety stock level without any bounds applied. For multi-echelon parts, this is the safety stock level required to meet the service level you have specified for the customer-facing part. If RapidResponse was able to calculate a safety stock level that meets your specified service level without violating safety stock bounds, this quantity matches the **OriginalQuantity** field. Corresponds to the **UnboundedQuantity** field in the **SafetyStockTimePhasedResult** table. **Note:** This field was added in Version 2016.2.	Quantity	

SafetyStockPart table

The **SafetyStockPart** table supports the Inventory Planning workbook. This table holds records pertaining to parts involved in single-echelon safety stock calculations (parts configured as safety stock items) and multi-echelon safety stock calculations (includes end-item assemblies configured as safety stock items as well as lower-level components brought into multi-echelon families under an end-item).

Most fields contained in this table report calculated parameters used in multi-echelon safety stock calculations. These details are based on values originally calculated for the **MEIOStage** and **MEIOStageResult** tables, and are copied to this table through a worksheet command in the Inventory Planning workbook. By copying the calculated results to this table, they can be saved and subsequently accessed again without the need to rerun the underlying analytic.

Additionally this table is used to set whether a part's recommended safety stock level, either from single or multi-echelon calculations, is used to overwrite (update) the part's existing input safety stock value. If the **CopySafetyStock** field is set to "Y" on the record for a given part, then the "CopySafetyStock" worksheet command in the Inventory Planning workbook will copy the recommended safety stock value (s) to either the **TimePhasedSafety** table (for time-phased results) or the **Part** table (for non time-phased results).

Table 6.291: SafetyStockPart (Mfg) fields

Field name	Description	Data type	Key
AverageDemand	Average historical demand calculated for the part in multi-echelon calculations. For customer facing end-items, this is based on demands placed directly on the part. For all lower level component parts, this is based on demand values propagated from higher level parts in the family. **Note: AverageDemand** should be expressed in **Type.IntervalsCalendar**.	Quantity	
AverageFuture Demand	Average of future bucketed demands for this part as determined by Netting calculations.	Quantity	
CopySafetyStock	Indicates if the calculated safety stock recommendations for the part, as reported in the **SafetyStockOverride** table, are used to overwrite (update) its existing input values defined in either the **TimePhasedSafety** or **Part** table. Valid values are: • **Y**—safety stock recommendations from either single or multi-echelon calculations are used to update the part's existing input values. This is typically done using a worksheet copy command run in the Inventory Planning workbook. • **N**—safety stock recommendations are not used to update existing input values.	Boolean	
CumulativeLead Time	Total time needed to process the part and all of its dependent parts in a multi-echelon family. Expressed in terms of the Gregorian (everyday) calendar.	Quantity	
Echelon	Indicates the level in the multi-echelon family where the part is located. For example, a customer-facing end item will have a value of 0, the stages/parts immediately below that a value of 1, the next level a value of 2, and so on.	Integer	
Family	A reference to the customer-facing end item at the head of the multi-echelon family (part) that this part belongs to. **Referenced table**: Part (nullable)	Reference	
Id	A unique string identifier for this record (typically, generated based on workbook commands in the Inventory Planning workbook).	String	Yes

Table 6.291: SafetyStockPart (Mfg) fields

Field name	Description	Data type	Key
IncomingService Time	Maximum service time found across all component parts directly below this one in a multi-echelon family. This is the maximum number of days within which the part can expect to have its immediate demands satisfied. Expressed in terms of the **MEIOReporCalendar** (on the **AnalyticsConfiguration** table).	Quantity	
LeadTime	Calculated time needed to process parts that are used in multi-echelon calculations This field relies on the part source's lead time, but is expressed in terms of the **MEIOReportCalendar** (on the **AnalyticsConfiguration** table) using the following expression: `PartSource.LeadTimeDate - RunDate`	Quantity	
OutgoingService Time	Recommended service time for the part (set and used by multi-echelon calculation. This is the maximum number of days within which the part guarantees it can satisfy the demands placed on it. For multi-echelon safety stock items, this is set to an input value specified on the safety stock item (if the input service time provided is greater than the cumulative lead time for the part, that cumulative lead time value is used instead). For all parts below those ends items in a multi-echelon family, this is calculated with the aim of minimizing total inventory holding costs. Expressed in terms of the **MEIOReporCalendar** (on the **AnalyticsConfiguration** table).	Quantity	
Part	A reference to the part the safety stock details on this record apply to. **Referenced table:** Part	Reference	Yes
PartSource	The primary part source for this part. This reference might be empty on records for parts belonging to multi-echelon families in certain situations. For example, if a buy part has multiple sources of the same priority as the primary source, then the part will have one record for each of those sources and an additional record with no part source reported (just the part itself). For more information about how part sources are reported in families, see "Create multi-echelon families" on page 1990. **Referenced table:** PartSource (nullable)	Reference	

Table 6.291: SafetyStockPart (Mfg) fields

Field name	Description	Data type	Key
SafetyStockItem	A reference to the safety stock item this record pertains to. This field is populated if the part has been configured as either a single-echelon or multi-echelon safety stock item (**ProcessingRule** field on SafetyStockItem will return either a value of either "SingleEchelon" or "MultiEchelon". If this record pertains to a lower level part brought into a multi-echelon family under a customer facing end item, then this reference is Null. **Referenced table:** SafetyStockItem (nullable) **Note: SafetyStockItem** should be expressed in **Type.IntervalsCalendar**.	Reference	
ServiceLevel	Service level used for this part when determining safety stock level. For parts configured as a safety stock item, this reports the service level provided as input on the **SafetyStockItem** record. For lower-level parts brought into multi-echelon families under a customer-facing end item, then the service level is typically set to that given on the item that brought it into the family. However, if a part is found under multiple items in a family, then this value is a weighted combination of those item's service levels.	Quantity	
StandardDeviation Demand	Standard deviation of historical demand calculated for the part.	Quantity	
StandardDeviation LeadTime	If this record pertains to a part in a multi-echelon family for much variable lead time is assumed, this field reports the calculated standard deviation of lead time for the part source.	Quantity	

Table 6.291: SafetyStockPart (Mfg) fields

Field name	Description	Data type	Key
Type	For parts involved in multi-echelon calculations, this identifies the part/stage type within the family (indicating how it was brought into the family). Valid values are: • **Buy**—a purchased part located at an internal stage in the family (**SafetyStockItem** reference will be Null). If the part has multiple sources of the same priority, there can be multiple records reported for the part (with different **Id** values). • **Customer**—a customer-facing end item where the part is referenced by a **SafetyStockItem** record with a **ProcessingRule** value of "MultiEchelon". • **InternalCustomer**—a customer-facing end item located at an internal stage in the family (the part is referenced by a **SafetyStockItem** record with a **ProcessingRule** value of "MultiEchelon"). For example, a safety stock item might have demand from external customers as well as demand from other internal parts (for example, as a transfer part). • **Make**—a manufactured part located at an internal stage in the family (**SafetyStockItem** reference will be Null). • **Transfer**—a part that is sourced from another site and located at an internal stage in the family (**SafetyStockItem** reference will be Null). Note that if a Transfer part has multiple sources, or if a given source supplies multiple parts with different transit times, there can be multiple records reported for the part (with different **Id** values). **Note:** For more information about the types of stages in a multi-echelon family, see "Create multi-echelon families" on page 1990.	String (Enum)	
UnitHoldingCost	The calculated cost of holding one unit of inventory for this part (based on the part's cost values and settings in **Part.InventoryHoldingRate** and **MultiEchelonSafetyStockRule.CostRule**). This value is required as input by multi-echelon calculations and used in determining the most cost effective placement of safety stock across the parts in a multi-echelon family.	Money	

SafetyStockPart table – calculated and set fields

The following describes the calculated and set fields in the **SafetyStockPart** table. For an introduction to this table, and descriptions of its input fields, see "SafetyStockPart table" on page 560.

Table 6.292: SafetyStockPart (Mfg) calculated fields

Field name	Description	Data type	Key
CalcFutureIntervals OfSupply	Calculated number of intervals of future demand (in terms of the **ReportCalendar**) that could be satisfied by the calculated safety stock on **RunDate** (where safety stock is taken from the **Quantity** field on the **SafetyStockOverride** table).	Quantity	
SafetyStock Overrides	Set of **SafetyStockOverride** records that reference this safety stock part.	Set	

SafetyStockRangeOfCoverage table

This table is used to define the range over which average demand is calculated for the purpose of calculating a part's recommended safety stock using the **SafetyStockItemType.TimePhasedProcessingRule** of "DaysOfSupplyForward" or "DaysOfSupplyBackward"

This table was added in Version 2016.2, Service Update 7.

Table 6.293: SafetyStockRangeOfCoverage fields

Field Name	Description	Data Type	Key
Count	The number of intervals over which average demand is calculated. For multi-echelon safety stock items, the intervals are days on the Everyday calendar. For single-echelon safety stock items, the intervals are days on the calendar specified in the **SafetyStockItemType.IntervalsCalendar** field.	Integer	
Offset	The start of the date range that average demand is calculated for, relative to the date for which safety stock is being computed. For multi-echelon safety stock items, this value is a number of days on the Everyday calendar. For single-echelon safety stock items, it is a number of calendar intervals on the calendar specified in the **SafetyStockItemType.IntervalsCalendar** field.	Integer	
Part	The part that the **Count** and **Offset** values apply to. **Referenced table**: Part	Reference	Yes

SafetyStockTimePhasedBounds table

This table supports the Inventory Planning workbook. It contains minimum and maximum safety stock bounds for parts and the dates on which these bounds start to apply. When safety stock bounds are used, RapidResponse does not recommend safety stock quantities outside of the specified bounds.

Safety stock bounds are specified either in terms of days of supply or quantity. The table can hold maximum and minimum values for both days of supply and quantity in the same record (four fields) but only one set (days of supply or quantity) can be used at a time. **SafetyStockItemType** settings control whether quantity bounds, days of supply bounds, or neither are used. For more information, see "Determining whether Days of Supply or Quantity bounds will be used" on page 1959.

A part can have multiple safety stock bounds records that become effective on different dates. The **Date** field in the **SafetyStockTimePhasedBounds** table specifies the date on which the safety stock bounds in that record start to apply. A bounds record remains effective until a bounds record with a later date overrides it.

If one or more records fall within a period, the one with the later date is used for that period.

This table was added in Version 2016.2.

Notes:

- When the **SafetyStockTimePhasedBounds** table contains records, safety stock is not allowed on artificial stages, even if **AnalyticConfiguration.AllowSafetyStockOnArtificialStages** is set to True.
- If a minimum bound is higher than the corresponding maximum bound, they cancel each other out, and are both treated as -1.

Table 6.294: SafetyStockTimePhasedBounds fields

Field name	Description	Data type	Key
Date	The date on which these bounds starts to apply.	Date	Yes
MaximumDaysOf Supply	Maximum days of supply for the part. **Default value**: -1. **Note:** Negative numbers are interpreted as positive infinity, and do not result in an upper bound being applied.	Quantity	
MaximumQuantity	Maximum quantity for the part. **Default value**: -1. **Note:** Negative numbers are interpreted as positive infinity, and do not result in an upper bound being applied.	Quantity	
MinimumDaysOf Supply	Minimum days of supply for the part. **Default value**: -1. **Note:** Negative numbers are interpreted as negative infinity, and do not result in a lower bound being applied.	Quantity	
MinimumQuantity	Minimum quantity for the part. **Default value**: -1. **Note:** Negative numbers are interpreted as negative infinity, and do not result in a lower bound being applied.	Quantity	

Table 6.294: SafetyStockTimePhasedBounds fields

Field name	Description	Data type	Key
Part	The part to which these bounds apply **Referenced table**: Part	Reference	Yes

SAP_MD04 table

The SAP_MD04 table contains stock requirement information, including supplies and demands for parts, created using the SAP MD04 transaction. This information is intended to be used for validating input data from SAP, and is not intended to be used for planning or decision making in RapidResponse.

The MD04 data consists of supply, demand, inventory, and allocation information for a part. This table can be used in worksheets to compare supply and demand values brought in to RapidResponse with the values as extracted directly from your SAP data source.

This table was added in RapidResponse 2016.2 Service Update 9.

Table 6.295: SAP_MD04 (Validation) fields

Field name	Description	Data type	Key
BAUGR_ PartNumber HigherLevel Assembly	The parent assembly that drives a dependent demand. This represents the BAUGR field in the SAP MDPS structure.	String	
Comments	Additional information about the supply and demand records.	String	
DAT00_ ReceiptDate	The date supply is planned to be available. This represents the DAT00 field in the SAP MDPS structure.	Date	
DAT01_ DeliveryDate	The date demand is planned to be delivered. This represents the DAT01 field in the SAP MDPS structure.	Date	
DEL12_ MRPElement Number	Production order numbers associated with supply and demand items, such as reservations, requirements, orders, and so on. This represents the DEL12 field in the SAP MDPS structure.	String	
DELET_ Scheduling Number	The scheduling line number for the purchase order, scheduling agreement, or sales order for this part. This represents the DELET field in the SAP MDPS structure.	String	
DELKZ_ MRPElement Type	The origin of the supply or demand item, such as a reservation, planned order, or production number. This represents the DELKZ field in the SAP MDPS structure.	String	

Table 6.295: SAP_MD04 (Validation) fields

Field name	Description	Data type	Key
DELNR_ MRPElement Data	The purchase requisition number, sales order number, planned order number, or purchase order number. This represents the DELNR field in the SAP MDPS structure.	String	
DELPS_ MRPElement Item	The reservation number of a dependent requirement of the production order. This represents the DELPS field in the SAP MDPS structure.	String	
Excluded	Whether this record is excluded from the comparison.	Boolean	
LIFNR_Vendor	The vendor number for a supply. This represents the LIFNR field in the SAP MDPS structure.	String	
MATNR_Material	The part number being analyzed. This represents the MATNR field in the SAP MDPS structure.	String	
MNG01_Recorded RequiredQuantity	The open quantity of a demand, supply, or inventory. This represents the MNG01 field in the SAP MDPS structure.	Integer	
STLKN_ ItemNodeNumber	Identifies the item node associated with this component in the BOM structure. This represents the STLKN field in the SAP MDPS structure.	String	
STLNR_ BillOfMaterial	The bill of material number identified in the production order. This represents the STLNR field in the SAP MDPS structure.	String	
STLTY_ BOMCategory	The bill of material type for the order. This can be one of the following values: • **M**—Material BOM • **K**—Order BOM This represents the STLTY field in the SAP MDPS structure.	String	
STPOZ_Counter	The item counter associated with the component in the bill of materials. This represents the STPOZ field in the SAP MDPS structure.	String	
WERKS_Plant	The site the part is present in. This represents the WERKS field in the SAP MDPS structure.	String	
Part	The part the supply and demand record is defined for. **Referenced table:** Part	Reference	

Table 6.295: SAP_MD04 (Validation) fields

Field name	Description	Data type	Key
Site	The site the supply and demand record is defined for. **Referenced table:** Site	Reference	

ScheduledReceipt table

The **ScheduledReceipt** table contains supply for a single part that has an assigned due date. Examples of supplies are: purchase orders and manufacturing orders.

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
ActionCode	The action code for an order line brought in from the Real-time Integration Service and included in a supplier collaboration process. This field is taken from the IDOC message and can contain the following values: • **Added**—Records defined by the message are added to the RapidResponse database. If the specified records already exist, the message fails. • **Changed**—Records defined by the message are modified, or inserted if they do not already exist in the RapidResponse database. • **Cancelled**—Records defined by the message are cancelled, which modifies the status of the order records. • **Deleted**—Records defined by the message are deleted. • **NotChanged**—Records defined by the message are not modified, so no actions are taken and the record is ignored. • **Locked**—Records defined by the message are locked, and cannot be modified. This field is in the **SupCollab** namespace. This field maps to the ACTION field in the E1EDP01 segment of the IDOC message.	String	
ActualDockDate	The actual date this supply arrived at the receiving dock, which is used in historical reporting. This field is in the **Solutions** namespace. **Note:** This field was added in the Version 2016.2, Service Update 5.	Date	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
ActualShipDate	The actual date this supply was shipped from the supplier, which is used in historical reporting. This field is in the **Solutions** namespace. **Note:** This field was added in the Version 2016.2, Service Update 5.	Date	
AllotmentOverride	A reference to the allotment override reserving component supply to this work order. The reserved supply is allocated to any work order(s) referencing the allotment override before it can be made available to other orders. **Referenced table:** AllotmentOverride (Nullable)	Reference	
AnchorDate	The anchor date associated with this supply. When the BOMType.DateRule is set to 'AnchorDate', the anchor date is used instead of the order start or due date in bill effectivity. If bill effectivity is based on anchor date, material allocations for an order are frozen despite subsequent rescheduling of the order. Anchor date logic is ignored if the AnchorDate equals d'Undefined' or if BOMType.DateRule is not 'AnchorDate'. **Default value**: Undefined	Date	
BOMAlternate	A reference to the alternate BOM used by this scheduled receipt. When Netting logic performs explosion calculations, only components that have **BillOfMaterial.Alternate** matched to either the value in this field or the first value in the **BOMAlternate** table (that is, common components) can potentially create new allocations for this scheduled receipt. In cases where this value differs from the value in **PartSource.BOMAlternate,** the value in this field is used, and the part source value is ignored. For more information about alternate BOMs, see "Alternate BOM calculations" on page 1759. **Referenced table**: BOMAlternate **Note:** This field is ignored for a given scheduled receipt unless **Order.Type.Source** = 'Make', and **Status.State** is set to either 'NotReleased', 'Planned', or 'ReleasedButNotAllocated'.	Reference	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
BuildStatus	Records whether an order was scheduled to completion. Valid values are: • **No_Build**—the order was not scheduled to start within the scheduling horizon. • **Partial**—the order was started, but not completely scheduled (finished) in the scheduling horizon. • **Complete**—the order was completely scheduled in the scheduling horizon. **Note:** This is an obsolete field. **Default value**: No_Build	String (Enum)	
BuyerCommit	For an order that is undergoing a supplier collaboration process, whether the buyer has committed to the order. A buyer can commit only after the supplier has committed. Valid values are: • **Y**—The buyer has committed to the order. • **N**—The buyer has not committed to the order. This field is in the **SupCollab** namespace.	Boolean	
ConfirmDate	For an order line committed to by a supplier, the due date of the confirmed order. This field provides data for an IDOC message response or order change. This field is in the **SupCollab** namespace. This field maps to the EDATU field in the E1EDP01.E1EDP20 segment of the IDOC message.	Date	
ConfirmQty	For an order line committed to by a supplier, the quantity the supplier has confirmed. This field provides data for an IDOC message response or order change. This field is in the **SupCollab** namespace. This field maps to the WMENG field in the E1EDP01.E1EDP20 segment of the IDOC message	Quantity	
ConfirmationDateTime	For an order line committed to by a supplier, the date and time the supplier confirmed the order. This field provides data for an IDOC message response or order change. This field is in the **SupCollab** namespace. The Date component of this field maps to the EDATU field in the E1EDP01.E1EDP20 segment of the IDOC message. The Time component of this field maps to the EZEIT field of the E1EDP01.E1EDP20 segment of the IDOC message.	DateTime	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
CurrentOperation	When a routing has been partially completed, this field is used to start the processing of the routing at an operation other than the first operation. When backward scheduling the order, all operations remaining will be scheduled relative to the due date of the order. When forward scheduling the order, all operations remaining will be scheduled relative to the MRP RunDate. This field is used by the Capacity Requirements Planning analytic. If this analytic is not being used, this field can be left blank. **Default value**: 0 **Note:** This field is applicable to operations defined through the **Operation** table only (for operations configured through the **CRPOperation** table, the **CRPOperationState** table should be used to indicate the current operation).	Integer	
CurrentOperation CompleteQuantity	The number of units completed by the current operation. The difference between the scheduled receipt's Quantity and this field indicates what remains to be processed by the current operation. This field is used by the Capacity Requirements Planning analytic. If this analytic is not being used, this field can be left blank. **Default value**: 0 **Note:** This field is applicable to operations defined through the **Operation** table only (for operations configured through the **CRPOperation** table, the **CRPOperationState.CompleteQty** should be used to indicate the amount of the current operation that is complete).	Integer	
DockDate	The date this supply is supposed to be delivered to the manufacturer's dock. Analytics will calculate from DueDate if DockDate is 'Undefined'.	Date	
DockToStock LeadTime	The time needed to move materials from the receiving dock to available inventory. This value is used in lead time calculations when the part source's lead time is overridden by the scheduled receipt (as set by the values on **SupplyStatus.LeadTimeOverride** and **SupplyType.LeadTimeRule**).	Quantity	
DrawingRevision	This field is used for internal purposes in RapidResponse. **Note:** This field is obsolete.	String	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
DueDate	The date the order is planned to be in stock. **Default value**: Future	Date	
ExpiryDate	The date on which the material in this supply expires (is no longer usable). If a value is provided in this field, it is also reported in the **CalcExpiryValue** field (as long as **PartType.ExpiryRule** is not set to "Ignore"). Otherwise, RapidResponse calculates the **CalcExpiryValue**. For guidelines about when to populate this field, see "Providing an expiry date for scheduled receipts" on page 1605. **Default value:** Undefined	Date	
ExpiryStartDate	The date on which the material in this supply starts to expire. If a value is provided in this field, it is also reported in the **CalcExpiryStartDate** field (as long as **PartType.ExpiryRule** is not set to "Ignore"). Otherwise, if a value is not provided in this field, then **CalcExpiryStartDate** is either determined from an analytic calculation or set to "Future" (depending on the configured expiry rules and types). This field is most typically used with expiry parts that are designated as limiting components. **Note:** This field was modified in Version 2016.2, Service Update 8 (previously, this was a calculated field and there was no way to provide it as input).	Date	
FixedLeadTime	The time needed to produce the material (independent of order quantity). This value is used in lead time calculations when the part source's lead time is overridden by the scheduled receipt (as set by the values on **SupplyStatus.LeadTimeOverride** and **SupplyType.LeadTimeRule**.	Quantity	
FocusIndicator	Flags an order to be reported with separate statistics. Focus orders are not scheduled differently; just their statistics are reported separately. Valid values are: • **Y** • **N** **Default value**: N **Note:** This field is obsolete. As a result, set this field to N.	String	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
InputModel	The model that is associated with this scheduled receipt. Based on the **SupplyStatus.ModelUsage** setting, either this model or the default model (the None model) is effective for the receipt. The actual usage of the receipt's effective model in analytic calculations is determined by the setting in the **Part.MUEPoolNettingType.ModelRule** field. This also depends on the Model-Unit Effectivity capability having been enabled. **Referenced table:** Model	Reference	
InputPool	The pool to which this scheduled receipt belongs. This field supports inventory pooling, and is used only when **Part.MUEPoolNettingType.PoolRule** is set to either "Net" or "Optional". If this field is left Null, then the pool from **Order.Pool** is used instead. Therefore, this field is typically used when the pools applicable to supplies more naturally correspond to individual line items rather supply orders, or when there is a need to inject pool information at the component level from scheduled assembly supply. **Referenced table:**Pool (Nullable) **NOTE**: This field was added in Version 2016.2, Service Update 19.	Reference	
LastModified	The date on which the order was last updated with data from the Real-time Integration Service. This field is also used to determine which order lines are no longer applicable. This field is in the **SupCollab** namespace.	Date	
Line	A unique identifier associated with each ScheduledReceipt record with the same Order.Type and Order.Id. **Default value**: blank string	String	Yes
LineValue	The total price of the order line brought in from the Real-time Integration Service and included in a supplier collaboration process. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the NETWR field of the E1EDP01 segment of the IDOC message.	Money	
Order	A value that determines the processing policies of this scheduled receipt. **Referenced table:** SupplyOrder **Reference Syntax:** Order.Type, Order.Id, Order.Site	Reference	Yes

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
OrderPriority	The priority associated with this order when **SupplyStatus.PrioritySource** is set to "FromInput". Ignored for all other PrioritySource values (unless the Netting analytic does not use this receipt to satisfy any demands). **Referenced table:** OrderPriority **Note:** The effective order priority on scheduled receipts is reported in the **EffOrderPriority** field.	Reference	
OriginalDueDate	The date on which the order was promised or contracted to be due. This is the DueDate that was originally on the ScheduledReceipt when the allocations were made. This may be different than the due date because the due date reflects the achievable target due date. **Default value**: Undefined	Date	
OriginalQuantity	This field should only be populated if it is used to calculate the **Allocation.EffQuantity** field (that is, if RapidResponse's mechanism of updating the Allocation every time there is a quantity change on the scheduled receipt is used). If populated, this field must contain the same value as the **Quantity** field. Imported allocations will have the **EffQuantity** (quantity that is used in netting calculations) recalculated based on the **ScheduledReceipt.OriginalQuantity * BillOfMaterial.QuantityPer**; that is, the imported **Allocation.RemainingQuantity** is not used in netting calculations. **Note:** For more information about automatic allocation adjustments, see "Automatic allocation adjustment" on page 1399).	Quantity	
OriginalRecordId	Identifies records that have been sent to your enterprise data sources in a closed-loop operation. This is a concatenation of the date the closed-loop operation was performed and the RecordId value for the record. When data is next updated, this value is compared to the new ScheduledReceipt records and ensures the records are not double-counted. The RecordId value is a number representing the sequence a record was created in. Each additional record added to the table increments the RecordId by one, and RecordId values are not reused when records are deleted. For more information about RecordId, see the *RapidResponse Resource Authoring Guide*. This field is in the **Solutions** namespace.	String	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
Part	Part for which this ScheduledReceipt exists. This field references the Part table. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	
PreShipLT	The time needed by the source to collect the inventory and prepare it for shipment. This value is used in lead time calculations when the source's lead time is overridden by the scheduled receipt (as set by the values on **SupplyStatus.LeadTimeOverride** and **SupplyType.LeadTimeRule)**.	Quantity	
PreviousQty	For an order line that was changed by an update from the Real-time Integration Service, the line's quantity before the change. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the AMENG field of the E1EDP01.E1EFP20 segment of the IDOC message.	Quantity	
Priority	Defines the priority of the Purchase Order. Valid values range from one to five. One represents the highest priority. Specify **X** for this field which represents no expressed priority. **Note:** This field is only applicable to RapidResponse legacy applications.	String (Enum)	
Processed	For an order brought in by the Real-time Integration Service and undergoing a supplier collaboration process, whether the order has been committed to by the supplier, accepted and committed to by the buyer, and the confirmation returned to your enterprise data source. Valid values are: • **Y**—The supplier has committed to the order, the buyer has accepted and committed to the order, and the order confirmation has been sent to your enterprise data source. • **N**—Either the supplier has not committed to the order or the buyer has not accepted or committed to the order. This field is in the **SupCollab** namespace.	Boolean	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
ProtectQuantity	Indicates whether the value in the Quantity field is modified when data is edited in a grouped worksheet. This field is used in the Advanced Data Editing dialog box as part of the expression that determines which records are not edited when a grouped worksheet is edited. Valid values are: • **Y**—the Quantity cannot be modified in summarized worksheets. • **N**—the Quantity can be modified. For more information, see "Protect a record" in the *RapidResponse Resource Authoring Guide*. **Default value**: N	Boolean	
Quantity	The total amount that will be available before testing (for example, before applying yield and accounting for any co-product or by-product supply). **Default value**: 0	Quantity	
ReviewedDateTime	The last time this supply had its status reviewed (by its supplier). **Default value**: Undefined	Date Time	
Routing	The routing for the order. It is a reference to the Routing table through the Routing.Id field. **Note:** This field is used by the Capacity Requirements Planning analytic. If this analytic is not being used, a value must still be entered in this field (in this case, it can be set to "None"). **Referenced table:** Routing	Reference	
SavedPriority	Duplicates OrderPriority, load with same data as OrderPriority. Not used by RapidResponse analytics, but might be used by applications, such as custom applications. **Referenced table:** OrderPriority	Reference	
SchedulePriority	A factor used in prioritizing and scheduling orders on work centers when due dates are the same. **Note:** This field is obsolete.	Integer	
SegmentQualifier	For an order line committed to by a supplier, identifies the order total IDOC segment for a shipment notification. This field is taken from the IDOC message. This field is in the **SupCollab** namespace.	String	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
ShipTo	Location to which this order is to be delivered. **Note:** This field is obsolete and only applicable to RapidResponse legacy applications.	String	
Shipment	The shipment that contains (or is planned to contain) this supply. **Referenced table:** Shipment (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
StartDate	The date that the order is scheduled to be released to the supplier or shop floor. **Note:** If the StartDate is undefined, this value is calculated. **Default value**: Undefined	Date	
StartUnit	The unit number (serial number) of the first unit in this receipt. **Default value**: -1	Integer	
Status	Processing rule modifiers on a per line basis used in conjunction with the Order.Type. A Firm status on a WorkOrder will generate NewAllocations, but Open ones already have Allocations. **Referenced tables:** SupplyStatus	Reference	
SupplierCommit	For an order that is undergoing a supplier collaboration process, whether the supplier has committed to the order. A supplier can commit only if the quantity and due date for the order have been accepted. Valid values are: • **Y**—The supplier has committed to the order. • **N**—The supplier has not committed to the order. This field is in the **SupCollab** namespace.	Boolean	
SupplierResponse	For orders undergoing a supplier collaboration process, the supplier's response to the order's proposed quantity and delivery date. Valid values are: • **Accepted**—The supplier accepts the order with the proposed quantity and delivery date. • **Changed**—The supplier accepts the order with a different quantity or delivery date. • **Rejected**—The supplier has rejected the order. This field is in the **SupCollab** namespace.	String	

Table 6.296: ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
TotalQuantity	The total quantity of parts this scheduled receipt includes (Quantity is the balance remaining, so the difference is received + scrap.)	Quantity	
Transaction Sequence	The sequence of the scheduled receipt. This value is assigned to transaction sequence on resulting dependent demands. If dependent demands have the same order priority and the value of **MaintainCommitments** is set to 'Y' for the dependent demands part, then supplies will be allocated by transaction sequence (smallest to largest). **Note:** On startup, the server checks the maximum value for a transaction sequence number and if the value is greater than 0x30000000 hex (or 805306368 decimal), it will reset the sequence number back to zero and re-sequence all the ScheduledReceipt and IndependentDemand records.	Integer	
TransitTime	The number of days the order must be shipped prior to it being available for use in manufacturing. This value is used in lead time calculations when the source's lead time is overridden by the scheduled receipt (as set by the values on **SupplyStatus.LeadTimeOverride** and **SupplyType.LeadTimeRule**).	Quantity	
Unconstrained Quantity	The quantity of this supply that has had constraint applied to it and does not require any more constraint. The constrained quantity is calculated, depending upon SupplyStatus.TargetAccum, as Quantity or TotalQuantity - UnconstrainedQuantity. A result of 0 or a negative value results in no load on the constraint. Values of UnconstrainedQuantity < 0 are ignored. UnconstrainedQuantity is ignored unless the scheduled receipt is limited by constraint.	Quantity	
UnitPrice	The unit price to be used for this order. In cost calculations, UnitPrice is assumed to represent material costs only. If UnitPrice is less than or equal to zero, use PartSource.EffUnitCost less labor and overhead costs for material costs instead. Labor and overhead costs are always calculated from PartSource fields.	Money	
VarLeadTime	A variable portion of lead time dependent on order quantity (this value is multiplied by the scheduled receipt quantity). This value is used in lead time calculations when the part source's lead time is overridden by the scheduled receipt (as set by the values on **SupplyStatus.LeadTimeOverride** and **SupplyType.LeadTimeRule**).	Quantity	

ScheduledReceipt table – calculated and set fields

The following describes the calculated and set fields in the ScheduledReceipt table. For an introduction to this table, and descriptions of its input fields, see "ScheduledReceipt table " on page 569.

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
AvailableDate	The date this supply is likely to be available based on materials, order status, and constraint.	Date	
AvailableStartDate	The date of the earliest constraint allocated to this supply (ship date or start date) converted to a start date. If incremental availability calculations are turned off, then all component materials must be available before constraint can be allocated. If incremental availability calculations are turned on, then only some set of component materials must be available before constraint can be allocated.	Date	
CalcExpiryDate	The calculated date on which the supply in this scheduled receipt will expire (is no longer usable). This field is calculated as follows: • If the **ExpiryDate** field is not "Undefined", then it is to the **ExpiryDate** value . • If the **ExpiryDate** field is "Undefined" and **PartType.ExpiryRule** is "Dependent", then it is set to the earliest calculated expiry date on any component supply allotted to this order. • If **ExpiryDate** is "Undefined" and **PartType.ExpiryRule** is "Self" or "SelfReset", then it is calculated by adding the value in **PartSource.IntervalsToExpiry** (expressed in terms of the part's expiry calendar) to either the supply's available date, dock date, or stock date (as determined by the setting in **PartSourceType.ExpiryDateRule**). • If **PartType.ExpiryRule** is "Ignore", then it is always set to "Future" regardless of any other field settings.	Date	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
CalcExpiryStart Date	The calculated date on which this supply starts to expire. This date is rolled up the product structure through parts using a "dependent" expiry type. For assembly parts with a **PartType.ExpiryRule** setting of "DependentStart", the earliest rolled up **ExpiryStartDate** from supply of a limiting component is then used to start expiry date calculations for the assembly supply. This field is populated as follows: • If an input value is found in **ExpiryStartDate**, then it is always used here (assuming the part's **PartType.ExpiryRule** is not set to "Ignore"). • If no input value is found and the part is designated as a limiting component (that is, the **ExpiryType.ProcessingRule** is set "LimitingComponent), then the component supply's due date, dock date, or start date is used (determined by **PartType.ExpiryDateRule**) • If no input value is found and the part has a **PartType.ExpiryRule** of "DependentStart" with limiting components in its structure, then the earliest expiry start date rolled up from a limiting component supply is sued. • Otherwise, set to "Future". **Note:** This field was added in Version 2016.2, Service Update 6. Previously, the calculated expiry start date was reported in the **ExpiryStartDate** field, however that field has been changed to an input field which can be used to specify a date on which expiration calculations should begin.	Date	
CalcRequestStart Date	The date on which work should start on this supply in order to meet the earliest request date on any driving independent demand it satisfies. Based on the original **RequestDate** exploded down from the driver **IndependentDemand** record and adjusted by lead time at each level in the product structure (if the supply directly satisfies an independent demand itself, then this is just the **RequestDate** adjusted by the part's lead time). If the driver demand is not an independent demand, or has an undefined **RequestDate**, then this is the date on which work should start on this supply in order to meet the due date on the driver.	Date	
CalcStartDate	The order start date as calculated by netting, including allowance for constraint. If StartDate is not d'undefined', StartDate + (Resched Date - OriginalDueDate Part.PlanningCalendars.TimeUnits) Part.PlanningCalendars.TimeUnits Otherwise, if StartDate is d'undefined', CalcStartDate is ReschedDate (dock date) converted to a start date.	Date	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
Constraint	The Constraint that has insufficient remaining capacity to make this supply available for its MaterialAvailableDate. If this supply is not late, this field is empty. **Referenced table**: Constraint	Reference	
CostOfGoodsSold	The total rolled-up costs of goods sold from all component supplies used to fill this order. This field summarizes the date-specific cost values reported in the **ScheduledReceiptCost** table and described on page "ScheduledReceiptCost table" on page 1178.	Money	
DemandDate	The date of the earliest demand that netting matches with this supply. DemandDate can differ from RecommendedDate for non-reschedulable supplies, if constraint limitations result in pre-build recommendations, and for parts that use safety lead time. **Note**: The date reported in this field must belong to the part's **PlanningCalendars.TimeUnits** calendar. If this supply is matched to an independent demand with a due date that does not fall on the part's **TimeUnits** calendar, it is adjusted back to the previous **TimeUnits** interval before being reported in this field.	Date	
DockDateFrom Start	The order dock date after adjustment based on any calendar rounding applied when determining the calculated start date. For example, parts that use "cumulative" lead time might end up with an earlier dock date from start based on rounding between time units, transit, ship, and lead time unit calendars. **Note:** This field was added in Version 2016.2	Date	
DueDateFromStart	The order due date after adjusting for any calendar rounding applied when determining the calculated start date. For example, parts that use "cumulative" lead time might end up with an earlier due date from start based on rounding between time units, transit, ship, and lead time unit calendars. **Note:** This field was added in Version 2016.2	Date	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
EarliestExpiryDate	The earliest date that any supplies being used to satisfy dependent demands from this receipt may expire. Typically, this is determined from the latest **EarliestExpiryDate** found on any demand that Netting calculations determine this supply will satisfy. Otherwise, if there are no consuming demands, this value is calculated by adding the number of expiry calendar intervals defined in **Part.MinimumShelfLife** to the supply due date. For non-expiry parts (**PartType.ExpiryRule is** set to "Ignore"), this field always reports "Past".	Date	
EffDockDate	The date this supply is supposed to be delivered to the receiving dock. If the supply has a valid delivery date, then EffDockDate is calculated from that. Otherwise, if DockDate is not 'Undefined', then equals DockDate. Otherwise, is calculated from DueDate.	Date	
EffDueDate	The date this supply is supposed to be available for use. If the supply has a valid delivery date, then EffDueDate is calculated from that. Otherwise, if DueDate is not 'Undefined', then equals DueDate. Otherwise, is calculated from DockDate.	Date	
EffectiveUnitPrice	The sum of all costs (material, labor and overhead) to produce this receipt, reported in stock units of measure. If the UnitPrice > 0, this value is calculated as `UOMConversion(UnitPrice + max(LaborCost, 0) + max(OverheadCost, 0) + max(OverheadCost2, 0)`. Otherwise, it is set to PartSource.EffUnitCost. Currency conversions on this field use the **EffDueDate** field's conversion rate.	Money	
EffOrderPriority	The effective order priority for this scheduled receipt. Depending on the setting in **SupplyStatus.PrioritySource**, this field either uses the OrderPriority value provided on the scheduled receipt record or is dynamically calculated based on the priorities of the demands satisfied by the receipt in netting calculations (these may not necessarily be the demands that capable-to-promise calculations later allocate to the scheduled receipt). For parts where priority is relevant (for example, when **PartType.CommitLevel** is set "High") , this value is used when allocating limited materials and constraint to an order, and is also propagated down to its dependent components. For more information about how order priority on scheduled receipts is calculated and used, see "Priorities on scheduled receipts" on page 1788. **Referenced table:** OrderPriority	Reference	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
EffQuantity	The quantity of this receipt that remains to be received into inventory, in inventory units of measure. This value is calculated by taking the Quantity and subtracting InTransitConsumedQuantity (if applicable), then adjusting by yield/scrap, considering any co-product or by-product supply, and applying unit of measure. If **SupplyType.ProcessingRule** is set to "Ignore" or **SupplyStatus.State** is set to "Historical", or the part's **PartType.ProcessingRule** is set to "Ignore", this field will always return 0 (zero). In such cases, none of the receipt can be used to cover demand by Netting.	Quantity	
EffShipDate	The date this supply was or should be shipped based on EffDueDate.	Date	
EffStartDate	The date the order was or should be started, calculated from DueDate, DockDate, and StartDate.	Date	
EstimatedExpiry Date	The date that the Netting analytic calculates this supply will expire. Used to determine which demands this receipt can potentially be used to satisfy (supply is only eligible for use if it is on or before the demand's **EarliestExpiryDate**). Note that this date might be different than the actual expiry date reported in **CalcExpiryDate** which takes supply availabilities into account. If a value is provided in the **ExpiryDate** field, then it is used here. Otherwise, calculated by adding the number of expiry calendar intervals defined in **PartSource.IntervalsToExpiry** to either the **EffectiveDueDate**, **StartDate,** or **DockDate** (as determined by the setting in **PartSourceType.ExpiryDateRule**). Non-expiry parts, where **PartType.ExpiryRule** is set to "Ignore", always return a value of "Future".	Date	
ExpiringQuantity	The amount of this scheduled receipt that will not be used before it expires (reaches its **CalcExpiryDate**).	Date	
FirstAvailableDate	The date the first portion of this supply should be available based on materials, order status, and constraint. This value is different than the AvailableDate field in cases where the order has been split due to incremental availability of component materials or constraint consumption in multiple periods.	Date	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
InTransit ConsumedQuantity	The amount of the scheduled receipt that has been consumed (reduced) by related in-transit scheduled receipts. This value is reported in supplier units of measure, and is not adjusted for yield. For more information about in-transit scheduled receipts, see "In-transit scheduled receipts" on page 1734.	Quantity	
MaterialAvailable Date	The date this supply is available calculated from MaterialStartDate.	Date	
MaterialStartDate	Date when all components are available and this order can be started. For work orders, this is based only on component availability. For purchase orders, this is the same as CalcStartDate.	Date	
Model	The model used when netting this record. Either the **Model** value referenced in the **InputModel** field or the default **Model** value ("None") is returned as determined by the settings in **Part.Type.MUEPoolNettting.ModelRule** and **SupplyStatus.ModelUsage.** The following criteria is used: • If **ModelRule** is set to "Net" or "NetWithoutForecast" and **ModelUsage** is set to "Normal", then the **InputModel** value is returned. • If **ModelRule** is set to "Net" or "NetWithoutForecast" and **ModelUsage** is set to "Ignore", then the default **Model** ("None") is returned. • If **ModelRule** is set to "ExplodeOnly" or "Ignore" then the default **Model** ("None") is returned. • If the optional Model-Unit Effectivity capability is not enabled, then the default model ("None") is always returned regardless of field settings. **Referenced table:** Model	Reference	
NeedQty	Quantity of the supply actually needed to cover the demand. This field is expressed in stock units, and inflated to cover yield.	Quantity	
NewProcessCost	Standard labor and overhead cost of planned orders to complete this order. Currency conversions on this field use the **EffDueDate** field's conversion rate.	Money	
NewPurchasesCost	Cost of new purchases required in order to complete this order (through all levels of the bill). Currency conversions on this field use the **EffDueDate** field's conversion rate.	Money	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
OnTimeQuantity	The quantity of this supply that could be completed on or before its ReschedDate (considering materials only) if it was possible to split the order. This value accounts for components available in time to apply lead time to complete the assembly for its ReschedDate including supplies and InKit materials.	Quantity	
ParameterDate	The date that was used to determine which time-phased parameter values are applicable to this supply (compared against the **EffectiveInDate** values on certain applicable tables). For example, if using time-phased yield then the **TimePhasedYield** record effective for the part source on this date determines the yield factor. Also, applies to time-phased **SourceConstraint** records as well as time-phased lead time logic (records in the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables). Depending on the status of the scheduled receipt, this date is set as follows: • For non-reschedulable receipts, the **DueDate** is used (set to "Past" if the **DueDate** is left undefined). • For reschedulable receipts, the **ReschedDate** is typically used (or the date from the first attempt at rescheduling the supply). However, if **SupplyStatus.AllowLatePlan** is set to "Y" and the **ReschedDate** then moves past the demand's due date, that demand due date is used instead. Additionally, if allotment overrides are used this is set to the receipt's effective stock date instead. And for part's that have **PartType.CommitLevel** set to "High", this date will not always correspond to **ReschedDate** (but will always correspond to the period from which the effective quantity of the receipt is calculated). **Note:** This field was added in Version 2016.2, Service Update 12.	Date	
PartSource	The PartSource record that corresponds to this supply. Used by calculations for Transfer (explosion), Ongoing, and Yield processing. For details about how this field is calculated, see "Part sources for scheduled receipts" on page 1389. **Referenced table:** PartSource	Reference	
PartSupplier	A reference to the entry in the PartSupplier table, if any, that has the same part and supplier as this ScheduledReceipt record (Part and Order.Supplier). **Referenced table**: PartSupplier	Reference	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
Pool	The pool used when netting this supply. The value returned here is affected by the **Part.MUEPoolNettingType.PoolRule** setting as follows. • If **PoolRule** is set to "Net" or "Optional" and the **InputPool** reference on this line item is populated, then that referenced pool is used. • If **PoolRule** is set to "Net" or "Optional" and the **InputPool** reference on this line item is Null, then the pool referenced by **Order.Pool** is used (from the supply order). • If **Part.MUEPoolNettingType** is "Ignore", "Forecast", or "ForecastOptional", the part's common pool, from the **Part.PlanningCalendars.CommonPool** reference, is always used. **Referenced table:** Pool	Reference	
RecommendedDate	This field considers rescheduling rules and expedite limits. It reports the date where Netting puts the scheduled receipt for the purpose of demand provision. If the supply is "RecommendOnly", it will report the date where netting wanted it without actually moving it there. Note that dependent demands are not generated with reference to this date.	Date	
Recommended DockDate	The dock date based on RecommendedDate.	Date	
ReschedDate	This field considers rescheduling rules and expedite limits. It reports the date where Netting puts the scheduled receipt in order to generate dependent demands. If the supply is "Reschedulable", then this field gives the same date as RecommendedDate; otherwise, it is the same as the DueDate.	Date	
ReschedDockDate	The dock date based on ReschedDate.	Date	
ShipDate	The date this supply was or should be shipped based on ReschedDate.	Date	
ShipDateFromStart	The order ship date after adjustment based on any calendar rounding applied when determining the calculated start date. For example, parts that use "cumulative" lead time might end up with an earlier ship date from start based on rounding between ship and lead time unit calendars. **Note:** This field was added in Version 2016.2	Date	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
StockUnitQuantity	Represents the supply quantity in inventory unit of measure (UOM), including any reduction for InTransitConsumedQuantity, and without applying yield or scrap factors. This field also does not account for the settings in either the SupplyStatus or SupplyType tables (and thus can return a positive value for orders that are ignored by Netting and have EffQuantity = 0). **Note**: **SupplierUOM** applies to **StockUnitQuantity**.	Quantity	
TaskNeedDate	The earliest date that this supply is required by any project task associated with it through a **TaskSupplyLink** record. If this order is not required by any tasks, then 'undefined' is reported. **Note:** This field belongs to the ProjMgmt namespace.	Date	
ToleranceProfile Zone	A reference to the entry in the ToleranceProfileZone table that applies to this ScheduledReceipt record. It is used in defining the acceptable levels of change when comparing scheduled receipts against baseline historical supply series. The ToleranceProfile for this record is the value referenced from the PartSupplier field, and the ToleranceProfileZone is set to the entry with the lowest ZoneEndInterval value where `Part.PlanningCalendars.RunDate.FirstDate + ZoneEndInterval ToleranceProfile.ToleranceCalendar > ScheduledReceipt.EffDueDate`. **Referenced table**: ToleranceProfileZone	Reference	
Allocations	Set of Allocation records for components that are required when making this order, but which have yet to be removed from inventory.	Set	
CostInfo	Set of **ScheduledReceiptCost** records that contain cost values associated with the set of supplies needed to complete this order. Used in calculations that populate the **CostOfGoodsSold** field.	Set	
CRPOperation States	Set of **CRPOperationState** records used to identify the current operation(s) and the quantity of those work order that has been completed or run through that operation.	Set	

Table 6.297: Calculated ScheduledReceipt (Mfg) fields

Field name	Description	Data type	Key
Notes	A Set of ScheduledReceipt_Notes records. This field can used to enter notes about an scheduled receipt. Each time a note value is added to this field through a worksheet, a new record is created in the ScheduledReceipt_Notes table with a date and timestamp, user ID, note text, and a reference to the ScheduledReceipt record against which it was created.	Note	
TaskSupplyLinks	Set of TaskSupplyLink records that link this scheduled receipt to project tasks.	Set	

Segment table

The **Segment** table describes the segments associated with parts made in a production wheel. This table is an optional and only contains data if your company defines segmentation strategies to use in production planning.

This table was added in Service Update 1902.

Table 6.298: Segment(Solutions) fields

Field name	Description	Data type	Key
Description	A description of the segment.	String	
Recurrence	A reference to the input table that defines how often a part can be produced in a selected production wheel. Referenced table: ProductionRecurrence	Reference	
Site	The site associated with this segment.	Reference	Yes
SortKey	How segments are organized. This field only displays values for segments that have an assigned sort strategy. For example, the Glenday Sieve technique sorts segments by color (Green > Yellow > Blue > Red).	String	
Strategy	The sorting strategy applied to the segments. This field only displays values for segments that have an assigned sorting strategy.	String	
Value	A unique identifier for this segment record. Only one segment can be associated with a part.	String	Yes

Segment table – calculated and set fields

The following describes the calculated and set fields in the **Segment** table. For an introduction to this table, and descriptions of its input fields, see "Segment table" on page 589

Table 6.299: Segment(Solutions) fields

Field name	Description	Data type	Key
PartSolutions	Set of PartSolution records associated with this Segment.	Set	

SequenceFlow table

The SequenceFlow table links the flow of activities within a process instance, meaning it links an activity to its prerequisite and subsequent activities.

Table 6.300: SequenceFlow (ProcOrch) fields

Field name	Description	Data type	Key
Source	The source activity of the link.		
Target	Target activity of the link.		

SequenceFlow table –calculated and set fields

The following describes the calculated and set fields in the SequenceFlow table. For an introduction to this table, and descriptions of its input fields, see "SequenceFlow table" on page 590.

Table 6.301: SequenceFlow (ProcOrch) fields

Field name	Description	Data type	Key
IsRecursive	Indicates if a recursive relationship exists between the prerequisite and subsequent activities.	Boolean	

Shipment table

The **Shipment** table stores the details about individual shipments from a supplier to the manufacturer.

Individual scheduled receipts must be associated with a shipment. Initially, you can supply a single default record as the default shipment.

Table 6.302: Shipment (Mfg) fields

Field name	Description	Data type	Key
Airport	An airport where this shipment is transferred. Typically, this is a reference to a **LogisticsLocation** record where **Type** is set to "Airport." **Referenced table**: LogisticsLocation (Nullable) **Note:** This table was added in Version 2016.2, Service Update 9	Reference	
Carrier	The name of the carrier for this shipment. **Referenced table:** Carrier (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
Customs	A customs location that this shipment passes through. Typically, this is a reference to a **LogisticsLocation** record where **Type** is set to "Customs." **Referenced table**: LogisticsLocation (Nullable) **Note:** This table was added in Version 2016.2, Service Update 9	Reference	
DeliveryDateTime	Date and time this shipment is scheduled to arrive. **Default value**: Undefined	Date Time	
Id	The name for this shipment. The name is arbitrary, but must be unique in combination with Supplier. The Shipment ID is needed as the key value while allowing other fields to be editable. **Default value**: Unassigned	String	Yes
Port	A port where this shipment is transferred. Typically, this is a reference to a **LogisticsLocation** record where **Type** is set to "Port." **Referenced table**: LogisticsLocation (Nullable) **Note:** This table was added in Version 2016.2, Service Update 9	Reference	
ShipDateTime	Date and time this shipment is scheduled to be shipped or was shipped. **Default value**: Undefined	Date Time	
Supplier	The supplier that is providing the goods being shipped in this shipment. **Referenced table:** Supplier **Reference Syntax**: Supplier.Id, Supplier.Site	Reference	Yes
Waybill	The waybill number for this shipment. **Default value**: blank string	String	

Shipment table – calculated and set fields

The following describes the calculated and set fields in the Shipment table. For an introduction to this table, and descriptions of its input fields, see "Shipment table" on page 590.

Table 6.303: Calculated Shipment (Mfg) fields

Field name	Description	Data type	Key
Scheduled Receipts	Set of ScheduledReceipt records included in this shipment.	Set	

ShipSet table

The **ShipSet** table is used to identify ship sets, which are groups of independent demands from the same demand order that are required to ship together as a whole order. Values are automatically loaded into this table from **IndependentDemand** records during a data import or update.

Table 6.304: ShipSet (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this ship set.	String	
Type	A reference to the ship set type. Used to control processing of the ship set when non-blocking allocations are used. **Referenced table:** ShipSetType (Nullable)	Reference	
Value	A unique identifier for this ship set.	String	Yes

ShipSet table – calculated and set fields

The following describes the calculated and set fields in the ShipSet table. For an introduction to this table, and descriptions of its input fields, see "ShipSet table" on page 592.

Table 6.305: Calculated ShipSet (Mfg) fields

Field name	Description	Data type	Key
Independent Demands	The set of independent demands that belong to this ship set.	Set	

Shop table

The **Shop** table contains information identifying unique shop floor locations used in a kanban material replenishment system. It is also used to set the date horizon for calculations that populate some fields in the **ShopKanban** table.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.306: Shop (Mfg) fields

Field name	Description	Data type	Key
Address	A unique identifier for a shop floor location.	String	Yes
Description	A description of the shop floor location.	String	
Site	The site where this shop is located. This field is optional. **Referenced table:** Site	Reference	Yes
KanbanHorizon	The number of days out from today that Kanban Manager considers when calculating the average daily demand for kanban parts at this shop address (past due demands are also included in this horizon).	Integer	

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

Shop table – calculated and set fields

The following describes the calculated and set fields in the Shop table. For an introduction to this table, and descriptions of its input fields, see "Shop table" on page 593.

Table 6.307: Calculated Shop (Mfg) fields

Field name	Description	Data type	Key
Allocations	The set of Allocation records associated with this shop floor location.	Set	
BOMs	The set of BillOfMaterial records associated with this shop floor location.	Set	
Kanbans	The set of ShopKanban records associated with this shop floor location.	Set	

ShopKanban table

The **ShopKanban** table is used to model parameters for a kanban material replenishment system within a factory. Each record in this table is uniquely identified by a part and shop floor location, and contains current and recommended details on bins at this ShopKanban. For example, it indicates the current number and size of bins for each ShopKanban, and also holds new calculated recommendations for these values.

This table supports the Kanban Manager workbook. It contains fields into which data should be imported if your company uses ShopKanbans, as well as fields that are populated by Kanban Manager calculations as discussed in . For more information, see "Kanban Manager calculations" on page 2239.

Table 6.308: ShopKanban (Mfg) fields

Field name	Description	Data type	Key
BinLocation	The exact storage location for this ShopKanban. **Default value**: blank string	String	
BinQty	The current approved quantity contained within each bin for this ShopKanban. **Default value**: 0	Integer	
Bins	Total number of bins currently approved for this ShopKanban. **Default value**: 0	Integer	
BinType	The type of bin used for this ShopKanban. **Default value**: blank string	String	
LastEvaluation Date	For a given scenario, the date on which Kanban Manager last evaluated, and possibly recommended changes to, the bin size and quantities for this ShopKanban. **Default value**: unassigned	Date	
MaximumBinQty	The maximum quantity of the part that can fit into the type of bin designated for this ShopKanban. This value establishes an upper limit for the bin quantity recommendation provided by Kanban Manager. If this value is < 0, then no upper limit is considered. **Default value**: 0	Integer	
Part	The name of a part which has been designated as a ShopKanban part. **Referenced table:** Part **Reference Syntax:** Part.Name, Part.Site	Reference	Yes

Table 6.308: ShopKanban (Mfg) fields

Field name	Description	Data type	Key
Replenishment Period	The frequency in which bin replenishment for this ShopKanban is desired. It is specified in work days, and decimals can be used to indicate partial work days. For example, a value of 2 indicates bins should be replenished every 2 days, and a value of .5 indicates bins should be replenished every half day. This value is used by Kanban Manager when calculating the recommended bin quantity. **Default value**: 0	Quantity	
Replenishment LeadTime	The number of work days it takes to replenish an empty bin of the part with a full one. This includes any transit times, and decimals can be used to indicated partial work days. This value is used by Kanban Manager when calculating the recommended number of bins. **Default value**: 0	Quantity	
SafetyFactor	A decimal factor used to increase the average daily demand value to account for normal variations in demand and delivery reliability. For example, specifying .25 indicates a 25% increase in average daily demand. **Default value**: 0	Quantity	
Shop	A shop floor location where consumption of the part occurs. **Referenced table:** Shop	Reference	Yes

ShopKanban table– calculated and set fields

The following describes the calculated and set fields in the ShopKanban table. For an introduction to this table, and descriptions of its input fields, see "ShopKanban table" on page 594. For information about the calculations used to populate these fields, see "Kanban Manager calculations" on page 2239.

Table 6.309: ShopKanban (Mfg) fields

Field name	Description	Data type	Key
AverageDaily Demand	The average daily demand calculated by Kanban Manager for this ShopKanban over the time horizon established by **Shop.KanbanHorizon**. This value includes any past due demands, and is used by Kanban Manager when recommending the bin quantity and number of bins. For information on how this field is calculated, see "Average daily demand" on page 2240. **Default value**: 0	Quantity	
Recommended BinQty	The recommended bin quantity for this ShopKanban as calculated by Kanban Manager. This value is calculated by multiplying AverageDailyDemand by the ReplenishmentPeriod, and is subject to both MaximumBinQty and PartSource.MultipleQty. For more information on how this field is calculated, see "Recommended bin quantity" on page 2241. **Default value**: 0	Integer	
Recommended Bins	The recommended number of bins for this ShopKanban as calculated by Kanban Manager. This value is calculated based on the current bin quantity. It does not consider any Kanban Manager recommendations for changes in bin quantity. For more information on how this field is calculated, see "Recommended number of bins" on page 2241. **Default value**: 0	Integer	

Site table

The **Site** table contains valid site codes. An entry should be provided for each site value in the enterprise data.

This table is referenced by many other tables. The Site field in these other tables is usually a key field, which keeps planning data from different sites separate. For example, there might be different constraints affecting production at different locations, or different quantities of a part on hand at different locations.

Types of sites

RapidResponse supports two types of sites: manufacturing/physical and inventory/logical.

Manufacturing/Physical

A manufacturing/physical site is an individual location within a company (or a company's subsidiary or a company's third-party manufacturing facilities) that segregates its supply and demand from other

locations, for example, a plant, a specific geographic location, and so on. Typically, manufacturing, assembly of products, or both occur at this type of site.

Data from manufacturing/physical sites is used to model supply and demand, and model the building of supply to support demand. The part data associated with manufacturing/physical sites includes a BOM structure.

Enterprise data sources are often used as a site identifiers.

Inventory/Logical

Inventory/logical sites are used to identify units, such as business units and inventory hubs. An inventory/logical site is often an inventory location, and it segregates its supply and demand from other locations. Examples include retail site, hub, VMI, and order consolidation.

These types of sites are primarily used to model a source of supply, demand, or both.

The part data associated with the inventory/logical site type does not involve transformation of one part into another (for example, processing, assembly, and so on) and should not include any BOM structures. RapidResponse analytics (including netting) must not be used in modeling exercises that explode BOM structures.

Table 6.310: Site (Core) fields

Field name	Description	Data type	Key
Address	The street address of the site. If this table is used to support maps, the address in this field determines the geocode that is used to mark the site's location on the map. To ensure that the correct geocode is used, follow the guidelines outlined in "Using address data to support maps" on page 1735. **Note:** This field was added in Version 2016.2, Service Update 9.	String	
ControlSet	The ControlSet record that data in this site is typically expected to use. **Referenced table:** ControlSet	Reference	
Currency	The default currency used for Money values at this site. **Referenced table**: Currency (Nullable)	Reference	
Description	Additional information about the site. **Default value**: blank string	String	

Table 6.310: Site (Core) fields

Field name	Description	Data type	Key
LicenseType	Indicates the type of site license. Valid values are: • **Logical**—This type of site is used to identify units, such as a specific business unit or a customer product line, within one physical site. In RapidResponse, this type of license is also referred to as Inventory/Logical. • **Physical**—This type of site is used to identify an individual location within a company that has separately defined enterprise data, such as a plant or a specific geographic location. In RapidResponse, this type of license is also referred to as Manufacturing/Physical. For more information, see "Types of sites" on page 596.	String (Enum)	
Region	The name of the region this site belongs to. For example, this might refer to a state or country. **Referenced table:** Region (Nullable)	Reference	
TimeZone	The name of the time zone associated with this site. **Referenced table:** TimeZone	Reference	
Type	The valid site types for reporting purposes only. Valid values are: • Distribution Center1 • Distribution Center2 • Distribution Center3 • Miscellaneous • ProductionFinishedGoods • ProductionSubAssembly • WarehouseFinishedGoods • WarehouseIncomingSupply **Default value**: ProductionFinishedGoods	String (Enum)	
Value	The key string used to reference a Site record (for example, from the Part table, or from the WorkCenter table if the **Capacity Planning - Constraints** application is enabled). **Default value**: blank string **Caution:** Avoid including leading or trailing spaces in Site names. In some cases, this can lead to the Site not being visible in the RapidResponse interface (for example, from a workbook's Site control).	String	

Site table – calculated and set fields

The following describes the calculated and set fields in the Site table. For an introduction to this table, and descriptions of its input fields, see "Site table" on page 596.

Table 6.311: Calculated Site (Core) fields

Field name	Description	Data type	Key
LastDataUpdate	The date and time of the last data update or import for this site. **Note:** This field is obsolete.	DateTime	
ABCCodes	The set of values in the ABCCode table associated with this site.	Set	
BonusSchedules	Set of BonusSchedule records associated with this site.	Set	
BuyerCodes	Set of BuyerCodes records associated with this site.	Set	
Constraints	Set of Constraint records associated with this site.	Set	
Customers	Set of Customer records associated with this site.	Set	
DeliveryRoutes	Set of DeliveryRoute records associated with this site.	Set	
DemandOrders	Set of DemandOrder records for this site.	Set	
Departments	Set of Department records at this site.	Set	
HistoricalDemand Orders	Set of HistoricalDemandOrder records associated with this site.	Set	
HistoricalShipments	Set of HistoricalShipment records associated with this site.	Set	
HistoricalSupply Orders	Set ofHistoricalSupplyOrder records associated with this site.	Set	
Parts	Set of Part records associated with this site.	Set	
PenaltySchedules	Set of PenaltySchedule records associated with this site.	Set	
PlannerCodes	Set of PlannerCode records associated with this site.	Set	
ProductionGroups	Set of ProductionGroup records associated with this site.	Set	
ProductionWheels	Set of ProductionWheel records associated with this site.	Set	
Projects	Set of Project records associated with this site.	Set	

Table 6.311: Calculated Site (Core) fields

Field name	Description	Data type	Key
Resources	Set of Resource records associated with this site.	Set	
Routings	Set of Routing records associated with this site.	Set	
SafetyLeadTime Profiles	Set of SafetyLeadTimeProfile records associated with this site.	Set	
Segments	Set of Segment records associated with this site.	Set	
Shops	Set of Shop records at this site.	Set	
SiteManagers	Set of SiteManager records associated with this site.	Set	
Sources	Set of Source records associated with this site.	Set	
SubstituteGroups	Set of SubstituteGroup records associated with this site.	Set	
Suppliers	Set of Supplier records associated with this site.	Set	
SupplyOrders	Set of SupplyOrder records for this site.	Set	
Warehouses	Set of Warehouse records at this site.	Set	
WorkCenter	Set of WorkCenter records associated with this site.	Set	

SiteManager table

This table is used to track information about management roles at different company sites.

Data in this table do not affect analytic functionality.

This table was added in Version 2016.2, Service Update 9 to support the Supply Chain Health dashboard, which is part of the S&OP Application. For more information about applications, see the *RapidResponse Applications Guide*.

Table 6.312: SiteManager (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this site manager's responsibilities.	String	
Site	The site that this manager is responsible for. **Referenced table:** Site	Reference	Yes
Type	The type of site manager. Because a site might have more than one manager—for example, a Plant manager and a Logistics manager—this key field makes it possible to include records for different types of managers at the same site. **Referenced table:** SiteManagerType	Reference	Yes
Value	Identifier for this management position.	String	

SiteManagerType table

This table is used to define different site manager types. For example, the roles of Plant manager and Logistics manager at a site might be filled by different people.

Data in this table do not affect analytic functionality.

This table was added in Version 2016.2, Service Update 9 to support the Supply Chain Health dashboard, which is part of the S&OP Application. For more information about applications, see the *RapidResponse Applications Guide*.

Table 6.313: SiteManagerType (Mfg) fields.

Field name	Description	Data type	Key
Description	Description of the site manager type.	String	
Value	Identifier for the site manager type.	String	Yes

SiteManagerType table – calculated and set fields

The following describes the set field in the SiteManagerType table. For an introduction to this table, and descriptions of its input fields, see "SiteManagerType table" on page 601.

Table 6.314: SiteManagerType (Mfg) fields

Field name	Description	Data type	Key
SiteManagerTypes	The SiteManager records associated with this SiteManagerType.	Set	

SiteMetric table

The **SiteMetric** table identifies the combination for each site and metric definition generated for a specific as of date that are used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.315: SiteMetric (Solutions) fields

Field name	Description	Data type	Key
AsOf	A reference to the date on which the site and metric definitions are generated.	Reference	Yes
Metric	A reference to the metric associated with this combination of site and metric definition data. **Referenced table:** Metric	Reference	Yes
Site	A reference to the part the associated with this combination of site and metric definition data. **Referenced table:** Site	Reference	Yes

SiteMetric table – calculated and set fields

The following describes the calculated and set fields in the SiteMetric table. For an introduction to this table, and descriptions of its input fields, see "SiteMetric table" on page 602.

Table 6.316: SiteMetric (Solutions) fields

Field name	Description	Data type	Key
SiteMetricValues	The set of site calculations that belong to this combination of site and metric definition data. **Referenced table:** SiteMetricValues	Set	

SiteMetricValues table

The **SiteMetricValues** table stores the site calculations for metrics used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 19.

Table 6.317: SiteMetricValues (Solutions) fields

Field name	Description	Data type	Key
Date	The date associated with this calculation.	Date	Yes
SiteMetric	A reference to the site and metric definition combination associated with this calculation. **Referenced table:** SiteMetric	Reference	Yes
Quantity	The quantity for the metric. The value in this field is used if Metric.ValueType= 'Quantity'. For more information, see "Metric table" on page 399.	Quantity	
Value	The money value for the metric. The value in this field is used if Metric.ValueType= 'Money'. For more information, see "Metric table" on page 399.	Money	

Source table

The **Source** table stores information about deliveries, as well as information pertaining to contact people at suppliers. This table is referenced from the PartSource table.

Table 6.318: Source (Mfg) fields

Field name	Description	Data type	Key
Carrier	The name of the transportation carrier for this source. **Referenced table**: Carrier **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	
ContactName	Full name for this source (often the contact person).	String	
DestinationSite	The site expected to consume from this source. **Referenced table:** Site	Reference	Yes
Email	The email address for the contact.	String	
EstimatedCost	An estimate of the shipping cost associated with using this source for an order. It can be used to express relative costs between sources. **Note:** This field was added in Version 2016.2, Service Update 9.	Money	

Table 6.318: Source (Mfg) fields

Field name	Description	Data type	Key
Id	Identifier for this source.	String	Yes
LeadTimeUnit	Calendar to use when calculating start date. **Referenced table:** Calendar	Reference	
Phone	The phone number for the contact.	String	
PreShipLT	The number of **Source.LeadTimeUnits** to allow between the supply-side stock date and the ship date for supplies using this source. This field gives time for the source to collect the inventory and prepare it for shipment. Values less than zero are treated as zero. **NOTE**: If this value is expected to change over time, the TimePhasedSourceLeadTime table can be used instead.	Quantity	
ShipCalendar	Calendar to use when determining Ship Date. **Referenced table:** Calendar	Reference	
Supplier	Identifies the supplier this record represents. **Referenced table:** Supplier **Reference Syntax:** Supplier.Id	Reference	Yes
TransitCalendar	A reference to the calendar in which the **TransitTime** value should be expressed. For example, if the trucks used to transport goods from the sourcing site to the destination site are not permitted on roads during weekends, this might be set to a standard Workday calendar. If this reference is left Null, then an Everyday calendar is assumed. **Referenced table:** Calendar (Nullable) **Note:** This field was added in Version 2016.2	Reference	
TransitTime	The amount of time required to ship supply from the sourcing site to the destination site. This field represents time between the **ShipDate** and **DockDate**, and is typically only applicable to transfer sources. This should be expressed in **TransitCalendar** intervals (or the Everyday calendar if the **TransitCalendar** reference is Null). **NOTE**: If this value is expected to change over time, the TimePhasedSourceLeadTime table can be used instead.	Quantity	

Table 6.318: Source (Mfg) fields

Field name	Description	Data type	Key
Transportation Mode	The transportation mode, including carrier and delivery route, for this source. **Referenced table**: TransportationMode (Nullable) **Note:** This field was added in Version 2016.2, Service Update 5.	Reference	

Source table – calculated and set fields

The following describes the calculated and set fields in the Source table. For an introduction to this table, and descriptions of its input fields, see "Source table" on page 603.

Table 6.319: Calculated Source (Mfg) fields

Field name	Description	Data type	Key
Collaborative Forecasts	Set of CollaborativeForecast records that reference this Source record. **Note:** This field is only applicable to RapidResponse legacy applications.	Set	
HistoricalSupply Actuals	Set of HistoricalSupplyActual records that reference this Source record.	Set	
PartSources	Set of PartSource records that reference this Source record.	Set	
TimePhasedLeadTimes	Set of TimePhasedSourceLeadTime records that reference this Source record.	Set	

SourceConstraint table

This table supports the Constraint Manager application, and defines constraint factors for part sources. Each record in this table defines the fixed and variable amounts of a constraint needed to produce supply from a given part source.

If constraint factors are not expected to change over time, then only one SourceConstraint record is required for a given constraint and part source combination. Otherwise, if constraint factors on a part source vary over time, then multiple time-phased SourceConstraint records can be defined.

Therefore, the combination of the Constraint, PartSource and EffectiveInDate fields must be unique for each record in this table. Otherwise, data integrity is violated and the record is not imported into this table.

Table 6.320: SourceConstraint (Mfg) fields

Field name	Description	Data Type	Key
CampaignFixed ConstraintFactor	If the part source is enabled for campaign planning (has **OrderPolicy.MaximumUsage** set to "CampaignPlanning"), this represents the major cleanup associated with the constraint (typically, equipment). It represents a fixed amount of constraint that gets consumed before each campaign can begin, and is time used to clean up or prepare the constraint to process supply of a particular part. Note that if the part source is not enabled for campaign planning, then the factor defined in this field is ignored. **Note:** This field supports the optional Campaign Planning capability.	Quantity	
Constraint	The constraint that this record references. Usually constraint will not be a constraint with Type=**Unconstrained** (RapidResponse calculations ignore such supplemental source constraints). A particular constraint record can only be used once in the set of SourceConstraints for the same part source. **Referenced table:** Constraint **Reference Syntax:** Constraint.Site, Constraint.Name	Reference	Yes
ConstraintFactor	The amount of the constraint required for each unit of supply from the referenced part source (in part source units). Thus, the variable amount of constraint required for a given supply is the product of the supply quantity and the value in this field.	Quantity	

Table 6.320: SourceConstraint (Mfg) fields

Field name	Description	Data Type	Key
EffectiveInDate	The date on which the fixed and variable constraint factors defined on this record become effective for the part source (inclusive). The factors are assumed to be effective until the next EffectiveInDate on a record for the same PartSource and Constraint combination. This field is intended to support situations where constraint factors vary over time. For example, an operating process might improve over time (requiring a factor decrease) or a machine might degrade over time (requiring a factor decrease). Similarly, if the constraint factors should be ignored over a period of time, then they can be set to zero (0). If constraint factors are never expected to change, then this field can be left at the default value. **Default value**: Past **Note:** This field was added in Version 2016.2, Service Update 3. **Note:** The earliest record whose effectivity overlaps with that of a given part source has its EffectiveInDate artificially extended to "Past" (this prevents an issue with setting the constraint factor to 0 but without a SourceConstraint to reference).	Date	Yes
FixedConstraint Factor	The fixed amount of the constraint required for each supply from the referenced part source (in part source units). Note that if the part source is enabled for campaign planning (has **OrderPolicy.MaximumUsage** set to "CampaignPlanning"), then this factor represents the minor clean-up between batches. It is a fixed amount of constraint consumed before each batch in a campaign (other than the first batch per campaign which has no minor clean up associated with it.	Quantity	
PartSource	The part source constrained by this constraint. **Referenced table**: PartSource **Reference Syntax:** PartSource.Part.Site, PartSource.Part.Name, PartSource.Source.Id, PartSource.Source.Supplier.Id, PartSource.BaseKey	Reference	Yes

SourceKanban table

The SourceKanban table is used to model parameters for a kanban material replenishment system from a given source. Each record in this table is associated with exactly one part source, and contains current and recommended details on bins at this SourceKanban. For example, it indicates the current number and size of bins for each SourceKanban, and also holds new calculated recommendations for these values.

This table supports the Kanban Manager workbook. It contains fields into which data should be imported if your company uses SourceKanbans, as well as fields that are populated by Kanban Manager calculations which are described in "SourceKanban table — calculated and set fields" on page 609. For information about the calculations used to populate these fields, see "Kanban Manager calculations" on page 2239.

Table 6.321: SourceKanban (Mfg) fields

Field name	Description	Data type	Key
BinLocation	The exact storage location for the SourceKanban. **Default value**: assigned	String	
BinQty	The current approved quantity contained within each bin for this SourceKanban. **Default value**: 0	Integer	
Bins	Total number of bin currently approved for this SourceKanban. **Default value**: 0	Integer	
BinType	The type of bin used for this SourceKanban. **Default value**: blank string	String	
KanbanHorizon	The number of days out from today that Kanban Manager considers when calculating the average daily demand for this SourceKanban. **Default value**: 0	Integer	
LastEvaluationDate	For a given scenario, the date on which Kanban Manager last evaluated, and possibly recommended changes to, the bin size quantities for this SourceKanban. **Default value**: unassigned	Date	
MaximumBinQty	The maximum quantity of the part that can fit into the type of bin designated for this SourceKanban. This value establishes an upper limit for the bin quantity recommendation provided by Kanban Manager. If this value is < 0, then no upper limit is considered. **Default value**: 0	Integer	
PartSource	The source of supply for the part designated as a SourceKanban part. **Referenced table**: PartSource **Reference Syntax**: PartSource.Part.Site, PartSource.Part.Name, PartSource.Source.Id, PartSource.Source.Supplier.Id, PartSource.BaseKey	Reference	Yes

Table 6.321: SourceKanban (Mfg) fields

Field name	Description	Data type	Key
Replenishment Period	The frequency in which bin responsibility for this SourceKanban is desired. It is specified in work days, and decimals can be used to indicate partial work days. For example, a value of 2 indicates bins should be replenished every 2 days, and a value of .5 indicates bins should be replenished every half day. This value is used by Kanban Manager when calculating the recommended bin quantity. **Default value**: 0	Quantity	
SafetyFactor	A decimal factor used to increase the average daily demand value to account for normal variations in demand and delivery reliability. For example, specifying .25 indicates a 25% increase in average daily demand. **Default value**: 0	Quantity	

SourceKanban table – calculated and set fields

The following describes the calculated and set fields in the SourceKanban table. For an introduction to this table, and descriptions of its input fields, see "SourceKanban table" on page 607. For information on the calculations used to populate this table, see "Kanban Manager calculations" on page 2239.

Table 6.322: SourceKanban (Mfg) fields

Field name	Description	Data type	Key
AverageDaily Demand	The average daily demand calculated by Kanban Manager for this SourceKanban over the time horizon established by KanbanHorizon. This value includes any past due demands, and is used by Kanban Manager when recommending the bin quantity and number of bins. For information on how this field is calculated, see "Average daily demand" on page 2240. **Default value**: 0	Quantity	
RecommendedBin Qty	The recommended bin quantity for this SourceKanban as calculated by Kanban Manager. This value is calculated by multiplying AverageDailyDemand by the ReplenishmentPeriod, and is subject to both the MaximumBinQty and PartSource.MultipleQty. For more information on how this field is calculated, see "Recommended bin quantity" on page 2241. **Default value**: 0	Integer	

Table 6.322: SourceKanban (Mfg) fields

Field name	Description	Data type	Key
Recommended Bins	The recommended number of bins for this SourceKanban as calculated by Kanban Manager. This value is calculated based on the current bin quantity. It does not consider any Kanban Manager recommendations for changes in bin quantity. However, this information is available in the Kanban Manager workbook. For more information on how this field is calculated, see "Recommended number of bins" on page 2241. **Default value**: 0	Integer	

StorageType table

The **Storage** table is used to define storage types for storage locations in the Distribution Requirements Planning application. For example, a storage type value could be Freezer or Cage. The settings in this table are referenced by the **PartSolution** table.

This table was added in the Version 2016.2, Service Update 5, to support the Distribution Requirements Planning application.

Table 6.323: StorageType (Solutions) fields

Field name	Description	Data type	Key
Value	A string identifier for this storage type. This field is referenced by **PartSolution.StorageType**.	String	Yes
Description	A description of this storage type. This field can used to enter a description about the storage type.	String	

SubProcess table

This table holds records that define hierarchical parent-child relationships between process instance activities. Parent activities are usually sub-processes and child activities are activities.

Table 6.324: SubProcess (ProcOrch) fields

Field name	Description	Data type	Key
Child	The child activity in the sub-process and child relationship. **Referenced table**: Activity (ProcOrch)	Reference	
Parent	The parent activity in the sub-process and child relationship. **Referenced table:** Activity (ProcOrch)	Reference	

SubProcess table—calculated and set fields

The following describes the calculated and set fields in the SubProcess table. For an introduction to this table, and descriptions of its input fields, see "SubProcess table" on page 610.

Table 6.325: SubProcess (ProcOrch) fields

Field name	Description	Data type	Key
IsRecursive	Indicates if a recursive relationship exists between sub-processes.	Boolean	

SubstituteGroup table

The **SubstituteGroup** table is used to define groups of substitutable components within an assembly. For example, a substitute group might define a group of components that can be substituted for one another within an assembly, or a group of components that can collectively be substituted for another group of components within an assembly.

All records in the **BillOfMaterial** table that define a group of substitutable components within a given assembly must reference the same **SubstituteGroup** record. Additionally, to ensure expected results from bom-level substitution, it is recommended that multiple assemblies not reuse the same group(s) to define their substitutable components. Thus there should typically be at least as many **SubstituteGroup** records as there are unique groups of substitutable components across all assemblies.

The **SubstituteGroup** table can also be used to define substitutable allocations under scheduled receipts. All records in the **Allocation** table that represent a group of substitutable allocations under a given scheduled receipt should reference the same SubstituteGroup record.

Note: For more information about using substitute groups, see "BOM level part substitution" on page 1565.

Table 6.326: SubstituteGroup (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this group of substitutable components.	String	
Site	The site associated with this group of substitutable components. Together with the Value field, this uniquely identifies each record in the table. **Referenced table**: Site	Reference	

Table 6.326: SubstituteGroup (Mfg) fields

Field name	Description	Data type	Key
Type	A value that defines the processing rules associated with this group of substitutable components. For example, can multiple components within the group be used within a single order for the group's assembly. **Referenced table:** SubstituteGroupType	Reference	
Value	An identifier for this group of substitute components. This value is referenced by each BillOfMaterial record that defines a group of substitutable components within a given assembly. It also referenced by each Allocation record that defines a group of substitutable components under a given scheduled receipt.	String	Yes

SubstituteGroup table – calculated and set fields

The following describes the calculated and set fields in the SubstituteGroup table. For an introduction to this table, and descriptions of its input fields, see "SubstituteGroup table" on page 611.

Table 6.327: Calculated SubstituteGroup (Mfg) fields

Field name	Description	Data type	Key
Allocations	Set of Allocation records which reference this SubstituteGroup.	Set	
BillOfMaterials	Set of BillOfMaterial records which reference this SubstituteGroup.	Set	

Supplier table

The **Supplier** table is a repository for supplier information. A Supplier may be a vendor for a purchase order, a work center for work orders, or a plant identifier for inter-plant orders.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.328: Supplier (Mfg) fields

Field name	Description	Data type	Key
Address	The street address of this supplier. If this table is used to support maps, the address in this field determines the geocode that is used to mark the supplier's location on the map. To ensure that the correct geocode is used, follow the guidelines outlined in "Using address data to support maps" on page 1735. **Note:** This field was added in Version 2016.2, Service Update 9.	String	
ContactName	Name of this supplier's contact person. **Note:** This field was added in Version 2016.2, Service Update 9.	String	
DefaultShipment	The name of the shipment to use as the default for this supplier. The field is a string so that data can be loaded. **Default value**: blank string	String	
Email	The email address of the contact person listed in the **ContactName** field. **Note:** This field was added in Version 2016.2, Service Update 9.	String	
Id	A value that identifies the supplier. **Default value**: unknown	String	Yes
Name	The name of the supplier. **Default value**: blank string	String	
Phone	The phone number of the contact person listed in the **ContactName** field. **Note:** This field was added in Version 2016.2, Service Update 9.	String	
Region	The name of the region associated with this supplier. **Referenced table:** Region (nullable)	Reference	
Site	The site associated with this supplier. This field is optional. **Referenced table:** Site	Reference	Yes
SupplierGroup	The group this supplier is in. **Referenced table**: SupplierGroup (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	

For more information about making the **Site** field optional, see “Modifying table and field properties” in the *RapidResponse Data Integration Guide*.

Supplier table – calculated and set fields

The following describes the calculated and set fields in the Supplier table. For an introduction to this table, and descriptions of its input fields, see "Supplier table" on page 612.

Table 6.329: Calculated Supplier (Mfg) fields

Field name	Description	Data type	Key
Contacts	Set of Source records referencing this supplier.	Set	
Orders	Set of SupplyOrder header records for existing orders from this supplier.	Set	
PartSuppliers	Set of PartSupplier records referencing this supplier.	Set	
Shipments	Set of Shipment records for this supplier.	Set	

SupplierGroup table

The SupplierGroup table defines groups that different suppliers can be placed under. If your company’s host system uses more than one name to refer to the same supplier, they can be placed in the same group, and the group name used to refer to the supplier.

Table 6.330: SupplierGroup (Mfg) fields

Field name	Description	Data type	Key
Id	The supplier group name	String	Yes
Description	A description of this group. This can be the supplier’s company name or a general classification for several different companies.	String	

SupplierGroup table – calculated and set fields

The following describes the calculated and set fields in the SupplierGroup table. For an introduction to this table, and descriptions of its input fields, see "SupplierGroup table" on page 614.

Table 6.331: Calculated SupplierGroup (Mfg) fields

Field name	Description	Data type	Key
Suppliers	The Supplier records in this group.	Set	

SupplierMetric table

The **SupplierMetric** table identifies the combination for each supplier and metric definition generated for a specific as of date that are used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 22.

Table 6.332: SupplierMetric (Solutions) fields

Field name	Description	Data type	Key
AsOf	A reference to the date on which the supplier and metric definitions are generated. **Referenced table:** MetricAsOf	Reference	Yes
Metric	A reference to the metric associated with this combination of supplier and metric definition data. **Referenced table:** Metric	Reference	Yes
Supplier	A reference to the part the associated with this combination of supplier and metric definition data. **Referenced table:** Supplier	Reference	Yes

SupplierMetric table – calculated and set fields

The following describes the calculated and set fields in the SupplierMetric table. For an introduction to this table, and descriptions of its input fields, see "SupplierMetric table" on page 615.

Table 6.333: SupplierMetric (Solutions) fields

Field name	Description	Data type	Key
SupplierMetricValues	The set of site calculations that belong to this combination of supplier and metric definition data. **Referenced table:** SupplierMetricValues	Set	

SupplierMetricValues table

The **SupplierMetricValues** table stores the supplier calculations for metrics used in live lens and comparison scorecards.

This table was added in Version 2016.2, Service Update 22.

Table 6.334: SupplierMetricValues (Solutions) fields

Field name	Description	Data type	Key
Date	The date associated with this calculation.	Date	Yes
SupplierMetric	A reference to the supplier and metric definition combination associated with this calculation. **Referenced table:** SiteMetric	Reference	Yes
Quantity	The quantity for the metric. The value in this field is used if Metric.ValueType= 'Quantity'. For more information, see "Metric table" on page 399.	Quantity	
Value	The money value for the metric. The value in this field is used if Metric.ValueType= 'Money'. For more information, see "Metric table" on page 399.	Money	

SupplyAllocation table

The SupplyAllocation table stores manual allocations of supply to demand entered or imported into RapidResponse. For example, a demand manager may create these manual supply allocations to reserve limited supply amongst multiple requesting demand sites. To model these manual allocations, corresponding orders are created in the PlannedOrder table and identified as “Manual” in the Origin field. As necessary, RapidResponse will also automatically generate planned orders at the PTFDate to satisfy any supply shortages introduced by the manual allocations.

Table 6.335: SupplyAllocation (Mfg) fields

Field name	Description	Data type	Key
DestinationPart	The part whose demand this supply allocation was created to satisfy. **Referenced table**: Part	Reference	
Id	An identifier used to uniquely identify this supply allocation record. **Default value**: SupplyAllocation	String	Yes

Table 6.335: SupplyAllocation (Mfg) fields

Field name	Description	Data type	Key
Model	The model that applies to this supply allocation. It is set to None if either of the following conditions apply: • Model-Unit Effectivity is not enabled • MUEPoolNettingType.ModelRule for the part is set to Ignore. **Referenced table**: Model	Reference	
OrderPriority	The priority associated with this supply allocation. **Referenced table**: OrderPriority	Reference	
Pool	The pool associated with this supply allocation. **Referenced table**: Pool	Reference	
Quantity	The number of units that this supply allocation is for. This value is expressed in inventory units of measure at the supplying site. However, the amount received at the destination site is adjusted for yield on the transfer part source (for example, to reflect any loss during transportation).	Quantity	
StartUnit	The start unit number that applies to this supply allocation. It is set to None if either of the following conditions apply: • Model-Unit Effectivity is not enabled. • MUEPoolNettingType.UnitRule is set to Ignore.	Integer	
SupplyDate	The date the supply is removed from the source part's site.	Date	
PartSource	The part source used for this supply allocation. **Referenced table**: PartSource **NOTE:** Manual supply allocations are not compatible with constrained part sources and can result in unexpected analytic results if used together. However, constraints at the sourcing site are respected as appropriate.	Reference	
Type	A value that determines the processing rules associated with this supply allocation. For example, whether the supply allocation is processed by RapidResponse analytics. **Referenced table**: SupplyAllocationType	Reference	

SupplyAllocation table – calculated and set fields

The following describes the calculated and set fields in the SupplyAllocation table. For an introduction to this table, and descriptions of its input fields, see "SupplyAllocation table" on page 616.

Table 6.336: SupplyAllocation (Mfg) fields

Field name	Description	Data type	Key
DestinationDate	The date this supply allocation is due at the destination site, based on the part source's lead time.	Date	
EffQuantity	The effective quantity, in supplier units of measure, of this supply allocation. If the **Type.ProcessingRule** is set to 'Ignore', this value is zero (0).	Quantity	
PartSupplier	A reference to the entry in the PartSupplier table, if any, that has the same part and supplier as this ScheduledReceipt record. **Referenced table**: PartSupplier	Reference	
ToleranceProfile Zone	A reference to the entry in the ToleranceProfileZone table that applies to this SupplyAllocation record. It is used in defining the acceptable levels of change when comparing supply allocations against baseline historical supply series. The ToleranceProfile for this record is the value referenced from the PartSupplier field, and the ToleranceProfileZone is set to the entry with the lowest ZoneEndInterval value where `Part.PlanningCalendars.RunDate.FirstDate + ZoneEndInterval ToleranceProfile.ToleranceCalendar > SupplyAllocation.DestinationDate`. **Referenced table**: ToleranceProfileZone	Reference	
Notes	A set of descriptive notes added to this supply allocation record. This field can be used to enter notes about an supply allocation record. Each time a note value is added to this field through a worksheet, a new record is created in the SupplyAllocation_Notes table with a date and timestamp, user ID, note text, and a reference to the SupplyAllocation record against which it was created.	Note	

SupplyOrder table

The **SupplyOrder** table is a repository for SupplyOrder information. Often supply orders are written up for multiple parts on a single order form. The header information is the common information about the whole order. Individual line items are entered in the **ScheduledReceipt** table. These line items reference the **SupplyOrder**. This information comes from work order (shop order), and purchase order files in most MRP systems.

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
CampaignIndex	If the supply order represents a production campaign, this field identifies that campaign and the **Line** item on each scheduled receipt belonging to the order can be used to indicate the batch number. Otherwise, this field should be left blank. All scheduled receipts within a campaign-enabled supply order are assumed to belong to the same campaign and should be for the same quantity (batch size). Each of the receipts in the order should also reference the same **PartSource** record which should have an **OrderPolicy.MaximumUsage** value of "CampaignPlanning". This enables campaign functionality such as contiguous constraint consumption and the use of major and minor cleanup times when running the order through a constraint. Note however that because scheduled receipts are provided as input, the batch size, campaign minimum, and campaign maximum defined on the part source, which are enforced on generated planned orders, can potentially be overridden on scheduled receipts (either unintentionally, or as part of a simulation exercise to test different batch settings). **Note:** This field supports the optional Campaign Planning. For more information, see "Campaign planning" on page 1707.	String	
Currency	For an order that is undergoing a supplier collaboration process, the currency the order's money values use. This value is not a reference to the Currency table, and is used for information only. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the CURCY field in the E1EDK01 segment of the IDOC message.	String	

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
DateQualifier	For an order received from the Real-time Integration Service, identifies how the date and time fields are used in the IDOC message that generated the order. This field is taken from the IDOC message and can contain the following values: • **003**—The date and time fields represent the order's requested delivery date. • **005**—The date and time fields represent a value-added service date. • **006**—The date and time fields represent the order's cancellation date. • **007**—The date and time fields represent the order's price determination date. • **014**—The date and time fields represent the validity start date for an agreement. • **022**—The date and time fields represent a date for services. • **023**—The date and time fields represent the valid start date for picking up a delivery order. • **024**—The date and time fields represent the valid end date for picking up a delivery order. This field is in the **SupCollab** namespace. This field maps to the IDDAT field of the E1EDK03 segment of the IDOC message.	String	
DocQualifier	For an order received from the Real-time Integration Service, identifies the type of order the IDOC message generates. This field is taken from the IDOC message and can contain the following values: • **001**—A customer purchase order. • **004**—A vendor quotation. • **007**—A collective. • **050**—A delivery order. • **066**—An external order. This field is in the **SupCollab** namespace. This field maps to the QUALF field of the E1EDK02 segment of the IDOC message.	String	

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
Flag	For an order that is undergoing a supplier collaboration process, indicates whether an order acknowledgement is required. This field is taken form the IDOC message and can contain the following values: • **Y**—An acknowledgment is required. • **N**—An acknowledgment is not required. This field is in the **SupCollab** namespace. This field maps to the KZABS field of the E1EDK01 segment of the IDOC message.	Boolean	
Id	A value that identifies an order number for a ScheduledReceipt record. **Default value**: blank string	String	Yes
IDOCDateTime	For an order received from the Real-time Integration Service, the date and time the order's IDOC message was generated. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. The Date component of this field maps to the DATUM field of the E1EDK02 segment of the IDOC message. The Time component of this field maps to the UZEIT field of the E1EDK02 segment of the IDOC message.	DateTime	
Organization	For an order received from the Real-time Integration Service, identifies the organization that generated the IDOC message for the order. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the ORGID field of the E1EDK14 segment of the IDOC message.	String	

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
OrgQualifier	For an order received from the Real-time Integration Service, identifies the division or organization that generates the order. This field is taken from the IDOC message and can contain the following values: • **006**—The order was generated by a division of your company. • **007**—The order was generated by a member of your distribution channel. • **008**—The order was generated by a sales organization. • **009**—The order was generated by a purchasing group. • **010**—The order was generated by a sales group. • **011**—The order was generated by a company. • **012**—The order was generated as a specific sales order type. • **013**—The order was generated as a specific purchase order type. • **014**—The order was generated by a purchasing organization • **016**—The order was generated by a sales office. This field is in the **SupCollab** namespace. This field maps to the QUALF field of the E1EDK14 segment of the IDOC message.	String	
PartnerFunc	For an order received from the Real-time Integration Service, the function provided by the trading partner. This field is taken from the IDOC message and can contain the following values: • **AG**—The partner is the organization the order is sold to. • **WE**—The partner is the organization the order is shipped to. • **RE**—The partner is the organization the order is billed to. • **RG**—The partner is paying for the order. This field is in the **SupCollab** namespace. This field maps to the PARVW field of the E1EDKA1 segment of the IDOC message.	String	
PaymentTerms	The term of payment for an order received from the Real-time Integration Service. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the QUALF field of the E1EDK18 segment of the IDOC message.	String	

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
Pool	The pool to which this order belongs. This field supports inventory pooling and is ignored unless **Part.MUEPoolNettingType.PoolRule** is set to 'Net'or 'NetOptional'. For all other **PoolRule** settings, or for implementations not using the Pool capabilities, all supplies belong to the part's common pool (as defined by the **Part.PlanningCalendars.CommonPool** reference). If any line items under the order have their **InputPool** reference populated, then that referenced pool is used instead. For example, this might be done when pools do not easily correspond to items associated with a supply order or when there is a need to inject pool information at the component level based on an assembly's scheduled supply. **Referenced table:** Pool **Reference Syntax:** Pool.Value	Reference	
PurchaseGroup	The name of a person or group of people operating within a segment of the purchasing organization. This field maps to the ORGID field of the E1EDK14 segment of the IDOC message. This field is in the **Solutions** namespace.	String	
Purchase Organization	The name of a company's purchasing organization. This field maps to the ORGID field of the E1EDK14 segment of the IDOC message. This field is in the **Solutions** namespace.	String	
Site	The site this supply order relates to. **Referenced table**: Site	Reference	Yes
Supplier	The supplier for the order. This field references the Supplier table. For information about this table, see "Supplier table" on page 612. **Referenced table:** Supplier **Reference Syntax:** Supplier.Id	Reference	
TextId	Identifies a note about an order that has been received from an enterprise data source through the Real-time Integration Service. This field is taken from the IDOC message. This field is in the **SupCollab** namespace. This field maps to the TDID field of the E1EDKT1 segment of the IDOC message.	String	

Table 6.337: SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
TextLine	For an order undergoing a supplier collaboration process, the notes provided by the buyer and supplier about the order. This field is taken from the IDOC message. Typically, these notes describe the quantity or delivery date changes made to the order, and are the primary channel of communication between the buyer and supplier. This field is in the **SupCollab** namespace. This field maps to the TDLINE field of the E1EDKT1.E1EDKT2 segment of the IDOC message.	Note	
Type	A value that determines the processing policies of ScheduledReceipts. This field references the SupplyType table. For information about this table, see "SupplyType table" on page 919. **Referenced table:** SupplyType	Reference	Yes

SupplyOrder table – calculated and set fields

The following describes the calculated and set fields in the SupplyOrder table. For an introduction to this table, and descriptions of its input fields, see "SupplyOrder table" on page 618.

Table 6.338: Calculated SupplyOrder (Mfg) fields

Field name	Description	Data type	Key
Lines	Set of line items (ScheduledReceipt records) for this order.	Set	

Task table

The **Task** table defines all individual work items or tasks that together form the basis of a project. Besides a reference to the applicable project, each record in this table contains task-level input details such as task name, the rule used for calculating duration, and any constraints on task start or finish dates. As necessary, a given task can also be assigned dependencies on other tasks, material availabilities, and resources that perform work towards task completion. Additionally, this table reports calculated project-level details such as start and finish dates, task duration, and any material, resource, or penalty costs applied to the project.

This table's records can be edited using the **Project Plan** workbook included with RapidResponse. When using this workbook, or any Task-based worksheet that has been automatically configured for project management, certain calculated fields become editable. For example, the calculated start date (**CalcStartDate**) and finish date (**CalcFinishDate**) can be edited, as can the calculated list of a task's predecessors (**PredecessorList**). For more information about worksheets automatically configured for project management, see the *RapidResponse Resource Authoring Guide*.

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
ActualCost	Used to specify any actual costs incurred while completing this task that aren't reflected in the defined resource or material costs. Useful for comparison against the **Cost** field to see how actual miscellaneous costs track against the planned miscellaneous costs.	Money	
ActualDuration	The actual number of days of work completed since the task was started. Used when calculating the **CalcDuration** and **CalcPercentComplete** fields. If populating this field, a value should typically be provided for the **RemainingDuration** field as well.	Quantity	
ActualFinishDate	The date work actually completed on the task (if applicable). For automatically scheduled tasks, if this field is populated then the date is also reported in **CalcFinishDate**. Also used in calculating the task state (for example, an undefined value here indicates the task has not yet finished), as well as by certain task calculations in projects where **ProjectType.PlanningRule** is set to "DateDriven".	Date	
ActualStartDate	The date work actually began on the task (if applicable). For automatically scheduled tasks, if this field is populated then the date is also reported in **CalcStartDate**. Also used in calculating the task state (for example, an undefined value here indicates the task has not yet started), as well as by certain task calculations in projects where **ProjectType.PlanningRule** is set to "DateDriven".	Date	
ActualWork	The actual amount of work that has already been completed on this task. Used when calculating the **CalcActualWork** and **CalcPercentComplete** fields.	Quantity	
ApplyRate Adjustment	Indicates if the resource rate adjustment specified in the **RateAdjustment** field will be applied to resources assigned to this task. Valid values are: • **N**—adjustment can be applied. • **Y**—adjustment is ignored.	Boolean	
BonusDate	The date on or before which bonuses can potentially be applied to this task. Bonuses are calculated based on the referenced **BonusSchedule** and **TaskType** values, and can be applied to projects that finish on or before this date (bonuses can accumulate between **CalcFinishDate** and **BonusDate**).	Date	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
BonusSchedule	The name of the bonuses schedule used for calculating project bonuses. Together with the rules defined on the project type, this determines how bonuses are calculated at the project level. **Referenced table:** BonusSchedule (Nullable)	Reference	
ConstraintDate	A date-based constraint to apply to scheduling of the task. The interpretation of this date is determined by the **ConstraintType** setting (all types other than "None", "AsLateAsPossible", and "AsSoonAsPossible" rely on this value). For example, this might define a date the task must finish on or a date the task cannot start before. Note that applying constraints to tasks limits the degree to which the project can be scheduled in RapidResponse, but might be useful when there are external factors influencing the task start or finish date.	Date	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
ConstraintType	Contains rules used to constrain task start or finish dates. Typically, these rules are applied to the date specified in the **ConstraintDate** field to either fix the task start or finish date, or provide boundaries for a possible start or finish date. Note that certain constraint types always honor the constraint date and might lead to conflicts with other scheduling parameters such as predecessor tasks or material order availability. The details of such conflicts are then reported in the **Conflict** table. Alternatively, other constraint types can be chosen that enforce the constraint date only so long as it does not lead to a conflict with other scheduling parameters. Valid values are: • **AsLateAsPossible**—task is scheduled to finish as late as it can (for tasks in project's with a ScheduleRule of "FromStart", this reverses the normal task scheduling logic). • **AsSoonAsPossible**—task is scheduled to start as soon as it can (for tasks in project's with a ScheduleRule of "FromStart", this reverses the normal task scheduling logic). • **FinishOn**—task is scheduled to finish on the constraint date unless this would conflict with another scheduling parameter. • **FinishOnOrAfter**—task is scheduled to finish on or after the constraint date unless this would conflict with another scheduling parameter. • **FinishOnOrBefore**—task is scheduled to finish on or after the constraint date unless this would conflict with another scheduling parameter. • **MustFinishOn**—task is always scheduled to finish on the constraint date. • **MustFinishOnOrAfter**—task is always scheduled to finish on or after the constraint date. • **MustFinishOnOrBefore**—task is always scheduled to finish on or before the constraint date. • **MustStartOn**—task is always scheduled to start on the constraint date. • **MustStartOnOrAfter**—task is always scheduled to start on or after the constraint date. • **MustStartOnOrBefore**—task is always scheduled to start on or before the constraint date specified. • **None**—task is scheduled without any constraint applied. If the task belongs to a project that has a **ProjectType.ScheduledRule** of "FromStart", then it is scheduled as early as possible. Otherwise, if the task belongs to a project that has a	String (Enum)	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
	ProjectType.ScheduledRule of "FromFinish", then it is scheduled as late as possible. • **StartOn**—task is scheduled to start on the constraint date unless this would conflict with another scheduling parameter. • **StartOnOrAfter**—task is scheduled to start on or after the constraint date unless this would conflict with another scheduling parameter. • **StartOnOrBefore**—task is scheduled to start on or before the constraint date unless this would conflict with another scheduling parameter. **Default value**: None **Note:** For more information about this field's usage, see "Task constraints" on page 2172 and "Constraint conflicts" on page 2174.		
Cost	Used to provide any planned costs associated with completing this task that aren't reflected in the defined resource or material costs. Subsequently used when calculating **Project.TaskCost**.	Money	
Description	Descriptive details about this task.	String	
Duration	An input value for the task duration used in determining the actual or calculated duration as reported in the **CalcDuration** field. The usage of this field is dependent on the setting in the **DurationType** field.	Quantity	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
DurationType	Contains rules used to set how the task duration is calculated and reported in the **CalcDuration** field. Valid values are: • **FixedDuration**—calculated task duration is typically fixed and set to the input value provided in the **Duration** field. Note that in run date-driven projects, if actual start date and either actual or expected finish dates are provided, the duration is calculated based on those dates. • **FixedUnits**—calculated task duration is determined by the resources assigned to the task (in units). The input **Duration** value is divided by the cumulative resource units assigned to the task through **ResourceAssignment.Units**. Otherwise, if no resources are assigned to the task, then the value provided in the **Duration** field is used instead. • **FixedWorkAndRate**—calculated task duration is determined by the provided work and rate values. For tasks without resources assigned, this is calculated by dividing the value in the **Work** field by the value in **Rate** field. For tasks with resources assigned, the work and rate values can be overridden by providing **ActualWork**, **RemainingWork**, and **Rate** values on the **ResourceAssignment** table. Otherwise, if no valid work and rate values are provided (for example, if Rate <= 0), then the value provided in the **Duration** field is used instead. **Default value:** FixedUnits For more information about how this field is used, see "Task durations" on page 2176.	String (Enum)	
ExpectedFinishDate	The date work on the task is expected to be completed. This value is used in calculating the finish date for **FixedDuration** tasks that are in progress in projects where the **ProjectType.PlanningRule** is set to "DateDriven".	Date	
FinishDate	If **SchedulingRule** is set to "Manual", this field can be used to provide an input finish date on which the task must finish. Otherwise, if **SchedulingRule** is set to "Automatic", then RapidResponse automatically calculates a realistic finish date based on other input parameters and this field is ignored (can be left undefined). **Note:** If there is a requirement to fix the finish date of automatically scheduled tasks on or before a given date, then a "FinishNoEarlierThan" task constraint can be used.	Date	
Id	A unique string identifier for this task.	String	Yes

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
Links	A string field used to hold URLs which can then be used to hyperlink to externally stored documents relevant to the task or project. For example, a link to related engineering specifications might be required.	String	
Milestone	Specifies if the task is a milestone. Typically, these are used to mark important points on the project schedule. For example, a milestone might indicate the point at which revenue can be recognized. Valid values are: • **N**—a normal task. • **Y**—a milestone task. Note that milestone tasks in projects where **ProjectType.AllowNonZeroMilestones** is set to "N" (the default setting) do not contain work to complete, do not generate resource loads, and always have a **CalcDuration** of 0. Otherwise, if **AllowNonZeroMilestones** is set to "Y", then a milestone task can represent actual work and have a positive calculated duration.	Boolean	
Name	A name to associate with this task. This value should make the task easy to recognize or identify by those working on the project.	String	
Notes	Descriptive notes appended to this task (for example, an explanation of why a task is expected to be late). Each time a note value is added to this field through a worksheet, a new record is created in the **Task_Notes** table with a datetime stamp, user ID, note text, and a reference to the Task record against which it was created.	Note	
OriginalStartDate	An optional field not used by analytic calculations, but which can be used to store a copy of the value returned by the CalcStartDate field for baseline comparison against future calculated results.	Date	
OriginalFinishDate	An optional field not used by analytic calculations, but which can be used to store a copy of the value returned by the CalcFinishDate field for baseline comparison against future calculated results.	Date	
PenaltyDate	The date from which penalties can potentially accumulate to this task. Penalty costs are calculated based on the referenced **PenaltySchedule** and **TaskType** values, and can be applied between this date and CalcFinishDate. For more information about how this field is used, see "Penalties" on page 2191.	Date	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
PenaltySchedule	The name of the penalty schedule used for calculating task penalties. Together with the rules defined on the task type, this determines how penalties are calculated at the task level. **Referenced table:** PenaltySchedule (Nullable)	Reference	
PointsCurve	A reference to the set of points that constitute the efficiency curve applied to resources assigned to this task when **Type.EfficiencyCurveRule** is set to "PointsCurve". **Referenced table:** PointsCurve (Nullable)	Reference	
Project	A reference to the project this task belongs to. **Referenced table:** Project	Reference	Yes
Ramp	Value used to construct a linear increasing efficiency curve. The resulting efficiency curve is then applied to resources assigned to this task when **Type.EfficiencyCurveRule** is set to "Ramp". Note that the value specified in this field determines the number of distinct points on the resulting curve, with the points spread evenly between 0 and 1. For example, if a value of 5 is specified, resources on this task have an efficiency of .2 on the first date, .4 on the second date, .6 on the third date, .8 on the fourth date, and 1 (full efficiency) on the fifth and all subsequent dates. Using this curve, a task that is otherwise expected to have a duration of 4 days, would have a duration of 6 days instead.	Integer	
Rate	If **Task**.**DurationType** is "FixedWorkAndRate", then this value can be used to determine the rate at which work is completed on the task (and hence impacts the calculated task duration). Otherwise, this field is ignored. For example, assume a task with Work of "5"and Rate of "1.25". The calculated task duration would then be 4; that is, (5 /1.25). **Note:** If the task has resources assigned to it, then this value can be overridden by providing values in the **ResourceAssignment.Rate** field.	Quantity	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
RateAdjustment	An optional percentage adjustment to be applied to the standard and overtime rates charged for any resources assigned to this task. This value is considered only if **ApplyRateAdjustment** is set to "Y". If the adjustment represents a premium charged on resources rates, a positive value should be specified. If the adjustment represents a discount applied to resource rates, a negative value should be specified. For example, .15 indicates a 15 percent premium while -.2 indicates a 20 percent discount. Note that this adjustment overrides any adjustments specified at the project level, however if there is an adjustment specified at a more granular task level, then that value is used instead of this one. For example, adjustments can also be specified for only certain resources applied to the task. For more information, see "Resource rate adjustments" on page 2189. However, if there is a more resource specific adjustment specified on the task, then that value is used instead of this one.	Quantity	
RemainingDuration	The amount of days remaining until the task is completed. Used when calculating the **CalcDuration** field. If populating this field, a value should typically be provided for the **ActualDuration** field as well.	Quantity	
Revenue	Used to provide a revenue amount associated with this task. Subsequently used when calculating **Project.TaskRevenue**.	Money	
SchedulingRule	A rule used to determine how the task's calculated start and finish dates (**CalcStartDate** and **CalcFinishDate**) are determined. Valid values are: • **Automatic**—task start and finish dates are automatically calculated by RapidResponse based on project start/finish dates, predecessor task dates, material order availability, constraints, and other parameters. • **Manual**—task start and finish dates are set to the input values provided in the **StartDate** and **FinishDate** fields. This option might typically be used in the early stages of project when all task details and timings are not known. **Default value**: Automatic **Note:** For more information about how tasks are scheduled, see "Task scheduling" on page 2167.	String (Enum)	

Table 6.339: Task (ProjMgmt) fields

Field name	Description	Data type	Key
SortKey	String value used to sort tasks in the project hierarchy. Typically, represented as a period-delimited list of integers (for example, 2.1.1).	String	
StartDate	If **SchedulingRule** is set to "Manual", this field can be used to provide an input start date on which the task must start. Otherwise, if **SchedulingRule** is set to "Automatic", then RapidResponse automatically calculates a realistic start date based on other input parameters and this field is ignored (can be left undefined).	Date	
Type	A value associated with processing rules that control certain calculations at the project level, such as how task duration and penalties are determined. **Referenced table:** TaskType	Reference	
Work	If **Task**.**DurationType** is "FixedWorkAndRate", then this value is used to determine the amount of work to be done on the task (and hence impacts the calculated task duration). Otherwise, this field is not used in calculating task duration. For example, assume a task with Work of "6"and Rate of "1.". The calculated task duration would then be 6; that is, (6 / 1). **Note:** If the task has resources assigned to it, then this value can be overridden by providing values in the **ActualWork** and **RemainingWork** fields on the **ResourceAssignment** table.	Quantity	

Task table – calculated and set fields

The following describes the calculated and set fields in the Task table. All calculated fields that are directly editable in any worksheets automatically configured for project management, such as in the Project Plan workbook, are also noted. For an introduction to this table, and descriptions of its input fields, see "Task table" on page 624.

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
ActualMaterial Cost	Total actual material cost summed across all demand and/or supply order dependencies associated with this task as follows: • For any portion of a material part requirement assigned to a supply order, the assigned quantity is multiplied by the **CostofGoodsSold** on the **ScheduledReceipt** record, and then divided by the **EffQuantity** on the **ScheduledReceipt** record. • For any portion of a material part requirement assigned to a demand order, the assigned quantity is multiplied by the value in **IndependentDemand.CostOfGoodsSold** and then divided by the value in **IndependentDemand.EffectiveDemand**.	Money	
Bonus	Bonuses earned at the task level. Task bonuses can only be earned on dates between **CalcFinishDate** and **BonusDate**, where the **BonusSchedule** field references a valid bonus schedule, and the referenced **ProjectType.BonusRule** value is other than "None". For more information about how this field is calculated, see "Bonuses" on page 2196. **Note:** All task-level bonuses are added to any project-level bonuses and together reported in the **Project.CumulativeBonus** field.	Money	
CalcActualDuration	The calculated number of days of work completed since the task was started. If the task belongs to a project where **Type.PlanningRule** is set to "Standard", this field always reports the value found in the **ActualDuration** field. Otherwise, if the task belongs to a project where **Type.PlanningRule** is set to "DateDriven", then this field is calculated based on the task's progress state as follows: • when **CalcState** returns "NotStarted", this field is set to zero (0). • when **CalcState** returns "InProgress", this field is calculated as the difference between the project's current working date and the task's calculated start date as follows: *Project.RunDate - CalcStartDate* • when **CalcState** returns "Completed', this field is set to the value in **CalcDuration**.	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
CalcActualWork	The calculated amount of work that has already been completed on this task. If the task has any resources assigned to it, this is the sum of all applicable **ResourceAssignment.ActualWork** values. Otherwise, if there are no resources assigned, this is set to the value in **ActualWork**.	Quantity	
CalcDuration	The amount of time this task is calculated to take. If the **SchedulingRule** field is set to "Automatic", then the duration is calculated as follows: • For normal tasks, this field is calculated based on the rule selected in the **DurationType** field as discussed in "Task durations" on page 2176. • For summary tasks, this value is calculated as the difference between the earliest calculated start date and latest calculated finish date on any sub-task. • For milestone tasks, this is always set to 0 if **ProjectType.AllowsNonZeroMilestones** is set to "N" (and calculated like a normal task otherwise). If the **SchedulingRule** field is set to "Manual", then the duration is calculated as follows: • For normal and summary tasks, if both the **StartDate** and **FinishDate** fields are defined, this is calculated as the difference between those two dates. Otherwise, set to the value in the input **Duration** field. • For milestone tasks, this is always set to 0 if **ProjectType.AllowsNonZeroMilestones** is set to "N" (and calculated like a normal task otherwise). **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable.	Quantity	
CalcFinishDate	The calculated finish date for this task. If **ProjectType.ScheduledRule** is "FromStart", this is set to **EarliestFinishDate**. Otherwise, if **ProjectType.ScheduleRule** is "FromFinish", this is set to **LatestFinishDate**. **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable	Date	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
CalcPercent Complete	Percentage of the task that is determined to be complete. This field holds values between 0 and 1 (with 1 meaning the task is 100% complete). For normal work tasks where **CalcDuration** > 0, this is calculated based on the number of days that have elapsed since work on the task started and the calculated task duration (*ActualDuration / CalcDuration*) under any of the following conditions: • **DurationType** is set to "FixedUnits". • **DurationType** is set to "FixedWorkAndRate". • **DurationType** is set to "FixedDuration" and **Project.Type.PercentCompleteRule** is set to "CalcDuration". Otherwise, calculated as whichever is greater of zero (0) or the amount of the original input duration that has not yet been completed (*Duration-CalcRemainingDuration/Duration*) under the following condition: • **DurationType** is set to "FixedDuration" and **Project.Type.PercentCompleteRule** is set to "OriginalDuration". For all milestone tasks belonging to projects where **ProjectType.AllowNonZeroMilestones** is set to "N", as well as well as any normal work tasks where **CalcDuration** = 0, calculated based on the following conditions: • If the task has predecessors, then 1 if every predecessor has **CalcPercentComplete** of "1", or 0 otherwise. • If the task does not have predecessors, then 1 if A**ctualStartDate** and **ActualFinishDate** are both defined, or 0 otherwise. For summary tasks, this returns a weighted average of the values in **CalcPercentComplete** from all of its subtasks. **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable	Quantity	
CalcPercent Received	For tasks with any material part requirements, this indicates the percentage of the required quantity that has already been received (based on the **Quantity** and **CalcPercentReceived** values on all **TaskPartLink** records associated with this task).	Quantity	
CalcPercentWork Complete	Percentage of work on this task that has already been completed (based on the **CalcActualWork** and **CalcWork** values).	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
CalcRemaining Duration	Amount of time remaining until this task is complete. Calculated as the difference between the number of days the task was calculated to take (CalcDuration) and the number of days that have already been worked on the task (CalcActualDuration)	Quantity	
CalcStartDate	The calculated start date for this task. If **ProjectType.ScheduledRule** is "FromStart", this is set to **EarliestStartDate**. Otherwise, if **ProjectType.ScheduleRule** is "FromFinish", this is set to **LatestStartDate**. For more information about calculated task start dates are determined, see "Task scheduling" on page 2167. **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable.	Date	
CalcState	The calculated state of progress for this task based on actual start and finish dates. Valid values are: • **Completed**—For normal work tasks, this indicates that both the **ActualStartDate** and the **ActualFinishDate** are defined. For milestone tasks, this indicates that either the **ActualStartDate** or the **ActualFinishDate** are defined. For summary tasks, this indicates that all child tasks report a value of "Completed" in this field. • **InProgress**—For normal work tasks, this indicates that the **ActualStartDate** is defined but the **ActualFinishDate** is undefined.For summary tasks, this indicates that at least one child task reports a value of "InProgress" in this field. • **NotStarted**—For normal work tasks, this indicates that the **ActualStartDate** is undefined. For milestone tasks, this indicates that both the **ActualStartDate** and the **ActualFinishDate** are undefined. For summary tasks, this indicates that all child tasks report a value of "NotStarted" in this field.	String (Enum)	
CalcWork	The calculated amount of work to be completed on this task. If the task has any resources assigned to it, this is the sum of all applicable **ResourceAssignment.Work** values. Otherwise, if there are no resources assigned, this is set to the value in **Work**.	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
Conflict	Indicates if the task has any scheduling conflicts that prevent the project plan from honoring all task parameters and limitations. Scheduling conflicts typically arise when a task constraint or material order link causes the task dates to move in the opposite direction from which the project is being scheduled. For example, when scheduling tasks from the project start date. a FinishNoLaterThan constraint might cause a conflict if its ConstraintDate pushes the task's start/finish dates earlier than they otherwise would be in respect of other scheduling parameters such as predecessor relationships. Valid values are: • **None**—The task does not have any conflicts. • **Primary**—The task has a conflict in the direction the project is currently scheduled. For example, if the task belongs to a project where **ProjectType.ScheduleRule** is set to "FromStart", this value indicates the task has a conflict as scheduled from the project start date and should be investigated. • **Secondary**—The task has a conflict in the opposite direction from which it is currently scheduled. For example, if the task belongs to a project where **ProjectType.ScheduledRule** is set to "FromStart", this value indicates the task would have a conflict if it were to be scheduled from the project finish date. **Note:** Details of each task conflict are reported in the **Conflict** table as described on page "Conflict table" on page 963.	String (Enum)	
Critical	Indicates if this task is on the critical path for its project. Tasks on the critical path are those which impact the project finish date (that is, tasks for which a delay would delay the project finish date). Valid values are: • **N**—task is not on critical path. • **Y**—task is on critical path.	Boolean	
CumulativeActual Cost	The sum of the **ActualCost** value on the current task and the **ActualCost** values found on all of its subtasks. This might be useful for reporting the miscellaneous costs associated with a summary task.	Money	
CumulativeActual MaterialCost	The sum of the **ActualMaterialCost** value on the current task and the **ActualMaterialCost** values found on all of its subtasks. This might be useful for reporting actual material costs associated with a summary task.	Money	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
CumulativeBonus	The sum of the **Bonus** value on the current task and the **Bonus** values found on all of its subtasks. This might be useful for reporting total bonuses associated with summary tasks.	Money	
CumulativeCost	The sum of the **Cost** value on the current task together with the **Cost** values found on all of its subtasks. This might be useful to reporting total costs associated with summary tasks.	Money	
CumulativeMaterial Cost	The sum of the **MaterialCost** value on the current task and the **MaterialCost** values found on all of its subtasks. This might be useful for reporting planned material costs associated with a summary task.	Money	
CumulativeMaterial Revenue	The sum of the **MaterialRevenue** value on the current task and the **MaterialRevenue** values found on all of its subtasks. This might be useful for reporting revenue associated with summary tasks.	Money	
CumulativePenalty Cost	The sum of the **PenaltyCost** value on the current task and the **PenaltyCost** values found on all of its subtasks. This might be useful for reporting total penalty costs associated with summary tasks.	Money	
CumulativeRevenue	The sum of the **Revenue** value on the current task together with the **Revenue** values found on all of its subtasks. This might be useful to reporting total revenue costs associated with summary tasks.	Money	
DaysLate	Indicates the number of days late the task is currently expected to be. Calculated as the difference between ActualDuration and the expected value of ActualDuration. A negative value in this field indicates a task that is ahead of schedule.	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
EarliestFinishDate	The earliest date this task can finish. The calculation of this date depends on the setting in the **SchedulingRule** field as follows: • If **SchedulingRule** is set to "Automatic", then this date is calculated based on factors such as the input project start date, project run date, predecessor/successor relationships, order dependencies, resource availability, constraints, and so on. • If **SchedulingRule** is set to "Manual" and the **FinishDate** field is populated, then the value in **FinishDate** is used. Otherwise, if **FinishDate** is not populated, this is calculated by adding the value in **CalcDuration** to **StartDate.** **Note:** If **ProjectType.ScheduleRule** is set to "FromStart", this also becomes the task finish date as reported in **CalcFinishDate**.	Date	
EarliestStartDate	The earliest date this task can start. The calculation of this date field depends on the setting in the **SchedulingRule** field as follows: • If **SchedulingRule** is set to "Automatic", then this date is calculated based on factors such as the input project start date, project run date, predecessor/successor relationships, order dependencies, resource availability, constraints, and so on. • If **SchedulingRule** is set to "Manual" and the **StartDate** field is populated, then the value in **StartDate** is used. Otherwise, if **StartDate** is not populated, this is calculated by subtracting the value in **CalcDuration** from **FinishDate.** **Note:** If **ProjectType.ScheduleRule** is set to "FromStart", this also becomes the task start date as reported in **CalcStartDate**.	Date	
EffectiveCalendar	The calendar used when calculating start and finish dates for this task. If the task belongs to a task type that has a calendar value defined, then that value (**Task.Type.Calendar**) is used. Otherwise, the calendar defined at the project level (**Project.Type.Calendar**) is used.	Reference	
FreeSlack	The amount of time, in working days, the task can be delayed without delaying a successor task. If the task has no successors, this reports the same value as **TotalSlack**.	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
Index	An index number that can be used to sort tasks across the entire project. Starts at 1 for the first top-level task in the project and increments by 1 for each additional task.	Quantity	
IsSummary	Indicates if the task is a summary task. Summary tasks roll up and summarize data from the sub-tasks (children) below them. Valid values are: For any portion of a material requirement assigned to a demand order, the assigned quantity is multiplied by the value in **IndependentDemand.CostOfGoodsSold** and then divided by the value in **IndependentDemand.EffectiveDemand**. • **N**—not a summary task. • **Y**—summary task.	Boolean	
LatestFinishDate	The latest date this task can finish. The calculation of this date field depends on the setting in the **SchedulingRule** field as follows: • If **SchedulingRule** is set to "Automatic", then this date is calculated based on factors such as the input project finish date, project run date, predecessor relationships, order dependencies, resource availability, constraints, and so on. • If **SchedulingRule** is set to "Manual" and the **FinishDate** field is populated, then the value in **FinishDate** is used. Otherwise, if **FinishDate** is not populated, this is calculated by adding the value in **CalcDuration** to **StartDate.** **Note:** If **ProjectType.ScheduleRule** is "FromFinish", this become the task finish date as reported in **CalcFinishDate**.	Date	
LatestStartDate	The latest date this task can start. The calculation of this date field depends on the setting in the **SchedulingRule** field as follows: • If **SchedulingRule** is set to "Automatic", then this date is calculated based on factors such as the input project finish date, project run date, predecessor/successor relationships, order dependencies, resource availability, constraints, and so on. • If **SchedulingRule** is set to "Manual" and the **StartDate** field is populated, then the value in **StartDate** is used. Otherwise, if **StartDate** is not populated, this is calculated by subtracting the value in **CalcDuration** from **FinishDate.** **Note:** If **ProjectType.ScheduleRule** is "FromFinish", this becomes the task start date as reported in **CalcStartDate**.	Date	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
MaterialCost	Total planned material cost summed across all material part requirements for the task. For each material part link defined, the quantity required is multiplied by the value in **Part.StdUnitCos**t.	Money	
MaterialRevenue	Revenue from all demand orders associated with this task. For each demand order, the quantity assigned to the task is multiplied by the value in **IndependentDemand.EffectiveUnitPrice**.	Money	
OutlineLevel	A number used to indicate the level of the task in the project structure. For example, top-level tasks have a value of "o", the next level of tasks down have values of "1", and so on.	Quantity	
OutlineNumber	A string value used to indicate the task's numerical position in the project's overall task hierarchy. For example, the first top-level task has a value of 1, and tasks underneath it have values such as 1.1., 1.2, and so on.	String	
Parent	A reference to the summary task, if any, this task belongs to. Otherwise, Null. **Referenced table:** Task	Reference	
PenaltyCost	Penalty costs incurred at the task level. Penalty costs can only be incurred on task dates between **PenaltyDate** and **CalcFinishDate**, where the **PenaltySchedule** field references a valid penalty schedule, and the referenced **TaskType.PenaltyRule** value is other than "None". For more information about how this field is calculated, see "Penalties" on page 2191. **Note:** Any task-level penalties are added to their project's penalties and together reported in the **Project.CumulativePenaltyCost** field.	Money	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
PredecessorList	A comma-delimited list of all predecessor's for this task (that is, those tasks that must come before this one) as defined in the **Predecessor** table. Predecessors tasks are referred to by **Index** value, and values in this field might also contain string details such as "SF" and "+5" to indicate the type of predecessor rule used along with any applicable offset value. For example, a value of "3FS+2" indicates the current task can start 2 days after the task with Index value of 3 finishes. **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable and values entered here are added or updated in the **Predecessor** table.	String	
ResourceActual FixedCost	Actual fixed costs incurred during project execution when assigning resources to this task. Rolled up from associated values specified in the **ResourceCostActual.FixedCost** field.	Money	
ResourceActual OvertimeCost	Actual overtime costs incurred during project execution from assigned resources working on this task in excess of their maximum hours per day. Rolled up from associated values reported in the **ResourceCostActual.OvertimeCost** field.	Money	
ResourceActual StandardCost	Actual standard costs incurred during project execution from assigned resources working on this task up to their maximum hours per day. Rolled up from associated values reported in the **ResourceCostActual.StandardCost** field.	Money	
ResourceFixedCost	Reports the sum of all fixed (one-time) costs incurred from the assignment of resources to this task. If this is a summary task, the sum of fixed resource costs from all its subtasks is reported.	Money	
ResourceOvertime Cost	Reports the sum of all overtime hourly costs incurred by resources while assigned to this task. If this is a summary task, the sum of overtime resource costs from all its subtasks is reported.	Money	
ResourceStandard Cost	Reports the sum of all standard hourly working costs incurred by resources while assigned to this task. If this is a summary task, the sum of standard resource costs from all its subtasks is reported.	Money	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
SiblingIndex	An index number that can be used for sorting sibling tasks (that is, tasks with the same Parent value). Starts at 1 for each grouping of sibling tasks.	Quantity	
SuccessorList	A comma-delimited list of all successors's for the this task (that is, those tasks that must come after this one) as defined in the Predecessor table. Values in this field might also contain string details such as "FS" and "-2" to indicate the type of predecessor rule used along with any applicable offset. **Note:** If using worksheets automatically configured for project management, such as in the Project Plan workbook included with RapidResponse, this field is directly editable and values entered here are added or updated in the **Predecessor** table.	String	
TargetPercent Complete	Expected value of **CalcPercentComplete** if the project is on schedule. This calculation is based on the date returned by the **TaskType.RunDate** calendar.	Quantity	
TargetPercent WorkComplete	For fixed work and rate tasks, the expected percentage of work complete done on the task assuming the project is on schedule. This calculation is based on the date returned by the **TaskType.RunDate** calendar.	Quantity	
TargetWork Complete	For fixed work and rate tasks, the expected amount of work complete done on the task assuming that the project is on schedule. For tasks that do not have **DurationType** set to "FixedWorkAndRate", this always returns 0. This calculation is based on the date returned by the **TaskType.RunDate** calendar.	Quantity	

Table 6.340: Calculated Task (ProjMgmt) fields

Field name	Description	Data type	Key
TotalSlack	The amount of time, in working days, the task can be delayed without delaying the overall project finish date. By default, this field is calculated as: *min (LatestStartDate - EarliestStartDate, LatestFinishDate - EarliestFinishDate)* However, a project level control setting, **ProjectType.TotalSlackRule**, allows for this calculation to instead always use the difference the latest and earliest start date, or always use the difference between the latest and earliest finish date (regardless of which is less). Additionally, this field will always return a value of 0 (zero) in either of the following cases: • The task has an "AsLateAsPossible" constraint and belongs to a project scheduled from start. • The task has an "AsSoonAsPossible" constraint and belongs to a project scheduled from finish with **Type.ASAPBackwardTotalSlackRule** set to "Zero".	Quantity	
Assignments	Set of ResourceAssignment records that reference this Task value.	Set	
BaselineTaskLinks	Set of BaselineTaskLink records that reference this Task value.	Set	
Children	Set of WBS records that reference this Task value as a parent task.	Set	
DemandLinks	Set of TaskDemandLink records that reference this Task value.	Set	
Parents	Set of WBS records that reference this Task value as a child task.	Set	
PartLinks	Set of TaskPartLink records that reference this Task value.	Set	
Predecessors	Set of Predecessor records that reference this Task value as their successor.	Set	
Successors	Set of Predecessor records that reference this Task value as their predecessor.	Set	
SupplyLinks	Set of TaskSupplyLink records that reference this Task value.	Set	

TaskDemandLink table

The **TaskDemandLink** table associates project tasks with specific demand orders for parts that are required to complete those tasks. The availability of the demands (as defined by records in the **IndependentDemand** table) can then be used to constrain a task's start or finish date, as well as being used in calculating material costs for a task.

Note that in order to create a valid link between a task and a demand, a record must also be defined in the **TaskPartLink** table to link the task to the associated part, and that **TaskPartLink** record must have a **Type** value of "Demand".

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse, values can be added to this table through the **Supply Chain** tab of the **Task Information** dialog box.

Table 6.341: TaskDemandLink (ProjMgmt) fields

Field name	Description	Data type	Key
ApplyConstraint	Indicates if the demand available date can potentially constrain the task start date or finish date. Valid values are: • **N**—available date of the linked demand does not impact the task start or finish date. • **Y**—available date of the linked demand impacts the task start date (if **Rule** = "StartDate") or task finish date (if **Rule** = "FinishDate"). **Default value:** Y	Boolean	
AssignedQuantity	The amount of the demand that has been assigned to this task.	Quantity	
Description	A description of this task and demand relationship.	String	
Independent Demand	A reference to a specific demand required to complete this task. **Referenced table:** IndependentDemand	Reference	Yes
Offset	The number of days to add to the demand's available date when determining the tasks' start date (if Rule = 'StartDate') or the task's finish date (if Rule = 'FinishDate'). This field is ignored if Rule = 'None' or ApplyConstraint = 'N'.. This field accepts positive, negative, and fractional values. For example, assuming a Rule of 'StartDate', an Offset of '2' means that the task cannot start earlier than two days after the demand's available date. Similarly, an Offset of '-1' means that the task cannot start earlier than the day before the demand's available date.	Quantity	

Table 6.341: TaskDemandLink (ProjMgmt) fields

Field name	Description	Data type	Key
PartLink	A reference to the task and part relationship that associates the task with the part it requires. **Referenced table:** TaskPartLink	Reference	
ReceivedQuantity	The amount of the demand that has already been received for this task.	Quantity	
Rule	Indicates how the availability of this demand constrains the task completion. Valid values are: • **FinishDate**—the task cannot finish any earlier than the demand's available date plus any applicable offset. Note that if **ApplyConstraint** is set to "N", then the task's finish date is not impacted by the constraint. • **None**—the task is not constrained by the demand available date. • **StartDate**—the task cannot start any earlier than the demand's available date plus any applicable offset. Note that if **ApplyConstraint** is set to "N", then the task's start date is not impacted by the constraint. **Default value:** None For more information about how this field is used, see "Supply and demand order links" on page 2171.	String (Enum)	
Task	A reference to the task that has a demand dependency. **Referenced table:** Task	Reference	Yes

TaskDemandLink table – calculated and set fields

The following describes the calculated and set fields in the TaskDemandLink table. For an introduction to this table, and descriptions of its input fields, see "TaskDemandLink table" on page 646.

Table 6.342: Calculated TaskDemandLink (ProjMgmt) fields

Field name	Description	Data type	Key
CalcPercent Received	The percentage of this demand's assigned quantity that has already been received by the task.	Quantity	
Critical	Indicates if the demand's availability determines when the task can be completed. Valid values are • N • Y	Boolean	

Table 6.342: Calculated TaskDemandLink (ProjMgmt) fields

Field name	Description	Data type	Key
MaterialCost	The cost associated with the quantity of parts assigned to this task from the referenced demand. Based on the sum of all values reported for this record in the **TaskDemandLinkCost** table.	Money	

TaskHyperlink table

The **TaskHyperlink** table is used to store links to internet locations containing more information about a particular project task. For example, links might be added to a page containing diagrams or specifications required for a particular task.

Records in this table are typically added using the **Add Links** dialog box available in the standard **Project Plan** workbook included with RapidResponse. Hyperlinks associated with a given task are then available from a popup menu in the **Links** column of the **Project Plan** worksheet.

Table 6.343: TaskHyperlink (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A short description that displays beside the link label in the Links column popup menu.	String	
Index	A numerical index value used to sort hyperlinks in the Links popup menu (lower values display at the top of a given popup).	Integer	
Label	The clickable text that displays for this link in the Links popup menu (shows instead of the URL).	Label	
URL	The URL this hyperlink should point to. The value provided must begin with a proper formatted protocol (http:)	URL	
Task	A reference to the task this hyperlink pertains to. **Referenced table:** Task	Reference	Yes

TaskPartLink table

The **TaskPartLink** table associates project tasks with any manufactured parts required to complete the task. Once the link between a task and part has been defined, the task can subsequently be set up to have dependencies on specific demands or supplies for the part. These can impact task start and finish dates, and are also used in calculating material costs for a task.

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse, values can be added to this table through the **Supply Chain** tab of the **Task Information** dialog box.

Table 6.344: TaskPartLink (ProjMgmt) fields

Field name	Description	Data type	Key
Description	A description of this task and part relationship.	String	
Part	A reference to a part required to complete the task. **Referenced table:** Part	Reference	Yes
Quantity	The amount of the part that is required to complete this task.	Quantity	
Received Quantity	The amount of the part that has already been received for this task. For tasks with supply or demand links defined, the **ReceivedQuantity** field on the **TaskSupplyLink** or **TaskDemandLink** table is used instead.	Quantity	
Task	A reference to the task that has a part dependency. **Referenced table:** Task	Reference	Yes
Type	Indicates which of demand orders for the part or supply orders for the part can be associated with this task. Valid values are: • **Demand**—this task, and hence its material costs as well as start or finish dates, can be tied to the availabilities of specific demands for the part. References made to the task through the **TaskDemandLink** table are valid and are used to associate the task with records in the **IndependentDemand** table. Any references to the task set up through the **TaskSupplyLink** table are ignored. • **Supply**—this task, and hence its material costs as well as start or finish dates, can be tied to the availabilities of specific supplies for the part. References made to the task through the **TaskSupplyLink** table are valid and are used to associate the task with records in the **ScheduledReceipt** table. Any references to the task set up through the **TaskDemandLink** table are ignored. **Default value:** Demand	String (Enum)	

TaskPartLink table – calculated and set fields

The following describes the calculated and set fields in the TaskPartLink table. For an introduction to this table, and descriptions of its input fields, see "TaskPartLink table" on page 648.

Table 6.345: Calculated TaskPartLink (ProjMgmt) fields

Field name	Description	Data type	Key
CalcPercent Received	The percentage of the part quantity that is required to complete the task and has already been received. For tasks with supply or demand links set up, the sum of the ReceivedQuantity field across all valid supply or demand links is used in calculating this value. Otherwise, for tasks with no supply or demand links, the ReceivedQuantity field in this table is used.	Quantity	
MaterialCost	Planned material cost for this part requirement on the task. Based on the sum of all values reported for this record in the **TaskPartLinkCost** table.	Money	
Unassigned Quantity	The quantity of the part that is required to complete the task, and has not yet been assigned from any valid supply or demand links.	Quantity	
DemandLinks	Set of TaskDemandLink records that reference this TaskPartLink value.	Set	
SupplyLinks	Set of TaskSupplyLink records that reference this TaskPartLink value.	Set	

TaskSupplyLink table

The **TaskSupplyLink** table associates project tasks with specific supply orders for parts that are required to complete those tasks. The availability of the supplies (as defined by records in the **ScheduledReceipt** table) can then be used to constrain a task's start or finish date, and is also used in calculating material costs for a task.

Note that in order to create a valid link between a task and a supply, a record must also be defined in the **TaskPartLink** table to link the task to the associated part, and that TaskPartLink record must have a **Type** value of "Supply".

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse, values can be added to this table through the **Supply Chain** tab of the **Task Information** dialog box.

Table 6.346: TaskSupplyLink (ProjMgmt) fields

Field name	Description	Data type	Key
ApplyConstraint	Indicates if the supply available date can potentially constrain the task start date or finish date. Valid values are: • **N**—available date of the linked supply does not impact the task start or finish date. • **Y**—available date of the linked supply impacts the task start date (if **Rule** = "StartDate") or task finish date (if **Rule** = "FinishDate"). **Default value:** Y	Boolean	
AssignedQuantity	The amount of the supply that has been assigned to this task.	Quantity	
Description	A description of this task and supply relationship.	String	
Offset	The number of days to add to the supply's available date when determining the task start date (if Rule = 'StartDate') or the task finish date (if Rule = 'FinishDate'). This field is ignored if Rule = 'None' or ApplyConstraint = 'N'. This field accepts positive, negative, and fractional values. For example, assuming a Rule of 'StartDate', an Offset of '3' means that the task cannot start earlier than three days after the supply's available date. Similarly, an Offset of '-2' means that the task cannot start earlier than two days before the supply's available date.	Quantity	
PartLink	A reference to the task and part relationship that associates the task with the part it requires. **Referenced table:** TaskPartLink	Reference	
ReceivedQuantity	The amount of the supply that has been already been received for the task.	Quantity	

Table 6.346: TaskSupplyLink (ProjMgmt) fields

Field name	Description	Data type	Key
Rule	Indicates how the availability of this supply constrains the task completion. Valid values are: • **FinishDate**—the task cannot finish any earlier than the supply's available date plus any applicable offset. Note that if **ApplyConstraint** is set to "N", then the task's finish date is not impacted by the constraint. • **None**—the task is not constrained by the supply available date. • **StartDate**—the task cannot start any earlier than the supply's available date plus any applicable offset. Note that if **ApplyConstraint** is set to "N", then the task's finish date is not impacted by the constraint. **Default value:** None For more information about how this field is used, see "Supply and demand order links" on page 2171.	String (Enum)	
ScheduledReceipt	A reference to a specific supply required to complete this task. **Referenced table:** ScheduledReceipt	Reference	Yes
Task	A reference to the task that has a supply dependency. **Referenced table:** Task	Reference	Yes

TaskSupplyLink table – calculated and set fields

The following describes the calculated and set fields in the TaskSupplyLink table. For an introduction to this table, and descriptions of its input fields, see "TaskSupplyLink table" on page 650.

Table 6.347: Calculated TaskSupplyLink (ProjMgmt) fields

Field name	Description	Data type	Key
CalcPercent Received	The percentage of this supply's assigned quantity that has already been received by the task.	Quantity	
Critical	Indicates if the supply's availability determines when the task can be completed.Valid values are: • N • Y	Boolean	
MaterialCost	The cost associated with the quantity of parts assigned to this task from the referenced supply. Based on the sum of all values reported for this record in the **TaskSupplyLinkCost** table.	Money	

TimePhasedDemandParameterSet table

The **TimePhasedDemandParameterSet** table is used to manually specify time-phased average and standard deviation of historical demands for a given safety stock item (part). These values can then be used as parameters in determining recommended safety stock levels on time-phased safety stock items. This table is intended for cases where there is limited historical demand data available for an item but the time-phased statistical parameters are known or estimated through some other means. Typically, one record should be specified for each period/season in a full cycle.

Table 6.348: TimePhasedDemandParameterSet(Mfg) fields

Field name	Description	Data type	Key
Average	Average historical demand for this item in the specified period (by index).	Quantity	
Id	A unique string identifier for this record.	String	Yes
Index	A zero-based index used to indicate the period or season to which this record's parameters apply within the full seasonal cycle. For example, if **CycleCalendar** is "Year" and **PeriodCalendar** is "Month", then 0 refers to January, 1 refers to February, and so on.	Integer	
SafetyStockItem	A reference to the safety stock item to which the parameters on this record apply. The values defined on this record are only used in safety stock calculations when this item references a **SafetyStockItemType** record that has **TimePhasedProcessingRule** set to "Use" and **NonStationaryDemandRule** set to "Manual". **Referenced table:** SafetyStockItem	Reference	Yes
StandardDeviation	Standard deviation of historical demand for this item in the specified period (by index).	Quantity	

TimePhasedLocationCapacity table

The **TimePhasedLocationCapacity** table is used to define the time-phased storage capacity of locations in a distribution network. The location in this table is referenced from the **Location** table. This table was added in the Version 2016.2, Service Update 5, to support the Distribution Requirements Planning application.

Table 6.349: TimePhasedLocationCapacity (Solutions) fields

Field name	Description	Data type	Key
Date	The date when this time-phased storage capacity record becomes effective.	Date	Yes
Location	The location for which this time-phased storage capacity record is specified. **Referenced table**: Location	Reference	Yes
StorageCapacity	The time-phased storage capacity quantity specified for the location.	Quantity	
StorageCapacity UOM	Defines the unit of measure used for the location. **Referenced table**: UnitofMeasure	Reference	

TimePhasedMaximumInventory table

The **TimePhasedMaximumInventory** table is used to set the maximum inventory quantity for a part. If a part has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and the part exceeds the set maximum inventory level at the sourcing site, this triggers a request for the finished goods to be transferred (pushed) to a destination site before it is required.

This table was added in Version 2016.2.

Table 6.350: TimePhasedMaximumInventory (Mfg) fields

Field name	Description	Data type	Key
Part	The name of the part for which the maximum inventory level is set. **Referenced table**: Part	Reference	Yes
Pool	A reference to the pool for the part. **Referenced table**: Pool	Reference	Yes
Date	Date when the maximum inventory value for the part becomes effective.	Date	Yes
Quantity	The amount of the maximum inventory that is set for the part starting on the Date. This value is reported in the unit of measure used by the part.	Quantity	

TimePhasedPartSourceLeadTime table

The **TimePhasedPartSourceLeadTime** table can be used to define time-phased lead time parameters at the part source level. For example, if fixed or dock-to-stock lead time values are expected to change over

time, this table can be used to hold those values.

Each record in this table applies to a given **PartSource** as of a specified **EffectiveInDate** and remains effective until the next **EffectiveInDate** found on a record for that same **PartSource**. When determining which set of lead time values apply to a given supply from the part source, the record with the nearest **EffecitveInDate** on or before the **PlannedOrder.ParameterDate** or **ScheduledReceipt.ParameterDate** is used. When this table is not used, or if it does not contain an effective record for a part source in a given period, the default **FixedLeadTime** and **DockToStockLeadTime** values from the **PartSource** table are used instead.

This table was added in Version 2016.2, Service Update 18. A **UseTimePhasedPartSourceLeadTime** field was also added to the **PartSourceType** table to give further control over whether values added to this table are used in lead-time calculations.

Table 6.351: TimePhasedPartSourceLeadTime(Mfg) fields

Field name	Description	Data type	Key
DockToStockLeadTime	The time required to move materials from the receiving dock to available inventory. Expressed in **Part.PlanningCalendars.TimeUnits** calendar intervals. **NOTE**: A negative value indicates to use the **DockToStockLeadTime** value from the previous (next earliest) record.	Quantity	
EffectiveInDate	The date on which the lead time values defined on this record become effective for the part source.	Date	Yes
FixedLeadTime	A fixed amount of time to produce each order for the part from this source. Expressed in **PartSource.Source.LeadTimeUnit** calendar intervals. **NOTE**: A negative value indicates to use the **FixedLeadTime** value from the previous (next earliest) record.	Quantity	
PartSource	The part source to which the time-phased lead time values on this record apply. **Referenced table**: PartSource	Reference	Yes

TimePhasedSafety table

The **TimePhasedSafety** table contains date and quantity values used to determine time-phased safety stock quantities for parts with certain **SafetyStockQuantityRule** settings (as defined on the applicable **SafetyStockPolicy** or **PartType** record).

For parts where the effective **SafetyStockQuantityRule** is set to either "TimePhasedQuantity" or "TimePhasedQuantityToLatest", this table is used to set the actual safety stock quantities that becomes effective on specified dates.

For parts where the effective **SafetyStockQuantityRule** is set to “TimePhasedRangeOfCoverage”, this table is used to define the number of days of the average daily requirement used in calculating the safety stock quantities within range of coverage zones that begin on specified dates.

Note that **TimePhasedRangeOfCoverage** does not calculate dynamic safety stock for the last period of demand if there is no future demand. For example, it is expected for RapidResponse to recalculate the safety stock level in April 2016 and drop it to zero. Inserting a new TimePhasedSafety record for 08-01-2016 doesn't impact the calculation. Changing the date on the new record to 04-01-2016 when the last demand is due, drops the SS level to zero, as expected.

For parts where the effective **SafetyStockQuantityRule** is set to either “TimePhasedSafetyDaysTotal” or “TimePhasedSafetyDaysMaximum”, this table is used to set the number of days of demand and a quantity that are used in calculating safety stock as of a specified date. Note these two settings are only available on the **SafetyStockPolicy** table (not included as options in **PartType.SafetyStockQuantityRule**).

For more information about using this table, see "Time-phased safety stock" on page 1403, or "Time-phased safety days" on page 1407, or "RangeOfCoverage calculations" on page 1409.

Table 6.352: TimePhasedSafety (Mfg) fields

Field name	Description	Data type	Key
Date	The date on which the safety stock quantity defined by this record becomes effective. The actual usage of this field depends on the **SafetyStockQuantityRule** setting as follows: • **TimePhasedSafetyDaysMaximum**—the date on which the values in the Days and Quantity fields become effective for use in calculating safety stock levels as the maximum of collected demands and specified quantity. • **TimePhasedSafetyDaysTotal**—the date on which the values in the Days and Quantity fields become effective for use in calculating safety stock levels as the sum of collected demands and specified quantity. • **TimePhasedQuantity**—the date on which the value in the **Quantity** field becomes effective as the safety stock level. • **TimePhasedQuantityToLatest**—the date on which the value in the **Quantity** field becomes effective as the safety stock level. • **TimePhasedRangeOfCoverage**—beginning of the range of coverage zone where the value in the **Days** field is used in calculating the safety stock quantity.	Date	

Table 6.352: TimePhasedSafety (Mfg) fields

Field name	Description	Data type	Key
Days	Number of days ahead to look when collecting demands for use in calculating dynamic safety stock levels. The specific usage of this field depends on the **SafetyStockQuantityRule** setting as follows: • **TimePhasedSafetyDaysMaximum**— the **SafetyStockPolicy.SafetyDaysCalendar** intervals ahead to collect demands for use in calculating the safety stock level (the maximum of the sum of collected demands and **Quantity** becomes the safety stock level on a given date). • **TimePhasedSafetyDaysTotal**—the number of **SafetyStockPolicy.SafetyDaysCalendar** intervals ahead to collect demands for use in calculating the safety stock level (the total of the sum of collected demands and **Quantity** becomes the safety stock level on a given date). • **TimePhasedQuantity**—this field is ignored. • **TimePhasedQuantityToLatest**—this field is ignored. • **TimePhasedRangeOfCoverage**— the number of days of the average daily requirement that calculated safety stock levels should satisfy within the zone that begins on the date defined in the **Date** field. **Note:** If a fractional quantity is specified when using either **TimePhasedSafetyDaysTotal** or **TimePhaseSafetyDaysMaximum**, the fractional amount indicates the percentage of demand to use from the last day being considered (for example, a value of 2.4 means to take the next two days of demand, plus forty percent of the demand in the third day). **Note:** If a negative value is specified when using either **TimePhasedSafetyDaysTotal** or **TimePhasedSafetyDaysMaximum**, then the previous (non-negative) Days value defined on a record for the part is used. **Note:** The **TimePhasedSafetyDaysTotal** and **TimePhasedSafetyDaysMaximum** options referred to above are not supported if using the legacy **PartType** settings for safety stock quantity.	Quantity	

Table 6.352: TimePhasedSafety (Mfg) fields

Field name	Description	Data type	Key
OrderPriority	The priority to assign to this safety stock requirement. If this field is left Null and **SafetyStockPolicy.DemandTransformationRule** is set to "Ignore", the part's default priority from **PartType.DefaultPriority** is used instead (if the **DemandTransformationRule** is set to "FirstInFirstOut" then prioritized safety stock should not be used). When the part's **SafetyStockPolicy.CTPHoldRule** is set to "Use", this priority determines how maintaining the safety stock requirement is prioritized against actual demands. When there is limited supply available to satisfy all demands, safety stock requirements are preferred to actual demands of the same or lower priority. As a result, supply allotments to lower-priority demands can be temporarily held to ensure safety stock levels are maintained. Note that Also that if a part has multiple safety stock requirements at different priority levels, each is handled separately. That is, an increase or decrease in the safety stock level at a given priority does not affect or change the part's safety stock levels at any other priorities. **Referenced table**: OrderPriority (Nullable) **NOTE**: This field was added in Version 2016.2, Service Update 18. It is supported for use with the **SafetyStockQuantityRule** of "TimePhasedQuantity" (other settings such as "TimePhasedSafetyDaysMaximum" and "TimePhasedSafetyDaysTotal" are not supported). For more information about how priorities on safety stock are used in capable-to-promise logic, see "Priority and safety stock holds in CTP" on page 1780	Reference	
Part	The Part this record applies to. **Referenced table:** Part **Reference Syntax**: Part.Name, Part.Site	Reference	
Pool	Indicates the pool in which the part's safety stock quantity should be planned. Applicable only when pooling logic is enabled and **MUEPoolNettingType.SafetyStockRule** is set to "ByPool". If this reference is null, the safety stock quantity is planned in the common pool. **Referenced table**: Pool (Nullable)	Reference	

Table 6.352: TimePhasedSafety (Mfg) fields

Field name	Description	Data type	Key
Quantity	The safety stock quantity that becomes effective on the date specified in the **Date** field. The actual usage of this field depends on the **SafetyStockQuantityRule** setting as follows: • **TimePhasedSafetyDaysMaximum**—the quantity compared against the sum of demands collected over some of forward-looking intervals (as set by the **Days** field) to determine the safety stock level on a given day (the maximum of these two values is used). • **TimePhasedSafetyDaysTotal**—the quantity added to the sum of demands collected over some number of forward-looking intervals (as set by the **Days** field) to determine the safety stock level on a given day (the total of these two values is used). • **TimePhasedQuantity**—the actual safety stock quantity that becomes effective on the specified date. • **TimePhasedQuantityToLatest**—the actual safety stock quantity that becomes effective on a specified date (excepting situations where the safety stock level decreases after the last demand for the part). • **TimePhasedRangeOfCoverage**—this field is ignored. **Note:** If a negative value is specified when using either **TimePhasedSafetyDaysTotal** or **TimePhasedSafetyDaysMaximum**, then the previous (non-negative) **Quantity** value defined on a record for the part is used. **Note:** The **TimePhasedSafetyDaysTotal** and **TimePhasedSafetyDaysMaximum** options referred to above are not supported if using the legacy **PartType** settings for safety stock quantity.	Quantity	

TimePhasedSourceLeadTime table

The **TimePhasedSourceLeadTime** table can be used to define time-phased lead time parameters at the source level. For example, if transit time or pre-ship lead time values are expected to change over time, this table can be used to hold those values.

Each record in this table applies to a given **Source** as of a specified **EffectiveInDate** and remains effective until the next **EffectiveInDate** found on a record for that same **Source**. When determining which set of lead time values apply to a given supply from the part source, the record with the nearest **EffecitveInDate** on or before the **PlannedOrder.ParameterDate** or **ScheduledReceipt.ParameterDate** is used. When this table is not used, or if it does not contain an effective record for a source in a given period, the default **PreShipLT**and **TransitTime** values from the **Source** table are used instead.

This table was added in Version 2016.2, Service Update 18. A **UseTimePhasedSourceLeadTime** field was also added to the **PartSourceType** table to give further control over whether values added to this table are used in lead-time calculations.

Table 6.353: TimePhasedSourceLeadTime (Mfg) fields

Field name	Description	Data type	Key
EffectiveInDate	The date on which the lead time values defined on this record become effective for the source.	Date	Key
Source	The source to which the time-phased lead time values on this record apply. **Referenced table**: Source	Reference	Key
PreShipLT	The amount of time needed collect the inventory at the sourcing site and prepare it for shipment to the destination site. Should be expressed in **Source.LeadTimeUnits** calendar intervals. **NOTE**: A negative value indicates to use the **PreShipLT** value from the previous (next earliest) record.	Quantity	
TransitTime	The amount of time required to ship supply from the sourcing site to the destination site. Should be expressed in **Source.TransitCalendar** intervals (or Everyday calendar units if the **TransitCalendar** reference is Null). **NOTE**: A negative value indicates to use the **TransitTime** value from the previous (next earliest) record.	Quantity	

TimePhasedYield table

The **TimePhasedYield** table can be used to define time-phased yield and coproduct yield values. For example, the part source for a new product might have a certain yield when the product is introduced and then higher yields at later points in time as production processes improve.

Each record in this table applies to a given **PartSource** as of a specified **EffectiveInDate** and remains effective until the next **EffectiveInDate** found on a record for that same **PartSource**. If this table is not used (or does not contain an effective record for a part source in a certain period), the default **Yield** and **CoProductYield** values from the **PartSource** table are used instead.

This table was added in Version 2016.2, Service Update 12. For more information about yield calculations, see "Yield" on page 1391

Table 6.354: TimePhasedYield (Mfg) fields

Field name	Description	Data type	Key
CoProduct Yield	The percentage of a primary product's supply production that is expected to actually end up as the primary product. The remainder is distributed between its co-products and by-products based on the **BillOfMaterial.QuantityPer** values on the records defining those co-product and by-product relationships.. This field respects the setting in the **OrderPolicy.YieldUsage** field, is applied after the **Yield** field, and is always considered by Netting calculations. If not using co-product and by-product logic in RapidResponse, this field should be set to 0 (to ensure the field is not used in Netting calculations). **NOTE**: A negative value indicates to use the **CoProductYield** value from the previous (next earliest) record.	Quantity	
EffectiveIn Date	The date on which the time-phased yield and/or co-product yield values on this record become effective for the part source.	Date	Yes
PartSource	The part source to which the time-phased yield and/or co-product yield values on this record apply. **Referenced table:** PartSource	Reference	Yes
Yield	The expected yield on supplies from this part source after consideration for scrap during production. New planned order quantities are subsequently inflated for yield while firm scheduled receipt quantities are reduced by yield. Note that the interpretation of the value in this field is affected by the **OrderPolicy.YieldUsage** setting for the part source. For example, if five percent of supply is expected to be lost as scrap and **YieldUsage** is set to "ScrapPercent", then this field should be set to "5", however if **YieldUsage** were set to "YieldFraction" then this field should be set to "0.95". **NOTE**: A negative value indicates to use the **Yield** value from the previous (next earliest) record.	Quantity	

TransferDestination table

The **TransferDestination** table is used to model and store routes for transferring inventory where you cannot specify a part source because it would cause a recursive relationship with an existing part source. For example, it could be used to transfer inventory to a site that is upstream in your distribution network.

This table was added in the Version 2016.2, Service Update 5, to support the Distribution Requirements Planning application.

Table 6.355: TransferDestination (Solutions) fields

Field name	Description	Data type	Key
Id	A string identifier for this transfer destination.	String	
Default	Indicates if this is a default transfer destination.	Boolean	
DockToStock LeadTime	The time required to move materials from the receiving dock to available inventory at the destination site. The setting in this field increases the effective lead time for all types of supply; ensure it is set to 0 unless needed. It is measured using **Part.PlanningCalendars.TimeUnits**. **Default value**: 0	Quantity	
Priority	The priority associated with this transfer destination.	Integer	
TransferCost	The cost that is specified for an inventory transfer that is using this transfer destination.	Money	
Part	The part for which this transfer destination exists. **Referenced table**: Part **Reference Syntax**: Part.Name, Part.Site	Reference	
Source	References the **Source** record that applies to the transfer destination record. **Referenced table**: Source	Reference	
TransferPart	References the **Part** record for the **Source** site that applies to the transfer destination. This field references the **Part** table. **Referenced table**: Part **Reference Syntax**: Part.Name, Part.Site	Reference	

TransportationMode table

The **TransportationMode** table defines different modes of transportation used to ship orders to customers or to ship transfer orders from one node to another in a distribution network. It is referenced by the **DeliveryRoute** and **Source** tables to indicate the method by which an order moves along a given route to the customer.

Table 6.356: TransportationMode(Mfg) fields

Field name	Description	Data type	Key
Description	A description of this mode of transportation.	String	
Value	A value that identifies this mode of transportation. For example, “Land” or “Rail”.	String	Yes
Carriers	Set of Carrier records that reference this value.	Set	
DeliveryRoutes	Set of DeliveryRoute records that reference this value.	Set	
Transportation Modes	Set of **TransportationMode** records that reference this value. **Note:** This field was added in Version 2016.2, Service Update 5.	Set	

UnitOfMeasureCategory table

The **UnitOfMeasureCategory** table is used to group categories of units. Typically units in the same category can be used to express numerical values at that scale. For example, a category might be mass, volume, or count.

Units that belong to a category are stored in the UnitOfMeasure table, including the relative ratio between the units in that category. For example: millimeters, centimeters, meters, and kilometers might be values for the category “Length”. For more information, see "UnitOfMeasure table" on page 931.

This table was added in Version 2016.2, Service Update 12.

Table 6.357: UnitOfMeasureCategory (Mfg) fields

Field name	Description	Data type	Key
Id	A value that identifies the category for a unit of measure.	String	Yes
Description	A description of the category.	String	

UnitOfMeasureCategory table – calculated and set fields

The following describes the calculated and set fields in the UnitofMeasureCategory table. For an introduction to this table, and descriptions of its input fields, see "UnitOfMeasureCategory table" on page 664.

Table 6.358: Calculated UnitOfMeasureCategory (Mfg) fields

Field name	Description	Data type	Key
UnitOfMeasures	Set of UnitOfMeasure records that reference this UnitOfMeasureCategory value.	Set	

Warehouse table

The **Warehouse** table stores Id and Name values for the warehouses used in identifying the locations for inventory.

This table's **Site** field is optional, and a system or data administrator can choose whether it uniquely identifies records in the table, or is ignored in queries, and not shown in insert definitions, dialog boxes, or the Data Sources and Mapping window.

Table 6.359: Warehouse (Mfg) fields

Field name	Description	Data type	Key
Id	An Id code for the warehouse. **Default value**: blank string	String	Yes
Name	The name of the Warehouse (not used in MRP calculations.) **Default value**: blank string	String	
Site	The site where this warehouse is located. This field is optional. **Referenced table:** Site	Reference	Yes

For more information about making the **Site** field optional, see "Modifying table and field properties" in the *RapidResponse Data Integration Guide*.

Warehouse table – calculated and set fields

The following describes the calculated and set fields in the Warehouse table. For an introduction to this table, and descriptions of its input fields, see "Warehouse table " on page 665.

Table 6.360: Calculated Warehouse (Mfg) fields

Field name	Description	Data type	Key
Locations	Set of Location records referencing this warehouse.	Set	

WBS table

The WBS table holds records that define hierarchical parent-child relationships between project tasks. Parent tasks are typically summary tasks, and child tasks are typically sub-tasks. A given task can have no more than one parent defined, and any tasks without a parent (that is, top-level tasks) are not shown in this table.

If using Task-based worksheets automatically configured for project management, such as in the **Project Plan** workbook included with RapidResponse records are automatically added to this table each time a new task is inserted.

Table 6.361: WBS (ProjMgmt) fields

Field name	Description	Data type	Key
Child	A reference to the child (sub) task in the parent-child relationship defined by this record. **Referenced table:** Task	Reference	Yes
Parent	A reference to the parent (summary) task in the parent-child relationship defined by this record. **Referenced table:** Task	Reference	

WBS table – calculated and set fields

The following describes the calculated and set fields in the WBS table. For an introduction to this table, and descriptions of its input fields, see "WBS table" on page 666.

Table 6.362: Calculated WBS (ProjMgmt) fields

Field name	Description	Data type	Key
IsRecursive	Indicates if a recursive relationship exists between the parent and child task. Valid values are • **N**—relationship is not recursive. • **Y**—relationship is recursive; the Parent task associated with this record is a descendant of the Child task through some other parent/child relationship(s) defined in this table.	Boolean	

WorkCenter table

The WorkCenter table describes a unique work center. The record includes the name, description, cost information, and a set of available capacity records for this work center.

Table 6.363: WorkCenter (Mfg) fields

Field name	Description	Data type	Key
Department	Department in which the work center resides. **Referenced table:** Department (this reference is nullable in new installations of RapidResponse 10.1 and later).	Reference	
Description	Description of the work center. **Default value**: blank string	String	
Name	The name of the work center. **Default value**: blank string	String	Yes
PlanningCalendars	Reference to a PlanningCalendars record that holds the work center's working calendar and run date. **Referenced table:** PlanningCalendars	Reference	
Site	The site that is associated with this work center. **Referenced table:** Site	Reference	Yes
StdLaborRunCost	Cost per standard hour for labor run time. **Default value**: 1	Money	
StdLaborSetupCost	Cost per standard hour for labor setup time.	Money	
StdMachineRun Cost	Cost per standard hour for machine run time. **Default value**: 1	Money	
StdMachineSetup Cost	Cost per standard hour for machine setup time. **Default value**: 1	Money	
Type	WorkCenter type and is a reference to the WorkCenterType table through the WorkCenterType.Value field. **Referenced table:** WorkCenterType	Reference	

WorkCenter table — calculated and set fields

The following describes the calculated and set fields in the WorkCenter table. For an introduction to this table, and descriptions of its input fields, see "WorkCenter table " on page 666.

Table 6.364: Calculated WorkCenter (Mfg) fields

Field name	Description	Data type	Key
Capacity	A set of Capacity records that describe the capacity for this work center.	Set	
LoadsDailyCurrent	A calculated set of LoadDailyCurrent records for this work center using the current due date.	Set	
LoadsDailyPlanned	A calculated set of LoadDailyPlanned records for this work center using the rescheduled scheduled receipt date.	Set	
LoadsDetailCurrent	A calculated set of LoadDetailCurrent records for this work center using the current due date.	Set	
LoadsDetailPlanned	A calculated set of LoadDetailPlanned records for this work center using the rescheduled scheduled receipt due date.	Set	
LoadWeeklyCurrent	A calculated set of LoadWeeklyCurrent records for this work center using the current due date.	Set	
LoadWeeklyPlanned	A calculated set of LoadWeeklyPlanned records for this work center using the rescheduled scheduled receipt due date.	Set	
Operations	A set of Operation records that use this work center.	Set	

CHAPTER 7: Control tables

Control tables contain rules and values that determine the calculations RapidResponse performs on input data, and hence the data produced in calculated tables. Control tables are configured during a RapidResponse deployment, and ensure that RapidResponse calculations emulate those found in your enterprise or host system. These initial settings can subsequently be modified by data or system administrators in RapidResponse. For simulation purposes, worksheets based on RapidResponse Control tables can be modified on a scenarios-by-scenario basis.

Each control table contains a Value field that defines values that are used in the corresponding Type field of the base table. For example, the **Type** field in the **Part** table references the **Value** field in the **PartType** table.

In addition to control tables, certain RapidResponse calculations can be configured using global settings. For more information, see "Configuring analytic calculations using global settings" on page 2203.

In this chapter, you'll learn about the following:

AggregatePartCustomerType table **670**
AllotmentOverrideType table **671**
AlternatePartType table **672**
AnalyticsConfiguration table **674**
AttributeMapType table **676**
AutomatedLeadTimeType table **678**
AutomatedYieldType table **683**
AvailableRule table **686**
BOMType table **687**
Calendar table **695**
CalendarDate table **700**
ConstraintType table **700**
ConstraintUnitOfMeasure table **703**
CRPUnitOfMeasure table **704**
DaysSupplyPolicy table **705**
DemandStatus table **713**
DemandType table **715**
DDMRPRule table **721**
DistributionPlanningRule table **722**
DRPConfiguration table **724**
ExpiryType table **726**
ForecastItemParametersType table **727**
HistoricalDemandCategoryType table **736**
IMConfiguration table **739**

IncrementalRule table 741
InventoryTransferType table 741
IPOConfiguration table 743
MEIOLeadTimeType table 744
MPSConfiguration table 746
MUEPoolNettingType table 749
MultiEchelonSafetyStockRule table 756
MultiLevelSearchRule table 758
OFConfiguration table 767
OnHandType table 768
OperationSequenceType table 768
OperationType table 770
OrderPolicy table 773
OrderPriority table 782
OutlierType table 792
PartSourceType table 797
PartType table 809
PlanningCalendars table 843
ProcessInstanceType table 848
ProductionWheelType table 849
ProjectType table 850
RangeOfCoverage table 855
ResourceType table 860
SafetyStockItemType table 861
SafetyStockPolicy table 874
ScenarioSetting table 887
ShipmentPolicy table 887
ShipSetType table 888
SHSCConfiguration table 889
SOPAnalyticsConfiguration table 892
SourceRule table 895
SpreadProfile table 899
SubstituteGroupType table 901
SupplyAllocationType table 903
SupplyStatus table 904
SupplyType table 919
TaskType table 925
ToleranceProfile table 930
ToleranceProfileZone table 930
UnitOfMeasure table 931
WorkCenterType table 933

Caution: Because changes to control table settings can causes significant changes in the data generated by RapidResponse calculations (across all sites), you should exercise caution when working with control tables after the initial RapidResponse integration.

AggregatePartCustomerType table

The **AggregatePartCustomerType** table is used to enable or disable records in the **AggregatePartCustomer** table for processing when determining disaggregation rates.

Table 7.1: AggregatePartCustomerType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this **AggregatePartCustomerType** record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this type.	String	
ProcessingRule	Indicates whether **AggregatePartCustomer** records that belong to this type are used in generating disaggregation rates. Valid values are: • **Ignore**—Aggregate part customers of this type are ignored during forecast disaggregation. That is, disaggregation is performed for detailed or individual part and customer combinations. • **Use**—Aggregate part customers of this type are considered during forecast disaggregation. That is, disaggregation is performed at the specified aggregate part-customer level.	String (Enum)	
Value	A unique string identifier for this type.	String	Yes
AggregatePart Customers	Set of **AggregatePartCustomer** records that belong to this type.	Set	

AllotmentOverrideType table

The **AllotmentOverrideType** table is referenced by the **AllotmentOverride** table and used to control processing of allotment overrides in RapidResponse calculations. Allotment overrides refer to specified quantities of lower-level components that are manually allotted to top-level supplies or demands. For example, allotment overrides might be used to manually allocate supply in cases where RapidResponse analytic calculations cannot easily achieve the desired allocation of limited components.

Table 7.2: AllotmentOverrideType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this AllotmentOverrideType record. **Referenced table:** ControlSet	Reference	Yes
Description	Additional information describing this AllotmentOverrideType record.	String	

Table 7.2: AllotmentOverrideType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Controls whether AllotmentOverride records, and their associated AllotmentOverrideDetails values, are enabled or disabled in RapidResponse calculations. Valid values are: • **Always**—AllotmentOverride records of this type are always enabled. • **AfterRunDate**—AllotmentOverrideDetails records associated with AllotmentOverride records of this type are enabled or disabled based on date. If AllotmentOverrideDetail.Date is on or after RunDate, that record is enabled. Otherwise, if AllotmentOverrideDetails.Date is before RunDate, that record is disabled. • **Ignore**—AllotmentOverride records of this type are always disabled.	String (Enum)	
Value	A unique identifier for this AllotmentOverrideType record. This field is referenced from the **Type** field on the **AllotmentOverride** table.	String	Yes
Allotment Overrides	Set of **AllotmentOverride** records that reference this AllotmentOverrideType value.	Set	

AlternatePartType table

This table is referenced by the AlternatePart table, and is used to indicate if a PrimaryPart and AlternatePart combination represents an interchangeable part relationship or a substitute part relationship. It also contains several settings specific to substitute part calculations.

Table 7.3: AlternatePartType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this AlternatePartType record. **Referenced table**: ControlSet	Reference	Yes
Description	Additional information describing this AlternatePartType value.	String	

Table 7.3: AlternatePartType (Mfg) fields

Field name	Description	Data type	Key
EffectivityRule	Determines how the EffectiveInDate and EffectiveOutDate fields on the AlternatePart record are used in substitute part calculations. Valid values are: • **Always**—the AlternatePart record is always effective. • **Exclusive**—the AlternatePart record is effective starting after the EffectiveInDate and ending before the EffectiveOutDate. • **Inclusive**—the AlternatePart record is effective starting on the EffectiveInDate and ending on the EffectiveOutDate. • **InclusiveExclusive**—the AlternatePart record is effective starting on the EffectiveInDate and ending before the EffectiveOutDate. • **Never**—the AlternatePart record is never effective. **Default value:** InclusiveExclusive **Note:** If the AlternatePart record defines an interchangeable part relationship, then it is always considered effective (regardless of the setting in this field or the EffectiveInDate/EffectiveOutDate value.	String (Enum)	
PoolRule	Indicates if the Pool value on the AlternatePart record is used in substitute part calculations. Valid values are: • **Ignore**—the Pool value on the AlternatePart table is ignored. • **Use**—the Pool value on the AlternatePart table is used (for customer-specific part substitution). **Default value**: Ignore	String (Enum)	

Table 7.3: AlternatePartType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Controls whether the pair of parts in the AlternatePart record have an interchangeable part relationship or a substitute part relationship. Valid values are: • **Ignore**—the part relationship is ignored. • **Interchangeable**—the parts are considered equivalent in fit, form, and function. The PrimaryPart and SubstitutePart are interchangeable, and demand for one can be satisfied by supply of the other. No preference is given as to which part's supply satisfies a demand. All planned orders and calculated results in an interchangeable relationship are generated for the **PrimaryPart** only. However, in order to be used in an interchangeable relationship, the **SubstitutePart** still requires at least one effective PartSource record (can include sources whose **OrderPolicy.OrderGenerationRule** is set to "NoOrders"). • **Substitute**—one part is considered a substitute for the other. Demand for the PrimaryPart can be satisfied by the SubstitutePart, however demand for the SubstitutePart cannot be satisfied by the PrimaryPart. Whenever possible, demand for the PrimaryPart will be satisfied by its own supply (that is supply of the SubstitutePart is only used when not enough of the PrimaryPart is available). However, planned orders and calculated results in a substitute relationship can be generated for both the PrimaryPart and SubstitutePart. **Default value**: Interchangeable	String (Enum)	
Value	A unique string identifier for this AlternatePartType record.	String	Yes
AlternateParts	Set of **AlternatePart** records that reference this AlternatePartType value.	Set	

AnalyticsConfiguration table

The **AnalyticsConfiguration** table is used to set global, scenario-based parameters that primarily apply to multi-echelon safety stock calculations. The **Inventory Planning Settings** worksheet in the **Inventory Planning** workbook included with RapidResponse can be used to edit values this table.

Because the underlying analytic uses an iterative approach to determine optimal safety stock levels, multiple iterations through a family of related parts can be expected in order to determine the best solution. Records in this table thus allow for practical limits to be placed on how long the multi-echelon analytic can potentially be run in search of the optimal solution (large families of parts typically take longer amounts of time for the analytic to process). If this table does not contain a record in a given scenario, then multi-echelon calculations within that scenario will run with values set to 500 iterations for **MEIOMaximumIterations** and 5 minutes for **MEIOMaximumTime**.

Table 7.4: AnalyticsConfiguration (Mfg) fields

Field name	Description	Data type	Key
MEIOAllowSafetyStockOnArtificialStages	When this field is set to "N" or there are records in the **SafetyStockTimePhasedBounds** table, safety stock is not allowed on artificial stages during multi-echelon inventory optimization. When this field is set to "Y" and there are no records in the **SafetyStockTimePhasedBounds** table, safety stock is allowed on artificial stages during multi-echelon inventory optimization. This might result in slightly lower safety stock recommendations. To edit the value of this field using the **Inventory Planning Settings** worksheet, all records must first be removed from the **SafetyStockTimePhasedBounds** table in the active scenario. **Default value:** "N" **Note:** This field was added in Version 2016.2.	Boolean	
MEIOFutureDemandInterval Count	Number of calendar intervals to collect future demands and calculate average demands for use in calculations reported in the **SafetyStockResult** and **SafetyStockTimePhasedResult** table. Note that these tables report calculated results from both single-echelon planning and multi-echelon inventory optimization calculations. **Note:** This field was added in Version 2014.4	Integer	
MEIOMaximum Iterations	Maximum number of iterations the analytic can make through a network of parts to determine the optimal levels of safety stock to hold across all parts in the network. If the best solution is found before the number of iterations specified, the analytic stops regardless. **Note:** Significant improvements in results are not typically found beyond 500 iterations.	Integer	
MEIOMaximum Time	Maximum amount of time (in minutes) the analytic can spend iterating through a network of parts to determine the optimal levels of safety stock to hold across all parts in the network. If the best solution is found before the number of minutes specified, the analytic stops regardless.	Integer	

Table 7.4: AnalyticsConfiguration (Mfg) fields

Field name	Description	Data type	Key
MEIOReport Calendar	If generating time-phased safety stock calculations, this specifies the calendar containing the dates on which safety stock results are generated. For example, if set to a Month calendar then safety stock can only be calculated on dates marking the beginning of a Monthly calendar (if no calendar is specified then an Everyday calendar is assumed). Note that the number of date markers in the selected calendar can impact the processing time taken to generate time-phased results. For example, it would take less time to generate results for each month than it would take to generate results for each day. **Referenced table:** Calendar (nullable)	Reference	
MEIOReport IntervalCount	Number of calendar intervals to report calculated results in the SafetyStockTimePhasedResult table. Note that this field reports calculated results from both single-echelon planning and multi-echelon inventory optimization calculations. **Note:** This field was added in Version 2014.4	Integer	
MLSTrialLimitPer Demand	The maximum number of trials that the multi-level search analytic can perform in search of the best path to satisfy a demand. **Default value**: 50000	Integer	

AttributeMapType table

The **AttributeMapType** table is referenced by the **AttributeMap** table and defines how the attribute rules on that table are interpreted and processed. Each record in this table can be configured to enable or disable attribute rules for use in supply allocation, forecast consumption, and/or forecast adjustment calculations.

This table is not shown in the *RapidResponse Data Model for Import* poster.

Table 7.5: AttributeMapType (Core) fields

Field name	Description	Data type	Key
Description	A description of this attribute map type.	String	
Forecast Adjustment	Specifies whether AttributeMap records of this type are considered when specifying adjustments or overrides on ForecastDetail records used in generating the S&OP consensus forecast. For example, adjustments or overrides might only be applied within a specific forecast category for a given part customer. Valid values are • **N**—AttributeMap records of this type are ignored when making adjustments or overrides on ForecastDetail records. • **Y**—AttributeMap records of this type are considered when making adjustments/overrides on ForecastDetail records. Typically, if using sales and operations planning in RapidResponse and either or both of the **ForecastConsumption** or **SupplySelection** fields are set to "Y", then this field should be set to "Y" as well. This ensures adjustments are made within matching combinations of attributes on **ForecastDetail** values and not mixed together. **Default value:** N **Caution:** If the same demand attribute is defined in two or more **AttributeMap** rules that reference **AttributeMapType** records with different values in thisfield, then all of those rules are treated as though **ForecastAdjustment** is set to "Y" (that is, a "Y" setting overrides any "N" settings).	Boolean	

Table 7.5: AttributeMapType (Core) fields

Field name	Description	Data type	Key
Forecast Consumption	Specifies whether AttributeMap records of this type are considered during the forecast consumption process. When attribute-based forecast consumption is enabled, then in order to consume a specific forecast, all enabled attribute values on an actual demand must match the corresponding attribute values on the forecast demand. For example, a given customers forecast might only be eligible to be consumed by actual demands from that same customer. Valid values are • **N**—AttributeMap records of this type are ignored during forecast consumption. • **Y**—AttributeMap records of this type are considered during forecast consumption. **Default value:** N **Note:** When this field is set to "Y", then the **MUEPoolNettingType**.**AttributeFlowToCommon** field can be used to set whether actual demands with specific attribute values can, after consuming all forecast demands with matching attribute values, begin to consume forecasts that do not have any attribute values defined or enabled.	Boolean	
SupplySelection	Specifies whether AttributeMap records of this type are considered when allocating supplies to demands. If attribute-based supply allotment is enabled, then only specific on hand inventory, scheduled receipts, and part sources might be eligible for use in satisfying particular independent, consensus forecast, and/or safety stock demands. For example, a given customer's demands might only accept supplies sourced from particular locations or manufacturing facilities. Valid values are • **N**—AttributeMap records of this type are ignored when allocating supply to demand. • **Y**—AttributeMap records of this type are considered when allocating supply to demand. **Default value**: N	Boolean	
Value	A unique identifier for this **AttributeMapType** record. This field is referenced from the **Type** field on the **AttributeMap** table.	String	Yes
AttributeMaps	Set of **AttributeMap** records that reference this **AttributeMapType** value.	Set	

AutomatedLeadTimeType table

The **AutomatedLeadTimeType** table contains control settings that define how the lead time corrections projected by the Machine Learning server are processed for **ItemSource** records of a

particular type.

For example, the zone threshold fields (**NearToleranceThreshold** and **OutsideToleranceThreshold**) determine which tolerance zone the lead time deviations for an **ItemSource** belong to (whether the **ItemSource** is defined as "inside tolerance", "near tolerance", or "outside tolerance"). Based on the lead time tolerance zones defined by the zone thresholds, the zone policy fields (**InsideToleranceAction**, **NearToleranceAction**, and **OutsideToleranceAction**) control whether the projected lead times are meant to be automatically accepted, manually reviewed, or ignored.

For more information about how lead time deviations are calculated, see "Self-Healing Supply Chain calculations" on page 2146.

This table was added in Version 2016.2, Service Update 21.

Table 7.6: AutomatedLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
HistoricalLeadTimeErrorMeasure	The error measure to report in **ItemSource.CalcHistoricalLeadTimeError**. Note that this field determines the units of the **NearToleranceThreshold** and **OutsideToleranceThreshold** fields. Valid values are: • **MeanAbsoluteDeviation** —Determines the absolute value of variance between the historical supply actual values and the "as designed" forecast values, expressed as **LeadTimeUnit**. The calculation is represented by the following equation: $\mathbf{MAD} = \frac{1}{n} \sum_{i=1}^{n} \lvert \mathbf{ActualLeadTime}_i - \mathbf{PartSourceLeadTime}_i \rvert$ • **MeanAbsolutePercentageError** —Determines the absolute value of variance between the historical supply actual values and the "as designed" forecast values, expressed as a percentage. The calculation is represented by the following equation: $\mathbf{MAPE} = \frac{1}{n} \sum_{i=0}^{n} \frac{\lvert \mathbf{ActualLeadTime}_i - \mathbf{PartSourceLeadTime}_i \rvert}{\mathbf{ActualLeadTime}_i}$ where: ◦ n is the number of **ItemSourcePartSourceHistoricalLeadTime** records for a given **ItemSource**. ◦ $\mathbf{ActualLeadTime}_i$ is **ItemSourcePartSourceHistoricalLeadTime.ActualLeadTime**. ◦ $\mathbf{PartSourceLeadTime}_i$ is **ItemSourcePartSourceHistoricalLeadTime.PartSourceLeadTime**. **Default value:** MeanAbsoluteDeviation **Note:** If there are no historical supply actuals in the historical horizon, then **MeanAbsolutePercentageError** reports 0.	Enum	
InsideToleranceAction	The action to be taken when lead time deviation falls inside the defined **NearToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected lead times and copies the lead time changes to the **TimePhasedPartSourceLeadTime** table. • **Ignore** —Ignores the projected lead times. • **Review** —Flags the projected lead times for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** Ignore is the recommended setting.	Enum	

Table 7.6: AutomatedLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
NearToleranceAction	The action to be taken when lead time deviation falls in between the **NearToleranceThreshold** and **OutsideToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected lead times and copies the lead time changes to the **TimePhasedPartSourceLeadTime** table. • **Ignore** —Ignores the projected lead times. • **Review** —Flags the projected lead times for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** AutoAccept is the recommended setting.	Enum	
NearToleranceThreshold	The tolerance threshold that defines the tolerance zones for the lead time deviations. The value for this threshold is defined by the **HistoricalLeadTimeErrorMeasure**. If the lead time deviation (**ItemSource.CalcHistoricalLeadTimeError**) is less than the **NearToleranceThreshold**, then the **ItemSource** is considered to be "inside tolerance". In this case, the **InsideToleranceAction** is used. If the lead time deviation is more than or equal to the **NearToleranceThreshold** but less than the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "near tolerance". In this case, the **NearToleranceAction** is used. **Note:** Only positive number values should be used for the threshold settings (values less than zero are treated as zero). **Note:** If the **NearToleranceThreshold** is more than the **OutsideToleranceThreshold** , then **ItemSource.CalcLeadTimeZone** = "None". In this case, the cell for the **OutsideToleranceThreshold** will be conditionally formatted with a yellow background and bold red text in the Automated Lead Time Type worksheet of the Control Tables workbook.	Quantity	

Table 7.6: AutomatedLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
OutsideToleranceAction	The action to be taken when lead time deviation falls outside the defined **OutsideToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected lead times and copies the lead time changes to the **TimePhasedPartSourceLeadTime** table. • **Ignore** —Ignores the projected lead times. • **Review** —Flags the projected lead times for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** Review is the recommended setting.	Enum	
OutsideToleranceThreshold	The tolerance threshold that defines the tolerance zones for the lead time deviations. The value for this threshold is defined by the **HistoricalLeadTimeErrorMeasure**. If the lead time deviation (**ItemSource.CalcHistoricalLeadTimeError**) is more than or equal to the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "outside tolerance". In this case, the **OutsideToleranceAction** is used. **Note:** Only positive number values should be used for the threshold settings (values less than zero are treated as zero). **Note:** If the **OutsideToleranceThreshold** is less than the **NearToleranceThreshold**, then **ItemSource.CalcLeadTimeZone** = "None". In this case, the cell for the **OutsideToleranceThreshold** will be conditionally formatted with a yellow background and bold red text in the Automated Lead Time Type worksheet of the Control Tables workbook.	Quantity	
ProcessingRule	Specifies how to process the projected lead times for the **ItemSource**. You can use this setting to override the tolerance actions. Valid values are: • **Ignore** —Does not run the Machine Learning service. • **Manual** —Equivalent to setting all tolerance actions to "Review". • **UseZonePolicy** —Uses the settings specified for the **InsideToleranceAction**, **NearToleranceAction**, and **OutsideToleranceAction** fields. **Default value:** Ignore **Note:** UseZonePolicy is the recommended setting.	Enum	

Table 7.6: AutomatedLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
Value	A unique string identifier for the **AutomatedLeadTimeType** record. This value is referenced from the **ItemSource.AutomatedLeadTimeType** field.	String	Yes
ItemSources	Set of **ItemSource** records that reference this **AutomatedLeadTimeType**. **Referenced table:** "ItemSource table" on page 379	Set	

AutomatedYieldType table

The **AutomatedYieldType** table contains control settings that define how the yield corrections projected by the Machine Learning server are processed for **ItemSource** records of a particular type.

For example, the zone threshold fields (**NearToleranceThreshold** and **OutsideToleranceThreshold**) determine which tolerance zone the yield deviations for an **ItemSource** belong to (whether the **ItemSource** is defined as "inside tolerance", "near tolerance", or "outside tolerance"). Based on the yield tolerance zones defined by the zone thresholds, the zone policy fields (**InsideToleranceAction**, **NearToleranceAction**, and **OutsideToleranceAction**) control whether the projected yields are meant to be automatically accepted, manually reviewed, or ignored.

For more information about how yield deviations are calculated, see "Self-Healing Supply Chain calculations" on page 2146.

Table 7.7: AutomatedYieldType (Mfg) fields

Field name	Description	Data type	Key
HistoricalYieldErrorMeasure	The error measure to report in **ItemSource.CalcHistoricalYieldError**. Note that this field determines the units of the **NearToleranceThreshold** and **OutsideToleranceThreshold** fields. Valid values are: • **MeanAbsoluteDeviation** —Determines the absolute value of variance between the historical receipts values and the "as designed" forecast values, expressed as **YieldUnit**. The calculation is represented by the following equation: $\mathrm{MAD} = \frac{1}{n} \sum_{i=1}^{n} \lvert \mathrm{ActualYield}_i - \mathrm{PartSourceYield}_i \rvert$ • **MeanAbsolutePercentageError** —Determines the absolute value of variance between the historical receipts values and the "as designed" forecast values, expressed as a percentage. The calculation is represented by the following equation: $\mathrm{MAPE} = \frac{1}{n} \sum_{i=0}^{n} \frac{\lvert \mathrm{ActualYield}_i - \mathrm{PartSourceYield}_i \rvert}{\mathrm{ActualYield}_i}$ where: ◦ n is the number of **ItemSourcePartSourceHistoricalReceipt** records for a given **ItemSource**. ◦ $\mathrm{ActualYield}_i$ is **ItemSourcePartSourceHistoricalReceipt.EffectiveHistoricalYield**. ◦ $\mathrm{PartSourceYield}_i$ is **ItemSourcePartSourceHistoricalReceipt.PartSourceYield**. **Default value:** MeanAbsoluteDeviation **Note:** If there are no historical supply actuals in the historical horizon, then **MeanAbsolutePercentageError** reports 0.	Enum	
InsideToleranceAction	The action to be taken when yield deviation falls inside the defined **NearToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected yields and copies the yield changes to the **TimePhasedPartSourceYield** table. • **Ignore** —Ignores the projected yields. • **Review** —Flags the projected yields for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** Ignore is the recommended setting.	Enum	

Table 7.7: AutomatedYieldType (Mfg) fields

Field name	Description	Data type	Key
NearToleranceAction	The action to be taken when yield deviation falls in between the **NearToleranceThreshold** and **OutsideToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected yields and copies the yield changes to the **TimePhasedPartSourceYield** table. • **Ignore** —Ignores the projected yields. • **Review** —Flags the projected yields for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** AutoAccept is the recommended setting.	Enum	
NearToleranceThreshold	The tolerance threshold that defines the tolerance zones for the yield deviations. The value for this threshold is defined by the **HistoricalYieldErrorMeasure**. If the yield deviation (**ItemSource.CalcHistoricalYieldError**) is less than the **NearToleranceThreshold**, then the **ItemSource** is considered to be "inside tolerance". In this case, the **InsideToleranceAction** is used. If the yield deviation is more than or equal to the **NearToleranceThreshold** but less than the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "near tolerance". In this case, the **NearToleranceAction** is used. **Note:** Only positive number values should be used for the threshold settings (values less than zero are treated as zero). **Note:** If the **NearToleranceThreshold** is more than the **OutsideToleranceThreshold** , then **ItemSource.CalcYieldZone** = "None". In this case, the cell for the **OutsideToleranceThreshold** will be conditionally formatted with a yellow background and bold red text in the Automated Yield Type worksheet of the Control Tables workbook.	Quantity	
OutsideToleranceAction	The action to be taken when yield deviation falls outside the defined **OutsideToleranceThreshold**. Valid values are: • **AutoAccept** —Automatically accepts the projected yields and copies the yield changes to the **TimePhasedPartSourceYield** table. • **Ignore** —Ignores the projected yields. • **Review** —Flags the projected yields for manual review to determine whether to ignore or accept the suggestions. **Default value:** Ignore **Note:** Review is the recommended setting.	Enum	

Table 7.7: AutomatedYieldType (Mfg) fields

Field name	Description	Data type	Key
OutsideToleranceThreshold	The tolerance threshold that defines the tolerance zones for the yield deviations. The value for this threshold is defined by the **HistoricalYieldErrorMeasure**. If the yield deviation (**ItemSource.CalcHistoricalYieldError**) is more than or equal to the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "outside tolerance". In this case, the **OutsideToleranceAction** is used. **Note:** Only positive number values should be used for the threshold settings (values less than zero are treated as zero). **Note:** If the **OutsideToleranceThreshold** is less than the **NearToleranceThreshold**, then **ItemSource.CalcYieldZone** = "None". In this case, the cell for the **OutsideToleranceThreshold** will be conditionally formatted with a yellow background and bold red text in the Automated Yield Type worksheet of the Control Tables workbook.	Quantity	
ProcessingRule	Specifies how to process the projected yields for the **ItemSource**. You can use this setting to override the tolerance actions. Valid values are: • **Ignore** —Does not run the Machine Learning service. • **Manual** —Equivalent to setting all tolerance actions to "Review". • **UseZonePolicy** —Uses the settings specified for the **InsideToleranceAction**, **NearToleranceAction**, and **OutsideToleranceAction** fields. **Default value:** Ignore **Note:** UseZonePolicy is the recommended setting.	Enum	
TimePhasedMinCOV	Determines whether the yield projected by the Machine Learning server in **ProjectedYield** is categorized as time-phased. If **TimePhasedMinCOV** is less than or equal to **ItemSource.CalcProjectedYieldCOV**, then **TimePhasedYield** is used instead of **PartSource.Yield**.	Quantity	
Value	A unique string identifier for the **AutomatedYieldType** record. This value is referenced from the **ItemSource.AutomatedYieldType** field.	String	Yes

AvailableRule table

The **AvailableRule** table is used to control how available dates for a part's supplies and demands are calculated. By default, available dates are determined using all applicable capable-to-promise rules. However, these rules can be ignored for parts that are considered unimportant in the calculation of availability (such as, vendor managed parts).

Note that values in this table are only used by parts that have their **Part.AvailableRule** reference field populated. Otherwise, if **Part.AvailableRule** is left null, a part's availability rule comes from **PartType.AvailableRule**.

Table 7.8: AvailableRule (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this AvailableRule record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this control setting.	String	
ProcessingRule	Specifies how the availability of a part's supplies and demands is calculated. Valid values are: • **FromType**—Available dates for supplies and demands of the part are calculated using the setting in **PartType.AvailableRule** (either "Ignore" or "Normal"). • **Ignore**—available date is calculated for each supply, but requirements are treated as if the dependent and independent demands had been covered by on hand supplies. This means that dependent requirements onto these parts are available at Past and independent requirements are available at DataDate. Typically, this setting is used for parts considered unimportant to the calculation of availability (for example, vendor managed parts or floor stock items). • **Normal**—Available dates for supplies and demands of the part are calculated using all applicable capable-to-promise rules and settings. **Default value:** FromType	String (Enum)	
Value	A unique string identifier for this control setting.	String	Yes
Parts	Set of **Part** records that reference this **AvailableRule** value.	Set	

BOMType table

The **BOMType** table is a control table that specifies the different types of bill records and the processing rules for them. The **BOMType.Value** is a text string assigned to each parent-child record in the BillOfMaterial table. Each **BOMType.Value** defined represents a set of configuration rules that control the behavior for that parent-child relationship.

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
BlowThrough	Specifies whether to use phantom BOM relationships or "Blow Through" bill logic. Dependent demand generated through this Bill record bypasses the netting of the component. Valid values are: • Y • N **Default value**: N **Note:** This setting is ignored (always treated as "N") on BOMType records used to define co-product or by-product configurations.	Boolean	
ControlSet	The control set associated with this BOMType record. **Referenced table:** ControlSet	Reference	Yes

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
DateRule	When determining bill date effectivity, this field specifies which date to test for. Valid values are: • **AnchorDate**—the supply record's anchor date (**ScheduledReceipt.AnchorDate**) is used to determine which components of the BillOfMaterial are effective for explosion purposes. • **BlowThroughCalcDueDate**— BillOfMaterial effectivity is determined using either the ScheduledReceipt.ReschedDate or PlannedOrder.DueDate of the real assembly. Unless the assembly is a phantom, this option is identical to CalcDueDate. • **CalcDueDate**—the **FirstEffectiveDate** and **LastEffectiveDate** values on the BillOfMaterial record are compared using the **ScheduledReceipt.ReschedDate** or **PlannedOrder.DueDate** to determine which BillOfMaterial records will have dependent demand generated on them. • **CalcDockDate**—a supply order's dependent demands are based on the components that are effective at the supply order's calculated dock date. That is, the **FirstEffectiveDate** and **LastEffectiveDate** values on the BillOfMaterial records are compared using either **ScheduledReceipt.ReschedDockDate** or **PlannedOrder.DockDate**. • **CalcStartDate**—the supply record's calculated start date (**ScheduledReceipt.CalcStartDate**) is used to determine which components of the BillOfMaterial are effective for explosion purposes. • **Ignore**—the supply record ignores the **BillOfMaterial.EffectiveInDate** and **BillOfMaterial.EffectiveOutDate** resulting in the component always being considered for explosion. **Default value:** Ignore	String (Enum)	
DependentDemand Model	Specifies the model to use when the parent model should not be used in order BOMs. When set to null, the model of the dependent demand is identical to the model of the parent. When this field is set to a model name, the referenced model is the model of the dependent demand. **Referenced table:** Model **Note:** This field was added in Version 2016.2.	Reference	

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
DependentUnit Rule	Specifies how dependent demands are handled with unit effectivity. If your company has not enabled Model-Unit Effectivity, this field is ignored. Valid values are: • **Split**—split dependent demand according to the unit number range. Demands are not split if the supply's unit is less than zero. • **First**—calculate the entire dependent demand according to StartUnit on the supply record. Do not split the order according to unit effectivity. **Default value**: Split **Note:** This setting is ignored on BOMType records that define co-product or by-product configuration.	String (Enum)	
Description	A text description of the BOMType.Value. **Default value**: blank string	String	
EffectivityRule	Specifies for each BOMType how ranges of units and dates are interpreted when determining effectivity. This field applies to all effectivity rules (ranges apply to dates and unit ranges). Valid values are: • **InclusiveExclusive**—the record is effective starting on start and ending before the end (recommended method). • **Never**—the record is ignored regardless of all dates and Model and Unit Effectivity rules. • **Always**—ignore all other effectivity rules and treat Bill as active. • **Exclusive**—use effectivity dates and units if applicable (not effective on the dates or units). • **Inclusive**—effective from start to end inclusive. **Default value**: InclusiveExclusive	String (Enum)	
ExplodeNegatives	Specifies how negative quantities per each of the BOMTypes are handled. Valid values are: • **Y**—negative QuantityPer is used in the explosion calculation. • **N**—negative QuantityPer is not used in the explosion calculation. **Default value:** N **Note:** This setting is ignored on BOMType records that define a co-product or by-product configuration (always treated as "Y" since QuantityPer values are required to be negative on all BillOfMaterial records that define co-products or by-products).	Boolean	

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
IgnoreAssembly Yield	Controls whether or not an assembly's yield is ignored from one component to another when calculating dependent demand quantities (that is, is the dependent demand quantity inflated for the assembly's yield or not). Valid values are: • **Y**—the component from the BOM record ignores the assembly's yield when calculating the dependent demand quantity (the quantity is not inflated in consideration of the assembly's yield). For example, if an assembly containing expensive components needs to be scrapped due to issues with another component, it might be desirable for the expensive components to be recovered from the assembly before scrapping it. • **N**—the component from the BOM record considers the assembly's yield when calculating the dependent demand quantity (the quantity is inflated in consideration of the assembly's yield). **Default value**: N **Note:** This setting is ignored on BOMType records that define a co-product or by-product configuration (always treated as "Y").	Boolean	
LimitEffective Date	Specifies for each **BOMType.Value** where an assembly has a past due demand whether effectivity of components is determined using the current DueDate (considering that it is prior to the **Assembly.RunDate**) or the RunDate itself. • **Y**—the **Assembly.RunDate** is used. This ensures that any past due demand uses BillOfMaterial records currently in effect (as of the RunDate). This setting overrides the explosion date (StartDate of the peg supply). As a result, bills not effective at RunDate are ignored (supply does not explode through them). • **N**—the **DueDate** is used. This ensures that the BillOfMaterial configuration at the time of the DueDate will be used. **Default value**: N	Boolean	

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
MPSConfig DemandSource	Specifies which components on the BillOfMaterial table are considered when looking for component actual sales to consume assembly forecast within a PlanningBOM relationship. Four conditions must be met for this to occur: • Assembly part has **PartType.ProcessingRule** set to 'MPSConfig' • The BillOfMaterial record defining the relationship between Assemblies and Components must have a **BOMType.MPSConfigDemandSource**='Y'. • The Component has independent demand records where **DemandType.ProcessingRule**='SalesActual' • The Assembly has independent demand records where **DemandType.ProcessingRule** = 'SalesForecast' or 'ProductionForecast'. Valid values are: • **Y**—this BOMType uses component consumption of assembly forecast within a PlanningBOM. • **N**—this BOMType does not use component consumption of assembly forecast within a PlanningBOM. **Default value**: N **Note:** This setting is ignored on all BOMType records that define a co-product or by-product configuration.	Boolean	
MUERule	Specifies how model and unit effectivity are to be applied. MUERule is used regardless of whether the parent or component parts are netted by model or unit. If Model-Unit Effectivity is not enabled then MUERule is treated as Ignore. Valid values are: • **Model**—the Bill record is effective if the Model value on the Bill record matches the **ScheduledReceipt.InputModel** or the model on the planned order. • **Unit**—the Bill record is effective if the StartUnit on the Supply record is in the range of the StartUnit and EndUnit from the Bill record. DependentUnitRule and EffectivityRule are tested to determine whether the unit is in range. • **ModelUnit**—use both model and unit tests simultaneously. Model matches and the unit must be in range for the Bill record to be effective. • **Ignore**—do not test model or unit for this Bill record. **Default value**: Ignore	String (Enum)	

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
ProductType	Specifies the component's relationship to the assembly. This setting determines whether dependent requirements are passed to the component, as well as how certain values on the BOM record are interpreted. Valid values are: • **Normal**—a normal dependent component used in the production of the assembly. • **CoProduct**—a co-product that can be manufactured and planned together with the assembly (primary) product due to common structure, components and process similarities. For example, a co-product might be a lower grade version of the primary product. Netting and CTP calculations for parts with a co-product relationship are performed at the same time. Therefore, two-way co-product relationships are supported. • **ByProduct**—a by-product that is generated from, and residual to, the production of the assembly (primary) product. For example, a by-product might be a chemical generated from the production of one or more assemblies in a given product structure. Netting and CTP calculations for a by-product part are performed after all of its BillOfMaterial assemblies have been planned. Therefore, two-way by-product relationships are not supported. For more information about co-products or by-products, see "Co-products and by-products" on page 1581. **Default value:** Normal	String (Enum)	
RatioRule	Specifies how the BillOfMaterial.OptionRatio is used. Valid values are: • **Ignore**—do not use the OptionRatio field in explosion • **Fraction**—value from 0 through 1 (1 means used in all assemblies) • **Percent**—value from 0 through 100 (100 means used in all assemblies) **Default value:** Fraction	String (Enum)	

Table 7.9: BOMType (Mfg) fields

Field name	Description	Data type	Key
RoundingRule	Specifies for each BOMType the rules for rounding numbers when exploding supplies through the BOM to place dependent demands on components. Valid values are: • **None**—do not round. • **Up**—round up to next integer. • **Down**—round down to next integer. • **Nearest**—round to nearest integer (.5 goes up). • **QtyPerOrder**—ignores the supply quantity on the order and pass dependent demand equal to the **BillOfMaterial.QuantityPer** value. For example, if the supply quantity is 10 and QuantityPer is 2, then the dependent demand will be 2. **Default value:** None	String (Enum)	
ScrapRule	Specifies how to handle scrap and yield values on BOMType records. Valid values are: • **Ignore**—do not use the Scrap factor • **YieldFraction**—value from 0 through 1 (1 = no loss) • **YieldPercent**—value from 0 through 100 (100 =no loss) • **ScrapFraction**—value from 0 through 1 (0 = no loss) • **ScrapPercent**—value from 0 through 100 (0 = no loss) • **InflationFraction**—formula = 1 / (1 + **BillOfMaterial.Scrap**) • **InflationPercent**—formula = 1 / (1 + **BillOfMaterial.Scrap** * 0.01) **Default value**: YieldFraction	String (Enum)	
UseLeadTime Offset	Specifies for each BOMType whether to use a separate lead-time factor on BillOfMaterial records. Valid values are: • **Y**—the lead-time value that is provided is used in all lead-time calculations. Setting this value to Y enables the use of the BillOfMaterial.LeadTimeOffset field as a factor in addition to Part and PartSource based lead-time elements such as FixedLeadTime, VariableLeadTime, and SafetyLeadTime, in calculating product lead time and start dates. • **N**—the lead-time value that is provided is not used in all lead time calculations. **Default value**: N	Boolean	
Value	The string value for this type of bill. This value appears in the BillOfMaterial.Type field. **Default value**: blank string	String	Yes
BillOfMaterials	Set of **BillOfMaterial** records that reference this BOMType value.	Set	

Calendar table

Calendars are used for date calculations. Common calendars used in manufacturing include shop calendars (workdays), and month calendars (defines beginning of monthly reporting periods). Each calendar has a name defined in the **Calendar** table. The calendar is defined by a list of dates associated with the calendar name.

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
BucketingName	An alternative name for this calendar (such as, the name of a supplier's internal calendar). This field is not used by RapidResponse analytics. **Default value**: blank string	String	
Description	A description of the calendar and the periods it represents.	String	
PlanningOnly	Used to exclude a calendar from appearing in the dialog boxes in which users choose worksheet bucket settings. This can be useful in eliminating calendars a user is not expected to see or to simply present a more manageable set of calendars. Note that regardless of this setting, all calendars are available for planning purposes (for example, for use by a PlanningCalendar record). Valid values are: • **N**—Calendar is available for planning purposes and is also available for selection by users when applying worksheet bucketing. This setting corresponds to how calendars always worked in versions prior to 2016.2. • **Y**—Calendar is available for planning purposes only. It will not show or be selectable in dialog boxes by users when choosing how to bucket data in worksheets. **Default value:** N **Note:** This field was added in Version 2016.2, Service Update 4.3.	Boolean	
Value	The name of a Calendar. A valid calendar name must begin with an alphabetic character, and can then include alphabetic, numeric, and underscore characters. No special characters are allowed. **Default value**: blank string	String	Yes
FirstDate	The earliest CalendarDate.Value for the calendar. This is a calculated field.	Date	

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
LastDate	The latest CalendarDate.Value for the calendar. This is a calculated field.	Date	
AsOfDateCalendar SafetyStockItem Types	Set of **SafetyStockItemType** records that use this calendar to define an "as of date".	Set	
ConstraintCalendar	Set of **Constraint** records using this Calendar when allocating constraint to load.	Set	
ConstraintDataDate	Set of **Constraint** records using this Calendar to define the current constraint period.	Set	
CycleCalendar SafetyStockItem Types	Set of **SafetyStockItemType** records that use this calendar as their cycle calendar.	Set	
DRPHorizonCalendars	Set of **DRPHorizon** records that reference this Calendar. This includes only records that are using the global default setting from the **DRPConfiguration** table.	Set	
DRPPartSiteHorizonCalendars	Set of **DRPHorizon** records that have that reference this Calendar. This includes only records that are using the settings from the **PartSolution** table.	Set	
Dates	Set of **CalendarDate** records that reference this Calendar.	Set	
Forecast Disaggregation Parameters InnerCalendars	Set of **ForecastDisaggregationParameters** records that use this Calendar as the inner calendar for forecast disaggregation.	Set	
Forecast Disaggregation Parameters OuterCalendars	Set of **ForecastDisaggregationParameters** records that use this Calendar as the outer calendar for forecast disaggregation.	Set	
ForecastItem ParametersTypes	Set of **ForecastItemParametersType** records that use this Calendar when bucketing actuals and generating the statistical forecast.	Set	
HistoricalSupply Headers	Set of **HistoricalSupplyHeader** records that reference this calendar.	Set	

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
LeadTimeCalendar SafetyStockItem Types	Set of **SafetyStockItemType** records that use this calendar as their lead time calendar.	Set	
MPSCalendars	Set of **MPSConfiguration** records that use this calendar for bucketing purposes.	Set	
MPSRunDate Calendars	Set of **MPSConfiguration** records that use this calendar to define the current system date.	Set	
MPSTolerance Calendars	Set of **MPSConfiguration** records that use this calendar to define the current system date.	Set	
OrderPriorities	Set of **OrderPriority** records that use this calendar.	Set	
PartCustomers	Set of **PartCustomer** records that use this calendar.	Set	
PeriodCalendar SafetyStockItem Types	Set of **SafetyStockItemType** records that use this calendar as their period calendar.	Set	
PlanningCalendars AllocationCalendar	Set of **PlanningCalendars** records that use this Calendar when performing fair and equal-share allocations.	Set	
PlanningCalendars CoByProduct PlanningInterval	Set of **PlanningCalendars** records that use this Calendar for defining co-product and by-product planning intervals.	Set	
PlanningCalendars DataDate	Set of **PlanningCalendars** records that use this Calendar for identifying when data was last updated.	Set	
PlanningCalendars ExpiryCalendars	Set of **PlanningCalendars** records that use this Calendar for material expiry planning.	Set	
PlanningCalendars Forecast	Set of **PlanningCalendars** records that use this Calendar for defining forecast intervals.	Set	
PlanningCalendars OrderPoint Increases	Set of **PlanningCalendars** records that use this Calendar for defining intervals used when calculating if an increase in the "order point" threshold should be applied.	Set	

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
PlanningCalendars PercentSafety Interval	Set of **PlanningCalendars** records that use this Calendar for bucketing in PercentOfDemand and FractionOfDemand safety stock calculations.	Set	
PlanningCalendars PeriodSupplyDue	Set of **PlanningCalendars** records that use this Calendar when setting valid planned order due dates for "days of supply" parts.	Set	
PlanningCalendars PeriodSupply Interval	Set of **PlanningCalendars** records that use this Calendar when accumulating intervals of demands for "days of supply" parts.	Set	
PlanningCalendars RunDate	Set of **PlanningCalendars** records that reference this Calendar when determining the date that data was extracted from the enterprise data source.	Set	
PlanningCalendars SecondCalendar	Set of **PlanningCalendars** records that use this Calendar for MPS and MPSConfig parts with forecast spreading.	Set	
PlanningCalendars Substitution ToleranceCalendar	Set of **PlanningCalendars** records that use this Calendar for part substitution calculations.	Set	
PlanningCalendars TimeFence	Set of **PlanningCalendars** records that use this Calendar for determining dates on which the demand time fence will fall.	Set	
PlanningCalendars TimeUnits	Set of **PlanningCalendars** records that use this Calendar to determine valid working dates for "units of time" parameters.	Set	
PlanningCalendars Weekly	Set of **PlanningCalendars** records that use this Calendar for certain weekly reporting calculations in RapidResponse.	Set	
ProjectRunDate	Set of **Project** records that use this Calendar for defining their run date.	Set	
ProjectType Calendars	Set of **ProjectType** records that use this Calendar for defining the project calendar.	Set	
ProjectType PenaltyCalendars	Set of **ProjectType** records that use this Calendar for defining the project penalty calendar.	Set	

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
RangeOfCoverage Average Requirement Intervals	Set of **RangeOfCoverage** records that use this Calendar for setting the intervals over which average daily demands are calculated.	Set	
RangeOfCoverage Intervals	Set of **RangeOfCoverage** records that use this Calendar for determining range of coverage zones.	Set	
ResourceTypes	Set of **ResourceType** records that use this Calendar for defining the resource calendar.	Set	
SafetyStockItem Types	Set of **SafetyStockItemType** records that use this calendar.	Set	
SafetyStockPolicies	Set of **SafetyStockPolicy** records that use this calendar.	Set	
ShipCalendar Sources	Set of **Source** records that use this Calendar when determining ship dates.	Set	
SOPBaseCalendars	Set of **SOPAnalyticsConfiguration** records that use this Calendar as the disaggregation calendar for forecast disaggregation.	Set	
SOPCycleCalendars	Set of **SOPAnalyticsConfiguration** records that use this Calendar to define the S&OP cycle.	Set	
SOPInnerCalendars	Set of **SOPAnalyticsConfiguration** records that use this Calendar as the inner calendar for forecast disaggregation.	Set	
SOPOuter Calendars	Set of **SOPAnalyticsConfiguration** records that use this Calendar as the outer calendar for forecast disaggregation.	Set	
SOPRunDate Calendars	Set of **SOPAnalyticsConfiguration** records that use this Calendar to define the current system date.	Set	
SourceLeadTime Units	Set of **Source** records that use this Calendar when determining start dates.	Set	
TaskType Calendars	Set of **TaskType** records that use this Calendar for defining the task calendar.	Set	

Table 7.10: Calendar (Core) fields

Field name	Description	Data type	Key
TaskType PenaltyCalendars	Set of **TaskType** records that use this Calendar for defining the task penalty calendar.	Set	
ToleranceProfile Baseline	Set of **ToleranceProfile** records that use this Calendar when identifying the baseline historical demand or supply series.	Set	
ToleranceProfile Tolerance	Set of **ToleranceProfile** records that use this Calendar when evaluating level of change.	Set	

Note: All calendar names must begin with an alphabetic character.

CalendarDate table

The **CalendarDate** table contains a list of dates that can be used in calendar tables. The dates are loaded from data located in the enterprise data source.

Note that the earliest date allowed on a **CalendarDate** record is `01-02-1970` and earlier dates are always taken to mean "Past". Similarly, the latest date allowed on a **CalendarDate** record is `12-31-2037` and later dates are always taken to mean "Future".

Table 7.11: CalendarDate (Core) fields

Field name	Description	Data type	Key
Calendar	Reference to the calendar name. This field references the Calendar table. For information about this table, see "Calendar table" on page 695. **Referenced table:** Calendar	Reference	
DisplayValue	A label that can be displayed in worksheet buckets in place of a period's Value.	String	
Value	A date that defines the beginning of a period in the named calendar. **Default value**: Undefined	Date	

ConstraintType table

This table supports the Constraint Manager application, and defines rules that control how each constraint is processed.

Table 7.12: ConstraintType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this ConstraintType record. **Referenced table:** ControlSet	Reference	Yes
Description	Additional information about this Type.	String	
EffectivityRule	Specifies how ConstraintAvailable effectivity dates are interpreted. Valid values are: • **InclusiveExclusive**—records are effective starting with Constraint period including In date and ending before the period containing Out date (this is the recommended method and the default). • **Always**—ignore effectivity dates and treat all ConstraintAvailable record as effective. • **Exclusive**—effective from the period after In to the period before Out. • **Inclusive**—effective from the period containing In to the period containing Out inclusive. • **Never**—no ConstraintAvailable records are effective. **Note:** Always only affects date effectivity, not CumulativeMax calculations.	String (Enum)	
FillSchedule	Controls whether netting tries to fill the constraint's schedule by planning to use constraint as early as possible, or if it only plans enough to satisfy demand dates. Valid values: • **Y**—Netting tries to fill constraint schedule to use constraint as early as possible. For "constrained" and "load only" constraints, Netting will pull orders earlier in to fill the constraint schedule and CTP will use the results for availability and constraint reporting. For "unconstrained" constraints, Netting does not try to fill in the constraint schedule , however if the **AvailableDateLimit** setting is "now", then CTP will schedule orders as early as possible with this setting. • **N**—Netting schedules constraint only to satisfy demand dates. This is the recommended value. Default: N If any constraint used by a part source is set to fill (Y), then all supplies using that part source will be planned to fill (regardless of whether their FillSchedule setting is Y or N). For more information, see "FillSchedule" on page 1665.	Boolean	

Table 7.12: ConstraintType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies how a constraint is applied. Valid values are: • **Constrained**—CumulativeMax and ConstraintAvailable limits are applied. • **CumMax**—load is calculated as if the Constraint has unlimited availability in every period. CumulativeMax calculations are effective for both order planning and available date calculations. • **LoadOnly**—load is calculated as if the Constraint has unlimited availability in every period. CumulativeMax and period ConstraintAvailable are ignored. • **PlanningOnly**—the Constraint is considered (treated as "Constrained") during netting calculations so that part source selection is made based on constraint availability. However, the Constraint is ignored (treated as "LoadOnly") during CTP available date calculations. • **Unconstrained**—the Constraint is never effective and is ignored from processing. This acts as if the Constraint had unlimited availability and Load is not calculated. • **UnconstrainedPlan**—the Constraint is ignored (treated as 'Unconstrained') during netting calculations so supplies are planned when needed based on material requirements only. However, the Constraint is considered (treated as 'Constrained') during Capable-to-Promise calculations so available dates reflect what is possible once constraints are taken into account. **Default:** Unconstrained **Note:** For more information about the effect of the ProcessingRule setting, see "ProcessingRule" on page 1652.	String (Enum)	
ReportLimit	The number of days from DataDate to generate **PeriodConstraintLoad** records. This value should be expressed in terms of the standard Gregorian (everyday) calendar. Constraint loads are reported beyond that in the **ConstraintLoad** and **ConstraintSpreadLoad** table , but no additional **PeriodConstraintLoad** records are produced.	Integer	

Table 7.12: ConstraintType (Mfg) fields

Field name	Description	Data type	Key
SchedulingRule	Specifies how constraint scheduling is performed during Netting calculations. Constraints are always first scheduled in a "backward" phase during which constrained supplies are matched to or planned for demands. If pre-planning is enabled and there is insufficient constraint available, then earlier periods are looked to first for additional constraint. By default, a subsequent "forward" phase is then applied to optimize the initial ordering of supplies. This phase can improve scheduling by ensuring high priority supplies are pre-planned first and ensuring related supplies/products are built in continuous blocks to reduce changeover costs. This setting allows disabling the forward phase which can produce a better schedule under certain conditions (such as, when there are parallel constraints). Valid values are: • **BackloadOnly**—Only the backward phase of constraint scheduling is performed (forward phase is disabled). If pre-planning is required when using this setting, low priority supplies will be pre-planned first. It is intended for cases where part sources are constrained by multiple different constraints (particularly if used with partial planning logi), and where supplies should be planned as close to the due date as possible. This setting is also recommended if multi-level search logic is heavily used. • **Normal**— Both the backward and forward phases of constraint scheduling are performed. If pre-planning is required when using this setting, high priority supplies will be pre-planned first. It is recommended for cases where supplies should use constraint in continuous blocks (such as, if there are high changeover costs) or where it is important to pre-plan higher priority supply first. This setting corresponds to how constraint scheduling was always performed prior to Service Update 1906. **Default value**: Normal **Note**: This field was added in Service Update 1906. For more information, see "SchedulingRule" on page 1667.	String (Enum)	
Value	Name for a particular ConstraintType.	String	Yes
Constraints	Set of **Constraint** records that reference this Constraint value.	Set	

ConstraintUnitOfMeasure table

This table supports the Constraint Manager application, and contains the units of measure names used by constraints.

Table 7.13: ConstraintUnitOfMeasure (Mfg) fields

Field name	Description	Data Type	Key
Constraints	Set of **Constraint** records that reference this ConstraintUnitOfMeasure value.	Set	
ControlSet	The control set associated with this ConstraintUnitOfMeasure record. **Referenced table:** ControlSet	Reference	Yes
Description	Additional information about this ConstraintUOM.	String	
UnitOfMeasure	The unit of measure associated with this ConstraintUnitOfMeasure record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
Value	A Unit of Measure that can be used by Constraints.	String	Yes

CRPUnitOfMeasure table

This table defines the different units of measurement used to interpret run times on an operation. It defines the calculation to convert the run time of an operation into standard hours of load.

Table 7.14: CRPUnitOfMeasure (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set that this CRPUnitOfMeasure record is associated with. **Referenced table:** ControlSet	Reference	Yes
CRPOperations	Set of **CRPOperation** records that reference this CRPUnitOfMeasure value.	Set	
Description	A description of this type of operation. An explanation of the code in the Value field. **Default value**: blank string	String	
Operations	Set of **Operation** records that reference this CRPUnitOfMeasure value.	Set	
ProcessingRule	• **UnitsPer Time** — uses the formula (1 / (RunTime * ScaleFactor) • **TimePerUnits** — uses the formula (RunTime * ScaleFactor)**Default value:** TimePerUnits	String (Enum)	

Table 7.14: CRPUnitOfMeasure (Mfg) fields

Field name	Description	Data type	Key
ScaleFactor	Specifies how to scale the value of the RunTime field on the Operation table. **Default value**: 1	Quantity	
UnitOfMeasure	The unit of measure that this CRPUnitOfMeasure record is associated with. **Referenced table:** UnitOfMeasure **Note**: This field was added in Version 2016.2, Service Update 9.	Reference	
Value	The string code for this type of operation. This value appears in the Operation records. **Default value**: blank string	String	Yes

DaysSupplyPolicy table

The **DaysSupplyPolicy** table contains settings that specify how demands should be accumulated and planned orders generated for parts that have days of supply logic enabled. Days of supply logic is meant to reduce the total number of planned orders generated by allowing a single planned order to satisfy all demands accumulated over a specified number of days or intervals. For example, all demands over each two-week period of demand might be satisfied by one planned order.

Each record in this table contains both short-term and long-term days of supply settings that apply to parts that reference it through the **Part.DaysSupplyPolicy** field. For example, demands might be accumulated weekly for the first two months in the horizon, and monthly after that. The **Fence** and **FenceCalendar** fields in this table are used to set the transition point between the short-term and long-term horizon for days of supply settings.

Note that for parts that only need one set of days of supply settings, then legacy days of supply settings as defined on the **Part, PartType**, and **PlanningCalendars** tables can be used instead. Alternatively for this case, the **DaysSupplyPolicy** table can still be used but with either the short-term horizon set large enough to cover the entire planning horizon, or with the short-term horizon set to "0" and the long-term horizon left to cover the entire planning horizon.

This table was added in Version 2016.2. For more information about days of supply logic, see "Days of Supply calculations" on page 1362.

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set that this **DaysSupplyPolicy** record is associated with. **Referenced table:** ControlSet	Reference	Yes

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
DaysSupplyRule	This setting determines how demands are accumulated and new orders generated within the short-term horizon for parts that use this days of supply policy. Valid values are: • **ByPeriod**—uses the values in **PeriodSupplyInterval** calendar, and over the number of intervals set by the **NumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). A single planned order to satisfy those demands is then generated on the first interval of the **PeriodSupplyDueCalendar** within the accumulation period. • **ByPeriodEnd**—demands are accumulated in intervals of the **PeriodSupplyInterval** calendar, and over the number of intervals set by the **NumberOfDaysSupply** field (starting from the due date of first unsatisfied demand). A single planned order is then generated on the last **PeriodSupplyDueCalendar** interval within the accumulation period. • **ByPeriodEndWithPriority**—demands of each priority level are accumulated separately in intervals of the **PeriodSupplyInterval** calendar, and over the number of intervals set by the **NumberOfDaysSupply** field (starting from the due date of first unsatisfied demand). For each group of prioritized demands, a single planned order is then generated on the last **PeriodSupplyDueCalendar** interval within the accumulation period. Note that the priority-based logic described above is only applicable to parts that have a **PartType.CommitLevel** setting of "High". For all other CommitLevel settings, this setting acts identically to the "ByPeriod" setting. • **ByPeriodWithPriority**—demands of each priority level are accumulated separately in intervals of the **PeriodSupplyInterval** calendar, and over the number of intervals set by the **NumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). For each group of prioritized demands, a single planned order is then generated on the first **PeriodSupplyDueCalendar** interval within the accumulation period. Note that the priority-based logic described above is only applicable to parts that have a **PartType.CommitLevel** setting of "High". For all other CommitLevel settings, this setting acts identically to the "ByPeriod" setting. • **FromDemand**—demands are accumulated in	String (Enum)	

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
	Part.PlanningCalendars.TimeUnits calendar intervals, and over the number of intervals set by the **NumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). A single planned order is then generated on the due date of the first unsatisfied demand within the collection interval. • **FromSupply**—similar to the "FromDemand" setting except that demands accumulate starting from the due date when the supply can be planned (instead of the due date of the unsatisfied demand). In cases where there is a past-due demand for a part using this setting, the supply due date will be later than the demand due date, and hence the days supply horizon will end later. In typical cases where the supply due date and demand due date are the same, this field acts identically to "FromDemand". • **Ignore**—Days of supply planning is disabled for parts that use this policy. **Default value**: FromDemand		
Description	A description of this days of supply policy.	String	
Fence	The number of periods after RunDate where the fence between short-term and long-term days of supply horizons is placed (the actual date is reported in **Part.DaysSupplyFenceDate**). Before the fence date, short-term days of supply settings are used, while on or after the fence date, long-term days of supply settings are used. This should be expressed in **FenceCalendar** intervals. For example, if FenceCalendar is "Week" and this value is "12", then DaysSupplyFenceDate is calculated 12 weeks out from RunDate. The short-term settings apply up to that calculated fence date and the long-term settings apply on and after that through to the end of the planning horizon.	Integer	
FenceCalendar	A reference to the calendar in which the Fence value is expressed. For example, Week or Month. **Referenced table**: Calendar	Reference	

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
LongTermDays SupplyRule	This setting determines how demands are accumulated and new orders generated within the long-term horizon for parts that use this days of supply policy. Valid values are: • **ByPeriod**—demands are accumulated in intervals of the **LongTermPeriodSupplyInterval** calendar, and over the number of intervals set by the **LongTermNumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). A single planned order to satisfy those demands is then generated on the first interval of the **LongTermPeriodSupplyDueCalendar** within the accumulation period. • **ByPeriodEnd**—demands are accumulated in **LongTermPeriodSupplyInterval** calendar intervals, and over the number of those intervals set by the **LongTermNumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). A single planned order is then generated on the last **LongTermPeriodSupplyDueCalendar** interval within the accumulation period. • **ByPeriodEndWithPriority**—demands of each priority level are accumulated separately in intervals of the **LongTermPeriodSupplyInterval** calendar, and over the number of intervals set by the **LongTermNumberOfDaysSupply** field (starting from the due date of first unsatisfied demand). For each group of prioritized demands, a single planned order is then generated on the last **LongTermPeriodSupplyDueCalendar** interval within the accumulation period. Note that the priority-based logic described above is only applicable to parts that have a **PartType.CommitLevel** setting of "High". For all other CommitLevel settings, this setting acts identically to the "ByPeriod" setting. • **ByPeriodWithPriority**—demands of each priority level are accumulated separately in **LongTermPeriodSupplyInterval** calendar intervals, and over the number of intervals set by the **LongTermNumberOfDaysSupply** field (starting from the due date of the first unsatisfied demand). For each group of prioritized demands, a single planned order is then generated on the first **LongTermPeriodSupplyDueCalendar** interval within the accumulation period. Note that the priority-based logic described above is only applicable to parts that have a **PartType.CommitLevel** setting of "High". For all other CommitLevel settings, this setting acts identically to the	String (Enum)	

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
	"ByPeriod" setting. • **FromDemand**—demands are accumulated in **Part.PlanningCalendars.TimeUnits** calendar intervals, and over the number of specified **LongTermNumberOfDaysSupply** intervals (starting from the due date of the first unsatisfied demand). A single planned order is then generated on the due date of the first unsatisfied demand within the accumulation period. • **FromSupply**—similar to the "FromDemand" setting except that demands accumulate starting from the due date when the supply can be planned (instead of the due date of the unsatisfied demand). In cases where there is a past-due demand for a part using this setting, the supply due date will be later than the demand due date, and hence the days supply horizon will end later. In typical cases where the supply due date and demand due date are the same, this field acts identically to "FromDemand". • **Ignore**—Days of supply planning is disabled for parts that use this policy. **Default value**: FromDemand		
LongTermNumber OfDaysSupply	The number of days or intervals of demand that each planned order is intended to satisfy within the long-term days of supply horizon. The interpretation of this field is dependent on the **LongTermNumber Of DaysSupply** setting as follows: • If using the "FromDemand" or "FromSupply", rule, this field should be expressed in intervals of the part's **PlanningCalendars.TimeUnits** calendar. • If using the "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd" or "ByPeriodEndWithPriority" rule, this field should be expressed in intervals of the **LongTermPeriodSupplyInterval** calendar. **Note:** A value of "0" or less in this field is treated as if it were set to "1"; that is, supply is planned to satisfy one day/interval of demand.	Quantity	

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
LongTermPeriod SupplyDateRule	When using any of the four "by period" days supply rules in the long-term planning horizon, this setting defines the dates that intervals in the **LongTermPeriodSupplyDueCalendar** and **LongTermPeriodSupplyIntervalCalendar** should align to. Valid values: • **DockDate**—long-term period calendars are aligned to dock dates. That is, demands are bucketed after consideration for dock-to-stock time and planned order dock dates must fall on intervals in the supply due calendar. This setting is useful for cases where dock-to-stock lead times vary between parts and their is a desire to co-ordinate the shipment of those parts at the sourcing site and subsequent receipt of those parts at the destination site. • **DueDate**—long-term period calendars are aligned to due dates. That is, demands are bucketed based on their due date and planned order due dates must fall on intervals in the supply due calendar. This setting corresponds to how days of supply calculations were always performed in versions prior to 2016.2. **Default value:** DueDate **Note:** This field was added in Version 2016.2, Service Update 4.3.	String (Enum)	
LongTermPeriod SupplyDueCalendar	Defines the calendar whose intervals define valid planned order due (or dock) dates within the long-term days of supply horizon. For example, if planned orders should only be due at the beginning of a week, then this might be set to a "Week" calendar. Or, if planned orders can be due on any working day, then this might be set to a "Weekly" calendar. Applicable only when **DaysSupplyRule** is set to "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd", or "ByPeriodEndWithPriority". **Referenced table**: Calendar	Reference	
LongTermPeriod SupplyInterval	Defines the calendar intervals over which demands are accumulated within the long-term days of supply horizon. For example, demands might be accumulated over weekly or monthly periods. Applicable when **LongTermDaysSupplyRule** is set to "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd", or "ByPeriodEndWithPriority". **Referenced table**: Calendar	Reference	

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
NumberOfDays Supply	The number of days or intervals of demand that each planned order is intended to satisfy within the short-term days of supply horizon. The interpretation of this field is dependent on the **NumberOfDaysSupply** setting as follows: • If using the "FromDemand" or "FromSupply", rule, this field should be expressed in intervals of the part's **PlanningCalendars.TimeUnits** calendar. • If using the "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd" or "ByPeriodEndWithPriority" rule, this field should be expressed in intervals of the **LongTermPeriodSupplyInterval** calendar. **Note:** A value of "0" or less in this field is treated as if it were set to "1"; that is, supply is planned to satisfy one day/interval of demand.	Quantity	
PeriodSupplyDate Rule	When using any of the four "by period" days supply rules in the short-term horizon, this setting defines the dates that the **PeriodSupplyDueCalendar** and **PeriodSupplyIntervalCalendar** should align to. Valid values: • **DockDate**—short-term period calendars are aligned to dock dates. That is, demands are bucketed after consideration for dock-to-stock time and planned order dock dates must fall on intervals in the supply due calendar. This setting is useful for cases where dock-to-stock lead times vary between parts and their is a desire to co-ordinate the shipment of those parts at the sourcing site and subsequent receipt of those parts at the destination site. • **DueDate**—short-term period calendars are aligned to due dates. That is, demands are bucketed based on their due date and planned order due dates must fall on intervals in the supply due calendar. This setting corresponds to how days of supply calculations were always performed in versions prior to 2016.2. **Default value:** DueDate **Note:** This field was added in Version 2016.2, Service Update 4.3.		

Table 7.15: DaysSuppyPolicy (Mfg) fields

Field name	Description	Data type	Key
PeriodSupplyDue Calendar	Defines the calendar whose intervals define valid planned order due (or dock) dates within the short-term days of supply horizon. For example, if planned orders should only be due at the beginning of a week, then this might be set to a "Week" calendar. Or, if planned orders can be due on any working day, then this might be set to a "Weekly" calendar. Applicable only when **DaysSupplyRule** is set to "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd", or "ByPeriodEndWithPriority". **Referenced table**: Calendar	Reference	
PeriodSupply Interval	Defines the calendar intervals over which demands are accumulated within the short-term days of supply horizon. For example, demands might be accumulated over weekly or monthly periods. Applicable only when **DaysSupplyRule** is set to "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd", or "ByPeriodEndWithPriority". **Referenced table**: Calendar	Reference	
Value	A unique string value for this days of supply policy. This is referenced from the **DaysSupplyPolicy** field on the **Part** table.	String	Yes
Parts	Set of **Part** records that use this days of supply policy.	Set	

DemandStatus table

The **DemandStatus** table determines how demand is handled by RapidResponse analytics. For example, you can set **ProcessingRule** field to **Ignore** resulting in the demand being ignored by netting calculations.

Historical sales data (demand that has been shipped to a customer) can be defined by setting the **ProcessingRule** field in this table to **Ignore**. When loading historical sales data, it's recommended that the amount already shipped be loaded into the **IndependentDemand.ShippedQty** field. It's also recommended that you specify zero in the **IndependentDemand.Quantity** field.

Table 7.16: DemandStatus (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this DemandStatus record. **Referenced table:** ControlSet	Reference	Yes
Description	The description of the demand status.	String	

Table 7.16: DemandStatus (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption DateRule	Specifies the date from which the forecast consumption interval is determined for any independent demands with this status. Valid values are: • **DataDate**—Demand forecast consumption interval is set based on the later of **DueDate** or **Part.PlanningCalendars.DataDate** (this usually corresponds to today's date). As a result, on-time independent demands will consume forecast based on their **DueDate**, while past-due demands will consume forecast based on the **DataDate** (and typically consume more of the forecast than they would at their due date). • **DueDate—**Demand forecast consumption interval is set based on **DueDate**. This means that all independent demands with this status, whether on-time or late, will consume forecast based on their **DueDate**. • **FromPartType**—Demand forecast consumption interval is set based on a part's **PartType.ForecastConsumptionDateRule** setting. This setting only contains "DueDate" and "DataDate" options, and does not support using the **RequestDate** to determine forecast consumption. • **RequestDate**—Demand forecast consumption interval is set based on **RequestDate**. This means that all independent demands using this status will consume forecast based on their **RequestDate** regardless of when that request date is. • **RequestOrDataDate**—Demand forecast consumption interval is set based on the later of **RequestDate** or **Part.PlanningCalendars.DataDate** (this usually corresponds to today's date). With this setting, any demands where the **RequestDate** is in the past will consume forecast based on the **DataDate** (and typically consume more of the forecast than they would at their request date). **Default value:** FromPartType	String (Enum)	

Table 7.16: DemandStatus (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption Override	If a line item belongs to a demand order where the **DemandType.ProcessingRule** is set to "SalesActual", this setting controls whether that individual line item can consume forecast. Valid values are: • **DoNotConsume**—line items with this setting are not eligible to consume forecast demands. • **Normal**—line items with this setting can consume forecast demands according to the configured forecast consumption logic. **Default value**: Normal **Note:** For more information about forecast consumption, see "Forecast consumption" on page 1431.	String (Enum)	
ModelUsage	Indicates how the model associated with a given independent demand is determined. Actual usage of that model in analytic calculations is then set by the part's **MUEPoolNettingType.ModelRule** value. Valid values are: • **Ignore**—model is set to the default model; this is the first value loaded in the **Model** table and typically has a value of "None". The input value provided in **IndependentDemand.Model** is ignored. • **Normal**—model is set to the input value provided in **IndependentDemand.Model**. **Default value:** Normal	String (Enum)	
ProcessingRule	Specifies if the demand is used in RapidResponse netting calculations. Valid values are: • **Ignore**—demand is ignored by netting • **Current**—demand is used by netting (at least one DemandStatus value must use this setting) **Default value:** Current	String (Enum)	
Value	The string value for this type of DemandStatus.	String	Yes
Independent Demands	Set of **IndependentDemand** records that reference this DemandStatus value.	Set	

DemandType table

The **DemandType** table defines the values for types of demands. It is used in **IndependentDemand** table to identify the different types of demands and their processing rules. It is also used with other demand tables and control tables to specify demand-processing behavior.

At least one DemandType must be defined. Typically, there will be at least one DemandType for each of the following demands: customer or sales orders, forecast, shipments, and dependent demand.

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this DemandType record. **Referenced table:** ControlSet	Reference	Yes
AllotmentOrder	Specifies priority in which supply is allocated to demands that have the same due date. The priority is ascending, meaning the lower the value the higher the priority. For example, demands with an AllotmentOrder of 1 receive supply before demands with an AllotmentOrder of 10 given that they have the same due date. This value must be a positive integer. **Default value**: 0	Quantity	
Description	A description of this type of demand. **Default value**: blank string	String	
KindPriority	The OrderPriority extended applications can use for IndependentDemands when they do not want to "bump" existing commitments (for customer orders, should be lower than the priority of any customer orders, yet above forecast). This field is not used by RapidResponse calculations. **Referenced table:** OrderPriority (The Default = OrderPriority with the highest Number, which is the lowest priority).	Reference	

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
PlanningRule	Specifies how the Netting and Capable-to-Promise analytics handle any unsatisfied demand for a particular demand type within a given period (as defined by the ForecastCalendar referenced from each part's PlanningCalendars record). For example, when there is limited supply within a given period, it might be acceptable to ignore any unsatisfied forecasted demand, but any actual demand may still need to be fully satisfied. This field applies only to independent demands (forecast or actuals), consensus forecast, input allocations, and transfer dependent demands. Valid values are: • **Full**— the demand order is required to be fully satisfied. In cases of limited supply, the demand may look to future forecast intervals to find supply. Assuming the same Priority and DueDate, demands using this value are processed before demands set to 'Partial'. • **Partial**— the demand order can only consume supplies in the number of forward-looking forecast intervals specified in the **Part.UnsatisfiedDemandTolerance** field (the default value of "0" means the demand can only consume supplies inside the same forecast interval or earlier). In cases of limited supply, the demand therefore may be left partially, or not at all, satisfied. If using this rule, it is recommended to also have incremental availability calculations turned on (for example, to enable seeing the available date associated with the satisfied portion of an order). **Default value**: Full **Note:** For more information, see "Ignoring unsatisfied demands" on page 1420. **Note:** For any parts with a DaysSupplyRule value of "ByPeriod" or "ByPeriodWithPriority" the setting in this field is ignored (always treated as "Full").	String (Enum)	

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	This value determines how the demand is handled in MRP netting. Valid values are: • **FromPart**—dependent demands are processed for forecast consumption on a component by component basis. Dependent demand for each component is treated as either **SalesActual** (processed for forecast consumption) or **Regular** (not processed for forecast consumption) as determined by the component's setting in **PartType.DependentDemandForecastConsumption**. This value should not be used in DemandType records that are referenced by IndependentDemand records. If it is, then those IndependentDemand records will be treated as 'Regular' demands. • **Ignore**—ignore this demand entirely. • **SalesForecast**—this is forecast of expected independent demand of type SalesActual. It is consumed (reduced) by SalesActual. Just the unconsumed forecast on or after the Demand Time Fence (DTF) is used as demand in MRP netting. A sales demand consumes a sales forecast within the bounds of the Forecast intervals on the Part's planning calendar. • **SalesActual**—actual demand that was forecasted as SalesForecast. The**Quantity** field plus the **ShippedQty** field of the IndependentDemand record is used to consume sales forecast; however, only the Quantity field is the actual demand used in MRP netting. At least one of the defined DemandTypes must have a ProcessingRule set to SalesActual. • **ProductionForecast**—forecast for production usually generated by explosion through a planning bill. It is not consumed by any actual demand, and thus the entire quantity is used by netting (for all part types). Also, demand time fence does not affect ProductionForecast. It is not considered committed (actual) demand and therefore supply created to satisfy ProductionForecast demand results in ATP for MPS parts. • **Regular**—demand used in MRP netting that does not consume forecast. It is real demand and reduces the ATP (consumes supply). **Note:** DemandType.ProcessingRule does not impact ConsensusForecast.	String (Enum)	

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
SafetyLeadTime Rule	Determines whether the **SafetyLeadTime** values specified on the **Part** and/or **PartSource** tables can be applied to demands of this type. Safety lead time is used to account for fluctuations in supply and demand and impacts planned order due dates for parts having **PartType.UseSafetyLeadTime** set to either "Y" or "Soft". Note also that this setting is only applicable to dependent demands for parts having **PartType. DependentDemandSafetyLeadTimeRule** set to "FromDemand", and consensus forecast and independent demands for parts having **PartType IndependentDemandSafetyLeadTimeRule** set to "FromDemand". Valid values are: • **Use**—Safety lead time is used for demands of this type. • **Ignore**—Safety lead time is ignored for demands of this type. **Default value:** Use	String (Enum)	

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
SafetyStockDemandUsage	Indicates whether demands of this type should be included in dynamic safety stock calculations and hence whether they impact dynamically calculated safety stock levels. For example, irregular types of demand or some types of transfer demands might need to be excluded from use in dynamic safety stock calculations. . This setting applies to demands for parts that use any of the following **SafetyStockQuantityRule** options: • FractionOfDemand • PercentOfDemand • RangeOfCoverage • RangeOfCoverageLeadTime • TimePhasedRangeOfCoverage • TimePhasedSafetyDaysMaximum • TimePhasedSafetyDaysTotal Valid values are: • **Use**—demands of this type are included in dynamic safety stock calculations (will impact calculated safety stock levels). • **Ignore**—demands of this type are excluded from dynamic safety stock calculations (will not impact calculated safety stock levels). **Default value:** Use **Note:** This field was added in Version 2016.2, Service Update 14. Prior to this there was no way to exclude specific demand types from being used in safety stock logic. **Note:** For dependent demands on parts that use either of the "TimePhasedSafetyDaysMaximum" or "TimePhasedSafetyDaysTotal" settings, this field along with **SafetyStockPolicy.SafetyDaysDependentDemandUsage** determines whether a given demand is used in safety stock logic. That is, in order for the demand to be used, the part must reference a **SafetyDaysDependentDemandUsage** setting other than "None" and the dependent demand must resolve to a value of "Use" in this field.		
SpreadProfile	The profile to use if SpreadRule = **Spread**. A default profile is often defined with no elements for non-forecast demands. **Referenced table:** SpreadProfile.Value	Reference	

Table 7.17: DemandType (Mfg) fields

Field name	Description	Data type	Key
SpreadRule	Specifies whether spreading is applied to forecasted demands. Spreading only applies if ProcessingRule is **SalesForecast**. Valid values are: • **None**—do not use spread. • **Spread**—spread the forecast using values in the SpreadProfile table. **Default value:** None	String (Enum)	
Value	The string value for this type of demand. This value appears in the **DemandOrder.Type** field. **Default value**: blank string	String	Yes
DemandOrders	Set of **DemandOrder** records that reference this DemandType value.	Set	
PartCustomerTypes	Set of **PartCustomer** records that reference this DemandType value.	Set	
SupplyTypes	Set of **SupplyType** records that reference this DemandType value.	Set	

DDMRPRule table

The **DDMRPRule** table contains DDMRP related configuration settings. It is used to identify the demand-processing behavior settings used to turn DDMRP calculations on or off for parts using this rule. This table supports the DDMRP application. This control table turns on Decoupled Lead Time calculations for each part.

This table was added in Version 2016.2, Service Update 27.

Table 7.18: DDMRPRule (Mfg) fields

Field name	Description	Data type	Key
Description	A description of this type ProcessingRule.	String	
ProcessingRule	Specifies the processing rule for DDMRP calculations. This field can have one of two values. Valid values are: **Ignore** -the DDMRP calculations are ignored and not calculated. **DDMRP** -the DDMRP calculations are calculated for parts choosing this rule. **Default value:** Ignore	String (Enum)	
Value	The unique value specified for each record.	String	Yes

DistributionPlanningRule table

The **DistributionPlanningRule** table is referenced by the **Part** table and determines if **TimePhasedMaximumInventory** records for the part are used in CTP calculations to trigger pushing of supply from the source site to the destination site(s).

If a part at the sourcing site has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and it exceeds the set maximum inventory level, this triggers a request for supply of the finished goods to be pushed (transferred) to the destination sites earlier than it is required there. Details of these pushed supply activities are reported in the **SupplyDemand.PushDate** field as well in **CTPActivity** records with a **Type** value of "PushTransferOut".

Note that the **TimePhasedMaximumInventory.Quantity** value sets the maximum inventory level to be held for a part at a souring site, and inventory above that level is then eligible to be pushed to the destination site. However there must be corresponding amounts of future demand at those sites for it to be allocated against (otherwise, inventory above the maximum might be held).

This table was added in Version 2016.2. For more information about how the settings in this table affect the transfer or pushing of supply, see "Distribution planning" on page 1717.

Table 7.19: DistributionPlanningRule fields

Field name	Description	Data type	Key

AllotmentRule	Indicates the type of fair share distribution allotment that is used for this part. Valid values are:
	• **Normal**—This uses a simple fair share allotment in the CTP calculation. If lateness occurs, priority becomes effective.
	• **ForwardBucketedFairShare**—This uses an enhanced fair share distribution allotment in the CTP calculations. The goal is to do the following:
	  • Start by satisfying the stock-outs at destination sites
	  • Once stock-outs are satisfied, bring destination sites up to their safety stock levels
	Allotments are performed in a planning cycle. The shortages that occur during one planning cycle are carried over into the following planning cycle.
		String(Enum)	

ControlSet	The control set associated with this **DistributionPlanningRule** record. **Referenced table**: ControlSet	Reference	Yes
Description	An optional description of this distribution planning rule. **Note:** This field was added in Version 2016.2, Service Update 27.	String	
Parts	Set of **Part** records that reference these **DistributionPlanningRules** values.	Set	

Table 7.19: DistributionPlanningRule fields

Field name	Description	Data type	Key
ProcessingRule	Indicates if the **TimePhasedMaximumInventory** record applicable to a given part is used to trigger the pushing of supply from source sites to destination sites before the supply is needed for demand at the destination. Valid values are: • **Disable**—The **TimePhasedMaximumInventory.Quantity** is not used in for the part. Thus, supply will not be pushed to destination sites before it is needed. • **Enable**—The **TimePhasedMaximumInventory**.**Quantity** is used for this part. If a part exceeds the set maximum inventory level at the sourcing site, this trigger requests for supply to be transferred (pushed) to the destination site(s) earlier than it is needed there. Note that these pushed supplies will respect the multiple quantity of the parent part source at the destination sites. When this setting is selected, the **AllotmentRule** setting further determines the type of fair share distribution allotment used and the **SupplyPushRule** setting can be used to generate push activities for all supplies instead of only early supplies. **Default value**: Disable	String(Enum)	

Table 7.19: DistributionPlanningRule fields

Field name	Description	Data type	Key
SupplyPushRule	Determines whether pushed supply activities are only generated for supplies that are early to satisfy demand at the destination site, or if they can also potentially be generated for all supplies (including those are at late to satisfy demand at the destination site). Valid values are: • **All**—Pushed supply actions are always generated when trasferring supply from the source site to the destination site. For supplies that are early to satisfy demand at the destination site, pushed supply actions are generated when the maximum inventory level is exceeded at the source site (identical to the "Early" setting). However, pushed supply actions are also generated in cases where supply is transferred either on time or late to satisfy demand at the destination site. With this setting, if supply is late to satisfy demand at the destination site, then pushed supply activities are still generated (the **SupplyDemand.PushDate** is populated and a **CTPActivity** record with **Type** set to "PushTransferOut" is generated). Additionally, dates on these pushed supplies are aligned as necessary to ensure the supply quantity being transferred to the destination site respects the parent part source's **MultipleQty**. • **Early**—When the maximum inventory level is exceeded at the source site, supply that is early to satisfy demand at the destination site is pushed to that site and only such early supply will generate pushed supply activities. This corresponds to how push-planning logic was always performed prior to Version 2016.2, Service Update 25. With this setting, if supply is late to satisfy demand at the destination site, then it is still "pulled" to that site but no pushed supply activities are generated. For example, the **SupplyDemand.PushDate** is not populated and there isn't a **CTPActivity** record with **Type** set to "PushTransferOut". Additionally, the supply quantities being transferred to the destination site might not align to the parent part source's **MultipleQty**. **Default value**: Early **Note**: This field was added in Version 2016.2, Service Update 25.	String (Enum)	
Value	A unique value used to identify the **DistributionPlanningRule** record.	String	Yes

DRPConfiguration table

The **DRPConfiguration** table contains settings that support the Distribution Requirements Planning (DRP) application in RapidResponse. This table contains exactly one record, and can be modified on a

scenario-by-scenario basis to model various planning scenarios.

The settings in the **DRPConfiguration** table are used in various DRP resources, such as the **Distribution Requirements Planning** workbook, but do not impact any analytic calculations. Note also that the settings in this table act as global defaults, some of which can be overridden at the part level using corresponding fields in the **PartSolution** table.

This table was added in the Version 2016.2, Service Update 5, to support the **Distribution Requirements Planning** application.

Table 7.20: DRPConfiguration (Solutions) fields

Field name	Description	Data type	Key
Calendar	A reference to the **Calendar** whose intervals define bucket sizes in the DRP application. For example, this might by a **Weekly** calendar. **Referenced table**: Calendar	Reference	
DRPFrozenWindow	Indicates the length of the **DRP Frozen Window** in **Calendar** intervals. The frozen window is then defined starting from **CalcStartDate** and ending after the number of intervals specified here. This window represents a near-term portion of the planning horizon within which changes to orders should be prevented or limited. This field value acts as a global default for all parts where **PartSolution**. **DRPFrozenWindow** is set to **-1**. Otherwise, if a non-negative value is provided in the **DRPFrozenWindow** field then it is used for the part. The part's effective **DRP frozen window**, from either the **PartSolution** or **DRPConfiguration** tables, is then reported in **PartSolution.EffectiveDRPFrozenWindow**.	Integer	
DRPHorizon	Indicates the length of the **DRP horizon** in **Calendar** intervals. The **DRP horizon** is then defined starting from **CalcStartDate** and ending after the number of intervals specified here. Within this horizon, the distribution requirements planner is actively executing the distribution requirements plan, such as aligning orders to demand and firming planned orders. This field value acts as a global default for all parts where **PartSolution.DRPHorizon** is set to **-1**. Otherwise, if a non-negative value is provided in the **DRPHorizon** field then it is used for the part. The part's effective **DRP Horizon**, from either the **PartSolution** or **DRPConfiguration** tables, is reported in **PartSolution.EffectiveDRPHorizon**.	Integer	
NewTransferType	A default transfer type that will be used for DRP inventory transfers. A reference to the **InventoryTransferType** table used to indicate the transfer type. **Referenced table**: InventoryTransferType	Reference	

Table 7.20: DRPConfiguration (Solutions) fields

Field name	Description	Data type	Key
PlanningHorizon	Indicates the length of the planning horizon in **Calendar** intervals. The planning horizon is then defined starting from **CalcStartDate** and ending after the number of intervals specified here. The planning horizon specifies the length of time that extends into the future for long-term planning. This field value acts as a global default for all parts where **PartSolution.DRPPlanningHorizon** is set to **-1**. Otherwise, if a non-negative value is provided in the **DRPPlanningHorizon** field then it is used for the part. The part's effective planning horizon, from either the **PartSolution** or **DRPConfiguration** tables, is reported in **PartSolution.EffectiveDRPPlanningHorizon**.	Integer	
SRIntransitStatus	A reference to the **SupplyStatus** table used to indicate the in-transit status of a DRP scheduled receipt. **Referenced table**: SupplyStatus	Reference	

ExpiryType table

The **ExpiryType** table contains rules controlling how lower-level component supplies are rolled up and used in calculating the expiry date on assemblies that reference a **PartType.ExpiryRule** setting of "DependentStart". Specifically, this table allows for designating components as "limiting components". A limiting component typically refers to a bottom-level component whose calculated available date should limit the expiry date on the higher-level assemblies where it is ultimately used (dependent on the assembly's **ExpiryRule** setting).

Note also that the settings in this table are always used in determining the **ExpiryStartDate** that is calculated on supplies for all parts that reference an **ExpiryRule** setting other than "Ignore" (the **ExpiryStartDate** is primarily used for calculating the expiry date on assemblies having an **ExpiryRule** setting of "DependentStart")

Table 7.21: ExpiryType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set that this **ExpiryType** record belongs to. **Referenced table:** ControlSet	Reference	Yes
Description	An optional description of this expiry type. **Note:** This field was added in Version 2016.2, Service Update 27.	String	

Table 7.21: ExpiryType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies whether the part is a limiting component or a normal component, and hence how the **ExpiryStartDate** is calculated for its supplies. The **ExpiryStartDate** calculated on component supplies is then rolled up the structure and subsequently impacts expiry calculations when used in assemblies having a **PartType.ExpiryRule** of "DependentStart". Typically, such assemblies should contain one or more limiting components. Valid values are: • **LimitingComponent**—If the part is a bottom-level component, then the **ExpiryStartDate** on its supplies is set to the date their expiry begins. By default, expiry begins on the available date but this is configurable using the **PartSourceType.ExpiryDateRule** setting. Additionally, input expiry start dates can be provided for on hand inventory and scheduled receipts. • **Normal**—If the part is a bottom-level component, then the **ExpiryStartDate** on its supplies is set to "Future". For "DependentStart" assemblies, the **ExpiryDate** on planned supplies is calculated by adding **PartSource.CTPIntervalsToExpiry** to the earliest **ExpiryStartDate** values rolled up from all component supplies. Note however that if the earliest **ExpiryStartDate** is "Future", then the assmebly supply's **AvailableDate** is used in the calculation instead. **Default value:** Normal Note: This field's usage was modified in Version 2016.2 (Service Update 6). For more information about how this field is used in **ExpiryStartDate** calculations, see "DependentStart" on page 1619.	String (Enum)	
Value	A unique string identifier for this record.	Quantity	
Parts	Set of **Part** records that reference this **ExpiryType** value.	Set	

ForecastItemParametersType table

This table is referenced by the **ForecastItemParameters** table. It contains the parameters used in calculating statistical forecasts for a given forecast item. For example, the statistical model and bucketing intervals used when calculating a statistical forecast are identified in this table.

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this ForecastItemParametersType record. **Referenced table:** ControlSet	Reference	Yes
CrostonsPredictRule	The value that determines if Croston's statistical forecasting method should report constant or sporadic predictions of future demand. Valid values are: • **Constant**—Croston's method should report constant predictions, or forecast values that are spread over all time periods. • **Sporadic**—Croston's method should report sporadic predictions, or forecast values that appear only at calculated intervals. **Default value**: Constant **Note:** When using the Croston's method, you can verify individual values for the final estimated non-zero demand at every InterarrivalValue and the final estimated arrival time using the **StatisticalForecastFitOutput** table. In the **ValueName** field, the **SmoothedValue** model output reports the final estimated non-zero demand at every InterarrivalValue and the **InterarrivalValue** model output reports the final estimated arrival time.	String (Enum)	
Description	A description of this ForecastItemParametersType record.	String	
DisaggregationActualsCategoryRule	The value that determines which historical demand category is used for statistical forecast disaggregation. Valid values are: • **Default**—The default historical demand category, as defined by the **SOPAnalyticsConfiguration** table, should be used for forecast disaggregation. • **UseOwn**—The historical demand category defined for the forecast item should be used for disaggregation. **Default value**: Default	String (Enum)	

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
FitMeasure	When "BestFit" is selected as the ForecastModel, this field indicates the measure used in selecting the statistical model that best fits the data. Valid values are: • **AdjustedRSquared**—Calculates the ratio between the regression sum of squares and total sum of squares measures similar to the RSquared measure, but is influenced by the number of values in the input data set. • **LagPercentageError**—Calculates the sum of differences between the simulated forecast and actuals in the first interval of the holdout period (after any offset), expressed as a percentage of all actuals in that interval. Applicable only when best fit is used with a holdout period. • **MeanAbsoluteError**—Determines the absolute value of the variance between the historical actual values and the predicted forecast values. • **MeanAbsoluteErrorByMean**—Normalizes the value returned by the mean absolute error measure by dividing it by the mean quantity of the input data. • **MeanAbsolutePercentageError**—Determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. • **MeanAbsoluteScaledError**—(non-seasonal) Determines the difference between the forecast error for a given period, which represents the actual value minus the forecast value for the same period, and then divides it by the mean absolute error of the one-step ahead naïve forecast. • **MeanError**—Determines the average variance between the historical actual values and the predicted forecast values. • **MeanPercentageError**—Determines the average variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. • **MeanSquareError**—Calculates the average of the residual sum of squares for the forecast. • **RegressionSumSquares**—Determines the difference between the total sum of squares of the input data and the residual sum of squares, and shows how the variance of the statistical forecast and input values differs from the variance of the input values and average of input values. • **ResidualSumSquares**—Calculates the sum of the square of differences between the historical actual values and the values predicted by the statistical forecast. • **RollingLagPercentageError**— Calculates the sum of differences between the simulated forecast and actuals in a specified number	String (Enum)	

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
	of intervals of the holdout period, and then expresses this as a percentage of all actuals in those intervals. The number of intervals to use with this measure is set by the RollingErrorIntervalCount field on the ForecastItemParameters table, and is applied from the start of the holdout region (after any offset). This setting is applicable only when best fit is used with a holdout period. • **RootMeanSquareError**—Determines the square root of the value produced by the mean square errors measure, and shows the correlation between the historical actual values and the results of the statistical forecast calculation. • **RSquared**—Determines the ratio between the regression sum of squares and total sum of squares measures, and shows how closely the forecast values fit the historical actual values. • **SymmetricMeanAbsolutePercentageError**—Determines the absolute difference between the forecast values and historical actual values and divides it by half the sum of the forecast values and actual values, expressed as a percentage. The sMAPE error rate is limited to 200%. **Note:** This field was modified in Version 2016.2, Service Update 10 and 13.		

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
ForecastModel	Indicates the statistical model that will be used to calculate the statistical forecast.Valid values are: • **AdditiveHoltWintersMethod**—Differs from the Holt-Winters model in that forecast points are calculated in an additive, rather than multiplicative, manner. Suitable for demand that has constant variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **ARIMA**—Calculates forecasts for a time period using a linear combination of actual values and error values in a time series. This model can be used on a stationary time series, or a differencing function can be applied to non-stationary data to make it stationary. For a more detailed description, see "Auto-Regressive Integrated Moving Average (ARIMA)" on page 2034. • **ARIMAX**—Is an extension of the ARIMA model, but also includes independent variables (regressors) in the forecast calculation. This makes ARIMAX a multiple linear regression model. To improve forecast accuracy, ARIMAX also takes into consideration autocorrelation that may be found in regressor residuals. For a more detailed description, see "ARIMAX" on page 2042. • **BestFit**—Fits each of the other forecast models and selects the one with the least margin of error. If the forecast item parameters reference a valid **BestFitProfile** record, then just the subset of models defined by the profile is used in the best fit process. Otherwise, if that **BestFitProfile** reference is Null, then all models (excepting "Rforecast") are used in the best fit process. For a more detailed description, see "Best fit" on page 2053. • **CrostonsMethod**— Uses exponential smoothing with a single smoothing constant for the forecast values, but calculates two different values; the first being the forecast values at a point and the second being the average length of time between forecasts. These values determine the forecast quantities and the periods that have forecast demands. Suitable for demand that is not constant, such as lumpy demand that contains periods of zero quantities. For a more detailed description, see "Croston's method" on page 2034. • **DoubleExponentialSmoothing**—Considers the data points and trends within the historical data, and applies a smoothing algorithm to both. The result is then used as the statistical forecast. Suitable for demand that is fairly stable but trending up or down over time. For a more detailed description, see "Double exponential smoothing" on page 2036. • **ElasticNet**—A multiple linear regression model that uses regularized, also known as penalized, regression methods. It blends Lasso and ridge regression techniques by penalizing both L1 and L2 norms to reduce the coefficient vector. To use ElasticNet, you must attach a minimum of two regressors to a forecast item or you won't be able to generate a statistical forecast. For a more detailed description, see "ElasticNet" on page	String (Enum)	

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
	2043. • **ExponentialSmoothing**— Considers data points within the historical data, and applies a smoothing algorithm. The result of the final data point is then used as a constant value for the statistical forecast. Suitable for demand that is fairly stable and lacks a trend. For a more detailed description, see "Exponential smoothing" on page 2037. • **ForecastImport**—The statistical forecast is set to values in the ImportedStatisticalForecast table. This input table is meant to hold statistical forecast values that were generated using an external tool and, once imported, can then be used by post-processing forecast logic in RapidResponse (for example, applying NPI/EOL profiles or disaggregating to the detailed part-customer level). For more information, see "ForecastImport" on page 2058. • **HoltWintersMethod**—Similar to double exponential smoothing in that it considers data points and trends within the historical data, but also accounts for seasonal trends. That is, the values for each point in the forecast and trend are further adjusted to account for seasonal demand changes. This model is the Holt-Winters multiplicative method, so it is suitable for demand that has proportional variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **Linear**—Performs a linear regression on a set of historical data points to generate a trend line that determines statistical forecast quantities. Suitable for demand that does not fluctuate. For a more detailed description, see "Linear" on page 2039. • **MovingAverage**—Calculates the simple moving average of a specified number of previous data points to generate a constant forecast. Suitable for demand that is fairly stable and does not change much over time. For a more detailed description, see "Moving average" on page 2039. • **MultipleLinearRegression**—Performs a linear regression using multiple independent variables to predict the outcome of one dependent variable. For a more detailed description, see "Multiple linear regression" on page 2043. • **RForecast**—Uses statistical functions defined in the R statistical programming language and generates a forecast using the function that provides the best fit with the item's data. Optionally, if populated, the RParameterSet reference on the ForecastItemParameters or BestFitProfileDetail tables can be used to define specific parameter values to be passed to named functions in R. This option supports the optional R Forecast capability. For more information, see "Rforecast" on page 2039. • **StepWiseARIMA**—Identical to the ARIMA model, but all parameters are calculated automatically. Step-wise ARIMA selects the order of difference		

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
	through KPSS unit-root tests and selects AR order and MA order by minimizing AIC. For a more detailed description, see "Step-wise ARIMA" on page 2039. **Default value:** BestFit **Notes:** • This field was modified in Version 2016.2, Service Update 5. • This field was modified in Version 2016.2, Service Update 5 and 27.		

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
HoldoutPeriod UsageRule	Specifies whether a holdout period is used during best fit statistical forecasting (applicable only to items where **ForecastModel** is set to "BestFit"). The holdout period is made up of a number of intervals immediately prior to **RunDate** that are held out (excluded) when fitting statistical models. Instead, historical data in the holdout period is used to evaluate simulated forecasts generated based on models fitted against historical demands before the holdout period. The model that most accurately predicts the actuals in the holdout period (based on the chosen fit measure), is then used to generate the statistical forecast. The duration of an item's holdout period is set by the **BestFitHoldoutPeriodIntervalCount** field on the **ForecastItemParameters** table. Valid values are: • **Ignore**—No holdout period is assumed, and the **BestFitHoldoutPeriodIntervalCount** field is ignored. Each statistical model is fitted against the full range of historical data being considered and the model with the least error is used to generate the forecast. This setting corresponds to how best fit forecasting was always performed in versions prior to 2015.3. • **SimpleHoldout**—The holdout period is used in best fit forecasting. Each model has a single set of fitted parameters generated based on historical data before the holdout period, and then used to generate a simulated forecast in the holdout period. The forecast is then compared against historical actuals from intervals in the holdout period. • **Simulation**—The holdout period is used in best fit forecasting. Each model has multiple sets of fitted parameters generated from a rolling window of data that begins in the area before the holdout period and moves later. Each set of parameters then generates a forecast value for comparison against actuals from a specific interval in the holdout period. This means that actuals later in the holdout period are compared against models fitted to more recent data. **Default value:** Ignore If **BestFitHoldoutPeriodIntervalCount** is set to 0 (zero) or less, then this setting is treated as if it were set to "Ignore". Also, if there is an invalid value in **BestFitHoldoutPeriodIntervalCount** then an error is reported and no statistical forecast generated (for example, this might occur if the value specified is greater than the range of historical data being considered). **Note:** For more information about using this field, see "Using best fit with a holdout period" on page 2053.	String (Enum)	

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
IntervalsCalendar	The calendar used to bucket actuals and generate the statistical forecast. Typically, this might be set to a Weekly or Monthly calendar. **Referenced table:** Calendar	Reference	
ModelConstant Usage	Controls whether forecast model constants (alpha, beta, gamma, and so on) are calculated by RapidResponse or taken from values that were previously calculated and are stored in an input table. If values are taken from an input table, the values are taken from either of the following tables: **ForecastItemFitOutput** or **ForecastItemParametersFitOutput**. In addition, for forecast items that are using Multiple Linear Regression or the Elastic Net forecast model to generate a statistical forecast, this field also determines whether constants are calculated or the values are taken from the ForecastItemParametersRegressionCoefficient table. Valid values are: • **Calculate**—Forecast model constants are calculated by RapidResponse. • **Use**—Use model constants given in the **ForecastItemFitOutput** table or **ForecastItemParametersFitOutput** tables. For Multiple Linear Regression or the Elastic Net, use regression coefficients reported in the **ForecastItemParametersRegressionCoefficient** table. **Note:** The "Use" option is not supported when **ForecastModel** is set to Rforecast or ARIMAX.	String (Enum)	
TrendDecay FactorRule	The value that determines if a decay factor is added to dampen or eliminate a forecast trend. You can apply trend decay to the following statistical forecasting methods: • Double exponential smoothing • Elastic Net • Holt-Winters multiplicative • Holt-Winters additive • Multiple Linear Regression The decay factor is specified in the **TrendDecayFactor** field on the **ForecastItemParameters** table. Valid values are: • **Use**—The decay factor should be added to the forecast calculation. • **Ignore**—The decay factor should not be added to the forecast calculation. **Default value**: Ignore	String (Enum)	
Value	A unique identifier for this record. This field is referenced from the **Type** field on the **ForecastItemParameters** table.	String	Yes

Table 7.22: ForecastItemParametersType (Mfg) fields

Field name	Description	Data type	Key
ForecastItem Parameters	Set of **ForecastItemParameters** records that reference this ForecastItemParametersType value.	Set	

Note: For more information about statistical forecast models and fit measures, see "Statistical forecasting" on page 2019.

HistoricalDemandCategoryType table

The **HistoricalDemandCategoryType** table is used to define how demand data associated with a given **HistoricalDemandCategory** value is processed in RapidResponse. This table supports Sales and Operations Planning in RapidResponse.

Table 7.23: HistoricalDemandCategoryType (Mfg) fields

Field name	Description	Data type	Key
AggregationRule	If ProcessingRule is set to "Target", this field is used to set how targets are aggregated. Valid values are: • **Average**—targets are aggregated as average values. • **None**— category is not used for aggregating targets. • **Sum**— targets are aggregates sum of values.	String (Enum)	
Description	A description of this HistoricalDemandCategoryType record.	String	
Disaggregation QuantityRule	The value that determines whether historical actuals or the consensus forecast should be used to calculate forecast disaggregation rates for categories of this type. Valid values are: • **Actuals**—Historical actuals, from a specified historical demand category, should be used for calculating disaggregation rates for forecast categories of this type. • **ConsensusForecast**—The consensus forecast, for the applicable part customer, should be used for calculating disaggregation rates for forecast categories of this type. **Default value**: Actuals	String (Enum)	

Table 7.23: HistoricalDemandCategoryType (Mfg) fields

Field name	Description	Data type	Key
EventAllow Negative	Specifies whether event phases can apply forecast quantity adjustments that result in a negative value. If this field is set to "No," forecast items belonging to this type of demand category cannot have their forecasts reduced below zero by event phases. For more information about event phases, see "Event Management" on page 2109. **Default value:** No **Note:** Regardless of this setting, an event phase cannot adjust the unit price of an item below zero. **Note:** This field was added in Version 2016.2.	Boolean	

Table 7.23: HistoricalDemandCategoryType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies how the historical demand category is used in RapidResponse, and how it affects consensus forecast calculations. Valid values are: • **Actual**—category is used for storing historical demand actuals. Values in this category can be used in calculating the statistical forecast. Ignored by consensus forecast calculations. • **Forecast**—category is used for storing streams of forecast demands or forecast adjustments. Values in this category can be used towards calculating the consensus forecast(based on the specified ConsensusForecastWeight value). Note that adjustment and override values specified in other historical category types can subsequently be used to modify or replace the calculated consensus forecast. • **ForecastOverride**—category is used for specifying values that override the calculated consensus forecast (weights are ignored). Note that this type of override can be modified by a RebalancingAdjustment or overridden by a RebalancingForecastOverride. • **None**—demands in this category are not used in any sales and operations planning calculations. Ignored by consensus forecast calculations • **RebalancingAdjustment**—category is used to increase or decrease either the **ConsensusForecast.CalculatedQuantity** or the **ForecastOverride** value (if used) during the demand and supply balancing phase (weights are ignored). Note that adjustments of this type are ignored if a RebalancingForecastOverride is specified. • **RebalancingForecastOverride**—category is used for specifying values to override the calculated consensus forecast during the demand and supply balancing phase (weights are ignored). Note that this type of override takes precedence over a ForecastOverride as well as any RebalancingAdjustment values that were applied. • **Target**—category is used for defining target metric values against which the S&OP annual plan is measured. Ignored by consensus forecast calculations	String (Enum)	

Table 7.23: HistoricalDemandCategoryType (Mfg) fields

Field name	Description	Data type	Key
UnitType	Specifies whether quantities or monetary values are used as historical demand or forecast targets. Valid values are: • **Quantity**—Values in the **Quantity** fields of the **ForecastDetail** and **HistoricalDemandSeriesDetail** tables are used as targets for historical demands and forecasts. • **Value**—Values in the **Value** fields of the **ForecastDetail** and **HistoricalDemandSeriesDetail** tables are used as targets for historical demands and forecasts. **Default value**: Quantity	String (Enum)	
Value	A unique identifier for this record. This field is referenced form the **Type** field on the **HistoricalDemandCategory** table.	String	Yes
HistoricalDemand Categories	Set of **HistoricalDemandCategory** records that reference this HistoricalDemandCategoryType value.	Set	

IMConfiguration table

This table defines the settings related to the Inventory Management application. It contains values which defines default targets, calendars, and IQR transitions. There should never be more than one record in this table.

Table 7.24: IMConfiguration (Solutions) fields

Field name	Description	Data type	Key
DefaultCustomer ServiceLevelTarget	The default customer service level target. Used when no On Time To Commit or On Time to Request target has been set for a part.	Quantity	
DefaultInventory QualityRatioTarget	The default IQR target. Used when no IQR target has been set for a part. The IQR target is set by inventory personnel in the **S&OP Annual Plan** workbook.	Quality	
DefaultSurplus Transition	When no PeriodsOfSupplyMaximum target is set, the point that inventory would be determined surplus.	Integer	
HorizonCalendar	The calendar used to define the Inventory Management planning horizon. **Referenced table:** Calendar	Reference	

Table 7.24: IMConfiguration (Solutions) fields

Field name	Description	Data type	Key
InventoryQuality RatioCalendar	Used to define the period type for the transitions between IQR categories. For example, if the calendar is months and the slow moving transition is 3, the transition to slow moving will occur at 3 months. **Referenced table:** Calendar	Reference	
ObsoleteTransition	The point that inventory would be considered obsolete instead of slow moving. Used only when **UseObsoleteTransition** is set to "Y".	Integer	
PeriodsOfSupply TargetCalendar	Used to define the target calendar for the **HistoricalDemandHeader** categories of PeriodsOfSupplyMinimum and PeriodsOfSupplyMaximum. **Referenced table:** Calendar	Reference	
RunDate	A reference to a calendar used to set the current system date (typically, this might be a calendar that has only one date belonging to it). The calculated **FirstDate** field on this calendar is then used in determining the starting point for the inventory management planning horizon. **Referenced table:** Calendar	Reference	
SlowMoving Transition	The point that inventory would be determined slow moving instead of surplus.	Integer	
UseObsolete Transition	Indicates whether to use a transition point from slow moving to obsolete. Valid values are: • **N**—Supply will only be considered obsolete if the total demand on the part is 0. • **Y**—The obsolete transition point is used. **Default value:** N	Boolean	
VisibleFuture Periods	Number of future periods to show in the Inventory Management planning horizon.	Integer	
VisibleHistorical Periods	Number of historical periods to show in the Inventory Management planning horizon.	Integer	
IMFutureHorizon	A calculated field indicating the latest date to show in the Inventory Management planning horizon.	Date	
IMPastHorizon	A calculated field indicating the earliest date to show in the Inventory Management planning horizon.	Date	

IncrementalRule table

This table is referenced by the Part table and holds values that can be used to turn incremental availability calculations on or off for each part. For more information about incremental availability, see "Incremental availability calculations" on page 1476.

Table 7.25: IncrementalRule (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this IncrementalRule record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of the incremental availability setting.	String	
ProcessingRule	Specifies whether incremental availability calculations are performed for parts using this setting. Valid values are: • **On**—Incremental availability is on. This means that supply orders for the part can be split if material becomes available over time, rather than all at once, or if constraint occurs in multiple periods. This value can be overridden on a supply-by-supply basis using the SupplyStatus.IncrementalSplit field. • **Off**—Incremental availability is off. Supply orders cannot be split with respect to material availability. **Default value**: Off	String (Enum)	
Value	A unique identifier for the incremental availability setting.	String	Yes
Parts	Set of **Part** records that reference this IncrementalRule value.	Set	

InventoryTransferType table

The **InventoryTransferType** table is referenced by the **InventoryTransfer** table, and contains rules that determine how inventory transfers are handled in RapidResponse. For example, each rule defines the status of the associated inventory transfers (already received, still in-transit, or so on), and whether that inventory transfer record should be processed by RapidResponse analytics.

Table 7.26: InventoryTransferType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this InventoryTransferType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this type of inventory transfer.	String	
ProcessingRule	Determines whether inventory transfers using this setting are processed by RapidResponse analytics. Valid values are: • **Ignore**—inventory transfers using this setting are not processed by RapidResponse analytics. • **Use**—inventory transfers using this setting are processed by RapidResponse analytics. **Default value**: Use	String (Enum)	
Status	The status of inventory transfers using this setting. This determines if the inventory transfer quantity should be subtracted from on hand inventory at the sourcing site (TransferPart), and supply created at the destination site (Part). Valid values are: • **ByDate**—Calculates the status to use based on the ship and stock dates of the inventory transfer as follows: "**InTransit**" is used if *ShipDate* < *Transfer Part.PlanningCalendars.DataDate* and *StockDate* >= *Part.PlanningCalendars.DataDate*. "**NotReleased**" is used if *ShipDate* >= *TransferPart.PlanningCalendars.DataDate*. "**Received**" is used if *StockDate* < *Part.PlanningCalendars.DataDate*. • **InTransit**—An active inventory transfer that has left the sourcing site, but not yet arrived at the destination site. RapidResponse will create supply for the Part at the destination site, but assumes that inventory at the sourcing site has already been decreased, and so no dependent demand is created for the TransferPart. • **NotReleased**—An active inventory transfer that has not yet left the sourcing site. RapidResponse creates supply for the Part at the destination site, and creates dependent demand for the TransferPart. • **Received**—An inactive inventory transfer that occurred in the past. RapidResponse assumes that inventory at the sourcing site has already been decreased, and also does not create supply at the destination site. **Default value**: NotReleased	String (Enum)	
Value	A unique identifier for this inventory transfer setting.	String	Yes

Table 7.26: InventoryTransferType (Mfg) fields

Field name	Description	Data type	Key
InventoryTransfers	Set of **InventoryTransfer** records that reference this InventoryTransferType value.	Set	

IPOConfiguration table

This table contains settings that support the Inventory Planning and Optimization application in RapidResponse. This table contains exactly one record, and can be modified on a scenario-by-scenario basis by either System Administrators or Data Administrators.

It contains default values used when configuring safety stock items such as the outlier type, the threshold, the smoothing intervals settings, and the multi-echelon processing rules. For more information about the default outlier type, see the **OutlierType** table. For more information about the multi-echelon processing rules, see the **MultiEchelonSafetyStockRule** table.

This table was added in Version 2016.2 Service Update 10 to support the **Inventory Planning and Optimization** application.

Table 7.27: IPOConfiguration fields

Field name	Description	Data type	Key
DefaultMEIO NoInventoryRule	The default multi-echelon "NoInventory" processing rule. **Referenced table**: MultiEchelonSafetyStockRule (Nullable)	Reference	
DefaultMEIO UseRule	The default multi-echelon "Use" processing rule. **Referenced table**: MultiEchelonSafetyStockRule (Nullable)	Reference	
DefaultOutlierType	A reference to the default outlier type, whose set of processing rules define how outliers in historical data are identified and subsequently processed for parts configured as safety stock items. **Referenced table**: OutlierType (Nullable)	Reference	
DemandOutlier Threshold	The default (minimum) number of standard deviations away from the mean a demand quantity must be in order to be considered an outlier. Any outliers detected are then removed, smoothed, or ignored, as specified by the setting in the **Type.DemandOutlierProcessingRule**.	Quantity	
Smoothing IntervalsAfter	When smoothing demand outliers, this indicates the default number of **Type.IntervalsCalendar** periods forward over which to spread the values. This occurs after any backward spreading.	Integer	
Smoothing IntervalsBefore	When smoothing demand outliers, this indicates the default number of **Type.IntervalsCalendar** periods backward over which to spread the values. This occurs before any forward spreading.	Integer	

MEIOLeadTimeType table

The **MEIOLeadTimeType** table sets how parts configured for variable/random lead times in multi-echelon safety stock calculations have their average and standard deviation of lead time parameters calculated.

Note that the calculated lead time parameter values resulting from these settings can be seen in the **CalcStandardDeviation** and **CalcAverage** fields on the **MEIOLeadTime** table.

Table 7.28: MEIOLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
Category	Specifies the category that historical supply records in the **HistoricalSupplyActual** table should belong to in order to be collected and used in calculating lead time parameters. This reference only needs to be populated if **ProbabilityDistribution** is set to "Normal" and **StandardDeviationRule** is set to "Calculate". **Referenced table:** HistoricalSupplyCategory (Nullable)	Reference	
ControlSet	The control set associated with this **MEIOLeadTimeType** record. **Referenced table:** ControlSet	Reference	
DistributionProfile Rule	For discrete probability distributions, this setting determines how the individual distribution profile points (as found in the **Value** field of the **MEIOLeadTimeDistributionProfilePoint** table) are interpreted. Valid values are: • **LeadTime**—Each distribution profile point represents an actual lead time value. • **Multiplier**—Each distribution profile point should be interpreted as a multiplier with lead time then calculated as *AverageLeadTime* * *Value*. This setting allows for the same profile to be used for parts with different average lead time values. **Default value:** LeadTime **Note:** This setting is only applicable when **ProbabilityDistributionRule** is set to "Discrete".	String (Enum)	

Table 7.28: MEIOLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
Probability Distribution	Specifies the probability distribution model that lead time values are assumed to follow, and hence how lead time parameters are calculated for this type. Valid values are: • **Discrete**—Lead time is modeled using a discrete probability distribution. Actual lead time values and their associated probabilities come from the distribution profile referenced by **MEIOLeadTime.DistributionProfile** (with the setting in the **DistributionProfileRule** field used to determine how the actual profile points are interpreted). • **Ignore**—Lead time variability is ignored for this type. In other words, it will use the fixed lead time value from the **MEIOLeadTime** table, but it will ignore the standard deviation so that there is no probability distribution. However, it may use a lead time value from the **PartSource** table only if there's no applicable record in **MEIOLeadTime** table. • **Normal**—Lead time is modeled using a normal probability distribution. Based on the setting in the **StandardDeviationRule** field, lead time parameters are either set to values provided as input on the **MEIOLeadTime** record or calculated based on historical supply records from the referenced **Category**. **Default value**: Ignore **Note:** For information about configuring lead time parameters for multi-echelon safety stock calculations, see "Configure multi-echelon lead time parameters (optional)" on page 1960.	String (Enum)	

Table 7.28: MEIOLeadTimeType (Mfg) fields

Field name	Description	Data type	Key
StandardDeviation Rule	For normal probability distributions, this setting determines how the average and standard deviation of historical supply lead times is determined (either calculated based on history or set to manually provided input values). Valid values are: • **Calculate**—average and standard deviation of lead time is calculated based on historical supply actuals. In order to be used in these calculations, historical supplies for a given part must belong to the same historical supply category as is referenced in the **Category** field, and those historical records must reference the same **Source** value as is referenced through **MEIOLeadTime.PartSource**. • **CoefficientOfVariation**—average of lead time is set to the input value provided in **MEIOLeadTime.Average** while the value in **MEIOLeadTime.StandardDeviation** is assumed to be the coefficient of variation. Therefore, standard deviation of lead time is calculated as **MEIOLeadTime.Average * MEIOLeadTime.StandardDeviation**. • **Manual**—average of lead time is set to the input value provided in **MEIOLeadTime.Average** while the standard deviation of lead time is set to the input value provided in **MEIOLeadTime.StandardDeviation**. **Default value:** Manual **Note:** This setting is only applicable when **ProbabilityDistributionRule** is set to "Normal".	String (Enum)	
Value	A unique string identifier for this control setting.	String	Yes
Items	Set of **MEIOLeadTime** records that use this control setting.	Set	

MPSConfiguration table

The **MPSConfiguration** table contains settings that support the MPS application in RapidResponse. This table contains exactly one record, and can be modified on a scenario-by-scenario basis to model various MPS planning scenarios. For example, the duration of the MPS planning horizon might be changed.

The settings in the **MPSConfiguration** table are used in various MPS resources, such as the **Master Scheduling** workbook, but do not impact any analytic calculations. Note also that the settings in this table act as global defaults, some of which can be overridden at the part level using corresponding fields in the **PartSolution** table.

Table 7.29: MPSConfiguration (Solutions) fields

Field name	Description	Data type	Key
AutoFirmWindow	Indicates the length of the auto-firm window. The auto-firm window defines a period after the MPS planning horizon in which a workbook command automatically converts planned orders into firm MPS orders. This value is added from the start of the latest **SOPAnalyticsConfiguration.CycleCalendar** interval that is contained in the MPS horizon. This field value should be expressed in **CycleCalendar** units. This field acts as a global default for all parts where **PartSolution.MPSAutoFirmWindow** is set to -1. Otherwise, if a non-negative value is provided in the **MPSAutoFirmWindow** field then it is used for the part. The part's effective MPS auto-firm window, from either the **PartSolution** or **MPSConfiguration** tables, is then reported in **PartSolution.EffectiveMPSAutoFirmWindow**.	Integer	
Calendar	A reference to the Calendar whose intervals define bucket sizes in the MPS application. For example, this might by a Weekly calendar. **Referenced table:** Calendar	Reference	
FrozenWindow	Indicates the length of the MPS frozen window in **Calendar** intervals. The frozen window is then defined starting from **CalcStartDate** and ending after the number of intervals specified here. This window represents a near-term portion of the planning horizon within which is strongly recommended to not make any changes to MPS orders (for example, because it may not be feasible to change material or capacity commitments). This field value acts as a global default for all parts where **PartSolution.MPSFrozenWindow** is set to -1. Otherwise, if a non-negative value is provided in the **MPSFrozenWindow** field then it is used for the part. The part's effective MPS frozen window, from either the **PartSolution** or **MPSConfiguration** tables, is then reported in **PartSolution.EffectiveMPSFrozenWindow**.	Integer	
Horizon	Indicates the length of the MPS planning horizon in **Calendar** intervals. The planning horizon is then defined starting from **CalcStartDate** and ending after the number of intervals specified here. Within this horizon, MPS orders are assumed to be fully under the authority of the master scheduler. This field value acts as a global default for all parts where **PartSolution.MPSHorizon** is set to -1. Otherwise, if a non-negative value is provided in the **MPSHorizon** field then it is used for the part. The part's effective MPS horizon, from either the **PartSolution** or **MPSConfiguration** tables, is reported in **PartSolution.EffectiveMPSHorizon**.	Integer	

Table 7.29: MPSConfiguration (Solutions) fields

Field name	Description	Data type	Key
RunDate	A reference to a calendar used to set the current system date (typically, this might be a calendar that has only one date belonging to it). The calculated **FirstDate** field on this calendar is then used in determining the starting point for the MPS planning horizon. **Referenced table:** Calendar	Reference	
Tolerance	Specifies whether tolerance settings in the **ToleranceInterval** and **ToleranceCalendar** fields are used by MPS resources to determine whether a supply is considered late. Valid values are: • **N**—tolerance settings are not used in determining order lateness. A supply is considered late if its available date is after the due date on the demand it satisfies. • **Y**—tolerance settings are used in determining order lateness. A supply is considered late if its available date is more than the specified number of **ToleranceCalendar** intervals after the demand due date. **Default value:** N	Boolean	
ToleranceCalendar	A reference to the calendar that determines how the **ToleranceInterval** value is interpreted. **Referenced table:** Calendar	Reference	
ToleranceInterval	This field sets the maximum limit on lateness for MPS orders. It is used in certain resources that use measures of lateness to report whether the master schedule is being achieved. If an MPS order is more than this number of **ToleranceCalendar** intervals late, then it is considered outside of tolerance and hence reported as late (subject to the setting in the **Tolerance** field).	Integer	
StartOffset	Number of **Calendar** intervals after the current system date (**RunDate.FirstDate**) at which to begin the MPS planning horizon.	Integer	
CalcEndDate	A calculated field indicating the end of the MPS planning horizon. This date is set by adding the value in the **Horizon** field to **CalcStartDate**.	Date	

Table 7.29: MPSConfiguration (Solutions) fields

Field name	Description	Data type	Key
CalcStartDate	A calculated field indicating the start of the MPS planning horizon. This date is set by adding the value in **StartOffset** to the current system date (as determined by the referenced value in **RunDate.FirstDate**).	Date	

MUEPoolNettingType table

The **MUEPoolNettingType** table supports the Model-Effectivity, Unit-Effectivity, and Pooling analytics and thereby allows finer control over processing rules for model, unit, and pool. This table is also used to enable attribute-based planning at the part table.

This information in the table is separated from the **PartType** table to allow generic part types like Make and Buy to be retained, while allowing for different netting rules within the type.

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
AttributeExcess Rule	When using attribute-based planning, this setting controls how excess available supply from planned orders created to satisfy one demand can be used (or not) by other subsequent demands. • **Ignore**—excess supply from a planned order created to satisfy one demand can only be used by other demands if their demand attributes match those of the original demand. That is, even if the supply attributes on the excess planned order (as injected from the **PartSource**) satisfy the requirements of a subsequent demand, but the demand attributes on that demand do not match those on the original demand, then the excess supply is not used. This setting corresponds to how attribute-based planning also handled excess planned supply prior to Version 2016.2. • **Use**— excess supply from a planned order created to satisfy one demand can be used by other demands as long as the supply attributes on the planned order (as injected from the **PartSource**) satisfy the attribute requirements of those demands. However, there is no need for the demand attributes on those other demands to match those on the original demand. This setting can help reduce the excess supply that might result from certian configurations of attribute-based planning (for example, when used with lot-sizing requirements). Note also that if using this setting, each PartSOurce should not only have a supply attribute defined (for demand-supply attribute matching), but also a demand attribute which can be propagated to demands on lower levels. **Default value**: Ignore **Note:** This field was added in Version 2016.2. For an example of its usage, see "Example 3: Allotments with attributes and excess supply" on page 1839.	String (Enum)	
AttributeFlowTo Common	If using attribute-based forecast consumption, this setting determines how actual demands can consume forecast based on their respective sets of attribute values. Valid values are: • **Forecast**—actual demand first consume from those forecast demands to which they have matching attribute values. If there is any remaining actual, then forecast consumption continues on any forecast demands that do not have attribute values defined on them. • **Ignore**—actual demands only consume from those forecast demands to which they have matching attribute values. **Default value:** Ignore	String (Enum)	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
AttributeRule	Used to enable or disable attribute-based calculations at the part level. Attributes can be added and used to identify relevant characteristics of demands or supplies which are then used to refine certain analytic calculations. Attributes are supported in supply allocation, forecast consumption, and forecast adjustment calculations. For example, a given customer's demands might only be able to receive supply containing components sourced from particular locations, or a given customer's forecast might only be eligible to be consumed by actual demands from that same customer. The actual rules that define relationships between attributes and how they are used in analytic calculations are set in the **AttributeMap** table. This field is then used to set the specific parts that should utilize those attribute relationships. Valid values are: • **Ignore**—attribute-based planning is disabled for parts using this type. If attribute-based planning logic is not being used at all, this setting can be left for all parts. • **Net**—attribute-based planning is enabled for parts using this type. This value should be set for any parts expected to either require attribute-based planning. **Default value**: Ignore Note also that supply attributes defined on the **PartSource** table can be propagated to all dependent demands under a given planned order regardless of the setting in this field. This field only controls whether attributes are used in planning calculations for a given assembly or component, not whether the attribute values themselves can be propagated through parts in the product structure.	String (Enum)	
ControlSet	The control set associated with this MUEPoolNettingType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of the type of Model Effectivity, Unit Effectivity, or Inventory Pooling configuration. This field is an explanation of the Value field. **Default value**: blank string	String	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
ModelRule	Identifies whether to process Model data. Regardless of the value of ModelRule, it is treated as Ignore unless Model-Unit Effectivity is enabled. Valid values are: • **ExplodeOnly**—no model nettings is performed for this part, but demand can be exploded by model. Demands and supplies are processed in the "default" model, and planned orders are created in the original model of the triggering demand. The model on the supply is then respected when exploding dependent demand on the part. • **Ignore**—no model netting is performed for this part. Model information on scheduled receipts is retained and may be used for explosion, but no Model data is copied to planned supply records. (The **ScheduledReceipts.InputModel** is set to 'None'. The first model in the Model table, needs to be populated with a value of 'None'). • **Net**—align supply and demand by model. Populate **ScheduledReceipt.InputModel** in planned supply with the value of the model being netted. • **NetWithoutForecast**—align supply and demand by model in netting calculations, but ignore the model when consuming forecast (actuals consume from "common" model). This option is useful when model is important for matching supply to demand, but not included on forecast records. **Default value**: Ignore	String (Enum)	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
PoolFlowTo Common	Specifies whether customer orders consume common forecast during independent and dependent demand forecast consumption. This field is also used by the Netting analytic to control where new supply is planned after using up all supply in both a customer's specified pool and the common supply pool. Valid values are: • **Ignore**—customer orders only consume forecast from the pool representing that customer's forecast. New planned orders can only be generated from part sources where the **PartSource.Pool** reference is a match for the demand pool or is "Null". • **Forecast**—customer orders first consume from the pool representing that customer's forecast. If any actual demands for that customer are left after all of the customer's forecast is consumed, then the remaining can consume from the common forecast. New planned orders can only be generated from part sources where the **PartSource.Pool** reference is a match for the demand pool or is "Null". • **NetCommon**—in forecast logic, customer specific forecast consumption is the same as with the Forecast option. In Netting logic, demand within a specific pool will be satisfied first by supply within that same pool pool, and then, if necessary, by supply in the common pool. Once supply in the common pool has been used, new supply is planned in the common pool. New planned orders can only be generated from part sources where the **PartSource.Pool** reference is either to the "Common" pool or is "Null" (part sources where **PartSource.Pool** is a match for the demand pool cannot be used). • **NetOriginal**— in forecast logic, customer specific forecast consumption is the same as with the Forecast option. In Netting logic, demand within a customer-specific pool will be satisfied first by supply in that customer's specified pool, and then, if necessary, by supply in the common pool. Once supply in the common pool has been used, new supply is planned in the (original) customer-specified pool. New planned orders can only be generated from part sources where the **PartSource.Pool** reference is a match for the demand pool or is "Null". **Default value:** Ignore	String (Enum)	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
PoolRule	Identifies whether to process pool data. If Pool is not licensed then the values of Net and Optional are treated as Ignore. However, Forecast and ForecastOptional settings can be used regardless of whether Pool has been enabled. Valid values are: • **Ignore**—no Inventory Pooling is performed for this part. Pool information on scheduled receipts is retained, but no Pool data is copied to planned supply records. New supply for parts with this setting can be created by any eligible part source and planned in the pool defined by **PlanningCalendars.CommonPool** (for example, the "common" or "unpooled" pool). This common pool is then typically propagated down to other supplies in the structure. However if the **PartSource.Pool** field is populated for a part with this setting (to pooling does not apply), then that pool can potentially be injected on supplies for lower-level components that use the "Net" or "Optional" settings (rather than planning them in the common pool). • **Net**—normal Inventory Pooling is performed based on the stated pool. Pool mappings (if any) are applied, and forecast is then consumed by pool, and netting performed by pool. Pool value on planned supplies is typically populated with the value of Pool being netted (either the value mapped in PoolMap or PoolMapOverride table, or if no mappings are provided the pool value from the demand is used). Note that the specific pool and part source used for planned orders is further dependent on the **PoolFlowToCommon** setting. • **Optional**—Inventory Pooling is performed using either the mapped pool (if any) or else the common pool. That is, if the demand's pool is listed in either the PoolMap table (based on PartType) or PoolMapOverride table (based on Part) then that pool is used. Otherwise, if no matching mappings are provided, the common pool is used. Note that specific pool and part source used for planned orders is further dependent on the **PoolFlowToCommon** setting. • **Forecast**—consume forecast by Pool but do not net by Pool. Apply any specific Pool Mapping for the part or part type to Actual and Forecast demands. Consume the forecast, and then treat all supplies and demands as CommonPool. When SalesActual and SalesForecast demands are processed, they are segregated in Model, Unit, and Pool combinations based on the input values and the pool mapping. • **ForecastOptional**—consume forecast by Mapped pool (if there is any mapping available, otherwise the CommonPool is used). Netting of unconsumed forecast is done by	String (Enum)	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
	CommonPool.		
SafetyStockRule	Identifies how safety stock calculations are performed for parts that have model, unit, and/or pool logic enabled. Valid values are: • **ByPool**—Safety stock calculations are enabled for all parts that have model, unit, and/or pool logic enabled. Safety stock is planned separately for each pool applicable to the part, but always using the default model and unit. Note that if using inventory pooling along with fixed safety stock requirements (where **SafetyStockQuantityRule** is set to "Fixed"), then the expected safety stock amounts might not be planned. If pools are specified on part sources, inventory, or demands, and a fixed safety stock quantity is required, it is recommended to use a **SafetyStockQuantityRule** of "TimePhasedQuantity" instead (the **TImePhasedSafety** table allows specifying safety stock levels by part and pool). • **Common**—Safety stock calculations are enabled for parts that have model, unit, and/or pool logic enabled and use either fixed or time-phased safety stock quantities (in other words, dynamic safety stock rules are ignored). Safety stock is planned for the part, but only within the common pool, default model, and default unit. • **Ignore**— Safety stock calculations are disabled for parts that have model, unit, and/or pool logic enabled. In other words, safety stock is not planned. This setting corresponds to how safety stock was always handled for parts netted by model, unit, and/or pool in versions prior to 2013.4. **Default value:** Ignore **Note:** For more information, see "Safety stock and pools" on page 1749.	String (Enum)	

Table 7.30: MUEPoolNettingType (Mfg) fields

Field name	Description	Data type	Key
UnitRule	Identifies whether to process unit data. If the Model-Unit Effectivity is not enabled, and UnitRule is 'Net', the UnitRule is processed as Ignore, otherwise the UnitRule is processed as described. Valid values are: • **Block**—perform block netting for the part.For information about block netting, see "Block netting" on page 1334. • **ExplodeOnly**—does not net by unit, but demand is exploded by unit. StartUnit is ignored when allotting supply to demand, but planned orders are generated by the StartUnit of the triggering demand. When exploding demand to lower levels, the StartUnit on ScheduledReceipts and PlannedOrders is used to determine the bill of material to use. • **Fixed**—do not net by unit, but generate new StartUnit values for planned supplies, each one using the same Part.NextUnit value • **Ignore**—no unit netting is performed for this part. StartUnit information on scheduled receipts is retained and may be used for explosion, but no StartUnit data is copied to planned supply records (the 'Unit Undefined' state is used on planned supply. StartUnit is –1) • **Increment**—do not net by unit, but generate new StartUnit values for planned supplies, by incrementing the Part.NextUnit value, if >=0. Otherwise, simply copy the Part.NextUnit value). • **Net**—align supply and demand by StartUnit (use data audits to detect supplies or demands where the quantity is greater than one). Populate StartUnit in planned supply with the value of the unit being netted. • **NetWithoutForecast**—align supply and demand by StartUnit in netting calculations, but ignore the StartUnit when consuming forecast (actuals consume from "common" unit). **Default value**: Ignore	String (Enum)	
Value	The name representing this type of Model Effectivity, Unit Effectivity, or Inventory Pooling configuration. **Default value**: blank string	String	Yes
Parts	Set of **Part** records that reference this MUEPoolNettingType value	Set	

MultiEchelonSafetyStockRule table

The **MultiEchelonSafetyStockRule** table is referenced by the **Part** table and supports multi-echelon inventory optimization. When a part is eligible to be brought into one or more multi-echelon families, the settings in this table control how that part is processed by multi-echelon safety stock calculations.

For example, individual parts can be included in, or excluded from, multi-echelon safety stock calculations (this might be useful if enabling multi-echelon calculations on a part-by-part basis).

This table is also used to configure how each part's inventory holding costs are calculated (the costs are required use by the logic that makes safety stock recommendations based on minimizing the total cost of holding inventory across a network of parts).

Table 7.31: MultiEchelonSafetyStockRule (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this **MultiEchelonSafetyStockRule** record. **Referenced table:** ControlSet	Reference	Yes
CostRule	Specifies the source of the part's cost values for use in calculating inventory holding costs. The cost value as determined by this setting is subsequently multiplied by **Part.InventoryHoldingRate** to determine the per unit cost of holding inventory for the part. Valid values are: • **RolledUp**—inventory holding costs are derived from rolled up material, labor, and/or overhead costs on the part's components. • **StdUnitCost**— inventory holding costs are derived from the part's standard unit cost value. **Default value**: RolledUp **Note:** For more information about inventory holding cost calculations, see "Calculate inventory holding costs for MEIO" on page 1996.	String (Enum)	
Description	A description of this multi-echelon safety stock rule. **Default value**: blank string	String	

Table 7.31: MultiEchelonSafetyStockRule (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies how parts using this rule are processed by multi-echelon safety stock calculations. Valid values are: • **Ignore**—ignored by multi-echelon safety stock calculations. Parts referencing this rule should not typically be brought into multi-echelon families or used in inventory optimization calculations. • **NoInventory**—used by multi-echelon safety stock calculations, but not eligible to hold inventory. Parts referencing this rule can be brought into multi-echelon families and their parameters used in inventory optimization calculations for the family. However, a recommended safety stock level is typically not calculated for these parts. • **Use**—used by multi-echelon safety stock calculations. Parts referencing this rule can be brought into multi-echelon families and used in inventory optimization calculations for safety stock items. As required, a recommended safety stock level will be calculated for these parts. **Default value**: Ignore **Note:** For examples of how this setting is used in creating multi-echelon families, see"Create multi-echelon families" on page 1990.	String (Enum)	
Value	A unique identifier for this multi-echelon safety stock rule. **Default value**: blank string	String	Yes
Parts	The set of parts that reference this **MultiEchelonSafetyStockRule** record.	Set	

MultiLevelSearchRule table

Specifies whether multi-level search logic is enabled for parts that reference a given value in this table.

When multi-level search logic is enabled for a part with multiple part sources or substitutes, the sourcing decisions made can consider all child component materials and constraints in order to help choose the path that will provide the supply either on time or least late. Otherwise, the standard sourcing logic is used (for example, when choosing a part source, source effectivity, constraints, PTF date, and sourcing rules are considered, but unlimited component availability is assumed).

Enabling multi-level search also provides support for non-blocking allocations by modifying the supply allotment process to consider sibling component availabilities. As necessary, the allocation of a given component's supply to a demand is synchronized with the supply of the demand's gating component (if any). This allows earlier supply of the gating component's sibling to be re-directed to other demands and potentially improve their availability. Otherwise, the standard allotment logic is used (for example,

component supply available dates, demand due dates, and order priorities are considered, but no knowledge of any sibling component allocations is assumed).

For more information about use of multi-level search logic and potential limitation on other analytics, see "Multi-level search logic" on page 1863.

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this MultiLevelSearchRule record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this multi-level search rule. **Default value**: blank string	String	
Future Availability	Specifies how multi-level search logic should plan when an order is determined to be available at Future. Valid values are: • **Ignore**—If an order is determined to have "future" availability, then it is not planned. Thus, requirements are not driven down to lower-level components and constraint. • **Plan**—If an order is determined to have "future" availability, it is still planned thus driving requirements down to lower-level components and constraints. This setting corresponds to how planning was always performed in versions prior to 2016.2. **Default value**: Plan **Note:** This field was added in Version 2016.2, Service Update 5.	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
LoggingRule	Indicates whether to log and report details of end-item demands which multi-level search logic determined couldn't be planned within a given lateness tolerance and were therefore left unsatisfied (demands can be left wholly or partially unsatisfied if their DemandType.PlanningRule is set to "Partial", otherwise they should always be planned even if late). Along with details of the end-item demands that couldn't be planned or satisfied, the specific trial supplies that ultimately contributed to those demands being left unsatisfied is reported. For example, an end-item demand might be left unsatisfied because there was insufficient constraint available to allow a lower-level component supply to be planned when needed. Valid values are: • **None**—details of demands that were left unsatisfied and the trial supplies that couldn't be planned in time to satisfy those demands are not reported. • **UnsatisfiedDemand**—details of demands that were left unsatisfied, and the trial supplies that couldn't be planned in time to satisfy those demands, are reported. Table such as **MLSActivity**, **TrialPlannedOrder,** and **TrialConstraintShortage** can be populated. **Default value:** Ignore **Note:** This field was added in Version 2016.2, Service Update 5.	String (Enum)	
NewPurchasesCost	Specifies whether rolled up material cost from purchase orders is considered during the multi-level source selection process. This setting can be used to help avoid selecting very low-yielding and/or expensive part sources. Valid values are: • **Ignore**—Part source selection criteria does not include rolled up material cost from purchase orders. • **UseBeforeTarget**—Part source selection criteria considers rolled up material costs from purchase orders before it considers **MLSPreTargetSequence** and **Target** values. That is, if two part sources of the same priority can provide supply equally well and component supply types/depth are equivalent, then the one with the lower rolled-up material cost from component purchase orders would be chosen (regardless of targets). **Default value:** Ignore **NOTE**: This field was added in Version 2016.2, Service Update 23.	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
PartSourceLimit	The maximum number of effective part sources that can potentially be evaluated when planning supply to satisfy a given demand for the part. Because large numbers of part sources typically lead to multi-level search logic performing a larger number of trial operations, placing a practical limit on the number of sources evaluated can improve runtime performance. In particular, if there is a very large number of low-yielding part sources, then setting this field appropriately can have a performance benefit with very little impact on solution quality. Any negative value, or zero (0), indicates that no limit should be placed on the number of effective part sources. **Default value**: 100 **NOTE**: This field added in Version 2016.2, Service Update 17. **NOTE**: This limit applies to effective part sources only (those part sources where a given demand date falls within the effective in and out dates). Any ineffective part sources are not counted against the specified limit.	Integer	
ProcessingRule	Indicates whether child component materials and availability are considered when sourcing supply of part from multiple part sources or substitutes, and whether sibling component allotments are considered when allocating supply of the part to the demands that require it. Valid values are: • **Ignore**—standard sourcing and supply allotment logic is used. That is unlimited child component availability is assumed when making supply sourcing decisions, and sibling component supplies are allocated to demands independently of one another. • **Use**—standard sourcing logic is modified to include considerations of component materials and availability when sourcing supply from multiple potential part sources or substitutes, and standard supply allotment logic is modified to include considerations of sibling component allotments when allocating a part's supply to higher-level demands. Note that even if an assembly part is set to "Use", multi-level search logic stops at component part(s) in its structure that are set to "Ignore". **Default value**: Ignore	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
Scheduled SupplyRule	In multi-level search logic, this field is used to specify whether part sources where component supply comes from on hand inventory should be preferred to part sources where components come from scheduled supply (scheduled receipts, inventory transfers, and so on). For example, when two part sources of the same priority are eligible to source a supply on time (with no preference based on input supply level), then the lowest level of component supply used by each is compared. At this level of source selection, on hand and scheduled component supply are always preferred to new planned orders, however the setting in this field determines whether there is a preference between on hand supply and scheduled supply. Valid values are: • **EqualToOnHand**—When evaluating the lowest level of component supply used by part sources, on hand inventory and scheduled supply are treated equally (both are preferred to new planned orders). • **PreferOnHand**—When evaluating the lowest level of component supply used by part sources, on hand inventory is preferred to scheduled supply (both are preferred to new planned orders). **Default value:** PreferOnHand **Note:** This field was added in Version 2016.2, Service Update 8.	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
ShortTerm Excess	Specifies how to handle potential short-term excess inventory that can result from using multi-level search logic along with priority planning, dynamic safety stock, lot-sizing, or constraints. Valid values are: • **Allow**— Short-term excess resulting from by pre-building supply is allowed. This setting corresponds to how planning was always performed in versions prior to 2016.2. • **Reduce**—Attempts are made to reduce short-term excess resulting from pre-building supply. Multi-level search logic might pull-in or delay supplies in order to reduce near-term excess. **Default value:** Allow **Note:** This field was added in Version 2016.2, Service Update 5. **Note:** When using the "Reduce" setting, the pulling in or delaying of planned orders cannot move them outside of their part source's effectivity. Thus in cases of multiple part sources with short date effectivity ranges, the use of the "Reduce" setting can potentially introduce temporary excess supply and/or shortages.	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
Source EffectivityRule	Indicates which planned supply dates are compared against the effective in and out dates on a part source during multi-level source selection. This setting therefore determines whether rolled up component availability should be used in the determination of assembly part source effectivity, and hence whether component availability outside of an assembly's part source effectivity can result in the assembly's demand being left unsatisfied. Valid values are: • **AvailableDate**—Effective dates on a part source are first compared against the planned order due date and then against the date that multi-level sourcing logic expects the order to be available. If found to be effective during these Netting calculations, the part source effective dates are further compared against the order's available date as calculated by CTP logic. Thus, planned component availability is used in determining assembly part source effectivity, and if planned component availability falls outside the effective range, the part source will not be selected (even if this means a demand will be planned late or left unsatisfied). Note that if planned component availability falls outside of assembly part source effectivity, the assembly's demand is left unsatisfied and an exception with Type of "MLSSourceEffectivity" is logged in the DemandException table. However, if planned component availability falls within assembly part source effectivity but actual component availability (as determined in CTP logic) subsequently falls outside of assembly part source effectivity, no demand exception is logged but the demand is still given an AvailableDate of "Future". • **DueDate**—Effective dates on a part source are compared against planned order due date only. Thus, planned component availability is not used in determining part source effectivity on an assembly. If a part source is effective on the supply due date, but some or all component availability is expected to fall outside that effective range, the part source can still be selected. • **PlannedAvailableDate**—Effective dates on a part source are first compared against the planned order due date and then against the date that multi-level sourcing logic expects the order to be available. Thus, this option considers planned component availability during source selection for an assembly. If a part source is effective on the supply due date, but some or all component availability is expected to fall outside the effective range, the part source will not be selected	String (Enum)	

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
	(even if this means a demand will be planned late or left unsatisfied). Note that if a part's demand is left unsatisfied because planned component availability falls outside part source effectivity, an exception with **Type** set to "MLSSourceEffectivity" is logged in the **DemandException** table. **Default value:** DueDate **Note:** This field was added in Version 2016.2, Service Update 4. The DueDate option corresponds to how multi-level source selection was always done in earlier versions. For more information about this field and an example of its usage, see "Multi-level part source effectivity" on page 1865.		

Table 7.32: MultiLevelSearchRule (Mfg) fields

Field name	Description	Data type	Key
Unsatisfied Demand Logged QuantityRule	When reporting unsatisfied demands due to partial planning logic used with multi-level search, this field controls the extent of the logging related to those unsatisfied demands and the supplies that couldn't satisfy them on time or within lateness tolerance. This can affect the number of records generated in tables such as MLSActivity, TrialPlannedOrder, and TrialConstraintShortage. Valid values are: • **FirstLatePortion**—Only the first portion of the supplies that will be planned late to satisfy the demand within tolerance is reported. For example, if constraint causes supply of a given component is planned to be available in five different portions and three of those portions fall on dates outside of the driver part's lateness tolerance, only the first of those segments is logged and the other two excluded from logging. • **FullQuantityPrimaryPath**—The entire unsatisfied demand quantity, along with all supply portions that would be too late to satisfy it on time (or within lateness tolerance), is reported. Constraint consumption occurs as normal and, in case when there is a lack of constraint available, demand logging might not continue all the way to lower levels in the structure. With this setting, all supplies logged as being outside of tolerance for the unsatisfied demand quantity are reported as being sourced from the primary part source at each level. • **FullQuantityPrimaryPathAfterNetting**—The unsatisfied demand quantity and supply portions late to satisfy it are reported after Netting logic completes and in the order that the demands were shorted. On-time input supply consumption is allowed, but only supplies leftover from the usual netting process can be used. Constraint consumption is disabled during the downwards pass (thus constraint availability does not change where dependent demands get exploded). **Default value:** FirstLatePortion **Note:** This field was added in Version 2016.2, Service Update 8. For details about how the primary part source (path) is determined when logging the full unsatisfied demand quantity, see "Primary part source used in MLS logging" on page 1867.	String (Enum)	
Value	A unique identifier for this multi-level search rule. **Default value**: blank string	String	Yes
Parts	The set of parts that reference this MultiLevelSearchRule record.	Set	

OFConfiguration table

The **OFConfiguration** table contains settings used by resources available with the Order Fulfillment application. For example, it allows defining the maximum threshold for both supply-related and customer-requested changes allowed against any single line item before that line is flagged for attention in certain Order Fulfillment resources.

Table 7.33: OFConfiguration (Solutions) fields

Field name	Description	Data type	Key
ActualsCategory	The category used to indicate closed customer orders. Order fulfillment resources that report historical data do so based on historical demand records that belong to this category. **Referenced table:** HistoricalDemandCategory (Nullable)	Reference	
Calendar	Defines current intervals for Order Fulfillment. Usually Month or Quarter. **Referenced table:** Calendar (Nullable)	Reference	
MaximumNumberOfCustomerChanges	The default number of customer-initiated changes allowed on a line item before certain application resources apply formatting to flag the order as having reached the maximum change threshold. When the maximum change threshold is reached, the order might typically require further investigation and/or being placed on hold pending resolution of the underlying customer issue. **Note:** If necessary, this change threshold can be specified for individual customers using the **MaximumNumberOfCustomerChanges** field on the **Customer** table.	Integer	
MaximumNumberOfSupplyChanges	The default number of supply changes allowed against any line item before certain application resources apply formatting to flag the order as having reached the maximum threshold. When the maximum change threshold is reached the order might typically require further investigation and/or being placed on hold pending resolution of the underlying supply issues. **Note:** If necessary, this change threshold can be specified for individual customers using the **MaximumNumberOfSupplyChanges** field on the **Customer** table.	Integer	

Table 7.33: OFConfiguration (Solutions) fields

Field name	Description	Data type	Key
RequestedStatus	A reference to the value in the **DemandStatus** table used to indicate a new customer request that has not yet been processed in RapidResponse. This should be a record with the **ProcessingRule** set to "Ignore". All new customer requests being brought into the Order Fulfillment application should reference this same **DemandStatus** value. **Referenced table:** DemandStatus (Nullable)	Reference	

OnHandType table

The **OnHandType** table is a control table that specifies whether the stock or inventory is considered in netting. In other words, this table specifies the rules for determining the availability of OnHand records in netting.

Table 7.34: OnHandType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this OnHandType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this type of OnHand. An explanation of the code in the Value field. **Default value**: blank string	String	
ProcessingRule	Specifies whether the stock is used in netting. Valid values are: • **Nettable** — considered in inventory netting calculations. • **NonNettable** — not considered in inventory netting calculations. **Default value**: Nettable	String (Enum)	
Value	The string value for this type of OnHand record. **Default value**: blank string	String	Yes
OnHands	Set of **OnHand** records that reference this OnHandType value.	Set	

OperationSequenceType table

The **OperationSequenceType** table contains processing rules applied to the name sequences of operations stored in the **OperationSequence** table. For example, it determines if a given sequence of

operations is the standard sequence in its routing or a parallel operation branching of the standard sequence.

Table 7.35: OperationSequenceType (Core) fields

Field name	Description	Data type	Key
AlignmentRule	When the duration of operations in a parallel sequence is different than the duration between the branch points in the standard sequence, this setting determined where those sequences are aligned. Valid values are: • **Earliest**—parallel sequences are aligned as early as possible. For example, if a parallel sequence is of shorter duration than the time between the branch and return operations in the standard sequence, the parallel sequence aligns with the start of the branch operation (with a float assumed at the end) • **Latest**—parallel sequences are aligned as late as possible. For example, if a parallel sequence is of shorter duration than the time between the branch and return operations in the standard sequence, the parallel sequence aligns with the end of the return operation (with a float assumed at the start). **Default value**: Earliest	String (Enum)	
ControlSet	The control set associated with this **OperationSequenceType** record. **Referenced table:** ControlSet	Reference	Yes
Description	An optional description of this operation sequence type.	String	
ProcessingRule	Specifies how a given operation sequence within a routing is processed. Valid values are: • **Ignore**—The operation sequence, and hence all of its operations, are ignored by the routing. • **Parallel**—The operation sequence is a parallel sequence that branches off the standard sequence in the routing. **OperationSequence** records that reference a type with this setting should have their **BranchOperation** and **ReturnOperation** fields populated to indicate the branching points from and to the standard sequence. • **Standard**—The operation sequence is treated as the standard sequence in the routing. Each routing should have one **OperationSequence** record referencing a type that uses this setting (all operations within a routing are ignored if it does not have a standard operation sequence). **Default value:** Ignore	String (Enum)	

Table 7.35: OperationSequenceType (Core) fields

Field name	Description	Data type	Key
Value	A unique identifier for this operation sequence type.	String	Yes
Operation Sequences	Set of **OperationSequence** records that reference this **OperationSequenceType** value.	Set	

OperationType table

This table controls the processing rules used for scheduling an operation.

Table 7.36: OperationType (Mfg) fields

Field name	Description	Data type	Key
BatchQuantityRule	When an operation has a batch quantity value of 0, this setting determines how it can process batched supplies received from the previous operation in the sequence. Valid values: • **IgnoreZero**—the operation can begin once it has received a single batch of supply from the previous operation in the sequence. • **UseZero**—the operation can begin only when the entire supply has been received from the previous operation in the sequence.	String (Enum)	
ControlSet	The control set associated with this OperationType record. **Referenced table:** ControlSet	Reference	Yes
DateRule	Specifies how date effectivity is to be applied. (DateRule is always applied, regardless of whether Model-Unit Effectivity or Pool have been enabled). Valid values are: • **AnchorDate**—tests the effective dates against the anchor date on the supply record. • **BlowThroughCalcDueDate**—tests the effective dates against the calculated due date on the supply record. Same as CalcDueDate. • **CalcDockDate**—tests the effective dates against the calculated dock date on the supply record. • **CalcDueDate**—tests the effective dates against the calculated due date on the supply record. • **CalcStartDate**—tests the effective dates against the supply record start date or calculated start date for planned loads • **Ignore**—ignores the effective dates (record treated as if effective from Past to Future) **Default value**: Ignore	String (Enum)	

Table 7.36: OperationType (Mfg) fields

Field name	Description	Data type	Key
DependentUnit Rule	This field supports Model and Unit- Effectivity. It defines how dependent load is to be handled with unit effectivity. Valid values are: • **Split**—split load according to the unit number range. Loads are not split if the supply's unit is less than zero. • **First**—calculate the entire load according to whether the supply StartUnit passes the unit range on the Bill record **Default value**: Split **Note:** If your company has not enabled Model-Unit Effectivity, this field is ignored.	String (Enum)	
Description	A description of this type of operation. An explanation of the code in the Value field. **Default value**: blank string	String	
EffectivityRule	Specifies how ranges of units and dates are to be interpreted when determining effectivity. This field applies to all effectivity rules (ranges apply to dates and unit ranges). Valid values are: • **InclusiveExclusive**— effective starting on start and ending before the end (recommended method). • **Never**— ignore this operation record regardless of all dates as well as Model and Unit-Effectivity rules. • **Always**— ignore all effectivity rules and treat the operation as active. • **Exclusive**— use effectivity dates and units if applicable (not effective on the dates or units). • **Inclusive**— effective from start to end inclusive. **Default value**: InclusiveExclusive	String (Enum)	

Table 7.36: OperationType (Mfg) fields

Field name	Description	Data type	Key
MUERule	This field supports, and is used to define application of, the Model and Unit-Effectivity analytics. Valid values are: • **Model**— the operation record is effective if the Model value on the operation record matches the Model on the Supply record • **Unit**— the operation record is effective if the StartUnit on the Supply record is in the range of the StartUnit and EndUnit from the operation record. DependentUnitRule and EffectivityRule are tested to determine whether the unit is in range. • **ModelUnit**— use both model and unit tests simultaneously. Model matches and the unit must be in range for the operation record to be effective. • **Ignore**— do not test model or unit for this operation record **Default value**: Ignore **Note:** If your company has not enabled Model-Unit Effectivity, MUERule is treated as Ignore.	String (Enum)	
TransitTime ProcessingRule	Valid values include: • **Hours** — interprets the TransitTime as the number of hours to move between operations • **Days** — interprets the TransitTime as the number of workdays it takes to move between operations without rounding (for example, moving from two hours into the day on operation 10 to operation 20 with one TransitTime Days, would move the time to two hours into the next workday). This rule should only be used with integer days and a fractional number will be rounded up to the next integer before calculating the transit time ending point. • **DaysRounding** — interprets the TransitTime as the number of workdays it takes to move between operations and start the next operation at the beginning of that day (for example, TransitTime would move ahead 1 day and if it is not at the beginning of the day, it will be moved to the beginning of the following workday). This rule should only be used with integer days and a fractional number will be rounded up to the next integer before calculating the transit time ending point. **Default value**: Hours	String (Enum)	

Table 7.36: OperationType (Mfg) fields

Field name	Description	Data type	Key
TransitTime SequenceRule	Controls whether to apply transit time before the queue time or after the wait time of the operation. Valid values are: • **BeforeOperation** — the time it takes to move from the previous operation to this operation • **AfterOperation** — the time it takes to move from this operation to the next operation. **Default value**: AfterOperation	String (Enum)	
Value	The string code for this type of operation. This is what will appear in the Operation records. **Default value**: blank string	String	Yes
CRPOperations	Set of **CRPOperation** records that reference this OperationType value.	Set	
Operations	Set of **Operation** records that reference this OperationType value.	Set	

OrderPolicy table

The **OrderPolicy** table is a control table that specifies the rules for determining the size and dates for placing orders. The start date is the next higher integer number of days from FixedLeadTime + (VarLeadTime * Quantity).

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this OrderPolicy record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of the OrderPolicy. **Default value**: blank string	String	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
LotSizeLastOrder	Specifies, for each order policy, whether to apply lot-sizing rules to the last planned order in the horizon. Lot-sizing rules include minimum order quantities (PartSource.MinimumQty), maximum order quantities (PartSource.MaximumQty), and multiple order quantities (PartSource.MultipleQty). Valid values are: • **Y**—applicable lot sizing rules are applied to the last planned order as they are applied to any other planned order. • **N**—lot-sizing rules are not applied to the last planned order. Only the quantity required is ordered; that is, the **PlannedOrder.NeedQty** and **PlannedOrder.Quantity** fields are the same. **Default value:** Y	Boolean	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
MaximumUsage	Specifies how the maximum order size, if any, is determined for each part source using this policy. Valid values are: • **Average**—if the required quantity exceeds **PartSource.MaximumQty**, use the **MaximumQty** to determine the number of planned orders to generate, and then average the required quantity across these orders. This option allows a given requirement to just be covered, respecting any Min, Max, or Mult policies, without bringing any supply in earlier than necessary. In order to use this option, a given requirement must have exactly one part source that can satisfy it, and the value of **PartSource.TakTime** for that source must be set to 0. If these conditions are not met, this option is equivalent to the "Use" option. • **CampaignPlanning**—each planned order created is for the exact quantity (batch size) that is specified in PartSource.MultipleQty. This setting plans supplies in production campaigns with the minimum and maximum quantities allowed per campaign then determined by the MinimumQty and MaximumQty fields on the PartSource record. Therefore, the minimum number of batches per campaign is calculated as `MinimumQty/MultipleQty`, while the maximum number of batches per campaign is calculated as `MaximumQty/MultipleQty`. Additionally, if the part source is constrained, then all batches in a given campaign must be scheduled through a contiguous block of the constraint. This option supports the optional Campaign Planning capability. • **Ignore**—do not use **PartSource.MaximumQty** value in determining planned order quantities. • **Kanban**—each planned order created for the part source is for the quantity defined by **SourceKanban.BinQty** (the current approved bin quantity). As well, the total supply of the associated part must not exceed the value of **SourceKanban.BinQty** * **SourceKanban.Bins** (the current number of bins in the system). When this setting is used, the values of **PartSource.MultipleQty, PartSource.MinimumQty**, **PartSource.MaximumQty** are all ignored. Also, if a part source using this setting has no corresponding entries in the **SourceKanban** table, then both **BinQty** and **Bins** are treated as zero and no planned orders are generated against the source. • **OverRideDaysSupply**—cover at least one day's requirements. Will not cover days of supply if the quantity would exceed maximum. (Note: this value is obsolete and is treated the same as "Use".) • **Use**—use the value in the **PartSource.MaximumQty** in determining the **PlannedOrder.Quantity**. The planned	String (Enum)	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
	orders that are generated will be less than or equal to the applicable **PartSource.MaximumQty** (may be more than one applicable part source) based on the required demand. It's possible that there could be more than one planned order for the same part source on the same day if the amount needed exceeds **PartSource.MaximumQty** (either the demand that is being satisfied or the demands if **Part.NumberofDaysSupply** is also used). **Note:** A global setting limiting the number of planned orders per day can result in orders that exceed **PartSource.MaximumQty**. For more information, see "Limiting the number of planned orders" on page 2204. **Default value:** Ignore		
MinimumUsage	Specifies the minimum usage setting for each order policy: • **Ignore**—do not use the **PartSource.MinimumQty** value in determining the **PlannedOrder.Quantity**. • **Use**—use the value in the **PartSource.MinimumQty** in determining the **PlannedOrder.Quantity**. The planned orders that are generated will be greater than or equal to the applicable **PartSource.MinimumQty** (there might be more than one applicable PartSource) based on the required demand. • **RoundMult**—use the value in the **PartSource.MinimumQty** rounded up to the value in the **PartSource.MultipleQty** in determining the **PlannedOrder.Quantity**. The PlannedOrders that are generated will be greater than the applicable **PartSource.MinimumQty** (there might be more than one applicable PartSource) but rounded to a multiple of the **PartSource.MultipleQty**. **Default value:** Use	String (Enum)	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
MultipleUsage	Specifies the multiple usage setting for each order policy: • **Ignore**—do not use the **PartSource.MultipleQty** value in determining planned order quantities. • **Use**—use the value in the **PartSource.MultipleQty** in determining the **PlannedOrder.Quantity**. The planned orders that are generated will have a quantity that is in increments (multiples) of the applicable **PartSource.MultipleQty** (there might be more than one applicable part source) based on the required demand quantity. • **Units**—the value in **PartSource.MultipleQty** is ignored and treated as if it was set to "1". As necessary, order quantities are rounded up to the next whole number (intended to ensure supply is not planned in fractional quantities). **Default value:** Use **Note:** If **MaximumUsage** is set to either "Kanban" or "CampaignPlanning", then this setting is ignored and planned orders are always set to the effective bin or batch size, respectively.	String (Enum)	
OrderGeneration Rule	Controls how RapidResponse generates planned orders. Valid values are: • **NoOrders**—do not generate planned orders • **AnyTime**—generate planned orders whenever demand requires it (including the past) • **AfterRunDate**—generate planned orders but only after **Part.PlanningCalendars.RunDate.FirstDate** • **AfterPTF**—generate planned orders but only on or after **PartSource.PTFDate**. Note that the calculation of **PTFDate** is affected by the setting in the **PTFRule** field. **Default value:** AnyTime	String (Enum)	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key

PTFRule	Specifies how the planning time fence date (**PartSource.PTFDate**) is calculated for part sources that use this order policy.
	Typically, the planning time fence might be set to a part's lead time or to a specified number of work days. If the **OrderGenerationRule** is set to "AfterPTF", the calculated **PTFDate** then represents the earliest date on which a new planned order from the part source is allowed to be due.
	Valid values are:
	• **Fence**—The **PTFDate** is determined by adding the **Part.PlanningTimeFence** value to **RunDate**.
	Note also that the **PTFUnit** setting affects this calculation as follows:
	  • If **PTFUnit** is set to "Part" then **PartSource.PTFDate** is calculated as (Part.PlanningCalendars.RunDate.FirstDate + PlanningTimeFence expressed in Part.PlanningCalendars.TimeUnits).
	  • IF **PTFUnit** is set to "Source" then **PartSource.PTFDate** is calculated as (Part.PlanningCalendars.RunDate.FirstDate + PlanningTimeFence expressed in Source.LeadTimeUnit), converted to Part.PlanningCalendards.TimeUnits date.
	• **LastDueFence**—the **PartSource.PTFDate** is calculated as the latest of either the **Fence** calculation (described above) or the **DueDate** of the last scheduled receipt or supply allocation assigned to the part source (expressed in **Part.PlanningCalendars.TimeUnits)**. This setting is similar to "LastDueFenceAll" but only considers the current part source.
	• **LastDueFenceAll**—the **PartSource.PTFDate** is calculated as the latest of either the **Fence** calculation (described above) or the **DueDate** of the last scheduled receipt or supply allocation assigned to any part source that provides the part (expressed in **Part.PlanningCalendars.TimeUnits)**. This setting is similar to "LastDueFence" but might be used when have the planning time fence synchronized across all of a part's sources is important.
	• **LastDueLead**—the **PTFDate** is calculated as the latest of either the **Lead** calculation (described below) or the **DueDate** of the last scheduled receipt or supply allocation assigned to the part source (expressed in **Part.PlanningCalendars.TimeUnits**). This setting is similar to "LastDueLeadAll" but only considers the current part source.
	• **LastDueLeadAll**—the **PTFDate** is calculated as the latest of either the **Lead** calculation (described below) or the
		String (Enum)	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
	DueDate of the last scheduled receipt or supply allocation assigned to any part source that provides the part (expressed in **Part.PlanningCalendars.TimeUnits**). This setting is similar to "LastDueFence" but might be used when have the planning time fence syncrhoznied across all of a part's sources is important. • **Lead**—The **PTFDate** is determined by adding both the part's lead time and **Part.PlanningTimeFence** value to **RunDate**. Thus, if the planning time fence should represent lead time only, ensure to leave **PlanningTimeFence** set to 0. Note also that the **PTFUnit** setting affects this calculation as follows: • If **PTFUnit** is set to 'Part' then **PartSource.PTFDate** is calculated as (Part.PlanningCalendars.RunDate.FirstDate as a start date, converted to a due date) + PartSource.PlanningTimeFence expressed in Part.PlanningCalendars.TimeUnits. • If **PTFUnit** is set to 'Source' then **PTFDate** is calculated as (Part.PlanningCalendars.RunDate.FirstDate + PlanningTimeFence expressed in Source.LeadTimeUnit) as a start date converted to a due date. **Default value:** Fence **Note:** With any of the "LastDue" options, supplies can be included or excluded from the **PTFDate** calculation by using the **PTFLastDueRule** setting on the SupplyType, SupplyStatus, and SupplyAllocationType tables. By default, all scheduled receipts and supply allocations are included. **Note:** This field was modified in Service Update G1908.		

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
PTFUnit	Affects calculations performed by settings in the **PTFRule** field when **Source.LeadTimeUnit, Source.ShipCalendar**, and **Part.PlanningCalendars.TimeUnits** do not all refer to the same Calendar (for example, this is often the case with "transfer" part sources). In such cases, this setting determines the time unit calendar (part or source's time units) to be used when calculating **PartSource.PTFDate**. Valid values are: • **Part**—Use when the planning time fence should be in the part's time units (**Part.PlanningCalendars.TimeUnits**), and applied after lead time when **PTFRule** is either "Lead", LastDueLead, or "LastDueLeadAll". • **Source**—Use when the planning time fence should be in the source's time units (**PartSource.Source.LeadTimeUnit**), and applied before lead time when **PTFRule** is either "Lead", "LastDueLead", or "LastDueLeadAll". **Default value**: Source	String (Enum)	
UseFixedLeadTime	Specifies for each order policy whether to consider the fixed lead time during MRP processing. Valid values are: • **Y**—the **PartSource.FixedLeadTime** is used in the calculation of **ScheduledReceipt.CalcStartDate**, **PlannedOrder.StartDate**, or both. • **N**—the **PartSource.FixedLeadTime** is not used in the calculation of **ScheduledReceipt.CalcStartDate**, **PlannedOrder.StartDate**, or both. **Default value**: N	Boolean	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
UseVarLeadTime	Specifies for each order policy whether to enable variable lead time during Netting calculations. Variable lead time is dependent on order quantity and used in calculating the start date on supplies (**ScheduledReceipt.CalcStartDate** or **PlannedOrder.StartDate**). Depending on the option selected in this field, variable lead time can be calculated based on a factor provided on the part source, set to the calculated time needed to run the order through all operations in its routing, or ignored altogether. Valid values are: • **CRP**—Variable lead time is enabled for supplies from this part source. To determine this portion of lead time, the time needed to run the supply through all work center operations in its routing is used. This can include float times specified in the **PartSource.CRPFloatBefore** and/or **PartSource.CRPFloatAfter** fields. Note that if the **SupplyType.CapacityProcessingRule** setting applicable to the supply is "Ignore", then this option is treated the same as "N". For more information about variable lead time based on work center capacity, see "Operations and supply lead time" on page 1532. • **N**—Variable lead time is ignored for supplies using this part source. • **Y**—Variable lead time is enabled for supplies from this part source. To determine this portion of lead time, **PartSource.VarLeadTime** is multiplied by either **PlannedOrder.Quantity** or **ScheduledReceipt.Quantity**. **Default value**: N	String (Enum)	
Value	The string code for this OrderPolicy. This is what will appear in the part record. **Default value**: blank string	String	Yes
YieldRounding Usage	Specifies whether decimal place rounding is applied to effective planned order and scheduled receipt quantities. Rounding can reduce or eliminate fractional amounts on effective supply quantities after applying yield. Valid values are: • **Ignore**—no rounding is applied to the effective quantities that remain on scheduled receipts and planned orders after applying yield. • **Use**—effective quantities on scheduled receipts and planned orders are rounded down after applying yield. Rounding is to the number of decimal places specified in the **Part.UnitOfMeasure.RoundingPrecision** field. **Default value**: Ignore	String (Enum)	

Table 7.37: OrderPolicy (Mfg) fields

Field name	Description	Data type	Key
YieldUsage	Specifies how to interpret yield values at the part level. This setting is applied against the **PartSource.Yield** field (to account for scrap) and the **PartSource.CoProductYield** (to account for co-product and by-product supply). For planned orders, yield is applied against the quantity required by the demand to return the **PlannedOrder.Quantity** (the amount required in consideration of yield, with the net amount received into inventory reported in **PlannedOrder.EffQuantity**. For scheduled receipts, yield is applied against **ScheduledReceipt.Quantity** to return the **ScheduledReceipt.EffQuantity**. For more information, see "Yield expressions for planned orders and scheduled receipts" on page 1393. Valid values are: • **Ignore**—do not use the Yield factor. It ignores the **Yield** and **CoProductYield** fields on the PartSource (even if populated) and the yield is always assumed to be 1 (no loss). • **YieldFraction**—value from 0 through 1 (1 means no loss). • **YieldPercent**—value from 0 through 100 (100 means no loss). • **ScrapFraction**—value from 0 through 1 (0 means no loss). • **ScrapPercent**—value from 0 through 100 (0 means no loss). • **ScrapPiece**—allows a fixed quantity per order to be scrapped. • **InflationFraction**—indicates amount to inflate supply using formula *1 / (1 + PartSource.Yield)* • **InflationPercent**—indicates amount to inflate supply using formula *1 / (1 + PartSource.Yield * 0.01)* **Default value**: YieldFraction set to 1 **Note:** The CoProductYield field only uses the YieldFraction, YieldPercent, and Ignore values. It treats InflationPercent, ScrapPiece, and ScrapFraction, as YieldFraction, and treats ScrapPercent and InflationPercent as YieldPercent.	String (Enum)	
PartSources	Set of **PartSource** records that reference this OrderPolicy value.	Set	

OrderPriority table

The **OrderPriority** table contains values that can be used to prioritize the order in which demands are satisfied or to prioritize the order in which supplies are consumed. The priority associated with each record in the **OrderPriority** table is defined through its **PlanningPriority** field. Together **OrderPriority** and **TransactionSequence** (found on the **IndependentDemand** and **ScheduledReceipt** tables) operate together to provide an unlimited number of levels of priority. That is, demands having the same

planning priority and due date are then sorted by TransactionSequence. Note that TransactionSequence is only considered if the value of the **MaintainCommitments** field is set to Y. For information about order priority, see "Order priority calculations" on page 1765.

This table is editable, therefore records can be added and modified.

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
AllocationRule	Specifies how supplies are allocated to demands within a given order priority. Valid values are: • **FirstComeFirstServed**—Supplies are allocated to demands based on priority and due date. • **FairShare**—If **PartType.SupplyAllocationRule** is set to 'ByAvailable', supplies are allocated to demands within one **PlanningCalendars.AllocationCalendar** interval based on demand size, with larger demands receiving more of the earliest available supply. • **EqualShare**—If **PartType.SupplyAllocationRule** is set to 'ByAvailable', supplies are allocated to demands within one **PlanningCalendars.AllocationCalendar** interval based on the number of orders, with each demand receiving the same amount of the earliest available supply. **Default value**: FirstComeFirstServed **Note:** The "FairShare" and "EqualShare" options can subsequently be turned off, or limited within a specific date horizon, at the part-level. This can be set using the **PartType.SupplyShareRule** field. For example, parts at one level in the product structure might require limited supplies to be shared amongst High priority demands, while parts at another level in the product structure might not need supplies shared amongst their High priority demands.	String (Enum)	
ControlSet	The control set associated with this OrderPriority record. **Referenced table:** Core::ControlSet	Reference	Yes
Constraint Allocation Calendar	Reference to a calendar defining the period over which demands for parts that share a common constraint through their part source(s), and belong to the same priority level, are collected together for the purpose of sharing potentially limited constraint availability between those part sources. For example, if set to a weekly calendar, then eligible demand due in a given week are collected together and, and if there is an insufficient amount of a shared constraint, the available amount of that constraint is shared between part sources based on the quantities of the demands being satisfied. Used only if **ConstraintAllocationRule** is set to "FairShare". **Referenced table:** Calendar (nullable)	Reference	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
Constraint AllocationRule	Indicates whether any limited constraint availability is to be shared fairly between different part sources that might require it to satisfy demands in each period as defined by **ConstraintAllocationCalendar** (for example, workday or week). Valid values are: • **FairShare**—Demands of a given priority level that share a common constraint through their part source(s) are processed together within each **ConstraintAllocationCalendar** period. If there is limited constraint available within the period, then it shared between part sources based on the quantity of the demand(s) being satisfied by each. Note that at the individual part-level, the fair-sharing of constraints can be either disabled or limited to a specific time horizon, using the **PartType.ConstraintShareRule** setting. • **FirstComeFirstServed**—Demands are processed one by one. If there is limited constraint available it is allocated based on a sequence of factors including priority, due date, part name. This setting corresponds to how constraint calculations were always performed in versions prior to 11.2. **Default value**: FirstComeFirstServed **Note:** For more information, see "FairShareAllocation" on page 1679.	String (Enum)	
Description	A description for this order priority.	String	
IntendedDriver Type	Specifies the types of orders that are expected to use this priority. It is used by some RapidResponse legacy applications; not directly used by RapidResponse calculations. Valid values are: • ScheduledReceipt • IndependentDemand (default) • SafetyStock	String (Enum)	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
Maintain Commitments	Specifies whether the transaction sequence is considered part of priority such that due dates on existing orders are maintained when new orders are dropped in. Values are: • **Y**—transaction sequence is considered part of priority. Typically, this setting should be used for orders that are truly committed to production (for example, the highest priority independent demands). • **N**—transaction sequence is not considered part of priority. This is the default value, and should be used for most priorities. If the **PartType.CommitLevel** field is set to Low, transaction sequence is ignored regardless of the setting on this field. **Caution:** More memory is consumed when orders have **MaintainCommitments** set to "Y". Therefore, as noted above, this setting should be limited to only those orders that are truly committed to production.	String (Enum)	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
MLSAllocation Rule	This setting controls how demands of a given priority level are processed in Netting when multi-level search logic is enabled. When there is insufficient material or constraint to cover all demands configured for "partial planning", this setting determines whether shortages throughout the product structure will be fairly shared between driver demands for a given period and priority level. Valid values are: • **FirstComeFirstServed**—Demands of a given priority level are processed individually and in due date order. This setting corresponds to how multi-level search logic always worked prior to Version 2016.2, Service Update 10. When demands of a given priority have to be left unsatisfied due to partial planning logic, then the shortages will tend to be unevenly distributed (limited material and/or constraint will go to demands that are processed first). • **Fairshare**—Within each allocation interval, demands of each priority level are broken up and processed iteratively. After demands are broken into defined increments, Netting then processes a defined portion of each demand before looping through the incremental demand portions again and this process continues until all demands have been fully processed. When multi-level search is used together with partial planning logic, this fair-sharing of constraints or material can help ensure that if some demand within a given period and priority level has to be left unsatisfied, then those shortages will be fairly distributed within each period. With this setting, the **MLSFairShareCalendar** and **MLSFairSliceFactor** fields define how demands are broken up for iterative processing. **Default value:** FirstComeFirstServed **Note:** This field was added in Version 2016.2, Service Update 10. Demands for parts where **PartType.CommitLevel** is set to "Medium" or "Low" are always processed as if this field were set to "FirstComeFirstServed".	String (Enum)	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
MLSFairshare Calendar	A reference to the calendar used to define the period over which demands of a given priority level are processed together in multi-level search logic. For example, all demands due in each Week might be processed together and any shortages fairly shared between those weekly demands. If this reference is left Null, then an Everyday calendar is used. **Referenced table:** Calendar (Nullable) **Note:** This field was added in Version 2016.2, Service Update 10, and only applies when the **MLSAllocationRule** field is set to "Fairshare".	Reference	
MLSFairshare SliceFactor	Specifies the number of incremental portions into which demands of a given priority level are split for processing by multi-level search logic. For example, if is set to "4", then a demand for 80 would be split and processed in four separate increments of 20 units. The value specified in this field should be between 1 and 10. Any value less than 1 is treated as 1 (demands are processed in whole), and any value greater than 10 is treated as 10 (at least 10% of each demand must be processed at a time). Note also that any splits for multi-level fair share logic will be rounded down to respect the part's **AllocationMultiple** setting. **Default value:** 1 **Note:** This field was added in Version 2016.2, Service Update 10, and only applies when the **MLSAllocationRule** field is set to "FairShare".	Integer	
Optimization Priority	Priority used by certain legacy applications in calculating customer service objective. High values indicate high logical priorities, and relative values are considered significant. For example, orders with an optimization priority of 10 are twice as important as those with an optimization priority of 5. Zero effectively excludes independent demands with this order priority form the customer service optimization objective. **Default value**: 0 **Note:** This field is obsolete.	Quantity	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
PeriodEffective	Specifies whether this type of priority is dependent on the period the demand occurs in. Valid values are: • **Y**—the priority value is period dependent. Within the current period, as defined by Part.PlanningCalendars.AllocationCalendar, demand priorities are taken from the PlanningPriority field. However, any unsatisfied demands from this interval are assigned a higher priority than demands in subsequent periods regardless of their PlanningPriority value. This setting can be used to try and ensure older past due demands are satisfied before demands in the current period. • **N**—the priority value is not period dependent. Demand priorities are always taken from the PlanningPriority field regardless of the period in which the demand occurs. **Default value:** N **Note:** For more information, see "Period effective priority" on page 1768.	Boolean	
PlanningPriority	The relative importance associated with each OrderPriority value. Lower values are considered the higher effective priority. Different OrderPriority records can share the same PlanningPriority value. If multiple orders share the same PlanningPriority value, they are treated the same, even if their OrderPriority differs. Therefore, it is recommended that you give each OrderPriority record a distinct PlanningPriority value.	Integer	
SchedulePriority	The constraint scheduling priority for supplies associated with this OrderPriority value (where lower values are considered the higher effective priority). SchedulePriority can be used to ensure high-priority supplies consume constraint as close to the due date as possible. If constraint availability then forces some supplies to be pre-planned, lower priority supplies are pre-planned earlier (and therefore held in inventory longer than higher priority supplies). This is useful in cases where high priority supplies are also higher value supplies. For more information on this field and considerations around its usage, see "SchedulePriority" on page 1659. **Default value**: 0	Integer	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
TwoDate PlanningRule	Specifies how demands of this priority are used in two-date planning logic. Typically, the capable-to-promise analytic allots supplies to demands to try and satisfy the date on which order shipment has been committed to the customer (**IndependentDemand.DueDate**). When two-date planning is enabled, after first planning to the due date, RapidResponse will try to move demand availability towards the original date on which the customer requested shipment of the order (**IndependentDemand.RequestDate**). Note that such improvements in demand availability are only made if they do not adversely affect the available date on other demands (unless those other demands belong to a lower priority level with a setting of "Ignore" in this field). For more information about two-date planning logic, see "Two-date planning" on page 1803. Valid values are: • **Ignore**—Two-date planning is not enabled for this priority. Demands of this priority level are only planned to try and satisfy their due date. Additionally, these demands might have their available dates adversely impacted by attempts to meet the request date on higher priority demands that are enabled for two-date planning. • **Protect**—Two-date planning is not enabled for this priority. Demands of this priority level are only planned to try and satisfy their due date. However, these demands are protected and will never have their availabilities made worse by attempts to meet the request date on higher priority demands enabled for two-date planning. • **Use**—Two-date planning is enabled for this priority. Demands of this priority level are first planned to their due date (less applicable safety lead time) by Netting logic. CTP calculations then initially try to satisfy that same date, before a second pass ultimately attempt to move the demand closer to the original customer request date (if possible). In order to try and achieve their RequestDate, these demands might adversely impact the available date of lower-priority demands that use the "Ignore" setting. • **UseWithHardSafetyLeadTime**—Two-date planning logic is enabled for this priority, and request dates are adjusted for safety lead time. Netting logic will plan to cover demands of this priority at the demand due date less safety lead time for the part. Capable-to-promise logic will initially try to satisfy the demand at its due date less safety lead time, and in a second pass will then	Reference	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
	try to satisfy the demand at its request date less safety lead time if this can be done without making other demands worse off (other than lower priority demands that use the "Ignore" setting in this field). Note though that within a given planning priority, demands that use this setting are processed before demands that use the "UseWithSoftSafetyLeadTime" setting. Thus, a demand with this setting might have its safety-lead time adjusted demand date satisfied even if it makes a "UseWithSoftSafetyLeadTime" demand late for its (non-adjusted) demand date. • **UseWithSoftSafetyLeadTime**—Two-date planning logic is enabled for this priority, and request dates are adjusted for safety lead time. Netting logic will plan to cover demands of this priority at the demand due date (does not apply safety lead time). Capable-to-promise logic will initially also try to satisfy the demand at that same due date, and in a second pass will then try to move the demand availability closer to its request date less safety lead time if this can be done without making other demands worse off (other than lower priority demands that use the "Ignore" setting in this field). **Default value**: Ignore **Note:** Ensure that supplies and part sources satisfying demands enabled for two-date planning have their **SupplyStatus.AvailableDateLimit** field set to "CalcRequestStartDate". This includes both supplies for the end-item itself as well as any component supplies needed to build the end-item. The "CalcRequestStartDate" setting ensures that supplies can be built early enough to satisfy the original customer request date. **Note:** Parts that have demands enabled for two-date planning should typically use a **PartType.ATPCTPRule** setting of "Latest" in order to allow pulling supplies in to the request date while reducing inventory carrying cost. If using the default **ATPCTPRule** of "Earliest", then results might not be as expected since this setting will try to ensure demands are satisfied with later supplies (assuming it doesn't impact the ability to satisfy the demand on time). This can leave earlier supply as excess rather than using it to try and meet customer request dates. **Note:** If demands of priorities using either the "UseWithSoftSafetyLeadTime" or "UseWithHardSafetyLeadTime" setting do not have a RequestDate populated, then CTP logic will ultimately try to satisfy the demands at their due date less safety lead time instead.		

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
	Note: This field was modified in Version 2016.2, Service Update 6.		
TwoDatePriority Limit	A calculated field reporting the highest **PlanningPriority** value found on any **OrderPriority** record, in a given control set, where the **TwoDatePlanningRule** setting is either "Use" or "Protect". This represents the lowest priority level whose demands are either enabled for, or protected against, two-date planning logic. Demands at priority levels below this one might be subject to having their available dates adversely impacted when two-date planning logic attempts to meet the request date on higher priority demands. Note also that if a record has a lower **PlanningPriority** (higher priority) than the value reported this field and has a **TwoDatePlanningRule** of "Ignore", it will be treated as if it were set to "Protect" instead.	Integer	
Value	The string value for this type of order priority.	String	Yes
DemandTypes	Set of **DemandType** records that reference this OrderPriority value.	Set	
Independent DemandsUnder OrderPriority	Set of **IndependentDemand** records that reference this OrderPriority value through their **OrderPriority** field.	Set	
Independent DemandsUnder SavedPriority	Set of **IndependentDemand** records that reference this OrderPriority value through their **SavedPriority** field.	Set	
PartCustomer TimePhased Attributes	Set of **PartCustomerTimePhasedAttributes** records that reference this OrderPriority value.	Set	
PartCustomer Types	Set of **PartCustomer** records that reference this OrderPriority value.	Set	
PartTypes	Set of **PartType** records that reference this OrderPriority value.	Set	
Scheduled ReceiptsUnder OrderPriority	Set of **ScheduledReceipt** records that reference this OrderPriority value through their **OrderPriority** field.	Set	

Table 7.38: OrderPriority (Mfg) fields

Field name	Description	Data type	Key
Scheduled ReceiptsUnder SavedPriority	Set of **ScheduledReceipt** records that reference this OrderPriority value through their **SavedPriority** field.	Set	
Supply Allocations	Set of **SupplyAllocation** records that reference this OrderPriority value.	Set	
TimePhasedSafetys	Set of **TimePhasedSafety** records that reference this OrderPriority value.	Set	

OutlierType table

The **OutlierType** table contains rules used in the detection of outliers in historical data for items configured for statistical forecasting and for inventory planning and optimization. Before an item's statistical forecast is generated or before an item's safety stock recommendations are calculated, rules in this table determine how outliers in historical data are identified as well as how to adjust any outliers that are detected.

For more information about using this field in the statistical forecasting process, see "Define outlier handling" on page 2022.

This table was modified in Version 2016.2, Service Update 5, to support the Inventory Planning and Optimization application. For more information about its usage in the safety stock configuration process, see "Safety stock item outlier processing" on page 1983.

Table 7.39: OutlierType (Mfg) fields

Field name	Description	Data type	Key
AboveThresholdRule	If an outlier is detected above the upper threshold, this rule specifies the type of adjustment made to reduce the historical actual quantity. The selected option determines the value used in place of the outlier quantity. Valid values are: • **Forecast**—For statistical forecasting only, the item's calculated value from **StatisticalForecastOutlier.Forecast** is generally used. The calculation of **Forecast** depends on the **DataRule** setting in this table as follows. If using the "MovingAverageError" setting, then the moving average up to the point where the outlier occurs is used. If using the optional "RstlError" setting, then the sum of the trend and seasonal components is used. If using the "Historical" setting, then **Forecast** returns -1 and the mean is used instead. • **Ignore**—the item's historical actual quantities are left as is without any outlier adjustments. • **Mean**—the item's average quantity across the historical demand actual periods is used. • **Median**—the item's median quantity across the historical demand actual periods is used. • **SmoothKeepExcess**—the item's outliers quantities are adjusted by using parameters defined on the item to smooth the quantity above the outlier threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains above the outlier threshold, it is left as is. • **SmoothRemoveExcess**—the item's outlier quantities are adjusted by using parameters defined on the item to smooth the quantity above the threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains above the outlier threshold, that excess is reduced to the demand outlier threshold for the item. • **Threshold**—the item's upper threshold is used (quantity is decreased exactly to the point where it is no longer considered an outlier). **Default value:** Threshold **Note:** For more information about how the selected smoothing options are applied, see "Processing outliers" on page 2025 for statistical forecasting or "Processing outliers" on page 1987 for safety stock items. **Note:** The Ignore, SmoothKeepExcess, and SmoothRemoveExcess enumeration values were added in Version 2016.2, Service Update 5.	String (Enum)	

Table 7.39: OutlierType (Mfg) fields

Field name	Description	Data type	Key
BelowThreshold Rule	If an outlier is detected below the lower threshold, this rule specifies the type of adjustment made to increase the historical actual quantity. The selected option determines the value used in place of the outlier quantity. Valid values are: • **Forecast**—For statistical forecasting only, the item's calculated value from **StatisticalForecastOutlier.Forecast** is generally used. The calculation of **Forecast** depends on the **DataRule** setting in this table as follows. If using the "MovingAverageError" setting, then the moving average up to the point where the outlier occurs is used. If using the optional "RstlError" setting, then the sum of the trend and seasonal components is used. If using the "Historical" setting, then **Forecast** returns -1 and the mean is used instead. • **Ignore**—the item's historical actual quantities are left as is without any outlier adjustments. • **Mean**—the item's average quantity across the historical demand actual periods is used. • **Median**—the item's median quantity across the historical demand actual periods is used. • **SmoothKeepExcess**—the item's outliers quantities are adjusted by using parameters defined on the item to smooth the quantity below the outlier threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains below the outlier threshold, it is left as is. • **SmoothRemoveExcess**—the item's outlier quantities are adjusted by using parameters defined on the item to smooth the quantity below the threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains below the outlier threshold, that excess is reduced to the demand outlier threshold for the item. • **Threshold**—the item's lower threshold is used (quantity is increased exactly to the point where it is no longer considered an outlier). **Default value:** Threshold **Note:** For more information about how the selected smoothing options are applied, see "Processing outliers" on page 2025 for statistical forecasting or "Processing outliers" on page 1987 for safety stock items.	String (Enum)	

Table 7.39: OutlierType (Mfg) fields

Field name	Description	Data type	Key
DataRule	A rule that determines the specific data used for outlier detection for an item. Valid values are: • **Historical**—the actual and unaltered historical data points are used in calculating outliers. • **MovingAverageError**—differences between the actual historical data points and the moving average is used. The moving average is calculated over a number of intervals as set by the **OutlierMovingAverageWindow** field on the **ForecastItemParameters** table (for statistical forecasting) and the **MovingAverageWindow** field on the **SafetyStockItem** table (for inventory planning and optimization). • **RstlError**—adjusted actuals, after removal of any seasonal and trend components, are used. This option is available only with the optional Rforecast capability and uses the stl function in the R statistical programming language. **Default value:** Historical	String (Enum)	
Description	An optional description of this type.	String	

Table 7.39: OutlierType (Mfg) fields

Field name	Description	Data type	Key
DetectionRule	A rule that determines the method for detecting outliers in an item's historical data. Note that most settings in this field rely on the **ForecastItemParameters.OutlierThreshold** field (for statistical forecasting) and the **SafetyStockItem.DemandOutlierThreshold** (for inventory planning and optimization) as input into outlier detection. For example, it might indicate the number of standard deviations away from the mean where a value is considered an outlier. Valid values are: • **IglewiczHoaglinMethod**—Uses a modified z-score to measure distance from the mean. A point is considered an outlier if it is outside the following range $[0.6745 * \mid (x_i - Md)/(MdAD)\mid] \geq Zscore$ then the data value x_i is an outlier. Where *Md* is the median, *MdAD* is the median absolute deviation, and is x_i the data value. • **Rtsclean**—Uses loess with non-seasonal data and a periodic stl decomposition with seasonal data to identify and detect outliers. When using this option, the **DataRule** setting is always treated as if it were set to "Historical" (actual historical data without decomposition must be used). Also, this option is available only with the optional Rforecast capability and uses the tsclean method in the R statistical programming language. • **StandardDeviation**—Assumes a normal distribution of data. Outliers are then identified as those points that are a specified number of standard deviations above or below the mean. For example, if **OutlierThreshold** is set to "3", then points more than three standard deviations away from the mean are outliers. • **Winsorizing**—Data is sorted by quantity and outliers are identified as those points that are outside a specified percentile in the ordered data. For example, if **OutlierThreshold** is ".05", then points below the 5th percentile and above the 95th percentile in the data are considered outliers. **Default value:** StandardDeviation	String (Enum)	
Value	A unique string identifier for this control setting.	String	Yes
ForecastItem Parameters	Set of **ForecastItemParameters** records that reference this **OutlierType** value.	Set	
SafetyStockItems	Set of **SafetyStockItem** records that reference this **OutlierType** value.	Set	

PartSourceType table

The **PartSourceType** table is a control table used to define the characteristics of each potential supply source (for example, Make, Buy, and Transfer). This table is referenced from the **PartSource** table.

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this PartSourceType record. **Referenced table:** ControlSet	Reference	Yes
Constraint DateRule	Specifies how constraint is applied and planned orders generated for this part source. Valid values are: • **Start**—the Constraint is applied at StartDate (the date when processing of the order starts), and a separate planned order created for each constraint period. If there is not enough constraint in a given period, then a new (earlier) planned order is created with a new DueDate based on that (earlier) start date. For part sources constrained by a fixed constraint, this value is treated the same as StartExtend (only one planned order is created). • **Ship**—the Constraint is applied at ShipDate (the date when the order is to be shipped), and a separate planned order created for each constraint period. If there is not enough constraint in a given period, then a new (earlier) planned order is created with a new DueDate based on that (earlier) start date. For part sources constrained by a fixed constraint, this value is treated the same as ShipExtend (only one planned order is created). Note that when a scheduled receipt references a **SupplyStatus.IgnoreStartDate** of "N", then constraint is applied at order **StartDate** plus lead time (which might not neccesarily equate to the **ShipDate**). • **DuringLeadTime**—the Constraint is applied between ShipDate and StartDate, and one planned order is created with StartDate extended if there is not enough constraint available within lead time (that is, between StartDate and ShipDate.) • **StartExtend**—the Constraint is applied at StartDate, and one planned order is created with StartDate extended if additional constraint periods are required. • **ShipExtend**—the Constraint is applied at ShipDate, and one planned order is created with ShipDate extended if additional constraint periods are required. **Default value**: Ship **Note:** If this field is set to "DuringLeadTime", "Start", or "StartExtend", then **PartSource.VarLeadTime** is always ignored. For more information about ConstraintDateRule, see "ConstraintDateRule" on page 1654.	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
Constraint MinimumRule	Controls whether the minimum quantity (as defined by **PartSource.MinimumQty**) is used when splitting supplies for constraint allocation (see "Respecting order policy rules" on page 1684) or incremental availability of component materials (see "Respecting order policy rules" on page 1481). Valid values are: • **Ignore**—the effective minimum quantity is not used when splitting supplies for constraints or incremental availability. • **MultiplePeriods**—the effective minimum quantity is used when splitting supplies for constraint or incremental availability (splits must be at least the size of the **PartSource.MinimumQty** value). Note that for constraint allocation, a single order might be allocated constraint from multiple constraint periods (not necessarily contiguous) in order to meet the minimum order quantity. Thus, this value is only compatible with single order **ConstraintDateRule** options (ShipExtend, StartExtend, or DuringLeadTime). Also, this value can be used to enforce minimum order quantities on part sources constrained by fixed constraints. **Note:** Constraint FairShare does not support the combination of PartSourceType.ConstraintMinimumRule with MultiplePeriods. • **Use**—the effective minimum quantity is used when splitting supplies for constraints or incremental availability (splits must be at least the size of the **PartSource.MinimumQty** value). Note that for constraint allocation, the minimum order quantity is enforced within each constraint period (for example, a workday or week). This means there must be sufficient constraint available within a given constraint period to meet the minimum quantity or supply cannot be planned there. Also, for part sources constrained by a fixed constraint, this value is treated the same as Ignore (minimum quantity is not used). **Default value**: Ignore	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
Constraint MultipleRule	Controls whether the lot size (as defined by **PartSource.MultipleQty**) is used when splitting supplies for constraint allocation (see page "Respecting order policy rules" on page 1684) or incremental availability of component materials. This setting is useful for parts that can only be built in defined batch quantities. Valid values are: • **Ignore**—the effective multiple quantity is not used when splitting supplies for constraints or incremental availability. • **MultiplePeriods**—the effective multiple quantity is used; that is, supply splits are multiples of **PartSource.MultipleQty**. Note that for constraint allocation, the order multiple is enforced across multiple constraint periods (for example, workday or week) as necessary. In other words, if an order is allocated constraint from multiple periods, then the total constraint consumed by the order should respect the multiple quantity (but the constraint consumed within just one of those periods might not). This value is only compatible with single order **ConstraintDateRule** options (ShipExtend, StartExtend, or DuringLeadTime) Also, this value can be used to enforce minimum order quantities on part sources constrained by fixed constraints. **Note:** Constraint FairShare does not support the combination of **PartSourceType.ConstraintMultipleRule** with **MultiplePeriods**. • **Use**—the effective multiple quantity is used; that is, supply splits are multiples of **PartSource.MultipleQty**. Note that for constraint allocation, the order multiple is enforced within each constraint period (for example, a workday or week). In other words, a given order can only be allocated constraint from a particular period in the defined multiple quantity. Also, for part sources constrained by a fixed constraint, this value is treated the same as Ignore (multiple quantity is not used). **Default value**: Use	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
CostRule	Specifies how the enterprise system calculates the effective cost of a part. Valid values are: • **UnitCost**—sets the PartSource.EffUnitCost to a fixed value equal to the value imported into the PartSource.UnitCost field or Part.StdUnitCost if UnitCost is 0. If neither of these fields are greater than 0 then the PartSource.EffUnitCost is 0. If PartSource.UnitCost > 0, set PartSource.EffUnitCost = UOMConversion (PartSource.UnitCost). Otherwise, if Part.StdUnitCost > 0, set PartSource.EffUnitCost = Part.StdUnitCost. Otherwise, PartSource.EffUnitCost = 0. • **MaterialLaborOverheadCosts**—calculates the PartSource.EffUnitCost by adding the material, labor, and overhead costs. If PartSource.MaterialCost + PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2 > 0 then set PartSource.EffUnitCost to the value of this summation converted to Part UOM (note that if the **PartSourceTimePhasedAttributes** table has an effective record for the part source then its values are used instead of those defined on the **PartSource** record). Otherwise, if Part.StdUnitCost > 0, set PartSource.EffUnitCost = Part.StdUnitCost. Otherwise, PartSource.EffUnitCost = 0. **Default value:** UnitCost **Note:** This setting also impacts the calculation of the **NewPurchaseCost** fields found on many tables in the RapidResponse data model.	String (Enum)	
Description	Additional information about this source type.	String	
DockToStock LeadTimeRule	Specifies whether the **DockToStockLeadTime** value on the **PartSource** record is used in lead time calculations. Valid values are: • **Ignore**—the **DockToStockLeadTime** value is always ignored. • **Use**—the **DockToStockLeadTime** value is used in lead time calculations. If **TransitRule** is set to "Cumulative", then **DockToStockLeadTime** is used in calculating effective lead time and hence the supply start date. Otherwise, if **TransitRule** is "Inclusive", then **DockToStockLeadTime** is only used in determining the calculated dock date **Default value:** Use	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
EffectivityRule	Specifies the effectivity rule that is used by the enterprise system to interpret the dates on PartSource records (which are time phased). Valid values are: • **InclusiveExclusive**—the PartSource record is effective beginning on the "In date" (EffectiveInDate) and ending on the day prior to the "Out date" (EffectiveOutDate). This is the recommended setting. • **Always**—effectivity dates are ignored and PartSource records are treated as effective. • **Exclusive**—the PartSource record is effective beginning on the day after the "In date" (EffectiveInDate) and ending on the day prior to the "Out date" (EffectiveOutDate) • **Inclusive**—the PartSource record is effective beginning on the "In date" (EffectiveInDate) and ending on the "Out date" (EffectiveOutDate). • **Never**—the PartSource record is never used for generating planned orders. However, it is a valid sourcing record for existing scheduled receipts. It is also excluded from the calculation of IsRecursive flags. The PartSource record might be maintained for historical purposes, or as a placeholder to be activated later by setting its EffectivityRule to another value.	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
ExpiryDateRule	Determines the point from which expiring supplies using this part source begin to expire. This is the date to which **PartSource.IntervalsToExpiry** is added to determine the estimated expiry date (used by the Netting analytic) and to which **PartSource.CTPIntervalsToExpiry** is added to determine the actual expiry date (calculated by the Capable-To-Promise analytic) on a given supply. Valid values are: • **DueDate**—The estimated expiry date is calculated by adding **IntervalsToExpiry** to the effective due date, and the actual expiry date is calculated by adding **CTPIntervalsToExpiry** to the available date. This setting should be used when expiration is expected to start from the point the supply is placed into stock. • **DockDate**—The estimated expiry date is calculated by adding **IntervalsToExpiry** to the effective dock date, and the actual expiry date is calculated by adding **CTPIntervalsToExpiry** to the available dock date. This setting should be used when expiration is expected to start from the point when the finished product is received but before it is placed into stock (as represented by the value in the **DockToStockLeadTime** field). For example, there might be an inspection process before a product can be received into stock, but during which it starts to expire. • **StartDate**— The estimated expiry date is calculated by adding **IntervalsToExpiry** to the effective start date, and the actual expiry date is calculated by adding **IntervalsToExpiry** to the available start date. This setting should only be used when all lead time components are meant to be included as part of expiry, and caution should be taken with its usage. If used with variable lead time (**VarLeadTime**), then this setting might result in planned orders that expire sooner than expected and potentially cannot be used. **Default value**: DueDate	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
ExpiryRule	Specifies how the expiry date for supplies from part sources of this type are calculated. The expiry date is then used to determine which demands can potentially be satisfied by the supply. This field is also used by calculations that set the earliest expiry date on dependent demands. By default, the **PartType.ExpiryRule** field sets how expiration is calculated for each part of a given type (from any of its sources), but this can be overridden at the part source level if necessary. Typically, this is done when a part has multiple sources of supply and not all of those sources should have their supply's expiration calculated in the same manner. For example, if supply is provided by a "make" source the expiry date might be calculated based on the part's components, but if it is provided from a "transfer" source then the expiry date might be taken directly from that incoming supply. Note also that the settings in this table are only applicable to parts where **PartType.ExpiryRule** is set to a value other than "Ignore" (if the part is not configured for expiry, then this field is always ignored). Valid values are: • **Dependent**—supplies expire based on characteristics of their components. The supply expiry date is calculated as the earliest expiry date on any component supply used to satisfy its dependent demands. This option is typically used for expiring assembly parts (if used on parts that do not explode to any components, then supply might not actually expire). When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands inherit their earliest expiry date directly from the earliest expiry date on the assembly demand. • **DependentLimit**—supplies expire based on either their own characteristics or those of their components. The supply expiry date is calculated as the earliest between its own calculated expiration (**AvailableDate + PartSource.CTPIntervalsToExpiry**) or the **ExpiryDate** found on any component supply used to satisfy its dependent demands. Note that if this option is used on parts that do not explode to any components, then supply might not actually expire. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands inherit their earliest expiry date directly from the earliest expiry date on the assembly demand.	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
	• **DependentStart**—supplies expire based on a mix of their own characteristics and those of their components. The supply **ExpiryDate** is calculated by adding the assembly's **PartSource.CTPIntervalsToExpiry** value to the earliest **ExpiryStartDate** rolled up from any component supply (by default, this will be set to the component's available date but expiry start dates can also be provided on input supplies). Note that this option is typically used for expiring assembly parts that contain limiting components in their structure (parts where **ExpiryType.ProcessingRule** is set to "LimitingComponent"). For more information, see "DependentStart" on page 1619 Note also that when an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the component's **MinimumShelfLife** value (in intervals of the component **ExpiryCalendar**). • **FromPartType**—expiry dates on supplies and earliest expiry dates on dependent demands are calculated based on the part's default setting in the **PartType.ExpiryRule** field. For example, this setting would typically be used on all part's that only have one source of supply. If a part has multiple sources of supply, then this setting can be used on those sources whose expiration should be calculated in the same manner defined by the part's default setting. • **Self**—a supply for a part of this type expires based on its own characteristics. The supply expiry date is calculated by adding the number of **ExpiryCalendar** intervals defined by the **PartSource.CTPIntervalsToExpiry** value to the supply's **AvailableDate**. This option is typically used for expiring assembly parts whose expiry is not determined by their components as well as for bottom-level component parts. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the assembly's **MinimumShelfLife** value (in terms of the assembly's **ExpiryCalendar**). • **SelfReset**—supplies for parts of this type expire based on their own characteristics. The supply expiry		

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
	date is calculated by adding the number of **ExpiryCalendar** intervals defined by the **PartSource.CTPIntervalsToExpiry** value to the supply's **AvailableDate**. This option is intended for use on assembly parts whose components should have the earliest expiry date on their dependent demands calculated to meet their own expiry objectives rather then those of the assembly. It is similar to "Self" except in how it impacts the earliest expiry date on dependent demands. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the component's **MinimumShelfLife** value (in intervals of the component **ExpiryCalendar**). **Default value:** FromPartType		
PlannedOrder SupplyStatus	Specifies how to control Order.Line level configuration settings (such as generation of dependent requirements) for planned orders. **Referenced table:** SupplyStatus	Reference	
PlannedOrder SupplyType	Specifies, for planned orders, how to control order level configuration settings (such as rescheduling logic and whether the order is a Make, Buy or Transfer). While PartTypes and PartSourceTypes might have already been named to reflect Make, Buy or Transfer, the SupplyType is the value that defines Make, Buy and Transfer logic and the PlannedOrderSupplyType is the link between these values. **Referenced table:** SupplyType	Reference	
PlannedOrder ToSR SupplyStatus	The supply status used when converting planned orders into scheduled receipts. This value is also used during the part source selection process . **Referenced table:** SupplyStatus	Reference	
PlannedOrder ToSR SupplyType	The supply type to use when converting planned orders into scheduled receipts. This field is also used as part of the criteria for assigning a part source to a scheduled receipt as discussed in "Part sources for scheduled receipts" on page 1389. **Referenced table:** SupplyType	Reference	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
PrePlanRule	When supplies are preplanned due to consuming constraint in an earlier period to try and satisfy a demand date, this setting controls the direction in which that constraint is consumed and hence may affect just how early pre-planning is done. Valid values are: • **Normal**— constraint is consumed from earliest date to latest (after having determined the earliest constraint bucket necessary for pre-planning to start. With this setting, preplanned quantities are likely to be seen in earlier constraint buckets, and in some cases preplanning might be seen as starting earlier than strictly required. This corresponds to how pre-planning was always done prior to Version 2016.2, Service Update 16. • **JustInTime**— constraint is consumed from latest constraint bucket to earliest. As necessary, orders are sized to fit available constraint capacity in the latest bucket. This setting is intended to reduce unnecessarily pre-planned quantities. **Default value**: Normal **NOTE**: This field was added in Version 2016.2, Service Update 16.	String (Enum)	
PreShipLead TimeRule	Specifies whether the **PreShipLT** value on the **Source** record is used in lead time calculations. Valid values are: • **Ignore**—the **PreShipLT** value is always ignored. • **Use**—the **PreShipLT** value is used in lead time calculations. If **TransitRule** is set to "Cumulative", then **PreShipLT** is used in calculating effective lead time and hence the supply start date. Otherwise, if **TransitRule** is "Inclusive", then **PreShipLT** is only used in determining the calculated ship date. **Default value:** Use	String (Enum)	
TransitTimeRule	Specifies whether the **TransitTime** value on the **Source** record is used in lead time calculations. Valid values are: • **Ignore**—the **TransitTime** value is always ignored. • **Use**—the **TransitTime** value is used in lead time calculations. If **TransitRule** is set to "Cumulative", then **TransitTime** is used in calculating effective lead time and hence the supply start date. Otherwise, if **TransitRule** is "Inclusive", then **TransitTime** is only used in determining the calculated ship date. **Default value:** Use	String (Enum)	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
TransitRule	Specifies how the enterprise system calculates lead time. Lead time values are considered in calculating a "start date" (PlannedOrder.StartDate or ScheduledReceipt.CalcStartDate). Valid values are: • **Cumulative**—lead time, and hence supply start date, is determined as the worst case scenario for producing a part. It is calculated by adding the following fields: PartSource.FixedLeadTime, PartSource.VarLeadTime, Part.LeadTimeAdjust, Source.PreShipLT, Source.TransitTime, and PartSource.DockToStockLeadTime. • **Inclusive**—lead time, and hence supply start date, is considered as the time when a part is ordered until it arrives in stock. Lead time is calculated by adding the following fields: PartSource.FixedLeadTime, PartSource.VarLeadTime, and Part.LeadTimeAdjust. **Note:** For more information about lead-time calculations, see "Lead time calculations" on page 1307. **Default value**: Inclusive	String (Enum)	
UseTimePhasedPartSource LeadTime	Specifies whether any applicable time-phased fixed or dock-to-stock lead time values are applied to the part source. This setting can be used to easily disable time-phased lead time logic without modifying existing input data in the **TimePhasedPartSourceLeadTime** table. Valid values are: • **Y**— time-phased lead time calculations are performed for this part source as long as records exist for it in the **TimePhasedPartSourceLeadTime** table. If a part source does not have any records defined in that table, then stationary lead time values from the **PartSource** table are used instead. • **N**—time-phased lead time calculations are not performed for the part source. This is regardless of whether or not records are defined for it in the **TimePhasedPartSourceLeadTime** table (stationary values from the **PartSource** table are always used). **Default value:** Yes **NOTE**: This field was added in Version 2016.2, Service Update 18.	Boolean	

Table 7.40: PartSourceType (Mfg) fields

Field name	Description	Data type	Key
UseTimePhasedSource LeadTime	Specifies whether any applicable time-phased transit or pre-ship lead time values are applied to the applicable source. This setting can be used to easily disable (or re-enable) time-phased lead time logic without modifying existing input data in the **TimePhasedSourceLeadTime** table. Valid values are: • **Y**—time-phased lead time calculations are performed for this part source as long as records exist for the source in the **TimePhasedSourceLeadTime** table. If a source does not have any records defined in that table, then stationary lead time values from the **Source** table are used instead. • **N**—time-phased lead time calculations are not performed for the part source. This is regardless of whether or not records are defined for the source in the **TimePhasedSourceLeadTime** table (stationary values from the **Source** table are always used). **Default value:** Yes **NOTE**: This field was added in Version 2016.2, Service Update 18.	Boolean	
Value	Identifier for this rule.	String	
PartSources	Set of **PartSource** records that reference this PartSourceType value.	Set	

PartType table

The **PartType** table contains the values that control how a part is processed. By setting up records appropriately, you can emulate the part types (also called source code) from the enterprise data source.

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Allocation QuantityRule	Specifies whether to include effective quantity in the criteria for sorting supplies and demands before the allotment of supplies to demands. Valid values are: • **Ignore**—Ignores the effective quantity when sorting supplies and demands before the supply-demand allotment. This setting can be used to satisfy demands on a first in, first out basis (if order ids have been assigned sequentially). • **Use**—Includes the effective quantity when sorting supplies and demands before the supply-demand allotment. This setting can be used to try and satisfy as many demands on time as possible (by ensuring smaller demands are satisfied before larger demands when there is insufficient quantity to satisfy all). **Default value**: Use	String (Enum)	
ATPCTPRule	Specifies, for each part type, where excess supply is allocated, and how to calculate for available to promise (ATP). Valid values are: • **Earliest**—demand is satisfied on time; that is, the **IndependentDemand.AvailableDate** will equal the **IndependentDemand.DueDate**. These demands are satisfied with the latest available supply that will satisfy the demand on time. This leaves earlier supplies available as ATP and therefore able to satisfy demands dropped in at the beginning of a scenario. • **Latest**—demand is satisfied with the earliest available supply that will make the **IndependentDemand.AvailableDate** as early as possible. This means ATP is calculated later and based on the remaining supply. **Default value:** Earliest **Note:** This control affects ATP results calculated by the Capable-to-Promise analytic and reported in various calculated fields. It does not affect results in the **AvailableToPromise** table which is based on Netting results.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
AvailableRule	Specifies how the availability for the part is determined. Valid values are: • **Ignore**—available date is calculated for each supply, but requirements are treated as if the dependent and independent demands had been covered by on hand supplies. Hence, dependent requirements onto these parts are available at Past and independent requirements are available at DataDate. Use this setting for vendor managed parts. • **Normal**—available date for supplies and demands are processed using all the rules for CTP. **Default value:** Normal **Note:** Any expiring parts should typically have this field set to "Normal". For more information, see "Setting a part's expiry rule" on page 1602. **Note:** The setting in this field can be overridden on parts that have their **AvailableRule** reference populated. For more information, see "AvailableRule table" on page 686.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
CommitLevel	Specifies, for each part type, how order commitments are maintained, and sets how priority is applied to netting (supply planning) and CTP (available date calculations). Valid values are: • **High**—Priority is used in both Netting and CTP. During Netting, priority is propagated from demands to supplies, and demands are processed in priority sequence. During CTP, priority is used when allocating supply to demand. This setting is appropriate for parts that have part sources with constraint and might have components with limited availability. • **MediumExplodePriority**—Priority is used in both CTP and Netting calculations. During Netting, priority is propagated from demands to supplies, however demands are processed in due date sequence instead of priority sequence. During CTP, priority is used when allocating supply to demand. Similar to "High", this setting is appropriate for parts that have part sources with constraint and might have components with limited availability, but is intended for parts set to use lot sizing policies, configured as the primary product in a coproduct byproduct structure, or as the primary component in a BOM level part substitution relationship. • **Medium**—appropriate for parts with only buy sources (with or without constraint) or make sources with no constraint. Priority is used in CTP, but not in netting. This means that priority is not propagated from demands to supplies, nor is priority used when allocating constraint to supplies for this part. However, priority is used when allocating supply to demand. For example, this setting would be used for purchased parts without supplier constraints. • **Low**—appropriate for parts where all part sources have no active constraint and where any make sources have unlimited availability. Priority is not used in either netting or CTP, nor is it used when allocating constraint. A low setting overrides a setting of **Y** on **OrderPriority.MaintainCommitments**. **Default value:** Low	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
ConstraintShare Rule	Specifies whether the part's demands are eligible for the fair-sharing of common constraints with other parts whose sources also require them. The OrderPriority.ConstraintAllocationRule field initially enables or disables the fair-sharing of limited constraint availability for all demands of a given priority level. This setting subsequently allows for constraint-sharing to be either turned off or limited at the part level. For example, it might be used to disable fair-sharing on lower level parts in the product structure who receive dependent requirements with priorities and fair-sharing policies inherited from higher-level assemblies. Valid values are: • **Ignore**—constraint sharing is always disabled for parts of this type, regardless of the setting in the ConstraintAllocationRule field. That is, when satisfying demands of a given priority level, potentially limited constraint is always allocated on a supply-by-supply basis (thus the demands satisfied by those supplies are satisfied on a first-come, first- served basis). • **Use**—constraint sharing is enabled for parts of this type on all demands belonging to priority levels where the ConstraintAllocationRule is set to "FairShare". Thus, when there is limited constraint availability to satisfy all demands on time, it is fairly shared between the sources supplying the different parts (thus, any resulting lateness is also shared between their demands). • **WithinFence**— constraint sharing is enabled for parts of this type on demands having priority levels where the ConstraintAllocationRule is set to "FairShare", and where the demand's due date falls within the constraint sharing window as defined by Part.ConstraintShareFence. Typically, used when the sharing of limited constraint is intended only to satisfy near-term demands, but where first-come, first-served constraint allocations are acceptable later in the planning horizon. **Default value**: Use	String (Enum)	
ControlSet	The control set associated with this PartType record. **Referenced table:** ControlSet	Reference	Yes

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
DaysSupply PrioritySource	For parts subject to days of supply logic (where **UseDaysSupply** = "Y"), this setting controls how the **OrderPriority** field on a **PlannedOrder** record generated in a given period is set. The order priority can be set to the part's default priority or calculated based on the priorities of the demands that the planned order satisfies in Netting. This field applies to all **DaysSupplyRule** options other than "ByPeriodWithPriority" and "ByPeriodWithPriorityEnd". When using either of these settings, a separate planned order is always created for each priority value found on one or more demands being satisfied in a given period. Valid values are: • **Average**—order priority is set to the average of the priorities found on all demands satisfied by the order in Netting calculations. That is, the **OrderPriority**.**PlanningPriority** values found on all demands being satisfied are summed and then divided by the number of demands. The **OrderPriority.Value** associated with the closest **PlanningPriority** value less than or equal to the average is then used. • **Default**—order priority is set to the part's default priority value as found in the **DefaultPriority** field. • **Highest**—order priority is set to the highest priority found on any demand satisfied by the order in Netting calculations. That is, the **OrderPriority.Value** associated with the lowest **OrderPriority**.**PlanningPriority** value referenced from the demands being satisfied is used. • **WeightedAverage**—order priority is set to a weighted average of the priorities found on all demands satisfied by the order in Netting calculations. That is, the **OrderPriority.PlanningPriority** values found on all demands being satisfied are multiplied by their respective demand quantities, summed together, and then divided by the total demand quantity being satisfied. The **OrderPriority.Value** associated with the closest **PlanningPriority** value less than or equal to the average is then used. As opposed to the "Average" option, this setting can help prevent planned order priorities from being inflated due to very small high priority demands. **Default value**: Default In order for demand priorities to be propagated to planned orders as described here, the part's **CommitLevel** setting must be "High". **Note:** For more information, see "Days of supply and planned order priority (alternate configuration)" on page 1371.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
DaysSupplyRule	For parts where **UseDaysSupply** = 'Y' and the **Part.DaysSupplyPolicy** reference is Null, this field defines the period considered when accumulating demands for planned order generation. Valid values are: • **ByPeriod—**uses the value in **Part.PlanningCalendars.PeriodSupplyInterval** to define the calendar used when accumulating demands (for example, weekly). Demands are accumulated over the number of calendar intervals set by the value in **Part.NumberOfDaysSupply** (starting from the due date of the first unsatisfied demand)**.** A single planned order is then generated on the first **Part.PlanningCalendars.Period SupplyDueIntervalCalendar** interval within the accumulation period. • **ByPeriodEnd**—demands are accumulated in intervals of the **Part.PlanningCalendars. PeriodSupplyInterval** calendar, and over the number of intervals set by the **Part.NumberOfDaysSupply** field (starting from the due date of first unsatisfied demand). A single planned order is then generated on the first **Part.PlanningCalendars.Period SupplyDueIntervalCalendar** interval within the accumulation period. • **ByPeriodEndWithPriority**—uses the value in **Part.PlanningCalendars. PeriodSupplyInterval** to define the calendar used when accumulating demands (for example, weekly), and accumulates demands over the number of calendar intervals set by the value in **Part.NumberOfDaysSupply** (starting from the due date of the first unsatisfied demand). If the **CommitLevel** is set to "High", then this setting also attempts to respect priority by breaking down the accumulated quantity into smaller units grouped by priority. However, if **CommitLevel** is set to "Low", "Medium", or "MediumExplodePriority" priority is ignored and this setting is identical to **ByPeriodEnd**. • **ByPeriodWithPriority**—uses the value in **Part.PlanningCalendars.PeriodSupplyInterval** to define the calendar used when accumulating demands (for example, weekly), and accumulates demands over the number of calendar intervals set by the value in **Part.NumberOfDaysSupply** (starting from the due date of the first unsatisfied demand). If the **CommitLevel** is set to "High", then this setting also attempts to respect priority by breaking down the accumulated quantity into smaller units grouped by priority. These quantities and priorities	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	are used when generating planned orders whose priorities are subsequently passed down to their dependent demands. However, if **CommitLevel** is set to "Medium", "MediumExplodePriority", or "Low", priority is ignored and this setting is identical to **ByPeriod**. • **FromDemand**—uses the value in **Part.PlanningCalendars.TimeUnits** to define the time unit used when accumulating demands (for example, work days). Demands are accumulated over the number of units set by the value in **Part.NumberOfDaysSupply** (starting from the due date of the first unsatisfied demand). • **FromSupply**—similar to **FromDemand** except that demands accumulate starting from the due date when the supply can be planned (instead of the due date of the unsatisfied demand). In cases where there is a past-due demand for a part subject to **DaysOfSupply**, the supply due date will be later than the demand due date, and hence the days supply horizon will end later. In typical cases where the supply due date and demand due date are the same, this field has the same effect as **FromDemand**. • **Ignore**—Days of supply planning is disabled for parts that use this policy (regardless of the setting in the **UseDaysSupply** field). **Default value**: FromDemand **Note:** For more information, see "Days of Supply calculations" on page 1362.		
DefaultPriority	The default order priority level for parts of this type. This is the priority assigned to new planned orders when **PartType.CommitLevel** is set to either "Medium" or "Low" as discussed in "Priorities on planned orders" on page 1790. . This is also the priority to use for all planned orders resulting from safety stock requirements when using the default **SafetyStockPolicy.DemandTransformationRule** setting of "Ignore". However, when using the "FirstInFirstOut" setting, then priorities on planned orders driven from safety stock requirements are set to the priority of the next subsequent real demand that the supply is seen to satisfy. Additionally, if using time-phased safety stock than priority on safety stock requirements can be explicitly set using the **TimePhasedSafety.OrderPriority** reference. For more information about using priority on safety stock, see "Priority and safety stock holds in CTP" on page 1780. **Referenced table:** OrderPriority **Default value:** OrderPriority with the highest Number.	Reference	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Dependent Demand Forecast Consumption	For dependent demands having a **Type.ProcessingRule** = 'FromPart', this field determines whether or not the demand for the component consumes forecast. Valid values: • **Y**—any dependent demands for this part that have a **Type.ProcessingRule** of 'FromPart' are processed for forecast consumption (that is, they are treated as if **Type.ProcessingRule** = 'SalesActual'. • **N**—any dependent demands for this part that have a **Type.ProcessingRule** of 'FromPart' are not processed for forecast consumption (that is, they are treated as if **Type.ProcessingRule** = 'Regular'. For more information, see "Dependent forecast consumption on a component by component basis" on page 1442. Default value: **N**	Boolean	
Dependent Demand Usage	Specifies the behavior of the dependent demand generated on each PartType. Valid values are: • **Ignore**—generate but do not use in netting. • **DoNotGenerate**—do not generate any dependent demand on this component • **Net**—Generate dependent demand on this component and use the demand in netting • **BlowThrough**—phantom part processing and dependent demand is passed straight through to components of this part. • **NetIfInventory**—use dependent demand in netting if the part has nettable inventory (**OnHand.Type.ProcessingRule** = 'Nettable'). If not, blow all dependent demand through to components of this part. • **NetIfActive**—Use dependent demand in netting if the part has nettable inventory (**OnHand.Type.ProcessingRule** = 'Nettable'), supply (**ScheduledReceipt.Status.IgnoreSupply** = 'N') or demand (**IndependentDemand.Status.ProcessingRule** = 'Current' and **IndependentDemand.Order.Type.ProcessingRule** not equal to 'Ignore') or dependent demand. If not, blow all dependent demand through to components of this part. **Default value**: Net	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Dependent SafetyLead TimeRule	Specifies whether orders satisfying dependent demands generated for parts of this type should use **SafetyLeadTime** as defined on the **Part** and/or **PartSource** tables. Note that this setting is only applicable if **UseSafetyTimeLeadTime** is set to either "Y" or "Soft" (safety lead time is always ignored when that option is set to "N") Valid values are: • **FromDemandType**—safety lead time usage on orders satisfying dependent demands for parts of this type is determined by demand type (using the **SafetyLeadTimeRule** setting from the applicable **DemandType** record). • **Ignore**—safety lead time is never applied on orders satisfying dependent demands for parts of this type. • **Use**—safety lead time is always applied on orders satisfying dependent demands for parts of this type. **Default value:** Use	String (Enum)	
Description	A description of this part type. **Default value**: blank string	String	
DTFRule	Specifies how unconsumed forecast before **Part.DTFDate** is processed. Valid values are: • **ForecastDate**—Any unconsumed forecast due before **DTFDate** is always ignored by Netting (supply is not planned to satisfy it).. • **ForecastInterval**—Unconsumed forecast due before the **DTFDate** is generally ignored by Netting, except when the forecast is due within the same **ForecastInterval** as DTFDate. In that case, the unconsumed forecast is treated as being due at DTFDate and Netting will plan supply to satisfy it. **Default value:** ForecastDate	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
ExcessRule	If a part is configured as a global or BOM-level substitute, this setting can be used to specify that only its excess current supply is made available to satisfy other part's demands. Within the excess window defined by **Part.ExcessFence**, existing supply on a part will then be used to satisfy its own demands first before any remainder (excess) is made available for use as a substitute. Valid values are: • **Fence**—If the part is configured as a bom-level substitute component, then its current supply should be used to satisfy any of its own demands first (regardless of order priority). The leftover or "excess" supply is then eligible to satisfy demand from other components. • **GlobalFence**—If the part is configured as a global substitute part, then its current supply should be first used to satisfy any of its own demands (within a given order priority level). The leftover or "excess" supply is then eligible to satisfy demands of the same priority level from other parts for which it has been configured as a substitute. With this setting, planned orders will only be generated to satisfy the part's own demands and not those of parts for which it is a substitute (excess resulting from those planned orders, however, can be made available to other parts). • **GlobalFenceBeforePriority—**If the part is configured as a global substitute part, then its current supply should be first used to satisfy any of its own demands (regardless of their priority level). The leftover or "excess" supply is then eligible to satisfy any demands from other parts for which it has been configured as a substitute. With this setting, planned orders will only be generated to satisfy the part's own demands and not those of parts for which it is a substitute (excess resulting from those planned orders, however, can be made available to other parts). • **Ignore**—Excess is not a consideration in substitution decisions. Input supply on a substitute component is used to satisfy the demands that require it first regardless of whether the demands are its own or those of the primary component in the group. Similarly, input supply on a globally substitutable part is used to satisfy the demands that require it first regardless of whether the demands are its own or those of another part. **Default value:** Ignore **Note:** If a part is configured for both global and BOM-level substitutions, the "GlobalFence" setting is always ignored (treated as "Ignore").	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
ExpiryPastDue Rule	Specifies how the earliest expiry date on past-due demands is calculated. Earliest expiry dates are normally determined starting from the demand due date and then adding the part's minimum shelf life. However, because past-due demands have due dates that are in the past, this results in the calculation of earliest expiry dates that are typically earlier than they should be. Thus, these demands might be allotted supplies which would otherwise be seen as expiring too soon to satisfy them. This field is applicable to all past-due independent demands for expiring parts, as well as past-due dependent demands for expiring parts that reference an **ExpiryRule** setting of either "Self", "SelfReset", or "DependentStart". Valid values are: • **DataDate**— Past-due demands have their earliest expiry dates calculated starting from **Part.PlanningCalendars.DataDate.FirstDate**(typically, this corresponds to today's date)**.** Thus, earliest expiry calculations are performed relative to the current date rather than a point in the past. • **DueDate**—Past due demands have their earliest expiry date calculated starting from their due date. This setting corresponds to how earliest expiry calculations were performed in all versions prior to 2015.3. **Default value:** DueDate	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
ExpiryRule	Specifies how the expiry date for supplies using parts of this type is calculated. The expiry date is then used to determine which demands can potentially be satisfied by the supply. This field is also used by calculations that set the earliest expiry date on dependent demands. Note also that the setting in this field can subsequently be overridden at the part-source level using the **PartSourceType.ExpiryRule** setting. This is intended for cases where a part has multiple sources, and not all of those sources should have their supply's expiration calculated in the same manner. For example, if provided by a "make" source the expiry date might be calculated from the part's components, and if provided by a "transfer" source the expiry date might be inherited from the incoming supply. Valid values are: • **Dependent**—supplies expire based on characteristics of their components. The supply expiry date is calculated as the earliest expiry date on any component supply used to satisfy its dependent demands. This option is typically used for expiring assembly parts (if used on parts that do not explode to any components, then supply might not actually expire). When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands inherit their earliest expiry date directly from the earliest expiry date on the assembly demand. • **DependentLimit**—supplies expire based on either their own characteristics or those of their components. The supply expiry date is calculated as the earliest between its own calculated expiration (**AvailableDate + PartSource.CTPIntervalsToExpiry**) or the **ExpiryDate** found on any component supply used to satisfy its dependent demands. Note that if this option is used on parts that do not explode to any components, then supply might not actually expire. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands inherit their earliest expiry date directly from the earliest expiry date on the assembly demand. • **DependentStart**—supplies expire based on a mix of their own characteristics and those of their components. The supply **ExpiryDate** is calculated by adding the assembly's **PartSource.CTPIntervalsToExpiry** value to the earliest **ExpiryStartDate** rolled up from supply of a limiting component (by default, this will be set to the component's available date but expiry start dates can also be provided on input supplies). Note that this option is typically used	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	for expiring assembly parts that contain limiting components in their structure (parts where **ExpiryType.ProcessingRule** is set to "LimitingComponent"). For more information, see "DependentStart" on page 1619. Note also that when an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the component's **MinimumShelfLife** value (in intervals of the component **ExpiryCalendar**). • **Ignore**— supplies for parts of this type do not expire, and can be used to satisfy demand on any date. The expiry date on supplies is always set to "Future" and the earliest expiry date on demands is always set to "Past". Note also that the **ExpiryRule** setting on the **PartSourceType** table is always ignored for parts using this setting (the part will never expire regardless of the setting at the part-source level). • **Self**—a supply for parts of this type expires based on its own characteristics. The supply expiry date is calculated by adding the number of **ExpiryCalendar** intervals defined by the **PartSource.CTPIntervalsToExpiry** value to the supply's **AvailableDate**. This option is typically used for expiring assembly parts whose expiry is not determined by their components as well as for bottom-level component parts. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the assembly's **MinimumShelfLife** value (in terms of the assembly's **ExpiryCalendar**). • **SelfReset**—supplies for parts of this type expire based on their own characteristics. The supply expiry date is calculated by adding the number of **ExpiryCalendar** intervals defined by the **PartSource.CTPIntervalsToExpiry** value to the supply's **AvailableDate**. This option is intended for use on assembly parts whose components should have the earliest expiry date on their dependent demands calculated to meet their own expiry objectives rather then those of the assembly. It is similar to "Self" except in how it impacts the earliest expiry date on dependent demands. When an assembly part of this type generates dependent demands on components subject to expiry, then those dependent demands have their earliest expiry date calculated starting from their own due date and adding the component's **MinimumShelfLife** value (in intervals of the component		

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	ExpiryCalendar). **Default value:** Ignore Any expiring parts should typically have their applicable **AvailableRule.ProcessingRule** or **Type. AvailableRule** value set to "Normal". For more information, see "Setting a part's expiry rule" on page 1602.		
FeatureBOM Rule	Used to enable or disable feature BOM logic for configurable end-item assembly parts of this type. When feature BOM logic is enabled, dependent demands are placed on particular assembly components by matching features specified on the demand (**IndependentDemand.FeatureList**) against feature rules that are specified on each BOM record (**BillOfMaterial.FeatureRule**). When feature BOM logic is disabled, dependent demands are placed on all of the assembly's components and any specified features or feature rules are ignored. Valid values are: • **HighVolume**—feature BOM logic is enabled. This setting is intended for configurable end-item assemblies that are expected to have extremely large volumes of orders per day which might impact system performance when performing certain full-level pegging calculations. As such, this setting limits or aggregates certain results returned by the full-level pegging analytic. Specifically, for records pertaining to component supplies in the **WhereConsumed** table, the driving demand is not reported and instead the associated **CTPPlannedOrder** record is reported as the driver. Further, in the **WhereConsumedForSupply** table only one record per planned order for a top-level part is reported, and multiple allotments from the same planned order are not reported. • **Ignore**—feature BOM logic is disabled and normal BOM explosion logic applies. • **Normal**—feature BOM logic is enabled. This setting is intended for configurable end-item assemblies that are not expected to have extremely large volumes of orders per day. With this setting, all full-level pegging calculations are performed and reported as normal. **Default value:** Ignore **Note:** When a part is enabled to be a configurable end-item, its demands can only be satisfied by planned orders (and not any input supplies).	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption Calendar	Specifies the calendar whose intervals are used in setting the forecast consumption window for demands on parts of this type. The forecast consumption window defines the date range over which a given actual demand can consume forecast. This window is, by default, set based on the demand's due date with a number of **Part.BeforeForecastInterval** calendar intervals added before the demand's due date and a number of **Part.AfterForecastInterval** calendar intervals added after the demand's due date. These two fields are assumed to be expressed in terms of the calendar chosen here. Valid values are: • **ForecastCalendar**—the forecast consumption window is defined in terms of the part's **PlanningCalendars.ForecastCalendar** value. For example, this might be a monthly calendar. • **SecondCalendar**—the forecast consumption window is defined in terms of the part's **PlanningCalendars.SecondCalendar** value. For example, this might be a workday calendar. **Default value:** ForecastCalendar **Note:** Based on configuration settings in the **ForecastConsumptionDateRule** field on both the **PartType** and **DemandStatus** tables, a given demand's forecast consumption window might be based on the **DataDate** or its **RequestDate** instead of **DueDate**.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption DateRule	Specifies the date from which the forecast consumption window for both independent and dependent demands is derived for parts of this type. Note that the impact of this setting is only seen on demands that are past due. Valid values are: • **DataDate**—demands for parts of this type have their forecast consumption window based on the later of **Part.PlanningCalendars.DataDate** or their **DueDate**. With this setting, past due demands will consume forecast relative to the current date rather than their due date (where there will typically be more forecast available to be consumed). • **DueDate**—demands for parts of this type always have their forecast consumption window based on their **DueDate**. With this setting, past due demands will consume forecast relative to the date past when they were originally due (where there will typically be less forecast to be consumed). **Default value**: DueDate **Note:** This field can be overridden on specific independent demands using the **DemandStatus** table's **ForecastConsumptionDateRule** setting which also includes an option to derive the forecast consumption window from the demand's **RequestDate**.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Forecast Consumption Order	Specifies the direction in which a customer order consumes forecast within its forecast interval. • **AlternatingBackward**—Forecast records are consumed starting from the customer order date, moving backward to the beginning of the previous calendar interval defined by Part.PlanningCalendars.SecondCalendar, and then alternating forward and backward in SecondCalendar intervals. • **AlternatingForward**—Forecast records are consumed starting from the customer order date, moving forward to the end of the next calendar interval defined by Part.PlanningCalendars.SecondCalendar, and then alternating backward and forward in SecondCalendar intervals. • **Backward**—Forecast records are consumed starting from the end of the forecast interval, and moving backwards to the start of the forecast interval. • **Forward**—Forecast records are consumed starting from the start of the forecast interval, and moving forwards to the end of the forecast interval. • **Normal**—Forecast records are consumed starting from the record on or before the customer order date, and moving backwards to the start of the forecast interval. After the start of the forecast interval is reached, consumption continues from the first record after the customer order date, and moving forwards to the end of the forecast interval. • **NormalNested**—Forecast records are consumed from the record on or before the customer order date, and moving backwards to the start of the forecast interval that contains the customer order date. After the start of the interval is reached, forecast is consumed from the customer order date to the end of the customer order's interval. Consumption then continues backwards from the start of the customer order's forecast interval to the start of the forecast interval, and then forward from the end of the customer order's interval to the end of the forecast interval. **Default value**: Normal **Note:** For all settings in this field other than AlternatingBackward and AlternatingForward, the forecast consumption window can be defined in terms of either the ForecastCalendar or the Second Calendar (based on the setting in **PartType.ForecastConsumptionCalendar**). For more information about forecast consumption, see "Forecast consumption" on page 1431.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Independent SafetyLead TimeRule	Specifies whether orders satisfying consensus forecast or independent demands for parts of this type should use **SafetyLeadTime** as defined on the **Part** and/or **PartSource** tables. Note that this setting is only applicable when **UseSafetyTimeLeadTime** is set to either "Y" or "Soft" (safety lead time is always ignored if that option is set to "N") Valid values are: • **FromDemandType**—safety lead time usage on orders satisfying consensus forecast or independent demands for parts of this type is determined by demand type. This is determined form the **DemandType.SafetyLeadTimeRule** field setting referenced by the applicable **IndependentDemand** or **PartCustomer** record). If the applicable **PartCustomer**record has a Null reference to **DemandType**, then a value of "Use" is assumed. • **Ignore**—safety lead time is never applied on orders satisfying consensus forecast or independent demands for parts of this type. • **Use**—safety lead time is applied on orders satisfying consensus forecast or independent demands for parts of this type. **Default value:** Use	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
OnTimeAllocation Rule	Following the normal allotment of supply to demand in CTP, this setting specifies whether any late supplies for the part can potentially be reallocated to improve the on-time quantity of demands. This process attempts to maximize the total on-time quantity across late demands for the part at the risk of making the available date on late demands worse than they otherwise would be. Valid values are: • **ImproveWithTolerance**—After the normal allotment of supply to demand, additional logic attempts to improve the total on-time quantity on late demands even if doing so makes the calculated available date on some or all of those late demands worse. When not all demands can be satisfied on time, any supplies that are late to their demand (or later than a tolerance defined in **Part.UnsatisfiedDemandTolerance**) are subject to additional on-time reallocation logic. Thus, any demand that is available late (or later than the number of forecast intervals specified in the **UnsatisfiedDemandTolerance** field) can potentially have its on-time quantity improved and/or its final available date made worse. • **Normal**—Normal allotments of supply to demand as made by capable-to-promise logic are used. If not all demands can be satisfied on time, the normal behavior of allocating limited supply to demands based on priority, earliest due date, and so on is used. **Default value**: Normal **NOTE**: This field was added in Version 2016.2, Service Update 27. For more information about its usage, see "Improve on-time quantity by reallocating late supply" on page 1471.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
OrderPointDate Rule	Indicates the date on which scheduled supply (scheduled receipts, inventory transfers, and supply allocations) for order point parts should be included when calculating whether the threshold for creating new planned orders has been crossed. Valid values are: • **Due**—Scheduled supply is considered as of its ReschedDate (calculated due date). As a result, supply that is already started, but not yet due, is ignored when determining if the order point threshold has been reached. Planned orders might therefore be created to cover this timing gap. . • **Start**— Scheduled supply is considered as of its CalcStartDate (calculated start date). As a result, the order point threshold will not be crossed if a sufficient quantity is available in a started, but not yet due, scheduled supply. This is the recommended value. **Default value**: Due **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead). Use of this field provides backward compatibility with versions prior to 2014.4.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
OrderPoint Increase	Indicates if a temporary increase in a part's order point threshold should be considered to guard against near-term spikes in demand that might push inventory below zero before more supply can be planned. This setting is only applicable to order point parts (that is, parts where **SafetyStockRule** is set to either "OrderPointAverage", "OrderPointOver", or "OrderPointUnder"). Additionally, this setting relies on the part's **PlanningCalendars.OrderPointIncrease** setting which specifies a calendar whose intervals are used, starting from the RunDate, to look for demand within the part's lead time. Valid values are: • **Calculate**— an increase in the order point is considered for each **PlanningCalendars.OrderPointIncrease** interval where there is demand within lead time. If the demand within lead time (plus any additional demand within the OrderPointIncrease interval) exceeds the effective order point, then the amount it exceeds it by is temporarily added to the order point. This increase is effective until the next OrderPointIncrease interval. • **Ignore**—order point threshold is never inflated to cover spikes in demand that will consume supply before it can be replenished. • **Sum**— an increase in the order point is considered for each **PlanningCalendars.OrderPointIncrease** interval where there is either demand within lead time or a value is provided in the **TimePhasedSafety** table. If the demand within lead time (plus any additional demand within the OrderPointIncrease interval) exceeds the effective order point, then the amount it exceeds it by is temporarily added to the order point. As well, if an effective record is found in the **TimePhased Safety** table, then its quantity is also added to the effective order point (if multiple **TimePhasedSafety** entries are found in the interval then the one with the largest **Quantity** is used). The total increase is effective until the next OrderPointIncrease interval. Note that if **SafetyStockQuantity** is "TimePhasedQuantity" or "TimePhasedQuantityToLatest", then this option and **Calculate** are the same. • **SumOffset**—identical to Sum, except that increases are never performed in the first interval (containing the SafetyStockDate). **Note:** This field is only used by parts having a Null reference in **Part.SafetyStockPolicy**, and gives backward compatibility with versions prior to 2014.4 (prior to **SafetyStockPolicy** table).	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies, for each PartType, a processing rule that describes the logic used for netting calculations. Valid values are: • **Ignore**—do not process this part. All supply and demand records for this part are ignored by netting and do not display in the Activity Table. Because all supply and demand are ignored, no planned orders or available to promise records are generated. This value is often used for reference or vendor managed parts. • **MRP**—Process the part as a standard MRP part. Forecast records (Independent demands with an **Order.Type.ProcessingRule** = 'SalesForecast') are ignored by netting. Available to promise records are not generated. Planned orders are created. • **MPS**—Forecast records (independent demands with a **DemandType.ProcessingRule** = 'SalesForecast') are consumed by sales orders (independent demands with a **DemandType.ProcessingRule** = 'SalesActual'). Available to promise records are generated. Forecast records (independent demands with a **DemandType.ProcessingRule** = 'ProductionForecast') are NOT consumed by sales orders but are valid demand for MPS Parts. • **MPSConfig**—used to explode forecast on a parent part and drive demand to its component parts. This configuration is used for forecasting by product family. The part representing the product family has a **PartType** of MPSConfig (planning BOMs, which are created to define the relationship between the forecasted part and its components, and have MPSConfigDemandSource = 'Y'). Actual demand for component parts consumes forecast at the parent part. Unconsumed forecast results in planned orders and constraints are ignored. No other types of supply or demand qualify for netting on MPSConfig parts. **Default value:** MRP **Note:** If a part does not have at least one active part source, then it is not processed by Netting logic regardless of the setting in this field (always treated as if it were set to "Ignore").	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
SafetyLeadTimeCalendarRule	Specifies the calendar used for safety lead time values provided on the **Part**, **PartSource**, and/or **SafetyLeadTimeProfileDetail** tables. Safety lead time is applied as a buffer against demand dates when determining planned order due dates for the part. Valid values are: • **Everyday**—Safety lead time is expressed in terms of the standard Everyday (Gregorian) calendar. This setting allows non-working days to count towards safety lead time. • **TimeUnits**— Safety lead time is expressed in terms of the part's **PlanningCalendars.TimeUnits** calendar (typically, this is a Workday calendar). This setting corresponds to how safety lead time was always applied prior to Service Update 1902. **Default value**: TimeUnits **Note:** This field was added in Service Update 1902. **Note:** Any demands that do not fall on the part's **PlanningCalendars.TimeUnits** calendar are first rounded back to the nearest **TimeUnits** interval before applying safety lead time (regardless of the setting in this field).		

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
SafetyStockDate Rule	Specifies, for each part type, the date on which to plan for safety stock. Valid values are: • **FirstDemand**—maintain safety stock level as of the date of the earliest demand record. The **PlannedOrder.DueDate** will be on the date of the first demand - (**Part.SafetyLeadTime + PartSource.SafetyLeadTime)**. • **FirstDemandOrLeadTime**—if the part has demands, then a planned order is generated to cover safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **RunDate + LeadTime** (if **OrderPolicy.OrderGenerationRule** is 'AnyTime' or 'AfterRunDate'), OR at the later of **RunDate + LeadTime** or **PTFDate** (if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'). • **FirstDemandOrRunDate**—if the part has demands, then a planned order is generated to cover safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **RunDate** (if **OrderPolicy.OrderGenerationRule** is 'AnyTime' or 'AfterRunDate'), OR at **PTFDate** if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'). • **FirstDemandOrPast**—if the part has demands, then generate a planned order to maintain safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **Past** if **OrderPolicy.GenerationRule** is 'AnyTime', OR at **RunDate** if **OrderPolicy.GenerationRule** is 'AfterRunDate', or at **PTFDate** if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'. • **LeadTime**—maintain safety stock level after RunDate (**Part.PlanningCalendars.RunDate.Value**) + **Part.FixedLeadTime** + **Part.LeadTimeAdjust** + **PartSource.Source.TransitTime** + (**Part.VariableLeadTime** * **Part.SafetyStockQty**). This calculation would result in a due date for the planned order to satisfy safety stock. If either or both of **Part.SafetyLeadTime** or **PartSource.SafetyLeadTime** is greater than 0 then this is added to the above calculation in creating the DemandDate on the PlannedOrder. The objective is to have the **PlannedOrder.StartDate** be achievable. However, lot sizing or other demands may make the start date earlier in the presence of **PartSource.VarLeadTime**. OnHand values may make the actual **PlannedOrder.Quantity** less and thus the **PlannedOrder.StartDate** later. • **LeadTimeReportUnplanned**—similar to the “LeadTime” option in that safety stock is only maintained/planned	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	after RunDate plus applicable lead times. However, with this setting safety stock levels are still calculated, for reporting purposes only, within lead time. These unplanned safety stock levels are reported in the Activity table with a Type value of "UnplannedSafety". • **Past**—maintain safety level even if it means placing a past due order. The **PlannedOrder.DueDate** may be Past. • **RunDate**—maintain safety stock level only after **Part.PlanningCalendars.RunDate.Value**. The due date of a planned supply would be equal to the run date. A planned order will have a DemandDate associated to it if this rule is used, and the DemandDate would be RunDate + SafetyLeadTime. Ordering policies or planning time fence may override any of these values. **Default value:** LeadTime **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead). Use of this field provides backward compatibility with versions prior to 2014.4. **Note:** When using the "LeadTime" or "LeadTimeReportUnplanned' settings, the lead time from the part's primary part source is used in determining the date from when safety stock can be planned. For example, if a secondary part source has shorter lead time and is used to satisfy a safety stock demand, it still cannot plan it any earlier than **RunDate** plus lead time from the primary part source. **Note:** This field was modified in Version 2016.2, Service Update 8.		

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
SafetyStock QuantityRule	Specifies how safety stock level is determined. Valid values are: • **FixedQuantity**—safety stock level calculated as a fixed quantity using the value in **Part.SafetyStockQty**. • **FractionOfDemand**—safety stock levels calculated the same as PercentOfDemand, except that the value is interpreted as a fraction (for example, 0.1 means 10%). • **PercentOfDemand**—safety stock levels calculated using a percentage (for example, 10 means 10%) of MRP total demand per defined interval. Demand intervals are calculated (beginning from the maximum of the **Part.PlanningCalendar.RunDate** or safety stock date) using the **Part.SafetyStockPolicy.PercentSafety Interval** value. The number of intervals is calculated using the **Part.PercentSafetyIntervalCount** value. All demands during this interval are summed and the **Part.PercentSafetyPercent** is applied against the total demand quantity which results in a DynamicSafetyStock quantity for that interval. • **RangeOfCoverage**—safety stock levels are calculated to satisfy a specified number of days of a part's average daily requirements, and can be configured for three distinct date zones. All parameters used to calculate these safety stock levels are configured through range of coverage profiles in the **RangeOfCoverage** table. If this option is selected for a part, the part's range of coverage profile is taken from the value in **SafetyStockPolicy.RangeOfCoverage**. For more information, see "RangeOfCoverage table" on page 855. • **RangeOfCoverageLeadTime**—safety stock levels are calculated to satisfy the part's average daily requirement over its lead time plus an optional buffer quantity. Most parameters used to calculate these safety stock levels are configured through range of coverage profiles in the RangeOfCoverage table. However, with this setting only the first zone is active and the average daily requirement is multiplied by lead time rather than the FirstZoneTarget value. • **TimePhasedQuantity**—safety stock levels change over time. The TimePhasedSafety table must be populated with the following fields: Part, Site, Date (earliest date this safety stock quantity is applicable), and Quantity. For example, safety of 5 is effective beginning 01-01-05 and a quantity of 10 is effective beginning 06-01-05. This means that a safety stock of 5 is no longer effective as of 05-31-05. For more information, see "TimePhasedSafety table" on page 655. This setting is identical to TimePhasedQuantityToLatest except in cases where the	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	part's safety stock level decreases after the last demand. • **TimePhasedQuantityToLatest**—This setting is identical to TimePhasedQuantity except in cases where the safety stock level decreases after the last demand for the part. If the safety stock level decreases after the last demand for the part, then the quantity of the last planned order will be reduced to ensure the resulting inventory level meets the latest value defined in the TimePhasedSafety table. If using lot sizing, the OrderPolicy.LotSizeLastOrder field should be set to "N" to ensure this behavior. Otherwise, if OrderPolicy.LotSizeLastOrder is set to "Y", then last planned order will only get as close as it can to the latest defined safety stock level while still respecting all lot-sizing rules. • **TimePhasedRangeOfCoverage**—safety stock levels are calculated to satisfy a specified number of days of a part's average daily demand, and are configurable for any number of zones. When using this setting, records in the TimePhasedSafety table are used to define the start of each range of coverage zone as well as the number of days of the part's average daily requirement that safety stock should cover in each of those zones. Additionally, parameters from the RangeOfCoverage table are used to specify how demands are collected for use in calculating the average daily requirement, as well to set the calendar whose intervals can be used to generate new safety stock levels. **Default value:** FixedQuantity **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead).		

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
SafetyStockRule	Specifies, for each part type, the rule used to control the maintenance of safety stock. For example, under what conditions will orders to replenish safety stock be generated? Valid values are: • **Always**—always maintain safety stock level as determined by the **SafetyStockQuantityRule** setting. If an order is required to bring inventory to the **Part.SafetyStockQty,** then generate a planned order or expedite an existing order as early as allowed based on order rescheduling rules (**Type.ProcessingRule /Status.RescheduleRule)**, Order generation rules (**PartSource.OrderPolicy.OrderGenerationRule**) and the **Part.SafetyStockDate** rule. • **IfActivity**—if there are non-ignored ScheduledReceipts (**Status.IgnoreSupply** does not equal 'Y' or the Quantity does not equal 0) or non-ignored IndependentDemand (**Order.Type.ProcessingRule** does not equal 'Ignore', **Status.ProcessingRule** does not equal 'Ignore', or Quantity is not equal to 0) or dependent demands. • **IfDemand**—orders for bringing inventory to safety level will be generated only if there is a non-ignored demand record for the part (**Order.Type.ProcessingRule** does not equal 'Ignore', and **Status.ProcessingRule** does not equal 'Ignore' or quantity is not equal to 0). This includes dependent and independent demand. • **Ignore**—do not maintain safety stock. Ignore the effective safety stock level. • **OrderPointAverage**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the **SafetyStockQuantityRule** field. The order quantity is determined by the value in **Part.AverageQty** (or **Part.SafetyStockQty** if **Part.AverageQty** is less than or equal to 0), and orders of this size are generated until inventory levels meet or exceed the calculated order point threshold. The start date of the generated planned orders matches the DueDate of the demand they are satisfying. • **OrderPointOver**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the SafetyStockQuantityRule field. Order quantities are set to the largest number that will bring inventory levels closest to the maximum of either the order point threshold or Part.AverageQty without exceeding it. If using this setting, it is recommended to have SafetyStockQuantityRule set to "FixedQuantity", and ensure Part.AverageQty is set to a value greater than Part.SafetyStockQty. RapidResponse will then plan orders using these two fields as an upper and lower bound,	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	respectively. • **OrderPointUnder**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the **SafetyStockQuantityRule** field. Order quantities are set to the smallest number that will bring inventory levels equal to or greater than the maximum of either the order point threshold or **Part.AverageQty**. This setting is intended to keep inventory levels above the order point threshold. The start date of the generated planned orders will match the DueDate of the demand they are satisfying. • **Soft**—safety stock is an available buffer. ReschedDate and RecommendedDate are left as DueDate if existing supplies retain inventory >=0 rather than >= SafetyStock. **Default:** Always **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead). Use of this field provides backward compatibility with versions prior to 2014.4.		
SalesActual Consumption Order	Specifies the order in which sales actuals are used to consume a part's forecast. Valid values are: • **ByDueDate**—Actual demands are used for consumption in due date order (earliest due actuals attempt to consume forecast first). • **ByPriority**— Actuals demands are used for consumption in priority order (higher priority actuals attempt to consume forecast first). If there are multiple sales actuals with the same priority, then the earliest due of those actuals consume forecast first. If **PartType.CommitLevel** is set to "Low", this setting is identical to "ByDueDate" (priority is not considered). When multiple sales actuals with the same due date are eligible to consume forecast based on the settings described above, then the independent demand with the lower **Order.Id** consumes first (or the demand with the smaller **Quantity** if the Ids are the same). **Default value**: ByDueDate **Note:** This field was added in Version 2016.2, Service Update 3.	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
Substitution Preference	Defines the sequence in which global part substitution logic uses different types of supply when trying to satisfy demand for the primary part configured in a substitute part relationship (this setting is only applicable to those primary parts). Note that within the different options described below, a preference between supply of a primary part and supply of its substitutes is determined based on comparing the value in the primary part's **Part.PrimarySubstitutionSequence** field, and the **AlternatePart.Sequence** field value associated with each of its substitutes. Input supply from part's associated with lower sequence values are generally used first (and only substitutes having a higher sequence value than the primary are eligible to have new planned orders created to satisfy demand from the primary part). Valid values are: • **OnHand**—preference is to satisfy primary part demand with existing on-hand inventory. That is, RapidResponse first looks for on hand inventory of the primary and substitute parts. If the demand remains unsatisfied, then scheduled receipts for primary and substitute parts are considered. If the demand still cannot be satisfied, then RapidResponse tries to create an on-time planned order for the primary part, and if this is not possible then it tries to create an on-time planned order on an eligible substitute. Finally, if the demand still cannot be satisfied, then the least late planned order is created on the primary or any of its eligible substitutes. • **Primary**—preference is to satisfy primary part demand with part's having lower **PrimarySubstitutionSequence** (primary) or **Sequence** (substitute parts) values. That is, RapidResponse first looks for on hand inventory of the part having the lower sequence value, followed by scheduled receipts of the part having the lowest sequence value, and then tries to create on-time planned orders for the part with the lowest sequence value (if eligible). If the demand still cannot be satisfied, then RapidResponse will look for supplies of the part having the next lowest sequence value. If, after all parts have been evaluated in sequence order, the demand still remains unsatisfied, then RapidResponse creates the earliest possible planned order for either the primary part or one of its eligible substitutes. • **ScheduledSupply**—preference is to satisfy primary part demand with existing on-hand inventory or scheduled supply. RapidResponse first looks for on hand inventory and then scheduled receipts of the primary part and/or its substitutes. Otherwise, if the demand cannot be satisfied by input supplies, then RapidResponse tries to create an on-time planned order for the primary part, and if this is	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
	not possible then it tries to create an on-time planned order on an eligible substitute. Finally, if the demand still cannot be satisfied, then the least late planned order is created on the primary or any of its eligible substitutes. For more information on substitution sequences and how they are used with the different settings in this field, see "Define when part substitutes are used" on page 1553. **Default**: ScheduledSupply		
Supply Allocation Rule	Specifies how supplies are allocated to demands during capable-to-promise calculations. Supplies are allocated to demands to try to satisfy demand dates. However, where supplies are unavoidably late, the late supplies are allocated to lower priority demands and higher priority demands are kept on time (or as close to on time as possible). Valid values are: • **ByAvailable**—Supplies are allocated by their available date (that is, the date that capable-to-promise calculations determine the supply should be available based on materials and constraint). In cases of expiring supply, when multiple supplies are available to satisfy a demand on time, those supplies are then allocated based on expiry date (earliest expiring supplies are used first). • **ByMRPPlan**—Supplies are always allocated by their demand date (either ScheduledReceipt.DemandDate or PlannedOrder.DemandDate). Demand date refers to the date of the first demand netting calculations determine should be satisfied by the supply. This setting is useful when an allotment results consistent with Netting calculations and certain enterprise systems is desired. However, this setting is generally not recommended as it gives allocation results that do not consider prioritized or constrained demands. **Default:** ByAvailable	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
SupplyShareRule	Specifies whether the part's demands are eligible for the fair or equal sharing of limited supplies. The **OrderPriority.AllocationRule** field is used to initially enable either the fair-sharing or equal-sharing of supply for all demands at a given priority level. This setting then allows for that supply sharing logic to be either turned off at the part level, or kept within a defined time horizon. For example, parts at one level in the product structure might need limited supplies to be shared amongst their High priority demands, while parts at another level in the product structure might not require supply sharing logic for their High priority demands. Valid values are: • **Ignore**—supply sharing is always disabled for parts of this type, regardless of the setting in the AllocationRule field. That is, when satisfying part demands of a given priority level, supplies are always allocated to those demands on a first-come, first-served basis. • **Use**—supply sharing is enabled for parts of this type on all demands belonging to priority levels where the AllocationRule is set to either "FairShare" or "EqualShare". Thus, when there is insufficient supply available to satisfy all prioritized on time, the supply is fairly or equally shared between all demand at a given priority level (thus, any resulting lateness is also shared between those demands). • **WithinFence**— supply sharing is enabled for parts of this type on demands having priority levels where the AllocationRule is set to either "FairShare" or "EqualShare", and where the due date falls within the supply sharing window as set by Part.SupplyShareFence. Typically, used when the sharing of limited supplies is intended only to satisfy near-term demands for the part, but where first-come, first-served supply allocations are acceptable later in the planning horizon. **Default value**: Use	String (Enum)	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
UseDaysSupply	Specifies whether a days of supply policy is used to accumulate demands and group planned orders based on settings in the **Part**, **PartType**, and **PlanningCalendars** table. Note that if there is a non-Null reference in **Part.SafetyStockPolicy**, then the referenced policy enables or disables days of supply logic and this setting is ignored. Valid values are: • **Y**—use Part.NumberOfDaysSupply to group planned orders according to the setting in DaysSupplyRule. Planned orders are grouped for the lesser of 'n' time units/periods or until a non-reschedulable supply (ScheduledReceipt.SupplyType.ProcessingRule = 'NonReschedulable') is seen. Planned order quantities will not exceed the PartSource.MaximumQty. • **N**—Part.NumberOfDaysSupply value is ignored. **Caution:** Avoid using UseDaysSupply with parts with components or constraint. UseDaysSupply can cause order commitments to be violated by new demands (Increased supply quantity may result in a supply being late, thereby making all demands for that supply late). **Recommendation:** Use part source ship calendars, rather than days supply, to align periodic deliveries. **Default value**: N **Note:** In most circumstances, UseDaysSupply = 'Y' overrides CommitLevel. However, if a rule that respects priority is applied (such as **ByPeriodEndWithPriority** and **ByPeriodWithPriority**), CommitLevel is used and priority is maintained.	Boolean	
UseLeadTime Adjust	Specifies, for each part type, whether a lead-time adjustment is applied on a part basis (aside from standard fixed, variable, and safety lead times). Valid values are: • **Y**—use **Part.LeadTimeAdjust** when calculating lead time. • **N**—do not use **Part.LeadTimeAdjust** when calculating lead time. **Default value:** Y	Boolean	
UseLTForFlatBill	Specifies whether a lead-time offset is included when calculating the FirstDate and LastDate fields in the FlatBillDown table. Valid values are: • **Y**—lead time is included. • **N**—lead time is ignored. **Default value**: Y	Boolean	

Table 7.41: PartType (Mfg) fields

Field name	Description	Data type	Key
UseNegativeOnHand	Specifies, for each part type, whether Negative Onhand quantities are considered in netting. Valid values are: • **Y**—negative total nettable on hand is treated as a demand at time Past. • **N**—any negative on hand is ignored. **Default value**: N	Boolean	
UseSafetyLeadTime	Determines whether **SafetyLeadTime** values as specified on the **PartSource, Part**, and/or **SafetyLeadTimeProfileDetail** tables are used for parts of this type. For example, when setting a planned order due date, safety lead time can be used to offset the demand due date to account for fluctuations in supply and demand (the supply is planned earlier than actually required by the demand). Valid values are: • **Y**—safety lead time is always used for parts of this type. • **N**—safety lead time is never used for parts of this type. • **Soft**—safety lead time is used for parts of this type, however reschedule recommendations are suppressed on scheduled receipts whose DueDate satisfies the demand due date. **Note: IndependentSafetyLeadTimeRule** and **DependentSafetyLeadTimeRule** settings provide further control over how safety lead time is applied when planning orders to satisfy specific types of demand. **Note:** If safety lead time should be applied to customer request dates, then the use of settings on the **OrderPriority.TwoDatePlanningRule** field, such as "UseWithHardSafetyLeadTime" or "UseWithSoftSafetyLeadTime", should be considered. **Default value:** N	String (Enum)	
Value	The string value for the part type. This value appears in the part record. **Default value**: blank string	String	Yes
Parts	Set of **Part** records that reference this PartType value.	Set	
PoolMaps	Set of **PoolMap** records that reference this PartType value.	Set	

PlanningCalendars table

Part calendars give the working schedule information for calculating dates. Multiple **PlanningCalendars** records can be used to allow different parts to have different operating schedules. Purchase parts may operate on schedules defined by the vendor.

Table 7.42: PlanningCalendars (Mfg) fields

Field name	Description	Data type	Key
Allocation Calendar	The calendar giving the dates for intervals to perform fair-share and equal-share allocation. All demands with the same **OrderPriority** value in the interval are used in the allocation calculation. This calendar also defines the intervals used for period effective priorities (that is, priority values where **OrderPriority.PeriodEffective** is set to 'Y'). All demands that aren't satisfied within a given interval are given a higher priority than any demands in subsequent intervals. **Referenced table**: Calendar	Reference	
CoByProduct PlanningInterval	The calendar whose intervals are used in co-product and by-product calculations. The specified calendar defines an interval over which demands on a primary product are collected and sorted together, and any required planned orders on the primary product are created at the beginning of the period. For example, if set to a "weekly" calendar, then all required planned orders would be created at the beginning of the week. This setting can be used to help ensure that generated co-product or by-product supplies are available to satisfy demands over the duration of the period, and can thereby help to reduce potential excess resulting from co-product and by-product supplies. Note that a given primary product and all of its associated co-products and by-products should reference the same CoByProductPlanningInterval calendar. If excess on co-products and by-products is not a concern, or this sort of "pre-planning" logic is not required, then this calendar should be set to an "everyday" calendar. This calendar is ignored by parts not involved in any co-product or by-product configurations. For more information, see "Specify co-product planning calendar intervals" on page 1589. **Referenced table:** Calendar	Reference	
CommonPool	Specifies the common pool to be used (by calculations such as forecast consumption and netting). For example, if the control setting in the **MUEPoolNettingType.FlowToCommonPool** field is "NetCommon", then when a new planned order is required it should be planned in the specified pool. **Referenced table:** Pool	Reference	
ControlSet	The control set associated with this PlanningCalendars record. **Referenced table**: ControlSet	Reference	Yes

Table 7.42: PlanningCalendars (Mfg) fields

Field name	Description	Data type	Key
DataDate	The calendar to use (First entry) as the date when data was last updated. **Referenced table:** Calendar	Reference	
Description	A description of this set of calendars. An explanation of the code in the Value field. **Default value**: blank string	String	
ExpiryCalendar	The calendar whose intervals are used in material expiry calculations. For example, this calendar is used along with the value provided in the **Part.MinimumShelfLife** field (for calculating the EarliestExpiryDate on demands) and along with the value provided in the **PartSource.CTPIntervalsToExpiry** field (for calculating the ExpiryDate on supplies). Typically, this might be set to an "Everyday" calendar. **Referenced table:** Calendar	Reference	
Forecast Calendar	The "outer" calendar giving forecast interval dates (buckets in which forecast is received). For example, Month. **Referenced table:** Calendar	Reference	
OrderPoint Increase	The calendar giving the intervals used in calculations determining if a temporary increase in a part's order point threshold should be applied. For more information about order point calculations, see "Planned orders with order point" on page 1351. **Referenced table:** Calendar	Reference	

Table 7.42: PlanningCalendars (Mfg) fields

Field name	Description	Data type	Key
PercentSafety Interval	Identifies the calendar used in calculating safety stock when **SafetyStockQuantityRule** is either "PercentOfDemand" or "FractionOfDemand". That is, the bucketing upon which the calculation is done is based on a period which is "N" intervals in length. Where intervals is defined by the referenced calendar value and "N" is set to **Part.PercentSafetyIntervalCount**. For example, to have a safety value that was based on four weeks of demand, this field could be set reference a "Weekly" calendar and **Part.PercentSafetyIntervalCount** set to "4". **Referenced table:** Calendar **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead). Use of this field provides backward compatibility with versions prior to 2014.4.	Reference	
PeriodSupply DueCalendar	Defines the calendar giving valid planned order due dates when **PartType.DaysSupplyRule** = "ByPeriod", "ByPeriodWithPriority", "ByPeriodEnd", or "ByPeriodEndWithPriority". For example, if planned orders should be due at the beginning of the week, then set this to a weekly calendar. Or, if planned orders should be due on the first demand date in a given interval, set this to a work day calendar (such as, Part.PlanningCalendars.TimeUnits). **Referenced table**: Calendar **Note:** This field is ignored for parts that have days of supply logic enabled through a reference to the **DaysSupplyPolicy** table, and equivalent calendar references on that table are then used instead.	Reference	
PeriodSupply Interval	Defines the calendar intervals over which demands are accumulated when **PartType.DaysSupplyRule** = "ByPeriod", "ByPeriodEnd", "ByPeriodEndWithPriority" or "ByPeriodWithPriority". For example, demands might be accumulated weekly or monthly. **Referenced table**: Calendar **Note:** This field is ignored for parts that have days of supply logic enabled through a reference to the **DaysSupplyPolicy** table, and equivalent calendar references on that table are then used instead.	Reference	

Table 7.42: PlanningCalendars (Mfg) fields

Field name	Description	Data type	Key
RangeOf Coverage	The range of coverage profile for parts using this **PlanningCalendars** record. This profile is relevant only for those parts having a **PartType.SafetyStockQuantityRule** value of 'RangeOfCoverage'. For more information about range of coverage profiles, see "RangeOfCoverage table" on page 855. **Referenced table**: RangeOfCoverage **Note:** This field is only used by parts that have a Null reference in **Part.SafetyStockPolicy** (otherwise, if that reference is populated then the corresponding field on the **SafetyStockPolicy** record is used instead). Use of this field provides backward compatibility with versions prior to 2014.4.	Reference	
RunDate	The Calendar.Value that is used to determine the date the data is extracted from the enterprise data source. **Note:** The RunDate.FirstDate calculated field can be used to get this date for reporting purposes. This method of getting the run date into the system allows the date to be in a table that is extracted but can be specified by a part property. **Referenced table:** Calendar	Reference	
SecondCalendar	The "inner" calendar over which forecast values should be spread. For example, Week or Workday. **Referenced table:** Calendar	Reference	
Substitution Tolerance Calendar	The calendar used in substitute part calculations when determining how far out to search for existing supply of a primary part before looking for supply of a given substitute part (or, depending on configuration, how far out to search for supply of a given substitute part before looking for supply of the primary part). This value is used in conjunction with the **Part.SubstitutionTolerance** field. **Referenced table**: Calendar	Reference	
TimeFence Calendar	The calendar giving valid dates on which the demand Time Fence (DTF) will fall. Dates not on a calendar marker are rounded back to the next previous calendar marker. **Referenced table:** Calendar	Reference	
TimeUnits	The calendar giving valid dates for 'units of time' parameters (usually WorkDay). This allows for calculations to be based on shop time not on elapsed time. **Referenced table:** Calendar	Reference	

Table 7.42: PlanningCalendars (Mfg) fields

Field name	Description	Data type	Key
Value	The name (string value) for this set of Calendars. This value appears in the Part.PlanningCalendars_Value field. It is recommended that this value not contain spaces, special characters, or numbers. If any of these are used in the calendar name, then unexpected errors might occur when the calendar is used in query expressions that perform date arithmetic. **Default value**: blank string	String	Yes
Weekly	The name of the calendar to be used for weekly reporting on CRP Analysis as well as when calculating average days of supply for Inventory Analysis. **Referenced table:** Calendar	Reference	
Parts	Set of **Part** records that reference this PlanningCalendars value.	Set	
WorkCenters	Set of **WorkCenter** records that reference this PlanningCalendars value.	Set	

ProcessInstanceType table

The ProcessInstance type table contains configurable settings that define processing at the process instance level. For example, this table sets the calendar for the process instance.

Table 7.43: ProcessInstanceType (ProcOrch) fields

Field name	Description	Data type	Key
Calendar	The calendar used for calculating the start and finish dates in the process instance. **Referenced table:** Calendar	Reference	
ControlSet	The control set associated with this ProcessInstance record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this process instance type.	String	
RunDate	The calendar used in calculating the activity due dates. **Referenced table:** Calendar	Reference	
Value	A unique string identifier for this type of process instance.	String	Yes
ProcessInstances	A set of process instances.	Set	

ProductionWheelType table

The **ProductionWheelType** table defines the rules and values used to manage production wheels in RapidResponse.

This table was added in Service Update 1902.

Table 7.44: ProductionWheelType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	A reference to the control set associated with this ProductionWheelType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of the production wheel type.	String	
UnsatisfiedSupplyRule	This setting determines what happens to unsatisfied supply quantities that are assigned to this cycle but cannot be built because the cycle has run out of constraint. Valid values are: **Defer**—Unsatisfied supply quantities are handled in the next cycle where the part can be built. **Ignore**—Unsatisfied supply quantities are ignored and not built in the cycle. Instead the supplies are assigned an available date of 'Future'.	String (Enum)	
UnusedConstraintRule	This setting determines how unused constraint is handled. Unused constraint is the constraint remaining after all supply quantities have been produced in the production wheel cycles where the minimum constraint consumption target has not been reached. Valid values are: **SatisfyFutureSupply**—Satisfy supplies that are normally produced in future cycles but are eligible to be built in the current cycle of this wheel. If there are still insufficient supplies to meet the minimum constraint consumption value for the production wheel, nothing else is built in this cycle. **Ignore**—If the constraint used in the current cycle does not meet the minimum constraint consumption value, ignore the value set for ProductionWheelConstraint.MinimumConstraintConsumption.	String (Enum)	
Value	A unique identifier for the production wheel type record.	String	Yes
ProductionWheels	A set of ProductionWheel records that reference this production wheel type.	Set	

ProjectType table

The **ProjectType** table contains configurable settings that define processing and calculations at the project level. For example, this table sets the project's working calendar, and specifies whether project dates are calculated from a given start date or finish date.

This table's records can be edited using the **Project Type** worksheet in the **Control Tables for Project Management** workbook.

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
AddActual DurationTo StartDate	When scheduling projects from finish, this setting indicates whether an in progress task's **StartDate**, or its **StartDate** + **ActualDuration**, is used when constraining a predecessor with either a "StartToStart" or "FinishToStart" relationship. Valid values are: • **N**—only the task's **StartDate** is used in determining predecessor dates. • **Y**—the task's **StartDate** + **ActualDuration** is used in determining predecessor dates. Additionally, if the task is complete, then its predecessor links are ignored. **Default value**: N	Boolean	
AllowNonZero Milestones	Specifies how the calculated duration on milestone tasks in the project is determined. Valid values are: • **N**—milestones always have their calculated duration set to 0 (zero). This is the default setting as milestones are used to mark important points in the project schedule and aren't typically assumed to have actual work associated with them. • **Y**—milestones have their duration calculated in the same way as normal tasks (based on actual work and resources associated with this task). With this setting, a milestone can have a duration other than 0 (zero). **Default value**: N	Boolean	

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
AllowTaskOnNonWorkingDay	Enables project tasks to potentially start or finish on non-working days if those days coincide with the start or finish dates on certain types of predecessor or successor tasks (and those tasks use different working calendars). Valid values • **N**—tasks are never allowed to start or finish on non-working days. • **Y**—tasks are allowed to start or finish on non-working days if any of the in projects scheduled from the start date, a task can finish on a non-working day if it has a "finish to finish" or "start to finish" relationship with a predecessor task. In projects scheduled from the finish date, a task can start on a non-working if a has a "start to start" or "start to finish" relationship with a successor task. **Default value**: N	Boolean	
ASAPBackwardTotalSlackRule	Indicates how total slack is calculated on tasks that have an "AsSoonAsPossible" constraint and belong to a project scheduled from finish. • **LateMinusEarly**—Total slack for such tasks is calculated as *min(LatestFinishDate - EarliestFinishDate, LatestStartDate - EarliestStartDate)*. • **Zero**—Total slack for such tasks always returns a value of 0. **Default value**: Zero	String (Enum)	
Bonus Accumulation Rule	If there are multiple **BonusScheduleByDate** or **BonusScheduleByInterval** records effective within the bonus period (between the project's **CalcFinishDate** and **BonusDate**), this setting determines how the bonus values on those records are used. Valid values are: • **Cumulative**—bonuses are calculated using the **OneTimeBonus** and **PerIntervalBonus** values defined on each effective entry in the bonus period and applied within their effective ranges only. With this setting there is the possibility for PerInterval bonuses that change over time along with multiple OneTime bonuses. • **FirstInterval**—bonuses are calculated using the **OneTimeBonus** and **PerIntervalBonus** values defined on the first effective entry within the bonus period and applied throughout the entire bonus period. This means either the **BonusScheduleByDate** record with the earliest **EffectiveOutDate** within the bonus period or the **BonusScheduleByInterval** with the highest **Interval** value within the bonus period is used. **Default value:** FirstInterval	String (Enum)	

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
BonusCalendar	The calendar used when calculating bonuses to be applied to a project. For example, should bonus revenue be earned on a daily or weekly basis. Referenced table: Calendar (this reference is nullable)	Reference	
BonusRule	Specifies the schedule, if any, to use for calculating project bonuses. This defines the one-time and per interval bonuses that can be earned and applied between the project's calculated finish date and bonus date. Valid values are: • **Date**—bonuses are applied based on dates and values defined in the **BonusScheduleByDate** table. The entries that apply to a given project are set through its **BonusSchedule** reference. • **Interval**—bonus values are applied based on bonus calendar intervals and values defined in the **BonusScheduleByInterval** table. The entries that apply to a given project are set through its **BonusSchedule** reference. • **None**—bonuses cannot be earned on projects of this type. **Default value:** None	String (Enum)	
Calendar	The calendar used when calculating start and finish dates on any tasks in the project for which the **TaskType.Calendar** reference is null. **Referenced table:** Calendar	Reference	
ControlSet	The control set associated with this **ProjectType** record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this project type.	String	
FreeSlack Calendar	Specifies the calendar used when calculating free slack (**Task.TotalSlack**) for tasks belonging to projects of this type. Valid values are: • **ProjectCalendar**—free slack is calculated using the project calendar (**ProjectType.Calendar**). • **TaskCalendar**— free slack is calculated using the task calendar (**TaskType.Calendar**). **Default value**: TaskCalendar	String (Enum)	

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
Penalty Accumulation Rule	In cases where there are multiple effective **PenaltyScheduleByInterval** or P**enaltyScheduleByDate** records found within the penalty window, this setting defines how the penalty costs from those effective records are apportioned to the applicable project or task. Valid values are: • **Cumulative**—penalty costs are calculated using the OneTime and PerInterval costs defined on each effective record found within the penalty window (based on effective date/interval). This setting allows for PerInterval costs that change over time along with multiple OneTime charges. • **LastInterval**—penalty costs are calculated using the OneTime and PerInterval costs defined on only the last effective record found within the penalty schedule (based on effective date/interval). This setting uses a constant PerInterval cost along with a single OneTime charge. **Default value:** LastInterval For more information about how this field is used, see "Penalties" on page 2191.	String (Enum)	
PenaltyCalendar	The calendar used when calculating penalties applied to a project. For example, should penalty costs be incurred on a daily or weekly basis. **Referenced table:** Calendar	Reference	
PenaltyRule	Specifies the schedule, if any, to use for calculating project penalty costs. This defines the one-time and per interval penalty costs that are applied between the project penalty date and finish date. Valid values are: • **Date**—penalty values are applied by dates defined in the **PenaltyScheduleByDate** table. The entries that apply to a given project are set through its **PenaltySchedule** reference. • **Interval**—penalty values are applied by calendar intervals defined in the **ProjectScheduleByInterval** table. The entries that apply to a given project are set through its **PenaltySchedule** reference. • **None**—penalties are not are calculated for projects of this type. **Default value:** None For more information about how this rule is used, see "Penalties" on page 2191.	String (Enum)	

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
Percent CompleteRule	For all fixed duration tasks in a project, this setting defines how the percentage complete on the task (**CalcPercentComplete**) is calculated. Valid values are: • **CalcDuration**—percentage complete on each fixed duration task is based on the amount of time the task is calculated to take as follows: *ActualDuration/CalcDuration* • **OriginalDuration**—percentage complete on each fixed duration task is based on the input value that originally specified the amount of time the task was expected to take as follows: *(Duration - CalcRemainingDuration)/Duration* **Default value**: CalcDuration **Note:** This field applies only when **Task.DurationType** is set to "FixedDuration". For all other **DurationType** settings, the task's percentage complete is always calculated based on **CalcDuration**.	String (Enum)	
PlanningRule	Indicates whether the project's current working date, as defined by the **Project.RunDate** field, should be considered when scheduling project tasks. For more information about project run dates, see "Project run dates" on page 2169. Valid values are: • **DateDriven**—the current working date is considered when scheduling tasks. Tasks that have not yet begun are never scheduled to start any earlier than **Project.RunDate** (typically, this setting is used to ensure tasks are not scheduled in the past). • **Standard**—the current working date is not considered when scheduling tasks. Tasks that have not yet started might be scheduled in advance of **Project.RunDate**. **Default value**: Standard **Note:** When this field is set to "DateDriven", any "FixedDuration" tasks in the project consider the task's **ActualStartDate**, **ActualFinishDate**, and/or **ExpectedFinishDate** values, along with the **RunDate** when calculating task durations. This can result in a **CalcDuration** result that differs from the input **Duration** value on the task.	String (Enum)	

Table 7.45: ProjectType (ProjMgmt) fields

Field name	Description	Data type	Key
ScheduleRule	Controls whether projects are scheduled backward from a given finish date, or scheduled forward from a given start date. Valid values are: • **FromFinish**—project dates are calculated backward from the project finish date. This option fixes the finish date and might be used to show late you can start a project. • **FromStart**—project dates are calculated forward from the project start date. This option fixes the start date and might be used to show when the project can finish. **Default value:** FromStart	String (Enum)	
TotalSlackRule	Specifies how total slack (**Task.TotalSlack**) is calculated for tasks belonging to projects of this type. Valid values are: • **Minimum**— Total slack is calculated as *min(LatestStartDate - EarliestStartDate, LatestFinishDate - EarliestFinishDate)* • **StartSlack**—Total slack is calculated as *LatestStartDate - Task.EarliestStartDate* • **FinishSlack**—Total slack is calculated as *LatestFinishDate - Task.EarliestFinishDate* **Default value:** Minimum	String (Enum)	
Value	A unique string identifier for this type of project. This value is referenced from the **Project.Type** field.	String	Yes
Projects	Set of **Project** records that reference this ProjectType value.	Set	

RangeOfCoverage table

This table is used to store range of coverage profiles. These profiles are used in safety stock calculations for parts where safety stock is meant to satisfy a number of days of the part's average daily demand (typically, within definable zones).

Profiles in this table are only applicable to parts whose effective **SafetyStockQuantityRule** setting, from either of the **SafetyStockPolicy** or **PartType** tables, is one of the following:

- **RangeOfCoverage**—safety stock is calculated to satisfy a number of days of the average daily demand within three distinct zones. Profiles in this table are used to determine how demands are collected for use in determining the average daily demand, the dates on which safety stock levels can be calculated, as well as the duration of the three coverage zones and the number of intervals of the average daily demand that safety stock should aim to satisfy in each zone.

  With this setting, all fields other than AverageRequirementIntervalLeadTimeCount are potentially applicable.
- **RangeOfCoverageLeadTime**—safety stock is calculated to satisfy the average daily demand over the part's lead time. Profiles in this table are used to determine how demands are collected for use

in determining the average daily demand as well as the dates on which safety stock levels can be calculated.

With this setting, all fields other than FirstZoneTarget, SecondZoneIntervalCount, SecondZoneTarget, and ThirdZoneTarget are potentially applicable.

- **TimePhasedRangeOfCoverage**— safety stock is calculated to satisfy a number of days of the average daily demand within any number of date-based zones. Profiles in this table are used to determine how demands are collected for use in determining the average daily demand as well as the dates on which safety stock levels can be calculated. Records in the **TimePhasedSafety** table are used to determine the start of each coverage zone as well as the number of days of the average daily requirement that the calculated safety stock levels should aim to satisfy within each of those zones.

  With this setting, all fields other than AverageRequirementIntervalLeadTimeCount, FirstZoneIntervalCount, FirstZoneTarget, SecondZoneIntercalCount, SecondZoneTarget, and ThirdZoneTarget are potentially applicable.

Notes:

- All RangeOfCoverage settings are also subject to optional upper and lower threshold limits that are defined in this table and used to determine whether a calculated change in the safety stock level is significant enough to warrant replacing the existing level.
- Certain range of coverage parameters can be overridden on a part by part basis using the **RangeOfCoveragePartOverride** table. For example, the fields used to set the duration over which demands are collected for use in calculating the average daily demand can be overridden. For more information, see "RangeOfCoveragePartOverride table" on page 509.
- **TimePhasedRangeOfCoverage** does not calculate dynamic safety stock for the last period of demand if there is no future demand. For example, it is expected for RapidResponse to recalculate the safety stock level in April 2016 and drop it to zero. Inserting a new TimePhasedSafety record for 08-01-0216 doesn't impact the calculation. Changing the date on the new record to 04-01-2016 when the last demand is due, drops the SS level to zero, as expected.
- For more information about range of coverage calculations, see "RangeOfCoverage calculations" on page 1409.

Table 7.46: RangeOfCoverage (Mfg) fields

Field name	Description	Data Type	Key
Average Requirement Interval	The name of the calendar in RapidResponse whose dates are used to set the intervals over which average daily demands are calculated. **Referenced table:** Calendar	Reference	
Average Requirement IntervalCount	The number of calendar units to consider demand from when calculating average daily demands. This value is used for each range of coverage zone.	Integer	

Table 7.46: RangeOfCoverage (Mfg) fields

Field name	Description	Data Type	Key
Average Requirement IntervalLead TimeCount	An optional parameter. It is applicable only when the applicable **SafetyStockQuantityRule** setting is "RangeOfCoverageLeadTime". It represents a factor of lead time that can be used to extend the period over which demands are accumulated when calculating the average daily demand. The value provided in this field is then multiplied by the longest lead time for the part expressed as a number of **AverageRequirementIntervals** (rounded up, if necessary). The result of that calculation is then added to the value in **AverageRequirementIntervalCount** to determine the total number of calendar units over which demands should be accumulated. **Default value**: 0	Integer	
Average Requirement IntervalOffset	A parameter allowing the average daily demand calculations to be offset a specified number of **AverageRequirementInterval** units from the start of the **RangeOfCoverageInterval**. For example, if **AverageRequirementInterval** is set to "Weekly" and this value is set to "3", then the average daily demand requirements will be calculated starting 3 weeks after the start of the **RangeOfCoverageInterval**. Default value: 0	Integer	
Average Requirement IntervalDuration Type	Indicates how days are interpreted when calculating the number of days in the average daily requirement window. Valid values are: • **TimeUnits**—Use the value in **Part.PlanningCalendars.TimeUnits** to calculate the number of days in the window. • **Everyday**—Use the standard everyday (Gregorian) calendar to calculate the number of days in the window. • **StandardDays**—Uses the product of **AverageRequirementIntervalCount** * **StandardDaysPerInterval** as the number of days in the window. **Default value**: TimeUnits	String (Enum)	
ControlSet	The control set associated with this RangeOfCoverage record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this range of coverage profile.	String	

Table 7.46: RangeOfCoverage (Mfg) fields

Field name	Description	Data Type	Key
FirstZoneInterval Count	The number of units in the calendar defined by **RangeOfCoverageInterval** that comprise the first range of coverage zone. If set to -1, this indicates that the first zone should extend to the end of the planning horizon. Typically, this is done when the zone target is set to the part's lead time (that is, where the applicable **SafetyStockQuantityRule** field is set to "RangeOfCoverageLeadTime"). This field is used only when the applicable **SafetyStockQuantityRule** field is set to either "RangeOfCoverage" or "RangeOfCoverageLeadTime'. **Note:** Past due demands are considered in this zone.	Integer	
FirstZoneTarget	The number of days of the average daily requirement to cover in the first zone. This field is used only when the applicable **SafetyStockQuantityRule** field is set to "RangeOfCoverage".	Quantity	
RangeOfCoverage Interval	The name of the calendar containing the dates on which safety stock levels can be calculated (applies to all range of coverage settings). For example, safety stock might be re-calculated weekly or monthly. This calendar also defines the dates used in calculating range of coverage zones when **SafetyStockQuantityRule** is set to "RangeOfCoverage". **Referenced table**: Calendar	Reference	
SecondZone IntervalCount	The number of units in the calendar defined by **RangeOfCoverageInterval** that comprise the second range of coverage zone. This field is used only when the applicable **SafetyStockQuantityRule** field is set to "RangeOfCoverage".	Integer	
SecondZoneTarget	The number of days of the average daily requirement to cover in the second zone. This field is used only when the applicable **SafetyStockQuantityRule** field is set to "RangeOfCoverage".	Quantity	
StandardDaysPer Interval	Used in calculating number of days in the average requirement window if **AverageRequirementIntervalDurationType** is set to "StandardDays". Otherwise, not used.	Integer	

Table 7.46: RangeOfCoverage (Mfg) fields

Field name	Description	Data Type	Key
ThirdZoneTarget	The number of days of the average daily requirement to cover in the third zone. This field is used only when the applicable **SafetyStockQuantityRule** field is set to "RangeOfCoverage".	Quantity	
ThresholdFactor Lower	A factor used along with the **Part.RangeOfCoverageBuffer field** to set the lower threshold limit for a given range of coverage interval as follows: • **LowerLimit** = Calculated SafetyStockLevel + (Part.RangeOfCoverageBuffer * ThresholdFactorLower) When the safety stock level is calculated for a range of coverage interval, this limit and an upper limit (using the **ThresholdFactorUpper** value) are also calculated. If the existing safety stock level (from the previous range of coverage interval) falls within these limits, then the new calculated level is discarded and the existing safety stock level remains effective. This is intended to avoid making minor changes to the safety stock level which might have only insignificant benefit. If, however, the existing safety stock level falls outside these limits, then the new calculated safety stock level becomes effective. **Default value:** 0	Quantity	
ThresholdFactor Upper	A factor used along with the **Part.RangeOfCoverageBuffer** field to set the upper threshold limit for a given range of coverage interval as follows: • **UpperLimit** = Calculated SafetyStockLevel + (Part.RangeOfCoverageBuffer * ThresholdFactorUpper) When the safety stock level is calculated for a given range of coverage interval, this limit and a lower limit (using the **ThresholdFactorLower** value) are also calculated. If the existing safety stock level (from the previous range of coverage interval) falls within these limits, then the new calculated level is discarded and the existing safety stock level remains effective. This is intended to avoid making minor changes to the safety stock level which might have only insignificant benefit. If, however, the existing safety stock level falls outside these limits, then the newly calculated safety stock level becomes effective. **Default value:** 0	Quantity	
Value	A unique identifier for this range of coverage profile. This field is referenced by the **SafetyStockPolicy** and **PlanningCalendars** table to determine the range of coverage profile applicable to a given part.	String	Yes

Table 7.46: RangeOfCoverage (Mfg) fields

Field name	Description	Data Type	Key
PlanningCalendars	Set of **PlanningCalendars** records that reference this RangeOfCoverage value.	Set	
SafetyStockPolicies	Set of **SafetyStockPolicy** records that reference this RangeOfCoverage value.	Set	

ResourceType table

The **ResourceType** table contains configurable settings that define processing and calculations at the resource level. For example, this table set the resource's working calendar, and specifies how standard and overtime costs are reported on resource loads.

This table's records can be edited using the **Resource Type** worksheet in the **Control Tables for Project Management** workbook included with RapidResponse.

Table 7.47: ResourceType (ProjMgmt) fields

Field name	Description	Data type	Key
Calendar	The calendar used in assigning resources and calculating loads against resources of this type. For example, this might typically be set to a workday calendar. **Referenced table:** Calendar	Reference	
ControlSet	The control set associated with this **ResourceType** record. **Referenced table:** ControlSet	Reference	Yes
CostAccrualRule	Specifies when standard and overtime costs are reported against resource loads generated from a given task. This impacts how values are reported in the **ResourceLoad.StandardCost** and **ResourceLoad.OvertimeCost** fields. Valid values are: • **Finish**—standard and overtime costs are reported on the last dates the resource is assigned to a given task. • **Prorated**—standard and overtime costs are reported on the actual dates they accrue during the duration of an assigned task. • **Start**—standard and overtime costs are reported against the finish date the resource is assigned to a given task. **Default value:** Prorated	String (Enum)	
ReportLimit	The number of Calendar periods after **Resource.DataDate** for which resource details are reported in the **ResourceDataByDate** tables.	Integer	

Table 7.47: ResourceType (ProjMgmt) fields

Field name	Description	Data type	Key
Value	A unique string identifier for this type of resource. This value is referenced from the **Resource.Type** field.	String	Yes
Resources	Set of **Resource** records that reference this ResourceType value.	Set	

SafetyStockItemType table

The **SafetyStockItemType** table contains control settings that define how safety stock items of a particular type are processed when generating safety stock recommendations. For example, fields in this table control item details such as whether the item is configured for time-phased safety stock calculations and how standard deviation of historical demands should be calculated.

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
AsOfDate Calendar	Calendar used when collecting historical forecasts for use in calculating forecast error (typically, this might be a monthly calendar). Each series of historical forecast demands identified in the **HistoricalDemandSeries** table contains an **AsOfDate**, and only one series within a given **AsOfDateCalendar** is considered effective. To calculate forecast error for a given "Date", the **HistoricalDemandSeriesDetail** record (if any) that satisfies the following three conditions is used: 1. Series.AsOfDate + 0 'AsOfDateCalendar' = Date - ForecastLag 'AsOfDateCalendar' 2. Has the earliest Series.AsOfDate of all records satisfying the first condition. 3. Series.LastOnAsOfDate = 'Y'. **Referenced table:** Calendar (Nullable) **Note:** This field is only applicable on items where **StandardDeviationDemandRule** is set to "ForecastError".	Reference	
Average Demand Rule	Indicates how the average demand value used in calculating the safety stock level is determined. Valid values are: • **Mean**—the mean (average value) is used. This is the default and typically appropriate for demand that is normally distributed, however other options are available for cases where the demand is not normally distributed. • **Median**—the median (middle value) is used. • **Mode**—the mode (most frequent value) is used. If there are multiple values found for the mode, the smallest of these is used. **Default value:** Mean **Note:** This setting only applies to non time-phased calculations where **StandardDeviationDemandRule** is set to "StandardDeviation". Also, this setting only affects the average demand value used when calculating safety stock. Average historical and average future demand calculations always return a mean.	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
BoundsRule	Indicates how minimum and maximum safety stock bounds, specified in the **SafetyStockTimePhased Bounds** table, are applied to this type of safety stock item or MEIO family. Valid values are: • **Ignore**—All maximum and minimum bounds specified in the **SafetyStockTimePhased Bounds** table are ignored. • **Quantity**—**MinimumQuantity** and **MaximumQuantity** values are used, while minimum and maximum days of supply are ignored. • **DaysOfSupply**— **MinimumDaysOfSupply** and **MaximumDaysOfSupply** values are used, while maximum and minimum quantities are ignored. **Default:** "Ignore" **Note:** This field was added in Version 2016.2.	String (Enum)	
ControlSet	The control set associated with this **SafetyStockItemType** record. **Referenced table:** ControlSet	Reference	Yes
CycleCalendar	The outer calendar used when an item is configured for time-phased safety stock calculations. The referenced calendar should typically define a full cycle for seasonal/trending data; for example, this might typically be a Yearly calendar. This calendar also applies when the Holt-Winters statistical model is used for estimating standard deviation of demand. **Referenced table**: Calendar (nullable) **Note:** The **CycleCalendar** and **IntervalsCalendar** settings determine the value to use for **SafetyStockItem.HistoricalIntervalCount**. This is to ensure that an adequate amount of historical demand and supply data is available for use in safety stock calculations. See "Define historical collection interval" on page 1956.	Reference	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
DemandOutlier ProcessingRule	Indicates how outliers found in historical data should be handled when processing safety stock items. The **Item.DemandOutlierThreshold** setting defines the number of standard deviations away from the mean an item quantity must be before it is considered an outlier (historical actuals are always first adjusted for any causal factors before checking/processing for outliers). Valid values are: • **Ignore**—historical actual quantities are left as is without any outlier adjustments. • **RemoveExcess**—outliers are removed from historical actuals by adjusting their quantities to the demand outlier threshold for the item. • **SmoothKeepExcess**—outlier quantities are adjusted by using parameters defined on the item to smooth the quantity above the outlier threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains above the outlier threshold, it is left as is. • **SmoothRemoveExcess**—outlier quantities are adjusted by using parameters defined on the item to smooth the quantity above the outlier threshold over a number of intervals backward and then forward. If, after smoothing, any excess remains above the outlier threshold, that excess is reduced to the demand outlier threshold for the item. **Default value:** Ignore **Note:** This field is obsolete (as of Version 2016.2, Service Update 5). To achieve the same functionality, use the **RemoveDemandOutliers** field on the **SafetyStockItem** table.	String (Enum)	
Description	An optional description of this safety stock item type.	String	
ExcludePastDue Demands	Whether demands that are due prior to the period containing the RunDate are excluded from time-phased safety stock calculations. When this value is set to "N," the demands due prior to the interval containing the RunDate are included in time-phased safety stock calculations and they are considered as belonging to the interval containing the RunDate. **Default value**: N **Note:** This field was added in Version 2016.2, Service Update 7.	Boolean	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
ForecastBias AdjustmentRule	Indicates whether an automatic forecast bias adjustment is applied when calculating the standard deviation of forecast error. Forecast bias refers to systematic over or under forecasting. However, the item's specified service level can be used to automatically adjust for bias based on forecast error distribution. Valid values are: • **Ignore**—no adjustment for forecast bias is performed. • **Use**—forecast bias adjustment is performed. The adjusted standard deviation of forecast error is calculated using the following formula: $\frac{(1-\theta(\beta))/\theta(\beta)-1}{\tau\beta.\mathrm{d}} \times \mu$ where • θ is the probability distribution of forecast error points *Forecast / (Forecast + Actual*). • $\theta(\beta)$ is the quantile point corresponding to β in the theta distribution. • μ is the mean forecast error. • $\tau\beta.\mathrm{d}$ is a student-t distribution with cumulative density of β and d degrees of freedom (where d is the number of historical points). **Note:** This field is only applicable on items where **StandardDeviationDemandRule** is set to "ForecastError". **Note:** When forecast error pooling is enabled, the forecast bias adjustment rule is ignored.	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
ForecastError Outlier ProcessingRule	Indicates how outliers are treated in measures of forecast error. Outliers are detected using the **ForecastErrorOutlierThreshold** value specified on the **SafetyStockItem** record (this specifies the number of standard deviations away from the mean that a forecast error point must be in order to be considered an outlier). Applicable on items where **StandardDeviationDemandRule** is set to "ForecastError". Valid values are: • **Ignore**—forecast error outliers are included when calculating the standard deviation of forecast error. • **Remove**—forecast error outliers are excluded when calculating the standard deviation of forecast error. • **RemoveExcess**—forecast error outliers are reduced to the outlier threshold defined as *Mean of ForecastError +- ForecastErrorOutlierThreshold * StandardDeviation of ForecastError* **Note:** This field is obsolete (as of Version 2016.2, Service Update 5). To achieve the same functionality, use the **RemoveDemandOutliers** field on the **SafetyStockItem** table.	String (Enum)	
IntervalsCalendar	Defines the calendar used in standard calculations for the safety stock item. For example, historical demands are bucketed, and average demand calculations are expressed, in terms of this calendar. This field also sets the calendar used for lead time values in cases where the **LeadTimeCalendar** is left Null. **Referenced table:** Calendar (nullable) **Note:** An everyday calendar is assumed if this field is not populated. **Note:** The **IntervalsCalendar** and **CycleCalendar** settings determine the value to use for **SafetyStockItem.HistoricalIntervalCount**. This is to ensure that an adequate amount of historical demand and supply data is available for use in safety stock calculations. See "Define historical collection interval" on page 1956.	Reference	
IntervalsPer Period	The number of **IntervalCalendar** units within a **PeriodCalendar**. When statistical models are used to determine the standard deviation of demand based on forecast error variability, this is used to convert results calculated in terms of the **PeriodCalendar** to units of the **IntervalCalendar**. For example, this might involve converting a monthly forecast error to daily forecast error. Default value: 1	Quantity	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
LeadTime Calendar	Defines the calendar used in lead time calculations for the safety stock item. For example, if using variable lead time in safety stock calculations, then historical lead times are calculated in terms of this calendar (the difference between the historical order date and supply date is expressed in units of this calendar). The input lead time specified on the **SafetyStockItem** record should also be specified in units of this calendar. If a valid calendar reference is not provided, then the **IntervalsCalendar** is used by default. In cases where **UseRollingLeadTimeCalendar** is set to "Y", then **LeadTimeCalendar** will always be ignored even if populated and **IntervalsCalendar** used instead. If history is monthly bucket and lead time is daily, then there are not enough points to meaningfully calculate rolling lead time demand. **Referenced table:** Calendar (Nullable) **Note:** LeadTimeCalendar only applies to SEIP.	Reference	
LeadTimePer Interval	Specifies the number of **LeadTimeCalendar** intervals within an **IntervalsCalendar**. For example, if **LeadTimeCalendar** is "Everyday" and **IntervalsCalendar** is "Week", this might be set to "7". The value in this field is used to ensure the standard deviation of demand parameter used in the calculation of safety stock is expressed in terms of the item's lead time calendar. **Note: SafetyStockItemType.LeadTimePerInterval** should never be less than 1 and should not have larger intervals than the **IntervalsCalendar**. When **SafetyStockItemType.LeadTimePerInterval** is less than 1, the value will automatically be treated as 1; thus resulting in an incorrect calculation of **SafetyStockResult.SafetyStock**. **Note:** For an example of how this field is used, see "Example 3: Fixed lead time (with forecast error)" on page 1972.	Quantity	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
MEIODemand Historicals PropagationRule	Enables forecast error pooling. The value of this field only matters if: • the part on the **SafetyStockItem** is the head of an MEIO Family • **TimePhasedProcessingRule** is either **Ignore**, **DaysOfSupplyBackward** or **DaysOfSupplyForward** • **StandardDeviationDemandRule** is set to **ForecastError** Valid values are: • **Use**—enables forecast error pooling for the entire MEIO family. In this case, forecast error pooling is used on all customer facing stages in this MEIO family, regardless of what was set on the **SafetyStockItemType.StandardDeviationDemandRule** . • **Ignore**—forecast error pooling is disabled for the entire MEIO family. **Default value:** Ignore **Note:** This field was added in Version 2016.2.	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
NonStationary DemandRule	Specifies how the standard deviation of historical demands is calculated for those items where **TimePhasedProcessingRule** is set to "Use" (applies to both single and multi-echelon items). The value calculated based on this setting is then used as a parameter when generating time phased safety stock recommendations. Valid values are: • **Decomposition**—A time series decomposition method is used to calculate standard deviation on historical demands within each period after removing trend from the data. This method is typically appropriate when there is more than two cycles (typically, years) of historical demand data available. • **HoltWinters**—HoltWinters seasonal forecasting method is used to calculate standard deviation of forecast error within each period. This method is typically appropriate when there is more than two cycles (typically, years) of historical data available. • **Manual**—Average and standard deviation of historical demands within each period is set to the values that have been provided in the **TimePhasedDemandParameterSet** table. This method is typically appropriate when there is little historical demand data available (less than a full cycle or year). • **Simple**—Standard or simple calculation of standard deviation is performed on bucketed historical demands within each period. This method is typically appropriate when there is between one and two cycles (typically, years) of historical demand data available. **Default Value**: Simple **Note:** If **TimePhasedProcessingRule** is set to "Ignore" or "DaysOfSupplyBackward" or "DaysOfSupplyForward", then the **StandardDeviationDemandRule** field applies instead.	String (Enum)	
PeriodCalendar	The inner calendar applicable if the item is configured for time-phased safety stock calculations. The referenced calendar should define a period or season within the cycle; for example, if the CycleCalendar is Year, then this might typically reference a Quarter or Month calendar. This calendar also applies when using statistical models (such as, exponential smoothing or Holt-Winters) to base the standard deviation of demand on forecast error variability. It is the calendar by which historical demand data should be bucketed when fitting data to these models (typically, monthly or quarterly calendar might be used). **Referenced table:** Calendar (nullable)	Reference	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
PeriodsPerCycle	The number of **PeriodCalendar** units within a **CycleCalendar**. For example, four quarters might make up a yearly cycle. This field applies to time-phased safety stock calculations, as well as when using the Holt-Winters method to calculate standard deviation of demand based on forecast error variability.	Integer	
ServiceLevelRule	Determines the measure of customer satisfaction that the recommended safety stock level is meant to achieve. This setting controls how the **ServiceLevel** value specified on the **SafetyStockItem** record is interpreted. Service levels are entered as percent values and, together with the setting in this field, determine the corresponding z-value or service factor (used as a multiplier when calculating safety stock levels). Valid values are: • **Cycle**—An event based service level that sets the probability of not stocking out when filling a demand within lead time. This refers to an alpha or type1 service level. • **FillRate**—A quantity based service level that sets the overall percentage of demand which should be satisfied on time. This refers to a beta or type2 service level. If using this option, the **MinimumOrderQuantity** field on the **SafetyStockItem** record should also be set for use in determining the z-value for a given fill rate service level. **Default value**: Cycle **Note:** This field applies only to single-echelon calculations. Multi-echelon safety stock items always use a Cycle fill rate regardless of the setting in this field.	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
Standard Deviation DemandRule	Specifies how the parameter representing standard deviation of historical demands is determined for safety stock items that are not configured for time phased calculations (if **TimePhasedProcessingRule** is set to "Use", then the **NonStationaryDemandRule** field applies instead). By default, the standard deviation is calculated based on historical actuals. However, standard deviation can also be calculated based on historical forecast error (difference between historical forecast and actuals). Additionally, various statistical models can be fitted for use in calculating and returning forecast error instead. If using these statistical models, a Period calendar might typically need to be specified. Valid values are: • **CrostonsMethod**—Croston's method is used to return forecast error variability (appropriate for lumpy or intermittent data). • **DoubleExponentialSmoothing**—double exponential smoothing is used to return forecast error variability (appropriate for stable data with a trend). • **ExponentialSmoothing**—exponential smoothing is used to return forecast error variability (appropriate for stable data with no trend). • **ForecastError**—standard deviation from forecast error is returned. Forecast error is calculated by comparing historical forecasts (from the HistoricalDemandSeriesDetail table) against historical actuals (from the HistoricalDemandActual table). The HistoricalDemandCategory and HistoricalForecastCategory reference fields on the SafetyStockItem record indicate the historical actual and historical forecast categories to use respectively. • **HoltWintersMethod**—Holt-Winters method is used to return forecast error variability (appropriate for data that is seasonal in nature). • **LinearRegression**—a simple linear regression is used to return forecast error variability. • **Manual**—standard deviation of historical demand and average historical demand are each set to values provided on the **SafetyStockItem** record (in the **StandardDeviationDemand** and **AverageDemand** fields respectively). This method is intended for cases where there is little or no historical demand data available, but the related demand parameters are known or estimated through some other means. • **StandardDeviation**—standard deviation from demands in the **HistoricalDemandActual** table is returned. Historical demands used for the item are those belonging in the category referenced by	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
	HistoricalDemandCategory on the **SafetyStockItem** record. **Default value**: StandardDeviation		
Supply VariabilityRule	Indicates if variability of historical supply lead times should be considered when calculating recommendations for safety stock levels and reorder point. If variability of historical supply lead time is to be considered, the required parameters can be either manually provided or calculated based on historical supply data. Note that variability of supply is only applicable when calculating stationary (non time-phased) safety stock for single-echelon items. Valid values are: • **Ignore**—Variability of historical supply lead times is ignored. A constant lead time, as provided on the safety stock item, is assumed. No records are reported for the item in the **SafetyStockHistoricalSupply** table. • **Manual**—Variability of historical supply lead time is considered in safety stock and reorder point calculations. Standard deviation of lead time and average lead time are each set to values provided on the **SafetyStockItem** record (in the **StandardDeviationLeadTime** and **LeadTime** fields respectively). Records are not generated for the item in the **SafetyStockHistoricalSupply** table. • **Use**—Variability of historical supply lead time is considered in safety stock and reorder point calculations. Historical lead times for supplies are calculated using the **Date** and **OrderDate** fields on the **HistoricalSupplyActual** table (both of these fields must be populated). Records are generated for the item in the **SafetyStockHistoricalSupply** table for each supply within the historical collection interval. **Default value:** Ignore	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
TimePhased ProcessingRule	Specifies whether the item is configured for time-phased safety stock calculations. Time-phased safety stock can be calculated for both single and multi-echelon items. The setting in this field also determines how the standard deviation of demand parameter is calculated. When set to "Use", the standard deviation of demand is calculated based on the setting in the **NonStationaryDemandRule** field. Otherwise, the standard deviation of demand is calculated based on the setting in the **StandardDeviationDemandRule** field. Valid values: • **DaysOfSupplyBackward**—A single safety stock value is initially calculated and then expressed as a number of intervals of the part's average historical demand. Time-phased safety stock values are then generated based on the total amount of future and/or historical demand found when looking back that number of intervals from a given point. • **DaysOfSupplyForward**— A single safety stock value is initially calculated and then expressed as a number of intervals of the part's average historical demand. Time-phased safety stock values are then generated based on the total amount of future demand found when looking ahead that number of intervals from a given point. • **Ignore**—Time-phased safety stock calculations are disabled for this item. A single safety stock level is calculated for the item and reported on RunDate. • **Use**—Time-phased safety stock calculations are enabled for this item with safety stock reported on RunDate as well as each subsequent period in which a new value applies. Appropriate cycle and period calendars should be defined when using this setting, with the standard deviation of demand and average historical demand parameters calculated for each period. Note also that fixed lead time is always assumed (variable lead time is not supported in time-phased calculations). **Default value:** Ignore. **Note:** For an example of how time-phased safety stock is calculated, see "Time-phased safety stock" on page 1977. For an example of how safety stock is calculated using days of supply, see "Days of supply and safety stock" on page 1981.	String (Enum)	

Table 7.48: SafetyStockItemType (Mfg) fields

Field name	Description	Data type	Key
UseRollingLead TimeDemand	Specifies which of rolling lead time demands or bucketed demand quantities are used to determine the average demand and standard deviation for use in calculating safety stock. Valid values are: • **N**—average demand and standard deviation calculations use bucketed demand quantities based on the safety stock item calendar (for example, daily buckets). Based on the setting in the **StandardDeviationDemandRule** field, either the standard deviation or standard error calculated for these bucketed demands is used and multiplied by the square root of lead time. • **Y**—average demand and standard deviation calculations use rolling lead time demand quantities. Based on the setting in the **StandardDeviationDemandRule** field, either the standard deviation or standard error calculated for these rolling lead-time demands is used. **Default value:** N	Boolean	
Value	A unique string identifier for this type of safety stock item. This value is referenced from the **SafetyStockItem.Type** field.	String	Yes

SafetyStockPolicy table

The **SafetyStockPolicy** table contains control settings specifying how netting calculations determine or maintain safety stock levels in RapidResponse.

This table is referenced from the **Part.SafetyStockPolicy** field, and is used by all parts that have that reference populated. Otherwise, if a part's **SafetyStockPolicy** reference is Null, then identical control settings on the **PartType** and **PlanningCalendars** tables are used instead. However, use of the **SafetyStockPolicy** table for managing safety stock controls is recommended as it allows for easier modeling of safety stock settings without the need to maintain additional **PartType** records (which hold many other planning policies). For example, a number of different combinations of safety stock settings might be required in what-if scenarios used to analyze recommendations generated by inventory management solutions in RapidResponse.

If upgrading from a version of RapidResponse earlier than 2014.4, a workbook is available that can create and then reference values in this table based on a part's existing **PartType** and **PlanningCalendars** settings. For more information, see "Moving safety stock settings to the SafetyStockPolicy table" on page 2409.

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this **SafetyStockPolicy** record. **Referenced table:** ControlSet	Reference	Yes
CTPHoldRule	Indicates whether priority on safety stock is considered in capable-to-promise logic. For example, priority can be used to ensure minimum safety stock requirements are maintained. Thus, when all demands cannot be satisfied on time, allotments of supply to demand can be held to ensure the higher priority safety stock level is maintained (or minimally violated if is not possible to always maintain it). Valid values are • **Ignore**— Priority on safety stock is not considered in capable-to-promise logic. Actual demands are always preferred to safety stock (regardless of priority), and supply allotments will never be held to ensure safety stock levels are respected. This setting corresponds to how safety stock was always treated in capable-to-promise logic prior to Version 2016.2, Service Update 18. • **Use**— Priority in capable-to-promise logic is considered in capable-to-promise logic. Maintaining safety stock is preferred to satisfying actual demands of a lower priority level, while actual demands of the same or higher priority are preferred to safety stock. As necessary, allotments of supply to (lower priority) actual demands can be held to ensure the safety stock level is maintained. Details of these held allotments are then reported in the **HoldStartDate** and **HoldEndDate** fields on the **SupplyDemand** and **CTPActivity** tables. **Default value:** Ignore **NOTE**: This field was added in Version 2016.2, Service Update 18. For more information about how priorities on safety stock are used in capable-to-promise logic, see "Priority and safety stock holds in CTP" on page 1780.	String (Enum	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
DemandTransformationRule	Specifies whether demand transformations are applied when safety stock requirements are present. This setting determines whether planned supplies take their priorities from the driver requirement (including safety stock) that led to their creation, or from the real demand that the supply is assumed to satisfy based on demand due date sequence. Valid values are: • **FirstInFirstOut**—Demand transformations are applied, and supplies take their priority from the real demand they are assumed to satisfy based on due date sequence. Thus supply created to satisfy safety stock requirements is given the priority of the next real demand it would ultimately satisfy. Such supplies might also be split if they are seen to satisfy multiple subsequent real demands of different priorities. Similarly, because real demands can be covered by supply created to satisfy safety stock, the supplies created to satisfy those real demands will then take their priority from subsequent/later demands. This setting is intended to ensure that planned supplies throughout the structure reflect the priority of the demands they ultimately satisfy, and prevent situations were higher priority demands might end up late because they are satisfied by released safety stock supply. When there are no real demands, the part's default priority is used. • **Ignore**—Demand transformation are not applied, and supplies take their priority from the driver demand or safety stock requirement that led to their creation. Thus, supply generated to satisfy safety stock requirements is always assigned the part's default priority (typically, this will be a lower priority). In cases where the safety stock level is reduced, this lower priority planned supply can then be consumed by real demands. Note that if these demands are of a higher priority, their priority might not be fully respected and/or propagated down to component supplies in the structure which can result in higher priority demands being satisfied late. This setting corresponds to how safety stock logic was always applied prior to Service Update 1905. **Default value:** Ignore **Note:** Supplies for parts that have a **CommitLevel** setting of "Medium" or "Low" are always planned using the default priority regardless of the setting in this field. **Note**: This field was added in Service Update 1905. For more information about the impact of this setting, see "Safety stock and priority adjustments" on page 1773.	String (Enum)	
Description	A description of this safety stock policy.	String	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
ExpiryRule	Specifies when expiring supply allocated to safety stock should be replenished. Valid values are: • **AtExpiry**: A new order is planned for when the safety stock supply will expire (at expiry date). • **AtPartMSL**: A new order is planned for when the safety stock supply will no longer be able to to satisfy demands in respect of the part's shelf life requirements (at expiry date less minimum shelf life). **Default value:** AtExpiry **Note:** This field was added in Version 2016.2	String (Enum)	
OrderPointDate Rule	Indicates the date on which scheduled supply (scheduled receipts, inventory transfers, and supply allocations) for order point parts should be included when calculating whether the threshold for creating new planned orders has been crossed. Valid values are: • **Due**—Scheduled supply is considered as of its ReschedDate (calculated due date). As a result, supply that is already started, but not yet due, is ignored when determining if the order point threshold has been reached. Planned orders might therefore be created to cover this timing gap. • **Start**— Scheduled supply is considered as of its CalcStartDate (calculated start date). As a result, the order point threshold will not be crossed if a sufficient quantity is available in a started, but not yet due, scheduled supply. This is the recommended value. **Default value**: Due	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
OrderPoint Increase	Indicates if a temporary increase in a part's order point threshold should be considered to guard against near-term spikes in demand that might push inventory below zero before more supply can be planned. This setting is only applicable to order point parts (that is, parts where **SafetyStockRule** is set to either "OrderPointAverage", "OrderPointOver", or "OrderPointUnder"). Additionally, this setting relies on the part's **PlanningCalendars.OrderPointIncrease** setting which specifies a calendar whose intervals are used, starting from the RunDate, to look for demand within the part's lead time. Valid values are: • **Calculate**— an increase in the order point is considered for each **PlanningCalendars.OrderPointIncrease** interval where there is demand within lead time. If the demand within lead time (plus any additional demand within the OrderPointIncrease interval) exceeds the effective order point, then the amount it exceeds it by is temporarily added to the order point. This increase is effective until the next OrderPointIncrease interval. • **Ignore**—order point threshold is never inflated to cover spikes in demand that will consume supply before it can be replenished. • **Sum**— an increase in the order point is considered for each **PlanningCalendars.OrderPointIncrease** interval where there is either demand within lead time or a value is provided in the **TimePhasedSafety** table. If the demand within lead time (plus any additional demand within the OrderPointIncrease interval) exceeds the effective order point, then the amount it exceeds it by is temporarily added to the order point. As well, if an effective record is found in the **TimePhased Safety** table, then its quantity is also added to the effective order point (if multiple **TimePhasedSafety** entries are found in the interval then the one with the largest **Quantity** is used). The total increase is effective until the next OrderPointIncrease interval. Note that if **SafetyStockQuantity** is "TimePhasedQuantity" or "TimePhasedQuantityToLatest", then this option and **Calculate** are the same. • **SumOffset**— identical to **Sum**, except that increases are never performed in the first interval (containing the SafetyStockDate). **Default value:** Ignore	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
PercentSafety Interval	Identifies the calendar used in calculating safety stock when **SafetyStockQuantityRule** is either "PercentOfDemand" or "FractionOfDemand". That is, the bucketing upon which the calculation is done is based on a period which is "N" intervals in length. Where intervals is defined by the referenced calendar value and "N" is set to **Part.PercentSafetyIntervalCount**. For example, to have a safety value that was based on four weeks of demand, this field could be set reference a "Weekly" calendar and **Part.PercentSafetyIntervalCount** set to "4". **Referenced table:** Calendar (Nullable) **Note:** If this reference is Null, then the part's **PlanningCalendars**.**PercentSafetyInterval** reference is used instead.	Reference	
RangeOf Coverage	A reference to range of coverage profile for parts using this policy. This profile is relevant only when **SafetyStockQuantityRule** is set to 'RangeOfCoverage'. **Referenced table:** RangeOfCoverage **Note:** If this reference is Null, then the part's **PlanningCalendars.RangeOfCoverage** reference is used instead.	Reference	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
SafetyStockDate Rule	Specifies the date on which to plan for safety stock for parts using this policy. Valid values are: • **FirstDemand**—maintain safety stock level as of the date of the earliest demand record. The **PlannedOrder.DueDate** will be the date of the first demand (**Part.SafetyLeadTime** + **PartSource.SafetyLeadTime**). • **FirstDemandOrLeadTime**—if the part has demands, then a planned order is generated to cover safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **RunDate + LeadTime** (if **OrderPolicy.OrderGenerationRule** is 'AnyTime' or 'AfterRunDate'), or at the later of **RunDat**e + **LeadTime** or **PTFDate** (if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'). • **FirstDemandOrRunDate**—if the part has demands, then a planned order is generated to cover safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **RunDate** (if **OrderPolicy.OrderGenerationRule** is 'AnyTime' or 'AfterRunDate'), or at **PTFDate** if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'). • **FirstDemandOrPast**—if the part has demands, then generate a planned order to maintain safety stock at the date of the first demand. If the part has no demands, then a planned order is generated to cover safety stock at either **Past** if **OrderPolicy.GenerationRule** is 'AnyTime', or at **RunDate** if **OrderPolicy.GenerationRule** is 'AfterRunDate', or at **PTFDate** if **OrderPolicy.OrderGenerationRule** is 'AfterPTF'. • **LeadTime**—maintain safety stock level after RunDate (**Part.PlanningCalendars.RunDate.Value**) + **PartSource.FixedLeadTime** + **Part.LeadTimeAdjust** + **PartSource.Source.TransitTime** + (**PartSource.VariableLeadTime** * **Part.SafetyStockQty**). Additionally, if either or both of **Part.SafetyLeadTime** or **PartSource.SafetyLeadTime** are greater than zero (0) then they are added to the above in determining **PlannedOrder.DemandDate**. This is meant to achieve a realistic **PlannedOrder.StartDate**. However, lot sizing or other demands might make the start date earlier in the presence of **PartSource.VarLeadTime**. Additionally, **OnHand** records for the part may make the actual **PlannedOrder.Quantity** less and thus the **PlannedOrder.StartDate** later. • **LeadTimeReportUnplanned**—similar to the "LeadTime" option in that safety stock is only maintained/planned after	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
	RunDate plus applicable lead times. However, with this setting safety stock levels are still calculated, for reporting purposes only, within lead time. These unplanned safety stock levels are reported in the **Activity** table with a **Type** value of "UnplannedSafety". • **Past**—safety stock can be maintained at any time even if it means placing orders in the past. • **Rundate**—safety stock can be maintained as of RunDate. **Default value:** LeadTime **Note:** When using the "LeadTime" or "LeadTimeReportUnplanned' settings, the lead time from the part's primary part source is used in determining the date from when safety stock can be planned. For example, if a secondary part source has shorter lead time and is used to satisfy a safety stock demand, it still cannot plan it any earlier than **RunDate** plus lead time from the primary part source. **Note:** Ordering policies or planning time fence may override any of the values described above. **Note:** This field was modified in Version 2016.2, Service Update 8.		
SafetyDays Calendar	When using a **SafetyStockQuantityRule** setting of either "TimePhasedSafetyDaysMaximum" or "TimePhasedSafetyDaysMaximum", this specifies the calendar whose intervals are used in collecting demands for use in calculating the safety stock level (that is, the calendar in which the value in **TimePhasedSafety.Days** is expressed). If this reference is left Null, then an Everyday calendar is assumed. Referenced table: Calendar (Nullable) **Note:** This field was added in Version 2016.2.	Reference	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
SafetyDays Dependent DemandUsage	When using a **SafetyStockQuantityRule** setting of either "TimePhasedDaysMaximum" or "TimePhasedDaysTotal", this field specifies the type of dependent demands that are collected and summed for use in calculating safety stock levels. This setting does not affect independent demands. Valid values are: • **All**—All types of dependent demands are used in the safety stock calculation. • **MakeOnly**—Dependent demands from "make" sources are used in the safety stock calculation. • **None**—Dependent demands are excluded from use in the safety stock calculation. • **TransferOnly**—Dependent demands from "transfer" sources are in the safety stock calculation. **Default value**: All **Note:** This field was added in Version 2016.2. **Note:** In addition to this setting, specific demand types can be included or excluded from use in safety stock calculations based on the **DemandType.SafetyStockDemandUsage** setting. That is, only dependent demands that reference a **SafetyStockDemandUsage** setting of "Use" are eligible for use in calculating dynamic safety stock levels (regardless of the setting in this field).	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
SafetyStock QuantityRule	Specifies how the safety stock level is determined for parts using this policy. Valid values are: • **FixedQuantity**—safety stock is set to the fixed input value provided in **Part.SafetyStockQty**. • **FractionOfDemand**—safety stock levels are dynamically calculated based using the value in Part.PercentSafetyPercent which is assumed to represent a fraction (for example, .1 means 10%) of total demand per defined interval. Demand intervals are calculated (beginning from the maximum of the **Part.PlanningCalendar.RunDate** or safety stock date) using the value specified in **PercentSafety Interval**. The number of intervals to use is calculated using the value in **Part.PercentSafetyIntervalCount**. All demands during this interval are summed and the **Part.PercentSafetyPercent** is applied against the total demand quantity resulting in a dynamic safety stock quantity for that interval. • **PercentOfDemand**—safety stock levels are calculated the same as "FractionOfDemand", except that the **Part.PercentSafetyPercent** value is interpreted as a percentage (for example, 10 means 10%). • **RangeOfCoverage**—safety stock levels are calculated to satisfy a specified number of days of a part's average daily requirements, and can be configured for three distinct date zones. All parameters used to calculate these safety stock levels are configured through range of coverage profiles in the **RangeOfCoverage** table. If this option is selected for a part, the part's range of coverage profile is taken from the value referenced in the **RangeOfCoverage** field. For more information, see "RangeOfCoverage table" on page 855. • **RangeOfCoverageLeadTime**—safety stock levels are calculated to satisfy the part's average daily requirement over its lead time plus an optional buffer quantity. Most parameters used to calculate these safety stock levels are configured through range of coverage profiles in the **RangeOfCoverage** table. However, with this setting only the first zone is active and the average daily requirement is multiplied by lead time rather than the FirstZoneTarget value. • **TimePhasedQuantity**—safety stock levels change over time. The **TimePhasedSafety** table must be populated with the following fields: Part, Site, Date (earliest date this safety stock quantity is applicable), and Quantity. For example, a Quantity of 5 is effective beginning 01-01-15 and a Quantity of 10 is effective beginning 06-01-15. This means that the safety stock of 5 is no longer effective as of 05-31-15. For more information, see	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
	"TimePhasedSafety table" on page 655. This setting is identical to "TimePhasedQuantityToLatest" except in cases where the part's safety stock level decreases after the last demand. • **TimePhasedQuantityToLatest**—This setting is identical to "TimePhasedQuantity" except in cases where the safety stock level decreases after the last demand for the part. If the safety stock level decreases after the last demand for the part, then the quantity of the last planned order will be reduced to ensure the resulting inventory level meets the latest value defined in the **TimePhasedSafety** table. If using lot sizing, the **OrderPolicy.LotSizeLastOrder** field should be set to "N" to ensure this behavior. Otherwise, if **LotSizeLastOrder** is set to "Y", then the last planned order will only get as close as it can to the latest defined safety stock level while still respecting all lot-sizing rules. • **TimePhasedRangeOfCoverage**—safety stock levels are calculated to satisfy a specified number of days of a part's average daily demand, and are configurable for any number of zones. When using this setting, records in the **TimePhasedSafety** table are used to define the start of each range of coverage zone as well as the number of days of the part's average daily requirement that safety stock should cover in each of those zones. Additionally, parameters from the **RangeOfCoverage** table are used to specify how demands are collected for use in calculating the average daily requirement, as well to set the calendar whose intervals can be used to generate new safety stock levels. • **TimePhasedSafetyDaysMaximum**—safety stock levels are calculated daily. Based on effective values in the **TimePhasedSafety** table, safety stock is set to the maximum of demand collected over a specified number of forward looking days (**Days**) or a given quantity (**Quantity**). For more information, see "Time-phased safety days" on page 1407. • **TimePhasedSafetyDaysTotal**—safety stock levels are calculated daily. Based on effective values in the **TimePhasedSafety** table, safety stock is set to the sum of demand collected over some number of forward looking days (**Days**) and a given quantity (**Quantity**). For more information, see "Time-phased safety days" on page 1407. **Default value:** FixedQuantity **Note:** This field was modified in Version 2016.2. **Note: TimePhasedRangeOfCoverage** does not calculate dynamic safety stock for the last period of demand if there is no future demand.		

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
	Note: For all dynamic rules, (that is, those other than **FixedQuantity** and the **TimePhasedQuantity*** settings, specific types of demands can be included or excluded from use in determining calculated safety stock levels based on the **DemandType.SafetyStockDemandUsage** setting.		

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
SafetyStockRule	Specifies how safety stock levels are maintained for parts using this policy. In other words, once the safety stock levels are known, under what conditions will orders to replenish safety stock be generated. Valid values are: • **Always**—always maintain the safety stock level as calculated based on the setting in **SafetyStockQuantityRule**. If an order is required to bring inventory to that level, then generate a planned order or expedite an existing order as early as allowed based on order rescheduling rules (**Type.ProcessingRule** / **Status.RescheduleRule**), order generation rules (**PartSource.OrderPolicy.OrderGenerationRule**) and the **Part.SafetyStockDate** rule. • **Ignore**—safety stock is never maintained regardless of the settings in other fields. • **IfDemand**—orders for bringing inventory to the safety level will be generated only if there is a non-ignored demand record for the part (**Order.Type.ProcessingRule** does not equal 'Ignore', and **Status.ProcessingRule** does not equal 'Ignore' or quantity is not equal to 0). This includes dependent and independent demand. • **IfActivity**—if there are non-ignored ScheduledReceipts (**Status.IgnoreSupply** does not equal 'Y' or the **Quantity** does not equal 0) or non-ignored IndependentDemands (**Order.Type.ProcessingRule** does not equal 'Ignore', **Status.ProcessingRule** does not equal 'Ignore', or **Quantity** is not equal to 0) or dependent demands. • **OrderPointAverage**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the **SafetyStockQuantityRule** field. The order quantity is determined by the value in **Part.AverageQty** (or **Part.SafetyStockQty** if **Part.AverageQty** is less than or equal to 0), and orders of this size are generated until inventory levels meet or exceed the calculated order point threshold. The start date of the generated planned orders matches the DueDate of the demand they are satisfying. • **OrderPointOver**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the **SafetyStockQuantityRule** field. Order quantities are set to the largest number that will bring inventory levels closest to the maximum of either the order point threshold or **Part.AverageQty** without exceeding it. If using this setting, it is recommended to have the **SafetyStockQuantityRule** field set to "FixedQuantity", and ensure **Part.AverageQty** is set to a value greater than **Part.SafetyStockQty**. RapidResponse will then plan orders	String (Enum)	

Table 7.49: SafetyStockPolicy (Mfg) fields

Field name	Description	Data type	Key
	using these two fields as an upper and lower bound, respectively. • **OrderPointUnder**—orders are generated when inventory levels fall below the order point threshold as determined by the setting in the **SafetyStockQuantityRule** field. Order quantities are set to the smallest number that will bring inventory levels equal to or greater than the maximum of either the order point threshold or **Part.AverageQty**. This setting is intended to keep inventory levels above the order point threshold. The start date of the generated planned orders will match the DueDate of the demand they are satisfying. • **Soft**—safety stock is an available buffer. **ReschedDate** and **RecommendedDate** are left as **DueDate** if existing supplies retain inventory >=0 rather than >= SafetyStock. **Default**: Always		
Value	A unique string identifier for this safety stock policy.	String	Yes
Parts	Set of Part records that reference this safety stock policy.	Set	

ScenarioSetting table

The **ScenarioSetting** table is used to specify the currency conversion date to use for converting calculated Money fields. This is typically specified as part of a scenario perspective, and is intended to be used to compare Money values calculated using conversion rates from different periods.

This table can contain only one record, and its value is specified by a perspective definition.

Table 7.50: ScenarioSetting (Core) fields

Field name	Description	Data type	Key
CurrencyAsOfDate	The historical currency as of date that contains the conversion rates to use for converting calculated Money fields. This value is used only if the value specified is earlier than the date used to determine when historical or actual rates are used to convert a value.	Date	

ShipmentPolicy table

The **ShipmentPolicy** table is used to indicate whether associated independent demands are eligible to be broken into partial shipments.

Table 7.51: ShipmentPolicy (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this ShipmentPolicy record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this shipment policy.	String	
PartialRule	Controls whether independent demands using this shipment policy are eligible to be split into partial shipments. Any independent demands that belong to a ship set cannot be split into partial shipments regardless of the setting on this field (for more information, see "ShipSet table" on page 592). Valid values are: • **Complete**—The entire independent demand quantity must be shipped as a whole. If using non-blocking allocations, component supply allocations are synchronized to the planned available date for the demand (determined by the gating component, if any). This allows any earlier component supplies associated with a late demand to be re-directed towards other demands that might otherwise be late. • **Partial**—The independent demand quantity can be split into partial shipments. If using non-blocking allocations, component supply allocations are synchronized to the due date for the demand. This allows earlier component supplies associated with the amount of a demand that will be late to be re-directed towards other demands that might otherwise be late. **Default value:** Partial Note: For more information about non-blocking allocations, see "Multi-level search logic" on page 1863.	String (Enum)	
Value	A unique identifier for this shipment policy. This field is referenced by the **ShipmentPolicy** field on the **IndependentDemand** table.	String	Yes
Independent Demands	Set of **IndependentDemand** records that reference this ShipmentPolicy value.	Set	

ShipSetType table

The **ShipSetType** table is used to enable or disable non-blocking allocations across the independent demands within a given ship set.

When non-blocking allocations are enabled at the part level, the allocation of a component's supply to a demand can be synchronized with the latest allocated supply from any of its sibling components in order to free up earlier supplies that can be used to potentially satisfy different demands that might otherwise have been late. If non-blocking allocations are further enabled at the ship set level, then

allocations of supply to the demands that belong to that ship set can be synchronized with the latest allocation of supply within that ship set (typically, the most gating). This can free up earlier supplies to satisfy other demands outside of the ship set (for example, demands belonging to another ship set or not associated with any ship set).

Table 7.52: ShipSetType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set this record belongs to. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this ship set type.	String	
MultiLevelSearchRule	Specifies whether non-blocking allocations are used within ship sets of this type. Valid values are: • **Ignore**—non-blocking allocations are disabled for demands in ship sets that reference this type. • **Use**—non-blocking allocations are enabled for demands in ship sets that reference this type. As necessary, component supply allocations are synchronized with the latest available supply that will be allotted to any demands in the ship set. This can potentially free up earlier component supplies to satisfy other demands or ship sets which might otherwise be late. Note that in order for this setting to take effect, non-blocking allocations must also be enabled at the part type level for the affected parts (that is, their **Part.MultiLevelSearchRule** value should be set to "Use"). **Default value:** Ignore	String (Enum)	
Value	A unique identifier for this ship set type.	String	Yes
ShipSets	Set of **ShipSet** records that reference this value.	Set	

SHSCConfiguration table

The **SHSCConfiguration** table contains control settings used by the Self-Healing Supply Chain.

This table was added in Version 2016.2, Service Update 25.

Table 7.53: SHSCConfiguration (Solutions) fields

Field name	Description	Data type	Key
CommitToParentScenario	Determines whether changes in the Self-Healing Working Plan scenario are committed to the parent scenario of Self-Healing Current Plan (Yes) or not (No). The Self-Healing Current Plan and the Approved Actions scenarios are hard-coded into the Commit Lead Time Corrections form. If the parent scenario for Self-Healing Current Plan is set to a different scenario than Approved Actions, then the Approved Actions scenario should be changed to the parent scenario that is used instead. Similarly, if the parent scenario for Self-Healing Current Plan is set to a different scenario than Approved Actions, then the scenario in the Auto Heal Lead Times - Approved Actions scheduled task that is part of the Self-Healing Lead Time automation chain should also be changed to point to the parent scenario that is used instead. Default: Yes	Boolean	
FutureYieldHorizon	Determines how many future periods of yield forecast results are sent from the Machine Learning service. Default: 26	Integer	
FutureYieldCalendar	Determines what calendar periods to use to define the yield forecast result from the Machine Learning service. Default: Weeks **Referenced table:**Calendar	Reference	
HistoricalHorizon	Determines how many historical periods to include in lead time analysis. For example, how many months of historical data will be included. Default: 24	Integer	
HistoricalYieldHorizon	Determines how many historical periods to include in yield analysis. For example, how many weeks of historical data will be included. Default: 156		
HorizonCalendar	Determines what calendar periods to use to define the lead time history. For example, months or weeks. Default: Months **Referenced table:**Calendar	Reference	

Table 7.53: SHSCConfiguration (Solutions) fields

Field name	Description	Data type	Key
LeadTimeType	When running the Machine Learning service for the first time, the Automated Lead Time Type value set here will be used for all ItemSource records. Default: Auto **Note:** After the first run of the Machine Learning service, you can change the AutomatedLeadTimeType from the default value and assign different AutomatedLeadTimeType records to ItemSource records. To change the AutomatedLeadTimeType for an ItemSource, use the Item Source worksheet in the Self-Healing Data workbook. **Note:** The AutomatedLeadTimeType record should be created first in the Automated Lead Time Type worksheet of the Control Tables workbook. **Referenced table:**AutomatedLeadTimeType	Reference	
YieldType	When running the Machine Learning service for the first time, the Automated Yield Type value set here will be used for all ItemSource records. Default: Null **Note:** After the first run of the Machine Learning service, you can change the AutomatedYieldType from the default value and assign different AutomatedYieldType records to ItemSource records. To change the AutomatedYieldType for an ItemSource, use the Item Source worksheet in the Self-Healing Data workbook. **Note:** The AutomatedYieldType record should be created first in the Automated Yield Type worksheet of the Control Tables workbook. **Referenced table:**AutomatedYieldType	Reference	
NewSupplyStatus	Displays the default OrderStatus to use for disabling ScheduledReceipt records for the Self-Healing Supply Chain. This variable should point to a record where **SupplyStatus.IgnoreSupply** = 'Y'. Default: Ignore **Referenced table:**SupplyStatus	Reference	

Table 7.53: SHSCConfiguration (Solutions) fields

Field name	Description	Data type	Key
OrderPolicy	Displays the default Order Policy to use for disabling PartSource records for the Self-Healing Supply Chain. This field should point to a record where **OrderPolicy.OrderGenerationRule** = 'NoOrders', **UseFixedLeadTime** = 'Y', **OrderPolicy.YieldUsage** ≠ 'Ignore', and **OrderPolicy.YieldUsage** ≠ 'ScrapPiece'. **Referenced table:**OrderPolicy	Reference	

SOPAnalyticsConfiguration table

The **SOPAnalyticsConfiguration** table is a control table used to define default parameters for forecasts and disaggregation rate calculations. It holds only one record which is populated using the **SOP Analytics Configuration** worksheet in the Control Tables workbook. This record can be modified on a scenario-by-scenario basis, allowing you to use this table to model various statistical forecast and disaggregation scenarios.

Note that many of the disaggregation parameters in this table can be overridden for a particular part-customer and forecast category combination by providing values in the **ForecastDisaggregationParameters** table, or they can be overridden for all part-customers associated with a given forecast category by providing values in the **ForecastDisaggregationParametersByCategory** table.

Table 7.54: SOPAnalyticsConfiguration (Mfg) fields

Field name	Description	Data type	Key
CalcForecastEnd Date	The calculated date on which to stop reporting disaggregation rates (exclusive). If a positive value is provided in **ForecastEndOffset**, then this is calculated as **CalcForecastStartDate** plus(offset by) that number of **CycleCalendar** intervals. Otherwise, this is calculated as **CalcForecastStartDate** plus one **CycleCalendar** interval.	Date	
CalcForecastStart Date	The calculated date on which to start reporting disaggregation rates (inclusive). If a positive value is provided in **ForecastStartOffset** , then this is calculated as **CalcHistoricalEndDate** plus(offset by) that number of **CycleCalendar** intervals. Otherwise, this is the same as **CalcHistoricalEndDate**.	Date	

Table 7.54: SOPAnalyticsConfiguration (Mfg) fields

Field name	Description	Data type	Key
CalcHistoricalEnd Date	The calculated last date on which historical data can be collected (exclusive). For example, this defines the date back from which historical data can be collected for use in determining disaggregation rates. Set to the nearest **CycleCalendar** interval on or before RunDate (for example, the beginning of the current month).	Date	
CycleCalendar	The calendar that reflects the S&OP cycle used to calculate the historical end date, the forecast start date, and the forecast end date. Also used to define holdout region and forecast lag associated with best fit calculations. This is typically a monthly calendar. **Referenced table:** Calendar	Reference	
Disaggregation ActualsCategory	The historical demand category used to calculate disaggregation rates. **Referenced table:** HistoricalDemandCategory	Reference	
Disaggregation Calendar	The disaggregation calendar is the one and only spreading interval used for all disaggregation. It defines the periods that orders can be disaggregated into, so forecast details are entered and stored at the disaggregation calendar level. It should be the smallest bucket needed for working with forecast quantities. Typical values include week and month. Note that this value can be overridden at the part customer level by providing a valid reference in the **PartCustomer.DisaggregationCalendar** field. Otherwise, if that reference is left Null for a given part customer, the value provided in this field is used. **Referenced table:** Calendar	Reference	
Disaggregation HistoricalInterval Count	The number of inner calendar periods before the forecast start date that are used to collect historical data and causal factors used to calculate disaggregation rates.	Integer	
Disaggregation InnerCalendar	The inner calendar is used in forecast disaggregation with seasonal trends to define the length of a season. This calendar is divided by the outer calendar to provide the index used to determine which period a forecast is disaggregated into. For example, with season month-in-year disaggregation, this would be set to a monthly calendar. Months with higher values in them would then receive more of the data being disaggregated. If forecast disaggregation is not seasonal, this should be set to the same value as the outer calendar. **Referenced table:** Calendar	Reference	

Table 7.54: SOPAnalyticsConfiguration (Mfg) fields

Field name	Description	Data type	Key
Disaggregation OuterCalendar	The outer disaggregation calendar is used to define the period over which a forecast is disaggregated. For example, with season month-in-year disaggregation, this would be set to a yearly calendar. The calendar referenced by this field cannot represent smaller intervals than the one referenced by the inner calendar (they can reference the same calendar if disaggregation is not seasonal). The outer calendar marker should also always land directly on the inner calendar marker. For example, with month-in-year disaggregation, the yearly (outer) calendar marker should land on a marker that defines the start of a monthly (inner) calendar. **Referenced table:** Calendar	Reference	
Forecast Disaggregation OverrideCategory	Supports default disaggregation override rates by forecast category. Override rates are used in place of the calculated disaggregation rates (either the default disaggregation rates or those for a particular part-customer and category). **Referenced table:** HistoricalDemandCategory	Reference	
ForecastEndOffset	Offset from the calculated forecast start date to stop reporting disaggregation rates (expressed in **CycleCalendar** intervals). For example, if using a monthly Cycle calendar, then a value of "24" indicates to report disaggregation rates for 24 months.	Integer	
ForecastStartOffset	Offset from the calculated historical end date to start reporting disaggregation rates (expressed in **CycleCalendar** intervals). For example if using a monthly Cycle calendar, then a value of "0" indicates to start reporting disaggregation rates at the beginning of the current month. This value also impacts the calculation of rolling weight horizons in consensus demand plan calculations. For information, see HistoricalDemandHeaderRollingWeight. **NOTE**: A negative value is treated the same as zero (0).	Integer	
RunDate	The current system date, used to calculate the historical end date, forecast start date, and forecast end date. **Referenced table:** Calendar	Reference	
RServerAvailable	Indicates whether the R Server is available. This is the server supporting the R programming language that gives access to some advanced forecasting functions and methods of outlier detection that are not otherwise supported in RapidResponse. Support for R is only available with the OnDemand RapidResponse Service, and if the optional R forecast capability has been configured.	Boolean	

SourceRule table

The **SourceRule** table determines how PartSource values are interpreted when a part has more than one active part source that is eligible to satisfy a given requirement. For example, this table can be used to determine how target percentage splits between suppliers are interpreted and thus used to determine the amount of the planned requirements sourced from each. This table is referenced from the **Part** table (**Part.SourceRule** field references this table).

Table 7.55: SourceRule (Mfg) fields

Field name	Description	Data type	Key
AllotmentRule	Specifies how a planned requirement is sourced when there are multiple eligible part sources with the same priority. That is, when there are one or more eligible **PartSource** records with the same **Priority** value, this setting indicates how their **Target** values are used in determining the sourcing of the requirements. Valid values are: • **Ongoing**—each planned requirement is fully allocated to the eligible part source that will be the furthest away from (and under) its ongoing target if not used to source that supply. The ongoing target for each **PartSource** record is determined by first dividing its **Target** field by the sum of the **Target** fields from all active **PartSource** records for a given **Priority.** The resulting value is then multiplied by the total quantity of supply allocated across all eligible part sources so far including the current requirement being sourced. These ongoing target values are then compared against the total supply allocated to each part source to determine which should supply the requirement. • **OngoingToDate**—each planned requirement is fully allocated to the eligible part source that has the lowest calculated quota rating based on all previously sourced supply. This option is similar to "OnGoing" but uses slightly different logic which does not include consideration for the quantity of the current requirement being sourced. The quota rating for each PartSource record is determined by dividing its Target field by the sum of the Target fields from all active PartSource records for a given Priority (this yields a target ratio). The total quantity allocated across all eligible part sources, up to but not including the current requirement, is then divided by the calculated target ratio to give the quota rating (with the lowest used to source the requirement). If multiple PartSource records have the same quota rating, the higher Target value is chosen, and if those are the same then the one with the lower SecondaryPriority value is (by default) chosen. • **Proportional**—each planned requirement is split between eligible part sources based on their proportional target values. The proportional target for each **PartSource** record is determined by dividing its **Target** field by the sum of the **Target** fields across all active **PartSource** records of a given **Priority** level. The proportional targets are then multiplied by the quantity of the current requirement to determine the amount to be provided by each source. If order multiples or minimums are applicable, then these are applied separately after the demand has been	String (Enum)	

Table 7.55: SourceRule (Mfg) fields

Field name	Description	Data type	Key
	split between all eligible part sources and can result in excess supply (for these cases, consider using "ProportionalLotSize"). • **ProportionalLotSize**—each planned requirement is split between eligible part sources based on their proportional target values. Proportional targets are recalculated after each portion of the supply is sourced and the outstanding requirement to be sourced adjusted for any lot-sized quantities. With this setting, the **PartSource** record with the largest **Target** is selected first, and its proportional target determined by dividing its **Target** value by the sum of the **Target** values across all active **PartSource** records of a given **Priority** level. The proportional target is then multiplied by the quantity of the requirement to determine the amount to be provided by the source. A supply is then planned for this initial amount and any lot-sizing applied at this time. The **PartSource** with the next largest **Target** is then selected, and its proportional target calculated based on the **Target** values across all eligible **PartSource** records that haven't yet been used for the current requirement. The proportional target is then multiplied by the remaining requirement quantity after adjusting for the previously sourced portions including any lot-sizing. This process repeats until the entire requirement has been sourced. If during the sourcing process multiple eligible **PartSource** records have the same **Target**, then the one with the lowest **SecondaryPriority** is (by default) chosen. Also, although use of this setting can help reduce excess from lot-sizing, the percentage supply splits will not necessarily reflect the specified **Target** values and in some cases eligible part sources might not be used at all (if there is enough excess from other supplies). **Default value:** Ongoing **Note:** If needed an initial allotment value can be specified in **PartSource.ToDateQty** to reflect any prior sourcing decisions not reflected in RapidResponse. This value is then always included in the calculations for the "Ongoing" and "OngoingToDate" options. **Note:** If using multi-level search logic for part source selection, the "OngoingToDate" and "ProportionalLotSize" options are not supported ("Ongoing" and "Proportional" logic respectively will be used instead). **Note:** This field was modified in Version 2016.2, Service Update 6. For more information about use of the settings in this field, see "Part source selection" on page 1377.		

Table 7.55: SourceRule (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this SourceRule record. **Referenced table:** ControlSet	Reference	Yes
DateEffectivityRule	When new supply is needed to satisfy a demand, this setting controls how sourcing is handled in cases where the demand date, less any safety lead time defined at the part level, falls outside of the effective window for all potential part sources. If this adjusted demand date does not fall between **FirstEffectiveDate** and **LastEffectiveDate** on any **PartSource** record, then the requirement will either be shifted into an effective period or left unsatisfied. Valid values are: • **AdjustedDemandDate**—If the adjusted demand date falls outside the effective date range for all part sources, then supply will not be planned for the demand which will be left unsatisfied. Any demands that go unsatisfied due to part source effectivity are logged in the **DemandException** table with a **Type** value of "SourceEffectivityWindow". • **Normal**—If the adjusted demand date falls outside the effective date range for all part sources, then supply can still be planned for the demand. Planned supply due dates must still be within the part source's effective window which will necessarily make them earlier or later than the demand date. However if **SupplyStatus.AllowLatePlan** is set to "Y", then in some cases planned supply due dates might be set beyond the part source's effectivity range. **Default value:** Normal **Note:** This field was added in Version 2016.2, Service Update 8.	String (Enum)	
Description	Additional information about this SourceRule record.	String	
PriorityRule	When sourcing planned requirements by priority, specifies how values in the **PartSource.Priority** and **PartSource.SecondaryPriority** fields are interpreted. Valid values are: • **Ascending**—lower numerical values equate to higher priorities (for example, part sources with lower **Priority** values will be used first when possible). • **Descending**—higher numerical values equate to higher priorities (for example, part sources with higher Priority values will be used first when possible). **Default value:** Ascending	String (Enum)	

Table 7.55: SourceRule (Mfg) fields

Field name	Description	Data type	Key
Value	Identifier for this rule.	String	Yes
Parts	Set of Part records that reference this SourceRule value.	Set	

SpreadProfile table

The **SpreadProfile** table defines a spreading profile used for forecast purposes. This table is referenced from **DemandType** table. For information about a spread profile, see "Forecast spreading" on page 1426.

You can define the characteristics of a spread profile. For example, to define a flat profile that spreads the forecast evenly over each SecondCalendar interval, define a SpreadProfile with **NumberOfPoints** equal to zero or one.

Table 7.56: SpreadProfile (Mfg) fields

Field name	Description	Data Type	Key
ControlSet	The control set associated with this SpreadProfile record. **Referenced table:** ControlSet	Reference	Yes
Description	The description of the demand spread profile.	String	
NumberOfPoints	The number of points (minimum of 0, maximum of 13) used to define the spreading function.	Integer	
Point1	Points defining the spreading function.	Quantity	
Point2	Points defining the spreading function.	Quantity	
Point3	Points defining the spreading function.	Quantity	
Point4	Points defining the spreading function.	Quantity	
Point5	Points defining the spreading function.	Quantity	
Point6	Points defining the spreading function.	Quantity	
Point7	Points defining the spreading function.	Quantity	
Point8	Points defining the spreading function.	Quantity	

Table 7.56: SpreadProfile (Mfg) fields

Field name	Description	Data Type	Key
Point9	Points defining the spreading function.	Quantity	
Point10	Points defining the spreading function.	Quantity	
Point11	Points defining the spreading function.	Quantity	
Point12	Points defining the spreading function.	Quantity	
Point13	Points defining the spreading function.	Quantity	
Value	The name of the demand spread profile.	String	Yes
DemandTypes	Set of **DemandType** records that reference this SpreadProfile value.	Set	

Sample data

The following table shows four example SpreadProfile records.

Table 7.57: Example records for the SpreadProfile table

Field	Example 1	Example 2	Example 3	Example 4
Description	No spreading is used. This is the default.	A profile that rises evenly.	A profile that has a sharp rise.	This uses 13 points.
NumberOfPoints	0	3	5	13
Point1	0	20	20	20
Point2	0	40	30	25
Point3	0	60	45	30
Point4	0	0	65	35
Point5	0	0	90	40
Point6	0	0	0	42
Point7	0	0	0	40
Point8	0	0	0	37
Point9	0	0	0	35
Point10	0	0	0	27
Point11	0	0	0	24
Point12	0	0	0	20
Point13	0	0	0	13
Value	Default	LinearRise	Exponential	Seasonal

SubstituteGroupType table

The **SubstituteGroupType** table is referenced by the **SubstituteGroup** table. It defines the processing rules that are associated with groups of substitutable components within a given assembly.

Table 7.58: SubstituteGroupType (Mfg) fields

Field name	Description	Data type	Key
ComponentRule	When using on hand or scheduled receipts to satisfy demand for a substitutable component, this field determines how different components from the group can be mixed together on a single assembly order. Valid values are: • **AssemblyUnit**—only one substitutable component can be used for each unit of assembly supply in the order. That is, component supply can only be used in increments equal to the component's QuantityPer value. • **Single**—only one substitutable component can be used for each assembly supply order. If planned orders are required to fully satisfy a demand, they are created on that same component. However, if the demand is only being satisfied by planned orders, then planned order creation is determined by the BillOfMaterial.Target field. • **Unrestricted**—no restrictions on how substitutable components can be mixed together on a supply order for the assembly. If planned orders are required to fully satisfy the demand, they created across the group's components in the ratio determined by the BillOfMaterial.Target field. **Default value**: Single	String (Enum)	
Component SourceRule	When planned orders are required to satisfy dependent demands placed on substitutable components, this setting determines how the requirements are split between those components. Valid values are: • **Ongoing**—each dependent requirement from the assembly is assigned to the one substitutable component in the group that will be furthest away from its ongoing target if not used to satisfy the demand. • **Proportional**—each dependent requirement from the assembly is split proportionally between substitutable components in the group based on their weighted targets as calculated from **BillOfMaterial.Target**. **Default value:** Proportional **Note:** For more information about this field's usage, see "Define planned order generation" on page 1571.	String (Enum)	

Table 7.58: SubstituteGroupType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this SubstituteGroupType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this SubstituteGroupType record.	String	
ProcessingRule	Controls whether component substitution is enabled for groups of this type. Valid values are: • **Ignore**—component substitution is disabled within the group. Demand on the primary component in the group can only be satisfied by its own supply. • **OptionRatio**—component substitution is disabled within the group. This type is used to group related option ratio components under an assembly. It allows incremental availability to be rolled up on each component within the group, and one unit of supply of the parent can therefore be built when any of the option components are available (otherwise, supply is required of each option component before a unit of the assembly can be built). • **Substitute**—component substitution is enabled within the group. Substitute components can be used to satisfy demand on the primary component in the group. **Default value**: Ignore	String (Enum)	
SubstituteGroups	Set of **SubstituteGroup** records that reference this SubstituteGroupType value.	Set	
SupplyPreference	When acceptable supply is available on multiple components in the group, this settings controls which of those components has its current supply used first. Valid values are: • **Earliest**—preference is given to the component with the earliest available supply that can be used. With this setting, onhand supplies are consumed first, and then the earliest scheduled supplies. Note that in cases where multiple components have supply available on the same date, then the one with the lowest value in the **BillOfMaterial.SubstitutionSequence** field is used first. • **Sequence**—preference is given to the component with the lowest value in the **BillOfMaterial.SubstitutionSequence** field. **Default value**: Sequence	String (Enum)	
Value	A unique identifier for this type of substitute group. This value is referenced by each record in the **SubstituteGroup** table that belongs to this group type.	String	Yes

SupplyAllocationType table

The **SupplyAllocationType** table is referenced by the **SupplyAllocation** table, and is used to determine if a manual supply allocation should be processed by RapidResponse analytics.

Table 7.59: SupplyAllocationType (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this SupplyAllocationType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this SupplyAllocationType record.	String	
ProcessingRule	Controls whether the manual supply allocation record is processed by RapidResponse. Valid values are: • **Ignore**—The supply allocation is ignored. • **Use**—The supply allocation is processed. **Default value**: Use	String (Enum)	
PTFLastDueRule	Specifies whether supply allocations of this type are considered in planning time fence logic for part sources having an **OrderPolicy.PTFRule** setting of either "LastDueFence", "LastDueLead", "LastDueFenceAll" or "LastDueLeadAll". Valid values are: • **Include**—Supplies of this type are included in the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules. This means the due date on the last supply allocation associated with an order of this type for the part can potentially be set as the part source's **PTFDate** (assuming it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). • **Exclude**—Supplies of this type are excluded from the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules. This means the due date on the last supply allocation associated with a supply of this type for the part will never be set as the part source's **PTFDate** (even if it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). **Default value**: Include **Note:** This field was added in Service Update 1908 (prior to this, it was not possible to exclude specific supply allocation types from the **PTFDate** calculation). For more information, see "Determining the PTFDate" on page 1344	String (Enum)	

Table 7.59: SupplyAllocationType (Mfg) fields

Field name	Description	Data type	Key
Supply Allocations	Set of SupplyAllocation records that reference this SupplyAllocationType value.	Set	
Value	A unique identifier for this SupplyAllocationType record. This field is referenced by the **Type** field on the **SupplyAllocation** table to determine the processing policies for a particular manual supply allocation.	String	Yes

SupplyStatus table

The **SupplyStatus** table controls the state of supplies (unreleased work orders, work orders, released work orders, planned purchase orders, and firm purchase orders). This table is referenced from the **PartSourceType** and **ScheduledReceipt** tables.

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
AllowLatePlan	This setting controls whether a supply's due date is allowed to be set later than its associated demand date and applies only when using constraints and/or multi-level search logic. When using constraints, this setting determines whether a supply's due date can be set later than would be needed to meet the demand date in cases where there is insufficient capacity available to meet that demand date. When using multi-level search, this setting controls whethers a supply's due date can be set later than neethe demand date in cases where there is insufficient capacity or availaiblity of component materials. This allows supplies to be scheduled when they are expected to have capacity, rather than to their (earlier) demand dates. This setting affects planned orders, as well as scheduled receipts that are reschedulable. Valid values are: • **Y**—If using constraints, supplies are planned for when they are expected to have capacity. If using multi-level search, supplies are planned for when they are expected to have both capacity and material availability. • **N**—Supplies are always planned to meet demand dates (This is the recommended value.) **Default value**: N For more information about AllowLatePlan, see "AllowLatePlan" on page 1663.	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
AvailableDate Limit	A supply order's **AvailableDate** is always limited by constraint and material availability, as well as the absolute available date limit as discussed in "AvailableDateLimit" on page 1666. Subject to these limitations, this field determines the earliest date that RapidResponse can set the available date for supply orders. Valid values are: • **CalcRequestStart—**build the supply based on its **CalcRequestStartDate.** Supply cannot be available earlier than **CalcRequestStartDate** + lead time, and constraint cannot be allocated any earlier than **CalcRequestStartDate** converted to a constraint date. Note that each supplies **CalcRequestStartDate** is derived from the original **RequestDate** provided on the (earliest) driving independent demand it satisfies, and which is then propagated down the product structure and adjusted for lead time at each component level. Therefore, this option is intended for use with supplies satisfying independent demands that are configured for two-date planning (have their **OrderPriority.TwoDatePlanningRule** set to "Use"). If this option is used with supplies having an undefined **CalcRequestStartDate**, then it is equivalent to using the "CalcStart" option (for example, this can occur with supplies satisfying consensus forecast demand or supplies satisfying independent demands with an undefined **RequestDate**). Additionally, if the calculated start date is earlier than the date determined from this setting, then it becomes the available date limit instead ("CalcStart" setting is used). • **CalcStart**—build the supply based on its calculated start date (**CalcStartDate** for scheduled receipts and **StartDate** for planned orders). Supply cannot be available earlier than **CalcStartDate/StartDate** + lead time , and constraint cannot be allocated any earlier than **CalcStartDate/StartDate** converted to a constraint date. • **Due**—build the supply so that the earliest it can be completed is the order's due date (even if rescheduled earlier). In addition, constraint cannot be allocated before **EffStartDate** (**StartDate** for planned orders) converted to a constraint date. • **Now**—if a supply order is unconstrained, it can be built as soon as materials are available (subject to the absolute limits on available date . If all material is available before the FirstEffectiveDate on the BillOfMaterial record through which the demand	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
	exploded, the available date on the supply is still reported based on material date and might end up being seen as available before the beginning of the associated BOM effectivity range. If a supply order is constrained and there is sufficient constraint to complete the order on time, then it will not start consuming constraint until its calculated start date (even if materials are available before that date). If a supply order is constrained and there is insufficient constraint to complete the order on time, then it can start consuming constraint as needed to meet the supply's rescheduled date (subject to the absolute limits on available date). **Default value**: CalcStart **Note:** For more information about how this field affects Constraint Analysis calculations, see "AvailableDateLimit" on page 1666. For more information about how this field affects incremental availability calculations, see "Defining when order splits can occur" on page 1480. **Note:** When push-planning logic (distribution planing) is used, some pushed supplies might up with available dates that are later than the limit determined by the AvailableDateLimit setting.		
CoByProduct Usage	Used with scheduled receipts for parts that are defined as the primary product in a co-product or by-product relationship. This field indicates whether co-product and by-product supply should be generated from the scheduled receipt. Valid values are: • **Generate**— Co-product and by-product supply is automatically generated from the primary product's scheduled receipt based on the values defined on the associated BillOfMaterial records. This setting is used for scheduled receipts for which the expected co-product or by-product supply has not been generated. • **DoNotGenerate**— Co-product and by-product supply is not generated from the scheduled receipt. This setting is used for scheduled receipts in the later stages of the production process where any co-product or by-product supplies have already been created and are being managed separately from the primary product. For more information, see "Co-products and by-products" on page 1581. **Default value**: Generate **Note:** If the **State** field is set to "Historical", then co-product and by-product supplies are not generated on the scheduled receipt regardless of the setting in this field.	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
ConstraintRule	Processing rule that affects how constraint is applied to scheduled receipts when calculating AvailableDate. This field makes it possible to prevent available date changes to scheduled receipts, even if total load exceeds the available constraint. For example, this may be used when a scheduled receipt represents a firm commitment that has been agreed to by an external supplier. Valid values are: • **Consume**—consume constraint, but do not consider its availability when calculating AvailableDate. This setting is intended for use with supplies that represent a firm commitment (for example, supplies that have been scheduled by an external system). • **BeforeAD**—consume Constraint and use its availability when calculating AvailableDate. This is the recommended setting. **Default:** Consume **Note:** This field is now obsolete and maintained only for backwards compatibility. It is recommended that you always set this field to **BeforeAD**. If you require the behavior provided by **Consume**, set the **RescheduleRule** field to **Scheduled.**	String (Enum)	
ControlSet	The control set associated with this SupplyStatus record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this type of SupplyStatus. **Default value**: blank string	String	
ExpediteLimit	Specifies the range that supplies can be expedited. Valid values are: • **Past**—date will be set to whatever date the supply is needed, including the past, if necessary. • **PTFDate**—supplies can only be expedited up to the calculated **PartSource.PTFDate**. Note that this option ignores the **OrderPolicy.LotSizeLastOrder** setting. For more information about the **PTFDate** calculation, see "Determining the PTFDate" on page 1344 • **RunDate**—supplies can only be expedited up to the Part.PlanningCalendars.RunDate. FirstDate. • **DueDate**—supplies can't be expedited. The only allowed recommended date changes are cancels and delay. **Default value:** Past	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
IgnoreStartDate	Specifies when rescheduling a supply's DueDate whether to always update the Supply StartDate and associated dependent demand Allocation DueDate. Valid values are: • **Y**—RapidResponse ignores the **ScheduledReceipt.StartDate** and **Allocation.DueDates.** Instead, **ScheduledReceipt.CalcStartDate** is used (calculated using the ScheduledReceipt.ReschedDate) to offset by the applicable lead-time fields. **SupplyType.ProcessingRule** and **SupplyStatus.RescheduleRule** are also taken into consideration for their effect on the **ScheduledReceipt.ReschedDate**. • **N**—RapidResponse uses the S**cheduledReceipt.StartDate** and **Allocation.DueDates**. **Default value:** N	Boolean	
IgnoreSupply	Specifies whether any type of supply should not be considered in netting and explosion. Valid values are: • **Y**—RapidResponse ignores all scheduled receipts associated with this SupplyStatus. • **N**—RapidResponse considers all scheduled receipts associated with this SupplyStatus. **Default value**: N	Boolean	
IncrementalSplit	For parts that have incremental availability calculations turned on, this field allows the calculations to be turned off on a supply-by-supply basis. Valid values are: • **FromPart**—Incremental availability calculations are performed for this supply if they have been turned on for the part (as set by the **Part.IncrementalRule** field). Otherwise, incremental availability calculations are not performed for the supply. • **Off**—Incremental availability calculations are not performed for this supply (regardless of the setting on the Part.IncrementalRule field). **Default value**: FromPart	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
LeadTime Override	Specifies whether lead time calculations use values defined on the part source, or are overridden with values defined on the scheduled receipt. This setting allows lead time overrides to be defined on a scheduled receipt by scheduled receipt basis. Valid values are: • **Yes**—Lead time values from the scheduled receipt are used in lead time calculations. • **No**—Lead time values from the part source associated with the scheduled receipt are used in lead time calculations. • **FromOrder**—The control setting on the supply order (**SupplyType.LeadTimeRule**) is used to determine whether lead time values from the scheduled receipt or part source are used in lead time calculations. **Default value**: No For more information on overriding lead time on scheduled receipts, see "Lead time overrides from scheduled receipts" on page 1328.	String (Enum)	
ModelUsage	Indicates how the model associated with a given scheduled receipt is determined. The value reported in the calculated **ScheduledReceipt.Model** field is also affected by this setting. Actual usage of the model in analytic calculations is then set by the part's **MUEPoolNettingType.ModelRule** value. Valid values are: • **Ignore**—model is set to the default model. This is the first value loaded in the **Model** table and typically has a value of "None". The input value provided in **ScheduledReceipt.InputModel** is ignored. • **Normal**—model is set to the input value provided in **ScheduledReceipt.InputModel**. **Default value:** Normal	String (Enum)	
PartialExplode	Specifies whether supply assigned this status only generates dependent demand when **BillOfMaterial.OperationNumber** is greater than or equal to **ScheduledReceipt.CurrentOperation** before generating dependent requirements (allocations). Valid values are: • **Y**—yes • **N**—supply assigned this status does not perform this test when calculating explosion. **Default value**: N	Boolean	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
PrioritySource	Controls how the priority on scheduled receipts and planned orders is set. For scheduled receipts, this setting controls how a scheduled receipt's effective order priority (**ScheduledReceipt.EffOrderPriority**) is determined. For planned orders from a given part source, this setting determines how a planned order's priority (**PlannedOrder.OrderPriority**) is determined. Valid values are: • **Average**— effective order priority is set to an average of the priorities associated with all demands satisfied by the supply. That is, the **OrderPriority.PlanningPriority** values found on all demands satisfied by the supply are summed together and then divided by the total number of demands; the **OrderPriority.Value** associated with the closest **PlanningPriority** value that is equal to or lower than that calculated average is then reported. • **FromInput**— The use of this value in setting priority depends on whether the supply is a scheduled receipt or a planned order. For scheduled receipts, the effective order priority is taken from the input **OrderPriority** value on the **ScheduledReceipt** record. For planned orders, the order priority is set to the input **OrderPriority** value on the triggering **IndependentDemand** record that led to the creation of this supply. • **Highest**— effective order priority is set to the highest priority associated with any demand satisfied by the supply during netting calculations. That is, the **OrderPriority.Value** associated with the lowest **OrderPriority.PlanningPriority** number on any demand satisfied by the supply is reported. • **WeightedAverage**— effective order priority is set to a weighted average of the priorities associated with all demands satisfied by the supply during netting calculations. That is, the **OrderPriority.PlanningPriority** value found on each demand satisfied by the supply is multiplied by the demand quantity, summed together, and then divided by the total demand quantity being satisfied; the **OrderPriority.Value** associated with the closest **PlanningPriority** value that is equal to or lower than that weighted average is then reported. **Default value**: FromInput **Note:** If a scheduled receipt does not satisfy any demands, it is treated as being set to FromInput.	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
	Note: Planned orders for parts that have PartType.CommitLevel set to either "Medium" or "Low" always use the part's default priority. **Note:** For more information about priorities on supplies, see "Priorities on scheduled receipts" on page 1788 and "Priorities on planned orders" on page 1790.		
PTFLastDueRule	Specifies whether scheduled receipts having this status are considered in planning time fence logic for part sources where the **OrderPolicy.PTFRule** setting is either "LastDueFence", "LastDueLead", "LastDueFenceAll" or "LastDueLeadAll". By default, scheduled receipts inherit this setting from the **SupplyType.PTFLastDueRule** setting referenced by the supply order they belong to, but this field allows overriding that setting on a scheduled receipt basis. Valid values are: • **FromOrder**—Scheduled receipts with this status are included or excluded from the **PTFDate** calculation based on the **SupplyType.PTFLastDueRule** setting used by their supply order. • **Include**—Scheduled receipts with this status are always included in the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules (regardless of the setting on the corresponding supply order).. This means the due date on the part's latest scheduled receipt having this status can potentially be set as the part source's **PTFDate** (assuming it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). • **Exclude**—Scheduled receipts with this status are always excluded from the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules (regardless of the setting on the corresponding supply order). This means the due date on the part's latest scheduled receipt having this status will never be set as the part source's **PTFDate** (even if it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). **Default value**: FromOrder **Note:** This field was added in Service Update 1908 (prior to this, it was not possible to exclude specific scheduled receipts from the **PTFDate** calculation). For more information, see "Determining the PTFDate" on page 1344	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
RescheduleRule	Specifies the reschedule rule to be applied to the ScheduledReceipt record (referencing the SupplyStatus table). This value overrides what has been set at the order level using the **SupplyType.ProcessingRule**. Valid values are: • **FromOrder**—supplies will use the rescheduling messaging and logic defined at the order level which is configured using the **SupplyType.ProcessingRule**. • **NonReschedulable**—supplies are assumed to be unable to be rescheduled even if required earlier or later than it is currently scheduled. The **ScheduledReceipt.DueDate** is the date used for netting and explosion purposes. With this setting, a new planned order might be created to cover the demand instead of using the scheduled receipt (depending on when the supply is required). • **Reschedulable**—supplies are assumed to be fully reschedulable. Recommended dates and action messages will be generated where required. **ScheduledReceipt.ReschedDate** is populated with the **ScheduledReceipt.RecommendedDate** and is used for netting and explosion. • **Received**—supply has been received into inventory at its destination (for example, the brand owner's site) and is not included as a supply in Netting calculations. If **SupplyType.ProcessingRule** is set to 'InTransit' or 'InTransitExact', the receipt consumes other scheduled receipts. However, if **SupplyType.ProcessingRule** is set to a value other than 'InTransit' or 'InTransitExact', the receipt is ignored. • **RecommendOnly**—supplies are moved so they align with demands, and may be expedited. RapidResponse recommends a rescheduled date (**ScheduledReceipt.RecommendedDate**), but does not change the effective due date (**ScheduledReceipt.ReschedDate** is populated with **ScheduledReceipt.DueDate**). With this setting, RapidResponse will try to use the scheduled receipt before planning any new planned orders. • **RecommendIfType**—supplies behave the same as the RecommendOnly option, provided the order level setting (**SupplyType.ProcessingRule**) is not set to NonReschedulable or Ignore. • **Scheduled**—supplies are non-reschedulable. If sufficient constraint is not available, the excess balance is applied to the last constraint period, and the constraint used in that period will exceed the amount of constraint available (thus overloading the constraint). This setting is intended for use with supplies that have been scheduled by another system, and which represent a	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
	firm commitment that has already been agreed to (and allowances made for). Note that with this setting, the supply's **AvailableDate** will be set to its **DueDate** even if there are material shortages . In cases where the **AvailableDateLimit** control uses a setting other than "Due", the **AvailableDate** might in some cases be set earlier than **DueDate** (for example, when the "CalcStart" setting is used and **EffDue** falls on a non-workday, the **AvailableDate** is set to the previous workday). **Default value:** FromOrder		
SortBeforeStart	Specifies how to prioritize supplies. In cases where two supplies have the same DueDate and a sort priority is not able to be determined by the **SupplyType.SortSameDate**, supply consumption priority is determined by comparing the supplies at the line level using SupplyStatus.SortBeforeStart. The lower the value the higher the priority. For a listing of all fields used when sorting scheduled receipts, see "Scheduled receipts usage" on page 1396. **Default value**: 1	Quantity	
SortSameStart	Specifies how to prioritize supply consumption priority. In cases where two supplies have the same DueDate and StartDate, and a sort priority cannot be determined by either the **SupplyType.SortSameDate** or by the **SupplyStatus.SortBeforeStart**, priority is determined by SortSameStart setting. The lower the value the higher the priority. For a listing of all fields used when sorting scheduled receipts, see "Scheduled receipts usage" on page 1396. **Default value**: 1	Quantity	
SplitLateSupply	Provides the option to split the on-time portion of a planned supply from the late portion. In other words, plan a portion of the supply on time even if the entire order cannot be planned on time. This field only affects orders using at least one constraint that is constrained. Valid values are: • **Y**—Plans some portion of this supply on time, if possible, then plans the balance later. If the part source is constrained by a fixed constraint, this value is treated the same as N (order is not split). • **N**—If this entire supply cannot be scheduled on time, then plan a higher priority order now, and this entire supply later. This is the recommended setting. **Default value**: Y	Boolean	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
State	Specifies whether RapidResponse generates dependent requirements, and also affects the lower bound on AvailableDate. **SupplyType.Source** is used in conjunction with the State setting to control the generation and timing of allocations. Valid values are: • **BackflushAndAllocated—**dependent requirements are not generated for this scheduled receipt. Input allocations are treated as due at "past". AvailableDate is never earlier than PlanningCalendars.DataDate.FirstDate. This value differs from ReleasedAndAllocated in that lead time is not incurred if dependent demands are present but satisfied by on hands. • **BackflushButNotAllocated—**dependent requirements are generated for this scheduled receipt, and all dependent requirements are treated as due at "past". AvailableDate is never earlier than Part.PlanningCalendars.DataDate.FirstDate. This value differs from ReleasedButNotAllocated in that lead time is not incurred if dependent requirements are present but all satisfied by on hands. • **Historical**—supply assigned this state is ignored by netting and does not generate dependent requirements (allocations). This value overrides the SupplyStatus.IgnoreSupply field. • **NotReleased**—dependent requirements are generated for this scheduled receipt. AvailableDate is never earlier than PlanningCalendars.DataFirst.FirstDate + lead time (sum of all applicable lead time fields). Lead time is always included in AvailableDate calculations for scheduled receipts with this State, even if no dependent requirements are present. • **NotReleasedAndAllocated**—dependent requirements (allocations) are not generated for this scheduled receipt. AvailableDate is never earlier than the PlanningCalendars.DataDate.FirstDate + LeadTime (sum of all applicable lead time fields). Lead time is always included in AvailableDate calculations for scheduled receipts with this State, even if no dependent requirements are present. • **Planned**—supply assigned this State behaves the same as 'NotReleased' except AvailableDate is never earlier than the PlanningCalendars.RunDate.FirstDate + LeadTime (sum of all applicable lead time fields). All planned orders are treated as if State is 'Planned'. • **ReleasedAndAllocated**—dependent requirements are not generated. AvailableDate is never earlier than PlanningCalendars.DataDate.FirstDate. Lead time is included in AvailableDate calculations for scheduled receipts with this State only if they have dependent	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
	requirements. Use for orders that have begun production and had components pulled from inventory. • **ReleasedButNotAllocated**—dependent requirements are generated. AvailableDate is never earlier than PlanningCalendars.DataDate.FirstDate. Lead time is included in AvailableDate calculations for scheduled receipts with this State only if they have dependent requirements. Use for orders that have begun production but not had components pulled from inventory. **Default value**: NotReleased		

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
TargetAccum	Specifies how scheduled receipt quantities are applied both when generating constraint load, and when accumulating allotments for a **PartSource** with a **Part.SourceRule.AllotmentRule** value of 'Ongoing'. This option is further subject to the setting in the **TargetAccumUsage** field. Valid values are: • **Ignore**—supply assigned this status does not perform accumulation logic in conjunction with the **SourceRule.AllotmentRule** nor does the supply generate load on or consume any constraint. Use this setting if neither constraint nor multi-sourcing logic is used. • **Remaining**—supply assigned this status adds the **ScheduledReceipt.Quantity** (the effective remaining quantity) to **PartSource.ToDateQty** in order to determine the 'current ToDateQty'. The **ScheduledReceipt.Quantity** (the effective remaining quantity) also generates fixed and variable constraint load. • **RemainingNoFixedLoad**—supply assigned this status adds the **ScheduledReceipt.Quantity** (the effective remaining quantity) to **PartSource.ToDateQty** in order to determine the 'current ToDateQty'. The **ScheduledReceipt.Quantity** (the effective remaining quantity) also generates variable constraint load, but does not generate fixed constraint load. • **Order**—supply assigned this status adds the **ScheduledReceipt.TotalQuantity** (the entire quantity of the order) to **PartSource.ToDateQty** in order to determine the 'current ToDateQty'. The **ScheduledReceipt.TotalQuantity** (the entire quantity of the order) also generates constraint load. **Default value:** Ignore **Note:** Applies to scheduled receipts only (planned orders always use the order quantity in calculating constraint load and allotment accumulation).	String (Enum)	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
TargetAccum Usage	This setting controls how the value chosen in the **TargetAccum** field is applied. It allows for scheduled receipt quantities to be used in determining how constraint load is generated, but ignored by the allotment accumulation logic applicable when **SourceRule.AllotmentRule** is "Ongoing" and which influences source selection when there are multiple eligible part sources. For example, scheduled receipt quantities might always generate constraint load, but multi-sourcing logic and hence target allotment accumulation might not be applicable for scheduled receipts until a particular point in the planning horizon. Valid values are: • **AccumulationAndConstraint**—the **TargetAccum** setting is used both in determining how scheduled receipt quantities generate constraint load, and in how those quantities are used when accumulating allotments for "Ongoing" part sources. • **Constraint**—the **TargetAccum** setting is used only in determining how scheduled receipt quantities generate constraint load. Scheduled receipts that use this option are ignored by the allotment accumulation logic applicable to "Ongoing" part sources. **Default value**: AccumulationAndConstraint	String (Enum)	
Value	The string value for this type of SupplyStatus. This value appears in the ScheduledReceipt.Status field. **Default value**: blank string	String	Yes
YieldRule	Specifies whether supplies use the yield ratio and coproduct yield ratio defined on the part source. For example, this setting might be used for scheduled receipts at a later stage of production where the yield factor no longer applies. Valid values are: • **Ignore**—supplies ignore the Yield and CoProductYield ratios defined on the part source. Yield on supplies using this setting is assumed to be 100%. • **Use**—supplies are reduced by the Yield and CoProductYield ratios defined on the part source. **Default value:** Use	String (Enum)	
PartSourceTypes_ PlannedOrder	Set of **PartSourceType** records that reference this SupplyStatus value through their **PlannedOrderSupplyStatus** field.	Set	

Table 7.60: SupplyStatus (Mfg) fields

Field name	Description	Data type	Key
PartSourceTypes_ PlannedOrder ToSR	Set of **PartSourceType** records that reference this SupplyStatus value through their **PlannedOrderToSRSupplyStatus** field.	Set	
PartSourceTypes_ Scheduled Receipts	Set of **ScheduledReceipt** records that reference this SupplyStatus value.	Set	

SupplyType table

The **SupplyType** table defines types for use in the **SupplyOrder** table, as well as the rules for processing scheduled receipts related to **SupplyOrder** records. It's also used in the **PartSourceType** record for defining characteristics for processing of planned orders.

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
Allocation DemandType	Specifies the type of demand (DemandType) that is assigned when generating dependent requirements (allocations) from supplies that are assigned this SupplyType. This field references the DemandType table. For more information, see "DemandType table" on page 715. **Referenced table:** DemandType	Reference	
Allocation ForecastRule	Specifies whether forecasts and sales actuals exploded from scheduled receipts are used in forecast generation and consumption respectively (dependent demands from planned orders are always included in forecast logic). Valid values are: • **Ignore**—Dependent demands from scheduled receipts do not generate forecast demands, nor do they consume other forecast demands. • **Use**—Dependent demands from scheduled receipts with **Order.Type.AllocationDemandType.ProcessingRule** = 'SalesForecast' generate forecast demands for MPS parts. Dependent demands from scheduled receipts with **Order.Type.AllocationDemandType.ProcesingRule** = 'SalesActual' consume forecast demands for MPS parts. **Default value**: Ignore	String (Enum)	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
BuyInProcess Rule	Specifies whether "Buy" type (purchased) supplies are included in in-process inventory calculations. Valid values are: • **Ignore**—Do not include this "Buy" supply type in in-process inventory calculations. • **IncludeDockToStock**—Includes this "Buy" supply type in in-process inventory calculations (for the duration of dock-to-stock lead time). **Default value**: Ignore	String (Enum)	
Capacity ProcessingRule	Specifies the operation scheduling method applied to supplies (planned and firm) in order to calculate load when using the Capacity Requirements Planning analytic. If this analytic is not being used then this field should be set to 'Ignore'. Valid values are: • **Forward**—scheduling starts with the first operation at the supply planned start date and proceeds forward through the operations. • **Backward**—scheduling starts with the last operation and schedules backward through the operations from the supplies planned completion date. • **Ignore**—operation scheduling is ignored. For open orders, the link to this field is from Scheduled Receipt to SupplyOrder to SupplyType. For planned orders the link to this field is through PartSourceType (PlannedOrderSupplyType) to SupplyType. **Default value**: Forward	String (Enum)	
ControlSet	The control set associated with this SupplyType record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this type of supply. An explanation of the code in the Value field. **Default value**: blank string	String	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
LeadTimeRule	Specifies whether lead time calculations use values defined on the part source, or are overridden with values defined on the scheduled receipt. This setting allows lead time overrides to be defined at the order level, and is only valid if **SupplyStatus.LeadTimeOverride** is set to "FromOrder". Valid values are: • **Override**—Lead time values from the scheduled receipt are used in lead time calculations. • **Ignore**—Lead time values from the part source associated with the scheduled receipt are used in lead time calculations. **Default value**: Ignore For more information on overriding lead time on scheduled receipts, see "Lead time overrides from scheduled receipts" on page 1328.	String (Enum)	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Specifies a processing rule for each scheduled receipt. The processing rule controls, at the order number or header level, both the reschedule recommendations as well as whether the supply is ignored. Valid values are: • **ExplodeOnly**—supply is ignored and considered as non-reschedulable. However, dependent requirements are generated from this type of supply and the **ScheduledReceipt.ReschedDate** (which is populated with the **ScheduledReceipt.DueDate**) is used for explosion. For example, a repair order may require components, but are shipped to the customer and do not get mixed with other supplies. Constraint, if applicable, may be consumed. • **Ignore**—supply is ignored in netting and calculated tables. It does, however, appear in the ScheduledReceipt table. This applies to order number level. • **InTransit**—consumes supply from scheduled receipts having the same part, model, unit, and pool. This consumption occurs in chronological date order, and occurs after any consumption done by 'InTransitExact' scheduled receipts. All scheduled receipts with this setting are also treated as 'non-reschedulable'. • **InTransitExact**—consumes supply from scheduled receipts having the same supplier, routing, part, model, unit, and pool. This consumption occurs in chronological date order. All scheduled receipts with this setting are also treated as 'non-reschedulable'. • **NonReschedulable**—supplies are assumed to be unable to be rescheduled even if required earlier or later than it is currently scheduled. The **ScheduledReceipt.DueDate** is the date used for netting and explosion purposes. With this setting, a new planned order might be created to cover the demand instead of using the scheduled receipt (depending on when the supply is required). • **RecommendOnly**—supplies are moved so they align with demands, and may be expedited. RapidResponse recommends a rescheduled date (**ScheduledReceipt.RecommendedDate**), but does not change the effective due date (**ScheduledReceipt.ReschedDate** is populated with **ScheduledReceipt.DueDate**). With this setting, RapidResponse will try to use the scheduled receipt before planning any new planned orders. • **Reschedulable**—supply can be fully reschedulable. Recommended dates and action messages are generated where required. **ScheduledReceipt.ReschedDate** is populated with the **ScheduledReceipt.RecommendedDate** and is used for netting and explosion. **Default value**: Reschedulable	String (Enum)	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
PTFLastDueRule	Specifies whether scheduled receipts belonging to orders of this type are considered in planning time fence logic for part sources having an **OrderPolicy.PTFRule** setting of either "LastDueFence", "LastDueLead", "LastDueFenceAll" or "LastDueLeadAll". Valid values are: • **Include**—Supplies of this type are included in the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules. This means the due date on the latest scheduled receipt for the part that belongs to an order of this type can potentially be set as the part source's **PTFDate** (assuming it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). • **Exclude**—Supplies of this type are excluded from the **PTFDate** calculation for part sources that use any of the "LastDue" planning time fence rules. This means the due date on the latest scheduled receipt for a part that belongs to an order of this type cannot be set as the part source's **PTFDate** (even if it is beyond the part source's fence or lead time value and later than any other scheduled supply from the part source). **Default value**: Include **Note:** This field was added in Service Update 1908 (prior to this, it was not possible to exclude specific supply types from the **PTFDate** calculation). For more information, see "Determining the PTFDate" on page 1344 **Note:** It is possible to override this setting at the scheduled receipt level using the PTFLastDueRule setting on the **SupplyStatus** table.	String (Enum)	
SortBeforeDate	Specifies how supplies are prioritized when being allocated to demands. RapidResponse sorts supply by comparing values in this field (sorting priority before sorting by DueDate) for reschedulable supplies only (**SupplyType.ProcessingRule** = 'Reschedulable'). For SupplyType values where the ProcessingRule = 'Reschedulable', this field represents the priority in which supplies should be allocated to demands. This must be a positive value and lower values have a higher priority. For a listing of all fields used when sorting scheduled receipts, see "Scheduled receipts usage" on page 1396. **Default value**: 0	Quantity	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
SortSameDate	Specifies how to sort supplies that have same DueDate. RapidResponse first sorts these supplies by comparing values in this field. This must be a positive value and lower values have a higher priority. For a listing of all fields used when sorting scheduled receipts, see "Scheduled receipts usage" on page 1396. **Default value**: 0	Quantity	
Source	Specifies a source for each SupplyType. The Source value works in conjunction with the BillOfMaterials to control the generation of dependent requirements (allocations) on components. It also works in conjunction with the PartSource, in the case of 'Transfer", to handle inter-site requirements. Valid values are: • **Make**—RapidResponse will look at **BillOfMaterial** records for the Assembly in order to determine which set of Component parts are required to create the supply for this part. Typically, this type of source represents cases where the site itself builds the supply for this part, and is associated with work orders and kitting operations where several different components are assembled together to form a new assembly. • **Buy**—RapidResponse does not generate any further dependent demands for supply using the PartSource. The expectation is that purchase orders are created and communicated to suppliers that manage their own planning. • **Transfer**—Indicates that supply for the part will be provided from another part and site. Unlike Make sources, it does not transfer to multiple components or consider BillOfMaterial records. Instead, supply of the part at the consuming site simply creates a demand for the part at the supplying site. Thus, the expectation is if the consuming site has a need for the part, it will obtain or source what it needs from the supplying site. **Default value:** Make	String (Enum)	
Value	Specifies the text string for this type of supply. This is what appears in records with a SupplyType field. This text string is assigned to each **Order.Id** in the SupplyOrder and ScheduledReceipt tables. **Default value**: blank string	String	Yes
PartSource Types_ PlannedOrder	Set of **PartSourceType** records that reference this SupplyType value through their **PlannedOrderSupplyStatus** field.	Set	

Table 7.61: SupplyType (Mfg) fields

Field name	Description	Data type	Key
PartSource Types_Planned OrderToSR	Set of **PartSourceType** records that reference this SupplyType value through their **PlannedOrderToSRSupplyStatus** field.	Set	
PartSource Types_ SupplyOrders	Set of **SupplyOrder** records that reference this SupplyType value.	Set	

TaskType table

The **TaskType** table contains configurable settings that define processing and calculations at the project level. For example, this table specifies the task's working calendar, and is also used to specify the rules that control how penalties are calculated at the task level.

This table's records can be edited using the **Task Type** worksheet in the **Control Tables for Project Management** workbook included with RapidResponse.

TaskType (ProjMgmt) fields

Field name	Description	Data type	Key
Bonus Accumulation Rule	If there are multiple **BonusScheduleByDate** or **BonusScheduleByInterval** records effective within the bonus period (between the task's **CalcFinishDate** and **BonusDate**), this setting determines how the bonus values on those records are used. Valid values are: • **Cumulative**—bonuses are calculated using the **OneTimeBonus** and **PerIntervalBonus** values defined on each effective entry in the bonus period and applied within their effective ranges only. With this setting there is the possibility for PerInterval bonuses that change over time along with multiple OneTime bonuses. • **FirstInterval**—bonuses are calculated using the **OneTimeBonus** and **PerIntervalBonus** values defined on the first effective entry within the bonus period and applied throughout the entire bonus period. This means either the **BonusScheduleByDate** record with the earliest **EffectiveOutDate** within the bonus period or the **BonusScheduleByInterval** with the highest **Interval** value within the bonus period is used. **Default value:** FirstInterval **Note:** For more information about its usage, see "Bonuses" on page 2196.	String (Enum)	
BonusCalendar	The calendar used when calculating bonuses to be applied to a task. For example, should bonus revenue be earned on a daily or weekly basis. **Referenced table:** Calendar (nullable)	Reference	
BonusRule	Specifies the schedule, if any, to use for calculating task bonuses. This defines the one-time and per interval bonuses that can be earned and applied between the task's calculated finish date and bonus date. Valid values are: • **Date**—bonuses are applied based on dates and values defined in the **BonusScheduleByDate** table. The entries that apply to a given task are set through its **BonusSchedule** reference. • **Interval**—bonus values are applied based on bonus calendar intervals and values defined in the **BonusScheduleByInterval** table. The entries that apply to a given task are set through its **BonusSchedule** reference. • **None**—bonuses cannot be earned by tasks of this type. **Default value:** None **Note:** For more information, see "Bonuses" on page 2196.	String (Enum)	

Field name	Description	Data type	Key
Calendar	The calendar to use when calculating start and finish dates for tasks of this type. If no calendar is specified, then **ProjectType.Calendar** is used instead. **Referenced table:** Calendar (nullable)	Reference	
ControlSet	The control set associated with this **TaskType** record. **Referenced table:** ControlSet	Reference	Yes
Description	A description of this task type.	String	
EfficiencyCurve Rule	Specifies the efficiency curve, if any, applied to resources when assigned to tasks of this type. Efficiency curves are sets of points used to model varying rates of resource efficiency during the completion of a task. For example, they might be used to model tasks on which resources have a lower rate of efficiency initially but then ramp up towards full efficiency as the task progresses. As such, efficiency curves impact values in the **CalcDuration** and **TargetWorkComplete** fields on the **Task** table. Valid values are: • **None**— No efficiency curve is used (a constant rate of efficiency is assumed). • **PointsCurve**—the efficiency curve referenced by the **Task.PointsCurve** value is used. • **Ramp**—a linear efficiency curve is built and used based on the value provided in **Task.Ramp**. **Default value:** None For more information about how this field is used, see "Leveling task resources" on page 2186.	String (Enum)	
FixedCost AccrualRule	Determines when the **Resource.FixedCost** value associated with using a resource is reported on resource loads generated from tasks of this type. This impacts the distribution of fixed cost values as seen in the **ResourceLoad.FixedCost** field. Valid values are: • **Finish**—the entire fixed cost is reported on the date of the last resource load generated from a given task. • **Prorated**—the fixed cost is spread evenly between all dates on which any resource load is generated from a given task. • **Start**—the entire fixed cost is reported on the date of the first resource load generated from a given task. **Default value:** Start	String (Enum)	

Field name	Description	Data type	Key
LevelResources	Indicates if resources can be assigned to a given task in excess of the effective maximum units defined on the resource (from either the **Resource** or **ResourceAvailable** tables). This setting impacts calculated task durations and associated resource loads in cases where a resource is over allocated to a task. Valid values are: • **N**—overloaded resources on a given task are not leveled. The calculated resource loads and task durations ignore resource availability as defined by the **MaximumUnits** field on either the **Resource** or **ResourceAvailable** tables. For example, assume the **ResourceAvailable** table defines a resource as having no units available on a given date where it is assigned to a task. With this setting, load is still assigned to the resource for that date (the resource is seen as overloaded) and the calculated task duration is not impacted. • **Y**—overloaded resources on a given task are leveled. The calculated resource loads and task durations respect resource availabilities as defined by the **MaximumUnits** field on either the **Resource** or **ResourceAvailable** tables. For example, assume the **ResourceAvailable** table defines a resource as being on vacation and having no units available on a given date where it is assigned to a task. With this setting, no load is assigned to the resource for that date (the loads moves to the next date where the resource is seen as available) and the calculated task duration is extended accordingly. **Default value:** N **Note:** For tasks where **Task.DurationType** is set to "FixedDuration", this field is always treated as "N". For more information about how this field is used, see "Leveling task resources" on page 2186.	Boolean	

Field name	Description	Data type	Key
Penalty Accumulation Rule	If there are multiple effective penalty costs defined in either the **PenaltyScheduleByDate or PenaltyScheduleByInterval** table, this setting defines how those costs are applied to tasks throughout the penalty schedule. Valid values are: • **Cumulative**—penalty costs are calculated using the OneTime and PerInterval costs defined on each effective entry on the penalty schedule. This setting allows for PerInterval costs that change over time along with multiple OneTime charges. • **LastInterval**—penalty costs are calculated using the OneTime and PerInterval costs defined on only the last effective entry on the penalty schedule. This setting uses a constant PerInterval cost along with a single OneTime charge. **Default value:** LastInterval For more information about how this field is used, see "Penalties" on page 2191.	String (Enum)	
PenaltyCalendar	The calendar used when calculating penalties applied to a task. For example, should penalty costs be incurred on a daily or weekly basis. **Referenced table:** Calendar (nullable)	Reference	
PenaltyRule	Specifies the schedule, if any, to use for calculating task penalty costs. This defines the one-time and per interval penalty costs that can be incurred and applied between the task penalty date and calculated finish date. Valid values are: • **Date**—penalty values are applied by dates defined in the **PenaltyScheduleByDate** table. The entries that apply to a given task are set through its **PenaltySchedule** reference. • **Interval**—penalty values are applied by calendar intervals defined in the **ProjectScheduleByInterval** table. The entries that apply to a given task are set through its **PenaltySchedule** reference. • **None**—penalties are not are calculated for projects of this type. **Default value:** None For more information about how this field is used, see "Penalties" on page 2191.	String (Enum)	
Value	A unique string identifier for this type of project. This value is referenced from the **Task.Type** field.	String	Yes
Tasks	Set of **Task** records that reference this TaskType value.	Set	

ToleranceProfile table

The ToleranceProfile table identifies each tolerance profile along with the baseline and tolerance calendar used by that profile.

Table 7.62: ToleranceProfile (Mfg) fields

Field name	Description	Data type	Key
BaselineCalendar	The calendar used in identifying the baseline historical demand or supply series. If the AsOfDate value in the HistoricalDemandSeries or HistoricalSupplySeries table belongs to this calendar, than that series is considered a baseline series. **Referenced table**: Calendar	Reference	
ControlSet	The control set associated with this ToleranceProfile record. **Referenced table:** ControlSet	Reference	Yes
Description	An optional description of this tolerance profile.	String	
Tolerance Calendar	The calendar used to calculate tolerance zones used when evaluating the level of forecast change. **Referenced table**: Calendar	Reference	
Value	A unique value used to identify this tolerance profile.	String	Yes
PartCustomers	The set of **PartCustomer** records that reference this ToleranceProfile value.	Set	
PartSuppliers	The set of **PartSupplier** records that reference this ToleranceProfile value.	Set	
Zones	The set of **ToleranceProfileZone** records that reference this ToleranceProfile value.	Set	

ToleranceProfileZone table

The ToleranceProfileZone table defines properties of the tolerance profiles used for historical data analysis. For example, it defines the acceptable increases and decreases allowed when comparing updated data against a baseline historical demand or supply series.

Table 7.63: ToleranceProfileZone (Mfg) fields

Field name	Description	Data type	Key
ControlSet	The control set associated with this ToleranceProfileZone record. **Referenced table:** ControlSet	Reference	Yes
Downside Percentage	The percentage decrease in historical demands or supplies that is acceptable within this zone. A negative value (for example, -1) implies that there is no limit.	Quantity	
ToleranceProfile	A reference to the tolerance profile that this zone belongs to. **Referenced table**: ToleranceProfile	Reference	
Upside Percentage	The percentage increase in historical demands or supplies that is acceptable within this zone. A negative value (for example, -1) implies that there is no limit.	Quantity	
ZoneEndInterval	The number of intervals in the tolerance calendar from the start date to end of this zone. For independent demands, start date is Part.PlanningCalendars.RunDate.FirstDate. For historical demands or historical supplies, start date is the immediately preceding baseline series' AsOfDate. If this value is negative or zero, then the entire entry is ignored. After the last ZoneEndInterval, any amount of increase or decrease is considered acceptable.	Integer	

UnitOfMeasure table

The **UnitOfMeasure** table determines all valid unit of measure codes. By entering records into this table, you can define a relative ratio between two units.

When calculating a statistical forecast, quantities are calculated using **UnitOfMeasure.BaseConversion**. However, a part could have a specific unit of measure conversion rate defined in the **PartUOMConversion** table. Therefore, **UnitOfMeasure.BaseConversion** is ignored if a part has a unit of measure conversion record in the **PartUOMConversion** table.

If **PartUOMConversion** records exist, **UnitOfMeasure.BaseConversion** is ignored when calculating quantities in the following tables:

- **ForecastItemParametersActual**
- **StatisticalForecast**
- **StatisticalForecastDetail**
- **StatisticalForecastDisaggregationRate**

Note: For more information about the PartUOMConversion table, see "PartUOMConversion table" on page 478.

Table 7.64: UnitOfMeasure (Mfg) fields

Field name	Description	Data Type	Key
BaseConversion	Factor to convert quantities in this unit to the base unit. If BaseConversion is less than or equal to zero (<= 0) for any UnitOfMeasure used when converting quantities, no conversion is performed. This avoids divide-by-zero problems. The following formula is used to convert a quantity from **Part.UnitOfMeasure** to a target **UnitOfMeasure**: $\frac{Quantity * Target.UnitOfMeasure.BaseConversion}{Part.UnitOfMeasure.BaseConversion}$	Quantity	
Category	The category associated with this UnitOfMeasure record. **Referenced table:** UnitOfMeasureCategory **Note:** : This field was added in Version 2016.2, Service Update 12.	Reference	
ControlSet	The control set associated with this UnitOfMeasure record. **Referenced table:** ControlSet	Reference	Yes
Description	Additional information about the units of measure or currency. **Default value**: blank string	String	
Rounding Precision	Indicates how much effective quantities on scheduled receipts and planned orders are rounded (this field is applicable only when **OrderPolicy.YieldUsage** is set to an option other than "Ignore" and **OrderPolicy.YieldRoundingUsage** is set to "Use"). After applying the yield factor, the EffQuantity field calculated on scheduled receipts and planned orders for a given part is rounded down to the number of decimal places specified in this field. If only whole order quantities are required, this value should be set to 0. Any value less than 0 is interpreted as 0, and any value greater than 9 is interpreted as 9. **Default value**: 9	Integer	
Value	Unique code for units of measure or currency. **Default value**: EA	String	Yes
ConstraintUnitOfMeasures	The set of **ConstraintUnitOfMeasures** records that reference this UnitOfMeasure value.	Set	
CRPUnitOfMeasures	The set of **CRPUnitOfMeasure** records that reference this UnitOfMeasure value.	Set	

Table 7.64: UnitOfMeasure (Mfg) fields

Field name	Description	Data Type	Key
CurveParameters	The set of **CurveParameter** records that reference this UnitOfMeasure value.	Set	
ForecastItemParameterUOMs	The set of **ForecastItemParameterUOM** records that reference this UnitOfMeasure value.	Set	
LocationCapacities	The set of **LocationCapacity** records that reference this UnitOfMeasure value.	Set	
Parts	The set of **Part** records that reference this UnitOfMeasure value.	Set	
PartSources	The set of **PartSource** records that reference this UnitOfMeasure value.	Set	
PartSuppliers	The set of **PartSupplier** records that reference this UnitOfMeasure value.	Set	
UOMConversions	The set of **PartUOMConversion** records that reference this UnitOfMeasure value.	Set	

Sample records

Assume the following records exist in this table.

Table 7.65: Example of a UnitOfMeasure record

Value	Description	BaseConversion
LB	Pound	1
Ton	Ton	2000

A planned requirement on a part for Quantity of 2000 (Stocking) is considered as an EffQuantity of 1 (purchasing). A purchase order with a Quantity of 1 is considered as an EffQuantity of 2000.

WorkCenterType table

This table identifies the switches and default capacity values for each type of work center.

Table 7.66: WorkCenterType (Mfg) fields

Field name	Description	Data type	Key
Value	The string code for this type of work center. This is what appears in the WorkCenter records. **Default value**: blank string	String	Yes
CapacityMethod	Specifies how to report the machine and labor capacity fields on the daily and weekly work center load records. Valid values are: • **HoursPerDay** — daily/weekly summary of reported capacity for this work center is: *HoursPerDay*NumberOfResource*Efficiency*Utilization*RunRatio* • **DemonstratedCapacity** — daily/weekly summary of reported capacity for this work center will use the DemonstratedCapacity provided. Operation overrides are ignored. • **MaximumCapacity** — daily/weekly summary of reported capacity for this work center will use the MaximumCapacity provided. Operation overrides are ignored. **Default value**: HoursPerDay	Enum	
CapacityReport Limit	Used to define the end of the reporting period for certain work-center based calculations. Represents a number of **PlanningCalendars.TimeUnits** after **RunDate** for reporting the following: • The last date used to include activity for consideration when calculating Capacity-based metrics in RapidResponse. It is recommended this value be the same for all WorkCenterTypes, in all scenarios. Metric results become confusing if the value is different between scenarios. • The latest possible date for reporting effective work center capacity values in the **WorkCenterCapacity** table. • Records showing Capacity, but not Load, in various calculated tables. Not that records are always created up to the last date on which there is Load on the work center. In addition, however, records showing Capacity (and no Load) will be created for every subsequent date up to this limit. **Default value**: 120	Quantity	
ControlSet	The control set associated with this WorkCenterType record. **Referenced table:** ControlSet	Reference	Yes
DefaultHoursPer Day	The total number of working hours in a day at this work center. Only used if there are no capacity records for the work center. **Default value**: 8	Quantity	

Table 7.66: WorkCenterType (Mfg) fields

Field name	Description	Data type	Key
DefaultLabor RunRatio	The number of hours of labor required for one standard hour of run time. This factor is used to convert work center run time load to a labor load. This value is used if there is no effective capacity record for the work center. **Default value**: 1	Quantity	
DefaultLabor SetupRatio	The number of hours of labor required for one standard hour of setup. This factor is used to convert work center setup time load to a labor load. This value is used if there is no effective capacity record for the work center. **Default value**: 1	Quantity	
DefaultMachine RunRatio	The number of hours of machine time required for one standard hour of run time. This factor is used to convert work center run time load to a machine load. This value is used if there is no effective capacity record for the work center. **Default value**: 1	Quantity	
DefaultMachine SetupRatio	The number of hours of machine time required for one standard hour of setup. This factor is used to convert work center setup time load to a machine load. This value is used if there is no effective capacity record for the work center. **Default value**: 1	Quantity	
DefaultNumber OfResources	The number of parallel resources available for scheduling all operations using this work center if there is no applicable Capacity record. This value must be non-negative. This value is used if there is no effective capacity record for the work center. **Default value**: 1	Quantity	
DefaultQueue Time	The pre-operation queue time used for scheduling all operations for this work center unless overridden (see Operation.QueueTimeOverride). This value is the number of hours to wait before setup can begin and must be greater than or equal to zero. Only used if there are no capacity records for the work center. **Default value**: 0	Quantity	
DefaultWaitTime	The post-operation wait time used for scheduling all operations for this work center unless overridden (see Operation.WaitTimeOverride). This value is the number of hours to wait after run time processing ends and must be greater than or equal to zero. Only used if there are no capacity records for the work center. **Default value**: 0	Quantity	
Description	A description of this type of work center. An explanation of the code in the Value field. **Default value**: blank string	String	

Table 7.66: WorkCenterType (Mfg) fields

Field name	Description	Data type	Key
ProcessingRule	Valid values are: • **Ignore** — no information about this work center will be generated on the Load table for any work center referencing this WorkCenterType • **OutsideProcessing** — no information is generated on the Load table but operations around this work center will be scheduled taking into account this operation's duration • **CRP** — generates all information for the Load table **Default value**: CRP	String (Enum)	
WorkCenters	The set of **WorkCenter** records that reference this WorkCenterType value.	Set	

CHAPTER 8: Calculated tables

RapidResponse calculated tables are not editable and hold records generated by RapidResponse calculations. For example, each time RapidResponse recommends a new planned order, RapidResponse creates a new record in the **PlannedOrder** table holding basic details of the order recommended by the netting analytic and a corresponding record in the **CTPPlannedOrder** table reporting order availability details as calculated by the capable-to-promise analytic.

This chapter describes each calculated table in the RapidResponse data model and contains reference information on each field within those tables. Note that a given calculated table might typically have many calculated reference fields that point to other tables that can include multiple references to the same table. However, each table has only one "owner" reference field as noted within the appropriate field descriptions in this chapter. The "owner" reference identifies the specific input table item from which the underlying analytic calculations being reported are triggered. For example, the **WhereConsumedForDemand** table is used to report supplies pegged to a particular independent demand, and its owner is **IndependentDemand**. When enabling item controls on calculated table worksheets, filtering should typically be applied through the "owner" reference in order to limit the number of records that need to be processed by underlying analytic calculations and hence reduce worksheet load times. For more information about filtering and owner references, see the the *RapidResponse Resource Authoring Guide*.

In this chapter, you'll learn about the following:

Activity table **940**
Tables referenced by the Activity table **951**
AvailableToPromise table **956**
BillOfMaterialFeatureRule table **957**
BlowThroughNewAllocation table **958**
BlowThroughPlannedAllocation table **959**
BOM_MUECUMLead table **960**
CoByProductSupply table **961**
Conflict table **963**
ConsensusForecast table **966**
ConsensusForecastDetail table **970**
ConsensusForecastRollingHorizonWeight table **971**
ConsensusForecastWeightByHeader table **972**
ConstraintLoad table **973**
ConstraintSpreadLoad table **974**
ConstraintUsed table **975**
CostOfGoodsSold table **977**
CostRollUp table **982**
CriticalPath table **984**

CTPActivity table 985
CTPCoByProductSupply table 996
CTPForecast table 998
CTPForecastCost table 999
CTPPlannedOrder table 1000
CTPSafetyStockHoldings table 1002
CTPSubstituteSupply table 1003
DataChange table 1005
DataChange_Historical table 1008
DataChange_Historical_Split table 1011
DataChange_Local table 1014
DataChange_Local_Split table 1016
DataChange_PendingUpdate table 1019
DataChange_PendingUpdate_Split table 1022
Demand table 1025
DemandException table 1028
DependentConsensusForecast table 1032
DependentDemand table 1033
DirectAssembly table 1036
DirectComponent table 1036
DisaggregationRateByPartCustomer table 1037
EventConsensusForecastDetail table 1039
EventDisaggregationRate table 1041
EventForecastDetailAdjustment table 1041
EventStatisticalDisaggregationRate table 1043
EventStatisticalForecastDetail table 1044
EventStatisticalForecastDetailAdjustment table 1044
FeatureBOMForIndependentDemand table 1045
FlatAssembly table 1046
FlatBillDown table 1046
FlatBillUp table 1051
FlatComponent table 1054
FlatDemand table 1055
FlatConstraint table 1058
FLPConstraint table 1058
Forecast table 1059
ForecastConsumption table 1063
ForecastItemParametersActual table 1065
ForecastItemParametersRegressionWeight table 1066
ForecastItemParametersRegressor table 1067
HistoricalDemandWaterfall table 1068
HistoricalSupplyWaterfall table 1070
IndependentDemandByFeature table 1072
IndependentDemandCost table 1073
IndependentDemandFeature table 1074
IndependentDemandFeatureSummary table 1075
InProcess table 1075
InventoryAnalysis table 1078
InventoryCTPAnalysis table 1081
InventorySummary table 1084
ItemSourceCandidate table 1087
ItemSourceIntervalEffectivePartSource table 1088
ItemSourcePartSourceEffectivityInterval table 1089
ItemSourcePartSourceHistoricalLeadTime table 1089

ItemSourcePartSourceHistoricalReceipt table 1091
LateSupply table 1092
LoadDailyCurrent table 1097
LoadDailyPlanned table 1098
LoadDetailCurrent table 1100
LoadDetailPlanned table 1102
LoadFLPPlanned table 1104
LoadOperationNewOrderPlanned table 1106
LoadOperationReceiptsCurrent table 1108
LoadOperationReceiptsPlanned table 1110
LoadWeeklyCurrent table 1112
LoadWeeklyPlanned table 1114
MaximumInventoryResult table 1115
MEIODriverPart table 1116
MEIOFamilyResult table 1117
MEIOLeadTimeOutlierResult table 1117
MEIOLeadTimeOutlierResultSummary table 1119
MEIOHistoricalSupply table 1121
MEIOParentPart table 1123
MEIOStage table 1123
MEIOStageHistoricalDemand table 1127
MEIOStageHistoricalForecast table 1127
MEIOStageLink table 1128
MEIOStageResult table 1128
MEIOStageTimePhasedResult table 1130
MLSActivity table 1131
NewAllocation table 1137
NewTransferAllocation table 1138
Numbers table 1138
Part_MUECumLead table 1139
Part_MUEPoolValidation table 1140
PartSourceCTPPlannedOrder table 1141
PartSourcePlannedOrder table 1141
PartSourceScheduledReceipt table 1142
PeriodConstraintLoad table 1142
PeriodsOfCoverage table 1144
PeriodsOfCTPCoverage table 1145
PlannedAllocation table 1147
PlannedOrder table 1148
PlannedTransferAllocation table 1154
ProductionCycle table 1155
ResourceDataByDate table 1156
ResourceLoad table 1157
SafetyStockHistoricalDemand table 1161
SafetyStockHistoricalForecast table 1162
SafetyStockHistoricalSupply table 1163
SafetyStockItemDemandParameterSet table 1164
SafetyStockItemFutureDemand table 1165
SafetyStockItemOutlierResult table 1166
SafetyStockItemOutlierResultSummary table 1167
SafetyStockResult table 1169
SafetyStockTimePhasedResult table 1174
ScheduledReceiptCost table 1178
StatisticalForecast table 1179

StatisticalForecastDetail table **1181**
StatisticalForecastDisaggregationRate table **1181**
StatisticalForecastFitOutput table **1182**
StatisticalForecastOutlier table **1192**
StatisticalForecastOutlierSummary table **1194**
StatisticalForecastPredictActual table **1196**
StatisticalForecastRegressionCoefficient table **1197**
StatisticalForecastResidualAutocorrelation table **1198**
StatisticalForecastResidualCount table **1199**
SubstituteSupply table **1200**
Supply table **1202**
SupplyDemand table **1205**
TaskDemandLinkCost table **1215**
TaskPartLinkCost table **1215**
TaskSupplyLinkCost table **1216**
TrialCoByProductSupply table **1216**
TrialConstraintShortage table **1218**
TrialConsumption table **1219**
TrialPlannedOrder table **1220**
TrialSubstituteSupply table **1222**
ValidatePlannedOrder table **1224**
WhereConsumed table **1225**
WhereConsumedForDemand table **1234**
WhereConsumedForForecast table **1243**
WhereConsumedForSupply table **1252**
WhereConsumedToHighVolumeOrder table **1262**
WorkCenterCapacity table **1265**

Note: RapidResponse also contains a number of fields that are populated with calculated data in Input tables. For more information, see "Input tables" on page 171.

Activity table

RapidResponse includes worksheets based on the **Activity** table that combine all OnHand, Supply, and Demand information. These worksheets can be used for projecting inventory balance or generating a complete transaction event report.

The **Activity** table only presents events or activities that are processed by netting (for example, where control table values are not set to ignore it). The description of the **Activity** table is divided between two tables:

- A description of the "Activity (Mfg) fields" on page 942.
- The tables and associated values that the **Activity** table references. See "Tables referenced by the Activity table" on page 951.

Notes:

- If interested in activity type information pertaining to capable-to-promise calculations, see "CTPActivity table" on page 985.

- By default, inactive IndependentDemand records are not included in the **Activity** table. However, these records can be included, as discussed in "Include inactive IndependentDemands in the Activity and CTPActivity tables" on page 2210.

Generation of TimeUnit records

RapidResponse creates Activity records when there is a transaction to report. For example, if an unsatisfied demand is satisfied with available inventory, two Activity records are created. One that shows the demand and the other showing the supply.

Activity records can also be generated based on time buckets. This type of Activity record is called a TimeUnit record. The value of the **Type** field is **TimeUnit**. TimeUnit records are only created if either an ActivityBeat record or an ActivityBeatBeforeRunDate record exists in the **Configuration** table. For information about setting up Configuration records, see "Calculating TimeUnit records" on page 2203.

If an **ActivityBeat** record exists with a value greater or equal to **0**, then the **Activity** table includes TimeUnit records on and after part RunDate. A value of **0** results in one TimeUnit record (or more if multiple model, unit, pool combinations) for the RunDate if there is a CalendarDate entry for that date.

If an **ActivityBeatBeforeRunDate** record exists and has a value greater or equal to **1**, then a TimeUnit record includes records before part RunDate.

TimeUnit records are generated for each Model, Unit, Pool combination netted by the Part.TimeUnit records are also generated for every CalendarDate entry for the **Part.PlanningCalendars.TimeUnits** calendar where:

```
(Part.PlanningCalendars.RunDate.FirstDate - Configuration
[ID=ActivityBeatBeforeRunDate].Value) <= CalendarDate.Value and
(Part.PlanningCalendars.RunDate.FirstDate > CalendarDate.Value)
```

or

```
(Part.PlanningCalendars.RunDate.FirstDate + Configuration
[ID=ActivityBeat].Value) >= CalendarDate.Value and
(Part.PlanningCalendars.RunDate.FirstDate <= CalendarDate.Value)
```

TimeUnit records are never generated for Past or Future date values. The following table shows an example of a TimeUnit record.

Table 8.1: Example of a TimeUnit record

Field	Value
Type	TimeUnit
Part	The part for this record
Model, PegModel	The Model used for netting
Pool, PegPool	The Pool used for netting
StartUnit	The Unit used for netting

Table 8.1: Example of a TimeUnit record

Field	Value
Allocation, Constraint, CTPPlannedOrder, Forecast, IndependentDemand, OnHand, PartSource, PegPart, PlannedOrder, ScheduledReceipt, Supplier	NULL reference
AssociatedDate, AvailableDate, AvailableStartDate DemandDate, DockDate, DueDate, RecommendedDate, ReschedDate, ShipDate, StockDate, SynchronizedDate	The TimeUnit date
AssociatedQty, ATPQuantity, BalanceDelta DemandQty, NewProcessCost, NewPurchasesCost, OnTimeQuantity, SupplyQty	0.0
DemandOrder	<blank>
DemandType (same as for OnHand)	NULL enum
OrderPriority (same as for OnHand)	NULL enum
SupplyType (same as for OnHand)	NULL enum
TransactionSequence	0

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
AdjustedDemand Date	For demands, this returns the DemandDate less the part's safety lead time (note that any safety lead time at the part source level is not included). For supplies, thisis the same as RecommendedDate (which considers safety lead time from both the part and part source). **Note:** This field was added in Version 2016.2.	Date	
Allocation	The Allocation record this entry relates to. **Referenced table:** Allocation	Reference	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The part the supply or demand activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply or demand data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, and this record pertains to a supply, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. Otherwise, if this record pertains to a demand, then Part is the part that needed the supply and AlternatePart is the part where the demand came from. **Referenced table**: Part	Reference	
AssociatedDate	The PromiseDate for demands, CalcStartDate for supply, RunDate for on hand.	Date	
AssociatedQty	If the **Type** field is 'DynamicSafety', this is the difference between the revised safety stock quantity and the previous safety stock quantity. Note that records of this type are only generated on dates when there is a dynamic or time-phased safety stock value calculated (or provided) and that value differs from the previous safety stock quantity. If the **Type** field is 'AvailableToPromise', this is **AvailableToPromise.Quantity**. For all **Type** values other than 'DynamicSafety' or 'AvailableToPromise', this field returns 0.		
BalanceDelta	The change in the inventory level due to the original quantity of the activity.	Quantity	
BillOfMaterial	The BOM record that corresponds to NewAllocation or PlannedAllocation records. **Referenced table:** BillOfMaterial	Reference	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
CoByProductSupply	If this record represents supply generated for a co-product or by-product, this field is a reference to the associated CoByProductSupply record (otherwise, this field is NULL). If necessary, the **BillOfMaterial.Type.Product** reference can be used to determine is this record represents supply of a co-product or a by-product. **Referenced table:** CoByProductSupply	Reference	
DemandDate	For supplies, date of the earliest demand matched by netting. For demands, ReschedDate.	Date	
DemandException	If this record represents a demand that was skipped because no planned order could be generated to meet its expiry requirements, this field is a reference to the associated DemandException record. This reference gives additional details on the skipped demand, such as the earliest date that any supply could expire if it were to be used in satisfying that demand. **Referenced table:** DemandException	Reference	
DemandOrder	The order number for independent demand activities.	String	
DemandQty	For demands, the total quantity of the demand, including ShippedQty (SupplyQty will be the shipped quantity or consumed forecast). For supplies, the quantity that has been received. • If Type = 'DynamicSafety', then this field represents the revised safety stock quantity. • If Type = 'AvailableToPromise', then this field is set to AvailableToPromise.SRQuantity.	Quantity	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
DemandType	The DemandType associated with this record. Valid values are: • **OnHand**—first value • **ScheduledReceipt**—first value • **PlannedOrder**—PartSource.Type. PlannedOrderSupplyType.AllocationDemand • **IndependentDemand**—Type • **Allocation**—SRAllocType • **NewAllocation**—SRAllocType • **NewTransferAllocation**—SRAllocType • **PlannedAllocation**—PartSource.Type. PlannedOrderSupplyType.AllocationDemandType • **PlannedTransferAllocation**—PartSource.Type. PlannedOrderSupplyType.AllocationDemand • **Forecast**—type of first Forecast record in bucket. • **AvailableToPromise**—first value **Referenced table:** DemandType	Reference	
DockDate	Date the transaction should be delivered to the receiving dock (calculated from ReschedDate). ReschedDockDate for scheduled receipts. Undefined unless planned order or scheduled receipt.		
DueDate	The date this supply (if it is a supply) is supposed to be available for use (EffDueDate). For demands, the date the demand is needed (based on parent supply ReschedDate if dependent - Allocation.ReschedDate, NewAllocation.DueDate).	Date	
EarliestExpiryDate	The earliest date that supply used to satisfy the demand associated with this record may expire.	Date	
EstimatedExpiry Date	If this record pertains to a supply, this is the date that the Netting analytic calculates that supply will expire. If the supply comes from on hand inventory or an inventory transfer then the supply's **ExpiryDate** value is reported. For all other types of supply, the supply's **EstimatedExpiryDate** value is reported.	Date	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
ExtendedType	Indicates whether this record is associated with an unsourceable demand due to material expiry or is a normal Activity record. Valid values are: • **DemandException**— a record representing an demand that was skipped because supply could not be planned to meet its expiry requirements. Typically, this occurs when RapidResponse is unable to generate a planned order with an expiry date on or after the demand's earliest expiry date. For more information, see "Unsourceable demands and the Activity/CTPActivity tables" on page 1626. • **Normal**—a normal Activity record (does not represent an unsourceable demand due to expiry).	String (Enum)	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
Forecast	The Forecast record associated with this demand (if Activity.Type = 'Forecast'). **Referenced table:** Forecast	Reference	
Independent Demand	Reference to the source of demand. Valid if forecast is placed as Independent Demand on this part. NULL if demand is not from an IndependentDemand record (for example, all dependent demand and ATP). **Referenced table:** IndependentDemand	Reference	
InventoryTransfer	The InventoryTransfer record associated with this Activity record. Valid only if Type is 'InventoryTransfer' or 'InventoryTransferAllocation'. **Referenced table:** InventoryTransfer	Reference	
Model	The model used when netting this record. **Referenced table:** Model	Reference	
OnHand	The OnHand record associated with this record. Valid only if Type is 'OnHand'. **Referenced table:** OnHand	Reference	
OrderPriority	The priority associated with this supply or demand. (for example, Part.Type.DefaultPriority for OnHand). **Referenced table:** OrderPriority	Reference	
Part	The part. **Referenced table:** Part	Reference	Yes

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
PartSource	The PartSource from ScheduledReceipt or PlannedOrder. Null if the Activity record does not originate from a ScheduledReceipt or PlannedOrder. **Referenced table:** PartSource	Reference	
PegModel	The model that is applicable to this supply or demand. The field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.ModelRule. **Referenced table:** Model	Reference	
PegPart	The source of demand if the record describes a dependent demand. This value is NULL for supply records (planned order, scheduled receipt and on hand) as well as independent demand, forecast, and ATP. **Referenced table:** Part	Reference	
PegPool	The pool that is applicable to this supply or demand. This field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.PoolRule. **Referenced table:** Pool	Reference	
PlannedOrder	The planned order, if any associated with this record: • If this record is a planned order supply for this part, the planned order supply. • If this record is dependent demand from a planned order, the parent (peg) planned order. • Otherwise Null. **Referenced table:** PlannedOrder	Reference	
Pool	This value represents the mapped pool used to satisfy demand. **Referenced table:** Pool	Reference	
Recommended Date	The recommended due date (stock date) for the supply. Same as DueDate if a demand.	Date	
ReschedDate	The revised due date (stock date) after rescheduling the supply. Based on its parent supply's ReschedDate (same as DueDate if a demand).	Date	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
ScheduledReceipt	If this record represents a supply from a ScheduledReceipt, then this field is a reference to that ScheduledReceipt. If this record represents a demand that is an allocation from a ScheduledReceipt, then this field is a reference to that ScheduledReceipt. In all other situations, this field is NULL. **Referenced table:** ScheduledReceipt	Reference	
ShipDate	The date the supply should be shipped from its supplier to meet ReschedDate. Copied from PlannedOrder or ScheduledReceipt. D'undefined' for other types of Activity records.	Date	
StartUnit	The start unit that is applicable to this supply or demand. The field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.UnitRule.	Integer	
StockUnitQuantity	Represents the supply quantity in inventory unit of measure (UOM) without applying yield or scrap factors. **Note**: **SupplierUOM** applies to **StockUnitQuantity**.	Quantity	
SubstituteSupply	Reference to the substitute supply associated with this record. **Referenced table:** SubstituteSupply	Reference	
Supplier	Reference to the name of the supplier. The supplier of the part. NULL if the supply is from on hand, or if the activity is a demand. **Referenced table:** Supplier	Reference	
SupplyOrder	The order number of ScheduledReceipt activities or the order number from scheduled receipts associated with allocations or new allocations.	String	
SupplyQty	For supplies, the total quantity of the supply (DemandQty will be the received quantity). For demands, the quantity that has been satisfied (shipped or consumed forecast). This quantity is reported in supplier units, and includes inflation for yield. If Type = 'AvailableToPromise', then this field is set to AvailableToPromise.SRQuantity.	Quantity	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
SupplyQtyUnitOfMeasure	Reference to the unit of measure used for the supply quantity that is associated with this record. **Referenced table:** "UnitOfMeasure table" on page 931. **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
SupplyType	The SupplyType associated with this record. If this record represents a demand record, RapidResponse assigns a blank string to this field. Otherwise, valid values are: • **OnHand**—first value • **ScheduledReceipt—**Type • **PlannedOrder**—PartSource.Type.PlannedOrderSupplyType • **IndependentDemand**—first value • **Allocation**—ScheduledReceipt.Type • **NewAllocation**—ScheduledReceipt.Type • **NewTransferAllocation**—ScheduledReceipt.Type • **PlannedAllocation**—PartSource.Type.PlannedOrderSupplyType • **PlannedTransferAllocation**—PartSource.Type.PlannedOrderSupplyType • **Forecast**—first value • **AvailableToPromise**—first value **Referenced table:** SupplyType	Reference	
Transaction Sequence	Sequence from supply or demand. 0 if not relevant or if there is no corresponding TransactionSequence field in the originating table.	Integer	

Table 8.2: Activity (Mfg) fields

Field name	Description	Data type	Owner
Type	Identifies the type of activity not ignored by netting. Valid values are: • **Allocation** • **AvailableToPromise** • **CoByProductSupply** • **DynamicSafety**(includes any activity from fixed safety stock). • **Forecast** • **IndependentDemand** • **InventoryTransfer** • **InventoryTransferAllocation** • **NewAllocation** • **NewTransferAllocation** • **OnHand** • **PlannedAllocation** • **OrderPointIncrease** • **PlannedTransferAllocation** • **PlannedOrder** • **ScheduledReceipt** • **SubstituteSupply** • **SubstituteAllocation** • **TimeUnit** • **UnplannedSafety** **Default value:** AvailableToPromise	String (Enum)	

Tables referenced by the Activity table

The following table outlines RapidResponse tables referenced by the Activity table. For a complete description of the fields on the Activity table, see the Fields on the "Activity table" on page 940.

Table 8.3: Tables referenced by the Activity table

Field name	OnHand	Scheduled Receipt	PlannedOrder	Independent Demand	Allocation	NewAllocation
Part						
PegPart	NULL	NULL	NULL	NULL	ScheduledReceipt.Part	Part to which this demand goes.
Type	On Hand	ScheduledReceipt	Planned Order	Independent Demand	Allocation	NewAllocation
AssociatedQty	0	0	0	0	0	0
DemandType		ScheduledReceipt.Order	Source	Independent Demand.Order.Type	SRAllocType	SRAllocType
SupplyType		ScheduledReceipt.Order.Type	PartSource.. PlannedOrder SupplyType		Scheduled Receipt.Type	ScheduledReceipt. Order.Type
Due Date	d' Past '	DueDate	DueDate	DueDate	ReschedDate	DueDate
ReschedDate	d'Past'	Resched Date	DueDate	DueDate	ReschedDate	DueDate
Recommended Date	d' Past '	Recommended Date	DueDate	DueDate	ReschedDate	Due Date
AssociatedDate	Run Date	CalcStart Date	Start Date	PromisedDate	Due Date	DueDate
Supply Qty	Quantity	Quantity	Quantity	ShippedQty	Total Quantity – Remaining Quantity	0
DemandQty	0	0	0	Quantity + Shipped Qty	Total Quantity	Quantity

Table 8.3: Tables referenced by the Activity table

Field name	OnHand	Scheduled Receipt	PlannedOrder	Independent Demand	Allocation	NewAllocation
Balance Delta	Quantity	EffQuantity	EffQuantity	Quantity	Remaining Quantity	Quantity
DemandOrder				Order.Id		
Supply Order		Order.Id			Scheduled Receipt. Order.Id	Scheduled Receipt.Order.Id
PegSR	NULL	NULL	NULL	NULL	Scheduled Receipt table from Allocation	Scheduled Receipt table
Scheduled Receipt	NULL	Scheduled Receipt	NULL	NULL	PegSR	NULL
Independent Demand	NULL	NULL	NULL	Independent Demand	NULL	NULL
Supplier	NULL	Scheduled Receipt. Order.Supplier	PartSource. Source.Supplier	NULL	NULL	NULL
Pool	Netted Pool	Pool	Pool	Netted Pool	MappedPool	MappedPool
PegPool	Pool	Order.Pool	Pool	Order. Pool	Scheduled Receipt.Pool	Scheduled Receipt.Pool
Model	Netted Model	Model	Model	Netted Model	Scheduled Receipt.Model	Scheduled Receipt.Model
Peg Model	Model	InputModel	Model	Model	Scheduled Receipt. InputModel	Scheduled Receipt. InputModel
Start Unit	StartUnit	StartUnit	StartUnit	StartUnit	StartUnit	StartUnit
Forecast	NULL	NULL	NULL	NULL	NULL	NULL

The following table displays additional RapidResponse tables referenced by the **Activity** table.

Table 8.4: Tables referenced by the Activity table

Field name	Planned Allocation	Forecast	AvailableTo Promise	NewTransfer Allocation	Planned Transfer Allocation
Part			Part		
PegPart	Part to which demand goes.	NULL	NULL	The Part for which the new transferred allocation exits.	The Part for which the planned transferred allocation exits
Type	Planned Allocation.	Forecast		PegSR.Order. .Allocation DemandType. DemandType	PartSource.Type. PlannedOrder SupplyType. Allocation DemandType. DemandType
AssociatedQty	0	0	Quantity	0	0
DemandType	PartSource.Type. PlannedOrder SupplyTyper	Part Alloc Type		PegSR.Order.Type	PartSource.Type. PlannedOrder SupplyType
SupplyType	PartSource.. PlannedOrder SupplyType	PartPO	Part.Type. Planned Order SupplyType (default)	NewTransfer Allocation	PlannedTransfer Allocation
Due Date	DueDate	Date	DueDate	DueDate	DueDate
ReschedDate	DueDate	Date	DueDate	DueDate	DueDate
Recommended Date	Due Date	Date	DueDate	DueDate	DueDate
AssociatedDate	Due Date	Date	DueDate	DueDate	DueDate
Supply Qty	0	The consumed quantity	SRQuantity	0	0
DemandQty	Quantity	Quantity	SR Quantity	Quantity	Quantity

Table 8.4: Tables referenced by the Activity table

Field name	Planned Allocation	Forecast	AvailableTo Promise	NewTransfer Allocation	Planned Transfer Allocation
Balance Delta	Quantity	Unconsumed Qty	0	Quantity	Quantity
Recommended Balance Delta	Quantity	Unconsumed Qty	0	Quantity	Quantity
DemandOrder					
Supply Order					
PegSR	NULL	NULL	NULL	NULL	NULL
Scheduled Receipt	NULL	NULL	NULL	NULL	NULL
Independent Demand	NULL	NULL	NULL	NULL	NULL
Supplier	NULL	NULL	NULL	NULL	NULL
Pool	Mapped Pool	Netted Pool	Pool	Mapped Pool	Mapped Pool
PegPool	Peg Planned Order. Pool	Original pool of the peg supply (or Independent Demand)	Pool	PegSR. Pool	Peg Planned Order. Pool
Model	Model	Netted Model	Model	PegSR.Model	PegPO.Model
Peg Model	Model	Original model of the peg supply (or Independent Demand)	Model	Model	Model
Start Unit	StartUnit	Start Unit	StartUnit	StartUnit	StartUnit
Forecast	NULL	NULL	NULL	NULL	NULL
DemandDate	DueDate	DueDate	DueDate	DueDate	NULL

Table 8.4: Tables referenced by the Activity table

Field name	Planned Allocation	Forecast	AvailableTo Promise	NewTransfer Allocation	Planned Transfer Allocation
ShipDate	NULL	NULL	NULL	NULL	NULL
DockDate	NULL	NULL	NULL	NULL	NULL

AvailableToPromise table

Available to Promise (ATP) is the uncommitted portion of inventory and production for a part. This usually drives sales quotas and delivery time slots for new sales orders (for example, customer orders).

ATP is calculated by daily (TimeUnits) bucketing of the supply records. The first ATP record is on the RunDate and takes into account inventory and all past due supply, as well as supply on the RunDate. After that, there is an ATP record for any workday (for example, from the TimeUnits calendar) for which there is a supply due, unless the ATP value for that date is zero. ATP is calculated using supply at DueDate.

Actual demands (for example, an independent demand whose type's processing rule is SalesActual or Regular) reduce the supply quantity in the closest, or preceding, non-zero ATP bucket(s). The exception is the first bucket (for example, at RunDate) which can go negative. If there is insufficient supply prior to a demand, the closest later supply is allocated. If demand exceeds supply, the ATP in the RunDate bucket will be negative to reflect the shortfall.

Two ATP quantities are calculated; the SRQuantity is calculated with only the scheduled receipts (for example, no planned orders), and Quantity is based on both scheduled receipts and planned orders.

Note: This table is based on results produced by the Netting analytic.

Table 8.5: AvailableToPromise (Mfg) fields

Field name	Description	Data type	Owner
DueDate	When the supply is available to promise.	Date	
Model	The model that applies to this supply. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.Model Rule is Ignore **Referenced table:** Model	Reference	
Part	The part that is available to promise. **Referenced table:** Part	Reference	Yes
Pool	The pool that applies to this supply. It is set to Unpooled if either of the following conditions exist: • Pool is not enabled • Part.MUEPoolNettingType.Pool Rule is Ignore **Referenced table:** Pool	Reference	

Table 8.5: AvailableToPromise (Mfg) fields

Field name	Description	Data type	Owner
Quantity	The amount of the supply on the due date that is not allocated to actual (real) demand. A negative balance indicates that there is more actual (non-forecast) demand than there is currently open and planned supply (negative balances can only be reported on RunDate).	Quantity	
SRQuantity	Same as Quantity except it only considers Scheduled Receipts as supply. A negative balance indicates that there is more actual (non-forecast) demand than there is currently open supply (negative balances can only be reported on RunDate).	Quantity	
StartUnit	The start unit that applies to this supply. It's set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is Ignore	Integer	

BillOfMaterialFeatureRule table

The **BillOfMaterialFeatureRule** table reports features according to the BOM records where they are used in feature rules. It contains one record for each valid feature included in the **BillOfMaterial.FeatureRule** field on a given record. It also identifies all features that are included in feature rules, but which are not found in the **Feature** table.

Table 8.6: BillOfMaterialFeatureRule (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	A reference to the BOM record on which the feature was included in a feature rule. The **BillOfMaterial.FeatureRule** field can be used to return the actual rule. **Referenced table:** BillOfMaterial	Reference	Yes
Feature	A reference to the name of a feature that was included in a feature rule. Null if the feature name cannot be found in the **Feature.Value** field on any record. **Referenced table:** Feature	Reference	
FeatureGroup Index	Indicates the depth, by position, of the feature within the **BillOfMaterialFeatureRule** field. For example, if the feature name is all that is found in the rule or is included in a simple rule listing potential features separated by "or" clauses, this value reports 0. Conversely, if the feature is contained in the feature rule after two "and" clauses, then this values reports "2".	Integer	

Table 8.6: BillOfMaterialFeatureRule (Mfg) fields

Field name	Description	Data type	Owner
MissingFeatures	A list of all feature names included in the **BillOfMaterial.FeatureRule** field that cannot be found in the **Value** field on any **Feature** record. These might be feature names that were typed incorrectly, or valid feature names that have not yet been added to the **Feature** table. If a feature name is not defined in the **Feature** table, it can still be used by feature BOM logic to determine where dependent demands are generated on configurable end-items, however the feature cannot be used in feature mapping and will not be reported in certain calculated tables.	String	

BlowThroughNewAllocation table

The **BlowThroughNewAllocation** table is a calculated table of the New Allocations for the parts that are bypassing netting at this level (for example, Phantom parts).

Table 8.7: BlowThroughNewAllocation (Mfg) fields

Field name	Description	Data type	Owner
DueDate	Date component is scheduled to be available to the manufacture of the assembly (assembly's StartDate+ BOM offset).	Date	
EarliestExpiryDate	The earliest date that supply can expire if it is to be used in satisfying this demand.	Date	
Model	The model that applies to the allocation. **Referenced table:** Model **Note:** This field was added in Version 2016.2.	Reference	
Part	The Part. **Referenced table:** Part	Reference	Yes
Quantity	Quantity of component needed.	Quantity	
ScheduledReceipt	Reference to the supply record that was exploded from. This is pegging. You can get the parent part and order information through this reference. **Referenced table:** ScheduledReceipt	Reference	
StartUnit	The start unit that applies to this allocation. It may differ from the ScheduledReceipt.StartUnit due to demand splitting.	Integer	

BlowThroughPlannedAllocation table

The **BlowThroughPlannedAllocation** table is a calculated table of the Planned Allocations for the parts that are bypassing netting at a given level (for example, Phantom parts).

Table 8.8: BlowThroughPlannedAllocation (Mfg) fields

Field name	Description	Data type	Owner
DueDate	The date that the component is scheduled to be available to the manufacturer of the assembly (assembly's StartDate + BOM offset).	Date	
EarliestExpiry Date	The earliest date that supply can expire if it is to be used in satisfying this demand.	Date	
Model	The model that applies to this allocation. It is set to None if the parent part is not netted by model. **Referenced table:** Model	Reference	
OrderPriority	The priority associated with this demand. **Referenced table:** OrderPriority	Reference	
Part	The Part. **Referenced table:** Part	Reference	Yes
PegDueDate	The date of the planned order or ATP. You can determine the exact planned order using this date and the PegPart (the RapidResponse data model will not generate two planned orders or ATPs on the same day).	Date	
PegPart	Pegging back to the part that the planned order or ATP is on. **Referenced table:** Part	Reference	
PegPartSource	The PartSource record used to create the planned order. PartSource can be used to find PartSource.Supplier and PartSource.SupplierPart (that were formerly found on the Part table.) **Referenced table:** PartSource	Reference	
Pool	The pool that applies to this allocation. It is set to Unpooled if the parent part is not netted by pool. **Referenced table:** Pool	Reference	
Quantity	Quantity of component needed.	Quantity	
Transaction Sequence	Transaction sequence from supply.	Integer	

Table 8.8: BlowThroughPlannedAllocation (Mfg) fields

Field name	Description	Data type	Owner
StartUnit	The start unit that applies to this allocation. It may differ from the PlannedOrder.StartUnit due to demand splitting.	Integer	
Type	The demand type from the PlannedOrder supply type or from a planning bill (see BOMType and PartType control tables). A copy of all the PlannedAllocation records that bypassed the netting of a part because of phantom logic. **Referenced table:** DemandType	Reference	

BOM_MUECUMLead table

The **BOM_MUECUMLead** table supports the Model-Effectivity and Unit-Effectivity analytics. This table is only available for use if you have enabled Model-Unit Effectivity.

If the BOM record is in effect (using date effectivity), this table is populated with the consolidated records of the Component.Part_MUECumLead table. Otherwise, no records are created. Records are consolidated by unique combinations of UseModel (when UseModel = False, then Model will = None), Model, StartUnit and EndUnit.

The Model, FirstEffectiveUnit, LastEffectiveUnit, and Type.MUERule of the BillOfMaterial record and the ModelRule, UnitRule of the Component.MUEPoolNettingType determine which records of the **Component.MUECumLead** are used for the consolidation.

If the BOM is a phantom (**BOMType.BlowThrough** = **Y** or **Component.DependentDemandUsage** = **BlowThrough)**, then the resulting CumLeadTime is adjusted by that of the component.

Table 8.9: BOM_MUECUMLead (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	The BillOfMaterial record for which the cumulative lead time applies. **Referenced table:** BillOfMaterial	Reference	Yes
CumLead	The cumulative lead time for the BillOfMaterial record for this Model and StartUnit / EndUnit range.	Quantity	
EndUnit	The effective ending unit (calculated using the BOMType.EffectivityRule**)**.	Integer	
Model	The model for which the cumulative lead time applies. **Referenced table:** BillOfMaterial	Reference	
MultiSiteCum LeadTime	The multi-site cumulative lead time for the BillOfMaterial record for this Model and StartUnit / EndUnit range.	Quantity	

Table 8.9: BOM_MUECUMLead (Mfg) fields

Field name	Description	Data type	Owner
StartUnit	The effective starting unit (calculated using the BOMType.EffectivityRule).	Integer	
UseModel	Indicates whether the above model is used for netting. Valid values are: • **True**—the model is used for netting • **False**—the model is ignored for the calculation of the CumLeadTime	Boolean	

CoByProductSupply table

The **CoByProductSupply** table reports Netting calculations, such as due dates and quantities, for co-product and by-product supplies that are triggered by production of a primary product.

Co-products are manufactured together with a primary product and typically have similar components and process similarities as the primary product. By-products are generated as a result of, and residual to, production of the primary product but they have some value.

Because they are generated from production of another product, co-product and by-product supplies do not explode to dependent components nor do they consume any constraint. As well, order priority is not reported on these types of supplies as they are assumed to have the same priority as the primary product's supply.

Table 8.10: CoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
BOM	The bill of material record that associates the primary product (assembly) with the co-product or by-product (component) whose supply is being reported here. The **BOM.Type.ProductType** reference can be used to show whether this is a co-product or by-product supply. **Referenced table:** BillOfMaterial	Reference	
DemandDate	The date of the demand matched to this supply by Netting calculations.	Date	
DueDate	The date when this supply is due. This is calculated starting from the DueDate of the primary product supply and adding the LeadTimeOffset value defined on the BOM record that associates the primary product with this co-product or by-product.	Date	

Table 8.10: CoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
EarliestExpiry Date	The earliest date that any supplies used to satisfy dependent demands generated from this supply can expire. Typically, this is determined from the latest **EarliestExpiryDate** found on a demand that Netting calculations determine this supply will satisfy. Otherwise, if there are no consuming demands, this value is calculated from the by adding the number of expiry calendar intervals defined in **Part.MinimumShelfLife** to the due date. Calculated from **StartDate** (adjusted for the primary part's expiry calendar), and then adjusted by **BOM.LeadTimeOffset** (in terms of the primary part's time units calendar) as well as **BOM.IntervalToExpiryOffset** (in terms of the primary part's expiry calendar). For non-expiry parts (**PartType.ExpiryRule** is set to "Ignore"), this field always reports "Past".	Money	
EstimatedExpiry Date	The date that the Netting analytic calculates this supply will expire. Used to determine which demands this supply can potentially be used to satisfy (supply is only eligible for use if it is on ot before the demand's **EarliestExpiryDate**). This date might be different from the actual expiry date reported in the **CTPCoProductSupply** table which takes supply availabilities into account. Non-expiry parts (**PartType.ExpiryRule** is set to "Ignore") always return a value of "Future".	Money	
Model	The model that applies to this supply. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is Ignore **Referenced table**: Model	Reference	
NeedQty	The amount of this co-product or by-product supply that is actually needed to satisfy demand for the part.	Quantity	
Part	A reference to the co-product or by-product part that this supply is for. **Referenced table:** Part	Reference	Yes
Pool	The pool that applies to this supply. It is set to Unpooled if either of the following conditions exist: • Pool is not enabled • Part.MUEPoolNettingType.PoolRule is Ignore **Referenced table:** Pool	Reference	

Table 8.10: CoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
PrimaryPart	A reference to the primary product whose planned order or scheduled receipt generated this co-product or by-product supply. **Referenced table:** Part	Reference	
PrimaryPlanned Order	A reference to the planned order for the primary product that generated this co-product or by-product supply. NULL if the primary product supply was a scheduled receipt. **Referenced table:** PlannedOrder	Reference	
Primary Scheduled Receipt	A reference to the scheduled receipt for the primary product that generated this co-product or by-product supply. NULL if the primary product supply was a planned order. **Referenced table:** ScheduledReceipt	Reference	
PrimarySupply Type	Indicates the type of the primary product's supply that triggered the generation of this co-product or by-product. Valid values are: • PlannedOrder • ScheduledReceipt	String (Enum)	
Quantity	The amount of this co-product or by-product supply. This is calculated by multipling the primary product supply's effective quantity (EffQuantity) by the QuantityPer defined on the BOM record that associates the primary product with this co-product or by-product (and then taking the absolute value of that result).	Quantity	
StartDate	The date when production of the supply should be started. This is set to the StartDate of the primary product supply that triggered this order.	Date	
StartUnit	The start unit number that applies to this supply. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is 'Ignore'	Integer	

Conflict table

The **Conflict** table identifies and reports details of task scheduling conflicts. A conflict indicates that a task cannot be scheduled in a manner that satisfies all limitations placed on it such as scheduling rules, predecessor/successor relationships, constraints, and material order requirements. These conflicts might lead to the generation of project plans and dates that are not necessarily desirable or achievable and should therefore be reviewed.

Task conflicts can occur in situations where a limitation placed on a task causes its dates to move in the opposite direction from which the task is being scheduled. Typically, this might be because of a task constraint or material order requirement. Task scheduling is set by the **ProjectType.ScheduleRule** setting on the project the task belongs to (either "FromStart" or "FromFinish"). For tasks in projects that are scheduled from their start date, a conflict results when a constraint attempts to move the task start/finish dates back and earlier than they would otherwise be. For tasks in projects that are scheduled from their finish date, a conflict results when a constraint attempts to move the task start/finish dates forward and later than they would otherwise be.

This table contains one record for each conflict found on an automatically scheduled task along with details of the constraint or material order requirement that caused the conflict. The **Project Plan** workbook included with RapidResponse also identifies any (primary) conflicts found on tasks within a project.

Table 8.11: Conflict (ProjMgmt) fields

Field name	Description	Data type	Owner
Constraint	If the scheduling conflict is caused by a task constraint, indicates the type of constraint (for example, "MustStartOn"). Otherwise, a value of "None" is returned.	String (Enum)	
Date	The date associated with the source of the task conflict as follows: • If the conflict was caused by a task constraint, then the date on which the task would have started if there was no constraint is reported. • If the conflict was caused by a material order link, then the supply or demand available date plus any applicable offset is reported • If the conflict was caused by the project run date, then the date the task would start on to respect all other scheduling parameters is reported. • If the conflict was caused by an actual start date on a task, then the date on which the task should start to satisfy the other scheduling parameters (such as, predecessor tasks) is reported. • If the conflict was caused by an actual or expected finish date on a task, then the date on which the task should finish to satisfy other scheduling parameters (such as, successor tasks) is reported.	Date	
Demand	If the conflict was caused by a demand order link, this is a reference to that **TaskDemandLink** record. The **IndependentDemand** record associated with the required material can also be accessed through this reference. Otherwise, null. **Referenced table:** TaskDemandLink	Reference	
Project	A reference to the project that the conflicted task belongs to. **Referenced table:** Project	Reference	Yes

Table 8.11: Conflict (ProjMgmt) fields

Field name	Description	Data type	Owner
Source	Indicates the scheduling parameter that led to the conflict reported on this record. For example, conflicts might typically be caused by task constraints or material order requirements. Valid values are: • **ActualFinishDate**—the conflict was caused by an actual finish date specified on the task. For example, in a project scheduled from finish, the ActualFinishDate might force the task to finish after a dependent successor task has already started. • **ActualStartDate**—the conflict was caused by an actual start date specified on the task. For example, in a project scheduled from the start, the ActualStartDate might force the task to start before a required predecessor task has been able to finish. • **Constraint**—the conflict was caused by a task constraint. The constraint date is honored and takes precedence over the other scheduling parameters. • **Demand**—the conflict was caused by the availability of an independent demand. The dates determined by the other scheduling parameters are honored and take precedence over the demand requirement. • **ExpectedFinishDate**—the conflict was caused by an expected finish date specified on the task. For example, in a project scheduled from finish, the ExpectedFinishDate might force the task to finish after a dependent successor task has already started (applicable to FixedDuration tasks only). • **RunDate**—the conflict was caused by the RunDate in a date-driven project. The RunDate is honored and takes precedence over the other scheduling parameters (for example, a task might be forced to finish after a dependent successor task or after the project finish date in order to respect the RunDate). • **Supply**—the conflict was caused by the availability of a scheduled receipt. The dates determined by the other scheduling parameters are honored and take precedence over the supply requirement.	String (Enum)	
Supply	If the conflict was caused by a supply order link, this is a reference to that **TaskSupplyLink** record. The **ScheduledReceipt** record associated with the required material can also be accessed through this reference. Otherwise, null. **Referenced table:** TaskSupplyLink	Reference	

Table 8.11: Conflict (ProjMgmt) fields

Field name	Description	Data type	Owner
Task	A reference to the task that has a conflict. **Referenced table:** Task	Reference	
Type	Indicates the phase in task scheduling where the conflict was detected. Because project tasks can be scheduled either from the project start date or the project finish date (based on a setting in the **ProjectType** table), conflicts caused by certain scheduling parameters can exist in either direction. Valid values are: • **Primary**—Indicates that the task conflict exists as the project is currently scheduled. For example, if the project has a **ProjectType.ScheduleRule** value of "FromStart", then this value indicates the conflict occurs when scheduling tasks from the project start date. This type of conflict typically requires investigation. • **Secondary**—Indicates a conflict that does not present itself as the project is currently scheduled. For example, if the project has a **ProjectType.ScheduleRule** of "FromStart", then this value indicates the conflict would occur if scheduling tasks from the project finish date. This type of conflict can typically be ignored.	String (Enum)	

ConsensusForecast table

The **ConsensusForecast** table contains the consensus forecast generated from the sales and operations planning process in RapidResponse. Each record in this table contains a calculated forecast amount for a part and customer combination on a given forecast date.

Forecast quantities in this table are calculated as a weighted combination of different forecast category quantities from the **ForecastDetail** table and event-based forecast adjustments on a given date. The weight assigned to a particular forecast category when calculating the consensus forecast for a part and customer combination is determined by the **ConsensusForecastWeight** field in one of the following tables:

- **HistoricalDemandHeaderTimePhasedAttributes**—this table stores time-phased forecast category weights per part customer. If it contains a record for a given forecast category and part customer combination, with an effective date range that includes the forecast date, then the category weight is taken from this table.
- **HistoricalDemandHeaderRollingWeight**—this table stores forecast weights to be used for a specific header and horizon combination when using rolling forecast weights. If a header has existing **HistoricalDemandHeaderTimephasedAttributes** records, this table is ignored.
- **HistoricalDemandHeader**—this table stores forecast category weights specific to a part customer when there is no requirement for time-phasing. If it contains a record for a given forecast category and part customer combination, and there is no effective record for that combination in either the **HistoricalDemandHeaderTimePhasedAttributes** table or the **HistoricalDemandHeaderRollingWeight** table, then the category weight is taken from this table.

- **HistoricalDemandCategoryRollingWeight**—this table stores forecast weights to be used for a specific category and horizon combination when using rolling forecast weights. If a header has existing **HistoricalDemandHeaderTimephasedAttributes** **HistoricalDemandHeaderRollingWeight**, or **HistoricalDemandHeader** records, this table is ignored.
- **HistoricalDemandCategory**—this table stores default forecast category weights to be used when there is no requirement for customer-specific weights. If there is no effective record for a given forecast category and part customer combination in of the **HistoricalDemandHeaderTimePhasedAttributes**, **HistoricalDemandHeaderRollingWeight**, **HistoricalDemandCategoryRollingWeight**, or **HistoricalDemandHeader** tables, then the category weight for the part customer is taken from this table.

Note that only categories with a **ConsensusForecastWeight** value greater than zero are used in calculating the consensus forecast values reported in this table. For example, if **ConsensusForecastWeight** was set to 1 for the Statistical forecast category and 0 for all other forecast categories, then the consensus forecast would be set to the statistical forecast value. Similarly, if **ConsensusForecastWeight** was set to 0.5 for the Finance forecast category, 0.4 for the Sales forecast category, and 0.2 for the Marketing forecast category, then the consensus forecast would be a summation of fifty-percent of the finance forecast, forty-percent of the sales forecast, and twenty-percent of the marketing forecast.

The **ConsensusForecast** table is used to drive supply planning in RapidResponse based on the consensus forecast generated during the sales and operations process. Consensus forecast quantities are subsequently reported in the **Forecast** table (subject to any forecast spreading logic) with a **ForecastSource** value of "ConsensusForecast". Availability of the forecast is given in the **CTPForecast** table, and details of all supplies used to satisfy the forecast can be seen in the **WhereConsumedForForecast** table.

If the **ForecastDetail** table contains a record for a given date, part, and customer where **Header.Category.Type.ProcessingRule** is set to "ForecastOverride" or "ReBalancingForecastOverride", then that record value is automatically reported in this table in the **OverrideQuantity** field instead of the calculated consensus forecast.

Note: ConsensusForecast records can only be generated for parts where the **PartType.ProcessRule** value is set to either "MPS" or "MPSConfig".

Table 8.12: ConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
CalculatedQuantity	The original calculated consensus forecast for this date. For example, this value might be different than the actual consensus forecast value reported in the Quantity field in cases where an override or adjustment was used for the consensus forecast.	Quantity	
Date	The date of this consensus forecast.	Date	

Table 8.12: ConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
EarliestExpiryDate	The earliest date that a supply can expire if it is to be used in satisfying this forecasted demand. For parts not configured for expiry, this field is always set to "Past". Calculated based on the demand **Date** and the **MinimumShelfLife** value from either the **TimePhasedPartCustomerAttributes** table (if used) or else the **PartCustomer** table (if > 0) or else the **Part** table. If provided, **ExpiryOffset** values on either the **TimePhasedPartCustomerAttributes** or **PartCustomer** tables are also used in this calculation.	Date	
EffectiveAllotment Override	A reference to the AllotmentOverride record (or Null) used to manually reserve component supply to the consensus forecast demand for this part and customer combination. **Referenced table:** AllotmentOverride	Reference	
EffectiveMinimum ShelfLife	For parts configured for expiry, this field represents the number of expiry calendar intervals to be added to the forecast date in order to determine its EarliestExpiryDate. For a given part and customer combination, this field is populated with the value from the **MinimumShelfLife** field value on either the **TimePhasedPartCustomerAttributes** table (if used), or the **PartCustomer** table (if > 0), or else the **Part** table.	Integer	
EffectivePriority	The order priority associated with this consensus forecast. For a given part and customer combination, this field is populated with the value from the **OrderPriority** field on either the **TimePhasedPartCustomerAttributes** table (if used), or else the **PartCustomer** table **Referenced table:** OrderPriority	Reference	

Table 8.12: ConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
EffectiveUnitPrice	The unit price effective for this forecast. This is typically useful in determining the revenue associated with the forecast. The calculated value in this field is based on the **EffectiveUnitPrice** value(s) found on the underlying **ForecastDetail** record(s) that make up the consensus forecast quantity. If there is only one associated **ForecastDetail** record, then its **EffectiveUnitPrice** is used here. Otherwise, if there are multiple associated **ForecastDetail** records, then this field contains a weighted average of the **EffectiveUnitPrice** values on those underlying records (each is given a weight relative to the quantity it contributes to the overall consensus forecast quantity). Currency conversions on this field use the **Date** field's conversion rate. **Note:** If the values that contribute to determining **ForecastDetail.EffectiveUnitPrice** are negative, the value reported in both **ForecastDetail.EffectiveUnitPrice** and **ConsensusForecast.EffectiveUnitPrice** is set to zero. For information about how the value in the **ForecastDetail.EffectiveUnitPrice** field is determined, see "EffectiveUnitPrice" on page 286.	Money	
OverrideQuantity	Reports override value made to the consensus forecast when generating the demand plan (typically, done using the **S&OP Consensus Demand Planning** workbook). This type of override originates from **ForecastDetail** records where the **HistoricalDemandCategoryType.ProcessingRule** is set to "ForecastOverride".	Quantity	
Part	The part this forecast applies to. **Referenced table**: Part	Reference	Yes
PartCustomer	The combination of the part and customer this forecast applies to. **Referenced table:** PartCustomer	Reference	
Quantity	The amount of the consensus forecast for this date.	Quantity	
Rebalancing Adjustment Quantity	Reports adjustments made to either the calculated consensus forecast or the forecast override quantity (if specified) when rebalancing the demand plan to the supply plan. The adjustments originate from **ForecastDetail** records where the **HistoricalDemandCategoryType.ProcessingRule** is "RebalancingAdjustment".	Quantity	

Table 8.12: ConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
Rebalancing OverrideQuantity	Reports overrides made to the consensus forecast when rebalancing the demand plan to the supply plan (typically, done using the **S&OP Demand Supply Balancing** workbook). This type of override takes precedence over any other override or adjustment values and originates from **ForecastDetail** records where the **HistoricalDemandCategoryType.ProcessingRule** is "ReBalancingForecastOverride".	Quantity	

ConsensusForecastDetail table

The **ConsensusForecastDetail** table identifies and reports details of **ForecastDetail** records that are used for generating the consensus forecast value for a particular part and customer during the sales and operations planning process and reported in the **ConsensusForecast** table. For example, it might report details of multiple weighted forecast categories that contribute to the consensus forecast on a given date along with the quantity that each of those categories contributes to the consensus forecast value.

Note: If a **ForecastDetail** record is affected by an event-based adjustment, it is reported in the analogous **EventConsensusForecastDetail** table instead of the **ConsensusForecastDetail** table. For more information, see "EventConsensusForecastDetail table" on page 1039.

Table 8.13: ConsensusForecastDetail(Mfg) fields

Field name	Description	Data type	Owner
Consensus Forecast	A reference to the **ConsensusForecast** record. This returns the calculated consensus forecast for a particular part and customer combination on a given forecast date. **Referenced table:** ConsensusForecast	Reference	
Date	The date from the **ForecastDetail** record, rounded down to the nearest date on the **PartCustomer.EffectiveDisaggregation Calendar**. **Note:** This field was added in Version 2016.2, Service Update 3.	Date	

Table 8.13: ConsensusForecastDetail(Mfg) fields

Field name	Description	Data type	Owner
Effective Quantity	The amount from the referenced **ForecastDetail** record that contributes to the consensus forecast total. This value will be different from the value in **ForecastDetail.Quantity** in cases where the consensus forecast is a weighted sum combining values from different forecast categories. Thus, the sum of all values in this field for each referenced record in the **ConsensusForecast** table should equate to the **ConsensusForecast.Quantity** value. In some cases, this field might return zero. For example, if the referenced **ForecastDetail** record represents either a negative forecast adjustment or a forecast stream whose quantity was reduced to zero by an adjustment.	Quantity	
Forecast Detail	A reference to a **ForecastDetail** record. This returns the forecast quantity and date for a particular part, customer, and forecast category combination that are contributing to the consensus forecast quantity. In cases where a forecast override or forecast rebalancing override is used, only the associated **ForecastDetail** record for that override is referenced in this table (that is, any other forecast details that contributed to the value found in **ConsensusForecast.CalculatedQuantity** are ignored in this table). **Referenced table:** ForecastDetail	Reference	
Part	The part whose consensus forecast this record pertains to. **Referenced table:** Part	Reference	Yes

ConsensusForecastRollingHorizonWeight table

The **ConsensusForecastRollingHorizonWeight** table reports the weight that is applied to a forecast category in a given horizon when using rolling forecast weights to create the consensus demand plan. The results are based on values from the **HistoricalDemandCategoryRollingWeight** and **HistoricalDemandHeaderRollingWeight** tables. It also reports the source of the forecast weight value. See "Consensus forecast category weights" on page 2073.

Note: If a header has existing **HistoricalDemandHeaderTimephasedAttributes** records, this table is ignored. Therefore, to use the rolling forecast weights, you need to make sure to delete any exsiting **HistoricalDemandHeaderTimephasedAttributes** records.

The **ConsensusForecastRollingHorizonWeight** table supports the Demand Planning application in RapidResponse. Values in this table are reported in the **S&OP Demand Planning Ratios** workbook. For more information about the Demand Planning application, see the RapidResponse Applications Guide.

This table was added in Version 2016.2, Service Update 12.

Table 8.14: ConsensusForecastRollingHorizonWeight (Mfg) fields

Field name	Description	Data type	Owner
Source	Indicates the source of the weight value that was used to calculate the consensus forecast. Valid Values are: • **HistoricalDemandHeaderRollingWeight**—Indicates that the consensus forecast weight was set using the **HistoricalDemandHeaderRollingWeight** table. • **HistoricalDemandHeader**—Indicates that the consensus forecast weight is from the **HistoricalDemandHeader** table. **HistoricalDemandCategory**—Indicates that the consensus forecast weight is from the **HistoricalDemandCategory** table. • **HistoricalDemandCategoryRollingWeight**— Indicates that the consensus forecast weight is from **HistoricalDemandCategoryRollingWeight** table.	Enum	
Weight	The value of the forecast weight that is associated with this category or header during a specific horizon.	Quantity	
Header	A reference to the header. **Reference table**: HistoricalDemandHeader	Reference	Yes
Horizon	A reference to the horizon. **Referenced table**: ConsensusForecastRollingHorizon	Reference	

ConsensusForecastWeightByHeader table

The **ConsensusForecastWeightByHeader** table reports the resulting consensus forecast weights by headers. The values that are reported are used by **ConsensusForecast** calculations. The results are based on the **ConsensusForecastWeight** that is set using one of the following input tables:

- HistoricalDemandCategory
- HistoricalDemandCategoryRollingWeight
- HistoricalDemandHeader
- HistoricalDemandHeaderRollingWeight
- HistoricalDemandHeaderTimePhasedAttributes

The **ConsensusForecastWeightByHeader** table is used if the **ProcessingRule** field on the **HistoricalDemandCategoryType** table is set to "Forecast", indicating that the historical demand category will be used towards calculating the consensus forecast. If the **HistoricalDemandHeaderTimePhasedAttributes** table contains a record for a given forecast category and part customer combination, with an effective date range that includes the forecast date, then it is used to generate the consensus forecast.

Note: For more information about consensus forecast weight calculations, see "Consensus forecast category weights" on page 2073.

Table 8.15: ConsensusForecastWeightByHeader (Mfg) fields

Field name	Description	Data type	Owner
Date	The date in the disaggregation calendar, which defines the periods that orders can be disaggregated into.	Date	
Header	A reference to the part customer and forecast category combination this set of values applies to. **Referenced table:** HistoricalDemandHeader	Reference	Yes
Weight	The weight for the header.	Quantity	

ConstraintLoad table

This table reports load on a Constraint from supplies. This shows when the Constraint was requested. Exactly one ConstraintLoad record is generated for each supply referencing an active Constraint except no ConstraintLoad record is generated for supplies where Load is 0, TargetAccum= **Ignore** or where Constraint.Type.ProcessingRule = **Unconstrained**.

Table 8.16: ConstraintLoad (Mfg) fields

Field name	Description	Data type	Owner
Constraint	The name of the Constraint. **Referenced table:** Constraint	Reference	Yes
CTPPlanned Order	The CTP planned order generating this load (Null if a scheduled receipt). **Referenced table:** CTPPlannedOrder	Reference	
Date	The date load is placed on the constraint. Either the ship date or start date associated with the later of material start date and resched date of the supply, normalized to the beginning of the constraint period.	Date	
IsPlanned	Indicates whether this load is from a planned order (Y) or scheduled receipt (N).	Boolean	
Load	The quantity of load placed on this constraint by this supply. Note that no unit of measure conversion is applied between supply units and Constraint units.	Quantity	

Table 8.16: ConstraintLoad (Mfg) fields

Field name	Description	Data type	Owner
Model	The model of the supply creating the load. **Referenced table:** Model	Reference	
Pool	The pool of the supply creating the load. **Referenced table:** Model	Reference	
Scheduled Receipt	The immediate scheduled receipt generating this load (Null if a planned order). **Referenced table:** ScheduledReceipt	Reference	
Source Constraint	The SourceConstraint that produced this load. **Referenced table:** SourceConstraint	Reference	
SupplyPart	The part whose supply is creating this Load. **Referenced table:** Part	Reference	

ConstraintSpreadLoad table

The **ConstraintSpreadLoad** table is generated for constraints where the Type.ProcessingRule is either Constrained, LoadOnly, or CumMax. It reports load that is scheduled, but can overload the available constraint. This table is similar to the **ConstraintLoad** table, except that a single supply can have several ConstraintSpreadLoad records (as opposed to one record per supply in ConstraintLoad).

ConstraintSpreadLoad is normally applied between the earliest date that constraint can be applied to the supply and the latest date that constraint can be applied and still meet the supply's ReschedDate. The constraint is initially applied up to the amount of constraint available (allowing for any previously allocated constraint). Any load that cannot be satisfied to complete the order for its ReschedDate is spread evenly between the early constraint date and the late constraint finish.

For more information about how constraint is spread across periods, see "ConstraintLoad" on page 1646 and "ConstraintSpreadLoad" on page 1646.

Table 8.17: ConstraintSpreadLoad (Mfg) fields

Field name	Description	Data	Owner
Constraint	The constraint the spread load record refers to. **Referenced table:** Constraint	Reference	Yes
CTPPlannedOrder	The planned order generating the load. This field is null if a scheduled receipt is generating the load. **Referenced table:** CTPPlannedOrder	Reference	
Date	The date the load is planned for this constraint. This considers load from other orders and constraint availability. Date is always reported as the start of the constraint period for the referenced constraint.	Date	

Table 8.17: ConstraintSpreadLoad (Mfg) fields

Field name	Description	Data	Owner
IsPlanned	Indicates whether this load is from a planned order or a scheduled receipt. Values are: • **Y**—Load is from a planned order. • **N**—Load is from a scheduled receipt.	Boolean	
Load	The quantity of load placed on the constraint by this supply in this period.	Quantity	
Model	The model of the supply creating the load. **Referenced table:** Model	Reference	
Overloaded	Indicates if *any* constraint associated with the supply is overloaded (does not have sufficient remaining capacity). Valid values are: • Y • N **Note:** For scheduled receipts where **Status.RescheduleRule = 'Scheduled'**, a value of **Y** indicates that one of the constraints is overloaded by the scheduled receipt.	Boolean	
Pool	The pool of the supply creating this load. **Referenced table:** Pool	Reference	
ScheduledReceipt	The immediate scheduled receipt generating this load. This field is null if a planned order is generating the load. **Referenced table**: ScheduledReceipt	Reference	
SourceConstraint	The source constraint that produced this load. **Referenced table**: SourceConstraint	Reference	
SupplyPart	The part whose supply is creating the load. **Referenced table:** Part	Reference	

ConstraintUsed table

This table reports when load is allotted to each supply that uses a Constraint. One or more ConstraintUsed records are generated for each ConstraintLoad record. This table supports the Constraint Manager application.

Table 8.18: ConstraintUsed (Mfg) fields

Field name	Description	Data type	Owner
Constraint	The name of the Constraint. **Referenced table:** Constraint	Reference	Yes
ConstraintLoad	The Constraint load being allocated. **Referenced table:** ConstraintLoad	Reference	
ConsumptionSequence	During constraint scheduling, the order that constraint is set to be consumed.	Integer	
Date	The date this amount of Constraint is allotted to this supply. Date is the beginning of the period that Constraint is available. It can be the period including the load or a subsequent period.	Date	
Load	The quantity of load satisfied in this period.	Quantity	
Overloaded	Indicates if *any* constraint associated with the supply is overloaded (does not have sufficient remaining capacity). Valid values are: • Y • N **Note:** For scheduled receipts where **Status.RescheduleRule = 'Scheduled'**, a value of **Y** indicates that one of the constraints is overloaded by the scheduled receipt.	Boolean	
ProductionCycleNumber	The cycle number that the constraint is used in. For example, a cycle number of 2 means that the constraint is used in the second cycle of the production wheel. Default value: 0	Integer	
ProductionWheel	A reference to the production wheel associated with this ConstraintUsed record. Reference table: ProductionWheel	Reference	

Table 8.18: ConstraintUsed (Mfg) fields

Field name	Description	Data type	Owner
Type	The type of constraint used during constraint scheduling. Valid values are: **Fixed**—the constraint associated with SourceConstraint.FixedConstraintFactor **Variable**—the constraint associated with SourceConstraint.ConstraintFactor **CampaignFixed**—the constraint associated with SourceConstraint.CampaignFixedConstraintFactor **MajorSetup**—the constraint associated with ProductionGroup.MajorSetup **MajorCleanup**—the constraint associated with ProductionGroup.MajorCleanup **MinorChangeOver**—the constraint associated with ProductionGroup.MinorChangeOver	Enumeration	

CostOfGoodsSold table

The **CostOfGoodsSold** table stores records created by RapidResponse for each supply quantity that satisfies an originating demand (independent or dependent). **CostOfGoodsSold** records provide extended costs on supplies apportioned to originating demands. This includes all sources of demand, excess scheduled receipts, excess on hand, and excess supply from planned orders due to lot sizing. For information about how RapidResponse calculates **CostOfGoodsSold** records, see "Cost of goods sold calculations" on page 1503.

Table 8.19: CostOfGoodsSold (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The part the supply activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, and this record pertains to a supply, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table**: Part	Reference	
CTPCoByProduct Supply	A reference to the CTP details for the co-product or by-product associated with this record. Costs for co-products and by-products come from the corresponding primary product in proportion to the percentage split associated with the co-product or by-product supply. This field is NULL unless supply comes from a co-product or by-product. **Referenced table:** CTPCoByProductSupply	Reference	
CTPForecast	A reference to the driver part's consensus forecast demand (as generated during the sales and operations planning process in RapidResponse) that is satisfied by this supply. NULL unless DriverType = 'ConsensusForecast'. **Referenced table:** CTPForecast	Reference	
CTPPlanned Order	A reference to the consumed planned order. This field is Null unless the supply is a planned order. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	The CTP Substitute Supply for this record. **Referenced table:** CTPSubstituteSupply	Reference	
DriverAvailable Date	The date the driver is available.	Date	
DriverDueDate	The DueDate of the driver for this demand.	Date	
DriverPart	The part driving the demand that consumes the supply. **Referenced table:** Part	Reference	
DriverQuantity	The quantity of the driving demand.	Quantity	

Table 8.19: CostOfGoodsSold (Mfg) fields

Field name	Description	Data type	Owner
DriverScheduled Receipt	A reference to the scheduled receipt ultimately driving this requirement. This field is Null unless DriverType is a ScheduledReceipt. **Referenced table:** ScheduledReceipt	Reference	
DriverType	Indicates the source of this demand. Valid values are: • **CoByProductSupply**—this supply is used to satisfy a co-product or by-product generated from a primary product supply. • **ConsensusForecast**—this supply is used to satisfy consensus forecast demand generated during the sales and operations planning process. • **Forecast**—this supply satisfies an unconsumed forecast. • **IndependentDemand**—this supply satisfies an independent demand. The IndependentDemand reference is valid. • **InventoryTransfer**—this supply satisfies an inventory transfer. • **OnHand**—this supply is excess on hand inventory not used in satisfying any demand. • **ScheduledReceipt**—this supply satisfies a scheduled receipt. The DriverScheduledReceipt reference is valid. • **PlannedOrder**—this supply satisfies a supply with no corresponding demand (therefore excess due to safety or planned order lot sizing). The Driver references are invalid.	String	
Independent Demand	A reference to the independent demand driving this requirement. This field is Null unless DriverType is IndependentDemand or Forecast. **Referenced table:** IndependentDemand	Reference	
Inventory Transfer	The inventory transfer satisfying this demand. **Referenced table:** InventoryTransfer	Reference	
LaborCost	Extended labor cost. This field is zero if SupplySource is OnHand, InKit, or None. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	

Table 8.19: CostOfGoodsSold (Mfg) fields

Field name	Description	Data type	Owner
MaterialCost	Extended material cost for this supply. Material costs are calculated for inventory supplies, in-kit supplies, purchased supplies, and allocated scheduled receipts. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
Model	The model used when netting this record. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule control value is Ignore. **Referenced table:** Model **Note:** This field is set to the first record in the Model table. The first record in the Model table should always be None.	Reference	
NeedDate	The date when this supply is needed to achieve on-time delivery for the driver.	Date	
NeedQuantity	The amount of this supply required to meet the demand.	Quantity	
OnHand	The inventory record satisfying this demand. Null unless the supply is OnHand. **Referenced table:** OnHand	Reference	
OverheadCost	Extended overhead cost. Zero if SupplySource is OnHand, InKit, or None. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	

Table 8.19: CostOfGoodsSold (Mfg) fields

Field name	Description	Data type	Owner
OverheadCost2	Secondary extended overhead cost. Zero if SupplySource is OnHand, InKit, or None. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
Part	The part whose supply is consumed. **Referenced table:** Part	Reference	Yes
PartSource	The part source used by the supply. **Referenced table:** PartSource	Reference	
Pool	The pool used when netting this record. It's set to Unpooled if either of the following conditions exist: • Pool is not enabled. • Part.MUEPoolNettingType.PoolRule control value is Ignore. **Referenced table:** Pool **Note:** This field is set to the first record in the Pool table. The first record in the Pool table should always be Unpooled.	Reference	
Scheduled Receipt	A reference to the scheduled receipt that is consumed. This field is Null if the supply is not a scheduled receipt. **Referenced table:** ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the supply, this represents the date a portion of the supply is available. If incremental availability calculations are turned off for the supply, this represents the date when the entire supply is available (except for cases where a planned order is split based upon constraint allocation).	Quantity	
SubstituteGroup	A reference to the substitute group associated with the supply. Null unless the supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt. **Referenced table:** SubstituteGroup	Reference	

Table 8.19: CostOfGoodsSold (Mfg) fields

Field name	Description	Data type	Owner
SupplyAvailable Date	The date when the entire supply is available.	Date	
SupplyDemand	The supply and demand being reported by this record. **Referenced table:** SupplyDemand	Reference	
SupplyDueDate	The effective due date of the supply being consumed.	Date	
SupplyQuantity	Total quantity of the supply.	Quantity	
SupplyQuantity UnitOfMeasure	Reference to the unit of measure used for the supply quantity that is associated with this record. **Referenced table:** "UnitOfMeasure table" on page 931. **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
SupplyStartUnit	The start unit used when netting this record. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule control value is Ignore.	Quantity	
SupplySource	The source of this supply record. Valid values are: • **CoByProductSupply**—supply is a co-product or by-product. • **CTPPlannedOrder**—normal planned order. • **InKit**—supply from InKit quantity on an allocation. • **InventoryTransfer**—supply is an inventory transfer. • **None**—no supply is available to satisfy this demand. • **OnHand**—supply is in on hand inventory. • **ScheduledReceipt**—normal scheduled receipt. • **SubstituteSupply**—supply comes from a substitute part.	String (Enum)	
WhereConsumed	The WhereConsumed record corresponding to this record. **Referenced table:** WhereConsumed	Reference	

CostRollUp table

The **CostRollUp** table stores rolled-up cost results, which are calculated based on effective bill components and effective part sources for every part. In cases where a part has multiple effective part sources at the highest priority, RapidResponse prorates the cost according to the target values on the relevant part sources.

The table description below uses the term **descendants**. This term includes all components that appear below a specific part assembly (including components of components and so on). Components that arise through transfer part sources are also included.

The term **effectively purchased** is used in the description of the **MaterialCostOnly** field. Effectively purchased parts do not have components. They are either purchased parts, parts with a make part source but no effective bill components, or parts with a transfer part source for which the transfer part name and site matches the part name and site. For information about how RapidResponse calculates **CostRollUp** records, see "Cost roll-up calculations" on page 1511.

Table 8.20: CostRollUp (Mfg) fields

Field name	Description	Data type	Owner
FirstDate	The beginning of the date range (inclusive) for which these rolled-up costs apply.	Date	
FirstUnit	The first unit to which these rolled-up costs apply.	Integer	
LaborCost	Rolled-up labor costs for this part, based on labor costs for all descendants of this part, including the part itself. Currency conversions on this field use the **FirstDate** field's conversion rate.	Money	
LastDate	The end of the date range (inclusive) for which these rolled-up costs apply.	Date	
LastUnit	The last unit to which these rolled-up costs apply.	Integer	
MaterialCostOnly	Rolled-up material purchase costs for this part. Based on all effectively purchased descendants of this part, including the part itself (assuming it's effectively purchased). Currency conversions on this field use the **FirstDate** field's conversion rate.	Money	
Model	The model to which these costs apply. **Referenced table:** Model	Reference	
OverheadCost	Rolled-up overhead costs for this part, based on overhead costs for all descendants of this part, including the part itself. Currency conversions on this field use the **FirstDate** field's conversion rate.	Money	
OverheadCost2	Rolled-up secondary overhead costs for this part, based on secondary overhead costs for all descendants of this part, including the part itself. Currency conversions on this field use the **FirstDate** field's conversion rate.	Money	

Table 8.20: CostRollUp (Mfg) fields

Field name	Description	Data type	Owner
Part	The part for which rolled-up costs are calculated. **Referenced table:** Part	Reference	Yes

CriticalPath table

The **CriticalPath** table can be used to identify all tasks that are on the critical path for a given project, as well as all gated tasks regardless of whether they are on a project's critical path.

Tasks on the critical path are those that directly impact the project finish date, and hence can potentially push out the project finish date if they finish late or pull in the project finish date if they can finish early. The **Critical** field can be used to find all tasks that are on a project's critical path.

Gated tasks are those whose completion is impacted by other items such as predecessor tasks, material order requirements, or constraints. If there are multiple items gating a given task in a project, then multiple records are reported in this table for that task and project combination. The **Source** field can be used to find all tasks that are gated, and other fields in this table used to provide details of the gating task and/or order.

Table 8.21: CriticalPath (ProjMgmt) fields

Field name	Description	Data type	Owner
Critical	Indicates whether the task is on the critical path for the project. Valid values are: • Y • N	Boolean	
GatingDemand	A reference to an independent demand that is gating the task reported on this record. This reference is valid only when the **Source** value is 'Demand'. **Referenced table:** IndependentDemand	Reference	
GatingSupply	A reference to a scheduled receipt that is gating the task reported on this record. This reference is valid only when the **Source** value is 'Supply'. **Referenced table:** ScheduledReceipt	Reference	
GatingTask	A reference to another task that is gating the task reported on this record. This reference is valid only when the **Source** value is 'Task'. **Referenced table:** Task	Reference	
Project	A reference to the project that the task being reported in this record pertains to. **Referenced table:** Project	Reference	Yes

Table 8.21: CriticalPath (ProjMgmt) fields

Field name	Description	Data type	Owner
Task	A reference to the task this record pertains to. This will be either a task that is on the critical path for its project, or that is gated by another task or material dependency, or both. **Referenced table:** Task	Reference	
Source	Indicates the type of dependency, if any, that is gating the task reported on this record and placing it on the project's critical path. Valid values are: • **Constraint**—task completion is impacted by a date-based constraint placed on the task. • **Demand**—task completion is impacted by the availability of a required demand order. • **None**—task completion is not impacted by any other tasks or material dependencies. • **Task**—task completion is impacted by the completion of another task. • **Supply**—task completion is impacted by the availability of a required supply order.	String (Enum)	

CTPActivity table

This table reports results of CTP calculations. It also shows AvailableDate and related information.

This table contains one record per supply, unless either:

- the incremental availability option is turned on for a supply in which case multiple records might be generated for that supply (as a result of component availability, constraint availability, or both).
- some portion of the supply will expire, in which case a second record is added with an ExtendedType field value of "Expiring" (The original record of this supply will have its ExtendedType field value of "Normal"). The second record reports the expiring quantity in the DemandQty field, the negative of the expiring quantity in the BalanceDelta field and has a SupplyQty equal to 0.

Activity table comparison

This table has some similarities to the Activity table. However, while the Activity table should be used when the reporting of planning actions is required, the CTPActivity table should be used when capable-to-promise results are required. Among the main differences between the Activity and CTPActivity tables:

- Although netting may run for a part, thus producing records in the Activity table, CTP may not run for that same part and therefore no CTPActivity table records would be produced. In particular, if a part has a recursive BOM structure below it, CTP calculations will not be performed even though Netting calculations are.
- Reschedulable scheduled receipts where none of the supply is needed are not reported in the CTPActivity table, though they are reported in the Activity table.

- The CTPActivity table reports supplies as they are used in capable-to-promise calculations to satisfy demands. In cases where there is an especially large variation in the quantities of supplies or demands (such as, 1,000,000:1), the quantity of a particular supply or demand reported in this table might be slightly different than the actual supply or demand quantity.

Note: If attribute-based planning is used to impact the allotment of supply to demand, all relevant demand and supply attributes at the part level can be reported on records in this table using the `Attribute<*>` syntax. Note that any demand attributes without a mapping in the **AttributeMap** table are excluded. For more information on the *Attribute*<*> syntax, see the *RapidResponse Resource Authoring Guide*).

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
Adjusted Demand Date	For demands, this returns the DemandDate less the part's safety lead time (note that any safety lead time at the part source level is not included). For supplies, thisis the same as RecommendedDate (which considers safety lead time from both the part and part source). **Note:** This field was added in Version 2016.2	Date	
Allocation	The Allocation record this entry relates to. **Referenced table:** Allocation	Reference	
AlternatePart	The part the supply or demand activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply or demand data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, and this record pertains to a supply, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. Otherwise, if this record pertains to a demand, then Part is the part that needed the supply and AlternatePart is the part where the demand came from. **Referenced table**: Part	Reference	
AssociatedDate	The PromiseDate for demands, CalcStartDate for supply, PlannedOrder.DueDate for dependent demands from planned orders, and RunDate for on hand.	Date	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
ATPQuantity	The quantity of this supply (0 if a demand) that is available to promise (allocated to excess, sales forecast, or production forecast).	Quantity	
AvailableDate	The date the entire supply is available, unless the incremental availability option is turned on for the supply in which case this represents the date a segment of the supply is available.	Date	
AvailableStart Date	If incremental availability is turned off for the supply, this field represents the date when work is expected to start on the supply (all materials and the first portion of constraint are available). If incremental availability is turned on for the supply, this field represents the date when work is expected to start on the segment of the supply (some set of materials and the first portion of constraint are available). This field is calculated from AvailableDate if there is no active constraint.	Date	
BalanceDelta	The change in the inventory level due to the original quantity of the activity.	Quantity	
Constraint	The Constraint whose insufficient capacity means the supply will not be ready for its MaterialAvailableDate. If this supply is not late, this field is empty. **Referenced table:** Constraint	Reference	
CTPCoByProduct Supply	Reference to the CTP cobyproduct associated with this supply. **Referenced table:** CTPCoByProductSupply	Reference	
CTPForecast	Reference to the consensus forecast that is the source of this demand. **Referenced table:** CTPForecast	Reference	
CTPPlanned Order	Reference to the CTP planned order record that is this supply or is the immediate parent of dependent demands from a planned order. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	Reference to the CTP substitute supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
DemandDate	For supplies, date of the earliest demand matched by netting (DemandDate). For demands, same as Activity.DueDate.	Date	
Demand Exception	If this record represents a demand that was skipped because no planned order could be generated to meet its expiry requirements, this field is a reference to the associated DemandException record. This reference gives additional details on the skipped demand, such as the earliest date that any supply could expire if it were to be used in satisfying that demand. **Referenced table:** DemandException	Reference	
DemandQty	For demands, the total quantity of the demand, including ShippedQty (SupplyQty will be the shipped quantity or consumed forecast). For supplies, the quantity that has been received. If the record has an **ExtendedType** value of "Expiring", this field contains the amount of the supply that will not be used before its expiry date.	Quantity	
DemandType	The DemandType associated with this record. **Referenced table:** Demand	Reference	
DockDate	Date the transaction should be delivered to the receiving dock (calculated from ReschedDate). ReschedDockDate for scheduled receipts. Undefined unless planned order or scheduled receipt.	Date	
DueDate	The date this supply (if it is a supply) is supposed to be available for use (EffDueDate). For demands, the date the demand is needed (based on parent supply ReschedDate if dependent - Allocation.ReschedDate, NewAllocation.DueDate). **Note:** For self-allocations, this field is always set to 'Past'. For more information about self-allocations, see "Self-allocations" on page 1424.	Date	
EarliestExpiry Date	If this record represents a demand, this is the earliest date that supply used to satisfy that demand may expire. If this record represents a supply, this is the earliest date that supply may expire if it is to be used in satisfying its dependent demands (taken from the latest EarliestExpiryDate of any demands it satisfies in Netting calculations.)	Date	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
Estimated ExpiryDate	If this record pertains to a supply, this is the date that the Netting analytic calculates that supply will expire and uses in determining which demands it can potentially supply satisfy. Note that this value might be different than the actual expiry date which is reported in the ExpiryDate field and considers all supply availabilities in its calculation. If the supply comes from on hand inventory or an inventory transfer then the supply's **ExpiryDate** value is reported here. For all other types of supply, the supply's **EstimatedExpiryDate** value is reported.	Date	
ExpiryDate	The data that the supply associated with this record will expire (is no longer usable).	Date	
ExpiryStart Date	Date on which to start supply expiry date calculations for assemblies having a **PartType.ExpiryRule** of "DependentStart (based on earliest ExpiryStartDate rolled up from a component supply). This field is calculated for all expiring parts that are either designated as limiting components or contain limiting components in their product structure. Otherwise, this field is set to "Future". For more information about how supply expiry start dates are used, see "DependentStart" on page 1618.	Date	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
ExtendedType	Indicates whether this record is associated with a special extended type or is a normal CTPActivity record. Valid values are: • **DemandException**— a record representing demand that was not met because supply could not be sourced to meet its expiry requirements. Typically, this occurs when RapidResponse is unable to generate a planned order with an expiry date on or after the demand's earliest expiry date. For more information, see "Unsourceable demands and the Activity/CTPActivity tables" on page 1626. • **Expiring**—a record representing the expiring portion of a supply (that is, the portion of a supply that will not be used before its expiry date). For records of this type, the DemandQty field contains the expiring quantity, the SupplyQty field is always set to zero (0), and the BalanceDelta field contains the inverse (negative) of the expiring quantity. For more information, see "Expiring quantities and the CTPActivity table" on page 1624. • **Normal**—a normal CTPActivity record (does not represent an expiry demand exception or the expiring portion of a supply). • **PushTransfer**—a record representing planned order supplies for a part that are arriving at a destination site earlier than requested. This occurs when the pre-built supply at the sourcing site exceeds the maximum inventory level that is set for the part. **Note:** This value was added in Version 2016.2.	String (Enum)	
Forecast	The Forecast record associated with this demand (if Activity.Type =Forecast). **Referenced table:** Forecast	Reference	
HoldEndDate	The date on which the supply, or portion of supply, reported on this record can actually be used without violating the minimum safety stock level (represented as a higher priority safety stock demand). This field is only populated if the part's **SafetyStockPolicy.CTPHoldRule** setting is "Use", incremental availability is enabled for the part, and some or all of the supply needed to be held from a demand it was allotted to in order to maintain safety stock. **NOTE**: This field was added in Version 2016.2, Service Update 18.	Date	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
HoldStartDate	The date from which the supply, or portion of supply, reported on this record should be held in order to respect a higher priority safety stock demand. In other words, this is the date that the minimum safety stock level (represented as higher priority safety stock) would be violated if the supply allotment were not held. This field is only calculated if the part's **SafetyStockPolicy.CTPHoldRule** setting is "Use", incremental availability is enabled for the part, and some or all of the supply needed to be held from a demand it was allotted to in order to maintain safety stock. **NOTE**: This field was added in Version 2016.2, Service Update 18.	Date	
Independent Demand	Reference to the source of demand. NULL if demand is not from an IndependentDemand record (for example, all dependent demand). **Referenced table:** IndependentDemand	Reference	
Inventory Transfer	The InventoryTransfer record associated with this CTPActivity record. Valid only if Type is 'InventoryTransfer' or 'InventoryTransferAllocation'. **Referenced table:** InventoryTransfer	Reference	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
LateBeyondTolerance Quantity	The amount of the late demand that is available outside of tolerance. That is, the amount of the demand that is later than its due date plus the number of **ForecastCalendar** intervals specified in the **Part.UnsatisfiedDemandTolerance** field. For example, if **ForecastCalendar** is "Month" and **UnsatisfiedDemandTolerance** is "0", this reports the amount of the demand that is available later than the end of the week in which it is due. Similarly, if **UnsatisifedDemandTolernace** were set to "2", this would return the amount of the demand that was available later than the end of the second month after which it was due. When using on-time reallocation logic (where **PartType.OnTimeAllocationRule** is set to "ImproveWithTolerance"), supply quantities that are allotted to demands beyond lateness tolerance can be reallocated to try and improve the on-time quantity on late demands (with the potential side effect of also making the available date on those late worse than they otherwise would be). Note that this field returns 0 (zero) for all supply records, as well as any demands that are on time or not later than tolerance. **Note**: This field was added in Version 2016.2, Service Update 27, and is calculated regardless of the part's **OnTimeAllocationRule** setting.	Quantity	
Model	The model used when netting this record. **Referenced table:** Model	Reference	
NewProcessCost	Standard labor and overhead cost of planned orders to complete this order. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used.	Money	
NewPurchases Cost	Cost of new purchases required in order to complete this order (through all levels of the bill). Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used.	Money	
OnHand	The OnHand record associated with this record. Valid only if Type is 'OnHand'. **Referenced table:** OnHand	Reference	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
OnTimeQuantity	The quantity of this activity that could be completed on or before its ReschedDate if it was possible to split the order.	Quantity	
OrderPriority	The priority associated with this supply or demand (Part.Type.DefaultPriority for OnHand). **Referenced table:** OrderPriority	Reference	
Part	The part this record relates to. **Referenced table:** Part	Reference	Yes
PartSource	The PartSource from ScheduledReceipt or PlannedOrder. Populated if the Activity record is based on a ScheduledReceipt, PlannedOrder, or dependent demand. Otherwise, Null. **Referenced table:** PartSource	Reference	
PegModel	The model that is applicable to this supply or demand. The field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.ModelRule.	String	
PegPart	The source of demand if the record describes a dependent demand. This value is NULL for supply records (planned order, scheduled receipt and on hand) as well as independent demand, forecast, and ATP. **Referenced table:** Part	Reference	
PegPool	The pool that is applicable to this supply or demand. This field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.PoolRule. **Referenced table:** Pool	Reference	
Pool	The pool used when netting this record. **Referenced table**: Pool	String	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
PushTransferID	A unique integer identifier for pre-built supply that was transferred earlier than needed (pushed) from a sourcing site to a distribution site as a result of the pre-built supply exceeding the maximum inventory level that was set for the part. This field only applies to parts that are enabled for push planning (have **DistributionPlanningRule.ProcessingRule** set to "Use". Additionally if **DistributionPlanningRule.SupplyShareRule** is set to "All", then this field is populated for all transfers from the sourcing site to distribution site (including late supplies). This is regardless of the maximum inventory level being exceeded and whether the supply was actually pushed earlier than needed. **Note:** This field was added in Version 2016.2.	Integer	
Recommended Date	The recommended due date (stock date) for the supply. Same as DueDate if a demand or planned order.	Date	
ReschedDate	The revised due date (stock date) after rescheduling the supply. Based on its parent supply's ReschedDate (same as Activity.DueDate if a demand.)		
Scheduled Receipt	If this record represents a supply from a ScheduledReceipt, then this field is a reference to that ScheduledReceipt. If this record represents a demand that is an allocation from a ScheduledReceipt, then this field is a reference to that ScheduledReceipt. In all other situations, this field is NULL. **Referenced table:** ScheduledReceipt	Reference	
ShipDate	The date the supply should be shipped from its supplier to meet ReschedDate. Copied from PlannedOrder or ScheduledReceipt. D'undefined' for other types of Activity records.	Date	
StartUnit	The start unit that is applicable to this supply or demand. The field is populated from the source supply or demand record regardless of the value for Part.MUEPoolNettingType.UnitRule.	Integer	
StockUnit Quantity	Represents the supply quantity in inventory unit of measure (UOM) without applying yield or scrap factors. **Note**: **SupplierUOM** applies to **StockUnitQuantity**.	Quantity	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
Supplier	Reference to the name of the supplier. The supplier of the part. NULL if the supply is from on hand, or if the activity is a demand. **Referenced table:** Supplier	Reference	
SupplyQty	For supplies, the total quantity of the supply (DemandQty will be the received quantity). For demands, the quantity that has been satisfied (shipped or consumed forecast). If the ExtendedType field is set to "Expiring", then this field is always set to zero (0).	Quantity	
SupplyQtyUnitOfMeasure	Reference to the unit of measure used for the supply quantity that is associated with this record. **Referenced table:**"UnitOfMeasure table" on page 931. **Note**: This field was added in Version 2016.2, Service Update 12.	Reference	
SupplyType	The SupplyType associated with this record.	String	
Synchronized Date	This field synchronizes the consumption of components based on the parent supply. In some inventory calculations, usage of this date is therefore more appropriate than **DueDate**. Calculated as follows: • For dependent demands, this is the date where the demand is required and expected to be consumed, based on the converting the parent supply's **FirstAvailableDate** to a **StartDate**. • For independent demands and unconsumed forecast, this is the demand AvailableDate. • For supplies, it is the **AvailableDate**.	Date	
Transaction Sequence	Sequence from supply or demand. 0 if not relevant or no corresponding Transaction Sequence field in the originating table.	Integer	

Table 8.22: CTPActivity (Mfg) fields

Field name	Description	Data type	Owner
Type	Identifies the type of activity record. Values are: • Allocation • AvailableToPromise (not used) • CoByProductSupply • ConsensusForecast • DynamicSafety (not used) • Forecast • IndependentDemand • InventoryTransfer • InventoryTransferAllocation • NewAllocation • NewTransferAllocation • OnHand • PlannedAllocation • PlannedOrder • PlannedTransferAllocation • PushTransferOut—Indicates that this demand transaction is an action indicating that the quantity should be pushed out early to the destination site in order to respect the maximum inventory level. This type applies only to source parts that are enabled for push planning (have **DistributionPlanningRule.ProcessingRule** set to "Enable". If those parts also have **DistributionPlanningRule.SupplyShareRule** set to "All", then this type will be found on all actions representing transfers from the source site to destination site. **Note:** This value was added in Version 2016.2. • ScheduledReceipt • SubstituteSupply • SubstituteAllocation	String (Enum)	

CTPCoByProductSupply table

The CTPCoByProductSupply table reports capable-to-promise calculations, such as available dates and on time quantities, for co-product and by-product supplies. Co-products are those that are manufactured together with a primary product and typically have similar components and process similarities as the primary product. By-products are those that are generated as a result of, and residual to, production of the primary product.

Table 8.23: CTPCoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
AvailableDate	The date when this supply will be available. This is calculated starting from the available date of the primary product supply that generated this co-product or by-product, and then adjusted by the **BillOfMaterial.LeadTimeOffset** value, if any, in the record that defines this co-product or by-product relationship.	Date	
CoByProduct Supply	A reference to the CoByProductSupply record that is associated with this CTP record. **Referenced table:** CoByProductSupply	Reference	
Expiring Quantity	The amount of this supply that will not be used before it expires (reaches its ExpiryDate and is no longer usable).	Quantity	
ExpiryDate	The date when this supply will expire. This is calculated starting from the expiry date of the primary product supply that generated this co-product or by-product, and then adjusted by some number of the primary products expiry calendar intervals. This adjustment, if any, is made in terms of the **BillOfMaterial.IntervalToExpiryOffset** value in the record that defines this co-product or by-product relationship.	Date	
ExpiryStart Date	Date on which to start supply expiry date calculations for assemblies having a **PartType.ExpiryRule** of "DependentStart" (based on earliest ExpiryStartDate rolled up from a component supply). This field is calculated for all expiring primary parts that are either designated as limiting components or contain limiting components in their product structure. Otherwise, this field is set to "Future". For more information about how supply expiry start dates are used, see "DependentStart" on page 1618.	Date	
FirstAvailable Date	If incremental availability is turned on for the part, this represents the date the first portion of the supply could be available. If incremental availability is turned off for the part, this is always the same as AvailableDate.	Date	
NewProcess Cost	Standard labor and overhead cost to complete this supply. Cost values used in this calculation are taken from the part source used in production of the primary product supply that generated this co-product or by-product. Cost is calculated by the percentage associated with this co-product or by-product in relation to the primary product. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	

Table 8.23: CTPCoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
NewPurchase Cost	Cost of new purchases to complete this supply. Cost values used in this calculation are taken from the part source used in production of the primary product supply that generated this co-product or by-product. Cost is calculated by the percentage associated with this co-product or by-product in relation to the primary product. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
OnTime Quantity	The amount of this supply that will be available by its due date.	Quantity	
Part	A reference to the co-product or by-product part that this supply is for. **Referenced table:** Part	Reference	Yes
PrimaryPart	A reference to the primary product whose planned order or scheduled receipt generated this co-product or by-product supply. **Referenced table**: Part	Reference	
PrimaryCTP PlannedOrder	A reference to the CTPPlannedOrder record associated with the primary product's planned order that generated this co-product or by-product. Null if the primary product supply is a scheduled receipt. **Referenced table:** CTPPlannedOrder	Reference	
Primary Scheduled Receipt	A reference to the primary product's scheduled receipt that generated this co-product or by-product. Null if the primary product supply is a planned order. **Referenced table:** ScheduledReceipt	Reference	

CTPForecast table

The **CTPForecast** table reports capable-to-promise calculations, such as available dates and on time quantities, pertaining to forecasted demands generated during the sales and operations planning process in RapidResponse. That is, this table only contains availability details of forecasts based on values in the **ForecastDetail** table and used in generating the consensus forecast (as reported in the **ConsensusForecast** table and in **Forecast** table records with a ForecastSource value of "ConsensusForecast").

This table is not shown on the *RapidResponse Calculated Data Model* poster.

Table 8.24: CTPForecast (Mfg) fields

Field name	Description	Data type	Owner
AvailableDate	The date at which supply is available to fully satisfy this forecasted demand.	Date	
DaysLate	The number of days late (in working days) this demand is expected to be.	Quantity	
Forecast	A reference to the forecast whose availability is reported in this record. Only Forecast records with a ForecastSource value of "ConsensusForecast" are referenced from this table. **Referenced table:** Forecast	Reference	
OnTime Quantity	The amount of this demand which can be satisfied on time.	Quantity	
Part	A reference to the part this forecast is for. **Referenced table:** Part	Reference	Yes

CTPForecastCost table

The **CTPForecastCost** table reports date-based cost of goods sold and material spending values for forecasts generated by the S&OP process in RapidResponse (that is, those forecasts whose details are reported in the **CTPForecast** tables). For a given forecast, this table contains one record for each date on which any component supplies pegged to the forecast in the **WhereConsumedForForecast** table are due.

This table is not shown on the *RapidResponse Calculated Data Model* poster. It is meant to replace the now obsolete **CostOfGoodsSoldForForecast** table which generated only one summary record per forecast. As a result, in cases where component supplies due on different dates were required to satisfy a given forecast, the **CostOfGoodsSoldForForecast** table would potentially give incorrect results if used in multi-currency environments.

Table 8.25: CTPForecastCost (Mfg) fields

Field name	Description	Data type	Owner
ConversionDate	The particular date to which the cost values reported on this record apply. Each forecast has one record added this table for each unique date on which one or more required component supplies are due based on the results in the **WhereConsumedForForecast** table. This date then becomes effective date for any related currency conversion calculations to ensure the appropriate date-sensitive rates are used.	Date	
CostOfGoods Sold	The total costs of goods sold from all component supplies due on this date and used in satisfying the referenced forecast. For a given date and set of component supplies pegged to this forecast in the **WhereConsumedForForecast** table, the **LaborCost**, **MaterialCost**, **OverheadCost**, and **OverheadCost2** fields are summed together to populate this field.	Money	
CTPForecast	A reference to the capable-to-promise details for the forecasted demand whose cost of goods sold and associated material spending is reported in this record. Details of the **Forecast** record itself can also be accessed through this reference. **Referenced table:** CTPForecast	Reference	
MaterialSpend	Total rolled-up material spending associated with component supplies due on this date and used in satisfying the referenced forecast. For any scheduled receipts or planned orders for purchased parts that are pegged to this demand in the **WhereConsumedForForecast** table, the **MaterialCost** field values are summed together to populate this field.	Money	
Part	A reference to the part associated with the forecasted demand whose cost of goods sold and associated material spending is reported in this record. **Referenced table**: Part	Reference	Yes

CTPPlannedOrder table

This table stores capable-to-promise information for planned orders based on things such as component and constraint availability. For example, you can determine if a supply is late due to lack of constraint or component materials by comparing the value of the MaterialAvailableDate field with the value of the AvailableDate field.

Table 8.26: CTPPlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
AvailableDate	The date this supply could be available based on materials, Constraint, and due date.	Date	
AvailableStart Date	The Start date calculated from earliest constraint allocation. If the incremental availability option is turned off for the supply, then all components must be available before any constraint be allocated. If the incremental availability option is turned on for the supply then only some set of components need be available before any constraint can be allocated. **NOTE**: If there is no active constraint, this date is calculated from AvailableDate. **NOTE**: If fixed constraint is required (**SourceConstraint.FixedConstraintFactor > 0**) and the amount of that fixed requirement exceeds the total constraint immediately available, then this date can end up being later than the earliest constraint allocation. In such situations, the total fixed constraint required might need to collected over several days/periods before the supply can start.	Date	
Constraint	The Constraint whose insufficient remaining capacity will make the entire supply not available for its MaterialAvailableDate. If this supply is not late, this field is empty. **Referenced table:** Constraint	Reference	
Expiring Quantity	The amount of this supply that will not be used before it expires (reaches its ExpiryDate).	Date	
ExpiryDate	The date on which the material in this supply expires (is no longer usable).	Date	
ExpiryStartDate	Calculated date on which to start supply expiry date calculations for assemblies having a **PartType.ExpiryRule** of "DependentStart". This field is calculated for all expiring parts that are either designated as limiting components or contain limiting components in their product structure. Otherwise, this field is set to "Future". For more information about how supply expiry start dates are used, see "DependentStart" on page 1618.	Date	
FirstAvailable Date	The date the first portion of this supply is available. This is different than the AvailableDate field in cases where the planned order has been split into multiple periods due to constraint consumption or incremental availability of component materials.	Date	

Table 8.26: CTPPlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
Material Available Date	Date when all materials are available (MaterialStartDate) converted to a Stock date.	Date	
MaterialStart Date	Date when all components are available for the entire supply. For work orders, this is based on component availability. For purchase orders, this is the same as StartDate.	Date	
NewProcess Cost	Standard labor and overhead cost of planned orders to complete this order. Labor and overhead costs values are taken from the applicable **PartSource** records or, the applicable **TimePhasedPartSourceAttributes** records (if they exist). Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
NewPurchases Cost	Cost of new purchases required in order to complete this order (through all levels of the bill). Note that fields used to calculate the various purchase values for supplies is determined by the setting in the **PartSourceType.CostRule** field. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
OnTime Quantity	The quantity of the entire planned order that could be available for DueDate, based on lead time and component availability only.	Quantity	
Part	The part this record relates to. **Referenced table:** Part	Reference	Yes
PlannedOrder	The planned order applicable to this record. **Referenced table:** PlannedOrder	Reference	

CTPSafetyStockHoldings table

The **CTPSafetyStockHoldings** table reports details of supply allotments that are available to support safety stock levels. For example, allotments of supply that have been temporarily held from their demands in order to ensure minimum safety stock levels are not violated (or are minimally violated if doing so cannot be avoided).

This table was added in Version 2016.2, Service Update 18. It is only populated for parts' where **SafetyStockPolicy.CTPHoldRule** is set to "Use" (where supply allotments can be temporarily held from their demands in order to protect higher priority safety stock requirements). For more information about held allotments and prioritized safety stock, see "Priority and safety stock holds in CTP" on page 1780.

Table 8.27: CTPSafetyStockHoldings(Mfg) fields

Field name	Description	Data type	Owner
Date	A date on which there is a supply allotment available towards maintaining the safety stock level. Records are generated on dates on which there is a change in the quantity of supply allotments available to support the safety stock level, as well as dates on which there is a change in the specific supply allotments that support a given safety stock level.	Date	
OrderPriority	The priority of the safety stock requirement for which the supply allotment was held. This is either the **OrderPriority** referenced on a **TimePhasedSafety** record for the part, or else the part's **PartType.DefaultPriority**. Note that this will always be a higher priority than that found on any actual demand whose allotment is held to respect the safety stock requirement. **Referenced table**: OrderPriority	Reference	
Part	The part with a supply allotment that has been held. **Referenced table**: Part	Reference	Yes
SupplyDemand	A reference to a **SupplyDemand** allotment that supports safety stock. The following types of allotments will be reported: • supply allotments made to actual demands which are then held to avoid violating a higher priority safety stock requirement. Held allotments are available between **HoldStartDate** and **HoldEndDate**. • supply that is being pushed (transferred) from a sourcing location to a distribution site in respect of the part's maximum inventory level. Pushed allotments are available between **SupplyAvailableDate** and **PushDate**. • supply allotments made to actually satisfy a safety stock requirement (records where **DemandSource** is set to "None"). **Referenced table**: SupplyDemand	Reference	

Note: A worksheet based on this table that grouped by **Part** and **Date** (and optionally **OrderPriority**), and then summed by the referenced **SupplyDemand.AllocatedQty** values, can show all supply available to support safety stock as of a given date.

CTPSubstituteSupply table

The **CTPSubstituteSupply** table shows the results of capable-to-promise calculations where supply of a substitute part was used to satisfy demand for a primary part.

Note that this table shows the results of global part substitute calculations only (it does not consider BOM-level part substitution).

Table 8.28: CTPSubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
AvailableDate	The date this supply should be available.	Date	
ExpiringQuantity	The amount of this supply that will not be used before its expiry date.	Date	
ExpiryDate	The date that this supply will expire.	Date	
ExpiryStartDate	Date on which to start supply expiry date calculations for assemblies having a **PartType.ExpiryRule** of "DependentStart" (based on earliest ExpiryStartDate rolled up from a component supply). This field is calculated for all expiring substitute parts that are either designated as limiting components or contain limiting components in their product structure. Otherwise, this field is set to "Future". For more information about how supply expiry start dates are used, see "DependentStart" on page 1618.	Date	
FirstAvailableDate	If incremental availability is turned on for the part, this represents the date the first portion of the supply could be available. If incremental availability is turned off for the part, this is the same as AvailableDate.	Date	
NewProcessCost	Standard labor and overhead cost rolled up from lower levels. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
NewPurchaseCost	New purchase costs rolled up from lower levels. Currency conversions on this field use the **DueDate** field's conversion rate.	Money	
OnTimeQuantity	The amount of the supply that could be available for the DueDate (based on component availability only).	Quantity	
Part	The primary part that needed supply from a substitute part. **Referenced table:** Part	Reference	Yes
SubstituteSupply	A reference to the related substitute supply record which provides more details on the supply and the primary part's demand it satisfies. **Referenced table:** SubstituteSupply	Reference	

DataChange table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates changes made to data in a given scenario as well as changes made in its parent that the scenario has not yet been updated with.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Note: User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.

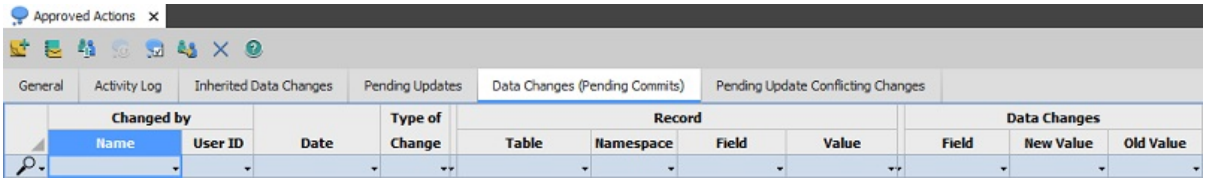
The DataChange table is used in the Changes to Root Scenario worksheet in the Administration Log workbook.

Changes to Root Scenario worksheet

RapidResponse 2016.2 introduced three tables that are almost identical to the DataChange table. Each of these tables run faster than the original DataChange table and are less memory intensive. The three tables each support a worksheet in the Scenario properties window.

- **Inherited Data Changes** tab (DataChange_Historical table)
- **Pending Updates** tab (DataChange_PendingUpdate table)
- **Data Changes (Pending Commits)** tab (DataChange_Local table)

Figure 8.1 Scenario properties - Data Changes (Pending Commits)



Using the DataChange tables and worksheets can help users better understand differences between a parent scenario and a child scenario. For example, suppose you are ready to commit data changes to a parent scenario. For each scenario in your Scenarios pane, the changes in the scenario can be viewed along with details of each change, such as the user who made the change, the type of action that resulted in the change, and the field values affected by the change. These changes are viewed through the Data Changes (Pending Commits) worksheet in the Scenario Properties tab.

Notes:

- For fields in this table containing delimited lists of values, the pipe character (|) is used to separate the values. If necessary, the delimiter character can be changed. Typically, this would be done if the delimiter character is used in your data set. For more information, see "Configuring data change tracking" in the *RapidResponse Administration Guide*.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.
- A data import deletes all existing data change records.

Table 8.29: DataChange fields

Field name	Description	Data	Owner
ChangeDate Time	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	

Table 8.29: DataChange fields

Field name	Description	Data	Owner
DataChange Source	Indicates the source of this change (for example, is the change in this scenario or its parent). Valid values are: • **Historical**—A change that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent. • **Local**—A change that occurred in this scenario (or was committed into this scenario). • **PendingUpdate**—A change made in the parent scenario, that has not yet been updated into this scenario.	String (Enum)	
Identification FieldNames	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	
Identification FieldValues	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalField Values	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`.	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	

Table 8.29: DataChange fields

Field name	Description	Data	Owner
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
Table	The name of the table where the change was made.	String	
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_Historical table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent.

This table is almost identical to the DataChange table except that it runs faster and is less memory intensive since only one type of source of change is reported (Historical). This table supports the Inherited Data Changes worksheet tab in the Scenario properties window.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.30: DataChange_Historical fields

Field name	Description	Data type	Owner
ChangeDate Time	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	
DataChange Source	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **Historical**—A change that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent.	String (Enum)	

Table 8.30: DataChange_Historical fields

Field name	Description	Data type	Owner
Identification FieldNames	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	
Identification FieldValues	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalField Values	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
Table	The name of the table where the change was made.	String	

Table 8.30: DataChange_Historical fields

Field name	Description	Data type	Owner
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_Historical_Split table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent.

This table is almost identical to the DataChange_Historical table except that it splits each identifying field and modified field from the DataChange_Historical table into a separate record. This reports the data changes in the Scenario properties window, and does not require an additional processing step to parse the DataChange record values into the worksheet format.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data

change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.31: DataChange_Historical_Split fields

Field name	Description	Data type	Owner
ChangeDate Time	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	
DataChange Source	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **Historical**—A change that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent.	String (Enum)	
Identification FieldNames	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	

Table 8.31: DataChange_Historical_Split fields

Field name	Description	Data type	Owner
Identification FieldValues	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalField Values	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
Line	The line number of the record split from the DataChange_ Historical table.	Integer	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
SplitGroupSequence	The sequence number assigned to each data change. All lines for a single change are assigned the same group sequence number, and the group represents a single row from the DataChange_PendingUpdate table.	Integer	
Table	The name of the table where the change was made.	String	

Table 8.31: DataChange_Historical_Split fields

Field name	Description	Data type	Owner
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_Local table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates a change that occurred only in this scenario (or was committed into this scenario).

This table is almost identical to the DataChange table except that it runs faster and is less memory intensive since only one type of source of change is reported (Local). This table supports the Data Changes (Pending Commits) worksheet tab in the Scenario properties window.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data change records after 45 days. However, the frequency in which these records are deleted can

be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.32: DataChange_Local fields

Field name	Description	Data	Owner
ChangeDateTime	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	
DataChangeSource	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **Local**—A change that occurred in this scenario (or was committed into this scenario).	String (Enum)	
IdentificationField Names	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	
IdentificationField Values	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	

Table 8.32: DataChange_Local fields

Field name	Description	Data	Owner
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalFieldValues	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
Table	The name of the table where the change was made.	String	
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_Local_Split table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates a change that occurred only in this scenario (or was committed into this scenario).

This table is almost identical to the DataChange_Local table except that it splits each identifying field and modified field from the DataChange_Local table into a separate record. This reports the data changes in the Scenario properties window, and does not require an additional processing step to parse the DataChange record values into the worksheet format.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.33: DataChange_Local_Split fields

Field name	Description	Data	Owner
ChangeDateTime	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	

Table 8.33: DataChange_Local_Split fields

Field name	Description	Data	Owner
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	
DataChangeSource	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **Local**—A change that occurred in this scenario (or was committed into this scenario).	String (Enum)	
IdentificationField Names	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	
IdentificationField Values	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalFieldValues	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
Line	The line number of the record split from the DataChange_ Local table.	Integer	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	

Table 8.33: DataChange_Local_Split fields

Field name	Description	Data	Owner
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
SplitGroupSequence	The sequence number assigned to each data change. All lines for a single change are assigned the same group sequence number, and the group represents a single row from the DataChange_PendingUpdate table.	Integer	
Table	The name of the table where the change was made.	String	
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_PendingUpdate table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes made in the parent scenario, that have not yet been updated into this scenario.

This table is almost identical to the DataChange table except that it runs faster and is less memory intensive since only one type of source of change is reported (PendingUpdate). This table supports the Pending Updates worksheet tab in the Scenario properties window.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no

modification record is created (as long as there have been no intervening scenario operations involving that scenario).

- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.34: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
ChangeDateTime	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	

Table 8.34: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
DataChangeSource	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **PendingUpdate**—A change made in the parent scenario, that has not yet been updated into this scenario.	String (Enum)	
IdentificationField Names	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	
IdentificationField Values	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalFieldValues	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	

Table 8.34: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
Table	The name of the table where the change was made.	String	
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

DataChange_PendingUpdate_Split table

This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes made in the parent scenario, that have not yet been updated into this scenario.

This table is almost identical to the DataChange_PendingUpdate table except that it splits each identifying field and modified field from the DataChange_PendingUpdate table into a separate record. This reports the data changes in the Scenario properties window, and does not require an additional processing step to parse the DataChange record values into the worksheet format.

The following types of changes are recorded in this table:

- **Insert**—for each record insertion in a scenario, a single record is added to this table. If an inserted record is subsequently modified, the insert record is updated with the new values and no modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Modify**—for each individual field value modified on a given record in a scenario, a single record is added to this table. If a single user makes multiple modifications to a given field value in a scenario, only one modification record is created (as long as there have been no intervening scenario operations involving that scenario).
- **Delete**—for each record deletion in a scenario, a single record is added to this table. When a record is deleted, any existing data change records pertaining to it (inserts or modifications) are also deleted. As well, any deletions that happen as a result of another deletion are not recorded.

Notes:

- User-initiated changes do not include adhoc data imports, data updates to a specific scenario, and automatic data modifications.
- RapidResponse automatically logs information about data change records in the DataChange table. Over time, these records can grow in size and potentially consume significant amounts of memory. By default, RapidResponse automatically deletes data

change records after 45 days. However, the frequency in which these records are deleted can be configured by an administrator. For more information, see the *RapidResponse Administration Guide*.

Table 8.35: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
ChangeDateTime	The date and time when the change to the scenario was made.	DateTime	
ChangedField Names	Delimited list of the fields affected by this change. The field names that display depend on the type of change as follows: • **Insert**—a delimited list of the field names from the record that is added. Usually a subset of all the field names on the record. • **Modify**—a delimited list of the names of the fields that have changed. • **Delete**—this field is blank.	String	
ChangedField Schema	Schema of value fields for changes made against referenced tables.	String	
ChangedField Values	Delimited list of the new values that now exist in the fields identified in **ChangedFieldNames**.	String	
ChangeRecord Handle	Record handle of the internal change record in RapidResponse.	Integer	
Comment	This field is reserved for future use in RapidResponse.	String	
DataChangeSource	Indicates the source of this change (for example, is the change in this scenario or its parent). A valid value is: • **PendingUpdate**—A change made in the parent scenario, that has not yet been updated into this scenario.	String (Enum)	
IdentificationField Names	A delimited list of fields from the table where the change occurred, that are used to identify the record that has changed. In a typical case, these are the fields which make up a table's primary key. An administrator can also define the fields used to identify records as discussed in "Configuring data change tracking" in the *RapidResponse Administration Guide*.	String	

Table 8.35: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
IdentificationField Values	A delimited list of values from the table where the change occurred and used to identify the record that has changed. These values correspond to the fields listed in **IdentificationFieldNames**.	String	
Origin	Always returns the value **User**. This field is maintained for compatibility purposes and can be ignored.	String (Enum)	
OriginalFieldValues	For changes of the type **Modify**, a delimited list of the original values from the changed fields (that is, those fields listed in **ChangedFieldNames**).	String	
Line	The line number of the record split from the DataChange_ PendingUpdate table.	Integer	
ScenarioPath	The name of the scenario where the changes have occurred. The notation of the scenario is based on the full path from the root scenario. The path levels are noted using **/~/**. For example, the full path for the Child scenario of Enterprise Data is noted as `/~/Enterprise Data/~/Child/~/`	String	
Sequence	When combined with **ChangeDateTime**, ensures that all changes resulting from a single user action can be grouped together. For example, if 10 records are modified as part of a single change action, they are all assigned the same **Sequence** (and **ChangeDateTime** value).	Integer	
Site	The corresponding site name for the data changes. **Referenced table**: Site	Reference	
SourceRecord Handle	Record handle associated with the record affected by the change (insertions and modifications only).	Integer	
SplitGroupSequence	The sequence number assigned to each data change. All lines for a single change are assigned the same group sequence number, and the group represents a single row from the DataChange_PendingUpdate table.	Integer	
Table	The name of the table where the change was made.	String	

Table 8.35: DataChange_PendingUpdate fields

Field name	Description	Data	Owner
Type	Indicates the type of change that occurred. Valid values are: • **Insert**—a new record was added. • **Modify**—an existing record was changed. • **Delete**—an existing record was deleted	String (Enum)	
User	The User ID of the user who made the change.	String	
UserRecord	A reference to the record that identifies the user who made the change. This is either the same value as User, or it might be NULL if the user has been deleted. **Referenced table**: User	Reference	

Demand table

The **Demand** table is a consolidation table that combines Allocation, IndependentDemand and DependentDemand together for a total demand picture for a part. There is one record for each demand schedule (quantity and due date) per given demand order (planned or firm). This table includes only those demands that are processed by Netting calculations (that is, it excludes demands where date Date is 'Undefined', or where OrderType.ProcessingRule is 'Ignore').

The **Demand** table also reports forecast. The interpretation of TotalQuantity and RemainingQuantity are extended to include forecast, after spreading.

Table 8.36: Demand (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The part the demand activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input demand data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part where the demand came from. **Referenced table**: Part	Reference	
DueDate	The date on which the demand is scheduled to be satisfied.	Date	

Table 8.36: Demand (Mfg) fields

Field name	Description	Data type	Owner
EarliestExpiry Date	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
Forecast	A reference to the Forecast record associated with this demand. NULL unless the demand comes from a forecast. **Referenced table:** Forecast	Reference	
Independent Demand	Reference to the IndependentDemand order, NULL if Demand is not an IndependentDemand. **Referenced table:** IndependentDemand	Reference	
IsDependent	Valid values are: • Y • N If 'Y', demand is a dependent demand, 'N' if it is an IndependentDemand or ConsensusForecast.	Boolean	
Inventory Transfer	The Inventory Transfer that is the source of this demand. **Referenced table:** InventoryTransfer	Reference	
Model	The model used when netting this record. **Referenced table:** Model	Reference	
Order	The supply order number Id if this is an Allocation. The demand order number Id if this is an IndependentDemand. Blank if it is a PlannedAllocation.	String	
Part	The part that has the demand. **Referenced table:** Part	Reference	Yes
PegModel	The model that is applicable to this demand. The field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.ModelRule. **Referenced table:** Model	Reference	
PegPart	If the demand is dependent demand (for example, IsDependent= 'Y'), PegPart is the assembly being made by the planned order or scheduled receipt. If the demand is from independent demand or from forecast, PegPart is NULL **Referenced table:** Part	Reference	

Table 8.36: Demand (Mfg) fields

Field name	Description	Data type	Owner
PegPool	The pool that is applicable to this demand. This field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.PoolRule. **Referenced table:** Pool	Reference	
PegSR	Reference to the ScheduledReceipt if this is an allocation, NULL if otherwise. **Referenced table:** ScheduledReceipt	Reference	
Pool	The pool used when netting this record. **Referenced table:** Pool	Reference	
PromisedDate	The date the part was promised to be available to (DueDate for dependent demands).	Date	
Remaining Quantity	The amount that is still needed. If the Demand is Forecast, this field shows the unconsumed portion of the forecast after spreading.	Quantity	
ReschedDate	The Date MRP moved the DueDate to (may be different from DueDate on dependent demands).	Date	
StartUnit	The start unit that is applicable to this demand. The field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.UnitRule.	Integer	
Substitute Supply	The substitute supply that creates the dependent demand **Referenced table:** SubstituteSupply	Reference	
TotalQuantity	The amount that was needed. If the Demand is Forecast, this field shows the total forecast (either dependent or independent) after spreading.	Quantity	
Type	Policies for processing this demand. **Referenced table:** DemandType	Reference	

Table 8.36: Demand (Mfg) fields

Field name	Description	Data type	Owner
UnitSelling Price	The per unit price for this order. If the demand is an independent demand (excluding forecast), then the IndependentDemand.EffectiveUnitPrice is used. For all other demand types (including forecast) the Part.AverageSellingPrice is used. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following case: • If the demand that drives this cost is an inventory transfer, the date from the **ShipDate** field of the InventoryTransfer record is used.	Money	
Unsatisfied Quantity	The portion of this demand that was left unsatisfied.	Quantity	

DemandException table

The **DemandException** table reports demands that were skipped and left unsatisfied either because RapidResponse could not plan supply to meet a part's material expiry requirements, or because the demand date (less safety lead time on the part) fell outside of the effectivity window for all part sources and the part's **SourceRule.DateEffectivityRule** setting was "AdjustedDemandDate", or because multi-level search logic determined that an assembly's components were not expected to be available within the effective date range defined for the assembly's part source(s).

For example, when RapidResponse attempts to generate a planned order to satisfy demand for an expiring part, it does as long as it can find a source where the estimated expiry date for the order is on or after the earliest expiry date for the demand. If it cannot find such a source, the demand is skipped and left unsatisfied. This might occur, for example, if the MinimumShelfLife value defined on the demand/part is greater than the IntervalsToExpiry value defined on the part source.

This table is therefore useful for identifying any demands which were left unsatisfied so that corrective action can subsequently be taken to adjust the supply plan and try to satisfy those demands. For more information about expiry calculations, see "Material expiration" on page 1601. For more information about part source effectivity, see "Part source selection" on page 1377. For more information about multi-level search calculations, see "Multi-level search logic" on page 1863.

Table 8.37: DemandException (Mfg) fields

Field name	Description	Data type	Owner
Constrained ExpiryDate	The latest estimated expiry date associated with any potential planned order that could be sourced from an eligible part source for this part. This date represents the best case scenario for being able to source the demand to meet expiry requirements, but will still be before the EarliestExpiryDate for the demand this exception applies to. This field is left Undefined for exceptions caused by part source effectivity or multi-level source effectivity.	Date	
Date	The date associated with the demand exception for this part. For exceptions caused by material expiry, this typically represents the due date of the demand that was left unsatisfied for the part. For exceptions caused by multi-level part source effectivity, this represents the earliest date that multi-level sourcing logic determined a supply could be planned for the demand (in consideration of component availability and constraint under the assembly). **Note:** The Type field indicates the cause of each demand exception.	Date	
EarliestExpiry Date	The earliest expiry date for the demand that could not be sourced. This represents the date before which any supply cannot expire if it is to be used in satisfying this demand (or demands). This field is left Undefined for exceptions caused by part source effectivity or multi-level source effectivity.	Date	

Table 8.37: DemandException (Mfg) fields

Field name	Description	Data type	Owner
DemandSource	The type of demand that was left unsatisfied due to either material expiry requirements or multi-level source effectivity. This information is useful in helping to track down the demand that was skipped and left unsatisfied. Valid values are: • **Allocation**—input dependent demand from a parent scheduled receipt. • **ConsensusForecast**—consensus forecast demand originating in the ForecastDetail table (other exceptions from forecast are reported under the type of demand they belong to, such as IndependentDemand). • **IndependentDemand**—independent demand for this part (including forecast). • **InventoryTransfer**—inventory transfer from a parent part. • **NewAllocation**—calculated dependent demand from a parent scheduled receipt. • **NewTransferAllocation**—calculated dependent demand from a from a parent transfer scheduled receipt. • **None**—demand to satisfy safety stock. • **OnHand**—demand originating from a negative OnHand record. • **PlannedAllocation**—dependent demand from a parent planned order. • **PlannedTransferAllocation**—dependent demand from a parent transfer planned order. • **SubstituteAllocation**—demand from a substitute allocation.	String (Enum)	
Model	The model, if any, associated with the demand(s) that could not be sourced. **Referenced table:** Model	Reference	
OrderPriority	The order priority associated with the demand(s) that could not be sourced. **Referenced table:** OrderPriority	Reference	
Part	The part whose demand could not be sourced due to material expiry limitations, part source effectivity or multi-level source effectivity. **Referenced table:** Part	Reference	Yes

Table 8.37: DemandException (Mfg) fields

Field name	Description	Data type	Owner
Pool	The pool, if any, associated with the demand(s) that could not be sourced. **Referenced table:** Pool	Reference	
Quantity	The amount of the demand that could not be sourced due to material expiry , part source effectivity, or multi-level source effectivity requirements. This value might be less than any single demand quantity. For example, if there is non-expiring supply available to satisfy part of a given demand then only the unsourceable portion of the demand would be reported in this table. Similarly, this value might be greater than any single demand quantity. For example, if a set of demands for a part existed on the same date and with the same priority, they might be processed together by RapidResponse.	Quantity	
Type	Indicates the cause of the demand exception being reported. Exceptions in this table are generated from either material expiry, part source effectivity, or multi-level sourcing logic. Valid values are: • **Expiry**—the exception was generated because non-expiring supply could not be found or planned for a part's demand. • **MLSSourceEffectivity**—the exception was generated because the date when supply could be planned for a part's demand, based on the planned availability and constraint on its components, is outside of the effective date range for the assembly's part source(s). This type of exception is only applicable to parts configured for multi-level sourcing and where **MultiLevelSearchRule.SourceEffectivityRule** is set to"PlannedAvailableDate". • **SourceEffectivityWindow**—the exception was generated because the demand date (less any safety lead time on the part) fell outside of the effectivity window for all part sources and the part's **SourceRule.DateEffectivityRule** setting was "AdjustedDemandDate". **Note:** This field was added in Version 2016.2, Service Update 4, and updated in Version 2016.2, Service Update 8.	String (Enum)	
Unit	The start unit associated with the demand that could not be sourced. Default value is -1.	Integer	

DependentConsensusForecast table

The **DependentConsensusForecast** table shows the result of exploding consensus forecasts through a planning BOM and hence the dependent demands that would be placed on the assembly's option components. These demands are not used in other planning (for example, they are not reported in CTPForecast or Forecast tables), but rather are intended for comparison against the statistical forecasts generated for the planning BOM components. For example, if the dependent forecast values generated here are significantly different from the component's statistical forecasts, this might indicate issues with the defined planning BOM ratios.

Table 8.38: DependentConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
BOM	A reference to the bill of material relationship that drove the dependent consensus forecast reported on this record. This table only calculates dependent consensus forecasts from **BillOfMaterial** records that satisfy the following conditions: • **Type.RatioRule** is either "Fraction" or "Percent". • **Assembly.Type.ProcessingRule** is either "MPS", "MPSConfig", or "Ignore". • **Component.Type.ProcessingRule** is "MPS". **Referenced table:** BillOfMaterial	Reference	
Date	The date of this dependent consensus forecast. Based on the original consensus forecast date (**ParentConsensusForecast.Date**) and offset by the lead time for the assembly's primary "Make" source (**ParentPartSource.EffLeadTime**) as well as any applicable lead time offset defined on the bill of material (**BOM.LeadTimeOffset**).	Date	
ParentConsensus Forecast	A reference to the consensus forecast generated for the assembly during the sales and operations planning process that led to the dependent forecast reported on this record. **Referenced table:** ConsensusForecast	Reference	
ParentPartSource	A reference to the primary "Make" part source for the assembly that the original consensus forecast pertains to. Used to retrieve the appropriate lead time values for calculating the dependent forecast date. **Referenced table:** PartSource	Reference	
Part	A reference to the component part that this dependent forecast was generated for. **Referenced table:** Part	Reference	Yes

Table 8.38: DependentConsensusForecast (Mfg) fields

Field name	Description	Data type	Owner
PartCustomer	A reference to the part and customer forecast this dependent forecast pertains to. Set to the record in the **PartCustomer** table that references the component part the dependent forecast is for and customer on the original consensus forecast for the assembly. Null if no such record exists. **Referenced table**: PartCustomer	Reference	
Quantity	The amount of dependent consensus forecast placed on the component. Based on the assembly's original consensus forecast quantity and then adjusted for factors such as quantity per, option ratio, and so on.	Quantity	

DependentDemand table

This is a consolidation table that combines **Allocation**, **NewAllocation** and **PlannedAllocation** tables to form dependent demand. Only records that are processed by netting at the Part are included. Any demand from the part's WhereUsed bills that are blown through will not be included in the dependent demand for this part. It will appear in the dependent demand of a lower part. This table shows the Unconsumed portion as RemainingQuantity if reporting dependent forecast.

Table 8.39: DependentDemand (Mfg) fields

Field name	Description	Data type	Owner
Allocation	Allocation creating this dependent demand. **Referenced table**: Allocation	Reference	
AlternatePart	The part the demand was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part where the demand came from. • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part where the demand came from. **Referenced table**: Part	Reference	
BillOfMaterial	The BillOfMaterial record used to create dependent demand. (Null if from an Allocation.) **Referenced table:** BillOfMaterial	Reference	

Table 8.39: DependentDemand (Mfg) fields

Field name	Description	Data type	Owner
DueDate	The date on which the demand is scheduled to be satisfied.	Date	
EarliestExpiry Date	The earliest date that a supply may expire if it is to be used in satisfying this demand. The calculation of this field depends on the **PartType.ExpiryRule** value found on the assembly/parent part that generated this demand: • If the assembly has a **PartType.ExpiryRule** of "Dependent", then this value is inherited directly from the **EarliestExpiryDate** on the parent supply. • If the assembly has a **PartType.ExpiryRule** of "Self", then this value is calculated starting from the **DueDat**e on this dependent demand record and adding the assembly's **Part.MinimumShelfLife** value (in units of its ExpiryCalendar). • If the assembly has a **PartType.ExpiryRule** of "SelfReset", then this value is calculated starting from the **DueDate** on this dependent demand record and adding the component's **Part.MinimumShelfLife** value (in units of its ExpiryCalendar). **Note:** If the part this demand is for has a **PartType.ExpiryRule** of "Ignore", then this field always returns a value of "Future".	Date	
IsForecast	Valid values are: • Y • N This field is set to 'Y' if the dependent demand has Type.ProcessingRule= 'SalesForecast'.	Boolean	
Inventory Transfer	The inventory transfer creating this dependent demand. **Referenced table:** InventoryTransfer	Reference	
Model	The model used when netting this record. **Referenced table:** Model	Reference	
Part	The part that has the demand. **Referenced table:** Part	Reference	Yes
PegModel	The model that is applicable to this demand. The field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.ModelRule. **Referenced table:** Model	Reference	
PegPart	The part that this demand will go into. **Referenced table:** Part	Reference	

Table 8.39: DependentDemand (Mfg) fields

Field name	Description	Data type	Owner
PegPool	The pool that is applicable to this demand. This field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.PoolRule. **Referenced table:** Pool	Reference	
PegSource	The source of this dependent demand. Valid values are: • **PlannedAllocation**—the source supply is a planned order, exploded through a BillOfMaterial. PlannedOrder and BillOfMaterial references are valid. • **NewAllocation**—the source supply is a scheduled receipt, exploded through a BillOfMaterial. PegSR and BillOfMaterial references are valid. • **Allocation**—the source supply is an Allocation record for a ScheduledReceipt. Allocation and PegSR references are valid. • **TransferAllocation**—the source supply is a planned Transfer order. PlannedOrder is valid. • **Transfer**—The source supply is a Transfer scheduled receipt. PegSR is valid. • **InventoryTransfer**—the source supply is an inventory transfer. • **Substitute**	String (Enum)	
PegSR	The ScheduledReceipt that creates this dependent requirement (Null if planned). **Referenced table:** ScheduledReceipt	Reference	
PlannedOrder	The planned order that creates this dependent requirement (Null unless planned). **Referenced table:** PlannedOrder	Reference	
Pool	The pool used when netting this record. **Referenced table:** Pool	Reference	
Remaining Quantity	The amount that is still needed. If the Demand is Forecast, this field shows the unconsumed portion of the forecast. This is the sum of UnconsumedQty from Forecast records associated with this dependent forecast.	Quantity	
ReschedDate	The Date MRP moved the DueDate to (may be moved if the supply is rescheduled).	Date	
StartUnit	The start unit that is applicable to this demand. The field is copied from the demand being reported regardless of the value for Part.MUEPoolNettingType.UnitRule.	Integer	

Table 8.39: DependentDemand (Mfg) fields

Field name	Description	Data type	Owner
SubstituteSupply	The substitute supply that created the dependent demand. **Referenced table:** SubstituteSupply	Reference	
TotalQuantity	The amount that was needed. If the Demand is Forecast, TotalQuantity shows the forecast (either dependent or independent) before spreading and consumption.	Quantity	
Type	The type of the supply in which this demand will be used. The demand type can be determined from the SupplyType using the AllocationDemandType field. **Referenced table:** SupplyType	Reference	
Unsatisfied Quantity	The portion of this dependent demand that was left unsatisfied.	Quantity	

DirectAssembly table

The **DirectAssembly** table reports immediate parent assemblies that can potentially use a given component part (and hence might cause dependent demand to be placed on the component). Assembly parts reported for a given component are those ultimately related through standard BillOfMaterial records, co/by-product relationships, global and BOM-level substitute relationships, Allocation records, and transfer PartSource records.

This table was added in Version 2016.2, Service Update 3. If knowledge of indirect assemblies found multiple levels above the component is required, use the **FlatAssembly** table.

Table 8.40: DirectAssembly (Mfg) fields

Field name	Description	Data type	Owner
Assembly	The name of a direct parent assembly that can potentially use the component. **Referenced table:** Part	Reference	
Part	The name of the component part that can potentially be used in the building of the parent assembly. **Referenced table:** Part	Reference	Yes

DirectComponent table

The **DirectComponent** table reports all immediate components parts that can potentially be used directly in a given assembly. This includes components related to the assembly through standard BillOfMaterial records, co/by-product relationships, BOMAlternate records, input Allocation records, and transfer PartSource records.

This table was added in Version 2014.1

Table 8.41: DirectComponent (Mfg) fields

Field name	Description	Data type	Owner
Component	The name of a component that can potentially be used directly in the assembly part. **Note:** Each assembly will have a record identifying itself as its own component. **Referenced table:** Part	Reference	
Part	The name of the assembly part that can directly use the component. **Referenced table:** Part	Reference	Yes

DisaggregationRateByPartCustomer table

The **DisaggregationRateByPartCustomer** table reports general disaggregation rates for part-customer and forecast category combinations.

The rates in this table are typically calculated based on either the default parameters provided in the **SOPAnalyticsConfiguration** table or forecast category specific parameters provided in the **ForecastDisaggregationParametersByCategory** table or part-customer and forecast category specific parameters provided in the **ForecastDisaggregationParameters** table. Additionally, if the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "Actuals", the **ForecastDisaggregationOverride** table can be used to specify rates for a particular part-customer and forecast category on specific dates and these will then be reported in the **DisaggregationRateByPartCustomer** table, rather than calculating them for that part-customer and category on that date. If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "ConsensusForecast", the adjustments in **ForecastDisaggregationOverride** table and the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are not used.

Notes:

- If causal quantities are reported in the **CausalFactorDetail** table, they are reflected in this table.
- In order for this table to calculate disaggregation rates for a particular forecast category, the name of a valid **HistoricalDemandCategory** value must be passed to the table using an analytic parameter. For more information, see "Calculate general disaggregation rates by forecast category" on page 2236.

Table 8.42: DisaggregationRateByPartCustomer (Mfg) fields

Field name	Description	Data type	Owner
Category	A reference to the forecast category that the disaggregation rates reported for the part customer on this record pertain to. **Referenced table:** HistoricalDemandCategory	Reference	
Date	The date in the disaggregation calendar, which defines the periods that orders can be disaggregated into.	Quantity	
EffectiveUnitPrice	The unit price used as the rate for disaggregation by revenue. • RapidResponse checks the **CustomerPrice** table for records with matching Part and Customer values that have effective dates less than or equal to the date of this record. If any matching records are found, then the **UnitPrice** from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, record with the larger value in the **CustomerPrice.Id** field is used. • However, if no matching **Part** and **Customer** values are found, RapidResponse next check the **CustomerPrice** table for records with a matching **Part** and blank **Customer** value (that is, ' ') that have effective dates less than or equal to the record date. If any matching records are found, then the **UnitPrice** from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above or if the value provided in **CustomerPrice.UnitPrice** is negative, the value in **Part.AverageSellingPrice** is reported here. However, if the value in **Part.AverageSellingPrice** is also negative, then **DisaggregationRateByPartCustomer.EffectiveUnitPrice** is set to zero. However, if a **PartCustomer** is a component of an aggregate (has aggregates), then no **DisaggregationRateByPartCustomer** records are generated for it. If a **PartCustomer** is an aggregate (has components), then it needs to gather history from itself and all its components. The formula for **EffectiveUnitPrice** gives a weighted average of the components' **EffectiveUnitPrice**. In particular, it maintains the following relation for revenue: Revenue = **Quantity** X **EffectiveUnitPrice**.	Money	

Table 8.42: DisaggregationRateByPartCustomer (Mfg) fields

Field name	Description	Data type	Owner
Header	A reference to the part customer and forecast category combination this set of values applies to. **Referenced table:** HistoricalDemandHeader	Reference	
PartCustomer	A reference to the part and customer combination the disaggregation rate applies to. Note that if records are added to the **AggregatePartCustomer** table, then results are not generated in this table for component **PartCustomer** values (only rates for aggregate **PartCustomer** values as well as **PartCustomer** values that do not belong to aggregate part customer groupings are reported here). **Referenced table:** PartCustomer	Reference	Yes
Quantity	The quantity used as the rate for disaggregation by unit. The quantity is calculated based on historical demand actuals or the consensus forecast, which is determined by the value set in **HistoricalDemandCategoryType.DisaggregationQuantityRule**. If an override quantity is found in **ForecastDisaggregationOverride.Quantity**, it will be reported here.	Quantity	
QuantityWithoutOverride	The default quantity for disaggregation by unit that is calculated based on historical demand actuals or the consensus forecast. If an override quantity is found in **ForecastDisaggretationOverride.Quantity**, it is not reported here. **Note:** This field was added in Version 2016.2, Service Update 15.	Quantity	

EventConsensusForecastDetail table

The **EventConsensusForecastDetail** table is used for Event Management. It holds information about forecast items that have event-based forecast adjustments applied and also belong to forecast categories that are used to generate the consensus forecast. The records in the **EventForecastDetailAdjustment** table are used as a source for the records in this table. For more information, see "Event Management and the RapidResponse data model — calculated tables" on page 2120.

This table is analogous to the **ConsensusForecastDetail** table, which uses the **ForecastDetail** table as a source and reports only the forecast details needed to generate the consensus forecast. Both the **EventConsensusForecastDetail** table and the **ConsensusForecastDetail** table are used to generate the forecast in the **ConsensusForecast** table.

This table was added in Version 2016.2, Service Update 3.

Note: To avoid duplicating information, forecast details that are reported in the **EventConsensusForecastDetail** table are not included in the **ConsensusForecastDetail** table. This means that applying an event phase to the forecast might reduce the number of records in the **ConsensusForecastDetail** table.

Table 8.43: EventConsensusForecastDetail (Mfg) fields

Field name	Description	Data type	Owner
AverageUnit Price	The weighted average of **EffectiveUnitPrice** for **ForecastDetail** records with the given **Header**, for the specified date bucket. This value comes from the corresponding field in the **EventForecastDetailAdjustment** table.	Money	
AverageUnit PriceAdjustment	The calculated event-based unit price adjustment for this forecast item after all event phases that affect it are applied.	Money	
Consensus Forecast	The **ConsensusForecast** record that this record affects. **Referenced table:** ConsensusForecast	Reference	
Date	The date in the Header.PartCustomer.EffectiveDisaggregationCalendar that represents the forecast bucket that this adjustment applies to.	Date	
Effective Quantity	Effective quantity for this forecast item before event-based adjustments.	Quantity	
Effective Quantity Adjustment	The calculated event-based quantity adjustment for this forecast item after all event phases that affect it are applied.	Quantity	
EventForecast Detail Adjustment	The EventForecastDetailAdjustment record from which this record is derived. **Referenced table:** EventForecastDetailAdjustment	Reference	
Header	The HistoricalDemandHeader record that identifies the forecast stream (part, customer, and historical demand category) for which results are shown. **Referenced table:** HistoricalDemandHeader	Reference	
Part	The part that this adjustment applies to. **Referenced table:** Part	Reference	Yes

EventDisaggregationRate table

This table is used for Event Management. It provides visibility into the calculations performed when applying event-based quantity adjustments to the forecast.

To apply quantity adjustments to forecast items, the affected forecast details reported in the **ForecastDetails** table are first bucketed using the calendar specified for the event phase. For example, if the event phase increases the forecast quantity by a certain amount for each month, the forecast details for the affected periods are bucketed using the month calendar. The result is reported in the **EventDisaggregationRate** table. For more information, see "Event Management and the RapidResponse data model — calculated tables" on page 2120.

Records might also be added to the **EventDisaggregationRate** table for forecast items that did not have forecast quantities previous to the event phase being applied.

Forecast items that are only subject to unit price adjustments are not included in the **EventDisaggregationRate** table.

This table was added in Version 2016.2, Service Update 3.

Table 8.44: EventDisaggregationRate (Mfg) fields

Field name	Description	Data type	Owner
Date	The date on the **EventPhase** calendar that begins the forecast bucket that this record applies to.	Date	
EventPhase	The event phase that affects this forecast item **Referenced table:** EventPhase	Reference	Yes
EventPhase Header	The event phase header that connects the **EventPhase** to an **HistoricalDemandHeader** that the event phase applies to. **Referenced table:** EventPhaseHeader	Reference	
Quantity	Bucketed forecast quantity associated with the **Date** and **HistoricalDemandHeader** referenced by **EventPhaseHeader**. The forecast quantity comes from the data in the **ForecastDetail** table. This is the forecast quantity before event-based forecast adjustments.	Quantity	
Weight	Ratio of the **Quantity** on the **Date** among the total of **Quantities** (on this **Date**) for valid **Headers** associated with the **EventPhase**	Quantity	

EventForecastDetailAdjustment table

This table is used for Event Management. It reports the adjustment details, including any quantity or unit price adjustments, for each forecast item that is affected by event-based adjustments. Detail records on this table for forecast items that contribute to the consensus forecast are used to populate

the **EventConsensusForecastDetail** table. For more information, see "Event Management and the RapidResponse data model — calculated tables" on page 2120.

A single record is included on this table for each affected forecast item in each affected **EventPhase.Calendar** bucket, and the results of all event-based forecast adjustments applied to it are calculated. For example, if a forecast item in a given date bucket is affected by an event phase that increases the unit price by $10, an event phase that decreases the quantity by 100, and an event phase that increases the quantity by 50, the value in the **AverageUnitPriceAdjustment** field would be $10, and the value in the **QuantityAdjustment** field would be -50.

This table was added in Version 2016.2, Service Update 3.

Table 8.45: EventForecastDetailAdjustment (Mfg) fields

Field name	Description	Data type	Owner
AverageUnit Price	The weighted average of **EffectiveUnitPrice** for F**orecastDetail** records with the given **Header** and the bucket specified by **Date**. If there are no **ForecastDetail** records in the appropriate period, the value is calculated in the same way as **EffectiveUnitPrice** on the **ForecastDetail** table on the **Date.**	Money	
AverageUnit Price Adjustment	Calculated adjustment to the average unit price.	Money	
Date	The date on the calendar specified in **Header.PartCustomer.EffectiveDisaggregationCalendar** field when the **EffectiveQuantityAdjustment** is applied.	Date	
Effective Quantity Adjustment	The value in the **QuantityAdjustment** field, unless the adjustment would result in a negative forecast amount and **Header.Category.Type.EventAllowNegative** is set to "false." This value is used in subsequent calculations, such as calculating the consensus forecast.	Quantity	
Header	The **HistoricalDemandHeader** record that identifies the part, customer, and historical demand category for which results are calculated. **Referenced table:** HistoricalDemandHeader	Reference	Yes
Original Quantity	Total of **ForecastDetail.Quantity** for records with the given **Header** and the bucket specified by **Date.** If there are no **ForecastDetail** records in the appropriate period, the value is 0.	Quantity	
Quantity Adjustment	The quantity to adjust the forecast by.	Quantity	

EventStatisticalDisaggregationRate table

This table is used for disaggregation of statistical forecast items affected by Event Management. Its results, along with data from other tables, are used to calculate the list of event-based statistical forecast adjustments that appears in the **EventStatisticalForecastDetailAdjustment** table. For more information, see "Reporting statistical forecast adjustments" on page 2123.

To calculate quantity adjustments for statistical forecast items, the affected forecast details reported in the **EventStatisticalForecastDetail** table are first bucketed using the calendar specified for the event phase. For example, if the event phase increases the forecast quantity by a certain amount for each month, the forecast details for the affected periods are re-bucketed using the month calendar.

Records might also be added to the **EventStatisticalDisaggregationRate** table for forecast items that did not have forecast quantities previous to the event phase being applied.

Statistical forecast items that are only subject to unit price adjustments are not included in the **EventStatisticalDisaggregationRate** table.

This table is analogous to the **EventDisaggregationRate** table, which is based on the records in the **ForecastDetails** table and reports results for other forecast categories in addition to the statistical forecast.

This table was added in Version 2016.2, Service Update 3.

Table 8.46: EventStatisticalDisaggregationRate (Mfg) Fields

Field name	Description	Data type	Owner
Date	The date on the **EventPhase** calendar that begins the forecast bucket that this record applies to.	Date	
EventPhase	The event phase that affects this forecast item. **Referenced table:** EventPhase	Reference	Yes
EventPhase Header	The event phase header that connects the **EventPhase** to an **HistoricalDemandHeader** that the event phase applies to. **Referenced table:** EventPhaseHeader	Reference	
Quantity	Bucketed forecast quantity associated with the **Date** and **HistoricalDemandHeader** referenced by **EventPhaseHeader**. The forecast quantity comes from the data in the **EventStatisticalForecastDetail** table. This is the forecast quantity before event-based forecast adjustments. If there are no **EventStatistical ForecastDetail** records in the appropriate period, the value is 0.	Quantity	
Weight	Ratio of the **Quantity** on the **Date** among the total of **Quantities** (on this **Date**) for valid **Headers** associated with the **EventPhase**	Quantity	

EventStatisticalForecastDetail table

This table is used for Event Management. It contains information about statistical forecast items that are affected by event phases. Its results, along with data from other tables, are used to calculate the event-based statistical forecast adjustments listed in the **EventStatisticalForecastDetailAdjustment**table. For more information, see "Reporting statistical forecast adjustments" on page 2123.

This table was added in Version 2016.2, Service Update 3.

Table 8.47: EventStatisticalForecastDetail (Mfg) fields

Field name	Description	Data type	Owner
Date	Date representing the forecast bucket. This date belongs to Header.PartCustomer.EffectiveDisaggregationCalendar.	Date	
Header	The HistoricalDemandHeader associated with the statistical forecast data. **Referenced table:** HistoricalDemandHeader	Reference	
ItemParameters	Reference to the ForecastItemParameters record associated with this forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes
Quantity	Forecast quantity associated with the **Date** and **Header.**	Quantity	

EventStatisticalForecastDetailAdjustment table

This table is used for Event Management. It reports the adjustment details, including any quantity adjustments, for each statistical forecast item that is affected by event-based adjustments. It is analogous to the **EventForecastDetailAdjustment** table, which also includes information about event-based adjustments to other forecast streams. For more information, see "Reporting statistical forecast adjustments" on page 2123.

The information in this table is not used to calculate the consensus forecast. Adjustments to statistical forecast items that affect the consensus forecast are reported along with other event-based adjustments in the **EventForecastDetailAdjustment** table and **EventConsensusForecastDetail** table, based on the data in the statistical forecast stream in the **ForecastDetail** table. In some cases, there might be differences between the event-based adjustment details reported here.

This table was added in Version 2016.2, Service Update 3.

Table 8.48: EventStatisticalForecastDetailAdjustment (Mfg) fields

Field name	Description	Data type	Owner
Adjustment	The event-based adjustment calculated for this forecast item, after all of the event phases that affect it have been applied.	Quantity	
Date	Date on the **Header.PartCustomer.EffectiveDisaggregationCalendar** calendar when the adjustment is applied.	Date	
Effective Adjustment	The value of the Adjustment field, possibly reduced to prevent the forecast from being adjusted to a negative quantity. For more information, see "Allowing events to reduce forecast below zero" on page 2133.	Quantity	
Header	The historical demand header of the forecast item affected by the adjustment. **Referenced table:** HistoricalDemandHeader	Reference	Yes

FeatureBOMForIndependentDemand table

The **FeatureBOMForIndependentDemand** table can be used to report all immediate components of an assembly, and their quantities, that are required to satisfy a given assembly demand. The calculations in this table include consideration of any applicable features when reporting component usage. Thus, for configurable end-items set up to use feature BOM logic in RapidResponse, it can be useful in identifying which components are actually needed to satisfy a given assembly demand based on the features selected for that demand.

Table 8.49: FeatureBOMForIndependentDemandFeature (Mfg) fields

Field name	Description	Data type	Owner
BOM	A reference to the BOM record that associates the component reported on this record with the assembly the demand is for. For example, details such as the quantity per, effectivity, and feature rule (if applicable) can be returned from this reference. **Referenced table:** BillOfMaterial	Reference	
Component	A reference to the component part being reported on this record. **Referenced table:** Part	Reference	
Independent Demand	A reference to the demand being reported on this record. For example, details such as order line, quantity, and feature list (if applicable) can be returned from this reference. **Reference table:** IndependentDemand	Reference	Yes

Table 8.49: FeatureBOMForIndependentDemandFeature (Mfg) fields

Field name	Description	Data type	Owner
Level	Indicates the level in the assembly's BOM structure that the component identified by this record belongs to. A value of "1" is reported for each record pertaining to an assembly and one of its immediate components. A value of "0" is reported for records that pertain just to the assembly.	Integer	
Quantity	The total amount of the component part required to satisfy this particular assembly demand (based on factors such as order quantity and quantity per on the BOM record).	Quantity	

FlatAssembly table

The **FlatAssembly** table reports all assemblies, at any level, that can potentially use a given component part (and hence might cause dependent demand to be placed on the component). This includes both direct parent assemblies that require the component, as well as other assemblies found multiple levels above the component through other part relationships. Assembly parts reported for a given component are those ultimately related through standard BillOfMaterial records, co/by-product relationships, global and BOM-level substitute relationships, Allocation records, and transfer PartSource records.

This table was added in Version 2016.2, Service Update 3. To report just the immediate parent assemblies for a component, use the **DirectAssembly** table.

Table 8.50: FlatAssembly (Mfg) fields

Field name	Description	Data type	Owner
Assembly	The name of an assembly that can potentially use the component. The assembly can be either immediately above the component or multiple levels above the component in the structure. **Referenced table:** Part	Reference	
Part	The name of the component part that can potentially be used in the building of the assembly. **Referenced table:** Part	Reference	Yes

FlatBillDown table

This table reports all components used to make a part (including multiple sites). Date effectivity on **BillOfMaterial** records (and **PartSource** records) are taken into account when determining which components will be processed at each level of the bill or material product structure. If a part has multiple effective part sources, only the results from exploding through the primary part source are reported.

FlatBillDown records generated for all valid **BillOfMaterial** and **PartSource** records regardless of their effectivity range as defined by the **PlanningCalendars.RunDate** field. If a **BillOfMaterial** or **PartSource**

record was effective before RunDate, it is used to generate the corresponding **FlatBillDown** record. To view **FlatBillDown** records generated from valid **BillOfMaterial** and **PartSource** records effective at RunDate, use the following filter: LastDate >= RunDate.

Note that **FlatBillDown** records are generated for all components under “make ” and "transfer" part sources (that is, where **PartSource.Type.PlannedOrderSupplyType.Source** = “Make” or "Transfer"), and as long as the **PartSource.BomAlternate** and **BOM.Alternate** reference fields point to the same value (typically, this might be the default or “blank” value). For "buy" part sources (that is, where **PartSource.Type.PlannedOrderSupplyType.Source** = “Buy”), just a record for the part itself is generated as there are no components to explode to.

Notes:

- If **FlatBillDown** records are required for all effective part sources, as opposed to just the primary part source, a workbook parameter can be used as described in "Include non-primary part sources in FlatBillDown and FlatBillUp calculations" on page 2217.
- If components under global substitutes should always be reported in this table, a workbook parameter can be used as described in "Include global substitutes and their components in FlatBillDown calculations" on page 2219.
- If assembly and components related through BOM-level substitute part relationships should always be reported in this table, a workbook parameter can be used as described in "Include substitute BOM components in FlatBillDown and FlatBillUp calculations" on page 2218.
- If records should be reported for one model of a part, a workbook parameter can be used as described in "Limiting FlatBillDown and FlatBillUp calculations by model" on page 2224.
- To specify an effective date against which PartSource and BillOfMaterial records must be evaluated for inclusion in FlatBillDown calculations, a workbook parameter can be added as described in "Apply a parameter date to FlatBillDown calculations" on page 2222.
- To report components under parts from inactive part sources that have **OrderPolicy.OrderGenerationRule** set to "NoOrders" and hence cannot generate new planned orders, a workbook parameter can be added. This enables FlatBillDown calculations to explode through such "NoOrders" parts as described in "Explode BOMs for "NoOrders" parts in FlatBillDown calculations" on page 2221.

Table 8.51: FlatBillDown (Mfg) fields

Field name	Description	Data type	Owner
BOM	The current BillOfMaterial record that generated this FlatBillDown record. The BOM.Component field is the reference to the current FlatComponent. The reference will be null if a transfer PartSource record is the origin of this FlatBillDown record. **Referenced table:** BillOfMaterial	Reference	
EffLeadTime	The effective lead time for the FlatComponent (taking into account safety, fixed and variable lead time as well as lead time adjustments) based on the PartSource used for the FlatBillDown record. For part sources where Type.TransitRule is set to “Cumulative”, this also includes pre-ship lead time, transit time, and dock-to-stock lead time.	Quantity	

Table 8.51: FlatBillDown (Mfg) fields

Field name	Description	Data type	Owner
FirstDate	The calculated beginning of the date range where the component can be used in building the top-level assembly is based on the component's parents lead times. This is the earliest date on which a planned order for the selected assembly (Part) can be due and still have the component (FlatComponent) effective (refers the Part.Source.EffectiveInDate). The above default logic needs to incorporate with the control table switch PartType.UseLTForFlatBill = 'Y'.	Date	
FirstUnit	The first unit in which this component can be used. Takes into account BOM (PartSource) unit effectivity and the previous (peg) FlatBillDown record effectivity. If the Part.MUEPoolNettingType.UnitRule field is **Ignore**, then all the BOM records (and PartSource records) under that part are effective, regardless of unit range that is defined on those records.	Date	
FlatComponent	The name of the component. It can be any level deep in the bill structure or in another site through a transfer PartSource. **Referenced table:** Part	Reference	
FlatIndex	A number used when displaying the indented flat bill (used for sorting the records in the worksheet).	Integer	
FlatLeadTimeDue	Number of TimeUnits intervals (usually Workdays) in advance of the Part's CustomerOrder DueDate that supply on the FlatComponent needs to be completed. **Note:** Any lead time associated with phantom assemblies will not be included in the calculation of this value.	Quantity	
FlatLeadTimeStart	Number of TimeUnits intervals (usually Workdays) in advance of the Part's CustomerOrder DueDate that supply on the FlatComponent needs to be started. **Note:** Any lead time associated with phantom assemblies will not be included in the calculation of this value.	Quantity	
FlatQuantityPer	The number of FlatComponents needed to build the Part. Takes into account the QuantityPer of the BOM record and the FlatQuantityPer of the previous (peg) FlatBillDown record.	Quantity	
InterSiteLevel	Is incremented (starts from 0) every time there is a cross site (usually due to a Transfer PartSource being processed).	Quantity	

Table 8.51: FlatBillDown (Mfg) fields

Field name	Description	Data type	Owner
IsModelEffective	If an input model is selected, the FlatBillDown records that are displayed are all ModelEffective, therefore the "IsModelEffective" flag will be set to Y, otherwise it is set to N. Valid values are: • **Y** • **N**	Boolean	
IsTransfer	Valid values are: • **Y**—if a transfer PartSource is the origin of the current record. In this situation, BOM reference is NULL. Use **PegPartSource** to get information about the FlatComponent's parent. **PegPartSource.TransferPart** is the FlatComponent. • **N**—if a transfer PartSource is not the origin of the current record.	Boolean	
LastDate	The calculated end of the date range where the component can be used in building the top-level assembly is based on the component's parents lead times. This is the latest date on which a planned order for the top-level assembly (Part) can be due and still have the component (FlatComponent) effective.	Date	
LastUnit	The last unit in which this component can be used.	Integer	
Level	The level on the bill structure the component part is at.	Integer	

Table 8.51: FlatBillDown (Mfg) fields

Field name	Description	Data type	Owner
Model	The user specified value in the field (entered manually as a value and assigned to the $RR_Analytics_FlatBill_Model variable) is used to calculate the FlatBillDown records for the BOM and return only the relevant records. The operation of filtering by model takes place for each BOM record in this order. 1. If the BOM record's model is "NeverEffective", it doesn't get calculated. 2. If the BOM record's model is the default model, it gets calculated irrespective of what RR_Analytics_FlatBill_Model was set to or what the BOM record's model effectivity is. The default model for a BOM record is “None”, which means it can take any model for explosion. 3. If RR_Analytics_FlatBill_Model is set to an asterisk (*), the BOM record gets calculated irrespective of what the BOM's model is or what its model effectivity is. 4. If the BOM record's model is not effective, it means that the MUE Rule for the BOM is not "Model" or "ModelUnit". In this case, the BOM record's model is treated as a default model and the record gets calculated irrespective of what RR_Analytics_FlatBill_Model was set to. 5. If RR_Analytics_FlatBill_Model is the same as the the BOM record's model, the BOM record gets calculated. 6. If RR_Analytics_FlatBill_Model is not the same as the BOM record's model, the BOM record doesn't get calculated. **Referenced table:** Model **Note:** More information on using the $RR_Analytics_FlatBill_Model variable to limit FlatBillDown records can be found on the "Limiting FlatBillDown and FlatBillUp calculations by model" on page 2224 page.	Reference	
Part	The name of the part whose components are being reported. **Referenced table:** Part	Reference	Yes
PartSource	The part source for the FlatComponent itself. That is, **FlatComponent = PartSource.Part**. **Referenced table:** PartSource	Reference	

Table 8.51: FlatBillDown (Mfg) fields

Field name	Description	Data type	Owner
PegPartSource	The part source for the immediate parent of the FlatComponent. This field points to the FlatComponent through either a transfer relationship or a bill of material. That is: • If **IsTransfer** = 'Y', then **FlatComponent** = **PegPartSource.TransferPart**. • If **IsTransfer** = 'N', then **FlatComponent** = **BOM.Component**. **Referenced table:** PartSource	Reference	

FlatBillUp table

This table reports all the assemblies at different levels where a part is used (including multiple sites). Date effectivity on **BillOfMaterial** and **PartSource** records are taken into account when determining which assemblies will be processed at each level of the bill product structure. If an assembly has multiple part sources at the same priority and over the same time frame, only the results from the primary part source are reported.

FlatBillUp records are generated for all valid **BillOfMaterial** and **PartSource** records, regardless of their effectivity range as defined by the **PlanningCalendars.RunDate** field. If a **BillOfMaterial** or **PartSource** record was effective before RunDate, it is used to generate the corresponding **FlatBillDown** record. To view **FlatBillUp** records generated from valid **BillOfMaterial** and **PartSource** records effective at RunDate, use the following filter: LastDate >= RunDate.

Notes:

- If records should be reported for all effective part sources, as opposed to just the primary part source, then a workbook parameter can be used as described in "Include non-primary part sources in FlatBillDown and FlatBillUp calculations" on page 2217.
- If assembly and components related through substitute part relationships should always be reported in this table, a workbook parameter can be used as described in "Include substitute BOM components in FlatBillDown and FlatBillUp calculations" on page 2218.
- If records should be reported for parts of only one model, then a workbook parameter can be used as described in "Limiting FlatBillDown and FlatBillUp calculations by model" on page 2224.

Table 8.52: FlatBillUp (Mfg) fields

Field name	Description	Data type	Owner
BOM	The current BillOfMaterial record that generated this FlatBillUp record. The **BOM.Assembly** field is the reference to the FlatAssembly. **Referenced table:** BillOfMaterial	Reference	
ChildPartSource	The part source for the immediate child of the FlatAssembly. This field points to the FlatAssembly through either a transfer relationship or a bill of material. That is: • If **IsTransfer** = 'N', **ChildPartSource.Part = BOM.Component**. • If IsTransfer = 'Y', **ChildPartSource.Part = PartSource.TransferPart**. **Referenced table:** PartSource	Reference	
EffLeadTime	The effective lead time for the FlatAssembly (taking into account safety, fixed, and variable lead time as well as lead time adjustments) based on the PartSource used for the FlatBillUp record. For part sources where the **Type.TransitRule** = 'Cumulative', this also includes pre-ship lead time, transit time, and dock-to-stock lead time.	Quantity	
FlatAssembly	The name of the assembly. It can be any level above the Part in the bill structure or in another site through a transfer part source. **Referenced table:** Part	Reference	
FirstDate	The calculated beginning of the date range where the Part is used in building this FlatAssembly. Takes into account BOM (PartSource) date effectivity, the previous (peg) FlatBillUp record effectivity, immediate component/TransferPart LeadTime and BOM offset.	Date	
FirstUnit	The first FlatAssembly unit (inclusive) in which this Part is used. Accounts for unit effectivity on BOM and previous (peg) FlatBillUp record.	Integer	
FlatIndex	A number that will be used when displaying the indented flat bill (used for sorting the records in the worksheet).	Quantity	
FlatLeadTimeDue	Number of days after Part supply start that the Part appears in supplies of the FlatAssembly (based on FlatAssembly supply due date).	Quantity	
FlatLeadTimeStart	Number of days after Part supply start that the Part appears in supplies of the FlatAssembly (based on FlatAssembly supply start date).	Quantity	

Table 8.52: FlatBillUp (Mfg) fields

Field name	Description	Data type	Owner
FlatQuantityPer	Number of parts required to make one FlatAssembly. Takes into account the QuantityPer of the BOM record and the FlatQuantityPer of the previous (peg) FlatBillUp record.	Quantity	
InterSiteLevel	Is incremented (starts from 0) every time there is a cross site (usually due to a Transfer PartSource being processed).	Quantity	
IsModelEffective	If an input model is selected, the FlatBillUp records that are displayed are all ModelEffective, therefore the "IsModelEffective" flag will be set to Y, otherwise it is set to N. Valid values are: • **Y** • **N**	Boolean	
IsTransfer	Valid values are: • **Y**—if a Transfer PartSource is the origin of the current record. In this situation, BOM reference is NULL. Use **PartSource** and **ChildPartSource** to get information about the FlatAssembly's child. **PartSource.Part** is the FlatAssembly, and **PartSource.TransferPart** is the FlatAssembly's child; **ChildPartSource.Part** is also the FlatAssembly's child. • **N**—if a transfer PartSource is not the origin of the current record.	Boolean	
LastDate	The calculated end of the date range where the Part is used in building this FlatAssembly.	Date	
LastUnit	The last FlatAssembly unit (inclusive) in which this Part is used.	Integer	
Level	The level in the bill structure the FlatAssembly is at (compared with Part).	Integer	

Table 8.52: FlatBillUp (Mfg) fields

Field name	Description	Data type	Owner
Model	The user specified value in the field (entered manually as a value and assigned to the $RR_Analytics_FlatBill_Model variable) is used to calculate the FlatBillUp records for the BOM and return only the relevant records. The operation of filtering by model takes place for each BOM record in this order. 1. If the BOM record's model is "NeverEffective", it doesn't get calculated. 2. If the BOM record's model is the default model, it gets calculated irrespective of what RR_Analytics_FlatBill_Model was set to or what the BOM record's model effectivity is. The default model for a BOM record is “None”, which means it can take any model for explosion. 3. If RR_Analytics_FlatBill_Model is set to an asterisk (*), the BOM record gets calculated irrespective of what the BOM's model is or what its model effectivity is. 4. If the BOM record's model is not effective, it means that the MUE Rule for the BOM is not "Model" or "ModelUnit". In this case, the BOM record's model is treated as a default model and the record gets calculated irrespective of what RR_Analytics_FlatBill_Model was set to. 5. If RR_Analytics_FlatBill_Model is the same as the the BOM record's model, the BOM record gets calculated. 6. If RR_Analytics_FlatBill_Model is not the same as the BOM record's model, the BOM record doesn't get calculated. **Referenced table:** Model **Note:** More information on using the $RR_Analytics_FlatBill_Model variable to limit FlatBillUp records can be found on the "Limiting FlatBillDown and FlatBillUp calculations by model" on page 2224 page.	Reference	
Part	The name of the part for which assemblies are calculated. **Referenced table:** Part	Reference	Yes
PartSource	The part source for the FlatAssembly itself. That is, **FlatAssembly = PartSource.Part**. **Referenced table:** Part	Reference	

FlatComponent table

The **FlatComponent** table reports all components parts, at any level, that can potentially be used in a given assembly. This includes both components directly used in building the assembly, as well as components found multiple levels below the assembly under other parts. Component parts reported for

a given assembly are those ultimately related through standard BillOfMaterial records, co/by-product relationships, global and BOM-level substitute relationships, Allocation records, and transfer PartSource records.

This table was added in Version 2014.1

Table 8.53: FlatComponent (Mfg) fields

Field name	Description	Data type	Owner
Component	The name of a component that can potentially be used at some level, either directly or indirectly, in building the assembly part. **Note:** Each assembly will have a record identifying itself as its own component. **Referenced table:** Part	Reference	
Part	The name of the assembly part that can use the component either directly or indirectly. **Referenced table:** Part	Reference	Yes

FlatDemand table

The **FlatDemand** table reports the dates on which component supplies would be needed to satisfy each independent demand. Each record in this table identifies a part and the need date for it's supply to achieve on-time delivery of a given end-item independent demand.

Component need dates are determined starting from the driver demand due date and then offsetting by lead time through all levels of the bill of material and across transfer sites as needed. Note that neither demand quantities nor existing input supplies are considered in this calculation. Thus need dates are shown for all components regardless of whether any upper-level demands will ultimately be satisfied by existing onhand or scheduled supply.

This table was added in Service Update 1902.

Table 8.54: FlatDemand (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	The bill of material entry that drove the dependent requirement shown on this record. The **BillOfMaterial.Component** reference corresponds to the part whose need date is reported on this record while the **BillOfMaterial.Assembly** reference corresponds to the assembly that drove demand to that part. Note that this field is Null if the record represents the independent demand itself. This field is also Null if the record represents a dependent requirement passed from a part at one site to another site through a transfer part source. **Referenced table:** BillOfMaterial	Reference	
IndependentDemand	The driver independent demand whose component need dates are being reported. **Referenced table:** IndependentDemand	Reference	Yes
Index	A generated index value that can be used for sorting records into an indented tree structure. This value is reset at 0 for each independent demand reported in this table, and increments by one for each component needed under a given independent demand.	Integer	
IsLeaf	Specifies whether the part is at the lowest level on its path within the bil of material structure, and hence whether it will pass dependent requirements to lower-level components. Valid values are: • **N**—The part is not at the lowest level in this path of the bill of material structure. Thus it can drive dependent requirements to lower-level components. • **Y**—The patt is found at the lowest level in this path of the bill of material structure. Thus it does not have any lower level components to which dependent requirements are driven. For example, this value would be seen for purchased parts.	Boolean	
Level	The level in the bill of material where the part reported on this record is found. This value can be used in building an indented tree structure. For example, end-items have a level of 0, their immediate components and sub-assemblies would have a level of 1, and immediate components of those sub-assemblies would have a level of 2.	Integer	

Table 8.54: FlatDemand (Mfg) fields

Field name	Description	Data type	Owner
NeedDate	The date by which supply of the part is needed in order to achieve on-time delivery of the driver independent demand. This date is determined starting from the due date of the driver independent demand and then offsetting by lead times at each level of the structure (excluding variable lead time and any safety lead time) . Note that, if necessary, need dates are always rounded back to a marker on the applicable **PlanningCalendars.TimeUnits** calendar before applying lead time at subsequent levels in the structure.	Date	
ParentPartSource	The part source for the parent part that drove the dependent requirement to the part reported on this record. **Referenced table:** ParentPartSource	Reference	
Part	A part for which supply would be needed to satisfy the driver independent demand. **Referenced table:** Part	Reference	
PartSource	The part source that would provide supply of the part reported on this record. For parts that have multiple part sources, those with an **OrderPolicy.OrderGenerationRule** setting other than "NoOrders" and which are effective on the **NeedDate** are considered first. If multiple part sources meet this criteria, then the highest priority of those part sources is chosen (by default, the smallest number in **PartSource.Priority** equates to the highest priority). If there are multiple effective part sources at the highest priority, then the following ordered criteria is used to determine the part source (based on ascending sort order). • Source.Id • Source.DestinationSite • Source.Supplier.Id • Source.Supplier.Site • Basekey **Referenced table:** PartSource	Reference	

Notes:

- Phantom parts and BOM records are reported in this table, however any lead times defined on them are ignored.
- This table does not include consideration for days of supply logic, attribute-based planning, model-unit effectivity, or forecast spreading.

FlatConstraint table

The **FlatConstraint** table reports all constraints that can potentially constrain a given part's sources of supply. This includes constraints used directly by the part itself as well as constraints used by any of its components (at any level).

Note: A part's associated constraints are not reported in this table if their referenced **ConstraintType** has a **ProcessingRule** of "Unconstrained" or an **EffectivityRule** of "Never', or if the associated **PartSource** record is never effective, or if the associated **SourceConstraint** record has **ConstraintFactor** and **FixedConstraintFactor** that are both less than or equal to zero (0),

Table 8.55: FlatConstraint (Mfg) fields

Field name	Description	Data type	Owner
Constraint	The name of a constraint that can potentially constrain either the part itself or one of its flat components. Constraints associated with a given part are not reported if they are used by a **PartSource** that is never effective, have a **Type.ProcessingRule** of "Unconstrained", have a **Type.EffectivityRule** of "Never", or the **SourceConstraint** record has both **FixedConstraintFactor** and **ConstraintFactor** that are <= 0. **Referenced table:** Part	Reference	
Part	The name of the part that can potentially be constrained by the constraint (or has components that might be constrained by the constraint). **Referenced table:** Part	Reference	Yes

FLPConstraint table

This table reports full level pegging and available supply information for Constraint usage.

Table 8.56: FLPConstraint (Mfg) fields

Field name	Description	Data type	Owner
AllocUsed	The amount of ConstraintUsed.Load allocated to this NeedQuantity (driver).	Quantity	
Constraint	The Constraint with this load. **Referenced table:** Constraint	Reference	Yes
ConstraintUsed	The Constraint load being allocated. **Referenced table:** ConstraintUsed	Reference	

Table 8.56: FLPConstraint (Mfg) fields

Field name	Description	Data type	Owner
NeedQuantity	The quantity of the supply that is pegged back to the driving order.	Quantity	
WhereConsumed	The WhereConsumed record associated with this use of Constraint. Note that AvailableDate, DriverType, IndependentDemand, ScheduledReceipt, DriverDueDate, DriverQuantity, DriverAvailableDate, Pool, Model, SupplyStartUnit and other data is available through this reference. **Referenced table:** WhereConsumed	Reference	

Forecast table

The records in this table show a summary of all demand records where Type is a SalesForecast. The original forecast **Quantity** is reported along with the total **ConsumedQuantity** that was consumed by sales actuals. Any remaining unconsumed forecast that is outside of the part's demand time fence is the effective demand quantity that should be planned for.

Forecast can be seen from independent demands, dependent demand from planned orders (PlannedAllocation and PlannedTransferAllocation), and consensus forecast demand. Forecast can also be generated from scheduled receipts if **SupplyType.AllocationForecastRule** is set to 'Use'.

Note: If a part does not have at least one PartSource defined, it will not have records generated in this table (even though forecast demands for such parts might be seen in other related tables like **IndependentDemand** and **ConsensusForecast**).

Table 8.57: Forecast (Mfg) fields

Field name	Description	Data type	Owner
Allocation	The Allocation record that created this forecast. Valid only if **ForecastSource** = 'Allocation'. **Referenced table**: Allocation	Reference	
ConsensusForecast	The ConsensusForecast record that created this forecast. Valid only if **ForecastSource** = 'ConsensusForecast'. **Referenced table:** ConsensusForecast	Reference	
ConsumedQuantity	The amount of the forecast quantity that is consumed by actual demand (actuals are represented by **IndependentDemand** records with an **Order.Type.ProcessingRule** setting of "SalesActual"). **Note**: This field was added in Version 2016.2, Service Update 30.0.	Quantity	

Table 8.57: Forecast (Mfg) fields

Field name	Description	Data type	Owner
Date	The date the demand is forecast to be required.	Date	
EarliestExpiryDate	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
EffectiveDemandQuantity	The amount of the original forecast quantity that should be planned for. Netting logic tries to assign or generate supply to cover the quantity reported here. This value represents unconsumed forecast that is outside the demand time fence. Unconsumed forecast due before the part's **DTFDate** is typically ignored and reports an effective quantity of 0 (unless the part's **Type.DTFRule** is "ForecastInterval" and the forecast is due within the same forecast interval as the **DTFDate**). **Note**: This field was added in Version 2016.2, Service Update 30.0.	Quantity	
EffectiveUnitPrice	The unit price effective for the forecast's part and customer. This value is useful in determining revenue and is calculated only if **ForecastSource** is set to 'Independent' or 'ConsensusForecast'. For these forecast sources, this field is calculated as follows: • RapidResponse checks the CustomerPrice table for records with matching Part and Customer values that have effective dates less than or equal to the date of this record. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. Note that if multiple records exist for a given part and customer combination with the same effective date, the EffectiveUnitPrice calculation uses the record with the larger value in the **CustomerPrice.Id** field. • However, if no matching Part and Customer values are found, RapidResponse next checks the CustomerPrice table for records with a matching Part and blank Customer value (that is, ' ') that have effective dates less than or equal to the record date. If any matching records are found, then the UnitPrice from the record with the latest effective date is reported here. • Finally, if no unit price can be found under any of the conditions mentioned above, the value in Part.AverageSellingPrice is reported here. For all other forecast sources, this field is always set to zero (0). Currency conversions on this field use the **Date** field's conversion rate.	Money	

Table 8.57: Forecast (Mfg) fields

Field name	Description	Data type	Owner
ForecastSource	Indicates the source of the forecast. Valid values are: • **Allocation**—forecast generated from an Allocation record on a scheduled receipt where Order.Type.AllocationDemand Type.ProcessingRule = 'SalesForecast'. The reference to Allocation is valid. • **BOM**—forecast was initiated at another part and was passed to this part through a Bill of Material record. The reference to PlannedAllocation is valid. • **BOMReceipt**—forecast was generated from a NewAllocation record on a scheduled receipt where **Order.Type.Allocation DemandType.ProcessingRule** = 'SalesForecast'. The reference to NewAllocation is valid. • **ConsensusForecast**—forecast was generated from a **ConsensusForecast** record. ConsensusForecast records are generated as weighted combinations of different streams or categories of forecast in the **ForecastDetail** table used to support the sales and operations planning process in RapidResponse. The reference to **ConsensusForecast** is valid. • **Independent**—forecast was initiated at this part. The reference to IndependentDemand is valid. • **Transfer**—forecast was initiated at another part and was passed to this part through a Transfer part source. The reference to PlannedTransferAllocation is valid. • **TransferReceipt**—forecast was generated from a **NewTransferAllocation** record coming from a scheduled receipt with **Order.AllocationDemandType .ProcessingRule** = 'SalesForecast'. The reference to NewTransferAllocation is valid.	String (Enum)	
Independent Demand	The IndependentDemand (forecast) creating this forecast. **Referenced table:** IndependentDemand	Reference	
Model	The model that applies to this forecast. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is Ignore **Referenced table:** Model	Reference	

Table 8.57: Forecast (Mfg) fields

Field name	Description	Data type	Owner
NewAllocation	The NewAllocation record that created this forecast. Valid only if **ForecastSource** = 'BOMReceipt'. **Referenced table**: NewAllocation	Reference	
NewTransfer Allocation	The NewTransferAllocation record that created this forecast. Valid only if **ForecastSource** = 'TransferReceipt'. **Referenced table**: NewTransferAllocation	Reference	
OrderPriority	The priority associated with this demand. Used when allocating limited supply to demands. **Referenced table:** OrderPriority	Reference	
Part	The part for which this forecast exists. **Referenced table:** Part	Reference	Yes
PartCustomer	A reference to the part and customer combination this forecast applies to. Valid only if ForecastSource is either "Independent" or "ConsensusForecast". **Referenced table:** PartCustomer	Reference	
PlannedAllocation	The PlannedAllocation record that created this forecast. **Referenced table:** PlannedAllocation	Reference	
PlannedTransfer Allocation	The PlannedTransferAllocation record that created this forecast. **Referenced table:** PlannedTransferAllocation	Reference	
Pool	The pool in which the forecast consumption was done. It is set to Unpooled if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.PoolRule is Ignore. **Referenced table**: Pool	Reference	
Quantity	The total amount forecast.	Quantity	

Table 8.57: Forecast (Mfg) fields

Field name	Description	Data type	Owner
StartUnit	The start unit that applies to this forecast. It is copied from the source independent demand record if the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is Net If these conditions do not exist, the field is set according to the Part.MUEPoolNettingType.UnitRule and Part.NextUnit.	Integer	
Unsatisfied Quantity	The portion of this demand that was left unsatisfied.	Quantity	

ForecastConsumption table

The **ForecastConsumption** table identifies the actual demands that consume forecast records. Forecast in RapidResponse typically comes from either consensus forecast records generated during the sales and operations planning process, or demand records (independent or dependent) where the referenced **DemandType.ProcessingRule** is set to "SalesForecast". Forecasts are then typically consumed (reduced) by actual demands that have a **DemandType.ProcessingRule** of "SalesActual". If a forecast is consumed by more than one actual demand, then a record is reported for each unique demand that consumes the forecast.

This table thereby provides insight into the exact order numbers, customers, and dates associated with the demands that consume specific forecasts. For example, this table might be used to see how actual sales consumes the current demand plan or just the forecast pertaining to a specific customer or item.

If any attributes were used to direct forecast consumption, they can be reported on the relevant records in this table using the `Attribute<*>` syntax (for more information, see the *RapidResponse Resource Authoring Guide*).

Table 8.58: ForecastConsumption (Mfg) fields

Field name	Description	Data type	Owner
ActualDueDate	The due date of the actual demand that consumes the forecast.	Date	
ActualQuantity	The total quantity of the actual demand that consumes the forecast.	Quantity	
Allocation	If **Source** is "Allocation", this returns a reference to the imported dependent demand that consumes the forecast. Otherwise, null. **Referenced table:** Allocation	Reference	

Table 8.58: ForecastConsumption (Mfg) fields

Field name	Description	Data type	Owner
Consumed Quantity	The amount of the forecast that is being consumed by the actual demand reported on this record.	Quantity	
Forecast	A reference to details of the forecast being consumed by the actual demand reported on this record. This link also provides a further reference to the underlying independent demand, dependent demand, or consensus forecast record. **Referenced table:** Forecast	Reference	
Independent Demand	If Source is "IndependentDemand", this returns a reference to an independent demand, with a Type.ProcessingRule value of "SalesActual", that consumed the forecast. Otherwise, null. **Referenced table:** IndependentDemand	Reference	
NewAllocation	If Source is "NewAllocation", this returns a reference to the dependent demand (from a scheduled receipt) that consumed the forecast. Otherwise, null. **Referenced table**: NewAllocation	Reference	
NewTransfer Allocation	If Source is "NewTransferAllocation", this returns a reference to the dependent demand (from a scheduled transfer order) that consumed the forecast. Otherwise, null. **Referenced table**: NewTransferAllocation	Reference	
Part	A reference to the part whose forecast is being consumed by an actual demand. **Referenced table**: Part	Reference	Yes
Planned Allocation	If Source is "PlannedAllocation", this returns a reference to the dependent demand (from a planned order) that consumed the forecast. Otherwise, null. **Referenced table**: PlannedAllocation	Reference	
PlannedTransfer Allocation	If Source is "PlannedTransferAllocation", this returns a reference to the dependent demand (from a planned transfer order) that consumed the forecast. Otherwise, null. **Referenced table**: PlannedTransferAllocation	Reference	

Table 8.58: ForecastConsumption (Mfg) fields

Field name	Description	Data type	Owner
Source	The type of actual demand that is consuming the forecast reported on this record (and hence indicates which of the demand references is valid). Valid values are: • Allocation • IndependentDemand • NewAllocation • NewTransferAllocation • PlannedAllocation • PlannedTransferAllocation	String (Enum)	

ForecastItemParametersActual table

The **ForecastItemParametersActual** table contains the bucketed adjusted history of a forecast item and any additional forecast items specified using the **ForecastItemParametersMap** table. The adjusted history for the forecast item is the sum of the historical actual demand and causal factors for the forecast item or items, bucketed by the calendar defined in **ForecastItemParameters.Type.IntervalsCalendar**. This table reflects the outliers in the **ForecastItemParametersOutlier** table. However, it does not respect the historical date ranges in the Forecast Item setups as the worksheet must filter the date buckets according to the task you are doing.

Quantities are calculated using the following rules:

- A quantity is calculated for each calendar interval.
- The quantity is the sum of the **HistoricalDemandActual** and **CausalFactorDetail** records for the forecast items.
- This forecast item's demand is used in the calculation if **ForecastItemParameters.UseOwnDemand** is set to **'Use'**, and if the **HistoricalDemandCategory** is the same as **ForecastItemParameters.ActualsCategory**.
- Other forecast items that are mapped to this one in the **ForecastItemParametersMap** table are used in the calculation, if the **HistoricalDemandCategory** is the same as **ForecastItemParameters.ActualsCategory**.
- Historical demands less than zero are ignored, but causal factors less than zero are included in the calculation.
- If the historical demand and causal factors are not available, a zero value is used.
- The final **Quantity** for each calendar interval must be greater than or equal to zero.

Table 8.59: ForecastItemParametersActual (Mfg) fields

Field name	Description	Data type	Owner
Date	The interval bucket date.	Date	
ForecastItem Parameters	The **ForecastItemParameters** record the history is calculated for. **Referenced table**: ForecastItemParameters	Reference	Yes
OutlierQuantity	The sum of outlier quantities for the forecast item and any items mapped to it.	Quantity	
Quantity	The adjusted demand for the item and any items mapped to it, calculated as the sum of the following: • **HistoricalDemandActual** records for the specified forecast items, if the **HistoricalDemandCategory** matches the **ForecastItemParameters.ActualsCategory**. If the historical demand is less than zero, it is ignored. • **CausalFactorDetails** records for the specified forecast items, if the **HistoricalDemandCategory** matches the **ForecastItemParameters.ActualsCategory**.	Quantity	
SelfOutlierQuantity	The sum of outlier quantities for just this **ForecastItemParameters** record.	Quantity	
SelfQuantity	The adjusted demand of just this **ForecastItemParameters** record, calculated as the sum of the following: • **HistoricalDemandActual** records for this **ForecastItemParameters** record, if the **HistoricalDemandCategory** matches the **ForecastItemParameters.ActualsCategory**. • **CausalFactorDetails** records for this **ForecastItemParameters** record, if the **HistoricalDemandCategory** matches the **ForecastItemParameters.ActualsCategory**. The item's history is used regardless of the value in the **ForecastItemParameters.UseOwnDemand** field.	Quantity	

ForecastItemParametersRegressionWeight table

The **ForecastItemParametersRegressionWeight** table reports the regression weights for forecast items that are using a multiple linear regression forecast model. Each record stores the regression weight used for one data point (**ForecastItemParametersRegressionWeight.Quantity**), which represents one **HistoricalIntervalCount** period bucket (**ForecastItemParametersRegressionWeight.Index**) starting at **RunDate**. The weight calculation reflects the **ForecastItemParameters.WeightsDecayFactor** that is set. The weights are used to adjust the relative influence of regressors over time in the statistical forecast. As

a result, the regressor values in more recent period buckets are given more importance than the values in earlier period buckets.

The **ForecastItemParametersRegressionWeight.Quantity** is calculated using the following formula:

For the t-th observation in the historical set

ForecastItemParametersRegressionWeight.Quantity = **WeightsDecayFactor**^ (**HistoricalIntervalCount** - **t**)

For more information about how **ForecastItemParametersRegressionWeight.Quantity** is calculated, see "Multiple linear regression calculations" on page 2047.

The **ForecastItemParametersRegressors** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 8.60: ForecastItemParametersRegressionWeight (Mfg) fields

Field name	Description	Data type	Key
Date	Date that represents the historical interval period bucket bucket.	Date	
ForecastItemParameters	A reference to the forecast item that is associated with the regression weights. **Referenced table**: ForecastItemParameters	Reference	
Index	An index value that is associated with one data point. The index value represents the number of historical intervals that occur before RunDate. For example, a regression weight of "2" at index "-1" doubles the weight of the most recent historical data point in the weighted least squares regression, a weight of "2" at index "-2" doubles the weight of the second-most recent point. The index values are always negative.	Integer	
Quantity	The regression weight for historical interval period bucket. This value cannot be negative.	Quantity	

ForecastItemParametersRegressor table

The **ForecastItemParametersRegressors** table contains the values for regressors that are associated with each **RegressionModelTerm**. They are bucketed using the same dates as **ForecastItemParametersActual**. If a **RegressionModelTerm.Lag** is specified for a model term, the lag is taken into account.

The **ForecastItemParametersRegressors** table uses values from the **RegressorValue** table to calculate **ForecastItemParametersRegressor.Quantity**. The **ForecastItemParametersRegressor.Quantity** represents the time-weighted average of the **RegressorValue.Quantity** records that occur from one **ForecastItemParametersActual** date to one day prior to the next **ForecastItemParametersActual** date. However, if within this interval there are dates with no **RegressorValue** records, a **RegressorValue.Quantity** remains in effect starting at **RegressorValue.Date** until one day prior to **RegressorValue.Date** of the next available **RegressorValue** record. Therefore, even if the next regressor

value record occurs only after the following **ForecastItemParametersActual.Date**, the value is still taken into consideration in the **ForecastItemParametersRegressor.Quantity** calculation for the new period bucket. If there is no **RegressorValue** record associated with a date that is equal to, or earlier than, the first **ForecastItemParametersActual.Date** in the historical data series, then a **RegressorValue.Quantity** of zero is applied. For more information about how regressor quantities are calculated, see "Multiple linear regression calculations" on page 2047.

It is also important to note that the values stored in this table are not only historical values. The values within the following range are taken into account in the **ForecastItemParametersRegressor.Quantity** calculation:

RunDate - **ForecastItemParameters.HistoricalIntervalCount** to **RunDate** + **ForecastItemParameters.ForecastIntervalCount**.

However, if **ForecastItemParameters.HistoryStartDate** is after **RunDate** - **ForecastItemParameters.HistoricalIntervalCount**, then the first date that is reported is **ForecastItemParameters.HistoryStartDate**. In addition, if **ForecastItemParameters.HistorySkipLeadingZeroes** is set to **Y**, and there are leading zeroes in the historical data, only the regressor values after the first non-zero value are considered. In other words, if there are regressor values whose dates match historical demand period buckets that have leading zeros, these regressor values are ignored.

The **ForecastItemParametersRegressors** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 8.61: ForecastItemParametersRegressor (Mfg) fields

Field name	Description	Data type	Key
Date	The interval bucket date for this regressor quantity, which is using the same date as **ForecastItemParametersActual**.	Date	
ForecastItemParameters	A reference to the **ForecastItemParameters** record that is associated with the regressor. **Referenced table**: ForecastItemParameters	Reference	Yes
Quantity	The value of the regressor, which includes **RegressionModelTerm.Lag** in the calculation.	Quantity	
RegressionModelTerm	A reference to the model term that is associated with the regressor. **Referenced table**: RegressionModelTerm	Reference	Yes

HistoricalDemandWaterfall table

This **HistoricalDemandWaterfall** table is a consolidation table that consolidates the **HistoricalDemandActual** and **HistoricalDemandSeriesDetail** tables. It can be used to construct waterfall reports showing one row for each series of historical demands and one column for each date bucket. This table allows access to historical forecast and actual demand data in a single report (for example, it can be used to analyze forecast accuracy by comparing against actual shipments).

Table 8.62: HistoricalDemandWaterfall (Mfg) fields

Field name	Description	Data type	Owner
ActualCategory	A reference to the category for this demand data as follows: • If **IsActual** = 'Y', this is a reference to **HistoricalDemandActual.Category.** • If **IsActual** = 'N', this is a reference to **Header.Category**. **Referenced table:** HistoricalDemandCategory	Reference	
Date	The date associated with this historical demand quantity as follows: • If **IsActual** = 'Y', the date comes from **HistoricalDemandActual.Date.** • If **IsActual** = 'N', the date comes from **HistoricalDemandSeriesDetail.Begin Date**.	Date	
Header	A reference to the historical demand header for this series. **Referenced table**: HistoricalDemandHeader	Reference	Yes
IsActual	Indicates if this demand is an actual. Valid values are: • **Y** (this entry comes from the **HistoricalDemandActual** table) • **N** (this entry comes from the **HistoricalDemandSeriesDetail** table)	Boolean	
IsBaseline	Identifies whether this historical demand series is a baseline series. A series is considered a baseline if its AsOfDate belongs to the ToleranceProfile.BaselineCalendar for the associated PartCustomer record. If multiple series have the same AsOfDate belonging to the baseline calendar, then the last series on this date is selected as the baseline. Valid values are: • Y • N	Boolean	

Table 8.62: HistoricalDemandWaterfall (Mfg) fields

Field name	Description	Data type	Owner
Quantity	The quantity of parts in this bucket for the series as follows: • If **IsActual** = 'Y', the quantity comes from **HistoricalDemandActual.Quantity** for this part, customer, date, and actual demand category. • If **IsActual** = 'N', the quantity comes from **HistoricalDemandSeries.Quantity** for this part, customer, series demand category, as-of date, and date. **Note:** If **IsActual** is set to 'N' and the **HistoricalDemandSeriesDetail.BeginDate** is earlier than the **HistoricalDemandSeries.As Of Date**, a quantity of zero is reported in this field. To prevent this from happening, you need to insert all past due amounts into the current bucket when entering any **HistoricalDemandSeriesDetail** data.	Quantity	
QuantityDelta	The difference between the quantity in this bucket for this series and the quantity from the previous baseline series in the same bucket. A positive value indicates this entry has a higher quantity in this bucket than that baseline series, and a negative value indicates this entry has a lower quantity in this bucket than that baseline series. Any series that occurs before the first baseline series reports a value of o (zero) here. All baseline series that occur after the first baseline series, have their QuantityDelta calculated against the previous baseline.	Quantity	
Series	A reference to the historical demand series. **Referenced table**: HistoricalDemandSeries	Reference	
ToleranceProfile Zone	A reference to the entry in the ToleranceProfileZone table that applies to this HistoricalDemandWaterfall record. It is used in defining the acceptable change in historical demands within a given zone or period. The ToleranceProfile for this record is the value referenced from Header.PartCustomer, and the ToleranceProfileZone is set to the entry with the lowest ZoneEndInterval value where the immediately preceding baseline has `Series.AsOfDate + ZoneEndInterval ToleranceProfile.ToleranceCalendar > HistoricalDemandWaterfall.Date`. **Referenced table**: ToleranceProfileZone	Reference	

HistoricalSupplyWaterfall table

This **HistoricalSupplyWaterfall** table is a consolidation table that consolidates the **HistoricalSupplyActual** and **HistoricalSupplySeriesDetail** tables. It can be used to construct waterfall reports showing one row for each series of historical supplies and one column for each date bucket. This table allows access to historical supply series and actual supplies in a single report.

Table 8.63: HistoricalSupplyWaterfall (Mfg) fields

Field name	Description	Data type	Owner
ActualCategory	A reference to the category for this supply data as follows: • If **IsActual** = 'Y', this is a reference to **HistoricalSupplyActual.Category.** • If **IsActual** = 'N', this is a reference to **Header.Category**. Referenced table: HistoricalSupplyCategory	Reference	
Date	The date associated with this historical supply quantity as follows: • If **IsActual** = 'Y', the date comes from **HistoricalSupplyActual.Date.** • If **IsActual** = 'N', the date comes from **HistoricalSupplySeriesDetail.Begin Date**.	Date	
Header	A reference to the historical supply header for this series. **Referenced table**: HistoricalSupplyHeader	Reference	Yes
IsActual	Indicates if this supply is an actual. Valid values are: • **Y** (this entry comes from the **HistoricalSupplyActual** table) • **N** (this entry comes from the **HistoricalSupplySeriesDetail** table)	Boolean	
IsBaseline	Identifies whether this historical supply series is a baseline series. A series is considered a baseline if its AsOfDate belongs to the ToleranceProfile.BaselineCalendar for the associated PartSupplier record. If multiple series have the same AsOfDate belonging to the baseline calendar, then the last series on this date is selected as the baseline. Valid values are: • Y • N	Boolean	
Quantity	The quantity of parts in this bucket for the series as follows: • If **IsActual** = 'Y', the quantity comes from **HistoricalSupplyActual.Quantity** for this part, supplier, date, and actual supply category. • If **IsActual** = 'N', the quantity comes from **HistoricalSupplySeriesDetail.Quantity** for this part, supplier, series demand category, as-of date, and date.	Quantity	

Table 8.63: HistoricalSupplyWaterfall (Mfg) fields

Field name	Description	Data type	Owner
QuantityDelta	The difference between the quantity in this bucket for this series and the quantity from the previous baseline series in the same bucket. A positive value indicates this entry has a higher quantity in this bucket than that baseline series, and a negative value indicates this entry has a lower quantity in this bucket than that baseline series. Any series that occurs before the first baseline series reports a value of 0 (zero) here. All baseline series that occur after the first baseline series, have their QuantityDelta calculated against the previous baseline.	Quantity	
Series	A reference to the historical supply series. **Referenced table**: HistoricalSupplySeries	Reference	
ToleranceProfile Zone	A reference to the entry in the ToleranceProfileZone table that applies to this HistoricalSupplyWaterfall record. It is used in defining the acceptable change in historical supplies within a given zone or period. The ToleranceProfile for this record is the value referenced from Header.PartSupplier, and the ToleranceProfileZone is set to the entry with the lowest ZoneEndInterval value where the immediately preceding baseline has `Series.AsOfDate + ZoneEndInterval ToleranceProfile.ToleranceCalendar > HistoricalSupplyWaterfall.Date`. **Referenced table**: ToleranceProfileZone	Reference	

IndependentDemandByFeature table

The **IndependentDemandByFeature** table can be used to return all independent demands that require a specified feature or group of features.

In order to specify the feature(s) that a demand requires to be reported in this table, the analytic parameter **RR_Analytics_FeatureBOM_FeatureRule** must be used. This parameter is implemented as a workbook variable and allows for the passing of individual feature names, list of features, or more complex feature rules as inputs.

The **RR_Analytics_FeatureBOM_FeatureRule** parameter is, by default, included as a variable in the standard **Orders by Feature** workbook which is recommended for use when a report of orders that use specified features is required. For more information about this workbook, see the *RapidResponse User Guide*. For information about implementing the **RR_Analytics_FeatureBOM_FeatureRule** parameter in custom workbooks, see "Enabling feature-based queries on IndependentDemandByFeature records" on page 2234.

Table 8.64: IndependentDemandByFeature (Mfg) fields

Field name	Description	Data type	Owner
Independent Demand	A reference to a demand that requires the specified features. **Referenced table:** IndependentDemand	Reference	
Part	A reference to the assembly part that the demand reported on this record is for. **Referenced table:** Part	Reference	Yes

IndependentDemandCost table

The **IndependentDemandCost** table reports date-specific cost of goods sold and material spending values from all component supplies pegged to a particular independent demand in the **WhereConsumedForDemand** table. For a given **IndependentDemand** record, this table reports cost of goods sold and material spend values for each date on which one or more component supplies are needed to satisfy the demand.

To report total rolled up cost of goods and material spending for each independent demand, the **IndependentDemand** table's **CostOfGoodsSold** and **MaterialSpend** fields can be used. These calculated fields summarize the values reported in this table and thereby ensure that all date-sensitive exchange rates that might apply during the production of an order are taken into account by RapidResponse during currency conversion calculations.

Table 8.65: IndependentDemandCost (Mfg) fields

Field name	Description	Data type	Owner
ConversionDate	The particular date to which the cost values reported on this record apply. Each demand has one record added to this table for each unique date on which one or more component supplies are needed in order to satisfy the demand. This date is then used as the effective date for any associated currency conversion calculations, and ensures the appropriate date-based rates are reflected on the rolled up cost values as reported in the **IndependentDemand** table.	Date	
CostOfGoodsSold	The total costs of goods sold from all component supplies needed on this date in order to satisfy the referenced independent demand. For a given need date and set of component supplies pegged to this demand in the **WhereConsumedForDemand** table, the **LaborCost, MaterialCost, OverheadCost**, and **OverheadCost2** fields are added together to populate this field.	Money	

Table 8.65: IndependentDemandCost (Mfg) fields

Field name	Description	Data type	Owner
Independent Demand	A reference to the independent demand to which the cost values reported on this record apply. **Reference table:** IndependentDemand	Reference	Yes
MaterialSpend	The total material spending from all component supplies needed on this date in order to satisfy the referenced independent demand. For a given need date and any scheduled receipts or planned orders for purchase parts pegged to this demand in the **WhereConsumedForDemand** table, the **MaterialCost** field values are summed together to populate this field.	Money	

IndependentDemandFeature table

The **IndependentDemandFeatureRule** table reports features according to the demand records where they are used in feature lists. It contains one record for each valid feature included in the **IndependentDemand.FeatureList** field on a given record (including features resulting from expanding the **FeatureList** field based on mappings in the **FeatureMap** table). It also identifies all features that are included in feature lists, but which are not found in the **Feature** table.

Table 8.66: IndependentDemandFeature (Mfg) fields

Field name	Description	Data type	Owner
Feature	A reference to the name of a feature that was included in a feature list. Null if the feature name cannot be found in the **Feature.Value** field on any record. **Referenced table:** Feature	Reference	
Independent Demand	A reference to the demand record on which the feature was included in a feature list. The **IndependentDemand.FeatureList** field can be used to return the initial list of features. **Reference table:** IndependentDemand	Reference	Yes
MissingFeatures	A list of all feature names included in the **IndependentDemand.FeatureList** field that cannot be found in the **Value** field on any **Feature** record. These might be feature names that were typed incorrectly, or valid feature names that have not yet been added to the **Feature** table. If a feature name is not defined in the **Feature** table, it can still be used by feature BOM logic to determine where dependent demands are generated on configurable end-items, however the feature cannot be used in feature mapping and will not be reported in certain calculated tables.	String	

IndependentDemandFeatureSummary table

The **IndependentDemandFeatureSummary** table can be used to return counts of the number of orders that require a given feature. This table contains one record for each feature needed to satisfy at least one demand for a configurable assembly part on a given date.

Table 8.67: IndependentDemandFeatureSummary (Mfg) fields

Field name	Description	Data type	Owner
Count	Total number of independent demands for the assembly part that are due on **DemandDueDate** and require this feature.	Integer	
DemandDueDate	Date on which a configurable assembly part has at least one independent demand due that requires this feature.	Date	
Feature	A reference to the feature. **Referenced table:** Feature	Reference	
Part	A reference to a configurable assembly part that has demands due on DemandDueDate which require this feature. **Referenced table:** Part	Reference	Yes

InProcess table

This table reports work in process (WIP) so it can be reported and summarized. Every supply order (scheduled receipt and planned order) has two corresponding records in this table: one with WIPStart = 'Y' and one WIPStart = 'N'. This way, data for individual orders can be found easily (by selecting WIPStart = 'Y' and using the data on that record) as can period-ending balances (by using Y and N records so the running balance correctly tracks WIP). When calculating running balance, use start dates to add to the balance and end dates to subtract from the balance.

Table 8.68: InProcess (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The part the supply activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table**: Part	Reference	
AvailableDate	The date this supply is likely to be completed by.	Date	
AvailableStart Date	The start date of an order based on AvailableDate as calculated by the CTP analytic. This includes consideration of both materials and capacity.	Date	
CalcStartDate	The calculated start date of an order (including any allowances for constraint). For "make" and "buy" supplies, this is based on either **ReschedDate** (for scheduled receipts) or **DueDate** (for other supply types) as calculated by netting. For "transfer" supplies, this is set to **ShipDate**.	Date	
CTPPlanned Order	The planned order associated with this WIP. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	The CTP substitute supply that creates the substitute supply. **Referenced table:** CTPSubstituteSupply	Reference	
DueDate	Due date for the supply.	Date	
IsPlanned	Valid values are: • Y • N Flags the supply as planned (Y) or as a ScheduledReceipt (N). This field is maintained for legacy purposes, and the new SupplySource field should typically be used instead.	Boolean	

Table 8.68: InProcess (Mfg) fields

Field name	Description	Data type	Owner
Inventory Transfer	The Inventory Transfer associated with this WIP. **Referenced table:** InventoryTransfer	Reference	
Model	The model used by netting calculations for the information in this record. **Referenced table**: Model	Reference	
Part	The part that is being made from this work in process supply. **Referenced table:** Part	Reference	Yes
PartSource	The part source for this supply. **Referenced table:** Part	Reference	
Pool	The model used by netting calculations for the information in this record. **Referenced table**: Pool	Reference	
Quantity	If WIPStart, the Quantity of this supply given in supplier units of measure, otherwise, the negative of the supply quantity.	Quantity	
QuantityUnitOfMeasure	Reference to the unit of measure used for the quantity associated with this record. **Referenced table:** "UnitOfMeasure table" on page 931. **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
ReschedDate	New due date as calculated by netting. This is the date that the WIP is to be closed (and appears as inventory).	Date	
Scheduled Receipt	The scheduled receipt associated with this work in process. **Referenced table:** ScheduledReceipt	Reference	
StartUnit	The unit used by netting calculations for the information in this record.	Quantity	
StockUnit Quantity	The supply quantity converted to inventory units of measure. There is no yield factor built into this field. **Note**: **SupplierUOM** applies to **StockUnitQuantity**.	Quantity	

Table 8.68: InProcess (Mfg) fields

Field name	Description	Data type	Owner
SupplySource	Indicates the type of supply associated with this record. Valid values are: • **ScheduledReceipt** • **PlannedOrder** • **InventoryTransfer** • **SubstituteSupply**	String (Enum)	
WIPStart	Valid values: • Y • N If Y, this record represents the beginning of the work. (Quantity >0). If N, this record represents the end of the work (Quantity <0).	Boolean	

InventoryAnalysis table

This table shows planned bucketed ending inventories, work in process quantities, and supply and demand activity data grouped together for each part. In each date bucket, one record is generated for each unique model, unit, and pool segregation corresponding to a given part.

The size of each bucket used in this table, as well as the date range over which these buckets are generated, is set using Configuration records as discussed in "Configuring analytic calculations using global settings" on page 2203. In addition, bucket sizes and date ranges can be specified for individual workbooks as discussed in "Specify workbook-specific Inventory Analysis buckets" on page 2225. Records in this table are organized into the generated buckets based on the DueDate of the corresponding supply or demand.

Table 8.69: InventoryAnalysis (Mfg) fields

Field name	Description	Data type	Owner
CumDemand	Total accumulated demand to the end of the current bucket.	Quantity	
CumPlanned Demand	Total accumulated planned demand to the end of the current bucket.	Quantity	
CumPlanned Supply	Total accumulated planned supply to the end of the current bucket.	Quantity	
CumSupply	Total accumulated supply to the end of the current bucket (scheduled receipt supply is reported at its ReschedDate, planned order supply is reported at its DueDate, and on hand inventory is reported in the past bucket).	Quantity	

Table 8.69: InventoryAnalysis (Mfg) fields

Field name	Description	Data type	Owner
InProcess	Projected in-process quantity from planned orders and scheduled receipts at the end of the current bucket (in inventory units). An order is considered in process if its planned start date is on or before the last date in the bucket, and its completion date is after the last date in the bucket. Start and completion dates are evaluated based on order type as follows: In-process if StartDate is on or before the bucket end date and DueDate is after the bucket end date. • **'Make' scheduled receipts**—In-process if CalcStartDate is on or before the bucket end date and ReschedDate is after the bucket end date. • **'Make' planned orders**—In-process if StartDate is on or before the bucket end date and DueDate is after the bucket end date. • **'Transfer' scheduled receipts**—In-process if ShipDate is on or before the bucket end date and ReschedDate is after the bucket end date. • **'Transfer' PlannedOrders**—In-process if ShipDate is on or before the bucket end date and DueDate is after the bucket end date. • **'Buy' scheduled receipts**—In-process if ReschedDockDate is on or before the bucket end date and ReschedDate is after the bucket end date. • **'Buy' planned orders**—In-process if DockDate is on or before the bucket end date and DueDate is after the bucket end date. **Note:** "Buy" supplies are reported in this field only if their **SupplyType.BuyInProcessRule** is set to "IncludeDockToStock".	Quantity	
Inventory	Projected inventory at the end of the current bucket. The reported inventory value can be negative to indicate that the planned supply is insufficient to satisfy planned demand. **Note:** When reporting inventory values summed across several parts or model, unit, pool segregations, use `Max (0, Inventory)` to ensure that planned shortages are ignored when calculating total inventory.	Quantity	
Model	The model, if any, used by netting calculations for the activity information in this record. **Referenced table:** Model	Reference	

Table 8.69: InventoryAnalysis (Mfg) fields

Field name	Description	Data type	Owner
Part	The Part that the activity information in this record pertains to. In each bucket, one record is generated for each unique model, unit, and pool segregation corresponding to the part. **Referenced table:** Part	Reference	Yes
Period	The date corresponding to the beginning of the current bucket. In each bucket, one record is generated for each model, unit, and pool combination for the part.	Date	
PeriodDemand	The sum of all demands due within the current bucket.	Quantity	
PeriodPlanned Demand	The sum of all planned demands due within the current bucket.	Quantity	
PeriodPlanned Supply	The sum of all planned supplies due within the current bucket.	Quantity	
PeriodSupply	The sum of all supplies due within the current bucket (scheduled receipt supply is reported at its ReschedDate, planned order supply is reported at its DueDate, and on hand inventory is reported in the past bucket).	Quantity	
PlannedIn Process	Projected in-process quantity from planned orders at the end of the current bucket (in inventory units). An order is considered in process if its planned start date is on or before the last date in the bucket, and its completion date is after the last date in the bucket. Start and completion dates are evaluated based on order type as follows: • **'Make' planned orders**—In-process if StartDate is on or before the bucket end date and DueDate is after the bucket end date. • **'Transfer' planned orders**—In-process if ShipDate is on or before the bucket end date and DueDate is after the bucket end date. • **'Buy' planned orders**—In-process if DockDate is on or before the bucket end date and DueDate is after the bucket end date. **Note:** "Buy" supplies are reported in this field only if their **SupplyType.BuyInProcessRule** is set to "IncludeDockToStock".	Quantity	
Pool	The pool, if any, used by netting calculations for the activity information in this record. **Referenced table:** Pool	Reference	

Table 8.69: InventoryAnalysis (Mfg) fields

Field name	Description	Data type	Owner
StartUnit	The unit used by netting calculations for the activity information in this record.	Quantity	
TotalDemand	The total demand, summed across all buckets considered in this table. This field is intended to simplify reporting excess inventory. There is considered to be excess when CumSupply (accumulated supply) exceeds TotalDemand.	Quantity	

InventoryCTPAnalysis table

This table shows bucketed ending inventories, work in process quantities, and supply and demand activity data grouped together for each part. The calculations reported in this table are based on projected material and constraint availability). In each date bucket, one record is generated for each unique model, unit, and pool segregation corresponding to a given part.

The size of each bucket used in this table, as well as the date range over which these buckets are generated, is set using Configuration records as discussed in "Configuring analytic calculations using global settings" on page 2203. In addition, bucket sizes and date ranges can be specified for individual workbooks as discussed in "Specify workbook-specific Inventory Analysis buckets" on page 2225. Records in this table are organized into these buckets based on the DueDate or SynchronizedDate of the corresponding demand, or AvailableDate of the corresponding supply.

Note: This table does not currently report push planning activity.

Table 8.70: InventoryCTPAnalysis (Mfg) fields

Field name	Description	Data type	Owner
CumDelayed	Total accumulated supply planned in or before the current bucket, but not projected to be available until a later date.	Quantity	
CumDemand	Total accumulated demand to the end of the current bucket (demand is reported at its SynchronizedDate).	Quantity	
CumPlanned Delayed	Total accumulated planned orders planned in or before the current bucket, but not projected to be available until a later date.	Quantity	
CumPlanned Demand	Total accumulated planned demand to the end of the current bucket (demand is reported at its SynchronizedDate).	Quantity	

Table 8.70: InventoryCTPAnalysis (Mfg) fields

Field name	Description	Data type	Owner
CumPlanned Supply	Total accumulated planned supply to the end of the current bucket (supply is reported at its AvailableDate). **Note:** Any planned supply expiring by the end of the current bucket is not included.	Quantity	
CumSupply	Total accumulated supply to the end of the current bucket (supply is reported at its AvailableDate). **Note:** Any supply expiring by the end of the current bucket is not included.	Quantity	
CumSynchronized Demand	Accumulated dependent demand from planned orders that have SynchronizedDate from Past to the end of the current bucket.	Quantity	
CumSynchronized PlannedDemand	Accumulated demand for all demands that SynchronizedDate from past to the end of the current bucket.	Quantity	
InProcess	Projected in-process quantity (in inventory units) at the end of the current bucket. Calculated based on the projected starting and available dates of supplies as follows: • **'Make' scheduled receipts and planned orders**—AvailableStartDate to AvailableDate • **'Buy' scheduled receipts and planned orders**—dock date calculated from AvailableStartDate to AvailableDate • **'Transfer' scheduled receipts and planned orders**—shipping date calculated from AvailableStartDate to AvailableDate	Quantity	
Inventory	Projected inventory at the end of the current bucket (calculated as CumSupply less CumDemand). **Note:** When reporting inventory values summed across several parts or model, pool, unit segregations, use `Max (0, Inventory)` to ensure that projected shortages are ignored when calculating total inventory.	Quantity	
Model	The model, if any, used by netting calculations for the activity information in this record. **Referenced table:** Model	Reference	

Table 8.70: InventoryCTPAnalysis (Mfg) fields

Field name	Description	Data type	Owner
Part	The Part that the activity information in this record pertains to. In each bucket, one record is generated for each unique model, unit, and pool segregation corresponding to the part. **Referenced table:** Part	Reference	Yes
Period	The date corresponding to the beginning of the current bucket. In each bucket, one record is generated for each unique model, unit, and pool segregation corresponding the part.	Date	
PeriodDemand	The sum of all demands due within the current bucket.	Quantity	
PeriodPlanned Demand	The sum of all planned demands due within the current bucket.	Quantity	
PeriodPlanned Supply	The sum of all supplies due within the current bucket (supply is reported at its AvailableDate).	Quantity	
PeriodPlanned SupplyExpiry	The sum of all planned supplies expiring within the current bucket.	Quantity	
PeriodSupply	The sum of all supplies due within the current bucket (supply is reported at its AvailableDate).	Quantity	
PeriodSupplyExpiry	The sum of all supplies expiring within the current bucket.	Quantity	
Period Synchronized Demand	The sum of all demands for the part where SynchronizedDate is in the current bucket.	Quantity	
Period Synchronized PlannedDemand	The sum of all planned demands for the part where SynchronizedDate is in the current bucket.	Quantity	

Table 8.70: InventoryCTPAnalysis (Mfg) fields

Field name	Description	Data type	Owner
PlannedInProcess	Projected in-process quantity from planned orders at the end of the current period. Calculated based on the projected starting and available dates of supplies as follows: • **'Make' scheduled receipts and planned orders**—AvailableStartDate to AvailableDate • **'Buy' scheduled receipts and planned orders**—dock date calculated from AvailableStartDate to AvailableDate • **'Transfer' scheduled receipts and planned orders**—shipping date calculated from AvailableStartDate to AvailableDate	Quantity	
Pool	The pool, if any, used by netting calculations for the activity information in this record. **Referenced table:** Pool	Reference	
StartUnit	The unit used by netting calculations for the activity information about this record.	Quantity	
TotalDemand	The total demand, summed across all buckets considered in this table.	Quantity	

InventorySummary table

The **InventorySummary** table supports the Model-Effectivity, Unit-Effectivity, and Inventory Pooling analytics. It represents the statistics compiled when performing netting on parts for a specific pool, model, and start unit. The statistics are based upon the ones calculated in the **Part** table, but are calculated by netting on each unique pool, model and start unit combination.

The Part.MUEPoolNettingType.PoolRule, ModelRule, and UnitRule will determine how the pool, model, and start unit combinations are used in netting. The rules are outlined in the following table.

Table 8.71: How netting uses pool, model and start unit

Value of Part.MUEPoolNettingType	Description
PoolRule = Ignore	Only Model and StartUnit can be used for the unique combination (Pool is set to Unpooled).
ModelRule = Ignore	Only Pool and StartUnit can be used for the unique combination (Model will be set to None).
UnitRule = Ignore, Next, or Increment	Only the Pool and Model can be used for the unique combination (StartUnit will be set to –1).

The following table shows examples of InventorySummary records.

Table 8.72: Examples of InventorySummary records

Value of Part.MUEPoolNettingType	Resulting record(s) in InventorySummary table
PoolRule = *Ignore* ModelRule = *Ignore* UnitRule = *Ignore*	Only one record is created in the InventorySummary table, with the following values: • Pool = *Unpooled* • Model = *None* • StartUnit = *-1*
PoolRule = *Ignore* ModelRule = *Net* UnitRule = *Net*	Records are created in the InventorySummary table for each unique combination of Model and StartUnit. The Pool is set to *Unpooled*.
PoolRule = *Net* ModelRule = *Ignore* UnitRule = *Ignore*	Records are created in the InventorySummary table for each unique Pool. The Model is set to *None* and StartUnit = *-1*.

The following table describes the fields in the **InventorySummary** table.

Table 8.73: InventorySummary (Mfg) fields

Field name	Description	Data type	Owner
FirstNeeded SupplyDate	The date when the on hand inventory of the part will run out, that is the first date when a supply other than on hand is needed to satisfy demand by the scheduled due date.	Date	
FirstShortageDate	The earliest date that inventory will drop below zero (not safety stock) if no rescheduling or new orders are generated.	Date	
MaxCritical Shortage	The greatest amount short for any day within the lead-time for a part. A shortage occurs when ending inventory is less than zero for the day.	Quantity	
Model	The model to which these statistics apply. **Referenced table:** Model	Reference	
Part	The part to which these statistics apply. **Referenced table:** Part	Reference	Yes
Pool	The pool to which these statistics apply. **Referenced table:** Pool	Reference	
StartUnit	The starting unit to which these statistics apply.	Integer	
TotalAllocations	Total quantity of allocations used in netting.	Quantity	

Table 8.73: InventorySummary (Mfg) fields

Field name	Description	Data type	Owner
TotalByProduct Supply	Total quantity of supply for this part that was generated as a by-product residual to the production of a primary product.	Quantity	
TotalCoProduct Supply	Total quantity of supply for this part that was generated as a co-product because of the production of a primary product.	Quantity	
TotalDemand	Total quantity of demand being requested. This summarization is faster than doing a summarized query on the planned order table.	Quantity	
TotalEffPlanned Orders	The total of the expected quantity to receive into inventory of all planned orders for the part. This calculation handles UoM conversion, and scrap/yield.	Quantity	
TotalEff Scheduled Receipts	The total of the expected quantity to receive into inventory of all scheduled receipts for the part. This calculation handles unit of measure (UOM) conversion, and scrap/yield.	Quantity	
TotalForecast	The total of all forecast records.	Quantity	
TotalIndependent Demand	Total quantity of independent demand used in netting. This does not include forecasted independent demands. That is, IndependentDemand records where **Order.Type.ProcessingRule** is set to "SalesForecast" are excluded from this calculation.	Quantity	
TotalIndependent DemandCount	The number of independent demand records that are reflected in the TotalIndependentDemand field (for example, records that were not ignored in MRP processing).	Quantity	
TotalInventory Transfer	The sum of effective supplies of Inventory Transfers for the part.	Quantity	
TotalInventory TransferAllocation	The sum of effective demands by Inventory Transfers for the part.	Quantity	
TotalNeeded Supply	Total quantity from ScheduledReceipt records that are needed to meet demand. This summarization is faster than doing a summarized query on the Supply table. This does not take into account the yield factor.	Quantity	

Table 8.73: InventorySummary (Mfg) fields

Field name	Description	Data type	Owner
TotalNettable OnHand	Total quantity of OnHand for the part that is available to meet demand. This summarization is faster than doing a summarized query on the OnHand table.	Quantity	
TotalNew Allocations	Total quantity of NewAllocations used in netting.	Quantity	
TotalNon Nettable OnHand	Total quantity of OnHand for the part that is not available for satisfying demand.	Quantity	
TotalPlanned Allocations	Total quantity of PlannedAllocations used in netting.	Quantity	
TotalPlanned Orders	Total quantity of planned orders being requested. This summarization is faster than doing a summarized query on the planned order table. This does not take into account the yield factor.	Quantity	
TotalSubstitute Allocation	The total amount of supply from this part used to satisfy demands for a primary part.	Quantity	
TotalSubstitute Supply	The total amount of supply from substitute parts used to satisfy demand for this part.	Quantity	
TotalSupply	Total quantity from ScheduledReceipt records that are available to meet demand. This summarization is faster than doing a summarized query on the Supply table. This does not take into account the yield factor.	Quantity	
TotalUnconsumed Forecast	The amount of forecast that is not covered by actual sales demand.	Quantity	

ItemSourceCandidate table

The **ItemSourceCandidate** table is a calculated table that identifies and reports combinations of **Part**, **Source**, and **HistoricalSupplyCategory** records that are **ItemSource** candidates. That is to say, this table provides a list of Parts and Sources of supply that have historical supply data that can be used by the Machine Learning service.

This table was added in Version 2016.2, Service Update 21.

Table 8.74: ItemSourceCandidate (Mfg) fields

Field name	Description	Data type	Owner
BOMAlternate	A reference to the alternate BOM for the **ItemSourceCandidate**. **Referenced table:** BOMAlternate (Nullable) **Reference syntax:** BOMAlternate.Value	Reference	
HistoricalSupplyCategory	A reference to the **HistoricalSupplyCategory** for which there exists at least one **HistoricalSupplyActual** record. **Referenced table:** HistoricalSupplyCategory	Reference	
Part	A reference to the **Part** for which there exists at least one **PartSource** record. **Referenced table:** Part	Reference	Yes
Routing	A reference to the sequence of operations (**Routing**) associated with the **ItemSourceCandidate**. **Referenced table:** Routing **Reference syntax:** Routing.Id, Routing.Site	Reference	
Source	A reference to the **Source** for which there exists at least one **PartSource** record. **Referenced table:** Source	Reference	

ItemSourceIntervalEffectivePartSource table

The **ItemSourceIntervalEffectivePartSource** table is a calculated table that reports the set of effective **PartSource** records for a given **ItemSourcePartSourceEffectivityInterval**. For more information, see the "ItemSourcePartSourceEffectivityInterval table" on page 1089.

This table was added in Version 2016.2, Service Update 21.

Table 8.75: ItemSourceIntervalEffectivePartSource (Mfg) fields

Field name	Description	Data type	Owner
Interval	A reference to the **ItemSourcePartSourceEffectivityInterval**. **Referenced table:** ItemSourcePartSourceEffectivityInterval	Reference	
ItemSource	A reference to the **ItemSource**. **Referenced table:** ItemSource	Reference	Yes

Table 8.75: ItemSourceIntervalEffectivePartSource (Mfg) fields

Field name	Description	Data type	Owner
PartSource	A reference to the **PartSource**. **Referenced table:** PartSource	Reference	

ItemSourcePartSourceEffectivityInterval table

The **ItemSourcePartSourceEffectivityInterval** table is a calculated table that reports **PartSource** "effectivity intervals", which are the intervals in which the set of effective **PartSource** records are constant for a given **ItemSource**. For each interval, the **ItemSourceIntervalEffectivePartSource** table enumerates the effective **PartSource** records.

That is to say, consider the set of matching **PartSource** records reported for a given **ItemSource**: for any given date, a subset of those **PartSource** records is effective. The effective subset changes on dates that are some **PartSource**'s **FirstEffectiveDate** or **LastEffectiveDate**. For more information, see the "ItemSourceIntervalEffectivePartSource table" on page 1088.

Note: The **FirstEffectiveDate** and **LastEffectiveDate** fields are different from **PartSource.FirstEffectiveDate** and **PartSource.LastEffectiveDate** because there may not be a PartSource with this exact pair of effective dates.

This table was added in Version 2016.2, Service Update 21.

Table 8.76: ItemSourcePartSourceEffectivityInterval (Mfg) fields

Field name	Description	Data type	Owner
FirstEffectiveDate	The start date of this interval.	Date	
ItemSource	A reference to the **ItemSource**. **Referenced table:** ItemSource	Reference	Yes
LastEffectiveDate	The (inclusive) end date of this interval.	Date	

ItemSourcePartSourceHistoricalLeadTime table

The **ItemSourcePartSourceHistoricalLeadTime** table is a calculated table that matches **HistoricalSupplyActual** records for **ItemSources** with lead times from effective **PartSource** records.

Matching **HistoricalSupplyActual** records with a **PartSource** is necessary to calculate the error between the designed lead times (**PartSourceLeadTime**) and the demonstrated lead times (**ActualLeadTime**).

The data in this table is passed to the Machine Learning server to calculate the results for **ProjectedLeadTime**, **Cluster**, and **ClusterProperty**.

Note: A PartSource record for an ItemSource needs to be effective after the MRPDate.

This table was added in Version 2016.2, Service Update 21.

Table 8.77: ItemSourcePartSourceHistoricalLeadTime (Mfg) fields

Field name	Description	Data type	Owner
ActualDate	The date (**HistoricalSupplyActual.Date**) associated with the actual shipment.	Quantity	
ActualHeader	A reference to the historical supply header (**HistoricalSupplyActual.Header**) associated with the record. This identifies the part, supplier, and supply category for the actual order. **Referenced table:** HistoricalSupplyHeader	Reference	
ActualLeadTime	The lead time from **HistoricalSupplyActual** in the **PartSource.Source.LeadTimeUnit** calendar. This value is given by **ActualDate** - **OrderDate** 'LeadTimeUnit', where **ActualDate** = **HistoricalSupplyActual.Date** and **OrderDate** is given by: • **HistoricalSupplyActual.OrderDate** (if defined); • or **ActualDate** - **HistoricalSupplyActual.LeadTime** '**HistoricalSupplyActual.Header.LeadTimeCalendar**', (if **LeadTime** is positive). However, if **OrderDate** is undefined and **LeadTime** is negative, then the **HistoricalSupplyActual** record is ignored. **Caution:** **HistoricalSupplyActual.OrderDate** overrides **HistoricalSupplyActual.LeadTime**, and not vice-versa. If using **LeadTime**, set **OrderDate** to "Undefined".	Quantity	
ActualLeadTimeDays	The lead time from **HistoricalSupplyActual** in the Everyday calendar. This value is given by **ActualDate** - **OrderDate** 'Everyday', where **ActualDate** = **HistoricalSupplyActual.Date** and **OrderDate** is given by: • **HistoricalSupplyActual.OrderDate** (if defined); • or **ActualDate** - **HistoricalSupplyActual.LeadTime** '**HistoricalSupplyActual.Header.LeadTimeCalendar**', (if **LeadTime** is positive). However, if **OrderDate** is undefined and **LeadTime** is negative, then the **HistoricalSupplyActual** record is ignored.	Quantity	
ActualQuantity	The number of units (**HistoricalSupplyActual.Quantity**) in the actual order.	Quantity	

Table 8.77: ItemSourcePartSourceHistoricalLeadTime (Mfg) fields

Field name	Description	Data type	Owner
ActualUnitPrice	The unit price (**HistoricalSupplyActual.UnitPrice**) for the historical supply.	Quantity	
ItemSource	The **ItemSource** associated with the historical supply. **Referenced table:** ItemSource	Reference	Yes
PartSource	The **PartSource** associated with the historical supply. **Referenced table:** PartSource	Reference	
PartSourceLeadTime	The **PartSource.FixedLeadTime** or a value from **TimePhasedPartSourceLeadTime**, in the **PartSource.Source.LeadTimeUnit** calendar.	Quantity	

ItemSourcePartSourceHistoricalReceipt table

The **ItemSourcePartSourceHistoricalReceipt** table is a calculated table that matches **HistoricalReceipt** records for **ItemSources** with yields from effective **PartSource** records.

Matching **HistoricalReceipt** records with a **PartSource** is necessary to calculate the error between the designed yields (**PartSourceYield**) and the demonstrated yields (**ActualYield**). The data in this table is passed to the Machine Learning server to calculate the results for **ProjectedYield**, **Cluster**, and **ClusterProperty**.

Note: A PartSource record for an ItemSource needs to be effective after the MRPDate.

Table 8.78: ItemSourcePartSourceHistoricalReceipt (Mfg) fields

Field name	Description	Data type	Owner
HistoricalReceipt	The **HistoricalReceipt** associated with the historical supply. **Referenced table:** HistoricalReceipt	Reference	
ItemSource	The **ItemSource** associated with the historical supply. **Referenced table:** ItemSource	Reference	Yes
PartSource	The **PartSource** associated with the historical supply. **Referenced table:** PartSource	Reference	

Table 8.78: ItemSourcePartSourceHistoricalReceipt (Mfg) fields

Field name	Description	Data type	Owner
PartSourceYield	The effective yield, based on **OrderPolicy**.**YieldUsage**.	Quantity	
EffectiveHistoricalYield	The effective yield, reported in **ItemSource**.**CalcYieldUsageRule** units, based on **HistoricalReceipt**.**ReceivedQuantity** / **HistoricalReceipt**.**StartQuantity**. Reports -1 if the yield is invalid.	Quantity	
AdjustedPartSourceYield	The adjusted yield, reported in **ItemSource**.**CalcYieldUsageRule** units, calculated using the following steps: • Convert invalid yield values to 100%. • Multiply the **PartSourceYield** by the co-product yield. This adjusted quantity matches the yield used to inflate demands in supply planning calculations.	Quantity	

LateSupply table

The records in this table show the detailed supply associated with a given independent demand and expected to be available too late for an on-time delivery of an order. For example, if an order is delayed and unable to satisfy demand due to lack of available components, a record is added to this table. The supply records are either scheduled receipts or planned orders.

When **LateSupply** records are created, RapidResponse filters all **WhereConsumed** records with AvailableDate greater than NeedDate or EffDueDate greater than NeedDate.

It's recommended that you use the **WhereConsumed** table as opposed to the **LateSupply** table. For information about the relationship between the **WhereConsumed** table and the **LateSupply** table, see "Relationship with LateSupply" on page 1225.

Table 8.79: LateSupply (Mfg) fields

Field name	Description	Data type	Owner
ActionSupply	Indicates if this supply requires direct action to achieve on time delivery of its top-level driver (**Y** or **N**). That is, the supply is late itself, not just that its components are late: • Supply is late (Available later than Need Date) And: • Resched Date is after the Need Date Or: • Constrained at this part (Constraint is making this supply later, that is, SupplyDemand.Constraint is not Null) Or: • Inside lead time (see InsideLeadTime field)	Boolean	
AvailableDate	The date when this supply record is expected to be available. This is based on time-fences, and expected component availability.	Date	
Constraint	The constraint that has insufficient remaining capacity to make this supply available for its MaterialAvailableDate. If this supply is not late, this field is empty. **Referenced table:** Constraint	Reference	
CTPCoProduct Supply	If the supply represented by this record is a co-product or by-product generated from production of a primary product, this is a reference to that co-product or by-product supply. **Referenced table:** CTPCoProductSupply	Reference	
CTPPlannedOrder	The CTP planned order supplying this quantity. Null unless IsPlanned. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	The substitute supply associated with this late supply. **Referenced table:** CTPSubstituteSupply	Reference	
DriverDueDate	The DueDate of the driver for this demand. DriverDueDate is valid regardless of the type of driver.	Date	

Table 8.79: LateSupply (Mfg) fields

Field name	Description	Data type	Owner
DriverNeed Quantity	The amount of the driver quantity that is ultimately satisfied by this pegged supply. This field indicates how much of the driver could be affected by this specific component supply, and hence can be used determine which supplies are having the greatest impact on a late order. **Note:** This field was added in Version 2016.2, Service Update 4. **Caution:** In linear BOM structures, this field can be used with NeedQuantity to calculate how many units of the component is ultimately needed to make a unit of the driver part. However in non-linear BOM structures, this can give misleading results as different values are likely to be returned based on the different sub-assemblies/paths under the driver where the lower-level component is used.	Quantity	
DriverQuantity	The total Quantity of the driver.	Quantity	
DueDate	The date the order is planned to be in stock.	Date	
Independent Demand	The IndependentDemand ultimately consuming this late supply. Includes forecast records. Note IndependentDemand references the top-level forecast, but other LateSupply fields use data from the lowest forecast (after spreading and explosion). **Reference table:** IndependentDemand	Reference	Yes
InsideLeadTime	Tracks whether the supply is inside its lead time. For PlannedOrders, if the order needs to start before RunDate, shows the number of work days before Run Date to start (otherwise, 0). For released Scheduled Receipts, indicates the supply has a ReschedDate before DataDate. Reports the difference in work days (otherwise 0). If unreleased Scheduled Receipts, indicates the supply has a CalcStartDate before DataDate. Reports the difference in work days (otherwise 0).	Integer	
Inventory Transfer	Indicates that the supply is an inventory transfer. **Referenced table:** InventoryTransfer	Reference	

Table 8.79: LateSupply (Mfg) fields

Field name	Description	Data type	Owner
Late	Number of periods by which the supply is late to achieve on-time delivery of its driver, based on its AvailableDate. Calculated as Supply.AvailableDate - where consumed need date using Part.PlanningCalendars.TimeUnits.	Integer	
LateAtHere	Number of periods by which the supply is late to satisfy the driver, based on the supply due date. Calculated as Supply.DueDate (adjusted to stock date) - need date from driver using Part.PlanningCalendars.TimeUnits.	Integer	
LatePart	A component part whose supply is late. **Referenced table:** Part	Reference	
LateQuantity	The amount of this supply record that is late for meeting the requirements of the particular demand.	Quantity	
NeedDate	The date when this supply record is needed in order to achieve on-time delivery for the particular demand.	Date	
NeedQuantity	The amount of this supply record that is required to meet the particular demand.	Quantity	
OrderPriority	The priority associated with this supply. **Referenced table:** OrderPriority	Reference	
PartSource	The part source applicable to this supply. (must always be valid since OnHand cannot be late). **Referenced table:** PartSource	Reference	
PegPart	The immediate parent to this part (whose supply is creating the demand if the demand is dependent). **Referenced table:** Part	Reference	
Pool	The pool used for netting this supply. (The driver Pool is available from IndependentDemand. Pool mapping may change the Pool.) **Referenced table:** Pool	Reference	
Quantity	Total quantity of the supply.	Quantity	

Table 8.79: LateSupply (Mfg) fields

Field name	Description	Data type	Owner
QuantityUnitOfMeasure	Reference to the unit of measure used for the quantity associated with this record. **Referenced table:**"UnitOfMeasure table" on page 931. **Note**: This field was added in Version 2016.2, Service Update 12.	Reference	
Scheduled Receipt	Reference to the source of supply. NULL if LateSupply is from a PlannedOrder. **Referenced table**: ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the supply, this represents the date a portion of the supply is available. If incremental availability calculations are turned off for the supply, this represents the date when the entire supply is available (except for cases where a planned order is split based upon constraint allocation).	Quantity	
StartUnit	The start unit that applies to the demand side of this record. It may differ from the IndependentDemand.StartUnit due to demand splitting.	Integer	
SupplyIsPlanned	Indicates that the supply is a planned order (Planned Order reference is valid, ScheduledReceipt reference is not valid). Otherwise, the ScheduledReceipt reference is valid. (OnHand cannot be late). Valid values are: • Y • N	Boolean	
SupplySource	Indicates the type of supply associated with this record. Valid values are: • CoByProductSupply • InventoryTransfers • PlannedOrders • ScheduledReceipts • SubstituteSupplies	String	
SupplyType	The type of the late supply. (OnHand is never late.) **Referenced table:** SupplyType	Reference	
Transaction Sequence	Transaction sequence of the supply.	Integer	

LoadDailyCurrent table

The LoadDailyCurrent table describes all load and capacity on a work center from scheduled receipts through operations and scheduled operations using the current due dates.

Records are generated for each day that has load. In addition, records are generated for each day greater than or equal to the MRP RunDate and up to the capacity report limit on days of the workday (time unit) calendar that do not have load. These non "load" records will have the daily capacity and have zero for the load numbers. This is how the total available capacity and number of machines for any given bucketing is calculated.

Table 8.80: LoadDailyCurrent (Mfg) fields

Field name	Description	Data type	Owner
Date	Date of this load.	Date	
Demonstrated Labor Capacity	The average number of standard labor hours that are available in a day for all workers.	Quantity	
Demonstrated MachineCapacity	The average number of standard machine hours of production in a day that has been demonstrated for this work center.	Quantity	
IsInsideCapacity ReportLimit	A flag that indicates whether the date that is represented by this record is inside the horizon that is established by the CapacityReportLimit for the work center.	String (Enum)	
LaborCapacity	Total labor capacity at this work center for this day.	Quantity	
LaborIdle	The number of standard labor hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center on the date generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center on the date generated during setup time.	Quantity	
MachineCapacity	Total machine capacity at this work center for this day.	Quantity	
MachineIdle	The number of standard machine hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	

Table 8.80: LoadDailyCurrent (Mfg) fields

Field name	Description	Data type	Owner
MachineRun	The number of standard machine hours of load placed on this work center on the date generated during run time.	Quantity	
MachineSetup	The number of standard machine hours of load placed on this work center on the date generated during setup time.	Quantity	
MaximumLabor Capacity	The maximum labor capacity for this work center, on this day, as described by the appropriate capacity record.	Quantity	
MaximumMachine Capacity	The maximum machine capacity for this work center on this day as described by the appropriate capacity record.	Quantity	
NumberOf Machines	The number of machines at this work center on this day.	Quantity	
NumberOfWorkers	The number of workers required for parallel resources at the work center on this day.	Quantity	
WorkCenter	Reference to the WorkCenter on which this load has been placed. **Referenced table:** WorkCenter	Reference	Yes

LoadDailyPlanned table

The **LoadDailyPlanned** table describes all load on a work center from scheduled receipts and planned orders through operations and scheduled operations using the rescheduled scheduled receipt dates. This load will match the allocation dates for the materials being used in the operations as planned by RapidResponse.

Table 8.81: LoadDailyPlanned (Mfg) fields

Field name	Description	Data type	Owner
Date	The date of this load.	Date	
Demonstrated LaborCapacity	The average number of standard labor hours that are available in a day for all workers.	Quantity	
Demonstrated MachineCapacity	The average number of standard machine hours that have been shown to be available in a day for all workers.	Quantity	
IsInsideCapacity ReportLimit	A flag that indicates whether the date that is represented by this record is inside the horizon that is established by the CapacityReportLimit for the work center.	Boolean	

Table 8.81: LoadDailyPlanned (Mfg) fields

Field name	Description	Data type	Owner
LaborCapacity	Total labor capacity at this work center for this day.	Quantity	
LaborIdle	The number of standard labor hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center on the date generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center on the date generated during setup time.	Quantity	
MachineCapacity	Total machine capacity at this work center for this day.	Quantity	
MachineIdle	The number of standard machine hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	
MachineRun	The number of standard machine hours of load placed on this work center on the date generated during run time.	Quantity	
MaximumLabor Capacity	The maximum labor capacity for this work center on this day as described by the appropriate capacity record.	Quantity	
MaximumMachine Capacity	The maximum machine capacity for this work center on this day as described by the appropriate capacity record.	Quantity	
NumberOf Machines	The number of machines, at this work center, on this day as described by the appropriate capacity record.	Quantity	
NumberOfWorkers	The number of workers for parallel operations that can be performed at this work center on this day as described by the appropriate capacity record.	Quantity	
Setup	The number of standard hours of load placed on this work center on the date generated during setup time.	Quantity	
SRLaborIdle	The number of standard labor hours spent at this work center on this date waiting for batches to arrive from the previous operation while working on scheduled receipts. This time is included in the SRLaborRun time.	Quantity	

Table 8.81: LoadDailyPlanned (Mfg) fields

Field name	Description	Data type	Owner
SRLaborRun	The number of standard labor hours of load placed on this work center on the date generated during run time from scheduled receipts.	Quantity	
SRLaborSetup	The number of standard labor hours of load placed on this work center on the date generated during setup time from scheduled receipts.	Quantity	
SRMachineIdle	The number of standard machine hours spent at this work center on this date waiting for batches to arrive from the previous operation while working on scheduled receipts. This time is included in the SRMachineRun time.	Quantity	
SRMachineRun	The number of standard machine hours of load placed on this work center on the date generated during run time from scheduled receipts.	Quantity	
SRMachineSetup	The number of standard hours of load placed on this work center on the date generated during setup time from scheduled receipts.	Quantity	
WorkCenter	Reference to the work center on which this load has been placed. **Referenced table:** WorkCenter	Reference	Yes

LoadDetailCurrent table

The LoadDetailCurrent table describes each load on a work center from scheduled receipts through operations and scheduled operations using the current due dates.

Table 8.82: LoadDetailCurrent (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **Operation** table. **Referenced table:** CRPOperation	Reference	
Date	The date of this load. If an operation spans several days, there will be a LoadDetailCurrent record generated for each date that has load.	Date	

Table 8.82: LoadDetailCurrent (Mfg) fields

Field name	Description	Data type	Owner
LaborIdle	The number of standard labor hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center on the Date generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center on the Date generated during setup time.	Quantity	
MachineIdle	The number of standard machine hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	
MachineRun	The number of standard machine hours of load placed on this work center on the Date generated during run time.	Quantity	
MachineSetup	The number of standard machine hours of load placed on this work center on the Date generated during setup time.	Quantity	
NextWorkCenter	The name of the next work center.	String	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	
OperationEndDate	The date that this operation ends on (end of wait time).	Date	
OperationStartDate	The date of the queue time for the operation.	Date	
PrevWorkCenter	The name of the previous work center.	String	
QtyRemaining	The number of units remaining to be processed from the order through the operation.	Quantity	
QtyThisDate	The number of units processed from the order through the operation on this date.	Quantity	
QtyToDate	The number of units processed from the order through the operation up to and including this date.	Quantity	

Table 8.82: LoadDetailCurrent (Mfg) fields

Field name	Description	Data type	Owner
ScheduledReceipt	Reference to the order that created this load. **Referenced table:** ScheduledReceipt	Reference	
Source	This field denotes the source of the load. Valid value is: **ScheduledReceipt.**	String (Enum)	
StartUnit	The start unit that applies to the portion of the supply that created this load.	Integer	
WorkCenter	Reference to the work center on which this load has been placed. **Referenced table:** WorkCenter	Reference	Yes

LoadDetailPlanned table

The LoadDetailPlanned table describes each load on a work center from scheduled receipts and planned orders, through operations and scheduled operations, using the rescheduled scheduled receipt dates.

Table 8.83: LoadDetailPlanned (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **Operation** table. **Referenced table:** CRPOperation	Reference	
Date	The date of this load. If an operation spans several days, there will be a LoadDetailPlanned record generated for each date that has load.	Date	
LaborIdle	The number of standard labor hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center on the Date generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center on the Date generated during setup time.	Quantity	

Table 8.83: LoadDetailPlanned (Mfg) fields

Field name	Description	Data type	Owner
MachineIdle	The number of standard machine hours spent at this work center on this date waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	
MachineRun	The number of standard machine hours of load placed on this work center on the Date generated during run time.	Quantity	
MachineSetup	The number of standard machine hours of load placed on this work center on the Date generated during setup time.	Quantity	
NextWorkCenter	The name of the next work center.	String	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	
OperationEndDate	The date on which this operation ends (for example, end of wait time).	Date	
OperationStartDate	The beginning of queue time.	Date	
Part	Reference to the part that created this load. **Referenced table:** Part	Reference	
PegModel	The model for the supply that created the load. **Referenced table:** Model	Reference	
PegPool	The pool for the supply that created the load. **Referenced table:** Pool	Reference	
PrevWorkCenter	The name of the previous work center.	String	
PlannedOrder	If Source = 'PlannedOrder', this field references the planned order that created the load. **Referenced table**: PlannedOrder	Reference	
QtyThisDate	The number of units processed from the order through the operation on this date.	Quantity	

Table 8.83: LoadDetailPlanned (Mfg) fields

Field name	Description	Data type	Owner
QtyToDate	The number of units processed from the order through the operation up to and including this date.	Quantity	
QtyRemaining	The number of units remaining to be processed from the order through the operation.	Quantity	
ScheduledReceipt	If Source = 'ScheduledReceipt', this field references the scheduled receipt that created the load. **Referenced table**: ScheduledReceipt	Reference	
Source	This field denotes the source of the load. Valid values are: • ScheduledReceipt • PlannedOrder	String (Enum)	
StartUnit	The start unit that applies to the portion of the supply that created this load.	Integer	
WorkCenter	The reference to the work center on which this load has been placed. **Referenced table:** WorkCenter	Reference	Yes

LoadFLPPlanned table

This table provides full level pegging and supply available information for load on work centers from planned orders and scheduled receipts. This table is calculated from data in the Part table.

Table 8.84: LoadFLPPlanned (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **Operation** table. **Referenced table:** CRPOperation	Reference	
AvailableDate	The date when the entire supply is available.	Date	
DriverAvailable Date	The date the driver is available.	Date	
DriverDueDate	The DueDate of the driver for this demand.	Date	

Table 8.84: LoadFLPPlanned (Mfg) fields

Field name	Description	Data type	Owner
DriverPart	The part that is driving the demand that is consuming this supply. **Referenced table:** Part	Reference	
DriverQuantity	The total Quantity of the driver.	Quantity	
Idle	Standard hours spent in idle time for this operation for WhereConsumed.NeedQuantity. This time is included in Run time and represents time spent waiting for batches to arrive from the previous operation.	Quantity	
LaborIdle	Standard labor hours spent in idle time for this operation for WhereConsumed.NeedQuantity. This time is included in LaborRun time and represents time spent waiting for batches to arrive from the previous operation.	Quantity	
LaborRun	Standard hours associated with labor run for this operation for WhereConsumed.NeedQuantity.	Quantity	
LaborSetup	Standard hours associated with labor setup for this operation for WhereConsumed.NeedQuantity.	Quantity	
MachineIdle	Standard machine hours spent in idle time for this operation for WhereConsumed.NeedQuantity. This time is included in MachineRun time and represents time spent waiting for batches to arrive from the previous operation.	Quantity	
MachineRun	Standard hours associated with Machine Run Time for this operation for WhereConsumed.NeedQuantity.	Quantity	
MachineSetup	Standard hours associated with Machine Setup for this operation for WhereConsumed.NeedQuantity.	Quantity	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	Yes
Part	The part for the Supply that is generating the load. **Referenced table:** Part	Reference	
Run	The quantity of run load allocated to this driver.	Quantity	

Table 8.84: LoadFLPPlanned (Mfg) fields

Field name	Description	Data type	Owner
ScheduledReceipt	The schedule receipt creating this load (Null if IsPlanned= 'Y'). **Referenced table:** ScheduledReceipt	Reference	
Setup	The quantity of setup load allocated to this driver.	Quantity	
SupplyIsPlanned	Valid values are: • **Y**—If this load is from a planned order • **N**—If the load is not from a planned order	Boolean	
SupplyPlanned Order	The planned order creating this load. **Referenced table:** CTPPlannedOrder	Reference	
SupplyResched Date	ReschedDate of the supply order (DueDate if PlannedOrder).	Date	
SupplyStartDate	Copied from CalcStartDate (scheduled receipt) or StartDate (planned order).	Date	
WhereConsumed	The WhereConsumed record for the supply associated with this work center load. **Referenced table:** WhereConsumed	Reference	

LoadOperationNewOrderPlanned table

This table describes the load on a particular date from a planned order through an operation. Loads reported in this table are given in real hours (after applying utilization and efficiency).

Table 8.85: LoadOperationNewOrderPlanned (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **Operation** table. **Referenced table:** CRPOperation	Reference	
EndDate	The date on which this operation ends. **Note:** This either represents the end of wait time or the end of transit time (based on the setting in **OperationType.TransitTimeSequenceRule**).	Date	

Table 8.85: LoadOperationNewOrderPlanned (Mfg) fields

Field name	Description	Data type	Owner
EndOffset	The number of hours into the hours per day when the operation ends.	Quantity	
Idle	The number of standard hours spent at this work center on this date waiting for batches to arrive from the previous operations. This time is included in the Run time.	Quantity	
NextWorkCenter	The name of the next work center.	String	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	
Part	Reference to the part that created the planned order that created this load. **Referenced table:** Part	Reference	
PegPool	The pool for the supply that created the load. **Referenced table:** Pool	Reference	
PegModel	The model for the supply that created the load. **Referenced table:** Model	Reference	
PlannedOrder	Reference to the planned order that created this load. **Referenced table**: PlannedOrder	Reference	
PrevWorkCenter	The name of the previous work center.	String	
QueueDate	The beginning of queue time.	Date	
QueueOffset	The number of hours into the hours per day when the Queue phase starts. **Note:** If transit time is either applied after the operation or set to zero, then **QueueDate** and **QueueOffset** will be the same as **StartDate** and **StartOffset**.	Quantity	
Routing	A reference to the routing code that the operation reported on this record belongs to. **Referenced table:** Routing	Reference	Yes

Table 8.85: LoadOperationNewOrderPlanned (Mfg) fields

Field name	Description	Data type	Owner
Run	The number of standard hours of load placed on this work center on the Date generated during run time.	Quantity	
RunDate	The date the run phase starts.	Date	
RunOffset	The number of hours into the hours per day when the run starts.	Quantity	
Setup	The number of standard hours of load placed on this work center on the Date generated during setup time.	Quantity	
SetupDate	The date the setup phase starts.	Date	
SetupOffset	The number of hours into the hours per day when the setup starts.	Quantity	
StartDate	The date on which the overall operation started.	Date	
StartOffset	The number of hours into the available hours per day when the operation starts. **Note:** If transit time is either applied after the operation or set to zero, then **StartDate** and **StartOffset** will be the same as **QueueDate** and **QueueOffset**. Otherwise, pre-operation transit time represents the difference between these two sets of values.	Quantity	
StartUnit	The start unit that applies to the portion of the supply that created this load.	Integer	
WaitDate	The beginning of wait time.	Date	
WaitOffset	The number of hours into the hours per day when the wait starts.	Quantity	

LoadOperationReceiptsCurrent table

This table describes the load on a particular date from a scheduled receipt through an operation scheduled using the current due dates. Loads reported in this table are given in real hours (after applying utilization and efficiency).

Table 8.86: LoadOperationReceiptsCurrent (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **Operation** table. **Referenced table:** CRPOperation	Reference	
EndDate	The date on which this operation ends. **Note:** This either represents the end of wait time or the end of transit time (based on the setting in **OperationType.TransitTimeSequenceRule**).	Date	
EndOffset	The number of hours into the hours per day when the operation ends.	Quantity	
Idle	The number of standard hours spent at this work center on this date waiting for batches to arrive from the previous operation.	Quantity	
NextWorkCenter	The name of the next work center.	String	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	
PrevWorkCenter	The name of the previous work center.	String	
QueueDate	The beginning of queue time.	Date	
QueueOffset	The number of hours into the hours per day when the Queue starts. **Note:** If transit time is either applied after the operation or set to zero, then **QueueDate** and **QueueOffset** will be the same as **StartDate** and **StartOffset**.	Quantity	
Routing	A reference to the routing code that the operation reported on this record belongs to. **Referenced table:** Routing	Reference	Yes
Run	The number of standard hours of load placed on this work center on the date generated during run time.	Quantity	

Table 8.86: LoadOperationReceiptsCurrent (Mfg) fields

Field name	Description	Data type	Owner
RunDate	The date the run phase starts.	Date	
RunOffset	The number of hours into the hours per day when the run starts.	Quantity	
ScheduledReceipt	Reference to the order that created this load. **Referenced table:** ScheduledReceipt	Reference	
Setup	The number of standard hours of load placed on this work center on the date generated during setup time.	Quantity	
SetupDate	The date the setup phase starts.	Date	
SetupOffset	The number of hours into the hours per day when the setup starts.	Quantity	
StartDate	The date on which the overall operation started.	Date	
StartOffset	The number of hours into the available hours per day when the operation starts. **Note:** If transit time is either applied after the operation or set to zero, then **StartDate** and **StartOffset** will be the same as **QueueDate** and **QueueOffset**. Otherwise, pre-operation transit time represents the difference between these two sets of values.	Quantity	
StartUnit	The start unit that applies to the portion of the supply that created this load.	Integer	
WaitDate	The beginning of wait time.	Date	
WaitOffset	The number of hours into the hours per day when the wait starts.	Quantity	

LoadOperationReceiptsPlanned table

This table describes the load on a work center from a scheduled receipt through an operation scheduled using the rescheduled dates. Loads reported in this table are given in real hours (after applying utilization and efficiency).

Table 8.87: LoadOperationReceiptsPlanned (Mfg) fields

Field name	Description	Data type	Owner
CRPOperation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the Operation table. **Referenced table:** CRPOperation	Reference	
EndDate	The date on which this operation ends. **Note:** This either represents the end of wait time or the end of transit time (based on the setting in **OperationType.TransitTimeSequenceRule**).	Date	
EndOffset	The number of hours into the hours per day when the operation ends.	Quantity	
Idle	The number of standard hours spent at this work center on this date waiting for batches to arrive from the previous operation.	Quantity	
NextWorkCenter	The name of the next work center.	String	
Operation	A reference to the work center operation that generated the load reported on this record. Null if the operation generating the load is defined in the **CRPOperation** table. **Referenced table:** Operation	Reference	
PrevWorkCenter	The name of the previous work center.	String	
QueueDate	The beginning of queue time.	Date	
QueueOffset	The number of hours into the hours per day when the Queue phase starts. **Note:** If transit time is either applied after the operation or set to zero, then **QueueDate** and **QueueOffset** will be the same as **StartDate** and **StartOffset**.	Quantity	
Routing	A reference to the routing code that the operation reported on this record belongs to. **Referenced table:** Routing	Reference	Yes
Run	The number of standard hours of load placed on this work center on the Date generated during run time.	Quantity	

Table 8.87: LoadOperationReceiptsPlanned (Mfg) fields

Field name	Description	Data type	Owner
RunDate	The date the run phase starts.	Date	
RunOffset	The number of hours into the hours per day when the run starts.	Quantity	
ScheduledReceipt	Reference to the order that created this load. **Referenced table:** ScheduledReceipt	Reference	
Setup	The number of standard hours of load placed on this work center on the Date generated during setup time.	Quantity	
SetupDate	The date the setup phase starts.	Date	
SetupOffset	The number of hours into the hours per day when the setup starts.	Quantity	
StartDate	The date on which the overall operation started.	Date	
StartOffset	The number of hours into the available hours per day when the overall operation started. **Note:** If transit time is either applied after the operation or set to zero, then **StartDate** and **StartOffset** will be the same as **QueueDate** and **QueueOffset**. Otherwise, pre-operation transit time represents the difference between these two sets of values.	Quantity	
StartUnit	The start unit that applies to the portion of the supply that created this load.	Integer	
WaitDate	The beginning of wait time.	Date	
WaitOffset	The number of hours into the hours per day when the wait starts.	Quantity	

LoadWeeklyCurrent table

The LoadWeeklyCurrent table describes all load and capacity on a work center from scheduled receipts through operations and scheduled operations using the current due dates. Records are shown for each week that has load.

Table 8.88: LoadWeeklyCurrent (Mfg) fields

Field name	Description	Data type	Owner
IsInsideCapacity ReportLimit	A flag that indicates whether the date that is represented by this record is inside the horizon that is established by the CapacityReportLimit for the work center.	Boolean	
LaborCapacity	Total labor capacity at this work center for this week.	Quantity	
LaborIdle	The number of standard labor hours spent at this work center in this week waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center in this week generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center during the week generated during setup time.	Quantity	
MachineCapacity	Total machine capacity at this work center for this week.	Quantity	
MachineIdle	The number of standard machine hours spent at this work center in this week waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	
MachineRun	The number of standard machine hours of load placed on this work center in this week generated during run time.	Quantity	
MachineSetup	The number of standard machine hours of load placed on this work center in this week generated during setup time.	Quantity	
MaximumLabor Capacity	The maximum labor capacity for this work center, in this week, as described by the appropriate capacity record.	Quantity	
MaximumMachine Capacity	The maximum machine capacity for this work center in this week as described by the appropriate capacity record.	Quantity	
WeekStarting	The beginning of the week for which the load is being reported.	Date	
WorkCenter	Reference to the WorkCenter on which this load has been placed. **Referenced table:** WorkCenter	Reference	Yes

LoadWeeklyPlanned table

The LoadWeeklyPlanned table describes all load on a work center in weekly buckets from scheduled receipts and planned orders through operations and scheduled operations using the rescheduled scheduled receipt dates. This load will match the allocation dates for the materials being used in the operations as planned by RapidResponse.

Table 8.89: LoadWeeklyPlanned (Mfg) fields

Field name	Description	Data type	Owner
IsInsideCapacity ReportLimit	A flag that indicates whether the date that is represented by this record is inside the horizon that is established by the CapacityReportLimit for the work center.	Boolean	
LaborCapacity	Total labor capacity at this work center for this week.	Quantity	
LaborIdle	The number of standard labor hours spent at this work center in this week waiting for batches to arrive from the previous operation. This time is included in the LaborRun time.	Quantity	
LaborRun	The number of standard labor hours of load placed on this work center in the week generated during run time.	Quantity	
LaborSetup	The number of standard labor hours of load placed on this work center in the week generated during setup time.	Quantity	
Machine Capacity	Total machine capacity at this work center for this week.	Quantity	
MachineIdle	The number of standard machine hours spent at this work center in this week waiting for batches to arrive from the previous operation. This time is included in the MachineRun time.	Quantity	
MachineRun	The number of standard machine hours of load placed on this work center in the week generated during run time.	Quantity	
MaximumLabor Capacity	The maximum labor capacity for this work center in this week as described by the appropriate capacity record.	Quantity	
Maximum Machine Capacity	The maximum machine capacity for this work center in this week as described by the appropriate capacity record.	Quantity	
Setup	The number of standard hours of load placed on this work center on the date generated during setup time.	Quantity	

Table 8.89: LoadWeeklyPlanned (Mfg) fields

Field name	Description	Data type	Owner
WeekStarting	The beginning of the week for which the load is being reported.	Date	
WorkCenter	Reference to the work center on which the reported load has been placed. **Referenced table:** WorkCenter	Reference	Yes

MaximumInventoryResult table

The **MaximumInventoryResult** table reports information about the parts that are configured to use the **DistributionPlanningRule.ProcessingRule** and also reports the quantity of maximum inventory that is set for the part. However, if the **TimePhasedMaximumInventory.Quantity** is set below the effective safety stock level for the part, the table reports the effective safety stock level as the maximum inventory quantity and indicates safety stock as the source of the maximum inventory.

The **MaximumInventoryResult** table was added in Version 2016.2. For more information, see "Distribution planning" on page 1717.

Table 8.90: MaximumInventoryResult (Mfg) fields

Field name	Description	Data type	Owner
Part	The part for which **DistributionPlanningRule** is enabled. **Referenced table**: Part	Reference	Yes
Pool	The pool that applies to this part, if applicable. **Referenced table**: Pool	Reference	
Date	Date when the maximum inventory value for the part becomes effective.	Date	
Quantity	The amount of the maximum inventory that is set for the part. If the **DistributionPlanningRule** is enabled but the **TimePhasedMaximumInventory** quantity is set below effective safety stock level for the part, then the effective safety stock level is used as the maximum inventory quantity. This value is reported in the unit of measure used by the part.	Quantity	

Table 8.90: MaximumInventoryResult (Mfg) fields

Field name	Description	Data type	Owner
Source	Indicates the source of the maximum inventory quantity on this date. Valid values are: • **TimePhasedMaximumInventory**—indicates that the maximum inventory quantity is **TimePhasedMaximumInventory.Quantity**. • **SafetyStock**—indicates that the maximum inventory quantity is the effective safety stock level for the part because **TimePhasedMaximumInventory.Quantity** that was set is below the effective safety stock level for the part.	Enum	

MEIODriverPart table

The **MEIODriverPart** table reports all driver parts for a particular part that has been brought into a multi-echelon family.

Driver parts can include customer-facing end items that have been configured for multi-echelon safety stock calculations (that is, parts whose **MultiEchelonSafetyStockRule** is set to "Use" or "NoInventory" and that are referenced by a **SafetyStockItem** record with a **ProcessingRule** of "MultiEchelon"), as well as other end-items brought into a multi-echelon family that drive demand to the lower-level components in the family. Note also that this table does not report details of any stages injected for modeling purposes (such as for modeling multiple sources of purchased supply). Only the actual parts themselves are reported in this table.

This table is useful for reporting the end-items that drive dependent demand down to the family members and hence which might result in safety stock recommendations being placed on the part. If knowledge of the parent assembly directly above the part into the family is required, the **MEIOParentPart** table can be used instead.

Table 8.91: MEIODriverPart (Mfg) fields

Field name	Description	Data type	Owner
DriverPart	A reference to the end item assembly that brought the part into a multi-echelon family. **Referenced table:** Part	Reference	
Part	A reference to the component that was brought into a multi-echelon family by the driver. A given part will have one record for each driver part it links to within a multi-echelon family. For example, the part might be found within the BillOfMaterial structure for multiple assemblies that have been configured as multi-echelon safety stock items. **Referenced table:** Part **Note:** For each valid driver part, there is also one record added to this table that references that part as both DriverPart and Part.	Reference	Yes

MEIOFamilyResult table

The **MEIOFamilyResult** table reports the number of iterations that multi-echelon safety stock optimization calculations made through each multi-echelon family, and time associated with those iterations, to achieve the current results. Note that configuration options are available to limit the maximum possible iterations performed on families in a given scenario. This can be set on a per scenario basis using the **Analytics Configuration** table. The Inventory Planning workbook can be used to add or modify values in that table.

Table 8.92: MEIOFamilyResult (Mfg) fields

Field name	Description	Data type	Owner
Family	The head of the multi-echelon family to which the results in this table apply. This always returns a reference to a part that has been configured as a multi-echelon safety stock item (has a **SafetyStockItem** record where **ProcessingRule** is set to "MultiEchelon"). If a family contains more than one multi-echelon safety stock item, the one that sorts first by alphabetical order is reported as the head of the family. **Referenced table:** Part	Reference	Yes
InventoryCost	The cost of holding safety stock and early arrival stock over MEIOStageTimePhasedResult report dates. If **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore," "DaysOfSupplyForward," or "DaysOfSupplyBackward," this is the average inventory cost over all report dates. **Note:** This field was added in RapidResponse 2016.2.	Quantity	
Iterations Performed	Total number of iterations made through this multi-echelon family to produce the current set of recommendations.	Integer	
RunTime	Total number of seconds spent iterating through this multi-echelon family to produce the current set of recommendations. **Note:** This field was modified in Version 2016.2 Service Update 6.	Quantity	
SafetyStock BoundsViolation	The sum of the absolute values of all safety stock bounds violations across all stages and report dates. **Note:** This field was added in RapidResponse 2016.2.	Quantity	

MEIOLeadTimeOutlierResult table

Similarly to the **SafetyStockItemOutlierResult** table, the **MEIOLeadTimeOutlierResult** table reports details related to outliers in historical data for items configured for multi-echelon safety stock

calculations using variable lead time parameters (assuming the part belongs to a multi-echelon family). For such safety stock items, there is one record created for each period of historical data used in generating the recommended safety stock.

Each record in the **MEIOLeadTimeOutlierResult** table indicates whether a given historical data point represents an outlier in the data set, and contains other useful details such as the quantity of the demand in the period by which it should be adjusted for outliers (if any).

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Table 8.93: MEIOLeadTimeOutlierResult fields

Field name	Description	Data type	Owner
Date	The historical date to which the outlier data reported on this record pertains. Each data reported here belongs to the item's **Type.IntervalsCalendar**. For example, dates might mark the beginning of a weekly or monthly period.	Date	
ErrorComponent	If seasonal decomposition is used when processing outliers, this reports the error remaining after removing seasonal and trend components. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
ExistingOutlier Adjustment	The quantity for the outlier, if any, that has already been specified for the item and period in the **SafetyStockItemOutlier** table.	Quantity	
IsOutlier	Indicates whether the data in this period represents an outlier within the item's historical data set. Valid values are: • **N**—the data point is not an outlier. • **Y**—the data point is an outlier.	Boolean	
LowerThreshold	Indicates a value below which a data point for the item is considered an outlier.	Quantity	
MEIOLeadTime	The **MEIOLeadTime** to which the outlier adjustment applies. **Referenced table**: MEIOLeadTime	Reference	Yes
Outlier Adjustment	The amount that the original Quantity on the record should be adjusted by to account for outliers. This recommended adjustment is then reflected in the **SuggestedQuantity** field. If the record does not represent an outlier, than a value of "0" (zero) is returned.	Quantity	

Table 8.93: MEIOLeadTimeOutlierResult fields

Field name	Description	Data type	Owner
Quantity	Total demand quantity for the forecast item and demand category in this period. This includes historical actuals, causal factors, and any existing outlier quantity.	Quantity	
Seasonal Component	If seasonal decomposition is used when processing outliers, this reports the seasonal component. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
Suggested Quantity	Demand quantity for the item in this period after adjustment for any detected outliers, as represented by the sum of the **Quantity** and **OutlierAdjustment**. If the record does not represent an outlier, then this returns the same value as the **Quantity** field.	Quantity	
Trend Component	If seasonal decomposition is used when processing outliers, this reports the trend component. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
UpperThreshold	Indicates a value above which a data point for the item is considered an outlier.	Quantity	

MEIOLeadTimeOutlierResultSummary table

Similarly to the **SafetyStockItemOutlierResultSummary** table, the **MEIOLeadTimeOutlierResultSummary** table contains summary information related to outlier detection in historical data for items configured for multi-echelon safety stock calculations using variable lead time parameters (assuming the part belongs to a multi-echelon family). The type of information reported in this table is generally either a characteristic of the data used in outlier detection, or a message generated when calculating outliers.

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Table 8.94: MEIOLeadTimeOutlierSummary fields

Field name	Description	Data type	Owner
Class	Indicates the type of data contained on this record. Valid values are: • **DataCharacteristic**—summary details about the historical data used when calculating outliers for the item. For example, the number of historical data points collected for the item. • **Message**—an error message generated during the process of calculating outliers. For example, if using methods that rely on the optionally configure R programming language, an error might be reported if a connection cannot be made to the R server.	String (Enum)	
MEIOLeadTime	The **MEIOLeadTime** to which the outlier adjustment applies. **Referenced table**: MEIOLeadTime	Reference	Yes
ValueDate	Used to report summary date values (populated when **ValueType** is "Date"). For example, the mean date is reported in this field.	Date	
ValueName	The specific piece of information being reported on this record. The actual value associated with this summary data is reported in one of **ValueDate**, **ValueQuantity**, or **ValueText** (data type dependent). Valid values are: • **Count**—the number of historical data intervals over which data was collected for the item. • **Error**—an error message indicating an issue such as an invalid or missing parameter value, or an error connecting to the R Server. • **Mean**—average quantity calculated from the historical demand periods considered. • **MeanDate**—the approximate midpoint of the dates used in the calculations. • **PercentageZeroActuals**—percentage of historical data periods with a 0 (zero) quantity. • **Variance**—measures the overall variance of the historical data quantities from the mean.	String (Enum)	
ValueQuantity	Used to report summary quantity values on the data used to detect outliers (populated when **ValueType** is "Quantity"). For example, mean and variance data values are reported in this field. **Note:** These quantity values are calculated on the quantities as they exist before any adjustment for outliers.	Quantity	

Table 8.94: MEIOLeadTimeOutlierSummary fields

Field name	Description	Data type	Owner
ValueText	Used to report summary string values (populated when **ValueType** is "Text"). For example, any error messages are reported in this field.	String	
ValueType	Indicates the type of data reported for the value on this record, and hence which Value field is populated. Valid values are: • **Date**—the **ValueDate** field is populated. • **Text**—the **ValueText** field is populated. • **Quantity**—the **ValueQuantity** field is populated.	String (Enum)	

MEIOHistoricalSupply table

The **MEIOHistoricalSupply** table reports details of historical supplies on parts for which supply lead time variability is assumed. Each record in this table is based on a record in the **HistoricalSupplyActual** input table, and contains data that can be used to determine lead-time parameters for use in multi-echelon safety stock calculations.

For a given record in the **HistoricalSupplyActual** table, a corresponding record is added to this table if the following conditions are satisfied:

- The **HistoricalSupplyActual** record has a valid **Source** reference.
- A record exists in the **MEIOLeadTime** table where the **PartSource** reference resolves to the same part and source combination as is referenced through the **HistoricalSupplyActual** record.
- The **MEIOLeadTime** record references an **MEIOLeadTimeType** record where the **Category** reference points to the same historical supply category as is referenced through **Header.Category** on the **HistoricalSupplyActual** record.

Table 8.95: MEIOHistoricalSupply (Mfg) fields

Field name	Description	Data type	Owner
Date	The actual date associated with this historical supply. For example, this might represent the date parts shipped from the dock.	Date	
IsLeadTime Outlier	Indicates if the original lead time value was considered an outlier. Valid values are: • **False**—the data point is not a lead time outlier. • **True**—the data point is a lead time outlier. If the original lead time was considered an outlier by less than 0.5 days, once the **LeadTime** is rounded to the nearest integer, then **LeadTime** = **OriginalLeadTime.** In this case, this field will report a value of "False". **Note:** This field was added in Version 2016.2, Service Update 5.	Boolean	
Item	A reference to the **MEIOLeadTime** record identifying the part and source this historical supply pertains to. Other details, such as how variable lead time is calculated for the item, can also be accessed through this reference. **Referenced table:** MEIOLeadTime	Reference	Yes
LeadTime	The lead time value after applying outlier adjustments from **MEIOLeadTimeOutlier**. Note that outlier adjustments can yield a fractional lead time. Values in this field are rounded to the nearest integer. **Note:** This field was modified in Version 2016.2, Service Update 5.	Integer	
OrderDate	The date associated with the ordering of this supply. **Note:** If the **OrderDate** field is not populated on the corresponding **HistoricalSupplyActual** record, this field is calculated using the **Date** and **LeadTime** values provided on that record. Otherwise, if either **Date** or **LeadTime** are not populated, the **HistoricalSupplyActual** record is ignored.	Date	
OriginalLead Time	The original lead time value for this date before applying any outlier adjustments. **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	
Quantity	The number of units in this actual shipment.	Quantity	

Note: Records in this table are only used in multi-echelon safety stock calculations if the part referenced through the **MEIOLeadTime** table (**Item** field) belongs to a multi-echelon family, and the **MEIOLeadTime** record uses both an **MEIOLeadTimeType.ProbabilityDistribution** setting of "Normal" and an **MEIOLeadTimeType.StandardDeviationRule** setting of "Calculate".

MEIOParentPart table

The **MEIOParentPart** table reports the immediate parent part for each part that has been brought into a multi-echelon family for use in determining optimal safety stock levels. Parent parts are determined based on actual bill of material structures and transfer part source relationships. Note also that this table does not report details of any stages injected for modeling purposes (such as to represent multiple purchase sources for a part). Only the actual parts themselves are reported in this table.

This table can be used to help understand the relationship between parts in multi-echelon families. If knowledge of the end item that was ultimately responsible for bringing a part into the family is required, the **MEIODriverPart** table can be used instead.

Table 8.96: MEIOParentPart (Mfg) fields

Field name	Description	Data type	Owner
Part	A reference to a component that was brought into a multi-echelon family. **Referenced table:** Part	Reference	Yes
ParentPart	A reference to the parent assembly that the component part belongs to. A given component will have one record for each parent assembly it belongs to within a multi-echelon family. For example, the part might be used by multiple assemblies within the multi-echelon family. **Referenced table:** Part	Reference	

MEIOStage table

The **MEIOStage** table identifies each stage in the families that are constructed for use in multi-echelon safety stock calculations. This includes real stages that can potentially hold inventory (for example, a part at a particular location), as well as artificial stages injected for modeling purposes (for example, stages added to support multiple potential sources for a buy part). Additional details pertaining to each stage, such as processing (lead) times and unit holding costs are also reported in this table.

Table 8.97: MEIOStage (Mfg) fields

Field name	Description	Data type	Owner
Cumulative LeadTime	Total time needed to process this stage and all of its dependent stages in the family. Expressed in terms of the Gregorian (everyday) calendar.	Integer	
Echelon	Represents the level in the multi-echelon family where the stage is located. For example, a customer-facing end item will have a value of 0, the stages immediately below that a value of 1, the next level of stages a value of 2, and so on.	Integer	
LeadTime	Time needed to process this stage (in days). This field relies on the part source's lead time but is reported in terms of the Everyday calendar using the following expression: *PartSource.LeadTimeDate - RunDate Everyday* Note that if a stage represents a buy part with multiple sources of the same priority, then the corresponding record with **Real** = "Y" will always have a value of 0 in this field (lead time is calculated on the stages injected to model the different sources). Similarly, if a stage represents a part with multiple transfer sources, or a transfer part that provides different transit times to two or more other parts, then stages are injected to model these situation and lead time calculated on those injected stages only.	Integer	
Part	A reference to the part represented by this stage. **Referenced table:** Part	Reference	Yes
PartSource	A reference to the part source supplying the part. This value depends on the type of part and number of potential part sources as follows: • For "make" parts—the primary part source is always reported. • For "buy" parts, if there is only one source then it is reported here. Otherwise, if there are multiple sources with the same priority as the primary part source, then additional records are injected and each references the applicable part source (and the main record where **Real** equals "Y" will be Null). • For "transfer" parts, if there is only source then this will be Null. Otherwise, if there are multiple sources with the same priority as the primary part source, then additional records are injected and each references the applicable part source (and the main record where **Real** equals "Y" will still be Null). **Referenced table:** PartSource	Reference	

Table 8.97: MEIOStage (Mfg) fields

Field name	Description	Data type	Owner
Real	Indicates if this record represents a real stage (part) in the family or an artificial stage injected by RapidResponse for modeling the family. For example, if a "buy" part has two sources of the same priority, there will be one record for that part having **Real** set to "Y" (this stage can hold inventory). Additionally, two records will be injected to model each of its part sources and these will each have **Real** set to "N". Valid values: • **Y**—this is a real stage (part) and eligible to hold inventory. • **N**—this is an artificial stage injected into the family. Inventory can be held at the artificial stage only if the **AnalyticsConfiguration.MEIOAllowSafetyStockOnArtificialStages** is set to "Y" and there are no records in and there are no records in the **SafetyStockTimePhasedBounds** table.	Boolean	
SafetyStock Item	The safety stock item, if any, this stage refers to. This field is Null unless the part is a finished good that has been configured for multi-echelon safety stock calculations (has a **SafetyStockItem** record with **ProcessingRule** set to "MultiEchelon"). **Referenced table:** SafetyStockItem	Reference	

Table 8.97: MEIOStage (Mfg) fields

Field name	Description	Data type	Owner
Type	Identifies the type of stage reported on this record. Valid values are: • **Buy**—a purchased part located at an internal stage in the family (**SafetyStockItem** reference will be Null). Note that if a Buy part has multiple sources of the same priority, then artificial stages are injected to model each of those sources separately and their **Real** fields are set to "N". Inventory can be held at the artificial stage only if the **AnalyticsConfiguration.MEIOAllowSafetyStockOnArtificialStages** is set to "Y" and there are no records in and there are no records in the **SafetyStockTimePhasedBounds** table. • **Customer**—a customer-facing end item where the part is referenced by a **SafetyStockItem** record with a **ProcessingRule** value of "MultiEchelon". Note that for this Type, if **Part.PrimaryPartSource** is not "Make", then the associated **PartSource** reference is Null and **LeadTime** reported as 0. • **InternalCustomer**—a stage injected to represent a customer-facing end item located at an internal stage in the family (where the part is referenced by a **SafetyStockItem** record with a **ProcessingRule** value of "MultiEchelon"). For example, a safety stock item might have demand from external customers as well as demand from other internal parts (such as, if configured as a transfer part). • **Make**—a manufactured part located at an internal stage in the family (**SafetyStockItem** reference will be Null). • **Transfer**—a part that is sourced from another site and located at an internal stage in the family (**SafetyStockItem** reference will be Null). Note that if a Transfer part has multiple sources, or if a given source supplies multiple parts with different transit times, then artificial stages are injected to model each of those relationships separately and their **Real** fields are set to "N". Inventory can be held at the artificial stage only if the **AnalyticsConfiguration.MEIOAllowSafetyStockOnArtificialStages** is set to "Y" and there are no records in and there are no records in the **SafetyStockTimePhasedBounds** table.	String (Enum)	
UnitHolding Cost	The calculated cost of holding one unit of inventory at this stage. Inventory holding costs might later be used in determining the best or optimal placement of safety stock within the family (based on minimizing the total cost of holding inventory in respect of the item's service level). **Note:** This field is calculated based on the Part record's specified **InventoryHoldingRate** value and **MultiEchelonSafetyStockRule.CostRule** setting. For more information, see "Calculate inventory holding costs for MEIO" on page 1996.	Money	

MEIOStageHistoricalDemand table

This table is only relevant for users that want to enable forecast error pooling. It contains records only for MEIO stages that are part of a family for which forecast error pooling is enabled.

This new table allows you to see the actual demand that is used in the calculations along with the forecast captured in the **MEIOStageHistoricalForecast** table.

This table was added in version 2016.2. For more information about forecast error pooling, see "Use forecast error pooling for safety stock (optional)" on page 1963.

Table 8.98: MEIOStageHistoricalDemand (Mfg) fields

Field name	Description	Data type	Owner
Date	The date that this bucketed quantity applies to.	Date	
Quantity	The quantity of demand.	Quantity	
Part	The part name. **Referenced table:** Part	Reference	Yes
Stage	The MEIO stage. **Referenced table:** Part	Reference	

MEIOStageHistoricalForecast table

This table is only relevant for users that want to enable forecast error pooling. It contains records only for MEIO stages that are part of a family for which forecast error pooling is enabled.

This new table allows you to see the actual forecast that is used in the calculations along with the demand captured in the **MEIOStageHistoricalDemand** table.

This table was added in version 2016.2. For more information about forecast error pooling, see "Use forecast error pooling for safety stock (optional)" on page 1963.

Table 8.99: MEIOStageHistoricalForecast (Mfg) fields

Field name	Description	Data type	Owner
Date	The date that this bucketed quantity applies to.	Date	
Quantity	The quantity of demand.	Quantity	
Part	The part name. **Referenced table:** Part	Reference	Yes
Stage	The MEIO stage. **Referenced table:** MEIOStage	Reference	

MEIOStageLink table

The **MEIOStageLink** table contains one record for each link that exists between stages in a multi-echelon family. For example, links are reported from each stage directly under the multi-echelon safety stock item to the stage representing the safety stock item itself. In cases where artificial stages are injected by RapidResponse, links are reported between those artificial stages and the appropriate real stages. This table can be useful for understanding the structure of multi-echelon families.

Table 8.100: MEIOStageLink (Mfg) fields

Field name	Description	Data type	Owner
FromStage	The stage where the link is made from. **Referenced table:** MEIOStage	Reference	
Part	A reference to the part reported in the **Stage** reference. **Referenced table:** Part	Reference	Yes
QuantityPer	The amount of the item in the **FromStage** reference required to build one unit of the item in the **Stage** reference	Quantity	
Stage	The stage that the link is made to. **Referenced table:** MEIOStage	Reference	

MEIOStageResult table

The **MEIOStageResult** table reports the optimal service times calculated for each stage (part) in a multi-echelon family. Service time represents the maximum time in which a given stage guarantees it will meet the demands placed on it. The service times reported in this table are those which the multi-echelon analytic determines will minimize the total cost of holding inventory across the family. Calculated parameters used in determining these service times, such as service level and average demand, are also reported for each stage.

For more information about the calculations reported by this table, see "Determine demand and supply parameters for MEIO" on page 1999.

Table 8.101: MEIOStageResult (Mfg) fields

Field name	Description	Data type	Owner
AverageDemand	Average historical demand calculated for the part represented by this stage. For customer facing end-items, this is based on demands placed directly on the part. For all lower level component parts, this is based on demands received from higher level stages in the family. Expressed in terms of the average demand per units of the Gregorian (everyday) calendar.	Quantity	
IncomingService Time	Maximum service time calculated across all stages/parts that are directly below this one in the multi-echelon family. This is the maximum number of days within which this stage will have its immediate demands satisfied (for example, from its component supplies). Expressed in terms of the Gregorian (everyday) calendar.	Integer	
OutgoingService Time	Recommended service time for this stage in the family. This is the maximum number of days within which the part guarantees it can satisfy the demands placed on it. For multi-echelon safety stock items, this is set to an input value specified on the safety stock item (if the input service time provided is greater than the cumulative lead time for the part, that cumulative lead time value is used instead). For all parts below those ends items in a multi-echelon family, this is calculated with the aim of minimizing total inventory holding costs. Expressed in terms of the Gregorian (everyday) calendar.	Integer	
Part	A reference to the part to which these stage results apply. **Referenced table:** Part	Reference	Yes
ServiceLevel	Service level applicable to this stage. If this stage refers to an end item configured for multi-echelon calculations, this returns the service level provided as input on the **SafetyStockItem** record. Otherwise, if this stage refers to a lower level part then the service level is set to that provided on the item that brought it into the family. However, if a part belongs to multiple end items in the family, then this is a weighted combination of those item's service levels.	Quantity	
Stage	The stage that these results were calculated for. **Referenced table:** MEIOStage	Reference	

Table 8.101: MEIOStageResult (Mfg) fields

Field name	Description	Data type	Owner
StandardDeviation Demand	Standard deviation of historical demand calculated for this stage (or standard deviation of historical forecast error if **SafetyStockItemType. StandardDeviationDemandRule** is set to "ForecastError"). For customer facing end-items, this is based on external demands placed directly on the part. For all lower level component parts, this is based on demands propagated from higher level stages in the family.	Quantity	

MEIOStageTimePhasedResult table

The **MEIOStageTimePhasedResult** table reports the recommended safety stock levels, if any, for each stage (part) in a multi-echelon family. The safety stock levels reported in this table are those which multi-echelon safety stock calculations determine will minimize the total cost of holding inventory across the family while ensuring the service level specified on the multi-echelon safety stock items can be met.

For more information about the calculations reported by this table, see "Recommend safety stock" on page 2005.

Table 8.102: MEIOStageTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
AverageDemand	Average historical demand calculated for the part represented by this stage. For customer facing end-items, this is based on demands placed directly on the part. For all lower level component parts, this is based on demands received from higher level stages in the family. If **SafetyStockItemType.TimePhased ProcessingRule** is set to "Use," this is the average demand over the part's net lead time for the period. Otherwise, it is expressed in terms of the average demand per unit of the Gregorian (everyday) calendar. **Note:** This field was added in Version 2016.2.	Quantity	
Date	The date on which the recommended safety stock level should be considered effective. If the associated multi-echelon safety stock item has been configured for time-phased calculations, then there is a record generated on the RunDate and on each subsequent date for which a change in the part's safety stock level is recommended. Otherwise, if the associated multi-echelon safety stock item is not configured for time-phased calculations, there is just a single record for a given part/stage (safety stock level is will be stationary).	Date	

Table 8.102: MEIOStageTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
EarlyArrival Stock	Amount of early arrival stock for this part. Early arrival stock refers to replenishment orders that arrive at a location before the associated customer order has shipped. Early arrival stock can occur for multi-echelon parts with lead time variability when a stage's outbound service time exceeds its inbound service time. Multi-echelon optimization calculations then attempt to minimize the total holding cost of both safety stock and early arrival stock.	Quantity	
Part	A reference to the part for which the safety stock recommendation on this record was made. **Referenced table:** Part	Reference	Yes
ServiceLevel	For a customer-facing item, this is the service level defined in the **SafetyStockItem.ServiceLevel** field. For lower level parts brought into a multi-echelon family under one or more customer-facing items, this value is calculated based on the service level and standard deviation of demand over net lead time for customer-facing items, and the quantity of the lower level part needed per unit of the customer-facing items. **Note:** This field was added in Version 2016.2.	Quantity	
SafetyStock	Recommended safety stock level for this part as of the date indicated.	Quantity	
Stage	The stage that the safety stock recommendation was calculated for. **Referenced table:** MEIOStage	Reference	
Standard Deviation Demand	Calculated standard deviation of demand for the part. **Note:** This field was added in Version 2016.2.	Quantity	

MLSActivity table

The **MLSActivity** table reports details of late components that are needed under an end-item driver assembly earlier than multi-level search logic determined they can be planned for availability. It relates late components supplies to end-item driver demands that were left wholly or partially unsatisfied because their planned availability, as determined by multi-level search logic, fell outside of a defined tolerance.

Each record in the **MLSActivity** table also identifies a specific supply which could not be planned in time to satisfy the demand need date, allowing for identification of the reasons why a driver demand was ultimately left unsatisfied. For example, sufficient constraint might not have been available to allow

completion of a component planned order by its need date. In other cases, end-item demands might be left unsatisfied when component supply is late due to planning time fence or part source effectivity.

This table was added in Version 2016.2, Service Update 5.

Note: Records are only ever generated for late components under end-item demands that have **DemandType.PlanningRule** set to "Partial" and where the driver part the demand is for has it's **MultiLevelSearchRule.ProcessingRule** set to "Use" and it's **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Additionally, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply segment).

Table 8.103: MLSActivity (Mfg) fields

Field name	Description	Data type	Owner
AdjustedNeedDate	The latest date by which the supply reported on this record would be needed to ensure it won't cause the end-item driver demand to be left unsatisfied.	Quantity	
AvailableStartDate	The start date for the supply. Calculated from the earliest allocated constraint (or if there is no associated constraint, calculated from **TrialAvailableDate**).	Date	
BelowPTFViolation	Indicates whether the trial supply reported on this record is generated from BOM explosion under a supply that couldn't be planned due to a planning time fence (PTF) violation. Valid values are: • **N**—the supply on this record was generated under a PTF violation. • **Y**— the supply on this record was generated under a PTF violaiton. **Note:** If the supply itself has a planning time fence violation, then it will report a value of "N" in ths field, and supplies on subsequent levels below it in the path will report a value of "Y".	Boolean	
BillOfMaterial	The BillOfMaterial record, if any, that led to the part whose **Referenced table:** BillOfMaterial	Reference	
ConstraintDue Date	The earliest date that a planned order from the part source providing this supply could be created (in consideration of planning time fence, constraint, and so on).	Date	

Table 8.103: MLSActivity (Mfg) fields

Field name	Description	Data type	Owner
DaysLate	The number of days late to satisfy the driving requirement that multi-level search logic determines the supply will be. This is calculated as the difference between **TrialAvailableDate** and **NeedDate**.	Integer	
DriverForecast	A reference to the independent demand that was not fully satisfied, and which is driving the supply requirement reported on this record. Null if **DriverType** is "IndependentDemand". **Referenced table:** Forecast	Reference	
DriverIndependentDemand	A reference to the independent demand that was not fully satisfied, and which is driving the supply requirement reported on this record. Null if **DriverType** is "Forecast". **Referenced table:** IndependentDemand	Reference	
DriverPart	The end-item part that is driving the demand requirement reported on this record. **Referenced table:** Part	Reference	Yes
DriverQuantity	The quantity of the end-item demand that is driving the requirement for the part reported on this record.	Quantity	
DriverType	Indicates the type of end-item demand that is driving the supply requirement reported on this record. Valid values are: • **Forecast**—Driver demand is a forecast. • **IndependentDemand**—Driver is an actual customer order.	String (Enum)	
EventDate	The date of the driver demand (offset by safety lead time, if applicable).	Date	
HasConstraintShortageByNeedDate	Indicates if the supply requires more constraint by its need date than is available. Valid values are: • **N**—the supply reported on this record does not have a constraint shortage. • **Y**—the supply reported on this record does have a constraint shortage. At least one entry in the **TrialConstraintShortage** table will point to this record.	Boolean	

Table 8.103: MLSActivity (Mfg) fields

Field name	Description	Data type	Owner
Index	A generated index value which can be used for sorting records into an indented supply tree. This value reset at 0 for each driver part reported in this table, and then increments by one for each additional record under that driver part.	Integer	
IsLeaf	Indicates if the supply has any late components below it. Valid values are: • **N**—the supply reported on this record does not have any late components below it (the supply's lateness is attributable only to the part itself). • **Y**—the supply reported on this record has one or more late components below it.	Boolean	
Level	The level in the supply structure where the supply reported on this record is found. This value can be used in building an indented supply tree. For example, a supply satisfying an end-item demand would have a value of "0", the next level of supply below it would have a value of "1", and so on.	Integer	
MaterialStartDate	The date when all components required for this supply will be available.	Date	
Model	The model, if any, that applies to this supply. **Referenced table:** Model	Reference	
NeedDate	The date by which the supply reported on this record would be needed to ensure the end-item driver demand can be satisfied.	Date	
NeedQuantity	The amount required from this supply to ultimately satisfy the driver demand.	Quantity	
OrderPriority	The priority assigned to the supply reported on this record. Based on configuration, typically set to the priority on the demand the supply is created to satisfy or else the part's default priority. **Referenced table:** OrderPriority	Reference	

Table 8.103: MLSActivity (Mfg) fields

Field name	Description	Data type	Owner
ParentPart	The immediate parent part that passed down the requirement for the supply reported on this record. Null if the record pertains to supply of the end item itself whose demand was left unsatisfied. **Referenced table:** Part	Reference	
ParentPartSource	The part source providing the parent part that passed down the requirement for the supply reported on this record. Null if the record pertains to supply of the end item itself whose demand was left unsatisfied. **Referenced table:** PartSource	Reference	
Part	The part the supply reported on this record pertains to. **Referenced table:** Part	Reference	
PartSource	Identifies the part source that provided the supply reported on this record. **Referenced table:** PartSource	Reference	
Pool	The pool, if any, that this supply belongs to. **Referenced table:** Pool	Reference	
PrePlanLimitDate	The earliest date that constraint is allowed to be be consumed by this supply (based on the absolute pre-plan limit).	Date	
ScheduledReceipt	Identifies the scheduled receipt whose supply details are reported on this record. Null unless **SupplyType** is set to "ScheduledReceipt" **Referenced table:** ScheduledReceipt	Reference	
StartUnit	The start unit of this supply.	Integer	

Table 8.103: MLSActivity (Mfg) fields

Field name	Description	Data type	Owner
SupplySource	Indicates the type of supply being reported and which multi-level search logic determined could not meet the need date (thus, contributing to the driver demand being left unsatisfied). The value reported in this field determines which of the supply references on the record are valid. Valid values are: • **None**—Indicates that multi-level search logic could not even plan a trial supply (for example, due to there not being an effective part source). • **ScheduledReceipt**—the supply is a scheduled receipt. • **TrialCoByProductSupply**—the supply is a trial co/by-product supply generated by multi-level search logic under a primary product's trial planned order or scheduled receipt. • **TrialPlannedOrder**—the supply is a trial planned order generated by multi-level search logic. • **TrialSubstituteSupply**—the supply is a trial substitute supply generated by multi-level search logic to try and cover demand for a primary part.	String (Enum)	
Transaction Sequence	The sequence assigned to the supply reported on this record. Depending on configuration settings, this is either taken from the demand the order satisfies or set to 0.	Integer	
TrialAvailableDate	The date the supply reported on this record would actually be completed and available in stock based on rolled up planned availability on components. For example, planning time fence and constraint availability on the part's components might make this date later than the planned due date.	Date	
TrialCoByProduct Supply	Identifies the trial co-product or by-product supply whose 32details are reported on this record. Null unless **SupplyType** is "TrialSubstituteSupply". **Referenced table:** TrialCoByProductSupply	Reference	
TrialPlannedOrder	Identifies the trial planned order whose supply details are reported on this record. Null unless **SupplyType** is set to "TrailPlannedOrder". **Referenced table:** TrialPlannedOrder	Reference	
TrialSubstitute Supply	Identifies the trial substitute supply whose supply details are reported on this record. Null unless **SupplyType** is set to "TrialSubstituteSupply". **Referenced table:** TrialSubstiututeSupply	Reference	

NewAllocation table

This is a calculated table that contains allocations (dependent demand items generated because a work order was created for a make item) from **ScheduledReceipt** records that did not have allocations imported into RapidResponse. Any allocations imported into RapidResponse are stored in the **Allocation** table.

Table 8.104: NewAllocation (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	The BOM record used to explode the requirements to the component from the Assemblies supply schedule. This reference can be used to get information about the part the demand is on as well as the part that caused the demand (single level peg). **Referenced table:** BillOfMaterial	Reference	Yes
DueDate	Date component is scheduled to be available to the manufacturer of the assembly (assembly's StartDate + BOM offset).	Date	
EarliestExpiryDate	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
ForecastQuantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	
MappedPool	The pool used in netting this demand. **Referenced table:** Pool.Value	Reference	
Quantity	Quantity of component needed.	Quantity	
ScheduledReceipt	Reference to the supply record from which this scheduled receipt was exploded. This is pegging. You can get the parent part and order information through this reference. **Referenced table:** ScheduledReceipt	Reference	
StartUnit	The first unit number of the part being made from the allocation represented by this record.	Integer	
TotalQuantity	Represents the value of **ScheduledReceipt.TotalQuantity** exploded through a bill of material entry.	Quantity	

NewTransferAllocation table

This table tracks dependent demands from ScheduledReceipt transfer orders.

Table 8.105: NewTransferAllocation (Mfg) fields

Field name	Description	Data type	Owner
DueDate	The date the dependent demand is needed to satisfy the ScheduledReceipt's ReschedDate.	Date	
EarliestExpiry Date	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
FlowToCommon Forecast Quantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
Forecast Quantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	
MappedPool	The pool used in netting this demand. **Referenced table:** Pool **NOTE**: This field was added in Version 2016.2, Service Update 19.	Reference	
PartSource	The transfer part source that creates this dependent demand. **Referenced table:** PartSource	Reference	Yes
Quantity	The quantity needed.	Quantity	
ScheduledReceipt	The Scheduled Receipt that creates this dependent demand. **Referenced table:** ScheduledReceipt	Reference	
TotalQuantity	Represents the value of **ScheduledReceipt.TotalQuantity** exploded through a transfer part source.	Quantity	
Unsatisfied Quantity	The portion of this dependent demand that was left unsatisfied.	Quantity	

Numbers table

The **Numbers** table contains only a single field named **Value** field, and can be used to generate a worksheet with a fixed number of records. Worksheets based on this table should typically include a

worksheet filter condition that defines the number of records to be returned in the worksheet. Without a filter condition, only one record is returned.

Typically the worksheet filter condition should at least contain a clause in the form of `Value <= Constant`. This sets the maximum number of records to be returned by the worksheet. For example, if the worksheet filter condition was `Value <= 100`, then 100 records would be returned in the worksheet (with Values between 1 and 100). Optionally, the "And" operator can be used to specify additional criteria for the filter. In such cases, if the full filter condition returns false then that **Value** is skipped and processing continues with the next one. For example, if the worksheet filter condition was `Value <= 100 AND <> 1`, then 99 records would be returned in the worksheet (with Values between 2 and 100).

If necessary, you can also use the **Value** results returned by the filter in other expressions. For example, if the worksheet filter condition was `value<=60`, then a column expression of `Today + (value-1)workday` would allow the worksheet to displays the dates of the next 60 workdays starting from today.

This table is not available from the Data Model dialog box. It is available only from the New Worksheet or Worksheet properties dialog boxes.

Note: If a worksheet filter is provided but does not contain an expected clause similar to `Value <= Constant`, then by default the maximum number of records to be returned is 1,000,000. For example, a filter of `Value >= 1` would return 1,000,000 records and a filter of `Value > 1` would return 999,999 records.

Table 8.106: Numbers (Core) fields

Field name	Description	Data type	Owner
Value	A number that defines the position of the record.	Quantity	

Part_MUECumLead table

The **Part_MUECumLead** table supports the Model-Effectivity and Unit-Effectivity analytics. When Model-Unit Effectivity is enabled this table is populated in the following manner:

Table 8.107: Populating the Part_MUECumLead (Mfg) fields

Value of part's Type.MakeBuy	Results
Make	The table is populated with the consolidated records of the part's Components.BOM_CumLead table. Records are consolidated by unique combinations of UseModel, Model, StartUnit and EndUnit.
Buy	The table is populated a single record with the following values: • UseModel = 'false' • StartUnit = –1 • EndUnit = –1 • CumLead = the part's CumLead field

This table is only available for use if you have enabled Model-Unit Effectivity.

Table 8.108: Part_MUECumLead (Mfg) fields

Field name	Description	Data type	Owner
CumLead	The cumulative leadtime for the part record for this Model and StartUnit / EndUnit range.	Quantity	
EndUnit	The effective ending unit. The following rules apply: • If MUEPoolNettingType.UnitRule is Net, EndUnit is copied from the Components.BOM_MUECumLead.EndUnit • If MUEPoolNettingType.UnitRule is not Net, it is assigned –1	Integer	
Model	The model for which the cumulative lead time applies. The following rules apply: • If MUEPoolNettingType.ModelRule is Net, Model is copied from the Components.BOM_MUECumLead.Model • If MUEPoolNettingType.ModelRule is not Net, it is None **Referenced table:** Model	Reference	
MultiSiteCumLeadTime	The multi-site cumulative leadtime for the part record for this Model and StartUnit / EndUnit range.	Quantity	
Part	The part record for which the cumulative lead time applies. **Referenced table:** Part	Reference	Yes
StartUnit	The effective starting unit. The following rules apply: • If MUEPoolNettingType.UnitRule is Net, StartUnit is copied from the Components.BOM_MUECumLead.StartUnit • If MUEPoolNettingType.UnitRule is not Net, it is assigned –1	Integer	
UseModel	Indicates whether the above model is used for netting. Valid values are: • **True**—the model is used for netting • **False**—the model is ignored for the calculation of the CumLeadTime	Boolean	

Part_MUEPoolValidation table

The **Part_MUEPoolValidation** table holds validation counts based on a part's model, unit, and pool settings. This table supports data validation and is used for internal purposes only.

Table 8.109: Part_MUEPoolValidation (Mfg) fields

Field name	Description	Data type	Owner
DifferencesThisLevel	Count of all dates of which the sum of order quantities in the **PlannedOrder** table differs from the sum of order quantities in the **BaselinePlannedOrdet** table (for this Part, Model, Unit, and Pool combination).	Integer	
DifferencesRollUp	This explosion includes all part sources for the part (not just the primary source).	Integer	
Model	Model that applies to this record. If **Part.MUEPoolNettingType.ModelRule** is not set to "Net", or if the Model-Unit Effectivity is not enabled, this field is set to the first value in the Model table (typically, this value is "None"). **Referenced table:** Model	Reference	
Part	The part the order validation details are for. **Referenced table:** Part	Reference	Yes
Pool	Pool that applies to this record. If **Part.MUEPoolNettingType.PoolRule** is not set to "Net", or if Pool is not enabled, this field set to the first value in the Pool table (typically, this value is "Unpooled"). **Referenced table:** Pool	Reference	
StartUnit	Start unit that applies to this record. If **Part.MUEPoolNettingType.UnitRule** is not set to "Net", or if Model-Unit Effectivity is not enabled this field is set to -1.	Integer	

PartSourceCTPPlannedOrder table

This table reports all CTPPlannedOrder records that use a PartSource.

Table 8.110: PartSourceCTPPlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
CTPPlannedOrder	A reference to the CTPPlannedOrder. **Referenced table:** CTPPlannedOrder	Reference	
PartSource	The PartSource used by the CTPPlannedOrder. **Referenced table:** PartSource	Reference	Yes

PartSourcePlannedOrder table

This table reports all the PlannedOrder records that use a PartSource.

Table 8.111: PartSourcePlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
PartSource	The PartSource used by the PlannedOrder. **Referenced table:** PartSource	Reference	
PlannedOrder	The PlannedOrder using this PartSource. **Referenced table:** PlannedOrder	Reference	Yes

PartSourceScheduledReceipt table

This table reports all the ScheduledReceipts that use a PartSource.

Table 8.112: PartSourceScheduledReceipt (Mfg) fields

Field name	Description	Data type	Owner
PartSource	The PartSource used by the PlannedOrder. **Referenced table:** PartSource	Reference	Yes
ScheduledReceipt	The ScheduledReceipt using this PartSource. **Referenced table:** ScheduledReceipt	Reference	

PeriodConstraintLoad table

This table reports bucketed load on a constraint. It reports both Load and UsedLoad. PeriodConstraintLoad records can be generated for each constraint period between DataDate and continuing out the number of days specified by ConstraintType.ReportLimit. There are never any PeriodConstraintLoad records generated after the specified limit.

Note: If a constraint does not have any load placed on it, then it is not reported in this table. For example, this can happen if all **SourceConstraint** records that reference the constraint have 0 (zero) fixed and variable constraint factors.

Table 8.113: PeriodConstraintLoad (Mfg) fields

Field name	Description	Data type	Owner
Available	The amount of constraint available in this period (**Constraint.Calendar**). This field is calculated using the **Rate** field on **ConstraintAvailable** records that are effective over a given period. If there are multiple **ConstraintAvailable** records effective over a given constraint period, they are summed together to give the total constraint available. Additionally, **ConstraintAvailable** records with a negative **Rate** value can be used to reduce the total constraint available for a given period.	Quantity	
Constraint	The name of the Constraint. **Referenced table:** Constraint	Reference	Yes
CurrentLoad	The Load in this period from scheduled receipts.	Quantity	
CurrentSpreadLoad	The SpreadLoad allocated in this period from scheduled receipts.	Quantity	
CurrentUsedLoad	The Load allotted in this period from Scheduled Receipts.	Quantity	
Overloaded	Indicates if any constraint associated with the supply is overloaded (does not have sufficient remaining capacity) and making the supply late (applies to constrained constraints only). Valid values are: • Y • N **Note:** For scheduled receipts where **Status.RescheduleRule = 'Scheduled'**, a value of **Y** indicates that one of the constraints is overloaded by the scheduled receipt.	Boolean	
Period	The beginning of each Calendar period for this Constraint. Note, there is exactly one record for each Constraint.Calendar period from Constraint.DataDate + 0 Constraint.Calendar to (Constraint.DataDate + Constraint.Type.ReportLimit) + 0 Constraint.Calendar.	Date	
TotalLoad	The quantity of load placed on this Constraint in this period.	Quantity	
TotalSpreadLoad	The total SpreadLoad allocated in this period from both scheduled receipts and planned orders.	Quantity	
TotalUsedLoad	The quantity allotted from this Constraint in this period.	Quantity	

PeriodsOfCoverage table

The **PeriodsOfCoverage** table calculates the excess supply that will be available at the end of a defined periods of coverage interval (for example, weeks or months), along with the number of subsequent periods of coverage intervals whose demands can be satisfied by that excess supply. For each unique part, model, unit, and pool combination that has activity, this table contains a record corresponding to each date defined within a part's **PlanningCalendars.TimeUnits** calendar.

This table is similar to the **PeriodsOfCTPCoverage** table except that its calculations are based on the Netting analytic. As such, supplies are considered at their rescheduled dates, and this table does not account for expiring supplies (that is, the calculation of total supply might include expiring supply).

Note: The calendar whose dates define the periods of coverage intervals used by the calculations that populate this table is set by a processing rule in RapidResponse. For more information, see "Defining a periods of coverage calendar" on page 2206.

Table 8.114: PeriodsOfCoverage (Mfg) fields

Field name	Description	Data type	Owner
Coverage	The number of subsequent periods of demand that can be satisfied by the excess supply available at the end of the periods of coverage interval containing this date. This value may be fractional. For example, assume that the PeriodsOfCoverageCalendar is set to "Weekly", and the ExcessSupply is calculated as 1,300. If the next three weeks have total demand for 600, 400, and 300 units respectively, then this field reports 3. However, if the next three weeks have total demand for 600, 400, and 400 units respectively, this field reports 2.75.	Quantity	
Date	The date on which coverage was calculated for the part. A record is generated in this table for each date defined within a part's **PlanningCalendars.TimeUnits** calendar.	Date	
DemandPeriod	The first date in the demand period in which the last of the excess supply reported on this record is fully consumed by demand.	Date	
ExcessSupply	The excess supply, if any, available at the end of the periods of coverage interval containing this date. Calculated by summing all supplies whose rescheduled date falls on or before this date, and then subtracting the sum of all demands whose rescheduled date falls on or before the last date in the periods of coverage interval containing this date. If the calculation returns a value greater than zero, then that value is reported in this field and a positive Coverage value will also be calculated. Otherwise, both this field and Coverage return 0 (zero).	Quantity	

Table 8.114: PeriodsOfCoverage (Mfg) fields

Field name	Description	Data type	Owner
Model	The pool, if any, used by netting calculations for the activity information in this record. **Referenced table:** Model	Reference	
Part	The part associated with this periods of coverage record. **Referenced table**: Part	Reference	Yes
Pool	The pool, if any, used by netting calculations for the activity information in this record **Referenced table:** Pool	Reference	
StartUnit	The unit used by netting calculations for the activity information in this record.	Integer	

PeriodsOfCTPCoverage table

The **PeriodsOfCTPCoverage** table calculates the excess supply that will be available at the end of a defined periods of coverage interval (for example, weeks or months), along with the number of subsequent periods of coverage intervals whose demands can be satisfied by that excess supply. For each unique part, model, unit, and pool combination that has activity, this table contains a record corresponding to each date defined within a part's **PlanningCalendars.TimeUnits** calendar.

This table is similar to the **PeriodsOfCoverage** table except that its calculations are based on the Capable-To-Promise analytic. As such, supplies are considered at their available dates, and this table accounts for expiring supplies (that is, the calculation of total supply subtracts supplies as they expire).

Notes:

- The calendar whose dates define the periods of coverage intervals used by the calculations that populate this table is set by a processing rule in RapidResponse. For more information, see "Defining a periods of coverage calendar" on page 2206.
- This table does not currently report push planning activity.

Table 8.115: PeriodsOfCTPCoverage (Mfg) fields

Field name	Description	Data type	Owner
CTPCoverage	The number of subsequent periods of demand that can be satisfied by the excess supply available at the end of the periods of coverage interval containing this date. This value may be fractional. For example, assume that the PeriodsOfCoverageCalendar is set to "Monthly", and the ExcessSupply is calculated as 1,000. If the next two months have total demand for 600 and 400 units respectively, then this field reports 2. However, if the next two months have total demand for 600, and 500 units respectively, this field reports 1.8.	Quantity	
Date	The date on which coverage was calculated for the part. A record is generated in this table for each date defined within a part's **PlanningCalendars.TimeUnits** calendar.	Date	
DemandPeriod	The first date of the demand period in which the excess supply reported on this record is fully consumed by demand.	Date	
ExcessSupply	The excess supply, if any, available at the end of the periods of coverage interval containing this date. Calculated by summing all supplies whose available date falls on or before this date, and then subtracting the sum of all demands whose rescheduled date falls on or before the last date in the periods of coverage interval containing this date. If this calculation returns a value greater than zero, then that value is reported in this field and a positive CTPCoverage value will also be calculated. Otherwise, both this field and CTPCoverage return 0 (zero).	Quantity	
Model	The pool, if any, used by netting calculations for the activity information in this record. **Referenced table:** Model	Reference	
Part	The part associated with this periods of coverage record. **Referenced table**: Part	Reference	Yes
Pool	The pool, if any, used by netting calculations for the activity information in this record **Referenced table:** Pool	Reference	
StartUnit	The unit used by netting calculations for the activity information in this record.	Integer	

PlannedAllocation table

Records in the **PlannedAllocation** table are calculated by the explode calculation for each BillOfMaterial record. The Type field of the BillOfMaterial record specifies a BOMType control record controlling the generation of PlannedAllocation records through the bill relationship. For each PlannedOrder on an Assembly part, a PlannedAllocation record may be generated. The TimeUnits calendar used in the lead-time offset calculation is specified by the Assembly part's PlanningCalendars. It contains dependent demand from planned orders (supply recommendations made by the MRP logic).

PlannedAllocation records can also be generated from **Forecast** or **AvailableToPromise** records if the assembly is a MPS part and the **BillOfMaterial.Type** is a MPS planning type of bill. For information about the BOMType table, see "BOMType table" on page 687.

Table 8.116: PlannedAllocation (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	The BOM record used to explode the requirements to the component from the Assemblies supply schedule. This reference can be used to get information about the part the demand is on as well as the part that caused the demand (single level peg). **Referenced table:** BillOfMaterial	Reference	Yes
DueDate	Date the component needs to be available to the manufacturer of the assembly. It is the assembly's PlannedOrder StartDate plus the effective LeadTimeOffset on the BillOfMaterial.	Date	
EarliestExpiryDate	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
ForecastQuantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	
MappedPool	The pool used in netting this demand. **Referenced table:** Pool.Value	Reference	
Model	The model that applies to this allocation. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is Ignore **Referenced table:** Model	Reference	

Table 8.116: PlannedAllocation (Mfg) fields

Field name	Description	Data type	Owner
PegPlannedOrder	The planned order that creates this planned allocation. **Referenced table:** PlannedOrder	Reference	
Quantity	The quantity of the component required (BillOfMaterial.Component).	Quantity	
StartUnit	The start unit number that applies to this allocation. It is set to-1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is Ignore	Integer	
Type	The demand from the PlannedOrder supply type or from a planning bill (see BOMType and PartType control tables). **Referenced table:** DemandType	Reference	

PlannedOrder table

The **PlannedOrder** table is a calculated table providing recommendations for new supply orders to be generated (receipts). The supply type is determined from the **PartSourceType.PlannedOrderSupplyType** control value referenced by the part source. There will never be more than one planned order generated for a particular part on the same TimeUnits interval (for example, work day). The order produced will cover all demand within the period. **PartSource.OrderPolicy** control values, such as those used when implementing parameters like days supply and lot sizing, can increase the effective coverage period and/or cause multiple planned orders to be generated on the same **TimeUnits** interval

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
BatchIndex	If the planned order is created as part of a production campaign, this represents the order's batch number (unique index value) within its campaign. For example, the first order scheduled in each campaign is given a value of "1", the second order a value "2", and so on. Otherwise, for all orders from part sources that aren't enabled for campaign planning (have an **OrderPolicy.MaximumUsage** value other than "CampaignPlanning"), this field returns "0".	Integer	
CalcRequest StartDate	The date on which work should start on this supply in order to meet the earliest request date on any earliest driving independent demand it satisfies. Based on the original **RequestDate** exploded down from the driver **IndependentDemand** record and adjusted by lead time at each level in the product structure (if the supply directly satisfies an independent demand itself, then this is just the **RequestDate** adjusted by the part's lead time). If the driver demand is not an independent demand, or has an undefined **RequestDate**, then this is the date on which work should start on this supply in order to meet the due date on the driver. **Note:** The date associated with a given planned order might be subject to limitations when multi-level search logic is enabled and the **MultiLevelSearchRule.ShortTermExcess** is set to "Reduce". For more information, see "Reducing short-term excess inventory" on page 1870.	Date	
CampaignIndex	If the planned order is created as part of a production campaign, this gives the unique index value of the campaign it belongs to. Campaigns for a given part source are numbered starting with 1 for the earliest planned campaign and then incrementing for each subsequent campaign. Otherwise, for all supplies from part sources that aren't enabled for campaign planning (have an **OrderPolicy.MaximumUsage** value other than "CampaignPlanning"), this field returns "0". **Note:** This field supports the optional Campaign Planning capability.	Integer	

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
DemandDate	The date of the demand associated with this planned order. This field is calculated as one of the following: • For order point parts (that is, parts with a SafetyStockPolicy.SafetyStockRule of 'OrderPointAverage', 'OrderPointOver', or 'OrderPointUnder'), this is the date when inventory for the part dropped below the order point threshold. • For all other parts, this reports the due date of the first demand satisfied by this supply. **Note:** The date reported in this field must belong to the part's **PlanningCalendars.TimeUnits** calendar. If the supply is created to cover an independent demand with a due date that does not fall on the part's **TimeUnits** calendar, it is adjusted back to the previous **TimeUnits** interval before being reported in this field.	Date	
DockDate	Date the supply is planned to be delivered to the receiving dock.	Date	
DockDateFrom Start	The adjusted dock date calculated by applying lead time forward from the order start date. This is meant to account for any calendar rounding incurred when applying lead time backward from the due date to determine order start. For example, parts that use "cumulative" lead time might end up with an earlier dock date from start based on rounding between time units, transit, ship, and lead time unit calendars. **Note:** This field was added in Version 2016.2, and its actual calculation is impacted by the applicable **TransitRule** setting. For more information, see "Lead time calendar rounding" on page 1319.	Date	
DueDate	The date the order is planned to be in stock.	Date	

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
DueDateFrom Start	The adjusted due date calculated by applying lead time forward from the order start date. This is meant to account for any calendar rounding incurred when applying lead time backward from the due date to determine order start. For example, parts that use "cumulative" lead time might end up with an earlier due date from start based on rounding between time units, transit, ship, and lead time unit calendars. **Note:** This field was added in Version 2016.2., and its actual calculation is impacted by the applicable **TransitRule** setting. For more information, see "Lead time calendar rounding" on page 1319.	Date	
EarliestExpiry Date	The earliest date that any supplies used to satisfy dependent demands generated from this order may expire. Typically, this is determined from the latest **EarliestExpiryDate** found on a demand that Netting calculations determine this supply will satisfy. Otherwise, if there are no consuming demands, this value is calculated by adding the number of expiry calendar intervals defined in **Part.MinimumShelfLife** to the supply due date. Note also that although planned orders used to satisfy dependent demands typically inherit their **EarliestExpiryDate** from those dependent demands, this can be overridden. If a positive value is given in the **MinimumShelfLifeThreshold** field on the **PartSource** used to supply the order, then those number of expiry calendar intervals are added to the order due date. If the resulting date is later than the earliest expiry date found on the dependent demand, it is reported in this field (otherwise, the date from the dependent demand is used instead). For non-expiry parts (**PartType.ExpiryRule is** set to "Ignore"), this field always reports "Past".	Date	
Estimated ExpiryDate	The date that the Netting analytic calculates this supply will expire. Used to determine which demands this order can potentially be used to satisfy (supply is only eligible for use if it is on or before the demand's **EarliestExpiryDate**). Calculated starting from **DueDate** and then adding the number of expiry calendar intervals defined in **PartSource.IntervalsToExpiry**. Note that this date might be different than the actual calculated expiry date reported in **CTPPlannedOrder.ExpiryDate** which takes supply availabilities into account. Non-expiry parts (**PartType.ExpiryRule** is set to "Ignore"), always return a value of "Future".	Date	

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
EffQuantity	The quantity of this order that is expected to be received into inventory after applying yield/scrap, considering any co-product or by-product supply, and converting the unit of measure to inventory units.	Quantity	
Line	The "line number" for this planned order. Orders planned for each part start at 1 and increment for each planned order.	Integer	
Model	The model that applies to this supply. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is Ignore **Referenced table:** Model	Reference	
NeedQty	Quantity actually needed to cover the demand. This field is expressed in stock units, and inflated to cover yield. **Note**: S**upplier UOM** applies to **NeedQty**.	Quantity	
OrderPriority	The priority assigned to this planned order. OrderPriority is calculated as follows: • If **Part.Type.CommitLevel** is "High" or "MediumExplodePriority", then this is set to the priority of the demand that drove the requirement for the planned order. • If **Part.Type.CommitLevel** is "Medium" or "Low", then this is set to the part's default priority (**Part.Type.DefaultPriority**). • If the planned order is generated to satisfy safety stock requirements, then this is set to **Part.Type.DefaultPriority** regardless. **Referenced table:** OrderPriority	Reference	
Origin	Indicates the origin of this planned order record. Valid values are: • **Manual**— The planned order record was created to model a manual supply allocation record in the SupplyAllocation table. • **Planned**—The planned order record was created by RapidResponse netting calculations.	String (Enum)	

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
ParameterDate	The date used to determine which time-phased parameter values are applicable to the supply (compared against the **EffectiveInDate** values on certain applicable tables). For example, if using time-phased yield then the **TimePhasedYield** record effective for the part source on this date determines the yield factor. Also, applies to time-phased **SourceConstraint** records, as well as time-phased lead time logic (records in the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables). This is typically set to the planned order's due date. However, when constraints are used this might be the order's stock date instead. For example, this can occur when constraint availability causes an order to be pre-planned. **Note**: This field was added in Version 2016.2, Service Update 12.	Date	
Part	The name of the part. **Referenced table:** Part	Reference	Yes
PartSource	The PartSource record that was used to create the planned order. PartSource can be used to find PartSource.Supplier and PartSource.SupplierPart **Referenced table:** PartSource	Reference	
Pool	The pool associated with this planned order. **Referenced table:** Pool	Reference	
Quantity	The amount required (including any allowances for yield, as well any co-product or by-product supply considerations). **Note**: S**upplier UOM** applies to **Quantity**.	Quantity	
ShipDate	The date the supply should be shipped from its supplier.	Date	
ShipDateFrom Start	The adjusted ship date calculated by applying lead time forward from the order start date. This is meant to account for any calendar rounding incurred when applying lead time backward from due date to determine order start. For example, parts that use "cumulative" lead time might end up with an earlier ship date from start based on rounding between ship and lead time unit calendars. **Note:** This field was added in Version 2016.2, and its actual calculation is impacted by the applicable **TransitRule** setting. For more information, see "Lead time calendar rounding" on page 1319.	Date	

Table 8.117: PlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
SourceKanban	The source kanban associated with this planned order. **Referenced table**: SourceKanban	Reference	
StartDate	The date to start the order (now adjusted to compensate for constraint).	Date	
StartUnit	The start unit number that applies to this supply. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is 'Ignore'	Integer	
StockUnit Quantity	Represents the supply quantity in inventory unit of measure (UOM) without applying yield or scrap factors. **Note**: S**upplier UOM** applies to **StockUnitQuantity**.	Quantity	
Transaction Sequence	The sequence of the planned order. Copied from the demand it satisfies if **CommitLevel** is set to 'High', or set to 0 if **CommitLevel** is set to 'Medium' or 'Low' or if the supply satisfies no specific demand (that is, from safety stock). The sequence is then assigned to transaction sequence on resulting dependent demands. If dependent demands have the same order priority and **MaintainCommitments** is set to 'Y' for dependent demand's part, then supplies will be allocated by sequence (smallest to largest).	Integer	

PlannedTransferAllocation table

The **PlannedTransferAllocation** table captures dependent demand from planned transfer orders.

Table 8.118: PlannedTransferAllocation (Mfg) fields

Field name	Description	Data type	Owner
DueDate	The date this order is available for use (stock date).	Date	
EarliestExpiryDate	The earliest date that a supply may expire if it is to be used in satisfying this demand.	Date	
FlowToCommon ForecastQuantity	The quantity of this order that consumed forecast. from the common forecast pool (that is, because all of the customer's forecast had already been consumed).	Quantity	
ForecastQuantity	The quantity of this order that consumed forecast. This includes both forecast consumed from the customer's forecast and forecast consumed from the common pool forecast.	Quantity	

Table 8.118: PlannedTransferAllocation (Mfg) fields

Field name	Description	Data type	Owner
MappedPool	The pool used in netting this demand. **Referenced table:** Pool **NOTE**: This field was added in Version 2016.2, Service Update 19.	Reference	
PartSource	The transfer part source that creates this dependent demand. **Referenced table:** PartSource	Reference	Yes
PlannedOrder	The planned order that creates this dependent demand. **Referenced table:** PlannedOrder	Reference	
Quantity	The quantity needed.	Quantity	
Unsatisfied Quantity	The portion of this dependent demand that was left unsatisfied.	Quantity	

ProductionCycle table

The **ProductionCycle** table reports on the cycles of a production wheel, including cycle start and end dates, constraint used in the cycle, and changes in constraint availability that might occur during the cycle.

This table was added in Service Update 1902.

Table 8.119: ProductionCycle(Mfg) fields

Field name	Description	Data type	Owner
Constraint	A reference to the constraint associated with this ProductionCycle record. **Referenced table:** Constraint	Reference	
ConstraintAvailable	The quantity of constraint available to one cycle of the production wheel. Measured in constraint units.	Quantity	
ConstraintUsed	The quantity of constraint used in one cycle of the production wheel. Measured in constraint units.	Quantity	
CycleEndDate	The date that the production wheel cycle ends. This field uses the same calendar as the production wheel.	Date	
CycleNumber	The cycle number for the wheel. For example, a cycle number of 3 means that the cycle is the third cycle in the production wheel.	Integer	
CycleStartDate	The date that the production wheel cycle starts. This field uses the same calendar as the production wheel.	Date	
ProductionWheel	A reference to the production wheel associated with this ProductionCycle record. **Referenced table:** ProductionWheel	Reference	

ResourceDataByDate table

The **ResourceDataByDate** table contains a record for each date on the resource's calendar (**ResourceType.Calendar**) starting from **Resource.DataDate** and ending the number of **ResourceType.Calendar** intervals after that as defined by the value in **ResourceType.ReportLimit**. This table is used for reporting resource details such as effective costs and working hours, along with the actual load placed on the resource in each period.

Table 8.120: ResourceDataByDate (ProjMgmt) fields

Field name	Description	Data type	Owner
Date	The date these resource details apply to.	Date	
FixedCost	The one-time fixed (set) cost incurred if the resource to a task on this date. Taken from **ResourceCosts.FixedCost** (if there is an effective record for the resource on this date), or else **Resource.FixedCost.**	Money	
HoursPerDay	The standard hours per day available from one unit of the resource on this date. Taken from **ResourceCalendatException.HoursPerDay** (if there is a day of week exception applicable for the resource on this date), or **ResourceAvailable.HoursPerDay** (if there is an effective record for the resource on this date), or else **Resource.HoursPerDay.**	Quantity	
Load	The total actual load placed on the resource on this date (summed across all tasks and projects the resource is assigned to). This value is expressed in terms of the effective units available. For example, if the effective **MaximumUnits** is 1 and **HoursPerDay** is 8, then a value of 1 in this field indicates the resource applied 8 hours of effort to the task on this date. Similarly, a value of .75 would indicate 6 hours of effort applied to the task, a value of 1.25 would indicate 10 hours of effort applied to the task, and so on.	Quantity	
OvertimeRate	The standard hourly rate charged if the resource is assigned to a task on this date (used until the resource's maximum units for this date are reached). Taken from **ResourceCosts.OvertimeRate** (if there is an effective record for the resource on this date), or else **Resource.OvertimeRate.** **Note:** This values does not consider any project or task-level rate adjustments.	Money	
Resource	A reference to the resource the details reported on this record pertain to. **Referenced table:** Resource	Reference	Yes

Table 8.120: ResourceDataByDate (ProjMgmt) fields

Field name	Description	Data type	Owner
StandardRate	The standard hourly rate charged when the resource is assigned to a task on this date (used until the resource's maximum units for this date are reached). Taken from **ResourceCosts.StandardRate** (if there is an effective record for the resource on this date), or else **Resource.StandardRate.** **Note:** This value does not consider any project or task-level rate adjustments.	Money	
Units	The maximum units available from the resource on this date. Each unit is expressed in terms of the effective **HoursPerDay** value. Taken from **ResourceAvailable.MaximumUnits** (if there is an effective record for the resource on this date), or else **Resource.MaximumUnits.**	Quantity	

ResourceLoad table

The **ResourceLoad** table contains a record for each date on which a resource has load generated on it from a specific task. Each record identifies a resource and task assignment, along with details such as the load generated from the task assignment and the associated resource costs. This table can be used to help understand the load placed against resources, including identifying any overloaded resources as well as the sources those loads, as well as all fixed, standard and overtime resource costs incurred from task assignments.

Table 8.121: ResourceLoad (ProjMgmt) fields

Field name	Description	Data type	Owner
Date	The date on which load was generated on this resource from the specified task assignment.	Date	
EffectiveStandard Rate	The hourly rate effective for the resource when working on this task assignment (including any rate adjustments).	Money	
EffectiveOvertime Rate	The hourly overtime rate effective for the resource when working on this task assignment (including any rate adjustments).	Money	

Table 8.121: ResourceLoad (ProjMgmt) fields

Field name	Description	Data type	Owner
EfficiencyFactor	Efficiency rate for the resource against the referenced task on this date. This field displays values between 0 and 1, where 1 indicates full resource efficiency. Rates less than one might be reported on resources subject to an efficiency curve. That is, resources assigned to tasks where **TaskType.EfficiencyProfileRule** is set to "Ramp" and **Task.Ramp** has a positive value, or where **TaskType.EfficiencyProfileRule** is set to"PointsCurve" and **Task.PointsCurve** references a valid points curve. In either of these cases, if the day of the resource assignment corresponds to a point on the associated efficiency curve with a value less than one, than an efficiency rate is applied. The efficiency rate does not impact the reported load or cost values on this date, but is an indication of how close to maximum efficiency the resource is working against the assigned task on this date and it will typically impact the calculated task duration and finish date.	Quantity	
FixedCost	Reports any fixed (set) cost incurred on this date from assigning the resource to the task. If there is a fixed cost to report, then the value to use comes from the **FixedCost** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on this date). Typically, the fixed cost is reported on the first day a resource is assigned to a task, however, based on the **TaskType.FixedCostAccrualRule** setting, it can instead be reported on the last day of the resource's task assignment or spread evenly throughout the resource's task assignment.	Money	

Table 8.121: ResourceLoad (ProjMgmt) fields

Field name	Description	Data type	Owner
Load	The load in units placed on the resource for this task assignment and date. This value can be seen as a factor of the effective maximum hours per day for the resource as defined on either the **Resource** or **ResourceAvailable** table. For example, if **Resource.MaxiumUnits** is 1 and **Resource.HoursPerDay** is 8, then a value of 1 in this field indicates the resource applied 8 hours of effort to the task on this date. Similarly, a value of .75 would indicate 6 hours of effort applied to the task, a value of 1.25 would indicate 10 hours of effort applied to the task, and so on. Note that if the date and resource defined on this record corresponds to a day with an effective record in the **ResourceCalendarException** table, the load reported in this field is still reported as a factor of the maximum hours per day as defined on either the **Resource** or **ResourceAvailable** tables. For example, assume a resource assigned at 100% to a task where **Resource.MaxiumUnits** is 1 and **Resource.HoursPerDay** is 8, and further assume a calendar exception on Friday's to indicate the resource has **HoursPerDay** of 4. In this case, the Load value would be .5 on Fridays, even though the resource's available hours are fully allocated to the task.	Quantity	
OvertimeCost	Reports the overtime cost incurred on a given date from the resource's work on this task. This value is calculated by multiplying the number of hours worked on the task that are in excess of the resource's maximum per day by the resource's hourly overtime rate (subject to any project or task-level rate adjustments). A resource's hourly overtime rate comes from the **OvertimeRate** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on this date), and the maximum hours per day comes from the combination of the **HoursPerDay** and **MaximumUnits** fields on either the **Resource** table or the **ResourceAvailable** table (if there is an effective record for the resource on this date). For example, if the effective **OverTimeRate** is $25, **HoursPerDay** is 8, **MaximumUnits** is 1, and the resource works for 10 hours against a given task on that date, the overtime cost is $50. Although by default, overtime costs are reported on each day that a resource incurs excess load from the assigned task, this is configurable. Based on options in **ResourceType.CostAccrualSetting,** overtime costs from a resource's task assignment can be accrued and reported on either the first day of the task assignment or the last day of the task assignment.	Money	

Table 8.121: ResourceLoad (ProjMgmt) fields

Field name	Description	Data type	Owner
OvertimeHours	The number of hours worked on the task that are in excess of the resources's maximum per day. If a resource is assigned to multiple tasks on the same date and the total hours worked exceeds the maximum for the resource, the overtime hours reported are split between the tasks in proportion to the hours assigned to each.	Quantity	
Project	A reference to the project containing the task that this resource was assigned to. **Referenced table:** Project	Reference	
Resource	A reference to the resource that had load generated on it from the referenced task assignment. **Referenced table:** Resource	Reference	Yes
StandardCost	Reports the standard cost incurred on a given date from the resource's work on this task. This value is calculated by multiplying the number of hours worked on the task that are not in excess of the resource's maximum per day by the resource's standard hourly rate (subject to any project or task-level rate adjustments). A resource's standard hourly rate comes from the **StandardRate** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on this date), and the maximum hours per day comes from the combination of the **HoursPerDay** and **MaximumUnits** fields on either the **Resource** table or the **ResourceAvailable** table (if there is an effective record for the resource on this date). For example, if the effective **StandardRate** is $20, **HoursPerDay** is 8, **MaximumUnits** is 1, and a resource works for 10 hours against a single task on a given date, the standard cost is $160. Although by default standard costs are reported on each day that a resource incurs load from the assigned task, this is configurable. Based on options in **ResourceType.CostAccrualSetting,** standard costs from a resource's task assignment can be accrued and reported on either the first day of the task assignment or the last day of the task assignment.	Money	
StandardHours	The number of hours worked on the task that are not in excess of the resources's maximum per day. If a resource is assigned to multiple tasks on the same date and the total hours worked exceeds the maximum for the resource, the standard hours reported are split between the tasks in proportion to the hours assigned to each.	Quantity	
Task	A reference to the task that this resource was assigned to. **Referenced table:** Task	Reference	

SafetyStockHistoricalDemand table

The **SafetyStockHistoricalDemand** table reports details of historical demand data (as taken from the **HistoricalDemandActual** table) for parts configured as safety stock items. For each safety stock item, this table reports different historical demand quantities pertaining to the historical demand category and within the date range specified on the **SafetyStockItem** record. These demands are bucketed by the calendar specified in the **SafetyStockItemType.IntervalsCalendar** field (by default, an everyday calendar is used) and one record is reported for each of these intervals that fall within the date range for the item.

Worksheets based on this table are required as input to the **HistoricalDemands** parameter of the **SafetyStock** function. This function uses historical and future looking data to provide recommended safety stock and reorder points to meet a specified service level, and also calculates details such as average historical demand.

In addition, the **SafetyStockHistoricalDemand** table reflects the results of the historical demand mapping of safety stock items.

Note: If you mapped a safety stock item to use the historical values from a different part using the **SafetyStockItemMapping** table, the **SafetyStockHistoricalDemand** table reflects the results of the mapped historical values.

Table 8.122: SafetyStockHistoricalDemand (Mfg) fields

Field name	Description	Data type	Owner
Date	The date this bucketed historical demand quantity applies to (demands are bucketed by the calendar referenced in **Item.Type.IntervalsCalendar** field).	Date	
IsOutlier	Indicates if the original demand quantity is considered an outlier. Outlier detection is based on the **Item.DemandOutlierThreshold** setting (the value in this field defines the minimum number of standard deviations away from the mean a demand quantity must be in order to be considered an outlier). Valid values are: • **N**—demand is not an outlier. • **Y**—demand is an outlier.	Boolean	
Item	A reference to the safety stock item. This identifies the specific part, and other details, associated with the historical demand reported on this record. **Referenced table:** SafetyStockItem	Reference	Yes
OriginalQuantity	The original quantity of the historical demand for this date; this is after any cleansing based on causal factors but before any adjustments for outliers. If there are no demands corresponding to a bucket date in the item's historical interval range, then a record is still generated and a 0 (zero) value reported in this field.	Quantity	

Table 8.122: SafetyStockHistoricalDemand (Mfg) fields

Field name	Description	Data type	Owner
Quantity	The quantity of the historical demand for this date after any adjustments for outliers have been made. Outlier adjustments are made based on the setting in the **Item.Type.DemandOutlierProcessingRule** field.	Quantity	
RollingLeadTime Quantity	The total quantity of historical demands due within the part's lead time based on this historical demand date. For example, if using a workday calendar and a constant lead time of 3 days, this value reports the total demand quantity that was due within three work days of this historical date. A value of -1 is reported in this field for dates where there is not enough data generated to determine demand with lead time (for example, this might apply to the last demands within the range of historical dates being reported)	Quantity	

SafetyStockHistoricalForecast table

The **SafetyStockHistoricalForecast** table reports bucketed historical forecast data for parts configured as safety stock items.

For each **SafetyStockItem** record where the **HistoricalForecastCategory** field has a non-null reference to the **HistoricalDemandCategory** table, forecast records in the **HistoricalDemandSeriesDetail** table referencing that same category through the **Series.Header.Category** reference are evaluated. These historical forecasts are then collected and grouped using **SafetyStockItem.AsOfDate** calendar intervals. If there are multiple **HistoricalDemandSeries** records whose **AsOfDate** falls within the interval, then only the earliest of those (and its associated Detail records) are collected.

One forecast record is then generated in the **SafetyStockHistoricalForecast** table for each **SafetyStockItemType.IntervalsCalendar** interval (within the item's defined date range). The historical forecast records in this table are then used, together with data in the **SafetyStockHistoricalDemand** table, when calculating the standard deviation of historical forecast error. Standard deviation of forecast error is calculated for single-echelon items where **SafetyStockItemType.StandardDeviationDemandRule** is set to "ForecastError".

Table 8.123: SafetyStockHistoricalForecast (Mfg) fields

Field name	Description	Data type	Owner
Date	The date the historical forecast on this record pertains to. Records are generated for calendar intervals set by a safety stock item's **Type.IntervalsCalendar** reference.	Date	
Item	A reference to the safety stock item. This identifies the specific part, and other details, associated with the historical forecast reported on this record. **Referenced table:** SafetyStockItem	Reference	Yes
Quantity	The historical forecast quantity for the date on this record.	Quantity	

SafetyStockHistoricalSupply table

The **SafetyStockHistoricalSupply** table reports details of historical supply data (as taken from the **HistoricalSupplyActual** table) for parts configured as safety stock items and which reference a **SafetyStockItemType.SupplyVariabilityRule** value of "Use". For each of those safety stock items, this table reports historical supply quantities pertaining to the historical supply category and within the date range specified on the **SafetyStockItem** record. As well, each supply's actual lead time is calculated for subsequent use in lead time variability calculations.

Worksheets based on this table are required as input to the **LeadTime** parameter of the **SafetyStock** function. This function uses historical and future looking data to provide recommended safety stock and reorder points to meet a specified service level, and also calculates details such as average lead time.

Note: If you mapped a safety stock item to use the historical values from a different part using the **SafetyStockItemMapping** table, the **SafetyStockHistoricalSupply** table reflects the results of the mapped historical values.

Table 8.124: SafetyStockHistoricalSupply (Mfg) fields

Field name	Description	Data type	Owner
Date	The date associated with the actual shipment of this historical supply.	Date	
IsLeadTimeOutlier	Indicates if the original lead time value was considered an outlier. Valid values are: • **False**—The data point is not a lead time outlier. • **True**—The data point is a lead time outlier. If the original lead time was considered an outlier by less than 0.5 days, once the **LeadTime** is rounded to the nearest integer, then **LeadTime** = **OriginalLeadTime**. In this case, this field will report a value of "False". **Note:** This field was added in Version 2016.2, System Update 5.	Boolean	
Item	A reference to the safety stock item. This identifies the specific part, and other details, associated with the historical supply reported on this record. **Referenced table:** SafetyStockItem	Reference	Yes
LeadTime	The lead time value after applying outlier adjustments from **MEIOLeadTimeOutlier**. Note that outlier adjustments can yield a fractional lead time. Values in this field are rounded to the nearest integer. **Note:** This field was modified in Version 2016.2, Service Update 5.	Integer	

Table 8.124: SafetyStockHistoricalSupply (Mfg) fields

Field name	Description	Data type	Owner
OriginalLeadTime	The original lead time value for this date before applying any outlier adjustments. **Note:** This field was added in Version 2016.2, Service Update 5.	Integer	
Quantity	The amount of this historical supply.	Quantity	

Note: HistoricalSupplyActual records that have an undefined **OrderDate,** or an **OrderDate** set to "Past" or "Future", are ignored by the calculations that populate this table.

SafetyStockItemDemandParameterSet table

The **SafetyStockItemDemandParameterSet** table reports average historical demands, and standard deviation of those demands, as calculated for each safety stock item. The values reported in this table are those as parameters by the function that produces single-echelon safety stock recommendations and the analytic that produces multi-echelon safety stock recommendations.

The values shown in this table might be useful in understanding the safety stock calculations performed for a particular safety stock item. Note that if an item is configured for time-phased safety stock calculations, multiple records will be produced per item and identified by Index value.

Table 8.125: SafetyStockItemDemandParameterSet (Mfg) fields

Field name	Description	Data type	Owner
Average	The safety stock item's average historical demands in the period this record pertains to (only a single value is returned for items that aren't time-phased).	Quantity	
Index	If the safety stock item is configured for time-phased calculations, then this is a zero-based index value that refers to the specific cycle period that the parameters on this record apply to. For example, if **Item.Type.CycleCalendar** is set to "Year" and **Item.Type.PeriodCalendar** is set "Month", 0 refers to the current month, 1 refers to the next month, 2 to the next month after that, and so on (in other words, at the beginning of a calendar year, 0 would refer to January, 1 would refer to February, and so on). Otherwise, if the safety stock item is not configured for time-phased calculations, then this returns 0 and only a single record is reported for the item.	Integer	

Table 8.125: SafetyStockItemDemandParameterSet (Mfg) fields

Field name	Description	Data type	Owner
Item	A reference to the safety stock item these results pertain to. **Referenced table:** SafetyStockItem	Reference	Yes
StandardDeviation	Standard deviation of the safety stock item's historical demands in the period this record pertains to (only one value is reported for items that aren't time-phased).	Quantity	

SafetyStockItemFutureDemand table

The **SafetyStockItemFutureDemand** table reports details of actual and forecast demands (as taken from the **Demand** table) for parts configured as safety stock items. For each safety stock item, this table reports future demand data within the date range specified on the **SafetyStockItem** record, and bucketed by the calendar specified in the **SafetyStockItemType.IntervalsCalendar** field (by default, an everyday calendar is used).

This table takes into consideration the value from **SafetyStockItem.FutureIntervalCount**. In addition, the values reported in this table are used to calculate the days of supply quantities for single-echelon safety stock in the **SafetyStockResult** table, as well as calculating future demand for multi-echelon safety stock calculations and impacting the results reported in **SafetyStockTimePhasedResult.FutureReorderPoint**.

Worksheets based on this table are required as input to the **FutureDemands** parameter of the **SafetyStock** function. This function uses historical and future looking data to provide recommended safety stock and reorder points to meet a specified service level.

Note: If you mapped a safety stock item to use the values from a different part using the **SafetyStockItemMapping** table, the **SafetyStockItemFutureDemand** table reflects the results of the mapped values.

Table 8.126: SafetyStockItemFutureDemand (Mfg) fields

Field name	Description	Data type	Owner
Date	The date this bucketed future demand quantity applies to (demands are bucketed by the calendar referenced in **Item.Type.IntervalsCalendar** field). Any demands before the run date are bucketed at run date.	Date	
Item	A reference to the safety stock item. This identifies the specific part, and other details, associated with the future demands reported on this record. **Referenced table:** SafetyStockItem	Reference	Yes
Quantity	Total quantity of actual and forecast demand for this date.	Quantity	

SafetyStockItemOutlierResult table

Similarly to the **StatisticalForecastOutlier** table, the **SafetyStockItemOutlierResult** table reports details related to outliers in historical data for items configured for safety stock. For such safety stock items, there is one record created for each period of historical data used in generating the safety stock recommendations.

Each record in the **SafetyStockItemOutlierResult** table indicates whether a given historical data point represents an outlier in the data set, and contains other useful details such as the quantity of the demand in the period by which it should be adjusted for outliers (if any).

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Table 8.127: SafetyStockItemOutlierResult fields

Field name	Description	Data type	Owner
Date	The historical date to which the outlier data reported on this record pertains. Each date reported here belongs to the item's **Type.IntervalsCalendar**. For example, dates might mark the beginning of a weekly or monthly period.	Date	
ErrorComponent	If seasonal decomposition is used when processing outliers, this reports the error remaining after removing seasonal and trend components. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
ExistingOutlier Adjustment	The quantity for the outlier, if any, that has already been specified for the item and period in the **SafetyStockItemOutlier** table.	Quantity	
IsOutlier	Indicates whether the data in this period represents an outlier within the item's historical data set. Valid values are: • **N**—The data point is not an outlier • **Y**—The data point is an outlier	Boolean	
Item	The **SafetyStockItem** to which this outlier adjustment applies. **Referenced table**: SafetyStockItem	Reference	Yes
LowerThreshold	Indicates a value below which a data point for the item is considered an outlier.	Quantity	

Table 8.127: SafetyStockItemOutlierResult fields

Field name	Description	Data type	Owner
OutlierAdjustment	The amount that the original **Quantity** on the record should be adjusted by to account for outliers. This recommended adjustment is then reflected in the **SuggestedQuantity** field. If the record does not represent an outlier, than a value of "0" (zero) is returned.	Quantity	
ProcessingRule	The data series to which the outlier belongs. Valid values are: • **Demand**—Indicates that the outlier belongs to the historical demand data series. • **LeadTime**—Indicates that the outlier belongs to the lead time variability data series. This value is used for single-echelon only.	String (Enum)	
Quantity	Total demand quantity for the forecast item and demand category in this period. This includes historical actuals, causal factors, and any existing outlier quantity.	Quantity	
Seasonal Component	If seasonal decomposition is used when processing outliers, this reports the seasonal component. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
Suggested Quantity	Demand quantity for the item in this period after adjustment for any detected outliers, as represented by the sum of the **Quantity** and **OutlierAdjustment**. If the record does not represent an outlier, then this returns the same value as the **Quantity** field.	Quantity	
TrendComponent	If seasonal decomposition is used when processing outliers, this reports the trend component. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
UpperThreshold	Indicates a value above which a data point for the item is considered an outlier.	Quantity	

SafetyStockItemOutlierResultSummary table

The **SafetyStockItemOutlierResultSummary** table contains summary information related to outlier detection in historical data for items configured for safety stock recommendations. The type of information reported in this table is generally either a characteristic of the data used in outlier detection, or a message generated when calculating outliers.

This table was added in Version 2016.2, Service Update 5 to support the **Inventory Planning and Optimization** application.

Table 8.128: SafetyStockItemOutlierResultSummary fields

Field name	Description	Data type	Owner
Class	Indicates the type of data contained on this record. Valid values are: • **DataCharacteristic**—summary details about the historical data used when calculating outliers for the item. For example, the number of historical data points collected for the item. • **Message**—an error message generated during the process of calculating outliers. For example, if using methods that rely on the optionally configure R programming language, an error might be reported if a connection cannot be made to the R server.	String (Enum)	
Item	A reference to the **SafetyStockItem** to which the outliers apply. **Referenced table**: SafetyStockItem	Reference	Yes
ProcessingRule	The data series to which the outlier belongs. Valid values are: • **Demand** • **LeadTime** (This value is for single-echelon only).	String (Enum)	
ValueDate	Used to report summary date values (populated when **ValueType** is "Date"). For example, the mean date is reported in this field.	Date	
ValueName	The specific piece of information being reported on this record. The actual value associated with this summary data is reported in one of **ValueDate**, **ValueQuantity**, or **ValueText** (data type dependent). Valid values are: • **Count**—the number of historical data intervals over which data was collected for the item. • **Error**—an error message indicating an issue such as an invalid or missing parameter value, or an error connecting to the R Server. • **Mean**—average quantity calculated from the historical demand periods considered. • **MeanDate**—the approximate midpoint of the dates used in the calculations. • **PercentageZeroActuals**—percentage of historical data periods with a 0 (zero) quantity. • **Variance**—measures the overall variance of the historical data quantities from the mean.	String (Enum)	

Table 8.128: SafetyStockItemOutlierResultSummary fields

Field name	Description	Data type	Owner
ValueQuantity	Used to report summary quantity values on the data used to detect outliers (populated when **ValueType** is "Quantity"). For example, mean and variance data values are reported in this field. **Note:** These quantity values are calculated on the quantities as they exist before any adjustment for outliers.	Quantity	
ValueText	Used to report summary string values (populated when **ValueType** is "Text"). For example, any error messages are reported in this field.	String	
ValueType	Indicates the type of data reported for the value on this record, and hence which Value field is populated. Valid values are: • **Date**—the ValueDate field is populated. • **Text**—the ValueText field is populated. • **Quantity**—the ValueQuantity field is populated.	String (Enum)	

SafetyStockResult table

The **SafetyStockResult** table reports single safety stock recommendations based on historical data for both single-echelon and multi-echelon safety stock items (as well as other parts brought into multi-echelon families). Additionally, a number of calculated parameters used in determining safety stock are reported in this table.

Note that values reported in this table are expressed in terms of the **AnalyticConfiguration.MEIOReportCalendar**. Thus, calendar conversions may be applied to ensure calculated parameters and results are expressed in terms of this calendar (with single-echelon results being converted from **SafetyStockItemType.IntervalsCalendar** to **MEIOReportCalendar**, and multi-echelon results converted from the everyday calendar to **MEIOReportCalendar**.

If safety stock items have been configured to report time-phased safety stock values, or if recommended historical and/or future looking reorder points are required, the **SafetyStockTimePhasedResult** table can be used instead.

Table 8.129: SafetyStockResult (Mfg) fields

Field name	Description	Data type	Owner
AverageDemand	Average of historical demand (or average of historical forecast if **SafetyStockItemType. StandardDeviationDemandRule** is set to "ForecastError"). For time-phased single-echelon items, this reports the average of the seasonal values.	Quantity	
AverageFuture Demand	Average of future (current/forecast) demand. • For single-echelon items, this is the average of future demands collected within the item's average demand profile (or all future demands if no profile is specified). In addition, this field takes into consideration the value from **SafetyStockItem.FutureIntervalCount**. • For multi-echelon items this is based on demand bucketed by **MEIOReportCalendar** and **MEIOFutureIntervalCount** (both fields are from the **AnalyticConfiguration** table).	Quantity	
Calculated ServiceLevel	The expected service level percentage, calculated using bounded safety stock. If **SafetyStock** = **UnboundedSafetyStock**, this value is the same as **ServiceLevel**. A value of -1 indicates that the service level cannot be calculated using the current settings. If **SafetyStockItemType. TimePhasedProcessingRule** for the item is set to "DaysOfSupplyForward" or "DaysOfSupplyBackward," the value of this field is set to -1. It is also set to -1 if safety stock is allowed on artificial stages and there is safety stock on a customer-facing artificial stage. **Note:** this field was added in Version 2016.2.	Quantity	
Calendar ConversionRate	Applicable calendar conversion rate used for the part when reporting values in this table. For multi-echelon items, this is the ratio between the **Everyday** calendar and the **MEIOReportCalendar**. For single-echelon items, this is the ratio between the **SafetyStockItem.IntervalsCalendar** and the **MEIOReportCalendar**.	Quantity	
EarlyArrivalStock	Amount of early arrival stock for this part. Early arrival stock refers to replenishment orders that arrive at a location before the associated customer order has shipped. Early arrival stock can occur for multi-echelon parts with lead time variability when a stage's outbound service time exceeds its inbound service time. Multi-echelon optimization calculations then attempt to minimize the total holding cost of both safety stock and early arrival stock. **Note:** This field was added in Version 2016.2.	Quantity	

Table 8.129: SafetyStockResult (Mfg) fields

Field name	Description	Data type	Owner
Incoming ServiceTime	Maximum service time calculated across all stages/parts that are directly below this one in the multi-echelon family. This is the maximum number of days within which this stage will have its immediate demands satisfied (for example, from its component supplies). Expressed in terms of the **MEIOReportCalendar**. **Note:** This field was added in Version 2016.2.	Quantity	
IntervalsOf Supply	The number of **MEIOReportCalendar** intervals of demand that can be satisfied by the SafetyStock value reported on this record.	Quantity	
LeadTime	Lead time for the part (based on input values or calculated from historical supplies). Note that this value is expressed in **MEIOReportCalendar** intervals (converted from the Everyday calendar for multi-echelon items, or from the SafetyStockItem.Intervals calendar for single-echelon items).	Quantity	
MaximumDays OfSupply	The maximum level of safety stock, expressed as a number of days of supply. • If **SafetyStockItemType.TimePhased ProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "DaysOfSupply," the value specified in **SafetyStockTimePhasedBounds.MaximumDaysOfSupply** is reported here. • Otherwise, the value of this field is -1. When the value in this field is negative, it is interpreted as positive infinity. **Note:** This field was added in Version 2016.2	Quantity	
MaximumSafety Stock	The maximum level of safety stock, expressed as a quantity. • If **SafetyStockItemType.TimePhased ProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "Quantity," the value specified in **SafetyStockTimePhasedBounds.MaximumQuantity** for the part is reported here. • If **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "DaysOfSupply," the average daily demand is multiplied by **SafetyStockTimePhasedBoundsRule.MaximumDaysOfSuppl y,** and the result is reported here. • Otherwise, the value of this field is -1. When the value in this field is negative, it is interpreted as positive infinity. **Note:** This field was added in Version 2016.2	Quantity	

Table 8.129: SafetyStockResult (Mfg) fields

Field name	Description	Data type	Owner
MEIOFamily	A reference to the item at the head of the multi-echelon family this part belongs to (or the item itself if it is the head of the family). This reference is Null for single-echelon items. **Referenced table:** Part	Reference	
MEIOStage	A reference to the MEIOStage this part represents. This reference is Null for single-echelon items. **Referenced table:** MEIOStage	Reference	
MinimumDays OfSupply	The minimum level of safety stock, expressed as a number of days of supply. If **SafetyStockItemType.TimePhased ProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "DaysOfSupply," the value specified in **SafetyStockTimePhasedBounds.MinimumDaysOfSupply** is reported here. Otherwise, the value of this field is -1. When the value in this field is negative, it is interpreted as negative infinity. **Note:** This field was added in Version 2016.2.	Quantity	
MinimumSafety Stock	The minimum level of safety stock, expressed as a quantity. • If **SafetyStockItemType.TimePhased ProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "Quantity," the value specified in **SafetyStockTimePhasedBounds.MinimumQuantity** for the part is reported here. • If **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore" and **SafetyStockItemType.BoundsRule** is set to "DaysOfSupply," the average daily demand is multiplied by **SafetyStockTimePhasedBoundsRule.MinimumDaysOfSupply,** and the result is reported here. • Otherwise, the value of this field is -1. When the value in this field is negative, it is interpreted as negative infinity. **Note:** This field was added in Version 2016.2.	Quantity	

Table 8.129: SafetyStockResult (Mfg) fields

Field name	Description	Data type	Owner
Outgoing ServiceTime	Recommended service time for this stage in the family. This is the maximum number of days within which the part guarantees it can satisfy the demands placed on it. For multi-echelon safety stock items, this is set to an input value specified on the safety stock item (if the input service time provided is greater than the cumulative lead time for the part, that cumulative lead time value is used instead). For all parts below those ends items in a multi-echelon family, this is calculated with the aim of minimizing total inventory holding costs. Expressed in terms of the **MEIOReportCalendar**. **Note:** This field was added in Version 2016.2.	Quantity	
Part	A reference to the part whose safety stock parameters are being reported on this table. **Referenced table:** Part	Reference	Yes
PartSource	A reference to the part source providing the part this record applies to. **Referenced table:** PartSource	Reference	
ReportCalendar	A reference to the calendar whose intervals the values on this record are expressed in. This returns the same calendar as is referenced from **AnalyticsConfiguration.MEIOReportCalendar**. **Referenced table:** Calendar	Reference	
SafetyStock	Safety stock level calculated based on historical data, with any relevant bounds rules applied.	Quantity	
SafetyStockItem	A reference to the **SafetyStockItem** configuration record applicable to the single-echelon or multi-echelon item reported on this record. Null for lower-level parts brought into a multi-echelon family under a safety stock item. **Referenced table**: SafetyStockItem	Reference	
ServiceLevel	For a customer-facing item, this is the service level defined in the **SafetyStockItem.ServiceLevel** field. For lower level parts brought into a multi-echelon family under one or more customer-facing items, this value is calculated based on the service level and **StandardDeviationDemand** for customer-facing items, and the quantity of the lower level part needed per unit of the customer-facing items.	Quantity	

Table 8.129: SafetyStockResult (Mfg) fields

Field name	Description	Data type	Owner
Standard Deviation Demand	Calculated standard deviation of historical demand (or standard deviation of historical forecast error if **SafetyStockItemType.StandardDeviation DemandRule** is set to "ForecastError").	Quantity	
Standard Deviation LeadTime	Calculated standard deviation of historical supply lead time.	Quantity	
Unbounded SafetyStock	For single-echelon items, this is the safety stock level recommendation without any bounds applied. For multi-echelon items, the value in this field is the same as **SafetyStock** if a solution that satisfies the service level and safety stock bounds is found. If no such solution can be found, the safety stock quantity from the lowest cost iteration is reported. This is generally the iteration that violates safety stock bounds by the smallest amount, while ensuring that the service level is satisfied. **Note:** This field was added in Version 2016.2.	Quantity	

SafetyStockTimePhasedResult table

The **SafetyStockTimePhasedResult** table reports time-phased safety stock recommendations for both single-echelon and multi-echelon safety stock items configured to generate time-phased results (as well as for other parts brought into multi-echelon families under a safety stock item).

Table 8.130: SafetyStockTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
AverageDemand	Average of historical demand (or average of historical forecast if **SafetyStockItemType.StandardDeviationDemandRule** is set to "ForecastError"). If **SafetyStockItemType.TimePhasedProcessingRule** is set to "Use," this is the average demand over the part's net lead time for the period. Otherwise, it is expressed in terms of the average demand per unit of the Gregorian (everyday) calendar. **Note:** This field was added in Version 2016.2.	Quantity	
CalculatedService Level	The expected service level percentage, calculated using bounded safety stock. If **SafetyStock** = **UnboundedSafetyStock**, this value is the same as **ServiceLevel**. A value of -1 indicates that the service level cannot be calculated using the current settings. If **SafetyStockItemType. TimePhasedProcessingRule** for the item is set to "DaysOfSupplyForward" or "DaysOfSupplyBackward," the value of this field is set to -1. It is also set to -1 if safety stock is allowed on artificial stages and there is safety stock on a customer-facing artificial stage. **Note:** This field was added in Version 2016.2.	Quantity	
CurrentSafety Stock	Current safety stock level for the part as determined from Netting calculations. For example, this might represent an input value provided in the **TimePhasedSafety** table or a value calculated by range of coverage logic.	Quantity	
Date	The date from which this safety stock recommendation is applicable.	Date	
FutureDemand	Calculated future demand for the part.	Quantity	
FutureReorder Point	Suggested reorder point to maintain the recommended safety stock level (based on average future demands). For multi-echelon safety stock items, this quantity is impacted by the details reported in the **SafetyStockItemFutureDemand** table.	Quantity	
HistoricalReorder Point	Suggested reorder point to maintain the recommended safety stock level (based on average historical demands).	Quantity	

Table 8.130: SafetyStockTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
MaximumDaysOf Supply	The maximum level of safety stock, expressed as a number of days of supply. If **SafetyStockItemType.BoundsRule** (for the head part) is set to "DaysOfSupply" and **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore," "DaysOfSupplyBackward," or "DaysOfSupplyForward," this field's value is equal to **SafetyStockTimePhasedBounds.MaximumDaysOfSupply** for this part. Otherwise, this field's value is -1. When the value in this field is negative, it is interpreted as positive infinity. **Note:** This field was added in Version 2016.2.	Quantity	
MaximumSafety Stock	The maximum level of safety stock, expressed as a quantity. If **SafetyStockItemType.BoundsRule** (for the head part) is set to "Quantity" and **SafetyStockItemType.TimePhasedProcessingRule** is set to "Use," this field's value is equal to **SafetyStockTimePhasedBounds.MaximumQuantity** for this part. If **SafetyStockItemType.BoundsRule** (for the head part) is set to "DaysOfSupply" and **SafetyStockItemType.TimephasedProcessingRule** is set to "Ignore," this field's value is equal to **AverageDemand** * **MaximumDaysOfSupply**. Otherwise, this field's value is -1. When the value in this field is negative, it is interpreted as positive infinity. **Note:** This field was added in Version 2016.2.	Quantity	
MinimumDaysOf Supply	The minimum level of safety stock, expressed as a number of days of supply. If **SafetyStockItemType.BoundsRule** is set to "DaysOfSupply" and **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore," "DaysOfSupplyBackward," or "DaysOfSupplyForward," this field's value is equal to **SafetyStockTimePhasedBounds.MinimumDaysOfSupply** for this part. Otherwise, this field's value is -1. When the value in this field is negative, it is interpreted as negative infinity. **Note:** This field was added in Version 2016.2	Quantity	

Table 8.130: SafetyStockTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
MinimumSafety Stock	The minimum level of safety stock, expressed as a quantity. If **SafetyStockItemType.BoundsRule** is set to "Quantity" and **SafetyStockItemType.TimePhasedProcessingRule** is set to "Use," this field's value is the value specified in **SafetyStockTimePhasedBounds.MinimumQuantity** for this part. If **SafetyStockItemType.BoundsRule** (for the head part) is set to "DaysOfSupply" and **SafetyStockItemType.TimePhasedProcessingRule** is set to "Ignore," this field's value is equal to **AverageDemand** * **MinimumDaysOfSupply**. Otherwise, this field's value is -1. When the value in this field is negative, it is interpreted as negative infinity. **Note:** This field was added in Version 2016.2.	Quantity	
Part	The part the safety stock recommendation on this record applies to. **Referenced table:** Part	Reference	Yes
SafetyStock	Recommended safety stock level for the part on this date as determined from either single-echelon or multi-echelon safety stock calculations. Any relevant bounds are applied. For single-echelon parts, this value is calculated as described in "Time-phased safety stock" on page 1977. For multi-echelon parts, this is the value calculated for the part's stage within the multi-echelon family (the result is as reported in the **MEIOStageTimePhasedResult** table).	Quantity	
SafetyStock Result	A reference to the **SafetyStockResult** record applicable to the part whose time-phased safety stock recommendation is being reported on this record. **Referenced table:** SafetyStockResult	Reference	
ServiceLevel	For a customer-facing item, this is the service level defined in the **SafetyStockItem.ServiceLevel** field. For lower level parts brought into a multi-echelon family under one or more customer-facing items, this value is calculated based on the service level and standard deviation of demand over net lead time for customer-facing items, and the quantity of the lower level part needed per unit of the customer-facing items. **Note:** this field was added in Version 2016.2	Quantity	

Table 8.130: SafetyStockTimePhasedResult (Mfg) fields

Field name	Description	Data type	Owner
Standard Deviation Demand	Calculated standard deviation of historical demand (or standard deviation of historical forecast error if **SafetyStockItemType.StandardDeviation DemandRule** is set to "ForecastError"). If SafetyStockItemType.TimePhasedProcessingRule is set to "Use," this is the standard deviation over net lead time. **Note:** This field was added in Version 2016.2.	Quantity	
Unbounded SafetyStock	For single-echelon items, this is the safety stock level recommendation without any bounds applied. For multi-echelon items, the value in this field is the same as **SafetyStock** if a solution that satisfies the service level and safety stock bounds is found. If no such solution can be found, the safety stock quantity from the lowest cost iteration is reported. This is generally the iteration that violates safety stock bounds by the smallest amount, while ensuring that the service level is satisfied. **Note:** This field was added in Version 2016.2.	Quantity	

ScheduledReceiptCost table

The **ScheduledReceiptCost** table reports date-specific cost of goods sold values from all component parts pegged to a particular scheduled receipt in the **WhereConsumedForSupply** table. For a given **IndependentDemand** record, this table reports cost of goods sold and material spend values for each date on which one or component supplies are needed to complete the production order.

To report total rolled up cost data for each scheduled receipt, the **CostOfGoodsSold** on the **ScheduledReceipt** table can be used. This calculated field summarize the values reported in this table and thereby ensure that all date-sensitive exchange rates that might apply during the completion of a production order are taken into account by RapidResponse during currency conversion calculations.

Table 8.131: ScheduledReceiptCost (Mfg) fields

Field name	Description	Data type	Owner
ConversionDate	The particular date to which the cost values reported on this record apply. Each receipt has one record added to this table for each unique date on which one or more component supplies are needed in order to complete the order. This date is then used as the effective date for any associated currency conversion calculations, and ensures the appropriate date-based rates are reflected on the rolled up cost values as reported in the **ScheduledReceipt** table.	Date	
CostOfGoods	The total cost of goods from all component supplies needed on this date in order to fill the referenced scheduled receipt For a given need date and set of component supplies pegged to this production order in the **WhereConsumedForSupply** table, the **LaborCost**, **MaterialCost**, **OverheadCost**, and **OverheadCost2** are added together to populate this field.	Money	
ScheduledReceipt	A reference to the scheduled receipt to which the cost values reported on this record apply. **Referenced table:** ScheduledReceipt	Reference	Yes

StatisticalForecast table

The **StatisticalForecast** table reports the result of the Predict statistical function calculations as future dated quantities. These calculations are based on the statistical model parameters and constants contained in the **StatisticalForecastFitOutput** table. If quantities are reported in the **CausalFactorDetail** and **ForecastItemParametersOutlier** tables, they are taken into account in the statistical forecast calculations.

Table 8.132: StatisticalForecast (Mfg) fields

Field name	Description	Data type	Owner
Date	The date the forecast value is calculated for.	Date	
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes

Table 8.132: StatisticalForecast (Mfg) fields

Field name	Description	Data type	Owner
Prediction IntervalLower	The lower limit for prediction intervals. This field is calculated only if the forecast was generated with **ForecastItemParameters.ConfidenceLevel** set to a positive value. In all other cases, a value of -1 is returned. In addition, the forecast must be generated using one of the following models: • ARIMA • ARIMAX • Double exponential smoothing • Exponential smoothing • Holt-Winters (multiplicative and additive) • Linear • Multiple Linear Regression • Rforecast • Step-wise ARIMA **Note:** The capability to calculate **Prediction IntervalLower** with statistical forecasts that are generated using the Linear model was added in Version 2016.2 Service Update 15. ARIMA and Step-Wise ARIMA were added in Version 2016.2 Service Update 16. ARIMAX and Multiple Linear Regression were added in Version 2016.2 Service Update 27.	Quantity	
Prediction IntervalUpper	The upper limit for prediction intervals. This field is calculated only if the forecast was generated with **ForecastItemParameters.ConfidenceLevel** set to a positive value. In all other cases, a value of -1 is returned. In addition, the forecast must be generated using one of the following models: • ARIMA • ARIMAX • Double exponential smoothing • Exponential smoothing • Holt-Winters (multiplicative and additive) • Linear • Multiple Linear Regression • Rforecast • Step-wise ARIMA **Note:** The capability to calculate **PredictionIntervalUpper** with statistical forecasts that are generated using the Linear model was added in Version 2016.2 Service Update 15. ARIMA and Step-Wise ARIMA were added in Version 2016.2 Service Update 16. ARIMAX and Multiple Linear Regression were added in Version 2016.2 Service Update 27.	Quantity	

Table 8.132: StatisticalForecast (Mfg) fields

Field name	Description	Data type	Owner
Quantity	The forecast quantity or money value calculated for a period.	Quantity	

StatisticalForecastDetail table

The **StatisticalForecastDetail** table holds current statistical forecast data at the detail level (data that has been disaggregated to the disaggregation calendar level for each part customer). Each record pertains to a given part, customer, and category combination, and shows details such as the forecast quantity and date. The values shown in this table are based on aggregate forecast values produced by the **StatisticalForecast** table. This table also reflects any quantities that are reported in the **CausalFactorDetail** and **ForecastItemParametersOutlier** tables.

Table 8.133: StatisticalForecastDetail (Mfg) fields

Field name	Description	Data type	Owner
Date	The forecast point date in the disaggregation calendar.	Date	
Header	A reference to the part customer and category combination this set of values applies to. **Referenced table:** HistoricalDemandHeader	Reference	
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes
PartCustomer	A reference to the part and customer combination the disaggregation applies to. **Referenced table:** PartCustomer	Reference	
Quantity	The forecast point quantity.	Quantity	

StatisticalForecastDisaggregationRate table

The **StatisticalForecastDisaggregationRate** table contains the calculated statistical forecast disaggregation rates. Disaggregation rates are determined by the settings in the **SOPAnalyticsConfiguration** table. This table also reflects any quantities that are reported in the **CausalFactorDetail** table.

Table 8.134: StatisticalForecastDisaggregationRate (Mfg) fields

Field name	Description	Data type	Owner
BaseCalendar Date	The date in the disaggregation calendar, which defines the periods that orders can be disaggregated into.	Date	
Date	The value representing the interval period of bucketed actuals, specified by **ForecastItemParametersType.IntervalsCalendar**. Typically this is set to a weekly or monthly calendar.	Date	
Header	A reference to the part customer and forecast category combination this set of values applies to. **Referenced table:** HistoricalDemandHeader	Reference	
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes
PartCustomer	A reference to the part and customer combination the disaggregation applies to. **Referenced table:** PartCustomer	Reference	
Quantity	The rate for disaggregation.	Quantity	

StatisticalForecastFitOutput table

The **StatisticalForecastFitOutput** table identifies the statistical model parameters used to calculate the statistical forecast, and calculates various statistical data characteristics and error measures of the statistical model. The data stored in this table is intended to be used to produce the data in the **StatisticalForecast** table and the **StatisticalForecastPredictActual** table.

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
BestFitProfile Detail	A reference to the detailed profile, and associated parameters, used in the process of fitting and selecting the best forecast model. This reference is populated on records where **Class** is "BestFit", and a best fit profile was used in selecting the model. **Referenced table:** BestFitProfileDetail	Reference	
Class	The type or category of forecast parameter being reported in the record. The name of the parameter is reported in the **ValueName** field for all classes except Message. Valid values are: • **BestFit**—the statistical model with the least margin of error. This field reports "BestFit" when **ForecastItemParametersType.ForecastModel** is set to "BestFit". The three models with the least margin of error are reported in the **ValueName** field. • **DataCharacteristic**—a characteristic of the input data points. • **ErrorMeasure**—an error measure of fitted values against the input data points. • **HoldoutPeriodErrorMeasure**—an error measure based on comparing simulated forecast values (generated by fitted models) against historical actuals in the holdout period. Applicable only when using best fit a holdout period (the **HoldoutPeriodUsageRule** is set to a value other than "Ignore") and the selected **FitMeasure** is either "LagPercentageError" or "RollingLagPercentageError". • **Message**—a message type generated from the statistical models. The message is available in the **ValueText** field. • **ModelConstant**—the constant values used by the forecast model. • **ModelOutput**—the output from the statistical model that will be used to generate forecast. • **StepWiseARIMA**—the ARIMA model with the least margin of error. This field reports "StepWiseARIMA" when **ForecastItemParametersType.ForecastModel** is set to "StepWiseARIMA". The three models with the least margin of error are reported in the **ValueName** field.	String (Enum)	

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
ForecastModel	The statistical model that will be used to calculate the statistical forecast. For more detailed information about the forecast models supported by RapidResponse, see "Statistical forecasting models" on page 2031. Valid values are: • **AdditiveHoltWintersMethod**—differs from the Holt-Winters model in that forecast points are calculated in an additive, rather than multiplicative, manner. Suitable for demand that has constant variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **ARIMA**—calculates forecasts for a time period using a linear combination of actual values and error values in a time series. This model can be used on a stationary time series, or a differencing function can be applied to non-stationary data to make it stationary. For a more detailed description, see "Auto-Regressive Integrated Moving Average (ARIMA)" on page 2034. • **ARIMAX**—Is an extension of the ARIMA model, but also includes independent variables (regressors) in the forecast calculation. This makes ARIMAX a multiple linear regression model. To improve forecast accuracy, ARIMAX also takes into consideration autocorrelation that may be found in regressor residuals. For a more detailed description, see "ARIMAX" on page 2042. • **BestFit**—fits each of the other forecast models and selects the one with the least margin of error. For a more detailed description, see "Best fit" on page 2053. • **CrostonsMethod**— uses exponential smoothing with a single smoothing constant for the forecast values, but calculates two different values; the first being the forecast values at a point, and the second being the average length of time between forecasts. These values determine the forecast quantities and the periods that have forecast demands. Suitable for demand that is not constant, such as lumpy demand that contains periods of zero quantities. For a more detailed description, see "Croston's method" on page 2034. • **DoubleExponentialSmoothing**—considers the data points and trends within the historical data, and applies a smoothing algorithm to both. The result is then used as the statistical forecast. Suitable for demand that is fairly stable but trending up or down over time. For a more detailed description, see "Double exponential smoothing" on page 2036. • **ElasticNet**—A multiple linear regression model that uses regularized, also known as penalized, regression methods. It blends Lasso and ridge regression	String (Enum)	

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
	techniques by penalizing both L1 and L2 norms to reduce the coefficient vector. To use ElasticNet, you must attach a minimum of two regressors to a forecast item or you won't be able to generate a statistical forecast. For a more detailed description, see "ElasticNet" on page 2043. • **ExponentialSmoothing**— considers data points within the historical data, and applies a smoothing algorithm. The result of the final data point is then used as a constant value for the statistical forecast. Suitable for demand that is fairly stable and lacks a trend. For a more detailed description, see "Exponential smoothing" on page 2037. • **ForecastImport**—the statistical forecast is set to values in the ImportedStatisticalForecast table which is meant to hold statistical forecast values that were generated using an external tool. Once imported, can then be used by post-processing forecast logic in RapidResponse (for example, applying NPI/EOL profiles or disaggregating to the detailed part-customer level). Errors are reported for this model if there aren't any forecast values provided in the expected date range. Warnings are reported for this model if there are forecast values provided but the date is not on the item's IntervalsCalendar (these are rounded to the previous date on the IntervalsCalendar) or if there are some dates on the item's IntervalsCalendar for which there are no forecast values provided in the expected date range (after any applicable date rounding). For a more detailed description, see "ForecastImport" on page 2058. • **HoltWintersMethod**—similar to double exponential smoothing in that it considers data points and trends within the historical data, but also accounts for seasonal trends. That is, the values for each point in the forecast and trend are further adjusted to account for seasonal demand changes. This model is the Holt-Winters multiplicative method, so it is suitable for demand that has proportional variation over a period. For a more detailed description, see "Holt-Winters multiplicative and additive" on page 2037. • **Linear**—performs a linear regression on a set of historical data points to generate a trend line that determines statistical forecast quantities. Suitable for demand that does not fluctuate. For a more detailed description, see "Linear" on page 2039. • **MovingAverage**—calculates the simple moving average of a specified number of previous data points to		

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
	generate a constant forecast. Suitable for demand that is fairly stable and does not change much over time. For a more detailed description, see "Moving average" on page 2039. • **MultipleLinearRegression**—Performs a linear regression using multiple independent variables to predict the outcome of one dependent variable. For a more detailed description, see "Multiple linear regression" on page 2043. • **StepWiseARIMA**—identical to the ARIMA model, but all parameters are calculated automatically. Step-wise ARIMA selects the order of difference through KPSS unit-root tests and selects AR order and MA order by minimizing AIC. For a more detailed description, see "Step-wise ARIMA" on page 2039.		
Index	An index value associated with this record. This value is incremented by one for each record added to the table.	Integer	
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes
ValueDate	The forecast parameter **MeanDate**, a data characteristic, is expressed as a date. This field is populated if the value in the **ValueType** field is "Date".	Date	

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
ValueName	The name of the forecast parameter. The **Class** field provides the category of the forecast parameter, then the **ValueName** field provides the specific parameter being reported, so various names could be reported for a given class. For example, if "ModelConstant" is the class of the parameter, the name could be "Alpha", indicating that the forecast model uses the alpha constant when performing its calculations (alpha's quantity is then reported in the **ValueQuantity** field). The parameter names that belong to the following classes are reported in this field and described below: • "Best fit" on page 1187 • "Data characteristics" on page 1187 • "Error measures" on page 1188 • "HoldoutPeriodErrorMeasure" on page 1189 • "Model constants" on page 1190 • "Model outputs" on page 1190 • "Step-Wise ARIMA" on page 1192 • "Message" on page 1192 ## Best fit • **ErrorMeasure**—the error measure used to determine the best fit. • The names of the three statistical models with the least margin of error (based on the selected measure). The **ValueText** field indicates the ranking of those models, and the **ValueQuantity** field indicates the calculated error measure. ## Data characteristics • **Count**—reports the number of values in your input set. • **MeanDate**—determines the average of all dates included in the statistical forecast, and the approximate midpoint of the dates used in the statistical calculations. • **MeanQuantity**—determines the average of all actual values in the input data. • **StandardErrorOfY**—determines how values differ from the mean. • **TotalSumSquares**—determines the total difference between each data point and the mean. • **Variance**—determines how data values differ from the mean. For more information about data characteristics, see "Data	String (Enum)	

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner

characteristics" on page 2059.

Error measures

- **AdjustedRSquared**—determines the ratio between the regression sum of squares and total sum of squares measures, but is influenced by the number of values in the input data set.
- **AkaikeInformationCriterion (AIC)**—determines how well a statistical model fits the values it was generated from using the log-likelihood result and double the number of estimated parameters. This measure is reported only if the ARIMA or Step-wise ARIMA models are used.
- **ForecastError**—determines forecast accuracy by showing the adjusted error of the predicted actuals compared to the historical actuals.
- **Hannan-QuinnCriterion (HQC)**—determines how well a statistical model fits the values it was generated from, using the log-likelihood result and the product of double the number of parameters and the logarithm of the logarithm of the number of observed samples. This measure is reported only if the ARIMA or Step-wise ARIMA models are used.
- **LogLikelihood**—determines the maximum likelihood for a statistical function. This measure is reported only if the ARIMA or Step-wise ARIMA models are used.
- **MeanAbsoluteError**—determines the absolute value of the variance between the historical actual values and the predicted forecast values.
- **MeanAbsoluteErrorByMean**—normalizes the value returned by the mean absolute error measure by dividing it by the mean quantity of the input data.
- **MeanAbsolutePercentageError**—determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage.
- **MeanAbsolutePercentageErrorByForecast**—determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage.
- **MeanError**—determines the average variance between the historical actual values and the predicted forecast values.
- **MeanPercentageError**—determines the average variance between the historical actual values and the

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner

predicted forecast values with respect to the actual values, expressed as a percentage.

- **MeanPercentageErrorByForecast**—determines the average variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage.
- **MeanSquareErrors**—calculates the average of the residual sum of squares for the forecast.
- **RegressionSumSquares**—determines the difference between the total sum of squares of the input data and the residual sum of squares, and shows how the variance of the statistical forecast and input values differs from the variance of the input values and average of input values.
- **ResidualSumSquares**—calculates the sum of the square of differences between the historical actual values and the values predicted by the statistical forecast.
- **RootMeanSquareErrors**—determines the square root of the value produced by the mean square errors measure, and shows the correlation between the historical actual values and the results of the statistical forecast calculation.
- **RSquared**—determines the ratio between the regression sum of squares and total sum of squares measures, and shows how closely the forecast values fit the historical actual values.
- **SchwarzBayesianInformationCriterion (SBC)**—determines how well a statistical model fits the values it was generated from using the log-likelihood result and the product of the number of parameters and logarithm of the number of observed samples. This measure is reported only if the ARIMA or Step-wise ARIMA models are used.
- **VarianceError**—determines if the forecast model produces forecasts with more stable forecast errors or unstable, varying forecast errors by showing how widely the errors are spread around the mean error.

For more information about error measures, see "Error measures" on page 2060.

HoldoutPeriodErrorMeasure

The following are reported when the Class is "HoldoutPeriodErrorMeasure":

- **LagActual**—sum of actuals within just the first interval of the holdout period used for measuring forecast error (after accounting for any offset).

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
	• **LagError**—sum of differences between the actuals and simulated forecast values in the first interval of the holdout period. • **LagPercentageError**—sum of differences between the actuals and forecast in the first interval of the holdout period, expressed as a percentage of all actuals in that interval. • **Mu**—used only for ARIMA, if a constant is included in the calculation. • **Phi**—used only for ARIMA, and must be provided for each auto-regressive and moving average lag included in the ARIMA calculation. • **RollingErrorIntervalCount**—sum of field in the **ForecastItemParameters** table, and is applied from the start of the holdout period (after accounting for any offset). • **RollingLagActual**—sum of actuals across the portion of the holdout period being used to measure forecast error. The number of intervals is set by the • **RollingLagError**—sum of differences between the actuals and simulated forecast values in the portion of the holdout period being considered. • **RollingLagPercentageError**—sum of differences between the actuals and simulated forecast values in the intervals being considered within the holdout period, expressed as a percentage of all actuals in those intervals. • **Theta**—used only for ARIMA, and must be provided for each auto-regressive and moving average lag included in the ARIMA calculation. ## Model constants • **Alpha**—a smoothing constant, required for exponential smoothing, double exponential smoothing, Holt-Winters, additive Holt-Winters, and Croston's method. • **Beta**—a seasonal trend smoothing constant, required for Holt-Winters and additive Holt-Winters. • **Gamma**—a trend smoothing constant, required for double exponential smoothing. ## Model outputs • **AutoRegressiveTerms**—the auto-regressive values considered for the ARIMA calculation. • **DifferenceLevel**—determines how many differencing functions are performed on the data when using the		

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
	ARIMA or ARIMAX model. If you are using an auto-selected difference level with ARIMAX, the auto-selected value is output here. • **FitError**—any error encountered during the model fit process. Can be either "None", "Error", or "CriticalError". The error message is displayed in the **ValueText** field with "Message" in the **Class** field. • **Forecast**—the calculated forecast quantity reported across all forecast points when using exponential smoothing, moving average, or Croston's method. • **IncludeConstant**—determines whether a constant value is included in the ARIMA and ARIMAX calculations. • **InterarrivalValue**—the final estimated inter-arrival time between forecasts, which is the unrounded SporadicInterval, when using the Croston's method. • **KPSS_UnitRootTest_(i)**—the calculated difference level used in the Step-wise ARIMA calculation. • **MovingAverageIntervalCount**—the specified number of historical periods used to calculate the moving average. • **MovingAverageTerms**—the moving average values considered for the ARIMA calculation. • **RegularizationParameter**—A parameter, also known as lambda, that controls the strength of the penalty term for coefficients that is used in Elastic Net regression. If you are using an auto-selected regularization parameter, the auto-selected value is output here. • **Root_AR_(i)**—the (i) root of the auto-regressive terms of the ARIMA calculation. • **Root_AR_(i)_Modulus**—the modulus of (i) root of the auto-regressive terms of the ARIMA calculation. • **Root_MA_(i)**—the (i) root of the moving average terms of the ARIMA calculation. • **Root_MA_(i)_Modulus**—the modulus of (i) root of the moving average terms of the ARIMA calculation. • **SeasonalIntervalCount**—the duration of a season, based on the intervals calendar; used for Elastic Net, Holt-Winters, additive Holt-Winters, and Multiple Linear Regression. If you are using an auto-selected seasonal interval count, the auto-selected value is output here. • **SeasonalityValue_(i)**—the value calculated for each season; used for Holt-Winters and additive Holt-Winters. • **SmoothedValue**—the value required for statistical models that use smoothing, namely exponential smoothing, double exponential smoothing, Holt-		

Table 8.135: StatisticalForecastFitOutput (Mfg) fields

Field name	Description	Data type	Owner
	Winters and additive Holt-Winters. For the Croston's method, it is the estimated non-zero demand at every InterarrivalValue. • **TrendValue**—the value used to represent a trend, used for double exponential smoothing, Holt-Winters, and additive Holt-Winters. **Note:** The **Interarrival** model output was added and the **SmoothedValue** was modified in Version 2016.2, Service Update 15. ## Step-Wise ARIMA • The names of the three ARIMA models with the least margin of error. ## Message • **Method**—If the optional Rforecast model was used, this provides a three-character string describing the model by error, trend, and season type (for example, MAM). • **RConnectionError**—This error may display if the optional R forecast capability has been enabled, and a connection could not be established with the R Server.		
ValueQuantity	Error measures, model constants, model outputs, and most data characteristics are expressed as quantities. This field is populated if the value in the **ValueType** field is "Quantity".	Quantity	
ValueText	Messages or arrays needed for ARIMA are expressed as strings. This field is populated if the value in the **ValueType** field is "Text".	String	
ValueType	The data type of the forecast parameter. The data types are: • Text • Quantity • Date	String (Enum)	

StatisticalForecastOutlier table

The **StatisticalForecastOutlier** table reports details related to outliers in historical data for items configured for statistical forecasting. For such forecast items, there is one record generated for each period of historical data used in generating the statistical forecast (as determined by the **ForecastItemParameters.HistoricalIntervalCount** setting).

Each record in the **StatisticalForecastOutlier** table indicates whether a given historical data point represents an outlier in the data set, and contains other useful details such as the quantity of demand in the period and the amount by which it should be adjusted for outliers (if any).

Note that this table is only populated for forecast items with a valid **ForecastItemParameters.OutlierType** reference. If the item's **ForecastItemParameters.OutlierType** reference is"Null", then the item will not have records generated in this table.

Table 8.136: StatisticalForecastOutlier (Mfg) fields

Field name	Description	Data type	Owner
Date	The historical date that the outlier data reported on this record pertains to. Each date reported here belongs to the item's **Type.IntervalsCalendar**. For example, dates might mark the beginning of a weekly or monthly period.	Date	
ErrorComponent	If seasonal decomposition is used when processing outliers, this reports the error remaining after removing seasonal and trend components. Applicable when **OutlierType.DataRule i**s set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
ExistingOutlier Quantity	The outlier quantity, if any, that has already been specified for the item and period in the **ForecastItemParametersOutlier** table. Only those **ForecastItemParametersOutlier** records where the **ProcessingRule** is set to "All" are reported and used in this table.	Quantity	
Forecast	Represents a value that can be used to replace detected outliers for items that reference an **OutlierType** record with a setting of "Forecast" in the **AboveThreshold** and/or **BelowThreshold** fields. The value calculated here is dependent on the item's **OutlierType.DataRule** setting as follows: • If **DataRule** = "MovingAverageError", this returns the calculated moving average in the item's history up to the period where the outlier was detected. • If **DataRule** is set to "RstlError", this returns the sum of the **TrendComponent** and **SeasonalComponent** values on this record. • If **DataRule** is set to "Historical", this field is not applicable and always returns a value of -1 (and the mean value will be used instead).	Quantity	
IsOutlier	Indicates if the data in this period represents an outlier within the item's historical data set. Valid values are: • **N**—the data point is not an outlier. • **Y**—the data point is an outlier.	Boolean	

Table 8.136: StatisticalForecastOutlier (Mfg) fields

Field name	Description	Data type	Owner
ItemParameters	A reference to the forecast item and its associated parameters (for example, the demand category to use for collecting history and the number of intervals over which to collect that data). **Referenced table:** ForecastItemParameters	Reference	
LowerThreshold	Indicates a value below which a data point for the item is considered an outlier.	Quantity	
Outlier Adjustment	The amount the original **Quantity** on the record should be adjusted by to account for outliers. This recommended adjustment is then reflected in the **SuggestedQuantity** field. If the record does not represent an outlier, than a value of 0 (zero) is returned.	Quantity	
Quantity	Total demand quantity for the forecast item and demand category in this period. This includes historical actuals, causal factors, and any existing outlier quantity.	Quantity	
Seasonal Component	If seasonal decomposition is used when processing outliers, this reports the seasonal component. Applicable when **OutlierType.DataRule** is set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
Suggested Quantity	Demand quantity for the item in this period after adjustment for any detected outliers. If the record does not represent an outlier, then this returns the same value as the **Quantity** field.	Quantity	
Trend Component	If seasonal decomposition is used when processing outliers, this reports the trend component. Applicable when **OutlierType.DataRule i**s set to "RstlError". Otherwise, a value of -1 is reported.	Quantity	
UpperThreshold	Indicates a value above which a data point for the item is considered an outlier.	Quantity	

StatisticalForecastOutlierSummary table

The **StatisticalForecastOutlierSummary** table contains summary information related to outlier detection in historical data for items configured for statistical forecasting. The type of information reported in this table is generally either a characteristic of the data used in outlier detection, or a message generated when calculating outliers.

Note that this table is only populated for forecast items with a valid **ForecastItemParameters.OutlierType** reference. If the item's **ForecastItemParameters.OutlierType** reference resolves to "Null", then the item will not have records generated in this table.

Table 8.137: StatisticalForecastOutlierSummary (Mfg) fields

Field name	Description	Data type	Owner
Class	Indicates the type of data contained on this record. Valid values are: • **DataCharacteristic**—summary details about the historical data used when calculating outliers for the item. For example, the number of historical data points collected for the item. • **Message**—an error message generated during the process of calculating outliers. For example, if using methods that rely on the optionally configured R programming language, an error might be reported if a connection cannot be made to the R Server.	String (Enum)	
ItemParameters	A reference to the forecast item and its associated parameters (for example, the demand category to use for collecting historical data and the number of intervals over which to collect that data). **Referenced table:** ForecastItemParameters	Reference	
ValueDate	Used to report summary date values, For example, the mean date is reported in this field	Date	
ValueName	The specific piece of information being reported on this record. The actual value associated with this summary data is reported in one of **ValueDate**, **ValueQuantity**, or **ValueString** (data type dependent). Valid values are: • **Count**—the number of historical data intervals over which data was collected for the item. • **Error**—an error message indicating an issue such as an invalid or missing parameter value, or an error connecting to the R Server. • **Mean**—average quantity calculated from the historical demand periods considered. • **MeanDate**—the approximate midpoint of the dates used in the calculation. • **PercentageZeroActuals**—percentage of historical data periods with a 0 (zero) quantity. • **Variance**—measures the overall variance of the historical data quantities from the mean.	String (Enum)	

Table 8.137: StatisticalForecastOutlierSummary (Mfg) fields

Field name	Description	Data type	Owner
ValueQuantity	Used to report summary quantity values on the data used to detect outliers (populated when **ValueType** is "Quantity"). For example, mean and variance data values are reported in this field. Note that these quantity values are calculated on the quantities as they exist before any adjustment for outliers.	Quantity	
ValueText	Used to report summary string values (populated when **ValueType** is "Text"). For example, any error messages are reported in this field.	String	
ValueType	Indicates the type of data reported for the value on this record, and hence which Value field is populated. Valid values are: • **Date**—The **ValueDate** field is populated. • **Text**—The **ValueText** field is populated. • **Quantity**—The **ValueQuantity** field is populated.	String (Enum)	

StatisticalForecastPredictActual table

The **StatisticalForecastPredictActual** table fits a set of historical actual demands to the parameters calculated for the statistical forecast, allowing you to measure how well the forecast method fits the points upon which it was based. In other words, the **StatisticalForecastPredictActual** table allows you to measure how well the model fits the actual data and does not necessarily find errors in the forecast. The calculations in the **StatisticalForecastPredictActual** table are based on the statistical model parameters and constants produced by the **StatisticalForecastFitOutput** table.

Table 8.138: StatisticalForecastPredictActual (Mfg) fields

Field name	Description	Data type	Owner
ActualQuantity	The historical actual quantity.	Quantity	
Date	The date the historical actual value is adjusted for.	Date	
ItemParameters	A reference to the input parameters used in generating the statistical forecast for a given forecast item. **Referenced table:** ForecastItemParameters	Reference	Yes
Quantity	The fitted or predict actual quantity.	Quantity	

StatisticalForecastRegressionCoefficient table

The **StatisticalForecastRegressionCoefficient** table contains the results of calculations of the coefficients for forecast items that use a multiple linear regression model to create a statistical forecast. The coefficients help determine how effective regressors, also known as independent variables, are at predicting future demand. If **ForecastItemParametersType.ModelConstantUsage** is set to **Use** and there is a **ForecastItemParametersRegressionCoefficient** record for a regressor, the quantities in this table are copied from the **ForecastItemParametersRegressionCoefficient** table. If **ForecastItemParametersType.ModelConstantUsage** is set to **Calculate**, the quantities in this table are calculated by R.

The reporting of the coefficient results also reflect the following:

- Values in the **ForecastItemParameters.WeightsDecayFactor**.
- Values in the **ForecastItemParameters.RegularizationParameter**.
- Values in the **ForecastItemParameters.ElasticNetParameter**.
- Trend and seasonality as reported by R.

Note: Depending on the forecast model that you choose, not all properties of the regression coefficients are reported. For example, if you are using the Elastic Net model, only the **Estimate** is reported. If you are using ARIMAX, only **Estimate** and **StandardErrors** are reported.

The **StatisticalForecastRegressionCoefficient** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 8.139: StatisticalForecastParametersRegressionCoefficient (Mfg) fields

Field name	Description	Data type	Key
Estimate	A value that indicates the direction and magnitude of the relationship between the regressors and demand. It represents the mean change in the regressors' response given one unit change in the demand. • A positive estimate indicates that an increase of the regressors results in an increase in demand. • A negative estimate indicates that an increase of the regressors results in a decrease in demand.	Quantity	
ItemParameters	A reference to the forecast item parameters that are associated with the coefficients. **Referenced table**: ForecastItemParameters	Reference	
Name	The unique name that is used to identify the model term that is associated with the coefficients.	String	

Table 8.139: StatisticalForecastParametersRegressionCoefficient (Mfg) fields

Field name	Description	Data type	Key
pValue	A value that indicates the statistical significance of the correlation between demand and the regressor. It reflects the probability that the observed relationship between demand and the regressors in the historical data would hold true were they not correlated. • A low p-value indicates that there is a correlation between the regressors and demand. • A high p-value indicates that there is little correlation between the regressors and demand.	Quantity	
Source	Indicates the source of the coefficient values. Valid values are: • **Calculation**: Indicates that this record was calculated. • **Input**: Indicates that this record was taken from **ForecastItemParametersRegressionCoefficient**.	Enum	
StandardError	A value that represents the standard deviation of the statistical sample mean. By analyzing the spread of the data points in the historical data, the standard error measures how closely the historical data represents the overall data set. The more data points are measured in the calculation of the mean results in a lower standard error. A lower standard error indicates the more representative the sample is of the overall data set. This value is always positive.	Quantity	
tStatistic	A value that represents the estimate divided by the standard error. You can use the tStatistic to determine if you should support or reject that the regression coefficient is equal to 0 (null hypothesis). A high t-statistic indicates a greater statistical significance, therefore, provides evidence that the regression coefficient is not equal to 0.	Quantity	

StatisticalForecastResidualAutocorrelation table

The records in the **StatisticalForecastResidualAutocorrelation** table report the autocorrelation function of the residuals for forecast items that are using a multiple linear regression model to generate a statistical forecast. Autocorrelation measures the relationship between the residuals in one observation to the residuals in previous, or lagged, observations to determine the correlation between the observations.

The **StatisticalForecastResidualAutocorrelation** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 8.140: StatisticalForecastResidualAutocorrelation (Mfg) fields

Field name	Description	Data type	Key
ConfidenceIntervalLower	The lower limit for autocorrelation confidence intervals. This field is calculated only if the forecast was generated with **ForecastItemParameters.ResidualAutocorrelationConfidenceLevel** set to a value between 0 and 1. In all other cases, a value of -1 is returned.	Quantity	
ConfidenceIntervalUpper	The upper limit for autocorrelation confidence intervals. This field is calculated only if the forecast was generated with **ForecastItemParameters.ResidualAutocorrelationConfidenceLevel** set to a value between 0 and 1. In all other cases, a value of -1 is returned.	Quantity	
ItemParameters	A reference to the forecast item that is associated with the autocorrelation record. **Referenced table**: ForecastItemParameters	Reference	
Lag	The past **ForecastItemParametersType.IntervalsCalendar** period for which the autocorrelation quantity applies.	Integer	
Quantity	The autocorrelation quantity that correspond to the forecast item.	Quantity	

StatisticalForecastResidualCount table

The **StatisticalForecastResidualCount** table reports the number of observations that have residuals with quantities that are within a particular range, or bin. The residual represents the difference between the actual demand for a forecast item in one period and the demand that was predicted in the same period. The **StatisticalForecastResidualCount** calculations are based on the forecast model parameters contained in the **ForecastItemParameters** table. This table is used for forecast items that are using multiple linear regression model.

Note: RapidResponse uses the Sturges formula to determine the number of bins to use for reporting the residuals.

The **StatisticalForecastRegressionCoefficient** table supports the Demand Planning and Statistical Forecasting application in RapidResponse. This table was introduced in version 2016.2 Service Update 27.

Table 8.141: StatisticalForecastResidualCount (Mfg) fields

Field name	Description	Data type	Key
ItemParameters	A reference to the forecast item that is associated with the residual count. **Referenced table**: ForecastItemParameters	Reference	
Bin	The quantity reported represents the left-hand edge/lower limit of the range.	Quantity	
Count	The quantity of residuals that fall within the bin.	Integer	

SubstituteSupply table

The **SubstituteSupply** table shows the results of Netting where supply of a substitute part was used to satisfy demand for a primary part.

Note that this table shows the results of global part substitute calculations only (it does not consider BOM-level part substitution).

Table 8.142: SubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The AlternatePart record that defined the relationship between the primary part that has the demand, and the substitute part used to satisfy the demand. **Referenced table:** AlternatePart	Reference	
CalcRequestStart Date	The date on which work should start on this supply in order to meet the earliest request date on any driving independent demand it satisfies. Based on the original **RequestDate** exploded down from the driver **IndependentDemand** record and adjusted by lead time at each level in the product structure. If the driver demand is not an independent demand, or has an undefined **RequestDate**, then this is the date on which work should start on this supply in order to meet the due date on the driver.	Date	
DueDate	The date the order is planned to be in stock.	Date	

Table 8.142: SubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
EarliestExpiryDate	The earliest date that any supplies used to satisfy dependent demand from this substitute supply may expire. Typically, this is determined from the latest **EarliestExpiryDate** found on a demand that Netting calculations determine this supply will satisfy. Otherwise, if there are no consuming demands, this value is calculated by adding the number of expiry calendar intervals defined in **Part.MinimumShelfLife** to the supply due date. For non-expiry parts (**PartType.ExpiryRule is** set to "Ignore"), this field always reports "Past".	Date	
EstimatedExpiry Date	The date that the Netting analytic calculates this substitute supply will expire. Used to determine which demands this supply can potentially be used to satisfy (supply is only eligible for use if it is on or before the demand's EarliestExpiryDate). This date might be different from the actual expiry date reported in CTPSubstituteSupply.ExpiryDate which takes supply availabilities into account. If the substitute supply comes from on hand inventory or an inventory transfer then the supply's **ExpiryDate** value is reported. For all other types of substitute supply the supply's **EstimatedExpiryDate** value is reported. Non-expiry parts (PartType.ExpiryRule is set to "Ignore"), always return a value of "Future".	Date	
IsSubstitute	Indicates the source of this record. Valid values are: • **Y**—The record results from part substitution logic. The reference to the AlternatePart table is valid. • **N**—The record results from flow-to-common logic. The reference to the AlternatePart table is null.	Boolean	
Line	The line number for this substitute supply. For each primary part, the Line begins at 1 and increments up by 1 for each substitute supply associated with the part.	Integer	
Model	The model of the demand satisfied by this substitute supply. **Referenced table**: Model	Reference	
OrderPriority	The priority of the demand satisfied by this substitute supply. **Referenced table**: OrderPriority	Reference	
Part	The primary part that needed supply from one of its substitute parts. **Referenced table:** Part	Reference	Yes

Table 8.142: SubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
Pool	The pool of the demand satisfied by this substitute supply. **Referenced table**: Pool	Reference	
Quantity	The amount of supply required (given in the primary part's unit of measure).	Quantity	
StartUnit	The start unit number from the demand satisfied by this supply.	Integer	
SubstitutePool	The pool that the substitute supply used to satisfy the demand belongs to. **Referenced table**: Pool	Reference	

Supply table

The **Supply** table is a consolidation table that combines multiple sources of supply such as scheduled receipts, planned orders, and inventory transfers. This table includes entries for parts where **Part.Type.ProcessingRule** is set to "Ignore".

Table 8.143: Supply (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The part the supply activity was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table**: Part	Reference	
CalcStartDate	The order start date as calculated by netting, including allowance for constraint.	Date	
CoByProductSupply	If the source of this supply is a co-product or by-product generated from the production of a primary product, this field is a reference to that co-product or by-product supply. **Referenced table:** CoByProductSupply	Reference	

Table 8.143: Supply (Mfg) fields

Field name	Description	Data type	Owner
CurrentOperation	When the routing has been partially completed, this field is used to start the processing of the routing at an operation other than the first operation. When backward scheduling the order, all operations remaining will be scheduled relative to the due date of the order. When forward scheduling the order, all operations remaining will be scheduled relative to the MRP RunDate.	Integer	
DemandDate	The date of the first demand that netting satisfies with this supply.	Date	
DueDate	The date the order is planned to be in stock.	Date	
EffQuantity	The quantity of this receipt that is expected to be received into inventory after applying yield/scrap and unit of measure conversion.	Quantity	
EarliestExpiryDate	The earliest date that any supply used to satisfy dependent demands generated from this supply may expire. Typically, based on the latest **EarliestExpiryDate** associated with any demands that the Netting analytic determines this supply will satisfy.	Date	
EstimatedExpiry Date	The date that the Netting analytic calculates that supply will expire. If the supply comes from an inventory transfer then the supply's **ExpiryDate** value is reported. For all other supply types reported in this table, the supply's **EstimatedExpiryDate** value is reported.	Date	
IsPlanned	Valid values are: • **Y**—if the supply is a PlannedOrder • **N**—if the supply is a ScheduledReceipt **Note:** This field is obsolete, and the SupplySource field should be used instead.	Boolean	
Inventory Transfer	The inventory transfer that is the source of this supply. **Referenced table:** InventoryTransfer	Reference	
Line	The Line number of the ScheduledReceipt or PlannedOrder for this supply.	String	

Table 8.143: Supply (Mfg) fields

Field name	Description	Data type	Owner
Model	The model that applies to this supply. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is Ignore **Referenced table:** Model	Reference	
NeedQty	NeedQty for ScheduledReceipts, NeedQty for PlannedOrders.	Quantity	
Order	If this is a ScheduledReceipt, the field will contain the order number. If this is a PlannedOrder, the field will be blank.	String	
Part	Name of the part. **Referenced table:** Part	Reference	Yes
PartSource	Copied from PlannedOrder or ScheduledReceipt. The PartSource associated with this supply. **Referenced table:** PartSource	Reference	
Pool	The pool that applies to this supply. It is set to Unpooled if either of the following conditions exist: • Pool is not enabled • Part.MUEPoolNettingType.PoolRule is Ignore **Referenced table:** Pool	Reference	
Quantity	The amount required (including allowances for yield).	Quantity	
QuantityUnitOfMeasure	Reference to the unit of measure used for the quantity associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
RecommendedDate	RecommendedDate for ScheduledReceipts, DueDate for PlannedOrders.	Date	
ReschedDate	The rescheduled due date. ReschedDate for ScheduledReceipts, DueDate for PlannedOrders.	Date	
ScheduledReceipt	Reference to the source of supply. NULL if Supply is not from a scheduled receipt. **Referenced table:** ScheduledReceipt	Reference	

Table 8.143: Supply (Mfg) fields

Field name	Description	Data type	Owner
ShipDate	The date the supply should be shipped from its supplier. This is copied from PlannedOrder or ScheduledReceipt.	Date	
StartDate	When it should be released to the vendor or production started.	Date	
StartUnit	The start unit number that applies to the immediate supply. This field is always set to -1 if either Model-Unit Effectivity is not enabled, or the Part.MUEPoolNettingType.UnitRule is set to "Ignore"	Integer	
StockUnitQuantity	Represents the supply quantity in inventory unit of measure (UOM) without applying yield or scrap factors.	Quantity	
SubstituteSupply	The substitute supply that created this supply record. **Referenced table:** SubstituteSupply	Reference	
Supplier	The supplier of the part. **Referenced table:** Supplier	Reference	
SupplySource	The source of this supply. Valid values are: • CoByProductSupply • InventoryTransfer • PlannedOrder • ScheduledReceipt • SubstituteSupply	String (Enum)	
Type	The supply type record used to process this supply record. **Referenced table:** SupplyType	Reference	

SupplyDemand table

The **SupplyDemand** table tracks single level allocations of supply to demand.

Note: If attribute-based planning is used to impact the allotment of supply to demand, relevant attributes can be reported on records in this table using the `Attribute<*>` syntax. This includes a part's supply attributes (for example, part source and input supply attributes) as well as attributes rolled up the product structure from component supplies. Note however that demand attributes are not reported in this table. For more information on the *Attribute*<*> syntax, see the *RapidResponse Resource Authoring Guide*).

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
AllocatedQty	The quantity of the supply allocated to the current demand. For parts where the DemandSource is expired, this field reports the expiring quantity of the supply.	Quantity	
Allocation	The Allocation record (or Null) that is the demand being satisfied. **Referenced table:** Allocation	Reference	
Allotment Override	A reference to the allotment override record whose ID that Netting calculations have passed down to the demand being allocated (otherwise Null). This is the AllotmentOverride.Id value associated with the originating independent demand, scheduled receipt, or assembly part source that component supply is being reserved for, and which then gets propagated down the structure during dependent demand explosion. When using allotment overrides (manual reservations of component quantities), this value is reported throughout the product structure between the component supply being reserved and the top-level item it is intended to satisfy. However, in some cases the allotment override reported in this field might not necessarily be used when determining the supply reported in this record was allocated to the demand reported in this record. For example, this might occur in cases where one supply goes into multiple originating independent demands where each refers to a different AllotmentOverride record, or at intermediate levels where no active AllotmentOverrideDetails records are present. **Referenced table:** AllotmentOverride	Reference	
Alternate DemandPart	The part the demand was created for. This value is different from the Part field value only if that part is interchangeable with this part. **Referenced table**: Part	Reference	
Alternate SupplyPart	The part the supply allocation was created for. This value is different from the Part field value only if that part is interchangeable with this part. **Referenced table**: Part	Reference	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
AvailableStart Date	The Start date for the supply (based on the earliest constraint consumption date converted to a start date). If incremental availability logic is disabled for the part, then all components must be available before any constraint can be allocated to the supply. However, if incremental availability is enabled for the part, then only some components need be available before constraint can be allocated to the supply. For more information about Incremental Availability, see "Incremental availability calculations" on page 1476. If there is no applicable/active constraint, then this field is based on **AvailableDate**.	Date	
BOM	The BillOfMaterial that drives the dependent demand. **Referenced table:** BillOfMaterial Beginning in Version 10, RapidResponse supports modeling multiple parts that come out of the same production. This is intended for situations where a primary product is being built, using its components and production processes, but where one or more other co-products or by-products are generated from that same production process in predictable percentages. For example, different grades of the same part might come out of one production process.	Reference	
Constraint	If this supply is late due to a constraint having insufficient remaining capacity to meet the supply's **MaterialAvailableDate,** then this returns a reference to the constraint causing the lateness. If a supply is made late by multiple constraints, only one of them is reported here. If the supply is made late by something other than a constraint, or if the supply is on time, then this field is empty. **Referenced table**: Constraint	Reference	
CTPCoByProduct Supply	The co-product or by-product supply that supplies this demand. **Referenced table:** CTPCoByProductSupply	Reference	
CTPForecast	A reference to the details of the consensus forecast associated with this demand. **Referenced table:** CTPForecast	Reference	
CTPPlanned Order	The CTP Planned Order which is the supply. **Referenced table:** CTPPlannedOrder	Reference	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
CTPSubstitute Supply	The CTP Substitute Supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	
Date	The date used when allocating the supply. The date is determined by the value set in PartType.SupplyAllocationRule. For ByAvailable, the available date of the supply is used. For ByMRPPlan, the demand due date calculated by netting for this supply is used.	Date	
DemandDueDate	The due date of the demand satisfied with this record. DueDate for independent demands and dependent demands. ReschedDate for Allocations.	Date	
DemandEarliest ExpiryDate	The earliest date that a supply is allowed to expire if it is to be used in satisfying the demand reported on this record. For non-expiry parts (**PartType.ExpiryRule** is set to "Ignore"), this field always returns "Past".	Date	
DemandFraction	The fraction of the total demand satisfied by AllocatedQty (for example, AllocatedQty / Demand.Quantity).	Quantity	
DemandOrder Priority	The order priority of the demand associated with this record. If **DemandSource** is "None" this reports an empty reference, and if **DemandSource** is "OnHand" this is assigned the highest priority (lowest **OrderPriority.PlanningPriority** value). For dependent demands, the pegged supply's order priority is reported. **Referenced table:** OrderPriority	Reference	
DemandShip Date	For all planned orders and scheduled receipts, this date represents the date by which the order must ship in order to meet DemandDueDate (considering the part source's shipping time). For all other supply sources, this field reports the same value as DemandDueDate.	Date	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
DemandSource	Indicates the type of demand. It also determines which reference is valid. Valid values are: • **Allocation**—Allocation and parent scheduled receipt references are valid. • **ConsensusForecast**— Demand originates from consensus forecast generated during sales and operations planning process in RapidResponse. CTPForecast reference is valid. • **DependentForecast**—ParentPlannedOrder and Forecast references are valid. • **Expired**—supply cannot be used to satisfy a demand because of its expiry date. • **ExplodeOnly**—dependent demand from an explode only scheduled receipt (that is, where Supply.Type.ProcessingRule is "ExplodeOnly"). • **IndependentDemand**—Independent demand reference is valid. • **IndependentForecast**—Forecast and IndependentDemand references are valid. • **InventoryTransfer**—Parent inventory transfer is valid. • **NewAllocation**—parent scheduled receipt and BOM references are valid. • **NewTransferAllocation**—parent scheduled receipt reference is valid. • **None**—used for excess supply, including safety stock, at this part level (excess at higher levels shows up as the normal dependent demand). • **OnHand**—used for negative onhand. • **PlannedAllocation**—BOM and parent planned order references are valid. • **PlannedTransferAllocation**—parent planned order reference is valid. • **ScheduledReceiptForecast**— demand side of the current record is a forecast generated from a scheduled receipt with **Order.Type.AllocationDemandType.ProcessingRule** = 'SalesForecast'. • **SubstituteAllocation**—ParentCTPSubstituteSupply reference is valid.	String (Enum)	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
ExpiryDate	The date this supply expires. Typically for expiry parts, this is set to the **ExpiryDate** on the supply record. However, if this allotment pertains to a top-level demand from a **ConsensusForecast** or **IndependentDemand** record for an expiring part, this field reports the supply **ExpiryDate** less the **ExpiryOffset** value (if provided on the applicable **PartCustomer**, **PartCustomerTimePhasedAttributes**, or **IndependentDemand** record).	Date	
Forecast	The specific Forecast record that is the demand (after spreading). **Referenced table:** Forecast	Reference	
Independent Demand	The Independent Demand (or Null) that is being directly satisfied by this supply. **Referenced table:** IndependentDemand	Reference	
Inventory Transfer	The Inventory Transfer that supplies the demand. **Referenced table:** InventoryTransfer	Reference	
InventoryTransfer Allocation	The Inventory Transfer that creates this demand. **Referenced table:** InventoryTransfer	Reference	
HoldEndDate	The date on which the supply allotment on this record can actually be used without violating the minimum safety stock level (represented as a higher priority safety stock demand). This field is only calculated if the part's **SafetyStockPolicy.CTPHoldRule** setting is "Use" and if the supply allotment needed to be held. With this setting, demand available dates can be limited by the **HoldEndDate** (if any) on their supply allotment(s), rather than by supply available dates. **NOTE**: This field was added in Version 2016.2, Service Update 18.	Date	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
HoldStartDate	The date from which the supply allotment on this record should be held in order to respect a higher priority safety stock demand. In other words, this is the date that the minimum safety stock level (represented as higher priority safety stock) would be violated if this allotment were not held. This field is only calculated if the part's **SafetyStockPolicy.CTPHoldRule** setting is "Use" and if the supply allotment needed to be held. **NOTE**: This field was added in Version 2016.2, Service Update 18.	Date	
MaterialStartDate	If incremental availability is turned on for this supply (as set by Part.IncrementalRule and SupplyStatus.IncrementalSplit), this is the date when all materials are available for the split supply. If incremental availability is turned off for this supply, this date is the same as the supply's material start date (date when all materials are available).	Date	
Model	The model used when netting this supply and demand. **Referenced table:** Model	Reference	
NewProcessCost	Standard labor and overhead cost of planned orders to complete this order. This field is calculated as: PartSource.OverheadCost + PartSource.LaborCost * Quantity) from all planned orders pegged back to this IndependentDemand (note that if there are effective **PartSourceTimePhasedAttributes** records then their values are used instead of those on the **PartSource**). Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InventoryTransfer, the **ShipDate** field of the inventory transfer record is used.	Money	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
NewPurchasesCost	The cost of planned order purchase cost attributed to this demand. This is calculated based on the supply's applicable **PartSourceType.CostRule** values as follows: • If set to "MaterialLaborOverheadCosts" then calculated based on the material, labor and overhead values on either the applicable **PartSourceTimePhasedAttributes** record (if one exists) or the **PartSource** record. • Otherwise if set to "UnitCost" then derived from the **UnitCost** value on the **PartSource** record. • And if the applicable calculation returns a value less than or equal to zero (<= 0), then derived from the **Part.StdUnitCost** value. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InventoryTransfer, the **ShipDate** field of the inventory transfer record is used.	Money	
OnHand	The on hand supply (or Null) that is being allocated. **Referenced table:** OnHand	Reference	
OnHandDemand	Demand coming from a negative OnHand record. **Referenced table**: OnHand	Reference	
OrderPriority	The priority associated with this supply. **Referenced table:** OrderPriority	Reference	
ParentCTPCo ByProduct Supply	If supply of a component is used to satisfy dependent demand from a parent supply that is associated with one or more co-product or by-product supplies, this is a reference to one of those co-product or by-product supplies. **Referenced table:** CTPCoByProductSupply	Reference	
ParentCTP PlannedOrder	The CTP Planned Order that creates the dependent demand. **Referenced table:** CTPPlannedOrder	Reference	
ParentCTP SubstituteSupply	The CTP Substitute Supply that creates the demand. **Referenced table:** CTPSubstituteSupply	Reference	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
ParentScheduled Receipt	The Scheduled Receipt that is the parent of the dependent demand. **Referenced table:** ScheduledReceipt	Reference	
Part	The part this supply demand relationship applies to. **Referenced table:** Part	Reference	Yes
PegCoByProduct Part	If supply of a component is used to satisfy dependent demand from a parent supply of a primary product associated with one or more co-products or by-products, this is a reference to one of those co-products or by-products. If there are no co-products or by-products associated with the parent supply, this field returns the same value as PegPart. Referenced table: Part	Reference	
PegPart	The part, if any, whose supply creates the dependent demand being satisfied by this record.	Reference	
PegPool	The pool of the parent supply that the dependent demand is associated with. This value can be different than the Pool field. **Referenced table:** Pool	Reference	
Pool	The pool used when netting this supply and demand. **Referenced table:** Pool	Reference	
PushDate	The date when the pre-built supply for this part should be transferred (pushed) from the sourcing site to a distribution site. This field only applies to parts that are enabled for push planning (have **DistributionPlanningRule.ProcessingRule** set to "Use"). By default, it is only populated when supply for the part exceeds its maximum inventory level and is then pushed out early to satisfy future demand. However, if **DistributionPlanningRule.SupplyShareRule** is set to "All", then this field is populated for all transfers from the sourcing site to distribution site regardless of the maximum inventory level or whether the supply is early to meet the demand. **Note:** This field was added in Version 2016.2.	Date	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
ScheduledReceipt	The scheduled receipt that satisfies the current demand. **Referenced table:** ScheduledReceipt	Reference	
SplitQuantity	If incremental availability is turned on for the supply, this represents the portion of the supply available based on either material or constraint availability. If incremental availability is turned off for the supply, this represents the total supply quantity for scheduled receipts, or the portion of a CTPPlannedOrder with constraint allocated to it.	Quantity	
SplitQuantityUnitOfMeasure	Reference to the unit of measure used for the quantity (supply available or total supply quantity) associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
StartUnit	Unit number used when netting this supply and demand. That is, the supply start unit.	Integer	
SubstituteGroup	A reference to the substitute group associated with this supply. Null unless the supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt. **Referenced table:** SubstituteGroup	Reference	
SupplyAvailable Date	The date this supply (which could be a split supply due to either material or constraint availability) is projected to be available. Past for on hand, and Future if never available.	Date	
SupplyEstimated ExpiryDate	The estimated expiry date for the supply reported on this record. This value is calculated by the Netting analytic and used in determining the demands that supply for an expiry part can potentially satisfy. For non-expiry parts (**PartType.ExpiryRule** is set to "Ignore"), this field always returns "Future".	Date	
SupplyFraction	The fraction of the total supply that is allocated to this demand (for example, AllocatedQty / Supply.Quantity).	Quantity	

Table 8.144: SupplyDemand (Mfg) fields

Field name	Description	Data type	Owner
SupplyReschedDate	The rescheduled date of the supply. Past for OnHand, ReschedDate for ScheduledReceipt, DueDate for PlannedOrder.	Date	
SupplySource	Indicates the type source of supply. It is used to determine which references are valid and is one of: • OnHand • ScheduledReceipt • PlannedOrder • SubstituteSupply • InventoryTransfer • CoByProductSupply • None	String (Enum)	

TaskDemandLinkCost table

The **TaskDemandLinkCost** table reports date-specific cost of goods sold values from all component supplies used to satisfy the portion of an independent demand required to complete a particular project task (as defined by a **TaskDemandLink** record). Rolled up cost values from all records in this table associated with a given **TaskDemandLink** record are reported in the **TaskDemandLink.MaterialCost** field.

Table 8.145: TaskDemandLinkCost (ProjMgmt) fields

Field name	Description	Data type	Owner
Date	The date these costs pertain to. One record is added to this table for each unique date on which one or more component supplies are needed to satisfy the independent demand referenced through **TaskDemandLink**.	Date	
TaskDemandLink	The **TaskDemandLink** record this cost applies to (identifies the independent demand, project task, and amount of the order needed to complete that task). **Referenced table:** TaskDemandLink	Reference	
Value	The dollar cost of all component supplies needed on this date.	Money	

TaskPartLinkCost table

The **TaskPartLinkCost** table reports date-specific cost of goods sold values from all component supplies used to satisfy the requirement for a given part needed to complete a particular project task (as defined

by a **TaskPartLink** record). Rolled up cost values from all records in this table associated with a given **TaskPartLink** record are reported in the **TaskPartLink.MaterialCost** field.

Table 8.146: TaskPartLinkCost (ProjMgmt) fields

Field name	Description	Data type	Owner
Date	The date these costs pertain to. One record is added to this table for each unique date on which one or more component supplies are needed to satisfy the requirement for the part referenced through **TaskPartLink**.	Date	
TaskPartLink	The **TaskPartLink** record this cost applies to (identifies the part, project task, and amount of the part needed to complete that task). **Referenced table:** TaskPartLink	Reference	
Value	The dollar cost of all component supplies needed on this date.	Money	

TaskSupplyLinkCost table

The **TaskSupplyLinkCost** table reports date-specific cost of goods sold values from all component supplies used to satisfy the portion of a scheduled receipt required to complete a particular project task (as defined by a **TaskSupplyLink** record). Rolled up cost values from all records in this table associated with a given **TaskSupplyLink** record are reported in the **TaskSupplyLink.MaterialCost** field.

Table 8.147: TaskPartLinkCost (ProjMgmt) fields

Field name	Description	Data type	Owner
Date	The date these costs pertain to. One record is added to this table for each unique date on which one or more component supplies are needed to satisfy the scheduled receipt referenced through **TaskSupplyLink**.	Date	
TaskSupplyLink	The **TaskSupplyLink** record this cost applies to (identifies the scheduled receipt, project task, and amount of the supply needed to complete that task). **Referenced table:** TaskSupplyLink	Reference	
Value	The dollar cost of all component supplies needed on this date.	Money	

TrialCoByProductSupply table

The **TrialCoByProduct** table logs details of co-product or by-product supplies (generated under a primary product's scheduled receipt or trial planned order) which could not be planned in time to cover the driver demands matched to them by Netting logic, and where those end-item demands were left

unsatisfied because multi-level search logic determined supply couldn't be planned for those demands within a specified tolerance of lateness. It is referenced by the **MLSActivity** table and provides additional details on records in that table with a **SupplySource** of "TrialCoByProductSupply".

This table was added in Version 2016.2, Service Update 5.

Note: Records are be logged in this table if the end-item part, having the unsatisfied demand, has its **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Also, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply portion).

Table 8.148: TrialCoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
DueDate	The date when this co-product or by-product supply is planned to be in stock.	Date	
Index	An index number for this trial co-product or by-product supply. Trial co/by product records start with an index of 1 and increment for each additional co/by product created for the part.	Integer	
Model	The model that applies to this supply. **Referenced table:** Model	Reference	
Part	The co-product or by-product part that this trial supply is for. **Referenced table:** Part	Reference	Yes
Pool	The pool that this supply belongs to. **Referenced table:** Pool	Reference	
PrimaryPart	The primary part whose supply led to the generation of the co-product or by-product reported on this record. **Referenced table:** Part	Reference	
PrimaryScheduled Receipt	The primary product's scheduled receipt that generated the co-product or by-product supply reported on this record. Null if **PrimarySupplyType** is "TrialPlannedOrder" **Referenced table**: Scheduled Receipt	Reference	

Table 8.148: TrialCoByProductSupply (Mfg) fields

Field name	Description	Data type	Owner
PrimarySupplyType	Indicates the type of supply on the primary product that generated the trial co-product or by-product supply reported on this record. Valid values are: • **ScheduledReceipt**—primary product supply was a scheduled receipt. • **TrialPlannedOrder**—primary product supply was a trial planned order.	String (Enum)	
PrimaryTrial PlannedOrder	The primary product's trial planned order that generated the co-product or by-product supply reported on this record. Null if **PrimarySupplyType** is "ScheduledReceipt"	Reference	
Quantity	The amount of this co-product or by-product supply.	Quantity	
StartDate	The start date of the primary product supply that generated this co-product or by-product supply.	Date	
StartUnit	The start unit of this supply.	Integer	
TrialAvailableDate	The date this trial co-product or by-product supply would actually be completed and available in stock based on rolled up planned component availability. For example, planning time fence and constraint availability on the primary part's components might make this date later than the planned due date.	Date	

TrialConstraintShortage table

The **TrialConstraintShortage** table reports when extra constraint would be needed, at any level of supply, to satisfy end-item driver demands that were left wholly or partially unsatisfied. The constraint shortages reported are those that occur when multi-level search logic determines that supply for end-items cannot be planned within a specified tolerance of lateness.

Each record in this table identifies the amount a constraint is short on a given date, and references the **MLSActivity** table for information on the specific demand that was left unsatisfied along with details on the trial supply that needed additional constraint in order for the driver demand to be fully satisfied.

This table was added in Version 2016.2, Service Update 5.

Note: Records are only logged in this table if the end-item part, having the unsatisfied demand, has its **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Also, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply portion).

Table 8.149: TrialConstraintShortage (Mfg) fields

Field name	Description	Data type	Owner
DriverPart	The end-item driver part whose demand was ultimately left unsatisfied due to the shortage of the constraint reported on this record. **Referenced table:** Part	Reference	Yes
LoadDate	The date on which constraint load was placed and a shortage is being reported. Based on the need date converted to a constraint consumption date.	Date	
MLSActivity	Reports details such as the end-item demand that was left unsatisfied and the associated supply affected by the constraint shortage. **Referenced table**: MLSActivity	Reference	
ShortQuantity	The difference between the amount of constraint available on **LoadDate** and the amount needed to fully satisfy the requirement from the driver demand.	Quantity	
SourceConstraint	Identifies the constraint with the shortage and the associated part source that requires additional constraint to provide supply by the need date. **Referenced table**: SourceConstraint	Reference	

TrialConsumption table

The **TrialConsumption** table reports details of constraints that multi-level search logic used on trial planned orders or scheduled receipts, but where not enough constraint was available by the need date and end-item driver demand was ultimately left unsatisfied because it could not be planned within a specified tolerance of lateness. Each record in this table identifies the amount of the constraint that multi-level search logic tried to use on a given date along with a reference to the trial planned order or scheduled receipt that required that constraint.

This table was added in Version 2016.2, Service Update 5.

Note: Records can only be logged in this table if the end-item part, with the unsatisfied demand, has its **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Also, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply portion).

Table 8.150: TrialConsumption (Mfg) fields

Field name	Description	Data type	Owner
IsPlanned	Indicates if the trial constraint consumption reported on this record comes from a trial planned order. Valid values: • **N**— constraint consumption does not come from a trial planned order (the **ScheduledReceipt** reference is valid). • **Y**— constraint consumption comes from a trial planned order (the **TrialPlannedOrder** reference is valid).	String (Enum)	
Load	The amount of load placed on the constraint by the trial consumption reported on this record.	Quantity	
LoadDate	The date that trial consumption placed load on this constraint.	Date	
Part	Identifies the part that required the constraint whose trial consumption is reported on this record. **Referenced table:** Part	Reference	Yes
ScheduledReceipt	The scheduled receipt responsible for the constraint consumption reported on this record. Null if **IsPlanned** is "Y". **Referenced table:** ScheduledReceipt	Reference	
SourceConstraint	The constraint and the associated part source that requires additional constraint to provide supply by the need date. **Referenced table:** SourceConstraint	Reference	
TrialPlannedOrder	The trial planned order responsible for the constraint consumption reported on this record. Null if **IsPlanned** is "N". **Referenced table:** TrialPlannedOrder	Reference	
UseDate	The date that trial consumption actually used the constraint.	Date	

TrialPlannedOrder table

The **TrialPlannedOrder** table reports details of planned orders which could not be planned in time to cover their driver end-item demands, and where those end-item demands were left unsatisfied because multi-level search logic determined supply couldn't be planned for those demands within a specified tolerance of lateness. It is referenced by the **MLSActivity** table and provides additional details on records in that table with a **SupplySource** of "TrialPlannedOrder". For example, constraint availability or the planning time fence might have prevented a component supply from being planned when it was needed and the assembly demand thereby left unsatisfied.

This table was added in Version 2016.2, Service Update 5.

Note: Records can only be logged in this table if the end-item part, with the unsatisfied demand, has its **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Also, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply portion).

Table 8.151: TrialPlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
DemandDate	The date of the demand that this trial planned order would satisfy.	Date	
DueDate	The date this trail planned order would be planned to be in stock.	Date	
EffQuantity	The amount of this supply that would be received into inventory after accounting for yield factor, unit of measure conversion, and so on.	Quantity	
Index	An index number for this trial planned order. Trial planned orders for each part start with an index of 1 and increment for each additional order created for the part.	Integer	
Model	The model, if any, that applies to this supply. **Referenced table:** Model	Reference	
OrderPriority	The priority assigned to this trial planned order. Based on configuration settings, this is typically taken from the demand the planned order is created to satisfy or else set to the part's default priority. **Referenced table:** OrderPriority	Reference	
Part	The part the trial planned order is for. **Referenced table:** Part	Reference	Yes
PartSource	Identifies the part source that provided the trial planned order. If a planned order could not be created for the component because the demand date was outside of the effectivity window for all part sources, this field returns a value of "None". **Referenced table:** PartSource	Reference	
Pool	The pool, if any, that this supply belongs to. **Referenced table:** Pool	Reference	

Table 8.151: TrialPlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
Quantity	The total amount of supply required (including allowances for yield).	Quantity	
ShipDate	The date this trial planned order would ship.	Date	
StartDate	The date this trial planned order would be started (adjusted to compensate for constraint where appropriate).	Date	
StartUnit	The start unit of this supply.	Integer	
Transaction Sequence	The sequence assigned to this trial planned order. Depending on configuration settings, this is either taken from the demand the order satisfies or set to 0.	Integer	
TrialAvailableDate	The date this trial planned order would actually be completed and available in stock based on rolled up planned availability on components. For example, planning time fence and constraint availability on the part's components might make this date later than the planned due date.	Date	

TrialSubstituteSupply table

The **TrialSubstituteSupply** table reports substitute supplies which could not be planned in time to cover the driver end-item demands of primary parts that needed them, and where those end-item demands were left unsatisfied because multi-level search logic determined supply couldn't be planned for those demands within a specified tolerance of lateness. It is referenced by the **MLSActivity** table and provides additional details on records in that table with a **SupplySource** of "TrialSubstituteSupply". For example, constraint availability or the planning time fence might have prevented a substitute supply from being planned when it was needed and the primary part's demand thereby left unsatisfied.

This table was added in Version 2016.2, Service Update 5. Note that this table shows the results of global part substitute calculations only (it does not consider BOM-level part substitution).

Note: Records can only be logged in this table if the end-item part, with the unsatisfied demand, has its **MultiLevelSearchRule.LoggingRule** set to "UnsatisfiedDemand". Also, the **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting impacts the amount of the unsatisfied demand and associated late supplies that are logged (either the full unsatisfied quantity or just the quantity related to the first late supply portion).

Table 8.152: TrialSubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
AlternatePart	The **AlternatePart** record that defined the relationship between the primary part that has demand, and the substitute part for which a trial supply was created to try and satisfy the demand. Null if **IfSubstitute** is set to "N". **Referenced table:** AlternatePart	Reference	
DueDate	The date this trail substitute supply would be planned to be in stock.	Date	
Index	An index number for this trial substitute supply. Trial substitute supplies for each part start with an index of 1 and increment for each additional order created for the part.	Integer	
IsSubstitute	Indicates the source of this record. Valid values are: Valid values are: • **N**—The record results from flow-to-common logic (**AlternatePart** reference is Null). • **Y**— The record results from part substitution logic (**AlternatePart** reference is valid).	Boolean	
Model	The model, if any, that applies to this supply. **Referenced table:** Model	Reference	
OrderPriority	The priority assigned to this trial substitute supply. Based on configuration settings, this is typically taken from the demand the substitute supply is created to satisfy or else set to the primary part's default priority. **Referenced table:** OrderPriority	Reference	
Part	The primary part having demand that the trial substitute supply was created for. **Referenced table**: Part	Reference	Yes
Pool	The pool, if any, that this supply belongs to. **Referenced table:** Pool	Reference	
Quantity	The amount of supply that would be required from this trial substitute supply (expressed in the primary part's unit of measure)	Quantity	
StartUnit	The start unit of this supply.	Integer	

Table 8.152: TrialSubstituteSupply (Mfg) fields

Field name	Description	Data type	Owner
SubstitutePool	The pool that the trial substitute supply created to satisfy the demand belongs to. **Referenced table:** Pool	Reference	
TrialAvailableDate	The date this trial substitute supply would actually be completed and available in stock based on rolled up planned availability on components. For example, planning time fence and constraint availability on the substitute part's components might make this date later than the planned due date.	Date	

ValidatePlannedOrder table

The **ValidatePlannedOrder** calculated table is a consolidation of imported baseline planned orders from the **BaselinePlannedOrder** table and calculated planned orders from the **PlannedOrder** table. For more information about planned orders, see "PlannedOrder table" on page 1148. This table supports data validation and is used for internal purposes only.

Table 8.153: ValidatePlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
BaselineQuantity	Quantity of the baseline (imported) planned order. Zero if no baseline planned order is due on this date.	Quantity	
CalculatedQuantity	Quantity of the planned order calculated by RapidResponse. Zero if no calculated planned order is due on this date.	Quantity	
DueDate	Due date of the baseline (imported) planned order.	DueDate	
Model	Model that applies to the baseline planned order. If **Part.MUEPoolNettingType.ModelRule** is not set to "Net", or if Model-Unit Effectivity is not enabled, this field is set to the first value in the Model table (typically, this value is "None"). **Referenced table:** Model	Reference	
Part	The part this planned order is for. **Referenced table:** Part	Reference	Yes
PlannedOrder	Reference to the planned order due date (valid only if a calculated planned order is due on this date). **Referenced table:** PlannedOrder	Reference	

Table 8.153: ValidatePlannedOrder (Mfg) fields

Field name	Description	Data type	Owner
Pool	Pool that applies to the baseline planned order. If **Part.MUEPoolNettingType.PoolRule** is not set to "Net", or if Pool is not enabled, this field set to the first value in the Pool table (typically, this value is "Unpooled"). **Referenced table:** Pool	Reference	
StartDate	Start date of the baseline (imported) planned order.	Date	
StartUnit	Start unit that applies to the baseline (imported) planned order. If **Part.MUEPoolNettingType.UnitRule** is not set to "Net", or if Model-Unit Effectivity is not enabled this field is set to -1.	Integer	

WhereConsumed table

This table pegs supply records to end item demands which consume them. It is suitable for bottom-up analysis to show where specific part supplies are ultimately used or consumed. Consumption can be driven from independent demands and by excess on a parent's scheduled receipt or planned order.

This table provides a complete picture of constrained material flow:

- Records are generated for unused supply (safety, unused supply from lot sizing, excess scheduled receipts, and excess OnHand).
- WhereConsumed now reports the SynchronizedDate, the date a supply is needed in order for the parent supply to complete for its AvailableDate.
- MRP dates are available through references to the supply and demand records.

Relationship with LateSupply

All fields in the **LateSupply** and **WhereConsumedForDemand** tables are now either directly or indirectly supported from the **WhereConsumed** table. Those supported indirectly are available as shown below:

- OrderPriority (SupplyDemand.OrderPriority)
- Constraint (SupplyDemand.Constraint)
- GatingPart (SupplyDemand.GatingPart)
- DueDate (SupplyDueDate)
- SupplyPlannedOrder (CTPPlannedOrder)
- AvailableDate (SupplyAvailableDate)
- LatePart (Part)
- LateQuantity (NeedQuantity)
- Quantity (SupplyQuantity)
- TransactionSequence (SupplySequence)

Note: workbook variables are available that allow a specific workbook/worksheet to limit the results from this table to those whose dates fall within a specified reporting horizon. For more information, see "Specify a reporting horizon for WhereConsumedForSupply records" on page 2230.

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
ActionSupply	Indicates if this supply requires direct action to achieve on time delivery of its top-level driver (**Y** or **N**). That is, the supply is late itself, not just that its components are late: • Supply is late (Available later than Need Date) And: • Resched Date is after the Need Date Or: • Constrained at this part (Constraint is making this supply later, that is, SupplyDemand.Constraint is not Null) Or: • Inside lead time (see InsideLeadTime field)	Boolean	
AlternatePart	The part the supply was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table**: Part	Reference	
ATP	Flags that this portion of supply is available to satisfy new demands (available to promise) for this part. That is, • Part = 'DriverPart' and DriverType = 'ScheduledReceipt', 'PlannedOrder', or 'Forecast'. • DriverType = 'Forecast' and ParentType = 'Forecast'.	String	
BillOfMaterial	The BillOfMaterial record which passed this dependent requirement. **Referenced table**: BillOfMaterial	Reference	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
ConstraintNeed Date	The date constraint would be needed for this supply for it to satisfy its Driver's due date. This is need date converted to either a ship or start date. **Note:** This field is only populated if the supply associated with the record has a gating constraint (otherwise, it is left blank).	Date	
CTPCoByProduct Supply	The co-product or by-product supply associated with this order. NULL unless the supply is a co-product or by-product generated due to production of a primary product. **Referenced table:** CTPCoByProductSupply	Reference	
CTPForecast	A reference to the driver part's consensus forecast demand (as generated during the sales and operations process) that is ultimately satisfied by this component supply. NULL unless DriverType is 'ConsensusForecast'. **Referenced table:** CTPForecast	Reference	
CTPPlannedOrder	The immediate planned order that is this supply. Null unless the supply is a planned order. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	The CTP Substitute Supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	
DriverAvailable Date	The date the driver is available.	Date	
DriverCTPCoBy ProductSupply	A reference to the co-product or by-product supply driving this requirement. NULL unless DriverType is 'CoByProductSupply'. **Referenced table:** CTPCoByProductSupply	Reference	
DriverCTP PlannedOrder	A reference to the planned order driver driving this requirement. This field is Null unless DriverType = 'PlannedOrder'. **Referenced table**: CTPPlannedOrder	Reference	
DriverDaysLate	The number of periods the driver is available later than its due date.	Quantity	
DriverDueDate	The DueDate of the driver for this demand. DriverDueDate is valid regardless of the type of driver.	Date	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
DriverInventory Transfer	A reference to the inventory transfer driving this requirement. This field is Null unless DriverType = 'InventoryTransfer'. **Referenced table:** InventoryTransfer	Reference	
DriverNeed Quantity	The amount of the driver quantity that is ultimately satisfied by this pegged supply. This field indicates how much of the driver could be affected by this specific component supply, and hence can be used determine which supplies are having the greatest impact on a late order. **Note:** This field was added in Version 2016.2, Service Update 4. **Caution:** In linear BOM structures, this field can be used with NeedQuantity to calculate how many units of the component is ultimately needed to make a unit of the driver part. However in non-linear BOM structures, this can give misleading results as different values are likely to be returned based on the different sub-assemblies/paths under the driver where the lower-level component is used.	Quantity	
DriverOrderID	If the driver is an IndependentDemand, reports its Order.ID. Otherwise, reports the origin of this demand which is 'Excess', 'ExplodeOnly' or 'OnHand' (if negative on hand).	String	
DriverOrderType	If the driver is an IndependentDemand, reports its Order.Type.Value. Otherwise, reports the origin of this demand (to satisfy excess) as one of: • OnHand (also includes net negative on hand) • PlannedOrder • ScheduledReceipt	String	
DriverPart	A part that is driving the demand that is consuming the supply. For forecast, this is the lowest-level part where DemandType.Processingrule is 'SalesForecast'. **Referenced table:** Part	Reference	
DriverQuantity	If the driver is an IndependentDemand, then this is the Quantity of the demand. If the driver is a supply, then it contains the excess represented by the driving allotment.	Quantity	
DriverScheduled Receipt	The scheduled receipt ultimately driving this requirement (hence, excess). Null unless DriverType is ScheduledReceipt. **Referenced table:** ScheduledReceipt	Reference	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
DriverSupply Demand	A reference to the supply and demand details associated with the root driver for this WhereConsumed record. **Referenced table**: SupplyDemand	Reference	
DriverType	Indicates the highest source of the demand being satisfied by this supply. The value will be one of: • **CoByProductSupply**—this supply is used to satisfy a co-product or by-product generated from a primary product supply. • **ConsensusForecast**—this supply ultimately satisifies demand from a consensus forecast generated during the sales and operations process in RapidResponse. The ConsensusForecast reference is valid. • **Forecast**—this supply satisfies an unconsumed forecast • **IndependentDemand**—this supply ultimately satisfies an independent demand. The IndependentDemand reference is valid. • **InventoryTransfer**—this supply satisfies an inventory transfer. • **OnHand**—this supply is either excess onhand inventory or is ultimately used to satisfy demand coming from a negative onhand value. If excess onhand, then DriverOrderId is "Excess" and DriveDueDate is "Undefined". If satisfying negative onhand, then DriverOrderId is "OnHand" and DriverDueDate is "Past". • **ScheduledReceipt**—this demand ultimately satisfies a supply order (no demand, so excess). The ScheduledReceipt reference is valid. • **PlannedOrder**—this demand ultimately satisfies a supply with no corresponding demand (therefore excess due to safety or planned order lot sizing). The driver references are invalid.	String (Enum)	
Independent Demand	Reference to the source of demand. This value is NULL if the demand originated as Excess. **Referenced table:** IndependentDemand	Reference	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
InsideLeadTime	Tracks whether the supply is inside its lead time. • For PlannedOrders, if the order needs to start before RunDate, shows the number of work days before Run Date to start (otherwise, 0). • For released Scheduled Receipts, indicates the supply has a ReschedDate before DataDate. Reports the difference in work days (otherwise 0). • If unreleased Scheduled Receipts, indicates the supply has a CalcStartDate before DataDate. Reports the difference in work days (otherwise 0).	Integer	
InventoryTransfer	The inventory transfer satisfying this demand. **Referenced table:** InventoryTransfer	Reference	
Late	Number of periods by which the supply is late to achieve on-time delivery of its driver, based on its AvailableDate. Calculated as Supply.AvailableDate - NeedDate using Part.PlanningCalendars.TimeUnits.	Integer	
LateAtHere	Number of periods by which the supply is late to satisfy the driver demand, based on the supply reschedule date. Calculated as (supply. ReschedDate - need date to this part to satisfy driver) using Part.PlanningCalendars.TimeUnits. If the driver is a ScheduledReceipt or Planned order for the current part, LateAtHere is 0 (this is excess supply). For dependent demands where the Driver is excess, the need date is based on the Driver EffDueDate.	Integer	
Model	The model that is used when netting this record. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is 'Ignore' **Referenced table:** Model	Reference	
NeedDate	The date when this supply is needed in order to achieve on-time delivery for the driver (from the driver DueDate). Supply EffDueDate if excess supply (DueDate for planned). This is reported as 'undefined' for parts where the applicable **Type.AvailableRule** or **AvailableRule.ProcessingRule** setting is 'Ignore', and for parts where demand flows through a parent that is set to 'Ignore'.	Date	
NeedQuantity	The amount of this supply required to meet the demand.	Quantity	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
NewProcessCost	Standard labor and overhead cost of planned orders to complete this portion of the supply (through all lower levels of the bill). Currency conversions on this field use the **DueDate** field's conversion rate, except in the following case: • If the supply is an on hand supply, the **Date** field of the on hand record is used.	Money	
NewPurchases Cost	Cost of new purchases required in order to complete this portion of the supply (through all lower levels of the bill). Currency conversions on this field use the **DueDate** field's conversion rate, except in the following case: • If the supply is an on hand supply, the **Date** field of the on hand record is used.	Money	
OnHand	The inventory record satisfying this demand. (Null if SupplyType is not OnHand.) **Referenced table:** OnHand	Reference	
ParentType	Identifies whether this demand is dependent and whether it is planned or a scheduled receipt. Valid values are: • **PlannedOrder**—the demand comes through a planned order. The PegCTPPlannedOrder reference is valid. • **ScheduledReceipt**—the demand comes through a scheduled receipt. The PegScheduledReceipt reference is valid. • **InventoryTransfer**—the demand comes from an inventory transfer. • **SubstituteSupply**—the demand comes through a substiutute supply. The PegCTPSubstituteSupply reference is valid. • **None**—the demand is not dependent. Neither PegScheduledReceipt, PegCTPPlannedOrder, PegInventoryTransfer, or PegCTPSusbtituteSupply references is valid.	String (Enum)	
Part	The part whose supply is being consumed. **Referenced table:** Part	Reference	Yes
PartSource	The PartSource used by the supply. The PartSource.Supplier field indicates the supplier. **Referenced table:** PartSource	Reference	
PegCTPPlanned Order	The planned order (if this is a dependent demand from a planned order) that creates the current demand. **Referenced table:** CTPPlannedOrder	Reference	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
PegCTP Substitute Supply	The parent CTP Substitute Supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	
PegInventory Transfer	If dependent demand comes from an inventory transfer, this is a reference to that inventory transfer. **Referenced table:** InventoryTransfer	Reference	
PegPart	Parent part that is the immediate source of this dependent demand (passes through phantoms). **Referenced table:** Part	Reference	
PegScheduled Receipt	The scheduled receipt (if this is a dependent demand from a scheduled receipt) that creates the current demand. **Referenced table:** ScheduledReceipt	Reference	
PegSupply Demand	A reference to the single-level supply and demand allocation details for the immediate parent order that required the supply reported on this record. This allows reporting SupplyDemand information for any incremental portions of the parent order. **Referenced table:** SupplyDemand **Note:** This field was added in Version 2016.2, Service Update 3.	Reference	
Pool	The pool that is used when netting this record. It is set to 'Unpooled' if either of the following conditions exist: • Pool is not enabled • Part.MUEPoolNettingType.PoolRule is 'Ignore' **Referenced table:** Pool	Reference	
Scheduled Receipt	A reference to the ScheduledReceipt that is consumed. (Null if the supply is not a ScheduledReceipt.) **Referenced table:** ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the supply, this represents the available date for a portion of the supply. If incremental availability calculations are turned off for the supply, this represents the available date for the entire supply (unless a planned order has been split due to constraint allocation).	Date	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
SubstituteGroup	A reference to the substitute group associated with this supply. Null unless the supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt. **Referenced table:** SubstituteGroup	Reference	
Supply AvailableDate	The date when the entire supply is available.	Date	
SupplyDemand	The supply and demand being reported by this record. Use SupplyDemand.DemandSource to determine which SupplyDemand references are valid for single level pegging (BOM, SR, and so on). **Referenced table:** SupplyDemand	Reference	
SupplyDueDate	Effective due date of the supply being consumed. This is EffDueDate for ScheduledReceipts, DueDate for PlannedOrders (Past if OnHand).	Date	
ID	If this supply is a ScheduledReceipt, ID reports its order ID. Otherwise, shows SupplyTypeString.	String	
SupplyQuantity	Total quantity of the supply.	Quantity	
SupplyQuantity UnitOfMeasure	Reference to the unit of measure used for the supply quantity associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	

Table 8.154: WhereConsumed (Mfg) fields

Field name	Description	Data type	Owner
SupplySource	The source of the supply record. Valid values are: • **CoByProductSupply**—supply is a co-product or by-product. • **CTPPlannedOrder**—the supply is a normal planned order. • **InventoryTransfer**—the supply is an inventory transfer. • **InKit**—the supply is from an InKit quantity on an input allocation. • **None**—no supply is available to satisfy this demand • **OnHand**—the supply is in inventory. • **ScheduledReceipt**—the supply is a normal scheduled receipt • **SubstituteSupply**—the supply is from a substitute part.—	String (Enum)	
SupplyStartDate	The order start date as calculated by netting, including allowance for constraint. From CalcStartDate for ScheduledReceipts, StartDate for Planned orders. (Past if OnHand).	Date	
SupplyStartUnit	The start unit that is used when netting this record. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is 'Ignore'	Integer	
SupplyType	The type of the late supply. (Null for OnHand). **Referenced table:** SupplyType	Reference	
SupplyType String	The type of the supply record. This is a customer-specific value. The value is OnHand if the supply is in inventory.	String	
Synchronized Date	The date when this supply is needed in order to achieve completion of the immediate parent supply for its FirstAvailableDate. (AvailableDate if not dependent demand).	Date	

WhereConsumedForDemand table

This table pegs independent demand records to the set of supplies that they consume. It is suitable for top-down analysis to show all supplies that are ultimately consumed by a particular demand. Calculations start from the IndependentDemand rather than from supply consumption and may be used when reporting full level pegging from independent demands.

The structure of **WhereConsumedForDemand** is very similar to **WhereConsumed** except that it only tracks to IndependentDemand (including Forecast). Note that all WhereConsumedForDemand from a

SalesForecast independent demand will reference the same IndependentDemand even if that forecast is spread.

It is recommended that you use the **WhereConsumed** table if you require dependent demand information. Also, the **WhereConsumedForForecast** table can be used if you require information on supplies consumed by consensus forecast demands generated during the sales and operations planning process in RapidResponse.

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
ActionSupply	Indicates if this supply requires direct action to achieve on time delivery of its top-level driver (**Y** or **N**). That is, the supply is late itself, not just that its components are late: • Supply is late (Available later than Need Date) And: • Resched Date is after the Need Date Or: • Constrained at this part (Constraint is making this supply later, that is, SupplyDemand.Constraint is not Null) Or: • Inside lead time (see InsideLeadTime field)	Boolean	
AlternatePart	The part the supply was created for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table**: Part	Reference	
BillOfMaterial	The BillOfMaterial record which passed this dependent requirement. (Null if Independent Demand). **Referenced table:** BillOfMaterial	Reference	
CTPCoByProduct Supply	The co-product or by-product supply associated with this order. NULL unless the supply is a co-product or by-product generated due to production of a primary product. **Referenced table:** CTPCoProductSupply	Reference	
CTPPlannedOrder	The immediate planned order that is this supply. Null unless the supply is a planned order. **Referenced table:** CTPPlannedOrder	Reference	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
CTPSubstitute Supply	The CTP Substitute Supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	
DriverAvailable Date	The date the driver is available.	Date	
DriverDueDate	The DueDate of the driver for this demand. DriverDueDate is valid regardless of the type of driver.	Date	
DriverNeed Quantity	The amount of the driver quantity that is ultimately satisfied by the pegged supply. This field indicates how much of the driver demand could be affected by this specific component supply, and hence can be used to determine which supplies are having the greatest impact on a late order. **Note:** This field was added in Version 2016.2, Service Update 4. **Caution:** In linear BOM structures, this field can be used with NeedQuantity to calculate how many units of the component is ultimately needed to make a unit of the driver part. However in non-linear BOM structures, this can give misleading results as different values are likely to be returned based on the different sub-assemblies/paths under the driver where the lower-level component is used.	Quantity	
DriverSupply Demand	A reference to the supply and demand details associated with the root driver for this WhereConsumedForDemand record. Referenced table: SupplyDemand	Reference	
DriverQuantity	The total Quantity of the driver.	Quantity	
Independent Demand	The IndependentDemand that is driving the given supply. **Reference table:** IndependentDemand	Reference	Yes
Index	A number used for sorting records when displaying an indented supply tree. This value is reset for each driving independent demand.	Integer	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
InsideLead Time	Tracks whether the supply is inside its lead time. For PlannedOrders, if the order needs to start before RunDate, shows the number of work days before Run Date to start (otherwise, 0). For released Scheduled Receipts, indicates the supply has a ReschedDate before DataDate. Reports the difference in work days (otherwise 0). If unreleased Scheduled Receipts, indicates the supply has a CalcStartDate before DataDate. Reports the difference in work days (otherwise 0).	Integer	
InventoryTransfer	The inventory transfer supplying this demand. **Referenced table:** InventoryTransfer	Reference	
LaborCost	The cost of labor associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.LaborCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0), if the SupplySource is either 'OnHand', 'InKit', or 'None', as well as for descendants of allocated scheduled receipts. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
Late	Number of periods by which the supply is late to achieve on-time delivery of its driver, based on its AvailableDate. Calculated as supply.AvailableDate - where consumed need date using Part.PlanningCalendars.TimeUnits. This field is a measure of how late the supply availability is.	Integer	
LateAtHere	Number of periods by which the supply is late to satisfy the driver, based on the supply reschedule date. Calculated as (supply. ReschedDate - need date at this part to satisfy driver) using Part.PlanningCalendars.TimeUnits. This field is a measure of how late the supply planning is.	Integer	
Level	Indicates the level in the supply structure that the current supply represents. It can be used when indenting supplies for display in an indented supply tree.	Integer	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
MaterialCost	The cost of materials associated with this allotment of supply. This value is calculated only for onhand inventory supplies, in-kit supplies, effectively purchased supplies, allocated scheduled receipts, and in-transit inventory transfers. For all other supplies, this field always returns a value of zero (0). This field is calculated as *NeedQuantity* * *EffectiveMaterialCost*, where *EffectiveMaterialCost* is set by supply type as follows: • **OnHand**—uses *OnHand.UnitCost* if > 0, otherwise *Part.StdUnitCost* if > 0, otherwise 0. • **InKit**—uses *Part.StdUnitCost* if > 0, otherwise 0. • **Purchased planned orders**—The part source's effective material cost is used. This is calculated as *PartSource.EffectiveMaterialCost * NeedQuantity*. • **Purchased or allocated scheduled receipts**—The scheduled receipt's unit cost is used, if it's a positive value; otherwise the part source's effective material cost is used. This calculated as *(? ScheduledReceipt.UnitPrice > 0 UOMConversion (ScheduledReceipt.UnitPrice) PartSource.EffectiveMaterialCost) * NeedQuantity* • **In-transit inventory transfer**—uses *Part.StdUnitCost* if > 0, otherwise 0. UOMConversion() converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
Model	The model that is used when netting this record. It is set to None if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.ModelRule is 'Ignore' **Referenced table:** Model	Reference	
NeedDate	The date when this supply record is needed in order to achieve on-time delivery for the particular driver. This is reported as 'undefined' for parts where the applicable setting in **Type.AvailableRule** or **AvailableRule.ProcessingRule** is set to 'Ignore', and for parts where demand flows through a parent that is set to 'Ignore'.	Date	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
NeedQuantity	The amount of this supply record that is required to meet the particular demand.	Quantity	
NewProcessCost	Standard labor and overhead cost of planned orders, for this need quantity, to complete this order. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
NewPurchasesCost	Cost of new purchases required, for this need quantity, in order to complete this order (through all levels of the bill). Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
OnHand	The inventory record satisfying this demand (Null unless SupplyType is OnHand). **Referenced table:** OnHand	Reference	
OverheadCost	The overhead cost associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. However, if the SupplySource is 'InventoryTransfer' then the unit TransferCost is used in OverHeadCost. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None', as well as for descendants of allocated scheduled receipts. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
OverheadCost2	The overhead cost associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None', as well as for descendants of allocated scheduled receipts. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
ParentType	Identifies whether this demand is dependent and what type of supply it comes through if so. Valid values are: • **PlannedOrder**—the demand comes through a planned order. The PegCTPPlannedOrder reference is valid. • **ScheduledReceipt**—the demand comes through a scheduled receipt. The PegScheduledReceipt reference is valid. • **InventoryTransfer**— the demand comes through an inventory transfer. The PegInventoryTransfer reference is valid. • **SubstituteSupply**—the demand comes through a substiutute supply. The PegCTPSubstituteSupply reference is valid. • **None**—the demand is not dependent. Neither the PegScheduledReceipt, PegCTPPlannedOrder, PegInventoryTransfer, or PegCTPSubstituteSupply references are valid.	String (Enum)	
Part	The part whose supply is being consumed. **Referenced table**: Part	Reference	
PartSource	The PartSource used by the supply. Note that PartSource.Supplier is the supplier. **Referenced table:** PartSource	Reference	
PegCTPPlanned Order	If dependent demand from a planned order, this contains the CTP information for that order. **Referenced table:** CTPPlannedOrder	Reference	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
PegCTPSubstitute Supply	The parent CTP Substitute Supply associated with this record. **Referenced table:** CTPSubstituteSupply	Reference	
PegInventory Transfer	If dependent demand comes from an inventory transfer, this is a reference to that inventory transfer. **Referenced table:** InventoryTransfer	Reference	
PegPart	Parent part that is the immediate source of this dependent demand (passes through phantoms). **Referenced table:** Part	Reference	
PegScheduled Receipt	The scheduled receipt (if this is a dependent demand from a scheduled receipt) that creates the current demand. **Referenced table:** ScheduledReceipt	Reference	
PegSupplyDemand	A reference to the single-level supply and demand allocation details for the immediate parent order that required the supply reported on this record. This allows reporting SupplyDemand information for any incremental portions of the parent order. **Referenced table:** SupplyDemand **Note:** This field was added in Version 2016.2, Service Update 3.	Reference	
Pool	The pool of the related supply record. **Referenced table:** Pool.Value	Reference	
ScheduledReceipt	The immediate scheduled receipt that is this supply (null unless a scheduled receipt). **Referenced table:** ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the supply, this represents the available date for a portion of the supply. If incremental availability calculations are turned off for the supply, this represents the available date for the entire supply (unless a planned order has been split due to constraint allocation).	Date	
SubstituteGroup	A reference to the substitute group associated with this supply. Null unless the supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt. **Referenced table:** SubstituteGroup	Reference	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
SupplyAvailable Date	The date when the entire supply is available.	Date	
SupplyDemand	The supply and demand being reported by this record. Use SupplyDemand.DemandSource to determine which SupplyDemand references are valid for single level pegging (BOM, SR, and so on). **Referenced table:** SupplyDemand	Reference	
SupplyDueDate	Effective due date of the supply (stock date). This is EffDueDate for ScheduledReceipts, DueDate for PlannedOrders. (Past if OnHand).	Date	
SupplyQuantity	Total quantity of the supply.	Quantity	
SupplyQuantity UnitOfMeasure	Reference to the unit of measure used for the supply quantity associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
SupplySource	The source of the supply record. Valid values are: • **CoByProductSupply**—supply is a co-product or by-product. • **CTPPlannedOrder**—normal planned order • **InKit**-the supply is from an InKit quantity on an input allocation. • **InventoryTransfer**—if the supply is an inventory transfer. • **None**—no supply is available to satisfy this demand • **OnHand**—if the supply is in inventory. • **ScheduledReceipt**—normal scheduled receipt • **SubstituteSupply**—supply comes from a substitute part.	String (Enum)	
SupplyStartDate	The order start date as calculated by netting, including allowance for constraint.	Date	
SupplyStartUnit	The start unit that is used when netting this record. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is 'Ignore'.	Integer	

Table 8.155: WhereConsumedForDemand (Mfg) fields

Field name	Description	Data type	Owner
SupplyTypeString	The type of the supply record. This is a customer-specific value. Valid values are: • **OnHand**—if the supply is in inventory • **SupplyType**—the SupplyType from a scheduled receipt or planned order	String	
SynchronizedDate	The date when this supply is needed in order to achieve completion of the immediate parent supply for its FirstAvailableDate (a stock date). This supply is AvailableDate for IndependentDemand.	Date	

WhereConsumedForForecast table

The **WhereConsumedForForecast** table pegs consensus forecast demands to the set of supplies they consume. It is suitable for top-down analysis to show all supplies, across all levels, that are ultimately consumed by a given forecast.

Each record in this table reports details of a component supply used in satisfying demand from a given forecast. Beyond component supply information, each record also contains details about supply of the driver part (the part the forecast was for) and supply of the immediate parent part (the part that passed dependent requirements to the component). For each driver part for a given forecast, there will also be at least one record that has its ParentType field set to "None" and its Level field set to "0". This is the root record for the forecast driver and only reports details of the supply for the driver part itself.

Note that this table only reports where supplies are to satisfy demand from consensus forecasts generated during the sales and operations planning process in RapidResponse. These are forecasts generated based on weighted combinations of forecast streams in the ForecastDetail table, and appear in the Forecast table with a ForecastSource value of 'ConsensusForecast'. If information on supplies used to satisfy forecasts from IndependentDemand records is required, the **WhereConsumedForDemand** table should be used instead.

Additionally, workbook variables are available that allow a specific workbook/worksheet to limit the results from this table to those whose dates fall within a specified reporting horizon. For more information, see "Specify a reporting horizon for WhereConsumedForForecast records" on page 2229.

This table is not shown on the *RapidResponse Calculated Data Model* poster.

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
ActionSupply	Indicates if the component supply will be late and requires action to achieve on time delivery of the top level driver part. Action is required if either the supply's available date and rescheduled date are later than its need date, the supply is made later by constraint, or the supply is inside its lead time (for more information, see the InsideLeadTime field description). Valid values are: • Y • N	Boolean	
AlternatePart	The name of the component part the supply is for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table:** Part	Reference	
BillOfMaterial	A reference to the BillOfMaterial record that passed the dependent requirement associated with this component supply (if applicable). This is the record that associates the component part with its parent. **Referenced table:** BillOfMaterial	Reference	
CTPCoByProduct Supply	If the component supply is a co-product generated because of the production of a primary part, this field contains a reference to that co-product. **Referenced table:** CTPCoByProductSupply	Reference	
CTPForecast	A reference to details of the forecast demand that is driving the requirement for this component supply. Only those Forecast records where the ForecastSource is 'ConsensusForecast' are reported through this reference. **Referenced table:** CTPForecast	Reference	
CTPPlannedOrder	If the component supply is a planned order, this field contains a reference to that planned order record. **Referenced table:** CTPPlannedOrder	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
CTPSubstitute Supply	If the component supply is from a part substitution, this field contains a reference to that substitute supply record. **Referenced table:** CTPSubstituteSupply	Reference	
DriverAvailable Date	The date that supply of the driver part is available to fully satisfy the forecasted demand.	Date	
DriverDueDate	The date that supply of the driver part is due in order to satisfy the forecasted demand.	Date	
DriverNeed Quantity	The amount of the driver quantity that is ultimately satisfied by the pegged supply. This field indicates how much of the driver demand could be affected by this specific component supply, and hence can be used to determine which supplies are having the greatest impact on a late order. **Note:** This field was added in Version 2016.2, Service Update 4. **Caution:** In linear BOM structures, this field can be used with NeedQuantity to calculate how many units of the component is ultimately needed to make a unit of the driver part. However in non-linear BOM structures, this can give misleading results as different values are likely to be returned based on the different sub-assemblies/paths under the driver where the lower-level component is used.	Quantity	
DriverPart	A reference to the driver part whose forecast is driving the allotment of this component supply. **Referenced table:** Part	Reference	Yes
DriverSupply Demand	A reference to the supply and demand details associated with the root driver for this WhereConsumedForForecast record. **Referenced table**: SupplyDemand	Reference	
DriverQuantity	The amount of the forecast for the driver part.	Quantity	
Index	A number that can be used when sorting records for display in an indented supply tree. Starting with the root record for the latest forecast for a given driver part, this field starts at 0 and increments by 1 as each supply allotment under the driver part is reported.	Integer	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
InsideLeadTime	Indicates the number of work days, if any, that the component supply is inside its lead time. • For planned orders, this field is calculated as the number of days, if any, prior to **Part.PlanningCalendars.RunDate** that the order needs to start. • For released scheduled receipts, this field is calculated as the number of days, if any, that the **ReschedDate** is earlier than **Part.PlanningCalendars.DataDate**. • For unreleased scheduled receipts, this field is calculated as the number of days, if any, that the **CalcStartDate** is earlier than **Part.PlanningCalendars.DataDate**.	Integer	
InventoryTransfer	If the component supply is from an inventory transfer, this field contains a reference to that inventory transfer record. **Referenced table:** InventoryTransfer	Reference	
Late	The number of periods, based on its AvailableDate, by which the component supply is late to achieve on-time delivery of the driver part's forecast. If the component's AvailableDate is later than the NeedDate, this field reports the number of **Part.PlanningCalendars.TimeUnits** between them. Otherwise, this field returns 0 (zero).	Integer	
LateAtHere	The number of periods, based on its DueDate or ReschedDate, by which the component supply is late to achieve on time delivery of the driver part's forecast. If the component's DueDate/ReschedDate is later than the NeedDate, this field reports the number of **Part.PlanningCalendars.TimeUnits** between them. Otherwise, this field returns 0 (zero).	Integer	
LaborCost	The cost of labor associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.LaborCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0), if the SupplySource is either 'OnHand', 'InKit', or 'None'. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
Level	A number indicating the component supply's position in the driver's part's supply structure. Starting at 0 for the driver part's root record, this field is incremented by one for each product structure level that is exploded (for example, a BOM level or a transfer part source). Note that the count is not incremented when passing through a phantom BOM or part. This field can be used when indenting records for display in an indented supply tree.	Integer	
MaterialCost	The cost of materials associated with this allotment of supply. This value is calculated only for onhand inventory supplies, in-kit supplies, effectively purchased supplies, allocated scheduled receipts, and in-transit inventory transfers. For all other supplies, this field always returns a value of zero (0). This field is calculated as *NeedQuantity* * *EffectiveMaterialCost*, where *EffectiveMaterialCost* is set by supply type as follows: • **OnHand**—uses *OnHand.UnitCost* if > 0, otherwise *Part.StdUnitCost* if > 0, otherwise 0. • **InKit**—uses *Part.StdUnitCost* if > 0, otherwise 0. • **Purchased planned orders**—The part source's effective material cost is used. This is calculated as *PartSource.EffectiveMaterialCost * NeedQuantity*. • **Purchased or allocated scheduled receipts**—The scheduled receipt's unit cost is used, if it's a positive value; otherwise the part source's effective material cost is used. This calculated as *(? ScheduledReceipt.UnitPrice > 0 UOMConversion (ScheduledReceipt.UnitPrice) PartSource.EffectiveMaterialCost) * NeedQuantity* • **In-transit inventory transfer**—uses *Part.StdUnitCost* if > 0, otherwise 0. UOMConversion() converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
Model	The model, if any, associated with this component supply. **Referenced table**: Model	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
NeedDate	The date when the component supply is needed in order to achieve on-time delivery of the forecast demand for the driver part. This is reported as 'undefined' for parts where the applicable **Type.AvailableRule** or **AvailableRule.ProcessingRule** setting is 'Ignore', and for parts where demand flows through a parent that is set to 'Ignore'.	Date	
NeedQuantity	The amount of the component supply that is needed in order to satisfy the forecast demand for the driver part.	Quantity	
NewPurchases Cost	Total of standard labor and overhead costs from planned orders for the component supply to satisfy the need quantity. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
NewProcessCost	Total of standard labor and overhead costs from planned orders for the component supply to satisfy the need quantity. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
OnHand	If the component supply comes from on hand inventory, this field contains a reference to that on hand record. **Referenced table**: OnHand	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
OverheadCost	The overhead cost associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. However, if the SupplySource is 'InventoryTransfer' then the unit TransferCost is used in OverHeadCost. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None'. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
OverheadCost2	The overhead cost associated with this allotment of supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None'. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
ParentCTP PlannedOrder	If this component supply satisfies dependent demand from a planned order, this field is a reference to that planned order record. **Referenced table:** CTPPlannedOrder	Reference	
ParentCTP SubstituteSupply	If this component supply satisfies dependent demand from a substitute supply, this field is a reference to that substitute supply record. **Referenced table:** CTPSubstituteSupply	Reference	
ParentPart	A reference to the immediate parent part that drove this allotment of component supply. **Referenced table:** Part	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
ParentInventory Transfer	If this component supply satisfies dependent demand from an inventory transfer, this field is a reference to that inventory transfer record. **Referenced table:** InventoryTransfer	Reference	
ParentScheduled Receipt	If this component supply satisfies dependent demand from a scheduled receipt, this field is a reference to that scheduled receipt record. **Referenced table:** ScheduledReceipt	Reference	
ParentSupply Demand	A reference to the single-level supply and demand allocation details for the immediate parent order that required the supply reported on this record. This allows reporting SupplyDemand information for any incremental portions of the parent order. **Referenced table:** SupplyDemand **Note:** This field was added in Version 2016.2, Service Update 3.	Reference	
ParentType	Identifies whether the demand for this component supply is dependent, and the type of supply that dependency comes through. Valid values are: • **PlannedOrder**—the demand comes through a planned order for a parent part. The ParentCTPPlannedOrder reference is valid. • **ScheduledReceipt**—the demand comes through a scheduled receipt for a parent part. The ParentScheduledReceipt reference is valid. • **InventoryTransfer**— the demand comes through an inventory transfer for a parent part. The ParentInventoryTransfer reference is valid. • **SubstituteSupply**—the demand comes through a substitute supply for a parent part. The PegCTPSubstituteSupply reference is valid. • **None**—the demand is not dependent. For example, the root record for each driver supply has this value.	String (Enum)	
Part	The component part whose supply is being consumed by the driver part forecast. **Referenced table:** Part	Reference	
PartSource	The part source used by the component supply. This reference also provides access to the associated supplier record (through **PartSource.Supplier**). **Referenced table:** PartSource	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
Pool	The pool, if any, this component supply belongs to. **Referenced table**: Pool	Reference	
ScheduledReceipt	If the component supply comes from a scheduled receipt, this field contains a reference to that scheduled receipt record. **Referenced table:** ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the component supply, or the supply comes from a planned order that has been split due to constraint allocation, this represents the date that a portion of the supply is available for use. Otherwise, this is equivalent to SupplyAvailableDate.	Date	
SubstituteGroup	If this component supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt, then this field is a reference to the substitute group that defines that relationship. **Referenced table**: SubstituteGroup	Reference	
SupplyAvailable Date	The date when the entire supply is available.	Date	
SupplyDemand	A reference to the supply and demand associated with the component supply record. **Referenced table**: SupplyDemand	Reference	
SupplyDueDate	The effective due date for the component supply. Set to **ScheduledReceipt.EffDueDate** for scheduled receipts, **PlannedOrder.DueDate** for planned orders, and "Past" if the supply comes from on hand inventory.	Date	
SupplyQuantity	Total quantity of the component supply.	Quantity	
SupplyQuantity UnitOfMeasure	Reference to the unit of measure used for the supply quantity associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	

Table 8.156: WhereConsumedForForecast (Mfg) fields

Field name	Description	Data type	Owner
SupplyStartUnit	The start unit that is used when netting the component supply record. It is set to -1 if either of the following conditions exist: • Model-Unit Effectivity is not enabled • Part.MUEPoolNettingType.UnitRule is 'Ignore'.	Integer	
SupplyStartDate	The date the component supply order is scheduled to be released to the shop floor or supplier.	Date	
SupplySource	The source of the component supply. Valid values are: • **CoByProduct**—the supply is from a co-product part. • **CTPPlannedOrder**—the supply is a normal planned order. • **InventoryTransfer**—the supply is from an inventory transfer. • **None**—indicates that no supply is available to satisfy the demand referenced in the allotment. • **OnHand**—the supply is from inventory. • **ScheduledReceipt**—the supply is a normal scheduled receipt • **SubstituteSupply**—the supply comes from a substitute part.	String (Enum)	
SupplyTypeString	If the component supply is either a scheduled receipt or planned order, this field returns the referenced value in the **SupplyType.Value** field that define characteristics and processing policies associated with that supply. Otherwise, this field returns one of the following strings indicating the supply type: • CoByProduct • FlowToCommon • InKit • InventoryTransfer • None • OnHand • SubstituteSupply	String	
SynchronizedDate	The date when this component supply is needed to achieve completion of its immediate parent supply.	Date	

WhereConsumedForSupply table

This table pegs supplies to the production orders where they are used. It is suitable for top-down analysis to report all supplies, across all levels, required to fill a given production order (for example, in an indented supply tree).

In RapidResponse, every active scheduled receipt, planned order, or substitute supply is reported as a driver supply in this table. For a given driver, one record is added to this table for each supply ultimately required in the production of that driver. Each record in this table identifies the driver supply and contains details about one of its component supplies. As well, the immediate parent part that drove dependent demand to that component is identified.

For each driver supply, there will be at least one record that has its ParentType field set to “None” and its Level field set to “0”. This is the root record for that driver and only reports details of the driver supply itself. If a given driver supply is allotted to multiple demands, there will be one root record for each group of allotments. For any active supply without dependencies (that is purchased supplies or released work orders with no allocations), these root records are the only records reported with that supply as the driver.

Note also that the calculations reported by this table can be limited to a specific number of part levels. For example, if a single level report of consumed supplies is required, the calculation can be limited to a single level, which returns the single-level result without calculating all parts at all levels consumed by the order. Limiting the calculations in this manner might also improve analytic performance and is controlled at the workbook level using an analytic parameter as discussed in "Specifying the number of levels used in WhereConsumedForSupply calculations" on page 2235.

Additionally, workbook variables are available that allow a specific workbook/worksheet to limit the results from this table to those whose dates fall within a specified reporting horizon. For more information, see "Specify a reporting horizon for WhereConsumedForSupply records" on page 2230.

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
ActionSupply	Indicates if the component supply will be late and requires action. Action is required if either the supply’s available date and rescheduled date are later than its need date, the supply is made later by constraint, or the supply is inside its lead time (for more information, see the InsideLeadTime field description). Valid values are: • Y • N	Boolean	
AlternatePart	The name of the component part the supply is for. This value is different from the Part field value only if that part is interchangeable or globally substitutable with this part as follows: • If the part belongs to a set of interchangeable parts, then Part is the primary part and AlternatePart is the part for the input supply data (or for dependent demands, this is where the demand came from). • If the part belongs to a set of globally substitutable parts, then Part is the part that needed the supply and AlternatePart is the part that provided the supply. **Referenced table:** Part	Reference	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
BillOfMaterial	A reference to the BillOfMaterial record that passed the dependent requirement associated with this component supply (if applicable). **Referenced table:** BillOfMaterial	Reference	
CTPCoByProduct Supply	If the component supply is a co-product or by-product generated due to production of a primary product, this field is a reference to that co-product or by-product supply. Otherwise, NULL. **Referenced table:** CTPCoByProductSupply	Reference	
CTPPlanned Order	If the component supply is a planned order, this field contains a reference to that planned order record. **Referenced table:** CTPPlannedOrder	Reference	
CTPSubstitute Supply	If the component supply is from a part substitution, this field contains a reference to that substitute supply record. **Referenced table:** CTPSubstituteSupply	Reference	
DriverAvailable Date	The date the supply driving the allotment is available.	Date	
DriverCTPCoBy ProductSupply	If the supply driving the allotment is a co-product or by-product generated because of the production of a primary part, this field contains a reference to that co-product or by-product supply. **Referenced table:** CTPCoByProductSupply	Reference	
DriverCTP PlannedOrder	If the supply driving the allotment is a planned order, this field contains a reference to that planned order record. **Referenced table:** CTPPlannedOrder	Reference	
DriverCTP SubstituteSupply	If the supply driving the allotment is a part substitution, this field contains a reference to that substitute supply. **Referenced table:** CTPSubstituteSupply	Reference	
DriverDueDate	The date the supply driving the allotment is due.	Date	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
DriverNeed Quantity	The amount of the driver quantity that is ultimately satisfied by the pegged supply. This field indicates how much of the driver supply could be affected by this specific component supply, and hence can be used to determine which supplies are having the greatest impact on a late order. **Note:** This field was added in Version 2016.2, Service Update 4. **Caution:** In linear BOM structures, this field can be used with NeedQuantity to calculate how many units of the component is ultimately needed to make a unit of the driver part. However in non-linear BOM structures, this can give misleading results as different values are likely to be returned based on the different sub-assemblies/paths under the driver where the lower-level component is used.	Quantity	
DriverPart	The name of the part whose supply is driving the allotment. **Referenced table:** Part	Reference	Yes
DriverScheduled Receipt	If the supply driving the allotment is a scheduled receipt, this field contains a reference to that scheduled receipt record. **Referenced table:** ScheduledReceipt	Reference	
DriverSupply Demand	A reference to the supply and demand details associated with the root driver for this WhereConsumedForSupply record. **Referenced table**: SupplyDemand	Reference	
DriverType	The type of supply that is driving the allotment. Valid values are: • CTPPlannedOrder • ScheduledReceipt • SubstituteSupply	String (Enum)	
DriverQuantity	The amount of the supply that is driving the allotment.	Quantity	
Index	A number that can be used when sorting records for display in an indented supply tree. Starting with the root record for the driving supply, this field starts at 0 and increments by 1 as each supply allotment is reported.	Integer	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
InsideLeadTime	Indicates the number of work days, if any, that the component supply is inside its lead time. For planned orders, this field is calculated as the number of days, if any, prior to **Part.PlanningCalendars.RunDate** that the order needs to start. For released scheduled receipts, this field is calculated as the number of days, if any, that the **ReschedDate** is earlier than **Part.PlanningCalendars.DataDate**. For unreleased scheduled receipts, this field is calculated as the number of days, if any, that the **CalcStartDate** is earlier than **Part.PlanningCalendars.DataDate**.	Integer	
Inventory Transfer	If the component supply is from an inventory transfer, this field contains a reference to that inventory transfer record. **Referenced table**: InventoryTransfer	Reference	
LaborCost	Cost of labor for with this allotment of component supply. This value is calculated as *NeedQuantity * UOMConversion (PartSource.LaborCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0), if the SupplySource is either 'OnHand', 'InKit', or 'None'.	Money	
Late	The number of periods, based on its AvailableDate, by which the component supply is late to achieve on time delivery of the driver supply. If the supply's AvailableDate is later than the NeedDate, this field reports the number of **Part.PlanningCalendars.TimeUnits** between them. Otherwise, this field returns 0.	Integer	
LateAtHere	The number of periods, based on its DueDate or ReschedDate, by which the supply is late to achieve on time delivery of the driver supply. If the supply's DueDate/ReschedDate is later than the NeedDate, this field reports the number of **Part.PlanningCalendars.TimeUnits** between them. Otherwise, this field returns 0.	Integer	
Level	A number indicating the component supply's position in the driver's supply structure. Starting at 0 for the driver supply's root record, this field is incremented by one for each product structure level that is exploded (for example, a BOM level or a transfer part source). Note that the count is not incremented when passing through a phantom BOM or part. This field can be used when indenting records for display in an indented supply tree.	Integer	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
MaterialCost	The cost of materials associated with this allotment of component supply. This value is calculated only for onhand inventory supplies, in-kit supplies, effectively purchased supplies, allocated scheduled receipts, and in-transit inventory transfers. For all other supplies, this field always returns a value of zero (0). This field is calculated as *NeedQuantity * EffectiveMaterialCost*, where *EffectiveMaterialCost* is set by supply type as follows: • **OnHand**—uses *OnHand.UnitCost* if > 0, otherwise *Part.StdUnitCost* if > 0, otherwise 0. • **InKit**—uses *Part.StdUnitCost* if > 0, otherwise 0. • **Purchased planned orders**—The part source's effective material cost is used. This is calculated as *PartSource.EffectiveMaterialCost * NeedQuantity*. • **Purchased or allocated scheduled receipts**—The scheduled receipt's unit cost is used, if it's a positive value; otherwise the part source's effective material cost is used. This calculated as *(? ScheduledReceipt.UnitPrice > 0 UOMConversion (ScheduledReceipt.UnitPrice) PartSource.EffectiveMaterialCost) * NeedQuantity* • **In-transit inventory transfer**—uses *Part.StdUnitCost* if > 0, otherwise 0. UOMConversion() converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure.	Money	
Model	The model, if any, associated with the component supply. **Referenced table:** Model	Reference	
NeedDate	The date when the component supply is needed in order to achieve on-time delivery of the driver supply. This is reported as 'undefined' for parts where the applicable **Type.AvailableRule** or **AvailableRule.ProcessingRule** setting is 'Ignore', and for parts where demand flows through a parent that is set to 'Ignore'.	Date	
NeedQuantity	The amount of the component supply that is needed in order to satisfy the driver supply.	Quantity	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
NewProcessCost	Total of standard labor and overhead costs from planned orders for the component supply to satisfy the driver supply's need quantity. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
NewPurchases Cost	Total cost of new purchases required for the component supply (across all BOM levels) to satisfy the driver supply's need quantity. Currency conversions on this field use the **DueDate** field's conversion rate, except in the following cases: • If the supply is an on hand supply, the **Date** field of the on hand record is used. • If the supply is an InKit, the Part.PlanningCalendars.RunDate.FirstDate value is used.	Money	
OnHand	If the component supply comes from on hand inventory, this field contains a reference to that on hand record. **Referenced table:** OnHand	Reference	
OverheadCost	The overhead cost associated with this allotment of component supply. Calculated as *NeedQuantity * UOMConversion(PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. However, if the SupplySource is 'InventoryTransfer' then the unit TransferCost is used in OverHeadCost. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None'.	Money	
OverheadCost2	The overhead cost associated with this allotment of component supply. Calculated as *NeedQuantity * UOMConversion(PartSource.OverheadCost)*, where *UOMConversion()* converts costs expressed in the supplier's unit of measure to costs expressed in the part's unit of measure. This field always returns a value of zero (0) if the SupplySource is either 'OnHand', 'InKit', or 'None'.	Money	
ParentCTP SubstituteSupply	If the component supply satisfies dependent demand from a part substitution, this field is a reference to that substitute supply record. **Referenced table:** "CTPSubstituteSupply table" on page 1003	Reference	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
ParentCTP PlannedOrder	If the component supply satisfies dependent demand from a planned order, this field is a reference to that planned order record. **Referenced table:** CTPPlannedOrder	Reference	
ParentInventory Transfer	If the component supply satisfies dependent demand from an inventory transfer, this field is a reference to that inventory transfer record. **Referenced table:** InventoryTransfer	Reference	
ParentPart	If the component supply satisfies dependent demand, this field is a reference to the immediate parent part that drove that demand. **Referenced table:** Part	Reference	
ParentScheduled Receipt	If the component supply satisfies dependent demand from a scheduled receipt, this is a reference to that scheduled receipt record. **Referenced table:** ScheduledReceipt	Reference	
ParentSupply Demand	A reference to the single-level supply and demand allocation details for the immediate parent order that required the supply reported on this record. This allows reporting SupplyDemand information for any incremental portions of the parent order. **Referenced table:** SupplyDemand **Note:** This field was added in Version 2016.2, Service Update 3.	Reference	
ParentType	If the component supply satisfies dependent demand from a parent part, this field indicates the type of supply that the demand came from. Valid values are: • **Inventory Transfer**—demand came through an inventory transfer. ParentInventoryTransfer reference is valid. • **None**—demand is not dependent. The indicates that the record refers to a supply on the driver part itself. • **PlannedOrder—**demand came through a planned order**.** ParentCTPPlannedOrder reference is valid. • **ScheduledReceipt—**demand came through a scheduled receipt. ParentCTPScheduledReceipt reference is valid. • **SubstituteSupply—**demand came through a part substitution. ParentCTPSubstituteSupply reference is valid.	String (Enum)	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
Part	The component part the supply is for. **Referenced table:** Part	Reference	
PartSource	The source used by the component supply. This reference also provides access to the associated supplier record (through **PartSource.Supplier**). **Referenced table:** PartSource	Reference	
Pool	The pool, if any, for the component supply. **Referenced table:** Pool	Reference	
Scheduled Receipt	If the component supply is a scheduled receipt, this field is a reference to that scheduled receipt record. **Referenced table:** ScheduledReceipt	Reference	
SplitSupply Available	If incremental availability calculations are turned on for the component supply, this field returns the available date associated with a portion of that supply. Otherwise, this field returns the available date for the entire current supply unless the supply has been split due to constraint allocation. For more information about incremental availability, see "Incremental availability calculations" on page 1476.	Date	
SubstituteGroup	A reference to the substitute group associated with this supply. Null unless the supply is for a substitutable component under a given bill of material structure, or associated with a substitutable allocation under a given scheduled receipt. **Referenced table:** SubstituteGroup	Reference	
SupplyAvailable Date	The date when the entire supply is available.	Date	
SupplyDemand	A reference to the supply and demand associated with this record. **Referenced table:** SupplyDemand	Reference	
SupplyDueDate	The date the component supply is due. This field is set to **ScheduledReceipt.EffDueDate** for scheduled receipts, **PlannedOrder.DueDate** for planned orders, and "Past" if the supply comes from on hand inventory.	Date	
SupplyQuantity	The unconsumed total quantity of the component supply.	Quantity	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
SupplyQuantity UnitOfMeasure	Reference to the unit of measure used for the supply quantity associated with this record. **Referenced table:** UnitOfMeasure **Note:** This field was added in Version 2016.2, Service Update 12.	Reference	
SupplySource	A string identifying the source type of the component supply. Valid values are: • **CoByProductSupply**—supply is a co-product or by-product. • **CTPPlannedOrder**—the supply is a normal planned order. • **InKit**—the supply is from an InKit quantity on an input allocation. • **InventoryTransfer**—the supply is an inventory transfer. • **None**—indicates that no supply is available to satisfy the demand referenced in the allotment. For example, this type might typically be seen when the part source has an **OrderPolicy.OrderGenerationRule** of "NoOrders" preventing it from planning new supply. Note that in such cases, the **SupplyQuantity** field is still populated and represents the amount of supply that is short and still needed to actually satisfy the requirement. • **OnHand**—the supply is in inventory. • **ScheduledReceipt**—the supply is a normal scheduled receipt • **SubstituteSupply**—the supply is from a substitute part.	String (Enum)	
SupplyStartDate	The date the component supply order is scheduled to be released to the shop floor or supplier.	Date	
SupplyStartUnit	The start unit number that applies to the component supply. This field is -1 either if Model-Unit Effectivity is not enabled, or the **Part.MUEPoolNettingType.UnitRule** is set to “Ignore”	Integer	

Table 8.157: WhereConsumedForSupply (Mfg) fields

Field name	Description	Data type	Owner
SupplyType String	If the component supply is either a scheduled receipt or planned order, this field returns the associated value in the **SupplyType.Value** field used to define characteristics and processing policies associated with that supply. Otherwise, this field returns one of the following strings indicating the supply type: • FlowToCommon • InKit • InventoryTransfer • None • OnHand	String	
Synchronized Date	The date when the component supply is needed to achieve completion of its immediate parent supply.	Date	

WhereConsumedToHighVolumeOrder table

The **WhereConsumedToHighVolumeOrder** table pegs each component supply to the highest level demand(s) that it ultimately satisfies. This information can be used to determine which end-item demand(s) might be impacted by a change in availability of a given component supply.

This table is intended for use with components used in configurable end-items that are expected to have very large numbers of demands (that is, final assembly parts where **PartType.FeatureBOMRule** is set to "HighVolume"). When component supplies are used to satisfy an independent demand from a high volume end-item, the actual **IndependentDemand** is not reported as the driver on the **WhereConsumed** record. Instead, the **CTPPlannedOrder** record representing the configurable end-item supply created to satisfy that independent demand is used as the driver. This is done for performance reasons as high-volume configurable end-items typically very large numbers of independent demands due on a given date; however, only a single planned order is created to satisfy those demands (within a given priority, model, unit, and pool combination).

For components that are not used in at least one high-volume, configurable end item, (only used by assembly's where **PartType.FeatureBOMRule** is either "Normal" or "Ignore"), the **WhereConsumed** table should be used to report the link between component supplies and the end item demands that consume them instead.

Note also that appropriate filtering should be used with worksheets based on the **WhereConsumedToHighVolumeOrder** table as there can be large numbers of records generated from demand for high volume, configurable end-items. Additionally, two analytic parameters can be configured to limit the date range over which records can be generated in this table as discussed in "Specify a reporting horizon for WhereConsumedToHighVolumeOrder records" on page 2232.

This table is not shown on the *RapidResponse Calculated Data Model Poster*. For more information about the Feature BOM analytic it supports, see "Feature BOMs" on page 1845.

Table 8.158: WhereConsumedToHighVolumeOrder (Mfg) fields

Field name	Description	Data type	Owner
ConstraintNeed Date	The date constraint would be needed for this supply in order to meet the driver demand's due date. Calculated as need date converted to either a ship or start date. **Note:** This field is only populated if the supply associated with the record has a gating constraint (otherwise, it is left blank).	Date	
CTPForecast	A reference to the consensus forecast that ultimately consumes the supply of the component reported on this record. NULL unless **DriverType** is "ConsensusForecast". **Referenced table:** CTPForecast	Reference	
DriverAvailable Date	The date that the driver order will be available.	Date	
DriverOrderId	A string identifying the highest level demand that ultimately consumes supply of the component reported on this record. Values used to identify the driver demand include: • The demand order id if the driver is an actual or forecast demand in **IndependentDemand** table. • A value of "ConsensusForecast" if the driver is a forecast demand in **CTPForecast** table. • A value of "Excess" if the driver is a supply not used to satisfy any demand. • A value of "OnHand" if the driver is a negative on hand value.	String	
DriverPart	A reference to the driver part whose demand ultimately consumes supply of the component part reported on this record. **Referenced table:** Part	Reference	
DriverPart Customer	A reference to the part customer combination associated with the driver demand reported on this record. Otherwise, NULL if the driver does not have a **PartCustomer** reference. **Referenced table:** PartCustomer	Reference	
DriverSupply Demand	A reference to details of the single-level supply to demand allocation between the driver part and its immediate parent (if any). **Referenced table:** SupplyDemand	Reference	

Table 8.158: WhereConsumedToHighVolumeOrder (Mfg) fields

Field name	Description	Data type	Owner
DriverType	Indicates the highest source of demand that is ultimately satisfied by the component supply reported on this record. Valid values are: • **CoByProductSupply**—the supply ultimately satisfies a co-product or by-product generated from a primary product supply. • **ConsensusForecast**—the supply ultimately satisfies a consensus forecast generated during the sales and operations process. The **CTPForecast** reference is valid. • **Forecast**—the supply ultimately satisfies an unconsumed forecast (from independent demand). • **IndependentDemand**— the supply ultimately satisfies an independent demand. The **IndependentDemand** reference is valid. • **InventoryTransfer**—the supply ultimately satisfies an inventory transfer. • **OnHand**—the supply is either excess onhand inventory or is ultimately used to satisfy demand coming from a negative onhand value. If the supply is excess onhand, then **DriverOrderId** is "Excess" and **DriveDueDate** is "Undefined". If the supply is satisfying negative onhand, then **DriverOrderId** is "OnHand" and **DriverDueDate** is "Past". • **ScheduledReceipt**—the supply ultimately satisfies a scheduled receipt not pegged to any demand (thus, it is excess). • **PlannedOrder**—the supply ultimately satisfies a planned order that is not pegged to any demand (typically, this excess is due to safety stock or lot sizing requirements).	String (Enum)	
Independent Demand	A reference to the independent demand that ultimately consumes supply of the component part reported on this record. NULL unless **DriverType** is "IndependentDemand" or "Forecast". **Referenced table:** IndependentDemand	Reference	
NeedDate	The date when supply of the component part is needed to ensure on-time delivery of the higher level demand from the driver part.	Date	
NeedQuantity	The amount of the component part that is required to satisfy demand from the driver reported on this record.	Quantity	
Part	A reference to the component part whose supply is being consumed by the driver demand reported on this record. **Referenced table:** Part	Reference	Yes

Table 8.158: WhereConsumedToHighVolumeOrder (Mfg) fields

Field name	Description	Data type	Owner
WhereConsumed	A reference to the **WhereConsumed** record that ties supply of the component part to the driver demand reported on this record. Note that if the driver part has **PartType.FeatureBOMRule** set to "HighVolume", then the referenced record will report the associated **CTPPlannedOrder** record as the driver rather than the demand itself. This can be used to report the immediate parent part that drove demand to the component, as well as other details on the component supply itself. **Referenced table:** WhereConsumed	Reference	

WorkCenterCapacity table

The **WorkCenterCapacity** table reports effective capacity for a given work center and date. This includes the number of working hours for that date, the number of resources available to work those hours, and any efficiency or utilization factors. Because work center capacity details can be provided as input in the **Capacity**, **CapacityOverride**, and **WorkCenterType** tables, the source of each set of effective capacity values is also indicated.

This table is not shown on the *RapidResponse Calculated Data Model* poster.

Table 8.159: WorkCenterCapacity (Mfg) fields

Field name	Description	Data type	Owner
Capacity	A reference to the **Capacity** record that defines work center capacity for the date reported on this record. Null if **Source** is "CapacityOverride" or "WorkCenterType". **Referenced table:** Capacity	Reference	
CapacityOverride	A reference to the **CapacityOverride** record that defines work center capacity for the date reported on this record. Null if **Source** is "Capacity" or "WorkCenterType". **Referenced table:** CapacityOverride	Reference	
Date	The date to which the work center capacity details reported on this record apply. If **Source** is "Capacity" or "WorkCenterType", then there should only be one record reported per work center on a given working day. However, if **Source** is "CapacityOverride", there might be multiple records reported per date if the work center has different shifts during the day.	Date	

Table 8.159: WorkCenterCapacity (Mfg) fields

Field name	Description	Data type	Owner
Efficiency	The factor used to scale operation setup and run times for this work center. In other words, a measurement of what can hypothetically be produced versus what is actually produced. When multiplied with the Utilization factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Note:** If **Source** is "WorkCenterType", this field always returns one.	Quantity	
NumberOf Resources	The number of resources available at this work center. Available daily capacity at the work center is adjusted using this value which must be non-negative.	Quantity	
Source	Indicates the source of the effective work center capacity values being reported for the date on this record. Valid values are: • **Capacity**—values come from the **Capacity** table. If a work center has an effective record defined in this table, it will be used unless overridden by a **CapacityOverride** record for a given day of the week. • **CapacityOverride**—reported values come from the **CapacityOverride** table. If a work center has an effective record defined in this table for a given day of the week, it will be used. • **WorkCenterType**—reported values come from the defaults defined by the work center's **Type** reference. These defaults, as stored in the **WorkCenterType** control table, are used if neither an effective **Capacity** record nor an effective **CapacityOverride** record exists for the work center.	String (Enum)	
Utilization	The factor used to scale operation setup and run times for this work center to allow for non-productive periods in the day (for example, breaks). When multiplied by the Efficiency factor, the resulting value is the usage factor. This is used for converting the run time and setup time from "standard hours" to real elapsed hours. **Note:** If **Source** is "WorkCenterType", this field always returns 1.	Quantity	

Table 8.159: WorkCenterCapacity (Mfg) fields

Field name	Description	Data type	Owner
WorkCenter	A reference to the work center whose capacity is being reported for the date shown on this record. **Referenced table:** WorkCenter **Note:** For each **WorkCenter** record, a value is reported in this table for each work day up until reaching the reporting limit calculated as: `RunDate + Type.CapacityReportLimit PlanningCalendars.TimeUnits`	Reference	
WorkingHour	Total number of working hours available at the work center on this date.	Quantity	

CHAPTER 9: Notes tables

Notes tables are responsible for storing note data. Note data includes a timestamp, a user ID, and a value.

The following RapidResponse tables have Note fields: **EngineeringChange**, **ForecastItemParametersOutlier**, **IndependentDemand**, **ScheduledReceipt, SupplyAllocation**, **SupplyOrder**, and **Task**. When a note value is entered into a Note field located in one of these tables, RapidResponse creates a note record and stores it in the corresponding Note table. For example, when a note is created for the IndependentDemand.Notes field, RapidResponse creates a note record and stores it in the **IndependentDemand_Notes** table.

If a Note field is added to a custom table, then RapidResponse creates a corresponding Note table. For example, assume a Note field named **Notes** is added to a custom table named **Location**. The name of the corresponding note table that gets created is **Location_Notes**. Each time a note value is added to the Note field in the **Location** custom table, RapidResponse creates a note record and stores it in the **Location_Notes** table.

If a Note field is added in a different namespace than the table is in, the table's namespace is reflected in the name of the Note table. For example, assume a custom **Notes** field in the **User** namespace is added to the **DemandOrder** table in the **Mfg** namespace, RapidResponse creates a Note table in the **User** namespace named **Mfg__DemandOrder_Notes**.

Notes tables are not displayed in the Data Model dialog box, and cannot be modified. Notes tables contain only the fields that are automatically added to them on creation. You also cannot map data to the fields in Notes tables, and they are not displayed in the Data Sources and Mapping window.

In this chapter, you'll learn about the following:

EngineeringChange_Notes table 1270
ForecastItemParametersOutlier_Notes table 1270
IndependentDemand_Notes table 1270
IndependentDemandSolution_CustomerNotes table 1271
IndependentDemandSolution_SupplyNotes table 1271
Mfg_SupplyOrder_TextLine table 1272
ScheduledReceipt_Notes table 1272
SupplyAllocation_Notes table 1273
Task_Notes table 1273

Note: The Notes tables described in this chapter are not shown in any of the data model posters included with RapidResponse.

EngineeringChange_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **EngineeringChange** table.

Table 9.1: EngineeringChange_Notes (Mfg) fields

Field name	Description	Data type	Key
Engineering Change	A reference to the engineering change record this note was added to. **Referenced table**: EngineeringChange	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserID	The ID of the user who created the note.	String	
Value	The text of the note.	String	

ForecastItemParametersOutlier_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **ForecastItemParametersOutlier** table.

Table 9.2: ForecastItemItemParametersOutlier_Notes (Mfg) fields

Field name	Description	Data type	Key
ForecastItem Parameters Outlier	A reference to the record this note was added to. **Referenced table**: ForecastItemParametersOutlier	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

IndependentDemand_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **IndependentDemand** table.

Table 9.3: IndependentDemand_Notes (Mfg) fields

Field name	Description	Data type	Key
Independent Demand	A reference to the independent demand record this note was added to. **Referenced table**: IndependentDemand	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

IndependentDemandSolution_CustomerNotes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the **CustomerNotes** field in the **IndependentDemandSolution** table. The **CustomerNotes** field is used by the Order Fulfillment application to record customer-requested changes made against individual line items during the order fulfillment process. Summary information, such as a count of the number of customer changes made against a given line item, can subsequently be retrieved using the **CustomerNotes** field.

Table 9.4: IndependentDemandSolution_CustomerNotes (Solutions) fields

Field name	Description	Data type	Key
Independent DemandSolution	References the **IndependentDemandSolution** record for the line item this change note applies to. **Referenced table:** IndependentDemandSolution	Reference	
TimeStamp	The date and time the note was created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

IndependentDemandSolution_SupplyNotes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the **SupplyNotes** field in the **IndependentDemandSolution** table. The **SupplyNotes** field is used by the Order Fulfillment application to record supply-related changes against individual line items during the

order fulfillment process. Summary information, such as a count of the number of supply related changes made agains a given line item, can subsequently be retrieved using the **Supply Notes** field.

Table 9.5: IndependentDemandSolution_SupplyNotes (Solutions) fields

Field name	Description	Data type	Key
Independent Demand Solution	References the **IndependentDemandSolution** record for the line item this change note applies to. **Referenced table:** IndependentDemandSolution	Reference	
TimeStamp	The date and time the note was created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

Mfg_SupplyOrder_TextLine table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the TextLine field in the SupplyOrder table.

Table 9.6: Mfg__SupplyOrder_TextLine (SupCollab) fields

Field name	Description	Data type	Key
SupplyOrder	A reference to the supply order record this note was added to. **Referenced table:** SupplyOrder (Mfg)	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

ScheduledReceipt_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **ScheduledReceipt** table.

Table 9.7: ScheduledReceipt_Notes (Mfg) fields

Field name	Description	Data type	Key
Scheduled Receipt	A reference to the scheduled receipt record this note was added to. **Referenced table**: ScheduledReceipt	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

SupplyAllocation_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **SupplyAllocation** table.

Table 9.8: SupplyAllocation_Notes (Mfg) fields

Field name	Description	Data type	Key
Supply Allocation	A reference to the supply allocation record this note was added to. **Referenced table**: SupplyAllocation	Reference	
TimeStamp	The date and time the note is created.	DateTime	
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

Task_Notes table

RapidResponse creates a new record and stores it in this table each time a note value is entered into the Note field in the **Task** table.

Table 9.9: Task_Notes (ProjMgmt) fields

Field name	Description	Data type	Key
Task	A reference to the task record this note was added to. **Referenced table**: Task	Reference	
TimeStamp	The date and time the note is created.	DateTime	

Table 9.9: Task_Notes (ProjMgmt) fields

Field name	Description	Data type	Key
UserId	The ID of the user who created the note.	String	
Value	The text of the note.	String	

CHAPTER 10: Referenced system tables

RapidResponse includes tables that provide system functionality. You cannot create worksheets based on system tables, but you can modify the appearance of worksheets based on system tables in system workbooks. Some system tables are referenced by other tables, and their fields can be viewed in worksheets.

These tables do not display in the Data Model dialog box, however, you might be able to map data to them using the Data Sources and Mappings window. Some of these system tables can also be included in the data update process. These tables are not included in the Data Origin workbook and their origin is not tracked. For more information, see the *RapidResponse Administration Guide*.

In this chapter, you'll learn about the following:

Currency table **1275**
CurrencyConversionActual table **1276**
RParameterName table **1277**
TimeZone table **1278**

Currency table

Each currency defined in your system is stored in this table. Currencies are defined by an administrator using the **Administration** pane in RapidResponse. Records should be added to this record only through the administration dialog boxes, not added directly to this table by inserting records or through data updates. For more information, see the *RapidResponse Administration Guide*.

Table 10.1: Currency fields

Field name	Description	Data type	Key
Description	Information about the currency.	String	
IsBase	Identifies whether this currency is the base currency. The base currency is used to perform all currency conversions. Valid values are: • **Y**—The currency is the base currency. • **N**—The currency is not the base currency.	Boolean	

Table 10.1: Currency fields

Field name	Description	Data type	Key
Name	The currency's name.	String	
Symbol	The symbol used to denote the currency.	String	
Value	The currency's code.	String	PK

Currency table - calculated and set fields

The following describes the calculated and set fields in the Currency table. Each field represents the set of records on another table that references the Currency table.

Table 10.2: Calculated Currency fields

Field name	Description	Data type	Key
Currency Conversion Actuals	The set of CurrencyConversionActual records associated with this currency.	Set	
Currency Conversion Forecasts	The set of CurrencyConversionForecast records associated with this currency	Set	
Sites	The set of Site records associated with this currency	Set	

CurrencyConversionActual table

Contains currency conversion rates, which are available in all scenarios. The latest rate with a conversion date on or before the current date is used to convert values. If a rate is modified, the same modification is present in all scenarios.

This table is a system table that is updated similarly to an input table. Updated rates should be brought in to this table using data updates, which makes the updated rates available in all scenarios.

If a new currency is added, you must update the data extracts and integration to include the conversion rates for the new currency, which ensures the currency returns meaningful values.

Any conversion rates specified for the base currency are ignored.

The data in this table is updated by performing data updates. However, this table is updated only if the data update provides data for the root scenario. This ensures actual conversion rates are not accidentally added. Data updates can only add records to this table.

If multiple data updates provide a record for a specific date, each of those records are added to the table. If multiple conversion rates are provided for a date, the rate that is used to perform conversions

cannot be predicted, which can result in incorrect results. When you provide data for this table, you should ensure only one rate is provided for a date.

Table 10.3: CurrencyConversionActual fields

Field name	Description	Data type	Key
Currency	The currency type. **Referenced table**: Currency	Reference	
Date	The date the conversion rate was calculated on.	Date	
Rate	The conversion rate as of the specified date.	Quantity	

RParameterName table

The **RParameterName** table is used to identify and maintain function names and the arguments that relate to each function in the R programing language. You can pass arguments to functions when using the "Rforecast" model to generate a statistical forecast. The following R functions are supported: "ets" and "forecast". The following table lists the arguments that are valid for each of the functions in RapidResponse:

ets	forecast
• model • alpha • beta • gamma • additive.only • lambda • biasadj • opt.crit • nmse • bounds • ic • restrict • allow.mulitplicative.trend	• lambda • allow.mulitplicative.trend • model

Note: The following values are ignored if defined as R parameters: damped, fan, find. frequency, h, level, lower, object, phi, upper, y. Most of these are automatically passed to R based on other field and data values in RapidResponse.

You need to specify argument and function names exactly as they are defined in the R programming language (including case), and values in this table are referenced by other tables in which R parameter values are defined. You can add records to this table using the **Application Settings** system workbook. If values are not added to this table, then optional parameters cannot be passed to R and the "Rforecast" model will run with a standard set of parameters instead.

This table was added in Version 2015.3, and supports the optional R forecast capability.

Table 10.4: RParameterName fields

Field name	Description	Data type	Key
Description	A description of this parameter.	String	
Function	The name of the statistical function in R to which the parameter applies. You can send arguments to R using the following functions: • ets • forecast	String	PK
Name	The name of the optional argument to be passed to the function. **Note:** For a detailed list of the valid arguments for each of the functions, see the table description above.	String	PK

TimeZone table

Contains all time zones defined in RapidResponse and specifies when daylight saving time begins and ends for each time zone.

Table 10.5: TimeZone fields

Field name	Description	Data type	Key
DSTName	The daylight saving time name. For example, Eastern Daylight Time.	String	
DSTOffset	How many minutes the time changes by when daylight saving time begins.	Quantity	
DSTStartDN	The day that daylight saving time begins on. This value is used with the **DSTStartDOW** value to determine which day daylight saving time begins on. Valid values are: • **1**—The **DSTStartDOW** value in the first week of the month. • **2**—The **DSTStartDOW** value in the second week of the month. • **3**—The **DSTStartDOW** value in the third week of the month. • **4**—The **DSTStartDOW** value in the fourth week of the month. • **5**—The **DSTStartDOW** value in the last week of the month. This value always represents the last week, even if the month does not contain five weeks. For example, to specify the first Sunday in a month, specify 0 for the **DSTStartDOW** field and 1 for this field.	Quantity	

Table 10.5: TimeZone fields

Field name	Description	Data type	Key
DSTStartDOW	The day of the week that daylight saving time begins on. Valid values are: • **0**—Sunday • **1**—Monday • **2**—Tuesday • **3**—Wednesday • **4**—Thursday • **5**—Friday • **6**—Saturday	Quantity	
DSTStartMonth	The month that daylight saving time begins on. Valid values are: • **0**—This time zone does not participate in daylight saving time. • **1**—January • **2**—February • **3**—March • **4**—April • **5**—May • **6**—June • **7**—July • **8**—August • **9**—September • **10**—October • **11**—November • **12**—December	Quantity	
DSTStartTime	The time of day that daylight saving time begins on, represented using a 24-hour format.	Time	
IsServer Timezone	Whether this time zone is the same time zone the RapidResponse Server computer is located in.	Boolean	
STName	The standard time name. For example, Eastern Standard Time.	String	
STOffset	How many minutes the time changes by when standard time begins.	Quantity	

Table 10.5: TimeZone fields

Field name	Description	Data type	Key
STStartDN	The day that standard time begins on. This value is used with the **STStartDOW** value to determine which day daylight saving time begins on. Valid values are: • **1**—The **STStartDOW** value in the first week of the month. • **2**—The **STStartDOW** value in the second week of the month. • **3**—The **STStartDOW** value in the third week of the month. • **4**—The **STStartDOW** value in the fourth week of the month. • **5**—The **STStartDOW** value in the last week of the month. This value always represents the last week, even if the month does not contain five weeks. For example, to specify the last Sunday in a month, specify 0 for the **STStartDOW** field and 5 for this field.	Quantity	
STStartDOW	The day of the week that standard time begins on. Valid values are: • **0**—Sunday • **1**—Monday • **2**—Tuesday • **3**—Wednesday • **4**—Thursday • **5**—Friday • **6**—Saturday	Quantity	
STStartMonth	The month that standard time begins on. Valid values are: • **0**—This time zone does not participate in daylight saving time. • **1**—January • **2**—February • **3**—March • **4**—April • **5**—May • **6**—June • **7**—July • **8**—August • **9**—September • **10**—October • **11**—November • **12**—December	Quantity	

Table 10.5: TimeZone fields

Field name	Description	Data type	Key
STStartTime	The time of day that standard time begins on, represented using a 24-hour format.	Time	
Value	The time zone's name.	String	PK

Part 2: Analytics

- Overview of RapidResponse analytics
- Calendars and date calculations
- Lead time calculations
- Material Requirements Planning calculations
- Master Production Scheduling calculations
- Inventory turns calculations
- Capable-to-Promise calculations
- Full-level pegging calculations
- Cost of goods sold calculations
- Cost roll-up calculations
- Capacity Requirements Planning (CRP) calculations
- Kanban calculations
- Interchangeable parts
- Global part substitution
- BOM level part substitution
- Co-products and by-products
- Material expiration
- Constraint Analysis calculations
- Production Planning calculations
- Campaign planning
- Distribution planning
- Multi-site calculations
- Model and Unit Effectivity calculations
- Inventory Pooling calculations
- Alternate BOM calculations
- Order priority calculations
- Two-date planning
- Attribute-based planning
- Feature BOMs
- Allotment overrides
- Multi-level search logic
- Inventory planning and optimization
- Statistical forecasting

- Forecast disaggregation
- Event Management
- Self-Healing Supply Chain
- Project management calculations
- Configuring analytic calculations using global settings
- Configuring analytic calculations using workbook parameters

CHAPTER 11: Overview of RapidResponse analytics

RapidResponse analytics perform calculations on your input data and generate calculated output. These analytics can be configured through control settings as discussed in "Control tables" on page 669. RapidResponse also includes modifiers that can extend and modify analytic calculations through the introduction of additional logic.

In this chapter, you'll learn about:

Core analytics **1285**
Analytic modifiers **1287**
Analytic compatibility and limitations **1291**

Core analytics

RapidResponse core analytics, both standard and optional, contain the calculations and formulas that are applied to your input data. Some of these analytics mimic the calculations found in standard ERP systems, and others perform calculations unique to RapidResponse. The results of these calculations are stored in RapidResponse calculated tables and fields.

The following table describes RapidResponse core analytics.

Table 11.1: RapidResponse core analytics

Analytic	Function
Material Requirements Planning (MRP)	Emulates the Material Requirement Planning (MRP) analytics found in common MRP and ERP systems. This includes netting, explosion, and safety stock calculations. • **Netting**—calculates net requirements for a part by matching demand with available inventory and scheduled receipts, provides reschedule recommendations for existing supplies, and recommends new planned orders (subject to any lot-sizing parameters) as necessary to satisfy the demands. • **Explosion**—calculates dependent demands for component parts, through all levels in the bill-of-material structure, that are ultimately required in the production of a parent assembly. • **Safety stock**—calculates and maintains a defined inventory level for each part, based on configurable fixed and dynamic methods, to protect against fluctuations in supply and demand. For more information, see "Material Requirements Planning calculations" on page 1331.
Capable-to-Promise (CTP)	Uses a patented method to allocate supplies to demands and determine realistic order available dates based on material availability and resource capacity throughout the supply chain. Where orders are late, CTP calculations can be used to report on-time and late quantities as well as to identify the gating part associated with those late demands. For more information, see "Capable-to-Promise calculations" on page 1453.
Full-Level Pegging (FLP)	Pegs lower level supplies back to the end item independent demands and higher level driver supplies that ultimately consume them. This makes it possible to determine which customer orders might be impacted by changes in supply availability, and further allows reporting of all component supplies required to fill a given demand or supply order. For more information, see "Full-level pegging calculations" on page 1493.
Forecast Consumption	Utilizes configuration parameters to determine how forecast on MPS parts is consumed by actual customer orders, and then produces a time phased unconsumed forecast table. For more information, see "Forecast consumption" on page 1431.
Available-to-Promise (ATP)	Calculates the amount of inventory and scheduled supply for MPS parts that is not committed to actual customer orders. This uncommitted supply is then shown as available to satisfy new demands (unconsumed forecast is considered available). For more information, see "Available to promise" on page 1449.
Bill of Material	Identifies all components at different levels used to make a part, and all assemblies at different levels where a part is used. Also identifies all end-items that use a particular component. The output from this analytic is stored in the FlatBillDown and FlatBillUp tables. For more information, see "FlatBillDown table" on page 1046 and "FlatBillUp table" on page 1051.
Capacity Requirements Planning (CRP)	Reports machine and labor profile for comparison with work center capacity to help identify work centers that are over or under utilized, manage resource load, and balance demand with the most efficient resource utilization over time. For more information, see "Capacity Requirements Planning (CRP) calculations" on page 1515.
Inventory Analysis	Helps identify projected inventory trends by reporting ending inventories, work in-process quantities, and supply & demand data grouped by part. Projected inventory turns over the next 365 days are also calculated. The output from this analytic is stored in the InventoryAnalysis and InventoryCTPAnalysis tables. For more information, see "InventoryAnalysis table" on page 1078 and "InventoryCTPAnalysis table" on page 1081.

Table 11.1: RapidResponse core analytics

Analytic	Function
Financial Analysis	Supports more informed business decisions and cost efficiencies based on the following calculations: • Projected Cost of Goods Sold—calculates the expected cost of orders over time, considering parameters such as actual cost from purchase orders and source dependent manufacturing costs. For more information, see "Cost of goods sold calculations" on page 1503. • Standard Cost Roll-Up—provides the ability to project the standard cost of an assembly or end-item for any future date, considering variables such as effective part sources, bill structure modifications, and changes in component part costs. For more information, see "Cost roll-up calculations" on page 1511.
Inventory Planning and Optimization	Supports improved planning of inventory levels to achieve stated customer service goals while also giving the option to minimize the cost from those inventory levels through strategic placement of that inventory. Two key capabilities are supported: • Single-echelon inventory planning—generates safety stock recommendations for a single part and site based on historical demand variability and, optionally, historical supply lead time variability. These recommendations are intended to achieve a specified service level for the part. • Multi-echelon inventory optimization— generates safety stock recommendations simultaneously across a network of related parts under one or more customer facing end-items, based on historical demand variability. The recommendations are meant to achieve specified service levels and service times for the end items, while also minimizing the overall inventory holding cost across all parts in the network. For more information, see "Inventory planning and optimization" on page 1935.

Analytic modifiers

RapidResponse uses analytic modifiers to extend and modify analytic calculations through the introduction of additional logic, as well as various infrastructure calculations that are utilized by one or more analytics. Modifiers introduce additional logic to be considered by core RapidResponse analytics, and therefore can impact the output generated by the analytics.

For example, the optional Constraint Analysis modifier enables CTP analytics to consider the impact of multiple capacity constraints associated with the sources of supply.

Constraints on a supply source might include the number of hours per day a machine can operate or the number of units per day a production line can produce. When these constraints are introduced, RapidResponse core analytics might produce different calculated output, such as different available dates associated with orders than would be the case without any supply constraints being considered.

The following table describes RapidResponse analytic modifiers.

Table 11.2: RapidResponse analytic modifiers

Modifier	Description
Multi-site	Supports analytic calculations across multiple sites in a company. This enables inter-site planning and analysis while supporting the transfer of supply and demand data between multiple site types, such as plants, distribution centers, and warehouses. For more information, see "Multi-site calculations" on page 1731.
Multi-sourcing	When replenishing a part, allows for consideration of multiple sources of supply each with potentially different planning characteristics. Parameters such as priority and target percentage can be used to control the distribution supply allotments (for example, they might reflect contractual requirements with suppliers). For more information, see "Planned orders" on page 1343.
Model and unit effectivity	Allows for models or units of a common part to be separated and netted independently of one another. Models are used where several variants of a configurable part exist and ensure that supply for a given model is allocated to demand for that model. Units are used when sequence designators (for example, serial numbers) are required for a part's supply or demand and ensure that supply for a given unit is allocated to demand for that same unit. For more information, see "Model and Unit Effectivity calculations" on page 1737.
Inventory pooling	Enables effective management of different pools of inventory for a part, and ensures that a pool of supplies and demands for a part can be segregated from other pools during netting. For example, pools might be used to prevent the intermingling of inventories for a given part that is associated with different customers, contracts, or projects. Enabling this analytic also enables the use of the Pool control in worksheets. However, even if this capability is not enabled, you can still enable forecast consumption by customer pool through use of the appropriate settings in the MUEPoolNetting.PoolRule field. For more information, see "Inventory Pooling calculations" on page 1745.
Attribute-based planning	Allows for supply of a particular part to be allotted to its demands based on particular attributes associated with the supplies and demands themselves. Attributes are custom data model elements that can be used to identify characteristics of supplies or demands that should have meaning in the supply process. For example, demands from particular customers might only be able to accept supply of the part if it was assembled at certain locations or contains components sourced from an approved list of suppliers. For more information, see "Attribute-based planning" on page 1817.
Interchangeable parts	Enables the grouping together of parts that are considered equivalent in form, fit, and function, and allows demand for any part in the interchangeable group to be satisfied by existing supply from any other part in the group. When new supply is required, it is created only on the part defined as the primary part in the group. For more information, see "Interchangeable parts" on page 1545.
Global part substitution	Allows for a primary part to have one or more similar parts defined as substitutes, from which existing supply can be used, or new supply planned, to satisfy demands for the primary part. This can help improve demand satisfaction on the primary part while also reducing the need for new planned orders and burning off excess inventory on the substitute parts. Can be extended to support different part substitutions on a customer by customer basis. For more information, see "Global part substitution" on page 1551.
BOM-level part substitution	Within a given assembly's BOM, allows a primary component to have one or similar components defined as substitutes from which existing supply can be used, or new supply planned, to satisfy dependent demands passed down from the assembly. This can help improve demand satisfaction on the assembly while reducing planned requirements on the primary component and burning off excess inventory on its substitutes. Can be extended to support substituting entire groupings of components within an assembly. For more information, see "BOM level part substitution" on page 1565.

Table 11.2: RapidResponse analytic modifiers

Modifier	Description
Constraint Analysis	Allows for the definition of fixed and variable rate-based constraints. These might typically represent the capacity of a process or supplier; for example, the number of hours per day a production line can operate or the number of units per day it can produce. By respecting material and resource constraints in analytic calculations, more realistic and achievable operating plans can be produced, while period load reporting can help identify any overloaded or underloaded constraints. For more information, see "Constraint Analysis calculations" on page 1629.
Production Planning	Models available capacity to balance part production using production wheel cycles to manage supplies and meet demand. Feasible production planning can minimize the time and costs of production and improve resource utilization. Production Planning is a complement to Capacity Planning (Constraints). For more information, see "Production Planning calculations" on page 1691.
Campaign Planning	Ensures that all planned orders generated from a given part source are in a defined batch quantity, with minimums and maximum on the number of batches that make up a given campaign. When scheduling a campaign through a constraint, all batches that make up the campaign must then be scheduled through contiguous blocks of the constraint. This capability is meant to support process industries where supply can only be produced in certain batches sizes, or where there are lengthy or expensive setup times associated with a constraint (typically, equipment), or where production efficiencies are realized as more batches of a part are produced. For more information, see "Campaign planning" on page 1707.
Alternate BOM	Allows a single assembly to be configured with more than one Bill Of Material each of which is modeled separately. For example, a single assembly might be sourced from multiple suppliers or produced on separate lines in a plant using slightly different component variants. For more information, see "Alternate BOM calculations" on page 1759.
Feature BOMs	Supports configurable end-item assemblies which might require, or not require, specific components or groups of components based on the group of features selected on a given demand. This allows a single assembly to be used to represent different configurations or models of the same base part. For more information, see "Feature BOMs" on page 1845.
Incremental Availability	Enables the reporting of available dates for partial supply segments taking into account all material and constraint availability. For example, when there is insufficient supply available on time, this makes it possible to show when different portions of a late customer order will become available. In cases where it is possible to split the order into partial shipments, this can lead to improved customer order satisfaction. For more information, see "Incremental availability calculations" on page 1476.
Priority	Allows different levels of priority or importance to be assigned to customer orders. When there is insufficient supply to satisfy all demands on time, this ensures that higher priority orders receive scarce supplies before lower priority orders (for example, priority might be given to high-value customer orders). Can also be used to define priority amongst supplies in environments with resource or material constraints. For more information, see "Order priority calculations" on page 1765.
Fair/equal share	When there is insufficient supply to satisfy all demands on time, fair and equal share logic can be used to ensure that the earliest available supply is shared between orders. Available supply can be shared either equally between all orders or proportionally based on order size. For more information, see "Allocating limited supplies" on page 1464.

Table 11.2: RapidResponse analytic modifiers

Modifier	Description
Two-date planning	Allows independent demands of specified priority levels to be planned to the original customer request dates as possible. This occurs after the capable-to-promise analytic first plans all demands to their due dates. A second pass calculation then attempts to satisfy the request date, and thereby improve customer satisfaction, on higher priority orders. Note that shifting supply allotments in order to achieve these customer request dates can adversely impact the availability of certain lower priority orders. For more information, see "Two-date planning" on page 1803.
Days of supply	Accumulates demand requirements over a specified time period and then attempts to satisfy those accumulated demands with a single planned order (subject to any other order policies). This can lead to fewer planned orders being created and a resultant reduction in the setup costs associated with new order generation. For more information, see "Days of Supply calculations" on page 1362.
Material expiry	Supports the modelling of items that are subject to material expiration by determining an expiry date on supplies and ensuring that a given demand can only be satisfied by supply that expires late enough to meet its shelf life requirements. Also provides visibility into the inventory risk from supplies projected to expire before they can be used, and identifies any demands that cannot be satisfied due to their expiry requirements. For more information, see "Material expiration" on page 1601.
Co-products and By-products	Supports configurations where multiple distinct parts emerge from a single primary part's production process. These parts can be either co-products (real parts which might typically be lower or higher grade versions of the primary part), or by-products (residual parts which might typically be seen as scrap with value). Planned supplies for the primary part are inflated to account for the expected co-product and by-product yield, and any of the generated co-product or by-product supplies are available to satisfy demands for their parts. For more information, see "Co-products and by-products" on page 1581.
Kanban Analysis	Supports kanban replenishment systems by ensuring that any new planned supplies are created to match the current approved bin quantity while also ensuring that the total net supply does not exceed the product of the current approved bin size and current approved bin quantity. For more information, see "Kanban calculations" on page 1541.
Allotment Overrides	Typically configured through a predefined RapidResponse resource, allotment overrides allow for component supply quantities to be reserved towards specific top-level demands and thereby processed before the normal allotment of supply to demand. This provides a technique for making manual refinements to supply allotments in RapidResponse in cases where the required demand availabilities and specific business goals are impractical to achieve using other analytic modifiers. For more information, see "Allotment overrides" on page 1853.
Multi-level search logic	Multi-level search logic is useful for parts where better part sourcing or substitution decisions could be made if knowledge of child component materials and constraints were known, as well as for parts where better supply allotment decisions could be made if knowledge of sibling component allocations were known. These decisions might then typically be reflected in improvements in on time demand availability. For more information, see "Multi-level search logic" on page 1863
Engineering Change	Engineering change helps you analyze engineering changes to determine the best date at which to introduce an engineering change. Choosing the right date can lower the cost of implementing an engineering change; it can help you reduce excess or obsolete inventory, and ensure customer satisfaction by eliminating late orders. The engineering change analytic supports the optional Engineering Change Manager capability. This capability consists of predefined resources including the Engineering Change Data and Engineering Change Manager workbooks. Calculations are performed in the workbooks. For more information see the *RapidResponse Administration Guide*.

Analytic compatibility and limitations

Some analytics and calculations might be disabled, or have certain limitations placed on them, when used in combination with other analytics. The following table highlights combinations of analytics that have potential compatibility issues. Click an X in the table to learn more about the limitation associated with a particular analytic combination.

Table 11.3: Analytic limitations

	Attribute-based planning	BOM Level Part Sub.	Campaign Planning	Co- and By- product	CRP Variable Lead Time	Days of Supply	Distribution Planning	Fair Share Constraint	Feature BOM	Global Part Sub.	Incremental availability	Kanban	Material Expiry	Multi-level search	Order Point	Safety Stock	Two-date Planning
Allotment Override	X		X											X			
ATP and CTP Rule													X				
Attribute-based planning						X	X							X		X	
BOM Substitution				X					X	X				X		X	
Campaign Planning						X		X	X			X					X
Co-Products	X											X					
Constrained Plannning					X	X						X		X	X		
Days of Supply	X		X					X		X			X	X	X		X
Dependent Forecast Consumption														X			
Dependent Unit Rule				X													
Explode Negatives				X													

Table 11.3: Analytic limitations

	Attribute-based planning	BOM Level Part Sub.	Campaign Planning	Co- and By- product	CRP Variable Lead Time	Days of Supply	Distribution Planning	Fair Share Constraint	Feature BOM	Global Part Sub.	Incremental availability	Kanban	Material Expiry	Multi-level search	Order Point	Safety Stock	Two-date Planning
Fair/Equal Share Allocation	X								X				X				X
Fair Share Constraint			X			X											X
Feature BOM		X	X							X	X	X		X	X	X	X
Fixed Constraint								X			X			X			
Forecast Consumption		X								X							
Full Level Pegging									X								
Global Part Substitution		X				X			X			X		X		X	
Ignore Assembly Yield				X													
Ignore Unsatisfied Demand								X									
Incremental Availability					X				X			X					
Input Supply Usage								X	X					X			
In-transit scheduled receipts	X																
Interchangeable Parts												X					
Inventory pooling															X	X	
Kanban	X		X	X					X	X	X			X	X		
Lot Sizing							X	X	X					X			X
Lot Size Last Order	X												X				

Table 11.3: Analytic limitations

	Attribute-based planning	BOM Level Part Sub.	Campaign Planning	Co- and By- product	CRP Variable Lead Time	Days of Supply	Distribution Planning	Fair Share Constraint	Feature BOM	Global Part Sub.	Incremental availability	Kanban	Material Expiry	Multi-level search	Order Point	Safety Stock	Two-date Planning
Material Expiry	X				X	X	X							X			X
Model/unit																X	
MPS Configuration		X		X	X				X	X				X			
Multi-Level Search	X	X			X	X			X	X						X	
Multi-Sourcing												X		X			
Negative dependent demand	X																
Option Ratio				X													
Order Point	X				X	X			X			X		X			
Order Priority							X					X	X	X			
Phantom Processing				X						X							
Production Planning			X						X			X	X	X			X
PTFDate						X											
Safety Lead Time															X		
Safety Stock	X	X							X	X				X			
Schedule Priority			X					X									X
Split Late Supply			X														
Scrap				X													

Table 11.3: Analytic limitations

	Attribute-based planning	BOM Level Part Sub.	Campaign Planning	Co- and By- product	CRP Variable Lead Time	Days of Supply	Distribution Planning	Fair Share Constraint	Feature BOM	Global Part Sub.	Incremental availability	Kanban	Material Expiry	Multi-level search	Order Point	Safety Stock	Two-date Planning
TaktTime			X									X		X	X		
Two-date planning			X			X	X	X	X				X				

CHAPTER 12: Calendars and date calculations

This section introduces the different date values that correspond to manufacturing events and appear in RapidResponse tables. It also outlines how date calculations are performed.

In this chapter, you'll learn about:

Calendar tables 1295
Dates and time spans 1296
RapidResponse date values 1297
Date constants 1303
RunDate and DataDate 1303
Date calculation rules 1304
Calendar date display values 1306

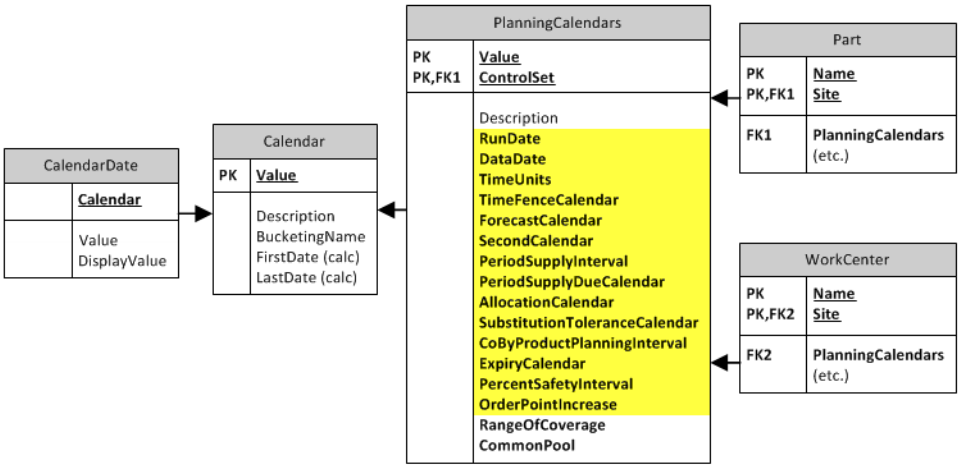
Calendar tables

There are three main tables in RapidResponse that hold calendar or calendar-related data. Each of these is briefly described in the following table.

Table	Description
Calendar	This table holds name of any commonly used calendars that might be required to support an organizations manufacturing and relative activities. For examples, Workday and Weekly calendars would be seen in most installations. In addition to the calendars defined here, there is an implicit Gregorian (everyday) calendar.
CalendarDate	This table contains actual date values along with references to the named calendar each belongs to. Thus, a given date might be defined on multiple records in this table with each record pointing to a different value in the Calendar table. For example, a given date might belong to both a Workday, Monday, and Weekly calendar.
PlanningCalendars	This table holds records that reference the Calendar value that applies to particular calculations or logic, and allows a single part to have multiple different working schedules. For example, a given part might use one calendar for forecasting, another for supply planning, and so on.

The following illustration shows the relationship between the three tables described above and identifies most of the commonly used fields on each.

Figure 12.1 Calendar related schema in RapidResponse



Note: The PlanningCalendars fields highlighted in yellow above indicate references to named values in the Calendar table.

Dates and time spans

The following two data types are used in date calculations:

- dates
- time spans

Date information is stored as a period of days. Values can range from January 2, 1970 to December 31, 2037. Spans of time are represented by quantities and the units are defined by the entries in the **Calendar** table. Date calculations involve adding or subtracting a time span to get a date value or calculating the time span between two dates.

Some of the parameters to the calculations represent a span of time (for example, the lead-times for ordering/making a part). Time span units are defined by calendars. Calendar names are listed in the **Calendar** table.

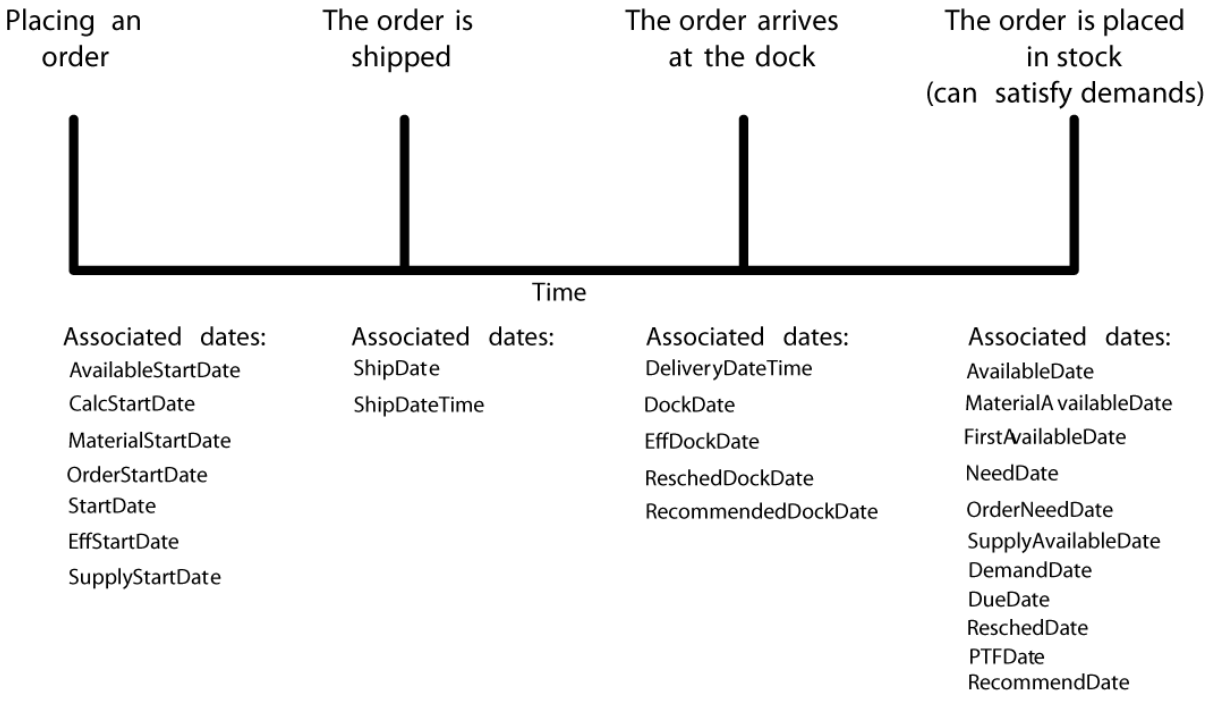
Each calendar has a set of records in the **CalendarDate** table. Each record in this table defines the beginning of a period for the associated calendar. For example, a workday calendar can have records in the **CalendarDate** table for all the dates the plant is operating. This would probably be Monday to Friday except for holidays. Suppose you had an order due date (Monday July 9, 2001) and wanted to calculate the start date, given a lead-time of five (5) workdays. You can use the calendar to step back the number of workdays to get the effective start date (Monday July 2, 2001). This means that lead-time calculations will skip non working days. The same types of date calculations can be done in filters and worksheet definitions. Calendar units either can be directly given in the formulas or can be specified by a field in the planning data.

If the part the order is created for is interchangeable with other parts, the calendar settings for the primary part in the interchangeable group are used for all parts in the group. For more information about interchangeable parts, see "Interchangeable parts" on page 1545.

RapidResponse date values

RapidResponse uses different date values while performing various calculations. Date values are associated with a specific manufacturing event. An example of a manufacturing event is when an order is shipped from a supplier to the manufacturer. The following diagram shows date values associated with manufacturing events.

Figure 12.2 Date values associated with manufacturing events



Date values associated with placing an order

There are eight date values associated with placing an order.

AvailableStartDate

The start date of an order based on AvailableDate (AvailableDate is associated with the order being placed in stock). AvailableStartDate considers both materials and capacity. This date value appears as:

- a field in the **CTPPlannedOrder** calculated table
- a field in the **CTPActivity** calculated table
- a field in the **InProcess** calculated table

- a calculated field in the **ScheduledReceipt** table
- a field in the **SupplyDemand** calculated table

CalcStartDate

The date the order was or should be started. This date value appears as:

- a field in the **InProcess** calculated table
- a field in the **Supply** calculated table
- a calculated field in the **ScheduledReceipt** table

For information about how CalcStartDate is calculated, see "ScheduledReceipt CalcStartDate field" on page 1637.

MaterialStartDate

The earliest date this order could be started, based on component availability. This date value appears as:

- a field in the **CTPPlannedOrder** calculated table
- a calculated field in the **ScheduledReceipt** table
- a field in the **SupplyDemand** calculated table

StartDate

The date the order should be released to the vendor or production started. This date value appears as:

- a field in the **BaselinePlannedOrder** table
- a field in the **CoByProductSupply** calculated table
- a field in the **PlannedOrder** calculated table
- a calculated field in the **ScheduledReceipt** table
- a field in the **Supply** calculated table

EffStartDate

The date the order was or should be started, based on EffDueDate and StartDate. This date value appears as a calculated field in the **ScheduledReceipt** table.

SupplyStartDate

The start date of the supply. This date value appears as:

- a field in the **WhereConsumed** calculated table
- a field in the **WhereConsumedForDemand** calculated table
- a field in the **WhereConsumedForForecast** calculated table
- a field in the **WhereConsumedForSupply** calculated table
- a field in the **LoadFLPPlanned** calculated table

Date values associated with shipping an order

There are two date values associated with shipping an order.

ShipDate

The date when the order should be shipped. This date value appears as:

- a calculated field in the **ScheduledReceipt** table
- a field in the **Activity** calculated table
- a field in the **CTPActivity** calculated table
- a field in the **InventoryTransfer** table
- a field in the **PlannedOrder** calculated table
- a field in the **Supply** calculated table

ShipDateTime

The date and time this order is scheduled to be shipped. This date value appears as a field in the **Shipment** table.

Date values associated with an order arriving at the dock

There are five date values associated with an order arriving at the manufacturer's dock.

DeliveryDateTime

The date and time this order is scheduled to arrive at the manufacturer's dock. This date value appears as a field in the **Shipment** table.

DockDate

The date this order is to be delivered to the manufacturer's dock. This date value appears as:

- a field in the **InventoryTransfer** table
- a field in the **ScheduledReceipt** table
- a field in the **PlannedOrder** calculated table
- a field in the **CTPActivity** calculated table
- a field in the **Activity** calculated table

EffDockDate

The date this order is to be delivered to the manufacturer's dock. This value is calculated by RapidResponse and appears as a calculated field on the **ScheduledReceipt** table.

ReschedDockDate

The calculated dock date value based on ReschedDate. This value appears as a calculated field on the **ScheduledReceipt** table.

RecommendedDockDate

The calculated dock date value based on RecommendedDate. This value appears as a calculated field on the **ScheduledReceipt** table.

Date values associated with an order being placed in stock

The following date values are associated with an order being placed in stock.

AvailableDate

The date the order is available based on materials, constraint, and due date. Once items in an order are available, they can be used to satisfy unsatisfied demands. This date value appears as:

- a calculated field in the **ScheduledReceipt** table
- a field in the **CTPActivity** calculated table
- a field in the **CTPCoByProductSupply** calculated table
- a field in the **CTPForecast** calculated table
- a field in the **CTPPlannedOrder** calculated table
- a field in the **CTPSubstituteSupply** calculated table
- a field in the **InProcess** calculated table
- a field in the **LoadFLPPlanned** calculated table
- a calculated field in the **IndependentDemand** table
- a calculated field in the **LateSupply** table

Note: For information about AvailableDate calculations, see "Available date calculations" on page 1456.

MaterialAvailableDate

Date the supply could be available, based on MaterialStartDate. RapidResponse considers constraints that belong to components that go into an assembly supply, but not the constraint on the assembly supply itself. This date value appears as:

- a calculated field in the **ScheduledReceipt** table
- a field in the **CTPPlannedOrder** calculated table

FirstAvailableDate

Date the first portion of this supply could be available based on materials, order status, and constraint. Different than the **AvailableDate** field in cases where the order is split due to incremental availability of components or constraint consumption in multiple periods. This date value appears as:

- a calculated field in the **ScheduledReceipt** table
- a field in the **CTPPlannedOrder** calculated table

NeedDate

The date when this order is needed in order to achieve on-time delivery for the particular demand. This date value appears as:

- a field in the **LateSupply** calculated table
- a field in the **WhereConsumed** calculated table
- a field in the **WhereConsumedForDemand** calculated table
- a field in the **WhereConsumedForForecast** calculated table
- a field in the **WhereConsumedForSupply** calculated table
- a field in the **WhereConsumedToHighVolumeOrder** calculated table

SupplyAvailableDate

The date when the immediate supply (order) is available. This date value appears as:

- a field in the **CostOfGoodsSold** calculated table
- a field in the **SupplyDemand** calculated table
- a field in the **WhereConsumed** calculated table
- a field in the **WhereConsumedForDemand** calculated table
- a field in the **WhereConsumedForForecast** calculated table
- a field in the **WhereConsumedForSupply** calculated table
- a field in the **WhereConsumedToHighVolumeOrder** calculated table

DueDate

The date this order is available for use. If the order is a dependent demand, DueDate is the date the demand is needed based on the ReschedDate of the parent supply. This date value appears as:

- a field on the **Allocation** table
- a field on the **Activity** calculated table
- a field on the **AvailableToPromise** calculated table
- a field on the **BlowThroughNewAllocation** calculated table
- a field on the **BlowThroughPlannedAllocation** calculated table
- a field on the **CTPActivity** calculated table
- a field on the **Demand** calculated table
- a field on the **DependentDemand** calculated table
- a field on the **IndependentDemand** table
- a field on the **InProcess** calculated table

- a field on the **LateSupply** calculated table
- a field on the **NewAllocation** calculated table
- a field on the **NewTransferAllocation** calculated table
- a field on the **PlannedAllocation** calculated table
- a field on the **PlannedOrder** calculated table
- a field on the **PlannedTransferAllocation** calculated table
- a field on the **ScheduledReceipt** table
- a field on the **SubstituteSupply** calculated table
- a field on the **Supply** calculated table

ReschedDate

The revised due date (stock date) after rescheduling the order. This date value appears as:

- a field on the **Activity** calculated table
- a calculated field on the **Allocation** table
- a field on the **CTPActivity** calculated table
- a field on the **Demand** calculated table
- a field on the **DependentDemand** calculated table
- a field on the **InProcess** calculated table
- a calculated field on the **ScheduledReceipt** table
- a field on the **Supply** calculated table

RecommendedDate

The recommended due date (stock date) for the order. Same as DueDate if a demand or planned order. This date value appears as:

- a field on the **Activity** calculated table
- a field on the **CTPActivity** calculated table
- a field on the **Supply** calculated table
- a field on the **ScheduledReceipt** table

DemandDate

The date of the first demand that netting satisfies with this supply. This date value appears as:

- a field on the **Activity** calculated table
- a field on the **CTPActivity** calculated table
- a field on the **PlannedOrder** calculated table
- a field on the **ScheduledReceipt** calculated table
- a calculated field on the **ScheduledReceipt** table

PTFDate

The date of the planning time fence calculated from the run date and PlanningTimeFence. This date value is used in determining planned order due dates (if OrderPolicy.OrderGenerationRule is **AfterPTF**).

This date value appears as:

- a calculated field on the **PartSource** table

Date constants

The following table lists date constants used in RapidResponse.

Table 12.1: Date constants

Date constant	Description
Past	Represents a date earlier than any calendar definition. It comes before all dates and is 'sticky' meaning any arithmetic on it will not change it because it is not known how much in the past it represents. Appears in reports as "Past".
Future	Represents a date later than any calendar definition. It comes after all dates and is 'sticky', meaning any arithmetic on it will not change it because it is not known how much in the future it represents. Appears in reports as "Future".
Undefined	A value has not been defined. All arithmetic involving an undefined value results in d'Undefined'. Undefined dates appear as a blank in all reports.
MRPDate	Depending on how RapidResponse is configured, either the earliest or latest date in any calendar referenced as **PlanningCalendars.RunDate.FirstDate**.
Today	Represents the current date as determined from the client system of a given user accessing RapidResponse. This may not necessarily represent the current date as defined on the installed/hosting RapidReponse. Similarly, users in different time zones, using the Today constant at exactly the same time, might get different values for Today based on their time zones.

Caution: The MRPDate constant should be used with care. For example, across sites or when data updates are performed, different parts might have different run dates. In these cases, the MRPDate constant might not always return the intended date. To avoid this, you should instead use `PlanningCalendars.RunDate.FirstDate` in your expression whenever you are working with a specific part.

Note: MRP date does not necessarily represent the date that data was extracted from the enterprise data source. **Part.PlanningCalendars.DataDate** represents the data extraction date for a particular part. **Part.PlanningCalendars.RunDate.FirstDate** represents the first date that action can be taken for a particular Part.

RunDate and DataDate

Each **PlanningCalendars** record has a reference to a **RunDate** calendar and a **DataDate** calendar. These references define dates that various analytic calculations will use to represent a "pseudo-today" date or planning date. It is expected that if the **Calendar** referenced by either of these **PlanningCalendars** fields contains more than one date in the **CalendarDates** table (this is typically an error), then the earliest date (FirstDate) is used.

The **Part.PlanningCalendars.DataDate.FirstDate** value gives the data extraction date for a particular part. The **Part.PlanningCalendars.RunDate.FirstDate** represents the first date that action can be taken

for a particular part. The **DataDate.FirstDate** value should never be later than the **RunDate.FirstDate**. However in some cases the **RunDate** might be adjusted as time goes by and hence could, for a time, be later than **DataDate**. For example, this might occur if data is only updated on a weekly basis in which case the **RunDate** can be adjusted while the **DataDate** remains stationary until the next update.

Typically, however, these two values will return the same date and each will be updated when new data is updated or imported into RapidResponse (for example, this might typically occur on a daily basis). This has the effect of resetting the planning horizon for the part.

Date calculation rules

Date and time calculations involve adding or subtracting a quantity and a date. The result is another date value. A time unit can be specified by using an existing calendar name or by performing the calculation with real days (for example, no calendar used). The following are date calculations rules:

- If a date is before the earliest date in the calendar, the result is 'Past'.
- If the date is after the latest date in the calendar, the result is 'Future'.
- Past and Future are sticky. This means that any calculation on Past or Future will result in the same date.
- If the date is 'Undefined', the result of any calculation applied to this date is also 'Undefined'.
- If zero units are added (or subtracted) from a date then the result is the date of the **CalendarDate** table entry earlier or equal to the date (for example, it rounds to the beginning of the period containing the date).
- If you define a date range with a calendar, and the specified date is not within the calendar's range, a date value of 'Past' or 'Future' is returned. For example, this occurs if you had specified weeks until the end of a calendar year and you wish to work in the next calendar year.
- If the resulting date is before the earliest calendar date, the result is 'Past'.
- If the resulting date is not defined as the beginning of a period in the set of CalendarDate records, the calculation considers the starting point the beginning of the calendar period. For example, if Monday to Friday are WorkDays, adding one Workday to either Friday, Saturday, or Sunday results in the Monday date.

When performing date arithmetic, RapidResponse evaluates the components of an expression on a strictly left-to-right basis. If you require a specific part of the expression to be evaluated first, ensure to enclose it in parentheses. Refer to the following examples.

Table 12.2: Examples of date arithmetic

Entering...	Returns
12-31-2014 - 5 Workday	The date that is five workdays prior to December 31, 2014.
12-31-2014 + 0 Week	The start of the work week containing December 31, 2014.
DueDate + 2 month	The first working date of the second month after the order is due.
DueDate + (2 + 1) Week	The start of the third work week after order is due.
(DueDate + 2) + 1 Week	The start of the work week containing the date that is 2 standard/gregorian calendar days after the order is due.

Differences between dates

RapidResponse can also calculate the difference between two dates. Similar to date arithmetic, a specific calendar or interval can be used to express the difference between any two dates. This is done using the following syntax:

```
Date1 - Date2 Calendar
```

The result of such a calculation is an integer number that represents the span of time between date1 and date2 in intervals of the named calendar. Essentially, this type of calculation is simply counting the number of the specified calendar intervals or markers found between two dates. If a calendar is not provided then RapidResponse assumes regular calendar days or 'everyday'. Refer to the following examples:

Entering...	Returns...
AvailableDate - DueDate Workday	The number of work days between the AvailableDate and the DueDate.
AvailableDate - DueDate	The number of standard calendar (Gregorian) days between the AvailableDate and the DueDate.
12-04-2014 - 10-20-2014 Month	The number of monthly calendar markers between December 4 and October 20 (in other words, 2).

Calendar date boundaries

As discussed earlier in this chapter, each **Calendar** table record has a set of **CalendarDate** records that reference it. On the **Calendar** table, there are two calculated fields that return the first date (**FirstDate**) and the last date (**LastDate**) for these **CalendarDate** records. As long as date calculations are on or use intervals and markers that stay within these two date boundaries, then the expected results will be as described in this chapter.

However, as soon as a date calculation for a particular calendar moves beyond these dates, it will result in either a value of "Past" (if earlier than the calendar's **FirstDate**) or "Future" (if later than the calendar's **LastDate**).

These "Past" and "Future" dates can be thought of as being "sticky". This means that, if while RapidResponse is processing a date expression, either the "Past" or "Future" values are reached, then all subsequent calculations in the expression will also return "Past" or "Future". This is because there is no way to indicate how far in the past or future a given date is.

In addition to the **FirstDate** and **LastDate** values calculated for a given calendar, there are also absolute date limits on the values that can be loaded into the **CalendarDate** table. The earliest date allowed on a **CalendarDate** record is `01-02-1970` and earlier dates are always taken to mean "Past". Similarly, the latest date allowed on a **CalendarDate** record is 12-31-2037 and later dates are always taken to mean "Future".

Calendar boundaries and differences between dates

The limitations imposed by dates extending beyond a given calendar's defined intervals might also be observed in the form of strange or unexpected integer values when subtracting one date from another. In particular, if the difference between two date values cannot be determined or would result in an

invalid value, then an integer value of `65,535` is always returned. This value is used internally by RapidResponse to represent the result of an invalid date calculation. Typically, this occurs when one or both of the date values used in calculating a difference has gone beyond the **FirstDate** or **LastDate** boundaries of the calendar being used. For example, consider the following expression:

```
AvailableDate - DueDate Workday
```

Normally, this expression returns the integer number of working days between an order's AvailableDate and its DueDate. If, however, the AvailableDate has been calculated as at "Future" (for example, because it is never available), then its is not possible to calculate the difference between Future and an actual date value in DueDate. In this, and similar cases, a value of `65,535` is used to represent the invalid or unresolvable nature of the calculation.

Calendar date display values

Entries in the CalendarDate table can have display values specified, which can be displayed in crosstab worksheet buckets in place of the date that each bucket begins with. For example, an entry in the Month calendar for August 2008 can have 'August 2008' specified as its display date. In worksheets that use the display values, the bucket for August will have 'August 2008' instead of '08-01-08' as its label.

To specify display names for calendar units, you must create a worksheet based on the CalendarDate table.

To specify display values for a calendar

1. On the **File** menu, point to **New**, and then click **Workbook**.
2. In the **New Workbook** dialog box, in the **Name** box, type a name for the workbook.
3. Click the **Worksheets** tab.
4. Click **New**, and then click **Table-based worksheet**.
5. In the **New Worksheet** dialog box, in the **Name** box, type a name for the worksheet.
6. In the **Table** list, click **CalendarDate**.
7. Click the **Columns** tab.
8. Click **Add Fields**.
9. In the **Add Fields** dialog box, add the following fields:
   - **Calendar**
   - **Value**
   - **DisplayValue**
10. Click **OK**, and then click **OK** again.
11. Click **Yes** to open the workbook.
12. In your worksheet, in the **Calendar** column, type the name of the calendar you want to add display values for.
13. In the **Display Value** column, type the value you want displayed for each calendar unit.
14. Click **Save Data**.
15. Repeat steps 12 - 14 for each calendar you want to specify display values for.

CHAPTER 13: Lead time calculations

Lead time refers to the amount of time it takes to complete a process. For example, how many days it takes for an item to arrive in stock once it is has been ordered. RapidResponse uses lead time values when performing calculations such as Netting, and has fields that store calculated lead time values. For example, **PartSource.EffLeadTime** shows the total lead time for a given part source, and **PartSource.LeadTimeDate** shows the lead time date for a given part source calculated from the run date and respecting all relevant calendars.

RapidResponse uses two types of lead time calculations, inclusive and cumulative. For a given part source, the **PartSourceType.TransitRule** control field determines if RapidResponse uses inclusive or cumulative lead-time calculations. That is, the **TransitRule** field determines whether any applicable pre-ship, transit, and dock-to-stock time are included in lead time and hence affect the calculated **StartDate** (other components of lead time like fixed and variable times are always included regardless of the **TransitRule** setting).

In this chapter, you'll learn about:

Lead time dates 1307
Lead time elements 1308
Lead time calendars 1317
Inclusive and cumulative lead time 1323
Lead time rounding 1327
Lead time overrides from scheduled receipts 1328

Lead time dates

There are a number of dates associated with a supply, each with its own meaning and derivation. Lead time affects each of them in different ways as described briefly in the following table.

Table 13.1: Supply dates and lead time

Date	Description
DemandDate	The date representing the due date (date from stock) of the demand to be satisfied. For independent demands, this is the input DueDate provided for the demand. For dependent demands, it is the DueDate that has been derived or calculated from the parent supply and offset through the related bill of material record.
DueDate	The date that the supply is expected or scheduled to be put into stock to cover the demand. This date is determined by offsetting the DemandDate by the effective safety lead time value applicable to the part and/or part source.
DockDate	The date that the supply actually completed and is received to the dock. This date is determined by offsetting the DueDate by any dock-to-stock time applicable to the part source.
ShipDate	The date the supply is shipped from the sourcing site, and typically only applicable to transfer sources. This date is determined by offsetting the DockDate by the transit time applicable the source.
BuiltDate	The date that the supply is completed at the sourcing site, and typically only applicable to transfer sources. This date is determined by offsetting the ShipDate by any pre-ship time applicable to the source. For transfer sources, this date corresponds to the demand date at the sourcing site. Note that this date is not reported as a field on any supply tables in RapidResponse. If necessary, it can be calculated in a worksheet column based on one of the other dates that are reported.
StartDate	The date that work on the supply should begin in order to complete by the DueDate (typically not applicable to transfer sources). It includes consideration for all applicable lead time lead time values (such as fixed and variable) and the calculation of this date is influenced by control settings as discussed later in this lesson.

Lead time elements

The various components of lead time are set in several different fields in RapidResponse. Typically, these lead time values apply at the part source level (defined on the **PartSource** table, or on tables referenced from it such as **Part** and **Source**).

Each field that holds lead time values is expected to represent a number of calendar intervals (such as, workday) so whole numbers should be provided in most of this fields (excepting variable lead time). For information about the different calendars used to hold lead time values, see "Lead time calendars" on page 1317. Additionally, not all lead time fields use the same underlying calendar, so calendar rounding may occur when calculating lead time. The implications of this are discussed in "Lead time rounding" on page 1327.

The following table describes fields on the **Part**, **Source**, and **PartSource** table that are used as input into lead time calculations. All fields listed can be used to provide stationary lead time values that aren't expected to change. However, if some lead times are likely to increase or decrease over time, then fields on the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables can be used instead as discussed in "Time-phased lead time" on page 1313

Table 13.2: Lead time elements

Table	Field	Description
PartSource	FixedLeadTime	Fixed amount of time to produce each order for the part from this source. This field is used along with the variable lead time and lead time adjustment to calculate the supply's **StartDate** (as determined from the **DueDate** or **BuiltDate**). Expressed in **PartSource.Source.LeadTimeUnits**.
PartSource	VarLeadTime	Variable amount of time to produce an order for the part from this source (can be fractional). The value provided here is multiplied by the order quantity to determine total variable lead time (if the supply quantity is not known an average quantity is used). This field is used along with the fixed lead time and lead time adjustment to calculate the supply's **StartDate** (as determined from the **DueDate** or **BuiltDate**). Expressed in **PartSource.Source.LeadTimeUnits**.
Part	LeadTimeAdjust	An adjustment applied to fixed lead time for all supply orders of the part. This field is used along with the fixed lead time and variable lead time to calculate the supply's **StartDate** (as determined from the **DueDate** or **BuiltDate**). Note that this field is rarely used. Expressed in **PartSource.Source.LeadTimeUnits**.
PartSource	DockToStock LeadTime	The amount of time needed to move the order from the receiving dock into available inventory. For example, this might include time for inspection of the supply. Expressed in **Part.PlanningCalendars.TimeUnits.**
Source	PreShipLT	The amount of time required to prepare supply for shipment. It represents the time between the **BuiltDate** and **ShipDate**, and is typically only applicable to transfer sources. Expressed in **PartSource.Source.LeadTimeUnits**.
Source	TransitTime	The amount of time required to ship supply from the sourcing site to the destination site. It represents the time between the **ShipDate** and **DockDate,** and is typically only applicable to transfer sources. Expressed in **Source.TransitCalendar** intervals (or the Everyday/Gregorian calendar in cases where the TransitCalendar reference is left Null).
PartSource	SafetyLeadTime	Amount of time added as a buffer to supplies from the part source to guard against fluctuations in supply and demand. It represents the time between the **DueDate** for supplies from this part source and the actual **DemandDate**. For example, it might be used to add an additional lead time amount for an unreliable supplier. Expressed in **Part.PlanningCalendars.TimeUnits** or the Everday/Gregorian calendar (depends on the **PartType.SafetyCalendarRule** setting applicable to the part).

Table 13.2: Lead time elements

Table	Field	Description
Part	SafetyLeadTime	Amount of time added as a buffer to all supplies for the part. It is meant to guard against fluctuations in supply and demand and is combined with any safety lead time value defined on the part source. Expressed in **Part.PlanningCalendars.TimeUnits** or the Everyday/Gregorian calendar (depends on the **PartType.SafetyLeadTimeCalendarRule** setting applicable to the part). **Note:** This field is only used if the **Part** record has a Null reference in the **SafetyLeadTimeProfile** field (or if there are date gaps within the referenced profile).
SafetyLead TimeProfile Detail	SafetyLeadTime	Amount of time added as a buffer, within a specified date range only, to all supplies for parts that use the safety lead time profile which the **SafetyLeadTimeProfileDetail** record belongs to. Meant to guard against fluctuations in supply and demand and is combined with any safety lead time value defined on the part source. Expressed in **Part.PlanningCalendars.TimeUnits** or the Everday/Gregorian calendar (depends on the **PartType.SafetyCalendarRule** setting applicable to a given part). **Note:** This field value is only ever used by parts that have a non-Null reference in the **SafetyLeadTimeProfile** field (parts that use time-phased safety lead time).For information about configuring safety lead time profiles, see "SafetyLeadTimeProfile table" on page 538.

Note: Any SafetyLeadTime values specified are not applied to any supplies generated to satisfy safety stock requirements.

Including or excluding lead time elements

Several control table settings determine whether or not certain components of lead time are used. Most of these are boolean-type fields that accept Y (yes) or N (no) values to indicate whether a particular lead time component is used. These controls are set on the **PartType** table (for enabling lead time values defined at the part level) as well as on the **OrderPolicy** and **PartSourceType** tables (for enabling lead time values defined at the source and part source level).

The following table lists the control table fields available for setting the lead time elements used in lead time calculations. Note that not all lead time fields have controls available to define their usage (are always assumed to be effective). However, any input lead time field can be effectively excluded by settings its value to 0.

Table 13.3: Controls for including/excluding lead-time elements

Table	Field	Description and valid values
OrderPolicy	UseFixedLeadTime	Specifies whether fixed lead time is used on part sources of a given order policy. Options are: • N (default) • Y
OrderPolicy	UseVarLeadTime	Specifies whether variable lead time is used on part sources of a given order policy. Options are: • CRP • N (default) • Y **Note:** The "CRP" option determines variable lead time based on the time needed to run supply through all work center operations in its routing (does not use the VarLeadTime value). This setting supports the Capacity Requirements Planning (CRP) analytic as described in "Operations and supply lead time" on page 1532.
PartSourceType	DockToStockLeadTime	Specifies whether dock-to-stock lead time defined for the part source is used lead time calculations. Options are: • Ignore • Use
PartSourceType	PreShipLeadTimeRule	Specifies whether pre-ship lead time defined for the source is used in lead time calculations. Options are: • Ignore • Use
PartSourceType	TransitTimeRule	Specifies whether transit time defined for the source is used in lead time calculations. Options are • Ignore • Use
PartSourceType	TransitRule	Specifies which lead time elements are used in determining supply start date. Options are: • Cumulative • Inclusive (default) **Note:** For more information, see "Inclusive and cumulative lead time" on page 1323.
PartType	UseLeadTimeAdjust	Specifies whether lead time adjustment is used for parts of this type. Options are: • N • Y (default)

Table 13.3: Controls for including/excluding lead-time elements

Table	Field	Description and valid values
PartType	UseSafetyLeadTime	Specifies whether safety lead time, as applicable to the part and/or part source, is used for parts of this type. Options are: • N (default) • Soft • Y **Note:** When safety lead is enabled for parts of a given part type (set to "Y" or "Soft"), there are three additional controls available to determine if it used with specific types of demand as described in the remainder of this table. For example, safety lead time might be enabled for use with independent and consensus forecast demands, but disabled for dependent demands. When safety lead time is disabled for parts of a given part type (set to "N"), then these three additional settings are not relevant.
PartType	DependentSafetyLeadTimeRule	For parts with safety lead time enabled, this specifies whether safety lead time is applied to supplies satisfying dependent demands for those parts. Options are: • FromDemand • Ignore • Use (default)
PartType	IndependentSafety LeadTimeRule	For parts with safety lead time enabled, this specifies whether safety lead time is applied to supplies satisfying consensus forecast and independent demands for those parts. Options are: • FromDemand • Ignore • Use (default)

Table 13.3: Controls for including/excluding lead-time elements

Table	Field	Description and valid values
DemandType	SafetyLeadTimeRule	Specifies whether safety lead time should be applied to demands of a given type. For example, safety lead time might be applied to actual customer demands but ignored on forecast demands. Options are: • Ignore • Use (default) **Note:** This setting only applies to consensus forecast and independent demands for parts with an IndependentSafetyLeadTimeRule setting of "FromDemand", and to dependent demands for parts with a DependentSafetyLeadTimeRule setting of "FromDemand".
OrderPriority	TwoDatePlanningRule	This field is used to enable (or disable) two-date planning logic whereby after demands after planned to their committed due dates, attempts are made to move those demands closer to the original customer request date this can be done without making other demands worse off (subject to the TwoDatePlanningRule setting on those other demands). However, this field also contains two settings ("UseWithHardSafetyLeadTime" and "UseWithSoftSafetyLeadTime") that enable special logic related to safety lead time (along with two-date planning capabilities). These both apply safety lead time to the request date and in the case of "UseWithSoftSafetyLeadTime" safety lead time is only ever applied to a demand if it can be done without making other demands worse off in the process If enabling safety lead time logic that relies on two-date planning logic and the two settings on the **OrderPriority.TwoDatePlanningRule** field mentioned above, ensure any PartSource providing supply to satisfy demands using these priority settings, as well as any scheduled receipts that might satisfy such demands, reference a **SupplyStatus.AvailableDateLimit** setting of "CalcRequestStart". Otherwise, supplies might not be planned as early as expected or needed. **Note:** For more information about how safety lead time works with two-date planning logic, see "Apply safety lead time at request date (optional)" on page 1807.

Time-phased lead time

If lead times are expected to change over time, then time-phased lead time values can be provided in the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables. These tables reference the **PartSource** and **Source** tables respectively. They thereby allow for lead time values to be defined for specific date ranges which then override the corresponding stationary values provided on the referenced **PartSource** or**Source** records.

Table 13.4: Time-phased lead time fields

Table	Field	Description
TimePhasedPartSourceLeadTime	FixedLeadTime	Fixed amount of time to produce each order for the part from the source. Expressed in **PartSource.Source.LeadTimeUnits**.
TimePhasedPartSourceLeadTime	DockToStock LeadTime	Amount of time needed to move the order from the receiving dock into available inventory. Expressed in **Part.PlanningCalendars.TimeUnits.**
TimePhasedSourceLeadTime	PreShipLT	Amount of time required to prepare supply for shipment from sourcing site to destination. Expressed in **PartSource.Source.LeadTimeUnits**.
TimePhasedSourceLeadTime	TransitTime	Amount of time required to ship supply from the sourcing site to the destination site. Expressed in **Source.TransitCalendar** intervals (or using the Everyday calendar if the **TransitCalendar** reference is Null).

Note: A negative value in any of the fields described above indicates to use the value in the previous (next earliest) record in the table. If no such record exists, than the default stationary value from the corresponding field in the **PartSource** or **Source** table is used instead.

EffectiveInDate and ParameterDate

Two main dates are involved when determining the time-phased values that apply to a given supply.

The **EffectiveInDate** field is found on both the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables. This input field specifies the date on which the time-phased lead times on a record become effective for the referenced **PartSource** or **Source**. The lead times then remain effective until the next **EffectiveInDate** found on a record referencing that same **PartSource** or **Source**.

The **ParameterDate** field is found on both the **PlannedOrder** and **ScheduledReceipt** tables. For planned orders, the **ParameterDate** is typically set to the order's due date. For scheduled receipts, the **ParameterDate** is typically set to either the input **DueDate** (for non-reschedulable receipts) or to the calculated **ReschedDate** (for reschedulable receipts).

To determine which time-phased lead time record(s) should apply to a given supply, the supply's **ParameterDate** is compared against the **EffectiveInDate** on the records pertaining to a given part source and source . The record in each table with the closest **EffectiveInDate** value that is on or before the **ParameterDate** is then used.

Note: The calculated **PTFDate**, **EffLeadTime**, and **EffFixedLeadTime** fields on each **PartSource** uses time-phased lead time values from the record(s) effective at the latter of **PartSource.EffectiveInDate** or **Part.RunDate**.
Similar logic applies to the **CumLeadTime** and **MultiSiteCumLeadTime** fields on the Part table (based on effective records for the part's primary part source), as well as to the **CumLeadTime** and **MultiSiteCumLeadTime** fields on the **BillOfMaterial** table (based on effective records for the component part's primary part source).

Example

Suppose the following **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** records pertain to the lone part source for a given part.

Table 13.5: Sample time-phased lead time records

TimePhasedPartSourceLeadTime			TimePhasedSourceLeadTime		
EffectiveIn Date	**Fixed LeadTime**	**DockToStock LeadTime**	**EffectiveIn Date**	**PreShipLT**	**Transit Time**
April 2, 2018	5	1	March 1, 2018	2	10
May 1, 2018	6	-1	March 30, 2018	-1	12
			April 30, 2018	-1	13

Further suppose that part has three demands, due on March 30th 2018, April 3rd 2018, May 15th 2018, and a planned order is required to cover each of these demands. The following table shows the time-phased lead time values applicable to the planned orders created for each of these demands based on ParameterDate.

Table 13.6: Lead times used based on ParameterDate

ParameterDate	Applicable lead time values...			
	Fixed LeadTime	**DockToStock LeadTime**	**PreShipLT**	**Transit Time**
March 30, 2018	Taken from PartSource table		2	12
April 3, 2018	5	1	2	12
May 15, 2018	6	1	2	13

Enabling and disabling time-phased lead time

Time-phased lead time values are subject to the same basic controls used for stationary lead times as discussed in "Including or excluding lead time elements" on page 1310. For example, if a part source has time-phased fixed lead times defined, its **OrderPolicy.UseFixedLeadTime** setting must still be set to "Y" in order for those lead times to be used in planning calculations.

Additionally the following controls allow for time-phased lead time logic to be turned off even if values are provided in the **TimePhasedPartSourceLeadTime** and **TimePhasedSourceLeadTime** tables.

Table 13.7: Controls for disabling or enabling time-phased lead time

Table	Field	Description
PartSourceType	UseTimePhasedPartSourceLeadTime	Specifies whether values defined in the **TimePhasedPartSourceLeadTime** table will be used in lead-time calculations. Options are: • N (lead-times from **PartSource** table are always used) • Y (time-phased lead times are used if provided, otherwise values from **PartSource** table are used)
PartSourceType	UseTimePhasedSourceLeadTime	Specifies whether values defined in the **TimePhasedSourceLeadTime** table will be used in lead-time calculations. Options are: • N (lead-times from **Source** table are always used) • Y (time-phased lead times are used if provided, otherwise values from **Source** table are used)

Time-phased lead time limitations

A small number of other analytics or calculations might be disabled, or not work as expected, for parts with demands that are enabled for time-phased lead time. The following table outlines other calculations having potential compatibility issues when using time-phased lead time logic:

Table 13.8: Time-phased lead time limitations on other analytics

Analytic / Calculation	Limitation
Days of supply	Because days of supply logic is intended to satisfy all demands due in a given period with a single planned order, time-phased lead times used for that planned order won't match necessarily match the lead times effective on the date when each of those demands was due. Additionally days of supply logic intended to collect and planned orders based on their dock date is not supported with time-phased lead time. If time-phased lead time logic is required, then the "DockDate" setting on the **PeriodSupplyDateRule** and **LongTermPeriodSupplyDateRule** fields of the **DaysSupplyPolicy** table should be avoided.
Constrained planning	In cases where limited constraint causes supply to be pre-planned in different constraint buckets when satisfying a given demand, multiple planned orders to satisfy that demand can be created with different due dates. When this occurs, all of these planned orders created to cover the demand will have the same calculated **ParameterDate**, but this date will not necessarily equate to the due date on all of those planned orders.
Order point	Because order point parts plan to start orders when projected inventory falls at or below the safety stock level, the **ParameterDate** used in determining time-phased lead time for supplies is calculated differently for this type of part (for example, where **SafetyStockQuantityRule** is set to "OrderPointUnder") On planned orders, the **ParameterDate** is set to the order's **StartDate** rather than its **DueDate**. On scheduled receipts, the **ParameterDate** is affected by the applicable**OrderPointDateRule** setting and whether the supply is reschedulable. For non-reschedulable receipts, if **OrderPointDateRule** is set to "Due", then the receipts **ParameterDate** is set to its **DueDate**. But if a non-reschedulable receipt has **OrderPointDateRule** set to "Start", then the receipt's **ParameterDate** is set to its **StartDate** (unless that date is undefined in which case **DueDate** is used instead). For reschedulable receipts, the **ParameterDate** is always set to the demand date.

Table 13.8: Time-phased lead time limitations on other analytics

Analytic / Calculation	Limitation
Safety stock date	When planning safety stock, certain **SafetyStockDateRule** settings ("LeadTime", "LeadTimReportUnplanned", and "FirstDemandOrLeadTime") rely on a part's lead time to set the date from which safety stock should be planned. When using time-phased lead time logic, the lead times used in the calculation to determine this "safety stock date" are those effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate**. In some cases, such as when the lead time used to determine the safety stock date is shorter than the lead time applicable to a supply created to satisfy a safety stock requirement shortly after that safety stock date, this can result in the planning an order to satisfy safety stock where the **StartDate** of the order is not achievable (is before the part's **RunDate**).
Range of coverage lead time	For part's where **SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime", safety stock is calculated to satisfy the part's average daily requirement over lead time. The lead time used for this calculation is the lead time effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate**. Around dates where time-phased lead times change, this can result in excessive safety stock being generated.
FlatBill	The effective lead time values reported for components and assemblies in the **FlatBillDown** and **FlatBillUp** table rely on the lead times effective at the latter of the part's **RunDate** or **PartSource.FirstEffectiveDate**. Time-phased lead times that become effective after these dates are not reflected in these tables.

Lead time calendars

Each of the lead time dates discussed in Lead time dates must belong to a particular defined calendar. Similarly, each of the input lead time elements discussed in Lead time elements are assumed to be expressed in units of a particular calendar. The relevant calendars are defined at the Part and PartSource level as described in the following table:

Table 13.9: Calendars used in lead time calculations

Calendar	Description and usage
Everyday	The standard everyday (Gregorian) calendar. Demands can fall on dates on this calendar, while supply dates must be on the specific calendars outlined below. The following values are expressed in this calendar unit: • SafetyLeadTime (PartSource, Part, and SafetyLeadTimeProfileDetail) . This calendar is only used when **PartType.SafetyLeadTimeCalendarRule** is set to "Everyday"
Part.PlanningCalendars.TimeUnits	The basic or default calendar used for a given part (for example, this might typically be the workday calendar). The following supply dates must be on this calendar: • DemandDate • DueDate • DockDate The following values are expressed in this calendar unit: • DockToStockLeadTime (PartSource) • SafetyLeadTime (PartSource, Part, and SafetyLeadTimeProfileDetail). This calendar is only used when **PartType.SafetyLeadTimeCalendarRule** is set to "TimeUnits"
PartSource.Source.ShipCalendar	The shipping calendar for a supply source (defines those dates on which supply can ship from sourcing site). The following supply date must be on this calendar: • ShipDate
PartSource.Source.LeadTimeUnit	The normal lead time calendar for a supply source. For manufactured parts, this is typically the same as the part's default TimeUnits calendar. For transfer and purchase parts, it is might be different. The following supply dates must be on this calendar: • BuiltDate • StartDate The following values are expressed in this calendar unit: • FixedLeadTime (PartSource) • LeadTimeAdjust (Part) • PreShipLT (Source) • VarLeadTime (PartSource)
PartSource.Source.TransitCalendar	The calendar used to define valid dates on which shipments from the sourcing site are moving towards the destination site (generally applicable to transfer sources only). For example, this is meant to account for dates on which vehicles transporting shipments cannot actually travel (such as, weekends). If this calendar is not used, then the Gregorian (everyday) calendar is assumed. The following value is expressed in this calendar unit: • TransitTime (Source)

Lead time calendar rounding

In cases where the different components of lead time rely on different calendars, then date rounding might occur in calculations when moving from one calendar to another. That is, supply dates can be pushed back until they land on a date belonging to the relevant calendar.

This is particularly true when cumulative lead time is used (that is, where the applicable **PartSourceType.TransitRule** value is set to "Cumulative"). For more information about the TransitRule, see "Inclusive and cumulative lead time" on page 1323.

Refer to the following for examples of calendar rounding:

1. DemandDate is on the everyday or gregorian calendar. Suppose the part's PlanningCalendars.TimeUnits calendar is a Monday to Friday workday calendar. Further suppose a particular DemandDate occurs on a Sunday. In this case, the effective DemandDate and hence DueDate for the associated order to satisfy is rounded back to the previous Friday (and potentially earlier if SafetyLeadTime is used).

   In other words, all demands for the part with a DemandDate on the Friday, Saturday, or Sunday would have the same DueDate in this situation.
2. ShipDate is on the part source's Source.ShipCalendar. This calendar defines the dates on which a source can make shipments to site where the supply is needed. Suppose this points to a Friday calendar (that is, shipments can only occur on Fridays) but all other relevant lead time calendars are expressed in workdays. This can have the impact of adding time due to calendar rounding when applying the TransitTime and PreShipLT values.

   For example, if the calculated DockDate is on a Friday and TransitTime is 3 days, the supply still must be ready to ship on previously Friday. For transfer sources, this will further impact due dates on related dependent demands at the sourcing site.

Adjusted dates after calendar rounding

When applying lead time back from a supply due date, calendar rounding can result in start, ship, and dock dates that are earlier than they would be otherwise be if all lead time values were expressed in the same calendar. However, the original due date for a supply is not subsequently adjusted to reflect these potentially earlier start dates. Similarly, the dock and ship dates are not adjusted for any calendar rounding done when applying lead time back from the due date.

To address situations where calendar rounding was needed during lead time calculations, three fields on the PlannedOrder and ScheduledReceipt tables can be used.. The following fields on each table can be used to report adjusted due, dock, and ship dates as calculated by applying lead time forward starting from the calculated supply start date.

- **ShipDateFromStart**
- **DockDateFromStart**
- **DueDateFromStart**

Note that the actual calculation of each of these fields depends on the **PartSourceType.TransitRule** applicable to a given part source setting as outlined in "Calculation of supply dates from start" on page 1320 (typically, these fields are useful when the **TransitRule** setting is "Cumulative"). For scheduled receipts, the calculations for these three fields are further affected by whether the input **StartDate** field

is used during rescheduling (that is, if **StartDate** is defined and **SupplyStatus.IgnoreStartDate** set to "N").

The following table outlines details the calculations of the supply dates calculated forward from start on the **PlannedOrder** and **ScheduledReceipt** tables.

Table 13.10: Calculation of supply dates from start

Field (Table)	Calculated as...
ShipDateFromStart (PlannedOrder)	If **TransitRule** is "Cumulative" then: • **StartDate** plus applicable lead time values (fixed, variable, lead time adjust, and pre-ship) If **TransitRule** is "Inclusive" then: • **DockDateFromStart** minus transit time.
DockDateFromStart (PlannedOrder)	If **TransitRule** is "Cumulative" then: • **ShipDateFromStart** plus transit time If **TransitRule** is "Inclusive" then: • **DueDateFromStart** minus dock-to-stock lead time.
DueDateFromStart (PlannedOrder)	If **TransitRule** is "Cumulative" then: • **DockDateFromStart** plus dock-to-stock lead time. If **TransitRule** is "Inclusive" then: • **StartDate** plus applicable lead time values (fixed, variable, and lead time adjust)
ShipDateFromStart (ScheduledReceipt)	If **TransitRule** is "Cumulative" then: • If **StartDate** is not defined or used, **CalcStartDate** plus applicable lead time values (fixed, variable, lead time adjust, and pre-ship) • If **StartDate** is defined and used, **DockDateFromStart** minus transit time. If **TransitRule** is "Inclusive" then: • **DockDateFromStart** minus transit time.
DockDateFromStart (ScheduledReceipt)	If **TransitRule** is "Cumulative" then: • If **StartDate** is not defined or used, **ShipDateFromStart** plus transit time • If **StartDate** is defined and used, **DueDateFromStart** minus dock-to-stock time If **TransitRule** is "Inclusive" then: • **DueDateFromStart** minus dock-to-stock lead time
DueDateFromStart (ScheduledReceipt)	If **TransitRule** is "Cumulative" then: • If **StartDate** is not defined or used, **DockDateFromStart** plus dock-to-stock lead time. • If **StartDate** is defined and used, **CalcStartStartDate** plus the difference between the **DueDate** and **StartDate**. If **TransitRule** is "Inclusive" then: • If **StartDate** is not defined or used, **CalcStartDate** plus applicable lead time values (fixed, variable, and lead time adjust) • If **StartDate** is defined and used, **CalcStartDate** plus the difference between the **DueDate** and **StartDate**.

Example: Supply dates from start

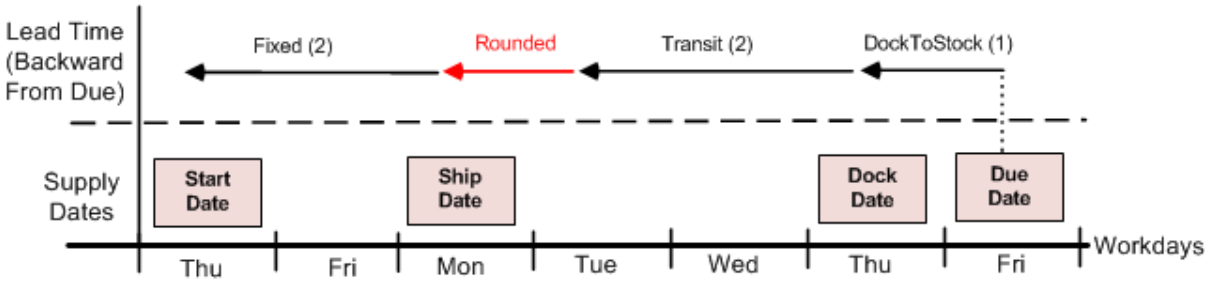
Suppose a part that uses "cumulative" lead time has the following lead time values and calendars applied (assume the Workday calendar represents a standard five day work week):

Table 13.11: Example lead time values and calendars

Lead time field	Value	Lead time calendar	Value
DockToStockLeadTime	1	TimeUnits	Workday
TransitTime	2	TransitCalendar	Workday
		ShipCalendar	Monday
FixedLeadTime	2	LeadTimeUnit	Workday

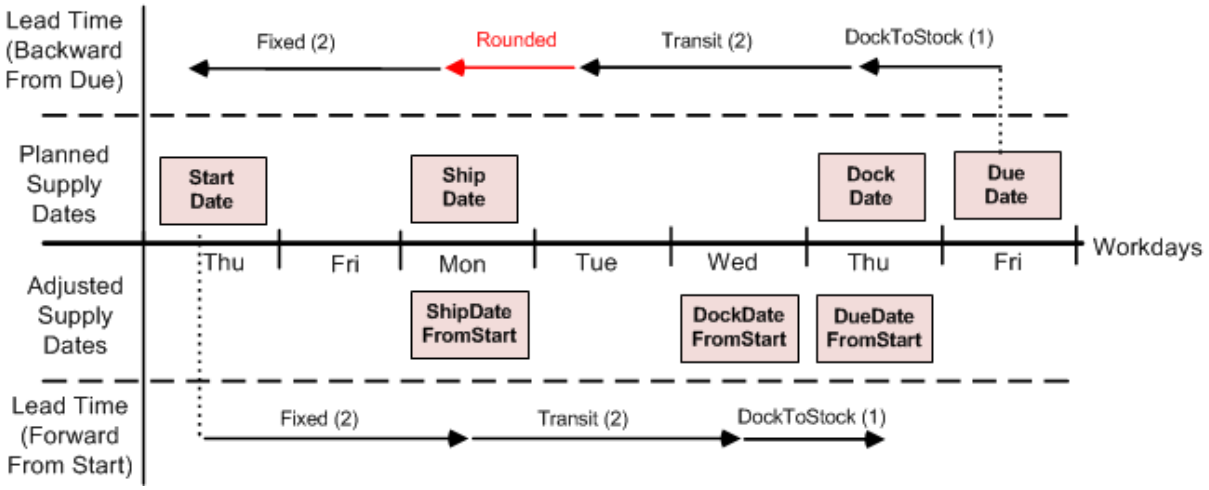
Given a demand due on a Friday to be satisfied by a planned order, applying the lead time values back from the due date leads to a **StartDate** on the previous Thursday as shown in the following illustration. In this case, note that applying 2 days of transit time back from the **DockDate** suggests the supply should ship on Tuesday, however since the **ShipCalendar** is set to "Monday", calendar rounding moves the actual **ShipDate** back to Monday. This results in a **StartDate** that is also one day earlier than is implied by cumulative lead time values alone.

Figure 13.1 Planned order dates are calculated back from the DueDate.



Then starting from the **StartDate** calculated on the planned order, ship, dock, and due dates from start are then calculated as shown in the following illustration. In this case, the note that the **DockDateFromStart** and **DueDateFromStart** are each one day earlier than the corresponding **DockDate** and **DueDate** values.

Figure 13.2 Planned order dates are calculated forward from the StartDate.



Safety lead time calendar

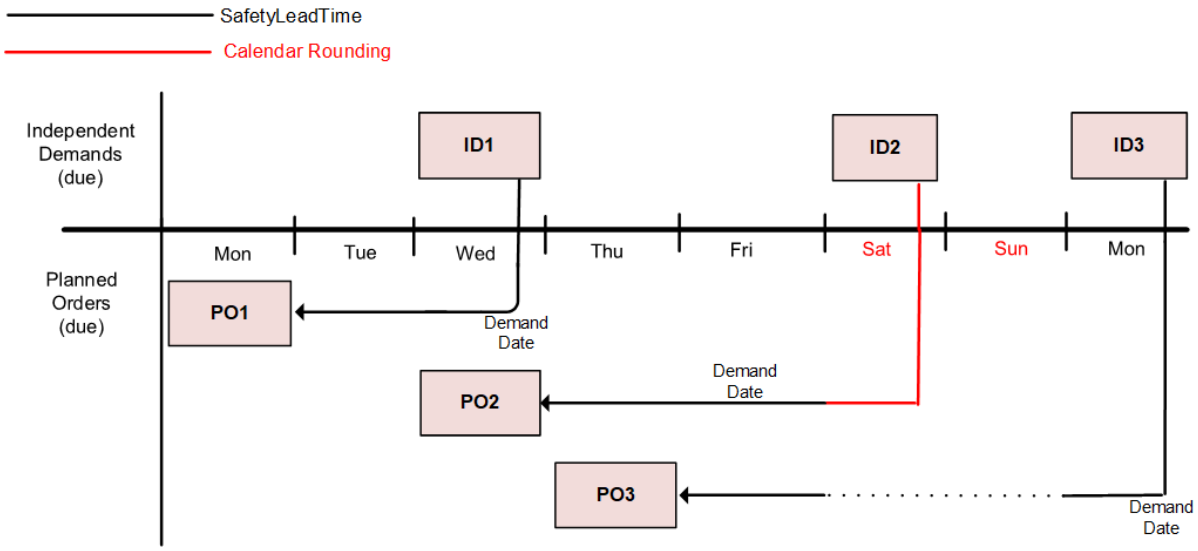
Safety lead time is used to provide a buffer between demand dates and the due dates on supplies needed to satisfy those demands. Based on the **PartType.SafetyLeadTimeCalendarRule** control, safety lead time can be applied in terms of the part's **TimeUnits** calendar (default) or the "Everyday" calendar. This setting can therefore affect planned order due dates when safety lead time is used.

For example, assume a part has the following relevant settings:

- **Part.SafetyLeadTime**- 2
- **PartType.UseSafetyLeadTime**- Y
- **PartType.SafetyLeadTimeCalendarRule**- TimeUnits
- **PlanningCalendars.TimeUnits**- Workday (Monday through Friday)

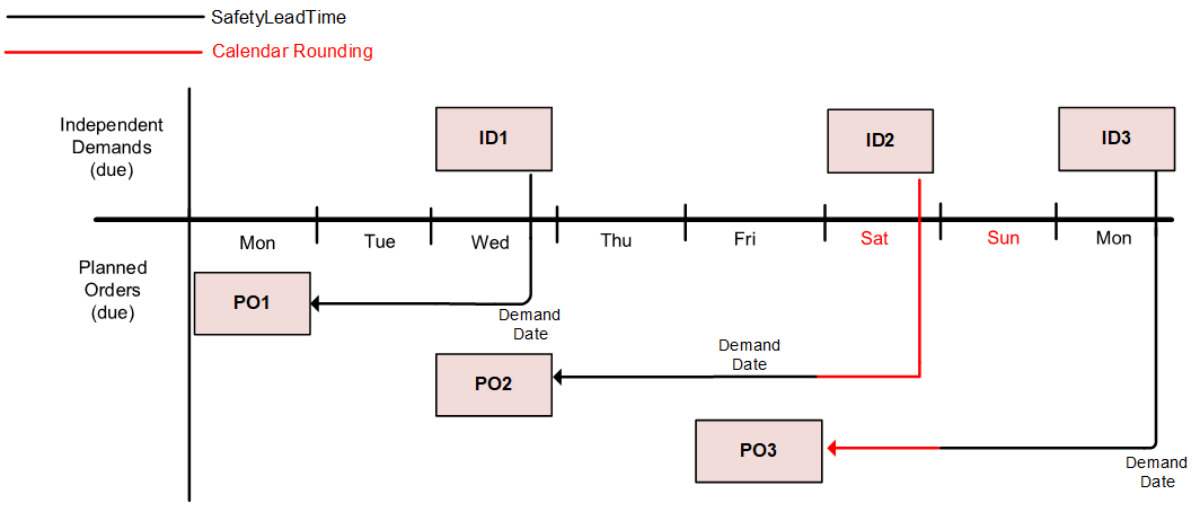
Given demands due on Wednesday, Saturday, and Monday then safety lead time is applied based on the part's **TimeUnits** calendar in order determine planned order due dates as shown in the following illustration. Thus, the Saturday and Sunday are not used in applying safety lead time to the Monday demand as they are not on the part's **TimeUnits** calendar. Note also that the demand due on Saturday is rounded to the previous **TimeUnits** interval (Friday) before applying safety lead time.

Figure 13.3 SafetyLeadTimeCalendarRule set to "TimeUnits"



Instead assume the part's **SafetyLeadTImeCalendarRule** were changed to "Everyday". In this case, supplies created to cover the demands due on Wednesday and Saturday would be processed the same as they were with the "TimeUnits" setting. However when planning supply to cover the demand due on Monday, safety lead time can now use non-working days and is applied from Monday through Saturday as shown in the following illustration. Note also that because planned order due dates must be on the **TimeUnits** calendar, the due date for this supply is rounded back to the Friday (however the planned order is still due one day later than it was with the "TimeUnits" setting).

Figure 13.4 SafetyLeadTimeCalendarRule set to "Everyday"



Inclusive and cumulative lead time

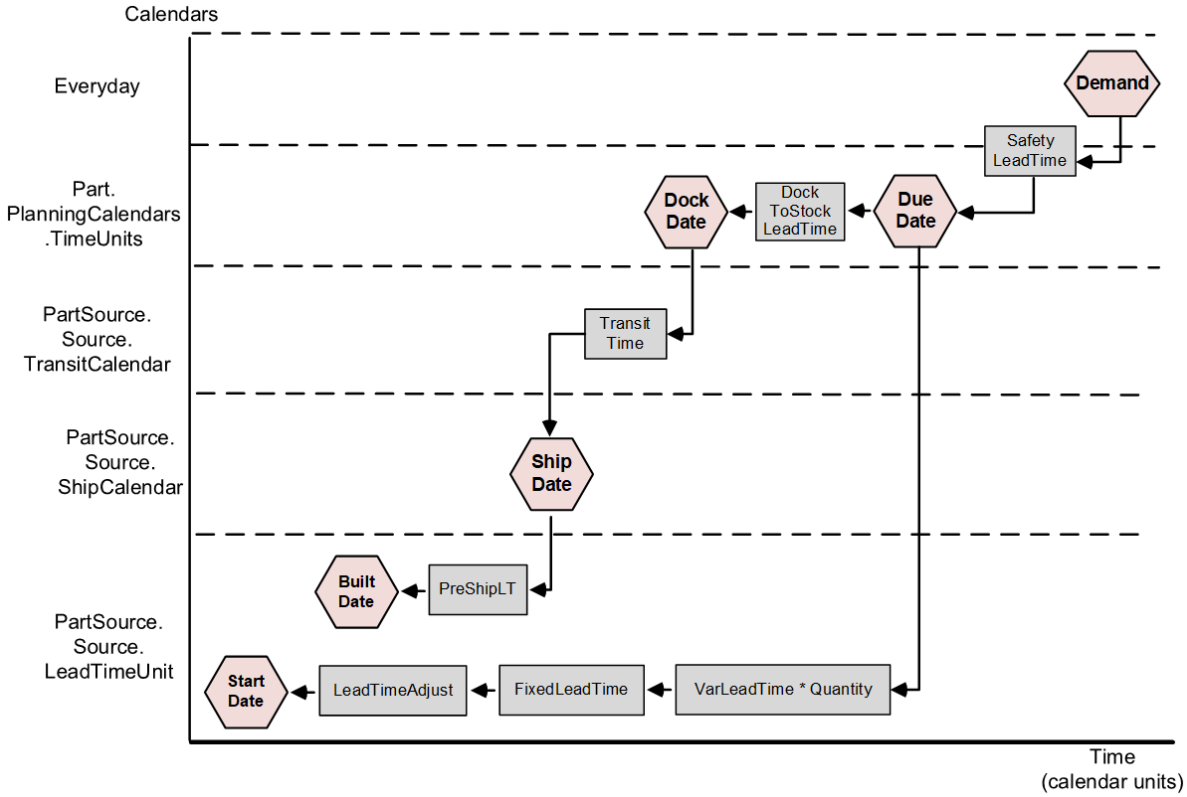
The **PartSourceType.TransitRule** control field determines how the various components of lead time are used in calculating total lead time and order start date for supplies from a given part source. Essentially, this setting controls whether any applicable pre-ship, transit, and dock-to-stock time are included in lead time and StartDate calculations (other parts of lead time like fixed and variable times are always included).

The TransitRule field has two settings; “Inclusive” (the default) and “Cumulative”.

Inclusive lead time

Inclusive lead time is used for all supplies having a transit rule of inclusive (that is, **PartSourceType.TransitRule** = 'Inclusive'). It calculates lead time and hence supply start dates using only fixed and variable lead times which are assumed to include all factors for pre-transit, shipping, and dock-to-stock time as shown in the following illustration. Note however that the pre-transit, shipping, and dock-to-stock times are still used in determining supply dock, ship and built dates.

Figure 13.5 Inclusive lead time calculations and applicable calendars

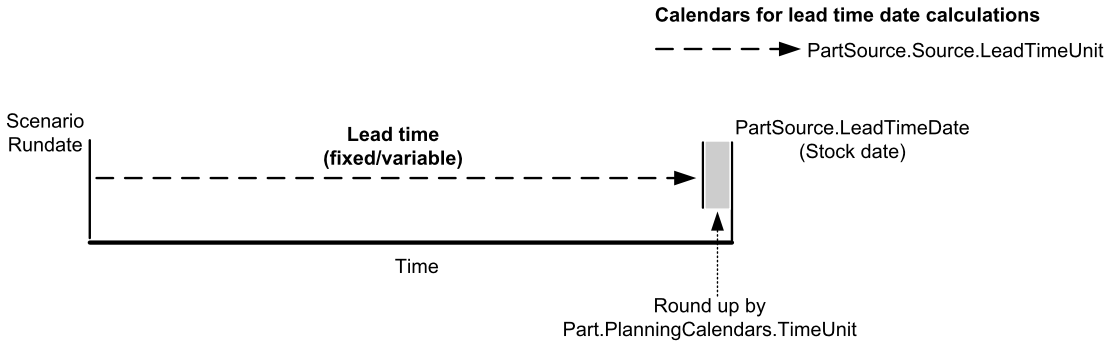


Note: The **PartType.SafetyLeadTimeCalendarRule** control setting determines whether safety lead time is applied using the part's **TimeUnits** calendar or the Everyday calendar. For more information, see "Safety lead time calendar" on page 1322.

Calculating the lead time date

The inclusive lead time date is calculated starting from the scenario run date (**Part.PlanningCalendars.RunDate.FirstDate**). Each of the components of inclusive lead time (fixed, variable, and adjusted) are then added to the run date. This calculation use the **PartSource.Source.LeadTimeUnit** calendar to adjust for any work stoppages (for example, weekends). The date resulting from adding all inclusive lead time components is then adjusted by the **Part.PlanningCalendars.TimeUnits** calendar to account for situations when the part and part source are using different calendars. This adjusted date is then reported in the **PartSource.LeadTimeDate** field as shown in the following illustration.

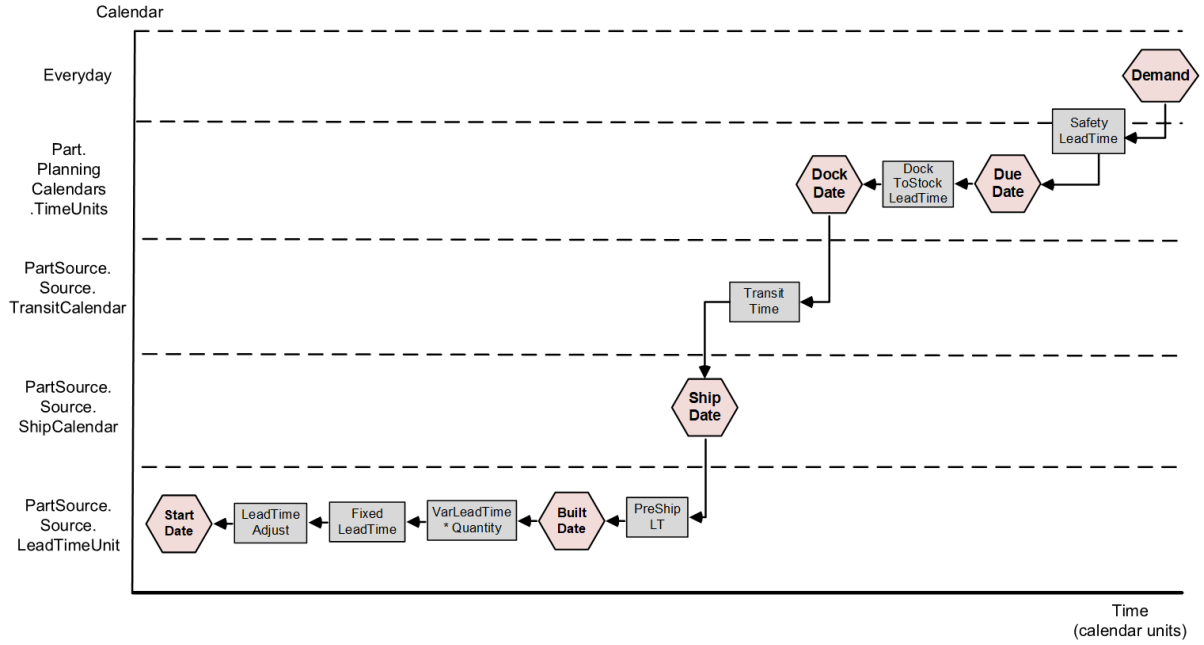
Figure 13.6 Inclusive lead time date calculations



Cumulative lead time

Cumulative lead time is used for any supplies having a transit rule of cumulative (that is, **PartSourceType.TransitRule** = 'Cumulative'). All applicable fields, including any DockToStockLeadTime, PreShipLT, and TransitTime values are used in determining total lead time for supply. To determine the required **StartDate** for a supply, fixed and variable lead times are applied back from the supply's calculated **BuiltDate** as shown in the following illustration. Note also that dependent demands for make or buy supplies are due at **StartDate,** and dependent demands for transfer supplies are due at **BuiltDate.**

Figure 13.7 Cumulative lead time and applicable calendars



Notes:

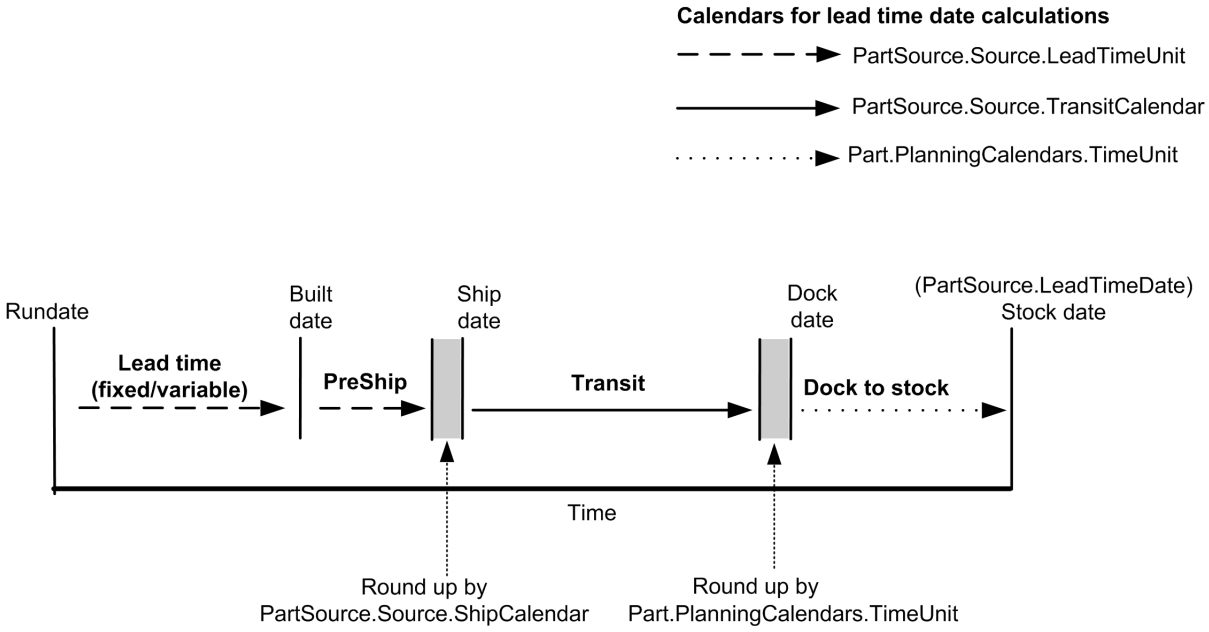
- The **PartType.SafetyLeadTimeCalendarRule** control setting determines whether safety lead time is applied using the part's **TimeUnits** calendar or in the "Everyday" calendar. For more information, see "Safety lead time calendar" on page 1322.
- For transfer sources, fixed and variable lead times at the destination site do not impact the corresponding due or start dates at the sourcing site (these are determined back from the **BuiltDate**). However, any fixed or variable lead time specified for a transfer source will still be used in determining a calculated **StartDate** at the destination site. For more information about lead time on transfer orders, see "Transfer lead time and explosion" on page 1732.

Calculating the lead time date

The cumulative lead time date is calculated starting from the scenario run date (**Part.PlanningCalendars.RunDate.FirstDate**). The following calculations are then performed.

1. Standard lead time components (fixed, variable, and adjusted) are added to the run date which gives the built date. This calculation use the **PartSource.Source.LeadTimeUnit** calendar to adjust for any work stoppages (for example, weekends). Note that this step does not apply to 'Transfer' part sources whose lead time date calculation starts from the built date.
2. Preship lead time is then added to the built date. This calculation also uses the **PartSource.Source.LeadTimeUnit** calendar to adjust for any work stoppages. Note that for 'Transfer' part sources this is the first calculation performed when determining lead time date.
3. The date resulting from adding preship lead time is adjusted up using the **PartSource.Source.ShipCalendar** calendar in order to find the next ship date.
4. Transit time is added to the ship date. This calculation uses the **PartSource.Source.TransitCalendar** calendar if populated (otherwise, the Everyday/Gregorian calendar is used and no work stoppages are assumed while in transit).
5. The date resulting from adding transit time is then rounded up using the **Part.PlanningCalendars.TimeUnits** calendar to determine the dock date.
6. Dock to stock time is added to the dock date. This calculation uses the **Part.PlanningCalendars.TimeUnit** calendar, and the resulting date is then reported in **PartSource.LeadTimeDate** as shown in the following illustration.

Figure 13.8 Cumulative lead time date calculations



Lead time rounding

Sometimes one or more lead time values specified might result in a non-integer value being returned. Typically, this can occur when using variable lead times. Because the **UseVarLeadTime** value is always multiplied by an order quantity, the resulting lead time quantity might frequently contain a fractional portion.

When a lead time value contains a fractional portion, the effective number of intervals to use in lead time calculations is determined by rounding as follows:

- If the fractional portion is less than .001, the fractional portion is ignored.
- Otherwise, if the fractional portion is greater than .001, round up to the next integer.

Example

Assume the following relevant input values for a particular part and source:

- PartSourceType.TransitRule— Inclusive
- PartSource.FixedLeadTime— 10
- PartSource.VarLeadTime— 0.005
- Part.LeadTimeAdjust— 0

Assuming an order for 2001 units, the number of days lead time required to build the order is calculated as:

- `(FixedLeadTime + (quantity * VarLeadTime))`
- `(10 + (2001 * .0005))`
- `(10 + 1.0005)`
- `11.0005` which by rule is rounded down to `11`.

Instead assume an order for 2002 units, in this case the number of days lead time required to build the order is calculated as:

- `(FixedLeadTime + (quantity * VarLeadTime))`
- (10 + (2002 * .0005))
- `(10 + 1.001)`
- `11.001` which by rule is rounded up to `12`.

Lead time overrides from scheduled receipts

If necessary, various values stored on the **Source** and **PartSource** table used in lead time calculations can be overridden with values stored on a **ScheduledReceipt** record. This might be commonly done for 'Transfer' scheduled receipts. The lead time values that can be stored on a scheduled receipt, and their equivalent Source/PartSource fields, are shown in the following table

Table 13.12: Leadtime values on the ScheduledReceipt table

ScheduledReceipt field	Equivalent to
DockToStockLeadTime	PartSource.DockToStockLeadTime
FixedLeadTime	PartSource.FixedLeadTime
PreShipLT	Source.PreShipLT
TransitTime	Source.TransitTime
VarLeadTime	PartSource.VarLeadTime

In order to use lead time values from the scheduled receipt, the following two fields are used:

- **SupplyStatus.LeadTimeOverride**—allows lead time values to be overridden at the scheduled receipt level. It can be set to either FromOrder (takes setting from SupplyType.LeadTimeRule field), No (lead time values should be taken from the part source), or Yes (lead time values taken from the scheduled receipt).
- **SupplyType.LeadTimeRule**—allows lead time values to be overridden at the order level. This field is used only if the value on the LeadTimeOverride field is set to 'FromOrder'. It can be set to either Ignore (lead time values taken from the part source) or Override (lead time values taken from the scheduled receipt).

Together the **LeadTimeRule** and **LeadTimeOverride** fields provide increased flexibility in whether a given scheduled receipt takes its lead time values from the scheduled receipt or the part source. The following table shows how different combinations of settings in these fields affect whether lead time values are taken from the part source or the scheduled receipt.

Table 13.13: Combinations of LeadTimeRule and LeadTimeOverride settings

SupplyType.LeadTimeRule	SupplyStatus.LeadTimeOverride	Lead time values come from
Override	Yes	ScheduledReceipt
Override	No	PartSource
Override	FromOrder	ScheduledReceipt

Table 13.13: Combinations of LeadTimeRule and LeadTimeOverride settings

SupplyType.LeadTimeRule	SupplyStatus.LeadTimeOverride	Lead time values come from
Ignore	Yes	ScheduledReceipt
Ignore	No	PartSource
Ignore	FromOrder	PartSource

Notes:

- All lead time values come from either the applicable part source or scheduled receipt. It is not possible to override just one lead time value (for example, TransitTime) on the scheduled receipt and get all other lead time values from the part source.
- Lead time overrides from scheduled receipts are not applicable to LeadTimeDate calculations.

CHAPTER 14: Material Requirements Planning calculations

RapidResponse performs Material Requirement Planning (MRP) calculations. This includes the calculation of a part's low-level code, a part's cumulative lead time, phantom processing, netting, explosion, and inventory turns. Results of RapidResponse calculations are stored in calculated tables or calculated fields located on input tables. For example, a record is added to the **PlannedOrder** table if netting calculations create a new planned order.

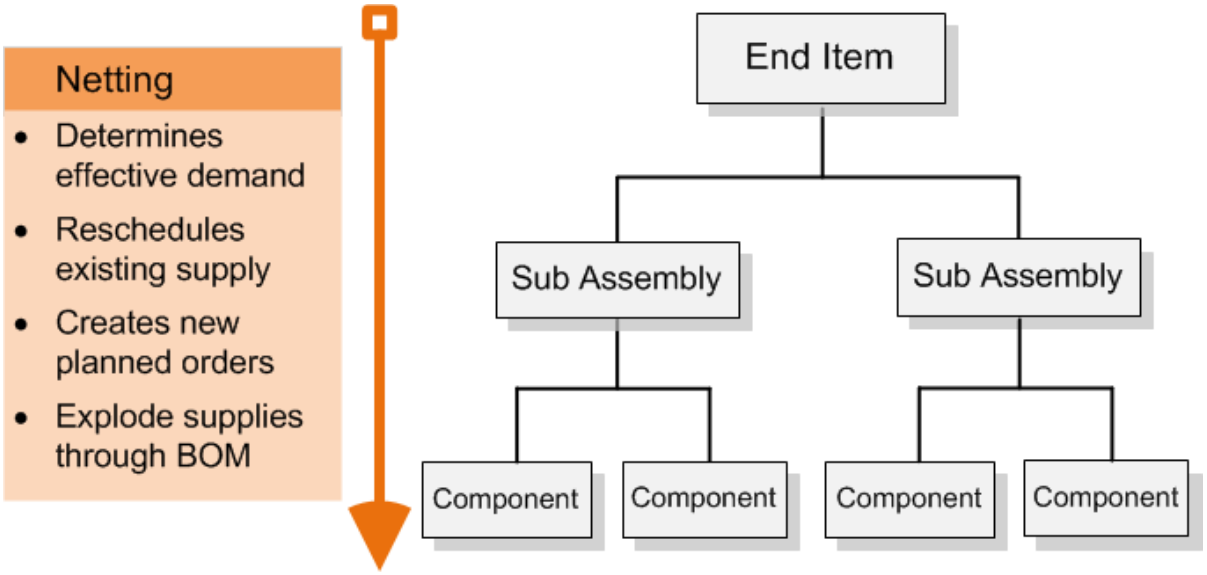
In this chapter, you'll learn about:

Netting basics 1331
Rescheduling supplies 1335
Explosion 1337
Planned orders 1343
Part source selection 1377
Phantom processing 1390
Yield 1391
Supply 1396
Demand 1397
Allocation data 1399
Safety stock 1400
Ignoring unsatisfied demands 1420
Recursive data 1423

Netting basics

Netting is a RapidResponse analytic that matches demand for a part with available supply. It does this by using available inventory (**OnHand** records), rescheduling scheduled receipts (**ScheduledReceipt** records), and recommending new planned orders.

Figure 14.1 Netting functions



Supply usage in Netting

Demands and supplies are netted on the basis of 'TimeUnits' buckets. If there is supply available, it is consumed. If not, the required quantity is passed to multi-sourcing functions to determine which **PartSource** is used. Demands due on a non-working period (determined by the **Part.PlanningCalendars.TimeUnits** field) are treated as being due at the end of the preceding work period. Supplies due on a non-working period (determined by the **Part.PlanningCalendars.TimeUnits** field) are treated as being available at the beginning of the subsequent working period.

Each demand bucket is processed from earliest to latest. Within each demand bucket, the Netting analytic performs the following basic process:

1. First, Netting checks to see if there is any existing OnHand that is available to cover the demand in the bucket. If so, as much of the available OnHand as is needed and available is allocated to the demand. If the demand in the bucket is now fully covered, then Netting proceeds to the next demand bucket and starts the process over.
2. Then, if there is not enough On Hand to cover the demand in the bucket, Netting will look for any scheduled supplies that can be used to cover the demand. Typically, this refers to any ScheduledReceipt records that are either on or before the need date or can be rescheduled to the need date. The "need date" is the due date of the demand backed off by the safety lead time on the part (**Part.SafetyLeadTime** and/or **PartSource.SafetyLeadTime**) as defined in **Part.PlanningCalendars.TimeUnits.** If a scheduled supply can be found to cover some or all of the demand then, as much of the scheduled supply as is needed or available is allocated to the demand. If the demand in the bucket is fully covered, then Netting proceeds to the next demand bucket and starts the process over.
3. Finally, if there is not enough existing supply (either on hand or scheduled receipts) to cover demand, only then will Netting create a planned order for the part. One or more effective part sources are selected for this purpose and one or more planned orders are created to cover the demand. If a demand cannot be satisfied from any eligible source at a given point (for example, due to effective dates on the PartSource records), it's set aside and satisfied by the first eligible

source that becomes effective. If a **PartSource** record does not exist for a given part, RapidResponse does not recommend planned orders for that part. When the demand in the bucket is covered, then Netting proceeds to the next demand bucket and starts the process over.

Note: In order for a part's demands and supplies to be processed by netting logic, the part must reference a **PartType.ProcessingRule** other than "Ignore" and have at least one valid **PartSource** record.

Example: Supply usage in the Netting analytic

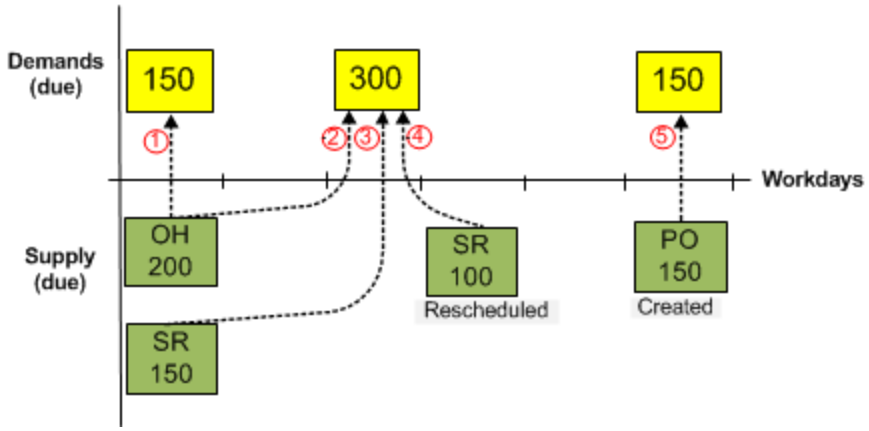
Assume the following set of input demands and supplies:

Table 14.1: Sample demands and supplies

Demand Date	Demand Quantity	Supply Date	Supply Type	Supply Quantity
June 2/14	150	Past	OnHand	200
June 4/14	300	June 2/14	ScheduledReceipt	150
June 9/14	1. 150	June 5/13	ScheduledReceipt	100

Assuming the scheduled receipts were reschedulable, then Netting calculations would align supply to demand as shown in the following illustration.

Figure 14.2 Netting with reschedulable input supply



The following details each of the steps performed:

1. Firstly, processes the first bucketed demand for 150 units due on June 2. Netting first looks for existing on hand inventory. There is sufficient on hand supply available, 200 units, so 150 of it is assigned to fully cover the demand.
2. Secondly, processes the next bucketed demand for 300 units due on June 4. The remaining on hand of 50 units is first assigned to this demand.

   With all on hand inventory gone and 250 units of this demand remaining unsatisfied, Netting next looks for scheduled supply on or before the demand need date.
3. 150 units of the scheduled receipt due on June 2 is assigned to this demand.

With no more scheduled supply available before the demand, Netting next looks for scheduled supply after the demand need date which can be rescheduled.

4. All 100 units of the scheduled receipt due on June 5 are rescheduled and assigned to this demand.
5. Thirdly, processes the last bucketed demand for 150 units due on June 9th. There is no remaining input supply available, so a new planned order for 150 units is created to fully cover the demand.

Note: In this example, the scheduled supplies are assumed to be fully reschedulable. If they were non-reschedulable, then a planned order for 100 units would have been required to satisfy the remainder of the demand on June 4 for 300 instead of rescheduling the later scheduled receipt available on June 5. In this scenario, that later scheduled receipt for 100 units would have been used towards satisfying the later demand due on June 9. For information about the rescheduling of supplies, see "Rescheduling supplies" on page 1335.

Block netting

When the **MUEPoolNettingType.UnitRule** control value is **Block**, RapidResponse performs block netting. Block netting does not affect how RapidResponse allots supply to demand. An existing supply is used to satisfy a demand (normal netting functionality). However, block netting determines how new planned orders are generated. The following specifies the block netting process:

1. The **StartUnit** field on open supplies (for example, OnHand records) is ignored.
2. Unused supplies are allotted to demands.
3. If there is no available supply to satisfy a demand, RapidResponse creates a new planned order. The **StartUnit** field on the **PlannedOrder** record is set to the largest of the **Part.NextUnit** field or the **StartUnit** field on the demand record.
4. Policy rules such as lot sizing and lot size last order are applied to the planned order based on the value of the **StartUnit** field.
5. Additional supply from a given planned order block can be used to satisfy a demand for the same block; however, a planned order from one block value cannot be used to satisfy demand from another block value.

Constraint

If constraints are loaded into RapidResponse, it considers constraint while performing netting. For information about how constraint affects netting, see "Matching supply to demand (2)" on page 1636.

Interchangeable parts

If a part is interchangeable with other parts, supplies of all parts in the interchangeable group are used to satisfy demands for any part in the interchangeable group. If demand cannot be satisfied with supply of the interchangeable parts, planned orders are generated for the primary, or leading, part.

For example, if parts A and B are interchangeable and part A is the primary part of the group, planned orders will be generated for part A, even if the demand is for part B. For more information about interchangeable parts, see "Interchangeable parts" on page 1545.

Rescheduling supplies

As discussed in "Example: Supply usage in the Netting analytic" on page 1333, whether a supply is reschedulable or not determines the demands it can be used to satisfy. That is, although a DueDate is provided as input on each **ScheduledReceipt** record, Netting might reschedule the supply or recommend a different due date based on when the supply is truly needed.

The ability to reschedule input supplies is controlled through a few key configuration settings, but scheduled receipts can generally be seen as belonging to one of the following three categories:

- **Reschedulable**—the scheduled receipt's dates can be moved, subject to some limits, and any dependent demand from the scheduled receipt will also be moved with it.
- **Recommend Only**—the scheduled receipt's dates can be moved, subject to some limits, but any dependent demand generated from the scheduled receipt will not be moved (based on the original due date).
- **Not Reschedulable**—the scheduled receipt's dates are firm and cannot be move at all (even if needed earlier).

Several control options determine which of the three general types of rescheduling apply to a given scheduled receipt as discussed in the following section. Respecting the applicable rescheduling rules, RapidReponse then calculates a **ReschedDate** on each **ScheduledReceipt** indicating when it should be due.

Controls over supply rescheduling

Whether or not a given scheduled receipt can be rescheduled depends on the **SupplyStatus.RescheduleRule** value referenced by the **ScheduledReceipt** record. This field controls rescheduling on an individual supply line basis and has the following options:

Table 14.2: RescheduleRule control field (SupplyStatus table)

Value	Description
FromOrder	Supply rescheduling is determined from the SupplyOrder. Whether the receipt can be rescheduled is determined by the **Order.Type.ProcessingRule** (points to **SupplyType** table). This is the default setting.
NonReschedulable	Supply is not reschedulable and dates are fixed (even if supply is needed earlier than scheduled).
Received	Supply has already been received into inventory and is ignored except to consume other "in-transit" scheduled receipts. Therefore, supply is not reschedulable (is effectively "ignored").
RecommendOnly	Supply is assumed to be reschedulable, however any dependent demand from the receipt is not rescheduled with it.
RecommendIfType	Supply is treated as if set to "RecommendOnly" unless the **Order.Type.ProcessingRule** is set to either "Ignored" or "NonReschedulable" in which case that setting is used instead.

Table 14.2: RescheduleRule control field (SupplyStatus table)

Value	Description
Reschedulable	Supply is assumed to be fully reschedulable. Any dependent demand from the receipt is also rescheduled with it.
Scheduled	Supply is not reschedulable. Similar to NonReschedulable, but if used in a constrained planning environment, constraint load can't be moved even if it results in overloading a constrained constraints.

As noted above, both the "FromOrder" and "RecommendIfType" settings in the **RescheduleRule** field rely on the rescheduling options defined on the **SupplyOrder** record through the **SupplyType.ProcessingRule** reference. The **ProcessingRule** field provides the following options which both impact rescheduling and also control whether scheduled receipts on the supply order should be ignored.

Table 14.3: ProcessingRule options (SupplyType table)

Value	Description
ExplodeOnly	Receipts are ignored expect to create dependent demand. Thus, considered as non-reschedulable (or more precisely, ignored).
Ignore	Receipts are ignored. Thus, considered as non-reschedulable.
InTransit	Receipts are used to consume other in-transit scheduled receipts. Thus, considered as non-reschedulable.
InTransitExact	Receipts are used to consume other in-transit scheduled receipts. Thus, considered as non-reschedulable.
NonReschedulable	Receipts are not reschedulable and dates are fixed (even if supply is needed earlier than scheduled).
RecommendOnly	Receipts are assumed to be reschedulable, however any dependent demand from a receipt is not rescheduled with it.
Reschedulable	Receipts are to be fully reschedulable. Any dependent demand from a receipt is also rescheduled with it. This is the default setting.

Limits on rescheduling

In the previous section, the controls that determine whether a scheduled receipt can be rescheduled at all were discussed. An additional control setting is available to limit how far back a scheduled receipt can be rescheduled (expedited). This is set through the **SupplyStatus.ExpediteLimit** field which contains the options outlined in the table below. Note also that there are no limits on how far out a reschedulable receipt can be delayed or canceled; the **ExpediteLimit** field only applies to those receipts that RapidResponse recommends should be expedited.

Table 14.4: ExpediteLimit options (SupplyStatus table)

Value	Description
DueDate	The scheduled receipt cannot be expedited at all. It can only be used on its DueDate, delayed to a later date, or cancelled entirely.
Past	There is no limit on how far the scheduled receipt can be expedited. The receipt is moved to where its needed to meet a demand (this includes being due at past if necessary). This is the default setting.
PTFDate	The scheduled receipt can be expedited as far back as the planning time fence date for its part source. This date is reported by the **PartSource.PTFDate** for the part source associated with the receipt. The planing time fence can be thought of as the earliest date on which a new planned order can be due. For more information, see
RunDate	The scheduled receipt can be expedited as far back as the part's run date as given by **Part.PlanningCalendars.RunDate.FirstDate**.

Note: Constraint availability can further limit the ability to expedite a scheduled receipt in RapidResponse.

RecommendedDate and ReschedDate

An input **DueDate** field should be populated on ScheduledReceipt records. However, based on where Netting calculations determine the receipt is actually needed, and subject to rescheduling and expedite rules, the following two related calculated fields are generated on scheduled receipts:

- **RecommendedDate**— This field considers rescheduling rules and expedite limits. It reports the date where Netting puts the scheduled receipt for the purpose of demand provision. If the supply is "RecommendOnly", it will report the date where netting wanted it without actually moving it there. Note that dependent demands are not generated with reference to this date.
- **ReschedDate**— This field also considers rescheduling rules and expedite limits. It reports that date where Netting puts the scheduled receipt in order to generate dependent demands. If the supply is "Reschedulable", then field is equivalent to the RecommendedDate. Otherwise, it is the same as the DueDate.

Explosion

Explosion is a RapidResponse calculation performed by the Netting analytic that creates allocations (dependent demand) for a component to be used in the production of an assembly. Allocation records created as a result of the Explosion calculation are stored in calculated tables such as the **NewAllocation** and the **PlannedAllocation** tables. For information about these calculated tables, see "Calculated tables" on page 937.

The **BillOfMaterial** table defines relationships between assembly and component parts. **BillOfMaterial** records contain a **Type** field (references the **BOMType** table) that defines the type of bill relationship and the processing characteristics. For example, does the product structure use a percent option ratio. For information about the **BOMType** table, see "BOMType table" on page 687.

The **BillOfMaterial.EffectiveInDate** and **BillOfMaterial.EffectiveOutDate** fields determine if a **BillOfMaterial** record is effective. Other rules are AnchorDate and Ignore. In addition to the effectivity rule, a bill record may be always effective, never effective or date effective.

Typically, each bill record has an effective date range (**BillOfMaterial.EffectiveInDate** and **BillOfMaterial.EffectiveOutDate**). A supply record must have a due date that falls in the range or it does not generate allocations. The actual date of the allocation record is the calculated start date (CalcStartDate) of the supply record, adjusted by the effective **LeadTimeOffset** field of the **BillOfMaterial** record.

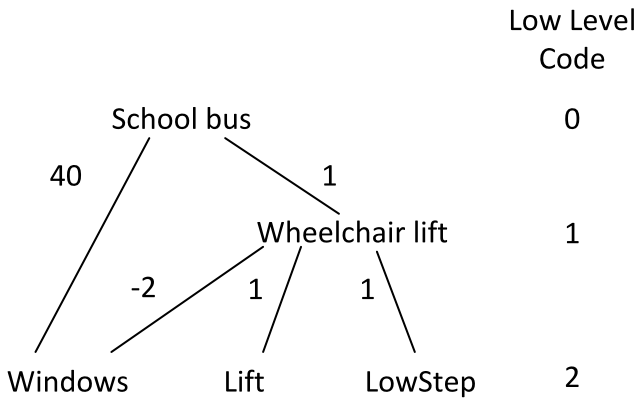
The quantity of a requirement of a supply is multiplied by the **BillOfMaterial.QuantityPer** field. The result can be rounded based on **BOMType.RoundingRule** control value.

Exploding bill records with a negative QuantityPer value

RapidResponse processes negative dependent demand at the component level to provide detailed and accurate product structure information.

To enable negative dependent demand in RapidResponse, set the **BOMType.ExplodeNegatives** control value to **Y**. If this field is not set to **Y**, then RapidResponse ignores a negative **BillOfMaterial.QuantityPer** value.

Figure 14.3 A bill of material product structure that generates negative dependent demand



The top level part is a school bus. Demand for one school bus causes a dependent demand for 40 windows. When a school bus has a wheelchair lift, it requires two fewer windows. This creates a dependent demand for -2 windows, resulting in a requirement for 38 windows as opposed to 40.

RapidResponse creates **NewAllocation** and **PlannedAllocation** records showing a negative quantity when it processes negative **BillOfMaterial** records. When positive demand is partially canceled by negative demand (instead of 40 windows, 38 are required), RapidResponse creates an **Activity** record with a **SupplyQty** field displaying the amount of positive dependent demand canceled.

In the previous example, RapidResponse creates an **Activity** record with a **SupplyQty** value of 2. The **DemandQty** field shows the original quantity of positive dependent demand (value of 40). Positive dependent demand that is completely canceled by negative demand is not reported in the **Activity** table; however, it's reported in the **NewAllocation** and **PlannedAllocation** tables.

Negative dependent demand is processed as regular demand and reduces positive demand if:

- Demand is on the same date.
- Demand uses the same Model, Unit, and Pool rules (if applicable).
- Demand is driven by the same PegOrder (planned order or scheduled receipt).

A portion of negative demand, which is not cancelled against a matching positive demand, is ignored when RapidResponse performs netting. However RapidResponse creates an **Activity** record showing dependent demand with both **SupplyQty** and **DemandQty** set to the ignored negative demand. For information about netting, see "Netting basics" on page 1331.

Because PegOrder is used to process negative **BillOfMaterial** records, any offsetting positive and negative **BillOfMaterial** records must either be on the same part or on one or more phantom parts below the parent part (that has the supply order). This results in the component ending up with positive and negative dependent demand from the same PegOrder. If a negative bill is passed through a normal part, there is no matching dependent demand on the component. For information about phantom parts, see "Phantom processing" on page 1390.

Consider the bill of material product structure shown previously. If the wheelchair lift is not a phantom part, then RapidResponse creates a planned order to satisfy the dependent demand generated by the school bus. Therefore the negative dependent demand on windows comes from a different PegOrder. Because there are two different PegOrders, the negative demand does not cancel the positive demand. This leads to additional and unnecessary demand for windows.

Explosion and scrap

When a parent supply explodes through a **BillOfMaterial** (BOM) record using a "Make" part source, the quantity of the parent supply will already have been inflated by its own **PartSource.Yield** value. Thus, this yield-inflated parent quantity is typically multiplied by the "quantity per" defined on the BOM to determine the dependent requirement placed on the component.

Additionally, a scrap factor can be defined directly on the **BillOfMaterial** to inflate the effective dependent demand quantity on the component rather than the supply itself. This might be done when there is something unique about the assembly-component combination defined on the BOM record that leads to loss of component supply. For example, the production process might destroy 5% of the component whenever it's used to build a given assembly. The following fields are used to configure scrap on the BOM.

Table 14.5: Fields for configuring scrap (BOM)

Table	Field	Description
BillOfMaterial	Scrap	An input factor by which component supply quantities are expected to be reduced when used in production of the assembly due to scrap/loss. The range of values considered here should be made in consideration of the ScrapRule setting.
BOMType	ScrapRule	A control setting that determines how the input Scrap value is interpreted and applied. Values are generally interpreted as either a ratio of component supply lost during assembly (Scrap*) or a ratio of component supply remaining after assembly (Yield*) and can be expressed as fractional or percentages. Valid options are: • **Ignore**—Scrap is ignored (no loss is assumed). • **YieldFraction**—Scrap values from 0 through 1 (1 means no loss). • **YieldPercent**— Scrap values from 0 through 100 (100 means no loss). • **ScrapFraction**—Scrap values from 0 through 1 (0 means no loss). • **ScrapPercent**—Scrap values from 0 through 100 (0 means no loss). • **InflationFraction**—Scrap indicates amount to inflate dependent demand by the formula 1 / (1 + BillOfMaterial.Scrap). • **InflationPercent**—Scrap indicates amount to inflate dependent demand by the formula 1 / (1 + BillOfMaterial.Scrap * .01)

Controlling explosions from the supply type

Explosion occurs on supply that is manufactured (make parts), and only manufactured parts have **BillOfMaterial** records defining required components. However, there are cases where a part can be both manufactured and purchased, or alternatively, where the part was previously manufactured and is now purchased. The **SupplyType.Source** control value determines if a part is manufactured or purchased.

Rescheduled dates

The explosion process can adjust the allocation's DueDate if the associated scheduled receipt is rescheduled, even if the allocation is pre-exploded (allocation records are imported into the **Allocation** table). The date of the allocation is adjusted by the same amount as the scheduled receipt's

RecommendedDate and ReschedDate. The **PlanningCalendar.TimeUnits** field of the assembly is used as the basis for the calculations.

Explosions for vendor managed parts

The type of a part (determined by the **Part.Type** field) can override the explosion and determine whether it generates allocations. **PartType.DependentDemandUsage** control value defines behavior where explosion generates allocations but netting does not use them. This type of part represents a vendor managed item. The demand for the item is calculated, but the vendor's enterprise data source manages the bucketing and netting of the part.

Explosion for parts

Assembly parts normally represent a separate stage in the overall production of a product. However, some sub-assemblies are never built or stocked, but instead represent a kit of parts to be assembled as part of a higher level operation. Some parts can be handled this way, in addition to being individually built. These kinds of behaviors are referred to as phantom parts. For information about phantom parts, see "Phantom processing" on page 1390.

Low level code

The **Part** table has a calculated field named **LowLevelCode**. It represents the length of the longest path up the bill of material product structure. The **LowLevelCode** field is calculated as the maximum number of WhereUsed (bill records where the part is the component) it takes to reach a part that has no parts above it (WhereUsed is empty). The calculation considers all **BillOfMaterial** records regardless of their type or effectivity. The low-level code of a top-level part is zero.

Anchor date logic and material explosion

Bill effectivity based on an anchor date on the scheduled receipt, rather than the order start or due dates, allows material allocations for a specific order to be frozen, regardless of the date on which it is eventually built. Individual bill records are effective based upon the anchor date on the supply and not on the start date of the supply record. Anchor date is passed through phantoms but is not passed through non-phantom parts (for example, not passed through netting).

Bill effectivity is controlled by the setting in the **BOMType.DateRule** control field. If anchor date logic is desired, the **BOMType.DateRule** control field should be set to **AnchorDate**. If normal bill explosion is desired, the **BOMType.DateRule** control field should be set to **CalcStartDate**.

Determining the type of effectivity to use is a two-step process. The following **BOMType** fields are considered:

- **EffectivityRule**—it can be one of: **Never**, **Always**, **InclusiveExclusive**, **Exclusive** or **Inclusive**.
- **DateRule**—it can be one of: **AnchorDate, BlowThroughCalcDueDate, CalcDockDate, CalcDueDate, CalcStartDate**, or **Ignore**.

Anchor date logic only affects material explosion of scheduled receipts or blow through new allocations. The anchor date is taken from the scheduled receipt or the referenced scheduled receipt. If anchor date is used, bill offsets do not apply to effectivity logic. However, bill offsets continue to apply for dependent demand due dates.

Visualizing the bill of material in a BOM chart

Some workbooks can include a chart that shows the bill of material. A BOM chart provides you with an up-to-date view of the product structure, and you can filter the chart to focus only on the components or sites (or other criteria) that are relevant to you. The BOM Visualization worksheet in the BOM Analysis workbook includes a BOM chart.

In a BOM chart you can:

- Control your view by moving the whole chart, moving individual nodes, or zooming in and out of the chart.
- Focus on a single component and its relationships.
- View substitutes, co-products, and by-products for all components or a specific component.
- Reset the chart to return to its default view.

For more information, see the *RapidResponse User Guide*.

Explosion and order BOMs

Assemblies can be exploded using either an order BOM or a material BOM. The order BOM can be defined at multiple levels in the product structure. The same assembly can use an order BOM for one order and a material BOM for another order. Once an assembly uses a material BOM instead of an order BOM, then all multi-level components below that assembly also use the material BOM. Existing supplies at multiple levels might be intended for a particular order as well. These supplies should be used to satisfy only the associated order's component demands.

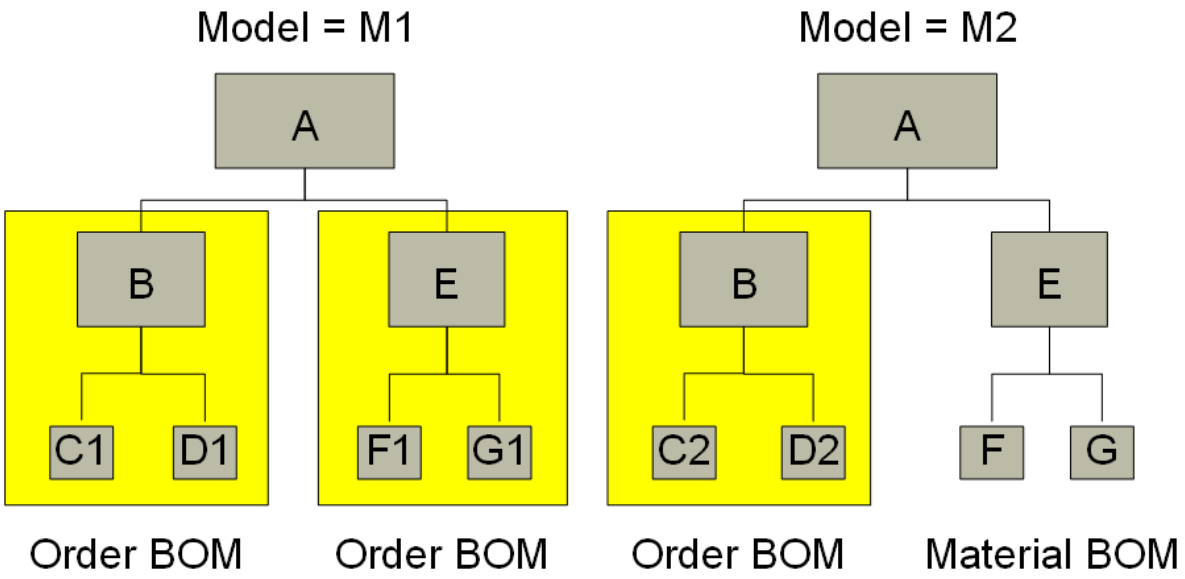
Supplies in the same assembly can be exploded using either an order BOM or a material BOM. Demand can be satisfied using order-specific components or standard components depending on whether the demand is order-specific of standard. Excess on a standard supply created to satisfy one demand can be used to satisfy another demand that should not be treated as order-specific.

The **DependentDemandModel** field on the **BOMType** control table does the following:

- If null, uses the value of the parent model field for component demands.
- If not null, uses this value for component demands.

For example, in the following figure, there are two demands for part E, each of which is for a different model.

Figure 14.4 Example of part E in two different orders



In this example, we need the following:

- A model to indicate we are in a material BOM, which we will call **Material**
- A **BOMType** with **MUERule** = **Model** and null **DependentDemandModel**, which we will call Model (it is likely that such a record would already exist)
- A **BOMType** with **MUERule** = **Model** and **DependentDemandModel** = **Material**, which we will call **Material**

The following table displays the BOM types for assembly A.

Table 14.6: BOM for assembly A

Assembly	Component	Model	QtyPer	BomType.MUERule
A	B	M1	1	Model
A	E	M1	1	Model
A	B	M2	1	Model
A	E	M2	1	Material

Now, when we explode A with model M2, the dependent demand for B still has model M2, but the dependent demand for E now has model **Material**, as desired. The remaining BOM entries can all use the existing behavior.

Planned orders

Planned order and rescheduling discussions are broken into the following areas:

- "Planning time fence" on page 1344
- "Planned orders with MRP rules" on page 1351
- "Planned orders with order point" on page 1351
- "Inventory levels and planned orders" on page 1358
- "Order Policy and planned orders" on page 1358
- "Planned order lot sizing" on page 1358
- "Days of Supply calculations" on page 1362
- "Shortage calculations" on page 1375
- "Limitations on the number of planned orders" on page 1376
- "Start date calculation for planned orders" on page 1376
- "Spreading planned orders with Takt Time" on page 1376

Notes:

- For information about RapidResponse creating planned orders while considering constraint, see "Creating planned orders" on page 1638.
- A Configuration record, named MaximumPlannedOrdersPerPeriod, controls the number of planned orders that RapidResponse generates for a specific time period. For information about this record, see "Limiting the number of planned orders" on page 2204.

Planning time fence

The planning time fence defines a boundary within which change may adversely affect the plan. In RapidResponse, it is represented by the **PartSource.PTFDate** field which is calculated out from the **RunDate.**

Conceptually, the **PTFDate** represents a date before which planned orders cannot be due. For example, if the planning time fence is defined such that the **PTFDate** is at the lead time of a part source, then any demand before the lead time date for the part source cannot be covered on time by a planned order. In such cases, if there is not enough on-hand inventory or scheduled supply to cover the near-in demand and that part source must be used, then the required planned order(s) will be placed at the **PTFDate** and the demand will be satisfied late.

Note that how the **PTFDate** is actually determined, and its impact on planned order generation, depends on several input and control configuration settings as discussed in the following section.

Determining the PTFDate

Several control table settings influence how the **PTFDate** is calculated for each part source. Additionally, this calculation can by affected by the part source's lead time and planning time fence values, as well as any scheduled receipts or supply allocations assigned to the part source.

The following fields are used by RapidResponse in calculating the **PTFDate**:

Table 14.7: Fields affecting calculation of PTFDate

Table	Field	Description
PartSource	PlanningTimeFence	Specifies the number of calendar intervals after **RunDate** to set the planning time fence. Expressed in either the part's "TimeUnits" calendar or the part source's "LeadTimeUnit" calendar as set by the value in **OrderPolicy.PTFUnit**. Note that the interpretation of this value depends on the **OrderPolicy.PTFRule** setting. Based on the **PTFRule** setting, the duration of the planning time fence can be set to either the value in this field, or the sum of this field and lead time. Therefore, for cases where the **PTFDate** is meant to be exactly at lead-time, this field should be set to zero (0).
OrderPolicy	PTFUnit	Specifies whether the part or part source's calendar should be used when calculating the planning time fence. Valid values are: • **Part**—planning time fence is calculated in terms of the part's time units (the calendar referenced from **Part.PlanningCalendars.TimeUnits**). • **Source**—planning time fence is calculated in terms of the source's time units (the calendar referenced from **PartSource.Source.LeadTimeUnit**). This is the default value.
OrderPolicy	PTFRule	Specifies how the duration of the planning time fence, and hence the calculated **PTFDate** field on the **PartSource** table, is determined for a given order policy. Valid values are: • **Fence**—The **PTFDate** is based on the value in the **PlanningTimeFence** field. This is the default value. • **LastDueFence**—The **PTFDate** is set to the later of the calculation described for the "Fence" option above, or the **DueDate** of the latest scheduled receipt or supply allocation assigned to the part source. • **LastDueFenceAll**—The **PTFDate** is set to the later of the calculation described for the "Fence" option above, or the **DueDate** of the latest scheduled receipt or supply allocation assigned to any part source that supplies the part. • **LastDueLead**—The **PTFDate** is set to the later of the calculation described for the "Lead" option below, or the **DueDate** of the latest scheduled receipt or supply allocation assigned to the part source. • **LastDueLeadAll**—The **PTFDate** is set to the later of the calculation described for the "Lead" option below, or the **DueDate** of the latest scheduled receipt assigned to any part source that supplies the part. • **Lead**—The **PTFDate** is based on lead time and the value in the **PlannningTimeFence** field (if provided). Thus, if **PTFDate** is meant to be set exactly to the lead time of the part source, then ensure the **PlanningTimeFence** value is 0 (zero).

Table 14.7: Fields affecting calculation of PTFDate

Table	Field	Description
SupplyAllocationType	PTFLastDueRule	Specifies whether supply allocations of a given type are considered when calculating the **PTFDate** for part sources that use any of the "LastDue" settings in **OrderPolicy.PTFRule**. Valid values are: • **Exclude**—the last due supply allocation of this type assigned to a given part source can be used in calculating **PartSource.PTFDate**. • **Include**—the last due supply allocation of this type assigned a given part source is not used in the logic to calculate **PartSource.PTFDate**.
SupplyType	PTFLastDueRule	Specifies whether scheduled receipts under supply orders of a given type are, by default, considered when calculating the **PTFDate** for part sources using any of the "LastDue" settings in **OrderPolicy.PTFRule**. Valid values are: • **Exclude**—for a given part source, the last due scheduled receipt under a supply order of this type can be used in calculating **PartSource.PTFDate**. • **Include**—for a given part source, the last due scheduled receipt under a supply order of this type can used in calculating **PartSource.PTFDate**.
SupplyStatus	PTFLastDueRule	Can be used to override the supply order's **SupplyType.PTFLastDueRule** setting on a scheduled receipt by scheduled receipt basis. Valid values are: • **Exclude**—scheduled receipts with this status are never used in the logic to calculate the **PTFDate** for part sources having any of the "LastDue" rules (regardless of the **SupplyType.PTFLastDueRule** setting on the supply order they belong to). • **FromOrder**—scheduled receipts with this status inherit and use the **SupplyType.PTFLastDueRule** setting from the supply order they belong to. • **Include**—scheduled receipts with this status can be used in the logic to calculate the **PTFDate** for part sources having any of the "LastDue" rules (regardless of the **SupplyType.PTFLastDueRule** setting on the supply order they belong to).

Note: The **OrderPolicy.OrderGenerationRule** setting determines whether the calculated **PTFDate** will impact planned order due dates from a given part source. For more information, see "OrderGenerationRule" on page 777.

Example 1: Planning time fence calculation and planned orders

Assume a RunDate of December 1, the lead time for the part is 3 days, and all relevant part and source calendars use a common Workday calendar.

Further, assume the following set of input demands and supplies.

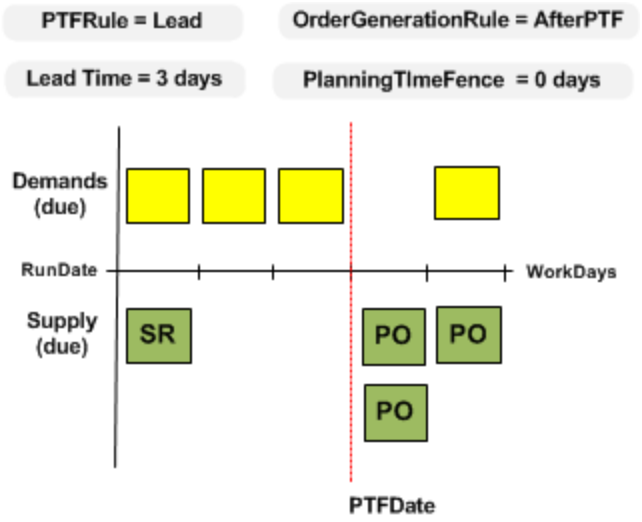
Demand Date	Demand Quantity	Supply Date	Supply Quantity
Dec 1	100	Dec 1	100
Dec 2	100		
Dec 3	100		
Dec 5	100		

Further assume the following settings.

- **PTFRule**— Lead
- **PlanningTimeFence**— 0 days
- **OrderGenerationRule**—AfterPTF

Based on the **PTFRule** setting, both lead time and **PlanningTimeFence** values are considered and the **PTFDate** is calculated as December 4. Based on the **OrderGenerationRule** setting, new planned supplies cannot be generated before the PTFDate and so the planned orders required to satisfy demands from December 2 and December 3 are both due on December 4. Refer to the following illustration.

Figure 14.5 PTFDate with "Lead" and "AfterPTF" settings

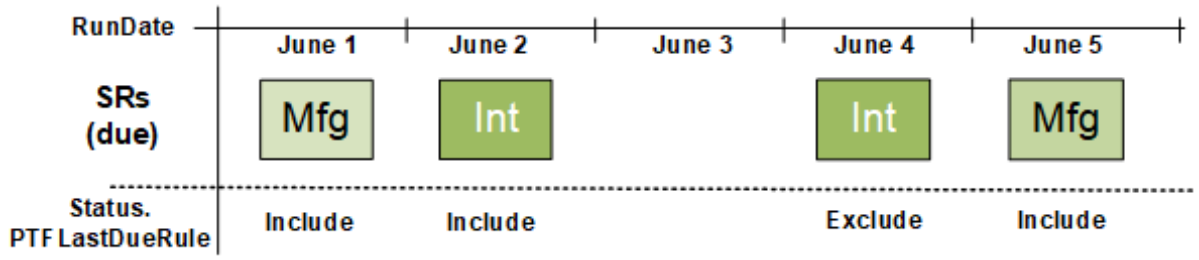


Example 2: Impact of scheduled supply on planning time fence

Depending on control settings, scheduled receipts and supply allocations can impact the **PTFDate** calculation for a given part source. For part sources that use any of the "LastDue" options in the **OrderPolicy.PTFRule** field, the due date from its last due scheduled receipt or supply allocation can potentially become its **PTFDate**.

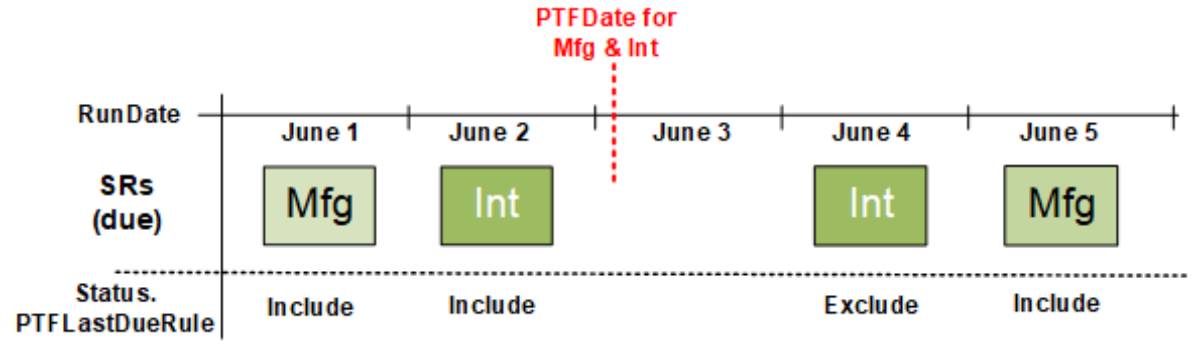
For example, assume a given part is provided by two sources; "Mfg" and "Int". Further assume that the **RunDate** is June 1st, each part source has a lead time of 2 days, and there is scheduled supply for the part as shown below along with the **PTFLastDueRule** applicable to each.

Figure 14.6 Scheduled receipts from two part sources



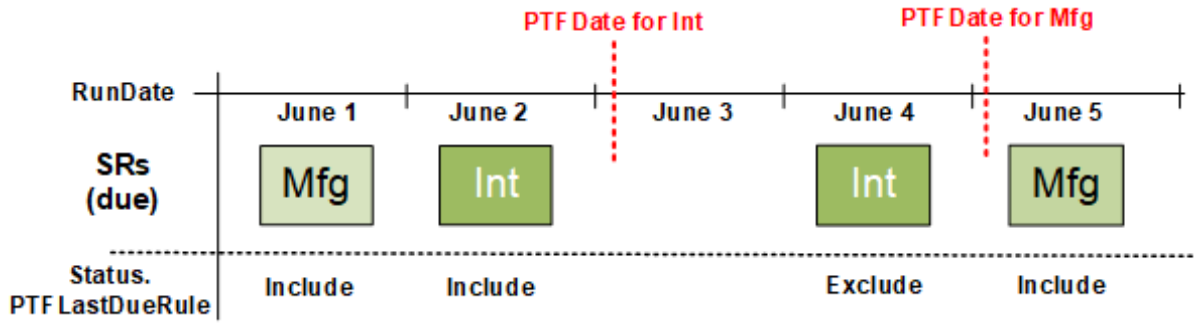
Assuming each part source had a **PTFRule** setting of "Lead", then their calculated **PTFDate** values would be just on lead time as shown below. In this case, the two part sources have an equivalent **PTFDate** of June 3rd as they happen to have identical lead times of 2 days.

Figure 14.7 PTFDate with "Lead" setting



Instead, if a **PTFRule** setting of "LastDueLead" was used, then their **PTFDate** values would be set as shown below. In this case, the part source "Mfg" has a **PTFDate** of June 5th corresponding to the last due scheduled receipt associated with this part source and which is later than lead time. However, the part source "Internal" still has a **PTFDate** of June 3rd which corresponds to its lead time. This is because its only scheduled receipt eligible for use in the planning time fence calculation is due within lead time (its other later scheduled receipt has a **PTFLastDueRule** of "Exclude")

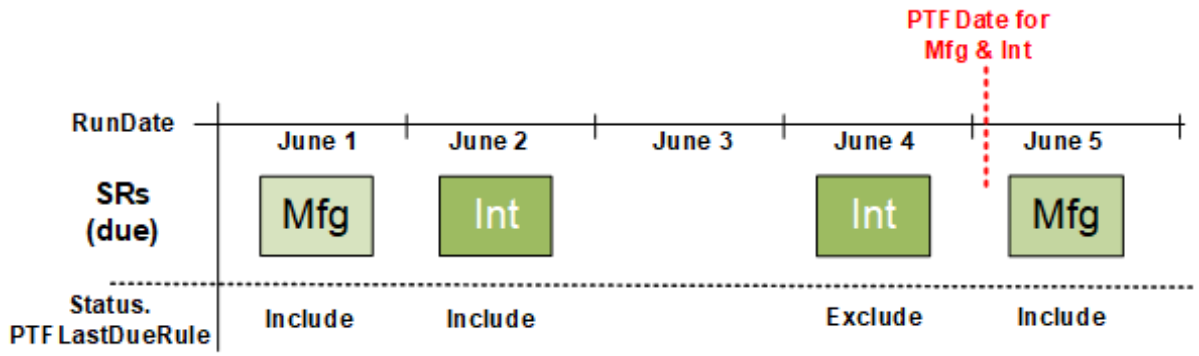
Figure 14.8 PTFDate with "LastDueLead" setting



Finally, if the PTFRule of "LastDueLeadAll" was used, then the **PTFDate** would be set as shown below. In this case, both part sources have a **PTFDate** of June 5th which corresponds to the latest due scheduled

receipt for the part from either part source. In other words, the due date from the last scheduled receipt from the "Mfg" part source is used as this supply is eligible for use in the planning time fence calculation and is outside of lead time for both part sources.

Figure 14.9 PTFDate with "LastDueLeadAll" setting



PTFDate and Days of Supply

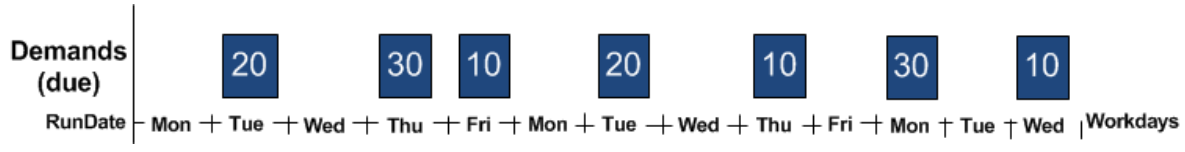
If using days of supply logic to satisfy groups of accumulated demands with a single planned order, the calculated **PartSource.PTFDate** can be impacted by the **PeriodSupplyDueCalendar**. This calendar is set on a part's referenced **PlanningCalendars** or **DaysSupplyPolicy** record and is typically used with period-based days of supply policies (such as, the "ByPeriod" or "ByPeriodEnd" settings). The **PeriodSupplyDueCalendar** specifies dates on which new planned orders can be generated to satisfy accumulated demands (for example, planned orders might always be due at the start of a Week or only on Thursdays).

Thus if the part uses period-based days of supply, that **PTFDate** must necessarily belong to a date on the **PeriodSupplyDueCalendar**. If the calculated **PTFDate** would otherwise fall in between dates on the part's**PeriodSupplyDueCalendar**, the **PTFDate** is adjusted ahead to the next subsequent date on the **PeriodSupplyDueCalendar**. In some cases, this can lead to near-term planned orders being made late or later than they otherwise would be.

For example, assume a part using days of supply logic has the following settings related to the planning time fence:

- **OrderGenerationRule**- AfterPTF
- **PTFRule**- Fence
- **PlanningTimeFence**- 3

Further assume the part has the following demands in its planning horizon:

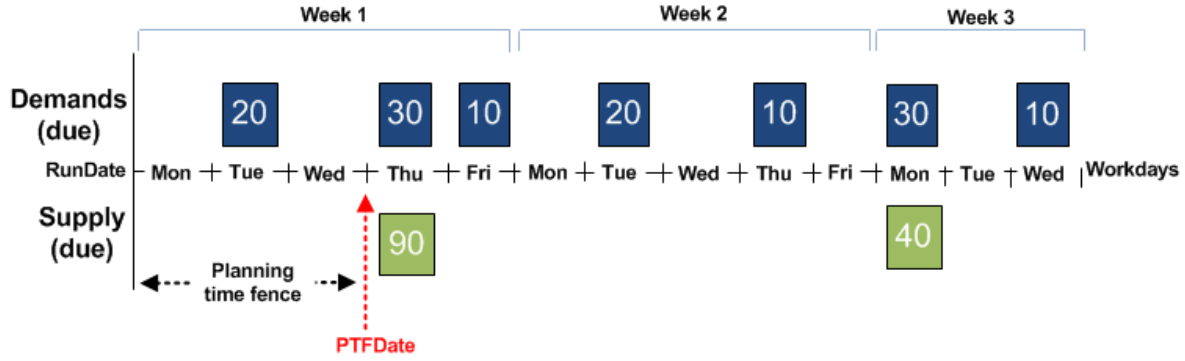


Finally, assume the part utilizes the following days of supply settings (indicating planned orders should satisfy two weeks of demand):

- **DaysSupplyRule**- ByPeriod
- **PeriodSupplyInterval**- Week
- **NumberOfDaysSupply**- 2
- **PeriodSupplyDueCalendar**- Workday

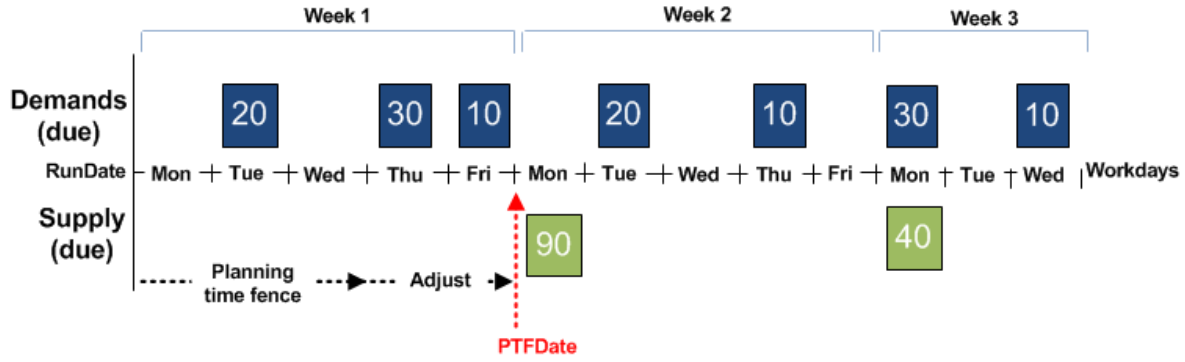
In this case, the part's **PTFDate** would be calculated and its planned orders generated as shown in the following illustration. In this case, the **PTFDate** is calculated three days out from **RunDate** which falls on the **PeriodSupplyDueCalendar** of "Workday". Thus, a planned order can be generated at this **PTFDate** and only 20 of the 90 units of demand in the first two week period is planned late.

Figure 14.10 PTFDate with PeriodSupplyDueCalendar set to Workday



Instead assume the same settings as shown previously but with the **PeriodSupplyDueCalendar** changed to "Week". In this case, the part's **PTFDate** is calculated and its planned orders generated as shown in the following illustration. Note that the **PTFDate** is again intially calculated three days out from **RunDate**. However, because this date falls in between the PeriodSupplyDueCalendar markers, it is then moved out to the next Week marker. Thus, the planned order covering the first two weeks of demand is not generated until the following Monday and 60 of the 90 units of demand in the first two week period is planned late.

Figure 14.11 PTFDate with PeriodSupplyDueCalendar set to Week



Note: For more information about days of supply logic, see "Days of Supply calculations" on page 1362.

Planned orders with MRP rules

When the **SafetyStockPolicy.SafetyStockRule** field is neither **OrderPointAverage**, **OrderPointOver**, or **OrderPointUnder**, planned orders are planned to arrive when the inventory is projected to fall below the safety stock level (determined by the **Part.SafetyStockQty** field).

Planned orders with order point

Order Point is often called Re-Order Point (or ROP) and is commonly used in "lean" production environments. With this method of planning, when the projected inventory balance drops below the re-order point, a replenishment order is then started. The parameters for determining the reorder point include such things as the time it takes to obtain a typical replenishment order, how big and variable the demand pattern is and what level of service you want to provide to that demand pattern.

Order point parts are those parts with a **SafetyStockPolicy.SafetyStockRule** setting of either "OrderPointAverage", "OrderPointOver", or "OrderPointUnder" (or if a part's **SafetyStockPolicy** reference is Null, then the **SafetyStockRule** setting on the referenced **PartType** record is used).

In order to generate planned orders for order point parts, RapidResponse needs to determine both the order point threshold and the order quantity. The order threshold is always set to the safety stock level (as determined by the **SafetyStockQuantityRule**), and so orders are started when projected inventory falls at or below the safety stock level. Thus for order point parts, lead time can be thought of as being applied forward from the start date for planned orders, rather than backwards from the due date (this implies that stock outs might occur if safety stock is set too low).

Note: This section assumes a part's order point settings are configured using the **SafetyStockRule**, **SafetyStockQuantityRule**, and **SafetyStockDateRule** settings are configured using the **SafetyStockPolicy** table. However, if a part's **SafetyStockPolicy** reference is Null, then equivalent fields on the P**artType** table are used instead (this corresponds to how settings were always configured in versions prior to 2014.4).

Order point threshold and quantity calculations

The order point threshold determines when RapidResponse generates planned orders. It is the point when inventory for a given part is projected to fall at or below the safety stock level. The safety stock level (and hence the order point threshold) used in this calculation depends on the setting in **SafetyStockPolicy.SafetyStockQuantityRule** as outlined in the following table:

Table 14.8: SafetyStockQuantityRule and order point threshold

SafetyStockQuantityRule	Order point threshold calculated as
FixedQuantity	The point when inventory is projected to fall at or below the value in **Part.SafetyStockQty**.
FractionOfDemand	The point when inventory is projected to fall at or below a specified fraction of the total demand (as set in **Part.PercentSafetyPercent**) calculated over a defined number of intervals beginning from **Part.PlanningCalendars.RunDate** (where the interval value is set by **Part.SafetyStockPolicy.PercentSafetyInterval** and the number of these intervals to use is set by **Part.PercentSafetyIntervalCount**).

Table 14.8: SafetyStockQuantityRule and order point threshold

SafetyStockQuantityRule	Order point threshold calculated as
PercentOfDemand	The point when inventory is projected to fall at or below a specified percentage of the total demand (as set in **Part.PercentSafetyPercent**) calculated over a defined number of intervals beginning from **Part.PlanningCalendars.RunDate** (where the interval is set by **Part.SafetyStockPolicy.PercentSafetyInterval** and the number of these intervals to use is set by **Part.PercentSafetyIntervalCount**).
RangeOfCoverage	The point when inventory is projected to fall at or below the average daily requirement, plus an optional buffer quantity, for a specified number of days within one of three configurable zones. For more information, see "RangeOfCoverage calculations" on page 1409.
RangeOfCoverageLeadTime	The point when inventory is projected to fall at or below the average daily requirement, plus an optional buffer quantity, within a part's lead time. For more information, see "RangeOfCoverage calculations" on page 1409.
TimePhasedQuantity	The point when inventory is projected to fall at or below the date effective value in **TimePhasedSafety.Quantity**.

Once the order point threshold has been reached, RapidResponse can create planned orders (with a start date equal to the due date of the demand they are satisfying). The actual order quantity used depends on the value in the **SafetyStockPolicy.SafetyStockRule** as outlined in the following table.

Table 14.9: SafetyStockRule and order point quantity

SafetyStockRule	Order quantity calculated as
OrderPointAverage	The value defined in **Part.AverageQty** (or **Part.SafetyStockQty** if **Part.AverageQty** is less than or equal to 0), and orders of this size are generated until inventory levels meet or exceed the order point threshold.
OrderPointOver	The largest number that brings inventory levels closest to the maximum of either the order point threshold or **Part.AverageQty** without exceeding it. This setting is intended to keep inventory levels in a range bounded by the order point threshold and **Part.AverageQty**. If using this setting, it's recommended to set **SafetyStockQuantityRule** to "FixedQuantity", and ensure that **AverageQty** is set to a value greater than **SafetyStockQty** .
OrderPointUnder	The smallest number that brings inventory levels equal to or greater than the maximum of either the order point threshold or **Part.AverageQty**. This setting is intended to keep inventory levels above the order point threshold.

Note: When the **SafetyStockPolicy.SafetyStockRule** field is set to any of the order point options outlined in "SafetyStockRule and order point quantity" on page 1352, it is recommended that the **Part.Type.UseSafetyLeadTime** field be set to "N". This ensures order point calculations are performed as expected.

Handling spikes in demand

When using order point logic, it is possible that significant spikes in demand might consume all supply and drive the inventory balance below zero before additional supply can be planned or obtained. To protect against this, RapidResponse can be configured to perform calculations that temporarily increase order point levels when such a spike in demand is detected. As well, if there are expectations or

knowledge of demand spikes that are not reflected in the demand data, these values can be imported into the **TimePhasedSafety** table and used by the order point calculations.

In order to apply temporary increases to order point levels, the **PlanningCalendars.OrderPointIncrease** and **SafetyStockPolicy.OrderPointIncrease** fields are used. The **PlanningCalendars.OrderPointIncrease** field specifies a calendar whose intervals are used, starting from the RunDate, to look for demand within a part's lead time. For each of these intervals where there is either demand within lead time or an entry in the **TimePhased** table, then order point calculations may be increased as specified by the setting in **Part.SafetyStockPolicy.OrderPointIncrease**. This field has the following values:

- **Calculate**— If the demand within lead time, plus any additional demand found for the within the **OrderPointIncrease** interval, exceeds the effective order point, then the amount it exceeds it by is temporarily added to the part's order point. The increase is effective until the next **OrderPointIncrease** interval.
- **Ignore**—no increase is made to the order point calculation. This is the default.
- **Sum**— If the demand within lead time, plus any additional demand found within the **OrderPointIncrease** interval, exceeds the effective order point, then the amount it exceeds it by is temporarily added to the order point, and if an effective record is found in the **TimePhased Safety** table, then its quantity is also added to the effective order point. The total increase is effective until the next **OrderPointIncrease** interval.
- **SumOffset—**identical to Sum, except that calculations to increase the order point threshold do not start until the second **OrderPointIncrease** interval after **Part.PlanningCalendars.RunDate.FirstDate**.

Scheduled supply usage in order point calculations

When calculating the initial inventory level for an order point part, the date when scheduled supply (scheduled receipts, supply allocations, and inventory transfers) should be included in the inventory balance can be set through the **SafetyStockPolicy.OrderPointDateRule** field. This field has the following values:

- **Due**—scheduled supply is considered as of its ReschedDate (calculated due date). As a result, supply that is already started, but not yet due, is ignored when calculating the inventory balance. Planned orders might potentially be created (unnecessarily) to cover this timing gap.
- **Start**—Scheduled supply is considered as of its CalcStartDate (calculated start date). As a result, the order point threshold will not be crossed if a sufficient quantity is available in a started, but not yet due, scheduled supply.

SafetyStockDateRule and order point calculations

Order point parts will respect the setting in the **SafetyStockPolicy.SafetyStockDateRule** field. Although all settings in this field are supported, order point parts should typically use either of the following:

- **Past**— safety stock is always maintained even if it means placing a past due order. This setting can be used to help flag and plan for the very early inventory shortages that are otherwise not handled by order point calculations.

- **RunDate**— safety stock is maintained only after **Part.PlanningCalendars.RunDate.FirstDate.Value**. This is the most typical setting for order point parts.

Notes:

- If **SafetyStockPolicy.SafetyStockQuantityRule** is set to either "PercentOfDemand" or "FractionOfDemand", then safety stock is never maintained before RunDate regardless of the setting in the **SafetyStockDateRule** field.
- For more information about **SafetyStockPolicy.SafetyStockDateRule**, see "SafetyStockDateRule" on page 880. Note that none of the "FirstDemand" settings are recommended for use with order point parts as this will always result in the first planned orders being late for the first demands by a period equal to the part's lead time. Similarly, the "LeadTime" setting is not recommended for order point parts as it can result in inventory shortages equivalent to a period of twice the lead time for a given part.

Limitations on other analytics

When using order point parts, limitations are placed on the analytic calculations outlined in the following table.

Table 14.10: Order point limitations on other analytics

Analytic / Calculation	Impact of order point
Constraint Analysis	Constraint analysis calculations are ignored/disabled for order point parts.
Days Of Supply	Days of supply calculations are ignored/disabled for order point parts.
Feature BOM	Order point logic is always disabled for configurable end items.
Kanban	Kanban analytic calculations are ignored/disabled for order point parts.
TaktTime	TaktTime calculations are ignored/disabled for order point parts.
Inventory Pooling	Inventory pooling logic should generally be avoided on order points parts. In particular, if there are part source pools or time-phased safety stock being used, the expected results might not be returned.

Examples

This section shows examples of order point calculations resulting from using the following SafetyStockRule and SafetyStockQuantityRule combinations.

- "OrderPointUnder and FixedQuantity" on page 1355
- "OrderPointUnder and PercentOfDemand " on page 1356
- "OrderPointAverage and TimePhasedSafety" on page 1356
- "OrderPointOver and FixedQuantity" on page 1357

These examples assume the following part and part-related values (unless noted otherwise):

- **Part.AverageQty**— 1000
- **Part.SafetyStockQty**— 2000
- **PartSource.MultipleQty**— 50
- **PartSource.EffLeadTime**— 2
- **PlanningCalendars.RunDate**— 06-02-08
- **OnHand.Quantity**— 2200

As well, all examples assume the following set of demands for the part.

Table 14.11: Sample demands

Order	Due Date	Quantity
1	06-11-08	770
2	06-11-08	1300
3	06-18-08	830
4	07-07-08	740

OrderPointUnder and FixedQuantity

"Planned orders and inventory balance (OrderPointUnder and FixedQuantity)" on page 1355 shows the planned orders and inventory levels resulting from setting **SafetyStockPolicy.SafetyStockRule** to "OrderPointUnder" and **SafetyStockPolicy.SafetyStockQuantityRule** to "FixedQuantity" (given the other part properties and set of demands shown above).

The first planned order is generated with a start date of 06-11-08 due to the two demands on that date which drives inventory levels down below the order point threshold (2000) to 130 units. The order is generated for 1900 units which is the smallest number that gets inventory back to or above the **SafetyStockQty** of 2000 while still respecting the **MultipleQty** of 50. The second planned order is generated in response to the demand on 06-18-08 taking inventory level below the order point threshold to 1200 units. In this case, RapidResponse is able to generate an order for 800 units which takes inventory exactly to the **SafetyStockQty**. Finally, the third planned order is generated when the demand on 07-07-08 takes inventory below the order point threshold to 1260 units. The order quantity of 750 units is the smallest number that gets inventory back to or above the **SafetyStockQty** of 2000 while still respecting the **MultipleQty** of 50, and the inventory balance ends up at 2010 units.

Table 14.12: Planned orders and inventory balance (OrderPointUnder and FixedQuantity)

Line	Start Date	Due Date	Quantity	Inventory Balance	Date
				2200	06-02-08
				130	06-11-08
1	06-11-08	06-13-08	1900	2030	06-13-08
				1200	06-18-08
2	06-18-08	06-20-08	800	2000	06-20-08
				1260	07-07-08
3	07-07-08	07-09-08	750	2010	08-09-08

OrderPointUnder and PercentOfDemand

"Planned orders and inventory balance (OrderPointUnder and PercentOfDemand)" on page 1356 shows the planned orders and inventory levels resulting from setting **SafetyStockPolicy.SafetyStockRule** to "OrderPointOver" and **SafetyStockPolicy.SafetyStockQuantityRule** to "PercentOfDemand" (given the other part properties and set of demands shown under "Examples" on page 1354). In addition, this example assumes the following part values that determine how the percent of demand calculations are performed:

- **Part.SafetyStockPolicy.PercentSafetyInterval**— Month
- **Part.PercentSafetyIntervalCount—** 1
- **Part.PercentSafetyPercent**— 40

Based on these settings, the order point threshold for the month of June is calculated as 1160 (.4 multiplied by the total monthly demand in June of 2900). The first planned order is created when the demands on 06-11-08 drive inventory levels down below this threshold to 130 units. The order is created for 1050 units which is the smallest number that gets inventory above the calculated order point threshold (1160 units) while still respecting the MultipleQty of 50. Similarly, the next planned order is created when the demand on 06-18-08 drives inventory levels down below the order point threshold to 350 units. One planned order is created for 850 units which is the smallest order size that can get inventory back above the order point threshold (1160) while still respecting the multiple quantity (50).

The order point threshold for the month of July is calculated as 296 units (.4 multiplied by the total monthly demand in July of 740 units). Because the inventory balance does not fall below this level, no planned orders are created in July.

Table 14.13: Planned orders and inventory balance (OrderPointUnder and PercentOfDemand)

Line	Start Date	Due Date	Quantity	Inventory Balance	Date
				2200	06-02-08
				130	06-11-08
1	06-11-08	06-13-08	1050	1180	06-13-08
				350	06-18-08
2	06-18-08	06-20-08	850	1200	06-20-08
				460	07-07-08

OrderPointAverage and TimePhasedSafety

"Planned orders and inventory balance (OrderPointAverage and TimePhasedSafety)" on page 1357 shows the planned orders and inventory levels resulting from setting **SafetyStockPolicy.SafetyStockRule** to "OrderPointAverage" and **SafetyStockPolicy.SafetyStockQuantityRule** to "TimePhasedSafety" (given the other part properties and set of demands shown under "Examples" on page 1354). In addition, this example assumes the following pair of records in the **TimePhasedSafety** table for our part.

- **TimePhasedSafety.Date—** 06-01-08 **TimePhasedSafetyQuantity**— 2000
- **TimePhasedSafety.Date**— 07-01-08 **TimePhasedSafetyQuantity**— 2500

The first planned orders are generated when the demands on 06-11-08 drive inventory levels down below the order point threshold for June (2000) to 130 units. In order to get inventory back above the **SafetyStockQty** level of 2000, two separate planned orders are created for the **AverageQty** of 1000 units each. The next planned order for 1000 units is created on 06-18-08 when the demand on that date for 830 units drives inventory below the order point threshold for June. Finally, the last planned for 1000 units is created on 07-01-08 when the new time phased safety quantity of 2500 comes into effect. Note that no planned orders are created in response to the demand on 07-07-08 as the inventory balance stays above the July order point threshold of 2500.

Table 14.14: Planned orders and inventory balance (OrderPointAverage and TimePhasedSafety)

Line	Start Date	Due Date	Quantity	Inventory Balance	Date
				2200	06-02-08
				130	06-11-08
1	06-11-08	06-13-08	1000	1130	06-13-08
2	06-11-08	06-13-08	1000	2130	06-13-08
				1300	06-18-08
3	06-18-08	06-20-08	1000	2300	06-20-08
4	07-01-08	07-03-08	1000	3300	07-03-08
				2560	07-07-08

OrderPointOver and FixedQuantity

"Planned orders and inventory balance (OrderPointOver and FixedQuantity)" on page 1358 shows planned orders and inventory levels resulting from setting **SafetyStockPolicy.SafetyStockRule** to "OrderPointOver" and **SafetyStockPolicy.SafetyStockQuantityRule** to "Fixed". This example assumes the same set of demands and other part properties as shown under "Examples" on page 1354, except for the following:

- **Part.SafetyStockQty**—1800
- **Part.AverageQty—**2000

The first planned order is generated with a start date of 06-11-08 due to the demands on that date which drives inventory levels down below the order point threshold (1800) to 130 units. The order is generated for 1850 which is the largest number that can keep inventory under **Part.AverageQty** (2000) while still respecting the **MultipleQty** of 50. The second planned order is generated on 06-18-08 due to the demand on that date taking inventory below the order point threshold (1800) , and a planned order for 850 units is generated which takes inventory to exactly to 2000 units. Finally, the last planned order is created on 07-07-08 due to inventory falling below the order point threshold, and the order is for 700 units which is the largest possible order size that keeps inventory under **Part.AverageQty** (2000 units) while still respecting the **MultipleQty** of 50.

Table 14.15: Planned orders and inventory balance (OrderPointOver and FixedQuantity)

Line	Start Date	Due Date	Quantity	Inventory Balance	Date
				2200	06-02-08
				130	06-11-08
1	06-11-08	06-13-08	1850	1980	06-13-08
				1150	06-18-08
2	06-18-08	06-20-08	850	2000	06-20-08
				1260	07-07-08
3	07-07-08	07-09-08	700	1960	08-09-08

Inventory levels and planned orders

OnHand records that have a **OnHandType.ProcessingRule** control value of **Nettable** are accumulated to obtain the starting inventory level. Daily bucketed demands then reduce the inventory. If the inventory drops below zero, or the safety stock level, new supply is obtained by rescheduling scheduled receipts. When a receipt is rescheduled, all of its quantity (reduced by yield) is used to increase inventory on the new date.

When netting runs out of scheduled receipts, it generates new planned orders. A record is added to the **PlannedOrder** table for each planned order. For information about this table, see "PlannedOrder table" on page 1148.

Order Policy and planned orders

Order policy for a given part is controlled by the following factors:

- The **PartSource.MinimumQty**, **PartSource.MaximumQty**, and **PartSource.MultipleQty** fields are used to control order minimums (minimum lot) , maximums (maximum order size) and multiples. For more information, see "Planned order lot sizing" on page 1358.
- If using period ordering (for example, look ahead for a specified number of days when planning supply), the **Part.NumberOfDaysSupply** field specifies the number of days or calendar intervals to look ahead. As well, the **PartType.DaysSupplyRule** controls the date from which the number of days supply value is applied. For more information, see "Days of Supply calculations" on page 1362.
- If using period ordering and maximum order size policies for a part, set the **PartSource.OrderPolicy.MaximumUsage** control value to **Use**.

For information about how order policy data is stored, see "OrderPolicy table" on page 773.

Planned order lot sizing

Planned order sizing during netting calculations is controlled by the **OrderPolicy** field on the **PartSource** table. This field references the **OrderPolicy** control table, which controls how RapidResponse creates planned orders.

The **PlanningCalendars** field of a **Part** record controls rounding of the DueDate. For example, some types of parts may only be shipped once a week on a particular shipping schedule.

Min, Max, and Multiple Quantities

One aspect of planned orders that the **OrderPolicy** control table defines is how the following three fields on the **PartSource** table are used:

- **MaximumQty**—The maximum lot size RapidResponse uses when planning supplies. If satisfying a demand larger than the maximum lot size, multiple orders are required.
  Use of this field is controlled by **OrderPolicy.MaximumUsage** (see "MaximumUsage" on page 775).
- **MinimumQty**—The minimum lot size RapidResponse uses when planning supplies. If satisfying a demand smaller than the minimum lot size, the order size is adjusted up.
  Use of this field is controlled by **OrderPolicy.MinimumUsage** (see "MinimumUsage" on page 776).
- **MultipleQty**—The target lot size RapidResponse uses when planning supplies. That is, order quantities should be set to a multiple of the value in this field.
  Use of this field is controlled by **OrderPolicy.MultipleUsage** (see "MultipleUsage" on page 777).

Caution: Use of the **MaximumQty** setting typically results in more planned orders than would otherwise be generated. If the total demand quantity is large and the **MaximumQty** is set to a small number, then this can result in very large numbers of planned orders being generated which can potentially impact system performance. Note also that a global setting can be used to place a cap on the number of planned orders generated per part each day as discussed in "Limiting the number of planned orders" on page 2204.

Of the three **PartSource** fields that control order size, RapidResponse gives the highest priority or precedence to the **MaximumQty**, followed by **MinimumQty**, and finally **MultipleQty**. That is, when calculating order size, RapidResponse first checks if the required demand quantity is less than the **MinimumQty**. If it is, then the order size is adjusted upward; otherwise the **MultipleQty** is applied. Finally, it checks to see if the order size exceeds the **MaximumQty** and adjusts the order size downward if it does.

The following table shows examples of how planned order size is calculated to satisfy different quantities of unsatisfied demand. Each example assumes the following order policy rules:

- **MinimumQty**—5
- **MinimumUsage**—Use
- **MultipleQty**—3
- **MultipleUsage**—Use
- **MaximumQty**—11
- **MaximumUsage**—Use

Table 14.16: Calculating order sizes

Unsatisfied demand quantity	Calculations and resulting planned order quantity
4	• The quantity needed is less than **MinimumQty**, so order size is adjusted up to 5. • **MultipleQty** is ignored. • The order size of 5 is less than the **MaximumQty** so no adjustment is required. **Result**— Planned order for 5 units.
7	• The quantity needed is greater than **MinimumQty**, so no adjustment is required. • **MultipleQty** is applied, so order size becomes 9 (rounded up to a multiple of 3). • The order of size of 9 is less than the MaximumQty, so no adjustment is required. **Result**—Planned order for 9 units.
10	• The quantity needed is greater than **MinimumQty**, so no adjustment is required. • **MultipleQty** is applied, so order size become 12 (rounded up to a multiple of3). • The order size of 12 is greater than **MaximumQty**, so order size is adjusted down to 11. **Result**—Planned order for 11 units

Notes:

- Depending on your **MinimumQty** and **MaximumQty** values, it is possible to end up with order sizes that do not respect the **MultipleQty** (as shown in the first and last examples above). Therefore, it's recommended that your **MinimumQty** and **MaximumQty** values be set to multiples of the **MultipleQty**.
- If using both order multiples and maximums, then **MultipleQty** should typically be set to a value less than **MaximumQty**. Otherwise, if **MultipleQty** is greater than **MaximumQty**, orders will be generated in excess of **MaximumQty** (that is, the order multiple target is also treated as the order maximum).

Order averaging using MaximumQty

When maximum and minimum order policies are used in generating planned orders, more supply than is needed to satisfy a given requirement may end up being created. For example, suppose an unsatisfied demand for 42 units on a given date for a part with the following order policies in effect:

- **MaximumQty**—20
- **MinimumQty**—10

In this situation, RapidResponse tries to create a planned order satisfying the 42 unit requirement, but is restricted by the MaximumQty and so creates an order for 20 units. RapidResponse then tries to create a planned order for the remaining 22 units, but is again restricted by the MaximumQty and so

creates another order for 20 units. Finally, RapidResponse tries to satisfy the remaining requirement for 2 units, but in respecting the MinimumQty it creates an order for 10 units instead. The net result is three planned orders created (of 20, 20, and 1o units) for a total of 50 units. This is 8 more units than are needed to satisfy the immediate requirement, but RapidResponse can use these units to satisfy later demand.

However, if you want to reduce this type of early supply planning, the "Average" option on **OrderPolicy.MaximumUsage** can be used to just cover the actual requirement while still respecting any Max, Min, and Mult order policies. In cases where the requirement for a part exceeds the MaximiumQty for the part source, this option divides the total requirement by the MaximumQty to determine the number of orders needed (rounding up, if necessary). It then averages the total requirement over this number of orders to determine the planned order size. For example, assume again an unsatisfied demand for 42 units on a given date for a part, using a source with OrderPolicy.MaximumUsage set to "Average", and the following order policies still in effect:

- **MaximumQty**—20
- **MinimumQty**—10

RapidResponse first divides the requirement for 42 units by the MaximumQty of 20 which returns 2.05. This value is then rounded up to 3 which becomes the number of planned orders to create. The total requirement of 42 units is then spread evenly across these 3 orders for an order size of 14 units each, with no excess supply created.

Note: The "Average" option on **OrderPolicy.MaximumUsage** can only be used if a given requirement has exactly one part source that can satisfy it, and the value of **PartSource.TaktTime** for that source is set to 0. If these conditions are not met, then "Average" is equivalent to the "Use" option. For more information on the **OrderPolicy.MaximumUsage** field, see "MaximumUsage" on page 775.

Quantity ordered

The recommended number of items that should be ordered is stored in the **Quantity** field in the **PlannedOrder** table. The value that is stored in the **NeedQty** field differs from the value in the **Quantity** field because the last planned order of an item will include more items than needed to cover remaining demand.

You can set the **LotSizeLastOrder** field in the **OrderPolicy** table to **N**, which results in lot sizing not being applied to the last order. In this situation, there is no difference between the **NeedQty** and **Quantity** fields on a **PlannedOrder** record.

Lot sizing

The following table shows the role the following PartSource fields play when RapidResponse generates a lot size value for planned orders: **MaximumQty**, **MinimumQty**, and **MultipleQty**.

Table 14.17: Lot size calculations

PartSource field	Description
MultipleQty	If the **OrderPolicy.MultipleUsage** field= Ignore then **PartSource.MultipleQty** = 0. If the **OrderPolicy.MultipleUsage** field= Units then **PartSource.MultipleQty** = 1. If the **OrderPolicy.MultipleUsage** field= Use then **PartSource.MultipleQty** = Max(0, PartSource.MultipleQty()).
MinimumQty	If the **OrderPolicy.MinimumUsage** field= Ignore then **PartSource.MinimumQty** = MinimumQty. If the **OrderPolicy.MinimumUsage** field= RoundMult then **PartSource.MinimumQty** = Multiple (PartSource.MinimumQty(), MultQty). If the **OrderPolicy.MinimumUsage** field= Use then **PartSourceMaximum.MinimumQty** = Max(0, PartSource.MinimumQty()).
MaximumQty	If the **OrderPolicy.MaximumUsage** field= Ignore then **PartSource.MaximumQty** = 0. If the **OrderPolicy.MaximumUsage** field= Use or OverRideDaysSupply then **PartSource.MaximumQty** = Max(0, PartSource.MaximumQty()).

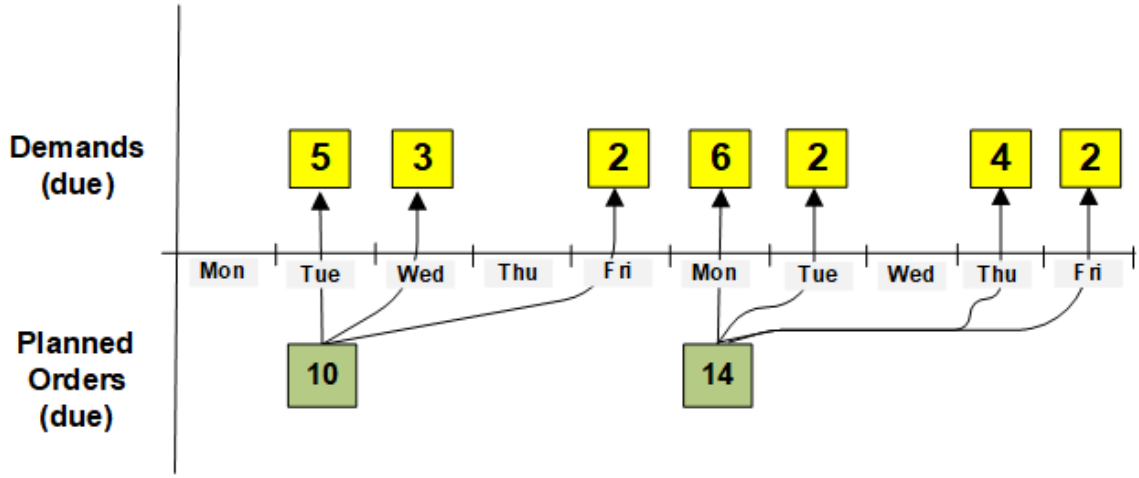
Note: For information about the OrderPolicy table, see "OrderPolicy table" on page 773.

Days of Supply calculations

Days of Supply logic can reduce the number of planned orders created. When using days of supply, if an unsatisfied demand is encountered, RapidResponse accumulates demand for the part over some number of periods and then tries to satisfy that accumulated demand with a single planned order (unless **PartSource.MaximumQty** would be exceeded).

For example, all demands due within each week might be accumulated and a single planned order created to satisfy those demands as shown in the following image.

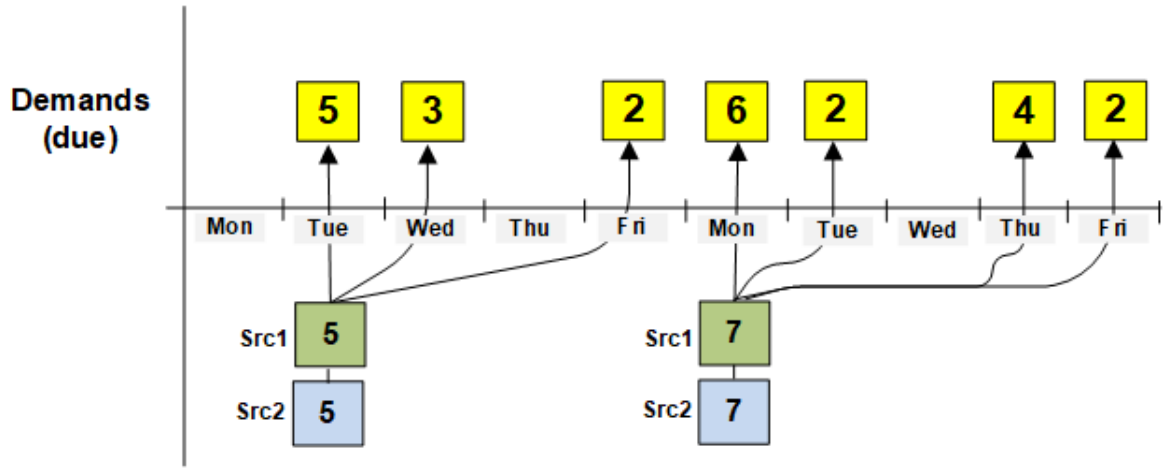
Figure 14.12 Planned orders generated with days of supply (single source)



Note that if a part has more than one effective part source, then each source can potentially plan supply in a given period resulting in multiple planned orders per part. For example, assume two equivalent part sources with the same priority and target for a part with **SourceRule.AllotmentRule** set to

"Proportional". In this case, if the demands were accumulated weekly then planned orders might be generated as shown in the following image.

Figure 14.13 Planned orders generated with days of supply (multi-source)



Enabling days of supply

Days of supply logic can be applied to a part in one of two different methods.

One method is by defining all days of supply settings in the **DaysSupplyPolicy** control table, and then referencing a record in that table from the **Part.DaysSupplyPolicy** field to apply those settings to the part. With this method, a non-Null reference in this field indicates that days of supply logic should be enabled for the part (unless the **DaysSupplyPolicy.DaysSupplyRule** field is set to "Ignore"). Note also that the **DaysSupplyPolicy** table allows for two time-phased sets of days of supply parameters; one which applies in a defined short-term horizon and the other which applies in a defined long-term horizon. The **Fence** and **FenceCalendar** fields on the **DaysSupplyPolicy** table are applied from RunDate and together determine the date where the short-term horizon ends and the long-term horizon begins.

The second method is by defining days of supply settings in a number of different fields across the **Part**, **PartType**, and **PlanningCalendars** tables. With this method, the part must reference a **PartType.UseDaysSupply** value of "Y" in order for days of supply logic to be enabled for the part. Note that this method only allows a single set of days of supply parameters which then apply across the entire planning horizon (no time-phased settings). This corresponds to how days of supply calculations were always performed in Version prior to 2016.2 (legacy method).

The following table outlines the main fields that should be configured depending on which method is used to configure days of supply logic. Note that many settings in the **DaysSupplyPolicy** table do not have a corresponding option if using the part-based legacy method.

Table 14.18: Fields for configuring days of supply logic

DaysSupplyPolicy configuration settings	Part-based legacy settings
Part.DaysSupplyPolicy	PartType.UseDaysSupply
DaysSupplyPolicy.Fence	

Table 14.18: Fields for configuring days of supply logic

DaysSupplyPolicy configuration settings	Part-based legacy settings
DaysSupplyPolicy.FenceCalendar	
DaysSupplyPolicy.DaysSupplyRule	PartType.DaysSupplyRule
DaysSupplyPolicy.LongTermDaysSupplyRule	
DaysSupplyPolicy.NumberDaysSupply	Part.NumberDaysSupply
DaysSupplyPolicy.LongTermNumberDaysSupply	
DaysSupplyPolicy.PeriodSupplyInterval	PlanningCalendars.PeriodSupplyInterval
DaysSupplyPolicy.LongTermPeriodSupplyInterval	
DaysSupplyPolicy.PeriodSupplyDueInterval	PlanningCalendars.PeriodSupplyDueInterval
DaysSupplyPolicy.LongTermPeriodSupplyDueInterval	
DaysSupplyPolicy.PeriodSupplyDateCalendar	
DaysSupplyPolicy.LongTermPeriodSupplyDateCalendar	
PlanningCalendars.TimeUnits	PlanningCalendars.TimeUnits

Days of supply rules

With either method used to apply days of supply logic, the specific way in which days of supply calculations are performed is further dependent on the **DaysSupplyRule** setting which has the following options (the remainder of this section discusses each of these options in more detail):

- **FromDemand**
- **FromSupply**
- **ByPeriod**
- **ByPeriodWithPriority**
- **ByPeriodEnd**
- **ByPeriodEndWithPriority**
- **Ignore** (disables days supply logic entirely)

Note: If planned order priorities should be based on priorities of the demands being satisfied, either the **ByPeriodWithPriority** or **ByPeriodEndWithPriority** option should be used, or the **PartType.DaysSupplyPrioritySource** field should be set to a value other than "Default". For more information, see "Days of supply and planned order priority (alternate configuration)" on page 1371.

FromDemand and FromSupply

The FromDemand and FromSupply options are similar except that FromDemand begins accumulating demands from the due date of the first unsatisfied demand and FromSupply begins accumulating demand from the due date when the supply can be planned. Both of these options accumulate demand

for a given number of time units (such as, work days), and then create supply to satisfy this accumulated demand. The time unit used for this accumulation is set by the calendar referenced in **Part.PlanningCalendars.TimeUnits**, and the number of these time units considered is set by the value in **Part.NumberofDaysSupply**.

The following table shows examples of the planned orders created by Days of Supply logic to satisfy given demands for a part under the following circumstances:

- **Part.Type.UseDaysSupply** is 'Y'
- **Part.Type.DaysSupplyRule** is 'FromDemand'
- **Part.PlanningCalendars.TimeUnits** is set to a work day calendar
- **Part.NumberOfDaysSupply** is set to '5'
- The **MaximumQty** is set to '250'

Starting from the demand on Tuesday (Week 1), all demands are accumulated over the 5 work days ending on the Monday in Week 2. A planned order is then created on Tuesday (Week 1) to satisfy this accumulated demand for 250 units. The next planned order is then created to satisfy the demand for 50 units on Wednesday (Week 2).

Table 14.19: Days Supply - 'From Demand' example

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand		125		50		75		50		
Planned Orders		250						50		

In the example shown in the previous table, the supply needed to satisfy the accumulated demands did not exceed **PartSource.MaximumQty**. In cases where accumulated demand does exceed maximum quantity then, on any given day, only as many planned orders at the **MaximumQty** as are necessary to satisfy the demand on that day are created. The days of supply horizon then resets when it next encounters unsatisfied demand, creates the required planned order(s) to satisfy demand on that day, and the process continues.

The following table shows how the results in the previous example would differ if the **MaximumQty** were set to '50'. Note that instead of creating 5 orders for 50 units on Tuesday (to satisfy the accumulated demand of 250 units), it instead creates only 3 orders for 50 units (to satisfy the demand on Tuesday). When unsatisfied demand is next encountered on Thursday (for 25 units) , the days of supply horizon is reset starting from this day and accumulates demand over the next 5 days (in this case 150 units) but only as many planned orders at 50 units (in this case 1) as are needed to satisfy Thursday's demand are created.

Table 14.20: Days Supply - 'From Demand' example

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand		125		50		75		50		
Planned Orders (if MaximumQty = 250)		250						50		
Planned Orders (if MaximumQty = 50)		50 50 50		50		50		50		

ByPeriod

Days of Supply logic can also be set to use the ByPeriod option. With this setting, instead of accumulating demand over a given number of Part.PlanningCalendars.TimeUnits, days of supply logic accumulates demand over a given number of intervals in a specified calendar. For example, demands might be accumulated on a weekly or monthly basis.

The calendar whose intervals are used for this accumulation is set by the calendar referenced in **Part.PlanningCalendars.PeriodSupplyInterval**, and the number of these intervals that define each accumulation period is set by the value in **Part.NumberOfDaysSupply**. As well, the due date for the planned order(s) created to satisfy this accumulated demand must come from the calendar referenced by **Part.PlanningCalendars.PeriodSupplyDueCalendar**. For example, this could be set up to ensure that planned orders are due on the first date in the accumulation period or on the first unsatisfied demand date in the accumulation period.

The following table shows the planned orders created to satisfy the same demands given in the previous example when the following settings are used instead:

- **Part.Type.UseDaysSupply** is 'Y'
- **Part.Type.DaysSupplyRule** is 'ByPeriod'
- **Part.PlanningCalendars.PeriodSuplyInterval** is Weekly
- **Part.NumberofDaysSupply** is '1'
- **Part.PlanningCalendars.PeriodSupplyDueCalendar** is Weekly

Assuming the Weekly calendar markers are set to Mondays, demands are accumulated from the date of the first demand in the week up to but not including the following Monday. The required planned order (s) are due on the Monday of each week.

Table 14.21: Days Supply - 'By Period' example

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand		100		100		50		50		
Planned Orders (MaximumQty = 250)	200					100				
Planned Orders (MaximumQty = 50)	50 50 50 50					50 50				

ByPeriodWithPriority

The ByPeriodWithPriority option is similar to ByPeriod in that it accumulates demand over a given number of intervals (**Part.NumberOfDaysSupply**) for a defined calendar (**Part.PlanningCalendars.PeriodSupplyInterval**). For a detailed example of how this logic works, see "ByPeriod" on page 1366.

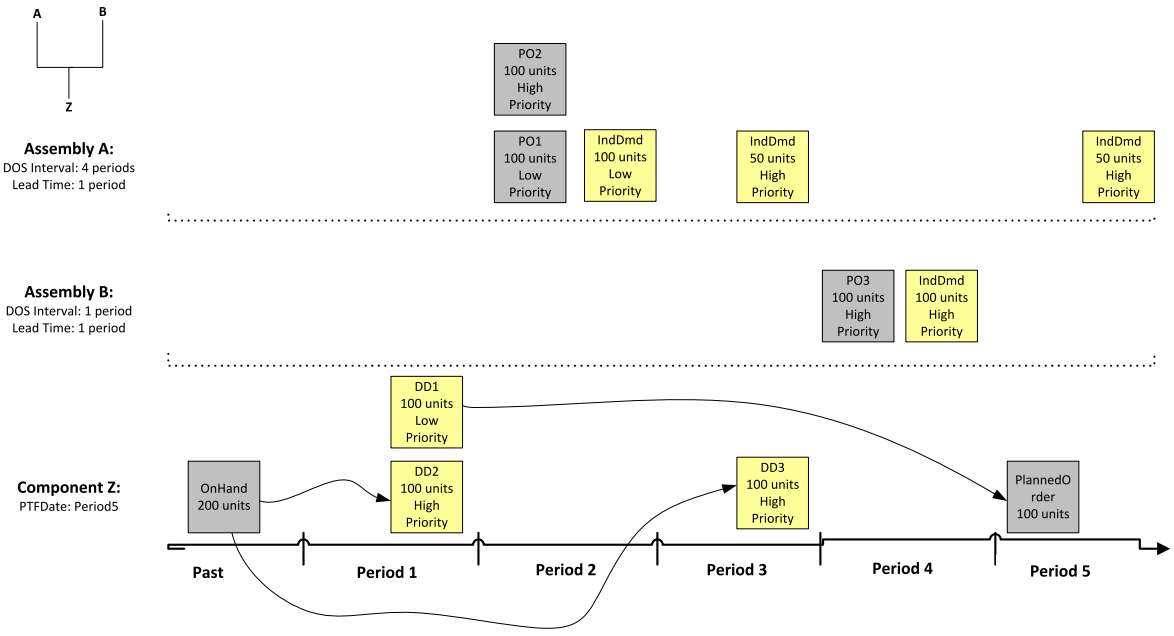
However, ByPeriodWithPriority differs from ByPeriod in that if a part's **PartType.CommitLevel** setting is "High", it will try to respect order priorities within its days of supply calculations. It does this by breaking the quantity accumulated in a given period of supply down into smaller quantities grouped by priority level. These quantities and priorities are then used when generating planned orders, and the priorities on the planned orders are subsequently passed down to their dependent demands.

To see the potential impact of considering priorities with days of supply calculations, consider the example shown in "ByPeriodWithPriority when CommitLevel = 'High'" on page 1367. In this simple example, both Assembly A and Assembly B use Component Z, and the following is assumed:

- **Assembly A**— has CommitLevel set to "High". Therefore, when it accumulates unsatisfied independent demands starting in period 2, it groups them by their priority levels (in this example, "low" and "high"). These groups of priorities are then used when creating the planned orders for Assembly A (in this example, a "high" priority order for 100 units, and a "low" priority order for 100 units).
- **Assembly B**—has CommitLevel set to "High", and only one unsatisfied independent demand. Therefore, the one planned order created for Assembly B takes its priority from that demand (in this example, a "high" priority order for 100 units).
- **Component Z**—has three dependent demands generated from the planned orders on Assembly A and Assembly B, each of which inherit the priority from their related planned order. These priorities are then considered when filling the dependent demand orders. Specifically, the existing on hand inventory is used to satisfy the two high priority dependent demands. Then a new planned order is created, in Period 5 due to the PTFDate, to satisfy the remaining low priority dependent demand.

As a result, the two high priority independent demands for Assembly A and the one high priority independent demand for Assembly B are satisfied on time, while the one low priority independent demand for Assembly A ends up being made late (due to availability of Component Z).

Figure 14.14 ByPeriodWithPriority when CommitLevel = 'High'



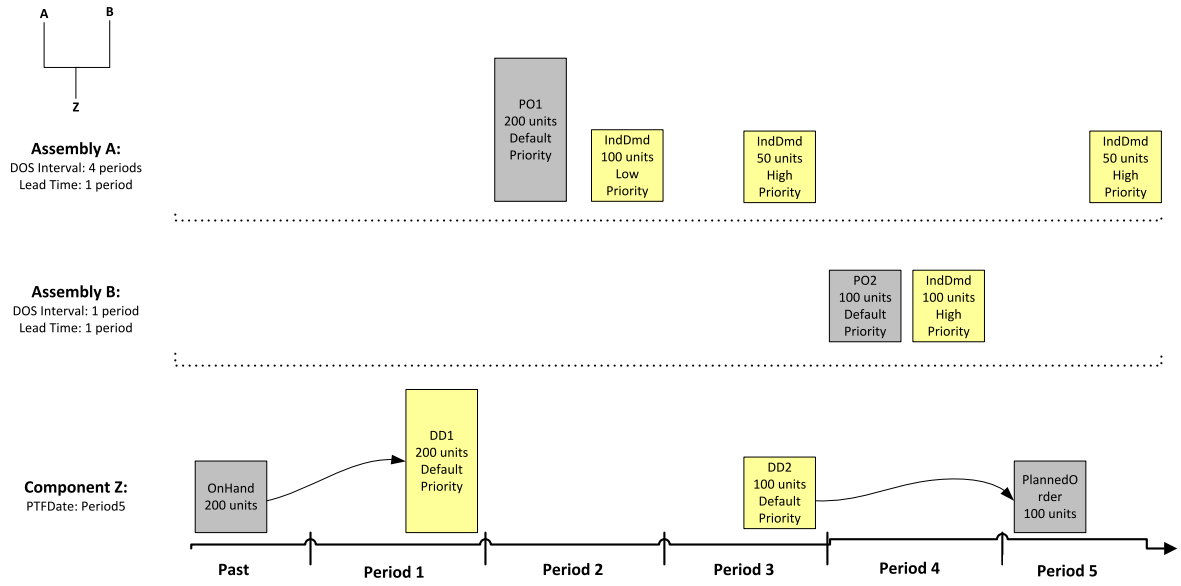
Note that if CommitLevel is set to either "Medium", "MediumExplodePriority" or "Low" then the ByPeriodWithPriority setting does not consider priority and instead assigns a default priority to the planned orders. To see the potential impact of not considering priorities with days of supply calculations, consider the example shown in "ByPeriodWithPriority and CommitLevel = 'Medium' or 'Low'" on page 1368.

In this simple example, both Assembly A and Assembly B use Component Z, and the following is assumed:

- **Assembly A**—has CommitLevel set to “Medium”. Therefore, all unsatisfied independent demands accumulated starting from period 2 are grouped together with a default priority. This default priority is assigned to the resulting planned order for all 200 units.
- **Assembly B**—has CommitLevel set to “Low” and only one unsatisfied demand. Therefore, a default priority is assigned to the resulting planned order for 100 units.
- **Component Z**—has two dependent demands generated from the planned orders on Assembly A and Assembly B, each of which inherit the default priority from their planned order. Therefore, priorities are not considered when filling these dependent demands which are instead satisfied in due date order. Specifically, the existing on hand inventory of 200 units is used to satisfy all of the first dependent demand. Then a new planned order is created, in Period 5 due to the PTFDate, to satisfy the other dependent demand.

As a result, all independent demands for Assembly A (both high and low priority) are satisfied on time, while the high priority independent demand for Assembly B is made late (due to availability of Component B). This contrasts with the example in "ByPeriodWithPriority when CommitLevel = 'High'" on page 1367 where earlier low priority demands were made late to ensure that later high priority demands could be satisfied on time.

Figure 14.15 ByPeriodWithPriority and CommitLevel = 'Medium' or 'Low'



Note: Using ByPeriodWithPriority with CommitLevel set to either “Medium” or “Low” is equivalent to using the ByPeriod setting.

ByPeriodEnd and ByPeriodEndWithPriority

The **ByPeriodEnd** and **ByPeriodEndWithPriority** options are similar to the **ByPeriod** and **ByPeriodWithPriority** options discussed earlier in this section. That is, they each collect demands over a specified number of **PeriodSupplyInterval** calendar intervals and satisfy them with a planned order due on the **PeriodSupplyDueInterval** calendar.

The difference is that **ByPeriodEnd** and **ByPeriodEndWithPriority** options are meant to plan supply to be due at the end of the period for which demand is being accumulated. That is, supply is planned on the last **PeriodSupplyDueCalendar** before the end of the accumulation period. With these options, the planned order generated can be expected to be late to satisfy some or all of the demands collected in the period.

For example, assume the following settings

- **Part.Type.UseDaysSupply** is 'Y'
- **Part.Type.DaysSupplyRule** is 'ByPeriodEnd'
- **Part.PlanningCalendars.PeriodSuplyInterval** is Weekly
- **Part.NumberofDaysSupply** is '2'
- **Part.PlanningCalendars.PeriodSupplyDueCalendar** is Friday

For a given set of demands shown in the first row of the following table, planned orders would be generated as shown in the second row of the table.

Table 14.22: Days Supply - ByPeriodEnd example 1

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand	300	200	250	250	175	150	125	200	250	100
Planned Orders										2000

Instead, assume the same settings except that the **PeriodSupplyDueCalendar** was changed to “Week”. In this case, planned orders would be generated as shown in the following table.

Table 14.23: Days Supply - ByPeriodEnd example 2

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand	300	200	250	250	175	150	125	200	250	100
Planned Orders						2000				

Finally, assume the PeriodSupplyDueCalendar and PeriodSupplyInterval were both changed to “Workday”, and NumberOfDaysSupply was changed to “5”. In this case, planned orders to meet the demands would be as shown in the following table..

Table 14.24: Days Supply - ByPeriodEnd example 2

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand	300	200	250	250	175	150	125	200	250	100
Planned Orders					1175					825

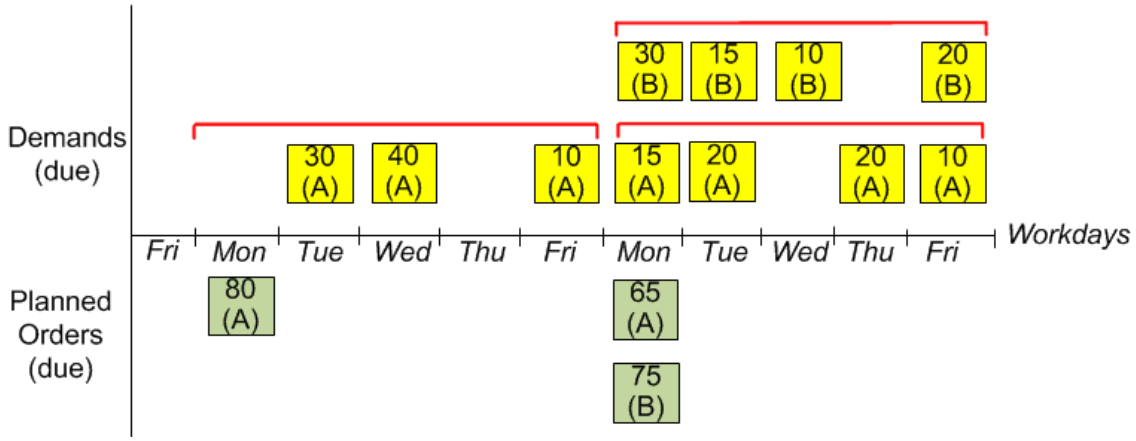
Dates used in period-based days of supply

When using any of the four period-based days of supply options (for example, ByPeriod), demands are collected over some number of **PeriodSupplyDateCalendar** intervals and the planned order generated to satisfy those collected demands must fall on an interval in the **PeriodSupplyDueCalendar**.

The **PeriodSupplyDateRule** and **LongTermPeriodSupplyDateRule** fields can be used to set which demand and supply dates are aligned to the specified period calendars. The default setting for each of these fields is "DueDate". With this option, demands are bucketed into intervals based on their due date, and planned order due dates are then placed on **PeriodSupplyDueCalendar** (or **LongTermPeriodSupplyCalendar**) intervals.

For example, assume two parts A and B each use a **DaysSupplyRule** of "ByPeriod", have their **PeriodSupplyDateCalendar** and **PeriodSupplyDueCalendar** set to "Week", and **NumberOfDaysSupply** set to "1". In this case, all of a part's demands due in a given week are collected together and the planned order created to satisfy them is due at the start of the week (on the "Week" calendar) as shown in the following image.

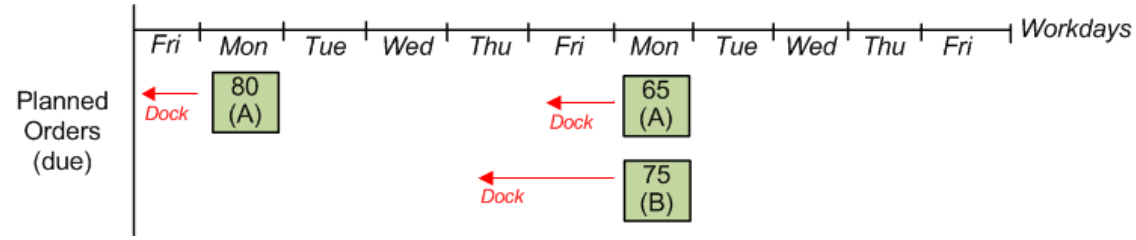
Figure 14.16 Planned orders with PeriodSupplyDateRule set to DueDate



The **PeriodSupplyDateRule** and **LongTermPeriodSupplyDateRule** fields also have a "DockDate" option. With this setting, demands are bucketed into intervals after considering dock-to-stock lead time for the part, and supply dock dates (rather than due dates) are then aligned to the **PeriodSupplyDueCalendar**. The "DockDate" setting can be useful for cases where different parts have different dock-to-stock lead times and there is a need to have supplies of those different parts arrive at the destination site on the same **DockDate** (and leave the sourcing site on the same **ShipDate)**.

For example, in the "DueDate" case shown in the preceding image, if **DockToStockLeadTime** was 1 day for part A and 2 days for part B, then supplies for these parts would arrive on different dock dates even though their due dates were aligned as shown below.

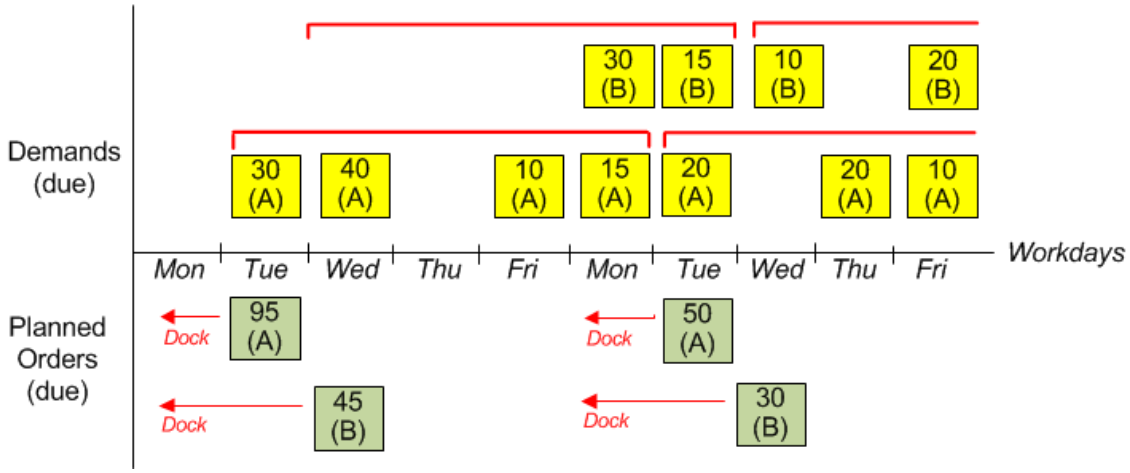
Figure 14.17 Dock dates for planned orders



Instead, assume this same example but with the **PeriodSupplyDateRule** set to "DockDate". In this case, the demand collection intervals are determined by offsetting the demand due dates by the applicable dock-to-stock lead times (1 day for A, and 2 days for B). The planned orders to satisfy these demands are then generated so that the DockDate for each falls at the start of the week (on the "Week" calendar) as shown in the following figure. Note that not only are the planned order due dates on different days

than they were with "DueDate" option, but the number and quantity of those planned orders has changed due to the shifting of the demand collection period.

Figure 14.18 Planned orders with PeriodSupplyDateRule set to DockDate

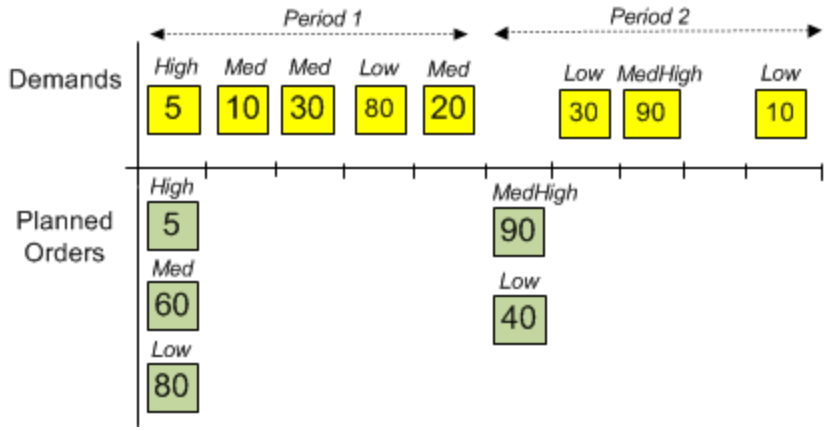


Note: The ability to specify date interpretation in period-based days of supply planning is only available if using the **DaysSupplyPolicy** table. If using the legacy part-based method for days of supply, then due dates are always aligned to period calendars (no option to use the dock date).

Days of supply and planned order priority (alternate configuration)

As discussed earlier in this previous section, the "ByPeriodWithPriority" and "ByPeriodEndWithPriority" settings can be used to ensure priorities on demands for parts subject to days of supply logic are passed on to the planned orders that satisfy them in Netting calculations. Subsequently, these priorities are propagated to the dependent demands generated from the planned orders. If there are demands of multiple priorities in a given days of supply period, then this setting will result in multiple planned orders in that period. For example, the following illustration shows a scenario where three planned orders are generated in Period 1 and two planned orders are generated in Period 2 to satisfy the different levels of prioritized demands found in each.

Figure 14.19 Planned orders from "ByPeriodWithPriority"

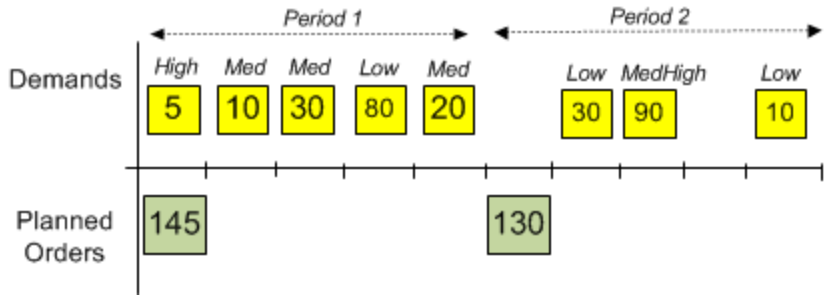


In cases where multiple potential planned orders per period are not desirable, then a **DaysSupplyRule** setting other than "ByPeriodWithPriority" should be selected. The **PartType.DaysSupplyPrioritySource** field is then used to determine how the part's planned orders have their priorities determined. The **DaysSupplyPrioritySource** field has the following options:

- **Average**—order priority is set to an average of the priorities found on demands satisfied by the order in Netting calculations. The average is determined based on the **OrderPriority**.**PlanningPriority** value for each demand over the total number of demands satisfied (closest **Value** less than or equal to the average is then used).
- **Default**—order priority is set to the part's default priority value as set by the value referenced in **PartType.DefaultPriority**. This is the default setting and the only one which does not consider the priorities of the demands being satisfied by the planned order.
- **Highest**—order priority is set to the highest priority on any demand being satisfied by the order in Netting calculations. In other words, the **Value** associated with the lowest **OrderPriority.PlanningPriority** value found on a demand being satisfied is used.
- **WeightedAverage**—order priority is set to a weighted average of the priorities found on demands satisfied by the order in Netting calculations. This average is determined based on both the **OrderPriority**.**PlanningPriority** value and quantity of each demand over the total quantity of demands satisfied (closest **Value** less than or equal to the weighted average is then used).

For example, if the same set of demands as shown previously belonged to a part having a **PartType.DaysSupplyRule** setting of "ByPeriod", then only one planned order in each of Period 1 and Period 2 would need to be generated as shown in the following illustration.

Figure 14.20 Planned orders from "ByPeriod"



In this case the priorities assigned to the planned order in each period would be calculated based on the part's **PartType.DaysSupplyPrioritySource** setting. For example, suppose the priorities shown in the previous illustration correspond to **Value** and **PlanningPriority** values in the **OrderPriority** table as shown below, and further suppose the applicable **PartSource.DefaultPriority** field points to a value of "Med".

Table 14.25: OrderPriority table values

Value	PlanningPriority
High	0
MedHigh	5
Med	10
MedLow	15
Low	20

Based on the applicable **DaysSupplyPrioritySource** setting, the priority assigned to each of the planned orders would be calculated as shown in the following table:

Table 14.26: Priority calculations based on "DaysSupplyPrioritySource" setting

DaysSupply PrioritySource	Priority for order in Period 1	Priority for order in Period 2
Average	• Med Calculated as *= 0 + 10 + 10 + 20 + 10 / 5* *= 50 / 5* *= 10* Thus, the closest PlanningPriority value less than or equal to 10 is 10 which then equates to a priority value of Med.	• MedLow Calculated as: *= 20 + 5 + 20 / 3* *= 45 / 3* *= 15* Thus, the closest PlanningPriority value less than or equal to 15 is 15 which then equates to a priority Value of MedLow.
Default	• Med Set to the Value referenced from PartType.DefaultPriority.	• Med Set to PartType.DefaultPriority field.
Highest	• High Set to Value associated with the lowest PlanningPriority on a demand being satisfied in period 1.	• MedHigh Set to Value associated with the lowest PlanningPriority on a demand being satisfied in period 2.
WeightedAverage	• MedLow Calculated as *=(0*5)+(10*10)+(30*10)+(80*20)+(20*10) / 145* *=0+100+300+1600+200 / 145* *= 2200 / 145* *= 15.2* Thus, the closest PlanningPriority value less than or equal to 15.2 is 15 which equates to a priority Value of MedLow.	• MedHigh Calculated as: *= (30*20)+(90*5)+(10*20) / 130* *= 600+450+200 / 130* *= 1250 / 130* *= 9.6* Thus, the closest PlanningPriority value less than or equal to 9.6 is 5 which equates to a priority Value of MedHigh.

Limitations on other analytics

When using days of supply, certain other analytic calculations might be disabled, or not work as expected, for parts configured to use days of supply planning logic. The following table outlines analytic calculations impacted by days of supply configurations.

Table 14.27: Days of Supply limitations on other analytics

Analytic / Calculation	Impact of Days of Supply
Order point	Days of supply calculations are automatically disabled (ignored) for order point parts.
Global part substitution	When satisfying any days of supply requirements on a primary part in a global part substitute relationship, any required planned orders can only be created on the primary part.
Material expiry	Days of supply is generally compatible with material expiry logic. However, in some cases expiry requirements might result in multiple supplies being generated on different days within a single days of supply period. When days of supply logic is enabled for an expiring part and a planned order is required in a given period, RapidResponse collects all demands within that period and tries to plan a single supply to meet the latest earliest expiry date of those demands. However, if the order cannot be planned with an expiry date to meet the latest earliest expiry date of the demands, then only the portion of the supply whose estimated expiry date can be met is included in the order and subsequent planned order(s) are created for the remaining quantity in the period..
Constrained planning	When multi-level search is used in constrained planning environments that use either lot-sizing or days of supply logic, the following limitations exist: • **Days of supply**- planned order due dates might not always fall on **PeriodSupplyDueCalendar** intervals. If constraint availability causes a supply to be pre-planned, then the planned order due date is set to the last date in the constraint bucket where the pre-planning occurred (which won't necessarily belong to the **PeriodSupplyDueCalendar**). • **Fair share (and DoS)**—the fair sharing of constraints might result in Days of Supply logic being ignored. Because each planned order represents the fair share of constraint for the demand being satisfied, those planned orders can then not be merged together to satisfy Days of Supply requirements. • **Lot-sizing**—if **PartSourceType.ConstraintMultipleRule** is set to "MultiplePeriods", then the multiple quantity (**PartSource.MultipleQty**) might not always be respected on planned orders. If the **MultipleQty** needs to be respected in environments that use both multi-level search and constrained planning, it is recommended to set **ConstraintMultipleRule** to "Use".
Attribute based planning	If using attribute-based planning, then days of supply logic will only merge planned orders together when the attribute value(s) on the triggering demands match.
Multi-level search	Days of supply should be avoided for parts that use multi-level search logic. If using multi-level search together with days of supply logic, then planned order due dates will not always fall in line with or respect days of supply policies.
Campaign planning	If using campaign planning with days of supply, be aware that the days of supply window might shift when supply from a part source enabled for campaign planning reaches the maximum number of batches and a new campaign is created.
Two-date planning	Two-date planning can be used with days of supply logic, however if a supply is used to satisfy multiple demands with different request dates, then the earliest of those is chosen and propagated down the structure.

Table 14.27: Days of Supply limitations on other analytics

Analytic / Calculation	Impact of Days of Supply
Planning time fence	If using period-based days of supply (for example, the "ByPeriod" or ByPeriodEnd" settings), then the calculated **PartSource.PTFDate** must fall on a date belonging to the part's applicable **PeriodSupplyDueCalendar**. In some cases, this can result in the planning time fence being set later than it otherwise would be and hence some planned orders being made later than they otherwise would be. For more information, see "PTFDate and Days of Supply" on page 1349.

Shortage calculations

The **Part** table has two calculated fields for determining actions to be taken for the part based on existing inventory level, demand, and supply information. These are:

- FirstNeededSupplyDate
- FirstShortageDate

FirstNeededSupplyDate is the date where the total demand to that point is greater than the nettable on hand. This is the date where the receipt is actually needed. This date may be later than the recommended date of the first receipt, because RapidResponse may be trying to satisfy safety stocking requirements. The **FirstNeededSupply** calculation assumes safety inventory is there exactly for the reason of pushing out the absolute **FirstNeededSupplyDate**.

FirstShortageDate is the date where there will be a shortage if no rescheduling or planned order action is taken.

As shown in the following table, Takt Time functionality can impact calculations involving days of supply logic. This example assumes that **PartSource.MaximumQty** is set to 50, and that **Part.NumberOfDaysSupply** is set to 5. Note that when Takt Time is turned off, some planned orders are created due on the Monday and others on Thursday. However, when Takt Time is used (in this case, set to -1), the logic considers the entire requirement for 300 due on the Monday, and creates six planned orders each for 50 units due on Monday and each of the five preceding work days.

Table 14.28: Planned order spreading with Takt Time and Days Supply logic

Day	Fri	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand							125			175	
Planned Orders (TaktTime = 0)							50 50 50			50 50 50	
Planned Orders (TaktTime = -1)		50	50	50	50	50	50				

Note: For more information about days of supply logic, see "Days of Supply calculations" on page 1362.

Limitations on the number of planned orders

By default, RapidResponse limits the number of planned orders for a part per planning period (typically a day) to 100 if the **OrderPolicy.MaximumUsage** field is set to **Use**. A global settings is available to modify this limit value.

This limit overrides the **PartSource.MaximumQty** field setting when the previously noted condition is in place.

If RapidResponse reaches the planned orders limit, the last order generated includes all the remaining unsatisfied demand.

For more information about how to control the planned order setting, see "Limiting the number of planned orders" on page 2204.

Start date calculation for planned orders

All supply records have a start date. This is the date production must start or the supplier needs notification of the order. For scheduled receipts, the **StartDate** field is set when they are defined. However, this date may be left undefined meaning it is calculated using the part's order policy from the **DueDate** and **Quantity** of the order.

The **CalcStartDate** field on a **ScheduledReceipt** record is calculated based on the **StartDate** and rescheduled due date (**ReschedDate**). It is the date on which the explosion process bases its calculations for allocations for components. For more information about how RapidResponse calculates the **CalcStartDate** field, see "ScheduledReceipt CalcStartDate field" on page 1637.

Spreading planned orders with Takt Time

In a typical case, when a single demand exceeds **PartSource.MaximumQty**, multiple planned orders are created due on the same date, where all but at most one has a quantity equal to the maximum quantity. With the Takt Time functionality in RapidResponse, however, when planned supply for a part exceeds **PartSource.MaximumQty**, the supply is split into smaller planned orders obeying **PartSource.MaximumQty** and then spread backwards or forwards in time. For example, if a work center has limited capacity, Takt Time could be used to spread load over a given number of time intervals so as not to exceed this capacity. In cases where Takt Time is used and there are still multiple planned orders on a given day, this could indicate a need to temporarily assign additional resources to increase throughput.

Takt Time functionality is controlled by the **PartSource.TaktTime** field. Setting this field to a positive integer separates consecutive planned orders out and ahead that number of time units (as defined by **Part.PlanningCalendars.TimeUnits**). Setting this field to a negative integer separates consecutive planned orders out and back that number of time units (as defined by **Part.PlanningCalendars.TimeUnits**). If this field is set to zero (the default setting), then Takt Time functionality is turned off for the part source.

When spreading planned orders backwards or forwards with Takt Time, the date representing the beginning or end of the effectivity range for the part source being used might be reached. In these cases, multiple planned orders are created on this date.

Notes:

- Use of the **PartSource.TaktTime** field overrides constraint functionality. That is, if a part source has its TaktTime value set to a non-zero value, the part source is then treated as being unconstrained even if constraints are assigned to it.
- When Takt Time is in effect, all planned orders created to satisfy a single required quantity use the same part source.

Examples

The following table shows examples of how different values in the **PartSource.TaktTime** field would affect the timing of planned orders to satisfy given demands. This example assumes that **PartSource.MaximumQty** is set to 50. Note that in this example, when TaktTime is set to -2, multiple planned orders for the part are still generated on some days. For example, there are two planned orders on the Tuesday (one to satisfy part of that days demand, and one offset by two days from Thursday),

Table 14.29: Planned order spreading with different Takt Time settings

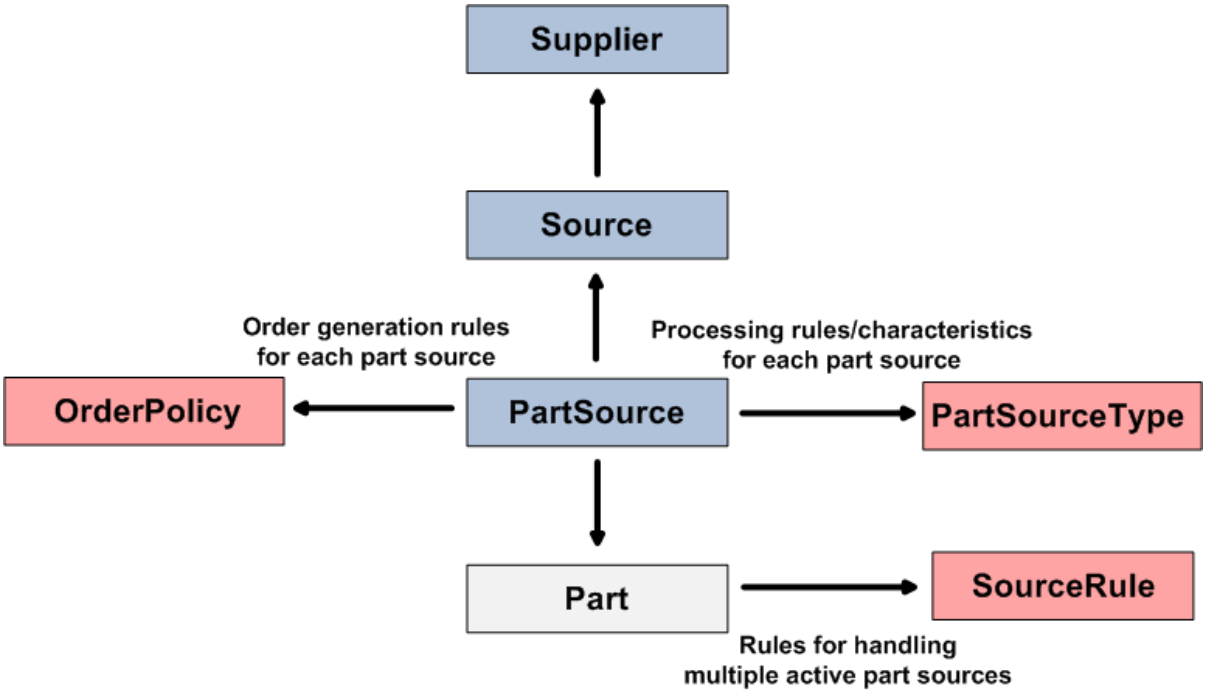
Day	Fri	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
Demand			100		100		100		50		
Planned Orders (TaktTime = 0)			50 50		50 50		50 50		50		
Planned Orders (TaktTime = 1)			50	50	50	50	50	50	50		
Planned Orders (TaktTime = -2)	50		50 50		50 50		50		50		

Part source selection

When demand cannot be satisfied by existing supply (on hand inventory or scheduled receipts), a new supply (planned order) is generated. Planned orders are only generated when there is at least one active source for the part, and RapidResponse allows parts to have more than one source of supply to satisfy a demand requirement (either dependent or independent demand). Multiple sources of supply might include different suppliers from which a part is purchased, a combination of make and transfer sources, different internal routings for make parts using the same BOM structure, or multiple make sources with different product structures configured using alternate BOMs. RapidResponse allows for allotments between different sources of supply to be configured based on rules such as priority and target percentage.

The following illustration shows some of the main tables used in configuring part sources in RapidResponse.

Figure 14.21 Tables associated with part source configuration



Part source selection process

When all on hand inventory and scheduled receipts are used up and new planned orders required to satisfy demand, RapidResponse looks for active sources that can best satisfy the demand while respecting source allotment rules defined on the part where appropriate.

The following table defines the basic part source selection criteria. Note that if attribute-based planning logic is used, then RapidResponse will attempt to create planned orders from part sources that can satisfy the attributes-based qualifications for demands. For more information, see "Supply allotment by attribute" on page 1827. Also if inventory pooling logic is enabled for a part and Pool values are specified on the PartSource records, then RapidResponse will attempt to created planned orders from part sources whose Pool is either a match for the demand pool or is Null. For more information, see "Pools and part source selection" on page 1751.

Table 14.30: Standard part source selection criteria

	Selection criteria	Description
1	On time availability (or least late)	RapidResponse looks for active part sources that can satisfy a planned requirement on time. This involves the following: • The **OrderPolicy.OrderGenerationRule** field setting determines when new orders can be provided from a source. If this field set to "NoOrder" then the part source is assumed to be inactive. If this field is set to "AfterRunDate" then new orders cannot be due before RunDate, and if this field is set to "AfterPTF" then new orders cannot be due before **PartSource.PTFDate**. • Effectivity on the part source is also considered as new planned orders from a given source cannot be earlier than its **PartSource.FirstEffectiveDate** or later than its **PartSource.LastEffectiveDate**. • Note also if a demand falls outside of the effectivity window for all potential part sources, then it can be either be shifted into effectivity for a part source or left unsatisfied. This is based on the **SourceRule.DateEffectivityRule** setting. • If a part source is constrained, then it must have available constraint at least up to the minimum order quantity requirement in order to be used. For more information about constraint, see "Constraint Analysis calculations" on page 1629. Based on the above criteria, RapidResponse equally prefers those part sources that can satisfy the planned requirement either on time or early. If no part source can provide supply on time, then the source than can provide supply the least late is preferred. If multiple part sources can provide the supply either on time or the least late, then the specified source allotment rules are used to determine the supply distribution. **Note:** The ability of a part source to satisfy a requirement for the part on time is based on its own effectivity, constraint, and planning time fence settings. Expected availability of component supplies is not considered (if it is required, then multi-level search logic should be used as discussed in "Multi-level search logic" on page 1863.
2	Priority	If more than one part source is able to provide supply either on time or the least late, then only those part sources that have the highest effective priority as defined by the value in **PartSource.Priority** are considered for use. The interpretation of the value in the **PartSource.Priority** field is affected by the **Part.SourceRule.PriorityRule** setting. If this rule is set to "Ascending" then smaller Priority values are considered to have higher effective priority, and if this rule is set to "Descending" then larger Priority values are considered to have higher effective priority.

Table 14.30: Standard part source selection criteria

	Selection criteria	Description
3	Target	If multiple part sources of the same priority are able to provide supply either on time or the least late, then the target values associated with each part source are used to determine the distribution of sourcing. The **PartSource.Target** field is used to specify the intended ratio or percentage split among part sources of a given priority. For example, if the first source has a **Target** of 4 and the next source has a **Target** of 1, this implies 80% of orders should be sourced from the first and 20% of orders sourced from the second. The actual usage of the **Target** values in the sourcing process is set by the value in the **Part.SourceRule.AllotmentRule** field which has the following four values: • **Ongoing**— targets are maintained on an ongoing basis with each requirement meant to be satisfied by only one source. Each planned requirement is therefore fully allocated to the eligible part source that will be furthest away from (and under) its ongoing target if not used to source that supply. This is the default value. The ongoing target for each **PartSource** record is determined by: 1. dividing its **Target** field by the sum of the **Target** fields across all active **PartSource** records (to give a target fraction) and then 2. multiplying that target fraction by the total quantity of supply sourced across all eligible part sources so far (this total quantity includes the current requirement being sourced). **Note:** If necessary, an initial allotment value can be set in the **PartSource.ToDateQty** field and this value is included in the calculation of supply sourced so far. Additionally, any scheduled receipt can be included or excluded from this calculation based on settings in the **TargetAccumUsage** and **TargetAccum** fields on the **SupplyStatus** table (generated planned orders are always included in ongoing calculations). • **OngoingToDate**—targets are maintained on an ongoing basis with each requirement meant to be satisfied by only one source. Each planned requirement is fully allocated to the eligible part source that has the smallest calculated ongoing quota rating based on all supply that has been sourced to date (excludes the current requirement). The quota rating for each **PartSource** record is determined by: 1. Dividing its **Target** field by the sum of the **Target** fields across all active **PartSource** records (to give a target fraction) and then 2. Dividing the total quantity of all supply sourced the part source (up to but not including the current requirement) over the target fraction. **Note:** If two or more part sources have the same calculated quota rating, then the one with largest **Target** is used, and if the **Target** values are the same then the one with the lowest **SecondaryPriority** is, by default, used (based on configuration). Also, if necessary, an initial allotment value can be set in the **PartSource.ToDateQty** field and this value is included in the ongoing calculation of supply sourced so far. Additionally, any scheduled receipts can be included or excluded from this calculation based on settings in the **TargetAccumUsage** and **TargetAccum** fields on the **SupplyStatus** table (generated planned orders are always considered in ongoing calculations). • **Proportional—**targets are maintained on each individual requirement which is split between eligible part sources. The proportional amount allocated to each source is determined by 1. Dividing its **Target** field by the sum of the **Target** fields across all eligible **PartSource** records (to give a target fraction) and then 2. Multiplying that target fraction by the quantity of the supply requirement to be satisfied. **Note:** If order multiples or minimums are applicable, these are applied separately after the requirement has been split between eligible sources and can lead to excess supply. For these cases, consider use of the **ProportionalLotSize** setting instead.

Table 14.30: Standard part source selection criteria

Selection criteria	Description
	• **ProportionalLotSize**—targets are maintained, as possible, on each individual requirement which is split between eligible part sources. However, with this setting lot sizing is applied after each individual supply split and the remaining requirement reduced by the lot-sized quantity before the next supply portion is assigned. The first proportional amount allocated to a source using this setting is determined by: 1. Selecting the part source with the largest **Target** (if multiple eligible part sources have the same **Target** then the **SecondaryPriority** value is used to pick the part source (by default, lower values are preferred). 2. The target fraction for the selected part source is determined by dividing its **Target** value by the sum of the **Target** values across all active **PartSource** records for the priority. 3. The target fraction is multiplied by the full amount of the supply requirement and this portion assigned to the selected part source. A planned order is then generated to cover the assigned amount of the requirement, with any applicable lot-sizing applied to the order quantity. The remaining supply to be sourced is determined by reducing the original requirement by the quantity of the lot-sized planned order. Each subsequent proportional amount allocated to a source using this setting is then determined by: 1. Selecting the part source with the next largest **Target** value (**SecondaryPriority** is again used if multiple sources have the same **Target** value). 2. The target fraction for the selected part source is determined by dividing its **Target** value by the sum of the **Target** values across all active **PartSource** records that haven't yet been assigned a split of the current requirement. 3. The target fraction is multiplied by the remaining amount of the current requirement and this portion assigned to the selected part source. A planned order is generated to cover the assigned amount with applicable lot-sizing applied to the order quantity. The remaining supply quantity to be sourced is determined after reducing the last planned order quantity, and the process repeats until the entire requirement is satisfied. **Note:** Although this setting can reduce overall excess from lot-sizing, the actual supply splits won't precisely reflect the Target values and in some cases eligible part sources might not be used at all (due to excess from other supply).

Notes:

- When determining on time (or least late) availability of orders from different part sources, material availability and constraints on components under the assembly being sourced are ignored during standard part source selection. In other words, unlimited component availability is assumed at the point when a part source is selected. If component supply under the assembly being sourced is to be considered during the sourcing process, then multi-level search logic should be enabled as discussed in "Multi-level search logic" on page 1863. Note however that the "OngoingToDate" and "ProportionalLotSize" options are not supported on parts that have multi-level search logic enabled (are treated as being set to "Ongoing" and "Proportional").
- Although a part can have multiple part sources, RapidResponse calculates a primary part source for use in certain types of reporting and application use. This value is reported on the calculated **Part**.**PrimaryPartSource** field. For example, this might be the part source that is

assigned the highest Target value within the highest effective Priority value amongst all sources.

Examples of part source selection

This section provides examples of part source allotments made in RapidResponse including the following:

- "Example 1: Proportional rule " on page 1382
- "Example 2: Lot sizing (Proportional vs ProportionalLotSize rules)" on page 1383
- "Example 3: ProportionalLotSize rule (with SecondaryPriority)" on page 1385
- "Example 4: Ongoing rule" on page 1386
- "Example 5: OngoingToDate rule" on page 1387

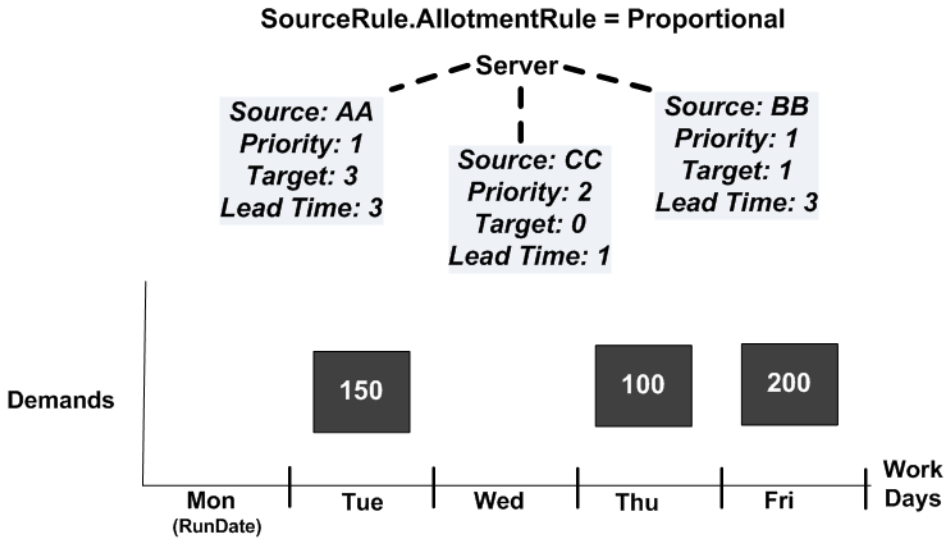
Notes:

- All examples in this section assume a **PartSource.PlanningTimeFence** value of "0", along with an **OrderPolicy.OrderGenerationRule** setting of "AfterPTF" and an **OrderPolicy.PTFRule** setting of "Lead". With this combination of settings, the planning time fence is equivalent to lead time and new supplies generated for demands within the part's lead time cannot be planned on time.
- For an overview of the part source selection process, see "Part source selection" on page 1377.

Example 1: Proportional rule

The following illustration shows details of three part sources used to supply the Server part, along with a set of demands for Server.

Figure 14.22 Demands to be satisfied from multiple potential sources



Using the "Proportional" AllotmentRule, the following table shows source allotments made to satisfy requirements from the demands shown in the previous illustration.

Table 14.31: Part source selection results with Proportional rule

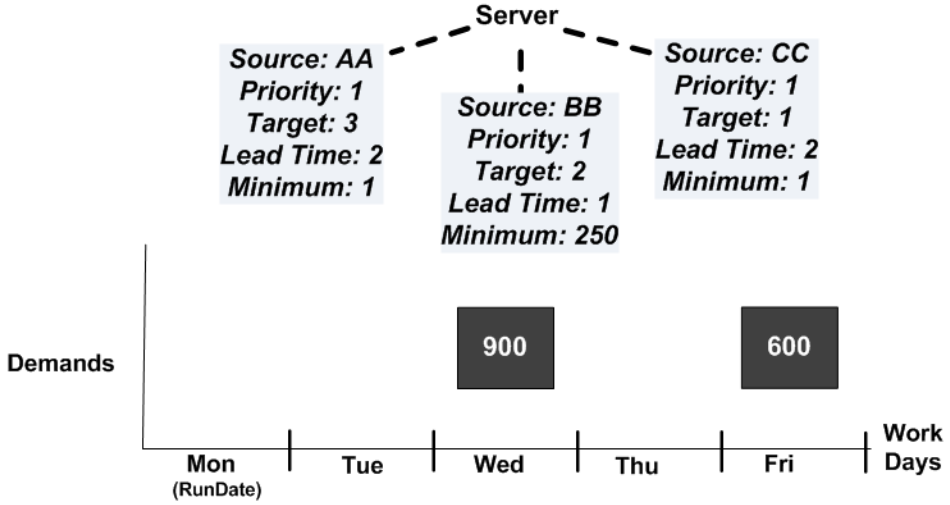
Demand Date	Source Allotment	Sourcing Details
Tuesday	AA: 0 BB: 0 CC: 150	• Because this demand is within lead time for both of the high priority sources AA and BB, it is fully allotted to lower priority source CC which is the only source able to provide it on time.
Thursday	AA: 75 BB: 25 CC: 0	• This demand can be sourced on time from all active sources, so the higher priority sources AA and BB are preferred. The allotment is split based on Target percentages, with 75 units sourced from AA and 25 units sourced from BB.
Friday	AA: 150 BB: 50 CC: 0	• This demand can also be sourced on time from all active sources, so the higher priority sources AA and BBare again preferred. The allotment is split based on Target percentages, with 150 units sourced from AA and 150 units from BB.

Example 2: Lot sizing (Proportional vs ProportionalLotSize rules)

When lot-sizing policies, such as order minimums or multiples are in effect, use of the "Proportional" setting can result in unnecessary amounts of excess supply being generated. This is due to the fact that the "Proportional" setting will split the entire requirement based on **Target** values before then applying lot-sizing to planned order quantities.

For example, consider the following set of demands for part "Server", to be supplied by new planned orders, as shown in the following figure (and note the **MinimumQty** of 250 defined on part source BB).

Figure 14.23 Demands to be satisfied from multiple potential sources



With an **AllotmentRule** of "Proportional" the following table shows source allotments made to satisfy requirements from demands shown in the figure.

Table 14.32: Part source selection results with Proportional rule

Demand Date	Source Allotment	Sourcing Details
Wednesday (900 units)	AA: 450 BB: 300 CC: 150	• All sources have identical **Priority** and can satisfy the demand on time. • Thus supply allotment follows **Target** ratio (3/6 or 50% AA, 2/6 or 33.3% to YY, and 1/6 or 16.7% to ZZ. • The portion alloted to source YY is above its minimum, so no lot sizing adjustment is required.
Friday (600 Units)	AA: 300 BB: 250 CC: 100	• All sources have identical **Priority** and can satisfy the demand on time. • Supply allotments for the requirement again follows the **Target** ratio of 3:2:1 (or, 3/6 or 50% of 600 to XX, 2/6 or 33.3% of 600 to YY, and 1/6 or 16.7% of 600 to ZZ). • The portion assigned to source BB (33.3% of 600, or 200 units) is below its minimum, so this supply is adjusted up to meet the **MinimumQty** of 250. Thus, 650 units are sourced even though only 600 were required leaving 50 units of excess supply at end of the week.

Instead, with an **AllotmentRule of** "ProportionalLotSize", the following table shows how sources allotments would be made to satisfy the same requirements shown in the figure in this example.

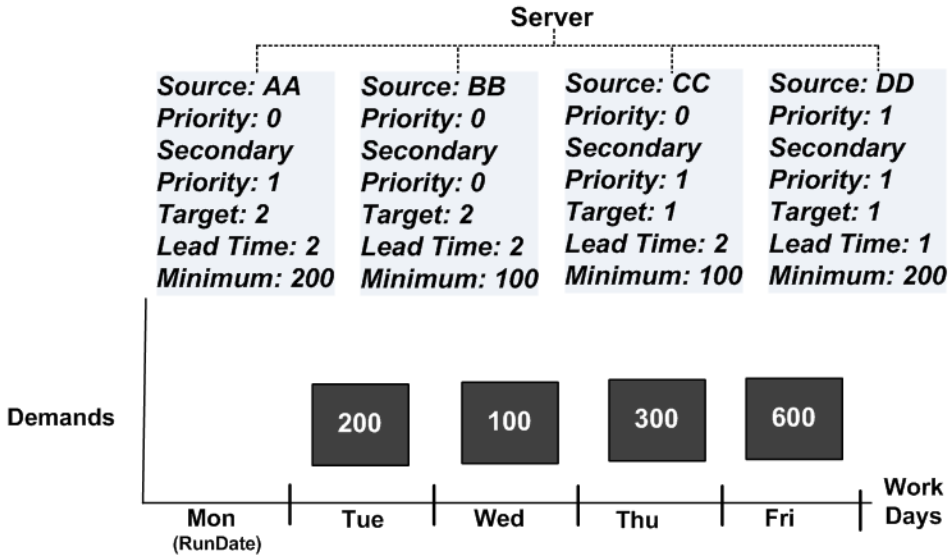
Table 14.33: Part source selection results with ProportionalLotSize rule

Demand Date	Source Allotment	Sourcing Details
Wednesday (900 units)	AA: 450 BB: 300 CC: 150	• All sources have identical **Priority** and can satisfy the demand on time. • First supply allotment is given to the source AA which has the highest **Target**. It receives 3/6 or 50% of the original requirement of 900, which is 450 units. • Next supply allotment is given to source BB which has the highest remaining **Target**. It receives 2/3 or 66.7% of the remaining requirement of 450, which is 300 units. • Final supply allotment is given to source CC which gets the entire remaining requirement of 150 units. **Note:** Because lot-sizing did not affect any supply quantities, the results for this requirement are identical to what was seen with the "Proportional" setting.
Friday (600 Units)	AA: 300 BB: 250 CC: 50	• All sources have the identical **Priority** and can satisfy the demand on time. • First supply allotment is given to source AA which has the largest **Target.** It receives 3/6 or 50% of the original requirement of 600 units, which is 300 units. • Next supply allotment is given to source BB which has the highest remaining **Target**. It receives 2/3 or 66.7% of the remaining requirement of 300, which is 200. Because 200 is below its minimum, the supply is adjusted up to the **MinimumQty** of 250 (and this lot-sized quantity is used to determine the remaining requirement to be sourced). • Final allotment is given to source CC which gets the entire remaining requirement of 50 units. **Note:** In this case, the excess supply of 50 that was seen with the "Proportional" setting has been eliminated. However, source CC received less than its expected target percentage of the requirement (received 8.3% of 600, or 50 units, whereas with "Proportional" it received 16.7% of 600, or 100 units).

Example 3: ProportionalLotSize rule (with SecondaryPriority)

Consider the following set of demands for part "Server", to be supplied by new planned orders, as shown in the following figure (and note the **MinimumQty** and **SecondaryPriority** values defined on each part source).

Figure 14.24 Demands to be satisfied from multiple potential sources



With an **AllotmentRule** of "ProportionalLotSize", the following table shows source allotments made to satisfy requirements from the demands shown in the figure in this example.

Table 14.34: Part source selection results with ProportionaLotSize rule

Demand Date	Source Allotment	Sourcing Details
Tuesday (200 units)	AA: 0 BB: 0 CC: 0 DD: 200	• Because none of the highest priority sources (Priority = 0) can satisfy this demand on time, it is sourced entirely from lower priority source DD which can satisfy it on time.
Wednesday (100 units)	AA: 0 BB: 100 CC: 0 DD: 0	• All of the higher priority sources can satisfy this demand on time. • Among these higher priority sources, AA and BB have the highest **Target** values so **SecondaryPriority** is considered next. • Based on **SecondaryPriority**, BB is used first. By target, it should be assigned 40 units of the supply, and after adjusting for **MinimumQty** it then generates a planned order for 100 units which covers the entire requirement.

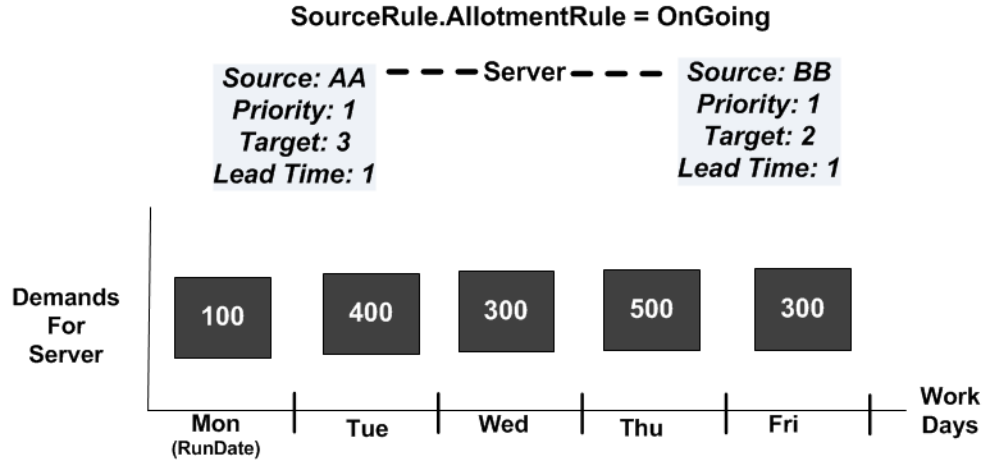
Table 14.34: Part source selection results with ProportionalLotSize rule

Demand Date	Source Allotment	Sourcing Details
Thursday (300 units)	AA: 200 BB 120 CC: 0 DD: 0	• All of the higher priority sources can satisfy this demand on time, so **Target** and **SecondaryPriority** are used. • Based on **SecondaryPriority** of the highest **Target** sources, BB is used before AA. By target, BB should be assigned 120 units of the supply (300/(2/5). Thus, no adjustment for **MinimumQty** is needed. • **AA** is used next. By target, it should also be assigned 120 units of the supply (180/(2/3)). After adjustment for **MinimumQty** it then generates a planned order for 200 units which covers the remaining portion of the requirement (plus 20 additional units).
Friday (580 units remaining)	AA: 232 BB: 232 CC: 116 DD: 0	• For this demand, none of the target allocations per source require any adjustment for **MinimumQty**. Thus the supply allocations following expected **Target** percentages (40%, 40%, and 20%). Note that no adjustments are made for eligible sources of this priority that were ignored during the previous allotments.

Example 4: Ongoing rule

The following illustration shows a set of demands for Server, along with details of two part sources of equivalent priority used to supply the Server part with Target values such that 60% (3/5) of ongoing requirements are meant to be satisfied by source AA and 40% (2/5) of ongoing requirements meant to be satisfied by source BB.

Figure 14.25 Demands to be satisfied from multiple potential sources



With an **AllotmentRule** of "Ongoing", the following table shows source allotments made to satisfy requirements from the demands shown in the figure in this example.

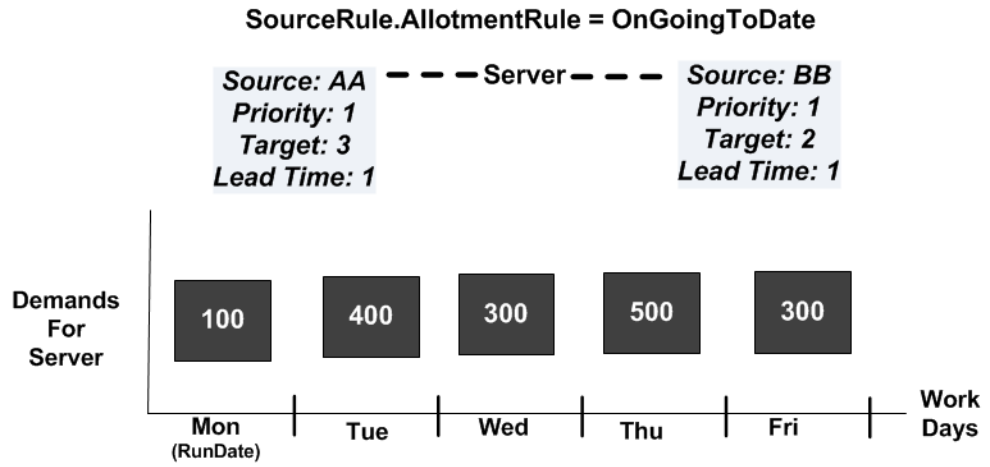
Table 14.35: Part source selection results with Ongoing rule

Demand Date	Source Allotment	Sourcing Details
Monday (100 units)	AA: 100 BB: 0	• Because both sources are of the same priority can satisfy this demand exactly one day late, they are considered equal and the allotment is based on **Target**. If not supplying this order, AA would be 60 units under its ongoing target (0:60) and BB would be 40 units under its ongoing target (0:40). Thus, the entire order is sourced from AA.
Tuesday (400 units)	AA: 400 BB: 0	• Both sources can satisfy this demand on time and are of equal priority, so the allotment is based on **Target**. If not supplying this order, AA would be 200 units under its ongoing target (100:300) and BB would also be 200 units under its ongoing target (0:200). Thus, the entire order is sourced from AA.
Wednesday (300 units)	AA: 0 BB 300	• Also, based on **Target**. If not supplying this order, AA would be still be 20 over its ongoing target (500:480) and BB would be 320 units under its ongoing target (0:320). Thus, the entire order is sourced from BB.
Thursday (500 units)	AA: 500 BB: 0	• Also, based on **Target**. If not supplying this order, AA would be 280 units under its ongoing target (500:780) and BB would be 220 units under its ongoing target (300:520). Thus, the entire order is sourced from AA.
Friday (300 units)	AA: 0 BB 300	• Also, based on **Target**. If not supplying this order, AA would still be 40 units over its ongoing target (1000:960) while BB would be 340 units under its ongoing target (300:640). Thus, the entire order is sourced from BB. At the end of this week, 1600 units have been sourced in total. Source AA is just over its ongoing target of 960 units or 60% (at 1000 units or 62.5%) and BB is just under its ongoing target of 640 units or 40% (at 600 units or 37.5%).

Example 5: OngoingToDate rule

The following illustration shows a set of demands for Server, along with details of two part sources of equivalent priority used to supply the Server part with Target values such that 60% (3/5) of ongoing requirements are meant to be satisfied by source AA and 40% (2/5) of ongoing requirements meant to be satisfied by source BB (this is equivalent to the scenario shown in the previous example except with a different **AllotmentRule** setting).

Figure 14.26 Demands to be satisfied from multiple potential sources



With an **AllotmentRule** of "OngoingToDate", the following table shows source allotments made to satisfy requirements from the demands shown in the figure in this example.

Table 14.36: Part source selection results with OngoingToDate rule

Demand Date	Source Allotment	Sourcing Details
Monday (100 units)	AA: 100 BB: 0	• Because both sources are of the same priority can satisfy this demand exactly one day late, they are considered equal and the allotment is based on quota rating. AA has a calculated quota rating of (0/(3/5) = **0**, and BB has a calculated quota rating of (0/(2/5) = **0**. Since these are the same, AA is selected to source the supply because it has a higher **Target**.
Tuesday (400 units)	AA: 0 BB: 400	• Both sources can satisfy this demand on time and are of equal priority, so the allotment is based on quota rating. AA has a quota rating of (100/(3/5)) = **167**, and BB has a quota rating of (0/(2/5) = **0**. Thus BB is selected to source the supply because it has a lower quota.
Wednesday (300 units)	AA: 300 BB 0	• Also, based on quota rating. AA has a quota rating of (100/(3/5)) = **167**, and BB has a quota rating of (400/(2/5) = **1000**. Thus AA is selected to source the supply because it has a lower quota.
Thursday (500 units)	AA: 500 BB: 0	• Also, based on quota rating. AA has a quota rating of (400/(3/5)) = **667**, and BB has a quota rating of (400/(2/5) = **1000**. Thus AA is selected to source the supply because it has a lower quota.
Friday (300 units)	AA: 0 BB 300	• Also, based on quota rating AA has a quota rating of (900/(3/5)) = **1500**, and BB has a quota rating of (400/(2/5) = **1000**. Thus BB is selected to source the supply because it has a lower quota. At the end of this week, 1600 units have been sourced in total. Source AA is just under its ongoing target of 960 units or 60% (at 900 units or 56.3%) and BB is just over its ongoing target of 640 units or 40% (at 700 units or 43.7%).

Part sources for scheduled receipts

RapidResponse calculates a part source for each scheduled receipt. This value is then stored in the **ScheduledReceipt.PartSource** field. If a given part has only one source, then that source is used to populate this field. But, if a given part has multiple sources, RapidResponse attempts to determine the appropriate source by comparing field values on the scheduled receipt against field values on the part source records.

This is done by evaluating the following prioritized list of conditions until there is exactly one matching part source record:

- PartSource with attributes matching attributes defined on the ScheduledReceipt (if attribute-based planning is enabled)
- PartSource.Pool is NULL or = ScheduledReceipt.InputPool (or if InputPool is Null then = ScheduledReceipt.Order.Pool)
- PartSource.Source.Supplier = ScheduledReceipt.Order.Supplier
- PartSource.Type.PlannedOrderSupplyType.Source = ScheduledReceipt.Order.Type.Source =
- PartSource.BOMAlternate = ScheduledReceipt.BOMAlternate
- PartSource.Routing = ScheduledReceipt.Routing
- PartSource.FirstEffectiveDate <= ScheduledReceipt.DueDate <= PartSource.LastEffectiveDate
- ScheduledReceipt.Order.Type = PartSource.Type.PlannedOrderToSRSupplyType

If the entire list of conditions above has been evaluated and RapidResponse has still not determined a source, one is selected from the list of sources using the following criteria:

- PartSource with the highest logical Priority
- PartSource with the greatest Target value
- PartSource with the earliest EffectiveInDate

Note: Scheduled receipt quantities associated with a part source can, optionally, be included in calculations used by the "Ongoing" allotment rule.

Including scheduled receipts in "ongoing" source selection logic

By default, scheduled receipt quantities are not accumulated and included in the ongoing part source selection logic. That is, only the accumulated planned order quantities, plus the optional **PartSource.ToDateQty** value, is used in determining how far from its intended target a given part source is. To determine whether a scheduled receipt's quantity is included in ongoing accumulation logic, the following fields on the referenced **SupplyStatus** control table are used.

Table 14.37: SupplyStatus field impacting ongoing accumulation logic

Field	Description
TargetAccum	Specifies whether a scheduled receipt's quantity is used in ongoing source accumulation logic for the part source. This setting also impacts how constraint load is generated. Valid values are: • **Ignore**—scheduled receipt quantity not included in ongoing accumulation logic. This is the default setting. • **Order**—scheduled receipt's **TotalQuantity** field used in source accumulation logic. This represents the order quantity before accounting for scrap and any amount already received (such as through a transfer). • **Remaining**—scheduled receipt's **Quantity** field used in source accumulation logic. This represents the remaining order quantity after accounting for scrap and any amount already received. • **RemainingNoFixedLoad**—same as Remaining option (differs only in how it impact constraint load)
TargetAccumUsage	Specifies whether the value chosen in the **TargetAccum** field is used for both ongoing source accumulation and generating constraint load, or only when generating constraint load. Valid values are: • **AccumulationAndConstraint**—TargetAccum setting is used for both source accumulation and constraint load. This is the default value. • **Constraint**—TargetAccum setting is used for constraint load only (the scheduled receipt is always ignored by source accumulation logic).

Phantom processing

Phantom logic represents a situation where dependent demand for a component part under an assembly is not processed, but passed unchanged (blown through) to the explode logic of its components. RapidResponse does not attempt to satisfy demand for a phantom component by rescheduling a scheduled receipt or creating a planned order.

A part's type might specify that all dependent demand is to be blown through. Alternatively, the phantom behavior can be specified only for a particular bill relationship by the type on a **BillOfMaterial** record. In this case, dependent demand generated by the explosion calculation on a phantom bill will bypass the netting of the phantom component. The netting either processes dependent demand, or puts it into one of the following tables:

- **BlowThroughNewAllocation** table
- **BlowThroughPlannedAllocation** table

NewAllocation records are placed in the **BlowThroughNewAllocation** table and **PlannedAllocation** records are placed in the **BlowThroughPlannedAllocation** table.

Phantom processing ignores **PartSource** records, simply exploding demands to lower level parts. If the part has no **BillOfMaterial** records, dependent demands are netted as if the part is a Buy part (phantom processing is not used).

If the **PartType.DependentDemandUsage** control field is **BlowThrough** (or **NetIfInventory** or **NetIfActive** and there is no inventory or no activity respectively), netting treats the part as a phantom part. All **PartSource** records are ignored and demands are immediately exploded through the product structure to its component parts.

BlowThroughNewAllocation

This represents **NewAllocation** records for a part that is not processed by RapidResponse netting calculations because the PartType or BillOfMaterialType specified phantom processing. The records are used as explosion from the part (for example, **BillOfMaterial** records with the part as the assembly).

BlowThroughPlannedAllocation

This represents **PlannedAllocation** records for a part that is not processed by RapidResponse netting calculations because the PartType or BillOfMaterialType specified phantom processing. The records are used as explosion from the part (for example, BillOfMaterial records with the part as the assembly).

Phantom dependent transfer demand

Dependent demand from transfer orders is usually netted, even if the component part is a phantom part. Refer to the following table:

Table 14.38: Dependent transfer demand

Part.Type.DependentDemandUsage value	Result on dependent transfer demand
DoNotGenerate	Not generated (ignored)
Ignore	Not generated (ignored)
BlowThrough	Generated and netted for the component part.
Net	Generated and netted for the component part.
NetIfActive	Generated and netted for the component part.
NetIfInventory	Generated and netted for the component part.

Yield

During the manufacturing process, some amount of an item might be lost or fail to meet required quality standards. The **Yield** field can be used to represent a value by which supply quantities from a part source are expected to be reduced during production because of scrap. For parts involved in a coproduct or byproduct structure, the **CoProductYield** field can be used to represent a value by which supply quantities from a part source should be reduced in consideration of all of a part's co-products and by-products that might come out of the production process. Together these two fields determine the effective yield on a part source.

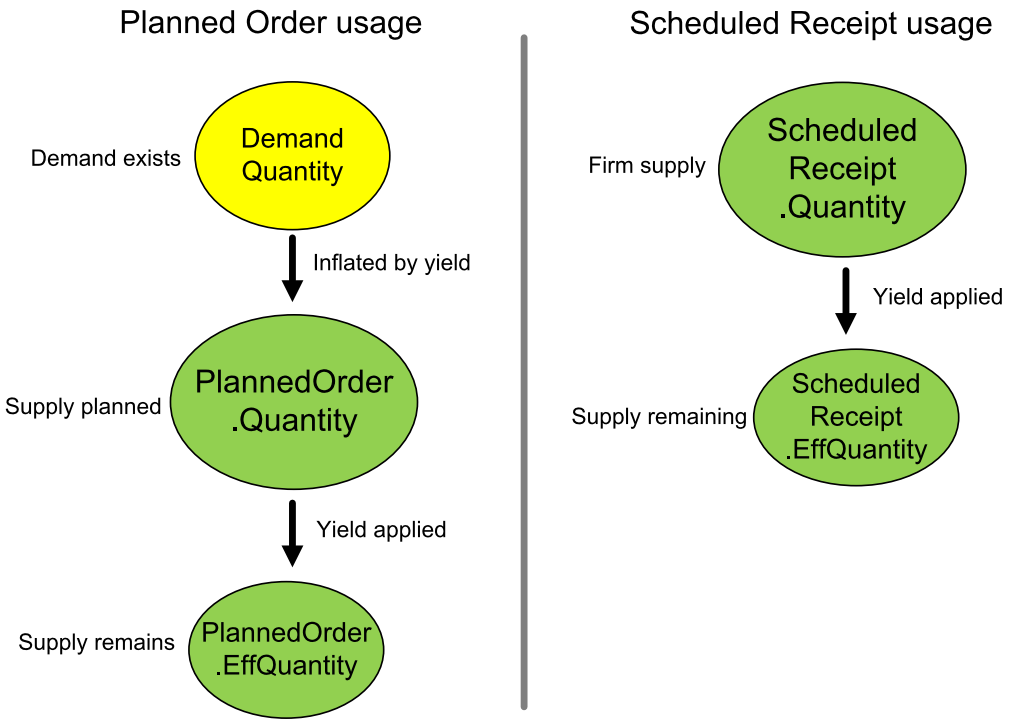
If the yield from a part source is expected to remain stationary over time, then a single set of yield values can be specified using the **Yield** and/or **CoProductYield** fields on the **PartSource** table itself. However, yield from a part source might also be expected to change over time (for example, as production processes are refined or improved). For these cases, the **Yield** and **CoProductYield** fields on the **TimePhasedYield** table should be used instead; a record can be added to this table referencing the applicable **PartSource** for each **EffectiveInDate** on which its yield values are expected to change). The

date which a given **PlannedOrder** or **ScheduledReceipt** used to determine the applicable **TimePhasedYield** record is then calculated and reported in the **ParameterDate** field on those tables.

When generating planned orders, the effective yield is applied inversely against the required demand quantity to determine the actual **PlannedOrder.Quantity** value. In other words, the demand quantity is inflated. This ensures the planned order quantity is made large enough to cover the demand with respect to yield. The amount of the planned order remaining after factors such as yield are applied, and hence available to satisfy demand is reported in **PlannedOrder.EffQuantity**.

Conversely, on scheduled receipts, the effective yield is applied directly against the input **ScheduledReceipt.Quantity** in order to determine the effective quantity. This effective quantity is calculated and reported in the **ScheduledReceipt.EffQuantity** field value. Thus, effective quantities on the scheduled represent how much demand this supply will actually be able to cover.

Figure 14.27 Yield and supplies



Note: Each part source's effective yield is reported in the **PartSource.EffYield** field. This calculated field considers both **Yield** and **CoProductYield** values as appropriate. However, if using time-phased yield, the value in this field is calculated using values on the earliest **TimePhasedYield** record for the part source that is effective on or after the part's **RunDate** is used.

YieldUsage setting

The actual interpretation of yield is affected by the **OrderPolicy.YieldUsage** field. This field has the following settings that determine how yield values are interpreted and applied:

- **Ignore**—do not apply any yield factor. Yield is always ignored, and treated as 1 (no loss).
- **YieldFraction**—value from 0 through 1 (1 means no loss).
- **YieldPercent**—value from 0 through 100 (100 means no loss).
- **ScrapFraction**—value from 0 through 1 (0 means no loss).
- **ScrapPercent**—value from 0 through 100 (0 means no loss).
- **ScrapPiece**—allows a fixed quantity per order to be scrapped.
- **InflationFraction**—formula = 1 / (1 + PartSource.Yield)
- **InflationPercent**—formula = 1 / (1 + PartSource.Yield * 0.01)

When generating planned orders, yield is applied against the demand quantity to determine the actual supply quantity reported in **PlannedOrder.Quantity** (as described in "Yield expression details for planned order quantities" on page 1393), and subsequently applied against that actual quantity to determine the resulting effective quantity reported in **PlannedOrder.EffQuantity** (as described in "Yield expression details for planned order effective quantities" on page 1394).

For scheduled receipts, yield is applied against the actual input **ScheduledReceipt.Quantity** field to return the effective quantity reported in **ScheduledReceipt.EffQuantity** (as described in "Yield expression details for scheduled receipts" on page 1394).

Yield expressions for planned orders and scheduled receipts

When generating planned orders, RapidResponse can inflate the actual supply quantities to account for the expected production yield (due to scrap or other factors). If there is a requirement to ensure these planned order quantities are not fractional, then order multiples can be used as discussed in "Planned order lot sizing" on page 1358. Specifically, if **OrderPolicy.MultipleUsage** is set to "Use" and **PartSource.MultipleQty** is set to a non-zero whole number, then fractional amounts are not included in the planned order quantities (which are rounded up the nearest specified multiple as required).

The following table shows how settings in the **OrderPolicy.YieldUsage** field impact usage of the yield factor when calculating planned order quantities.

Table 14.39: Yield expression details for planned order quantities

YieldUsage Setting	PlannedOrder.Quantity expression
YieldFraction	=Demanded Quantity / Yield
YieldPercent	=Demanded Quantity / (Yield * 0.01)
ScrapFraction	=Demanded Quantity / (1 - Yield)
ScrapPercent	=Demanded Quantity / (1 - (Yield * 0.01))
ScrapPiece	=Demanded Quantity + Yield

Table 14.39: Yield expression details for planned order quantities

YieldUsage Setting	PlannedOrder.Quantity expression
InflationFraction	=Demanded Quantity / (1 / (1 + Yield))
InflationPercent	=Demanded Quantity / (1 / (1 + Yield * 0.01))
Ignore	=Demanded Quantity

Yield is also used when calculating effective planned order and scheduled receipt quantities to reduce the actual supply quantity down to the expected output after accounting for factors like scrap. The following table shows how settings in the **OrderPolicy.YieldUsage** field affect yield factor usage when calculating effective planned order quantities.

Table 14.40: Yield expression details for planned order effective quantities

YieldUsage Setting	PlannedOrder.EffQuantity expression
YieldFraction	=PlannedOrder.Quantity * Yield
YieldPercent	=PlannedOrder.Quantity * (Yield * 0.01)
ScrapFraction	=PlannedOrder.Quantity * (1 - Yield)
ScrapPercent	=PlannedOrder.Quantity * (1 - (Yield * 0.01))
ScrapPiece	=PlannedOrder.Quantity - Yield
InflationFraction	=PlannedOrder.Quantity * (1 / (1 + Yield))
InflationPercent	=PlannedOrder.Quantity * (1 / (1 + Yield * 0.01))
Ignore	=PlannedOrder.Quantity

The following table shows how settings in the **OrderPolicy.YieldUsage** field affect yield factor usage when calculating effective scheduled receipt quantities.

Table 14.41: Yield expression details for scheduled receipts

YieldUsage Setting	ScheduledReceipt.EffQuantity expression
YieldFraction	=ScheduledReceipt.Quantity * Yield
YieldPercent	=ScheduledReceipt.Quantity * (Yield * 0.01)
ScrapFraction	=ScheduledReceipt.Quantity * (1 - Yield)
ScrapPercent	=ScheduledReceipt.Quantity * (1 - (Yield * 0.01))
ScrapPiece	=ScheduledReceipt.Quantity - Yield
InflationFraction	=ScheduledReceipt.Quantity * (1 / (1 + Yield))
InflationPercent	=ScheduledReceipt.Quantity * (1 / (1 + Yield * 0.01))
Ignore	=ScheduledReceipt.Quantity

Note that after applying the yield factor, effective planned order and scheduled receipt quantities might be fractional. However rounding options are available to reduce or eliminate fractional amounts as discussed in "Rounding effective supply quantities" on page 1395. If necessary, yield usage can also be disabled for select scheduled receipts as discussed in "Ignoring yield on scheduled receipts" on page 1395.

Note: The **CoProductYield** field only uses the Ignore, YieldFraction and YieldPercent values. It treats ScrapPiece, ScrapFraction, and InflationFraction as YieldFraction, and treats ScrapPercent and InflationPercent as YieldPercent. For more information about co-products and by-products, see "Co-products and by-products" on page 1581.

Rounding effective supply quantities

Applying a yield factor might result in fractional amounts reported on some planned order and scheduled receipt effective quantities. In some cases, it might be desirable to save and accumulate these fractional supply amounts over time. In other cases – for example, where it is not possible to stock fractional amounts from individual orders – it might be preferable to reduce or scrap those fractional amounts entirely. For this purpose, rounding rules can be configured and applied against effective supply quantities.

To enable rounding of effective supply quantities for a given part source, the associated **OrderPolicy.YieldRoundingUsage** value should be set to "Use". With this setting, effective quantities on planned orders and scheduled receipts are rounded down to the number of decimal places specified by the **UnitOfMeasure.RoundingPrecision** value referenced by the part. The following table shows an example of how different rounding configurations impact the effective supply quantity.

Table 14.42: Impact of rounding on effective supply quantities

Quantity	Yield	Yield Usage	Yield Rounding Usage	Rounding Precision	EffQuantity
175	.985	Fractional	Ignore	9	172.375
			Use	5	172.375
			Use	2	172.37
			Use	1	172.3
			Use	0	172

Ignoring yield on scheduled receipts

Scheduled receipts can be configured to ignore the yield setting on the part source it uses. For example, this might be done for scheduled receipts that are at a late stage of production where the yield factor no longer applies. Yield usage on scheduled receipts is controlled through the **SupplyStatus.YieldRule** which has the following two values:

- **Ignore**—supplies ignore the yield ratio defined on the part source. Yield on supplies using this setting is assumed to be 100%.
- **Use**—supplies are reduced by the yield ratio defined on the part source.

Ignoring assembly yield on dependent demands

RapidResponse can be configured to ignore an assembly's yield when calculating dependent demand. For example, if an assembly contains expensive components, and those expensive components will not be scrapped if there are production issues with the assembly, then it might be desirable for those expensive components to not be inflated by the assembly's yield during planning. This is controlled by setting the **BOMType.IgnoreAssemblyYield** field to "Y".

Supply

The scheduled supply for a part is stored in the **ScheduledReceipt** table. An order is a record in the **SupplyOrder** table and is referenced by the **Order** field in the **ScheduledReceipt** table. The usage of the supply is controlled by the **ScheduledReceipt.Order.Type** field and by the **ScheduledReceipt.Status** field.

When a supply record is processed, the **RecommendedDate** field is set to reflect when it's used in netting and the **ReschedDate** field is set to either the recommended date or the original due date depending upon whether the order is reschedulable. The value of the **SupplyStatus.RescheduleRule** field determines if an order is reschedulable.

The part source's yield factor is used to adjust scheduled supply to account for defective parts either during manufacture or in inspection of a purchase order. The exact usage of the Yield value is determined by the order policy for a part source. The amount of a scheduled receipt or planned order's quantity that satisfies demand (adjust inventory) is the amount left after adjusting for yield.

Scheduled receipts usage

Scheduled receipts are sorted in the following order for usage in netting:

- Order.Type.SortBeforeDate
- EffDueDate
- Order.Type.SortSameDate
- OrderPriority + TransactionSequence
- Status.SortBeforeStart
- EffStartDate
- Status.SortSameStart
- Quantity
- Order.ID
- Line
- Order.Type
- Part.PrimaryAlternatePart
- Order.Site

RapidResponse fields that affect supply

The following RapidResponse fields affect supply:

- If **ScheduledReceipt.Status.IgnoreSupply** control value is **Y** or **.Type.ProcessingRule** control value is **Ignore**, the scheduled receipt record is not used in netting.
- Records with zero or negative quantity are ignored (**ScheduledReceipt.Quantity**).
- If both **DueDate** and **DockDate** date values are undefined, then the supply is ignored. If either or both are defined (including future), the supply is used.
- If **Type.ProcessingRule** control value is **RecommendOnly** or **Reschedulable**, the supply is only used when necessary to keep the inventory above the part's **SafetyStockQty**. The **RecommendedDate** field is set to the date where the supply is required. The **ReschedDate** field is set to the RecommendedDate for reschedulable and to the DueDate for recommend only. If **.Type.ProcessingRule** is **Non-Reschedulable**, the amount of the supply is used to increase the inventory level on the DueDate, regardless of the demand situation. The **RecommendedDate** and **ReschedDate** fields are both set to the DueDate.
- If a supply that is **Reschedulable** or **RecommendOnly** is not needed, its **RecommendedDate** field is set to **d'Undefined'**.
- If **SupplyStatus.IgnoreStartDate** control value is set to **Y**, RapidResponse ignores the **ScheduledReceipt.StartDate** and **Allocation.DueDate** fields while calculating the **ScheduledReceipt.AvailableDate** field. Instead, RapidResponse uses the part source's stated lead-time. For information about lead-time, see "Lead time calculations" on page 1307.
- If **SupplyStatus.RescheduleRule** = 'Received', and **SupplyType.ProcessingRule** has a value other than 'InTransit' or 'InTransitExact', the supply is ignored.

Demand

Demand information is stored in the following RapidResponse tables:

- **IndependentDemand** table
- **Allocation** table
- **NewAllocation** table
- **PlannedAllocation** table
- **Forecast** table

IndependentDemand table

The **IndependentDemand** table specifies demand that is not dependent on other demand. Examples are customer orders and service parts. Sales forecast is also entered into this table but is not used directly by netting. Instead, it's used by forecast consumption on MPS parts to generate records in the **Forecast** table. For more information about this table, see "IndependentDemand table" on page 358.

Allocation table

The **Allocation** table includes demand for a part to be used in a work order (ScheduledReceipt) for a parent part. These demands are entered into the enterprise data source when the work order is

generated. They are pre-allocated dependent demand. For information about allocation data, see "Allocation data" on page 1399.

NewAllocation table

The **NewAllocation** table includes the demand generated for firm planned orders for manufactured parts (ScheduledReceipt) that did not have their dependent demand pre-allocated. For more information about this table, see "NewAllocation table" on page 1137.

PlannedAllocation table

The **PlannedAllocation** table includes the demand generated for a component of a manufactured part where the RapidResponse is recommending a new planned order. For more information about this table, see "PlannedAllocation table" on page 1147.

Forecast table

The **Forecast** table is generated from the other demand tables giving a detailed sales forecast and the amount left unconsumed. Records in this table are generated for MPS parts.

Demand records

Each demand record has a **DemandType** record associated with it. This determines the type of demand and how it is used in netting. The following table lists fields in various tables that reference the **DemandType** table.

Table 14.43: Demand tables and fields that reference DemandType

Table	Field name	Description
IndependentDemand	Type	Describes the independent demand.
Allocation	ScheduledReceipt.Order.Type. AllocationDemandType	The demand type that generated this allocation record.
NewAllocation (calculated table)	ScheduledReceipt.Order.Type. AllocationDemandType	The demand type that generated this allocation record.
PlannedAllocation (calculated table)	Type	The demand type from the PlannedOrder supply type or from a planning bill type.

DemandType processing rules

The **DemandType.ProcessingRule** control value determines how demand is used in netting. These rules are described in the following table.

Table 14.44: DemandType processing rules

Processing rule	Description
Ignore	The demand record is not used.
SalesForecast	The demand record is not directly used in netting. Instead, it is used by MPS parts for forecast consumption logic and generating an unconsumed forecast demand in the **Forecast** table.
SalesActual	Real demand in netting. It's also used to reduce SalesForecast in the MPS forecast consumption logic.
ProductionForecast	Dependent demand from MPS planning logic. This is used as demand for netting and represents demand for planned production that does not yet have actual customer requirements.
Regular	Demand used in netting but not in MPS forecasting logic. Some MRP systems have a type called "Abnormal demand", meaning it is demand that was not forecast. Demand on non MPS parts may also be classified as regular (although Sales actual would work equally as well since there would be no MPS logic).

Forecast consumption considers the whole order quantity. On **IndependentDemand** records, quantity is used in netting and the sum of Quantity and ShippedQty is used for ForecastConsumption. On **Allocation** records, the RemainingQuantity is used in netting and TotalQuantity is used in ForecastConsumption.

Notes:

- The **SalesActual**, **ProductionForecast** and **Regular** demand types are processed by the netting logic as reducing inventory levels on the DueDate of the demand.
- Forecast records reduce inventory on the Date of the forecast.
- The quantity used in netting is the remaining quantity to be delivered not the total order quantity.

Allocation data

Dependent demand that was pre-exploded in the enterprise data source as part of the order generation process can be imported into the **Allocation** table. Although these records are pre-exploded, the Explosion calculations suggests a ReschedDate as a replacement for the input DueDate (for example, this might occur if the **ScheduledReceipt** is recommended to be rescheduled by netting). Certain Quantity values on Allocation records are also calculated by RapidResponse.

Automatic allocation adjustment

Automatic allocation adjustment refers to **Allocation** records being updated in response to the **Quantity** field on the parent **ScheduledReceipt** record changing. The **Allocation.RemainingQuantity** field may be outdated if the parent **ScheduledReceipt.Quantity** field is modified.

RapidResponse calculates a value for the **Allocation.EffQuantity** field. Instead of using the **Allocation.RemainingQuantity** field to report allocation information, the **Allocation.EffQuantity** field is used.

In order for RapidResponse to calculate an adjusted value for the **Allocation.EffQuantity** field, data must be imported into the **ScheduledReceipt.OriginalQuantity** field. If data is not imported into this field, RapidResponse assigns the value of the **Allocation.RemainingQuantity** fieldto the **Allocation.EffQuantity** field**.** For information about the **Allocation.EffQuantity** field, see "Allocation table" on page 182.

The following table describes the automatic allocation adjustment calculation. This calculation determines the effective quantity per on the allocation. It then determines the new **EffQuantity** on the allocation based on changed quantity on the scheduled receipt.

Table 14.45: Automatic allocation adjustment calculation

Step	Description	Calculation
1	Check the input data	Calculate: **QuantityToUse** = ? (ScheduledReceipt.OriginalQuantity > 0 ScheduledReceipt.OriginalQuantity) ScheduledReceipt.Quantity If ScheduledReceipt.QuantityToUse <= 0 or ScheduledReceipt.TotalQuantity <= 0 or Allocation.TotalQuantity <= 0 then: **Allocation.EffQuantity** = Allocation.RemainingQuantity
2	Determine effective quantity per unit (first part)	For ScheduledReceipts, calculate: **SRTotal** = Max(ScheduledReceipt.TotalQuantity, QuantityToUse) For Allocations, calculate: **ALTotal**=Max(Allocation.TotalQuantity, Allocation.RemainingQuantity)
3	Determine effective quantity per unit (second part)	Calculate EffQuantityPer which is the larger of the quantity per calculated from the total supply, total allocation and as calculated from the remaining supply quantity and the remaining allocation. Calculated as: **EffQuantityPer**=Max(ALTotal/SRTotal, Allocation.RemainingQuantity/ScheduledReceipt.OriginalQuantity)
4	Determine InKit quantity	Calculate InKit as: **InKit** = Max(0, ALTotal - RemainingQuantity - (SRTotal - QuantityToUse)*EffQuantityPer)
5	Resolve the Allocation	Calculate Allocation.EffQuantity as: **Allocation.EffQuantity** = ScheduledReceipt.Quantity * EffQuantityPer - InKit

Safety stock

Safety stock is intended to provide a buffer level of inventory to compensate for orders in excess of forecast within the cumulative lead time of a part. RapidResponse calculations create orders as necessary to maintain this level. This section covers the following:

- "Safety stock policy configuration" on page 1401
- "Basic safety stock rules" on page 1402
- "Safety stock calculations" on page 1402
- "Analytic limitations with safety stock" on page 1418

Note: RapidResponse also supports single and multi-echelon inventory optimization methods that generate safety stock recommendations based on historical data and a specified service level. For more information, see "Inventory planning and optimization" on page 1935.

Safety stock policy configuration

Controls for many basic safety stock settings, such as how the safety stock level is determined and the conditions under which it is maintained, are available in the **SafetyStockPolicy** table. This table is referenced from the **Part.SafetyStockPolicy** field and it's settings are used by all parts for which that reference is not Null.

For parts where the **SafetyStockPolicy** reference is Null, a series of corresponding control settings on the part's referenced **PartType** and **PlanningCalendar** records are used instead. This provides backward compatibility with versions of RapidResponse prior to 2014.4. However, use of the **SafetyStockPolicy** table is recommended as it supports easier modeling of different combinations of safety stock settings across scenarios without the need to create additional **PartType** or **PlanningCalendars** records (each of which contains many other planning settings unrelated to safety stock). Additionally, certain settings added in recent versions of RapidResponse are defined only on fields in the **SafetyStockPolicy** table (but not on corresponding fields in the **PartType** table).

Unless otherwise noted, this chapter assumes a part's basic safety stock settings are configured using fields on the **SafetyStockPolicy** table. As appropriate, the equivalent **PartType** and **PlanningCalendars** fields can be assumed instead. The following table lists the main control fields on the **SafetyStockPolicy** table and their equivalent settings on other control tables.

Table 14.46: SafetyStockPolicy fields and equivalents

SafetyStockPolicy fields	Equivalent setting (used when Part.SafetyStockPolicy is Null)
ExpiryRule	None
OrderPointDateRule	PartType.OrderPointDateRule
OrderPointIncrease	PartType.OrderPointIncrease
SafetyDaysCalendar	None
SafetyDaysDependentDemandUsage	None
SafetyStockDateRule	PartType.SafetyStockDateRule
SafetyStockQuantityRule	PartType.SafetyStockQuantityRule
SafetyStockRule	PartType.SafetyStockRule
PercentSafetyInterval	PlanningCalendars.PercentSafetyInterval
RangeOfCoverage	PlanningCalendars.RangeOfCoverage

Additionally, if a part references a valid **SafetyStockPolicy** record, but either of that record's **PercentSafetyInterval** or **RangeOfCoverage** values are Null, then the equivalent fields from the part's **PlanningCalendars** reference are used instead.

Notes:

- A series of calculated fields on the **Part** table report the effective safety stock setting for each part. For example, the **EffectiveSafetyStockRule** field reports the setting from the **SafetyStockRule** field applicable to each part (as taken from either the **SafetyStockPolicy** or **PartType** record). For more information, see "Part table — calculated and set fields" on page 428.
- If upgrading from an earlier version of RapidResponse, a workbook is available that can be used to create and reference **SafetyStockPolicy** records based on a part's existing **PartType**

and **PlanningCalendars** settings. For more information, see "Moving safety stock settings to the SafetyStockPolicy table" on page 2409.

Basic safety stock rules

Safety stock calculations in RapidResponse are controlled by setting values in the following three fields on the **SafetyStockPolicy** table (or on the **PartType** table if a part's **SafetyStockPolicy** reference is Null):

Note: In order for safety stock to be calculated at all, **Part.MUEPoolNettingType.SafetyStockRule** must be set to a setting other than **Ignore** or else safety stock will not be calculated.

SafetyStockRule

The **SafetyStockPolicy.SafetyStockRule** control value determines how the safety stock level is maintained (for example, safety stock might always be maintained, never maintained, or maintained only when certain conditions are met).

SafetyStockDateRule

The **SafetyStockPolicy.SafetyStockDateRule** control value determines the dates on which to plan orders used to maintain the safety stock level (for example, safety stock orders can only be planned as of the date of the first demand or only planned outside of lead time).

SafetyStockQuantityRule

The **SafetyStockPolicy.SafetyStockQuantityRule** control value determines how the safety stock level is calculated. Safety stock levels can be calculated statically (for example, safety stock can be set to a single fixed value or different fixed values that become effective on specified dates).

Alternatively, safety stock levels can be calculated dynamically (for example, to satisfy a part's average daily requirements). Based on actual demand patterns, safety stock levels will then rise and/or fall over time. Thus, when the safety stock rises, a new order is required to satisfy the new level, and when the safety stock drops, the difference between the old and current safety stock quantity is then available to satisfy demand.

Note: If model, unit, or pool logic is enabled, safety stock for a part can be planned in a specific pool, the common pool, or ignored altogether. For more information, see "Safety stock and pools" on page 1749.

Safety stock calculations

Orders for safety stock are created when a demand order causes projected inventory to fall below a predefined level. The determination of the safety stock level for each part is influenced by the applicable **SafetyStockQuantityRule** setting. Based on the setting in this field, safety stock can be set to a specified constant value (or series of time-phased constant values), or it can be dynamically calculated from demand using a number of different methods.

The following safety stock calculations are discussed in this section:

- Fixed safety stock
- "Time-phased safety stock" on page 1403
- "FractionOfDemand and PercentOfDemand" on page 1404
- "Time-phased safety days" on page 1407
- "RangeOfCoverage and RangeOfCoverageLeadTime" on page 1410
- "TimePhasedRangeOfCoverage" on page 1412
- Demand usage in dynamic safety stock calculations
- "Priority and safety stock" on page 1418

Note: All safety stock quantities, regardless of how they are provided or calculated, are reported in the **Activity** table on records with a **Type** of "DynamicSafety". For each of these records, the **AssociatedQty** field reports the change in the safety stock level on a given date. In cases where a dynamically calculated (or time-phased) safety stock quantity is equivalent to the previous safety stock quantity, a new **Activity** record is not generated.

Fixed safety stock

A constant value is used for the safety stock level when **SafetyStockQuantityRule** is set to 'FixedQuantity'. The level is set to the value in **Part.SafetyStockQty** and maintained as a constant value throughout the planing horizon.

Time-phased safety stock

When the **SafetyStockQuantityRule** is set to "TimePhasedQuantity", records in the **TimePhasedSafety** table define a series of constant safety stock levels. On each date specified in the **Date** field for a given record, the safety stock level is changed to the value in the **Quantity** field on that record. For more information, see "TimePhasedSafety table" on page 655.

The following table shows an example of time-phased safety stock levels for a part.

Table 14.47: TimePhasedSafety values

Part	Date	Quantity
CPU	06-01-2016	25
CPU	10-01-2016	35
CPU	01-01-2017	40
CPU	03-01-2017	20

On October 1, 2016 the safety stock level for part CPU increases from 25 to 35. If there are less than 35 units of the part in stock, a planned order is created to raise the stock to 35. The safety stock level stays at 35 until the end of the year. On January 1, 2017 the safety stock level increases to 40. If there are less than 40 units of the part in stock, a planned order is created to raise the stock to 40. Finally, on March

01, 2017 the safety stock level drops to 20. This means there would be 20 units previously held as safety stock that are then available to cover demand.

The **SafetyStockQuantityRule** can also be set to "TimePhasedQuantityToLatest". This option is identical to "TimePhasedQuantity" except in how it handles a drop in safety stock after the last demand in the horizon. With "TimePhasedQuantityToLatest", if there is such a drop in the safety stock level, the quantity of the last planned order is reduced to ensure the resulting inventory level meets the latest safety stock level defined in **TimePhasedSafety** table.

For example, using the values shown in "TimePhasedSafety values" on page 1403, assume the latest demand for the part is on February 20, 2017 and for 70 units. Given a balance of 40 units and no scheduled supply, a planned order for 50 units would be created (thus leaving the balance at 20 units after the demand is satisfied). Instead if **SafetyStockQuantityRule** was "TimePhasedQuantity", that last planned order would have been for 70 units.

Note: TimePhasedSafety records with **Date** = 'Past' are ignored by netting calculations.

FractionOfDemand and PercentOfDemand

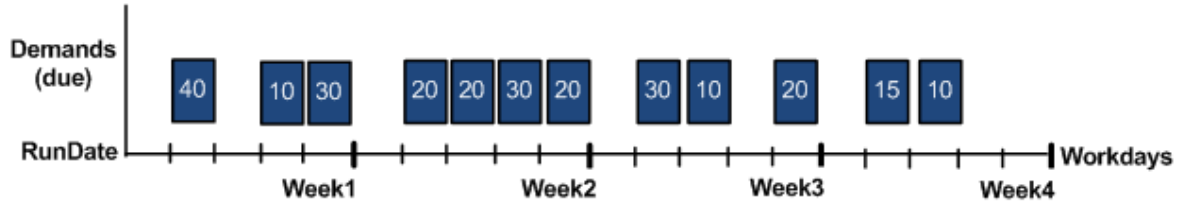
When the **SafetyStockQuantityRule** is set to "FractionOfDemand" or "PercentOfDemand", safety stock is dynamically calculated as a fraction or percentage of the demand in each specified period. For example, safety stock might be set to 15 percent of the total demand in each month. Similarly, safety stock might be set to .1 of the total demand in each three-week period.

The following table outlines other fields that should be set when using the "FractionOfDemand" or "PercentOfDemand" settings:

Field	Usage
SafetyStockPolicy.PercentSaferyInterval	The calendar whose intervals are used for demand collection and setting the safety stock level. For example, safety stock might be calculated based on demand found in each "Month" or "Week".
Part.PercentSafetyIntervalCount	The number of **PercentSafetyInterval** calendar markers over which to collect demand for use in setting the safety stock level. For example, safety stock might be calculated based on demand found in each "2" week period.
Part.PercentSafetyPercent	The percentage or fraction of total collected demand to set the safety stock level to. For example, safety stock might set to ten percent of the demand found in each two-week period . If using "FractionOfDemand" setting then a value of ".1" would be provided, and if using "PercentOfDemand' than a value .
SafetyStockPolicy.SafetyStockDateRule	Demands for dynamic safety stock calculations are collected starting from **RunDate** and in specified intervals of the **PercentSafetyInterval** calendar after that. However, the part's **SafetyStockDateRule** setting, along with it's lead time and/or demands, can influence when a part's initial safety stock level is generated. Subsequent safety stock levels will be on specified intervals of the **PercentSafetyInterval** calendar. Refer to the examples below to see differences between use of the **SafetyStockDateRule** settings of "RunDate" and "LeadTime".

Example 1 - Workday safety intervals

Suppose a part has the set of demands shown in the image below (assume RunDate is a Monday).

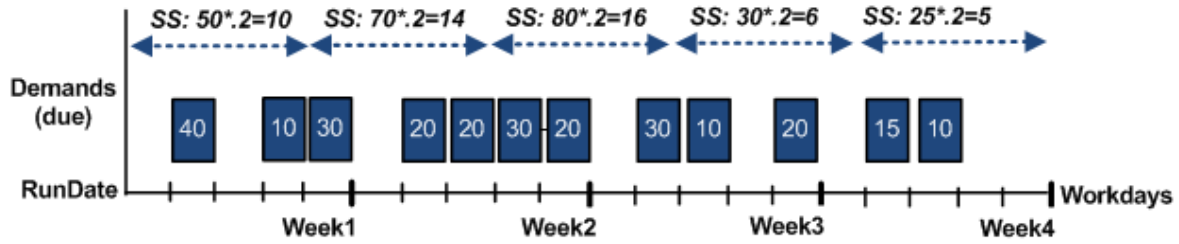


Further suppose the part has the following safety-stock related settings:

- **SafetyStockQuantityRule:** FractionOfDemand
- **PercentSafetyInterval:** Workday
- **PercentSafetyIntervalCount:** 5
- **PercentSafetyPercent:** .2
- **SafetyStockDateRule**: RunDate
- **EffLeadTime**: 2

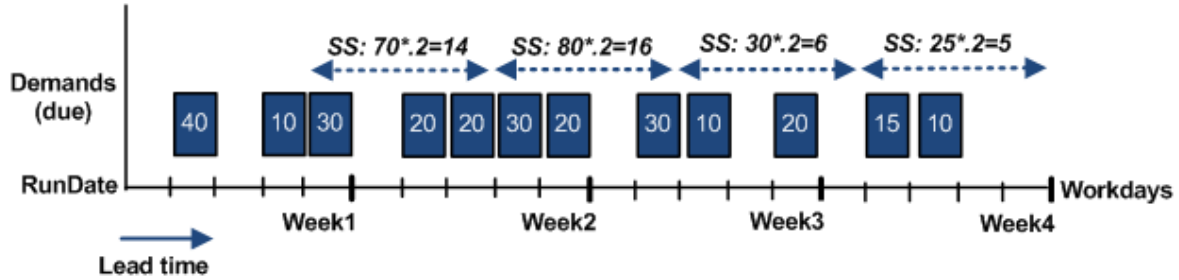
Thus safety stock would be set to 20% of the total demand found in each 4-day period. As determined by the **SafetyStockQuantityRule** setting, safety stock is then calculated starting from RunDate and recalculated every 4 work days after that. Refer to the following image.

Figure 14.28 SafetyStockDateRule set to "RunDate"



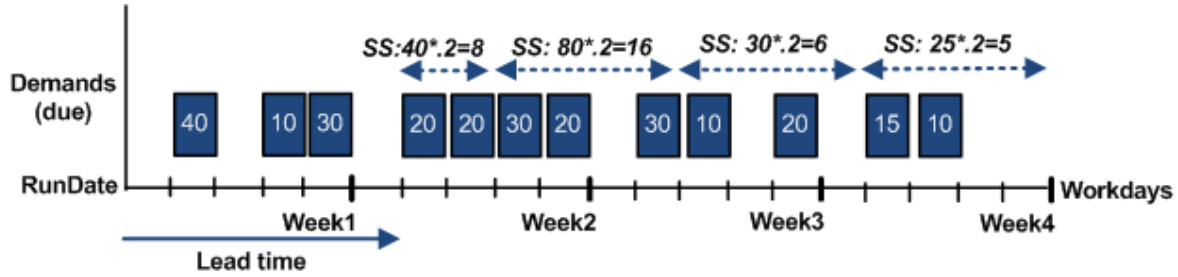
Instead assume the part has its **SafetyStockDateRule** setting changed to "LeadTime". With this setting, safety stock levels would be generated starting from the latter of the part's lead time or the date on the **PercentSafetyInterval** calendar that is **PercentSafetyIntervalCount** calendar markers after **RunDate**. Refer to the following illustration, and note that the first safety stock level from the previous image is no longer generated, but the subsequent levels are identical.

Figure 14.29 SafetyStockDateRule set to "LeadTime" (lead time of 2 days)



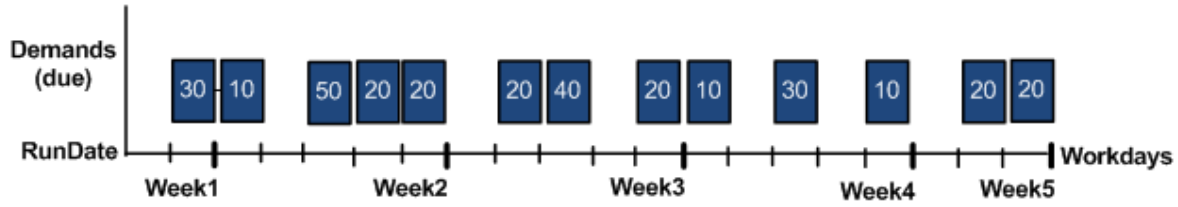
Finally assume the same settings and use of the "LeadTime" rule, but with the part having a lead time of 6 days instead. In this case, the safety stock levels would be generated as shown in the following illustration. Note that the first safety level is generated at lead time and only based on two days of demand, with the subsequent safety stock levels identical to what is shown in the previous image.

Figure 14.30 SafetyStockDateRule set to "LeadTime" (lead time of 6 days)



Example 2- Weekly safety intervals

Suppose a part has the set of demands shown in the image below (assume RunDate is a Thursday).

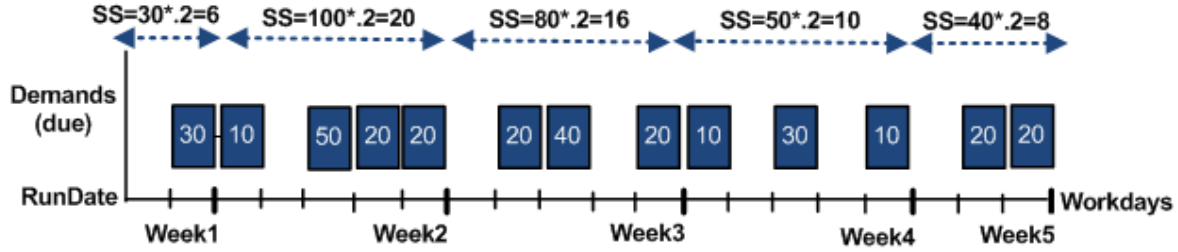


Further suppose the part has the following safety-stock related settings:

- **SafetyStockQuantityRule:** FractionOfDemand
- **PercentSafetyInterval:** Week
- **PercentSafetyIntervalCount:** 1
- **PercentSafetyPercent:** .2
- **SafetyStockDateRule**: RunDate
- **EffLeadTime**: 3

Thus safety stock would be set to 10% of the total demand found in each weekly interval. As determined by the **SafetyStockQuantityRule** setting, safety stock is then calculated starting from RunDate and recalculated at the start of every week after that. Refer to the following image.

Figure 14.31 SafetyStockDateRule set to "RunDate"



Instead assume the part has its **SafetyStockDateRule** setting changed to "LeadTime". With this setting, safety stock levels would be generated starting from the latter of the part's lead time or the date on the **PercentSafetyInterval** calendar that is **PercentSafetyIntervalCount** calendar markers after **RunDate**. Refer to the following illustration. Note that safety stock is no longer generated in Week 1, and the safety stock level in Week 2 no includes fewer demands to account for the part's lead time.

Figure 14.32 SafetyStockDateRule set to "LeadTime" (lead time of 3 days)

Finally assume the same settings and use of the "LeadTime" rule, but with the part's lead time reduced to 1 day instead. In this case, the safety stock levels would be generated as shown in the following illustration. Note that there is no safety stock level generated in Week 1, but all subsequent safety stock levels are generated on Week markers and provide identical results as were seen with the "RunDate" setting.

Figure 14.33 SafetyStockDateRule set to "LeadTime" (lead time of 1 day)

Time-phased safety days

When the **SafetyStockQuantityRule** is set to either "TimePhasedSafetyDaysTotal" or "TimePhasedSafetyDaysMaximum", a part's safety stock level is recalculated daily using parameters defined in the **TimePhasedSafety** table.

On a given day, the part's effective **TimePhasedSafety** record is determined based on the **Date** field (using the closest record where the **Date** value is on or before the day for which safety stock is being calculated).

The **Quantity** and **Days** fields on the effective **TimePhasedSafety** record then specify the inputs used in calculating the safety stock level as follows.

- **Quantity—** a fixed quantity that is always used in the days safety stock calculation.
- **Days**— a number of **SafetyStockPolicy.SafetyDaysCalendar** intervals over which the part's demands are collected and summed; the summed total is then used in the safety stock calculation (if the **SafetyDaysCalendar** reference is Null, the Everyday calendar is used). If a fractional value is specified in this field, the fractional amount indicates the percentage of demand to use from the last day being considered (for example, a value of 2.4 means to take the next two days of demand, plus forty percent of the demand in the third day).

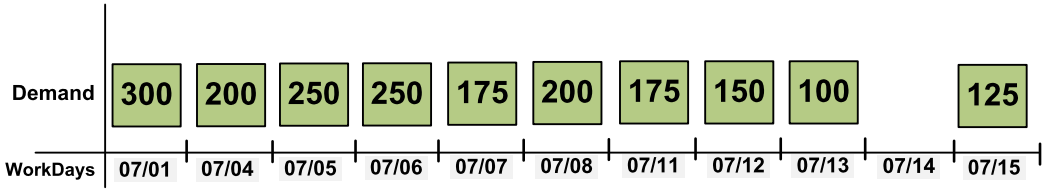
Note: If a negative value is specified in either the Days or Quantity field, then the previous (non-negative) Days or Quantity value defined on a record for the part is used. For example, if the Quantity value needs to change over a series of time-phased records but the Days value should remain constant, then a value of -1 can be specified in the Days field for all records meant to inherit the previous value.

Safety stock calculations then continue throughout the horizon with new levels generated daily (excluding days where the calculation returns the same value as the previous day). Safety stock recommendations are never made after the date of the last demand for the part. If safety stock should not be maintained after a specific date regardless of the demand picture, then a **TimePhasedSafety** record with **Quantity** and **Days** both set to "0" for that date should be added. For more information, see the **SafetyStockQuantityRule** field in the "SafetyStockPolicy table" on page 874.

Example

Suppose RunDate is 07-01-16 and part CPU4 has demands as shown in the following illustration:

Figure 14.34 Demands for CPU4



Further assume the following values in the **TimePhasedSafety** table:

Table 14.48: TimePhasedSafety values

Part	Date	Days	Quantity
CPU4	07-01-16	3	500
CPU4	07-11-16	2	300

If the "TimePhasedSafetyDaysTotal" setting is used, then the following safety stock levels are calculated for CPU4:

Table 14.49: TimePhasedDaysTotal safety stock calculation

Date	Safety stock	Calculated As...
07-01-16	1200	500+200+250+250
07-04-16	1175	500+250+250+175
07-05-16	1125	500+250+175+200
07-06-16	1050	500+175+200+175
07-07-16	1025	500+200+175+150
07-08-16	925	500+175+150+100
07-11-16	550	300+150+100
07-12-16	400	300+100+0
07-13-16	425	300+0+125
07-15-15	300	300

Instead assume the same demands and settings as shown previously except using the "TimePhasedSafetyDaysMaximum" setting and with **TimePhasedSafety.Date** on the second record set to 07-09-16 (instead of 07-11-16). In this case, the following safety stock levels are calculated for CPU4.

Table 14.50: TimePhasedSafetyDaysMaximum safety stock calculation

Date	Safety stock	Calculated As...
07-01-16	700	MAX (500, (200+250+250))
07-04-16	675	MAX (500, (250+250+175))
07-05-16	625	MAX (500, (250+175+200))
07-06-16	550	MAX (500, (175+200+175))
07-07-16	525	MAX (500, (200+175+150))
07-08-16	500	MAX (500, (175+150+100)
07-09-16	325	MAX (300, (175+150))
07-11-16	300	MAX (300(150+100))

Note: Safety stock is calculated every day, including non-work days. However, in the examples above because there are no demands on those non-work days, the calculated safety stock levels on 07-02-16, 07-03-16, and 07-10-11 give the same safety stock level as the previous work day.

RangeOfCoverage calculations

Range of coverage safety stock calculations apply to parts where the effective **SafetyStockQuantityRule**, from the either the **SafetyStockPolicy** table or the **PartType** table, is set to one of "RangeOfCoverage", "RangeOfCoverageLeadTime" or "TimePhasedRangeOfCoverage".

For more information about configuring these settings, see "RangeOfCoverage and RangeOfCoverageLeadTime" on page 1410 or "TimePhasedRangeOfCoverage" on page 1412.

RangeOfCoverage and RangeOfCoverageLeadTime

If the effective **SafetyStockQuantityRule** is set to "RangeOfCoverage", then safety stock levels are calculated to satisfy a specified number of days of the average daily demand for the part in three distinct date zones.

However, if the effective **SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime", then safety stock levels are calculated to satisfy the part's average daily requirement over its lead time.

Range of coverage safety stock calculations rely on profiles, or groups of control settings, defined in the **RangeOfCoverage** table. The range of coverage profile used by a given part is set by the reference in **Part.SafetyStockPolicy.RangeOfCoverage**, and range of coverage calculations include the following basic elements:

1. Define the range of coverage interval— A range of coverage interval is defined in the **RangeOfCoverage.RangeOfCoverageInterval** field. This field defines the calendar containing the dates when safety stock levels can be re-calculated. For example, if this field is set a Monthly calendar, then safety stock levels can be re-calculated each month. The calendar referenced in this field is also used to define range of coverage zones.
2. **Define range of coverage zones**—If **SafetyStockPolicy.SafetyStockQuantityRule** is set to "RangeOfCoverage", then three separate date zones can be specified, each with its own range of coverage settings. However, if **SafetyStockPolicy.SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime", then only the first zone is active.

   Range of coverage zones are defined by specifying the number of range of coverage intervals that comprise each zone (where **RangeOfCoverage.FirstZoneIntervalCount** specifies the length of the first zone, **RangeOfCoverage.SecondZoneIntervalCount** specifies the length of the second zone, and everything else is assumed to be in the third zone). For example, the first zone might comprise the first 3 months, the second the next 5 months, and the third all remaining months. A value of -1 can also be specified in the **RangeOfCoverage.FirstZoneIntervalCount** field to indicate that this zone extends to the end of the planning horizon.
   The three range of coverage date zones are calculated as follows:

   - **First Zone**—FROM `Part.PlanningCalendars.RunDate.FirstDate + 0 RangeOfCoverageInterval` TO `(Part.PlanningCalendars.RunDate.FirstDate + RangeOfCoverage.FirstIntervalCount RangeOfCoverage.RangeOfCoverageInterval) - 1`
   - **Second Zone**—FROM `Part.PlanningCalendars.RunDate.FirstDate + RangeOfCoverage.FirstZoneIntervalCount RangeOfCoverage.RangeOfCoverageInterval` TO `(Part.PlanningCalendars.RunDate.FirstDate + (RangeOfCoverage.FirstZoneIntervalCount + RangeOfCoverage.SecondZoneIntervalCount) RangeOfCoverage.RangeOfCoverageInterval) - 1`
   - **Third Zone**—FROM `Part.PlanningCalendars.RunDate.FirstDate + (RangeOfCoverage.FirstZoneIntervalCount + RangeOfCoverage.SecondZoneIntervalCount) RangeOfCoverage.RangeOfInterval` ONWARDS
3. **Calculate average daily demand**—Average daily demand for a part is calculated by summing total effective demand for the part across a given number of intervals. The number of intervals to consider demand from is set by the value in

RangeOfCoverage.AverageRequirementIntervalCount and the calendar defining these intervals is set by the value in **RangeOfCoverage.AverageRequirementInterval**. For example, assuming a monthly range of coverage interval, if **RangeOfCoverage.AverageRequirement Interval** is a "Weekly" calendar and **RangeOfCoverage.AverageRequirementIntervalCount** is "10" then the average daily demand calculation includes demands due from the start of each month until 10 weeks after the start of each month. If necessary, these calculations can be offset a number of time units from the start of the range of coverage interval by specifying a value in **RangeOfCoverage.AverageRequirementIntervalOffset**.

Once demand has been summed across the specified number of intervals, then average daily demand is calculated by dividing the total demand by the number of days in the intervals (as determined by the value in **RangeOfCoverage.AverageRequirementIntervalDurationType**).

Note: If **SafetyStockPolicy.SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime", then the duration over which average daily demand is calculated can be extended by some factor of a part's lead time. This is done by providing a value in the **RangeOfCoverage.AverageRequirementLeadTimeCount** field which is then multiplied by the number of AverageRequirementIntervals that represent the longest lead time for the part. The result of that calculation is then added to the value in **AverageRequirementIntervalCount**.

4. Determine safety stock level—If **SafetyStockPolicy.SafetyStockQuantityRule** is set to "RangeOfCoverage" then the safety stock level for each zone is calculated by multiplying the average daily demand by the number of days of demand that safety stock is meant to cover in that zone (as set by the values in FirstZoneTarget, SecondZoneTarget, and ThirdZoneTarget, respectively). For example, if the calculated average daily demand is 500 units for a RangeOfCoverage interval in the first zone and FirstZoneTarget is set to "4", then the safety stock level for that RangeOfCoverageInterval is set to 2000.

   However, if **SafetyStockPolicy.SafetyStockQuantityRule** is set to "RangeOfCoverageLeadTime", then the safety stock level is calculated by multiplying the average daily demand by the part's lead time. For example, if the calculated average daily demand is 500 units and lead time is 3 days, then the safety stock level is set to 1500.

   Once the safety stock level has been calculated, an optional buffer quantity is added to it if a value is found in **Part.RangeOfCoverageBuffer**. This field is used to account for some expected demand variability.

5. **Apply optional threshold check—**To avoid making minor changes to the safety stock level that might have only insignificant benefits, a check can be applied to the calculated safety stock levels in each range of coverage interval. Using the value in **Part.RangeOfCoverageBuffer** together with the **RangeOfCoverage.ThresholdFactorUpper** and **RaageOfCoverage.ThresholdFactorLower,** upper and lower limits are calculated to determine if the new calculated safety stock level should replace the one from the previous range of coverage interval. The upper and lower limits are calculated as follows:

   - **UpperLimit** = `CalculatedSafetyStockLevel + (Part.RangeOfCoverageBuffer * RangeOfCoverage.ThresholdFactorUpper)`
   - **LowerLimit** = `CalculatedSafetyStockLevel -(Part.RangeOfCoverageBuffer * RangeOfCoverage.ThresholdFactorLower)`

   If the existing safety stock level (from the previous **RangeOfCoverageInterval**) is between these limits, then the new calculated level is discarded and the existing safety stock level remains effective. This is intended to avoid making minor changes to the safety stock level which may have insignificant benefit. If, however, the existing safety stock level falls outside these limits, then the new calculated safety stock level become effective.

Notes:

- By default, all range of coverage parameters for a given part are stored in the **RangeOfCoverage** table (accessed through the part's **SafetyStockPolicy.RangeOfCoverage** reference). However, if required, certain range of coverage parameters can be overridden on a part by providing records in the **RangeOfCoveragePartOverride** table. For more information about the **RangeOfCoverage** table, see "RangeOfCoverage table" on page 855, or for more information about the **RangeOfCoveragePartOverride** table, see "RangeOfCoveragePartOverride table" on page 509 .
- For parts using range of coverage calculations, an additional safety stock demand with quantity given by **Part.SafetyStockQty** is generated on the date indicated by **Part.SafetyStockPolicy.SafetyStockDateRule**. To avoid having this additional demand generated, ensure the value of **Part.SafetyStockQty** is set to 0.

TimePhasedRangeOfCoverage

If the effective **SafetyStockQuantityRule** is "TimePhasedRangeOfCoverage", then safety stock levels are calculated to satisfy a specified number of days of the part's average daily demand. The number of days of demand that safety stock should cover can be set within any number of definable date-based zones.

Time-phased range of coverage safety stock calculations rely on records defined in the **TimePhasedSafety** table, as well as profiles (groups of related settings) defined in the **RangeOfCoverage** table. The range of coverage profile used for a given part is set by the reference in **SafetyStockPolicy.RangeOfCoverage** (if the **SafetyStockPolicy** reference is "Null", then the **PlanningCalendars.RangeOfCoverage** reference is used instead).

Time-phased range of coverage calculations for a part include the following basic steps.

1. **Determine range of coverage zones**—Time-phased range of coverage zones are defined using the **TimePhasedSafety** table. Each record in this table contains a **Date** field indicating the date on which a range of coverage zone should start, as well as a **Days** field indicating the number of days of the average daily requirement that all calculated safety stock levels within that zone are meant to cover.
2. **Determine dates on which safety stock is calculated**— Within each range of coverage zone, the actual dates on which new safety stock levels can be calculated are those belonging to the **RangeOfCoverage.RangeOfCoverageInterval** calendar. For example, if this field references a Monthly calendar, then new safety stock levels can be recommended at the beginning of each month.

   Note also that if the **Date** value on a **TimePhasedSafety** record does not fall on **RangeOfCoverageInterval** calendar markers, then they are considered effective as of the beginning of the current interval. For example, if using a "Monthly" **RangeOfCoverageInterval** calendar, and a **TimePhasedSafety** record has a **Date** of 12-20 (December 20), then it would be considered effective and used to define a range of coverage zone beginning on the first working day in December. Additionally, if there are multiple **TimePhasedSafety** records whose **Date** resolves to the same **RangeOfCoverageInterval** marker, then the latest of those records is used.
3. **Calculate average daily demand**— The average daily demand for a part is calculated by summing total effective demand for the part across a specified number of calendar intervals. The number of intervals over which to consider demand is set by the value in **RangeOfCoverage.AverageRequirementIntervalCount** and the **RangeOfCoverage.AverageRequirementInterval** reference defines the calendar in which those

intervals are expressed. For example, assuming a Monthly range of coverage interval, if **RangeOfCoverage.AverageRequirement Interval** is set to "Week" and **RangeOfCoverage.AverageRequirementIntervalCount** is "8", then the average daily demand calculation includes demands due from the start of each month and over each of the next 8 weeks.

Once total effective demand has been summed across the specified number of intervals, the average daily demand is calculated by dividing that demand over the number of working days in the intervals from which it was collected (as determined by the **RangeOfCoverage.AverageRequirementIntervalDurationType** setting).

4. **Calculate safety stock level**—In order to calculate the safety stock level for a given date within a range of coverage zone, the average daily demand is multiplied by the number of days demand that safety stock is meant to cover within that zone (as taken from the **Days** field on the effective **TimePhasedSafety** record).
   Additionally, if the optional **Part.RangeOfCoverageBuffer** field is populated, then it is added to the safety stock value (typically, this done to provide a buffer against some expected demand variability).
5. **Apply threshold check**—To avoid making minor changes to the safety stock level that might have only insignificant benefits, an optional check can be applied to the calculated safety stock levels in each range of coverage interval. This involves calculating upper and lower threshold limits based on the newly calculated safety stock level and other fields as shown below.
   - **UpperLimit** = `CalculatedSafetyStockLevel + (Part.RangeOfCoverageBuffer * RangeOfCoverage.ThresholdFactorUpper)`
   - **LowerLimit** = `CalculatedSafetyStockLevel -(Part.RangeOfCoverageBuffer * RangeOfCoverage.ThresholdFactorLower)`

   Then, as long as the existing safety stock level (from the previous **RangeOfCoverageInterval**) falls outside these limits, the new calculated safety stock level becomes effective. Otherwise, the new calculated level is discarded and the existing safety stock level remains effective.

Note: TimePhasedRangeOfCoverage does not calculate dynamic safety stock for the last period of demand if there is no future demand. For example, it is expected for RapidResponse to recalculate the safety stock level in April 2016 and drop it to zero. Inserting a new TimePhasedSafety record for 08-01-0216 doesn't impact the calculation. Changing the date on the new record to 04-01-2016 when the last demand is due, drops the SS level to zero, as expected.

Example: TimePhasedRangeOfCoverage

Suppose a part Gamer with an effective **SafetyStockQuantityRule** setting of "TimePhasedRangeOfCoverage" and the following **RangeOfCoverage** settings:

- **RangeOfCoverageInterval**— Week
- **AverageRequirementInterval**— Week
- **AverageRequirementIntervalCount**— 2
- **AverageRequirementIntervalDurationType**— TimeUnits (workday)

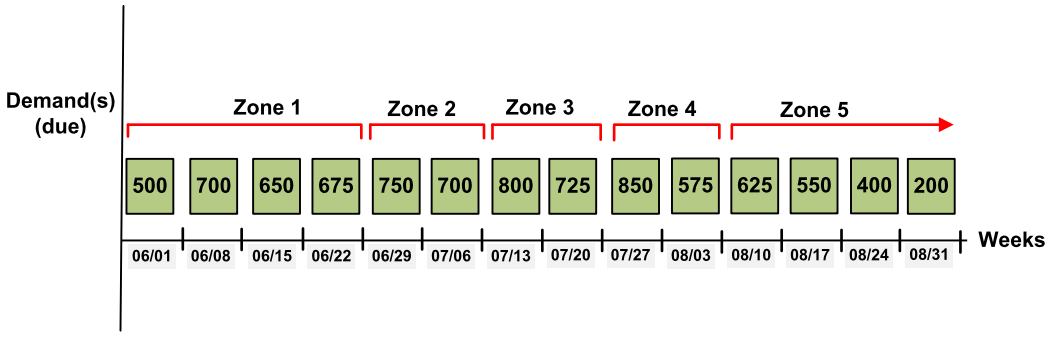
Further, suppose a set of **TimePhasedSafety** records used to define range of coverage zones for the Gamer part as shown below:

Table 14.51: TimePhasedSafety records

Part	Date	Days
Gamer	06-01-15	3
Gamer	07-01-15	2.5
Gamer	07-15-15	2
Gamer	08-01-15	1
Gamer	08-10-15	0

Finally, assume weekly demand quantities for the part as shown in the following illustration. In this case, each date shown is assumed to belong to the "Week" calendar and marks the beginning of a week containing five (5) working days. As such, **Date** values on **TimePhasedSafety** record that do not belong to the "Week" calendar are adjusted to the current "Week" marker where they define the beginning of range of coverage zones as shown below.

Figure 14.35 Demands used in calculating average daily demand



Thus, safety stock quantities for the part would be calculated as shown in "Calculated safety stock quantities for RangeOfCoverageTimePhased" on page 1414 below. Note that safety stock quantities are calculated weekly, and each is meant to cover a number of days of the part's average daily demand as defined on the effective **TimePhasedSafety** record, with the average daily demand being determined from demands summed over each 2 week (10 day) period.

Table 14.52: Calculated safety stock quantities for RangeOfCoverageTimePhased

Date	Average Daily Demand	ADD Calculation	Safety Stock Quantity	SSQ Calculation
06-01-15	120	(500 + 700) /10	360	120 * 3
06-08-15	135	(700 + 650) /10	405	135 * 3
06-15-15	132.5	(650 + 675) /10	397.5	132.5 * 3
06-22-15	142.5	(675 + 750) /10	427.5	142.5 * 3
06-29-15	145.00	(750 + 700) /10	362.5	145 * 2.5

Table 14.52: Calculated safety stock quantities for RangeOfCoverageTimePhased

Date	Average Daily Demand	ADD Calculation	Safety Stock Quantity	SSQ Calculation
07-06-15	150	(700 + 800) /10	375	150 * 2.5
07-13-15	152.5	(800 + 725) /10	305	152.5 * 2
07-20-15		(725 + 850) /10	315	157.5 * 2
07-27-15	142.5	(850 + 575) /10	142.5	142.5 * 1
08-03-15	120	(575 + 625) /10	120	120 *1
08-10-15	1175	(625 +550) /10	0	117.5 * 0

Note: The results shown above assume that the **RangeOfCoverageBuffer**, **ThresholdFactorUpper**, and **ThresholdFactorLower** fields are all set to "0".

Demand usage in dynamic safety stock calculations

Parts configured for dynamic safety stock will, by default, have all types of demands considered when calculating their safety stock levels. However, the **DemandType.SafetyStockDemandUsage** and **SafetyStockPolicy.DependentDemandUsage** settings can be used to exclude certain types of demands from use in dynamic safety stock calculations.

For example, it might a requirement to exclude certain irregular demand types from influencing safety stock levels, or there might be a need to include demands from make sources but exclude demands from transfer sources.

SafetyStockDemandUsage

The **DemandType.SafetyStockDemandUsage** field applies to demands for parts that use any **SafetyStockQuantityRule** other than "FixedQuantity" or "TimePhasedQuantity". For independent demands, the applicable setting can be determined from the **Order.Type** reference, and for dependent demands the applicable setting can be determined from the **Type.AllocationDemandType** reference.

The **SafetyStockDemandUsage** field has the following two options:

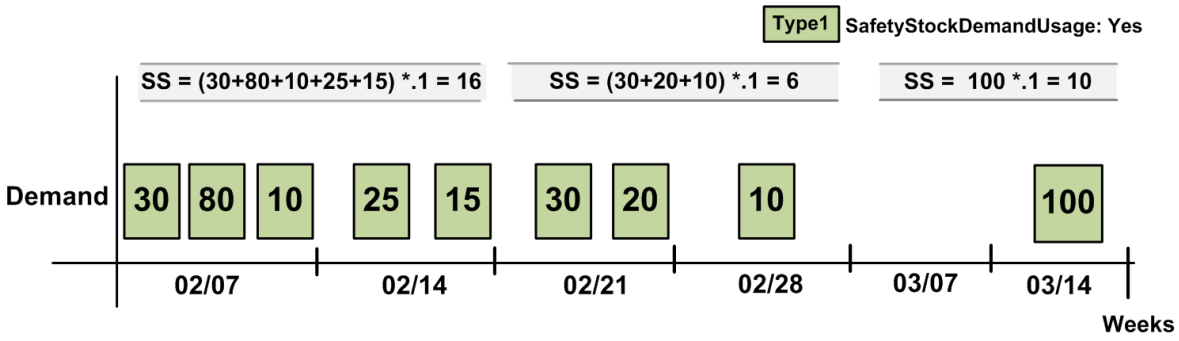
- **Use**-include demands that reference this setting in dynamic safety stock calculations. This is the default setting
- **Ignore**--exclude demands that reference this setting from dynamic safety stock calculations.

Example

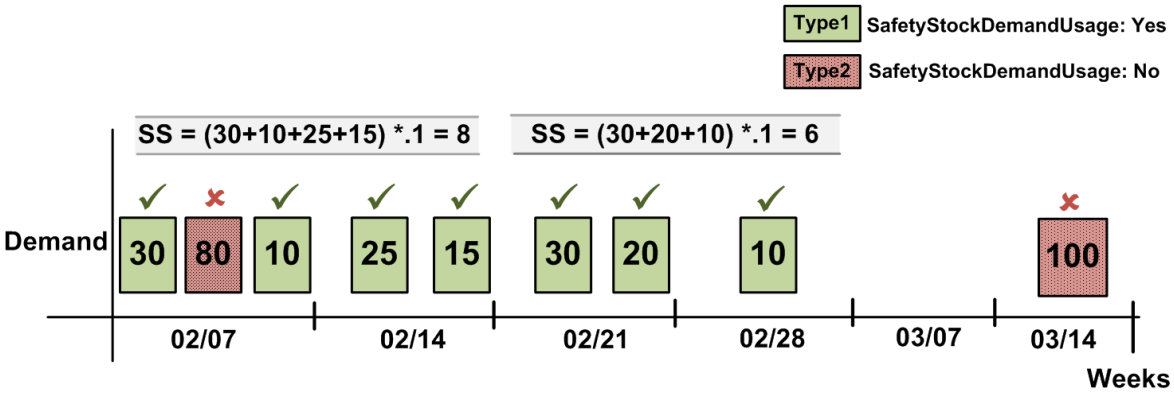
Suppose a part has the following safety stock settings:

- **SafetyStockPolicy.SafetyStockQuantityRule:** FractionOfDemand
- **SafetyStockPolicy.PercentSafetyInterval:** Week
- **PercentSafetyIntervalCount:** 2

If all demands for the part referenced a **SafetyStockDemandUsage** setting of "Yes", then safety stock levels might be calculated as shown in the following illustration.



Instead, if some of those same demands referenced a **SafetyStockDemandUsage** setting of "No", then safety stock levels would be calculated as shown in the following illustration.



DependentDemandUsage

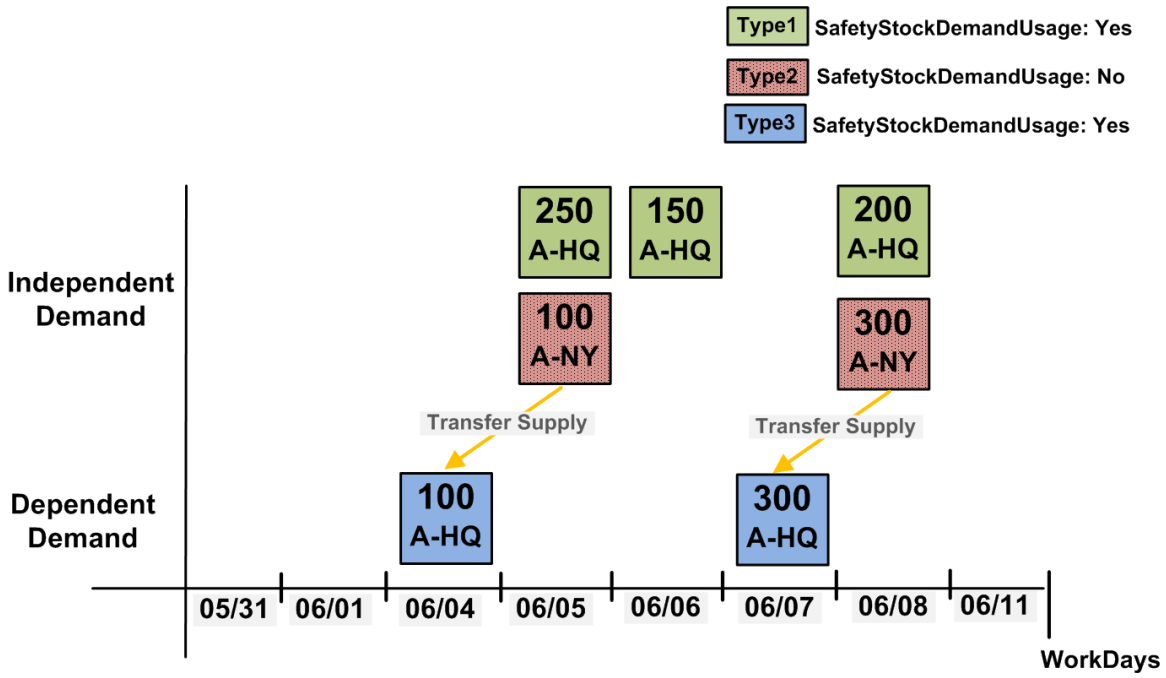
For just those parts that use a **SafetyStockQuantityRule** of either "TimePhasedSafetyDaysMaximum" or "TimePhasedSafetyDaysTotal", the **SafetyStockPolicy.DependentDemandUsage** setting can be used to exclude specific types of dependent demands from impacting the calculation of safety stock levels. Combined with the **SafetyStockDemandUsage** setting, this field then determines whether specific dependent demands for a part are used in dynamic safety stock logic.

The **DependentDemandUsage** field has the following four options:

- **All**-all of the part's dependent demands can be used in safety stock calculations (as long as those demands reference a **SafetyStockDemandUsage** setting of "Use") . This is the default option.
- **MakeOnly**-only dependent demands from "make" sources can be used in safety stock calculations (as long as those demands reference a **SafetyStockDemandUsage** setting of "Use").
- **TransferOnly**-only dependent demands from "transfer" sources can be used in safety stock logic (as long as those demands reference a **SafetyStockDemandUsage** setting of "Use").
- **None**-dependent demands for the part are never used in safety stock calculations (regardless of the **SafetyStockDemandUsage** settings applicable to its demands).

Example

Suppose Part A at the HQ site has independent demands placed directly on it as well as well as demand transferred from the NY site with each of those demands referencing one of the three **SafetyStockDemandUsage** settings shown in the following illustration (this example assumes 1 day of transit time between HQ and NY).



Further suppose Part A at HQ referenced a **SafetyStockPolicy.SafetyStockQuantityRule** setting of "TimePhasedDaysTotal", and the following values were defined on a record in the **TimePhasedSafety** table:

Part	Site	Date	Quantity	Days
A	HQ	06-01	25	3

If the referenced **SafetyStockPolicy.DependentDemandUsage** value was set to "All" or "Transfer", then safety stock levels would be calculated as in the following table:

Table 14.53: TimePhasedDaysTotal safety stock calculation for A-HQ

Date	Safety stock	Calculated As...
06-01-18	525	25+250+150
06-04-18	725	25+250+150+300
06-05-18	675	25+150+300+200
06-06-18	525	25+300+200
06-07-18	225	25+200
06-08-19	25	25

Instead if the referenced **SafetyStockPolicy.DependentDemandUsage** value was changed to "None" or "Make", then safety stock levels would be calculated as in the following table:

Table 14.54: TimePhasedDaysTotal safety stock calculation for A-HQ

Date	Safety stock	Calculated As...
06-01-18	425	25+250+150
06-05-18	375	25+150+200
06-06-18	225	25+200
06-08-19	25	25

Note that in this example, the same changed results shown in the previous table could also have be achieved by changing the **SafetyStockDemandUsage** setting for Type2 demands to "No".

Priority and safety stock

By default, when **SafetyStockPolicy.DemandTransformationRule** is set to "Ignore", the priority on orders generated to satisfy safety stock requirements is set to the part's default priority as determined by the **PartType.DefaultPriority** reference. Otherwise, when **SafetyStockPolicy.DemandTransformationRule** is set "FirstInFirstOut", the priority on orders generated to satisfy safety stock is set to the priority of the real demand it satisfies.

Optionally, when **DemandTransformationRule** is "Ignore" and for time-phased safety stock only, the priority on a given safety stock requirement can be explicitly set using the **TimePhasedSafety.OrderPriority** reference. This can then be the priority set on planned orders generated to satisfy the safety stock requirement (dependent on the parts **PartType.CommitLevel** setting). Otherwise, if the **OrderPriority** reference is left Null, then the default priority from the part type is used as the planned priority.

For more information on how priority is assigned to supply generated by safety stock requirements, see "Safety stock and priority adjustments" on page 1773. For more information on how priority on safety stock is used when allotting supply to demand, see "Priority and safety stock holds in CTP" on page 1780.

Analytic limitations with safety stock

The following table outlines other analytic calculations which might not work as expected when configured for parts using certain safety stock policies.

Table 14.55: Safety stock limitations with other analytics

Analytic / Calculation	Impact on safety stock
Model/Unit	Maintenance of safety stock for parts netted by model and/or unit depends on the **MUEPoolNettingType.SafetyStockRule** setting as follows: • **Ignore**—safety stock is not planned for these parts. • **Common**—fixed or time-phased safety stock is maintained using the default model and/or unit, but safety stock is not planned if the part uses a "dynamic" safety stock policy (for example, "PercentOfDemand" or "RangeOfCoverage"). • **ByPool**—safety stock is planned using the default model and/or unit (all safety stock policies are supported).
Inventory pooling	Maintenance of safety stock for parts netted by pool depends on the **MUEPoolNettingType.SafetyStockRule** setting as follows: • **Ignore**—safety stock is not planned for these parts. • **Common**—fixed or time-phased safety stock is maintained in the common pool, but safety stock is not planned if the part uses a "dynamic" safety stock policy (for example, "FractionOfDemand" or "RangeOfCoverage"). Also, records in the **TimePhasedSafety** table referencing a pool other than Common pool are ignored. • **ByPool**—safety stock is planned in the applicable pool (all safety stock policies are supported).
Global part substitution	Safety stock requirements on a primary part can be satisfied by inventory or scheduled supply of its substitute parts. However, if planned orders are required to satisfy safety stock requirements they can only be created on the primary part (not on any of its substitutes). Additionally dynamic safety stock logic is not fully supported on parts configured as global substitutes.
BOM-level part substitution	Safety stock requirements on substitutable components can only be satisfied by their supply, and safety stock requirements are not used in the calculation of a substitute component's excess supply. Additionally dynamic safety stock logic is not fully supported on components configured as bom-level substitutes.
Attribute-based planning	Attribute-based requirements are only supported on safety stock demands from certain methods set using the **TimePhasedSafety** table (against which attributes can be defined). If safety stock is required with attribute-based planning, then the **SafetyStockQuantityRule** should be set to one of "TimePhasedSafety", "TimePhasedSafetyToLatest", "TimePhasedSafetyRangeOfCoverage", "TimePhasedRangeOfCoverage". Attribute-based requirements are not supported on safety stock with any **SafetyStockQuantityRule** other than the four mentioned above (none of "Fixed", "FractionOfDemand", "PercentOfDemand", "RangeOfCoverage", "RangeOfCoverageLeadTime", and "TimePhasedRangeOfCoverage" are supported).
Feature BOM	Safety stock is never planned for configurable end-items (it can, however, be planned for components under a configurable end-item).
Multi-level search	Component parts that have multi-level search logic enabled should not use dynamic safety stock policies. The only **SafetyStockQuantityRule** options supported for components using multi-level search are "FixedQuantity", "TimePhasedQuantity", and "TimePhasedQuantityToLatest". However, all safety stock policies (including dynamic policies such as "FractionOfDemand", "RangeOfCoverage", and so on) are supported for top-level assembly parts that have multi-level search logic enabled.

Ignoring unsatisfied demands

RapidResponse analytic calculations can be optionally configured to ignore the unsatisfied portions of a specified demand type within a given period. For example, when there is limited supply within a given period, it might be acceptable to ignore any unsatisfied forecast demand, but a requirement to always process demand from actual customer orders.

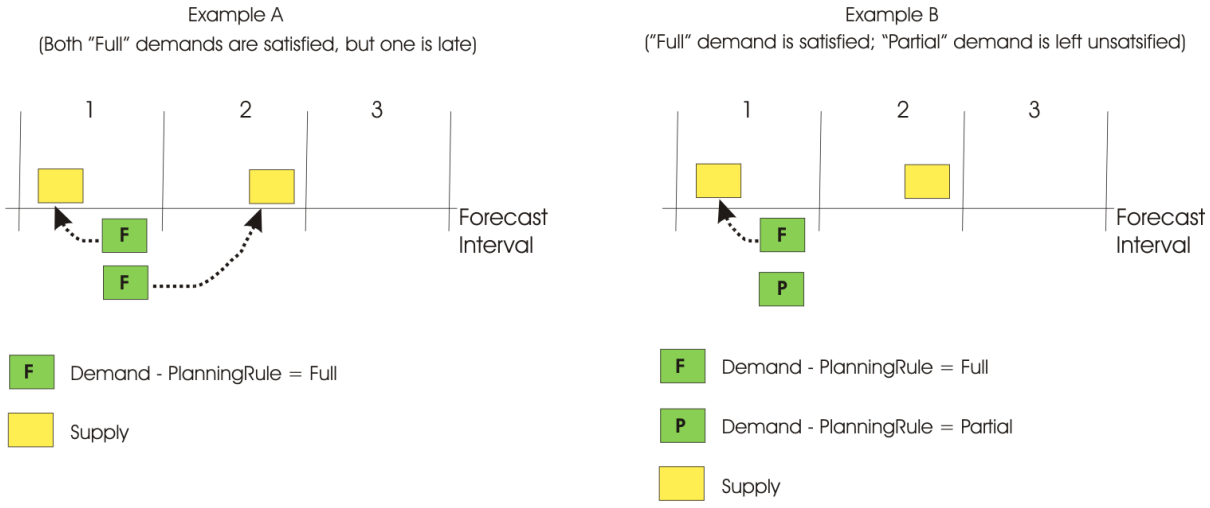
The processing of unsatisfied demands by RapidResponse is controlled by the **DemandType.PlanningRule** field which has two options "Full" and "Partial." ("Full" is the default setting.) This control setting applies to demand records in the following tables:

- IndependentDemand (both forecast and sales actuals)
- ConsensusForecast
- Allocation
- PlannedTransferAllocation
- NewTransferAllocation

When the **PlanningRule** is set to "Full" for a particular demand type, its demands are required to be fully satisfied and Netting calculations will assign or plan supplies for the demand as required. When there is limited supply within a period, this means the demand can not only consume supplies in the current forecast interval or earlier, but can also look ahead to future intervals to find required supply.

Additionally, assuming the same demand due date and priority, demands with a **PlanningRule** of "Full" take precedence over demands with a PlanningRule of Partial. These cases are illustrated in the following figure. In Example A, there are two "Full" demands due in Interval 1, but only enough supply to satisfy one of them, so the other demand looks ahead until it finds available supply which occurs in Interval 2. In Example B however, there is one "Full" demand due in Interval 1 as well as one "Partial" demand due at the same time. In this case, the "Full" demand is satisfied first and uses the lone supply in Interval 1. This leaves no available supply on-time for the "Partial" demand (note that depending on configuration, it may or may not be able to look ahead to future intervals for supply).

Figure 14.36 "Full" PlanningRule setting



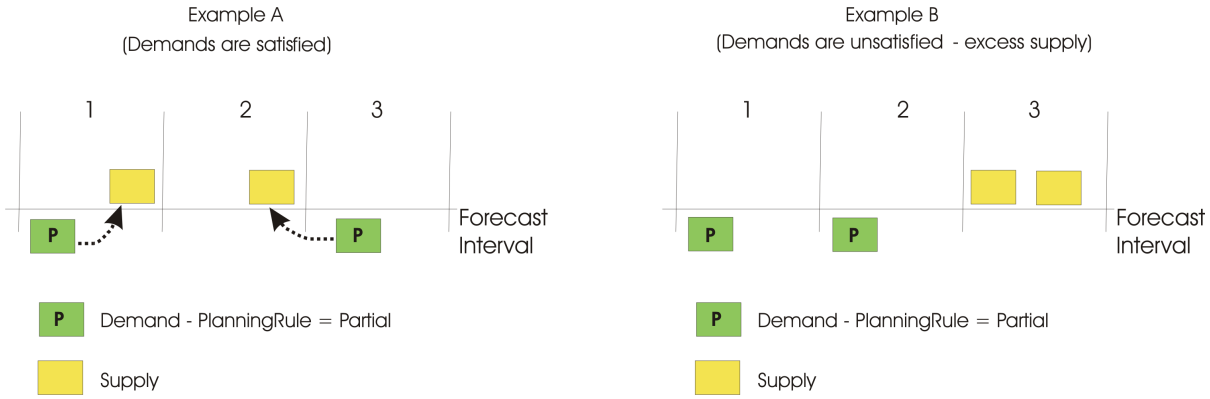
When the **PlanningRule** is set to "Partial," the **UnsatisfiedDemandTolerance** field on the **Part** table defines the number of forecast intervals that demands of this type can be late before being ignored and left unsatisfied (Netting will not assign or plan supplies for the demand). For example, in cases where

the decision to ignore demand needs to be based on a lower level component, "Partial" demands can use a demand tolerance to indicate how many intervals in the future that the demand can consume supply from.

When the **UnsatisfiedDemandTolerance** field is set to "0" (the default value), the part's demands can only consume supplies due within the same forecast interval or earlier. For example, a given scheduled receipt could only be used to satisfy the demand if its ReschedDate is in the same forecast interval as the demand's due date or earlier. Similarly, a planned order can only be created to satisfy the demand if the minimum order due date for the part source is in the same forecast interval or earlier.

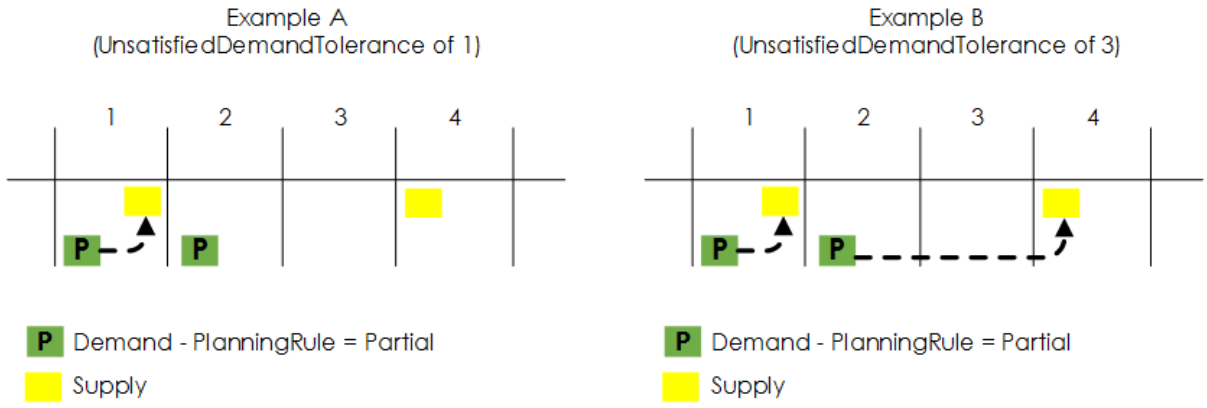
For example, consider the examples shown in the following figure, which assume the part's **UnsatisfiedDemandTolerance** is set to "0". In Example A, both demands are satisfied as they are able to find supply within their respective forecast intervals or earlier (though the demand in Period 1 will be satisfied late). However, in Example B, both demands are left unsatisfied as neither can find supply within its forecast interval or earlier. Note also in Example B that there is excess supply resulting as none of the demands are able to use the supplies due in Interval 3.

Figure 14.37 "Partial" PlanningRule setting



Instead, consider the examples shown in the following figure, where positive values are assumed in the part's **UnsatisfiedDemandTolerance** field. In Example A, the demand tolerance is one interval, so the demand in Interval 2 cannot be satisfied. However, in Example B, the demand tolerance is three, so the demands in Interval 1 and Interval 2 are satisfied.

Figure 14.38 Impact of UnsatisfiedDemandTolerance setting



Notes:

- Safety stock demands are not associated with a DemandType and are therefore not eligible for partial planning and will always be fully satisfied.
- Safety lead time is considered in partial planning logic, however if using partial planning then it is recommended to specify safety lead time at the part level (on the **Part** or **SafetyLeadTimeProfile** tables). Using partial planning logic along with safety lead time at the part-source level (**PartSource.SafetyLeadTime**) is not supported and might not always give the expected results.
- If component availability throughout a given product structure cannot be guaranteed (for example, due to constraints, lead time, or order policies on the part sources that supply components), then it is recommended to also enable multi-level search along with partial planning. This ensures that the decision to leave demand portions unsatisfied more accurately reflects the expected availability of a part's component supplies. See "Multi-level search logic" on page 1863 and "Partial planning" on page 1866

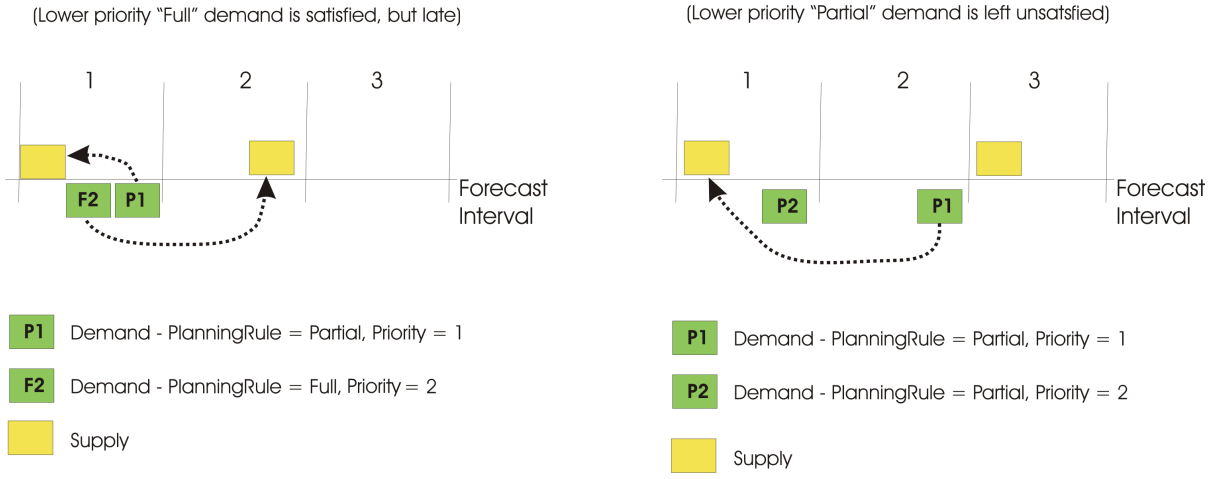
Impact of priority

For RapidResponse implementations that make use of the Priority analytic, the impacts of priority should be strongly considered when used with DemandType.PlanningRule settings. Because priority is the first sort criteria used when planning demand orders, it can significantly influence the expected handling of unsatisfied demands.

The following illustration shows two examples of how Priority settings might influence calculations involving unsatisfied demands. In Example A, the "Partial" demand in Interval 1 is satisfied first because it has the higher priority, forcing the earlier "Full" demand to look for and use supply from a later interval. If priority was not used in this case, the "Full" demand would have been satisfied first and been on time, and the "Partial" demand left unsatisfied. In Example B, the "Partial" demand in Interval 2 is satisfied first because it has the higher priority, and uses the supply in Interval 1 to ensure it is satisfied on time. As a result, the "Partial" demand in Interval 1 is left unsatisfied because the only remaining supply is in a future interval. If priority was not used in this case, the demand in Interval 1 would have been satisfied and on time, and demand in Interval 2 would have been satisfied but late.

For more information about priority, see "Order priority calculations" on page 1765.

Figure 14.39 Impact of priority on PlanningRule setting



Recursive data

Recursive data is often found in relation to records in certain input tables such as **BillOfMaterial** , **PartSource**, and **Allocation**. It should generally be avoided as RapidResponse analytics such as Netting does not plan completely when there are recursive data structures.

A recursive data structure refers to a situation wherein to calculate A, the results from B are needed; however, in order to get those results from B, the results of A must first be calculated.

For example, consider an effective **BillOfMaterial** record where A is the assembly and B is the component and there is another effective **BillOfMaterial** record where B is the assembly and A is the component. This is considered recursive, and RapidResponse can neither plan for A or B as each requires the planning results of the other in order to plan themselves.

Or consider two **PartSource** records; a make source for Part A at site NY, and a transfer source for Part A at site HQ (where HQ was the destination site and NY was the sourcing site). Neither of these records would be flagged as recursive. But If a third **PartSource** record was added and it was a transfer source for Part A at site NY (where NY was the destination site and HQ was the sourcing site), then these records would be now considered recursive as A and A1 would each require the planning results of the other in order to plan themselves

Recursion can happen in and between several input tables. For example, within the Mfg namespace it can occur in any input table that relates two **Part** records with a parent and child relationship. The following tables in RapidResponse can potentially contain recursive data:

- Allocation (Mfg)
- AlternatePart (Mfg)
- BillOfMaterial (Mfg)
- Constraint (Mfg)
- PartSource (Mfg)
- Predecessor (ProjMgmt)
- SequenceFlow (ProcOrch)
- SubProcess (ProcOrch)
- WBS (ProjMgmt)

In each of the tables listed above, there is a calculated Boolean field named **IsRecursive** that is set to Y when RapidResponse detects that this record is part of a recursive loop. Otherwise, the **IsRecursive** field on these tables returns a value of "N".

Notes:

- The effective date range for part relationships is not relevant in determining whether or not it is recursive. That is, if the A-B BOM record is effective from January to March and the B-A BOM record is effective from April to June, the relationship is still flagged as recursive.
- Recursive relationships are not confined to immediate parent and child relationships. For example, a recursive loop might be defined by a **BillOfMaterial** record with A as the assembly and B as the component, and then a second **BillOfMaterial** record with B as the assembly and C as the component, and then a **ScheduledReceipt** record for C with an associated **Allocation** record for component A.

Self-allocations

A self-allocation is a special form of input **Allocation** record where the part on the referenced **ScheduledReceipt** record is the same as the part the part being allocated on the **Allocation** record. In other words, the parent and child part in this situation resolve to the same part number.

As discussed in the previous section, this would typically indicate a "recursive" situation which cannot be properly planned in RapidResponse. However, this is one of the few recursive data conditions that are specifically allowed in RapidResponse. As such, self-allocation records are not marked as recursive (the **Allocation.IsRecursive** field returns a value of “N” for this type of recursive situation).

This is because RapidResponse considers a self-allocation as a legitimate demand, rather than as an instance of recursion. Thus, the AvailableDate field on the ScheduledReceipt record will not depend on the availability of the self-allocation. The self-allocation is allowed before any other demand and is treated as a negative on-hand quantity.

A self-allocation usually represents a rework order where some assembly is being returned for rework. It is intended that the rework order will consume itself via its self-allocation. Thus, in this type of scenario, the scheduled receipt represents the supply and the self-allocation represents the demand.

CHAPTER 15: Master Production Scheduling calculations

The Master Production Scheduling (MPS) analytic plans top level supplies to meet customer orders and forecast demand.

Master Production Scheduling (MPS) parts are defined in the **PartType** table by setting the **ProcessingRule** control field to "MPS" (or "MPSConfig" when dealing with planning BOMs). On these parts, forecast records in the **IndependentDemand** and/or **ConsensusForecast** tables are processed against actual demand to get an unconsumed forecast for demand to the netting calculation. In addition, AvailableToPromise (ATP) records are generated. ATP shows the date when supply, that is not needed to cover committed orders, is scheduled to be due. If the ATP commitments are not met, an excess or shortage can occur.

In this chapter, you'll learn about:

Demand types 1425
Forecast spreading 1426
Forecast consumption 1431
MPS configuration 1442
Dependent forecast on scheduled receipts 1446
Forecast consumption by pool/customer 1447
Available to promise 1449

Demand types

With forecasting and MPS logic, there are three general types of demand to consider; Regular, SalesActual and SalesForecast. They are (mostly) determined by the **DemandType.ProcessingRule** setting which determines how a given demand is processed by Netting logic. For example, on IndependentDemands, the **Order.Type.ProcessingRule** defines the type of demand and hence how it is processed.

The **DemandType.ProcessingRule** is usually set to one of the first three options listed in the table below. The last three options listed are less common and are either used ignore the demand completely or will logically resolve to one of the first three options.

Table 15.1: DemandType.ProcessingRule options

Value	Description
SalesForecast	Represents forecast demand and can be consumed by SalesActual demand in the same forecast consumption interval. The unconsumed portion of this demand is ignored prior to the demand time fence and generates demand after it.
SalesActual	Represents an actual customer order that was forecasted and, as such, it can consume SalesForecast demand. The amount of this demand that can consume SalesForecast is the **Quantity** plus the **ShippedQty** on the independent demand. Note that the ability to consume forecast can be overridden on individual demand lines.
Regular	This is actual demand that is not part of any forecast. It is always added to the total demand picture and cannot consume any SalesForecast demand.
Ignore	Demand that is to be ignored completely.
ProductionForecast	This is considered forecast demand (usually dependent demand from an MPSConfig part) but it is not eligible to be consumed by SalesActual. However, it still affects the ATP calculations as sales forecast (that is, supply allocated to these demands is considered "available to promise").
FromPart	This is only used for dependent demands that are exploded from a supply. (should only be used on **DemandType** records referenced by **the SupplyType.AllocationDemandType** field). This setting thereby allows for the component part's **PartType.DependentDemandForecastConsumption** setting to determine if the dependent demand is SalesActual (**DependentDemandForecastConsumption**=Y) or Regular (**DependentDemandForecastConsumption**=N). In other words, it allows the component part to determine if the exploded demand is supposed to be able to consume forecast or not.

Note: The general forecast spreading and consumption logic also applies to the consensus forecast as calculated and generated from the Sales and Operations Planning (S&OP) process in RapidResponse (if applicable). In other words, the consensus forecast can be thought of as functionally equivalent to independent demands of type "SalesForecast".

Forecast spreading

Forecast spreading is a planning technique that spreads forecast values over a period of time. For example, you can define an evenly spread forecast over a six month period that results in a master schedule that is evenly spread and less lumpy.

The following RapidResponse fields control forecast spreading:

- **DemandType.ProcessingRule**—forecast spreading is applicable only to demands where this control field is set to "SalesForecast".
  For independent demands, the **DemandType** table is referenced through the **DemandOrder** record. In other words, the **IndependentDemand** record must have **Order.Type.ProcessingRule** set to "SalesForecast". For consensus forecast demands, the **DemandType** table is referenced through the **PartCustomer** record. In other words, the applicable **PartCustomer** record must have **DemandType.ProcessingRule** set to "SalesForecast".
- **DemandType.SpreadRule**—this control field must be set to "Spread" for RapidResponse to consider the values in the **SpreadProfile** table.

- **DemandType.SpreadProfile**—this value must reference a valid **SpreadProfile** record. For information about the **SpreadProfile** table, see "SpreadProfile table" on page 899.
- **PartType.ProcessingRule**—this field must be either **MPS** or **MPSConfig** for RapidResponse to perform forecasting for the part.
- **PlanningCalendars.ForecastCalendar**—this calendar provides the date interval value is used as the forecast **from** calendar. For example, forecast values might be collected over a monthly period.
- **PlanningCalendars.SecondCalendar**—this calendar provides the date interval to be used as the forecast **to** calendar. For example, forecast values might be spread over workdays.
- **Part.SpreadForecastInterval**—an optional parameter that can be used to set the number of ForecastCalendar intervals over which forecast spreading logic is applied to eligible demands (spreading is only applicable to forecast demands due within this number of intervals out from **Part.PlanningCalendars.RunDate.FirstDate**). Otherwise, if this field is not used (set to 0), then forecast spreading logic is applied throughout the entire planning horizon.

Forecast spreading process

For each demand (independent or dependent) with a **DemandOrder.Type.ProcessingRule** control value of **SalesForecast**, a **Forecast** record is created for each **SecondCalendar** date in each ForecastCalendar interval containing the due date of the initial demand.

For each ForecastCalendar interval, the number of SecondCalendar dates in that interval are counted (the interval uses InclusiveExclusive rules for the first and last date). If there are no SecondCalendar dates in the interval (for example, the SecondCalendar interval is the same as the Forecast interval), the initial SalesForecast date and quantity are retained as the spread forecast.

If there is one SecondCalendar date in the interval, the spread forecast quantity is the SalesForecast quantity and the date is the SecondCalendar date. If there are more than one SecondCalendar dates in the interval, the SalesForecast quantity is divided among the SecondCalendar dates using an interpolation of the SpreadProfile points (these points are defined in the **SpreadProfile** table).

The following table shows how each field in the **Forecast** table is calculated.

Table 15.2: Calculation of the Forecast table

Field	Calculation
Part	The part this forecast record applies to.
Date	If (**DemandType.SpreadRule** = **Spread**) and there is at least one **SecondCalendar** entry in the ForecastCalendar interval containing the DueDate from the Independent or Dependent demand, the following occurs: • One record is generated for each SecondCalendar.Date in the forecast period. • If there are no SecondCalendar dates in the forecast period, Date=demand due date.
Quantity	If at least one **SecondCalendar** date in the ForecastCalendar interval: • Quantity is calculated using the spread profile calculation (see below). • Otherwise Quantity is assigned the **Quantity** value from the demand record (for example, the **Quantity** value from the **IndependentDemand** record).
UnconsumedQty	The portion of this Forecast.Quantity that remains after forecast has been consumed by SalesActual demands (for example, an **IndependentDemand** record with a value of **Order.Type.ProcessingRule** = **SalesActual**).

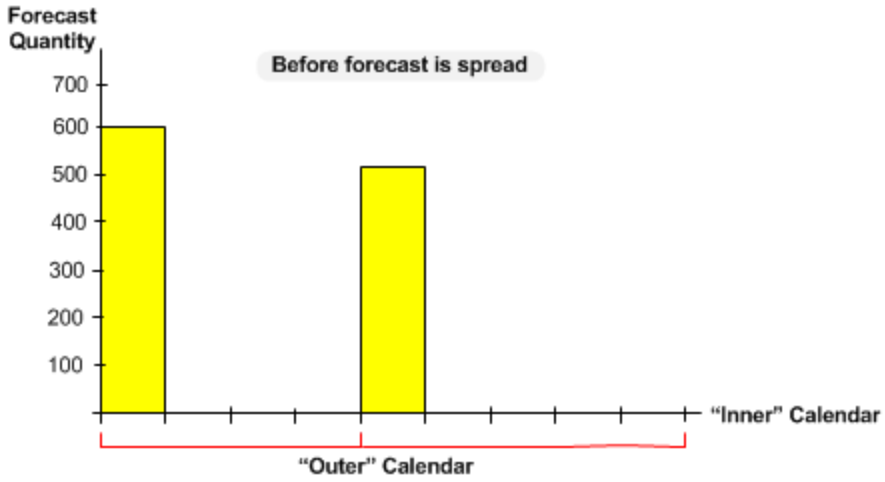
Table 15.2: Calculation of the Forecast table

Field	Calculation
Pool	The pool that this forecast applies to. This pool value is used by the netting calculation.
Model	The model that this forecast applies to. This model value is used by the netting calculation.

Example: Basic forecast spreading

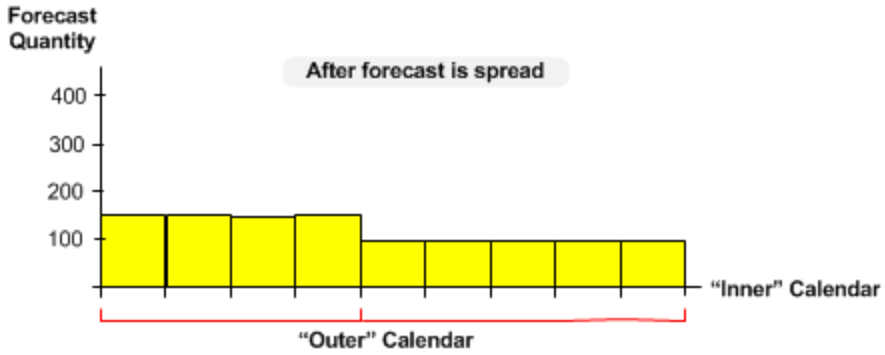
Assume a part's forecast is received monthly (**ForecastCalendar** = "Month"), as shown in the following illustration, with 600 units forecast at the start of the first month and 500 units forecast at the start of the second month.

Figure 15.1 Forecast before spreading



If the part's forecast is to be spread weekly (**SecondCalendar** = "Week"), and the applicable **DemandType**.**SpreadValue** value is set to "Spread", then the forecast values would be spread as shown in the following illustration, with 600 units spread over the four Weekly intervals in the first month (150 units per week) and 500 units spread over the four Weekly intervals in the second month (120 units per week). Note that, by default, forecast values from the "Outer" calendar are spread evenly over each "Inner" calendar but this can be configured through special spreading profiles.

Figure 15.2 Forecast after spreading



Spread profile calculation

RapidResponse performs the spread profile calculation to calculate the **Quantity** field in the **Forecast** table when there is more than one SecondCalendar date value in a ForecastCalendar interval (also assuming there is a demand (Independent or Dependent) with **DemandOrder.Type.ProcessingRule** = **SalesForecast**). The following table lists the spread profile calculation.

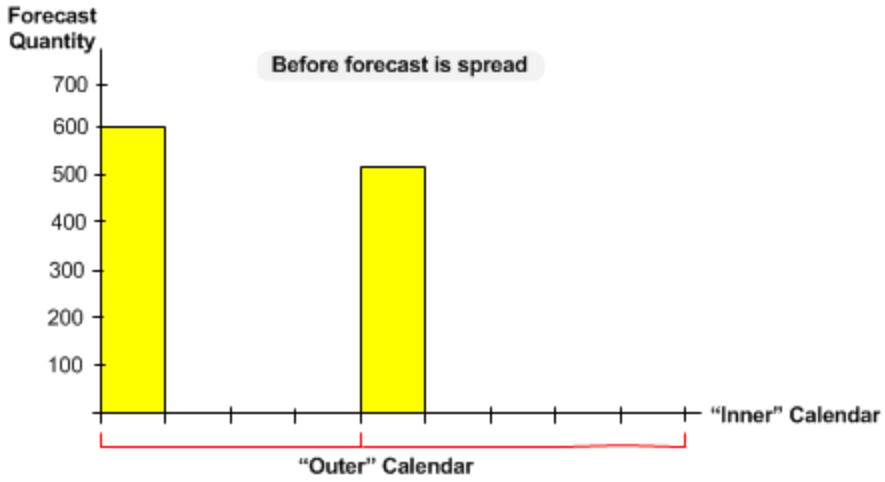
Table 15.3: Spread profile calculation

Rule	Calculation
A	If **SpreadProfile.NumberOfPoints** <=1 • Each spread forecast quantity in the ForecastCalendar period is calculated as: **Quantity** = (ForecastQuantity) / (Number of SecondCalendar periods in the ForecastCalendar period)
B	If **SpreadProfile.NumberOfPoint**s = Number of SecondCalendar periods in the ForecastCalendar period • The **Nth** spread forecast quantity is calculated as: **Quantity** = (ForecastQuantity) * (SpreadProfile.Point(N))/ (Sum of SpreadProfile.Point(1) to SpreadProfile.Point(Number of Points))
C	If **SpreadProfile.NumberOfPoints** <>Number of SecondCalendar periods in the ForecastCalendar period Calculate equivalent function points: • Linearize SecondCalendar. Convert each SecondCalendar date in the ForecastCalendar period to an equivalent number (X-value) representing the SpreadProfile.Point(x). • The X-value for the **Nth** SecondCalendar date is:1 + ((N-1)* (SpreadProfile.NumberOfPoints - 1) / (number of SecondCalendar dates in the ForecastCalendar period - 1)) Using the X-value just calculated: • the whole-number portion selects the Ith Point in the SpreadProfile.· • Equivalent function point(N) = PointI + (Point (I+1) - PointI)*(Xvalue - I) Calculate forecast quantity • The **Nth** spread forecast quantity is calculated as: **Quantity** = (ForecastQuantity) * (equivalent function point(N))/ (Sum of all equivalent function points)

Example: Forecast spreading with a profile

Assume a part's forecast is received monthly (**ForecastCalendar** = "Month"), as shown in the following illustration, with 600 units forecast at the start of the first month and 500 units forecast at the start of the second month.

Figure 15.3 Forecast before spreading



Further assume the part's forecast is to be spread weekly (**SecondCalendar** = "Week"), and the applicable **DemandType** record has a **SpreadValue** value set to "Spread" and references a **SpreadProfile** with the following points defined.

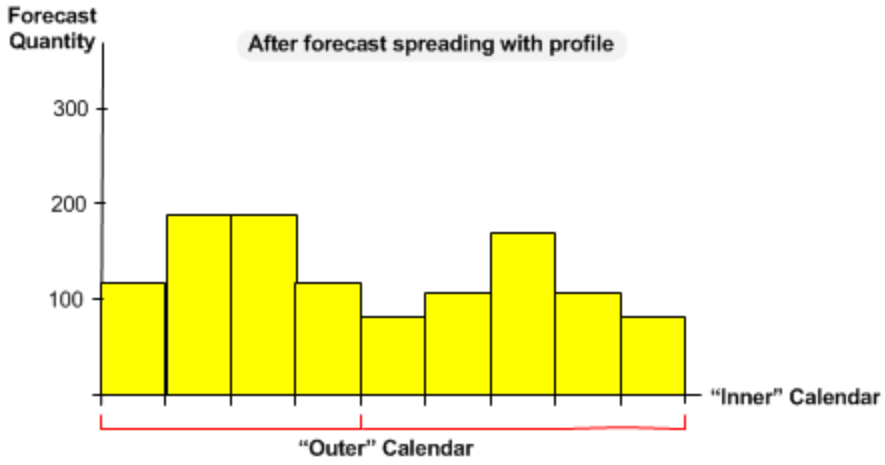
Table 15.4: Samples SpreadProfile value

Fleld	Value
NumberOfPoints	5
Point1	20
Point2	30
Point3	40
Point4	30
Point5	20

Thus, the profile curve resulting from these values would look similar to the following:

The forecast values would be spread as shown in the following illustration, with 600 units spread over the four Weekly intervals in first month (in amounts of 112.50, 187.50, 187.50, and 112.50) and 500 units spread over the five Weekly intervals in the second month (in amounts of 71.43, 107.14, 142.86, 107.14, and 71.43).

Figure 15.4 Forecast after spreading (with profile)

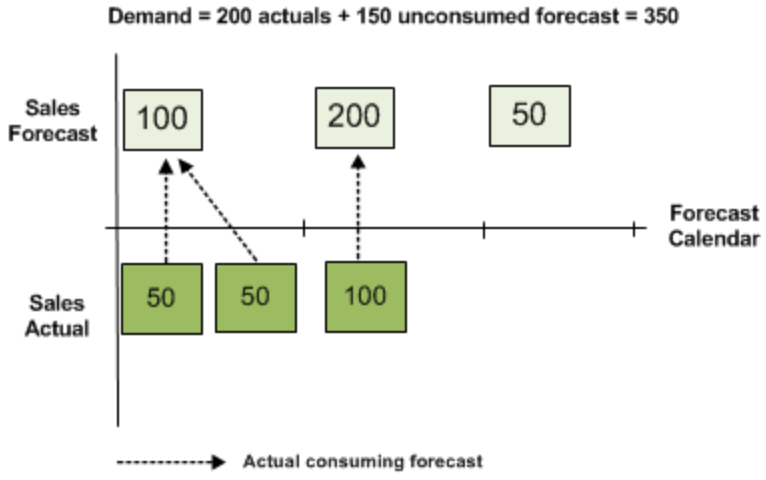


Forecast consumption

Forecast is represented by demand records (independent or dependent) with a **DemandType.ProcessingRule** control value of **SalesForecast**, as well as by **ConsensusForecast** records generated by the sales and operations planning in RapidResponse. Forecast represents an expected demand level. Actual sales demand typically comes from **IndependentDemand** records belonging to demand orders where the **DemandType.ProcessingRule** is set to **SalesActual** (it's possible to set up RapidResponse to generate **SalesActual** demand as dependent demand, but this is not found in typical enterprise data sources). If a given **IndependentDemand** record references a **DemandStatus** record where the **ForecastConsumptionOverride** value is set to "Normal" (the default setting), then that line item can consume forecast. Otherwise, if it references a record where **ForecastConsumptionOverride** is set to "DoNotConsume", then that line item is not eligible to consume forecast.

Sales actuals reduce the forecast because that portion of the forecast is replaced by the actual demands. If the actual exceeds the forecast, there is no unconsumed forecast. RapidResponse then plans for the actual demand instead of the originally forecasted amount. For example, assume three "SalesForecast" demands for 100, 200, and 50 units (350 total), and three "SalesActual" demands for 50, 50, and 100 units (200 total) as shown in the following figure.

Figure 15.5 Forecast consumption



In this case, the first forecast for 100 is completely consumed by the two 50 unit "SalesActual" demands. The remaining "SalesActual" is then able to consume 100 units of the second forecast demand which has 100 units remaining (unconsumed). The third and final forecast demand for 50 units is thus left completely unconsumed. Thus the demand that should be planned for by Netting is the total "SalesActual" of 200 units plus the unconsumed "SalesForecast" of 150 units; in other words 350 is the total demand that should be planned for.

Note: For **IndependentDemand** records that reference a **DemandType.ProcessingRule** setting of "SalesForecast", both the **Quantity** as well as the **ShippedQty** can consume forecast (only the Quantity is used in standard netting logic, though).

Demand time fence

If necessary, a near-term window can be configured in which any unconsumed forecast is ignored and not planned for by Netting, while unconsumed forecast outside the window will drive demand as normal. This window is referred to as the demand time fence.

The demand time fence in RapidResponse for a given part can be found in the calculated **Part.DTFDate** field. This gives the date before which any unconsumed SalesForecast demand is ignored and does not contribute to the total demand for a part. The DTFDate field is calculated as follows:

```
PlanningCalendars.RunDate.FirstDate + DemandTimeFence
PlanningCalendars.TimeFenceCalendar
```

The following table outlines the fields used to configure the demand time fence.

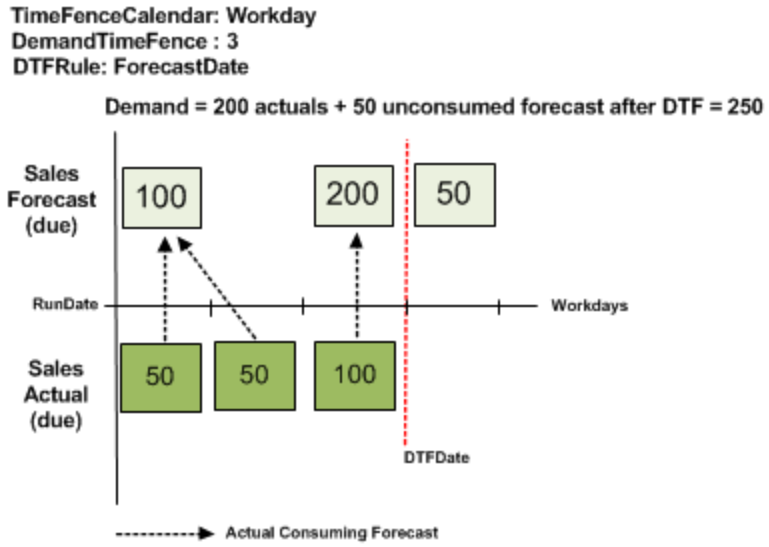
Table 15.5: Demand time fence configuration

Table	Field	Usage
Part	DemandTimeFence	Number of calendar intervals after RunDate to set the demand time fence. Any unconsumed forecast within the demand time fence will be ignored (not planned for in Netting). If set to o (zero), then the demand time fence is placed at the RunDate and all unconsumed forecast drives demand (in cases where the RunDate does not fall on a TimeFenceCalendar marker, it set to the previous marker in the calendar).
PlanningCalendars	TimeFenceCalendar	Defines the calendar whose intervals are used for setting the demand time fence as reported in **Part.DTFDate**.
PartType	DTFRule	Defines how unconsumed forecast before **Part.DTFDate** is processed. Valid values are: • **ForecastDate**—Any unconsumed forecast due before **DTFDate** is always ignored by Netting (supply is not planned to satisfy it). • **ForecastInterval**—Unconsumed forecast due before the **DTFDate** is generally ignored by Netting, except when the forecast is due within the same **ForecastInterval** as DTFDate. In that case, the unconsumed forecast is treated as being due at DTFDate and Netting will plan supply to satisfy it.

Note: The demand time fence is applied after, and does not affect, forecast spreading and consumption logic. It only impacts how unconsumed forecast is subsequently handled by Netting.

For example, assume again three "SalesForecast" demands for 100, 200, and 50 units (350 total), and three "SalesActual" demands for 50, 50, and 100 units (200 total) as shown in the following figure.

Figure 15.6 Impact of demand time fence



In this example, the first forecast for 100 is completely consumed by the two 50 unit "SalesActual" demands. The remaining "SalesActual" is then able to consume 100 units of the second forecast demand which has 100 units remaining (unconsumed). The third and final forecast demand for 50 units is thus left completely unconsumed.

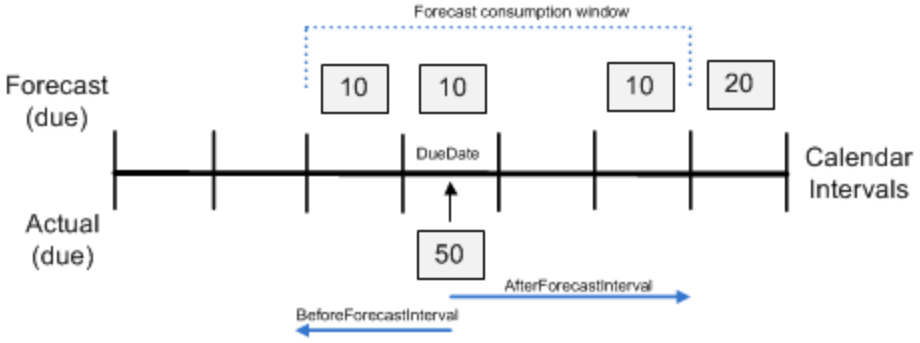
However, note in this example the **DemandTimeFence** and **TimeFenceCalendar** settings place the **DTFDate** three days after the **RunDate**. Based on the **DTFRule** setting of "ForecastDate", this means the unconsumed forecast of 100 units from the second SalesForecast demand should be ignored as it is due before the **DTFDate**.

The demand that should be planned for by Netting is the total "SalesActual" of 200 units plus the 50 units of unconsumed "SalesForecast" that is on/after the **DTFDate**. Thus, the true demand to be planned for in Netting is 250 units.

Forecast consumption window

Actuals and forecast do not always line up; for example, forecast is often entered only as weekly or monthly values. A forecast consumption window, or interval, allows a given actual demand order to reduce only those forecasts that are within a defined and configurable date range. For example, in the following illustration, the actual demand for 50 units is able to consume each of the three demands for 10 units, but not the one demand for 20 units as it falls outside of the forecast consumption window.

Figure 15.7 Forecast consumption window



The start of the forecast consumption window for a given demand is, by default, calculated by taking its **DueDate** and subtracting the number of calendar intervals defined by the value in **Part.BeforeForecastInterval.** This gives the number of forecast calendar intervals back that forecast can be consumed (inclusive). For example, a value of "0" indicates to consume from the **DueDate** to the beginning of the current forecast interval, and a value of "1" indicates to consume from the **DueDate** to the beginning of the previous forecast interval.

The end of a forecast consumption window for a given demand is, by default, calculated by taking its **DueDate** and adding a number of calendar intervals defined by the value in **Part.AfterForecastInterval**. This gives the number of forecast calendar intervals ahead that forecast can be consumed (exclusive). For example, a value of a value of "1" indicates to consume from the **DueDate** to the end of the current forecast interval, and a value of "2" indicates to consume from the **DueDate** to the end of the next forecast interval.

Forecast consumption window configuration settings

Several control settings are available that determine exactly how the forecast consumption window for a given demand is derived. This includes two settings on the **PartType** table.

The **PartType.ForecastConsumptionCalendar** option sets the calendar whose intervals the values in the **BeforeForecastInterval** and **AfterForecastInterval** fields on the **Part** table are defined in. This field has two options; "ForecastCalendar" and "SecondCalendar". ForecastCalendar is the default option and defines the forecast consumption window in terms of **Part.PlanningCalendars.ForecastCalendar** intervals (the "outer" calendar in which forecast is collected). The SecondCalendar option defines the forecast consumption window in terms of **Part.PlanningCalendars.SecondCalendar** (the "inner" calendar, if any, to which forecast is spread).

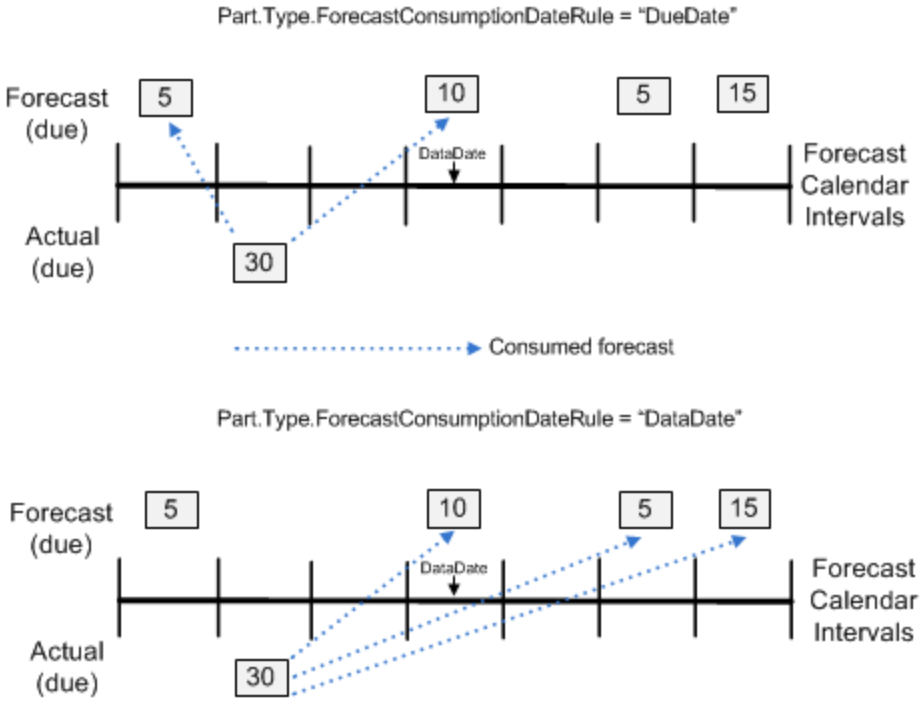
The **PartType.ForecastConsumptionDateRule** option is used to set the date from which all independent and dependent demands for a part will, by default, have their forecast consumption window determined. This field has two options; "DueDate" and "DataDate". DueDate is the default option and sets the forecast consumption window based on each demand's due date. The DataDate option is similar in that it will also set the forecast consumption window based on a demand's due date as long as that demand is on-time.

However, when using the "DataDate" option with a demand that is past-due (has a **DueDate** earlier than **Part.PlanningCalendars.DataDate.FirstDate**), the demand will have its forecast consumption window set relative to the **DataDate** (typically, it equates to today's date). This setting allows for past due demands to potentially consume more of the current forecast then they would if consuming based on their original **DueDate**.

For example, assume a part where the **PartType.ForecastConsumptionCalendar** setting is to "ForecastCalendar", **BeforeForecastInterval** is set to "2", and **AfterForecastInterval** is set to "4". The

following illustration shows how the different settings in **PartType.ForecastConsumptionDateRule** might impact the forecast consumption for a past due independent demand.

Figure 15.8 Impact of **PartType.ForecastConsumptionDateRule** setting

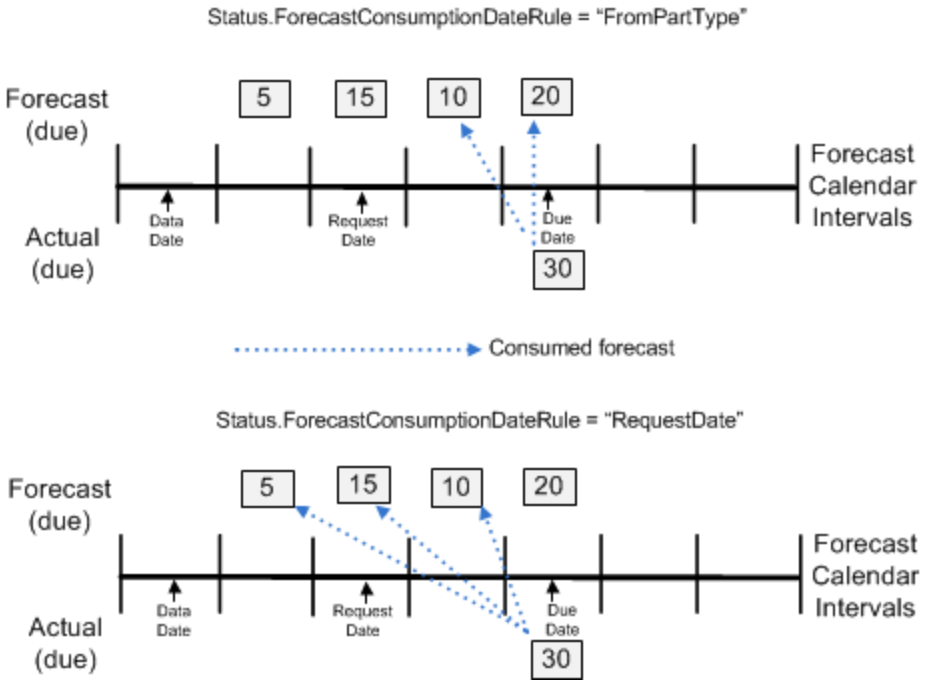


The forecast consumption window can also be configured at the independent demand level using the **DemandStatus**.**ForecastConsumptionDateRule** field. The default value for this field is "FromPartType" in which case the demand uses the **PartType** setting defined for its part. This field also contains "DueDate" and "DataDate" options with identical behavior to those same settings on the **PartType** table, but allow for a given independent demand to override the default **PartType** setting if required.

The **DemandStatus.ForecastConsumptionDateRule** field also contains two options, "RequestDate" and "RequestOrDataDate", that are not available on the **PartType**. When using the "RequestDate" option, an independent demand will always consume forecast relative to its **RequestDate**. However, with the "RequestOrDataDate" option, an independent demand will consume forecast relative to its **RequestDate** only if that request date is on or after **Part.PlanningCalendars.DataDate.FirstDate.** If the **RequestDate** is in the past, then forecast consumption is made relative to the **DataDate** instead. These two options might typically be used in cases where the customer requested delivery date more closely aligns with the forecast than does the scheduled delivery date.

For example, assume a part where **PartType.ForecastConsumptionDateRule** is set to "DueDate", **BeforeForecastInterval** is set to "1", and **AfterForecastInterval** is set to "2". The following illustration shows how a given independent demand might have its consumption affected by the **DemandStatus.ForecastConsumptionDateRule** setting.

Figure 15.9 Impact of DemandStatus.ForecastConsumptionDateRule setting



Note: Based on the applicable control settings, the actual date an independent demand will use to set its forecast consumption window is reported in the calculated **IndependentDemand.ForecastConsumptionDate** field.

Sales actual consumption order

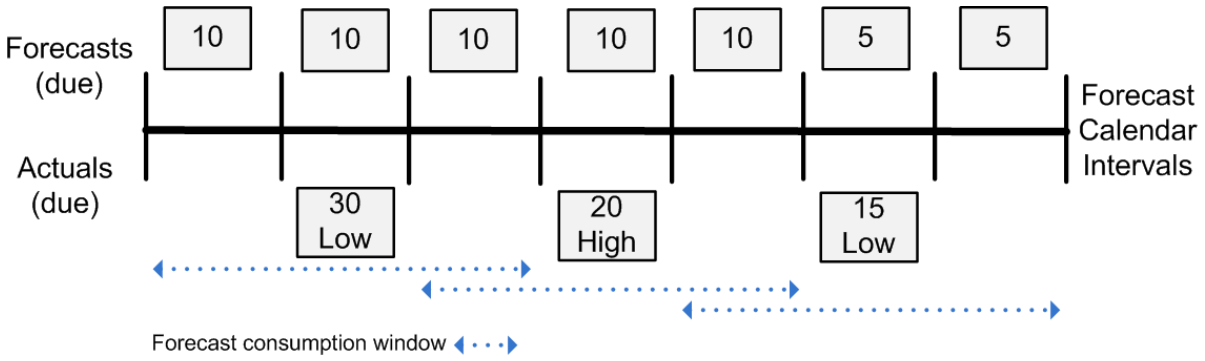
As discussed previously, each actual demand (where **Type.ProcessingRule** is set to "SalesActual") can consume forecast within its forecast consumption window. The **PartType.SalesActualConsumptionOrder** setting can be used to specify the order in which these actual demands are processed and used to consume forecast.

With the default **SalesActuualConsumptionOrder** of "ByDueDate", actual demands are processed in due date sequence with earlier due actuals therefore eligible to consume forecast first. In cases where there are multiple actual demands due on the same date, demands with a lower **Order.Id** value will consume forecast first and if there are still multiple eligible demands then those with a lower **Quantity** will consume forecast first.

The **SalesActualConsumptionOrder** field can also be set to "ByPriority" to allow actual demands to consume in priority order. That is, higher priority actual demands (those with lower **PlanningPriority** values) will be eligible to consume forecast first. Within each given priority level, actuals demands are still processed in **DueDate** order (and by **Order.Id** and **Quantity** if there are multiple demands of the same priority level on a given date).

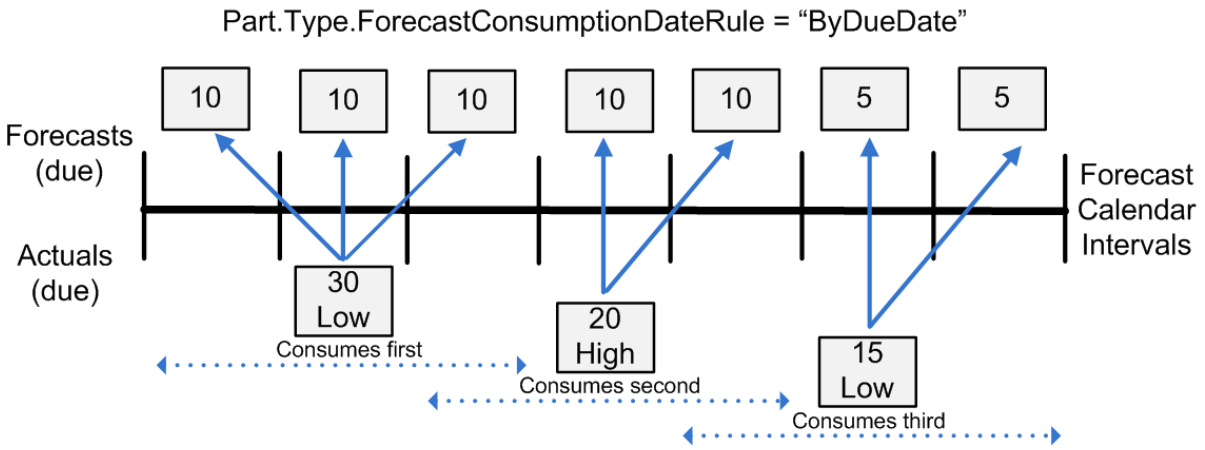
For example, assume a part has a **BeforeForecastInterval** of "1" and an **AfterForecastInterval** of "2" along with the series of prioritized demands shown in the following figure.

Figure 15.10 Prioritized sales actual demands



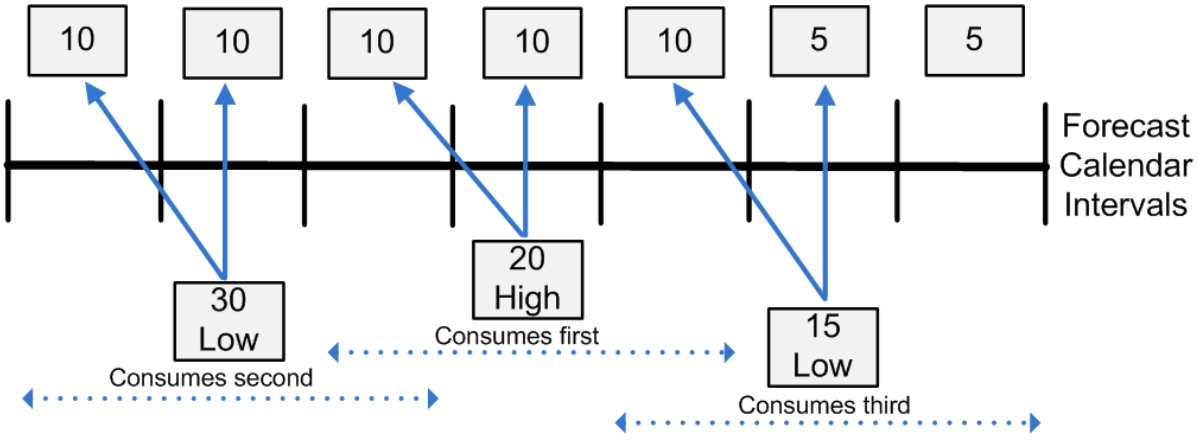
With the "ByDueDate" setting, forecast consumption starts with the earliest due actuals shown in the following figure. Thus, starting with the low priority demand for 30 units, each actual demand consumes what it can within its forecast consumption window before the next due actual is processed. Note that in this case, all forecast is consumed.

Figure 15.11 Sales actuals consuming in due date order



Instead, with the "ByPriority" setting, forecast consumption would start with the High priority demand for 20 units as shown in the following figure. Because this demand is eligible to consume forecast first, it ends up consuming some of the forecast which would otherwise be consumed by the Low priority demand for 30 units. After the High priority demand is done consuming forecast, consumption continues with the two Low priority demands which are processed in DueDate order. Note that in this case, the last 5 units of forecast ends up being left unconsumed

Figure 15.12 Sales actuals consuming in priority order



Notes:

- This example assumes the default **PartType.ForecastConsumptionOrder** setting of "Normal" (a given actual first consumes from its **DueDate** back to the start of the forecast consumption window, and then continues after **DueDate** forward to the end of the forecast consumption window). For more information on this control setting, see "Forecast consumption order" on page 1438.
- If a part has a **PartType.CommitLevel** setting of "Low", then changing the **SalesActualConsumptionOrder** setting has no impact (priority is never used in determining the order in which sales actuals are eligible to consume forecast).

Forecast consumption order

As discussed previously, actual demands are eligible to consume forecast within their forecast consumption window and are processed in an order determined by the part's **PartType.SalesActualConsumptionOrder** setting.

Within its forecast consumption window, a given actual will consume available forecasts in an order determined by the part's **PartType.ForecastConsumptionOrder** setting. The possible values for this field are:

- Normal
- NormalNested
- Forward
- Backward
- AlternatingForward
- AlternatingBackward

If multiple forecasts are due on the same date, the order in which they are consumed is determined by the following:

- Order priority, with higher priority forecasts consumed first.
- Allotment order, with lower allotment values consumed first. This is considered if two or more forecasts have the same order priority.
- Quantity, with lower quantities consumed first. This is considered if two or more forecasts have the same allotment order.

Order priorities are considered only when **PartType.CommitLevel = 'High'**. Otherwise, the priority is ignored, and the forecasts due on the same date are consumed by allotment order and quantity. For more information about order priorities, see "Order priority calculations" on page 1765.

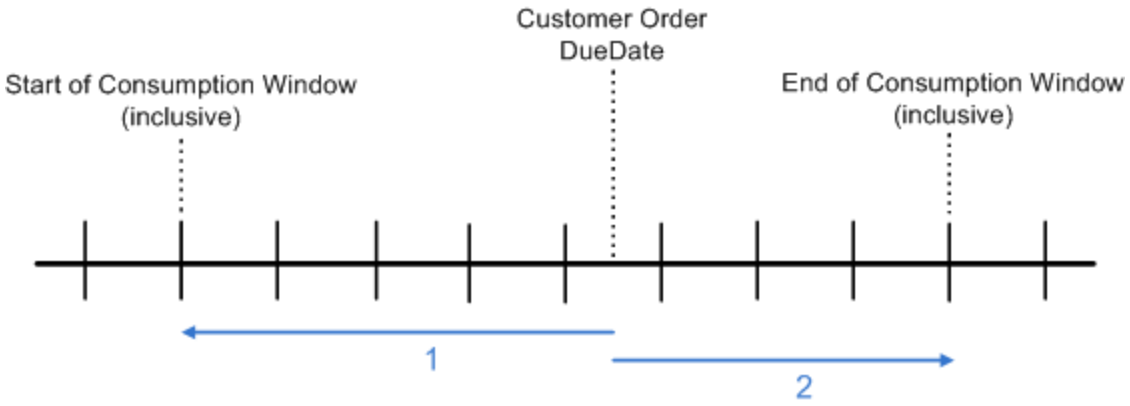
Notes:

- The **ForecastConsumption** table can be used to see which actual demands consume specific forecasts and in what amount. For more information, see "ForecastConsumption table" on page 1063.
- Forecasts are never reduced below zero.

Normal forecast consumption

The default setting for the PartType.ForecastConsumptionOrder field is "Normal". As shown in the following illustration, this setting first consumes forecast from the same date as the actual demand due date. If the actual demand quantity is not fully covered by this forecast, consumption continues backwards until the start of the forecast interval is reached. If the actual demand quantity is still not fully covered by those forecasts, consumption resumes with the first record found after the order due date, and continues forward until either the entire demand quantity is used or the end of the forecast interval is reached.

Figure 15.13 "Normal" forecast consumption order

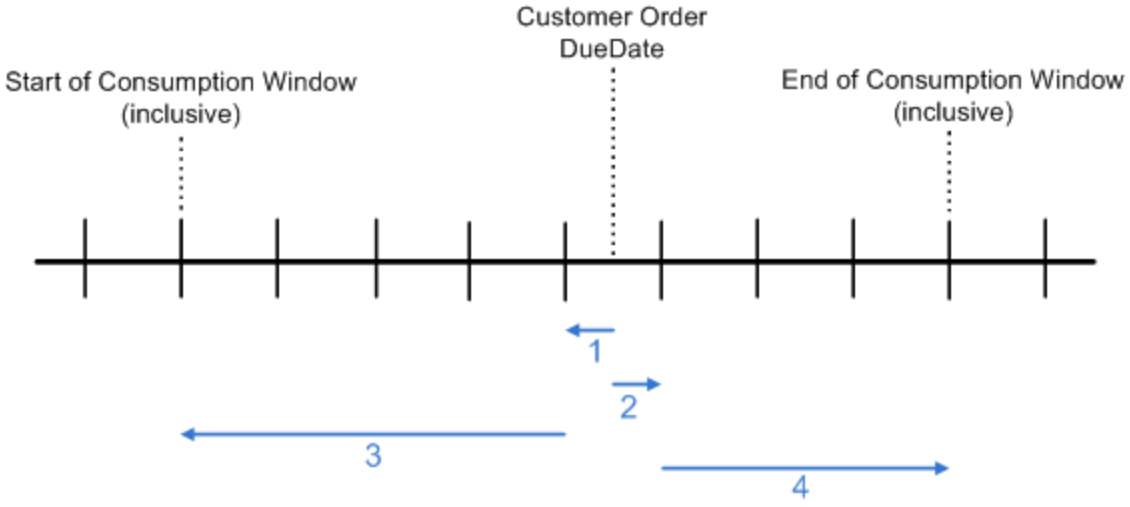


NormalNested forecast consumption

The "NormalNested" setting first consumes forecast in the actual demand order's **Part.PlanningCalendars.ForecastCalendar** interval, and then continues consuming forecast similarly to the "Normal" setting, as shown in the following illustration. The forecast is consumed from the same date as the actual demand, until the beginning of the actual order's ForecastCalendar interval is reached. The consumption then continues forwards from the actual demand due date until the end of the actual order's ForecastCalendar interval is reached. If the actual demand quantity is not fully consumed by this forecast, consumption continues backwards from the start of the actual order's

ForecastCalendar interval to the beginning of the forecast interval. If the actual demand quantity is still not fully satisfied, consumption continues forward from the end of the actual order's ForecastCalendar interval until either the demand is fully satisfied or the end of the forecast interval is reached.

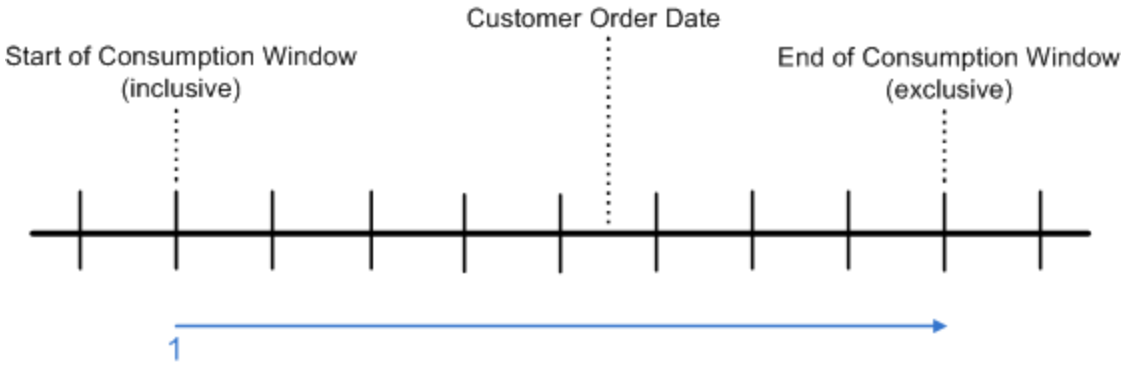
Figure 15.14 "NormalNested" forecast consumption orde



Forward and Backward forecast consumption

The "Forward" and "Backward" settings each consume forecast in a single direction. When "Forward" is used, forecast is consumed beginning at the start of the forecast interval, and moving forward until either the entire demand quantity is used or the end of the forecast interval is reached. This is shown in the following illustration. Conversely, when "Backward" is used, forecast is consumed beginning at the end of the forecast interval, and moving backward until either the entire demand quantity is used or the start of the forecast interval is reached.

Figure 15.15 "Forward" forecast consumption order

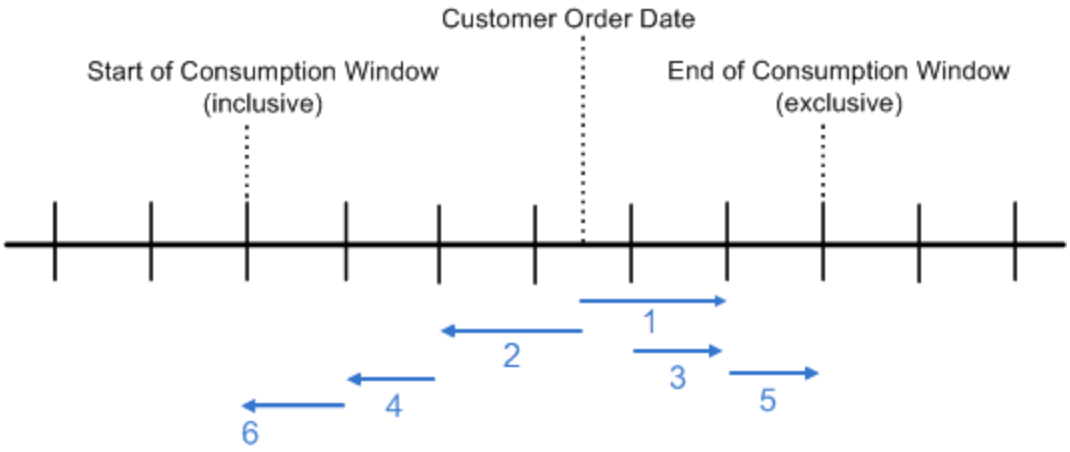


AlternatingForward and AlternatingBackward forecast consumption

The "AlternatingForward" and "AlternatingBackward" settings each consume forecast in back and forth intervals defined by the **Part.PlanningCalendars.SecondCalendar** value. These settings are meant to initially consume forecast that is as close to the demand as possible.

When "AlternatingForward" is used, forecast is consumed starting from the customer order date and moving forward until the end of the next SecondCalendar interval is reached. Consumption then continues in the SecondCalendar interval preceding the customer order date, and continues back and forth until either the entire demand quantity is used or all forecast within the forecast interval has been consumed. This is shown in the following illustration. Conversely, when "AlternatingBackward" is used, forecast is consumed starting from the customer order date and moving backward until the beginning of the preceding SecondCalendar interval is reached. Consumption then continues in the SecondCalendar interval after the customer order date, and continues back and forth until either the entire demand quantity is used or all forecast within the forecast interval has been consumed.

Figure 15.16 "AlternatingForward" forecast consumption order



Both a forecast and its unconsumed quantity are reported in the **Forecast** table. Only unconsumed forecast with a date later than or equal to the DTFDate (demand time fence date) is used in netting. The DTFDate is calculated by adding the Part's DemandTimeFence quantity to the RunDate (PlanningCalendars.RunDate.FirstDate). Forecast consumption happens before the demand time fence. However, netting does not use the resulting unconsumed forecast.

Dependent forecast consumption on components

If forecast is placed on component-level parts, any dependent demands generated for those components can potentially consume forecast.

Thus, if a component has forecast placed on it, that component part should be set up as an MPS part; in other words, its **PartType.ProcessingRule** field should be set to "MPS". Additionally, any forecast demands placed on that component should reference a **DemandType.ProcessingRule** setting of "SalesForecast".

In cases where forecast is made against component parts that receive dependent demands, those dependent demands can then be set up to consume forecast. This configuration is made on the assembly supply that generates the dependent demand and relies on the **SupplyType.AllocationDemandType** field which is a reference to the **DemandType** table.

Then, in order for dependent demands generated from an assembly supply to be eligible to consume component forecast, the assembly supply's **AllocationDemandType** setting must reference a **DemandType** record where the **ProcessingRule** is set to "SalesActual". Note that if the assembly supply is a planned order, then the **SupplyType** reference is made from the **PartSourceType** table. However, if

the assembly supply is a scheduled receipt, the **SupplyType** reference is made from the **ScheduledReceipt** record itself, and the **SupplyType.AllocationForecastRule** can be further used to control whether dependent demands exploded from the scheduled receipt are used in forecast logic.

Dependent forecast consumption on a component by component basis

Dependent demands can be considered for forecast consumption on a component by component basis. This is controlled by the 'FromPart' setting on the **DemandType.ProcessingRule** field. When this setting is used, dependent demands look to the value in **PartType.DependentDemandForecastConsumption**, either Y(es) or N(o), to determine if a particular component should be used in forecast consumption. As a result, dependent demands from the same supply might end up being processed for forecast consumption for one component and ignored for another.

If forecast consumption on a component basis is desired behavior, ensure the following:

- All scheduled receipts have **Type.AllocationDemandType.ProcessingRule** = 'FromPart'.
- **PartType.DependentDemandForecastConsumption** is set to 'Y' for at least one part type.
- All parts that are to have dependent demands consuming forecast reference a **PartType** that has **DependentDemandForecastConsumption** set to 'Y'.
- The 'FromPart' option is not used in any **DemandType** records referenced by IndependentDemand records (otherwise, those records will be treated as 'Regular' demands and not processed for forecast consumption).

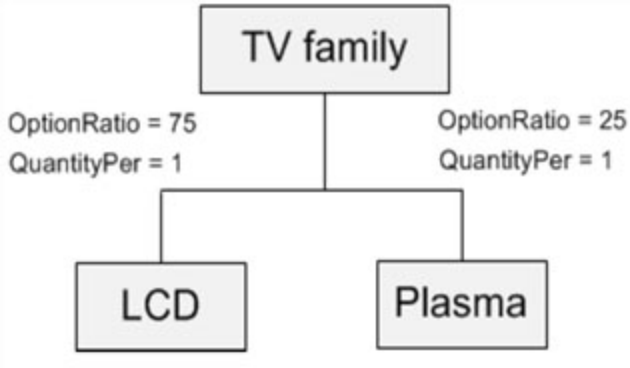
MPS configuration

MPS configuration is used for forecasting by product family. In this configuration, forecast created from a parent part (MPSConfig part) drives demand for component parts within a family (MPS parts). Actual demand for these MPS parts then consumes or reduces forecast from the parent MPSConfig part. However, if any forecasts have also been placed on an MPS part, its actual demands will consume those forecasts first before moving to the MPSConfig part.

Setting up MPS configuration data

To use MPS configuration, planning BOMS are required to define the relationship between the MPSConfig parent part and the MPS component parts. For example, the following illustration shows a simple MPS configuration with a TV family consisting of a LCD model and a Plasma model.

Figure 15.17 A TV family planning bill of material



In order for MPS configuration calculations to be performed, specific data must be configured in the **Part**, **BillOfMaterial**, **IndependentDemand**, and **PartSource** tables. The following outlines the relevant data to be setup in each of these tables.

Table 15.6: Tables used in MPS configuration setup

Table	Details
Part	All parts in an MPS configuration should exist in the **Part** table. • The MPSConfig (parent) part should have a **PartType.ProcessingRule** value of 'MPSConfig'. This setting is used to explode forecast on the MPSConfig part and drive demand to its components. The actual demand for those component parts can subsequently consume its forecast, and any unconsumed forecast results in planned orders being generated. • The MPS (component) parts should have a **PartType.ProcessingRule** value of 'MPS'. This setting is used to ensure that actual sales orders for these parts consume forecast orders from the MPSConfig part. Note that if any forecast is placed directly on an MPS part, it will be consumed before the forecast on the MPSConfig part is consumed.
BillOfMaterial	The planning BOMs that specify the relationship between MPSConfig (parent) and MPS (component) parts are defined in the **BillOfMaterial** table. One record is required for each MPSConfig and MPS part combination, and the following fields are of particular relevance: • **Assembly.Name**—identifies the MPSConfig part. • **Component.Name**—identifies the MPS part. • **OptionRatio—**the ratio by which unconsumed forecast for the MPSConfig part generates dependent demand for the MPS part. Depending on the setting in Type.RatioRule, values between 0 - 1 (fractions) or 1 - 100 (percentages) can be specified. • **Type.RatioRule**—indicates how the OptionRatio value is interpreted (as a percentage, fraction, or so on). • **QuantityPer—**the number of MPS parts required for each MPSConfig part. By default, this is set to 1. • **Type.MPSConfigDemandSource**— should be set to **Y** to indicate the MPS part is eligible to consume forecast from its MPSConfig parent.

Table 15.6: Tables used in MPS configuration setup

Table	Details
IndependentDemand	The records that specify the quantity of each forecasted or actual demand are stored in the **IndependentDemand** table. Forecasts should have an **Order.Type.ProcessingRule** value of 'SalesForecast', and actual demands that consume forecast should have an **Order.Type.ProcessingRule** value of 'SalesActual'.
PartSource	There must be at least one record defined in the **PartSource** table for each part in an MPS configuration. If there is not a record in this table, RapidResponse does not process the part (for example, planned order records are not created).

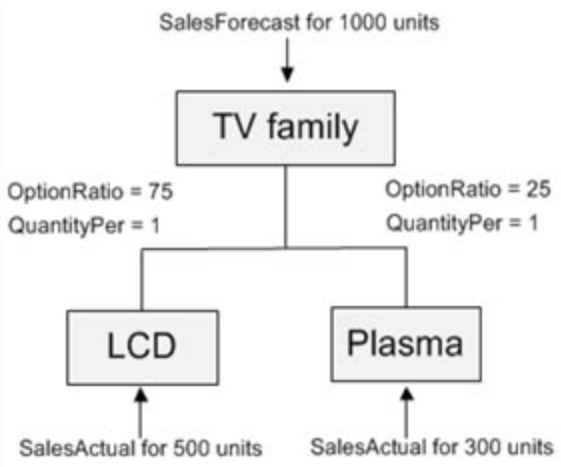
Notes:

- To allow forecast to explode from the assembly part to its option components, but without allowing those components to consume the assembly's forecast, set the parent assembly to have a **PartType.ProcessingRule** other than "MPSConfig" (that is, "MPS"). The "MPSConfig" setting is only strictly required when components should be able to consume the parent assembly's forecast.
- For detailed information about each of the tables and fields mentioned above, see the reference section of this guide.
- MPS configuration also respects forecast consumption by customer pool. For example, common pool forecast might be placed on the MPSConfig part and customer specific forecast might be placed on the MPS parts. This also enables reporting what portion of an order consumed from the customer's forecast and what portion consumed from the common pool. For more information, see "Forecast consumption by pool/customer" on page 1447.

Example (forecast on MPSConfig part only)

Suppose an MPS configuration with settings and demands as shown below.

Figure 15.18 Forecast and demands for MPS configuration



The original forecast for 1,000 units of the TV family creates dependent demand for the LCD model and Plasma model. Dependent demand for the LCD model is 750 units (1,000 multiplied by the Option ratio of 75) and for the Plasma model is 250 units (1,000 multiplied by the Option ratio of 25).

However, subsequent actual demands for 500 units of the LCD model and 300 units of the Plasma model consume a total of 800 units of the original forecast. The new unconsumed forecast quantity of 200 units for the TV family is then exploded down to the MPS parts. As a result, the dependent forecast for the LCD model is now 15o units (200 units multiplied by the Option ratio of 75) and the dependent forecast for the Plasma model is now 50 units (200 units multiplied by the Option ratio of 25).

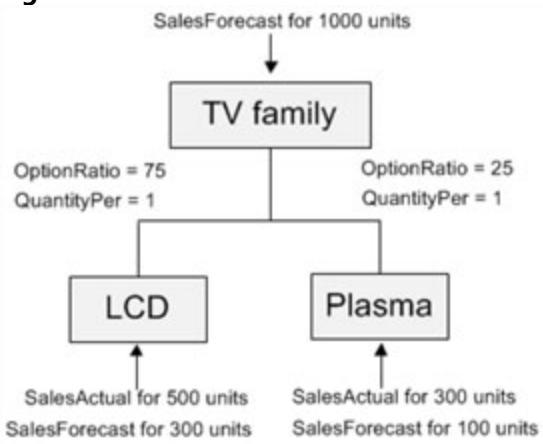
Assuming there is no available supply, RapidResponse creates the following calculated records:

- A **PlannedOrder** record for 200 units of **TV family** (equal to the unconsumed forecast)
- A **PlannedOrder** record for 650 units of **LCD model** (150 to cover the forecast and 500 to cover the actual demand).
- A **PlannedOrder** record for 350 units of **Plasma model** (50 to cover the forecast and 300 to cover the actual demand).

Example (forecast on MPSConfig and MPS parts)

Suppose an MPS configuration with settings and demands as shown below.

Figure 15.19 Forecasts and demands for MPS configuration



This case is identical to the previous example except that it assumes forecasts are also placed directly on the MPS parts (LCD model and Plasma). In this case, the actual demand for these parts first consumes their own forecasts, before consuming forecast from the MPSConfig part.

The actual demand of 500 units for the LCD model first consumes all 300 units of LCD model forecast, and the remainder then consumes 200 units of forecast from the TV family reducing it to 800 units. Similarly, the actual demand of 300 units for the Plasma model first consumes all 100 units of the Plasma model forecast, and the remainder then consumes 200 units of forecast from the TV family further reducing it to 600 units.

This unconsumed forecast quantity from the TV family is then exploded down to the MPS parts. As a result, the dependent forecast for the LCD model is 450 units (600 units multiplied by the Option ratio of 75) and the dependent forecast for the Plasma model is now 150 units (600 units multiplied by the Option ratio of 25).

Assuming there is no available supply, RapidResponse creates the following calculated records:

- A **PlannedOrder** record for 600 units of **TV family** (equal to the unconsumed forecast)
- A **PlannedOrder** record for 950 units of **LCD model** (450 to cover the forecast and 500 to cover the actual demand).
- A **PlannedOrder** record for 450 units of **Plasma model** (150 to cover the forecast and 300 to cover the actual demand).

Dependent forecast on scheduled receipts

RapidResponse calculations always use dependent demands from planned orders in forecast logic. It is also possible to specify that dependent demands from scheduled receipts with **Order.Type.AllocationDemandType.ProcessingRule** of 'SalesForecast' generate forecast demand for MPS parts, and that dependent demands from scheduled receipts with **Order.Type.AllocationDemandType.ProcessingRule** of 'SalesActual' consume forecast demands from MPS parts. This is done by setting the value of **SupplyType.AllocationForecastRule** to 'Use', and will typically result in more demands for MPS parts.

The forecast consumption of 'SalesActual' dependent demands considers both the Quantity and TotalQuantity on the originating ScheduledReceipt record. This is similar to the logic used with independent demands, where the quantity consuming forecast is calculated as `IndependentDemand.Quantity` + `IndependentDemand.ShippedQuantity`. This means even if the scheduled receipt has a Quantity = 0, but a TotalQuantity > 0, then it should be exploded to generate SalesActual dependent demands for MPS components. Such demands would be used exclusively for forecast consumption, and are otherwise ignored by Netting calculations. The actual quantity to be used when consuming forecast is calculated as
`Max(TotalQuantity - Quantity, 0) + Quantity`
In other words, if the TotalQuantity is greater than the Quantity, the TotalQuantity value is used. Otherwise, the Quantity value is used.

Allocations and forecast consumption

Similar to the processing of scheduled receipts, there might be Allocation records that are ignored by Netting calculations, but that are processed by forecast consumption. The forecast consumption of allocations considers both the RemainingQuantity and TotalQuantity on the allocation. This means that an Allocation record where Quantity = 0, which would be ignored by Netting, is used in forecast consumption if **ScheduledReceipt.Order.Type.AllocationDemandType.ProcessingRule** = 'SalesActual' and **Allocation.TotalQuantity** > 0. The total amount to be used for forecast consumption is calculated as follows:

EffTotalQty = `Max(TotalQuantity, RemainingQty) + ShippedWithSR`
where
ShippedWithSR = `(Max(SR.TotalQuantity, SRQtyToUse) - SRQtyToUse)*QtyPer`

SRQtyToUse = `OriginalQty > 0 ? OriginalQty : Quantity`

QtyPer = `Max(Max(Allocation.TotalQty, Allocation.RemainingQty)/ Max (SR.TotalQuantity, SRQtyToUse), Allocation.RemainingQty/SRQtyToUse)`

Forecast consumption by pool/customer

Using inventory pooling, RapidResponse allows for customer specific forecasts and forecast consumption by customer.

For a given part, sales forecasts can be segregated into different pools by customer, as well as a common pool that represents all non-customer specific forecasts. Actual sales for a customer will first try to consume forecast from that given customer's pool before moving to the common pool (after all forecast in the customer pool has been consumed).

Notes:

- Forecast consumption by pool is always available, even if Pool is not enabled. However, certain other pool capabilities are available only if Pool is enabled. For more information about inventory pooling, see "Inventory Pooling calculations" on page 1745.
- Forecast consumption by customer, or other relevant entities, can also be configured through the use of attributes. For more information, see "Attribute-based planning" on page 1817.

Setting up forecast consumption by customer

In addition to the standard forecast settings described earlier in this chapter, several RapidResponse fields need to be configured to support forecast consumption by customer. The following outlines the relevant data to be setup in these fields.

Table 15.7: Fields used in setting up forecast consumption by pool

Field	Details
DemandOrder.Pool	Specifies the customer pool a given demand order belongs to. A value should be added to the Pool table corresponding to each customer whose forecasts will be segregated by pool. Note that the Customer field on a given demand order has no impact on the forecast consumption logic.
Part.PlanningCalendars.CommonPool	Part table reference to the CommonPool field on the PlanningCalendars table. It defines the common pool for the part. This pool is used when the part's forecast consumption logic is looking for common pool forecast (for example, if there is no customer specific forecast to consume).
Part.MUEPoolNettingType.PoolRule	Part table reference to the PoolRule field on the MUEPoolNettingType table. It specifies how, or if, the pool is used in the forecast consumption logic.
Part.MUEPoolNettingType.PoolFlowToCommon	Part table reference to the Pool field on the MUEPoolNettingType table. It indicates if actual sales orders for the part can consume forecast from the common pool forecast after all forecast from the customer specific pool has been consumed. This field has two settings; Forecast and Ignore. It should be set to Forecast if you want actual sales orders to be able to consume from the common pool, and Ignore if you do not.

RapidResponse also allows for reporting what portion of an actual demand consumed from the customer forecast, and what portion of an actual demand consumed from the common forecast. This is reported through the following two fields on the **IndependentDemand** table:

- **ForecastQuantity**—the amount of the order that consumed forecast. This includes both forecast consumed from the customer's forecast pool and forecast consumed from the common pool forecast.
- **FlowToCommonForecastQuantity**—The amount of the order that consumed forecast from the common forecast pool (either because all of the customer's forecast had already been consumed, or because no pool value was provided on the order).

Note: These fields are also calculated on other RapidResponse tables (for example, the Activity table).

Example

Suppose a part whose properties include the following:

- **Name**—CD player
- **Type.ProcessingRule**—MPS
- **PlanningCalendars.CommonPool**—Common
- **MUEPoolNettingType.PoolRule**—Forecast
- **MUEPoolNettingType.Pool**—Forecast

Further suppose a series of forecast and actual independent demands for the part from customers BetterBuy and CircuitTown which include the following values:

Order.Id	Order.Type. ProcessingRule	Order.Pool	DueDate	Quantity
1	SalesForecast	Common	07-07-08	1000
2	SalesForecast	CircuitTown	07-07-08	800
3	SalesForecast	BetterBuy	07-07-08	400
4	SalesActual		07-08-08	500
5	SalesActual	CircuitTown	07-09-08	600
6	SalesActual	CircuitTown	07-10-08	400
7	SalesActual	BetterBuy	07-11-08	500

In this scenario, forecast consumption happens as follows:

1. Order 4 on July 8 has no customer pool specified, and so its entire 500 units consume from the Common pool. This reduces the Common pool to 500 units.
2. Order 5 on July 9 first attempts to consume from the specified CircuitTown pool, and because sufficient forecast can be found there consumes only from that pool. This reduces the CircuitTown pool by 600 units down to 200 units.
3. Order 6 on July 10 also attempts to consume from the CircuitTown pool, but can only find 200 of the required 400 units. So it consumes the remaining 200 units from the CircuitTown pool, before

moving to the Common pool and consuming 200 more units. This reduces the Common pool to 300 units.

4. Order 7 on July 11 attempts to consume 500 units from the specified BetterBuy pool, but can only find 400 units. It consumes all this available forecast from the BetterBuy pool, and then moves to the Common pool and consumes another 100 units. This reduce the Common pool to 200 units.

In the end, all customer-specific forecast from the CircuitTown and BetterBuy pools have been completely consumed, and 200 units of unconsumed forecast remains for the common pool. As well, the following ForecastQuantity and ForecastQuantity values are reported on each of the actual demands.

Order.Id	ForecastQuantity	ForecastQuantity
4	500	500
5	600	0
6	400	200
7	500	100

MPS configuration and forecast consumption by pool

Forecast consumption by pool can be used in conjunction with MPS configuration for forecasting by product family. This includes cases where forecast is placed both on the parent (MPSConfig) part and component (MPS) part. For example, common pool forecast might be placed on the parent part and customer-specific forecast might be placed on the component part. In this case, forecast consumption would happen in the following order:

1. Customer specific forecast for MPS part.
2. Common pool forecast for MPS part.
3. Customer specific forecast for MPSConfig part.
4. Common pool forecast for MPSConfig part.

Note: For more information about MPS configuration, see "MPS configuration" on page 1442.

Available to promise

Available-To-Promise is a standard MPS calculation used to indicate how much supply has not already been committed to real customer demand. It is generally the portion of supply that is either allocated to forecast or not allocated to any demand. Thus, it is the amount of available supply that can be used to accept (promise) new customer orders.

In RapidResponse, available-to-promise information is reported by both the Netting (ATP) and Capable-To-Promise analytics (ATPCTP).

ATP

The Netting version of ATP can be thought of as a more traditional ATP calculation and provides a date-based report without specific references to actual supply records. Additionally, the results reported in this table are made without knowledge of the actual or final allotments of supply to demand as

calculated by the Capable-to-Promise analytic. Rather these calculations rely on the date-based allotments that would be assumed by Netting if priorities were not involved at all. Priorities are not included in the traditional (and Netting-based) ATP calculations.

The results of the ATP calculations are reported in the **AvailableToPromise** table. For each unique part, this calculated table contains one record on the MRPDate and then subsequent records are generated on each **Part.PlanningCalendar.TimeUnit** date where a supply is due. Each record contains references to **Part**, **Model**, **StartUnit**, and **Pool** to identify the specific item the **ATP** being reported refers to, as well as a **DueDate** and two quantity fields; **SRQuantity** and **Quantity**. The **SRQuantity** field reports a more traditional measure of ATP and considers only open supply (that is, **OnHand** and **ScheduledReceipt** records) while the **Quantity** field includes amounts from planned orders as well.

Both the **SRQuantity** and **Quantity** fields can report negative balances, but this will only ever occur at **RunDate**. A negative balance in the **SRQuantity** field indicates that there is more actual (non-forecast) demand than there is currently open supply. A negative balance in the **Quantity** field indicates there is more actual (non-forecast) demand than there is currently open and planned supply. If such a negative balance is reported in either field, that field will then report zero-values on all subsequent records in the planning horizon.

ATPCTP

The Capable-to-Promise version of ATP can be thought of as a more real-time ATP calculation and considers the actual allotments of supply to demand as determined by capable-to-promise calculations. Thus, the actual ATP results may depend on things such as order priorities and supply availability (as discussed in "Capable-to-Promise calculations" on page 1453). Additionally, because this version of ATP is related directly to the supplies that are actually allocated to forecast or excess, the values are reported directly on each supply they pertain to. For example, it is reported in the **CTPActivity.ATPQuantity** field on activity records pertaining to specific supplies.

CHAPTER 16: Inventory turns calculations

RapidResponse calculates several fields to be used in determining the projected inventory turns for the next year. Because any demand or supply that is scheduled before the MRP RunDate cannot happen until the MRP RunDate or later, it is considered part of the first year activity. 365 days (real days, not work days) from the MRP RunDate is used as the cutoff point in calculating the turns parameters.

This chapter discusses the **Part** table fields which store the results of inventory turns calculations.

TotalFirstYearDemand

This is the total quantity of demand that netting consumes to the end of the first year.

AverageFirstYearInventory

This is the daily (real days not work days) average inventory level, including safety stock. Parts are in inventory from the DueDate of their supply until consumed by a demand. The date of the demand does not count as a stocking day but the date of the supply does. Thus an order that arrives one day and leaves the next will only count as being in inventory one day. If supply and demand dates lineup completely, and there is no safety stock the average first year, inventory is zero. Rescheduled dates are used for allocations and scheduled receipts (not recommended but only the automatic rescheduling authorized by the processing rules).

AverageFirstYearInProcess

This is the quantity of the part in the process of being obtained. The duration of an order is the period from its CalcStartDate to the ReschedDate. The period is influenced by the lead-time and the relationship between the workday calendar (PlanningCalendars.TimeUnits) and real days. For example, one workday lead-time starting on a Monday might be one real day but on a Friday, it would be three.

The quantity of an order (before applying yield) is multiplied by the duration of the order in real days. All supplies in the first year are added together and divided by 365 to get the average first year in process. This is often referred to as WIP (work in process) on manufactured parts.

The average first year in process is calculated for all types of parts. However, turns calculations only use WIP in their calculations because purchase order production is the vendor's responsibility, whereas

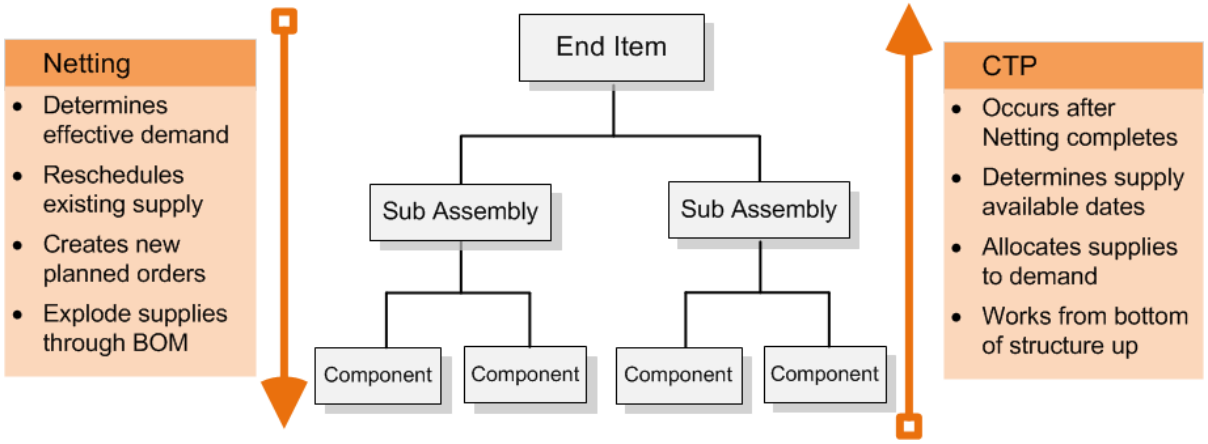
inventory of purchased material is the purchaser's responsibility. If an order starts in the first year but ends after it, only the number of days in the first year are considered.

Turns for a part can be determined using the above three factors. However, if turns across a set of parts are to be determined, each of the factors need to be normalized using StdUnitCost and summed individually before calculating the ratio. The RapidResponse data model contains a **Turns** table that performs this calculation. The **Turns** table is used to calculate turns across a set of parts. This table has a **Part** field for selecting a subset of the parts. However, a query on the **Turns** table always results in a single record that is the sum for all applicable parts. This allows the query to calculate the ratio.

CHAPTER 17: Capable-to-Promise calculations

As shown in the following illustration, the Capable-to-Promise analytic is dependent on the results generated by the Netting analytic and is then used to determine material shortages, allotment of materials, and the dates when supplies and demands are available. If, for example, an assembly cannot be built on time because one of its components is late, the assembly's available date is calculated and the late component is identified as the "gating part".

Figure 17.1 Capable-to-Promise functions



In this chapter, you'll learn about:

Capable-to-promise tables 1454
Sample data used in CTP examples 1454
Available date calculations 1456
Gating part calculations 1457
Gating constraint calculations 1458
Late supply calculations 1458
VMI parts and supply demand allocation records 1458
Allocation of supply to demand 1458
Allocating limited supplies 1464
Improve on-time quantity by reallocating late supply 1471
Incremental availability calculations 1476
CTP data examples 1488

Capable-to-promise tables

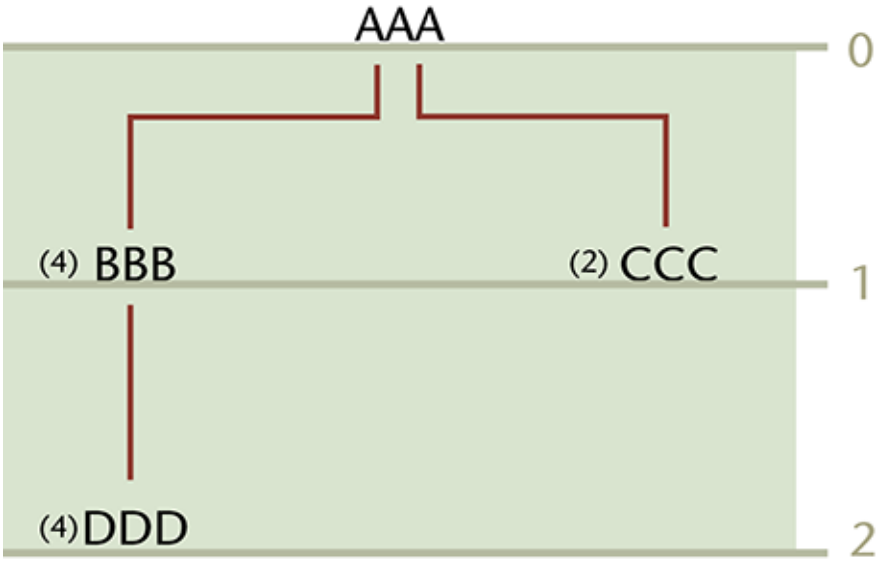
The RapidResponse data model includes tables that store capable-to-promise (CTP) information. For example, if a customer order cannot be satisfied on time because one of its components is late, a record is added to the **LateSupply** table. The following is a list of RapidResponse calculated capable-to-promise tables:

- **CTPActivity**—reports the results of constraint calculations in addition to supply and demand information reported in the planning sheet. This table also shows AvailableDate for supplies and demands. For more information about this table, see "CTPActivity table" on page 985.
- **CTPPlannedOrder**—reports planned orders that are recommended by RapidResponse. One difference between the **CTPPlannedOrder** table and the **PlannedOrder** table is the **CTPPlannedOrder** table contains an **AvailableDate** field. For more information about this table, see "CTPPlannedOrder table" on page 1000.
- **SupplyDemand**—reports single level allocations of supply to demand. For information about this table, see "SupplyDemand table" on page 1205.
- **LateSupply**—reports supplies that are late to satisfy an independent demand. For example, if a work order is late, RapidResponse creates a **LateSupply** record. For information about this table, see "LateSupply table" on page 1092.
- **WhereConsumed**—reports how a supply on a given part is apportioned to originating demands. This table also provides information about full-level pegging. For more information about this table, see "WhereConsumed table" on page 1225.
- **WhereConsumedForDemand**—reports how a supply on a given part is apportioned to originating independent demands. For more information about this table, see "WhereConsumedForDemand table" on page 1234.
- **WhereConsumedForSupply**—reports how a given component supply on a part is apportioned to its higher level driving supplies. For more information about this table, see "WhereConsumedForSupply table" on page 1252.

Sample data used in CTP examples

The following diagram shows a bill of material product structure used throughout this chapter.

Figure 17.2 Bill of material product structure



The finished product in this bill of material product structure is item **AAA**. Two components that go into item **AAA** are **BBB** and **CCC**. The Quantity Per of component **BBB** is four (it takes four units of item **BBB** to create a single **AAA** item). The Quantity Per of component **CCC** is two. Item **DDD** is a required component to build item **BBB**. The Quantity Per of item **DDD** is four. If there is a net requirement to build 100 units of item **AAA**, the following components must be available:

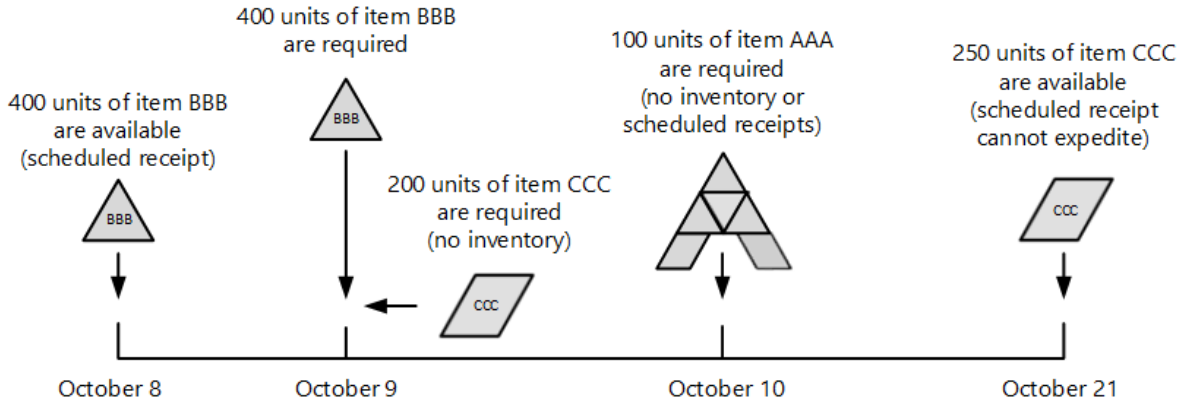
- 200 units of item **CCC**
- 400 units of item **BBB**
- 1600 units of item **DDD**

Capable-to-Promise data example

The following data example is used throughout this chapter to demonstrate Capable-to-Promise analytics. The planning calendar for all parts in this section is **Workday** and lead-time for all parts is **1**. Assume the following conditions exist:

- Independent demand for 100 units of item **AAA** is due **October 10**.
- Dependent demand for 400 units of item **BBB** is due **October 9**. However, a scheduled receipt for 400 units of item **BBB** is available on **October 8** that can satisfy the demand.
- Dependent demand for 200 units of item **CCC** is due **October 9**.
- There is a scheduled receipt for 250 units of **CCC** due **October 21**. RapidResponse is unable to expedite this supply because the value of the **SupplyStatus.RescheduleRule** field is **NonReschedulable**.
- RapidResponse is unable to create a planned order for item **CCC** because the value of the **PartSource.OrderPolicy.OrderGenerationRule** field is **NoOrders**.

Figure 17.3 Capable-to-promise sample data



Capable-to-Promise with interchangeable parts

Assume that item **CCC** is interchangeable with item **EEE**, and item **CCC** is the primary part in the interchangeable group.

A scheduled receipt for 100 units of item **EEE** is due **October 15**. RapidResponse is unable to expedite this supply because the value of the **SupplyStatus.RescheduleRule** field is **NonReschedulable**.

When the supply of item **EEE** is available, it is used to satisfy the demand for item **CCC**. However, the demand cannot be satisfied with this supply, and the remaining supply for item **CCC** or **EEE** is still required.

When the scheduled receipt for item **CCC** is received on **October 21**, it is used to satisfy the demand for item **CCC**, and 150 units of excess supply of item **CCC** remain.

Available date calculations

One of the primary purposes of the CTP analytic is to determine availability of supply and demand (in other words, the **AvailableDate**). Supply availability calculations start at the bottom of the product structure (Buy parts) and are based on a number of factors. For example, on-hand inventory is considered available at past but the demand to which it is allocated normally cannot be considered available before the **DataDate** (**Part.PlanningCalendars.DataDate.FirstDate**) for the part. Scheduled receipts for Buy parts are generally considered available on the **ScheduledReceipt.ReschedDate** which might be limited by things like constraints or reschedule rules. Similarly, planned orders for these parts are considered to be available on the **PlannedOrder.DueDate** which similarly might be limited by things like constraints or the order generation parameters such as planning time fence.

Once the supply availability is known for the Buy part, the supplies are then allocated or allotted to the demands on the Buy part (the specifics of how that allotment is done is covered in the next lesson). The demands for the Buy part will then derive their available dates from the supplies that are allotted to them. The latest available date found across all supplies allotted to a demand will determine the available date of the demand itself.

When these demands, whose availability has just been determined, are dependent demands (exploded from a parent supply), then the availability of the demand is then transferred up the product structure to the supply that the demand came from. The availability is offset by the appropriate lead times and offsets in order to determine the availability of the parent supplies. Again, in cases where the parent

supply may have exploded to several component demands, the latest availability of the dependent demands determines the availability of the parent supply.

Then supplies are matched up with the demands at the parent level in order to determine availability of the demands at the parent and explode them from there up to the next level. This process then continues up the entire product structure until the original driving demand is reached (for example, this might typically be a customer order for an end item).

Note: If constraints are used, then RapidResponse also considers constraint while calculating AvailableDate as discussed in "Constraint Analysis calculations" on page 1629.

Example (AvailableDate)

In the Capable-to-Promise data example, the supply (scheduled receipt) for item **CCC** is too late to satisfy the demand on time. This delay affects the AvailableDate of item **AAA.** Instead of the independent demand being satisfied on **October 10**, it cannot be satisfied until **October 22** (one day for lead-time). As a result, RapidResponse creates the following calculated values:

- The **AvailableDate** field on the **IndependentDemand** record is assigned **October 22**.
- The **AvailableDate** field on the **ScheduledReceipt** record is assigned **October 21**.

Note: For a complete listing of all RapidResponse tables that contain an **AvailableDate** field, see "AvailableDate" on page 1300.

Gating part calculations

The GatingPart calculation calculates what part, if any, whose availability determines the AvailableDate of an independent demand. GatingPart information is stored in the **GatingPart** field in the **IndependentDemand** table (**GatingPart** is a calculated field).

In the Capable-to-Promise data example, the supply (scheduled receipt) for item **CCC** is too late to satisfy the demand on time. Therefore item **CCC** affects the availability of item **AAA**. Instead of the independent demand being satisfied on **October 10**, it cannot be satisfied until **October 22** (one day for lead-time). As a result, RapidResponse creates the following calculated value:

- The **GatingPart** field on the **IndependentDemand** record is assigned **CCC**.

Gating part calculations with interchangeable parts

If the late part is interchangeable with other parts, the gating part can be any part in the interchangeable group. The part in the interchangeable group that has the latest supply is reported as the gating part.

In the Capable-to-Promise with interchangeable parts example, supply of item **EEE** was used to satisfy part of the demand for item **AAA**. However, because the supply of item **CCC** is the latest part in the interchangeable group, RapidResponse creates the following calculated value:

- The **GatingPart** field on the **IndependentDemand** record is assigned **CCC**.

When the supply of item **CCC** is received on **October 21**, the order is satisfied on **October 22**.

Gating constraint calculations

The GatingConstraint calculation specifies the constraint associated with a GatingPart. GatingConstraint information is stored in the **GatingConstraint** field on the **IndependentDemand** table (**GatingConstraint** is a calculated field).

Note: For information about constraint, see "Constraint Analysis calculations" on page 1629.

Late supply calculations

The Late Supply calculation determines all late supplies used by each independent demand (including forecast). The results of the late supply calculation are stored in the **LateSupply** table. When records are created in the **LateSupply** table, RapidResponse filters all **WhereConsumed** records where AvailableDate is greater than NeedDate or EffDueDate is greater than NeedDate.

In the Capable-to-Promise data example, the scheduled receipt for item **CCC** cannot satisfy the demand on time because it arrives too late. As a result, item **CCC** is considered late supply and RapidResponse creates the following calculated records:

- A **LateSupply** record for item **CCC**. The value of the **NeedDate** field is **October 9**. The value of the **AvailableDate** field is **October 21**.
- A **LateSupply** record for item **AAA**. The value of the **NeedDate** is **October 10**. The value of the **AvailableDate** field is **October 22**.

VMI parts and supply demand allocation records

Vendor Managed Inventory (VMI) parts can generate supply demand allocation (SupplyDemand) records. For VMI parts, the **Part.AvailableRule.ProcessingRule** (if populated) or **Part.Type.AvailableRule** (if the **AvailableRule** reference is null) should be set to **Ignore**.

In turn, the AvailableDate on demands is set to Past and OnTimeQty = Quantity (that is, the demand is treated as being satisfied from OnHand). The NewProcessCost and NewPurchasesCost records are calculated based on how supply is allocated to this demand (that is, the same demand calculation for regular parts). This creates WhereConsumed (and WhereConsumedForDemand) records. Because the demands are available as if satisfied from OnHand, RapidResponse doesn't calculate a NeedDate on components of a VMI part.

Note: Because lateness is defined as (**AvailableDate** - **NeedDate**), no LateSupply records can exist on or under a VMI part.

Allocation of supply to demand

The Capable-To-Promise analytic is also responsible for the actual allotment of supply to demand as reported in the calculated **SupplyDemand** table. This is a key function of the CTP analytic as the specific

allotments of supply segments to demand will, in turn, influence other calculations discussed in this chapter such as AvailableDate and GatingPart.

The actual allocation of supply to demand can be broken down into the following three main areas described in this section.

- "Collect and sort demand and supply" on page 1459
- "Allotment pre-processing" on page 1461
- "Allotment process" on page 1462

Collect and sort demand and supply

In order to allocate supply to demand, the CTP analytic first collects all supplies and demands into sorted lists for processing.

The supply list contains all available supply segments where each supply segment represents a portion of a supply available on a given date. Thus, if the entire supply becomes available on the same date, it consists of just one segment. The supply list is sorted principally by descending supply available date with many additional criteria considered after that.

The demand list is broken into two separate lists. One is sorted primarily by due date and the other is sorted primarily by priority. Both are sorted in reverse (descending) order and many additional (identical) criteria are considered in each list.

Supply sort criteria

The following table lists the criteria used by CTP for sorting supplies for use in the allotment process. Note that the sorting is done in descending order (that is, latest available supplies sort to the top of the list and earliest available supplies sort to the end of the list).

Table 17.1: Supply sort criteria for CTP allotments

	Supply Sort Criteria
1	AvailableDate
2	Demand date
3	SupplyType.SortBeforeDate
4	OrderPriority
5	ReschedDate (or Date for OnHand supply)
6	SupplyType.SortSameDate
7	SupplyStatus.SortBeforeStart
8	StartDate (calculated)
9	SupplyStatus.SortSameStart
10	RecommendedDate
11	DueDate (if defined, else DockDate + Part.DockToStockLeadTime)
12	SupplyExpiryDate

Table 17.1: Supply sort criteria for CTP allotments

	Supply Sort Criteria
13	Yielded effective quantity (if PartType.AllocationQuantityRule = Use)
14	ID1 - depends on supply type: • Location.Id (OnHand) • SupplyOrder.Id (ScheduledReceipt) • Part.Name (PlannedOrder) • PrimaryPart.Name (SubstituteSupply)
15	ID2 - depends on supply type: • Location.Warehouse.Id (OnHand) • <blank> (ScheduledReceipt) • Part.Site (PlannedOrder) • Primary.Part.Site (SubstituteSupply)
16	SupplyType.Value (or Part.Site for OnHand)
17	Line (String field on ScheduledReceipt, or Part.Name for OnHand)
18	Line (Integer field on PlannedOrder)
19	Pool
20	Model
21	Unit
22	RecordNumber (internal index value)

Demand sort criteria

The following table lists the two criteria used by CTP for sorting demands for use in the supply allotment process (the two lists are identical other than reversing the position of the OrderPriority and DueDate criteria).

Note that sorting for most items in the list is done in descending order (for example, the latest due demands and lowest priority demands sort to the top of their respective lists). The one exception to this is the EarliestExpiryDate which sorts in ascending order (earlier EarliestExpiryDates first).

Table 17.2: Demand sort criteria for CTP allotments

	ByDueDate Sort Critera	ByPriority Sort Criteria
1	Demand Date	OrderPriority (PlanningPriority + TransactionSequence)
2	OrderPriority (PlanningPriority + TransactionSequence)	Demand Date
3	EarliestExpiryDate	EarliestExpiryDate
4	DemandType.AllotmentOrder	DemandType.AllotmentOrder
5	Peg Demand Date	Peg Demand Date
6	Effective quantity (if PartType. AllocationQuantityRule is 'Use')	Effective quantity (if PartType. AllocationQuantityRule is 'Use')

Table 17.2: Demand sort criteria for CTP allotments

	ByDueDate Sort Critera	ByPriority Sort Criteria
7	RequestDate	RequestDate
8	Order.Id (Part.Name for planned dependent demands)	Order.Id (Part.Name for planned dependent demands)
9	Order.Site (Part.Site for planned dependent demands)	Order.Site (Part.Site for planned dependent demands)
10	Type.Value (from the DemandType or SupplyType driving the demand)	Type.Value (from the DemandType or SupplyType driving the demand)
11	Line (string value)	Line (string value)
12	Line (integer if from planned order)	Line (integer if from planned order)
13	Model/Unit/Pool	Model/Unit/Pool
14	DemandType (in order) • IndependentDemand • BlowThroughNewAllocation • NewTransferAllocation • NewAllocation • Allocation • BlowThroughPlannedAllocation • PlannedAllocation • PlannedTransferAllocation • IndependentForecast • DependentForecast • ForecastFromSR	DemandType (in order) • IndependentDemand • BlowThroughNewAllocation • NewTransferAllocation • NewAllocation • Allocation • BlowThroughPlannedAllocation • PlannedAllocation • PlannedTransferAllocation • IndependentForecast • DependentForecast • ForecastFromSR
15	RecordNumber (internal index)	RecordNumber (internal index)

Allotment pre-processing

Once all available supplies and demand have been collected into sorted lists, some pre-processing is performed before the actual allotment of supply to demand begins.

Specifically, any excess supply that exists for the part may need to be “trimmed” from the list. That is, if the total demand for the part is less than the total available supply, then the excess supply should be allocated to a demand source of ‘None’ (used to indicate that supply is not consumed by real demand). In order to determine which supplies should be considered as excess, the **PartType.ATPCTPRule** control setting is used. This setting has two options: “Latest” and “Earliest”, as described below.

- If **PartType.ATPCTPRule** is set to ‘Latest’, then latest available supplies are deemed to be excess. That is, the excess amount is taken from the top of the sorted supply list and allocated to the "None" demand. Because the latest available supplies are already known, these supplies can be trimmed even before the allotment process begins.
- If **PartType.ATPCTPRule** is set to ‘Earliest’ (default setting), then the earliest available supply that will not otherwise impact demand availability is deemed to be excess. However, which supply this is cannot be determined until the actual allotment process begins. Thus, the “None” is given the excess supply quantity and used during the allotment process until the excess quantity has been allocated to those “earliest” supplies.

In other cases, there might be excess demand (more total demand than there is supply available to satisfy it). Excess demand can be trimmed from the allotment list by allocating them to the 'None' supply. In order to determine which demands are excess, the sorted 'ByPriority' list is used and demands are removed from the bottom of this list until the remaining demand equals the amount of supply available to satisfy it. This ensures only the lowest priority (and latest) demands are left unsatisfied and assigned to the 'None' supply.

Note: If there are any allotment overrides specified, these are dealt with before this process begins. For more information about allotment overrides, see "Allotment overrides" on page 1853.

Allotment process

After the allotment pre-processing tasks described in the previous section have been performed, the remainder of the sorted demand list and sorted supply list should be exactly balanced (possibly including the use of 'None' supply/demand).

Starting with the latest available supply by date, the CTP analytic then iterates through the remaining segments of supply. For each supply segment, the following basic process occurs:

- The demand from the top of the "ByDueDate" list of demands that has not been fully allocated yet is retrieved. This should be the last unallocated demand by due date.
- The supply segment is compared against the demand to see if it is available "on-time" to satisfy the demand.
- If the supply is late for the demand then:
  - If there is some of the "None" demand remaining that was set up earlier, then the supply is allocated to that "None" demand. This is meant to allocate late supply segments to the "None" demand until it is fully accounted for.
  - Otherwise, if there is no remaining "None" demand, then the allotment process switches to the "ByPriority" and assigns the supply to the top demand in this list that is not yet fully allocated. This should be the lowest priority and latest due unsatisfied demand.
- However, if the supply segment is on time for the demand then it allocated to the demand that was originally selected from the "ByDueDate" list.
- If after allocating the supply segment to a given demand, the supply still has an unused portion, then that supply takes the next demand from the top of the ByDueDate list and the process repeats.
  - Otherwise, the next available supply from the top of the supply list (latest available supply segment with an unused quantity) is taken and the process repeats using that next supply.

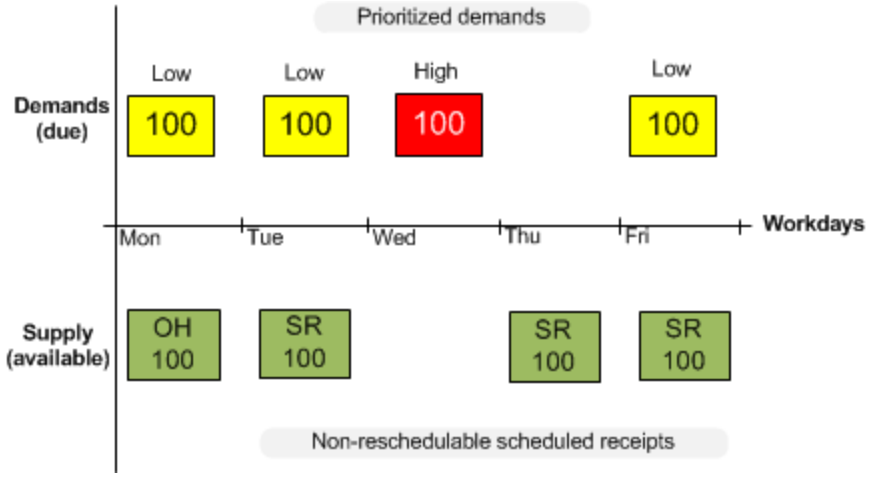
    Note that if there is no demand left to allocate to a given supply, then the rest of the supply is allocated to the "None" demand.

Thus, available supplies are allocated to demands in reverse date order unless the allocation would result in the demand being late. When that occurs, the allotment process uses the sorted 'ByPriority' list and give the late available supply to the lowest priority and latest demand instead. This ensures that earlier available supply is only "stolen" from lower priority demands and given to higher priority demands when that higher priority demand would otherwise be late.

Allotment process (example)

Suppose a set of prioritized demands for a given part, along with available input supply from on hand inventory and scheduled receipt as shown in the following illustration. Further suppose that the scheduled receipts have a **SupplyStatus.RescheduleRule** of "NonReschedulable" and demands are within that part's lead time. Thus, only the supplies shown as is can be used to satisfy these demands.

Figure 17.4 Available supplies and prioritized demands

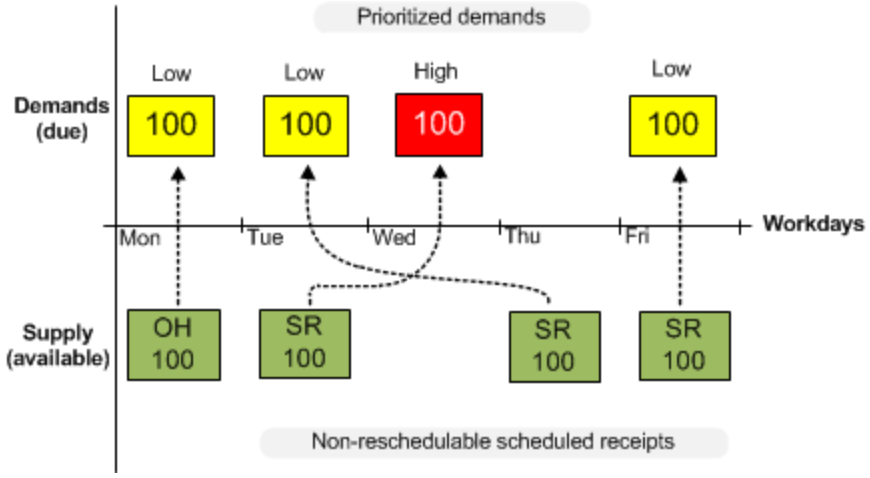


Recall that the supply allotment takes supply from the top of the sorted 'ByAvailable' list and tries to match it to demand from the top of the reverse 'ByDueDate' list, except when it cannot meet the demand on time in which case it uses demands from the sorted 'ByPriority' list. With this in mind, the basic process for allocating these supplies to the demands would be as follows:

- The latest available supply (Friday) is taken and compared against the latest due demand (Friday). The supply is able to satisfy that demand on time and so would be allocated to it.
- The latest available supply remaining (Thursday) is taken and compared against the latest due demand remaining (Wednesday). The supply is not able to satisfy that demand on time, thus the allotment process switches to use the sorted 'ByPriority' list.
  - The available Thursday supply is now allotted to the lowest priority and latest due demand remaining in the 'ByPriority' list. Thus, the Thursday supply is allotted to the Tuesday demand which will be made late as a result.
- The latest available supply remaining (Tuesday) is taken and compared against the latest due demand remaining (Wednesday). The supply is able to satisfy that demand on time and so would be allocated to it. Note that this has ensured the High priority demand will be available on time.
- Finally, the remaining available supply (Monday) is allocated to the remaining demand (Monday). The supply is able to satisfy the demand on time.

As a result, the final allocation of supply to demand would be as seen in the following illustration.

Figure 17.5 Supplies allocated to demands



Note: For complete information about order priority and its configuration, see "Order priority calculations" on page 1765.

Allocating limited supplies

The capable-to-promise analytic allocates supplies to demands. Typically, the demands are allocated supply on a first come, first served basis. The first orders in the demand horizon are allocated supply, and other demands are not allocated any until additional supply is received. However, you might want to allocate some supply to each demand so that every customer gets something on time, even if the orders are late. You can do this by changing how the supplies are allocated.

How supply gets allocated to demands can be modified for all demands with the same priority. The **OrderPriority.AllocationType** field specifies whether demands with a specific order priority are allocated supply on a first come first served, fair share, or equal share basis as described below:

- **FirstComeFirstServed**—Supply is allocated by due date and, if used, priority. In cases, where there is not enough supply to satisfy every demand on time and priorities are used, then the highest-priority demands are satisfied first and the lowest-priority demands are allocated the late supply. This is the default allocation type in RapidResponse.

  Note that supply allocations are typically made for each component part without knowledge of the allocations made on any of its sibling parts. In some cases, however, knowledge of sibling component allocations could lead to better allocations on a given component (and hence, better demand availabilities). For example, if on-time supply of a limited component is allocated to a demand which is otherwise late due to late supply of another (sibling) component, then the on-time component supply might be better directed elsewhere. This can be achieved using non-blocking allocations as discussed in "Multi-level search logic" on page 1863.

- **FairShare**—Supply is allocated so that all demands in the **AllocationCalendar** interval are allocated the earliest available supply, and larger demands in the interval receive a larger portion of that supply than smaller demands. The remainder of each demand is allocated from a later supply.

- **EqualShare**—Supply is allocated so that all demands in the **AllocationCalendar** interval are allocated supply, and each demand receives an equal amount of the earliest available supply. The remainder of each demand is allocated from a later supply.

Allocation types are generally handled at the order priority level. For example, all orders at a medium priority should spread out allocations equally, but high priority orders from a specific customer to be allocated supply first, the medium priority uses equal share allocation and the high priority order uses first come first served allocation. For more information about order priorities, see "Order priority calculations" on page 1765.

Fair share and equal share allocations are used in cases where supply is not available to satisfy every demand of a given priority level on time. The available supply is spread over demands with the same order priority in the demand horizon, within an interval defined by the **PlanningCalendars.AllocationCalendar** field. Demands with the same order priority in one interval must be fully allocated before supply is allocated to demands in other intervals. For example, if PlanningCalendars.AllocationCalendar = 'Week' and OrderPriority.Value = 'MedFairShare', then the MedFairShare demands due in one week are fully allocated before MedFairShare demands due in the next week are considered.

Fair share and equal share allocation logic can subsequently be turned off at the part-level using the **PartType.SupplyShareRule** setting. By default, this setting is set to "Use"' and fair or equal share logic is always determined from each demand's priority level. However, if **SupplyShareRule** is set to "Ignore", then fair and equal share logic is always disabled for parts of a given type. For example, priority levels using fair share logic might be being passed down from assembly demands to dependent demands on lower-level components. In this case, if fair sharing of supplies is only meant to apply to parts at a certain level in the product structure, then those parts should reference a **SupplyShareRule** of "Use" and other parts which do not require fair sharing of supplies should reference a **SupplyShareRule** of "Ignore".

Parts can also be configured to use fair or equal share logic as specified a demand's priority level, but only for demands due within a defined near-term window. In this case, the part should reference a **PartType.SupplyShareRule** value of "WithinFence" and the **Part.SupplyShareFence** field sets the number of **AllocationCalendar** intervals from **RunDate** within which a demand must be due in order to be eligible for supply sharing (demands later than this are always satisfied on a first-come, first-served basis).

Order priorities are considered only if **PartType.SupplyAllocationRule = 'ByAvailable'**. Only parts that have a part type using this value will be allocated using fair share or equal share allocation.

Allocated quantities can be rounded, where possible, depending on the value in the **Part.AllocationMultiple** field. Each allocation quantity is rounded to the nearest multiple of the value in this field. For example, if Part.AllocationMultiple = 1.0, every allocated quantity is rounded to the closest whole number. However, if Part.AllocationMultiple = 2.0, each allocated quantity is rounded to the nearest multiple of two, which will always be an even number. If Part.AllocationMultiple is zero or negative, the allocated quantities are not rounded, which can result in fractional quantities being allocated.

If the demand or the supply being allocated is fractional, the last allocation of the supply is not rounded, regardless of the value of Part.AllocationMultiple. This might happen if a part has a fractional yield quantity, or if **Part.UnitOfMeasure <> 'EA'**.

Note: For fair-sharing of supply on parts that use multi-level search logic, the **MLSAllocationRule** and related fields on the **OrderPriority** tables should be used instead. For more information, see "Fairshare of material and constraints" on page 1868.

Fair share and equal share allocation example

Assume you have the following demands and supplies in a week. In this example, Part.AllocationMultiple = 0.0, so the allocated quantities are not rounded.

Table 17.3: Supplies and demands for fair share and equal share allocation

	07-21-08	07-22-08	07-23-08	07-24-08	07-25-08
Demands	6	4			10
Supplies	6		4		10

Allocation logic begins by allocating the supply available on 07-21-08. The supply is allocated to all demands in the interval up to and including the last demand that would be late if first come first served allocation was used. In this case, the supply is allocated to the demands due on 07-21-08 and 07-22-08. The fair share and equal share allocations ensure that some portion of supply is allocated to every demand on time, and the rest allocated when the next supply is available.

Next, the supply available on 07-23-08 is allocated to the demands due on 07-21-08 and 07-22-08. No other demands require this supply, because the demand due on 07-25-08 can be completely satisfied by the supply available the same day.

Depending on whether fair share or equal share allocations are used, the following allocations are made:

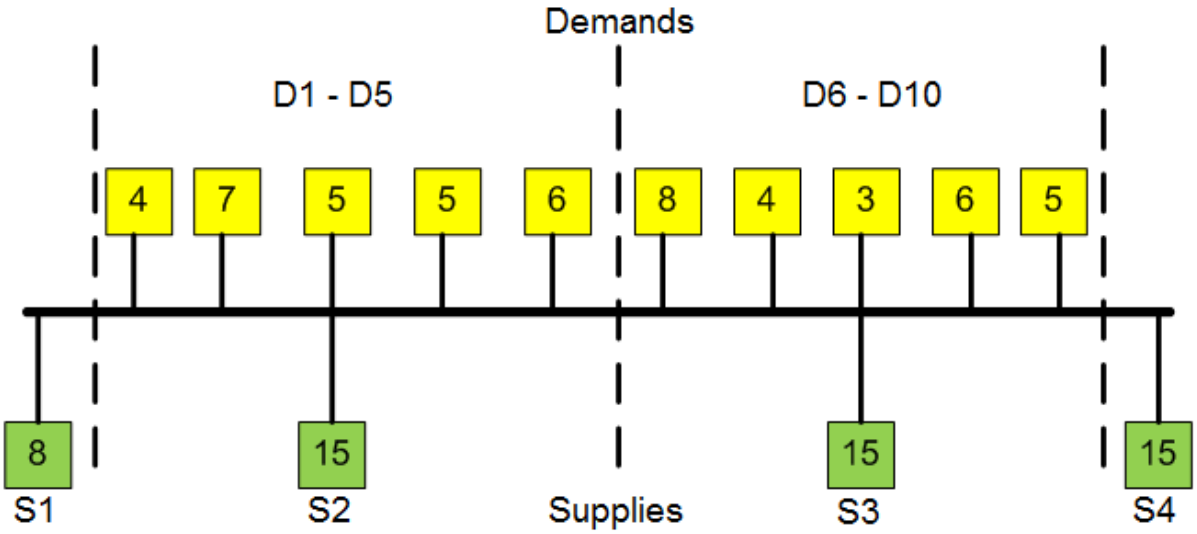
Table 17.4: Fair share and equal share allocations

Demand	Supply1	Supply2	Supply3
Fair share allocation			
07-21-08	3.6	2.4	
07-22-08	2.4	1.6	
07-25-08			10
Equal share allocation			
07-21-08	3	3	
07-22-08	3	1	
07-25-08			10

Allocation over multiple intervals

Assume you have demands and supplies spread over several weeks. The demands have PlanningCalendars.AllocationCalendar = ''Week'', and Part.AllocationMultiple = 1.0. The supplies and demands in each allocation interval are shown in the following illustration.

Figure 17.6 Supplies and demands in allocation intervals



The PlanningCalendars.AllocationCalendar value specifies that supply is allocated to demands in one week at a time. The Part.AllocationMultiple value specifies that all allocated quantities are rounded to the closest whole number.

Fair share allocation

The first supply allocated is S1, which is available before the first interval. This supply provides eight units, and is allocated to demands D1 and D2, which are due before the next supply, S2, is available. The supply is split between the two demands, with D2 receiving a greater portion because it is a larger demand. The allocations for these demands are:

Table 17.5: Fair share allocation for supply S1

Demand	Quantity	Allocated	Remaining
D1	4	3	1
D2	7	5	2

Next, supply S2 for 15 units is allocated. This supply is allocated to every demand in the interval. Demands D3, D4, and D5 are larger than D1 and D2, so they receive a greater portion of S2. The allocations for these demands are:

Table 17.6: Fair share allocation for supply S2

Demand	Quantity	Allocated	Remaining
D1	1	1	0
D2	2	1	1
D3	5	4	1
D4	5	4	1
D5	6	5	1

The next supply, S3 for 15 units, is available in the second interval. The supply is required to satisfy D2, D3, D4, and D5, so it is allocated to these demands.

With the first interval fully allocated, the demands in the second interval are then allocated, using the 11 units remaining from supply S3 and, after S3 is fully allocated, the 15 units from supply S4. The final allocations for each demand from each supply are:

Table 17.7: Summary of fair share allocations

Demand	S1	S2	S3	S4	On Time
D1	3	1			3
D2	5	1	1		5
D3		4	1		4
D4		4	1		4
D5		5	1		5
D6			3	5	0
D7			2	2	0
D8			1	2	1
D9			3	3	3
D10			2	3	2

Equal share allocation

The first supply allocated is S1, which is available before the first interval. This supply provides eight units, and is allocated to demands D1 and D2, which are due before the next supply, S2, is available. The supply is split evenly between the two demands. The allocations for these demands are:

Table 17.8: Equal share allocation for supply S1

Demand	Quantity	Allocated	Remaining
D1	4	4	0
D2	7	4	3

The next supply allocated is S2, for 15 units. This supply is on time for demands D3, D4, and D5. Supply S2 is allocated so that each demand has as close to the same amount of on time supply as possible allocated to it. The allocations for this supply are:

Table 17.9: Equal share allocation for supply S2

Demand	Quantity	Allocated	Remaining
D2	3	1	2
D3	5	5	0
D4	5	5	0
D5	6	4	2

Supply S3 for 15 units is then allocated to fill in the late quantities of demands D2 and D5. After the demands in the first interval are fully allocated, the remaining 11 units of S3 are allocated, with the allocations making an equal amount of supply on time for demands D8, D9, and D10. Demands D6 and

D7 are allocated no on time supply, but have an equal share of the earliest available supply, S3, allocated to them. After S3 is fully allocated, the 15 units of S4 are allocated to the remaining demands in the second interval. The final allocations for each demand from each supply are:

Table 17.10: Summary of equal share allocations

Demand	S1	S2	S3	S4	On Time
D1	4				4
D2	4	1	2		4
D3		5			5
D4		5			5
D5		4	2		4
D6			3	5	0
D7			2	2	0
D8			2	1	2
D9			2	4	2
D10			2	3	2

Fair and equal share allocation with expiring supply

Fair and equal share allocation can also be used with parts that have expiring supply. In this situation, the logic must ensure that expiration is respected while making the allocations. Thus, for all demands of the same priority level within a given AllocationCalendar interval, late supply can only be shared if there are other demands that are able to use to use that late supply and if those demands have allocated supply that the late demand can also use.

For all demands of a given priority within a single AllocationCalendar interval, if there is late supply but all supplies are eligible to be used to satisfy all demands, then normal fair or equal share allocations are performed as described in the previous section. That is, allocations are made as they would be for non-expiring supplies.

However, if there is late supply and at least one supply that can't be used for one or more demands, then RapidResponse cannot perform normal fair or equal share allocation without potentially assigning supply to a demand that can't use it. Instead, it first creates blocks of supplies with each block consisting of all supplies that expire on the same day. and then associates all demands whose earliest expiry date falls on or before a given expiry date are then associated with the supply block for that date. Any blocks of supplies that can be used for the same demands are then merged together.

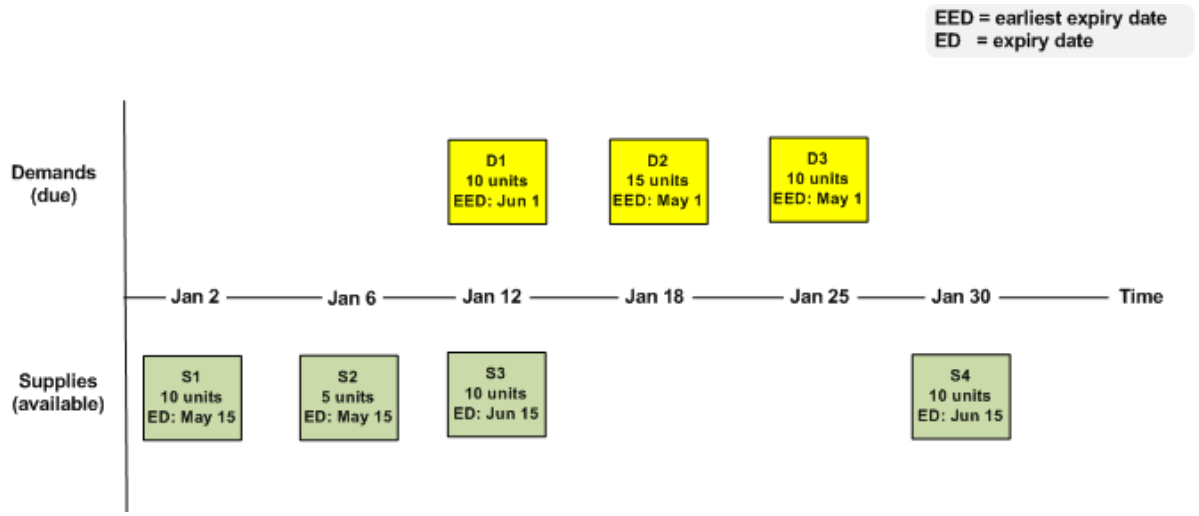
Fair or equal share allocations can then be made using the demand and supplies within each supply block. However, if the total demands and supplies in a block do not have the same quantity, then additional "carry over" supply is temporarily created to represent the difference and fair or equal share allocations are then made.

Example

Assume the set of expiring supplies and demands as shown in the following illustration. This example assumes all demands are of the same priority, and the AllocationCalendar is set to a monthly calendar.

If using the default Allocation rule of FirstComeFirstServed Allocation rule, then demand D1 would be satisfied on time by supply S3, demand D2 would satisfied on time by supplies S1 and S2, and demand D3 would be satisfied late by supply S4.

Figure 17.7 Demands and supplies with expiry dates



However, suppose a requirement to fairly share the late supply among all eligible demands using the FairShare AllocationRule. In this case, because not all supplies are eligible to be used by all demands, supplies would be grouped together into blocks based on expiry date and associated with the demands that can use them. This means that supplies S1 and S2 with an expiry date of May 15th would be grouped together and associated with demands D2 and D3 (the only demands eligible to use this supply based on their earliest expiry date). Supplies S2 and S4 with an expiry date of June 15th would then be grouped together and associated with all demands D1, D2, and D3 as they all have earliest expiry dates making them eligible to use these demands.

Once the appropriate blocks of supply and demands have been determined, allocations then begin in the first supply block. In this block, there is a total of 15 units of supply, along with 25 units of demand that are eligible to use it. This means there will be 10 units of carry over supply. Starting at the end of the current block with this carry over supply, supplies are then allocated to the demands in a ratio equal to the portion of total demand in the block that each represents. That is, forty percent of the carry over supply (4 units) is allocated to D3 and sixty percent (six units) of the carry over supply is allocated to D2. Then all five units of S2 are allocated to D3, and one unit of S1 is also allocated to D3 with the remaining 9 units going to D2. These allocations are shown in the following table.

Table 17.11: Allocations for first supply block

Demands \ Supplies	S1 (10)	S2 (5)	Carry over (10)
D2 (15)	9	---	6
D3 (10)	1	5	4

After the first supply block has been fully allocated, allocations can be made in the second supply block consisting of S2 and S4. These supplies can be used to satisfy demand D1 as well as the remaining quantities on D2 and D3 (the carry over supply from the previous block).

Allocations begin at the end of the interval with the late supply S4 which is allocated amongst the three demands in a ratio equivalent to the portion each makes up of the total demand for this block. That is D3 receives 20% of the late supply (2 units), D2 receives 30% of the late supply (3 units), and D1 receives

50% of the late supply (5 units). S3 is then allocated to the remaining unsatisfied quantities on each demand as shown in the following illustration.

Table 17.12: Allocations for second supply block

Demands \ Supplies	S3 (10)	S4 (10)
D1 (10)	5	5
D2 (6)	3	3
D3 (4)	2	2

After both blocks of supplies have been allocated, the complete allocation of supplies to demands would be as seen in the following table. Note that the late supply S4 is now shared between all three demands, and part of demand D3 can now be satisfied on time by the earlier supplies S1, S2, and S3.

Table 17.13: Late supply shared by all demands

Demands \ Supplies	S1 (10)	S2 (5)	S3 (10)	S4 (10)
D1 (10)	---	---	5	5
D2 (15)	9	---	3	3
D3 (10)	1	5	2	2

Note: When order priorities are used with parts that expire, the priorities are always assumed to be period effective. For more information about expiring supply, see "Material expiration" on page 1601.

Improve on-time quantity by reallocating late supply

As described in "Allocation of supply to demand" on page 1458, capable-to-promise logic starts from the latest available supply and tries to use it to satisfy the latest due demand with supply allotments continuing in reverse date order from there. In cases where there is insufficient supply to satisfy all demands on-time, higher priority and earlier demands are given preference and assigned supply to either make them available on-time or at least minimize their lateness.

Instead, if improving the total on-time quantity across late demands is more important than when the final late portions of those demands become available, additional logic can be applied.

Configure on-time reallocation logic

To maximize the on-time quantity for a part's late demands, the **PartType.OnTimeAllocationRule** control setting can be used. This field has the following settings:

- **Normal**— This is the default setting and does not attempt to maximize the on-time quantity of late demands. Standard allotment logic that aims to minimize lateness on demands based on factors such as priority and due date is used.
- **ImproveWithTolerance**—After the normal allotment of supply to demand, any supply portions that are late to their demand (and beyond tolerance) can potentially be reallocated with the aim of

improving the on-time quantity on other late demands. Note that this setting can make the overall available date on some late demands worse than they would otherwise be. Thus it should only be used if the availability of demands that are late beyond tolerance doesn't matter or is less important than improving the on-time quantity on late demands.

Demands that are considered late beyond tolerance are those available at or later than a specified number of forecast calendar intervals after their due date. Thus, the following fields should be set when using the "ImproveWithTolerance" setting as they are used in determining demand tolerance dates:

- **Part.UnsatisfiedDemandTolerance**—The number of forecast intervals after demand due dates to set the part's tolerance for lateness. This field is also used by partial planning logic in Netting.
- **PlanningCalendars.ForecastCalendar**—The part's referenced forecast calendar and used in determining tolerance. For example, if **ForecastCalendar** is "Month" and **UnsatisfiedDemandTolernace** is "0", a demand is considered late beyond tolerance if available after the end of the month in which it is due. Similarly, if **ForecastCalendar** is "Week" and **UnsatisfiedDemandTolerance** is "2", a demand is considered late beyond tolerance if available after the end of the second full week following its due date.

Note: If all late demands should be considered beyond tolerance and therefore eligible to have their supplies used in on-time allocation logic, then **ForecastCalendar** should be set to "Everyday" and **UnsatisfiedDemandTolerance** set to "0".

Reallocation of late supplies

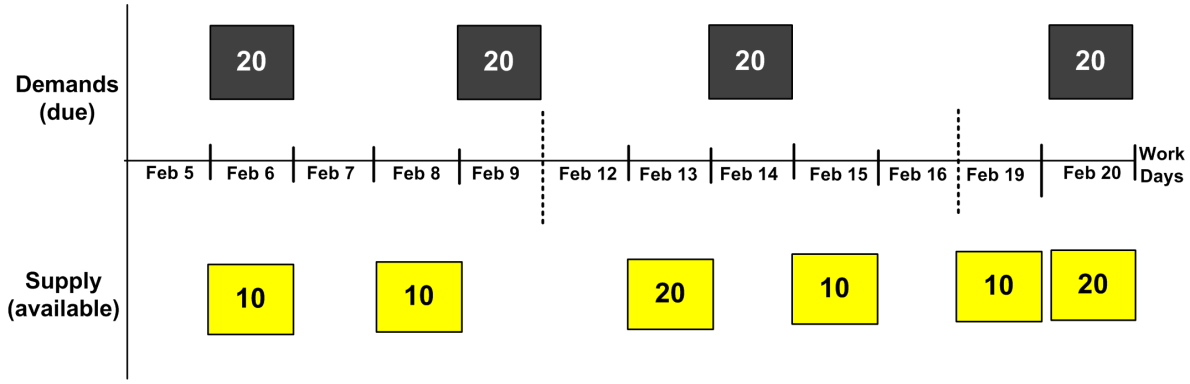
For parts with **PartType.OnTimeAllocationRule** set to "ImproveWithTolerance", the following additional logic is applied after the normal CTP supply allotment process (and immediately before distribution planning, if used):

1. Supplies that are allotted to real demands (that is, not the "None" demand) are evaluated. For all supplies where the available date exceeds the associated demand's tolerance date, the following occurs:
   - The supply is removed from the demand (no longer allotted to it).
   - The supply is added to a list of "free" supplies.
   - The demand is added to a list of "unsatisfied" demands.
2. The lists of newly free supplies and unsatisfied demands are sorted as follows:
   - Free supplies are sorted in ascending order by available date (earliest to latest).
   - Unsatisfied demands are sorted in ascending order using the standard demand sort criteria (highest priority or earliest due). The part's **PartType.CommitLevel** setting determines whether the ByDueDate or ByPriority list is used.
3. To maximize the on-time quantity of unsatisfied demands (where possible), the following occurs for each demand in the sorted list of unsatisfied demands:
   - While the demand has an unsatisfied quantity, the latest supply in the free supply list that is available on or before the demand's tolerance date is located (assuming it exists). If such a supply does exist, it is allotted to the demand (splitting it if needed) The supply is removed from the free supply list.
   - If the demand can be fully satisfied by on-time supply, it is then removed from the unsatisfied demand list.

4. To then allot supply to the remaining list of sorted unsatisfied demands, the following occurs:
   - While the demand has an unsatisfied quantity, the earliest supply in the list of free supplies is taken and allotted to the demand (splitting the supply if needed). The supply is removed from the free supply list.

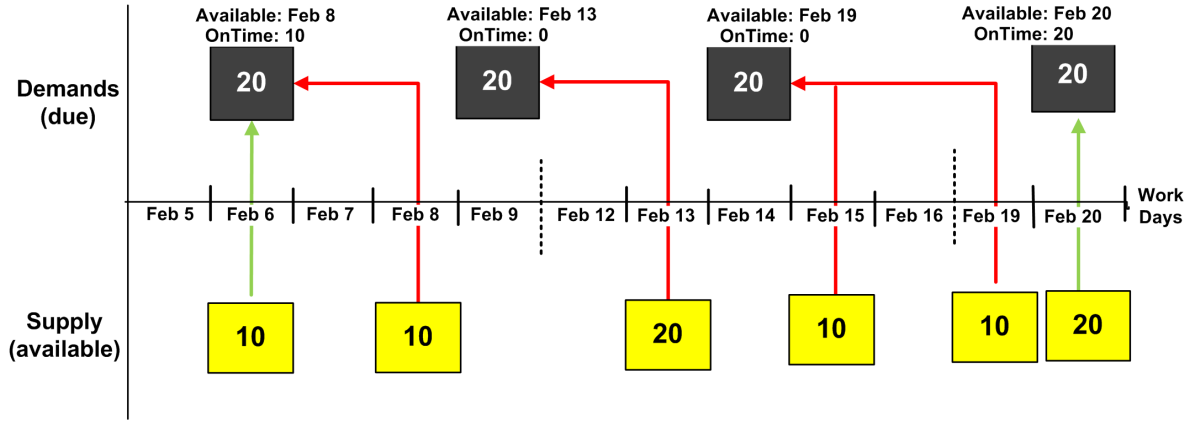
Example

Suppose the following set of demands and supplies for a given part. All demands are assumed to be of the same priority level and the part is assumed to reference the default **OnTimeAllocationRule** setting of "Normal".



The standard allotments of supply to demand would be used as shown in the following illustration. Limited supply is allocated such that the earliest demands are assigned supplies to minimize their lateness and make them available as close to their due date as possible. For example, the first two supplies are assigned to the demand due on February 6 in order to minimize its lateness (10 units are available on-time and remainder is just 2 days late). However, only 30 of the 80 units of total demand are available on time.

Figure 17.8 Supply allotments with OnTimeAllocationRule of Normal



Instead assume the part had the following settings:

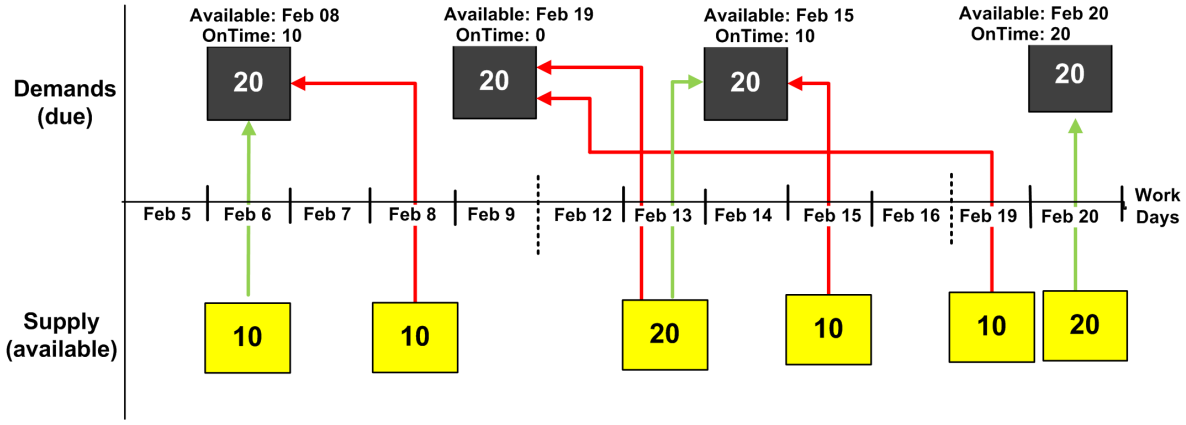
- **Type.OnTimeAllocationRule**—ImproveWithTolernace
- **PlanningCalendars.ForecastCalendar**—Week
- **UnsatisfiedDemandTolerance**—0

In this case, supplies available later than the end of the week in which their associated demand is due can be reallocated. The following occurs based on the original supply allotments shown above (dashed lines indicate the end of a Week interval):

1. The following supplies are removed from their allotted demands:
   - Supply for 20 units available on February 13 (demand on February 9 is unsatisfied)
   - Supply for 10 units available on February 19 (demand on February 14 is partially unsatisfied)
2. To maximize the on-time quantity on the unsatisfied demands, the following occurs:
   - The demand due on February 9 is checked but there is no free supply available on or before its tolerance date.
   - The unsatisfied portion of the demand due on February 14 is assigned the 10 units of free supply available on February 13 (the latest free supply available on or before its tolerance date). This satisfies the demand
3. To allot supply to the remaining unsatisfied demands, the following occurs:
   - The demand due on February 9 is assigned the remaining 10 units of free supply available on February 15.
   - The demand due on February is next assigned the 10 units of free supply available on February 19 which fully satisfies the demand.

Thus the new allotments of supply to demand would be as shown in the following illustration. Note that on-time quantity of late demands has been improved as 10 units of the demand due on February 14 are now on time (previously the entire demand was late) and 40 units of demand in total are now on time. However, the demand due on February 9 has been made an additional week later in the process.

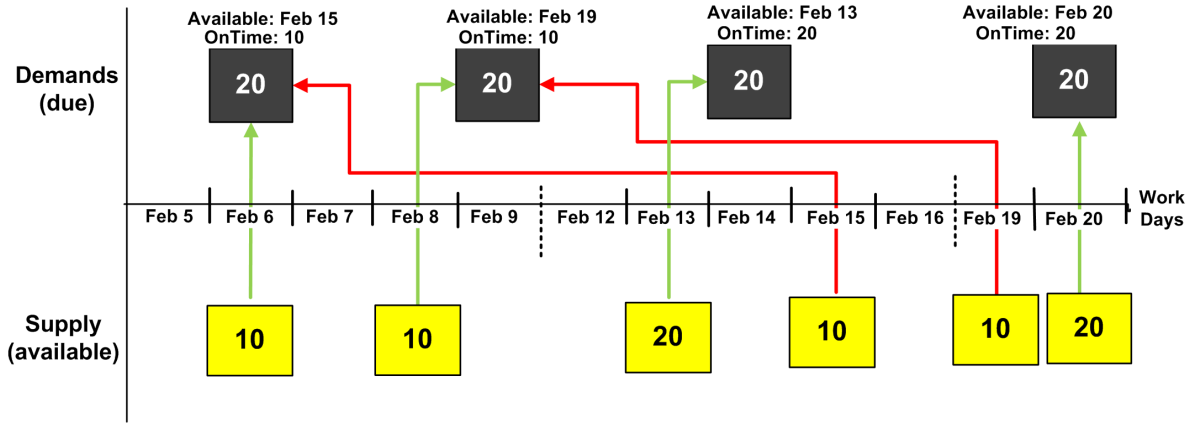
Figure 17.9 OnTimeAllocationRule of ImproveWithTolerance (weekly forecast calendar)



Instead assume the **ForecastCalendar** was set to an "Everyday" calendar. With this setting, a similar reallocation process as described above would be followed but all initial supply allotments other than the on-time supply assigned to the demand on February 20 and the 10 units of on-time supply allotted to the demand on February 6th. That is, all supply available any later than the associated demand's due date is considered to be beyond lateness tolerance.

Following the on-time reallocation process, the final allocations of supply to demand would as seen in the following illustration. Note that 60 units of demand are now seen as on time as the reallocation process improved the on-time quantity on both the demand due February 10 and the demand due on February 14 which is fully on-time. However, the final available date on the demands due on February 6 and February 9 has been made later than it initially was.

Figure 17.10 OnTimeAllocationRule of ImproveWithTolerance (everyday forecast calendar)



Limitations on other analytics

If using on-time reallocation logic, certain other analytic calculations might be disabled or not work as expected for parts that have **PartType.OnTimeAllocationRule** set to "ImproveWithTolerance".

Table 17.14: OnTimeAllocationRule limitations on other analytics

Analytic / Calculation	Impact of "ImproveWithTolerance" setting
Multi-level constraints	If two or more parts belong to the same CTP family (for example, because they are used in the same product structure and utilize the same constraint) then only those at the highest level in the family structure can make use of on-time reallocation logic. For parts below the top level in a multi-level constraint family, the **OnTimeAllocationRule** setting of "ImproveWithTolerance" is ignored (treated the same as "Normal"). Note that if a part found below the top-level in a CTP family structure has **OnTimeAllocationRule** set to "ImproveWithTolerance", it is flagged in the **Master Data Summary** worksheet of the **Data Integrity - Material** workbook.
Safety stock holds	Safety stock holding logic is applied after on-time reallocation logic in CTP. However, because safety stock holds are meant to satisfy demands later while on-time reallocations are meant to satisfy demands earlier, it is not recommended to use these two features together. Note that if a part references both a **SafetyStockPolicy.CTPHoldRule** setting of "Use" and a **PartType.OnTimeAllocationRule** settng of "ImproveWithTolerance", it is flagged in the **Master Data Summary** worksheet of the **Data Integrity - Material** workbook.
ByMRPPlan	The **PartType.SupplyAllocationRule** setting of "ByMRPPlan" should not be used together with on-time reallocation logic. The "BYMRPPlan" setting is meant to allocate supplies by their demand date.
Manual allocations	On-time reallocation logic will not attempt to improve on supply allotments that have been provided as input into tables such as **AllotmentOverride**, **InventoryTransfer**, or **SupplyAllocation**.

Table 17.14: OnTimeAllocationRule limitations on other analytics

Analytic / Calculation	Impact of "ImproveWithTolerance" setting
Fair and equal share allocation	Because fair and equal share logic is applied before on-time reallocation logic in CTP, it is possible that some fair or equal share allocations can be undone when reallocating to improve on-time quantities.
Two-date planning	Because two-date planning is applied before on-time reallocation logic in CTP, it is possible that some improvements towards meeting the demand's request date can be undone when reallocating to improve on-time quantities.
Attribute-based planning	The reallocation of supplies will only occur amongst demands that have the same unique set of attribute values.
Material expiry	On-time reallocation logic generally respects expiry dates and supplies won't be reallocated to demands if the supply expires sooner than the demand requires. However, during the reallocation process it is possible for a supply to be reallocated to a demand such that the original demand is left unsatisfied (if there are no other supplies that satisfy its expiry requirements).

Incremental availability calculations

Incremental availability is an optional setting that allows splitting of specified scheduled receipts and planned orders based on the availability of component materials and constraint. As a result, the different dates at which some portion of a supply could be available can be reported. With this information, it's possible to see when different portions of a late order will be available thus potentially improving customer order satisfaction. For example, a customer may be interested in receiving the different portions of a late order as they become available rather than waiting for the entire order to be complete.

This section discusses the following aspects of incremental availability calculations:

- Incremental availability basics—Explains and provides simple examples of how incremental availability calculations are performed and reported.
- Constraints and incremental availability—Explains the impact of incremental availability on constrained planned calculations.
- Controlling incremental availability calculations—Explains the different fields which turn incremental availability calculations on and off, and also influence how order splits are made.
- Best practices for incremental availability results—Explains how to configure RapidResponse to perform incremental availability calculations depending on different requirements you may have.
- Analytic limitations with incremental availability— Lists the limitations on other analytics when incremental availability is used.

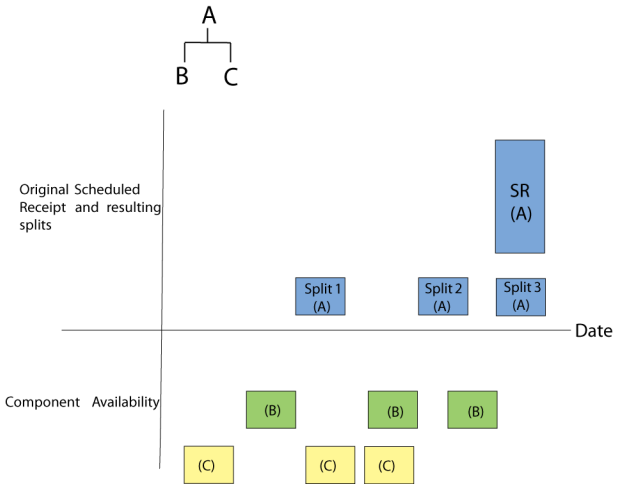
Incremental availability basics

When incremental availability is turned on, it's possible to see the different dates at which portions (or splits) of a given supply order could be ready. This involves considering the availability of an assembly's components over time (for example, some might be available now from on hand inventory and others might be available from planned orders).

RapidResponse calculates incremental availability by first determining the date at which there is some supply available for all of an assembly's components. Then, the quantity of the assembly that can be

built from the available supplies is recorded. Finally, the available date for the split supply is calculated taking into account constraints (if applicable) and assembly lead time. This process is then repeated until all supply splits for the assembly are recorded as shown in the following illustration.

Figure 17.11 Incremental availability creates splits of supply orders



Incremental availability can be especially useful where late customer orders are concerned. With incremental availability calculations turned off, only the on-time quantity (the amount available by the due date) and the available date (the date the order is satisfied) are available. With incremental availability calculations turned on, it is possible to see precisely when different portions of a late customer order could become available between the due and available dates.

Looking at incremental availability data

Incremental availability information is stored in the **SupplyDemand** table. The splitting of supplies results in additional records being added to this table. These records indicate the order(s) associated with the split, as well as details associated with each split. This includes the following SupplyDemand fields:

- **MaterialStartDate**—The date when all component materials are available for this split supply.
- **AvailableStartDate**—The date this split could theoretically start if it were an actual supply on its own (considers both material and constraint availability).
- **AllocatedQty**—The quantity of the split supply.
- **SupplyAvailableDate**—The date this split supply could be available.

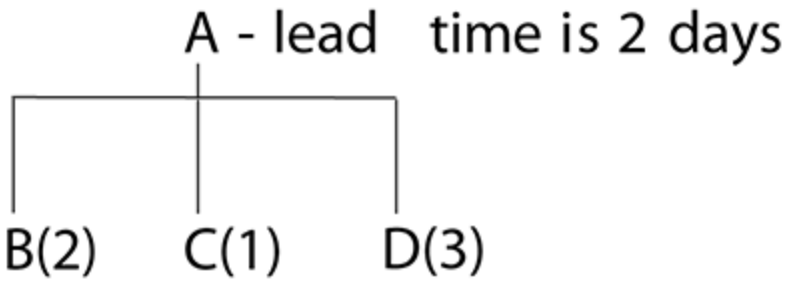
Notes:

- For more information about the **SupplyDemand** table, see "SupplyDemand table" on page 1205.
- When using incremental availability calculations, if a given supply split has **SupplyAvailableDate** <= **NeedDate**, then this split is seen by RapidResponse as being on time even if the original complete supply will be late.

Incremental availability example

This section shows a simple numerical example of how incremental availability calculations are performed. Assume a bill of material structure for product A as shown in the following illustration (the numbers in parentheses for each of the components is the quantity per assembly for that component).

Figure 17.12 Bill of material example



Now suppose a scheduled receipt for 10 units of A, which leads to allocations of 20 units for component B, 10 units for component C, and 30 units for component D respectively. Further, assume that the availability of these components is as outlined in the following table.

	6/10	6/17	6/18	6/19	6/21	6/22	6/23	6/24
B	4				10		6	
C		5		5				
D			15			12		3

This would result in five splits for the order in the **SupplyDemand** table with details as shown in the following table.

AllocatedQty	MaterialStartDate	AvailableDate
2	6/18	6/20
3	6/21	6/23
2	6/22	6/24
2	6/23	6/25
1	6/24	6/26

Note that in this example if incremental availability calculations were turned off, there would only be one SupplyDemand record for 10 units with **MaterialStartDate** of 6/24 and **AvailableDate** of 6/26.

Constraints and incremental availability

If your company is using constraints, incremental availability calculations can influence the Constraint Analysis results. Supplies are first split by incremental availability of component materials. These splits are then loaded separately onto constraints. Potentially, this means that constraint could be used earlier or more efficiently than is possible without incremental availability of component materials. In periods where there is insufficient constraint available, further splitting of the split supply can be done in order to find available constraint.

Controlling incremental availability calculations

RapidResponse provides control settings that turn incremental availability calculations on and off, and also influence the dates and quantities associated with the order splits resulting from incremental availability calculations.

Note: For examples of how to configure incremental availability calculations for some common requirements you may have, see "Best practices for incremental availability results" on page 1484.

Turning incremental availability calculations on and off

Incremental availability is turned on and off at the part level. This is done by setting the **Part.IncrementalRule** field for the part. **Part.IncrementalRule** references the **IncrementalRule** table which contains all incremental availability settings that have been defined. For example, you may want to define separate incremental rule settings for parts associated with particular customers.

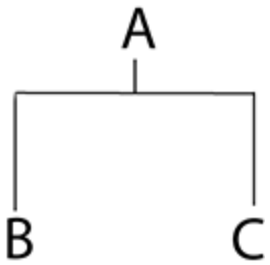
Each of the settings in the IncrementalRule table has its **ProcessingRule** field set to either **On** (incremental availability calculations are performed on supplies of this part) or **Off** (incremental availability calculations are not performed on supplies of this part). For more information, see "IncrementalRule table" on page 741.

If necessary, it is possible to turn incremental availability off for a part on a supply-by-supply basis (for example, you may have turned incremental availability calculations on for a part, but do not want incremental availability calculations for its scheduled receipts). This is done by setting the **SupplyStatus.IncrementalSplit** field to **Off** for the supply. For more information, see "IncrementalSplit" on page 909.

Dependent demands and incremental availability calculations

While the previous section outlined the fields used to turn incremental availability on and off for parts and orders, there are some special considerations where dependent demands are concerned. Because dependent demands are a direct result of supplies at the assembly level, whether or not incremental availability calculations are performed for them is determined by the supply that generates the dependent demand.

Figure 17.13 Simple product assembly example



Further, assume that A has its IncrementalRule setting set to ON, while B and C have it set to OFF. A new order for A would calculate incremental availability data (due to respecting A's IncrementalRule setting). However, resulting dependent demands for B and C would also calculate incremental availability data (regardless of their IncrementalRule settings). This is because it is only the parent supply that is of concern, and not the setting on the dependent demand parts themselves.

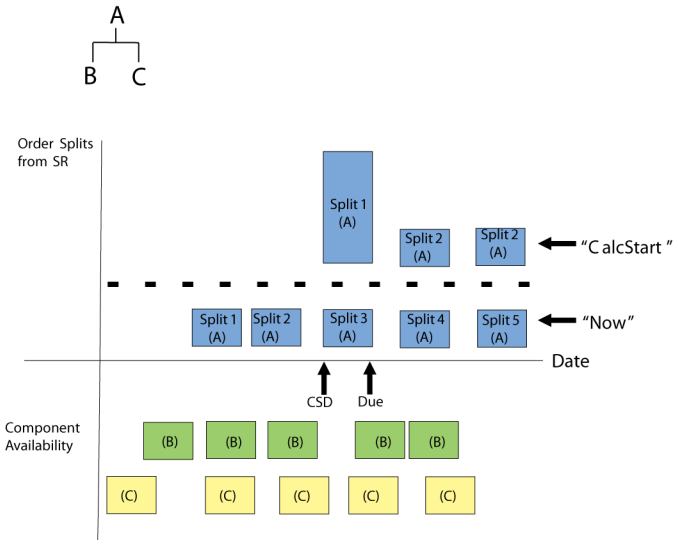
Defining when order splits can occur

The **SupplyStatus.AvailableDateLimit** field can be used to affect the dates at which splits of an order can occur. This field has three options with the following impacts:

- **Now**—Each order split is treated as its own record. In cases where the supply order is unconstrained, the order can be built as soon as materials are available. However, in cases where the supply order is both constrained and there is sufficient constraint available to complete it on time, then it will not start consuming constraint before its calculated start date. If there is sufficient constraint to complete it on time, then the order may be built once materials are available.
- **CalcRequestStart**—Build the supply based on its **CalcRequestStartDate.**Supply cannot be available earlier than **CalcRequestStartDate** + lead time, and constraint cannot be allocated any earlier than **CalcRequestStartDate** converted to a constraint date. If the calculated start date is earlier than this date, then it becomes the available date limit instead ("CalcStart" setting is used).
- **CalcStart**—All order splits on or before the supply's CalcStartDate are combined into one record, and the MaterialStartDate for this record is the date associated with the latest of those splits. Each order split after the supply's CalcStartDate is treated as its own record, and the MaterialStartDate for each record is the date of that split. This option might be used when you are only concerned with the incremental availability of the late portions of orders.
- **Due**—All order splits on or before the supply's EffStartDate are combined into one record, and the MaterialStartDate for this record is the date associated with the latest of those splits. Each order split after the supply's EffStartDate is treated as its own record, and the MaterialStartDate for each record is the date of that split.

The following illustration shows how order splits might appear differently when **Now** is used as compared to when **CalcStart** is used (assuming an unconstrained supply order).

Figure 17.14 Incremental splits using "CalcStart" and "Now" AvailableDateLimit options



Note: For more information about this field, see "AvailableDateLimit" on page 906.

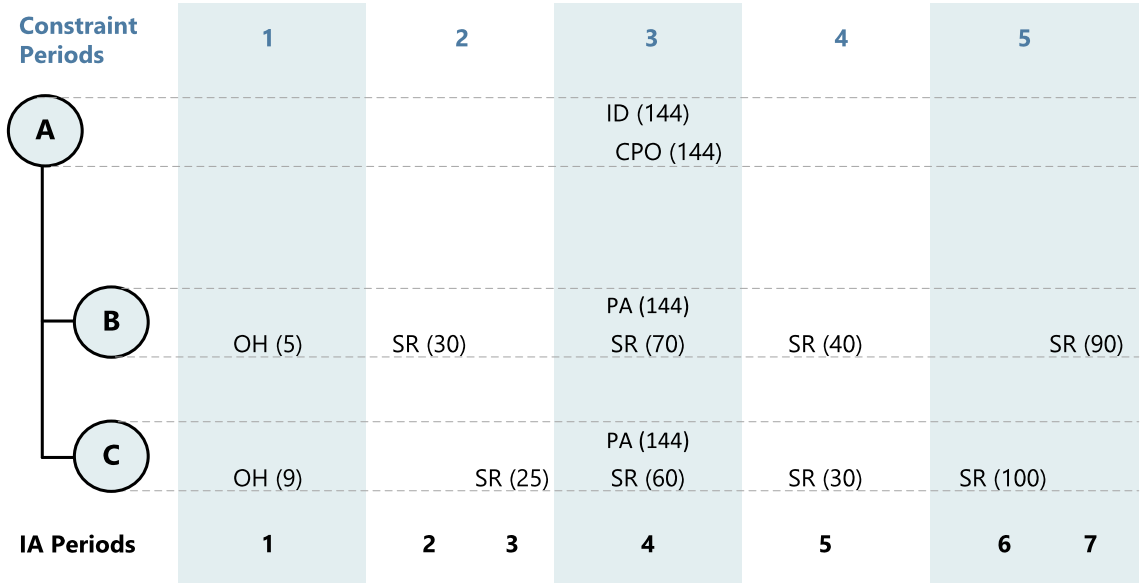
Respecting order policy rules

When incremental availability calculations are turned on, component availability can potentially cause a supply order to be split into a large number of very small splits across a wide range of dates. To avoid this, settings on the **PartSourceType** table can be used to force incremental availability splits to respect order policy rules such as lot size and minimum order quantity. The following two fields are used:

- **PartSourceType.ConstraintMultipleRule**—By setting this value to 'Use', material order splits are forced to respect lot size (ensures splits are multiples of the value in **PartSource.MultipleQty**).
- **PartSourceType.ConstraintMinimumRule**—By setting this value to 'Use', material order splits are forced to respect the minimum order quantity (ensures splits are at least the value of **PartSource.MinimumQty**). In cases where **MinimumQty** is not a multiple of the value in **MultipleQty**, then **MinimumQty** takes precedence (each split is at least the value of **MinimumQty** rounded up to the next multiple of **MultipleQty**).

When these fields are used, incremental availability calculations accumulate partially available components until there are enough for order splits that satisfy the order policy rules. As an example, assume demand for part A and the availability of its components B and C as shown in the following illustration. Further assume that for part A the value of **PartSource.MinimumQty** is '30', and the value of **PartSource.MultipleQty** is '12'.

Figure 17.15 Component availability



- **ID**—Independent demand
- **CPO**—CTP planned order
- **PA**—Planned allocation

- **OH**—On hand
- **SR**—Scheduled receipt
- **IA**—Incremental availability

Based on the component availability shown in the illustration above, the following table shows the order splits that would result from different combinations of settings in the **ConstraintMultipleRule** and **ConstraintMinimumRule** fields. Note that in this case, setting either or both of these fields to 'Use' changes the size of the various order splits and also results in fewer splits.

Table 17.15: Incremental availability splits based on order policy settings

Constraint Multiple Rule	Constraint Minimum Rule	Period 1	Period 2	Period 3	Period 4	Period 5	Period 6
Ignore	Ignore	5	4	25	66	30	14
Use	Ignore	0	0	24	72	24	24
Ignore	Use	0	0	34	66	30	14
Use	Use	0	0	30	60	36	18

Note: For more information about the fields used to control the size of splits resulting from incremental availability calculations, see "ConstraintMultipleRule" on page 800 and "ConstraintMinimumRule" on page 799.

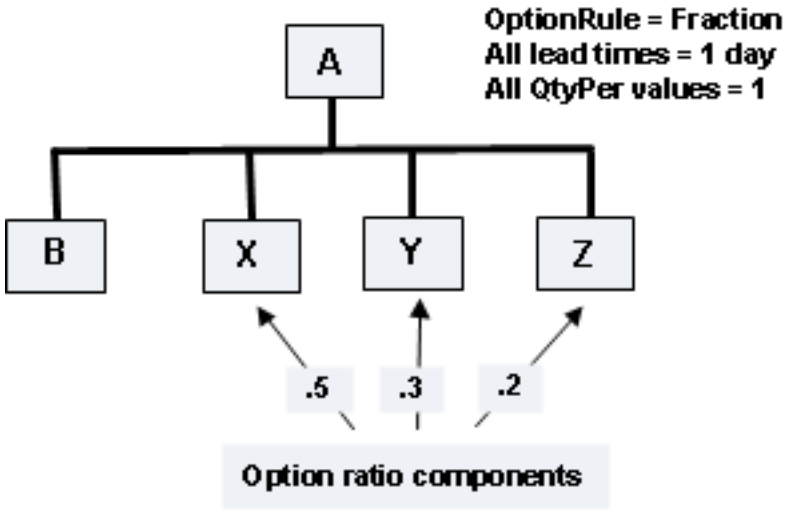
Incremental availability with BOM option ratios

As discussed earlier in this section, incremental availability calculations can be enabled or disabled at the part level using the **Part.IncrementalRule** reference. For parts configured to use BOM option ratios, incremental availability is further configurable based on how availability of the optional components is meant to affect the assembly's availability.

By default, incremental availability calculations assume that all option components are required to build one unit of the parent supply. That is, in order to build an incremental portion of the total assembly supply, there must be available supply of each option component. The amount of each option component used to build an incremental amount of the parent supply must then be proportional to their option ratios (as defined in the **BillOfMaterial.OptionRatio** field).

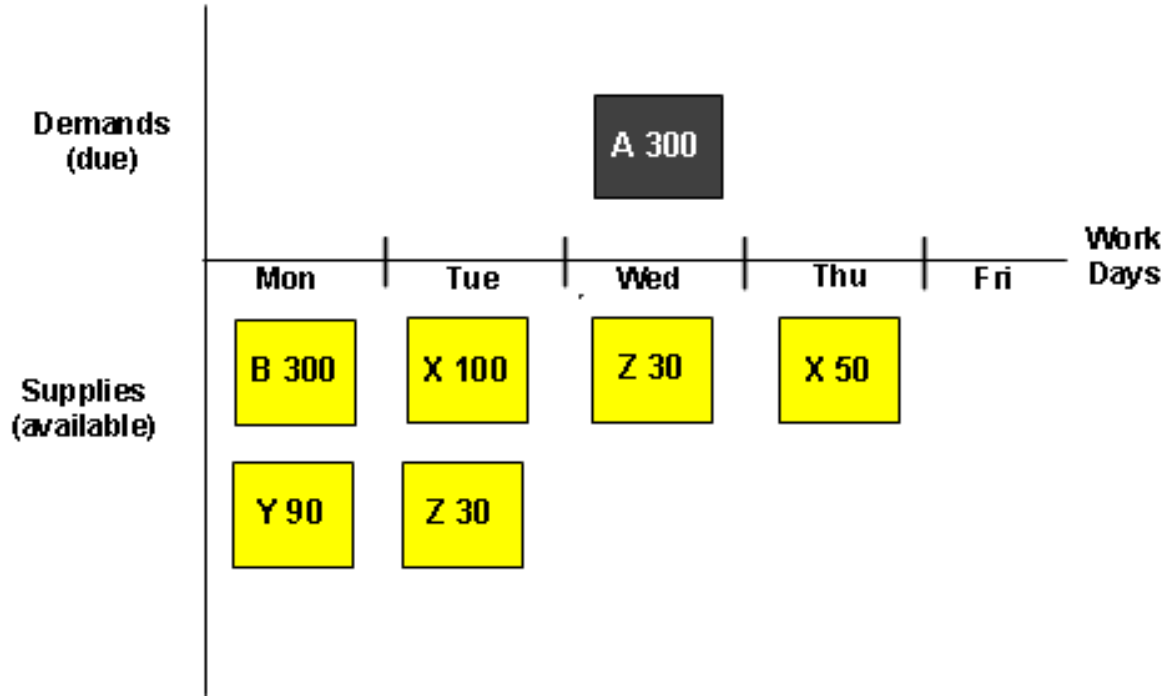
For example, assume a simple BOM structure using BOM option ratios as shown in the following illustration, where Assembly A can be configured to order using a standard base component B, and one of three optional components X (50%), Y (30%), and Z (20%).

Figure 17.16 Bill of material with option ratio component



Further assume a demand of 300 units for assembly A, which drives dependent demand of 300 units to component B, 150 units of component X, 90 units to Y, and 60 units to Z. Also assume the set of component supplies required to satisfy this demand available as shown in the following illustration.

Figure 17.17 Assembly demand and component supplies



In this scenario, the incrementally available portions of the demand for A would be reported as follows**:**

- **150 units available on Wednesday (on time)**—built using a mix of 75 units of X, 45 units of Y, and 30 units of Z (along with 150 units of B).

- **50 units available on Thursday (late)**— built using a mix of 25 units of X, 15 units of Y, and 10 units of Z (along with 50 units of B).
- **100 units available on Friday (late)**—build using a mix of 50 units of X, 25 units of Y, and 20 units of Z (along with 100 units of B).

Alternatively, RapidResponse can be configured so that only supply of one option component, within a grouping of related option components, is required to build a unit of the assembly supply. This is done by adding records to the **SubstituteGroup** table which are then used to group associated option components together. There should be one record for each group of related option components under an assembly, and each of these should have a **SubstituteGroupType.ProcessingRule** value of "BomOption". Each BillOfMaterial record that associates an assembly with an option ratio should then reference the appropriate SubstituteGroup record.

For example, in the previous example, different availabilities would be reported if the BillOfMaterial records associating assembly A with each of the option components X, Y, and Z all referenced to the same record in the SubstituteGroup table and that record had a Type.ProcessingRule of "BomOption". In this scenario, the incrementally available portions of the demand for A would be reported as follows**:**

- **220 units available on Wednesday (on time)**—built using all supplies of option components X, Y, and Z available on Monday and Tuesday (along with 220 units of B).
- **30 units available on Thursday (late)**—built using the supply of option component Z available on Wednesday (along with 30 units of B).
- **50 units available on Friday (late)**—built using the supply of option component X available on Thursday (along with 5o units of B).

Best practices for incremental availability results

This section provides suggestions on how to configure incremental availability calculations for certain requirements you may have. The following are discussed:

- "No incremental availability information required" on page 1484
- "Incremental availability information on all parts and their supplies" on page 1485
- "Incremental availability throughout a product structure" on page 1485
- "Incremental availability and components used in multiple product structures" on page 1485
- "Propagating incremental availability up through a product structure" on page 1486
- "Incremental availability for planned orders only (do not split scheduled receipts)" on page 1486
- "Incremental availability after supply due date only" on page 1487
- "Incremental availability for specific part sources" on page 1487

No incremental availability information required

If you do not require incremental availability calculations, no changes are required to the settings that install with RapidResponse. By default, **IncrementalRule.ProcessingValue** is set to "Off" for all parts, and **SupplyStatus.IncrementalRule** is set to "FromPart" for all supplies. If you are currently generating incremental availability calculations and want to turn them off everywhere, you should restore these defaults.

Incremental availability information on all parts and their supplies

It is not recommended that you generate incremental availability calculations for all parts as this increases the memory and execution time of the Capable-to-Promise and Full-Level Pegging analytics. However, there are some situations where you may want to do so (for example, as part of a "what-if" scenario). To generate incremental availability for all parts, they should have their **IncrementalRule.ProcessingRule** set to "On". The default setting on **SupplyStatus.IncrementalRule** (that is, "FromPart") ensures that incremental availability calculations are generated for all supplies.

Incremental availability throughout a product structure

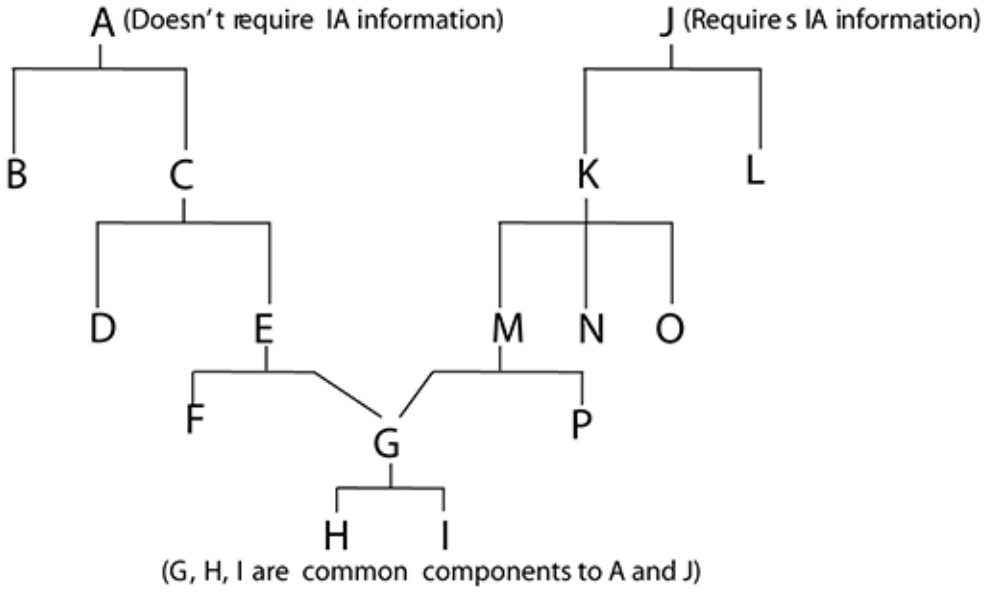
In many cases, having incremental availability calculations turned on for a single level in a product structure can provide some beneficial information. In general, though, if you are interested in incremental availability for a given product, you should set not only its **Part**.**IncrementalRule.ProcessingRule** to "On", but also ensure that **Part**.**IncrementalRule.ProcessingRule** is set to "On" for each of its components. This ensures the complete flow of material availability throughout the product structure.

Incremental availability and components used in multiple product structures

Often the same component is used in multiple product structures, which may have different requirements for incremental availability calculations. This can be managed by setting the **IncrementalRule.ProcessingRule** setting throughout the different product structures as necessary.

For example, in the illustration below, component G is used in both product structure A (which doesn't require incremental availability calculations), and product structure J (which does require incremental availability calculations). In this case, J and all its components (including G, H, and I) should have their **IncrementalRule.ProcessingRule** set to "On". This ensures the complete flow of incremental availability calculations throughout the product structure. On the other hand, A and components B, C, D, E, and F should have their IncrementalRule.ProcessingValue set to "Off". This ensures incremental availability calculations are not performed on A and also prevents incremental availability calculations on G, H, and I from propagating up its product structure.

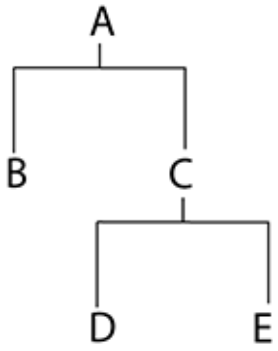
Figure 17.18 Common components and incremental availability



Propagating incremental availability up through a product structure

In some cases, you may have incremental availability data on components that you want to propagate up a products structure. For example, in the diagram to the right, assume that D and E are purchased components identified as parts for which incremental availability calculations would prove beneficial. In order to ensure potential supply splits on these parts are providing the maximum benefits, they need to be propagated up the product structure. In this example, that means that, A and C also need to have incremental availability calculations turned on (in addition to D and E).

Figure 17.19 Product structure example



Incremental availability for planned orders only (do not split scheduled receipts)

Because scheduled receipts represent a firm commitment to build an order, you may not want to split them. However, you may still be interested in seeing how planned orders can be split based on component material availability. The following indicates how to calculate incremental availability calculations for a part's planned orders, but not its scheduled receipts.

1. Ensure the part's **IncrementalRule.ProcessingRule** value is set to "On" (that is, incremental availability is turned on for the part).
2. Identify all **SupplyStatus** records referenced by the part's scheduled receipts.
3. Do one of the following:
   - If the records identified in Step 2 are used only to set the status for scheduled receipts, then set their **IncrementalSplit** value to "Off" (this prevents the generation of incremental availability calculations for the receipts).
   - If the records identified in Step 2 are also used to set the status for planned orders (from **PartSourceType.PlannedOrderSupplyStatus**), then create copies of these records. The copies should have a unique name and have their **IncrementalSplit** value set to "Off".
4. If you created new **SupplyStatus** records in Step 3, ensure the necessary scheduled receipt records are modified to point to them (from **ScheduledReceipt.Status**).

Incremental availability after supply due date only

In some cases, you may not be interested in incremental availability data before an order's due date. That is, you might only be interested in seeing incremental availability for any late portions of a supply. In these cases, the following settings can be used:

- **Part.IncrementalRule.ProcessingRule**— "On"
- **SupplyStatus.IncrementalSplit**— "FromPart"
- **SupplyStatus.AvailableDateLimit**— "CalcStart"

To see full incremental availability data, both before and after due date, change the **SupplyStatus.AvailableDateLimit** value to "Now".

Note: For an illustration showing the impact of using "CalcStart" as opposed to "Now", see "Defining when order splits can occur" on page 1480.

Incremental availability for specific part sources

Incremental availability calculations are turned on and off at the part level. If necessary, it is also possible to restrict incremental availability calculations to specific part sources.

By default, if incremental availability calculations are turned on for a given part, they are also on for its part sources, and all planned orders and scheduled receipts using the source can be split based on incremental availability. Incremental availability can subsequently be turned off for a given part source by manipulating the **SupplyStatus.IncrementalSplit** field as follows.

1. For all scheduled receipts against the part source, set the **Status.IncrementalSplit** value to **Off** (this ensures scheduled receipts using the part source ignore incremental availability).
2. For the part source, ensure the **Type.PlannedOrderSupplyStatus.IncrementalSplit** value is set to **Off** (this ensures that planned orders generated against the source ignore incremental availability).

Analytic limitations with incremental availability

A small number of other analytic calculations might be disabled, or not work as expected, for parts set up to use incremental availability logic.

Table 17.16: Incremental availability limitations on other analytics

Calculation / Setting	Impact of incremental availability
Kanban	Incremental availability logic is not applied to kanban parts. This is because incremental availability has the effect of splitting supplies as their component materials become available, and kanban bins are meant to be processed as a whole unit.
Feature BOM	Incremental availability calculations are always forced on for configurable end items (regardless of the part's **IncrementalRule.ProcessingRule** setting). As such demands due on the same date and satisfied by the same planned order, might have different available dates.
Fixed Constraint	Calculated results might not always be as expected for parts that have incremental availability enabled and are provided by part sources that use fixed constraint. During Netting calculations, each supply for such parts will consume the fixed amount of constraint (for example, representing a setup or cleanup operation), however during CTP calculations each incrementally available portion of that supply will consume the fixed amount of constraint. This difference between how Netting and CTP consume fixed constraint can potentially lead to shortages for overloaded constraints.

CTP data examples

The following CTP examples relate to the bill of material product structure specified in the "Sample data used in CTP examples" on page 1454.

- Matching supply to demand
- Material is allotted to demand based on order priority
- A component is affected by constraint

Matching supply to demand

RapidResponse creates records for the following calculated tables when supply is allocated to demand: **SupplyDemand** and **WhereConsumed**. Assume the following conditions exist:

- Independent demand for 100 units of item **AAA** is due **October 10.** There are no **OnHand** or **ScheduledReceipt** records for item **AAA**.
- Dependent demand for 400 units of item **BBB** is due **October 9**. There are no **OnHand** or **ScheduledReceipt** records for item **BBB**.
- Dependent demand for 1600 units of **DDD** is due **October 8**. However, there are 1600 units in inventory that can satisfy the demand (the **Quantity** field in the **OnHand** record for item **DDD** is 1600 and the value of the **OnHand.Type.ProcessingRule** field is **Nettable**).
- Dependent demand for 200 units of item **CCC** is due **October 9**.
- There is a **ScheduledReceipt** record for 300 units for item **CCC** due **October 21**. RapidResponse is able to expedite this supply because the value of the **SupplyStatus.RescheduleRule** field is **Reschedulable** and the value of the **SupplyType.ProcessingRule** is **Reschedulable**.

Result

RapidResponse expedites the **ScheduledReceipt** record to **October 9**. A **SupplyDemand** record is created that specifies the scheduled receipt for item **CCC** has been allocated to satisfy the dependent

demand for item **CCC**.

RapidResponse creates a **WhereConsumed** record that shows the scheduled receipt being allocated to the dependent demand for item **CCC**. The **WhereConsumed** record also shows additional information such as the part that is driving demand (in this example, item **AAA** is driving demand).

Because there are enough items of **DDD** in inventory to create 400 units of **BBB**, RapidResponse creates a new planned order for 400 units of item **BBB**. RapidResponse assigns the value **October 9** to the **AvailableDate** field on the **CTPPlannedOrder** record.

Because there are enough components to satisfy the demand of 100 units of item **AAA**, RapidResponse creates a planned order for 100 units of item **AAA**. The **AvailableDate** field on the **CTPPlannedOrder** record is assigned **October 10**. The **AvailableDate** field on the **IndependentDemand** record is also assigned **October 10**.

Material is allotted to demand based on order priority

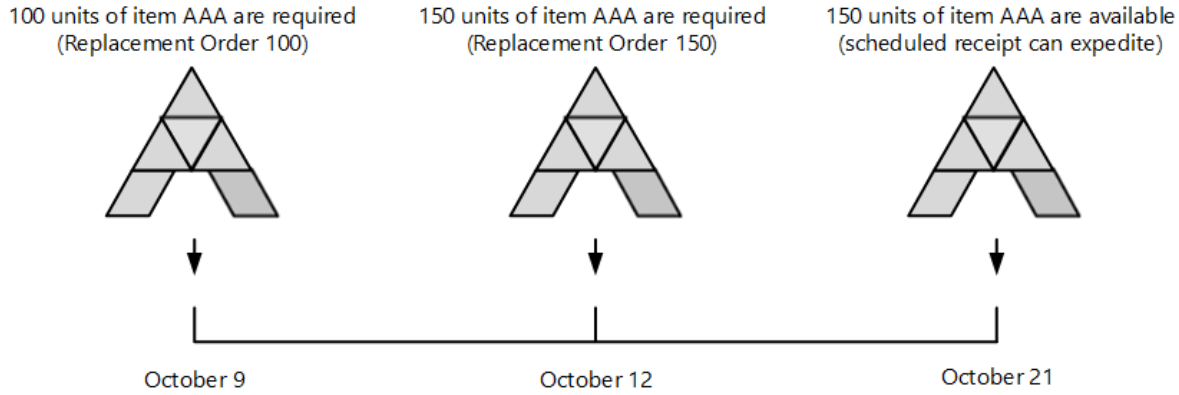
In this example, there are two separate independent demand records for item **AAA.** The first independent demand for 100 units is meant to increase inventory for replacement parts and is due **October 9**. The second independent demand represents a customer order for 150 units and is due **October 12**.

Data example

Assume the following conditions exist:

- Independent demand for 100 units of item **AAA** is due **October 9.** The value of the **Order.Id** field is **ReplacementOrder100**.
- Independent demand for 150 units of item **AAA** is due **October 12.** The value of the **Order.Id** field is **CustOrder500**.
- There are no **OnHand** records for item **AAA**.
- A **ScheduledReceipt** record for 150 units for item **AAA** is due **October 21**. RapidResponse is able to expedite this supply because the value of the **SupplyStatus.RescheduleRule** field is **Reschedulable** and the value of the **SupplyType.ProcessingRule** field is **Reschedulable**.
- RapidResponse cannot recommend new planned orders for item **AAA** because the value of the **PartSource**.**OrderPolicy.OrderGenerationRule** field is **NoOrders**.

Figure 17.20 OrderPriority data



Both independent demands have the same priority

If both **IndependentDemand** records have the same **OrderPriority** value, RapidResponse satisfies the demand with the earlier due date. Assume the following conditions exist:

- The value of the **OrderPriority** field is **Medium** on the **IndependentDemand** record where **Order.Id** is **ReplacementOrder100**.
- The value of the **OrderPriority** field is **Medium** on the **IndependentDemand** record where **Order.Id** is **CustOrder500**.

Because both independent demands have the same priority, RapidResponse expedites the **ScheduledReceipt** record to **October 9** and satisfies the independent demand where **Order.Id** is **ReplacementOrder100**. The **AvailableDate** field on this **IndependentDemand** record is assigned **October 9**.

The independent demand where **Order.Id** is **CustOrder500** is not satisfied because there is not enough supply. As a result, RapidResponse assigns **Future** to the **AvailableDate** field on this **IndependentDemand** record.

This example is not efficient because the demand that is satisfied is meant for inventory while a customer order is not filled. A better solution is to assign an appropriate **OrderPriority** value to each independent demand.

Independent demands have different priorities

If both **IndependentDemand** records have different **OrderPriority** values, RapidResponse satisfies the demand with the higher **OrderPriority** value. Assume the following conditions exist:

- The value of the **OrderPriority** field is **Low** on the **IndependentDemand** record where **Order.Id** is **ReplacementOrder100**.
- The value of the **OrderPriority** field is **High** on the **IndependentDemand** record where **Order.Id** is **CustOrder500**.

RapidResponse expedites the **ScheduledReceipt** record to **October 9**. However, RapidResponse uses the supply to satisfy the **IndependentDemand** record that contains the High **OrderPriority** value. The **AvailableDate** field on this **IndependentDemand** record is assigned **October 9**.

The other independent demand cannot be satisfied because there is not enough supply. As a result, RapidResponse assigns the value **Future** to the **AvailableDate** field on the **IndependentDemand** record with the Low **OrderPriority** value.

For more information about order prioritization, see "Order priority calculations" on page 1765.

Material is allotted to demand based on sequence

After RapidResponse considers the **OrderPriority** value, it considers the value of the **TransactionSequence** field (this field appears on the **IndependentDemand** and **ScheduledReceipt** tables). RapidResponse allocates material to a demand record with a smaller **TransactionSequence** value.

The value of the **OrderPriority.MaintainCommitments** field must be **Y** for RapidResponse to allocate material based on the **TransactionSequence** value. If the value of **MaintainCommitments** is **N**, RapidResponse does not consider **TransactionSequence**.

Because the **TransactionSequence** field is an input field, it is assigned a value when data is imported into RapidResponse. When the **OrderPriority** field on a **ScheduledReceipt** or **IndependentDemand** record is modified, RapidResponse can be configured to automatically update the **TransactionSequence** field. If this automatic modification is made, the field is then set to be higher than any other **TransactionSequence** value in the RapidResponse data model.

Assume the highest **TransactionSequence** value in the RapidResponse data model is 100. If the value of the **TransactionSequence** field of an **IndependentDemand** record is 50 and its **OrderPriority** field is updated, the **TransactionSequence** field is incremented to 101.

Caution: It is strongly recommended that you do not modify the value of the **TransactionSequence** field.

Note: In order for a priority change on a scheduled receipt or independent demand to trigger an automatic update of the corresponding **TransactionSequence** field, the **TransactionSequenceEnabled** configuration record must be set as discussed in "Enabling automatic maintenance of TransactionSequence values" on page 2207.

Data example

Assume the following conditions exist:

- Independent demand for 100 units of item **AAA** is due **October 9.** The value of the **OrderPriority** field is **Medium** (the value of **MaintainCommitments** is **Y**)**.** The value of the **TransactionSequence** field is **25**.
- Independent demand for 150 units of item **AAA** is due **October 12.** The value of the **OrderPriority** field is **Medium** (the value of **MaintainCommitments** is **Y**). The value of the **TransactionSequence** field is **10**.
- There are no **OnHand** records for item **AAA**.
- A **ScheduledReceipt** record for 150 units for item **AAA** is due **October 21**. RapidResponse is able to expedite this supply because the value of the **SupplyStatus.RescheduleRule** field is **Reschedulable** and the value of the **SupplyType.ProcessingRule** field is **Reschedulable**.
- RapidResponse cannot recommend new planned orders for item **AAA** because the value of the **PartSource**.**OrderPolicy.OrderGenerationRule** field is **NoOrders**.

Because both **IndependentDemand** records have an **OrderPriority** value of **Medium**, RapidResponse considers the value of the **TransactionSequence** field. It expedites the **ScheduledReceipt** record to **October 9** and uses this supply to satisfy the **IndependentDemand** with the smaller **TransactionSequence** value. The **AvailableDate** field on the **IndependentDemand** with the **TransactionSequence** value of **10** is assigned **October 9**.

The other independent demand cannot be satisfied because there is not enough supply. As a result, RapidResponse assigns the value **Future** to the **AvailableDate** field on the **IndependentDemand** record with the **TransactionSequence** value of **25**.

A component is affected by constraint rate

This example describes how Capable-to-Promise calculations are affected when a component of an assembly is affected by constraint. For information about constraint, see "Constraint Analysis

calculations" on page 1629.

This example uses constraint on the **PartSource** record belonging to item **CCC**. Assume the following conditions exist:

- Independent demand for 100 units of item **AAA** is due **October 10.**
- There are no **OnHand** or **ScheduledReceipt** records for item **AAA**.
- Dependent demand for 400 units of item **BBB** is due **October 9**. However, there are 500 units of item **BBB** in inventory that is able to satisfy the demand (the **Quantity** field in the **OnHand** record for item **BBB** stores 500 and the value of the **OnHand.Type.ProcessingRule** field is **Nettable**).
- Dependent demand for 200 units of item **CCC** is due **October 9**.
- There are no **OnHand** or **ScheduledReceipt** records for item **CCC.** However, RapidResponse is able to recommended new planned orders for item **CCC** because the value of the **PartSource**.**OrderPolicy.OrderGenerationRule** field is **Anytime**.
- The part source for item **CCC** has available constraint. However, the amount of constraint available for each time period is 50 (the **Rate** field on the **ConstraintAvailable** record is 50).

Result

RapidResponse creates as many planned orders during a time period as possible to satisfy demand. In this example, only 50 units of part **CCC** can be ordered during each workday (assume **Constraint.Calendar** = **Workday**). Because net requirements of item **CCC** is 200 and during each workday only 50 units can be ordered, RapidResponse recommends four separate planned orders (4 * 50 (Constraint Rate) = 200).

Assuming all planned orders must be available for use by **October 9** to satisfy demand, RapidResponse schedules four planned orders backwards in time from the DueDate (**October 9**).

The following shows the due date of all four planned orders.

Table 17.17: Planned order due dates

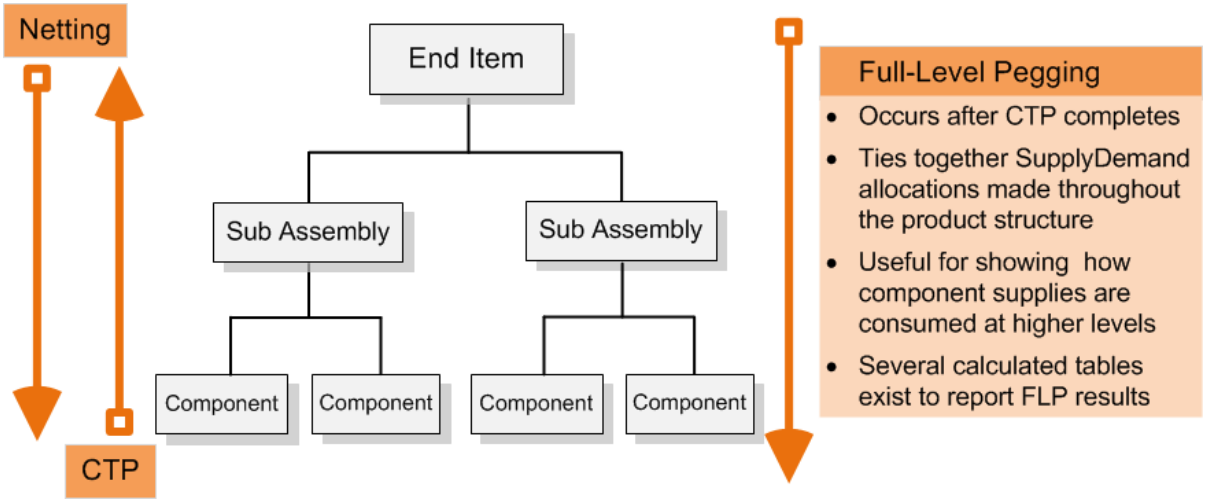
Planned Order	Rate	DueDate	Quantity
1	50	Oct 4	50
2	50	Oct 7	50
3	50	Oct 8	50
4	50	Oct 9	50

Because there are enough units of **CCC** to satisfy the demand of 100 units of item **AAA**, RapidResponse creates a planned order for 100 units of item **AAA**. The **AvailableDate** field on the **CTPPlannedOrder** record is **October 10**. The **AvailableDate** field on the **IndependentDemand** record is also **October 10**.

CHAPTER 18: Full-level pegging calculations

The Full-Level Pegging (FLP) analytic relies on output produced by the Capable-to-Promise analytic; specifically the single-level allotments reported in the calculated **SupplyDemand** table. The main function of the FLP analytic is to connect these allotments and use them to generate various allotment details in consideration of the product structure as a whole. For example, this can be used to show which top-level driver orders would be impacted by changes in allotted component supplies.

Figure 18.1 Full-Level Pegging functions



In this chapter, you'll learn about:

Full-level pegging tables 1493
Full-level pegging data example 1496
Full-level pegging through allocations 1499
Full-level pegging reporting horizon 1501

Full-level pegging tables

Several tables are available that report details from the FLP analytic. Their use depends on whether a view up or down the product structure is required, and the particular types of demands or supplies whose details are of interest. This section provides an overview of the following tables:

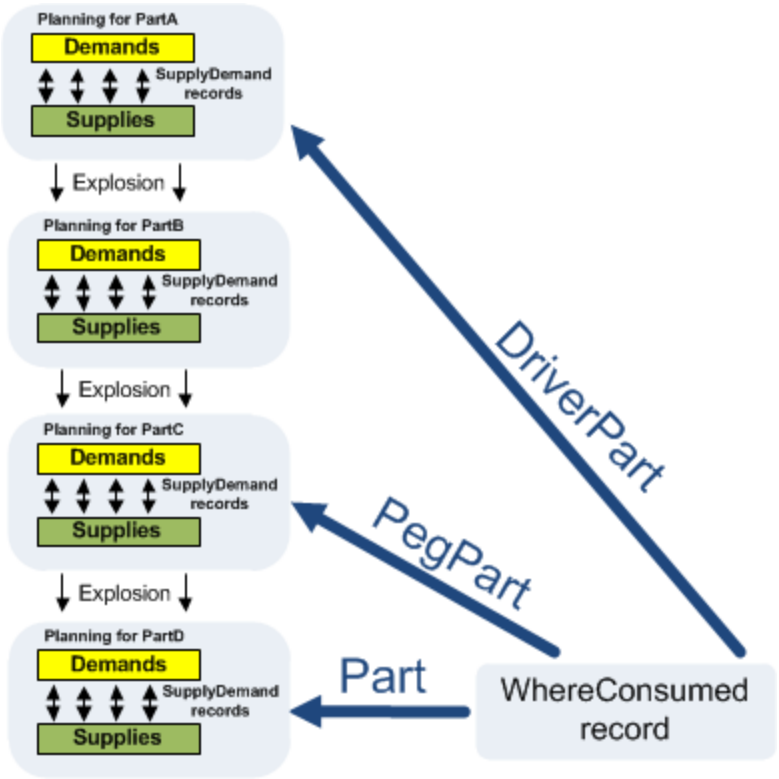
- "WhereConsumed" on page 1494
- "WhereConsumedForDemand" on page 1495
- "WhereConsumedForSupply" on page 1495
- "WhereConsumedForForecast" on page 1496
- "WhereConsumedToHighVolumeOrder" on page 1496

WhereConsumed

The **WhereConsumed** table is the most typically used table reporting FLP results. It operates from the perspective of a given component part and is used to look up the product structure at the allotments made above it. Each record in this table identifies a given component supply and the demand it satisfies, along with the immediate parent part and supply that drove demand to the component, and the top-level driver part and supply that drove demand down the product structure.

Note that as shown in the following illustration, when there are more than three levels in a product structure, there is no easy way to see a complete picture of all allotments made throughout the structure. Instead, reports based on the **WhereConsumed** table are meant to focus on specific components and where they are used.

Figure 18.2 Parts reported on WhereConsumed record



Note: The **WhereConsumed** table is owned by its **Part** reference. Thus when applying Part and Site filter controls to this table, they should typically be applied through **Part** and **Part.Site**. For more information about this table, see "WhereConsumed table" on page 1225.

WhereConsumedForDemand

The **WhereConsumedForDemand** table is meant to look down the product structure from the perspective of the driver allotment and to report all the dependent **SupplyDemand** allotments that will be used to satisfy the driver allotment. Essentially, for each given supply that ultimately pegs back to the driver through **SupplyDemand** relationships, a single record is added to this table reporting the **DriverPart** (assembly) and **Part** (component somewhere in the structure). Thus, there will be as many records for a given driver order as there are individual supplies that end up being consumed by it (at any level).

This table should be used when the focus in on determining all of the supplies that are consumed by a specific top-level demand. Hence, it can be used to identify all the supplies needed to satisfy the driver on time or for whom a change in availability might impact the driver order's status.

Unlike the **WhereConsumed** table then, the **WhereConsumedForDemand** table enables the reporting of the entire path and product structure of the allotments under a given driver order. The **WhereConsumedForDemand** table also contain a Index and Level fields that hold integer values useful for sorting and indenting fields under the driver in order to build a complete allotment tree.

Note: The **WhereConsumedForDemand** table is owned by its **IndependentDemand** reference. Thus, when applying Part and Site filter controls to this table, they should typically be applied through **IndependentDemand.Part** and **IndependentDemand.Part.Site**. For more information about this table, see "WhereConsumedForDemand table" on page 1234.

WhereConsumedForSupply

WhereConsumedForSupply is another table meant for top-down analysis. However, it used to report full-level pegging details where a given production order is considered as the driver. For example, this might typically useful in cases where the production environment from a "Master Schedule" rather than from demands and those master schedule records are loaded as **ScheduledReceipt** records. In such case, the **WhereConsumedForSupply** table can be used to determine what dependent supplies are going into the master schedule **ScheduledReceipt** records.

In other cases, it might be useful to show all supplies, from all levels in the structure, that are used to satisfy a given production order. For example, to help understand what component supplies could potentially impact a given production order if they were changed.

Finally, note that the **WhereConsumedForSupply** table can be a very expensive table to use. Reports based on this table might incur significant costs in terms of processing time and memory usage. Therefore, if using a report based on this table, it is recommended to first filter to the smallest number of driver parts possible.

Note: The **WhereConsumedForSupply** table is owned by its **DriverPart** reference. Thus when applying Part and Site filter controls to this table, they should typically be applied through **DriverPart** and **DriverPart.Site**. For more information, see "WhereConsumedForSupply table" on page 1252.

WhereConsumedForForecast

The **WhereConsumedForForecast** table is suitable for top-down analysis and is virtually identical to the **WhereConsumedForDemand** table discussed earlier in this section. Except **WhereConsumedForForecast** is meant to report all supplies that peg back to a given consensus forecast demand as generated from the S&OP process in RapidResponse.

Note: The **WhereConsumedForForecast** table is owned by its **DriverPart** reference. Thus when applying Part and Site filter controls to this table, they should typically be applied through **DriverPart** and **DriverPart.Site**. For more information about this table, see "WhereConsumedForForecast table" on page 1243.

WhereConsumedToHighVolumeOrder

The **WhereConsumedToHighVolumeOrder** table is meant for bottom-up analysis; albeit in very specific cases. It pegs each component supply to the highest level demand that it ultimately satisfies. This information can be used to determine which end-item demand(s) might be impacted by a change in availability of a given component supply.

This table is intended for use with components used in configurable end-items that are expected to have very large numbers of demands (that is, final assembly parts where **PartType.FeatureBOMRule** is set to "HighVolume"). When component supplies are used to satisfy an independent demand from a high volume end-item, the actual **IndependentDemand** is not reported as the driver on the **WhereConsumed** record. Instead, the **CTPPlannedOrder** record representing the configurable end-item supply created to satisfy that independent demand is used as the driver. This is done for performance reasons as high-volume configurable end-items typically have very large numbers of independent demands due on a given date; however, only a single planned order is created to satisfy those demands (within a given priority, model, unit, and pool combination).

For components that are not used in at least one high-volume, configurable end item, (only used by assembly's where **PartType.FeatureBOMRule** is either "Normal" or "Ignore"), the **WhereConsumed** table should instead be used to report the link between component supplies and the end item demands that consume them.

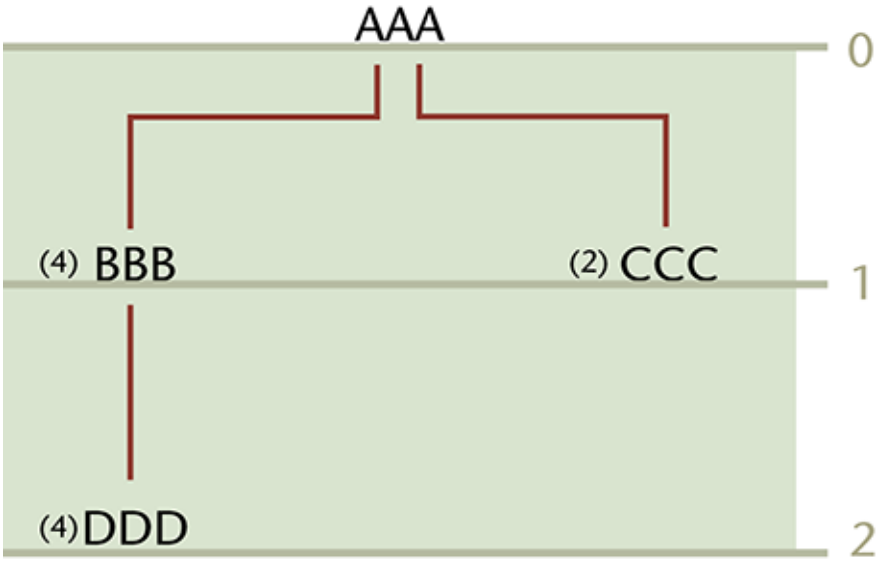
Note also that appropriate filtering should be used with worksheets based on the **WhereConsumedToHighVolumeOrder** table as there can be large numbers of records generated from demand for high volume, configurable end-items.

Note: The **WhereConsumedToHighVolumeOrder** table is owned by its **Part** reference. Thus when applying Part and Site filter controls to this table, they should typically be applied through **Part** and **Part.Site**. For more information about this table, see "WhereConsumedToHighVolumeOrder table" on page 1262.

Full-level pegging data example

The following diagram shows a bill of material product structure used in this example.

Figure 18.3 Bill of material product structure



The finished product in this bill of material product structure is item **AAA**. Two components that go into item **AAA** are **BBB** and **CCC**. The Quantity Per of component **BBB** is four (it takes four units of item **BBB** to create a single **AAA** item). The Quantity Per of component **CCC** is two. Item **DDD** is a required component to build item **BBB**. The Quantity Per of item **DDD** is four.

Full-level pegging example details

Assume the following:

- Independent demand for 100 units of item **AAA** is due **October 10**.
- Dependent demand for 400 units of item **BBB** is due **October 9** and there are no **ScheduledReceipt** or **OnHand** records for this part.
- Dependent demand for 200 units of item **CCC** is due **October 9** and there are no **ScheduledReceipt** or **OnHand** records for this part.
- Dependent demand for 1600 units of item **DDD** is due **October 8**.
- RapidResponse is able to create planned orders for all parts because the value of all **PartSource.OrderPolicy.OrderGenerationRule** fields is **Anytime**.

In this example, RapidResponse creates four **WhereConsumed** records (one **WhereConsumed** record for each supply allocated to a demand that pegs back to the top-level driver item in the bill of material product structure) and four **CTPPlannedOrder** records.

WhereConsumed records

The **WhereConsumed** record created for item **DDD** contains the following values:

- **PegPart** is assigned **BBB**
- **DriverPart** is assigned **AAA**
- **NeedQuantity** is assigned **1600**

The **WhereConsumed** record created for item **BBB** contains the following values:

- **PegPart** is assigned **AAA**
- **DriverPart** is assigned **AAA**
- **NeedQuantity** is assigned **400**

The **WhereConsumed** record created for item **CCC** contains the following values:

- **PegPart** is assigned **AAA**
- **DriverPart** is assigned **AAA**
- **NeedQuantity** is assigned **200**

The **WhereConsumed** record created for item **AAA** contains the following values:

- **PegPart** is empty
- **DriverPart** is assigned **AAA**
- **NeedQuantity** is assigned **100**

Note: For information about the **WhereConsumed** table, see "WhereConsumed table" on page 1225.

CTPPlannedOrder records

The **CTPPlannedOrder** record created for item **DDD** contains the following values:

- **Part** is assigned **DDD**
- **PlannedOrder.Quantity** is assigned **1600**
- **AvailableDate** is assigned **October 8**

The **CTPPlannedOrder** record created for item **BBB** contains the following values:

- **Part** is assigned **BBB**
- **PlannedOrder.Quantity** is assigned **400**
- **AvailableDate** is assigned **October 9**
- The **CTPPlannedOrder** record created for item **CCC** contains the following values:
- **Part** is assigned **CCC**
- **PlannedOrder.Quantity** is assigned **200**
- **AvailableDate** is assigned **October 9**

The **CTPPlannedOrder** record created for item **AAA** contains the following values:

- **Part** is assigned **AAA**
- **PlannedOrder.Quantity** is assigned **100**
- **AvailableDate** is assigned **October 10**

Full-level pegging with interchangeable parts

Suppose part **DDD** is interchangeable with part **FFF**, which is the primary part in the interchangeable group. In this case, **FFF** will be pegged to **BBB**.

The **WhereConsumed** and **CTPPlannedOrder** records will both be created for **FFF**.

The **WhereConsumed** record created for item **FFF** contains the following values:

- **PegPart** is assigned **BBB**
- **DriverPart** is assigned **AAA**
- **AlternatePart** is assigned **FFF**
- **NeedQuantity** is assigned **1600**

The **CTPPlannedOrder** record created for item **FFF** contains the following values:

- **Part** is assigned **FFF**
- **PlannedOrder.Quantity** is assigned **1600**
- **AvailableDate** is assigned **October 8**

Note: For more information about interchangeable parts, see "Interchangeable parts" on page 1545.

Full-level pegging through allocations

RapidResponse calculates the **ScheduledReceipt.OnTimeQuantity** field with partial allocations differently than previous versions due to a change in the way it calculates full-level pegging. If **ScheduledReceipt.TotalQuantity** or **Allocation.TotalQuantity** fields are greater than **RemainingQuantity** and **Quantity,** then RapidResponse performs the full-level pegging through allocations calculation. For more information, see "Full-level pegging through allocations calculation" on page 1500.

RapidResponse calculates the quantity of material placed into an allocation kit and considers quantity taken from the allocation kit to build complete units of the scheduled receipt. The quantity of a component that remains in the allocation kit is also used in the calculation.

It's recommended that the following input fields have accurate data:

- Allocation.TotalQuantity
- ScheduledReceipt.TotalQuantity

For example, ensure that amount of a component needed is specified in the **Allocation.TotalQuantity** field.

Using fractions

If the ratios between allocation quantities and scheduled receipt quantities are whole numbers (this defines the effective quantity per assembly), then pegging allocations are also in whole numbers.

In some situations, fractional pegging is unavoidable. For example, if scrap is allocated in fractional quantity, then fractional demand is accounted for in the pegging (as well as lower level parts).

Full-level pegging through allocations calculation

This calculation resolves the effective quantity per of the original allocation. It determines the number of units remaining in the scheduled receipt that requires some portion of the remaining allocation before the work order can be built. The RemainingQuantity is allocated among the last units in the scheduled receipt.

RapidResponse allocates scrap requirements, including fractional remainder, proportionally to the unit (s) to be built, whether they use supply from the kit or from the remaining allocation. AvailableDate calculations are not affected by full-level pegging through allocations calculations because AvailableDate is based on the availability of the entire allocation. For information about AvailableDate calculation, see "Determining AvailableDate (5)" on page 1641.

The following table describes the steps involved in the full-level pegging through allocations calculation.

Table 18.1: Full-level pegging through allocations calculation

Step	Description	Calculation
1.	Determine effective quantity per unit (first part)	For ScheduledReceipts, calculate: **SRTotal**=Max(ScheduledReceipt.TotalQuantity, ScheduledReceipt.Quantity) For Allocations, calculate: **ALTotal**=Max(Allocation.TotalQuantity, Allocation.RemainingQuantity)
2.	Determine effective quantity per unit (second part)	Calculate the EffQuantity which is the larger of the quantity per calculated from the total supply, total allocation, and as calculated from the remaining supply quantity and the remaining allocation. EffQuantityPer is defined as the quantity of the component, through this allocation required to make a single unit of the scheduled receipt part. Calculated as: **EffQuantityPer**=Max(ALTotal/SRTotal, Allocation.RemainingQuantity/ScheduledReceipt.Quantity)
3.	Resolve the allocation	Calculate the quantity that remains in the allocation kit. This value is calculated as the quantity received into the allocation kit less the amount that is used from the kit to satisfy completed supply units. Calculated as: **InKit** = Max(0, ALTotal-RemainingQuantity-((SRTotal-ScheduledReceipt.Quantity) * EffQuantityPer) **Note:** This value is reported in the **Allocation.InKit** field. For information about this field, see "Allocation table" on page 182.

Table 18.1: Full-level pegging through allocations calculation

Step	Description	Calculation
4.	Calculate the OnTimeQuantity (first part)	Allocate any InKit quantity to the first units of the ScheduledReceipt to be satisfied using EffQuantityPer. This is treated like on hand inventory.
5.	Calculate the OnTimeQuantity (second part)	After the InKit quantity has been accounted for, allocate Allocation.RemainingQuantity to ScheduledReceipt.Quantity.

Full-level pegging reporting horizon

Tables reporting full-level pegging results can potentially contain large numbers of generated records which can make it difficult for worksheet users to find specific results of interest and can also adversely impact system performance.

To limit full-level pegging results to a specific date range, a reporting horizon (or fence) can be applied to the results generated from a number of different tables. For each of the tables outlined below, two analytic parameters are available and can be used to set the first and last date respectively for which results can be reported. Note that the parameters are implemented as workbook variables and therefore apply to worksheets within a given workbook only.

Table 18.2: Analytic parameters for full-level pegging reporting horizon

Table	Applicable Parameters
WhereConsumed	• **RR_Analytics_WhereConsumed_BeginDate**—defines a date on or after which component supplies must be due if they are to be reported and pegged to end-item demands in worksheets based on the **WhereConsumed** table. In other words, this represents the earliest **SupplyDueDate** that will be reported. • **RR_Analytics_WhereConsumed_EndDate**—defines a date on or before which component supplies must be due if they are to be reported and pegged to end-item demands in worksheets based on the **WhereConsumed** table. In other words, this represents the latest **SupplyDueDate** that will be reported.
WhereConsumedForForecast	• **RR_Analytics_WhereConsumedForForecast_BeginDate**—defines a date on or after which consensus forecast demands must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForForecast** table. In other words, this represents the earliest **DriverDueDate** that will be reported. • **RR_Analytics_WhereConsumedForForecast_ EndDate**—defines a date on or before which consensus forecast demands must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForForecast** table. In other words, this represents the latest **DriverDueDate** that will be reported.

Table 18.2: Analytic parameters for full-level pegging reporting horizon

Table	Applicable Parameters
WhereConsumedForSupply	• **RR_Analytics_WhereConsumedForSupply_BeginDate**—defines a date on or after which production orders must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForSupply** table. In other words, this represents the earliest **DriverDueDate** that will be reported. • **RR_Analytics_WhereConsumedForSupply_EndDate**—defines a date on or before which production orders must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForSupply** table. In other words, this represents the latest **DriverDueDate** that will be reported.
WhereConsumedToHigh VolumeOrders	• **RR_Analytics_WhereConsumedHighVolume_BeginDate**—defines a date on or after which component supplies must be due if they are to be reported and pegged to end-item demands in the **WhereConsumedToHighVolumeOrder** table. • **RR_Analytics_WhereConsumedHighVolume_EndDate**—defines a date before which component supplies must be due if they are to be reported and pegged to end-item demands in the **WhereConsumedToHighVolumeOrder** table.

Note: For more information about configuring these parameters in workbooks, see "Configuring analytic calculations using workbook parameters" on page 2209.

CHAPTER 19: Cost of goods sold calculations

The Projected Cost of Goods Sold (PCOGS) calculation determines the projected cost of products sold by adding material, labor, and overhead costs. RapidResponse stores this financial data in the **CostOfGoodsSold** table. For information about this table, see "CostOfGoodsSold table" on page 977.

PCOGS results should be analyzed over a time horizon (for example, monthly or quarterly). Individual customer orders should not be analyzed separately.

Consider a scenario that satisfies a customer order and the PCOGS calculation for the order is $100. Next, assume a new customer order is added to the scenario. This new order might use some of the supplies that are assigned to the original customer order. The COGS value for the new order may be $90, but that number does not represent the true cost of the new order because the COGS value for the first customer order increases in value due to changed allocations. A new order may affect existing ones. Therefore, it is more accurate to analyze PCOGS calculation results for the entire plan over a specified time horizon.

In this chapter, you'll learn about:

Allocated scheduled receipts **1503**
Unallocated scheduled receipts **1505**
Calculating PCOGS material costs **1505**
Calculating PCOGS labor costs **1508**
Calculating PCOGS overhead costs **1509**

Note: Certain calculations described in this chapter use cost values taken either directly from a **PartSource** record or, if applicable, from the effective **PartSourceTimePhasedAttributes** record associated with the part source. For more information about these tables, see "PartSource table" on page 459 and "PartSourceTimePhasedAttributes table" on page 476.

Allocated scheduled receipts

An allocated scheduled receipt is reserved supply, does not generate new allocations, and the value of the **ScheduledReceipt.Status.State** control field is either **NotReleasedAndAllocated** or **ReleasedAndAllocated.** For information about the **ScheduledReceipt.Status.State** control field, see "SupplyStatus table" on page 904.

Allocated scheduled receipts may not have a complete set of allocations available (allocation records are stored in the **Allocation** table). For example, some material requirements might be completely kitted, and as a result, allocations are not stored in the **Allocation** table.

RapidResponse considers allocated scheduled receipts as supply that uses defined allocations and does not create new allocations. Costs for an allocated scheduled receipt are calculated; however, other costs accumulated through supplies that either directly or indirectly satisfy allocations belonging to the allocated scheduled receipt aren't considered.

An allocated scheduled receipt's material cost is based on the value of the **ScheduledReceipt.UnitPrice** field as long as a positive value has been provided. If no UnitPrice is defined on the scheduled receipt, then material cost is based on the **PartSourceTimePhasedAttributes.MaterialCost** value as long as there is an effective record for the part source in the **PartSourceTimePhasedAttributes** table, and based on **PartSource.MaterialCost** otherwise. For information about effective material cost calculations, see "Calculating PCOGS material costs" on page 1505.

The labor cost on an allocated scheduled receipts is based on the **PartSourceTimePhasedAttributes.LaborCost** value as long as there is an effective record for the part source in the **PartSourceTimePhasedAttributes** table, and based on **PartSource.LaborCost** otherwise

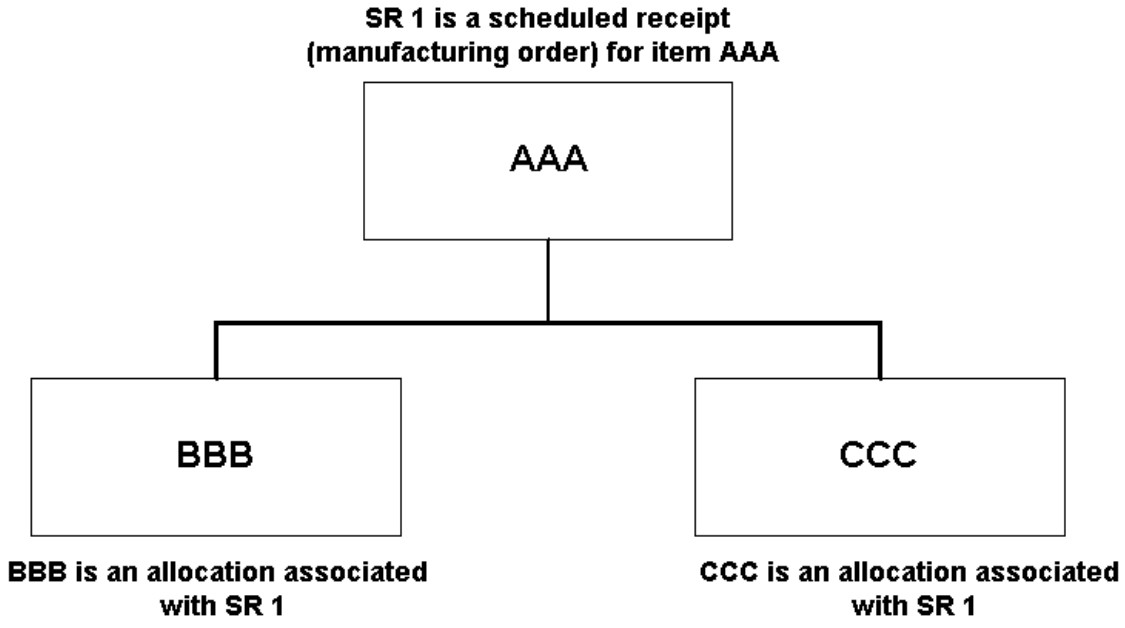
The overhead cost on an allocated scheduled receipt is based on the **OverheadCost** and **OverheadCost2** fields in the **PartSourceTimePhasedAttributes** as long as the part source has an effective record in the table, and based on the **OverheadCost** and **OverheadCost2** fields in the **PartSource** table otherwise.

Note: The **CostOfGoodsSold** table does not include records for supply quantities that either directly or indirectly satisfy allocations belonging to allocated scheduled receipts.

Allocated scheduled receipt example

Assume an allocated scheduled receipt for item **AAA** exists (**AAA** is a make part). Next, assume there are two allocations associated with this scheduled receipt. The allocations are **BBB** and **CCC**. Refer to the following diagram.

Figure 19.1 An allocated scheduled receipt



Because **SR 1** is an allocated scheduled receipt, its allocations, **BBB** and **CCC**, are not considered in the PCOGS calculation. RapidResponse only considers the effective material cost, labor cost, and overhead cost for item **AAA**.

Unallocated scheduled receipts

Unallocated scheduled receipts may have allocations associated with them. RapidResponse creates **CostOfGoodsSold** records for each supply below an unallocated scheduled receipt. Costs associated with these supplies are calculated. For example, consider the scheduled receipt shown in "Allocated scheduled receipt example" on page 1504.

If **SR 1** is an unallocated scheduled receipt, the cost of supplies satisfying allocations for items **BBB** and **CCC** are considered in the PCOGS calculation.

Calculating PCOGS material costs

Material cost is defined as the cost of materials associated with a supply and is not calculated for all records in the **CostOfGoodsSold** table. Material cost is calculated for terminating supplies. For all other supplies, material cost is set to zero.

Terminating supply

A terminating supply does not generate dependent demand. The following are considered terminating supply:

- **Allocated scheduled receipts—**the value of the ScheduledReceipt.Status.State control field is either NotReleasedAndAllocated or ReleasedAndAllocated.
- **Inventory supplies—**the value of the **CostOfGoodsSold.SupplySource** field is **OnHand**.
- **InKit quantities—**the value of the **CostOfGoodsSold.SupplySource** is **InKit**.
- Effectively purchased supplies—supplies that are purchased and do not generate dependent demand. Effectively purchased supplies are:
  - **Buy parts—**supplies that are bought. The **SupplyType.Source** control value is **Buy.**
  - **Transfer supplies—**supplies that are transferred and do not generate dependent demand. The **SupplyType.Source** control value is **Transfer** and the following conditions are true: The value of the **PartSource.Part.Name** field is equal to the **PartSource.TransferPart.Name** field. The value of the **PartSource.Part.Site** field is equal to the **PartSource.TransferPart.Site** field.
  - **Make parts—**supplies that are made but have no dependent demand. The **SupplyType.Source** control value is **Make**.

Material and new purchase costs

Material cost differs from new purchase costs in two ways (**NewPurchasesCost** is a calculated field appearing in various tables such as the **ScheduledReceipt** table):

- Material costs include material costs associated with not only planned orders, but also scheduled receipts and inventory.
- New purchase costs are based on the **PartSource.EffUnitCost** field, while material costs are based on **PartSource.EffUnitCost** - UOMConversion (**PartSource.LaborCost** + **PartSource.OverheadCost** + **PartSource.OverheadCost2**).

Effective material cost

Effective material cost is based on the **PartSource.EffUnitCost** field and is used in the material cost calculation (this value is used throughout this chapter). RapidResponse calculates effective material cost by subtracting labor and overhead costs from **PartSource.EffUnitCost**. If this value is not positive, then effective material cost is zero.

Material cost calculations

Material costs for a supply are calculated by multiplying the NeedQuantity of the supply by a cost value associated with that supply. The calculated material cost is then stored in the **CostOfGoodsSold.MaterialCost** field.

The following table explains the fields used by RapidResponse as input into the material cost calculation based on the type of supply. Note that some material cost calculations are affected by the **PartSourceType.CostRule** control setting. For more information about this field, see "CostRule" on page 801.

Table 19.1: RapidResponse material costs calculation

Supply	Material cost calculation
OnHand (inventory)	• If the **OnHand.UnitCost** field is a positive value, then `NeedQty * OnHand.UnitCost` • Otherwise, if the **Part.StdUnitCost** is a positive value, then `NeedQty * Part.StdUnitCost` • Otherwise, zero.
InKit quantities	• If the **Part.StdUnitCost field** is a positive value, then `NeedQty * Part.StdUnitCost` • Otherwise, zero is used.

Table 19.1: RapidResponse material costs calculation

Supply	Material cost calculation

Effectively purchased planned orders	If **PartSourceType.CostRule** = 'MaterialLaborOverhead'
	• If the part source has an effective record in the **PartSourceTimePhasedAttributes** field, then
	`NeedQty * PartSourceTimePhasedAttributes.MaterialCost`
	• Otherwise
	`NeedQty * PartSource.MaterialCost`
	• Otherwise, zero is used.
	If **PartSourceType.CostRule =** 'UnitCost', then
	• If PartSource.UnitCost is a positive value
	`PartSource.UnitCost – (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, if Part.StdUnitCost is a positive value
	`Part.StdUnitCost – (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, zero is used.

Effectively purchased scheduled receipts	If **PartSourceType.CostRule** = 'MaterialLaborOverhead'
	• If **ScheduledReceipt.UnitPrice** value is a positive value.
	`NeedQty * UOMConversion (ScheduledReceipt.UnitPrice)`
	• Otherwise, if the part source has an effective record in the **PartSourceTimePhasedAttributes** field, then
	`NeedQty * UOMConversion (PartSourceTimePhasedAttributes.MaterialCost)`
	• Otherwise
	`NeedQty * UOMConversion (PartSource.MaterialCost)`
	• Otherwise, zero is used.
	If **PartSourceType.CostRule =** 'UnitCost'
	• If **PartSource.UnitCost** is a positive value, then
	`NeedQty * PartSource.UnitCost – UOMConversion (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, if **Part.StdUnitCost** is a positive value, then
	`NeedQty * Part.StdUnitCost – (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, zero is used.

Table 19.1: RapidResponse material costs calculation

Supply	Material cost calculation

Allocated scheduled receipts	If **PartSourceType.CostRule** = 'MaterialLaborOverhead'
	• If **ScheduledReceipt.UnitPrice** value is a positive value.
	`NeedQty * UOMConversion (ScheduledReceipt.UnitPrice)`
	• Otherwise, if the part source has an effective record in the **PartSourceTimePhasedAttributes** field, then
	`NeedQty * UOMConversion (PartSourceTimePhasedAttributes.MaterialCost)`
	• Otherwise
	`NeedQty * UOMConversion (PartSource.MaterialCost)`
	• Otherwise, zero is used.
	If **PartSourceType.CostRule =** 'UnitCost'
	• If **PartSource.UnitCost** is a positive value, then
	`NeedQty * PartSource.UnitCost – UOMConversion (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, if **Part.StdUnitCost** is a positive value, then
	`NeedQty * Part.StdUnitCost – (PartSource.LaborCost + PartSource.OverheadCost + PartSource.OverheadCost2)`
	• Otherwise, zero is used.

In-transit inventory transfer	• If the **Part.StdUnitCost field** is a positive value, then `NeedQty * Part.StdUnitCost` • Otherwise, zero is used.
All other supplies (for example, unallocated scheduled receipts)	Zero is always used.

Calculating PCOGS labor costs

Labor cost is the cost of labor associated with a supply and is calculated for most **CostOfGoodsSold** records. The result of the labor cost calculation is stored in the **LaborCost** field of the **CostOfGoodsSold** table.

If the supply's part source has an effective record in the **PartSourceTimePhasedAttributes** table, then labor cost is calculated as:

```
NeedQuantity * UOMConversion (PartSourceTimePhasedAttributes.LaborCost)
```

Otherwise, labor cost is calculated as:

```
NeedQuantity * UOMConversion (PartSource.LaborCost)
```

Note that the **LaborCost** field is set to zero for all **CostOfGoodsSold** records where:

- **CostOfGoodsSold.SupplySource** field is **OnHand**.
- **CostOfGoodsSold.SupplySource** field is **InKit**.
- **CostOfGoodsSold.SupplySource** field is **None**.

Calculating PCOGS overhead costs

Overhead cost is cost associated with a supply and is calculated for most **CostOfGoodsSold** records. Results are stored in the **OverheadCost** and **OverheadCost2** fields in the **CostOfGoodsSold** table**.**

Note that the **OverheadCost** and **OverheadCost2** fields are set to zero for **CostOfGoodsSold** records where:

- **CostOfGoodsSold.SupplySource** field is **OnHand**.
- **CostOfGoodsSold.SupplySource** field is **InKit**.
- **CostOfGoodsSold.SupplySource** field is **None**.

If the supply's part source has an effective record in the **PartSourceTimePhasedAttributes** table, then overhead costs are calculated as:

- `NeedQuantity * UOMConversion (PartSourceTimePhasedAttributes.OverheadCost)`
- `NeedQuantity * UOMConversion (PartSourceTimePhasedAttributes.OverheadCost2)`

Otherwise, overhead costs are calculated as:

- `NeedQuantity * UOMConversion (PartSource.OverheadCost)`
- `NeedQuantity * UOMConversion (PartSource.OverheadCost2)`

CHAPTER 20: Cost roll-up calculations

The Cost roll up calculation determines rolled-up material, labor, and overhead costs for each part, based on effective bill of material components and effective part sources. This financial data is stored in the **CostRollUp** table. For information about this table, see "CostRollUp table" on page 982.

Rolled-up costs might change over time due to effective components. For example, an effective part source during summer months may be less expensive than an effective part source during winter months. Rolled-up costs can be compared with other effective bill of material components and part sources to determine which components and part sources are cost efficient. Rolled-up costs can also be compared with the value of the **Part.StdUnitCost** field.

In this chapter, you will learn about:

Calculating CR material costs **1511**
Calculating CR labor costs **1512**
Calculating CR overhead costs **1513**

Note: Certain calculations described in this chapter use cost values taken either directly from a **PartSource** record or, if applicable, from the effective **PartSourceTimePhasedAttributes** record associated with the part source. For more information about these tables, see "PartSource table" on page 459 and "PartSourceTimePhasedAttributes table" on page 476.

Calculating CR material costs

Material costs are calculated for effectively purchased descendants only (this avoids double-counting the material costs for "make" and "purchase" items). Effectively purchased descendants have no dependent demands and are defined as:

- **PartSource.Type.PlannedOrderSupplyType.Source** control value is **Buy**.
- **SupplyType.Source** control value is **Transfer** and the following conditions are true:
  - The value of the **PartSource.Part.Name** field is equal to the **PartSource.TransferPart.Name** field.
  - The value of the **PartSource.Part.Site** field is equal to the **PartSource.TransferPart.Site** field.
- **SupplyType.Source** control value is **Make** but no effective components exist.

FlatNetQuantityPer in the BillOfMaterial table

FlatNetQuantityPer is used in the material cost calculation (this calculation is shown in a following section). As long as the bill of material components have no negative **QuantityPer** (**BillOfMaterial.QuantityPer** field), **FlatNetQuantityPer** is the number of components needed to build an assembly part. It takes into account the **QuantityPer** field from the current **BillOfMaterial** record and the **FlatNetQuantityPer** of its immediate parent part.

In the case where bill of material components include negative **QuantityPer** values, RapidResponse consolidates all the **QuantityPer** field values for the same component of the same part to arrive at the **NetQuantityPer** for that component. If the **NetQuantityPer** is negative, it's treated as zero.

NetQuantityPer is multiplied by the **FlatNetQuantityPer** of its immediate parent to determine **FlatNetQuantityPer**. For information about negative **QuantityPer** values, see "Exploding bill records with a negative QuantityPer value" on page 1338.

Material cost equation

Material cost is stored in the **MaterialCostOnly** field in the **CostRollUp** table. The equation to calculate the material cost for effectively purchased descendants of a part (including the part itself) is:

MaterialCost = Sum of (EffectiveMaterialCost * FlatNetQuantityPer) for all descendants of this part, including the part itself

Depending on the type of supply, the effective material cost calculation may be based on the supply's UnitCost, the StdUnitCost on the Part record, or the MaterialCost stored on the PartSource record or in a PartSourceTimePhasedAttributes record referencing the part source. Material cost calculations can also be affected by the PartSource.CostRule setting. For information about how material cost is determined, see "Material cost calculations" on page 1506.

Calculating CR labor costs

Labor cost is the cost of labor associated with a supply and is summed for all descendants of a part and then added to the labor cost for the part. This allows for the possibility of incurring labor cost at all levels of the supply chain. The result is stored in the **LaborCost** field in the **CostRollUp** table. The equation to calculate labor cost is:

LaborCost = Sum of (UOMConversion (LaborCost) * FlatNetQuantityPer) for all descendants of this part, including the part itself

The LaborCost used in this calculation is either taken from a part source (PartSource.LaborCost) or from an effective time phased record referencing the part source (PartSourceTimePhasedAttributes.LaborCost). For information, see "Calculating PCOGS labor costs" on page 1508.

Calculating CR overhead costs

Overhead costs are overhead cost associated with a supply and are summed for all descendants of a part and then added to the overhead cost for the part itself. This allows for the possibility of incurring overhead cost at all levels of the supply chain. Results are stored in the **OverheadCost** and **OverheadCost2** fields in the **CostRollUp** table. The equation to calculate overhead costs are:

OverheadCost = Sum of (UOMConversion (OverheadCost) * FlatNetQuantityPer) for all descendants of this part, including the part itself

OverheadCost2 = Sum of (UOMConversion OverheadCost2) * FlatNetQuantityPer) for all descendants of this part, including the part itself

The OverheadCost and OverheadCost2 values used in this calculation are either taken from a part source (PartSource.OverheadCost and PartSource.OverheadCost2) or from an effective time phased record referencing the part source (PartSourceTimePhasedAttributes.OverheadCost and PartSourceTimePhasedAttributes.OverheadCost2). For information, see "Calculating PCOGS labor costs" on page 1508.

CHAPTER 21: Capacity Requirements Planning (CRP) calculations

RapidResponse supports capacity requirements planning (CRP) by calculating the machine and labor load profile on work centers for comparison with work center capacity. Work orders defined in the **ScheduledReceipt** table reference a routing, with all operations belonging to sequences that reference the same routing run during processing of the work order. Similarly for planned orders, a routing referenced on the part source is used for determining the required operations and processing load at work centers.

The output of certain capacity requirements calculations can also be linked to and affect results produced by the Netting analytic. For example, the time calculated to run a supply through all operations in its routing, based on available work center capacity, can be used in calculating lead time and hence supply start dates.

In this chapter, you'll learn more about:

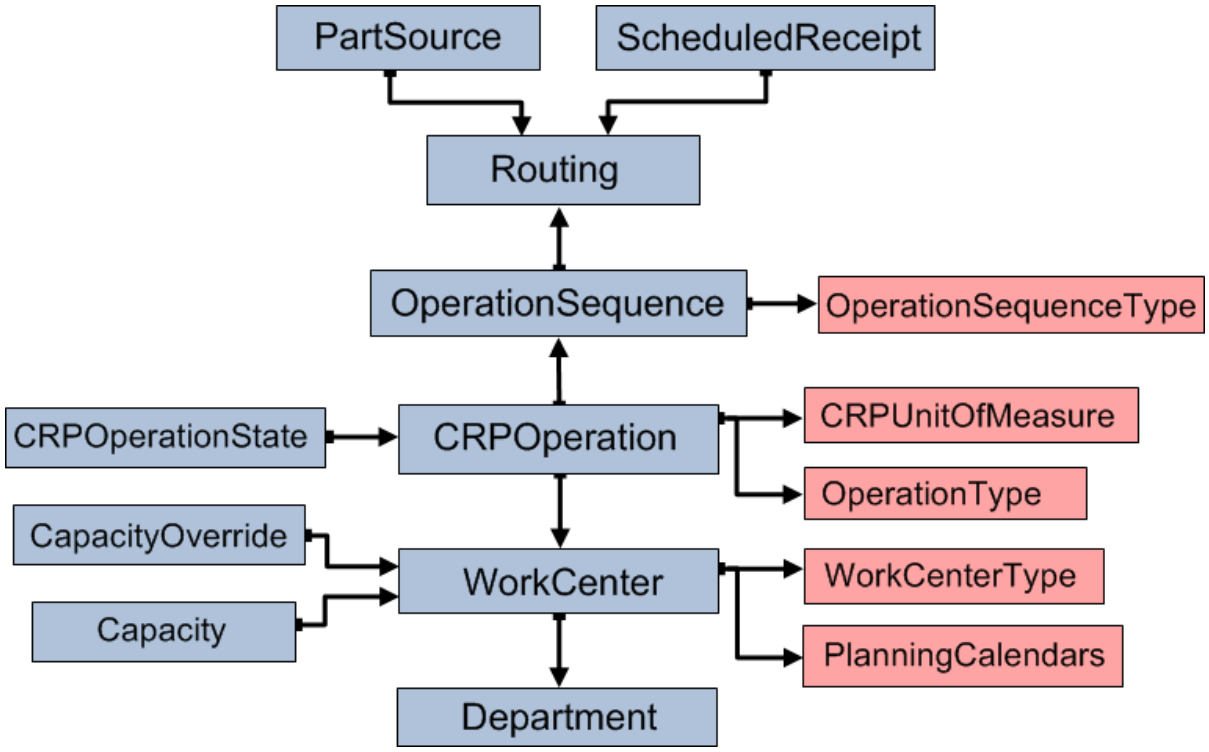
Capacity Requirements Planning data model **1515**
About Capacity Requirements Planning calculations **1519**
Scheduled receipts and planned orders **1520**
Routings **1521**
Operations and sequences **1522**
Work centers and capacity **1534**
Operation loads **1536**
WorkCenter loads **1538**
Limitations on other analytics with CRP **1540**

Capacity Requirements Planning data model

The RapidResponse data model contains input, control, and calculated tables designed specifically to support capacity requirements planning. There are also other standard (non-Capacity specific) tables that hold fields pertaining to capacity planning.

The following illustration shows the typical input and control tables required to configure Capacity calculations.

Figure 21.1 Tables for configuring Capacity Requirements Planning



The remainder of this section provides a brief overview of each capacity-related table along with links to more detailed reference information.

Input tables

There are several input tables in the RapidResponse data model that support Capacity planning. These include tables that hold routing and operations data, work center and capacity data, as well as standard RapidResponse tables that contain fields related to Capacity planning.

Table 21.1: Capacity planning input tables

Table	Description
WorkCenter	Describes a unique work center, including name, description, cost information, and its set of available capacity records. For more information, see "WorkCenter table " on page 666.
Department	Describes a work center department (typically, a group of related work centers). For more information, see "Department table" on page 274.
Capacity	Specifies available capacity at a work center. For example, the number of hours available per day. This is used to schedule the duration of any given operation on the work center. For more information, see "Capacity table" on page 234.

Table 21.1: Capacity planning input tables

Table	Description
CapacityOverride	Specifies available capacity at a work center on a particular day of the week. For example, this table can be used to define shift-based capacity. Records here override the work center's default capacity values stored in the Capacity table. For more information, see "CapacityOverride table" on page 236.
CRPOperation	Defines each of the work center operations required in the production of a part, along with duration of those operations, and the order in which operations within a given sequence should occur. For more information, see "CRPOperation table" on page 256.
OperationSequence	Groups work center operations together in a particular sequence. Each routing requires exactly one standard sequence and can, optionally, have one or more parallel sequences that branch off that standard sequence. For more information, see "OperationSequence table" on page 410.
Routing	Contains the names of routings that are used to associate parts and supply orders with the sequence(s) of work center operations that are required in their production. For more information, see "Routing table" on page 534.
AlternateRouting	Used as a placeholder for alternate routings. It links routings to parts for a list of alternate routings. For more information, see "AlternateRouting table" on page 193.
CRPOperationState	Identifies ongoing work center operations for partially completed scheduled receipts. This allows work center scheduling to consider only portions of supplies and operations that have yet to complete. For more information, see "CRPOperationState table" on page 260.
Operation (supports previous versions)	Can be used to identify each of the operations required in the production of a part, and describes the order and duration of the operations, as well as any differences (or overrides) in capacity available at the work center during the processing of the operation. For more information, see "Operation table " on page 407. **Note:** As of Version 2014.4, this table is maintained for backward compatibility with previous versions of RapidResponse. All existing resources based on this table will continue to function as before. However, for any new configurations, and in all cases where parallel operations and/or lot splitting is required, the **CRPOperation** table should be used instead.
PartSource	A standard RapidResponse table that contains several fields affecting Capacity planning. For example, the Routing reference determines the operations required in production of planned orders from the part source. For more information, see "PartSource table" on page 459.
ScheduledReceipt	A standard RapidResponse table that contains several fields affecting Capacity planning. For example, the Routing reference determines the operations required in production of planned orders from the part source. For more information, see "ScheduledReceipt table " on page 569.
BillOfMaterial	A standard RapidResponse that contains a field, Operation, that is used in Capacity planning with ongoing operations on scheduled receipts. For more information, see "BillOfMaterial table" on page 217.

Control tables

There are several control tables that impact Capacity calculations.

Table 21.2: Capacity planning control tables

Table	Description
CRPUnitOfMeasure	Defines different units of measurement used to interpret the run times of an operation (how run times of an operation are converted into standard hours of load). For more information, see "CRPUnitOfMeasure table" on page 704.
WorkCenterType	Identifies the switches and default capacity values for each type of work center. For example, whether load is calculated for the work center. For more information, see "WorkCenterType table" on page 933.
OperationType	Sets the processing rules used for scheduling an operation. For example, how to apply any transit time incurred when moving between operations. For more information, see "OperationType table" on page 770.
OperationSequenceType	Contains processing rules applied to an operation sequences within a routing. For example, does the operation sequence represent the standard sequence in the routing or a parallel sequence. For more information, see "OperationSequenceType table" on page 768.
OrderPolicy	A standard table containing a field, **UseVarLeadTime**, that can be used to include work center processing time as a variable lead time for the supply. For more information, see "OrderPolicy table" on page 773.
PartSourceType	A standard table containing a field, **PlannedOrderSupplyType**, that affects Capacity planning. For more information, see "PartSourceType table" on page 797.
SupplyType	A standard table containing a field, **CapacityProcessingRule**, that determines the operation scheduling method applied to supplies when calculating work center load. For more information, see "SupplyType table" on page 919.

Calculated tables

The following calculated tables in the RapidResponse data model contain Capacity planning information. These contain load data, both using current order due dates (current) and the RapidResponse recommended due dates (planned). Each of these are combined to produce a single current view of load and a single planned view of load.

Table 21.3: Capacity planning calculated tables

Table	Description
LoadDailyCurrent	Describes all load and capacity on a work center from scheduled receipts through operations and scheduled operations using the current due dates. Records are generated for each day that has load. For more information, see "LoadDailyCurrent table" on page 1097.
LoadDetailCurrent	Describes each load on a work center from scheduled receipts through operations and scheduled operations not using the current due dates. For more information, see "LoadDetailCurrent table" on page 1100.
LoadDailyPlanned	Describes all load on a work center from scheduled receipts and planned orders through operations and scheduled operations using the rescheduled scheduled receipt dates. For more information, see "LoadDailyPlanned table" on page 1098.

Table 21.3: Capacity planning calculated tables

Table	Description
LoadDetailPlanned	Describes each load on a work center from scheduled receipts and planned orders through operations and scheduled operations using the rescheduled scheduled receipt dates. For more information, see "LoadDetailPlanned table" on page 1102.
LoadFLPPlanned	Provides full level pegging and supply available info for load on work centers from planned orders and scheduled receipts. For more information, see "LoadFLPPlanned table" on page 1104.
LoadOperationReceiptsCurrent	Describes the load on a work center from a scheduled receipt through an operation scheduled using the current due dates. For more information, see "LoadOperationReceiptsCurrent table" on page 1108.
LoadOperationReceiptsPlanned	Describes the load on a particular date from a scheduled receipt through an operation scheduled using the rescheduled dates. For more information, see "LoadOperationReceiptsPlanned table" on page 1110.
LoadOperationNewOrderPlanned	Describes the load on a work center from a planned order through an operation. For more information, see "LoadOperationNewOrderPlanned table" on page 1106.
WorkCenterCapacity	Reports effective work center capacity by date and indicates which of the Capacity, CapacityOverride, or WorkCenterType tables the effective values originate from. For more information, see "WorkCenterCapacity table" on page 1265.

About Capacity Requirements Planning calculations

A routing is the sequence of operations that must be performed in order to produce the part from its direct components. Each operation specifies a number of standard setup hours and a piece run time to perform the operation at a single work center.

Each work center has time phased capacity records. These records define the factors required to convert the setup and run standard hours into elapsed hours for scheduling, and from that schedule, calculate the machine and labor load hours on the work center. The capacity also defines the operating schedule of the work center and its capacity limits.

RapidResponse creates two different perspectives of the load:

- The results in tables referred to as the Current load tables are calculated using only the scheduled receipts as currently scheduled (their input DueDate). This makes no use of the MRP calculation results and presents a picture of the current schedule. All of these table names end in the word Current. For example, the **LoadDailyCurrent** table.
- The results in tables referred to as the Planned load tables are calculated using the dates and orders generated by MRP calculations. The ReschedDate is used for scheduled receipts. Planned orders are also used to generate load. (See the MRP/MPS analytic for a description of how the

SupplyType control table controls the calculation of the ReschedDate.) All of these table names end in the word Planned. For example, the **LoadDailyPlanned** table.

The Capacity Requirements Planning analytic can be used to give a factory load projection over periods of time such as weeks, months or even years. This could be used to ensure that load and capacity levels are in alignment. The accuracy of these longer projections depends on the proper MPS forecast. Note that Capacity Requirement Planning represents an Infinite CRP model. Workcenters are loaded based on the MRP lead-times and average operation queue times, regardless of the existence of other orders due at the same time that may be competing for resources.

CRP load is generated in two steps. The first step is to schedule each work order through the operations associated with its routing. The operation parameters are converted to elapsed duration using the effective work center capacity parameters, such as Efficiency and Utilization. Operations are scheduled in a forward or backward direction, depending on the type of order. It is important to note that in a typical installation, only one method is chosen for all order types.

Forward scheduling starts with the first operation at the date the order will start production and proceeds forward through the operations. Backward scheduling starts with the last operation and the date the order is due to be completed. One record is produced for each operation for each work order. This intermediate result, needed for scheduling purposes, is stored in several tables referred to as Operation load tables (for example, **LoadOperationReceiptsCurrent** table) They provide references back to the operation and the order. They give the elapsed duration of the operation's setup and run phases, as well as a detailed date and offset into the workday of each phase of the load.

Table 21.4: Operation load scheduling phase result tables related to input data

Table	Current	Planned
ScheduledReceipt	LoadOperationReceiptsCurrent	LoadOperationReceiptsPlanned
PlannedOrder	N/A	LoadOperationNewOrderPlanned

The second step is to use the operation load records to generate a machine and labor load profile on the work center. The elapsed times calculated in the first step that were needed for scheduling purposes, are converted back to standard hours using parameters from the effective record on the **Capacity** table. Then, machine and labor ratios are used to convert the standard hours into work center machine and labor load.

Table 21.5: WorkCenter load tables (consolidation of Operation loads)

Information type	Current	Planned
Detailed load breakdown	LoadDetailCurrent	LoadDetailPlanned
Daily bucketed load and capacity	LoadDailyCurrent	LoadDailyPlanned
Weekly bucketed load and capacity	LoadWeeklyCurrent	LoadWeeklyPlanned

Note that the duration of an operation load may span several days. These records are broken into daily pieces when the detail work center loads are calculated. In addition, the results are consolidated into Daily and Weekly tables that give a one record per day/week version of the detail work center loads. This is done within the capacity limits for that day to allow load limit comparison and summaries.

Scheduled receipts and planned orders

Work orders are represented in RapidResponse as line items of supply orders and are stored in the **ScheduledReceipt** table. They are the drivers for creating load records through routing operations. The

type of supply order is defined by records in the **SupplyType** control table.

The **SupplyType** determines if a scheduled receipt is a work order (versus a purchase order) and needs to generate load in the Current and Planned capacity calculations.

Planned orders are created by MRP calculations as recommendations for new work orders and are stored in the **PlannedOrder** table. The PlannedOrderSupplyType, found on the **PartType** table, refers to the **SupplyType** table. This determines whether planned orders are work orders or purchase orders and, therefore, whether they will be used in the planned capacity calculations.

The **CapacityProcessingRule** in the **SupplyType** table controls whether load should be generated for the order type, as well as the appropriate scheduling method. Typically, only work orders will generate load. If load is to be generated, the **CapacityProcessingRule** determines if orders will be scheduled through the routing in a forward or backward direction. Normally only one of the two scheduling methods should be used in an implementation.

Possible uses of forward or backward scheduling are:

- **Forward** — used for orders where early completion is a priority. Slack time could potentially build up.
- **Backward** — used if minimizing early completion of orders is a priority. Due date will be achieved with no slack built up

RapidResponse does two separate load calculations with results in different tables. The current calculation uses only the scheduled orders (scheduled receipts). The order completion date is the DueDate field and the start date is the StartDate field unless it is undefined, in which case it is calculated from the DueDate.

If StartDate is undefined, an order start date is calculated as:

```
DueDate – (Part.EffFixedLeadTime + Part.EffVarLeadTime * Quantity)
```

The part's PlanningCalendars.TimeUnits is used as the shop calendar for the calculation.

The planned calculation uses the same dates that the MRP calculations use for the order's component allocations. The load will line up with the planned availability of the components. Both scheduled receipts and planned orders are used in the planned calculation. For scheduled receipts, the order completion date is the **ReschedDate** and the start date is the **CalcStartDate**. For planned orders, it is the **DueDate** and **StartDate**.

For both current and planned calculations, if the order start or completion date is not a working day, the date is pushed back to the most recent date that is a workday (defined by the **Part.PlanningCalendars.TimeUnits** calendar).

Routings

A routing identifies the sequence of operations that must be performed in order to manufacture a part from its direct components.

The **Routing** table holds the actual routing codes that group together operations. This table is referenced by the **OperationSequence**table which should identify at least one standard sequence of operations belonging to a given routing (multiple sequences are supported if parallel operations within a routing are required).

The **OperationSequence** table is in turn referenced by records in the **CRPOperation** table identifying the actual operations that belong to the sequence. For more information about configuring operations, see "Operations and sequences" on page 1522.

The **Routing** table then serves as the link between work orders and the specific work center operations required in their production. It is referenced both by the **PartSource** table, which uses it to determine all operations required in the production of a new planned order, as well as the **ScheduledReceipt** table which uses it to determine all operations required in production of the receipt.

Notes:

- To provide support for operations defined in earlier versions, the **Routing** table can also be directly referenced by operation records stored in the **Operation** table. However, configuring operations using the **CRPOperation** and **OperationSequence** tables is the current recommended practice.
- Optionally, the **AlternateRouting** table can be used to relate routings to parts as alternatives to the active routing identified in **PartSource.Routing**. A part can have multiple alternate routings and a routing can be used for multiple parts as an alternate routing. Note that these relationships are only for reporting purposes and are not used in the load calculations.

Operations and sequences

Operations are the individual steps performed at work centers when making a part from its immediate components. To define operations in RapidResponse, the **CRPOperation** table should be used. Each record in this table specifies a required work center, provides details on up to six different operation phases, and references a control setting in the **OperationType** table that defines operation processing.

Each **CRPOperation** record also references a record in the **OperationSequence** table which further references a **Routing** record. When a given scheduled receipt or planned order is run through that **Routing**, the **EffectiveInDate** and **EffectiveOutDate** values on the **CRPOperation** record are compared against the start date of the supply to determine if the operation is effective for processing the supply (additional control over effectivity is available through **OperationType.EffectivityRule**).

This section provides details on configuring operations in RapidResponse. This includes the following:

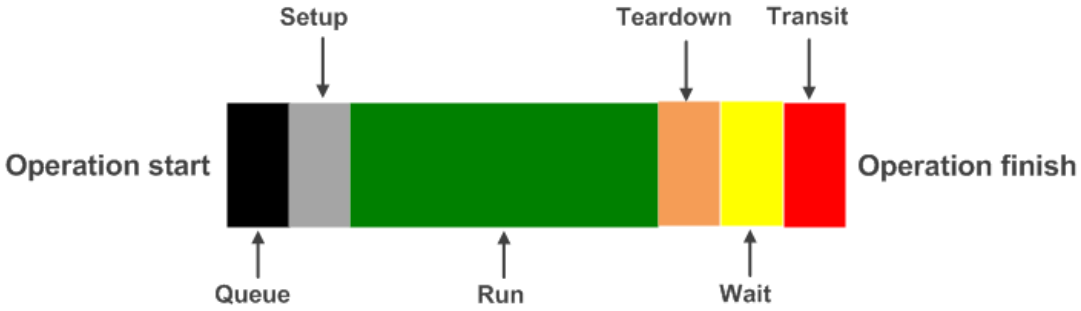
- "Operation phases and durations" on page 1523
- "Sequencing and parallel operations" on page 1524
- "Operation lot splitting" on page 1526
- "Ongoing operations" on page 1526
- "Overlapping operations" on page 1527
- "Aligning dependent requirements to operations" on page 1531
- "Operations and supply lead time" on page 1532

Note: To provide compatibility with previous versions of RapidResponse, operations can also be stored in the **Operation** table which directly references the **Routing** table and does not make use of **OperationSequence** values or provide support for related capabilities such as parallel operations. For more information, see "Operation table " on page 407.

Operation phases and durations

As shown in the following illustration, each work center operation can consist of up to six different phases which are applied to supplies being processed by the operation.

Figure 21.2 Operation phases



The following table outlines each of the phases of an operation that can be configured using fields in the **CRPOperation** table.

Table 21.6: CRPOperation phases

Phase	Configuration and description
Queue	The **QueueTimeOverride** field can be used to specify a number of hours to wait before operation setup can begin at the work center. Queue time is also specified in the **Capacity.QueueTime** field. The value on the **Capacity** record applicable to the work center is used unless a non-negative value is provided in **QueueTimeOverride**.
Setup	The **SetUpHours** field is used to specify the number of standard hours needed to set up a machine for use in the operation.
Run	The **RunTime** field is used to specify the amount of time required for the operation to process one unit of supply. By default, this is the number of hours required to run through one unit of supply at the operation. However, the **RunTimeUOM** field (reference to **CRPUnitOfMeasure** table) is used to determine the actual formula and scaling factors for converting the specified run time to standard hours per part. Once determined, the standard hours are multiplied by the order quantity to determine the total hours of run time needed for the supply.
Tear down	The **TearDownTime** field can be used to specify the standard number of hours needed for tear down of the work center after the run time phase has completed. For example, the time to remove parts and restore the work center to its normal state in preparation for the next order.
Wait	The **WaitTimeOverride** field can be used to specify a number of hours of waiting time after the operation run, and any applicable tear down, has completed at the work center. Wait time is also specified in **Capacity.WaitTime,** and the value on the **Capacity** record applicable to the work center is used unless a non-negative value is provided in **WaitTimeOverride**.
Transit	The **TransitTime** field is used to specify the amount of time needed to move an order between operations. By default, this is the number of hours required to move an order to the next operation in the sequence. However, the **OperationType.TransitTimeProcessingRule** setting determines if this value is interpreted as hours or days, and the **OperationType.TransitTimeSequenceRule** setting determines if the transit time is applied after or before the operation.

Subsequently, when determining work center load, the standard number of hours for the setup, run, and tear down phases are adjusted by any efficiency and utilization factors defined at the work center. The **WorkCenter.Efficiency** field can be used to specify a ratio between standard hours of work and the actual number of hour it typically takes to accomplish the work, while the **WorkCenter.Utilization** specifies the fraction of the workday during which the work center is actually available to do a job (accounting for factors such as break periods). By applying the efficiency and/or utilization factors to standard hours, an estimate of the number of elapsed hours the job will take to complete is determined. The elapsed time values of setup run, and tear down are then used in scheduling the work order(s) to perform the operations.

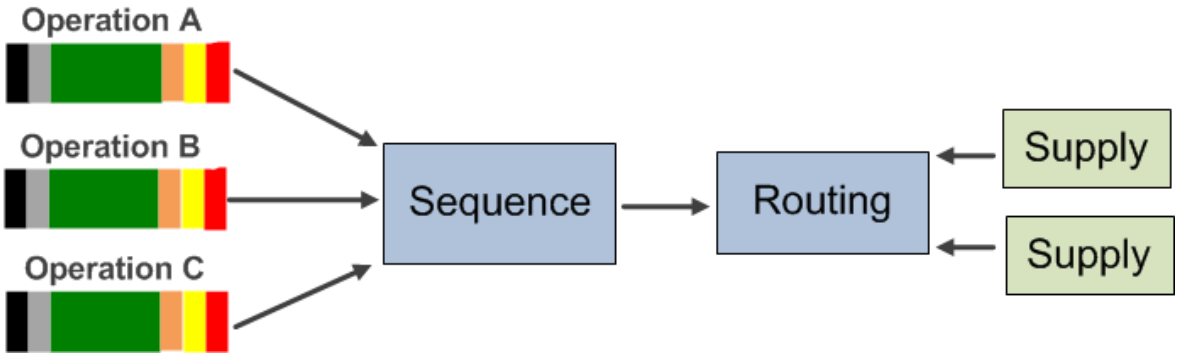
Notes:

- For complete information on how operation load is calculated from each operation phase, see "Operation loads" on page 1536.
- All **CRPOperation** fields described in "CRPOperation phases" on page 1523 are Quantity fields (thus, fractional hours can be specified as required).

Sequencing and parallel operations

Each sequence of operations within a given routing should be defined in the **OperationSequence** table. Records in this table are uniquely identified by their **Id** field and **Routing** table reference. Operation records, as defined in the **CRPOperation** table, are then tied to the supply routings where they are required via their **Sequence** reference as shown in the following illustration.

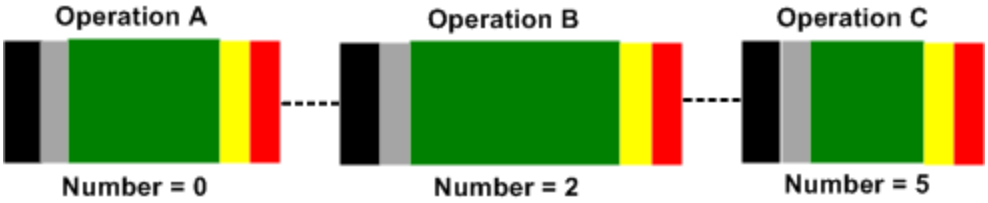
Figure 21.3 Operations belong to a sequence and routing



Each routing should have exactly one standard sequence of operations and, optionally, one or more parallel operations that branch off the standard sequence. To indicate a record in the **OperationSequence** table represents a standard sequence, it should reference an **OperationSequenceType.ProcessingRule** value of "Standard". All operations that are considered part of the routing's standard sequence should then reference that **OperationSequence** record.

The order in which operations within the standard sequence are performed is then determined by the **CRPOperation.Number** field. This field holds integer values and operations within a given sequence are ordered from lowest to highest value as shown in the following illustration. Note that if two operations within a sequence have the same **Number** value, then scheduling is determined by **WorkCenter.Name** (alphabetical) and **EffectiveInDate** (earliest to latest).

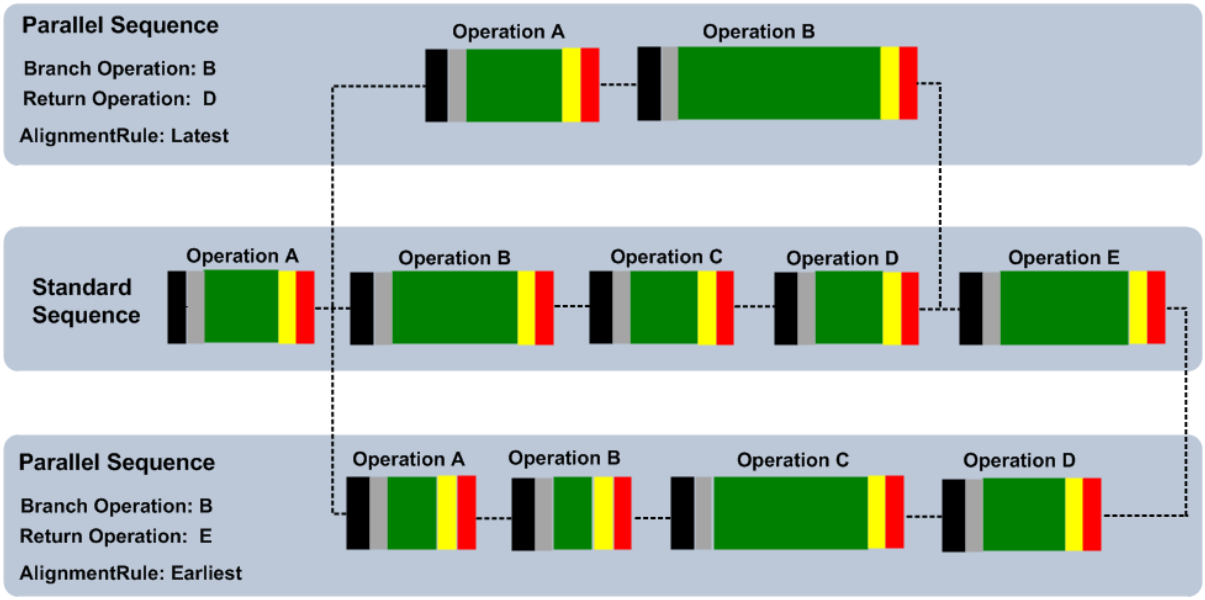
Figure 21.4 Operations ordered within a standard sequence



If a routing requires more than one sequence of operations, parallel sequences can be defined. A parallel sequence represents a group of operations that can be run concurrently with operations in the standard sequence. To indicate a record in the **OperationSequence** table represents a parallel sequence, it should reference an **OperationSequenceType.ProcessingRule** value of "Parallel". Operations within each parallel sequence are then ordered according to their **CRPOperation.Number** value.

The **BranchOperation** and **ReturnOperation** fields on the **OperationSequence** record are used to schedule parallel operations relative to the standard sequence. Each of these string fields should contain a valid **CRPOperation.Id** value for an operation in the standard sequence. **BranchOperation** indicates the operation before which the parallel sequence should branch, while **ReturnOperation** indicates the operation after which the parallel sequence branch should return or join back.

Figure 21.5 Parallel operations within a Routing



Note that if the duration of operations in the parallel sequence does not line up with the duration of the operations between its branching points in the standard sequence, the **AlignmentRule** field on the applicable **OperationSequenceType** record is used. This field has options of "Earliest", to align the start of the earliest operations at the branching point, and "Latest" to align the finish of the latest operations at the return point.

Additionally, if **BranchOperation** does not refer to a valid **CRPOperation.Id** value, the first operation in the standard sequence is used as the branching point. Similarly, if the **ReturnOperation** field is left blank or does not refer to a valid **CRPOperation.Id** value, the last operation in the standard sequence is used as the branching point. To indicate the actual branch points for a parallel sequence,

EffectiveBranchOperation and **EffectiveReturnOperation** fields are calculated on the **OperationSequence** record.

Note: If a routing does not have a standard sequence defined, then any parallel sequences belonging to the routing will be ignored (along with their **CRPOperation** records). Additionally if a routing has more than one standard sequence defined, RapidResponse will designate one as the standard sequence and ignore the others (along with their **CRPOperation** records).

Operation lot splitting

When an order is processed by an operation, it can potentially be split across multiple work center resources that can work concurrently on the operation. This can shrink down the actual time needed to run through the operation and allow it to finish quicker than it would if only one resource were assigned to it. For example, if three resources were applied to an operation that normally takes one resource six hours to complete, the operation could potentially be completed within 2 hours.

To enable operation lot splitting, the **CRPOperation.MaximumResources** field is used. This field specifies the maximum number of work center resources that can be concurrently applied to the operation. The actual number of work center resources that will be applied to the operation is then the minimum of the **MaximumResources** field and the actual number of resources available at the work center as specified in the **NumberOfResources** field on the **Capacity** record applicable to the work center.

Note: For more information about work center capacity and load from operations, see "Work centers and capacity" on page 1534.

Ongoing operations

An existing work order (scheduled receipt) might be partially completed by the work center operations in its routing when imported into RapidResponse. In cases of such ongoing operations, only the remaining portions of the order should be scheduled through their yet to be completed operations.

For an operation to be configured as ongoing (in progress), the **CRPOperationState** table is used. This table references both a **ScheduledReceipt** record to indicate the work order that has been partially been processed, as well as a **CRPOperation** record to indicate the specific operation, within a given operation sequence, that is in currently in progress. Also when an operation is marked as ongoing, this further indicates that all previous (non-batched) operations within the same operation sequence have already completed and do not need to be scheduled.

To specify the portion of an order that has already been completed by the current operation, the **CRPOperation.CompleteQty** field can be used. Note that for orders processed in batches, this field should indicate the total amount that has been processed across all batches. The difference between the **ScheduledReceipt.Quantity** field and **CRPOperation.CompleteQty** then gives the amount of the order that still needs to be scheduled through the current operation.

The **CRPOperationState** table also contains fields to indicate actual start dates and offsets (hours into the day) for specific operation phases. These are used to indicate the detailed state of in progress operations, and at least one actual start date must be specified on a **CRPOperationState** record for the operation to actually be considered in progress. Typically, a start date and offset should be should be specified for the phase that is currently in progress, or about to start, and load from the operation is

only generated from that point onward. For example, the **ActualProcessStartDate** field can be used to indicate that the operation is currently in progress at the run phase and earlier phases, such as setup, have already completed.

If start dates and offsets are specified for multiple operation phases, the one representing the latest phase within an operation is considered to represent the current state of the operation and the others are considered to be completed. That is, the start date fields are evaluated in the following order and the first one that contains a date is used to define the current state of the operation:

1. **ActualTearDownStartDate / ActualTearDownOffset—**indicates when work center down tear down began for the operation.
2. **ActualProcessStartDate / ActualProcessingOffset—**indicates when the actual processing, or run time, of the order began for the operation.
3. **ActualSetupStartDate / ActualSetupOffset—**indicates when work center setup began for the operation.
4. **ActualStartDate / ActualStartOffset—**indicates the overall start time for the operation.

Additionally, an **ActualFinishDate** field is available to indicate that a referenced operation has already completed. This field is available in case maintaining details of historical operations is required, but has no algorithmic significance. However, note that any time the **ActualFinishDate** is populated on a **CRPOperationState** record, the referenced operation is assumed to be complete and any other date and quantity values provided on the record are ignored.

Finally, note that if parallel operations are configured within a supply routing, the scheduled receipt can have concurrent ongoing operations across sequences. However, a given scheduled receipt should typically not have more than one ongoing operation within a given operation sequence. If there are multiple **CRPOperationState** records linking the same scheduled receipt to different operations in the same sequence, then the latest non-batched operation found within the set is considered to be actually ongoing and the others are considered finished. Also, If there is at least one ongoing operation in the standard sequence that falls between the branching and return points of any parallel sequences, then the applicable **OperationSequenceType.AlignmentRule** setting maybe ignored.

Notes:

- For more information about the fields on the **CRPOperationState** table, see "CRPOperationState table" on page 260.
- For compatibility with previous versions, if work center operations are defined in the **Operation** table instead of the **CRPOperation** table, the **CurrentOperation** and **CurrentOperationCompleteQuantity** fields on the ScheduledReceipt table can be used to define the ongoing state of an operation. For more information about these fields, see "ScheduledReceipt table " on page 569.

Overlapping operations

Operations can be configured to run sequentially or to overlap.

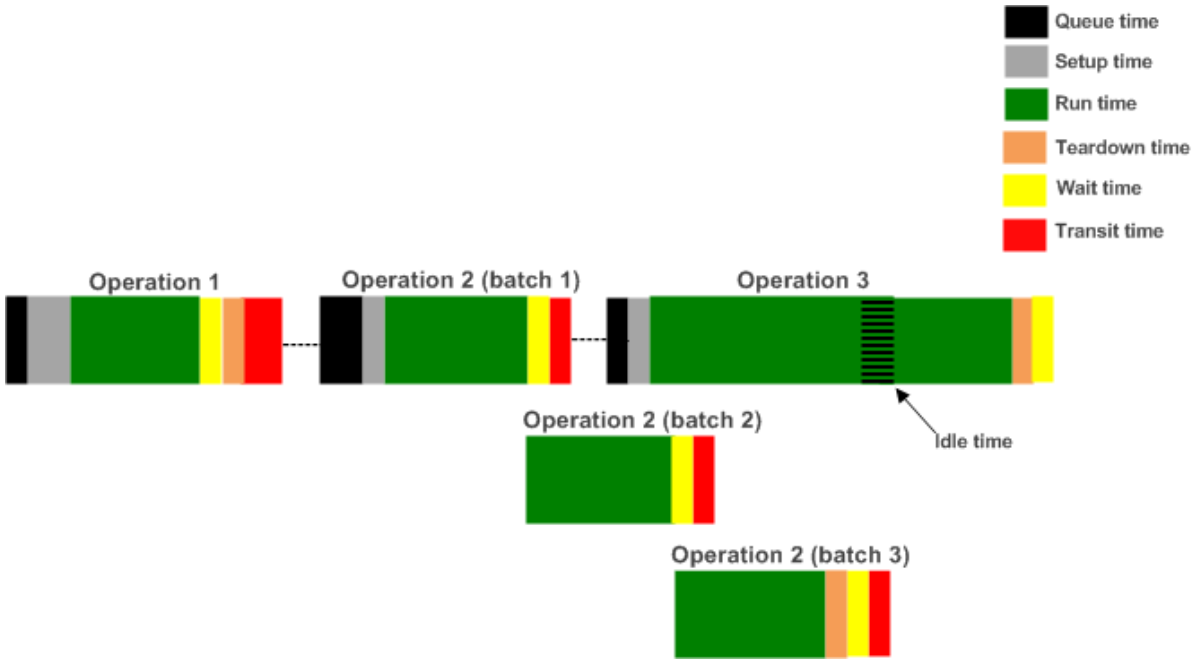
When an operation is set to run sequentially, each of its phases must complete the processing of a supply before that supply can be moved to the next operation in the sequence for processing. This can be seen in the following figure, which shows three sequential operations.

Figure 21.6 Routing with sequential operations only

With overlapping operations, a given operation does not necessarily need to finish before the next operation in the sequence can begin. This can be seen in the following figure, where Operation 2 processes a supply in distinct batches. After each batch is processed, it is then sent ahead to the next operation in the sequence for processing. Because subsequent operations can then start sooner, this typically results in the routing finishing earlier than it otherwise would. However, an operation may also incur idle time during its run phase while waiting for batches of supply to be received from the previous operation.

Note also that when an operation is processed in batches, each batch incurs per unit run time along with any applicable wait and transit times. However, only the first batch in an operation will incur any queue and/or wait times, while only the last batch in the operation will incur any applicable tear down time.

Figure 21.7 Routing sequence with overlapping operations



To configure overlapping operations, the **CRPOperation.BatchQuantity** field is used to specify each operation's batch size. This indicates the amount of an order that the operation must process before

sending that batch ahead to the next operation. When the next operation receives the supply batch, it can begin processing it as long as the amount received satisfies its own batch size. For example, suppose the current operation has its **BatchQuantity** set to "5". If the next operation also has a **BatchQuantity** of "5", it can start as soon as it receives the first batch of five units. Instead, if that next operation in the sequence has its **BatchQuantity** set to "10", it cannot start until it has received two supply batches of five units.

For operations that aren't meant to produce supply in batches, the **BatchQuantity** field should be set to "0". This is the default setting and indicates that the operation must process each order in its entirety before sending it ahead to the next operation in the sequence. When a given operation has a **BatchQuantity** of "0", but receives supply in batches from the previous operation, the **OperationType.BatchQuantityRule** field is used to define when that operation can start. If **BatchQuantityRule** is set to "UseZero", the operation cannot start until the previous operation has processed the entire order. Otherwise, if **BatchQuantityRule** is set to "IgnoreZero", the operation can start as soon as it receives a batch of supply from the previous operation (note that the example shown in "Routing sequence with overlapping operations" on page 1528 assumes that Operation 3 uses the "IgnoreZero" setting).

Note: Overlapping operations should not be used with days rounding logic applied to the transit time.

Example - Sequential vs. Overlapping operations

Assume a routing is made up of three operations with the time, in hours, for each operation phase as shown in the following table.

Table 21.7: Sample operation times in hours (fixed times and run time per unit)

Operation	Queue	Setup	RunTime	Teardown	Wait	Transit
1	.25	.50	.75	0	.50	1
2	.50	1	3	.75	.50	.50
3	.25	1	1.5	.50	1	0

Also assume a supply for 6 units is being processed and the operations in its routing are forward scheduled starting on June 23, 2014 with each day consisting of 8 working hours.

If each operation in the routing has **CRPOperation.BatchQuantity** = 0, then the dates and offsets associated with each operation would be as shown in "Dates and offsets sequential operations" on page 1529. Note that the first operation begins at the start of the working day on June 23 and the final operation ends 7 hours and 45 minutes into the working day on June 27.

Table 21.8: Dates and offsets sequential operations

Op	Queue		Setup		Run		Wait		End	
#	Date	Offset	Date	Offset	Date	Offset	Date	Offset	Date	Offset
1	06-23	0.00	06-23	0.25	06-23	0.75	06-23	5.25	06-23	6.75
2	06-23	6.75	06-23	7.25	06-24	0.25	06-26	3.00	06-26	4.00
3	06-26	4.00	06-26	4.25	06-26	5.25	06-27	6.75	06-27	7.75

Further, "Run time with sequential operations" on page 1530 shows the total run times for each of the operations outlined in the previous table. In this case, all run times represent actual processing time and are equal to run time per unit multiplied by the number of units processed with no idle time incurred.

Table 21.9: Run time with sequential operations

Operation	Run (hours)	Idle (hours)
1	4.5	0
2	18	0
3	9	0

Instead assume that the **CRPOperation.BatchQuantity** value for Operation 2 is set to "2", with the other two operations still having a **BatchQuantity** of "0" and assuming a **Type.BatchQuantityRule** of "IgnoreZero" for each.

In this case, the dates and offsets associated with each operation, including batch times for Operation 2, would be as shown in "Dates and offsets with an overlapping operation" on page 1530. Note that the first operation still begins at the start of the working day on June 23, but the final operation now ends only 30 minutes into the working day on June 27. Thus, the total time to complete the routing has been reduced by 7 hours and 15 minutes (due to Operation 3 starting sooner and being able to process batches of supply received from Operation 2).

Table 21.10: Dates and offsets with an overlapping operation

Op	Queue		Setup		Run		Wait		End	
#	Date	Offset	Date	Offset	Date	Offset	Date	Offset	Date	Offset
1	06-23	0.00	06-23	0.25	06-23	0.75	06-23	5.25	06-23	6.75
2 (B1)	06-23	6.75	06-23	7.25	06-24	0.25	06-24	6.25	06-24	7.25
2 (B2)	---	---	---	---	06-24	6.25	06-25	4.25	06-25	5.25
2 (B3)	---	---	---	---	06-25	4.25	06-26	3.00	06-26	4.00
3	06-24	7.25	06-24	7.50	06-25	0.50	06-26	7.50	06-27	0.50

Further, "Total run time with an overlapping operation" on page 1530 shows the total run times for each of the operations outlined in the previous table. Note that the run time reported for Operation 3 is now longer than shown in "Total run time with an overlapping operation" on page 1530 as it includes 5.5 hours of idle time waiting for batches from Operation 2 to arrive.

Table 21.11: Total run time with an overlapping operation

Operation	Run (hours)	Idle (hours)
1	4.5	0
2	18	0
3	14.5	5.5

More specifically, the run time reported for Operation 3 can be broken down into the following portions:

Table 21.12: Operation 3 run time details

Task	From (Date + Offset)	To (Date + Offset)	Duration (Hours)
Processing Batch 1 from Operation 2	06-25 + 0.50	06-25 + 3.50	3.00
Idle/Waiting for Operation 2, Batch 2	06-25 + 3.50	06-25 + 5.25	1.75
Processing Batch 2 from Operation 2	06-25 + 5.25	06-26 + 0.25	3.00
Idle/Waiting for Operation 2, Batch 3	06-26 + 0.25	06-25 + 4.00	3.75
Processing Batch 3 from Operation 3	06-26 + 4.00	06-26 + 7.00	3.00

Note: For more information about each of the operation "Date" and "Offset" field values referenced in this section, see "Operation loads" on page 1536.

Processing partially complete operations

As discussed in "Ongoing operations" on page 1526, the **CRPOperationState** table can define partially complete operations. For example, if **ScheduledReceipt.Quantity** is 100 and a record in the **CRPOperationState** table has **CompleteQty** of 40 along with valid references to that **ScheduledReceipt** and a **CRPOperation** record, this means that the referenced operation has processed 40 units of the supply with 60 units remaining to process.

Because different operations may take different times to process their current batches, RapidResponse performs the following set of calculations for subsequent operations when **CompleteQuantity** > 0.

1. If **CRPOperation.BatchQuantity** = 0, then subsequent operations must process the entire **Quantity**.
2. Otherwise if **CompleteQuantity** < **BatchQuantity**, then the current operation has not completed the first batch and subsequent operations must process the entire **Quantity**.
3. Otherwise if **CompleteQuantity** > **BatchQuantity**, calculate TimeA, the difference between the time when the subsequent operation starts processing the remaining quantity and the start time of the supply on the current operation, as well as TimeB, the product of the subsequent operation's run time and the **CompleteQuantity** (that is, the time it takes the subsequent operation to process the quantity already completed by the current operation). Then
   - if TimeA <= TimeB, Time A is included in the subsequent operation's run time, along with the time it takes the subsequent operation to process the remaining quantity. Queue and setup times are not included for the subsequent operation.
   - otherwise if TimeA > TimeB, TimeB is included in the subsequent operation's run time along with the time it takes the subsequent operation to process the remaining quantity. Queue and setup times for the subsequent operation are included before its run time.

Aligning dependent requirements to operations

RapidResponse generates dependent demands from new planned orders and unallocated scheduled receipts. By default, the due date on the component demands is set to the start date of the assembly supply that generated them. This implies that all component supplies must be available before work can begin on the assembly.

However, dependent demand due dates can instead be aligned to the start date of specific work center operations within the assembly supply's routing. This allow component supplies to be due at the actual point where they are needed in the production process. For example, this might be useful in cases of assembly's with long production times where certain components are otherwise due much earlier than they are truly needed.

To align dependent demand due dates to specific work center operations, the **BillOfMaterial.Operation** field is used. This field is a nullable reference to the **CRPOperation** table and should refer to a valid operation within the assembly supply's routing. The referenced operation can exist within either the routing's standard sequence, or one of its optional parallel sequences. If the **Operation** reference is left blank, then dependent demands generated for the component are always placed at the start date of the assembly supply that generated them.

Note: For imported allocations on scheduled receipts, the **Allocation.Operation** field can be used to indicate the specific operation in the scheduled receipt's routing where the component is needed.

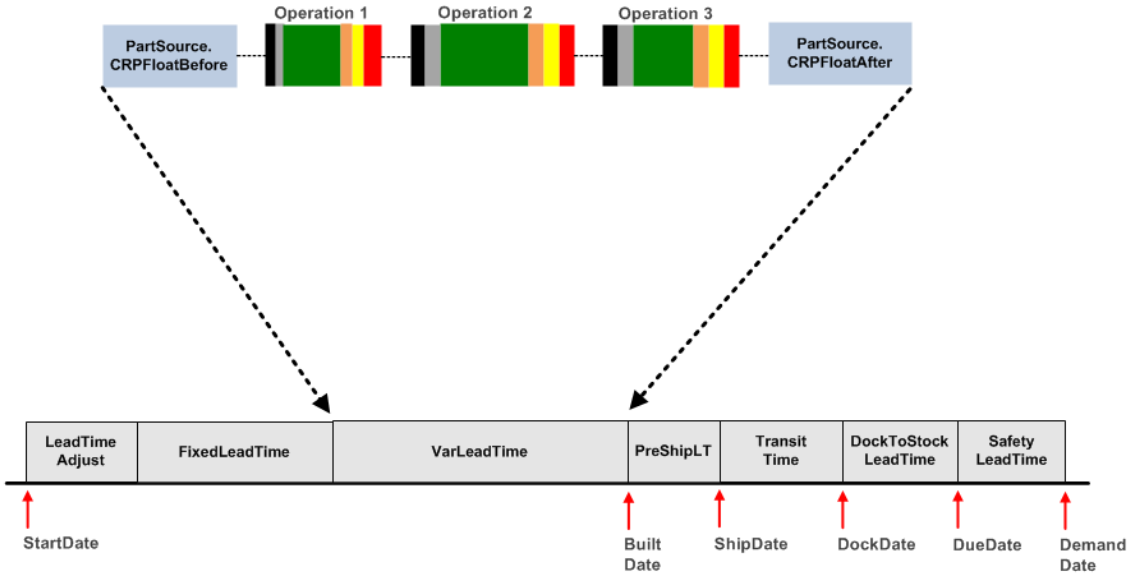
Operations and supply lead time

Supply lead times and start dates, as calculated by the Netting analytic, can be tied to work center capacity and the actual time needed to process a supply through all work center operations in its routing. For example, if there is limited work center capacity for a period, then the time taken to process the supply might be longer. Similarly, if additional work center resources are made available to the operations in its routing, a supply might complete sooner. To align supply lead times with actual work center operation times, variable lead time can be used as shown in the following illustration.

Variable lead time is enabled at the part source level by the referenced control value in the **OrderPolicy.UseVarLeadTime** field. When this option is set to "CRP", the time to process the order across all operations in the supply's routing determines the variable portion of lead time with the other lead time elements applied around it based on fixed input field values. Otherwise, if **UseVarLeadTime** is set to "Y" (variable lead time is calculated as **Quantity * PartSource.VarLeadTime**) or "N" (variable lead time is disabled), then there is not a link between work center processing times and supply lead time.

When using variable lead time based on work center capacity (UseVarLeadTime = "CRP"), fixed float times can also be added before and/or after the calculated time needed to run through the supply's routing. As shown in the following illustration, these floats are specified on two fields in the PartSource table. The CRPFloatAfter field specifies an optional amount of time to leave in the schedule after the final operation in the routing completes. For example, this can act as a buffer to account for unexpected delays or interruptions incurred during work center operations. The CRPFloatBefore field specifies an optional amount of time to leave in the schedule before the first operation in the routing begins. For example, this can act as a buffer to account for unexpected delays in the receipt of component supplies.

Figure 21.8 Operations and cumulative lead time



Additionally the "CRP" variable lead time is used in determining the point from which operation scheduling and load generation begins for an order. This can be further influenced by whether inclusive or cumulative lead time is used (as determined by the applicable setting in **PartSourceType.TransitRule**), and whether the operations in a supply's routing are backward or forward scheduled as follows:

- When **SupplyType.CapacityProcessingRule** = "Backward", operation scheduling and load generations starts from the supply due date if either the **TransitRule** is "Inclusive", or the **TransitRule** is "Cumulative" and **UseVarLeadTime** is either "N" or "Y". Otherwise, if **TransitRule** is "Cumulative" and **UseVarLeadTime** is "CRP", then backward operation scheduling starts from the "BuiltDate".
- When **SupplyType.CapacityProcessingRule** = "Forward", operation scheduling and load generation starts from the supply start date if **UseVarLeadTime** is either "Y" or "N". Otherwise, if **UseVarLeadTime** is "CRP", forward operation scheduling starts from *StartDate + (LeadTimeAdjust + FixedLeadTime)*.

Notes:

- For more information about inclusive and cumulative lead time, as well as details on how to enable or disable each lead time element from use in determining supply lead time and start dates, see "Lead time calculations" on page 1307.
- If a supply's **SupplyType.CapacityProcessingRule** is set to "Ignore", then an **OrderPolicy.UseVarLeadTime** setting of "CRP" is treated the same as "N" (variable lead time is not used). For more information about **CapacityProcessingRule** see, "Scheduled receipts and planned orders" on page 1520.
- Work center capacity is evaluated independently for each order. That is, the same total capacity is made available to each operation loaded onto the work center and this can result in overloaded work centers. Therefore, to ensure the calculated operation and supply schedules reflect what is actually achievable, work centers should be monitored for overloads. For more information, see "WorkCenter loads" on page 1538.

Work centers and capacity

The **WorkCenter** table defines the target of operations and represents the resources needed to perform an operation. For example, a work center typically represents a machine (or machine group or work area with tooling) and the labor required for operating it. Each record in the **WorkCenter** table is uniquely identified by the **Name** and **Site** field combination, and also contains fields for defining labor and machine costs and the work center.

The **WorkCenter** table has a **Type** value that references control settings defined in the **WorkCenterType** table. This control table defines the processing rules and the default parameters that are associated with each type of **WorkCenter**. For example, the **DefaultHoursPerDay** field is used to specify the default number of working hours per day that are assumed for any work centers that do not have **Capacity** record(s) defined. The **WorkCenter** table also has a **PlanningCalendars** reference that determines the work center's working calendar and run date.

Finally, work centers are grouped into departments for reporting and filtering. The **Department** table defines unique department names in the **Id** field. This field is referenced by the **Department** field in the **WorkCenter** table.

Note: For more information about work centers, see "WorkCenter table " on page 666.

Capacity

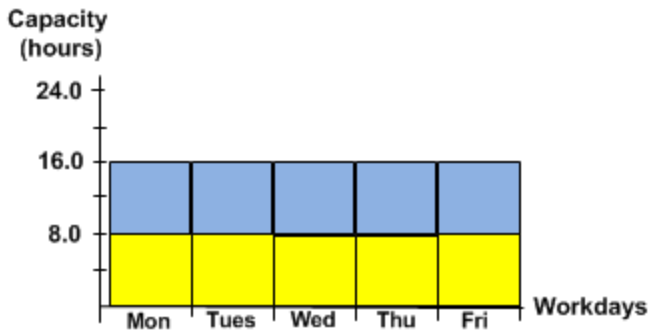
Each **WorkCenter** record is typically referenced by one or more records in the **Capacity** table. Capacity records contain information about the work center's operating schedule and characteristics such as number of working hours per day, number of resources available, efficiency and utilization rates, as well as machine and labor load ratios. For example, suppose a lone **Capacity** record for a work center with values as shown in the following table.

Table 21.13: Sample values on a Capacity record

WorkCenter	Hours PerDay	NumberOf Resources	Efficiency	Utilization
AA	8.0	2	1	1

Assuming that work center "AA" uses a Monday through Friday planning calendar, then the daily capacity for the work center would be as seen in the following figure, with total work center capacity for each day being 16 hours (*2 * 8 * (1 * 1)*).

Figure 21.9 Daily work center capacity (from Capacity record)



Note that when multiple resources are specified on a **Capacity** record, they are assumed to operate in parallel. When operation splitting is enabled, this typically results in an earlier finish. For example, if a given operation were allowed to be split between the two resources shown in this example, it could place up to 16 hours of load on the work center on a given day. Otherwise, it could only place up to 8 hours of daily load on the work center.

Each **Capacity** record also has an **EffectiveInDate** that defines when its parameters can first be used in load calculations for the work center. A **Capacity** record then stops being effective when another record with a later **EffectiveInDate** is found referencing the same work center. Note also that if multiple **Capacity** records are added having the same **WorkCenter** and **EffectiveInDate** values, only one of these will be used in capacity and load calculations (the others get ignored).

Day and shift-based capacity

Work center capacity can also be specified in the **CapacityOverride** table. This table allows for work center capacity details such as hours per day, efficiency, and utilization to be defined by specific day of the week. This table also has an **EffectiveInDate** field to indicate when the capacity values become effective. For each day of the week on which a work center has an effective record defined in the **CapacityOverride** table, that record is used to define a work center's capacity. On days of the week for which an effective record is not specified in the **CapacityOverride** table, the work center's effective **Capacity** record is used.

For example, assume work center AA has a default **Capacity** record with values as shown in "Sample values on a Capacity record" on page 1534, but also has just 4 working hours per day on Mondays. In this case, a record with values similar to the following could be added to the **CapacityOverride** table.

Table 21.14: CapacityOverride record for a day of the week

WorkCenter	DayOf Week	Index	Hours PerDay	NumberOf Resources	Efficiency	Utilization
AA	Monday	0	4	2	1	1

Additionally, a work center can have multiple **CapacityOverride** records added for a given day of the week to indicate separate working shifts in the day. The capacity from each of these shifts is then summed together to give the work center's total capacity for the day. However, note that in order for multiple **CapacityOverride** records having the same **WorkCenter**, **DayOfWeek**, and **EffectiveInDate** to be used together, each must have a unique **Index** value (otherwise, just one of the records will be used in capacity/load calculations, and the others ignored).

For example, assume work center AA has a default **Capacity** record with values as shown in "Sample values on a Capacity record" on page 1534, but also has an additional 8 hour shift added on Fridays with one resource assigned and an efficiency rate of 75% assumed. In this case, records with values similar to the following could be added to the **CapacityOverride** table.

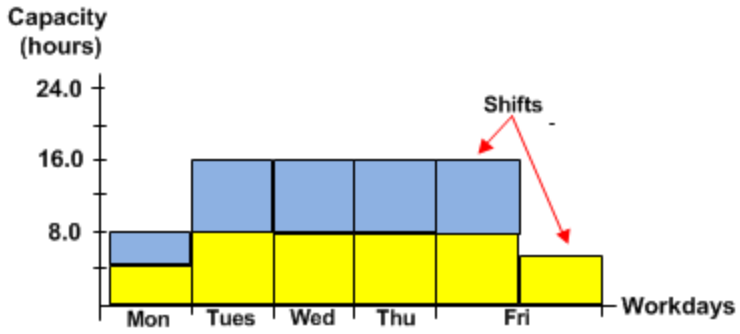
Table 21.15: Multiple CapacityOverride records for the same day of the week

WorkCenter	DayOf Week	Index	Hours PerDay	NumberOf Resources	Efficiency	Utilization
AA	Friday	0	8	2	1	1
AA	Friday	1	8	1	.75	1

Thus, based on the **CapacityOverride** records shown in "CapacityOverride record for a day of the week" on page 1535 and "Multiple CapacityOverride records for the same day of the week" on page 1535, daily capacity for work center "AA" would be as seen in the following figure. Note that based on the

overrides, work center capacity on Monday is 8 hours (*2 * 4 * (1 * 1)*) and work center capacity on Friday is 22 hours (*((2 * 8 * (1 * 1)) + (1 * 8 * (.75 * 1))*).

Figure 21.10 Daily work center capacity (with CapacityOverride records)



Note: For more information about the tables used to define work center capacity, see "Capacity table" on page 234 and "CapacityOverride table" on page 236.

Operation loads

For each work order (scheduled receipt or planned order) that references a **SupplyType.CapacityProcessingRule** other than "Ignore", an operation load record is generated for each operation in its routing. Depending on the type of order, operation load records are stored in the following calculated tables:

- **LoadOperationNewOrderPlanned** (for planned orders)
- **LoadOperationReceiptsCurrent** (for scheduled receipts; current due date)
- **LoadOperationReceiptsPlanned** (for scheduled receipts, recommended due date)

Each record in these tables reports the actual hours duration of the setup and run phases of the load, and contains fields giving detailed dates and offset (hours into the working day), for each operation phase. For example, a **StartDate** value of "06-30-14" and **StartOffset** value of "1.25" indicates the operation started one hour and fifteen minutes into the work centers available working hours on June 30th. The following pairs of fields identify the different phases of each operation:

Table 21.16: LoadOperation date and offset fields

Fields	Operation phase reported
StartDate / StartOffset	The date and hours into the work center's working day at which the overall operation begins. If **OperationType.TransitSequenceRule** is set to "AfterOperation", these fields return the same values as the **QueueDate** and **QueueOffset** fields. Otherwise, if **TransitSequenceRule** is "BeforeOperation", then these fields account for pre-operation transit time.
QueueDate / QueueOffset	The date and hours into the working day at which pre-operation queue time begins (time spent waiting for the start of the setup phase).
SetupDate / SetupOffset	The date and hours into the working day at which work center setup begins for the operation. This occurs after the queue phase has completed.

Table 21.16: LoadOperation date and offset fields

Fields	Operation phase reported
RunDate /RunOffset	The date and hours into the working day at when run time or actual supply processing begins for the operation. This occurs after the setup phase has completed.
WaitDate / WaitOffset	The date and hours into the working day at which the wait phase begins for the operation. This is time the supply has to wait before moving to the next operation and occurs after the run phase and any applicable tear down time has completed.
EndDate / EndOffset	The date and hours into the working day at which the overall operation is seen as completed. If **OperationType.TransitSequenceRule** is set to "AfterOperation", this represents the end of operation transit time. Otherwise, if **TransitSequenceRule** is "BeforeOperation", then this represents the end of operation run time.

A work order is scheduled through the effective operation records in its routing by either the forward scheduling or backward scheduling sequence, as specified by the order type. At each operation step, the elapsed hours of the six phases of the operation are used in sequence either forward or backward to calculate the points in a day that the phases will start. The point in time of the end of one operation defines the beginning of the next for forward scheduling. In backward scheduling, this is reversed.

Finally, the **PrevWorkCenter** and **NextWorkCenter** fields give details about where the assembly is coming from before the operation, and where the assembly is going to after the operation.

Note: When an operation date marker falls at the end of the current day/beginning of next day, it reports the beginning of the next day for the start of an operation (for example, StartDate and StartOffset) and the end of the current day for the end of an operation (for example, EndDate and EndOffset).

Dates and operating schedules

An operation or phase may span several days. The work center parameters defining working days and hours in each day are used to determine the days, and number of working hours offset into the day, at which each operation's phases begin. Production only happens on working days. The **WorkCenter** table has a PlanningCalendars field identifying the **PlanningCalendars** table record that defines the operating schedule for the work center. This is the same as is used by the **Part** table in MRP processing. The TimeUnits field on the **PlanningCalendars** table identifies a calendar for spans of time. It usually represents working days. The units used in operations are in hours or fractions of hours. The work center Capacity record's value for HoursPerDay gives the number of hours available for production per working day. Because a work center can have multiple Capacity records with different EffectiveInDate values, the hours per day can vary with the date.

Scheduling

There are two methods available to schedule a sequence of operations. Forward scheduling starts with the first operation at the beginning of the day of the order start date and proceeds through each operation's pre-transit, queue, setup, run, wait and post-transit. Backward scheduling starts with the last operation scheduling it to complete (for example, end of the post-transit) at the end of the workday before the due date. The supply order type determines which scheduling method is used and is usually defined to operate in the same mode as the enterprise data source. It is possible to have different types

of orders use different scheduling methods. It may be useful to schedule high priority orders using forward scheduling. This ensures that they have room for delays. It also means that low priority orders are scheduled backwards, so as not to finish early and cause storage problems on the floor if there are changes.

The choice of scheduling method is made at integration time by defining appropriate **SupplyType** control table entries for the various types of work orders and setting the **CapacityProcessingRule** appropriately. The rule can also be set to **Ignore**. This will cause the capacity processing to be skipped for that type of order (for example, a sub-contracted order).

The combination of the workday calendar and time phased hours per day allows capacity planning to schedule loads at a very detailed level. It is important to note that many of the factors used in Capacity Planning are estimates or average values (for example, queue times, efficiency and utilization factors). Capacity Planning is necessary for getting a feel for the overloading or under loading of work centers for Resource Planning in the medium and long-range time frames.

WorkCenter loads

WorkCenter load tables allow the entire daily load for a work center to be accessible in a single table as machine and labor load numbers. There is a detail table, a daily bucketed table, and a weekly bucketed table for each of the two planning methods (Current and Planned). The daily bucketed version includes available capacity values along with the load and is useful for comparing the load picture against capacity. The work center load tables are the ones usually used in capacity reports. The load numbers are not equivalent to elapsed time, but rather are represented as standard units.

Detail WorkCenter load

The load detail tables are the result of calculating machine and labor loads from the operation load results, and distributing the loads of multi-day duration records across multiple daily detail records. The load for each of machine and labor is broken down into setup and run load. The transit, queue, and wait phases of an operation do not produce load. They only affect the timing of when the load is generated. Work center detail load records are only produced for days that are within the setup or run phases of an operation load.

Operation load record Setup and Run durations field values are converted back to standard hours by multiplying by the work center efficiency and utilization. Separate loads on machine and labor are calculated from this standard hour load by multiplying by one the LoadRatio parameters on the **Capacity** table.

Loads with setup and run times that span several days are spread across the days by using several detail records. The load is apportioned to the days based on the number of hours that day contributes to the total setup or run duration. Parts are only produced during the run phase of an operation so the order (scheduled receipt or planned order) quantity is apportioned to the detail load records using the same ratio as the run time load. Three quantity fields are added to reflect the portion of the job completed on a specific date:

- **QtyThisDate —** is the amount produced by the load record in that day
- **QtyRemaining —** is the amount left to be produced in subsequent days
- **QtyToDate —** is the amount of the order produced on this date and any previous days on which the order was being worked

The OperationStartDate and OperationEndDate are the QueueDate and EndDate respectively from the operation load.

The detail results of the current capacity calculation are stored in the **LoadDetailCurrent** table. The orders always come from scheduled receipts so the order is identified by a reference to the **ScheduledReceipt** table. The operation is identified by a reference to the **Operation** table.

The Planned capacity calculation produces two tables from the **Operation** table. One for scheduled receipts and the other for planned orders. The **LoadDetailPlanned** table stores the detail load from both these tables. The order therefore cannot be referred to directly because of the multiple source tables. Instead, the order is identified by the following: type: Order Id; Line for scheduled receipts; and by Part and due date for planned orders. Relevant order fields are replicated on the load record. The Operation field references the **Operation** table.

The Source field specifies the tables that generated the load (scheduled receipt or planned order).

The PrevWorkCenter and NextWorkCenter fields identify the previous and next work centers from which the part is coming or going, based on the effective operation. The fields are strings, not references, to the **WorkCenter** table. The PrevWorkCenter value will be blank for the first operation of the effective routing and NextWorkCenter will be blank for the last.

Daily summary

There are two daily summary tables: one for the current calculation and one for the planned calculation. They are called LoadDailyCurrent and LoadDailyPlanned. One record per day that has load is produced, giving the total load. This is a daily bucketing of the detail load consolidation. In addition all working days between the RunDate and the WorkCenterType CapacityReportLimit or last load (whichever is greater) have a record generated even if they have no load. This is to report available capacity.

The available machine and labor capacities are computed for each day from the effective capacity record for that day. The calculation depends on the work center type standard hour conversion rule and makes use of the machine and labor run ratios to display the capacities for machine and labor separately. There are three possibilities for the MachineCapacity and LaborCapacity fields:

- **HoursPerDay —** formula is HoursPerDay * NumberOfResources * Efficiency * Utilization * RunRatio
- **Demonstrated capacity —** uses the DemonstratedCapacity
- **Maximum capacity —** uses the MaximumCapacity

The MaximumCapacity and DemonstratedCapacity come from the effective WorkCenter value.

The NumberOfMachines and NumberOfWorkers are calculated by multiplying the NumberOfResources by the MachineRunRatio or LaborRunRatio.

In the **LoadDailyPlanned** table there is an additional set of four load fields. These are the load values just from ScheduledReceipt orders. The fields are named SRMachineSetup, SRLaborSetup, SRMachineRun, SRLaborRun. The other fields are the load from both planned orders and scheduled receipts.

Weekly summary

There are two weekly summary tables: one for the current calculation and one for the planned calculation. They are called LoadWeeklyCurrent and LoadWeeklyPlanned. One record per week is

produced giving the total load. This is a weekly bucketing of the daily load consolidation.

Limitations on other analytics with CRP

As discussed earlier in this chapter, certain Capacity Requirements Planning logic impacts the results generated by other analytics such as Netting. The use of various CRP settings is also incompatible with, or causes limitations on, other analytics and calculations as discussed in the table below.

Table 21.17: Limitations on other analytics when using Capacity Requirements Planning (CRP)

Analytic	Limitation
Constraint Analysis	Constraints that reference any setting in **ConstraintType.ProcessingRule** other than "LoadOnly" are automatically disabled on parts that use variable CRP lead time (but "LoadOnly" constraints are supported on parts that use CRP variable lead time).
Multi-level search logic	Variable CRP lead time is automatically disabled for parts in an MLS family.
MPSConfig	Variable CRP lead time is disabled for MPSConfig part sources. The Operation reference on BillOfMaterial and PartSource records for MPSConfig parts is also ignored.
BOM substitution	BOM substitution is not supported across different components tied to the start of different operations.
Material expiry	If a part source is configured for variable CRP lead time on parts that expire, a setting other than "StartDate" in **PartSource.ExpiryDateRule** should be used.
Order point	It is recommended not to enable variable CRP lead time for order point parts.
Incremental availability	Incremental availability is automatically disabled for parts that use CRP variable lead time (where the part source has **OrderPolicy.VarLeadTime** set to "CRP").

CHAPTER 22: Kanban calculations

Support for kanban material replenishment systems in RapidResponse calculations is provided through optional kanban logic. Kanban calculations in RapidResponse can be used to ensure that some maximum number of bins are in process at any one time. As well, it ensures that any new planned orders created will be for the current approved bin quantity. This chapter explains how kanban calculations impact both the netting analytic and capable-to-produce analytics, and outlines some limitations kanban has on other analytics. It also introduces the key data model fields used in kanban calculations.

In this chapter, you'll learn about:

Enabling kanban calculations **1541**
Impacts on netting calculations **1542**
Impacts on capable-to-promise calculations **1542**
Limitations on other analytics with kanban **1543**

Enabling kanban calculations

Kanban calculations in RapidResponse are enabled at the part source level by setting the value of **OrderPolicy.MaximumUsage** to "Kanban". With this setting, planned orders using the source are set to the current approved bin quantity, and total net supply resulting from a new planned order cannot exceed the product of the current approved bin quantity and the current total approved number of bins in the system. For more information about this setting, see "MaximumUsage" on page 775.

Kanban calculations also require that a given part source has a corresponding entry in the **SourceKanban** table. This table is used to model kanban parameters for a given source. For example, **SourceKanban.BinQty** defines the current approved quantity contained within each bin, and **SourceKanban.Bins** defines the total number of bins in the system. Note that SourceKanban records do not have effectivities on them, but kanban parameters can be made dynamic (time-phased) by creating additional PartSource records. That is, to change kanban parameters (for example, BinQty) over time, a new PartSource record should be created with the desired effectivity, along with a new SourceKanban record that refers to that new PartSource record. For more information about the SourceKanban table, see "SourceKanban table" on page 607.

Note: RapidResponse also includes the Kanban Manager workbook. This application can be used to calculate new bin sizing recommendations based on input data and kanban parameters. However, kanban logic in the netting and capable-to-promise analytics can be enforced with or without use of this application. Similarly, the Kanban Manager workbook can be used without enforcing kanban logic in the netting and capable-to-promise analytics. For more information about Kanban Manager, see "Kanban Manager calculations" on page 2239.

Impacts on netting calculations

Kanban logic impacts calculations performed by the netting analytic by setting a maximum limit on the number of kanban bins that can be in process at any one time. This limit is then enforced by controlling when planned orders can start as well as the size of those planned orders.

In order to enforce kanban limits, the netting analytic keeps track of the total net supply for a part. Within in a given period, total net supply for a part is calculated by summing the in-process (bins being re-filled) and projected inventory (parts in bins) where:

- **Projected inventory** = projected inventory from previous period + supply receipts - demands
- **In process** = in process from previous period + supply starts - supply receipts

Once total net supply is known, planned orders can then only be created for a given period if total net supply falls at or below the kanban replenishment trigger.

The kanban replenishment trigger is calculated as the kanban limit minus the current approved bin quantity, where the kanban limit is calculated as the product of the current approved bin quantity (**SourceKanban.BinQty**) and the current approved number of bins **(SourceKanban.Bins)**. For example, if the current approved bin quantity is set to 50 and the current approved number of bins is set to 5, then the kanban limit is calculated at 250 and the kanban replenishment trigger is calculated at 200.

In other words, the kanban replenishment trigger is reached when at least one bin becomes empty. Any planned orders then required (for example, after a demand consumes some existing supply) are made for the current approved bin quantity, and total net supply resulting from these orders is not allowed to exceed the kanban limit.

Note that when the netting analytic creates planned orders, the kanban logic only ensures that the total net supply is at or below the kanban replenishment trigger on the planned order start date. It does not check through subsequent periods in lead time to ensure that the kanban levels are being respected, nor does it attempt to reschedule scheduled receipts to ensure that kanban limits are respected. This means that if there are scheduled receipts due after kanban logic starts to create planned orders, then these receipts might cause total net supply to exceed the kanban limits. Therefore, it is recommended that the planning time fence be set far enough out to include all scheduled receipts so as to ensure they are due before any planned orders can be created. For example, this can be achieved by setting **PartSource.OrderPolicy.PTFRule** to either “LastDueLead” or “LastDueFence”.

Impacts on capable-to-promise calculations

Besides ensuring that netting calculations plan supplies that respect the kanban limits, kanban logic is also applied in capable-to-promise calculations. For example, it is possible that component availability

can delay some supplies, and cause them to be bunched more closely than netting calculations originally planned. When kanban logic is applied, capable-to-promise calculations provide available dates that can be met while still respecting kanban limits.

Similarly to the netting analytic, the capable-to-promise analytic keeps track of the total net supply for a part in order to enforce the kanban limits. Within a given period, CTP total net supply for a part is calculated as the sum of available inventory and in process where:

- **Available inventory** = available inventory from previous period + available receipts - demands
- **In process** = in process from previous period + available starts - available receipts

Once CTP total net supply is known, available starts in CTP are limited not only by the requirement that CTP total net supply cannot exceed the kanban limits, but also by the material start date on supplies.

That is, once a planned order has all the materials it needs, the capable-to-promise analytic attempts to add it to the total net supply. The AvailableStartDate is initialized to the later of MaterialStartDate or the "AvailableStartDateLimit" (as set by the value on **Status.AvailableDateLimit**). If CTP total net supply on this date is at or below the kanban replenishment trigger, then a CTP planned order is created. However, if CTP total net supply on this date is above the kanban replenishment trigger, then the CTP analytic look forward in time until a period is found where CTP total net supply is at or below the kanban replenishment trigger, and creates the CTP planned order on that date. This date is then reported in the CTPPlannedOrder.AvailableStartDate field.

Note: It is recommended that kanban supplies have their **Status.AvailableDateLimit** value set to "CalcStart". This is intended to ensure that, whenever possible, CTP calculations will add supplies to kanban bins in the same sequence that netting calculations planned them and avoid planning them earlier than netting intended.

Limitations on other analytics with kanban

As discussed earlier in this chapter, kanban logic impacts the results generated by the netting and capable-to-promise analytics. Enabling kanban calculations also has implications for a number of other analytics. For example, some may not be compatible with kanban logic or may override kanban logic as discussed in the following table.

Table 22.1: Limitations on other analytics when using kanban logic

Analytic	Limitation
Constraint Analysis	Kanban part sources cannot be subject to constraints. Any constraints on the capacity to produce or purchase a kanban part should already be accounted for in the kanban parameters. Therefore, any constraints associated with a kanban part source through SourceConstraint records are ignored.
Incremental Availability	Incremental availability logic does not apply to kanban parts. This is because incremental availability has the effect of splitting supplies as their component materials become available, and kanban bins are meant to be processed as a whole unit.

Table 22.1: Limitations on other analytics when using kanban logic

Analytic	Limitation
Multi-sourcing	Kanban parts can have only one part source effective on a given date. If a kanban part has more than one part source effective at a given time, the kanban logic will only use one of them (typically, the one with the latest minimum order due date). However, it is possible to phase out one part source and phase another in order to support time-phased kanban parameters. Once a new part source becomes effective, then the previous part source is considered no longer effective, even if its effectivity overlaps with the new part source's effectivity. Also, scheduled receipts can use non-kanban part sources (that is, part sources with **OrderPolicy.MaximumUsage** set to a value other than 'Kanban').
Order Point	Any part that uses order point logic (that is, has **SafetyStockPolicy.SafetyStockRule** set to either OverPointOver, OverPointUnder, or OverPointAverage) cannot use kanban logic. Order point logic has some similarities to kanban logic but is set at a higher level (part as opposed to part source), and therefore will take precedence over the kanban logic if both are set.
Prioritization	Order priorities do not apply to kanban parts. This is because kanban logic in the netting and capable-to-promise analytics require that demands be processed in due date sequence. Therefore, all kanban parts are treated as if **Part.Type.CommitLevel** = "Low" (regardless of what the actual setting on the part is).
Takt Time	Takt time logic is not applicable to kanban part sources. This is because takt time logic affects planned orders when the required quantity exceeds **PartSource.MaximumQty**, but kanban part sources respect the kanban bin quantities rather than the maximum quantity.
Interchangeable Parts	Only the primary part in an interchangeable parts group can use kanban logic. Scheduled receipts for alternate parts in the interchangeable group do not consume bin quantities.
Substitute Parts	Kanban logic is disabled for parts involved in a substitute part relationship.
Co-product and By-product Parts	Using Kanban logic with parts configured in a co-product or by-product relationship will lead to unexpected results. Specifically, any co-product supply does not consume available kanban bins. Supply on the primary product can consume kanban bins, but may lead to invalid results.
Campaign planning	Kanban logic cannot be used on part sources that are enabled for campaign planning (they are mutually exclusive settings in **OrderPolicy.MaximumUsage**).
Feature BOM	Kanban logic is always disabled for configurable end items.

CHAPTER 23: Interchangeable parts

You can specify sets of parts as being interchangeable. When demands are received for an interchangeable part, supply of all parts interchangeable with that part are used to satisfy the demand. If parts are interchangeable, orders for one part can consume supply for the other. For example, if you have two suppliers who manufacture similar engine mounting brackets, you can use those brackets interchangeably in assemblies. Supplies of both brackets are used to satisfy demands for either of the brackets during the engine production.

Interchangeable parts are arranged into groups, consisting of a primary part and one or more alternate parts. The primary part defines the group, and is used for most planning calculations. Supply of every part in the group can be used to satisfy demands for any other part in the group.

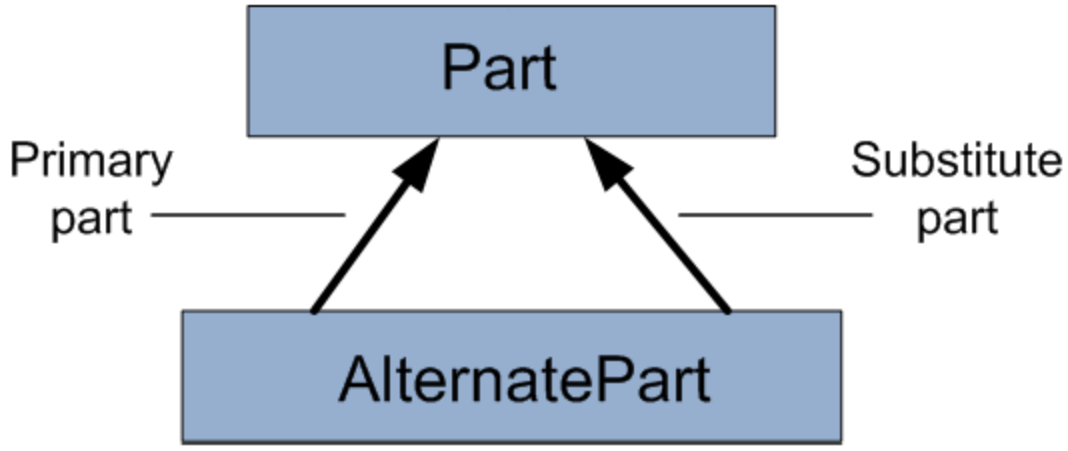
In this chapter, you will learn about:

Setting up interchangeable parts 1545
Analytics with interchangeable parts 1549

Setting up interchangeable parts

Interchangeable parts are defined by creating part groups in the **AlternatePart** table. This table has fields to define the primary part, in the **PrimaryPart** field, and alternate parts, in the **SubstitutePart** field, for the group. Both of these part fields reference records in the Part table, as shown in the following illustration.

Figure 23.1 Tables to define interchangeable parts



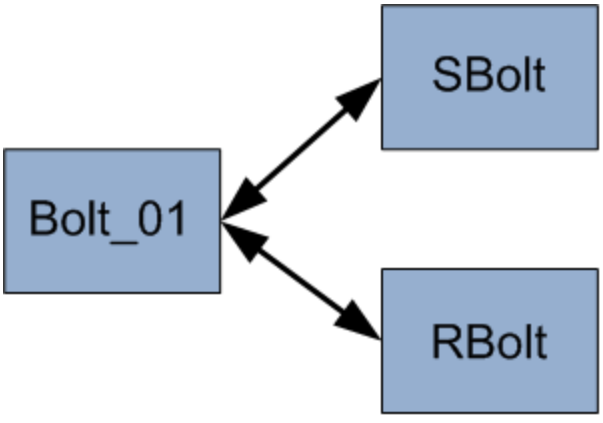
One primary part can have many alternate parts. All planned orders for parts in the group are created for the primary part, and safety stock is replenished only for the primary part. This means that if a planned order must be created to satisfy a demand for a part in the interchangeable group, that planned order is created for the primary part. In addition, the primary part's processing rules, planning calendars, and forecast calendars are used for all calculations for any part in the group of interchangeable parts. However, in order to be used in an interchangeable relationship, the **SubstitutePart** still requires at least one effective **PartSource** record (this can include even those sources whose **OrderPolicy.OrderGenerationRule** is set to "NoOrders").

You can also specify a name for the interchangeable part group.

You can report interchangeable parts by specifying that each part in the interchangeable group reference the same record in the **ReferencePart** table. This is not a requirement for the parts to be considered interchangeable, and is used only for reporting.

The following illustration shows an example of an interchangeable group named SteelBolt. In this example, the SteelBolt group has one primary part, Bolt_01, and two alternate parts, SBolt and RBolt.

Figure 23.2 An interchangeable part group



The SteelBolt interchangeable part group can be defined with the following records in the AlternatePart table. The BaseKey field contains the interchangeable group's name, and is not used in RapidResponse analytics or calculations.

Table 23.1: AlternatePart field values to define an interchangeable group

BaseKey	PrimaryPart	SubstitutePart
SteelBolt	Bolt_01	SBolt
SteelBolt	Bolt_01	RBolt

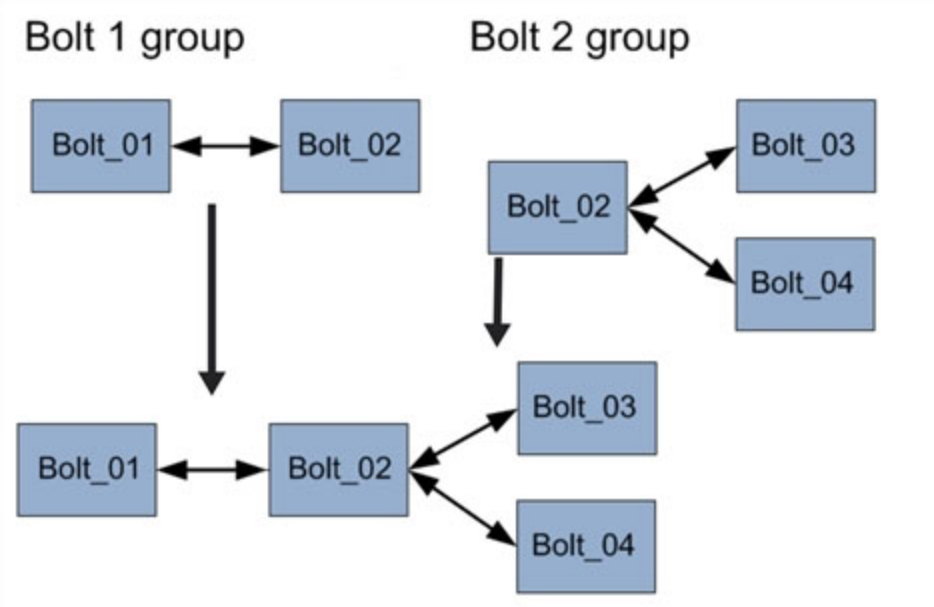
These parts can now be used interchangeably, and orders for one can consume supply of the others. All planned orders created for any of the parts in the group are created for Bolt_01.

Chained interchangeable parts

If a part is a primary part in one interchangeable group and an alternate part in another, the interchangeable groups are considered to be chained. If the groups are chained, the parts in each group are interchangeable with each other. In other words, if two interchangeable groups are chained, they form a single group.

For example, the following illustration shows two chained groups, Bolt 1 and Bolt 2. Part Bolt_02 is an alternate part in the Bolt 1 group, and is the primary part in the Bolt 2 group. These two groups are chained, and are treated as a single group with primary part Bolt_01.

Figure 23.3 Chained interchangeable groups



In this example, Bolt_01 is the primary part for the entire group because it is the only primary part that is not also an alternate part. Bolt_02, Bolt_03, and Bolt_04 are all alternates of Bolt_01. This type of interchangeable group can be defined with the following AlternatePart records.

Table 23.2: AlternatePart field values for chained interchangeable groups

BaseKey	PrimaryPart	SubstitutePart
Bolt 1	Bolt_01	Bolt_02
Bolt 2	Bolt_02	Bolt_03
Bolt 2	Bolt_02	Bolt_04

These parts function the same as other interchangeable groups. Supplies of each part in the group can be used to satisfy demands for any of the others, and any planned orders are created for Bolt_01.

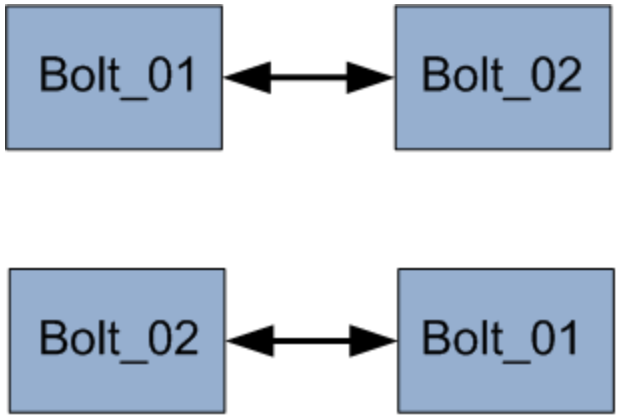
Recursive interchangeable groups

In some cases, two parts might be defined as alternates of each other. This interchangeable group is recursive, and because both parts are alternates for one another, there is no way to determine which of the parts is the primary part of the group. A recursive interchangeable group is considered invalid, and is ignored.

If a group is recursive, the **AlternatePart.IsRecursive** field is set to **'Y'**.

For example, in the following illustration, the Bolt_01 and Bolt_02 parts form an invalid recursive interchangeable group.

Figure 23.4 Recursive interchangeable part groups



These invalid interchangeable groups are defined with the AlternatePart records in the following table.

Table 23.3: AlternatePart field values for recursive interchangeable groups

BaseKey	PrimaryPart	SubstitutePart
SteelBolt	Bolt_01	Bolt_02
OtherBolt	Bolt_02	Bolt_01

When creating interchangeable groups, ensure that no groups are recursive.

Analytics with interchangeable parts

Interchangeable parts affect the following RapidResponse analytics.

Table 23.4: Analytics affected by interchangeable parts

Analytic	Effect of interchangeable parts
Netting	Supplies for any part in the interchangeable group are used to satisfy demands for any parts in the group. For more information, see "Netting basics" on page 1331.
Explosion	When a bill of materials that includes an interchangeable part is exploded to create allocations, the allocations are created for the part specified in the BOM structure. However, supplies of other parts in the interchangeable group can be used to satisfy the allocation demand. For more information about explosion and allocations, see "Explosion" on page 1337.
Planned Orders	Planned orders are created only for the primary part in a group of interchangeable parts, regardless of which part in the group had the demand driving the planned order. If planned orders are spread with Takt Time, the Takt Time settings for the primary part of the interchangeable group are used. For more information about generating planned orders, see "Planned orders" on page 1343. For more information about Takt Time, see "Spreading planned orders with Takt Time" on page 1376.
Days of Supply	Days of supply is calculated only for the primary part in the interchangeable group. The demand for the entire group of interchangeable parts is used in the days of supply calculation. For more information about days of supply, see "Days of Supply calculations" on page 1362.
In-Transit	In-Transit scheduled receipts for interchangeable parts consume scheduled receipts for all parts in the interchangeable group. However, InTransitExact scheduled receipts consume only scheduled receipts for the part specified by the scheduled receipt. For more information, see "In-transit scheduled receipts" on page 1734.
Safety Stock	Safety stock is calculated for the primary part in the interchangeable group only, based on its safety stock policies. The safety stock policies for alternate parts are ignored. For safety stock policies that accumulate demand, the total demand in a period is calculated for the entire interchangeable group. For more information, see "Safety stock" on page 1400.
Forecast Consumption	The forecast consumption rules for the primary part in the group are used for all parts in the group. All forecast and actual orders for the group are processed together as one set of forecasts and one set of actual orders. This means actual orders for any part in the group can consume forecasts for any other part in the group. For more information, see "Forecast consumption" on page 1431.
Forecast Consumption by Customer	Actual orders for parts in the interchangeable group can consume forecast for other parts in the group, as long as the parts have the same customer pool. In addition, if the primary part in the group is configured to use the common pool, actual orders for all parts in the group can move to the common pool for consumption. For more information, see "Forecast consumption by pool/customer" on page 1447.
Available-to-Promise	Available-to-Promise calculations include all parts in the interchangeable group, but results are shown only for the primary part, not for the alternate parts. For more information, see "Available to promise" on page 1449.
Capable-to-Promise	Demand for one part in the group can be allocated supply for another part in the group. For more information, see "Capable-to-Promise calculations" on page 1453.

Table 23.4: Analytics affected by interchangeable parts

Analytic	Effect of interchangeable parts
Gating Part	The part in the interchangeable group with the latest supply is reported as the gating part. For more information, see "Gating part calculations" on page 1457.
Full-Level Pegging	Full-level pegging results are reported against the primary part in the interchangeable group. For more information, see "Full-level pegging with interchangeable parts" on page 1499.
Kanban Analysis	A kanban replenishment system can be defined only for the primary part in the group. Quantities of alternate parts do not consume bin quantities of the primary part. Planned orders to replenish bins are created for the primary part in the group. For more information, see "Impacts on netting calculations" on page 1542.
Bill of Material	Allocations for a part in a bill of material can be satisfied using supplies of interchangeable parts. For more information, see "Explosion" on page 1337.
Inventory Analysis	Inventory analysis is performed only for the primary part in the group. The results of the inventory analysis calculations are stored in the **InventoryAnalysis** table (see "InventoryAnalysis table" on page 1078) and **InventoryCTPAnalysis** table (see "InventoryCTPAnalysis table" on page 1081).
Cost of Goods Sold	The cost of all interchangeable parts are used in the cost of goods sold calculation. For more information, see "Cost of goods sold calculations" on page 1503.
Cost Roll-up	Rolled-up costs are calculated only for the primary part in the group. Costs for alternate parts in the group are not calculated. For more information, see "Cost roll-up calculations" on page 1511.

Interchangeable parts can also affect some analytic modifiers.

Table 23.5: Analytic modifiers affected by interchangeable parts

Analytic modifier	Effect of interchangeable parts
Constraint Analysis	Interchangeable parts are all considered to be members of the same constraint family. For more information, see "Families and multi-level constraints" on page 1633.
Model-Unit Effectivity and Inventory Pooling	Supplies of interchangeable parts are used to satisfy demands only if they have the same model, unit, and pool as the demand. Any parts with different values cannot be used to satisfy the demand. Planned orders created to replenish stock are created for the primary part in the interchangeable group, and have the demand's model, unit, and pool values. For more information, see "Pool, model and unit netting" on page 1748.

CHAPTER 24: Global part substitution

In some cases, when there is not enough supply of a part to satisfy its demand(s), there might be other similar parts whose supply could be used to satisfy demand for the original part. In RapidResponse, this can be configured by defining substitute part relationships in the AlternatePart table. For a given part, one or more other parts can be set as substitutes, and a preference can be specified amongst multiple substitutes to indicate which should be used first. If necessary, a given substitute relationship can be set as effective within a specified time frame only. In addition, different part substitutes, or preferences amongst substitutes, can be configured on a per customer basis.

In this chapter, you will learn about:

Setting up part substitution **1551**
Part substitution calculations **1562**
Limitations on other analytics **1562**

Notes:

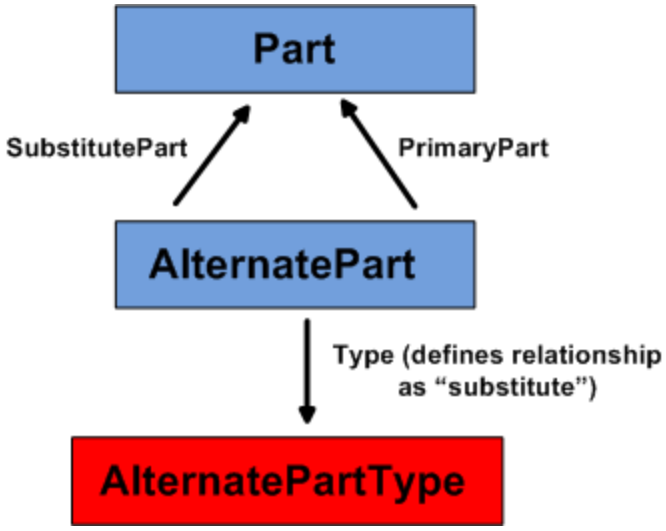
- Substitute part relationships act in one direction only. That is, while a substitute part can be used in place of the original part, supply of the original part cannot be used to satisfy demand for a substitute. However, bidirectional part relationships can be configured in RapidResponse by defining interchangeable parts. For more information, see "Interchangeable parts" on page 1545.
- Substitute parts can also be configured just within the context of a specific assembly. That is, component parts may be configured as substitutes of one another when being used to satisfy demand from an assembly. For more information, see "BOM level part substitution" on page 1565.
- BOM charts in RapidResponse provide you with a view of the bill of material. You can use a BOM chart to view components and their global part substitutions. The BOM Analysis workbook includes a BOM chart in the BOM Visualization worksheet. For more information, see the *RapidResponse User Guide*.

Setting up part substitution

Part substitution is defined using the **AlternatePart** table. As shown in the following illustration, this table contains two references to the **Part** table. One reference identifies the primary part (the part requiring a substitute), and the other reference identifies the substitute part (the part that can be used in place of the primary part). Because the **AlternatePart** table is used to define interchangeable parts as well as

substitute parts, it also contains a reference to the **AlternatePartType** table. This table is used to indicate whether a given pair of parts have an interchangeable or substitute relationship, and also contains other settings affecting part substitution calculations.

Figure 24.1 AlternatePart table



If a given part has more than one substitute, a separate record should be added to the **AlternatePart** table for each primary part and substitute part combination. Using the **Sequence** field on the **AlternatePart** table, a preference can be specified for which of multiple possible substitutes should be used first (with lower values indicating a more preferred substitute). For example, there might be a preference to use a lower-cost substitute as opposed to a higher-quality substitute. The following table shows some sample values used to indicate a preference amongst substitutes for the part ChipA.

Table 24.1: Defining preference amongst substitutes in AlternatePart table

PrimaryPart	SubstitutePart	Type.ProcessingRule	Sequence
ChipA	ChipX	Substitute	0
ChipA	ChipY	Substitute	1
ChipA	ChipZ	Substitute	2

Note: For complete reference information, see "AlternatePart table" on page 190.

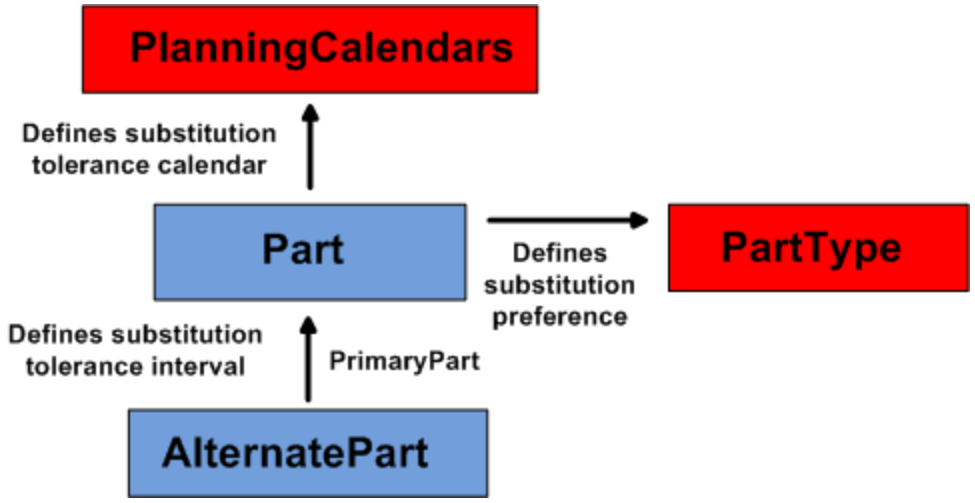
Specify date effectivity

Substitute part relationships can be specified as effective within a given time period using the **EffectiveInDate** and **EffectiveOutDate** fields on the **AlternatePart** table. Together with the setting in **Type.EffectivityRule**, they define the first and last dates on which the **AlternatePart** record is considered effective.

Define when part substitutes are used

When satisfying demand for the primary part configured in a substitute part relationship, several settings determine the types of supply that are preferred as well as whether supply of the primary part or supply of its substitute(s) are preferred. As shown in the following illustration, these settings are primarily defined from the **Part** table record for the primary part in a part substitution relationship.

Figure 24.2 Part-related controls affecting substitution calculations



The **PlanningCalendars** table reference defines a **SubstitutionToleranceCalendar** to be used in part substitution calculations. Together with the value in the **Part.SubstitutionTolerance** field, it defines the part's tolerance interval. A tolerance interval is a period of time added to the primary part's demand due date that indicates how far out to search for input supply of one part before looking for input supply of other parts. For example, if the **SubstitutionToleranceCalendar** is set to "Week" and **SubstitutionTolerance** is set to "4", then (by default) the primary part will look for its own existing input supply for four weeks before looking for substitute supply. Note also that if **SubstitutionTolerance** is "0", the part will only consider input supply for a given part that is actually on time to meet the demand before looking for input supply from other parts.

The **PartType** table reference includes the **SubstitutionPreference** control setting for specifying a preference between different types of supply when satisfying demand from the primary part. This field has the following three options:

- **OnHand**—Demand is satisfied with on-hand inventory where possible.
- **Primary**—Demand is satisfied with on-time supply from primary or substitute part having the lowest sequence value where possible (as described below).
- **ScheduledSupply**—Demand is satisfied with existing on-hand or scheduled supply, where possible, before generating new planned orders.

Within each of the three options outlined above, the use of either the primary part or substitute part's input supply is further influenced by the combination of the **Part.PrimarySubstitutionSequence** value on the primary part and the **AlternatePart.Sequence** value on each record defining one of its substitute parts. Together, these fields set the order or preference in which input supply is consumed between a primary part and its substitutes with parts that have lower sequence values being consumed first.

For example, if a substitute part represents a higher grade version of the primary part, then the substitute's **Sequence** value should be set higher than the primary's **PrimarySubstitutionSequence** value. In this case, existing supplies from the primary part would be used first, with supply of the more expensive substitute used only when necessary. Conversely, if a substitute part represents an older revision of the primary part, then the substitute's **Sequence** value should be set lower than the primary's **PrimarySubstitutionSequence** value. In this case, existing supply of the older revisions would be used up before moving to supply of the newer revision. Note also that new planned orders are never generated on substitute parts having a **Sequence** value lower than the **PrimarySubstitutionSequence** value defined on the primary part.

The following table outlines how different settings in the **SubstitutionPreference** determine primary and substitute supply usage based on the applicable **Sequence** and **PrimarySubstitutionSequence** values.

Table 24.2: SubstitutionPreference and SubstitutionSequence rules

Substitution Preference	Supply Usage
OnHand	If the **Part.PrimarySubstitutionSequence** field value (primary part) is less than or equal to the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the PrimaryPart. 2. On-hand inventory of the SubstitutePart. 3. Scheduled receipt for the PrimaryPart within tolerance interval. 4. Scheduled receipt for the SubstitutePart within tolerance interval. 5. Excess from on-time existing planned order of the PrimaryPart. 6. Excess from on-time existing planned order of the SubstitutePart. 7. Earliest scheduled receipt, from either primary or substitute, outside of tolerance interval (if available before the earliest planned order). 8. Create on-time planned order on the PrimaryPart. 9. Create on-time planned order on the SubstitutePart. 10. Excess from least late existing planned order on either the PrimaryPart or SubstitutePart. 11. Create least late planned order on either the PrimaryPart or SubstitutePart. Otherwise, if the **Part.PrimarySubstitutionSequence** field value (primary part) is greater than the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the SubstitutePart. 2. On-hand inventory of the PrimaryPart. 3. Scheduled receipt for the SubstitutePart within tolerance interval. 4. Scheduled receipt for the PrimaryPart within tolerance interval. 5. Excess from on-time existing planned order of the SubstitutePart. 6. Excess from on-time existing planned order of the PrimaryPart. 7. Earliest scheduled receipt, from either substitute or primary, outside of tolerance interval (if available before the earliest planned order). 8. Create on-time planned order on the PrimaryPart. 9. Excess from least late existing planned order on either the PrimaryPart or SubstitutePart. 10. Create late planned order on the PrimaryPart.

Table 24.2: SubstitutionPreference and SubstitutionSequence rules

Substitution Preference	Supply Usage
Primary	If the **Part.PrimarySubstitutionSequence** field value (primary part) is less than or equal to the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the PrimaryPart. 2. Scheduled receipt for the PrimaryPart within tolerance interval. 3. Excess from on-time existing planned order of the PrimaryPart. 4. Create on-time planned order on the PrimaryPart. 5. On-hand inventory of the SubstitutePart. 6. Scheduled receipt for the SubstitutePart within tolerance interval. 7. Excess from on-time existing planned order of the SubstitutePart. 8. Earliest scheduled receipt for either the PrimaryPart or SubstitutePart outside of tolerance interval, but available before the earliest planned order on either. 9. Create on-time planned order on the SubstitutePart. 10. Excess from least late existing planned order from either part. 11. Create least late planned order for either the PrimaryPart or SubstitutePart. Otherwise, if the **Part.PrimarySubstitutionSequence** field value (primary part) is greater than the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the SubstitutePart. 2. Scheduled receipt for the SubstitutePart within tolerance interval. 3. Excess from on-time existing planned order of the SubstitutePart. 4. On-hand inventory of the PrimaryPart. 5. Scheduled receipt for the PrimaryPart within tolerance interval. 6. Excess from on-time existing planned order of the PrimaryPart. 7. Earliest scheduled receipt for either the PrimaryPart or SubstitutePart outside of tolerance interval, but available before the earliest planned order. 8. Create on-time planned order on the PrimaryPart. 9. Excess from least late existing planned order from either part. 10. Create late planned order for the PrimaryPart.

Table 24.2: SubstitutionPreference and SubstitutionSequence rules

Substitution Preference	Supply Usage
ScheduledSupply	If the **Part.PrimarySubstitutionSequence** field value (primary part) is less than or equal to the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the PrimaryPart. 2. Scheduled receipt for the PrimaryPart within tolerance interval. 3. On-hand inventory of the SubstitutePart. 4. Scheduled receipt for the SubstitutePart within tolerance interval. 5. Excess from on-time existing planned order of the PrimaryPart. 6. Excess from on-time existing planned order of the SubstitutePart. 7. Earliest scheduled receipt, from either primary or substitute, outside of tolerance interval (if available before the earliest planned order). 8. Create on-time planned order on the PrimaryPart. 9. Create on-time planned order on the SubstitutePart. 10. Excess from least late existing planned order on either the PrimaryPart or SubstitutePart. 11. Create least late planned order on either the PrimaryPart or SubstitutePart. Otherwise, if the **Part.PrimarySubstitutionSequence** field value (primary part) is greater than the **AlternatePart.Sequence** field value (substitute part), then supplies are used in the following order: 1. On-hand inventory of the SubstitutePart. 2. Scheduled receipt for the SubstitutePart within tolerance interval. 3. On-hand inventory of the PrimaryPart. 4. Scheduled receipt for the PrimaryPart within tolerance interval. 5. Excess from on-time existing planned order of the SubstitutePart. 6. Excess from on-time existing planned order of the PrimaryPart. 7. Earliest scheduled receipt, from either substitute or primary, outside of tolerance interval (if available before the earliest planned order). 8. Create on-time planned order on the PrimaryPart. 9. Excess from least late existing planned order on either the PrimaryPart or SubstitutePart. 10. Create late planned order on the PrimaryPart.

Notes:

- In standard part substitution logic, when RapidResponse needs to create planned orders to try and provide on time supply for the primary part, it considers constraint availability and planning time fence only on the primary part and its substitute parts themselves. It does not consider component availability under those parts as part of the criteria for determining whether planned orders can be sourced on time (that is, unlimited component availability is assumed at all levels under the primary and substitute parts). In some scenarios, this can lead to less desirable planning results than might be possible if component availability was considered during planned order generation. To include consideration of component availability when choosing between part substitutes on which to place a planned order, multi-level search logic can be enabled as discussed in "Multi-level search logic" on page 1863.
- To use up all current supply on the primary part, but only plan new supplies on the substitute parts, you can set each part source for the primary part to have a **PartSource.OrderPolicy.OrderGenerationRule** setting of “No Order”.

Example: SubstitutionPreference configuration

Assume a primary part "APrime" has two substitutes "XSub" and "YSub" with sequence values as shown in the following table.

Table 24.3: AlternatePart values

PrimaryPart.Name	PrimaryPart.PrimarySubstitutionSequence	AlternatePart.Name	Sequence
APrime	1	XSub	0
APrime	1	YSub	2

If using the **SubstitutionPreference** option of "OnHand", the supply preference amongst this set of substitutable parts when satisfying a demand for primary part "APrime" would be as follows

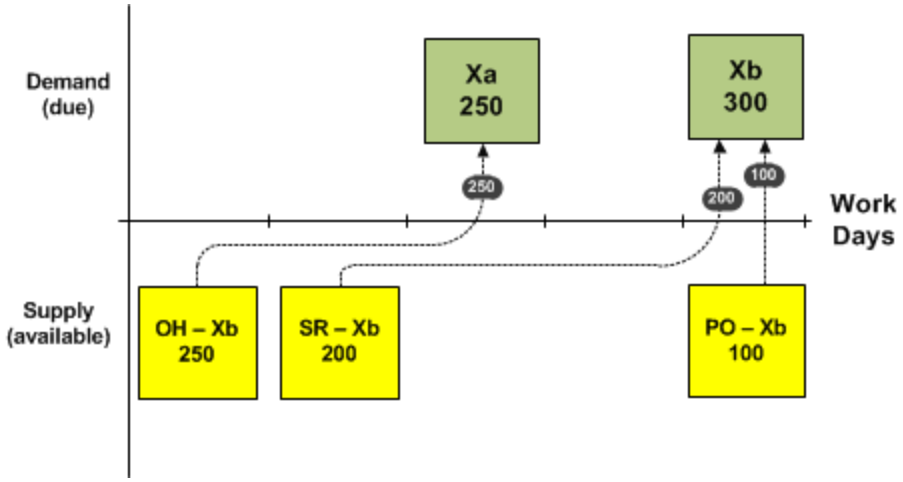
1. On-hand inventory of XSub.
2. On-hand inventory of APrime.
3. On-hand inventory of YSub.
4. Scheduled receipt of XSub (within tolerance).
5. Scheduled receipt of APrime (within tolerance).
6. Scheduled receipt of YSub (within tolerance).
7. Excess from existing on-time planned order for XSub.
8. Excess from existing on-time planned order for APrime.
9. Excess from existing on-time planned order for YSub.
10. Earliest scheduled receipt from either XSub, APrime, or YSub (outside of tolerance, but before the earliest planned order)
11. Create on-time planned order for APrime.
12. Create on-time planned order for YSub.
13. Excess from least late planned order on either XSub, APrime, or YSub.
14. Create least late planned order on APrime or YSub.

Limit substitute usage to excess supply

Global part substitution calculations can be configured so that only excess current supply from a substitute part is made available to satisfy demands from other parts. The specific handling of excess supply for a substitute part is set by the **PartType.ExcessRule** option it references.

By default, the **ExcessRule** is set to "Ignore" and supply of substitute parts are generally used to satisfy the demands that require them first. This is regardless of whether the demands are for the part itself or another part for which it is substitutable. For example, assume part "Xb" is a substitute for primary part "Xa" and each has a lead time of 4 days. Further assume the set of demands and input supplies shown in the following figure (with all demands assumed to be at the same priority level). In this case, there is no input supply available to satisfy the earlier demand of 250 units for "Xa" and a new planned order would make this demand late. Thus, the demand for "Xa" is satisfied by the available on hand supply from "Xb". The later demand for "Xb" is then allocated the existing scheduled receipt and a planned order for 100 units is required to satisfy the remainder of this demand. Note that all demands are available on time.

Figure 24.3 Part substitution with ExcessRule set to "Ignore"



In other cases, it might be necessary to protect or reserve existing supply of substitute parts that lack alternative sources of supply. Similarly, if an older substitute part is being phased out it might be desirable to reserve existing supplies for its own later demands in order to prevent unnecessary planned orders from being generated. In these cases, the **ExcessRule** can be set to either "GlobalFence" or "GlobalFenceBeforePriority".

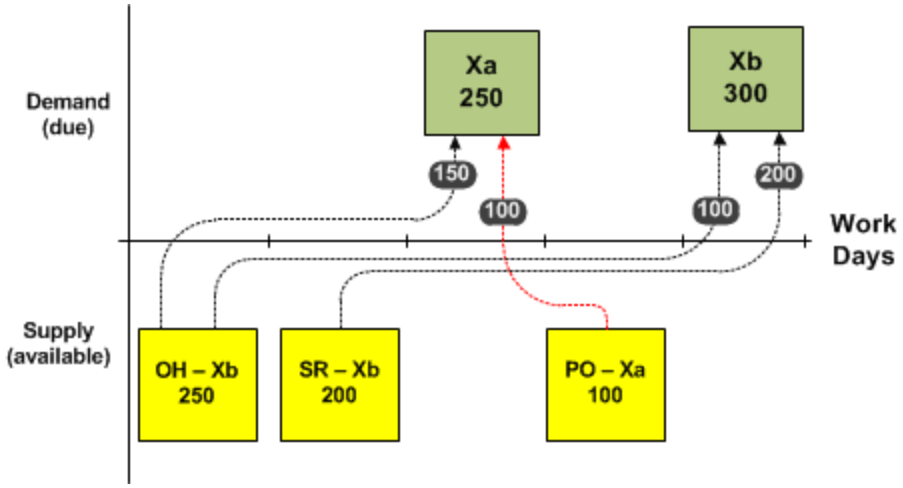
When a substitute part uses the "GlobalFence" option, all current input supplies are reserved and first used to satisfy all its own demands, at a given priority level, before any remainder or excess supply is then made available to satisfy demands, of that same priority level, from other parts for which it is a substitute. Alternatively, when a substitute part uses the "GlobalFenceBeforePriority" option, its current input supplies are reserved and first used to satisfy any of its own demands, regardless of priority level, before any remainder or excess supply is then made available to satisfy demands from other parts for which it is a substitute. For example, this setting might be applied in situations where a substitute part has it own demands, but represents an older revision of the primary part and for which creating new supply is not desirable.

Also, with either of the "GlobalFence" or "GlobalFenceBeforePriority" settings, planned orders can be generated on substitute parts to satisfy their own demands but not to satisfy demand originating from other parts (however, other parts can potentially use excess resulting from planned orders on the substitute part). Additionally, this excess logic can be confined to a defined excess window stating from **RunDate** and extending out the number of **PlanningCalendars.TimeUnits** specified in the **Part.ExcessFence** field. However, if only excess supply should ever be made available for substitution, the part's **ExcessFence** should be set to a negative value (such as, -1).

GlobalFence example

Assume the same set of input supplies and demands as shown in the preceding figure, but with part "Xb" having **Type.ExcessRule** set to "GlobalFence" and **ExcessFence** set to -1. This results in supply allocations as seen in the following figure. Note that because substitute part "Xb" has demand for 300 and current supply of 450, this means only 150 units are deemed to be excess and therefore eligible to be used by the demand for part "Xa". The remaining 100 units of the demand for primary part "Xa" must be satisfied by a planned order which makes the demand late. However, the objective of not generating unnecessary planned orders on part "Xb" would be achieved.

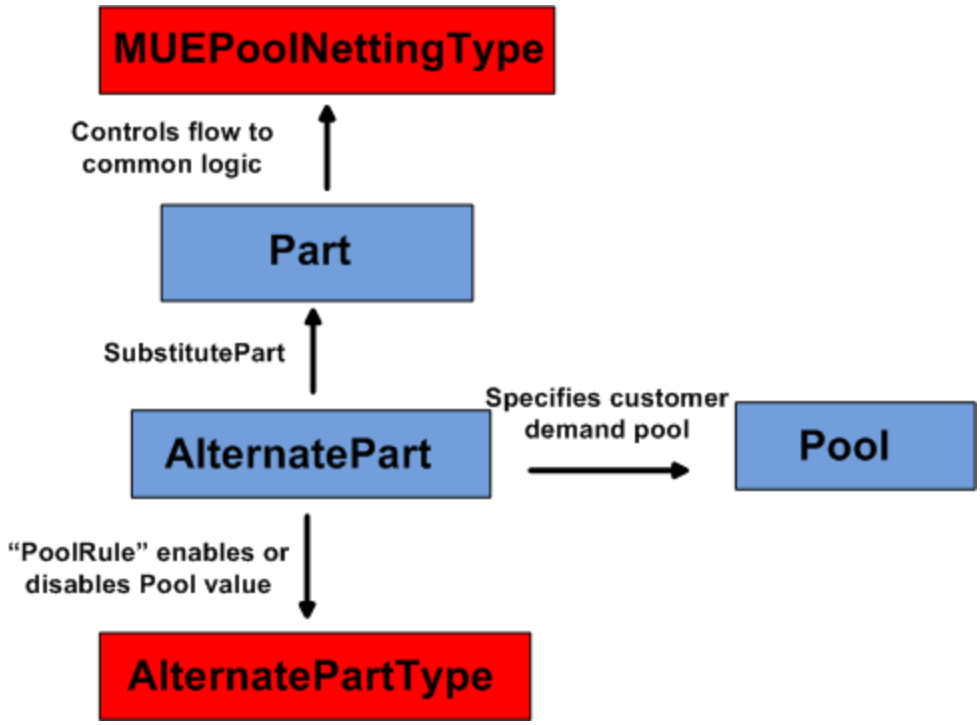
Figure 24.4 Part substitution with ExcessRule set to "GlobalFence"



Enable customer-specific part substitution

In some cases, individual customers may have their own limited approved list of parts that they're willing to accept as substitutes. And different customers may have their own preference amongst the substitutes they're willing to accept. In these cases, customer-specific part substitution can be defined using Inventory Pooling in RapidResponse. As shown in the following illustration, the **AlternatePart** table contains a reference to the Pool table which can be used to model a given customer's demands.

Figure 24.5 Pool table enables customer-specific substitutes



To enable customer-specific substitution, a given **AlternatePart** record needs a valid reference to a **Pool** table value (representing the customer's demand pool) and to have its **Type.PoolRule** value set to "Use". Then in cases where there the customer has demand for a given primary part, RapidResponse will only consider those substitutes that are defined for the customer. For example, the following table shows sample **AlternatePart** records for where different substitutes are allowed for the ChipA part depending on the customer who has the demand. In this example, the customer CircuitTown is willing to accept substitute supply of either part ChipX or ChipY (with a preference for ChipX), customer BestShop is only willing to accept substitute supply of part ChipZ, and the customer BetterBuy is willing to accept substitute supply from either part ChipZ or ChipY (with a preference for ChipZ). A common pool is also defined to manage supply since a single part can be eligible to supply multiple customers. Note also in this example that for all customers, supply of the primary part ChipA is always preferred to any of the substitute parts.

Table 24.4: Defining customer (pool) specific substitute preferences

Pool	Primary Part	Substitute Part	Type. Processing Rule	Type.Pool Rule	Sequence	Primary Part. Primary Substitution Sequence
CircuitTown	ChipA	ChipX	Substitute	Use	0	-1
CircuitTown	ChipA	ChipY	Substitute	Use	1	-1
BestShop	ChipA	ChipZ	Substitute	Use	0	-1
BetterBuy	ChipA	ChipZ	Substitute	Use	0	-1
BetterBuy	ChipA	ChipY	Substitute	Use	1	-1
Common	ChipA	ChipX	Substitute	Use	0	-1
Common	ChipA	ChipY	Substitute	Use	1	-1
Common	ChipA	ChipZ	Substitute	Use	2	-1

In cases where a common pool is configured, the flow-to-common logic in RapidResponse determines if part substitution occurs only in the demand pool or in the common pool as well. If the flow-to common logic is enabled for a part (that is, **Part.MUEPoolNettingType.PoolFlowToCommon** is set to "NetOriginal" or "NetCommon"), then part substitution can occur in the demand pool and the common pool. However, if the flow-to-common logic is not enabled for a part, then part substitution occurs only in the demand pool.

For example, suppose flow-to-common logic is turned off for parts ChipX and ChipY. When requiring substitute supply for part ChipA and customer CircuitTown, RapidResponse would first try to find supply for ChipX from the CircuitTown pool, and then look for supply of ChipY in the CircuitTown pool.

However, if flow-to-common logic was turned on for parts ChipX and ChipY, RapidResponse would first try to find supply for ChipX from the CircuitTown pool, next look for supply of ChipX in the Common pool, then look for supply of ChipY in the CircuitTown pool, and finally look for supply of ChipY in the Common pool

Notes:

- When the setting is "NetOriginal", then once all supply in the common pool has been used, new supply is planned for the customer-specific pool that has the demand. However, if the setting is "NetCommon", then once all supply in the common pool has been used, new

supply is planned for the common pool.

- For more information about inventory pooling, see "Inventory Pooling calculations" on page 1745.

Part substitution calculations

The results of part substitution calculations are reported in the **SubstituteSupply** and **CTPSubstituteSupply** tables. SubstituteSupply reports Netting calculations where supply of a substitute part was used to satisfy demand for a primary part. For example, the name of the part requiring supply from one its substitutes, and quantity and due date of the order. CTPSubstituteSupply reports Capable-To-Promise calculations where supply of a substitute part was used to satisfy demand for a primary part. For example, the date when the substitute supply should be available.

As well, a "SubstituteSupply" supply type is reported in many calculated tables indicating that a part substitution was the source of the supply for the primary part, and a "SubstituteAllocation" demand type is reported in many calculated tables indicating that a part substitution was the source of dependent demand on the substitute part.

Note: For more information, see "SubstituteSupply table" on page 1200 and "CTPSubstituteSupply table" on page 1003.

Limitations on other analytics

When using part substitution, certain other analytic calculations may be disabled, or not work as expected, for the parts configured in a substitute part relationship. The following table outlines calculations impacted by part substitution.

Table 24.5: Part substitution limitations on other analytics

Analytic / Calculation	Impact of Part Substitution
Safety Stock	Safety stock requirements on a primary part can be satisfied by inventory or scheduled supply of its substitute parts. However, if planned orders are required to satisfy safety stock requirements they can only be created on the primary part (not on any of its substitutes).
Dynamic safety stock	If parts involved in a global substitute relationship also use dynamic safety stock logic, then the generated safety stock levels might not be as expected. Dynamic safety stock levels are calculated based on tdemands placed on parts before any substitution decisions have been made. This can result in higher than expected safety stock levels on the primary part and/or lower than expected safety stock levels on its substitutes. Therefore it is recommended to avoid dynamic safety stock settings on parts involved in global part substitution (that is, such parts should typically not use **SafetyStockQuantityRule** settings of "FractionOfDemand", "PercentOfDemand", "RangeOfCoverage", "RangeOfCoverageLeadTime", "TimePhasedRangeOfCoverage", "TimePhasedSafetyDaysMaximum", or "TimePhasedSafetyDaysTotal").
Days Of Supply	When using supply to satisfy Days of Supply requirements on a primary part, any planned orders required can only be created on the primary part.
Kanban	Kanban calculations are disabled for any part involved in a substitute part relationship.

Table 24.5: Part substitution limitations on other analytics

Analytic / Calculation	Impact of Part Substitution
Forecast Consumption	The dependent demand (SubstituteAllocation) created on a substitute part does not consume forecast.
MPS Configuration	MPSConfig parts are not eligible for part substitution.
Feature BOM	Global part substitution cannot be performed on any of the immediate components of a configurable end-item.
Phantom Parts	Global part substitution logic might lead to unpredictable results if used on phantom parts.
Multi-level search logic	Global part substitution is supported for parts that have multi-level search logic enabled, but the ability to define an excess window for the substitution is not supported. That is, if multi-level search logic is enabled then the **PartType.ExcessRule** settings of "GlobalFence" and "GlobalFenceBeforePriority" should be avoided.
BOM level part substitution	Global part substitution is ignored when substitutable parts are also configured as bom-level substitutes. If two parts are configured as substitutable components within an assembly and also configured as global substitutable parts, then the BOM level part substitution settings will override the global part substitution settings

CHAPTER 25: BOM level part substitution

A component used in the production of an assembly might not always have sufficient supply available to satisfy dependent demand exploded down from the assembly. However, there could be available supply on other similar components which could be used instead. This allows the demand to be satisfied, while also reducing the need to create new planned orders and using up existing inventory of the other components.

In RapidResponse, an assembly can be configured to have substitutable component parts that can be used in place of one another. This involves designating one component as the primary component (the component which the assembly will first look to for supply), and designating all other components as substitutes which can potentially be used when there is insufficient supply of the primary component.

A preference can be specified as to the order in which substitute components should be used, and other settings are available to control calculations such as how planned order generation should be spread amongst substitute components.

RapidResponse also allows for an entire group of components used in the production of a a given assembly to be substituted with another group of components. And if existing allocations are being imported into the Allocation table, RapidResponse allows for substitutable allocations to be defined under a given scheduled receipt.

In this chapter, you'll learn about:

Substitute BOM configuration **1566**
Substituting groups of components **1574**
Substitutable allocations **1576**
Limitations on other analytics **1578**

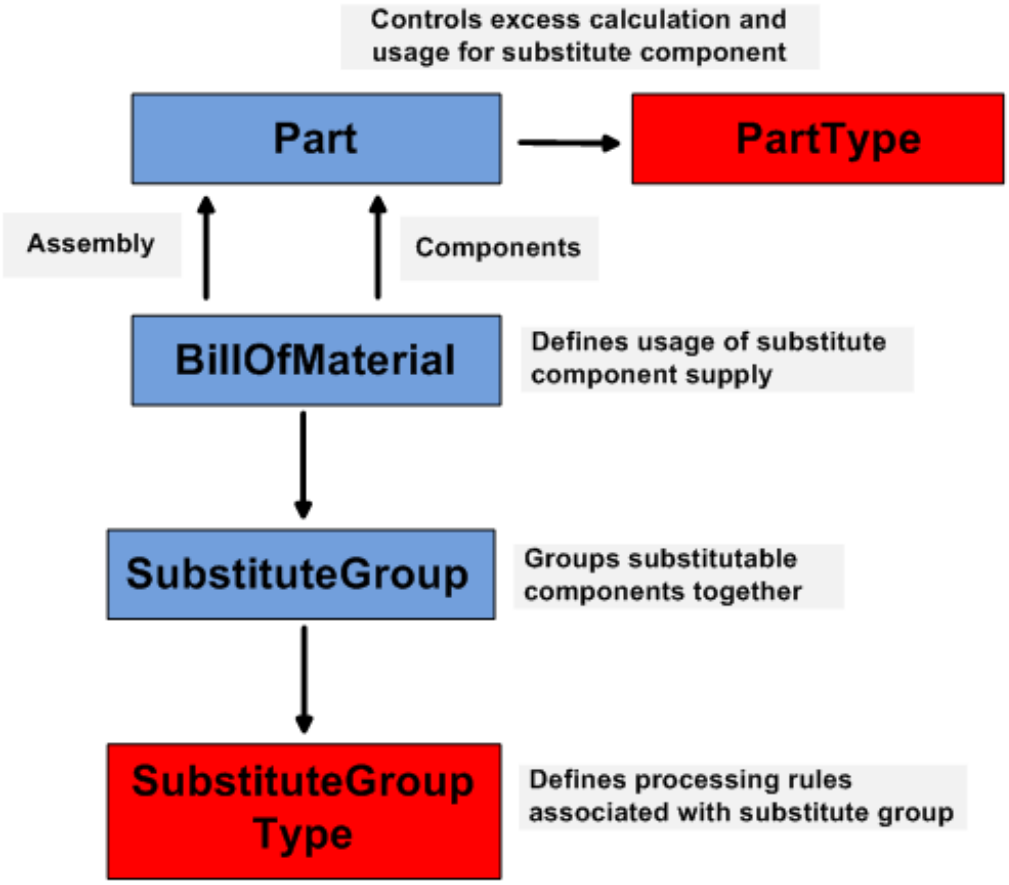
Notes:

- This chapter discusses configuring BOM level part substitution calculations which are applicable to component parts within a given assembly. RapidResponse also allows for parts to be configured as substitutes for one another on a global basis (that is, outside of a bill of material context). For more information, see "Global part substitution" on page 1551.
- BOM charts in RapidResponse provide you with a view of the bill of material. You can use a BOM chart to view components and their BOM level part substitutions. The BOM Analysis workbook includes a BOM chart in the BOM Visualization worksheet. For more information, see the *RapidResponse User Guide*.

Substitute BOM configuration

As shown in the following illustration, BOM level part substitution is configured through records in the BillOfMaterial table and their references.

Figure 25.1 Main tables used in BOM substitution



The section introduces the basic concepts involved in BOM level part substitution, and discusses configuring RapidResponse to substitute one or more of an assembly's individual components with other similar components. It contains discussion on the areas outlined in the following table:

Table 25.1: Tasks involved in setting up BOM level part substitution

Task	Description
Define substitutable components	This involves ensuring appropriate records are added to the **SubstituteGroup** and **SubstituteGroupType** tables to enable BOM level part substitution, and then creating the **BillOfMaterial** records that define a given assembly's substitutable components. For more information, see "Define substitutable components" on page 1567.
Specify usage of substitute supply	This involves specifying the control settings that determine how current supply on substitutable components get used in BOM level part substitution. For example, RapidResponse can be configured so that only a component's excess supply is available for use as a substitute, or to set how different substitutes may be mixed together on a single supply for the assembly. For more information, see "Specify usage of substitute supply" on page 1569.
Define planned order generation	This involves specifying the ratio in which planned orders should be generated across an assembly's substitutable components (in cases, where there is insufficient current supply available). For example, planned orders to satisfy a given demand can be split proportionally between substitutable components based on specified targets or only generated on the substitutable component that is furthest from its target. Alternatively, RapidResponse might be configured so that planned orders are only created on the primary component in a group. For more information, see "Define planned order generation" on page 1571.
Specify date effectivity	If substitutable components or their properties are to change over time, this can be modeled using BOM effectivity. For more information, see "Specify date effectivity" on page 1574.

Note: For information about substituting an entire group of components with another group of components, see "Substituting groups of components" on page 1574.

Define substitutable components

Substitutable components within an assembly are grouped and defined using the **SubstituteGroup**, **SubstituteGroupType** and **BillOfMaterial** tables.

Create SubstituteGroup and SubstituteGroupType records

BOM level part substitution requires records to be added to the **SubstituteGroup** table. The table is referenced by the **BillOfMaterial** table and is used to define groups of substitutable components together within an assembly. It is also strongly recommended to avoid using the same **SubstituteGroup** record for groups under multiple assemblies (there should be a record specific to each group of substitutable components under a given assembly).

Records must also be added to the **SubstituteGroupType** table. This control table is referenced by each **SubstituteGroup** record and define processing rules associated with BOM level part substitution as discussed later in this section.

Note: For reference information about these tables, see "SubstituteGroup table" on page 611 and "SubstituteGroupType table" on page 901.

Specify an assembly's substitutable components

To define substitutable components within an assembly, **BillOfMaterial**records are required to associate the assembly with the primary component as well as each of the substitute components that can be used in place of the primary component. This includes providing a value in the **SubstitutionSequence** field to define the order in which the assembly will look for current supply (onhand and scheduled receipts) within a group of substitutable components.

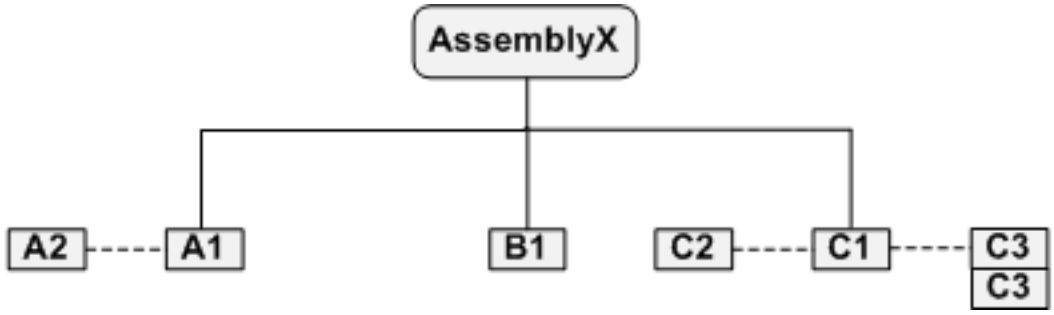
The primary component must be assigned a value of 0 in this field, and substitute components with lower values in this field will be used before substitutes with higher values. Additionally note that the BOM record defining the primary component must be effective in order to ensure the expected bom substitution behavior.

As well, each record that associates the assembly part with one of a group of substitutable components must point to the same record in the **SubstituteGroup** table (and there must be one and only one primary BOM inside each substitute group). This is necessary to ensure RapidResponse treats those components as substitutable. Additionally, each of an assembly's substitutable components under a given **SubstituteGroup** should have a unique **SubstitutionSequence** value within that group (duplicate **SubstitutionSequence** values in a group of substitute components can result in excess dependent demand being generated).

Example

Assume a simple bill of material structure as shown in the following illustration where AssemblyX requires 1 unit each of primary components A1, B1, and C1. In this example, A1 has one substitute component (A2), B1 has no substitute components, and C1 has two substitute components (C2 which is the preferred substitute, and C3 of which two units are required as substitutes for one unit of C1)

Figure 25.2 BOM structure for Assembly with substitutable components



The following table shows values that might be included in BillOfMaterial records to model the BOM structure shown in the previous illustration. This assumes the **SubstituteGroup** table includes a value, ASub, to group component A1 and its substitutes, and another value, CSub, to group component C1 and its substitutes.

Table 25.2: Sample BillOfMaterial records for Assembly with substitutable components

Assembly	Component	Substitute Group.Value	Substitution Sequence	Quantity Per
AssemblyX	A1	ASub	0	1
AssemblyX	A2	ASub	1	1
AssemblyX	B1	BSub	0	1
AssemblyX	C1	CSub	0	1
AssemblyX	C2	CSub	1	1
AssemblyX	C3	CSub	2	2

Note: The SubstitutionSequence field determines the preference for supply between substitutable components in a group as long as the **SubstituteGroupType.SupplyPreference** field is set to "Sequence" (which is the default). However, this field also has an "Earliest" setting which can be used to override the SubstitutionSequence value, and instead use the substitute component that has the earliest supply that can be used.

Enable or disable substitute calculations

For each substitute group defined in the **SubstituteGroup**table, BOM level part substitution can be turned on or off for the group through its reference to the **SubstituteGroupType** control table.

When a record in the **SubstituteGroup** table has a **Type.ProcessingRule**value of "Substitute", BOM substitution calculations are turned on for the group. However, when a record in the **SubstituteGroup** table has a **Type.ProcessingRule** value of either "Ignore" or "OptionRatio", BOM substitution calculations are turned off for the group.

Specify usage of substitute supply

Control settings are available that set how current supply on substitute components can be used in BOM level part substitution.

Use only excess supply on substitute components

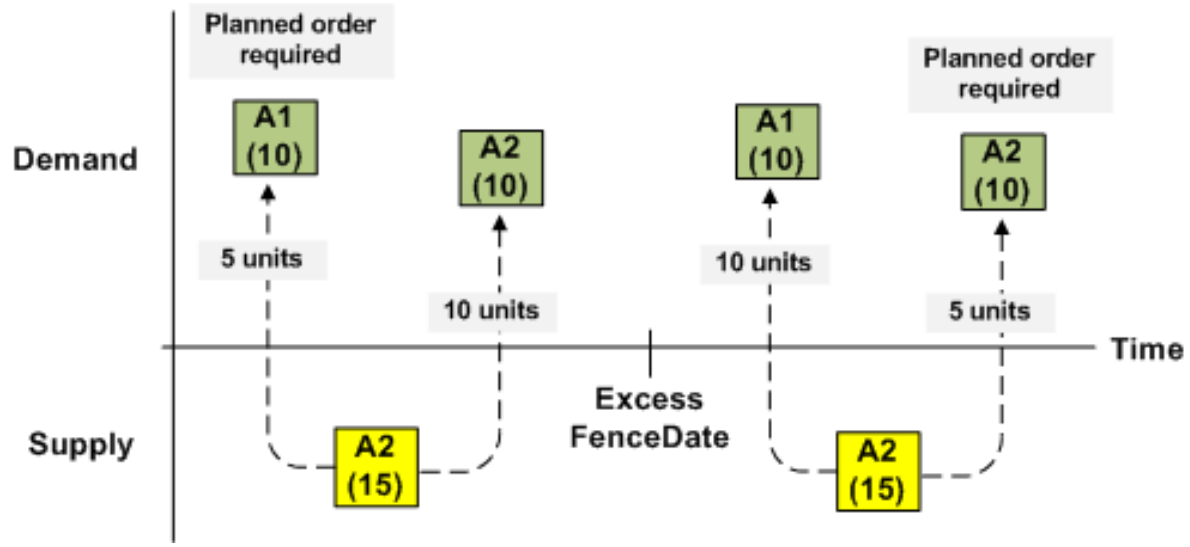
RapidResponse can be configured so that within a defined excess window, current supply (onhand or scheduled receipts) on a substitute component must first be used to satisfy any of its own demand (that does not itself have substitution options). Only the excess that remains is then eligible to be used in BOM substitution calculations. This can prevent supply from being taken away from lower priority or later demands that might not have alternatives available. Outside of the excess window, supply on a substitute component is used by the demand that requires it first (based on priority, due date, and so on) regardless of whether the demand is its own or that of a substitutable component.

For example, assuming the same BOM structure shown in "BOM structure for Assembly with substitutable components" on page 1568, the following illustration shows how supply from substitute component A2 might be split between its own demand and demand on the primary component A1.

Note that within the excess window (that is, before ExcessFenceDate), A2 first fully satisfies its own demand for 10 units before the remainder is made available for use as a substitute on A1. Outside of the excess window, A2 first satisfies the earlier demand on A1 which leaves insufficient supply to satisfy its own later demand. This example assumes the supplies shown are not reschedulable.

The impact of excess window on BOM level part substitution is shown in the following diagram.

Figure 25.3 Impact of excess window on BOM level part substitution



The excess window used in BOM substitution is configured on a part basis, and so can be set differently across components. The **Part.ExcessFence** field is used to specify the number of intervals that are added to the value in **PlanningCalendars.RunDate.FirstDate** to define the end of the excess window. For example, an excess window might be set to 2 weeks or 1 month. In order for a component part's excess window to be used in BOM substitution, the part must reference a value in the **PartType.ExcessRule** field that is set to "Fence". If this field is set to "Ignore", then the part's excess window is not used.

Note: Reschedulable scheduled receipts originally planned outside the excess window, may be moved within the excess window and thereby used in a component part's excess calculation.

Combine substitute components on a single assembly supply order

When using any of a component's current supply (on hand or scheduled receipts) to satisfy dependent demand from the assembly, the **SubstituteGroupType.ComponentRule** field can be used to control how supply from different substitute components in a group can be mixed together on a single assembly order. This field contains the following values:

- **AssemblyUnit**—only one component in the group can be used for each unit of assembly supply in the order. That is, component supply can only be used in increments equal to the component's QuantityPer value.
- **Single**—only one component in the group can be used for each assembly supply order. The component with the lowest Sequence value that can satisfy the entire demand will be used;

otherwise the component with the lowest Sequence value that can satisfy any of the demand is used. If planned orders are required to fully satisfy the demand, they must be created on the same component whose current supply was used.

- **Unrestricted**—there are no restrictions on how substitutable components can be mixed together on a supply order for the assembly. Available supply is collected starting from the component with the lowest Sequence value and moving to the component with the highest Sequence value until the demand can be satisfied. If planned orders are required to fully satisfy the demand, they are created across the group's components in the ratio determined by the **BillOfMaterial.Target** field as discussed in the following section.

Define planned order generation

If RapidResponse cannot find enough current supply (on hand or scheduled receipts) from either the primary component used in an assembly or any of its substitute components, planned orders need to be created. The **BillOfMaterial.Target** field can be used to set the ratio in which dependent demand from the assembly is exploded down to each substitutable component, and hence the total quantity from planned orders created for each component.

For example, assuming the same BOM structure shown in "Use only excess supply on substitute components" on page 1569, the following Target values could be used to specify that planned orders to satisfy dependent demand from AssemblyX are only created on the primary component in each substitute group (in this case, components A1 and C1).

Table 25.3: Target values for creating planned orders on primary components only

Assembly	Component	Substitute Group.Value	Substitution Sequence	Quantity Per	Target
AssemblyX	A1	ASub	0	1	1
AssemblyX	A2	ASub	1	1	0
AssemblyX	B1		0	1	0
AssemblyX	C1	CSub	0	1	1
AssemblyX	C2	CSub	1	1	0
AssemblyX	C3	CSub	2	2	0

Similarly, planned orders can be created on the substitute components in a group, or on some combination of the primary and substitute component. For example, when planned orders are used to satisfy dependent demand from AssemblyX, the following Target values could be used to specify that within group ASub planned orders are only created on the substitute component, and within group CSub fifty-percent of the total planned requirement should be satisfied by the primary component, with the remainder split evenly between the two substitute components.

Table 25.4: Target values for creating planned orders on substitute and primary components

Assembly	Component	Substitute Group.Value	Substitution Sequence	Quantity Per	Target
AssemblyX	A1	ASub	0	1	0
AssemblyX	A2	ASub	1	1	1

Table 25.4: Target values for creating planned orders on substitute and primary components

Assembly	Component	Substitute Group.Value	Substitution Sequence	Quantity Per	Target
AssemblyX	B1		0	1	0
AssemblyX	C1	CSub	0	1	4
AssemblyX	C2	CSub	1	1	2
AssemblyX	C3	CSub	2	2	2

Proportional vs Ongoing substitution

Note also that if planned orders are required to satisfy dependent demands being passed down from an assembly to a group of substitutable components, then each substitute group's **SubstituteGroupType.ComponentSourceRule**setting determines how these dependent requirements are split between the components. This control setting has two options; "Proportional" and "OnGoing".

When the **ComponentSourceRule** is set to "Proportional" (default setting), then each dependent requirement is split proportionally between substitutable components based on their weighted targets as calculated from **BillOfMaterial.Target**. The weighted target for each component in the group is determined by dividing its Target value by the sum of the **Target** values across all other substitutable components in the group.

For example, using the **BillOfMaterial** records and **Target** values shown in "Target values for creating planned orders on substitute and primary components" on page 1571 above, the weighted target for each substitutable component in group "CSub" would be calculated as follows:

- **C1**: 4 / 4 + 2 +2 = 50%
- **C2**: 2 / 4 + 2 + 2 = 25%
- **C3:** 2 / 4 + 2 + 2 = 25%

Thus, a series of demands for "AssemblyX" would have their dependent requirements split between the substitutable components in group "CSub" as shown in the following table.

Table 25.5: Planned orders on substitutable components — "Proportional"

Demand (AssemblyX)		Dependent requirements / planned orders		
Date	**Quantity**	**C1**	**C2**	**C3**
Dec 1	500	500 * .50 * 1 = 250		500 * .25 * 2 = 250
Dec 5	300	300 * .50 * 1 = 150	300 * .25 * 1 = 75	
Dec 8	400	400 * .50 * 1 = 200		400 * .25 * 2 = 200

Conversely, when the **ComponentSourceRule** is set to "OnGoing", then each given dependent requirement passed to a group of substitutable components is satisfied by the component which would be furthest away from (and under) its ongoing target if not used to satisfy that requirement. With these settings, the ongoing target is determined by multiplying each component's weighted **Target** value by the total dependent requirements that have been passed down from planned orders on the assembly (including the current requirement).

In order to determine the distance from the target for a given substitutable component, the total dependent requirements from the assembly that were already satisfied by the component are added to the associated **BillOfMaterial.SubstitutionToDateQty** value. **SubstitutionToDateQty** is an input field used to represent the quantity, in component supply units, that has already been allocated to this substitute BOM. For example, this field might represent known component allocations on input supplies. The total of **SubstitutionToDateQty** values across all components in the group are also included in the total sum of demand received from the assembly (after any applicable adjustment for **QuantityPer**). Thus, the distance from target for each substitute component is calculated as:

```
AllocatedQtyToDate - ((TotalPreviousDemands + CurrentDemand) * TargetRatio)
```

For example, assume the set of BOM records for substitutable components C1, C2, and C3 under AssemblyX given earlier in this section, but with **SubstitutionToDateQty** values added as shown in the following table.

Table 25.6: Target values for creating planned orders on substitute and primary components

Assembly	Component	Quantity Per	Target	Substitute Group.Value	SubstitutionTo DateQty
AssemblyX	C1	1	4	CSub	100
AssemblyX	C2	1	2	CSub	0
AssemblyX	C3	2	2	CSub	400

Further, assuming the series of demands for "AssemblyX" shown in "Calculated distance from Ongoing target (and resulting planned orders)" on page 1573, each component's ongoing distance from target would be calculated as shown in that table. Note that in each case a planned order is created on the substitutable component that would be furthest from target if not used to satisfy the entire requirement

Table 25.7: Calculated distance from Ongoing target (and resulting planned orders)

Demand (AssemblyX)		Distance from target		
Date	**Quantity**	**C1**	**C2**	**C3**
Dec 1	500	100-(300+500*.5)=-300 Planned order for 500	0-(300+500*.25)=-200 No planned order	200-(300+500*.25)=0 No planned order
Dec 5	300	600-(800+300*.5)=+50 No planned order	0-(800+300*.25)=-275 Planned order for 300	200-(800+300*.25)=-75 No planned order
Dec 8	400	600-(1100+400*.5)=-150 No planned order	300-(1100+400*.25)=-75 No planned order	200-(1100+400*.25)=-175 Planned order for 800

Notes:

- If dependent demand from the assembly can be partially satisfied by current supply (onhand or scheduled receipts), then planned order generation is also affected by the setting in the **SubstituteGroupType.ComponentRule** field as discussed in "Combine substitute components on a single assembly supply order" on page 1570.
- In cases where the QuantityPer differs between substitutable components, the percentage split specified in the Target field is applied before the QuantityPer value. That is, the Target determines the percentage of dependent demand satisfied by a component's planned orders but not necessarily the percentage of planned orders created on the component.

- In some cases, particularly early in the planning horizon, it might be preferable to override the Target percentages and instead place planned orders that choose the substitute whose lower level component supply availabilities will make satisfy the demands on-time (or least late). This can be done by enabling multi-level search logic as discussed in "Multi-level search logic" on page 1863.
- If a substitute BOM is not valid for a given dependent demand (for example, because the demand is outside the effective date range or is for a different model), then that invalid BOM is excluded from the Target calculation.

Tip: If planned orders should be evenly spread across all components in the group, then each can be assigned the same **Target** value (for example, 1 could be specified).

Specify date effectivity

Because BOM level part substitution is defined through the BillOfMaterial table, BOM effectivity is respected. This means substitute components and properties of substitute components can be specified as effective within a given time period using the EffectiveInDate and EffectiveOutDate fields. For example, there might be a need to have time-phased Target ratios that determine planned order creation for a group of substitutable components. The following table shows sample BillOfMaterial values that specify dependent demand from AssemblyX to be satisfied by planned orders should be split evenly between component A1 and A2 in January, but split on a 3:1 basis between component A1 and component A2 for the rest of the quarter.

Table 25.8: Specifying time-phased Target values in BillOfMaterial table

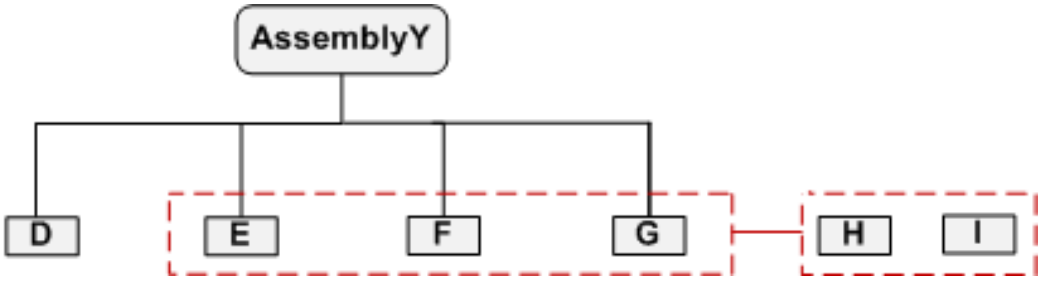
Assembly	Component	Substitute Group.Value	Substitution Sequence	Effective InDate	Effective OutDate	Target
AssemblyX	A1	ASub	0	01-01-09	01-31-09	50
AssemblyX	A2	ASub	1	01-01-09	01-31-09	50
AssemblyX	A1	ASub	0	02-01-09	03-31-09	75
AssemblyX	A2	ASub	1	02-01-09	03-31-09	25

Substituting groups of components

RapidResponse can also be configured to substitute a group of components within an assembly with another group of components. This type of BOM substitution uses the same principles as discussed in "Substitute BOM configuration" on page 1566, however as an additional step phantom BOM records must be created to group the associated components together.

For example, assume the following BOM structure where AssemblyY has a group of components E, F, and G, which can be collectively substituted with a group of components containing H and I.

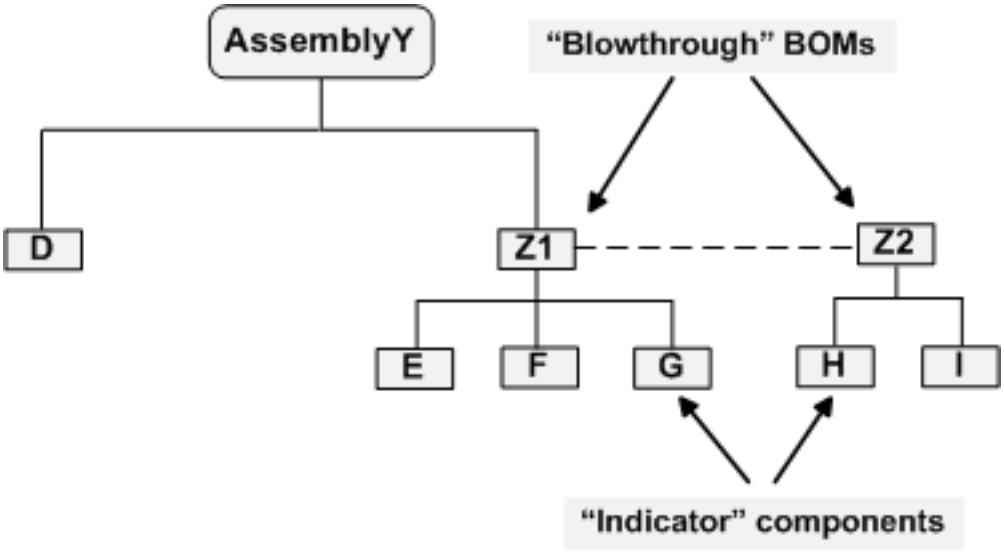
Figure 25.4 Assembly with group of substitutable components



To enable group substitution in RapidResponse, phantom BOM records should be added to the **BillOfMaterial** table (that is, records where the **BOMType.BlowThrough** value is set to "Y"). One phantom assembly is required to group each set of substitutable components, and should be added as a component under the assembly to which the substitutable components belong.

For example, in the following illustration phantom assembly Z1 is used to group components E, F, and G, while phantom assembly Z2 is used to group components H and I. When group substitution is used, the parameters that control substitution are specified on the phantom BOM records, and passed down to the components underneath. For example, the **SubstituteGroup** reference, substitution preference and planned order generation are all specified between the main assembly and the phantom.

Figure 25.5 Phantom BOMs are used to group components for substitution



Within each group of components under a phantom assembly, an indicator component must also be specified. The indicator component is the component whose supply determines if that groups components can be used. That is, if available supply is found on the indicator component, then the group can be used. When supply on the indicator is used up, then component supply from the next preferred group is considered. The indicator component is assumed to be the component in the group with the lowest **SubstitutionSequence** value.

The following table shows some values that might be included in BillOfMaterial records to model group substitution for the BOM structure shown in the previous illustration. This table assumes that a value, SubGroup1, has been added to the **SubstituteGroup** table to be define these component groups as substitutable. It also assumes that Z1 is the preferred component group to take supply from, and any planned orders to satisfy dependent demand from the assembly will be split 80 percent on component

group Z1 and 20 percent on component group Z2. The **SubstitutionSequence** values of -1 for components G and I indicate that they are the indicator components within groups Z1 and Z2 respectively.

Table 25.9: Sample BillOfMaterial records for assembly with group of substitutable components

Assembly	Component	Type. Blowthrough	Substitute Group.Value	Substitution Sequence	Target
AssemblyY	D	N		0	0
AssemblyY	Z1	Y	SubGroup1	0	80
AssemblyY	Z2	Y	SubGroup1	1	20
Z1	E	N		0	0
Z1	F	N		0	0
Z1	G	N		-1	0
Z2	H	N		-1	0
Z2	I	N		0	0

Note: If no indicator can be determined from the SubstitutionSequence values for a group's components, RapidResponse will automatically designate the component with the highest **Part.StdUnitCost** value as the indicator.

Substitutable allocations

As discussed earlier in this chapter, substitutable component supplies can be used to satisfy demand exploded from an assembly part in RapidResponse. In some cases, typically either when an order is close to being released to the shop floor or when deviations have been made from the standard bill of materials, supply allocations for a scheduled receipt are imported into the Allocation table in RapidResponse. If any of the imported allocations are considered substitutable under a given scheduled receipt, then that should be indicated on the relevant Allocation records. This ensures RapidResponse can determine incremental availability and on time quantity of the scheduled receipt based on when supplies of each of the substitutable allocation parts are available.

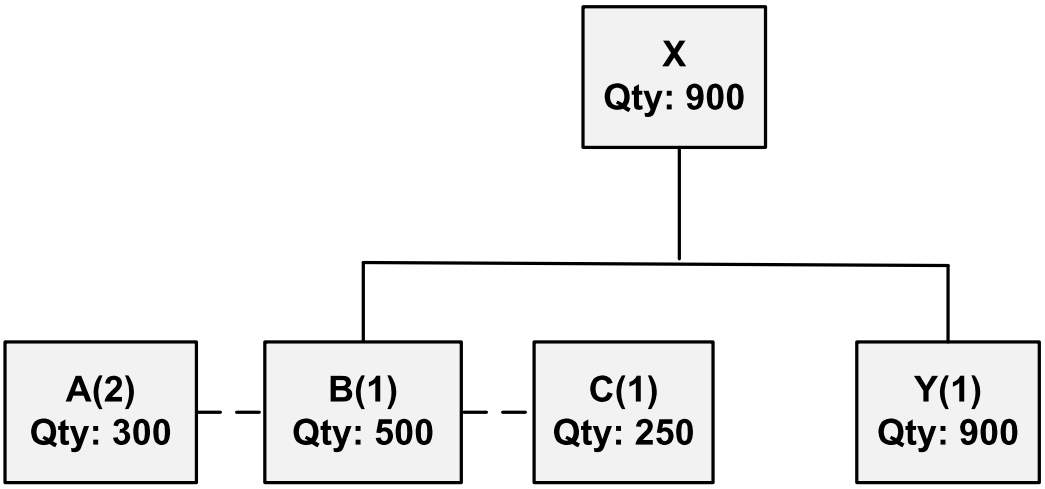
To define a group of allocations under a scheduled receipt as substitutes, each of the records must point to the same **SubstituteGroup** record, and that referenced record must have a **Type.ProcessingRule** of "Substitute". As well, each substitutable allocation should have its **QuantityPer** field populated to indicate the flat quantity per between the allocation part and the part the associated scheduled receipt is for (including any consideration for scrap). This quantity per value is used when rolling up availability of the allocation part to the scheduled receipt for determining incremental availability, as well as in "InKit" calculations for substitutable allocations. For allocations that aren't considered substitutable, the QuantityPer field can be left at its default value of 0 (zero), as RapidResponse will calculate an effective quantity per based on the ratio between the allocation part and the part the associated scheduled receipt is for.

Example

Suppose a structure as shown in the following illustration where a scheduled receipt for part X has imported allocations for component parts A, B, C, and Y (where A, B, and C are assumed to be

substitutable for one another).

Figure 25.6 Substitutable allocations under a scheduled receipt



Assuming the structure shown above, the following table shows sample Allocation table records that could be used to model the substitutable components.

Table 25.10: Sample Allocation record values (including substitutable allocations)

Part	SR. Part	SR. Order ID	SR. Line	Remaining Quantity	Quantity Per	Substitute Group	Substitute Group.Type. ProcesingRule
A	X	Wo-03	1	300	2	Group1	Substitute
B	X	Wo-03	1	500	1	Group1	Substitute
C	X	Wo-03	1	250	1	Group1	Substitute
Y	X	W0-03	1	900	0		Ignore

Next assume that the order WO-03 for X is due on June 20, and supplies of the allocated components are available as detailed in the following table.

Table 25.11: Supplies for allocated components

Part	Quantity	Available Date
A	300	June 10
B	400	June 15
B	100	June 22
C	250	June 30
Y	1000	June 10

In this scenario, 550 units of the order for X could be satisfied on time, with 150 of those on time units built using component supplies A and Y, and 400 built using component supplies B and Y. The remainder of the order for X, requiring either the late supply of B or the supply of C, would be late for the June 20 due date. Note that if substitution was not enabled on the allocations, then the order for X could not be built until supply of all components was available, in this case making the entire order late.

Limitations on other analytics

When using BOM level part substitution, a small number of other analytics and calculations might be disabled, or not work as expected, for parts configured for BOM level substitution. The following table outlines other calculations impacted by BOM level part substitution

Table 25.12: BOM level part substitution limitations on other analytics

Analytic / Calculation	Impact of BOM level part substitution
Forecast Consumption	The dependent demand created on substitutable component does not consume forecast.
Global part substitution	If two parts are configured as substitutable components within an assembly and also configured as global substitutable parts, then the BOM level part substitution settings will override the global part substitution settings
MPS Configuration	MPSConfig parts cannot be used in BOM level part substitution.
Safety Stock	Safety stock requirements on substitutable components can only be satisfied by their supply, and safety stock requirements are not used in the calculation of a substitute component's excess supply.
Dynamic safety stock	If components involved in a bom-level part substitute relationship also use dynamic safety stock logic, then the generated safety stock levels might not be as expected. Dynamic safety stock levels are calculated based on the dependent demands placed on components before any bom-level part substitution decisions have been made. This can result in higher than expected safety stock levels on the primary component and/or lower than expected safety stock levels on the substitute components. Therefore it is recommended to avoid dynamic safety stock settings on components involved in bom-level part substitution (that is, such parts should typically not use **SafetyStockQuantityRule** settings of "FractionOfDemand", "PercentOfDemand", "RangeOfCoverage", "RangeOfCoverageLeadTime", "TimePhasedRangeOfCoverage", "TimePhasedSafetyDaysMaximum", or "TimePhasedSafetyDaysTotal".
Feature BOM	BOM-level part substitution cannot be performed on configurable end-items.
Multi-level search logic	Multi-level search logic is generally supported on parts that are configured for bom-level part substitution. However the ability to define an excess window, such that substitution is only made using excess current supply, is not supported. That is, if multi-level search logic is enabled then the **PartType.ExcessRule** setting of "Fence" should be avoided as it might not return the expected results.
Multi-level search logic (with partial planning)	Multi-level search logic with partial planning logic (DemandType.PlanningRule = "Partial") is supported in conjunction with BOM Substitution as long as the relevant SusbtituteGroupType.ComponentRule setting is either "AssemblyUnit" or "Unrestricted". However, if ComponentRule is set to "Single" then the substitution logic will not work as expected (MLS partial planning will not provide partial supply).

Note: BOM level part substitution is also not performed on any scheduled receipts that have had their allocations imported into the Allocation table. However, allocations imported into this table can be configured as substitutes under a given scheduled receipt as described in "Substitutable allocations" on page 1576.

CHAPTER 26: Co-products and by-products

RapidResponse can be configured to support generating co-product and by-product supplies from the production of a primary product. This allows support for configurations where multiple distinct parts emerge from a single production process.

This chapter contains an overview of co-products and by-products, provides instruction on configuring co-products and by-products in RapidResponse, and explains how their calculated output is generated.

In this chapter, you'll learn about:

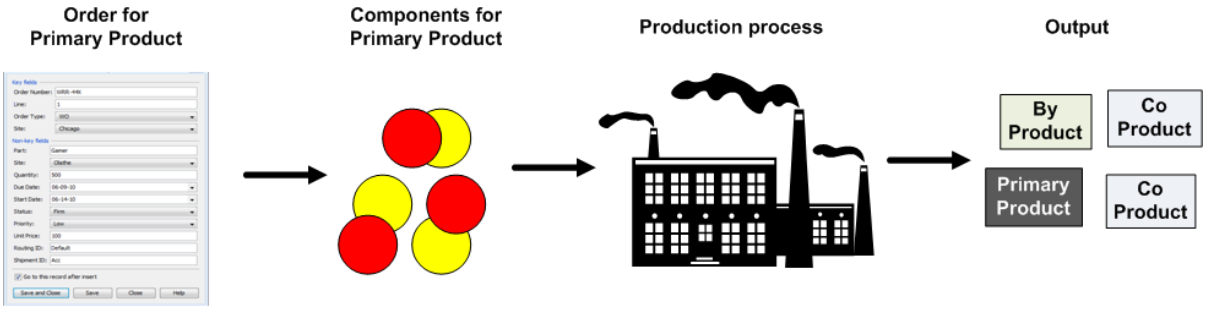
Co-product and by-product basics **1581**
Co-product and by-product data model **1582**
Configuring co-products and by-products **1583**
Co-product and by-product calculations **1596**
Limitations on other analytics **1598**

Note: BOM charts in RapidResponse provide you with a view of the bill of material. You can use a BOM chart to view components and their co-products and by-products. The BOM Analysis workbook includes a BOM chart in the BOM Visualization worksheet. For more information, see the *RapidResponse User Guide*.

Co-product and by-product basics

As shown in the following illustration, co-products and by-products are products that are generated during, and because of, the production of another part.

Figure 26.1 Co-product and by-product overview



The parts that have real demand triggering supply production are known as primary products. When these primary products are made, co-product and by-product supplies are generated from their scheduled receipts or planned orders. For example, 10% of a primary product's output might come out as a by-product, 30% as a co-product, and the remainder as the primary product itself.

Because the co-product and by-product supply is derived from production of the primary product, it is the primary product whose components and availability determines the availability of any co-product or by-product supply. In addition, planning policies defined on the primary product part affect the generation of co-product and by-product supplies (as opposed to any planning policies defined on the co-product or by-product parts themselves). For example, yield ratios, lot sizing and sourcing policies defined on a primary product affect the generation of co-product and by-product supply. As well, constraint and capacity limitations on a primary product can affect the availability of co-product and by-product supply.

Although co-product and by-product supplies are both derived from production of primary product and do not explode or drive demand to any dependent components, they have a few differences.

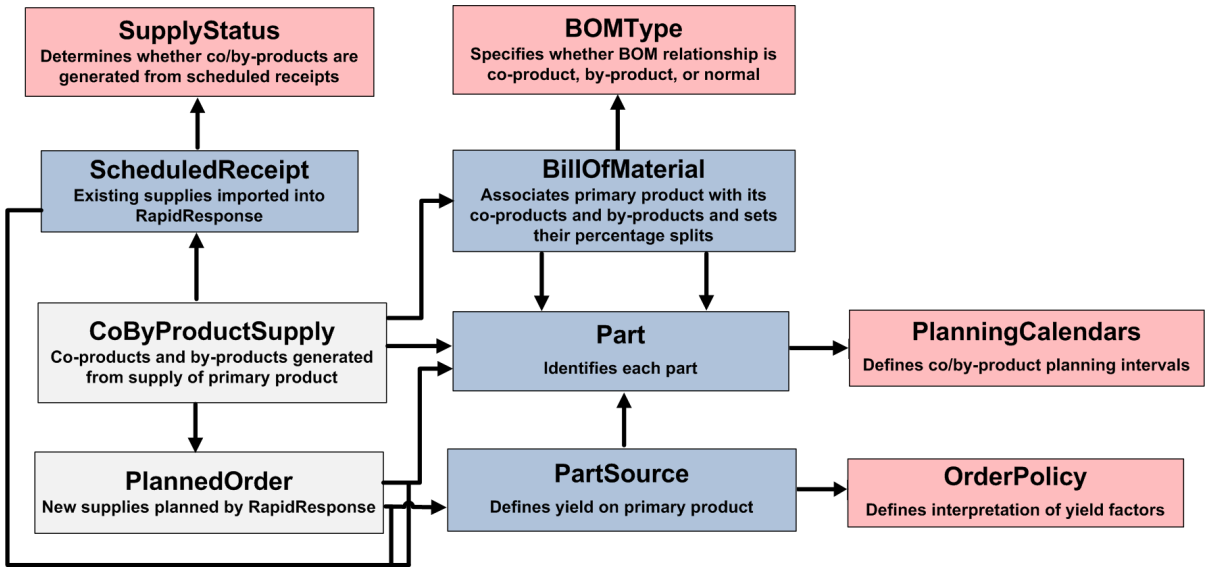
Co-products are parts manufactured alongside the supply of a primary product to which they are functionally similar and with which they might typically share common components or manufacturing processes. For example, a co-product might be a lower quality grade version of a primary product. In such cases, a testing process might be run after production to determines the quality and capabilities of the output. Based on the testing the results, a given item can be labeled as the primary product or one of its co-products. In RapidResponse, the netting and CTP analytics process and perform calculations for parts related through co-product relationships at the same time.

By-products are parts generated from and considered residual to the production of a primary product. For example, a by-product might be a chemical generated during the production of one or more assemblies (primary products) in a given product structure. In other cases, a by-product might be scrap material with value that is salvaged or recovered during the production process of the primary product. In RapidResponse, the netting and CTP analytics process and perform calculations for a by-product part only after all of its BillOfMaterial assemblies have been planned.

Co-product and by-product data model

Co and by-products are configured at the BOM-level which allows for co-product and by-product supplies to be generated from planned orders and scheduled receipts for a primary product. The following illustration shows the main tables used to support co-products and by-products in RapidResponse, while the remainder of this section provides details on related configuration points and explains the calculated output.

Figure 26.2 Main tables supporting co-products and by-products



Configuring co-products and by-products

This section discusses setting up co-products and by-products in RapidResponse. The following table outlines the main configuration points for co-product and by-product logic.

Table 26.1: Co-product and by-product configuration points

Step	Description
Define product types and percentage splits.	To define co-product and by-product configurations in RapidResponse, BillOfMaterial records should be created that associate primary products with their co-product and by-products. As well, the percentage split of supply output between primary products and their co-products and by-products needs to be defined. For more information, see "Define product types and percentage splits" on page 1584.
Define co-product planning intervals	A planning calendar can be specified to indicate a time interval over which demands for a primary product should be collected and processed together. This can help reduce potential excess supply. For more information, see "Specify co-product planning calendar intervals" on page 1589.
Define due and available date offsets.	If co-products and by-products should have different due and available dates than the primary product supply they are derived from, an offset can be specified to set how much earlier or later they should be. For more information, see "Define due date and available date offsets" on page 1591.
Define expiry offsets.	If using material expiration, and co-products and by-products should have different expiry dates from the primary product supply they are derived from, an offset can be specified to set how much earlier or later they should expire. For more information, see "Define expiry date offset" on page 1591.

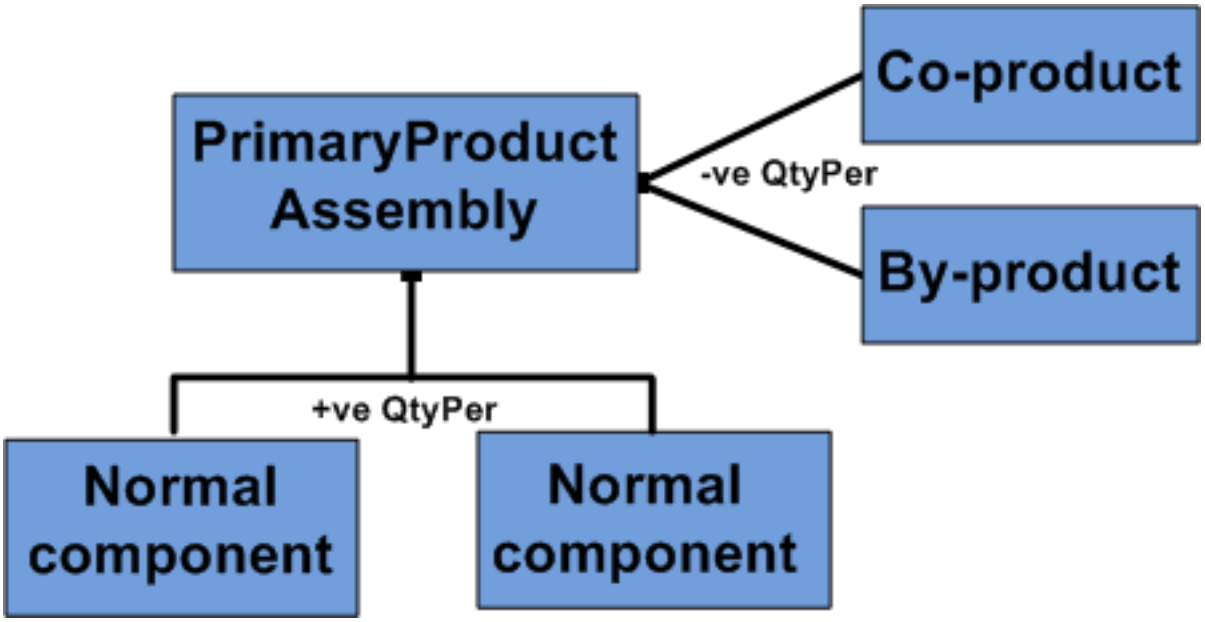
Table 26.1: Co-product and by-product configuration points

Step	Description
Specify co-product generation on scheduled receipts	A control setting is available to set whether co-products and by-products should be generated for given scheduled receipts. This allows for modelling scheduled receipts in the later stages of production that have already had co-product and by-product supply generated. For more information, see "Managing co-products and by-products on scheduled receipts" on page 1592.
Specify appropriate order priority settings.	If using order priority in RapidResponse, attention should be paid to the PartType.CommitLevel option; in particular setting this option to "High" could result in unintended results such as excess co-product supply. For more information, see "Using order priority with co-products" on page 1592.
Consider use of substitute part logic.	In some cases, supply of a co-product can be used to satisfy demand for a primary product. In these cases, co-products can potentially be set up as part substitutes in RapidResponse. For more information, see "Using part substitution with co-products" on page 1594.

Define product types and percentage splits

Co-products and by-products are configured using **BillOfMaterial** records. As shown in the following illustration, the primary product should be set as an assembly part with each of its co-products and by-products defined as components underneath it (in addition to all of the normal components used in production of the primary product).

Figure 26.3 Bill of material relationships define co-products and by-products



Negative **QuantityPer** values then need to be set on the **BillOfMaterial** records that define co-products and by-products to indicate the percentage of planned primary product supply that ends up as each co-product or by-product.

As well, the **PartSource.CoProductYield** value associated with the primary product should be set to indicate the percentage of supply that is expected ends up as the primary product itself. The remaining percentage is then distributed between its co-products and by-products based on the negative **QuantityPer** values specified on the appropriate BOM records

If the coproductyield on the primary product is expected to vary over time (for example, due to process changes), then the **CoProductYield** on the **TimePhasedYield** table should be used instead. That is, a record can be added to this table referencing the applicable **PartSource** for each **EffectiveInDate** on which its coproductyield is expected to change. For more information, see "TimePhasedYield table" on page 661.

Notes:

- A given primary product can have more than one set of co-product relationships. For example, there could be different ways to make the primary and co-products. These can be setup using alternate BOMs. For more information, see "Alternate BOM calculations" on page 1759.
- Planning policies defined on the primary product part affect generation of co-product and by-product supplies.

Define product types on components

For all components defined under a primary product assembly, the **BOMType.ProductType** control setting is used to define the component's relationship to the primary product. This field has the following three settings:

- **Normal**—a normal dependent component used in the production of the assembly. This value should be used on the BillOfMaterial records that associate the primary product (assembly) with each of the actual components used in its production.
- **CoProduct**—a co-product that can be manufactured and planned together with the assembly (primary) product due to common structure, components and process similarities. For example, a co-product might be a lower grade version of the primary product. This value should be used on the BillOfMaterial records that associate the primary product (assembly) with each of its co-products. Netting and CTP calculations for parts with a co-product relationship are performed at the same time. Therefore, two-way co-product relationships are supported.
- **ByProduct**—a by-product that is generated from, and residual to, the production of the assembly (primary) product. For example, a by-product might be a chemical generated from the production of one or more assemblies in a given product structure. This value should be used on the BillOfMaterial records that associate the primary product (assembly) with each of its co-products. Netting and CTP calculations for a by-product part are performed after all of its BillOfMaterial assemblies have been planned. Therefore, two-way by-product relationships are not supported.

Define co-product and by-product percentages

In order to indicate the percentage of a primary part's production that ends up as either a co-product or by-product, a negative QuantityPer value should be specified on each BillOfMaterial record that associates a primary product assembly with any of its co-products or by-products. This value is used by RapidResponse in determining the amount of co-product and by-product supply that is generated from an order for the primary product. The QuantityPer value to use for a co-product or by-product configuration is based on the following calculation:

```
- (Co-Product or By-Product % / Primary Product %)
```

Define primary product percentage

The portion of a primary product's supply production that ends up as the primary product itself is specified in the **PartSource.CoProductYield** field associated with the primary product supply's part source (on either the PartSource table . The **CoProductYield** field accepts fractional or percentage values (subject to the **OrderPolicy.YieldUsage** setting), and indicates the expected yield of the primary product after accounting for any co-products or by-products that come out of the production process.

Note: CoProductYield is combined with PartSource.Yield to determine the effective yield on the part source (**PartSource.EffYield**). For more information about yield, see "Yield" on page 1391.

Examples of co-product and by-product configurations

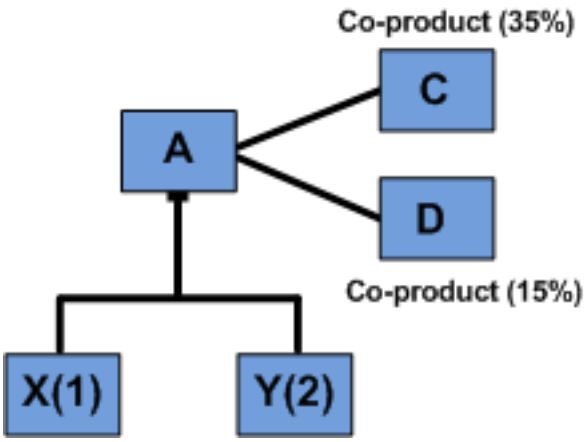
This section provides examples of BillOfMaterial and PartSource values that might be set to define some sample co-product or by-product configurations. The following are shown:

- "Example 1 - Configuring co-products" on page 1586
- "Example 2 - Configuring a by-product" on page 1587
- "Example 3 - Configuring co-products and a by-product" on page 1587
- "Example 4 - Configuring two-way co-products" on page 1588

Example 1 - Configuring co-products

Assume a BOM structure as shown in the following illustration, where A is a primary product that requires one unit component X and two units of component Y to build it, and also has two co-product parts C and D (where 35% of the production of A ends up as part C and 15% ends up as D).

Figure 26.4 The primary product has two co-products



In order to support the configuration shown in the previous illustration, the **PartSource.CoProductYield** value associated with part A should be set to .5. As well, the following table shows sample BillOfMaterial values to support this configuration; the right-most column gives details of the calculation used to determine the QuantityPer values on the co-products.

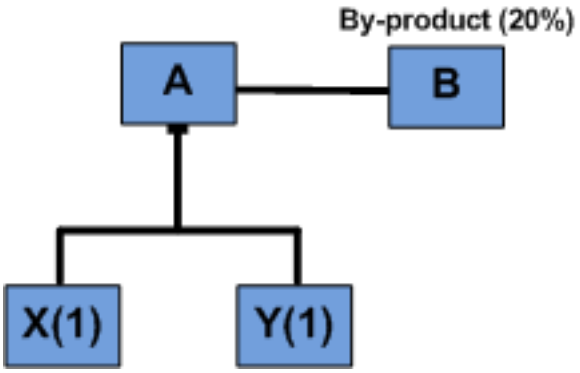
Table 26.2: Sample BillOfMaterial values to support the figure

Assembly	Component	Type.ProductType	QuantityPer	
A	X	Normal	1	
A	Y	Normal	2	
A	C	CoProduct	-.7	-(.35 / .5)
A	D	CoProduct	-.3	-(.15 / .5)

Example 2 - Configuring a by-product

Assume a BOM structure as shown in the following illustration, where A is a primary product that requires one unit each of components X and Y to build it, and also has one by-product part, B (where 20% of the production of A ends up as part B).

Figure 26.5 The primary product has one by-product



In order to support the configuration shown in the previous illustration, the **PartSource.CoProductYield** value associated with part A should be set to .8. As well, the following table shows sample BillOfMaterial values to support this configuration; the right-most column gives details of the calculation used to determine the QuantityPer value on the by-product.

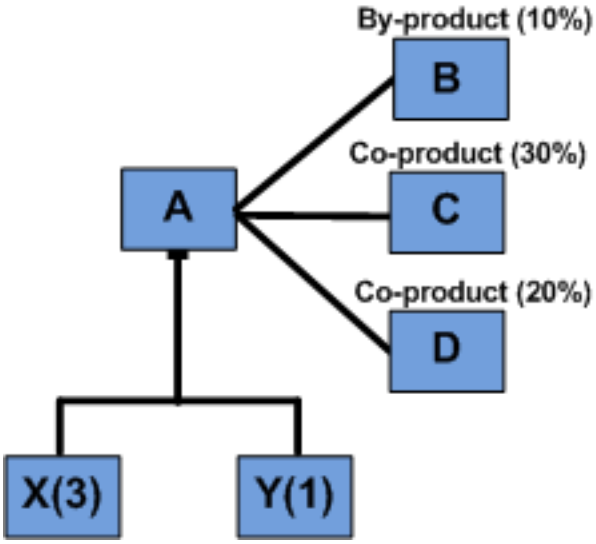
Table 26.3: Sample BillOfMaterial values to support the figure

Assembly	Component	Type.ProductType	QuantityPer	
A	X	Normal	1	
A	Y	Normal	1	
A	B	ByProduct	-.25	-(.2 / .8)

Example 3 - Configuring co-products and a by-product

Assume a BOM structure as shown in the following illustration, where A is a primary product that requires three units of component X and 1 unit component Y to build it, and also has two co-product parts C and D, and one by-product part B (where 10% of the production of A ends up as B, 30% as C, and 20% as D).

Figure 26.6 The primary product has two co-products and one by-product



In order to support the configuration shown in the previous illustration, the **PartSource.CoProductYield** value associated with part A should be set to .4. As well, the following table shows sample BillOfMaterial values to support this configuration; the right-most column gives details of the calculation used to determine the QuantityPer values on the co-products and by-product.

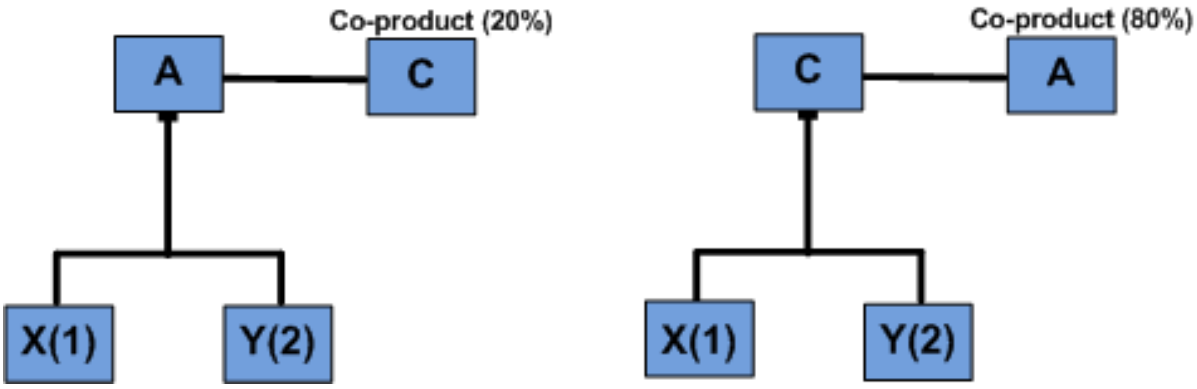
Table 26.4: Sample BillOfMaterial values to support the figure

Assembly	Component	Type.ProductType	QuantityPer	
A	X	Normal	3	
A	Y	Normal	1	
A	B	ByProduct	-.25	-(.1 / .4)
A	C	CoProduct	-.75	-(.3 / .4)
A	D	CoProduct	-.5	-(.2 / .4)

Example 4 - Configuring two-way co-products

Assume BOM structures for a pair of two-way co-products as shown in the following illustration. In this example A is a primary product that requires one unit of component X and two units of component Y to build it, and has C as a co-product (where 20% of the production of A ends up as C). And C is also a primary product that requires one unit of component X and two units of component Y to build it, and has A as a co-product (where 80% of the production of C ends up as A).

Figure 26.7 Two-way co-products are configured



In order to support the configuration shown in the previous illustration, the **PartSource.CoProductYield** value associated with part A should be set to .8, and the **PartSource.CoProductYield** value associated with part C should be set to .2. As well, the following table shows sample BillOfMaterial values to support this configuration; the right-most column gives details of the calculation used to determine the QuantityPer value on the co-products.

Table 26.5: Sample BillOfMaterial values to support the figure

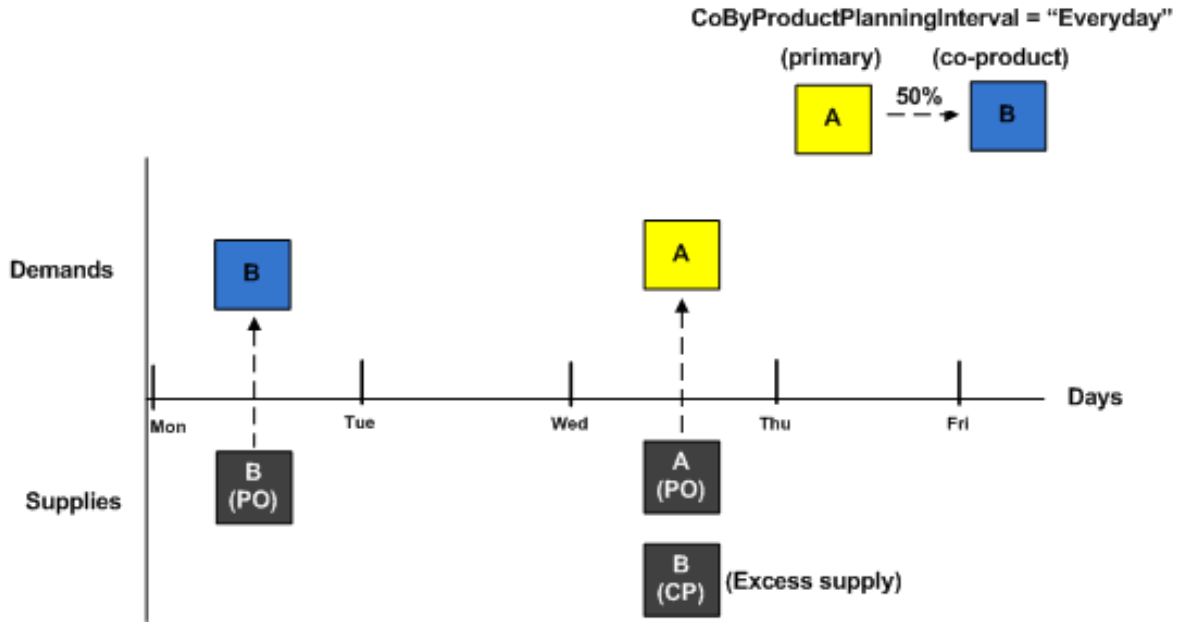
Assembly	Component	Type.ProductType	QuantityPer	
A	X	Normal	1	
A	Y	Normal	2	
A	C	CoProduct	-.25	-(.2 / .8)
C	X	Normal	1	
C	Y	Normal	2	
C	A	CoProduct	-4	-(.8 / .2)

Specify co-product planning calendar intervals

When using co-product and by-product logic in RapidResponse, there is a potential for excess supply on the co-product parts if there is not enough demand to consume their generated supply. The **PlanningCalendars.CoByProductPlanningInterval** calendar can therefore be used to specify a period of time over which demands on the primary product are collected together, and any required planned orders are then created at the beginning of that period. This ensures that resulting co-product supplies that get generated are then available to satisfy demands over the duration of the specified interval.

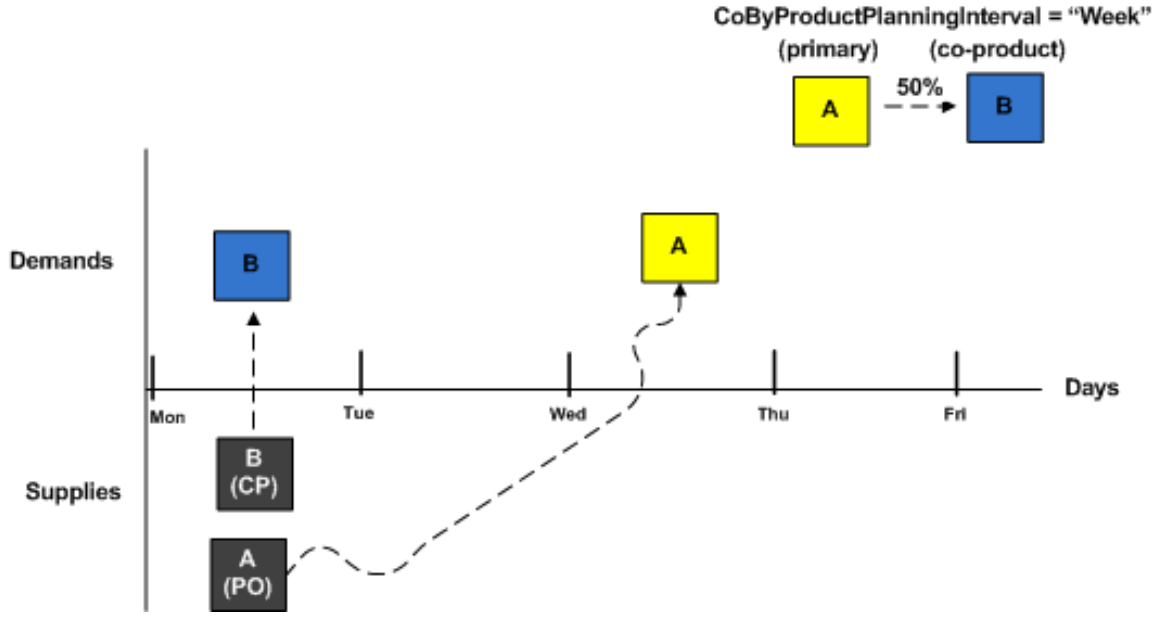
For example, in the following illustration part B is defined as a co-product of part A, the CoByProductPlanningInterval calendar is assumed to be set to an “everyday” calendar, and no onhand or scheduled receipt supplies are assumed. In this situation, the demand for B on Monday is satisfied by a planned order for B, and the demand for A on Wednesday is satisfied by a planned order for A which also results in co-product supply being generated for B.

Figure 26.8 Excess supply is created on co-product part



However, as shown in the following illustration, if the **CoProductPlanningInterval** were set to a "weekly" calendar, then the excess could have been avoided. In this case, the planned order for A would be created at the start of the weekly period, along with the resulting co-product supply on B. This co-product supply is then available to satisfy the demand for B on Monday with no need to create a planned order on B to satisfy its demand.

Figure 26.9 Co-product supply is used to satisfy earlier demand



Notes:

- A given primary product and all of its associated co-products and by-products should reference the same **CoByProductPlanningInterval** calendar.
- A given primary product and all of its associated co-products and by-products should reference the same **CoByProductPlanningInterval** calendar.
- If excess on co-products is not a concern, or the sort of "pre-planning" logic described in this section is not desirable, then the **CoByProductPlanningInterval** calendar should be set to an "everyday" calendar.

Define due date and available date offsets

Because co-products and by-products are generated from the production of a primary product, the due and available dates of co-product and by-product supplies are determined from the due and available dates of the primary product supply they were derived from. In some cases, though, co-product and by-product supplies might require different due and available dates than the associated primary product supply. For example, there could be additional labelling or packaging time associated with a co-product that would make it available later than the primary product supply.

To specify due date and available date offsets on co-product and by-products, the **BillOfMaterial.LeadTimeOffset** field can be used (subject to the setting in **BOMType.UseLeadTimeOffset**). For records that define co-product or by-product configurations, the LeadTimeOffset field accepts both positive and negative values to indicate the direction and size of the offset. The value provided in this field is interpreted in terms of the primary product's **PlanningCalendars.TimeUnits** (typically, workdays) and represents the amount of time to add or subtract from the primary product's available and due dates when determining the co-product or by-product's due and available dates.

Define expiry date offset

If using material expiration, RapidResponse can be configured to support different expiry dates between a primary product and any of its co-products or by-products.

On the record that associates a primary product with a given co-product or by-product, the **BillOfMaterial.IntervalToExpiryOffset** field can be used to specify the number of intervals (in terms of the primary product's expiry calendar) than a co-product or by-product's expiry date should be offset from the primary product's expiration. If a co-product or by-product has a longer shelf life than the primary product, then this field should be set to a positive value. Similarly, if a co-product or by-product has a shorter shelf life than the primary product, then this field should be set to a negative value.

Expiration on a co-product or by-product is also affected by its **PartType.ExpiryRule** setting. If this field is set to "Self", then the IntervalToExpiryOffset value is combined with the primary product's **PartSource.CTPIntervalsToExpiry** and/or **PartSource.IntervalsToExpiry** values (both expressed in terms of the primary product's PlanningCalendars.ExpiryCalendar), and the resulting number of intervals is then added to the co-product or by-product's available date.

Otherwise, if the Expiry Rule on the co-product or by-product part is set to "Dependent", then its expiry date is rolled up from the primary product supply.

Notes:

- Based on the setting in the **BillOfMaterial.LeadTimeOffset** field, co-product or by-product supplies might have different available dates than the primary product supply they were derived from which can further affect the difference in their expiry dates. For more information, see "Define due date and available date offsets" on page 1591.
- For more information about material expiry, see "Material expiration" on page 1601.

Managing co-products and by-products on scheduled receipts

Co-product and by-product supplies are automatically generated from primary product scheduled receipts and planned orders based on the settings defined on the relevant BillOfMaterial records. If required, individual scheduled receipts can be configured to not generate co-products and by-products. For example, in later stages of a scheduled receipt's production where the co-product and by-product supplies have already been created and are being managed separately, there would be no need for RapidResponse to generate co-products and by-products.

The **SupplyStatus.CoByProductUsage** setting controls whether RapidResponse creates co-product and by-product supplies from a primary product's scheduled receipt. This field has the following two values:

- **Generate—** Co-product and/or by-product supply is automatically generated from the primary product's scheduled receipt based on the QuantityPer values defined on the associated BillOfMaterial records. This setting is typically used on scheduled receipts for which the expected co-product or by-product supply has not been generated. This is the default setting in RapidResponse.
- **DoNotGenerate—** Co-product and by-product supply are not generated from the scheduled receipt. This setting is typically used on scheduled receipts in the later stages of production process where any co-product or by-product supplies have already been created and are being managed separately from the primary product.

Note: If a scheduled receipt has a **SupplyStatus.State** value of "Historical", it will not generate co-product and by-product supplies regardless of the CoByProductUsage setting.

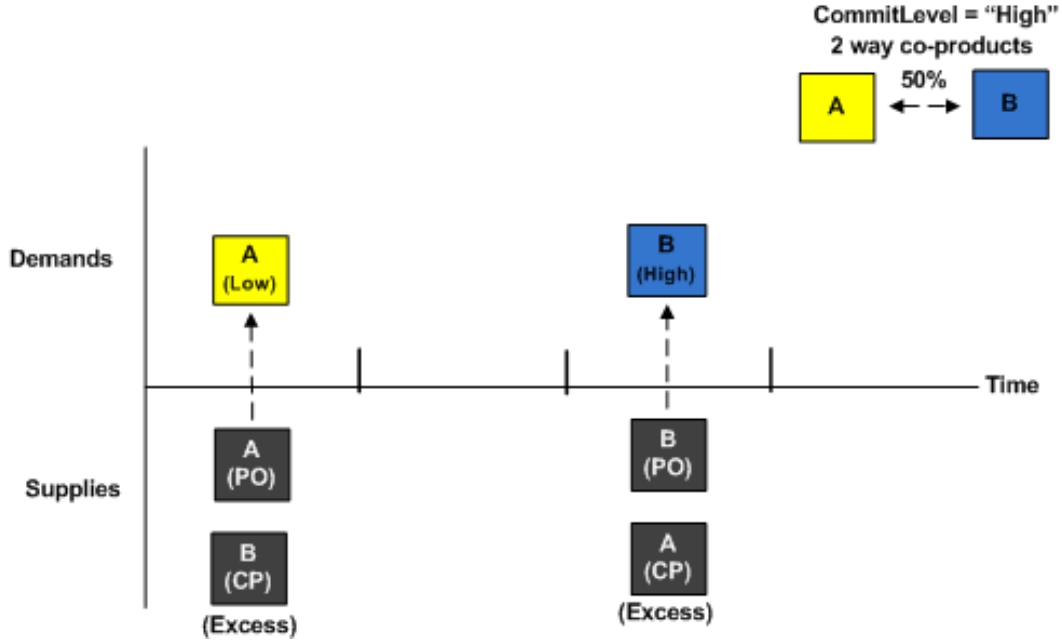
Using order priority with co-products

If order priority is used with parts involved in a co-product configuration, then caution should be taken if setting the PartType.CommitLevel to "High". This setting causes the Netting analytic to process demands in priority sequence and pass priorities from demands to planned supplies, and ensures that the Capable-To-Promise analytic respects priorities when allocating supply to demand. However, processing demands in priority sequence can potentially lead to excess for parts defined in a co-product configuration.

For example, in the following illustration the PartType.CommitLevel is assumed to be "High", A and B are two-way co-products with a 50/50 percentage split, and there is an early Low priority demand for A as well as a later High priority demand for B. In this case, the Netting analytic processes the high priority demand for B first, resulting in 1 planned order for B and a corresponding co-product supply for A.

Netting then processes the low priority demand for A, resulting in 1 planned order for A and a corresponding co-product for B. This results in excess supply as neither of the co-product supplies can be used to satisfy any demands.

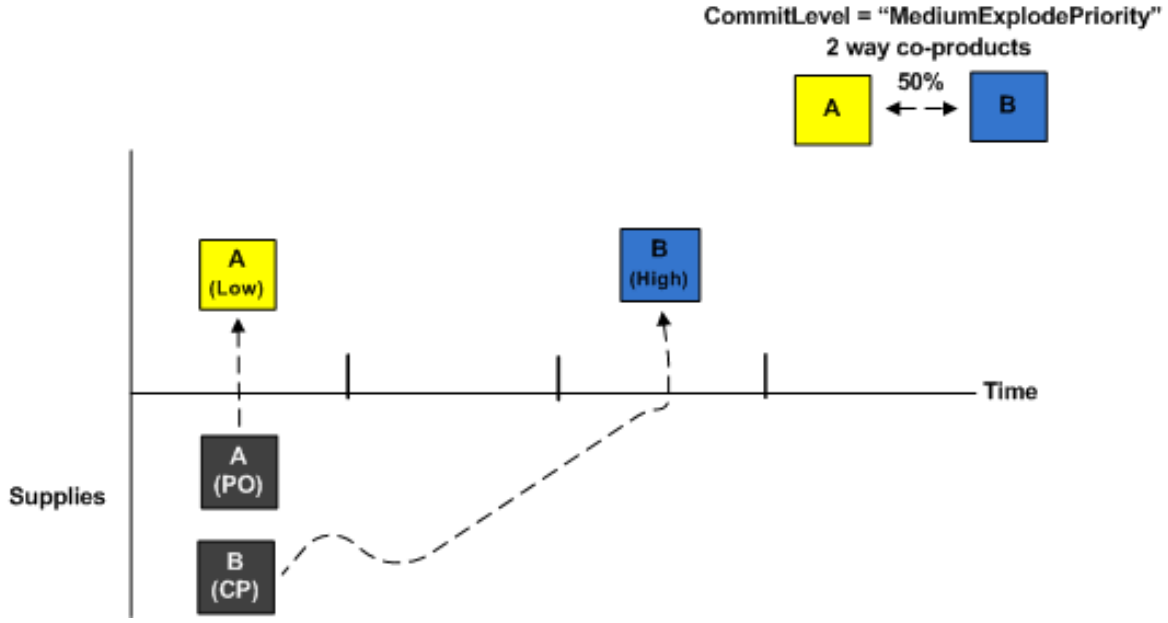
Figure 26.10 Demands are processed in priority sequence (high to low)



Therefore, if order priority is used with parts setup in a co-product configuration, the PartType.CommitLevel field should typically be set to "MediumExplodePriority". This setting is similar to "High" in that Netting propagates priorities from demands to planned supplies, and CTP uses priority when allocating supply to demand. However, this setting differs from high in that Netting processes demands by date instead of priority sequence.

For example, if in the previous example, the PartType.CommitLevel was set to "MediumExplodePriority", then the excess situation could be avoided. That is, by allowing Netting to processing demands in date sequence, the low priority demand for A would be processed first, resulting in a planned order for A and a corresponding co-product supply for B. This co-product supply could then be used to satisfy the later demand for B.

Figure 26.11 Demands are processed in date sequence



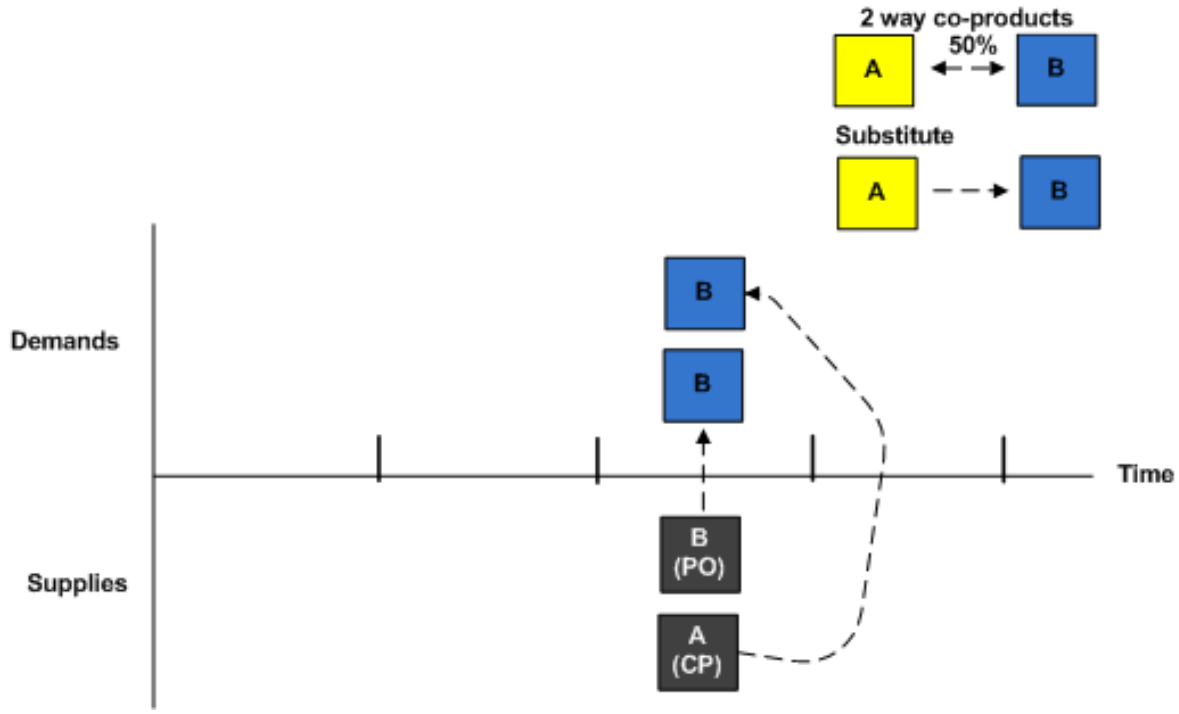
Using part substitution with co-products

In some cases, substitutable parts might be set up in co-product configurations in RapidResponse. For example, a high and low grade version of a part might be co-products that get produced in the same manufacturing process, and the high grade version could also be considered an acceptable substitutable for the low grade version of the part.

In these situations, demands for the substitutable parts that are due on the same day are sorted and processed together, and any generated co-product supply is consumed as it is created.

For example, in the following illustration parts A and B are set up as two-way co-products with a 50/50 percentage split, A is assumed to be an acceptable substitute for B, and there is no existing supply for either part. There are 2 units of demands for B due on the same day, which results in 1 planned order to satisfy a demand B, and 1 co-product supply for A which is then used as a substitute to satisfy the other demand for A.

Figure 26.12 Substitutable co-products

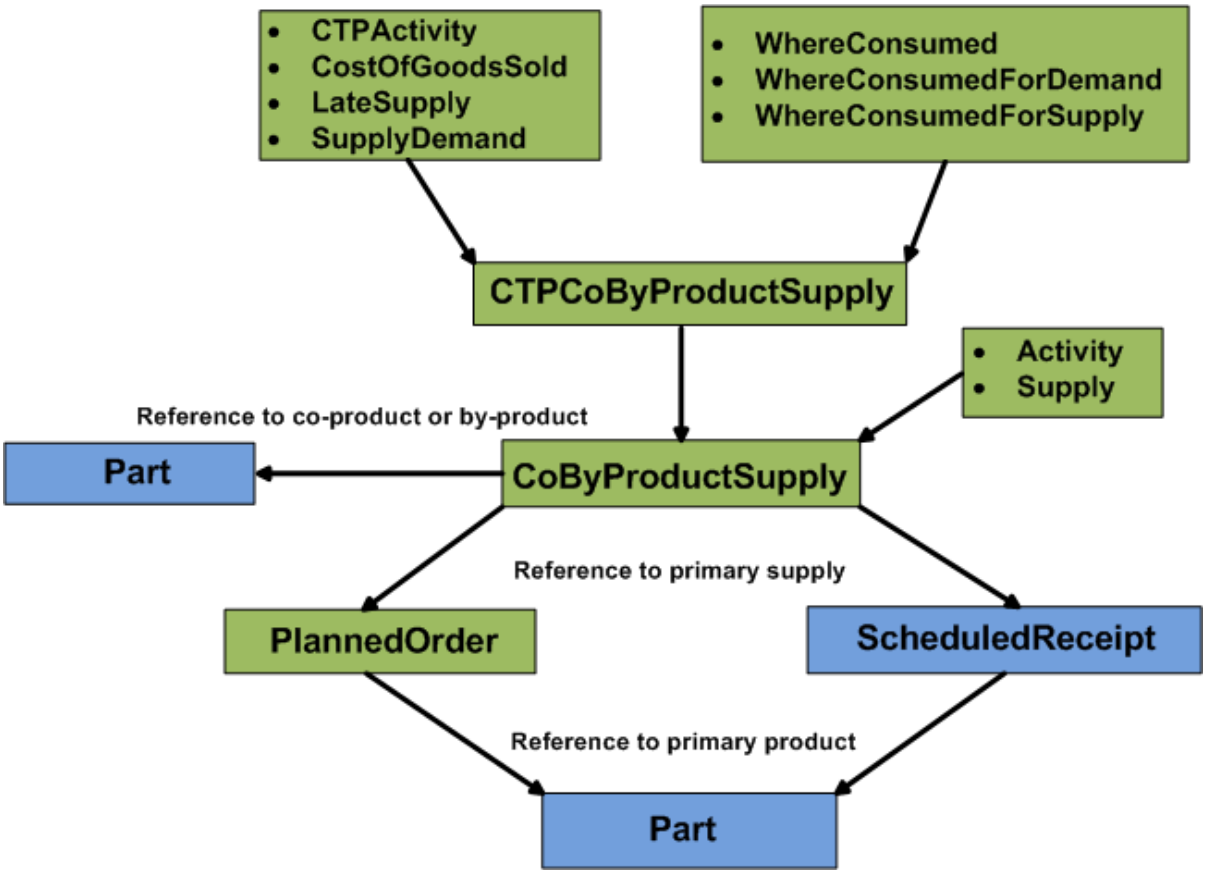


Note: In order to avoid potential excess when substitutable parts are configured as co-products, the CoByProductPlanningInterval calendar can be used to set a time window within which demands for substitutable parts are collected together (treated as if due on the same day). For more information, see "Specify co-product planning calendar intervals" on page 1589.

Co-product and by-product calculations

As shown in the following illustration, a number of calculated tables in RapidResponse report or reference details pertaining to co-product and by-product calculations.

Figure 26.13 Tables used in reporting co-product and by-product supply

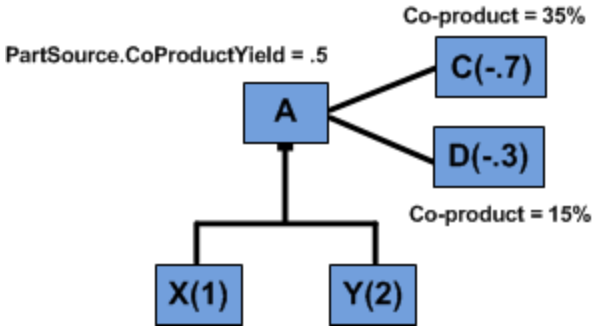


Co-product and by-product supplies (example)

The CoByProductSupply and CTPCoByProductSupply tables are the main tables used to report co-product and by-product calculations in RapidResponse. CoByProductSupply reports details of netting calculations, such as due date and quantity, and CTPCoByProductSupply reports details of capable-to-promise calculations, such as available date and on time quantity. The calculations reported in these tables are based on the details of the primary product supply the co-product or by-product supply was derived from, as well as the BillOfMaterial and other related values used to define the co-product and by-product configurations.

For example, assume a BillOfMaterial structure as shown in the following illustration where A has two co-products C and D.

Figure 26.14 The primary product has two co-products.



Further assume:

- a demand for 200 units of Part A
- a demand for 100 units of Part C (due on the same date as the demand for A)
- no on-hand or scheduled supplies for any of the parts.

In this example, two records would be generated in the CoByProductSupply table with details as shown below. The Quantity of each is based on the primary product supply's effective quantity, in this case 200, multiplied by the QuantityPer on the record that defines the co-product configuration (the absolute value of this calculation is reported). Note also the NeedQuantity field can used to report the amount of the co-product supply that gets used to satisfy a demand for the co-product part.

Table 26.6: Sample CoByProduct values

Part	PrimaryPart	PrimarySupplyType	Quantity	NeedQuantity
C	A	PlannedOrder	140	100
D	A	PlannedOrder	60	0

Notes:

- For complete reference information on these tables, see "CoByProductSupply table" on page 961 and "CTPCoByProductSupply table" on page 996.
- The total amount of co-product or by-product supply on a given part is reported in the TotalCoProductSupply and TotalByProductSupply fields on both the Part and InventorySummary tables.

Primary product supplies (example)

The CoByProductSupply table also provide references to the driver ScheduledReceipt or PlannedOrder record for the primary product that generated supply of the co-product or by-product. These supply records have their quantities inflated based on the **PartSource.CoProductYield** field to account for any of co-product or by-product supplies, while their effective quantities report the actual amount of supply that ends up as the primary product.

For example, based on the example shown in "Co-product and by-product supplies (example)" on page 1596, the demand for 200 units would be multiplied by the CoProductYield value of .5 to determine the

required quantity of 400. The primary product's component supplies are then based on the (inflated) Quantity value as shown in the following illustration.

Table 26.7: Sample PlannedOrder values

Part	Quantity	EffQuantity
A	400	200
X	400	400
Y	800	800

Note: This example assumes the **PartSource.Yield** value is set to 1.

Component allocations (example)

Although co-product and by-product supplies do not explode to any dependent components, and are generated from the production of their primary product's supplies, any of the primary product's components that get used are allocated in RapidResponse between that primary product and its co-product and by-product supplies.

For example, based on the example shown in "Co-product and by-product supplies (example)" on page 1596, the following table shows WhereConsumed records reporting how the resulting planned order for component X and the planned order for component Y get allocated between the primary product supply and its co-products.

Table 26.8: Sample WhereConsumed values

Part	Quantity	DriverPart	DriverType	NeedQuantity
X	400	A	IndependentDemand	200
X	400	C	IndependentDemand	100
X	400	C	CoByProductSupply	40
X	400	D	CoByProductSupply	60
Y	800	A	IndependentDemand	400
Y	800	C	IndependentDemand	200
Y	800	C	CoByProductSupply	80
Y	800	D	CoByProductSupply	120

Limitations on other analytics

A small number of other analytics, calculations, and BOM-related settings might be disabled, or not work as expected, for parts set up in a co-product or by-product configuration. The following table outlines other calculations impacted by co-product and by-product logic.

Table 26.9: Co-product and by-product limitations on other analytics

Calculation / Setting	Impact of co-products and by-products
BOM substitution	Parts involved in a co-product relationship can still be defined as BOM-level substitutes. However, the **SubstituteGroup** field is ignored on the **BillOfMaterial** record that defines a co-product or by-product relationship (between the primary part and co or by-product part). Instead, to define BOM substitution between co-products, this should be on different **BillOfMaterial** records between the assembly (where co-products are used) and co-products (as components).
Dependent Unit Rule	The **Type.DependentUnitRule** setting is always treated as being set to "First" for any BOM record that defines a co-product or by-product configuration.
Explode Negatives	The **Type.ExplodeNegatives** setting is ignored on records that define a co-product or by-product configuration (always treated as "Y" as **QuantityPer** values are required to be negative on **BillOfMaterial** records that define co-products or by-products).
Ignore Assembly Yield	BOM records that define a co-product or by-product configuration are always treated as though **Type.IgnoreAssemblyYield** is set to "N".
Kanban	Using Kanban logic with parts configured in a co-product or by-product relationship will lead to unexpected results. Specifically, co-product supply does not consume kanban bins and can therefore result in the kanban limit being exceeded. Supply on the primary product can consume kanban bins, but may lead to invalid results.
MPS Config	MPS configurations (that is, where **Type.MPSConfigDemandSource** is set to "Y") are ignored for any BOM record that defines a co-product or by-product relationship.
Option ratio	The **BillOfMaterial.OptionRatio** setting is ignored for all bill of material records that define a co-product or by-product configuration.
Phantom processing	BlowThrough configurations (that is, where **Type.Blowthrough** is set to "Y") are ignored for any BOM record that defines a co-product or by-product relationship.
Scrap	Scrap rules defined on any BOM record that defines a co-product a or by-product relationship are ignored.

CHAPTER 27: Material expiration

Expiry dates can be provided or calculated on supplies in RapidResponse to indicate a date after which a supply is assumed to no longer be usable (expired). Demands can then be configured to use only those supplies that haven't expired before a specified date based on the demand's shelf life requirements. RapidResponse then ensures that demands are satisfied with non-expired supplies, and also calculates the expiring quantity on supplies to give visibility into the inventory risk from expiring supplies.

This chapter provides an overview of the tables and fields in which data must be set up in order to enable material expiration, and explains how expiry related calculations are performed.

In this chapter, you will learn about:

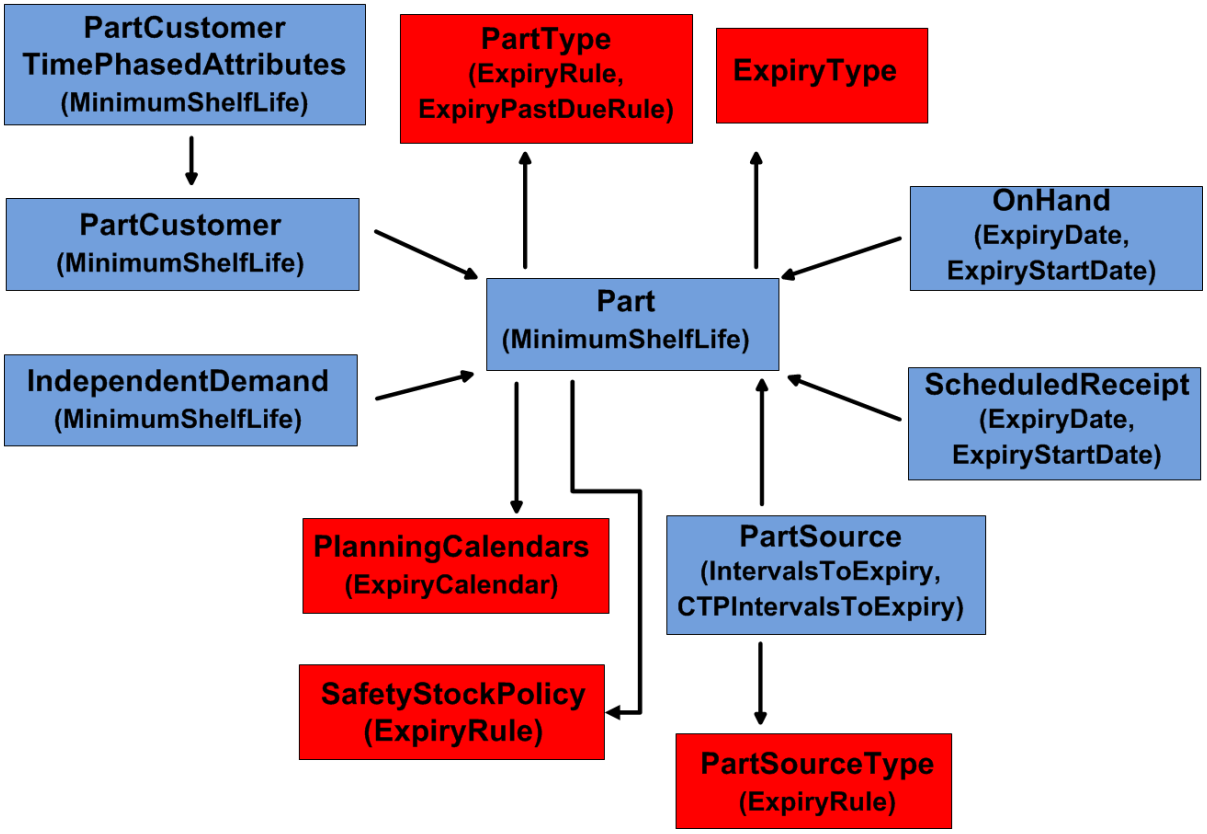
Setting up expiry data **1601**
Expiry calculations **1609**
Limitations on other analytics **1626**

Setting up expiry data

To enable material expiration logic in RapidResponse, appropriate input data and control settings must be provided. This includes data that specifies or calculates the expiration of supplies, as well as data that indicates how demands determine if they can use a given supply based on its expiration.

The following illustration shows the main input and control tables for which data related to material expiry might typically need to be configured.

Figure 27.1 Main tables used in configuring material expiration



Specifying supply expiry

Expiry dates can be specified directly on on hand and scheduled receipt records. For these current supplies, part-level settings can also influence how the supply's expiry date is treated. The same part-level settings are also used to set how expiry dates are determined for planned supplies.

Setting a part's expiry rule

The main control setting used to enable expiry and determine how expiry dates are calculated for supplies is the **ExpiryRule** field. A version of this field is available on both the **PartType** table and the **PartSourceType** table.

The **PartType.ExpiryRule** field is used to enable expiry by part and provides the default expiry settings applicable to any part sources that provide the part. This field has the following six options that determine how supplies of a particular type expire:

- **Dependent**—supply **ExpiryDate** is based on the earliest of all expiry dates found across all component supplies used to satisfy dependent demand generated from the supply. Care should be taken to ensure that supplies for parts of this type will actually explode to some components;

otherwise the supply might never expire. This setting is typically used for expiring assembly parts.

- **DependentLimit—**supply **ExpiryDate** is based on the earliest of either it own calculated expiry date (**AvailableDate + PartSource.CTPIntervalsToExpiry**) or the earliest expiry date found on one of its component supplies.
- **DependentStart**—supply **ExpiryDate** is determined by adding the assembly's **PartSource.CTPIntervalsToExpiry** value to the earliest **ExpiryStartDate** rolled up from a component supply. By default, the **ExpiryStartDate** on component supplies will be their available date (subject to configuration), and this date can optionally be provided as input on **OnHand** and **ScheduledReeipt** records. The "DependentStart" setting is typically used for expiring assembly parts that contain "limiting components" (components where **ExpiryType.ProcessingRule** is set to "LimitingComponent" as discussed in "Define a part's expiry type" on page 1604).
- **Ignore**—supply for the part never expires, regardless of any other settings. For example, if an **ExpiryDate** is specified on a scheduled receipt record, but its part has an **ExpiryRule** of "Ignore", the receipt will never expire. All supplies for part of this type have their **ExpiryDate** reported as "Future". Thus, in order for supply from sources providing the part to expire, the part should reference a value other than "Ignore" in this field.
- **Self** —supply **ExpiryDate** is calculated starting from the supply's available date and then adding its **PartSource.CTPIntervalsToExpiry** value (expressed in expiry calendar intervals). This setting should typically be used for bottom-level components as well as for any assembly's whose expiration is not dependent on their components.
- **SelfReset**—supply **ExpiryDate** is calculated the same as for the "Self" option. However, this setting and "Self" differ in how they affect earliest expiry date calculations on dependent demands as discussed in "Earliest expiry for dependent demands" on page 1612.

The **PartSourceType.ExpiryRule** setting can be used to override or set the expiry rule at the individual part source level for any part that references a value other than "Ignore" in **PartType.ExpiryRule.** This setting contains all options found on the **PartType** field other than "Ignore". It also has a "FromPart" option (the default setting), which is used to indicate that the part source should calculate supply expiration as determined from the setting in **PartType.ExpiryRule**. Otherwise, if expiration should be calculated differently from the part's default for a given part source, then an option other than "FromPart" should be chosen.

For example, suppose a part's **Type.ExpiryRule** setting was "Self". Further, suppose the part had a "Make" source whose supplies should expire based on the part's own characteristics, and a "Transfer" source whose supplies should expire based on the characteristics of their components. In this case, the following expiry-related settings might be defined in the **PartSourceType** table.

Table 27.1: Sample ExpiryRule settings in PartSourceType

Value	ExpiryRule
Make	FromPart
Transfer	Dependent

Note also that when configuring part types for expiring parts, attention should be paid to settings in **Part.AvailableRule.ProcessingRule** (if populated) or **PartType.AvailableRule** (if the AvailableRule reference is not populated). Parts with a value of "Ignore" for that setting will have an available date calculated, however dependent requirements onto these parts are always treated as available at Past and independent requirements onto these parts are treated as available at DataDate. This may lead to unexpected impacts on expiry calculations.

For example, a component part with an **ExpiryRule** of "Self" and an **AvailableRule/ProcessingRule** of "Ignore" will still have an expiry date calculated on its supplies. However, if a parent of that component has an ExpiryRule of "Dependent", it will ignore the expiry date found on the component supply and, because the dependent requirement is assumed to be available at "Past", have its calculated expiry date set to "Future" and never expire (assuming no other component restricts the expiry of the parent supply).

Therefore it is recommended that expiring parts have their **AvailableRule/ProcessingRule** set to "Normal". For more information about this setting, see "AvailableRule" on page 811.

Defining a part's expiry calendar

The calendar used in the date calculations that set expiry dates on a part's supplies is determined by the part's **PlanningCalendars.ExpiryCalendar** value. Typically, this might be set to an "Everyday" calendar. The same calendar is also used when calculating the earliest expiry date on demands for the part.

Define a part's expiry type

All parts have a nullable reference to the **ExpiryType** table. This table has a **ProcessingRule** setting used to designate a part as either a limiting component or a normal component (any parts with a Null reference to **ExpiryType** are automatically treated as normal components). When a part is designated as a limiting component (has an **ExpiryType.ProcessingRule** setting of "LimitingComponent"), then expiry start dates are calculated on its supplies are rolled up the product structure for use in calculating the expiry date on assembly supplies if the assembly references a **PartType.ExpiryRule** setting of "DependentStart". For more information, see "DependentStart" on page 1619.

Defining expiry intervals on part sources

All part sources associated with a part that is subject to expiration should have the **PartSource**.**CTPIntervalsToExpiry** and **PartSource.IntervalsToExpiry** fields populated. These fields are used to indicate the duration of time that it takes supply from this part source to expire, and each should be expressed in intervals of the part's **PlanningCalendars.ExpiryCalendar**.

The **IntervalsToExpiry** field is used by the Netting analytic to calculate an estimated expiry date on supplies. The estimated expiry date is determined by adding the number of **ExpiryCalendar** intervals to the supply due date, and is used in determining if a given part source is eligible to supply a particular demand. Note that for parts where the **ExpiryRule** is set to "Dependent", this value should be set to the same value as that specified for the component of the part that has the smallest **IntervalsToExpiry** value (expires the fastest) minus any lead time difference between the two parts.

The **CTPIntervalsToExpiry** field is used by the Capable-to-Promise analytic to calculate the actual expiry date on supplies for expiring parts. Depending on the **ExpiryRule** setting, the **CTPIntervalsToExpiry** field is applied against different starting points to calculate the expiry date. For example, if the part has an **ExpiryRule** of "Self", then supplies from the part source expire a specified number of **ExpiryCalendar** intervals after their available date. However, if the **ExpiryRule** is "DependentStart", then supplies from the part source expire a specified number of **ExpiryCalendar** intervals after the earliest expiry start date rolled up from a component supply (typically, the component's available date).

Note that if the **CTPIntervalsToExpiry** field is set to 0 or a negative value, then the **IntervalsToExpiry** field value is used in both estimated expiry and actual expiry date calculations (this corresponds to how expiry calculations were always done in versions prior to 2016.2, Service Update 5). However, because these two dates can be calculated from different starting points based on configuration (for example, from when the assembly starts to expire versus when one of its components starts to expire), they should often be different if the estimated expiry date is meant to best approximate the expiry date.

Note: For more information about expiry date calculations, see "Calculating the expiry date on supplies" on page 1616.

Defining when supply expiration begins

The point from which supply using a given part source begins to expire is configurable in RapidResponse using the **PartSourceType.ExpiryDateRule** field. This field sets the date from which the **PartSource.CTPIntervalsToExpiry** value is added when calculating a supply expiry date. The **ExpiryDateRule** field has the following three options.

- **DueDate**—expiration is calculated starting from the supply available date (date on which the supply is in stock). This is the default value in RapidResponse.
- **DockDate**—expiration is calculated starting from the supply dock date (date on which the supply is at the receiving dock). **PartSource.DockToStockLeadTime** is included as part of the supply's expiry time.
- **StartDate**— expiration is calculated starting from the supply start date. All lead time components are included as part of the supply's expiry time, and caution should be taken with usage of this setting (it should be avoided if using variable lead time).

Notes:

- For more information about these options, see "ExpiryDateRule" on page 803.
- For more information about how the **ExpiryDateRule** field is used in expiry date calculations, see "Calculating the expiry date on supplies" on page 1616.

Providing an expiry date for on hand inventory

In order for on hand inventory to expire, a date must be provided in the **OnHand.ExpiryDate** field. This is an input field and the only way to indicate expiry on on hand records. If the on hand supply is assumed to never expire, then this field can be set to "Undefined" or "Future".

Note: If the part's **PartType.ExpiryRule** field is set to "Ignore", then the **ExpiryDate** on the on hand record is ignored.

Providing an expiry date for scheduled receipts

If the expiry date for a scheduled receipt is known, it can be provided in the **ScheduledReceipt.ExpiryDate** input field. This value is then reported in the **ScheduledReceipt.CalcExpiryDate** which represents the actual expiry date for the receipt. If the **ExpiryDate** field is set to "Undefined" or left blank, then the **CalcExpiryDate** is calculated based on the part's **PartType.ExpiryRule** setting as discussed earlier in this section.

The following table provides general guidelines on when to populate the **ExpiryDate** field for a scheduled receipt.

Table 27.2: Recommendations for populating ScheduledReceipt.ExpiryDate

If the PartType.ExpiryRule is...	Then...
Dependent / DependentLimit / DependentStart	• If the scheduled receipt has been released and exploded, set this field to the expected expiry date. • If the scheduled receipt has been released but not exploded, set this field to "Undefined' or leave it blank so that the expiry date is calculated based on component availability. • If the scheduled receipt has not been released, set this field to "Undefined" or leave it blank so that the expiry date is calculated based on component availability.
Ignore	• Any value provided in this field is ignored, as expiry calculations will be turned off for all of the part's supplies.
Self / SelfReset	• If the scheduled receipt has been released, set this field to the expected expiry date. • If the scheduled receipt has not been released, set this field to "Undefined" or leave blank so that the expiry date will be calculated based on the receipt's availability (for example, this might be its **AvailableDate**) and the part source's **CTPIntervalsToExpiry** value.

Notes:

- The **ExpiryDate** should also typically be set to "Undefined" or left blank for all reschedulable scheduled receipts (unless they are already exploded).
- A value of "Future" can be specified in the **ExpiryDate** field to indicate that a given scheduled receipt should never expire.

Specify when expiring safety stock is replenished

When supply of a part subject to expiration is held as safety stock, a configuration setting determines when new supply is planned to replenish expiring safety stock. This is set using the **SafetyStock.ExpiryRule** field which has the following two options:

- **ExpiryDate**—if safety stock supply will expire, a replenishment order is planned (due) at the supply's expiry date. This is the default setting and corresponds to how expiry logic worked prior to Version 2016.2 However, with this setting the safety stock supply can end up being held for some period of time during which, although it has not expired, it would also not be eligible to satisfy any demands due to minimum shelf-life requirements.
- **AtPartMSL**—if safety stock supply will expire, a replenishment order is planned (due) at the point when the supply (that is, at the supply's expiry date less the part's minimum shelf life). Note if using this setting, the part's minimum shelf life should typically be less than or equal to the minimum shelf life explicitly set on any of the part's independent demands (otherwise, safety stock supply might be unable to satisfy such demands).

Note: If safety stock settings are maintained using legacy fields on the **PartType** and **PlanningCalendars** tables (instead of **SafetyStockPolicy**), there is no field equivalent to **ExpiryRule** and supply to replenish expiring safety stock supply is always planned at the supply's expiry date. Therefore, if replenishment of expiring safety stock should consider minimum shelf life requirements, the part's safety stock settings must be set via the **SafetyStockPolicy** table.

Specifying how demands use expiring supply

When a part has expiring supplies, demands need to determine which of those supplies they can potentially use. This involves calculating a date before which supply must not expire if it is to be used in satisfying a given demand (the earliest expiry date).

Setting a part's expiry rule

Earliest expiry dates can be calculated on a part's demands as long its **PartType.ExpiryRule** is not set to "Ignore". That is, earliest expiry date calculations can be performed for a part as long as its **ExpiryRule** is set to "Self", "SelfReset", "Dependent", "DependentLimit", or "DependentStart". For information about how the **ExpiryRule** impacts earliest expiry date calculations, see "Calculating the earliest expiry date on a demand" on page 1611.

Defining a part's expiry calendar

The calendar used in the date calculations that set the earliest expiry dates on a part's demands is determined by the part's **PlanningCalendars.ExpiryCalendar** value. Typically, this might be set to an "Everyday" calendar. The same calendar is also used when calculating the expiry dates on supplies for the part.

Providing a part's minimum shelf life

The required shelf life for each part can be specified in the **Part.MinimumShelfLife** field. This field is, by default, used in determining the earliest expiry date on a part's demands (such as, independent demands and consensus forecast demands). The **MinimumShelfLife** value specifies a number of **ExpiryCalendar** intervals to add to the due date for this part's independent and consensus forecast demands in order to calculate the earliest expiry date of any supply that can be used to satisfy those demands.

For example, if a part's **PlanningCalendars.ExpiryCalendar** is set to "Month" and this value is set to "4", then an independent demand for the part with a DueDate of June 1, 2009 would have its **EarliestExpiryDate** field calculated as October 1, 2009.

A **MinimumShelfLife** of zero (0) can be specified to indicate that the earliest expiry date should be set exactly to the due date of the part's demand.

Providing a minimum shelf life on independent demands

If populated, the **IndependentDemand.MinimumShelfLife** overrides the part-level minimum shelf life value on independent demands. This field specifies the number of **Part.PlanningCalendars.ExpiryCalendar** intervals to add to the due date on a particular independent demand in order to calculate the earliest expiry date of any supply that can be used to satisfy that demand.

A **MinimumShelfLife** of zero (0) can be specified, along with an **ExpiryOffset** of zero (0), to indicate that the earliest expiry date should be set exactly to the due date of the demand. And a negative value (for example, -1) can be specified to indicate that the **EarliestExpiryDate** should be calculated based on the **Part**.**MinimumShelfLife** value instead.

Optionally, the **ExpiryOffset** field can be used to extend a given demand's minimum shelf life before exploding the demand to generate dependent demands on its components. This allows for the expiration requirements of all required components to be limited on a demand-by-demand basis (without affecting the expiry of those components when used in other demands). For more information about how this field is used, see "Demand-specific component expiry" on page 1621.

Note that when the calculated expiry dates are subsequently rolled up the product structure, the **ExpiryDate** on the **SupplyDemand** record reporting the supply allotment made to the top-level demand is then adjusted by the **ExpiryOffset** value. For more information, see "Demand-specific component expiry" on page 1621.

Providing a minimum shelf life for consensus forecast demands

A minimum shelf life value can be specified at the part-customer level for use in determining the earliest expiry date on **ConsensusForecast** demands generated for a part and customer combination (during the S&OP process).

If minimum shelf life requirements vary over time, then the **MinimumShelfLife** field on the **PartCustomerTimePhasedAttributes** table should be used. If a non-negative value is provided in this field, then it is used for that part and customer combination and within the effective date range specified on the record. However, if minimum shelf life requirements for a part and customer do not change over time, then the **MinimumShelfLife** field on the **PartCustomer** table should be used instead. If a non-negative value is provided in this field (and a negative value provided on the **PartCustomerTimePhasedAttributes** table), then the **PartCustomer** value is always used in determining the earliest expiry date.

Optionally, the **ExpiryOffset** field can also be used to extend the minimum shelf life for a given part-customers consensus forecast before exploding that forecasted demand to generate dependent demands on its components. This field is available on both the **PartCustomer** and **PartCustomerTimePhasedAttributes** tables and allows for the expiry requirements of all required components to be limited on a part-customer by part-customer basis (without affecting the expiry of those same components when used in demands for other part customers). For more information about how this field is used, see "Demand-specific component expiry" on page 1621.

Otherwise, if the **MinimumShelfLife** value is negative on both the **PartCustomer** and **PartCustomerTimePhasedAttribute** tables, then **Part.MinimumShelfLife** is used in determining the earliest expiry date on consensus forecast demands for the part and customer combination.

Note: For more information about how **MinimumShelfLife** fields are used in determining expiry requirements, see "Calculating the earliest expiry date on a demand" on page 1611.

Providing a minimum shelf life override by part source

Depending on the setting in the **PartType.ExpiryRule** field, the calculation of an earliest expiry date on dependent demands relies on the **MinimumShelfLife** from either the component itself or the assembly part that drove the demand, or it may be inherited from the driving demand. Planned orders then typically take their earliest expiry date directly from the dependent demand they are being created to satisfy (that is, the value in **DependentDemand.EarliestExpiryDate**)

However, the part's minimum shelf life can be overridden (extended) at the source level by specifying a positive value in **PartSource.MinimumShelfLifeThreshold.** This field might typically be populated for buy or transfer sources if they are expected top have longer shelf life requirements than those passed down from the item at the top of the product structure. In such cases, the **MinimumShelfLifeThreshold** is added to the order due date and if the resulting date is later than the

earliest expiry date from the dependent demand, it is used in **PlannedOrder.EarliestExpiryDate**. Otherwise, the earliest expiry date from the dependent demand is used instead.

Define calculation of earliest expiry on past-due demands

The **PartType.ExpiryPastDueRule** control field field can be used to specify whether the earliest expiry date calculation for past-due demands should start from the demand due date (as is done with all other demands) or if it should start from **Part.PlanningCalendars.DataDate.FirstDate** (typically, this corresponds to today's date and has the effect of starting the earliest expiry calculation from the current date rather than some point in the past).

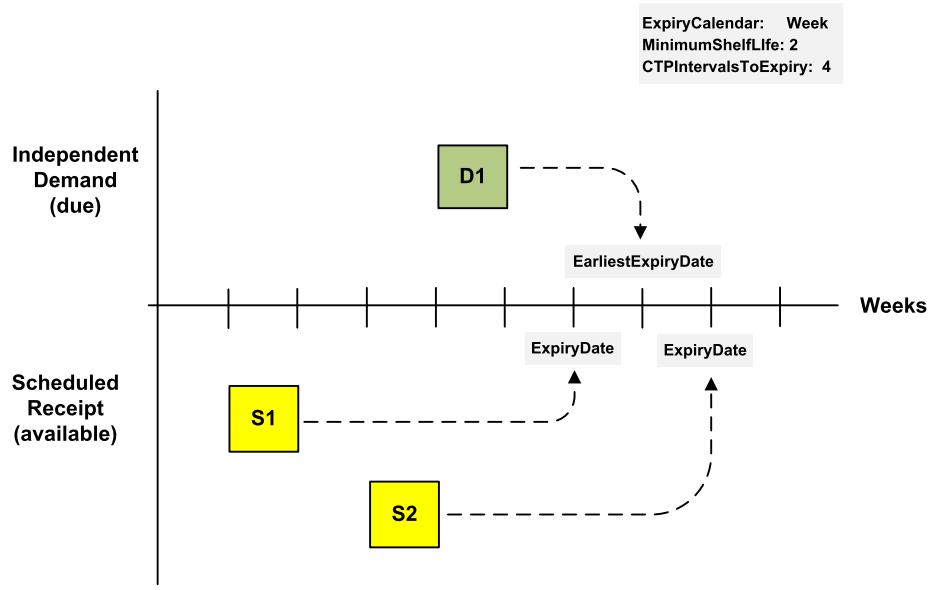
The **ExpiryPastDue** field has two settings; "DataDate" and "DueDate" (the default setting). For more information about how these settings are used, see "Earliest expiry calculation on past-due demands" on page 1614.

Expiry calculations

After material expiry data is set up in RapidResponse, the Netting and Capable-to-Produce analytics will calculate and consider expiry dates and quantities as appropriate. Specifically, expiry dates are calculated for supplies and earliest expiry dates are calculated on demands.

When supplies are allotted to demands, RapidResponse can then calculate and consider expiry dates and quantities as appropriate. This ensures that only those supplies that don't expire before a demands earliest expiry date are allotted to that demand. For example, in the following illustration supply S1 cannot be used to satisfied demand D1 because it expires too soon, however supply S2 can be used to satisfy the demand. RapidResponse also reports the expiring quantity on supplies to indicate how much of the supply will expire before it can be used by any demand.

Figure 27.2 Material expiry example

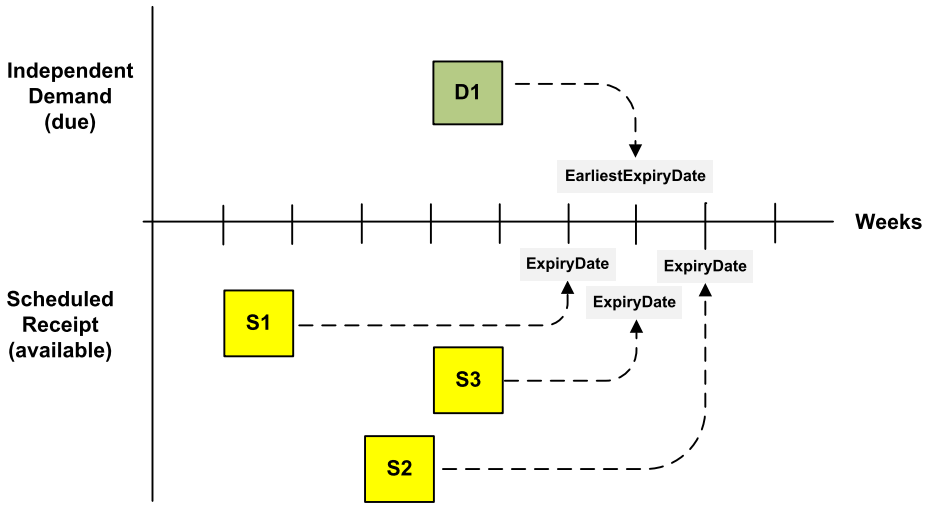


If more than one expiring supply is on-time and available to satisfy a demand for a part, RapidResponse can be configured to allocate those on-time supplies in order of expiry. This is done by ensuring the part's **PartType.SupplyAllocationRule** setting is "ByAvailable" (this is the default setting in RapidResponse). All expiring supplies whose available date is on or before the demand due date are then sorted by expiry date, and the earliest expiring supply (the one with the least amount of shelf life remaining) is allocated to the demand. In cases where the available supplies can potentially be used to satisfy multiple demands on-time, the demand with the earliest due date receives the earliest expiring supply. This functionality is useful for cases where a group of supplies are available in one sequence, but expire in a different sequence.

For example, in the following illustration, there is one demand, D1, and three supplies, S1, S2, and S3. In this example, all supplies are available before the demand due date, but S1 expires too soon to be allocated to D1. This leaves only S2 and S3 eligible to satisfy the demand. Assuming a SupplyAllocationRule of "ByAvailable", S3 would be allocated to the demand first because it expires sooner even though S2 is available sooner.

If multiple eligible supplies have the same expiry date, then available date is used as a secondary criteria in allocating the supplies. For example, in the above illustration, if S2 and S3 had the same expiry date, then S2 would be allocated first because it is available earlier. In situations where no supplies can be made to satisfy a demand on time, then the available date becomes the main criteria in allocating those supplies instead of the expiry date. Also, if the part's **PartType.SupplyAllocationRule** setting is "ByMRPPlan" (and not "ByAvailable"), then supplies are allocated based on the date of the demand they are initially allocated to during Netting calculations.

Figure 27.3 Multiple expiring supplies available to satisfy demand



Note: Expiry calculations are only performed for parts that have a **PartType.ExpiryRule** setting other than "Ignore" Any supply for a part where the ExpiryRule is "Ignore" will have its expiry date set to "Future", and any demand for a part where the ExpiryRule is "Ignore" will have its earliest expiry date set to "Past".

Calculating the earliest expiry date on a demand

Calculating the earliest expiry date on a demand

Earliest expiry dates are calculated for different types of demand in RapidResponse. These dates are used to specify a date before which any supply must not expire if it is to be used in satisfying a given demand. The **EarliestExpiryDate** calculation also depends on the type of demand that has expiry requirements. The following are discussed in this section:

- "Earliest expiry for independent demands" on page 1611
- "Earliest expiry for consensus forecast" on page 1611
- "Earliest expiry for dependent demands" on page 1612
- "Overriding the earliest expiry on dependent demands" on page 1614

Earliest expiry for independent demands

The **IndependentDemand.EarliestExpiryDate** field returns the earliest date that a supply can expire if it is to be used in satisfying the demand. This field is calculated by adding a specified number of calendar intervals to the demand due date. These calendar intervals are expressed in terms of the part's **ExpiryCalendar** and are set by the **MinimumShelfLife** on either the **IndependentDemand** record (if populated) or the **Part** record. If **MinimumShelfLife** is provided on the **IndependentDemand** record, then the earliest expiry date can be further extended before exploding the demand to dependent components using the **ExpiryOffset** field. This field allows the expiry requirements to be adjusted for all components under a specific end-item.

That is, if a non-negative value is found in **IndependentDemand.MinimumShelfLife**, then the **EarliestExpiryDate** is calculated as:

```
IndependentDemand.DueDate + (MinimumShelfLife + ExpiryOffset)
Part.PlanningCalendars.ExpiryCalendar
```

Otherwise, the **EarliestExpiryDate** is calculated as:

```
IndependentDemand.DueDate + Part.MinimumShelfLife
Part.PlanningCalendars.ExpiryCalendar
```

Note: A value of 0 (zero) can be specified in the **MinimumShelfLife** field, along with a value of 0 (zero) in the **ExpiryOffset** field, to indicate the **EarliestExpiryDate** should be set to exactly the demand **DueDate**.

Earliest expiry for consensus forecast

The **ConsenusForecast.EarliestExpiryDate** field returns the earliest date that a supply can expire if it is to be used in satisfying the forecast. This field is calculated by adding a specified number of calendar intervals to the demand date. These calendar intervals are expressed in terms of the part's **ExpiryCalendar** and are set by the **MinimumShelfLife** on either an effective **PartCustomerTimePhasedAttributes** record (if one exists) or the **PartCustomer** record (if defined), or else the **Part** record.

If **MinimumShelfLife** is provided on a **PartCustomerTimePhasedAttributes** or **PartCustomer** record, then the earliest expiry date can be further extended before exploding the demand to dependent

components using the **ExpiryOffset** field on these tables. This field allows the expiry requirements to be adjusted for all dependent components under a specific part-customer's demands.

Earliest expiry for dependent demands

An **EarliestExpiryDate** field also appears on all records in the **DependentDemand** table. For dependent demands, the calculation of this date is determined by the **PartType.ExpiryRule** setting on the immediate parent part that drove the demand as follows.

- **Dependent / DependentLimit**—if the assembly uses either of these settings, then the **EarliestExpiryDate** on its dependent demands are directly inherited from the **EarliestExpiryDat**e found on the parent supply that drove the dependencies. This setting should be used on assembly's who have components that impact the overall expiry of the assembly itself.
- **Self**—if the assembly uses this setting, then the **EarliestExpiryDate** on each of its dependent demands is calculated starting from **DependentDemand.DueDate** and then adding a number of calendar intervals as defined by the assembly part's **MinimumShelfLife** and **PlanningCalendars.ExpiryCalendar** values. This setting should be used on assembly's whose immediate component supplies should satisfy the expiry requirements of the assembly part that drove the dependent demand (in other words, the component's impact the overall expiry of the assembly).
- **SelfReset / DependentStart**—if the assembly uses either of these settings, then the **EarliestExpiryDate** on its dependent demands is calculated starting from **DependentDemand.DueDate** and then adding a number of calendar intervals as defined by the component part's **MinimumShelfLife** and **PlanningCalendars.ExpiryCalendar** values. This setting should be used on assembly's whose immediate component supplies need only to satisfy their own expiry requirements (in other words, where the components do not impact the overall expiry of the parent assembly itself and only need not to last enough to be assembled into the parent).

  Usage of the SelfReset option can prevent situations where eligible component supplies would otherwise be seen as expiring too soon to be used in the assembly (particularly for components with short shelf lives). See the next section for an example of this type of situation.

Note: In cases where a dependent demand is created that does not originate from an independent demand driver, the **EarliestExpiryDate** on the dependent demand is derived from the parent supply (that is not allocated to a demand). This is calculated as:

```
PeggedSupply.DueDate +
PeggedPart.MinimumShelfLifePeggedPart.PlanningCalendars.ExpiryCalendar
```

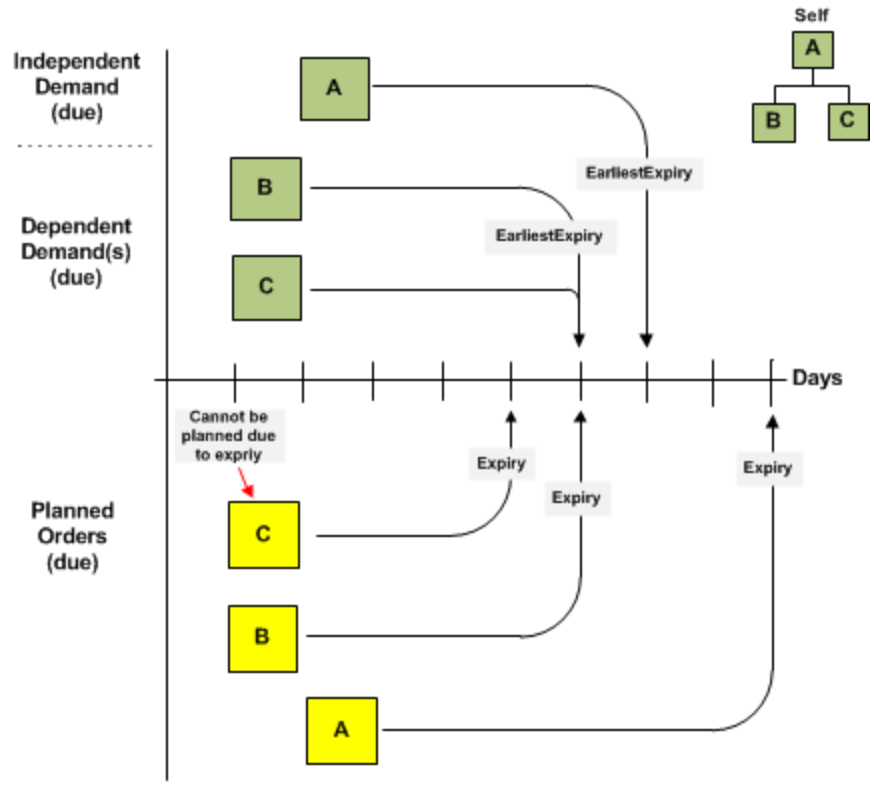
Self vs SelfReset example

Suppose an assembly part "A" has two components "B" and "C", and each part has a lead time of 1 day. Further suppose each part's **ExpiryCalendar** value is set to the "Everyday" calendar and the following set of expiry parameters are provided:

Part	IntervalsToExpiry	MinimumShelfLife
A	6	4
B	4	3
C	3	2

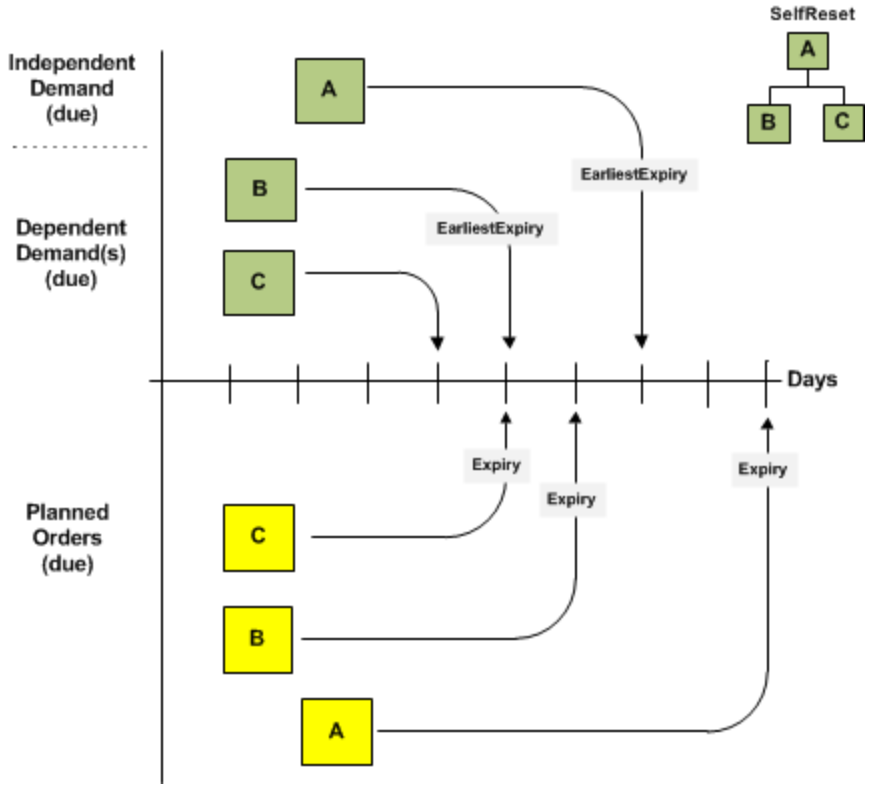
If part A has a **Type.ExpiryRule** value of "Self', then the earliest expiry date on the dependent demands placed on components B and C would be based on part A's minimum shelf life (4 days). In this case, new supply of B can be planned to satisfy dependent demands generated from demand on A, however new supply of C cannot. This is because its **EarliestExpiryDate**, calculated based on the **MinimumShelfLife** of the assembly, will always be later than the date on which new planned supply would expire (supply of C expires in 3 days, but is required to last for at least 4 days based on the **MinimumShelfLife** defined on assembly A).

Figure 27.4 The impact of the "Self" option



Instead if part A has a **Type.ExpiryRule** value of "SelfReset", then the earliest expiry date on the dependent demands placed on components B and C would be based on their own minimum shelf life values (3 and 2 days, respectively). In this case, new supply of C can now be planned to satisfy dependent demands generated from demand on A. This is because its supplies are still seen as expiring within 3 days but they now need only to last for 2 days (based on the component's own expiry requirements rather than those of the assembly). And as before, new supply of component B can still be planned to satisfy dependent demands from A (with their **EarliestExpiryDate** now calculated as being a day earlier than it was).

Figure 27.5 The impact of the "SelfReset" option



Overriding the earliest expiry on dependent demands

As described in the previous section, the **EarliestExpiryDate** on a dependent demand is calculated to satisfy either its own expiry requirements, or those of the assembly supply it satisfies, based on the applicable **PartType.ExpiryRule** setting. In either case, when planned orders are generated to satisfy those dependent demands, they typically inherit the **EarliestExpiryDate** from the dependent demand.

However, if the **PartSource** used to generate the planned order has a positive value in the **MinimumShelfLifeThreshold** field, then that number of expiry calendars intervals is added to the planned order due date. If the resulting date is later than the **EarliestExpiryDate** on the **DependentDemand** being satisfied, then it is used in **PlannedOrder.EarliestExpiryDate**.Otherwise, the **EarliestExpiryDate** from the **DependentDemand** record is used.

Earliest expiry calculation on past-due demands

As discussed earlier in this section, the earliest expiry date on demands is typically calculated by adding the applicable **MinimumShelfLife** value to the demand's **DueDate**. However, in case where demands are seen as past-due (have a **DueDate** earlier than **Part.PlanningCalendars.DataDate.FirstDate**), it may not be desirable to start earliest expiry calculations from the due date. This is because the due date on these demands represents a point in the past and therefore will result in an earliest expiry date that is earlier than it otherwise should be. As a result, such demands might end up being allotted supplies that would otherwise be seen as expiring too soon to be used in satisfying them.

For control over the calculation of earliest expiry dates on past-due demands, the **PartType.ExpiryPastDueRule** setting can be used. This control field applies to all past-due independent

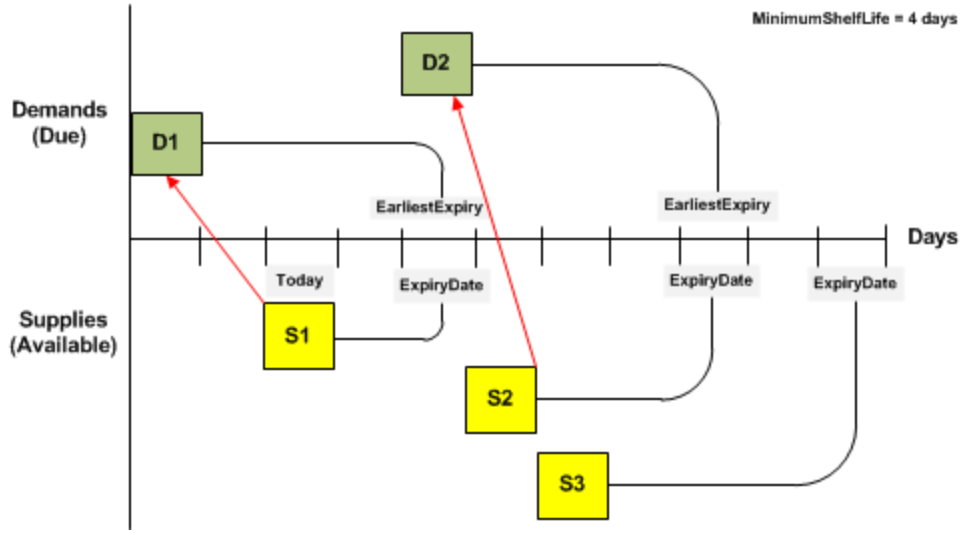
demands, as well as past-due dependent demands for parts that reference an **ExpiryRule** of either "Self", "SelfReset", or "DependentStart". This setting has the following two options:

- **DataDate**— Past-due demands have their earliest expiry dates calculated starting from **Part.PlanningCalendars.DataDate.FirstDate** (typically, this corresponds to today's date).
- **DueDate**—Past due demands have their earliest expiry date calculated starting from their due date. This is the default setting in RapidResponse (and corresponds to how past-due demands were always handled in versions prior to 2015.3).

Thus the supplies which get allotted to a given past-due demand, and hence to other subsequent demands, may differ depending on which of the **ExpiryPastDueRule** settings is used.

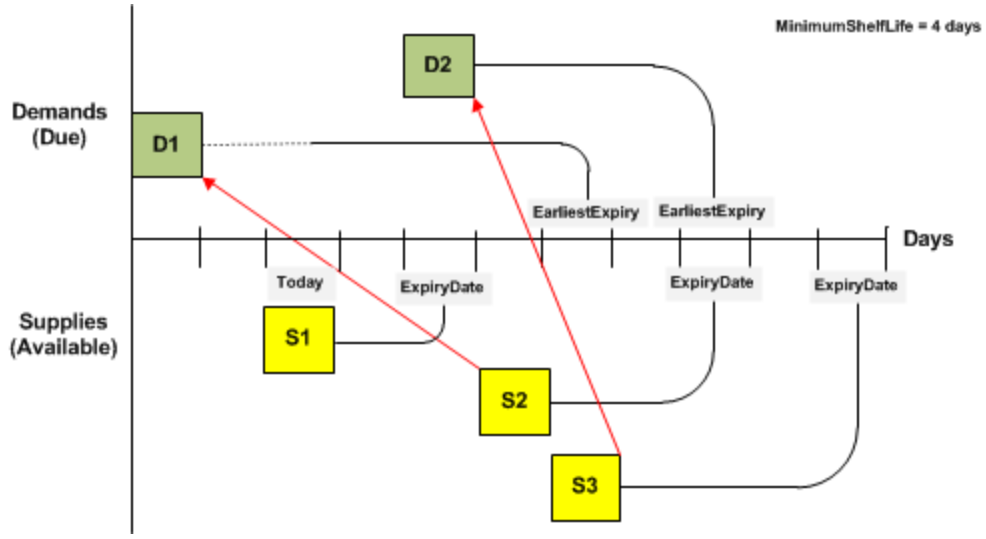
For example, suppose a part with a **MinimumShelfLife** of four (4) days, an **ExpiryPastDueRule** of "DueDate", and two independent demands, with one of these demands being past-due, as shown in the following illustration. In this case, the past-due demand of D1 is given the supply S1 which is available at **DataDate** (today), and the current demand D2 is allotted the next available supply S2. Note in this case, the supply allotted to the past-due demand actually expires two days after it is used to satisfy that demand itself despite the demand having a minimum shelf life requirement of four days.

Figure 27.6 Earliest expiry date calculated based on DueDate



Instead assume the **ExpiryPastDueRule** were set to "DateDate" as shown in the following illustration. In this case, the past-due demand D1 would have its earliest expiry date calculated starting from **DataDate** (today), which would make supply S1 ineligible to satisfy this demand based on its expiring too soon. Therefore, the demand D1 would be instead be satisfied by supply S2, and the subsequent demand D2 would now be satisfied by supply S3.

Figure 27.7 Earliest expiry date calculated based on DataDate



Calculating the expiry date on supplies

Expiry dates are calculated for different types of supply in RapidResponse. In order for a supply to be used in satisfying a demand, its **ExpiryDate** must be later than or equal to the **EarliestExpiryDate** specified on the demand. The **ExpiryDate** calculation also depends on the type of supply that is expiring. The following are discussed in this section:

- "OnHand expiry" on page 1616
- "Inventory transfer expiry" on page 1616
- "Scheduled receipt expiry" on page 1617
- "Planned order expiry" on page 1618
- "Demand-specific component expiry" on page 1621
- "Example: ExpiryDate calculations (Self vs Dependent vs DependentStart)" on page 1621

OnHand expiry

The expiry date of onhand inventory is reported in the **OnHand.ExpiryDate** field. This is an input field, and if it is not populated then the supply represented by the OnHand record is assumed to never expire.

Inventory transfer expiry

The expiry date of inventory transfers is reported in the calculated **InventoryTransfer.ExpiryDate** field. This field uses the earliest **ExpiryDate** found on any OnHand record used to supply the transfer.

Scheduled receipt expiry

Two expiry dates are calculated on scheduled receipts, an estimated expiry date, and an actual expiry date. The estimated expiry date is calculated by the Netting analytic and the actual expiry date is calculated by the Capable-to-Promise analytic.

The estimated expiry date calculated on each scheduled receipt is used to determine which demands the receipt can be used to satisfy. In order for a scheduled receipt to be used to cover a given demand in Netting logic, its calculated **EstimatedExpiryDate** field value must be later than or on the **EarliestExpiryDate** of the demand.

If a value is provided in the **ScheduledReceipt.ExpiryDate** field, it is subsequently used in **ScheduledReceipt**.**EstimatedExpiryDate**. Otherwise, the estimated expiry date on a scheduled receipt is calculated by adding some number of expiry calendar intervals (as specified on the part source) to the date on which the receipt begins to expire. Estimated supply expiration begins on either its effective due date, dock date, or start date as defined by the **PartSourceType.ExpiryDateRule** setting. For example, with the default **ExpiryDateRule** setting of "DueDate", the following calculation sets the estimated expiry date on scheduled receipts:

```
ScheduledRecipt.EffDueDate + PartSource.IntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

Note that because the **EstimatedExpiryDate** value calculated by the Netting analytic does not take supply availabilities into account, it might be different than the actual **ExpiryDate** subsequently calculated by Capable-To-Promise analytic which does consider availability. The actual expiry date of scheduled receipts is subsequently calculated and reported in the **ScheduledReceipt**.**CalcExpiryDate** field. If the **ScheduledReceipt.ExpiryDate** input field is populated with a date, then that date is automatically reported in the **CalcExpiryDate** field as well.

Otherwise, the **CalcExpiryDate** field is calculated in one of four ways depending on the part's **PartType.ExpiryRule** setting as follows.

Dependent

For parts where the **ExpiryRule** is "Dependent", the **CalcExpiryDate** is set to the earliest of any calculated expiry date found on any component supply used to satisfy dependent demand from the scheduled receipt.

Note: If the part using this setting does not actually explode to any components, then it will not expire and its **CalcExpiryDate** will be set to "Future" (this condition also applies to part's using the "DependentLimit" and "DependentStart" options described below).

DependentLimit

For parts where the **ExpiryRule** is "DependentLimit", the **CalcExpiryDate** for an assembly supply is set to the earliest of either any expiry date found on a component supply that was used in the assembly, or the receipt's own calculated expiry as determined from:

```
ScheduledReceipt.AvailableDate + PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

DependentStart

For assemblies where the **ExpiryRule** is "DependentStart", the **CalcExpiryDate** for a scheduled receipt is calculated as :

```
ScheduledReceipt.CalcExpiryStartDate + PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

This assumes that at least some of the assembly's components, typically those at the bottom of the product structure, reference an **ExpiryType.ProcessingRule** setting of "LimitingComponent". When a component is designated as a "LimitingComponent", it's supplies have an **ExpiryStartDate** calculated. Those **ExpiryStartDate** values are set to either the component supply's effective due date (default), dock date, or start date as defined by the **PartType.ExpiryDateRule** setting. For on hand inventory and scheduled receipts, an **ExpiryStartDate** can also (optionally) be provided as input.

Once calculated or set on lower-level supplies, the **ExpiryStartDate** is then rolled up the product structure, through any normal components, until ultimately reaching the assembly supply. The **ExpiryStartDate** on the scheduled receipt is then set to the earliest of all **ExpiryStartDate** values rolled up from its component supplies, and this date is used as the point from which the receipt's expiration start. In other words, the assembly's intervals to expiry is then added to the earliest rolled up expiry start date from a component supply.

Note: If all of the assembly's components use an **ExpiryType.ProcessingRule** setting of "Normal" (there are no limiting components/no expiry start dates are provided), then the receipt's **CalcExpiryDate** is calculated starting from its own available date rather than the rolled up expiry start date.

Self / SelfReset

For parts where the **ExpiryRule** is "Self" or "SelfReset", the **CalcExpiryDate** is set by adding some number of expiry calendar intervals (as specified on the part source) to the date on which the receipt begins to expire. Supply expiration begins on either its available date, dock date, or start date as defined by the applicable **PartSourceType.ExpiryDateRule** setting. For example, with the default **ExpiryDateRule** setting of "DueDate", the following sets the actual expiry date on scheduled receipts:

```
ScheduledReceipt.AvailableDate + PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

Note: If the **PartSource.CTPIntervalsToExpiry** field is set to 0 or a negative value, then the **PartSource.IntervalsToExpiry** value is used in all expiry date calculations instead.

Planned order expiry

Two expiry dates are calculated for planned orders, an estimated expiry date, and an actual expiry date. The estimated expiry date is calculated by the Netting analytic, and the actual expiry date is calculated by the Capable-to-Promise analytic.

During the creation of planned orders, RapidResponse determines an estimated expiry date for each planned order by adding some number of expiry calendar intervals (as specified on the part source) to the date on which the order begins to expire. This value is reported in **PlannedOrder.EstimatedExpiryDate**. The estimated supply expiration begins on either its effective due

date, dock date, or start date as defined by the **PartType.ExpiryDateRule** setting. For example, with the default setting of "DueDate", the following calculation populates the **EstimatedExpiryDate** on planned orders:

```
PlannedOrder.DueDate + PartSource.IntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

For a given part source, a planned order is then only generated if this estimated expiry date, minus the value in (**Part.SafetyLeadTime + PartSource.SafetyLeadTime)**, is equal to or later than the **EarliestExpiryDate** of the demand that triggered it. Note that because the **EstimatedExpiryDate** value calculated by the Netting analytic does not take supply availabilities into account, it might be different than the actual **ExpiryDate** subsequently calculated by the Capable-To-Promise analytic which does consider availability.

For any planned orders that are generated, RapidResponse subsequently calculates the actual expiry date. This value is reported in the **CTPPlannedOrder.ExpiryDate** field, and its calculation is made in one of four different ways depending on the setting in the **PartType.ExpiryRule** field as follows.

Note: If all of the assembly's components use an **ExpiryType.ProcessingRule** setting of "Normal" (there are no limiting components/no expiry start dates are provided), then the receipt's **CalcExpiryDate** is calculated starting from its own available date rather than the rolled up expiry start date.

Dependent

If a part's **ExpiryRule** is set to "Dependent", then the **ExpiryDate** is set to the earliest expiry date found on any component supply used to satisfy dependent demand from the planned order.

DependentLimit

For assemblies where the **ExpiryRule** is "DependentLimit", the **ExpiryDate** on a **CTPPlannedOrder** record is set to the earliest of either any expiry date found on a component supply that was usebd in the assembly, or the planned order's own calculated expiry as determined from:

```
CTPPlannedOrder.AvailableDate + PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

DependentStart

For assemblies where the **ExpiryRule** is "DependentStart", the **CalcExpiryDate** for a planned order is calculated as:

```
CTPPlannedOrder.ExpiryStartDate +
PlannedOrder.PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

This assumes that at least some of the assembly's components, typically those at the bottom of the product structure, reference an **ExpiryType.ProcessingRule** setting of "LimitingComponent". When a lower-level component is designated as a "LimitingComponent", it's supplies have an **ExpiryStartDate** calculated. Those **ExpiryStartDate** values are set to the component supply's effective due date, dock date, or start date as defined by the **PartType.ExpiryDateRule** setting. However, when a lower-level component has **ExpiryType.ProcessingRule** set to "Normal", then its supplies have their **ExpiryStartDate** value set to "Future".

Once calculated on lower-level supplies, the **ExpiryStartDate** is then rolled up the product structure, through any normal components, until ultimately reaching the assembly supply. The planned order's **ExpiryStartDate** is then set to the earliest **ExpiryStartDate** values rolled up from its component supplies, and this date is the point from which the planned order's expiration starts. That is, the assembly's intervals to expiry is added to the earliest rolled up expiry start date from a component supply.

Note also that the **PartType.ExpiryRule** setting further impacts the **ExpiryStartDate** values rolled up through the product structure. The following table shows how different combinations of the **PartType.ExpiryRule** and **ExpiryType.ProcessingRule** settings affect the rolled up **ExpiryStartDate** values for different types of supplies.

If ExpiryRule is...	Self/SelfReset		Dependent/DependentLimit/DependentStart	
And ProcessingRule is..	Normal	LimitingComponent	Normal	LimitingComponent
PlannedOrder	Future	AvailableDate	Earliest rolled up expiry start date	Earliest of AvailableDate and rolled up expiry start dates
ScheduleReceipt	CalcExpiryStartDate	CalcExpiryStartDate	Earliest of CalcExpiryStartDate and rolled up expiry start dates	Earliest of CalcExpiryStartDate and rolled up expiry start dates
OnHand	ExpiryStartDate	ExpiryStartDate	ExpiryStartDate	ExpiryStartDate
InventoryTransfer	ExpiryStartDate (or else the primary part's expiry start date	ExpiryStartDate (or else the primary part's expiry start date	ExpiryStartDate (or else the primary part's expiry start date	ExpiryStartDate (or else the primary part's expiry start date
SubstituteSupply or CoByProduct	Primary part's expiry start date	Primary part's expiry start date	Primary part's expiry start date	Primary part's expiry start date

Note: If all components in the assembly's structure use an **ExpiryType.ProcessingRule** setting of "Normal" (there are no limiting components and no expiry start dates are provided), then the order's **ExpiryDate** is calculated starting from its own **AvailableDate** rather than the rolled up expiry start date of "Future".

Self / SelfReset

If the **ExpiryRule** is set to "Self" or "SelfReset", then the **ExpiryDate** is calculated by adding some number of expiry calendar intervals (as specified on the part source) to the date on which the order begins to expire. Supply expiration begins on either its available date, dock date, or start date as defined by the **PartType.ExpiryDateRule** setting. For example, with the default setting of "DueDate", the following calculation sets the actual expiry date on planned orders:

```
CTPPlannedOrder.AvailableDate + PlannedOrder.PartSource.CTPIntervalsToExpiry
Part.PlanningCalendars.ExpiryCalendar
```

Note: If the **PartSource.CTPIntervalsToExpiry** field is set to 0 or a negative value, then the **PartSource.IntervalsToExpiry** value is used in all expiry date calculations instead.

Demand-specific component expiry

In some cases, the expiration requirements of components used in satisfying end-item demands may vary depending on the specific independent or consensus forecast demand in which they are being used. However, the expiration of the component supplies are always dependent on the **PartSource.CTPIntervalsToExpiry** value which has no correlation to the driving demand.

In cases where component supply expiry requirements should be adjusted (reduced) when used in a particular end-item demand, the **ExpiryOffset** field on the **IndependentDemand** table should be used. Similarly, in cases where component supply expiry requirements should be adjusted when used in a particular part customer's consensus forecast demands, the **ExpiryOffset** field on either the **PartCustomer** or **PartCustomerTimePhasedAttributes** tables should be used.

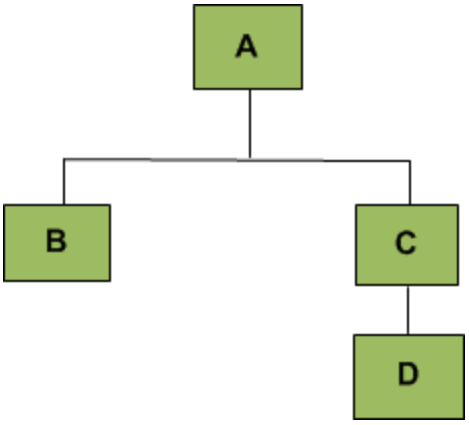
When a positive value is specified in the **ExpiryOffset** field, that number of expiry intervals is added to the **MinimumShelfLife** on the end-item demand in order to extend its calculated **EarliestExpiryDate** before exploding the demand down to any dependent components. Thus, depending on the applicable **ExpiryRule** setting, all dependent demands placed on components under the end-item will reflect the extended earliest expiry date and only supplies that can meet these limited expiry requirements will be used.

Subsequently, when calculated supply available and/or expiry dates are rolled up the product structure, the **ExpiryOffset** value is used when reaching the specific allotment of supply used to satisfy the top-level demand. In other words, the **ExpiryDate** on the **SupplyDemand** record reporting an allotment of supply to the top-level demand gets adjusted (reduced by) the value specified in the applicable **ExpiryOffset** field. This adjustment is made in order to reflect the shortened expiry requirements on the lower-level components when used towards satisfying the specific demand. Note however that while the top-level allotment of supply to demand is adjusted to account for the limited component supply expiry, the actual **ExpiryDate** values calculated on the lower-level supplies themselves are not affected.

Example: ExpiryDate calculations (Self vs Dependent vs DependentStart)

Suppose a product structure under assembly A as shown in the following illustration.

Figure 27.8 BillOfMaterial for assembly subject to expiration



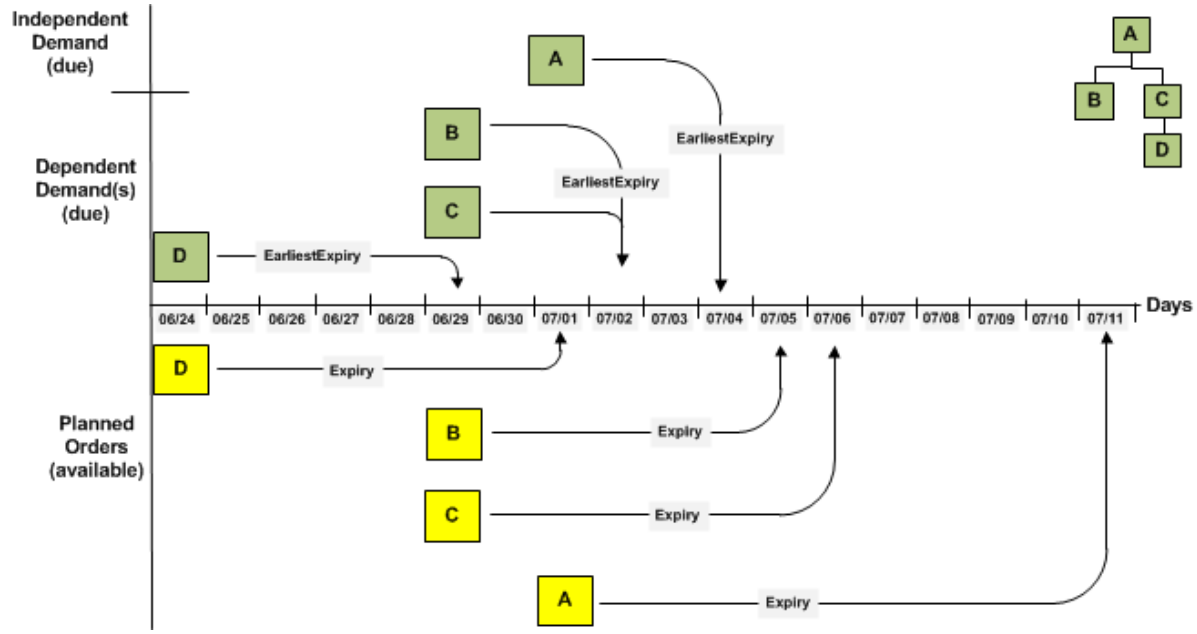
Further, suppose the following expiry-related input and control settings for each of the parts found in the product structure.

Table 27.3: Expiry-related settings

Settings / Part	A	B	C	D
PartSource. EffLeadTime	2	1	3	1
PartSource. IntervalsToExpiry	10	6	7	7
PartSource. CTPIntervalsToExpiry	10	6	7	7
Part. MinimumShelfLife	3	5	3	4
PartType. ExpiryRule	Self	Self	Self	Self
PartType. AvailableRule	Due	Due	Due	Due
ExpiryType. ProcessingRule	Normal	Limiting Component	Normal	Limiting Component
PlanningCalendars. ExpiryCalendar	Everyday	Everyday	Everyday	Everyday
PlanningCalendars. TimeUnits	Everyday	Everyday	Everyday	Everyday

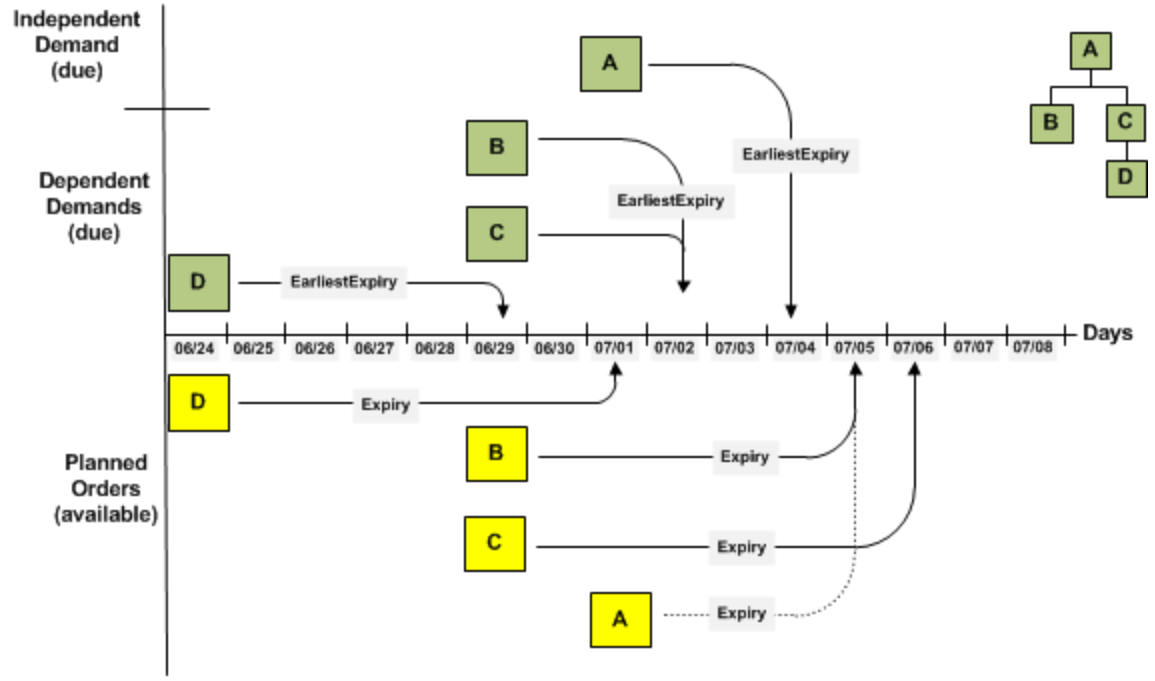
Assume a demand for Assembly A due on July 01 with no existing supplies for A or any of the components in its product structure. This leads to the set of dependent demands and planned orders created to satisfy them, along with calculated EarliestExpiryDate and ExpiryDate values, as shown in the following illustration. In this scenario, because top-level assembly A's **ExpiryRule** set to "Self", its **ExpiryDate** is calculated based on only own its properties. That is, planned order expiration is determined starting from its **AvailableDate** (July 01) and then adding **PartSource.IntervalsToExpiry** (10 days) to give a calculated **ExpiryDate** of July 11.

Figure 27.9 ExpiryDate calculations for "Self" assembly A



Instead if A's **ExpiryRule** was set to "Dependent", then its **ExpiryDate** would be calculated as shown in the following illustration. That is, the **ExpiryDate** for A is set to the earliest **ExpiryDate** on any supply used to satisfy a dependent demand generated from the requirement for A. In this case, the **ExpiryDate** of July 5 is inherited from the planned order created for component B.

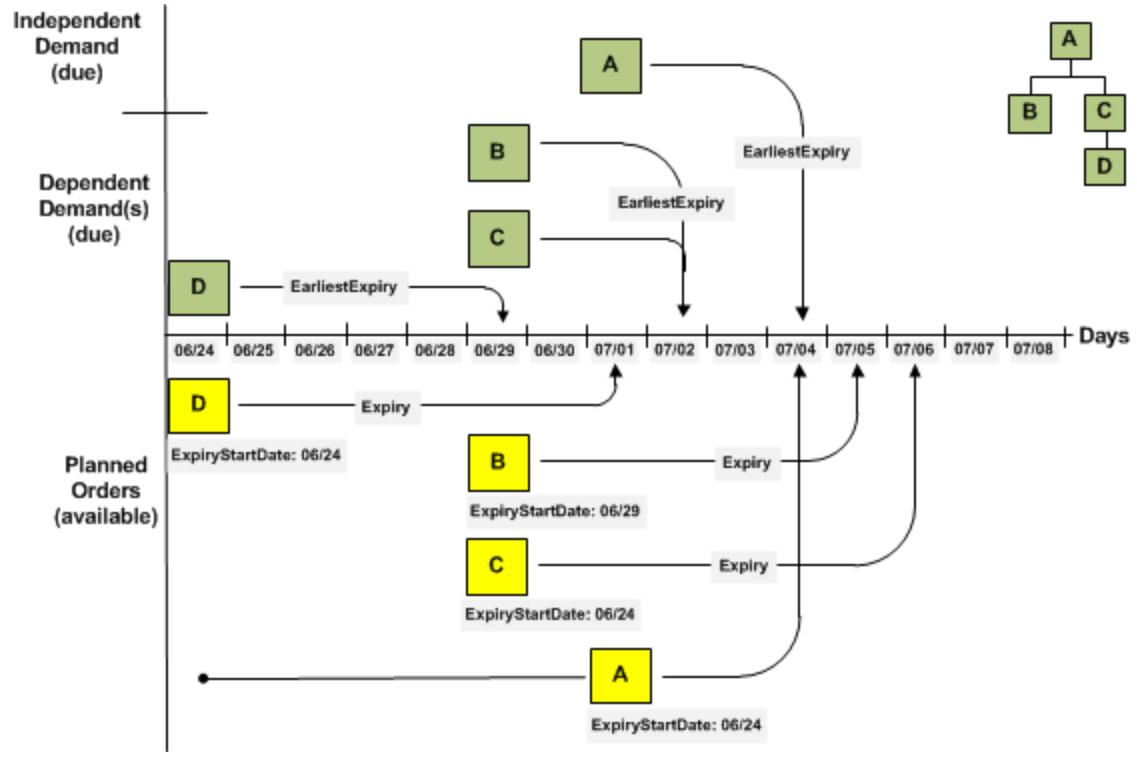
Figure 27.10 ExpiryDate calculations for "Dependent" assembly A



And finally if A's **ExpiryRule** was instead set to "DependentStart", then the **ExpiryDate** on its supply would be calculated by adding its **CTPIntervalsToExpiry** value (10 days) to the earliest of any

ExpiryStartDate value rolled up from a limiting component (in this case, the June 24 value rolled up from component D). Thus, the **ExpiryDate** for A's supply is determined to be July 04 as shown in the following illustration (June 24 + 10). Note also that in this case if the **ExpiryDate** on normal component C had been earlier than July 04, than it would have been used as A's **ExpiryDate** instead.

Figure 27.11 ExpiryDate calculations for "DependentStart" assembly A



Calculating the expiring quantity on a supply

Any amount of a supply which reaches it expiry date without being used to satisfy a demand is reported as an expiring quantity. For example, if a scheduled receipt for 2000 units has an expiry date of August 1, and only 1600 units of this supply can be consumed by eligible demands before August 1, then an expiring quantity of 400 is reported to show the portion of the receipt that will expire and become unusable supply. A calculated **ExpiringQuantity** field is reported on supplies in the following tables:

- CTPPlannedOrder
- CTPSubstituteSupply
- InventoryTransfer
- OnHand
- ScheduledReceipt

Expiring quantities and the CTPActivity table

Supplies with expiring quantities are treated specially in the CTPActivity table. The expiring quantity of a supply is not reported on the original record of the supply in the CTPActivity table. Instead, a second record for the supply is added with its ExtendedType field set to "Expiring" to indicate the record

represents only the expiring portion of the supply. The original record for the supply will have a value of "Normal" in the ExtendedType field.

In addition to the "Expiring" value in the **ExtendedType** field, the expiring portion of a supply has the following CTPActivity fields set differently than the original record of the supply:

- **ATPQuantity**—always set to zero (0).
- **BalanceDelta**—shows the expiring quantity represented as a negative value.
- **DemandQty**— shows the expiring quantity.
- **NewProcessCost—**always set to zero (0).
- **NewPurchasesCost**—always set to zero (0).
- **SupplyQty**—always set to zero (0).

Expiring quantities and the SupplyDemand table

Supplies with expiring quantities are also treated specially in the SupplyDemand table. Records in this table that represent the expiring portion of a supply have a DemandSource value of "Expired" and the expiring quantity is reported in the AllocatedQty field.

Note that records in the SupplyDemand table are only set as "Expired" if RapidResponse has ever attempted to use the supply in satisfying a demand but was unable to because of its expiry date. If an expiring supply was never tested for use in satisfying a demand, then its DemandSource is set to "None" and the supply is considered to be excess.

Unsourceable demands due to material expiry

As discussed previously in this chapter, when RapidResponse attempts to generate a planned order to satisfy demand for an expiring part, the estimated expiry date on the planned order must be on or after the earliest expiry date on the demand it is meant to satisfy. In some cases, however, RapidResponse might be unable to generate a planned order that can meet these expiry requirements.

This might happen, for example, if the effective MinimumShelfLife value on the demand is greater than the IntervalsToExpiry value defined on the part source(s) that can provide the supply. Because the MinimumShelfLife value is added to the demand due date to determine its earliest expiry date, and the IntervalsToExpiry value is added to the supply due date (depending on its **PartSourceType.ExpiryDateRule** setting) to determine its estimated expiry date, such a configuration would ensure supply that always expires too soon to satisfy the demand. As a result, the demand would be left unsatisfied.

Also, when demand is due at past for an expiring part (for example, from negative OnHand records or coming through a backflush scheduled receipt), this demand can only be satisfied by on hand inventory or scheduled receipts that are due at past. Thus, if a demand due at past required a planned order to provide supply, that demand, too, would be left unsatisfied.

Whenever RapidResponse encounters demand which cannot be sourced due to expiry requirements, it skips over the demand, and continues processing other demands. In order to help identify these skipped demands, the **DemandException** table can be used. This table reports all demands that were skipped and could not be satisfied due to their expiry requirements, along with details such as the amount of the demand that can't be sourced and the date it is required. For more information, see "DemandException table" on page 1028.

Unsourceable demands and the Activity/CTPActivity tables

When a demand cannot be sourced due to expiry limitations, records are also added to the **Activity** and **CTPActivity** tables to capture these details. These records are identified by an **ExtendedType** value of **DemandException**, They report details on the demand that could not be sourced such as the amount of the demand and its earliest expiry date, and also provide links to the corresponding record in the **DemandException** table.

The following table described the fields which are populated on Activity or CTPActivity records with an **ExtendedType** value of **DemandException**. All other base fields on these records are set to their data type's default, and all other reference fields on these records are left empty.

Table 27.4: Fields populated on Activity / CTPActivity records for unsourceable demands

Field	Description
AlternatePart	Reference to the primary part.
AssociatedDate	Date associated with the demand that cannot be sourced (typically, the due date).
DemandException	A reference to the associated DemandException record.
DemandQty	The amount of the demand that cannot be sourced.
DueDate	Date associated with the demand that could not be sourced (typically, the due date).
EarliestExpiryDate	Date before which supply cannot expire
ExtendedType	Returns a value of "DemandException" to indicate the record represents an unsourceable demand.
Model	The model, if any, associated with the demand that cannot be sourced.
OrderPriority	The priority associated with the demand that cannot be sourced.
Part	The part whose demand could not be sourced due to expiry requirements.
Pool	The pool, if any, associated with the demand that cannot be sourced.
StartUnit	The start unit associated with the demand that cannot be sourced (defaults to -1).
Type	The type of demand that cannot be sourced (for example, IndependentDemand).

Limitations on other analytics

When using material expiry, a small number of other analytics or calculations might be disabled, or not work as expected, for parts set up for expiration. The following table outlines other calculations impacted by material expiration:

Table 27.5: Material expiry limitations on other analytics

Analytic / Calculation	Impact of Material Expiry
ATPCTPRule	The **PartType.ATPCTPRule** setting is ignored for all parts where material expiry is enabled.
Days of Supply	Material expiration is compatible with Days of Supply logic in RapidResponse. However, in some cases expiry requirements might result in multiple supplies being generated on different days within a single days of supply period. When days of supply logic is enabled for an expiring part and a planned order is required in a given period, RapidResponse collects all demands within that period and tries to plan a single supply to meet the latest earliest expiry date of those demands. However, if the order cannot be planned with an expiry date to meet the latest earliest expiry date of the demands, then only the portion of the supply whose estimated expiry date can be met is included in the order and subsequent planned order(s) are created for the remaining quantity in the period.
Order priorities	All order priorities on parts configured for material expiry are assumed to be period effective. Note that if attribute-based planning is also used along with expiry and priority, then these priorities might not always be respected (priorities might not be respected across groups of demand attributes). For more information about period effective priorities, see "Period effective priority" on page 1768.
Fair share and equal share allocation	Fair share and equal share allocation logic is applicable but only within blocks of supplies as grouped by expiry date. For more information, see "Fair and equal share allocation with expiring supply" on page 1469.
Lot size last order	When **OrderPolicy.LotSize LastOrder** is set to "N" on part source for a part with material expiry enabled, the last order quantity on the part source is reduced by the amount that remains to expire on the planned order.
Two-date planning	Two-date planning is automatically disabled for any parts subject to expiry. That is, independent demands for parts where **PartType.ExpiryRule** is set to a value other than "Ignore" will only be planned to their due date regardless of the demand's referenced setting in **OrderPriority.TwoDatePlanningRule**.

CHAPTER 28: Constraint Analysis calculations

RapidResponse performs Constraint Analysis calculations to determine the date when supplies and demands are available (referred to as AvailableDate) while considering multiple constraints associated with the source of supply. A single source of supply can be constrained (restricted) by multiple constraints.

Constraint can be fixed or variable factor representing the capacity of a process or supplier. Constraint is based on a rate, which is the amount of constraint available per time period. For example, a constraint on a manufacturing process could be the number of units per day that a production line can produce, or the number of production hours per day it operates.

RapidResponse considers constraint both when planning supply and when calculating the AvailableDate for supplies.

In this chapter, you'll learn about:

Understanding constraint **1629**
Constraint Analysis data model tables **1630**
Constraints on part sources **1631**
Families and multi-level constraints **1633**
Constraint Analysis process overview **1635**
Planning with fixed and variable constraints **1642**
Planning with multiple constraints **1643**
Planning with multi-level constraints **1644**
Reporting constraint loads **1645**
Constraint Analysis control settings **1651**
Control settings and calculated fields in Netting **1685**

Understanding constraint

Constraint is defined in the **Constraint** table, which is an input table.

Examples of constraint are:

- Units per day available from a production line.
- A fixed number of units associated with the supply source.
- The number of hours that a machine can operate.

A part source refers to the source RapidResponse uses to replenish supply to satisfy a demand and is defined in the **PartSource** table. A part source can be constrained by multiple constraints such as:

- A constraint that allows 50 items per day and has 1000 items available.
- A constraint that allows a total of 100 items a day and has 500 items available.

Both constraints must be available for RapidResponse to use the part source. In this situation, RapidResponse can recommend a total of 500 planned orders (defined by the second constraint) at a rate of 50 per day (defined by the first constraint). For more information about multiple constraints, see "Planning with multiple constraints" on page 1643.

If a part source is not restricted by constraint, then it has no associated constraints. Different part sources can use the same constraint.

Constraint Analysis data model tables

The following RapidResponse table types store Constraint Analysis information:

- Input tables
- Calculated tables
- Control tables

Constraint Analysis input tables

The following are Constraint Analysis input tables:

- **Constraint—**defines all constraint(s) used by RapidResponse. For more information about this table, see "Constraint table" on page 247.
- **SourceConstraint—**associates constraints with the part sources they constraint, and defines fixed and variable constraint factors (which can be time-phased). For more information about this table, see "SourceConstraint table" on page 605 and "Constraints on part sources" on page 1631.
- **ConstraintAvailable—**defines the availability of constraint over a period of time. For more information about this table, see "ConstraintAvailable table" on page 249.

Constraint Analysis calculated tables

The following are Constraint Analysis calculated tables:

- **ConstraintLoad**—reports load on a constraint from supplies (for example, how much constraint is used in a given time period). For more information about this table, see "ConstraintLoad table" on page 973.
- **ConstraintSpreadLoad**—reports the spread load on a constraint from supplies (for example, how much constraint would theoretically be used in a given time period if availability were not an issue). For information about this table, see "ConstraintSpreadLoad table" on page 974.
- **ConstraintUsed**—reports when load is allotted to each supply that uses constraint. For more information about this table, see "ConstraintUsed table" on page 975.
- **FLPConstraint**—reports full level pegging and supply available information for constraint usage. For more information about this table, see "FLPConstraint table" on page 1058.

- **PeriodConstraintLoad**—reports bucketed load on a constraint. For more information about this table, see "PeriodConstraintLoad table" on page 1142.

Note: For detailed information about how constraint load is reported in these tables, see "Reporting constraint loads" on page 1645.

Constraint Analysis control tables

The following are Constraint Analysis control tables:

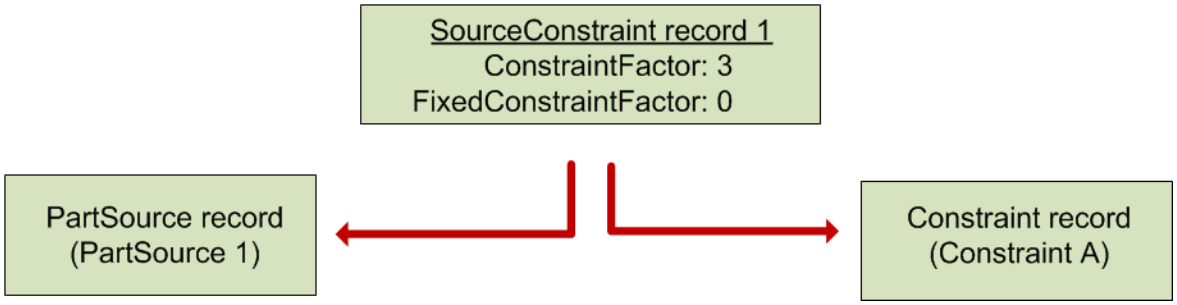
- **ConstraintType**—defines the rules that control how each constraint is processed. For more information about this table, see "ConstraintType table" on page 700.
- **ConstraintUnitOfMeasure**—defines units of measure used by constraints. For more information about this table, see "ConstraintUnitOfMeasure table" on page 703.

Note: There are also some fields on other tables that control how constraint is used. For more information about the different control fields that affect Constraint Analysis calculations, see "Constraint Analysis control settings" on page 1651.

Constraints on part sources

The **SourceConstraint** table is used to associate constraints with the part sources they constrain. Each **SourceConstraint** record references both a **PartSource** record that identifies a source of supply, and a **Constraint** record that identifies a valid constraint as shown in the following illustration. The constraint referenced in the **Constraint** record then constrains the part source referenced in the **PartSource** record.

Figure 28.1 SourceConstraint associating a PartSource with a Constraint

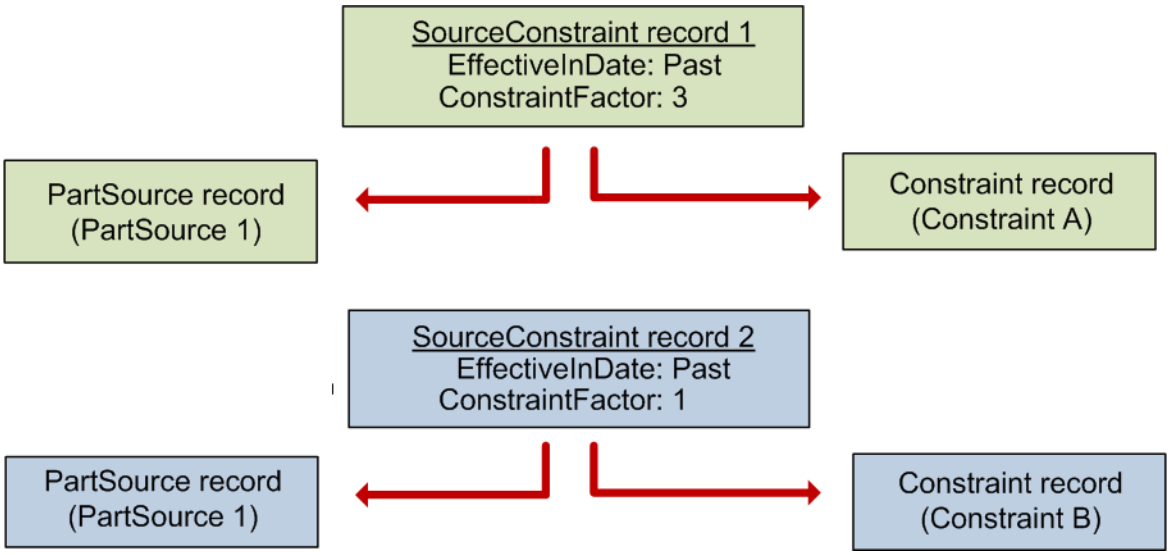


The **SourceConstraint** table also contains two fields that affect the amount of constraint required by supplies from the referenced part source. The **SourceConstraint.FixedConstraintFactor** field defines a fixed amount of constraint to be consumed by each supply from the referenced part source, and the **SourceConstraint.ConstraintFactor** defines a variable amount of constraint to be consumed by each unit within that supply.

To constrain a part source with multiple constraints, additional **SourceConstraint** records are required that reference the same **PartSource** record and additional or different **Constraint** records. For example,

in the following illustration PartSource 1 is constrained by both Constraint A and Constraint B (with separate **SourceConstraint** records required for each).

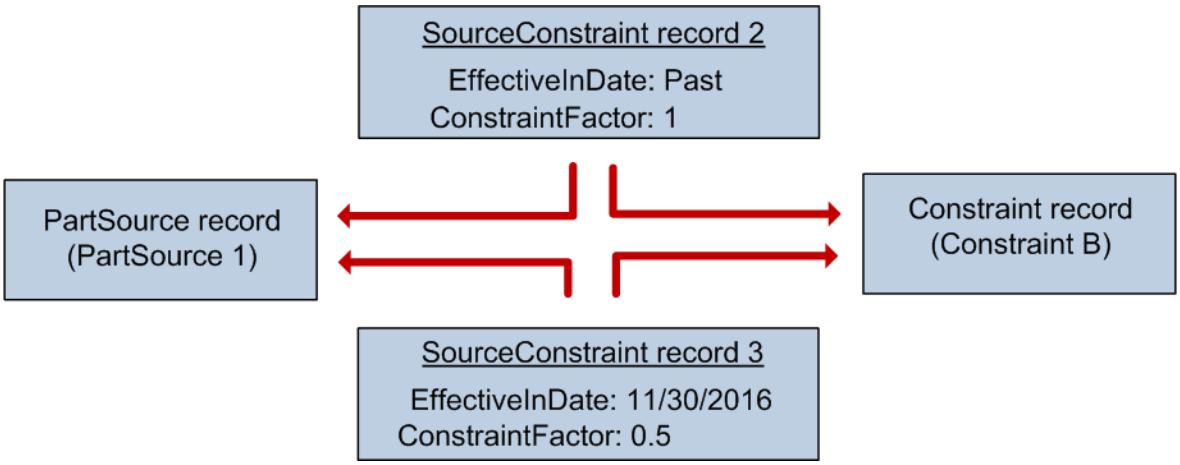
Figure 28.2 PartSource associated with multiple Constraints



If the fixed or variable constraint factors being applied to a part source vary over time, then **SourceConstraint** records can be time-phased. For example, if a manufacturing process improves over time then the constraint factor might need to be decreased. Each **SourceConstraint**record has an **EffectiveInDate** field specifying the date on which the fixed and variable constraint factors on the record become effective. The factors are considered effective until the next **EffectiveInDate** on a **SourceConstraint** record for the part source and constraint combination.

If the constraint factor for a given part source is not going to change over time, then only one **SourceConstraint** record is needed for a given constraint and part source combination (and its **EffectiveInDate** can be set to the default value of "Past"). But if constraint factors change over time, then two or more **SourceConstraint** records are needed to define the time-phased factors for a constraint and part source combination as shown in the following illustration (assuming one constraint factor effective up to 11/29/2016 and a lesser constraint factor from 11/30/2016 onward).

Figure 28.3 PartSource with time-phased constraint factors.



When determining which **SourceConstraint** record applies to a given supply from a part source, the supply's calculated **ParameterDate** is used and the **SourceConstraint** with the closest **EffectiveInDate** on or before that **ParameterDate** is chosen. For information about how this date is calculated on planned orders see "ParameterDate" on page 1153 and for information about how this date is calculated on scheduled receipts see "ParameterDate" on page 586.

In cases where constraint consumption straddles multiple **SourceConstraint**records, the factors from the chosen **SourceConstraint** record are used throughout. For example, in the previous illustration, if a supply was due on 11/30/2016 and consumed constraint between 11/28/2016 and 11/30/2016 it would use a **ConstraintFactor** of 0.5 on each of those days (even though the ConstraintFactor effective on 11/28/2016 and 11/29/2016 is 1.0).

Notes:

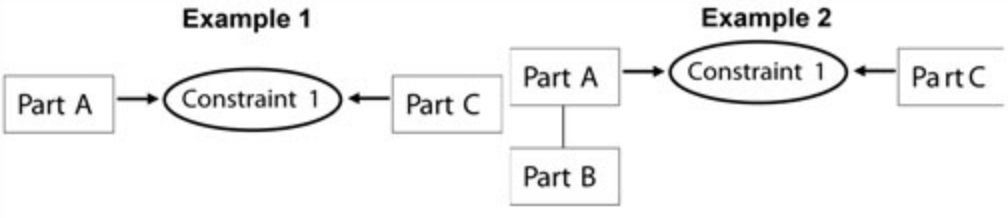
- Any **SourceConstraint** record with an **EffectiveInDate** after **PartSource.LastEffectiveDate** will never become effective.
- If there are multiple **SourceConstraint** records with **EffectiveInDate** on or before **PartSource.FirstEffectiveDate,** only the latest of these can ever be effective (at the beginning of the part source's effectivity).
- A part source's **SourceConstraint.Constraint.Type.ProcessingRule** further determines how constraints are applied. For more information, see "ProcessingRule" on page 1652.

Families and multi-level constraints

RapidResponse data can include multi-level constraints. A typical example of a multi-level constraint is a constraint used at different levels of a product structure (such as, a metal stamping or welding machine used throughout the production process).

Constrained planning calculations support multi-level constraints through multi-level families. A family consists of all parts and constraints related through **SourceConstraint** relationships (for complete information on SourceConstraint relationships, see "Constraints on part sources" on page 1631). The following illustration shows examples of simple single-level constraint families. In each case, the family is composed of Part A, Part C, and Constraint 1.

Figure 28.4 Two single level constraint families

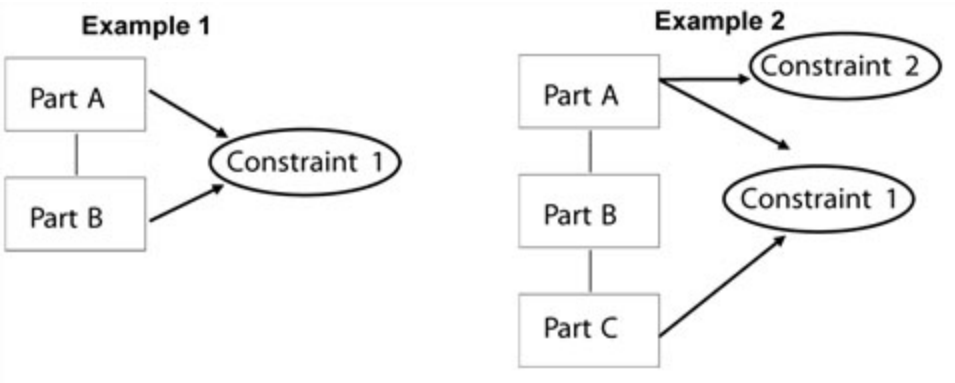


A constraint family is considered multi-level if it includes parts where at least one part is related through product structure relationships to another part in the family (either directly or indirectly). Product structure relationships refer to bill of material entries for parts with 'Make' part sources, transfer parts with 'Transfer' part sources, or allocations on scheduled receipts.

Within a multi-level constraint family, each constraint is considered multi-level even if it is used at only one level in the family. The following illustration shows examples of typical multi-level constraint families. Note that in the example on the right, Part B is included in the family even though it does not use any

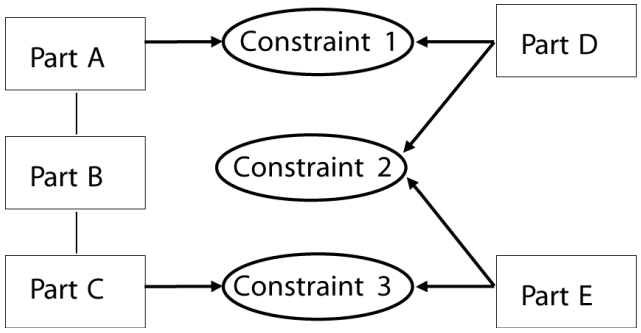
constraints (because it is an intervening part between A and C), and Constraint 2 is considered multi-level even though it is only used by Part A (because all constraints in a multi-level family are considered multi-level).

Figure 28.5 Two typical multi-level constraint families



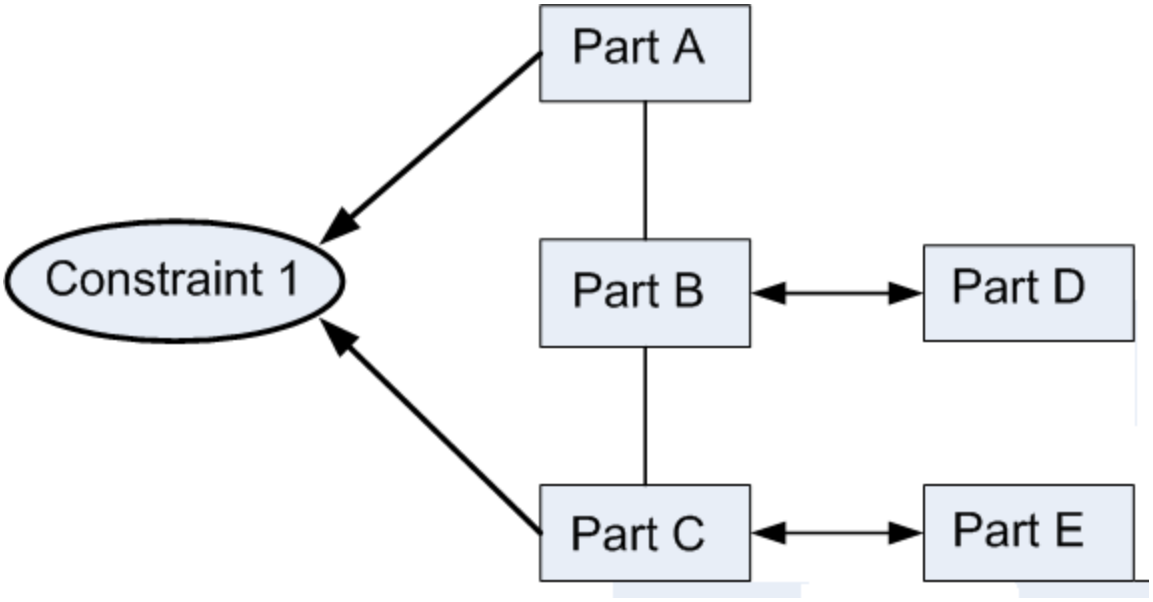
The following illustration shows a more complex example of a multi-level constraint family. In this example, both Part A and its descendant Part C use different constraints, however they still belong to a multi-level family due to other **SourceConstraint** relationships in the family. Specifically, Part A shares Constraint 1 with Part D, while Part D shares Constraint 2 with Part E, and Part E shares Constraint 3 with Part C. Part B is still included in the family, as it is an intervening part between A and C. Thus a multi-level family is created because it includes both a part and a descendant of that part (that is, Parts A and C).

Complex multi-level constraint family



If a part in a constraint family is interchangeable with other parts, every part in the interchangeable group is a member of the constraint family. The following illustration shows an example of a multi-level constraint family with interchangeable parts. In this example, Part A and its descendent Part C use Constraint 1. Part B is included in the constraint family because it is an intervening part between Part A and Part C. Part B is interchangeable with Part D, and Part C is interchangeable with Part E, making Part D and Part E members of the constraint family.

Figure 28.6 Multi-level constraint family with interchangeable parts



Notes:

- For simplicity, the illustrations in this section assume each part is has only one part source, and parts are illustrated as having constraints. In actuality, parts may have multiple part sources, part sources are what have constraints, and SourceConstraint records link constraints to part sources.
- For information about how constrained Netting and Capable-to-Promise calculations differ for multi-level constraints, see "Planning with multi-level constraints" on page 1644.

Constraint Analysis process overview

RapidResponse considers constraint twice. It first considers constraint during netting (a downward pass through the bill of material product structure). It also considers constraint during capable-to-promise (CTP) calculations (an upward pass through the bill of material product structure).

The objective of netting is to propagate demand from top of the bill of material product structure to the bottom, consuming existing supplies as it goes, and generating reschedule recommendations and planned orders based on available constraints. Typically, supplies are planned in line with the date when they are needed, with some planned early (pre-planned) if necessary to ensure that other supplies meet their required date. However, this is not always the case. If sufficient constraint is not available, higher priority orders may take precedence over lower priority orders due earlier. For more information about how priority affects planning, see "OrderPriority rules" on page 1657.

During netting, RapidResponse performs the following basic tasks which are discussed later in this section:

1. Determines effective demand.
2. Matches supply to demand (generates reschedule recommendations and planned orders based on available constraints).
3. Explodes supplies to become dependent demand.

During the upward pass through the product structure, RapidResponse performs Capable-to-Promise calculations (CTP). During CTP, RapidResponse performs the following basic tasks which are discussed later in this section.

4. Determines MaterialStartDate for each supply.
5. Determines AvailableDate for each supply based on constraints and material start date.
6. Allots supply to demands.

Notes:

- The remainder of this section discusses each of the steps in the Constraint Analysis process. There are also control settings you can configure to control how Constraint Analysis calculations are carried out for a particular constraint or constraints. For more information about these, see "Constraint Analysis control settings" on page 1651.
- For more information about Netting, see "Netting basics" on page 1331. For more information about Capable-to-Promise, see "Capable-to-Promise calculations" on page 1453.

Determining effective demand (1)

The first step of the Constraint Analysis process is determining if unsatisfied demand exists. An example of unsatisfied demand is an **IndependentDemand** record for a given part. Once RapidResponse locates unsatisfied demand, it attempts to satisfy it by matching supply to the demand.

Matching supply to demand (2)

The second step of the Constraint Analysis process is the attempt by RapidResponse to satisfy demand with supply. The first location RapidResponse searches for supply is records in the **OnHand**table. If insufficient quantities of supply exist in the **OnHand** table, then RapidResponse next searches records in the **ScheduledReceipt** table.

If the demand part is interchangeable with other parts, supply for all parts in the interchangeable group is used to satisfy the demand. For more information about interchangeable parts, see "Interchangeable parts" on page 1545.

Rescheduling scheduled receipts

There may be **ScheduledReceipt** records that can satisfy demand; however, the due date (the date the scheduled receipt is available for use) is too late. As a result, RapidResponse attempts to recommend a new due date so demand can be satisfied. The **ReschedDate**is a calculated field that specifies the date the scheduled receipt should be due to satisfy demand. RapidResponse considers constraint when rescheduling scheduled receipts.

You can compare the value of the **ScheduledReceipt.DueDate** field and the **ScheduledReceipt.ReschedDate** field to determine if the scheduled receipt has been rescheduled. If these two values are the same, then the scheduled receipt hasn't been rescheduled. If **ScheduledReceipt.ReschedDate** is an earlier date, then the scheduled receipt was expedited. If **ScheduledReceipt.ReschedDate**is a later date, then the scheduled receipt was delayed.

Note that not all scheduled receipts can be rescheduled. Whether or not a scheduled receipt can be rescheduled is determined by values in the **SupplyType.ProcessingRule** and

SupplyStatus.RescheduleRule fields. For more information, see "Processing Scheduled Receipts" on page 1685.

ScheduledReceipt CalcStartDate field

The **ScheduledReceipt.CalcStartDate** field specifies the date a scheduled receipt should be started based on available constraint. The calculation of **ScheduledReceipt.CalcStartDate** depends upon:

- If there is sufficient constraint available when planning the scheduled receipt.
- If the scheduled receipt is reschedulable. For more information about how reschedulability is determined, see "Rescheduling rules" on page 1653.
- Whether the scheduled receipt has a defined or undefined start date, or whether its **SupplyStatus.IgnoreStartDate** is set to **Y**.
- Whether the AllowLatePlan field is set to Yes or No. For more information, see "AllowLatePlan" on page 1663.
- If constraint is used at StartDate, ShipDate or DuringLeadTime. This is defined by setting on the **PartSourceType.ConstraintDateRule** field. For more information, see "ConstraintDateRule" on page 1654.

Note: To see how CalcStartDate is calculated for different combinations of the factors outlined above, see "Processing Scheduled Receipts" on page 1685.

Unconstrained Quantity

RapidResponse calculates the amount of constraint needed (constraint load) using:

- If **SupplyStatus.TargetAccum** = **Ignore**, then 0.
- If **SupplyStatus.TargetAccum** = **Remaining**, then Quantity - **ScheduledReceipt.UnconstrainedQuantity**.
- If **SupplyStatus.TargetAccum** = **Order**, then TotalQuantity - **ScheduledReceipt.UnconstrainedQuantity**.

The resulting quantity, **ConstrainedQuantity**, is multiplied by **SourceConstraint.ConstraintFactor** to calculate the load.

Constraint allocation

When RapidResponse allocates constraint, it consumes in constraint calendar periods including and after **Constraint.DataDate**. The **SupplyStatus** table determines whether a scheduled receipt needs allocated constraint. For example, the **SupplyStatus.ConstraintRule** field specifies the processing rules to be applied when processing constraint.

Implementations using constraint at StartDate need to ensure that the quantity of the order already started is recorded in the **ScheduledReceipt.UnconstrainedQuantity** field for any partially started orders (if fully started, then the **SupplyStatus.TargetAccum** could be set to **Ignore** and **UnconstrainedQuantity** is irrelevant).

Minimum allocation

When allocating constraint, RapidResponse only allocates constraint if a minimum amount of constraint is available in the period. For scheduled receipts, the minimum allocation is the lesser of the effective part source order multiple (if **PartSource.Type.ConstraintMultipleRule** is set to **Use**), and the remaining load for the order.

Scheduled receipts with shipments

Scheduled receipts with a **Shipment** record with valid DeliveryDateTime are treated specially. Their **EffDueDate** and **ReschedDate** are set from the**Shipment** record and these orders are always seen as **NonReschedulable**.

Allocations

Allocation.ReschedDate is intended to be the dependent demand date corresponding to the associated scheduled receipt's ReschedDate.

If **SupplyStatus.IgnoreStartDate** is set to **N**, then **Allocation.ReschedDate** for dependent demand from **Allocation** records is calculated by adjusting **Allocation.ReschedDate** by the same number of **ScheduledReceipt.PartSource** time units as **ReschedDate** differs from **RecommendedDate** for the associated scheduled receipt. If **SupplyStatus.IgnoreStartDate** is set to **Y**, then **Allocation.ReschedDate** is equal to **ScheduledReceipt.CalcStartDate**.

A special situation is a self-allocation where the allocation is for the same part as the associated scheduled receipt. In this case, the **ReschedDate** on the allocation is set to **Allocation.DueDate**. For more information about self-allocations, see "Self-allocations" on page 1424.

Transfer Orders

Transfer supplies (scheduled receipts with **SupplyType.Source** control value of **Transfer** and have a valid transfer part source with TransferPart <> Part and planned orders with a valid Transfer part source) are special because their dependent demand is at ship date, not start date. The supplying part then applies its own lead time when planning supply to meet the ship date.

Note also that if the value of **Source.PreShipLT** (pre-ship lead time) is greater than zero, the transfer dependent demand date is that number of source lead time units prior to the reported ShipDate.

OnTimeQuantity

OnTimeQuantity is the quantity of the supply that could be started on its CalcStartDate if it was possible to split the order. This is the same result as before except where the effective lead time on the scheduled receipt is extended due to constraint limitations.

Creating planned orders

If a net requirement exists after RapidResponse searches records in the **OnHand** and **ScheduledReceipt** tables, then RapidResponse attempts to create planned orders to satisfy demand. Planned orders are generated to align with the date they are needed (Demand Date).

Pre-planning limits

Assume there are multiple part sources from which RapidResponse can choose to replenish supply and each part source is constrained by one constraint. If a part source does not have enough constraint to create a planned order for its demand date, RapidResponse can either search for available constraint in earlier time periods or use a lower priority part source in the current time period that may have available constraint (there are various reasons, such as cost, why a part source might have a lower priority value).

A field on the **PartSource** table named **PrePlanLimit** controls whether RapidResponse plans orders in earlier time periods using a specific part source or attempts to plan orders using lower priority part sources within the current time period. For more information about the **PrePlanLimit** field, see "PartSource table" on page 459.

Valid values for the **PrePlanLimit** field are:

- **>0**—use this part source (and others at the same priority) in the **PrePlanLimit** number of earlier periods before trying lower priority part sources in the current period.

- **0**—test other part sources in the current period before trying this source in earlier periods. (This is the default value for this field).
- **<0**—test part sources at the same priority as this source in the current period, then test this source (and others at the same priority) in earlier periods, up to the planning time fence (defined in the **PartSource.PlanningTimeFence** field) for this part source.

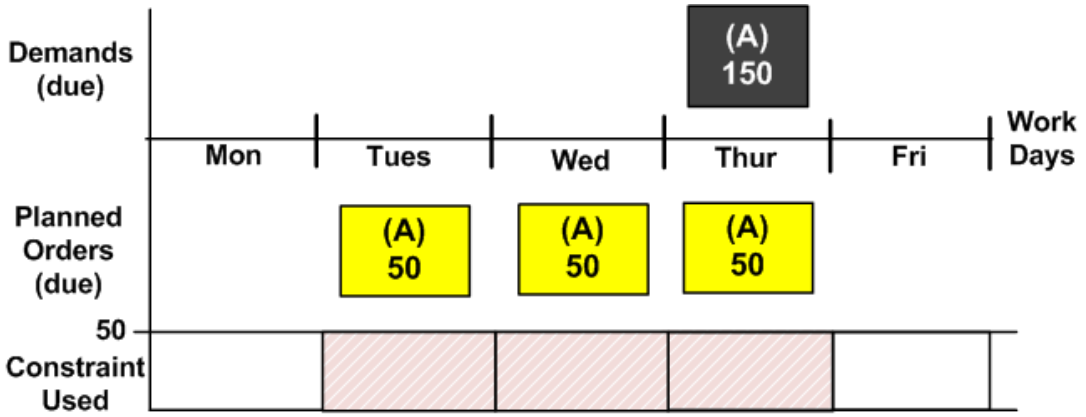
Additionally, a field on the **PartSource** table, **AbsolutePrePlanLimit**, sets the maximum number of calendar days that constraint can be planned or allocated to supplies using a given part source before its normal constraint allocation date. By default, this field is set to -1 with any negative value implying there is no limit to how early RapidResponse can pre-plan constraint for supplies using the part source. Typically, it is recommended this value be set high enough so there is time to build the largest supply within its limit.

For example, assume Part A has exactly one part source that uses a production constraint and the following constraint-related settings are defined:

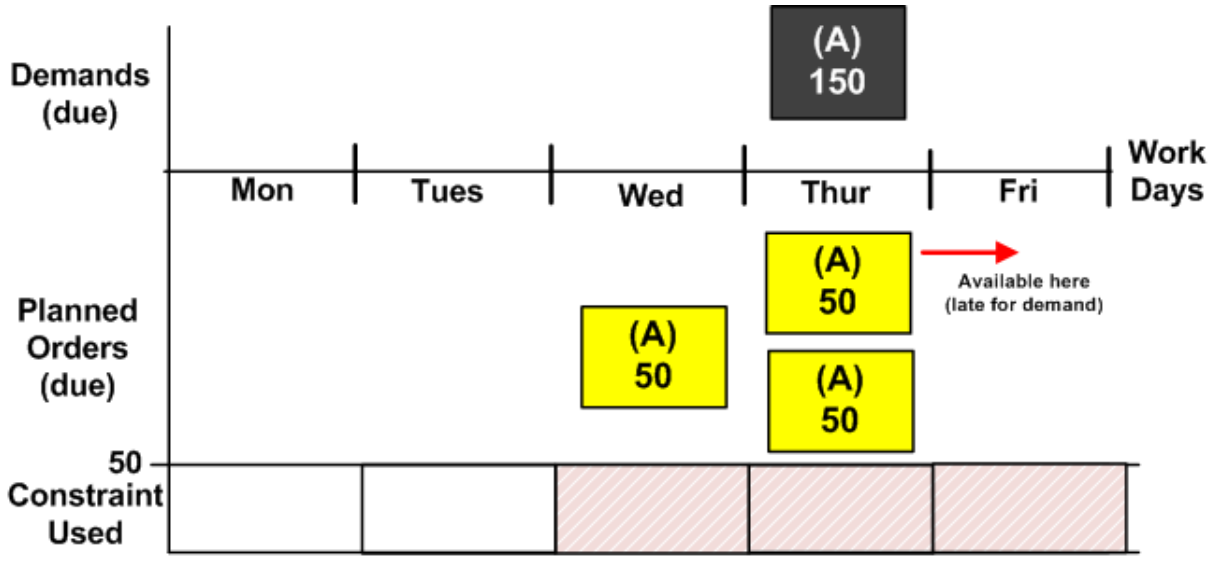
- **Constraint.Calendar**- Workday
- **ConstraintAvailable.Rate**- 50
- **SourceContraint.ConstraintFactor**- 1
- **ConstraintType.ProcessingRule**- Constrained
- **PartSourceType.ConstraintDateRule**- Ship

If **AbsolutePrePlanLmit** is set to -1, then a given demand for 150 units of A might be sourced and supplies as shown in the following illustration.

Figure 28.7 AbsolutePrePlanLimit set to -1



Instead assume the **AbsolutePrePlanLimit** were set to 1. In this case, supplies to satisfy that same demand for 150 would be planned as shown in the following illustration. In this case, due to the limitation on pre-planning, two planned orders are planned on the Thursday but constraint availability only one of them can be available on time.



Note: The interpretation of **AbsolutePrePlanLimit** values of 0 (zero) is configurable through a global setting (can mean unlimited pre-planning or no pre-planning allowed). For more information see "Configuring AbsolutePrePlanLimit usage" on page 2206.

CumulativeMax

Order(s) are planned for those available part source(s) up to the amount of constraint available (the maximum amount of constraint associated with a part source is named **CumulativeMax** and appears as a field on the **Constraint** table). A planned order quantity is limited by constraint available and therefore may be less than the total required. Supply allocated to a part source may result in more than one planned order with the same DemandDate, but the total planned orders with the same DemandDate will represent at least one lot size.

When there are no earlier part sources available, RapidResponse searches forward, looking for the first part source(s) that become active (value of the **FirstEffectiveDate** field on the **PartSource** table). If that part source is effective and past its OrderGeneration fence at DemandDate, DueDate is set to DemandDate (assuming **SupplyStatus.AllowLatePlan** is **N**). Otherwise, DueDate is set to the later of **FirstEffectiveDate** and OrderGeneration fence for that part source. Once all constraint(s) associated with a part source are consumed, the part source is no longer used to create a planned order.

Notes:

- For constraints with **ConstraintType.ProcessingRule** set to either **Constrained** or **CumMax,** RapidResponse does not plan orders exceeding the **CumulativeMax** field on the **Constraint** table.
- For information about the impact of **SupplyStatus.AllowLatePlan**, see "AllowLatePlan" on page 1663.

Constraint Rate

The amount of constraint available during each time period is constraint rate (this value is determined by the **Rate** field in the **ConstraintAvailable** table). For more information about the **ConstraintAvailable.Rate** field, see "ConstraintAvailable table" on page 249.

Exploding supplies (3)

Exploding supplies is the process of creating allocations (gross requirements) for a component to be used in the production of an assembly. When there are not sufficient **ScheduledReceipt** or **OnHand** records for the components in the bill of material product structure, RapidResponse attempts to create planned orders to satisfy demand. The process of determining effective demand, matching supply to demand, and exploding supply is repeated until the end of the bill of material product structure is reached. Once the end is reached, RapidResponse begins the upward pass (CTP calculations) through the product structure.

Determining MaterialStartDate (4)

Once RapidResponse completes netting (downward pass through the bill of material product structure), it calculates MaterialStartDate for each supply. MaterialStartDate on supply records (for example, planned order records) is the date when all components are available to start work on the order. RapidResponse considers constraints used by components that go into an assembly supply when determining the availability of the components, but not the constraint on the assembly supply itself.

Determining AvailableDate (5)

Once MaterialStartDate is calculated, RapidResponse uses this date to determine AvailableDate for the supply. AvailableDate is the date a supply is available based on materials and constraint. In addition, it may be limited by the setting in the AvailableDateLimit field as discussed in "AvailableDateLimit" on page 1666. AvailableDate appears as:

- A calculated field in the **ScheduledReceipt** table.
- A field in the **CTPActivity** calculated table.
- A field in the **CTPPlannedOrder** calculated table.
- A field in the **FLPConstraint** calculated table.
- A calculated field in the **IndependentDemand** table.
- A calculated field in the **LateSupply** table.

Allotting supply to demand (6)

Once AvailableDate of a supply is determined, RapidResponse allots supply to demand, which determines the AvailableDate of the demand (the **AvailableDate** calculated field is assigned a value). RapidResponse repeats the process of determining the MaterialStartDate, AvailableDate and allotting supplies to demands until it reaches the top of the bill of material product structure.

When allotting supply to demand, RapidResponse does so either on the basis of the supply's AvailableDate or the date when it was needed (the supply's demand date). The date used is determined by the setting on PartType.SupplyAllocationRule as discussed in "SupplyAllocationRule" on page 840.

Planning with fixed and variable constraints

RapidResponse supports both fixed and variable constraint consumption. Fixed constraint consumption should be used when constraint usage is dependent on the number of batches, and variable constraint consumption should be used when constraint usage is dependent on the size of each batch.

The amount of fixed constraint used by each supply from a given part source is determined by the value in the **SourceConstraint.FixedConstraintFactor** field. For example, if the FixedConstraintFactor is set to 3, then each supply created against the associated part source will consume three units of constraint.

The amount of variable constraint used by each supply from a given part source is determined by both the value in the **SourceConstraint.ConstraintFactor** field and the size of the supply. For example, if the ConstraintFactor is 5 and the supply quantity is 40, then the supply would consume 200 units of constraint.

In some cases, a constraint might be subject to both fixed and variable consumption. For example, there might be a fixed amount of constraint used in setting up a machine to process an order, and then an additional variable amount of constraint associated with processing each unit of that order.

When a supply consumes both a fixed and variable amount of a given constraint, the fixed amount is consumed first. Variable constraint will then start in the last period where that fixed constraint is consumed. Note that if there is not enough constraint available to plan the entire required supply, then the fixed constraint is consumed first and the remainder determines the supply quantity. Therefore, if there is not enough constraint to satisfy the required fixed constraint for a supply, then no supply can be planned.

The following illustration shows an example of constraint consumption for a supply source consuming both a fixed and variable amount of constraint from a single constraint. This example assumes a **FixedConstraintFactor** of 1, a **ConstraintFactor** of 3, and that 4 units of the constraint are available in each period. Note that the supply for a quantity of 1 in Period 1 consumes exactly all 4 units of constraint in that period. Conversely, the supply for a quantity of 4 in Period 5 requires a total of 10 units of constraint. This is more than is available in that period, and so constraint consumption must start in Period 3 with the fixed constraint being consumed at the beginning of that period.

Figure 28.8 Fixed and variable supply from single constraint

Period	1	2	3	4	5
Supply	1				3
Total Load (fixed+variable)	1+3=4				1+9=10
Constraint Available					

Similarly, if constraint is required from multiple constraints, fixed constraint is still consumed first and the variable constraint again begins in the last period where fixed constraint was consumed. For example, the following illustration shows an example of constraint consumption for a supply source a fixed amount of constraint from one constraint (Constraint1) and a variable amount of constraint from a second constraint (Constraint2). This example assumes that Constraint1 has a **FixedConstraintFactor** of 2 with 4 units of constraint available in each period, and that Constraint2 has a **ConstraintFactor** of 4 with 6 units of constraint available in each period.

Note that there is sufficient amount of each constraint to satisfy the supply for a quantity of 1 in Period 1. However, the supply for a quantity of 3 units in Period 5 requires 12 units of variable constraint from Constraint2 which consumes all of its available constraint in both Period 4 and Period 5. As such, the required consumption of 2 units of fixed constraint from Constraint1 must also begin in Period 4.

Figure 28.9 Fixed and variable supply from separate constraints

Period	1	2	3	4	5
Supply	1				3
Total Load1 (fixed)	2				2
Constraint1 Available					
Total Load2 (variable)	4				12
Constraint2 Available					

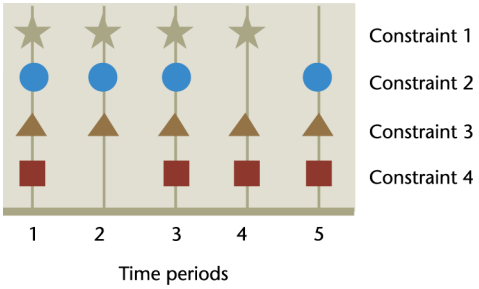
Planning with multiple constraints

If a part source is constrained with multiple variable only constraints (**SourceConsraint.ConstraintFactor** >0 and **SourceConstraint.FixedConstraintFactor** = 0), then all constraints must be available for RapidResponse to use the part source in a time period. If one or more variable constraints does not have at least the minimum amount of constraint required in the period, then no constraint is consumed in that time period. RapidResponse checks previous and subsequent time periods until one is found where all constraints are available.

When more than one part source exists, RapidResponse tests for earlier available constraint using the highest priority part source. The number of earlier periods is defined by the**PartSource.PrePlanLimit**

and **PartSource.AbsolutePrePlanLimit** fields (in the following figure, there are five time periods). If available constraint cannot be located, RapidResponse uses a lower priority part source.

Figure 28.10 A part source constrained by four constraints



The above diagram assumes that the value of the **PartSourceType.ConstraintDateRule** control value is **Ship**. This is the default value for this field and means constraint is applied corresponding to the date when the supply is shipped (for more information, see "ConstraintDateRule" on page 1654).

Assume time period five represents the date when a scheduled receipt is to ship. Because the part source is constrained by four constraints, RapidResponse determines if there is available constraint. However in time period five, constraint one is unavailable. As a result, RapidResponse searches time period four for constraint one.

In time period four, constraint one is available; however, constraint two is missing. RapidResponse checks time period three where all four constraints are available. RapidResponse uses as much as possible until its needs are satisfied. If RapidResponse needs are not satisfied, it searches for more constraint in a previous time period. In time period two, constraint four is missing. Finally RapidResponse locates period one where there is sufficient constraint to meet its needs.

If RapidResponse cannot locate enough constraint in a previous time period, it searches a later time period and consumes available constraint. A later time period delays the date a supply is available.

Note: If a constraint has a positive value in **SourceConstraint.FixedConstraintFactor**, then parallel constraint consumption as described above is not guaranteed. That is, fixed constraints might be used before the variable constraints/variable constraint portions and not necessarily within the same time period. For more information about planning with variable and fixed constraints, see "Planning with fixed and variable constraints" on page 1642.

Planning with multi-level constraints

The calculations discussed earlier in this chapter pertain to single-level constraints. When dealing with multi-level constraints, these calculations still apply, however there are a few other considerations involved. This section discusses how Netting and Capable-to-Promise calculations differ with multi-level constraints.

Note: For an introduction to multi-level constraints, see "Families and multi-level constraints" on page 1633.

Netting impacts

Netting calculations for multi-level constraints are similar to those for single level constraints. They begin at the highest logical level in a given multi-level family, and perform constrained netting as discussed in "Constraint Analysis process overview" on page 1635.

However, within a multi-level family, constraint usage is carried down from level to level. This means that lower levels in the family can only use the constraint that remains after consumption at higher levels in the family. As a result, when constraints are overloaded, it is possible that assembly supplies may consume all available constraint between the date constraint is available and the date the supply is needed. This would leave no constraint remaining for component supplies. Therefore, it is recommended that the **SupplyStatus.AllowLatePlan** be set as discussed in "Multi-level constraints and AllowLatePlan" on page 1664.

Capable-to-Promise impacts

As described earlier in this chapter, the Capable-to-Promise analytic calculates the available dates for supplies. When calculating the available date for a supply, it depends on the supply's material start date, which further depends on the available date of component supplies, and the calculation of these dates requires the components to be loaded onto their constraints. Loading the components onto their constraints further requires the loading of all supplies in the family. This is done so that when there is insufficient constraint, high-priority supplies that are ready can be loaded first (this requires knowing all supplies which could possibly consume constraint in a period).

However when multi-level constraints are involved, this means that available dates on component supplies are needed to determine the assembly supply's material start date and load the supply on constraints, but at the same time the assembly supply needs to be loaded on its constraints to determine the available dates on the component supplies. Therefore when multi-level constraints are encountered, Capable-to-Promise simultaneously considers component and constraint availability at all levels within the family, including the availability of sub-assemblies within the family. The results of these calculations are then reported in the Supply fields such as MaterialStartDate, AvailableDate, and FirstAvailableDate.

Reporting constraint loads

Constraint Analysis tables in RapidResponse enable the reporting of loads placed on a constraint by supplies, as well as the amount of constraint actually used. This includes identifying overloaded constraints (constraints where the load exceeds the amount of constraint available).

The following tables are discussed in this section:

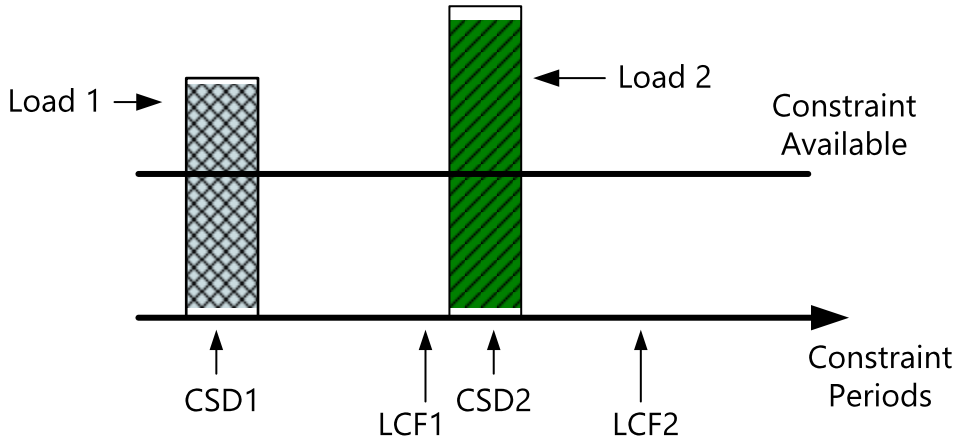
- ConstraintLoad
- ConstraintSpreadLoad
- PeriodConstraintLoad
- ConstraintAvailable (input)
- ConstraintUsed

ConstraintLoad

The **ConstraintLoad** table reports load placed on a constraint by supplies.

This table generates one record for each supply referencing an active constraint. It indicates the amount of constraint being requested in a given period. Each constraint load is reported as a single point load at the supply's constraint start date (that is, the earliest date at which constraint can be applied to the supply). As well, no consideration is given to the amount of constraint actually available. The following illustration shows a simple case showing the load applied to a constraint from two supplies. Note that each load is reported within a single period and, in this case, each load exceeds the amount of constraint available.

Figure 28.11 Reporting constraint load



CSD = Constraint Start Date (earliest date at which constraint can be applied to a supply)

LCF = Late Constraint Finish (latest date at which constraint can be applied to a supply and still meet its ReschedDate

Note: For more information about the **ConstraintLoad** table, see "ConstraintLoad table" on page 973.

ConstraintSpreadLoad

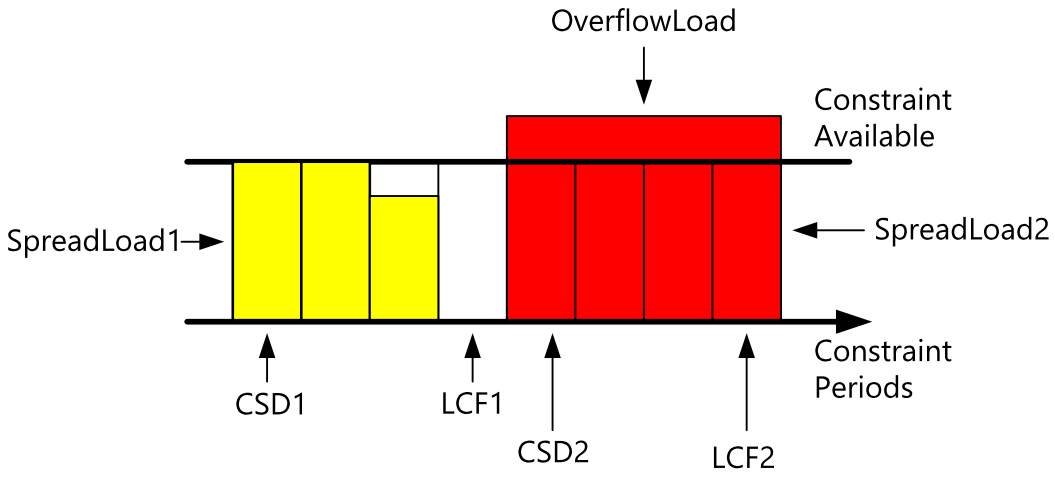
The **ConstraintSpreadLoad** table is similar to the **ConstraintLoad** table as it reports load placed on a constraint by supplies. In this table, a single supply can generate multiple spread load records if insufficient constraint is available in a given period. **ConstraintSpreadLoad** is intended to provide a

good estimate of the constraint level needed to satisfy supplies once the component materials become available.

As with the **ConstraintLoad** table, spread load is initially applied at the period containing the supply's constraint start date. However, if insufficient constraint is found, then additional spread load records are generated for subsequent periods until enough constraint is found or the supply's late constraint finish is reached (that is, the latest date at which constraint can be applied to a supply and still meet its ReschedDate). If sufficient constraint is still not found between the constraint start date and late constraint finish, then the remaining load (overflow) is spread evenly between these two dates.

The following illustration shows how ConstraintSpreadLoad records are generated for a simple case where spread load is applied to a constraint from two supplies. SpreadLoad1 shows a case where sufficient constraint is available between constraint start date and late constraint finish. SpreadLoad2 shows a case where there is insufficient constraint available and overflow load generated.

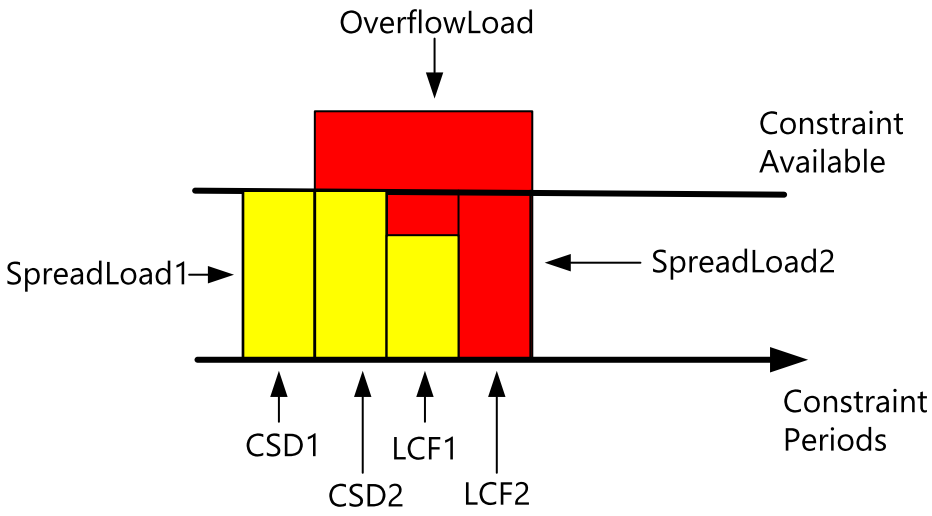
Figure 28.12 Reporting constraint spread load



The previous illustration shows a case where each supply's constraint start date precedes its late constraint finish date, and there is no overlap between the constraint start and late constraint finish date associated with any of the supplies. If either of these conditions did not hold, then spread load would be applied differently.

In the first case, if a supply's late constraint finish date precedes its constraint start date, then it is not possible to spread the load between these two periods. Instead the entire spread load would be applied at the constraint start date (similar to how load is reported in the **ConstraintLoad** table). In the second case, the overlapping of dates could potentially push out the date at which spread load can be applied as shown in the following illustration. In this example, the constraint start date for supply 2 overlaps the late constraint finish date for supply 1, and because all available constraint is being allocated to supply 1, the actual date at which constraint can be applied to supply 2 is pushed out (to the next period in this case). Note that the overflow from SpreadLoad2 is still reported between its constraint start date and late constraint finish.

Figure 28.13 Overlapping spread load



Note: For more information about the **ConstraintSpreadLoad** table, see "ConstraintSpreadLoad table" on page 974.

PeriodConstraintLoad

The **PeriodConstraintLoad** table reports bucketed load, spread load, and used load values across current supplies as well as from both current and planned supplies. This table contains one record for each **Period** (defined by **Constraint.Calendar** intervals) starting from **DataDate** and continuing over a horizon defined by number of days specified in the **ConstraintType.ReportLimit** field

Also, the amount of constraint available in each period is reported in the **Available** field, and the **Overloaded** field can flag whether the load placed on a constraint in a given period exceeds the actual constraint available for the period. The constraint available reported or used by this and other calculated tables is based on input values provided in the **ConstraintAvailable** table.

Note: For more information about the **PeriodConstraintLoad** table, see "PeriodConstraintLoad table" on page 1142.

ConstraintAvailable (input)

The **ConstraintAvailable** table is an input table used to specify the availability of a given constraint over time. Each record is effective over the date range defined by the **EffectiveInDate** and **EffectiveOutDate** fields, with the **ConstraintType.EffectivityRule** setting providing further control over how the effective in and out dates are treated. The **Rate** fieldindicates the amount of the constraint available in each constraint period (that is, each **Constraint.Calendar** interval).

For example, if **Constraint.Calendar** was "Workday" and **Rate** was "50", this indicate 50 units of constraint is available for each work day starting from **EffectiveInDate** and ending the bay before **EffectiveOutDate**. For more information, see "ConstraintAvailable table" on page 249.

Effective dates and calendar intervals

If values in the **EffectiveInDate** and **EffectiveOutDate** fields don't align to constraint calendar intervals (with respect to the **ConstraintType.EffectivityRule** setting), then calendar rounding occurs and results might not always be as expected. For example, assume the default **EffectvityRule** of "InclusiveExclusive", a **Constraint.Calendar** setting of "Month", and the following sets of effective dates:

- If **EffectiveInDate** is "05/16/2018" and **EffectiveOutDate** is "07/31/2018", then the record is effective from May 1, 2018 to June 30 2018.
- If **EffectiveInDate** is "05/01/2018" and **EffectiveOutDate** is "08/01/2018", then the record is effective from May 1, 2018 to July 31, 2018.
- If **EffectiveInDate** is "05/01/2018" and **EffectiveOutDate** is "05/31/2018", then the record is never effective.

Multiple records per calendar interval

If a constraint has multiple effective records over a given constraint period, then the available constraint is determined by summing the **Rate** value from those records. Additionally, any negative **Rate** values will reduce the available constraint for the period. For example, assume a given constraint has **Constraint.Calendar** set to "Week", **ConstraintType.EffectivityRule** set to "InclusiveExclusive", and the folllowing **ConstraintAvailable** records defined.

EffectiveInDate	EffectiveOutDate	Rate
10-01-18	10-15-19	75
10-15-18	10-22-18	40
10-17-18	10-22-18	60
10-19-18	10-29-18	10
10-22-18	11-14-18	85
11-05-18	11-19-18	-5

The amount of the constraint available per period would then be calculated and reported in the **PeriodConstraintLoad** table as follows. This example assumes that all Weekly calendar markers are set to Mondays (for example, October 1, 8. 15, 22, and so on).

Period	Available	(Calculated As...)
10-01-18	75	(75)
10-08-18	75	(75)
10-15-18	110	(40+60+10)
10-22-18	95	(10+85)
10-29-18	85	(85)

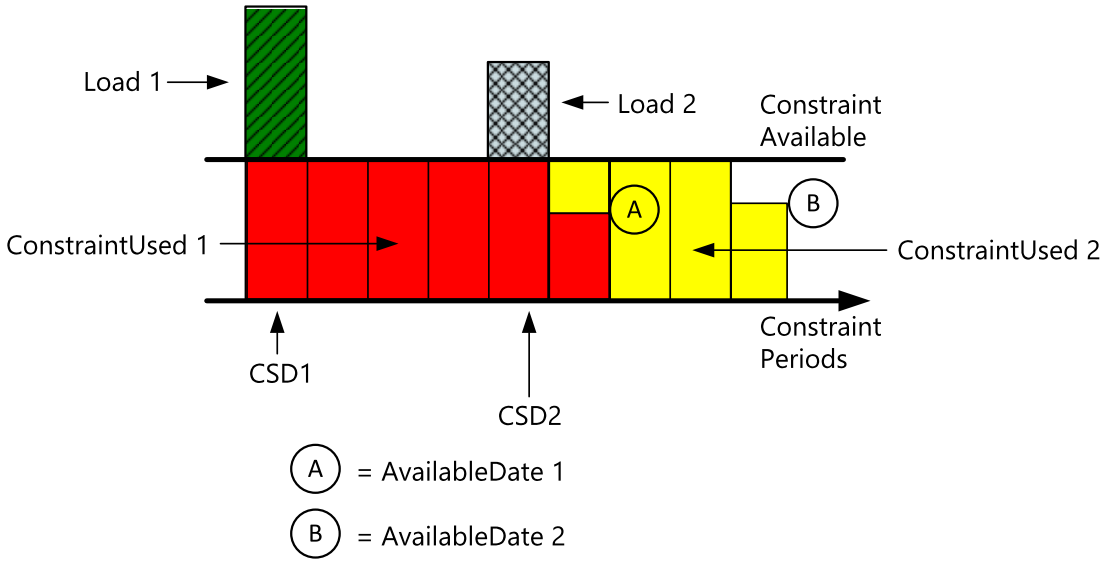
Period	Available	(Calculated As...)
11-05-18	80	(85-5)
11-12-18	0	(Max (0, -5))

ConstraintUsed

The **ConstraintUsed** table shows when load is allotted to each supply that uses a constraint. It is intended to report the actual planned consumption of a constraint. ConstraintUsed initially looks for available constraint in the period containing the supply's constraint start date. If insufficient constraint is found in this period, it then looks in as many subsequent periods as is necessary until the necessary constraint is found. ConstraintUsed usually respects constraint available, so if sufficient constraint cannot be found to satisfy the supply, then the remaining load for that supply will be unsatisfied and no ConstraintUsed records generated for the unsatisfied load. The only exception to this rule is scheduled receipts where **SupplyStatus.RescheduleRule** is set to **Scheduled**. These receipts are seen as having been scheduled by some other system, and any unsatisfied load remaining at late constraint finish date is seen as constraint used at late constraint finish. For more information, see "SupplyStatus table" on page 904.

The following illustration shows a simple case of how ConstraintUsed is reported. Note that in this case ConstraintUsed for the first supply uses up constraint available in several future periods and pushed out the start date and available date for the second supply. This diagram also compares this to how ConstraintLoad is reported for the same case.

Figure 28.14 Reporting constraint and used contraint load



Note: For more information about the **ConstraintUsed** table, see "ConstraintUsed table" on page 975.

Constraint Analysis control settings

This section discusses some of the control settings that determine how Constraint Analysis calculations are carried out. The settings discussed in this section are outlined in the following table.

Table 28.1: Control settings for Constraint Analysis calculations

Setting	Description
ConstraintType.ProcessingRule	Determines how load is reported and netting and CTP calculations are carried out for a particular constraint type. For more information, see "ProcessingRule" on page 1652.
Rescheduling rules	Determines whether or not a scheduled receipt can be rescheduled. For more information, see "Rescheduling rules" on page 1653.
ConstraintDateRule	Determines when constraint is allocated to a supply, and how available date is calculated for that supply. For more information, see "ConstraintDateRule" on page 1654.
OrderPriority rules	Determines the relative importance of orders. This is used when insufficient constraint is available to complete all orders on time. For more information, see "OrderPriority rules" on page 1657.
SchedulePriority	Allows for high priority orders to be scheduled as close to the due date as possible. For more information, see "SchedulePriority" on page 1659.
SplitLateSupply	Determines whether or not orders are always completed in contiguous blocks. For more information, see "SplitLateSupply" on page 1662.
AllowLatePlan	Determines whether or not supply dates cannot be later than original demand dates if insufficient constraint is available. For more information, see "AllowLatePlan" on page 1663.
FillSchedule	Determines whether RapidResponse calculations try to fill up the schedule for a constraint by planning to use it as early as possible. For more information, see "FillSchedule" on page 1665.
AvailableDateLimit	Sets limits, beyond constraint and material availability, on when a supply can be ready. For more information, see "AvailableDateLimit" on page 1666.
ScheduingRule	Determines whether constraint scheduling includes a "forward" phase to optimize constraint usage by planning supplies in continuous blocks where possible. For more information, see "SchedulingRule" on page 1667
PrePlanRule	When constraint availability makes pre-planning of supplies necessary, this setting controls whether that pre-planning is done as late as strictly possible. For more information, see "PrePlanRule" on page 1676.
FairShareAllocation	If there is insufficient constraint in a period, allows for the available constraint to be shared fairly between multiple part sources that require it. For more information, see "FairShareAllocation" on page 1679.
Order policy rules	In cases of insufficient constraint, consumption can be forced to respect order policy rules (such as, lot size and minimum order quantity). For more information, see "Respecting order policy rules" on page 1684.

Note: The remainder of this section provides detailed information about each of the control settings outlined in the preceding table. To see the differences in calculated fields generated during Netting that could be expected from using different combinations of some of these settings, see "Control settings and calculated fields in Netting" on page 1685.

ProcessingRule

The ProcessingRule value is set on the **ConstraintType** table. The ProcessingRule value set for each constraint determines how certain calculations are processed for the constraint. It impacts Netting and AvailableDate calculations, as well as how load is reported in the ConstraintLoad, SpreadLoad, and ConstraintUsed tables. The following table shows the various impacts of each of the five settings available on the **ConstraintType.ProcessingRule** field.

Table 28.2: ProcessingRule settings

Processing Rule	Netting	Load Reporting	AvailableDate (CTP)
Constrained	Uses Constraint Rate and Max Constraint for source selection, and to set CalcStartDate.	• **ConstraintLoad**: At CSD. • **SpreadLoad**: Between CSD and LCF. • **ConstraintUsed**: At CSD, up to CumulativeMax and Constraint Available.	Calculated using materials and constraint available, as well as Cumulative Max
CumMax	Uses Cumulative Max.	• **ConstraintLoad**: Calculated. At CSD. • **SpreadLoad**: Calculated. At CSD unless ConstraintAvailable defined. • **ConstraintUsed**: At CSD.	Future if Cumulative Max exceeded, otherwise considers materials only
LoadOnly	Ignore.	• **ConstraintLoad**: At Constraint Start Date (CSD). • **SpreadLoad**: Between CSD and LCF. • **ConstraintUsed**: At CSD.	Considers material availability, but ignores constraint
Unconstrained	Ignore.	• **ConstraintLoad**: Null • **SpreadLoad**: Null • **ConstraintUsed:** Null	Ignores constraint
PlanningOnly	Uses Constraint Rate and Max Constraint for source selection, and to set CalcStartDate.	• **ConstraintLoad**: At Constraint Start Date (CSD). • **SpreadLoad**: Between CSD and LCF. • **ConstraintUsed**: At CSD.	Considers material availability, but ignores constraint
UnconstrainedPlan	Ignore.	• **ConstraintLoad**: Null • **SpreadLoad**: Null • **ConstraintUsed**: Null	Calculated using materials and constraint available, as well as Cumulative Max

Note: For more information about **ProcessingRule** and the **ConstraintType** table, see "ConstraintType table" on page 700.

Rescheduling rules

The following fields determine whether or not RapidResponse can reschedule a scheduled receipt:

- **SupplyType.ProcessingRule**—if this control value is set to **NonReschedulable**, then the effective due date on the **ScheduledReceipt** record cannot be adjusted. However, if this field is set to **Reschedulable**, then the effective due date may be adjusted (still depends on the value of **SupplyStatus.RescheduleRule**). For more information about this field, see "SupplyType table" on page 919.
- **SupplyStatus.RescheduleRule**—if this control value is set to **NonReschedulable**, then the effective due date on the **ScheduledReceipt** record cannot be adjusted. However, if this field is set to **Reschedulable**, then the effective due date may be adjusted. For more information about this field, see "SupplyStatus table" on page 904.
- **SupplyStatus.ConstraintRule**—if this control value is set to Consume, and if the selected part source has ConstraintType.ProcessingRule set to Constrained or CumMax, then the scheduled receipt is seen as NonReschedulable. It is recommended that this value always be set to **BeforeAD**. For more information about this field, see "SupplyStatus table" on page 904.

The following table shows how the values in the ProcessingRule and RescheduleRule fields affect the due date of a **ScheduledReceipt** record.

Table 28.3: Results of SupplyType and SupplyStatus fields

Value of SupplyStatus.RescheduleRule	Value of SupplyType.ProcessingRule	Result
FromOrder	Reschedulable	Reschedulable
FromOrder	Ignore	Ignore
FromOrder	NonReschedulable	NonReschedulable
FromOrder	RecommendOnly	RecommendOnly
FromOrder	ExplodeOnly	NonReschedulable
RecommendIfType	Reschedulable	RecommendOnly
RecommendIfType	Ignore	Ignore
RecommendIfType	NonReschedulable	NonReschedulable
RecommendIfType	RecommendOnly	RecommendOnly
RecommendIfType	ExplodeOnly	NonReschedulable
NonReschedulable	Reschedulable	NonReschedulable
NonReschedulable	Ignore	Ignore
NonReschedulable	NonReschedulable	NonReschedulable
NonReschedulable	RecommendOnly	NonReschedulable
NonReschedulable	ExplodeOnly	NonReschedulable

Table 28.3: Results of SupplyType and SupplyStatus fields

Value of SupplyStatus. RescheduleRule	Value of SupplyType. ProcessingRule	Result
Reschedulable	Reschedulable	Reschedulable
Reschedulable	Ignore	Ignore
Reschedulable	NonReschedulable	Reschedulable
Reschedulable	RecommendOnly	Reschedulable
Reschedulable	ExplodeOnly	NonReschedulable
RecommendOnly	Reschedulable	RecommendOnly
RecommendOnly	Ignore	Ignore
RecommendOnly	NonReschedulable	RecommendOnly
RecommendOnly	RecommendOnly	RecommendOnly
RecommendOnly	ExplodeOnly	NonReschedulable
Scheduled	Reschedulable	NonReschedulable
Scheduled	Ignore	Ignore
Scheduled	NonReschedulable	NonReschedulable
Scheduled	RecommendOnly	NonReschedulable
Scheduled	ExplodeOnly	NonReschedulable

Note: The **SupplyStatus.ExpediteLimit** control value limits the extent of expediting scheduled receipts. For example, if you set this field to **DueDate**, RapidResponse is unable to adjust **ScheduledReceipt.ReschedDate** before **DueDate**. For more information about the **ExpediteLimit** field, see "SupplyStatus table" on page 904.

ConstraintDateRule

The **ConstraintDateRule** option is set on the **PartSourceType** table. This option is used to help determine the following:

- When netting applies constraint
- How planned orders are generated
- How CTP calculates AvailableDate

This section discusses the five settings available for **ConstraintDateRule**; Start, StartExtend, Ship, ShipExtend, and DuringLeadTime.

Note: Certain **ConstraintDateRule** options will automatically disable variable lead time (set through **PartSource.VarLeadTime**) as noted throughout this section.

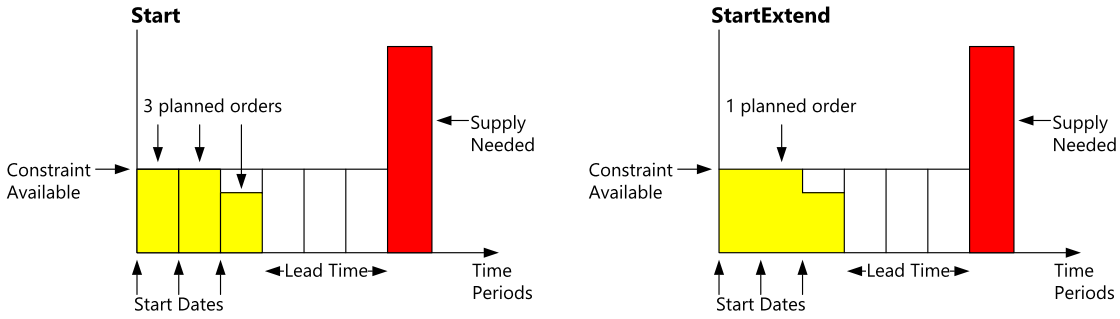
Start and StartExtend

Start and StartExtend both apply constraint starting at the start date for the order. Typically, these settings are used for internal production constraints and affect planned order generation as follows:

- **Start**—if only variable constraint is consumed, then a new planned order is generated for each period in which constraint is used. The AvailableDate for the order is calculated as a stock date from the start date of the last constraint allocated to the supply. If fixed constraint is consumed, either on its own or in addition to variable constraint, then this option is treated the same as StartExtend.
- **StartExtend**—a single planned order is generated for the entire supply, with the start date for the order extended if additional constraint periods are required. The AvailableDate for the order is calculated as a stock date from the start date of the last constraint used.

The following illustration shows an example of how planned orders for a supply could be generated in cases where the Start and StartExtend rules were used. This example assumes only variable constraint consumption.

Figure 28.15 Planned orders using Start and StartExtend



Notes:

- When either the "Start" or "StartExtend" options are enabled, variable lead time as set through **PartSource.VarLeadTime** is automatically disabled. This is because available constraint helps determine the StartDate, DueDate, and ShipDate for the supply with lead time thereby extended or reduced based on supply quantity. Similarly, the **VarLeadTime** field represents a variable portion of lead time that is based on supply quantity. Thus, these two settings should not be combined together as the variable lead time may already account for the constraint run time (or similarly, constraint run time may already account for variable lead time). Thus, RapidResponse automatically disables the variable portion of lead time set through the **VarLeadTime** field.
- When either the "Start" or "StartExtend" options are enabled, variable lead time as set through **PartSource.VarLeadTime** is automatically disabled. This is because available constraint helps determine the StartDate, DueDate, and ShipDate for the supply with lead time thereby extended or reduced based on supply quantity. Similarly, the **VarLeadTime** field represents a variable portion of lead time that is based on supply quantity. Thus, these two settings should not be combined together as the variable lead time may already account for the constraint run time (or similarly, constraint run time may already account for variable lead time). Thus, RapidResponse automatically disables the variable portion of lead time set through the **VarLeadTime** field.

- The illustration above assumes a **PartSourceType.PrePlanRule** setting of "Normal" (the default setting). However, when the **PartSourceType.PrePlanRule** setting of "JustInTime", is used along with a **ConstraintDateRule** of "Start", then constraint consumption might be different than shown above. For more information, see "PrePlanRule" on page 1676.

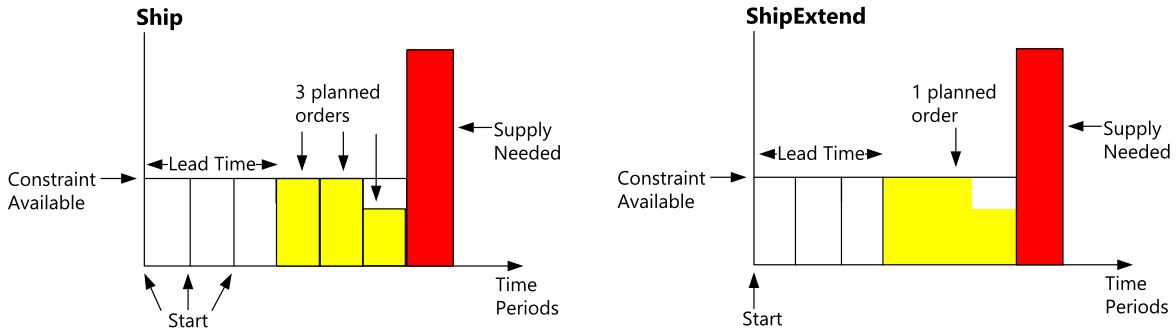
Ship and ShipExtend

Ship and ShipExtend both apply constraint starting at the ship date for the supply. Typically, these settings are used for supplier constraints and affect planned order generation as follows:

- **Ship**—if only variable constraint is consumed, then a new planned order is generated for each period in which constraint is used. The AvailableDate for the order is calculated as a stock date from the ship date of the last constraint allocated.
- **ShipExtend**—a single planned order is generated for the entire supply, with the start date for the order extended if additional constraint periods are required. The AvailableDate for the order is calculated as a stock date from the start of the last constraint used as a ship date.

The following illustration shows how planned order(s) for a supply might be generated in cases where the Ship and ShipExtend rules were used. This example assumes only variable constraint consumption.

Figure 28.16 Planned orders using Ship and ShipExtend



Notes:

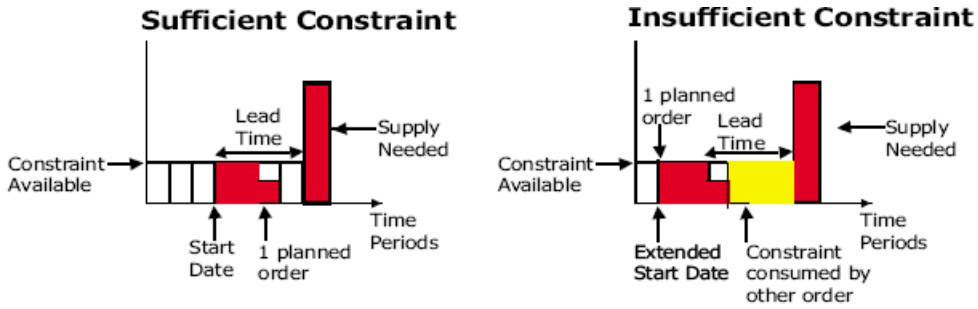
- "Ship" and "ShipExtend" are the only **ConstraintDateRule** settings which are compatible with variable lead time as defined in **PartSource.VarLeadTime**. With these settings, when converting consumed period to DueDate/AvailableDate, lead time elements between ShipDate and DueDate are used (no variable lead time). However, when converting consumed period to StartDate, lead time elements between ShipDate and StartDate are used (including any variable lead time). Finally, when applying material available availability in Capable-to-Promise logic, the material start is converted to ShipDate with variable lead time respected in that conversion.
- The illustration above assumes a **PartSourceType.PrePlanRule** setting of "Normal" (the default setting). However, when the **PartSourceType.PrePlanRule** setting of "JustInTime", is used along with a **ConstraintDateRule** of "Ship", then constraint consumption might be different than shown above. For more information, see "PrePlanRule" on page 1676.

DuringLeadTime

DuringLeadTime applies constraint between an order's start date and ship date. A single planned order is created for each supply. If insufficient constraint is available during lead time, then the order's start date is extended. AvailableDate for the order is calculated as the later of (a) the stock date from the ship date, using the last constraint allocated, and (b) the stock date from the start date.

The following illustration shows how a planned order for a supply might be generated using the DuringLeadTime setting, both when sufficient constraint is available during lead time and when it is not. Notice that start date for the order is only extended in the case where insufficient constraint is available.

Figure 28.17 Planned orders using DuringLeadTime



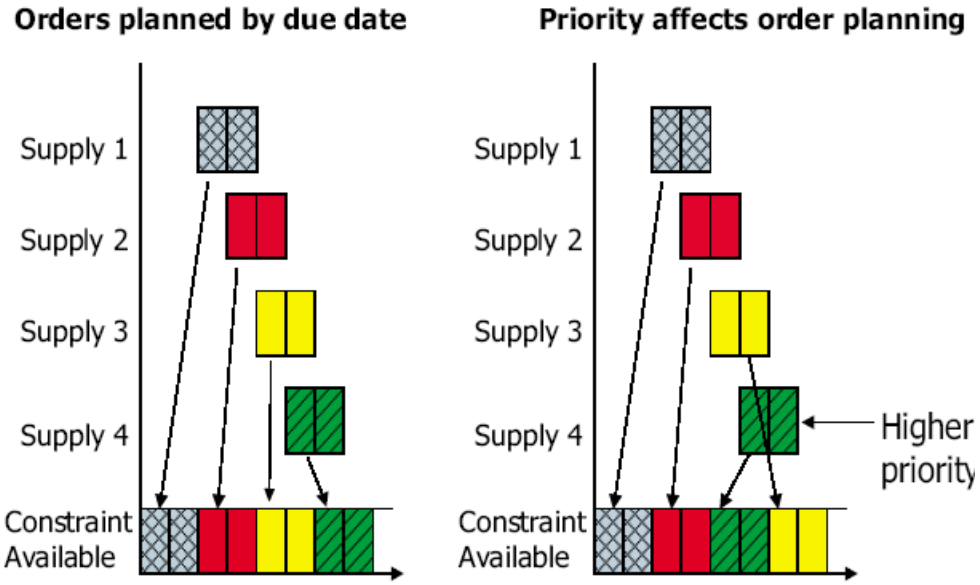
Note: When the "DuringLeadTime" option is enabled, variable lead time as set through **PartSource.VarLeadTime** is automatically disabled. This is because available constraint helps determine the StartDate, DueDate, and ShipDate for the supply with lead time thereby extended or reduced based on supply quantity. Similarly, the **VarLeadTime** field represents a variable portion of lead time that is based on supply quantity. Thus, these two settings cannot be combined together as the variable lead time may already account for the constraint run time (or similarly, constraint run time may already account for variable lead time). Thus, RapidResponse automatically disables the variable portion of lead time set through the **VarLeadTime** field.

OrderPriority rules

When a constraint is overloaded, RapidResponse will pre-plan orders (move them forward) to make use of available constraint. Typically, orders should be planned in due date sequence, with some orders planned early if necessary to ensure the due dates for other supplies can be met. The exception to this is where a higher priority order will not have enough available constraint to meet its due date. In this case, the higher priority order can be bumped ahead of lower priority order which will then finish late.

The following illustration shows an example of the two cases discussed above. In the image on the left, all supplies are assumed to have the same priority and are planned in due date sequence. Note that Supply 1 and Supply 2 are planned early, and that Supply 4, the latest due order, finishes late. However, in the image on the right, Supply 4 is assumed to have a higher priority than the other orders. Therefore, it is bumped ahead of Supply 3 to ensure that it finished on time. This also means that Supply 3 now finishes late.

Figure 28.18 Effect of priority on order planning



Fields that determine Priority

The following table shows the fields that affect how priority is used in RapidResponse calculations.

Table 28.4: Fields determining priority values

Field	Description
OrderPriority.PlanningPriority	An integer that determines the importance associated with each value in the **OrderPriority** table. Lower values indicate higher priorities.
OrderPriority.MaintainCommitments	Indicates whether or not TransactionSequence is considered as part of priority.
PartType.CommitLevel	Indicates where priority (PlanningPriority + MaintainCommitments) is used. Values are: • **High**: Priority used in both Netting and CTP. • **Med**: Priority used in CTP, but not Netting. • **Low**: Priority not used in either CTP or Netting.

Notes:

- For more information about PlanningPriority, MaintainCommitments, and the **OrderPriority** table, see "OrderPriority table" on page 782. For more information about CommitLevel and the **PartType** table, see "PartType table" on page 809.
- The **OrderPriority.SchedulePriority** field can be used to allow higher priority orders to consumer constraint before lower priority orders. This results in higher priority orders being scheduled as close as possible to their due date. For more information, see "SchedulePriority" on page 1659.

Sample Priority values

The following table shows a typical set of OrderPriority values, and the type of supplies that might be associated with each.

Table 28.5: Typical priority values

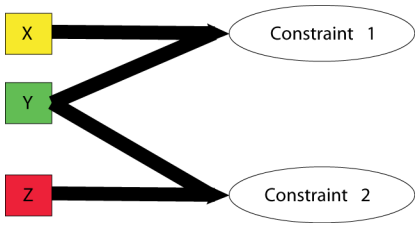
Planning Priority	Value	Maintain Commitments	Used For
0	Open	N	Open work orders already in production.
1	High	Y	Highest priority customer orders. That is, those orders that can bump normal orders if necessary.
2	Normal	Y	Normal customer orders. Can be used for order promising, but these orders can also be bumped by high priority orders.
3	Low	N	Low priority customer orders. These orders may frequently be bumped by normal and high priority orders.
4	Forecast	N	Satisfying forecasted demand. These orders are superseded by all real customer orders.
5	Buffer	N	Lowest priority demands (for example, safety stock).

Note: It is recommended that each OrderPriority record be given a distinct PlanningPriority value.

SchedulePriority

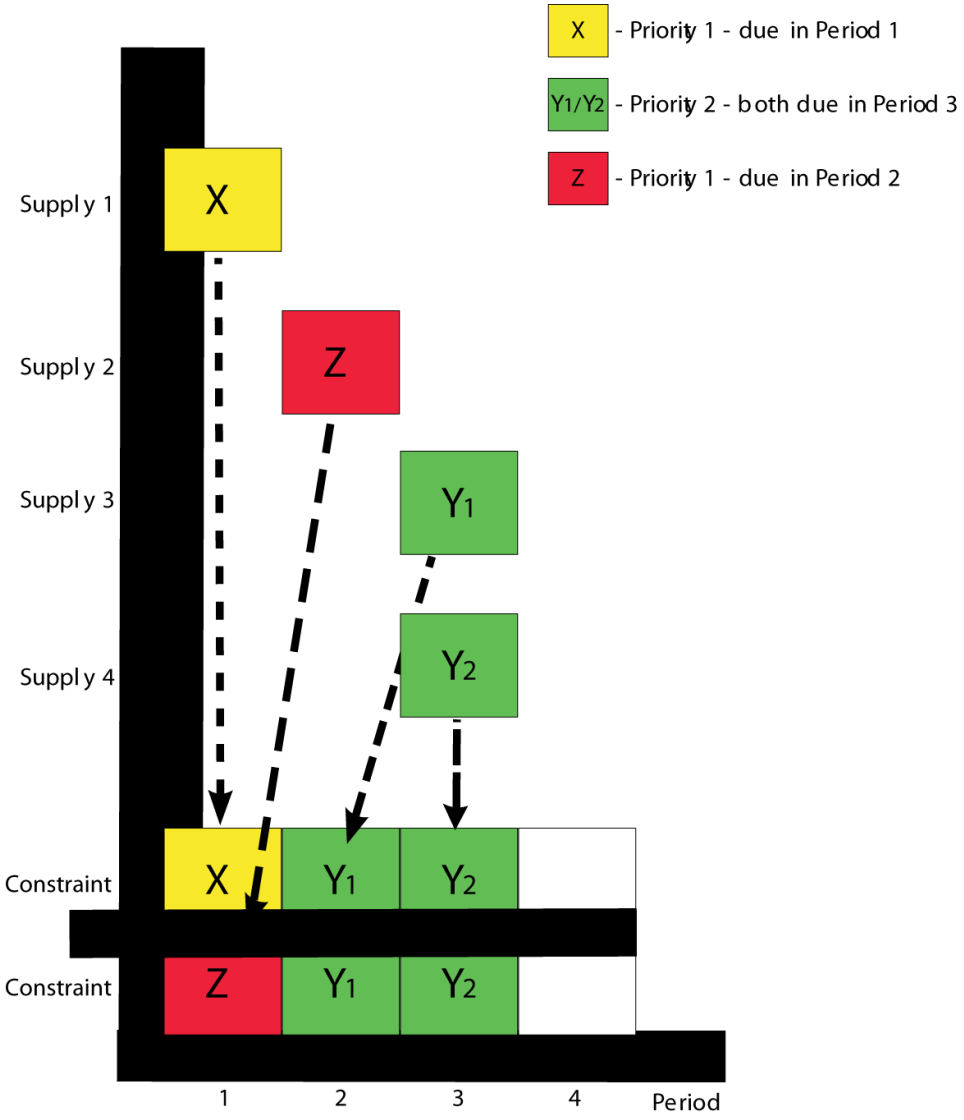
RapidResponse schedules all supplies belonging to a given family together. A family consists of all parts and constraints related through **SourceConstraint** relationships. The following illustration shows a simple example, used throughout this section, of a family consisting of parts X, Y and Z (for more information on families, see "Families and multi-level constraints" on page 1633).

Figure 28.19 Constraint family example



Typically, RapidResponse tries to plan supplies in line with their due date. If there is not sufficient constraint available, then some supplies might be planned early (pre-planned) in order to ensure that other supplies can find enough constraint to meet their due date. This can result in situations where high-priority supplies are pre-planned in order to ensure that lower priority supplies can be available on time. This is illustrated in the following example, where Supply Z is planned one period early in order to ensure enough constraint is available to build both Supply Y1 and Y2 on time.

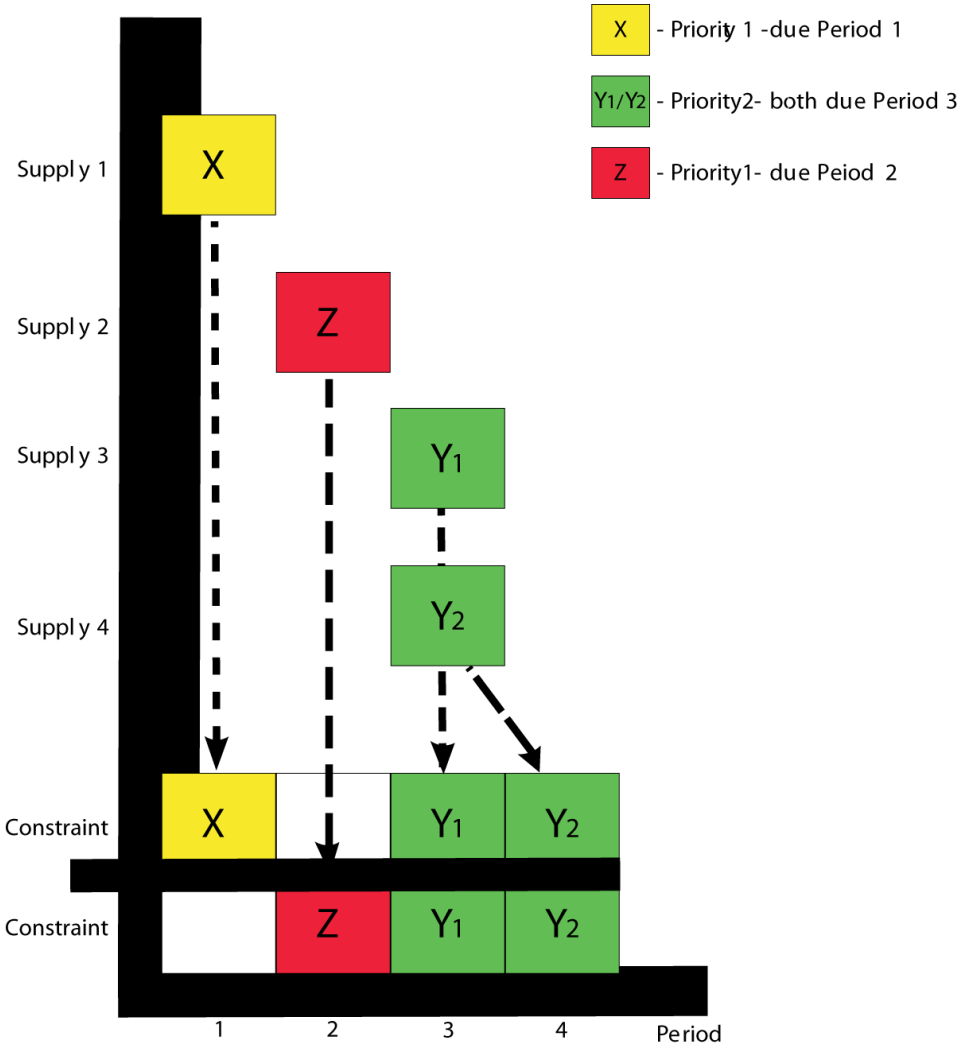
Figure 28.20 Constraint scheduling with SchedulePriority turned "Off"



The **OrderPriority.SchedulePriority** field can be used to configure RapidResponse to plan high-priority supplies as close to their due date as possible. This might be done in cases where high priority supplies are of very high value to ensure they are held in inventory for as little time as possible.

When using SchedulePriority, supplies within a family are grouped and scheduled by their schedule priority. High priority supplies are scheduled first and planned as close to their due date as possible. Lower priority supplies are then scheduled onto whatever constraint remains after the higher-priority supplies have been scheduled. For example, the following illustration shows a case where higher priority supplies X and Z are scheduled first and built to be ready just in time for their due dates in Period 1 and 2 respectively. Lower priority supplies Y1 and Y2 and then scheduled after that and cannot find enough remaining constraint until Periods 3 and 4 respectively. The net result is that Y2 is now late, and contrasts with the example shown in the previous figure, where the pre-planning of Supply Z allowed all supplies to be built on time.

Figure 28.21 Constraint scheduling with SchedulePriority turned "On"



Considerations for using SchedulePriority

By default in RapidResponse, SchedulePriority has no effect (all values are set to 0). If your company opts to use SchedulePriority, the following should be kept in mind:

- The values specified in the **SchedulePriority** field should increase in line with the values in the **Planning Priority** field. For example:

Value	PlanningPriority	SchedulePriority
High	10	0
Medium	20	1
Low	30	2

- SchedulePriority respects the setting in the **PartType.CommitLevel** field. This field should be set to 'High' to ensure priorities are propagated from demands to planned supplies.

- SchedulePriority does not consider the **TransactionSequence** value (regardless of the setting of **OrderPriority.MaintainCommitments**).
- Lower priority supplies are more likely to be made late (as illustrated in "Constraint scheduling with SchedulePriority turned "On"" on page 1661), or have their constraint consumption split across periods.
- To turn SchedulePriority off, all **OrderPriority.SchedulePriority** values should be set to the same number (for example, 0).

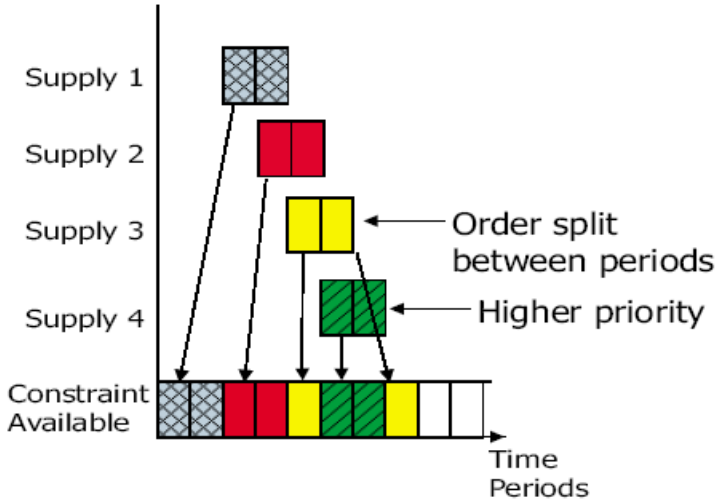
SplitLateSupply

The SplitLateSupply option is set on the **SupplyStatus** table. When dealing with a set of planned orders where load exceeds the constraint available for a given period, this gives the option of whether or not RapidResponse should always plan to use contiguous blocks of constraint for a given order.

Typically, orders are planned to finish on time and constraints are allocated in contiguous blocks. However, when a constraint is overloaded, this is not always possible. If SplitLateSupply is set to Y, RapidResponse, when necessary, attempts to split the allocation of constraint between the part of the supply that can be finished on time, and the part that cannot. Note that RapidResponse only attempts to split the allocation of constraint if all constraint consumption for a given order is variable. That is if any fixed constraint is consumed, the allocation of constraint will not be split and the SplitLateSupply setting is treated as if it was set to N.

The following illustration shows how constraint might be allocated in a case where SplitLateSupply is set to Y. Note that Supply 1 and Supply 2 can both be finished on time. Supply 3 could also be finished on time, but this would ensure that Supply 4, which has a higher order priority associated with it, would finish late. Therefore, half work begins on Supply 3, then switches to Supply 4 until it is completed (on time), and then the late portion of Supply 3 is completed.

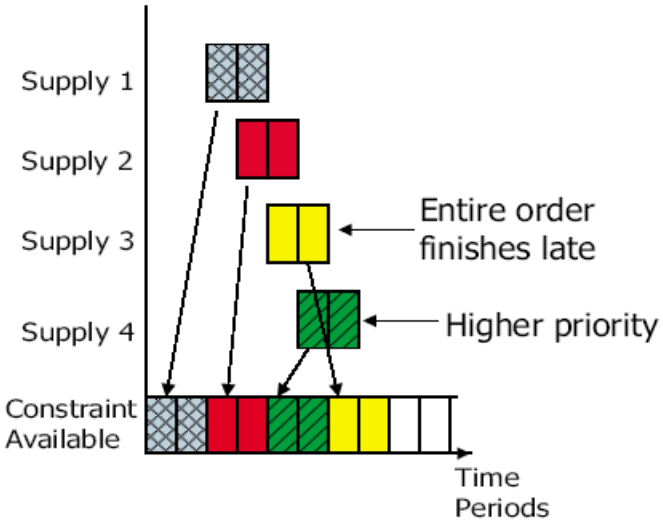
Figure 28.22 SplitLateSupply set to Y



Setting SplitLateSupply to N, forces RapidResponse to try to use contiguous blocks of constraint for a given order. Therefore, if an entire order cannot be finished on time, RapidResponse instead builds a higher order priority order (which may finish early), and then builds the lower priority order later. This can help minimize the setup and tear down costs associated with switching back and forth between work on separate orders.

The following illustration shows the identical supplies and priorities as in "SplitLateSupply set to Y" on page 1662, but indicates how constraint is allocated when SplitLateSupply is set to N. Supply 1 and 2 are planned as before. However, because orders are being kept together, all of Supply 3 is pushed out and finished late. As a result work on Supply 4 (the higher priority order) starts sooner and the order is completed early.

Figure 28.23 SplitLateSupply set to N



Note: For more information about SplitLateSupply and the **SupplyStatus** table, see "SupplyType table" on page 919.

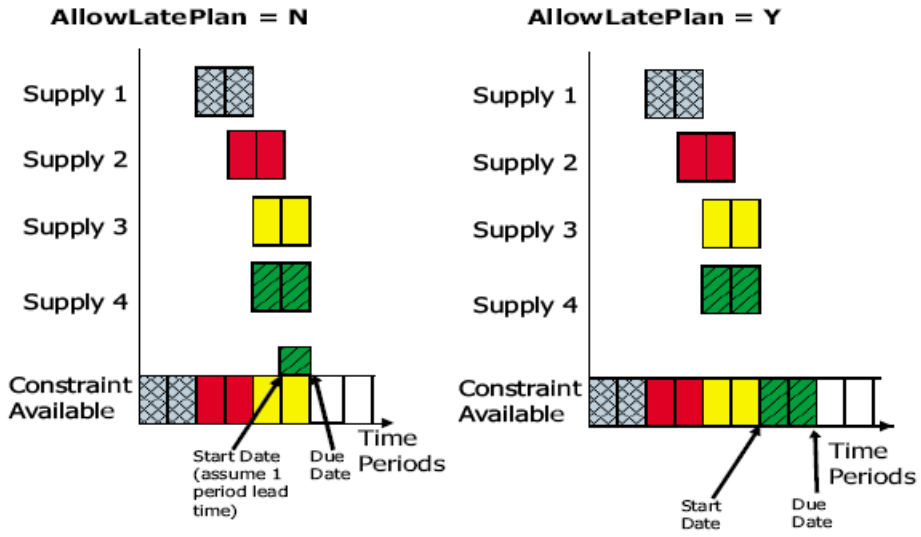
AllowLatePlan

The AllowLatePlan option is set on the **SupplyStatus** table. This controls whether or not a supply's start date and due date can be later than its associated demand date in cases where sufficient constraint is not available when needed.

Typically, supplies are planned to be due when needed (that is, to meet the demand date). Dependent demands are planned for the corresponding start date. This is the case even if there is not sufficient constraint to complete the supply when needed. In some cases, it may be preferable to plan supplies in line with constraint availability. For example, by planning supplies to be ready only when there is available capacity, the cost of holding these supplies in inventory can be reduced.

The following illustration shows an example of how a supply's start and due dates would look when the AllowLatePlan option is set to Y and N respectively. When AllowLatePlan is set to N, the supply is planned to meet the demand due date. However, if AllowLatePlan is set to Y, the supply is planned to start only when there is available capacity. All dependent demands for the supply are then planned to satisfy this later date.

Figure 28.24 AllowLatePlan effect on supply due dates



Notes:

- If using multi-level search logic, then the **AllowLatePlan** setting of "Y" also allows for the due dates on an assembly's supplies to be later than the demand due date when any of its components will be late due to insufficent constraint or material availability. For more information about multi-level search, see "Multi-level search logic" on page 1863.
- For more information about AllowLatePlan and the **SupplyStatus** table, see "SupplyStatus table" on page 904.

Multi-level constraints and AllowLatePlan

When dealing with multi-level constraints, all assembly supplies in a family are planned before any component supplies in the family. In cases where a constraint is overloaded, assemblies might consume all available constraint between the date the constraint is first available, and the date it is needed to create supplies to satisfy demand. As a result, no constraint would remain to plan component supplies.

The situation described above is affected by the **AllowLatePlan** setting. Specifically, if **AllowLatePlan** is set to 'Y', then no constraint would be available for component supplies until after the demand date, and component supplies would not be planned until after their assembly supplies. However, if AllowLatePlan is set to 'N', then supplies are allowed to overload the constraint, and the Capable-to-Produce analytic determines which supplies receive constraint first.

Therefore, it is recommended that **AllowLatePlan** be set to 'N' for all SupplyStatus values referenced by scheduled receipts or planned orders for components belonging to multi-level families.

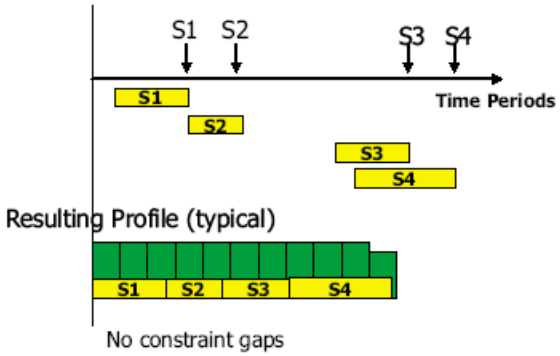
Note: For information on multi-level constraints, see "Families and multi-level constraints" on page 1633.

FillSchedule

The FillSchedule option is set on the **ConstraintType** table. Setting this option to Y for a constraint forces Netting to try to fill up the schedule for a constraint by planning to use as much of it as early as possible (subject to component availability). For example, for some key constraints it may be desirable to plan the use of constraint early so if opportunity arises later, there will be available capacity that would not otherwise have been there without this pre-planning. Note that when using numerous constraints or lot sizing on constraint use, it may not always be possible to use all constraint available.

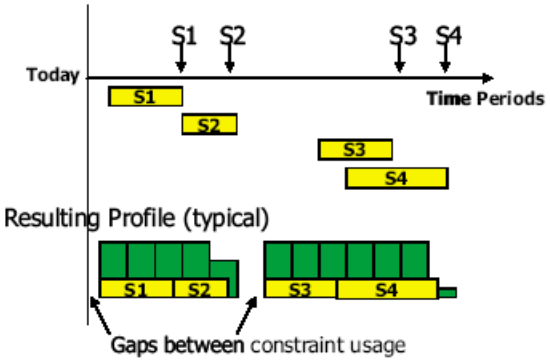
The following illustration shows four supplies, with their due dates, and a typical constraint load profile that might result from having the FillSchedule option set to Y. Note that the allotment of constraint begins as early as possible, and continues until all supplies have been satisfied with constraint. This results in minimizing any gaps in usage of the constraint.

Figure 28.25 Fill Schedule set to Y



If the FillSchedule option is set to N, then netting only plans to use enough constraint to satisfy demand due dates. This can result in gaps in the usage of a constraint for periods where its allocation is not strictly needed to satisfy due dates. The following illustration uses the same supplies and due dates as the previous illustration, but shows a typical constraint load profile that might result from having the FillSchedule option set to N.

Figure 28.26 Fill Schedule set to N



Note: For more information about FillSchedule and the **ConstraintType** table, see "ConstraintType table" on page 700.

AvailableDateLimit

As discussed earlier in this chapter, limitations on constraint and material availability can make the AvailableDate for a supply later than it would otherwise be. In addition to these limitations, all supplies have an absolute limit on their AvailableDate. This limit depends on whether the supply is a planned order or a scheduled receipt. If the supply is a scheduled receipt, this limit further depends on the state of the supply as outlined in the following table.

Table 28.6: AbsoluteLimit for supplies

Supply type	State	AbsoluteLimit
PlannedOrder	• All	Part.PlanningCalendars.RunDate + lead time
ScheduledReceipt	• BackflushAndAllocated • BackflushButNotAllocated • ReleasedButNotAllocated • ReleasedAndAllocated	Part.PlanningCalendars.DataDate
ScheduledReceipt	• Planned • NotReleased • NotReleasedAndAllocated	Part.PlanningCalendars.DataDate + lead time

Finally, in addition to the AbsoluteLimit and the limitations imposed by constraint and material availability, you can specify how early, before its due date that RapidResponse can set the AvailableDate for a supply. This option is set on the AvailableDateLimit field of the **SupplyStatus** table. The settings available for this field are defined in the following table:

Table 28.7: AvailableDateLimit settings

Setting	Limits on AvailableDate
Now	If a supply order is unconstrained, it can be built as soon as materials are available subject to the absolute limits defined in the preceding table. If all material is available before the FirstEffectiveDate on the BillOfMaterials record through which the demand exploded, the available date on the supply is still reported based on material date and might end up being seen as available before the beginning of the associated BOM effectivity range. If a supply order is constrained and there is sufficient constraint to complete the order on time, then it will not start consuming constraint until its calculated start date (even if materials are available before that date). If a supply order is constrained and there is insufficient constraint to complete the order on time, then it can start consuming constraint as needed to meet the supply's rescheduled date subject to the absolute limits defined in the preceding table.
CalcRequest Start	Build the supply based on its **CalcRequestStartDate.** Supply cannot be available earlier than **CalcRequestStartDate** + lead time, and constraint cannot be allocated any earlier than **CalcRequestStartDate** converted to a constraint date. **Note**: If the calculated start date is earlier than this date, then it becomes the available date limit instead ("CalcStart" setting is used)

Table 28.7: AvailableDateLimit settings

Setting	Limits on AvailableDate
CalcStart	Build the supply based on its original start date or its calculated start date (through netting). The available date for the supply is limited by material and constraint availability, as well as the AbsoluteLimit defined in the preceding table. In addition, constraint cannot be allocated before CalcStart (StartDate for planned orders) converted to a constraint date. Therefore, AvailableDate is further limited by CalcStart plus lead time.
Due	Build the supply so the earliest it can be completed is the order's due date. The available date for the supply is limited by material and constraint availability, as well as the AbsoluteLimit. In addition, constraint cannot be allocated before EffStartDate (StartDate for planned orders) converted to a constraint date. Therefore, AvailableDate is further limited by EffDueDate (DueDate for planned orders).

Notes:

- For more information about AvailableDateLimit and the **SupplyStatus** table, see "SupplyStatus table" on page 904.
- When push-planning logic (distribution planning) is used, some pushed supplies might end up with available dates that are later than the limit determined by the AvailableDateLimit setting.

SchedulingRule

After demands for the current part or family are collected into lists sorted primarily by priority and/or due date, constrained scheduling has two main phases within the Netting analytic. These are a "backward" scheduling phase and an optional "forward" scheduling phase.

Backward Phase

The backward phase is performed first and refers to the direction in which constraint is looked for beyond the current constraint period. That is, if there is insufficient constraint to satisfy a demand in the current bucket, then earlier periods are next looked to for additional constraint (subject to any pre-planning limits).

In this phase, sorted and prioritized demands are processed one by one, and supplies are assigned or created as needed to satisfy those demands. When new planned orders are created, they consume constraint based on demand date, applicable pre-planning settings, and constraint availability.

To support the subsequent forward phase, the logic in this phase also keeps tracks of the following two items on each constrained supply:

- The earliest constraint period in which the supply consumed constraint (know as **LatestConstraintStart** or **LCS**).
- The later of either the latest constraint period in which the supply consumed constraint or the demand date (known as **MaxConstraintDemand** or **MaxCD**).

Forward Phase

The constraint consumption picture determined in the backward phase is then cleared so that the forward phase can begin. This phase refers to the order in which constraint is consumed, and here it is supplies rather than demands that are sorted. The main purpose of this phase is to identify supplies

that use constraint in multiple buckets and realign supply constraint usage such that supplies are processed in continuous blocks where possible. This can help reduce switchover costs and minimize fixed constraint usage.

The following three general steps occur in this stage

1. A zone from each supplies **LatestConstraintStart** to its **MaxConstraintDemand** is determined:
   - For supplies that share the same constraint, this phase then identifies supplies whose zone fully engulfs another supply.
   - The **LatestConstraintStart** on supplies whose zone is engulfed by another is then extended and pushed earlier to match the other supply. This adjusted **LCS** is used in the third step described below.
2. Supplies are sorted into a prioritized list:
   - Scheduled receipts are sorted first (given preference).
   - Planned orders are sorted next. Their priority is determined primarily by ordered factors such as **MaxCD** (earliest), **DueDate** (earliest), and **OrderPriority** (highest).
3. The forward scheduling of constraints is performed:
   - Based on the sorted supply list, each supply consumes constraint in turn (one at a time).
   - Each supply starts looking for constraint from its modified **LCS** (respecting absolute pre-plan limits, if applicable) and moving forward in time to find required constraint.
   - Based on forward constraint consumption, planned order dates are adjusted accordingly, and if **ConstraintDateRule** is set to "Start" or "Ship" then the supplies are split into multiple orders based on this consumption.

Disabling forward scheduling

Optionally, the forward phase of constraint scheduling can be disabled so that only backward scheduling is performed. This is configured using the **ConstraintType.SchedulingRule** field which has the following settings:

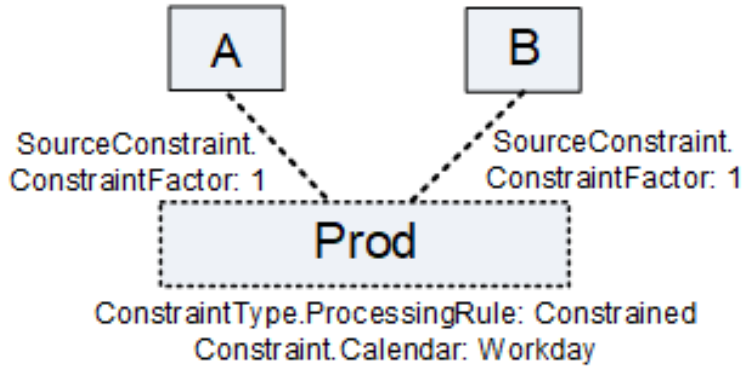
- **Normal**—Both the backward and forward phases of constraint scheduling are performed in Netting. This option should be used when it is important for supplies to use constraint in continuous blocks . If constraint pre-planning is required, then higher priority supplies will typically be pre-planned earlier. This is the default option and its usage provides backward compatibility with RapidResponse versions prior to Service Update 1906.
- **BackloadOnly**—Only the backward phase of constraint scheduling is performed in Netting. This option can be used when normal constraint scheduling gives sub-optimal results. Typically, this is used for cases where a part source requires multiple constraints. In such cases, the forward phase of scheduling might not always be able to find availability for all required constraints in a given period which can push supplies to be due later than necessary. If constraint pre-planning is required, then lower priority supplies will typically be pre-planned earlier.
  If using this setting, ensure the applicable **ConstraintType.ProcessingRule** is set to "Constraint" and the **PartSourceType.ConstraintDateRule** is set to either "Ship" or "Start". For more details on configuration limitations that exist with the "BackloadOnly" setting, see "Limitations on "BackloadOnly" scheduling" on page 1675.

Example 1: High changeover costs ("Normal" is preferred)

Suppose constraint "Prod" represents a machine with high changeover costs, and this constraint is required to build both parts "A" and "B" as shown below (with each requiring 1 unit of constraint per unit of supply built). Each part is further assumed to have a lead time of 1 day, and use a

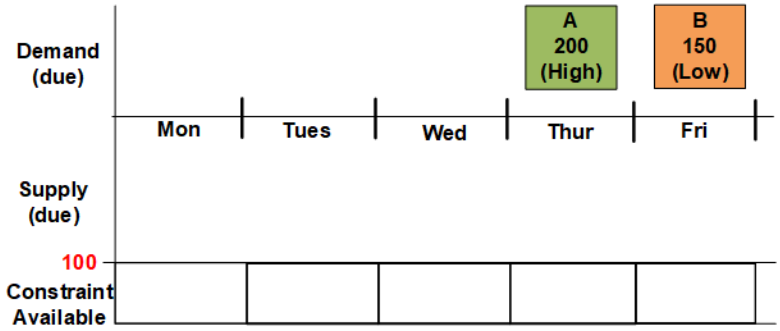
PartType.CommitLevel setting of "High", and each part source is assumed to use a **PartSourceType.ConstraintDateRule** setting of "Start".

Figure 28.27 Parts with a shared constraint



Further suppose the following prioritized demands for parts "A" and "B", with no on hand or scheduled supply, and enough constraint available to build 100 units of total supply per day.

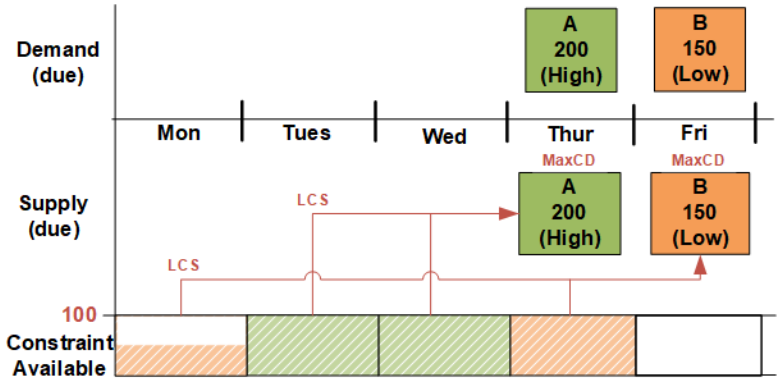
Figure 28.28 Prioritized demands and available constraint



With either **SchedulingRule**, the backward phase first creates planned supplies that use constraint as shown below. In this case, the higher priority demand for "A" due on Thursday is processed first and uses up all 100 units of constraint on Wednesday before moving back and using all 100 units of constraint available on Tuesday. Next, the lower priority demand for "B" due on Friday is processed and first uses all 100 units of constraint available on Thursday. Because there is no constraint remaining on either Wednesday or Tuesday, it then looks further back and finding the remaining 50 units of constraint needed on Monday.

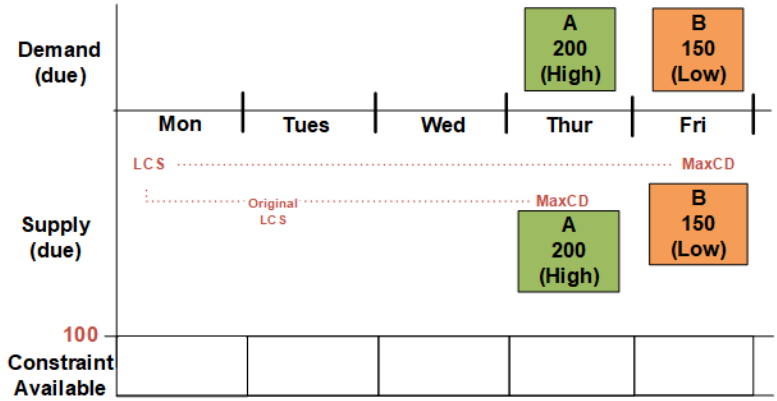
Note in this case that both supplies would have **MaxCD** set to their demand dates of Thursday and Friday, respectively (both demands are being planned on time), and **LatestConstraintStart** would be set to Monday for the supply of "A" and Tuesday for the supply of "B"

Figure 28.29 Backward phase of constraint scheduling



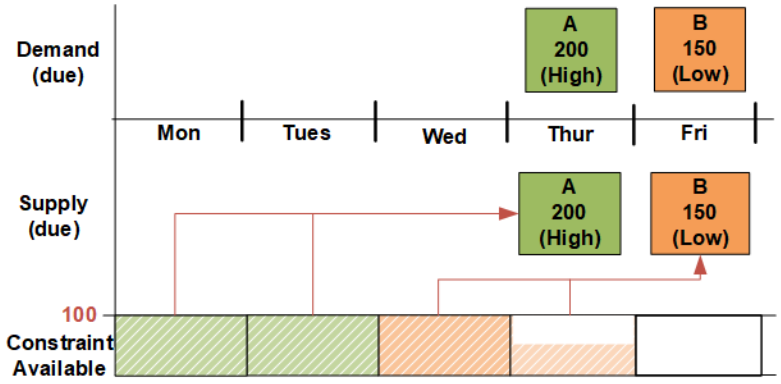
With the default **SchedulingRule** of "Normal", the forward constraint scheduling phase subsequently runs. Here it is determined that the supplies for "A" and "B" each share the same constraint and the zone for the supply of "B" completely engulfs the supply for "A". Thus, the **LatestConstraintStart** on the supply for "A" is moved earlier to match that of "B" as shown in the following illustration.

Figure 28.30 Adjusting LateConstraintStart in forward phase



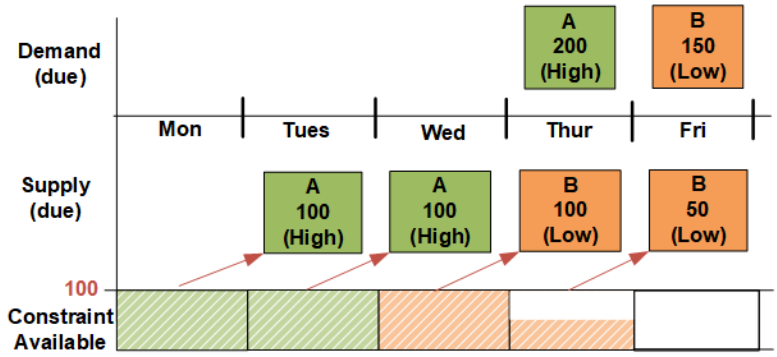
Next the supplies are sorted, with the higher priority supply for "A" being prioritized over "B" in this case due to an earlier **MaximumConstraintDate**. Thus, the supply for "A" starts consuming constraint from its adjusted **LatestConstraintStart** and consumes 100 units of constraint on Monday and again on Tuesday. Next in turn, the supply for "B" would consume 100 units of constraint on Wednesday, and then finds the remaining 50 units of constraint needed the following day on Thursday.

Figure 28.31 Forward phase of constraint scheduling



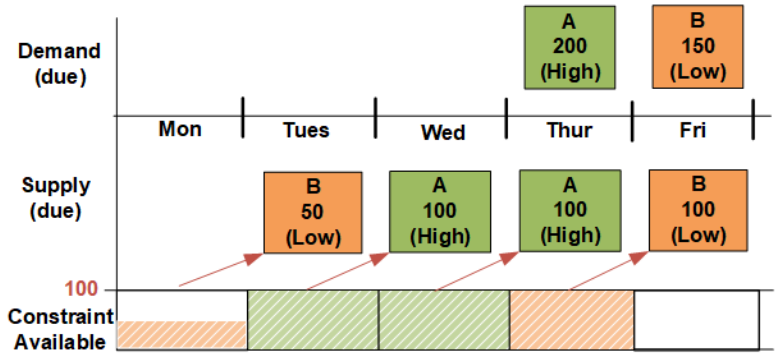
Based on the **ConstraintDateRule** setting of "Start", planned order splits from "Normal" constraint scheduling would be as shown in the following illustration with all supplies planned on time to their demands. And the forward scheduling phase has ensured the supplies consume constraint in continuous chunks with the higher priority supplies being pre-planned earlier.

Figure 28.32 Planned order splits with "Normal" scheduling rule



Instead, if the **SchedulingRule** were changed to "BackloadOnly", then supplies would be planned based on the initial backward phase of constraint scheduling (as seen earlier in this example). Thus, planned orders would be generated as shown in the following image. Note that both demands are still planned on time, however the supplies are no longer loaded on constraints in continuous chunks and the earlier pre-planned supply is a lower priority supply.

Figure 28.33 Planned order splits with "BackloadOnly" scheduling rule

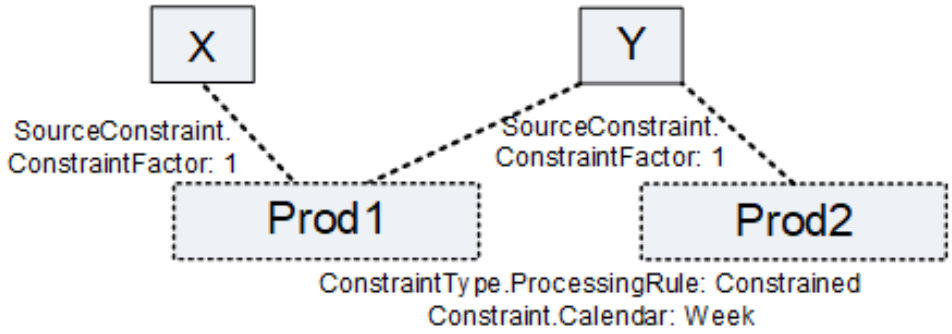


Therefore if there is an assumption of high changeover costs, then the "Normal" setting is generally recommended. However for some use cases, the "BackloadOnly" setting will give better results. See the following example.

Example 2: Parallel constraints with partial planning ("BackloadOnly" is preferred)

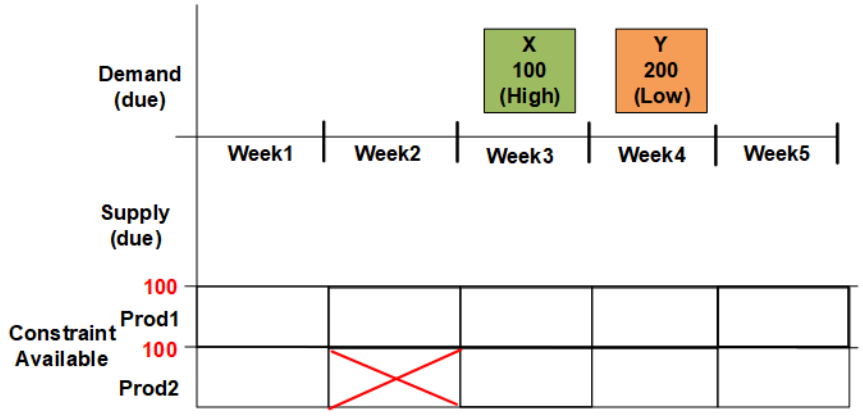
Suppose constraint "Prod1" is required to build both parts "A" and "B", while part "B" also requires constraint "Prod2" in addition to "Prod1". As shown below, each part is assumed to require one unit of each of its applicable constraints to build one unit of supply. Each part is further assumed to have a lead time of 1 week and use a **PartType.CommitLevel** setting of "High, while each part source references a **PartSourceType.ConstraintDateRule** setting of "Start".

Figure 28.34 Shared and parallel constraints



Further suppose the following prioritized demands for parts "X" and "Y". No on hand or scheduled supply is assumed, and there are 100 units of each of constraint of "Prod1" and "Prod2" available each week other than week 2 where "Prod2" is not available and assumed to have been scheduled for maintenance.

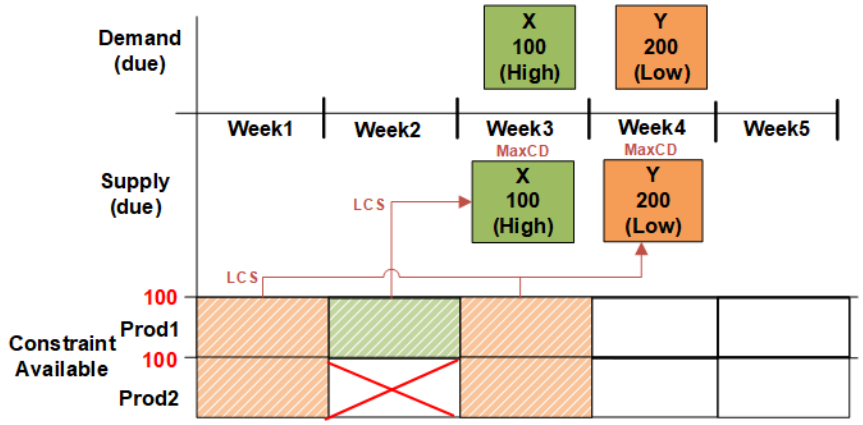
Figure 28.35 Prioritized demands and available constraints



With either **SchedulingRule**, the higher priority demand for "A" due in Week 3 is processed first and uses up all 100 units of constraint available in Week 2 in order to plan the supply on time. The lower priority demand for "B" is processed next and first consumes all 100 units of both constraint available in Week 3. Because there is no available constraint in Week 2, it then looks back to Week 1 to find the

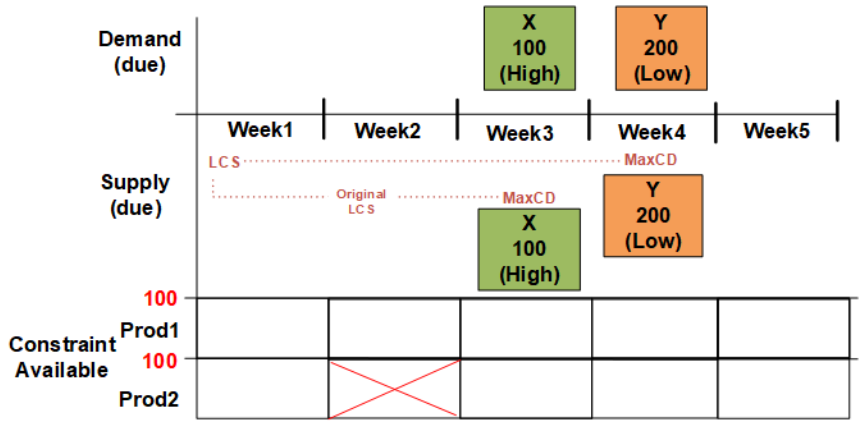
additional 100 units required of each constraint. Note also that each supply would have its **MaxCD** and initial **LatestConstraintStart** set as shown in the following illustration.

Figure 28.36 Backward phase of constraint scheduling



With the default **SchedulingRule** of "Normal", the forward constraint scheduling phase subsequently runs. Because the zone for the supply of "Y" fully engulfs the supply of "X", the **LatestConstraintStart** on "X" is moved earlier to Week 1 to match "Y" as shown in the following illustration.

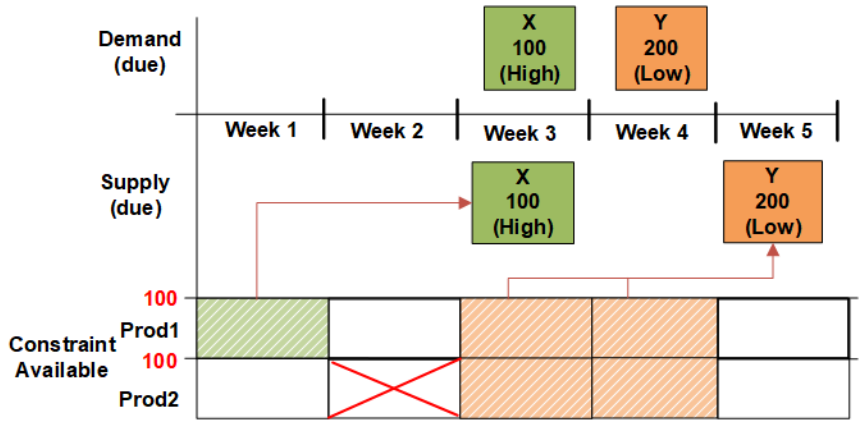
Figure 28.37 Adjusting LatestConstraintStart in forward phase



Next the supplies are sorted, with the higher priority supply for "X" being prioritized over "Y" in this case due to an earlier **MaxCD**. Thus, the supply for "X" starts consuming constraint from its adjusted **LatestConstraintStart** and uses all 100 units of constraint "Prod1" available on Monday.

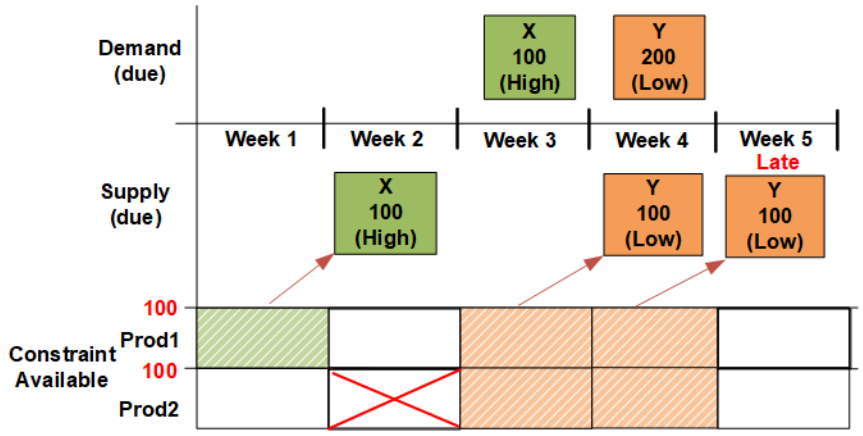
The supply for "Y" is then processed and, starting from its **LatestConstraintStart** (also Monday), looks for availability of both constraints "Prod1" and "Prod2". Supply of this part cannot be built in Week 1 as all of constraint "Prod1" has already been used, nor can it be built on Week 2 as constraint "Prod2" has no availability there. Thus the supply of part "Y" doesn't start consuming constraint until Week 3 where it consumes all 100 units of both constraints "Prod1" and "Prod2", and then again in Week 4 where it consumes all 100 units of each constraint.

Figure 28.38 Forward phase of constraint scheduling



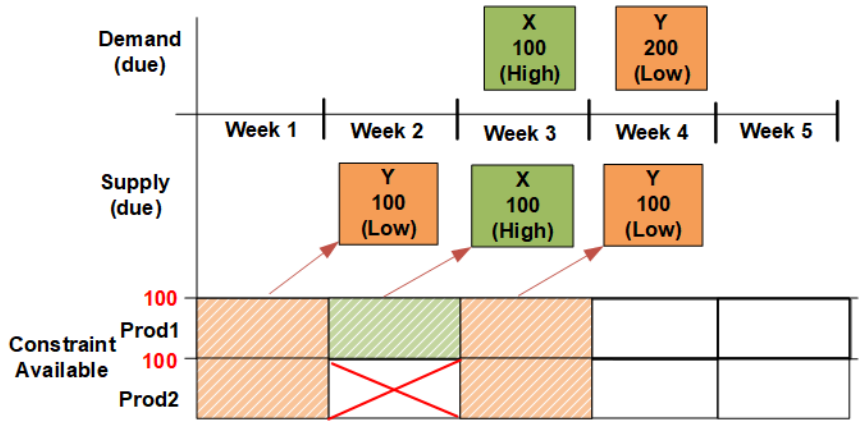
Based on the **ConstraintDateRule** setting of "Start", planned order splits from normal constraint scheduling would be as shown in the following illustration. In this case, forward scheduling has ensured the supplies consume constraint in continuous chunks with the higher priority supplies being pre-planned earlier. However, because the forward scheduling phase was unable to obtain parallel availability of both constraints "Prod1" and "Prod2" until Week 3, this resulted in some supply of "Y" being planned late.

Figure 28.39 Planned order splits with "Normal" setting



Instead, if the **SchedulingRule** were changed to "BackloadOnly", then supplies would be planned based on the initial backward phase of constraint scheduling (as seen earlier in this example). Thus, planned orders would be generated as shown in the following image. Note that this means the supplies are no longer loaded on constraints in continuous chunks and the earlier pre-planned supply is a lower priority supply. However, it also allows for all supplies to be planned on time.

Figure 28.40 Planned order splits with "BackloadOnly" setting



Therefore when part sources require multiple different constraints, the "BackloadOnly" setting is generally recommended as the forward scheduling phase is not guaranteed to find aligned availability of all required constraints in one or more periods even if the backward phase did so.

Additionally, if partial planning logic is used along with parallel constraints, the "BackloadOnly" setting is further recommended as decisions to give up on demands or not is made in the backward phase of constraint scheduling. That is, if "Normal" constraint scheduling is used with parallel constraints and partial planning logic, it is possible for demands to be satisfied late and beyond their lateness tolerance rather than ignored.

Limitations on "BackloadOnly" scheduling

When the **PartSourceType.SchedulingRule** setting is "BackloadOnly", and hence the forward phase of constraint scheduling is disabled, a small number of incompatibilities or limitations on other analytic calculations are introduced.

Table 28.8: SchedulingRule limitations

Analytic / Calculation	Impact on "BackloadOnly" scheduling
Multi-level search (reduce excess)	Multi-level search logic is supported with the BackloadOnly setting. However, the option to reduce short-term excess that can be generated from multi-level search logic is not supported when BackloadOnly scheduling is used. That is, the **MultiLevelSearchRule.ReduceExcess** field is always treated as being set to "Allow" when the BackloadOnly setting is used (the "Reduce" setting has no effect).
ConstraintDateRule	The **PartSourceType.ConstraintDateRule** settings of "ShipExtend" and "StartExtend" are not supported, and will be treated as if set to "Ship" and "Start" respectively. The "DuringLeadTime" setting is respected but results might not be as expected. Only the Ship and Start options are recommended for use with BackloadOnly scheduling.
Fixed constraints	The use of fixed constraints is not recommended for use with the BackloadOnly scheduling. Values provided in **SourceConstraint.FixedConstraintFactor** will be respected, however the BackloadOnly setting will generally create more supply splits which can lead to additional fixed constraint consumption.
Non-constrained constraints	BackloadOnly scheduling is only supported for constraints that have a **ConstraintType.ProcessingRule** setting of "Constrained". However, the "BackloadOnly" setting is automatically disabled for constraints that use any other **ProcessingRule** ("CumMax", "LoadOnly", "PlanningOnly", "Unconstrained", or "UnconstrainedPlan").

Table 28.8: SchedulingRule limitations

Analytic / Calculation	Impact on "BackloadOnly" scheduling
Production planning	The use of production wheels and production planning logic is not supported when BackloadOnly scheduling is used. Production planning relies on the forward phase of constraint scheduling which is disabled when the "BackloadOnly" seeting is used.
Multi-level constraints with partial planning	Partial planning logic with multi-level constraints is generally supported in BackloadOnly scheduling as long as there is ample constraint available. However, in cases of limited constraint available then upper level parts might typically use up the constraint available and prevent lower level components from being built on time. To avoid this problem, when there are multi-level constraints and limited constraint available with the BackloadOnly setting, it is recommended to enable multi-level search logic.
PrePlanRule	The **PartSourceType.PrePlanRule** of "JustInTime" is not supported with BackloadOnly scheduling. However, the BackloadOnly setting on its own will generally plan as close to just as time as is possible .
SchedulePriority	The use of **OrderPriority.SchedulePriority** to plan high priority supplies as close to their due date as possible is not supported with BackloadOnly scheduling. The **SchedulePriority** setting is applied during the forward phase of constraint scheduling which is disabled by the BackloadOnly setting.
AbsolutePrePlanLimit	The use of **PartSource.AbsolutePrePlanLimit** is supported with BackloadOnly scheduling. To ensure the expected results, smaller constraint calendars should generally be associated with larger pre-planning periods. Additionally, if also using multi-level constraints along with multi-level search logic, avoid setting the **AbsolutePrePlanLimit** to "0" as this can lead to unnecessary lateness.
AllowLatePlan	All **SupplyStatus.AllowLatePlan** settings are supported with BackloadOnly scheduling. However, the "Y" setting is recommended to ensure better consistency between Netting and CTP calculations especially when parallel or multi-level constraints are involved.
Constraint families	If one or more parts in a constrained constraint family (related through **SourceConstraint** relationships) use the "BackloadOnly" setting, then BackloadOnly scheduling is automatically enabled for all other parts in the family (regardless of their **SchedulingRule** setting).

PrePlanRule

Supplies can be pre-planned or built before they're actually required to satisfy a demand based on rate (constraint available per time period) and to ensure other supplies can meet their due dates (as described in "Pre-planning limits" on page 1638).

When pre-planning is applicable, the **PartSourceType.PrePlanRule** setting determines how pre-planning is applied and whether any pre-planned quantities are more likely to be in earlier or later constraint buckets within the range where pre-planning is required. The **PrePlanRule** field has two settings; "JustInTime" and "Normal" (the default value and equivalent to how constraint pre-planning was always applied prior to Version 2016.2, Service Update 16).

When **PrePlanRule** is set to "Normal" then, after the earliest constraint bucket in which pre-planning should start has been determined, constraint is consumed from earliest date to the latest within the required range. With this setting, pre-planned quantifies are more likely to be seen in earlier constraint buckets and some, pre-planning might occur earlier than is strictly required.

When **PrePlanRule** is set to "JustInTime" then, constraint is consumed starting from the latest constraint bucket in the required range to the earliest. As necessary, orders are sized to fit available constraint capacity in the latest bucket. This setting is intended to reduce occurences of unnecessarily early pre-planned quantities and any pre-planned quantities are more likely to be seen in later constraint buckets.

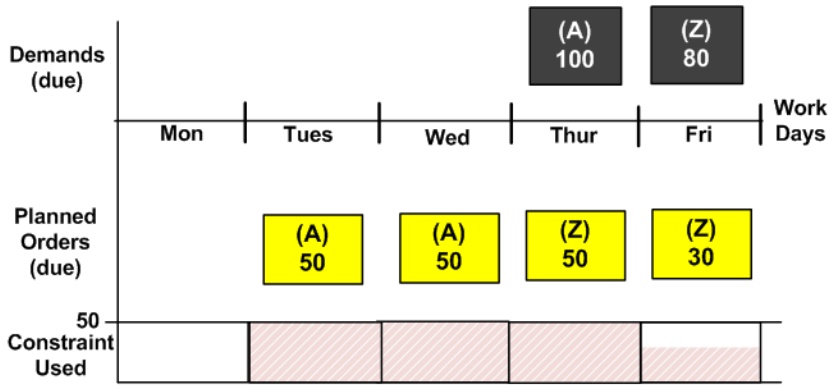
Example

Assume two parts, Part A and Part B, each have exactly one part source that uses a common production constraint and the following constraint-related settings are defined (identically) for each:

- **Constraint.Calendar**- Workday
- **ConstraintAvailable.Rate**- 50
- **SourceContraint.ConstraintFactor**- 1
- **ConstraintType.ProcessingRule**- Constrained
- **PartSourceType.ConstraintDateRule**- Ship

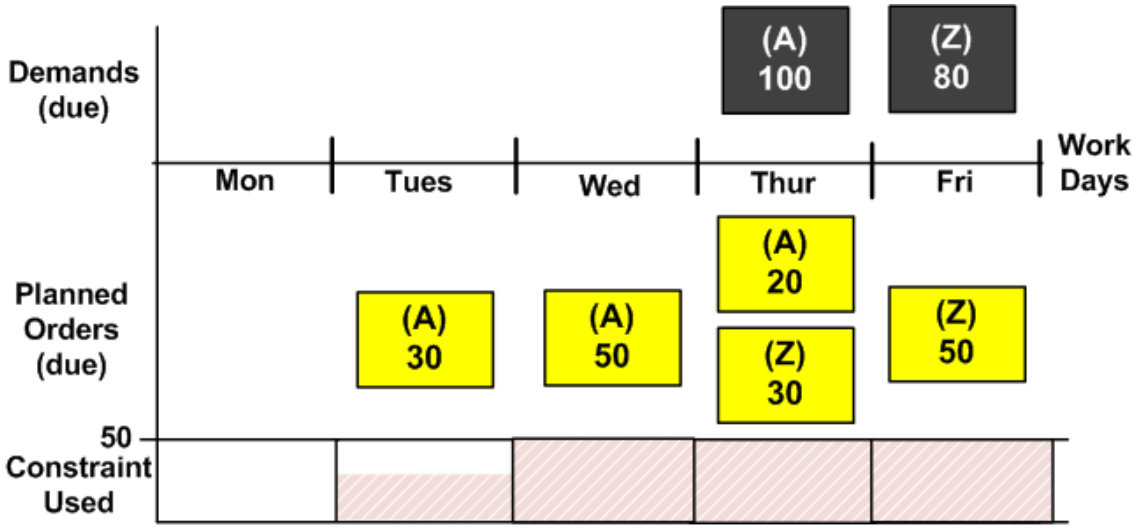
If the PrePlanRule setting of "Normal" were used for both part sources, then planned supplies created to satisfy a pair of demands for these parts might be as shown in the following illustration. In this case, the supply for 100 units Part A needs to be preplanned both because there is only enough constraint to build 50 units a day and also because the latter demand for Part Z has a constraint requirement that overlaps with the requirement from A. Considering both these factors, the earliest bucket in which pre-planning of this supply is required would be Tuesday. Thus with this setting, pre-planning of supply for A would start on Tuesday and continue through Wednesday (consuming all constraint available in each of those two days). Planning of supply to satisfy the demand for 80 units of Z would then start on Thursday, consuming all constraint available that day, with another smaller supply for Z planned on Friday that consumes 30 units of the constraint available that day.

Figure 28.41 Normal setting



Instead, if the PrePlanRule setting of "JustInTime" were used for both part sources, then planned supplies to satisfy this pair of demands might be as shown in the following illustration. With this setting, because not all of the constraint on Thursday is actually needed to satisfy the latter demand for Z, a supply for 20 units of A could be planned here, and the remaining supply is planned moving earlier, consuming all 50 units of constraint available on Wednesday and 30 units of the constraint available on Tuesday. Similarly, supply for Z is planned starting in the Friday bucket and the remainder planned in the Thursday bucket. Thus in this example, 20 fewer units of supply of each are pre-planned with the "JustInTime" setting as compared to the "Normal" setting.

Figure 28.42 JustInTime setting



Limitations on "JustInTime" preplanning

When the logic to minimize any pre-planning is enabled (**PartSourceType.PrePlanRule** = "JustInTime"), a small number of incompatibilities or limitations on other analytic calculations are introduced.

Table 28.9: PrePlanRule limitations

Analytic / Calculation	Impact on "JustInTime" preplanning
ConstraintDateRule settings	**PartSourceType.ConstraintDateRule** settings of "StartExtend", "ShipExtend", and "DuringLeadTime" are not compatible with"JustInTime" pre-planning. When they are used, the "JustInTime" setting behaves the same as the "Normal" setting. In other words, the "JustInTime" pre-planning logic is only compatible with **ConstraintDateRule** settings of "Start" and "Ship".
FillSchedule	A **ConstraintType.FillSchedule** setting of "Y" is not compatible with "JustInTime" pre-planning. When it used, the "JustInTime" setting behaves the same as the "Normal" setting. If using "JustInTime" pre-planning logic, the **FillSchedule** setting should be left at "N".
FixedConstraint	Fixed constraint consumption is not compatible with "JustInTime" pre-planning. When fixed constraint usage is specified (using the **SourceConstraint.FixedConstraintFactor** field), the "JustInTime" setting behaves the same as the "Normal" setting.
Campaign planning	Campaign production planning logic is not compatible with "JustInTime" pre-planning. When campaign planning is enabled (**OrderPolicy.Maximum** usage is set to "CampaignPlanning"), the "JustInTime" setting behaves the same as the "Normal" setting.
Multi-level search (MLS)	"JustInTime" pre-planning is compatible with multi-level search logic. In cases where multi-level search is not enabled, and a component lower down in the BOM structure is later found to be unavailable, supply for higher level parts might be canceled. If this occurs, gaps in constraint usage can occur without earlier pre-planned supplies being able to be moved into those gaps to minimize unnecessary pre-planning. Because constraint scheduling in Netting otherwise assumes complete component availability, enabling multi-level search for such parts can help determine component availability before constraint scheduling and thereby prevent such unfillable gaps in constraint usage.

Table 28.9: PrePlanRule limitations

Analytic / Calculation	Impact on "JustInTime" preplanning
MLS smoothing	MLS smoothing logic is compatible with "JustInTime" pre-planning, but in some cases might not result in complete minimization of constraint pre-planning.
Lot-sizing (on last order)	If lot-sizing is used and **OrderPolicy.LotSizeLastOrder** is set to "N", then "JustInTime" pre-planning logic might not fully minimize pre-planning with respect to the last planned order (only).
Days of supply	Days of supply logic is compatible with "JustInTime" pre-planning, but can result in a greater number of smaller planned orders being generated if a single planned order cannot be scheduled for a single constraint period.

FairShareAllocation

Sometimes many different parts might share a common constraint through their respective part sources. In cases where there is an insufficient amount of that constraint available to satisfy all demands where it is required, a configuration option is available to set how limited constraint availability should be allocated between different parts that need it. This option is primarily controlled through the **ConstraintAllocationRule** and **ConstraintAllocationCalendar** fields on the **OrderPriority** table. For finer control at the part level, the **PartType.ConstraintShareRule** field can be used.

The **OrderPriority.ConstraintAllocationRule** field is used to specify the way in which limited constraint is actually allocated between multiple part demands of a given priority level and contains two values; **FirstComeFirstServed** and **FairShare**.

With the **FirstComeFirstServed** option, prioritized demands are processed one at a time and, if there is limited constraint available, it is allocated on a supply by supply basis based on a large number of factors including demand priority, due date and, when all other things are equal, part name. This is the default setting corresponds to how constraint calculations were always performed in versions prior to 11.2.

With the **FairShare** option, all demands of the same priority level, due within a specified period, and that share a common constraint through their part sources are processed together. If there is insufficient constraint to supply the entire of group of demands, then the limited constraint is allocated between parts in a ratio proportional to the quantity of the demands being satisfied for each. In other words, parts with large demand quantities in a period will be allocated more of the limited constraint, and parts with smaller demand quantities will receive proportionally less constraint. Note that this allocation of constraint is based strictly on actual demand quantities and does not include consideration for any differences in constraint factor. Additionally the **OrderPriority.ConstraintAllocationCalendar** is used to reference the calendar that defines the period over which demands are collected and processed for the purpose of sharing limited available constraint. For example, constraint might be shared between all parts with demands due on each workday or all demands due within a weekly period.

When using the **FairShare** option, the **PartType.ConstraintShareRule** field applies and determines whether a part's demands follow the **ConstraintAllocationRule** setting effective for demands of a given priority level. If **ConstraintShareRule** is set to "Ignore", then the part's demands are never eligible for the fair-sharing of constraints. Otherwise, if **ConstraintShareRule** is set to "Use" (default setting) or "WithinFence", the part's demands respect the **ConstraintAllocationRule** setting effective for a given priority level. If using the "WithinFence" option, the **Part.ConstraintShareFence** field also applies and can be used to set a number of **ConstraintAllocationCalendar** intervals from **RunDate** within which the part's demands must fall to be eligible for fair-sharing of constraints. (demands outside this window are not eligible for constraint sharing).

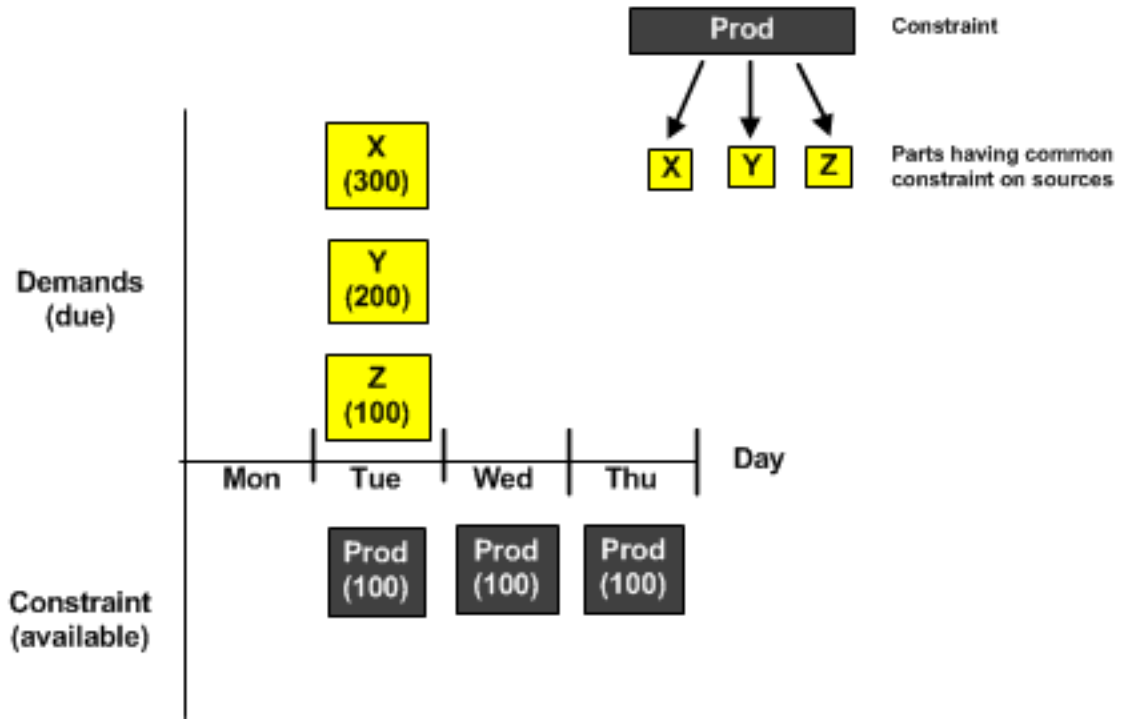
Notes:

- If setting the **ConstraintAllocationRule** field to “FairShare”, then the **IncrementalRule.ProcessingRule** value should be set to “On” for the parts sharing the constraint. This ensures the benefits from the fair sharing of constraints are reflected by allowing planned orders to be split and incremental available dates reported on demands. For more information, see "Turning incremental availability calculations on and off" on page 1479.
- If setting the **ConstraintAllocationRule** field to "FairShare", then the appropriate **SupplyStatus.AllowLatePlan** values should be set to "Y" into order to ensure expected fair-sharing of constraints in cases where there are overloaded constraints. If a constraint is overloaded and Netting moves the load to the next constraint calendar bucket, then expected fair-sharing might not occur in cases where **SupplyStatus.AllowLatePlan** is set to "N'.
- For sharing of the actual supplies generated for a particular part, the “FairShare” and “EqualShare” settings on the **OrderPriority.AllocationRule** field can be used. For more information, see "Allocating limited supplies" on page 1464.
- For fair-sharing of constraint on parts that use multi-level search logic, the **MLSAllocationRule** and related fields on the **OrderPriority** tables should be used instead. For more information, see "Fairshare of material and constraints" on page 1868.

Example 1: Constraint sharing by day

Assume three parts X, Y, and Z each of which share a common requirement for a constraint, Prod, through their part sources. Further assume that each part has a **PartType.ConstraintShareRule** of “Use”. Finally assume the set of demands, all belonging to the same priority level, for these parts and availability of the Prod constraint as shown in the following illustration.

Figure 28.43 Demands for parts that share constraint (ConstraintAllocationCalendar = Workday)



With the default **ConstraintAllocationRule** setting of "FirstComeFirstServed", because all demands are due on the same date and share the same priority, other factors determine which demand will be processed first and hence is first in line to receive the limited constraint. For example, if other factors are equal and the part name is used to determine processing order, then the available constraint would first be allocated for use in supply satisfying part X. In this scenario, X would receive all 300 units of the Prod constraint shown as available. Thus, on time quantities for each of three demands would be as follows:

- X: 100 on time
- Y: 0 on time
- Z: 0 on time

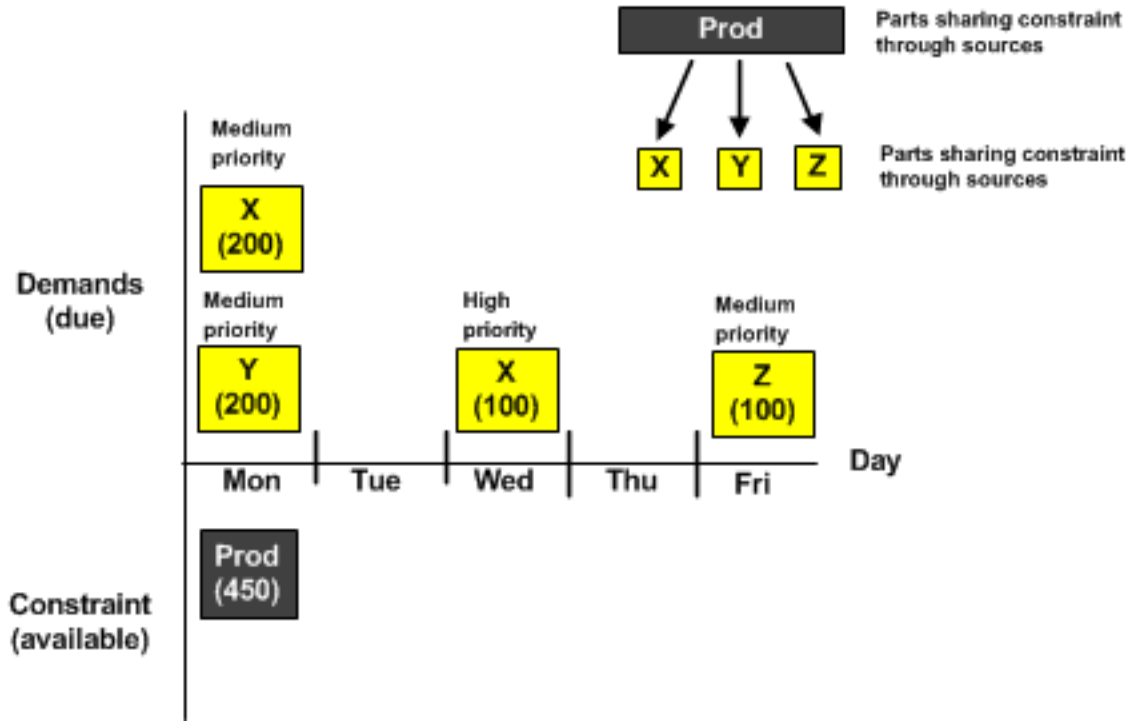
Instead if the **ConstraintAllocationRule** were set to "FairShare" with the **ConstraintAllocationCalendar** set to reference a "Workday" calendar, then all demands shown on Tuesday would be processed together and the limited constraint fairly shared based on the demand quantity ratio. Thus, on time quantities for each of the three demands would then be as follows:

- X: 50 on time
- Y: 33 on time
- Z: 17 on time

Example 2: Constraint sharing by week

Assume three parts X, Y, and Z each of which share a common requirement for a constraint, Prod, through their part sources. Further assume that each part has a **PartType.ConstraintShareRule** of "Use". Finally assume the set of prioritized demands for these parts and availability of the Prod constraint as shown below.

Figure 28.44 Demands for parts that share constraint (ConstraintAllocationCalendar = Week)



With the default **ConstraintAllocationRule** setting of "FirstComeFirstServed", limited constraint would be allocated to demands in this scenario based on priority first, and then due date, and then other factors (eventually including part name) used to choose between demands with equivalent priorities and due dates. Thus, on time quantities for each of the four demands would be as follows:

- Mon X: 200 on time
- Mon Y: 150 on time
- Wed X: 100 on time
- Fri Z: 0 on time

Instead if the **ConstraintAllocationRule** were set to "FairShare" and the **ConstraintAllocationCalendar** set to reference a "Weekly" calendar, then the lone high priority order would still receive limited constraint first, and all medium priority demands would be processed together and the remaining constraint fairly shared between them based on their demand quantity ratios. Thus, on time quantities for each of the four demands would then be as follows:

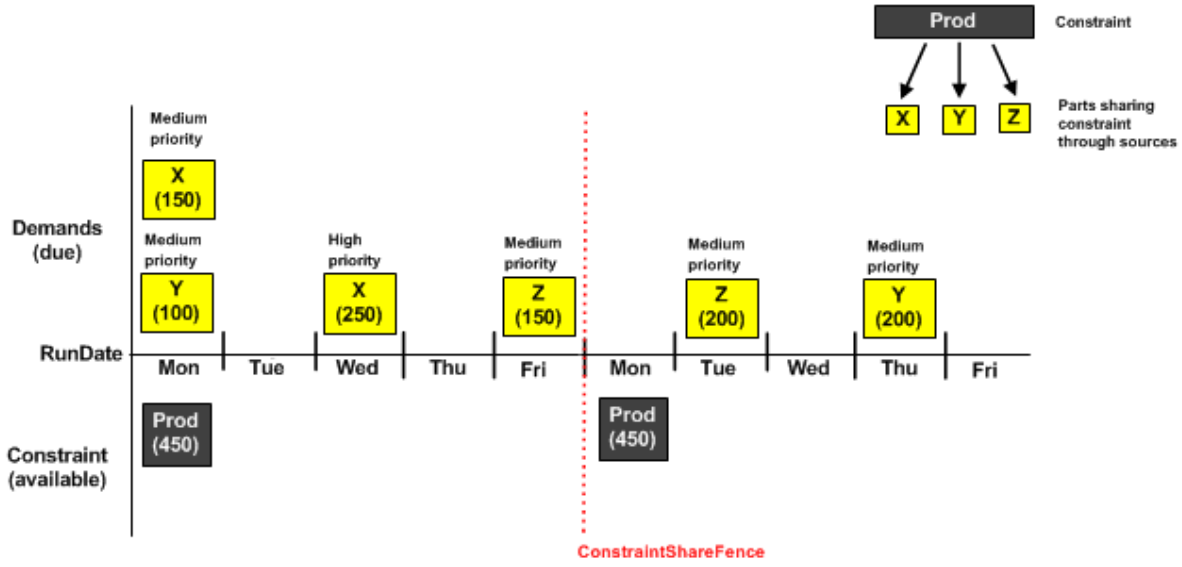
- Mon X: 140 on time
- Mon Y: 140 on time
- Wed X: 100 on time
- Fri Z: 70 on time

Example 3: Constraint sharing window

Assume three parts X, Y, and Z that share a common requirement for a constraint, "Prod", through their part sources (with **ConstraintFactor** of "1" for each). Further assume that each part has a **Type.ConstraintShareRule** setting of "WithinFence" and a **ConstraintShareFence** value of "1".

Finally assume the set of prioritized demands for these parts and availability of the Prod constraint as shown in the following illustration, with each priority level assumed to have **ConstraintAllocationRule** set to "FairShare" and **ConstraintAllocationCalendar** set to reference a "Weekly" calendar.

Figure 28.45 Demands for parts that share constraint



In the first week, there is insufficient constraint to satisfy all orders on time and the lone High priority order is processed first and fully receives the 250 units of constraint it requires. That leaves 200 units of constraint to be fairly shared between the three Medium priority orders which require constraint in the weekly period. The constraint is shared in a ratio based on those three demand quantities with on-time amounts for each demand in the week as follows:

- Mon X: 75 on time (75 late)
- Mon Y: 50 on time (50 late)
- Wed X: 250 on time (0 late)
- Fri Z: 75 on time (75 late)

Thus, 200 units demand in the first week are late and need to consume constraint from the second week. This leaves 250 units of constraint available which is insufficient to satisfy the two Medium priority demands due in this Weekly period. However, because these demands fall outside the constraint sharing window and they are allocated on a first-come, first-served basis, with on-time amounts for the demands as follows:

- Tue Z: 200 on time (0 late)
- Thu Y: 50 on time (150 late)

Limitations on the FairShareAllocation setting

When fair share constraint logic is enabled, a small number of incompatibilities or limitations on other analytic calculations are introduced.

Table 28.10: FairShareAllocation limitations

Limitation	Description
Input supply (scheduled receipt) usage	If a constraint was consumed by scheduled receipts in a period, it will not be shared.
Fixed constraint	Fair share constraint logic is automatically disabled for fixed constraints.
Campaign Planning	Fair share constraint logic is automatically disabled on part sources that are enabled for campaign planning.
Lot sizing	Fair sharing of constraints might result in certain lot sizing rules on part sources being violated. For example, the MaximumQty might be ignored when planning new supply.
Days of Supply	Fair sharing of constraints might result in Days of Supply logic being ignored. Because each planned order represents the fair share of constraint for the demand being satisfied, those planned orders can then not be merged together to satisfy Days of Supply requirements.
Two-date planning	Two-date planning should not be used together with fair share constraint logic. Fair share constraint logic enables the Netting analytic to share limited constraint between demand's based on their due dates. However, if using two-date planning and the recommended **AvailableDateLimit** setting of "CalcRequestStart", then the CTP analytic might schedule those constraints using different dates resulting in undesirable planning results.
Schedule Priority	The use of constraint scheduling priorities (OrderPriority.SchedulePriority) should be avoided when using fair share constraint logic. Because SchedulePriority is used to plan and schedule higher priority supplies as close as it can to the demand due date, this can produce undesirable results when trying to share constraint across supplies.

Table 28.10: FairShareAllocation limitations

Limitation	Description
Ignoring unsatisfied demand	If **DemandType.PlanningRule** is set to "Partial", then demand that cannot be satisfied within a specified tolerance is ignored and the demand left wholly or partially unsatisfied. However this partial planning is not compatible with constraint fair sharing and this logic should not be used together. Note if multi-level search is enabled, then a form of fair-sharing can be enabled together with partial planning by using the **OrderPriority.MLSAllocationRule** setting of "FairShare".
Overloaded constraints	If a constraint is overloaded and Netting logic moves the load into the next constraint calendar bucket, then constraint fair sharing results might not be as expected in cases where **SupplyStatus.AllowLatePlan** is set to "N". To ensure expected constraint fair-sharing when there are overloaded constraints, set the appropriate **SupplyStatus.AllowLatePlan** values to "Y" (allows constraints to be scheduled when they have capacity rather than to their demand dates which might be earlier).

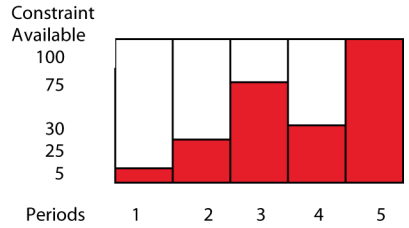
Respecting order policy rules

In cases where not enough constraint is available in a period to fully satisfy a supply, settings on the **PartSourceType** table can be used to force constraint consumption to respect order policy rules.The following two fields are used:

- **PartSourceType.ConstraintMultipleRule—**By setting this value to "Use", constraint consumption is forced to respect lot size (constraint can only be consumed in multiples of the value in **PartSource.MultipleQty**). For part sources constrained by a fixed constraint, this value is treated the same as "Ignore" (minimum quantity is not used).
- **PartSourceType.ConstraintMinimumRule**—By setting this value to "Use", constraint can only be consumed in levels that are at least the minimum order quantity (ensures constraint consumption is at least the value of **PartSource.MinimumQty**). In cases where **MinimumQty** is not a multiple of the value in **MultipleQty**, then **MinimumQty** takes precedence (constraint consumption is at least the value of **MinimumQty** rounded up to the next multiple of **MultipleQty**). For part sources constrained by a fixed constraint, this value is treated the same as "Ignore" (minimum quantity is not used).

When these fields are used, RapidResponse calculations do not consume constraint until there is enough available to satisfy the selected order policy rules. As an example, the following illustration shows the amount of constraint available across a range of dates to satisfy a given supply.

Figure 28.46 Constraint available across a range of dates to satisfy a given supply



Based on the constraint availability shown in the illustration above, the following table shows how constraint would be consumed in different periods based on the combinations of settings in the **ConstraintMultipleRule** and **ConstraintMinimumRule** fields. Note that in this case, setting either or

both of these fields to 'Use' changes the amount of constraint consumed in each period, and also results in constraint being consumed in fewer periods.

Table 28.11: Constraint consumption splits based on order policy settings

Constraint Multiple Rule	Constraint Minimum Rule	Period 1	Period 2	Period 3	Period 4	Period 5
Ignore	*Ignore*	5	25	75	30	9
Use	*Ignore*	0	24	72	24	24
Ignore	*Use*	0	0	75	30	39
Use	*Use*	0	0	72	30	42

Note: For more information about the order policy fields used when consuming constraint, see "ConstraintMultipleRule" on page 800 and "ConstraintMinimumRule" on page 799.

Control settings and calculated fields in Netting

The previous section in this chapter discussed the control settings that can be used to configure Constraint Analysis calculations. This section shows the effects that different combinations of these settings have on calculated date fields set by RapidResponse during netting calculations. The effects are shown for both scheduled receipts and planned orders. Note that some of these fields are set to the earlier (min) or later (max) of two different dates.

Processing Scheduled Receipts

The following table shows the effect that different control settings have on Constraint Analysis calculations when dealing with scheduled receipts. As well, the impact of whether or not the scheduled receipt had a defined StartDate is considered.

Table 28.12: Effects of control settings on scheduled receipt dates

Resched Rule	Allow Late Plan	Start Date	Calculated fields
NotReschedulable	All Values	Defined	• CalcStartDate = StartDate. • ReschedDate = EffDueDate. • RecommendedDate = EffDueDate (for Scheduled and NonReschedulable) OR BwdStockDate (for RecommendOnly). • ShipDate = Shipment.ShipDateTime (if the SR has a shipment record) OR ShipFromStock(EffDueDate) (if the SR does not have a shipment record). • TransferStart = MIN (ShipDateFromFirstTimeConstraintUsed, ShipFromStock(EffDueDate)).
		Undefined	• CalcStartDate = MIN (StartDateFromFirstTimeConsumed, StartDateFromEffDueDate). • ReschedDate = EffDueDate • RecommendedDate = EffDueDate (for Scheduled and NonReschedulable) OR DemandDate (for RecommendOnly). • ShipDate = ShipFromStock(ReschedDate). • TransferStart = MIN (ShipDateFromFirstConstraintUsed, ShipFromStock (ReschedDate)).
Reschedulable	No	Any Value	• CalcStartDate = MIN (StartDateFromFirstTimeConsumed, StartDateFromReschedDate.) • ReschedDate = MAX (ExpediteLimit, DemandDate.) • RecommendedDate = ReschedDate. • ShipDate = ShipFromStock(ReschedDate) • TransferStart = MIN (ShipDateFromFirstConstraintUsed, ShipFromStock (ReschedDate)).
	Yes		• CalcStartDate = MIN (StartDateFromReschedDate, StartDateFromFirstTimeConsumed). • ReschedDate = MAX (ExpediteLimit, StockDateFromLastTimeConstraintUsed.) • RecommendedDate = ReschedDate • ShipDate = ShipFromStock (ReschedDate) • TransferStart = MIN (ShipDateFromFirstConstraintUsed, ShipFromStock (ReschedDate))

Notes:

- StartDateFromFirstTimeConsumed is the date when the constraint was first used for this supply converted to a start date.
- BwdStockDate is the date that the supply was needed (as calculated when processing demands by priority and date, subject to planning time fence and expedite limit).
- StockDateFromLastTimeConstraintUsed is the date when constraint was last used for this supply converted to a stock date.
- ShipDateFromFirstConstraintUsed is the date when the constraint was first used for this supply, converted to a ship date.

Processing Planned Orders

The following table shows the effects that various control settings have on some calculated fields associated with planned orders.

Table 28.13: Effects of control settings on planned order dates

Constraint Date Rule	Split Late Supply	Allow Late Plan	Calculated Fields
StartExtend OR ShipExtend OR DuringLeadTime	No	No	One PlannedOrder is generated. • DueDate = MIN (StockDateFromLastTimeConstriantLooked, BwdStockDate). • StartDate = MIN (StartDateFromDueDate, StartDateFromFirstConstraintUsed.) • ShipDate = ShipFromStock(DueDate) • TransferStart = MIN (ShipDateFromDueDate, ShipDateFromFirstConstraintUsed.)
		Yes	One PlannedOrder is generated. • DueDate = StockDateFromLastTimeConstraintUsed. • StartDate = MIN (StartDateFromDueDate, StockDateFromFirstConstraintUsed). • ShipDate = ShipFromStock(DueDate) • TransferStart = MIN (ShipDateFromDueDate, ShipDateFromFirstConstraintUsed).
	Yes	No	Maximum of 2 PlannedOrders generated (one for the on time portion, and one for the late portion). OnTime portion: • DueDate = StockDateFromLastTimeConstraintUsed. • StartDate = MIN (StartDateFromDueDate, StartDateFromFirstTimeConsumed). • ShipDate = ShipFromStock(DueDate) • TransferStart = MIN (ShipDateFromDueDate, ShipDateFromFirstTimeConstraintUsed). Late portion: • DueDate = BwdStockDate. • StartDate = StartDateFromDueDate • ShipDate = ShipFromStock(DueDate) • TransferStart = ShipDateFromDueDate.
		Yes	Maximum of 2 PlannedOrders generated (one for the ontime portion, and one for the late portion). OnTime portion: Dates are calculated the same as for AllowLatePlan = No (above). Late portion: • DueDate = StockDateFromLastTimeConstraintUsed. • StartDate = MIN (StartDateFromDueDate, StartDateFromFirstConstraintUsed.) • ShipDate = ShipFromStock(DueDate) • TransferStart = MIN (ShipDateFromDueDate, ShipDateFromFirstConstraintUsed).

Table 28.13: Effects of control settings on planned order dates

Constraint Date Rule	Split Late Supply	Allow Late Plan	Calculated Fields
Ship OR Start	All Values	No	Multiple PlannedOrders generated (one for every period where constraint was allocated). • DueDate = StockDateFromStartDate. • StartDate = MIN (StartDateFromConstraintUsed, StartDateFromStockDate (BwdStockDate)). • ShipDate = ShipFromStock (DueDate). • TransferStart = ShipDateFromDueDate.
		Yes	Multiple PlannedOrders could be generated (one for every period where constraint was allocated). • DueDate = StockDateFromStartDate. • StartDate = StartDateFromConstraintUsed. • ShipDate = ShipFromStock (DueDate). • TransferStart = ShipDateFromDueDate.

Notes:

- StockDateFromLastTimeConstraintLooked is the date when RapidResponse last looked for constraint to consume this supply.
- StartDateFromFirstConstraintUsed is the date when the constraint was first used for this supply converted to a start date.
- BwdStockDate is the date that the supply was needed (as calculated when processing demands by priority and date, subject to planning time fence and expedite limit).
- StockDateFromLastTimeConstraintUsed is the date when constraint was last used for this supply converted to a stock date.
- ShipDateFromFirstConstraintUsed is the date when the constraint was first used for this supply, converted to a ship date.

CHAPTER 29: Production Planning calculations

Production Planning uses constraint to model part production in a wheel. Each wheel is divided into cycles that process groups of parts. Each cycle consumes constraint in part production and in changeovers. RapidResponse calculates the number of cycles in a wheel and how constraint and supplies are managed in those cycles.

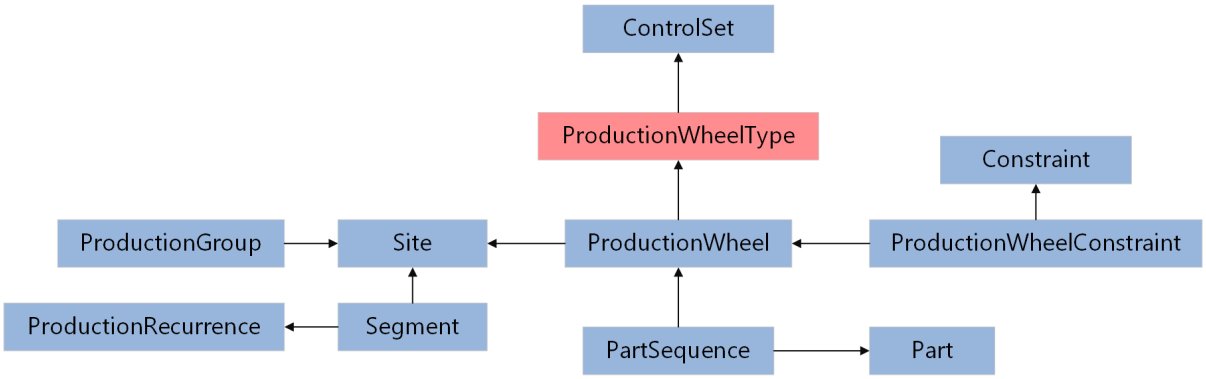
In this chapter, you'll learn about:

Production Planning data model **1691**
Managing production wheel constraint **1693**
Part sources in production wheels **1698**
Scheduled receipts and planned orders **1699**
Production wheel durations and cycle lengths **1699**
Production recurrence **1702**
Limitations with other analytics **1703**

Production Planning data model

The RapidResponse data model includes input, control, and calculated tables designed to support production planning. In addition, other standard tables that are not specific to production planning also hold fields that are used for production planning.

The following illustration shows the input and control tables required to configure production planning calculations.



Input tables

There are several input tables in the RapidResponse data model that support production planning. These include tables that hold production wheel, cycle, and constraint data in addition to standard RapidResponse tables that contain fields related to production planning.

Table	Description
PartSequence	Describes the properties for a part sequence. See "PartSequence table" on page 452.
ProductionGroup	Describes the properties for production groups, including the constraint used for changeovers. See "ProductionGroup table" on page 493.
ProductionRecurrence	Defines how often a part can be produced on a production wheel. See "ProductionRecurrence table" on page 495.
ProductionWheel	Describes the parameters that control production wheel cycles. See "ProductionWheel table" on page 496.
ProductionWheelConstraint	Describes the constraint parameters that affect production wheels. See "ProductionWheelConstraint table" on page 498.
Segment	Describes the segments a production wheel is divided into if segmentation strategies are used in planning. See "Segment table" on page 589.

Control tables

There are several control tables that impact production planning calculations.

Table	Description
ProductionWheelType	Defines the rules and values used to manage production wheels in RapidResponse. See "ProductionWheelType table" on page 849.

Calculated tables

The following calculated tables in the Rapidresponse data model contain production planning information. These contain ….

Table	Description
ProductionCycle	Describes the cycles in a production wheel including cycle length and changes to constraint availability. See "ProductionCycle table" on page 1155.

Managing production wheel constraint

The constraint required by a production wheel is divided between the cycles of the wheel where constraint is used up (ProductionCycle.ConstraintUsed) to produce each part and to set up, clean up, and change over between producing individual parts and groups of parts. Each cycle in the production wheel is assigned a fixed amount of constraint (ProductionCycle.ConstraintAvailable).

The order that constraint is used in a wheel is determined by the cycle number (ConstraintUsed.ProductionCycleNumber). When production wheels are configured in production planning, a processing rule must be set to define how the constraint is used in the wheel. To perform the full set of production planning calculations, ConstraintType.ProcessingRule must be set to Constrained.

Part sources are typically associated with multiple constraints. The primary constraint is the constraint that is associated with a production wheel (ProductionWheelConstraint). The secondary constraints are constraints on the part source that are not associated with a production wheel.

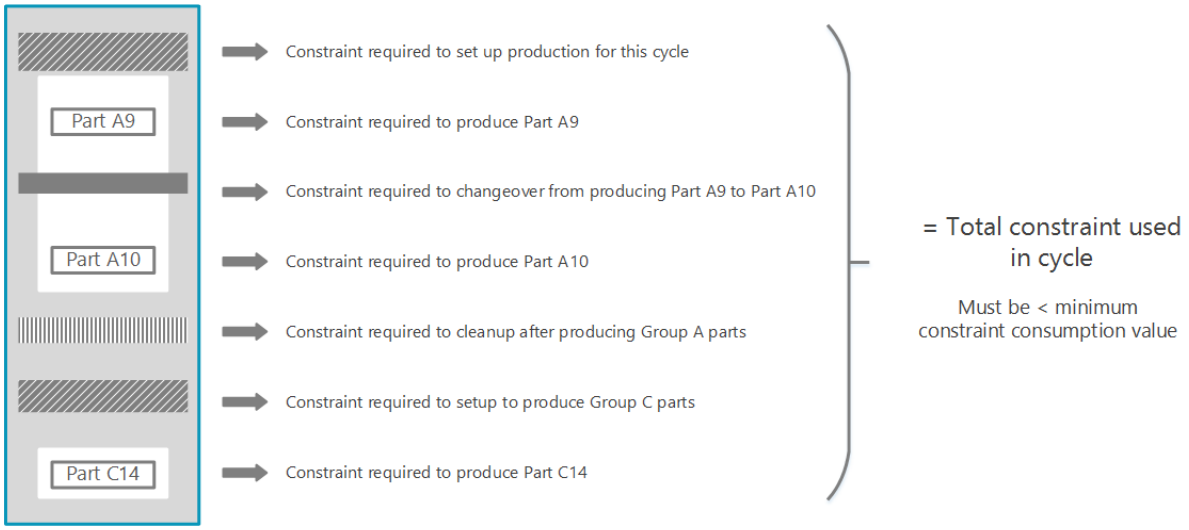
For production planning, all secondary constraints should be set to LoadOnly. If any secondary constraints for a part source are constrained, that can affect planning that uses the primary constraint. See "Insufficient constraint" on page 1698.

Each part is allocated constraint based on values from the SourceConstraint and ConstraintAvailable tables. To see the constraint for a part, use the Production Planning Manager workbook.

Constraint used for changeover is designated as either minor or major changeover (ConstraintUsed.Type) and in a cycle, constraint is used to:

- Produce each part in a group.
- Changeover from producing one part to producing another part in the same production group. This can include setup, cleanup, or both setup and cleanup (ProductionGroup.MinorChangeover).
- Setup production before processing a specific production group or cycle (ProductionGroup.MajorSetup).
- Cleanup production after processing a specific production group or cycle (ProductionGroup.MajorCleanup).

The diagram below demonstrates how a cycle might allocate constraint to produce two groups of parts.



Caution: Production wheels that are effective over the same period should not reference the same constraint to avoid incorrect calculations and poor planning results.

Tip: For part sources that use multiple constraints, set the constraints no referenced in the production wheel (non-primary constraints) to LoadOnly. See "Part sources in production wheels" on page 1698.

Minimum and maximum constraint

Each cycle is assigned a minimum fraction of constraint that should be consumed in the cycle (ProductionWheelConstraint.MinimumConstraintConsumption).

Each production group is assigned a maximum fraction of constraint (ProductionGroup.MaximumConstraintConsumption) they can use in a cycle. For example Group A is assigned a maximum value of 0.5 and Group B is assigned a value of 0.25. In a cycle that has a total of 500 units of constraint available, Group A cannot use more than 250 constraint and Group B cannot use more than 125 constraint.

The default for the maximum constraint consumption value is set to 0 and the counter is reset at the start of each cycle.

Caution: Do not change constraint factors (fixed or variable) in the middle of a cycle to avoid affecting minimum and maximum constraint consumption calculations.

Note: When production wheels are configured in production planning, a processing rule must be set to define how the constraint is used in the wheel. To perform the full set of production planning calculations, ConstraintType.ProcessingRule must be set to Constrained.

Handling unused constraints and unsatisfied supplies

One the parameters that must be set for a production wheel is how it handles unused constraint amounts (ProductionWheelType.UnusedConstraintRule) and unsatisfied supply quantities

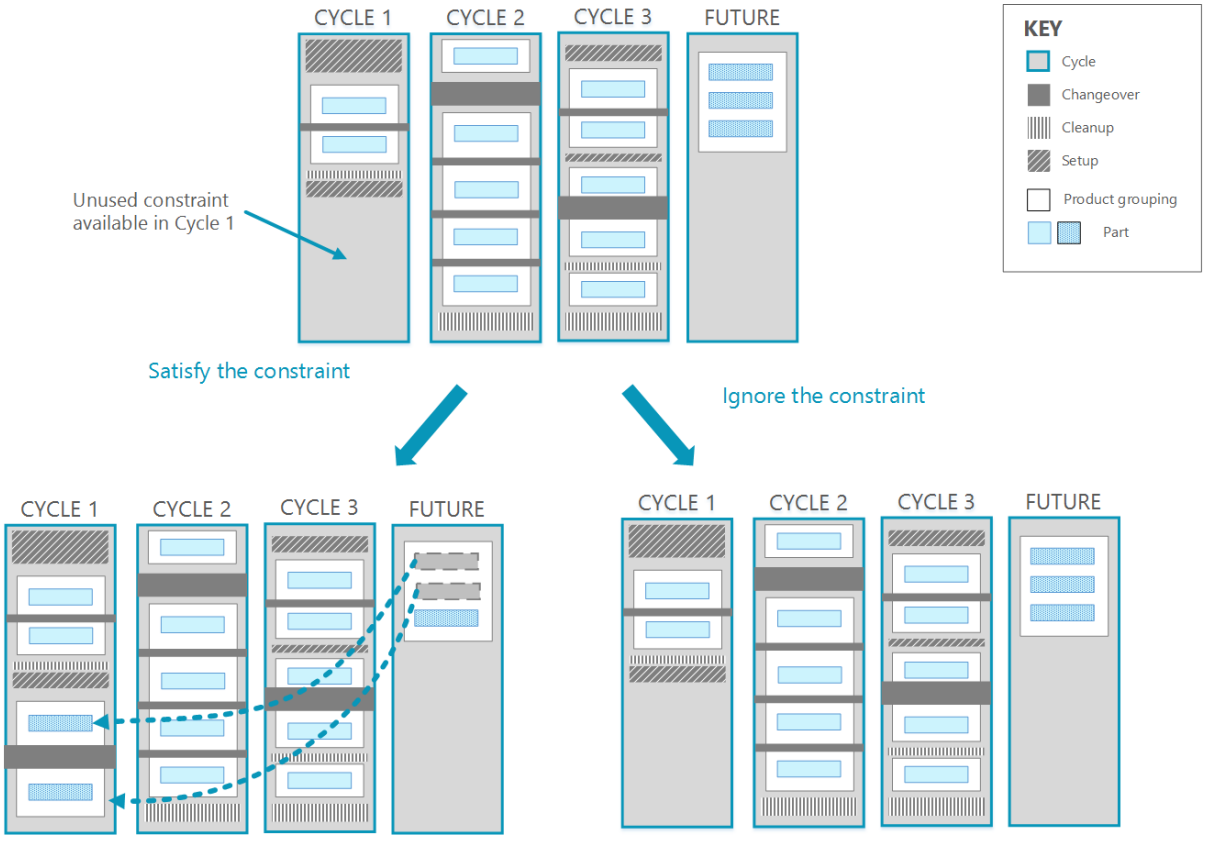
(ProductionWheelType.UnsatisfiedSupplyRule) in cycles.

Unused constraint

There are two ways to handle unused constraint remaining after all supply quantities have been produced in a production wheel:

- **Satisfy the constraint**—If parts produced in future cycles are eligible to build in the current cycle, those parts are assigned to the current cycle.
- **Ignore the constraint**—No additional parts are moved from future cycles.

The diagram below illustrates how both unused constraint options affect a production wheel.



If preplanning is possible, to control preplanning in earlier cycles the absolute preplan limit ((PartSource.AbsolutePrePlanLimit) is used in conjunction with the unused constraint rule to move supplies to earlier cycles. The preplan limit date is rounded to the start of the production wheel cycle it falls on.

Situations when parts are not eligible to be built in a current cycle:

- The minimum constraint consumption value has been reached.
- There is insufficient demand to meet the minimum constraint consumption value.
- The part lot size is too large to fit into the cycle.

- The maximum constraint consumption value has been reached
- The absolute preplan limit has been set to prevent demand from being pulled in.

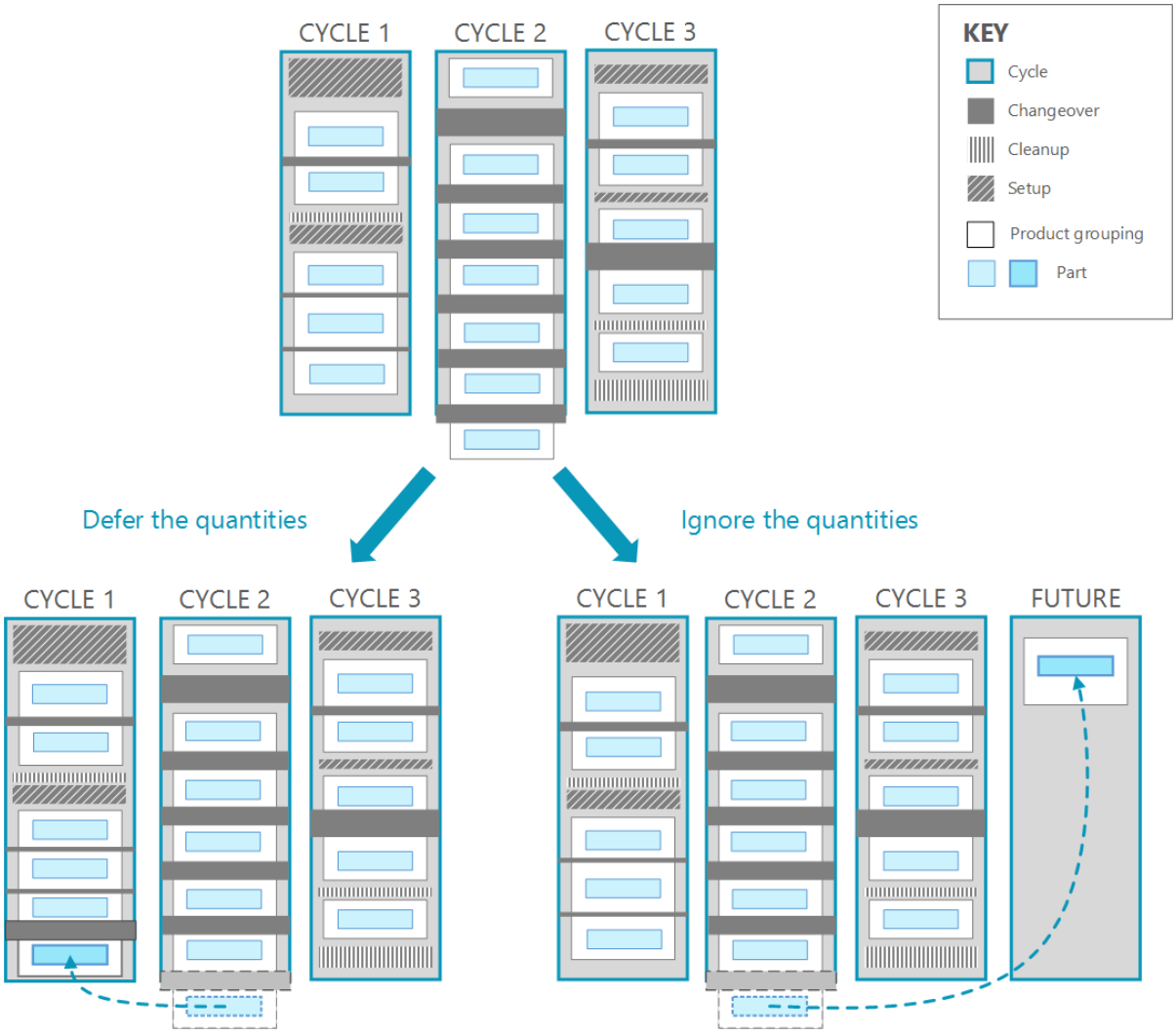
Note: When supply quantities are pulled in to current cycles, they may not follow their assigned part sequence order because previously scheduled supplies are not reshuffled.

Unsatisfied supply

There are two ways to handle supply quantities assigned to a production cycle that remain unsatisfied because the cycle has run out of constraint:

- **Defer the quantities**—The unsatisfied supply is handled in the next cycle where it can be built.
- **Ignore the quantities**—Unsatisfied supply is ignored. Instead it is assigned an available date of 'Future'.

The diagram below illustrates how both unsatisfied supply options affect a production wheel.



Changeover in production

Constraint is used to model all changeovers required in production:

Minor changeover—the amount of constraint required to change from producing one part to another part in a production group.

Major changeovers

- **Setup**—the amount of constraint required to set up production. Setup typically takes place between production groups in a cycle however setup might also be required at the start of a cycle.
- **Cleanup**—the amount of constraint required to clean up production. Cleanup typically takes place between production groups in a cycle however cleanup might also be required at the end of a cycle.

If parts with different attributes are assigned to different production groups, changeover is not applied between the parts in the group. Instead, it is recommended that these parts be modeled as separate parts. See "Attribute-based planning" on page 1817.

Overriding major changeovers

The OverrideMajorChangeover table is an optional table that describes the constraint required for setup and cleanup when changing from one production group to another production group in a cycle. See "OverrideMajorChangeover table" on page 414.

By default, the constraint required for changeovers is defined in the ProductionGroup table (see "ProductionGroup table" on page 493) however you can set overrides for the changeover between specified production groups in the OverrideMajorChangeover table.

For example, 4 different production groups are processed on one production wheel: Group A, Group B, Group C, and Group D. Group A, B, and D are produced on a regular basis whereas Group C is only produced on special order from a customer and it requires a thorough cleanup before production to minimize cross-contamination of materials.

Use the OverrideMajorChangeover table to set an override records when changing over from Group A, B, or D to Group C to ensure appropriate cleanup before it is produced in any cycle of the wheel.

Production Group				
From	**Site**	**To**	**Cleanup**	**Setup**
Group A	HQ	Group C	35	5
Group B	HQ	Group C	40	5
Group D	HQ	Group C	50	5

Note: OverrideMajorChangeover.FromProductionGroup and OverrideMajorChangeover.ToProductionGroup are non-nullable fields.

Part sources in production wheels

In production planning, parts that have multiple parts sources use the established criteria to select a part source:

1. On time availability
2. PartSource priority
3. Target

For more on part source selection, see "Part source selection" on page 1377.

The effective period for the part sources used in a wheel should match that effective period of that production wheel. If the effective periods do not match, those cycles with part sources that are not effective for the full cycle follow existing RapidResponse behaviour that allows for constraint to be used outside of part source effectivity. See "Multi-level part source selection criteria" on page 1877.

Multiple production wheels running at the same time

When there are multiple production wheels that are effective in the same time period, the wheels are processed using the following criteria:

1. **Production wheel priority**—the wheel with the lowest priority value is processed first (ProductionWheel.Priority).
2. **Production wheel name**—the wheels are sorted alphabetically (ProductionWheel.Name).

Insufficient constraint

Secondary constraints for a part source should be set to LoadOnly to maintain planning quality. Setting ConstraintType.ProcessingRule to LoadOnly prevents insufficient constraint for subsequent cycles or wheels where there is not enough constriant for them to produce supplies.

If you are not able to set all secondary constraints to LoadOnly, then do one of the following :

- Assign enough secondary constraints to all the wheels.
- Use production recurrence, maximum constraint consumption, production wheel type, and absolute preplan limits to configure each wheel so that no wheel will use up constraint for other wheels.

Caution: Production wheels that share the same constraint cannot be effective in the same time period.

Note: For more about constraints in production planning, see "Managing production wheel constraint " on page 1693

Scheduled receipts and planned orders

Production Planning uses scheduled receipts that are reschedulable to enable the receipts to be shifted within production cycles, deferred to future cycles, or pulled in from future cycles. See "Rescheduling supplies" on page 1335.

When a reschedulable receipt is pulled in from a future cycle, the expedite limit date (SupplyStatus.ExpediteLimit) is extended to the start of that cycle to prevent supplies from being moved to early.

When a reschedulable receipt is pulled in from a future cycle and production quantities allow for preplanning, absolute preplan limit logic is applied. The Absolute Preplan Limit date (PartSource.AbsolutePrePlanLimit) is rounded to the start of the production cycle it falls on and with the value specified to manage unused constraint in the wheel (ProductionWheelPolicy.UnusedConstraintRule) controls the preplanning of supplies in the earlier cycles. See "Handling unused constraints and unsatisfied supplies" on page 1694.

Production for a reschedulable receipt in a cycle starts when:

- Constraint is available in the cycle.
- The maximum constraint consumption value for the production group has not been reached.

Caution: Until production has completed the cycle end date and maximum constraint consumption value are ignored possibly causing constraint to be consumed from subsequent cycles. It is recommended that you ensure there is sufficient constraint to avoid starving future cycles in the production wheel.

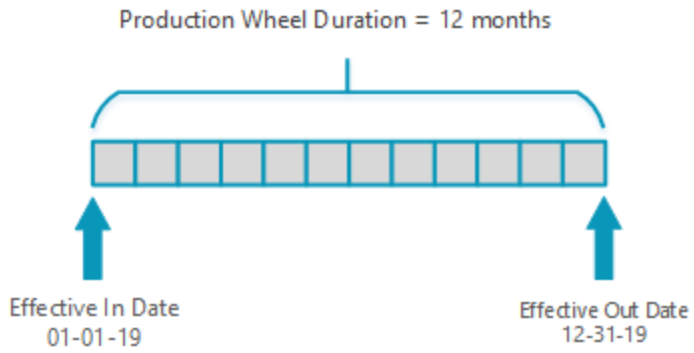
In Production Planning, when unused supply quantities are deferred or pulled into previous cycles, planned orders might be split.

Note: Lot sizing is only applied to planned orders. Additionally, when not using fixed lost sizes the preferred **OrderPolicy.MinimumUsage** setting with production planning logic is "RoundMult". Settings other than "RoundMult" will increase the likelihood that when production wheels need to spit the supply quantity, the last split might not obey the specified lot sizes.

Production wheel durations and cycle lengths

Production wheels are defined to run for a specific duration and are divided into production cycles. Users specify the duration and cycle lengths and RapidResponse calibrates the wheel to run as many cycles as it can over that duration.

The duration that the wheel is effective for is defined by its start date (ProductionWheel.EffectiveInDate), stop date (ProductionWheel.EffectiveOutDate), and calendar (ProductionWheel.Calendar).



The specific dates the wheel can start (ProductionWheel.FirstEffectiveDate) and stop production on (ProductionWheel.LastEffectiveDate) are calculated dates that account for the selected wheel calendar and defined cycle length in the wheel.

Wheel calendar

The calendar specified for the wheel (ProductionWheel.Calendar) affects when the wheel can start and stop by shifting the dates to fit into the calendar.

For example, for a production wheel with the following parameters:

Field	Value
EffectiveInDate	September 1
EffectiveOutDate	November 15
Calendar	Weekday

SEPTEMBER

S	M	T	W	R	F	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

OCTOBER

S	M	T	W	R	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

NOVEMBER

S	M	T	W	R	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

KEY

Production wheel duration

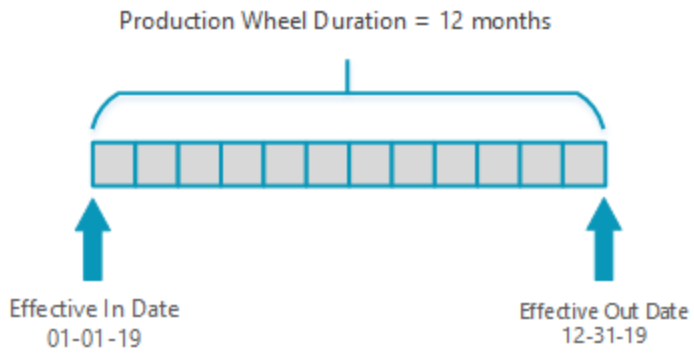
First and last effective dates calculated by RapidResponse

The first effective date is calculated as September 4 and the last effective date is calculated as November 17.

Cycle length

The length of the cycles in a production wheel is set by the number of intervals specified for the cycle and by the wheel calendar (ProductionWheel.Calendar + ProductionWheel.CalendarInterval).

For example, if ProductionWheel.CalendarInterval = 4 and ProductionWheel.Calendar = Month, the cycle length is 4 months. In a production wheel that runs for 12 months, 3 cycles would run for that wheel duration.

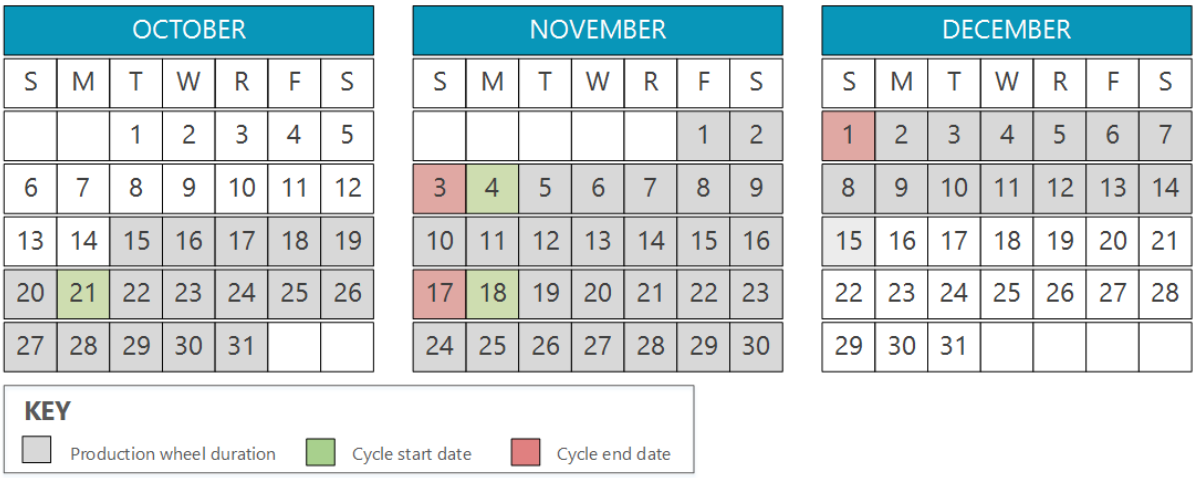


Production wheels can only process fully cycles. If a specified cycle length results in partial cycles in the wheel, RapidResponse shifts the first and last effective dates to ensure no partial cycles are processed in the wheel.

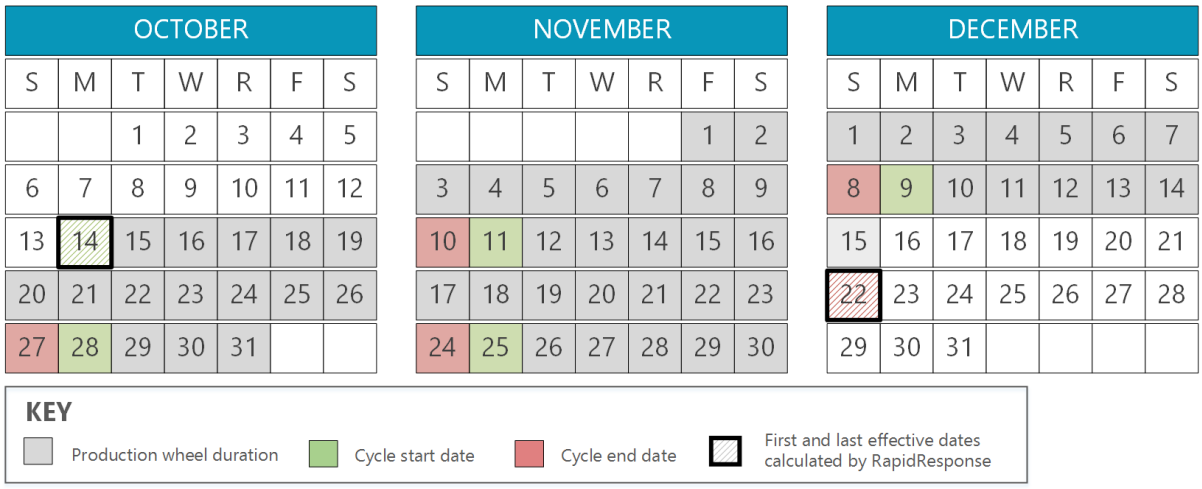
For example, for a production wheel with the following parameters:

Field	Value
EffectiveInDate	October 15
EffectiveOutDate	December 15
Calendar	Week
CalendarInterval	2

The cycle length is 2 weeks. If the production wheel was configured with just these parameters, only 3 full cycles can run in that wheel.



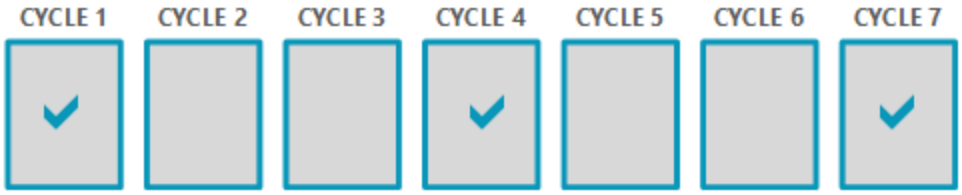
Instead, RapidResponse shifts the wheel duration by calculating a first effective date of October 15 and a last effective date of December 22, enabling 5 full cycles (ProductionCycle.CycleStartDate -> ProductionCycle.CycleEndDate) to run in that wheel.



Production recurrence

How often a part can be produced in a production wheel is determined by the production recurrence. Recurrence is assigned to a part (PartSource.ProductionRecurrence) or to the segment associated with a part (Segment.Recurrence).

The recurrence value (ProductionRecurrence.Recurrence) identifies which cycles a part can be produced in. For example, a recurrence of 3 enables a part to be produced in every third cycle of a production wheel (Cycles 1, 4, and 7).



Note: Cycles in a production are assigned a cycle number (ProductionCycle.CycleNumber).

Optionally, a recurrence can include an offset value (ProductionRecurrence.Offset) that shifts which cycle a production recurrence starts in. For example, a part with a recurrence of 1 can be produced in every cycle of a production wheel. If that part has an offset of 2, then that part can be produced in every cycle of the production wheel from cycle 2 onwards.

Limitations with other analytics

Some Production Planning logic affects results generated by other analytics in RapidResponse such as being incompatible with or causing limitations to other calculations or settings. The table below outlines these limitations and compatibility details.

Analytic or calculation	Details
Scheduled Receipts	Production Planning only supports reschedulable receipts. See "Scheduled receipts and planned orders" on page 1699.
Lead Times	Only SafetyLeadTime is compatible with Production Planning calculations. The time required to produce parts is modeled using fixed and variable constraint factors. See "Managing production wheel constraint " on page 1693.
Safety Stock	Production Planning limits safety stock behavior by building safety stock too early, late, or not at all. This is because: • Safety stock is built using both supply sorting criteria and production planning rules that set the demand date for all supplies in the last day of the cycle. • Planned orders to replenish safety stock can only be generated when actively included in a production wheel. • Planned orders to replenish safety stock are not generated when they do not meet the criteria for order generation. See "Define planned order generation" on page 1571.

Analytic or calculation	Details
Attribute-based Planning	In Production Planning, parts with different attributes should be modeled as separate parts. This is because switching between part sources for the same part does not require the use of any constraint. Therefore, the constraint required for changeover between these parts is not accounted for.
Co-products	Co product supply quantities are allotted to demand as if available from the start of a production cycle. See "CoByProductSupply table" on page 961. **Caution:** It is possible for co product supply to be very late if production wheel constraint is overloaded.
Absolute PrePlan Limit	When supply quantities are large enough to allow for preplanning, the absolute preplan limit date is rounded to the start of the production wheel cycle it falls on. This date is used in conjunction with the unused constraint rule to move supplies to earlier cycles.
PartSources with multiple constraints	To avoid planning gaps and under utilization of the primary constraint in a production wheel, part sources that use multiple constraints should set non-primary constraints to LoadOnly.
Multiple production wheels referencing the same constraint	To avoid poor planning results and calculations errors, production wheels that have overlapping effectively periods should not reference the constraint.
Time-phased SourceConstraint	To avoid affecting minimum and maximum constraint consumption calculations, fixed and variable constraint factors should not change in the middle of a cycle. See "SourceConstraint table" on page 605.
Control settings for Constraint Analysis	The settings are not compatible: • AllowLatePlan • FairShareAllocation • FillSchedule • PrePlanRule - JustInTime logic • SchedulePriority See "Constraint Analysis control settings" on page 1651.

Analytic or calculation	Details
Planned Order calculations	These calculations are not compatible: • Days of Supply • Order Point
• Campaign Planning • Capacity Requirements Planning (CRP) • Feature BOMs • Kanban • Material Expiry • Multi-Level Search logic (MLS) • Two-date planning	Production Planning is not compatible with these calculations.

CHAPTER 30: Campaign planning

RapidResponse supports campaign planning in which new supplies for a part are planned in defined batch sizes and grouped together into production campaigns. Each campaign then represents a set amount of the part to be produced and scheduled through a constrained production line as an uninterrupted process. The number of batches within any given campaign is a function of the demand being satisfied, but also subject to configurable limits. For example, supply of part might be producible only in batched quantities of fifty, with each campaign required to consist of at least five but no more than twenty batches.

Production campaigns are typically applicable in process industries where supplies are produced in consistent batch sizes and efficiencies are realized by producing many batches together as part of a larger process. For example, if there are long and expensive setup times associated with the equipment required to make a part, it might be cost effective to only produce supply of that part in long production runs. Similarly, if optimal product quality is not achieved with the first supply produced, but rather improves gradually over time, then it might be advantageous to produce supply in large batches.

In this chapter, you'll learn about:

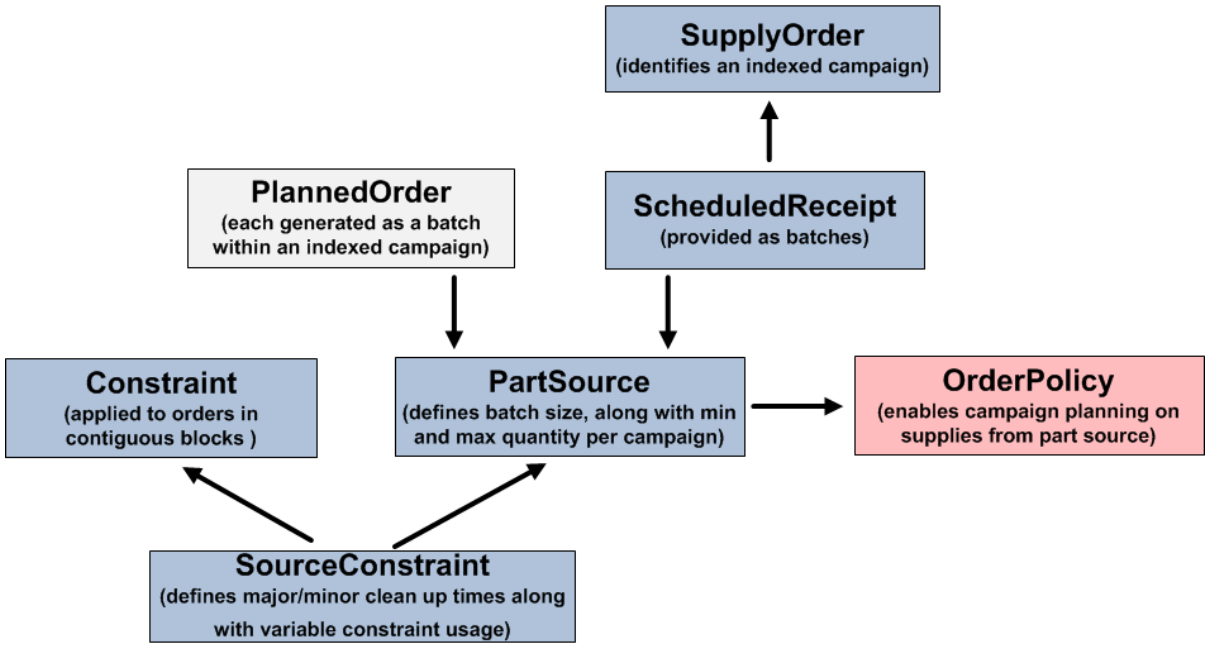
Campaign planning data model **1707**
Configuring campaigns **1708**
Campaign planning calculations **1710**
Limitations on other analytics **1715**

Campaign planning data model

Campaign planning is enabled at the part source level, and ensures supplies are generated for the required batch size and respect the minimum and maximum batches per campaign. Further, when batches of a campaign are scheduled through a constrained constraint, they can only consume that constraint in contiguous blocks.

The following illustration shows the main tables used to support campaign planning in RapidResponse, while the remainder of this chapter provides details on configuring and understanding the output of campaign planning calculations.

Figure 30.1 Main tables supporting campaign planning



Configuring campaigns

Production campaigns should be configured to ensure planned orders from relevant part sources are generated in the required batch size and that each campaign respects the minimum and maximum allowable number of batches.

Additionally, when production constraints are involved, the relevant fixed and variable constraint rates should be specified which are used in determining the size of the contiguous constraint block required to process a given campaign.

The following table outlines the main configuration steps involved in configuring campaigns in RapidResponse.

Table 30.1: Campaign planning configuration

Step	Description
Enable campaign planning logic	Campaign planning logic is enabled at the **OrderPolicy** level by setting the **MaximumUsage** option to "CampaignPlanning". Therefore, any **PartSource** records whose planned orders are to be generated in batches within production campaigns must reference an **OrderPolicy.MaximumUsage** value of "CampaignPlanning".
Define batch and campaign sizes	When a part source is enabled for campaign planning, the following three fields on the **PartSource** record should be used to define batch and campaign sizes: • **MultipleQty**—the intended batch size. All planned orders from the part source are built for this quantity. Note that if this field is set to "0", then campaign planning is disabled regardless of the setting in **OrderPolicy.MaximumUsage**. • **MinimumQty**—the minimum total order quantity that must exist across all batches in a campaign from this part source. This means, the minimum number of batches per campaign is calculated as `MinimumQty/MultipleQty` (rounded up to the next whole number if necessary). Thus, whenever new supply is required, batch-sized orders are generated at least until the minimum number of batches is reached. • **MaximumQty**—the maximum total order quantity that can exist across all batches in a campaign from this part source. This means, the maximum number of batches per campaign is calculated as `MaximumQty/MultipleQty` (rounded up to the next whole number if necessary). Thus, whenever new supply is required, an additional campaign must be generated each time the maximum number of batches is reached.
Define constraint factors	The following fields on the **SourceConstraint** table are used to specify the fixed and variable constraint factors applicable when processing campaign batches: • **ConstraintFactor**—the amount of the constraint required for a single unit of supply from the part source. This value multiplied by the supply quantity determines the variable amount of constraint consumed (it will be the same for each supply in a campaign due to fixed batch quantities). • **CampaignFixedConstraintFactor**—a fixed amount of constraint consumed at the beginning of each campaign. It represents major cleanup time needed to prepare the constraint to process the campaign as a whole. • **FixedConstraintFactor**—a fixed amount of constraint consumed in between campaign batches. It represents minor cleanup time needed to prepare the constraint to process the next batch in a given campaign (applied after each batch other than the last one in a campaign). **Note:** Campaign planning is only supported on a single constrained constraint per part source.

Table 30.1: Campaign planning configuration

Step	Description
Configure input supply batches	Input supplies can be set up for production campaigns. Campaigns are defined at the supply order level and **SupplyOrder.CampaignIndex** field is used to identify the campaign (or left blank if the supply order does not represent a campaign). If a supply order represents a campaign, each scheduled receipt under the order is assumed to represent a batch in the campaign and the **ScheduledReceipt.Line** field acts as the batch number. In this case, each scheduled receipt should be for the same **Part**, reference the same **PartSource**, and have the same quantity. Note also that because scheduled receipts are provided as input values, RapidResponse does not enforce the batch and campaign sizes defined on the associated **PartSource** record. For example, it is possible to simulate changes to the default batch quantities when converting planned orders to scheduled receipts. Finally, to enable campaign planning logic, the **PartSource** referenced by the scheduled receipts within a campaign should typically reference an **OrderPolicy.MaximumUsage** value of "CampaignPlanning". Among other things, this setting ensures that all receipts in the campaign are scheduled through their constraint in a contiguous block
Specify global batch limit	A global limit applicable to all part sources across all scenarios is used to set a maximum on the number of batches for any single campaign. This is intended to improve system performance by limiting the total number of planned orders generated (for example, when there is a very large demand quantity combined with a very small batch quantity). By default, the batch limit per campaign is 100. If this limit is reached, then 99 orders in the campaign are set to the batch quantity and 1 order set for the remaining quantity. If necessary, the batch limit can be changed by adding a configuration record as follows: 1. From the Go menu, click Administration. 2. Under System Settings, click Configuration. 3. Click the Processing Rules worksheet. 4. From the Edit menu, click Insert Record. 5. In the Insert Record dialog box, do the following: • In the **Id** box, type **MaxBatchCountPerCampaign**. • In the **Value** box, type the new campaign batch limit. Click **Save and Close**. 6. Restart RapidResponse Server **Note:** Only administrators can access the **RapidResponse Administration** workbook or restart RapidResponse Server.

Campaign planning calculations

Campaign planning logic is enabled for a **PartSource** if it references an **OrderPolicy.MaximumUsage** value of "CampaignPlanning". This setting impacts both planned order generation and constraint consumption.

Planned order generation

If a part source is enabled for campaign planning, then planned orders from the part source are generated in batches. Each batch (planned order) is generated for exactly the quantity defined in the **PartSource.MultipleQty** field and belongs to a specific campaign. Campaign and batch numbers are managed by part and reported in the **CampaignIndex** and **BatchIndex** fields on the **PlannedOrder**

table. Additionally, all applicable lead time elements are applied equally to each batch with the exception of safety lead time which is only applied to the last batch in a given campaign.

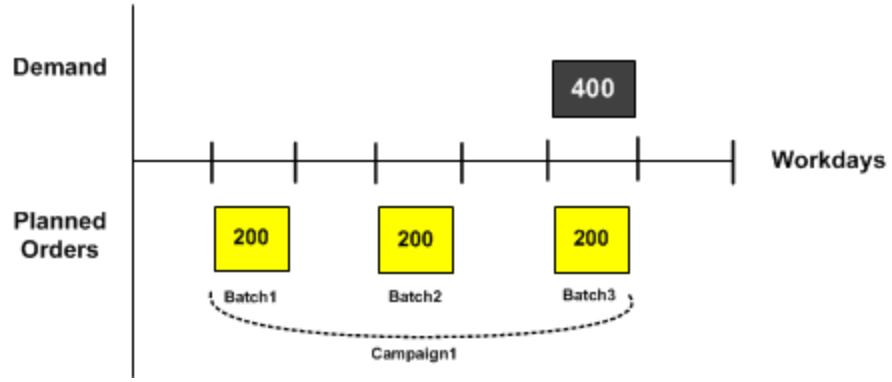
In order to control the number of batches that can belong to a given campaign, the **PartSource.MinimumQty** and **PartSource.MaximumQty fields** are used.

The **MinimumQty** determines the minimum total order quantity that must exist across all batches in a given campaign. Thus, the minimum number of batches that must exist in a campaign is determined as `MinimumQty/MultipleQty` (rounded up if the result is not a whole number). For example, assume a **PartSource** has the following settings:

- **EffectiveLeadTime—** 2
- **MultipleQty—**200
- **MinimumQty—**500

If **OrderPolicy.MaximumUsage** is set to "CampaignPlanning", then a single demand for 400 units would result in three planned orders (batches) generated as shown in the following illustration. Note that in order to respect both the batch quantity and minimum number of batches, there are 200 units of supply generated in excess of the actual demand quantity.

Figure 30.2 Batches within a single campaign

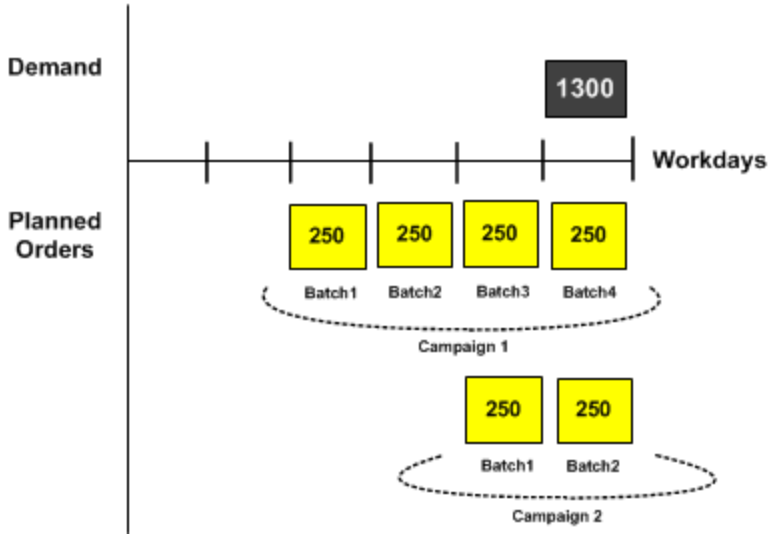


Similarly, the **MaximumQty** determines the maximum total order quantity that can potentially exist across all batches in a given campaign. Thus, the maximum number of batches that can exist in a campaign is determined as `MaximumQty/MultipleQty` (rounded up if the result is not a whole number). For example, assume a **PartSource** with the following values:

- **EffectiveLeadTime—**1
- **MultipleQty—**250
- **MaximumQty—**1000
- **MinimumQty—**500

If **OrderPolicy.MaximumUsage** is set to "CampaignPlanning", then a single demand for 1300 units would result in two campaigns generated as shown in the following illustration. Note that the first campaign plans batches until the maximum quantity of 1000 is reached which leaves 300 units of demand to be satisfied by a second campaign. In order to respect both the batch minimum, this campaign requires exactly two batches which results in total supply exceeding the actual demand quantity by 200 units.

Figure 30.3 Batches across multiple campaigns



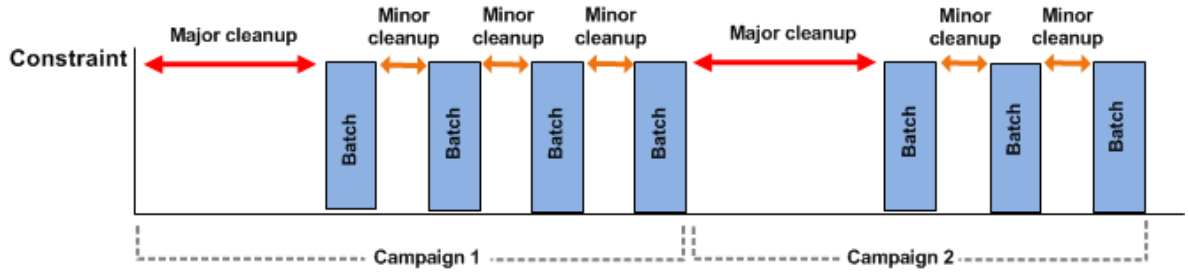
Notes:

- Each individual campaign batch is considered available for use (such as, by higher level assemblies) once it is complete. A batch's availability is not dependent on completion of the overall campaign.
- All planned orders within a given campaign must have the same priority. When priorities are propagated from demands to the supplies that satisfy them, the highest of those priorities is assigned to each order batch in the campaign.

Constraint consumption

Constraint consumption logic is affected when supplies from part sources enabled for campaign planning are scheduled through a constraint as shown in the following illustration.

Figure 30.4 Campaign-based constraint consumption



The following describes the three different sources of constraint consumption within a production campaign:

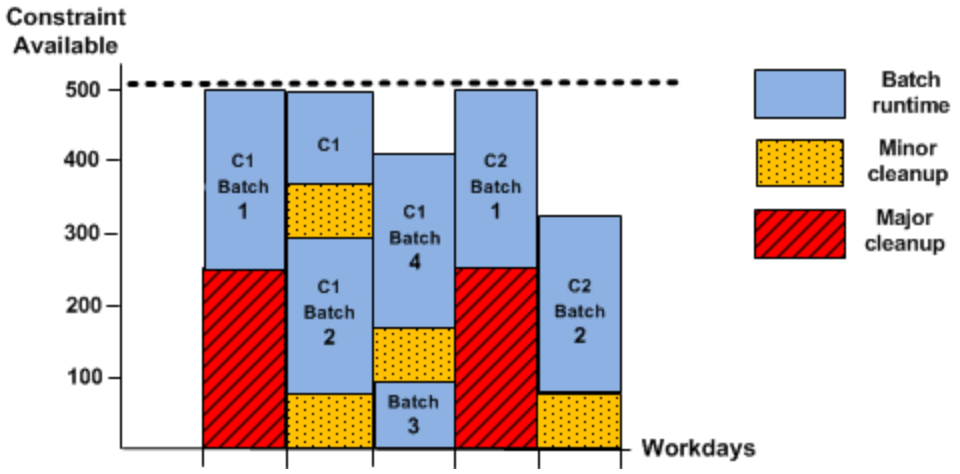
- Major cleanup—A quantity consumed at the beginning of each campaign in order to prepare the constraint (typically, equipment) to process the campaign. This is a fixed quantity applied to each campaign from the part source and is set by the value in **SourceConstraint.CampaignFixedConstraintFactor**.
- Minor cleanup—A quantity consumed between batches within each campaign in order to prepare the constraint to process the next batch in the campaign. This is a fixed quantity applied after each batch in a campaign (other than the last) and is set by the value in **SourceConstraint.FixedConstraintFactor**.
- Batch runtime—Quantity of constraint needed to process a supply batch from the part source. Calculated by multiplying **SourceConstraint.ConstraintFactor** by the supply quantity (because all planned orders from the part source are for the same batch size, the runtime for each batch will also be the same).

Additionally, all constraint consumption by the batches within a given campaign must be done in contiguous blocks. For example, assume the same two campaigns containing supply batches as shown in "Batches across multiple campaigns" on page 1712. Further assume the part source requires use of a constraint and the following properties apply:

- **Constraint.Rate—**500
- **SourceConstraint.ConstraintFactor—**1
- **SourceConstraint.CampaignFixedConstraintFactor—**250
- **SourceConstraint.FixedConstraintFactor—**50

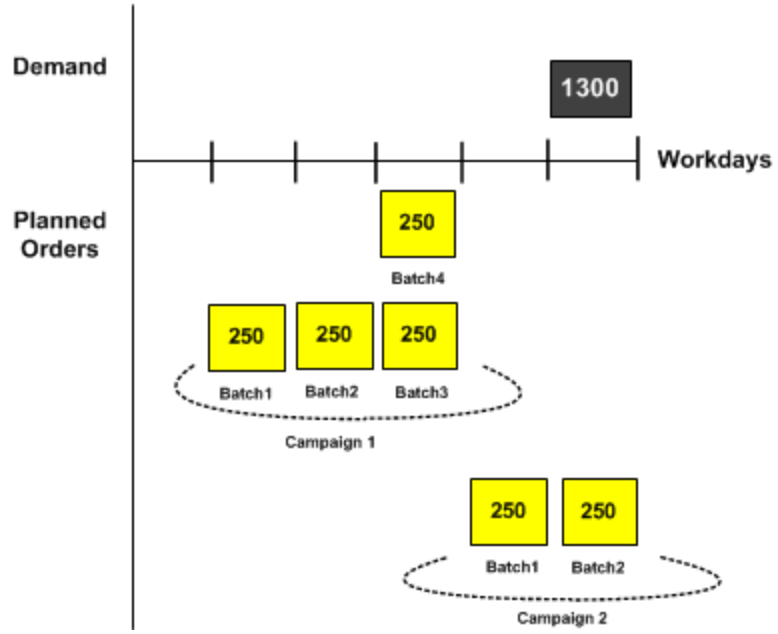
Constraint consumption from the two campaigns would be as shown in the following illustration.

Figure 30.5 Constraint consumed by campaign batches



Scheduling the campaign batches through the constraints in these blocks would then change the original planned order dates to those shown in the following illustration. Note that each campaign is still planned in blocks although the batches in Campaign 1 are moved earlier due to limited constraint availability.

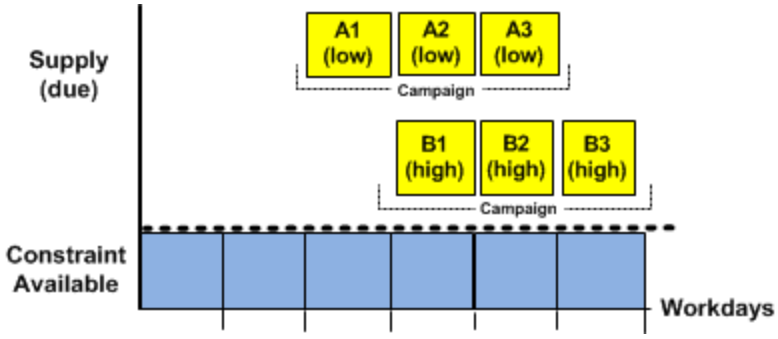
Figure 30.6 Order dates after constraint consumption



Additionally, because batches within a campaign must consume constraint in contiguous blocks, this can result in some orders being made available later than they would be if not planned in a campaign. This might typically occur when there are gaps in constraint availability and/or if there are multiple part sources requiring the same constraint.

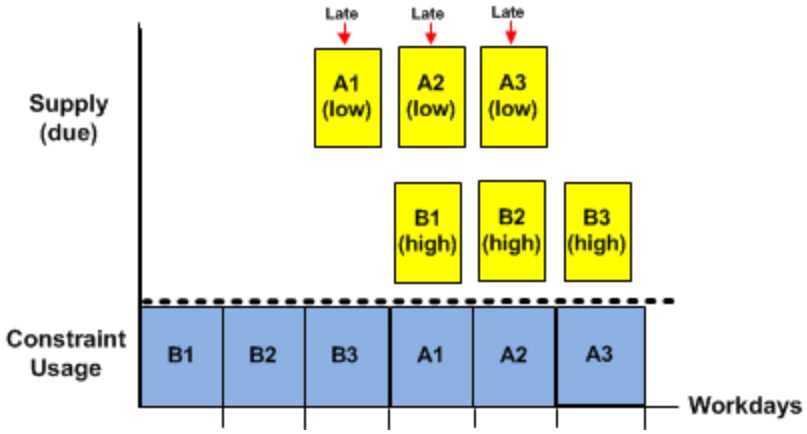
For example, assume a set of prioritized orders for parts A and B. Further assume each of their part sources requires use of the same constrained constraint, and there is only enough constraint available to build one supply/batch per period as shown in the following illustration.

Figure 30.7 Campaign batches with a common constraint



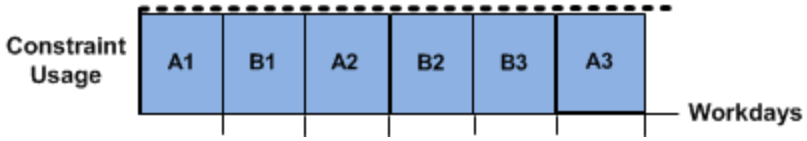
In order to ensure the high priority orders for B could be satisfied on time, they would need to consume constraint from earlier periods, and this consumption must occur in a contiguous block as shown in the following illustration. As a result, all supplies for part A are made late as they must also consume constraint in contiguous blocks and do not receive constraint until the campaign for part B has been processed. Note that this example assumes there are no fixed constraint factors, and that pre-planning is allowed.

Figure 30.8 Campaigns must consume constraint in contiguous blocks



If the same prioritized orders shown above were not associated with campaigns, then high priority supplies for B would only consume from earlier periods as necessary to ensure they are on time. This would allow supplies for A to consume constraint around B as shown in the following illustration (resulting in only A3 being late).

Figure 30.9 Constraint consumption without campaigns



Limitations on other analytics

A small number of other analytics or calculations might be disabled, or not work as expected, for parts planned from part sources with campaign planning logic enabled. The following table outlines other calculations impacted by campaign planning:

Table 30.2: Campaign planning limitations on other analytics

Analytic / Calculation	Limitation
Feature BOM	Campaign planning cannot be used with parts that are also configured in feature BOM relationships (where the assembly's **PartType.FeatureBOMRule** field is set to "Normal" or "HighVolume").
Kanban	Kanban logic cannot be used on part sources that are enabled for campaign planning (they are mutually exclusive settings in **OrderPolicy.MaximumUsage**).
Takt Time	Takt Time logic is automatically turned off on part sources that are enabled for campaign planning. That is, **PartSource.TaktTime** is treated as being set to "0".
Fair Share Constraint	Fair sharing of constraints is automatically turned off for parts using part sources enabled for campaign planning. That is, **PartType.ConstraintShareRule** is treated as being set to "Ignore".

Table 30.2: Campaign planning limitations on other analytics

Analytic / Calculation	Limitation
Split Late Supply	The splitting of late supplies, for the purposes of constraint consumption is automatically turned off for supplies that use part sources enabled for campaign planning. That is, **SupplyStatus.SplitLateSupply** is treated as being set to "N".
Schedule Priority	The ability to ensure that higher priority orders consume constraint as close to their due date as possible during any pre-planning is disabled for supplies from part sources enabled for campaign planning. That is, **OrderPriority.SchedulePriority** is treated as being set to "0".
Days of Supply	The days of supply window may shift when supply from a part source enabled for campaign planning reaches the maximum number of batches and a new campaign is created.
Allotment Override	When allotment overrides are used to reserve limited component supplies to specific demands, campaign planning batch sizes and policies may result in over reservations of supply.

CHAPTER 31: Distribution planning

Distribution planning includes an optional setting that enables the distribution, or pushing, of pre-built supply from a sourcing site to distribution sites. The amount of pre-built supply that is specified to be pushed is determined by the maximum inventory level that is set for a part. When the inventory for a part exceeds the set maximum inventory level, RapidResponse generates a request to push a quantity of the pre-built supply to one or more distribution sites. For example, if a company is pre-building to meet future demand and their manufacturing facility cannot store the finished goods, they can decide to push the finished goods into their distribution network before the finished goods are required.

Pushing can also be considered when pre-building finished goods to anticipate seasonal demand or to support a campaign planning process where a company may be pre-building well in advance of demand to optimize capacity utilization.

In addition to the pushing option, distribution planning also includes fair share logic to ensure that the pushing to distribution centers is performed in a fair and efficient manner. Fair share includes a forward-bucketed option that provides a method for satisfying shortages at distribution centers that are caused by demand from customer orders or forecast as fairly and efficiently as possible. Once the shortages are resolved, the analytic also ensures the replenishment of distribution centers to their safety stock levels in a fair and efficient manner.

This section discusses the following about distribution planning calculations:

Configuring distribution planning for pushing **1717**
Distribution planning calculations **1718**
Forward-bucketed fair share **1726**
Limitations on other analytics **1730**

Configuring distribution planning for pushing

In RapidResponse, distribution planning for pushing is configured on a part-by-part basis. To configure distribution planning for pushing, there are a few important requirements:

- Only sourcing parts in **Transfer PartSource** relationships can be configured to enable the pushing of supply (earlier than required) to the destination sites. However, parts that are components in a **Make** relationship (i.e. a component in a BillofMaterial) can also be configured for pushing, but the part needs to be able to be pushed to another level in the supply network. For example, for some

component parts, there might not be a location in the supply network for the part to be pushed.

- Incremental availability must be turned on for the part.

The maximum inventory level can also be set at multiple levels on a finished good part. Therefore, once the maximum level of inventory is reached for a part at one site, it generates a request to push the supply to the next distribution site.

Configuring a part for distribution planning involves two steps. First, distribution planning needs to be enabled for the part and then set a maximum inventory level for the part. Then, when a part exceeds the maximum inventory level that you set, it triggers a request to push the pre-built supply from the sourcing site to the distribution site.

The following table outlines the main configuration steps involved in setting up distribution planning for a part.

Table 31.1: Distribution planning for pushing configuration

Step	Description
Enable distribution planning rule for a part	Ensure the part references a **DistributionPlanningRule** record where the **DistributionPlanningRule.ProcessingRule** is set to **Enable**. The **AllotmentRule** and **SupplyPushRule** settings in this table also influence how supply can be pushed. For more information, see "DistributionPlanningRule table" on page 722. **NOTE**: To support supply pushing, the part must have incremental availability turned on (that is, it should reference an **IncrementalRule.ProcessingRule** setting of "On").
Define the maximum inventory quantity for a part	When a part is enabled for distribution planning it also requires a **TimePhasedMaximumInventory** record that defines the maximum inventory quantity for the part. For more information, see "TimePhasedMaximumInventory table" on page 654. Note that if the **TimePhasedMaximumInventory.Quantity** field is set below the effective safety stock level for the part, for example **0**, then the effective safety stock level is used as the maximum inventory level in the calculation.

Distribution planning calculations

Distribution planning logic is enabled for a part if it references a **DistributionPlanningRule.ProcessingRule** value of **Enable**. When the pre-built inventory at the sourcing site exceeds the set maximum inventory level , this generates a request to transfer (push) a portion of the pre-built supply of the finished goods to a distribution site. When RapidResponse performs distribution planning calculations to determine if pre-built inventory can be pushed, the quantity that is specified to be pushed needs to bring the inventory to the maximum inventory level for the date that it's set to be transferred.

Additionally, fair share distribution logic is also considered in the calculation to determine the allocations. Fair share distribution ensures that the pre-built supply is pushed to distribution sites in a fair and equitable manner. RapidResponse considers any future demand for a site to determine the quantity that it will recommend be pushed to the site. The push requests follow the existing **PartSource** paths on the dependent demands. Therefore, RapidResponse will not recommend to push to distribution sites that are not requesting the product.

The supply that is allocated to be pushed uses the following:

- PlanningCalendars.AllocationCalendar
- Part.AllocationMultiple
- PartSource.MultipleQty (tries to respect)

RapidResponse also ensures that the quantities recommended for pushing meet the parent part source's required MultipleQty (lot size). For example, assume a part exceeds the maximum inventory that was set and has 400 units at the sourcing site, however, the lot size for the part at the distribution site is set to 500 units. In this case, RapidResponse will not generate a push request if there is not enough available supply (500 units) at the sourcing site to push to the distribution site while respecting the lot size requirements.

The **TimePhasedMaximumInventory.Quantity** should not be set lower than the effective safety stock level for the part because RapidResponse will use the effective safety stock level in the calculation instead. For example, if the **TimePhasedMaximumInventory.Quantity** is set to **0** units and the effective safety stock level is set to **500** units, then **500** units is used as the maximum inventory level in the calculation.

Note: To ensure the parent part source's **MultipleQty** is respected in cases where both minimum and multiple quantities are applicable, the **OrderPolicy.MinimumUsage** should be set to "RoundMult". If **MinimumUsage** is set to "Use", then distribution planning logic might not always respect the parent part source's **MultipleQty**.

Pushed supply activity

Push planning details are reported in the **CTPActivity** and **SupplyDemand** tables as follows.

- **CTPActivity**-demand activity is generated at the source site when the maxmimum inventory level is exceeded and pre-built supply is pushed out early to meet future demand at a destination/distribution site. This is represented on records that have the **PushTransferId** field populated and the **Type** field set to "PushTransferOut". The receipt of these pushed transfers is then seen on supply activity at the destination site. This is represented by records that have the **ExtendedType** field set to "PushTransfer".
- **SupplyDemand**-the **PushDate** field is populated on records that represent supply being pushed out early from the sourcing site to meet future demand at the destination/distribution site. This represents the date on which the supply should be transferred to bring the inventory level back down the maximum quantity.

Note that, by default, the activity described above is only generated when supply is actually pushed out early due to the part's maximum inventory level being exceeded. Otherwise, supply of a part is seen as being pulled by the demand it satisfies and pushed supply actions are not generated. For example, this would occur if supply was available late to meet the demand it satisfies at the destination site or if the supply is on-time but the maximum inventory level was not exceeded. In these cases, planned transfers would be used to represent the demand pulling the supply from the source site.

However, if the part's **DistributionPlanningRule.SupplyPushRule** is set to "All" (rather than default of "Early"), then pushed supply actions are generated for all transfers of supply from the sourcing site to meet demand at destination sites (assuming the **DistributionPlanningRule.ProcessingRule** is also set to "Enable"). This ensures consistent reporting of transfers of supply from source sites to the destination/distribution sites, and also ensures that these pushed supply transfers are aligned to respect the **MultipleQty** on the parent part at the destination site. That is, supplies are only recommended for pushing when a full lot size is accumulated and available dates on the pushed supply actions are aligned to ensure this. Otherwise, when supplies are pulled to satisfy demand, it is the **AvailableDate** on the supplies that is seen to indicate when the supply is available to be be shipped.

For an example of the impact of the **SupplyShareRule** setting, see "Example of distribution planning calculations with late supplies" on page 1723.

Examples

This section includes the following examples of push planning calculations:

- "Example of basic distribution planning calculations" on page 1720
- "Example of multi-tiered distribution planning calculations" on page 1721
- "Example of fair share distribution planning calculations" on page 1722
- "Example of distribution planning calculations impacted by lot sizes" on page 1723
- "Example of distribution planning calculations with late supplies" on page 1723

Example of basic distribution planning calculations

If a part is enabled for distribution planning and a **TimePhasedMaximumInventory** quantity, the distribution planning calculation determines if there is pre-built supply that is available to be pushed from a sourcing site to distribution sites.

In the following example, the manufacturing site (Mfg1) sources the central distribution centers (DC). Now, assume a part has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and has the following setting at Mfg1:

- **Maximum Inventory**—500 units per month

Figure 31.1 Example of basic distribution planning calculations

		June	July	Aug	Sept	Oct	Nov	Dec
DC1	Demand			0	0	500	500	500
	Netting Supply					500	500	500
	Push Transfer			250	250	250	250	
	Pull							500
DC2	Demand			0	0	500	500	500
	Netting Supply					500	500	500
	Push Transfer			250	250	250	250	
	Pull							500
Mfg1	Demand				1000	1000	1000	
	Netting Supply	500	500	500	500	500	500	
	CTP Supply	500	500	500	500	500	500	
	Ending Inventory	500	500	500	500	500	0	

When RapidResponse performs the distribution planning calculation, it determines if **TimePhasedMaximumInventory** value of 500 units was exceeded at the manufacturing facility (Mfg1) and then calculates how much pre-built inventory can be pushed to the distribution sites.

In the example, the manufacturing site Mfg1 produced 500 units in both July and August, which is reflected in the CTP Supply. As a result, this caused the **TimePhasedMaximumInventory** value to be exceeded by 500 units during these months. To maintain the ending inventory level for site Mfg1 at 500,

RapidResponse generates push requests to transfer 250 units to both distribution sites, DC1 and DC2, every month from August to November.

However, there is demand for 500 units per month at DC1 and DC2 starting in October. Once the pushed supply is consumed by the demand, RapidResponse starts pulling the demand in December.

Example of multi-tiered distribution planning calculations

You can also set the maximum inventory level at multiple tiers on a finished good part. Therefore, once the maximum level of inventory is reached for a part at one site, it generates a request to push the supply the next distribution site.

In the following example, the manufacturing site (Mfg1) sources the central distribution centers (CDC1 and CDC2) and the central distribution centers source the regional distribution centers (RDC1 and RDC2). Now, assume a part has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and has the following settings:

- Mfg1 **Maximum Inventory**—250 units per month
- CDC1 **Maximum Inventory**—400 units per month
- CDC2 **Maximum Inventory**—400 units per month

Figure 31.2 Example of multi-tiered distribution planning calculations

		Jan	Feb	Mar	Apr	May	June	July	Aug	Sept	Oct	Nov	Dec	
RDC1	Demand										500	500	500	
	Netting Supply										100	500	500	
	Push Transfer					100	125	125	125	125	125	275		
	Pull												500	
	Ending Inventory					100	225	350	475	600	225	0	0	
RDC2	Demand										500	500	500	
	Netting Supply										100	500	500	
	Push Transfer					100	125	125	125	125	125	275		
	Pull												500	
	Ending Inventory					100	225	350	475	600	225	0	0	
CDC1	Demand										500	500	500	
	Netting Supply										500	500	500	
	Push Transfer		125	125	125	125	125	125	125	125	125			
	Pull											125	250	
	Ending Inventory		125	250	375	400	400	400	400	400	400	250	0	
CDC2	Demand										500	500	500	
	Netting Supply										500	500	500	
	Push Transfer		125	125	125	125	125	125	125	125	125			
	Pull											125	250	
	Ending Inventory		125	250	375	400	400	400	400	400	400	250	0	
Mfg1	Demand	0	0	0	0	0	0	0	0	0	1000	1000	1000	
	Netting Supply	250	250	250	250	250	250	250	250	250	250	250	250	
	CTP Supply	250	250	250	250	250	250	250	250	250	250	250	250	
	Ending Inventory	250	250	250	250	250	250	250	250	250	250	250	0	

When RapidResponse performs the distribution planning calculation, it first determines if **TimePhasedMaximumInventory** value of 250 units was exceeded at the manufacturing facility (Mfg1) and then calculates how much pre-built inventory can be pushed to the first tier of the distribution sites.

Starting in February, the manufacturing site Mfg1 produced 250 units per month, which is reflected in the CTP Supply. The **TimePhasedMaximumInventory** at Mfg1 is exceeded by 250 units in each month starting in February. Therefore, RapidResponse starts pushing the pre-built inventory to the first tier distribution sites, CDC1 and CDC2, in order to maintain the Ending Inventory level at 250 at the Mfg1 site.

Both CDC1 and CDC2 have their maximum inventory levels set to 400. When the maximum inventory level is exceeded in April, RapidResponse generates push requests to start transferring units to distribution sites RDC1 and RDC2. From October to December, there is demand for 1000 units per month. Once the pushed supply is consumed, RapidResponse starts pulling the demand in November for CDC1 and CDC2. Then it starts pulling demand for RDC1 and RDC2 in December.

Example of fair share distribution planning calculations

In the previous examples, the quantities of pre-built supply are equally pushed to the distribution sites. However, if there are future demands, RapidResponse fair share distribution logic considers any future demand for a site to determine the quantity that it recommends be pushed to the site. The push requests follow the existing **PartSource** paths on the dependent demands. Therefore, RapidResponse will not recommend to push to distribution sites that are not requesting the product.

In the following example, the manufacturing site (Mfg1) sources the central distribution centers (CDC). Now, assume a part has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and has the following setting at Mfg1:

- Mfg1 **Maximum Inventory**—250 units per month

Figure 31.3 Example of fair share distribution planning calculations

		Dec	Jan	Feb	Mar	Apr	May	June	July	Aug	Sept	Oct	Nov
CDC1	Demand		0	0	0	0	0	0	0	0	700	500	300
	Netting Supply										700	500	300
	Push Transfer		145	145	145	145	145	140	140	140	145	85	0
	Pull												125
	Ending Inventory		145	290	435	580	725	865	1005	1145	590	175	0
CDC2	Demand		0	0	0	0	0	0	0	0	500	400	600
	Netting Supply										500	400	600
	Push Transfer		105	105	105	105	105	110	110	110	105	165	0
	Pull												375
	Ending Inventory		105	210	315	420	525	635	745	855	460	225	0
Mfg1	Demand	0	0	0	0	0	0	0	0	0	1200	900	900
	Netting Supply	250	250	250	250	250	250	250	250	250	250	250	250
	CTP Supply	250	250	250	250	250	250	250	250	250	250	250	250
	Ending Inventory	250	250	250	250	250	250	250	250	250	250	250	0

When RapidResponse performs the distribution planning calculation, it determines that the **TimePhasedMaximumInventory** value of 250 units was exceeded at the manufacturing facility (Mfg1). In addition, it also takes into consideration the future demand in September, October, and November for both CDC1 and CDC2 in order to determine how much supply should be pushed.

Starting in December, the manufacturing site Mfg1 produced 250 units per month, which is reflected in the CTP Supply. The **TimePhasedMaximumInventory** at Mfg1 is exceeded by 250 units in each month starting in January. Therefore, RapidResponse starts pushing the pre-built inventory to CDC1 and CDC2 based on based on the dependent demands quantities in September. Once the dependent demands in September have been fully satisfied, then the allotment quantities are based on the dependent demand quantities for October in order to maintain the ending inventory level at 250 at the Mfg1 site.

Once the pushed supply is consumed, RapidResponse starts pulling the demand in November for CDC1 and CDC2.

Example of distribution planning calculations impacted by lot sizes

In most instances, RapidResponse will enforce the lot-sizing of a **PartSource** (MultipleQty). Therefore, it will not generate a push request if the quantity of pre-built supply violates the lot size at the distribution site.

For example, assume a part has the **DistributionPlanningRule.ProcessingRule** set to **Enable** and has the following settings:

- Mfg1 **Maximum Inventory**—250
- CDC1 **Lot size**—Multiple 25
- CDC2 **Lot size**—Multiple 10

Figure 31.4 Example of distribution planning calculations impacted by lot sizes

		Dec	Jan	Feb	Mar	Apr	May	June	July	Aug	Sept	Oct	Nov
CDC1	Demand		0	0	0	0	0	0	0	0	500	500	500
	Netting Supply										500	500	500
	Push Transfer		125	125	125	125	125	125	125	125	125	125	0
	Pull												250
	Ending Inventory		125	250	375	500	625	750	875	1000	625	250	0
CDC2	Demand		0	0	0	0	0	0	0	0	500	500	500
	Netting Supply										500	500	500
	Push Transfer		130	120	130	120	130	120	130	120	130	120	0
	Pull												250
	Ending Inventory		130	250	370	500	620	750	870	1000	620	250	0
Mfg1	Demand	0	0	0	0	0	0	0	0	0	1000	1000	1000
	Netting Supply	250	250	250	250	250	250	250	250	250	250	250	250
	CTP Supply	250	250	250	250	250	250	250	250	250	250	250	250
	Ending Inventory	250	245	250	245	250	245	250	245	250	245	250	0

When RapidResponse performs the distribution planning calculation, it determines that the **TimePhasedMaximumInventory** value of 250 units was exceeded at the manufacturing facility (Mfg1). In addition, it also takes into consideration the lot sizes that are set for CDC1 (Lot size—Mult 25) and CDC2 (Lot size—Mult 10) in order to determine how much supply should be pushed.

Starting in January, the **TimePhasedMaximumInventory** at Mfg1 is exceeded by 250 units. Therefore, RapidResponse starts pushing the pre-built inventory to CDC1 and CDC2 bin January to maintain the ending inventory level at 250 at the Mfg1 site. However, due to the lot size requirements set for CDC1 and CDC2, 5 units cannot be pushed in January, March, May, July, and September. As a result, the ending inventory at Mfg1 for those months is 245, which is below the maximum inventory level by 5 units.

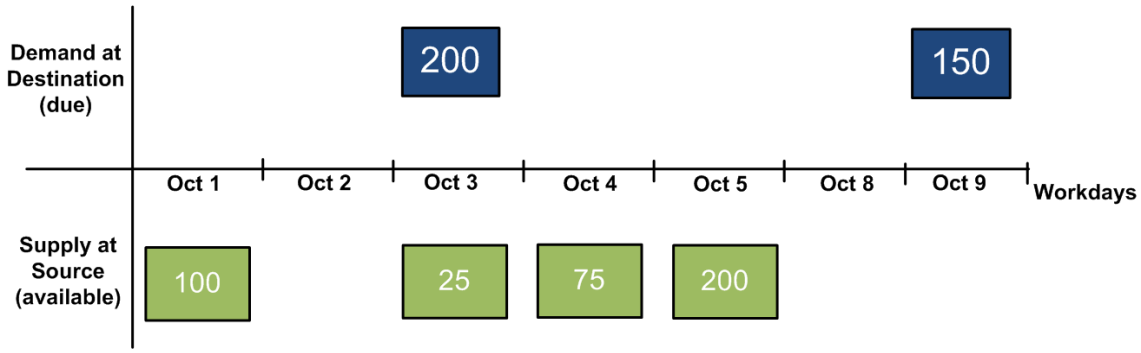
Once the pushed supply is consumed, RapidResponse starts pulling the demand in November for CDC1 and CDC2.

Example of distribution planning calculations with late supplies

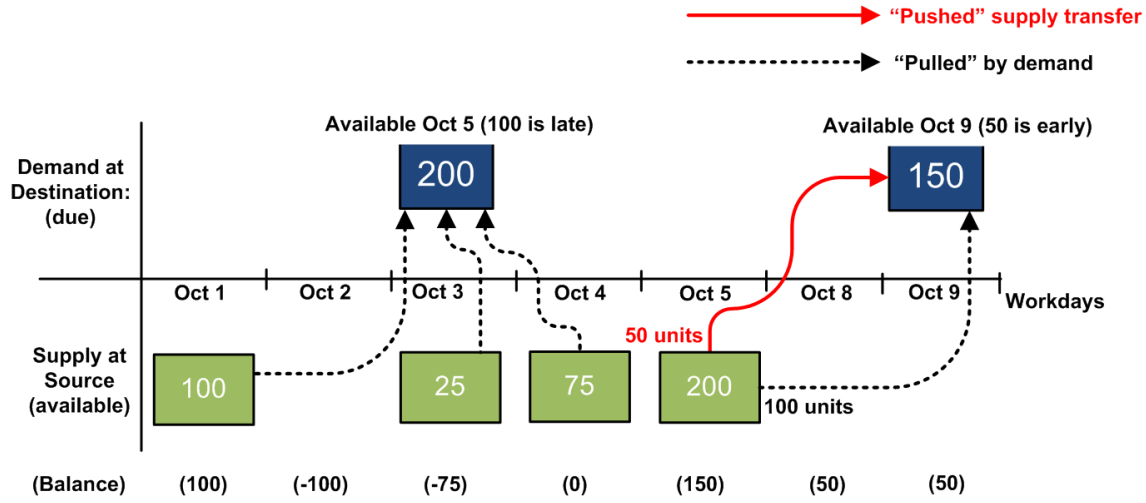
This example shows the impact of the **DistributionPlanningRule.SupplyPushRule** setting.

Assume the following:

- A part manufactured at a sourcing site has **DistributionPlanningRule.ProcessingRule** set to "Enable"
- The **TimePhasedMaximumInventory.Quantity** for this part is 150
- 1 day of transit time is required to ship supply from the source site to the distribution site that receives demand.
- The **MultipleQty** for the parent part at the distribution site is 50
- Scheduled supply for the part at the source site, and demand for the parent part at the distribution site, is as shown in the following illustration



With this configuration, supply at the source site is pushed out early to the destination site when the part's maximum inventory level of 150 is exceeded. As shown in the following illustration, this occurs on October 5th, when 50 units of supply are pushed out 1 day earlier than needed to meet the October 9th demand at the distribution site. This brings inventory of the part at the source site back to the maximum inventory level. All other supplies and supply portions are pulled by the demands that require them.



If the part at the source site has **DistributionPlanningRule.SupplyPushRule** set to "Early", then pushed supply activity is only generated for the supply portion that was actually pushed out early. Thus, the **CTPActivity** values shown in the following table are reported for the part at the source site.

Note that a single demand record with **Type** set to "PushTransferOut" is reported on October 5th corresponding to the 50 units that were shipped early to their demand at the destination site. All other demand records at the source site, representing dependent demands that pulled supply to the distribution site, are reported with a **CTPActivity.Type** value of "PlannedTransferAllocation".

Table 31.2: CTPActivity values - SupplyPushRule set to "Early"

DemandDate	AvailableDate	Type	DemandQty	SupplyQty
Oct 2	Oct 1	ScheduledReceipt		100
Oct 2	Oct 4	PlannedTransferAllocation	200	
Oct 2	Oct 3	ScheduledReceipt		25
Oct 2	Oct 4	ScheduledReceipt		75
Oct 8	Oct 5	ScheduledReceipt		200
Oct 5	Oct 5	PushTransferOut	50	
Oct 8	Oct 8	PlannedTransferAllocation	150	50

Instead, if the part at the source site has **DistributionPlanningRule.SupplyPushRule** set to "All", then pushed supply actions would be generated corresponding to all supply portions that are transferred to satisfy demand at the distribution site. Thus, the **CTPActivity** values shown in the following table are reported for the part at the sourcing site.

Note that "PlannedTransferAllocation" records are not shown here, but would still be generated albeit with a **BalanceDelta** value of "0" and can typically be ignored as a result. Instead all demand portions being satisfied by transferred supply are now shown on records with a **Type** of "PushTransferOut". This includes late supplies, and is regardless of whether the supplies are being pushed to or pulled by the demands. Note also that the "PushTransferOut" for 25 units is seen as being available on October 4 even though the corresponding scheduled receipt is available October 3. This is done to respect the **MultipleQty** on the parent part at the destination (the pushed transfer action here is not recommended until the supply from October 4th is also accumulated and allows the transfer to be made in multiples of 50).

Table 31.3: CTPActivity values - SupplyPushRule set to "All"

DemandDate	AvailableDate	Type	DemandQty	SupplyQty
Oct 2	Oct 1	ScheduledReceipt		100
Oct 2	Oct 2	PushTransferOut	100	
Oct 2	Oct 3	ScheduledReceipt		25
Oct 2	Oct 4	ScheduledReceipt		75
Oct 4	Oct 4	PushTransferOut	25	
Oct 4	Oct 4	PushTransferOut	75	
Oct 8	Oct 5	ScheduledReceipt		200
Oct 5	Oct 5	PushTransferOut	50	
OCt 8	Oct 8	PushTransferOut	100	

Forward-bucketed fair share

You can use the forward-bucketed fair share allotment logic to allocate limited supplies to distribution sites. The forward-bucketed fair share logic is different than the fair share option that is available on the **OrderPriority.AllocationRule** field because it moves from earliest to latest buckets and also shares on-time supply, instead of only late supply. For example, when demands are not fully satisfied in one bucket, the shortage is carried to the next bucket. RapidReponse then recalculates the ratios based on the shortages in the earlier buckets and the demands in the current bucket. The bucket intervals are defined by the Allocation calendar.

Once the shortages are resolved, the forward-bucketed fair share logic also takes into consideration the safety stock level at the distribution sites when calculating the allocation ratios. The forward-bucketed fair share option also ensures the replenishment of safety stock levels at the distribution centers.

The forward-bucketed fair share capabilities are supported by the **AllotmentRule** field on the **DistributionPlanningRule** control table.

Notes:

- The **ForwardBucketedFairShare** allotment logic does not take **OrderPriority** into consideration when allocating the supply to distribution sites. For example, **ForwardBucketedFairShare** will satisfy the independent demand of the sourcing site before satisfying the dependent demands from distribution sites during bucket processing.
- The **ForwardBucketedFairShare** allotment logic does not take expiry dates into consideration when allocating the supply to distribution sites.

Satisfying shortages using forward-bucketed fair share logic

The forward-bucketed fair share logic allocates supply that is available at the sourcing site in order to satisfy shortages at distribution sites. To calculate how much supply should be allocated to each distribution site, RapidResponse does the following:

- Determines the Total Stock Recovery Ratio (TSRR) by dividing the total available supply in a given bucket by the sum of the inventory shortages that are found at the distribution sites.
- Multiplies the TSRR by the shortage at each distribution site to determine the amount of supply to be allocated to each distribution site.

If shortages persist and there is still available supply, RapidResponse calculates the Stock Recovery Ratio (SRR) to determine where the remainder of the supply should be allocated. The SRR is calculated by dividing the current amount of supply that was received in the bucket by the original amount of the inventory shortage.

Once the shortages are resolved, if the sourcing site still has available supply and the distribution sites are below their safety stock levels, the forward-bucketed fair share logic calculates how to allocate the supply to replenish the safety stock. The calculations are as follows:

- Determines the Total Stock Sufficiency Ratio (TSSR) by adding the available supply in a given bucket with the total of the inventory that is found at the distribution sites. It then divides the amount by the sum of the safety stock levels that are set for the distribution sites.

- Multiplies the TSSR by the current safety stock level at each of the distribution sites to determine the amount of supply to be allocated to each site.

If safety stock levels are not fully replenished and there is still available supply, RapidResponse calculates the Stock Sufficiency Ratio (SSR) to determine where the remainder of the supply should be allocated. The SSR is calculated by dividing the current level of safety stock that was satisfied in the bucket by the desired safety stock level for the site.

Example of satisfying shortages at distribution sites

In the following example, there are shortages for Part A at distribution centers DC1 and DC2, but there is supply available at the manufacturing site Mfg1.

Table 31.4:

		06-01-15	06-08-15	06-15-15	06-22-15
DC1	**Demand**	700		1,400	400
	Supply	1,200			**200**
	Inventory	1,500	1,500	100	-100
	Safety stock	700			
DC2	**Demand**	500		1,500	500
	Supply	1,000		**400**	
	Inventory	1,000	1,000	-100	-600
	Safety stock	500	1,000		
Mfg1	**Demand**		2,500	1,200	
	Supply		**600**	2,000	1,100
	Inventory	0	-1,900	-1,100	0

The details reveal that:

- Part A at manufacturing site Mfg1 has 600 units of supply in the 06-08-15 bucket.
- Distribution centers DC1 and DC2 have demand for 2500 units in the same period. However, you can assume that real demand from forecast and sales actuals total 800 units and another 1700 units of the demand is required to replenish safety stock levels.

The forward-bucketed fair share logic focuses on the 800 units of real demand for Part A at distribution centers DC1 and DC2. The demand is broken down as follows:

- DC1 has demand for 300 units
- DC2 has demand for 500 units

RapidResponse performs the following calculation to determine the TSRR, which is used to calculate how much supply should be allocated to each distribution site:

- 600 units total supply / 800 units total shortage = 0.75

RapidResponse then multiplies the demand at each distribution center by the 0.75 TSRR and rounds down the amounts as follows:

- DC1 300 units X 0.75 = RoundDownToMult(225) 200 units
- DC2 500 units X 0.75 = RoundDownToMult(375) 300 units

Initially, Mfg1 had 600 units of supply in the 06-08-15 bucket. After the first round of calculations, 100 units of supply is still available at site Mfg1.

To determine where the 100 units of supply should be allocated, RapidResponse calculates the Stock Recovery Ratio (SRR). The SRR is calculated by dividing the current amount of supply that was received in the bucket by the original amount of the inventory shortage.

In the example, RapidResponse performs the following calculations to determine the SRR values:

- DC1 200 units current supply / 300 units original inventory shortage = 0.6666
- DC2 300 units current supply / 500 units original inventory shortage = 0.6

The distribution site with the smallest SRR value would receive the supply. In the example, DC2 has the smallest SRR and, therefore, will receive the 100 units.

In summary, the distribution sites received the following supply:

- DC1 200 units
- DC2 400 units

Example of satisfying shortages and replenishing safety stock

In the following example, there are 2000 units of supply available for Part A at the manufacturing site Mfg1 in the 06/15/15 bucket. The distribution centers DC1 and DC2 require 3100 units during the same bucket.

Table 31.5:

		06-01-15	06-08-15	06-15-15	06-22-15	06-29-15
DC1	**Demand**	700		1,400	400	700
	Supply	1,200			200	1,100
	Inventory	1,500	1,500	100	-100	300
	Safety stock	700				
DC2	**Demand**	500		1,500	500	500
	Supply	1,000		400	900	
	Inventory	1,000	1,000	-100	-600	-200
	Safety stock	500	1,000			500
Mfg1	**Demand**		2,500	1,200		1,200
	Supply		600	2,000	1,100	1,200
	Inventory	0	-1,900	-1,100	0	

By examining the 3100 units of demand from DC1 and DC2, the units reveal that:

- 1400 units of demand are from forecast and sales actuals (800 units for DC1 and 600 units for DC2).
- 1700 units of the demand is required to replenish safety stock levels (700 units for DC1 and 1000 units for DC2)

The forward-bucketed fair share logic starts by focusing 1400 units of demand for Part A at distribution centers DC1 and DC2. The demand is broken down as follows:

- DC1 has demand for 800 units
- DC2 has demand for 600 units

RapidResponse performs the following calculation to determine the TSRR, which is used to calculate how much supply should be allocated to each distribution site:

- 2000 units total supply / 1400 units total shortage = 1.42 (The ratio is greater than 1, which indicates that there is enough supply to satisfy all of the demand. Therefore, the TSRR is 1.)

RapidResponse then multiplies the demand at each distribution center by the 1 TSRR as follows:

- DC1 800 units X 1 = RoundDownToMult(800) 800 units
- DC2 600 units X 1 = RoundDownToMult(600) 600 units

This resolves all of the demand shortages at DC 1 and DC2. However, there is still supply available to replenish the safety stock. The forward-bucketed fair share logic moves onto calculating the amount of supply that can be allocated to replenishing safety stock at each of the sites. RapidResponse performs the following calculation to determine the TSSR:

- ((0+0) total inventory at distribution sites + 600 units of available supply in bucket) / (700 + 1000) sum of the safety stock levels for the distribution sites = 0.352 Total Stock Sufficiency Ratio (TSSR)

RapidResponse then multiplies the TSSR by the safety stock level at each distribution site to determine the amount of supply that needs to be allocated to each distribution site:

- DC1 700 units X 0.352 = RoundDownToMult(246) 200 units
- DC2 1000 units X 0.352 = RoundDownToMult(352) 300 units

Initially, Mfg1 had 600 units of available supply to replenish the safety stock. After the first round of allocations, 100 units of supply is still available at site Mfg1.

To determine where the 100 units of supply should be allocated, RapidResponse calculates the Stock Sufficiency Ratio (SSR).

In the example, RapidResponse performed the following calculations to determine the SSR values:

- DC1 200 units current level of safety stock that was satisfied in the bucket / 700 desired safety stock level for the site = 0.29
- DC2 300 current level of safety stock that was satisfied in the bucket / 1000 units desired safety stock level for the site = 0.3

The distribution site with the smallest SSR value would receive the supply to replenish its safety stock level. In the example, DC1 has the smallest SRR and, therefore, will receive the 100 units.

In summary, the distribution sites received the following supply:

- DC1 1100 units (800 + 200 + 100)
- DC2 900 units (600 + 300)

RapidReponse then proceeds to the next allocation bucket.

Limitations on other analytics

A small number of other analytics or calculations might be disabled, or not work as expected, for parts with distribution planning enabled. The following table outlines calculations impacted by distribution planning:

Table 31.6: Distribution planning limitations on other analytics

Analytic/Calculation	Limitation
Attribute-based planning (ABP)	The fair-share allotment logic that is associated with distribution planning does not fully respect ABP. Supply may be available earlier then required if the sourcing part has **DistributionPlanningRule.ProcessingRule** set to **Enable**.
Two-date planning	Two-date planning is automatically disabled for parts that have the **DistributionPlanningRule.ProcessingRule** set to **Enable**.
Lot-sizing	When recommending pushed supply transfers from a sourcing site, distribution planning logic ensures those transfers are aligned to respect the multiple quantity of the part at the destination site. That is, pushed supply transfers aren't recommended until a full-lot size has been accumulated. However, minimum and maximum quantities might not be respected by distribution planning logic. Additionally, if minimum and multiple quantities are both applicable and the minimum and multiple values are not the same, then **OrderPolicy.MinimumUsage** must be set to "RoundMult". If MinimumUsage is set to "Use", then distribution planning logic might not always respect the multiple.
Order priority	If fair-share is enabled using **DistributionPlanningRule.AllotmentRule** set to **ForwardBucketedFairShare**, the fair-share logic does not take **OrderPriority** into consideration when allocating supplies to distribution sites.
Expiry	If fair-share is enabled using **DistributionPlanningRule.AllotmentRule** set to **ForwardBucketedFairShare**, the fair-share logic does not take expiry dates into consideration when allocating supplies to distribution sites.

CHAPTER 32: Multi-site calculations

Using RapidResponse you can analyze and manipulate part and other data for multiple sites within and across your organization. Additionally, transfer part relationships can be configured such that demand for a part at one site is satisfied by supply of a part at another site (typically, though not always, these parts will have the same Name value).

In this chapter, you'll learn more about:

Sites **1731**
Transfer part sources **1731**
Transfer scheduled receipts **1734**
Using address data to support maps **1735**

Sites

RapidResponse allows for visibility, analysis, and simulation across multiple sites within an organization. A site is defined as an individual location within a company that has separately defined planning data. This could be a a single geographic location, a plant, or a department within a plant. Thus, if a given part needs to be netted separately in two different locations, those locations will probably be defined as sites in RapidResponse. Similarly, if a given part needs to have different planning parameters such as lead-time or order policy information or safety stock in two different locations, those locations will be defined as Sites.

Sites are defined within the Site table and references are made from other data tables such as Part, WorkCenter, Constraint, PartSource, BillOfMaterial, OnHand, IndependentDemand, and ScheduledReceipt to the Site table to ensure data integrity. These site fields on other data tables are usually part of the record key on that table. For tables like, PartSource, BillOfMaterial, IndependentDemand and ScheduledReceipt, the Site key is specified through a key reference to another table like Part, SupplyOrder or DemandOrder which directly reference the Site table.

The sites that are modeled in RapidResponse may be isolated and independent of each other or they may participate with each other as trading partners in the supply chain (for example, supply of part at one site might be used to satisfy demand for the part at another site).

Transfer part sources

A **PartSource** record is the primary source of information about how a part can be sourced or supplied. Thus, to support transfers of supply/demand between sites, a **PartSource** record is required at the

consuming site that tells RapidResponse that supply for the part should be provided by a part at the sourcing site. Thus, each PartSource record has a Type reference which determines how the part is to be sourced. The sourcing method for each source can be determined through the **Type.PlannedOrderSupplyType.Source** reference (points to the **SupplyType** table) which has the following three possible values:

Table 32.1: SupplyType.Source values

Option	Description
Buy	Indicates that RapidResponse will not generate any further dependent demands for supply using the PartSource. The expectation is that purchase orders are created and communicated to suppliers that manage their own planning.
Make	Indicates that RapidResponse will look at BillOfMaterial records for the Part in order to determine which set of component parts are required to create the supply for this part. Typically, this type of source represents cases where the site itself builds the supply for this part, and is associated with work orders and kitting operations where several different components are assembled together to form a new assembly. This is the default setting.
Transfer	Indicates that supply for the part will be provided from another part and site. Unlike Make sources, it does not transfer to multiple components or consider BillOfMaterial records. Instead, supply of the part at the consuming site simply creates a demand for the part at the supplying site. Thus, the expectation is if the consuming site has a need for the part, it will obtain or source what it needs from the supplying site. This is the setting required to support transfer operations as described throughout this lesson.

Additionally, each **PartSource** record has two distinct references to the **Part** table (**Part.Name**/**Part.Site** and **TransferPart.Name**/**TransferPart.Site**). On non-transfer **PartSource** records, these references resolve to the same part and site combination.

On transfer **PartSource** records, it is not required that the **Part.Name** and the **TransferPart.Name** be the same, though they typically are (for example, they might represent the same part number at different locations). However, the **Part.Site** and **TransferPart.Site** values should be different for transfer sources with **Part.Site** identifying the consuming or demand site and **TransferPart.Site** identifying the sourcing or supply site.

Note: A transfer relationship between parts can exist in one direction only. For example, if Part A at Site 1 sources its supply from Part A at Site 2, then it is not possible for Part A at Site 2 to source its supply from Part A at site 1. Such a configuration would be flagged as recursive data.

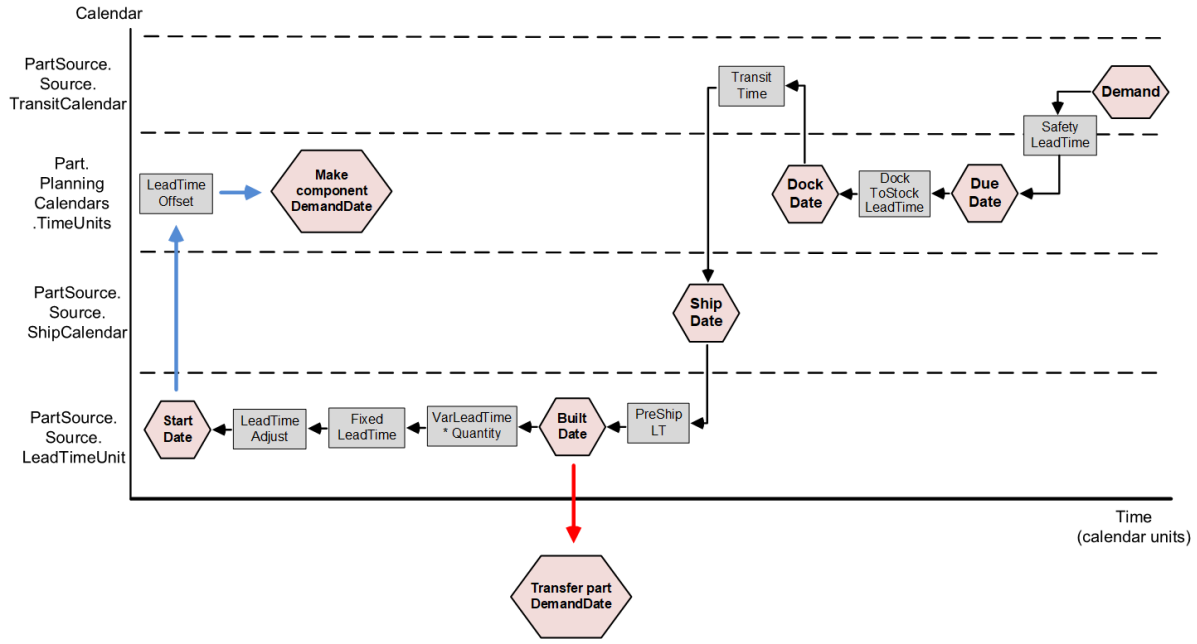
Transfer lead time and explosion

When exploding through a transfer **PartSource** record, the supplying part's demand date is not derived from the parent supply **StartDate** as is the case with other source types. Since RapidResponse is modeling the transfer of the supply from the supplying site to the consuming site, the demand is placed on the supplying site as of the **BuiltDate** of the supply at the consuming site.

Refer to the following illustration. From this, it can be seen that all lead time elements and calendars used to determine the **DemandDate** at the supplying site (red arrow) are defined by the consuming site. That is, the transfer supply places demand on the consuming site after accounting for safety, dock-to-stock, transit, and pre-ship lead time from the transfer part source at consuming site. Note however that the demand is placed on the supplying site without including fixed, variable, or adjusted lead time

at the consuming site. These values are used to determine a start date for the transfer order at the consuming site, but they have no effect on the transfer of demand to the sourcing site that will actually provide the supply.

Figure 32.1 Cumulative lead time used for transfer part source



Note: The **PartType.SafetyLeadTimeCalendarRule** control setting determines whether safety lead time is applied using the part's **TimeUnits** calendar or in the "Everyday" calendar. For more information, see "Safety lead time calendar" on page 1322.

UOM conversion

While there is no Quantity-Per associated with explosion through a transfer PartSource, there can be a difference in the supply quantity at the consuming site and the demand quantity it places on the supplying site. Normally, the supply (planned order) at the consuming site is expressed in the **PartSource.SupplierUOM**. If the supplying site **Part.UnitOfMeasure** is different than the **SupplierUOM** then the supply quantity is 'converted' using the **UnitOfMeasure.BaseConversion** value effective for the two units-of-measure.

Given that the **TransferPart** on the consuming site's **PartSource** record points to the supplying site part, the conversion factor can be determined completely from the consuming site's **PartSource** record as:

```
ConversionFactor = SupplierUOM.BaseConversion /
TransferPart.UnitOfMeasure.BaseConversion
```

Thus, the demand quantity at the sourcing site is determined from the original supply quantity at the consuming site as follows:

```
Demand = SupplyQuantity * ConversionFactor
```

Transfer scheduled receipts

Scheduled receipts can also be set up as transfer orders. Similar to part sources for planned orders, each transfer receipt needs to reference a **SupplyType.Source** value of "Transfer". This is set through the **Order.Type.Source** reference on the scheduled receipt (**Type** is defined at the **SupplyOrder** level).

Additionally, to be treated as a transfer order, a **ScheduledReceipt** record also needs to reference a valid "transfer" PartSource record. As is discussed in Lesson 8 and the topic "Part sources for scheduled receipts" on page 1389, a set of standard criteria is used to calculate and assign a **PartSource** record to each scheduled receipt (including comparing values against the **ScheduledReceipt** record such as supplier and type). Thus, it is important to ensure that there is a "Transfer" **PartSource** record defined that can potentially be associated with the **ScheduledReceipt** so that it knows the correct **TransferPart** to explode to.

In-transit scheduled receipts

Sometimes a consuming site might have an open transfer supply order that the sourcing site has already shipped some portion of, but which has not yet been received at the consuming site. To reconcile this difference and prevent explosion through demand quantities that have already shipped, "in-transit" versions of **ScheduledReceipt** records can be created by the sourcing site to indicate that some or all of the order is in-transit. These ScheduledReceipts are made against the part at the consuming site, but reflect information as known at the sourcing site. Consequently, the original scheduled receipts are reduced or consumed by the amount that is currently in-transit.

In-transit scheduled receipts are meant to provide the latest shipping information on a transfer order as is known at the sourcing site. Although these receipts typically provide information from the sourcing site, their **Part** and **Site** fields must reference the part and site values at the consuming site. Additionally, the **Quantity** field on the scheduled receipt is used to indicate the amount that is in-transit. Finally, to indicate that a given **ScheduledReceipt** record is modelling an in-transit shipment, it should reference one of the following two **ProcessingRule** values in the **SupplyType** table (as defined through the receipt's **Order.Type** reference):

- **InTransit**—consumes supply from scheduled receipts having the same part, model, unit, and pool. This consumption occurs in chronological date order, and occurs after any consumption done by 'InTransitExact' scheduled receipts. All scheduled receipts with this setting are also treated as 'non-reschedulable'.
- **InTransitExact**—consumes supply from scheduled receipts having the same supplier, routing, part, model, unit, and pool. This consumption occurs in chronological date order, and occurs before any 'InTransit' scheduled receipts. All scheduled receipts with this setting are also treated as 'non-reschedulable'.

Thus, before the Netting analytic performs its standard calculations, any in-transit scheduled receipts are first used to reduce or consume the quantities of related transfer orders. Consumption begins first with "InTransitExact" receipts which consume supply from orders that have the same supplier, routing, and part. "InTransit" receipts are then used to consume supply from any remaining orders for the same part (this may include orders already partially consumed by InTransitExacts). Both "IntransitExact" and "Intransit" scheduled receipts consume supply in chronological date order, and respect all relevant model, unit, and pool settings.

As a result, when Netting calculates dependent demands from a scheduled receipt, it does so based on the order quantity remaining after in-transit consumption, and not on the original order quantity. The

amount consumed from a given **ScheduledReceipt** record is reported in the calculated **InTransitConsumedQuantity** field.

Note also that once a transfer supply order has been fully received at the consuming location, its **SupplyStatus.Reschedule** field can be set to 'Received' as any normal scheduled receipts with this are no longer eligible to be consumed. Therefore, any 'InTransit' or 'InTransitExact' receipts created to model in-transit portions of the order should also be deleted (otherwise, they will continue to potentially consume from matching or eligible supply orders).

Caution: Although typically intended for transfers, in-transit records can reduce the effective amount of dependent demand placed on the source for a "make" scheduled receipt that matches the 'InTransit' or 'InTransitExact' consumption criteria.

Using address data to support maps

Some RapidResponse implementations might include dashboards that display locations on maps. Four tables have Address fields that can be used to display locations on maps:

- CustomerDestination
- LogisticsLocation
- Site
- Supplier

Address format guidelines

Addresses are converted to geocodes so that their locations can be displayed on maps. For this to work, the addresses must be accurate and complete.

Use the following guidelines to ensure that address data will display on a map:

- Use the address format defined by the national postal service of a country.
- If they are not included in the address format defined by a country's postal service, do not include additional details such as:
  - Business names
  - Unit numbers
  - Floor numbers
  - Suite numbers
- Avoid using the building name at an address. Use the street number instead.
- Avoid specifying cross streets or intersections for an address. Use the street number instead.
- Do not include non-address details such as nearby landmarks.

Minor variations in the address format might not be a problem. For example, both of the following address formats for Kinaxis headquarters result in the correct geocode being selected:

- 700 Silver Seven Rd, Kanata, ON K2V 1C3
- 700 Silver Seven Road, Kanata, Ontario, Canada

CHAPTER 33: Model and Unit Effectivity calculations

Effectivity is a method for determining if a bill or operation record is used to generate dependent demand. Model Effectivity and Unit Effectivity selects individual bill records or operation records based on characteristics of the supply order being exploded. The data displayed in all RapidResponse worksheets are calculated using bill and operation effectivity rules and netting.

Explosions determine if a bill is effective based on the following settings from the **BOMType** and **OperationType** tables:

- a date factor
- a model and unit factor

Model and Unit Effectivity are enabled as a single capability (Model-Unit Effectivity). You can use model and unit effectivity by themselves, or together. Similarly, you can enable model and unit netting individually, or in combination with each other. Netting is performed separately for each unique combination of model and unit.

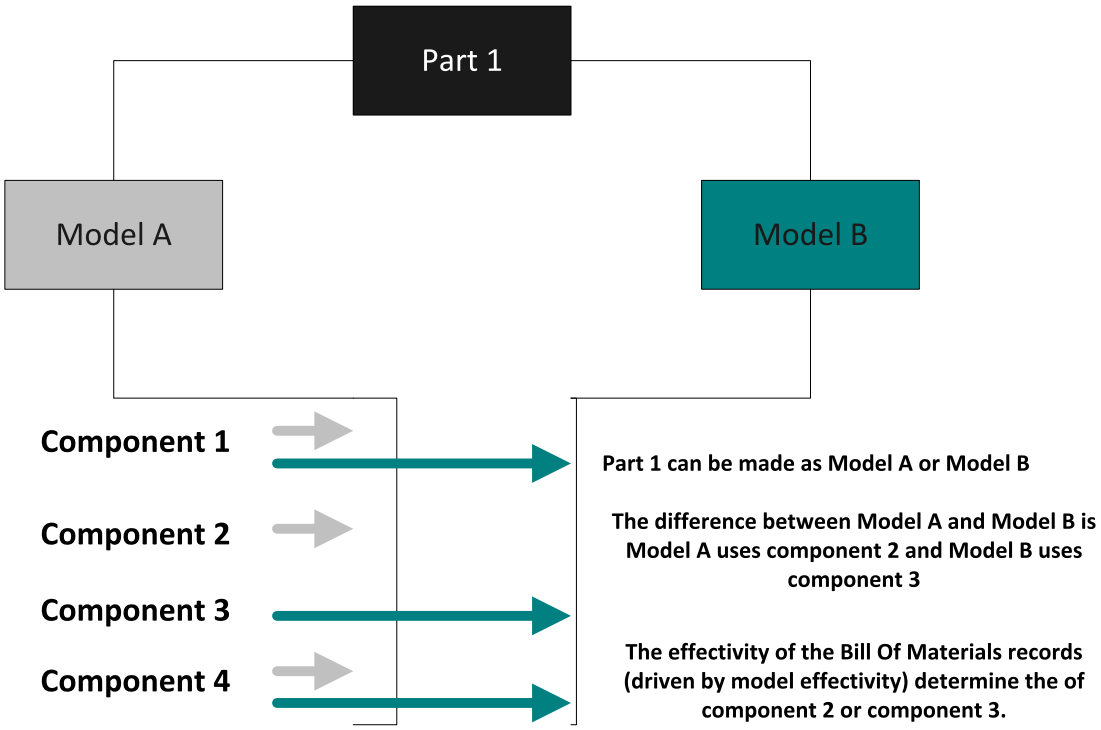
In this chapter, you'll learn more about:

Model effectivity 1737
Unit effectivity 1738
Explosion based on effectivity 1739
Cumulative lead time 1741
Capacity load calculations 1742

Model effectivity

Model effectivity specifies that the bill record is effective if the model number on the bill record matches the model number on the supply record. A model is a variant of a specific part. It uses the same part number, but is different from other models of the same part. RapidResponse uses a single part number for each model and part combination. Supply of one model is not used to satisfy demand for a different model.

Figure 33.1 Example of model effectivity



Example of model effectivity

For the purpose of model effectivity, consider aircraft engines. An aircraft engine has a part number. There are different models that represent left and right side mounting. The right side model uses a right side fuel pump and engine mounts. The left side model uses a left side fuel pump and engine mounts. The bill of material for the engine uses model effectivity to generate dependent demand for either right side mounts and fuel pumps or left side mounts and fuel pumps.

Unit effectivity

This type of effectivity recognizes a range of unit numbers. A unit is a sequence designator for supply or demand on a part. Unit numbers often correspond to serial numbers. RapidResponse **BomType** and **OperationType** control tables specify how units on orders are interpreted.

The bill record is effective if the StartUnit of the supply record is within the range of the StartUnit and EndUnit on the bill record.

During explosion, some orders will have a portion of their units that fall within the unit range and a portion that do not. In these situations, one of the following can occur:

- Dependent demands may be split based on the units within the range.
- Dependent demand from the entire order can be based on the starting unit number.

Example of unit effectivity

Airframe manufacturers often attach options to an aircraft based on a serial number of the aircraft (for example, the tail number). Unit effectivity uses the unit number of the supply that is being exploded to determine effectivity for bill of material and operation records.

Explosion based on effectivity

If your company has enabled Model-Unit Effectivity, the methods used to determine whether a bill is effective include model and unit logic as well as date effectivity. Unit testing is performed in explosion regardless of the value of **Part.MUEPoolNettingType** field.

Effectivity depends on the following fields in the **BOMType** table.

Table 33.1: Fields in the BOMType table

Value	Explanation
EffectivityRule	Defines how ranges of units and dates are to be interpreted when determining effectivity. This field applies to all effectivity rules (ranges apply to dates and unit ranges) and can make a record always effective or always ineffective.
DateRule	This determines how date effectivity is to be applied. The DateRule is always applied, regardless of which analytics have been enabled.
MUERule	This determines how model and unit effectivity are applied. If your company has not enabled Model-Unit Effectivity, this field is treated as Ignore.
DependentUnitRule	If unit effectivity is being used, this field determines how dependent demands are handled.

Testing for effectivity

The test for effectivity involves a series of steps. The first step is to test the **BOMType.EffectivityRule** control value for the bill record. This value is one of the values specified in the following table.

Table 33.2: Effectivity test for BillOfMaterial records

Value	Description
Never	Explosion completely ignores the bill record.
Always	Explosion always uses the bill, regardless of other effectivities on the record.
InclusiveExclusive	Explosion includes start dates and units in the effective range, but not end dates and units.
Exclusive	Explosion excludes start dates, start units, end dates and end units from the effective range.
Inclusive	Explosion includes start dates, start units, end dates and end units in the effective range.

DateRule test

The next step is to test the **BOMType.DateRule** control value, which is one of the following.

Table 33.3: DateRule test

Value	Description
AnchorDate	Explosion uses the bill record if the supply's Anchor date falls within the bill's EffectiveInDate and EffectiveOutDate range.
BlowThroughCalcDueDate	Explosion uses the bill record if the real assembly supply's ReschedDate (for ScheduledReceipt records) or the DueDate (for PlannedOrder records) falls within the bill's EffectiveInDate and EffectiveOutDate range. Unless the assembly is a phantom, this option is identical to CalcDueDate
CalcDockDate	Explosion uses the bill record if the supply's ReschedDockDate (for ScheduledReceipt records) or the DockDate (for PlannedOrder records) falls within the bill's EffectiveInDate and EffectiveOutDate range.
CalcDueDate	Explosion uses the bill record if the supply's ReschedDate (for ScheduledReceipt records) or the DueDate (for PlannedOrder records) falls within the bill's EffectiveInDate and EffectiveOutDate range.
CalcStartDate	Explosion uses the bill record if the supply's CalcStartDate (for ScheduledReceipt records) or the StartDate (for PlannedOrder records) falls within the bill's EffectiveInDate and EffectiveOutDate range.
Ignore	Explosion does not test date effectivity for this record.

MUERule test

The next step is to test the **BOMType.MUERule** control value. If your company has not enabled Model-Unit Effectivity, explosion always ignores this control value. The rule will have one of the following values.

Table 33.4: MUERule test

Value	Description
Model	Explosion uses the bill record if the supply's model matches the bill record's model (for example, BillOfMaterial.Model). Unit on the **BillOfMaterial** table is ignored.
Unit	Explosion uses the bill record according to the values of the following fields: • Supply.StartUnit • BOMType.DependentUnitRule • BillOfMaterial.StartUnit and BillOfMaterial.EndUnit (Model is ignored) Explosion looks at the BOMType.DependentUnitRule: • If BOMType.DependentUnitRule = Split, explosion uses the bill record if any unit between Supply.StartUnit and (Supply.StartUnit + Quantity) is between the bill record's StartUnit and EndUnit (using BOMType. EffectivityRule for inclusive/exclusive checking). Explosion adjusts the StartUnit and Quantity of the resulting allocation to match the units within the bill record's StartUnit and EndUnit range. • If BOMType.DependentUnitRule = First, explosion uses the bill record if the Supply.StartUnit falls within the range of bill record's StartUnit and EndUnit (using BOMType EffectivityRule for inclusive/exclusive checking). The resulting allocation record will cover the entire supply. **Note:** If EffectiveEndUnit is < 0, every supply is treated as being before the end unit test and therefore, every supply passes the end unit test. Similarly, if EffectiveStartUnit < 0, every supply is treated as being after the start unit test and therefore, every supply passes the start unit test.
ModelUnit	Explosion uses the bill record if both the supply's model matches and the StartUnit on the supply record is within the StartUnit and EndUnit range from the bill record.
Ignore	Explosion does no testing of model and unit effectivity for this bill record.

Dependent demand

If the bill record is effective, explosion then populates dependent demand as follows:

- The value in **Supply.Model** (for example, **ScheduledReceipt.Model**) is copied to Demand.Model
- The StartUnit is the first unit representing the dependent demand (depending upon the DependentUnitRule). If either Unit or Model-Unit processing is occurring, and the DependentUnitRule is **Split**, the allocation record reflects just that portion of the supply for which the Unit number falls within the StartUnit/EndUnit range.

Exploding scheduled receipts and planned orders

If either Unit or Model-Unit effectivity is used when exploding scheduled receipts (work orders) and planned orders, the StartUnit of the resulting DependentDemand is based on the **BOMType.DependentUnitRule** field.

A DependentUnitRule of **Split** applies only to exploding scheduled receipts and planned orders.

Exploded Forecast and AvailableToPromise are all treated as First, regardless of the value of the **BOMType.DependentUnitRule** control value.

Cumulative lead time

Cumulative lead time, as shown on the Bill Browser tree, will reflect the maximum cumulative lead time for the model and unit selected in their respective controls. This is different for different models and units due to effectivity in the bill of materials.

If your company has enabled Model-Unit Effectivity, two calculated tables are populated:

- **Part_MUECumLead** table
- **BOM_MUECumLead** table

Part_MUECumLead table

The **Part_MUECumLead** table supports the Model-Effectivity and Unit-Effectivity analytic. This table accomplishes the following:

- Represents a set referencing Part.
- Lists the maximum cumulative lead times for all models and unit ranges through this part's bill of materials.
- Contains at least one record for each part. These records are: Model= None and Unit = -1.

Note: For more information about the **Part_MUECumLead** table, see "Part_MUECumLead table" on page 1139.

BOM_MUECumLead table

The **BOM_MUECumLead** table supports the Model Unit Effectivity analytic. If the BOM record is in effect (using date effectivity), this table is populated with the consolidated records of the **Component.Part_MUECumLead** table. Otherwise, no records are created. Records are consolidated by unique combinations of UseModel (when UseModel = 'False', then Model will = 'None'), Model, StartUnit and EndUnit. This table accomplishes the following:

- Represents a set referencing each BillOfMaterial record.
- Lists the cumulative lead times for all models and unit ranges through this bill record.
- It is calculated by summarizing the component part's Part_MUECumLead for model and unit combinations that will be netted individually.

Capacity load calculations

If your company has enabled Model-Unit Effectivity, Operation effectivity is expanded to include model, unit and anchor date effectivity, in addition to standard date effectivity. Unit testing is performed in explosion regardless of the value of Part.MUEPoolNettingType.MUERule.

Like bill effectivity, operation effectivity depends upon the following OperationType fields:

- EffectivityRule
- DateRule
- MUERule
- DependentUnitRule

EffectivityRule test

The first step is to test the EffectivityRule for the operation. The rule will be one of the following.

Table 33.5: EffectivityRule test

Value	Description
Never	Explosion logic completely ignores the operation.
Always	Explosion always uses this operation, regardless of other effectivities on the record.
InclusiveExclusive	Explosion includes start dates and units in the effective range, but not end dates and units.
Exclusive	Explosion excludes start dates, start units, end dates and end units from the effective range.
Inclusive	Explosion includes start dates, start units, end dates and end units in the effective range.

DateRule test

The next step is to test the DateRule. The rule will be one of the following.

Table 33.6: DateRule test

Value	Description
CalcStartDate	Explosion logic uses the operation if the supply's CalcStartDate (for ScheduledReceipt records) or the StartDate (for PlannedOrder records) falls within the operation's EffectiveInDate and EffectiveOutDate range.
AnchorDate	Explosion logic uses the operation if the supply's Anchor date falls within the operation's EffectiveInDate and EffectiveOutDate range.
Ignore	Explosion does not test date effectivity for this operation.

MUERule test

Now the MUERule is tested. If Model-Unit Effectivity is not enabled, explosion always ignores the MUERule. The rule will be one of the following:

Table 33.7: MUERule test

Value	Description
Model	Explosion uses the operation if the supply's model matches the operation's model (for example, Operation.Model). StartUnit on the **Operation** table is ignored.
Unit	Explosion uses the operation according to the values of the following fields: • OperationType.DependentUnitRule • Operation.StartUnit • Operation.EndUnit (Model is ignored) Explosion looks at the DependentUnitRule: • If DependentUnitRule = Split, explosion uses the operation if any unit between Supply.StartUnit and (Supply.StartUnit + Quantity) is between the operation's StartUnit and EndUnit (using OperationType.EffectivityRule for inclusive/exclusive checking). Explosion adjusts the StartUnit and RunTime of the resulting load record to match the units within the operation's StartUnit and EndUnit range. If there is a split, all loads receive setup for the corresponding operation. • If DependentUnitRule = First, explosion uses the operation if the StartUnit falls within the range of the Operation StartUnit and EndUnit (using theOperationType.EffectivityRule for inclusive/exclusive checking). The resulting load record will cover the entire supply.
ModelUnit	Explosion uses the operation if the supply's model matches the operation's model (for example, Operation.Model). Explosion then looks at StartUnit and performs the same testing as described above.
Ignore	Explosion does no testing of model and unit effectivity for this operation.

CHAPTER 34: Inventory Pooling calculations

RapidResponse netting calculations align supply plans for a given part with its demand. If scheduled receipts are insufficient, RapidResponse recommends new planned orders.

When pool, model, or unit are relevant, netting behaves a little differently. It matches supply and demand for a part, and plans orders for each unique combination (for example, Inventory Pooling) of the part. Netting ensures that a scheduled receipt for a specific model, unit, and pool is allocated to a demand for the same model, unit and pool. For parts where Model, Unit, and Inventory Pooling are not applicable, RapidResponse handles all supplies for the part as common inventory, that can be allocated to any demand for the part.

Model, unit, and pool also determine which parts can be used as substitutes for other parts in an interchangeable group. Only parts that have the same model, unit, and pool can be used interchangeably.

This chapter discusses optional Model-Unit Effectivity capability and the optional Pool capability.

In this chapter, you'll learn more about:

Understanding pools **1745**
Pool netting **1745**
Pool, model and unit netting factors **1746**
Pool, model and unit netting **1748**
Pools and part sources **1750**

Understanding pools

A pool is a set of supplies and demands that are segregated from other supplies and demands for the same part. Some manufacturers use pools to manage supplies for one customer or project separately from supplies for other customers or projects.

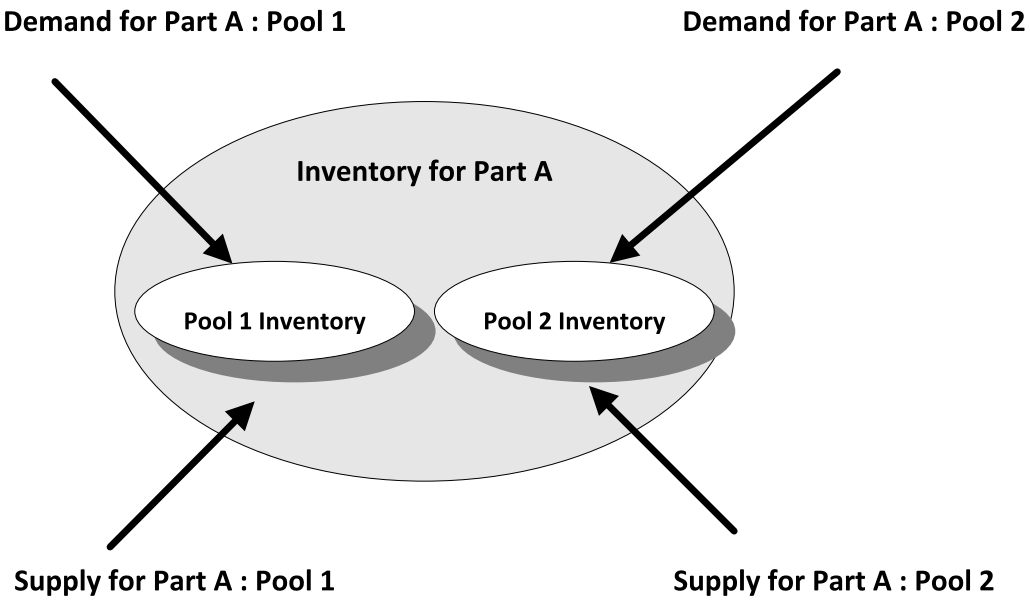
Pools allow manufacturers to isolate supply and demand for a part on a customer or project basis. Compare this to multi-site, which allows for different sites (which could represent customers or projects) but which also uses a separate bill of material product structure.

Pool netting

RapidResponse supports distinct netting for different models, pools and units including new planned orders on the same day. Netting treats each unique, active combination of Model, StartUnit and Pool for a

given part separately. Demand is bucketed by day, after being grouped by part, site, pool, model and unit.

Figure 34.1 Pool netting



Inventory for Part A is segregated into Pool 1 and Pool 2. So even though supply and demand are all for Part A, they are planned separately because of the inventory pools.

The analytics for unit support distinct sub-assemblies for different units (unit netting). When netting by unit, the unit analytic can generate unit numbers for planned orders. When netting by unit, supplies will have the same unit number as demands.

Pool, model and unit netting factors

Pool, model and unit netting requires either Pool or Model-Unit Effectivity to be enabled. They are controlled by the following factors:

- Part records
- MUEPoolNettingType records
- PoolMap records
- PoolMapOverride table

Part records

The **Part** table includes a reference field to the **MUEPoolNettingType** control table, which controls the model, pool and unit netting for the part. The **Part** table also includes a **NextUnit** field. In conjunction with the **MUEPoolNettingType.UnitRule** field, **NextUnit** determines how the StartUnit is propagated

from the part, as well as the unit number to use as the StartUnit for planned orders and forecast to be exploded from the part.

MUEPoolNettingType records

The **MUEPoolNettingType** control table allows finer control over processing rules for model, unit, and pool.

Processing Model, Unit, and Pool data

The **MUEPoolNettingType** table contains the following fields that determine whether model, unit, and pool data is processed by RapidResponse analytics:

- **ModelRule**—specifies whether a part's netting is performed by model, and also allows for demand to be exploded by model, without netting by the model.
- **PoolRule**—specifies whether a part's netting is performed by pool, and whether forecast consumption is done by pool.
- **UnitRule**—specifies whether a part's netting is performed by unit, and also allows for demand to be exploded by unit without netting by unit

Flow to common

The **MUEPoolNettingType** table contains the **PoolFlowToCommon** field.

The setting in the **PoolFlowToCommon** field is used by RapidResponse forecast logic. It specifies whether actual orders from a given customer should consume only forecast from that customer's specified forecast pool or, if all forecast for the customer has been consumed, allow any remaining orders from the customer to consume from the common forecast pool.

The **PoolFlowToCommon** field setting is also used by RapidResponse netting logic when satisfying demands. It specifies whether, after having used up all supply from a customer's specified pool as well as that in the common pool, any new supply that is required to satisfy the customer's demand should be planned in the customer-specific pool or in the common pool.

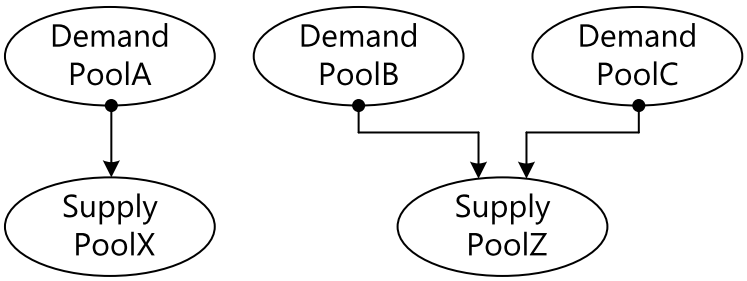
Notes:

- For more information about the MUEPoolNettingType table and its options, see "MUEPoolNettingType table" on page 749.
- The common pool associated with a given part is determined by its **PlanningCalendars.CommonPool** reference.
- The Part table includes many calculated fields that report summary information for each part. When pool-based netting flow-to-common logic is enabled, those fields that report a sum (for example, **TotalDemand**) are summed across all pools whereas those fields that report the first of something (for example, **FirstShortageDate**) will report the first value from just one of those pools.

PoolMap records

You can import values into the **PoolMap** table to determine which supply pool will satisfy a demand for a part belonging to a specific demand pool.

Figure 34.2 The relationship between Supply and Demand pools



In the above diagram, when there is demand for a given part belonging to **DemandPoolA**, only a part from **SupplyPoolX** can satisfy the demand. If there are no parts in **SupplyPoolX**, then RapidResponse generates a new planned order to cover the demand. If there is demand for a part belonging to either **DemandPoolB** or **DemandPoolC**, a part from **SupplyPoolZ** can satisfy the demand. However, a part from **SupplyPoolX** cannot satisfy demand for a part belonging to **DemandPoolB** or **DemandPoolC**. For information about the **PoolMap** table, see "PoolMap table" on page 487.

PoolMapOverride records

You can import values into the **PoolMapOverride** table to determine which supply pool will satisfy a demand for a part belonging to a specific demand pool. The values in this table override the values in the **PoolMap** table. With respect to the figure illustrating "PoolMap records" on page 1748, the same principles apply; however, the difference being a Part is considered, not a PartType. Therefore when there is a demand for a Part belonging to DemandPoolA, only a Part from SupplyPoolX can satisfy the demand. For more information, see "PoolMapOverride table" on page 487.

Pool, model and unit netting

Netting considers pool data if the value in the **MUEPoolNettingType.PoolRule** field is **Net**, and Pool has been enabled. If either of these conditions do not exist, Pool fields always store the first record that is loaded into the **Pool** table (usually **Unpooled**) and is ignored.

Netting includes different model values in the calculations if the value in **MUEPoolNettingType.ModelRule** field is **Net** or **NetWithoutForecast**, and your company has enabled **Model-Unit Effectivity**. If either of these conditions do not exist, the **Model** field in the data always comes from the first record that is loaded into the **Model** table. The first record in the Model table is the default value and is typically **None**.

Unit values are considered in the calculations if the value in **MUEPoolNettingType.UnitRule** field is **Net**, and your company has enabled Model-Unit Effectivity. If either of these conditions do not exist, all units are netted together. In this case, the value for the **StartUnit** field is -1.

Note: Model values are provided as input on scheduled receipts and independent demands, however an additional control setting, **ModelUsage**, is available on both the **DemandStatus** and **SupplyStatus** tables to indicate whether the provided input value or the default Model should be used in Netting.

Planned orders

Netting populates the Model, StartUnit and Pool fields on planned orders with the Model, StartUnit and Pool being netted. If pool netting is not in effect, the value for Pool is **Unpooled**. If model netting is not in effect, the value for Model will be **None**. If unit netting is not in effect, the value for StartUnit is –1. When the MUEPoolNettingType.UnitRule is not **Net**, or if Model-Unit Effectivity is not enabled, StartUnit is generated from Part.NextUnit.

Safety stock and pools

When part is netted by pool (for example, references a value of “Net” in any of the **ModelRule**, **PoolRule**, or **UnitRule** fields on the **MUEPoolNettingType** table), safety stock planning for each part is dependent on the referenced setting in the **MUEPoolNettingType.SafetyStockRule** field. This allows safety stock for a part to be planned on a pool by pool basis, in the common pool only, or ignored altogether. The **SafetyStockRule** field contains the following three options:

- **ByPool**—Separate safety stock quantities are maintained for each pool applicable to the part (always using the default model and unit). For example, if a part has a **SafetyStockPolicy.SafetyStockQuantityRule** setting of “PercentOfDemand”, then separate safety stock quantities are calculated for each pool the part is netted by. Similarly, if a part has a **SafetyStockPolicy.SafetyStockQuantityRule** setting of “TimePhasedSafety”, then separate safety stock quantities can be inputted for different part and pool combinations. This option is intended for parts that are netted by pool and require safety stock to be maintained in different pools (if used with a **PoolRule** setting other than “Net”, safety stock is planned in the common pool). Note that if using inventory pooling along with fixed safety stock requirements (where **SafetyStockQuantityRule** is set to "Fixed"), then the expected safety stock amounts might not be planned. If pools are specified on part sources, inventory, or demands, and a fixed safety stock quantity is required, it is recommended to use a **SafetyStockQuantityRule** of "TimePhasedQuantity" instead (the **TImePhasedSafety** table allows specifying safety stock levels by part and pool).
- **Common**—Safety stock is maintained for the part using the common pool along with the default model and unit. This option is only applicable to parts that have a **SafetyStockPolicy.SafetyStockQuantityRule** setting of “Fixed”, “TimePhasedQuantity”, or “TimePhasedQuantityLatest”. If used with dynamic safety stock rules (such as, “PercentOfDemand” or “RangeOfCoverage”), safety stock is not planned. This option is intended for parts that are netted by model, unit, and/or pool but require only a single pool of fixed/time-phased safety stock to be maintained. Note also that **TimePhasedSafety** records that reference pools other than the common pool are included.

- **Ignore**—Safety stock is never maintained for the part. This option is intended both for parts that not netted by model, unit or pool, as well as for parts that are actually netted by model, unit, or pool but for which safety stock is not required. This is the default setting and ensures calculations are performed consistent with those in versions prior to 2013.4.

Note: For information about the settings that control how a part's safety stock levels are determined, see "Shortage calculations" on page 1375.

Generating the StartUnit number

When **MUEPoolNettingType.Unit** is **Net** and Model-Unit Effectivity is enabled, StartUnit numbers for supply and demand will match. The StartUnit does not need to be generated. The StartUnit on planned orders and forecast will be the unit from the demand that is being satisfied. Similarly, when considering scheduled receipts, the input data already contains a relevant StartUnit number.

Therefore, there are only two situations when StartUnit needs to be generated:

- for forecast
- for planned orders not using Unit netting

In these situations, unit logic depends upon the value in the following fields on the **Part** table: MUEPoolNettingType.UnitRule and NextUnit. Their interaction is described in the table below. For forecast explosion, the StartUnit from the forecast record is passed through parts with unit netting. For parts not using unit netting, StartUnit is generated using UnitRule and NextUnit.

Table 34.1: StartUnit values

UnitRule	NextUnit	Interpretation
Ignore	N/A	StartUnit on planned orders and forecast is set to –1.
Net	N/A	Netting is performed by Unit. Planned orders and forecast simply acquire the StartUnit that is being netted.
Fixed	Used	The value of NextUnit is copied into StartUnit for planned order and forecast.
Increment	<0	Unit number generation for this part is disabled. StartUnit on all planned orders and forecast is set to –1.
Increment	>=0	Unit number generation for this part is enabled. StartUnit is calculated from Part.NextUnit. All forecast and planned orders for the part are sorted separated by pool, model and due date. The first forecast data to explodes is assigned StartUnit = Part.NextUnit. The next forecast record is assigned Part.NextUnit + (Quantity on first forecast) etc. StartUnit numbers are assigned to all forecast for the part. Next, all planned orders for the part are processed using the same calculation, with startUnit for the first PlannedOrder being set to Part.NextUnit + (sum of all forecast quantities for the part).

Pools and part sources

Pools can be specified on **PartSource** records in the (nullable) **Pool** reference field.

When inventory pooling is enabled for a part (**MUEPoolNettingType.PoolRule** is set to "Pool" or "Optional"), the **PartSource.Pool** value can be used in the part source selection process. The **Pool** reference can also be used to inject pool data at the dependent demand level in cases where an assembly part does not use pooling logic but its lower-level components should.

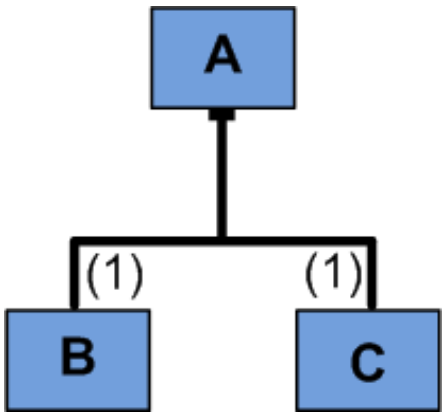
Pools and part source selection

When inventory pooling is enabled and the optional **Pool** reference is left Null on all **PartSource** records for a part, then standard part source selection logic (as described in "Part source selection process" on page 1378) determines which part sources can potentially be used for any new planned orders needed to satisfy a given demand. Planned orders generated from the selected part source(s) will have a **Pool** value that matches the demand being netted or the common pool (depending on the part's **MUEPoolNettingType.PoolRule** setting). **Pool** values are then propagated down the product structure to dependent demands and supplies on lower level components that have inventory pooling enabled.

For example, assume a simple BOM structure with relevant PartSource records as shown in the following illustration. Further assume each part has **SourceRule.AllotmentRule** set to "Proportional" and a **PlanningCalendars.CommonPool** reference pointing to the "Unpooled" pool.

Table 34.2: PartSource records for items in product structure

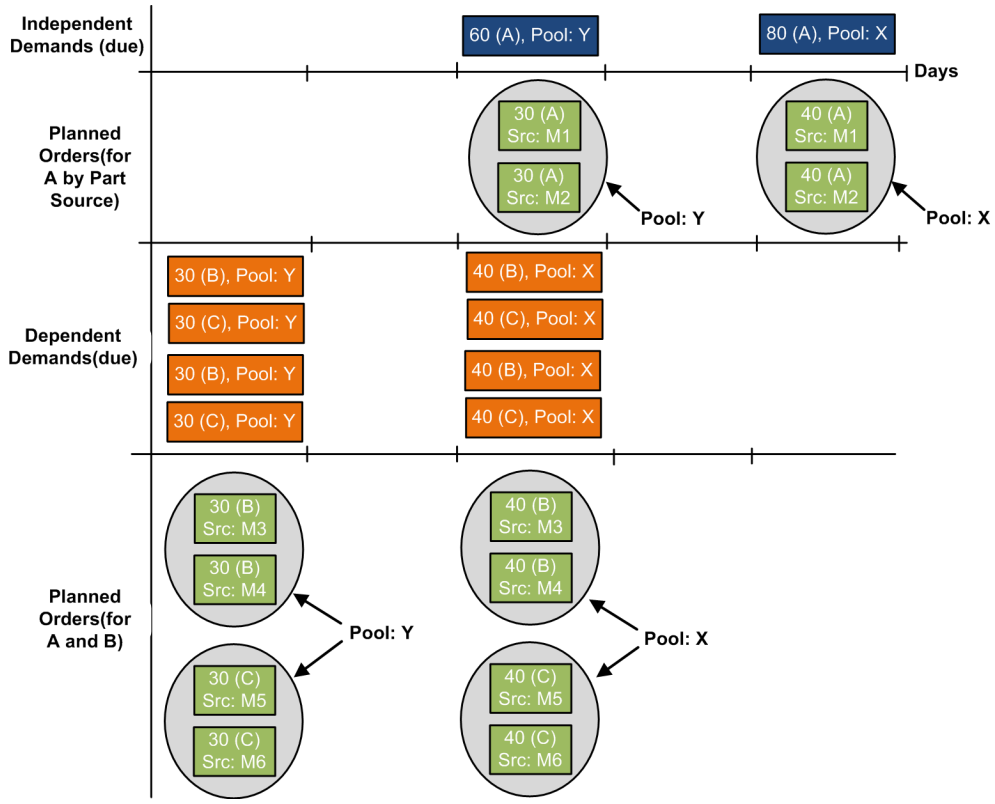
Part	Source	Pool	Priority	Target	EffLead Time
A	M1	Null	0	0	2
A	M2	Null	0	0	2
B	M3	Null	0	0	1
B	M4	Null	0	0	1
C	M5	Null	0	0	1
C	M6	Null	0	0	1



Given independent demands for end-item A and no available input supplies, dependent demands and planned supplies would be generated as shown in the following illustration (this assumes a **PoolRule** setting of "Use" and **PoolFlowToCommon** setting of "Ignore" for all parts). In this case, note that supply for each demand is sourced from all of the part's part sources.

Figure 34.3 Sourcing with Null Pool reference on PartSource

.

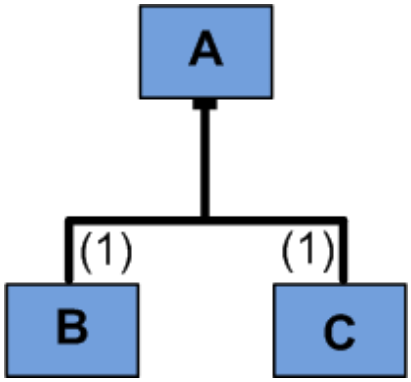


However, when inventory pooling is enabled for a part and the optional **PartSource.Pool** reference is populated on some or all part sources, then the referenced **Pool** value(s) influence part source selection. As long as the part's **MUEPoolNettingType.PoolFlowToCommon** setting is other than "NetCommon" (that is, one of "Ignore", "NetCommon", or "Forecast") then new planned orders can only be provided from part sources where the referenced **Pool** is either a match for the demand pool or is set to "Null". Any part sources that explicitly reference a Pool that isn't a match for the demand pool cannot be used in this case. Note also that if there are records where **PartSource.Pool** is set to "Null" these are considered equally as good as those that explicitly reference a matching pool and the part source(s) selected will depend on other criteria.

For example, consider the same **PartSource** records as shown in the previous example, but with the **Pool** values populated as shown below.

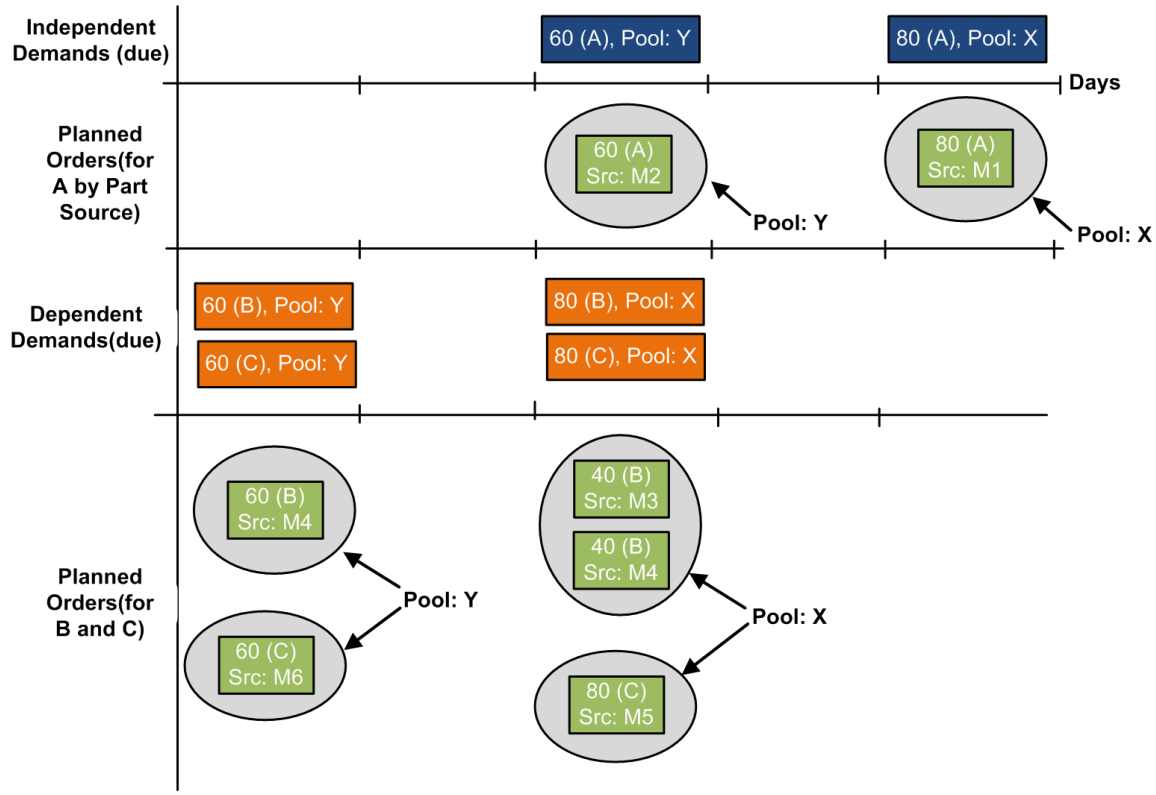
Table 34.3: PartSource records for parts in product structure

Part	Source	Pool	Priority	Target	EffLead Time
A	M1	X	0	0	2
A	M2	Y	0	0	2
B	M3	X	0	0	1
B	M4	Null	0	0	1
C	M5	X	0	0	1
C	M6	Y	0	0	1



Given the same independent demands for end-item A as shown previously with no available input supplies, dependent demands and planned supplies would now be generated as shown in the following illustration (this assumes a **PoolRule** setting of "Use" and **PoolFlowToCommon** setting of "Ignore" for all parts). In this case, note that supply for each demand is sourced from only those part sources where the **Pool** value is a match for the demand or is Null. For example, to satisfy demand for part B in pool X, both part sources are used (one referencing Pool X and one with a Null reference). However, the demand for part C in pool X can only be satisfied by supply from the part source referencing Pool X (the other part source references Pool Y and is therefore ineligible for this demand).

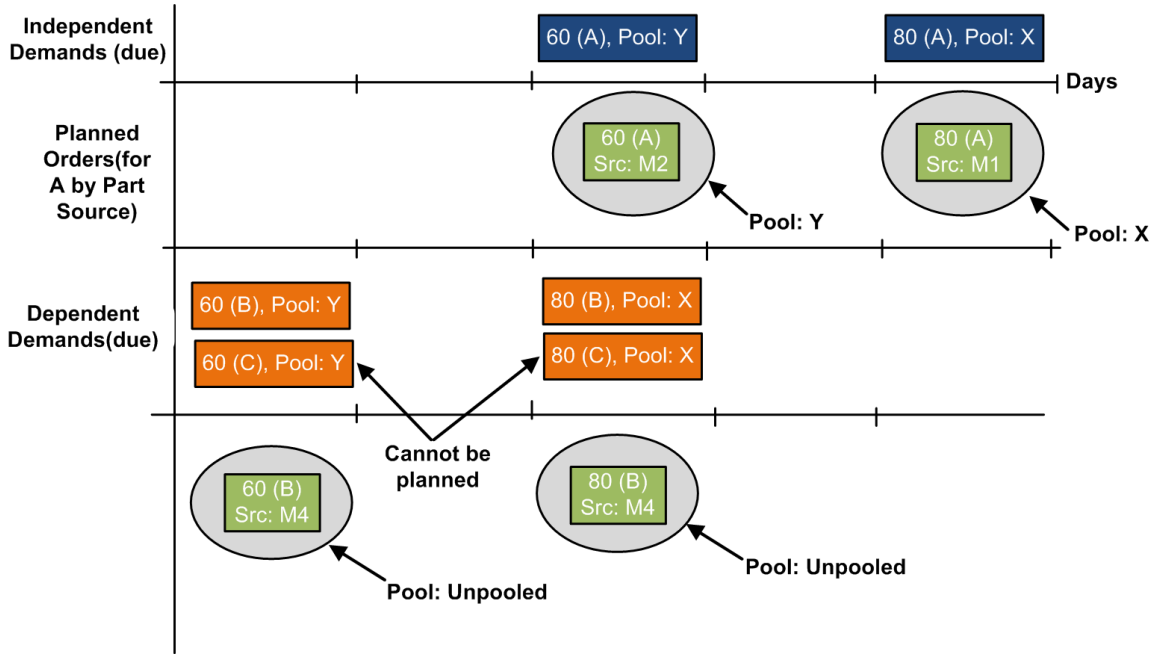
Figure 34.4 Sourcing impacted by PartSource.Pool reference values



Note that when **PartSource.Pool** values are provided for use in part source selection, caution should be taken with use of the **MUEPoolNettingType.PoolFlowToCommon** setting of "NetCommon". Parts with this setting are only be able to plan supply in the common pool. This implies that only part sources where the Pool reference either points to the common pool or is Null will be able to plan supply for the part. In other words, if a part source's **Pool** reference is a match for a specific (non-common) demand pool, then that part source is ineligible to satisfy the demand.

For example, assume the same parts and **PartSource** records as shown in the previous example but with lower-level components B and C both using a **PoolFlowToCommon** setting of "NetCommon". In this case, supplies would be sourced as shown below. Note that all supply of B is now provided from source M4 as it is the only one with a Null **Pool** reference. Additionally, no supply of C can be planned as it does not have a **PartSource** with a Null or common pool reference (this implies the end-item demands for part A cannot ultimately be satisfied.

Figure 34.5 Impact of NetCommon setting on sourcing when using PartSource.Pool



Note: The **PartSource.Pool** field was added in Version 2016.2, Service Update 19.

Inject pools at the dependent demand level

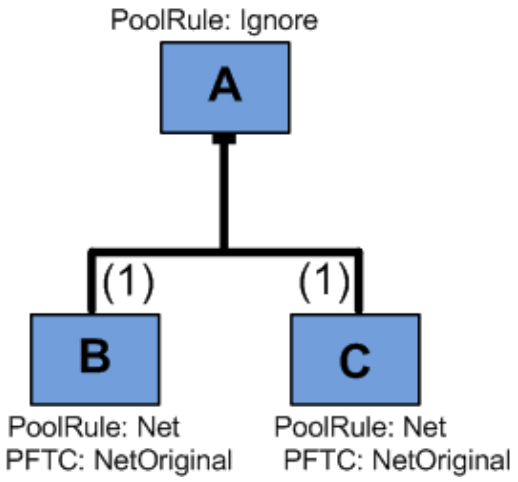
Typically when inventory pooling is enabled for a part, the pool applicable to a demand is matched by its supplies and subsequently propagated down to dependent demands on lower-component components in the product structure. Similarly, when inventory pooling is disabled for a part, the part's supplies are then planned in the common pool which is propagated down to dependent demands on lower-level components in the product structure.

For example, assume a simple BOM structure with relevant PartSource records as shown in the following illustration where pooling is disabled on the end-item assembly but enabled on its components. Further

assume each part has its **SourceRule.AllotmentRule** set to "Proportional" and references a **PlanningCalendars.CommonPool** value of "Common".

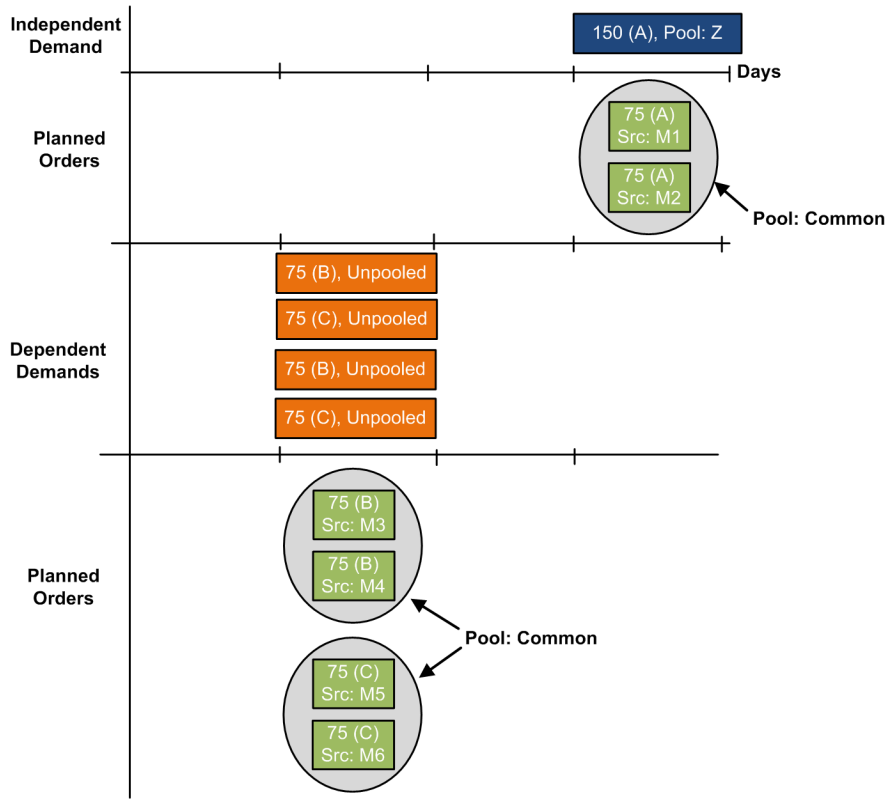
Table 34.4: PartSource records for parts in product structure

Part	Source	Pool	Priority	Target	EffLead Time
A	M1	Null	0	0	2
A	M2	Null	0	0	2
B	M3	Null	0	0	1
B	M4	Null	0	0	1
C	M5	Null	0	0	1
C	M6	Null	0	0	1



Given a single independent demand for end-item A with no available input supplies, dependent demand and planned supplies would be generated as shown in the following illustration. Note that all supplies in the product structure are planned in the "Common" pool (as determined from the **PoolRule** setting on end-item A).

Figure 34.6 Pool propagated throughout the product structure

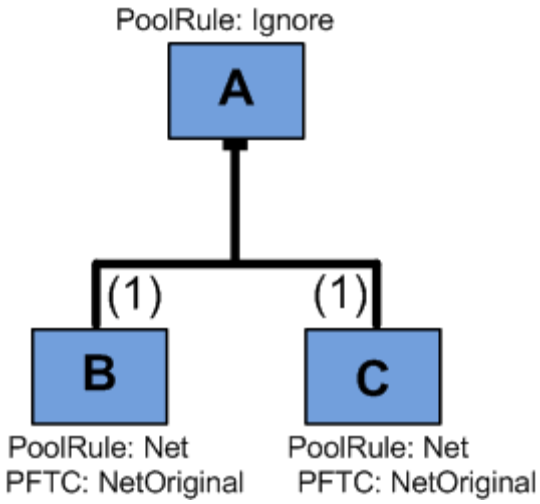


However, in cases where inventory pooling is disabled for a part but the **Pool** reference is populated on **PartSource** records for that part, then the referenced pools can be propagated down the product structure and ultimately used on dependent demands for lower-level components where pooling is enabled. That is, the assembly part would have its supplies planned in the "common" pool, but supplies for lower-level components that do have inventory pooling enabled would be planned in the pool that was defined on the **PartSource** record.

For example, assume the same **PartSource** records as shown in the previous example but with **Pool** values populated as shown below.

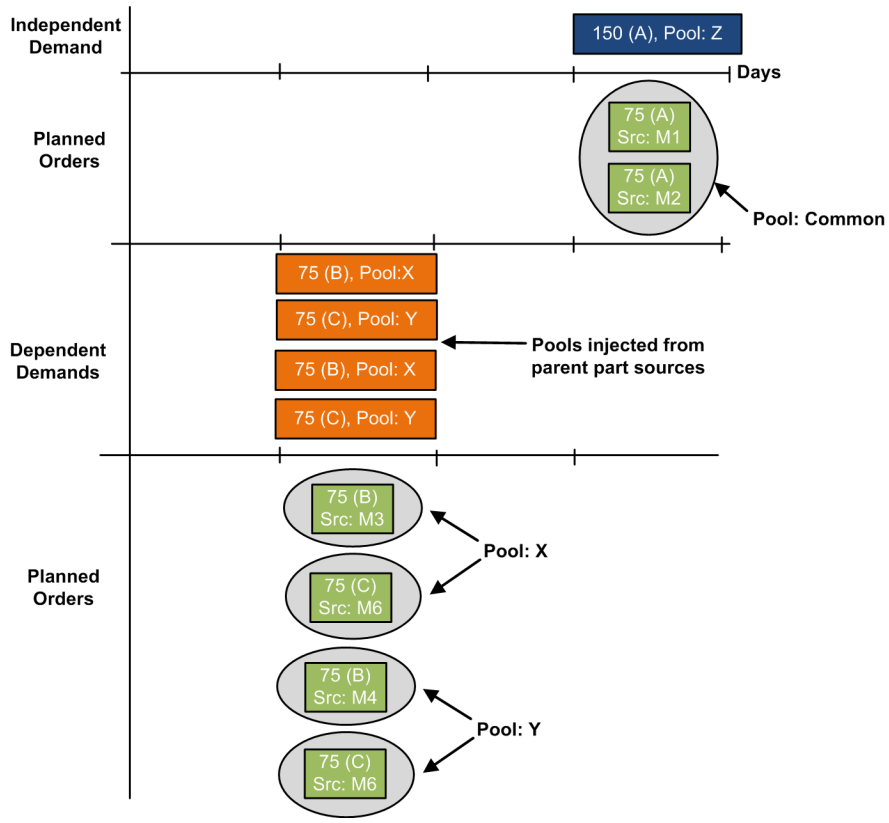
Table 34.5: PartSource records for parts in product structure

Part	Source	Pool	Priority	Target	EffLead Time
A	M1	X	0	0	2
A	M2	Y	0	0	2
B	M3	X	0	0	1
B	M4	Y	0	0	1
C	M5	Z	0	0	1
C	M6	Null	0	0	1



In this case, new supplies for item A are still planned in the common pool but the Pool values referenced by its part sources are injected at the next level and used to determine the specific pools in which the component supplies should be planned. Also, because Pool values are assumed to now be specified on the component's part sources, this further determines which of those sources can be used to provide supplies for B and C as shown in the following illustration.

Figure 34.7 Pools injected at dependent demand/component level



Note: Pools can also be injected to dependent demands under scheduled receipts (that is, in cases where the assembly demand is satisfied by a non-allocated scheduled receipt rather than a planned order). For such cases, the **ScheduledReceipt.InputPool** field should typically be populated as this reference determines the pool to inject at the component level (as long as it is not Null). If **ScheduledReceipt.InputPool** is Null, then the common pool is propagated down the the structure instead for part's that do not use inventory pooling. If using a process to convert generated planned orders to scheduled receipts, ensure to set the **ScheduledReceipt.InputPool** to reference the same **PlannedOrder.PartSource.Pool** value found in the original calculated record.

CHAPTER 35: Alternate BOM calculations

If the Alternate BOM logic in RapidResponse is used, a single assembly can have more than one bill of material. This allows each to be modeled separately. For example, the same assembly might be sourced from multiple suppliers each of which uses a different BOM. Alternatively, two lines of a manufacturing plant might produce the same assembly differing only in the material used to package it.

In this chapter, you'll learn about:

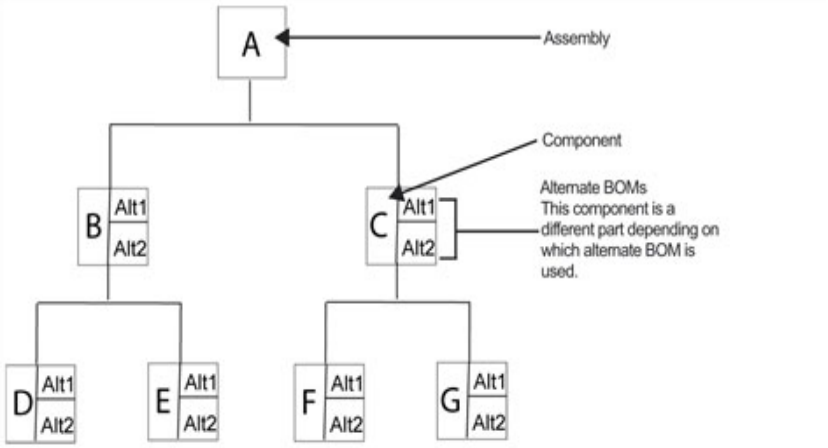
Alternate BOM basics **1759**
Effects on Netting calculations **1761**
Best practices for using alternate BOMs **1762**
Default BOMAlternate record **1764**

Alternate BOM basics

Alternate BOM data is contained in the **BOMAlternate** table. Values in this table can be automatically loaded during a data import or update based on records in the **BillOfMaterial**, **PartSource**, and **ScheduledReceipt** tables. Additionally, one record in this table is the "default" record and contains a special or common alternate code to be used by components that belong to all alternate BOM configurations under a given assembly. Typically, this special **BOMAlternate** record will have an empty string in the **Value** field. For more information about verifying which record is the default, see "Default BOMAlternate record" on page 1764.

Using alternate BOMs, a single assembly can have multiple bills of material if necessary. For example, an assembly might have two different part sources, each of which has a bill of material made up of slightly different but functionally equivalent components. In this case, each entry in the **BillOfMaterial.Alternate** field for a given assembly's component references the alternate BOM to which it belongs (for example, Alt1 and Alt2). The following illustration shows such a case.

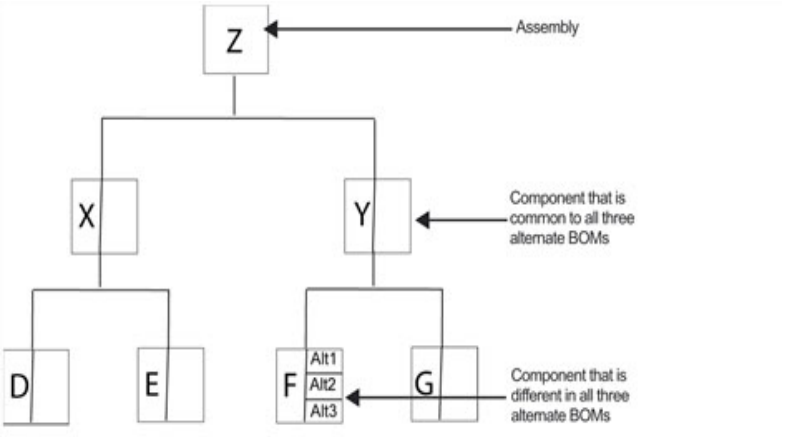
Figure 35.1 Bills of material with no common components



Alt1 and Alt2 are BOMs for Assembly A that require different components. Each component is a different part depending on which alternate BOM is used. For example, component C could be part DR-0201 in Alt1 and DR-0210 in Alt2. Parts required for Alt1 cannot be used in Alt2.

It might also be the case that alternate BOMs for an assembly differ only in select components (that is, they have many of the same components). In this case, each entry in the **BillOfMaterial.Alternate** field for an assembly's common components references the first entry in the BOMAlternate table (by default, this value is blank). Only those components which differ between bills of material should reference specific named alternate BOMs (for example, Alt1, Alt2, and Alt3). The following illustration shows such a case.

Figure 35.2 Bills of material with some common components



Alt1, Alt2, and Alt3 are BOMs for Assembly Z. All the components except one are the same in each of the three alternate BOMs. Because components X, Y, D, E, and G are common to the three BOMs, they have blank values in their **BillOfMaterial.Alternate** fields. Component F is a different part in the three BOMs, and the part required for Alt1 cannot be used in Alt2 or Alt3.

Note: For complete reference information about the **BOMAlternate** table, see "BOMAlternate table" on page 229.

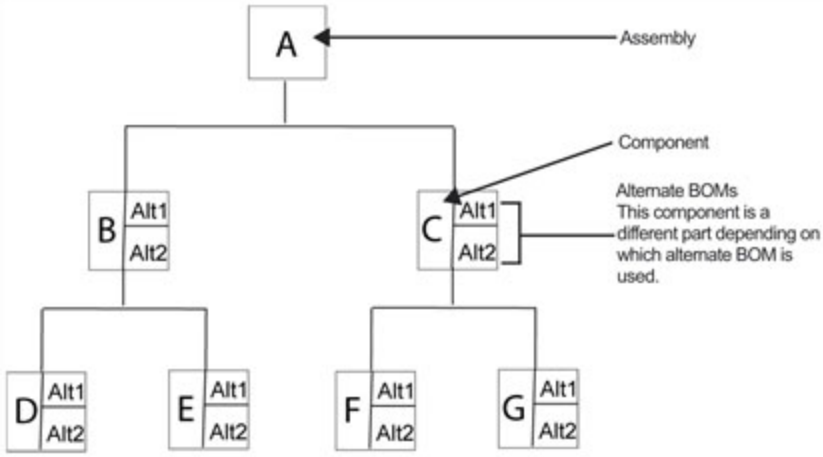
Effects on Netting calculations

When netting calculations generate planned orders, logic is used to select the best part source. This logic is unchanged by the presence of alternate BOMs. However, when a 'Make' part source is exploded to generate dependent demands for its components, these calculations are affected by the presence of alternate BOMs.

Each part source references an alternate BOM through the **PartSource.BOMAlternate** field to determine which bill of material to use when generating dependent demands. For a given assembly then, dependent demands are generated only for entries in the **BillOfMaterial** table that are either common components or where **BillOfMaterial.Alternate** matches the value in **PartSource.BOMAlternate**.

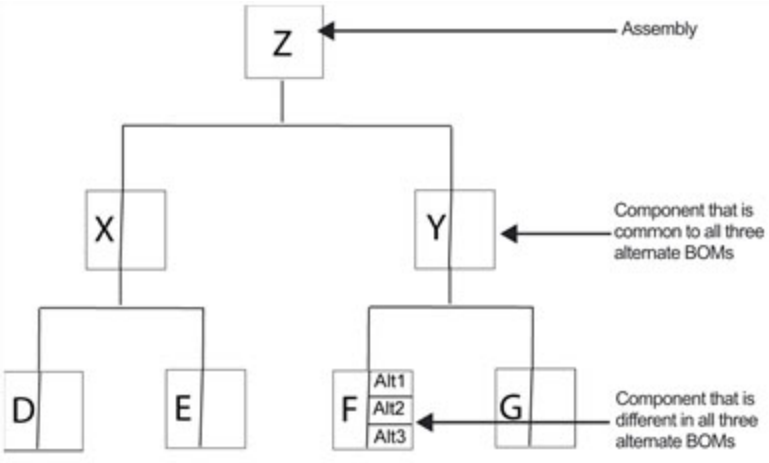
For example, assume a planned order for Assembly 'A' as depicted in "Bills of material with no common components" on page 1760, where the **PartSource.BOMAlternate** value is 'Alt1'.

Figure 35.3 Bills of material with no common components



This would generate dependent demands for only those components with a **BillOfMaterial.Alternate** value of 'Alt1'. Similarly, suppose a planned order for Assembly 'Z' as depicted in "Bills of material with some common components" on page 1760, where the **PartSource.BOMAlternate** value is 'Alt3'.

Figure 35.4 Bills of material with some common components



This would generate dependent demands for those components with a **BillOfMaterial.Alternate** value of either 'Alt3' or ' ' (common components).

When netting calculations explode scheduled receipts, similar alternate BOM logic is used as with planned orders. The main difference is that the value in **ScheduledReceipt.BOMAlternate** is compared to **BillOfMaterial.Alternate** when determining which components to generate dependent demands for. In cases where **ScheduledReceipt.BOMAlternate** differs from **PartSource.BOMAlternate**, the **ScheduledReceipt.BOMAlternate** value takes precedence and is used when generating dependent demands. The **ScheduledReceipt.BOMAlternate** value is only relevant in cases where **Order.Type.Source** = 'Make', and **Status.State** is set to either 'NotReleased', 'Planned', or 'ReleasedButNotAllocated'.

Best practices for using alternate BOMs

Depending on how, or if, your company is using alternate BOM logic, your input data needs to be set up differently. Alternate BOM values are automatically loaded from the **BillOfMaterial**, **PartSource**, and **ScheduledReceipt** table into the AlternateBOM table. This section covers three basic scenarios and how data should be loaded into the **AlternateBOM** table in each case.

- "Alternate BOMs are not used" on page 1762.
- "Alternate BOMs used with common components" on page 1762.
- "Alternate BOMs without any common components" on page 1763.

Alternate BOMs are not used

If your company is not using any alternate BOM logic, then no consideration needs to be given to alternate BOMs when setting up your input data. In this situation, only one "default" record is needed in the **BOMAlternate** table and it should typically have a value of ' ' (blank). By default, all references to this table (**BillOfMaterial.Alternate**, **PartSource.BOMAlternate**, and **ScheduledReceipt.BOMAlternate**) are also set to a value of ' ' (blank). As a result, all effective entries in the **BillOfMaterial** table apply to all 'make' part sources and to all 'make' scheduled receipts that are exploded to create new allocations.

Alternate BOMs used with common components

If your company is using alternate BOM logic to model alternate BOMs that use some common components, ensure that alternate BOM input data is provided for RapidResponse tables as outlined in the following table.

Table 35.1: Considerations for input data with alternate BOMs and common components

Table	Do the following
BOMAlternate	Ensure exactly one record is first imported directly into this table. This becomes the first value in the table, and is the one referred to by all common components. By default, it will have a value of ' ' (blank), though it can be changed if necessary. **Note:** If this blank value is not imported before alternate BOM information is added to other tables, the alternate BOMs might not function properly.
BillOfMaterial	For a given assembly, ensure the following: • All components that are common to all 'make' part sources and 'Make' scheduled receipts for the assembly should have their **BillOfMaterial.Alternate** value set to reference the default record in the **BOMAlternate** table. • All components that are not common across all 'Make' part sources and 'Make' scheduled receipts for the assembly should have one entry in the **BillOfMaterial** table for each alternate they belong to, and each entry should reference a distinct named alternate in the **BOMAlternate** table.
PartSource	For a given part source, ensure the following: • If the part source uses only common components, its **PartSource.BOMAlternate** value should be set to reference the default record in the **BOMAlternate** table. • If the part source uses both common components and components specific to an alternate BOM, its **PartSource.BOMAlternate** value should be set to the name of that alternate BOM.
ScheduledReceipt	For a given scheduled receipt, ensure the following: • If the scheduled receipt uses only common components, its **ScheduledReceipt.BOMAlternate** value should be set to refernece the default record in the **BOMAlternate** table. • If the scheduled receipt uses both common components and components specific to an alternate BOM, its **ScheduledReceipt.BOMAlternate** value should be set to the name of that alternate BOM.

Alternate BOMs without any common components

Table	Do the following
BOMAlternate	Ensure exactly one record is first imported directly into this table. It should have a value of ' ' (blank). Ensure that no records in the **BillOfMaterial**, **PartSource**, or **ScheduledReceipt** table reference this record. **Note:** If this blank value is not imported before alternate BOM information is added to other tables, the alternate BOMs might not function properly.
BillOfMaterial	Ensure that each bill of material record refers to a named alternate BOM (that is, a record other than the default record in the **BOMAlternate** table).
PartSource	Ensure that each part source record refers to a named alternate BOM (that is, a record other than the default record in the **BOMAlternate** table).
ScheduledReceipt	Ensure that each scheduled receipt record refers to a named alternate BOM (that is, a record other than the default record in the **BOMAlternate** table).

Default BOMAlternate record

It is recommended that the default record in the BOMAlternate table be set to have a <blank> **Value**. However, in order to double-check which record is set as the default, create a worksheet on the **BOMAlternate** table that shows the Value and Description fields; it is often useful to also include a count of BillOfMaterials, PartSources and ScheduledReceipts that use each BOMAlternate. Then add a new column with the column expression of `System::IsDefault`. The record with a check mark in the **IsDefault** column is the default **BOMAlternate** record as shown in the following illustration.

Figure 35.5 Record with a check mark in the IsDefault column

BOMAlternate

	Description	Value	Bill Of Materials	Part Sources	Scheduled Receipts	IsDefault
1			5,354	3,752	4,486	☑
2	Alter1	1	1,634	1,378	7,134	☐
3	Alter2	2	7	6	0	☐

CHAPTER 36: Order priority calculations

The priority analytic modifier prioritizes orders. This ensures that important orders are filled first when there is not enough supply to fill every order on a specific date. For example, a customer order can be assigned a higher priority than a forecast, or an order for a large customer can be given a higher priority than one for a smaller customer. High-priority orders are filled, and low-priority orders are delayed until they can be filled.

Order priorities apply only when there is insufficient supply to fill all orders on a certain date. If there is enough supply, the orders are filled according to their due dates.

In this chapter you will learn about:

Order priority tables and fields **1765**
Priority calculations **1767**
Prioritized demand examples **1795**
Prioritized supply examples **1799**

Notes:

- In cases where priority logic cannot easily achieve a specific required allotment of limited component supply, allotment overrides can be used to manually reserve a quantity of component supply to a given demand order. For more information about allotment overrides, see "Allotment overrides" on page 1853.
- Order priorities also support two-date planning logic which allows planning supplies to the original customer request date on independent demands instead of the standard planning to the committed due date. For more information, see "Two-date planning" on page 1803.

Order priority tables and fields

Priority is controlled through RapidResponse data model fields. These fields are spread across six tables.

Table 36.1: Order priority tables and fields

Table	Fields
OrderPriority	Value PlanningPriority AllocationRule MaintainCommitments PeriodEffective TwoDatePlanningRule
PartType	DefaultPriority SupplyAllocationRule CommitLevel UseDaysSupply
IndependentDemand	OrderPriority TransactionSequence
ScheduledReceipt	OrderPriority TransactionSequence
SupplyStatus	PrioritySource
DemandType	AllotmentOrder

Note: The **PartType.UseDaysSupply** field is not used in priority calculations, but its setting can override the **PartType.CommitLevel** value, which is used in priority calculations. The **PartType.UseDaysSupply** default value of **N** should not be changed when using order prioritization.

Order priorities are expressed as String values in the **OrderPriority** table. Each **Value** has an associated **PlanningPriority** integer value. Lower **PlanningPriority** values relate to higher priority. An example of **Values** to represent **PlanningPriority** values can be seen in the following table.

Table 36.2: OrderPriority values

Value	PlanningPriority
Very High	10
High	20
Urgent	25
Medium	40
Normal	50
Low	70
Safety Stock	90

The unique **OrderPriority** records for each **OrderPriority.PlanningPriority** value can be used for easily changing order priorities. An **OrderPriority** record can be created with an associated **MaintainCommitments** value for further priority customization.

For more information about the tables and fields used in order prioritization calculations, see "OrderPriority table" on page 782, "PartType table" on page 809, "IndependentDemand table" on page 358, "ScheduledReceipt table " on page 569, and "DemandType table" on page 715.

Priority calculations

Order priorities have an effect on both Capable-to-Promise (CTP) and netting order plans. Exactly how the orders in the system are affected is determined by performing priority calculations. The data obtained from these calculations is used by RapidResponse to determine the order in which demands are filled and how long low-priority orders have to wait.

RapidResponse performs the following priority calculations as discussed throughout this section:

- "Effective priority" on page 1767
- "Period effective priority" on page 1768
- "Effective transaction sequence" on page 1769
- "Updating TransactionSequence values" on page 1769
- "Priorities on dependent demands" on page 1770
- "Prioritized demand order" on page 1771
- "CTP supply allotments" on page 1771
- Priority and safety stock holds in CTP
- "Priorities on scheduled receipts" on page 1788
- "Priorities on planned orders" on page 1790
- "Constraint planning and usage" on page 1794

Effective priority

The effective priority of an order is the priority RapidResponse uses for order sequencing. The effective priority is determined by using the following fields:

- **PartType.CommitLevel**
- **PartType.DefaultPriority**
- **OrderPriority.PlanningPriority**

The effective priorities are calculated according to the following table.

Table 36.3: Effective priority

PartType.CommitLevel	Netting effective priority	CTP effective priority
High	OrderPriority. PlanningPriority	OrderPriority. PlanningPriority
MediumExplodePriority	OrderPriority. PlanningPriority	OrderPriority. PlanningPriority
Medium	PartType.DefaultPriority	OrderPriority. PlanningPriority
Low	PartType.DefaultPriority	PartType.DefaultPriority

Netting effective priority

If the **PartType.CommitLevel** is set to "Medium" or "Low", order priority has no effect during Netting. When new supply is planned, its priority is always set to the part's default priority. If the **PartType.CommitLevel** is set to "MediumExplodePriority" or "High", then order priority is used during

Netting and supplies take their priority from the demands they are created to cover. However, when using the "High" setting demands are processed in priority sequence, while when using with the "MediumExplodePriority" setting, demands are processed in due date sequence as discussed in "CommitLevel of High " on page 1791 and "CommitLevel of MediumExplodePriority" on page 1792.

CTP effective priority

When **PartType.CommitLevel** is set to "Low", order priority has no effect on CTP and the part's default priority is used as the effective priority. That is, in cases of limited supply, priority is ignored and demands are still filled in due date sequence. However, when **PartType.CommitLevel** is set to either "Medium", "MediumExplodePriority", or "High", then in cases of limited supply demands will be filled in priority sequence (higher priority demands are satisfied before lower priority demands).

Note: The results of the effective priority calculation are held internally by RapidResponse and not included in the data model.

Period effective priority

By default, effective priorities are assumed to be absolute in RapidResponse regardless of the dates on which their demands fall. This means that RapidResponse will try to ensure that higher priority orders are filled in time. In cases where there is not enough supply to satisfy all orders on time, then these high priority orders will get the earlier supply and lower priority orders will likely receive late supply.

As necessary, however, period effective priority can be enabled in RapidResponse. This involves configuring OrderPriority records so that the priorities on demands are only effective within a specified calendar interval (for example, weekly or monthly). When period effective priority is enabled, demands due within a specified time period will respect their relative effective priorities, however any unsatisfied demands from a previous period are treated as having a higher priority than demands due in the current period.

Period effective priority is set through the **OrderPriority.PeriodEffective** field. If this field is set to "Y", then starting within the current date interval,demands using a given priority value take their priority from the PlanningPriority field. However, any unsatisfied demands remaining from the current period will be assumed to have a higher priority than demands in subsequent periods. As a result, RapidResponse will attempt to satisfy those past due demands before demands due in the current period.

The date intervals used for determining period effective priority are set by a part's **PlanningCalendars.AllocationCalendar** value. For example, if this field is set to a monthly calendar then all demands due within a given month will respect their relative effective priorities. However, any demands which remain unsatisfied at the end of the month will be assumed to have a higher priority than demands in subsequent months (regardless of their PlanningPriority value).

If period effective priority is not required for a given priority value, then the **OrderPriority.PeriodEffective** value should be set to "N". As required, some or all order priority values can be configured as period effective. For more information and examples of period effective priority, see "Setting period effective priorities (example)" on page 1797.

Note: Part's with a **PartType.CommitLevel** of "High" or "MediumExplodePriority" have their priority value propagated to their dependent demands, however the date effective component of priority does not propagate to the dependent demands. As a result, when working with period effective priorities, some dependent component demands could have a higher effective priority than their driver demands.

Effective transaction sequence

The effective transaction sequence of an order is used for determining priority when two orders have the same **OrderPriority.PlanningPriority** value. The transaction sequence value is incremented when an order is created or modified. When using transaction sequences to determine priority, the order with the lowest transaction sequence is given highest priority.

For the duration of this section, **TransactionSequence** is used to represent both **IndependentDemand.TransactionSequence** and **ScheduledReceipt.-TransactionSequence**. The calculations are the same regardless of which is used.

The effective transaction sequence calculation uses the following values:

- **PartType.CommitLevel**
- **OrderPriority.MaintainCommitments**

The effective transaction sequences are calculated according to the following table.

Table 36.4: Effective transaction sequence

PartType. CommitLevel	OrderPriority. Maintain-Commitments	Netting effective transaction sequence	CTP effective transaction sequence
High	Y	TransactionSequence	TransactionSequence
	N	0	0
Medium	Y	0	TransactionSequence
	N	0	0
Low	Y	0	0
	N	0	0

If the effective transaction sequence is **0**, then the transaction sequence is not used for determining the order's priority.

Updating TransactionSequence values

The **TransactionSequence** field of the **ScheduledReceipt** and **IndependentDemand** tables acts as a global count of how many transactions have been added to or changed in the table, and is used to sort orders with the same priorities. When a record in the table is added or modified, its **TransactionSequence** value is incremented, or changed to be one more than the previous highest **TransactionSequence** value in the table.

If a **TransactionSequence** value is not specified when the initial data is imported into RapidResponse, it defaults to **0.**

For example, consider the following sample **IndependentDemand** table.

Table 36.5: Sample IndependentDemand table

Order.Id	DueDate	OrderPriority	Transaction Sequence
23-098	October 9	High	0
23-099	October 9	Medium	1
23-100	October 9	Low	2
23-101	October 9	Medium	3

If order 23-100 has its **OrderPriority** changed to **Medium**, the **TransactionSequence** value is increased to **4**, one more than the previous highest value of **3**, which is assigned to order 23-101.

If a new order is then added, its **TransactionSequence** field will be **5**, one more than the previous highest value. The table would then look like this.

Table 36.6: Sample IndependentDemand with changes

Order.Id	DueDate	OrderPriority	Transaction Sequence
23-098	October 9	High	0
23-099	October 9	Medium	1
23-100	October 9	Medium	4
23-101	October 9	Medium	3
23-102	October 9	High	5

The **TransactionSequence** value is incremented to ensure that prioritized orders are still filled according to plan. An order with a changed priority is filled after all other orders with that priority have been filled, ensuring the supplies that have been set aside for those orders actually get used to fill them. For more information about how orders are filled, see "Prioritized demand order" on page 1771.

Note: In order for a priority change on a scheduled receipt or independent demand to trigger an automatic update of the corresponding **TransactionSequence** field, the **TransactionSequenceEnabled** configuration record must be set as discussed in "Enabling automatic maintenance of TransactionSequence values" on page 2207.

Priorities on dependent demands

The priority of a dependent demand comes from the supply that creates it. The effective priority and effective transaction sequence for a dependent demand are calculated the same way as for independent demands. For more information, see "Effective priority" on page 1767 and "Effective transaction sequence" on page 1769.

The priority fields, **OrderPriority** (planned orders) and **ScheduledReceipt TransactionSequence**, are passed from the demand driver to the dependent demand according to the following table.

Table 36.7: Priorities on dependent demands

Demand driver	Dependent demand
ScheduledReceipt	Allocations NewAllocations NewTransferAllocations
PlannedOrder	PlannedAllocations PlannedTransferAllocations

If a **ScheduledReceipt** creates a **NewAllocations** dependent demand, the **NewAllocations** will have the same **OrderPriority** and **TransactionSequence** as that **ScheduledReceipt**.

Prioritized demand order

Prioritized demands are filled in the sequence of their priorities. RapidResponse sorts demands by the criteria in the following list. If two orders have the same value for any given criterion, RapidResponse sorts them according to the next criterion in the list.

1. **Effective demand period**—for period effective priorities, demands due in one period (as set by the part's PlanningCalendars.AllocationCalendar value) are given a higher priority than demands in subsequent periods. That is, any past due demands are given the highest priority. For non-period effective priorities, the period in which the demand date falls is not relevant. For more information, see "Period effective priority" on page 1768.
2. **Effective priority**—within an effective demand period, demands are prioritized based on their priority values as set by the demand's OrderPriority.PlanningPriority value. For non-period effective priorities, the PlanningPriority is assumed to be absolute (that is, the date the demand falls on does not influence priority).
3. **Effective transaction sequence**—if multiple orders have the same PlanningPriority then they are filled according to their IndependentDemand.TransactionSequence value (this field is incremented each time an order is added or modified). For more information, see "Effective transaction sequence" on page 1769.
4. **Attribute weight**—if any orders within the same priority are associated with active attribute rules , then demands having more attribute-based supply limitations are satisfied before those having fewer attribute-based supply limitations,, and demands not having any attribute-based supply limitations are satisfied last. For more information about attributes, see "Attribute-based planning" on page 1817.
5. **Calculated due date** (**ReschedDate** or **DueDate**)— If multiple orders have the same effective priority, transaction sequence and order priority, they are filled in due date sequence.
6. **Allotment order**—If multiple orders have the same due date, they are filled according to their associated **DemandType.AllotmentOrder** values. Lower values in this field are given higher priority.

CTP supply allotments

Order priorities are considered only if there is not enough supply to fill all demands. If enough supply exists, orders are filled in the order of their **IndependentDemand.DueDate** values, no matter what **OrderPriority.PlanningPriority** values are associated with each **IndependentDemand**. For more information about how RapidResponse fills demand orders, see "Capable-to-Promise calculations" on page 1453.

If the available supply is insufficient to fill all of the independent demands, then RapidResponse uses the **IndependentDemand.OrderPriority** values (and, by extension, the associated **OrderPriority.PlanningPriority** values) to decide which orders get filled and which get made late.

The order with the latest **DueDate** is matched with the supply with the latest **AvailableDate**. If the **AvailableDate** is after the **DueDate**, RapidResponse looks for the unsatisfied demand with the lowest priority and latest **DueDate**, and allots the supply to that demand. This makes the low-priority demand late, and fills the demand with the later **DueDate**.

After the supply has been exhausted, the orders that are late will most likely be low-priority, and the demands that have been filled will most likely be high-priority.

For example, consider two orders for 150 units of the same part. Order 1 has an **OrderPriority** value of **High**, and Order 2 has an **OrderPriority** value of **Medium**. Order 2 is due two days before Order1, but there is not enough supply to fill both orders.

Two **ScheduledReceipts**, each for 100 units, for the part are available the day before Order 2's **DueDate**. Another **ScheduledReceipt**, for another 100 units, is available the day after Order 1's **DueDate**.

Because there is insufficient supply, the higher-priority order, Order 1, is filled first using the supply from the **ScheduledReceipts** received before Order 2's **DueDate**. Order 2 has the remainder of those **ScheduledReceipts** allocated to it, but must wait for the rest of its supply. When the third **ScheduledReceipt** is received, its supply is used to finish filling Order 2. Order 1 is on time, Order 2 is late.

Note: Supply allotments are typically made for each component part without knowledge of the allotments made on any of its sibling parts. In some cases, however, knowledge of its sibling component allotments could lead to better allotments on a given component (and hence, better demand availabilities). For example, if on-time supply of a limited component is allocated to a demand which is otherwise late due to late supply of another (sibling) component, then the on-time component supply might be better directed elsewhere. This can be achieved using non-blocking allocations as discussed in "Multi-level search logic" on page 1863.

CTP allocation with OnHand supply

RapidResponse allocates **OnHand** supplies to low-priority demands (those with a high numerical value in **OrderPriority.PlanningPriority**) as long as high-priority demands can be filled on time with **ScheduledReceipts** or **PlannedOrders**. If the high-priority demands cannot be filled, **OnHand** supplies are allocated to the demand with the earliest **DueDate** and highest priority.

For example, consider two orders for 150 units of the same part. Order 1 has an **OrderPriority** value of **High**, and Order 2 has an **OrderPriority** value of **Medium**. Order 2 is due two days before Order1, but there is not enough supply to fill both orders.

A **ScheduledReceipt** for 75 units of the part is available two days before Order 2's **DueDate**, and there are 100 units on hand. A **PlannedOrder** for another 200 units is due the day after Order 1's **DueDate**.

Because there is not enough supply to fill both orders on time, the on hand supply and **ScheduledReceipt** are allocated to Order 1, and the **PlannedOrder** is allocated to Order 2. Order 1 is on time, Order 2 is late.

If the **PlannedOrder** was due the day before Order 1 was due, the on hand supply and **ScheduledReceipt** would be allocated to Order 2, and the **PlannedOrder** to Order 1. Both orders would be on time.

Safety stock and priority adjustments

As discussed in "Safety stock" on page 1400, safety stock is used to provide a buffer to guard against variability in demand and supply. For example, it might be used to satisfy orders in excess of forecast within a period or to compensate for unexpected supply disruptions. Based on a fixed or calculated safety stock level, Netting plans supply to maintain the safety stock level as required.

To control how supply to satisfy safety stock requirements is planned and its priority set relative to actual demands, the **DemandTransformationRule** control field on the **SafetyStockPolicy** table can be used. This field has two options, "Ignore" and "FirstInFirstOut".

With the "Ignore" setting, planned supplies generally take the priority of their driver requirement and an increase in the safety stock level is seen as a low priority demand. To satisfy such a safety stock demand, planned orders are generated with the part's default priority as determined from the **PartType.DefaultPriority** reference (typically, a low priority). For parts that use a **PartType.CommitLevel** setting of "High" or "MediumExplodePriority", this default priority level on safety stock supplies is then propagated to the dependent demands placed on lower level components in the structure.

Note that "Ignore" is the default **DemandTransformationRule** setting and corresponds to how safety stock logic was always applied prior to Service Update 1905. However a limitation of this setting is that if supplies created to satisfy stock demands end up being used to satisfy real demands, then the priority on those real demands can be lost or hidden which might lead to adverse planning results. For example, this might typically be seen when a drop in the safety stock level results in the newly freed up supply being used to satisfy a real demand. For such cases, the "FirstInFirstOut" setting can be used instead.

With the "FirstInFirstOut" setting, each planned supply is assumed to have been created to eventually satisfy a real demand. When there is an increase in the safety stock level, the priority on a new planned order created to satisfy it is set to the priority found on the next real demand for the part based on due date sequence. This priority can then also be propagated to any dependent demands placed on lower level components in the part's structure (depending on **CommitLevel**).

Note that once supply driven by a safety stock requirement is assumed to have been used to satisfy a real demand, this implies that some or all of the supply driven by that same real demand will then be used to satisfy a different/subsequent demand. This process of "demand transformations" can then potentially continue throughout the horizon will both safety stock and real demand requirements taking their priorities from the later real demands they are assumed to satisfy. Additionally, if the safety stock (or other) requirement being satisfied is large enough to satisfy more than one real demand, then the planned supply might be split into multiple orders to match the priorities on those demands.

Example: Demand transformations and priority

Assume a part has the following independent demands and safety stock requirements and that the part's default priority is set to "Buffer" which is assumed to be the lowest priority.

Table 36.8: Prioritized demands and safety stock levels

IndependentDemand records			TimePhasedSafety records	
DueDate	**Quantity**	**OrderPriority**	**DueDate**	**Quantity**
August 1	300	Medium	August 2	800
August 5	400	Low	August 8	1000
August 12	600	High	August 19	500
August 16	800	Medium	August 22	0
August 20	200	High		

When the **DemandTransformationRule** is set to "Ignore", new supplies are planned as shown in the table below. In this case, there are six planned orders created and each of these has a priority corresponding to that of the driving independent demand or safety stock requirement. Note also that there is a supply planned for both safety stock increases as well as for five of the six real demands. However, there is no planned order corresponding to the High priority demand for 200 units on August 20th as this requirement is covered by supply released by the safety stock reduction on August 19.

Table 36.9: PlannedOrder records with "Ignore" setting

DueDate	Quantity	Priority
August 1	300	Medium
August 2	800	Buffer
August 5	400	Low
August 8	200	Buffer
August 12	600	High
August 16	800	Medium

Instead if **DemandTransformationRule** were set to "FirstInFirstOut", then each planned order would takes its priority from the real demand that it is assumed eventually satisfy (up until the end of the horizon when there are no more real demands to satisfy). Note also with this setting that nine planned orders would be created as opposed to six with "Ignore". This is because some supplies driven by safety stock requirements are seen to eventually satisfy more than one prioritized demand. For example, there is a total of 800 units of supply planned on August 1st which is driven by the safety stock requirement on that date. However, this supply is seen as covering both the 400 unit "Low" priority demand due on August 5th and 400 units of the 600 unit "High" priority demand due on August 12th as shown in the following table.

Table 36.10: PlannedOrder records with "FirstInFirstOut" setting

DueDate	Quantity	Priority	Supply priority is taken from...
August 1	300	Medium	Demand due on August 1
August 2	400	Low	Demand due on August 5
August 2	400	High	Demand due on August 12
August 5	200	High	Demand due on August 12

Table 36.10: PlannedOrder records with "FirstInFirstOut" setting

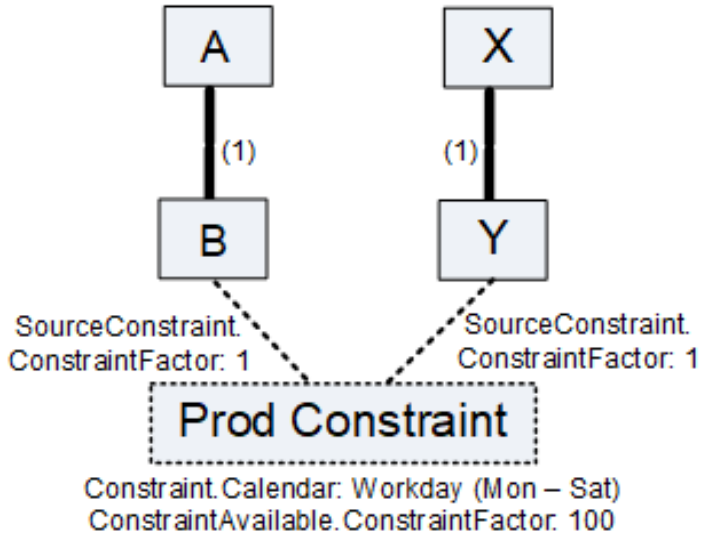
DueDate	Quantity	Priority	Supply priority is taken from...
August 5	200	Medium	Demand due on August 16
August 8	200	Medium	Demand due on August 16
August 12	400	Medium	Demand due on August 16
August 12	200	High	Demand due on August 20
August16	800	Buffer	Safety stock (default priority)

Example: DemandTransformationRule and planning

The following shows a simplified example of how the **SafetyStockPolicy.DemandTransformationRule** setting can impact planning results based on how demands transformations and the priorities set on supplies.

Suppose two assemblies A and X built using components that share a common constraint, which allows for 100 units of component supplies to be built each day, as shown below. All parts are assumed to have lead time of 1 day, and use a **PartType.CommitLevel** setting of "High".

Figure 36.1 Assemblies with components that share a constraint



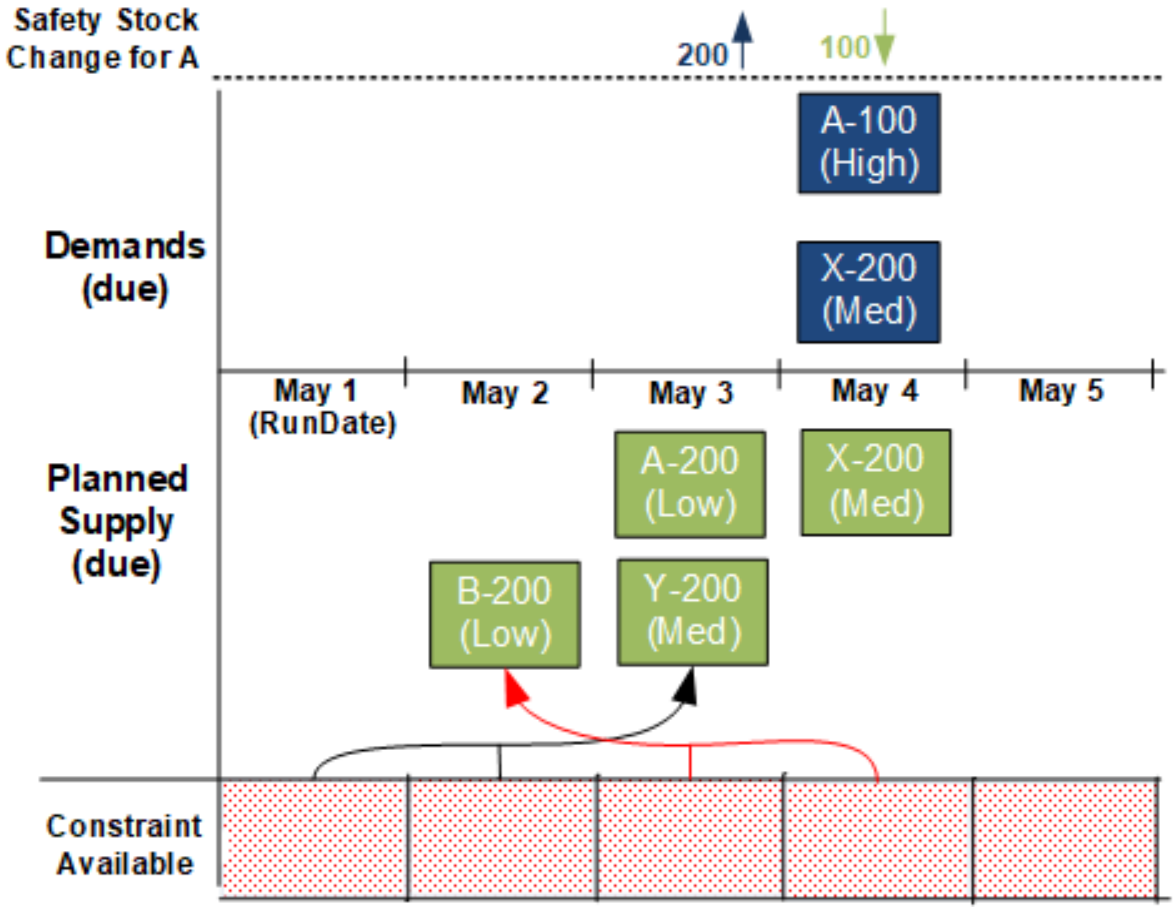
Assume a high priority demand for assembly A and medium priority demand for assembly X, along with an initial safety stock requirement and subsequent safety stock reduction for A as shown below. Further assume that no input supplies exist for any of the parts, and that part A shown references a **PartType.DefaultPriority** of "Low".

Table 36.11: Prioritized demands and safety stock requirements for parts A and X

IndependentDemand records				TimePhasedSafety records		
Part	**DueDate**	**Quantity**	**Priortiy**	**Part**	**Date**	**Quantity**
A	May 4	100	High	A	May 3	200
X	May 4	200	Med	A	May 4	100

With the default **DemandTransformationRule** setting of "Ignore, planned orders would be generated as shown below.

Figure 36.2 Planned supplies - DemandTransformationRule = "Ignore"

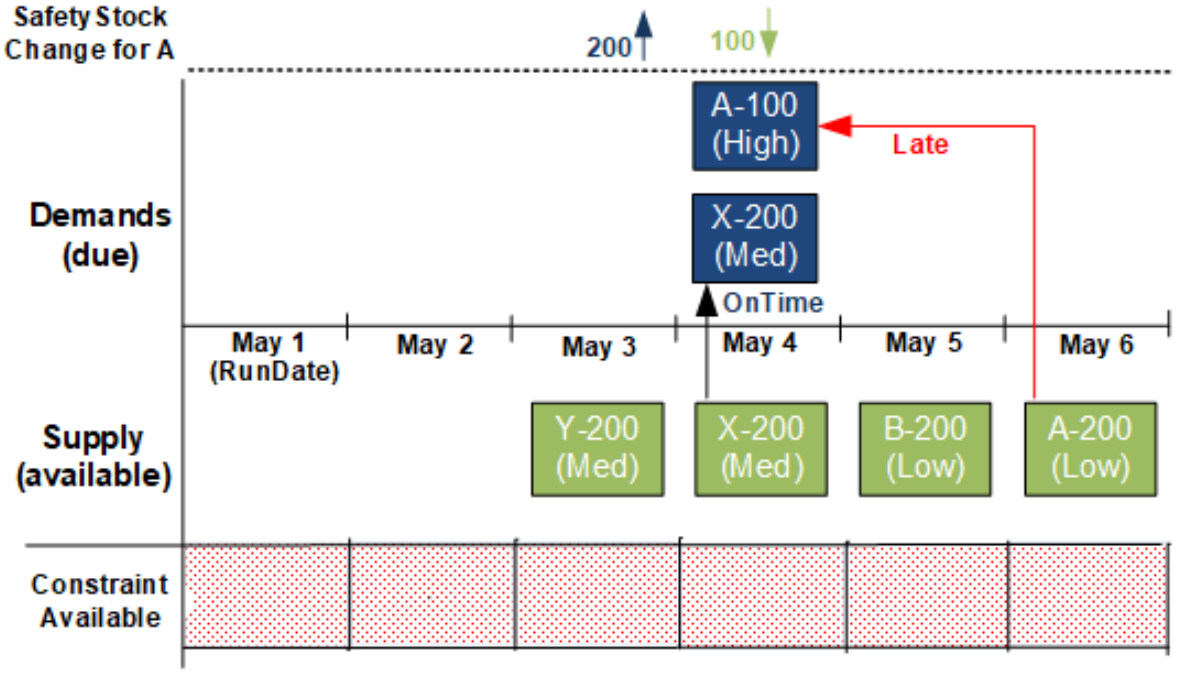


Note the following:

- The 200 unit planned order for A is generated to cover the initial satisfy stock requirement on May 3rd and therefore assigned a "Low" priority which is also propagated to the subsequent dependent demand and planned order generated on component B (this assumes that A's default priority is set to "Low").
- No further supplies are planned for assembly A as the subsequent 100 unit reduction in the safety stock level provides sufficient supply to cover the 100 unit demand on May 4th. Thus, the 100 unit "High" priority demand is covered by the released "Low" priority safety stock supply and the priority on this demand is not propagated to lower levels.
- The 200 unit planned order for X is generated to cover the 200 unit independent demand due on May 4th. The planned order inherits the "Medium" priority from the demand it covers and this priority is also propagated to the dependent demand and planned order generated for its component Y.
- Note that based on the priorities on the lower level components, the limited constraint available is assigned to the medium priority supply for component Y before later constraint is assigned to the low priority supply for component B.

Thus, based on demand/supply priorities and limited constraint, supply availability would be as seen below.

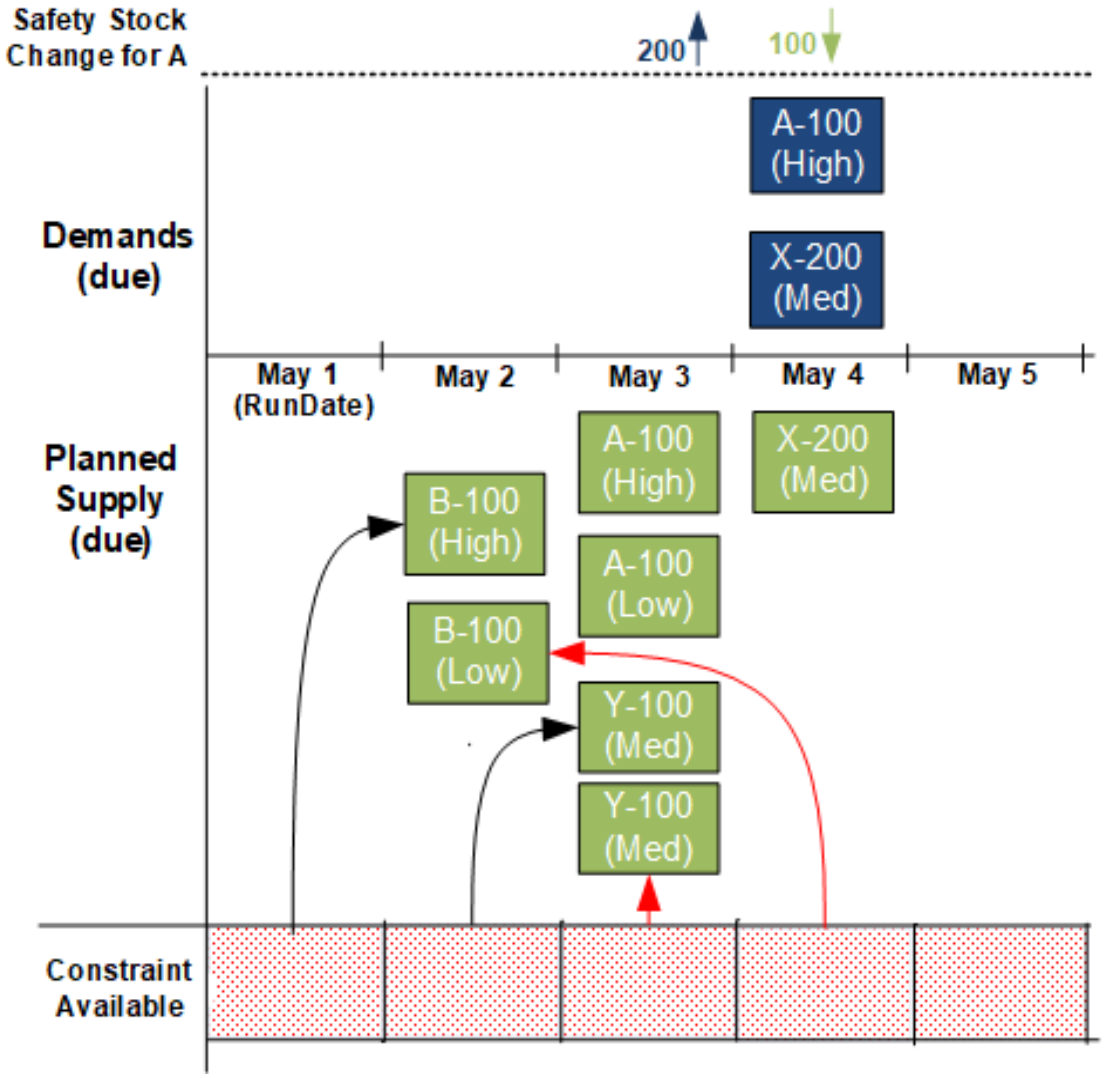
Figure 36.3 Supply availability- DemandTransformationRule = "Ignore"



Note that the medium priority demand for X due on My 4th is satisfied by medium priority supply and is available on time. However, the high priority demand for A due on the same date is available late with low priority supply for A being used to satisfy this demand as well as the safety stock requirement.

Instead assume the **DemandTransformationRule** setting was "FirstInFirstOut". In this case, supplies would be planned as shown below:

Figure 36.4 Planned supplies - DemandTransformationRule = "FirstInFirstOut"



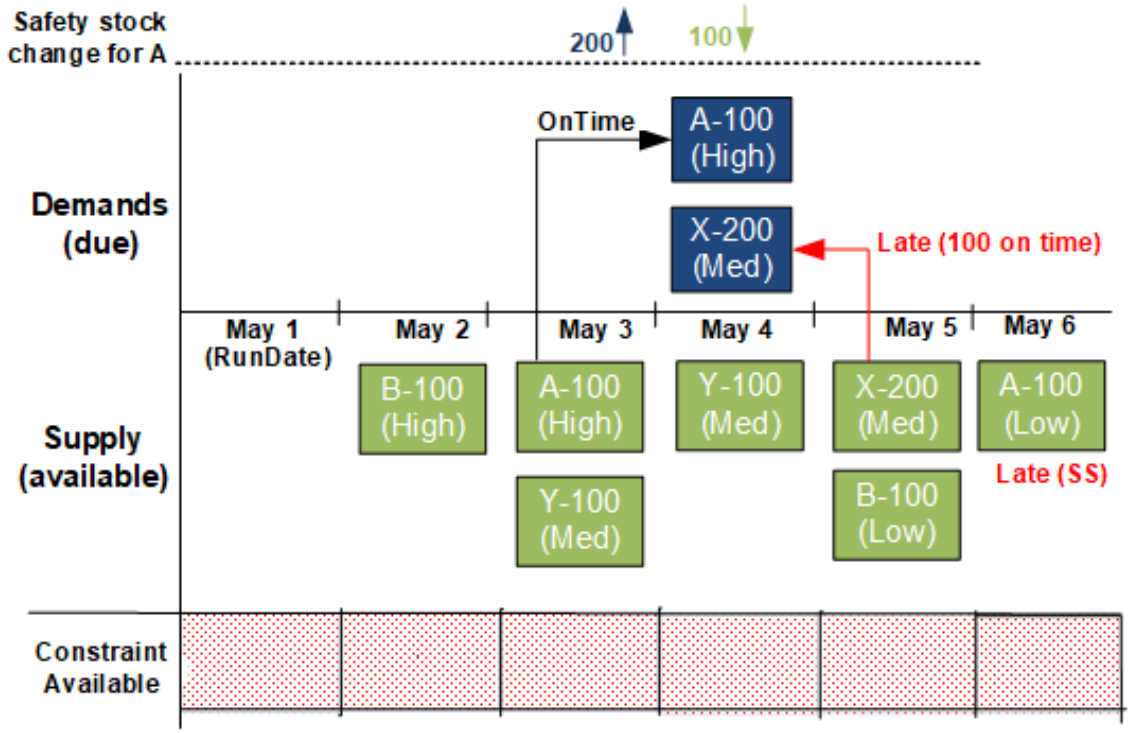
Note the following:

- Two separate planned orders for 100 units of assembly A of 100 units are now generated and both due on May 3rd, but with differing priorities.
- The "High" priority supply for A takes its priority from the "High" priority demand on May 4th that it will ultimately satisfy (after the reduction in the safety stock level for which it would originally be planned). The "Low" priority supply for A does not ultimately cover any real demand and thus receives the part's default priority.
- Similarly, there are two separate dependent demands and planned orders now placed on lower-level component B which inherit their respective priorities from the higher-level demands they are created for.
- As in the previous case, a 200 unit planned order of "Medium" priority for X is generated to cover the 200 unit independent demand due on May 4th, and this priority lvel is propagated to the dependent demand and planned supplies for component Y.

- Note in this case now the limited constraint available is first assigned to the "High" priority demand for B, then the "Medium" priority demands for Y, and finally the "Low" priority demand for B.

Based on the demand/supply priorities and constraint available seen in the previous illustration, supply availability would be as seen below.

Figure 36.5 Demand availability - DemandTransformationRule = "FirstInFirstOut"



The main difference compared to the previous case with the "Ignore" setting is that the "High" priority demand of 100 units for A is now satisfied on time as it's priority has been preserved and passed down to the supplies that ultimately end up satisfying it. Additionally there is Low" priority supply for A that is available late, but this supply is now for 100 units and intended solely to satisfy safety stock. Also note that the "Medium" priority demand for X is now satisfied late (although 100 units of the demand could be satisfied on time).

Limitations on other analytics

When demand transformations and safety stock priority adjustments are enabled , a small number of other analytics or calculations might be disabled or not work as expected. The following table outlines other calculations impacted by when the **DemandTransformationRule** setting is "FirstInFirstOut".

Table 36.12: DemandTransformationRule limitations with other analytic calculations

Analytic / Calculation	Impact on demand transformation
Material expiry	Demand transformations are not supported on parts subject to material expiration. Because planned orders generated when material expiry is enabled could expire, supplies created to fulfill safety stock requirements are treated differently than those that satisfy real demands. However, demand transformation logic effectively transforms safety stock demands to the subsequent real demands. Thus use of these analytics together could result in unexpected results and supply expiration.
Attribute-based planning	Demand transformations are not supported with attribute-based planning as demand transformation logic may not respect attributes while applying the transformation logic.
Prioritized safety stock	Demand transformation logic treats increased in the safety stock level as low-priority demands. Transformation involving high-priority safety stock is not supported. Thus this logic is not expected to be used with the holding of safety stock allotments.
Partial planning	Partial planning is not fully supported with demand transformation logic. Partial planning is intended to ignore or short portions of a demand quantity that can't be satisfied by its due date or within a certain tolerance of its due date. However, when demand transformation logic is logic the due date of the transformed demand is not considered in partial planning decisions.
Multi-level search	Demand transformation logic relies on a complete demand picture for a particular part (and model, unit, pool combination). However, because multi-level search is a "depth-first" analytic that processes one demand at a time across all levels of the structure, the full demand picture is only known for the top-level part with fixed driving demands.
Global substitution	Demand transformation logic relies on a complete demand picture for a particular part (and model, unit, pool combination). However similar to multi-level search logic, the full demand picture is not known for alternate parts.
BOM level part substitution	Demand transformation logic relies on a complete demand picture for a particular part (and model, unit, pool combination). However similar to multi-level search logic, the full demand picture is not known for substitute components.

Priority and safety stock holds in CTP

When the **SafetyStockPolicy.DemandTransformationRule** field is set to the default "Ignore" setting, priority on orders generated to satisfy safety stock requirements is set to the part's default priority as determined by the **PartType.DefaultPriority** reference. Typically, the **DefaultPriority** will be set to reference a low priority level.

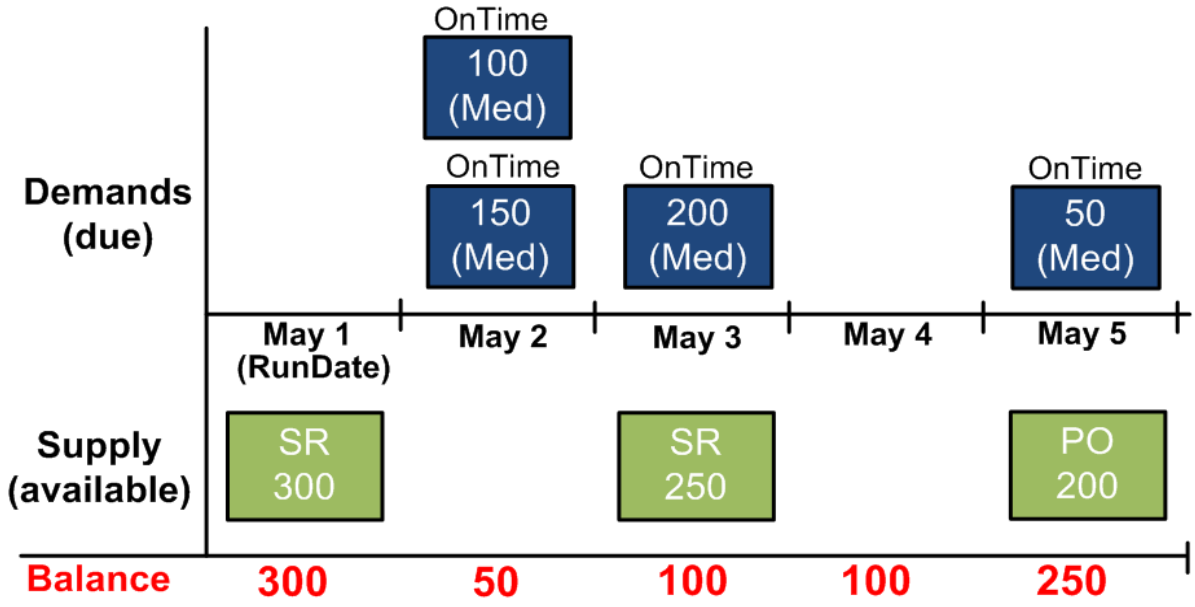
Optionally, for time-phased safety stock only, the priority on a given safety stock requirement can be explicitly set using the **TimePhasedSafety.OrderPriority** reference. Otherwise, if the **OrderPriority** reference is left Null, then the default priority from the part type is used regardless.

Regardless of the priority on safety stock or how that priority is determined, any actual demands are (by default) given preference over safety stock in Capable-to-Promise logic. This means that inventory levels are allowed to temporarily fall below the safety stock level if doing so ensures that actual demands are satisfied on time (or less late than they otherwise would be). For example, assume a part "AA" with **SafetyStockQuantityRule** set to "TimePhasedSafety" has the following records defined in the **TimePhasedSafety** table:

Table 36.13: TimePhasedSafety values

Part	Date	Quantity
AA	Past	300
AA	May 4	250

Further, assume this part has a lead time of 4 days, along with the set of demands and supplies shown in the following illustration. In this case, all demands shown can be satisfied on time. However, in order to ensure these demands can be satisfied on time, the inventory balance is allowed to fall below the safety stock level from May 2nd until a new planned order is available on May 5th.



Records reporting details of allotments of supply to demand would then be reported in the **SupplyDemand** table as follows. Note that records shown where **DemandSource** is set to "None" and **DemandDueDate** is "Undefined" represent supply allotments made to satisfy a safety stock requirement.

Table 36.14: SupplyDemand allotments

Date	Allocated Quantity	Supply AvailableDate	Demand DueDate	DemandSource	IndependentDemand.AvailableDate
May 1	150	May 1	May 2	IndependentDemand	May 1
May 1	100	May 1	May 2	IndependentDemand	May 1
May 1	50	May 1		None	
May 3	200	May 3	May 3		May 3
May 3	50	May 3		None	
May 5	150	May 5		None	
May 5	50	May 5	May 5	IndependentDemand	May 5

Configure prioritized safety stock

In some cases, there might be a need to ensure that, when there is limited supply, safety stock levels are still respected and thereby given preference over some (or all) actual demands. For example, if customer sales can only be realized from an available stock of inventory, there might be a high priority need to ensure a minimum safety stock is always maintained.

To enable support for prioritized safety stock in capable-to-promise logic, the **SafetyStockPolicy.CTPHoldRule** field can be used. This field has the following options:

- **Ignore**- priority on safety stock is ignored in capable-to-promise logic. Actual demands are always given preference over safety stock regardless of priority.
- **Use**- priority on safety stock is used in capable-to-promise logic. Safety stock requirements are given preference over lower-priority demands (demands of a higher or equivalent priority to safety stock are still given preference over maintaining the safety stock level).

Typically, if using a **CTPHoldRule** of "Use", then safety stock levels should be defined using a **SafetyQuantityRule** that relies on values provided in the **TimePhasedSafety** table (for example, the "TimePhasedQuantity" setting). This allows for priority to be explicitly set on safety stock requirements and supports different levels of prioritized safety stock for a part. For example, there might be a minimum safety stock level (with high priority) that should always be maintained and an additional buffer level of safety stock (with medium or low priority) that should be maintained only when doing so doesn't impact actual demand availability. Note that if a part has safety stock of different priority safety stock level contributing to its overall safety stock, each is maintained separately and a change in one doesn't impact, or require a change in, the other levels.

If using safety stock methods that don't utilize values in the **TimePhasedSafety** table, then the priority on safety stock will always be set to the part's default priority (the priority referenced in **PartType.DefaultPriority**). Thus, safety stock requirements are only ever given preference over those actual demands that have a lower priority than the part's default priority (and they would always be preferred to them).

Additionally, any parts that use a **CTPHoldRule** of "Use" should have incremental availability logic enabled. That is, the parts should reference an **IncrementalRule** record where the **ProcessingRule** is set to "On". Otherwise, some details related to prioritized safety stock will not be accurately reflected or reported.

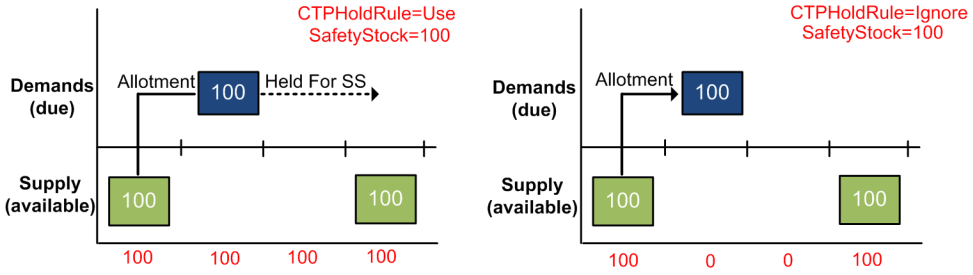
Note: The ability to define priorities on **TimePhasedSafety** records was added in Version 2016.2, Service Update 18. Currently this is only supported when using a **SafetyStockQuantityRule** of "TimePhasedQuantity". Support for other time-phased methods ("TimePhasedSafetyQuantityToLatest", "TimePhasedSafetyDaysTotal", and "TimePhasedSafetyDaysMaximum" will be added in a future service update.

Prioritized safety stock in capable-to-promise calculations

When there is limited supply available to satisfy all requirements on time, safety stock requirements are given preference over actual demands that have a lower priority (but actual demands of an equal or higher priority level are still given preference over safety stock).

This is done by holding some normal allotments of supply to demand, as determined by capable-to-promise logic, until those allotments can be made without violating the safety stock level. Or, if there is no way ensure the higher priority safety stock level is maintained, allotments of supply to demand are held to ensure safety stock levels are minimally violated.

In either case, when determining which allotments to hold, the duration of these holds is minimized as possible and further preference is given to holding allotments that satisfy small portions of a demand over allotments that satisfy large portions of a demand. Note that the holding of supplies to satisfy higher priority safety stock is dependent on the part's **SafetyStockPolicy.CTPHoldRule** setting as shown in the following illustration:



The details of held allotments are calculated and reported in the **SupplyDemand** table. The following two fields are populated on any allotments of supply to demand that should be held in order to respect the safety stock level (assuming the part uses a **SafetyStockPolicy.CTPHoldRule** setting of "Use"):

- **HoldStartDate**—The date from which the allotment of supply needed to be held to respect (or minimally violate) the safety stock level. This is date the safety stock level would be violated if the supply were actually used by the demand.
- **HoldEndDate**—The date on which the allotment of supply can be actually used by the demand without violating the safety stock level.

If a given demand is subject to any held allotments of supply, then its **AvailableDate** cannot be any earlier than the latest **HoldEndDate** on any of those allotments. Thus, the demand can end up being later than the availability of its allotted supplies would otherwise dictate.

The **HoldStartDate** and **HoldEndDate** values are also reported in fields the **CTPActivity** table. This allows for details of held allotments to be reported on specific supplies.

Additionally, the calculated **CTPSafetyStockHoldings** table can be used to report details of all a part's supply allotments that are available to support safety stock levels. This table contains records on each that the either the amount of supply a part has available to safety stock changes or the specific supplies available to that safety stock level change. It combines allotments of supply to actual demand that have been held, allotments of supply to safety stock (where DemandSource is "None"), as well as supply that is eligible to be pushed to a destination site due to exceeding the maximum inventory level. For more information about this table, see "CTPSafetyStockHoldings table" on page 1002.

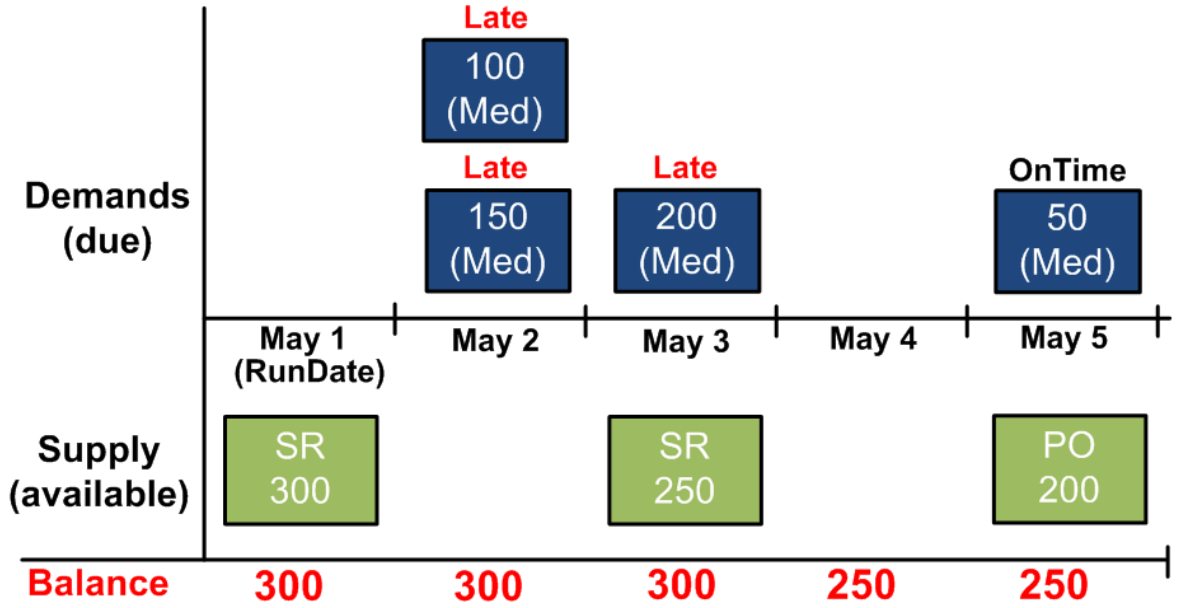
Example of prioritized safety stock (one priority level)

Assume a part "AA" that references a **SafetyStockPolicy** record with **SafetyStockQuantityRule** set to "TimePhasedSafety" and **CTPHoldRule** set to "Use", along with the following records defined in the **TimePhasedSafety** table:

Table 36.15: TimePhasedSafety values

Part	Date	Quantity	OrderPriority
AA	May 1	300	High
AA	May 4	250	High

Further, assume this part has a lead time of 4 days, along with the set of demands and supplies shown in the following illustration (these are the same set of demands and supplies used earlier in this section). In this case, note that the demands shown are of a lower priority than the safety stock requirements. Thus, although there is enough supply available to satisfy all demands on time, several of the supply allotments need to be held in order to ensure the higher priority safety stock level is not violated. As a result, the demands due on May 2nd and May 3rd are available late.



The **SupplyDemand** table would then report details of these allotments, including hold dates, as follows (the rows with undefined **DemandDueDate** values represent allotments to the "None" demand source). Note that drop in the safety stock level from 300 to 250 on May 4th means that 50 units of the demand due on May 3rd can potentially be made available just one day late, while the rest of this demand's allotted supply remains held until May 5th and is two days late.

Table 36.16: SupplyDemand allotments

Date	Allocated Quantity	Supply AvailableDate	Demand DueDate	IndependentDemand .AvailableDate	HoldStart Date	HoldEnd Date
May 1	150	May 1	May 2	May 3	May 1	May 3
May 1	100	May 1	May 2	May 3	May 1	May 3
May 1	50	May 1				
May 3	150	May 3	May 3	May 5	May 3	May 5
May 3	50	May 3	May 3	May 5	May 3	May 4
May 3	50	May 3				
May 5	150	May 5				
May 5	50	May 5	May 5	May 5		

Further, records showing supply available to safety stock would be reported in the **CTPSafetyStockHoldings** table on May 1st, 3rd, 4th, and 5th. The summed

SupplyDemand.AllocatedQty for each of those dates, representing the supply available to safety stock, would be as faollows:

Table 36.17: CTPSafetyStockHoldings

Date	SupplyDemand. AllocatedQuantity	Calculated as the sum of the following supply allotments by date
May 1	300	150(May 1) + 100(May 1) + 50(May 1)
May 3	300	50(May 1) + 150(May 3) + 50(May 3) + 50(May 3)
May 4	250	50(May 1) + 150(May 3) + 50(May 3)
May 5	250	50(May 1) + 50(May 3) + 150(May 5)

Example of prioritized safety stock (multiple priority levels)

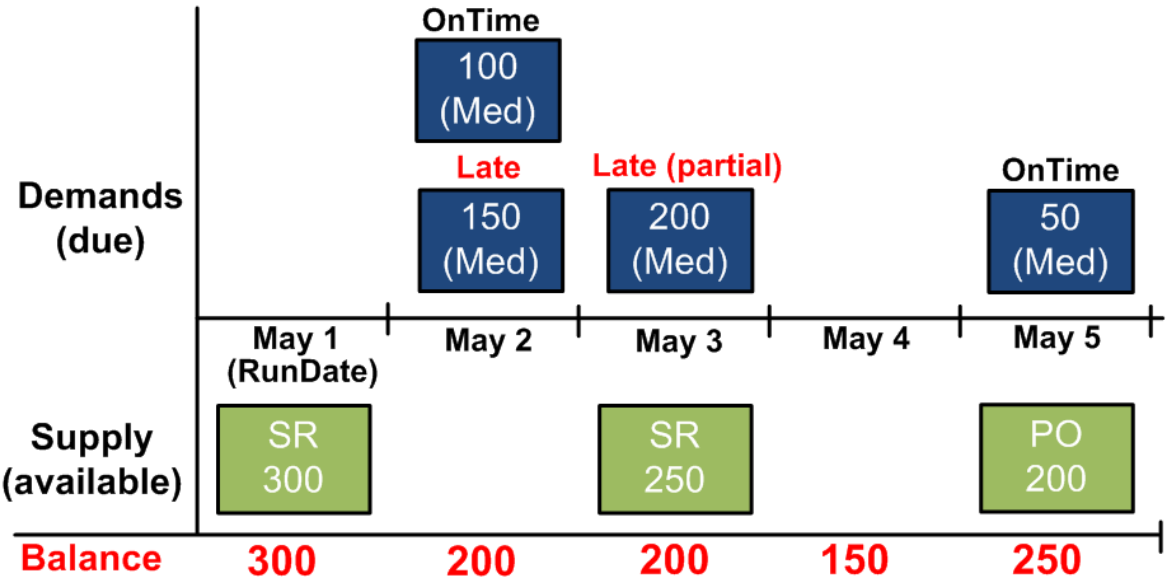
Assume a part "AA" that references a **SafetyStockPolicy** record with **SafetyStockQuantityRule** set to "TimePhasedSafety" and **CTPHoldRule** set to "Use", along with the following records defined in the **TimePhasedSafety** table. In this case, the "High" priority safety stock is meant to represent a minimum safety stock level that should always be maintained and the "Med" priority stock represents an additional buffer stock that should be maintained as long as doing so does not impact the availability of actual demands:

Table 36.18: TimePhasedSafety values

Part	Date	Quantity	OrderPriority
AA	May 1	200	High
AA	May 1	100	Med
AA	May 4	150	High
AA	May 15	50	Med

Further, assume this part has a lead time of 4 days, along with the set of demands and supplies shown in the following illustration. In this case, note that the demands shown have a Medium priority which is a lower priority than just a portion of the safety stock requirements.

Thus, the demand for 100 units on May 2nd can be satisfied on time as the Low priority portion of the safety stock level is allowed to be temporarily violated. However, the other demand on May 2nd cannot be satisfied on time as its allotment of supply needs to be held to ensure the High priority portion of safety stock is maintained. Similarly, 100 units of the demand on May 3rd can potentially be satisfied on time as 10o units of supply is held to ensure the High priority portion of safety stock is maintained.



The **SupplyDemand** table would then report details of these allotments, including hold dates, as follows (the rows with undefined **DemandDueDate** values represent allotments to the "None" demand source). Note that drop in the High priority portion of the safety stock level from 200 to 150 on May 4th means that 50 units of the late portion of the demand due on May 3rd can be available just one day late, while the remaining 50 units of allotted supply remains held until May 5th and is two days late.

Table 36.19: SupplyDemand allotments

Date	Allocated Quantity	Supply AvailableDate	Demand DueDate	IndependentDemand .AvailableDate	HoldStart Date	HoldEnd Date
May 1	150	May 1	May 2	May 3	May 1	May 3
May 1	100	May 1	May 2	May 1		
May 1	50	May 1				
May 3	100	May 3	May 3	May 5		
May 3	50	May 3	May 3	May 5	May 3	May 4
May 3	50	May 3	May 3	May 5	May 3	May 5
May 3	50	May 3				
May 5	100	May 5				
May 5	50	May 5	May 5	May 5		
May 5	50	May 5				

Further, records showing supply available to safety stock would be reported in the **CTPSafetyStockHoldings** table on May 1, , 4, 5, and 15. The summed **SupplyDemand.AllocatedQty** for each of those dates and priority levels, representing the supply available to safety stock, would be as follows:

Table 36.20: CTPSafetyStockHoldings

Date	Priority	SupplyDemand.AllocatedQuantity	Calculated as the sum of the following supply allotments by date
May 1	High	200	150(May 1) + 50(May 1)
May 3	High	200	50(May 1) + 50(May 3) + 50 (May 3) + 50(May 3)
May 4	High	150	50(May 1) + 50(May 3) + 50(May 3)
May 5	High	150	50(May 5) + 50(May 5) + 50(May 5)
May 5	Low	100	50(May 1) + 50(May 3)
May 15	High	150	50(May 1) + 50(May 3) + 50(May 5)

Limitations on prioritized safety stock

When prioritized safety stock is configured for a part, a small number of other analytics or calculations might be disabled, or not work as expected. The following table outlines other calculations impacted by when prioritized safety stock is enabled in capable-to-promise logic.

Table 36.21: Prioritized safety stock limitations with other analytic calculations

Analytic / Calculation	Impact of respecting priority on safety stock
Material expiry	Material expiration is not considered when holding allotments of supply to demand. Thus, supply might be held and not released to the demand later than expiry rules would otherwise allow.
Fair share (supply)	If fair share of supply has been configured, the holding of supply allotments does not attempt to ensure holds respect the fair share requirements.
Allotment overrides	Allotment overrides, as well as other types of hard allotments such as allotments to negative on hand and allotments to InventoryTransferAllocation demands, cannot be held in respect of safety stock..
Attribute-based planning	Attributes are not considered in the process of holding allotments. Thus, the attributes on held supplies might not match the attribute requirements on safety stock demands.
Partial planning	Demands configured for partial planning can have their supply allotments held past the date (from tolerance) where the demand would otherwise be left unsatisfied.
Push planning	Push planning is generally compatible with prioritized safety stock and supply allotments pushed to a destination site due to exceeding the maximum inventory are available to safety stock between SupplyAvailableDate and PushDate. However, because any pushing of supplies occurs before the holding of supply allotments, if supply is pushed out immediately it is never be available for holding to support high priority safety stock.

Table 36.21: Prioritized safety stock limitations with other analytic calculations

Analytic / Calculation	Impact of respecting priority on safety stock
Multi-level search (and safety stock drops)	Drops in lower priority safety stock levels are generally eligible to be used by higher priority safety stock requirements. However, when multi-level search logic is enabled, this is only true if multi-level search smoothing logic is used (if its not, then such safety stock drops cannot be consumed and too much supply might end up being planned).
SafetyStockQuantityRule settings	Prioritized safety stock based on settings in the **TimePhasedSafety** table is only supported when **SafetyStockQuantityRule** is set to "TimePhasedQuantity". Other options such as "TimePhasedSafetyDaysTotal " and "TimePhasedSafetyDaysMaximum" are not supported. Other **SafetyQuantityRule** options are limited to a single (non-time phased) priority on safety stock based on the part type's "default priority"

Priorities on scheduled receipts

Priorities on scheduled receipts are used when allocating constraint and are also passed to the dependent demands generated by a receipt.

Scheduled receipts have **OrderPriority** and **TransactionSequence** input fields that can be used to set their priority. Scheduled receipts can also be configured to have their effective order priority dynamically calculated based on the priorities of the demands they satisfy during netting calculations (TransactionSequence is always based on the provided input value). The effective order priority on a scheduled receipt can be found in the calculated **EffOrderPriority** field.

Calculating scheduled receipt priorities

The **SupplyStatus.PrioritySource** control field is used to determine how the effective order priority on scheduled receipts is set. Based on the setting in this field, a scheduled receipt either takes its effective order priority from the value provided on input or has it dynamically calculated based on the priorities of the demands it satisfies during netting calculations. Note that the demands a receipt is seen as satisfying during netting calculations, may not necessarily be the ones it is ultimately allocated to by capable-to-promise calculations.

The PrioritySource field has four settings as described in the following table.

Table 36.22: SupplyStatus.PrioritySource settings

Value	Impact on effective order priority
Average	Effective order priority is set to an average of the priorities associated with all demands satisfied by the receipt during netting calculations. This setting sums the **OrderPriority.PlanningPriority** values found on all demands satisfied by the scheduled receipt, and then divides by the total number of demands satisfied by the receipt. This calculated average is then compared against all **OrderPriority.PlanningPriority** values in the same control set as the input order priority on the scheduled receipt. The **OrderPriority.Value** associated with the nearest **PlanningPriority** that is equal to or less than the calculated average is then reported.
FromInput	Effective order priority is always set to the value in the OrderPriority input field found on the scheduled receipt record.

Table 36.22: SupplyStatus.PrioritySource settings

Value	Impact on effective order priority
Highest	Effective order priority is set to the highest priority associated with any demand satisfied by the receipt during netting calculations. This setting looks for the lowest **OrderPriority.PlanningPriority** value found on any demand satisfied by the receipt. The **OrderPriority.Value** found on that record is then reported. **Note:** If multiple demands are found with the same PlanningPriority, this setting gives preference to values in the same control set as the input order priority on the scheduled receipt.
WeightedAverage	Effective order priority is set to a weighted average of the priorities associated with all demands satisfied by the receipt during netting calculations. This setting multiplies the **OrderPriority.PlanningPriority** value found on each demand satisfied by the receipt by the demand quantity, sums all these values together, and then divides by the total quantity of demands being satisfied by the receipt. This calculated average is then compared against all **OrderPriority.PlanningPriority** values in the same control set as the input order priority on the scheduled receipt. The **OrderPriority.Value** associated with the nearest **PlanningPriority** that is equal to or less than the calculated average is then reported.

In some cases, when using the Average, Highest, or WeightedAverage setting, multiple OrderPriority records being looked up within a control set might have the same PlanningPriority. When this happens, the following fields on the relevant OrderPriority records are used as an additional sort criteria towards determining the effective order priority.

- SchedulePriority (low to high)
- MaintainCommitments (Y before N)
- Value (alphabetical, A - Z)

Notes:

- If a given scheduled receipt is not used to satisfy any demands, the receipt's effective order priority is always set to the **ScheduledReceipt.OrderPriority** input value (regardless of its PrioritySource setting).
- For parts configured to use a days of supply policy (that is, where **PartType.UseDaysSupply** is set to "Y"), the priority on a demand is only considered and propagated down its dependent demands and the supplies used to satisfy them if the **PartType.UseDaysSupplyRule** is set to "ByPeriodWithPriority: For more information about this setting, see "ByPeriodWithPriority" on page 1366, and for more information about days of supply logic generally, see "Days of Supply calculations" on page 1362,

Reporting effective priorities on scheduled receipts

Based on the **SupplyStatus.PrioritySource** setting, an effective order priority is always calculated and reported in the **ScheduledReceipt.EffOrderPriority** field. This field represents the order priority to be used on scheduled receipts in cases where priority is relevant for a part as determined by the **PartType.CommitLevel** setting. In cases where priority is not relevant, the part's default priority (**PartType.DefaultPriority**) is used instead, and hence propagated to the receipt's dependent demands.

The following table shows how different CommitLevel settings affect the priority used on scheduled receipts by the Netting and Capable-to-Promise analytics.

Table 36.23: Impact of PartType.CommitLevel setting on priority used by analytics

CommitLevel	Priority in Netting	Priority in CTP
High	EffOrderPriority	EffOrderPriority
MediumExplodePriority	EffOrderPriority	EffOrderPriority
Medium	DefaultPriority	EffOrderPriority
Low	DefaultPriority	DefaultPriority

Priorities on planned orders

When there is insufficient supply available to satisfy demand for a part, a new planned order is created by Netting logic. Each planned order has a calculated reference that points to a value in the **OrderPriority** table.

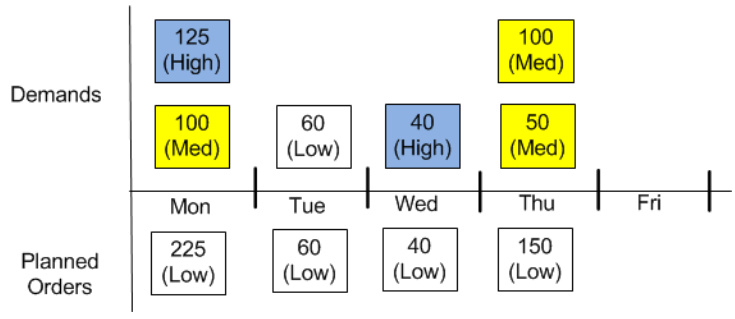
The priority reference set on a given planned order is determined by the part's **PartType.CommitLevel** setting and, potentially, by the priority on the demand that triggered the creation of the planned order.

CommitLevel of Low or Medium

If the **PartType.CommitLevel** setting for a part is "Low" or "Medium", then the priority on any planned order for it will be set to the part's default priority (as determined by the **PartType.DefaultPriority** reference).

For example, assume a part references a **PartType** record where the **CommitLevel** is set to "Medium" and the **DefaultPriority** references an **OrderPriority.Value** of "Low". In response to a given series of prioritized demands for a part, all required planned orders would be created with a priority of "Low" as shown in the following figure:

Figure 36.6 Planned orders when CommitLevel is Medium (or Low)



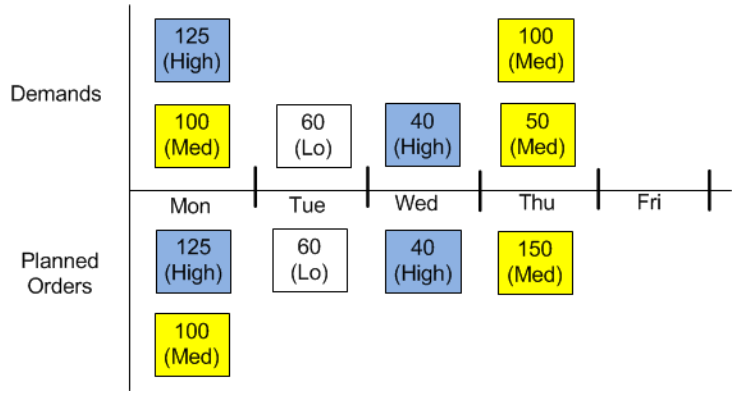
Instead, if priorities should be propagated from demands to supplies, a **CommitLevel** of either "MediumExplodePriority" or "High" should be used. With these settings, the priority on a new planned order is typically set to the priority on the demand that triggered the creation of that planned order, however these two settings differ in how demands are processed by Netting logic.

CommitLevel of High

If the **PartType.CommitLevel** is set to "High", higher priority demands are processed first. That is, demands are processed in priority sequence (lower **PlanningPriority** values first) and then in due date sequence within a given priority level. This means that if a part requires new supply for multiple demands of differing priorities on a given date, then a separate planned order is typically created for each of those priority levels.

For example, assume the same set of prioritized demands as seen in "Planned orders when CommitLevel is Medium (or Low)" on page 1790, but with a **CommitLevel** of "High". In this case, demands are processed in priority order and planned orders created with priorities as shown in the following illustration.

Figure 36.7 Planned orders when CommitLevel is High



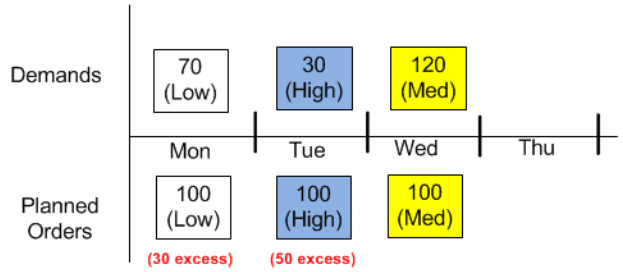
Note: The **CommitLevel** of "High" should be avoided on parts that either use lot sizing policies, are configured as the primary product in a coproduct/byproduct structure, or are configured as the primary component in a BOM-level part substitution relationship. In these situations, processing demands in priority order can sometimes lead to undesirable results such as unnecessary amounts of excess supply.

For example, the following figure shows an example where minimum lot-size requirements of 100 would lead to excess. Because demands are processed in priority order when CommitLevel is "High", the following occurs:

- The High priority demand for 30 on Tuesday is processed first and a high priority planned order for 100 is created to satisfy the minimum quantity.
- Next, the Medium priority demand for 120 on Wednesday is processed. Because there is available/excess supply of 70, only 50 units are needed but a medium priority planned order for 100 is created to satisfy the minimum quantity.
- Finally, the Low priority demand for 70 on Monday is processed, and a low priority planned order for 100 is created to satisfy the minimum quantity.

  Thus, 80 units of excess supply would be created in total.

Figure 36.8 Planned orders with CommitLevel of High and MinimumQty of 100

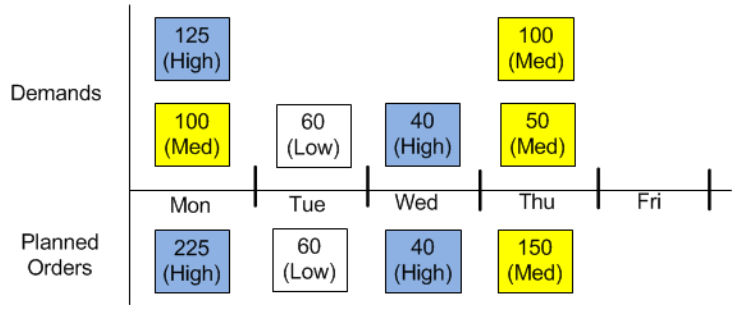


CommitLevel of MediumExplodePriority

The **PartType.CommitLevel** of "MediumExplodePriority" is similar to "High" in that it will propagate priority from demands to planned supplies for a part. However, with this setting demands are processed in due date sequence (instead of priority sequence).

Thus, if a part requires new supply for multiple demands of differing priorities on a given date, then a single planned order is typically created for those demands as shown in the following figure. Note that this is similar to the CommitLevel of "High" example shown in "Planned orders when CommitLevel is High" on page 1791 except that only one planned order is now generated on the Monday (for information on how priority is determined on planned orders covering demands of different priorities, see "Priority on supplies covering multiple demand priorities" on page 1793).

Figure 36.9 Planned orders with CommitLevel of "MediumExplodePriority"



Note: The **CommitLevel** of "MediumExplodePriority" is intended for parts that either use lot sizing policies, are configured as the primary product in a coproduct/byproduct structure, or are configured as the primary component in a BOM-level part substitution relationship. In these cases, use of the "MediumExplodePriority" setting can mitigate against issues, such as excess supply, which might arise when using the "High" setting.

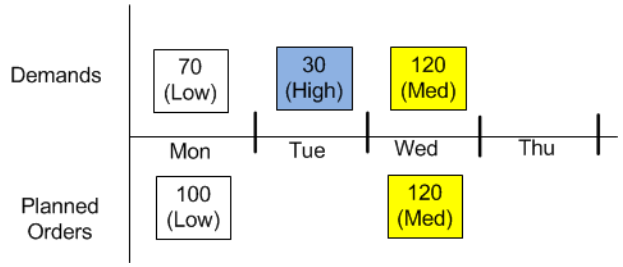
For example, the following figure, "Planned orders with CommitLevel of MediumExplodePriority and MinimumQty of 100" on page 1793,illustrates as situation where the "MediumExplodPriority" setting can prevent excess resulting from minimum lot-size requirements of 100. Because demands are processed in due date order when the **CommitLevel** is set to "MediumExplodePriority", the following occurs:

- The Low priority demand for 70 on Monday is processed first and a low priority planned order for 100 units is created to satisfy the minimum requirement.

- Next, the High priority demand for 30 on Tuesday is processed. However, no new supply is planned for it because there is available/excess supply from the earlier Low priority planned order.
- Finally, the Medium priority demand for 120 on Wednesday is processed and a new medium priority planned order is created.

  Thus, no excess supply is planned in this case (recall that with the "High" priority setting, 80 units of excess supply would be created for these same demands).

Figure 36.10 Planned orders with CommitLevel of MediumExplodePriority and MinimumQty of 100



However, because a Low priority supply is used to cover the High priority demand while a later Medium priority supply is used to cover the Medium priority demand, this might lead to undesirable results. For example, if constraints were used and there was limited constraint in the week, it would first be assigned to the Medium priority supply which is planned later than would allow the High priority demand to be satisfied on time.

Priority on supplies covering multiple demand priorities

Sometimes supplies created to cover demands of one priority level might end up being used to cover demands of other priority levels as well. For example, this might with occur if part if or when lot-sizing policies are in place.

To provide control over how priority is set on planned orders that cover demands of multiple priority levels during Netting, the **SupplyStatus.PrioritySource** setting can be used (for a given part source, this is referenced from the **PartSourceType.Status** field).

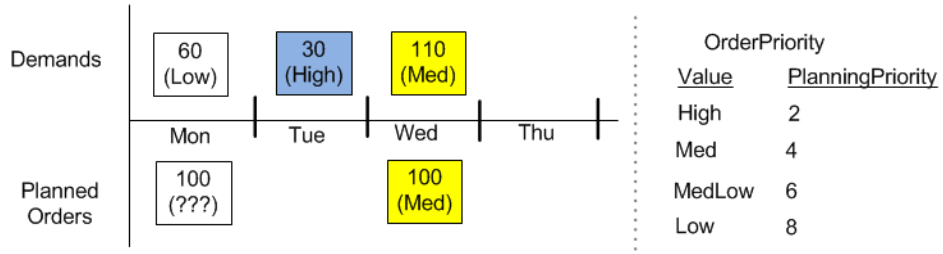
The **PrioritySource** field has the following four options:

- **FromInput**— Planned order priority is set to the priority on the demand that triggered the creation of the supply. This is the default setting and corresponds to how priorities were always set prior to Version 2016.2, Service Update 3.
- **Highest**— Planned order priority is set to the highest priority found on any demand that the supply covers (**OrderPriority** record with the lowest **PlanningPriority** value is used).
- **Average**— Planned order priority is set to the average of all priorities on demands covered by the supply. That is, the **OrderPriority.PlanningPriority** value found on all demands satisfied by the supply are summed together and divided by the total number of demands. The **OrderPriority** record with the closest **PlanningPriority** value equal or less than the calculated average is then used.
- **WeightedAverage**—Planned order priority is set to a weighted average of all priorities on demands covered by the supply. That is, the **OrderPriority.PlanningPriority** value found on each demand covered by the supply is multiplied by the demand quantity, these values are all then summed together and divided by the total quantity of demands covered by the supply. The

OrderPriority record with the closest **PlanningPriority** value equal or less than this weighted average is then used.

For example, assume the set of prioritized demands, generated supplies, and OrderPriority records shown in the following figure.

Figure 36.11 Planned orders with CommitLevel of MediumExplodePriority and MinimumQty of 100



Based on different PrioritySource settings, the Priority on the planned order due on Monday would be calculated as shown in the table:

PrioritySource	Priority calculation	Priority
FromInput	Set to priority on Monday demand that triggered the supply.	Low
Highest	Set to priority on Tuesday demand that has the highest planning priority of all demands the supply covers.	High
Average	8 + 2 + 4 = 14 / 3 = 4.67. Set to priority with closest PlanningPriority that is equal to or less than 4.67.	Med
WeightedAverage	((8 * 60) + (2 * 20) + (4 * 20)) = 600 / 100 = 6 Set to priority with closest PlanningPriority that is equal to or less than 6.	MedLow

Notes:

- Planned orders for parts with a **CommitLevel** of "Medium" or "Low" always use the default priority regardless of the **PrioritySource** setting.
- If a part uses days of supply logic, then a different group of settings are used to determine the priority on planned orders. For more information, see "Days of supply and planned order priority (alternate configuration)" on page 1371.

Constraint planning and usage

If the Netting analytic determines there is not enough constraint to plan supplies on time, RapidResponse assigns constraint to high-priority supplies to ensure they're delivered on time. Lower-priority supplies are planned late or allowed to overload the constraint, depending on the setting of **SupplyStatus.AllowLatePlan**.

If the Capable-To-Promise analytic determines there is not enough constraint to build all supplies on time, the available constraint is allocated to high-priority orders. Lower-priority orders are scheduled to receive their required constraint late, which makes those orders late.

Prioritized demand examples

This section describes several examples of how order priorities can be used to fill demands. Each example describes a business scenario and lists the relevant table and field values that must be set to support the goals of the scenario.

The following scenarios are presented:

- "Orders are not prioritized (example)" on page 1795
- "Basic order prioritization (example)" on page 1795
- "Maintaining current customer commitments with order drop-ins (example)" on page 1796
- "Controlling priorities for orders with the same DueDate (example)" on page 1797
- "Setting period effective priorities (example)" on page 1797

Orders are not prioritized (example)

Company X has been using its ERP system to fulfill customer orders by the date they are due, and has no desire to change. Orders will not be assigned priorities.

To meet the goals of this scenario, modify the ordered parts and prioritized orders to use the following settings.

Table 36.24: Settings for no order priorities

Change to	Table	Field	Value
Parts	PartType	CommitLevel	Low
Orders	OrderPriority	MaintainCommitments	N

These settings must be applied to all parts and orders. Orders are filled according to their due dates, and no new order ever takes priority over an older one with the same due date.

Basic order prioritization (example)

Company X has begun to see the value of allowing some orders to be filled before others, such as filling customer orders instead of rebuilding safety stock, filling customer orders before forecasts, or filling some customer orders before other customer orders.

Recently, a customer has requested to move shipment of a large order up by a week. Company X's master scheduler has been unable to fulfill this request by creating scenarios that shuffle the due dates of other orders, and decides to change the priority of the customer's order.

After running a simulation with the priority changed to **High**, RapidResponse shows the customer's order can be filled in time for the new due date, but two low-priority orders will be made late. Company X's management approves the schedule change.

To meet the goals of this scenario, modify the ordered parts and prioritized orders to use the following settings.

Table 36.25: Settings for basic order prioritization

Change to	Table	Field	Value
Parts	PartType	CommitLevel	Medium
All orders	OrderPriority	MaintainCommitments	N
Customer's order	OrderPriority	PlanningPriority	20
	IndependentDemand	OrderPriority	High

Note: You can use other values for **OrderPriority.PlanningPriority** and **IndependentDemand.OrderPriority**. The values used in this example are suggestions that indicate a high priority.

Maintaining current customer commitments with order drop-ins (example)

Company X has acquired a reputation for fulfilling customer orders very reliably, resulting in an increase in orders with a wide variety of due dates

Because the order's **DueDate** field is the primary criterion RapidResponse uses when planning orders, a new problem has sprung up for Company X. When a new high-priority order is added with an earlier **DueDate** than another high-priority order, the new order tends to take the stock that the older order needs. This is beginning to lead to shortages.

To prevent new orders from delaying existing orders, Company X has configured RapidResponse to maintain commitments. When a new order is added, the supply already allocated to other demands stays allocated, and the new order must wait for more supply before it can begin to get filled. The commitment to the customer is maintained, and the order is filled on time.

To meet the goals of this scenario, modify the ordered parts and prioritized orders to use the following settings.

Table 36.26: Settings to maintain commitments

Change to	Table	Field	Value
Parts	PartType	CommitLevel	High
All orders	OrderPriority	MaintainCommitments	Y
	IndependentDemand	TransactionSequence	
High-priority orders	OrderPriority	PlanningPriority	20
	IndependentDemand	OrderPriority	High

Notes:

- You can use other values for **OrderPriority.PlanningPriority** and **IndependentDemand.OrderPriority**. The values used in this example are suggestions that indicate a high priority.
- Orders with **OrderPriority.MaintainCommitments** set to **Y** are maintained regardless of their priority. However, use of this setting can potentially consume more memory, and it

should only be used for those orders that are truly committed to production (for example, the highest priority independent demands).

- The **IndependentDemand.TransactionSequence** field is used here for determining how orders are filled when more than one has the same priority. This is likely to occur if multiple orders with the same earlier **DueDate** are added. The order added first is filled first, the order added last is filled last.

Controlling priorities for orders with the same DueDate (example)

Company X has begun to do a lot of business with a major chain, BigMart, and always gives their orders a higher priority than any others. A number of orders, including one for BigMart are due today, but cannot be filled because of an equipment malfunction. Production should be back to normal tomorrow, and the late orders will ship then.

To ensure the late order for BigMart is filled before the other orders due tomorrow, Company X sets the **DemandType.AllotmentOrder** field of all the orders except the BigMart order so their values are higher than the RapidResponse default of **0**. This ensures that the BigMart order has supplies allocated to it ahead of other orders with the same due date and priority. The BigMart order is completed and shipped the next day, and is a day late.

Shifting the BigMart order back a day will most likely encroach on supplies reserved for other orders, so **OrderPriority.MaintainCommitments** must be set to **N**. If this is not done, supplies needed for the BigMart order might be assigned to other orders, making the BigMart order even later. The commitments to the other orders must be broken so the very high-priority BigMart order can be filled.

To meet the goals of this scenario, modify the ordered parts and non-BigMart orders to use the following settings.

Table 36.27: Settings to control priorities for orders with the same DueDate

Change to	Table	Field	Value
Parts	PartType	CommitLevel	Low
Orders	OrderPriority	MaintainCommitments	N
	DemandType	AllotmentOrder	10

Notes:

- you can use other values for **DemandType.AllotmentOrder**. The value used in this example is just a suggestion that indicates a low priority.
- **PartType.CommitLevel** is set to **Low** so the effective transaction sequence is not considered during order sequencing.

Setting period effective priorities (example)

Suppose requirements for three distinct levels of priority to be enforced during each weekly period, and also for any past due demands from a previous week to be treated as having a higher priority than all demands due in the current week. This could be achieved by specifying OrderPriority values such as the ones in the following table. The PlanningPriority field is used to indicate the relative priorities and the PeriodEffective field value of "Y" enables period effectivity on those priorities. As well, since weekly effective priorities are needed the part's **PlanningCalendars.AllocationCalendar** value should be set to

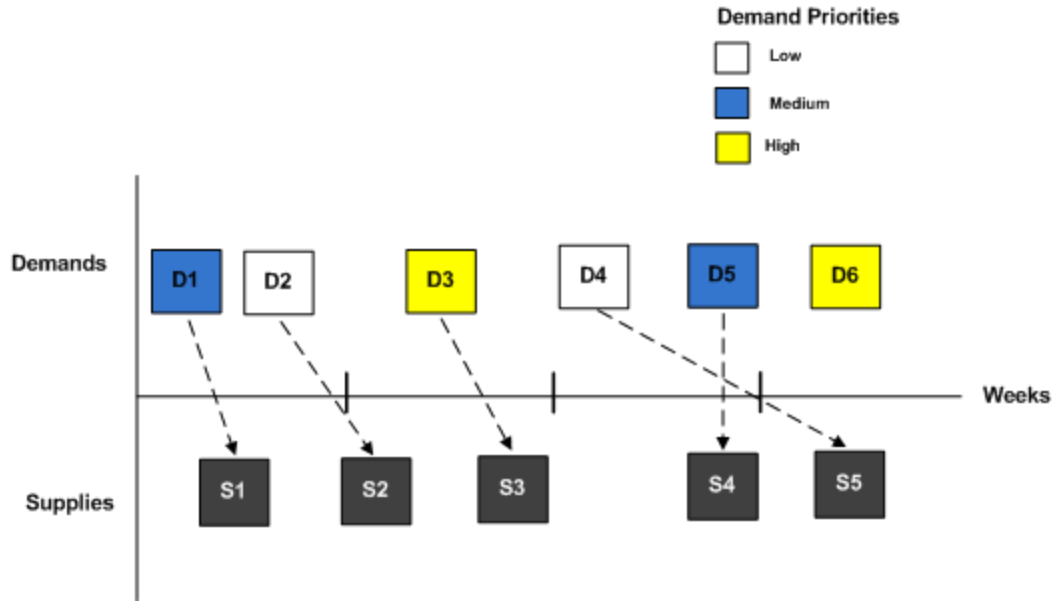
a weekly calendar.

Table 36.28: Period effective values in the OrderPriority table

Value	PlanningPriority	PeriodEffective
High	2	Y
Medium	5	Y
Low	7	Y

The following illustrations shows an example of how demand orders using the priority values in the table, "Period effective values in the OrderPriority table" on page 1798, might be satisfied by limited supply over a four week period. Note that in Week 1, there is only enough supply to satisfy the medium priority demand D1. This causes the low priority demand D2 to look for supply in Week 2 and, because this demand is past due, it is given the priority over the actual "High" priority demand D3. As a result, D2 is satisfied by the earlier supply S2, and D3 can only be satisfied by supply S3 thus making it late. Similarly, in Week 3 there is only enough supply to satisfy the medium priority demand D5, which causes the low priority demand D4 to look for supply in Week 4. There it is given priority over the high priority demand D6 and consumes the only available supply thus leaving D6 unsatisfied at the end of Week 4.

Figure 36.12 Period effective priorities



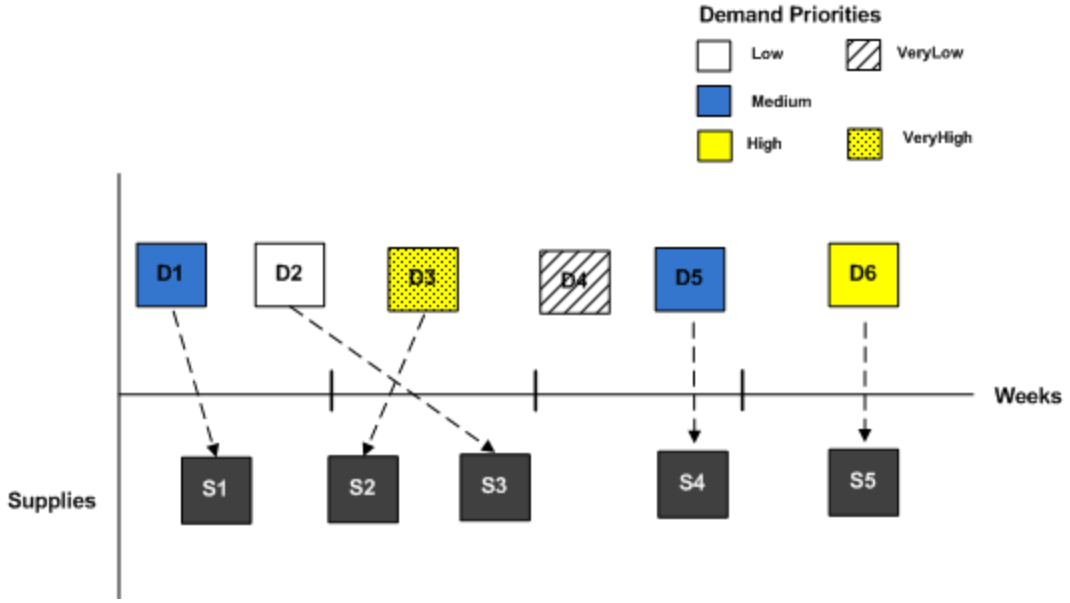
There might also be a requirement that even when using period effective priorities certain types of orders should be assigned the highest or lowest priority regardless of the period the demand falls in. For example, there might be a need to ensure orders from a particularly important customer are always satisfied first. This can be achieved by adding OrderPriority values such as the ones in the following table. The VeryHigh value is given the lowest PlanningPriority value and set as not period effective so that orders referencing it are always given the highest priority. Similarly, the VeryLow value is given the lowest PlanningPriority and set as not period effective so that orders referencing it are always given the lowest priority.

Table 36.29: Period effective and non-period effective values in the OrderPriority table

Value	PlanningPriority	PeriodEffective
VeryHigh	0	N
High	2	Y
Medium	5	Y
Low	7	Y
VeryLow	10	N

The following illustration shows an example of how demand orders using the priority values defined in the table, "Period effective and non-period effective values in the OrderPriority table" on page 1799, are satisfied by limited supply over a four week period. This example assumes the same set of supplies and demands as in the previous figure, except that demand D3 in Week 2 is now assigned a priority of VeryHigh and demand D4 in Week 3 is now assigned a priority of VeryLow. As a result, note that demand D3 is now given priority to receive the earlier supply S2 in Week 2, and the low priority demand D2 from period 1 receives the later supply in that period S2. In Week 3, the limited supply is only enough to satisfy the Medium priority demand D5 while the VeryLow priority demand D4 is unsatisfied. And because D4 is not period effective, it remains with the lowest priority in Week 4 even though it is past due, thus allowing the High priority demand D6 to be satisfied.

Figure 36.13 Period effective and non-period effective priorities



Prioritized supply examples

This section describes several examples of how order priorities can be used to plan supply usage and distribution. Each example describes a business scenario and lists the relevant table and field values that must be set to support the goals of the scenario.

The following scenarios are presented:

- "Protecting released orders" on page 1800
- "Overriding protected released orders" on page 1800
- "Scheduling a production release sequence" on page 1801
- "Setting default priorities to ensure replenishing safety stock is low priority" on page 1802

Protecting released orders

Company X wants to ensure that when a supply order is sent to the shop floor, the materials it needs are not taken by another order. Without the necessary materials, supply orders can pile up and lead to inefficient usage of manufacturing equipment, late orders, and inefficient use of time.

To set materials aside, Company X begins assigning priorities to the **ScheduledReceipt**s that they release for production. This ensures that the necessary materials are not used for any other order, and the supply is created on time.

To meet the goals of this scenario, modify the released orders to use the following settings.

Table 36.30: Settings for protecting released orders

Change to	Table	Field	Value
Released orders	OrderPriority	MaintainCommitments	N
	OrderPriority	PlanningPriority	20
	ScheduledReceipt	OrderPriority	High

Note: You can use other values for **OrderPriority.PlanningPriority** and **ScheduledReceipt.OrderPriority**. The values used in this example are suggestions that indicate a high priority.

Overriding protected released orders

Company X has set a policy of assigning low priorities to demands for forecast and supply stock and high priorities to customer orders. However, these priority settings do not automatically extend to any scheduled receipts created to fill supply shortages.

Company X's planner is reviewing the week's **ScheduledReceipts** and notices a late customer order. Two **ScheduledReceipts** with the same priorities are in the system for the same constrained supply and scarce materials, one scheduled for forecast, the other for the late customer order. The planner wants to ensure the constraint and materials are used for the customer order.

By setting the **OrderPriority.PlanningPriority** and **ScheduledReceipt.OrderPriority** fields, the planner sets the forecast **ScheduledReceipt** to a lower priority and the customer order **ScheduledReceipt** to a higher priority. The customer order is completed first, and the forecast order waits until more materials are available to complete it.

To meet the goals of this scenario, modify the orders to use the following settings.

Table 36.31: Settings to override protected released orders

Change to	Table	Field	Value
Customer order	ScheduledReceipt	OrderPriority	High
	OrderPriority	PlanningPriority	20
Forecast order	ScheduledReceipt	OrderPriority	Low
	OrderPriority	PlanningPriority	70

Note: You can use other values for **OrderPriority.PlanningPriority** and **ScheduledReceipt.OrderPriority**. The values used in this example are suggestions that indicate a high priority for the customer order and a low priority for the forecast order.

Scheduling a production release sequence

Company X wants to schedule production so parts with similar properties or common materials are grouped together. This reduces setup times and turns out more supply during the day. In addition, if a material shortage occurs, it likely occurs at the end of the day, giving the planner time to resolve it.

In order to group their production, Company X wants to ensure the orders at the beginning of the day have access to materials and capacity. They accomplish this by assigning every production order a different priority, and then filling those orders from highest to lowest priority.

To meet the goals of this scenario, modify the ordered parts and prioritized orders to use the following settings.

Table 36.32: Settings to schedule production

Change to	Table	Field	Value
Parts	PartType	CommitLevel	High
All orders	OrderPriority	MaintainCommitments	Y
First order	OrderPriority	PlanningPriority	20
	ScheduledReceipt	OrderPriority	High
Other orders	OrderPriority	PlanningPriority	Higher value (lower priority) than the previous order.
	ScheduledReceipt	OrderPriority	A unique value for each PlanningPriority value
	ScheduledReceipt	TransactionSequence	

Give the first order the lowest value in its **OrderPriority.PlanningPriority** field, and all other orders a value greater than the order before them. For example, if the first order is given an **OrderPriority.PlanningPriority** of **20**, the second order could be given an **OrderPriority.PlanningPriority** of **25**, the third **30**, the fourth **35**, and so on.

If unique priorities are not assigned, the **ScheduledReceipt.TransactionSequence** value determines the order in which they are produced.

Because every order has **OrderPriority.MaintainCommitments** set to **Y**, none of the other orders can steal materials or constraint from any other order. This ensures that an order, once begun, does not unexpectedly run out of materials, and allows the planner to substitute another set of orders if the supply of materials runs out.

Setting default priorities to ensure replenishing safety stock is low priority

Company X is facing a slight shortage of one of its most popular products. Orders keep coming in, and are filled as quickly as possible. In addition to the customer orders, a planned order to rebuild safety stock of this product is issued every time the supply runs out.

Company X wants to ensure that any new supply of the product is allocated to customer orders and not to safety stock. Changing the priority every time a new planned order is created is becoming tedious for the planner, so Company X finds an alternative.

By setting the **PartType.DefaultPriority** for the product to a much lower priority, Company X ensures that new planned orders created for the product are also given low priorities. Any customer orders that come in have higher priorities, and are filled first.

To meet the goals of this scenario, modify the ordered parts and prioritized orders to use the following settings.

Table 36.33: Settings to ensure customer orders are filled before safety stock

Change to	Table	Field	Value
Parts	PartType	DefaultPriority	Low
	PartType	CommitLevel	Medium
Orders	IndependentDemand	OrderPriority	Medium or High

By setting **PartType.CommitLevel** to **Medium**, any **PlannedOrder**s for the product will be given the priority value in the **PartType.DefaultPriority** field. Alternatively, any demands for the product are given the order's priority, which will be either **Medium** or **High**. Incoming customer demands are always higher-priority than automatically-generated planned orders, and are therefore filled first.

CHAPTER 37: Two-date planning

Typically, the capable-to-promise analytic plans supply to try and satisfy demands by their due date. On independent demands, this represents the date on which completion of the order has been promised to the customer. However, this date can sometimes be later than the date on which the customer originally requested for the order to be completed. To enable consideration of the customer's original request date in the supply allotment process, two-date planning can be used.

Two-date planning is optional priority-based logic that runs after an initial standard allotment of supply to demand. It attempts to move the available date on certain independent demands closer to their original request date, thereby improving customer satisfaction. For example, it might be possible to make a high-priority actual demand available at its request date by rescheduling supply or reallotting certain component supplies from a low-priority forecast order.

In this chapter, you'll learn about:

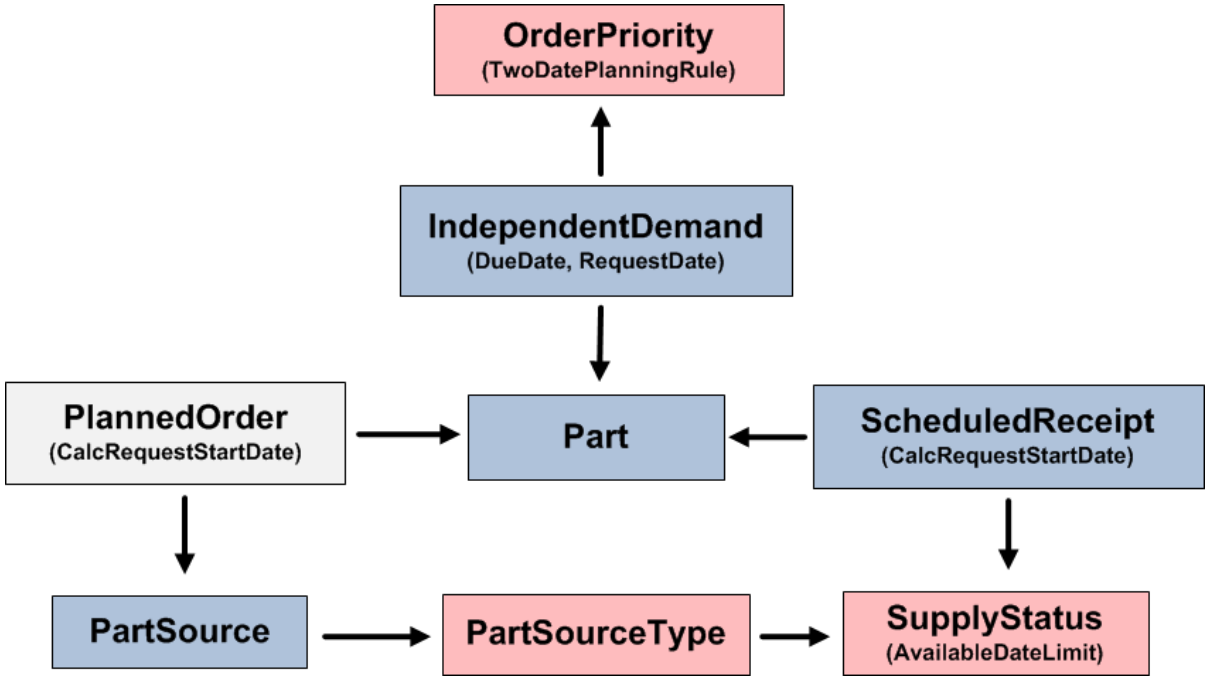
Two-date planning configuration **1803**
Two-date planning calculations **1810**
Limitations on other analytics **1814**

Note: For detailed information about the standard supply allotment calculations performed by the Capable-to-Promise analytic, see "Allocation of supply to demand" on page 1458.

Two-date planning configuration

Configuration of two-date planning logic involves providing the original customer request date on the appropriate demands, enabling two-date planning logic at the order priority level, and ensuring that the required sources are able to provide supply early enough to meet the customer request date. The relationship between the main tables involved in two-date planning logic is shown in the following illustration.

Figure 37.1 Main tables supporting two-date planning



The remainder of this section describes the basic tasks involved in setting up two-date planning logic in RapidResponse. The following configuration points are discussed:

- "Provide request date on independent demands" on page 1804
- "Enable two-date planning by priority level" on page 1805
- "Ensure supply sources can plan to request dates" on page 1806
- "Apply safety lead time at request date (optional)" on page 1807

Provide request date on independent demands

All independent demands are first planned to their due date as provided in **IndependentDemand.DueDate** (if **DueDate** is not populated, the demand is effectively ignored and supplies are not allotted to it)**.** This field holds the date on which order completion has been committed to the customer and on which it should therefore be ready to put into stock.

If a given independent demand should also be used in two-date planning logic, then **IndependentDemand.RequestDate** should be populated. This field holds the date on which the customer originally requested for the order to be completed to stock. If the demand does not need to be planned to the customer's request date, then the **RequestDate** field can be left blank (or populated if it is useful for reporting purposes).

In addition, all independent demands that should be planned to their **RequestDate** should also reference an **OrderPriority** record that has been enabled for two-date planning.s

Enable two-date planning by priority level

Two-date planning logic is enabled at the order priority level using the **OrderPriority.TwoDatePlanningRule** control field. This means that all independent demands of a given priority level are enabled or disabled for two-date planning as a group.

The **TwoDatePlanningRule** setting controls whether independent demands of a given priority level should be planned to their **RequestDate** where possible**.** This field has settings as described below:

- **Use**—independent demands of this priority level are enabled for two-date planning. After the capable-to-promise analytic initially allots supplies to all independent demands in order to try and meet their **DueDate**, two-date planning calculations perform a second round of allotments that attempt to move the available date on demands referencing this setting closer to their original **RequestDate**. This can involve rescheduling or reallotting supplies as compared to the original planning results, but such changes are generally only made if they do not adversely affect the availability of other demands as calculated during the initial round of supply allotments. Note however that lower priority demands which are not enabled for, nor protected against, two-date planning logic might be made worse by attempts to meet the **RequestDate** on independent demands using this setting.

  Two date-planning can also be enabled with the **UseWithHardSafetyLeadTime** and **UseWithSoftSafetyLeadTime** settings. However, these settings are intended for cases where safety lead time should be applied to request dates and/or where safety lead time should only be considered for a demand if its use won't make other demands worse off. For more information on these settings, see "Apply safety lead time at request date (optional)" on page 1807.
- **Protect**—independent demands of this priority level are not enabled for two-date planning. The capable-to-promise analytic allots supplies to independent demands using this setting to try and meet their **DueDate**, but these demands are generally ignored by two-date planning calculations. However, the initial available dates achieved on demands using this setting are protected against two-date planning logic. Thus, independent demands enabled for two-date planning will only be moved closer to their initial customer requested date if this can be accomplished without adversely impacting the initial **AvailableDate** achieved on demands referencing this setting.
- **Ignore**— independent demands of this priority level are not enabled for two-date planning. The capable-to-promise analytic initially allots supplies to independent demands using this setting to try and meet their **DueDate**, but these demands are generally ignored by two-date planning calculations. Additionally, the initial available date achieved on a demand using this setting can be made worse off if it results in moving a demand that both belongs to a higher priority level and is enabled for two-date planning closer to its request date. This is the default setting.

Typically, the **TwoDatePlanningRule** setting of "Use" should be associated with higher priorities (lower **PlanningPriority** values), while "Protect" and "Ignore" are then associated with the next lower levels of priorities as shown in the following table.

Table 37.1: Sample values in OrderPriority table

Value	Plannning Priority	TwoDatePlanningRule
High	0	Use
MediumHigh	2	Use
Medium	5	Protect
Low	10	Ignore

Additionally note that, within a given control set, a **TwoDatePriorityLimit** is calculated and reports the lowest priority (highest **PlanningPriority** value) from any **OrderPriority** record having a **TwoDatePlanningRule** of either "Use" or "Protect". All priority levels above this level are automatically protected during two-date planning calculations, and will not have their availabilities made worse when attempting to meet the **RequestDate** on higher priority demands. This is regardless of the **TwoDatePlanningRule** setting on those higher priorities (if set to "Ignore", they will be treated as if they were set to "Protect" instead).

For example, assume the following set of priority values. In this case, the "MediumHigh" priority level is automatically protected during two-date planning and would not have its demand availabilities impacted by attempts to satisfy the **RequestDate** on "High" priority demands. Therefore, in this example, only "Low" priority demands would potentially have their availabilities made worse off by two-date planning logic.

Table 37.2: Sample values in OrderPriority table

Value	Planning Priority	TwoDatePlanningRule	TwoDatePriorityLimit
High	0	Use	5
MediumHigh	2	Ignore	5
Medium	5	Protect	5
Low	10	Ignore	5

Note: Parts that have demands enabled for two-date planning should typically use a **PartType.ATPCTPRule** setting of "Latest" in order to allow pulling supplies in to the request date while reducing inventory carrying cost. If using the default **ATPCTPRule** of "Earliest", then results might not be as expected since this setting will try to ensure demands are satisfied with later supplies (assuming it doesn't impact the ability to satisfy the demand on time). This can leave earlier supply as excess rather than using it to try and meet customer request dates.

Ensure supply sources can plan to request dates

All scheduled receipts and planned orders have a **CalcRequestStartDate** field. This calculated field indicates the date on which work on a supply should commence in order to achieve the **RequestDate** (if provided) on the independent demand(s) it ultimately satisfies.

For supplies of the end-item assembly itself, the **CalcRequestStartDate** is derived from the earliest **RequestDate** found on any independent demand that the supply satisfies and offset by the assembly's lead time. For supplies of component parts used in the assembly, the **CalcRequestStartDate** is based on the earliest **RequestDate** found on any independent demand which ultimately uses the component supply, and then offset by lead times through each level of the product structure down to the component.

Thus, in order to potentially achieve the customer request date, all relevant supply sources need to be configured to provide supplies by their **CalcRequestStartDate**. This configuration is made through the **SupplyStatus.AvailableDateLimit** control field. This field has four options which, subject to material and constraint availability and limitations, determine the earliest date on which supply can be made available as described below:

- **CalcRequestStart** (recommended)—supply is built to **CalcRequestStartDate**. As material and constraint availability permits, this setting allows supply to be built that can ultimately satisfy the customer **RequestDate** specified on end-item independent demands.

  The **CalcRequestStart** setting should typically be applied to all scheduled receipts and/or part sources that are used towards satisfying independent demands enabled for two-date planning (that is, those referencing a **TwoDatePlanningLimit** setting of "Use", "UseWithHardSafetyLeadTime", or "UseWithSoftSafetyLeadTime"). Thus, such scheduled receipts should have **SupplyStatus.AvailableDateLimit** set to "CalcRequestStart" and planned orders shouldhave **PartSource.Type.Planned OrderSupplyStatus.AvailableDateLimit** set to "CalcRequestStart". Note that this recommendation applies to both the supplies/sources used to satisfy the end-item independent demands as well as those that provide any component parts used in those end-items. Additionally, If the calculated start date is earlier than the date determined from this setting, then it becomes the available date limit instead ("CalcStart" setting is used).
- **CalcStart** (default)—supply is built based on its calculated start date (**CalcStartDate** for scheduled receipts and **StartDate** for planned orders). If this setting is used in conjunction with two-date planning, then supplies would typically not be planned early enough to satisfy the original customer request dates.
- **Due** —supply is built so that the earliest it can be completed is its due date (even if rescheduled earlier). If this setting is used in conjunction with two-date planning, then supplies would typically not be planned early enough to satisfy the original customer request dates.
- **Now**—supply is built as soon as it can be based on material and constraint availability. If this setting is used in conjunction with two-date planning and there is sufficient material and constraint available, then supplies would typically be able to satisfy the customer request dates. However, the supply ends up being made available earlier than it is actually needed.

Notes:

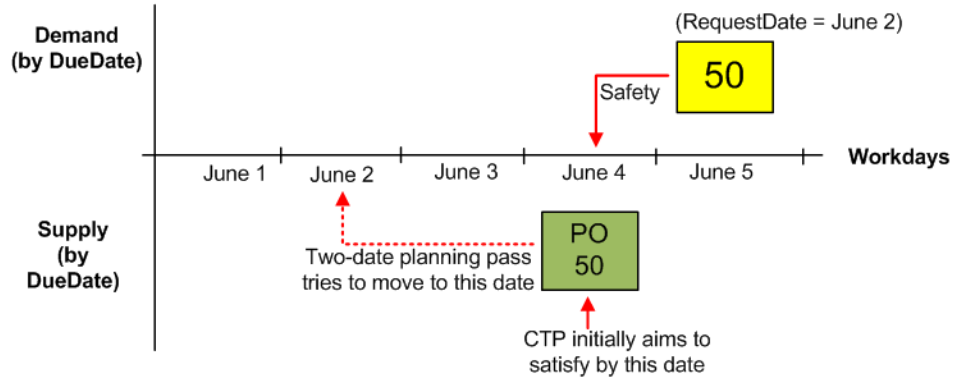
- When push-planning logic (distribution planning) is used, some pushed supplies might end up with available dates that are later than the limit determined by the AvailableDateLimit setting.
- For more information on supply availability limits, see "AvailableDateLimit" on page 1666.

Apply safety lead time at request date (optional)

Safety lead time is an optional lead time component that can be applied against the demand due date when planning supply. It serves as a buffer meant to guard against fluctuations in demand and supply. For example, Netting can plan supply to meet the demand due date less safety lead time and CTP logic will then attempt to satisfy the demand at that adjusted demand date.

Optionally, the setting in the **TwoDatePlanningRule** field can be used to influence where and when safety lead time is applied for demands of a given priority level. When this field is set to "Use", Netting will plan supplies at the demand due date less safety lead time and CTP will try to satisfy demands at the same adjusted date as shown in the following figure(which assumes **SafetyLeadTme** for the part is 1 day). Note that safety lead time is also handled identically on demands belonging to priorities where the **TwoDatePlanningRule** setting is either "Protect" or "Ignore" (but, after the intial supply allotment, two-date planning logic does not subsequently try to move such demands closer to their request date).

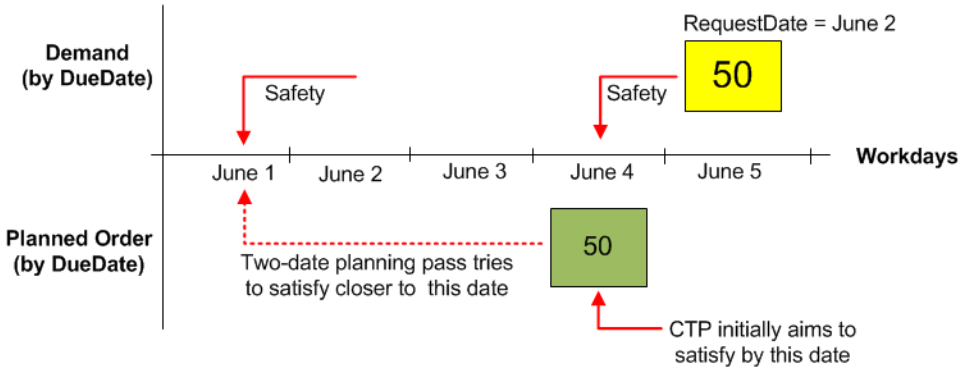
Figure 37.2 TwoDatePlanningRule set to Use (SafetyLeadTime = 1 day)



The **TwoDatePlanningRule** field also contains two other settings meant specifically for cases where safety lead time should either be applied to the request date or only applied if doing so doesn't make other demands of a given priority level worse off. These settings are "UseWithHardSafetyLeadTime" and "UseWithSoftSafetyLeadTime".

The "UseWithHardSafetyLeadTime" setting treats safety lead time similar to the "Use" setting, however it also adjusts the original request date by safety lead time. With this setting, Netting will again plan supplies at the demand due date less safety lead time and CTP will try to satisfy demands at that demand adjusted date. However, in this case after the initial allotment of supply to demand, two-date planning logic will try to satisfy the demand closer to its request date less safety lead time, as shown in the following figure. Thus with this setting, meeting the safety lead-time adjusted due date might make other lower priority demands worse off. However, demands are only moved closer to their safety lead-time adjusted request date if this can be done without making any other demandw worse off (excepting lower priority demands that use the **TwoDatePlanningRule** setting of "Ignore").

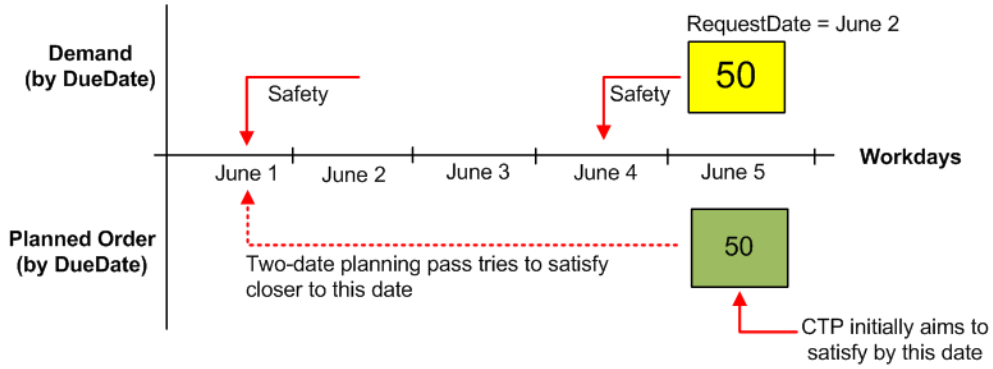
Figure 37.3 TwoDatePlanningRule set to UseWithHardSafetyLeadTIme (SafetyLeadTIme = 1 day)



Instead, if using the "UseWithSoftSafetyLeadTime" setting, safety lead time is typically only ever applied if doing so won't make other demands worse off. With this setting, Netting will plan supplies at the demand due date (without applying safety lead time) and CTP will try to satisfy that same demand due date. Where possible, after the initial allotment of supply to demand, two-date planning logic will then try to satisfy the demand closer to the request date less safety lead time as shown in the following figures. Thus with this setting, safety lead time would only be applied to a demand if doing so wouldn't

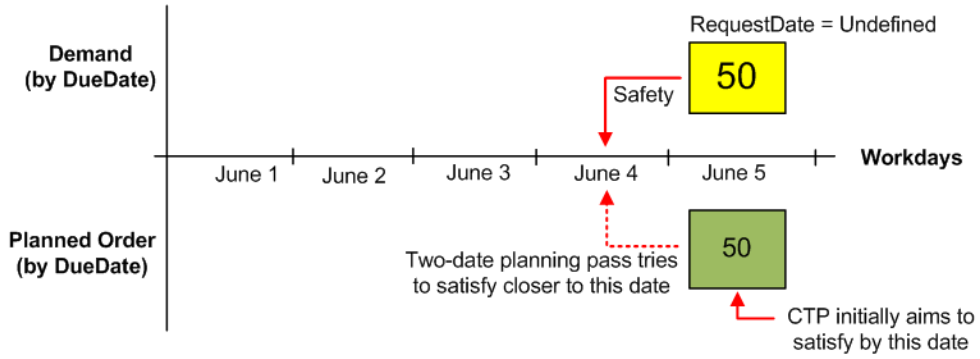
make any other demands worse off (excepting those of a lower priority that have their **TwoDatePlanningRule** set to "Ignore".

Figure 37.4 TwoDatePlanningRule set to UseWithSoftSafetyLeadTime (SafetyLeadTime = 1 day)



Also if either of the "UseWithSoftSafetyLeadTime" or "UseWithHardSafetyLeadTime" settings are referenced by an independent demand with an undefined **RequestDate**, then two-date planning logic will use the safety-lead time adjusted demand due date instead as shown in the following figure. This allow the determination of whether to use "soft" safety lead time to be made in cases where request dates are not provided.

Figure 37.5 TwoDatePlanningRule set to UseWithSoftSafetyLeadTime (SafetyLeadTIme = 1 day)



Notes:

- The ability to apply safety lead time at the request date and use two-date planning logic only applies to safety lead time defined at the part level (on either the **Part** or **SafetyLeadTimeProfile** tables). The logic described in this section does not consider any safety lead time defined on the **PartSource** table. For more information about the basic tables and fields used to setup safety lead time, see "Lead time elements" on page 1308.
- If safety lead time with two-date planning logic matters on components below the top-level parts that have independent demands, then **Priority** and hence the specified **TwoDatePlanningRule** needs to be propagated down the product structure. To do so, ensure the appropriate parts use a **PartType.CommitLevel** setting of "High" or "MediumExplodePriority".

- If a part has demands that reference different **Priority** records, all having the same **PlanningPriority** value but some using the **TwoDatePlanningRule** setting of "UseWithSoftSafetyLeadTime" and others using "UseWithHardSafetyLeadTime", the demands referencing the "UseWithHardSafetyLeadTIme" setting are processed first. This means that within a given priority level, the "UseWithHardSafetyLead" demands can potentially be satisfied by their safety lead time adjusted due date even if doing so means making the "UseWithSoftSafetyLeadTime" demands miss their due date altogether. For an example of this and other safety lead time cases, see

Two-date planning calculations

Typically, when the capable-to-promise analytic allots supplies to demands it tries to satisfy all independent demands by their **DueDate** (representing the date on which order completion, and hence shipment of the order, has been committed to the customer). In cases where there is insufficient material or constraint availability to satisfy all demands on time, other factors such as order priority are used to determine which demands receive limited available supplies.

When two-date planning logic is enabled, all independent demands are still initially planned to their provided **DueDate** (less applicable safety lead time), and the demand availabilities achieved by these initial allotments are maintained internally by RapidResponse. However, subsequent attempts are then made to satisfy some independent demands closer to the date on which the customer originally requested completion of the order. This two-date planning logic is implemented at two different stages in Capable-to-Promise calculations.

The first stage is enabled by a **SupplyStatus.AvailableDateLimit** setting of "CalcRequestStart" and occurs during material and constraint availability calculations prior to the actual allotment of supply to demand. During this stage, supplies that reference a **SupplyStatus.AvailableDateLimit** of "CalcRequestStart" will, as possible, be planned to start at their **CalcRequestStartDate** value. **CalcRequestStartDate** is a calculated date field on **ScheduledReceipt**, **PlannedOrder**, and **SubstituteSupply** records indicating the date the supply should start in order to meet the **RequestDate** on the **IndependentDemand** record it ultimately satisfies. It is based on the driver independent demand's original **RequestDate** and then offset by lead time at each level throughout the product structure (or set directly to **RequestDate** on supplies satisfying the independent demands themselves).

The second stage is enabled by an **OrderPriority.TwoDatePlanningRule** setting of Use" and occurs after the initial allotment of supply to demand. During this stage, a second round of allotments, attempts are made to move independent demands having an **OrderPriority.TwoDatePlanningRule** setting of "Use" closer to their **RequestDate** (if populated). For example, this might involve rescheduling component supplies earlier or reallotting supplies that were initially given to other demands.

Note that independent demands with a **TwoDatePlanningRule** setting of "Use" will only have their availability moved closer to the provided **RequestDate** if this can be done without adversely impacting the availability of any other independent demands that are either of the same or a higher priority level, or that reference an **OrderPriority.TwoDatePlanningRule** setting of either "Use" or "Protect". That is, if such demands were seen as on time during the initial round of supply allotments, they cannot be made late, and if they were seen as late during the initial round of allotments, they cannot be made later.

However, independent demands that are of a lower priority level and which have an **OrderPriority.TwoDatePlanningRule** set of "Ignore" might be made worse off during the round of allotments that plans select demands to their **RequestDate**. For example, a "High" priority demand having a **TwoDatePlanningRule** of "Use" might take supplies that were initially allotted to "Low" priority

demand having a **TwoDatePlanningRule** of "Ignore", even if this means making that lower priority demand late.

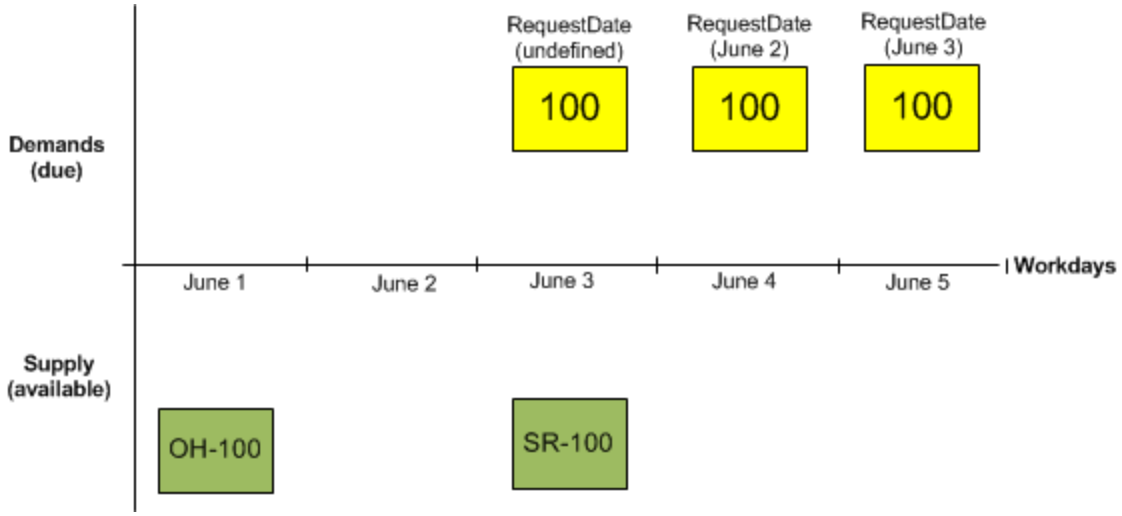
Examples

This section shows simple examples of how two-date planning can impact the standard allotment of supply to both prioritized and non-prioritized demands.

Example 1: Single priority level

Assume three independent demands due for a part, with request dates defined on some of these demands, and enough firm input supply available to satisfy two of these demands as shown in the following illustration. Also assume that total lead time to source a new planned order for the part is four (4) days.

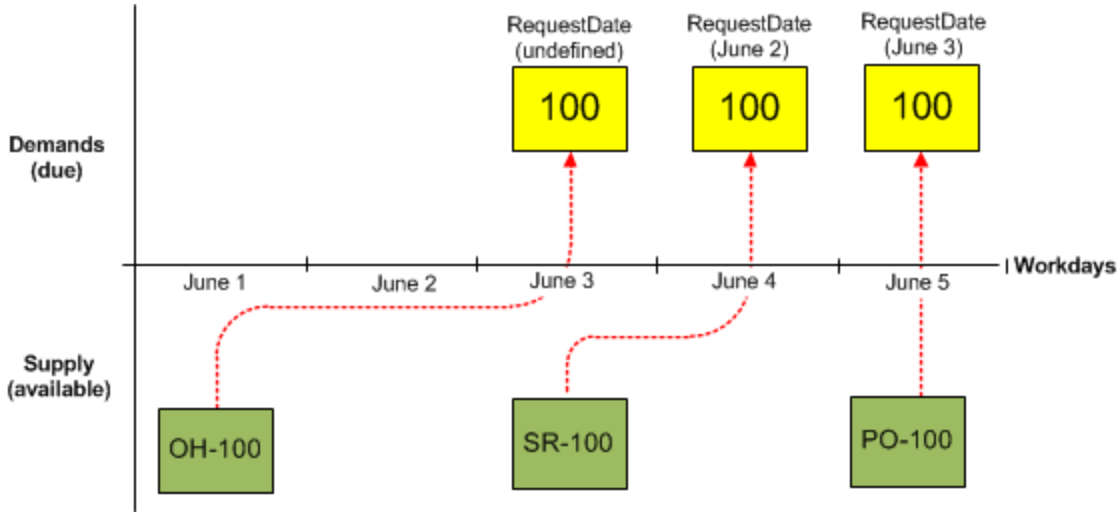
Figure 37.6 Input demands and supplies



Further assume that all demands belong to the same priority level and that the referenced **OrderPriority** record has a **TwoDatePlanningRule** of "Use".

During the initial round of allotments, all demands are planned to their due date as shown in the following illustration. Note that the demand on June 3rd receives the onhand supply and is available on June 1, while the demand on June 4 receives the firm scheduled receipt and is available on June 3 (both demands are early), while a new planned order is required for the demand on June 5 and available at its due date.

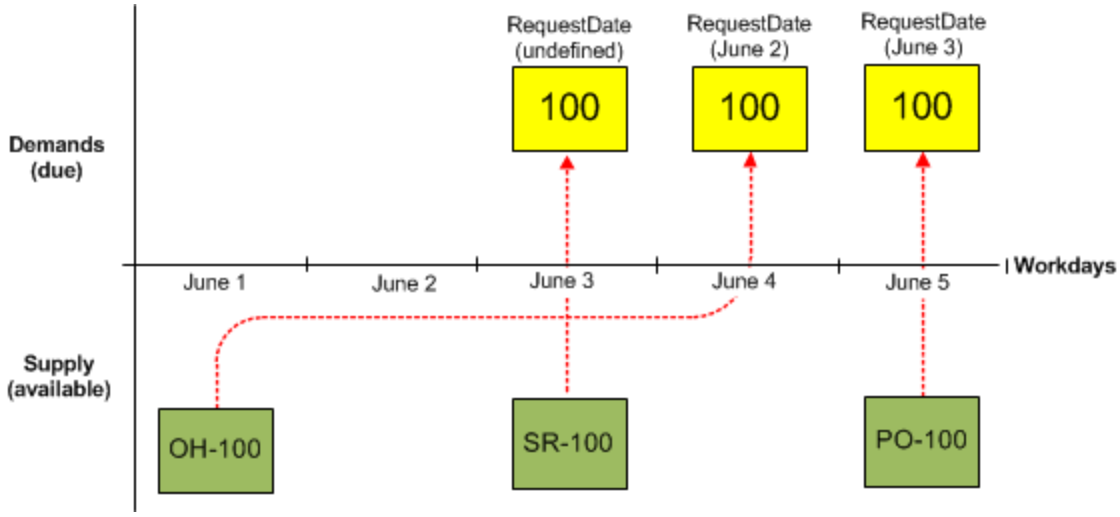
Figure 37.7 Initial planning to DueDate



During the second round of allotments, the capable-to-promise analytic now looks for opportunities to plan closer to the **RequestDate** values defined on the independent demands due on June 2 and June 3. The results of these allotments are shown in the following illustration.

Note that the demand due on June 4 can now be moved towards, and actually meet, its request date by taking the onhand supply originally allotted to the demand due on June 3. This is allowable because the demand due on June 3 can still be satisfied on time by using the scheduled receipt due on June 3 (that is, it is no worse off than it was in the initial allotments). Note also that the demand due on June 5 could meet its request date if it were to be allotted the June 3 scheduled receipt, but this is not allowable as it would force the June 3 demand to be satisfied by a planned order and thereby made late (thus, being made worse off than it was in the initial allotments).

Figure 37.8 Subsequent planning to RequestDate diagram

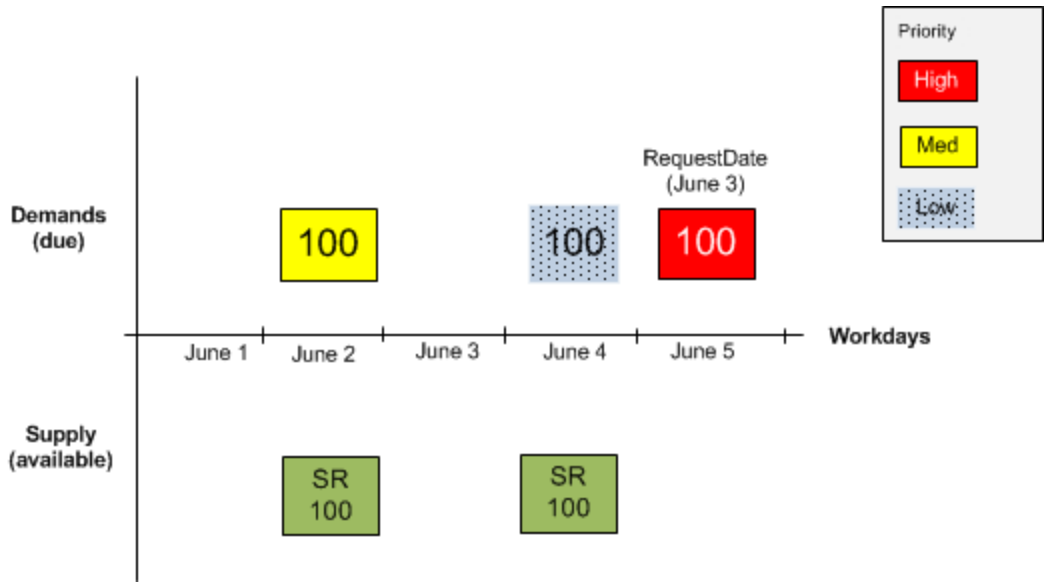


Example 2: Multiple priority levels

Assume three independent demands of different priorities due for a part, with a request date defined

on the highest priority of these demands, and two firm scheduled receipts available to satisfy demand as shown in the following illustration. Also assume that total lead time to source a new planned order for the part is four (4) days.

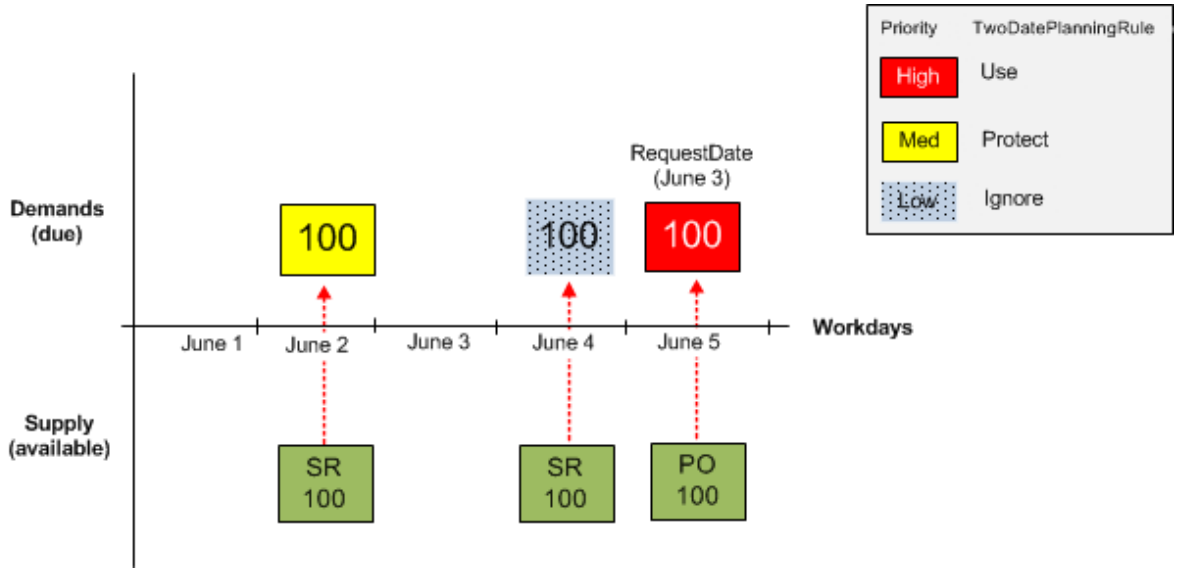
Figure 37.9 Input demands and supplies diagram



Further assume that each of the three order priority levels used by the demands reference a different **TwoDatePlanningRule** setting as shown in the following illustration. In the initial round of allotments, this setting and any provided **RequestDate** values are ignored, as all demands are planned to their due date.

That is, the demands on June 2 and June 4 are satisfied by the existing scheduled receipts available on those same dates, and a new planned order is created to meet the demand due on June 5. As a result, all demands are seen as on time.

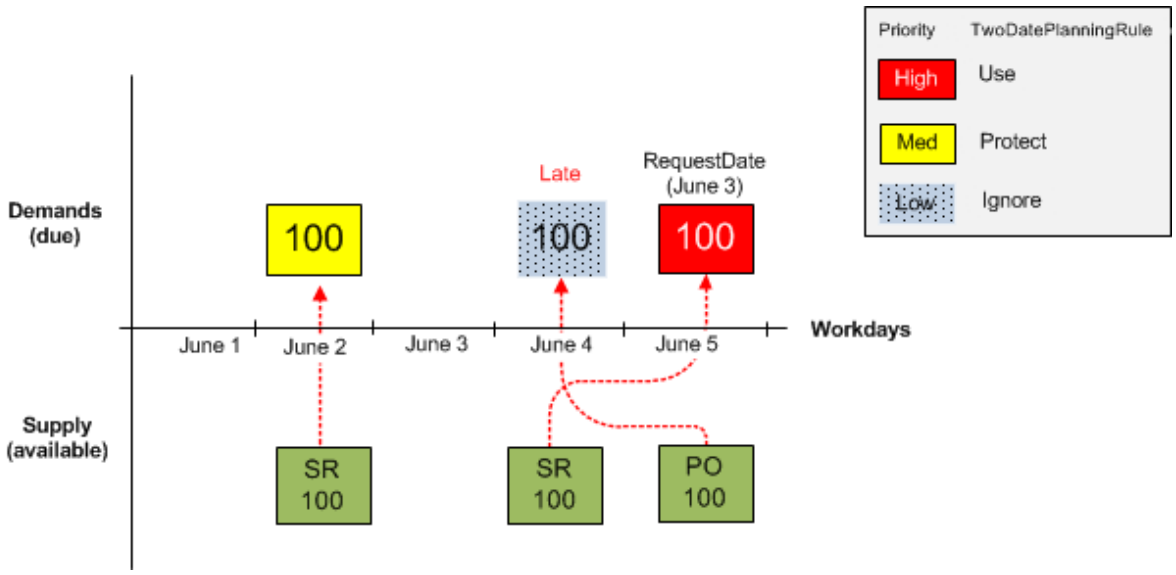
Figure 37.10 Initial planning to DueDate



During the second round of allotments, RapidResponse attempts to move the "High" priority demand with a **TwoDatePlanningRule** setting of "Use" to its request date of June 3. As new orders cannot be planned any earlier than June 5, improving availability to the request date can only be achieved by using the existing input supply. In this case, the scheduled receipt available June 2 could actually satisfy the "High" priority demand by its request date, however this would make the "Medium" priority demand late (worse off than it was during the initial allotments) and this demand is protected by a **TwoDatePlanningRule** of "Protect".

Instead, the scheduled receipt due on June 4, which was originally given to the "Low" priority demand due on June 4, is allotted to the "High" priority demand due on June 5. As a result, the "Low" priority demand due on June 4 can only be satisfied by a new planned order. When compared to the initial allotments, this has the effect of moving the "High" priority demand one day closer to its request date while ultimately making the "Low" priority demand available late as shown in the following illustration.

Figure 37.11 Subsequent planning to RequestDate



Limitations on other analytics

A small number of other analytics or calculations might be disabled, or not work as expected, for parts with demands that are enabled for two-date planning. The following table outlines other calculations having potential compatibility issues with two-date planning logic:

Table 37.3: Two-date planning limitations on other analytics

Analytic / Calculation	Limitation
Expiry	Two-date planning is automatically disabled for any parts subject to expiry. That is, parts where **PartType.ExpiryRule** is set to a value other than "Ignore" will always be planned to their due date regardless of the **TwoDatePlanningRule** setting applicable to a given demand.
Fair Share Constraint	The use of fair share constraint logic (a setting of "FairShare" in **OrderPriority.ConstraintAllocationRule**) should be avoided on demands/priorities that are enabled for two-date planning. Fair share constraint logic enables the Netting analytic to share limited constraint between demand's based on their due dates. However, if using two-date planning and the recommended **AvailableDateLimit** setting of "CalcRequestStart", then the CTP analytic might schedule those constraints using different dates resulting in undesirable planning results.
Fair/Equal Share Allocation	Two-date planning is automatically disabled on order priorities set up for the fair sharing of supplies. That is, demands where **OrderPriority.AllocationRule** is set to either "EqualShare" or "FairShare" will only be planned to their due date regardless of their **TwoDatePlanningRule** setting.
Feature BOM	Two-date planning logic is not supported on configurable end-items (parts having a **PartType.FeatureBOMRule** setting of either "Normal" or "HighVolume"). For each priority level, such parts only have one planned order generated to satisfy all demands with a given due date (no consideration is given to the request date).
Schedule Priority	The ability to ensure that higher priority orders consume constraint as close to their due date as possible during any pre-planning is not supported when two-date planning logic is enabled. That is, **OrderPriority.SchedulePriority** is treated as being set to "0".
Lot Sizing	Lot-sizing rules are supported when two-date planning is enabled, however if a supply is used to satisfy multiple demands with different request dates, the earliest of those is chosen and propagated down the structure.
Days of Supply	Days of supply logic is supported when two-date planning is enabled, however if a supply is used to satisfy multiple demands with different request dates, the earliest of those is chosen and propagated down the structure.
Forecast consumption	When using two-date planning, an option is available to let independent demands consume forecast relative to their original request date rather than their due date (if **DemandStatus.ForecastConsumptionDateRule** is set to "RequestDate"). Note however that this setting only applies to independent demands. Any dependent demands generated as a result of satisfying the independent demand will always consume forecast relative to their due date (or data date) even if the driver independent demand used a **ForecastConsumptionDateRule** of "RequestDate".
Campaign Planning	Two-date planning logic is compatible with campaign planning. Note, however, that each batch in a campaign will have its **CalcRequestStartDate** based on the earliest request date found on a demand it satisfies with **CalcRequestStartDate** values likely differing across batches in the campaign.

CHAPTER 38: Attribute-based planning

Attributes are custom data model elements used to identify particular characteristics of supplies or demands on a given table. Rules can then be constructed that use attribute values to control or refine certain analytic calculations.

This chapter discusses the analytics that support attributes, outlines the steps involved in creating and configuring attributes, and describes the impact they can have on analytic calculations. Finally, examples are shown of sample attributes being configured along with the impact they would have on calculated output in RapidResponse.

In this chapter, you'll learn about:

Supported analytics **1817**
Configuring attributes **1818**
Attribute calculations **1827**
Attribute-based planning examples **1832**

Supported analytics

Attributes can be enabled to provide refinements over certain analytic calculations in RapidResponse as outlined in the following table. For more details on how attributes impact these calculations, refer to the subsequent sections in this chapter.

Table 38.1: Analytics that can utilize attributes

Analytic Calculation	Description / Attribute Impact
Supply allotment	When RapidResponse allots supplies of a part to the demands where it is required, factors such as due date and priority are typically used to determine which existing supplies and/or part sources are used to satisfy particular demands. When attributes are enabled, rules can be defined to further limit which supplies of a part can be allotted to particular demands. This allows for rule-based partitioning of the demands and supplies for a single part and ensures that supplies are created and/or allotted based on the attributes associated with each. For example, one customer might only accept supply of an end item if it was assembled at a particular location, while another customer might only accept supply of that item if it contains components sourced from a list of approved suppliers.
Forecast consumption	Forecast for a part is typically represented by demand records of type "SalesForecast" as well as consensus forecast records generated during sales and operations planning process, and can be consumed/reduced by demands of type "SalesActual" for that part (optionally, pooling logic can be used to refine consumption). When attributes are enabled, rules can be defined to limit which actual demands for a part can be used to consume specific forecast for that part. For example, part forecast from a specific customer or region might only be eligible to be consumed by actual demands from that same customer or region. Further configuration is available to define how any common (non-attribute specific) forecast is consumed.
Forecast adjustment	Adjustments and overrides made to the consensus forecast generated during the sales and operations process are typically made against the total consensus quantity for a given part customer and forecast date, and any adjustments to be applied against multiple underlying forecast detail records are made in a sequence determined by RapidResponse. When attributes are enabled, rules can be defined specifying which of the underlying forecast detail records the adjustment/override should be applied to. For example, a forecast adjustment might only be applied within a particular forecast stream (such as, the marketing forecast) that contributes to the consensus forecast.

Configuring attributes

This section outlines the basic steps in configuring attribute-based planning in RapidResponse. This involves ensuring the required control settings are defined, creating attributes, enabling and defining them for usage on select tables through the use of attributes fields, and then creating rules that specify how specific attributes and their values should be interpreted and used in supply allotment, forecast consumption, and forecast adjustment calculations.

The following steps are discussed:

- "Configure control table settings" on page 1818
- "Create attributes" on page 1820
- "Create and define attribute fields" on page 1821
- "Define attribute rules" on page 1825

Configure control table settings

If implementing attributes, certain settings on two relevant control tables should be configured to ensure support for the required attribute-based calculations.

The **AttributeMapType** table is used to set how the attribute rules that users later configure are used and interpreted. Each control value in this table can be used to enable attributes in supply allotment, forecast consumption, and/or forecast adjustment calculations. As required, records can be added to the **AttributeMapType** table to enable the required combinations of attribute calculations based on the three boolean fields outlined below (for complete reference information about this table, see "AttributeMapType table" on page 676).

Table 38.2: AttributeMapType fields

Field	Description
SupplySelection	Specifies whether rules of this type are considered when allocating supplies to demands. When attribute-based supply allotment is enabled, only specific on hand inventory, scheduled receipts, and part sources might be eligible for use in satisfying particular independent demands, consensus forecast demands, or safety stock demands. For example, demands from a given customer might only accept supply assembled at a particular location.
ForecastConsumption	Specifies whether rules of this type are considered during the forecast consumption process. When attribute-based forecast consumption is enabled, then specific part forecasts can only be consumed by actual demands that have matching attribute values. For example, a given customers forecast might only be eligible to be consumed by actual demands specifically from that customer.
ForecastAdjustment	Specifies whether rules of this type are considered when specifying adjustments or overrides to made against the forecast detail records used when generating the S&OP consensus forecast. For example, a forecast adjustment might only be applied to a specific forecast category for the part customer.

As well, the **MUEPoolNettingType** table is used to enable or disable attribute-based planning at the part-level, and provides a related configuration option, based on the two fields outlined below (for complete reference information on this table, see "MUEPoolNettingType table" on page 749)

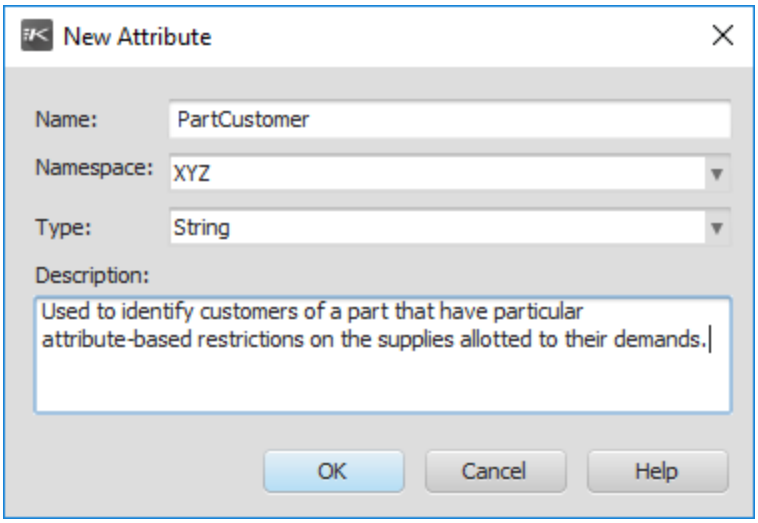
Table 38.3: MUEPoolNettingType fields (for attributes)

Field	Description
AttributeRule	Used to enable or disable attribute-based calculations for all parts that reference a given type in this table based on the following two settings: • **Ignore**—attribute-based planning is disabled for parts using this type. • **Net**—attribute-based planning is enabled for parts using this type. Note that supply attributes defined on the **PartSource** table can be propagated to all dependent demands under a given planned order regardless of the setting in this field. The **AttributeRule** setting only controls whether attributes are used in planning calculations for a given assembly or component, not whether the attribute values themselves can be propagated from or through parts in the product structure.
AttributeExcessRule	Controls how excess supply from planned orders created to satisfy one demand can potentially be used to satisfy other subsequent demands. For example, this might impact the use of excess supply arising from lot-sizing requirements (such as, minimum order quantities). • **Ignore**—excess supply from a planned order created to satisfy one demand can only be used by other demands if their demand attributes match those of the original demand. If the supply attributes on the excess planned order (as injected from the **PartSource**) satisfy the requirements of a subsequent demand, but the demand attributes on that demand do not match those on the original demand, then the excess supply is not used. • **Use**—excess supply from a planned order created to satisfy one demand can be used by other demands as long as the supply attributes on the planned order (as injected from the **PartSource**) satisfy the attribute requirements of those demands. However, there is no need for the demand attributes on those demands to match the demand attributes on the original demand. For an example of how this settng can affect the use of excess planned supply, see "Example 3: Allotments with attributes and excess supply" on page 1839.
AttributeFlowToCommon	When attribute-based forecast consumption is used, this field determines how any common (non-attribute specific) forecast is handled based on the following two settings: • **Ignore**—actual demands only consume from forecast demands to which they have matching attribute values. • **Forecast**—actual demands first consume from forecast demands to which they have matching attribute values. If actual demand remains after all matching forecast has been consumed, then that demand can consume any common forecast.

Create attributes

Attributes are custom data model elements created by data administrators. They are meant to represent particular characteristics of demands or supplies. Demand attributes can be used in the supply allotment, forecast consumption, and/or consensus forecast adjustment calculations. They typically return details of actual or forecasted end item orders such a customer name or part customer combinations. Supply attributes can be used in the supply allotment process only and typically return details of inventory, actual work or purchase orders, and part sources that provide new planned orders.

Each attribute consists of a name, a specified data type, an optional description, and must be defined in a custom namespace as shown in the following illustration.



Demand characteristics that get modeled as attributes might be things like customer names, part customer combinations, and due dates.

Supply characteristics that get modeled as attributes might be things like supplier names, plant locations, manufacturing technologies, and test dates.

Note: Attributes can be created as any data type other than reference, time, or datetime. However, if an attribute needs to return values referenced in other tables, it should be set to the string data type and the required reference field(s) specified when subsequently defining the attribute field definitions (typically, these expression might concatenate multiple referenced values together).

For more information about creating attributes, see the *the RapidResponse Data Integration Guide*.

Create and define attribute fields

Once the required demand and supply attributes have been created, they need to be enabled on the tables where they are required. To enable a given attribute on a table, an attribute field based on the attribute should be added. Attribute fields are created by data administrators and inherit their name, namespace, and data type from their attribute.

Tables for attribute fields

Attribute fields can technically be defined on any input table. However, for the purposes of enabling them for the use in analytic calculations, attribute fields should only be added within a select set of demand and supply tables as outlined in "Demand tables for attribute fields " on page 1822and "Supply tables for attribute fields" on page 1823.

Six specific demand tables can be used to enable attributes as detailed below. Depending on the table, the attributes might then be available for use in supply allotment, forecast consumption, and/or forecast adjustment calculations.

Table 38.4: Demand tables for attribute fields

Table	Description
ForecastDetail	For demand attributes that identify characteristics of the detailed forecast records used towards generating the consensus forecast for a particular part customer during the sales and operations planning process. For example, a particular forecast category and/ or customer value. The attribute values might then be used to define rules specifying how particular forecast adjustments and overrides are to be applied within the different forecast detail records used in generating the consensus forecast. Typically, these values might then be used later to control supply selection and/or forecast consumption calculations relating to consensus forecast demand.
IndependentDemand	For demand attributes that identify characteristics of independent demands (actual or forecast) for end-item assemblies. For example, order customers, part customer combinations, or due dates. The attributes might then be used when defining supply selection rules for particular demands, or to enable attribute-based forecast consumption for particular demands.
PartCustomer and PartCustomerTime PhasedAttributes	For demand attributes that identify characteristics of part customers that are then associated with the consensus forecast demands generated for a given part customer during the sales and operations planning process. For example, a customer, or part and customer combination. The attributes might then be used when defining supply selection rules for particular forecast demands, or to enable attribute-based forecast consumption on those demands.
TimePhasedSafety	For demand attributes that identify characteristics of demands generated to represent defined safety stock requirements. For example, safety stock used to support certain customers might have a requirement that it can only be sourced from a particular location or supplier. Separately netted collections of safety stock are then maintained for each unique set of attribute values associated with a given part. If safety stock is then used to satisfy actual demand, it can only be used by those demands with matching attribute values. For example, suppose a "Customer" attribute is enabled on the **TimePhasedSafety** table. Further suppose a given part has a record added to this table on a given date for **Quantity** of "500" with **Customer** attribute value of "EMC". If on a subsequent date a record is added for that same part and attribute value with a **Quantity** of "300", the 200 units of freed up supply could only be used to satisfy independent demands where the **Customer** attribute value was either "EMC" or left blank (undefined). Demands with defined **Customer** attributes other than "EMC" would be ineligible to use the supply resulting from such a safety stock drop. Note also that if a part has independent and/or consensus forecast demands with attributes defined on them, but there aren't any attributes defined on time-phased safety stock, then the attributebased demands will be processed and satisfied before the safety stock demands. This means that any firm supplies (on hand and scheduled receipts) will first be consumed by the actual demands with attribute values defined. If there is insufficient firm supply remaining to satisfy the safety stock requirements, then new planned orders would need to be generated to satisfy safety stock which may result in temporarily holding excess supply for the part.

Three specific supply tables can be used to enable attributes as detailed below. These tables are only meant for attributes being used in the supply allotment process.

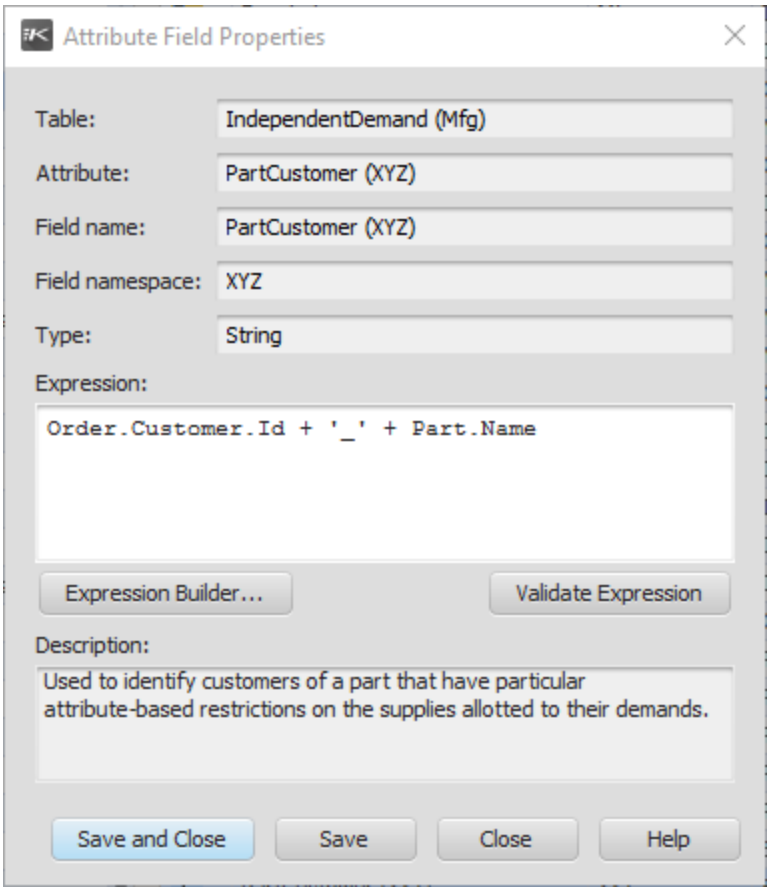
Table 38.5: Supply tables for attribute fields

Table	Description
OnHand	For supply attributes applicable to characteristics of on hand inventory (and subsequently any associated inventory transfers). For example, the date inventory was tested or the warehouse at which it is held. Also used to track any applicable supply attributes pertaining to the production of the parts now in inventory or the lower-level components that went into them. For example, the plant where a part was assembled or the supplier that produced a component used in its production. Thus, attributes should be carefully tracked on inventory records that will be imported into RapidResponse.
PartSource	For supply attributes applicable to characteristics of the part sources used to provide new planned orders. For example, the fabrication plants that make a particular chip, or the suppliers from which components are sourced. Can also be used for demand attributes that identify part sources having particular qualifications for the components underneath the planned orders they create. For example, this can be used to model an internal production limitation at an intermediate level in the product structure.
ScheduledReceipt	For supply attributes applicable to characteristics of actual work and purchase orders. For example, the manufacturing locations at which a part are assembled or the supplier from which it was purchased. Also can be used to track applicable characteristics of the component supplies that went into the order. For example, the fabrication plant that provided a lower-level component chip used in the order. Thus, attributes should be carefully tracked on work order and purchase order records that will be imported into RapidResponse. Can also be used for demand attributes used to identify scheduled receipts that have particular qualifications for the components underneath them. For example, this might be used to model an intermediate level limitation or to specify qualifications for the components on an unallocated scheduled receipt.

Defining attribute field values

Attribute fields are similar to custom calculated fields and require query expressions that define how the attribute's value is to be determined for the records on the table.

Attribute field expressions can contain any combination of input fields on or referenced from a given table, operators, and constants. For example, the following illustration shows a PartCustomer attribute being enabled on the IndependentDemand table with an expression that concatenates the referenced order id, an underscore character, and the referenced part name used to return the attribute value for each record in the table.



When creating attribute fields, any custom input fields required to hold the data that determines the attribute value should be created and applied to the data model. For example, if a supply attribute identifies the country in which supply was produced or the name of the fabrication plant that manufactured a particular component, then custom fields are typically required because these entities are not defined in the standard RapidResponse data model.

If an attribute field expression returns a blank value for a given supply record (for example, because the field the expression is based on is empty), then that record is considered eligible to satisfy any demands; including those having an attribute-based requirement for the supply. This allows for attribute requirements to be satisfied and propagated through demands on levels in the product structure where the attribute is not applicable down to the lower-level dependent components where the attribute does matter. For this reason, care should also be taken to ensure that attribute fields do return actual values at the levels in the product structure where they truly matter.

As well, if a supply attribute is not enabled on a given supply table, then any demands having rule-based requirements for the attribute cannot use the supplies held in that table. For example, if a supply attribute is enabled on the ScheduledReceipt and PartSource tables but not the OnHand table, then demands having a requirement for the attribute cannot be satisfied by on hand inventory. If all records in a given table should considered a match for a supply attribute (for example, because the attribute does not apply or matter to the supply type), then the attribute field expression can be set to return an empty value (that is, a blank string, undefined date, or 0).

For more information about creating attribute fields, see the *RapidResponse Data Integration Guide*.

Tip: If certain supply records should be ignored by all demands that have attribute-based requirements, then consider assigning a common attribute value to those supplies which is not going to be found on any demands. For example, a value of "Ignore" might be used.

Define attribute rules

Once the appropriate control settings, attributes and attribute fields are created, attribute rules can be defined using the **AttributeMap** table. Each record in this table has a **Type** field that references the **AttributeMapType** control table and determines the type of attribute-based calculations the rule defines. For example, a given rule might reference a type enabling it for use only in supply allotment, or it might reference a type enabling it for use in both forecast consumption and forecast adjustment calculations.

The **FromAttributeName** field is then used to specify the name of a particular demand attribute and the **FromAttributeValue** field is used to provide a value identifying the demands that should use this rule. For example, if **FromAttributeName** is "Customer", then a **FromAttributeValue** might identify the name of a particular customer with limitations on the supplies it can accept or whose forecast can only be consumed by its own demands.

Similarly, the **ToAttributeName** field is then used to specify the name of a particular supply attribute for which the identified demands have attribute-based qualifications, and the **ToAttributeValue** field is used to provide the values the supply attribute must possess in order to be allotted to those demands (this can include the use of certain standard RapidResponse operators for specifying multiple values or value ranges as discussed in "Operators supported in attribute rules" on page 1826). For example, if **ToAttributeName** is "Supplier", then **ToAttributeValue** might identify a list of qualified component suppliers.

Note that **ToAttributeName** and **ToAttributeValue** fields are only relevant to rules enabled for use in supply allotment. For rules not pertinent to supply allotment, the **ToAttributeName** can be set to any valid attribute (typically, the same as the **FromAttributeName** and the **ToAttributeValue** can be left blank. For example, suppose two attributes are defined; a demand attribute "Customers" that identifies customer names, and a supply attribute "Foundry" that identifies fabrication plants. Further suppose that both customer AMC and customer EMS have limitations on the foundries they can accept supply from and require forecast consumption based on customer values. Such a situation might be addressed by records similar to the following.

FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue	Type.Value	Type.ForecastConsumption	Type.SupplySelection
Customer	AMC	Foundry	Fab1, Fab2	Stnd	Y	Y
Customer	EMS	Foundry	Fab3	Stnd	Y	Y

Alternatively, if customer EMS had no attribute-based limitations on the supplies it could accept but still required forecast consumption by attribute, then records similar to the following might be defined.

Table 38.6: Attribute rules referencing the same Type

FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue	Type.Value	Type.ForecastConsumption	Type.SupplySelection
Customer	AMC	Foundry	Fab1, Fab2	Stnd	N	Y
Customer	EMS	Customer		FCOnly	Y	N

If both the **FromAttributeValue** and **ToAttributeValue** are left blank, then all attribute values are considered a match for the attribute rule. For example, the following record could enable forecast consumption across all Customer attribute values

Once defined, attribute rules can be enabled or disabled as needed for simulations or other purposes by setting the **Enabled** field to either "Y" (enabled) or "N" (disabled). When an attribute rule is enabled, its syntax is validated and any errors found reported in the calculated **Errors** field. An attribute rule is not used in supply allotment calculations until it is enabled and any reported errors fixed. For complete reference information about the **Attribute Map** table and its fields, see "AttributeMap table" on page 200.

For detailed examples of setting up attribute rules to support different scenarios, see "Attribute-based planning examples" on page 1832.

Note: If using attribute-based planning for supply allotment and the **FromAttributeName** and **ToAttributeName** fields are populated but the **FromAttributeValue** and **ToAttributeValue** fields are left blank, this implies a universal string match. That is, each **ToAttributeValue** would be considered a match for the same **FromAttributeValue**.

Operators supported in attribute rules

When defining the **ToAttributeValue** in an attribute rule, either a single value can be specified or certain operators can be used to specify multiple values or value ranges for the field. The supported operators depend on the attribute's data type as outlined below.

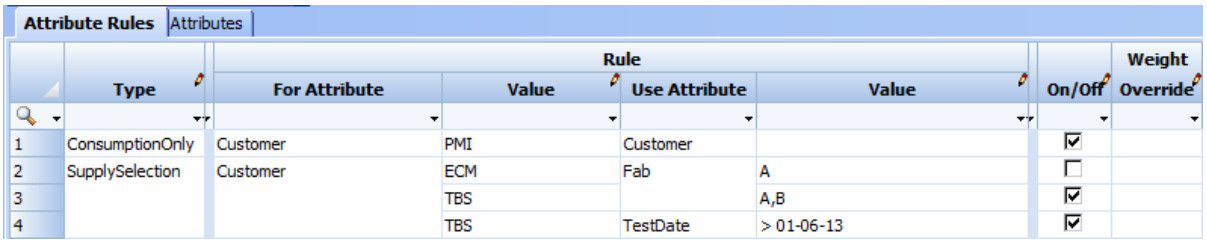
Table 38.7: Supported operators in attribute rules

Operator	Description
,	Used as an "or" operator for string values to specify that attributes can match any value within a specified list (for example, 'NW', 'SW', 'SE').
<>	Used as a "not" operator, and indicates values other than a given string, date, or number.
<	Indicates values before a given date, or less than a given number.
<=	Indicates values before/on a given date, or less than/equal to a given number.
>	Indicates values after a given date, or greater than a given number.
> =	Indicates values after/on a given date, or greater than/equal to a given number.
..	Indicates an inclusive range between two dates or numbers (for example, 500..750).

Attribute Rules workbook

Note that in order to simplify the setting up of attribute rules, records are typically added to the **AttributeMap** table using the standard **Attribute Rules** workbook included with RapidResponse. For

more information about this workbook, which is shown in the following illustration, see the the *RapidResponse Applications Guide*.



Attribute Rules | Attributes

	Type	Rule				On/Off	Weight Override
		For Attribute	Value	Use Attribute	Value		
1	ConsumptionOnly	Customer	PMI	Customer		☑	
2	SupplySelection	Customer	ECM	Fab	A	☐	
3			TBS		A,B	☑	
4			TBS	TestDate	> 01-06-13	☑	

Attribute calculations

Attributes can impact the results produced by the supply allotment, forecast consumption, and forecast adjustment analytic calculations. This section describes the following attributed-related calculations:

- "Supply allotment by attribute" on page 1827
- "Forecast consumption by attribute" on page 1830
- "Forecast adjustment by attribute" on page 1831

Supply allotment by attribute

If input supply of the part cannot be found, then RapidResponse will attempt to create a planned order from a part source that can satisfy all attributes-based qualifications for the demand. This can be a part source whose attributes return values that are either equal to values, or are found within ranges of values, specified in the **ToAttributeValue** field for the attribute rules pertinent to the demand. If a part source contains actual values that do not match that specified in **ToAttributeValue** for a rule pertinent to the demand being satisfied, then is not considered a match and is ineligible for use by the that demand. Note that if a given attribute is not defined for the part source (is blank), this is also considered a match.

As attribute rules are satisfied and dependent demands are generated under a planned order, all demand attributes are passed to the next and subsequent levels for evaluation. For this reason, a part source is also considered to match a rule if its attribute field expression returns a blank value (either an empty string, an undefined date, or a 0). This is intended to allow matches on intermediate levels of the structure where an attribute rule is not applicable, while ensuring the attributes are propagated down to lower levels for evaluation.

Note: Once supply is used by a demand in Netting logic, subsequent Capable-to-Promise logic ensures that the supply quantity can only be allocated to demands that have a matching set of attributes as that original demand. This protects against potential shortages that could arise if Capable-to-Promise logic reallocated demands outside of that attribute combination.

Attribute weights

An attribute weight is calculated and assigned from each attribute rule based on the data type of the supply attribute and the amount of supply choice the associated demands are expected to have based

on the values given for that supply attribute.

The calculated attribute weight is then used before due date in determining the order in which demands should be processed by Netting logic. This means when **PartType.CommitLevel** is set to "Low", "Medium", or "MediumExplodePriority", attribute weight can override the standard due date processing sequence. And when **PartType.CommitLevel** is set to "High", attribute weight can override due date processing sequence within a given priority level.

The use of attribute weights is thereby meant to ensure demands expected to have fewer supply choices available to them, based on attribute rules, are processed first. This can help to reduce potential excess supply that might be generated from trying to satisfy large numbers of demands with many different attribute requirements on the supplies they can use.

Table 38.8: Weight calculations for attribute rules

Data Type(s)	ToAttributeValue contains	Weight
Boolean	Single item	1.0
Date, Integer, Quantity	Single item	1.0
Date, Integer, Quantity	Range (<, <=, >=, >, ..)	.5
String	Single item,	1.0
String	List of items	1 / number of items in list
String	Not (<>) item/list of items	

Note: The weights calculated for each attribute rule can be overridden, and hence the order in which demands having attribute qualifications processed can be changed, by specifying a non-zero value in the **AttributeMap.Weight** field. For more information about this field, see "AttributeMap table" on page 200.

Compatibility with other analytics

Attributes can be configured for use in supply allotment calculations as described earlier in this section. This logic is compatible with most other analytic calculations in RapidResponse. However, in some cases, parts enabled for attribute-based logic might produced unexpected results if combined with the following analytic calculations:

Table 38.9: Impacts/limitations on other calculations

Impact area	Description
Co-products	In netting logic, co/by-product supplies take their attribute values from the primary supply's attributes. In capable-to-promise logic, co/by-product supplies take their attribute values from the primary supply's rolled up attributes.
Days of Supply	New orders are only merged together when the attribute value(s) on the triggering demands match.
Fair/equal share	When using attribute-based planning, there is limited support for fair-share and equal-share logic. That is, sharing will only happen within/between demands that have the same demand attribute values, or do not have any demand attribute values.

Table 38.9: Impacts/limitations on other calculations

Impact area	Description
Allotment override	Because allotment overrides reserve supply up front before Netting logic has a chance to process demands, no information available about demand attributes or their mappings at that point. Therefore, allotment overrides are not supported with attribute-planning.
Safety stock	Attributes can be defined on demands originating from dynamic or time-phased safety stock settings that make use of the **TimePhasedSafety** table. That is, if a part's effective **SafetyStockQuantityRule** setting is "TimePhasedQuantity", "TimePhasedQuantityToLatest", "TimePhasedSafetyDaysTotal", or "TimePhasedSafetyDaysMaximum", then demand attributes can be defined on safety stock requirements by adding attribute fields to the **TimePhasedSafety** table. However, attributes are not supported on demands originating from any safety stock method other than the four **SafetyStockQuantityRule** settings mentioned above. Additionally, if attributes are enabled on the **TimePhasedSafety** table for parts with a **SafetyStockQuantityRule** setting of "TimePhasedQuantityToLatest", then those parts are treated as though they were set to "TimePhasedQuantity". This mean even if there is a decrease in the safety stock level after the last demand, the last planned order is planned to meet the full demand requirement and not reduced in respect of the safety stock drop.
Material expiry	Attribute-based planning logic is generally supported for parts subject to expiry. However, defining attributes on demands originating from any safety stock requirements (including from dynamic or time-phased safety stock methods as described in the previous row), is not supported on parts subject to expiry.
Lot size last order	When OrderPolicy.LotSizeLastOrder is set to "N" on part sources for a part having attributes enabled on its supplies/demands, the last order quantity is still planned to respect all lot-size requirements. That is, the part source is treated as if LotSizeLastOrder were set to "Y" which can result in excess supply being generated on that last order.
Order point	Order point logic is not fully supported and might produce unexpected results if used on parts with attributes enabled on their supplies/demands. Therefore, **SafetyStockRule** settings of "OrderPointUnder", "OrderPointOver", and "OrderPointAverage" should be avoided on parts that are expected to be used in attribute-based planning logic.
Kanban	Because the specific demand that triggered the requirement for a new supply cannot always be determined when using Kanban logic, attribute-based planning is not fully supported or compatible with Kanban logic.
Negative dependent demand	Negative dependent demand only cancels positive demands that have the same demand attribute values.
In-transit scheduled receipts	For scheduled receipts belonging to an order where the **SupplyType.ProcessingRule** is either "InTransit" or "InTransitExact", if the part is enabled for attribute-based planning (has **MUEPoolNettingType.AttributeRule** set to "Net"), then any populated attributes are used by the intransit logic. That is, if both scheduled receipts have a nonblank value for a given attribute then the attribute values must match for consumption to occur. As well, if a given attribute has a blank value on either scheduled receipt, then that attribute is considered a match. Note this logic applies to all attributes populated given on a scheduled receipt and does not depend on AttributeMap rules being defined to enable it.
Multi-level search	Caution should be taken if using attribute-based planning along with multi-level search logic. The use of these two analytics together, particular alongside other capabilities such as lot-sizing and priority, can result in excess supply being planned.

Forecast consumption by attribute

A part's forecast in RapidResponse is typically represented by demand records (independent or dependent) where the reference **DemandType.ProcessingRule** is set to "SalesForecast", and by **ConsensusForecast** records generated during the sales and operations planning process. The forecast for the part can then be consumed (reduced) by its actual demands having a referenced **DemandType.ProcessingRule** set to "SalesActual (optionally, inventory pooling logic can be configured to further refine forecast consumption logic). If relevant attribute rules have been enabled, the attribute values specified on the rules can further limit which actual demands for a part can be used to consume particular forecasts. For example, attributes might be used to limit forecast consumption by customer or some other entity.

When RapidResponse processes a part's forecast demands associated with one or more **AttributeMap** rules where **AttributeMapType.ForecastConsumption** is set to "Y", the forecast can only be consumed by actual demands that match each of the attribute values defined in the **FromAttributeValue** field. For example, suppose one rule is defined **where FromAttributeName** references an attribute identifying customer names and **FromAttributeValue** contains "BCI", and a second rule is defined where **FromAttributeName** references an attribute identifying geographic order regions and **FromAttributeValue** contains "NorthWest". In this case, any forecast demands for customer BCI associated with the NorthWest region could only be consumed by actual demands from customer BCI in the NorthWest region.

If all forecast demands for a particular attribute value or set of attribute values have been consumed and there is still a remaining quantity its actual demands, then these actuals can continue to consume any forecasts for the part with undefined attribute values (that is, where the relevant attribute values are blank or not enabled in any attribute rules). This is dependent on the part's **MUEPoolNettingType.AttributeFlowToCommon** setting. When this field is set to "Ignore" (default value), the actual demands will stop consuming forecast after all forecast with matching attribute values has been consumed. Optionally, when this field is set to "Forecast" than the actual demands will continue to consume all demands for the part that have undefined attribute values.

Further, the **AttributeFlowToCommon** setting can be used together with the **PoolFlowToCommon** setting on the **MUEPoolNettingType** table to support combining pool-based forecast consumption and attribute-based forecast consumption. When forecast consumption by pool and attribute-based forecast consumption are used together, then actual demands pertaining to a specific (non-common) pool will consume associated forecast demands in the following sequence

1. Forecast from the same pool and with matching attribute values.
2. Forecast from the same pool and with undefined attribute values.
3. Forecast from the common pool and with matching attribute values.
4. Forecast from the common pool and with undefined attribute values.

Notes:

- For more information about forecast consumption, including details of configuring pool-based consumption, see "Forecast consumption" on page 1431.
- in order to see how actual demands for a part are used to consume particular forecast demands, the **ForecastConsumption** table can be used. This table reports one record for each unique demand that consumes a particular forecast. If any attributes were used to direct forecast consumption, they can be reported on relevant records in this table using the `Attribute<*>` syntax in a column expression.

Forecast adjustment by attribute

Normally, when forecast adjustments or overrides are applied against **ForecastDetail** records used in generating the S&OP consensus forecast (**ConsensusForecast** table), the adjustment or override is applied against the total consensus forecast quantity. Further, if there are multiple forecast streams contributing to the consensus forecast, the particular stream or streams that have their effective quantities adjusted is automatically determined by an internal processing sequence.

For example, consider a series of **ForecastDetail** records with effective quantities contributing to the consensus forecast as shown in the following table. Normally, the resulting consensus forecast for this part customer on 08-01-13 would be 165 as the adjustment would be applied against, the total effective quantity (205) contributing to the consensus forecast from all underlying records.

Table 38.10: ForecastDetail records that contribute to the consensus forecast

PartCustomer	Date	Category	Quantity
CPU|BetterBuy	08-01-13	MarketingForecast	80
CPU|BetterBuy	08-01-13	SalesForecast	25
CPU|BetterBuy	08-01-13	Statistical	100
CPU|BetterBuy	08-01-13	ForecastAdjustment	-40

However, if relevant attribute rules have been enabled, attribute values can be used to direct the particular forecast stream(s) against which a forecast adjustment or override is applied. That is, when RapidResponse processes an adjustment/override associated with one or more **AttributeMap** rules where **AttributeMapType.ForecastAdjustment** is set to "Y", the adjustment/override is only applied against other **ForecastDetail** records with matching attribute values. This can in turn impact the total consensus forecast quantity calculated for a given part customer and forecast date (in the case of an override or negative adjustment).

For example, consider the set of **ForecastDetail** records shown previously, but with rules enabling attribute-based forecast adjustments and an attribute "CatAdj" added to the **ForecastDetail** table to identify different demand categories (forecast streams) as shown in the following table. In this case, the resulting consensus forecast would be 180 instead of 165 (because of matching attribute values during the adjust process, the adjustment is only applied to the SalesForecast category which would get reduced to, but not below, zero).

Table 38.11: ForecastDetail records that contribute to the consensus forecast (with attributes)

PartCustomer	Date	Category	CatAdj	Quantity
CPU|BetterBuy	08-01-13	MarketingForecast	Mktg2	80
CPU|BetterBuy	08-01-13	SalesForecast	Sales1	25
CPU|BetterBuy	08-01-13	Statistical	Stat3	100
CPU|BetterBuy	08-01-13	ForecastAdjustment	Sales1	-40

Further, the settings on the **ForecastConsumption** and **SupplySelection** fields of the **AttributeMapType** table are relevant to the reporting of the consensus forecast. If both these fields are set to "N" but **ForecastAdjustment** is set to "Y", then only one **ConsensusForecast** record is reported per forecast date for a given part customer and all attributes are blended together as shown in the following table.

Table 38.12: Blended ConsensusForecast record

PartCustomer	Date	Quantity	Attribute<*>
CPU|BetterBuy	08-01-13	180	

Instead, if either or both of the **SupplySelection** and **ForecastConsumption** fields are set to "Y", then one **ConsensusForecast** record is generated for each unique attribute combination. Thus, any attribute-adjusted values can be seen as shown in the following table. This might also be relevant if the attributes values are subsequently relevant to the forecast consumption and/or supply allotment calculations.

Table 38.13: Attribute-specific ConsensusForecast records

PartCustomer	Date	Quantity	Attribute<*>
CPU|BetterBuy	08-01-13	100	Stat3
CPU|BetterBuy	08-01-13	80	Mktg2
CPU|BetterBuy	08-01-13	0	Sales1

Note: The **ConsensusForecastDetail** table can also be used to report which **ForecastDetail** records contribute to a given **ConsensusForecast** record along with the effective quantity each contributes.

Attribute-based planning examples

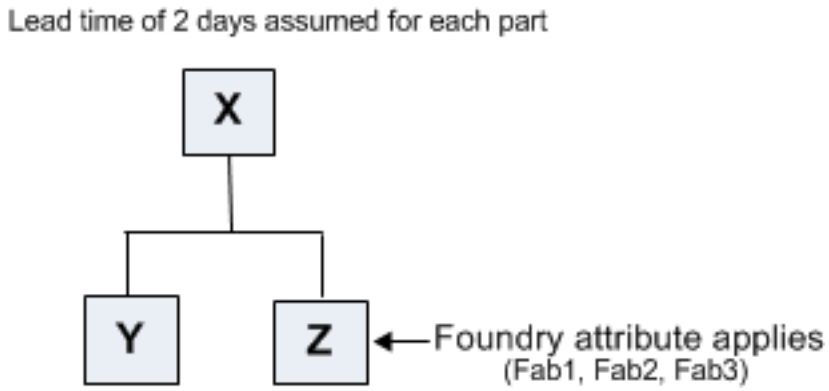
This section walks through examples showing the set up or output from attributes used to model specific customer-based requirements for supply allotments and forecast consumption.

- "Example 1: Allotment with single supply attribute" on page 1832
- "Example 2: Allotment with multiple supply attributes" on page 1836
- "Example 3: Allotments with attributes and excess supply" on page 1839
- "Example 4: Forecast consumption" on page 1841

Example 1: Allotment with single supply attribute

Assume a BOM structure, as shown in the following illustration, where assembly X is made up of components Y and Z with component Z having three different sources. Each source represents a particular foundry where the component is produced.

Figure 38.1 BOM with attributes applicable to a component



Further assume there are four customers for part X, each of which has different requirements for the supply they can accept based on the foundry where the supply of included component Z was produced as follows:

- Customer AMC— can accept supply containing component Z from any foundry.
- Customer BPM—can accept supply containing component Z from Fab1 or Fab2.
- Customer CPT—can accept supply containing component Z from Fab2.
- Customer DMO—can accept supply containing component Z from Fab3.

To setup these requirements in RapidResponse, the following steps could be taken.

Create attributes

In this scenario, two attributes are required. One to model the part customers with the particular attribute-based requirements, and one to model the different foundries used to produce supplies of component Z. Two named-attributes are created with data types as follows:

- PartCustomer (String)
- Foundry (String)

For the purposes of this example, both of these attributes are assumed to be added to the custom Xyz namespace which is assumed to have a dependency on the Mfg namespace.

Define attribute fields

To enable the PartCustomer attribute, an attribute field based on it is added to the **IndependentDemand** table to identify part and customer combinations. The expression for this field can be based on fields in the standard RapidResponse data model as follows:

```
Order.Customer.Id + '_' + Part.Name
```

To enable the Foundry attribute, a custom input field should first be added to each of the **OnHand**, **ScheduledReceipt**, and **PartSource** tables to hold the name of the foundry associated with a supply or supply source as this information is not contained in the standard RapidResponse data model. Assume these new input fields are all named Foundry; the expression for this attribute field on each of the three supply tables is then as follows:

```
Foundry
```

Create attribute rules

Before defining the required attribute rules, at least one record must exist in the **AttributeMapType** table with a **SupplySelection** field setting of "Y". In this case, a record with a **Value** of "SpSlct" is added. Attribute rules for each part customer's supply limitations can be defined in the **AttributeMap** table as follows:

Table 38.14: AttributeMap table records

Type	FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue
SpSlct	xyz::PartCustomer	BPM_X	xyz::Foundry	Fab1, Fab2
SpSlct	xyz::PartCustomer	CPT_X	xyz::Foundry	Fab2
SpSlct	xyz::PartCustomer	DMO_X	xyz::Foundry	Fab3

Note: A rule is not required for customer AMC because it does not have any foundry-based requirements for the components used in the supply it receives.

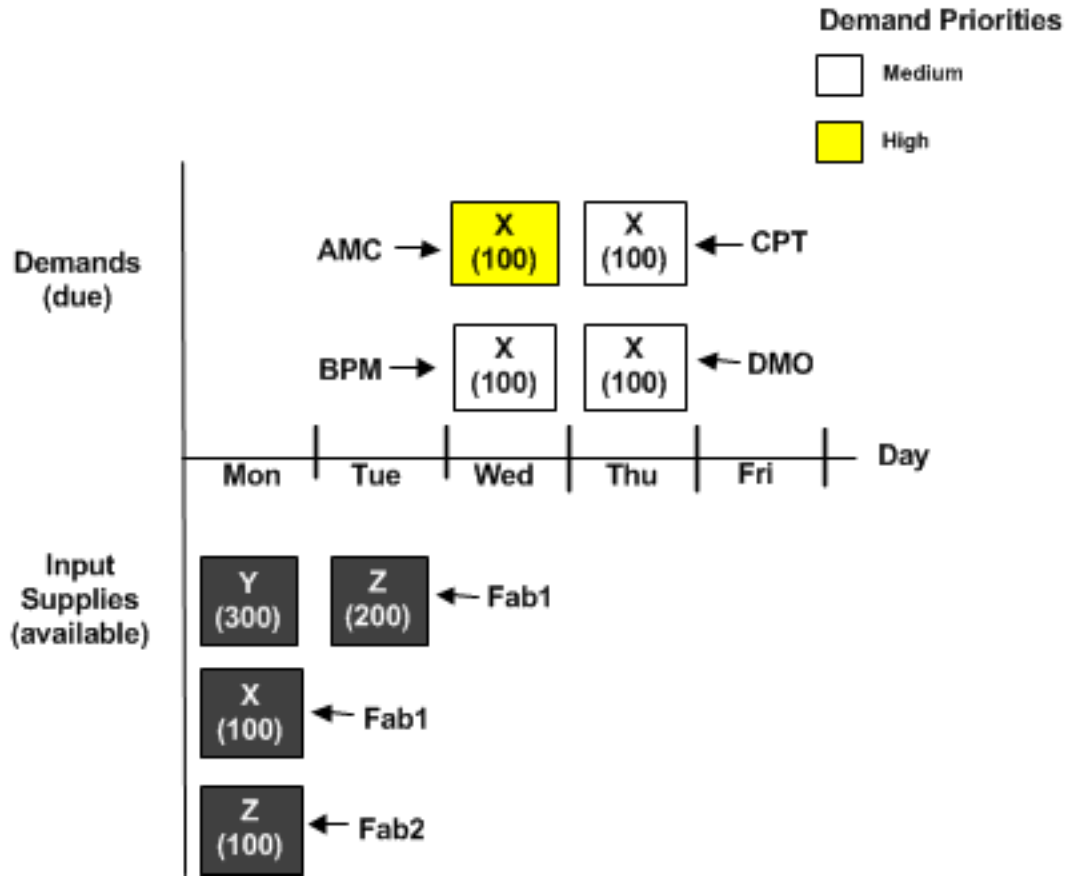
Enabled attribute-based planning

To enable attribute-based planning logic where it is required, settings should be made at the part-level through the **MUEPoolNettingType** reference. In this example, both part X and part Z should have a **MUEPoolNettingType.AttributeRule** value of "Net" in order to enable attribute-based planning, while part Y should have an **MUEPoolNettingType.AttributeRule** setting of "Ignore" as it does not require planning by attributes.

Supply allotments

Once the attribute rules have been created and enabled, they can be used by RapidResponse when allotting supply to demand. For example, assume the set of demands and supplies as shown in the following illustration. The customer associated with each demand is shown and, where applicable, the Foundry value associated with supplies of component Z (or associated with the supply of component Z used in a higher-level assembly) is also shown.

Figure 38.2 Demands and supplies with attributes



In this scenario, supply allotments would be made as follows:

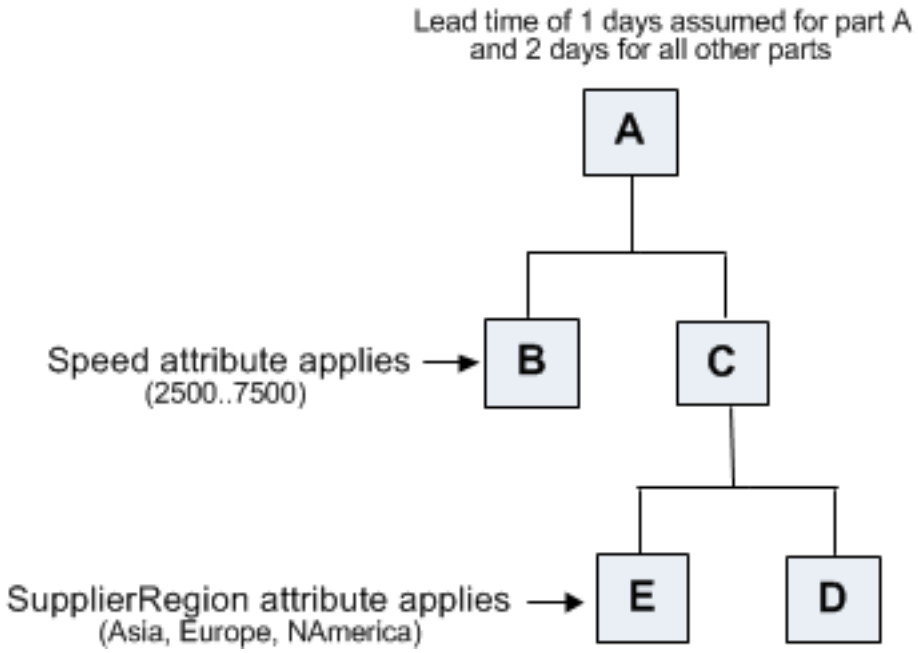
- The demand from customer AMC due on Wednesday is fully satisfied by the input supply for X. Note that because the demand has the highest priority, it receives this limited input supply. This demand is available on Monday (on time).
- The demand from customer BPM due on Wednesday is satisfied by a planned order built using 100 units of the input supply of Y available Monday and the input supply of Z available on Tuesday. This demand is available on Friday (late).

  Note that this demand could be satisfied on time if you could use the input supply of Z available on Monday, but this supply is allotted to the demand from customer CPT instead due to it having a higher attribute weight (fewer supply choices).
- The demand from customer CPT due on Thursday is satisfied by a planned order built using 100 units of the input supply of Y available Monday and the input supply of Z available on Monday. This demand is available on Thursday (on time).
- The demand from customer DMO due on Thursday is satisfied by a planned order built using 100 units of the input of Y available Monday and a planned order for Z sourced from Fab3 and available on Wednesday (required because no input supply for Z exists from this foundry). This demand is available on Friday (late).

Example 2: Allotment with multiple supply attributes

Assume a BOM structure, as shown in the following illustration, where assembly A is made of two levels of components; including component B, which is manufactured at several different plants, each of which produces a version of the component having a different measurable speed, and component E, which is purchased from several different suppliers, each of which is located in a particular geographic region.

Figure 38.3 BOM with attributes applicable to a component



Further assume there are four customers for part A, each of which has different requirements for the supply of A they can accept based on the region from which component E was sourced, and/or the speed associated with component B as follows:

- Customer AMC— can only accept supply containing component E sourced from suppliers in NorthAmerica or Europe.
- Customer BPM—can only accept supply containing component E sourced from suppliers in Europe and component B with speed between 5000 and 7500.
- Customer CPT—can only accept supply containing component E sourced from suppliers in Asia.
- Customer DMO—can accept supply containing component E sourced from suppliers in Asia and component B with speed less than 4000.

To setup these requirements in RapidResponse, the following steps could be taken.

Create attributes

In this scenario, three attributes are required. One to model the part customers with the particular attribute-based requirements, one to identify the region in which suppliers of component E are located, and one to model the speed of component Z. Three named-attributes are created with data types as follows:

- PartCustomer (String)
- SupplierRegion (String)
- Speed (Integer)

For the purposes of this example, each of these attributes is assumed to belong to the custom Xyz namespace which is assumed to have a dependency on the Mfg namespace.

Define attribute fields

To enable the PartCustomer attribute, an attribute field based on it is added to the **IndependentDemand** table to identify part and customer combinations. The expression for this field can be based on fields in the standard RapidResponse data model as follows:

```
Order.Customer.Id + '_' + Part.Name
```

To enable the SupplierRegion attribute, a custom input field should first be added to each of the **OnHand**, **ScheduledReceipt**, and **PartSource** tables to hold the name of the region associated with a particular supplier as this information is not contained in the standard RapidResponse data model. Assume these new input fields are all named SupplierRegion; the expression for this attribute field on each of the three supply tables is then as follows:

```
SupplierRegion
```

Similarly, to enable the Speed attribute, a custom input field should first be added to each of the **OnHand**, **ScheduledReceipt**, and **PartSource** tables to specify the speed of components associated with particular supplies or supply sources. Assume these new input fields are all named Speed; the expression for this attribute field on each of the three supply tables is then as follows:

```
Speed
```

Create attribute rules

Before defining the required attribute rules, at least one record must exist in the **AttributeMapType** table with a **SupplySelection** field setting of "Y". In this case, a record with a **Value** of "SpSlct" is added. Attribute rules for each part customer's supply limitations can then be defined in the **AttributeMap** table as shown below:

Table 38.15: AttributeMap table records

Type	FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue
SpSlct	xyz::PartCustomer	AMC_A	xyz::SupplierRegion	NAmerica, Europe
SpSlct	xyz::PartCustomer	BPM_A	xyz::SupplierRegion	Europe
SpSlct	xyz::PartCustomer	BPM_A	xyz::Speed	5000..7500
SpSlct	xyz::PartCustomer	CPT_A	xyz::SupplierRegion	Asia
SpSlct	xyz::PartCustomer	DMO_A	xyz::SupplierRegion	Asia
SpSlct	xyz::PartCustomer	DMO_A	xyz::Speed	< 4000

Enable attribute-based planning

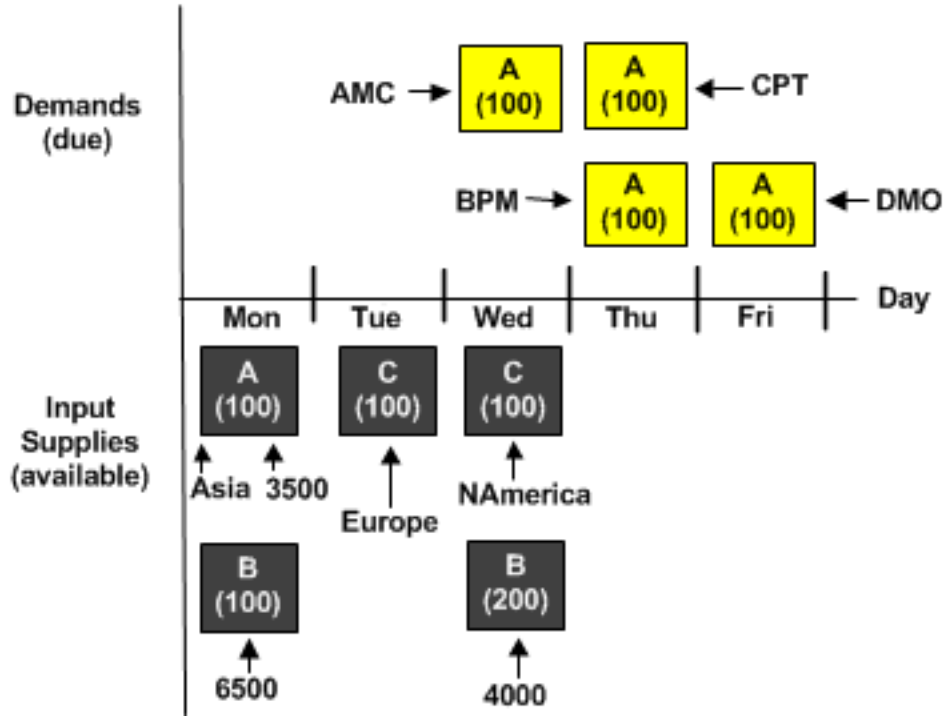
To enable attribute-based planning logic where it is required, settings should be made at the part-level through the **MUEPoolNettingType** reference. In this example, parts A, B, C, and E should have a **MUEPoolNettingType.AttributeRule** value of "Net" in order to enable attribute-based planning, while

part D should have an **MUEPoolNettingType.AttributeRule** setting of "Ignore" as it does not require planning by attributes.

Supply allotments

Once the attribute rules have been created and enabled, they can be used by RapidResponse when allotting supply to demand. For example, assume the set of demands and supplies as shown in the following illustration. The PartCustomer associated with each demand is shown and, where applicable, the SupplierRegion attribute value associated with supplies of component E (or supply of component E used in a higher-level assembly) and Speed attribute value associated with component B (or supply of component B used in a higher-level assembly) is also shown.

Figure 38.4 Demands and supplies with attributes



In this scenario, supply allotments would be made as follows:

- The demand from customer AMC due on Wednesday is satisfied by a planned order for A built using all of the input supply of C available on Wednesday and 100 units of the input supply of B available on Wednesday. This demand is late (available on Thursday).

  Note that this demand could be satisfied on time if it were able to use the earlier compatible input supplies of B and C available on Monday and Tuesday. However, these are allotted to demand from customer CPT on Thursday due a higher weight associated with its attribute rules.

- The demand from customer BPM due on Thursday is satisfied by a planned order for A using the input supply of B available Monday and the input supply of C available on Tuesday. This demand is available on time.
- The demand from customer CPT due on Thursday is satisfied by a planned order for A built using the remaining 100 units of the input supply of B available Wednesday and a new planned order for

C (which must include a planned order for part E sourced from a supplier in Asia region). This demand is late (available on Friday).

- The demand from customer DMO due on Friday is satisfied by the input supply of A due on Monday. This demand is available on time.

Note: If the demand from customer CPT could receive the input supply of A due on Monday, it would be satisfied on time and the demand from customer DMO could still be satisfied on time by creating planned orders for all required components. This could be accomplished by assigning a higher order priority to the order from customer CPT or overriding the Weight value on its attribute rule with a higher number. However, in this scenario 100 units of the input supply of B available on Wednesday would be left unused.

Example 3: Allotments with attributes and excess supply

Assume the following:

- A supply attribute "Fab" and a demand attribute "PartCustomerQ"
- A part "BB" that can be sourced from two sources, each of which has the "Fab" attribute enabled through an attribute-based field, with relevant details as shown in the following table.

Table 38.16: Sample PartSource values

Part	Source	Priority	MinimumQty	*Fab
BB	Mfg	0	500	Fab1
BB	Int	1	500	Fab2

- Two independent demands of equivalent priority for part "BB", one from each of customer BMO and ECS, with relevant details and demand attributes as shown in the following table.

Table 38.17: Sample IndependentDemand values

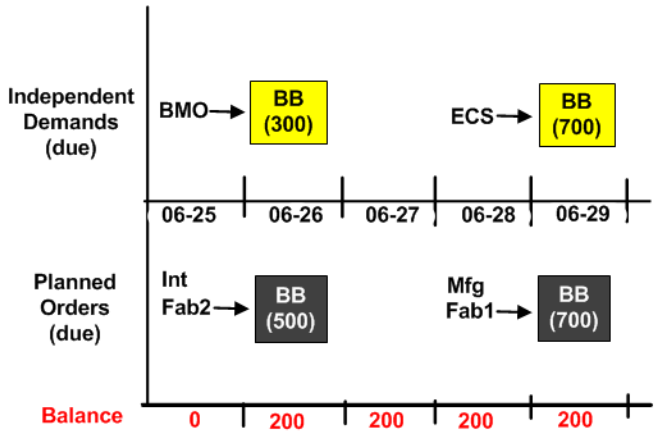
Part	Customer	DueDate	Quantity	*PartCustomerQ
BB	BMO	06-26-18	300	BMO_BB
BB	ECS	06-29-18	800	ECS_BB

- AttributeMap records that map values between these supply and demand attributes as shown in the following table:

Table 38.18: Sample AttributeMap values

FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue
PartCustomerQ	BMO_BB	Fab	Fab2
PartCustomerQ	ECS_BB	Fab	Fab1,Fab2

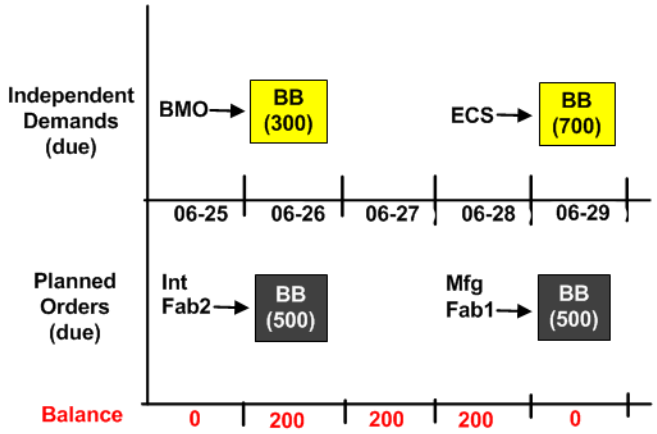
Assuming part "BB" referenced the default **MUEPoolNettingType.AttributeExcessRule** setting of "Ignore", then planned orders to satisfy the two independent demands would be as shown in the following illustration.



In this case, the demand on June 26 is processed first and satisfied by a planned order from source "Int" as it is the only part source that matches the required supply attribute of "Fab2" for demands with attribute "BMO_BB". Because of the minimum order quantity, this planned order is generated for 500 units which is 200 in excess of the demand.

Next the demand on June 29 is processed which has a demand attribute of "ECS_BB". This demand can be satisfied by a planned order from either source "Mfg" (with matching supply attribute of "Fab1") or from source "Int" (with matching supply attribute of "Fab2"). In this case, part source "Mfg" is chosen due to a higher Priority and a planned order for 700 units is generated to satisfy the entire demand. Note that the excess supply of 200 units from the previous planned order cannot be used as the demand attributes of "BMO_BB" and "BMO_ECS" do not match (if the second demand were for customer BMO than the excess supply could be used).

Instead, assume part "BB" referenced a **MUEPoolNettingType.AttributeExcessRule** setting of "Use". With this setting, planned orders to satisfy the two independent demands would then be as shown in the following illustration.



In this case, the demand on June 26 is again satisfied by a planned order for 500 units from source "Int", and 200 units of excess supply is again carried forward.

However, when the demand on June 29 with demand attribute of "ECS_BB" is processed, the excess planned order supply can be used to partially satisfy this demand as it has a matching supply attribute of "Fab1". This is because the **AttributeExcessRule** setting of "Use" only requires that the supply attribute on the excess planned order be a match for the demands attribute, but it doesn't require the demand attribute to match that of the original demand. Next the part source "Mfg", which has a higher

Priority along with a matching supply attribute of "Fab2", is chosen to satisfy the remainder of the demand and a planned order for 500 units is generated.

Example 4: Forecast consumption

Assume a part "Gamer" has a set of forecast demands, coming from both independent demands and the consensus forecast, with customers and quantities as shown in the following illustration.

Forecast Source	Date	Customer	Quantity
ConsensusForecast	May 1	BetterBuy	500
IndependentDemand	May 2	ECS	800
IndependentDemand	May 3	BCI	900
IndependentDemand	May 4	AmeriComp	400

Also assume the set of actual demands from various customers as shown in the following table.

Table 38.19: Actual demands

Date	Customer	Quantity
May 11	ECS	400
May 12	BetterBuy	100
May 13	BCI	500
May 14	ECS	300
May 15	AmeriComp	300
May 18	BetterBuy	700
May 19	BCI	400

Finally, assume the Gamer part has its **PartType.ForecastConsumptionOrder** field set to "Forward" (that is, forecast consumed from the start of the forecast interval and moving forward to the interval end). Forecast consumption then happens as follows:.

Table 38.20: Forecast consumption

ActualCustomer	ActualDate	ActualQuantity	ForecastCustomer	Forecast Date	ForecastQuantity	ConsumedQuantity
ECS	May 11	400	BetterBuy	May 1	500	400
BetterBuy	May 12	100	BetterBuy	May 1	500	100
BCI	May 13	500	ECS	May 2	800	500
ECS	May 14	300	ECS	May 2	800	300
AmeriComp	May 15	300	BCI	May 3	900	300
BetterBuy	May 18	700	BCI	May 3	900	600
BetterBuy	May 18	700	AmeriComp	May 4	400	100
BCI	May 19	400	AmeriComp	May 4	400	300

Instead, assume a requirement that forecasts for the part from customer BetterBuy can only be consumed its own actual demands, and forecasts from customer BCI can only be consumed by its own actual demands. Also, assume any leftover actuals from either BetterBuy or BCI can consume from other customer forecasts.

These requirements could be satisfied by setting up attributes in RapidResponse as outlined in the following steps.

Create attributes

In this scenario, only one attribute is required which will be used to identify the customers associated with specific actual and forecast demands.

The attribute is named "Customer", belongs to the String data type, and is added to the custom "Xyz" namespace which is assumed to have a dependency on the Mfg namespace.

Define attribute fields

Because forecasts are assumed to come from the independent demands as well as the consensus forecast, the Customer attribute needs to be enabled on two tables through the use of attribute fields.

First, an attribute field based on the Customer attribute is added to the **IndependentDemand** table, and the field uses the following expression:

```
Order.Customer.Id
```

Second, another attribute field based on the Customer attribute is added to the **ForecastDetail** table, and this field uses the following expression:

```
Header.PartCustomer.Customer.Id
```

Create attribute rules

Before defining the required attribute rules, at least one record must exist in the **AttributeMapType** with a **ForecastConsumption** field setting of "Y". In this case, a record with a **Value** of "FcstConsume" is added. This record is then referenced by the attribute rules added to define customer-specific forecast consumption for customers BetterBuy and BCI as shown below.

Table 38.21: AttributeMap table records

Type	FromAttributeName	FromAttributeValue	ToAttributeName	ToAttributeValue
FcstConsume	xyz::Customer	BetterBuy	xyz::Customer	
FcstConsume	xyz::Customer	BCI	xyz::Customer	

Enable attribute-based planning for parts

To enable attribute-based planning logic where it is required, settings should be made at the part-level through the **MUEPoolNettingType** reference. In this example, the part Gamer should reference a **MUEPoolNettingType** value where the **AttributeRule** field is set to "Net" and the **AttributeFlowToCommon** field is set to "Forecast".

Forecast consumption output

After the attributes and attribute rules have been created and enabled, forecast consumption pertaining to the set of forecasts and actuals shown now as shown in the following table. Note that

forecast for customer BCI is now consumed only by actual demand from BCI, and forecast for customer BetterBuy is now consumed only by actual demand from BetterBuy (because there is more actual demands than forecast from BetterBuy, the remainder is able to consume non-attribute specific forecast).

Table 38.22: Customer specific forecast consumption

ActualCustomer	ActualDate	ForecastCustomer	ForecastDate	ConsumedQuantity
ECS	May 11	ECS	May 2	400
BetterBuy	May 12	BetterBuy	May 1	100
BCI	May 13	BCI	May 3	500
ECS	May 14	ECS	May 2	300
AmeriComp	May 15	ECS	May 2	100
AmeriComp	May 15	AmeriComp	May 4	200
BetterBuy	May 18	BetterBuy	May 1	400
BetterBuy	May 18	AmeriComp	May 4	200
BCI	May 19	BCI	May 3	400

CHAPTER 39: Feature BOMs

Feature BOMs are used to support configurable end-item assemblies which might require, or not require, specific components and groups of components based on the features selected on a demand. For example, a configurable end-item might be an automobile with components such as different transmissions, tire packages, and audio systems available as selectable features under that. To support feature BOMs in RapidResponse, each independent demand for a configurable end-item should have a list of selected features specified on it, and this feature list is then compared against feature-based rules defined on each BOM record associating a component with an configurable end-item to determine if the component is needed in supply built to satisfy the demand.

This chapter discusses the steps involved in setting up features and feature BOMs in RapidResponse, and explains how standard supply planning and allotment calculations in RapidResponse are impacted by feature BOM logic.

In this chapter, you'll learn about:

Setting up Feature BOMs **1845**
Feature BOM calculations **1848**
Limitations on other analytics **1850**

Setting up Feature BOMs

This section outlines the basic steps in configuring feature BOM logic in RapidResponse. This includes the following:

- "Define features" on page 1845
- "Configure control settings" on page 1846
- "Define feature rules on BOM records" on page 1846
- "Specify feature lists for independent demands" on page 1847
- "Define required feature maps" on page 1847

Define features

All required feature codes should be identified in the **Feature** table. Each record in this table contains a **Name** field to identify the feature and a **Description** field to provide more detail on the feature. For example, might refer to a transmission type or the set of wheels/tires that go into an automobile.

Note that feature codes can be specified on BOM and demand records, and hence used in planning calculations, even if they have not been added to this table. However, it is strongly recommended to define them in the **Feature** table first. This provides a reference for all valid feature names, makes possible the mapping and high level feature codes that expand to larger groups of features, and ensures that details pertaining to the feature can be reported in certain calculated feature-based tables.

Configure control settings

Feature BOM logic is enabled at the part level in RapidResponse using the **PartType.FeatureBOMRule** control field. This field has the following three settings:

- **Ignore**—feature BOM logic is disabled. Standard BOM explosion and supply planning calculations are performed for parts of this type. This is the default value.
- **Normal**—feature BOM logic is enabled for parts of this type. Demands for parts of this type can only be satisfied by planned orders and dependent demands are placed on their immediate components based on the evaluation of feature lists defined on independent demands against features rules defined on BillOfMaterial records.
- **HighVolume**—feature BOM logic is enabled for parts of this type. Identical to the "Normal" option except it performs certain aggregations during full-level pegging calculations and is intended for parts expected to have extremely high volumes of orders per day which would otherwise result in poor performance of the full level pegging analytic. For more information, see "Full level pegging limitation" on page 1849.

Thus, appropriate **PartType** records need to be defined and the **FeatureBOMRule** field should be set to either "Normal" or "HighVolume" for each configurable end-item assembly (depending on expected order volume), and set to "Ignore" for all other parts.

Define feature rules on BOM records

Records should exist in the **BillOfMaterial** table for a configurable end-item and each of its immediate components. The **BillOfMaterial.FeatureRule** is then used to define which of these immediate components are standard components that are always required to satisfy demands for the end-item, and which of these are feature-based components that might be required (or not) based on the features specified on a given demand.

If a record defines a standard component under a configurable end-item assembly, then its **FeatureRule** field should be left blank. This ensures the component is always included in supply built for the assembly regardless of the features defined on the demand it is satisfying.

If a record defines a feature-specific component under a configurable end-item assembly, the **FeatureRule** field should be used to specify the conditions under which the component is required in supplies satisfying demands for the assembly. In the simplest case, the **FeatureRule** field might contain just the name of a single feature. If that feature is subsequently included on a given demand for the assembly, then the component is required to satisfy that demand; otherwise, the component is not required.

Besides feature names, the **BillOfMaterial.FeatureRule** field can also make use of logical operators to create more complex rules that specify combinations of features an assembly demand must include in order to require a given component. The following operators are supported:

- **And**—used to specify multiple features that must all be present on an assembly demand for the component to be required in supply satisfying that demand. The ampersand character (&) is used to represent "and". For example, `A & B & C` would indicate that the component is only required if a given demand contains all of the features A, B, and C.
- **Or**—used to specify multiple features any one (or more) of which must be present on an assembly demand for the component to be required in supply satisfying that demand. The comma (,) is used to represent "or" and features combined with this operator should also be enclosed in parentheses. For example, `(A, B, C)` would indicate that the component is only required if a given demand contains any of the features A, B, or C.
- **Not**—used to specify a feature that must not be included on a demand for the component to be required in supply satisfying that demand. An exclamation (!) is used to represent "not". For example, `!B` indicates the component is only required if a given demand does not contain feature B.

Note: The character used to represent the "and" operator is configurable using a global setting as discussed in "Defining the And operator for feature rules" on page 2208.

Specify feature lists for independent demands

Feature rules on **BillOfMaterial** records are compared against features specified on independent demands to determine if a given component is required to satisfy an assembly demand. The features required for a given independent demand are specified in the **IndependentDemandFeatureList** table. Each record in this table references an **IndependentDemand** record and specifies one feature required for that demand. The list or set of all features specified for a given independent demand can then be accessed through the **IndependentDemand.FeatureList** set field.

Dependent demands are then placed on only those immediate assembly components where the **BillOfMaterial.FeatureRule** value evaluates to true when compared against the list of features specified on the demand (a blank **FeatureRule** always evaluates to true).

For example, if the **FeatureList** contains `A, C, D, E` then this would indicate that features A, C, D, and E are included on the demand. A **FeatureRule** value of `C & E` would evaluate to true against this list as would a **FeatureRule** value of `(B, D, F)`. However, a **FeatureRule** value of `A & B` would evaluate to false against this list.

Note: If the **FeatureList** set field is empty for a configurable end-item demand (meaning no records in the **IndependentDemandFeatureList** table reference that independent demand), then dependent demands are only placed on the standard components belonging to that assembly.

Define required feature maps

The ability to take a high-level feature code and convert or expand it to a more detailed group of features is also supported. This is done using the **FeatureMap** table in RapidResponse.

Each record in the **FeatureMap** table contains a reference to the configurable assembly part the features on the record pertain to, along with **FromRule** and **ToFeature** fields used to define the feature mapping. The **FromRule** field is a string identifying a valid feature code that might be included in the list

of features specified on demands in the **IndependentDemand.FeatureList** field. The **ToFeature** field is a reference to a feature code value that exists in the **Feature** table. Typically, a given **FromRule** value will be defined on multiple records each referencing a different **ToFeature** value.

For example, suppose a given high-level feature code "A" applicable to part "SUV" should map to features "XX", "YY", and "ZZ". To support this, records could be added to the **FeatureMap** table as follows:

Part	FromRule	ToFeature
SUV	A	XX
SUV	A	YY
SUV	A	ZZ

Thus, when a high level feature code included on a given demand for a part matches a **FeatureMap.FromRule** value, it is expanded to the set of all associated features referenced in the **ToFeature** field. This expanded set of features is what is then compared against **BillOfMaterial.FeatureRule** record values in order to determine the components required in satisfying the demand

Note: The **FeatureMap** table also supports date effectivity and the ability to define a sequence in which expansion of mapped features occurs. For complete reference information about the **FeatureMap** table, see "FeatureMap table " on page 281.

Feature BOM calculations

When the Netting analytic encounters an independent demand for a configurable end-item assembly (that is, a part where **PartType.FeatureRule** is set to "Normal" or "HighVolume"), it must satisfy the demand with a new planned order. If any input supplies, such as on hand inventory or scheduled receipts, exist for a configurable end-item they are ignored as they might not necessarily have been built using the right group of components to match the features selected on the demand.

In order for Netting to place dependent demands on the appropriate components and create any required planned orders for those components, it must evaluate the features included on each demand. That is, when processing demand for a configurable end-item, the set of features required on the demand (**as** represented by the **FeatureList** set field on the **IndependentDemand** record) is evaluated against the feature rules specified on each record associating the end-item assembly with one of its immediate components (**BillOfMaterial.FeatureRule**). If the feature rule defined for a given component matches against any of the feature(s) included on the assembly demand, then dependent demands are placed on that component. If the feature rule defined for a given component does not match against any of the feature(s) included on the demand, then dependent demands are not placed on the component and is assumed not to be required in supply built to satisfy the demand. Note also that if **BillOfMaterial.FeatureRule** is left blank, this indicates a standard component for the assembly which is assumed to be required in satisfying all of the assembly's demands regardless of the features included on them.

Note also that if there are multiple demands for a configurable end-item due on the same date, then just a single planned order is created to cover all of those demands (within a given priority level and pertaining to the same model, unit, and pool combination). This is regardless of the features selected on those demands. Thus, different portions of the same planned order for a configurable end-item might

typically explode down different component paths and as such only select portions of that planned order might be able to satisfy a given demand.

When the Capable-to-Promise analytic subsequently allots supplies to demand, each demand for a configurable end-item is guaranteed to be satisfied by at least a portion of the planned order created to satisfy it during Netting calculations based on applicable features. As well, because a single planned order will typically have been created in Netting to cover multiple demands for a configurable end-item, RapidResponse forces incremental availability calculations on for each configurable end-item (regardless of whether it has been enabled through the **Part.IncrementRule.ProcessingRule** value or not). This is to ensure that different demands being satisfied by the same planned order, but with potentially different features and hence components going into them, are not adversely impacted by the availability of components that they do need. As such, orders for the same part that are due on same date and satisfied by the same planned order might have different available dates based on their selected features and related component requirements.

Full level pegging limitation

Complete full-level pegging information pertaining to configurable end-items can be generated and reported in RapidResponse. However, a configuration setting is available that can be used to aggregate certain full-level pegging calculations and thus generate only a subset of the records that are otherwise created. This is intended to improve the performance or speed of full-level pegging calculations on highly configurable end-items that are expected to have extremely high volumes of orders per day.

For configurable end-items expected to have a moderate amount of order per day, then feature BOM logic can be enabled for the part using the "Normal" setting on the **PartType.FeatureBOMRule** field. With this setting, complete full level pegging results are reported in RapidResponse.

However, for highly configurable end-items expected to have large numbers of demands per day, then the "HighVolume" setting on the **PartType.FeatureBOMRule** field should be used instead. This setting enables feature BOM logic and is identical to the "Normal" option with the exception of specific aggregations it performs on results reported in the following two tables:

- **WhereConsumed**—for records pertaining to component supplies used to satisfy demand from a given configurable end-item on a given date (and belonging to the same priority, model, unit, and pool values), the driving independent demand is not reported and instead the associated **CTPPlannedOrder** used to satisfy the demands due on that date is reported as the driver instead.

  As well, only a single **WhereConsumed** record per component supply allotted to any or all demands for the configurable end-item on a given date is reported (with the "Normal" option, one record is reported for each allotment of component supply to a demand). As a result, fewer records can be expected in this table for parts associated with the "HighVolume" option as compared to parts associated with the "Normal" option.

  However, if knowledge of the specific demand for a high-volume, configurable assembly that a component supply is used in is required, the **WhereConsumedToHigh VolumeOrder t**able can be used instead. This table pegs all component supplies to the highest level demand that they satisfy, and thereby can be used to determine the demands impacted by component availability. Note that although this table pegs all component supplies to the highest level demand they satisfy, it should only be used for reporting on components used in high-volume, configurable, end-items (if those components are also used in non-configurable and/or non-high-volume assemblies, then this table can report all pegging details in one place). However, for components that are only ever used in assemblies where the **PartType.FeatureBOMRule** is either "Ignore" or "Normal", the **WhereConsumed** table should be used instead.

- **WhereConsumedForSupply**—only one record per planned order for each top-level part is reported, and multiple allotments from the same planned order for a component are not reported. Instead, just one record is generated for the component supply quantity and if the supply is allotted to multiple demands for the configurable end-item on a given date, then only one of those allotments is chosen for reporting. Thus, similar to the WhereConsumed table, this table can be expected to have fewer records for parts associated with the "HighVolume" option as compared to parts associated with the "Normal" option.

Notes:

- For a full list of limitations introduced for parts set up as configurable end-items, see "Limitations on other analytics" on page 1850.
- For more information about the **WhereConsumedToHighVolumeOrder** table, see "WhereConsumedToHighVolumeOrder table" on page 1262.

Calculated feature tables

To help identify where features are included on demands, as well as to locate potential issues with feature BOM configurations, a number of calculated tables report details pertaining to feature BOMs in RapidResponse. The following calculated tables are available:

- **BillOfMaterialFeatureRule**—reports all features included in the feature rule on a given BillOfMaterial record and identifies any of those features that are not yet defined in the Feature table. For more information, see "BillOfMaterialFeatureRule table" on page 957.
- **FeatureBOMForIndependentDemand**—used to report all immediate components of a configurable end-item that are required to satisfy a given demand, based on its selected features, and the amount of each component required to satisfy the demand. The standard Feature Planning workbook is included with RapidResponse is based on this table. For more information, see "FeatureBOMForIndependentDemand table" on page 1045.
- **IndependentDemandByFeature**—uses an analytic parameter to report all orders that include a specifiable feature or group of features. The standard Orders by Feature workbook included with RapidResponse is based on this table. For more information, see "IndependentDemandByFeature table" on page 1072.
- **IndependentDemandFeature**—used to report all features included in the features list on a given IndependentDemand record (including any features expanded from the initial feature list based on mappings in the FeatureMap table) and identifies any of those features that are not yet defined in the Feature table. For more information, see "IndependentDemandFeature table" on page 1074.
- **IndependentDemandFeatureSummary**—used to count and report the number of independent demands that each feature is included in. For more information, see "IndependentDemandFeatureSummary table" on page 1075.

Limitations on other analytics

When a part is set up as a configurable end-item for the purpose of utilizing feature BOM logic in RapidResponse, certain limitations on other analytics are introduced. For parts where the **PartType.FeatureBOMRule** is set to either "Normal" or "HighVolume", the impacts on the RapidResponse analytic calculations outlined in the following table should be considered.

Table 39.1: Feature BOM limitations on other analytics

Limitation	Description
Input supply usage	Demands for configurable end-items can only be satisfied by planned orders. This means any input supplies for these configurable end-items in the form of on hand inventory, inventory transfers, scheduled receipts and so on, are ignored and cannot be used to satisfy demands for the part.
Full-level pegging	For parts where FeatureBOMRule is set to "HighVolume" only, certain aggregations are performed; such as, not reporting the driver demand for component supplies and instead referencing the associated **CTPPlannedOrder** record. This limitation primarily impacts the results generated in the **WhereConsumed** table, and is meant to improve analytic performance on parts that are expected to have extremely high volumes of orders per day. **Note:** The **WhereConsumedToHighVolumeOrder** table can be used to work around this limitation. This table was added in Version 2014.4 and pegs component supplies to the highest level demand source they are used to satisfy (even if those demands are for high-volume parts).
Safety stock	Safety stock is never planned for configurable end-items (it can, however, be planned for components under a configurable end-item).
Lot sizing	Any minimum and multiple lot size rules defined on configurable end items are ignored. Maximum lot size rules are, however, respected if the end item is sourced from a part source utilizing takt time logic.
Order point	Order point logic is always disabled for configurable end items.
Kanban	Kanban logic is always disabled for configurable end items.
Fair share and equal share	Fair and equal share allocation logic is not supported for configurable end-items.
BOM substitution	BOM-level part substitution cannot be performed on configurable end-items.
Global substitution	Global part substitution cannot be performed on any of the immediate components of a configurable end-item.
MPS Config	Parts that are set up as configurable end-items should not also be set up as MPSConfig parts (MPSConfig settings are ignored on configurable end-items).
Incremental availability	Increment availability calculations are always forced on for configurable end items (regardless of the part's **IncrementalRule.ProcessingRule value**). As such demands due on the same date and satisfied by the same planned order, might have different available dates.

Table 39.1: Feature BOM limitations on other analytics

Limitation	Description
Multi-level search	Multi-level search logic is supported on configurable end-items, however this should be done with caution as these calculations might perform slowly on parts that have high volumes of orders.
Campaign planning	Campaign planning cannot be used with parts that are also configured in feature BOM relationships (where the assembly's **PartType.FeatureBOMRule** field is set to "Normal" or "HighVolume").
Two-date planning	Two-date planning logic is not supported on parts that are also configured in feature BOM relationships (parts having a **PartType.FeatureBOMRule** setting of either "Normal" or "HighVolume").

CHAPTER 40: Allotment overrides

When component supplies are unavoidably late to satisfy all demands on time, RapidResponse analytics attempt to allocate limited supply based on available/due dates, demand priorities, fair or equal-share logic, and so on. In cases where order priority techniques cannot achieve the desired allotments of component supply, or are otherwise impractical to implement, allotment overrides can be used to try and improve demand availabilities or meet other business goals.

Allotment overrides allow for specified quantities of limited component supplies to be reserved towards particular top-level demands. Components can be reserved at any level in the BOM structure and RapidResponse will attempt to direct the indicated component quantities up the structure so that they are ultimately consumed by the intended top-level items. For example, some amount of a bottom-level component might be reserved for demands of a particular assembly or split between demands from certain customers.

Included with RapidResponse is the standard Allotment Overrides workbook. This workbook shows the top-level items where a component's supply is required along with how much is currently being allotted to each, and indicates any shortages in a given period. Typically, this workbook is then be used to define any desired overrides to the supply allotments generated by RapidResponse.

In this chapter, you'll learn about:

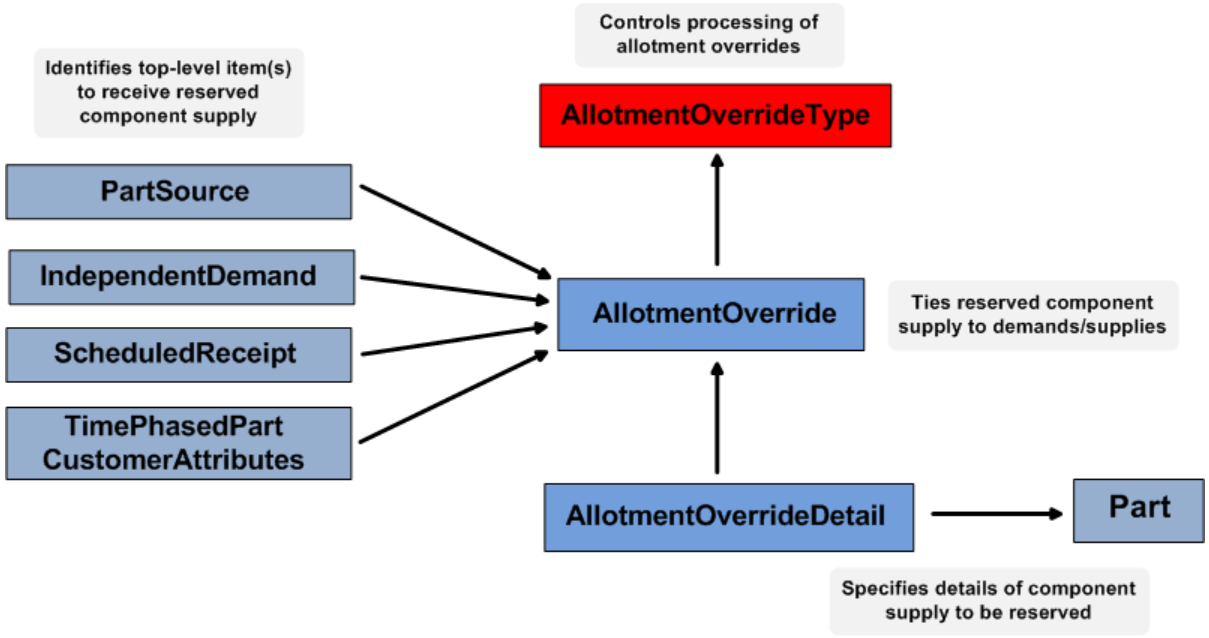
Setting up allotment overrides **1853**
Allotment override calculations **1856**
Allotment override examples **1857**

Setting up allotment overrides

By looking at the results of RapidResponse analytic calculations, critical component shortages can be identified along with the higher level demands (or supplies) that require these components and the amounts of each component that have been allotted to each. As necessary, these component allotments can then be manually modified to try and achieve defined business goals such as increasing the number of on-time demands.

RapidResponse includes several tables designed to hold details pertaining to allotment overrides and these are typically populated from within RapidResponse. Several other standard RapidResponse tables are also used to indicate the items where allotment overrides are used. The relationship between those tables involved in the allotment override process is shown in the following illustration.

Figure 40.1 Allotment override data model



This section discusses the basic steps required in configuring allotment overrides using the RapidResponse tables shown above.

Note: If setting up allotment overrides in RapidResponse, it is recommended to enable incremental availability for the part whose component supply is being reserved, as well as any parts between it and the top level part for which the reserved supply is intended. This ensures that incremental portions of on time supply can be properly passed up multiple levels of the product structure to reach the intended top level item. For more information about incremental availability, see "Incremental availability calculations" on page 1476.

Create processing rules

In order to use allotment overrides in RapidResponse, values must be defined in the **AllotmentOverrideType** control table. The main purpose of this table is to control when a given allotment override will be used in RapidResponse calculations. For example, settings can be added so that associated allotment overrides are always enabled or enabled only if the date associated with the reservation of component supply is on or after RunDate.

For more information, see "AllotmentOverrideType table" on page 671.

Define allotment overrides

Once any required processing rules have been setup, allotment overrides can be defined by adding values to the **AllotmentOverride** table. This table holds string values that uniquely identify each allotment override in RapidResponse. For example, an Id in this table might concatenate the assembly part and site or the customer and site combination the override pertains to. Through references from

other tables, the **AllotmentOverride** table is then used to tie together actual reservations of component supply with the higher-level demands or supply where they are to be consumed.

If it is expected that a given component's supply will be reserved through multiple allotment overrides (for example, for multiple assembly parts or different customer's demands), then use of the **AllotmentOverride.Sequence** field can be considered. This field defines the processing order of allotment overrides in RapidResponse, with lower values being processed first and thus taking precedence over allotment overrides with higher Sequence values.

For more information, see "AllotmentOverride table" on page 187.

Reserve component supply

After values are added to the **AllotmentOverride** table, the actual reservations of component supply can be made using the **AllotmentOverrideDetail** table. This table references the **AllotmentOverride** table and contains details for each allotment override. For example, it identifies the component part to be reserved (can be any number of levels below the assembly for which it is being reserved), the amount of the component's supply to reserve, and the date on which to look for actual supply to provide the quantity being reserved. Note that the AllotmentOverrideDetail table does not provide the ability to specify the specific supply that will be used to provide the reserved component quantity.

As necessary, multiple **AllotmentOverrideDetail** records can reference a single **AllotmentOverride** value. For example, reservations of several components might be grouped together in a single allotment override pertaining to a given assembly.

For more information, see "AllotmentOverrideDetail table" on page 189.

Allocate reserved component supply

Finally, reserved component supplies can be directed towards the higher-level assembly items where they are meant to be consumed. This is done by ensuring the higher-level demands or supplies that are to receive the reserved components reference the same **AllotmentOverride** value as the **AllotmentOverrideDetail** records that identify those reserved component quantities. For the purpose of receiving allotments of reserved components, AllotmentOverride reference fields are available on the following four tables:

- **IndependentDemand**—reserves component supply to top-level demands. Useful for ensuring component supply is available to demands for a specific assembly or from a specific customer.
- **PartCustomerTimePhasedAttributes**—reserves component supply to a specific part customer's consensus forecast demands. Useful when RapidResponse is used as part of the sales and operations planning process to generate consensus forecasts from multiple streams.
- **PartSource**—reserves component supply to dependent demands generated from a specific part source. Useful for reserving component supply to items in the middle of the product structure (normal allotment logic is applied at higher levels).
- **ScheduledReceipt**—reserves component supply to work orders (top-level or otherwise). Useful for scheduled receipts that drive production for which there is no associated demand.

As necessary, many records in these tables can reference a single value in the **AllotmentOverride** table. For example, if a given amount of a component is intended to be reserved for a group of customer

demands, then each of those customer demands should reference the same **AllotmentOverride** value as the **AllotmentOverrideDetail** record that reserves the component quantity.

Note: Allotment overrides in RapidResponse cannot be used to reserve component supply to demand generated from safety stock requirements.

Allotment override calculations

Allotment overrides are considered by both the Netting and Capable-to-Promise analytics which then attempt to ensure that any demands or supplies associated with allotment overrides are processed first. The specified quantities of reserved components associated with those demand or supplies are allotted first before moving to the normal RapidResponse allotment of supply to demand (based on due date, priority, fair or equal share rules, and so on).

During Netting calculations, any demands that have allotment overrides associated with them are processed before normal demands that do not have allotment overrides defined. Netting also ensures that allotment overrides are propagated to lower levels in the structure. This is done by setting the AllotmentOverride.Id from a given demand to the supply that satisfies it, and then propagating this value through the structure during dependent demand explosion.

Similarly, during Capable-to-Promise calculations, demands that have allotment overrides associated with them are processed before other demands. In order to find the specified quantity of a reserved component, RapidResponse starts looking for supply beginning from the date defined in the AllotmentOverrideDetail.Date field. If sufficient supply is not available on that date then RapidResponse looks through earlier date buckets as required, and if sufficient supply still cannot be found in those earlier buckets then RapidResponse looks in date buckets later than AllotmentOverrideDetail.Date until the full amount of the reserved quantity is found.

In cases where multiple allotment overrides require supply of the same reserved component and there is insufficient supply to meet the reserved quantity, the AllotmentOverride.Sequence field is used to prioritize which allotment overrides receive the reserved quantities first (with lower Sequence values indicating a higher priority). If there are multiple demands associated with the same allotment override and there is insufficient supply to meet the reserved quantity, then standard RapidResponse allotment logic is used to determine which of those demands receive the reserved quantities first (based on priority, due date, and so on).

Conversely, if the amount of a component's supply reserved through any allotment overrides is more than is needed to satisfy the demands associated with those allotment overrides, the remainder is made available to other demands (that is, RapidResponse will not allot more of a reserved component than is actually needed to satisfy demands associated with allotment overrides).

After processing an allotment override, the incremental allotment of supply to demand is tagged with the associated AllotmentOverride Id, and Capable-to-Promise logic then attempts to ensure that the incremental amounts of the reserved lower level component supply are directed up the product structure to demands with compatible AllotmentOverride Ids. This is done in order to try and ensure the reserved component reaches its intended top-level target, and can typically be seen through the AllotmentOverride reference field on the SupplyDemand table. For example, if a bottom level had supply reserved to a top-level demand through a given AllotmentOverride, that AllotmentOverride Id could be reported on the SupplyDemand record that allots supply of the bottom level component to its parent, the SupplyDemand record that allots supply of the bottom level component's parent to its parent, and so on up the structure on SupplyDemand until the top level item is reached. However, in some cases the AllotmentOverride reference on the SupplyDemand table might not necessarily have been used when determining the supply reported in a given SupplyDemand record was allocated to the demand

reported in that record. For example, this might occur in cases where one supply goes into multiple originating independent demands and where each refers to a different AllotmentOverride record or at intermediate levels where no active AllotmentOverrideDetails records are present.

For examples showing the impact allotment overrides might have on standard supply allotments in RapidResponse, see "Allotment override examples" on page 1857.

Allotment override examples

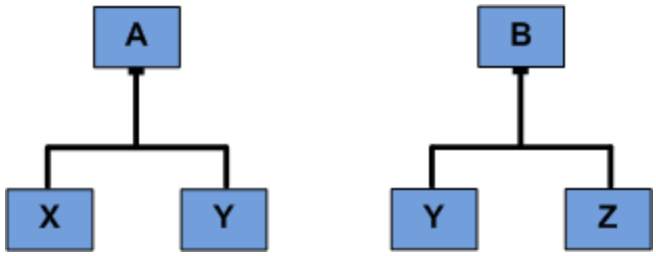
This section provides the following examples of defining allotment overrides in RapidResponse to alter the default allocations of limited component supply in order to improve the on-time delivery of demands.

- "Example 1: allotment override associated with a single demand" on page 1857
- "Example 2: allotment overrides associated with multiple demands" on page 1859

Example 1: allotment override associated with a single demand

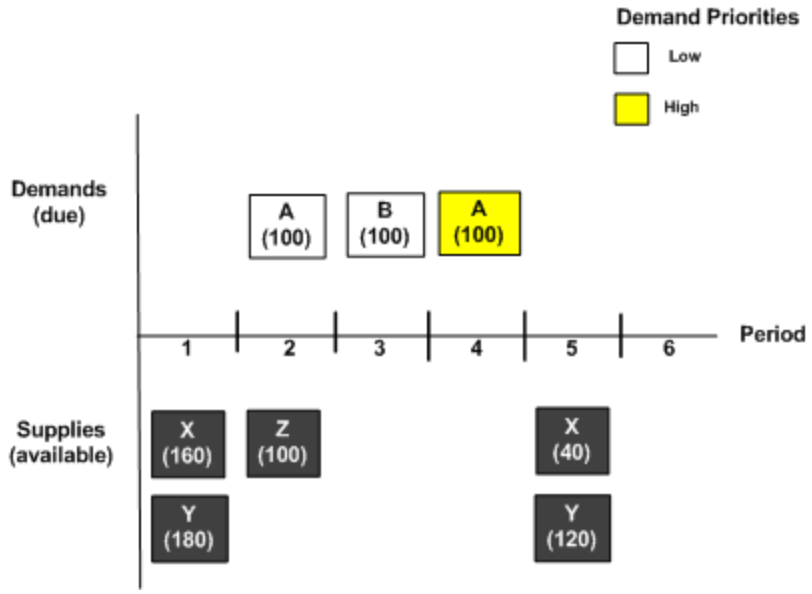
Suppose two simple BOM structures at site HQ as shown in the following illustration, where Assembly A and Assembly B are each made up of two components, including a common component Y.

Figure 40.2 BOM structure for assemblies with common component Y



Further suppose the set of prioritized demands for A and B, along with component supply availability, as shown in the following illustration. In particular, note that there is not enough supply available of either component X or common component Y to meet all demands on-time.

Figure 40.3 Demands and supplies for allotment override



In the standard RapidResponse allotment logic, the limited supply of 180 units of component Y is first allocated to the higher priority demand for A in period 4 and, after satisfying it, the remainder is then allocated to the lower priority demands in due date sequence.

This results in the demand in period 4 being completely satisfied, the demand in period 2 being partially satisfied, and the demand in period 3 being completely late as outlined in the following table. Note that although the demand for A in Period 2 receives 80 units of Y on time, only 60 units of the order for A itself is ultimately on time because not enough of component X is available until a later date.

Table 40.1: On-time quantities of independent demands before allotment override

Part.Name	Part.Site	Order Priority	Due	Quantity	OnTime	Late
A	HQ	High	Period4	100	100	0
A	HQ	Low	Period2	100	60	40
B	HQ	Low	Period3	100	0	100

If this allotment was not desirable, allotment overrides could be used to force a different allocation of component supply. For example, suppose a requirement for the high priority demand to still be fully satisfied, but for the remainder of the earlier supply of Y to be shared so that 20 units of the demand for B could be available on time.

This could be accomplished by first creating an AllotmentOverride record with the following values. In this case:

Table 40.2: AllotmentOverride values

Id	Sequence	Type.ProcessingRule
B:HQ	0	AfterRunDate

Next, a single record can be added to the AllotmentOverrideDetail table to reserve the required component supply as follows:

Table 40.3: AllotmentOverrideDetail values

AllotmentOverride.Id	Component.Name	Component.Site	Date	Quantity
B:HQ	Y	HQ	Period1	20

Finally, the IndependentDemand record with the order for part B would be updated so that its AllotmentOverride reference field points to the newly created AllotmentOverride. This ensures that the reserved component supply of 20 units of Y is first allocated to the demand for part B, after which normal RapidResponse logic allocates the remaining 150 units of on time component supply of Y based first on priority and then in due date sequence. This results in the following modified on-time quantities of the demands with 20 units of the demand for B now available on-time, but no change in the on-time quantities for A:

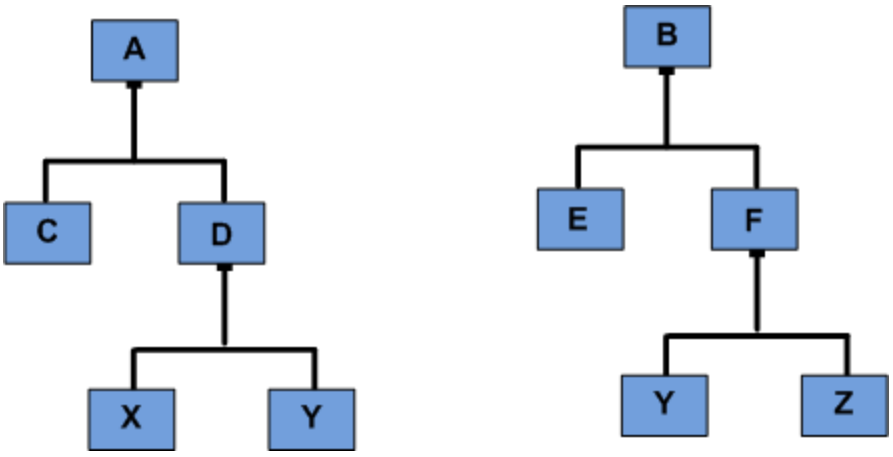
Table 40.4: On-time quantities of independent demands after allotment override

Part	Site	Priority	Allotment Override.Id	Due	Quantity	OnTime	Late
A	HQ	High	NULL	Period4	100	100	0
A	HQ	Low	NULL	Period2	100	60	40
B	HQ	Low	B:HQ	Period3	100	20	80

Example 2: allotment overrides associated with multiple demands

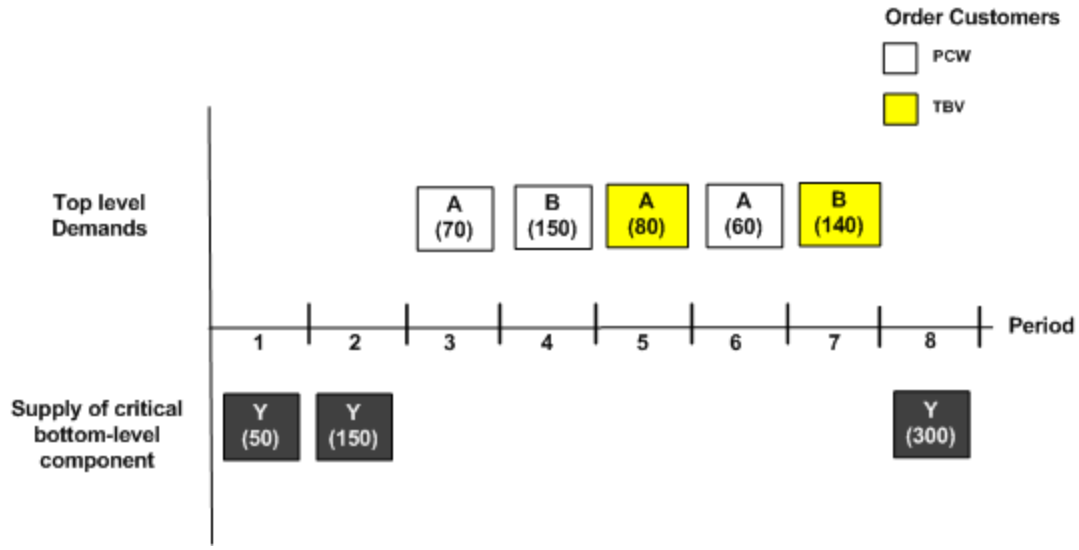
Suppose two simple BOM structures at site HQ as shown in the following illustration, where Assembly A and Assembly B each share a common component Y several levels down the structure.

Figure 40.4 BOM structures for assemblies with common component Y



Further suppose the set of non-prioritized demands from customers PCW and TBV for parts A and B as shown in the following illustration. Also shown is the availability of supplies for component Y, some of which is too late to ensure all demands for A and B can be satisfied on time (for example, this might be due to constraints). There are assumed to be no constraints or supply limitations on any of the other components. That is, the ability to satisfy the demands for A and B is gated only by supply constraints on Y.

Figure 40.5 Demands and critical component supply for allotment override



In the standard RapidResponse allotment logic, the on-time component supply of 200 units of component Y is allocated to demands in due date sequence. This results in the demand in period 2 being on-time, the demand in period 3 being partially on-time, and all subsequent demands being completely late as outlined in the following table (note that only demands from customer PCW receive portions on time).

Table 40.5: On-time quantities of independent demands before allotment overrides

Part.Name	Part.Site	Order.Customer	Due	Quantity	OnTime	Late
A	HQ	PCW	Period3	70	70	0
B	HQ	PCW	Period4	150	130	20
A	HQ	TBV	Period5	80	0	80
A	HQ	PCW	Period6	60	0	60
B	HQ	TBV	Period7	140	0	140

If this allotment was not desirable, allotment overrides could be used to force a different allocation of component supply. For example, suppose a requirement to share the earlier supply of component Y evenly between demands from customer PCW and demands from customer TBV.

This could be accomplished by first creating two AllotmentOverride records, one for each customer and site combination as shown in the following table. Note that Sequence values are used to indicate overrides for customer TBV should be processed before those for customer PCW.

Table 40.6: AllotmentOverride values

Id	Sequence	Type.ProcessingRule
PCW:HQ	1	AfterRunDate
TBV:HQ	0	AfterRunDate

Next, records are added to the AllotmentOverrideDetail table to reserve the required component supply of 100 units of Y to each customers allotment override.

AllotmentOverrideDetail values

AllotmentOverride.Id	Component.Name	Component.Site	Date	Quantity
PCW:HQ	Y	HQ	Period1	100
TBV:HQ	Y	HQ	Period1	100

Finally, each customer's IndependentDemand records should be set to reference their associated AllotmentOverride record. IndependentDemands with the order for part B would be updated so that its AllotmentOverride reference field points to the newly created AllotmentOverride. This gives the modified on-time quantities of demands as shown in the following table with each customer's demands receive 100 units of the on time supply of component Y. Note that this allotment results in two fully on-time orders as compared to one previously.

Table 40.7: On-time quantities of independent demands after allotment overrides

Part.Name	Part.Site	Order.Customer	AllotmentOverride.Id	Due	Quantity	OnTime	Late
A	HQ	PCW	PCW:HQ	Period2	70	70	0
B	HQ	PCW	PCW:HQ	Period3	150	30	120
A	HQ	TBV	TBV:HQ	Period4	80	80	0
A	HQ	PCW	PCW:HQ	Period5	60	0	60
B	HQ	TBV	TBV:HQ	Period6	140	20	120

CHAPTER 41: Multi-level search logic

Multi-level search logic is useful for parts where better part sourcing or substitution decisions could be made if knowledge of child component materials and constraints were known, as well as for parts where better supply allotment decisions could be made if knowledge of sibling component allocations were known. These decisions might then typically be reflected in improvements in on time demand availability.

Typically, when choosing between multiple part sources, as well as when making part substitution decisions, RapidResponse does not consider constraint and material availability on components under the part being sourced. For example, when choosing a part source, source effectivity, constraints, PTF date, and sourcing rules are considered, but unlimited component availability is assumed. However, when multi-level search logic is enabled for a part, supply sourcing decisions are expanded to consider component materials and constraints in order to help choose the path that will provide the supply either on time or least late.

As well, when RapidResponse allocates a component's supplies to demands, it typically considers factors such as demand due dates, supply available dates, and order priorities, but the allotments made on any sibling components are not considered. However, when multi-level search logic is enabled for a part, sibling component allotments are taken into account. This allows all sibling component's allotments to be synchronized with the supply of the gating component (if any) and allows earlier supply of those components to be re-directed to other demands and potentially improve overall availability.

This chapter discusses the tables and fields used to support multi-level search logic, and its impact on part source selection, global and BOM-level part substitution, as well as the supply allotment process.

You will learn about the following:

Multi-level search configuration **1863**
Multi-sourcing **1876**
Substitution **1909**
Non-blocking allocations **1916**
Limitations on other analytics **1926**
MLS infrastructure changes (optional and limited release) **1929**

Multi-level search configuration

This section provides an overview of the settings and configuration points that should be considered when using multi-level search logic.

The following is discussed:

- "Enabling multi-level search logic" on page 1864
- "Incremental availability" on page 1865
- "AvailableRule" on page 1865
- "Multi-level part source effectivity" on page 1865
- "Partial planning" on page 1866
- "Logging unsatisfied demands from partial planning" on page 1866
- "Fairshare of material and constraints" on page 1868
- "Limiting the number of part sources evaluated" on page 1870
- "Other control settings to consider" on page 1871
- "Pre-defined resources supporting MLS" on page 1873

Enabling multi-level search logic

Multi-level search logic is enabled at the part level. Each record in the **Part** table has a nullable reference to the **MultiLevelSearchRule** table. When used, this table controls multi-level search logic for referencing parts based on its **ProcessingRule** field, which has the following two values:

- **Ignore**—Multi-level search logic is disabled for all parts that reference a type using this option. This means that both part source selection and part substitution decisions are made without knowledge of planned component availabilities under the assembly being supplied, and supply allotment decisions for a given component are made without knowledge of the allocations made on any of its sibling components. This is the default setting in RapidResponse, and corresponds to the sourcing logic described in "Part source selection" on page 1377.
- **Use**—Multi-level search logic is enabled for all parts that reference a type using this option. This means that sourcing decisions made when choosing a part source or substitute to provide supply are modified to include considerations of the component materials and constraints under the part being sourced. This is intended to help pick the supply source that can provide the supply either on time or least late. Note that if setting this field to "Use" for an assembly, and a component in the middle of its structure is set to "Ignore", then sourcing decisions would consider component availability up to that component but not any of the components directly below it (regardless of their MultiLevelSearchRule setting). As well, the decisions made when allocating component supplies to demands are modified to include knowledge of sibling component allotments. When dealing with demands that cannot be fully supplied on time, this allows sibling component supply allotments to be synchronized with the availability of gating (most late) component in order to potentially free up earlier component supplies to satisfy other demands.

Note that enabling multi-level search logic can result in RapidResponse taking longer than normal to return calculated analytic results. This is particularly true when applied to a significant number of parts in large data sets. As well, multi-level search logic is incompatible with, or imposes limitations on, certain other RapidResponse analytics as discussed in "Limitations on other analytics" on page 1926.

Therefore, as possible, multi-level search logic should be limited to only those parts for which it will truly make a difference. Typically, this means the assembly parts whose demand availabilities you want to improve, as well as the component(s) whose materials or constraints affect the on time availabilities of those upper-level demands.

For example, if there is a requirement to consider constraint and material availability for all components at all levels under a given assembly, then multi-level search logic should be set for the assembly and all of its components. In other cases, though, it might be sufficient to just enable multi-level search logic down to a certain level or path in the structure.

Similarly, if attempting to change component supply allocations to improve availabilities on demands for a given part, it might be sufficient to just enable multi-level search logic for the part(s) whose demands are to be improved along with the specific component(s) who can affect the availabilities on those assembly demands.

Incremental availability

Parts that have multi-level search logic enabled should typically have incremental availability turned on. That is, their IncrementalRule.ProcessingRule value should be set to "On".

This ensures the benefits of multi-level search logic can be reflected on higher-level demands by allowing planned orders to be split and incremental available dates reported and rolled up to demands as portions of a late supply become available. For more information, see "IncrementalRule table" on page 741.

AvailableRule

Parts that have multi-level search logic enabled should also typically have their applicable AvailableRule.ProcessingRule or Type.AvailableRule value set to "Normal". With this setting, available dates are calculated using all applicable capable-to-promise rules and settings.

Use of the "Normal" setting is required to ensure multi-level search logic is used in analytic calculations for the part. If AvailableRule is set to "Ignore" for a given part, then multi-level search logic will be automatically disabled for that part regardless of its MultiLevelSearchRule setting.

Multi-level part source effectivity

With either standard or multi-level sourcing logic, planned orders from a given source can only be due within the part source's effective range (in other words, cannot be due before **PartSource.FirstEffectiveDate** or after **PartSource.LastEffectiveDate**). If a demand falls outside of effectivity for all eligible part sources, then the demand can be shifted into an effective range (dependent on the **SourceRule.DateEffectivityRule** setting).

When using multi-level search logic, an additional control setting allows specifying whether planned availability on an assembly's components should affect the determination of part source effectivity for the assembly itself. If planned component availability is determined to be outside of assembly part source effectivity, then the part source will not be selected (even if this means the demand is made late or goes unsatisfied.
This logic is configured through the **MultiLevelSearchRule.SourceEffectivityRule** field which has the following options:

- **DueDate**— effective dates on an assembly part source are compared against the planned order due date only.

  Thus, planned component availability, based on factors such as planning time fence and constraint, does not impact whether a part source is considered effective (but can still impact which effective part source is ultimately be chosen to supply a given demand). This is the default setting.

- **PlannedAvailableDate**— effective dates on an assembly part source are compared against the planned order due date, as well as against the planned available date determined by multi-level search logic (based on factors such as planning time fence and constraint on the assembly's components).

  Thus, planned component availability, based on factors such as planning time fence and constraint, can impact which of an assembly's part sources are considered effective as well as which of the effective part sources are ultimately chosen to supply a given demand. Additionally, if planned component availability falls outside of effectivity for all of the assembly's part sources, then the demand will be left unsatisfied and an exception with **Type** set to "MLSSourceEffectivity" will be logged and reported in the **DemandException** table.

- **AvailableDate**—identical to the "PlannedAvailableDate" setting in how planned component availability is considered during multi-level part source selection.

  However, as an additional criterion, if planned component availability falls within assembly part source effectivity (as determined by Netting logic) but actual component availability (as determined in CTP logic) subsequently falls outside of assembly part source effectivity, then the demand should have its **AvailableDate** set to "Future" (however, no **DemandException** record is generated in this case)

Notes:

- For more information about the **DemandException** table in which demands left unsatisfied due to source effectivity are reported, see "DemandException table" on page 1028.
- For an example of how multi-level search logic can affect the determination of part source effectivity, see "Example 6: Impact of multi-level source effectivity" on page 1891.

Partial planning

The DemandType.PlanningRule field has a "Full" setting for demands that should always be satisfied and a "Partial" setting for demands that Netting logic should give up on and leave unsatisfied if they cannot be satisfied/planned within a given tolerance interval.

For parts using multi-level search logic, the planned availability for the demand, based on expected availability from component supplies at all levels, is considered in this determination of whether to give up on demand. That is, a demand might be left unsatisfied because multi-level search logic determines one or more component supplies will be planned too late to allow the demand to be satisfied within a specified tolerance interval. For example, component supplies might be planned too late due to lead time or constrained part sources.

Thus, unless availability of component supply is ensured throughout the product structure, multi-level search logic is generally recommended for use when enabling partial planning. For more information about partial planning, see "Ignoring unsatisfied demands" on page 1420.

Logging unsatisfied demands from partial planning

If parts using multi-level search logic have demands configured for partial planning, then any demand portions left unsatisfied due to partial planning, along with details of the component supplies that led to those demands going unsatisfied, can (optionally) be reported in a series of tables. Logging of unsatisfied demands is controlled through the **MultiLevelSearchRule.LoggingRule** field. With the default setting of "Ignore", details of unsatisfied demands from partial planningare not logged.

However, when **LoggingRule** is set to "UnsatisfiedDemand", then details of demands that were left unsatisfied due to partial planning and multi-level search logic are logged.

With the "UnsatisfiedDemand" setting, the following calculated tables can potentially be populated with details related to supplies that could not be planned to meet a specified lateness tolerance on demand:

- MLSActivity (see "MLSActivity table" on page 1131)
- TrialCoByProductSupply (see "TrialCoByProductSupply table " on page 1216)
- TrialConstraintShortage (see "TrialConstraintShortage table" on page 1218)
- TrialConsumption (see "TrialConsumption table" on page 1219)
- TrialPlannedOrder (see "TrialPlannedOrder table" on page 1220)
- TrialSubstituteSupply (see "TrialSubstituteSupply table" on page 1222)

For parts that use a MultilLevelSearchRule.LoggingRule of "UnsatisfiedDemand", the MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule further determines the portion of the unsatisfied demand and associated late supply portions that get logged. It has the following options:

- **FirstLatePortion**— this is the default setting, and results in only the first late supply portion that caused multi-level search logic to leave the demand unsatisfied are reported (along with that corresponding portion of the unsatisfied demand). For example, if a given component supply is too late to satisfy the demand within tolerance, and constraint causes that supply to be available in multiple portions/days then only the first of those portions making the demand late beyond tolerance is reported (subsequent or later portions would not be reported).
- **FullQuantityPrimaryPath**— the entire unsatisfied demand quantity is reported along with all supply portions that would be too late to satisfy it on time (or within lateness tolerance) are reported. Constraint consumption occurs as normal and, in case when there is a lack of constraint available, demand logging might not continue all the way to lower levels in the structure. Also, all supplies needed for the unsatisfied demand are assumed to be sourced from the primary part source determined by multi-level search logic.
- **FullQuantityPrimaryPathAfterNetting**— the unsatisfied demand quantity and supply portions late to satisfy it are reported after Netting logic completes and in the order that the demands were shorted. On-time input supply consumption is allowed, but only supplies leftover from the usual netting process can be used. Constraint consumption is disabled during the downwards pass (thus constraint availability does not change where dependent demands get exploded). Also, all supplies needed for the unsatisfied demand are assumed to be sourced from the primary part source determined by multi-level search logic.

Primary part source used in MLS logging

Note also that with either of the "FullQuantity" settings, the "primary" part source used when logging details of unsatisfied demands is determined as follows.

If there is one or more effective part sources then the following criteria is used to determine the primary:

1. Priority (highest, based on PriorityRule)
2. MLSPreTargetSequence (lowest value)
3. Target (highest value)
4. SecondaryPriority (highest,based on PriorityRule)
5. BaseKey (lowest value)
6. Supplier (lowest value)
7. Source (lowest value)

If there aren't any effective part sources, but one or more effective substitute sources, then the following criteria is used to determine the primary:

1. Sequence (lowest value)
2. Primary preferred to substitute
3. Type.PoolRule (preference for "Use")
4. FirstEffectiveDate (earliest)
5. SubstitutePart.Name (lowest value)
6. SubstitutePart.Site (lowest value)
7. Not using flow-to-common

Otherwise, if there aren't any effective part sources or any effective substitute sources, then the following criteria is used to determine the primary from the non-effective part sources:

1. Latest EffectiveOutDate effective before order date
2. Earliest EffectiveInDate effective after order date

Fairshare of material and constraints

Demands within a given priority level are typically processed in due date sequence. Thus earlier demands will typically be processed first and can be planned to use limited material and/or constraint throughout the product structure. If multi-level search is used together with partial planning logic, this means earlier demands within a given tolerance interval can be planned first and use up potentially limited material and/or constraints, while later demands within the tolerance interval will be left disproportionately short when not all demands can be planned.

To allow a more equitable treatment of demands, multi-level share logic can be applied in Netting calculations. This ensures that when demands of a given priority level must be left unsatisfied (left short), the shortness will be fairly distributed amongst those demands within a given interval. To configure multi-level fair share logic, the following fields on the OrderPriority table should be used.

Table 41.1: OrderPriority fields used to configure multi-level fair share logic

Field	Description
MLSAllocationRule	Determines how demands of a given priority level are processed in Netting when multi-level search logic is enabled. Used to enable or disable multi-level fair share logic based on the following options: • **FirstComeFirstServed**—Demands of a given priority level are processed individually and in due date sequence. In other words, fair share logic is disabled (this is the default setting). • **Fairshare**—Demands of a given priority level are broken up into smaller pieces, based on demand size, and these smaller pieces are processed in an alternating fashion from earliest to latest. In other words, fair share logic is enabled and limited component materials or constraint should be more fairly shared between prioritized demands.
MLSFairshareCalendar	A reference to the calendar used to define the period over which demands of a given priority level are processed together in multi-level search logic. For example, all demands due in each Week might be processed together and any shortages fairly shared between those weekly demands (if left undefined, default Everyday calendar is used).

Table 41.1: OrderPriority fields used to configure multi-level fair share logic

Field	Description
MLSFairShareSliceFactor	Specifies the number of incremental portions into which demands of a given priority level are split for processing by multi-level search logic. For example, if set to "3", and there were demands of 30 and 45, they would be processed in three increments of 10 and 15 respectively. Values between 1 and 10 should be specified (a value less than 1 is treated as 1, and a value greater than 10 is treated as 10).

For examples of fair sharing within multi-level search logic, see "Example 9: MLS FairShare (with material shortage)" on page 1901 and "Example 10: MLS FairShare (with limited constraint)" on page 1904.

Note: When multi-level search logic is not used, then fair sharing of supplies is configured through the **OrderPriority.AllocationType**, **PlanningCalendars.AllocationCalendar**, and **PartType.SupplyShare** fields. Similarly, fair-sharing of constraints without multi-level search logic is set through the **OrderPriority.ConstraintAllocationRule** and **PartType.ConstraintShareRule** fields. However, when multi-level search logic is used then the fields noted in the table above should be used for fair-sharing of supplies and/or constraint.

Including material cost as a selection criteria

When choosing between part sources to supply a requirement, the total cost of purchased materials needed might vary depending on the source chosen. For example, if a given part source has relatively low yields or expensive components in its structure, then the total rolled up cost from using that source to supply a demand might be greater than other potential sources. For such cases, multi-level search logic can be configured to give preference to sources that provide supply with a lower rolled up material purchase cost. This can be used to differentiate between sources of the same priority that can otherwise provide the assembly supply on-time and that have equivalent component supply types/depth.

If material purchase cost should be used in the multi-level source selection process, the **NewPurchasesCost** field on the **MultiLevelSearchRule** control table can be used. If using this functionality, ensure that appropriate **PartSource.MaterialCost** values are provided for sources providing purchased parts throughout the structure. The **MaterialCost** field and supply quantities are the main factors used in determining total purchase cost.

The **MultiLevelSearchRule.NewPurchasesCost** field has the following options:

- **Ignore**— Part source selection criteria does not include rolled up material cost from purchase orders. This is the default setting.
- **UseBeforeTarget**— Part source selection criteria considers rolled up material costs from purchase orders before it considers **MLSPreTargetSequence** and **Target** values. That is, if two part sources of the same priority can provide supply equally well and component supply types/depth are equivalent, then the one with the lower rolled-up material cost from component purchase orders will be used to supply the requirement. If material costs from new purchase orders are equivalent, then the source that has the lowest material cost from input supplies (such as, scheduled receipts) and existing planned orders is preferred.

Note: For an example of how material purchase cost is used in multi-level search logic, see "Example 7: Impact of new purchases cost" on page 1893.

Reducing short-term excess inventory

Short-term excess inventory can result from the pre-building of supply due to order priority, dynamic safety stock, lot-sizing, and/or constrained planning. When using multi-level search logic, the **MultiLevelSearchRule.ShortTermExcess** setting can be used to limit the amount of excess supply held in the short term.

The **ShortTermExcess** field has two settings; "Allow" and "Reduce". The default setting is "Allow" which allows excess inventory to potentially build up in the short term as it otherwise might from certain planning logic in Netting calculations.

Instead if using the "Reduce" setting, multi-level search logic might pull in or delay supplies in order to reduce short-term excess inventory. This occurs in two passes of balancing supplies and demands.

During the first pass, supplies are sorted by the date multi-level search expects them to be available, and demands are sorted by due date. The following potential impacts on planning results might occur:

- Dependent demand dates are updated as planned orders on higher levels are planned on different dates and with different start dates.
- Planned order dates changes as supplies are balanced based on demands.
- Planned order priorities are raised based on the demand covered by the supply.

During the second pass, supplies are again sorted by the date multi-level search expects them to be available, and demands are sorted by request date. The following potential impact on planning results might occur:

- Planned order request dates (used in two-date planning logic) are updated based on the demand being satisfied. If multi-level search logic determines the planned order will be late, the request date is then limited to the DemandDate of the planned order.

Additionally with the "Reduce" setting, planned orders might sometimes be split to allow further reductions in excess supply. For example, this might occur if temporary excess supply occurs when pulling in planned orders to satisfy demand and the order being pulled in is more than needed.

Notes:

- For an example of the impact of the **ShortTermExcess** setting, see "Example 11: Reduction of short-term excess inventory" on page 1907.
- When using the **ShortTermExcess** setting of "Reduce", the pulling in or delaying of planned orders cannot move them outside of their part source's effectivity. Thus in cases of multiple part sources with short date effectivity ranges, the use of the "Reduce" setting can potentially introduce temporary excess supply and/or shortages.

Limiting the number of part sources evaluated

When multi-level search logic is enabled, the criteria for selecting a part source to supply a given demand is as outlined in "Multi-level part source selection criteria" on page 1877.

If necessary, an upper limit can be placed on the number of effective part sources that are potentially evaluated by multi-level search logic when planning supply to satisfy each demand for a given part. This is set through the **MultiLevelSearchRule.PartSourceLimit** control field.

Because the number of effective part sources is typically proportional to the number of trials performed by multi-level search logic, placing a limit on the number of part sources can improve run time performance. In particular, if a part has a large number of low-yielding part sources, then placing a practical limit on the number that can be evaluated will typically improve performance of the multi-level search analytic while having limited impact on solution quality.

To set a limit on the number of effective part sources considered, provide the appropriate positive value in the **PartSourceLimit** field. For example, with the default setting of 100, no more than 100 part sources having **EffectiveInDate** and **EffectiveOutDate** values that bound the demand date will be evaluated by multi-level search logic when planning the demand. Note that when a part source limit is applied, the following criteria determines the order in which part sources are sorted and hence evaluated for suitability in satisfying a demand:

1. Lowest **PartSource.Priority** value (that is, higher priority sources are considered first)
2. Lowest **PartSource.MLSPreTarrgetSequence** value
3. Highest **PartSource.Target** value
4. Earliest **PartSource.EffectiveInDate** value
5. Lowest (internal) **PartSource.RecordId** value

Otherwise, if no limit should be placed on the number of effective part sources considered, the **PartSourceLimit** field should set to zero (0) or any negative value.

Note: The **PartSourceLimit** setting was added in Version 2016.2, Service Update 17. That Service Update also includes other performance improvements in the multi-level search analytic and related bug fixes as discussed in "Defect fixes and calculation changes" on page 2331.

Other control settings to consider

Additionally, the following other control settings should also be considered when implementing multi-level search logic:

Table 41.2: Control settings to consider when enabling multi-level search logic

Control setting	Detail
PartType.AllocationQuantityRule	Parts that have multi-level search logic enabled are compatible with this setting which, when set to “Use”, includes demand size in the criteria for allocating a part’s limited component supplies (after due date, priority, and so on). However, when this option is set to “Use” for a single part that has multi-level search logic enabled, all other parts processed with that part are automatically treated as being set to “Use” as well. For example, if this rule were set to “Use” for just a single component configured for non-blocking allocations under an assembly, then its related sibling components would inherit this rule as well. Therefore, to disable this rule for a component that has multi-level search logic, ensure it is disabled on all other parts processed with that component.
PartSourceType.ConstraintMultipleRule	For parts that use multi-level search logic along with constraints and also have lot-sizing requirements, the **ConstraintMultipleRule** setting of "Use" should be used. This ensures that the multiple quantity is properly respected (it might not always be if **ConstraintMultipleRule** is set to "MultiplePeriods"). Additionally, the "Use" setting can help mitigate against the possibiilty of planned orders sometimes being created below the specified minimum quantity. For more information on this limitation, see "Lot-sizing with constraints" on page 1929
ShipmentPolicy.PartialRule	For parts that have multi-level search logic enabled, this setting controls how non-blocking allocations are used under a given independent demand. If an independent demand references the default ShipmentPolicy.PartialRule of “Partial”, which indicates the order is eligible for partial shipments, then component supply allocations are synchronized to the due date of the demand as possible. As a result, when a demand cannot be satisfied on time, earlier component supplies can potentially be released to satisfy other orders for whatever portion of the demand cannot be satisfied on time. Otherwise, if the independent demand references a ShipmentPolicy.PartialRule of “Complete”, then component supply allocations are synchronized to the calculated available date for the demand. As a result, when a demand cannot be satisfied on time, all earlier component supplies can potentially be released to satisfy other orders. If an independent demand belongs to a ship set, this setting has no impact (always treated as “Complete”).

Table 41.2: Control settings to consider when enabling multi-level search logic

Control setting	Detail
ShipSetType.MultiLevelSearchRule	If multi-level search logic is enabled for parts with demands in a ship set, then this rule determines whether non-blocking allocations are considered across the demands in a ship set. When a ship set has its Type.MultiLevelSearchRule set to “Use”, then component allocations under demands within the ship set are synchronized with those of the latest available demand in the ship set. This allows earlier supplies to be released and potentially used to improve availabilities on demands outside of the ship set. Otherwise, if Type.MultiLevelSearchRule is set to “Ignore”, then non-blocking allocations are not considered at the ship set level.
SupplyStatus.AllowLatePlan	This setting controls whether supplies can be planned later than their associated demand dates. When **AllowLatePlan** is set to "N" (which is the default), then supplies are always planned to meet their demand date (although they might not always be available by that demand date). For part's that use multi-level search and have **AllowLatePlan** set to "Y", then a part's planned order due dates can be set to the date when the supply is realistically expected to be available based on any expected lateness of its components due to material availability and/or constraint. Thus if multi-level search logic determines a supply will not the demand date, its due date can be set later than the demand date (otherwise, the due date will still be planned to satisfy the demand date). **Note:** For parts that do not use multi-level search, the **AllowLatePlan** setting of "Y" only allows the late planning of a part's supplies when there is insufficient constraint available to it.

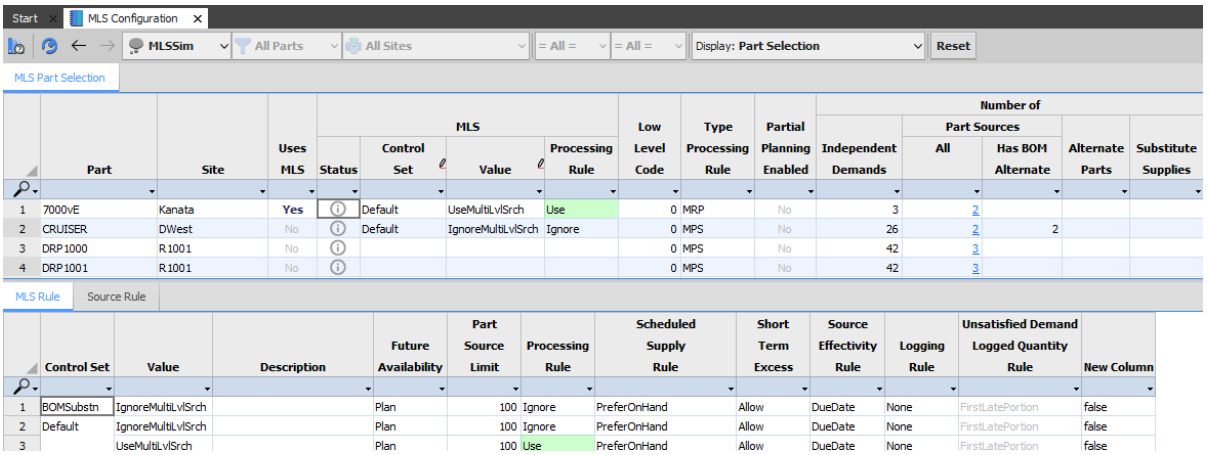
Pre-defined resources supporting MLS

Several pre-defined resources are available that can assist with enabling and/or measuring the impacts of multi-level search logic. This includes the following:

- "MLS Configuration workbook" on page 1873
- "Control Tables workbook" on page 1875
- MLS Performance Manager dashboard

MLS Configuration workbook

The **MLS Configuration** workbook can be used to identify parts that currently have multi-level search enabled or which might benefit from enabling multi-level search. This workbook allows you to view the configuration points for multi-level search functionality which are discussed throughout this section. It also provides insight into potential data issues or settings that might adversely impact multi-level search calculations.



	Part	Site	Uses MLS	MLS Status	MLS Control Set	MLS Value	MLS Processing Rule	Low Level Code	Type Processing Rule	Partial Planning Enabled	Number of Independent Demands	Number of Part Sources All	Number of Part Sources Has BOM Alternate	Number of Alternate Parts	Number of Substitute Supplies
1	7000vE	Kanata	Yes		Default	UseMultiLvlSrch	Use	0	MRP	No	3	2			
2	CRUISER	DWest	No		Default	IgnoreMultiLvlSrch	Ignore	0	MPS	No	26	2	2		
3	DRP1000	R1001	No					0	MPS	No	42	3			
4	DRP1001	R1001	No					0	MPS	No	42	3			

	Control Set	Value	Description	Future Availability	Part Source Limit	Processing Rule	Scheduled Supply Rule	Short Term Excess	Source Effectivity Rule	Logging Rule	Unsatisfied Demand Logged Quantity Rule	New Column
1	BOMSubstn	IgnoreMultiLvlSrch		Plan	100	Ignore	PreferOnHand	Allow	DueDate	None	FirstLatePortion	false
2	Default	IgnoreMultiLvlSrch		Plan	100	Ignore	PreferOnHand	Allow	DueDate	None	FirstLatePortion	false
3		UseMultiLvlSrch		Plan	100	Use	PreferOnHand	Allow	DueDate	None	FirstLatePortion	false

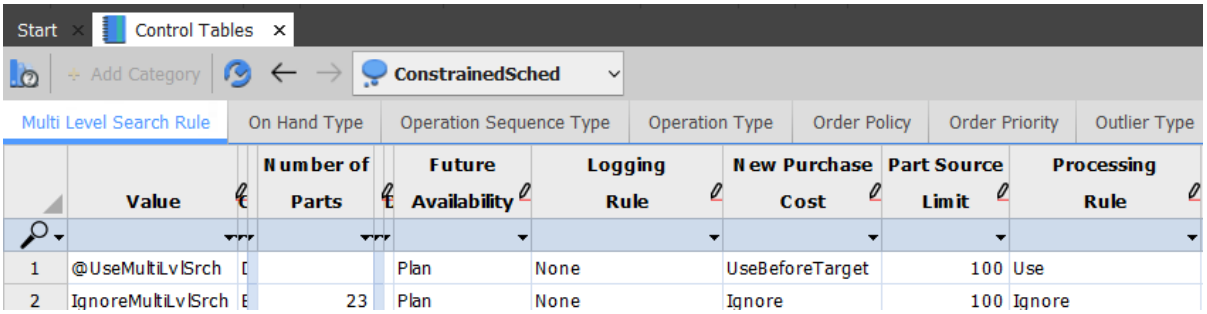
The following main worksheets are included in the **MLS Configuration** workbook:

- **MLS Part Selection**: This worksheet can report all parts that have multi-level search logic enabled. It can also be used to identify parts that don't currently use multi-level search logic but which might benefit from using it. For example, parts that have many part sources or which have substitute parts might see better planning results when muti-level search logic is enabled. When this worksheet is selected, the **MLS Rule** worksheet displays in the lower workbook pane.
  - **MLS Rule**: This worksheet shows all records in the **MultiLevelSearchRule** control table and the settings on each. Parts can have their **MultiLevelSearchRule** setting changed using the **MLS Part Selection** worksheet.
- **MLS Part Status:** This worksheet can be used to identify parts with configuration issues that might adversely impact multi-level search results or performance. For example, if a part uses certain control settings that are not compatible or recommended for use with multi-level search logic these are reported.
- **MLS Configuration**: This worksheet shows certain global configuration settings from the **AnalyticsConfiguration** table along with details on multi-level search trial operations.
- **MLS Activity Log**: This worksheet reports details about component supplies that expected to be late to satisfy a top-level demand (independent demand or forecast).
- **Recursive Path**: This worksheet can be used to identify recursive data structures where a part depends on itself (for example, if a part is found as a component within its own product structure). Recursive data structures should be avoided and fixed if found as they can cause RapidResponse analytics to get into endless loops or produce incomplete results.
- **MLS Cloning**: This worksheet allows enabling improved technical infrastructure supporting multi-level search logic in Service Update 1908. This new infrastructure provides generally improved performance and scalability with multi-level search and, in some specific cases, may produce better solution quality. If enabling this capability or encountering any issues with its usage, please contact Kinaxis Customer Support. Note that this new infrastructure will become the default and only option in a future service update.

Note: For more information about the **MLS Configuration** workbook, see the *RapidResponse Applications Guide*.

Control Tables workbook

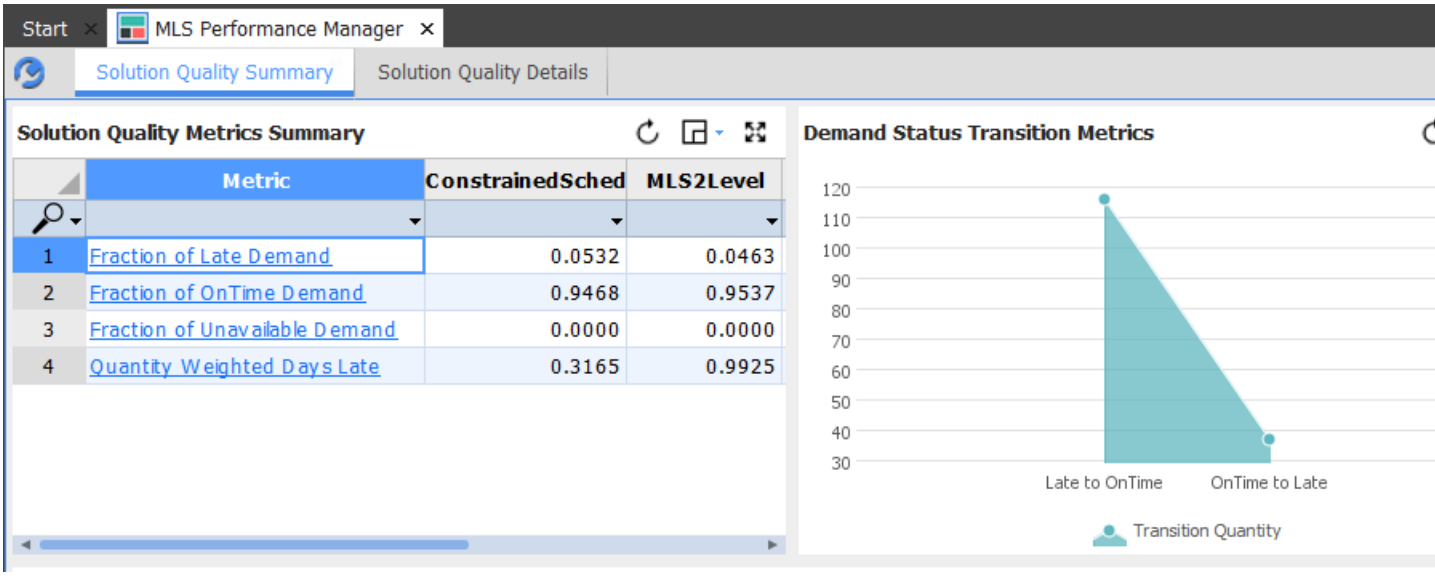
The **Control Tables** workbook shows all records and settings in the **MultiLevelSearchRule** table for any given scenario. Additional worksheets in this workbook always provides access to many of the other relevant configuration points and settings discussed throughout this section.



	Value	Number of Parts	Future Availability	Logging Rule	New Purchase Cost	Part Source Limit	Processing Rule
1	@UseMultiLvlSrch		Plan	None	UseBeforeTarget	100	Use
2	IgnoreMultiLvlSrch	23	Plan	None	Ignore	100	Ignore

MLS Performance Manager dashboard

The **MLS Performance Manager** dashboard allows comparison of solution quality between two selected scenarios. This can be useful in determining any potential benefits from enabling multi-level search on certain parts. For example, you can compare the number of late orders in each bucket between a scenario that doesn't use multi-level search logic and one that has it enabled on select parts.



	Metric	ConstrainedSched	MLS2Level
1	Fraction of Late Demand	0.0532	0.0463
2	Fraction of OnTime Demand	0.9468	0.9537
3	Fraction of Unavailable Demand	0.0000	0.0000
4	Quantity Weighted Days Late	0.3165	0.9925

The following main reports are included in the **MLS Performance Manager** dashboard. Each report presents summary information and allow drilling down for more detail on issues pertaining to a specific metric, priority level, or time bucket.

- **Solution Quality Metrics Summary**— Contains various demand fulfillment metrics that allow comparing the percentage of demands that are expected to on-time, late, and left unsatisfied

between the two scenarios. A weighted metric based on how many days late the late demands are is also presented.

- **Demand Status Transition Metrics**— Shows demands whose availability status changed between the two scenarios. For example, you can identify demands that were late in one scenario but on time in another or demands that were late in one scenario but left unsatisfied in another.
- **Demand Order Status by Due Date**—Shows demands grouped into specified buckets and identifies the number on-time, late, or unsatisfied demands per scenario in each bucket.
- **OnTime Demand by Priority**—Compares number of on-time demands at each order priority level between the two scenarios.
- **Late Demand by Priority**—Compares number of late demands at each order priority level between the two scenarios.
- **Unavailable Demand by Priority**—Compares number of demands left unsatisfied at each order priority level between the two scenarios.
- **Categorized Late Quantity**—Reports supplies expected to be late to their demands in each of the scenarios, broken down into four configurable date buckets.

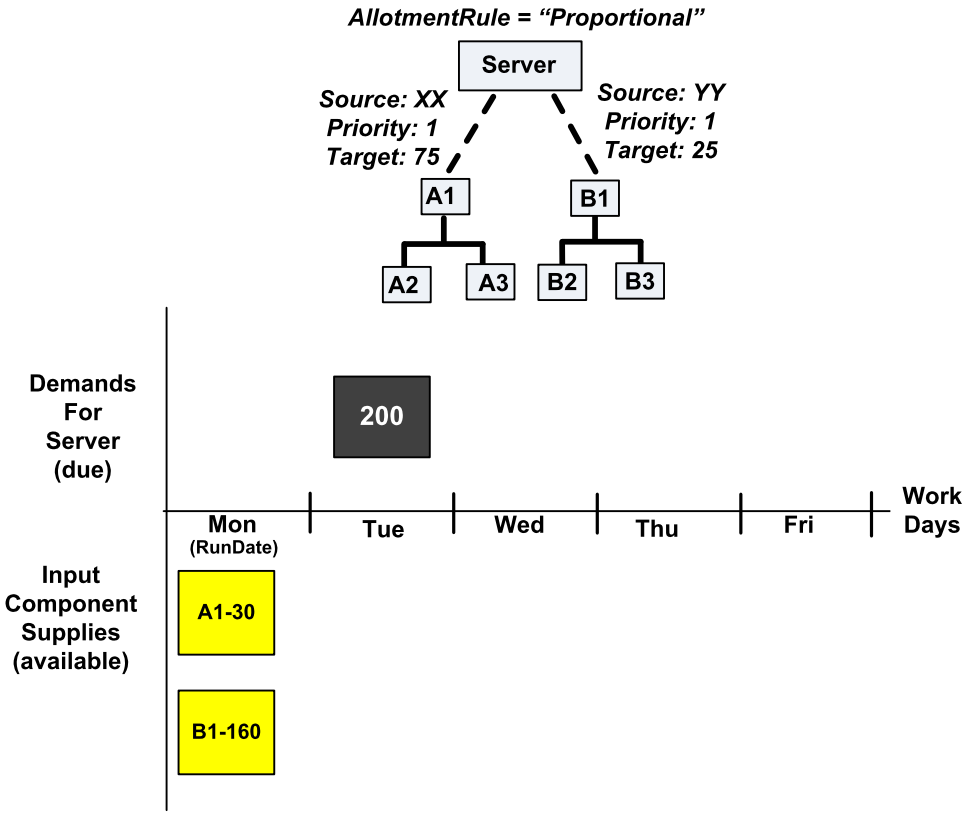
Multi-sourcing

In cases where there is insufficient on hand inventory or scheduled supply available to meet demand, which requires planned orders to be generated, RapidResponse chooses a part source to supply those orders. When choosing from multiple part sources, RapidResponse considers part source effectivity, planning time fence, and constraint associated with the assembly part being sourced as well as the part's sourcing rules as discussed in "Part source selection" on page 1377.

However, constraint and material availability on component parts are not considered during this process as RapidResponse assumes unlimited component availability during standard part source selection. In some cases, this can lead to potentially undesirable planning results.

For example, assume a part with two Make part sources (set up using alternate BOMs) that has a demand and available component supplies as shown in the following illustration. Further assume all parts have a lead time of one day (that is, the earliest a new order could be made available entirely from planned orders at levels of the structure would be Thursday).

Figure 41.1 Situation that might be improved with multi-level part source selection



Using the standard part source selection logic, 150 units would be sourced from Source XX and 50 units would be sourced from Source YY (in accordance with their Target values). With this source allotment, all 50 units sourced from YY would be on time but only 30 of the units sourced from XX would be on time with the remaining 120 late and not available until Thursday (due to the need to create planned orders on all of its lower-level components to satisfy those remaining 120 units). However, if component availability were considered then on time availability could be improved by sourcing more of the order from YY which has unused input component supply available. Potentially, 160 units could be sourced on time from YY which would leave 4o units to be sourced from XX only 10 of which would be late.

Multi-level part source selection criteria

When making part source selection decisions for parts that have multi-level search logic enabled, RapidResponse considers part source effectivity, planning time fence, and constraint associated with the assembly part being sourced as well as the part's sourcing rules as discussed in "Part source selection" on page 1377. In addition, the assembly part's component constraints and availability are considered in order to try to ensure sourcing decisions provide orders on time (or as close to on time as possible) while also potentially using up existing input component supplies before generating new planned orders.

The following table outlines the basic criteria used by RapidResponse when making multi-level part source selection decisions. For examples of how these criteria might be used when selecting which of

multiple part sources to place new planned orders against, see "Examples of multi-level part source selection" on page 1881.

Table 41.3: Multi-level part source selection criteria

	Selection Criteria	Description
1	On time order availability	When selecting a part source to supply a planned order, RapidResponse prefers those sources which, considering all component availability, can create the planned order on time. All sources that can create the planned order on time are considered equal in this criteria (no special consideration is given to sources which can supply the order early). If no part source can provide planned orders on time to satisfy a given requirement, then RapidResponse prefers those sources which can satisfy the requirement the least number of days late. If multiple part sources can satisfy a requirement on time (or least late), then subsequent criteria are used to choose between those sources.
2	Part source priority	If multiple part sources are eligible to satisfy a requirement on time (or least late), then RapidResponse prefers part sources with higher effective priorities to those with lower effective priorities. That is, it will source as much as it can from a higher priority source before considering lower priority sources. Note that priority is determined from **PartSource.Priority** with lower values in this field, by default, equating to higher effective priorities (but this is configurable based on the **SourceRule.PriorityRule** setting). If there are multiple part sources of a given priority eligible to provide a supply, then subsequent potential criteria are evaluated, as detailed through the rest of this table, until the part source(s) that should provide the supply can be chosen.
3	Component supply level (input supply only)	If multiple part sources of a given priority level are eligible to satisfy a requirement, and the lowest level supply being supplied comes fully from input supply, then preference is given to the source that can provides that input supply from a higher level in the product structure. This criteria is intended to potentially reduce the number of new orders that are required to assemble lower level components throughout the structure.

Table 41.3: Multi-level part source selection criteria

	Selection Criteria	Description
4	Component supply type (bottom level component)	If all prior criteria have been evaluated and multiple part sources are still considered equal, then the lowest level of component supply used in each source's path is compared. Preference is then given to sources based on the supply type used at that bottom level along with the **MultiLevelSearchRule.ScheduledSupplyRule** setting as follows: If **ScheduledSupplyRule** is set to "PreferOnHand" (default setting), then component supply is preferred in this order: 1. On Hand Inventory 2. Scheduled Supply (scheduled receipts, inventory transfers, supply allocations, and/or co-products) 3. Planned Orders Otherwise, if **ScheduledSupplyRule** is set to "EqualToOnHand", then component supply is preferred in this order: 1. On Hand Inventory or Scheduled Supply (scheduled receipts, inventory transfers, supply allocations, and/or co-products). 2. Planned orders **Note:** This part of the selection criteria is intended to try and use up on hand and scheduled component supply before creating new planned orders. If a mix of component supply types is used at the bottom component level under a source, then that level is assumed to be the least preferred of those supply types. In other words, "On Hand Inventory" means all component supply at the bottom level must be from on hand, "Scheduled Supply" means all component supply at the bottom level must be from either scheduled supply or a mix of scheduled supply and on hand, and "Planned Orders" means any or all supply at the bottom level is from planned orders.
5	Scheduled receipts (substitution & pooling)	If all prior criteria have been evaluated and multiple part sources are still considered equal, then sources that can use scheduled receipts at any component level (including unallocated scheduled receipts) are preferred over sources requiring new planned orders. This might help fully allocate or use up existing scheduled receipts before planning new supplies. This criterion only applies when using multi-level search with either global part substitution or inventory pooling logic as follows: • When using global part substitution and if **PartType.SubstitutionPreference** is set to "ScheduledSupply", a source that can use component supply from a scheduled receipt, even if is unallocated, is then preferred over one that requires a new planned order. • Similarly when using inventory pooling logic with **MUEPoolNettingType.PoolRule** set to "Net", if multiple sources can satisfy a demand equally well and are considered equivalent in priority and depth/level, then a source that can utilize an unallocated scheduled receipt in one pool (for example, the common pool) is preferred over a sources that requires a new planned order to be created in another pool (for example, the demand pool). **NOTE**: This criterion was added in Service Update G1903 and supports better and more consistent use of scheduled receipts when using multi-level search with global part substitution and/or inventory pooling logic.

Table 41.3: Multi-level part source selection criteria

	Selection Criteria	Description
6	Planned expiry date (material expiry)	When material expiry logic is used and multiple sources are otherwise equal, the date that supply from each is planned/expected to expire (including consideration for expiration of its components) is then evaluated. Sources that can provide supply with an earlier planned expiry date are then preferred. This is intended to help make sourcing decisions that minimize supply risk and reduce the total amount of component material that will ultimately expire. For an example of how this criterion is used in multi-level search logic, see "Example 8: MLS and material expiry " on page 1898.
7	New purchase cost	If all prior criteria have been evaluated and multiple sources are still considered equal, the source with the lowest material purchase cost is chosen. Note that this optional criteria applies only if **MultiLevelSearchRule.NewPurchasesCost** is set to "Use". Otherwise (if **MultiLevelSearchRule.NewPurchasesCosts** is set to "Ignore"), then the cost of new purchases is not part of the source selection criteria. To determine the cost associated with a new planned order for a purchased part, the order quantity (**PlannedOrder.Quantity**) is multiplied by the part source's material cost value (**PartSource.MaterialCost**). To determine the cost associated with a new planned order for a make or transfer part, the new purchase costs from its dependent demands are rolled up and summed together (that is, it is the sum of material costs from new purchase orders generated to satisfy the driving requirement). This part of the selection criteria is meant to help avoid selecting sources that have relatively lower yields and hence higher purchased component costs in their path. If the costs for new purchase costs are equal, then costs from existing input supplies and previously created planned supplies are considered next. For an example of how this criterion is used in multi-level part source selection, see "Example 7: Impact of new purchases cost" on page 1893
8	Existing supply cost	If all prior criteria have been evaluated and multiple sources are still considered equal, the source with the lowest rolled up cost from existing supply is chosen. Note that this optional criteria applies only if **MultiLevelSearchRule.NewPurchasesCost** is set to "Use". Otherwise, (if **MultiLevelSearchRule.NewPurchasesCosts** is set to "Ignore"), then the cost of existing supply is not part of the source selection criteria. For purchased parts, the determination of existing supply cost depends on the type of supply as follows: • **Scheduled receipts**—the quantity of the receipt used for the requirement is multiplied by **PartSource.MaterialCost**. • **Planned orders (existing)**—the quantity of the previously created order that is used for the requirement is multiplied by **PartSource.MaterialCost**. • **On hand inventory**—the quantity of inventory used for the requirement is multipled by **OnHand.UnitCost** (as long as the UnitCost is positive). Otherwise, the quantity is multiplied by the **MaterialCost** defined on the first effective **PartSource** record as determined from the dependent demand's due date. • **Co-products**—the quantity of co-product supply used for the requirement is multiplied by the **MaterialCost** defined on the first effective **PartSource** record as determined from the dependent demand's due date. For make and transfer parts, existing supply cost is based on rolled up existing supply costs from its dependent demands (that is, it is the sum of purchase costs from existing supply portions used to satisfy the driving requirement).

Table 41.3: Multi-level part source selection criteria

	Selection Criteria	Description
9	MLSPreTargetSequence	If all prior criteria have been evaluated and multiple sources are still considered equal, then the record with the lowest **PartSource.MLSPreTargetSequence** value is used to supply the requirement. The **MLSPreTargetSequence** field acts as an additional selection criterion indicating a preference between part sources of a given priority. For example, this field might be used to specify a preference between secondary part sources.
10	Part source target	If all prior criteria have been evaluated, and multiple part sources are still considered equal for sourcing a particular requirement, then their specified percentage splits are used to determine sourcing (as set on the **PartSource.Target** field). The part's **SourceRule.AllotmentRule** setting is also used to determine how those **Target** values are used when sourcing a requirement from multiple potential sources: • **Ongoing**— the outstanding requirement is fully allotted to the part source with the highest calculated Target quantity. • **Proportional**—the outstanding requirement is first allotted to the part source that has the highest remaining Target quantity up to the point where is Target is reached. Sourcing then shifts to the part source with the next highest remaining Target quantity remaining until the requirement is fully sourced. **Note: AllotmentRule** settings of "OngoingToDate" and "ProportionalLotSize" are not supported with multi-level search logic. If these settings are used with parts enabled for multi-level search logic, the "Ongoing" and "Proportional" settings are applied instead.
11	SecondaryPriority	In cases where, after applying targets, multiple sources are still considered equal for supplying a requirement, the **PartSource.SecondaryPriority** field is used to choose the source. For example, if the "ongoing" target for two sources is identical. By default, lower values in the **SecondaryPriority** field equate to higher effective priorities (but this is configurable based on the **SourceRule.PriorityRule** setting).

Examples of multi-level part source selection

This section shows the following examples of RapidResponse part source selection decisions made when multi-level search logic is enabled for the parts being sourced:

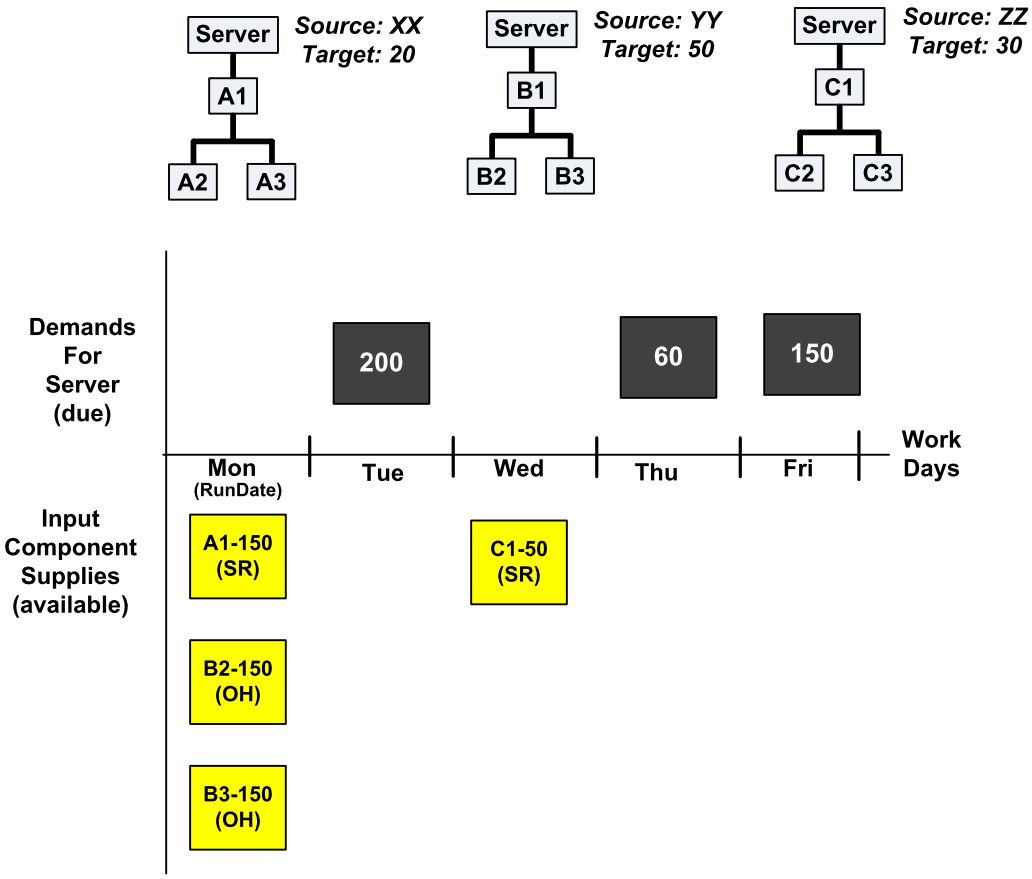
- "Example 1: Proportional rule (without priorities)" on page 1882
- "Example 2: Proportional rule (with priorities)" on page 1884
- "Example 3: Ongoing rule (without priorities)" on page 1886
- "Example 4: Ongoing rule (with priorities)" on page 1888
- "Example 5: Usage of MLSPreTargetSequence" on page 1889
- "Example 6: Impact of multi-level source effectivity" on page 1891
- "Example 7: Impact of new purchases cost" on page 1893
- "Example 8: MLS and material expiry " on page 1898

- Example 9: MLS FairShare (with material shortage)
- "Example 10: MLS FairShare (with limited constraint)" on page 1904
- "Example 11: Reduction of short-term excess inventory" on page 1907

Example 1: Proportional rule (without priorities)

Assume multiple "make" part sources (configured using alternate BOMs) for the part Server with sourcing rules as shown in the following illustration. Further assume that all parts have a 1 day lead time.

Figure 41.2 Targets with Proportional rule



Based on the set of Server demands and available component supplies shown in the previous illustration (with all scheduled receipts assumed to be fully allocated), the following table describes the source allotments made by part source.

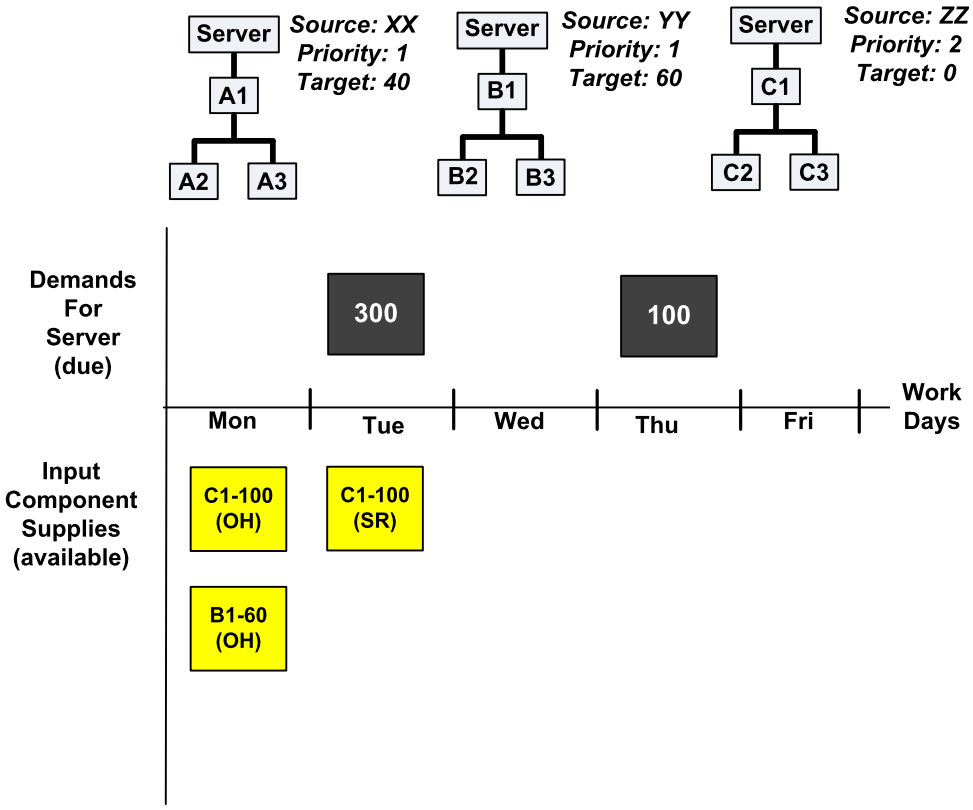
Table 41.4: Multi-level source selection results with Proportional rule

Demand Date	Source Allotments	Source Selection Details
Tuesday	XX: 150 YY: 50 ZZ: 0	• Source XX has component supply available early enough to satisfy the demand on time and so it is preferred to Source ZZ and Source YY, which do not. Thus Source XX is preferred and a planned order of 150 units is sourced from XX using all of the scheduled receipt for A1 as input. • The remainder of the demand is then sourced from YY as it can satisfy the demand the least late since component parts B2 and B3 are already on hand. A planned order of 50 units is created on B1 using 50 units each of the B2 and B3 on hand inventory as inputs.
Thursday	XX: 0 YY: 10 ZZ: 50	• All sources can potentially satisfy this demand on time, however only Source YY and Source ZZ have input supplies available and so are preferred to Source XX which requires planned orders at all levels in its structure to satisfy the demand. • Because the input component supply available under Source ZZ is at a higher level in the product structure than that available under Source YY, it is preferred and a planned order of 50 units is sourced from ZZ using all of the scheduled receipt for C1 as input. • The remainder of the demand is then sourced from YY using 10 units each of the B2 and B3 on hand inventory as input (planned order created on B1).
Friday	XX: 15 YY: 90 ZZ: 45	• All sources can potentially satisfy this demand on time, however only Source YY has any input component supply available and so it is preferred (Source XX and ZZ require planned orders on all components). A planned order of 90 units is sourced from YY using all the remaining on hand inventory of B2 and B3 (planned order created on B1). • The remainder of the demand is then sourced between XX and ZZ based on their target percentages (YY has already exceeded its target of 75 for this demand). ZZ is furthest away from its target so it is used first and a planned order for 45 units is sourced bringing it to its target for this demand. The remaining 15 units is then sourced from XX because it is the only source still under its target.

Example 2: Proportional rule (with priorities)

Assume multiple "make" part sources (configured using alternate BOMs) for the part Server with sourcing rules as shown in the following illustration. Further assume that all parts have a 1 day lead time.

Figure 41.3 Priorities and Targets with Proportional rule



Based on the set of Server demands and available component supplies shown in the previous illustration (with all scheduled receipts assumed to be fully allocated), the following table describes the source allotments made per part source

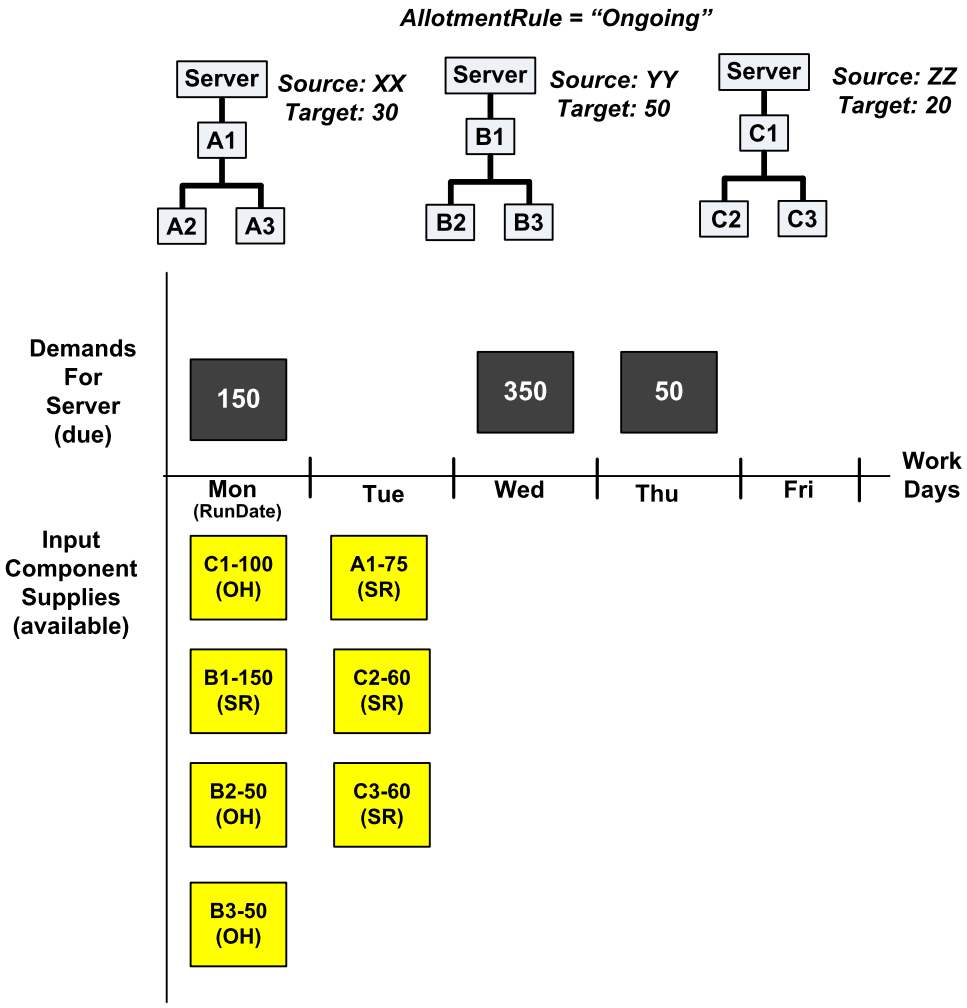
Table 41.5: Multi-level source selection results with Priorities and Proportional rule

Demand Date	Source Allotment	Source Selection Details
Tuesday	XX: 80 YY: 120 ZZ: 100	Only Source YY and Source ZZ are able to source supply of Server on time (due to having on hand component inventory), so they are preferred to Source XX which cannot provide any Server supply on time. • Source YY is preferred to Source ZZ because it has a higher effective priority and is used first with a planned order sourced from it for **60 units** (using up all of its available component inventory). • Next **100 units** are sourced from lower priority Source ZZ because none of the higher priority sources can satisfy any of the remaining demand on time (using up all of ZZ's available component inventory). The remaining 140 units of demand for Server cannot be sourced on time, but both higher priority sources Source XX and Source YY can satisfy it one day late. Each of these sources have component scheduled receipts, at the same level in their product structure, available Tuesday. Thus, their Targets are used to determine sourcing. • XX sources **80** units of the 140 unit requirement to reach its target of 40% of the total requirement satisfied by the higher priority sources (80/200 = 40). • YY sources **60** units of the 140 unit requirement to reach its target of 60% of the total requirement satisfied by the higher priority sources (120/200 = 60)
Thursday	XX: 40 YY: 60 ZZ: 0	All Sources can satisfy this demand on time, but Source XX and Source YY are preferred because they have higher effective priority (regardless of the fact that lower priority Source ZZ has unused component scheduled receipt quantities that could be used rather than generating new planned orders). • Source XX sources **40 units** in keeping with its target proportion. • Source YY sources **60 units**in keeping with its target proportion.

Example 3: Ongoing rule (without priorities)

Assume multiple "make" part sources (configured using alternate BOMs) for the part Server with sourcing rules as shown in the following illustration. Further assume that all parts have a 1 day lead time.

Figure 41.4 Targets with OnGoing rule



Based on the set of Server demands and available component supplies shown in the previous illustration (with all scheduled receipts assumed to be fully allocated), the following table describes the source allotments made per part source

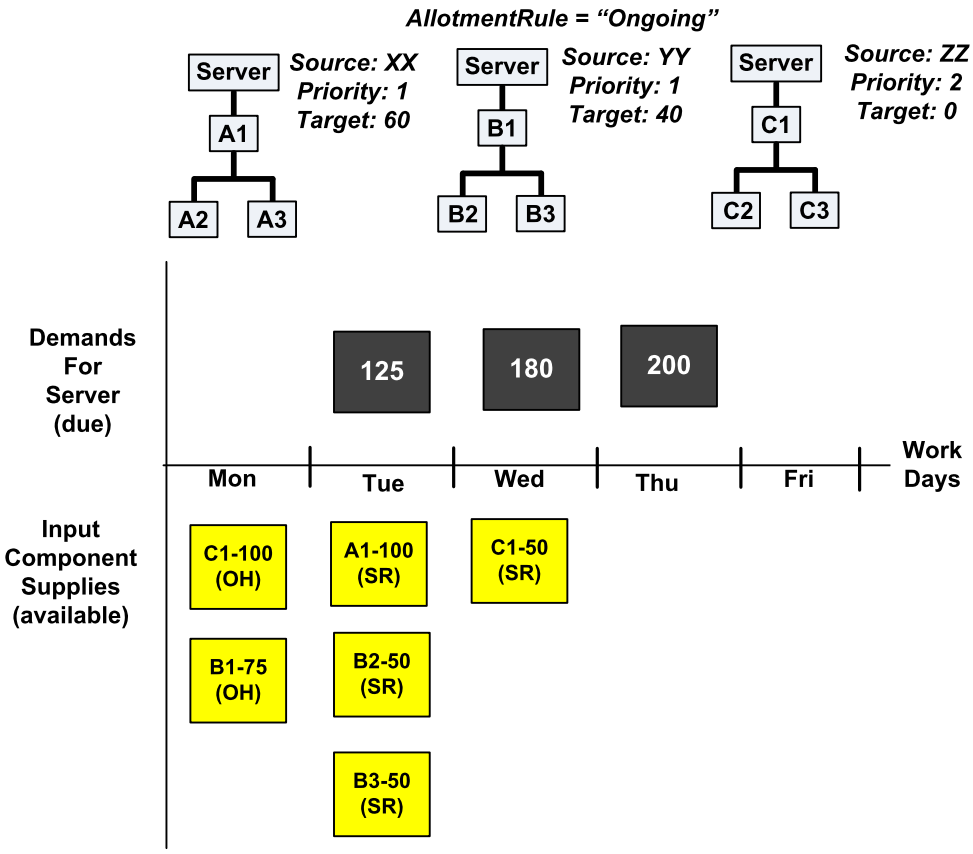
Table 41.6: Multi-level source selection results with OnGoing rule

Demand Date	Source Allotments	Source Selection Details
Monday	XX: 0 YY: 50 ZZ: 100	This demand cannot be satisfied on time from any source, so those Sources that can satisfy it least late are preferred. Both Source YY and Source ZZ have component supply available early enough to satisfy the demand one day late. • Source ZZ is used before YY because its available component supply comes from on hand inventory which is preferred to using component supply from scheduled receipts. **100 units** of Server are sourced using all available on hand supply of C1 as input. • Source YY is used to provide the remaining **50 units** of Server for this requirement, using a portion of the scheduled receipt for B1 as input.
Wednesday	XX: 75 YY: 215 ZZ: 60	Both Source XX and Source YY can satisfy some portion of this order on time, and each of these sources has a scheduled receipt available for the component directly under Server. • Source YY is preferred because it has a higher Target. **100 units** are sourced using up the remainder of the scheduled receipt for B1 as input. • Source XX, which has a lower Target value, is then used to source the next **75 units** using up the entire scheduled receipt for A1 as input. • Source YY also has scheduled receipts on lower level components C2 and C3. This is less preferred than using input supplies at a higher level in the structure but is the only option for providing more on time supply. An additional **50 units** are sourced from YY (planned order also created on B1). The remaining 125 units cannot be satisfied on time from any source, but each source can potentially provide supply one day late. • Source ZZ is used first because it is the only eligible source which still has input component supply remaining. It provides **60 units** using scheduled receipts on lower level components C2 and C3 as input (planned order also created on C1). • Because no more input component supply is available under any Source, the remaining quantity is allotted by Target. The final **65 units** are sourced from Source YY because it has the highest calculated Target quantity. Planned orders are created on all components under the structure.
Friday	XX: 0 YY: 50 ZZ: 0	• Each source can potentially satisfy this demand on time, and no input component supply is available under any structure. All **50units** are sourced from Source YY because it has the highest calculated Target quantity.

Example 4: Ongoing rule (with priorities)

Assume multiple "make" part sources (configured using alternate BOMs) for the part Server with sourcing rules as shown in the illustration below. Further assume that all parts have a 1 day lead time.

Figure 41.5 Priorities and Targets with OnGoing rule



Based on the set of Server demands and available component supplies shown in the previous illustration (with all scheduled receipts assumed to be fully allocated), the following table describes the source allotments made per part source.

Table 41.7: Source selection results with Priorities and Ongoing rule

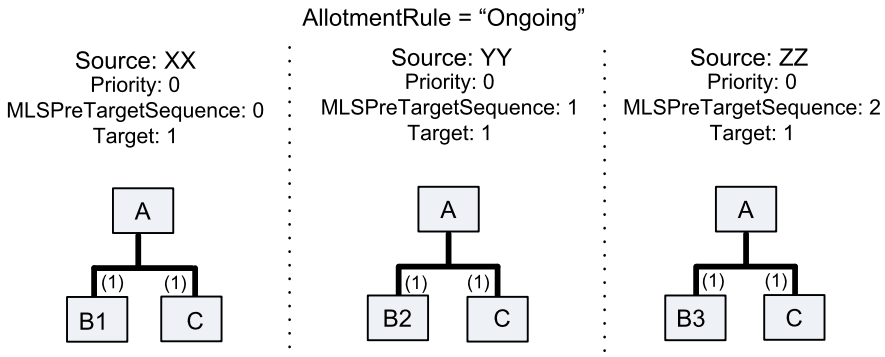
Demand Date	Source Allotments	Source Selection Details
Tuesday	XX: 0 YY: 75 ZZ: 50	Only Source YY and Source ZZ have component availability allowing them to satisfy this demand on time. • Source YY is used first because it has a higher effective priority. **75 units** are sourced using up the available on hand of B1 as input. • Source ZZ is used second because it has a lower effective priority. **50 units** are sourced using up half of the available on hand of component C1 as input.
Wednesday	XX: 100 YY: 30 ZZ: 50	Only Source XX and Source ZZ have component availability allowing them to satisfy any of this demand on time. • Source XX is used first because it is again the only higher priority that can provide supply for this demand on time (this time using the entire scheduled receipt for component A1 on Tuesdays as input). **100 units** are sourced from XX. • Lower priority Source ZZ is next because it is the only other source that can source supply for this demand on time (using the remaining on hand supply of component C1 as input). **50 units** are sourced from ZZ. None of sources can satisfy the remaining demand for 30 on time, but all Sources can potentially source this remaining requirement one day late. • Source YY is preferred because it is a higher priority source and has existing scheduled receipts for lower level components available (Source XX is also a higher priority Source but does not have any component supply remaining, and Source ZZ does have input component supply but is a lower priority source). The remaining **30 units** are sourced from YY using scheduled receipts for lower level components B2 B3 as input (a planned order is also created for B1).
Thursday	XX: 180 YY: 20 ZZ: 0	All sources can fully satisfy this requirement on time. Sources XX and YY are preferred because they have a higher effective priority than ZZ. • Source YY is used first because it still has some input component supply available from B2 and B3. **20 units** are sourced from YY using up the remaining component supply of B2 and B3 (and creating another planned order on B1). • The remaining requirement for **180 units** is sourced from XX because it has a higher calculated Target quantity. Planned orders are also created for all components in the structure.

Example 5: Usage of MLSPreTargetSequence

Assume three make part sources for assembly part A, configured using alternate BOMs, with applicable settings for each part source as shown in the following image. Lead time for each part is 1 day.

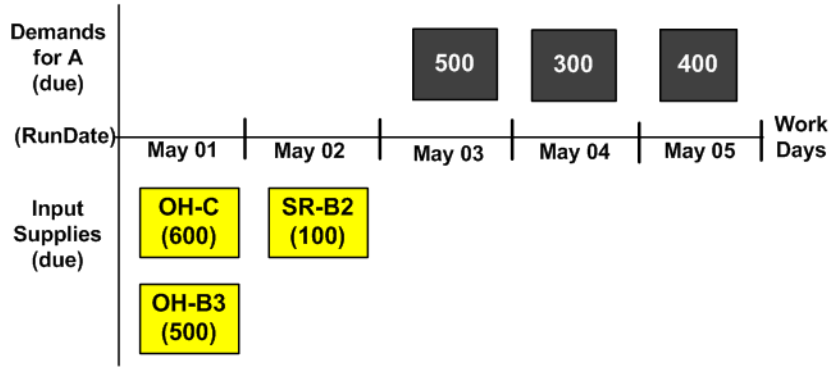
With these settings, all part sources have the same **Priority** value, thus when each source can satisfy a requirement on time, input supply type/usage and **MLSPreTargetSequence** will be the main determinants in making the sourcing decision.

Figure 41.6 Part sources with different MLSPreTargetSequence values



For example, assume the set of independent demands for assembly A, along with input supply availability on its components, as shown in the following image.

Figure 41.7 Assembly demands and component supplies



Based on the demands and supplies shown in the prior illustration, the following table describes the source allotments made per part source for each of the three requirements for “A”. This assumes the **ScheduledSupplyRule** is set to “PreferOnHand”.

Table 41.8: Source selection using MLSPreTargetSequence and “PreferOnHand”

Demand Date	Source Allotments	Source Selection Details
Wednesday	XX: 0 YY: 0 ZZ: 500	All part sources can satisfy this demand on time and have the same Priority. • Source ZZ is selected because it can provide it using on hand component inventory at the lowest level in the path.
Thursday	XX 200 YY: 100 ZZ: 0	All part sources can satisfy this demand on time and have the same Priority. • Source YY can provide a portion of this requirement using scheduled supply (100 units), so it is selected and used first. • After the first 100 units are sourced, no more on hand or scheduled component supply is available. Thus, the remainder is sourced from XX which has the lowest value in MLSPreTargetSequence.

Table 41.8: Source selection using MLSPreTargetSequence and "PreferOnHand"

Demand Date	Source Allotments	Source Selection Details
Friday	XX: 400 YY: 0 ZZ: 0	All part sources can satisfy this demand on time and have the same Priority, and none can provide any using on ahnd or scheduled supply on lower-level components. • Thus the entire requirement is sourced from XX because it has the lowest MLSPreTargetSequence value.

Instead if the **ScheduledSupplyRule** were set to "EqualToOnHand", then source allotments made per part source for each of three requirements for "A" would be as shown in the following table.

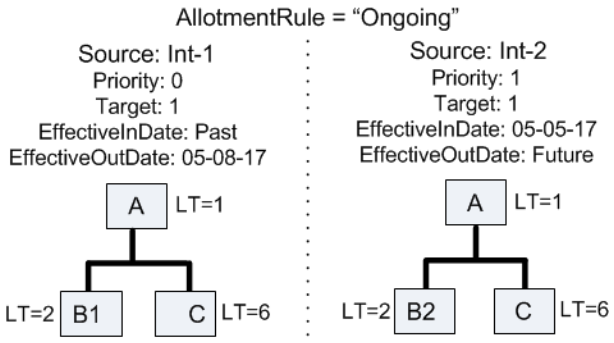
Table 41.9: Source selection using MLSPreTargetSequence and "EqualToOnHand"

Demand Date	Source Allotments	Source Selection Details
Wednesday	XX 0 YY: 100 ZZ 400	All part sources can satisfy this demand on time and have the same Priority. Both source YY and ZZ can satisfy (at least) a portion of this requirement using either on hand or scheduled supply. • YY has the lower MLSPreTargetSequence value and is used first. It is able to provide 100 units of the requirement using on hand or scheduled supply. • ZZ, having the higher MLSPreTargetSequence value is used next. It can provide the remainder of the requirement using on hand or scheduled supply.
Thursday	XX 200 YY: 0 ZZ: 100	All part sources can satisfy this demand on time and have the same Priority. • Source ZZ can provide a portion of this requirement using scheduled supply (100 units), so it is selected/used first. • After the first 100 units are sourced, no more on hand or scheduled component supply is available. Thus, the remainder is sourced from XX which has the lowest value in MLSPreTargetSequence.
Friday	XX: 400 YY: 0 ZZ: 0	All part sources can satisfy this demand on time and have the same Priority, and none can provide any using on ahnd or scheduled supply on lower-level components. • Thus the entire requirement is sourced from XX because it has the lowest MLSPreTargetSequence value.

Example 6: Impact of multi-level source effectivity

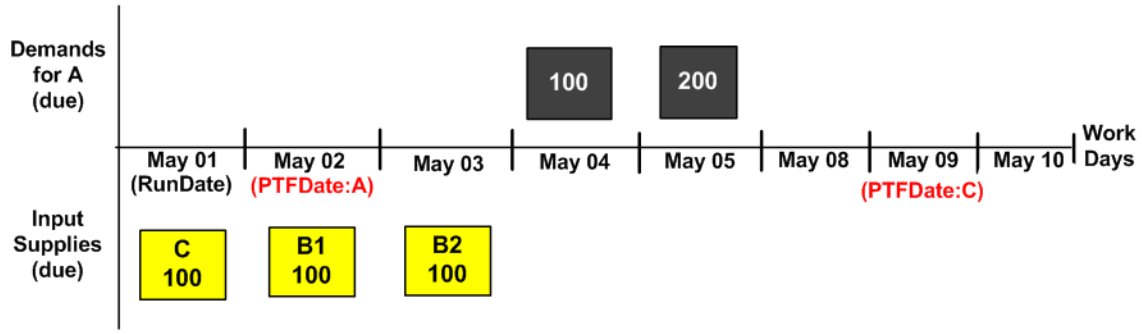
Assume two make part sources for a part, configured using alternate BOMs, with effective lead times and other settings for each source as shown in the following figure. All **QtyPer** values are 1.

Figure 41.8 Part sources with different effective dates



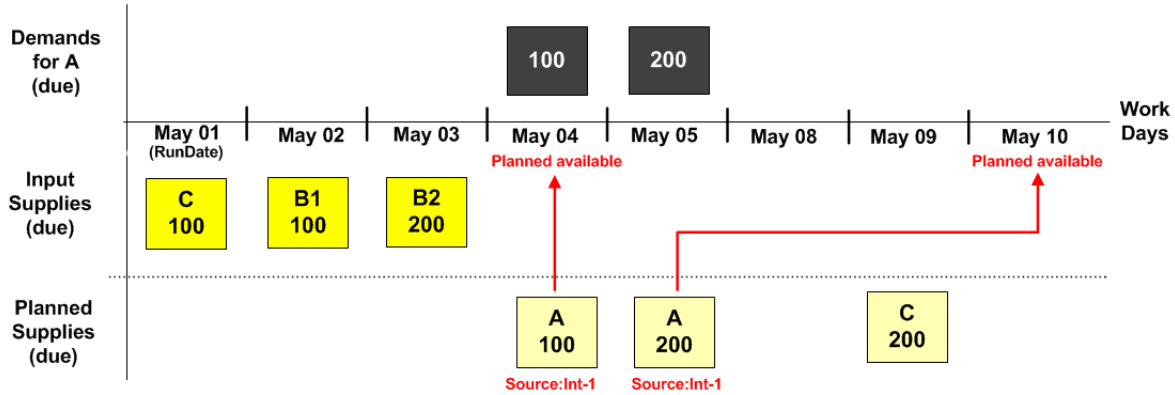
Further suppose the following set of input demands and supplies for assembly A and each of its components. In this case, to fulfill the demand for A, new planned supply for and it's component C is required.

Figure 41.9 Input demands and supplies



When the **MultiLevelSearchRule.SourceEffectivityRule** is set to "DueDate", then part source effectivity is determined based on when supply from eligible sources can be planned (without consideration of rolled up planned availability on components). Thus, planned orders would be generated as shown in the following figure, with both supplies for A provided by source Int-1.

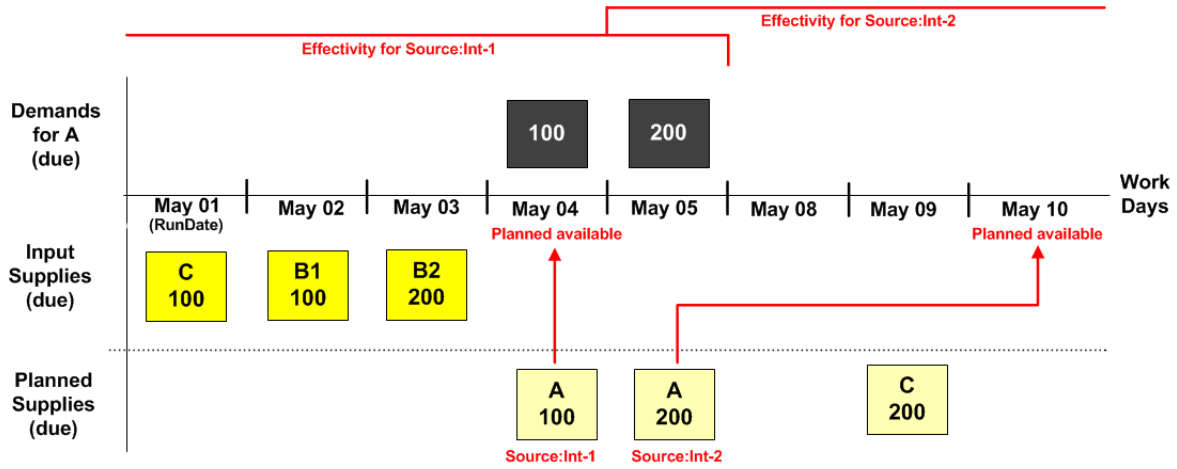
Figure 41.10 SourceEffectivityRule set to "DueDate"



Note that the planned order for 200 was sourced from Int-1 even though it's planned availability, based on planned multi-level component availability, falls outside the effective date range for the source (note also that the part source is no longer effective at the point when all required components are planned to be available).

Instead, if **MultiLevelSearchRule.SourceEffectivityRule** were set to "PlannedAvailableDate", then part source effectivity is determined based on when supply from eligible sources is expected to be available (based on rolled up planned availability from components). Thus, planned orders would be generated as shown in the following figure, with the supply for 100 still sourced from Int-1 but the supply for 200 now sourced from lower priority source Int-2. Note that the due date on this order is the same as it was previously, but is now provided by a part source that will still be effective on the planned available date.

Figure 41.11 SourceEffectivityRule set to "PlannedAvailableDate"

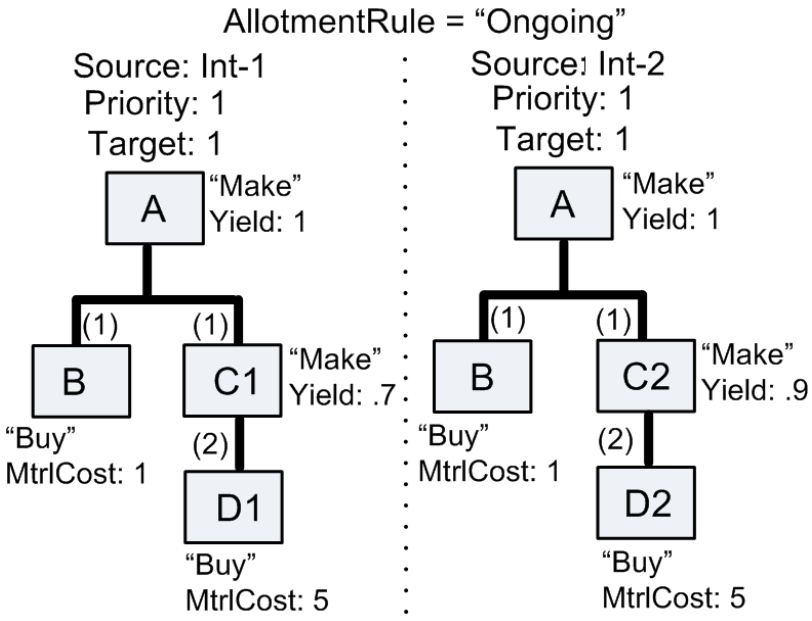


Finally, assume there was no other source than Int-1 effective on May 5th. In this case, with the "PlannedAvailableDate" setting, the demand for 200 would be left unsatisfied and an exception record generated in the **DemandException** table.

Example 7: Impact of new purchases cost

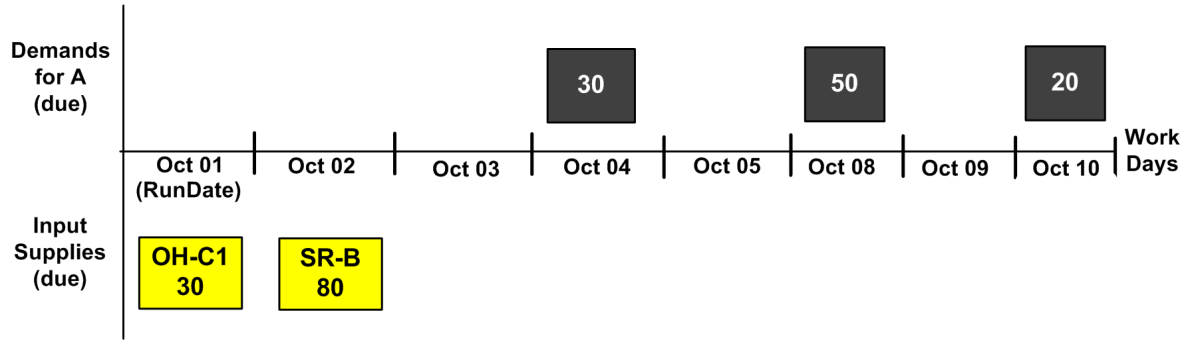
Assume a part that uses the **SourceRule.AllotmentRule** setting of "Ongoing' has two make part sources, configured using alternate BOMs, with relevant yield factors and material costs associated with the part sources in the structures as shown in the following illustration. Also assume each part source has a total lead time of 1 day.

Figure 41.12 Part sources using alternate BOMs



Further assume the set of independent demands shown for end-item A, along with available input supply for component C1 and common component B, as shown in the following illustration.

Figure 41.13 Input demands and supplies



If the parts all have multi-level search logic enabled but **MultiLevelSearchRule.NewPurchasesCost** is set to "Ignore", then sourcing decisions for the three demands for A would be as described in the following table.

Table 41.10: NewPurchasesCost = "Ignore"

Demand Date	Source Allotments	Source Selection Details

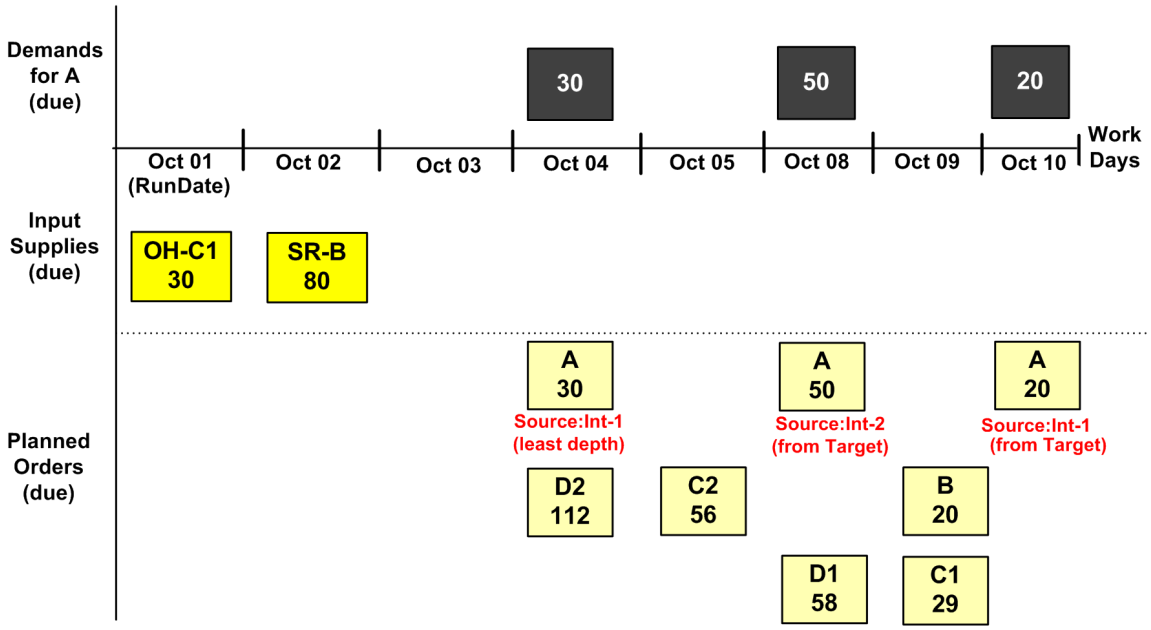
October 4	Int-1: 30
	Int-2: 0
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Source Int-1 is selected because it can provide it using the least depth of component supply.
		  • Both sources can make use of the scheduled receipt for common component B.
		  • Source Int-1 can use the on-hand inventory for sub-assembly C1 and does not require any new supply of component D1.
		  • Source Int-2 would have required a new planned order for sub-assembly C2 along with a corresponding planned order for component D2

October 8	Int-1: 0
	Int-2: 50
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Both part sources for A require new component supplies and at the same levels in the structure.
		• Source Int-2 is selected because it would be furthest from its ongoing target if not selected.
		  • If Source Int-1 is not selected then it is 10 away from its ongoing target of 40 (80 * 1/2)
		  • If Source INn-2 is not selected then it is 40 away from its ongoing target of 40 (80 * 1/2)

October 10	Int-1: 20
	Int-2: 0
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Both part sources for A require new supplies of all components.
		• Source Int-1 is selected because it would be furthest from its ongoing target if not selected.
		  • If Source Int-1 is not selected then it is 30 away from its ongoing target of 50 (80 * 1/2)
		  • If Source Int-2 is not selected then it is still at its ongoing target of 50 (80 * 1/2)

Thus, planned orders for "A" and its various components would be as shown in the following illustration.

Figure 41.14 Planned orders when MultiLevelSearchRule.NewPurchasesCost set to "Ignore"



Instead if **MultiLevelSearchRule.NewPurchasesCost** was set to "Use", then sourcing decisions for the three demands for A would be as described in the following table:

Table 41.11: NewPurchasesCost = "Ignore"

Demand Date	Source Allotments	Source Selection Details

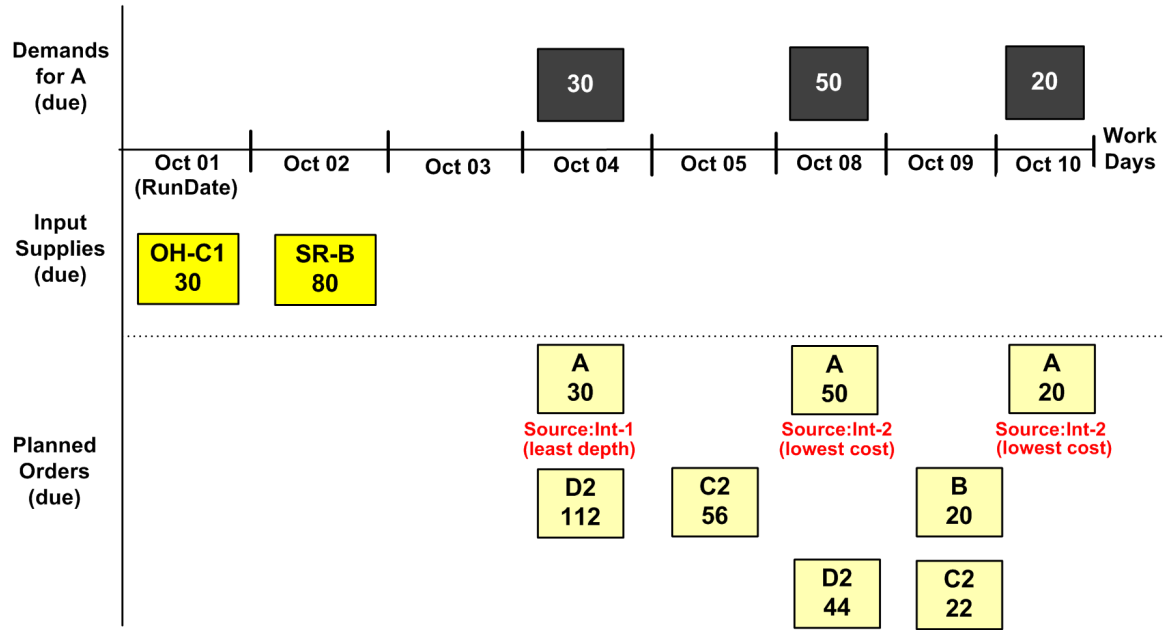
October 4	Int-1: 30
	Int-2: 0
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Source Int-1 is selected because it can provide it using the least depth of component supply.
		  • Both sources can make use of the scheduled receipt for common component B.
		  • Source Int-1 can use the on-hand inventory for sub-assembly C1 and does not require any new supply of component D1.
		  • Source Int-2 would have required a new planned order for sub-assembly C2 along with a corresponding planned order for component D2

October 8	Int-1: 0
	Int-2: 50
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Both part sources for A require new component supplies and at the same levels in the structure.
		• Source Int-2 is selected because its rolled up cost from new component purchases is lower.
		• New material purchase cost to use Int-2 is $560 as opposed to $720 if Int-1 were used.
		  • If Source Int-1 is selected, a planned order for 72 units of sub-assembly C1 is needed (50 units of dependent demand from A which is then inflated for yield factor of .70). This 72 unit supply then generates dependent demand of 144 units on component D1. Those 144 units multiplied by the material unit cost of $5 from D1 would then give the total cost of new material purchases.
		  • If Source Int-2 is selelcted, a planned order for 56 units of sub-assembly C2 is needed (50 units of dependent demand from A which is then inflated for yield factor of .90). This 56 unit supply then generates dependent demand of 112 units on component D2. Those 112 units multiplied by the material cost of $5 from D2 would then give the total cost of new material purchases.

October 10	Int-1: 0
	Int-2: 20
		• Both part sources for A can satisfy this demand on time and have the same Priority.
		• Both part sources for A require new supplies of all components.
		• Source Int-2 is selected because its rolled up cost from new component purchases is lower.
		• New material purchase cost to use Int-2 is $240 as opposed to $310 if Int-1 were used.
		  • Each source would require a planned order for 20 units of common component B, which is multiplied by material unit cost of $1 for B.
		  • If Source Int-1 is selected, a planned order for 29 units of sub-assembly C1 is needed (20 units of dependent demand from A which is then inflated for yield factor of .70). This 29 unit supply then generates dependent demand of 59 units on component D1. Those 59 units multiplied by the material unit cost of $5 from D1 gives 290, which added to the $20 cost from B would then give the total cost of new material purchases.
		  • If Source Int-2 is selelcted, a planned order for 22 units of sub-assembly C2 is needed (20 units of dependent demand from A which is then inflated for yield factor of .90). This 20 unit supply then generates dependent demand of 44 units on component D2. Those 44 units multiplied by the material cost of $5 from D2 gives 220, which added to the $20 cost from B would then give the total cost of new material purchases.

Thus, planned orders for "A" and its various components would be as shown in the following illustration.

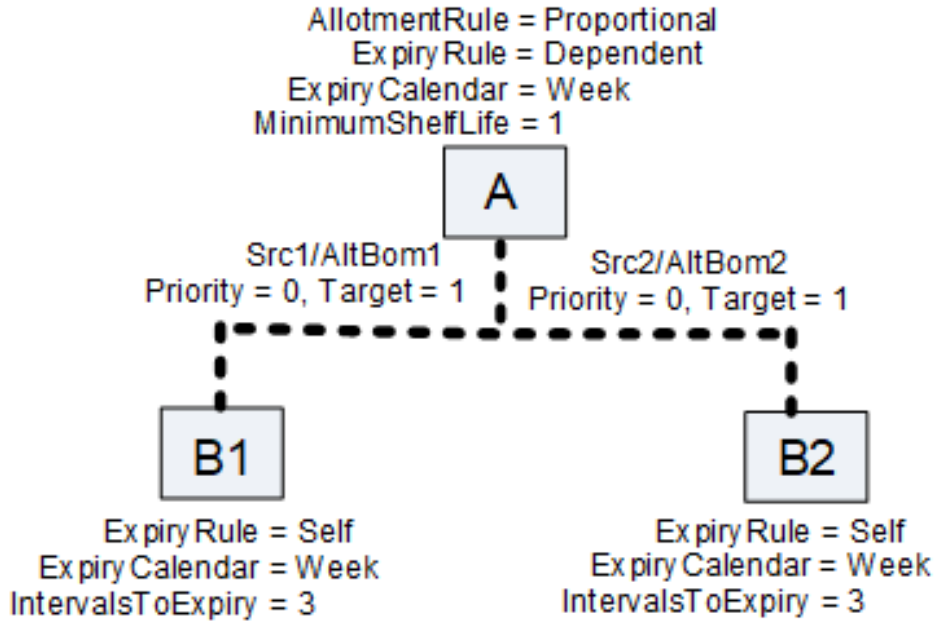
Figure 41.15 Planned orders when MultiLevelSearchRule.NewPurchasesCost set to "Use"



Example 8: MLS and material expiry

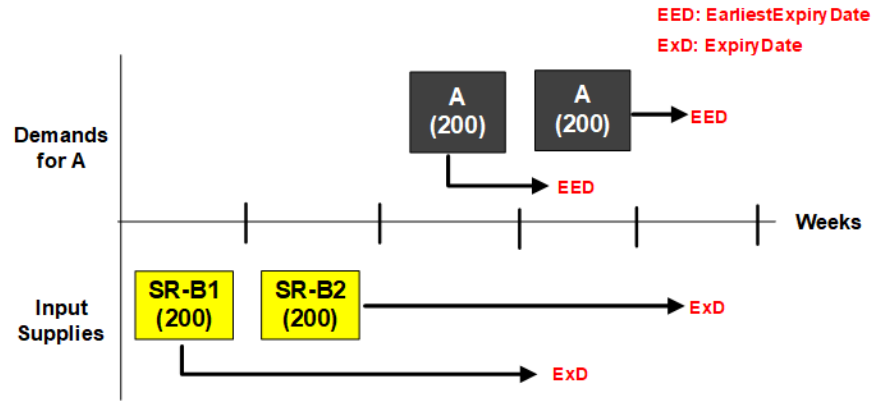
Suppose assembly A is subject to material expiration and can be built using either component B1 or component B2 based on two different part sources and alternate BOMs. Assume that all lead times and quantity per values are set to "1", and relevant expiry and part source selection properties for each part and part source are as shown in the following illustration.

Figure 41.16 Assembly with two alternate BOMs



Further suppose the following set of independent demands for assembly A and available input supplies (scheduled receipts) for components B1 and B2 (all demands and supplies are assumed to fall at the beginning of each week). Calculated earliest expiry and expiry dates are also shown.

Figure 41.17 Assembly demands and component supplies

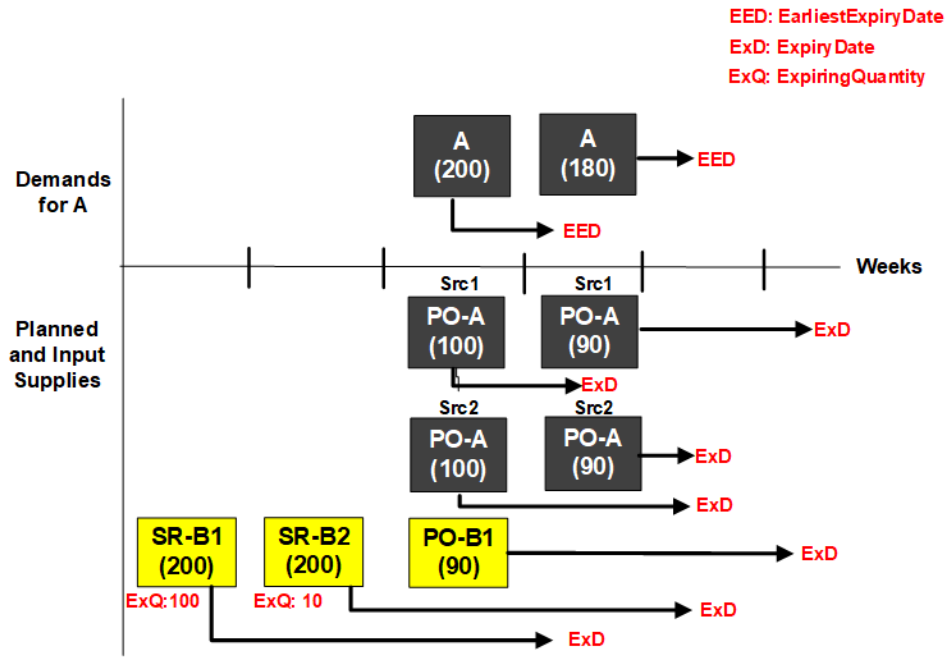


Without multi-level search logic enabled, supplies to fulfill the demands for assembly A would be sourced as follows.

- The demand for 200 units in Week 3 is equally split between the two sources based on Targets as each source has the same priority and can supply the demand equally well (on-time). Thus a planned order for 100 units is created on each source for A and using 100 units of each component's scheduled receipt.
- The demand for 180 units in Week 4 is equally split between the two sources based on Target as each can supply the demand on-time. Thus a planned order for 90 units is created on each source for A and using up 90 of the remaining units of component B2's scheduled receipt and also requiring a new planned order for 90 units on component B1 as the remainder of its scheduled receipt expires before too early to be used.

Refer to the following illustration which also shows the expiring quantity on each supply.

Figure 41.18 Sourcing decisions for assembly A with MLS disabled

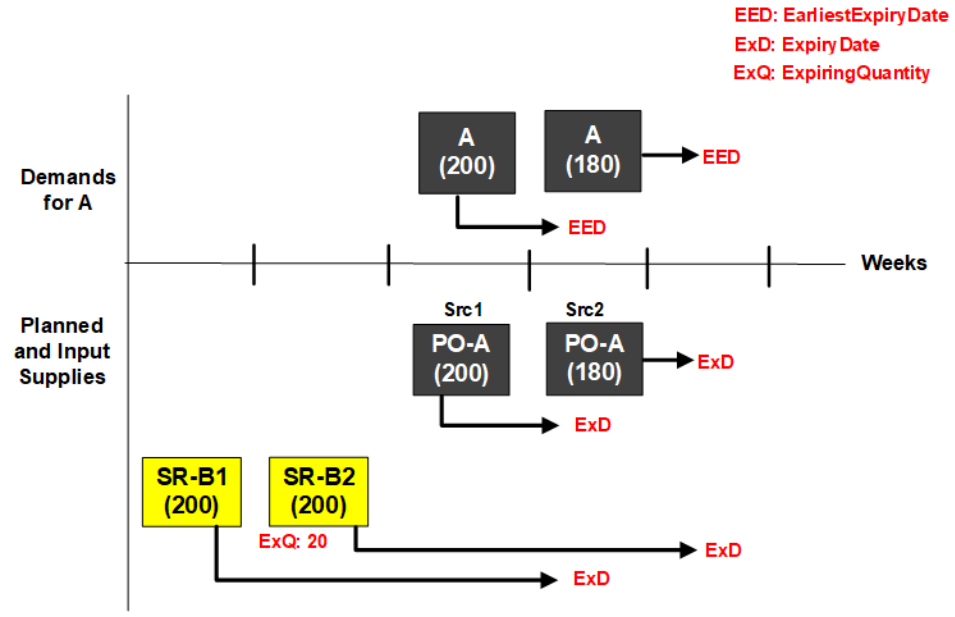


Instead assume that multi-level search logic is enabled for parts A, B1, and B2. In this case, supplies to fulfill the demand on A would be sourced as follows.

- The demand for 200 units in Week 3 is sourced entirely from Src1. Both part sources can fulfill this demand equally well, are of the same priority, and each would make use of the same component supply type/depth. However, supply from Src1 has an earlier planned expiry than Src2 (based on rolled up component expiry dates). Thus, Src1 is preferred based on the criterion to use earlier expiring supply first and mitigate overall expiry risk. Note that this uses all up existing input supply on component B1.
- The demand for 180 units in Week 4 is sourced entirely from Src2. Both part sources can fulfill this demand equally well and are of the same priority. However, because Src2 can be built using existing input supply of its component B2 it is preferred to Src1 which would require a new planned order on its component B1.

Refer to the following illustration which also shows the expiring quantity on each supply. Note that as compared to the previous illustration where multi-level search was not enabled, 90 less units of supply is now projected to expire and no planned orders are required on any of assembly A's components (only existing component supply is used).

Figure 41.19 Sourcing decisions for assembly A with MLS enabled

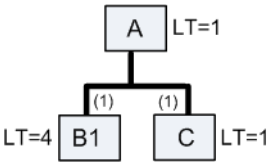


Note: Although multi-level search logic can help to make better sourcing and substitution decisions on parts subject to material expiration (such as, by using up existing and earlier expiring supply), some limitations exist when using multi-level search and material expiry logic together. See "Material expiry" on page 1929.

Example 9: MLS FairShare (with material shortage)

Suppose a simple BOM structure for assembly A has two components with quantity per and lead time values as shown in the following illustration.

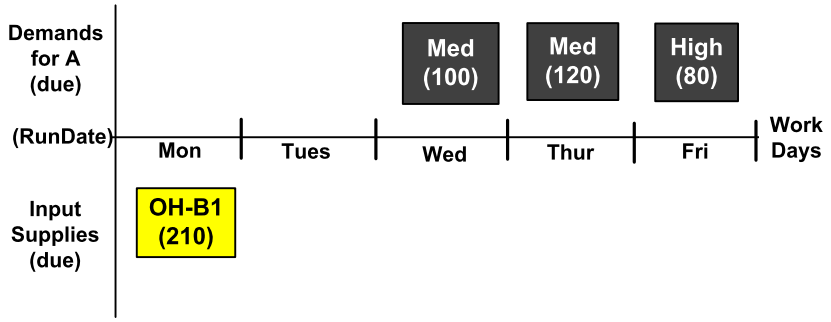
Figure 41.20 BOM for assembly with 2 components



Further suppose the set of prioritized demands for A, with the part's **PartType.CommitLevel** being set to "High". In this example, all demands are assumed to use the **DemandType.PlanningRule** setting of "Partial". Additionally, assembly A's **PlanningCalendars.ForecastCalendar** is assumed to be "Week" and its **UnsatisfiedDemandTolerance** left at "O". Therefore all demands shown must be satisfied with supply in the week they are due or else the unsatisfied portion of the demand will be ignored (never satisfied).

Thus, given the four day lead time on component B1, this means the only new supply that can be planned for A within that week is that which uses the on hand inventory of B1. In other words, although there is 300 units of demand for "A" no more than 210 units of supply of "A" can be planned within the week.

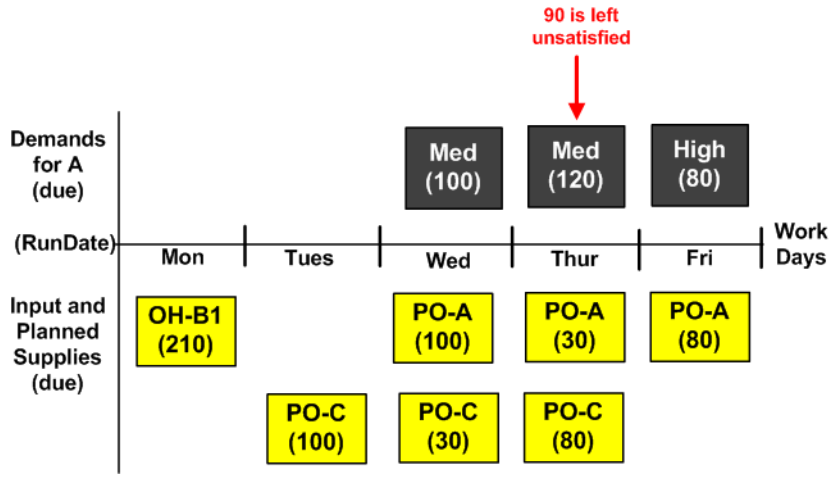
Figure 41.21 Prioritized assembly demands and firm component supplies



Assuming each priority's **OrderPriority.MLSAllocationRule** field is set to the default value of "FirstComeFirstServed", demands will be processed in priority order and then by due date within each priority as follows:

1. The High priority demand on Friday is processed first, and a planned order for the full 80 units of A is generated (along with a corresponding planned order for component "C" and reduction in on hand units of component B1 from 210 to 130).
2. The Medium priority demand on Wednesday is processed next, and a planned order for the full 100 units of A is generated (along with a corresponding planned order for component "C" and reduction in on hand units of component B1 from 130 to 30).
3. The Medium priority demand on Thursday is processed last. Due to limited material supply of component B1, only 30 units of supply for A can be planned within the week and the remaining 90 units of this demand is left short (this is the only demand with an unsatisfied portion).

Figure 41.22 Planned supplies with standard "first come, first served" multi-level logic



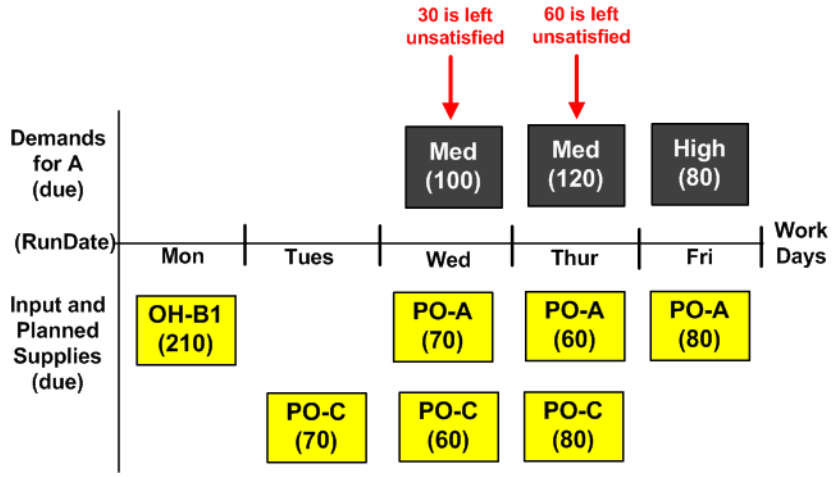
Instead assume the OrderPriority.MLSAllocationRule field for the "Med" priority level was set to "FairShare", with the MLSFairShareCalendar set to "Week" and MLSliceFactor set to "2". In this case, the demands would be processed as follows:

1. The High priority demand on Friday is still processed first, and a planned order for the full 80 units of A is generated (along with a corresponding planned order for component "C" and reduction in on hand units of component B1 from 210 to 130).
2. Then, the two Med priority demands within the week are collected and each is divided into two equal portion of 50 and 60 respectively. These demand portions are then processed in alternating rounds, as shown in the table below until as much of the demand as possible has been planned within the week:

Round	B1 supply remaining	Wed demand processed	B1 supply remaining	Thur demand processed
1	130	50	80	60
2	20	20	0	0

In this case, the unsatisfied demand portions are shared between the two "Med" priority demands as shown in the following illustration. However a greater portion (50%) of the Thursday demand has been left short as compared to the Wednesday demand (30%).

Figure 41.23 Planned supplies with "fair share" multi-level logic (SliceFactor of 2)



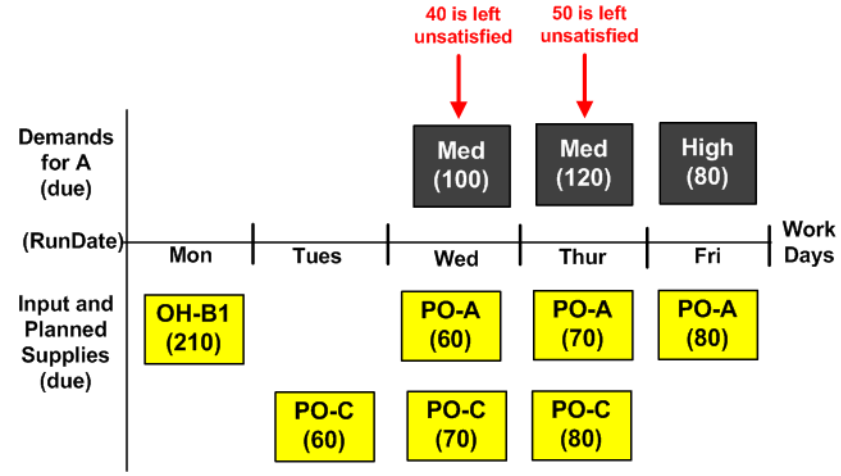
Instead assume the **MLSFairShareSliceFactor** were set to "5" with each of the two Med priority demands being split into 5 equal portions. In this case, the demands would be processed as follows:

1. The High priority demand on Friday is still processed first, and a planned order for the full 80 units of A is generated (along with a corresponding planned order for component "C" and reduction in on hand units of component B1 from 210 to 130).
2. Then, the two Med priority demands within the week are collected and each is divided into five equal portion of 20 and 24 respectively. These demand portions are then processed in alternating rounds, as shown in the table below until as much of the demand as possible has been planned within the week

Round	B2 supply remaining	Wed demand processed	B2 supply remaining	Thur demand processed
1	130	20	110	24
2	86	20	66	24
3	42	20	22	22
4	0	0	0	0
5	0	0	0	0

In this case, the unsatisfied demand portions are again shared between the two "Med" priority demands. However, a roughly fair portion of each demand has been left short (40% of the Wednesday demand and 42% of the Thursday demand).

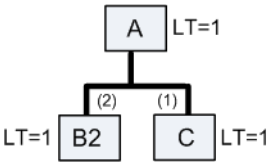
Figure 41.24 Planned supplies with "fair share" multi-level logic (SliceFactor of 5)



Example 10: MLS FairShare (with limited constraint)

Suppose a simple BOM structure for assembly A has two components with quantity per and lead time values as shown in the following illustration.

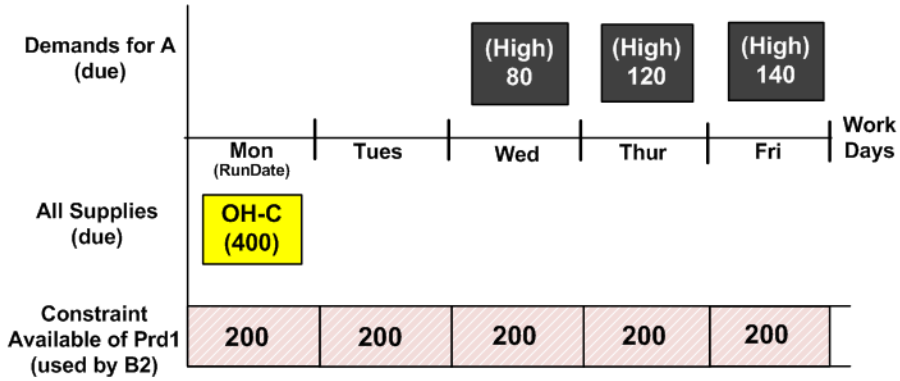
Figure 41.25 BOM for assembly with 2 components



Further suppose the set of demands as shown in the example below, with the part's **PartType.CommitLevel** being set to "High". In this example, all demands are assumed to use the **DemandType.PlanningRule** setting of "Partial". Additionally, assembly A's **PlanningCalendars.ForecastCalendar** is assumed to be "Week" and its **UnsatisfiedDemandTolerance** left at "O". Therefore all demands shown must be satisfied with supply in the week they are due or else the unsatisfied portion of the demand will be ignored (never satisfied).

Finally, suppose that the part source providing component B requires constraint "Prd1" with **PartSourceType.ConstraintDateRule** set to "StartExtend" and the applicable **SourceConstraint.ConstraintFactor** value set to "1" (thus only 300 of the 340 units demanded of A within the week can be satisfied within the week, the constraint available on Thursday and Friday results in component supply too late to satisfy demands of A within the week).

Figure 41.26 High priority demands for assembly A and firm component supplies

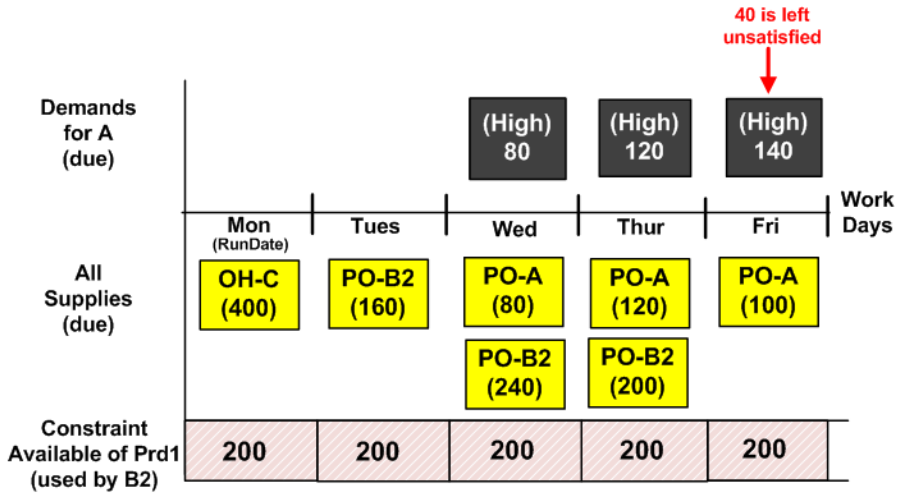


Assuming the "High" priority's OrderPriority.MLSAllocationRule field is set to the default value of "FirstComeFirstServed", these High priority demands will be processed in due date order within the week as follows:

1. The demand on Wednesday is processed first, and a planned order for the full 80 units of A is generated (along with a corresponding planned order for component B2 for 160 which would start on Monday).
2. The demand on Thursday is processed next, and a planned order for the full 120 units of A is generated (along with a corresponding planned order for component "B2" for 240 units which would start on Monday).
3. The demand on Friday would be processed last and, due to the limited constraint available to produce component B2 in time to allow this demand to be satisfied in the week, a planned for just 100 units would be generated (along with a corresponding planned for component "B2" for 200 units which would start on Wednesday).

   Thus, 40 units of the demand on Friday would be left short as shown in the following figure.

Figure 41.27 Planned supplies with standard "first come, first served" multi-level logic



Instead assume the **OrderPriority.MLSAllocationRule** value for the "High" priority level was set to "FairShare", with **MLSFairShareCalendar** set to "Week" and MLSliceFactor set to "8". In this case, the demands would be processed as follows:

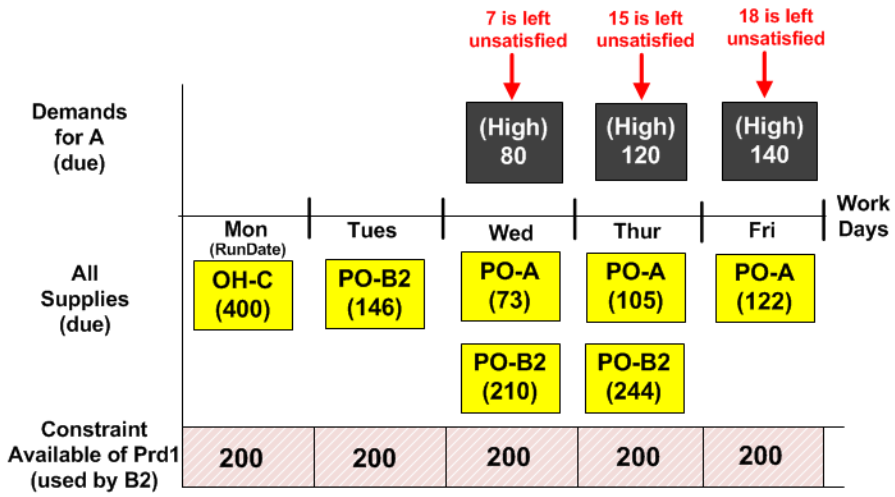
- All three demands within the week are collected and each is divided into five equal portions. These demand portions are then processed in alternating rounds, as shown in the following table until as much of the demand as possible (300 units) has been planned within the week based on available component constraint.

  Note also that Wednesday and Thursday demands can be split into 8 equal portions of 10 and 15 respectively. However, the Friday demand cannot and assuming a Part.AllocationMultiple of "1" for part A, the Friday demand would be split using the calculations shown in the last column.

Round	Wed demand processed	Thur demand processed	Fri demand processed	Friday demand slice calculated as
1	10	15	17	140/8= 17.5 Round down to 17
2	10	15	17	123/7= 17.5 Round down to 17
3	10	15	17	106/6= 17.6 Round down to 17
4	10	15	17	89/5= 17.8 Round down to 17
5	10	15	18	72/4 = 18
6	10	15	18	54/3 = 18
7	10	15	18	36/2 = 18
8	3	0	0	18/1 = 18

Thus a relatively fair portion of each demand is left short as shown in the following figure.

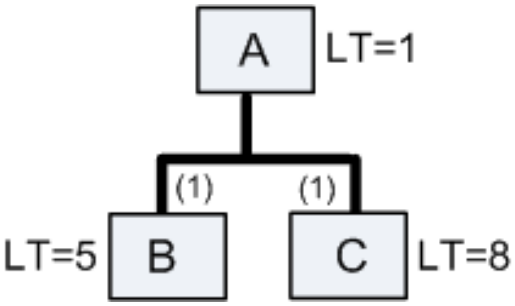
Figure 41.28 Planned supplies with "fair share" multi-level logic (SliceFactor set to "8")



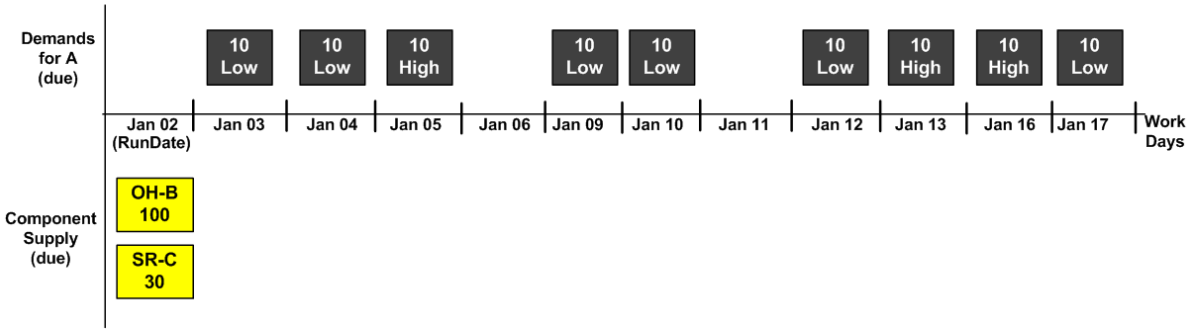
Example 11: Reduction of short-term excess inventory

The following shows an example of how the **MultiLevelSearchRule.ShortTermExcess** setting can impact the amount of excess inventory held in the short term.

Suppose a simple BOM structure where assembly "A" is made up of components "B" and "C" with lead time and quantity per values as shown in the following illustration. Assume also that a **PartType.CommitLevel** setting of "High" is used and the part source providing "A" has a **MinimumQty** of 30 units.

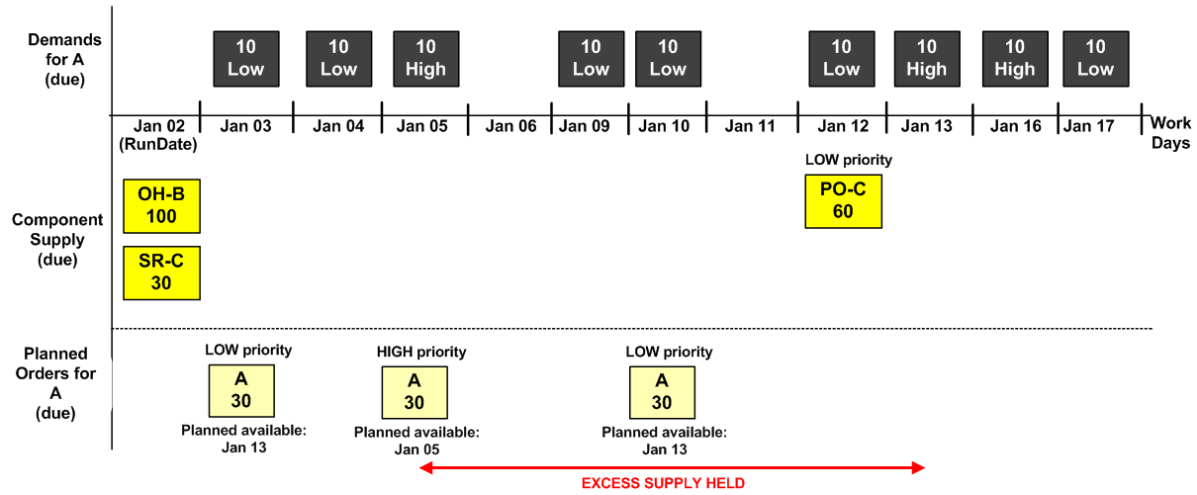


Further suppose the following set of prioritized demands for "A" along with available supplies for components "B" and "C" (assume no current or planned supply exists for "A").



If **MultiLevelSearchRule.ShortTermExcess** is set to "Allow", then planned orders for A are generated as shown below. Note that the high priority demand due on Jan 5 is processed first and a high priority planned order created to cover it. To meet the minimum lot size, this order is increased to 30 units and subsequently used for the later high priority demands. Low priority planned orders are then created to satisfy the other demands which are of low priority. Note that this results in supply being planned earlier than it needs to be and some supply from the high priority planned order held as short-term excess.

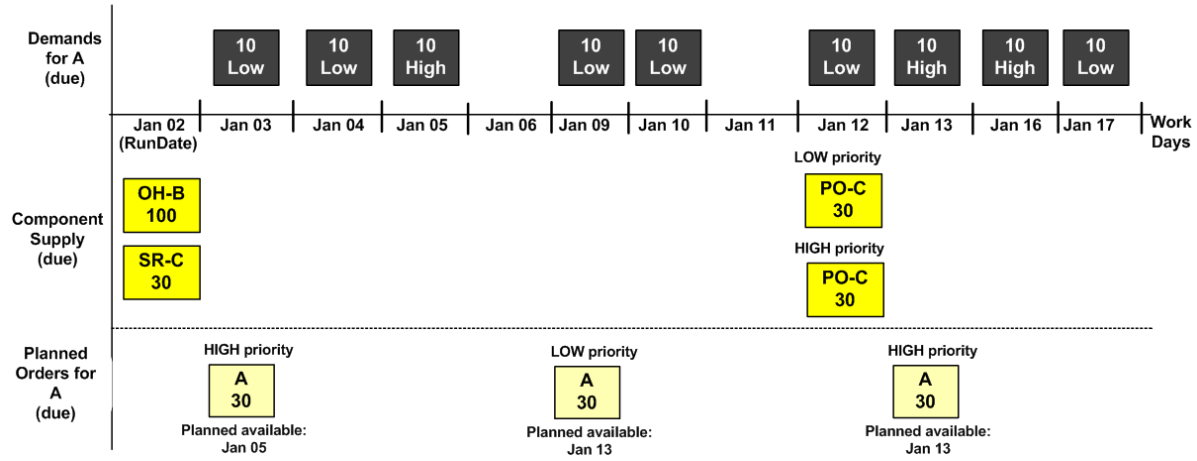
Figure 41.29 ShortTermExcess set to "Allow"



Instead if **MultiLevelSearchRule.ShortTermExcess** is set to "Reduce", then planned orders for A are generated as shown below. In this case, note that two high priority planned orders are generated instead of one. This is because supply from the first of these high priority orders can also be used for the earliest low priority demands without making the later high priority demands worse off than they were before. A second high priority orders than satisfies the later high priority demands and subsequent low priority demand, with a single low priority planned order created for the remaining demands.

Thus, the excess supply resulting from the High priority planned order that was seen previously held has been reduced. Note also with this setting that two separate planned orders for component C are now created; one with high priority and one with low priority (as opposed to the single low priority order shown in the previous illustration).

Figure 41.30 ShortTermExcess set to "Reduce"



Substitution

Multi-level search logic can be used to improve supply sourcing decisions made when choosing either between global part substitutes to supply a part or BOM-level substitutes to provide supply of a component part under a given assembly.

Multi-level search logic with part substitution

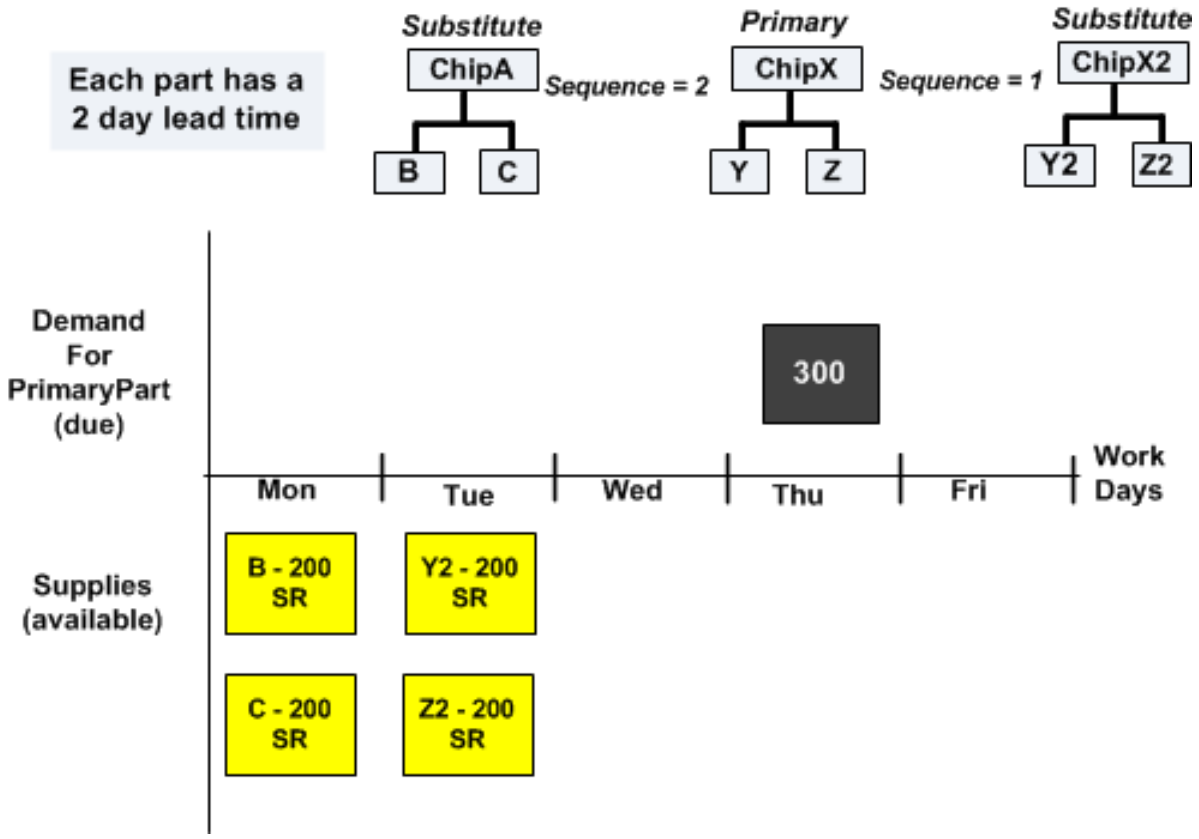
As discussed in "Global part substitution" on page 1551, when there is insufficient on hand inventory or scheduled supply available to satisfy demand on a primary part, planned orders need to be created to satisfy the demand. When choosing whether to create planned orders on a primary part or one its substitutes, RapidResponse first attempts to create on time planned orders for the primary part. If this is not possible, RapidResponse next prefers to try and create on time planned orders on substitute parts. If sufficient on time orders cannot be generated on the primary part or any of its substitutes, then RapidResponse will generate the earliest orders it can on either the primary or substitute parts.

In standard part substitution logic, when RapidResponse needs to create planned orders to try and provide on time supply for the primary part, it considers constraint availability and planning time fence only on the primary part and its substitute parts themselves. It does not consider component availability under those parts as part of the criteria for determining whether planned orders can be sourced on time (that is, unlimited component availability is assumed at all levels under the primary and substitute parts). In some scenarios, this can lead to less desirable planning results than might be possible if component availability was considered during planned order generation.

Example: Improve availability with substitution and MLS

Assume three simple BOM structures exist for a primary part (ChipX) and its two substitute parts (ChipX2 and ChipA), and a planning period beginning on Monday, as shown in the following illustration. A demand exists for the primary part, and no on hand or scheduled supply is available so planned orders are required. In this scenario, the primary part and each of its substitutes would be seen as being able to provide a planned order by Wednesday and therefore on time to satisfy the Thursday due date, with the primary part preferred and a planned order of 300 units created on ChipX. This situation could be improved with multi-level sourcing logic.

Figure 41.31 Situation that could be improved with multi-level sourcing logic



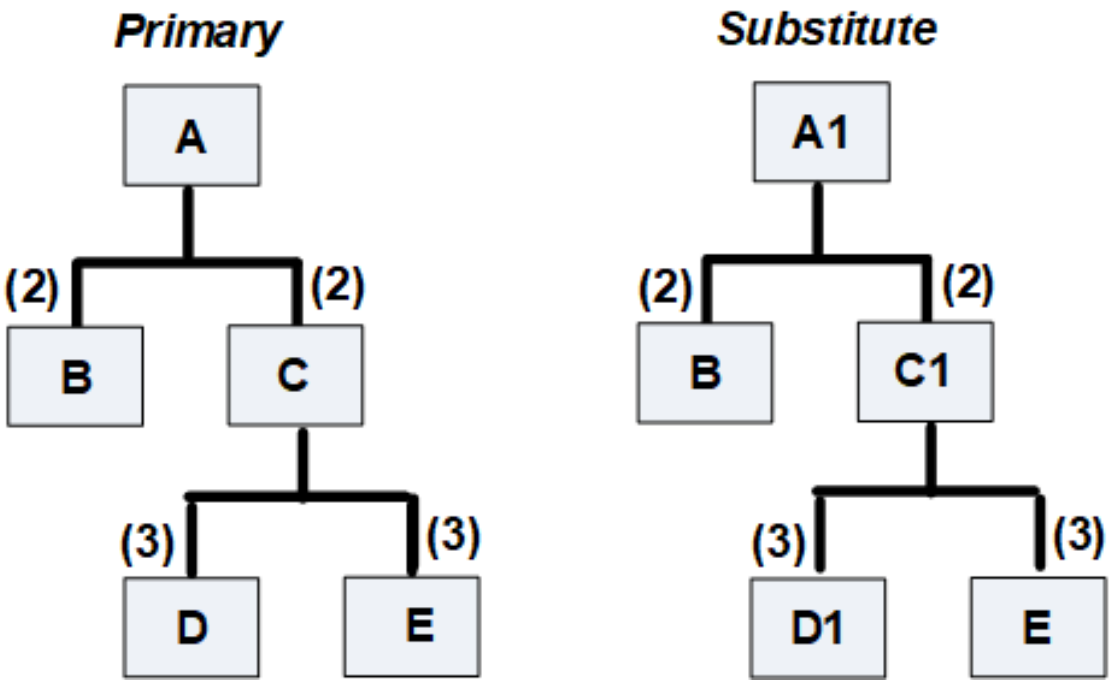
However, due to the lack of available component supplies under the primary part ChipX, planned orders would first need to be created for components Y and Z. These components would not be available until Wednesday, thus delaying the start of the planned order for ChipX and making the entire order available (and late) on Friday.

If multi-level search logic were enabled on the parts in these structures, then potentially better sourcing decisions could be made by considering the availability of scheduled component supplies under the two substitute parts. That is, due to the scheduled supplies of components B and C, 200 units of ChipA could potentially be produced in advance of the demand's Thursday due date. Similarly, due to scheduled supplies of components Y2 and Z2, 200 units of ChipX2 could potentially be produced by the demand's Thursday due date. This is preferable to sourcing supply from the primary part which would be late, and so RapidResponse would generate a planned order for 200 units on ChipX2 and a planned order for 100 units on Chip A (Chip X2 would provide the larger portion of the substitute supply due to its having the lower Sequence value).

Example: Reduce excess with substitution and MLS

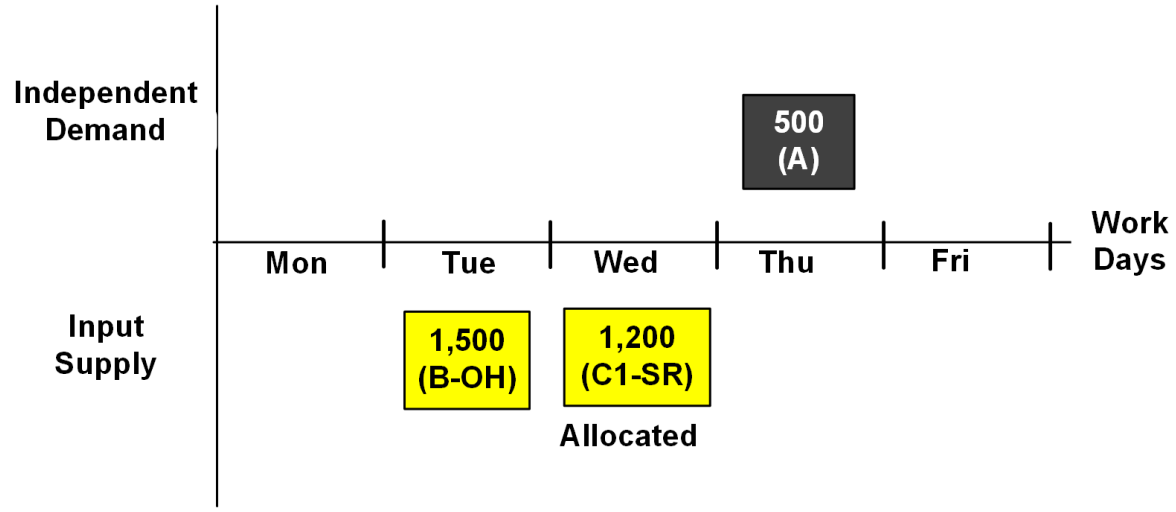
Suppose two BOM structures, one for primary Part A and one for it's global substitute Part A1, as shown below where the two assemblyies share some common components. Further suppose that

- **PartType.SubstitutionPreference** is set to "ScheduledSupply".
- **PrimarySubstitutionSequence** and Sequence values are all set to "0"
- All lead times are 1 day



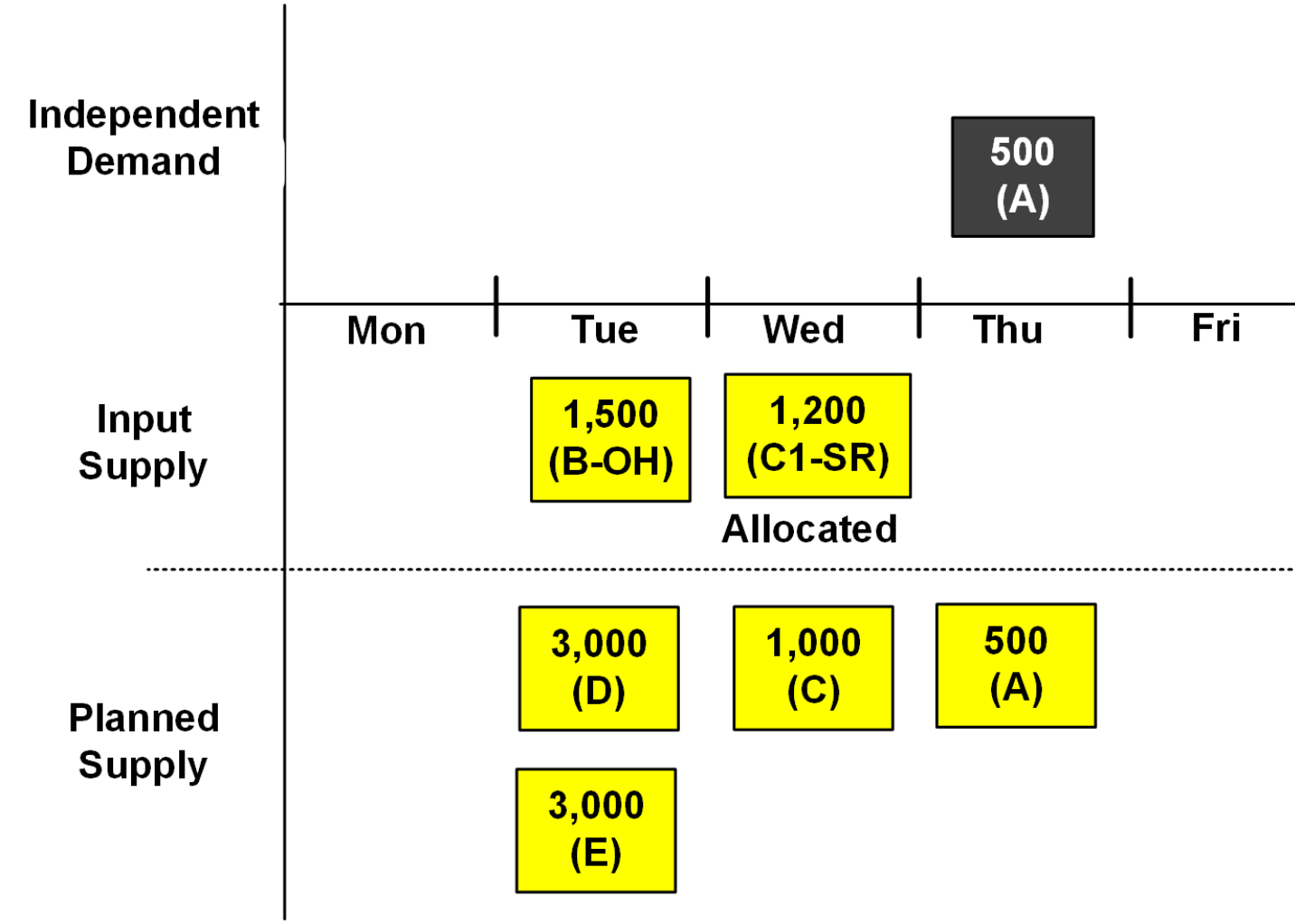
Assume the following set of the demands and input supplies for the parts shown above. In this case, a single independent demand for primary part A, as well as on hand inventory for common component B and an allocated scheduled receipt for component C1 which is used only in substitute part A1.

Figure 41.32 Input demand and supplies



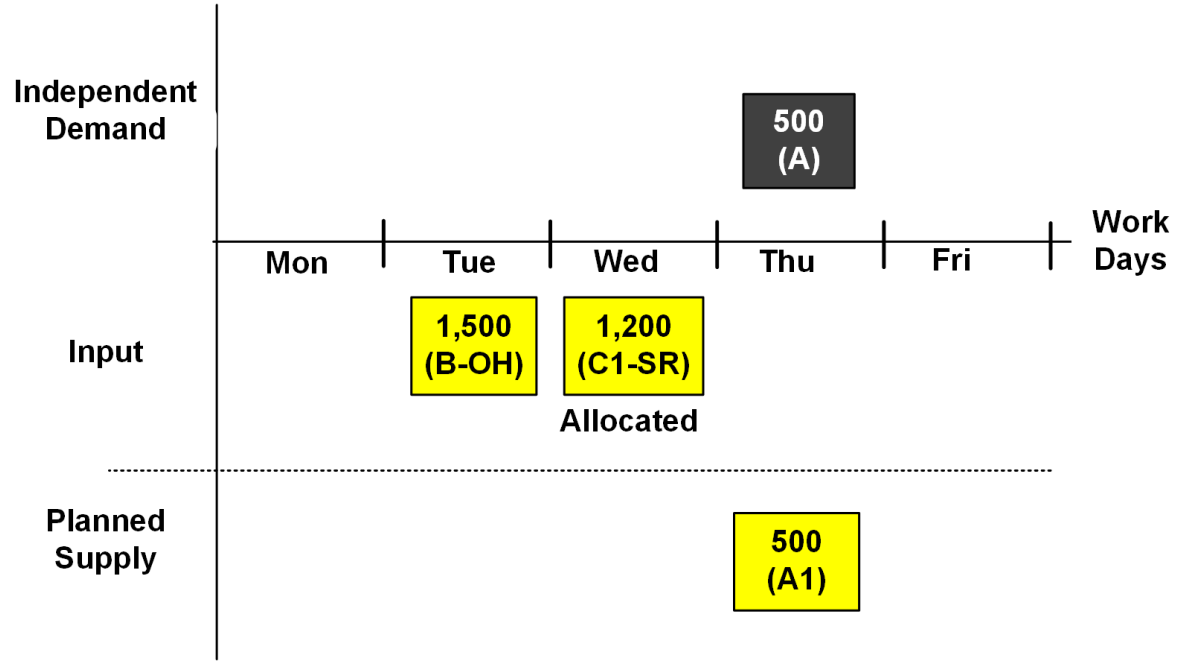
Without multi-level search enabled, global part substitution logic determines there is no existing supply available on either the primary part A or its substitute A1. Since both parts could satisfy the demand on-time with a new planned order, the primary part A is used to satisfy the entire demand as shown in the following illustration. Note that the already created and allocated scheduled receipt for component C1 remains as excess while new planned supply is required for bottom-level components D and E.

Figure 41.33 Planning with multi-level search disabled



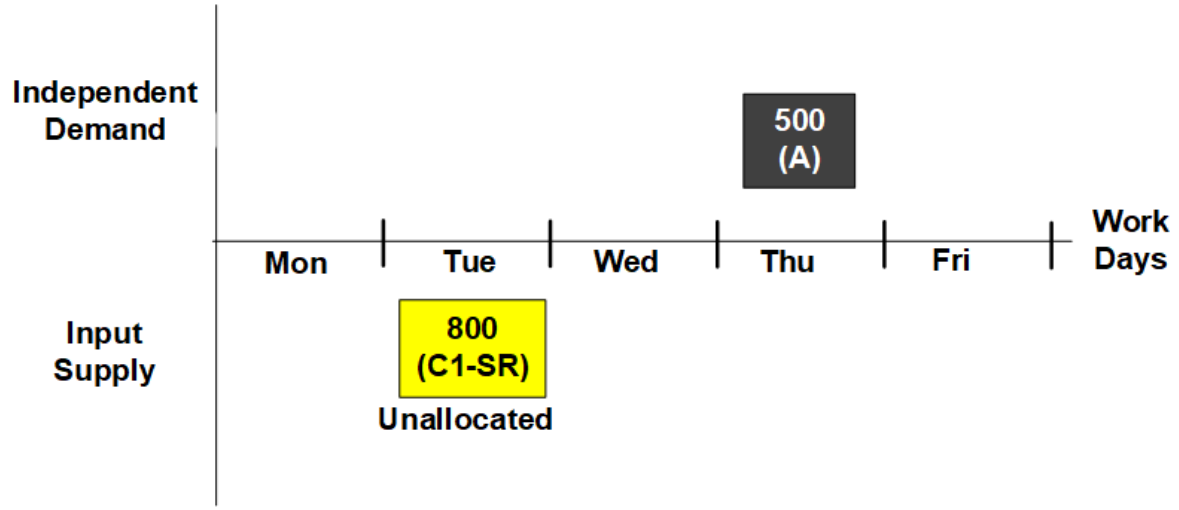
However if multi-level search is enabled, then component supply availability is considered. Although both parts can satisfy the demand on time, alternate part A1 is preferred as it is able to utilize just input supplies for the middle-level components whereas using primary part A would again have required planning new supplies being planned for the bottom-level components. Note that all but 200 units of the scheduled receipt for component C1 would be used up in this case.

Figure 41.34 Planning with multi-level search enabled



Instead assume the same lone demand for primary part but with the only input supply being an unallocated scheduled receipt of 800 units for component C1 as shown in the following illustration. If multi-level search were not enabled, then global substitution logic would fully satisfy the demand with a new planned order on component A and generating corresponding dependent demands and planned supplies on all of its components.

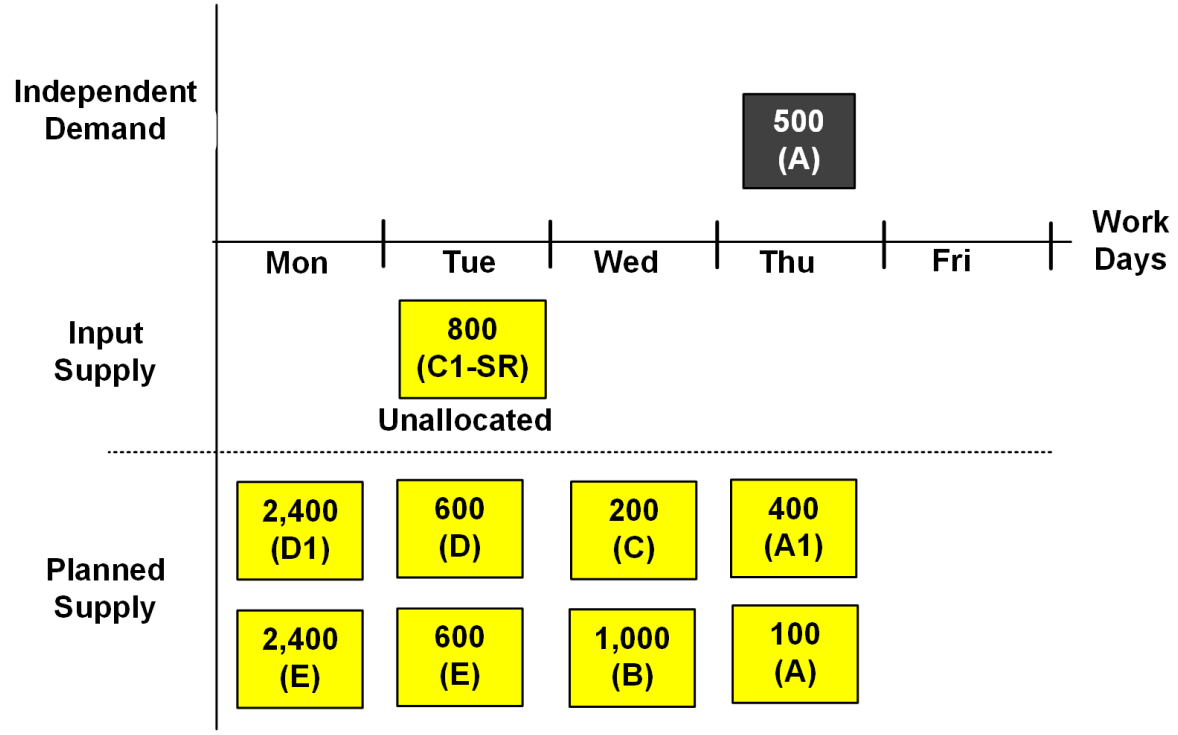
Figure 41.35 Input demand and supply



However, when multi-level search logic is enabled, preference is given to using up the existing 800 units scheduled receipt of A1's lower-level component C1 (even though this receipt is unallocated and requires new planned orders from it's components D1 and E). Thus a planned order for 400 units is planned on substitute part A1 which fully uses up the unallocated scheduled receipt on C1 (as well as

requiring new planned orders on all other components in its structure). The remaining 100 units of the independent demand is then satisfied by a planned order on primary part A which is preferred to its substitute when it can supply the demand equally well (and further leads to new planned orders on all parts in its structure).

Figure 41.36 Unallocated scheduled receipt under substitute used before planning new supply on primary

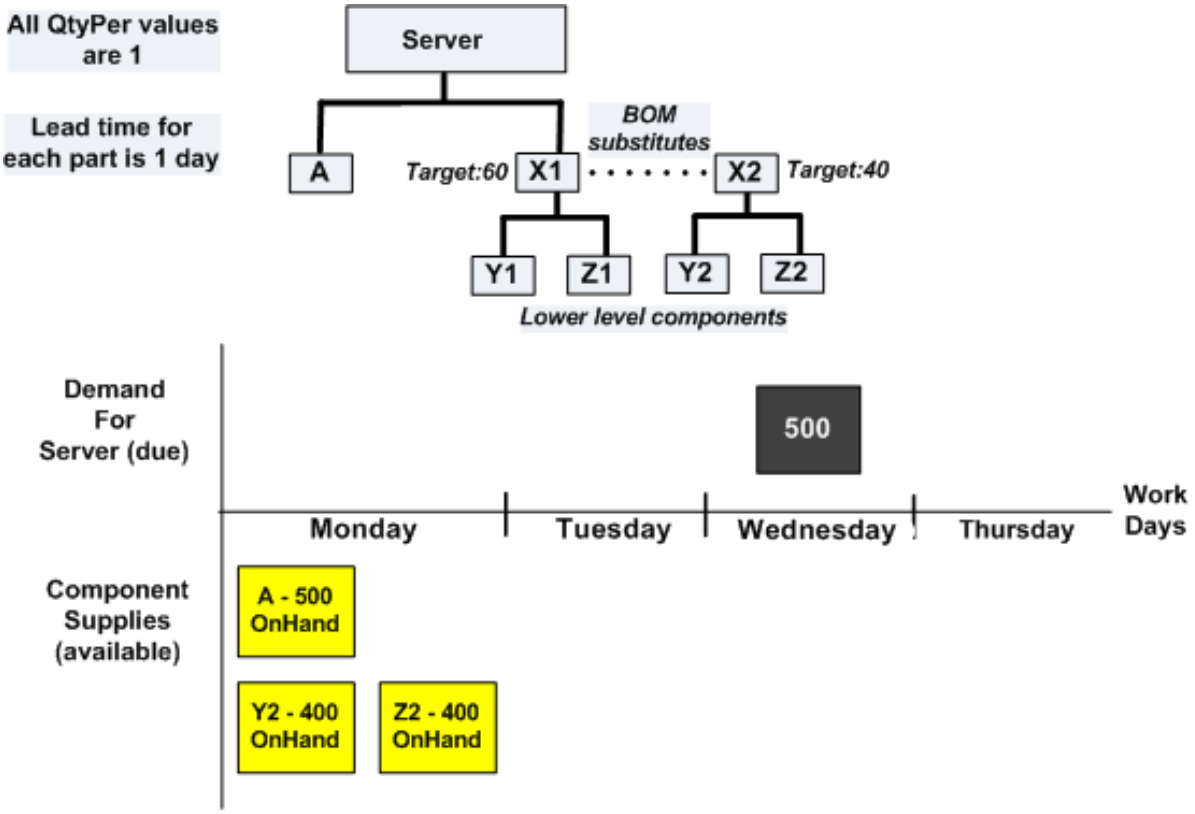


Multi-level search logic with BOM substitution

If RapidResponse cannot find enough current supply (on hand or scheduled receipts) from either the primary component used in an assembly or any of its substitute components, planned orders need to be created. As discussed in "BOM level part substitution" on page 1565, the **BillOfMaterial.Target** field can be used to set the ratio in which dependent demand from the assembly is exploded down to each substitutable component.

For example, assuming the BOM structure and set of demands and supplies shown in the following illustration, dependent demand of 500 units is passed down from Server assembly demand due on Wednesday to each of its components.

Figure 41.37 Substitute BOM relationship and lower level components



In this scenario Component A is able to satisfy their requirement entirely from on hand inventory. However, because neither primary component X1 nor its substitute component X2 have any on hand or scheduled supply available, planned orders are required on these substitutable components. Based on their Target values, 300 planned orders are created on X1 (60% of 500) and 200 planned orders are created on X2 (40% of 500).

In some cases, particularly early in the planning horizon, it might be preferable to override the Target percentages to improve order availability based on lower level component supply availability. For example, in the configuration shown in the previous example, the 200 planned orders created on X2 would be available on time to supply the Server demand on due on Wednesday (due to on hand inventory of X2's lower level components Y2 and Z2). However, all 300 planned orders created on X1 would be available too late to supply the Server demand on time (due to needing subsequent planned orders on X1's lower components Y1 and Y2).

However, if multi-level search logic were enabled on the parts in the structures, better on time availability of the Server demand could have been achieved. By considering availability of lower level components under X1 and X2, RapidResponse would have placed the first 400 planned order quantity on X2, all of which would have been on time to satisfy the demand originating from the Server part. Because all paths to satisfy the remaining dependent 100 units would be equally late these would be distributed between X1 and X2 based on their Target values (60 units on X1 and another 40 units on X2).

Notes:

- When BOM substitution logic is configured to allow substitutions between groups of

related components, and multi-level search logic is enabled on the component parts used in the BOM substitution relationship, the availability of all components within the group is considered when deciding which component group to use. Otherwise, when multi-level search logic is not used, the decisions on which group of components to use is determined by an "indicator" component within each group. For more information, see "Substituting groups of components" on page 1574.

- When multi-level search logic is enabled on the component parts used in a BOM substitution relationship, support for a BOM substitution excess window in which only excess on hand and scheduled supply of substitute parts is used in BOM substitution is disabled. For more information about the excess window, see "Specify usage of substitute supply" on page 1569.

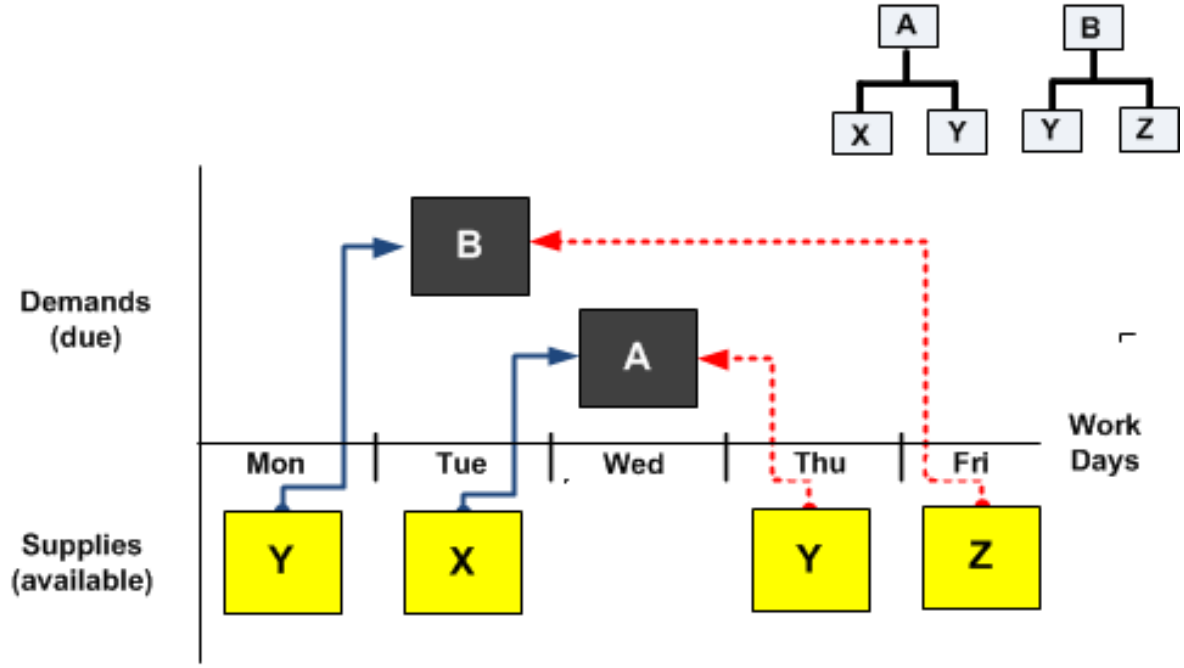
Non-blocking allocations

Typically, the allocation of component supplies to demands in RapidResponse considers demand due date, supply available date, priorities, and so on. When allocations are made for a given component supply, no knowledge of the allocations made on any of its sibling components is assumed. In cases where a common component is used across multiple assemblies, and there is limited on time supply of that component, this might result in less optimal allotments than would be made if knowledge of all sibling component allotments was known. In these cases, multi-level search logic can be enabled to support non-blocking allocations in RapidResponse.

Non-blocking allocations refers to allocations that consider the impact of supply allotments made on other sibling components. In cases where supply of a given component is unavoidably late, RapidResponse will attempt to synchronize allotments of any sibling components of that gating component. This allows for earlier supplies of those sibling components to potentially be released to satisfy other demands that might otherwise be late.

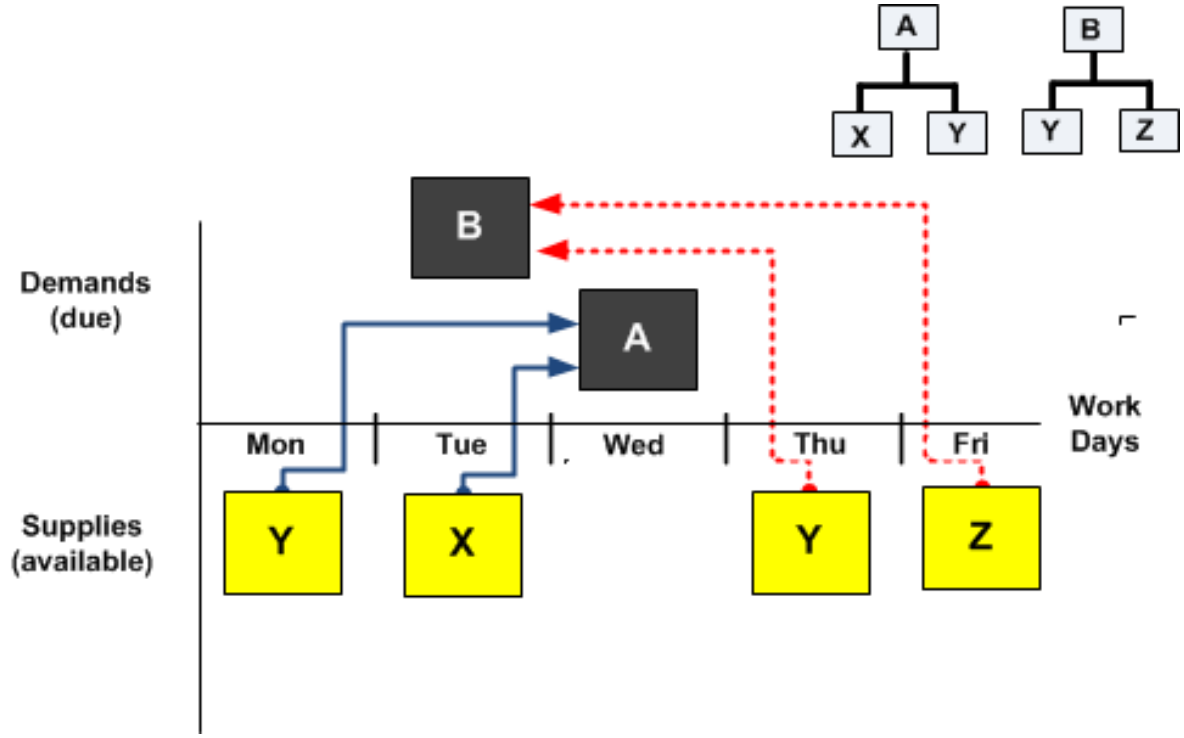
The following illustration shows a simple example of supply allotments that could be improved in RapidResponse with non-blocking allocations. In this scenario, there is one demand each for assembly A and B, and these assemblies share a common component Y that can satisfy only one of those assemblies on time. Using the standard allotment logic, the on time supply of Y is used to satisfy the earlier demand from assembly B, and the late supply of Y is given to the later demand from assembly A. However, despite receiving the on time supply of Y, note that the demand on B is late due to receiving late supply of its other component Z. At the same time, the demand for A is made late due to receiving the later supply of Y although its other component, X, provides supply on time.

Figure 41.38 Allotments that could be improved by non-blocking allocations



The situation shown above could be improved if enabling non-blocking allocations by setting the **MultiLevelSearchRule.ProcessingRule** reference to "Use" for parts B, Y, and Z. Then, when allocating supplies to the demand for B, RapidResponse would be able to identify its gating supply of component Z and, as a result, allot it the later Thursday supply of Y. This would free up the earlier Monday supply of Y to be allotted to the demand for A as shown in the following illustration. This would make the demand for A now on time, without affecting the availability of the demand for B which is still determined by gating supply of component Z.

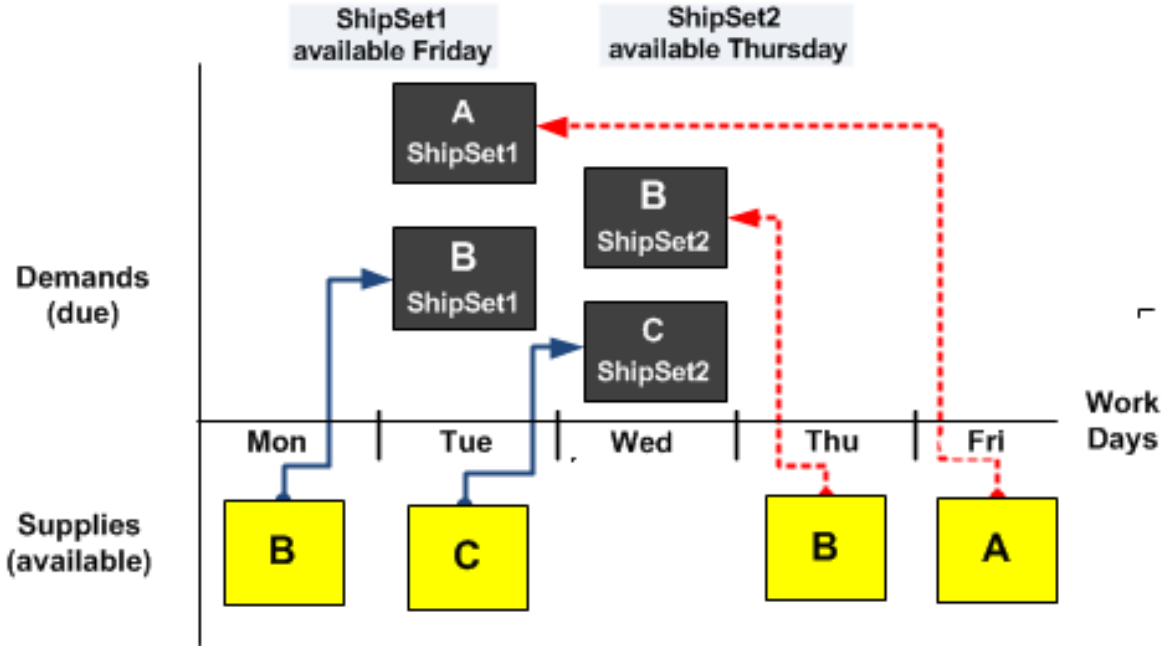
Figure 41.39 Allotments are improved with non-blocking allocations



Note that, in this scenario, if the supplies shown on Thursday and Friday were reversed (that is, if Z were available on Thursday and Y were available on Friday), then the supply allocations made would be identical to those made without non-blocking allocations. This is because RapidResponse would not change the supply allocations made in order to make the demand for A on time if it also meant making the already late demand for B worse off than it would otherwise be.

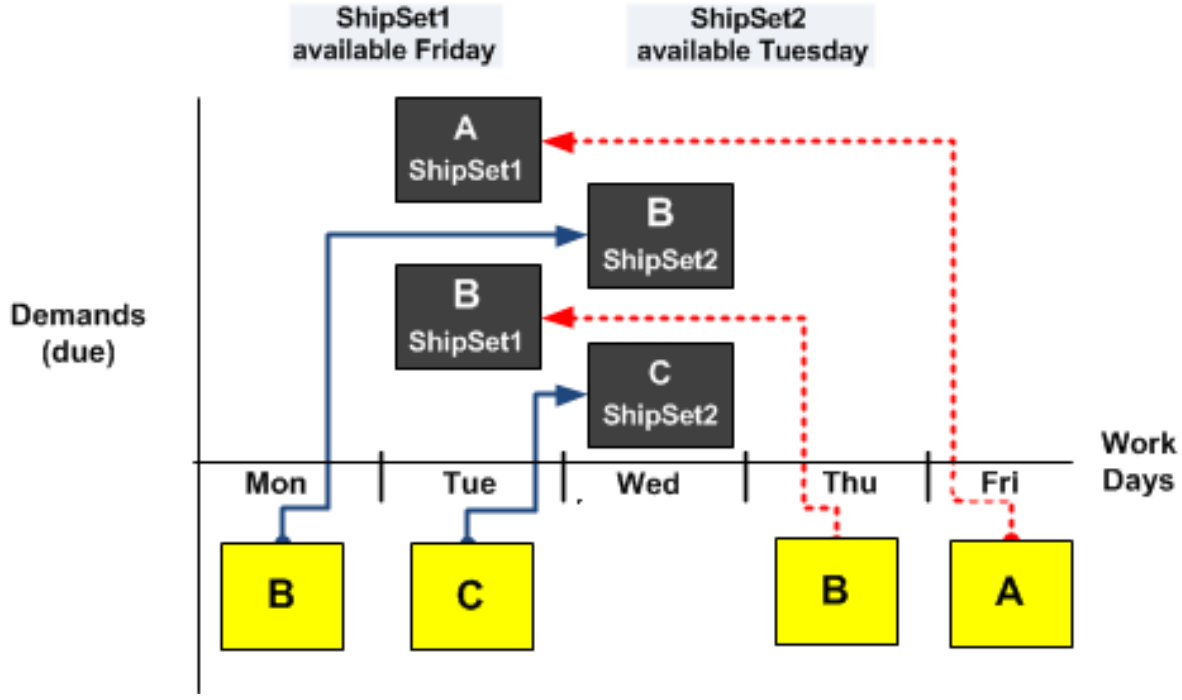
Non-blocking allocations can also be beneficial at the ship set level by allowing supply allotments within a ship set to be synchronized with the latest available date of any demands in that ship set. For example, assume 3 assemblies with independent demands belonging to 2 separate ship sets, and supplies available to satisfy those demands as shown in the following illustration. Using standard allotment logic, with the demands being satisfied by due date, neither ship set is available on time to provide all of its demands by their due date.

Figure 41.40 Allotments within ship sets before non-blocking allocations



This situation could be improved if enabling non-blocking allocation on the ship set by setting the **ShipSetType.MultiLevelSearchRule reference** to "Use" for ShipSet1 (assuming it was also enabled for the relevant part). RapidResponse would then identify that the demand for A, not available until Friday, is determining the ship set available date for ShipSet1. As such, any supply for B due on before Friday would be considered equally good within ShipSet1, and the later supply for B, available Thursday, could then be used instead to satisfy the demand for B due on Tuesday. This frees up the earlier supply of B, available on Monday, to satisfy the demand for B which is due on Wednesday and belonging to ShipSet2. As a result, ShipSet2 available on time to provide all of its demands by their due date, while not making ShipSet1 any worse than it was originally.

Figure 41.41 Allotments across ship sets are improved with non-blocking allocations



Examples

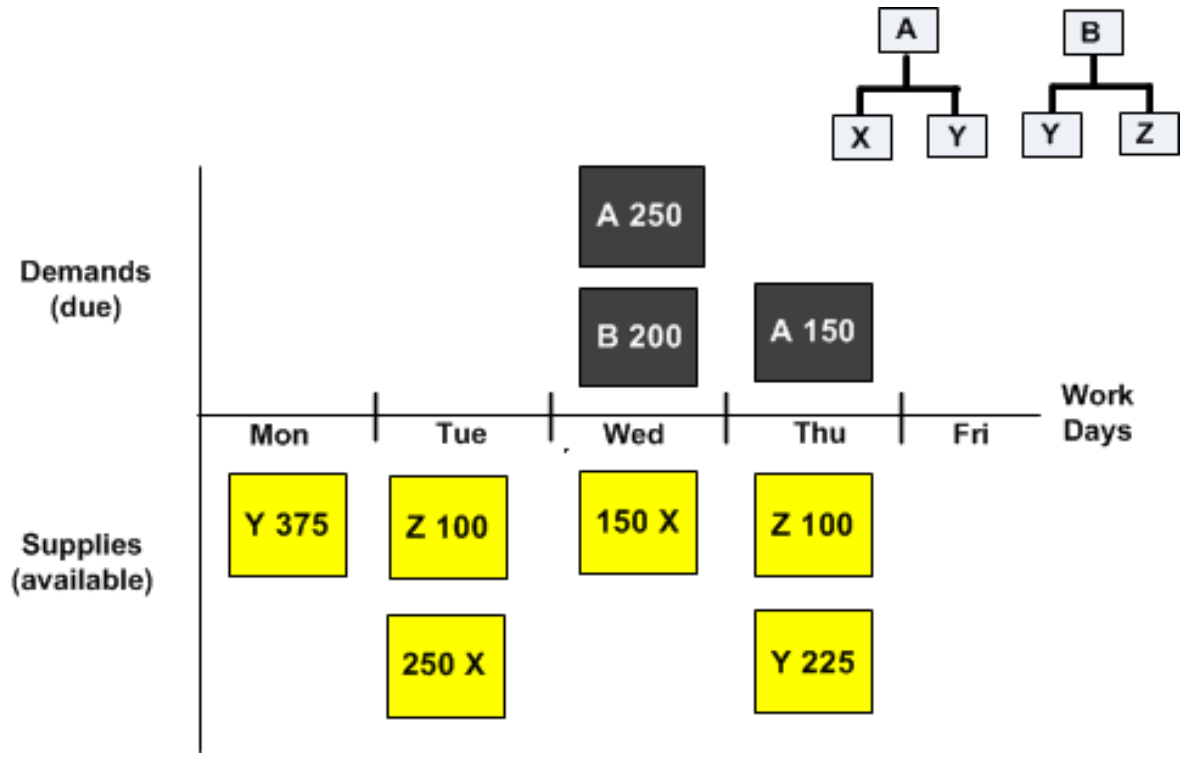
This section provides more detailed examples of supply allotment decisions made by RapidResponse with and without non-blocking allocations; both for demands that belong to a ship set as well as demands that do not belong to a ship set. The following scenarios are discussed:

- "Example 1: Normal demands" on page 1921
- "Example 2: Demands within a ship set" on page 1923

Example 1: Normal demands

Assume that Assembly A and Assembly B both share a common component Y and that there are demands for these two parts as shown in the following illustration. Further assume that each part has a lead time of 1 day.

Figure 41.42 Normal demands



With the standard supply allotment logic, each demand is allocated the available component supplies in due date order. Because there is not enough supply of shared component Y to satisfy both Wednesday demands on time, it is given first to the demand for B due to its being the smaller order (assuming its PartType.Allocation QuantityRule is set to the default value of "Use"). Supply allotments and on time demand availabilities would be as shown in the following table.

Table 41.12: Supply allotments without non-blocking allocations

Demands	Supply Allotments
B (Wed)	• 200 units of the supply of Y from Monday. • 100 units of the supply of Z from Tuesday. • 100 units of the supply of Z from Thursday. **100 units** of this demand are available on time and **100 units** are late (available Friday).
A (Wed)	• 175 units of the supply of Y from Monday. • 250 units of the supply of X from Tuesday. • 75 units of the supply of Y from Thursday. **175 units** of this demand are on time, and **75 units** are late (available Friday).

Table 41.12: Supply allotments without non-blocking allocations

Demands	Supply Allotments
A (Thu)	• 150 units of the supply of X from Wednesday. • 150 units of the supply of Y from Thursday. All **150 units** of this demand are late (available Friday).

In this scenario, potentially better allotments could be made to improve demand availabilities. If enabling non-blocking allocations (that is, setting the **MultiLevelSearchRule.ProcessingRule** reference to "Use" for these parts) then RapidResponse could re-allocate some of the supply of Y away from the demand for B (which does not have enough supply of its component to be fully available on time). This supply goes toward making the demand for A on Wednesday now fully on time and demand for A on Thursday partially on time. The supply allotments and on time availabilities of demands would be as shown in the following table (assuming the independent demands used the default **ShipmentPolicy.PartialRule** value of "Partial" which indicates that partial demand shipments are allowed).

Table 41.13: Supply allotments with non-blocking allocations - Partial shipment policy

Demands	Supply Allotments
B (Wed)	• 100 units of the supply of Y from Monday. • 100 units of the supply of Z from Tuesday • 100 units of the supply of Y from Thursday. • 100 units of the supply of Z from Thursday. **100 units** of this demand are available on time and **100 units** are late (available Friday).
A (Wed)	• 250 units of the supply of Y from Monday. • 250 units of the supply of X from Tuesday. All **250 units** of this demand are on time.
A (Thu)	• 25 units of the supply of Y from Monday. • 150 units of the supply of X from Wednesday. • 125 units of the supply of Y Thursday. **25 units** of this demand are on time, and **125 units** are late (available Friday).

Finally, assume the same settings as described in the previous block, but with the **ShipmentPolicy.PartialRule** on part B set to "Complete", indicating that the demand is expected to ship as a whole. In this scenario, more of the common component of Y is re-allocated away from the demand for B, and allotments would then be as shown in the following table. The net result is more of the demand for A on Thursday being satisfied on time and none of the demand for B being satisfied on time (but its available date remaining the same).

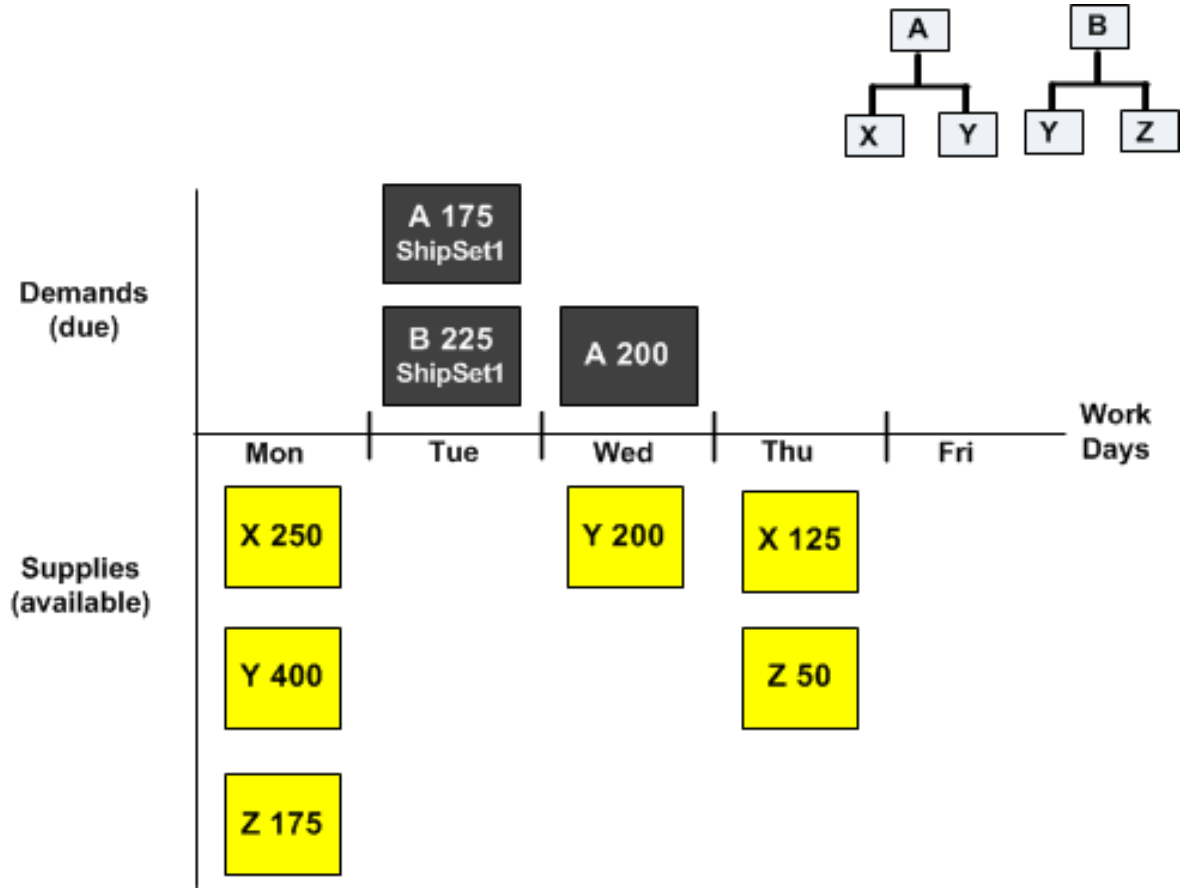
Table 41.14: Supply allotments with non-blocking allocations - Complete shipment policy

Demands	Supply Allotments
B (Wed)	• 100 units of the supply of Z from Monday. • 200 units of the supply of Y from Thursday. • 100 units of the supply of Z from Thursday All **200 units** of this demand are late (available Friday).
A (Wed)	• 250 units of the supply of Y from Monday. • 25o units of the supply of X from Tuesday. All **250 units** of this demand are on time.
A (Thu)	This demand is the latest of the lower priority demands and receives the limited on time component supplies last. Allotments are as follows: • 125 units of the supply of Y from Monday. • 150 units of the supply of X from Wednesday. • 25 units of the supply of Y from Thursday **125 units** of this demand are on time and **25 units** are late (available Friday).

Example 2: Demands within a ship set

Assume there are initial demands for assembly A and assembly B which share a common component and belong to the same ship set, as well as a subsequent demand for assembly A that is not associated with any ship set, as shown in the following illustration. Further assume that each assembly has a lead time of 1 day, and that the demand outside of the ship set has its **ShipmentPolicy.PartialRule** set to "Partial" (demands within a ship set are always treated as having a PartialRule of "Complete").

Figure 41.43 Demands within a ship set



With the standard supply allotment logic, in this case allocating supplies to demands by due date, each demand is allocated the available supplies as outlined in the following table:

Table 41.15: Supply allotments without non-blocking allocations

Demands	Supply Allotments
A (Tue)	• 175 units of the supply of Y from Monday. • 175 units of the supply of X from Monday. All **175 units** of this demand are seen as available on time (though its availability is ultimately tied to the rest of its ship set).
B (Tue)	• 225 units of the supply of Y from Monday. • 175 units of supply of Z from Monday. • 50 units of the supply of Z from Thursday. **175 units** of this demand are ontime and **50 units** are late (available on Friday; this date also determines the available date on the ship set).
A (Wed)	• 200 units of the supply of Y from Wednesday • 75 units of the supply of X from Monday. • 125 units of the supply of X from Thursday All **200 units** of this demand are late.

In this scenario, potentially better allotments could be made to improve the on time availability of the order for assembly A due on Wednesday, without impacting the other order's availability. If just enabling non-blocking allocations at the part level for these parts (that is, setting the **MultiLevelSearchRule.ProcessingRule** reference to "Use" for these parts) then RapidResponse would re-direct some of the earlier supply of common component "Y" from the demand for B due on Tuesday to the later demand for A due on Wednesday (up to the amount of the order for A that can be satisfied on time). Allotments would now be as shown in the following table:

Table 41.16: Supply allotments with non-blocking allocations - at part levels

Demands	Supply Allotments
A (Tue)	• 175 units of the supply of Y from Monday. • 175 units of the supply of X from Monday All **175 units** of this demand are seen as available on time (though its availability is ultimately tied to the rest of its ship set).
B (Tue)	• 150 units of the supply of Y from Monday • 175 units of the supply of Z from Monday • 75 units of the supply of Y from Wednesday • 50 units of the supply of Z from Thursday **150 units** of this demand are on time and **75 units** are late (the whole order, and hence the ship set, is available on Friday).
A (Wed)	• 75 units of the supply of Y from Monday. • 75 units of the supply of X from Monday. • 125 units of the supply of Y from Wednesday. • 125 units of the supply of X from Thursday **75 units** of this demand are on time and **125 units** are late (available on Friday).

Note that although a portion of the demand for A due on Wednesday is now available on time, further improvements could be seen by enabling non-blocking allocations on ShipSet1 (that is, setting its **ShipSetType.MultiLevelSearchRule** value to "Use"). In this case, RapidResponse could also re-allot the earlier supply of X from the Tuesday demand for A to the Wednesday demand for A as the Tuesday demand's availability is limited by its ship set availability. Because more of component supply X is now made available to the Wednesday demand, this demand also receives an additional amount of the on time supply of component supply Y. Allotments would now be as shown in the following table.

Table 41.17: Supply allotments with non-blocking allocations - enabled at part and ship set level

Demands	Supply Allotments
A (Tue)	• 175 units of the supply of Y from Monday. • 50 units of the supply of X from Monday. • 125 units of the supply of X from Thursday **50 units** of this demand are seen as available on time and **125 units** are late (available Friday).
B (Tue)	• 25 units of the supply of Y from Monday. • 175 units of the supply of Z from Monday. • 200 units of the supply of Y from Wednesday. • 50 units of the supply of Z from Thursday. **25 units** of this demand are on time and **200 units** are late (available Friday).
A (Wed)	• 200 units of the supply of Y from Monday. • 200 units of the supply of X from Monday. All **200 units** of this demand are on time.

As a net result, the demand for A due on Wednesday is now fully available on time, while the ship set remains available on Friday. Although smaller portions of each of the individual demands within the ship set are seen as on time, the ship set available date is identical to the initial allotment shown in this example when none of the Wednesday demand was on time.

Limitations on other analytics

Although multi-level search logic can help improve decisions made by RapidResponse either when choosing a source to supply an order or when allocating component supplies to assembly demands, it introduces limitations on certain other RapidResponse analytics. When enabling multi-level search logic for a given part, the impacts on the analytics outlined in the following table should be considered.

Table 41.18: Multi-level search limitations on other analytics

Analytic / Calculation	Impact of multi-level sourcing
Safety stock	Component parts that have multi-level search logic enabled should not use dynamic safety stock policies. The only **SafetyStockQuantityRule** options supported for components using multi-level search are "FixedQuantity", "TimePhasedQuantity", and "TimePhasedQuantityToLatest". However, all safety stock policies (including dynamic policies such as "FractionOfDemand", "RangeOfCoverage", and so on) are supported for top-level assembly parts that have multi-level search logic enabled.
Order point	Parts that have multi-level search should not be set up as order point parts. The PartType.SafetyStockRule values of “OrderPointAverage”, “OrderPointOver”, and “OrderPointUnder” are not supported when a part has multi-level search logic enabled.
MPS config	Multi-level search logic is not supported for MPSConfig parts (always disabled for those parts).
Dependent forecast consumption	Dependent forecast consumption is not supported on parts that have multi-level search logic enabled (only independent demands on part enabled for multi-level search logic are eligible for forecast consumption).
BOM substitution	BOM-level part substitution is generally supported for parts that have multi-level search logic enabled. However the ability to define an excess window, such that only excess current supply is used for the substitution, is not supported. That is, if multi-level search logic is enabled then **PartType.ExcessRule** settings other than "Ignore" should be avoided as they might not return the expected results.
BOM Substitution (with partial planning)	In cases where multi-level search logic is implemented with partial planning logic (DemandType.PlanningRule = “Partial”), BOM Substitution is supported as long as the relevant SusbtituteGroupType.ComponentRule setting is either “AssemblyUnit” or “Unrestricted”. However, if ComponentRule is set to “Single” then the substitution logic will not work as expected (MLS partial planning will not provide partial supply).

Table 41.18: Multi-level search limitations on other analytics

Analytic / Calculation	Impact of multi-level sourcing
Global substitution	Global part substitution is generally supported for parts that have multi-level search logic enabled. However the ability to define an excess window, such that only excess current supply is used for the substitution, is not supported. That is, if multi-level search logic is enabled then **PartType.ExcessRule** settings other than "Ignore" should be avoided as they might not return the expected results. Additionally the criterion for multi-level search logic to prefer sources that would use a scheduled receipt (even if it unallocated) at any level over sources that require a new planned order is only supported when **PartType.SubstitutionPreference** is set to "ScheduledSupply" (for example, unexpected results might be seen with the "OnHand" option).
Kanban	Kanban calculations are turned off for any parts that have multi-level search logic enabled.
Feature BOM	Multi-level search logic is supported on configurable end-items, however this should be done with caution as these calculations might perform slowly on parts that have high volumes of orders.
Input supply usage	Sometimes, during the transition period between when input supplies are used up and planned orders are required, input supply might be used in cases where the creation of a new planned order would result in better overall availability when considering later demands (however, input supply that has already been used to cover a demand cannot be reclaimed or recovered for use elsewhere).
Priority	When multi-level search is enabled, non-blocking allocation logic still respects order priority and is only applied across demands having the same priority level. In other words, a lower priority demand will not take limited component supply that would otherwise be given to a higher priority demand, even if it can do so without making that higher priority demand worse off. Non-blocking allocation logic will only redirect supplies that would otherwise be given to demands of the same priority level.
Takt time	Takt time calculations are turned off for any parts that have multi-level search logic enabled.
AllotmentRule options in multi-sourcing	Multi-level search logic does not support **SourceRule.AllotmentRule** options of “OngoingToDate” or “ProportionalLotSize”. If parts configured for multi-level search have their **AllotmentRule** set to “OngoingToDate” they will use the “Ongoing” option instead, and if parts configured for multi-level search have their **AllotmentRule** set to “ProportionalLotSize” they will use the “Proportional” option instead. Additionally, the ability to reduce build up of short-term excess supply with the **MultiLevelSearchRule.ShortTermExcess** setting of "Reduce" is not supported on parts that use a **SourceRule.AllotmentRule** setting of "Proportional" (can cause supplies to be delayed further than they should based on proportional targets).
Attribute-based planning	Caution should be taken if using multi-level search logic along with attribute-based planning. The use of these two analytics together, particularly alongside other capabilities such as lot-sizing and priority, can result in excess supply being planned. Additionally, the ability to reduce build up of short-term excess supply with the **MultiLevelSearchRule.ShortTermExcess** setting of "Reduce" is not fully supported on parts when attributes are involved (might result in supplies being delayed further than they should based on proportional targets).

Table 41.18: Multi-level search limitations on other analytics

Analytic / Calculation	Impact of multi-level sourcing
Allotment overrides	Allotment overrides are generally supported and compatible with multi-level search logic. However, care should be take to avoid using multi-level search logic with allotment overrides that represent an over-reservation of component supply (that is, where the amount of the override exceeds the effective requirement for the item). In some cases, this combination can then lead to unwanted excess supply being generated. Additionally, if multi-level search logic is used along with allotment overrides for a part that has "reschedulable" scheduled receipts, those receipts might not always be rescheduled correctly with respect to the override (for example, might be rescheduled earlier than needed).
Inventory pooling	Inventory pooling logic is generally supported with multi-level search. However, if using multi-level search along with inventory pooling and constraints, excess planned orders might be generated in some cases where an allocated scheduled receipt exists that could have been used to satisfy the demand instead.
Days of supply	Days of supply planning should be avoided for parts that use multi-level search logic. If using multi-level search logic, then planned order due dates will not always fall in line with or respect days of supply policies.
FixedConstraintFactor	When incremental availability is turned on, as is recommended for multi-level search, planned orders are generated when the material and constraint is available. This satisfies the total demand quantity in split planned orders, however, each order consumes the entire fixed constraint amount. This results in an overstatement of the total amount of fixed constraint consumed.

Table 41.18: Multi-level search limitations on other analytics

Analytic / Calculation	Impact of multi-level sourcing
Lot-sizing with constraints	When using multiple-level search with constraints, the creation of planned orders for less than the minimum order quantity can occur if there is insufficient constraint or material for a part or, more likely, one of its components. The logic will always try to ensure there is enough component material collected and rolled up to satisfy the assembly's lot size. However, in cases where a component is severely limited in material or constraint, then lesser quantities that do not allow respecting the assembly's lot size might be rolled up. Therefore if the part source has lot-sizing requirements that need to be respected in environments that use both multi-level search and constrained planning, it is recommended to ensure that **MultipleQty** is set to a positive value (for example, this might typically be the same as the **MinimumQty**) and that the **PartSourceType.ConstraintMultipleRule** is set to "Use". This can help mitigate against planned orders being created for less than the minimum quantity as described above. However, these settings can have a small impact on solution quality and might leave small amounts of unconsumed component material when they are not large enough to push an entire lot through the network. **NOTE**: The **ConstraintMultipleRule** setting of "Use" also ensures the multiple itself is generally respected when using multi-level search along with constraints (it might not be in some cases where the "MultiplePeriods" rule is used instead).
Material expiry	Support for material expiry logic when using multi-level search was added in Version 2016.2, Service Update 27. This enabled multi-level selection decisions to include consideration for expiry risk such as by choosing paths that use earlier expiring component supply. However, when material expiry is used together with multi-level search, demands might not all be processed in date sequence (which is otherwise the case with material expiry), and the following limitations should be kept in mind. • The **MultiLevelSearchRule.ShortTermExcess** setting of "Reduce" is not supported with material expiry. Thus if multi-level search and material expiry are enabled together, the **ShortTermExcess** setting of "Allow" should be used. • The ability for material expiry logic to refresh safety stock supply to mitigate expiry risk is not supported when multi-level search is enabled. And although the combination of multi-level search and material expiry along with attribute-based planning is supported, this combination of analytics is not supported if there is also safety stock requirements. Thus, multi-level search should generally be avoided on parts that require material expiry, attribute-based planning, and safety stock (or consider disabling safety stock in scenarios that use multi-level search along with expiry and ABP). • Other limitations that are specific to material expiry (regardless of using multi-level search) are also applicable as discussed in "Limitations on other analytics" on page 1626. For example, when using material expiry, any order priorities are automatically assumed to be period effective (thus fields such as **PlanningCalendars.AllocationCalendar** should be set appropriately).

MLS infrastructure changes (optional and limited release)

A Service Update expected to be released in early 2020 will contain significant changes to the underlying infrastructure supporting multi-level search logic. When these changes are introduced, the intent,

configuration, and benefits of the multi-level search analytic will remain the same.

However, the new multi-level search infrastructure should result in improved performance and stability while also supporting scalability and future enhancements to the analytic. In addition, under certain specific conditions, changes or improvements to solution output might be seen.

In this service update (1908), you can optionally enable a limited release of the new multi-level search infrastructure. This enables you to see any performance or output changes you are likely to encounter when the new infrastructure is officially released as a permanent replacement for the old infrastructure.

Note however that this it is not yet an officially supported enhancement and further changes might still be made before the official release. Therefore, it should be used with caution and any significant or unexpected changes to solution output should be reported to Kinaxis Customer Support for investigation.

This section covers the following:

- "Overview of MLS infrastructure changes" on page 1930
- "Enable new MLS infrastructure" on page 1931
- Measure impact of new MLS infrastructure

Note: The **MLS Configuration** workbook and **MLS Performance Manager** dashboard discussed in this section are available in the **Non-Application Specific** resource package. If you or your organization need access to these resources, please contact your RapidResponse administrator or Kinaxis Customer Support.

Overview of MLS infrastructure changes

Changes to the multi-level search infrastructure will generally result in some noticeable performance and stability improvements, while also supporting future scalability and planned feature enhancements. Additionally, these architectural changes might result in better solution output in some cases due to improvements in how multi-level search trial operations process material and constraint.

Typical differences that might be observed when using the new MLS infrastructure include the following areas:

Constraint and material utilization

- Resolution of an issue where materials and/or constraint could sometimes be left somewhat under-utilized or over-utilized.
- Better and more accurate part source selection decisions due to improved material and constraint utilization might be reflected throughout the product structure.

Scheduled receipts

- Better component supply decisions under scheduled receipts as dependent demands do not have to be satisfied by a single order.
- In some cases, satisfying dependent demands with multiple supplies now allows for consuming small amounts of excess supply from different paths under a scheduled receipt.
- Improved constraint consumption under some scheduled receipts.

New purchase costs

- New purchase costs have been moved higher in the MLS selection criteria.
- Minimizing purchase costs is now preferred over using least depth of supply.
- If using the optional MLS selection criteria to minimize purchase costs, differences might be seen in cases where very low-yielding part sources that are higher in the structure were previously chosen (lower-level but higher yielding sources might now be chosen).

Performance

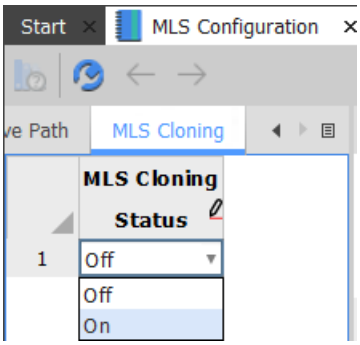
- Faster solution performance due to ability to roll up, in a single pass, multiple on hand and excess supply orders to meet a demand quantity (previously multiple passes were required)
- Faster solution performance due to memory allocation savings of 5-20% in typical scenarios.

Note: If you are concerned with any significant or unexpected differences encountered when using the new MLS infrastructure, please contact Kinaxis Customer Support.

Enable new MLS infrastructure

The limited release of the new multi-level search infrastructure can be manually enabled on a scenario-by-scenario basis. For example, if you have an existing scenario that already provides MLS solution output you are satisfied with, you might create a child of that scenario and enable the new infrastructure there. This would allow you to then compare the two scenarios and determine any differences between the new and old versions of the MLS infrastructure.

The limited version of the new MLS infrastructure can be enabled in a given scenario by using the **MLS Configuration** workbook to add a configuration record.



To enable or disable the new MLS infrastructure

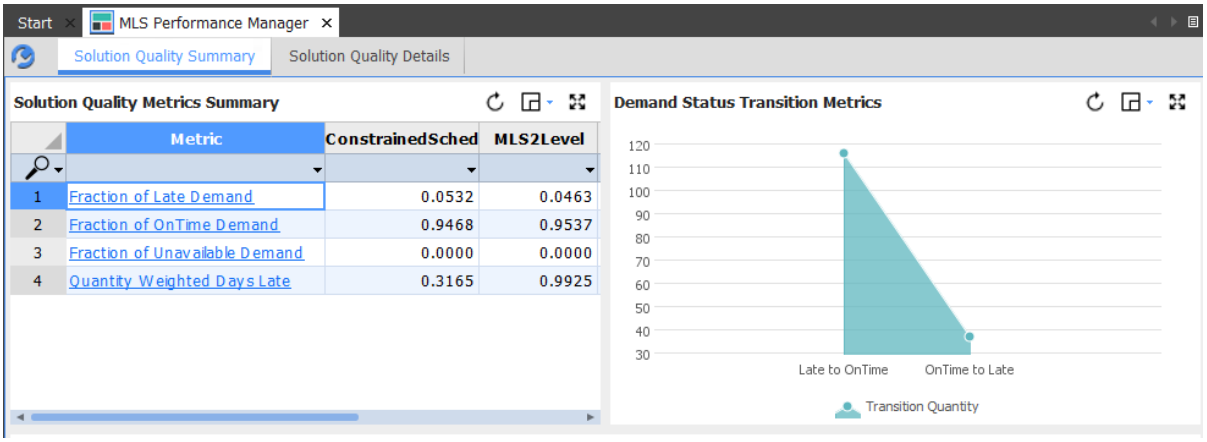
1. Ensure the scenario(s) in which you want to enable the new MLS infrastructure has been created.
2. In **Explorer**, double-click to open the **MLS Configuration** workbook.
3. Click the **MLS Cloning** worksheet.
4. From **Scenario** control, select the scenario on which you want to enable the new MLS infrastructure.
5. From the **Edit** menu, click **Insert Record** and then click **Save and Close**.

6. Click in the **MLS Cloning Status** column and toggle one of the following:
   - **On**— enables the new MLS infrastructure.
   - **Off**—disables the new MLS infrastructure. This is the default setting and uses the previous MLS infrastructure instead.
7. Save your change.

Measure impact of new MLS infrastructure

If using the new multi-level search infrastructure, you should evaluate the scenarios in which it is enabled against any established metrics or key performance indicators your organization normally uses to measure MLS solution quality. Any unexpected differences or degraded solution quality should be reported to Kinaxis Customer Support.

You can also make use of the new **MLS Performance Manager** dashboard included with RapidResponse. This dashboard reports a number of demand fulfillment metrics and allows a comparison between two specified scenarios. Typically, you might use this compare a scenario that uses the new MLS infrastructure with one that uses the old MLS infrastructure. For example, you can compare measures of on-time and lateness in each scenario and identify any demands whose availability status changed between the two scenarios.



1. In **Explorer**, double-click to open the **MLS Performance Manager** dashboard.
2. In the **Data Settings** pane under **Scenarios**, ensure exactly two scenarios are selected for comparison.
3. As appropriate, set the part filter, site filter, and any other data settings.
4. On the **Solution Quality Summary** tab, review the following metrics. As necessary, you can drill down to underlying worksheets for more detail.
   - **Solution Quality Metrics Summary**—contains various demand fulfillment metrics that report the percentage of demands that are expected to be on-time, late, and left unsatisfied in each of the two scenarios. A weighted metric based on the number of days late is also shown.
   - **Demand Status Transition Metrics**—summary report of demands whose availability changed between the two scenarios. For example, you can identify demands that were late in one scenario but on time in the other.

- **Demand Order Status by Due Date**—reports demands grouped into configurable buckets and identifies the number on-time, late, or unsatisfied demands per scenario in each bucket.

5. On the **Solution Quality Details** tab, review the following metrics. As necessary, you can drill down to underlying worksheets for more detail.
   - **OnTime Demand by Priority**—Compares on-time demands at each order priority level between the two scenarios.
   - **Late Demand by Priority**—Compares late demands at each order priority level between the two scenarios.
   - **Unavailable Demand by Priority**—Compares demands left unsatisfied at each order priority level between the two scenarios.
   - **Categorized Late Quantity**—Compares supplies expected to be late to their demands in each of the two scenarios, reported in four configurable date buckets.

Note: For more information on these MLS metrics, refer to the help included with the MLS Performance Manager dashboard and associated worksheets.

CHAPTER 42: Inventory planning and optimization

Safety stock represents an inventory reserve that is maintained as a buffer against demand variability and uncertainty. Determining the appropriate safety stock level for a part typically requires balancing the cost of potential lost sales when stockouts occur against the capital and holding costs incurred from carrying excess inventory. In RapidResponse, the safety stock level for a part is provided as an input value in either the **Part** table (for a stationary value) or the **TimePhasedSafety** table (for time-phased values). Based on the safety stock values provided, RapidResponse planning analytics ensure supplies are created as necessary to maintain the desired inventory levels for each part.

Optionally, RapidResponse can be configured to recommend safety stock levels for a part using either single-echelon inventory planning (SEIP) or multi-echelon inventory optimization (MEIO) methods. Based on a part's historical demand variability and other input parameters, safety stock levels are recommended to ensure adequate inventory is on hand to achieve a given customer service level while reducing the costs associated with holding inventory beyond that level. After review of the recommendations, automated processes can be used to replace the input safety stock levels with the calculated recommendations.

This chapter provides an overview of both single-echelon inventory planning and multi-echelon inventory optimizations, describes the key tables and fields used to support each, outlines the steps required to configure a part for either single or multi-echelon methods, and describes details of the calculations performed by each.

In this chapter, you'll learn about:

Single and multi-echelon basics **1936**
Safety stock item data model **1937**
Safety stock item configuration **1946**
Single-echelon safety stock calculations **1968**
Safety stock item outlier processing **1983**
Multi-echelon safety stock calculations **1989**
Automated safety stock updates **2012**
Comparing single-echelon settings and multi-echelon settings **2014**

Note: The Inventory Planning workbook included with RapidResponse can be used to configure parts as safety stock items and reports details of single and multi-echelon safety stock calculations. For more information about the Inventory Planning workbook, see the *RapidResponse Applications Guide* or refer to the workbook help.

Single and multi-echelon basics

Single-echelon inventory planning and multi-echelon inventory optimization are two methods for determining the appropriate levels of safety stock to be maintained. In RapidResponse, single-echelon inventory planning and multi-echelon inventory optimization relies on an analytic.

Single-echelon inventory planning

Single-echelon inventory planning involves the generation of recommended safety stock levels for a single part at a time. Although any number of parts might be configured for single-echelon calculations, the recommendations for each part are made independently of any other related parts.

Figure 42.1 Single-echelon inventory planning for a part

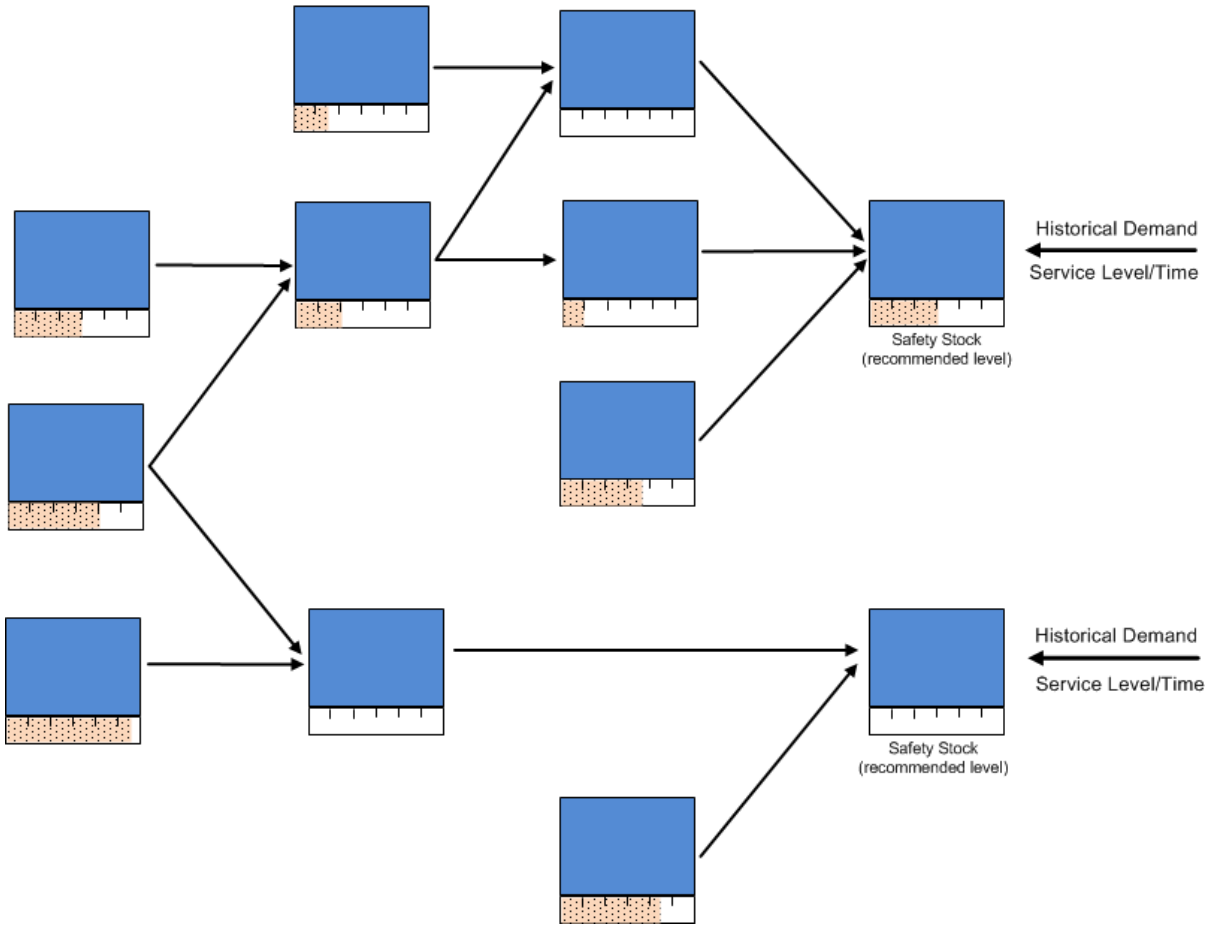
Single-echelon calculations consider both the variability of a part's historical demands and, optionally, the variability of its historical supply lead times. Parameters based on historical data are calculated and then used as input in statistical formulas that generate the recommended safety stock level(s) for the part. The recommendations are made to satisfy a specified customer service level and can be either stationary or time-phased.

Suggested reorder points, based on both historical and future looking data, required to maintain the recommended safety stock levels are also generated for each part.

Multi-echelon inventory optimization

Multi-echelon inventory optimization involves the simultaneous generation of recommended safety stock levels for multiple parts related through bill of material and part source relationships. Not only do these recommendations determine the amount of safety stock needed to satisfy defined customer service requirements, but the strategic placement of that safety stock across the network or family of related parts is also taken into account. For example, it might be more cost effective to hold inventory stores of lower level components and sub-assemblies than of the final end item itself.

Figure 42.2 Multi-echelon inventory optimization for a network of parts



Multi-echelon inventory optimization considers historical demand data which is collected for customer-facing end items only and those demand values are then propagated to the dependent parts in the network. For each end item, a desired customer service level is also required as input along with a service time. The service time defines the maximum time within which the item guarantees its orders can always be able to be satisfied. Optionally, each part in a multi-echelon family can either assume a fixed-lead time based on **PartSource** values (by default), or variable lead time can be assumed in which lead time parameters, such as average and standard deviation of demand, are calculated from actual historical supply records.

Based on the calculated demand parameters, optimization methods are then used to determine the optimal guaranteed service time for all other parts within the network. The service times returned are those which ensure the specified service level(s) can be satisfied while minimizing the total cost of holding inventory across the network of parts. The optimized service times are then used in statistical formulas to calculate the safety stock level(s) to maintain for each part. These safety stock recommendations can be either stationary or time-phased.

Safety stock item data model

A number of input, control, and calculated tables are available to define parts as safety stock items, configure those items to generate safety stock recommendations, and subsequently report those

recommendations and their related parameters. This section provides a high-level overview of the tables that are available to support single-echelon inventory planning, and multi-echelon inventory optimization, or both.

- "Single-echelon data model" on page 1938
- "Multi-echelon data model" on page 1942

Notes:

- In addition to the tables outlined in this section, other related tables are available to support updating any input fixed or time-phased safety stock values that have been provided with the recommendations made by single and multi-echelon calculations. For more information, see "Automated safety stock updates" on page 2012.
- Also in addition to the tables outlined in this section, as of Version 2016.2, Service Update 5, other related tables are available to support outlier detection and correction for both single and multi-echelon calculations. For more information, see "Safety stock item outlier processing" on page 1983

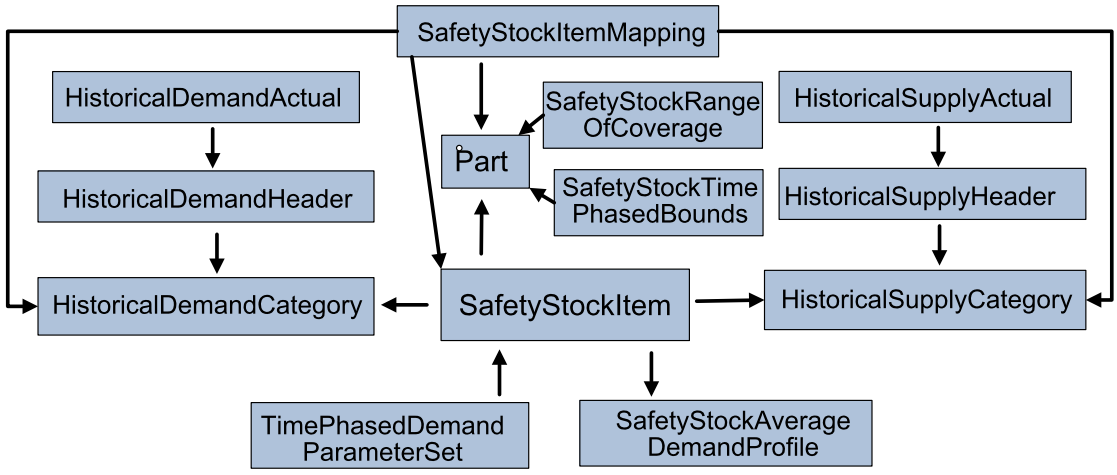
Single-echelon data model

The section provides an overview of the key input, control, and calculated tables used to support single-echelon inventory planning.

Input tables

The main input table used to support single-echelon calculations in RapidResponse is the **SafetyStockItem** table. As shown in the following illustration, this table references the part for which safety stock recommendations should be calculated, along with the specific categories of historical data that should be used as input.

Figure 42.3 Single-echelon input tables



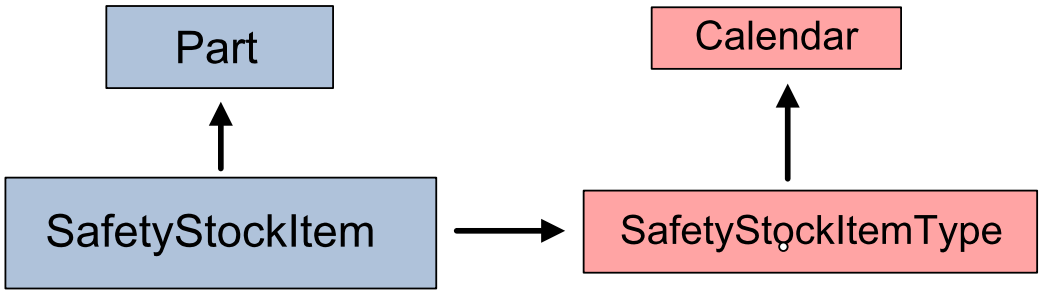
The following describes the input tables shown in the previous illustration.

- **SafetyStockItem**—identifies the items (parts) for which safety stock recommendations should be generated and contains item-specific parameters that define how those calculations are performed (for example, the service level that should be supported and whether an item is subject to single or multi-echelon processing). For more information, see "SafetyStockItem table" on page 541.
- **Part**—contains details of the specific part that the SafetyStockItem and SafetyStockTimePhasedBounds record is being defined for and to which recommended safety stock levels, reorder points, and safety stock bounds apply. For more information, see "Part table " on page 415.
- **SafetyStockTimePhasedBounds**—contains minimum and maximum safety stock bounds for parts and the dates on which these bounds start to apply. For more information, see "SafetyStockTimePhasedBounds table" on page 565.
- **SafetyStockRangeOfCoverage**—defines the range over which average demand is calculated for the purpose of calculating a part's recommended safety stock using the "DaysOfSupplyForward" or "DaysOfSupplyBackward" processing rule. For more information, see "SafetyStockRangeOfCoverage table" on page 565.
- **HistoricalDemandCategory**—identifies categories of historical demands, and is referenced from the SafetyStockItem table to determine the category of historical demands that are used in safety stock calculations for an item. For more information, see "HistoricalDemandCategory table" on page 329.
- **HistoricalDemandActual**— contains the actual details of historical customer orders/shipments that are used as input in determining average and standard deviation of historical demands. These are key parameters used in determining the recommended safety stock levels and reorder points for the item. The records used in this table are those that match an item's HistoricalDemandCategory reference. For more information, see "HistoricalDemandActual table" on page 325.
- **SafetyStockAverageDemandProfile**—contains optional profiles based on lead time multiples and other parameters that define a horizon over which historical and/or future demands are collected for use in calculating average demand when determining recommended reorder points. For more information, see "SafetyStockAverageDemandProfile table" on page 540.
- **HistoricalSupplyCategory**—identifies categories of historical supplies, and is referenced from the SafetyStockItem table to determine the category whose historical supplies are used to determine variability of lead time for an item (if applicable). For more information, see "HistoricalSupplyCategory table" on page 350.
- **HistoricalSupplyActual**—contains the actual details of historical supply orders, such as the date an order was placed and the date it was actually received, for use when calculating variability of lead time for the safety stock item. The records used in this table are those that match an item's HistoricalSupplyCategory reference.For more information, see "HistoricalSupplyActual table" on page 347.
- **SafetyStockItemMapping**—allows a safety stock item to be mapped to other parts and have their demand and/or supply data included in safety stock calculations for the item (for example, different revisions of a part might be mapped). For more information, see "SafetyStockItemMapping table" on page 554.
- **TimePhasedDemandParameterSet**—allows for manually specifying time-phased demand parameters (average and standard deviation of demand) in cases where there is no sufficient historical data available to calculate them (for non time-phased items, parameters can be manually specified in the SafetyStockItem table). For more information, see "TimePhasedDemandParameterSet table" on page 653.

Control tables

A single control table defines the common configuration parameters that can be used by single-echelon safety stock items as shown in the following illustration.

Figure 42.4 Single-echelon control tables



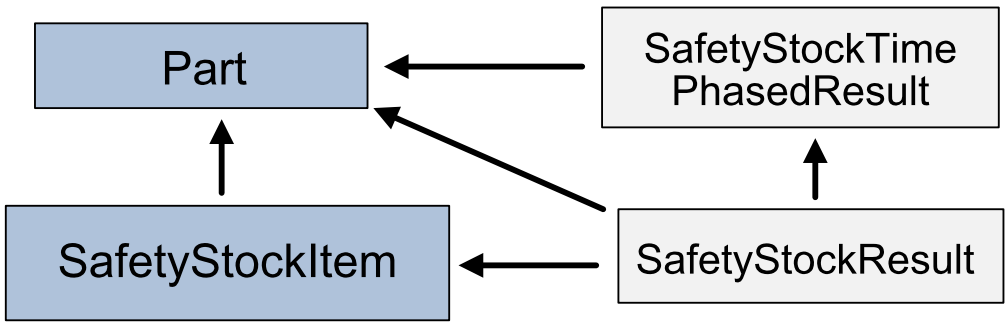
The following describes the control tables shown in the previous illustration.

- **SafetyStockItemType**—contains control settings applied to safety stock items such as the calendars to use for safety stock calculations, the method used for determining standard deviation, and whether an item should generate stationary or time-phased safety stock results. For more information, see "SafetyStockItemType table" on page 861.
- **Calendar**—contains the named calendars referenced by various fields on the SafetyStockItemType table. For example, the names of the inner and outer calendars used in time-phased calculations. For more information, see "Calendar table" on page 695.

Calculated tables

As shown in the following illustration, a series of calculated tables report details of single-echelon calculations.

Figure 42.5 Single-echelon calculated tables



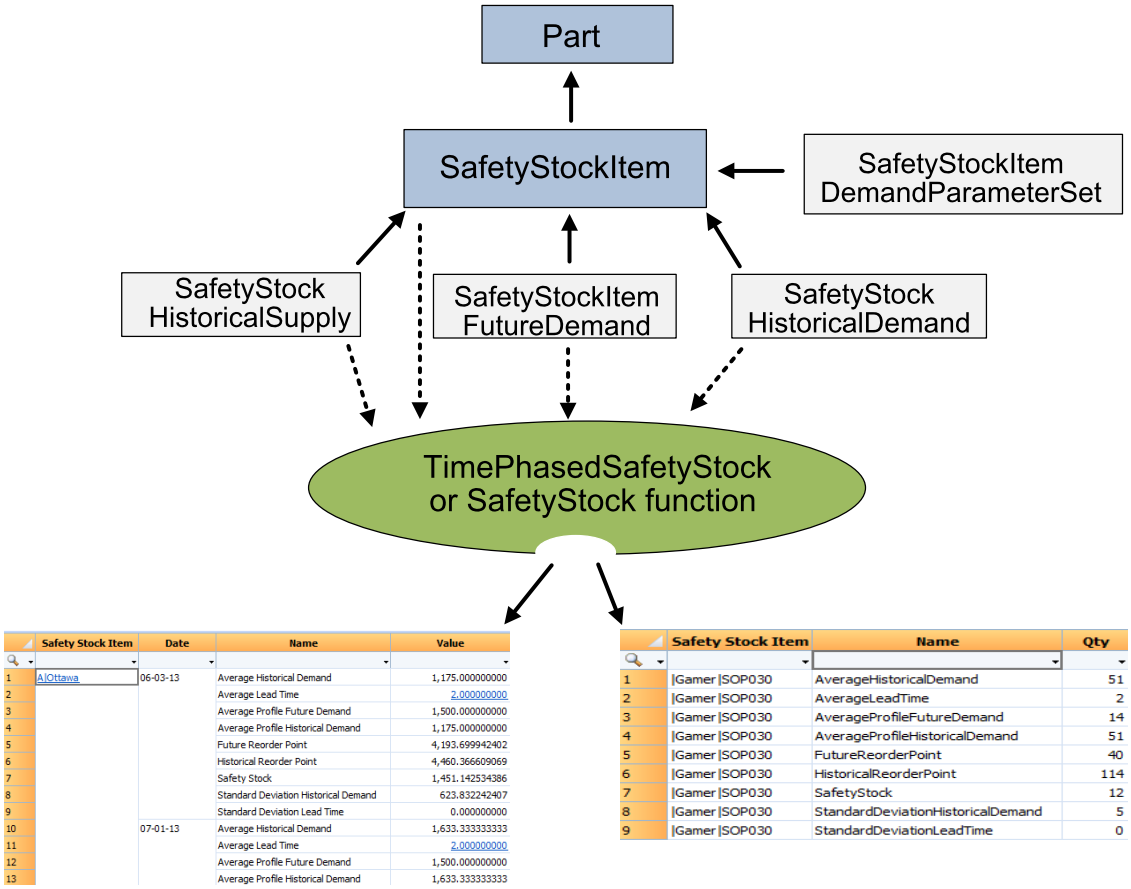
The following describes the calculated tables shown in the previous illustration.

- **SafetyStockResult**— reports safety stock calculated from historical data along with related parameters applicable to both multi-echelon inventory optimization and single-echelon inventory planning calculations**.** For more information, see "SafetyStockResult table" on page 1169.
- **SafetyStockTimePhasedResult**— reports time-phased safety stock and reorder points recommendations from both multi-echelon inventory optimization and single-echelon inventory planning calculations. For more information, see "SafetyStockTimePhasedResult table" on page 1174.

Legacy calculated tables and functions

A series of calculated tables are populated for single-echelon safety stock items and contain calculated details of each item's historical demands, future demands, and historical supplies. As shown in the following illustration, worksheets based on many of these tables are subsequently required as inputs by the TimePhasedSafetyStock and SafetyStock functions that produced single-echelon safety stock and reorder point recommendations in all versions prior to Version 2014.4. As of Version 2014.4, however, use of the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables are recommended instead because they report calculated results without the need for transformation worksheets and also allow reporting both single and multi-echelon results from the same table.

Figure 42.6 Single-echelon calculated tables providing support for legacy functions



	Safety Stock Item	Date	Name	Value
1	AlOttawa	06-03-13	Average Historical Demand	1,175.000000000
2			Average Lead Time	2.000000000
3			Average Profile Future Demand	1,500.000000000
4			Average Profile Historical Demand	1,175.000000000
5			Future Reorder Point	4,193.699942402
6			Historical Reorder Point	4,460.366609069
7			Safety Stock	1,451.142534386
8			Standard Deviation Historical Demand	623.832242407
9			Standard Deviation Lead Time	0.000000000
10		07-01-13	Average Historical Demand	1,633.333333333
11			Average Lead Time	2.000000000
12			Average Profile Future Demand	1,500.000000000
13			Average Profile Historical Demand	1,633.333333333

	Safety Stock Item	Name	Qty
1	|Gamer |SOP030	AverageHistoricalDemand	51
2	|Gamer |SOP030	AverageLeadTime	2
3	|Gamer |SOP030	AverageProfileFutureDemand	14
4	|Gamer |SOP030	AverageProfileHistoricalDemand	51
5	|Gamer |SOP030	FutureReorderPoint	40
6	|Gamer |SOP030	HistoricalReorderPoint	114
7	|Gamer |SOP030	SafetyStock	12
8	|Gamer |SOP030	StandardDeviationHistoricalDemand	5
9	|Gamer |SOP030	StandardDeviationLeadTime	0

The following describes the calculated tables shown in the previous illustration.

- **SafetyStockHistoricalDemand**—reports bucketed historical demands for safety stock items that pertain to the historical demand category and date range specified on the SafetyStockItem record. For more information, see "SafetyStockHistoricalDemand table" on page 1161.
- **SafetyStockHistoricalSupply**—for safety stock items that consider supply variability, reports historical supplies that pertain to the historical supply category and date range specified on the SafetyStockItem record. For more information, see "SafetyStockHistoricalSupply table" on page 1163.
- **SafetyStockItemFutureDemand**—reports bucketed current and future demands for safety stock items, as found in the Demand table, for the date range specified on the SafetyStockItem record. For more information, see "SafetyStockItemFutureDemand table" on page 1165.
- **SafetyStockItemDemandParameterSet**—not used by the safety stock function, but reports calculated demand parameters (average and standard deviation of historical demands) that are generated and used by those functions. For more information, see "SafetyStockItemDemandParameterSet table" on page 1164.

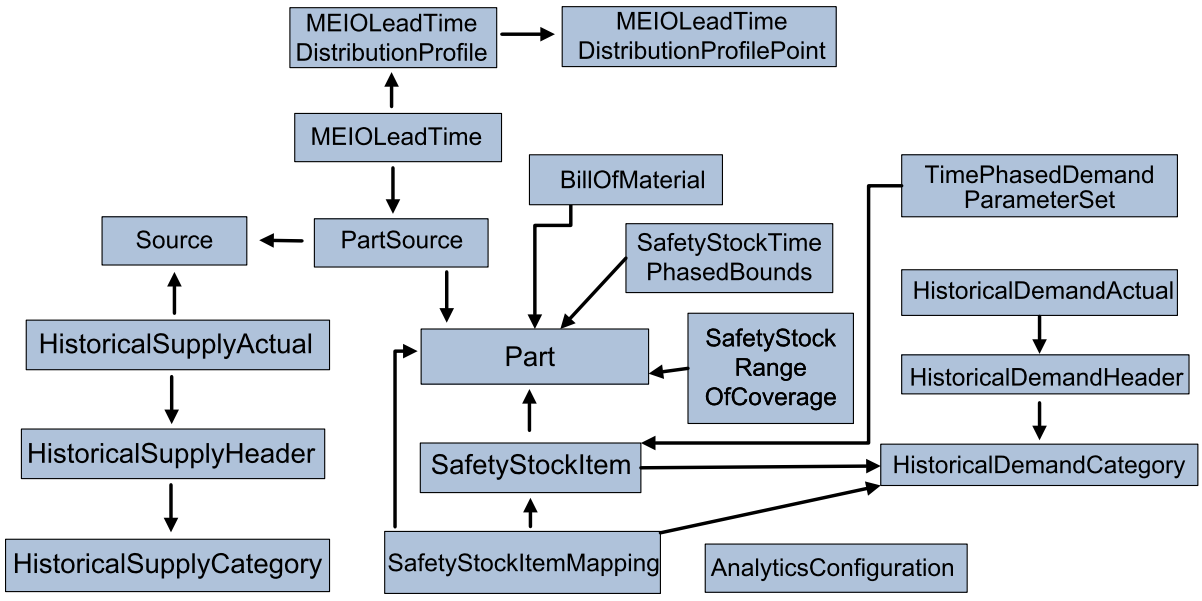
Multi-echelon data model

This section provides an overview of the key input, control, and calculated tables used to support multi-echelon inventory optimization.

Input tables

The main input table used to support multi-echelon calculations in RapidResponse is the **SafetyStockItem** table. As shown in the following illustration, this table references the end-item part under which families of parts should be built and safety stock recommendations subsequently calculated. It also references the historical demand category containing the end item's historical data from which demand parameters are generated.

Figure 42.7 Multi-echelon input tables



The following describes the input tables shown in the previous illustration.

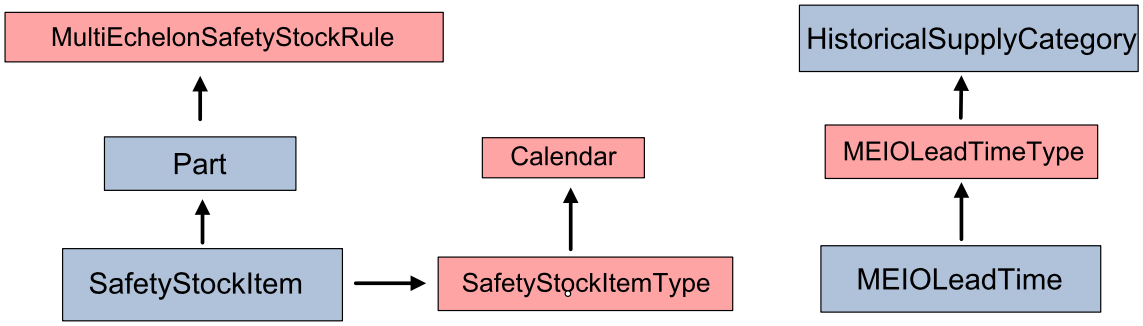
- **SafetyStockItem**—identifies the items (parts) for which safety stock recommendations should be generated and contains item-specific parameters that define how those calculations are performed (for example, the service level and service times that should be satisfied, and whether an item is subject to single or multi-echelon processing). For more information, see "SafetyStockItemType table" on page 861.
- **Part**—contains details of the specific part that each multi-echelon SafetyStockItem and SafetyStockTimePhasedBounds record is being defined for. For more information, see "Part table " on page 415.
- **BillOfMaterial**—identifies assembly-component relationships in a bill of material. Used in determining the component parts that can be brought into a multi-echelon family under a given end item. For more information, see "BillOfMaterial table" on page 217.
- **PartSource**—identifies sources of supply for a part. Used in building multi-echelon families and contains other details used as input when determining optimal safety stock levels such as unit cost and lead times. Also if referenced by a record in the **MEIOLeadTime** table, then lead time parameters for the **PartSource** can be calculated from historical supply data.For more information, see "PartSource table" on page 459.
- **SafetyStockTimePhasedBounds**—contains minimum and maximum safety stock bounds for parts and the dates on which these bounds start to apply. For more information, see "SafetyStockTimePhasedBounds table" on page 565.
- **SafetyStockRangeOfCoverage**—defines the range over which average demand is calculated for the purpose of calculating a part's recommended safety stock using the "DaysOfSupplyForward" or "DaysOfSupplyBackward" processing rule. For more information, see "SafetyStockRangeOfCoverage table" on page 565.
- **HistoricalDemandCategory**—identifies categories of historical demands, and is referenced from the SafetyStockItem table to determine the category of historical demands for the item that are used as input in making safety stock recommendations for an item. For more information, see "HistoricalDemandCategory table" on page 329.
- **HistoricalDemandActual**— contains the actual details of historical customer orders/shipments that are used as input in determining average and standard deviation of historical demands for use in determining the recommended safety stock levels for the item. The records used in this table are those that match an item's HistoricalDemandCategory reference within a defined collection interval. For more information, see "HistoricalDemandActual table" on page 325.
- **MEIOLeadTime**—used to enable lead time variability for parts involved in multi-echelon safety stock calculations (if not used for a given part, then fixed lead time based on values on the **PartSource** table is used). For more information, see "MEIOLeadTime table" on page 393.
- **HistoricalSupplyActual**—contains the actual details of historical supply orders, such as the date an order was placed and the date it was actually received. Used when variable supply lead time is assumed. For more information, see "HistoricalSupplyActual table" on page 347.
- **HistoricalSupplyCategory**—identifies categories of historical supplies, and is referenced by the **MEIOLeadTimeType** control table in order to indicate the category of historical supplies which are used in calculating historical supply lead time for a particular type. For more information, see "HistoricalSupplyCategory table" on page 350.
- **Source**—indicates a source of supply, and referenced by the **HistoricalSupplyActual** table to indicate which of a part's sources the lead time details on a historical supply record should apply to. For more information, see "Source table" on page 603.
- **MEIOLeadTimeDistributionProfile**—if lead time variability is assumed and lead times follow a discrete distribution, this table holds named profiles identifying those distribution profiles. For more information, see "MEIOLeadTimeDistributionProfile table" on page 396.

- **MEIOLeadTimeDistributionProfilePoint**—if lead time variability is assumed and lead times follow a discrete distribution, this table holds the discrete lead time values and probability weighting for use in those profiles. For more information, see "MEIOLeadTimeDistributionProfilePoint table" on page 397.
- **TimePhasedDemandParameterSet**—allows for manually specifying time-phased demand parameters (average and standard deviation of demand) in cases where there is no sufficient historical data available to calculate them (for non time-phased items, parameters can be manually specified on the SafetyStockItem record). For more information, see "TimePhasedDemandParameterSet table" on page 653.
- **SafetyStockItemMapping**—allows a safety stock item to be mapped to other parts and have their demand included in safety stock calculations for the item (for example, different revisions of a part might be mapped). For more information, see "SafetyStockItemMapping table" on page 554.
- **AnalyticsConfiguration**—contains scenario-specific values that specify certain settings applied to all multi-echelon item calculations in the scenario. For example, the amount of time the analytic is allowed to run in search of the best solution is specified here. For more information, see "AnalyticsConfiguration table" on page 674.

Control tables

A single control table defines the most common configuration parameters that can be used by multi-echelon safety stock items, while an additional part-level control is available to set if and how eligible parts are used in multi-echelon families. If lead time variability is enabled, then an additional control table is available to define how the required lead time parameters are calculated. Refer to the following illustration.

Figure 42.8 Multi-echelon control tables



The following describes the control tables shown in the previous illustration.

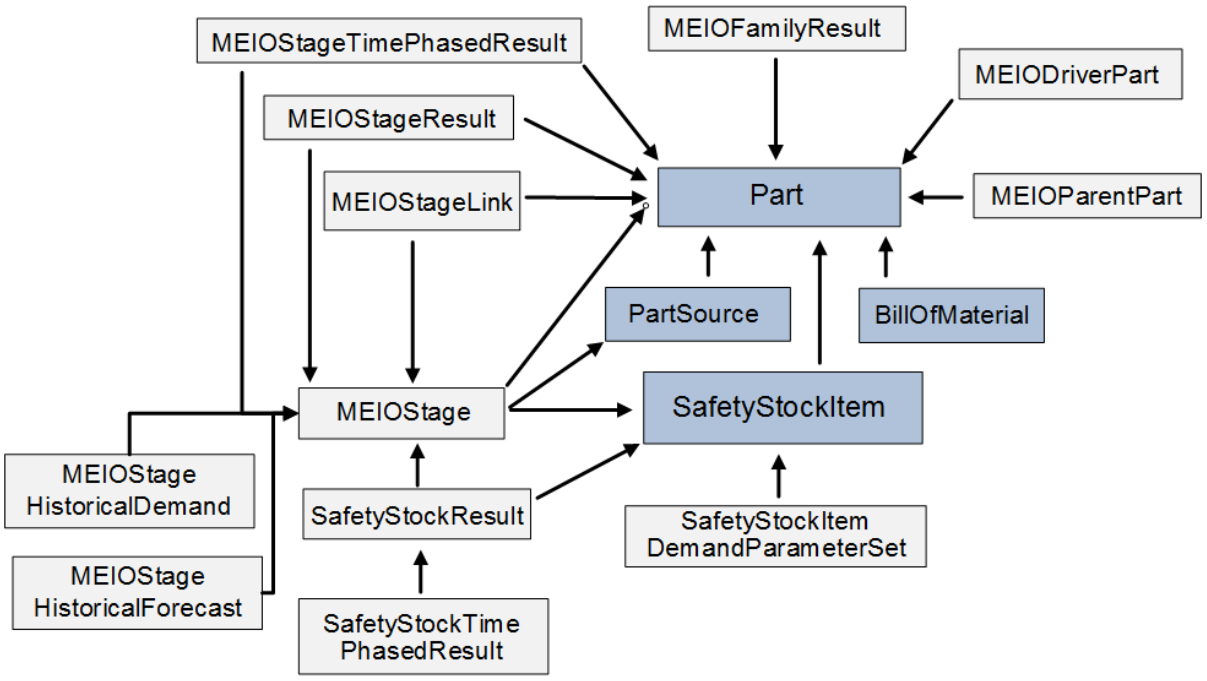
- **SafetyStockItemType**—contains control settings applied to safety stock items such as the calendars to use for safety stock calculations, the method used for determining standard deviation, and whether an item should generate stationary or time-phased safety stock results. For more information, see "SafetyStockItemType table" on page 861.
- **MultiEchelonSafetyStockRule**—provides settings that control whether individual parts are brought into multi-echelon families and how those parts are used with regards to safety stock calculations. For example, parts in multi-echelon families can be set as eligible or ineligible to carry safety stock. For more information, see "MultiEchelonSafetyStockRule table" on page 756.

- **MEIOLeadTimeType**—provides settings that control how average and standard deviation of lead time are calculated. For example, should a normal or discrete lead time distribution be assumed. The settings in this table are applicable to parts for lead time variability and should be assumed in multi-echelon safety stock calculations (that is, parts that have been configured in the **MEIOLeadTime** table). For more information, see "MEIOLeadTimeType table" on page 744.
- **Calendar**—contains the named calendars referenced by various fields on the SafetyStockItemType table. For example, the names of the inner and outer calendars used in time-phased calculations. For more information, see "Calendar table" on page 695.

Calculated tables

A number of calculated tables are used to hold both the safety stock recommendations generated by multi-echelon calculations, as well as details of the various calculated parameters and relationships used in those calculations.

Figure 42.9 Multi-echelon calculated tables



The following describes the calculated tables shown in the previous illustration.

- **MEIOStageTimePhasedResult**—reports the recommended safety stock level(s) for parts as generated by multi-echelon calculation. This can include recommendations both for the safety stock items themselves as well as the dependent or related parts brought into multi-echelon families. For more information, see "MEIOStageTimePhasedResult table" on page 1130.
- **MEIOStageResult**—reports demand parameters calculated for use in multi-echelon safety stock calculations (for example, average and standard deviation of demand) along with final optimized service times. For more information, see "MEIOStageResult table" on page 1128.

- **MEIOFamilyResult**—reports details associated with the generation of safety recommendations within a given family. For example, the total time to arrive at the final set of recommendations made across the family. For more information, see "MEIOFamilyResult table" on page 1117.
- **MEIOStage**—identifies each stage (typically, a part and/or a source of supply for the part) within a multi-echelon family along with source details such as cost and lead times. For more information, see "MEIOStage table" on page 1123.
- **MEIOStageLink**—identifies the links between all stages in a family, along with their quantity per values (from BOM relationships). For more information, see "MEIOStageLink table" on page 1128.
- **MEIOStageHistoricalDemand**—reports the propagated demand for each MEIO stage based on the historical demand of customer facing demand. For more information, see "MEIOStageHistoricalDemand table" on page 1127.
- **MEIOStageHistoricalForecast**—reports the propagated forecast for each MEIO stage based on the historical forecast of customer facing demand. For more information, see "MEIOStageHistoricalForecast table" on page 1127.
- **MEIODriverPart**—reports the end-item parts that drive demand to lower-level parts in multi-echelon families. For more information, see "MEIODriverPart table" on page 1116.
- **MEIOParentPart**—reports the immediate parent parts above each item in a multi-echelon family. For more information, see "MEIOParentPart table" on page 1123.
- **MEIOHistoricalSupply**—reports details of historical supplies on parts for which supply lead time variability is assumed, and which are used to calculate lead time parameters for use in multi-echelon safety stock calculations. For more information, see "MEIOHistoricalSupply table" on page 1121.
- **SafetyStockItemDemandParameterSet**—reports calculated demand parameters (average and standard deviation of historical demands) that are generated and used by safety stock calculations. For more information, see "SafetyStockItemDemandParameterSet table" on page 1164.
- **SafetyStockResult**—reports safety stock calculated from historical data along with related parameters applicable to both multi-echelon inventory optimization and single-echelon inventory planning calculations. For more information, see "SafetyStockResult table" on page 1169.
- **SafetyStockTimePhasedResult**—reports time-phased safety stock and reorder points recommendations from both multi-echelon inventory optimization and single-echelon inventory planning calculations. For more information, see "SafetyStockTimePhasedResult table" on page 1174.

Safety stock item configuration

This section outlines the steps that should be taken to set up and configure a part as a safety stock item.

1. "Define IP&O parameters" on page 1947
2. "Provide historical data for safety stock calculations" on page 1947
3. "Configure control settings for safety stock calculations" on page 1948
4. "Set safety stock bounds (optional)" on page 1959
5. "Configure multi-echelon lead time parameters (optional)" on page 1960
6. "Define safety stock item mappings (optional)" on page 1962
7. "Define single-echelon average demand profiles (optional)" on page 1962
8. "Use forecast error pooling for safety stock (optional)" on page 1963

Note: Some configuration steps are optional, while others apply only to single-echelon safety stock items or only to multi-echelon safety stock items.

Define IP&O parameters

Inventory Planning and Optimization parameters for the configuration of safety stock items are defined in the **IPOConfiguration** worksheet.

The **IPOConfiguration** worksheet is used primarily to define the default configuration of outlier processing, and of the multi-echelon safety stock rule that determines how parts brought into multi-echelon families are processed in safety stock calculations. **OutlierType** and **MultiEchelonSafetyStockRule** records must be created via the **Control Tables** workbook before adding them to this table.

The **IPOConfiguration** worksheet holds only one record. As each scenario includes a version of this record, you can use this table to model various outlier adjustment and multi-echelon family scenarios.

These records are typically populated when RapidResponse is deployed. These initial settings can subsequently be modified by data or system administrators in RapidResponse. If you are upgrading from RapidResponse 2016.2, Service Update 9 (or earlier), populate this record before using analytics to calculate safety stock recommendations.

To define default IP&O parameters in IPOConfiguration

1. In the **Explorer**, click the **Control Tables** workbook.
2. Click the **IPOConfiguration** worksheet.
3. Double-click in a table cell to edit a value.
4. Click **Save Data**.

Provide historical data for safety stock calculations

Safety stock recommendations for both single and multi-echelon item rely on calculated parameters such as average historical demands and standard deviation of historical demands. Therefore, historical demand data must be provided for any parts designated as safety stock items. This data should be imported into the **HistoricalDemandActual** table which identifies details of historical demands such as historical order date and quantity. Through this table's **Header** field, key references are made to the **HistoricalDemandCategory** and **PartCustomer** tables. The **HistoricalDemandCategory** is used to identify different categories of demands, such as shipments or customer orders. In order to be used in safety stock calculations for the part, care should be taken to ensure the required **HistoricalDemandActual** records reference the same category value as is referenced from the **SafetyStockItem** record for the part. The **PartCustomer** is used to define unique part and customer combinations in historical data and is typically used to support Sales and Operations Planning in RapidResponse; therefore, records might need to be added to this table as necessary.

Optionally, supply lead time variability can be considered in safety stock calculations. In this case, calculated details such as average lead time are used and historical supply data should be provided for these items. This supply data should be imported into the **HistoricalSupplyActual** table which identifies details of historical supplies such as dates and quantities of actual shipments. In order for lead time

variability to be calculated, ensure to populate the **OrderDate** field on this table; this field is used to specify a date associated with the ordering of an item and the difference between the **Date** and **OrderDate** fields is used to calculate actual lead time for the historical supply. Through this table's Header field, key references are made to the **PartSupplier** and **HistoricalSupplyCategory** and tables.

Also optionally, historical forecast error variability can be considered in safety stock calculations (this applies when the item's **Type.StandardDeviationDemandRule** field is set to "ForecastError"). In this case, detailed historical forecast demand data should be imported into the **HistoricalDemandSeriesDetail** table. These forecasts should belong to a historical demand category as determined by their **Series.Header.Category** reference, and this category should be the same one as referenced by the item's **SafetyStockItem.HistoricalForecastCategory** field.

The **PartSupplier** table is used to define unique part and supplier combinations in historical data and records might need to be added to this table as required. The **HistoricalSupplyCategory** is used to identify different categories of supplies, and in order to be used in safety stock calculations for the part, care should be taken to ensure the required **HistoricalSupplyActual** records reference the same supply category value as is referenced from the **SafetyStockItem** record for the part.

Note: If there is insufficient historical demand data and/or insufficient historical supply data, then fields on the **SafetyStockItem** table can be used to manually provide single stationary values for parameters such as standard deviation of demand and average lead time of supply. For items subject to time-phased safety stock calculations, the **TimePhasedDemandParameterSet** table can be used to enter a period based set of demand parameters.

Configure control settings for safety stock calculations

In order to support both single and multi-echelon safety stock items, one or more control records should be added to the **SafetyStockItemType** table. This table is referenced by the **SafetyStockItem** table.

Additionally, if multi-echelon safety stock items are being defined, one or more control records should be added to the **MultiEchelonSafetyStockRule** table. This table is referenced by the **Part** table.

SafetyStockItemType settings

The **SafetyStockItemType** table holds settings that define key configuration parameters that can be referenced/reused by multiple records in the **SafetyStockItem** table. For example, item details such as how standard deviation of historical demands should be calculated and how the standard deviation of historical demands should be calculated. Note that many, but not all, fields in the **SafetyStockItemType** table apply to both single and multi-echelon safety stock calculations.

The following table outlines some key configuration tasks that are done using the **SafetyStockItemType** table. For complete reference information about this table, see "SafetyStockItemType table" on page 861.

Note: Previously, outlier processing in historical actual and forecast demand data was controlled through the **DemandOutlierProcessingRule** and the **ForecastErrorOutlierProcessingRule** settings on the **SafetyStockItemType** table (these fields are now hidden and not recommended, though existing resources that use them will continue to function). As of Version 2016.2, Service Update 5, outlier processing is applicable to historical actual and forecast demands, as well as to historical lead times on supplies in the context of both single-echelon and multi-echelon stages. These options are configured through several new input fields on the **SafetyStockItem** and **MEIOLeadTime** tables.

Table 42.1: SafetyStockItemType settings

Task	Description
Turn on or off time-phased processing	Safety stock calculations can be stationary or time-phased. Stationary safety stock calculations generate single values for parameters such as average and standard deviation of demand, and use them to generate a single safety stock recommendation for each item (at the RunDate). This is appropriate for items with demand that does not typically change much over time or between seasons. Time-phased safety stock calculations generate period-based values for parameters such as average and standard deviation of demand, and use them to generate safety stock recommendations that vary over time. This is appropriate for items with demand that is subject to trend or seasonality. If enabling time-phased safety stock calculations, a number of special calendars and related settings also apply. RapidResponse also supports "days of supply" based safety stock calculations in which a single safety stock value is initially calculated and then converted to a number of days (or intervals) of the part's average demand. This number of intervals is then used as a horizon over which backward or forward looking demands are collected and used to calculate safety stock values that vary over time. Use the **TimePhasedProcessingRule** field to turn on or off time-phased safety stock methods. The value selected in this field also determines whether the **StandardDeviationDemandRule** or the **NonStationaryDemandRule** control setting is used in determining how the standard deviation of demand parameter is calculated. For more information, see "Specify rule for standard deviation calculation" on page 1952.

Table 42.1: SafetyStockItemType settings

Task	Description
Define applicable calendars	A number of calendars can be defined for each safety stock item type through (nullable) references to the **Calendar** table. The following fields define calendars used in safety stock calculations. • **IntervalsCalendar**—this is the main calendar used in safety stock calculations and applies whether calculations are stationary or time-phased. For example, historical demands are bucketed, and average demand calculations are expressed, in terms of this calendar. If this reference is Null, an everyday (Gregorian) calendar is used. • **LeadTimeCalendar**—an (optional) calendar used in lead time calculations for the safety stock item. Historical lead times are calculated in terms of this calendar, and input lead time on the **SafetyStockItem** record is specified in terms of this calendar. If this reference is Null, then lead time calculations are performed in terms of the **IntervalsCalendar** instead**.** In other words, this field should be populated if the lead time and intervals calendar are different (for example, if lead time is daily and intervals are weekly). • **CycleCalendar**—when time-phased safety stock calculations are enabled, this defines the outer-calendar used to define a full cycle of seasonal data. For example, this might typically be set to a Yearly calendar. • **PeriodCalendar**—when time-phased safety stock calculations are enabled, this defines the inner-calendar used to define a season or period within a full cycle of data. Historical demand parameters, such as standard deviation of demand, are then generated by period. For example, this might typically be set to a Month or Quarter calendar. This calendar is also applicable when statistical models are used to calculate standard deviation based on forecast error. If using certain calendars listed above, the following fields apply and are used to express relationships between those calendars: • **IntervalsPerPeriod**—number of **IntervalCalendar** units in the **PeriodCalendar**. Used to convert results between calendars when standard deviation of demand is calculated using statistical forecast models. • **LeadTimePerInterval—**number of **LeadTimeCalendar** units in the **IntervalsCalendar**. Used in converting simple standard deviation or forecast error from intervals to lead time units when calculating safety stock. • **PeriodsPerCycle**—number of **PeriodCalendar** units in the **CycleCalendar**. Used in defining seasonal cycles for time-phased safety stock. Also applicable if using HoltWinters method to estimate standard deviation of demand. **Note:** The **CycleCalendar** and **IntervalsCalendar** settings determine the value to use for **SafetyStockItem.HistoricalIntervalCount**. This is to ensure that an adequate amount of historical demand and supply data is available for use in safety stock calculations. See "Define historical collection interval" on page 1956.
Use forecast error pooling	When demand is stationary, the MEIO algorithm requires one average demand and one standard deviation of demand for each MEIO stage. However, the **SafetyStockItemDemandParameterSet** table contains average demand and standard deviation demand only for customer facing stages, as only they have a corresponding **SafetyStockItem**. For a non-customer facing stage, the standard way to calculate average demand and standard deviation is by propagating the values from the customer facing stage. Optionally, when forecast error pooling is enabled on the MEIO family, the average demand and standard deviation of demand for a non-customer facing stage is calculated based on the demands and the forecasts propagated from the customer facing stage. The propagated demands and forecasts are shown in **MEIOStageHistoricalDemand** and **MEIOStageHistoricalForecast**. The average demand and standard deviation are calculated based on these demands, and forecasts can only be seen in **MEIOStageResult**.

Table 42.1: SafetyStockItemType settings

Task	Description
Specify rule for applying safety stock bounds	A safety stock item can have minimum and maximum bounds defined in terms of absolute quantity or days of supply. Quantity and days of supply bounds cannot both apply at the same time. The **BoundsRule** field is used to specify whether quantity bounds, days of supply bounds, or no bounds are applied.
Enable/disable rolling lead time demand	Specifies whether bucketed demand quantities within the **Intervals** calendar (typically, this might be daily) or demands collected and bucketed within a part's lead time, are used when calculating average historical demand and standard deviation of demand. By default, bucketed demand quantities are used. The **UseRollingLeadTimeDemand** field controls whether bucketed demands or demands within lead time are used.
Specify rule for calculating average demand	Either the mean, median, or mode can be used as a measure of the average within a set of historical demands. By default, the mean is used. However, median or mode might be appropriate in cases of demand that is not normally distributed. The **AverageDemandRule** field is used to specify how average demand should be calculated. This field applies only to single-echelon calculations where **StandardDeviationDemandRule** is set to "StandardDeviation".
Specify rule for standard deviation calculation	In order to generate a safety stock recommendation, the average and standard deviation of historical demands needs to be calculated. The standard deviation gives a measure of the variance within those demands and hence is used as a parameter towards determining the level of safety stock that should be kept in order to guard against future variability. For most configurations, RapidResponse supports determining standard deviation through simple known formulas based on historical demand and/or historical forecast error, as well as using statistical methods such as exponential smoothing and Holt-Winters to estimate forecast error. For all non-time phased safety stock calculations (as well as those time-phased calculations based on days of supply), the **StandardDeviationDemandRule** field is used to select the measure of standard deviation. For standard time-phased safety stock calculations, the **NonStationaryDemandRule** field is used to select the measure of standard deviation. This field applies to both single and multi-echelon calculations. **Note:** These fields each include an option to use manually provide demand parameters instead of calculating them. If used, appropriate values should be provided in the **SafetyStockItem** or **TimePhasedDemandParameterSet** table. **Note:** The **StandardDeviationDemandRule** includes an option ("ForecastError") to calculate standard deviation based on the difference between historical forecast demands and historical actual demands. When this option is used, data should be provided in both a historical actual and a historical forecast category, and several other parameters apply as described in the next cell. For more detail on how standard deviation is calculated from historical forecast error, see "Example 3: Fixed lead time (with forecast error)" on page 1972.
Specify forecast error parameters	When the **StandardDeviationDemandRule** setting is "ForecastError", several additional settings on the control record are relevant. This includes the following fields: • **AsOfDateCalendar**—specifies the calendar by which historical forecasts are collected for use in calculating forecast error. Typically, this might be a monthly calendar. • **ForecastBiasAdjustmentRule**—specifies whether the standard deviation of forecast error is automatically adjusted to account for forecast bias (systematic over or under forecasting). This feature is not available with forecast error pooling.

Table 42.1: SafetyStockItemType settings

Task	Description
Specify service level rule	For single-echelon items only, specifies how the service level specified on the safety stock item is interpreted (and hence how the z-value or service factor used in safety stock calculations is determined). Service level rules define the percentage-based measure of customer satisfaction that the safety stock policy is meant to satisfy. Both alpha/type1 (probability of not stocking out) and beta/type (percent of overall demand that should be satisfied on time) service levels are supported. The **ServiceLevelRule** field is used to specify how the service level is interpreted. Note that multi-echelon items always use an alpha/type1 service level.
Turn on or off supply variability	For single-echelon items, safety stock calculations can either use a constant input lead time as provided on the **SafetyStockItem** record, or variable lead times can be used in which case average and standard deviation of lead time for a part is calculated (based on the differences between order and ship dates in the provided historical supply data). The **SupplyVariabilityRule** field is then used to control how lead time is determined and used in single-echelon safety stock calculations. Note that fixed lead times are always used in time-phased safety stock calculations regardless of this setting. **Note:** For multi-echelon items and parts within multi-echelon families, either a fixed safety stock value as calculated based on the applicable **PartSource** record(s) is used or, if variable lead time should be modeled, a series of tables are used to configure how lead time parameters are calculated as discussed in "Configure multi-echelon lead time parameters (optional)" on page 1960.

MultiEchelonSafetyStockRule settings

The **MultiEchelonSafetyStockRule** table holds part-level settings that define how a given part can be used in multi-echelon safety stock calculations. For example, this allows for specific parts or groups of parts to be brought into, or excluded from, the multi-echelon families used to generate safety stock recommendations.

If an item is configured for multi-echelon safety stock calculations, related parts that should be brought into its family should have their **MultiEchelonSafetyStockRule** reference set appropriately (this includes the part referenced on the **SafetyStockItem** record itself).

The following table outlines some key configuration tasks that are done using the **MultiEchelonSafetyStockRule** table. For complete reference information about this table, see "MultiEchelonSafetyStockRule table" on page 756.

Table 42.2: MultiEchelonSafetyStockRule settings

Task	Description
Specify part eligibility	Parts can be included or excluded from all multi-echelon families to which they might potentially belong. For example, safety stock recommendations might be enabled or disabled for particular levels or groups of parts at a time. Parts can also be brought into multi-echelon families, and used to generate parameters required for safety stock calculations, but disabled from holding inventory themselves (in other words a part and its demand parameters can contribute to the safety stock recommendations made across the network, but no safety stock recommended for that part itself). Use the **ProcessingRule** field to set how parts can be used in multi-echelon safety stock calculations.
Set cost rule	Inventory holding costs for parts in multi-echelon families are required in order to determine the optimal strategic placement of safety stock. For a given part, these costs can be calculated based on part source attributes (labor, material, and overhead costs) including rolled-up values from lower components, or based on the part's standard unit cost. Use the **CostRule** field to set how inventory holding costs are calculated. The **Part.InventoryHoldingRate** field also applies and is used to set the marginal rate of holding inventory (by default, this field is set to 0.20 for all parts).

Configure safety stock items

In order for single or multi-echelon safety stock recommendations to be generated for a given part by the underlying functions or analytic, the part must be set up as a safety stock item with the applicable configuration parameters set as required. Parts are configured for safety stock recommendations using the **SafetyStockItem** table.

When generating the safety stock recommendations, the presence of outliers in historical data can limit the ability to generate a reasonably accurate recommendation. In order to identify and process outliers in the historical data, several configuration options are available in RapidResponse. These are mostly set through the **OutlierType** control table, as well as supporting fields in the **SafetyStockItem** table.

The following outlines the basic configuration steps needed to define a new item in this table. Note that certain fields apply only to single-echelon calculations and others apply only to multi-echelon calculations. For complete reference information about this table, see "SafetyStockItem table" on page 541.

Table 42.3: Defining a SafetyStockItem record

Tasks	Description
Select the part	The **Part** reference serves as the **SafetyStockItem** table's primary key and is used to specify the specific item or part for which safety stock recommendations should be generated based on the parameters defined on the record.
Select a safety stock item type	The **Type** reference is used to specify the record in the **SafetyStockItemType** control table record that determines generic processing rules for the item's safety stock calculations such as whether time-phased or stationary safety stock values should be generated, and how standard deviation of historical demands should be calculated.

Table 42.3: Defining a SafetyStockItem record

Tasks	Description
Choose single or multi-echelon processing	Each safety stock item can be configured for either single or multi-echelon safety stock recommendations using the **ProcessingRule** field. A single-echelon item will generate safety stock levels for just the part itself. These recommendations are intended to satisfy the item's service level and are based on the item's historical demand variability and, optionally, historical supply variability (otherwise fixed lead times are used). Historical and future looking reorder points are also calculated. A multi-echelon item can potentially generate safety stock levels for both the part itself as well as other related parts brought into its family through BillOfMaterial and PartSource relationships. The safety stock is placed across the network of parts in a manner that minimizes inventory holding cost, while satisfying the end item's service level and service time. These recommendations use historical demand parameters (typically, calculated based on variability of the safety stock item's historical demands and propagated to lower-level items in the family), and optionally, historical supply variability (otherwise fixed lead times are used). **Note:** If a part is configured as a single-echelon safety stock item, but it is subsequently brought into a multi-echelon family under another item, then its safety stock value as recommended by multi-echelon calculations is the one reported in the **SafetyStockOverride** table.
Specify service level	An intended customer service level percentage should be specified for each item in the **ServiceLevel** field, and safety stock calculations are then made with the intention of achieving that service level. Typically, to ensure meaningful results, service levels between .50 and .99 should be provided. Note that specific interpretation of the service level value depends on the **SafetyStockItemType**.**ServiceLevelRule** setting as follows: • If set to "Cycle", then the value specifies the probability of not stocking out. • If set to "FillRate", then the value specifies the overall percent of demand that should be satisfied on time, and a value should also be provided in the **OrderQuantity** field (used to determine the service factor).
Specify service time	For multi-echelon items only, a service time value should be specified in the **ServiceTime** field. It represents the maximum number of days allowable between when a customer orders the item and when the item is shipped. This value is then used as input in multi-echelon safety stock calculations, with service times for all other parts in the network determined as part of the optimization calculation.

Table 42.3: Defining a SafetyStockItem record

Tasks	Description
Select historical categories	Each safety stock item should reference the specific **HistoricalDemandCategory** value associated with the historical demand data that should be used in determining required parameters such as average historical demand and standard deviation of those demands. Only those demands from the **HistoricalDemandActual** table that reference the same category (through the **Header** reference) are used towards generating safety stock levels for the item. Optionally, for single-echelon items only, additional historical categories need to be specified under the following conditions: • If **SafetyStockItemType.SupplyVariabilityRule** is set to "Use" (meaning supply lead time variability is considered during safety stock calculations), then the item's **HistoricalSupplyCategory** field should reference the category associated with the historical supplies to be used in determining the item's average lead time. Only those supplies from the **HistoricalSupplyActual** table that reference the same category (through the **Header** reference) are then used in calculations generating safety stock levels for the item. • If **SafetyStockItemType.StandardDeviationDemandRule** is set to "ForecastError" (meaning standard deviation is calculated from forecast error), then the item's **HistoricalForecastCategory** field should reference the category associated with the historical forecast demands to be compared against historical actuals for the purpose of calculating forecast error. Only those forecast demands from the **HistoricalDemandSeriesDetail** table that reference the same category through their **Series.Header.Category** reference are collected for use in calculating forecast error. When calculating standard deviation of forecast error, several additional fields apply as discussed in "Specify forecast error parameters" on page 1958.
Define historical collection interval	The period over which historical demand data (and optionally historical supply data) is collected for use in determining safety stock parameters can be specified on each safety stock item as follows: • If a value is provided in the **HistoricalIntervalCount** field, then this specifies the number of **Type.IntervalsCalendar** periods back from the RunDate over which to collect historical data. • Otherwise, if explicit date values are provided in the **HistoricalStartDate** and **HistoricalEndDate** fields then these set the beginning and end of the historical data collection period. • Otherwise, all available history is collected for the item. **Note:** If **HistoricalSkipLeadingZeros** is set to "Y" and one or more buckets at the beginning of the period for which historical demand is collected have a zero demand or forecast quantity, these buckets are not considered when calculating safety stock. For more information, see "HistoricalSkipLeadingZeros" on page 547. **Note:** When specifying a **SafetyStockItem.HistoricalIntervalCount** value for a safety stock item, it needs to adequately reflect both the **SafetyStockItemType.IntervalsCalendar** and **SafetyStockItemType.CycleCalendar** that are selected for that item. This is to ensure that an adequate amount of historical demand and supply data is available for use in safety stock calculations. For example, if **IntervalsCalendar** is set to "monthly" and **CycleCalendar** is "yearly", you would need a minimum **SafetyStockItem.HistoricalIntervalCount** of "12" to provide adequate history.
Specify outlier processing	Any special processing of outliers that might be found in historical demand data should be specified. For example, outlier quantities might be smoothed over a number of intervals or removed entirely from the data. By default, outliers are adjusted according to the threshold rules on **OutlierType** (adjustments for any causal adjustments are always made before, and regardless of, any applicable outlier processing). **Note:** As of Version 2016.2, Service Update 5, the **Type.DemandOutlierProcessingRule** is replaced by the **DemandOutlierType**. With this change, outliers found in historical data are no longer removed by default, instead outliers are adjusted according to the threshold rules on **OutlierType**.

Table 42.3: Defining a SafetyStockItem record

Tasks	Description
Specify parameters for outlier processing	Together with **RemoveDemandOutliers**, each item allows further control over how outliers in its data are processed. The following fields are used: • **DemandOutlierType**—references the **OutlierType** that defines how to process outliers in historical demand or historical forecast error. • **DemandOutlierThreshold**—number of intervals away from the mean a demand quantity must be in order to be considered an outlier. The threshold value is interpreted depending on the specified **OutlierType.DetectionRule**. If **SafetyStockItemType.SupplyVariabilityRule** is set to "Use", the following fields are used: • **LeadTimeOutlierType**—references the **OutlierType** that defines how to process outliers in historical lead time. • **LeadTimeOutlierThreshold**—number of intervals away from the mean a demand quantity must be in order to be considered an outlier. The threshold value is interpreted depending on the specified **OutlierType.DetectionRule**. For complete information about the settings on the **OutlierType** table, see "OutlierType table" on page 792.
Provide average lead time value	For single-echelon items, an average or fixed lead time value should be specified in the **LeadTime** field (expressed in terms of **Type.IntervalsCalendar** units). This value is used in safety stock calculations for items that do not use supply variability. **Note:** For multi-echelon items that do not use supply variability, a fixed lead time value is calculated based on values from the applicable **PartSource** record(s).
Define future collection interval	For single-echelon items only, the number of **Type.IntervalsCalendar** periods after the run date over which current/forecast demand data should be collected to determine future average demands for use in reorder point calculations. If no value is specified, all future demand data is used. The **FutureIntervalCount** field is required when defining future collection interval. **Note:** The **FutureIntervalCount** field applies to single-echelon safety stock calculations and is applicable if the **Type.SupplyVariabilityRule**= "Ignore" or "Manual" or if no historical supply data is available.

Table 42.3: Defining a SafetyStockItem record

Tasks	Description
Specify forecast error parameters	If **Type.StandardDeviationDemandRule** is set to "ForecastError", the following fields are used to specify certain aspects of historical forecast error calculations: • **ForecastLag**—number of **Type.AsOfDateCalendar** intervals to go back and look at historical forecasts for comparison against the historical actual demands from a given period. For example, should a given period's actuals be compared against the forecast generated the previous month or the forecast generated two months ago. • **DemandOutlierThreshold**—number of standard deviations away from the mean that a forecast error point must be in order to be considered an outlier. Outliers are then processed according to the **RemoveDemandOutliers** field. **Note:** As of Version 2016.2, Service Update 5, **ForecastErrorOutlierThreshold** is replaced by the **DemandOutlierThreshold** field. With this change, outliers found in historical forecast error are adjusted according to the threshold rules on **OutlierType**.
Provide manual demand and/or supply parameters (for cases of limited historical data)	If **Type.StandardDeviationDemandRule** is set to "Manual" the following fields can be used to manually input demand parameters: • AverageDemand • StandardDeviationDemand If **Type.SupplyVariabilityRule** is set to "Manual" the following fields can be used to manually input supply parameters: • LeadTime • StandardDeviationLeadTime **Note:** For items configured for time-phased safety stock, manual demand parameters can be entered in the **TimePhasedDemandParameterSet** table instead.

Note: Values in the **SafetyStockItem** table can be modified using the **Item Configuration** worksheet of the **Inventory Planning** workbook included with RapidResponse.

Set global multi-echelon parameters

Certain parameters affect how long multi-echelon safety stock calculations can potentially run in search of the optimal solution for the placement of safety stock across the network. These parameters apply globally within each scenario in which they are defined and are set using the following fields in the **AnalyticsConfiguration** table:

- **MEIOMaximumIterations**—the maximum number of iterations that the analytic can run in search of the best or most optimal solution (the analytic will stop if the best solution is found before the maximum iterations is reached).
- **MEIOMaximumTime**—the maximum amount of time (minutes) that the MEIO analytic can run in search of the best or most optimal solution (the analytic will stop if the best solution is found before the maximum time is reached).
- **MEIOReportCalendar**—for time-phased safety stock calculations, defines the calendar on whose dates safety stock results can be generated. This reference is nullable and an everyday calendar is assumed if it is not populated; however, setting it to a calendar with fewer dates (for example, Month) can potentially reduce analytic processing time.

Notes:

- Multi-echelon safety stock results will not be generated in a scenario if either **MEIOMaximumIterations** or **MEIOMaximumTime** is set to zero (0).
- Values in the **AnalyticsConfiguration** table can be edited in the **Inventory Planning Settings** worksheet of the **Inventory Planning** workbook included with RapidResponse.

Set safety stock bounds (optional)

Any part can have maximum and minimum safety stock bounds specified for it in the **SafetyStockTimePhasedBounds** table. When safety stock bounds are in effect, RapidResponse does not recommend safety stock levels outside of the bounds specified for the part. The **Time-Phased Safety Stock Bounds** worksheet in the **Inventory Planning** workbook can be used to edit safety stock bounds.

Why place bounds on safety stock?

There are often practical limits on safety stock levels. For example, a site might have limited storage capacity or a company might have a contractual obligation to maintain a minimum level of safety stock for a part. The safety stock solution that would be ideal if these limits did not exist might not be reasonable in the real world.

Although it is possible to ignore or override any safety stock recommendations that fall outside of a practical range (see "Automated safety stock updates" on page 2012), there can be advantages to using safety stock bounds to get safety stock recommendations that fall within a practical range to begin with:

- For multi-echelon inventory optimization, applying inventory bounds can increase the likelihood of getting a safety stock recommendation that both satisfies the service level defined for the item and falls within an acceptable range.
- When safety stock bounds are used, the **CalculatedServiceLevel** field in the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables provides estimates of the service levels that can be achieved without violating safety stock bounds. Significant impacts on service levels resulting from bounds can be identified by comparing the calculated service levels to the service levels specified in the safety stock item configurations.
- Safety stock bounds persist until they are changed, whereas manual overrides applied using the **Inventory Planning** workbook prior to automated safety stock updates must be retyped each time safety stock is calculated.

Determining whether Days of Supply or Quantity bounds will be used

You can specify inventory bounds in terms of days of supply or quantity. While it is possible to include both types of bounds in the same record in the **SafetyStockTimePhasedBounds** table, both sets of bounds cannot be applied at the same time. Which bounds, if any, are used depends on two settings in the **SafetyStockItemType** table: **BoundsRule** and **TimePhasedProcessingRule**.

Table 42.4: Effect of SafetyStockItemType settings on type of safety stock bounds used

Bounds Rule	TimePhasedProcessingRule			
	DaysOfSupply Backward	DaysOfSupply Forward	Ignore	Use
DaysOfSupply	DaysOfSupply	DaysOfSupply	DaysOfSupply	none
Quantity	Quantity	Quantity	Quantity	Quantity
Ignore	none	none	none	none

For multi-echelon families, the **TimePhasedProcessingRule** and **BoundsRule** that apply to the head part are applied to all parts in that family.

Specifying when safety stock bounds start to apply

A part can have multiple safety stock bounds records that become effective on different dates. The **Date** field in the **SafetyStockTimePhasedBounds** table specifies the date on which the safety stock bounds in that record start to apply.

For time-phased safety stock items, different bounds rules can apply to different intervals.

- If one or more records have dates that fall within an interval, the one with the latest date is used.
- If no records have dates that fall within an interval, the previous interval's bounds, if any, apply. If it is the first interval, and there is a bounds rule with a date prior to the beginning of that interval, that record's bounds apply, but otherwise, no bounds are applied.

For safety stock items that are not time-phased, only the rule that applies on the RunDate is used.

Configure multi-echelon lead time parameters (optional)

Each part in a multi-echelon family requires a lead time value for use in determining safety stock.

By default, a single fixed lead time is assumed for each item in a multi-echelon family and then used in calculating safety stock as described in "Safety stock for parts with fixed lead time" on page 2005. Fixed lead time values are calculated using values on the applicable **PartSource** record as follows:

```
PartSource.LeadTimeDate - RunDate Everyday
```

Optionally, one or more parts in a multi-echelon family can be configured as having a variable or random lead time distribution. In this case, parameters representing the average and standard deviation of lead time are required for use in safety stock calculations as described in "Safety stock for parts with lead time variability" on page 2008. To enable lead time variability for parts in multi-echelon families, the **MEIOLeadTime** and **MEIOLeadTimeType** tables are used. One record should be added to the **MEIOLeadTime** table for each part and source combination for which variable supply lead time should be considered in multi-echelon calculations. This table subsequently references **MEIOLeadTimeType** which is a control table containing the processing rules that specifies how the average and standard deviation parameters are determined.

RapidResponse supports calculating lead time parameters under the assumption of either normal or discrete lead time distributions. If normal lead time distributions are assumed, parameters can be either calculated from historical data or provided as input. Otherwise, if discrete lead time distributions are

assumed, the parameters are calculated based on input profiles that must be provided. The following table provides an overview of the basic steps for configuring lead time variability for a part and source under each of these three assumptions.

Table 42.5: Enabling variable lead time in multi-echelon safety stock calculations

Lead time distribution	Configuration steps
Normal (using input values)	• **Define MEIOLeadTimeType settings**—A record must exist in this table with **ProbabilityDistribution** set to "Normal" and **StandardDeviationRule** set to either "Manual" or "CoefficientOfVariation". • **Create MEIOLeadTime record**—A record should be added to this table referencing both the appropriate **PartSource,** as well as an **MEIOLeadTimeType** record containing the settings outlined above. • **Provide lead time parameters**—Input lead time parameters should be specified in the **Average** and **StandardDeviation** fields on the **MEIOLeadTime** record. **Average** should always hold a value representing the average supply lead time.**StandardDeviation** field should then hold either the standard deviation of supply lead time if the **Type.StandardDeviationRule** is set to "Calculate" or its coefficient of variation if the **Type.StandardDeviationRule** is set to "CoefficientOfVariation".
Normal (calculated from history)	• **Provide historical supply data**—historical supply data for the part should be imported to the **HistoricalSupplyActual** table. Both the **Date** and **OrderDate** on each historical supply record should be defined to allow the calculation of actual supply lead time (otherwise, a non-negative value should be specified in the **LeadTime** field). Also, the **Source** reference on each historical supply record should be populated (data on the record is only used in calculating lead time parameters for parts whose **MEIOLeadTime** records match this source value through their **PartSource** reference). • **Define MEIOLeadTimeType settings**—a record must exist in this table with **ProbabilityDistribution** set to "Normal" and **StandardDeviationRule** set to "Calculate". Additionally, the **Category** field on this record should reference the same historical category as is specified on the **HistoricalSupplyActual** records to be considered in determining lead time parameters for this type. • **Create MEIOLeadTime record**—a record should be added to this table referencing both the appropriate **PartSource** (historical supply actual records for the same part and source are used in calculating lead time parameters), as well an **MEIOLeadTimeType** record containing the settings outlined above.
Discrete (using a profile)	• **Provide distribution profile**—a named profile should be added to the **MEIOLeadTimeDistributionProfile** table with its discrete lead time points specified in referencing records in the **MEIOLeadTimeDistributionProfilePoint** table. Each point in this table consists of a lead time value (or multiplier) and a weight. The probability of a given lead time value is then calculated by dividing its weight across the sum of all weights in the profile. • **Define MEIOLeadTimeType settings**—a record must exist in this table with **ProbabilityDistribution** set to "Discrete", and the **DistributionProfileRule** field should be set to either "LeadTime" if profile points contain actual lead time values or "Multiplier" if profile points contain multipliers to be used with average lead time values in determining the lead time points. • **Create MEIOLeadTime record**—a record should be added to this table referencing both the appropriate **PartSource,** as well as an **MEIOLeadTimeType** record containing the settings outlined in the previous step. As well, the **DistributionProfile** field should reference the **MEIOLeadTimeDistributionProfile** record associated with the appropriate lead time distribution values.

Define safety stock item mappings (optional)

Sometimes historical demand data, historical supply data, and/or future demand data for one part might need to be used as input to the safety stock calculations being performed for another part. For example, a part might use historical data from a different part revision or another similar part when it does not have sufficient data to produce reliable results.

To map data between parts, the **SafetyStockItemMapping** table is used. This table references both a safety stock item (**SafetyStockItem** field) and another part (**SourcePart** field) whose data should be used in safety stock calculations for the item. Data from the source part is then used as input towards safety stock calculations for the safety stock item as if it were data for the item itself based on the following three field settings on this table:

- **HistoricalDemandCategory**—if this reference is set, the source part's historical demand data from the selected category is used as input to the item's calculations.
- **HistoricalSupplyCategory**—if this reference is set, the source part's historical supply data from the selected category is used as input to the item's calculations. Applicable to single-echelon items only.
- **IncludeFutureDemand**—if set to "Y", then future/forecast demands from the source part are used as input to the item's calculations. Applicable to single-echelon items only.

Note: If a given safety stock item requires its own data be taken from multiple demand and/or supply categories, then the **SafetyStockItemMapping** table can be used to map the item to itself with the additional categories specified on the mapping record (both the **SafetyStockItem** and **SourcePart** references should resolve to the same part in this case).

Define single-echelon average demand profiles (optional)

For single-echelon safety stock items, RapidResponse generates two suggested reorder points based on both historical average and future average demands for each safety stock item. By default, these calculations use the collection intervals defined on the item (or all historical demands and all future demands if no interval ranges are specified).

Optionally, the **SafetyStockAverageDemandProfile** table can be used to create profiles that define the specific horizon over which the average demand values used to determine reorder points (only) are calculated. These profiles are based on the following three fields in the table:

- **Offset**—a fixed amount of time used to offset the start/end of the horizon over which average demands are calculated.
- **Multiplier**—a multiplier applied to the item's average lead time which is then used to determine the basic size of the horizon over which average demands are calculated. For example, if lead time is 3 days and this value is set to 10, then demands would be collected over a horizon of 30 days (subject to additional time from Extend as below).
- **Extend**—a fixed amount of time to add to or extend the horizon over which average demands are calculated.

Note that the **SafetyStockAverageDemandProfile** is referenced by two separate fields on the **SafetyStockItem** table which are used to reference profiles for average historical demands and average future demands respectively.

If a profile is referenced from the item's **HistoricalAverageDemandProfile** field**,** then the start and end dates of the profile are determined as follows (where **LastDate** refers to the latest date in the safety stock item's historical collection interval; typically this might be **RunDate**):

- **Start:**`LastDate - Offset - AverageLeadTime*Multiplier - Extend`
- **End:**`LastDate - Offset`

Similarly, if a profile is referenced from the item's **FutureAverageDemandProfile** field, then the start and end dates of the profile for future demand are determined as follows:

- **Start**: `RunDate + Offset`
- **End:**`RunDate + Offset + AverageLeadTime*Multiplier + Extend`

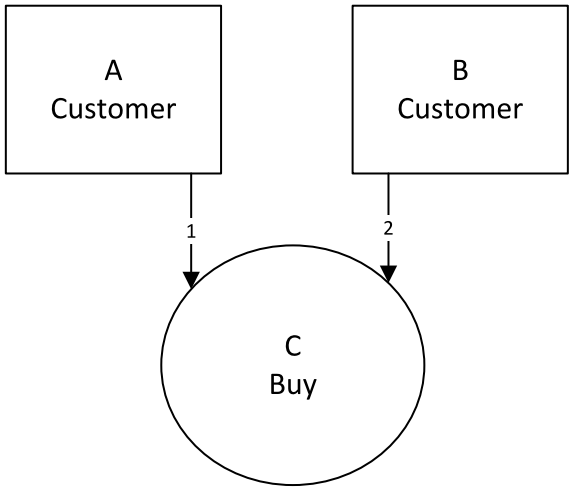
Use forecast error pooling for safety stock (optional)

To run MEIO with the StandardDeviationDemandRule = *ForecastError* setting, standard deviation of forecast error needs to be calculated for every MEIO stage. When *not* using forecast error pooling, computing standard deviation of forecast error is a two-step process. In step one, the standard deviation of forecast error for all customer-facing stages is calculated. In step two, the standard deviation for non-customer facing stages is calculated by propagating the standard deviations from step one.

When using forecast error pooling, on the other hand, standard deviation is calculated a different way at non-customer facing stages. Instead of propagating *standard deviations* from customer-facing stages, the actual *forecast errors* are propagated to non-customer facing stages. Each non-customer facing stage's standard deviation of forecast error is then calculated directly from its forecast errors.

As shown in the following figure, assume that forecast error pooling is enabled for both A and B. The numbers '1' and '2' on the arrows represent the **BillOfMaterial.QuantityPer** for the customer facing stages.

Figure 42.10 Simple MEIO Network diagram



The demands and forecasts are shown in the following table.

Table 42.6: Demands and forecasts

Stage	Bucket (month)	Demand	Forecast
A	Jan	410	470
	Febr	350	320
	March	560	680
	April	290	260
B	Jan	700	600
	Febr	810	952
	March	640	560
	April	690	780

In this example, when forecast error pooling is not enabled, RapidResponse computes the following values:

- For A:
  - Average demand = $\frac{(470+320+680+260)}{4} = 435.5$
  - Standard Deviation of forecast error = $\sqrt{\frac{(470-410)^2+(350-320)^2+(560-680)^2+(290-260)^2}{(4-1)}} = 81.2$
- For B:
  - Average demand = $\frac{(600+952+560+780)}{4} = 723$
  - Standard Deviation of forecast error = $\sqrt{\frac{(700-600)^2+(810-952)^2+(640-560)^2+(690-780)^2}{(4-1)}} = 122.0$
- For C:
- Average demand = $1 \times 435.5 + 2 \times 723 = 1878.5$
  = (BillOfMaterial.QuantityPer of A * Average of A) + (BillOfMaterial.QuantityPer of B * Average of B)
- Standard Deviation of forecast error = $\sqrt{(1 \times 81.2)^2 + (2 \times 122)^2} = 257.2$
  = √((BillOfMaterial.QuantityPer of A * SD of A)^2 + (BillOfMaterial.QuantityPer of B * SD of B)^2)

When forecast error pooling is enabled, average demand and standard deviation of forecast error for C are calculated by first propagating the demands and forecasts for C from stages A and B.

Table 42.7: Demand and forecasts with forecast error pooling enabled

Stage	Bucket (month)	Demand Formula	Demand Result	Forecast Formula	Forecast Result
C	Jan	410+2*700	1810	470 + 2*600	1670
	Febr	350+2*810	1970	320 + 2*952	2224
	March	560+2*640	1840	680 + 2*560	1800
	April	290+2*690	1670	260 + 2*780	1820

Based on these values:

- Average demand C = $\frac{(1670+2224+1800+1820)}{4} = \frac{3757}{2} = 1878.4$
- Standard Deviation of forecast error C = $\sqrt{\frac{(1810-1670)^2+(1970-2224)^2+(1840-1800)^2+(1670-1820)^2}{3}} = 189.92$

This is a family level feature. To turn on this feature, the **SafetyStockItemType** used for the head part of the family must have the following settings:

- **TimePhasedProcessingRule** = **Ignore** or **DaysOfSupplyBackward** or **DaysOfSupplyForward**
- **StandardDeviationDemandRule** = **ForecastError**
- **MEIODemandHistoricalsPropagationRule** = Use

The same bucket size is required for all customer facing stages. Regardless of what was set on their respective **SafetyStockItem**, each customer facing stage uses the interval specified on MEIO family head **Part.SafetyStockItem.IntervalsCalendar** for bucketing.

All other settings (that is, **HistoricalStartDate, HistoricalEndDate,** and **HistoricalIntervalCount**) can differ between safety stock items. We therefore have conventions for dealing with missing data.

In the following table, the first example is changed to assume that data was only collected data for stage B starting in February.

Table 42.8: Demand and forecast with overlapping intervals

Stage	Bucket (month)	Demand	Forecast
A	Jan	410	470
	Febr	350	320
	March	560	680
	April	290	260
B	Jan	NA	NA
	Febr	810	952
	March	640	560
	April	690	780

In this case, the results for stage C in the following table are calculated assuming that demand and forecast for stage B for January are 0 and 0, respectively.

Table 42.9: Demand and forecasts with overlapping intervals with forecast error pooling enabled

Stage	Bucket (month)	Demand Formula	Demand Result	Forecast Formula	Forecast Result
C	Jan	410+2*0	410	470 + 2*0	470
	Febr	350+2*810	1970	320 + 2*952	2224
	March	560+2*640	1840	680 + 2*560	1800
	April	290+2*690	1670	260 + 2*780	1820

Based on these values:

- Average demand C = $\frac{(470+2224+1800+1820)}{4} = \frac{3157}{2} = 1578.5$
- Standard Deviation of forecast error C = $\sqrt{\frac{(410-470)^2+(1970-2224)^2+(1840-1800)^2+(1670-1820)^2}{3}} = 175.32$

In the following table, the first example above is changed to assume that data has been collected for stage A from January to April and for stage B from August to November. In this case, stage A has a historical end date that is set in April, which is earlier than the historical end date for stage B. However, if the historical end date for stage A was set to be the same as stage B, there would be buckets for the missing months.

Table 42.10: Demand and forecast with non-overlapping intervals

Stage	Bucket (month)	Demand	Forecast
A	Jan	410	470
	Febr	350	320
	March	560	680
	April	290	260
B	Aug	700	600
	Sept	810	952
	Oct	640	560
	Nov	690	780

In this case, results for stage C are calculated assuming that missing demands and forecasts for stage A and stage B are all 0. In other words, it would be the same as starting with the following table.

Table 42.11: Demand and forecast with non-overlapping intervals and missing values filled in

Bucket (month)	Demand A	Forecast A	Demand B	Forecast B
Jan	410	470	0	0
Febr	350	320	0	0
March	560	680	0	0

Table 42.11: Demand and forecast with non-overlapping intervals and missing values filled in

Bucket (month)	Demand A	Forecast A	Demand B	Forecast B
April	290	260	0	0
Aug	0	0	700	600
Sept	0	0	810	952
Oct	0	0	640	560
Nov	0	0	690	780

Results for stage C are calculated with non-overlapping intervals and missing values filled in as shown in the following table.

Table 42.12: Demand and forecast with non-overlapping intervals and missing values filled in with forecast error pooling enabled

Stage	Bucket (month)	Demand Formula	Demand Result	Forecast Formula	Forecast Result
C	Jan	410+2*0	410	470 + 2*0	470
	Febr	350+2*0	350	320 + 2*0	320
	March	560+2*0	560	680 + 2*0	680
	April	290+2*0	290	260 + 2*0	260
	Aug	0+2*700	1400	0+2*600	1200
	Sept	0+2*810	1620	0+2*952	1904
	Oct	0+2*640	1280	0+2*560	1120
	Nov	0+2*690	1380	0+2*780	1560

Based on these values:

Average demand C = $\frac{(470+320+680+260+1200+1904+1120+1560)}{8} = \frac{3757}{4} = 939.25$

Standard Deviation of forecast error C = $\sqrt{\frac{(410-470)^2+(350-320)^2+(560-680)^2+(290-260)^2+(1400-1200)^2+\left(1620-1904\right)^2+\left(1280-1120\right)^2+\left(1380-1560\right)^2}{6}} = 181.87$

When forecast error pooling is enabled, the forecast bias adjustment feature is not available for any stage. For more information about the forecast bias adjustment feature, see "SafetyStockItemType table" on page 861.

When forecast error pooling is enabled, the forecast error outliers feature is available for the customer facing stages. The corrected data, either by removing a forecast or adjusting it, is then propagated to non-customer facing stages. For more information about the forecast error outliers feature, see "SafetyStockItemType table" on page 861.

Single-echelon safety stock calculations

Single echelon safety stock calculations rely on historical actual demand data for a given **SafetyStockItem** (part). Data from a specified historical demand category, and within the range of dates in the historical collection interval, is collected and bucketed by the item's **Intervals** calendar for use in the safety stock calculation. This data is automatically adjusted based on any causal factors defined in the **CausalFactorDetail** table and, optionally, further adjusted for outliers as defined in the item's configuration.

The adjusted historical demand data is then used to calculate demand parameters for use in determining safety stock and reorder point. Specifically, the average historical demand and standard deviation (or forecast error) of those demands for each item. Based on configuration options, these demands parameters can be calculated based on either the bucketed demands (using Intervals calendar) or rolling lead time demands, and the output produced can be either stationary or time-phased.

Optionally, historical supply data can be collected and these (non-bucketed) values then used to calculate average and standard deviation of supply lead times for use as parameters in determining safety stock. Otherwise, if supply variability is not applicable, a fixed lead time value as provided on the safety stock item can be used instead.

Based on the calculated historical demand and supply parameters, as well as the input service level specified on each item, recommended safety stock and reorder point levels are calculated based on standard formulas. The results are presented in analytic based on either the **SafetyStock** or **TimePhasedSafetyStock** functions.

The remainder of this section describes the following aspects of single-echelon safety stock calculations in more detail:

- **Safety stock with fixed lead time**—describes and shows examples of the basic single echelon safety stock calculations performed when fixed lead times are assumed and provided. For more information, see "Safety stock with fixed lead time" on page 1968.
- **Safety stock with variable lead time**—describes and shows examples of the basic single echelon safety stock calculations performed when variable lead times are assumed and calculated from historical supply. For more information, see "Safety stock with variable lead time" on page 1974.
- **Time-phased safety stock**—describes and shows examples of the basic process for calculating safety stock values that change over time in response to seasonal demand patterns or trend. For more information, see "Time-phased safety stock" on page 1977.
- **Days of supply and safety stock**—describes and shows examples of safety stock values that vary over time based on a part's average requirements. For more information, see "Days of supply and safety stock" on page 1981.
- **Safety stock bounds for SEIP**—describes and shows an example of the process for applying safety stock bounds. For more information, see "Safety stock bounds for SEIP" on page 1983.

Safety stock with fixed lead time

When variability of supply lead time is not considered for a given **SafetyStockItem** record (**Type.SupplyVariabilityRule** = "Ignore"), then the fixed lead time as provided on the item is used in calculating safety stock. The recommended safety stock level is intended to provide a buffer against demand variability within this fixed lead time using the following basic formula:

$$\text{SafetyStock} = Z_{\alpha} * \sigma D * \sqrt{LT}$$

where:

- Z_{α} is a normal distribution z-value determined from the specified service level; note that both alpha/type1 (Cycle) and beta/type2 (FillRate) service levels are supported.
- σD^2 is the standard deviation of bucketed historical demands. Based on control settings, this can be calculated using simple methods using historical demands, historical forecast error, or measures of forecast error determined from statistical models. Note also that this parameter is initially calculated in terms of **Type.IntervalsCalendar** units. However if the interval and lead time calendars are different, then the parameter is subsequently divided by the setting in **Type.LeadTimePerInterval** value in order to convert it to lead time units before the safety stock calculation is performed.
- LT is the fixed lead time value provided in **SafetyStockItem.FixedLeadTime**.

Note that if rolling lead time demands are used instead of bucketed demands (that is, if **SafetyStockItemType.UseRollingLeadTimeDemands** is set to "Y"), then the following formula is used instead:

$$\text{SafetyStock} = Z_{\alpha} * \sigma LD$$

where:

- σLD is the standard deviation of lead time demands (both simple standard deviation and measures of forecast error are supported via control settings).

Once the safety stock level has been determined, both historical and future looking reorder points can be calculated. These are intended to ensure coverage of average demand over lead time plus safety stock, and each uses the basic formula below.

$$ReorderPoint = \mu D * \text{LT} + \text{SafetyStock}$$

where:

- μD depends on the reorder point being calculated, and is either the average of historical demands within the historical demand profile (or else the average of historical demands calculated previously if no profile is specified, or else the average of historical forecast if standard deviation is calculated from actual forecast error) or the average of current/future demands within the future demand profile (or else the average of all future demands if no profile is specified).
- LT is the fixed lead time value provided in **SafetyStockItem.LeadTime**.
- SafetyStock is the recommended safety stock value as calculated previously.

Example 1: Fixed lead time (with standard deviation)

The following shows an example of how safety stock is determined when the standard deviation parameter is determined using the simple standard deviation method, and lead time is assumed to be fixed.

Assume a **SafetyStockItem** record for a given part includes the following input and control settings:

- **ProcessingRule** = SingleEchelon
- **ServiceLevel** = .95
- **LeadTime** = 2
- **Type.IntervalsCalendar** = Workday

- **Type.LeadTimeCalendar** = Workday
- **Type.ServiceLevelRule** = Cycle
- **Type.StandardDeviationDemandRule** = StandardDeviation
- **Type.UseRollingLeadTimeDemand** = N
- **Type.SupplyVariabilityRule** = Ignore

Further assume the following set of daily historical demand data is available and bucketed by the Intervals calendar as shown in the following table (assume RunDate is Monday June 3, 2013).

Table 42.13: Sample historical demands

Demand Date	Quantity	Bucket Date	Quantity
May 27, 2013	500	May 27, 2013	500
May 28, 2013	200	May 28, 2013	525
May 28, 2013	325	May 29, 2013	450
May 29, 2013	450	May 30, 2013	570
May 30, 2013	170	May 31, 2013	520
May 30, 2013	400		
May 31, 2013	520		

To determine the required demand parameters for the safety stock calculation, average historical demand is first calculated as follows:

$$500+ \\ 525+ \\ 450+ \\ 570+ \\ \frac{520}{5} = \\ 513$$

Then once average demand has been calculated, it can be used to determine the standard deviation of historical demand as follows:

$$\sqrt{\frac{(500-513)^2+(525-513)^2+(450-513)^2+(570-513)^2+(520-513)^2}{4}}$$

$$=\sqrt{\frac{7580}{4}}=\sqrt{1895}=43.53$$

Once the standard deviation of historical demand has been calculated, it can be used in the basic SafetyStock formula shown in "Safety stock with fixed lead time" on page 1968, along with fixed lead time, and the Z-value corresponding to the specified service level as follows:

$$\mathbf{SafetyStock} = 1.645 * 43.53 * \sqrt{2}$$

$$= 101.26$$

Once the safety stock value has been determined, the reorder point(s) can be calculated. For example, using the ReorderPoint formula shown in "Safety stock with fixed lead time" on page 1968, the historical reorder point would be calculated as follows (assuming no average demand profile is specified).

$$HistoricalReorderPoint = (513 * 2) + 101.26$$

$$= 1127.26$$

Example 2: Fixed lead time (using rolling lead time demands)

The following shows an example of how safety stock is determined when the standard deviation parameter is determined using the simple standard deviation method, and rolling demands within lead time are considered, with lead time assumed to be fixed.

Assume a SafetyStockItem record for a given part has the following input and control settings.

- **ProcessingRule** = SingleEchelon
- **ServiceLevel** = .95
- **LeadTime** = 2
- **Type.IntervalsCalendar** = Workday
- **Type.LeadTimeCalendar** = Workday
- **Type.ServiceLevelRule** = Cycle
- **Type.StandardDeviationDemandRule** = StandardDeviation
- **Type.UseRollingLeadTimeDemand** = Y
- **Type.SupplyVariabilityRule** = Ignore

Further assume the following set of daily historical demand data is available and bucketed by the Intervals calendar as shown within the following table (assume RunDate is Monday June 3, 2013).

Table 42.14: Sample historical demands

Demand Date	Quantity	Bucket Date	Quantity
May 27, 2013	500	May 27, 2013	500
May 28, 2013	200	May 28, 2013	525
May 28, 2013	325	May 29, 2013	450
May 29, 2013	450	May 30, 2013	570
May 30, 2013	170	May 31, 2013	520
May 30, 2013	400		
May 31, 2013	520		

To determine the required demand parameters for the safety stock calculation, average historical demand within lead time is first calculated as follows (using the available points for which demands within lead time can be seen).

$$\begin{array}{l}(\\ 500+\\ 525)+(\\ \frac{525+450)+(450+570)+(570+520)}{4}\end{array} = 1027.5$$

Once average demand within lead time has been calculated, the standard deviation of those rolling lead time demands can be calculated as follows:

$$\sqrt{\frac{(1025-1027.5)^2+(975-1027.5)^2+(1020-1027.5)^2+(1090-1027.5)^2}{3}}$$

$$= \sqrt{\frac{6725}{3}} = \sqrt{2241.66} = 47.34$$

Once the standard deviation of demands within lead time has been calculated, it can be used in the basic SafetyStock formula shown in "Safety stock with fixed lead time" on page 1968, along with the Z-value corresponding to the specified service level as follows:

$$SafetyStock = 1.645 * 47.34$$

$$= 77.87$$

Once the safety stock value has been determined, the historical and future reorder point(s) can be calculated. Assume the set of current/ forecast demands shown in the following table.

Table 42.15: Sample future demands

Date	Demand
June 7, 2013	2300
June 14, 2013	2000
June 21, 2013	2500
June 28, 2013	2100
July 5, 2013	1800

Further suppose the safety stock item has its **FutureAverageDemandProfile** reference pointing to a record with the following values:

- **Offset** —5
- **Multiplier**—5
- **Extend**—0

Based on a RunDate of June 3 2013, future demands are then collected starting on June 10 (`RunDate + 5 Workday`) and over a 10 day period (`RunDate + (5 + (5*2) + 0) Workday`). The forecasts within this value give an average future demand value of 450 units (`4500/10`). The future order point using average demand within lead time and safety stock is then calculated as follows:

$$FutureReorderPoint = (450 * 2) + 487.01$$

$$= 1387.01$$

Example 3: Fixed lead time (with forecast error)

The following shows an example of how safety stock is determined when the standard deviation parameter is determined using historical forecast error, with lead time assumed to be fixed.

Assume a **SafetyStockItem** record for a given part has the following input values and control settings.

- **ProcessingRule** = SingleEchelon
- **ServiceLevel** = .95
- **LeadTime** = 14
- **ForecastLag** = 1
- **Type.IntervalsCalendar** = Month
- **Type.AsOfDateCalendar** = Month
- **Type.LeadTimeCalendar** = Everyday
- **Type.LeadTimePerInterval** = 30

- **Type.ServiceLevelRule** = Cycle
- **Type.StandardDeviationDemandRule** = ForecastError
- **Type.UseRollingLeadTimeDemand** = N
- **Type.SupplyVariabilityRule** = Ignore

Further assume there are monthly bucketed **HistoricalDemandActual** records for the part with quantities as shown in the following table (assume **RunDate** is Monday June 2, 2014):

Table 42.16: Monthly historical demands

Month	Quantity
February 2014	400
March 2014	1200
April 2014	1000
May 2014	500

Finally, assume there are five sets of **HistoricalDemandSeries** records identifying historical forecast series for the part, with each series made up of **HistoricalDemandSeriesDetail** records containing forecast quantities for the subsequent four months as shown in the following table:

Table 42.17: Monthly historical forecast

Series AsOfDate	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep
Jan 31 2014		600	575	500	550				
Feb 28 2014			400	600	500	575			
Mar 31 2014				550	600	475	525		
Apr 30 2014					525	500	475	500	
May 31 2014						575	600	625	600

Next, because **ForecastLag** is set to "1", each actual is compared against forecast from the previous period. For example, historical actuals from May are compared against the forecasted value from the April 30th **AsOfDate** which was"525" (if **ForecastLag** were set to "2", then the March 31st value of 600 would be used instead). As a result, forecast error is determined from historical actuals and forecast as shown in the following table:

Table 42.18: Forecast errors

Month	Actual Quantity	Forecast Quantity	Forecast Error
February 2014	400	600	
March 2014	1200	400	800
April 2014	1000	550	450
May 2014	500	525	75

Standard deviation of forecast error can then be calculated as follows:

$$\sqrt{\frac{(-200)^2+(800)^2+(450)^2+(75)^2}{3}}$$

$$=\sqrt{\frac{888125}{3}}=\sqrt{296041.66}=544.10$$

Once the standard deviation of forecast error is known, it can be used in the basic SafetyStock formula shown in "Safety stock with fixed lead time" on page 1968, along with fixed lead time, and the Z-value corresponding to the specified service level as shown below. Note that the standard deviation parameter is adjusted for the lead time calendar as part of this calculation (using the square root of the value in **LeadTimePerInterval**).

$$\text{SafetyStock}=1.645 * \frac{544.10}{\sqrt{30}} * \sqrt{14}$$

$$=1.645 * 99.339 * 3.742$$

$$=611.49$$

Finally, once the recommended safety stock value has been determined, the reorder point(s) can be calculated. For example, using the ReorderPoint formula shown in "Safety stock with fixed lead time" on page 1968, the historical reorder point would be calculated as shown below. Note that average demand is calculated based on the historical forecast values, and is then further adjusted for the lead time calendar:

$$HistoricalReorderPoint=\left(\frac{4}{\frac{50+525+550+600}{4}}\right)\times 14+611.49$$

$$=\frac{493.75}{30}\times 14+611.49$$

$$=841.79$$

Note: For an example of calculating time-phased days of supply using the data and calculations shown in this example, see "Example: Days of supply and safety stock" on page 1982.

Safety stock with variable lead time

When variability of supply lead time is considered for a given **SafetyStockItem** record (**Type.SupplyVariabilityRule** = "Use" and historical supply data is provided), average lead time for the item is calculated based on when historical supplies were ordered and actually received. The average lead time for each supply is determined from the **Date** and **OrderDate** fields on the **HistoricalSupplyActual** table and expressed in units of the **SafetyStockItem.LeadTimeCalendar** as follows (note also that a **LeadTime** field on **HistoricalSupplyActual** can be populated if the required order date fields are not known):

$$LeadTime=OrderDate-Date\,/\,\text{LeadTimeCalendar}\,/$$

The recommended safety stock level then is intended to provide a buffer against demand variability, while also accounting for variations in supply lead time, using the following basic formula:

$$\text{SafetyStock}=Z_\alpha * \sqrt{\mu\text{LT} * \sigma D^2 + \mu D^2 * \sigma\text{LT}^2}$$

where:

- Z_α is a normal distribution z-value determined from the specified service level; note that both alpha/type1 (Cycle) and beta/type2 (FillRate) service levels are supported.
- $\mu\mathrm{LT}$ is the average of lead time as calculated from historical supplies.
- σD is the standard deviation of historical demands. Based on control settings, this can be calculated using simple methods based on either historical demands or historical forecast error, as well as measures of forecast error determined from statistical models. Note also that this parameter is initially calculated in terms of **Type.IntervalsCalendar** units. However if the interval and lead time calendars are different, then the parameter is subsequently divided by the setting in **Type.LeadTimePerInterval** value in order to convert it to lead time units before the safety stock calculation is performed.(calculations using both simple standard deviation and measures of forecast error are supported via control settings).
- μD is the average historical demand.
- $\sigma\mathrm{LT}$ is the standard deviation of lead time on historical supplies.

Note that if rolling lead time demands are used instead of bucketed demands (that is, if **SafetyStockItemType.UseRollingLeadTimeDemands** is set "Y"), then the formula shown above is modified to:

$$\mathrm{SafetyStock} = Z_\alpha * \sqrt{\sigma\prime D^2 + \mu D^2 * \sigma\mathrm{LT}^2}$$

where:

- $\sigma\prime D$ is the standard deviation of lead time demands (both simple standard deviation and measures of forecast error are supported by way of control settings).

Once the safety stock level has been determined, both historical and future looking reorder points can be calculated. These are intended to ensure coverage of demand over average lead time plus safety stock, and each uses the basic formula below.

$$ReorderPoint = \mu\mathrm{LT} * \mu D + \mathrm{SafetyStock}$$

where:

- $\mu\mathrm{LT}$ is the average lead time calculated on historical supplies.
- μD is the average historical demand.
- $\mathrm{SafetyStock}$ is the recommended safety stock value.

Example: variable lead time

The following shows an example of how safety stock is determined when the standard deviation parameter is determined using the simple standard deviation method, and lead time is assumed to be variable (based on history).

Assume a **SafetyStockItem** record for a given part includes the following input and control settings:

- **ProcessingRule** = SingleEchelon
- **ServiceLevel** = .95
- **LeadTime** = 2
- **Type.IntervalsCalendar** = Workday
- **Type.ServiceLevelRule** = Cycle
- **Type.StandardDeviationDemandRule** = StandardDeviation

- **Type.UseRollingLeadTimeDemand** = N
- **Type.SupplyVariabilityRule** = Use

Further assume the following set of daily historical demand data is available and bucketed by the Intervals calendar as shown within the following table (assume RunDate is Monday June 3, 2013).

Table 42.19: Sample historical demands

Demand Date	Quantity	Bucket Date	Quantity
May 27, 2013	500	May 27, 2013	500
May 28, 2013	200	May 28, 2013	525
May 28, 2013	325	May 29, 2013	450
May 29, 2013	450	May 30, 2013	570
May 30, 2013	170	May 31, 2013	520
May 30, 2013	400		
May 31, 2013	520		

To determine the required demand parameters for the safety stock calculation, average historical demand within lead time is first calculated as follows:

$$\begin{array}{l} 500+ \\ 525+ \\ 450+ \\ 570+ \\ \frac{520}{5} = \end{array}$$
$$513$$

Then once average demand has been calculated, it can be used to determine the standard deviation of historical demand as follows:

$$\sqrt{\frac{(500-513)^2+(525-513)^2+(450-513)^2+(570-513)^2+(520-513)^2}{4}}$$

$$=\sqrt{\frac{7580}{4}}=\sqrt{1895}=43.5316$$

Further assume the following set of historical supply records is available for the part where **Date** indicates the date on which an order was received, and **OrderDate** indicates the date the order was placed.

Table 42.20: Sample historical supplies

Date	OrderDate
May 29, 2013	May 27, 2013
May 30, 2013	May 27, 2013
May 30, 2013	May 28, 2013
May 31, 2013	May 28, 2013

Table 42.20: Sample historical supplies

Date	OrderDate
May 31, 2013	May 28, 2013
May 31, 2013	May 29, 2013

Thus, the difference between the OrderDate and Date values on historical supplies can be used to calculate average lead time as follows:

$$\frac{2+3+2+3+3+2}{6} = 2.5$$

Once average historical lead time is known, the standard deviation of supply lead time can also be determined as follows:

$$\sqrt{\frac{(2-2.5)^2+(3-2.5)^2+(2-2.5)^2+(3-2.5)^2+(2-2.5)^2+(2-2.5)^2}{5}}$$

$$= \sqrt{\frac{1.5}{5}} = \sqrt{3} = .5477$$

Once the average and standard deviation of historical demands, and average and standard deviation of historical lead times have been calculated, they can be used in the SafetyStock formula shown in "Safety stock with variable lead time" on page 1974, along with the Z-value corresponding to the specified service level, to determine the recommended safety stock level as follows:

$$SafetyStock = 1.645 * \sqrt{2.5 * 43.5316^2 + 513^2 * .5477^2}$$

$$= 1.645 * \sqrt{83652.8726}$$

$$= 475.85$$

Finally, to calculate reorder point, average demand across lead time should be added to the calculated safety stock level. For example, the historical reorder point would be:

$$HistoricalReorderPoint = 2.5 * 513 + 475.85$$

$$= 1758.35$$

Time-phased safety stock

If demand for an item is typically not stationary (for example, subject to seasonality or trend), then time-phased safety stock and reorder point recommendations can be generated. Time-phased safety stock calculations apply to items where the **Type.TimePhasedProcessingRule** is set to "Use", and use the following calendars:

- **Cycle calendar**—defines a full seasonal cycle of data and is set by each items **Type.CycleCalendar** reference. This can be thought of as the "outer calendar" and might typically be set to a yearly calendar.
- **Period calendar**—defines the individual periods or seasons within a full cycle and is set by each item's **Type.PeriodCalendar** reference. This can be thought of as the "inner calendar" and might typically be set to a monthly or quarterly calendar. The **Type.PeriodsPerCycle** value also applies and defines the number of periods that make up a full cycle.

These two calendars are then used when generating time-phased demand parameters. For each period, historical demands are collected from all applicable cycles in the historical collection interval. In other words, period data might be collected from a single cycle or across multiple cycles. Average historical demand and standard deviation of historical demand is then calculated separately for each period in the cycle. These calculated parameters can be seen in the **SafetyStockItemDemandParameterSet** table. Each set of time-phased demand parameters is also assigned a zero-based index value where 0 indicates the current period. For example, if using Monthly periods and the current month is November, then 0 represents November, 1 represents December, 2 represents January, and so on.

Once the historical demand parameters for each period have been calculated and indexed, they can be used as input in calculating safety stock. When calculating safety stock for a given period or point in time, the demand parameters used as input in determining a time-phased safety stock quantity are those belonging to the indexed value(s) found across the item's lead time if an order were to be received at the current period in time (in other words, it assumes the order was made in the past).

The basic formula for time-phased safety stock thus requires a backward looking approach that makes use of the demand parameters that were effective at each point within the item's lead time as follows:

$$SafetyStock = Z_{\alpha(t)} \sqrt{\sum_{i=1}^{LT} \sigma D^2 \left(t - \mathrm{LT} + i\right)}$$

where:

- $Z_{\alpha(t)}$ is a normal distribution z-value determined from the specified service level; note that both alpha/type1 ("Cycle") and beta/type2 ("FillRate") service levels are supported. This value will be stationary if using the "Cycle" option but time-phased if using the "FillRate" option.
- σD is the standard deviation of historical demands for a given indexed period in the cycle. Note that simple calculations of standard deviation, as well as time-series decomposition and HoltWinters methods are supported based on the setting in the **SafetyStockItemType.NonStationaryDemandRule** field.
- t is the current period in time (where the order is expected to have been placed at point *t - LT*). This parameter represents a given date or period on which safety stock is being determined.
- LT is the fixed lead time value provided for the item (note that variable lead time is not supported in time-phased calculations). This defines the number of backward looking periods from which the effective standard deviation of demand is summed.

Once time-phased safety stock values have been calculated, the required reorder quantities can be determined. To determine reorder quantity for a given point in time, a forward looking approach is used. This involves summing the applicable average demand values across the item's lead time together with the safety stock value effective at the end of lead time as follows:

$$ReorderPoint = \left(\sum_{i=1}^{LT} \mu D \left(t + i\right) \right) + SafetyStock \left(t + LT\right)$$

where:

- μD is the average historical demand or average future demand. Note that future order point calculations, as well as historical order point calculations that rely on an average demand profile, calculate and use a single average demand value across all periods. However, historical order points that do not use a profile make use of the time-phased average demand values as calculated previously.
- t is the current point in time (where an order placed will be received at point *t - LT*).
- LT is the fixed lead time value provided for the item.
- $SafetyStock$ is the recommended safety stock value as calculated for a given period.

Example: Time-phased safety stock calculation

Assume a **SafetyStockItem** record for a part includes the following input and control settings:

- **ProcessingRule**—SingleEchelon
- **LeadTime**—2
- **Type.TimePhasedProcessingRule**—Use
- **Type.IntervalsCalendar**—Month (typically, this might be set to a daily calendar).
- **Type.LeadTimeCalendar**— Month
- **Type.CycleCalendar**—Year
- **Type.PeriodCalendar**—Quarter
- **Type.PeriodsPerCycle**—4
- **Type.ServiceLevelRule**—Cycle
- **Type.NonStationaryDemandRule**—Simple
- **Type.UseRollingLeadTimeDemand**—N

Further assume the RunDate is Monday June 3, 2013 and that historical demand data is available for each of the previous 13 months as shown in the following table.

Table 42.21: Sample historical demand data

Month	Demand
May 2012	1000
June 2012	2000
July 2012	2200
August 2012	1500
September 2012	1200
October 2012	450
November 2012	250
December 2012	200
January 2013	100
February 2013	200
March 2013	400
April 2013	500
May 2013	1200

In order to calculate time-phased safety stock, the required demand parameters are calculated within each cycle period. In this case, demand parameters are required for each of the four quarterly periods within a yearly cycle.

For example, to calculate average historical demand for the second quarter (period) in a yearly cycle, the available historical demand values for each of the months (intervals) within that quarter can be used. In this case, historical data from April and May of 2012 as well as historical data from May and June of 2013 is available. Thus, average historical demand for this quarterly period is calculated as follows:

$$\frac{1000+2000+500+1200}{4} = 1175$$

Once the period's average historical demand has been calculated, it can be used to determine the standard deviation of historical demands within the period as follows:

$$\sqrt{\frac{(1000-1175)^2+(2000-1175)^2+(500-1175)^2+(1200-1175)^2}{3}}$$

$$= \sqrt{\frac{1167500}{3}} = \sqrt{389166.66} = 623.83$$

The following table shows the average historical demand and standard deviation values that are calculated for each of the four cycle periods. Note that because the current period containing the RunDate falls within the second quarter (April - June), this period is assigned the Index value of 0 (incremented by 1 for each subsequent period).

Table 42.22: Time-phased demand parameters

Index	Period (Quarter)	Average Demand	Standard Deviation
0	April - June	1175.00	623.83
1	July -September	1633.33	513.16
2	October - December	300.00	259.81
3	January - March	233.33	152.75

Then to generate a safety stock recommendation for a given point in time, the appropriate time-phased standard deviation values should be applied as determined by the SafetyStock formula shown in "Time-phased safety stock" on page 1977. For example, based on a two month lead-time for the part, the recommended safety stock for July would be calculated as shown below. Note that standard deviation of historical demands from the current period as well as the previous period are used in this case.

$$= 1.645\sqrt{\sum_{i=1}^{2} \sigma D^2 \left(t - 2 + i\right)}$$

$$= 1.645\sqrt{\sigma^2(\text{July} - 2 + 1) + \sigma^2(\text{July} - 2 + 2)}$$

$$= 1.645\sqrt{\sigma^2(\text{June}) + \sigma^2(\text{July})}$$

$$= 1.645\sqrt{\sigma^2(\text{Index0}) + \sigma^2(\text{Index1})}$$

$$= 1.645\sqrt{623.83^2 + 513.16^2}$$

$$= 1.645\sqrt{652497.05}$$

$$= 1.645 * 807.77$$

$$= 1328.78$$

Similarly in the month of August, the recommended safety stock value would be calculated as shown below. Note that in this case, only the standard deviation from the current period applies and hence a different safety stock level is determined than in July.

$$SafetyStock = 1.645\sqrt{\sigma^2(\text{August} - 2 + 1) + \sigma^2(\text{August} - 2 + 2)}$$

$$= 1.645\sqrt{\sigma^2(\text{July}) + \sigma^2(\text{August})}$$

$$= 1.645\sqrt{\sigma^2(\text{Index1}) + \sigma^2(\text{Index1})}$$

$$= 1.645\sqrt{513.16^2 + 513.16^2}$$

$$= 1.645\sqrt{526666.66}$$

$= 1.645 * 725.71$

$= 1193.79$

Once the recommended safety stock levels have been determined, the applicable reorder points can also be found. For example, the following shows the calculation of historical reorder point for June:

$$HistoricalReorderPoint = \left(\sum_{i=1}^{2} \mu D\left(June + i\right) \right) + SafetyStock\left(June + 2\right)$$

$$= \mu D\left(July\right) + \mu D\left(August\right) + SafetyStock\left(August\right)$$

$$= 1633.33 + 1633.33 + 1193.79$$

$$= 4460.65$$

Days of supply and safety stock

Time-phased safety stock levels can be calculated based on a part's average daily requirements. This applies to parts where **Type.TimePhasedProcessingRule** is set to either "DaysOfSupplyBackward" or "DaysOfSupplyForward".

These options are suitable for producing safety stock values that vary over time, but use a standard deviation parameter based on the **Type.StandardDeviationDemandRule** setting rather than the **Type.NonStationaryDemandRule** setting. Thus, days of supply calculations can be used to generate time-phased safety stock values for parts whose standard deviation is calculated based on methods such as forecast error.

In order to calculate time-phased safety stock using days of supply, a single safety stock value is first calculated (as would be done if **Type.TimePhasedProcessingRule** were set to "Ignore"). This calculated value is then used along with the average historical demand to determine the days of supply parameter as follows.

$$\frac{Calculated\ \text{Safety Stock}}{Average\ \text{Historical Demand}} = DoS$$

Note that the days of supply parameter (DoS) for an item is expressed in terms of the calendar it references through **Type.IntervalsCalendar**. For example, it might represent a number of days, weeks, or months of supply. Once the days of supply value has been determined, it sets the horizon over which the part's historical and/or future looking requirements are collected and summed to determine the safety stock level for a given date.

The expression used to calculate safety stock with days of supply depends on the **Type.TimePhasedProcessingRule** as shown in the following illustration.

DaysOfSupplyBackward	DaysOfSupplyForward
$SS(t) = \sum_{i=1}^{DoS} D\left(t - i + 1\right)$	$SS(t) = \sum_{i=1}^{DoS} D\left(t + i - 1\right)$

where:

- DoS represents the number of **SafetyStockItemType.IntervalsCalendar** over which a part's historical and/or future looking demands should be collected and summed in order to calculate safety stock for a given date. Note that this number might be fractional in which case only that fraction/percentage of demand is taken from the last of the intervals in which demand is being collected (either the earliest interval used with "DaysOfSupplyBackward" or the latest interval used with "DaysOfSupplyForward").

- D represents the demand collected in a given interval. If the date demand is being collected for is before RunDate, then this is a historical demand as taken from the **HistoricalDemandActual** table. Otherwise, if the date demand is being collected for is on or after RunDate, then this is the current/forecast demand as taken from the **Demand** table.
- t represents the date or interval for which time-phased safety stock is being calculated. For example, this might be the beginning of a particular weekly or monthly period.
- i represents a point in the days of supply horizon.

Example: Days of supply and safety stock

Assume a **SafetyStockItem** record for a given part has the input values and control settings shown below. Note that these settings are identical to those previously shown in "Example 3: Fixed lead time (with forecast error)" on page 1972 except for the **TimePhasedProcessingRule** setting which was previously assumed to be "Ignore". Additionally, assume the same input data and calculated output as were shown in "Example 3: Fixed lead time (with forecast error)" on page 1972.

- **ProcessingRule** = SingleEchelon
- **ServiceLevel** = .95
- **LeadTime** = 14
- **ForecastLag** = 1
- **Type.IntervalsCalendar** = Month
- **Type.AsOfDateCalendar** = Month
- **Type.LeadTimeCalendar** = Everyday
- **Type.LeadTimePerInterval** = 30
- **Type.ServiceLevelRule** = Cycle
- **Type.StandardDeviationDemandRule** = ForecastError
- **Type.UseRollingLeadTimeDemand** = N
- **Type.SupplyVariabilityRule** = Ignore
- **Type.TimePhasedProcessingRule** = DaysOfSupplyBackward

In order to calculate time-phased safety stock using the "DaysOfSupplyBackward" setting, a single safety stock value (as used in non-time phased cases) and an average historical demand both need to be determined. Recall from "Example 3: Fixed lead time (with forecast error)" on page 1972 that safety stock was calculated to be *611.49* and average historical demand was calculated to be *493.*75. From this, the days of supply parameter can be calculated as follows:

$$\frac{611.49}{493.75} = DoS$$

$$\frac{611.49}{493.75} = 1.24$$

Because the days of supply value is expressed in terms of the **IntervalsCalendar**, this means the calculated safety stock value represents 1.24 months of supply. To calculate time-phased safety stock for a given month, this value is then used to set the horizon (number of months) over which actuals/forecasts are summed to determine time-phased safety stock as shown in the following equation:

$$\mathrm{SS(t)} = \sum_{i=1}^{1.24} D\,(t - i + 1)$$

For example, assume that **RunDate** is Monday June 2, and there are monthly bucketed historical actual demands (originating in the **HistoricalDemandActual** table) and monthly bucketed current/forecast demands (from the **Demand** table) as shown in the following table.

Table 42.23: Monthly historical and current/forecast demands

	FEB	MAR	APR	MAY	JUN	JUL	AUG
Historical Actual	400	1,200	1,000	500			
Current/Forecast					550	475	500

The time-phased safety stock value generated for the month of June would then be calculated as follows:

$$\mathbf{SS(June)} = D(June - 1 + 1) + D(June - 1.24 + 1)$$

$$\mathbf{SS(June)} = D(June - 0) + D(June - .24)$$

$$\mathbf{SS(June)} = (June\ \mathbf{forecasts}) + (\mathbf{May\ actuals})\ * .24$$

$$\mathbf{SS(June)} = (550) + (500 * .24)$$

$$\mathbf{SS(June)} = 670$$

Similarly, the time-phased safety stock value generated for the month of July would then be calculated as follows:

$$\mathbf{SS(July)} = D(July - 1 + 1) + D(July - 1.24 + 1)$$

$$\mathbf{SS(July)} = (July\ \mathbf{forecasts}) + (June\ \mathbf{forecasts}\ * .24)$$

$$\mathbf{SS(July)} = (475) + (550 * .24)$$

$$\mathbf{SS(July)} = 607$$

Safety stock bounds for SEIP

Safety stock bounds, if any, are applied after all other SEIP calculations have been performed. The optimal safety stock level, calculated without any consideration for safety stock bounds, is recorded in the **UnboundedSafetyStock** fields in the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables.

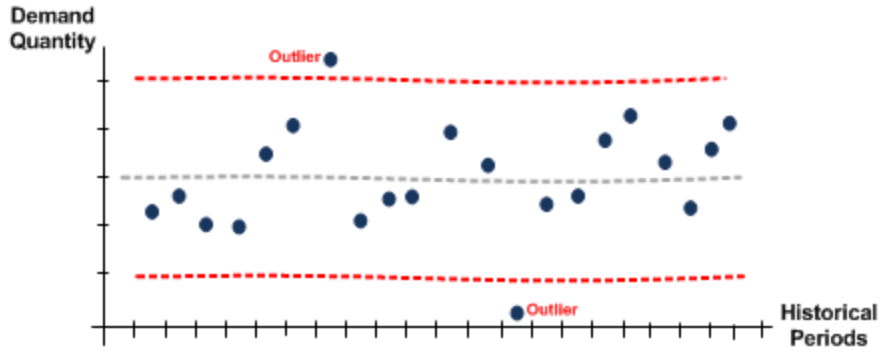
Next, safety stock bounds are considered:

- If the unbounded safety stock value does not violate safety stock bounds, it is copied into the **SafetyStock** field.
- If the unbounded safety stock value violates a safety stock bound, the bound that was violated is copied into the **SafetyStock** field instead. For example, if the unbounded safety stock value is 112 and a maximum quantity bound of 100 applies, **SafetyStock** is 100.

Safety stock item outlier processing

Outliers refer to points in the data that are generally not consistent with the overall pattern of the data. For example, they might refer to points that are more than a specified distance away from the average point in the data.

Figure 42.11 Outliers in historical actuals



When calculating safety stock, the presence of outliers in historical data can limit the ability to generate reasonably accurate recommendations. In order to identify and process outliers in the historical data, several configuration options are available in RapidResponse. These are mostly set through the **OutlierType** control table, as well supporting fields in the **SafetyStockItem** and **MEIOLeadTime** input tables.

Note: The **Cleanse Historical Data** workbook is used to do this process: the **Outlier Details** worksheet is used to configure the outlier types and parameters, while the **Manage Outliers** worksheet is used to manage the suggested outlier adjustments.

Safety stock item outlier data model

Previously, outlier processing in historical actual and forecast demand data was controlled through the **DemandOutlierProcessingRule** and the **ForecastErrorOutlierProcessingRule** settings on the **SafetyStockItemType** table (these fields are now hidden and not recommended, though existing resources that use them will continue to function).

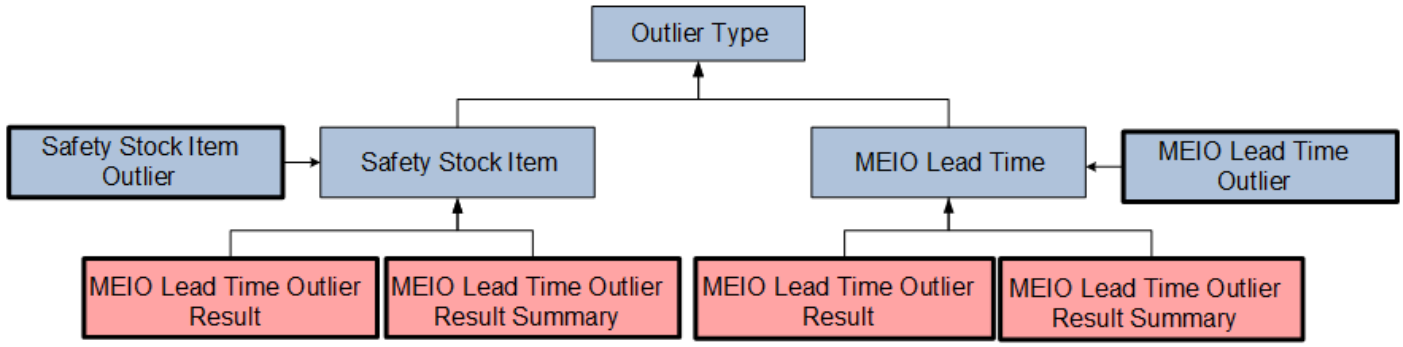
As of Version 2016.2, Service Update 5, outlier processing is applicable to historical actual and forecast demands, as well as to historical lead times on supplies in the context of both single-echelon and multi-echelon stages. These options are configured through several new input fields on the **SafetyStockItem** and **MEIOLeadTime** tables.

If the **SafetyStockItem.DemandOutlierType** is null (blank), then RapidResponse will use the previous outlier processing to calculate outliers through the **SafetyStockItemType.DemandOutlierProcessingRule**.

On the **SafetyStockItem** table, both the **DemandOutlierType** and the **LeadTimeOutlierType** fields reference the **OutlierType** that defines how to process outliers in historical data. The **DemandOutlierType** field is used for outliers in historical demand or historical forecast error, while the **LeadTimeOutlierType** field is used for outliers in historical lead time for SEIP only. On the **MEIOLeadTime** table, the **LeadTimeOutlierType** field is used for outliers in historical lead time for MEIO only.

The following figure shows the tables used in the Inventory Planning and Optimization outlier data model. The tables with a thick border are new as of Version 2016.2, Service Update 5.

Figure 42.12 Tables used for IP&O outlier processing



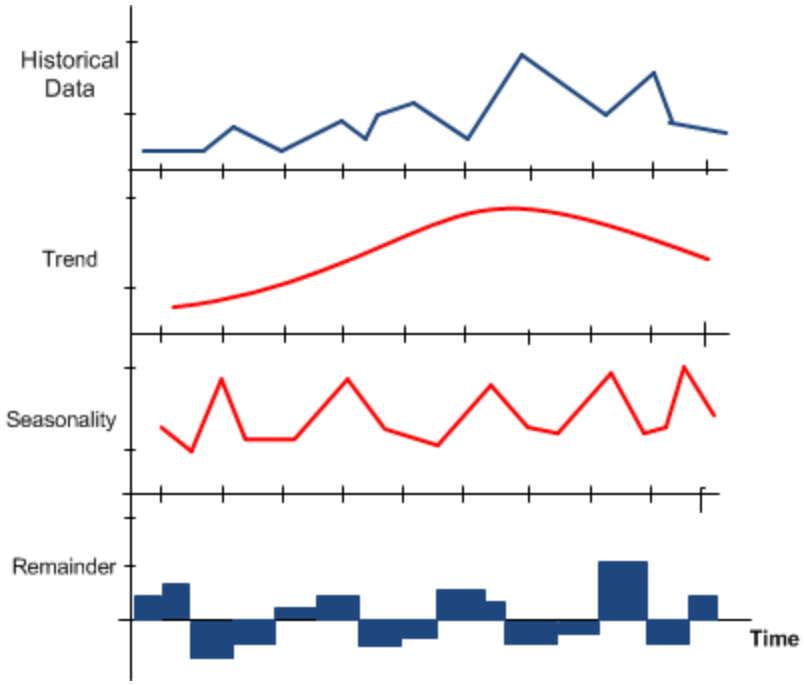
Data used for outlier detection

The **OutlierType.DataRule** setting specifies the actual data points to be evaluated for the presence of outliers. With the default setting of "Historical", outlier detection is performed against raw data points from historical date periods used as input in generating the historical demand.

Instead, if the **DataRule** is set to "MovingAverageError", then outlier detection is made against points that represent the difference between the historical actual quantity in each period and the moving average calculated using a defined number of points before that actual. With this setting, the number of periods used in calculating the moving average is set by the **DemandOutlierMovingAverageWindow** field on the **SafetyStockItem** table and the **LeadTimeOutlierMovingAverageWindow** field on the **SafetyStockItem** and **MEIOLeadTime** tables.

Finally, if the optional Rforecast capability has been enabled, the **DataRule** can be set to "RstlError". This option uses the stl function in the R programming language to return a set of adjusted actuals for use in detecting outliers. That is, outliers are then detected using historical data after any seasonal or trend components have been removed as shown in the following illustration.

Figure 42.13 Decomposed time series data



Rules for outlier detection

Using data points as determined by the selected **DataRule**, outliers are then detected based on the **OutlierType.DetectionRule** setting. Most options in this field are used in conjunction with the **DemandOutlierThreshold** field on the **SafetyStockItem** table and the **LeadTimeOutlierThreshold** field on the **SafetyStockItem** and **MEIOLeadTime** tables to calculate upper and lower thresholds. Points above or below those threshold values are considered to be outliers.

The **DetectionRule** contains the following four settings;

- StandardDeviation (see "StandardDeviation" on page 1986)
- IglewiczHoaglinMethod (see "IglewiczHoaglinMethod" on page 1986)
- Winsorizing (see "Winsorizing" on page 1987)
- Rtsclean (see "Rtsclean" on page 1987)

StandardDeviation

This is the default setting in the **DetectionRule** field. It defines outliers as being more than a specified number of standard deviations away from the mean. With this setting, the **DemandOutlierThreshold** and **LeadTimeOutlierThreshold** values are used to indicate the number of standard deviations above or below the mean where a point is considered an outlier.

For example, assume the following historical actuals, along with a **DetectionRule** of "StandardDeviation" and a **DemandOutlierThreshold** value of "3".

Jan	Feb	Mar	Apr	May	June
600	550	400	600	650	500

In this case, the mean would be calculated as follows:

$$\frac{500+650+600+400+550+600}{6} = 550$$

And the standard deviation calculated as:

$$\sqrt{\frac{(500-550)^2 + (650-550)^2 + (600-550)^2 + (400-550)^2 + (550-550)^2 + (600-550)^2}{5}} = 89.44$$

Thus, a point would be considered an outlier if less than 281.68 (that is, *550-(89.44*3)*) or greater than 818.32 (that is, *550+(89.44*3)*).

IglewiczHoaglinMethod

This setting uses a modified z-score to measure distance from the mean. With this setting, a point is considered an outlier if it is outside the range shown in the following formula

$$[Md - X*(\text{MAD}/.6745), Md + X*(\text{MAD}/.6745)]$$ where

- *Md*—median,
- MAD—median absolute deviation
- *X*— OutlierThreshold

For example, assume again the following historical actuals, along with a **DetectionRule** of "IglewiczHoaglinMethod".

Jan	Feb	Mar	Apr	May	June
600	550	400	600	650	500

Further assuming a **DemandOutlierThreshold** of "2.5", the outlier thresholds would be calculated as follows:

$$[575 - 2.5 * (66.67 / .6745), 575 + 2.5 * (66.67 / .6745)] = [389.68, 760.32]$$

Thus, a point would be considered an outlier if less than 389.68 or greater than 760.32.

Winsorizing

This setting allows for outliers to be defined as those points outside of a specified percentile in the sorted data set. This can be useful for removing upper and lower tails in the large data sets. With this setting, the **DemandOutlierThreshold** and **LeadTimeOutlierThreshold** fields are used to define the upper and lower percentiles to be removed from the data, and a value between .01 and .49 should be provided.

For example, assume the following set of 12 months of historical actuals and a **DemandOutlierThreshold** value of "0.2".

Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
50	100	55	65	70	55	80	30	95	85	60	45

In this case, sorting the actuals in ascending order gives the following ordered set:
[30, 45, 50, 55, 55, 60, 65, 70, 80, 85, 95, 100]

Using the sorted data, the 20th percentile point can be found as 12 * .2 = 2.4 (rounded up to the 3rd ordered point, gives 50) and the 80th percentile can be found as 12 * .8 = 9.6 (rounded up to the 10th ordered point gives 85). Thus, a point would be considered an outlier if less than 51 or greater than 84.

Rtsclean

Optionally, if the Rforecast capability has been enabled, the **DetectionRule** option can be set to "Rtsclean". This setting uses the "tsclean" method from the R statistical programming language. To identify outliers, this method uses loess for non-seasonal data and a periodic stl decomposition for seasonal data. To estimate missing values, linear interpolation is used for non-seasonal series, and a periodic stl decomposition is used with seasonal series

Note also that the "Rtsclean" setting should only be used with pure historical data that has not had any decomposition applied. As a result, when the **DetectionRule** is set to "Rtsclean", the **DataRule** setting is always treated as if it were set to "Historical".

Processing outliers

Once an outlier has been detected within an item's historical data, it should be adjusted to a level closer to the general pattern of the data set. To support outlier adjustments, the **AboveThresholdRule** and **BelowThresholdRule** settings on the **OutlierType** table can be used.

The **AboveThresholdRule** field specifies how data points above the upper outlier threshold are handled, while the **BelowThresholdRule** field specifies how data points below the lower outlier threshold are handled. Otherwise, these two fields are conceptually similar and contain the same four options for adjusting outlier data as follows:

- **Forecast**—the outlier quantity is replaced by a calculated value dependent on the **DataRule** setting. If using the default "Historical" option, then this option is the same as "Mean" (replaces the outlier with the average from the data). Otherwise, if using the "MovingAverageError" option, the moving average up to the point where the outlier was found is used, and if using the "RstlError" option, the sum of the trend and seasonal components are used to replace the outlier.
- **Ignore**—the outlier quantity is left as is without any adjustments.
- **Mean**—the outlier quantity is replaced by the mean (average) point in the data set.
- **Median**—the outlier quantity is replaced the median (middle) point in the data set.
- **SmoothKeepExcess**—the outlier quantity is smoothed above the threshold over a number of intervals backward and then forward. If after smoothing, any excess remains above the outlier threshold, it is left as is.
- **SmoothRemoveExcess**—the outlier quantity is smoothed above the threshold over a number of intervals backward and then forward. If after smoothing, any excess remains above the outlier threshold, that excess is reduced to the outlier threshold for the item.
- **Threshold**—the outlier quantity is replaced by the lower threshold (if below) or the upper threshold level (if above).

If you are using any of the smoothing options for **OutlierType.AboveThresholdRule** or **OutlierType.BelowThresholdRule**, there are a few considerations to keep in mind. The following table describes the order in which the selected smoothing options are applied.

When you	Result
Specify only one of the threshold rules to use either **SmoothKeepExcess** or **SmoothRemoveExcess**. For example, you set **SmoothKeepExcess** to the **AboveThresholdRule**, but do not apply smoothing to **BelowThresholdRule**.	The smoothing correction is applied to the outliers in the input data series. Then, if outliers remain in the corrected data, one of the following non-smoothing rules will be applied, as applicable: Forecast, Ignore, Mean, Median, or Threshold.
Specify both threshold rules to use either **SmoothKeepExcess** or **SmoothRemoveExcess**. For example, you set **SmoothRemoveExcess** to both the **AboveThresholdRule** and **BelowThresholdRule**.	The smoothing correction is first applied in the input data series only to outliers that are "above" the threshold. Then, if outliers that are "below" the threshold remain in the corrected data, the non-smoothing rule is applied.

Calculated outlier details

Outliers in historical actual and historical forecast demand data for both single and multi-echelon safety stock items, as well as in historical lead time data for single-echelon safety stock items are calculated based on the **SafetyStockItem** table, and the results are stored in the **SafetyStockItemOutlierResult** and the **SafetyStockItemOutlierResultSummary** tables. The Lead Time in **SafetyStockHistoricalSupply** and the Quantity in **SafetyStockHistoricalDemand** will be corrected based on the outlier adjustments specified in the **SafetyStockItemOutlier** table.

Similarly, outliers in historical lead time data for multi-echelon safety stock items are calculated based on the **MEIOLeadTime** table, and the results are store in the **MEIOLeadTimeOutlierResult** and the **MEIOLeadTimeOutlierResultSummary** tables. The Lead Time in the **MEIOHistoricalSupply** will be corrected based on the data in the **MEIOLeadTimeOutlier** table.

Managing outlier adjustments

Previously, outlier adjustments were automatically applied to any outliers that are detected. As of Version 2016.2, Service Update 5, you can manually adjust any suggested outlier adjustments.

Outlier results that used to be reported in the **SafetyStockHistoricalDemand** table are now reported in the **SafetyStockItemOutlierResult** table. Those records can now be copied as they are in the **SafetyStockItemOutlier** table, or can be modified or rejected.

In this version, if the outlier type(s) and parameters are changed, RapidResponse recalculates the outliers, and their suggested adjustments are reported in the **SafetyStockItemOutlierResult** table. The "Recalculate Outliers" command should be run only when changing outlier types and parameters to re-copy the results from **SafetyStockItemOutlierResult** to **SafetyStockItemOutlier**. The command will:

1. Delete any records where there has been a change in the date between the revised **SafetyStockItemOutlierResult** calculations and the **SafetyStockItemOutlier** input table.
2. Insert or update all other records.

You can use the "Lock Override" function to lock your manual override of the suggested historical demand quantity for an outlier. To lock the override, place a checkmark in the **Lock Override** column and click **Save Data**.

Note: The **Manage Outliers** worksheet in the **Cleanse Historical Data** workbook is used to do this process.

Multi-echelon safety stock calculations

Multi-echelon safety stock calculations generate recommended safety stock levels across a network or family of related parts based on parameters applicable to one or more customer facing end items and propagated to lower-level components in the network. The recommended safety stock levels are meant to satisfy the specified service levels and service time provided on the end item(s) while also minimizing inventory holding cost through strategic placement of that safety stock amongst the parts in the network.

This section describes each of the following aspects of multi-echelon safety stock calculations:

- **Create multi-echelon families**—describes how multi-echelon families are constructed under safety stock items. For more information, see "Create multi-echelon families" on page 1990.
- **Calculate inventory holding costs**—describes how the per unit holding cost of inventory for a part is calculated. Inventory holding cost values are required by multi-echelon calculations for use in determining optimal inventory placement. For more information, see "Calculate inventory holding costs for MEIO" on page 1996.
- **Determine demand and supply parameters**—discusses the demand and supply parameters required as input by multi-echelon calculations and how they are determined. For more information, see "Determine demand and supply parameters for MEIO" on page 1999.
- **Recommend safety stock**—describes the model used for determining the levels of safety stock required and how that safety stock is placed across a network of parts. For more information, see "Recommend safety stock" on page 2005.

- **Applying safety stock bounds in MEIO**—describes how safety stock bounds affect safety stock recommendations. For more information, see "Apply safety stock bounds in MEIO" on page 2010.

Create multi-echelon families

In multi-echelon inventory optimization, multi-echelon families are created under one or more customer-facing parts that have a **SafetyStockItem** record where the **ProcessingRule** value is set to "MultiEchelon". Each family is created based on **BillOfMaterial** and **PartSource** relationships under the end item part(s) and defines a group of parts that should be processed together when generating safety stock recommendations. Each part brought into a multi-echelon family can also be thought of as a stage within the family. This term is used in calculated RapidResponse tables such as MEIOStage.

The **MultiEchelonSafetyStockRule** control table is also used in building multi-echelon families. This table is referenced by the **Part** table, its **ProcessingRule** field contains the following values which govern whether a part can be used in multi-echelon families:

- **Ignore**—the part is not eligible to be brought into multi-echelon families regardless of its BillOfMaterial or PartSource relationships.
- **NoInventory**—the part can be brought into multi-echelon and its parameters used in safety stock calculations. However, safety stock recommendations are not meant to be generated for the part.
- **Use**—the part can be brought into multi-echelon and its parameters used in safety stock calculations. As applicable, safety stock recommendations are generated for the part.

Thus, family construction begins by scanning through the BillOfMaterial under a given item and evaluating the **MultiEchelonSafetyStockRule.ProcessingRule** setting on the component parts within it. If an eligible component has a primary part source of type "make" then the part is brought into the family, and additional components under it are further evaluated and brought into the family as appropriate. Note that each stage created to model a part within the family also notes the applicable part source. Similarly, when components with a primary part source type of "buy" or "transfer" are found, they too are brought into the family as applicable based on the part-level setting.

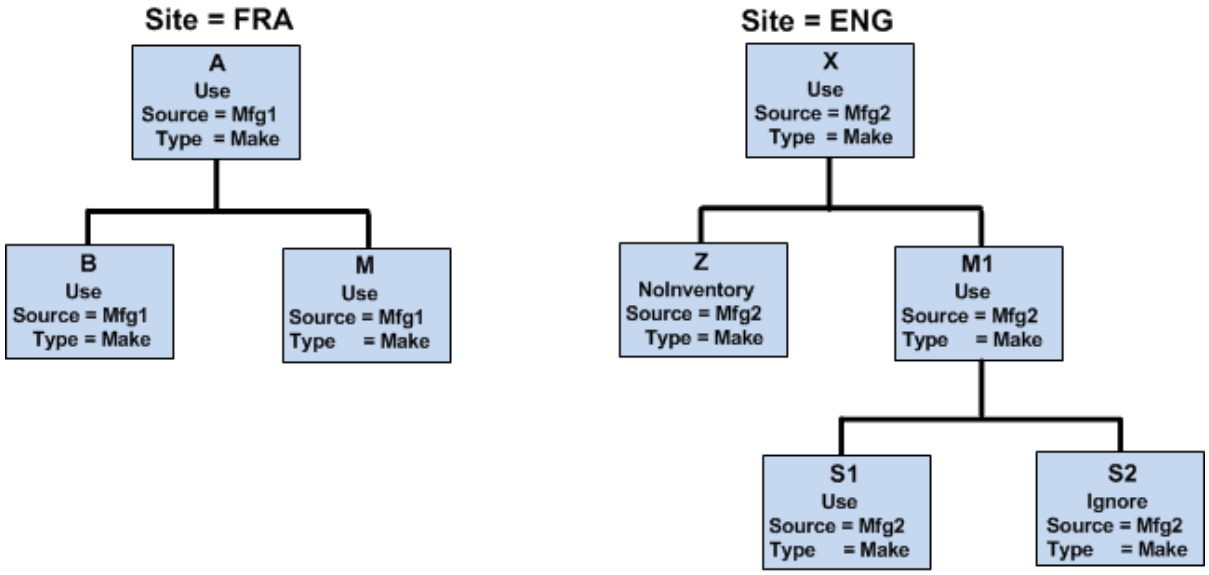
However, additional special logic is required for "buy" and "transfer" parts when building a family. If there are no other part sources with the same priority as a primary part source of type "buy" then only that source is brought into the family. However, if there are additional "buy" sources with the same priority as the primary, then each of those sources are also brought into the family. To model this, RapidResponse creates a "real" stage in the family to represent the part itself, and injects additional "artificial" stages to model each of the different "buy" part sources for the part (that have the same priority as the primary part source). For example, different cost details might pertain to each of the applicable part sources and they are therefore modeled separately.

Similarly, for transfer sources, an artificial stage is always injected to model the source and part brought in from another site, with the part itself noted on the "real" stage. This supports modeling situations where a part being transferred has different processing or lead times depending on its destination site. Refer to examples 2 and 3 later in this section for illustrations of how buy and transfer parts are handled in multi-echelon families.

Example 1: Safety stock items with only make sources

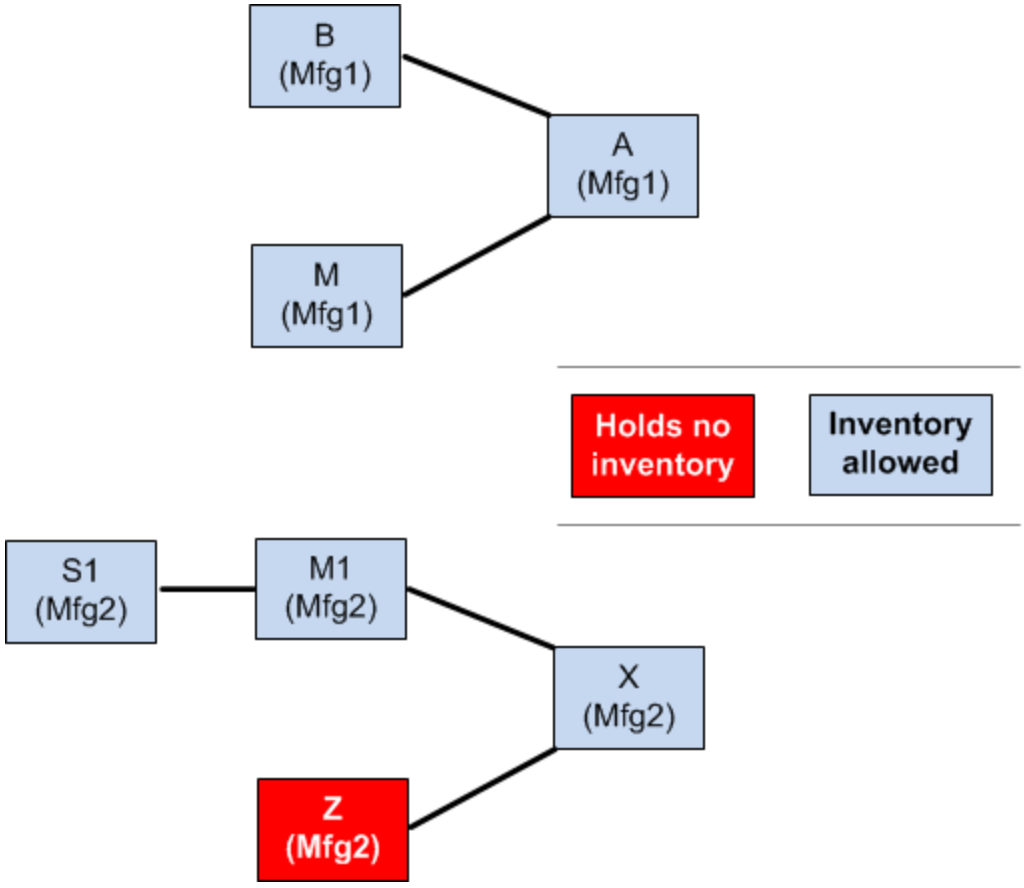
Suppose two multi-echelon safety stock items, A and X, found at separate sites with part sources and **MultiEchelonSafetyStockRule.ProcessingRule** values as shown in the following illustration.

Figure 42.14 BOMs under SafetyStockItems at different sites



To model this situation, two separate families would be created as shown in the following illustration. Note that component S2 is excluded from the family, and Z is included but designated not to hold safety stock.

Figure 42.15 Family result



Details of family construction are noted in the **MEIOStage** table. The following table identifies some pertinent values noted in this table for a family as shown in the previous illustration. Note that the **Real** field is a Boolean used to indicate if the stage is a real stage ("Y") or an artificial stage injected for modeling purposes ("N"). In this case, all reported stages are real stages. Note also that the **Echelon** field identifies the numerical level of the stage in the field (starting at 0 for customer-facing end items). For information about all other fields reported in the MEIOStage table, see "MEIOStage table" on page 1123.

Table 42.24: MEIOStage table values

MEIO Family	Echelon	Part	Source	Stage Type	Real
A	0	A	Mfg1	Customer	Y
A	1	B	Mfg1	Make	Y
A	1	M	Mfg1	Make	Y
X	0	X	Mfg2	Customer	Y
X	1	Z	Mfg2	Make	Y

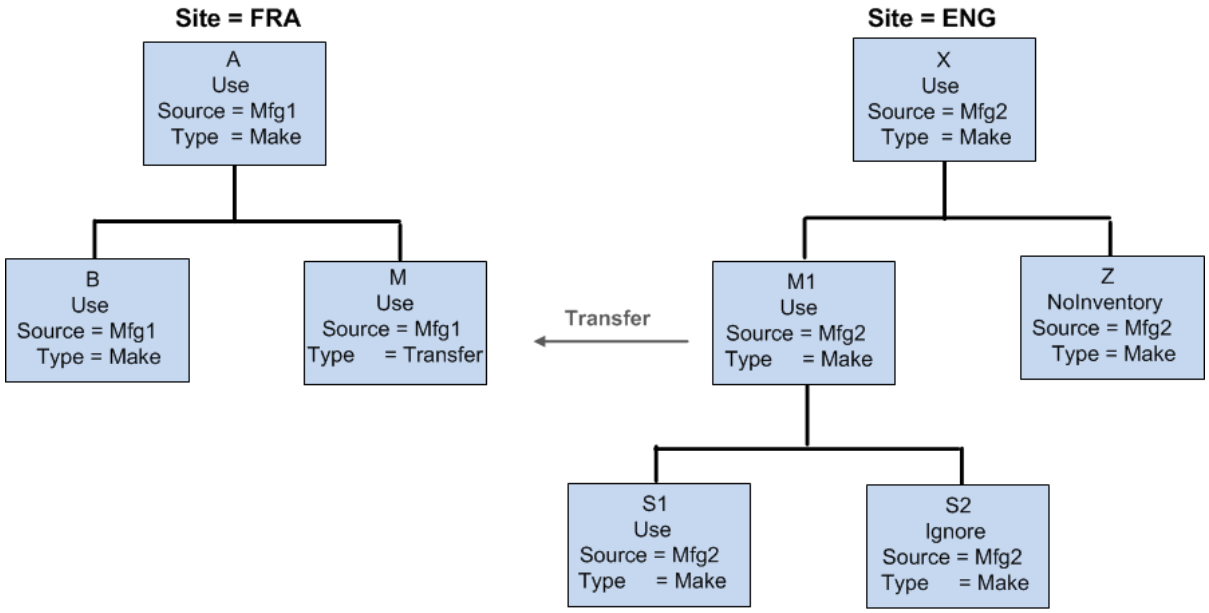
Table 42.24: MEIOStage table values

MEIO Family	Echelon	Part	Source	Stage Type	Real
X	1	M1	Mfg2	Make	Y
X	2	S1	Mfg2	Make	Y

Example 2: Safety stock items with make and transfer sources

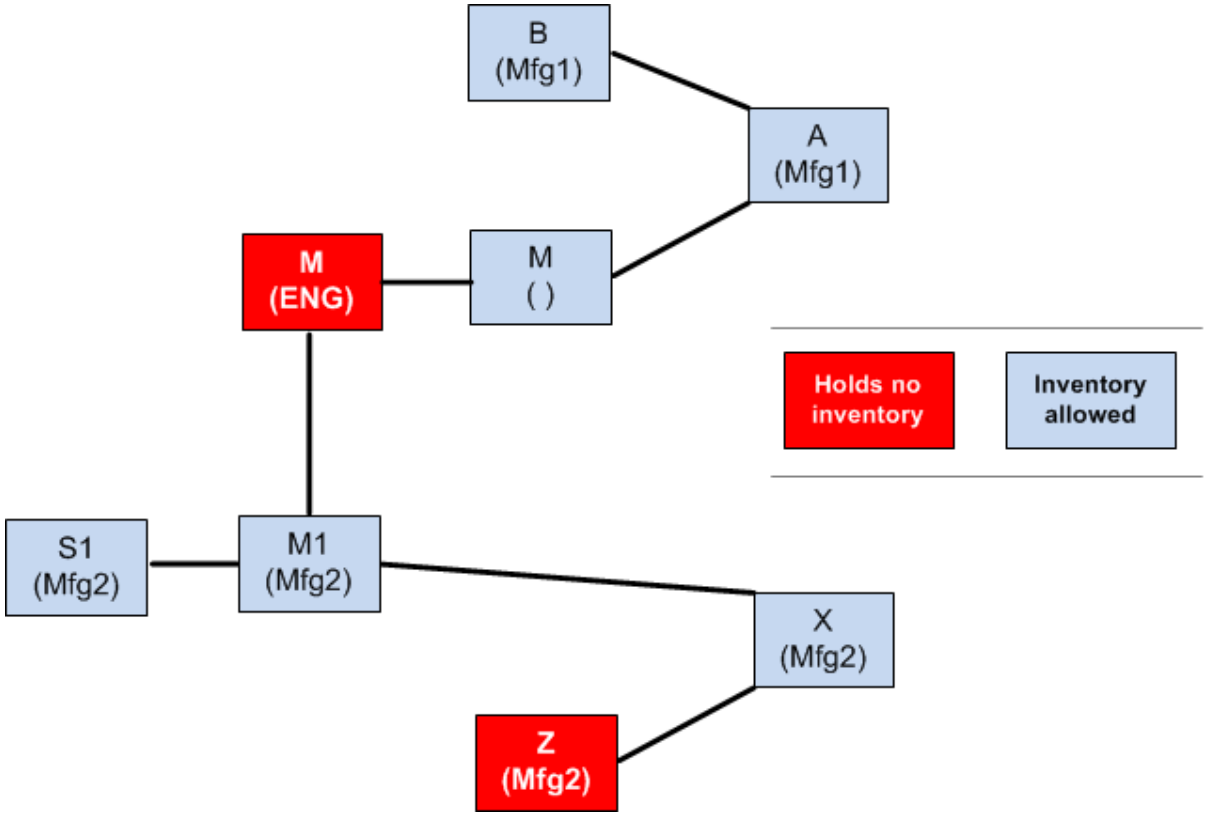
Suppose two multi-echelon safety stock items, A and X, found at separate sites with part sources and **MultiEchelonSafetyStockRule.ProcessingRule** values as shown in the following illustration. Note that this is an almost identical scenario as shown in the previous example, with the exception that part "M" at the "FRA" site is now sourced by way of transfer from the "ENG" site.

Figure 42.16 BOMs under SafetyStockItems at different sites



To model this situation, a single family would be created as shown in the following illustration. This is due to the fact that A and X are now linked by way of a PartSource relationship. Note also that component S2 is excluded from the family, and Z is included but designated not to hold safety stock.

Figure 42.17 Family result



The following table identifies some pertinent values noted in the **MEIOStage** table for a family as shown in the previous illustration.

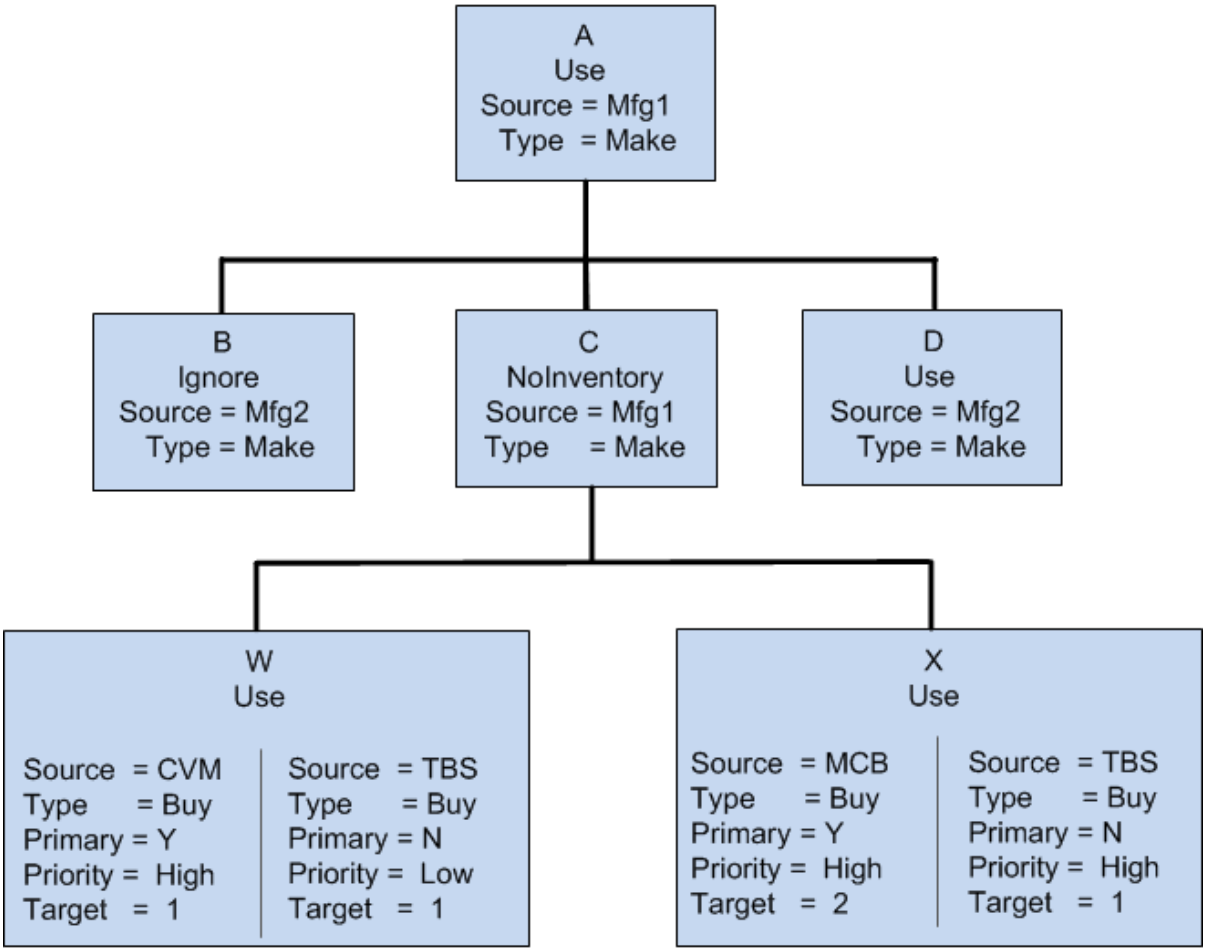
Table 42.25: MEIOStage table values

MEIO Family	Echelon	Part	Source	Stage Type	Real
A	0	A	Mfg1	Customer	Y
A	0	X	Mfg2	Customer	Y
A	1	B	Mfg1	Make	Y
A	1	Z	Mfg2	Make	Y
A	1	M		Transfer	Y
A	1	M1	ENG	Transfer	N
A	2	M1	Mfg2	Make	Y
A	3	S1	Mfg2	Make	Y

Example 3: Safety stock item including multiple “buy” sources

Suppose one multi-echelon safety stock items, A, with some components provided by “make” sources and other components provided by “buy” sources along with **MultiEchelonSafetyStockRule.ProcessingRule** values as shown in the following illustration. Note also that each of the two “buy” components have multiple effective part sources.

Figure 42.18 BOM under a SafetyStockItem

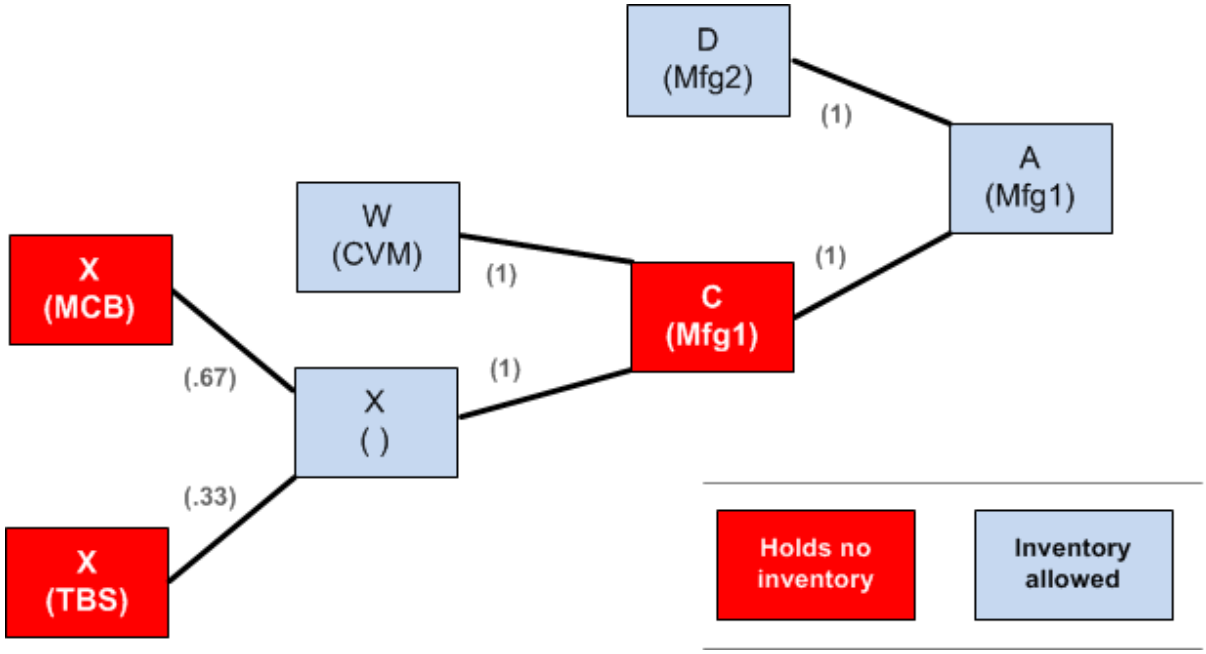


To model this situation, a single family would be created as shown in the following illustration. Note that component B has been excluded from the family, and component C is included but designated not to carry safety stock.

Also, note that for the "buy" component W, only a single stage is contained within the family. This is because its non-primary part source was of a lower priority than the primary. Conversely, component X has a single "real" stage as well as two stages injected to model each of its different part sources. This is because the non-primary part source was of the same priority as the primary. In this example, the **SafetyStockTimePhasedBounds** table contains no records, and **AnalyticsConfiguration**.**AllowSafetyStockOnArtificialStages** is set to True so we can see how calculations work when safety stock is allowed on artificial stages.

The QtyPer values between the stages in the family are also shown. These are based on the QtyPer values shown in the BOM in the preceding figure. Note the artificial stages injected under the real stage for part "X" have QtyPer values set relative to their corresponding **PartSource.Target** values. To see how these QtyPer values are used in cost calculations, refer to "Example 2: Multi-source costs (buy parts)" on page 1998.

Figure 42.19 Family result



The following table identifies some pertinent values noted in the **MEIOStage** table for a family as shown in the previous illustration.

Table 42.26: MEIOStage table values

MEIO Family	Echelon	Part	Source	Stage Type	Real
A	0	A	Mfg1	Customer	Y
A	1	C	Mfg2	Make	Y
A	1	D	Mfg1	Make	Y
A	2	W	CVM	Buy	Y
A	2	X		Buy	Y
A	2	X	MCB	Buy	N
A	2	X	CVM	Buy	N

Calculate inventory holding costs for MEIO

In order to determine optimal safety stock levels, per unit cost of holding inventory for each part needs to be determined. Subsequently when attempting to place required safety stock to minimize the holding cost of inventory, these calculated costs can be used.

To determine per unit cost of holding inventory for a given part, the inventory holding rate for a unit of each part is required. This is expressed as a percentage of the part's calculated cost and is stored in the **Part.InventoryHoldingRate** field. This field should contain values between 0 and 1, and by default it is set to .20 for each part (that is, the default inventory holding rate for each part is 20 percent of its calculated cost). In addition to the inventory holding rate, each part's calculated unit cost is required.

This cost is then multiplied by the inventory holding rate to determine the per unit holding cost of inventory for the part.

The part's unit cost for use in determining inventory holding costs is calculated based on its referenced **MultiEchelonSafetyStockRule.CostRule** value. This control field has the following two options:

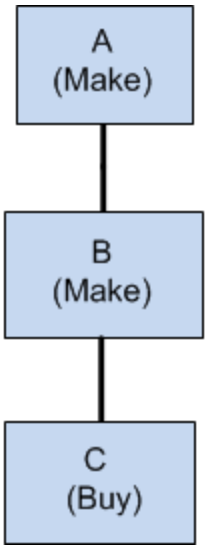
- **RolledUp**—inventory holding costs are based on rolled up cost values from all the part's components using cost values defined on the PartSourceTimePhasedAttributes table (if applicable), or else the PartSource table if time-phased cost values have not been specified. For make and transfer parts, this includes any applicable labor and overhead costs defined on the source. For buy sources, this includes material costs (plus any labor or material costs if defined). As well, if a "buy" or "transfer" part has multiple sources with the same priority as the primary sources, then weighted combinations of those sources' cost values are used in the cost roll up (with the weighting based on Target values). Note also that if using this setting, then rolled-up values might include costs from parts that are not necessarily in the multi-echelon family. For example, if a component has its MultiEchelonSafetyStockRule set to "Ignore", then its cost values will still be included on the assemblies that use it, although a cost value for the component itself will not be calculated.
- **StdUnitCost**—inventory holding rates are calculated from the part's standard unit cost value (**Part.StdUnitCost** field). In other words, per unit inventory holding cost is the product of **StdUnitCost** and **InventoryHoldingRate**.

Once calculated based on the parameters discussed above, the per unit holding cost for each part in a multi-echelon family is reported in the **MEIOStage.UnitHoldingCost** field. The following shows examples of how the **UnitHoldingCost** value is calculated.

Example 1: Single source costs

Suppose a simple BillOfMaterial structure as shown in the following illustration where each part is assumed to have exactly one part source.

Figure 42.20 Simple BOM structure



Further suppose the applicable part and part source costs are as shown in the following table.

Table 42.27: Sample cost values

Part	Part. StdUnitCost	PartSource. LaborCost	PartSource. MaterialCost	PartSource. OverheadCost
A	30	60	0	10
B	15	40	0	0
C	5	0	15	0

If the **Part.InventoryHoldingRate** for each part is .20, then the unit holding cost as would be reported in **MEIOStage.UnitHoldingCost** is as shown in the table below. Note that this example shows the results that would apply based on each of the two **MultiEchelonSafetyStockRule.CostRule** settings:

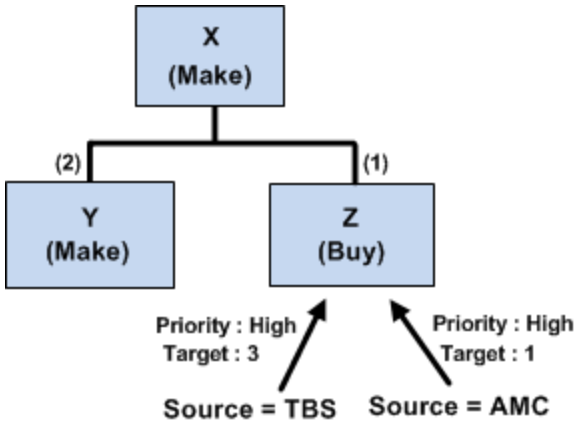
Table 42.28: Sample UnitHoldingCost calculations

Part	CostRule: StdUnitCost	CostRule: RolledUp
A	(30 * .20) = $6	3 + 8+ (10 * .2) + (60 * .2) = 3 + 8 + 2 + 12 = $25
B	(15 * .20) = $3	3 + (40 * .20) = 3 + 8 = $11
C	(5 * .20) = $1	(15 * .20) = $3

Example 2: Multi-source costs (buy parts)

Suppose a simple BillOfMaterial structure as shown in the following illustration. In this case, note that the purchased component "Z" has two part sources, and the primary and non-primary source both have the same priority.

Figure 42.21 Purchased component with two part sources



Further suppose the applicable part and part source costs are as shown in the following table.

Table 42.29: Sample cost values

Part	Source	Part. StdUnitCost	PartSource. LaborCost	PartSource. MaterialCost	PartSource. OverheadCost
X	Mfg	40	75	0	5
Y	Mfg	20	30	0	0
Z	TBS	5	0	14	2
Z	AMC	5	0	13	

If the **Part.InventoryHoldingRate** for each part is .20, then the unit holding cost that would be reported in **MEIOStage.UnitHoldingCost** is as shown in the table below. Note that this example shows the results and calculations that would apply based on a **MultiEchelonSafetyStockRule.CostRule** setting of "RolledUp". Note also that the Target values for the two sources under "buy" component Z are used in determining how each contributes to the overall cost value

Table 42.30: UnitHoldingCost calculation with "RolledUp" rule

Part	RolledUp cost calculation
X	(6 * 2) + 3.05 + (75 + 5) * .20 = 12 + 3.05 + 16 = $31.05
Y	30 * .20 = $6
Z	(14 + 2) * .2 * .75 + 13 * .2 * .25 = 2.4 + .65 = $3.05

Determine demand and supply parameters for MEIO

To generate safety stock recommendations for parts in a multi-echelon family, a number of input and calculated parameters are required.

Demand variability

Multi echelon safety stock calculations rely on historical actual demand data for the customer facing end items set up in the **SafetyStockItem** table. Data from a specified historical demand category is collected and bucketed by the item's **Intervals** calendar for use in the safety stock calculation. This data is automatically adjusted based on any causal factors defined in the **CausalFactorDetail** table and, optionally, further adjusted for outliers as defined in the item's configuration.

The adjusted historical demand data is then used to calculate demand parameters for use in determining safety stock. Specifically, the average historical demand and standard deviation (or forecast error) of those demands is calculated for each end item. Based on configuration options, these demand parameters can be calculated based on either the bucketed demands (using Intervals calendar) or rolling lead time demands, and can be either stationary or time-phased.

Average and standard deviation of demand are also required on all the lower level sub-assemblies and components under a multi-echelon item. For a given part in a multi-echelon family, these parameters are determined using demand parameter values propagated from the part's immediate parent based on the effective QtyPer between the assembly and component.

Service

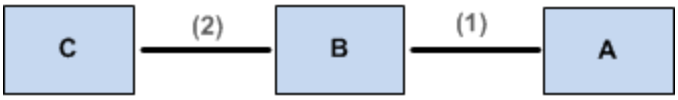
Each multi-echelon safety stock item also requires an input service level to define the measure of customer service that the item's safety stock level should seek to achieve. All parts below a customer-facing end item in a multi-echelon also require a service level. These service levels are calculated based on the multi-echelon item(s) the part ultimately pegs to. If a part only pegs to a single multi-echelon item, then it inherits that item's service level. Otherwise, if a part pegs to two or more multi-echelon items, then its service level is calculated as a weighted combination of those item's service levels (based on the item's standard deviation of demand).

Multi-echelon safety stock items also require a service time which is provided as an input value. This represents a time within which the item guarantees its demands can always be satisfied (thus, in the event of a stock-out, its assumed extraordinary measures will be taken to satisfy the demand). Service times are also applicable to the lower level sub-assemblies and components within the multi-echelon family, however they are calculated values determined by the process that optimizes safety stock levels as described in "Recommend safety stock" on page 2005.

Example 1: Calculating demand parameters (single successor)

Suppose a simple family of parts under multi-echelon safety stock item "A" with QtyPer values as shown in the following illustration.

Figure 42.22 Each part has one successor



Further assume the **SafetyStockItem** record for part "A" includes the following relevant input and control settings:

- **Type.IntervalsCalendar**—Everyday
- **Type.UseRollingLeadTimeDemand**—N
- **Type.StandardDeviationDemandRule**—StandardDeviation
- **ServiceLevel**—0.95

Finally, assume the set of collected historical demands for end items "A" and "X" is as shown in the following table.

Table 42.31: Historical demands for "A"

Date	Demand
May 27, 2013	500
May 28, 2013	525
May 29, 2013	450
May 30, 2013	570
May 31, 2013	520

To determine the required demand parameters, average historical demand for end item “A” is calculated based on its collected historical demands as follows:

$$\frac{500+525+450+570+520}{5} = 513$$

Then once average demand has been calculated, it can be used to determine the standard deviation of historical demand for “A” as follows:

$$\sqrt{\frac{(500-513)^2+(525-513)^2+(450-513)^2+(570-513)^2+(520-513)^2}{4}}$$

$$= \sqrt{\frac{7580}{4}} = \sqrt{1895} = 43.53$$

The historical demand values, as well as the item’s service level, are then propagated to the lower level items in the family as shown in the following table.

Table 42.32: Propagated parameters

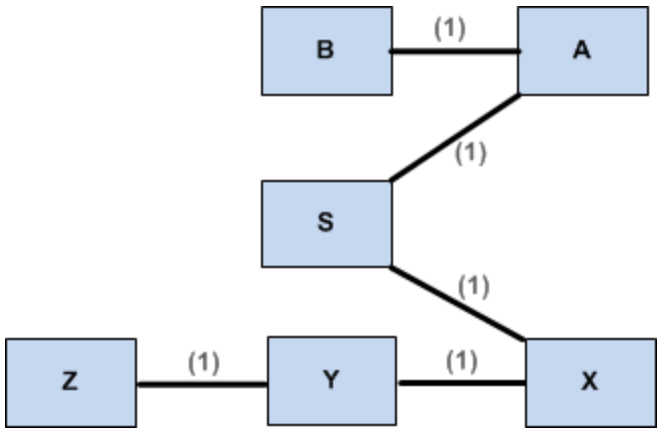
Part	Average Demand	Standard Deviation	Service Level
A	513	43.5316	.95
B	513	43.5316	.95
C	1026	87.0632	.95

For an example of how these parameters are used in making safety stock level recommendations across the family, see "Example 1: Recommend safety stock (single successor)" on page 2006.

Example 2: Calculating demand parameters (multiple successors)

Suppose a family of parts under multi-echelon safety stock items “A” and “X” with QtyPer values as shown in the following illustration.

Figure 42.23 Part has multiple successors in family



Further assume the **SafetyStockItem** record for part "A" has a ServiceLevel of ".95" and the **SafetyStockItem** record for "X" has a ServiceLevel of ".90", with each referencing the same **SafetyStockItemType** record including the following settings:

- **Type.IntervalsCalendar**— Everyday
- **Type.StandardDeviationDemandRule**—StandardDeviation
- **Type.UseRollingLeadTimeDemand**— N

Finally, assume the set of collected historical demands for end items "A" and "X" is as shown in the following table.

Table 42.33: Historical demands for end items

Date	Demand for X	Demand for A
May 27, 2013	80	50
May 28, 2013	100	60
May 29, 2013	60	65
May 30, 2013	85	50
May 31, 2013	90	75

To determine the required demand parameters, average historical demand for end item "X" is calculated based on its collected historical demands as follows:

$$\frac{80+100+60+85+90}{5} = 83$$

The standard deviation of demand for "X" can be calculated as follows.

$$\sqrt{\frac{(80-83)^2+(100-83)^2+(60-83)^2+(85-83)^2+(90-83)^2}{4}}$$

$$= \sqrt{\frac{880}{4}} = \sqrt{220} = 14.8324$$

Similarly, to determine the required demand parameters, average historical demand for end item "A" is calculated based on its collected historical demands as follows:

$$\frac{50+60+65+50+75}{5} = 60$$

The standard deviation of demand for "A" can then be calculated as follows.

$$\sqrt{\frac{(50-60)^2+(60-60)^2+(65-60)^2+(50-60)^2+(75-60)^2}{4}}$$

$$= \sqrt{\frac{450}{4}} = \sqrt{112.5} = 10.6066$$

The historical demand values as well as the service levels from the two end items are then propagated to the lower level items in the family as shown in the following table.

Table 42.34: Propagated parameters

Part	Average Demand	Standard Deviation	Service Level
A	60	10.6066	.95
B	60	10.6066	.95
S	143	18.2346	.9215
X	83	14.8324	.90
Y	83	14.8324	.90
Z	83	14.8324	.90

Note that the demand parameters and service level calculated for part "S" reflect parameter values passed down from both "A" and "X". To see an example of how these weighted parameters are used in calculating safety stock, see "Example 2: Recommend safety stock (multiple successors)" on page 2007.

Lead time

Each part in a multi-echelon family also requires a lead time value for use in determining safety stock.

By default, this value is assumed to be known and constant. For each part (and source), lead time for use in multi-echelon calculations is determined and expressed in terms of the Gregorian (everyday) calendar as follows:

```
PartSource.LeadTimeDate - RunDate Everyday
```

Optionally, one or more parts within a multi-echelon family can be configured as having a variable lead time distribution. To model variable lead time for a part used in multi-echelon calculations, a series of input and control table settings should be configured as described in "Configure multi-echelon lead time parameters (optional)" on page 1960.

When variability of lead time is enabled for a part, lead time parameters representing the calculated average of historical supply lead time and standard deviation of historical supply lead time are calculated for the part. These parameters can be calculated based on historical supply data, set to input values, or determined from discrete lead time distribution profiles. In each case, the lead time parameters are then used in the calculations that generate recommended safety stock levels as described in "Safety stock for parts with lead time variability" on page 2008.

Example 1: Calculating lead time parameters (normal distribution)

Suppose a series of **HistoricalSupplyActual** records with values as shown in the following table:

Table 42.35: HistoricalSupplyActual values and effective lead time.

Part	Source	Quantity	Date	OrderDate	LeadTime	Category	Effective lead time
XYZ	Mfg	300	04-09-14		6	Shipment	6
XYZ	Mfg	500	04-29-14	04-20-14	-1	Shipment	9
XYZ	Mfg	2000	05-12-14	05-07-14	-1	Shipment	5
XYZ	Mfg	500	05-22-14	05-18-14	8	Shipment	4
XYZ	Mfg	600	05-29-14	05-24-14	-1	Shipment	5

Note: The **LeadTime** values stated in the **HistoricalSupplyActual** table are ignored in all but the first case because both the **Date** and **OrderDate** values are provided, allowing the historical effective lead time to be calculated from those dates.

Further suppose an **MEIOLeadTime** record is added for part XYZ that includes the following settings:

- **PartSource.Part.Name**—XYZ
- **PartSource.Source.Id**—Mfg
- **Type.Category**—Shipment
- **Type.ProbabilityDistribution**—Normal
- **Type.StandardDeviationRule**—Calculate

In this case, the average historical supply lead time for the part and source (as reported in **MEIOLeadTime.CalcAverage**) would be calculated as:

$$\frac{6+9+5+4+5}{5} = 5.8$$

And, the standard deviation of historical supply lead time for the part and source (as reported in **MEIOLeadTime.CalcStandardDeviation**) would be calculated as:

$$\sqrt{\frac{(6-5.8)^2+(9-5.8)^2+(5-5.8)^2+(4-5.8)^2+(5-5.8)^2}{4}}$$

$$= \sqrt{\frac{14.8}{4}} = \sqrt{3.7} = 1.92$$

Example 2: Calculating lead time parameters (discrete distribution)

Suppose a series of **MEIOLeadTimeDistributionProfilePoint** records with values as shown in the following table:

Table 42.36: MEIOLeadTImeDistribution ProfilePoint values

Profile Name	ValueS	Weight
Profile1	10	3
Profile1	7	6
Profile1	5	1

Further suppose an **MEIOLeadTime** record is added for part XYZ that includes the following settings:

- **PartSource.Part.Name**—XYZ
- **PartSource.Source.Id**—Mfg
- **DistributionProfile.Name**—Profile1
- **Type.ProbabilityDistribution**—Discrete
- **Type.DistributionProfileRule**—LeadTime

In this case, the average historical supply lead time for the part and source (as reported in **MEIOLeadTime.CalcAverage**) would be calculated as:

$$\frac{(10\times3)+(7\times6)+(5\times1)}{(3+6+1)} = \left(77/10\right) = 7.7$$

And, the standard deviation of historical supply lead time for the part and source (as reported in **MEIOLeadTime.CalcStandardDeviation**) would be calculated as:

$$\sqrt{\frac{\left((10-7.7)^2\times3\right)+\left((7-7.7)^2\times6\right)+\left((5-7.7)^2\times1\right)}{10}}$$

$$\sqrt{\frac{26.1}{10}} = \sqrt{2.61} = 1.62$$

Recommend safety stock

Multi-echelon safety stock calculations determine safety stock levels so as to ensure demands can be satisfied within each item's guaranteed service time while minimizing the inventory holding cost required to do so. The actual calculations depend on whether a given part in a multi-echelon family has lead times that are fixed or variable.

Note: If the specified ServiceTime for a SafetyStockItem exceeds the cumulative lead time on the part, then multi-ecehlon safety stock logic will not recommend any safety stock for the part. This combination of input parameters is assumed to represent a "make-to-order" item where product availability is not expected any quicker than the duration of the production/delivery cycle.

Safety stock for parts with fixed lead time

Safety stock is only recommended within the net lead time for a part with net lead time being defined as follows:

$$\mathbf{NLT} = SI + T - S$$

where:

- SI is the item's incoming service time which indicates the maximum time within which all of the items components can be ready. It is set to the maximum quoted service time found on all of the item's immediate children in the family.
- T is the fixed lead time for the item (includes all processing, transportation, and waiting time). Calculated from the item's part source as `PartSource.LeadTimeDate - RunDate Everyday`.
- S is the applicable service time for the item representing the maximum time within which its orders are guaranteed to be ready. Service time is provided as input on the customer-facing end-items that are configured for multi-echelon calculations, and is the key decision variable calculated for all other parts in a multi-echelon family.

Multi-echelon safety stock calculations rely on the assumption that external demand for an end item is bounded over a time horizon. That is, safety stock for the part can be planned to account for demand up to a maximum threshold as given by the following formula (any demand beyond this threshold is assumed to require other measures, such as expediting or sub-contracting):

$$\mathrm{D(t)} = \mu D * NLT + Z\alpha * \sigma D * \sqrt{NLT}$$

where:

- NLT represents the maximum replenishment or net lead time for the part.
- μD is the average historical demand.
- Z_{α} is a normal distribution z-value determined from the applicable service level.
- σD is the standard deviation of historical demand.

The level of safety stock that should ideally be maintained in inventory for a particular multi-echelon item is then given by the following formula:

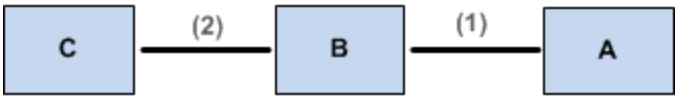
$$\mathrm{SafetyStock} = Z\alpha * \sigma D * \sqrt{SI + T - S}$$

Based on the formula shown above, RapidResponse make use of iterative optimization methods to evaluate and determine a recommended service time for each item in the multi-echelon family (excluding the safety stock items themselves on which the service time is given). Service times are optimized so as to minimize the total expected holding cost of safety stock across all parts in the network. Thus, once the optimal service times have been determined, they can be used in the formula shown above to calculate the safety stock level, if any, to maintain for each part.

Example 1: Recommend safety stock (single successor)

Suppose a simple family of parts under multi-echelon safety stock item "A" with QtyPer values as shown in the following illustration.

Figure 42.24 Parts in simple multi-echelon family



Further suppose the following set of demand and supply parameters apply to the parts in this multi-echelon family.

Table 42.37: Safety stock parameters

Part	Average Demand	Standard Deviation	Service Level	Lead Time	UnitHoldingCost
A	513	43.5316	.95	2	20
B	513	43.5316	.95	3	10
C	1026	87.0632	.95	7	2

If the input **ServiceTime** for A is specified as 0, then the optimal service time as determined from iterative methods is 3 for part B and 0 for part C. Thus, safety stock for each item in the family is calculated as shown in the formulas below with the cost of holding this safety stock in inventory being $3960.31.

$$\mathrm{SS(A)} = 1.645 * 43.5316 * \sqrt{3 + 2 - 0} = 71.609 * \sqrt{5} = 160.12 \text{ units}$$

$$\mathrm{SS(B)} = 1.645 * 43.5316 * \sqrt{0 + 3 - 3} = 71.609 * \sqrt{0} = 0 \, units$$

$$\mathrm{SS(C)} = 1.645 * 87.0632 * \sqrt{0 + 7 - 0} = 143.219 * \sqrt{7} = 378.89 \, units$$

Instead, suppose the input **ServiceTime** for A is modified to 2. In this case, the optimal service time as determined from iterative methods in RapidResponse is 0 for both part B and 0 for part C. Thus, safety stock for each item in the family is calculated as shown in the formulas below with total cost of holding this safety stock in inventory now being $1997.98. Note that safety stock is no longer held on the more expensive end-item A, and is instead moved to component B, as the service guarantee for A can now be met within lead time.

$$\mathrm{SS(A)} = 1.645 * 43.5316 * \sqrt{0 + 2 - 2} = 71.609 * \sqrt{0} = 0 \text{ units}$$

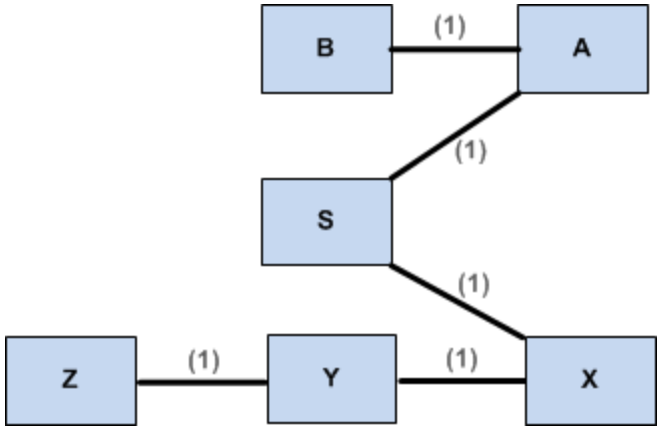
$$\mathrm{SS(B)} = 1.645 * 43.5316 * \sqrt{0 + 3 - 0} = 71.609 * \sqrt{3} = 124.02 \, units$$

$$\mathrm{SS(C)} = 1.645 * 87.0632 * \sqrt{0 + 7 - 0} = 143.219 * \sqrt{7} = 378.89 \, units$$

Example 2: Recommend safety stock (multiple successors)

Suppose a family of parts under multi-echelon safety stock items "A" and "X" with QtyPer values as shown in the following illustration.

Figure 42.25 Part has multiple successors in family



Further suppose the following set of demand and supply parameters apply to the parts in this multi-echelon family.

Table 42.38: Safety stock parameters

Part	Average Demand	Standard Deviation	Service Level	Lead Time	UnitHoldingCost
A	60	10.6066	.95	2	30
B	60	10.6066	.95	1	14
X	83	14.8324	.90	2	20
Y	83	14.8324	.90	3	12
Z	83	14.8324	.90	7	4
S	143	18.2346	.9215	4	10

If the input service times provided are:

- A = 0 days
- X = 3 days

Then RapidResponse determines the optimal service times for the other parts in the family to be:

- B = 1 day
- Y= 1 day
- Z = 0 days
- S = 1 day

Thus, safety stock for each item in the family is calculated as shown in the formulas below with the total cost of holding this safety stock in inventory being $1877.18.

$$\mathbf{SS(A)} = \mathbf{1.645} * \mathbf{10.6066} * \sqrt{\mathbf{1} + \mathbf{2} - \mathbf{0}} = \mathbf{17.447} * \sqrt{\mathbf{3}} = \mathbf{30.22\ units}$$

$$\mathbf{SS(B)} = \mathbf{1.645} * \mathbf{10.6066} * \sqrt{\mathbf{0} + \mathbf{1} - \mathbf{1}} = \mathbf{17.447} * \sqrt{\mathbf{0}} = \mathbf{0}\,units$$

$$\mathbf{SS(X)} = \mathbf{1.282} * \mathbf{14.8324} * \sqrt{\mathbf{1} + \mathbf{2} - \mathbf{3}} = \mathbf{19.015} * \sqrt{\mathbf{0}} = \mathbf{0}\,units$$

$$\mathbf{SS(Y)} = \mathbf{1.282} * \mathbf{14.8324} * \sqrt{\mathbf{0} + \mathbf{3} - \mathbf{1}} = \mathbf{19.015} * \sqrt{\mathbf{2}} = \mathbf{26.89}\,units$$

$$\mathbf{SS(Z)} = \mathbf{1.282} * \mathbf{14.8324} * \sqrt{\mathbf{0} + \mathbf{7} - \mathbf{0}} = \mathbf{19.015} * \sqrt{\mathbf{7}} = \mathbf{50.29}\,units$$

$$\mathbf{SS(S)} = \sqrt{(\mathbf{1.282} * \mathbf{14.8324})^2 + (\mathbf{1.645} * \mathbf{10.6066})^2} * \sqrt{\mathbf{0} + \mathbf{4} - \mathbf{1}}$$

$$= \sqrt{\mathbf{666}} * \sqrt{\mathbf{3}} = \mathbf{44.69}\,units$$

Safety stock for parts with lead time variability

Safety stock is calculated within the average net lead time for a part with net lead time being defined as follows:

$$\mathbf{NetLeadTime} = SI + \mu L - S$$

where:

- SI is the item's incoming service time which indicates the maximum time within which all of the items components can be ready. It is set to the maximum quoted service time found on all of the item's immediate children in the family.
- μL is the average lead time calculated for supplies from the part source (calculated based on actual historical supply data, provided input values, or discrete distribution profiles).
- S is the applicable service time for the item representing the maximum time within which its orders are guaranteed to be ready. Service time is provided as input on the customer-facing end-items that are configured for multi-echelon calculations, and is the key decision variable calculated for all other parts in a multi-echelon family.

The level of safety stock that should ideally be maintained in inventory for a particular multi-echelon item subject to lead time variability is then given by the following formula:

$$\mathrm{SafetyStock(Si, S)} = Z_{\alpha} * \sqrt{Q\left(S - Si\right) * \left(\sigma D^{2}\right) + \left(\mu D^{2}\right) * R\left(S - Si\right)}$$

where:

- Z_{α} is a normal distribution z-value determined from the applicable service level.
- Q is a function returning the average of net lead time when it is positive. For details about this function, see "Formulas for positive net lead time" on page 2009.
- σD is the standard deviation of historical demand.
- μD is the average of historical demand.
- R is the variance of the positive part of the net lead time random variable. In other words, R is the variance of net lead time conditional on it being positive. For details about this function, see "Formulas for positive net lead time" on page 2009.

Note that when lead time is assumed to be variable or random, it is possible to realize negative net lead time at a given stage in the model. This means the stage will be seen as having replenished more orders than it has actually shipped, and occurs when the function $Q(S - Si)$ is greater than $Si + \mu L - S$ (in other words, the average of net lead time when it is positive is strictly greater than the average of net lead time). This leads to the possibility of early arrival stock as calculated by the following equation.

$$EarlyArrival\mathrm{Stock(SI, S)} = \mu_{D}\left[Q\left(S - SI\right) - \left(SI + \mu L - S\right)\right]$$

The sum of safety stock and early arrival stock at a given stage represents its total average inventory as shown below. Thus, when lead time variability is assumed, multi-echelon safety stock calculations attempt to minimize the cost of holding this total inventory.

$$TotalInventory = SafetyStock\left(SI, S\right) + \mathrm{EarlyArrivalStock(SI, S)}$$

Formulas for positive net lead time

As outlined in the previous section, the calculation of safety stock with lead time variability relies on two functions, Q and R, which are further described below and dependent on whether lead time is assumed to follow a normal or discrete distribution (based on the setting in **MEIOLeadTime.ProbabilityDistribution**).

Q is the average of the positive part of net lead time. In other words, this function gives the mean of net lead time conditional on it being positive:

$$Q\left(T\right) = H^{2}\left(T\right) - T\left(1 - H^{1}\left(T\right)\right)$$

R is the average of the positive part of net lead time. In other words, this function gives the variance of net lead time conditional on it being positive:

$$R\left(T\right) = T^{2}H^{1}\left(T\right)\left(1 - H^{1}\left(T\right)\right) - 2TH^{1}\left(T\right)H^{2}\left(T\right) + H^{3}\left(T\right) - \left(H^{2}\left(T\right)\right)^{2}$$

where:

- H^{1} is the probability that the lead time value is at most T: $H^{1}\left(T\right) := P\left(L <= T\right)$
- H^{2} is the conditional average lead time value: $H^{2}\left(T\right) := E\left(L \mid L > T\right)$

- H^3 is the difference between the variance of the lead time squared and the mean squared:

$$H^3\left(T\right) := Var\left(L \mid L > T\right) - \mu_L^2 = E\left(L^2\right)$$

And if lead time has a discrete distribution, then:

$$H^1\left(T\right) = \sum_{l=0}^{T} P(L = l)$$

$$H^2\left(T\right) = \sum_{l=\mathrm{T}+1}^{\infty} lP(L = l)$$

$$H^3\left(T\right) = \sum_{l=\mathrm{T}+1}^{\infty} l^2 P\left(L = l\right)$$

Otherwise, if lead time has a normal distribution, then

$$H^1\left(T\right) = \Phi\left(\frac{T-\mu L}{\sigma L}\right) - \Phi\left(\frac{0-\mu L}{\sigma L}\right)$$

$$H^2\left(T\right) = \mu L\left(1 - \Phi\left(\frac{T-\mu L}{\sigma L}\right)\right) + \sigma L\phi\left(\frac{T-\mu L}{\sigma L}\right)$$

$$H^3\left(T\right) = \left(\sigma_L^2 + \mu_L^2\right)\left(1 - \Phi\left(\frac{T-\mu L}{\sigma L}\right)\right) + \left(\sigma_L^2 \frac{T-\mu L}{\sigma L} + 2\sigma L\mu L\right)\phi\left(\frac{T-\mu L}{\sigma L}\right)$$

where:

- Φ is the cumulative distribution function of the standard normal distribution.
- ϕ is the probability density function of the standard normal distribution.

Apply safety stock bounds in MEIO

When evaluating the total cost of an iteration, the multi-echelon safety stock analytic penalizes any iteration in which safety stock bounds are violated. The sum of the absolute values of all safety stock bound violations in an iteration is multiplied by a large constant and added to the cost of that iteration.

Placing a high cost on safety stock violations ensures that iterations in which safety stock bounds are violated are assigned a higher cost than iterations without safety stock bounds violations. If all iterations have safety stock bounds violations, the ones with the smallest violations are assigned the lowest cost. Results from the lowest cost iteration are reported in the **UnboundedSafetyStock** fields in the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables.

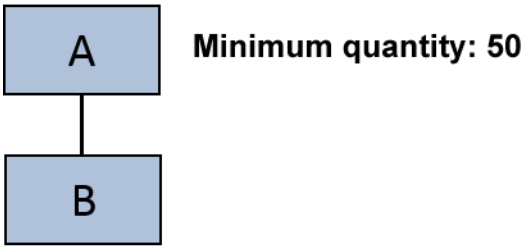
If an unbounded safety stock value does not violate safety stock bounds, it is copied into the **SafetyStock** field.

If an unbounded safety stock value violates a safety stock bound, the bound that was violated is copied into the **SafetyStock** field instead. For example, if the unbounded safety stock value is 112 and a maximum quantity bound of 100 applies, **SafetyStock** is 100.

Example 1: Apply safety stock bounds (solution exists)

Consider the simplest possible MEIO family, which has two parts: A customer-facing part, A, which has one part, B, in its bill of materials. Part A has a higher unit holding cost than part B.

Figure 42.26 Simple MEIO family with an inventory bound



A minimum safety stock quantity bound of 50 applies to part A, and part B has no safety stock bounds. Four iterations of the RapidResponse MEIO analytic have been run:

Table 42.39: Iterations of the MEIO Analytic for an MEIO family with one safety stock bound

Iteration	Safety Stock Quantity		Inventory Holding	Bounds violations	Rank
	Part A	Part B	Cost	(absolute value)	
First	45	45	$3330	5	4
Second	48	42	$3444	2	3
Third	51	38	$3540	0	1
Fourth	54	34	$3636	0	2

In the first iteration, safety stock for A is 45, which is five less than the minimum safety stock bound for this part, so the absolute value of the safety stock bounds violations is five. The safety stock constant is several orders of magnitude larger than the actual inventory holding cost, so adding a penalty of five times the safety stock constant to this iteration's overall cost makes it the highest cost iteration, rather than the lowest cost iteration.

The second iteration also has safety stock bounds violations, so it ranks below any iterations without bounds violations, but above the first iteration, which violates the safety stock bounds by a larger amount.

The third and fourth iterations do not have any safety stock bounds violations, so they are not penalized. The third iteration has the lowest inventory holding cost of the two solutions without safety stock violations, so the safety stock numbers from the third iteration are reported in the **UnboundedSafetyStock** fields of the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables.

Because RapidResponse was able to find a solution that satisfied the service level for A and did not violate safety stock bounds, the **UnboundedSafetyStock** values are copied into the **SafetyStock** fields.

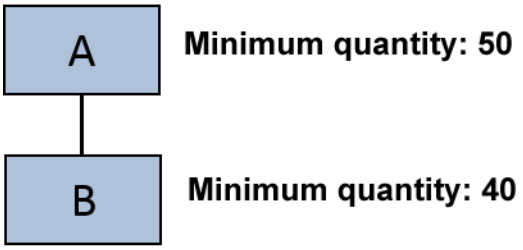
RapidResponse recommends a safety stock level of 51 for A and 38 for B.

The **CalculatedServiceLevel** value matches the **ServiceLevel** value because a solution was found that satisfies service level requirements without violating safety stock bounds.

Example 2: Apply safety stock bounds (solution does not exist)

Consider the same MEIO family as in the previous example. Part A still has a minimum safety stock quantity of 50, and now Part B has a minimum safety stock quantity of 40.

Figure 42.27 Simple MEIO family with two inventory bounds



Four iterations of the MEIO analytic have been run:

Table 42.40: Iterations of the MEIO Analytic for an MEIO family with two safety stock bounds

Iteration	Safety Stock Quantity		Inventory Holding	Bounds violations	Rank
	Part A	Part B	Cost	(absolute value)	
First	45	45	$3330	5 + 0 = **5**	3
Second	48	42	$3444	2 + 0 = **2**	1
Third	51	38	$3540	0 + 2 = **2**	2
Fourth	54	34	$3636	0 + 6 = **6**	4

All of the iterations have safety stock bounds violations. Because a solution could not be found that satisfies the service level without violating safety stock bounds, all of the iterations are penalized for bounds violations.

The second and third iterations have the lowest safety stock bounds violations. Because they both have the same bounds violation (two), the same safety stock bounds violation penalty is applied to both. The second iteration is ranked higher since it has the lower inventory holding cost, so the safety stock numbers from the second iteration are reported in the **UnboundedSafetyStock** fields of the **SafetyStockResult** and **SafetyStockTimePhasedResult** tables.

Because the value in **UnboundedSafetyStock** for A violates an inventory bound, the bound that was violated is copied into the **SafetyStock** field instead. The **UnboundedSafetyStock** value for B does not violate any safety stock bounds, so it is copied into the **SafetyStock** field.

RapidResponse recommends a safety stock level of 50 for A and 42 for B.

With the safety stock bounds applied, the service level specified in the **SafetyStockItem** table for part A, the head part in this MEIO family, is not satisfied. An estimate of the service level that can be expected with the safety stock bounds applied is reported in the **CalculatedServiceLevel** field.

Automated safety stock updates

As described earlier in this chapter, safety stock recommendations and related calculations from both single-echelon inventory planning and multi-echelon inventory optimization are consolidated in the calculated **SafetyStockResult** and **SafetyStockTimePhasedResult** tables (with supporting details contained in many other tables).

Note that the recommended safety stock values are not automatically used by planning analytics such as Netting. However, if deemed acceptable after review, the recommended values can be copied over the existing input values that were provided in either the **Part.SafetyStockQty** field (for fixed safety stock) or

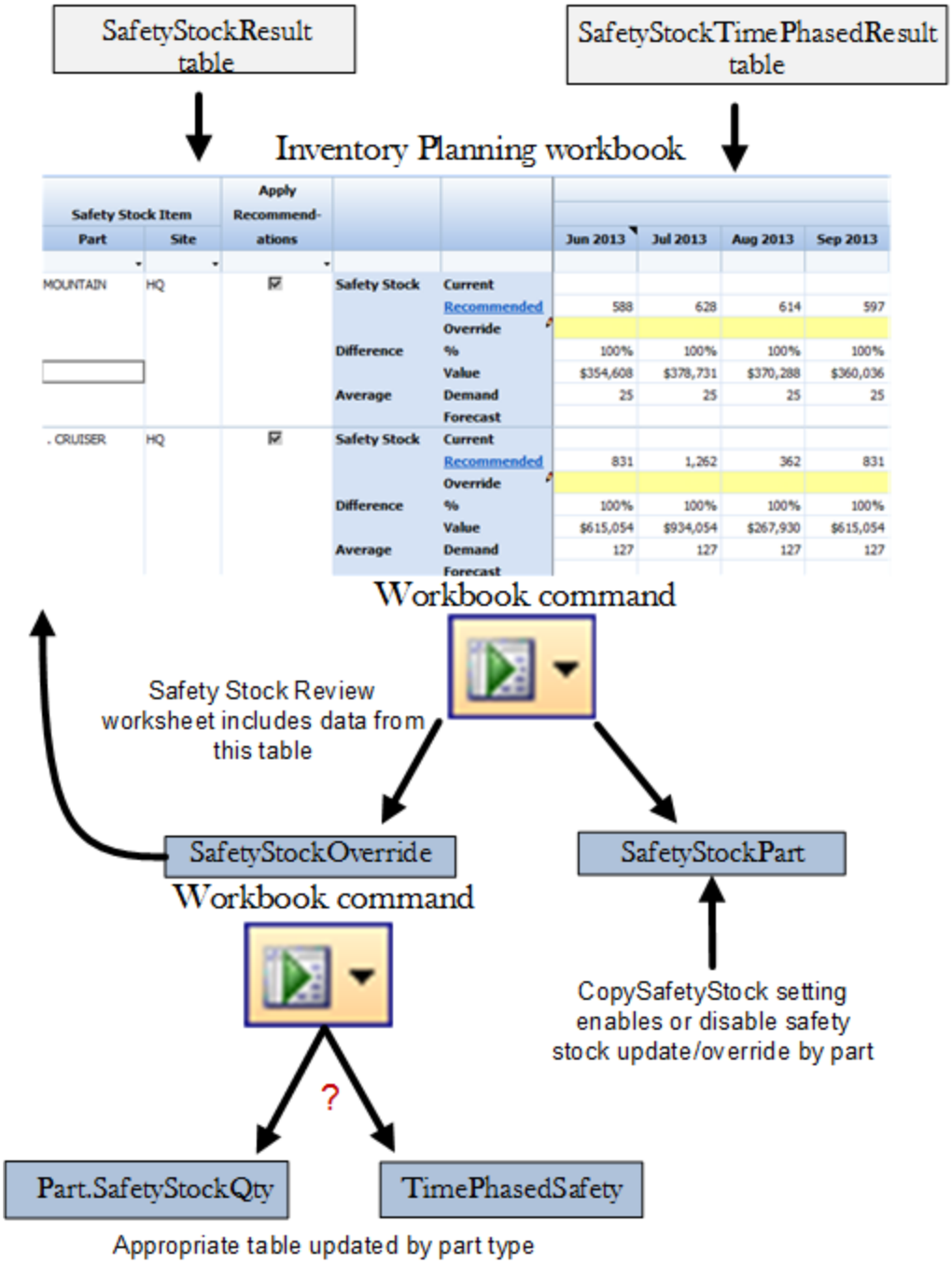
the **TimePhasedSafety** table (for time-phased safety stock). This can be done manually, or using workbook data modifications commands such as those included with the Inventory Planning workbook included with RapidResponse.

The data modification commands in the Inventory Planning workbook make use of two special input tables: **SafetyStockOverride** and **SafetyStockPart**.

The **SafetyStockOverride** table is used by a data modification command that copies the results of both single and multi-echelon safety stock calculations into the table. This preserves the underlying calculated results in a single table and allows them to be accessed without rerunning the analytic or function. If necessary, this table also enables manual overrides to be made against one or more of the recommended safety stock quantities for each item. The **SafetyStockPart** table contains records for all parts that are either configured as single-echelon safety stock items, configured as multi-echelon safety stock items, or are lower-level parts brought into a family under a multi-echelon item. The **CopySafetyStock** field on this table is used to set whether the workbook's data modification commands will replace the relevant input safety stock value(s) with the recommended ones. Note that the **SafetyStockPart** table also contains copies of many calculated parameters and other details pertinent to multi-echelon calculations.

The following illustration shows the use of the Inventory Planning workbook to automatically update input safety stock values with those recommendations generated by single and multi-echelon calculations.

Figure 42.28 Data modification commands update existing input values



Comparing single-echelon settings and multi-echelon settings

The **SafetyStockItem** and **SafetyStockItemType** settings discussed in this chapter might apply only to SEIP, only to MEIO, or both. Additionally, in MEIO, some **SafetyStockItem** and **SafetyStockItemType**

settings are family settings. Family settings are settings taken from the head part and are applied to every part of the family. This section summarizes which **SafetyStockItem** and **SafetyStockItemType** settings apply to SEIP, MEIO, or both, as well as which **SafetyStockItem** and **SafetyStockItemType** settings are family settings.

SafetyStockItem settings

The following table indicates whether each setting on the **SafetyStockItem** table applies to SEIP, MEIO, or both, and whether each setting is a family setting.

Table 42.41: SafetyStockItem settings

Field name	SEIP	MEIO	Family setting
AverageDemand	•	•	
DemandOutlierMovingAverageWindow	•	•	
DemandOutlierThreshold	•	•	
DemandOutlierType	•	•	
DemandSmoothingAfterIntervalCount	•	•	
DemandSmoothingBeforeIntervalCount	•	•	
ForecastErrorOutlierThreshold	•	•	
ForecastLag	•	•	
FutureAverageDemandProfile	•		
FutureIntervalCount	•	•	
HistoricalAverageDemandProfile	•		
HistoricalDemandCategory	•	•	
HistoricalEndDate	•	•	
HistoricalForecastCategory	•	•	
HistoricalIntervalCount	•	•	
HistoricalSkipLeadingZeros	•	•	
HistoricalStartDate	•	•	
HistoricalSupplyCategory	•	•	
LeadTime	•	**See Note (1) below.**	
LeadTimeOutlierMovingAverageWindow	•	**See Note (2) below.**	
LeadTimeOutlierThreshold	•		
LeadTimeOutlierType	•		
LeadTimeSmoothingAfterIntervalCount	•		
LeadTimeSmoothingBeforeIntervalCount	•		
OrderQuantity	•		
Part	•	•	

Table 42.41: SafetyStockItem settings

Field name	SEIP	MEIO	Family setting
ProcessingRule	•	•	
RemoveDemandOutliers	•	•	
ServiceLevel	•	•	
ServiceTime		•	
StandardDeviationDemand	•	•	
StandardDeviationLeadTime	•		
TimePhasedReportCount	•	•	•
Type	•	•	

Notes:

- **(1)** Ignored in MEIO except when UseRollingLeadTimeDemand = Y.
- **(2)** The LeadTime fields (**LeadTimeOutlierMovingAverageWindow**, **LeadTimeOutlierThreshold**, **LeadTimeOutlierType**, **LeadTimeSmoothingAfterIntervalCount**, and **LeadTimeSmoothingBeforeIntervalCount**) in this table are used only for SEIP. For MEIO, use the same fields on the **MEIOLeadTime** table.

SafetyStockItemType settings

The following table indicates whether each setting on the **SafetyStockItemType** table applies to SEIP, MEIO, or both, and whether each setting is a family setting.

Table 42.42: SafetyStockItemType settings

Field name	SEIP	MEIO	Family setting
AsOfDateCalendar	•	•	•
AverageDemandRule	•	•	
BoundsRule		•	•
CycleCalendar	•	•	•
DemandOutlierProcessingRule	•	•	
ExcludePastDueDemands	•	•	
ForecastBiasAdjustmentRule	•	•	
ForecastErrorOutlierProcessingRule	•	•	
IntervalsCalendar	•	•	**See Note (1) below.**
IntervalsPerPeriod	•	•	•
LeadTimeCalendar	•		
LeadTimePerInterval	•		

Table 42.42: SafetyStockItemType settings

Field name	SEIP	MEIO	Family setting
MEIODemandHistoricalsPropogationRule		•	•
NonStationaryDemandRule	•	•	
PeriodCalendar	•	•	•
PeriodsPerCycle	•	•	•
ServiceLevelRule	•		
StandardDeviationDemandRule	•	•	**See Note (2) below.**
SupplyVariabilityRule	•		
TimePhasedProcessingRule	•	•	•
UseRollingLeadTimeDemand	•	• **See Note (3) below.**	

Notes:

- **(1)** Family setting enforced when family StandardDeviationDemandRule = ForecastError and MEIODemandHistoricalsPropogationRule = Use.
- **(2)** Family setting enforced when StandardDeviationDemandRule = ForecastError and MEIODemandHistoricalsPropogationRule = Use.
- **(3)** UseRollingLeadTimeDemand is used in both SEIP and MEIO; however, UseRollingLeadTimeDemand= 'Y' is not recommended in MEIO.

CHAPTER 43: Statistical forecasting

An important aspect of demand planning is transforming historical demand data into actionable predictions of future demand, or generating the statistical forecast. RapidResponse supports statistical forecasting, allowing demand planners to adjust historical actuals in the case of data errors or anomalies, select the forecast items, and define the forecast parameters. The benefits of statistical forecasting extend beyond demand planning, as sales, marketing, or finance users can utilize the predicted forecast quantities as measuring benchmarks for their own forecast inputs.

Note: In RapidResponse 2014.2 and earlier, statistical forecasting calculations were supported by transformation worksheets. This method of generating the statistical forecast is still available to you, but it is recommended that you transition to analytics to produce statistical forecasts. Analytics calculations are more efficient, yet simpler to produce than transformation worksheet calculations, and the calculated data is easier to maintain. Additionally, as of RapidResponse 2014.4, profile variables are no longer used to configure statistical forecasting settings or calculate disaggregation rates. For more information on transitioning to analytics for statistical forecasting, see "Transitioning to analytics for forecasting and disaggregation" on page 2413.

In this chapter, you will learn about:

Statistical forecasting tables **2019**
Statistical forecast configuration **2020**
Statistical forecasting operations **2030**
Statistical forecasting models **2031**
Multiple linear regression tables **2044**
Additional forecasting options **2052**
Data characteristics **2059**
Error measures **2060**

Statistical forecasting tables

The RapidResponse data model includes the following tables that support statistical forecasting calculations.

Table 43.1: Statistical forecasting tables

Table	Description
ForecastItem	This input table holds the configuration of the forecast items for which statistical forecasts can be generated. For more information, see "ForecastItem table" on page 292.
ForecastItem Parameters	This input table contains the parameters used in generating a statistical forecast for a given forecast item. For more information, see "ForecastItemParameters table" on page 296.
ForecastItem ParametersType	This control table contains the parameters used in generating a statistical forecast for a given forecast item. For more information, see "ForecastItemParametersType table" on page 727.
ForecastItem ParametersActual	This calculated table contains the bucketed adjusted history of a forecast item and any additional forecast items specified using the **ForecastItemParametersMap** table. For more information, see "ForecastItemParametersActual table" on page 1065.
ForecastItem ParametersFit Output	This input table saves the fit output generated by **StatisticalForecastFitOutput**. For more information, see "ForecastItemParametersFitOutput table" on page 313.
ForecastItem ParametersMap	This input table defines mappings between forecast items, which allows the historical demand from one item to be used in calculating the statistical forecast of another. For more information, see "ForecastItemParametersMap table" on page 321.
ForecastItem ParametersOutlier	This input table stores outlier adjustments at the forecast item level, which means that items that belong to more than one forecast category can use different outlier adjustments for each category. For more information, see "ForecastItemParametersOutlier table" on page 322.
ImportedStatistical Forecast	This input table is available for cases where the statistical forecast is being generated by tool or application external to RapidResponse and then imported into RapidResponse (instead of being generated based on parameters in the tables described above). For more information, see "ImportedStatisticalForecast table" on page 356.
StatisticalForecast FitOutput	This calculated table identifies the statistical model parameters that are used to calculate the statistical forecast, and calculates various statistical data characteristics and error measures of the statistical model. For more information, see "StatisticalForecastFitOutput table" on page 1182.
StatisticalForecast	This calculated table reports statistical forecast values as future dated quantities for a number of specified periods. For more information, see "StatisticalForecast table" on page 1179.
StatisticalForecast PredictActual	This calculated table fits a set of historical actual demands to the parameters calculated for the statistical forecast, allowing you to determine how accurately the model can forecast the actual data that it is based on and find errors in the forecast. For more information, see "StatisticalForecastPredictActual table" on page 1196.

Statistical forecast configuration

To ensure that RapidResponse's forecasting operations have the data required to perform their calculations, and that the calculated forecast values are realistic demand predictions, the following actions must be performed:

- "Define S&OP parameters" on page 2021
- "Provide historical data" on page 2021
- "Define outlier handling" on page 2022
- "Define forecast parameters" on page 2027
- "Define unit of measure" on page 2027

Define S&OP parameters

S&OP parameters for forecasting are defined in the **SOPAnalyticsConfiguration** table. This table is used primarily to define the default configuration of forecast disaggregation, but it also includes fields used to define the forecast.

The **SOPAnalyticsConfiguration** table holds only one record. As each scenario includes a version of this record, you can use this table to model various statistical forecast and disaggregation scenarios.

This record is typically populated when RapidResponse is deployed. If you are upgrading from RapidResponse 2014.2 (or earlier), populate this record before using analytics to calculate statistical forecasts. For more information on how to populate the **SOPAnalyticsConfiguration** table, see "Transitioning to analytics for forecasting and disaggregation" on page 2413.

Provide historical data

Historical demand data is contained in the **HistoricalDemandActual** table, which typically holds data related to historical actual customer shipments on a given date. It references the **HistoricalDemandHeader** table, which identifies the part, customer, and demand category for the actual shipment (by references to the **HistoricalDemandCategory** and **PartCustomer** tables). The records in the **HistoricalDemandActual** table that use the same category referenced by the **ForecastItemParameters.ActualsCategory** field are used to represent the demand history from which statistical forecasts are derived.

Historical demand can contain data errors and causals that should not be reflected in the statistical forecast. Data errors are instances in which there is no historical demand for a period, or at least one historical demand in the period has a zero value. Causals are specific events that impact demand, such as promotions or price discounts. Historical data is corrected for data errors and causals using the **CausalFactor** table, which stores the causal factors that are used to adjust the data. It references the **HistoricalDemandHeader** table, which identifies the part, customer, and historical actual data that the causal factor applies to, and the **CausalFactorCategory** table, which provides the category the causal factor belongs to. The **CausalFactor** table is referenced by the **CausalFactorDetail** table, which provides date and quantity details of the causal factor.

Historical demand must also be adjusted for outliers, or instances when the demand quantity is out of the typical range for a forecast item. Unlike data errors and causals, outlier adjustments are stored at the forecast item parameters level by the **ForecastItemParametersOutlier** table (added in RapidResponse 2014.4), which references the **ForecastItemParameters** table. Making outlier adjustments at the forecast item parameters level means that forecast items used with more than one forecast category in **ForecastItemParameters** can use a different set of outlier adjustments for each category.

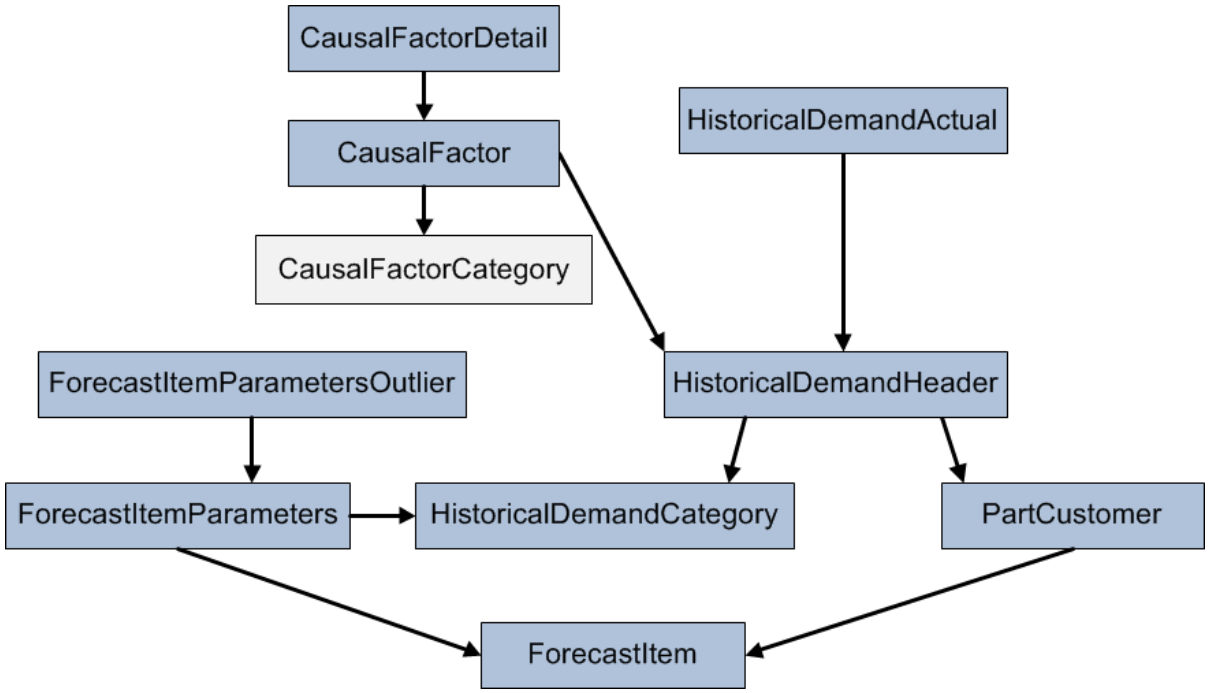
Because outlier adjustments used to be grouped with data errors and causals, customers using Version 2014.4 (or later) must ensure that any outlier records have been moved from **CausalFactorDetail** to **ForecastItemParametersOutlier** if they would like to make outlier adjustments at the forecast item level. For more information on populating the **ForecastItemParametersOutlier** table, see "Move outlier adjustments to a new table" on page 2415.

The history used to generate the statistical forecast is the sum of the **HistoricalDemandActual**, **CausalFactorDetail**, and **ForecastItemParametersOutlier** records for the forecast items. Before they can be used in the statistical forecast, the historical actuals are bucketed by the calendar defined in

ForecastItemParameters.Type.IntervalsCalendar, and stored in the **ForecastItemParametersActual** table.

The relationships between the tables that are required to provide historical data are illustrated in the following image.

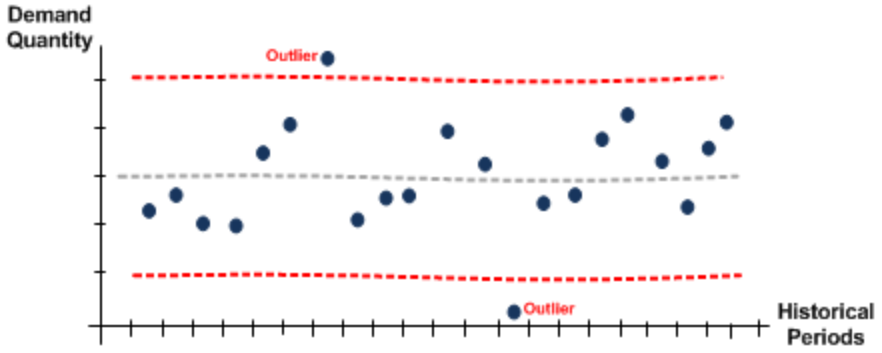
Figure 43.1 Tables used to provide historical demand information



Define outlier handling

Outliers refer to points in the data that are generally not consistent with the overall pattern of the data. For example, they might refer to points that are more than a specified distance away from the average point in the data.

Figure 43.2 Outliers in historical actuals



When generating the statistical forecast, the presence of outliers in historical data can limit the ability to generate a reasonably accurate forecast. In order to identify and process outliers in the historical data, several configuration options are available in RapidResponse. These are mostly set through the **OutlierType** control table, as well supporting fields in the **ForecastItemParameters** table.

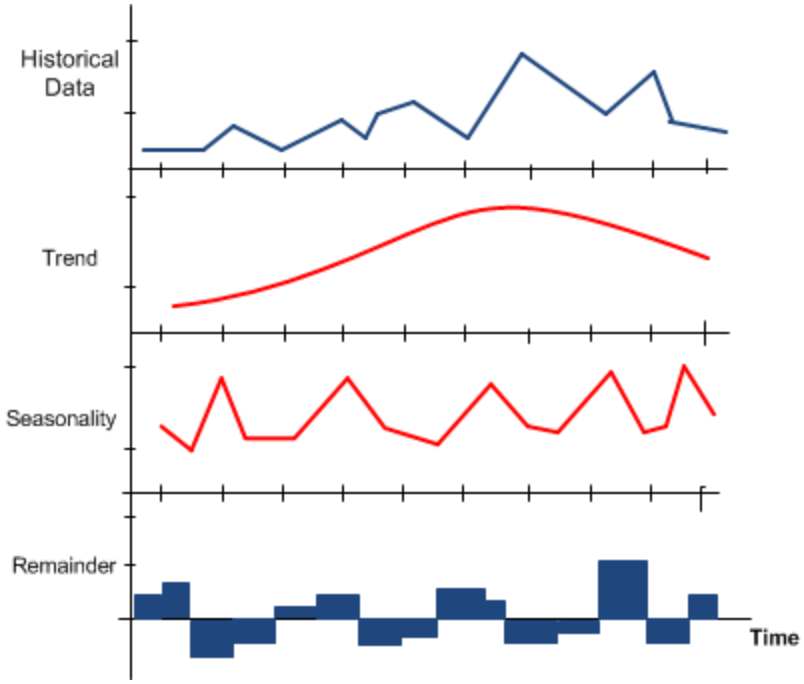
Data used for outlier detection

The **OutlierType.DataRule** setting specifies the actual data points to be evaluated for the presence of outliers. With the default setting of "Historical", outlier detection is performed against raw data points from historical date period used as input in generating the statistical forecast.

Instead, if the **DataRule** is set to "MovingAverageError", then outlier detection is made against points that represent the difference between the historical actual quantity in each period and the moving average calculated using a defined number of points before that actual. With this setting, the number of periods used in calculating the moving average is set by the **ForecastItemParameters.OutlierMovingAverageWindow** field.

Finally, if the optional Rforecast capability has been enabled, the **DataRule** can be set to "RstlError". This option uses the stl function in the R programming language to return a set of adjusted actuals for use in detecting outliers. That is, outliers are then detected using historical data after any seasonal or trend components have been removed as shown in the following illustration.

Figure 43.3 Decomposed time series data



Rules for outlier detection

Using data points as determined by the selected **DataRule**, outliers are then detected based on the **OutlierType.DetectionRule** setting. Most options in this field are used in conjunction with the **ForecastItemParameters.OutlierThreshold** field to calculate upper and lower thresholds. Points above or below those threshold values are considered to be outliers.

The **DetectionRule** contains the following four settings;

- StandardDeviation (see "StandardDeviation" on page 2024)
- IglewiczHoaglinMethod (see "IglewiczHoaglinMethod" on page 2024)
- Winsorizing (see "Winsorizing" on page 2025)
- Rtsclean (see "Rtsclean" on page 2025)

StandardDeviation

This is the default setting in the **DetectionRule** field. It defines outliers as being more than a specified number of standard deviations away from the mean. With this setting, the **OutlierThreshold** value is used to indicate the number of standard deviations above or below the mean where a point is considered an outlier.

For example, assume the following historical actuals, along with a **DetectionRule** of "StandardDeviation" and an **OutlierThreshold** value of "3".

Jan	Feb	Mar	Apr	May	June
600	550	400	600	650	500

In this case, the mean would be calculated as follows:

$$\frac{500+650+600+400+550+600}{6} = 550$$

And the standard deviation calculated as:

$$\sqrt{\frac{\left(500-550\right)^2 + \left(650-550\right)^2 + \left(600-550\right)^2 + \left(400-550\right)^2 + \left(550-550\right)^2 + \left(600-550\right)^2}{5}} = 89.44$$

Thus, a point would be considered an outlier if less than 281.68 (that is, *550-(89.44*3)*) or greater than 818.32 (that is, *550+(89.44*3)*).

IglewiczHoaglinMethod

This setting uses a modified z-score to measure distance from the mean. With this setting, a point is considered an outlier if it is outside the range shown in the following formula

$$[Md - X*(\text{MAD}/.6745), Md + X*(\text{MAD}/.6745)]$$ where

- *Md*—median,
- MAD—median absolute deviation
- *X*— OutlierThreshold

For example, assume again the following historical actuals, along with a **DetectionRule** of "IglewiczHoaglinMethod".

Jan	Feb	Mar	Apr	May	June
600	550	400	600	650	500

Further assuming an **OutlierThreshold** of "2.5", the outlier thresholds would be calculated as follows:

$$[575 - 2.5*(66.67/.6745), 575 + 2.5*(66.67/.6745)] = [389.68, 760.32]$$

Thus, a point would be considered an outlier if less than 389.68 or greater than 760.32.

Winsorizing

This setting allows for outliers to be defined as those points outside of a specified percentile in the sorted data set. This can be useful for removing upper and lower tails in the large data sets. With this setting, the **OutlierThreshold** field is used to define the upper and lower percentiles to be removed from the data, and a value between .01 and .49 should be provided.

For example, assume the following set of 12 months of historical actuals and an **OutlierThreshold** value of "0.2".

Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
50	100	55	65	70	55	80	30	95	85	60	45

In this case, sorting the actuals in ascending order gives the following ordered set:
[30, 45, 50, 55, 55, 60, 65, 70, 80, 85, 95, 100]

Using the sorted data, the 20th percentile point can be found as 12 * .2 = 2.4 (rounded up to the 3rd ordered point, gives 50) and the 80th percentile can be found as 12 * .8 = 9.6 (rounded up to the 10th ordered point gives 85). Thus, a point would be considered an outlier if less than 51 or greater than 84.

Rtsclean

Optionally, if the Rforecast capability has been enabled, the **DetectionRule** option can be set to "Rtsclean". This setting uses the "tsclean" method from the R statistical programming language. To identify outliers, this method uses loess for non-seasonal data and a periodic stl decomposition for seasonal data. To estimate missing values, linear interpolation is used for non-seasonal series, and a periodic stl decomposition is used with seasonal series

Note also that the "Rtsclean" setting should only be used with pure historical data that has not had any decomposition applied. As a result, when the **DetectionRule** is set to "Rtsclean", the **DataRule** setting is always treated as if it were set to "Historical".

Processing outliers

Once an outlier has been detected within an item's historical data, it should be adjusted to a level closer to the general pattern of the data set. To support outlier adjustments, the **AboveThreshold** and **BelowThreshold** settings on the **OutlierType** table can be used.

The **AboveThreshold** field specifies how data points above the upper outlier threshold are handled, while the **BelowThreshold** field specifies how data points below the lower outlier threshold are handled. Otherwise, these two fields are conceptually similar and contain the same four options for adjusting outlier data as follows:

- **Forecast**—the outlier quantity is replaced by a calculated value dependent on the **DataRule** setting. If using the default "Historical" option, then this option is the same as "Mean" (replaces the outlier with the average from the data). Otherwise, if using the "MovingAverageError" option, the moving average up to the point where the outlier was found is used, and if using the "RstlError" option, the sum of the trend and seasonal components are used to replace the outlier.
- **Mean**—the outlier quantity is replaced by the mean (average) point in the data set.
- **Median**—the outlier quantity is replaced the median (middle) point in the data set.
- **SmoothKeepExcess**—the outlier quantity is smoothed above the threshold over a number of intervals backward and then forward. If after smoothing, any excess remains above the outlier threshold, it is left as is.

- **SmoothRemoveExcess**—the outlier quantity is smoothed above the threshold over a number of intervals backward and then forward. If after smoothing, any excess remains above the outlier threshold, that excess is reduced to the outlier threshold for the item.
- **Threshold**—the outlier quantity is replaced by the lower threshold (if below) or the upper threshold level (if above).

If you are using any of the smoothing options for **OutlierType.AboveThresholdRule** or **OutlierType.BelowThresholdRule**, there are a few considerations to keep in mind. The following table describes the order in which the selected smoothing options are applied.

When you	Result
Specify only one of the threshold rules to use either **SmoothKeepExcess** or **SmoothRemoveExcess**. For example, you set **SmoothKeepExcess** to the **AboveThresholdRule**, but do not apply smoothing to **BelowThresholdRule**.	The smoothing correction is applied to the outliers in the input data series. Then, if outliers remain in the corrected data, one of the following non-smoothing rules will be applied, as applicable: Forecast, Ignore, Mean, Median, or Threshold.
Specify both threshold rules to use either **SmoothKeepExcess** or **SmoothRemoveExcess**. For example, you set **SmoothRemoveExcess** to both the **AboveThresholdRule** and **BelowThresholdRule**.	The smoothing correction is first applied in the input data series only to outliers that are "above" the threshold. Then, if outliers that are "below" the threshold remain in the corrected data, the non-smoothing rule is applied.

Calculated outlier details

Two calculated tables, **StatisticalForecastOutlier** and **StatisticalForecastOutlierSummary** report details on outliers found in historical data used in generating the statistical forecast.

For a given forecast item, the **StatisticalForecastOutlier** table contains one record for each period of historical actual data used in generating the statistical forecast (as determined by **ForecastItemParameters.HistoricalIntervalCount**) Each record in this table contains a boolean field, **IsOutlier**, indicating whether the period represents an outlier in the item's historical data set. For periods identified as outliers, other details such as the recommended adjustment to the quantity are reported. Additionally, the calculated upper and lower threshold values used in determining whether a point is an outlier are reported in this table. For complete reference information on this table, see "StatisticalForecastOutlier table" on page 1192.

The **StatisticalForecastOutlierSummary** then reports summary data characteristics used in calculating a forecast item's outliers. For example, the number of periods of history used in calculating outliers and the mean quantity across those periods is reported. For more information on this table, see "StatisticalForecastOutlierSummary table" on page 1194

Note: The two outlier tables discussed above are only populated for forecast items that reference a valid **OutlierType** record through the **ForecastItemParameters** table. If an item has **ForecastItemParameters.OutlierType** set to "Null", then the item will not have records generated in these tables.

Define forecast parameters

The **ForecastItemParameters** table contains the input parameters that are used in generating a forecast for a given forecast item. This table has two key references that identify the unique parameters for the forecast item:

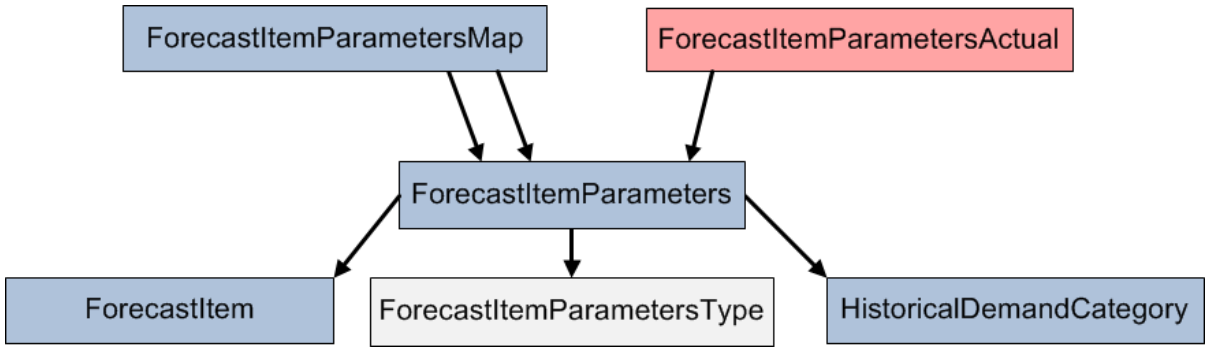
- The **ForecastItem** table, which identifies the item the forecast parameters apply to.
- The **HistoricalDemandCategory** table, which identifies the category that the forecast belongs to.

For a statistical forecast, the category would be "Statistical". The **ForecastItemParameters** table also references the **ForecastItemParametersType** table in order to define parameters such as forecast model, fit measure, and intervals calendar. It is possible to use the history from one item to calculate the statistical forecast of another using the **ForecastItemParametersMap** table. This table references **ForecastItemParameters** for the forecast item parameters the mapping is defined for and the forecast item parameters that are mapped to the item.

One of the forecast parameters that can be defined is the **ForecastItemParameters.UnitofMeasure**. For more information about this field, see "Define unit of measure" on page 2027.

The relationships between the tables required to define forecast parameters are illustrated in the following image.

Figure 43.4 Tables used to define forecast parameters



Define unit of measure

When configuring a forecast item, you may need to assign a unit of measure to the forecast item. For example, if individual part-customers that reference the forecast item are using different units of measure or if you want to report a forecast item in a unit of measure that's different than the part-customers that reference it, assigning a forecast item unit of measure ensures that all units are correctly converted in a statistical forecast. In addition, to successfully use the **Unit of Measure** control in any of the workbooks, it's necessary to assign a forecast item unit of measure to forecast items even if they are reported in the same unit of measure as their part-customers. To specify a forecast item unit of measure, you need to assign a unit of measure to a forecast item using the **ForecastItemParameters.UnitofMeasure** field. By specifying the **ForecastItemParameters.UnitofMeasure**, this also ensures that the historical demand for the part-customers is aggregated and dissagregated using a consistent unit of measure.

For example, suppose that parts *Battery A* and *Battery B* have the **Part.UnitOfMeasure** field specified as "Unit" and both parts also reference forecast item *Batteries*. If *Batteries* has the **ForecastItemParameters.UnitofMeasure** field specified as "Case", the historical demand quantities for *Battery A* and *Battery B* would be converted from "Unit" to "Case" for reporting quantities of *Batteries* in the statistical forecast.

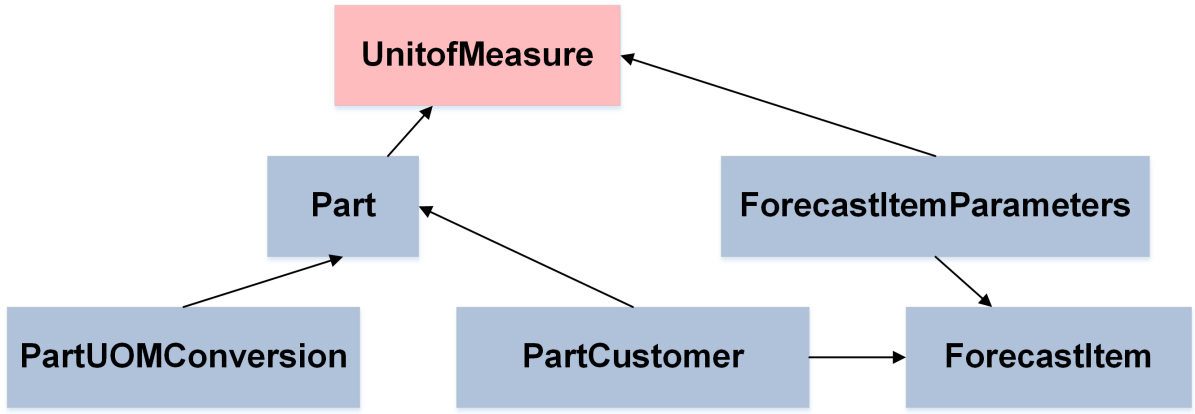
When a **ForecastItemParameters.UnitofMeasure** is specified, RapidResponse converts the historical demand quantities found in **HistoricalDemandActual.Quantity** from the unit of measure specified in **HistoricalDemandActual.Header.PartCustomer.Part.UnitOfMeasure** to the unit of measure specified in **ForecastItemParameters.UnitOfMeasure**, and reports these values in **ForecastItemParametersActual.Quantity**.

Note: If **ForecastItemParameters.UnitofMeasure** is not specified, the quantities reported in **ForecastItemParametersActual.Quantity** are an aggregated sum of **HistoricalDemandActual.Quantity** without converting any units of measure. Therefore, if the part-customers are using different units of measure, the quantities reported in the statistical forecast will be comprised of different units.

Unit of measure conversion

In RapidResponse, quantities are converted from one unit of measure to another by using the factor that is specified in **UnitOfMeasure.BaseConversion**, which is a factor to convert quantities in one unit to the base unit. The base unit represents the smallest unit of measure in a group of units and it is used to convert to other units of measure. For example, "Unit" could be set as a base unit of measure and it would be used to convert quantities to "Case" or "Pallet". However, if **Part.UnitOfMeasure** has a different base unit value than the one that is given in **UnitOfMeasure.BaseConversion**, a part-specific unit of measure conversion factor needs to be entered using the **PartUOMConversion** table to ensure that the quantities are correctly converted to a different unit of measure.

Figure 43.5 The following illustration shows the tables that are associated with defining units of measure and conversion factors for reporting quantities in the statistical forecast.



Example of unit of measure conversion

The following example demonstrates how to use the **PartUOMConversion** table to ensure that quantities are converted using the correct unit of measure in the statistical forecast. First, suppose parts *Battery A* and *Battery B* have **Part.UnitOfMeasure** set to "Unit" and both parts also reference forecast

item *Batteries*, which has **ForecastItemParameters.UnitofMeasure** set to "Case". Let's assume that the **UnitOfMeasure.BaseConversion** factor for "Unit" is 1 (base unit) and the factor for "Case" is 100. "Case" is set to a factor of 100 because it is equivalent to 100 "Units" (base units). If *Battery A* is stocked in individual units, the existing **UnitOfMeasure.BaseConversion** values would accurately convert "Unit" quantities to "Case".

Note: If a part has a unit of measure conversion record in the **PartUOMConversion** table, the **UnitOfMeasure.BaseConversion** value is ignored and quantities in the following tables are calculated using **PartUOMConversion.Factor**: **ForecastItemParametersActual**, **StatisticalForecast**, **StatisticalForecastDetail**, and **StatisticalForecastDisaggregationRate**.

Now, continuing with the example, suppose that one "Unit" of *Battery B* is equivalent to two batteries, instead of just one like *Battery A*. If the existing **UnitOfMeasure.BaseConversion** values are used to convert quantities of *Battery B*, the quantities reported would be incorrect. A quantity of 100 units of *Battery B* would be converted to only one case, but the correct amount would be two cases.

The following table represents three period buckets of **HistoricalDemandActual.Quantity** for parts *Battery A* and *Battery B*. Notice that the value in **ForecastItemParametersActual.Quantity** for *Batteries* is using the existing **UnitOfMeasure.BaseConversion** values of 1 for "Unit" and 100 for "Case". As a result, the quantity of cases is incorrect.

Part	UnitofMeasure.BaseConversion	February 13	February 20	February 27
Battery A – Unit		100	200	300
Battery B – Unit		100	200	300
Conversion calculation	100	200/100=2	400/100=4	600/100=6
Batteries – Case		**2**	**4**	**6**

To ensure that the quantity of units is correctly converted, a **PartUOMConversion** record needs to be set for *Battery B* for the conversion to the unit of measure "Case". The **PartUOMConversion.Factor** uses the following formula to convert values from **Part.UnitOfMeasure** to a target **UnitOfMeasure**:

$$\text{Quantity} \times \text{PartUOMConversion.Factor}$$

Therefore, *Battery B* would require a **PartUOMConversion.Factor** of 0.02 to correctly convert quantities from "Unit" (equivalent to 2 units) to "Case".

It is important to note that the formula for **PartUOMConversion.Factor** is different than the one that is used by **UnitOfMeasure.BaseConversion**, which converts quantities from one base unit of measure to a target unit of measure:

$$\text{Quantity} \times \frac{\text{Part.UnitofMeasure.BaseConversion}}{\text{Target UnitofMeasure.BaseConversion}}$$

Therefore, unlike the **PartUOMConversion.Factor**, the **UnitofMeasure.BaseConversion** formula takes into consideration the value of the base unit of measure in the conversion.

To continue with the example, the following table shows how quantities of *Battery A* and *Battery B* were converted from "Unit" to "Case". Quantities of *Battery A* were converted by using **UnitofMeasure.BaseConversion**. Quantities of *Battery B* were converted using **PartUOMConversion.Factor**.

Part	UOM Conversion	February 13	February 20	February 27
Battery A	UnitofMeasure. BaseConversion 1	100/100 = 1	200/100 = 2	300/100 = 3
Battery B	PartUOMConversion. Factor .02	100 * .02 = 2	200 * .02 = 4	300 * .02 = 6

In summary, using the **PartUOMConversion.Factor** ensured that *Battery B* units were correctly converted to cases. The following table represents three period buckets of **HistoricalDemandActual.Quantity** for parts *Battery A* and *Battery B* with the sum **ForecastItemParametersActual.Quantity** for *Batteries* showing the correct number of cases.

Part	UOM Conversion	February 13	February 20	February 27
Battery A – Unit		100 Units	200 Units	300 Units
Battery A – Case	UnitofMeasure. BaseConversion 1	1 Cases	2 Cases	3 Cases
Battery B – Unit		100 Units	200 Units	300 Units
Battery B – Case	PartUOMConversion. Factor .02	2 Cases	4 Cases	6 Cases
Batteries – Case		**3 Cases**	**6 Cases**	**9 Cases**

Statistical forecasting operations

With the data configuration requirements met, RapidResponse analytics perform the following operations to generate the statistical forecast and to ensure the forecast's validity. Data produced by the operations is stored in the appropriate calculated table automatically.

Table 43.2: Statistical forecasting operations

Operation	Description
Fit the forecast	This operation determines how a statistical forecast is calculated, and the statistical model that the forecast conforms to. It also provides information about the forecast models, such as data characteristics and error measure values. For example, this operation may determine that the exponential smoothing model will generate the most realistic forecast values given your historical actuals. You can specify the forecast parameters that are used as input to this operation, which allows you to override any calculated values and to customize the forecast. For example, you can specify which forecasting model is used to generate the forecast. The data calculated by this operation is required for the following operations and is stored in the **StatisticalForecastFitOutput** table. For more information, see "StatisticalForecastFitOutput table" on page 1182.
Predict forecast values	This operation calculates statistical forecast values using data generated by the fit the forecast operation. The forecast values are stored in the **StatisticalForecast** table. For more information, see "StatisticalForecast table" on page 1179. **Note:** Forecasts generated in the **StatisticalForecast** table are made at the forecast item level. Based on configurable disaggregation parameters the statistical forecast is then disaggregated to the detailed part and customer level and reported in the **StatisticalForecastDetail** table. For more information about statistical forecast disaggregation, see "Forecast disaggregation" on page 2083.
Predict actual values	This operation fits a set of historical actual demands to the parameters calculated for the statistical forecast by the fit forecast operation. This generates a set of data points that the statistical forecast parameters would produce if they were used to forecast historical actuals. When you compare these data points to real historical actuals, you can determine the accuracy of the statistical forecast or find errors. The data calculated by this operation is stored in the **StatisticalForecastPredictActual** table. For more information, see "StatisticalForecastPredictActual table" on page 1196.

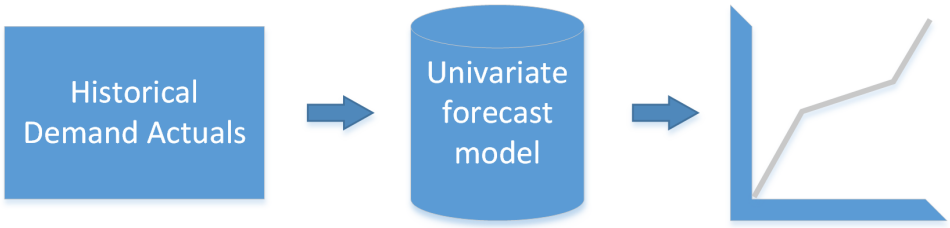
Statistical forecasting models

RapidResponse supports a number of forecasting models and options that you can use to generate a statistical forecast for a forecast item. Each forecast model performs a unique calculation, and is best suited to specific data conditions. The available statistical forecast models are grouped into two categories: univariate time series and multiple linear regression. There are also additional options to assist you in choosing the right model for your data.

The forecast model to be used for each item is set by the **ForecastItemParametersType.ForecastModel** field.

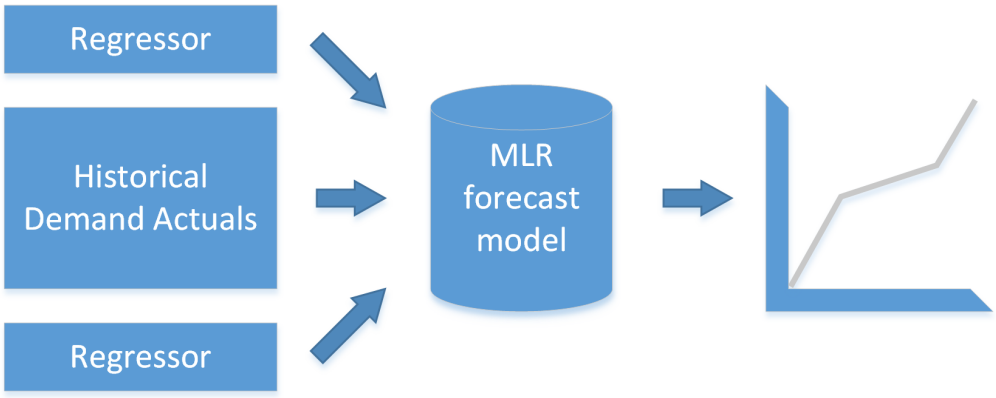
Univariate time series forecast models

Univariate time series forecast models, such as Linear or ARIMA, use historical demand actuals data in statistical forecast calculations to predict future demand. Different univariate forecast models support different data patterns, such as seasonality or trends. RapidResponse includes various models that support these different patterns. For more information, see "Univariate time series forecasting models" on page 2032.



Multiple Linear Regression forecast models

Multiple linear regression (MLR) forecast models offer more advanced statistical forecasting capabilities. MLR models use historical demand actuals data and additional historical data from regressors (independent variables) that may drive demand for a product, such as holidays, weather, or promotions, in statistical forecast calculations to predict future demand. They are designed to enhance the ability to predict demand driven by these past events or trends. For more information, see "Multiple Linear Regression forecasting models" on page 2042



Additional forecasting options

RapidResponse also includes additional options for generating a statistical forecast. For example, you can use Best Fit to let RapidResponse analyze the historical actual data and automatically select one of the available forecast models that best suits the data. For more information, see "Additional forecasting options" on page 2052

Note: For more information about the disaggregation logic that is applied to statistical forecast values whether they be imported or generated in RapidResponse, see "Forecast disaggregation" on page 2083.

Univariate time series forecasting models

Univariate time series forecasting models are useful for generating a statistical forecast that use available historical demand data to predict future demand. A time series is comprised of a series of well-defined data points taken sequentially over time. For example, historical demand data is considered a time series.

This section includes the following information:

- "How to choose a univariate time series forecast model" on page 2033
- "List of univariate time series forecasting models" on page 2033
- "Univariate time series forecast model parameters" on page 2041

How to choose a univariate time series forecast model

Forecast models are designed to support specific data patterns. The model that you choose to generate a statistical forecast would depend on the data pattern found in the historical sales data for the forecast item. Choosing a forecast model that best suits your forecast item's data pattern will yield better results. The following table shows the various data patterns and it also lists the corresponding forecast models.

Data pattern	Category	Models	Use if historical sales data
	Constant	Exponential smoothing Moving average	Shows stable demand that does not fluctuate. For example, it could apply to a mature product with stable demand.
	Trend	ARIMA Double-exponential smoothing Linear Step-wise ARIMA RForecast	Shows demand that is consistently growing or declining. For example, it could apply to a new or end-of-life product.
	Seasonal	Holt-Winters multiplicative and additive* RForecast	Shows calendar-driven shifts in demand. For example, it could apply to a seasonal product.
	Sporadic	Croston's method	Shows irregular or sudden shifts in demand. For example, it could apply to a product with intermittent demand.

Note: You can use Holt-Winters with historical data that has both seasonality and trend. However, it is not recommended for historical data that has only trend.

List of univariate time series forecasting models

In RapidResponse, the following univariate time series forecasting models are available:

- "Auto-Regressive Integrated Moving Average (ARIMA)" on page 2034
- "Croston's method" on page 2034
- "Double exponential smoothing" on page 2036

- "Exponential smoothing" on page 2037
- "Holt-Winters multiplicative and additive" on page 2037
- "Linear" on page 2039
- "Moving average" on page 2039
- "Rforecast" on page 2039
- "Step-wise ARIMA" on page 2039

Auto-Regressive Integrated Moving Average (ARIMA)

The ARIMA model considers regressive terms and moving average terms for distinct time periods or lags, and calculates forecasts for a time period using a linear combination of actual values and error values in a time series. This model can be used on a stationary time series, or a differencing function can be applied to non-stationary data to make it stationary, which ensures the mean and variance of the time series do not change over time. The ARIMA model is not suited for seasonal data.

You can specify the difference level, which determines how many differencing functions are performed on the data. The differencing function can be written $Y(t) - Y(t-1)$ for each point, where *Y* is the data value and *t* is the time period. This function is applied to the data as many times as specified.

After the differencing function is performed, an Auto-Regressive Moving Average (ARMA) calculation is performed using the following formula, where μ is a constant value, φ is the coefficient of the auto-regressive function, θ is the coefficient of the moving average function, *t* is the time, y is the actual value at point *t*, *e* is a white noise error process at point *t*, *p* is the number of auto-regressive terms, and *q* is the number of moving average terms:

$$y_t = \mu + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \ldots + \phi_p y_{t-p} + e_t + \theta_1 e_{t-1} + \ldots + \theta_q e_{t-q}$$

The ARIMA calculation considers auto-regressive and moving average values for each lag. Depending on how you want to calculate the function, you can specify the lags to be used in each of the auto-regressive and moving average calculations. For example, if you specify "AutoRegressiveTerms = (1,2,4)" and "MovingAverageTerms=(1,3)", the auto-regressive calculation is performed for lags 1, 2, and 4, and the moving average calculation is performed for lags 1 and 3. If you specify a single lag, the brackets specified for these parameters can modify how the calculation runs. For example, specifying "AutoRegressiveTerms = 4" means that all lags up to the 4th are considered; however, "AutoRegressiveTerms = (4)" means only the 4th lag is considered.

Croston's method

The Croston's method model is exponential smoothing that accounts for intermittent or sporadic demand. This method is best suited for demand that is not constant, such as lumpy demand that contains periods of zero quantities, or that has few non-zero data points in a period.

Croston's method uses a single smoothing constant for the forecast values, but calculates two different values. The first calculated value, Z_i, is the quantity of forecast demand at a point. The second calculated value, X_i, is the average length of time between forecasts. Basically, these values provide the forecast quantities and the periods that have forecast demands. A forecast quantity will be followed by X_i periods of no forecasts, then another forecast quantity, then another X_i periods of no forecasts, and so on.

The forecast quantity and average length between forecasts are calculated using a smoothing function if the demand at a point is not zero; otherwise, the values from the previous point are repeated.

Croston's method is calculated using the following formulas, where Z_i is the forecast value, X_i is the average length between forecast values, q is the number of periods between two non-zero demands, and alpha (α) is the smoothing constant:

$$Z_{i+1} = \begin{cases} \alpha y_i + (1-\alpha) Z_i, & y_i \neq 0 \\ Z_i, & y_i = 0 \end{cases}$$

$$q_{i+1} = \begin{cases} 1, & y_i \neq 0 \\ q_i + 1, & y_i = 0 \end{cases}$$

$$X_{i+1} = \begin{cases} \alpha q_i + (1-\alpha) X_i, & y_i \neq 0 \\ X_i, & y_i = 0 \end{cases}$$

$$Z_0 = y_1$$

$$X_0 = \begin{cases} 1 \text{ if } y_1 \neq 0 \\ 2 \text{ if } y_1 = 0 \end{cases}$$

$$q_0 = 1$$

$$\text{FittedValue}_i = \frac{Z_{i-1}}{X_{i-1}}$$

The Z_n and X_n values are calculated using the formulas listed above and the forecast can be reported using one of two ways: "Constant" (the default field value) or "Sporadic". These methods of reporting can be selected using **ForecastItemParametersType.CrostonsPredictRule**.

When reporting the forecast using "Constant" predictions, the forecast values are spread over all time periods. When using "Sporadic" predictions, the forecast values appear only every X_n intervals. Let

$$\text{ConstantForecast} = \frac{Z_n}{X_n},$$

$\text{SporadicInterval} = \text{Round}\ (\text{X}_n)$, and

$$\text{SporadicForecast}_i = \begin{cases} \dfrac{\text{ConstantForecast} \times \text{ForecastIntervalCount}}{\left\lceil \frac{\text{ForecastIntervalCount}}{\text{SporadicInterval}} \right\rceil}, & i \bmod \text{SporadicInterval} = 0 \\ 0, & \text{otherwise} \end{cases}.$$

Notes:

- SporadicInterval is rounded to the nearest integer.
- You can verify individual values for Z_n and X_n by using the **StatisticalForecastFitOutput** table. In the **ValueName** field, the **SmoothedValue** model output reports the Z_n value and the **InterarrivalValue** model output reports the and X_n value.

When **ForecastItemParametersType.CrostonsPredictRule** is set to "Constant":

$$\text{Forecast}_i = \text{ConstantForecast}$$

When **ForecastItemParametersType.CrostonsPredictRule** is set to "Sporadic":

$$\text{Forecast}_i = \text{SporadicForecast}_i$$

Sporadic predicted values are greater than constant predicted values, sporadic values are not averaged over time; ultimately, however, the total sporadic forecast demand equals the total constant forecast demand.

Double exponential smoothing

The double exponential smoothing model considers input data points and trends within the data, and applies a smoothing algorithm to both. The result of the data points and trend are used as the statistical forecast. This method is best used for demand that is mostly linear, but that trends upwards or downwards over time.

For this model, the smoothing formulas require different smoothing constants, which are values between zero and one. The values for each point in the forecast and the trend are calculated separately, and then added to determine where the statistical forecast point should be.

Double exponential smoothing is calculated using the following equation, where *F* is the smoothed value, *T* is the trend value, alpha (α) is the smoothing constant, gamma (γ) is the trend smoothing constant, and *N* is the number of periods:

$$F_{i+1} = \alpha y_{i+1} + (1 - \alpha)\,(F_i + T_i)$$

$$T_{i+1} = \gamma(F_{i+1} - F_i) + (1 - \gamma)T_i$$

The initial values for this function are calculated by backcasting the function. A linear regression sets the initial trend value for the backcasting function, while the last actual statistical forecast input value sets the smoothed value. The final values calculated by the backcasting function are the initial values used by the double exponential smoothing function.

Using the results of these formulas, forecast values that fit the input values can be calculated, and future forecast values can be predicted from the final forecast and trend values, using the following formulas:

$$\mathbf{FittedValue}_i = F_{i-1} + T_{i-1}$$

$$\mathbf{Forecast}_{N+m} = F_N + mT_N$$

$$m = 1, 2, \ldots$$

If you have historical data that follows a trend that will not continue, either by stopping completely or lessening over time, you can eliminate or dampen the trend in your statistical forecast when using double exponential smoothing. The **ForecastItemParametersType.TrendDecayFactorRule** field is set to "Ignore" by default. However, if the field is set to "Use", a decay factor (*k*) is added to the statistical calculation. Double exponential smoothing with trend decay or elimination is calculated using the following formula:

$$\mathbf{Forecast}_{N+m} = F_N + ((1 - k^m)/(1 - k))T_N$$

The decay factor is set by the value in the **ForecastItemParameters.TrendDecayFactor** field. If the trend should decay over time, this value must fall between zero and one. For example, assuming the trend decreases by 10 percent each period, the value in the field is "0.9". If neither decay nor elimination is required, the field value should be "1" (the default field value) or a value greater than one. If the value is "0", the decay formula is interpreted as $F_N + T_N$. ll values less than zero result in complete elimination of the trend.

The **ForecastItemParametersType.TrendDecayFactorRule** field can be set to "Use" indefinitely, as all levels of decay (no decay, some decay, or complete trend elimination) can be determined by the decay factor. However, using this field to control trend decay and elimination, rather than changing the decay factor value, means fewer manipulated records in the database.

The double exponential smoothing model also includes the capability to specify confidence levels to calculate prediction intervals when generating the statistical forecast. For more information, see "ForecastItemParameters table" on page 296 and "StatisticalForecast table" on page 1179.

Exponential smoothing

The exponential smoothing model calculates a statistical forecast by combining fitted actuals with the latest historical actual demands. This model considers the available historical input data, but assigns gradually decreasing significance to each data point as it ages. It is suitable for forecasting demand that is fairly stable and does not exhibit a trend or seasonality.

When using this model a smoothing constant, alpha (α), is required and applied to the historical data points when generating the statistical forecast. Alpha is always a value between 0 and 1, with a larger alpha giving higher weightings to the more recent data points, and a smaller alpha giving higher weightings to the earlier data points. Alpha is automatically determined by RapidResponse when calculating a fitted value for the historical actuals, and is set as the value that minimizes the residual sum of squares.

When fitting the actuals, each fitted value combines the previous period's historical actual (y) with its fitted value, after applying the smoothing constant alpha (α), as shown in the following formula:

$$\mathbf{FittedValue}_{k+1} = \alpha y_k + (1-\alpha)\mathbf{FittedValue}_k$$

The initial value for this function is set to the final data point calculated by backcasting the function. The initial value for the backcasting function is the last actual value used as input for the statistical forecast. The final value produced by the backcasting function is the first fitted value used for the exponential smoothing function.

The statistical forecast values are constant, and are equal to the final fitted value, as shown in the following formula:

$$\mathbf{Forecast} = \alpha y_N + (1-\alpha)\mathbf{FittedValue}_\mathbf{N}$$

The exponential smoothing model also includes the capability to specify confidence levels to calculate prediction intervals when generating the statistical forecast. For more information, see "ForecastItemParameters table" on page 296 and "StatisticalForecast table" on page 1179.

Holt-Winters multiplicative and additive

The Holt-Winters models are similar to the double exponential smoothing model in that they consider data points and trends. However, the Holt-Winters models also account for seasonal trending, and are best suited for demand that rises and falls over a period. Multiplicative Holt-Winters (referred to as HoltWintersMethod in the data model) should be used when demand has proportional variation over the period; additive Holt-Winters (referred to as AdditiveHoltWintersMethod in the data model) should be used when demand has constant variation over the period. Both models require data for all previous periods, and cannot be used if there are any gaps in the historical buckets.

For the Holt-Winters models, the smoothing formulas require different smoothing constants, which are values between zero and one. The values for each point in the forecast and the trend are adjusted to account for the seasonal demand change, and then added to determine where the statistical forecast point should be.

The seasonal trend is smoothed along with the trend and forecast demands. These are calculated using the following formulas, where *F* is the smoothed value, *S* is the seasonal trend value, *T* is the trend value, *p* is the length of the season, alpha (α) is the smoothing constant, beta (β) is the seasonal trend smoothing constant, and gamma (γ) is the trend smoothing constant:

$$F_i = \alpha \frac{y_i}{S_{i-p}} + \left(1 - \alpha\right)\left(F_{i-1} + T_{i-1}\right)$$

$$S_i = \beta \frac{y_i}{F_i} + \left(1 - \beta\right) S_{1-p}$$

$$T_i = \gamma(F_i - F_{i-1}) + (1 - \gamma) T_{i-1}$$

The initial values for a Holt-Winters function are calculated by backcasting the function. The initial values for the backcasting function are set by using a linear regression with seasonal dummies to set the trend and seasonal trend values, and the last actual statistical forecast input value sets the initial smoothed value. The final values calculated by the backcasting function are the initial values used by the Holt-Winters functions.

Using the results of these formulas, forecast values that fit the input values can be calculated, and future forecast values can be predicted from the final forecast and trend values. The Holt-Winters multiplicative method uses the following formulas:

$$\mathbf{FittedValue}_i = (F_{i-1} + T_{i-1})\, S_{i-p}$$

$$\mathbf{Forecast}_{N+m} = (F_N + mT_N)\, S_{N-p+m},\ m = 1, 2, \ldots$$

The Holt-Winters additive method uses the following formulas:

$$\mathbf{FittedValue}_i = (F_{i-1} + T_{i-1}) + S_{i-p}$$

$$\mathbf{Forecast}_{N+m} = (F_N + mT_N) + S_{N-p+m},\ m = 1, 2, \ldots$$

If you have historical data that follows a trend that will not continue, either by stopping completely or lessening over time, you can eliminate or dampen the trend in your statistical forecast when using a Holt-Winters model. The **ForecastItemParametersType.TrendDecayFactorRule** field is set to "Ignore" by default. However, if the field is set to "Use", a decay factor (*k*) is added to the statistical calculations. With the decay factor added, the Holt-Winter's multiplicative formula becomes the following:

$$\mathbf{Forecast}_{N+m} = F_N + T_N \frac{1-k^m}{1-k}$$

With the decay factor added, the Holt-Winter's additive formula becomes the following:

$$\mathbf{Forecast}_{N+m} = F_N + T_N \frac{1-k^m}{1-k} + S_{N-p+m}$$

The decay factor is set by the value in the **ForecastItemParameters.TrendDecayFactor** field. If the trend should decay over time, this value must fall between zero and one. For example, assuming the trend decreases by 10 percent each period, the value in the field is "0.9". If neither decay nor elimination is required, the field value should be "1" (the default field value) or a value greater than one. If the value is "0", the multiplicative decay formula is interpreted as $F_N + T_N S_{N-p+m}$, and the additive decay formula is interpreted as $F_N + T_N + S_{N-p+m}$. All values less than zero result in complete elimination of the trend.

The **ForecastItemParametersType.TrendDecayFactorRule** field can be set to "Use" indefinitely, as all levels of decay (no decay, some decay, or complete trend elimination) can be determined by the decay factor. However, using this field to control trend decay and elimination, rather than changing the decay factor value, means fewer manipulated records in the database.

The Holt-Winters multiplicative and additive models also include the capability to specify confidence levels to calculate prediction intervals when generating the statistical forecast. For more information, see "ForecastItemParameters table" on page 296 and "StatisticalForecast table" on page 1179

Linear

The linear forecast model performs a linear regression on a set of historical data points to calculate a trend line that determines the statistical forecast quantities; the forecast quantities are never less than zero. The linear model is best used for demand that does not contain seasonal variations.

The linear forecast model also includes the capability to specify confidence levels to calculate prediction intervals when generating the statistical forecast. For more information, see "ForecastItemParameters table" on page 296 and "StatisticalForecast table" on page 1179.

Moving average

The moving average model calculates a statistical forecast by taking the average of the actual historical demands from a specified number of prior periods. The number of periods considered by this model is determined by the setting in the **ForecastItemParameters.MovingAverageIntervalCount** field and expressed in terms of the calendar referenced by the **ForecastItemParametersType.IntervalsCalendar** field. Each new forecast period drops the oldest of the demands considered by the previous forecast, and replaces it with a newer one. For example, if the **MovingAverageIntervalCount** was set to "12" and the **IntervalsCalendar** was set to "Month", the forecast is calculated as the average of the previous twelve months of actual demand.

When fitting the actuals, the sum of all historical actuals (y) are divided by the interval count (k), up to the final actual demand point (N), as shown in the following formulas:

$$\mathbf{FittedValue}_{k+1} = \frac{y_1+\ldots+y_k}{k}$$

$$\mathbf{FittedValue}_{k+2} = \frac{y_2+\ldots+y_{k+1}}{k}$$

$$\mathbf{FittedValue}_{N} = \frac{y_{N-k}+\ldots+y_{N-1}}{k}$$

The forecast values are constant, calculated using the following formula:

$$\mathbf{Forecast} = \frac{y_{N-k+1}+\ldots+y_{N}}{k}$$

Because all previous data points being considered by this model are weighted equally, it is most suitable for demand that is stable and does not change much over time.

Step-wise ARIMA

The Step-wise ARIMA model is identical to the ARIMA model, but all parameters are calculated automatically. The difference level is calculated using Kwiatkowski-Phillips-Schmidt-Shin unit-root tests and parameter values are calculated to minimize errors according to Akaike information criterion values. Step-wise ARIMA fits values and generates forecasts for all lags. Any statistically insignificant lags are excluded from the calculations. Step-wise ARIMA should be used for forecasting few forecast item parameters, and is not suited for a bulk process.

The Step-wise ARIMA model also include the capability to specify confidence levels to calculate prediction intervals when generating the statistical forecast. For more information, see "ForecastItemParameters table" on page 296 and "StatisticalForecast table" on page 1179

Rforecast

The **Rforecast** model allows for the use of functions as defined in the R programming language to be

used in generating the statistical forecast. When you select the Rforecast model, RapidResponse passes the relevant historical actuals as well as a number of argument values to the "ets" or "forecast" function in R. The specific forecasting function in R that provides the best fit, based on the chosen error measure, is then used to generate the item's statistical forecast.

When using the **Rforecast** model, a number of arguments in the R "forecast" method take values from fields in the **ForecastItemParameters** table. The following fields have values that are either used by, or in determining, R arguments:

- **ConfidenceLevel**—sets the confidence level for prediction intervals.
- **ForecastIntervalCount**—defines the number of periods over which to forecast data.
- **SeasonalIntervalCount**—specifies the number of intervals in a season. If < 0, then seasonal models are disabled. If = 0, then R automatically detects seasonality. If > 0, then the value is used in running seasonal models.
- **TrendDecayFactor**—specifies the factor to use to dampen a trend. If < 0, then trend is ignored. If = 0, then R automatically detects trend and the forecast model can select decay factor as part of the fit. If > 0 and < 1, then the value is used for trend dampening. If >= 1, then trend dampening is ignored.

Additionally, this model allows passing a number of additional and completely optional argument values to named functions within the R language. For example, values might be passed to the optional "lamba" argument for the "ets" or "forecast" functions. To use optional arguments, records must be added to the **"RParameterName table" on page 1277** to specify the value to associate with a named argument when passed to an R function. Each **RParameter** record then has a reference to the **RParameterSet table** which groups related parameter values together. A given **ForecastItemParameters** record can then, optionally, reference a set of parameter values to pass to R through its **RParameterSet** reference (an **RParameterSet** reference also exists on **BestFitProfileDetail**). Otherwise, if the **RParameterSet** reference is Null, then no optional parameters are assumed and the "Rforecast" model passes just a standard set of parameters to R. Typically, R will select a reasonable model and produce a good forecast result using just these standard parameters (though, optional parameters can be provided as more advanced cases warrant).

The following table lists the arguments that are valid for each of the R functions in RapidResponse:

ets	forecast
• model • alpha • beta • gamma • additive.only • lambda • biasadj • opt.crit • nmse • bounds • ic • restrict • allow.mulitplicative.trend	• lambda • allow.mulitplicative.trend • model

Notes:

- The following values are ignored if defined as parameter values in the **RParameter** tables (most are automatically passed to R based on data and field values in RapidResponse): damped, fan, find.frequency, h, level, lower, object, phi, upper, y.
- The Rforecast model and supporting R tables are only available if the optional Rforecast capability has been enabled. For more information about this optional capability, see "R integration" on page 168.

Univariate time series forecast model parameters

When you configure a statistical forecast for a forecast item, you need to set various parameters to improve the accuracy of the forecast. These parameters are stored in the "ForecastItemParameters table" on page 296. Some of these parameters apply only to certain forecast models. The following table lists all of the Univariate time series forecast models and the parameters that correspond to each model:

Note: For information about the parameters that are applicable to multiple linear regression forecast models, see "MLR forecast model parameters" on page 2043.

	ARIMA	Best Fit	Croston's	Double-exponential smoothing	Exponential smoothing	Holt-Winters	Linear	Moving average	Step-wise ARIMA	R
AutoRegressive Terms	•									
BestFitForecastLag		•								
BestFitHoldout PeriodInterval Count		•								
Constant	•									
ConfidenceLevel	•			•	•	•	•		•	•
DifferenceLevel	•									
MovingAverage IntervalCount		•						•		
MovingAverage Terms	•									
RParameterSet										•
SeasonalInterval Count		•				•				•
TrendDecayFactor				•		•				•

Multiple Linear Regression forecasting models

RapidResponse includes multiple linear regression (MLR) forecasting models to generate statistical forecasts. MLR is a statistical forecasting method that examines multiple regressors to predict the future demand of a forecast item. A regressor, which is also known as an independent variable, causal factor, or predictor, is a factor that may drive the demand for a forecast item. The regressors that you use to determine the future demand for the forecast item in the statistical forecast could include product promotions, holidays, weather, or any factor that is known to drive the demand for a product.

A series of RapidResponse data model tables and fields support the use of multiple linear regression models with statistical forecasting. There are also some calculation considerations with regards to these tables and fields.

The MLR forecasting models use the R programming language to perform calculations. Therefore, MLR forecasting models are available only if R has been enabled. It is also important to note that R is available only with the On-Demand RapidResponse Service. For more information, see "R integration" on page 168.

This section includes the following information:

- "List of multiple linear regression models" on page 2042
- "MLR forecast model parameters" on page 2043
- "Multiple linear regression tables" on page 2044
- "Multiple linear regression calculations" on page 2047

List of multiple linear regression models

RapidReponse includes the following MLR forecasting models:

- "ARIMAX" on page 2042
- "ElasticNet" on page 2043
- "Multiple linear regression" on page 2043

ARIMAX

The ARIMAX is a time series forecast model is an extension of the ARIMA model, but also includes independent variables (regressors) in the forecast calculation. This makes ARIMAX a multiple linear regression model. To improve forecast accuracy, ARIMAX also takes into consideration autocorrelation that may be found in regressor residuals.

Like ARIMA, you can specify the difference level, which determines how many differencing functions are performed on the data. The differencing function can be written $Y(t) - Y(t-1)$ for each point, where *Y* is the data value and *t* is the time period. This function is applied to the data as many times as specified. Unlike ARIMA and more like StepwiseARIMA, the number of autoregressive and moving-average terms is automatically selected and cannot be set manually.

After the differencing function is performed, an Auto-Regressive Moving Average (ARMAX) calculation is performed using the following formula:

$$Y_t = \beta_1 x_{1,t} + \beta_2 x_{2,t} + \ldots + \beta_r\, x_{r,t} + \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} - \theta_1 z_{t-1} - \ldots - \theta_q z_{t-q} + z_t$$

Where x_t is an independent variable at time *t* and β is its coefficient, θ is the coefficient of the moving average function, *t* is the time, Y is the actual value at point *t*, z_t is a white noise error process at point *t*, *p* is the number of auto-regressive terms, and *q* is the number of moving average terms.

ElasticNet

The ElasticNet model is a multiple linear regression model that uses regularized, also known as penalized, regression methods. It blends Lasso and ridge regression techniques by penalizing both L1 and L2 norms to reduce the coefficient vector. To use ElasticNet, you must attach a minimum of two regressors to a forecast item or you won't be able to generate a statistical forecast.

Note: If the ElasticNet model two or more regressors, two of the regressors must be without coefficients provided in ForecastItemParametersRegressionCoefficients. If you want to provide all coefficients as input in this table, then you must switch from ElasticNet to MultipleLinearRegression forecast model.

ElasticNet is calculated using the following equation:

$$\widehat{\beta} \equiv \frac{argmin}{\beta} \| y - X\beta \|^2 + \lambda \left(\alpha ||\beta||^2 + (1-\alpha)||\beta||_1 \right)$$

Where y represents historical demand actuals, $\|y - X\beta\|^2$ is the sum of residuals squared, $\lambda \left(\alpha ||\beta||^2 + (1-\alpha)||\beta||_1 \right)$ is the regularization term, where λ is the regularization parameter, and where α is the ElasticNet parameter.

Multiple linear regression

The multiple linear regression model performs a linear regression using multiple independent variables to predict the outcome of one dependent variable. To use the multiple linear regression forecast model, you must attach a minimum of one regressor to a forecast item or you won't be able to generate a statistical forecast. Multiple linear regression is calculated using the following formula:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} + E$$

Where y_i represents the historical demand actuals, x_{i1} and x_{i2} represent independent variables (regressors), E represents the residuals, β_0 represents the y-intercept at time zero, β_1 represents regression coefficient for regressor x_{i1}, and β_2 represents regression coefficient for regressor x_{i2}.

MLR forecast model parameters

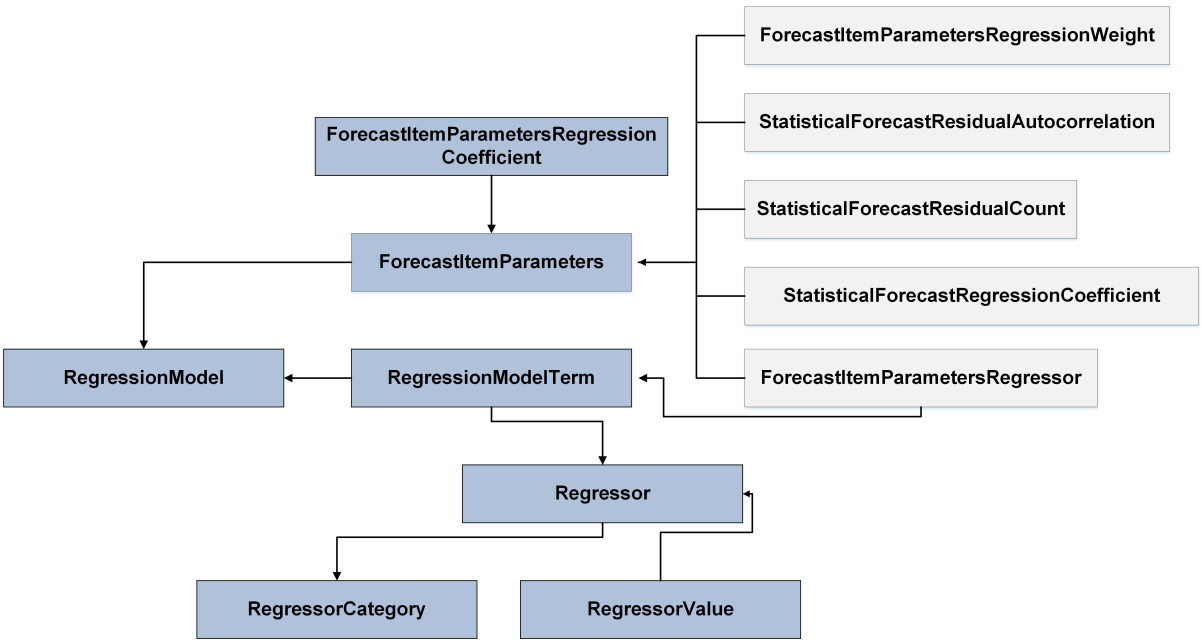
When you configure a forecast item to use a specific forecast model, you need to set various parameters to improve the accuracy of the forecast. These parameters are stored in the "ForecastItemParameters table" on page 296. Some of these parameters apply only to certain forecast models. The following table lists all of the MLR forecast models and the parameters that correspond to each model:

	ARIMAX	Elastic Net	Multiple Linear Regression
AutoRegressiveTerms			
BestFitForecastLag			
BestFitHoldoutPeriodIntervalCount			
Constant	•		
ConfidenceLevel	•		•
DifferenceLevel	•		
ElasticNetParameter		•	
MovingAverageIntervalCount			
MovingAverageTerms			
RegularizationParameter		•	
ResidualAutocorrelationConfidenceLevel	•	•	•
RParameterSet	•	•	•
SeasonalIntervalCount		•	•
TrendDecayFactor		•	•
WeightsDecayFactor		•	•

Note: For information about the parameters that are applicable to univariate forecast models, see "Univariate time series forecast model parameters" on page 2041.

Multiple linear regression tables

The RapidResponse data model includes a group of tables that support statistical forecasting using multiple linear regression. The following illustration shows the input tables (blue) and calculated tables (grey) that were added for multiple linear regression and how they relate to the ForecastItemParameters table and each other. If a table includes a reference field, the arrow in the diagram points to the table that is being referenced. The ForecastItemParameters table contains the input parameters, including any parameters related to MLR, that are used in generating a statistical forecast for a given forecast item.



Input tables

The following table lists the input tables that were added for MLR and provides a short description of each table. The tables that were added store information about regressors, also known as independent variables, that are used when generating a statistical forecast with a multiple linear regression model.

Table	Description
ForecastItemParametersRegressionCoefficient	Saves the regression coefficients that are generated by **StatisticalForecastRegressionCoefficient**, which is a calculated table.
RegressionModel	Stores the names of regression models that are defined for use with MLR statistical forecasting models.
RegressionModelTerm	Stores information about the model terms that are associated with regressors.
Regressor	Stores records for regressors that are used with MLR statistical forecasting models.
RegressorCategory	Stores regressor categories.
RegressorValue	Stores individual values for a regressor on a given date.

Calculated tables

The following table lists the calculated tables that were added for MLR and provides a short description of each table. The tables contain values that are associated with regressors with mutliple linear regression models, such as regression regression weights and coefficients.

Table	Description
StatisticalForecastRegressionCoefficient	Contains the results of calculations of the coefficients for forecast items that use an MLR forecast model.
ForecastItemParametersRegressionWeight	Reports the regression weights for forecast items that are using an MLR forecast model.
ForecastItemParametersRegressor	Contains the values for regressors that are associated with each **RegressionModelTerm**.
StatisticalForecastResidualAutocorrelation	Reports the autocorrelation function of the residuals for forecast items that use an MLR forecast model.
StatisticalForecastResidualCount	Reports the number of observations that have residuals with quantities that are within a particular range, or bin.

Other tables

The following table lists other tables that include fields that are related to MLR and provides a list of the MLR-related fields that were added.

Table	MLR-related fields
BestFitProfileDetail	DifferenceLevel ElasticNetParameter ForecastModel RegularizationParameter SeasonalIntervalCount TrendDecayFactor
ForecastItemFitOutput	ForecastModel

Table	MLR-related fields
ForecastItemParameters	ConfidenceLevel Constant DifferenceLevel ElasticNetParameter RegressionModel RegularizationParameter ResidualAutocorrelationConfidenceLevel SeasonalIntervalCount TrendDecayFactor WeightsDecayFactor
ForecastItemParametersFitOutput	ForecastModel
ForecastItemParametersType	ForecastModel ModelConstantUsage
StatisticalForecastFitOuput	ForecastModel

Multiple linear regression calculations

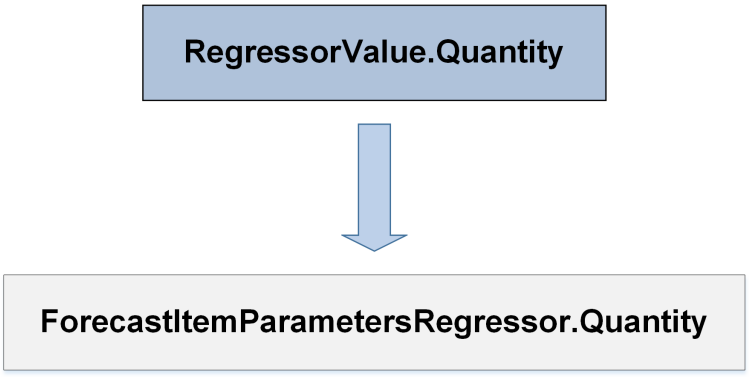
When you apply a multiple linear regression forecast model to a forecast item, RapidReponse performs calculations on the regressors to generate a statistical forecast for this item. This section describes the calculations that are specific to the regressors that are used with multiple linear regression forecast models.

In this section, you will learn:

- "How regressor values are calculated" on page 2047
- "How regressor weight decay is calculated" on page 2049

How regressor values are calculated

When you include a regressor, such as past promotions or holidays, in a statistical forecast, the dates for the regressor values may be intermittent. These values and their associated dates are reported in the **RegressorValue** table. The values in the **RegressorValue** table are then used by the **ForecastItemParametersRegressors** table to calculate **ForecastItemParametersRegressor.Quantity**, as shown in the following illustration.



The **ForecastItemParametersRegressor.Quantity** is then bucketed using **ForecastItemParametersRegressor.Date**, which uses the same date as **ForecastItemParametersActual.Date**. Therefore, there is a calculation to determine the regressor values in **ForecastItemParametersRegressor.Quantity** bucket.

The **ForecastItemParametersRegressor.Quantity** is the time-weighted average of the **RegressorValue.Quantity** records that are in the interval between two **ForecastItemParametersActual** dates. However, if within this interval there are dates with no **RegressorValue** records, a **RegressorValue.Quantity** remains in effect starting at **RegressorValue.Date** until one day prior to **RegressorValue.Date** of the next available **RegressorValue** record. Therefore, even if the next regressor value record occurs only after the following **ForecastItemParametersActual.Date**, the value is still taken into consideration in the **ForecastItemParametersRegressor.Quantity** calculation for the new period bucket. The formula used to determine **ForecastItemParametersRegressor.Quantity** is as follows:

> **ForecastItemParametersRegressor.Quantity** = [(**D**/**DM**)*__RegressorValueA__ + (**D**/**DM**)*__RegressorValueB__]
>
> Where **D** represents the number of days the **RegressorValue.Quantity** is in effect in a given month and **DM** represents the number of days in the given month.

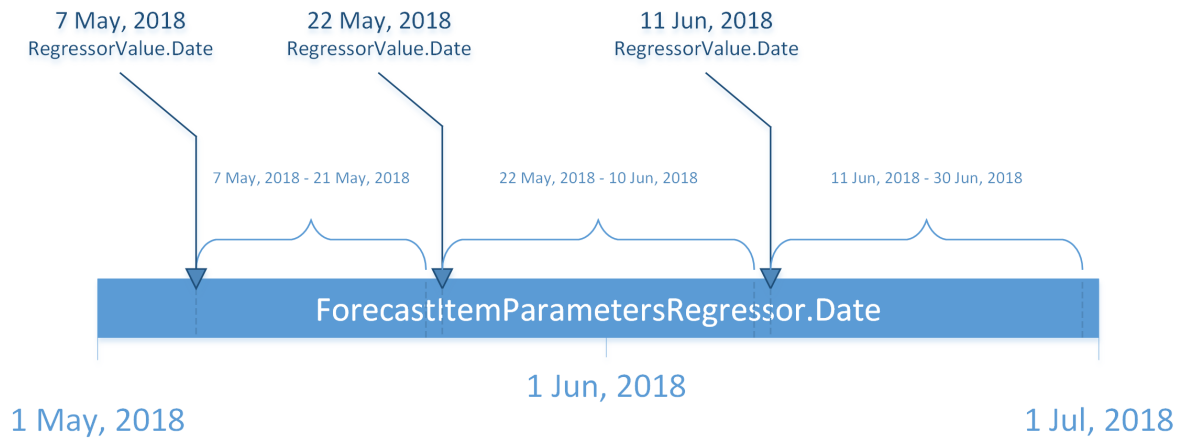
If there is no **RegressorValue** record associated with a date that is equal to, or earlier than the first **ForecastItemParametersActual.Date** in the historical data series, then a **RegressorValue.Quantity** of zero is applied.

It is also important to note that the values stored in this table are not only historical values. The values within the following range are taken into account in the **ForecastItemParametersRegressor.Quantity** calculation:

RunDate - **ForecastItemParameters.HistoricalIntervalCount** to **RunDate** + **ForecastItemParameters.ForecastIntervalCount**.

However, if **ForecastItemParameters.HistoryStartDate** is after **RunDate** - **ForecastItemParameters.HistoricalIntervalCount**, then the first date that is reported is **ForecastItemParameters.HistoryStartDate**. If **ForecastItemParameters.HistorySkipLeadingZeroes** is also set to **Y**, and there are leading zeroes in the historical demand data, then only the regressor values after the first non-zero value are considered. In other words, if there are regressor values whose dates match historical demand period buckets that have leading zeros, these regressor values are ignored.

The following illustration shows how three individual **RegressorValue.Quantity** records span one **RegressorValue.Date** to another. The regressor values are reported on the following dates: May 7, 2018, May 22, 2018, and June 11, 2018. The illustration also shows the effective period for each regressor value. A time-weighted average of the values is then calculated for **ForecastItemParametersRegressor.Quantity** values, which is bucketed using the following **ForecastItemParametersRegressor.Date:** May 1, 2018, Jun 1, 2018, and Jul 1, 2018.



Now, using the example above, suppose we apply the following **RegressorValue.Quantity** for each **RegressorValue.Date**:

RegressorValue.Date	RegressorValue.Quantity	Effective Days	Used in calculation for period bucket
7-May-18	1	15 days in May (May 7 to May 21)	1-May-2018
22-May-18	3	10 days in May 10 days in June (May 22 to June 10)	1-May-2018 and 1-Jun-2018
11-Jun-18	2	20 Days in June (June 11 to June 30)	1-Jun-2018

Using the **RegressorValue.Quantity** in the example above, RapidReponse would calculate a time-weighted average for these values using the following formulas:

ForecastItemParametersRegressor.Quantity for May 1 = [(15/31)*1 + (10/31)*3]

ForecastItemParametersRegressor.Quantity for June 1 = [(10/30)*3 + (20/30)*2]

The results of the calculations would be as follows:

ForecastItemParametersRegressor.Date	ForecastItemParametersRegressor.Quantity
1-May-18	1.45
1-Jun-18	2.33

How regressor weight decay is calculated

You can set a regression weight decay factor for a regressor to adjust the relative influence of regressors over time in the statistical forecast. When you set a weight decay factor, the regressor values in more recent period buckets are given more importance than the values in earlier period buckets. You

can set a weight decay factor using **ForecastItemParameters.WeightsDecayFactor**. When a weight decay factor is set, RapidResponse calculates the weights that are applied to the regressor values and reports them in the **ForecastItemParametersRegressionWeight**, as shown in the following illustration:

Each record stores the regression weight used for one data point (**ForecastItemParametersRegressionWeight.Quantity**), which represents one **HistoricalIntervalCount** period bucket (**ForecastItemParametersRegressionWeight.Index**) starting at **RunDate**.

The weights reported in the **ForecastItemParametersRegressionWeight** table are then used in the calculation of the statistical forecast for the forecast item. They are also taken into account in the following:

- Fitted value reported in **StatisticalForecastPredictActual.Quantity.**
- Residual value (**StatisticalForecastPredictActual.Quantity** - **StatisticalForecastPredictActual.PredictedQuantity**), including the **StatisticalForecastResidualCount** table.

The **ForecastItemParametersRegressionWeight.Quantity** is calculated using the following formula:

For the **t-th** observation in the historical set

ForecastItemParametersRegressionWeight.Quantity = **WeightsDecayFactor**^ (**HistoricalIntervalCount** - t)

The following table illustrates the calculated regressor weights used when **WeightsDecayFactor** is set to .98 and .95 for a forecast item with a **HistoricalIntervalCount** set to 24 months.

Date	t (index of observation)	ForecastItemParametersRegressionWeight.Quantity WeightsDecayFactor .98	ForecastItemParametersRegressionWeight.Quantity WeightsDecayFactor .95
Jan-2017	1	0.63	0.31
Feb-2017	2	0.64	0.32

Mar-2017	3	0.65	0.34
Apr-2017	4	0.67	0.36
May-2017	5	0.68	0.38
Jun-2017	6	0.70	0.40
Jul-2017	7	0.71	0.42
Aug-2017	8	0.72	0.44
Sep-2017	9	0.74	0.46
Oct-2017	10	0.75	0.49
Nov-2017	11	0.77	0.51
Dec-2017	12	0.78	0.54
Jan-2018	13	0.80	0.57
Feb-2018	14	0.82	0.60

Mar-2018	15	0.83	0.63
Apr-2018	16	0.85	0.66
May-2018	17	0.87	0.70
Jun-2018	18	0.89	0.74
Jul-2018	19	0.90	0.77
Aug-2018	20	0.92	0.81
Sep-2018	21	0.94	0.86
Oct-2018	22	0.96	0.90
Nov-2018	23	0.98	0.95
Dec-2018	24	1.00	1.00

Additional forecasting options

RapidResponse includes additional forecasting options Best Fit and Smart Forecast Selection that analyze the historical actual data for a forecast item and then automatically select one of the available forecast models that best suits the data. In addition, you can import a statistical forecast that was calculated using an external tool or application into RapidResponse.

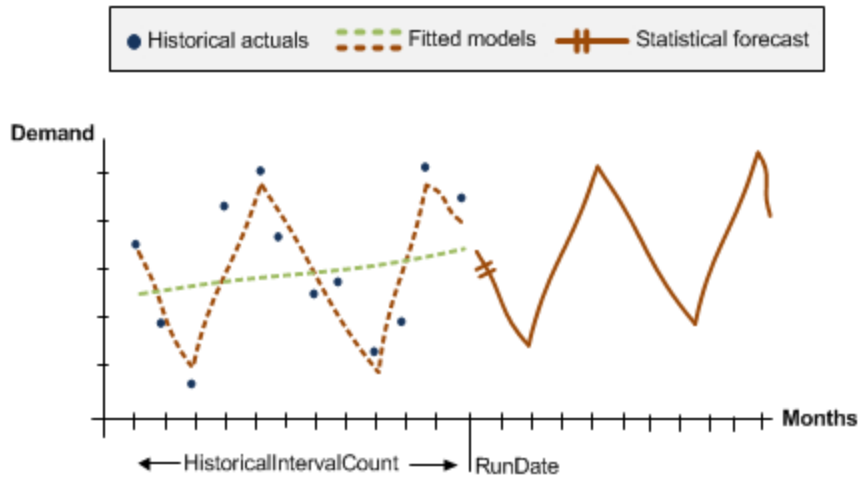
- "Best fit" on page 2053
- "ForecastImport" on page 2058

Best fit

The best fit model fits each of the other standard forecast models to historical actual data and then selects the model with the least error for use in generating the statistical forecast.

The best fit model is applied to forecast items that reference a value of "BestFit" in the **ForecastItemParametersType.ForecastModel** field, and the error measure applicable to a given item is set by the **ForecastItemParametersType.FitMeasure** field. The selected fit measure is then used to evaluate the difference between the actual historical data points and the fitted values generated by the model. For example, the "MeanError" fit measure chooses the model that produces the lowest average variance between the fitted points and the actual historical data points. The model with the lowest margin of error, based on the chosen fit measure, is then selected and used to generate the item's statistical forecast at RunDate as shown in the following figure.

Figure 43.6 BestFit forecast model selection



The item's statistical forecast as generated from the selected model is then reported in the **StatisticalForecast** table, and subsequently disaggregated to the part-customer level and reported in the **StatisticalForecastDetail** table. For information about the disaggregation process, see "Forecast disaggregation" on page 2083. Additionally, error measures and other details pertaining to the fitting process are reported for the selected forecast model in the **StatisticalForecastFitOutput** table. This table also identifies the forecast models that were the second and third best fits (had the next lowest error margins).

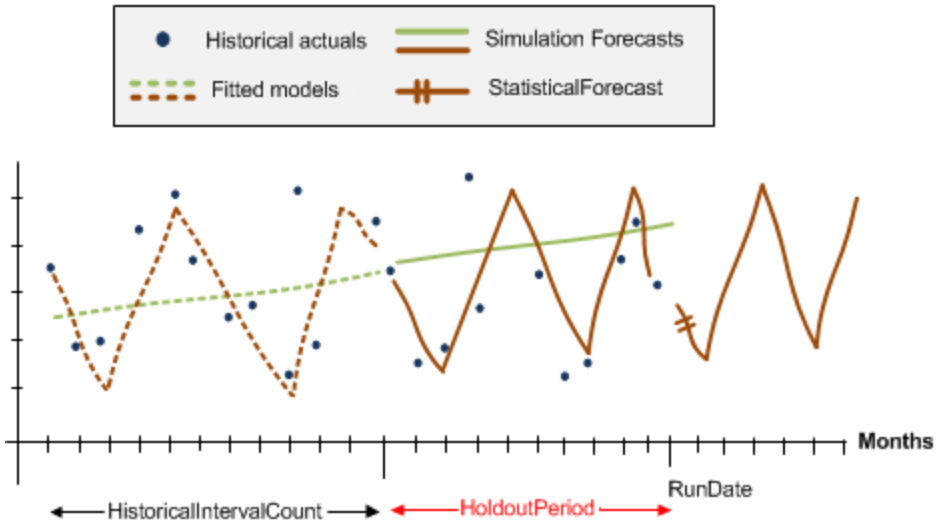
By default all historical data within the **HistoricalIntervalCount** specified for the item is used during the best fit process, and all statistical models are evaluated. Optionally, a specified portion of the historical data can be held out of the sample and used for simulation purposes, while profiles can be created to identify the specific models that should be used during the best fit process.

Using best fit with a holdout period

A near-term portion of the historical data set can, optionally, be held out of the sample used to fit statistical models during the best fit process. Instead this holdout region is used for testing purposes as the statistical models are fitted based on the earlier historical data and then used to generate forecasts in the holdout region.

Subsequently, the model whose forecast most closely predicts the actuals in the holdout region is then selected and used to generate the statistical forecast at RunDate as shown in the following figure.

Figure 43.7 BestFit forecast model selection with holdout



This means the selected **FitMeasure** evaluates the forecast models based not on how well they fit known data, but rather on how accurately they can predict data in future periods.

Before generating the statistical forecast, the selected model is refitted based on all applicable history including the holdout period. For example, suppose three years of historical data is used with the last of those years held out of the sample. In this case, the statistical models would be fitted against the first two years of history, and then used to generate a simulated forecast to be compared against the actuals in the third year. The model whose forecast resulted in the lowest forecast error would be refitted based on all three years of history and then used to generate the statistical forecast.

Define the holdout period

To define a holdout period, the following two fields on the **ForecastItemParameters** table are used:

- **BestFitHoldoutPeriodIntervalCount** —specifies the duration of the holdout period and is expressed in **SOPAnalyticsConfiguration.CycleCalendar** intervals. For example, assuming the **CycleCalendar** is "Month", then a value of "6" indicates to hold out the most recent six months of history when fitting the models, and instead use the actuals in those six months for the purpose of measuring the accuracy of the simulated forecast values.
- **BestFitForecastLag—**specifies the interval in the holdout period from which the simulated forecast is first evaluated, and is expressed in intervals of the **SOPAnalyticsConfiguration.CycleCalendar**. By default, this field is set to "1" which refers to the first interval in the holdout period. Optionally, this value can be changed to offset the calculation of forecast error later into the holdout period (typically, this might be done to account for lead time of the item).

Together, these two fields thereby define the cycle calendar intervals used for evaluating simulated forecasts produced by fitted models. For example, if **CycleCalendar** is "Month", **BestFitHoldoutPeriodIntervalCount** is 6", and **BestFitForecastLag** is "3", then forecast errors are calculated in the last four months of the holdout period.

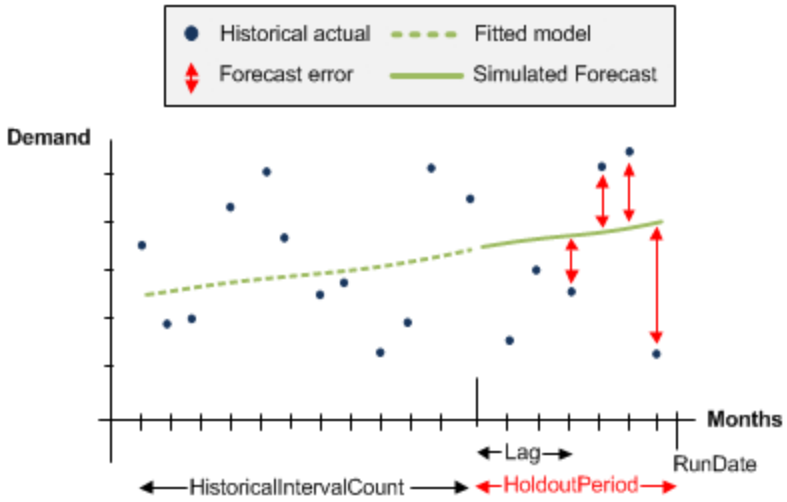
Enable holdout period usage

The actual usage of the holdout period in best fit forecasting is controlled by the **HoldoutPeriodUsageRule** setting on the **ForecastItemParametersType** table. When this field is set to "Ignore" (the default setting), then the holdout period is not used in best fit forecasting for the item.

In order to enable use of the holdout period, the **HoldoutPeriodUsageRule** must be set to either "SimpleHoldout" or "Simulation". These two options differ in how they generate model parameters and hence generate simulated forecasts for use in calculating forecast error for the different models.

When the "SimpleHoldout" option is selected, each model is fitted once against the range of historical data before the holdout period, and a single set of model parameters is generated. These parameters are then used to calculate a simulated forecast in the holdout period. The points on the simulated forecast are compared against the known actuals in each interval of the holdout period in order to calculate forecast error.

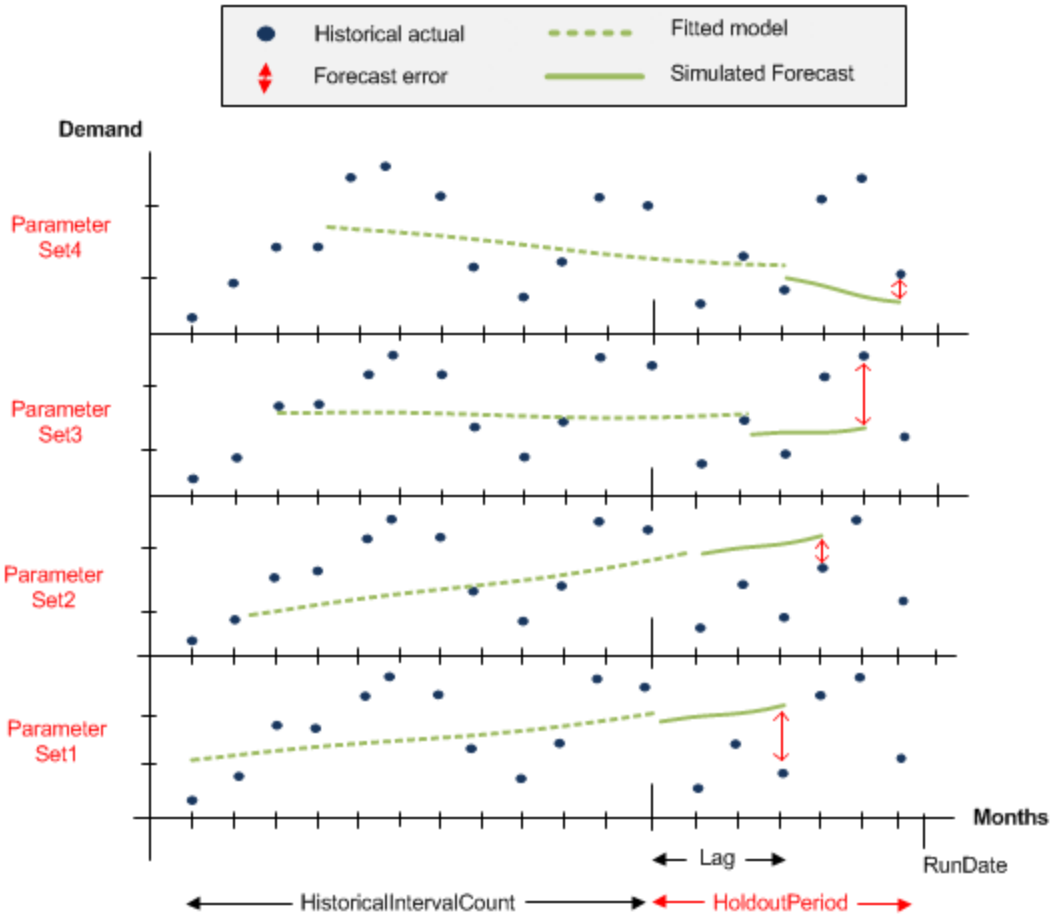
For example, assume the applicable **CycleCalendar** and **IntervalsCalendar** values are both set to "Month", **BestFitHoldoutPeriodIntervalCount** is set to "6", and **BestFitForecastLag** is set to "3". In this case, forecast errors for a given model using "SimpleHoldout" might be calculated as seen in the following illustration.



When the "Simulation" option is selected, each model is fitted multiple times against a rolling window of historical data that starts with the same range of data used for "SimpleHoldout", but which ultimately moves into the holdout region itself. This results in multiple sets of fitted parameters generated for each model. Each of the model's parameter sets then generates a simulated forecast value that is used to calculate forecast error for a specific interval in the holdout period.

For example, assume again the applicable **CycleCalendar** and **IntervalsCalendar** values are both set to "Month", **BestFitHoldoutPeriodIntervalCount** is set to "6", and **BestFitForecastLag** is set to "3". In this case, forecast errors for a given model using "Simulation" might be calculated as seen in the following illustration with four different sets of fitted parameters determined for the model. Note that as the fitted parameter sets are calculated based on the later periods of history are used in calculating forecast for correspondingly later periods in the holdout period.

Figure 43.8 Simulation and forecast error



Holdout fit measures

When using the "BestFit" model with a holdout period, forecast errors are generated based on the difference between the historical actuals in the holdout period and the forecast values the model predicts in that holdout period.

Specific error measures are available in **ForecastItemParametersType.FitMeasure** for measuring errors based on difference between actuals in the holdout period and the predicted forecast values. These error measures are "LagPercentageError", which uses differences between the actuals and simulated forecast values in the first interval of the holdout period only, and "RollingLagPercentageError", which uses differences between the actuals and simulated forecast values in a configurable number of holdout period intervals (based on **ForecastItemParameters.RollingErrorIntervalCount**). In each case, the error is expressed as a percentage of all actuals in the interval(s) being considered.

For more information about these fit measures, see "Lag percentage error" on page 2063 and "Rolling lag percentage error" on page 2067.

Applying best fit profiles

By default, all standard statistical models are evaluated during the best fit process and applicable model parameters, such as **HistoricalIntervalCount**, are taken from the item's **ForecastItemParameters** record. Instead, profiles can be defined that specify a subset of models to be considered during the best fit

process along with the parameters to be used when fitting each of those models. For example, if a subset of models are known to work best for a given forecast item, then a profile made up of just those models can be created and applied to the item. This can eliminate processing time spent by the analytic in evaluating models that are unlikely to produce a good fit with the item's data. Additionally, it is possible for a profile to contain multiple instances of the same forecast model with each instance specifying different parameters to use in the fitting process. For example, separate instances of the "HoltWintersMethod" might be added to a profile with each defining different seasonal intervals.

To define best fit profiles, the **BestFitProfile** and **BestFitProfileDetail** tables are used. The **BestFitProfile** table simply identifies each named profile and is referenced by the **BestFitProfileDetail** table. Each record in the **BestFitProfileDetail** table then identifies a specific statistical model along with a set of applicable parameters to be used in fitting the model. Thus, each **BestFitProfile** entry is made up all models and associated parameters that reference it from records in the **BestFitProfileDetail** table.

In order to use a profile in best fit calculations, a forecast item must reference a named profile through the **BestFitProfile** reference on the **ForecastItemParameters** table. When this reference is populated, the forecast item only considers models that belong to the referenced profile as shown in "Tables supporting best fit profiles" on page 2057.

For example, if a **ForecastItemParameters** record references a profile, the **HistoricalIntervalCount** value used for fitting each model comes from the appropriate **BestFitProfile** record and the **HistoricalIntervalCount** value on the **ForecastItemParameters** record is ignored. Otherwise, if the **BestFitProfile** reference is left "Null", then all statistical models are considered during the best fit process and the applicable model parameters from the **ForecastItemParameters** table are used.

Figure 43.9 Tables supporting best fit profiles

In cases where profiles are used in best fit forecasting, the actual **BestFitProfileDetail** record that defined an item's chosen forecast model and parameters is reported in the **CalcBestFitProfileDetail** reference on the **ForecastItemParameters** table. Optionally, if satisfied with the chosen model, it can be locked to the item by adding a reference to that same record in **ForecastItemParameters.BestFitProfileDetail**. This prevents any other models from being selected for the item in subsequent best fit processes.

Note: There may be instances where you want a forecast item to reference a group of statistical models that are defined in **BestFitProfile**, but you do not want to use the model parameter values that are associated with the profile in the **BestFitProfileDetail** table and apply the values from **ForecastItemParameters** instead. To do this, you can set **ForecastItemParameters.IgnoreBestFitProfileDetailParameters** to **"Y"** which ignores the model parameter values in the **BestFitProfileDetail** table and applies the **ForecastItemParameters** values. However, if a profile contains multiple instances of the same forecast model, ignoring the **BestFitProfileDetail** parameters will apply the same parameter values to all instances of the model. See "IgnoreBestFitProfileDetailParameters" on page 302.

Note: The optional "Rforecast" model is not considered alongside the standard models evaluated by the best fit process for items that have a null **BestFitProfile** reference. Therefore, profiles must be set up for any items that are meant to consider "Rforecast" in the best fit forecasting process.

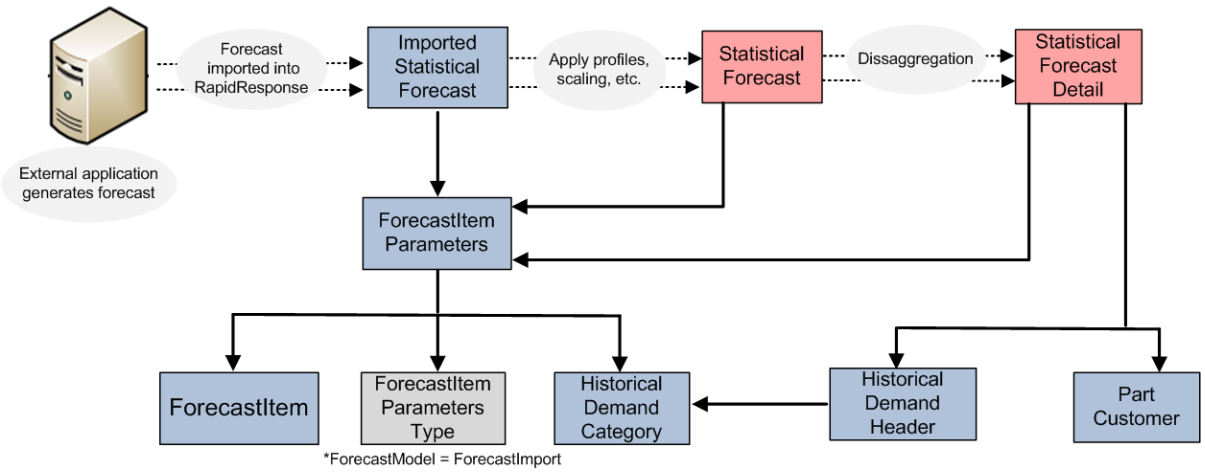
ForecastImport

Although RapidResponse provides many forecasting models that can be used when generating a statistical forecast, it is also possible to import a statistical forecast that was calculated using an external tool or application. Statistical forecasts imported into RapidResponse can then make use of the same subsequent forecasting logic and calculations that are applied to generated statistical forecasts. For example, profiles and scaling factors can be applied to the imported statistical forecast before it is moved to the **StatisticalForecast** table and disaggregation logic can be applied before detailed forecast records at the part-customer level are added to the **StatisticalForecastDetail** table.

In order to make use of an imported statistical forecast, an item's applicable **ForecastItemParameters** record must reference a value of "ForecastImport" in **ForecastItemParameters**.**Type.ForecastModel**. The item's statistical forecast values can then be imported into the **ImportedStatisticalForecast** table. Each record in this table should reference the appropriate **ForecastItemParameters** record for the item and identify a **Date** and **Quantity** for the forecast point. Each **Date** value should resolve to a date on the **ForecastItemParameters.Type.IntervalsCalendar** (imported dates not belonging to that calendar are then rounded back to the nearest previous date that does belong to that calendar). Additionally, forecast values should be provided for each **IntervalsCalendar** marker from the forecast start date (on or after **SOPAnalyticsConfiguration.CalcHistoricalEndDate)** and over the number of specified **ForecastItemParameters.ForecastIntervalCount** intervals. Missing **IntervalsCalendar** dates in this range are flagged with a warning, and dates outside of this range are ignored by forecast processing logic.

The following illustration shows the relationship between the basic tables used in importing the statistical forecast and subsequent processing and reporting on those imported forecast values.

Figure 43.10 Statistical forecast imported into RapidResponse



Note: For more information about the disaggregation logic that is applied to statistical forecast values whether they be imported or generated in RapidResponse, see "Forecast disaggregation" on page 2083.

Data characteristics

The **StatisticalForecastFitOutput** table reports data characteristics, which are calculations that provide useful information about your input data points. Because each forecast model is designed for unique data conditions, you can use data characteristics to choose the best model for your forecast. Data characteristics are sometimes used to calculate other data characteristics or error measures. The following data characteristics can be reported:

- "Count" on page 2059
- "Mean date" on page 2059
- "Mean quantity" on page 2059
- "Total sum of squares" on page 2060
- "Variance" on page 2060
- "Standard error" on page 2060

Count

The count data characteristic is the number of values in your input set. The count is used by most other data characteristics and error measures to calculate averages.

Mean date

The mean date data characteristic is the average of all dates included in the statistical forecast. For this calculation, the date values are converted to integers and averaged; the result is converted back to a date.

The mean date can indicate if periods are missing from the statistical calculations, and to verify that the right dates are used. Also, it determines the approximate midpoint of the dates used in the statistical calculations.

The mean date is calculated using the following formula, where *X* is the integer value of each date:

$$\frac{\sum_{i=1}^{\text{Count}} x_i}{\text{Count}}$$

Mean quantity

The mean quantity data characteristic is the average of all actual values in the input data. It is used in other measures to determine how a value varies from the average or as a baseline for comparisons.

The mean quantity is calculated using the following formula, where y or y_i is the quantity at each data point:

$$\frac{\sum_{i=1}^{\text{Count}} y_i}{\text{Count}}$$

Total sum of squares

The total sum of squares data characteristic is the total difference between each data point and the mean, which is calculated using the mean quantity of the data.

The total sum of squares is calculated using the following formula, where y or y_i is the quantity at each data point:

$$\sum_{i=1}^{\text{Count}} \left(Y_i - \text{MeanQty}\right)^2$$

Variance

The variance data characteristic reports how data values differ from the mean. It is used to show how uniform or dispersed the input data is, and can help you determine which statistical forecast model to use. For example, if the variance is low, you can use the linear model.

The variance is calculated using the following formula:

$$\frac{\text{TotalSumSquares}}{\text{Count}-1}$$

Standard error

The standard error data characteristic reports how values differ from the mean. This characteristic is most relevant if the input data is level, with little variance.

The standard error is calculated using the following formula, where *X* represents dates and y represents quantities:

$$\sqrt{\frac{1}{\text{Count}-2} \times \left(\left(\sum_{i=1}^{\text{Count}} \left(y_i - \text{MeanQty}\right)^2\right) - \left(\frac{\left(\sum_{i=1}^{\text{Count}} \left(y_i - \text{MeanQty}\right) \times \left(x_i - \text{MeanDate}\right)\right)^2}{\sum_{i=1}^{\text{Count}} \left(x_i - \text{MeanDate}\right)^2}\right)\right)}$$

Error measures

In any statistical calculation, some margin of error exists; minimizing the margin of error is crucial when producing an accurate statistical forecast. Error measures can help you determine which forecast model has the least margin of error. Error measures fit forecast values against the input data points, allowing you to see how accurately the predicted values match the input values.

The error measures are by the reported by the **StatisticalForecastFitOutput** table and some error measures are also available when selecting an option in the **ForecastItemParametersType.FitMeasure** field for the purposes of choosing a forecast model when using the "BestFit" model.

The following table lists the error measures are reported by the **StatisticalForecastFitOutput** table and which are available from the **ForecastItemParametersType.FitMeasure** field when using the "BestFit" model. For more information about each error measure, click the name of the error measure.

Error measure	StatisticalForecastFitOutput	ForecastItemParametersType.FitMeasure
"Adjusted R-squared" on page 2062	•	•
"Akaike information criterion (AIC)" on page 2062	•	
"Forecast error" on page 2063	•	
"Hannan-Quinn criterion (HQC)" on page 2063	•	
"Log-likelihood" on page 2064	•	
"Lag percentage error" on page 2063		•
"Mean absolute error" on page 2064	•	•
"Mean absolute percentage error" on page 2064	•	•
"Mean absolute percentage error by forecast" on page 2064	•	
"Mean absolute error by mean" on page 2065	•	•
"Mean absolute scaled error (non-seasonal)" on page 2065	•	•
"Mean error" on page 2066	•	•
"Mean percentage error" on page 2066	•	•
"Mean percentage error by forecast" on page 2066	•	
"Mean square errors" on page 2066	•	•

Error measure	StatisticalForecastFitOutput	ForecastItemParametersType.FitMeasure
"Regression sum of squares" on page 2067	•	•
"Residual sum of squares" on page 2067	•	•
"Rolling lag percentage error" on page 2067	•	•
"Root mean square error" on page 2068	•	•
"R-squared" on page 2067	•	•
"Schwarz Bayesian information criterion (SBC)" on page 2068	•	
"Symmetric mean absolute percentage error" on page 2069	•	•
"Variance error" on page 2069	•	
Weighted MAPE (see "Mean absolute error by mean" on page 2065)		

Adjusted R-squared

The Adjusted R-squared measure determines the ratio between the regression sum of squares and total sum of squares measures, similarly to the R-squared measure. However, the adjusted R-squared measure is influenced by the number of values in the input data set, so that more values mean that any one value has less of an effect on the ratio. In other words, the adjusted R-squared measure is less responsive when more values are contained in the input.

The adjusted R-squared measure is calculated using the following formula, where p=1, as p represents the number of independent variables used in the input data (the only independent variable is the date):

$$1 - \left(1 - \mathrm{RSquared}\right) \times \frac{\mathrm{Count} - 1}{\mathrm{Count} - p - 1}$$

To use the adjusted R-squared measure to calculate the best fit, select "AdjustedRSquared" in the **ForecastItemParametersType.FitMeasure** field.

Akaike information criterion (AIC)

The Akaike information criterion measure determines how well an ARIMA model fits the values it was generated from, using the log-likelihood result and double the number of estimated parameters. Models that produce a smaller value using this measure are preferred.

The AIC is calculated using the following formula, where k is the number of parameters estimated:

$$-2\text{LogLikelihood} + 2k$$

This value is reported in the fit output only if the ARIMA or Step-wise ARIMA models are used.

Forecast error

The forecast error measure determines forecast accuracy by showing the adjusted error of the predicted actuals compared to the historical actuals.

The forecast error measure is calculated using the following formula:

$$\sqrt{\left(\frac{\text{ResidualSumSquares}}{(\text{Count}-2)}\right)}$$

This value is reported in the **StatisticalForecastFitOutput** table.

Hannan-Quinn criterion (HQC)

The Hannan-Quinn criterion measure determines how well an ARIMA model fits the values it was generated from, using the log-likelihood result and the product of double the number of parameters and the logarithm of the logarithm of the number of observed samples. Models that produce a smaller value using this measure are preferred.

The HQC is calculated using the following formula, where *n* is the number of observations in the sample and *k* is the number of parameters estimated.

$$-2\text{LogLikelihood} + 2k\log\log n$$

This value is reported in the fit output only if the ARIMA or Step-wise ARIMA models are used.

Lag percentage error

The lag percentage error measure is reported only when using the "BestFit" forecast model with a holdout period (**ForecastItemParametersType** has **ForecastModel** set to "BestFit" and **HoldoutPeriodUsageRule** set to either "SimpleHoldout" or "Simulation").

This measure calculates the sum of differences between the actual historical data points in the first interval of the holdout period and the simulated forecast values generated in the first interval of the holdout period, and expresses this difference as a percentage of all actual historical data points in that interval.

The lag percentage error measure is calculated using the following formula:

$$100 \times \frac{|y - f(x)|}{y}$$

where:

- *y*—represents the sum of historical actuals in the first cycle calendar interval considered in the holdout period (after offset for optional BestFitForecastLag).
- *f(x)*—represents the sum of simulated forecast values generated by the model in the first cycle calendar interval considered in the holdout period (after offset for optional BestFitForecastLag).

Note: To use the lag percentage error measure when determining best fit, select "LagPercentageError" in the **ForecastItemParametersType.FitMeasure** field.

Log-likelihood

The log-likelihood measure determines the maximum likelihood for a statistical function, using the sample size, the logarithm of the sample size, and the logarithm of the residual sum of squares.

The log-likelihood is calculated using the following formula, where *n* is the number of observations in the sample:

$$-\frac{n}{2}\left(1+\log 2\pi-\log n\right)-\frac{n}{2}\log \mathrm{ResidualSumSquares}$$

This value is reported in the fit output only if the ARIMA or Step-wise ARIMA models are used.

Mean absolute error

The mean absolute error (MAE) measure, also refered to as Mean Absolute Deviation (MAD), determines the absolute value of the variance between the historical actual values and the predicted forecast values.

The mean absolute error is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$\frac{\sum_{i=1}^{\mathrm{Count}}|y_i-f(x_i)|}{\mathrm{Count}}$$

To use the mean absolute error measure to calculate the best fit, select "MeanAbsoluteError" in the **ForecastItemParametersType.FitMeasure** field.

Mean absolute percentage error

The mean absolute percentage error measure determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. Zero values in the input data are ignored, and are not included in the total count of points.

The mean absolute percentage error is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$100\times\frac{\sum_{i=1}^{\mathrm{Count}}\left|\frac{y_i-f(x_i)}{y_i}\right|}{\mathrm{Count}}$$

To use the mean absolute percentage error measure to calculate the best fit, select "MeanAbsolutePercentageError" in the **ForecastItemParametersType.FitMeasure** field.

Mean absolute percentage error by forecast

The mean absolute percentage error by forecast measure determines the absolute value of the variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage. Zero values in the input data are ignored, and are not included in the total count of points.

The mean absolute percentage error by forecast is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$100 \times \frac{\sum_{i=1}^{\text{Count}} \left| \frac{y_i - f(x_i)}{f(x_i)} \right|}{\text{Count}}$$

This value is reported in the **StatisticalForecastFitOutput** table.

Mean absolute error by mean

The mean absolute error by mean measure, also referred to as Mean Absolute Deviation by Mean, normalizes the value returned by the mean absolute error measure by dividing it by the mean quantity of the input data. If the mean quantity is zero, a value of 'INF' is returned.

The mean absolute error by mean is calculated using the following formula:

$$100 \times \frac{\text{MeanAbsoluteError}}{\text{MeanQty}}$$

Note: You can use Mean Absolute Error by Mean as an alternative to weighted MAPE in determining forecast error for both intermittent and non-intermittent time series.

To use the mean absolute error by mean measure to calculate the best fit, select "MeanAbsoluteErrorByMean" in the **ForecastItemParametersType.FitMeasure** field.

Mean absolute scaled error (non-seasonal)

Mean absolute scaled error (MASE) (non-seasonal) is the mean absolute error of the forecast divided by the mean absolute error of the one-step ahead naïve forecast. It is recommended for comparing the forecasts for different parts.

The mean absolute scaled error by forecast is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*.

The mean absolute scaled error (non-seasonal) is calculated using the following formula:

$$\text{MASE} = \frac{\frac{1}{\text{Count}} \sum_{i=1}^{\text{Count}} |f(x_i) - y_i|}{\frac{1}{\text{Count}-1} \sum_{i=2}^{\text{Count}} |y_i - y_{i-1}|}$$

Mean absolute scaled error (seasonal)

Mean absolute scaled error (MASE) (seasonal) is the mean absolute error of the forecast divided by the mean absolute error of the seasonal naïve forecast (seasonal).

The mean absolute scaled error by forecast is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*.

The mean absolute scaled error (seasonal) is calculated using the following formula:

$$\text{MASE} = \frac{\frac{1}{T} \sum_{i=1}^{T} |y_i - \text{Fitted}_i|}{\frac{1}{T-M} \sum_{i=M+1}^{T} |y_i - y_{i-M}|}$$

Mean error

The mean error measure determines the average variance between the historical actual values and the predicted forecast values.

The mean error is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$\frac{\sum_{i=1}^{\text{Count}} (f(x_i) - y_i)}{\text{Count}}$$

To use the mean error measure to calculate the best fit, select "MeanError" in the **ForecastItemParametersType.FitMeasure** field.

Mean percentage error

The mean percentage error measure determines the average variance between the historical actual values and the predicted forecast values with respect to the actual values, expressed as a percentage. Because this calculation does not contain an absolute value, the percentage can be expressed as positive or negative, which shows whether the input is greater than or less than the predicted values. Zero values in the input data are ignored, and are not included in the total count of points.

The mean percentage error is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$100 \times \frac{\sum_{i=1}^{\text{Count}} \frac{(f(x_i) - y_i)}{y_i}}{\text{Count}}$$

To use the mean percentage error measure to calculate the best fit, select "MeanPercentageError" in the **ForecastItemParametersType.FitMeasure** field.

Mean percentage error by forecast

The mean percentage error by forecast measure determines the average variance between the historical actual values and the predicted forecast values with respect to the forecast values, expressed as a percentage. Because this calculation does not contain an absolute value, the percentage can be expressed as positive or negative, which shows whether the input is greater than or less than the predicted values. Zero values in the input data are ignored, and are not included in the total count of points.

The mean percentage error by forecast is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$100 \times \frac{\sum_{i=1}^{\text{Count}} \frac{(f(x_i) - y_i)}{f(x_i)}}{\text{Count}}$$

This value is reported in the **StatisticalForecastFitOutput** table.

Mean square errors

The mean square errors measure calculates the average of the residual sum of squares for the forecast. This measure determines, on average, how the calculated and actual values differ.

The mean square errors measure is calculated using the following formula:

$$\frac{\text{ResidualSumSquares}}{\text{Count}}$$

To use the mean square errors measure to calculate the best fit, select "MeanSquareError" in the **ForecastItemParametersType.FitMeasure** field.

R-squared

The R-squared measure determines the ratio between the regression sum of squares and total sum of squares measures. This value shows how closely the forecast values fit the historical actual values, with a higher value indicating a better fit.

This measure is also referred to as the coefficient of determination.

The R-squared measure is calculated using the following formula:

$$\frac{\text{ResidualSumSquares}}{\text{TotalSumSquares}}$$

To use the R-squared measure to calculate the best fit, select "RSquared" in the **ForecastItemParametersType.FitMeasure** field.

Regression sum of squares

The regression sum of squares error measure determines the difference between the total sum of squares of the input data and the residual sum of squares. This is used to show how the variance of the statistical forecast and input values differs from the variance of the input values and average of input values. Low values represent a close match.

The regression sum of squares is calculated using the following formula:

$$\text{TotalSumSquares-ResidualSumSquares}$$

To use the regression sum of squares measure to calculate the best fit, select "RegressionSumSquares" in the **ForecastItemParametersType.FitMeasure** field.

Residual sum of squares

The residual sum of squares error measure calculates the sum of the square of differences between the historical actual values and the values predicted by the statistical forecast. This measure is also referred to as the sum of squared errors.

The residual sum of squares is calculated using the following formula, where y represents the quantity of a data point and $f(x)$ represents the result of the statistical forecast calculation for the date value given by *x*:

$$\sum_{i=1}^{\text{Count}} (Y_i - f(x_i))^2$$

To use the residual sum of squares measure to calculate the best fit, select "ResidualSumSquares" in the **ForecastItemParametersType.FitMeasure** field.

Rolling lag percentage error

The rolling lag percentage error measure is reported only when using the "BestFit" forecast model with a holdout period (the **ForecastItemParametersType** record has **ForecastModel** set to "BestFit" and **HoldoutPeriodUsageRule** set to either "SimpleHoldout" or "Simulation").

This measure calculates the sum of differences between the actual historical data points simulated forecast values across a specified number of intervals in the holdout period, and expresses this difference as a percentage of the actual historical data points in those intervals. The number of intervals considered for this error measure is set by the **RollingErrorIntervalCount** field on the **ForecastItemParameters** table.

The rolling lag percentage error measure is calculated using the following formula:

$$100 \times \frac{\left| \sum_{i=1}^{\text{Count}} (yi - f(xi)) \right|}{\sum_{i=1}^{\text{Count}} yi}$$

where:

- *1*—represents the first cycle calendar interval considered in the holdout period (this includes consideration for **BestFitForecastLag** field value).
- *Count*—represents the number of cycle calendar intervals to be considered in the holdout period as defined by the **RollingErrorIntervalCount** field value.
- *y*—represents the sum of historical actuals within a given cycle calendar interval in the holdout period.
- *f(x)*—represents the sum of simulated forecast values generated by the model within a given cycle calendar interval in the holdout period.

Note: To use the rolling lag percentage error measure when determining best fit, select "RollingLagPercentageError" in the **ForecastItemParametersType.FitMeasure** field.

Root mean square error

The root mean square error measure determines the square root of the value produced by the mean square errors measure. This is used to show the correlation between the historical actual values and the results of the statistical forecast calculation, with smaller values representing closer correlation.

This measure is also referred to as the standard deviation of error.

The root mean square errors measure is calculated using the following formula:

$$\sqrt{\frac{\text{ResidualSumSquares}}{\text{Count}}}$$

To use the root mean square errors measure to calculate the best fit, select "RootMeanSquareError" in the **ForecastItemParametersType.FitMeasure** field.

Schwarz Bayesian information criterion (SBC)

The Schwarz Bayesian information criterion measure determines how well an ARIMA model fits the values it was generated from, using the log-likelihood result and the product of the number of parameters and logarithm of the number of observed samples. Models that produce a smaller value using this measure are preferred.

The SBC is calculated using the following formula, where *n* is the number of observations in the sample and *k* is the number of parameters estimated.

$$-2\text{LogLikelihood} + k \log n$$

This value is reported in the fit output only if the ARIMA or Step-wise ARIMA models are used.

Symmetric mean absolute percentage error

The symmetric mean absolute percentage error determines the absolute difference between the forecast values and historical actual values and divides it by half the sum of the forecast values and actual values, expressed as a percentage. The sMAPE is limited to 200%. Because it has a finite maximum value, this makes sMAPE is a good alternative to Mean Absolute Percent Error (MAPE) because it reduces the influence of demand values that are zero or close to zero.

The symmetric mean absolute percentage error is calculated using the following formula, where *y* represents the actual demand value and *Fitted* represents the forecast value:

$$\mathbf{sMAPE} = \sum_{i=1}^{N} \frac{\mathbf{200}|y_i - \mathbf{Fitted}_i|}{|y_i + \mathbf{Fitted}_i|}$$

Variance error

The variance error measure determines if the forecast model produces forecasts with more stable forecast errors or unstable, varying forecast errors by showing how widely the errors are spread around the mean error.

The variance error is calculated using the following formula:

$$\sum_{i=1}^{\mathbf{Count}} \frac{(f(x_i) - y_i - \mathbf{MeanError})^2}{\mathbf{Count} - 1}$$

This value is reported in the **StatisticalForecastFitOutput** table.

CHAPTER 43: Consensus forecast

The consensus forecast combines the results of the statistical forecast and leverages the market intelligence of collaborators within an organization to arrive at a single, agreed upon consensus forecast for the current S&OP cycle. The purpose of combining all forecasts is to improve forecast accuracy and reduce errors by minimizing the bias that might be associated with one forecast category.

In this chapter, you will learn about:

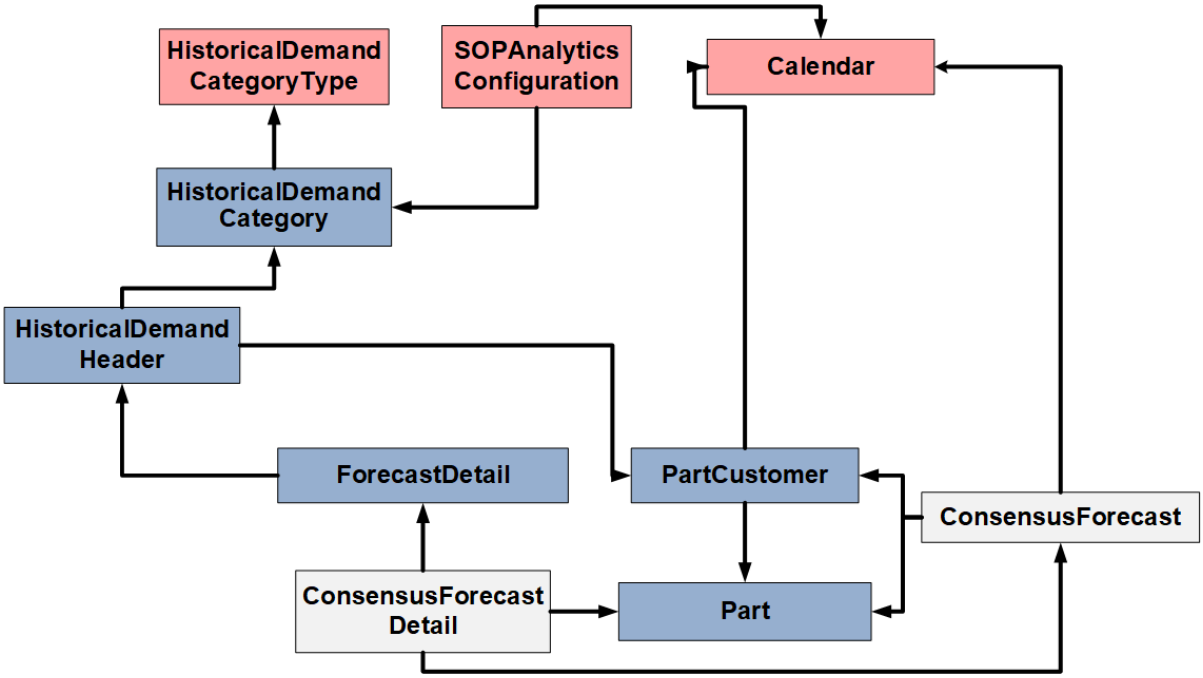
Consensus forecast data model **2070**
Consensus forecast calculations **2072**

Consensus forecast category weights **2073**
Consensus forecast adjustments and overrides **2079**

Consensus forecast data model

In this section, we'll explore the input, control, and calculated tables that support the creation of consensus forecasts during consensus demand planning. The table listed in this section are involved in inputting, configuring, and calculating consensus forecasts. The **ConsensusForecast** table is the calculated table that holds the consensus forecast records for a part customer and also combines the results of the statistical forecast, as well as the forecasts from different forecast categories.

The following diagram shows the input tables (blue), control tables (pink), and the calculated tables (grey) that are involved in consensus demand planning and how they relate to the **ConsensusForecast** table. If a table includes a reference field, the arrow in the diagram points to the table that is being referenced.



The following section describes the tables shown in the illustration and briefly outlines their usage in consensus demand planning.

Table 43.3: Consensus demand planning tables

Table	Type	Description
Calendar	Control	Identifies named calendars. The **PartCustomer** table references this table to define the disaggregation calendar for the part-customer, which affects how the consensus demand plan is disaggregated. See "Calendar table" on page 695.
ConsensusForecast	Calculated	Reports the consensus forecast for a part-customer and reflects any weights associated with the individual forecast categories, as well any adjustments and overrides. Depending on configuration, it can also be used in calculating disaggregation rates for a given part, customer, and forecast category combination. See "ConsensusForecast table" on page 966.
ConsensusForecastDetail	Calculated	Identifies and reports **ForecastDetail** records that are used to generate the consensus forecast for a particular part and customer and reported in the **ConsensusForecast** table. It also reflects the weighted quantity for the individual forecast categories on a given date. See "ConsensusForecastDetail table" on page 970.
ForecastDetail	Input	Holds detailed forecast data for part-customer and category combinations (after any disaggregation calculations have been performed) on a given date. Forecast quantities in the **ConsensusForecast** table are calculated as a weighted combination of different forecast category quantities from the **ForecastDetail** table. See "ForecastDetail table" on page 283.
HistoricalDemandCategory	Input	Identifies categories of forecasted and actual demand. It also defines the default **ConsensusForecastWeight** that is applied to a forecast category which determines how much the category contributes to the calculated consensus forecast. If there is no effective record for a given forecast category and part customer combination in the **HistoricalDemandHeaderTimePhasedAttributes**, **HistoricalDemandHeaderRollingWeight**, **HistoricalDemandCategoryRollingWeight**, or weight in **HistoricalDemandHeader** tables, then the category weight for the part customer is taken from this table. See "HistoricalDemandCategory table" on page 329 and "Consensus forecast category weights" on page 2073.

Table 43.3: Consensus demand planning tables

Table	Type	Description
HistoricalDemand CategoryType	Control	Defines how demand data that is associated with a given demand category is processed in RapidResponse. For example, it determines whether historical actuals or the consensus forecast should be used to calculate disaggregation rates for a statistical forecast category. See "HistoricalDemandCategoryType table" on page 736.
HistoricalDemandHeader	Input	Identifies the unique part, customer, and historical demand category combinations. It also holds the **ConsensusForecastWeight** field. This is the override value for the **HistoricalDemandCategory.ConsensusForecastWeight** . If this is set to anything other than -1.0, then the category value has been overridden by this. See "HistoricalDemandHeader table" on page 334 and "Consensus forecast category weights" on page 2073.
Part	Input	Identifies a unique part and site. To generate a consensus forecast for a part, the Part.Type processing rule must be set to either "MPS" or "MPSConfig". See "Part table " on page 415.
PartCustomer	Input	Contains unique part and customer combinations for which historical data is provided and forecasts can be generated in the S&OP process. It also allows overriding the standard disaggregation calendar on a part-customer basis. For more information, see "PartCustomer table" on page 444.
SOPAnalytics Configuration	Control	Specifies the default values for disaggregation settings such as the historical demand category, number of historical intervals, and calendars to use when calculating disaggregation rates. This table should hold a single record per scenario. For more information, see "SOPAnalyticsConfiguration table" on page 892.

Consensus forecast calculations

In RapidResponse, the consensus forecast that is generated during consensus demand planning is the weighted sum of the results of the statistical forecast and the forecast quantities from each of the contributing forecast categories, for example finance, marketing, sales, or other groups. The forecast quantities on a given date for a given part, customer, and forecast category combination are stored in the **ForecastDetail** input table. It is based on aggregate values that are entered or maintained through various resources in RapidResponse.

The **ForecastDetail.Quantity**, also reflects any adjustments or overrides that the demand planner applied to the forecast quantities during consensus demand planning or demand supply plan balancing. See "Consensus forecast adjustments and overrides" on page 2079.

The **ForecastDetail.Quantity** value that is associated with each forecast category on a given date is then multiplied by any forecast weights, also known as forecast ratios, that the demand planner applied to a category to determine the importance that is given to each of the forecasts that contribute to the consensus demand plan. If, however, overrides or adjustments are made, these weights are ignored. See "Consensus forecast category weights" on page 2073.

The following calculated tables are then used to report consensus forecast quantities:

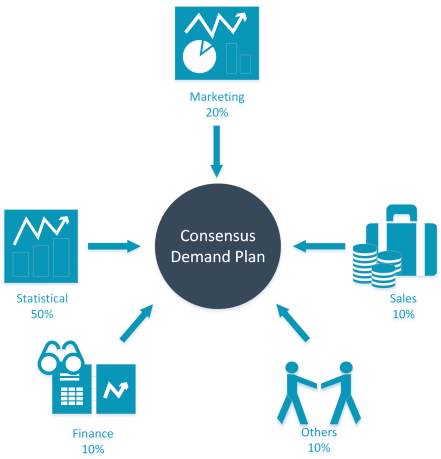
- **ConsensusForecastDetail**: Reports the forecast quantity and date for a particular part, customer, and forecast category combination that are contributing to the consensus forecast. It also reflects the forecast weights that may be assigned to each category, as well as forecast overrides or forecast rebalancing overrides.
- **ConsensusForecast**: Contains the calculated quantity for a part and customer combination on a given forecast date. The forecast weights that may be assigned to each category are reflected, but are summed together. This table also reports any forecast quantities that were changed using an override or an adjustment.

The **ConsensusForecast** table is also used to drive supply planning in RapidResponse based on the consensus forecast generated during the sales and operations planning process. Consensus forecast quantities are subsequently reported in the **Forecast** table (subject to any forecast spreading logic) with a **ForecastSource** value of "ConsensusForecast". Availability of the forecast is given in the **CTPForecast** table, and details of all supplies used to satisfy the forecast can be seen in the **WhereConsumedForForecast** table.

Another important aspect of consensus forecast calculations is disaggregation. Forecast disaggregation logic takes forecast values entered or calculated at summary levels and distributes them to detailed part and customer combinations. The topic of forecast disaggregation is not discussed in this chapter, but is discussed in detail in "Forecast disaggregation" on page 2083.

Consensus forecast category weights

When combining forecasts, the demand planner can define the weight of importance to give to the individual forecasts that contribute to the consensus demand plan. In RapidResponse, the individual forecasts are organized into forecast categories using the **HistoricalDemandCategory** table. In the following example, after reviewing forecast accuracy metrics, the demand planner determines that the sales, finance, and other forecast categories are less accurate at predicting demand than the other forecasts. As a result, they reduce the weight of these categories so that they contribute less to the consensus demand plan.



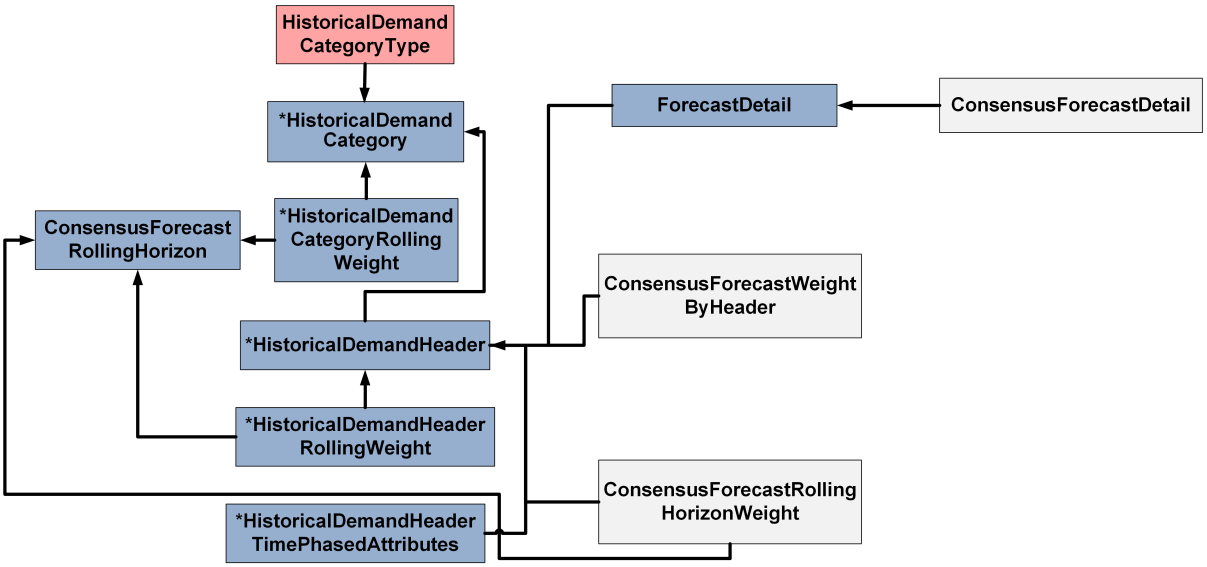
In RapidResponse, there are different methods of weighting each of these forecast categories. This section will discuss the tables that are involved with each of the different methods for adjusting weights, as well as how the forecast category weights are calculated.

Note: If a **RebalancingForecastOverride**, **RebalancingAdjustment** and **ForecastOverride** record exists for a forecast quantity on a given date for a given part, customer, and forecast category combination, the forecast category weight is ignored. See "Consensus forecast adjustments and overrides" on page 2079.

Forecast category weights and tables

In RapidResponse, the forecast category weight that is used in calculations is entered in the **ConsensusForecastWeight** field. This field is found in five different input tables. The table that you choose to enter the forecast category weight determines which input table is used in the calculation.

The following diagram shows the input tables (blue), control tables (pink), and calculated tables (grey) that are involved in using forecast category weights. The input tables that include a **ConsensusForecastWeight** field are marked with *. If a table includes a reference field, the arrow in the diagram points to the table that is referenced.



It's important to note that the choice of input table changes only how the weight is applied and for how long it's in effect, but the way the weight is calculated remains the same regardless of the method that you choose to enter it. The calculations will be discussed in more detail later in this section.

The following table describes the different ways of entering forecast category weights along with the input table to use to enter the weight. The table also identifies which tables take precedence over others.

To enter	Use table
Default forecast category weights.	HistoricalDemandCategory A **ConsensusForecastWeight** value entered in this table is used in consensus forecast calculations only if there are no effective **ConsensusForecastWeight** records found in any other tables.
Default forecast category rolling weights. See "Configuring rolling forecast category weights" on page 2076.	HistoricalDemandCategoryRollingWeight A **ConsensusForecastWeight** value entered in this table takes precedence over **HistoricalDemandCategory** table in consensus forecast calculations.
Forecast category weights for specific part-customers.	HistoricalDemandHeader A **ConsensusForecastWeight** value entered in this table takes precedence over **HistoricalDemandCategoryRollingWeight** and **HistoricalDemandCategory** tables in consensus forecast calculations. However, if a header has existing **HistoricalDemandHeaderTimephasedAttributes** or **HistoricalDemandHeaderRollingWeight** records, the weight value in this table is ignored.

To enter	Use table
Forecast category rolling weights for a specific part-customer and duration. See "Configuring rolling forecast category weights" on page 2076.	HistoricalDemandHeaderRollingWeight A **ConsensusForecastWeight** value entered in this table takes precedence over most other **ConsensusForecastWeight** values in consensus forecast calculations except for **HistoricalDemandHeaderTimephasedAttributes** . If a header has existing **HistoricalDemandHeaderTimephasedAttributes** records, the weight value in this table is ignored.
Forecast category weights for a specific part-customer and set duration.	HistoricalDemandHeaderTimePhasedAttributes A **ConsensusForecastWeight** value entered in this table takes precedence over all other **ConsensusForecastWeight** values in consensus forecast calculations.

There are two calculated tables that report the results of weight calculations. They are as follows:

Table 43.4:

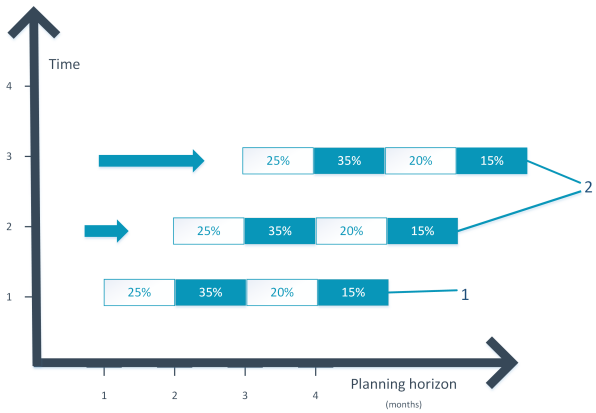
Table	Type	Description
"ConsensusForecastWeightByHeader table" on page 972	Calculated	Reports values that are used by **ConsensusForecast** calculations. The results are based on the **ConsensusForecastWeight** that is set using one the input tables that include the **ConsensusForecastWeight** field (see previous table). The **ConsensusForecastWeightByHeader** table is also used if the ProcessingRule field on the **HistoricalDemandCategoryType** table is set to **Forecast**, indicating that the historical demand category will be used towards calculating the consensus forecast.
"ConsensusForecastRollingHorizonWeight table" on page 971	Calculated	Reports the weight that is applied to a forecast category in a given horizon when using rolling forecast weights to create the consensus demand plan. The weights entered using **HistoricalDemandCategoryRollingWeight** and **HistoricalDemandHeaderRollingWeight** are reported here. If a header contains existing **HistoricalDemandHeaderTimephasedAttributes** records, this table is ignored.

Configuring rolling forecast category weights

A demand planner can use rolling forecast weights to assign weights to each forecast category for a defined period of time during the planning horizon. Then, as time moves on, so do the rolling weights that are assigned to the forecast categories. This allows the demand planner to reuse forecast weights into the future, without the need to continually enter weights for new period buckets as the planning

horizon moves forward. The entire duration of the rolling weights horizon is based on the S&OP cycle calendar.

The following illustration shows 1) a set of rolling forecast weights that are assigned to one forecast category and 2) how the same series of weights moves forward in the planning horizon as time advances.



To configure rolling forecast category weights, you need to:

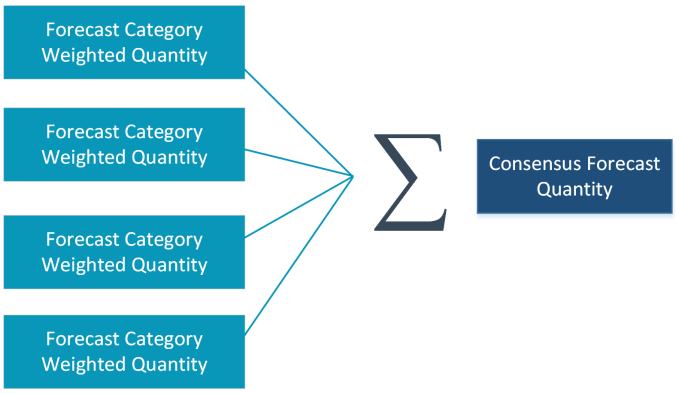
1. Define the order and duration of the horizons that you want to use for applying **ConsensusForecastWeight** in the "ConsensusForecastRollingHorizon table " on page 246.
2. Define which **HistoricalDemandCategory** will use rolling forecast weights in the "HistoricalDemandCategoryRollingWeight table " on page 333.
3. Specify a **ConsensusForecastWeight** for each **HistoricalDemandCategory** for a given horizon in the "HistoricalDemandCategoryRollingWeight table " on page 333.

Consensus forecast weight calculations

As previously mentioned, the consensus forecast weight for a given part and customer combination is calculated the same way regardless of the input table that you choose to enter the **ConsensusForecastWeight** value. The **ForecastDetail.Quantity** value that is associated with each forecast category on a given date is multiplied by the value in the **ConsensusForecastWeight** field.

and the weighted results are reported in **ConsensusForecastDetail.Quantity**.

Then the sum of the weighted **ConsensusForecastDetail.Quantity** for each forecast category on a given date is reported in **ConsensusForecast.CalculatedQuantity**.



When you enter **ConsensusForecastWeight**, the value is typically between 0 and 1. Only values that are greater than zero are included in the consensus forecast calculation. For example, if **ConsensusForecastWeight** was set to 1 for the Statistical forecast category and 0 for all other forecast categories, then the consensus forecast would be set to use only the Statistical forecast value in consensus forecast calculations. In addition, the sum of the **ConsensusForecastWeight** values for all categories can be below or above 1.

The following illustrations show **ForecastDetail.Quantity** for four different forecast categories on a given date (shown in teal), the category weights (shown in red) that are applied to each category, as well as the weighted **ConsensusForecastDetail.Quantity** (shown in dark blue).

In this example, the sum of the category weights is 1.

Sales	Marketing	Finance	Statistical
4,700	4,000	3,000	5,000
.25	.25	.25	.25
1,175	1,000	750	1,250

In this example, the sum of the category weights is .8.

Sales	Marketing	Finance	Statistical
4,700	4,000	3,000	5,000
.2	.2	.2	.2
940	800	600	1,000

In this example, the sum of the category weights is 1.1.

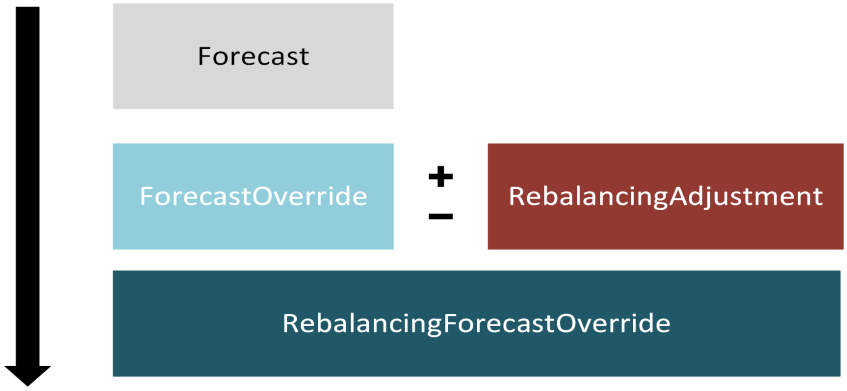
Sales	Marketing	Finance	Statistical
4,700	4,000	3,000	5,000
.4	.2	.5	.0
1,880	800	1,500	0

Consensus forecast adjustments and overrides

When evaluating the consensus forecast, the demand planner has the ability to change the consensus forecast quantities that are calculated by RapidResponse, including the sum of the weighted forecast. They can make changes by adjusting (increasing or decreasing) or overriding (replacing) the consensus forecast quantities. The changes are made either when reviewing the consensus demand plan or when rebalancing demand to supply.

The adjustments and overrides are made to the forecast quantities for a given PartCustomer on a given date. They are based on the **HistoricalDemandHeader** table for a given date and they are reflected in **ForecastDetail.Quantity**. The **HistoricalDemandCategoryType.ProcessingRule** that is set for the **ForecastDetail** record determines how the adjustment or override is calculated.

The following diagram lists each **HistoricalDemandCategoryType.ProcessingRule** in the order in which it would be applied during the consensus demand planning process, as well as the relative precedence of each processing rule. For example, the **RebalancingForecastOverride** rule takes precedence over all other processing rules.



The following table lists the processing rules, how they impact the consensus forecast calculations, and in which field the quantity is reported.

If HistoricalDemandCategoryType. ProcessingRule is set to	How it affects consensus forecast calculations	Quantity reported in
Forecast	Stores forecast demands and forecast adjustments for forecast categories. Values using this processing rule can be used towards calculating the consensus forecast (based on the specified **ConsensusForecastWeight** value).	**ConsensusForecast.CalculatedQuantity** and also reflected in **ConsensusForecast.Quantity**
ForecastOverride	Specifies the value to use to override the weighted sum of the forecast. This override is applied during the consensus demand planning phase. Forecast category weights are ignored. If a **RebalancingAdjustment** is specified, the **ForecastOverride** quantity is either increased or decreased. If **RebalancingForecastOverride** is specified, the **ForecastOverride** value is ignored.	**ConsensusForecast.OverrideQuantity** and also reflected in **ConsensusForecast.Quantity**
RebalancingAdjustment	Specifies the amount to increase or decrease either the **ConsensusForecast.CalculatedQuantity** or the **ConsensusForecast.OverrideQuantity** value (if used) during the demand and supply balancing phase. Forecast category weights are ignored. If a **RebalancingForecastOverride** is specified, the **RebalancingAdjustment** value is ignored.	**ConsensusForecast.Rebalancing Adjustment Quantity** and also reflected in **ConsensusForecast.Quantity**

If HistoricalDemandCategoryType. ProcessingRule is set to	How it affects consensus forecast calculations	Quantity reported in
RebalancingForecastOverride	Specifies the value to use to override the consensus forecast during the demand and supply balancing phase. This rule overrides any previous changes that were made. Forecast category weights are ignored. If a **RebalancingForecastOverride** is specified, the value takes precedence over **RebalancingAdjustment**, **ForecastOverride**, and **Forecast**.	**ConsensusForecast.RebalancingOverrideQuantity** and also reflected in **ConsensusForecast.Quantity**

CHAPTER 44: Forecast disaggregation

Forecast disaggregation logic takes forecast values entered or calculated at summary levels during the S&OP process and distributes them to detailed part and customer combinations. This distribution is based on calculated and configurable disaggregation rates, before then spreading the disaggregated values across defined calendar intervals. Both general forecast disaggregation and statistical forecast disaggregation are supported in RapidResponse.

General forecast disaggregation refers to the distribution of forecast values entered at aggregate levels using hierarchies in standard RapidResponse resources. For example, a financial forecast entered at the product group level in the S&OP Finance Operating Plan workbook is then distributed to the detailed part and customer level and reported in the **ForecastDetail** table. Statistical forecast disaggregation refers to the distribution of the statistical forecast calculated for forecast items configured in RapidResponse. Statistical forecast values at the item level are reported in the **StatisticalForecast** table, with the **StatisticalForecastDetail** table then used to hold values disaggregated to the detailed part and customer level.

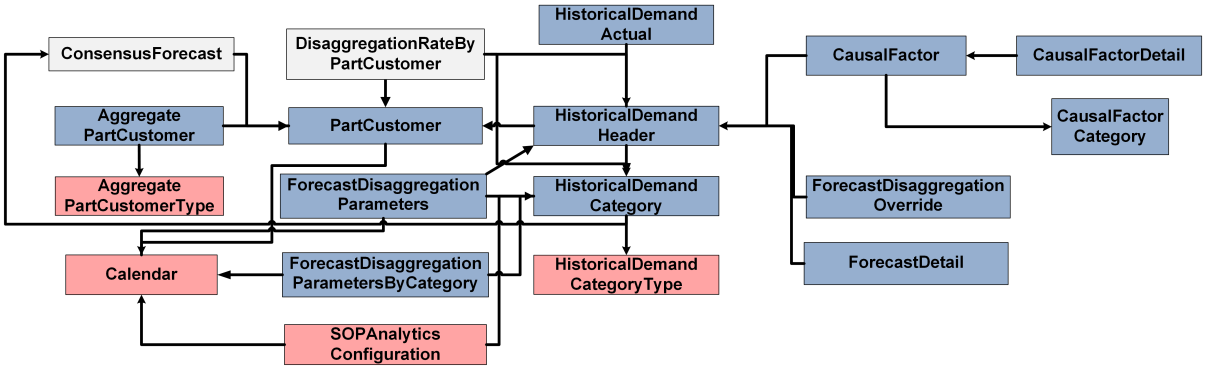
In this chapter, you will learn about:

General disaggregation data model **2083**
General disaggregation rates **2087**
Configuring general forecast disaggregation **2088**
Statistical disaggregation data model **2098**
Statistical disaggregation rates **2102**
Configuring statistical forecast disaggregation **2103**

General disaggregation data model

The RapidResponse data model includes a number of tables that either provide data used as input into forecast disaggregation calculations, hold settings used to determine how disaggregation calculations are performed, or report and use the results of those calculations. The following illustration shows the relationships between these input, control, and calculated tables.

Figure 44.1 Tables used in general forecast disaggregation



The following section describes each of the tables shown in the illustriation, and briefly outlines their usage in general forecast disaggregation.

Table 44.1: General forecast disaggregation tables

Table	Type	Description
AggregatePart Customer	Input	Groups part-customers together when a forecast does not need to be fully disaggregated to detailed part and customer combinations. For more information, see "AggregatePartCustomer table" on page 179.
AggregatePart CustomerType	Control	Enables or disables the aggregate part-customer definitions that are used in forecast disaggregation calculations. For more information, see "AggregatePartCustomerType table" on page 670.
Calendar	Control	Identifies named calendars (and referenced by various disaggregation settings such as the inner and outer disaggregation calendars). For more information, see "Calendar table" on page 695.
CausalFactor	Input	Stores causal factors used to adjust historical data for data errors or unusual demand events before a statistical forecast is generated. It references the part customer and historical actual the causal factor applies to (through the HistoricalDemandHeader reference), as well as the category the causal factor is associated with (through the CausalFactorCategory reference). For more information, see "CausalFactor table" on page 239.
CausalFactorCategory	Input	Stores categories defined in your company for grouping causal factors. For example, causal factor categories might include things such as promotions and weather events. For more information, see "CausalFactorCategory table" on page 240.

Table 44.1: General forecast disaggregation tables

Table	Type	Description
CausalFactorDetail	Input	Includes date and quantity details that belong to a given causal factor through the reference to the CausalFactor table. The quantities in this table are taken into account by the DisaggregationRateByPartCustomer table and are also reflected in the quantities that are reported in the ForecastDetail table. For more information, see "CausalFactorDetail table" on page 241.
ConsensusForecast	Calculated	Reports the consensus forecast for a part customer. Depending on configuration, it can be used in calculating disaggregation rates for a given part, customer, and forecast category combination. For more information, see "ConsensusForecast table" on page 966.
DisaggregationRate ByPartCustomer	Calculated	Reports the effective disaggregation rate for a part-customer and forecast category on a given date (either calculated or provided as an input/override). Any causal quantities that are reported in CausalFactorDetail are also taken into account. For more information, see "DisaggregationRateByPartCustomer table" on page 1037.
ForecastDetail	Input	Holds detailed forecast data for part-customer and category combinations (after any disaggregation calculations have been performed). Disaggregated values are typically added to this table based on values entered in S&OP workbooks. Any causal quantities that are reported in CausalFactorDetail are also taken into account. For more information, see "ForecastDetail table" on page 283.
Forecast Disaggregation Override	Input	Specifies general disaggregation rates for part-customer and forecast category combinations. This table is used only if you choose to manually insert rates instead of having the rates calculated by RapidResponse. For more information, see "ForecastDisaggregationOverride table" on page 290.
Forecast Disaggregation Parameters	Input	Holds many of the default disaggregation settings on a part-customer and forecast category basis. It overrides the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates. For more information, see "ForecastDisaggregationParameters table" on page 287.

Table 44.1: General forecast disaggregation tables

Table	Type	Description
Forecast Disaggregation ParametersBy Category	Input	Holds many of the default disaggregation settings on a forecast category basis. It overrides the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates. For more information, see "ForecastDisaggregationParametersByCategory table" on page 288.
HistoricalDemand Actual	Input	Holds historical demand data in various categories which, depending on configuration, can be used in calculating disaggregation rates for a given part, customer, and forecast category combination. For more information, see "HistoricalDemandActual table" on page 325.
HistoricalDemand Header	Input	Identifies unique part, customer, and historical demand category combinations. For more information, see "HistoricalDemandHeader table" on page 334.
HistoricalDemand Category	Input	Identifies categories of forecasted and actual demand. For more information, see "HistoricalDemandCategory table" on page 329.
HistoricalDemand CategoryType	Control	Defines how demand data that is associated with a given demand category is processed in RapidResponse. For example, it determines whether historical actuals or the consensus forecast should be used to calculate disaggregation rates for a statistical forecast category. For more information, see "HistoricalDemandCategoryType table" on page 736.
PartCustomer	Input	Contains unique part and customer combinations for which historical data is provided and forecasts can be generated in the S&OP process. Also allows overriding the standard disaggregation calendar on a part-customer basis. For more information, see "PartCustomer table" on page 444.
SOPAnalytics Configuration	Control	Specifies the default values for disaggregation settings such as the historical demand category, number of historical intervals, and calendars to use when calculating disaggregation rates. This table should hold a single record per scenario. For more information, see "SOPAnalyticsConfiguration table" on page 892.

General disaggregation rates

General forecast disaggregation rates are calculated and stored in the **DisaggregationRateByPartCustomer** table. The rates in this table are used to disaggregate different forecast streams whose values are entered using several standard S&OP workbooks included with RapidResponse. For example, the S&OP Sales Forecast workbook allows members of the Sales group to enter sales forecast data against various levels of the hierarchy (such as, by product group or customer group), and then disaggregates these forecasts down to detailed part and customer combinations. The disaggregated forecast quantities are then stored in the **ForecastDetail** table.

Each record in the **DisaggregationRateByPartCustomer** table holds a calculated **Quantity** (disaggregation rate) pertaining to a particular part-customer and forecast category combination as of a given **Date**. The disaggregation rate on each record is then effective until the next **Date** a record is found for that same part-customer and forecast category combination. To support the option to disaggregate forecasts by order revenue, rather than quantity, each record in this table also reports an **EffectiveUnitPrice** value.

Note that in order for the **DisaggregationRateByPartCustomer** table to calculate and report disaggregation rates for any given forecast category, the category name must be passed to the table using an analytic parameter. For more information, see "Calculate general disaggregation rates by forecast category" on page 2236.

Usage of general disaggregation rates

Disaggregation rates are used in determining the disaggregated forecast quantities that are then stored in the **ForecastDetail** table. The actual rates applied during forecast disaggregation are based on ratios determined from the **Quantity** field. For example, suppose the following set of records in the **DisaggregationRateByPartCustomer** table.

Date	Category	PartCustomer.Part	PartCustomer.Customer	Quantity
07-01-15	SalesForecast	Gamer	BCI	400
07-01-15	SalesForecast	Mini	BCI	160
07-01-15	SalesForecast	Server	BCI	0
07-01-15	SalesForecast	Gamer	ECS	350
07-01-15	SalesForecast	Mini	ECS	130
07-01-15	SalesForecast	Server	ECS	560
			SUBTOTAL	1600

Further, suppose parts Gamer, Mini, and Server all belong to the same **ProductGroup** and a sales forecast of 500 units for 07-01-15 is entered against that **ProductGroup** using the S&OP Sales Forecast workbook. The different part-customer combinations would then receive the following disaggregated forecast quantities to be stored in the **ForecastDetail** table (spread across disaggregation calendar intervals as required)**:**

- Gamer-BCI: 125 units (25% as determined from 400/1600)
- Mini-BCI: 50 units (10% as determined from 160/1600)

- Server-BCI: 0 units (0%)
- Gamer-ECS: 109 units (21.875% as determined from 350/1600)
- Mini-ECS: 41 units (8.125% as determined from 130/1600)
- Server-ECS: 175 units (35% as determined from 560/1600)

However, in cases where there are already **ForecastDetail** records for the part, customer, forecast category and period into which the forecast is being disaggregated, then those quantities are used to determine the disaggregation ratios rather than the calculated values in the **DisaggregationRateByPartCustomer** table. For example, suppose the following two records exist in the **ForecastDetail** table.

Date	Header. Category	PartCustomer .Part	PartCustomer. Customer	Quantity
07-01-15	SalesForecast	Gamer	ECS	60
07-01-15	SalesForecast	Mini	BCI	90

In this case, if a sales forecast of 500 units for 07-01-15 is entered against the same **ProductGroup** using the S&OP Sales Forecast workbook, then disaggregation calculations would assign 200 units to Gamer:ECS and 300 units to Mini:BCI.

Configuring general forecast disaggregation

General forecast disaggregation in RapidResponse is based on many configurable parameters. For example, the source of data used to determine disaggregation rates as well as the calendars used to bucket and disaggregate data can be specified. Most disaggregation parameters can be specified at the global or default level as well as overridden for individual part-customers and forecast categories.

This section covers the following aspects of configuring general forecast disaggregation:

- Specify range for calculating disaggregation rates
- Use historical actual or consensus forecast data
- Specify calendars for disaggregation
- Define aggregate part customer relationships
- Override calculated disaggregation rates

In addition, examples of different combinations of forecast disaggregation settings and the calculated disaggregation rates they produce can be seen in "Examples: Configuring general disaggregation rates" on page 2093.

Specify range for calculating disaggregation rates

The **DisaggregationRateByPartCustomer** table can report disaggregation rates in a date range bounded by **SOPAnalyticsConfiguration.CalcForecastStartDate** (inclusive) and **SOPAnalyticsConfiguration.CalcForecastEndDate** (exclusive). These two fields rely on input parameters in the **SOPAnalyticsConfiguration** table, and are calculated as follows:

- **CalcForecastStartDate**—`CalcHistoricalEndDate + MAX (0, ForecastStartOffset) CycleCalendar`

  where `CalcHistoricalEndDate = RunDate.FirstDate + 0 CycleCalendar`

- **CalcForecastEndDate**—`CalcForecastStartDate + MAX (1, ForecastEndOffset) CycleCalendar`

For example, if **CycleCalendar** were set to "Month", **ForecastStartOffset** set to "0", and **ForecastEndOffset** set to "24", then disaggregation rates could be calculated from the start of the current month and continuing for the next two years (24 months).

The **CalcHistoricalEndDate** value shown in the calculation above is also used to determine the latest date (exclusive) on which historical data can be collected for use in calculating disaggregation rates. For example, if set to a Monthly calendar, then historical data can be collected up to, but not including, the start of the current month.

Note: The individual dates on which disaggregation rates are actually reported within a defined range is determined by the disaggregation calendar. For more information, see "Specify calendars for disaggregation" on page 2090.

Use historical actual or consensus forecast data

Data used to calculate disaggregation rates for a given part-customer and forecast category can come from either historical demand actuals or the consensus forecast. This is set at the forecast category level using the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table. This field has two settings; "Actuals" (the default setting) and "ConsensusForecast".

Actuals

When the **DisaggregationQuantityRule** field is set to "Actuals", then disaggregation rates for the forecast category are calculated using historical actual demand data. That is, disaggregation rates for the forecast category are set proportionally to the distribution of historical demands between part-customers. The historical demand category to use, along with the number of intervals over which to collect historical data from that category, is configurable for each part-customer and forecast category combination. However, default values for these settings should also be provided.

Default disaggregation parameters are specified in the **SOPAnalyticsConfiguration** table which should contain exactly one record per scenario. On this table, the **DisaggregationActualsCategory** reference indicates the historical demand category whose data should be used and the **DisaggregationHistoricalIntervalCount** field indicates the number of historical intervals back from **CalcHistoricalEndDate** to collect data from the specified category when calculating disaggregation rates. The value specified in **DisaggregationHistoricalIntervalCount** should be expressed in **InnerCalendar** intervals (for information about the inner calendar, see "Inner and Outer calendars" on page 2091). For example, if **DisaggregationActualsCategory** is set to "ActualDemand", and **DisaggregationHistoricalIntervalCount** is set to "12", then twelve **InnerCalendar** intervals (typically, months) of historical demand from the ActualDemand category are used in calculating disaggregation rates (by default).

If disaggregation parameters other than the defaults are required for particular part-customers and forecast categories, then the **ForecastDisaggregationParameters** and **ForecastDisaggregationParametersByCategory** tables can be used.

Records in the **ForecastDisaggregationParameters** table apply to specific part-customer and forecast category combinations as defined by the table's **Header** reference. Records in the **ForecastDisaggregationParametersByCategory** apply universally across a forecast category as defined by the table's **Category** reference. On each of these tables, the **ActualsCategory** reference indicates the historical demand category whose data should be used, and the **HistoricalIntervalCount** field indicates the number of historical intervals over which to collect data from the specified category when calculating disaggregation rates.

Thus, if a record exists in the **ForecastDisaggregationParameters** table for a given part-customer and forecast category combination, then its parameters will always be used when calculating disaggregation rates for the part-customer. Otherwise, if a record exists in the **ForecastDisaggregationParametersByCategory** table for a given forecast category, then its parameters will be used when calculating disaggregation rates for part-customers associated with that category. Otherwise, if there are no records applicable to a given part-customer in either of these tables, then that part-customer's disaggregation rates will be calculated using default parameters in the **SOPAnalyticsConfiguration** table.

ConsensusForecast

When the **DisaggregationQuantityRule** is set to "ConsensusForecast", all disaggregation rates pertaining to a given forecast category are set using the calculated consensus forecast values for its part customers. This means all part-customers associated with a forecast category of this type will have a disaggregation rate for the category determined from their calculated consensus forecast quantity. With this option, the range over which to collect the consensus forecast quantities for determining disaggregation rates is between the dates defined by **CalcForecastStartDate** and **CalcForecastEndDate** (exclusive) on the **SOPAnalyticsConfiguration** table.

Note: If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "ConsensusForecast" to calculate forecast disaggregation rates, the disaggregation rates are derived directly from the consensus forecast with no additional modifications applied. The adjustments in the **ForecastDisaggregationOverride** table and the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are not used.

Specify calendars for disaggregation

In order to calculate general disaggregation rates, several calendars should be defined. If the **DisaggregationQuantityRule** is set to "Actuals", the disaggregation calendar, inner calendar, and outer calendar should be defined. If the **DisaggregationQuantityRule** is set to "ConsensusForecast", only the disaggregation calendar should be defined, the inner and outer calendars are ignored. For each of these calendars, a default value can be specified in the **SOPAnalyticsConfiguration** table, while part-customer and forecast category specific values can be used to override the defaults as required.

Disaggregation calendar

The disaggregation calendar holds the actual dates on which disaggregation rates can be reported in the **DisaggregationRateByPartCustomer** table. This calendar serves as the base calendar for disaggregation and thereby defines the intervals that disaggregated forecast quantities are ultimately stored in. Therefore, the disaggregation calender should be the same or smaller than the inner calendar. If this calendar is different (smaller) than the inner calendar, then disaggregated forecast

quantities are spread evenly from inner calendar intervals to disaggregation calendar intervals before being added to the **ForecastDetail** table. For example, if the inner calendar is "Month" and the disaggregation calendar is "Week", then disaggregated forecast quantities are spread evenly across all weeks in a given month.

To specify a default disaggregation calendar, the **DisaggregationCalendar** field on the **SOPAnalyticsConfiguration** table is used. If a given part-customer requires a disaggregation calendar other than the default, it can be specified in the **DisaggregationCalendar** field on the **PartCustomer** table. For example, a standard monthly calendar might be used as the default disaggregation calendar, but some customers might use a fiscal month calendar and others a weekly calendar.

Note: For examples of how different calendar settings affect the calculation of forecast disaggregation rates, see "Example 1: Monthly disaggregation" on page 2093 and "Example 2: Quarter in year disaggregation" on page 2095.

Inner and Outer calendars

The inner and outer calendars are used to bucket historical data for use in calculating disaggregation rates.

If the historical data is not seasonal in nature, then the inner and outer calendars should be set to the same calendar value. For example, a monthly inner and outer calendar might typically be used to bucket data when determining disaggregation rates. In this case, the disaggregation rate applicable to each point on the inner calendar will be the same.

If data is seasonal in nature, then the outer calendar is used to define the overall period in which seasons are contained, and the inner calendar is used to define the length of each season. For example, the outer calendar might be year, while the inner calendar might be month or quarter. In this case, the inner calendar should be set to a smaller calendar value than the outer calendar, and the disaggregation rate applicable to each point on the inner calendar will typically be different.

To specify default inner and outer calendars, the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are used. If a given part-customer and forecast category combination requires inner and outer calendar settings other than those defaults, then the **InnerCalendar** and **OuterCalendar** fields on a **ForecastDisaggregationParameters** record added for that part-customer and forecast category should be used. For example, the default inner and outer calendars might be set to month, and then overridden for part-customers that have seasonal demand trends. Alternatively, if all part-customers associated with a given forecast category should use the same calendar overrides, then the **InnerCalendar** and **OuterCalendar** fields on a **ForecastDisaggregationParametersByCategory** record added for that forecast category should be used.

Define aggregate part customer relationships

As described earlier in this chapter, forecasts entered against hierarchies in RapidResponse are disaggregated to forecast category and part-customer combinations. Each **PartCustomer** record typically associates an actual part with a unique customer for that part. However, in some cases, it might not be necessary to disaggregate the forecast down to such a detailed level. For example, a part's forecast might be made at the aggregate level for all customers within a given sales territory. Similarly, a customer's forecast might be made at the aggregate level for all parts belonging to a given product family.

To support forecasting at higher levels of aggregation, the **AggregatePartCustomer** table can be used to associate actual part customers with aggregate or summary part customers. The **Aggregate** field on this table should reference a **PartCustomer** record being used to aggregate actual part customers together, while the **Component** field should reference a **PartCustomer** record representing an actual part and customer combination being aggregated. Therefore, records for both real and aggregate part-customers must be added to the **PartCustomer** table before relationships can be defined using the **AggregatePartCustomer**.

Once a part-customer is defined as a component on an **AggregatePartCustomer** record, then disaggregation rates will not be calculated for that actual part-customer nor will forecast be disaggregated to that part-customer. Instead, disaggregation will only be made to the level of the aggregate **PartCustomer** (with disaggregation rates for each aggregate calculated by summing actuals from its associated component part-customers). Since a given aggregate part-customer record is typically associated with multiple component part-customers, this implies fewer records will be generated in the **DisaggregationRateByPartCustomer** and **ForecastDetail** tables when aggregate part customers have been defined.

Notes:

- In order for a given **AggregatePartCustomer** value to be used in calculating forecast disaggregation rates, it must reference an **AggregatePartCustomerType** record that has a **ProcessingRule** setting of "Use". For more information, see "AggregatePartCustomerType table" on page 670.
- The **PartCustomer.ForecastItem.Path** values for both the "Aggregate" and "Component" part customers must match in order for the aggregate part-customer disaggregation logic to work.
- For an example of configuring aggregate part customers, see "Example 3: Aggregate part-customer disaggregation" on page 2096.

Override calculated disaggregation rates

If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "Actuals," the calculated disaggregation rates can be manually overridden at the part-customer level. For example, override rates might be specified in cases where there is not enough historical data available to calculate the disaggregation rate for a particular part-customer, or where history does not reflect the desired forecast distribution.

The **ForecastDisaggregationOverride** table allows for manually providing disaggregation rates for part-customer and forecast category combinations (as defined through the **Header** reference). The **Quantity** field is used to specify the new disaggregation rate and the **Date** field indicates the date on which the disaggregation rate becomes effective. The **Quantity** value is then considered effective from the specified **Date** until the next **DisaggregationCalendar** marker (at which point another rate override can be provided or else the calculated disaggregation rate will be used).

Note that only **ForecastDisaggregationOverride** records with **Date** values that actually fall on markers belonging to the **DisaggregationCalendar** are considered valid. For example, if using a monthly disaggregation calendar and August 3, 2015 defines the beginning of the working calendar for August, then only records with a **Date** of 08/03/2015 would be used as overrides for August. If a record was added with a **Date** of 08/02/2015 or 08/10/2015, it would be ignored.

Additionally, a default disaggregation override category can be specified using the **ForecastDisaggregationOverrideCategory** reference on the **SOPAnalyticsConfiguration** table. If a

ForecastDisaggregationOverride record is then added for that default category and a particular part-customer, the disaggregation rate specified is then applied to all forecast categories for that part-customer (unless those other categories have their own **ForecastDisaggregationOverride** record specified for the part-customer on the effective date). If the same disaggregation rate override should be applied to all of a part customer's forecast categories, this can reduce the number of records that need to be created and maintained.

Examples: Configuring general disaggregation rates

This section includes the following examples of how the various configuration settings discussed earlier in this chapter impact calculated disaggregation rates:

- "Example 1: Monthly disaggregation" on page 2093
- "Example 2: Quarter in year disaggregation" on page 2095
- "Example 3: Aggregate part-customer disaggregation" on page 2096

Example 1: Monthly disaggregation

Suppose by default that all disaggregation rates should be generated monthly and calculated based on (non-seasonal) monthly data using 6 months of history from the "Shipments" demand category. This requirement could be supported using the following values on the **SOPAnalyticsConfiguration** record.

Table 44.2: SOPAnalyticsConfiguration values

Actuals Category	Disaggregation Historical IntervalCount	Disaggregation Calendar	Disaggregation InnerCalendar	Disaggregation OuterCalendar
Shipments	6	Month	Month	Month

Further assume a **RunDate** of June 2, 2015 and the following set of historical actual demands in the "Shipment" category for each part-customer.

Table 44.3: Historical monthly demand actuals in "Shipment" category

Part: Customer	May 2015	Apr 2015	Mar 2015	Feb 2015	Jan 2015	Dec 2014	Nov 2014	Oct 2014	Sep 2014
Gamer:BCI	775	600	785	625	800	650	700	675	805
Gamer:ECS	740	675	825	750	650	725	800	850	760
Server:BCI	450	500	475	500	550	600	525	610	520
Server:ECS	600	575	625	475	550	650	575	500	480

Assuming the "SalesForecast" forecast category has been defined for each part-customer, the following monthly disaggregation rates would be calculated for each part-customer. Note that the monthly rates would be repeated for each month until the date in **SOPAnalyticsConfiguration.ForecastEndDate** was reached.

Table 44.4: DisaggregationRateByPartCustomer values

Date	Category	Part: Customer	Quantity	Details of disaggregation rate calculation
06/01/15	SalesForecast	Gamer:BCI	705.83	(775+600+785+625+800+650) / 6
06/01/15	SalesForecast	Gamer:ECS	727.50	(740+675+825+750+650+725) / 6
06/01/15	SalesForecast	Server:BCI	512.50	(450+500+475+500+550+600) / 6
06/01/15	SalesForecast	Server:ECS	579.17	(600+575+625+475+550+650) / 6
07/01/15	SalesForecast	Gamer:BCI	705.83	
07/01/15	SalesForecast	Gamer:ECS	727.50	
07/01/15	SalesForecast	Server:BCI	512.50	
07/01/15	SalesForecast	Server:ECS	579.17	

Assume a further requirement that forecasts for customer ECS should be disaggregated weekly. This could be accomplished by providing a valid **DisaggregationCalendar** reference on the relevant **PartCustomer** records as shown in the following table.

Table 44.5: Overriding DisaggregationCalendar on PartCustomer records

Part	Customer	DisaggregationCalendar
Gamer	ECS	Week
Server	ECS	Week

As a result, disaggregation rates would now be spread and reported weekly for customer ECS, while still reported monthly for customer BCI. Note that the weekly disaggregation rates vary based on the number of weeks in a given month.

Table 44.6: DisaggregationRateByPartCustomer values (after DisaggregationCalendar override)

Date	Category	Part: Customer	Quantity	Details of disaggregation rate calculation
06/01/15	Shipments	Gamer:BCI	705.83	(775+600+785+625+800+650) / 6
06/01/15	Shipments	Gamer:ECS	145.50	((740+675+825+750+650+725) / 6) / 5
06/01/15	Shipments	Server:BCI	512.50	(450+500+475+500+550+600) / 6
06/01/15	Shipments	Server:ECS	115.83	((600+575+625+475+550+650) / 6) / 5
06/08/15	Shipments	Gamer:ECS	145.50	
06/08/15	Shipments	Server:ECS	115.83	
06/15/15	Shipments	Gamer:ECS	145.50	
06/15/15	Shipments	Server:ECS	115.83	
06/22/15	Shipments	Gamer:ECS	145.50	
06/22/15	Shipments	Server:ECS	115.83	
06/29/15	Shipments	Gamer:ECS	145.50	
06/29/15	Shipments	Server:ECS	115.83	

Table 44.6: DisaggregationRateByPartCustomer values (after DisaggregationCalendar override)

Date	Category	Part: Customer	Quantity	Details of disaggregation rate calculation
07/01/15	Shipments	Gamer:BCI	705.83	
07/01/15	Shipments	Server:BCI	512.50	
07/06/15	Shipment	Gamer:ECS	181.88	((740+675+825+750+650+725) / 6) / 4
07/06/15	Shipment	Server:ECS	144.79	((600+575+625+475+550+650) / 6) / 4

Example 2: Quarter in year disaggregation

Assume the following **SOPAnalyticsConfiguration** record defines default disaggregation parameters.

Table 44.7: Default SOPAnalyticsConfiguration settings

Actuals Category	Disaggregation Historical IntervalCount	Disaggregation Calendar	Disaggregation InnerCalendar	Disaggregation OuterCalendar
Shipments	6	Month	Month	Month

Further assume a **RunDate** of June 2, 2015 and the following monthly historical demands from the "Shipments" category for three customers of the "Chip" part.

Table 44.8: Historical demand actuals in "Shipments" category

Part Customer	Mar 2015	Feb 2015	Jan 2015	Dec 2014	Nov 2014	Oct 2014	Sep 2014	Aug 2014	Jul 2014	Jun 2014	May 2014	Apr 2014
Chip:BCI	600	550	400	800	725	600	1025	1050	975	800	875	980
Chip:ECS	560	450	500	600	675	650	990	1150	1100	925	1000	910
Chip:LZR	575	500	450	575	700	750	1100	950	1000	950	925	860

Finally, assume a requirement that all customers for part "Chip" should have their monthly disaggregation rates calculated based on seasonal demand data from four quarters ago. This requirement could be satisfied by adding records to the **ForecastDisaggregationParameter** table as follows. Note that the records shown would override disaggregation parameters for the "SalesForecast" category only. Assuming the same overrides should apply to other forecast categories, then separate **ForecastDisaggregationOverride** records could be added for those categories, or the **SOPAnalyticsConfiguration.ForecastDisaggregationOverrideCategory** field set to reference the "SalesForecast" category.

Table 44.9: ForecastDisaggregationParameter settings

Header. Category	Header. PartCustomer	Actuals Category	Historical IntervalCount	Inner Calendar	Outer Calendar
SalesForecast	Chip:BCI	Shipments	4	Quarter	Year
SalesForecast	Chip:ECS	Shipments	4	Quarter	Year
SalesForecast	Chip:LZR	Shipments	4	Quarter	Year

Based on the settings shown previously, the following table shows some of the **DisaggregationRateByPartCustomer** records that would be generated for a particular part-customer combination (Chip:ECS) and category (SalesForecast).

Table 44.10: DisaggregationRateByPartCustomer values for one PartCustomer

Date	Category	Part: Customer	Quantity	Disaggregation rate calculation
06/01/15	SalesForecast	Chip:ECS	945	(925+1000+910) / 3
07/01/15	SalesForecast	Chip:ECS	1080	(1100+1150+990) / 3
08/03/15	SalesForecast	Chip:ECS	1080	
09/01/15	SalesForecast	Chip:ECS	1080	
10/01/15	SalesForecast	Chip:ECS	641.67	(650+675+600) / 3
11/02/15	SalesForecast	Chip:ECS	641.67	
12/01/15	SalesForecast	Chip:ECS	641.67	
01/01/16	SalesForecast	Chip:ECS	503.33	(500+450+560) / 3
02/01/16	SalesForecast	Chip:ECS	503.33	
03/01/16	SalesForecast	Chip:ECS	503.33	
04/01/16	SalesForecast	Chip:ECS	945	
05/02/16	SalesForecast	Chip:ECS	945	

Example 3: Aggregate part-customer disaggregation

Suppose a single **SOPAnalyticsConfiguration** record that defines disaggregation rates for all part customers as shown in the following table.

Table 44.11: Default SOPAnalyticsConfiguration settings for monthly disaggregation

Actuals Category	Disaggregation Historical IntervalCount	Disaggregation Calendar	Disaggregation InnerCalendar	Disaggregation OuterCalendar
ActualDemand	4	Month	Month	Month

Further assume the following actual part and customer combinations have been defined.

Table 44.12: PartCustomer values

Part	Customer
ChipX	BCI
ChipX	ECS
ChipX	LZR
ChipX	QQ
ChipX	ZZ

However, instead of disaggregating forecast to the detailed part and customer level, suppose a requirement that part forecasts only need to be disaggregated to the customer region level with

customers “BCI”, “ECS”, and “LZR” associated with the “East” region, and customers “QQ” and “ZZ” associated with the “West” region.

To model this situation, the following values could to be added to the **Customer** table.

Table 44.13: Customer values

Id
West
East

Next, appropriate **PartCustomer** records could be added to identify the new regional levels at which the forecast for “ChipX” should be aggregated.

Table 44.14: PartCustomer values

Part	Customer
ChipX	East
ChipX	West

AggregatePartCustomer records should be added to associate the detailed or component part-customer combinations with the aggregate part-customers for which disaggregation rates should be calculated.

Table 44.15: AggregatePartCustomer values

Component .Part	Component. Customer	Aggregate. Part	Aggregate. Customer
ChipX	BCI	ChipX	East
ChipX	ECS	ChipX	East
ChipX	LZR	ChipX	East
ChipX	QQ	ChipX	West
ChipX	ZZ	ChipX	West

Finally, assume a RunDate of June 2, 2015 and the following set historical demand actuals in the “ActualDemand” category pertaining to the component part-customers.

Table 44.16: Historical monthly demand actuals in “ActualDemand” category

Part: Customer	May 2015	Apr 2015	Mar 2015	Feb 2015	Jan 2015	Dec 2014
ChipX:BCI	775	600	785	625	800	650
ChipX:ECS	740	675	825	750	650	725
ChipX:LZR	450	500	475	500	550	600
ChipX:QQ	600	575	625	475	550	650
ChipX:ZZ	575	400	600	500	675	550

Based on the **AggregatePartCustomer** relationships that have been defined, the following **DisaggregationRateByPartCustomer** records would be generated for each period and forecast

category relevant to these aggregate part-customers. Note that disaggregation rates would be calculated for the component part-customers associated with these aggregates.

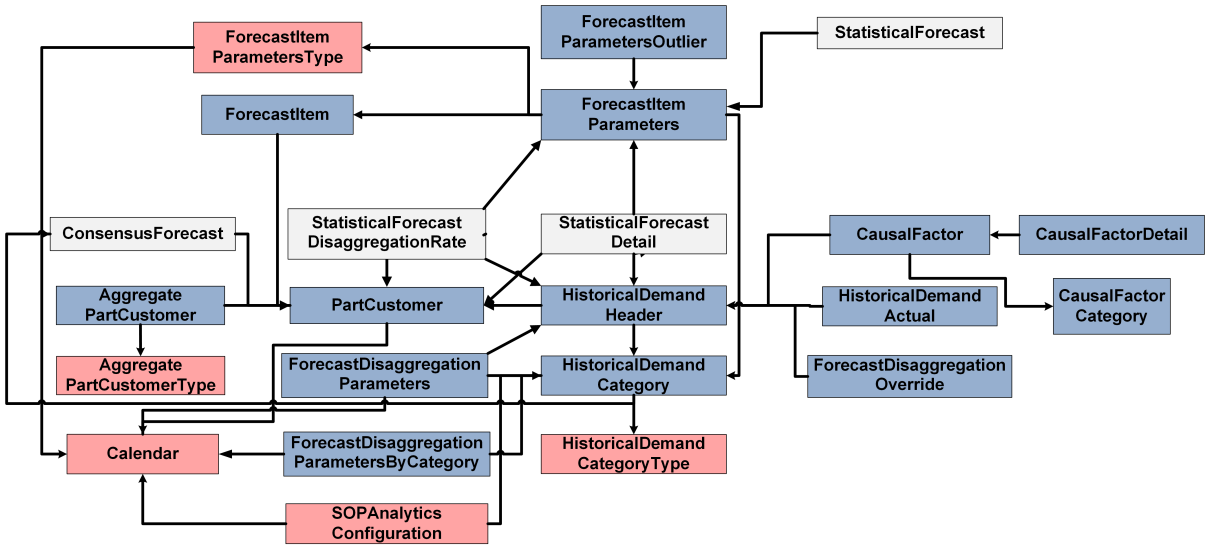
Table 44.17: DisaggregationRateByPartCustomer values (for aggregate part-customers)

Date	Part: Customer	Quantity	Disaggregation rate calculation
06/01/15	ChipX:East	1806.25	(775+600+785+625)+(740+675+825+750)+(450+500+475+500) / 4
06/01/15	ChipX:West	1087.50	(600+575+625+475)+(575+400+600+ 500) / 4

Statistical disaggregation data model

The RapidResponse data model includes a number of tables that either provide data used as input into statistical forecast disaggregation calculations, hold settings used to determine how disaggregation calculations are performed, or report and use the results of those calculations. The following illustration shows the relationships between these input, control, and calculated tables.

Figure 44.2 Tables used to disaggregate the statistical forecast



The following describes each of the tables shown in the figure and briefly outlines their usage in statistical forecast disaggregation.

Table 44.18: Statistical forecast disaggregation tables

Table	Type	Description
AggregatePart Customer	Input	Groups part-customers together for cases where the statistical forecast does not need to be fully disaggregated to detailed part and customer combinations. For more information, see "AggregatePartCustomer table" on page 179.
AggregatePart CustomerType	Control	Enables or disable aggregate part-customer definitions for use in forecast disaggregation calculations. For more information, see "AggregatePartCustomerType table" on page 670.
Calendar	Control	Identifies named calendars (and referenced by various disaggregation settings such as the inner and outer disaggregation calendars). For more information, see "Calendar table" on page 695.
CausalFactor	Input	Stores causal factors used to adjust historical data for data errors or unusual demand events before a statistical forecast is generated. It references the part customer and historical actual the causal factor applies to (through the HistoricalDemandHeader reference), as well as the category the causal factor is associated with (through the CausalFactorCategory reference). For more information, see "CausalFactor table" on page 239.
CausalFactorCategory	Input	Stores categories defined in your company for grouping causal factors. For example, causal factor categories might include things such as promotions and weather events. For more information, see "CausalFactorCategory table" on page 240.
CausalFactorDetail	Input	Includes date and quantity details that belong to a given causal factor through the reference to the CausalFactor table. The quantities in this table are taken into account by StatisticalForecastDetail and StatisticalForecastDisaggregationRate. For more information, see "CausalFactorDetail table" on page 241.
ConsensusForecast	Calculated	Contains the consensus forecast generated from the sales and operations planning process in RapidResponse. Each record in this table contains a calculated forecast amount for a part and customer combination on a given forecast date. For more information, see "ConsensusForecast table" on page 966.
Forecast Disaggregation Override	Input	Specifies general disaggregation rates for part-customer and forecast category combinations. This table is used only if you choose to manually insert rates instead of having the rates calculated by RapidResponse. For more information, see "ForecastDisaggregationOverride table" on page 290.

Table 44.18: Statistical forecast disaggregation tables

Table	Type	Description
Forecast Disaggregation Parameters	Input	Holds many of the default disaggregation settings on a part-customer and forecast category basis. It overrides the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates. For more information, see "ForecastDisaggregationParameters table" on page 287.
Forecast Disaggregation ParametersBy Category	Input	Holds many of the default disaggregation settings on a forecast category basis. It overrides the default parameters set by the **SOPAnalyticsConfiguration** table when determining disaggregation rates. For more information, see "ForecastDisaggregationParametersByCategory table" on page 288.
ForecastItem	Input	Defines forecast items, made up of combinations of levels from one or more hierarchies, for which statistical forecasts can be generated. For more information, see "ForecastItem table" on page 292.
ForecastItem Parameters	Input	Holds parameters used in generating a statistical forecast for a given forecast item. For more information, see "ForecastItemParameters table" on page 296.
ForecastItem ParametersOutlier	Input	Stores outlier adjustments at the forecast item level, which means that items that belong to more than one forecast category in ForecastItemParameters can use different outlier adjustments for each category. The quantities in this table are taken into account in the disaggregated quantities reported in StatisticalForecastDetail. For more information, see "ForecastItemParametersOutlier table" on page 322.
ForecastItem ParametersType	Control	Contains the settings used in generating a statistical forecast for a given forecast item, and provides a control that determines if the historical actual category is used in calculating the statistical forecast, as well as calculating disaggregation rates. For more information, see "ForecastItemParametersType table" on page 727.
HistoricalDemand Actual	Input	Holds historical demand data in various categories which, depending on configuration, can be used in calculating statistical disaggregation rates for a given part, customer, forecast item, and category. For more information, see "HistoricalDemandActual table" on page 325.
HistoricalDemand Category	Input	Identifies categories of forecasted and actual demand. For more information, see "HistoricalDemandCategory table" on page 329.

Table 44.18: Statistical forecast disaggregation tables

Table	Type	Description
HistoricalDemand CategoryType	Control	Defines how demand data that is associated with a given demand category is processed in RapidResponse. For example, it determines whether historical actuals or the consensus forecast should be used to calculate disaggregation rates for a statistical forecast category. For more information, see "HistoricalDemandCategoryType table" on page 736.
HistoricalDemand Header	Input	Identifies unique part, customer, and historical demand category combinations. For more information, see "HistoricalDemandHeader table" on page 334.
PartCustomer	Input	Contains unique part and customer combinations for which historical data is provided and forecast can be generated in the S&OP process. Also allows overriding the standard disaggregation calendar on a part-customer basis. For more information, see "PartCustomer table" on page 444.
SOPAnalytics Configuration	Control	Specifies the default values for disaggregation settings such as the historical demand category, number of historical intervals, and calendars to use when calculating disaggregation rates. This table should hold a single record per scenario. For more information, see "SOPAnalyticsConfiguration table" on page 892.
StatisticalForecast	Calculated	Holds the statistical forecast calculated for a given forecast item (that is, before any disaggregation is applied). The history used to generate the statistical forecast is the sum of the HistoricalDemandActual, CausalFactorDetail, and ForecastItemParametersOutlier records for the forecast items. For more information, see "StatisticalForecast table" on page 1179.
StatisticalForecast Detail	Calculated	Holds the current statistical forecast data after it has been disaggregated to the detailed part and customer combinations. This table reflects any quantities that are reported in CausalFactorDetail and ForecastItemParametersOutlier. For more information, see "StatisticalForecastDetail table" on page 1181.
StatisticalForecast DisaggregationRate	Calculated	Holds disaggregation rates (calculated based on settings provided in tables like SOPAnalyticsConfiguration) that are used to disaggregate the statistical forecast to the detailed part and customer level. Any causal quantities that are reported in CausalFactorDetail are taken into account. For more information, see "StatisticalForecastDisaggregationRate table" on page 1181.

Statistical disaggregation rates

Statistical forecasts are generated for forecast items (which are made up of combinations of levels from one or more hierarchies in RapidResponse), and then reported in the **StatisticalForecast** table. After the statistical forecast has been calculated, the forecast quantities must be disaggregated down to the detail level using calculated disaggregation rates stored in the **StatisticalForecastDisaggregationRate** table.

Each record in the **StatisticalForecastDisaggregationRate** table holds a calculated **Quantity** (fractional disaggregation rate) pertaining to a particular part-customer and set of forecast item parameters used in generating the forecast for a given forecast item. The rates in this table are then used to produce the disaggregated forecast quantities that get reported in the **StatisticalForecastDetail** table.

For example, suppose a **StatisticalForecast** record containing a monthly forecast as shown in the following table.

Table 44.19: StatisticalForecast values

Date	ItemParameters. ForecastCategory	ItemParameters. ItemPath	Quantity
07-01-15	Statistical	System	500

Further, suppose a **StatisticalForecastDisaggregationRate** record containing rates for the "System" forecast item as shown in the following table.

Table 44.20: StatisticalForecasDisaggregationRate values

Date	ItemParameters. ForecastCategory	ItemParameters. ItemPath	Part	Customer	Quantity
07-01-15	Statistical	System	Gamer	BCI	.25
07-01-15	Statistical	System	Mini	BCI	.15
07-01-15	Statistical	System	Server	ECS	.60

Assuming monthly disaggregation, the following corresponding records would then be generated in the **StatisticalForecastDetail** table.

Table 44.21: StatisticalForecasDisaggregationRate values

Date	ItemParameters. ForecastCategory	ItemParameters. ItemPath	Part	Customer	Quantity
07-01-15	Statistical	System	Gamer	BCI	125
07-01-15	Statistical	System	Mini	BCI	75
07-01-15	Statistical	System	Server	ECS	300

Configuring statistical forecast disaggregation

Statistical forecast disaggregation rates are based on many configurable parameters that can be specified at both the global or default level as well as overridden at the part-customer level.

This section covers the following aspects of configuring general forecast disaggregation:

- "Use historical actual or consensus forecast data" on page 2103
- "Specify calendars for disaggregation" on page 2104
- "Define aggregate part-customer relationships" on page 2105
- "Override calculated disaggregation rates" on page 2107

Use historical actual or consensus forecast data

Data used to calculate statistical disaggregation rates for a given part-customer and statistical forecast category can come from either historical demand actuals or the consensus forecast. This is set at the statistical forecast category level using the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table. This field has two settings; "Actuals" (the default setting) and "ConsensusForecast".

Actuals

When the **DisaggregationQuantityRule** field is set to "Actuals", then disaggregation rates for the statistical forecast are calculated using historical actual demand data. That is, disaggregation rates for the statistical forecast category are set proportionally to the distribution of historical demands between part-customers. The historical demand category to use, along with the number of intervals over which to collect historical data from that category, is configurable for each part-customer and forecast category combination. However, default values for these settings should also be provided.

Default disaggregation parameters are specified in the **SOPAnalyticsConfiguration** table which should contain one record per scenario. On this table, the **DisaggregationActualsCategory** reference indicates the historical demand category whose data should be used as long as the associated forecast item references a **ForecastItemParametersType.DisaggregationActualsCategoryRule** setting of "Default". Otherwise, if **DisaggregationActualsCategoryRule** is set to "UseOwn", then the historical demand category to use for calculating disaggregation rates is automatically set to the **ForecastItemParametersType.ActualsCategory** reference value.

In either case, the **DisaggregationHistoricalIntervalCount** field on the **SOPAnalyticsConfiguration** table indicates the number of historical intervals back from **CalcHistoricalEndDate** to collect data from the specified category when calculating disaggregation rates. The **DisaggregationHistoricalIntervalCount** value should be expressed in intervals of the **InnerCalendar** (for information about the inner calendar, see "Inner and Outer calendars" on page 2091). For example, if **ForecastItemParametersType.DisaggregationActualsCategoryRule** is set to "Default", **DisaggregationActualsCategory** is set to "ActualDemand", and **DisaggregationHistoricalIntervalCount** is set to "12", then twelve **InnerCalendar** intervals (typically, months) of historical demand from the ActualDemand category is used in calculating statistical disaggregation rates (by default).

If disaggregation parameters other than the defaults are required for particular part-customers and forecast categories, then the **ForecastDisaggregationParameters** and **ForecastDisaggregationParametersByCategory** tables can be used.

Records in the **ForecastDisaggregationParameters** table apply to specific part-customer and forecast category combinations as defined by the table's **Header** reference. Records in the **ForecastDisaggregationParametersByCategory** apply universally across a forecast category as defined by the table's **Category** reference. On each of these tables, the **ActualsCategory** reference indicates the historical demand category whose data should be used, and the **HistoricalIntervalCount** field indicates the number of historical intervals over which to collect data from the specified category when calculating statistical forecast disaggregation rates.

Thus, if a record exists in the **ForecastDisaggregation** table for a given part-customer and forecast category combination, then its parameters will always be used when calculating disaggregation rates for the part-customer. Otherwise, if a record exists in the **ForecastDisaggregationByCategory** table for a given forecast category, then its parameters will be used when calculating disaggregation rates for part-customers associated with that category. Otherwise, if there are no records applicable to a given part-customer in either of these tables, then that part-customer's disaggregation rates will be calculated using default parameters in the **SOPAnalyticsConfiguration** table.

ConsensusForecast

When the **DisaggregationQuantityRule** is set to "ConsensusForecast", all disaggregation rates pertaining to a statistical forecast category are set using the calculated consensus forecast values for its part customers. This means all part-customers associated with a forecast category of this type will have the statistical disaggregation rate for the category determined from their calculated consensus forecast quantity. With this option, the range over which to collect the consensus forecast quantities for determining disaggregation rates is between the dates defined by **CalcForecastStartDate** and **CalcForecastEndDate** (exclusive) on the **SOPAnalyticsConfiguration** table.

Note: If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "ConsensusForecast" to calculate forecast disaggregation rates, the disaggregation rates are derived directly from the consensus forecast with no additional modifications applied. The adjustments in the **ForecastDisaggregationOverride** table and the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are not used.

Specify calendars for disaggregation

In order to calculate general disaggregation rates, several calendars should be defined. If the **DisaggregationQuantityRule** is set to "Actuals", the disaggregation calendar, inner calendar, and outer calendar should be defined. If the **DisaggregationQuantityRule** is set to "ConsensusForecast", only the disaggregation calendar should be defined, the inner and outer calendars are ignored. For each of these calendars, a default value can be specified in the **SOPAnalyticsConfiguration** table, while part-customer and forecast category specific values can be used to override the defaults as required.

Disaggregation calendar

The disaggregation calendar holds dates on which disaggregation rates can be reported in the **StatisticalForecastDisaggregationRate** table. This calendar serves as the base calendar for

disaggregation and thereby defines the intervals that disaggregated statistical forecast quantities are ultimately stored in. Therefore, the disaggregation calender should be the same or smaller than the inner calendar. If this calendar is different (smaller) than the inner calendar, then disaggregated forecast quantities are spread evenly from inner calendar intervals to disaggregation calendar intervals before being added to the **StatisticalForecastDetail** table. For example, if the inner calendar is "Month" and the disaggregation calendar is "Week", then disaggregated statistical forecast quantities are spread evenly across all weeks in a given month.

To specify a default disaggregation calendar, the **DisaggregationCalendar** field on the **SOPAnalyticsConfiguration** table is used. If a given part-customer requires a disaggregation calendar other than the default, it can be specified in the **DisaggregationCalendar** field on the **PartCustomer** table. For example, a standard monthly calendar might be used as the default disaggregation calendar, but some customers might use a fiscal month calendar and others a weekly calendar.

Inner and Outer calendars

The inner and outer calendars are used to bucket historical data for use in calculating disaggregation rates.

If the data is not seasonal in nature, then the inner and outer calendars should be set to the same calendar value. For example, a monthly inner and outer calendar might typically be used to bucket data when determining disaggregation rates. In this case, the disaggregated quantity for each point on the inner calendar will be the same.

If data is seasonal in nature, then the outer calendar is used to define the overall period in which seasons are contained, and the inner calendar is used to define the length of each season. For example, the outer calendar might be year, while the inner calendar might be month or quarter. In this case, the inner calendar should be set to a smaller calendar value than the outer calendar, the disaggregated quantity for each point on the inner calendar will typically be different.

To specify default inner and outer calendars, the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are used. If a given part-customer and forecast category combination requires inner and outer calendar settings other than those defaults, then the **InnerCalendar** and **OuterCalendar** fields on a **ForecastDisaggregationParameters** record added for that part-customer and forecast category should be used. For example, the default inner and outer calendars might be set to month, and then overridden for part-customers that have seasonal demand trends. Alternatively, if all part-customers associated with a given forecast category should use the same calendar overrides, then the **InnerCalendar** and **OuterCalendar** fields on a **ForecastDisaggregationParametersByCategory** record added for that forecast category should be used.

Define aggregate part-customer relationships

As discussed earlier in this chapter, statistical forecasts are typically generated and then disaggregated down to the part-customer level where each **PartCustomer** record typically associates an actual part with a unique customer for that part.

However, in some cases, while historical data identifies the actual part and customer associated with a demand, it might not be necessary to generate the forecast to such a detailed level. For example, a part's forecast might be made at the aggregate level for all customers within a given sales region. Similarly, a customer's forecast might be made at the aggregate level for all parts belonging to a given product family.

To support statistical forecasts at higher levels of aggregation, the **AggregatePartCustomer** table can be used. This table is used to associate or link aggregate **PartCustomer** records with actual (or component) part and customer combinations that are also defined in the **PartCustomer** table. If a statistical forecast is being disaggregated to parts involved in an **AggregatePartCustomer** relationship, the disaggregation will only be made to the level of aggregate **PartCustomer**. This implies that fewer records would therefore be generated in the **StatisticalForecastDetail** and **StatisticalForecastDisaggregationRate** tables.

Example

For example, assume historical demand data for the following part and customer combinations.

Table 44.22: PartCustomer values

Part	Customer
Processor	AAA
Processor	BBB
Processor	CCC
Processor	YYY
Processor	ZZZ

Further, assume that instead of disaggregating the statistical forecast to the part and customer level, it should be done at the customer region level with customers "AAA", "BBB", and "CCC" associated with the "NorthWest" region, and customers "YYY" and "ZZZ" associated with the "SouthEast" region. To model this situation, the following values should be added to the **Customer** table.

Table 44.23: Customer values

Id
NorthWest
SouthEast

Next, appropriate **PartCustomer** records should be added to identify the new regional levels at which the statistical forecast for "Processor" should be aggregated.

Table 44.24: PartCustomer values

Part	Customer
Processor	NorthWest
Processor	SouthEast

Finally, **AggregatePartCustomer** records can be added to associate the detailed or component part-customer combinations with the aggregate part-customer combinations for which **StatisticalForecastDetail** records should be generated.

Table 44.25: AggregatePartCustomer values

Component.Part	Component.Customer	Aggregate.Part	Aggregate.Customer
Processor	AAA	Processor	NorthWest
Processor	BBB	Processor	NorthWest
Processor	CCC	Processor	NorthWest
Processor	YYY	Processor	SouthEast
Processor	ZZZ	Processor	SouthEast

Note: In order for a given **AggregatePartCustomer** value to be used in calculating forecast disaggregation rates, it must reference an **AggregatePartCustomerType** record that has a **ProcessingRule** setting of "Use". For more information, see "AggregatePartCustomerType table" on page 670.

Override calculated disaggregation rates

If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "Actuals," the calculated disaggregation rates can be manually overridden at the part-customer level. Calculated disaggregation rates can be manually overridden at the part-customer level. For example, override rates might be specified in cases where there is not enough historical data available to calculate the disaggregation rate for a particular part-customer, or where history does not reflect the desired forecast distribution.

The **ForecastDisaggregationOverride** table allows for manually providing disaggregation rates for part-customer and statistical forecast category combinations (as defined through the **Header** reference). The **Quantity** field is used to specify the new disaggregation rate and the **Date** field indicates the date on which the disaggregation rate becomes effective. The **Quantity** value is then considered effective from the specified **Date** until the next **DisaggregationCalendar** marker (at which point another rate override can be provided or else the calculated disaggregation rate will be used).

Note that the **Quantity** specified on the **ForecastOverride** table overrides the bucketed historical actual quantity used to calculate the associated disaggregation rate reported in the **StatisticalForecastDisaggregationRate** table (it does not directly override the actual fractional disaggregation rates reported in this table). Therefore, overrides made against statistical disaggregation rates should be made in the context of the underlying historical quantities on which the disaggregation rates for the part-customer and statistical forecast quantity are based.

Additionally, only **ForecastDisaggregationOverride** records with **Date** values that actually fall on markers belonging to the **DisaggregationCalendar** are considered valid. For example, if using a monthly disaggregation calendar and June 1, 2015 defines the beginning of the working calendar for June, then only records with a **Date** of 06/01/2015 would be used as overrides for June. If a record was added with a **Date** of 06/10/2015, it would be ignored.

CHAPTER 45: Event Management

The Event Management analytic can be used to adjust the forecast when an event is expected to increase or decrease either of the following:

- **Demand** for one or more items
- **Unit price** of one or more items

The following are examples of such events.

- A product is new, and is being introduced into the market.
- The sales force for a product is increased.
- A targeted marketing campaign is applied to a product.
- A significant increase or decrease is made to the price of a product.
- A competitor plans to introduce a competing product.
- Patent protection ends for a product.
- A product is approaching its end of life.

Some changes, such as increased demand as a result of a promotion, might be temporary. Other changes, such as reduced demand after a competitor introduces a generic version of a drug, might affect the product for the remainder of its life. An event might have more than one phase, such as a steep increase in demand for a product during an advertising campaign, followed by a gradual decline to normal levels.

While it is possible to manually edit forecast values in individual forecast streams or create a separate forecast stream for applying adjustments to the forecast, using the Event Management analytic might be more efficient. After the parameters of an event phase are defined, the event phase can be applied to multiple forecast streams and for multiple parts and customers. If a similar event occurs in the future, it can be modeled by copying the record containing the parameters that were used for the original event, changing name and the dates, and reusing it.

The Event Management application

The RapidResponse Event Management application includes predefined resources to facilitate the creation and management of events and event phases. For more information about these resources, see the *RapidResponse Applications Guide*.

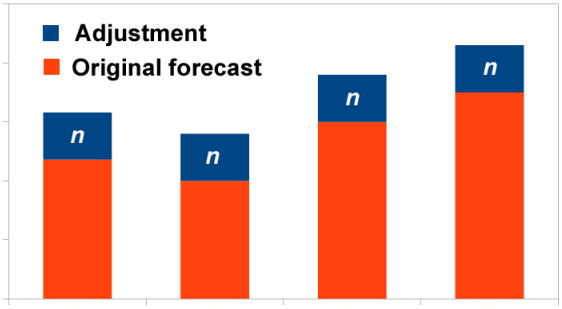
In this chapter, you will learn about:

Action types for event phases **2110**
Event Management and the RapidResponse data model — Input tables **2112**
A process for adjusting forecast values using Event Management **2114**
Using curves to adjust the forecast **2116**
Event Management and the RapidResponse data model — calculated tables **2120**
Date bucketing and disaggregation of event phase forecast adjustments **2124**
Adjusting the forecast by negative amounts **2133**

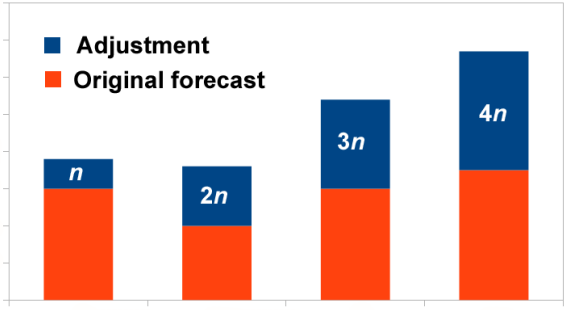
Action types for event phases

The following types action can be taken to make adjustments to forecast values:

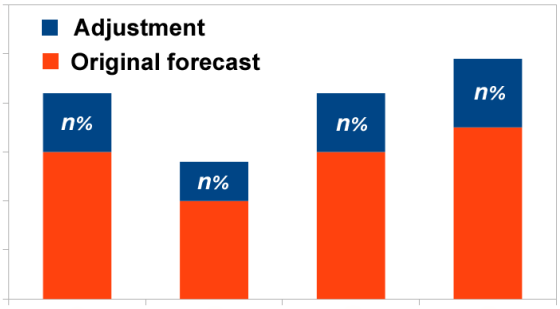
- **Quantity**—A specified value (*n*) is added to the forecast in each bucket within the effective date range.



- **CompoundedQuantity**—a specified value (*n*) is added to the forecast in the first bucket within the effective date range. Multiples of *n* are added to subsequent buckets in the range. For more information, see "Determining compounding buckets for an event phase" on page 2126.

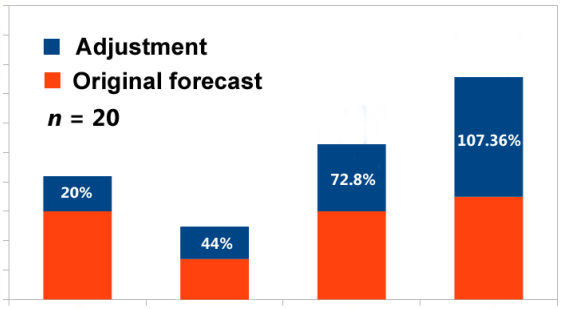


- **Percentage**—The forecast in each bucket within the effective date range is increased or decreased by a specified percentage of its original value.

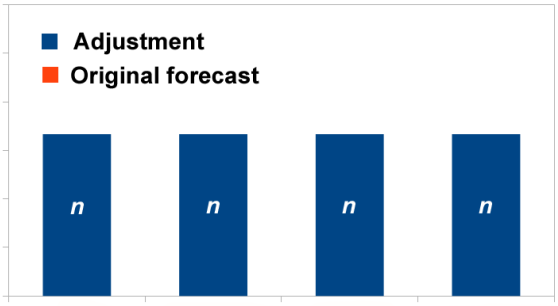


- **CompoundedPercentage**—a specified percentage (*n%*) of the original forecast value is added to the forecast in the first date bucket within the effective date range. For each compounding bucket, an additional *n%* is added to the forecast. This amount is compounded. For example, if the forecast in the first bucket is increased by 20%, the forecast for the next compounding interval is increased by 44% (an additional 20%, plus 20% of the original 20% adjustment). For more information, see "Determining compounding buckets for an event phase" on page 2126.

  CompoundedPercentage example, where *n* = 20



- **Constant**—The forecast in each bucket within the effective date range is set to a constant. The original forecast values are replaced.



- **Curve**—Points on a curve are used to determine the quantities to add to the original forecast. User-defined parameters control the shape of the curve. The types of curves are discussed in the section, "Using curves to adjust the forecast" on page 2116.

Most or all of the preceding charts show forecast adjustments that increase the forecast in each date bucket. (In the example for Constant, it is not possible to tell what the forecast was before the adjustments were applied.) If negative quantities or percentages are added to the forecast, though, the forecast is reduced.

Note: By default, forecast values cannot become negative as a result of event phase adjustments; they will not be reduced below zero. You can change this behavior for demand quantity adjustments, but not for unit price adjustments. For more information, see "Adjusting the forecast by negative amounts" on page 2133.

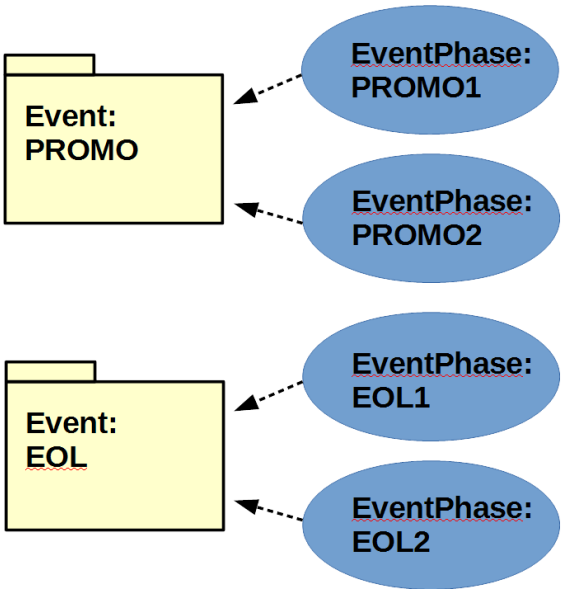
Event Management and the RapidResponse data model – Input tables

This section discusses how input tables support the Event Management analytic.

Events and event phases

Each event, represented by a record on the **Event** table, is made up of one or more event phases, represented by records on the **EventPhase** table. An **EventPhase** record contains information including when the phase begins and ends, the calendars used, and how the forecast should be adjusted during that phase.

Figure 45.1 Organization of events and event phases



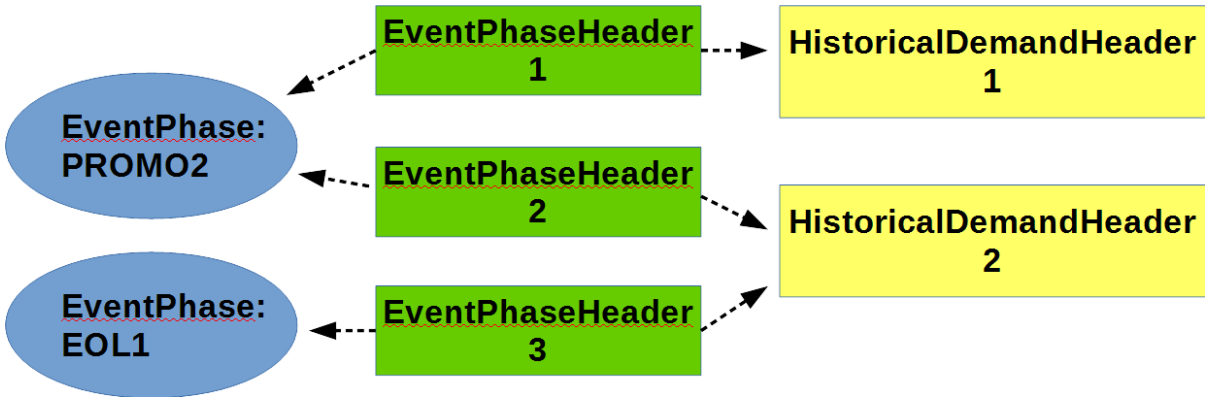
Connecting event phases to the affected forecast

The records in the **EventPhase** table do not directly specify which forecast values are affected by the adjustments. The **EventPhaseHeader** table connects **EventPhase** records with records on the **HistoricalDemandHeader** table, which identifies each unique part, customer, and historical demand

category combination for which forecast can be created. For more information about the **HistoricalDemandHeader** table, see "HistoricalDemandHeader table" on page 334.

Each **EventPhaseHeader** record references one **EventPhase** record and one **HistoricalDemandHeader** record, linking the two, as shown in the following illustration.

Figure 45.2 Connecting EventPhase records to HistoricalDemandHeader records



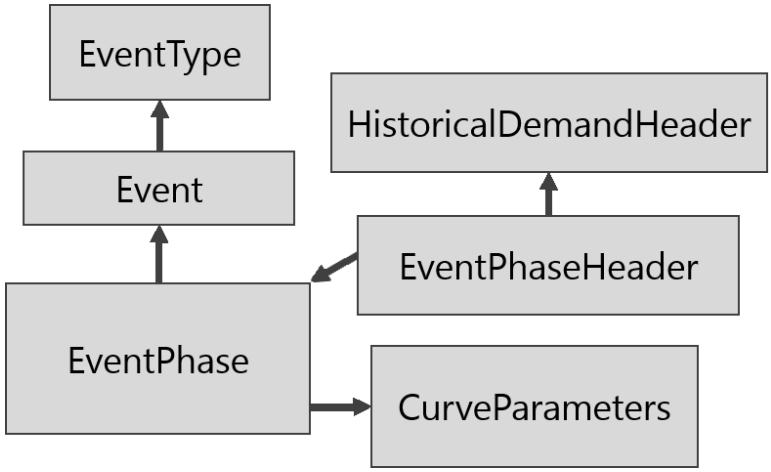
An event phase can be connected with multiple historical demand headers. Likewise, a historical demand header can be connected with multiple event phases, potentially belonging to different events.

Note: When using attribute-based planning, it might be necessary to link an event phase with the same demand header multiple times for different attributes. For information, see "Id" on page 280.

Summary of input tables used for Event Management

The following illustration shows the main input tables used for event management. The arrows represent reference fields. For example, the **Event** table has a reference to the **EventType** table.

Figure 45.3 Event Management input tables



CurveParameters—Holds parameters that define curves that can be used to adjust the forecast when the "Curve" **ActionType** is specified on the **EventPhase** table. For information about fields in this table, see "CurveParameters table" on page 263.

Event—Used to logically group event phases, which are used to apply adjustments to forecast quantities or unit prices. This table is mainly used for reference, and does not contain settings that affect analytics. For information about fields in this table, see "Event table" on page 276.

EventPhase—Holds sets of parameters that can be used to adjust (increase or decrease) forecast quantities or unit prices. This is the main table used to define the adjustments that the Event Management analytic applies to the forecast. Event phases are applied to the forecast using event phase headers. For information about fields in this table, see "EventPhase table" on page 276.

EventPhaseHeader— Used to associate event phases with historical demand headers, so that the event phases can then be used to adjust the forecast under the historical demand headers. For information about fields in this table, see "EventPhaseHeader table" on page 279.

EventType—Used to categorize events by type. For example, a trade promotion or consumer promotion. This table is for reference, and does not contain settings that affect analytics. For information about fields in this table, see "EventType table" on page 280.

HistoricalDemandHeader—Identifies each unique part, customer, and historical demand category combination for which forecast can be created. For information about fields in this table, see "HistoricalDemandHeader table" on page 334.

A process for adjusting forecast values using Event Management

The following is an outline of the general process for adjusting forecast values using event management. The order of the steps is not critical, though if records are created in the order described here, the necessary records on referenced tables will already exist for every reference field.

Create EventType

If necessary, create an **EventType** record to represent the type of event that you are modeling. This information is included in the data model to help organize event data and does not affect analytic calculations. The **EventType** table has just two input fields: **Value** and **Description**.

Create Event

If necessary, create a new **Event** record to represent the event that the event phase or phases you are about to create belongs to. The **Event** table is used to group related event phases and includes related information, including the planner code associated with the event and the date the event was created. It does not affect analytic calculations.

Create CurveParameters

If you will be using a curve to adjust the forecast, ensure that there is a record in the **CurveParameters** table describing the curve you want to use. For more information about types of curves and the settings on the **CurveParameters** table used to define them, see "Using curves to adjust the forecast" on page 2116.

Create EventPhase

Create a new record on the **EventPhase** table to define the parameters of the event.

1. Enter a **Name** for the event phase, and associate it with an **Event**.
2. Select an adjustment type in the **AdjustmentType** field. Valid options are "Quantity" or "UnitPrice."
3. Select an action type in the **ActionType** field. For a summary of valid action types, see "Action types for event phases" on page 2110.
4. Provide the values needed to determine the adjustment amounts. The relevant fields vary, depending on the action type selected.

   Table 45.1: Important EventPhase settings by ActionType

ActionType	Important settings
Quantity	Specify the adjustment quantity in the **Quantity** field.
CompoundedQuantity	Specify the adjustment quantity in the **Quantity** field. Also specify a **CompoundingCalendar** and **CompoundingIntervalCount**. For more information, see "Determining compounding buckets for an event phase" on page 2126.
Percentage	Specify the adjustment percentage in the **Percentage** field.
CompoundedPercentage	Specify the adjustment percentage in the **Percentage** field. Also specify a **CompoundingCalendar** and **CompoundingIntervalCount**. For more information, see "Determining compounding buckets for an event phase" on page 2126.
Constant	Specify the new quantity in the **Quantity** field.
Curve	Specify a set of **CurveParameters** to use.

5. Specify a **Calendar,** a **StartDate**, and an **EndDate** for the event phase. These settings are used to calculate the effective dates of the event and determine date bucketing for the phase's forecast adjustments. For more information, see "Date bucketing and disaggregation of event phase forecast adjustments" on page 2124.

Tip: If the event phase is similar enough to one that has already been created, it can be modeled by copying the record containing the parameters that were used for the original event, changing name and the dates, and reusing it.

Apply EventPhase to forecast

Create records on the **EventPhaseHeader** table to associate the event phase with all historical demand headers to which it applies. (This assumes that all **HistoricalDemandHeader** records needed to generate the forecast have already been created.) For more information, see "Connecting event phases to the affected forecast" on page 2112.

Using curves to adjust the forecast

For more complex forecast adjustments, consider using a curve.

When curves are used to adjust the forecast, the parameters that define the curve are determined using the **CurveParameters** table. The generated curve values (points) are added to the existing forecast quantities on the time markers of the calendar specified in the **EventPhase.Calendar** field.

Depending on the curve parameters, the resulting adjustment might be negative in one or more calendar periods. That is, a negative value might be added to the existing forecast.

The following types of curves are available:

- Linear curve
- Exponential curve
- Bass Diffusion Model

The type of curve is determined by the **CurveParameters.Type** field.

Adjusting the forecast using a linear curve

A linear curve is a straight line. Points on this type of curve can be calculated using the formula

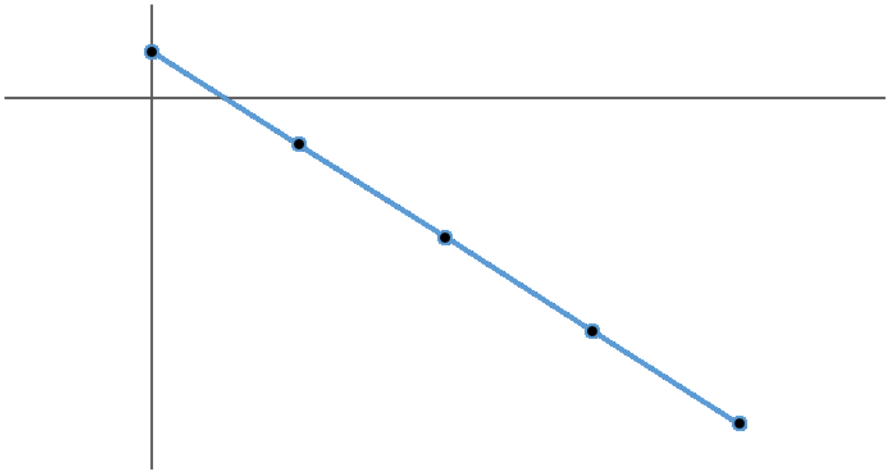
*InitialValue + LinearRate * time*

where *time* is the number of time dates on the event phase calendar since the first point on the curve.

InitialValue determines the demand or unit price adjustment at the beginning of the event phase (t=0), and **LinearRate** determines the slope of the line.

The following illustration shows a linear curve where there is a small positive adjustment to the forecast for the first time point on the curve, followed by negative adjustments for the remainder of the points.

Figure 45.4 Example of a linear curve



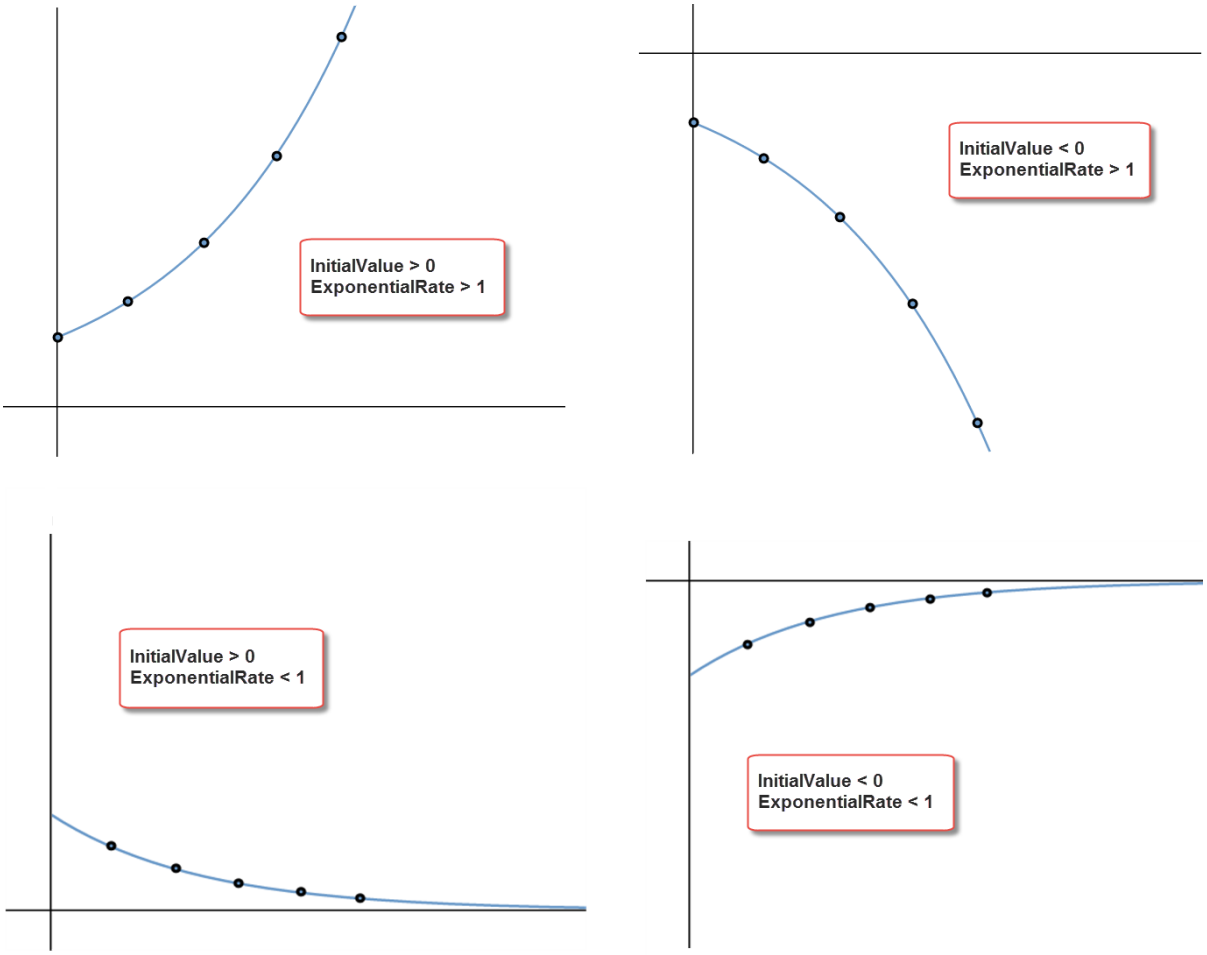
Adjusting the forecast using an exponential curve

Points on an exponential curve can be calculated using the formula

*InitialValue * ExponentialRate* ^{time}

InitialValue determines the demand or unit price adjustment at the beginning of the event phase (t=0). While **InitialValue** can be negative, **ExponentialRate** cannot be negative, since that would not produce a valid curve.

Figure 45.5 Examples of exponential curves



Modeling introduction of a new product using the Bass Diffusion Model

The Bass Diffusion Model, proposed by Frank Bass in a 1969 article in the journal *Management Science*, models the simplified behavior of consumers when a new product enters the market. It is used to predict the rate of adoption of the new product in a population of consumers.

In RapidResponse, this type of curve is mainly intended to be used to adjust demand forecast during new product introduction, where existing demand forecast values are zero. Using this type of curve to adjust unit price is not recommended.

In the Bass Diffusion Model, consumers are divided into two groups:

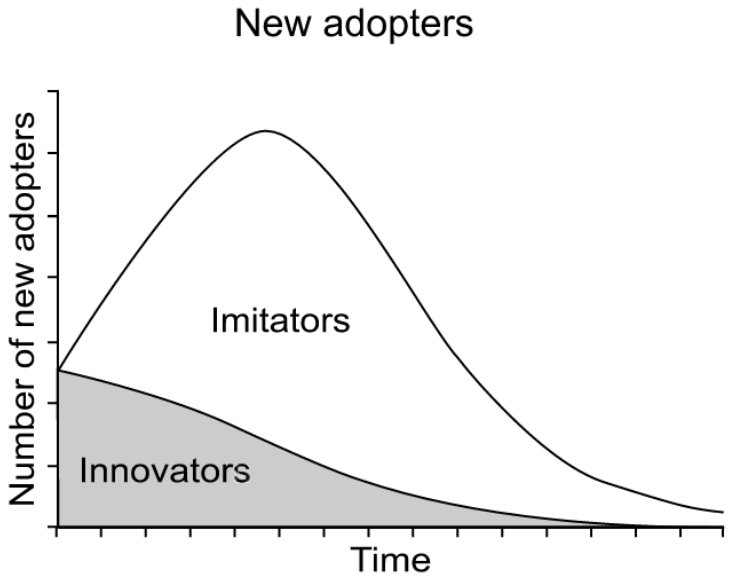
- **Innovators** buy the new product right away when it enters the market.
- **Imitators** wait for somebody else to buy a new product before they buy it.

The shape of the adoption curve varies based on three factors:

- **Innovation rate (*p*)** The proportion of innovators in a population. If a value greater than zero is not specified for the innovation rate, the curve will be flat; without innovators to buy the product first, the imitators will never buy it. The innovation rate is represented in the data model by **CurveParameters.InnovationRate**.
- **Imitation rate (*q*)** The proportion of imitators in the population. The imitation rate is represented in the data model by **CurveParameters.ImitationRate**.
- **Population size (*m*)** The number of potential consumers in the population. This can also be understood as the market size, or the maximum possible demand for the product. In the RapidResponse data model, the population size is the difference between **CurveParameters.InitialValue** and **CurveParameters.MaximumValue**. If the population size is zero, the curve is constant and flat.

The following illustration shows a typical curve generated using the Bass Diffusion Model. The shaded area under the curve represents the new adopters who are innovators, and the white area under the curve represents the new adopters who are imitators.

Figure 45.6 Bass diffusion curve



Source: https://en.wikipedia.org/wiki/Bass_diffusion_mode

The following Bass Diffusion Model formula gives the projected number of consumers who will have adopted the new product by time *t*:

$$S\left(p,q,m,t\right) = m \cdot \frac{1-e^{-(p+q)t}}{1+\frac{q}{p}\cdot e^{-(p+q)t}}$$

p = Innovation rate

q = Imitation rate

m = population size

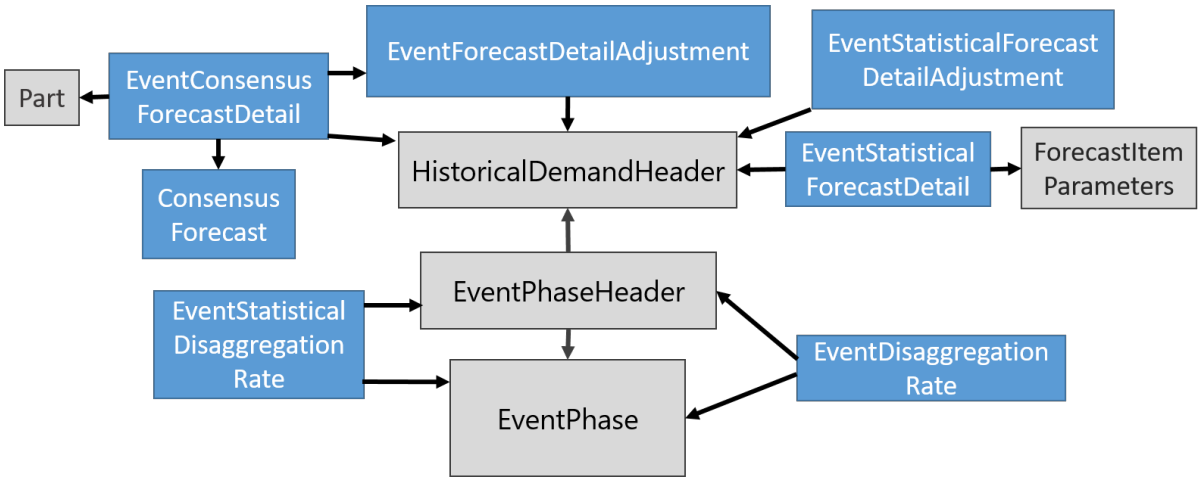
t = discrete units of time

Event Management and the RapidResponse data model – calculated tables

Previous sections in this chapter discussed setting up events and event phases and associating them with forecast records. In the following sections, the calculated tables used for Event Management and the analytics used to apply event-based adjustments to the forecast are described.

Summary of calculated tables used for Event Management

Figure 45.7 Calculated tables used for Event Management



- **EventConsensusForecastDetail—**Reports details of **EventForecastDetail Adjustment** records that are used in generating the consensus forecast value for part and customer combinations. For information about fields in this table, see "EventConsensusForecastDetail table" on page 1039.
- **EventDisaggregationRate**—Reports time-phased, disaggregated forecast adjustment rates for use in calculating forecast adjustments. For information about fields in this table, see "EventDisaggregationRate table" on page 1041.
- **EventForecastDetailAdjustment**—Reports event-based adjustments made to forecast quantity values from the **ForecastDetail** table. It can also include items that were not included in the **ForecastDetail** table, but now have positive quantities as a result of quantity adjustments. For information about fields in this table, see "EventForecastDetailAdjustment table" on page 1041.
- **EventStatisticalDisaggregationRate—**Reports time-phased, disaggregated forecast adjustment rates for use in calculating statistical forecast adjustments. This information is also included in the **EventDisaggregationRate** table. For information about fields in this table, see "EventStatisticalDisaggregationRate table" on page 1043.

- **EventStatisticalForecastDetail**—Contains statistical forecast details from the **StatisticalForecast** table against which event-based adjustments can be applied. For information about fields in this table, see "EventStatisticalForecastDetail table" on page 1044.
- **EventStatisticalForecastDetailAdjustment**—Reports event-based adjustments made to statistical forecast quantity values from the **StatisticalForecast** table. It can also include items that were not included in the **StatisticalForecast** table, but now have positive quantities as a result of quantity adjustments. For information about fields in this table, see "EventStatisticalForecastDetailAdjustment table" on page 1044.

Building the consensus forecast with event adjustments

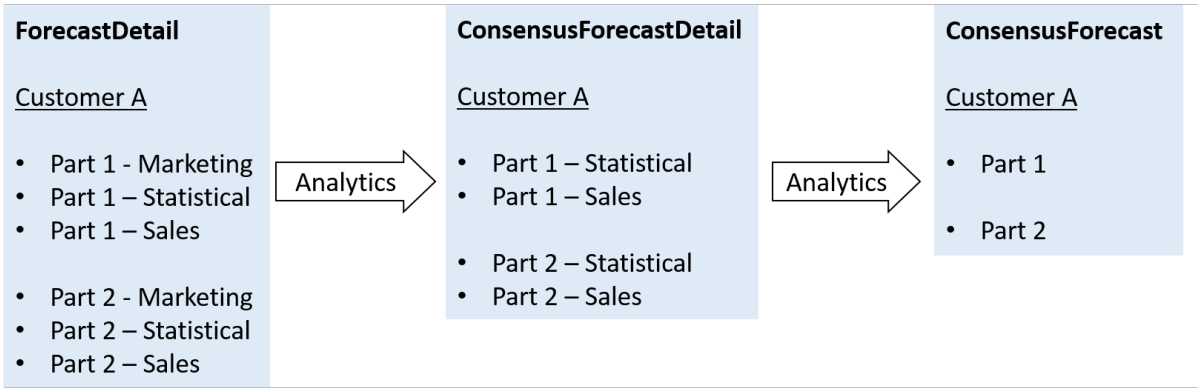
This section discusses how applying event phases to the forecast changes the flow of data and the process used to calculate the consensus forecast.

Calculating the consensus forecast without event phase adjustments

When no event phases affect the forecast, the consensus forecast is built using weighted forecast values from the **ConsensusForecastDetail** table for each customer and part combination in each date bucket. The consensus forecast is reported on the **ConsensusForecast** table.

The **ConsensusForecastDetail** table contains a subset of the data from the **ForecastDetail** table, which reports forecast data for all forecast categories. Only forecast data that affects the consensus forecast are included in the **ConsensusForecastDetail** table. In the following simplified illustration, the Statistical and Sales forecast categories are used to build the consensus forecast.

Figure 45.8 Building the consensus forecast without event phase adjustments

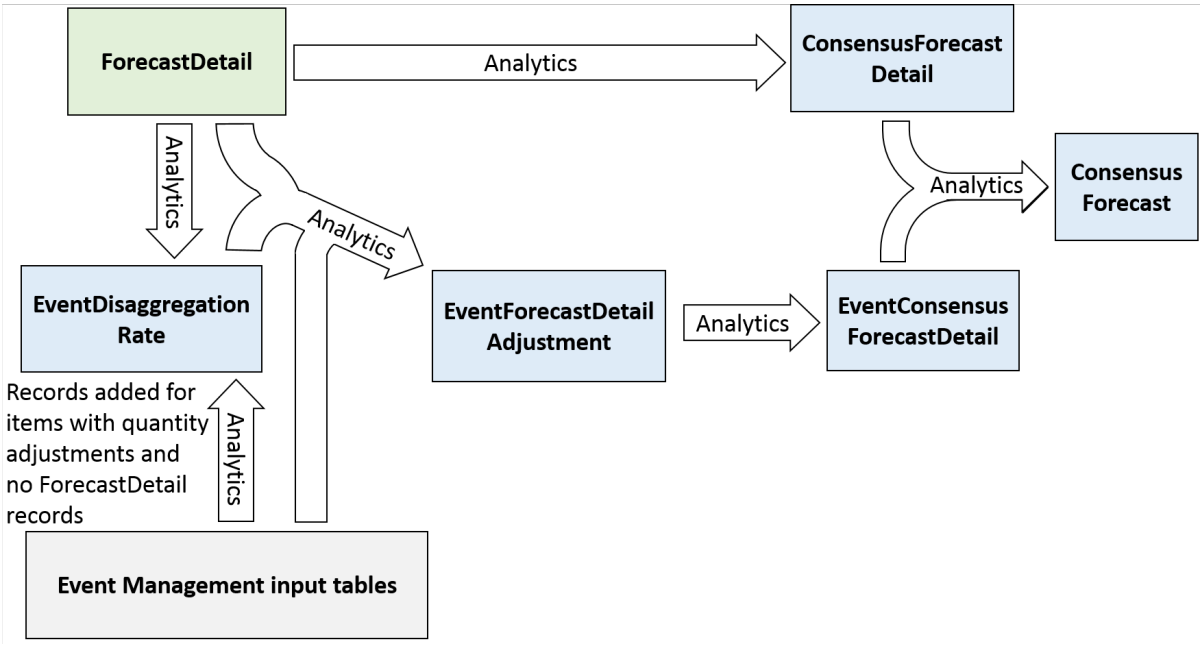


Calculating the consensus forecast with event phase adjustments

When event phases affect forecast and forecast records in categories that are used to build the consensus forecast, the flow of data is changed only for items affected by event phases.

As shown in the following illustration, **ForecastDetail** data that contribute to the consensus forecast and are not adjusted by event phases are still accounted for in the **ConsensusForecastDetail** table. **ForecastDetail** data affected by event phases and contribute to the consensus forecast are adjusted and reported in the **EventConsensusForecastDetail** table.

Figure 45.9 Building the consensus forecast when there are event phase adjustments



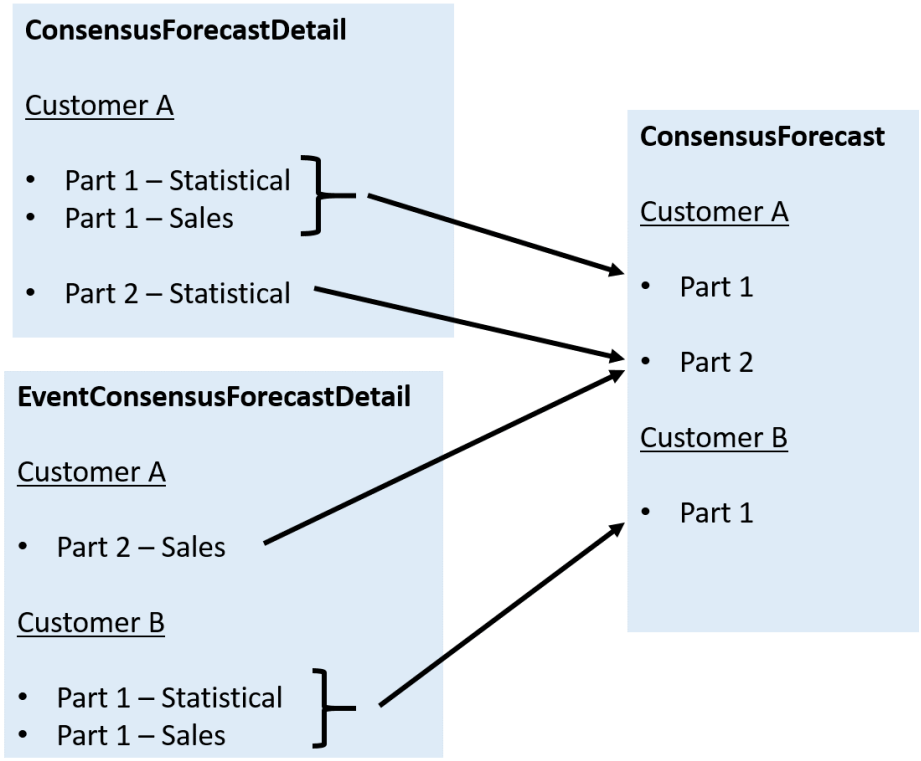
ForecastDetail records that are subject to event-based quantity adjustments are re-bucketed using the **EventPhase** calendar and used to populate the **EventDisaggregationRate** table. Records are also added to the **EventDisaggregationRate** table when there are quantity adjustments added to **HistoricalDemandHeader** records for which there are no **ForecastDetail** records.

Event phase quantity adjustments are calculated and applied to the forecast, which is bucketed using the **EventPhase** calendar. The adjusted forecast amounts are then bucketed into the effective disaggregation calendar used in the **ForecastDetail** and **ConsensusForecastDetail** tables. Unit price adjustments are also applied. (Records do not appear on the **EventDisaggregationRate** table for items that only have price adjustments). The results are listed in the **EventForecastDetailAdjustment** table.

The **EventConsensusForecastDetail** table gets its data from the **EventForecastDetailAdjustment** table, but includes only data that contribute to the consensus forecast. It is analogous to the **ConsensusForecastDetail** table.

The consensus forecast is built using weighted forecast values from the **ConsensusForecastDetail** table and the **EventConsensusForecastDetail** table, as shown in the following simplified illustration, where the Sales and Statistical forecasts contribute to the consensus forecast.

Figure 45.10 Detail records contributing to the consensus forecast



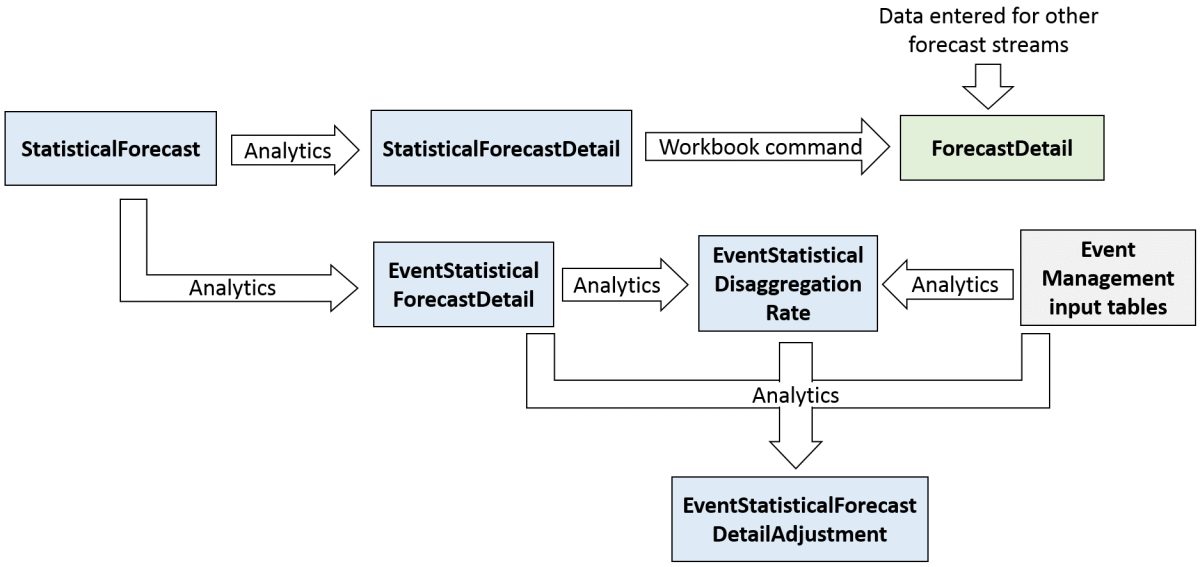
Reporting statistical forecast adjustments

Statistical forecast adjustments are reported separately from other forecast streams in the following three tables:

- EventStatisticalForecastDetail
- EventStatisticalDisaggregationRate
- EventStatisticalForecastDetailAdjustment

These tables are provided for informational purposes. They are not used to generate the consensus forecast.

Figure 45.11 Statistical forecast tables for event management



The forecast record adjustments reported in these tables are also reported in the analogous **EventConsensusForecastDetail**, **EventForecastDisaggregationRate,** and **EventForecastDetailAdjustment** tables, which also contain data about adjustments to other forecast streams.

There might be differences in the details of the event-based adjustments to the statistical forecast stream reported in the EventStatistical tables and the details reported for the same forecast in the Event tables that are calculated based on values from the **ForecastDetails** table. This can occur if values in the **StatisticalForecast** table have changed since the last time the data in the **ForecastDetail** were updated with data from the **StatisticalForecastDetail** table—an operation that is generally performed using a workbook command.

The statistical tables for event management can be used to get an idea of how event phases will affect the statistical forecast before the statistical forecast is copied into the ForecastDetails table.

Date bucketing and disaggregation of event phase forecast adjustments

An **EventPhase** record contains two references to the **Calendar** table:

- **Calendar** —The calendar used when applying adjustments for this event phase. Input parameters, such as **Quantity** and **Percent** values are specified in terms of this calendar. For example, if the **ActionType** field is set to "Quantity," the **Quantity** field is set to "200," and the **Calendar** field is set to "Week," the forecast is adjusted by 200 per week. The **FirstEffectiveDate** and the **LastEffectiveDate** are dates on this calendar.
- **CompoundingCalendar**—this calendar is used to determine compounding intervals when the **ActionType** field is set to "CompoundedQuantity" or "CompoundedPercentage." If no calendar is specified, the Everyday calendar is used. The **CompoundingIntervalCount** field determines the number of these calendar intervals in a single compounding bucket.

Determining the effective dates and date buckets for an event phase

The **FirstEffectiveDate** is the first date on which an adjustment is applied. It is the date on the calendar specified in the **Calendar** field that begins the date bucket that contains the **StartDate**. The **FirstEffectiveDate** and the **StartDate** are the same if the **StartDate** is included in the specified calendar.

The **LastEffectiveDate** is the last date on the calendar specified in the **Calendar** field that falls before the **EndDate**. The **LastEffectiveDate** cannot be the same as the **EndDate**. If the **EndDate** is a date that is included in the specified calendar, the **LastEffectiveDate** is the date on that calendar that comes before the **EndDate.**

The **FirstEffectiveDate** and the **LastEffectiveDate** can be the same. In this case, the event phase applies to a single date bucket.

FirstEffectiveDate and LastEffectiveDate: exceptions

The following situations are special cases in which the **FirstEffectiveDate** and the **LastEffectiveDate** are determined differently.

Table 45.2: FirstEffectiveDate and LastEffectiveDate exceptions

Case	Result
StartDate is set to "Past" or "Undefined"	The StartDate is adjusted to the SOP RunDate (**SOPAnalyticsConfiguration.RunDate)**.
EndDate is set to "Future" or "Undefined"	The LastEffectiveDate is adjusted to the SOP ForecastEndDate (**SOPAnalyticsConfiguration.CalcForecastEndDate**).

If, after the above adjustments have been made if applicable

- the **StartDate** or **EndDate** is still not a specific date (for example, if the **StartDate** is set to "Future") or
- the **StartDate** is after or the same as the **EndDate**

then both the **FirstEffectiveDate** and the **LastEffectiveDate** are set to "Undefined" and the event phase is ignored.

Note: The Event Management analytic does not make adjustments to the forecast before the RunDate.

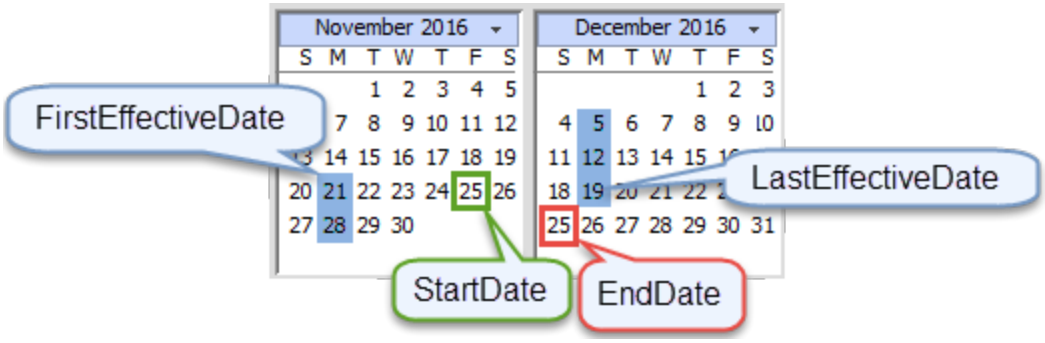
Example: determining effective dates for a quantity adjustment

The Jurassic Toy Company's holiday advertising campaign is expected to increase the demand for robot dinosaurs by 10,000 per week over the month leading up to Christmas. An event phase is created to model this change.

- **ActionType**: Quantity
- **Quantity**: 10,000
- **Calendar**: Week

- **StartDate**: November 25, 2016
- **EndDate**: December 25, 2016
- **FirstEffectiveDate** is the date on the Week calendar that begins the date bucket that contains **StartDate.** In this case, it is November 21, 2016.
- **LastEffectiveDate** is the last date on the Week calendar that falls before the **EndDate**. In this case it is December 19, 2016.

Figure 45.12 Dates for the holiday promotion event phase



EventPhaseHeader records are then created by the user to associate the event phase with the appropriate historical demand headers. As a result, the robot dinosaur demand quantity forecast is increased by 10,000 each week for the weeks of November 21, November 28, December 5, December 12, and December 19.

Determining compounding buckets for an event phase

When **EventPhase.ActionType** is set to "CompoundedPercentage" or "CompoundedQuantity," compounding buckets determine when compounding occurs. Compounding buckets can coincide with the date buckets defined by the calendar specified in the **EventPhase.Calendar** field or they can be different.

The first compounding bucket of an event phase includes the **EffectiveStartDate**. The date ranges for subsequent compounding buckets depend on two fields:

- **CompoundingCalendar** determines which calendar is used for compounding.
- **CompoundingIntervalCount** determines the number of dates on the compounding calendar per compounding bucket. By default, this value is 1.

The compounding bucket that the **EventPhase.Calendar** date falls in determines the compounding for the **EventPhase.Calendar** bucket.

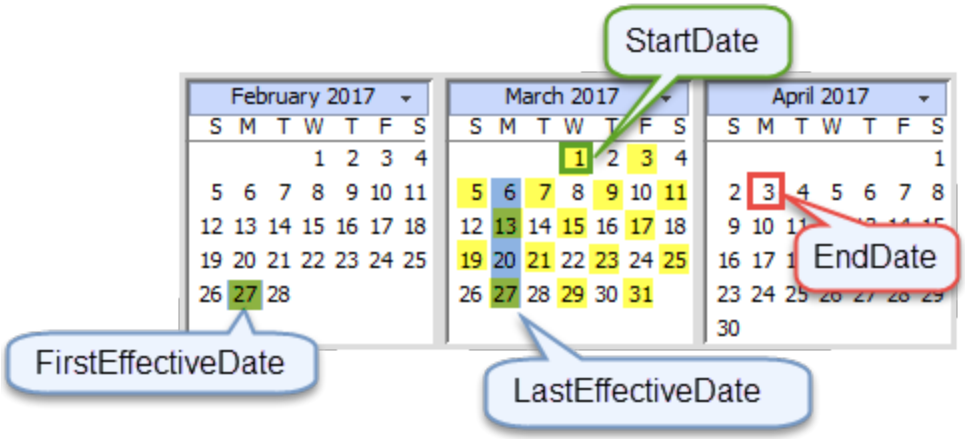
Example: determining effective dates and compounding buckets for a CompoundedQuantity adjustment

Following the Christmas toy marketing campaign, the Jurassic Toy Company is expecting a gradual increase in demand for replacement battery packs over the month of March. The following event phase is created:

- **ActionType**: CompoundedQuantity
- **Quantity**: 100
- **Calendar**: Week
- **CompoundingCalendar**: Everyday
- **CompoundingIntervalCount**: 2
- **StartDate**: March 1, 2017
- **EndDate**: April 3, 2017
- **FirstEffectiveDate** is the date on the Week calendar that begins the date bucket that contains **StartDate.** In this case, it is February 27, 2017.
- **LastEffectiveDate** is the last date on the Week calendar that falls before the **EndDate**. In this case it is March, 27, 2017.

The following illustration shows the calendar dates for the event phase. Dates on the Week calendar (Mondays) are highlighted in blue. Days that mark the beginnings of compounding buckets are highlighted in yellow. February 27, March 13, and March 27 are Mondays that also begin new compounding buckets.

Figure 45.13 Dates for battery demand increase phase



EventPhaseHeader records are then created by the user to associate the event phase with the appropriate historical demand headers. As a result, the battery pack demand forecast is adjusted as follows:

Table 45.3: Battery pack demand adjustments by week

Week of	Compounding Bucket	Forecast Quantity Adjustment(Quantity * compounding bucket number)
February 27	1	**+100** (100 * 1)
March 6	4	**+400** (100 * 4)
March 13	8	**+800** (100 * 8)
March 20	11	**+1100** (100 * 11)
March 27	15	**+1500** (100 * 15)

Event phase forecast adjustments and forecast disaggregation

The calendar specified in the **PartCustomer.EffectiveDisaggregationCalendar** field determines date buckets used for disaggregating the original forecast for a given part and customer combination. When an **EventPhaseHeader** record connects an **EventPhase** with a **HistoricalDemandHeader**, the event phase's calendar and the effective disaggregation calendar used for the original forecast can be the same or different.

Using the same calendar for event phase bucketing and forecast disaggregation will usually produce the best results. The examples given later in this section include situations in which a mismatch between these two calendars results in an inappropriate distribution of forecast adjustments.

When multiple dates on the event phase calendar fall into the same date bucket on the effective disaggregation calendar

For Quantity, CompoundedQuantity, Constant, or Curve action types, the adjustment calculated for each date on the event phase calendar is added to the **PartCustomer.EffectiveDisaggregationCalendar** date bucket that contains that date.

For more information, see "Example: effective disaggregation calendar with larger date buckets than event phase calendar" on page 2128.

For Percentage and CompoundedPercentage action types, the forecast is adjusted by average of the percentages calculated for the **EventPhase.Calendar** dates that fall within the **PartCustomer.EffectiveDisaggregationCalendar** date bucket.

When multiple dates on the effective disaggregation calendar fall into the same date bucket on the event phase calendar

For Quantity, CompoundedQuantity, Constant, or Curve action types, the adjustment quantity calculated for the event phase calendar is spread proportionately over all of the date buckets on the effective disaggregation calendar that begin in the event phase date bucket, according to the original forecast values for each. If the original forecast in a date bucket is negative, the absolute value of the negative number is used to determine spreading of the initial forecast.

For more information, see "Example: effective disaggregation calendar with smaller date buckets" on page 2130.

For Percentage and CompoundedPercentage action types, the forecast in each date bucket is adjusted **PartCustomer.EffectiveDisaggregationCalendar** by the percentage calculated for the **EventPhase.Calendar** date bucket in which it begins.

Example: effective disaggregation calendar with larger date buckets than event phase calendar

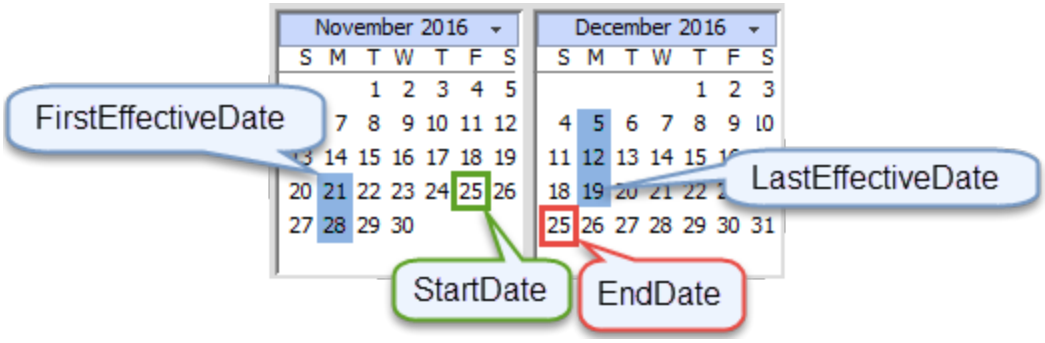
This example builds on "Example: determining effective dates for a quantity adjustment" on page 2125 and "Example: determining effective dates and compounding buckets for a CompoundedQuantity adjustment" on page 2126.

The Jurassic Toy Company has been making some changes to forecasting procedures. The **PartCustomer.EffectiveDisaggregationCalendar** field has been changed from "Week" to "Month" for the robot dinosaur and battery pack forecast records.

Effect on holiday robot dinosaur promotion event phase

For the holiday promotion, recall that the demand forecast for robot dinosaurs is increased by 10,000 per week for the weeks of November 21, November 28, December 5, December 12, and December 19.

Figure 45.14 Dates for holiday promotion



The dates on the Week calendar that begin the first two date buckets for which forecast is increased (November 21 and 28) fall in the November date bucket. Therefore, the robot dinosaur forecast for November is increased by 20,000.

The dates on the Week calendar that begin the remaining three event phase date buckets fall in the month of December, so the forecast for December is increased by 30,000.

Notice that the total amount of the forecast adjustment remains the same: 50,000.

The planner who originally created the event phase decides that distribution of the demand forecast increase is no longer a reasonable approximation of the expected outcome of the marketing campaign. The marketing campaign is expected to increase sales between November 25 and December 24. While only 20% of this period falls in the month of November, 40% of the demand forecast increase is added to the November forecast.

To correct this, the planner changes the event phase's **ActionType** to "Curve" and sets **Calendar** to "Month. In the **CurveParameters** field, the planner specifies a curve with the following parameters:

- **InitialValue**: 10,000
- **Type**: LinearCurve
- **LinearRate**: 30,000

The result is that the November forecast is adjusted by 10,000 and the December forecast is adjusted by 40,000.

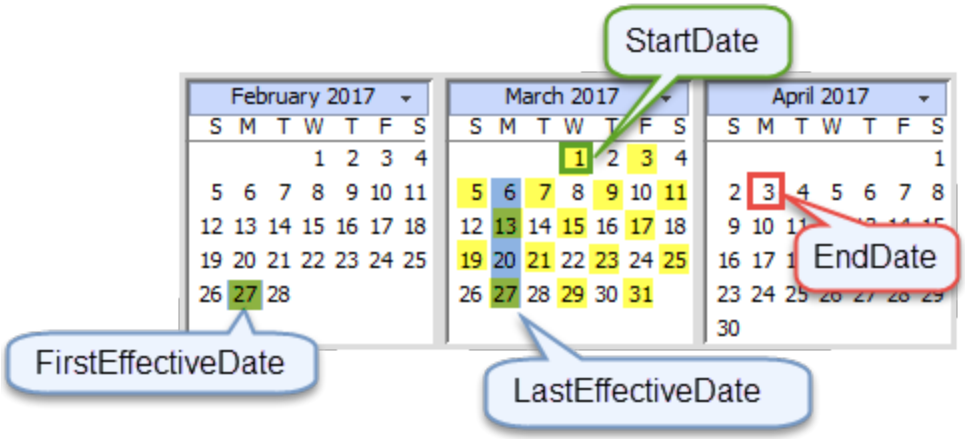
For more information about specifying curve parameters, see "Using curves to adjust the forecast" on page 2116.

Effect on battery pack demand-increase event phase

For the event phase representing an increased demand for battery packs in March, the following forecast adjustments are made:

- February 27: 100
- March 6: 400
- March 13: 800
- March 20: 1100
- March 27: 1500

Figure 45.15 Dates for battery demand increase phase



The **FirstEffectiveDate** (February 27) falls in the February date bucket. Therefore, the battery pack demand forecast for February is increased by 100.

The event phase's remaining weeks all begin in March, so the forecast for March is increased by 3800 (400 + 800 + 1100 + 1500).

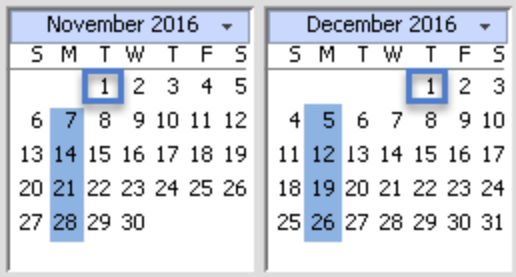
Example: effective disaggregation calendar with smaller date buckets

At the Jurassic Toy Company, the **PartCustomer.EffectiveDisaggregationCalendar** for robot dinosaur forecast records has just been changed (again) from "Month" to "Week." As described in "Effect on holiday robot dinosaur promotion event phase" on page 2129, an event phase was set up to model an increase in demand for robot dinosaurs between November 25 and December 24, 2016. The event phase uses the Month calendar and applies a linear curve to increase the demand forecast for robot dinosaurs, increasing it by 10,000 in November and 40,000 in December.

When the **EventPhase.Calendar** field and the **PartCustomer.EffectiveDisaggregationCalendar** field were both set to "Month," two month-long forecast buckets were affected and the demand was split appropriately between them. However, now that the Week calendar is the effective disaggregation calendar for the robot dinosaur demand forecast, the demand for each month is disaggregated into four Week buckets.

The following illustration shows the dates that begin Month buckets (blue boxes) and the days that begin Week buckets (blue highlights) in this period.

Figure 45.16 Month and Week buckets for November and December 2016



Four Week buckets begin in November (on November 7, 14, 21, and 28). The demand increase of 10,000 for the month of November is distributed among these four buckets. Four Week buckets begin in December (on December 5, 12, 19, and 26). The demand increase of 40,000 for the month of December is distributed among these four buckets.

The forecast increases for each month are distributed proportionately among the week buckets that begin in that month, according to the existing forecast quantities.

Table 45.4: Disaggregation of demand adjustments over the event phase

Week of	Original forecast	Forecast adjustment
November		
7	500 (10%)	+1000 (10%)
14	1000 (20%)	+2000 (20%)
15	1500 (30%)	+3000 (30%)
21	2000 (40%)	+4000 (40%)
Total	5000 (100%)	+10,000 (100%)
December		
5	2000 (20%)	+8000 (20%)
12	2000 (20%)	+8000 (20%)
19	3500 (35%)	+14,000 (35%)
26	2500 (25%)	+10,000 (25%)
Total	10000 (100%)	+40,000 (100%)

Since the marketing campaign does not begin until November 25, it does not make sense that it would increase the forecast beginning the week of November 7. At the end of the event, there is a similar problem: the demand increase attributable to the marketing campaign is expected to end on December 24th, but it continues through the week of December 26.

The planner who is responsible for this event returns the event phase's settings to their original values, using the Week calendar and increasing the forecast by a quantity of 10,000 per week, as described in "Example: determining effective dates for a quantity adjustment" on page 2125.

Spreading quantity adjustments within a date bucket

When **EventPhase.ActionType** is set to "Quantity," "CompoundedQuantity," or "Curve" the adjustment quantity for each date bucket is spread across the forecast quantities that the event phase applies to in each forecast stream. (**EventPhaseHeader** records determine which forecast quantities the event phase applies to.) Adjustments are spread proportionally, based on the existing forecast. Spreading is calculated independently for each date bucket and each forecast stream.

When the **EventPhase.ActionType** is set to "Constant," the **EventPhase.Quantity** amount is spread across the forecast quantities that it applies to in each forecast stream. Adjustments are spread proportionally, based on the existing forecast. This is different in that the end result is proportional, rather than the size of the adjustment.

Note: If the original forecast value is negative, its absolute value is used for the purpose of spreading forecast adjustments.

Example: Spreading Quantity adjustments within date buckets

An event phase with the following settings is created:

- **ActionType:** Quantity
- **Quantity:** 100
- **Calendar:** Month
- **FirstEffectiveDate:** November 1, 2016
- **LastEffectiveDate:** December 1, 2016

The event phase is applied to the marketing forecast and the statistical forecast for a product for two customers, A and B. The following tables show the forecast adjustments that are applied as a result

Table 45.5: Example - Forecast adjustments for November 1, 2016

	Customer A		Customer B		Total for both customers	
	Original	Adj.	Original	Adj.	Original	Adj.
Marketing Forecast	500	+50	500	+50	1000	+100
Statistical Forecast	300	+60	200	+40	500	+100

Table 45.6: Example - Forecast adjustments for December 1, 2016

	Customer A		Customer B		Total for both customers	
	Original	Adj.	Original	Adj.	Original	Adj.
Marketing Forecast	1200	+60	800	+40	2000	+100
Statistical Forecast	500	+50	500	+50	1000	+100

Adjusting the forecast by negative amounts

You can use Event Management to reduce forecast quantities or unit prices.

By default, forecast values cannot become negative as a result of event phase adjustments; they will not be reduced below zero. You can change this behavior for demand quantity adjustments, but not for unit price adjustments.

On the **EventForecastDetailAdjustment** and **EventStatisticalForecastDetailAdjustment** tables, the **QuantityAdjustment** field shows the calculated adjustment for each forecast record. If this adjustment does not reduce the forecast below zero or if negative results are allowed, the same quantity is reported in the **EffectiveQuantityAdjustment** field. However, if the adjustment would reduce the forecast below zero and negative results are not allowed, the adjustment amount required to reduce the forecast to exactly zero is reported in the **EffectiveQuantityAdjustment** field.

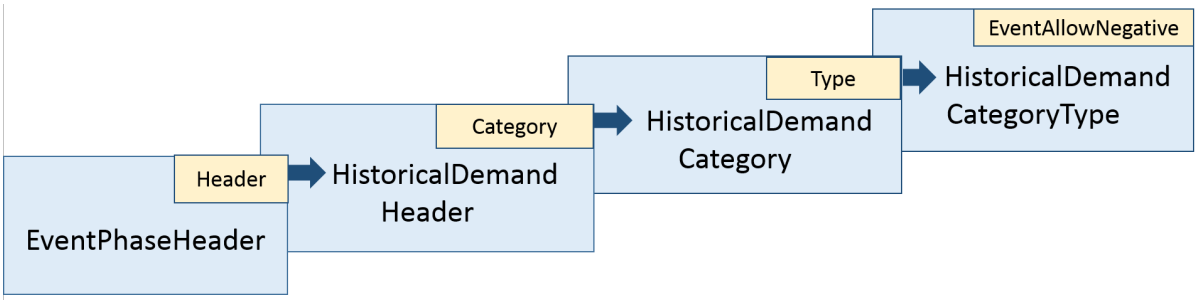
For example, a quantity adjustment of -200 is applied to an original forecast quantity of 100. Negative results are not allowed. A value of -200 is reported in the **QuantityAdjustment** field, -100 is reported in the **EffectiveQuantityAdjustment** field, and the forecast quantity is reduced to zero.

Allowing events to reduce forecast below zero

In some forecast streams, it might be appropriate to allow events to reduce the forecast below zero. For example, a forecast stream used to apply adjustments to the consensus forecast might be expected to contain negative forecast quantities.

The **HistoricalDemandCategoryType.EventAllowNegative** field controls whether event-based adjustments are allowed to make the forecast for items belonging to a type of historical demand category negative. The following illustration shows the path from the **EventPhaseHeader** table to the **HistoricalDemandCategoryType** table.

Figure 45.17 Path from EventPhaseHeader to HistoricalDemandCategoryType



If **EventAllowNegative** is set to "Yes," forecast records belonging to this type of demand category can have their forecast quantities (but not unit prices) reduced below zero by the application of event phases.

CHAPTER 46: Self-Healing Supply Chain

There are many supply chain attributes that have significant impact on inventory cost, late revenue, and other factors. The Self-Healing Supply Chain uses Machine Learning services to automatically detect and correct deviations in these supply chain attributes in the planning data. The Self-Healing Supply Chain works alongside existing RapidResponse capabilities and applications, like Aggregate Supply Planning, to drive better performance and efficiency due to the monitoring and correcting of inaccurate planning parameters.

The list of parameters considered by the Self-Healing Supply Chain service is likely to extend over time. The current list includes:

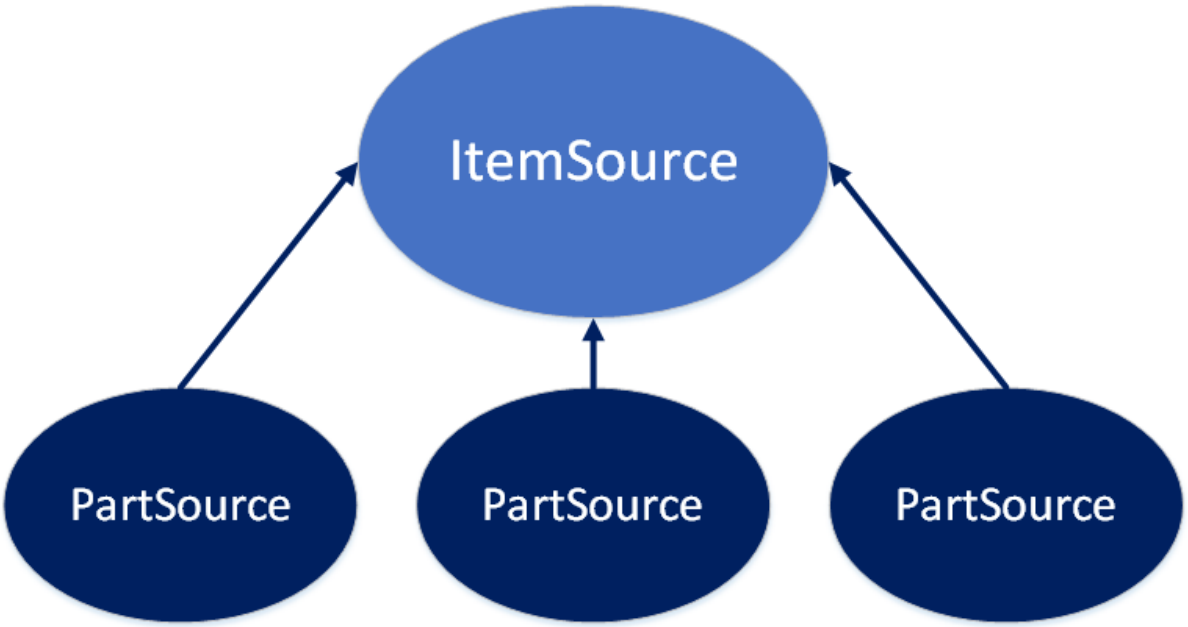
- Lead time for Buy part sources
- Yields for manufactured parts

Typically, the parameter values addressed by the Self-Healing Supply Chain apply at the part source level (defined on the **PartSource**, which references the **Source**, and **TimePhasedPartSourceLeadTime** tables). For the Self-Healing Supply Chain, lead time and yield values (such as deviations and corrections) apply at the item source level (defined on the **ItemSource** table, which references the **Part** and **Source** tables) for multiple sources of supply.

Self-Healing Supply Chain data does not directly reference a **PartSource** because ERP systems are not aware of the RapidResponse part source definitions and applicable part sources are time-phased. Thus, ItemSource records are used.

The figure below shows a set of matching **PartSource** records for a given **ItemSource**.

Figure 46.1 Matching PartSources for a given ItemSource



For more information about lead time and yield calculations, see "Lead time calculations" on page 1307 and "Yield" on page 1391.

This chapter provides an overview of the Self-Healing Supply Chain, describes the key tables and fields used to support the calculations, outlines the steps required to configure how projected parameter values are processed for **ItemSource** records, and describes details of the calculations that are performed.

Note: The Self-Healing Supply Chain application is only available if it has been enabled, and is only available with the On-Demand RapidResponse service. In these cases, the Self-Healing Supply Chain data model and calculations will be available. For more information, see "Self-Healing Supply Chain calculations" on page 2146.

In this chapter, you'll learn about:

Self-Healing Supply Chain model **2136**
Self-Healing Supply Chain data flow **2141**
Self-Healing Supply Chain calculations **2146**
Self-Healing Supply Chain configuration **2153**

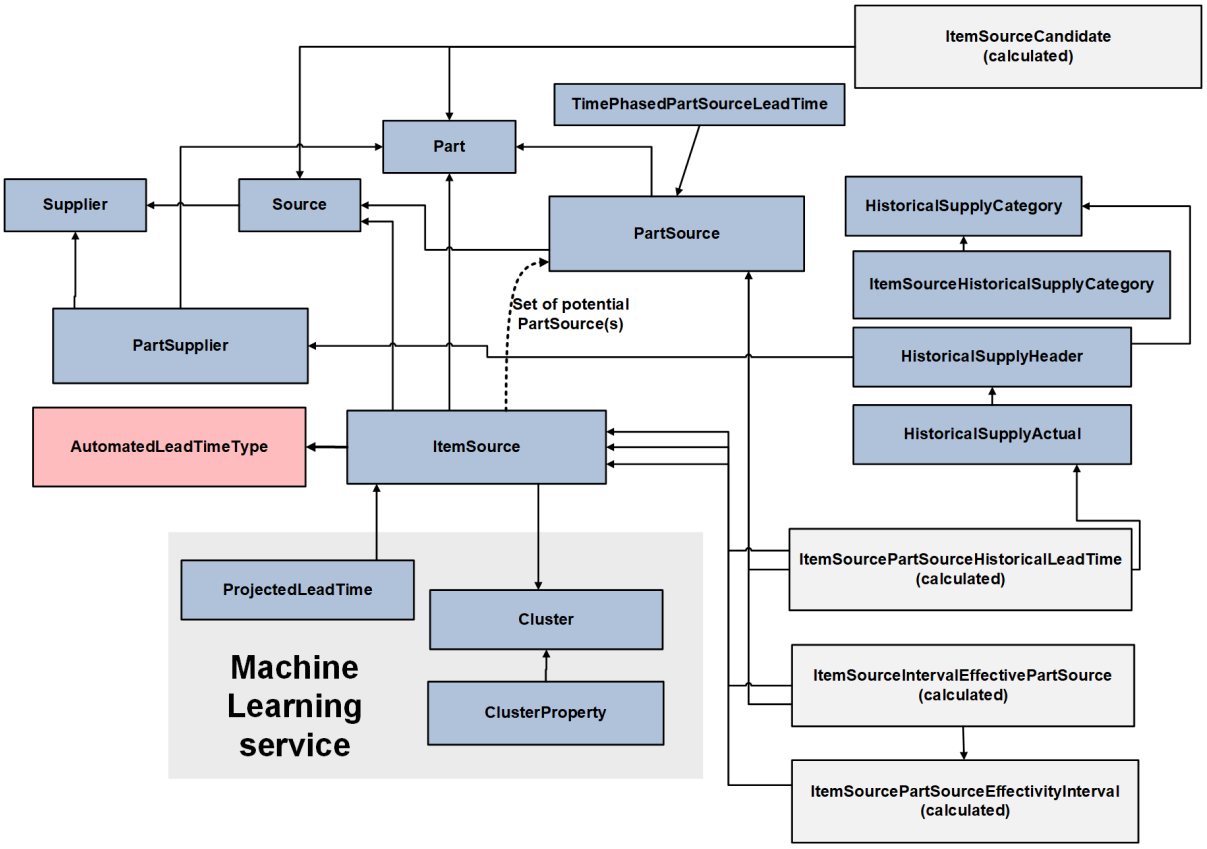
Self-Healing Supply Chain model

This section provides an overview of the key input, control, and calculated tables used in the RapidResponse data model to support the Self-Healing Supply Chain calculations. For more information, see "Self-Healing Supply Chain calculations" on page 2146.

Lead time data model

The following figure shows the tables used for the Self-Healing Supply Chain as part of Lead Time healing in RapidResponse. The tables with a thick border are new as of Version 2016.2, Service Update 21.

Figure 46.2 Tables used for the Self-Healing Supply Chain for Lead Time



The following describes the main tables shown in the figure above, and briefly outlines their usage in the Self-Healing Supply Chain for Lead Time. All tables belong in the Mfg namespace, unless noted otherwise.

Table 46.1: Self-Healing Supply Chain tables for Lead Time

Table	Description
AutomatedLeadTimeType	This control table contains the parameters used in defining how the lead times projected by the Machine Learning service are processed for **ItemSource** records of a particular type. For more information, see "AutomatedLeadTimeType table" on page 678.
Cluster	This input table contains groups of **ItemSource** records with similar lead time characteristics, as reported by the Machine Learning service. For more information, see "Cluster table" on page 243.

Table 46.1: Self-Healing Supply Chain tables for Lead Time

Table	Description
ClusterProperty	This input table contains the key properties and values for a particular **Cluster**, as reported by the Machine Learning service. For more information, see "ClusterProperty table" on page 244.
HistoricalSupplyActual	This input table contains supply data related to historical orders placed against suppliers on a given date. For more information, see "HistoricalSupplyActual table" on page 347.
HistoricalSupplyCategory	This input table identifies both different categories of historical supply series and historical supply actuals. For more information, see "HistoricalSupplyCategory table" on page 350.
HistoricalSupplyHeader	This input table identifies each unique part, supplier, and historical supply category combination. For more information, see "HistoricalSupplyHeader table" on page 351.
ItemSource	This input table defines combinations of **Part** and **Source** that are associated with historical supply. The table is populated based on data in the calculated **ItemSourceCandidate** table. For more information, see "ItemSource table" on page 379.
ItemSourceCandidate	This calculated table reports a list of **Part** and **Source** combinations associated with historical supply data that can be used by the Machine Learning service. For more information, see "ItemSourceCandidate table" on page 1087.
ItemSourceHistoricalSupplyCategory	This input table defines a set of **HistoricalSupplyCategory** records that will be excluded from analysis by the Self-Healing Supply Chain's Machine Learning service. For more information, see "ItemSourceHistoricalSupplyCategory table" on page 389.
ItemSourceIntervalEffectivePartSource	This calculated table reports the set of effective **PartSource** records for a given **ItemSourcePartSourceEffectivityInterval**. For more information, see "ItemSourceIntervalEffectivePartSource table" on page 1088.
ItemSourcePartSourceEffectivityInterval	This calculated table reports **PartSource** "effectivity intervals", which determine the intervals in which a set of effective **PartSource** records are constant for a given **ItemSource**. For more information, see "ItemSourcePartSourceEffectivityInterval table" on page 1089.
ItemSourcePartSourceHistoricalLeadTime	This calculated table matches **HistoricalSupplyActual** records for **ItemSources** with lead times from effective **PartSource** records. For more information, see "ItemSourcePartSourceHistoricalLeadTime table" on page 1089.
Part	This input table identifies a unique part and site. For more information, see "Part table " on page 415.
PartSource	This input table contains multiple sources of supply for a part. For more information, see "PartSource table" on page 459.

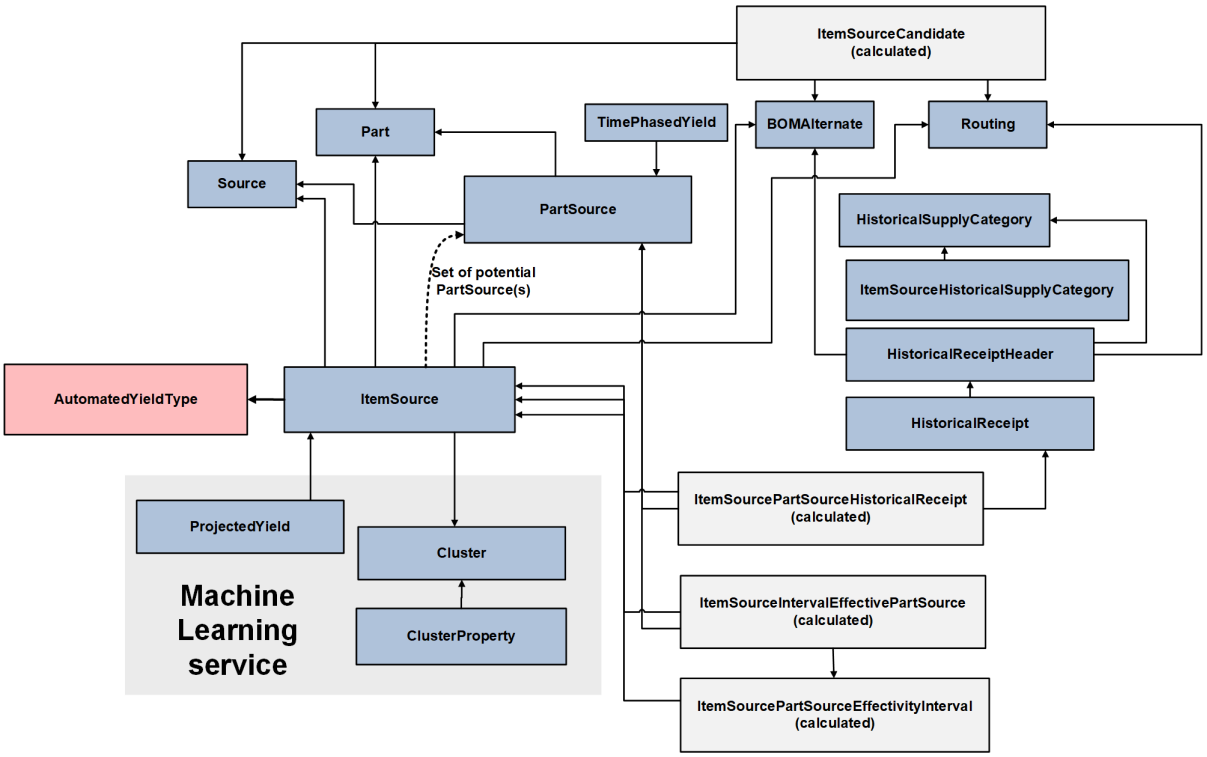
Table 46.1: Self-Healing Supply Chain tables for Lead Time

Table	Description
PartSupplier	This input table identifies each unique part and supplier combination used in historical data. For more information, see "PartSupplier table" on page 477.
ProjectedLeadTime	This input table contains lead time information projected by the Machine Learning service for a set of **ItemSource** records. The accepted lead time changes are then copied to the **TimePhasedPartSourceLeadTime** table. For more information, see "ProjectedLeadTime table" on page 505.
Source	This input table stores information about deliveries, as well as information pertaining to contact people at suppliers. For more information, see "Source table" on page 603.
Supplier	This input table is a repository for supplier information. For more information, see "Supplier table" on page 612.

Yield data model

The following figure shows the tables used for the Self-Healing Supply Chain as part of Yield healing in RapidResponse.

Figure 46.3 Tables used for the Self-Healing Supply Chain for Yield



The following describes the main tables shown in the figure above, and briefly outlines their usage in the Self-Healing Supply Chain for Yield. All tables belong in the Mfg namespace, unless noted otherwise.

Table 46.2: Self-Healing Supply Chain tables for Yield

Table	Description
AutomatedYieldType	This control table contains the parameters used in defining how the yields projected by the Machine Learning service are processed for **ItemSource** records of a particular type. For more information, see "AutomatedYieldType table" on page 683.
BOMAlternate	This input table contains alternate BOMs for configurations using alternate BOM logic. For more information, see "BOMAlternate table" on page 229.
Cluster	This input table contains groups of **ItemSource** records with similar characteristics in the attribute to be healed (e.g. lead time or yield), as reported by the Machine Learning service. For more information, see "Cluster table" on page 243.
ClusterProperty	This input table contains the key properties and values for a particular **Cluster**, as reported by the Machine Learning service. For more information, see "ClusterProperty table" on page 244.
HistoricalReceipt	This input table contains the historical supply yield data. For more information, see
HistoricalReceiptHeader	This input table identifies each unique part, supplier, BOM alternate, routing, and historical yield category combination. For more information, see "HistoricalReceiptHeader table" on page 345.
HistoricalSupplyCategory	This input table identifies both different categories of historical supply series and historical supply actuals. For more information, see "HistoricalSupplyCategory table" on page 350.
ItemSource	This input table defines combinations of **Part** and **Source** that are associated with historical yields. The table is populated based on data in the calculated **ItemSourceCandidate** table. For more information, see "ItemSource table" on page 379.
ItemSourceCandidate	This calculated table reports a list of **Part** and **Source** records with historical yield data that can be used by the Machine Learning service. For more information, see "ItemSourceCandidate table" on page 1087.
ItemSourceHistoricalSupplyCategory	This input table defines a set of **HistoricalSupplyCategory** records that will be excluded from analysis by the Self-Healing Supply Chain's Machine Learning service. For more information, see "ItemSourceHistoricalSupplyCategory table" on page 389.
ItemSourceIntervalEffectivePartSource	This calculated table reports the set of effective **PartSource** records for a given **ItemSourcePartSourceEffectivityInterval**. For more information, see "ItemSourceIntervalEffectivePartSource table" on page 1088.
ItemSourcePartSourceEffectivityInterval	This calculated table reports **PartSource** "effectivity intervals", which determine the intervals in which a set of effective **PartSource** records are constant for a given **ItemSource**. For more information, see "ItemSourcePartSourceEffectivityInterval table" on page 1089.

Table 46.2: Self-Healing Supply Chain tables for Yield

Table	Description
ItemSourcePartSourceHistoricalReceipt	This calculated table matches **HistoricalReceipt** records for **ItemSources** with yields from effective **PartSource** records. For more information, see "ItemSourcePartSourceHistoricalReceipt table" on page 1091.
Part	This input table identifies a unique part and site. For more information, see "Part table " on page 415.
PartSource	This input table contains multiple sources of supply for a part. For more information, see "PartSource table" on page 459.
ProjectedYield	This input table contains yield information projected by the Machine Learning service for a set of **ItemSource** records. The accepted yield changes are then copied to the **TimePhasedPartSourceYield** table. For more information, see "ProjectedYield table" on page 506.
Routing	This input table contains the routing codes used to group the sequence of operations used in the making of a part. For more information, see "Routing table" on page 534.
Source	This input table stores information about deliveries, as well as information pertaining to contact people at suppliers. For more information, see "Source table" on page 603.
TimePhasedYield	This input table contains time-phased yield values. For more information, see "TimePhasedYield table" on page 661.

Self-Healing Supply Chain data flow

The Self-Healing Supply Chain uses machine learning algorithms to automatically detect, measure, and correct deviations between how planning attributes were designed (as posted in the planning data for the **PartSource** records) and how they actually perform (as demonstrated in the **HistoricalSupplyActual** or **HistoricalReceipt** data for the **PartSource** records).

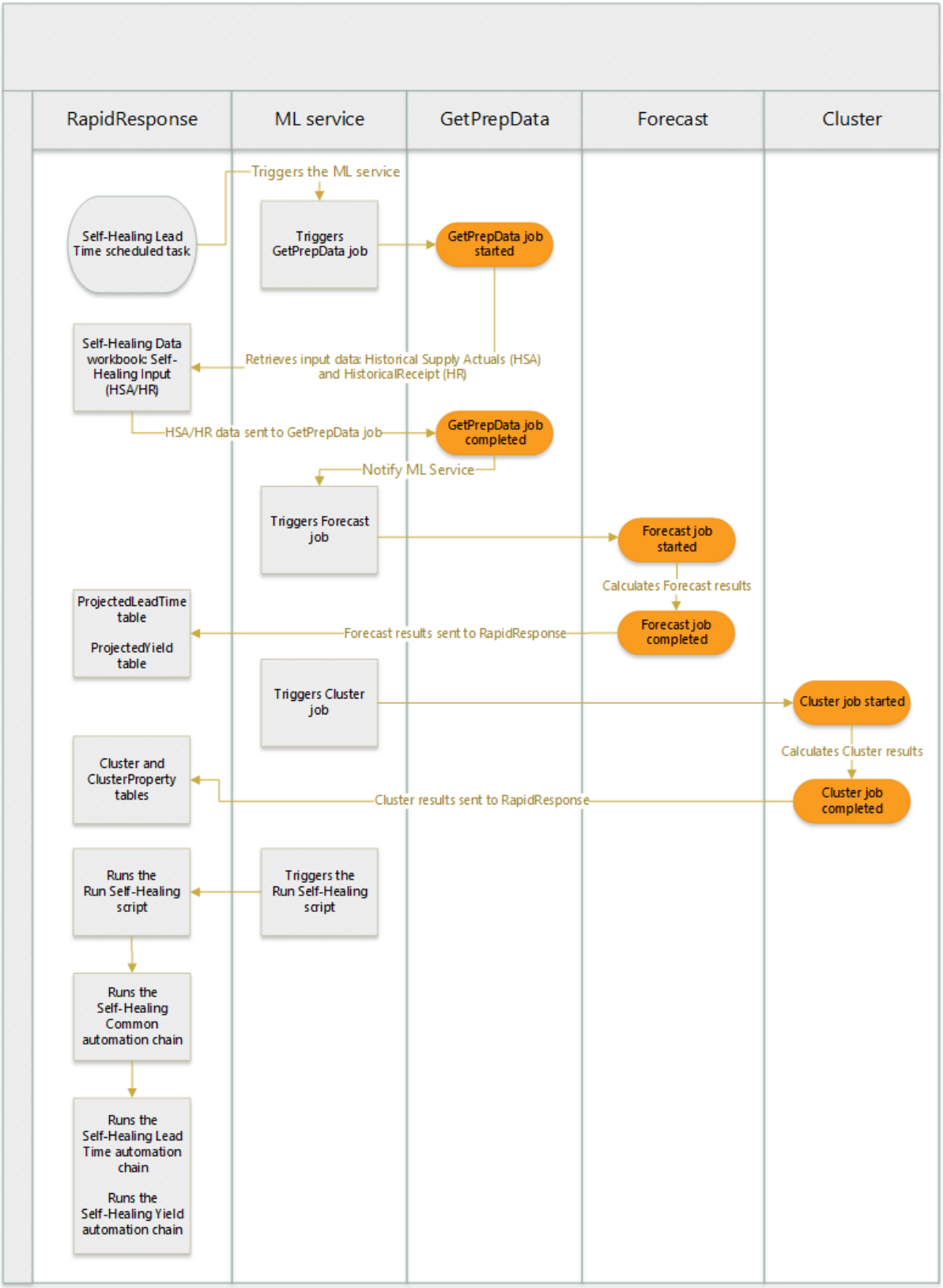
This section describes the flow of the Self-Healing Supply Chain data between RapidResponse and the Machine Learning service.

As shown in the figure below, the Machine Learning service performs the following techniques (also called jobs):

- "GetPrepData" on page 2143
- "Cluster" on page 2145
- "Forecast" on page 2146

The graphic below shows the sequence of the jobs that are part of the Machine Learning service and the flow of the Self-Healing Supply Chain data. The Machine Learning service is triggered by the Self-Healing Common scheduled task. For more information about the Self-Healing Common scheduled task, see the *RapidResponse Administration Guide*.

Figure 46.4 Self-Healing Supply Chain data flow



Note: The status of the machine learning jobs for the Self-Healing Supply Chain can be monitored with the **Job Status** worksheet in the **Self-Healing Job Status** workbook. The **Job Status** worksheet reports the status of the GetPrepData, Forecast, and Cluster jobs. For more information, see "Job table" on page 389.

The following sections describe the jobs that are part of the Machine Learning service in more detail.

GetPrepData

Collect Lead Time data

GetPrepData retrieves **HistoricalSupplyActual** (HSA) data for "Buy" part source types from the **ItemSourcePartSourceHistoricalLeadTime** table in RapidResponse.

Collect Yield data

GetPrepData retrieves **HistoricalReceipt** (HR) data from the **ItemSourcePartSourceHistoricalReceipt** table in RapidResponse.

Cleanse data

GetPrepData cleanses the historical data to remove data errors and outliers, and prepares the HSA and HR data sets in the correct format for the Cluster and Forecast jobs.

GetPrepData removes the following to cleanse the historical data:

- Records with null values for **Part**, **Part.Site**, **Supplier**, **Supplier.Site**, and **LeadTime** or **Yield**.
- Duplicate records, such as two or more records with identical values for the fields listed in the "Data requirements" on page 2144 section below.
- Records with invalid values, such as negative lead times.
- Outliers.
- Yield records above 1 (100%) or below 0 (0%).

Note: After data cleansing, if there is no data for Cluster and Forecast, GetPrepData will first send a "No Data" status and then send a "Job Complete" status.

Cleansing outliers

An outlier is a data point (or a **HistoricalSupplyActual** or **HistoricalReceipt** record) with lead times or yields that lie outside of the overall pattern in a time series (or a data set). To identify outliers in the **HistoricalSupplyActual** or **HistoricalReceipt** data set for an **ItemSource**, the interquartile range (IQR) method is used.

The IQR method identifies a data point as an outlier if it is more than *1.5 * IQR* above the third quartile or below the first quartile. That is to say, low outliers are below *Q1 - 1.5 * IQR* and high outliers are above *Q3 + 1.5 * IQR*.

The IQR method includes the following steps:

1. Calculates the median, the quartiles, and the IQR.
2. Calculates *1.5 * IQR* below the first quartile to check for low outliers.
3. Calculates *1.5 * IQR* above the third quartile to check for high outliers.

The calculations result in an upper and lower bound. Any records with lead times or yields that are below the lower bound and above the upper bound are removed from the data set.

Example

For example, assume the following historical actuals in the data:

Jan	Feb	Mar	Apr	May	Jun	Jul
5	11	12	12	12	13	25

1. Calculating the median, the quartiles, and the IQR:
   a. First quartile (also known as the 25th percentile or *Q1*) = *11.5*
   b. Second quartile (also known as the 50th percentile or *Q2*) = *12*= Median
   c. Third quartile (also known as the 75th percentile or *Q3*) = *12.5*
   d. *IQR = Q3 - Q1*

      where:

      $$\mathbf{IQR} = \mathbf{12.5 - 11.5}$$
      $$\mathbf{IQR} = \mathbf{1}$$
2. Calculating *1.5 * IQR* below the first quartile to check for low outliers:

   $$\mathbf{Q1 - 1.5 \times IQR}$$
   $$= \mathbf{11.5 - 1.5 \times 1}$$
   $$= \mathbf{11.5 - 1.5}$$
   $$= \mathbf{10}$$

   Thus, a data point is considered a low outlier if less than 10. In this case, 5 is considered a low outlier.
3. Calculating *1.5 * IQR* above the third quartile to check for low outliers:

   $$\mathbf{Q3 + 1.5 \times IQR}$$
   $$= \mathbf{12.5 + 1.5 \times 1}$$
   $$= \mathbf{12.5 + 1.5}$$
   $$= \mathbf{14}$$

   Thus, a data point is considered a high outlier if greater than 14. In this case, 25 is considered a high outlier.

Based on the IQR calculations above, there are two outliers (5 and 15) in the example data set. Therefore, those two data points will be removed.

Data requirements

The Cluster and Forecast jobs for analysis by the Machine Learning server use data provided by the **Self-Healing Input** worksheet in the **Self-Healing Data** workbook. The following fields are required by the Machine Learning service:

Input Data	Required for Lead Time	Required for Yields
Part	Y	Y
Site	Y	Y
Supplier - ID	Y	Y
Source - ID	Y	Y
Available Date	Y	Y
Actual Lead Time	Y	
Lead Time Everyday	Y	
Actual Yield		Y
ItemSource.Id (calculated)	Y	Y

For more information, see "ItemSourcePartSourceHistoricalLeadTime table" on page 1089 and "ItemSourcePartSourceHistoricalReceipt table" on page 1091.

Cluster

Cluster retrieves the data that has been prepared by GetPrepData and performs the Cluster calculation. The Cluster results are uploaded to the **Cluster** table in RapidResponse, while the top key lead time and yield attributes are uploaded to the **ClusterProperty** table for the **ItemSource**.

For example, **ClusterProperty** provides results that show the top trending lead time and yield characteristics for a specific **Cluster** and also returns a label (cluster name) for the **ItemSource**.

Note: The results for **Cluster** and **ClusterProperty** are displayed in the **Lead Time Deviation Treemap** workbook in RapidResponse, and the results for yield are displayed in the **Yield Corrections** workbook..

For clustering, the following fields should have multiple values:

- **Part.Site**
- **Supplier.ID**
- **Source.ID**
- **Supplier.Site**
- **Part.Site.Region**
- **Supplier.Site.Region**
- **Source.TransportationMode**

Having multiple values for these fields improves the labels for the clusters.

For more information, see "Cluster table" on page 243 and "ClusterProperty table" on page 244.

Forecast

Forecast retrieves the data that has been prepared by GetPrepData and performs the Forecast calculation.

The lead time Forecast results are uploaded to the **ProjectedLeadTime** table in RapidResponse for the **ItemSource**, and are processed based on the **AutomatedLeadTimeType** settings. For more information, see "AutomatedLeadTimeType table" on page 678.

The yield Forecast results are uploaded to the **ProjectedYield** table in RapidResponse for the **ItemSource**, and are processed based on the **AutomatedYieldType** settings. For more information, see "AutomatedYieldType table" on page 683.

The accepted lead time changes are copied to the **TimePhasedPartSourceLeadTime** table, either because they should be accepted automatically or because the user has accepted them manually. For more information, see "ProjectedLeadTime table" on page 505.

The accepted yield changes are copied to the **TimePhasedYield** table, either because they should be accepted automatically or because the user has accepted them manually. For more information, see "ProjectedYield table" on page 506.

Note: The results for lead time **Forecast** are displayed in the **Lead Time Healing** workbook in RapidResponse, and the results for yield are displayed in the **Yield Corrections** workbook.

Self-Healing Supply Chain calculations

Self-Healing Supply Chain logic is enabled for an **ItemSource** if it references a valid **AutomatedLeadTimeType** or a valid **AutomatedYieldType**. These fields are valid if:

- The **ProcessingRule** is set to "UseZonePolicy" or "Manual",
- And if at least one of the **InsideToleranceAction**, the **NearToleranceAction**, and the **OutsideToleranceAction** fields is set to "AutoAccept" or "Review".

For more information about the settings that are required to configure the Self-Healing Supply Chain, see "Self-Healing Supply Chain configuration" on page 2153.

With the Self-Healing Supply Chain logic enabled, the following calculations are performed by the Machine Learning service:

- "ItemSource candidacy" on page 2146
- "PartSource effectivity" on page 2147
- "Error measure calculation" on page 2150

ItemSource candidacy

For lead time healing, a **Part**, **Source**, and **HistoricalSupplyActual** combination is a candidate for **ItemSource** if:

1. There exists a **PartSource** where:
    a. **PartSource.Part** = **Part**;
    b. **PartSource.Source** = **Source**;
    c. **PartSource.FirstEffectiveDate** ≠ Undefined;
    d. **PartSource.FirstEffectiveDate** ≠ Future; and
    e. **PartSource.FirstEffectiveDate** ≤ **PartSource.LastEffectiveDate**;
2. There exists at least one matching **HistoricalSupplyActual** record.

For yield healing, a **Part**, **Source**, and **HistoricalReceipt** combination is a candidate for **ItemSource** if:

1. There exists a **PartSource** where:
    a. **PartSource.Part** = **Part**;
    b. **PartSource.Source** = **Source**;
    c. **PartSource.FirstEffectiveDate** ≠ Undefined;
    d. **PartSource.FirstEffectiveDate** ≤ **PartSource.LastEffectiveDate**;
    e. **PartSource.FirstEffectiveDate** ≠ Future; and
    f. **PartSource.BOMAlternate**= **BOMAlternate**;
    g. **PartSource.Routing**= **Routing**;
    h. **OrderPolicy.YieldUsage** ≠ ScrapPiece;
    i. **OrderPolicy.YieldUsage** ≠ Ignore; and
2. There exists at least one matching **HistoricalReceipt** record.

For more information, see "ItemSourceCandidate table" on page 1087.

PartSource effectivity

Matching PartSource and HistoricalSupplyActual or HistoricalReceipt

Although **HistoricalSupplyActual** and **HistoricalReceipt** have no reference to **PartSource**, they have a reference to **Part** and a nullable reference to **Source**. Therefore, a **PartSource** record matches a **HistoricalSupplyActual** or **HistoricalReceipt** record if and only if there exists at least one **HistoricalSupplyActual** or **HistoricalReceipt** where the following conditions are true:

Table 46.3:

HistoricalSupplyActual (Lead Time)	HistoricalReceipt (Yield)
HistoricalSupplyActual.Header.PartSupplier.Part = **PartSource.Part**	**HistoricalReceipt.Header.PartSupplier.Part** = **PartSource.Part**
PartSource.FirstEffectiveDate <= **HistoricalSupplyActual.Date** <= **PartSource.LastEffectiveDate**	**PartSource.FirstEffectiveDate** <= **HistoricalReceipt.LineReceivedDate** <= **PartSource.LastEffectiveDate**
One of the following conditions is true: • **HistoricalSupplyActual.Source** = **PartSource.Source** • **HistoricalSupplyActual.Source** = Null and **HistoricalSupplyActual.Header.PartSupplier.Supplier** = **PartSource.Source.Supplier**	One of the following conditions is true: • **HistoricalReceipt.Header.Source** = **PartSource.Source** • **HistoricalReceipt.Header.Source** = Null and **HistoricalReceipt.Header.PartSupplier.Supplier** = **PartSource.Source.Supplier**

That is to say, if **HistoricalSupplyActual.Source** or **HistoricalReceipt.Header.Source** are non-null, then they must match **PartSource.Source**. Otherwise, it will match to **Supplier**. No matching will occur when **HistoricalSupplyActual.Source** or **HistoricalReceipt.Header.Source** points to a source without a **PartSupplier**. In this case, no **ItemSourcePartSourceHistoricalLeadTime** or **ItemSourcePartSourceHistoricalReceipt**is generated for this **HistoricalSupplyActual** or **HistoricalReceipt**.

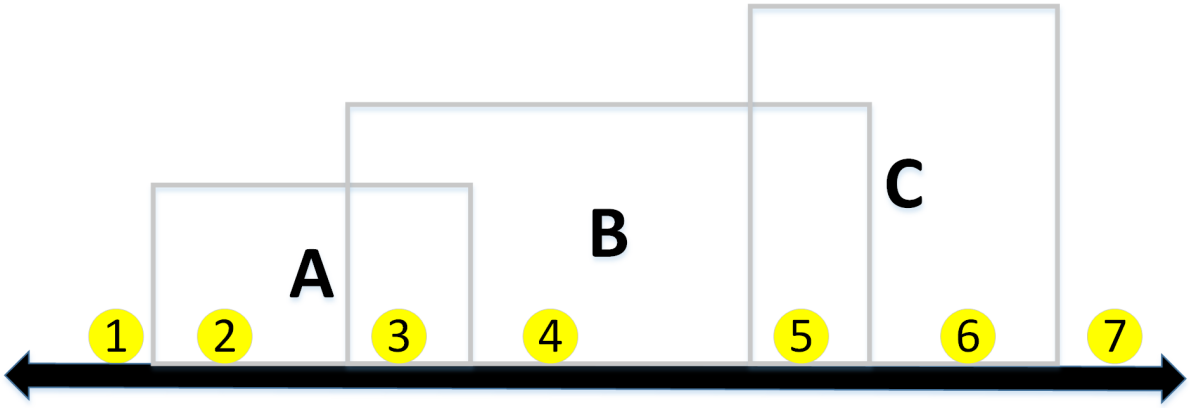
The **FirstEffectiveDate** and **LastEffectiveDate** calculated fields both take into account the **PartSourceType.EffectivityRule**.

Note: If a **Supplier** has more than one **Source**, then a **HistoricalSupplyActual** or **HistoricalReceipt**record may match multiple **PartSource** records or even multiple **ItemSource** records. In this case, an **ItemSourcePartSourceHistoricalLeadTime** or **ItemSourcePartSourceHistoricalReceipt** record is generated for each **ItemSource**/**PartSource** pair.

Example: PartSource effectivity

The figure below shows effective **PartSource** intervals for an **ItemSource** record.

Figure 46.5 Effective PartSource intervals



For a given **ItemSource** record, the **ItemSourcePartSourceEffectivityInterval** table reports **PartSource** "effectivity intervals", which are intervals in which the set of effective **PartSource** records are constant.

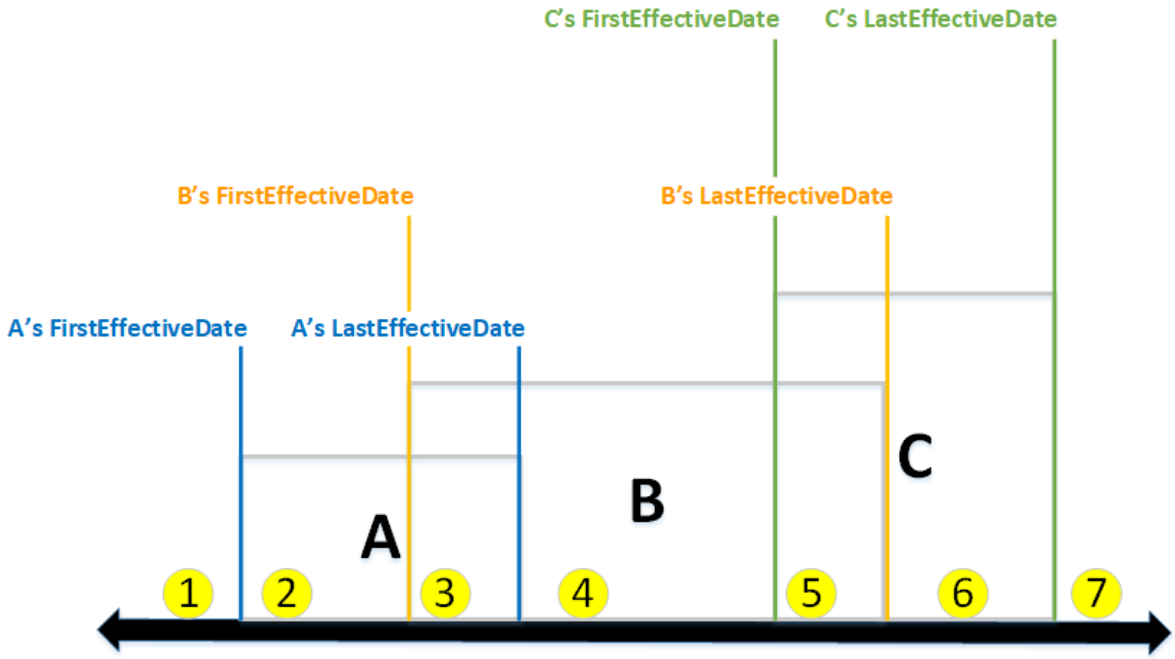
Table 46.4: ItemSourcePartSourceEffectivityInterval example

Effective interval	Effective PartSource records
1	Null
2	A
3	A and B
4	B
5	B and C
6	C
7	Null

For any given date, a subset of those **PartSource** records are effective. The effective subset changes only on dates that are some **PartSource**'s **FirstEffectiveDate** or **LastEffectiveDate**.

For more information, see "ItemSourcePartSourceEffectivityInterval table" on page 1089.

Figure 46.6 Effective PartSource records



For each interval, the **ItemSourceIntervalEffectivePartSource** table enumerates the effective **PartSource** records, as shown below.

Table 46.5: ItemSourceIntervalEffectivePartSource example

ItemSource record	Effective interval	Effective PartSource records
ItemSource 1	2	A
ItemSource 1	3	A
ItemSource 1	3	B
ItemSource 1	4	B
ItemSource 1	5	B
ItemSource 1	5	C
ItemSource 1	6	C

For the given **ItemSource**, intervals 3 and 5 are effective **PartSource** intervals, and both have multiple **PartSource** records that are effective at once.

For more information, see "ItemSourceIntervalEffectivePartSource table" on page 1088.

Multiple PartSources effective at once

PartSource has three key fields including **Part**, **Source**, and **BaseKey**. This allows a **HistoricalSupplyActual** or **HistoricalReceipt**record to match more than one effective **PartSource**, even when **HistoricalSupplyActual.Source** or **HistoricalReceipt.Header.Source**are non-null.

When multiple **PartSource** records are effective at once, all matches of **HistoricalSupplyActual** and **PartSource**, and all matches of **HistoricalReceipt** and **PartSource** are used in the **ItemSource.CalcHistoricalLeadTimeError** or **ItemSource.CalcHistoricalYieldError** calculation, respectively. In this case, rather than calculating the mean absolute deviation against a single **PartSource**, the field calculates the error between all **HistoricalSupplyActual** or **HistoricalReceipt** records related to each **PartSource** and its effective lead times or yields over the effectivity range of that **PartSource**. The **ItemSource** number (**ItemSource.CalcHistoricalLeadTimeError** or **ItemSource.CalcHistoricalYieldError**) reported would then be the average of all the errors for all the associated **PartSource** records (the sum of the deviations across all **PartSource** records, divided by the number of valid **HistoricalSupplyActual** or **HistoricalReceipt**records).

Tip: The **Data Integrity - Self-Healing Supply Chain** workbook is used to detect **PartSource** records that are effective at the same time for any **ItemSource**.

Error measure calculation

The lead time deviation (error) calculations are done in RapidResponse using **HistoricalSupplyActual** data, and the yield deviation calculations are done using **HistoricalReceipt** data.

For a given **ItemSource**:

- **ItemSourcePartSourceHistoricalLeadTime** calculates the absolute mean difference between **PartSource.FixedLeadTime** and **HistoricalSupplyActual.LeadTime**.
- **ItemSourcePartSourceHistoricalReceipt** calculates the absolute mean difference between **PartSource.Yield** and the effective yield, based on **HistoricalReceipt.ReceivedQuantity**/**HistoricalReceipt.StartQuantity**.

The deviations and projected values (corrections) are then processed based on the following **AutomatedLeadTimeType** or **AutomatedYieldType** settings:

- Tolerance threshold settings
- Tolerance action settings

Tolerance threshold settings

The tolerance threshold settings (**NearToleranceThreshold** and **OutsideToleranceThreshold**) determine which tolerance zone the deviations for an **ItemSource** belong to (whether the **ItemSource** is defined as "inside tolerance", "near tolerance", or "outside tolerance").

For example, consider the tolerance threshold settings defined in the table below. This example uses lead time, but the concepts apply equally to yield use cases.

Table 46.6: Tolerance threshold settings

Tolerance zone	Tolerance threshold	Definition
Near tolerance	5	The mean absolute deviation is in between 5 and 9 days.
Outside tolerance	10	The mean absolute deviation is 10 days or more.

Tolerance action settings

Based on the lead time tolerance zones defined by the thresholds, the tolerance action fields (**InsideToleranceAction**, **NearToleranceAction**, and **OutsideToleranceAction**) determine how to process the projected lead times for an **ItemSource** using one of the following available tolerance actions:

- **AutoAccept** —Automatically accepts the projected lead times and copies the lead time changes to the **TimePhasedPartSourceLeadTime** table.
- **Ignore** —Ignores the projected lead times.
- **Review** —Flags the projected lead times for manual review to determine whether to ignore or accept the suggestions.

The following table shows the recommended actions to take related to the tolerance zones (tolerance categories) for lead time deviation.

Table 46.7: Tolerance action settings

Tolerance zone	Tolerance action	Definition
Inside tolerance	Ignore	If the **ItemSource** has a lead time deviation that is less than 5 days, then ignore the projected lead times.
Near tolerance	AutoAccept	If the **ItemSource** has a lead time deviation that is greater than or equal to 5 days, and is less than 10 days, then auto-accept the projected lead times and copy the changes to the **TimePhasedPartSourceLeadTime** table.
Outside tolerance	Review	If the **ItemSource** has a lead time deviation that is greater than or equal to 10 days, then flag it for manual review.

The following shows examples of how the lead time deviation is calculated.

Example 1: Fixed lead time records

The three following tables show an example with fixed lead time.

If all historical supply actuals fall within the effective time window of a single part source (see "PartSource effectivity" on page 2147), lead time deviation for each supply actual is calculated against a single, fixed value from that part source.

As shown below, RapidResponse calculates the absolute mean of deviation for the lead time based on the above **NearToleranceThreshold** and **OutsideToleranceThreshold** settings to determine the lead time deviation for each **ItemSource**.

Table 46.8: ItemSource1

	Shipment 1	Shipment 2	Shipment 3	Shipment 4	Shipment 5
Fixed lead time	12	12	12	12	12
Actual lead time	8	8	8	8	8
Difference	-4	-4	-4	-4	-4
Absolute difference	4	4	4	4	4

The mean for **ItemSource1** is calculated as *20/5 = 4 days*. Based on the tolerance threshold settings defined on the **AutomatedLeadTimeType**, the lead time deviation for **ItemSource1** is categorized as "inside tolerance". As a result, the projected lead times will be ignored.

Table 46.9: ItemSource2

	Shipment 1	Shipment 2	Shipment 3	Shipment 4	Shipment 5
Fixed lead time	12	12	12	12	12
Actual lead time	6	6	20	18	18
Difference	-6	-6	8	6	6
Absolute difference	6	6	8	6	6

The mean for **ItemSource2** is calculated as *32/5 = 6.4 days*. Based on the tolerance threshold settings defined on the **AutomatedLeadTimeType**, the lead time deviation for **ItemSource2** is categorized as "near tolerance". As a result, the projected lead times will be auto-accepted.

Table 46.10: ItemSource3

	Shipment 1	Shipment 2	Shipment 3	Shipment 4	Shipment 5
Fixed lead time	12	12	12	12	12
Actual lead time	1	Null	1	1	1
Difference	-11		-11	-11	-11
Absolute difference	11		11	11	11

The mean for **ItemSource3** is calculated as *44/4 = 11 days*. Based on the tolerance threshold settings defined on the **AutomatedLeadTimeType**, the lead time deviation for **ItemSource3** is categorized as "outside tolerance". As a result, the projected lead times will require review in order to be manually auto-accepted or ignored.

Example 2: Time-phased lead time records

The following table shows an example with time-phased lead time.

If the historical supply actuals fall within the effective time windows of multiple part sources (see "PartSource effectivity" on page 2147), or if lead time for a part source varies over time, due to the use of a **TimePhasedPartSourceLeadTime**, lead time deviation for each supply actual is calculated against the appropriate lead time value for the time interval in which the supply actual falls.

As shown below, RapidResponse calculates the absolute mean of deviation for the lead time based on the above **NearToleranceThreshold** and **OutsideToleranceThreshold** settings to determine the lead time deviation for each **ItemSource**.

Table 46.11: ItemSource1

	Shipment 1	Shipment 2	Shipment 3	Shipment 4	Shipment 5	Shipment 6
Fixed lead time	10	10	15	15	10	10
Actual lead time	18	18	24	24	18	18
Difference	8	8	9	9	8	8
Absolute difference	8	8	9	9	8	8

The mean for **ItemSource1** is calculated as *50/6 = 8.3 days*. Based on the tolerance threshold settings defined on the **AutomatedLeadTimeType**, the lead time deviation for **ItemSource1** is categorized as "near tolerance". As a result, the projected lead times will be auto-accepted.

For more information about the fields on **AutomatedLeadTimeType**, see "AutomatedLeadTimeType table" on page 678.

For more information about time-phased lead time, see "Lead time elements" on page 1308.

Self-Healing Supply Chain configuration

Lead Time and Yield

AutomatedLeadTimeType and **AutomatedYieldType**settings are configured to determine how lead time and yield values projected by the Machine Learning server are processed for **ItemSource** records of a particular type.

Defining the **AutomatedLeadTimeType** and **AutomatedYieldType** involves four main steps:

1. Defining the processing rule.
2. Defining the tolerance thresholds (zone thresholds).
3. Defining the tolerance actions (zone policies).
4. Defining the historical lead time and yield error measures.

Additionally, once the **AutomatedLeadTimeType** and **AutomatedYieldType** records are defined, the **AutomatedLeadTimeType**, **AutomatedYieldType**, and the **ItemSourceHistoricalSupplyCategory** must be configured for the **ItemSource** records.

The following table outlines the main configuration steps involved in configuring the Self-Healing Supply Chain in RapidResponse.

Note: These settings are typically configured by your deployment team during the initial setup.

Table 46.12: Self-Healing Supply Chain configuration

Step	Description
Define IM configuration settings	The IM configuration settings are used to define the calendars and dates used for the planning horizons. In the **Control Tables** workbook, use the **IM Configuration** worksheet to set the following parameters: • **RunDate** = "MRPDate" • **HorizonCalendar** = "Month" • **VisibleHistoricalPeriods** = "24" • **VisibleFuturePeriod** = "12" • **InventoryQualityRatioCalendar** = "Month" **Note:** The value for the **InventoryQualityRatioCalendar** setting is not used by the Machine Learning service, but is required to successfully create a record with the values above. **Note:** If you change the **MRPDate**, it will impact the accuracy of the inventory cost and late revenue that is reported for the ItemSource records in the Self-Healing Lead Time and Self-Healing Yield Impact Analysis dashboards in RapidResponse. For more information about the **IM Configuration** settings, see "IMConfiguration table" on page 739.

Table 46.12: Self-Healing Supply Chain configuration

Step	Description
Define AutomatedLeadTime Type and AutomatedYieldType settings	The **AutomatedLeadTimeType** and **AutomatedYieldType** settings are used to define the sets of rules used to calculate the planning parameter deviation and how to process the projected parameter values. Settings in the **AutomatedLeadTimeType** table affect the processing of lead time planning parameters, and settings in the **AutomatedYieldType** table affect the processing of yield planning parameters. **Defining the processing rule** Use the **ProcessingRule** to specify how to process the projected parameter values. Valid options are: • **Ignore**—Does not run the Machine Learning service. • **Manual**—is equivalent to setting all tolerance action fields to "Review". • **UseZonePolicy**—uses the settings specified for the tolerance action fields. **Defining the tolerance thresholds (zone thresholds)** Use the **NearToleranceThreshold** and the **OutsideToleranceThreshold** settings to define the tolerance zones for the deviations. The deviations are determined by **ItemSource.CalcHistoricalLeadTimeError** and **ItemSource.CalcHistoricalYieldError**. • If the deviation is less than the **NearToleranceThreshold**, then the **ItemSource** is considered to be "inside tolerance". In this case, the **InsideToleranceAction** is used. • If the deviation is more than or equal to the **NearToleranceThreshold** but less than the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "near tolerance". • If the deviation is more than or equal to the **OutsideToleranceThreshold**, then the **ItemSource** is considered to be "outside tolerance". In this case, the **OutsideToleranceAction** is used. **Defining the tolerance actions (zone policies)** Use the **InsideToleranceAction**, the **NearToleranceAction**, and the **OutsideToleranceAction** settings to define the following actions for each of the tolerance zones: • **Ignore**—Ignores the projected parameter values. This is the recommended setting for the **InsideToleranceAction**. • **AutoAccept**—Automatically accepts the projected parameter values and copies the parameter changes to the **TimePhasedPartSourceLeadTime** or **TimePhasedYield** table, as appropriate. This is the recommended setting for the **NearToleranceAction**. • **Review**—User reviews and manually accepts the projected parameter values. This is the recommended setting for the **OutsideToleranceAction**. **Defining the historical lead time and yield error measures** Use the **HistoricalLeadTimeErrorMeasure** to specify the error measure to report in **ItemSource.CalcHistoricalLeadTimeError**, and use the **HistoricalYieldErrorMeasure** to specify the error measure to report in **ItemSource.CalcHistoricalYieldError**. These fields determine the units of the **NearToleranceThreshold** and **OutsideToleranceThreshold** fields. Valid options are: • **MeanAbsoluteDeviation** —Determines the absolute value of variance between the historical actual values and the projected forecast values, expressed as **LeadTimeUnit**.

Table 46.12: Self-Healing Supply Chain configuration

Step	Description
	• **MeanAbsolutePercentageError** —Determines the absolute value of variance between the historical actual values and the projected forecast values, expressed as a percentage. **Creating an Automated Lead Time Type record and an Automated Yield Type record** In the **Control Tables** workbook, the **Automated Lead Time Type** worksheet allows you to insert, modify, or delete records in the **AutomatedLeadTimeType** table, and the **Automated Yield Type** worksheet allows you to insert, modify, or delete records in the **AutomatedYieldType** table. To create a new **AutomatedLeadTimeType**, do the following: 1. Open the **Control Tables** workbook. 2. Select the **Automated Lead Time Type** worksheet. 3. Ensure that your private scenario is selected in the scenario control. 4. Click **Insert Record** on the worksheet toolbar to create a new automated lead time type. 5. In the **Create Automated Lead Time Type** form, fill in the fields as applicable. The following rules ensure the integrity of the data that is inserted in the form: • The **Value** for the Automated Lead Time type must be unique. • Both the **Near ToleranceThreshold** and the **OutsideToleranceThreshold** must be positive numbers. • The **NearToleranceThreshold** should be smaller than the **OutsideToleranceThreshold**. 6. Click **OK** to insert the record. To create a new **AutomatedYieldType**, do the following: 1. Open the **Control Tables** workbook. 2. Select the **Automated Yield Type** worksheet. 3. Ensure that your private scenario is selected in the scenario control. 4. Click **Insert Record** on the worksheet toolbar to create a new automated yield type. 5. In the **Create Automated Yield Type** form, fill in the fields as applicable. The following rules ensure the integrity of the data that is inserted in the form: • The **Value** for the Automated Yield type must be unique. • Both the **Near ToleranceThreshold** and the **OutsideToleranceThreshold** must be positive numbers. • The **NearToleranceThreshold** should be smaller than the **OutsideToleranceThreshold**. 6. Click **OK** to insert the record. **Note:** It is recommended to create at least three **AutomatedLeadTimeType** and three **AutomatedYieldType** records with different processing rules. For more information about the **AutomatedLeadTimeType** settings, see "AutomatedLeadTimeType table" on page 678.

Table 46.12: Self-Healing Supply Chain configuration

Step	Description
	For more information about the **AutomatedYieldType** settings, see "AutomatedYieldType table" on page 683.
Set the AutomatedLeadTimeType for ItemSource records	**Before the first run** Before the first run of the Machine Learning service, the **LeadTimeType** and **YieldType**control fields are used to define the default **AutomatedLeadTimeType** and **AutomatedYieldType**to create the **ItemSource** records. 1. In the **Control tables** workbook, use the **SHSC Configuration** worksheet to set the **LeadTimeType** and **YieldType**. Before running the Machine Learning service for the first time, the **Item Source** worksheet in the **Self-Healing Data** workbook will not have any records. Once you have run the Machine Learning service for the first time, the **Item Source** worksheet will be populated with **ItemSource** records. Separate item sources will be created for lead time and for yield, so for the same part and source, there should be one item source for lead time, and one itemsource for yields, and all records should have the same **AutomatedLeadTimeType** or **AutomatedYieldType**, respectively. **After the first run** After the first run of the Machine Learning service, you can change the **AutomatedLeadTimeType** and **AutomatedYieldType**from the default values and assign different **AutomatedLeadTimeType** and **AutomatedYieldType**records to **ItemSource** records. 2. In the **Self-Healing Data** workbook, use the **Item Source** worksheet to review existing **ItemSource** records and to modify the default **AutomatedLeadTimeType** or **AutomatedYieldType**, if necessary. For more information about the **ItemSource** table, see "ItemSource table" on page 379.

Table 46.12: Self-Healing Supply Chain configuration

Step	Description
Define the HistoricalSupplyActual data	By default, 24 months of **HistoricalSupplyActual** data is required as input to calculate the **ItemSourcePartSourceHistoricalLeadTime**. The Machine Learning service uses the data from **ItemSourcePartSourceHistoricalLeadTime** for 12 months of forecasting (for **ProjectedLeadTime**) and clustering (for **Cluster** and **ClusterProperty**). The **ItemSourceHistoricalSupplyCategory** table is used to define the **HistoricalSupplyCategory** records to be excluded from the historical data that is sent to the Machine Learning service. 1. In the **Self-Healing Data** workbook, use the **Item Source Historical Supply Category** worksheet to review the **HistoricalSupplyActual** data that is used by the Machine Learning service. 2. Open the **Self-Healing Data** workbook. 3. Select the **Item Source Historical Supply Category** worksheet. 4. Ensure that your private scenario is selected in the scenario control. 5. Click **Insert Record** on the worksheet toolbar to add **HistoricalSupplyCategory** records to exclude. 6. In the **Insert Record - ItemSourceHistoricalSupplyCategory** dialog box, select a Historical Supply Category. 7. Click **Save and Close** to insert the record. Any records that are added will be ignored. If there are no records in the **Item Source Historical Supply Category** worksheet, then all historical supply categories will be included and analyzed by the Machine Learning server. For more information about the **ItemSourceHistoricalSupplyCategory** table, see "ItemSourceHistoricalSupplyCategory table" on page 389.
Define the HistoricalReceipt data	By default, 156 weeks (36 months) of **HistoricalReceipt** data is required as input to calculate the **ItemSourcePartSourceHistoricalReceipt**. The Machine Learning service uses the data from **ItemSourcePartSourceHistoricalReceipt** for 26 months of forecasting (for **ProjectedYield**) and clustering (for **Cluster** and **ClusterProperty**). The **ItemSourceHistoricalSupplyCategory** table is used to define the **HistoricalSupplyCategory** records to be excluded from the historical data that is sent to the Machine Learning service. 1. In the **Self-Healing Data** workbook, use the **Item Source Historical Supply Category** worksheet to review the **HistoricalReceipt** data that is used by the Machine Learning service. 2. Open the **Self-Healing Data** workbook. 3. Select the **Item Source Historical Supply Category** worksheet. 4. Ensure that your private scenario is selected in the scenario control. 5. Click **Insert Record** on the worksheet toolbar to add **HistoricalSupplyCategory** records to exclude. 6. In the **Insert Record - ItemSourceHistoricalSupplyCategory** dialog box, select a Historical Supply Category. 7. Click **Save and Close** to insert the record. Any records that are added will be ignored. If there are no records in the **Item Source Historical Supply Category** worksheet, then all historical supply categories will be included and analyzed by the Machine Learning server. For more information about the **ItemSourceHistoricalSupplyCategory** table, see "ItemSourceHistoricalSupplyCategory table" on page 389.

CHAPTER 47: Project management calculations

RapidResponse supports project management capabilities allowing for the planning and analysis of projects along with their associated tasks and resources. Simulations can be run to measure how changes to the project plan impact project start and finish dates, task durations, resource costs, revenue recognition, and so on. For example, the impact of authorizing overtime hours can be seen both from a cost perspective and in terms of expediting project completion.

Because RapidResponse utilizes a single unified data model, the interdependency between project management and supply chain operations can also be seen. Projects can be created that utilize materials whose production was also modeled in RapidResponse. This means, for example, the impact that a delay in the availability of a required supply order has on the project plan can be instantly seen with the project finish date automatically recalculated to reflect the slippage.

This chapter provides an overview of the data model tables used to support integrated project management in RapidResponse, and gives details of calculations performed by project management analytics in RapidResponse.

In this chapter, you'll learn about:

Integrated project management data model **2161**
Task scheduling **2167**
Task durations **2176**
Resource efficiency **2181**
Resource availability and exceptions **2183**
Leveling task resources **2186**
Project costs **2188**
Penalties **2191**
Bonuses **2196**

Note: By default, project management analytics in RapidResponse report each task's finish date at the end of the final day of the task. If required, this can be changed, using a global setting, to report each task's finish date at the beginning of the next day after the final day of the task. For more information, see "Configuring task finish date calculations" on page 2208.

Integrated project management data model

This section provides an overview of all tables that support integrated project management in RapidResponse. All tables described in this section belong to the ProjMgmt namespace and can be used

to hold data imported from a project management application for analysis, reporting, and simulation in RapidResponse. New projects and their associated details can also be created and managed directly in RapidResponse.

Additionally, certain project management tables contain references to tables in the Mfg namespace used for managing supply chain and manufacturing operations. This linkage allows for supply chain details to be reflected at the project level and any unplanned events responded to accordingly. For example, the impact that a change in supply availability has on project completion, or that a change in part supplier has on project cost, can be immediately seen.

Project tables

Projects are temporary task-based endeavors undertaken to achieve a specific and measurable end such as delivering a good or service. Basic details of each project are contained in the **Project** table, and additional details are stored and reported in other related tables. The following tables contain project-related data (tables are assumed to be input tables unless noted otherwise).

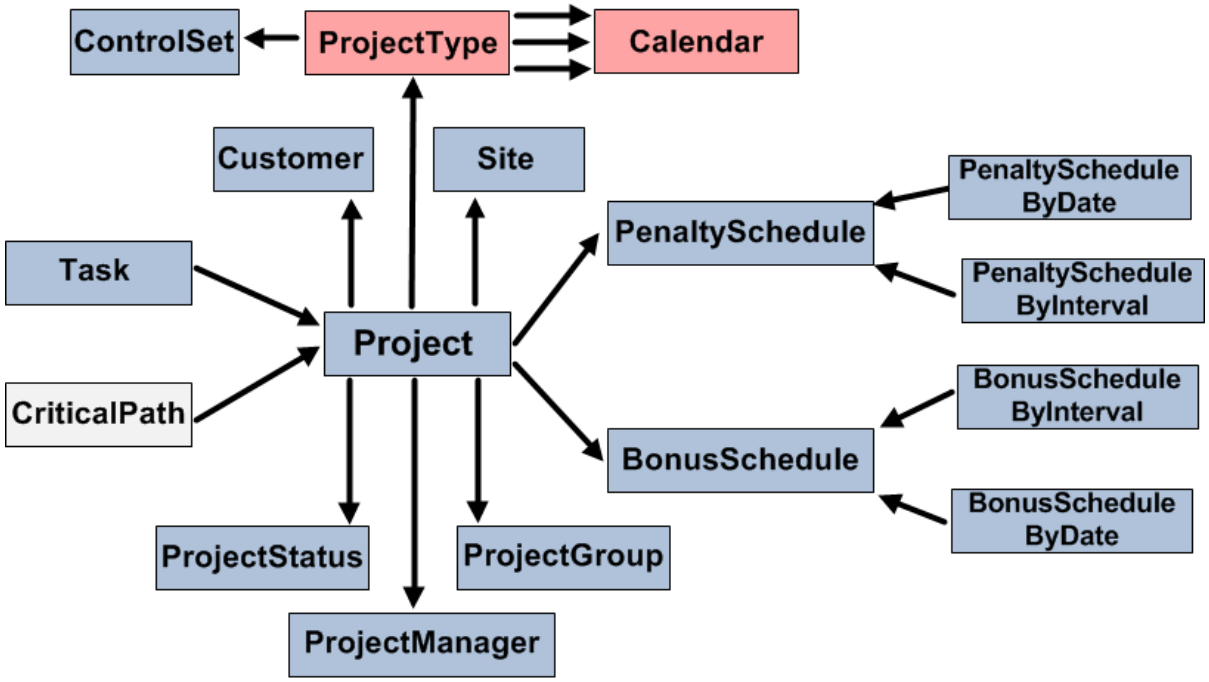
- **Project**— holds project names and other input data such as the start or finish dates to use as a basis for task scheduling within the project. This table also contains calculated fields reporting details such as project-wide material and resource costs along with realistic start and finish dates as determined by RapidResponse analytics. For more information, see "Project table" on page 500.
- **ProjectStatus**—holds values that can be used to indicate the current status of a project, typically in regards to its level of completion. For more information, see "ProjectStatus table" on page 508.
- **ProjectManager**—holds the names of all individuals that can be assigned overall responsibility for a project. For more information, see "ProjectManager table" on page 508.
- **ProjectGroup**—holds group names that can be used to categorize and group similar types of project together. For more information, see "ProjectGroup table" on page 507.
- **PenaltySchedule**—holds the names of penalty schedules that can be applied to a project to determine any charges that should be incurred if it is not complete by a specified penalty date. The details of each named penalty schedule can be defined in two related tables. For more information, see "PenaltySchedule table" on page 480.
- **PenaltyScheduleByDate**—contains time-phased records that specify one-time and recurring penalty charges associated with a named penalty schedule and potentially applicable to projects using that schedule. Any penalties are applied in the context of the project's penalty calendar, and the effectivity of records in this table is determined by specified date values. For more information, see "PenaltyScheduleByDate table" on page 481.
- **PenaltyScheduleByInterval**—contains records that specify the one-time and recurring penalty charges associated with a named penalty schedule and potentially applicable to projects using that schedule. Any penalties are applied in the context of a specified number of the project's penalty calendar intervals, and the effectivity of records in this table is determined by specified calendar intervals. For more information, see "PenaltyScheduleByInterval table" on page 482.
- **BonusSchedule**—holds the names of bonus schedules that can be applied to a project to determine any bonuses that should be applied to it if it is complete by a specified bonus date. The details of each named bonus schedule can be defined in two related tables. For more information, see "BonusSchedule table" on page 230.
- **BonusScheduleByDate**— contains date-effective records that specify one-time and recurring bonuses associated with a named bonus schedule and potentially applicable to projects using that schedule. Any bonuses are applied in the context of he project's bonus calendar, and the

effectivity of records in this table is determined by date. For more information, see "BonusScheduleByDate table" on page 231.

- **BonusScheduleByInterval**—holds interval-based records, expressed in terms of bonus calendar interval, used to define the one-time and recurring bonus revenue applicable to a specified bonus schedule. For more information, see "BonusScheduleByInterval table" on page 232.
- **CriticalPath**—a calculated table that reports all tasks on the critical path for a project. This provides a means to identify those tasks that directly impact the project's finish date along with any related items that are gating those tasks (for example, tasks might be gated by material requirements). Also shows details of all tasks being gated by a particular item regardless of whether the gated-task is on the critical path. For more information, see "CriticalPath table" on page 984.
- **ProjectType**—a control table containing processing rules and settings that impact project-level calculations, such as whether the project should be scheduled from its start or finish date and what calendar any applicable penalty charges should be based on. For more information, see "ProjectType table" on page 850.

The following illustration shows the relationships between the project-related tables outlined in this section.

Figure 47.1 Project tables



Task tables

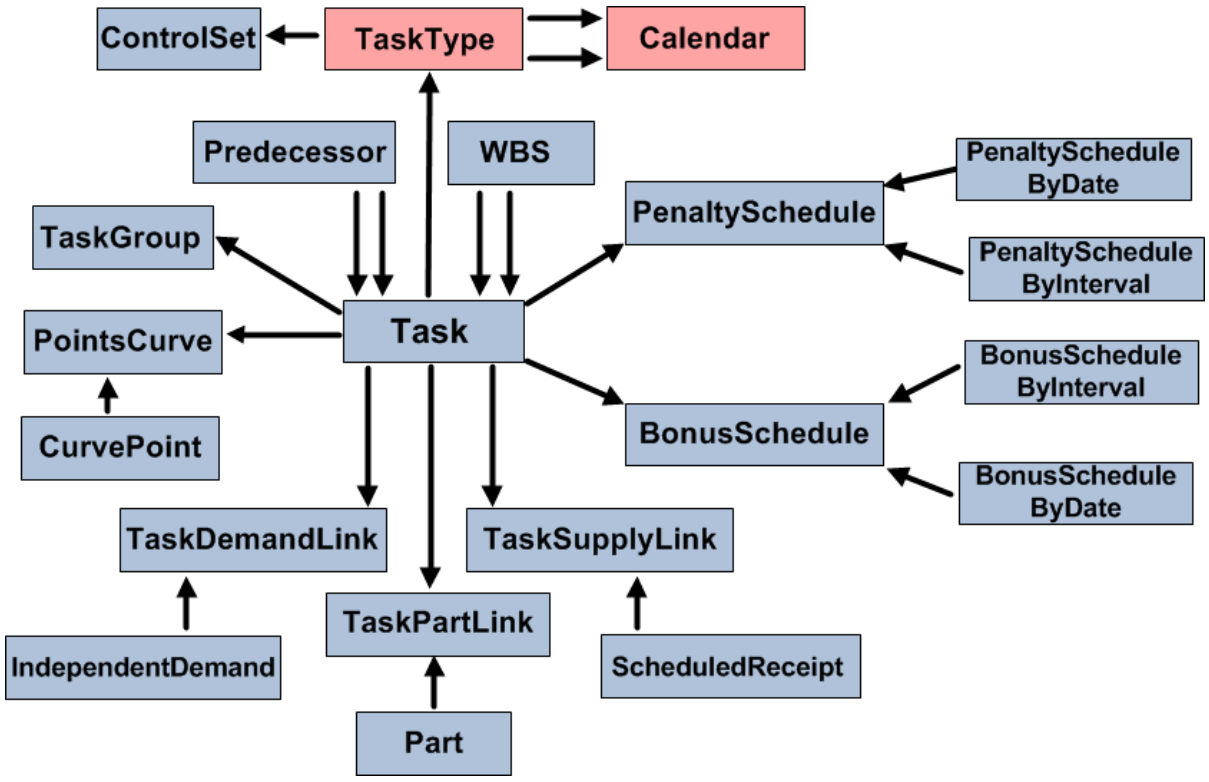
Tasks are the building blocks of projects and identify all discrete work items required towards project completion. Basic details of each task are contained in the **Task** table, and each record in this table is associated with a specific project defined in the **Project** table. Additional task data is stored or reported in other related tables. The following list outlines tables containing task-related data in RapidResponse.

- **Task**—holds task names and ids, along with other input details such as the project reference, constraint types, and the rules used in determining task durations. This table also contains calculated fields reporting details included task start date, finish date, and duration as determined by RapidResponse analytics. For more information, see "Task table" on page 624.
- **WBS**— holds work breakdown structure data identifying the hierarchical relationship between subtasks and summary tasks in the project structure. For more information, see "WBS table" on page 666.
- **Predecessor**—identifies dependencies between predecessor and successor tasks within the project structure and specifies how these dependencies impact task scheduling. For more information, see "Predecessor table" on page 488.
- **PointsCurve**—identifies named efficiency curves that can be used to define a distribution of efficiency for any assigned resources over the duration of a task. Typically, used for tasks on which resources require some period of time to ramp up before working at full efficiency. For more information, see "PointsCurve table" on page 484.
- **CurvePoint**— used to specify the actual data points that make up each efficiency curve as defined in the PointsCurve table. For more information, see "CurvePoint table" on page 265.
- **TaskPartLink**— identifies any material requirements for the task and references the applicable Part table record(s). Used in calculating certain material costs for tasks and enables the adding of task dependencies on specific demand or supply orders for the specified parts. For more information, see "TaskPartLink table" on page 648.
- **TaskSupplyLink**— defines task dependencies on specific supply orders including a reference to the applicable ScheduledReceipt table record(s). Used in calculating material costs for tasks and allows for supply availabilities, and any changes therein, to be immediately reflected in realistic task start and finish dates. For more information, see "TaskSupplyLink table" on page 650.
- **TaskDemandLink**—defines task dependencies on specific demand orders including a reference to the applicable IndependentDemand table record(s). Used in calculating material costs for tasks and allows for demand availabilities, and any changes therein, to be reflected in task start and finish dates, For more information, see "TaskDemandLink table" on page 646.
- **BaselineTaskLink**—used to store details of a task's demand or supply order dependencies whenever a project is baselined in RapidResponse. For more information, see "BaselineTaskLink table" on page 207.
- **PenaltySchedule**— holds the names of penalty schedules that can be applied to a task to determine any charges that should be incurred if it is not complete by a specified penalty date. The details of each named penalty schedule can be defined in two other tables. For more information, see "PenaltySchedule table" on page 480.
- **PenaltyScheduleByDate**—contains date-effective records that specify one-time and recurring penalty charges associated with a named penalty schedule and potentially applicable to tasks using that schedule. Any penalties are applied in the context of the project's penalty calendar intervals, and the effectivity of records in this table is determined by date. For more information, see "PenaltyScheduleByDate table" on page 481.
- **PenaltyScheduleByInterval**—contains date-effective records that specify one-time and recurring bonuses associated with a named bonus schedule and potentially applicable to tasks using that schedule. Any bonuses are applied in the context of the task's bonus calendar, and the effectivity of records in this table is determined by date. For more information, see "PenaltyScheduleByInterval table" on page 482.
- **BonusSchedule**— holds the names of bonus schedules that can be applied to a task to determine any bonuses that should be applied to it if it is complete by a specified bonus date. The details of each named bonus schedule can be defined in two related tables. For more information, see "BonusSchedule table" on page 230.

- **BonusScheduleByDate**— contains date-effective records that specify one-time and recurring bonuses associated with a named bonus schedule and potentially applicable to tasks using that schedule. Any bonuses are applied in the context of the task's bonus calendar, and the effectivity of records in this table is determined by date. For more information, see "BonusScheduleByDate table" on page 231.
- **BonusScheduleByInterval**—contains interval-based records that specify the one-time and recurring bonuses associated with a named bonus schedule and potentially applicable to tasks using that schedule. Any bonuses are applied in the context of the task's bonus calendar, and the effectivity of records in this table is determined by specified calendar intervals. For more information, see "BonusScheduleByInterval table" on page 232.
- **TaskType**— a control table containing processing rules and settings that impact task-level calculations, such as whether any resources on the task are subject to an efficiency curve and what calendar to use when calculating task durations. For more information, see "TaskType table" on page 925.

The following illustration shows the relationships between the task-related tables outlined in this section.

Figure 47.2 Task tables



Resource tables

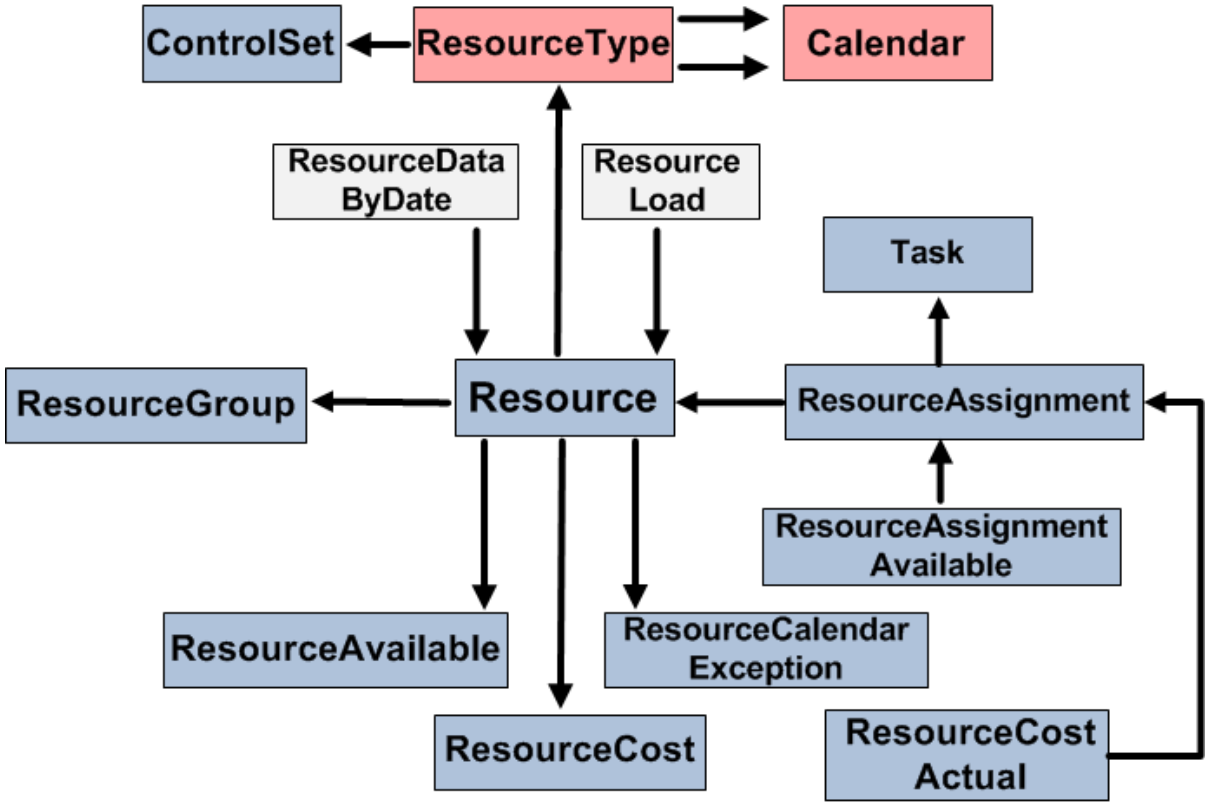
Resources are the people (or equipment) assigned to work on and complete project tasks. Basic details of each resource are defined in the **Resource** table and additional resource data, including the

assignment of resources to specific tasks, is stored or reported in other related tables. The following tables contain resource-related data in RapidResponse.

- **Resource**—holds resource names, and other input details such as work hours available per day, and the potential fixed, standard, and overtime costs associated with assigning the resource to work on a task. For more information, see "Resource table" on page 519.
- **ResourceAssignment**—used for the actual assignment of resources to specific tasks in defined units of working time. For more information, see "ResourceAssignment table" on page 522.
- **ResourceAssignmentAvailable**—allows for time-phased adjustments to specific resource assignments. For more information, see "ResourceAssignmentAvailable table" on page 525.
- **ResourceAvailable**—allows for time-phased adjustments to the default resource availabilities defined in the Resource table. For more information, see "ResourceAvailable table" on page 526.
- **ResourceCalendarException**—allows for specific day-of-week based adjustments to the effective resource availabilities from either the Resource or ResourceAvailable tables. For more information, see "ResourceCalendarException table" on page 527.
- **ResourceCosts**— allows for time-phased adjustments to the default fixed, standard, and overtime costs associated with resource usage as defined in the Resource table. For more information, see "ResourceCosts table" on page 530.
- **ResourceGroup**—holds group names that can be used to categorize and group similar types of resources together. For more information, see "ResourceGroup table" on page 531.
- **ResourceCostActual**— used to input the actual hours worked by a resource on a given date, and reports the calculated costs associated with those hours (typically used once a project has moved past the planning stage into execution for comparing actual and estimated costs). For more information, see "ResourceCostActual table" on page 528.
- **ResourceLoad**—a calculated table used to report resource load and cost details for each date on which the resource is assigned against a given task. For more information, see "ResourceLoad table" on page 1157.
- **ResourceDataByDate**—a calculated table used to report the effective resource availabilities, costs, and loads incurred within a customizable date range for a resource. For more information, see "ResourceDataByDate table" on page 1156.
- **ResourceType**—a control table containing processing rules and settings that impact resource-level calculations, such as when records are generated in the ResourceDataByDate table and what working calendar to use when assigning resources. For more information, see "ResourceType table" on page 860.

The following illustration shows the relationships between the resource-related tables outlined in this section.

Figure 47.3 Resource tables



Task scheduling

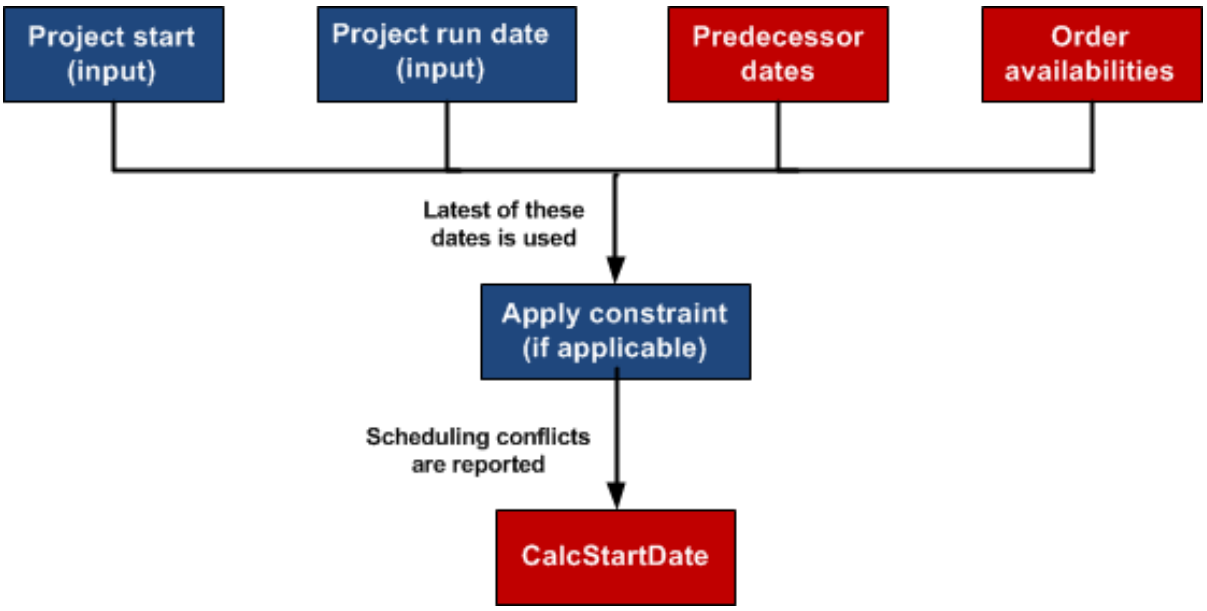
Project tasks can have their calculated start date (**Task.CalcStartDate**) and finish date (**Task.CalcFinishDate**) either automatically determined by RapidResponse based on specified parameters and processing rules, or manually set to provided input start and finish dates. To control how a task is scheduled, the **Task.SchedulingRule** field can be used. This field has two options; "Manual" and "Automatic".

If **SchedulingRule** is set to "Manual", calculated task start and finish dates are based on the input values provided in the **StartDate**, **FinishDate,** and/or **Duration** fields. This option is typically intended for use only in the early stages of a project where all task details, timing, and relationships might not be known. When using manual task scheduling, if both the **StartDate** and **FinishDate** fields are defined, then the dates provided there are subsequently reported in the **CalcStartDate** and **CalcFinishDate** fields, respectively. However, if only the **StartDate** field is defined, then it is reported in **CalcStartDate** and **CalcFinishDate** is determined by adding the value in the **Duration** field to **StartDate**.Similarly, if only the **FinishDate** field is defined, then it is reported in **CalcFinishDate** and **CalcStartDate** is determined by subtracting the value in the **Duration** field from **FinishDate.**

If **SchedulingRule** is set to "Automatic", which is the default setting in RapidResponse, then any values provided in the input **StartDate** and **FinishDate** fields are ignored. Instead, project tasks are scheduled based on a number of other parameters and rules shown in the illustration below. However, if a task in an automatically scheduled project has already started, the **Task.ActualStartDate** field can be used to

indicate when it actually started. This input date is then reported in the **CalcStartDate** field. Similarly, if a task in an automatically scheduled project has already completed, a date can be specified in **Task.ActualFinishDate** field to indicate when it actually finished.

Figure 47.4 Potential determinants of automatically calculated task start dates



The remainder of this section describes the key scheduling factors used by RapidResponse when automatically calculating task start (or finish) dates.

Project start or finish dates

As a basis for task scheduling within a project, input project start or finish dates should be provided in the **Project.StartDate** and **Project.FinishDate** fields respectively. These fields are meant to serve as default start or finish dates for the project, with their usage dependent on the **ProjectType.ScheduleRule** setting that should be defined for each project (through the **Project.Type** reference).

For projects with a **ScheduleRule** of "FromStart", the project is scheduled forward from the input start date. This means the **Project.StartDate** value provided is, by default, used as the earliest potential date on which a task in the project can be scheduled (with subsequent tasks scheduled out after that).

Similarly, for projects with a **ScheduleRule** of "FromFinish", the project is scheduled backward from the input finish date. This means the **Project.FinishDate** value provided is, by default, used as the latest potential date on which a task in the project can be scheduled to finish.

Note: Based on the scheduling of all tasks in the project, calculated project start and finish dates are also determined and reported in the **Project.CalcStartDate** and **Project.CalcFinshDate** fields.

Project run dates

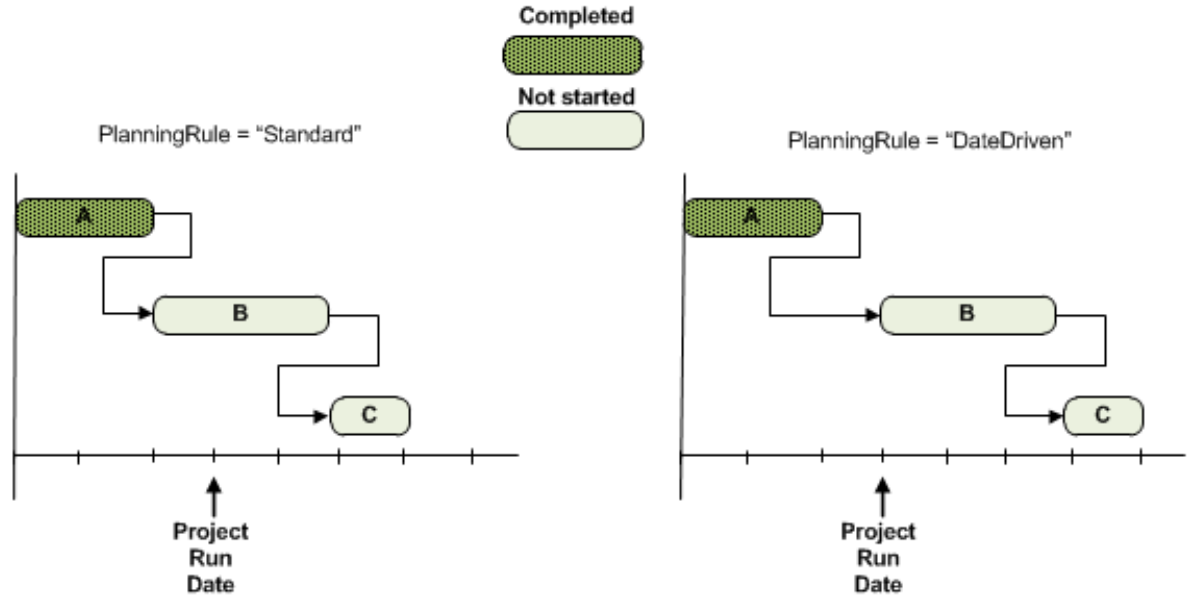
Each project can have a run date specified in the **Project.RunDate** field. This value indicates the current working or "as of date" for a project, and marks the split between past and future task activity. Depending on how a project is being planned, this might typically set to the current date or a date in the very recent past .

In run date-driven projects, the value in the **RunDate** field is used as the first date on which new task work can begin. This is meant to ensure that any tasks that have not yet started are not scheduled in the past. To set a project as date-driven, it should reference a **ProjectType.PlanningRule** value of "DateDriven" (if this field is set to "Standard", which is the default, **RunDate** is ignored during task scheduling). As appropriate, each task can then use the **Task.ActualStartDate** and **Task.ActualFinishDate** fields to indicate if and when the tasks actually started and/or finished (when these fields are populated, their values are subsequently reported in the **Task.CalcStartDate** and **Task.CalcFinishDate** fields, respectively).

The **ActualStartDate** and **ActualFinishDate** fields are used to determine the project state, as reported in **Task.CalcState**, and hence how the **RunDate** value impacts task scheduling.If both **ActualStartDate** and **ActualFinishDate** are populated then a task is considered completed and scheduling is not affected by the **RunDate** value.

However, if **ActualStartDate** is undefined, this indicates that the task has not started. In calculating when to start such a task, RapidResponse will not determine a start date any earlier than **Project.RunDate**. This is the case even if the task's predecessors, or other scheduling parameters, dictate that it could start sooner as shown in the following illustration.

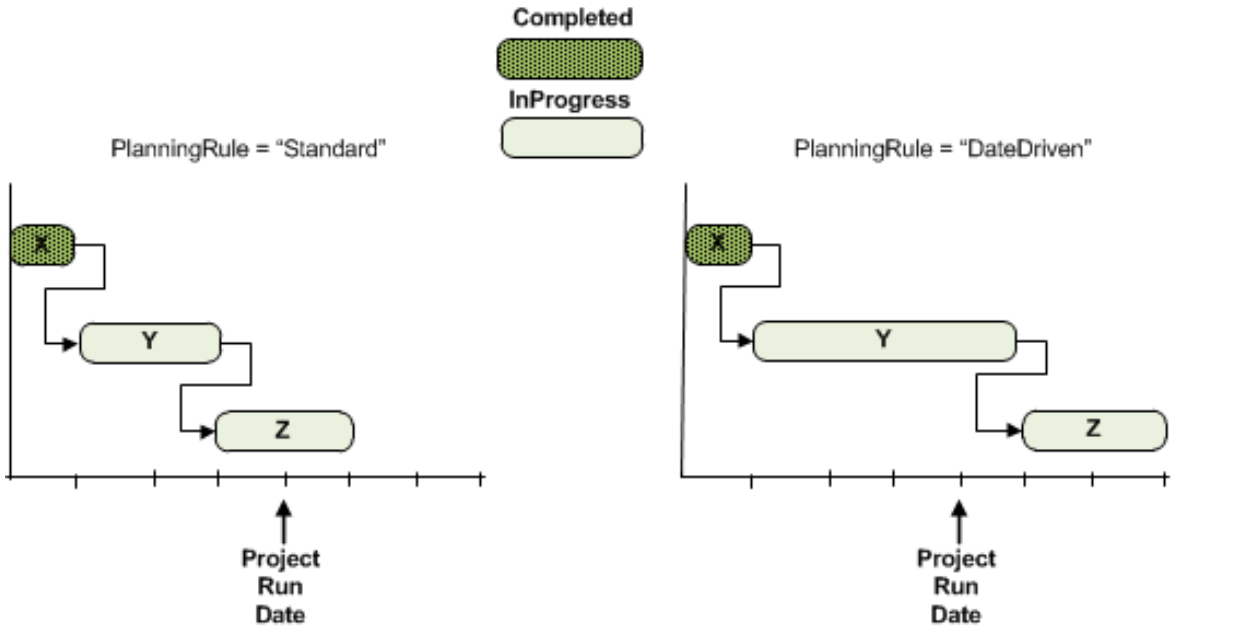
Figure 47.5 Impact of RunDate on task that hasn't started



Similarly, if the **ActualStartDate** field is populated but the **ActualFinishDate** is not, the task is considered in progress. In this case, the **Task.ExpectedFinishDate** input field can be used to specify the date when the task is expected to finish. For "FixedDuration" tasks only, the **ExpectedFinishDate** is then also reported as the task's **CalcFinishDate**. However, if the **ExpectedFinishDate** value is earlier than the

project's **RunDate**, or if no **ExpectedFinishDate** is provided, the **CalcFinishDate** is typically extended past **RunDate** as shown in the following illustration.

Figure 47.6 Impact of RunDate on FixedDuration task in progress



Notes:

- The **Project.RunDateTimeOfDayField** can be used to indicate if the **Project.RunDate** value refers to the start or end of the specified date.
- If run date driven projects are scheduled from the project finish, task scheduling conflicts can potentially arise as discussed in "Run date conflicts" on page 2175.

Predecessor relationships

A logical sequence of dependencies can be defined between tasks in a project. This is done by creating predecessor relationships. If a task has one or more other tasks designated as predecessors, than those tasks should typically be scheduled first and their dates used to constrain the other task in the relationship (known as a successor).

These relationships are defined in the **Predecessor** table which identifies each pair of task dependencies. This table contains two references to the **Task** table, **Predecessor**, identifying a predecessor task, and **Task**, identifying a successor task. These relationships can then be used to constrain the start or finish dates of the dependent task. How these predecessor relationships actually constrain task dates is determined by the **Predeccesor.Rule** and **Predecessor.Offset** fields. The **Rule** field determines which task dates are either constrained or used to constrain other dates, and the **Offset** allows a lag period to be specified between the tasks.

The specific application of these two fields is further influenced by whether a project's **ProjectType.ScheduleRule** value is set to "FromStart" or "FromFinish". With projects scheduled from the project start date, predecessor tasks constrain the scheduling of their successor tasks. Whereas in

projects scheduled from the project finish date, successor tasks constrain the scheduling of their predecessor tasks. This relationship is shown in the following table:

Table 47.1: Predecessor.Rule usage based on ScheduleRule settings

ProjectType.ScheduleRule	Predecessor.Rule	Impact
FromStart	FinishToFinish	Predecessor.CalcFinishDate + Offset constrains the successor's finish date (cannot finish before).
	FinishToStart	Predecessor.CalcFinishDate + Offset constrains the successor's start date (cannot start before).
	StartToFinish	Predecessor.CalcStartDate + Offset constrains the successor's finish date (cannot finish before).
	StartToStart	Predecessor.CalcStartDate + Offset constrains the successor's start date (cannot start before).
	None	Dependency ignored when calculating successor dates.
FromFinish	FinishToFinish	Task.CalcFinishDate - Offset constrains the predecessor's finish date (cannot finish after))
	FinishToStart	Task.CalcStartDate - Offset constrains the predecessor's finish date (cannot finish after)
	StartToFinish	Task.CalcFinishDate - Offset constrains the predecessor's start date (cannot start after)
	StartToStart	Task.CalcStartDate - Offset constrains the predecessor's start date (cannot start after)
	None	Dependency ignored when calculating predecessor dates.

Notes:

- All predecessors or successors defined for a given task, can be seen in the **Task** table through the calculated **Task.PredecessorList** and **Task.SuccessorList** fields. These fields are also directly editable, which forces records to be automatically created in the **Predecessor** table, if using workbooks that are automatically configured for project management such as in the **Project Plan** workbook included with RapidResponse.
- For in progress tasks within projects scheduled from finish, the **ProjectType.AddActualDurationToStartDate** can be set to "Y" if **ActualDuration** should be added to **StartDate** in order to determine where predecessor tasks having either a "StartToStart" or "FinishToStart" relationship can actually start.
- In progress tasks within projects scheduled from finish to have their **ActualDuration** added to **StartDate** when pushing dates on predecessors having a "StartToStart" or "FinishToStart" relationship. Additionally, if such a task is 100% complete, then its predecessor links are ignored.

Supply and demand order links

Project tasks might require specific material parts in order to complete. As such, a task's start or finish might also be dependent on the availability of specific demand or supply orders to provide the required parts. If delays or other issues arise with these orders, the task dates should be reflected accordingly.

In order for task to have material requirements associated with them, records must be added to the **TaskPartLink** table. Each record in this table references both a task in the **Task** table and a part that task requires in **Part** table. Once a task and part association has been made, the value in the **TaskPartLink.Type** field determines whether the task can be further linked to either specific demand order lines (**IndependentDemand** records) or supply order lines (**ScheduledReceipt** records).

In order to define links between demands and tasks, records should be added to the **TaskDemandLink** table (which references the **IndependentDemand** and **Task** tables), and in order to define links between supplies and tasks, records should be added to the **TaskSupplyLink** table (which references the **ScheduledReceipt** and **Task** tables). Each of these task linking tables contains **Rule** and **Offset** fields that can be used to specify how task start or finish dates are constrained. The **Rule** field determines which task dates are constrained by the order's availability, and the **Offset** field can be used to specify a lag period between order availability and impacted task date. Additionally, each of these tables contains an **ApplyConstraint** field with settings of "Y" (the default) and "N" which determine if a given order link is allowed to impact the task's start or finish date.

The following table outlines how settings are used to constrain task start or finish dates.

Table 47.2: Impact of supply and demand task links

Order Link From	Rule	Impact
TaskDemandLink	FinishDate	**IndependentDemand.AvailableDate + Offset** defines the earliest date the task can finish (as long as **ApplyConstraint** is set to "Y").
	StartDate	**IndependentDemand.AvailableDate + Offset** defines the earliest date the task can start (as long as **ApplyConstraint** is set to "Y").
	None	Task dates are not constrained by the demand.
TaskSupplyLink	FinishDate	**ScheduledReceipt.AvailableDate + Offset** defines the earliest date the task can finish (as long as **ApplyConstraint** is set to "Y").
	StartDate	**ScheduledReceipt.AvailableDate + Offset** defines the earliest date the task can start (as long as **ApplyConstraint** is set to "Y").
	None	Task dates are not constrained by the supply.

Notes:

- The offset value specified can be either positive or negative. For example, suppose a scheduled receipt's availability constrains a task start date, but the actual material is not needed until the second day of the task. In this case, an **Offset** of -1 could be specified to indicate that the task can start one day before the supply is available.
- The earliest date that a given independent demand is needed by any project task is reported in the **IndependentDemand.TaskNeedDate** field, and the earliest date that a given scheduled receipt is needed by any project task is reported in the **ScheduledReceipt.TaskNeedDate** field.

Task constraints

Besides constraints imposed on a task's start or finish date by project dates, other task dependencies, and order availabilities, tasks can have explicit constraints, which are typically date-based, applied to them. Typically, this is done by specifying a date in the **Task.ConstraintDate** field and then selecting a

rule from the **Task.ConstraintType** field that specifies how the provided date is used in constraining the task. For example, a task's start date might be fixed to a particular date or given a boundary before which it cannot start.

Task constraints might be useful in cases where external factors limit the date or dates when a given task must start or finish. However, caution should be taken when using task constraint dates as they limit RapidResponse's ability to schedule a task as efficiently as possible.

The following table outlines the available constraint type rules:

Table 47.3: Task constraint types

ConstraintType	Impact on task
AsLateAsPossible	Finishes as late as it can. For tasks in projects with a ScheduleRule of "FromStart", this reverses the normal task scheduling logic.
AsSoonAsPossible	Starts as soon as it can. For tasks in projects with a ScheduleRule of "FromFinish", this reverses the normal task scheduling logic.
FinishOn	Should finish on the specified ConstraintDate.
FinishOnOrAfter	Should finish on or after the specified ConstraintDate.
FinishOnOrBefore	Should finish on or before the specified ConstraintDate.
MustFinishOn	Must finish on the specified ConstraintDate.
MustFinishOnOrAfter	Must finish on or after the specified ConstraintDate.
MustFinishOnOrBefore	Must finish on or before the specified ConstraintDate.
MustStartOn	Must finish on the specified ConstraintDate.
MustStartOnOrAfter	Must start on or after the specified ConstraintDate.
MustStartOnOrBefore	Must start on or before the specified ConstraintDate.
None	None. Task scheduling ignores the specified ConstraintDate.
StartOn	Should start on the specified ConstraintDate.
StartOnOrAfter	Should start on or after the specified ConstraintDate.
StartOnOrBefore	Should start on or before the specified ConstraintDate.

Note that any task constraint types listed in the "Task constraint types" on page 2173 table whose type name begins with the word "Must" are considered mandatory and will have their dates honored even if they cause conflicts with other scheduling components (for example, task dependencies such as predecessors tasks or material order links). To avoid conflicts, other non-mandatory constraint types can be chosen instead. For example, the MustStartOn and StartOn constraint types are identical except that MustStartOn always starts on the constraint date (even if it generates a conflict), whereas StartOn honors the constraint date as long as it does not conflict with other scheduling components which are then given precedence.

Task conflicts

As discussed earlier in this chapter, automatically scheduled tasks can have their start and finish dates calculated based on parameters such as project scheduling rules and dates, predecessor/successor relationships, material order requirements, and task constraints. In certain cases, it is not possible for RapidResponse to generate a project plan that simultaneously respects all scheduling parameters or

limitations placed on the tasks in the project. When this happens, a task conflict is generated and the scheduling decisions that are subsequently made might be unexpected and lead to a project plan that is not necessarily achievable or desirable.

Task conflicts can potentially arise when certain scheduling parameters move a task in the opposite direction from which the project is being scheduled. For example, a constraint date might cause a task's start/finish dates to move in such a manner that not all scheduling parameters or limitations can be simultaneously respected. When this occurs, a conflict is generated and reported in the **Conflict** table. This table can then be used to investigate the details of each conflict which might need to be resolved in order to produce an achievable project plan.

Note: For more information about the **Conflict** table, see "Conflict table" on page 963.

Constraint conflicts

Certain types of constraints can cause task conflicts in projects that are scheduled from their start date (have **ProjectType.ScheduleRule** set to "FromStart"), as well as in projects that are scheduled from their finish date (have **ProjectType.ScheduleRule** set to "FromFinish").

When tasks are scheduled out from the project start date, conflicts can be caused by constraints that move tasks earlier by specifying dates on or before which the constraint must start/finish. The following constraint types can potentially lead to conflicts in projects scheduled from the start date:

- MustFinishOn
- MustFinishOnOrBefore
- MustStartOn
- MustStartOnOrBefore

Conversely, when tasks are scheduled back from the project finish date, conflicts can be caused by constraints that move tasks later by specifying dates on or after which the constraint must start/finish. The following constraint types can lead to conflicts in projects scheduled from the finish date:

- MustFinishOn
- MustFinishOnOrAfter
- MustStartOn
- MustStartOnOrAfter

Regardless of how a project is scheduled, when a constraint leads to a task scheduling conflict, the constraint date is honored and takes precedence over the other scheduling parameters. This can lead to the generation of project plans that are not necessarily achievable or desirable.

For example, suppose in a project scheduled from its start date that a given task has a FinishToStart relationship with a predecessor and a 1 day offset is specified. If the predecessor finishes on June 18 2013, then the task would be scheduled to start on June 20 2013. However, if a "MustStartOnOrBefore" constraint was also specified on the task with a ConstraintDate of June 17 2013, then a task conflict would be generated and the task would be forced to start on that date (before its predecessor has actually finished). Assuming the task was truly dependent on its predecessor's completion, then the resulting project plan would not be achievable and corrective action would be required to ensure achievable start and finish dates on the conflicted task.

Material order conflicts

Material order links typically only cause task scheduling conflicts when projects are scheduled from their finish date (have **ProjectType.ScheduleRule** set to "FromFinish"). Conflicts can then potentially arise when the available date on a required demand or supply order moves tasks later and results in start/finish dates that are later than those determined by other scheduling parameters such as predecessor/successor relationships. When such a conflict occurs, the dates determined by the other scheduling parameters are honored and given precedence over the order requirement. This can lead to the generation of unachievable project plans if tasks are forced to start or finish in advance of the availability of the materials they require.

Note that it is also possible for a conflict to exist between a material order link and a constraint even while all other scheduling parameters are being respected. This happens when a constraint applied to a task forces that task to be scheduled in advance of the available date of a linked demand or supply order. This type of conflict can occur in either projects scheduled from start or from finish.

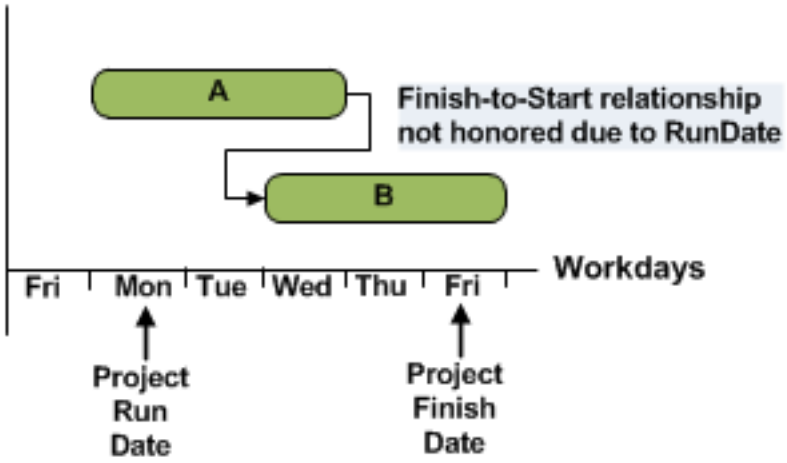
For example, suppose in a project scheduled from start, that based on its predecessor's finish dates a given task could potentially start on August 1, has a StartNoLaterThan constraint with a ConstraintDate of August 3, and a supply order link that constrains the task start date and is associated to a scheduled receipt with an AvailableDate of August 5. In this situation, the conflict only exists between the supply order link and the constraint. When this type of conflict occurs, two conflict records are generated in the **Conflict** table (one for the constraint and one for the order link), and the date associated with the constraint is given precedence and used.

Run date conflicts

Project run dates can only cause task scheduling conflicts in projects scheduled from their finish date (have **ProjectType.ScheduleRule** set to "FromFinishDate"). If the project is date-driven (has **ProjectType.PlanningRule** set to "DateDriven"), then the **Project.RunDate** value marks the current or "as of" date for the project and any tasks that have yet to start are prevented from starting before that date. This can occasionally lead to conflicts where a given task might be forced to finish after the specified project finish date, or where a predecessor task cannot finish in time to satisfy a successor task that depends on it.

For example, suppose the project run date is set to a given Monday, and the project finish date is that Friday. Further, suppose there are two tasks that have yet to start; task A which has a duration of 3 days, and task B which also has a duration of three days and has A as a predecessor of type "FinishToStart". In this scenario, task A cannot start any earlier than Monday which is too late to satisfy the dependency from task B as shown in the following illustration. Note that if the project were not date-driven, then task A would be allowed to start on the proceeding Friday and the predecessor relationship honored.

Figure 47.7 RunDate conflict



Actual start date and finish date conflicts

If the **Task.ActualStartDate** field is used to specify a date on which a task actually started, this can potentially lead to calculated task start dates that are in conflict with those determined from other scheduling parameters. Similarly, if either of the **Task.ActualFinishDate** or **Task.ExpectedFinishDate** fields are used to specify dates on which a task actually (or is expected to) finish, this can lead to calculated task finish dates that are in conflict with those determined from other scheduling parameters. These types of conflicts can occur in either projects scheduled from their start date or projects scheduled from their finish date.

For example, assume a project scheduled from start where the **ActualStartDate** on a task is populated and defines a start date earlier than what would otherwise be determined by RapidResponse. This might force the task to start before a required predecessor task could realistically complete in which case a conflict is logged. Similarly, assume a project scheduled from finish where the **ActualFinishDate** on a task is populated and defines a finish date later than would otherwise be determined by RapidResponse. This might force the task to finish after a dependent successor has already begun in which case a conflict is logged.

Task durations

The previous section discussed the main dependencies influencing when automatically scheduled tasks can start or finish. Another factor that impacts calculated task start and finish dates is how long it actually takes to complete a task once underway. This value is calculated by RapidResponse and reported in the **Task.CalcDuration** field.

The **Task** table contains a number of input fields with values that can be used when calculating a task's duration. For example, the **Duration** field can be used to specify an expected duration for the task, the **Work** field can be used to specify the amount of work required to complete a task, and the **Rate** field can be used to specify the rate at which the specified amount of work is completed. The specific input fields that get used when calculating task durations is determined by the **Task.DurationType** setting.

Because tasks typically require resources (people or machines) to complete them, the usage of these resources also impacts calculated task duration. The **Resource** table identifies resources that can be applied to tasks, along with the resource's **HoursPerDay** and **MaximumUnits** values. The **MaximumUnits** value is expressed in terms of **HoursPerDay** and indicates the number of hours the resource can work before it is over allocated and incurs any applicable overtime charges. For example, if

HoursPerDay is "8", and **MaximumUnits** is "1", then the resource has 8 standard working hours available per day.

The **ResourceAssignment** table is used to assign resources and their effort levels to tasks. This table references both the **Task** and **Resource** tables, and contains a **Units** field to indicate how much of a resource's daily availability should be allocated to a task. For example, if **Resource.HoursPerDay** is "8", and **Resource.MaximumUnits** is "1", then a **ResourceAssignment.Units** value of ".5" allocates 4 hours of the resource per day to the task.

Thus based on the input values provided on the task, as well as any resource assignments and the availabilities of those resources, RapidResponse can determine the calculated task duration. Note that in some cases the calendar associated with the resource (**ResourceType.Calendar**) might differ from the calendar associated with the task (**TaskType.Calendar** or **ProjectType.Calendar**). In these cases, only dates that fall on both the resource calendar and the task calendar are used in scheduling the task and determining loading of the resource.

The calculation of a task's duration also depends on the rule chosen from the **Task.DurationType** field which has three settings; "FixedDuration", "FixedUnits", and "FixedWorkAndRate". The remainder of this section discusses how each of these options affect calculated task durations.

Notes:

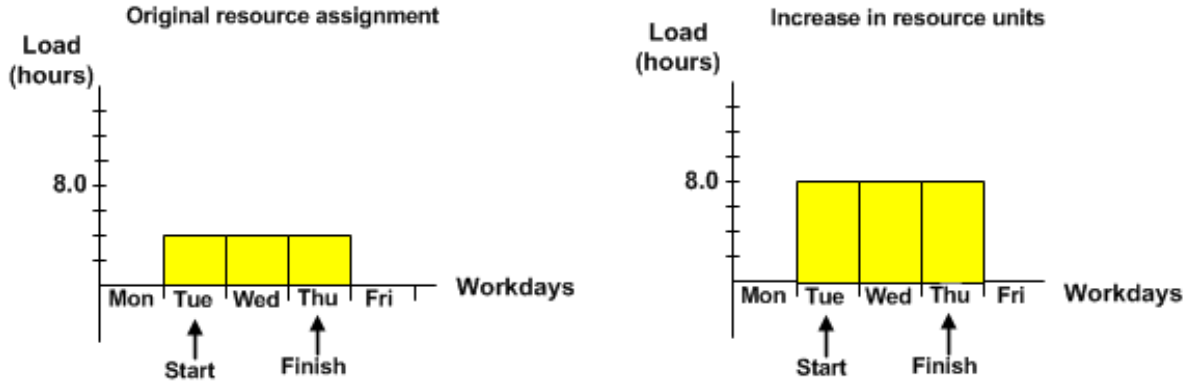
- For milestone tasks, calculated durations are always reported as 0 in projects where **ProjectType.AllowNonZeroMilestones** is set to "N" (which is the default). Otherwise, if **AllowNonZeroMilestones** is set to "Y", then durations on milestones are calculated the same as for normal tasks. For summary tasks, calculated durations are based on the difference between the latest **CalcFinishDate** for a subtask and the earliest **CalcStartDate** for a subtask.
- This section assumes that a resource's hours per day and maximum units available are taken from values on the **Resource** table. Optionally, time-phased **HoursPerDay** and **MaximumUnits** values for a resource can also be specified using the **ResourceAvailable** table. As well, the **HoursPerDay** a resource works can be overridden for specific days of the week using the **ResourceCalendarException** table. These are useful for cases where a resource's availability is expected to vary over time (for example, due to a vacation or temporary change in working schedule). For more information see "Resource availability and exceptions" on page 2183.

FixedDuration

The "FixedDuration" setting is meant to fix the task duration to the input duration value. That is, the **Task.CalcDuration** typically returns the number of days specified in the **Task.Duration** field. However, if values are provided in either **Task.ActualDuration** ( ) or **Task.RemainingDuration** ( )fields, then **Task.CalcDuration** is calculated as the sum of these two fields. As well, increasing resource assignment units on "FixedDuration" tasks does not impact the task duration, but rather just the amount of load placed on the resources that are required to complete it.

For example, assume a task with **Duration** of "3", and a single resource, having **HoursPerDay** of "8" and **MaximumUnits** of "1", assigned to the task at **Units** of ".5". As shown in the following illustration, the calculated duration would be 3, and 4 hours of load per day placed on the resource. However, if the **Units** assigned was increased to "1", then 8 hours of load per day is placed on the resource, but the calculated duration remains at 3 days.

Figure 47.8 "FixedDuration" task (diagrams) showing original resource assignment and increase in resource units.



Note that the calculations described previously are always applicable to "FixedDuration" task in projects that use the default **ProjectType.PlanningRule** setting of "Standard". However, in projects where **ProjectType.PlanningRule** is set to "DateDriven", the calculated task duration might differ from the specified input duration(s) in cases where actual/expected start and finish dates are provided on the task as follow.

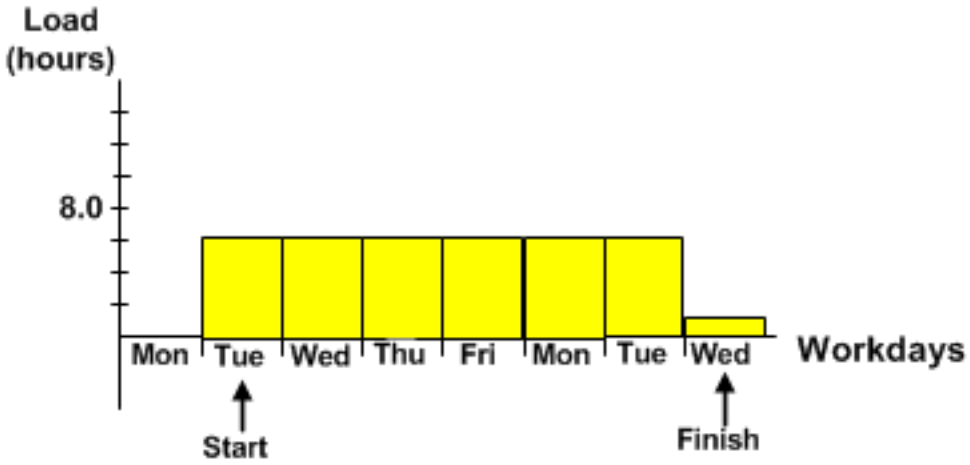
- If both **Task.ActualStartDate** and **Task.ActualFinishDate** are populated, the calculated task duration is the difference between these two dates.
- If only **Task.ActualStartDate** and **Task.ExpectedFinishDate** are populated, the calculated task duration is typically the difference between these two dates. However, if **Task.ExpectedFinishDate** occurs before **Project.RunDate**, the calculated task duration is then the difference between **Task.ActualStartDate** and **Project.RunDate** plus one day.
- If only **Task.ActualStartDate** is populated, the calculated task duration is typically the difference between **Task.ActualStartDate** and **Project.RunDate** plus the value in **Task.RemainingDuration**. However, if **Task.RemainingDuration** is set to "0", then the calculated task duration is the difference between **Task.ActualStartDate** and the later of either **Task.ActualStartDate** plus **Task.Duration** or the **Project.RunDate.**

FixedUnits

The "FixedUnits" setting calculates task duration based on a combination of the input duration and any resource assignments made against the task. That is, task duration is calculated as **Task.Duration** divided by cumulative **ResourceAssignment.Units** from all resources assigned to it.

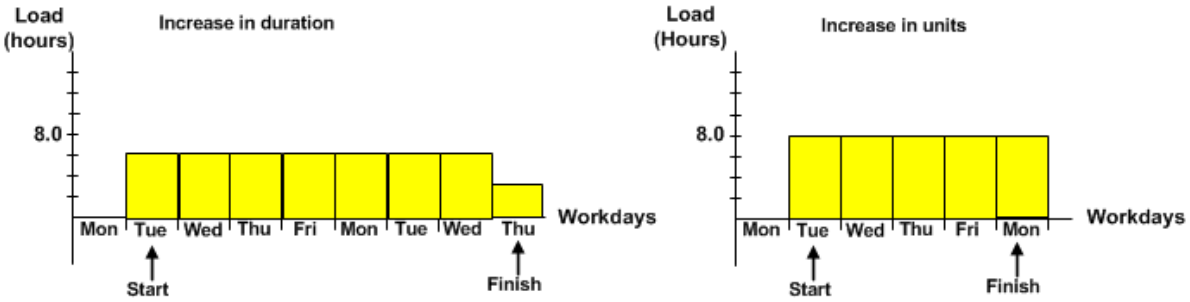
For example, assume a task with **Duration** of "5", and a single resource, having **HoursPerDay** of "8" and **MaximumUnits** of "1", assigned to the task at **Units** of ".8". The calculated duration would then be 6.25 days (5 / .8), with 6.4 hours of load per day (8 * .8) placed on the resource, excepting the last day which requires only 1.6 hours, as shown in the following illustration.

Figure 47.9 "FixedDuration" task



With the "FixedUnits" setting, an increase in input duration leads to an increase in the calculated duration. For example, as shown in the following illustration, if **Duration** of "5" in the previous example were changed to "6" and other settings left as is, then the calculated duration would be extended to 7.5 days. Conversely, an increase in resource units assigned leads to a decrease in the calculated duration. For example, as shown in the following illustration, if the **Units** value of ".8" in the previous example were changed to "1" and other settings left as is, then the calculated duration would be shortened to 5 days.

Figure 47.10 Impact of changes on FixedUnits task (diagrams). First shows an increase in duration. Second shows an increase in units.



Note: If multiple resources are assigned to the same task, their assignment units are combined together to determine the calculated task duration. For example, if a task with an input duration of 9 days has a given resource assigned at 1 units and another resource assigned at .5 units, then the calculated task duration is 6 days (9/1.5).

FixedWorkAndRate

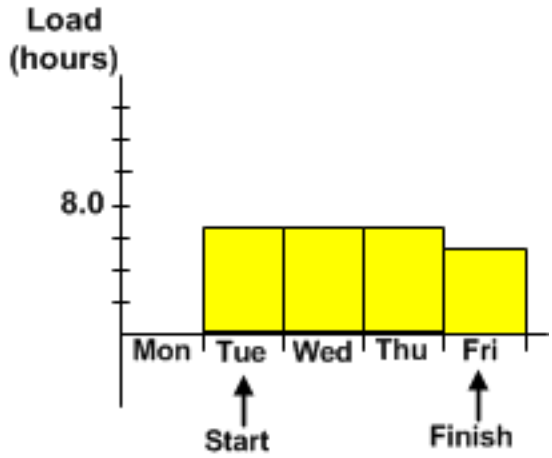
The "FixedWorkAndRate" option bases calculated task durations on input work and rate values specified on either the task or its resource assignments.

If the task does not have any resources assigned to it, then the **Task.Work** field determines the amount of work to be done on the task, and the **Task.Rate** field determines the rate at which that work is completed on that task. In other words, **Task.CalcDuration** is calculated as `Work/Rate`. For example, if **Work** is "8" and **Rate** is "2", then **CalcDuration** is 4.

If the task has resources assigned to it, then fields on the resource assignment can override those on the task (if valid values are given). The **ResourceAssignment.Rate** field can be used to specify the rate at which the resource completes work on the task and the **ResourceAssignment.Units** indicates the amount of the resource applied at that rate, while the combination of the **ResourceAssignment.ActualWork** and **ResourceAssignment**.**RemainingWork** fields define the amount of work to be done on the task. In this situation, **Task.CalcDuration** is then calculated as `(ActualWork+RemainingWork)/(Rate*Units)`.

For example, assume a task with a single resource assigned to it that has **HoursPerDay** of "8" and **MaximumUnits** of "1". Further, suppose the **ResourceAssignment** record has **ActualWork** of "0", **RemainingWork** of "3", **Rate** of ".8", and **Units** of "1". The calculated duration would be 3.75 days (3 / .8), with 6.4 hours of load per day placed on the resource (8 * .8), excepting the last day which requires only 4.8 hours of resource effort, as shown in the following illustration.

Figure 47.11 "FixedWorkAndRate" task



With the "FixedWorkAndRate" setting, an increase in work leads to an increase in the calculated duration. For example, as shown in the following illustration, if **RemainingWork** of "3" in the previous example were changed to "4" and other settings left as is, then the calculated duration would be extended to 5 days. Conversely, an increase in rate (or units) leads to a decrease in the calculated duration. For example, as shown in the following illustration, if the **Rate** value of "1" in the previous example were changed to "1.25" and other settings left as is, then the calculated duration would be shorted to 3 days.

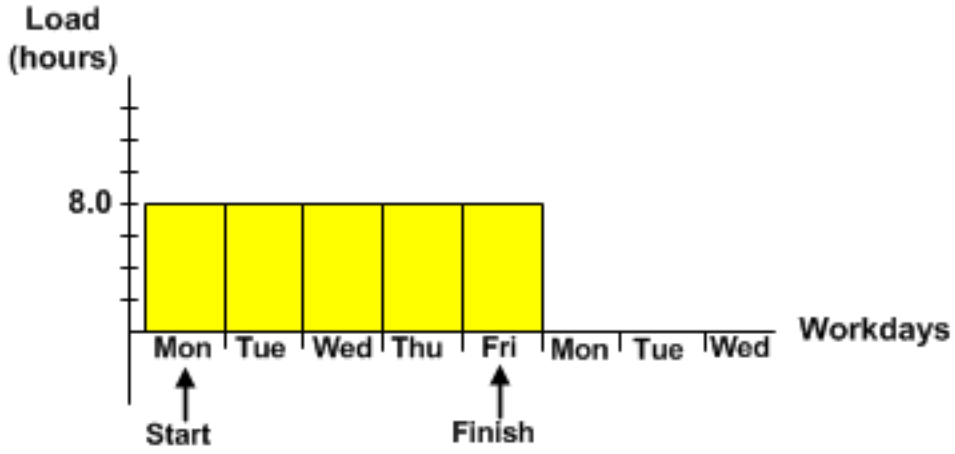
Figure 47.12 Impact of changes on "FixedWorkAndRate" task.(diagrams) First diagram: Increase in work. Second diagram: Increase in rate.

Note: If there are not valid (positive) work and rate values provided in either the **Task** or **ResourceAssignment** tables, then the value provided in the **Task.Duration** field is reported instead.

Resource efficiency

By default, when task durations are calculated in RapidResponse, any assigned resources are assumed to work at a constant level of efficiency during the course of the task. For example, assume a resource with **HoursPerDay** of "8" and **MaximumUnits** of "1", is assigned at a **Units** value of "1" to a task having an input duration of 5 days. In this case, the calculated task duration would be 5 as the resource is assumed to work at maximum efficiency throughout as shown in the following illustration.

Figure 47.13 FixedUnit task with full resource efficiency assumed



However, in cases where resources are not expected to produce at constant rates of efficiency on a task, an efficiency curve can be used. Efficiency curves contain point values between 0 and 1, where 1 indicates full efficiency, and are used to define variable rates of efficiency for all resources assigned to a task. This might typically be used for very complicated tasks that require additional time for any assigned resources to ramp up before they can contribute fully.

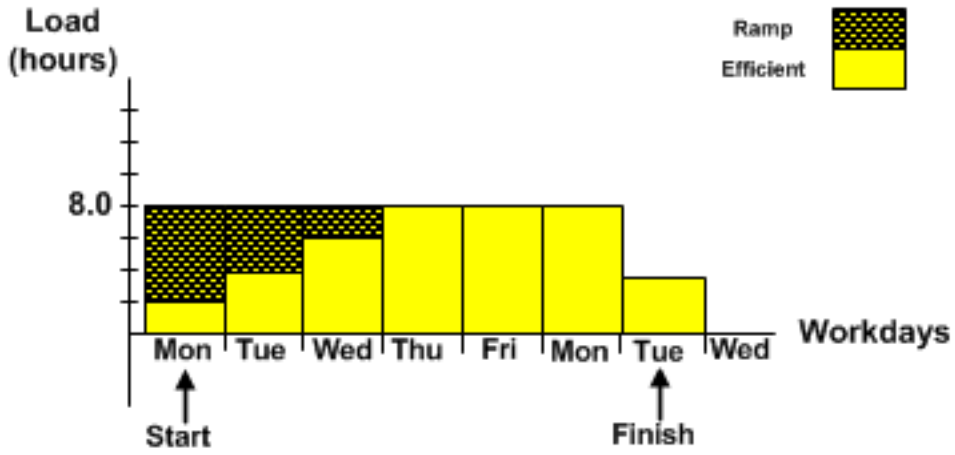
Efficiency curves are enabled at the task level by the setting in the referenced **TaskType.EfficiencyCurveRule** field. This field has the following three options:

- **None**—no efficiency curve is used (default).
- **Ramp**—a linear increasing efficiency curve is created and applied.
- **PointsCurve**—a named efficiency curve from the **PointsCurve** table is applied.

If using the **Ramp** option, the efficiency curve is automatically created and applied based on the quantity value specified in the **Task.Ramp** field. This value is used to define the number of points in the curve with those points spread evenly between 0 and 1. For example, if **Ramp** is "4", then the curve points would be .25 on the first day, .5 on the second day, .75 on the third day, and 1 on the fourth and all subsequent days.

The following illustration shows the impact of applying this ramp to the example task shown in the previous figure. Note that applying a ramp to a task does not affect the daily load on the assigned resources, just the amount of productive work generated from those loads and hence increases calculated task durations. In this case, the previously calculated task duration of 5 days now becomes 6.5 days.

Figure 47.14 Ramp applied



If using the **Curve** option, then the efficiency curve used is the one defined by the **Task.Curve** reference. This field references a value in the **PointsCurve** table which contains all named efficiency curves that have been defined in RapidResponse. Each of these named efficiency curves is then made up of all **CurvePoint** table values that reference it.

Each **CurvePoint** record identifies the associated **PointsCurve** value, and also contains **Index** and **Quantity** values, where **Index** refers to a numbered point on the curve while **Quantity** defines the efficiency of the resource at that point and should be a value between 0 and 1 (where 1 indicates full efficiency). The last value as specified Index defines the end of the curve and is applied for each subsequent day until the end of the task is reached.

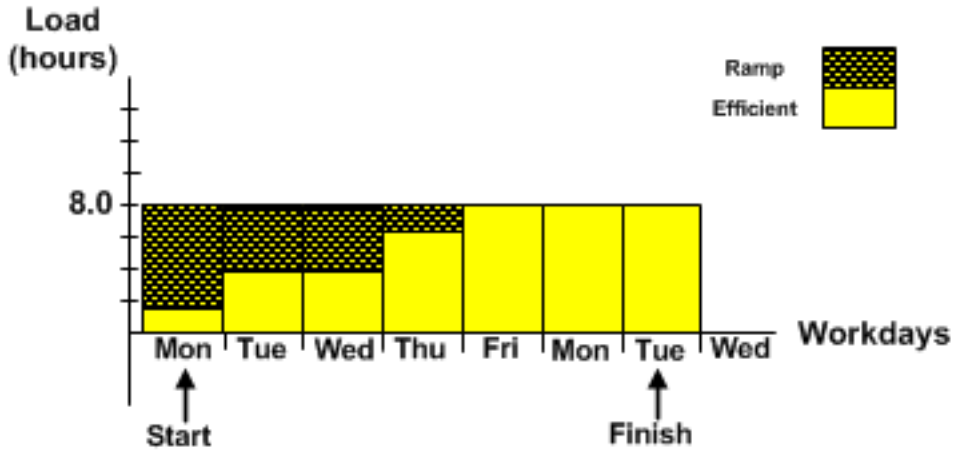
For example, suppose a **PointsCurve** named "Sample" meant to specify resource efficiencies against task of 20% on the first day, 50% on the second and third day, 80% on the fourth day, and 100% on the fifth and all subsequent days. Associated records in the **CurvePoint** table would then look similar to the following.

Table 47.4: "Sample" CurvePoint records

Curve	Index	Quantity
Sample	1	.2
Sample	2	.5
Sample	3	.5
Sample	4	.8
Sample	5	1.0

The following illustration shows the impact of applying this curve to the example task shown in the first resource efficiency example. Note that applying a curve to a task does not affect the daily load on the assigned resources, just the amount of productive work generated from those loads and hence increases calculated task durations. In this case, the previously calculated task duration of 5 days now becomes 7 days.

Figure 47.15 PointsCurve applied



Resource availability and exceptions

As discussed previously, a resource's standard work availability is, by default, defined by the combination of the **HoursPerDay** and **MaximumUnits** fields in the **Resource** table. The standard availability is then assumed to be effective for all dates on the resource's working calendar. In some cases, however, it might be necessary to override a resource's availability for a specified period of time, or to specify an exception to the number of hours per day worked on a specific day of the week.

Resource available

A resource's **HoursPerDay** and **MaximumUnits** values can be overridden for a specified period of time using the **ResourceAvailable** table. By adding a record to this table for a given resource, the corresponding values on the **Resource** table are overridden as of the effective date given on the record. These new **HoursPerDay** and **MaximumUnits** values then remain in effect for that resource until the next effective date found on a **ResourceAvailable** record for the resource.

For example, suppose the resource YZ is going to be on vacation for the entire month of June, and returning to work on July 1. This could be modeled by adding values to the **ResourceAvailable** table similar to the following.

Table 47.5: ResourceAvailable values

Resource	EffectiveDate	HoursPerDay	MaximumUnits
YZ	June 1	8	0
YZ	July 1	8	1

Similarly, if an extra unit of resource XY is temporarily made available to double its capacity for a 2 week period beginning on July 10, this could be modeled by adding values to the **ResourceAvailable** table similar to the following.

Table 47.6: ResourceAvailable values

Resource	EffectiveDate	HoursPerDay	MaximumUnits
YZ	July 10	8	2
YZ	July 24	8	1

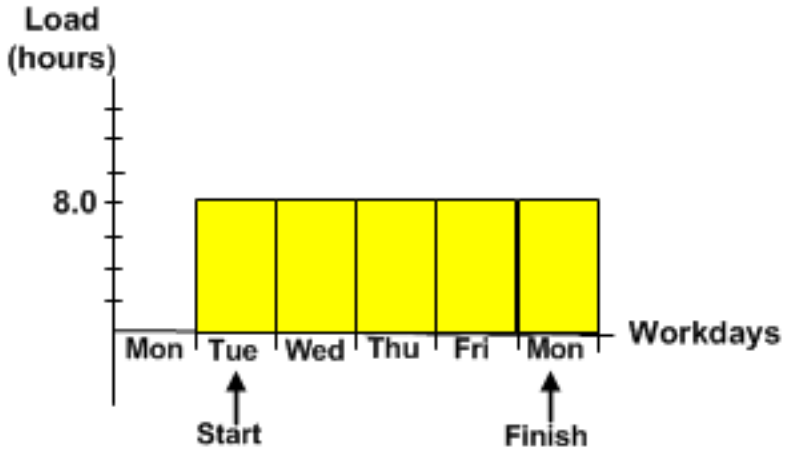
Note that regardless of the **MaximumUnits** specified on either the **Resource** or **ResourceAvailable** table, resources might potentially be assigned in excess of these maximum units. By default, this is then typically reflected in overloaded resources and overtime rates potentially being charged. However, this is configurable for cases where resources should not be overloaded. To ensure that a resource is not overloaded and limit the potential for overtime costs, at the individual task only, the level resources option can be used as discussed in "Leveling task resources" on page 2186. For example, this might be useful to ensure loads are not placed against a resource while it is on vacation or otherwise unavailable.

Note: The **Overloaded** field on the **Resource** table indicates if a resource is overloaded at any point from assignments across all projects, and the **OverloadedDate** field indicates the first date on which the resource is overloaded (has resource assignments that exceed its maximum units available).

Resource calendar exception

As discussed previously, resource availabilities can be defined on the **Resource** table using the **HoursPerDay** and **MaximumUnits** fields, and optionally overridden beginning from one or more effective dates using the **HoursPerDay** and **MaximumUnits** fields on the **ResourceAvailable** table. The effective resource availability is then applied to all dates on the resource's working calendar in the effective range. For example, assume a resource on a standard 5 day work week, with effective **HoursPerDay** of 8 and **MaximumUnits** of 1. If the resource is fully assigned to a "FixedUnits" task with an input duration of 5 days and starting on a Tuesday, then the task would have a calculated duration of 5 days and finish the following Monday as shown in the following illustration.

Figure 47.16 Task without resource calendar exception



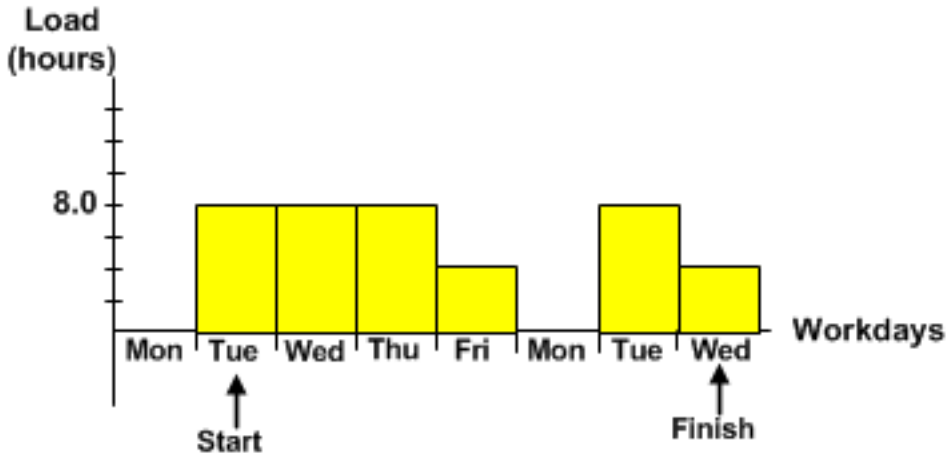
Sometimes, a resource's working hours per day might vary based on the day of the week. In such cases, the **ResourceCalendarException** table can be used to override a resource's effective **HoursPerDay** for a given **DayOfWeek** value as of a particular date. For example, suppose a resource is going to work a shortened week for the month of August by taking each Monday off and only working half days on Friday, before returning to the standard work week in September. To model this, records could be added to the **ResourceCalendarException** table with values similar to the following.

Table 47.7: ResourceCalendarException values

EffectiveDate	DayOfWeek	HoursPerDay
August 1	Monday	0
August 1	Friday	4
September 1	Monday	8
September 1	Friday	8

Now assume the same task as depicted in the following figure starts on a Tuesday in July. In this case, due to the limited availability of the resource to perform work on Mondays and Fridays, the task would not finish until the following Wednesday as shown in the following illustration, and would have a calculated duration of 6.5 days rather than 5.

Figure 47.17 Task with resource calendar exceptions on Monday/Friday



Note: Values entered in the **ResourceCalendarException** table are only applicable if the **DayOfWeek** value corresponds to a day on the resource's working calendar. For example, if a resource has a Monday to Saturday working calendar, then any records added where the DayOfWeek is set to "Sunday" would be ignored.

Leveling task resources

When resources are assigned to a task, the daily load placed on those resources, and hence the calculated task duration for most task types, is driven by the value specified in **ResourceAssignment.Units**. In some cases, this can lead to overloaded resources during the course of a task and hence overtime hours and charges incurred from those resources.

At the individual task level only, overloaded resources on a task can be leveled using a setting in the referenced **TaskType.LevelResources** field. This field has the following two options:

- **N**—resources assigned to a given task in excess of the effective maximum units for the resource (from either the **Resource** or **ResourceAvailable** tables) are not leveled. This might mean shorter task durations but the incurring of overtime costs.
- **Y**—resources assigned to a given task in excess of the effective maximum units for the resource (from either the **Resource** or **ResourceAvailable** tables) are leveled. This might mean longer task durations but not incurring overtime costs.

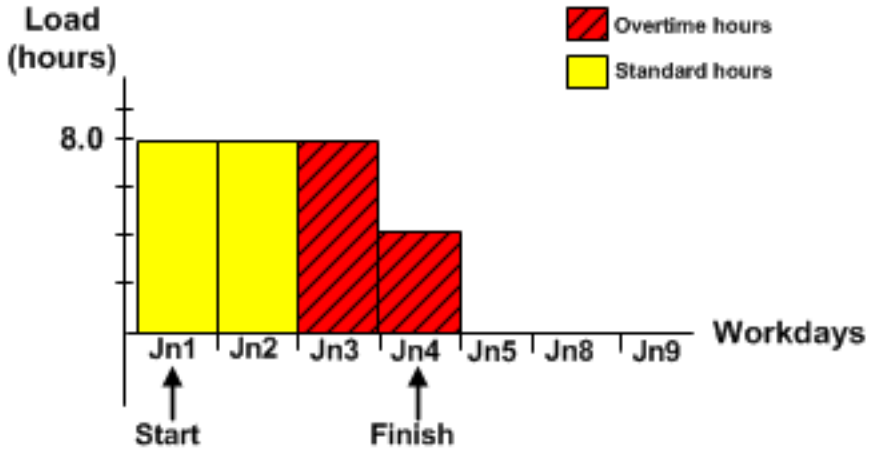
Leveling task resources might be required in cases where it is not desirable or feasible for the resource required on a task to work more than the maximum specified units. For example, suppose values in the **ResourceAvailable** table set up for a given resource and indicating the start of a vacation period on June 4 and returning to work on June 8.

Table 47.8: ResourceAvailable values

EffectiveDate	HoursPerDay	MaximumUnits
April 1	8	1
June 3	8	0
June 8	8	1

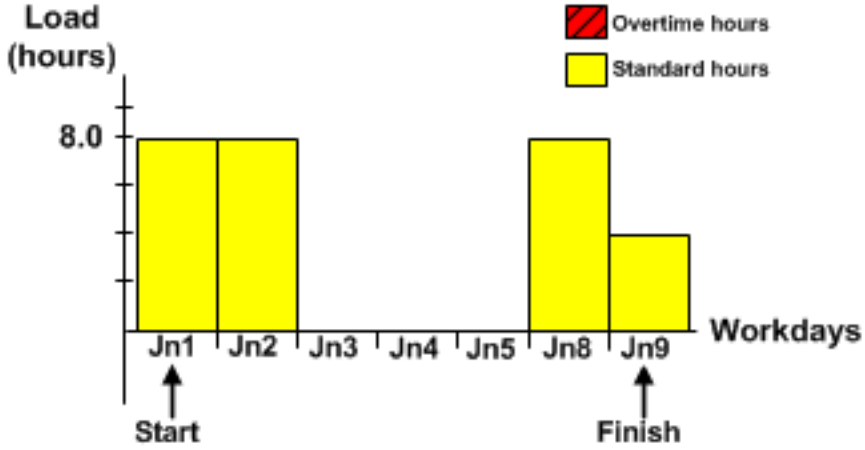
Further, suppose the resource is assigned to a "FixedUnits" task that starts on June 1 with an input duration of 3.5 days as shown in the following illustration. If **LevelResources** is set to "N", then load is still placed on the resource on June 3 and June 4, although the resource is reported as overloaded and any applicable overtime rates used for the hours assigned. The calculated task duration is reported as 3.5 days as shown in the following figure.

Figure 47.18 Task with resource assignments exceeding maximum units



Instead, if **LevelResources** is set to "Y", then no load is placed on the resource during its vacation period (the dates where the effective **MaximumUnits** is "0") as shown in the following illustration. As such, the task does not finish until the vacation period ends and calculated task duration is now reported as 6.5 days as shown in the following figure.

Figure 47.19 Task after LevelResource option applied



Note: The LevelResources option has no impact on "FixedDuration" tasks (always treated as if set to "N").

Project costs

Costs are typically an important metric in the overall project plan. A number of different cost values are reported in the RapidResponse both at the overall project level, and for the individual tasks that comprise a project. This includes costs incurred from resource and material usage, as well as penalty costs incurred from failing to meet agreed upon dates or milestones.

Resource costs

Costs associated from using resources to complete project tasks can be broken down into fixed, standard, and overtime costs. These costs can also be reported and aggregated at the individual resource level by using the calculated **ResourceLoad** table as discussed in "ResourceLoad table" on page 1157.

Fixed costs

At the task level, these are fixed (set) costs incurred from the assignment of a resource to a task. Fixed cost values from assigning a resource come from the **FixedCost** field on either the **Resource** table or the **ResourceCosts** table (if it has an effective record for the resource on a given assignment date). The **Task.ResourceFixedCost** field then reports the sum of the effective **FixedCost** values from each resource that has been assigned to the task.

At the project level, all fixed costs incurred against tasks are rolled up and reported in the **Project.ResourceFixedCost** field.

Standard costs

At the task level, these are variable costs incurred for each standard hour per day that a resource works on an assigned task. The standard hourly rates charged come from the **StandardRate** field on either **Resource** table or the **ResourceCosts** table (if it has an effective record the resource on a given date). The effective standard rate is then charged for each hour worked up to the maximum standard hours per day for the resource (as determined by the product of the effective **HoursPerDay** and **MaximumUnits** values for the resource). The **Task.StandardCost** field then reports the total standard cost calculated from each resource that has been assigned to the task.

At the project level, all standard costs incurred against tasks are rolled up and reported in the **Project.StandardCost** field.

Overtime costs

At the task level, these are variable costs incurred for each overtime hour per day that a resource works on any assigned tasks. The hourly overtime rates charged come from the **OvertimeRate** field on either **Resource** table or the **ResourceCosts** table (if it has an effective record the resource on a given date). The effective overtime rate is then charged for each hour worked in excess of the maximum standard hours per day for the resource (as determined by the product of the effective **HoursPerDay** and **MaximumUnits** values for the resource). The **Task.OvertimeCost** field then reports the total overtime cost calculated from each resource that has been assigned to the task.

Note that if multiple tasks use a given resource on a day on which it works overtime hours, the overtime hours and costs are split proportionally between the tasks based on the resource assignment ratio. For example, suppose a resource with **HoursPerDay** of 8 and **MaximumUnits** of 1 (that is, a standard 8

hours work day). If the resource works 6 hours against one task on a given day, and 4 hours against another task, then 1.2 hours of overtime would be charged against the first task and .8 hours of overtime would be charged against the second task. Similarly, the standard hour costs would also be split proportionally between the tasks based on resource assignment ratio.

At the project level, all overtime costs incurred against tasks are rolled up and reported in the **Project.OvertimeCost** field.

Actual resource costs

Once a project moves past the planning stage into execution, the actual resource costs incurred can be tracked using the **ResourceCostActual** table. This table allows for entering the details of actual work on tasks so that the associated resource costs can be compared against the estimate resource costs determined by the project plan (for example, this might be useful in a resource burndown chart).

For a given resource assigned to a task on a particular date, the **OvertimeHours** and **StandardHours** fields on the **ResourceCostActual** table can be used to enter the resource's actual working time(s) on the task. The hourly rates effective for the resource, including any rate adjustments, are then used to calculate the cost of those hours worked which are reported in the **OvertimeCost** and **StandardCost** fields. If any fixed costs are associated with the resource assignment on a given date, they can be directly entered in the **FixedCost** field.

For a given task, all actual costs from resource assignments to the task are summed together and reported in the following group of fields on the **Task** table (the same fields exist on the **Project** table for reporting actual costs rolled up from all tasks in a project).

- ActualFixedCost
- ActualOvertimeCost
- ActualStandardCost

Resource rate adjustments

As discussed previously, standard and overtime rates are defined for each resource and then charged accordingly when the resource is assigned to project tasks. If required, rate adjustments can be applied to the resource's normal rates when working on specific projects and/or tasks. For example, a discount rate might be applied to all resources working on an important customer project, or a premium rate might be charged for all engineering resources working on a specific task.

Rate adjustments can be specified using a **RateAdjustment** field found on six different input tables. However, only one rate adjustment is ever considered effective and applied when calculating standard and overtime costs for a particular resource assignment. If there are multiple rate adjustments applicable to a given resource assignment, then precedence is typically given to the more narrowly defined rate adjustment as described in the following table (lower numbers indicate precedence over higher numbers).

Table 47.9: Rate adjustment precedence

	Table	Adjustment applies to
1	ResourceAssignment	A specific resource working on a given task (overrides any other applicable adjustments).
2	ResourceGroupTaskOverride	All members of a resource group working on a given task (unless overridden).
3	Task	Any resource working on a given task (unless overridden).

Table 47.9: Rate adjustment precedence

	Table	Adjustment applies to
4	ResourceProjectOverride	A specific resource working on any task in a given project (unless overridden).
5	ResourceGroupProjectOverride	All members of a resource group working on any task in a given project (unless overridden).
6	Project	Any resource working on any task in a given project (unless overridden).

When a resource rate adjustment is effective, the standard and overtime rates charged for a given resource are then adjusted by the value in the applicable **RateAdjustment** field. For example, if a resource normally charges a standard rate of $10.00, then a rate adjustment of ".2" results in an effective standard rate of $12.00. Similarly, a rate adjustment of "-.05" would result in an effective standard rate of $9.50.

Notes:

- The effective rates for a given resource assignment are reported in the **EffectiveStandardRate** and **EffectiveOvertimeRate** fields on the **ResourceLoad** table.
- The **EffectiveRateAdjustmentSource** field on the **ResourceAssignment** indicates the table, if any, where the effective rate adjustment is defined and the **EffectiveRateAdjustment** field on that table indicates the amount of the adjustment.

Material costs

RapidResponse calculates both the planned and actual material costs needed to complete a task.

Planned material costs for a task are based on the original material requirements defined using the **TaskPartLink** table. This table references both the **Task** and **Part** tables and identifies the specific parts and the amount of each required to complete a given task. The amount of each part required (**TaskPartLink.Quantity**) is then multiplied by its standard unit cost (**Part.StdUnitCost**) to give the planned material cost for that part. The sum of the planned material costs for all parts assigned to a given task is then reported in the **Task.MaterialCost** field.

Actual material cost calculations can come from either any independent demands or any scheduled receipts linked to the task. This depends on whether the planned material requirement has a **TaskPartLink.Type** value of "Demand" or "Supply".

If a given material requirement for a task is of type "Demand", then the **TaskDemandLink** table can be used to link the **Task** record to one or more actual demands in the **IndependentDemand** table and assign a specific amount of the order(s) to the task. The **TaskDemandLink.AssignedQuantity** field is then multiplied by the **IndependentDemand.CostOfGoodsSold** and the resulting value divided by **IndependentDemand.EffectiveDemand** to determine the cost of the assigned material requirement.

If a given material requirement for a task is of type "Supply", then the **TaskSupplyLink** table can be used to link the **Task** record to one or more actual supplies in the **ScheduledReceipt** table and assign a specific amount of the order(s) to the task. The **TaskSupplyLink.AssignedQuantity** field is then multiplied by the **ScheduledReceipt.EffectiveUnitPrice** field to determine the cost of the assigned requirement.

The sum of the actual material costs from all demand and/or supply orders assigned to a given task is the reported in the **Task.ActualMaterialCost** field.

At the project level, the planned material costs across all tasks in the project are rolled up and reported in the **Project.MaterialCost** field. Similarly, actual material costs across all tasks in the project are rolled up and reported in the **Project.ActualMaterialCost** field.

Penalty costs

Penalty costs can be incurred if specific project dates or agreed upon milestones are not met and can be incurred at both the task and project levels. At the task level, penalty costs are reported in the **Task.PenaltyCost** field, while the **Task.CumulativePenaltyCost** field can be used to report the task's penalties combined with those of its child tasks (for example, this might be used for reporting on summary tasks). At the project level, penalty costs specific to the project are reported in the **Project.PenaltyCost** field, while the **Project.CumulativePenaltyCost** field reports the sum of the project penalty costs with the penalty costs from all of its non-summary tasks.

For complete details on how penalties are applied and calculated against projects or tasks in RapidResponse, see "Penalties" on page 2191.

Other task costs

To provide a complete picture of task costs, any planned miscellaneous costs not associated with the assignment of either resource or materials to a task, and not related to penalties levied against a task, can be provided in the **Task.Cost** field. At the project level, these miscellaneous costs are rolled up and reported in the **Project.TaskCost** field.

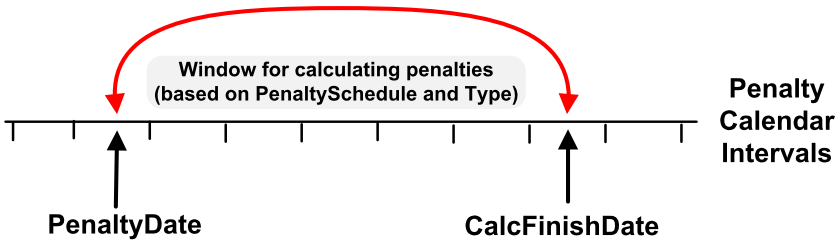
To track how the planned costs compare against the actual miscellaneous costs incurred during the course of a task, the **Task.ActualCost** field can be used to record actual costs values as they accrue during project execution. At the project level, these miscellaneous actual costs are rolled up and reported in the **Project.ActualTaskCost** field.

Penalties

Missing a key deadline or milestone in a project might typically result in financial penalties being levied as agreed upon between a company and its customers. These might be lump sum penalties applied when a key date is missed, recurring penalties applied at defined intervals after the date has slipped, or both. A calculated **PenaltyCost** field is found on both the **Project** and **Task** tables and reports any applicable penalties incurred at the project and task-levels respectively.

The **Project** and **Task** tables each contain a **PenaltyDate** field used to define the beginning of a window when project penalties will be calculated. The end of this window is determined by the project or task's **CalcFinishDate** field. In other words, penalties are only incurred when a given project or task has a calculated finish date that is later than its specified penalty date as shown in the following illustration.

Figure 47.20 Penalty window



Within the defined penalty window, the dates on which penalties can be incurred is determined by the markers on the penalty calendar. For projects, this is set by the **ProjectType.PenaltyCalendar** reference and for tasks this is set by the **TaskType.PenaltyCalendar** reference. The effective penalty costs applied on any or all of these dates is then determined by a referenced penalty schedule (and related **ProjectType** or **TaskType** settings). RapidResponse supports both date-based and interval-based penalty schedules.

Note: The **Project** and **Task** tables also each contain a **CumulativePenaltyCost** field. For projects, this field sums the project's penalty cost together with the penalty cost from all of its tasks. For tasks, this field sums the task's penalty cost with that from all of its subtasks (for example, this might be useful to report on summary tasks).

Date-based penalty schedules

Projects and tasks each get their penalty schedule from the value they reference in the **PenaltySchedule** table. If the **ProjectType.PenaltyRule** or **TaskType.PenaltyRule** field is set to "Date", then the effective penalty schedule is made up of those date-based records in the **PenaltyScheduleByDate** table that reference the same **PenaltySchedule** value as the **Task** or **Project** record. This setting is useful for situations where penalties are attached to specific calendar dates, and schedules defined in the **PenaltyScheduleByDate** table are thereby not typically reusable from project to project.

Each record in the **PenaltyScheduleByDate** table identifies the penalty schedule it belongs to, along with a one-time cost (single lump-sum penalty) and a per interval cost (applied on each applicable penalty calendar marker). **PenaltyScheduleByDate** records also contain an **EffectiveDate** field that specifies the first date that penalty costs defined on the record are effective. The penalty costs remain effective until the next **EffectiveDate** defined on a record for that same penalty schedule.

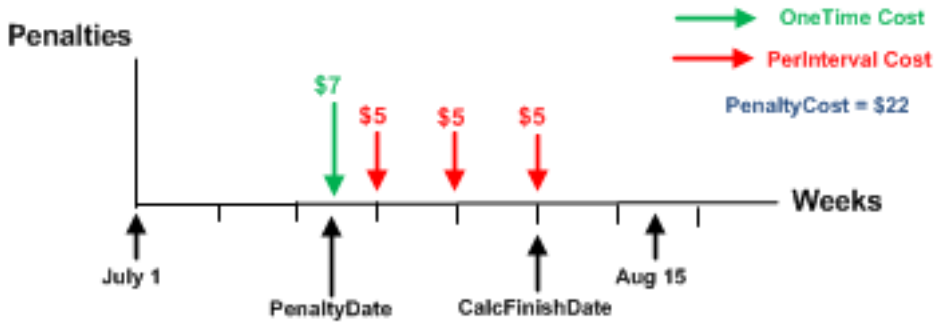
For example, suppose a series of **PenaltyScheduleByDate** records that reference a given **PenaltySchedule** value, PS1, as shown in the following table.

Table 47.10: Sample PenaltyScheduleByDate values

Schedule	EffectiveDate	OneTimeCost	PerIntervalCost
PS1	05-15-13	0	3
PS1	07-01-13	7	5
PS1	08-15-13	11	8
PS1	10-01-13	15	10

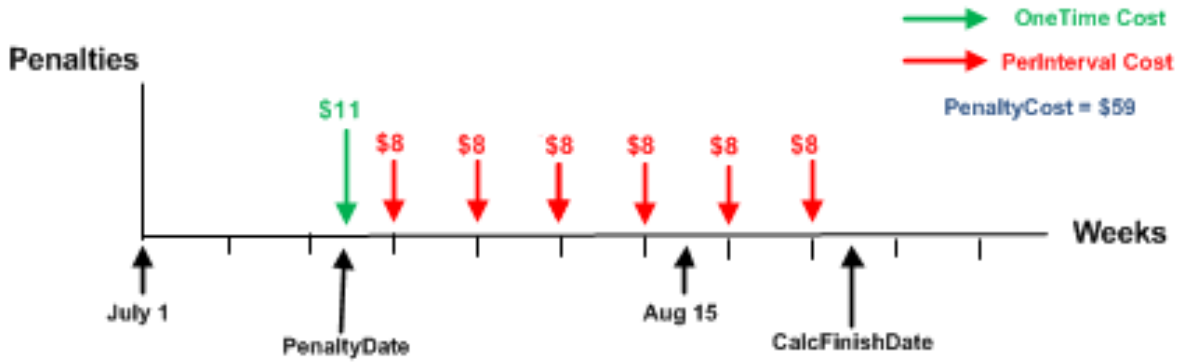
Further assume a task using a weekly penalty calendar and the PS1 penalty schedule, along with a penalty date and calculated finish date as shown in the following figure In this case, only the **PenaltyScheduleByDate** record with the **EffectiveDate** of 07-01-13 is effective within the defined penalty window. Therefore, this record's **OneTimeCost** and **PerIntervalCosts** are used and reflected in the total penalty costs charged to the task as seen in the figure.

Figure 47.21 Date rule with single effective entry



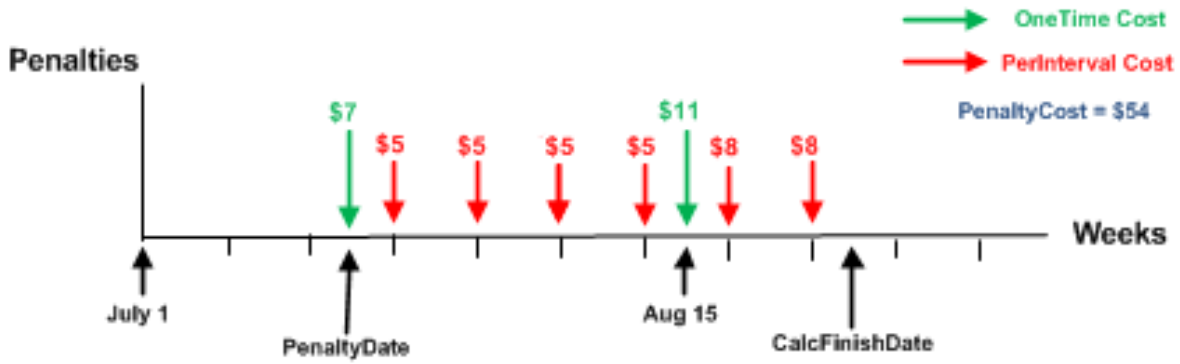
Sometimes, there might be multiple **PenaltyScheduleByDate** records effective within a given penalty window. For example, suppose the task shown previously has a later calculated finish date as shown in the following figure. In this case, the record with an **EffectiveDate** of 07-01-13 as well as the record with an **EffectiveDate** of 08-15-13 are both effective within the penalty calendar window. In such cases, the setting on the **TaskType.PenaltyAccumulationRule** field determines how the penalty costs on these records are used (a similar field is found on the **ProjectType** table for project-level penalties). By default, the **PenaltyAccumulationRule** field is set to "LastInterval" which specifies that penalty costs are taken only from the latest of the effective records and then applied throughout the penalty window. In this example, the one-time and per interval penalty costs are taken exclusively from the record with an **EffectiveDate** of 08-15-13 as shown in the figure.

Figure 47.22 Date rule with "LastInterval" accumulation



The **PenaltyAccumulationRule** field also contains a "Cumulative" option which can be used to specify that the penalty costs from each effective record are applied only within the portion of the penalty window that overlaps their effective dates. With this option, multiple one-time costs and variable per-interval costs are possible. In this example, the penalty costs would be combined from the records effective on 07-01-13 and 08-15-13 as shown in the following figure.

Figure 47.23 Date rule with "Cumulative" accumulation



Interval-based penalty schedules

Projects and tasks each get their penalty schedule from the value they reference in the **PenaltySchedule** table. If the **ProjectType.Penalty Rule** or **TaskType.PenaltyRule** field is set to "Interval", then the effective penalty schedule is made up of those records in the **PenaltyScheduleByInterval** table that reference the same **PenaltySchedule** value as the **Task** or **Project** record. This setting is useful for situations where penalties are not attached to specific calendar dates but instead are relative to the project or task dates, and schedules defined in the **PenaltyScheduleByInterval** table might typically be reusable from project to project.

Each record in the **PenaltyScheduleByInterval** table identifies the penalty schedule it belongs to, along with a one-time cost (single lump-sum penalty) and a per interval cost (applied on each applicable penalty calendar marker). **PenaltyScheduleByInterval** records also contain an **Interval** field that specifies the number of penalty calendar intervals from the **PenaltyDate** for which the costs defined on the record become effective. For example, an **Interval** value of 2 indicates the costs are effective beginning on the second penalty calendar interval after the **PenaltyDate**. The penalty costs remain effective until the next **Interval** defined on a record for that same penalty schedule.

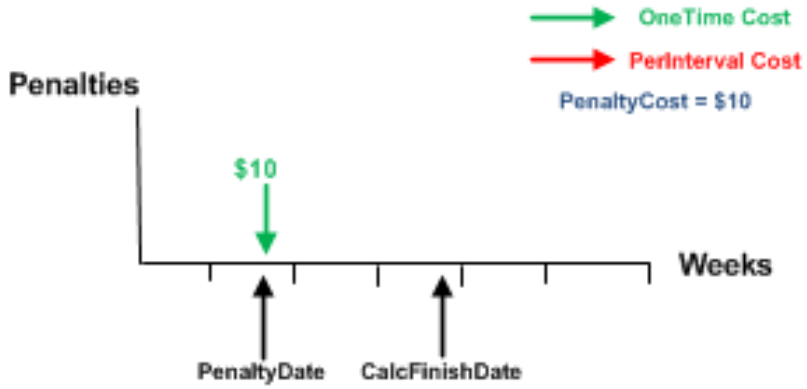
For example, suppose a series of **PenaltyScheduleByInterval** records that reference a given **PenaltySchedule** value, PS2, as shown in the following table.

Table 47.11: Sample PenaltyScheduleByInterval values

Schedule	Interval	OneTimeCost	PerIntervalCost
PS2	0	10	0
PS2	3	25	2
PS2	6	40	5
PS2	10	100	6

Further assume a task using a weekly penalty calendar and the PS2 penalty schedule, along with a penalty date and calculated finish date as shown in the following figure. In this case, because **CalcFinishDate** is 2 penalty calendar intervals after **PenaltyDate**, only the **PenaltyScheduleByInterval** record with an **Interval** of 0 is effective within the penalty window. Therefore, this record's **OneTimeCost** and **PerIntervalCosts** are used and reflected in the total penalty costs charged to the task as seen in the figure.

Figure 47.24 Interval rule with one effective entry

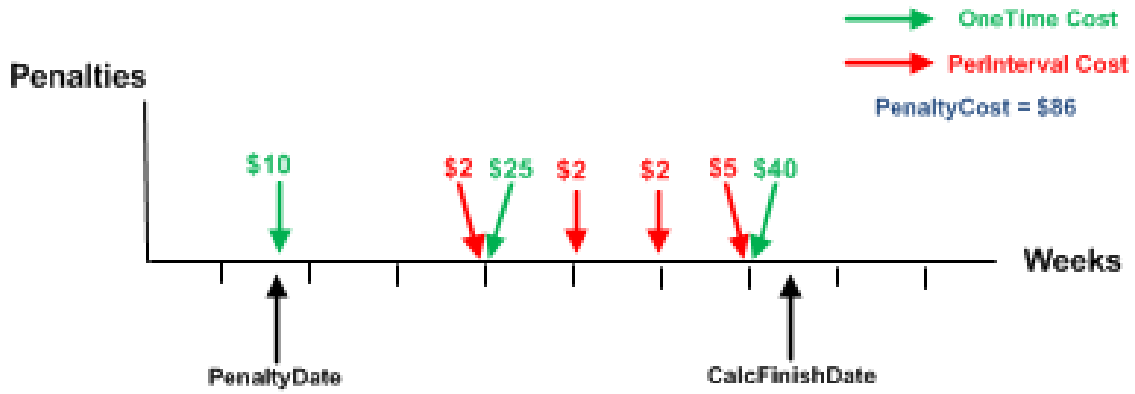


Sometimes, there might be multiple **PenaltyScheduleByInterval** records effective within a given penalty window. For example, suppose the task shown previously has a later calculated finish date as shown in the following figure. In this case, because there are 6 penalty calendar markers, between **PenaltyDate** and **CalcFinishDate**, the records with **Interval** values 0, 3, and 6 are all effective within the penalty window. In such cases, the setting on the **TaskType.PenaltyAccumulationRule** field determines how the penalty costs on these records are used (a similar field is found on the **ProjectType** table for project-level penalties). By default, the **PenaltyAccumulationRule** field is set to "LastInterval" which specifies that penalty costs are taken only from the latest of the effective records and then applied throughout the penalty window. In this example, the **OneTime** and **PerInterval** penalty costs are taken exclusively from the record with an **Interval** value of 6 as shown in the figure.

Figure 47.25 Interval rule with "LastInterval" accumulation

The **PenaltyAccumulationRule** field also contains a "Cumulative" option which can be used to specify that the penalty costs from each effective record are applied only within the portion of the penalty window that overlaps their effective dates. With this option, multiple one-time costs and variable per-interval costs are possible. In this example, the penalty costs would be combined from the records effective on Interval 0, 3, and 6 as shown in the following figure.

Figure 47.26 "Interval rule with "Cumulative" accumulation

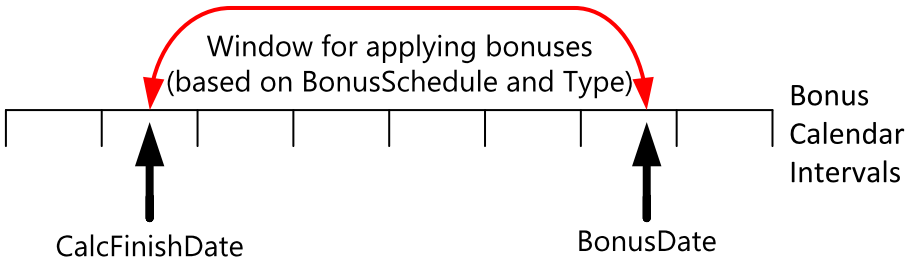


Bonuses

Projects or tasks that finish in advance of planned dates might sometimes be eligible to receive bonus payments as agreed upon between a company and its customers. These can be one-time bonuses paid as long as a project or task finishes on or before a specified date and/or recurring bonus payments made for each interval in advance of a specified date that a project or task finishes. A calculated **Bonus** field is found on both the **Project** and **Task** tables and reports bonuses earned at the project and task-levels respectively. As well, a calculated **CumulativeBonus** field is available for use in reporting rolled up bonus values. For projects, this field sums the project level bonus together with bonuses from all of its tasks. For tasks, this field sums the task's bonus with that from all of its subtasks (for example, this might be useful to report on summary tasks).

The **Project** and **Task** tables each contain a **BonusDate** field used to define the end of a window within which bonuses can be earned. The beginning of this bonus window is determined by the project or task's **CalcFinishDate** field. In other words, bonuses are only earned when a given project or task has a calculated finish date that is on or earlier than its specified bonus date as shown in the following illustration.

Figure 47.27 Bonus window



Within the bonus window, the dates on which bonuses can be earned is determined by the markers on the bonus calendar (for example, bonuses might accrue daily, weekly, or monthly). On projects, the bonus calendar is set by the **ProjectType.BonusCalendar** reference and on tasks it is set by the **TaskType.BonusCalendar** reference.

The effective bonus values that actually get applied on these dates is then determined by a payment schedule set through the **BonusSchedule** reference on the **Project** and **Task** tables (as well as related **ProjectType** or **TaskType** settings). RapidResponse supports both date-based and interval-based bonus schedules.

Date-based bonus schedules

Projects and tasks each derive their bonus schedule from the value they reference in the **BonusSchedule** table. If the **ProjectType.BonusRule** or **TaskType.BonusRule** field is set to "Date", then the project or task's bonus schedule is made up of those records in the **BonusScheduleByDate** table that reference the same **BonusSchedule** value as the **Task** or **Project** record. This setting is useful for situations where bonuses are attached to specific calendar dates, and schedules defined in the **BonusScheduleByDate** table are thereby not typically reusable from project to project

Each record in the **BonusScheduleByDate** table identifies the bonus schedule it belongs to, along with a one-time bonus (single lump-sum bonus) and a per interval cost (recurring bonus payable on each bonus calendar interval). **BonusScheduleByDate** records also contain an **EffectiveOutDate** field that specifies the last date that bonus values defined on the record are effective. The bonus values on the record are assumed to have become effective after the immediately preceding **EffectiveOutDate** defined on a record for the same bonus schedule.

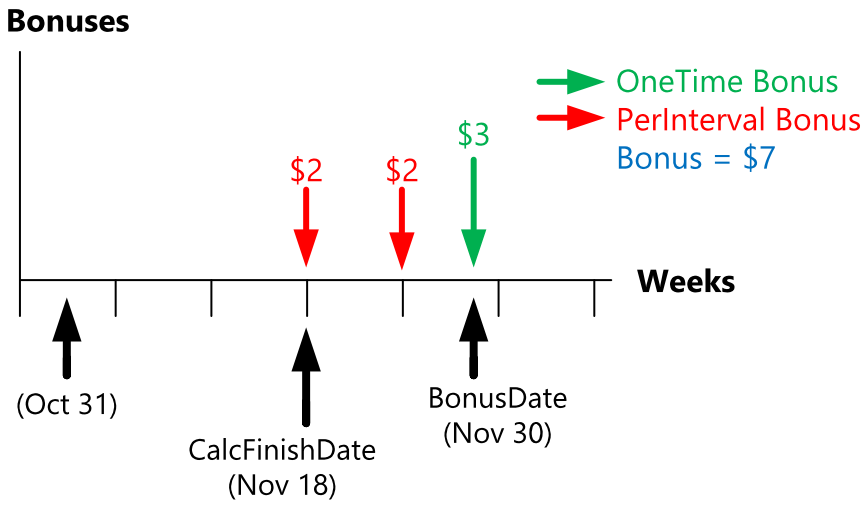
For example, suppose a series of **BonusScheduleByDate** records that reference a given **BonusSchedule** value, Bonus1, as shown in the following table.

Table 47.12: Sample BonusScheduleByDate values

Schedule	EffectiveOutDate	OneTimeBonus	PerIntervalBonus
Bonus1	09-30-13	7	6
Bonus1	10-31-13	5	4
Bonus1	11-30-13	3	2

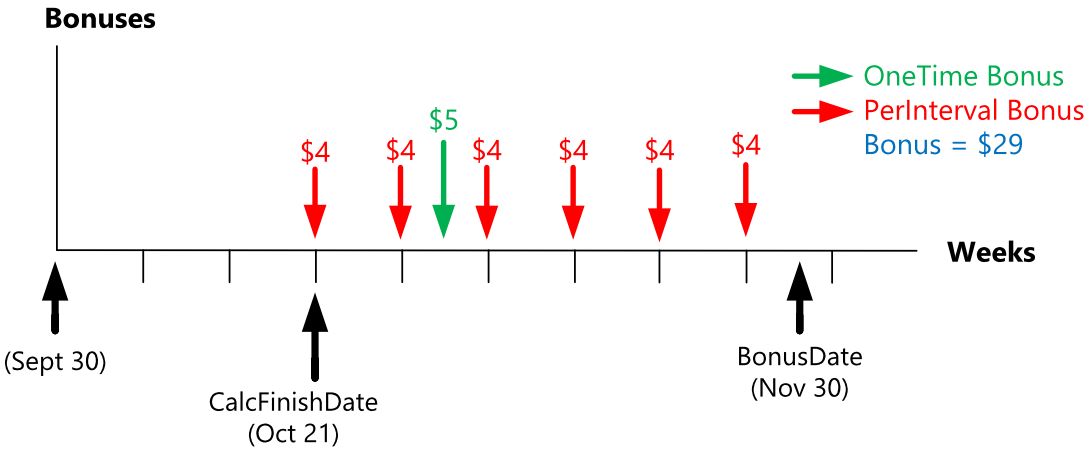
Further assume a task using a weekly bonus calendar and the **Bonus1** bonus schedule, along with a bonus date and calculated finish date for the task as shown in the following figure. In this case, only the **BonusScheduleByDate** record with the **EffectiveOutDate** of 11-30-13 is effective within the bonus window defined by **CalcFinishDate** and **BonusDate**. Therefore, this record's **OneTimeCost** and **PerIntervalCosts** are used to calculate the total bonus earned by the task.

Figure 47.28 Date rule with one effective entry



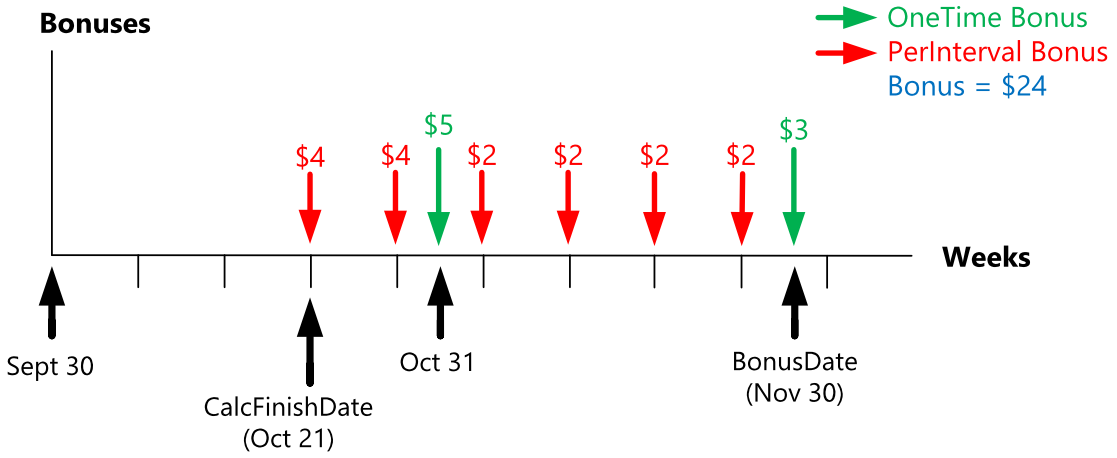
Sometimes, there might be multiple **BonusScheduleByDate** records effective within a given bonus window. For example, suppose the task shown previously has an earlier calculated finish date as shown in the following figure. In this case, the record with an **EffectiveOutDate** of 11-30-13 and the record with an **EffectiveOutDate** of 10-31-13 are both effective within the bonus calendar window. In such cases, the setting on the **TaskType.BonusAccumulationRule** field determines how the bonus values from these records are used (a similar field is found on the **ProjectType** table for project-level bonuses). By default, the **BonusAccumulationRule** field is set to "FirstInterval" which specifies that bonus values are taken only from the earliest of the effective **BonusScheduleByDate** records and then applied throughout the bonus window. In this example, the bonus is calculated exclusively using only one-time and per interval bonuses defined on the record with an **EffectiveOutDate** of 10-31-13 as shown in the following figure.

Figure 47.29 Date rule with "FirstInterval" accumulation



The **BonusAccumulationRule** field also contains a "Cumulative" option which can be used to specify that the bonus values from each effective **BonusScheduleByDate** record are used but applied only during the portion of the bonus window for which they are effective. With this option, multiple one-time bonuses and variable per-interval bonuses are possible. In this example, the bonus is calculated using values from the records with EffectiveOutDate values of 10-31-13 and 11-30-13 as shown in the following figure.

Figure 47.30 Date rule "Cumulative" accumulation



Interval-based bonus schedules

Projects and tasks each derive their bonus schedule from the value they reference in the **BonusSchedule** table. If the **ProjectType.BonusRule** or **TaskType.BonusRule** field is set to "Interval", then the project or task's bonus schedule is made up of those records in the **BonusScheduleByInterval** table that reference the same **BonusSchedule** value as the **Task** or **Project** record. This setting is useful for situations where bonuses are not attached to specific calendar dates but instead are relative to the project or tasks dates, and schedules defined in the **BonusScheduleByInterval** table might typically be reusable from project to project.

Each record in the **BonusScheduleByInterval** table identifies the bonus schedule it belongs to, along with a one-time bonus (single lump-sum bonus) and a per interval cost (recurring bonus payable on each bonus calendar interval).

The **BonusScheduleByInterval** table also contains an **Interval** field that is used to determine the last date on which bonus values defined on the record are effective. To determine that last effective date, a number of bonus calendar intervals, as specified by the value in the **Interval** field, is subtracted from the project or task's **BonusDate**. The bonus values on the record are assumed to have become effective after the last effective date defined by the preceding **BonusScheduleByInterval** record (that is, the record on the same bonus schedule with the next highest **Interval** value).

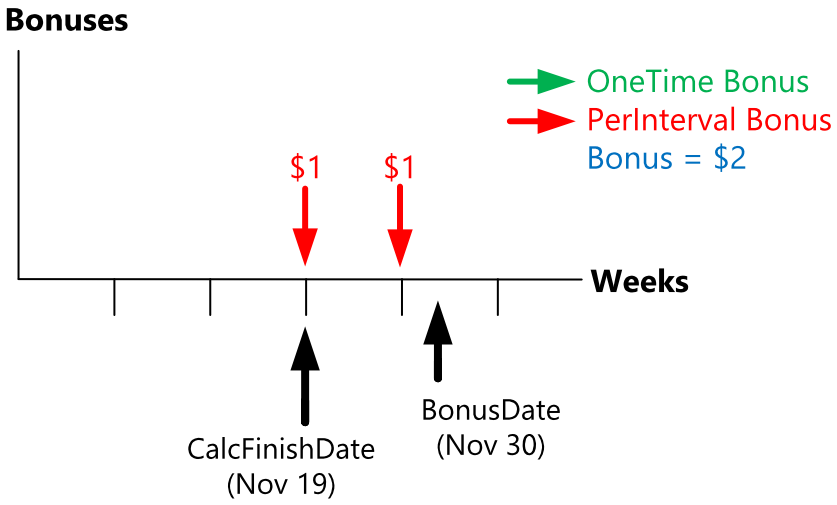
For example, suppose a series of **BonusScheduleByInterval** records that reference a given **BonusSchedule** value, Bonus2, as shown in the following table.

Table 47.13: Sample BonusScheduleByInterval values

Schedule	Interval	OneTimeBonus	PerIntervalBonus
Bonus2	0	0	1
Bonus2	2	3	3
Bonus2	4	6	5
Bonus2	8	9	7

Further assume a task using a weekly bonus calendar and the **Bonus2** bonus schedule, along with a bonus date and calculated finish date for the task as shown in the following figure. In this case, only the **BonusScheduleByInterval** record with the **Interval** value of 0 is effective within the bonus window defined by **CalcFinishDate** and **BonusDate**. Therefore, this record's **OneTimeCost** and **PerIntervalCosts** values are used to calculate the total bonus earned by the task.

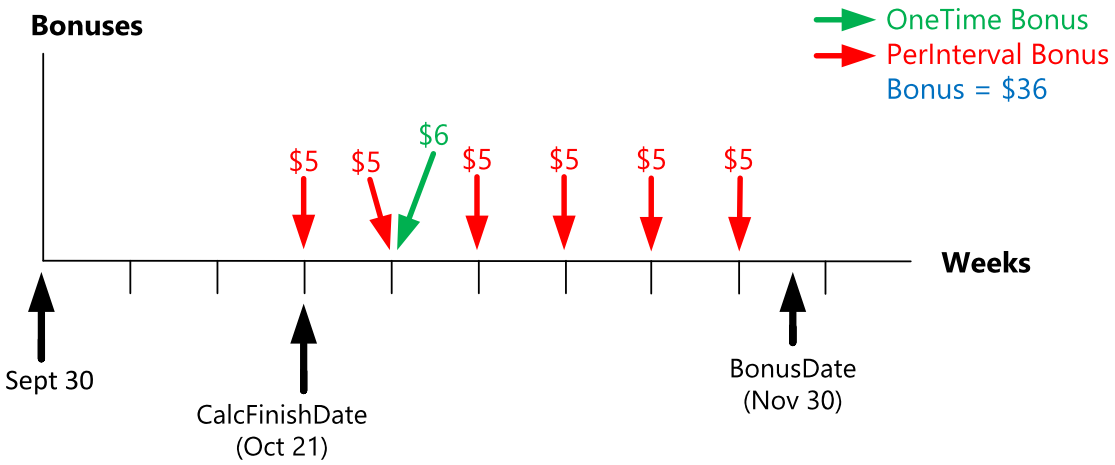
Figure 47.31 Interval rule with one effective entry



Sometimes, there might be multiple **BonusScheduleByInterval** records effective within a given bonus window. For example, suppose the task shown previously has an earlier calculated finish date as shown in the following figure. In this case, the records with Interval values of 0, 2, and 4 are all effective within the bonus calendar window. In such cases, the setting on the **TaskType.BonusAccumulationRule** field determines how the bonus values from these records are used (a similar field is found on the **ProjectType** table for project-level bonuses).

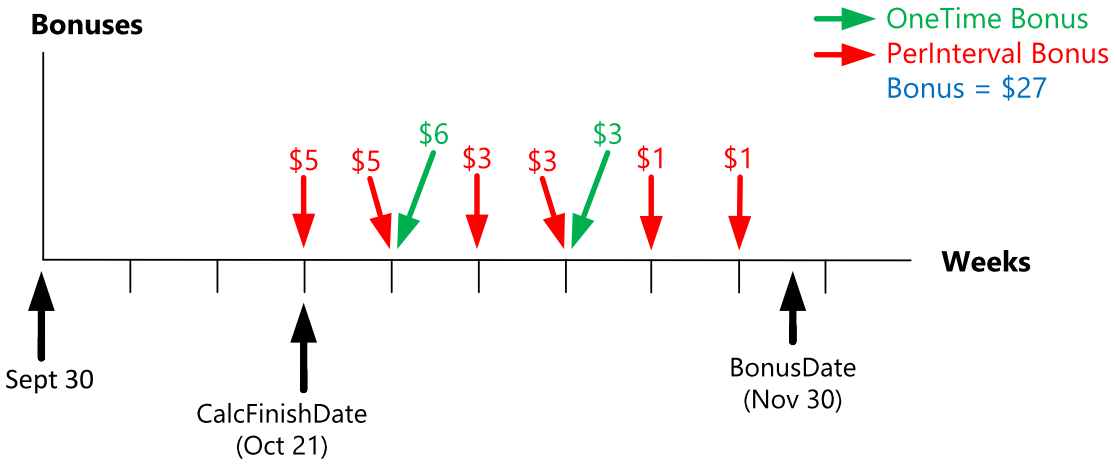
By default, the **BonusAccumulationRule** field is set to "FirstInterval". This option specifies that bonus values are taken only from the **BonusScheduleByInterval** record that has the largest **Interval** value of the records effective within the bonus window (the earliest effective of those records), and then applied through the entire bonus window. In this example, the bonus is calculated exclusively using only the one-time and per interval bonuses defined on the record with an **Interval** value of 4 as shown in the following figure.

Figure 47.32 Interval rule with "FirstInterval" accumulation



The **BonusAccumulationRule** field also contains a "Cumulative" option which can be used to specify that the bonus values from each effective **BonusScheduleByInterval** record are used but applied only during the portion of the bonus window for which they are effective. With this option, multiple one-time bonuses and variable per-interval bonuses are possible. In this example, the bonus is calculated using values from the records with **Interval** values of 4, 2, 0 as shown in the following figure.

Figure 47.33 Interval rule with "Cumulative" accumulation



CHAPTER 48: Configuring analytic calculations using global settings

Control tables in the RapidResponse data model contain rules and values that determine how RapidResponse performs calculations that produce values stored in calculated tables and fields. For more information about control tables, see "Control tables" on page 669.

In addition to control tables, RapidResponse system administrators can use global settings to configure how some analytics are calculated.

In this chapter, you'll learn about:

Calculating TimeUnit records **2203**
Limiting the number of planned orders **2204**
Configuring buckets for Inventory Analysis tables **2205**
Defining a periods of coverage calendar **2206**
Configuring AbsolutePrePlanLimit usage **2206**
Enabling automatic maintenance of TransactionSequence values **2207**
Configuring task finish date calculations **2208**
Defining the And operator for feature rules **2208**

Note: Most procedures described in this section require RapidResponse system administrator permissions. Therefore, you might need to ask your RapidResponse administrator to make modifications for you.

Calculating TimeUnit records

RapidResponse can be configured to create additional Activity and CTPActivity table records based on time buckets. These types of records are called TimeUnit records. You can use TimeUnit records with advanced query operations to view bucketed supply and demand information.

By default, RapidResponse creates Activity records when netting occurs. For example, assume a requirement for 100 units of part AC-0101 exists. By performing netting, RapidResponse satisfies this requirement by recommending a planned order for 100 units. After RapidResponse performs this netting, it creates two Activity records. The first Activity record shows a requirement of 100 units of part AC-0101. The second Activity record shows a planned order for 100 units of the part. For information about netting, see "Netting basics" on page 1331.

In addition to transaction-based Activity and CTPActivity records, RapidResponse can be configured to create TimeUnit records. TimeUnit records can be configured to be created before RunDate, after RunDate, or both before and after RunDate.

Note: If you are using the On-Premises version of RapidResponse, system administrators can configure this setting. If you are using the On-Demand RapidResponse service, this setting can be configured by the Master Administrator at Kinaxis.

To configure RapidResponse to create TimeUnit records

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Analytics** category.
3. Do either or both of the following:
   - In the **Time Unit bucket days after run date** box, specify the number of normal calendar days over which Activity and CTPActivity records are generated for each **Part.PlanningCalendars.TimeUnits** value after **Part.PlanningCalendars.RunDate.FirstDate**.
   - In the **Time Unit bucket days after run date** box, specify the number of normal calendar days over which Activity and CTPActivity records are generated for each **Part.PlanningCalendars.TimeUnits** value defined before **Part.PlanningCalendars.RunDate.FirstDate**
4. Click **OK**.
5. Click **Insert**.

   Any resulting changes to calculated results are visible to users after their next RapidResponse sign in. If you want to verify the impact of your change, sign out of and then back into RapidResponse.

Caution: Using TimeUnit records adds to RapidResponse memory requirements and processing time. As such, they should only be configured if required. By default, RapidResponse does not generate TimeUnit records for the Activity and CTPActivity tables.

Note: TimeUnit records are created for every unique model and pool combination that has transaction for a part (including parts not associated with any models or pools). For more information about how TimeUnit records are calculated, see "Activity table" on page 940.

Limiting the number of planned orders

By default, when the **OrderPolicy.MaximumUsage** field is set to **Use,** RapidResponse limits the number of planned orders per part in each planning period to 100 (typically, a planning period is a day). If RapidResponse reaches the planned order limit, the last order generated includes all the remaining unsatisfied demand, and if necessary this last order quantity will override the **PartSource.MaximumQty setting.**

Note: If you are using the On-Premises version of RapidResponse, system administrators can configure this setting. If you are using the On-Demand RapidResponse service, this setting can be configured by the Master Administrator at Kinaxis.

To change the planned order limit

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Analytics** category.
3. In the **Maximum planned orders** box, specify the maximum number of planned orders that can be created per part in a given planning period.
4. Click **OK**.

   Any resulting changes to calculated results are visible to users after RapidResponse next generates calculated results. With On-Premises RapidResponse, you can also force the change to take effect by stopping and restarting RapidResponse Server.

Configuring buckets for Inventory Analysis tables

The Inventory Analysis analytic calculates grouped inventories, work in process quantities, and supply and demand data grouped together by part. Output from this analytic is stored in the InventoryAnalysis and InventoryCTPAnalysis tables.

The data in the Inventory Analysis tables is stored in date buckets.

By default, daily buckets are used, however you can specify a different calendar to define bucket size. For example, you might want the data grouped into monthly buckets. You can also specify the number of date buckets generated before and after the MRPDate. By default, 0 buckets are generated before MRPDate, and 12 buckets are generated after MRPDate. As well any data before the earliest defined bucket is grouped into a "Past" bucket, and any data after the latest defined bucket is grouped into a "Future" bucket.

You can also specify Inventory Analysis buckets that are specific to a workbook. For more information, see "Specify workbook-specific Inventory Analysis buckets" on page 2225.

For information about the tables containing Inventory Analysis data, see "InventoryAnalysis table" on page 1078 and "InventoryCTPAnalysis table" on page 1081.

For information about MRPDate, see "Date constants" on page 1303.

To set default bucketing for Inventory Analysis data

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Inventory Analysis** category.
3. Do one or more of the following:
   - In the **Calendar** list, select the calendar used to define bucket size.
   - In the **Buckets after anchor date** box, specify the number of buckets to be generated after MRPDate.
   - In the **Buckets before anchor date** box, specify the number of buckets to be generated before the MRPDate.
4. Click **OK**.

   Any resulting changes to calculated results are visible to users after their next RapidResponse sign in. If you want to verify the impact of you change, sign out of and then back into RapidResponse.

Defining a periods of coverage calendar

Periods of coverage calculations in RapidResponse determine the number of future periods of demand that can be covered by the excess supply available at the end of a given period. The results of these calculations are stored in the **PeriodsOfCoverage** and **PeriodsOfCTPCoverage** tables, and also reported in the Periods of Coverage rows in the Planning Sheet workbook.

The date interval used in periods of coverage calculations is set by the calendar specified in the **Periods of coverage calendar** global setting. By default, this setting points to a blank (' ') calendar which in most RapidResponse installations corresponds to an "Everyday" calendar. You can then change this setting to reference the calendar whose periods of coverage you want to report. Typically, this would be a weekly or monthly calendar.

To define a periods of coverage calendar

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Analytics** category.
3. In the **Periods of coverage calendar** list, select the calendar whose intervals are used in calculating periods of coverage.
4. Click **OK**.

   Any resulting changes to calculated results are visible to users after their next RapidResponse sign in. If you want to verify the impact of you change, sign out of and then back into RapidResponse.

Note: For more information about periods of coverage calculations, see "PeriodsOfCoverage table" on page 1144 and "PeriodsOfCTPCoverage table" on page 1145.

Configuring AbsolutePrePlanLimit usage

The **PartSource.AbsolutePrePlanLimit** field sets the maximum number of calendar days before its normal constraint allocation date that constraint can be planned or allocated to supplies using a given part source. Negative values in this field are treated specially and mean that constraint preplanning should be unlimited for supplies using the part source.

A zero value in the AbsolutePrePlanLimit field is also treated specially, but its meaning is configurable. Depending on how you have configured it, an AbsolutePrePlanLimit value of zero can mean unlimited constraint preplanning is allowed for supplies using the part source (this is identical to providing a negative value), or an AbsolutePrePlanLimit value of zero can mean preplanning is disabled for supplies using the part source.

To configure the interpretation of zero values in the PartSource.AbsolutePrePlanLimit field

1. In the **Administration** pane, under **System Settings**, click **Global Settings.**
2. Expand the **Analytics** category.
3. In the **Constraint preplanning** list, select one of the following:

- **No—**constraint preplanning cannot be disabled. If a value of zero (0) is specified in the **PartSource.AbsolutePrePlanLimit** field, there is no limit to how early constraints can be preplanned for supplies using the part source.
- **Yes—**constraint preplanning can be disabled. If a value of zero (0) is specified in the **PartSource.AbsolutePrePlanLimit** field, constraint preplanning is not allowed for supplies using the part source. This is the recommended setting.

4. Click **OK**.

   Any resulting changes to calculated results are visible to users after their next RapidResponse sign in. If you want to verify the impact of you change, sign out of and then back into RapidResponse.

Enabling automatic maintenance of TransactionSequence values

The TransactionSequence field on the IndependentDemand and ScheduledReceipt tables is used as part of the allocation sort criteria for orders that share the same priority.

RapidResponse can automatically modify the TransactionSequence field on the IndependentDemand and ScheduledReceipt tables when an order has its priority changed. That is, if a given IndependentDemand or ScheduledReceipt record has its OrderPriority value changed, the TransactionSequence field for that record can then be modified and set to one number higher than the current highest TransactionSequence value on that table. This behavior can be enabled by setting the appropriate Configuration record as described in the following procedure.

To control automatic updating of TransactionSequence values

1. In the **Administration** pane, under **System Settings**, click **Global Settings.**
2. Expand the **Analytics** category.
3. In the **Automatically update TransactionSequence** list, select one of the following:
   - **No**—when an IndependentDemand or ScheduledReceipt record has its OrderPriority value modified, there is no impact on that record's TransactionSequence value. This is the recommended setting.
   - **Yes**—when an IndependentDemand or ScheduledReceipt record has its OrderPriority value modified, that record's TransactionSequence value is set to one number higher than the highest TransactionSequence value in the table.
4. Click **OK**.

   The change takes effect immediately.

Note: The ability to automatically maintain TransactionSequence values as described in this topic will be removed in a future version of RapidResponse. If your organization relies on the automatic maintenance of TransactionSequence values, please contact Kinaxis Customer Support.

Configuring task finish date calculations

If leveraging integrated project management in RapidResponse, how a task's calculated finish date is reported in RapidResponse can be configured. The finish dates can either be set to end of the last day of each task, or to the beginning of the next day after the last day of the task. The end of the last day reporting method is the default setting used in RapidResponse, and corresponds to the behavior found in most widely used project management applications.

To specify how a task's finish date is reported

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Analytics** category.
3. In the **Project management finish** dates list, select one of the following:
   - **End of day**
   - **Beginning of next day**

Defining the And operator for feature rules

Feature BOM capabilities in RapidResponse allow for supplies of configurable end items to be built using components that vary based on the particular features associated with the demands being satisfied. To indicate which features of an assembly that a particular component pertains to, a rule is can be specified in the **BillOfMaterial.FeatureRule** field.

This rule typically consists of feature names along with some combination of and, or, and not operators. By default, an ampersand character (&) is used to indicate "and", while a comma (,) is always used to indicate "or", and an exclamation (!) always used to indicate "not". However, the character used to indicate "and" is configurable if necessary. For example, if the ampersand character is expected to be used in some of your company's feature code names, then a different character should be chosen to represent the and operator.

To specify the And operator for feature rules

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Client** category.
3. In the **AND operator for features** box, type any single character. Ensure this is a character that is not expected to appear in any actual feature codes.

Note: The specified character is also used to "and" features together when working with the standard **Orders by Features** workbook included with RapidResponse. This workbook is used to report orders associated with a specific feature or features. For more information, see the *RapidResponse Applications Guide*.

CHAPTER 49: Configuring analytic calculations using workbook parameters

Analytics can have their calculated results configured by specifying values for analytic parameters. Values for these parameters are specified using a workbook variable. The value specified for the variable is provided for the parameter when the analytic runs (such as, when the workbook is opened or data settings are modified). This allows for certain analytic calculations to be configured at the individual workbook level.

In this chapter, you'll learn about:

Include inactive IndependentDemands in the Activity and CTPActivity tables **2210**
Include inactive ScheduledReceipts in the Activity and CTPActivity tables **2211**
Include inactive OnHand in the Activity and CTPActivity tables **2212**
Duplicate supplies and demands in the Activity and CTPActivity tables **2212**
Merge PartSources and BOMs in FlatBillDown and FlatBillUp calculations **2215**
Include non-primary part sources in FlatBillDown and FlatBillUp calculations **2217**
Include substitute BOM components in FlatBillDown and FlatBillUp calculations **2218**
Include global substitutes and their components in FlatBillDown calculations **2219**
Explode BOMs for "NoOrders" parts in FlatBillDown calculations **2221**
Apply a parameter date to FlatBillDown calculations **2222**
Limiting FlatBillDown and FlatBillUp calculations by model **2224**
Specify workbook-specific Inventory Analysis buckets **2225**
Specify a reporting horizon for WhereConsumed records **2227**
Specify a reporting horizon for WhereConsumedForForecast records **2229**
Specify a reporting horizon for WhereConsumedForSupply records **2230**
Specify a reporting horizon for WhereConsumedToHighVolumeOrder records **2232**
Enabling feature-based queries on IndependentDemandByFeature records **2234**
Specifying the number of levels used in WhereConsumedForSupply calculations **2235**
Calculate general disaggregation rates by forecast category **2236**

Note: For more information about workbook variables, see the *RapidResponse Resource Authoring Guide*.

Include inactive IndependentDemands in the Activity and CTPActivity tables

By default, the **Activity** and **CTPActivity** tables do not include inactive **IndependentDemand** records. Inactive independent demands are defined as those where the associated part has part sources and a **Type.ProcessingRule** other than "Ignore", and the demand has a **DueDate** other than 'Undefined' and also meets at least one of the following conditions:

- **Status.ProcessingRule** is "Ignore"
- **Order.Type.ProcessingRule** is "Ignore"
- **Quantity** and **ShippedQuantity** are both <= 0

In some cases, you might want reports that include inactive **IndependentDemand** records along with supply and netting requirements in the **Activity** and **CTPActivity** tables. This can be done using the **RR_Analytics_Activity_IncludeInactiveIDs analytics parameter.** This parameter is set at the workbook level and applies to any **Activity** or **CTPActivity** table worksheets within a given workbook.If the variable's value is "false", inactive **IndependentDemand** records are not included. If the value is "true", inactive **IndependentDemand** records are included.

To include inactive IndependentDemands in Activity table records

1. Open a workbook which contains, or will contain, **Activity** and/or **CTPActivity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and the click **Boolean Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_IncludeInactiveIDs**.
6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.
8. In the **Initial value** column, specify which value is the variable's default.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_Activity_IncludeInactiveIDs`

Include inactive ScheduledReceipts in the Activity and CTPActivity tables

By default, the **Activity** and **CTPActivity** tables do not include inactive **ScheduledReceipt** records. Inactive scheduled receipts are defined as those where the associated part has part sources and a **Type.ProcessingRule** other than "Ignore", and the supply has a **DueDate** other than 'Undefined' and also meets at least one of the following conditions:

- **Status.IgnoreSupply** is "Y"
- **Status.State** is "Historical"
- **Order.Type.ProcessingRule** = '"Ignore"
- **Quantity** = 0

In some cases, you might want reports that include inactive **ScheduledReceipt** records along with supply and netting requirements in the **Activity** and **CTPActivity** tables. This can be done using the **RR_Analytics_Activity_IncludeInactiveSRs analytics parameter.** This parameter is set at the workbook level and applies to any **Activity** or **CTPActivity** table worksheets within a given workbook.If the variable's value is false, inactive **ScheduledReceipt** records are not included. If the value is true, inactive **ScheduledReceipt** records are included.

To include inactive ScheduledReceipts in Activity records

1. Open a workbook which contains, or will contain, **Activity** and/or **CTPActivity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and the click **Boolean Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_IncludeInactiveSRs**.
6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.
8. In the **Initial value** column, specify which value is the variable's default.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_Activity_IncludeInactiveSRs`

Include inactive OnHand in the Activity and CTPActivity tables

By default, the **Activity** and **CTPActivity** tables do not include inactive **OnHand** records. Inactive onhands are defined as those where **Type.ProcessingRule** is "Non-Nettable".

In some cases, you might want reports that include inactive **OnHand** records along with supply and netting requirements in the **Activity** and **CTPActivity** tables. This can be done using the **RR_Analytics_Activity_IncludeInactiveOHs analytics parameter.** This parameter is set at the workbook level and applies to any **Activity** or **CTPActivity** table worksheets within a given workbook.If the variable's value is false, inactive **OnHand** records are not included. If the value is true, inactive **OnHand** records are included.

To include inactive OnHand in Activity records

1. Open a workbook which contains, or will contain, **OnHand** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and the click **Boolean Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_IncludeInactiveOHs**.
6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.
8. In the **Initial value** column, specify which value is the variable's default.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_Activity_IncludeInactiveOHs`

Duplicate supplies and demands in the Activity and CTPActivity tables

By default, each active scheduled receipt, planned order, independent demand, and dependent demand is reported once in each of the **Activity** and **CTPActivity** tables. If necessary, each of these records can be duplicated up to ten times in the **Activity** and **CTPActivity** tables. For example, there might be a requirement to have the same independent demand reported on multiple dates (for example, once on its due date and once on its available date).

This duplication can be done using four separate analytic parameters. These parameters are set at the workbook level and applies to any **Activity** or **CTPActivity** table worksheets within a given workbook.

Each parameter accepts quantity values between 0 and 10 indicating the number of times a given demand or supply should be duplicated in the Activity and CTPActivity tables. For example, "0" means records aren't duplicated (only one record per entry), and "4" means records are duplicated four times (five records per entry). The following parameters are available:

- **RR_Analytics_Activity_DuplicateSRs**
- **RR_Analytics_Activity_DuplicateCPOs**
- **RR_Analytics_Activity_DuplicateIDs**.
- **RR_Analytics_Activity_DuplicateDDs**

Note: Duplicating demand and supply records can also be accomplished through the use of composite worksheets. For information about composite worksheets, see the *RapidResponse Resource Authoring Guide*.

To duplicate ScheduledReceipt records in the Activity tables

1. Open a workbook which contains, or will contain, **Activity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_DuplicateSRs**.
6. In the **Minimum** box, type **0**.
7. In the **Maximum** box, type **10**.
8. In the **Initial value** column, specify a default value. For example, if set to "3" then each active **ScheduledReceipt** record will be reported four times in the **Activity** and **CTPActivity** table.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_Activity_DuplicateSRs`

To duplicate PlannedOrders records in the Activity tables

1. Open a workbook which contains, or will contain, **Activity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_DuplicateCPOs**.
6. In the **Minimum** box, type **0**.
7. In the **Maximum** box, type **10**.
8. In the **Initial value** column, specify a default value. For example, if set to "1" then each active **PlannedOrder** record will be reported twice in the **Activity** and **CTPActivity** tables.

9. If you want the variable to be modifiable by workbook users, do the following:
    - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
    - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
    - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
      `$RR_Analytics_Activity_DuplicateCPOs`

To duplicate IndependentDemand records in the Activity tables

1. Open a workbook which contains, or will contain, **Activity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_DuplicateIDs**.
6. In the **Minimum** box, type **0**.
7. In the **Maximum** box, type **10**.
8. In the **Initial value** column, specify a default value. For example, if set to "3" then each active **IndependentDemand** record will be reported four times in the **Activity** and **CTPActivity** tables.
9. If you want the variable to be modifiable by workbook users, do the following:
    - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
    - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
    - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
      `$RR_Analytics_Activity_DuplicateIDs`

To duplicate IndependentDemand records in the Activity tables

1. Open a workbook which contains, or will contain, **Activity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_DuplicateIDs**.
6. In the **Minimum** box, type **0**.
7. In the **Maximum** box, type **10**.
8. In the **Initial value** column, specify a default value. For example, if set to "3" then each active **IndependentDemand** record will be reported four times in the **Activity** and **CTPActivity** tables.
9. If you want the variable to be modifiable by workbook users, do the following:
    - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
    - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.

- In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
  `$RR_Analytics_Activity_DuplicateIDs`

To duplicate DependentDemand records in the Activity tables

1. Open a workbook which contains, or will contain, **Activity** table worksheets.
2. On the toolbar, click **Workbook Properties**, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **Name** box, type **RR_Analytics_Activity_DuplicateDDs**.
6. In the **Minimum** box, type **0**.
7. In the **Maximum** box, type **10**.
8. In the **Initial value** column, specify a default value. For example, if set to "2" then each active **DependentDemand** record will be reported four times in the **Activity** and **CTPActivity** tables.
9. If you want the variable to be modifiable by workbook users, do the following:
10. Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
    - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
    - In the worksheets based on the **Activity** and/or **CTPActivity** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
      `$RR_Analytics_Activity_DuplicateDDs`

Merge PartSources and BOMs in FlatBillDown and FlatBillUp calculations

When FlatBillUp or FlatBillDown records are generated, a result is calculated for each PartSource and BillOfMaterial record that applies to a part. In some cases, a part might have multiple PartSources or BillOfMaterials that are identical except for their effective dates. For example, if a new PartSource record is generated for a part every month, over time that part will have multiple PartSources, with the same value for each field except the effective date.

Instead of generating records for each identical PartSource or BillOfMaterial record, you can merge the identical records using a parameter on the FlatBillUp and FlatBillDown analytics. This generates the FlatBillUp or FlatBillDown record only once for each PartSource and BillOfMaterial record. When identical PartSource and BillOfMaterial records are merged, the earliest effective date is used as the effective date, and the effectivity period is extended to the latest effective date. This option is set at the workbook level and therefore applies to all worksheets that are based on the FlatBillUp or FlatBillDown table within that workbook.

This option is specified using a Boolean variable. If the variable's value is false, identical PartSource and BillOfMaterial records are not merged, and a result is calculated for each record. If the value is true, identical PartSource and BillOfMaterial records are merged, and only one result is calculated.

PartSource records are considered identical if the values in the following fields are the same.

- EffectivePriority
- Yield
- CoProductYield
- Type
- Source
- TransferPart
- BOMAlternate

BillOfMaterial records are considered identical if the values in the following fields are the same.

- Component
- Type
- EffQtyPer
- LeadTimeOffset
- EffOptRatio
- SubstituteGroup
- Alternate
- FirstEffUnit
- LastEffUnit
- EffModel

To merge identical PartSource and BillOfMaterial records in FlatBillUp or FlatBillDown analytics

1. Open a workbook which contains, or will contain, worksheets that perform flat bill calculations.
2. On the toolbar, click **Workbook Properties**, and then click Workbook Properties.
3. Click the **Variables** tab.
4. Click **New**, and then click **Boolean Variable**.
5. In the **Name** box, type **RR_Analytics_FlatBill_MergeIdentical**.
6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.
8. In the **Initial value** column, specify which value is the variable's default.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **FlatBillDown** and/or **FlatBillUp** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_FlatBill_MergeIdentical`

Include non-primary part sources in FlatBillDown and FlatBillUp calculations

The calculated **FlatBillDown** and **FlatBillUp** tables report the results of explosions through BOM records. The **FlatBillDown** table reports all components used to make a given part, while **the FlatBillUp** table reports all assemblies where a given part is used.

By default, the records generated in the **FlatBillDown** and **FlatBillUp** tables only contain results applicable to the primary part source for a given part. This means if an assembly has multiple part sources, the **FlatBillDown** table only reports details of components required to build the assembly when sourced from the primary part source. It does not, however, report details of component requirements when sourcing the part from other potential sources. For example, if a non-primary part source is associated with an alternate BOM, the results of exploding that alternate BOM would not be reported in the **FlatBillDown** table. Similarly, if an assembly has global part substitutes, the **FlatBillDown** table does not report details of those substitute parts or their components.

If any non-primary part sources should be included when generating **FlatBillDown** and **FlatBillUp** records, the **RR_Analytics_FlatBill_MultiSourceExplode** analytic parameter can be used. This parameter can be defined as a workbook boolean variable, and applied to any worksheets based on the **FlatBillDown** or **FlatBillUp** tables within that workbook. Valid options for this parameter/variable are:

- **False**—Only results from the primary part source for a given part are reported. This setting corresponds to the behavior seen when not using the parameter/variable. For example, for a given assembly, the **FlatBillDown** table would report only those components required when the assembly was sourced strictly from the primary part source.
- **True**—Results from all eligible part sources for a given part are reported. For example, for a given assembly, the **FlatBillDown** table would include records for components found from all eligible part sources (and not just the primary). Note also that if the assembly had global substitutes, then those susbtitutes along with all of their components would be reported (if just the substitutes should be included but theircomponents excluded from FlatBillDown reports, then the **RR_Analytics_FlatBill_AlternatePartExplode** parameter can be used as discussed in "Include global substitutes and their components in FlatBillDown calculations" on page 2219).

Note: The primary part source used in **FlatBillDown** calculations considers date effectivity and is therefore not necessarily limited to the part source with **PrimaryPartSource** = Yes.

To enable multi-source explosion for FlatBillDown/FlatBillUp records

1. Open a workbook that contains, or will contain, worksheets based on either of the **FlatBillDown** or **FlatBillUp** tables.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then click **Boolean** variable.
5. In the **Name** box, type **RR_Analytics_FlatBill_MultiSourceExplode**.
6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.

8. In the **Initial value** column, specify which value is the variable's default. **False** indicates that only primary part sources are considered, while **True** indicates that all effective part sources are considered.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **FlatBillDown** and/or **FlatBillUp** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but should typically be hidden):
     `$RR_Analytics_FlatBill_MultiSourceExplode`

Include substitute BOM components in FlatBillDown and FlatBillUp calculations

The calculated **FlatBillDown** and **FlatBillUp** tables report the results of explosions through BOM records. The **FlatBillDown** table reports all components used to make a given part, while the **FlatBillUp** table reports all assemblies where a given part is used.

When BOM-level part substitution is enabled, substitute components are included in **FlatBillUp** and **FlatBillDown** calculations as long as new planned orders can be placed on those substitutes in order to satisfy demand from the primary component. This is based on **BillOfMaterial.Target** values. For example, if the primary component and its substitute components all have positive **Target** values, or all have zero (0) **Target** values, then the substitute components are included in **FlatBillUp** and **FlatBillDown** calculations.

However, if new planned orders are not allowed on substitute components in order to satisfy demand for the primary component, then those substitute components will not be included in **FlatBillUp** and **FlatBillDown** calculations. In other words, if the primary component has a positive **BillOfMaterial.Target** value and its substitute components all have zero (0) values in the **BillOfMaterial.Target** field, then the substitute components are excluded from **FlatBillUp** and **FlatBillDown** calculations.

For cases where substitute components should always be reported on **FlatBillDown** and **FlatBillUp** records if BOM-level part substitution is enabled, the **RR_Analytics_FlatBill_SubstituteComponents** analytic parameter can be used. This parameter is configured at the workbook level and can be applied to any worksheets based on the **FlatBillDown** or **FlatBillUp** tables within that workbook.

To include substitute components in FlatBillDown/FlatBillUp records

1. Open a workbook that contains, or will contain, worksheets based on either of the **FlatBillDown** or **FlatBillUp** tables.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then click **Boolean** variable.
5. In the **Name** box, type **RR_Analytics_FlatBill_SubstituteComponents**.

6. In the **False value** area, in the **Data value** box, type **false**.
7. In the **True value** area, in the **Data value** box, type **true**.
8. In the **Initial value** column, specify which value is the variable's default.
   - **True** indicates that substitute components are always considered in FlatBillDown/FlatBillUp calculations when bom-level part substitution is enabled.
   - **False** indicates that substitute components are considered in FlatBillDown/FlatUp calculations only when bom-level part substitution is enabled and new planned orders can be generated on those substitutes in response to demand from the primary component. This is equivalent to not using the parameter at all.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - To add a toggle button to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **FlatBillDown** and/or **FlatBillUp** tables, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_FlatBill_SubstituteComponents`

Include global substitutes and their components in FlatBillDown calculations

The calculated **FlatBillDown** table is based on explosion through **BillOfMaterial** records, and thereby reports all components used to make a given assembly. By default, this table only reports components used when sourcing from the assembly's primary part source. Thus components sourced from non-primary part sources are excluded, as are any of the assembly's substitute parts and their components.

If a workbook has the optional **RR_Analytics_FlatBill_MultiSourceExplode** parameter enabled as a workbook variable, then FlatBillDown reports in that workbook can include components from all of an assembly's eligible part sources. Setting this parameter to explode across all of an assembly's eligible part sources will also include details of any of that assembly's substitute parts and their components. For more information about this parameter,see "Include non-primary part sources in FlatBillDown and FlatBillUp calculations" on page 2217.

For specific control over whether an assembly's global substitutes and their components are included in worksheets based on the **FlatBillDown** table, the **RR_Analytics_FlatBill_AlternatePartsExplode** parameter can be added as a workbook list variable. This parameter can be used in workbooks that also use the **RR_Analytics_FlatBill_MultiSourceExplode** parameter or on its own (however several of its options either rely on or are influenced by the setting in the **RR_Analytics_FlatBill_MultiSourceExplode** parameter).

The **RR_Analytics_FlatBill_AlternatePartsExplode** parameter supports the options shown in the following list:

- **FromMultiSourceExplode**- Global substitutes and their components are excluded/included based on the workbook setting for the **RR_Analytics_FlatBill_MultiSourceExpode** parameter. If that parameter is set to "True", then global substitutes and their components are always included in

FlatBillDown reports (across all eligible part sources). However, if that parameter is set to "False", then global substitutes and their components are always excluded from FlatBillDown reports.

- **Exclude**- Components under global substitutes are always excluded from FlatBillDown reports. If the **RR_Analytics_FlatBill_MultiSourceExplode** parameter is enabled and set to "True", then all of an assembly's global substitutes are reported but the components under them are not. In all other cases, however, neither the global substitutes nor their components are reported.
- **Include**- Components under an assembly's global substitutes are included in FlatBillDown reports, along with the global substitutes themselves. This is irrespective of the setting in **RR_Analytics_FlatBill_MultiSourceExplode** (however, that parameter still influeces whether just the primary part source or all eligible part sources are exploded). Note that because including components under substitute parts can significantly increased the number of FlatBillDown records generated, this setting can potentially impact worksheet performance (particularly when used with an **RR_Analytics_FlatBill_MultiSourceExplode** setting of "True").

Note: The **RR_Analytics_FlatBill_AlternatePartsExplode** parameter was added in Version 2016.2, Service Update 20.0.

To enable inclusion/exclusion of global substitutes in FlatBillDown reports

1. Open a workbook that contains, or will contain, worksheets based on the **FlatBillDown** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then click **List** variable.
5. In the **Name** box, type **RR_Analytics_FlatBill_AlternatePartsExplode**.
6. In the **Label** box, type an optional label for the control to be displayed on the workbook toolbar.
7. Select the following check boxes:
   - **Available to users**
   - **Add to worksheet toolbar**
   - **Show label on toolbar**
8. Select the **List contains fixed values** check box, and then do the following:
   - Type **Exclude** in both the **Display Value** and **Data Value** cells.
   - Click **Add Row**.
   - Type **FromMultiSourceExplode** in both the **Display Value** and **Data Value** cells.
   - Click **Add Row**.
   - Type **Include** in both the **Display Value** and **Data Value** cells.
9. Optionally, make any required changes in the **Default value** area or add text in the **Description** box.
10. Click **OK**.
11. To allow workbook users to view and set the control, ensure a column with the following expression is added to worksheets based on the **FlatBillDown** table (typically, such a column can be hidden).

    ```
    $RR_Analytics_FlatBill_AlternatePartsExplode
    ```

Explode BOMs for "NoOrders" parts in FlatBillDown calculations

The **OrderPolicy.OrderGenerationRule** control field defines when a given part source can generate planned orders (for example, at any time or only outside of the planning time fence). The **OrderGenerationRule** setting of "NoOrders" is used to prevent a given part source from generating new planned orders. For example, it might be applied to historical or inactive part sources. By default, FlatBillDown calculations do not explode through parts from make or transfer part sources that use this "NoOrders" setting. Thus the components needed under such parts are not shown in the **FlatBillDown** table, although the "NoOrders" parts themselves are reported.

To allow explosion through "NoOrders" parts and enable reporting of their components in FlatBillDown reports, the **RR_Analytics_FlatBill_ExplodeNoOrders** parameter is available. For example, it can be used if you want to report cost rollups for historical BOMs or if you want to identify all components needed to make a part that is not currently planned or built.

The **RR_Analytics_FlatBill_ExplodeNoOrders** parameter can be added as a workbook variable and accepts the following values:

- **False**- When performing FlatBillDown calculations, BOM explosion stops at parts from sources that reference an **OrderGenerationRule** of "NoOrders". Thus "NoOrders" parts can be reported in the **FlatBillDown** table but their components are not identified. This setting is equivalent to not using the parameter at all.
- **True**- When performing FlatBillDown calculations, BOM explosion continues through parts from make or transfer sources that reference an **OrderGenerationRule** of "NoOrders". Thus components needed under "NoOrders" parts can be reported in the **FlatBillDown** table. This setting will typically increase the number of **FlatBillDown** records generated.

Note: As discussed earlier in this section, several other analytic parameters provide control over FlatBillDown calculations. The **RR_Analytics_FlatBill_ExplodeNoOrders** parameter is evaluated after all other FlatBillDown parameters applicable to a given workbook and worksheet. For example, if this parameter is set to "True" and **RR_Analytics_FlatBill_MultiSourceExplode** is enabled and set to "True", then all "NoOrders" parts from all eligible part sources are exploded. Instead if **RR_Analytics_FlatBill_MultiSourceExplode** were set to "False", then only "NoOrders" parts found under the primary part source would need to be exploded.

To control BOM explosion for "NoOrders" parts in FlatBillDown reports

1. Open a workbook that contains, or will contain, worksheets based on the **FlatBillDown** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then click **Boolean Variable**.
5. In the **Name** box, type **RR_Analytics_FlatBill_ExplodeNoOrders**.
6. In the **Label** box, type a label to use for this variable on the worksheet toolbar button and **Data Settings** dialog box (for example, Explode NoOrders?). If you do not specify a label, the parameter name displays instead.

7. In the **False value** area, do the following:
   - In the **Data value** box, ensure a text string of **false** is specified.
   - In the **Display value** box, type a value to use for this option in the **Data Settings** dialog box (for example, N or No).
8. In the **True value** area, do the following:
   - in the **Data value** box, ensure a text string of **true** is specified.
   - In the **Display value** box, type a value to use for this option in the **Data Settings** dialog box (for example, Y or Yes).
9. In the **Initial value** column, select which value should be the variable's default for the workbook.
   - **True**- by default, BOM explosion continues through "NoOrders" parts and hence details of their components can be reported in **FlatBillDown** worksheets.
   - **False** by default, BOM explosion stops at "NoOrders" parts and hence details of their components cannot be reported in **FlatBillDown** worksheets.
10. If you want the variable setting to be modifiable by workbook users, do the following:
    - Select the **Available to users** check box (if not selected, the variable's initial value is always used).
      This adds the variable in the workbook's **Data Settings** dialog box
    - Optionally, select the **Add to worksheet toolbar** check box.
      This adds a button to the workbook toolbar for toggling the variable value,
    - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
    - In worksheets based on the **FlatBillDown** table, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but should typically be hidden from view):
      `$RR_Analytics_FlatBill_ExplodeNoOrders`

Apply a parameter date to FlatBillDown calculations

The calculated **FlatBillDown** table is based on explosion through **BillOfMaterial** records, and thereby reports all components used to make a given assembly.

By default, this only includes results from **BillOfMaterial** records found when exploding through the assemby's primary part source. However, various analytic parameters can be used to impact the scope of results returned by **FlatBillDown** records. For example, if the **RR_Analytics_FlatBill_ MultiSourceExplode** parameter is used and set to "True", then the **FlatBillDown** table will show results from exploding **BillOfMaterial** records across of the assembly's eligible part sources. Further when exploding the **BillOfMaterial** records applicable to a given part source, all eligble **BillOfMaterial** records are reported regardless of date.

To add consideration for date effectivity to **FlatBillDown** calculations, and potentially limit or refine which records are returned, the **RR_Analytics_FlatBill_ParameterDate** parameter can be used. When used, this date parameter is compared against records in certain date-effective tables to determine which should be used by or included in **FlatBillDown** calculations. This includes the following tables:

PartSource	Only part sources that are effective on the specified date will be considered. For example, if a parameter date of "11-30-2018" is specified and **RR_Analytics_FlatBill_MultiSourceExplode** is set to "True", then all of the assembly's part sources effective on November 30, 2018 would be considered. However, if **RR_Analytics_FlatBill_MultiSourceExplode** is set to "False", then only the part source considered to be primary part source for the assembly on October 30, 2018 would be considered (note that this might always be same part source identified as the primary by the **Part.PrimaryPartSource** field).
BillOfMaterial	Only bill of material entries that are effective on the specified date will be used. For example, if a parameter date of "11-30-2018" is specified and **RR_Analytics_FlatBill_MultiSourceExplode** is set to "False", then only **BillOfMaterial** records found when sourcing from the primary part source and which are effective on November 30, 2018 are used. Thus records which are no longer effective for an assembly/component on the specified date, or which have yet to become effective for an assembly/component on the specified date, will be excluded.
TimePhasedPartSourceLeadTime	If this table has been used to define time-phased lead times for part sources, then the record effective on the specified parameter date will be used (rather the values on the **PartSource** record). For example, if a parameter date of "11-30-2018" is specified, then the lead times defined in the **TimePhasedPartSourceLeadTime** record effective on November 30, 2018 for the assembly's part source(s) are used. Note that if time-phased lead times are applicable and the **RR_Analytics_Flat_Bill_ParameterDate** parameter is not used, then the **TimePhasedPartSourceLeadTime** record effective at the later of the part's **RunDate** or **PartSource.FirstEffectiveDate** is used.
TimePhasedSourceLeadTime	Similar to **TimePhasedPartSouceLeadTime.** If this table has been used to define time-phased lead times at the source level, then only **TimePhasedSourceLeadTime** records effective on the specified parameter date will be used for an assembly's part source(s).

The **RR_Analytics_FlatBill_ParameterDate** parameter can be enabled as a workbook variable and then applied to worksheets based on the **FlatBillDown** table.

Worksheet users can then specify a parameter date for use in **FlatBillDown** reports. The specified parameter date is used for top-level assemblies and, after being offset for applicable lead times, is passed down to and used by lower-level components. For example, if a date value of "11-30-2018" is specified for **RR_Analytics_FlatBill_ParameterDate** and a top-level assembly has an effective lead time of 3 days, then its immediate components would use a parameter date of November 27, 2018. This ensures that only pertinent details of components that could be used to build new assembly supply due on the specified parameter date are used in **FlatBillDown** calculations.

To apply a parameter date to FlatBillDown reports

1. Open a workbook that contains, or will contain, worksheets based on the **FlatBillDown** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then click **Date** variable.
5. In the **Name** box, type **RR_Analytics_FlatBill_ParameterDate**.
6. In the **Label** box, type an optional label for the control to be displayed on the workbook toolbar.

7. In the **Description** box, type an optional description for the variable. For example, this can help other worksheet authors understand the intention of the variable.
8. To make the variable available to workbook users, do the following:
   - Select the **Available to users** check box.
   - Select the **Add to worksheet toolbar** check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
   - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should not display a label).
9. Optionally, in the **Initial value** box, specify the default date to use for this variable in all of the workbook's worksheets. Alternatively specifying a value of **Undefined** in the box or selecting the **None** option means the variable won't be applied until a worksheet user provides a value.
10. Click **OK**.
11. To allow workbook users to view and set the control, ensure a column with the following expression is added to worksheets based on the **FlatBillDown** table (typically, such a column can be hidden).

    ```
    $RR_Analytics_FlatBill_ParameterDate
    ```

Note: The **RR_Analytics_FlatBill_ParameterDate** parameter was added in Version 2016.2, Service Update 22.0.

Limiting FlatBillDown and FlatBillUp calculations by model

By default, when you run **FlatBillDown** or **FlatBillUp** calculations against a specific part, all models of that part are included. The **RR_Analytics_FlatBill_Model** analytics parameter ensures that only records with a specified model are computed, which decreases the memory requirement for **FlatBillDown** and **FlatBillUp** calculations.

The **RR_Analytics_FlatBill_Model** parameter can be tied to an application-level model filter. This way, whenever the model filter is altered, the **RR_Analytics_Flatbill_Model** automatically changes. Thereby, the analytics computation can be limited by the selection of the model filter.

The **RR_Analytics_FlatBill_Model** parameter can be configured as a text variable or a list variable.

To consider all models, set the **RR_Analytics_FlatBill_Model** parameter to an asterisk (*), a single-quoted asterisk ('*'), or do not set the parameter.

To limit FlatBillDown and FlatBillUp calculations by model

1. Open a workbook containing worksheets based on the **FlatBillUp** or **FlatBillDown** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. To define this variable as a text variable, do the following:
   - Click **New**, and then click **Text Variable**.
   - In the **Name** box, type **RR_Analytics_FlatBill_Model**.

- In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box (for example, **Model**).
- Ensure that the **Available to users** check box is selected.
- In the **Initial value** area, specify the variable's default value.
- In the **On opening the workbook** area, select **Always use the initial value**.
- Click **OK**.

5. To define this variable as a list variable, do the following:
   - Click **New**, and then click **List Variable**.
   - In the **Name** box, type **RR_Analytics_FlatBill_Model**.
   - In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box (for example, **Model**).
   - Ensure that the **Available to users** check box is selected.
   - In the **Initial value** area, specify the variable's default value.
   - Select the **List contains query-generated values** check box.
   - From the **Table** list, select **Model**.
   - In the **Display value** box, type **Value**.
   - In the **Data values** box, type **Value**.
   - Click **OK**.

To associate the RR_Analytics_FlatBill_Model filter to an applications model filter

1. Open a workbook containing worksheets based on the **FlatBillUp** or **FlatBillDown** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then **List Variable**.
5. In the **Name** box, type **RR_Analytics_FlatBill_Model**.
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box (for example, **Model**).
7. Ensure that the **Available to users** check box is cleared.
8. In the **Initial value** area, specify the variable's default value.
9. In the **On opening the workbook** area, select **Always use the initial value**.
10. Select the **List contains query-generated values** check box.
11. From the **Table** list, select **Model**.
12. In the **Display value** box, type **$SelectedModel**.
13. In the **Data values** box, type **$SelectedModel**.
14. Click **OK**.

Specify workbook-specific Inventory Analysis buckets

In workbooks that calculate Inventory Analysis analytics, you can specify Inventory Analysis buckets that are used in only that workbook. These buckets can be used to calculate Inventory Analysis results for a specific period without modifying the results of other Inventory Analysis worksheets.

To specify Inventory Analysis buckets for a workbook, you can pass the bucket size and the number of buckets as parameters to the Inventory Analysis analytic. Values for the parameters are specified using workbook variables. The values specified are used in all worksheets based on the InventoryAnalysis or InventoryCTPAnalysis tables in the workbook.

To enable workbook specific bucketing of Inventory Analysis data

1. Open a workbook containing worksheets based on either the **InventoryAnalysis** or **InventoryCTPAnalysis** tables.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. To define a variable used to set the calendar that defines bucket size, do the following:
   - Click **New**, and then **List Variable**.
   - In the **Name** box, type **RR_Analytics_InventoryAnalysis_Calendar.**
   - In the **Label** box, provide a name to be used for this control in the Data Settings dialog box (for example, Bucketing calendar).
   - Ensure the **Available to users** check box is selected.
   - In the **Initial value** area, specify the variable's default value.
   - Select the **List contains query-generated values** check box.
   - From the **Table** list, select **Calendar**.
   - In the **Display value** box, type **Value**.
   - Click **OK**.
5. To define a variable used to specify the number of buckets before the planning date, do the following:
   - Click **New**, and then click **Quantity Variable**.
   - In the **Name** box, type **RR_Analytics_InventoryAnalysis_BucketsBefore**.
   - In the **Label** box, provide a name to be used for this control in the Data Settings dialog box.
   - Ensure the **Available to users** check box is selected.
   - In the **Minimum** and **Maximum** boxes, specify the limits on the number of buckets worksheet users can specify.
   - In the **Initial value** area, specify the variable's default value.
   - Click **OK**.
6. To define a variable used to specify the number of buckets after the planning date, do the following:
   - Click **New**, and then click **Quantity Variable**.
   - In the **Name** box, type **RR_Analytics_InventoryAnalysis_BucketsAfter**.
   - In the **Label** box, provide a name to be used for this control in the Data Settings dialog box.
   - Ensure the **Available to users** check box is selected.
   - In the **Minimum** and **Maximum** boxes, specify the limits on the number of buckets worksheet users can specify.
   - In the **Initial value** area, specify the variable's default value.
   - Click **OK**.
7. If you want to make a given variable modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.

- To add a calendar control to the workbook toolbar for setting this variable, select the **Add to worksheet toolbar** check box.
- In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
- In the worksheets based on the **InventoryAnalysis** and/or **InventoryCTPAnalysis** tables, add columns referencing each of the newly created variables to be exposed using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
  ```
  $RR_Analytics_InventoryAnalysis_Calendar
  $RR_Analytics_InventoryAnalysis_BucketsBefore
  $RR_Analytics_InventoryAnalysis_BucketsAfter
  ```

Specify a reporting horizon for WhereConsumed records

The **WhereConsumed** table is suitable for bottom-up analysis. It is used to peg each component supply to both the highest level demand it satisfies, along with the immediate parent supply that drove demand to the component.

To allow users of reports based on the **WhereConsumed** table to more quickly and easily retrieve the specific results they are interested in, the following two analytic parameters are available to specify a date-based reporting horizon in a given workbook or worksheet:

- **RR_Analytics_WhereConsumed_BeginDate**—defines a date on or after which component supplies must be due if they are to be reported and pegged to end-item demands in worksheets based on the **WhereConsumed** table. In other words, this represents the earliest **SupplyDueDate** that will be reported.
- **RR_Analytics_WhereConsumed_EndDate**—defines a date on or before which component supplies must be due if they are to be reported and pegged to end-item demands in worksheets based on the **WhereConsumed** table. In other words, this represents the latest **SupplyDueDate** that will be reported.

These two parameters can added as workbook variables and then used to specify a default reporting horizon for records in all **WhereConsumed** worksheets in a given workbook. Optionally, the variables can be configured to give users of a specific **WhereConsumed** worksheet the ability to specify their own reporting horizon.

To specify the earliest date for which WhereConsumed records are reported

1. Open a workbook containing worksheets based on the **WhereConsumed** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumed_BeginDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:

- Select the **Available to users** check box.
- Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
- Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).

8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the earliest **SupplyDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumed** table for which worksheet users should be able to specify a value for the variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumed_BeginDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

To specify the latest date for which WhereConsumed records are reported

1. Open a workbook containing worksheets based on the **WhereConsumed** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumed_EndDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:
    - Select the **Available to users** check box.
    - Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
    - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).
8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the latest **SupplyDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumed** table for which worksheet users should be able to specify a value for the variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumed_EndDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

Specify a reporting horizon for WhereConsumedForForecast records

The **WhereConsumedForForecast** table is suitable for top-down analysis. It is used to report all component supplies used to satisfy a consensus forecast demand as generated from the S&OP process.

To allow users of reports based on the **WhereConsumedForForecast** table to more quickly and easily retrieve the specific results they are interested in, the following two analytic parameters are available to specify a date-based reporting horizon in a given workbook or worksheet:

- **RR_Analytics_WhereConsumedForForecast_BeginDate**—defines a date on or after which consensus forecast demands must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForForecast** table. In other words, this represents the earliest **DriverDueDate** that will be reported.
- **RR_Analytics_WhereConsumedForForecast_EndDate**—defines a date on or before which consensus forecast demands must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForForecast** table. In other words, this represents the latest **DriverDueDate** that will be reported.

These two parameters can added as workbook variables and then used to specify a default reporting horizon for records in all **WhereConsumedForForecast** worksheets in a given workbook. Optionally, the variables can be configured to give users of a specific **WhereConsumedForForecast** worksheet the ability to specify their own reporting horizon.

To specify the earliest date for which WhereConsumedForForecast records are reported

1. Open a workbook containing worksheets based on the **WhereConsumedForForecast** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedForForecast_BeginDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:
   - Select the **Available to users** check box.
   - Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
   - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).
8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the earliest **DriverDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumedForForecast** table for which worksheet users should be able to specify a value for the variable:

- Right-click in the worksheet, and then click **Worksheet Properties**.
- Ensure the **Data** tab is selected.
- Click **New Column,** and type the following in the **Expression** box:
  `$RR_Analytics_WhereConsumedForForecast_BeginDate`
- Click the **Hide column** check box.

11. Click **OK** and then **OK** again.

To specify the latest date for which WhereConsumedForForecast records are reported

1. Open a workbook containing worksheets based on the **WhereConsumedForForecast** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedForForecast_EndDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:
   - Select the **Available to users** check box.
   - Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
   - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).
8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the latest **DriverDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumedForForecast** table for which worksheet users should be able to specify a value for the variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumedForForecast_EndDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

Specify a reporting horizon for WhereConsumedForSupply records

The **WhereConsumedForSupply** table is suitable for top-down analysis. It is used to report all component supplies across all levels that are used to satisfy a given production order (such as, a scheduled receipt or planned order).

To allow users of reports based on the **WhereConsumedForSupply** table to more quickly and easily retrieve the specific results they are interested in, the following two analytic parameters are available to specify a date-based reporting horizon in a given workbook or worksheet:

- **RR_Analytics_WhereConsumedForSupply_BeginDate**—defines a date on or after which production orders must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForSupply** table. In other words, this represents the earliest **DriverDueDate** that will be reported.
- **RR_Analytics_WhereConsumedForSupply_EndDate**—defines a date on or before which production orders must be due if they and their pegged component supplies are to be reported in worksheets based on the **WhereConsumedForSupply** table. In other words, this represents the latest **DriverDueDate** that will be reported.

These two parameters can added as workbook variables and then used to specify a default reporting horizon for records in all **WhereConsumedForSupply** worksheets in a given workbook. Optionally, the variables can be configured to give users of a specific **WhereConsumedForSupply** worksheet the ability to specify their own reporting horizon.

To specify the earliest date for which WhereConsumedForSupply records are reported

1. Open a workbook containing worksheets based on the **WhereConsumedForSupply** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedForSupply_BeginDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:
   - Select the **Available to users** check box.
   - Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
   - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).
8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the earliest **DriverDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumedForSupply** table for which worksheet users should be able to specify a value for the variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumedForSupply_BeginDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

To specify the latest date for which WhereConsumedForSupply records are reported

1. Open a workbook containing worksheets based on the **WhereConsumedForSupply** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedForSupply_EndDate.**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box and/or worksheet toolbar.
7. Optionally, to make the variable available to workbook users, do the following:
   - Select the **Available to users** check box.
   - Select the **Add to worksheet** toolbar check box (or leave unchecked if the variable should only be available in the **Data Settings** dialog box).
   - Select the **Show label on toolbar** check box (or leave unchecked if the toolbar control should appear without a label).
8. In the **Initial value** box, specify the default value to use for this variable in all worksheets (the latest **DriverDueDate** for which records should be reported). If an initial value is not provided, then a value of "Past" is assumed unless a worksheet user specifies a value for the variable.
9. Click **OK** and then **OK** again.
10. In each worksheet based on the **WhereConsumedForSupply** table for which worksheet users should be able to specify a value for the variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box: `$RR_Analytics_WhereConsumedForSupply_EndDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

Specify a reporting horizon for WhereConsumedToHighVolumeOrder records

The **WhereConsumedToHighVolumeOrder** table pegs each component supply to the highest level demand(s) where it is ultimately used. Because this table is intended for use with components required in configurable end-items that have large volumes of demands, large numbers of records can potentially be generated in this table.

To allow users of reports based on the **WhereConsumedToHighVolumeOrder** table to more quickly and easily retrieve the specific results they are interested in, the following two analytic parameters are available:

- **RR_Analytics_WhereConsumedHighVolume_BeginDate**—defines a date on or after which component supplies must be due if they are to be reported and pegged to end-item demands in the **WhereConsumedToHighVolumeOrder** table.

- **RR_Analytics_WhereConsumedHighVolume_EndDate**—defines a date before which component supplies must be due if they are to be reported and pegged to end-item demands in the **WhereConsumedToHighVolumeOrder** table.

These parameters can added as workbook variables and then enabled on worksheets based on the **WhereConsumedToHighVolumeOrder** table. This allows users of those worksheets to specify a the date range over which records can be reported in the worksheet.

To enable specifying the earliest date for which WhereConsumedToHighVolumeOrder results can be generated

1. Open a workbook containing worksheets based on the **WhereConsumedToHighVolumeOrder** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedHighVolume_BeginDate**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box.
7. Ensure the **Available to users** check box is selected.
8. Optionally, to add the variable to the worksheet toolbar, do the following:
   - Select the **Add to worksheet** toolbar check box.
   - Select the **Show label on toolbar** check box (or leave blank if the toolbar control should appear without a label).
9. Click **OK** and then **OK** again.
10. For each worksheet based on the **WhereConsumedToHighVolumeOrder** table, do the following to enable use of the new variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumedHighVolume_BeginDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

To enable specifying the latest date for which WhereConsumedToHighVolumeOrder results can be generated

1. Open a workbook containing worksheets based on the **WhereConsumedToHighVolumeOrder** table.
2. Right-click in any worksheet, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **Date Variable**.
5. In the **Name** box, type **RR_Analytics_WhereConsumedHighVolume_EndDate**
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box.
7. Ensure the **Available to users** check box is selected.
8. Optionally, to add the variable to the worksheet toolbar, do the following:

- Select the **Add to worksheet** toolbar check box.
- Select the **Show label on toolbar** check box (or leave blank if the toolbar control should appear without a label).

9. Click **OK** and then **OK** again.
10. For each worksheet based on the **WhereConsumedToHighVolumeOrder** table, do the following to enable use of the new variable:
    - Right-click in the worksheet, and then click **Worksheet Properties**.
    - Ensure the **Data** tab is selected.
    - Click **New Column,** and type the following in the **Expression** box:
      `$RR_Analytics_WhereConsumedHighVolume_EndDate`
    - Click the **Hide column** check box.
11. Click **OK** and then **OK** again.

Enabling feature-based queries on IndependentDemandByFeature records

The calculated **IndependentDemandByFeature** table contains references to the **Feature**, **Part**, and **IndependentDemand** tables. It can be used to report all demands that match a specified feature, list of features, or feature rule. In order to pass the names of the actual features to be evaluated by the table's underlying calculations, an analytic parameter is required.

By default, this parameter is implemented as a string variable in the standard **Orders by Feature** workbook included with RapidResponse. When needing to report demands by feature, the use of this workbook is recommended as it provides assistance in building potentially lengthy or complex feature lists and rules. However, a workbook variable can also be added for this purpose to any workbook containing worksheets based on the **IndependentDemandForFeature**. This allows for the Data Display Settings dialog box to be used in passing feature values to the analytic parameter.

To enable querying IndependentDemandbyFeature records by feature

1. Open a workbook that contains, or will contain, worksheets based on the **IndependentDemandByFeature** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New** and then **New Text Variable**.
5. In the **Name** box, type **RR_Analytics_FeatureBOM_FeatureRule**.
6. In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box.
7. Ensure the **Available to users** check box is selected.
8. In the worksheet(s) based on the **IndependentDemandByFeature** table, add a column that references the newly created variable using the following syntax:
   `$RR_Analytics_FeatureBOM_FeatureRule`

Specifying the number of levels used in WhereConsumedForSupply calculations

The **WhereConsumedForSupply** table details all supplies across all levels, under a given item, required to fill a production order for the item. In workbooks that report supplies, if not all levels of detail are required, the **WhereConsumedForSupply** analytic can be set to calculate a limited number of levels. You can also limit the number of reported levels to improve worksheet performance.

To specify the number of product levels for a workbook, you can pass the number of levels as a parameter to the full-level pegging analytic that produces results in the **WhereConsumedForSupply** table. The value for the parameter is specified using a workbook variable. The specified value is used in all worksheets based on the **WhereConsumedForSupply** table in the workbook.

If a negative value is specified for the variable, the calculation uses all levels. If zero is specified, the calculation uses only the product level of the part itself. If a number greater than zero is specified, the calculation uses that number of levels.

The variable to pass the number of levels must be a quantity variable, which specifies the minimum and maximum values that can be set. In this case, the minimum value should be -1 to allow users to view all levels, and the maximum should be your maximum product level depth.

If you allow workbook users to modify the variable, they can specify any value in the range you have specified. If you do not allow users to modify the variable, you must set the variable's initial value to the number of levels to use in the workbook.

To specify the number of levels used in WhereConsumedForSupply calculations

1. Open a workbook that contains or will contain a worksheet based on the **WhereConsumedForSupply** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **Quantity Variable**.
5. In the **New Quantity Variable** dialog box, in the **Name** box, type **RR_Analytics_ WhereConsumedForSupply_MaxLevel**
6. In the **Minimum** box, type **-1**.
7. In the **Maximum** box, type the largest value the variable can contain.
8. In the **Initial value** area, specify the variable's default value.
9. If you want the variable to be modifiable by workbook users, do the following:
   - Select the **Available to users** check box. This adds the variable to the workbook's **Data Settings** dialog box.
   - In the **On opening the workbook** area, specify whether you want the workbook to always open with the default value or with a given user's last selected value.
   - In the worksheets based on the **WhereConsumedForSupply** table, add a column that references the newly created variable using the following syntax (this column is required to expose the variable to users, but can typically be hidden):
     `$RR_Analytics_WhereConsumedForSupply_MaxLevel`

Calculate general disaggregation rates by forecast category

The **DisaggregationRateByPartCustomer** table stores calculated disaggregation rates applied to different streams of forecast data entered in RapidResponse applications. For example, when members of the Finance group enter the financial forecast using the S&OP Finance Operating Plan workbook, these rates are used to disaggregate forecast data made against higher levels of a hierarchy down to detailed part and customer combinations.

In order for the **DisaggregationRateByPartCustomer** table to calculate and report disaggregation rates for a given forecast category, the name of the category as defined in **HistoricalDemandCategory.Value** needs to be passed to the table using an analytic parameter named **RR_Analytics_Disaggregation_ForecastCategory**. Therefore, this parameter should be added as a workbook variable to workbooks that are meant to either use or report calculated general disaggregation rates.

To enable calculation of forecast category disaggregation rates

1. Open a workbook that contains or will contain a worksheet based on the **DisaggregationRateByPartCustomer** table.
2. Right-click in the workbook, and then click **Workbook Properties**.
3. Click the **Variables** tab.
4. Click **New**, and then click **List Variable**.
5. In the **New List Variable** dialog box, do the following:
   - In the **Name** box, type **RR_Analytics_Disaggregation_ForecastCategory**.
   - In the **Label** box, provide a name to be used for this control in the **Data Settings** dialog box.
   - To make the variable available to workbook users, select both the **Available to users** check box and the **Add to worksheet** toolbar check box.
   - In the **Initial value** area, specify the variable's default value (if any).
   - Select the **List contains query-generated values** check box.
   - From the **Table** list, select **HistoricalDemandCategory.**
   - In the **Display values** box, type **Value**.
   - In the **Data values** box, type **Value**.
   - Click **Ok**.
6. In the worksheet(s) based on the **DisaggregationRateByPartCustomer** table, ensure there is at least one column or filter expression that references the newly created variable using the following syntax (this is required to expose the variable to worksheet users):
   `$RR_Analytics_Disaggregation_ForecastCategory`

   If a column expression is used expose the variable to users, it can subsequently be hidden.

Note: Actual examples of the use of this parameter in RapidResponse can be seen in certain S&OP workbooks such as S&OP Forecast Disaggregation, S&OP Annual Plan, S&OP Finance Operating Plan, and S&OP Sales Forecast.

Part 3: Configuring resource calculations

- Kanban Manager calculations
- Multi-enterprise resources
- Data validation resources
- Clear to Build resources

CHAPTER 50: Kanban Manager calculations

The Kanban Manager workbook can be used to support a kanban material replenishment system. A kanban system uses a given number of standard containers (bins) to hold an established quantity of a component. When all components within a bin are used, the bins empty condition becomes a signal for replenishment of the component.

The RapidResponse data model includes two tables, **ShopKanban** and **SourceKanban**, which can hold details of a kanban system for a factory location or source (supplier) respectively. Each of these tables also contain fields that the Kanban Manager workbook can populate with recommendations for new bin sizes or number of bins. By reviewing these recommendations as demands and other variables change, Kanban Manager users can potentially avoid material shortages or excess inventory that might otherwise arise.

This chapter describes the calculations performed by Kanban Manager, as well as the macros and profile variables that can be used to configure these calculations.

In this chapter, you'll learn about:

Kanban Manager tables **2239**
ShopKanban calculations **2240**
SourceKanban calculations **2242**
Configuring Kanban Manager calculations **2244**

Note: RapidResponse can also enforce kanban logic in calculations performed by the netting and capable-to-promise analytics, to ensure that only a given number of bins can be in process at one time. This logic only pertains to SourceKanbans (and not to ShopKanbans), and can be enforced with or without use of the Kanban Manager workbook.

Kanban Manager tables

The RapidResponse Data Model contains three tables to specifically support the Kanban Manager workbook as described below:

Table 50.1: Kanban Manager tables

Table	Description
Shop	This tables identifies unique shop floor locations for kanban parts. It is referenced by the ShopKanban table and, together with the Part record, uniquely identifies a ShopKanban record. For more information about this table, see "Shop table" on page 593.
ShopKanban	This tables holds details relating to a kanban material replenishment system within a factory. Each ShopKanban record is uniquely identified by a Part and Shop combination. It contains information such as the existing bin quantity and number of bins for each ShopKanban. It also holds new recommendations for these values calculated by Kanban Manager, as well as parameters used by these calculations (such as, the desired replenishment period and maximum bin quantity allowed). For more information about this table, see "ShopKanban table" on page 594. For more information about the Kanban Manager calculations used by this table, see "ShopKanban calculations" on page 2240.
SourceKanban	This tables holds details relating to a kanban material replenishment system for a particular source, such as a supplier. Each SourceKanban record is uniquely identified by its PartSource record. It contains information such as the existing bin quantity and number of bins for each SourceKanban. It also hold new recommendations for these values calculated by Kanban Manager, as well as parameters used by these calculations, such as the desired replenishment period and maximum bin quantity allowed. For more information about this table, see "SourceKanban table" on page 607. For more information about the Kanban Manager calculations used by this table, see "SourceKanban calculations" on page 2242.

ShopKanban calculations

When a kanban analysis is run, Kanban Manager calculates new recommended bin quantities and number of bins for the selected ShopKanban parts. It also calculates the average daily demand for the ShopKanban parts to then use as input into the other two calculations. This section describes each of these three calculations.

Average daily demand

The first value calculated by Kanban Manager during a Kanban Analysis is the average daily demand for a part. This value populates **ShopKanban.AverageDailyDemand,** and is then available for use in the calculations that define the recommended bin quantity and recommended number of bins.

For each ShopKanban part contained in a selected filter, average daily demand is calculated by summing the RemainingQuantity of all DependentDemand records due on dates within the kanban horizon for this Shop (including any past due demands), and then dividing by the number of days defined by the kanban horizon. The expression for this calculation is:

```
RemainingQuantity / (? Allocation.ScheduledReceipt <> ''
Allocation.Shop.KanbanHorizon BillOfMaterial.Shop.KanbanHorizon)
```

Example

Suppose a ShopKanban part has a total of 50 DependentDemand records created from Allocations and due within a given kanban horizon, each for a Quantity of 100. Further, suppose that the kanban

horizon for this shop is set to 40 days. The average daily demand for the part would be calculated as follows:

```
AverageDailyDemand = 5000 / 40
AverageDailyDemand = 125
```

Recommended bin quantity

The second value calculated by Kanban Manager during a kanban analysis is a new recommended bin quantity for the part at this shop location. This value populates **ShopKanban.RecommendedBinQty**.

For each ShopKanban record, the recommended bin quantity is calculated by multiplying the AverageDailyDemand (calculated previously) by the ReplenishmentPeriod (in days), and then rounding up to the nearest multiple of the part primary source's multiple quantity. The lower of this value or the MaximumBinQty then becomes the new recommended bin quantity. The expression for this calculation is:

```
Min (Mult(AverageDailyDemand * ReplenishmentPeriod),
Part.PrimarySource.MultipleQty) ? MaximumBinQty > 0 MaximumBinQty 9999999
```

Note: If no MaximumBinQty is specified, a value of 9999999 is used.

Example

Suppose a ShopKanban part with the following values:

- AverageDailyDemand — 125
- ReplenishmentPeriod — 3 days
- Part.PrimarySource.MultipleQty — 10
- MaximumBinQty — 500

The recommended bin quantity for the part would be calculated as follows:

```
RecommendedBinQty = Min (Mult((125 * 3), 10) 500)
RecommendedBinQty = Min (Mult ((375), 10) 500)
RecommendedBinQty = Min (380, 500)
RecommendedBinQty = 380
```

Recommended number of bins

The third value calculated by Kanban Manager is a new recommended number of bins for the part and shop defined for this ShopKanban record. This value populates **ShopKanban.RecommendedBins**.

For each ShopKanban record, the recommended number of bins is calculated by multiplying AverageDailyDemand (calculated previously) by ReplenishmentLeadTime (in days), increasing by a SafetyFactor, and dividing by the BinQty. This value is then rounded up to nearest whole number and increased by one. Then the greater of this number or 2 becomes the new recommended number of bins (thus ensuring there will always be a minimum of 2 bins). The expression for this calculation is:

```
Max(RoundUp((AverageDailyDemand * ReplenishmentLeadTime * (SafetyFactor +
1)) / BinQty ) + 1, 2)
```

Note: This calculation uses the existing bin quantity, and not the recommended bin quantity. However, it is possible to calculate the recommended number of bins based on the recommended bin quantity using the **ShopKanbanCalcNumBins** macro as discussed in "ShopKanbanCalcNumBins" on page 2246.

Example

Suppose a ShopKanban record the following values:

- AverageDailyDemand — 125
- ReplenishmentLeadTime — 2 days
- SafetyFactor — .2
- BinQty — 120

The recommended bin quantity for the part would be calculated as follows:

```
RecommendedBins = Max(RoundUp((125 * 2 *(.2 + 1)) / 120 ) + 1, 2)
RecommendedBins = Max(RoundUp((250 * 1.2) / 120 ) + 1, 2)
RecommendedBins = Max(RoundUp(300 / 120) + 1, 2)
RecommendedBins = Max(RoundUp(2.5) + 1, 2)
RecommendedBins = Max(3 + 1, 2)
RecommendedBins = Max(4, 2)
RecommendedBins = 4
```

SourceKanban calculations

During a kanban analysis, Kanban Manager calculates new recommended bin quantity and number of bins for the selected SourceKanban parts. It also calculates the average daily demand for the SourceKanban parts for use as input into the other two calculations. This section describes each of the these calculations.

Average daily demand

The first value calculated by Kanban Manager during a kanban analysis is the average daily demand for a part. This value populates **SourceKanban.AverageDailyDemand,** and is then available for use in the calculations that define the recommended bin quantities and recommended number of bins.

For each SourceKanban part contained in a selected filter, average daily demand is calculated by summing the Quantity of all PlannedOrder records due on dates within the kanban horizon for this Source (including any past due demands), and then dividing by the number of days defined by the kanban horizon. The expression for this calculation is:

```
Quantity / PartSource.SourceKanban.KanbanHorizon
```

Example

Suppose a SourceKanban part has a total of 50 PlannedOrder records due within a given Kanban Horizon, each for a Quantity of 100. Further, suppose that the Kanban Horizon for this source is set to 40 days. The average daily demand for the part would be calculated as follows:

```
AverageDailyDemand = 5000 / 40
```

```
AverageDailyDemand = 125
```

Recommended bin quantity

The second value calculated by Kanban Manager during a kanban analysis is a new recommended bin quantity for the SourceKanban part. This value populates **SourceKanban.RecommendedBinQty**.

For each SourceKanban record, the recommended bin quantity is calculated by multiplying the AverageDailyDemand (calculated previously) by the ReplenishmentPeriod (in days), and then rounding up to the nearest multiple of the part source's multiple quantity. The lower of this value or the MaximumBinQty then becomes the new recommended bin quantity. The expression for this calculation is:

```
Min (Mult(AverageDailyDemand * ReplenishmentPeriod), PartSource.MultipleQty)
? MaximumBinQty > 0 MaximumBinQty 9999999
```

Note: If no MaximumBinQty is specified, a value of 9999999 is used.

Example

Suppose a SourceKanban record with the following values:

- AverageDailyDemand — 125
- ReplenishmentPeriod — 3 days
- PartSource.MultipleQty — 10
- MaximumBinQty — 500

The recommended bin quantity for the part would be calculated as follows:

```
RecommendedBinQty = Min (Mult((125 * 3), 10) 500)
RecommendedBinQty = Min (Mult((375), 10) 500)
RecommendedBinQty = Min (380, 500)
RecommendedBinQty = 380
```

Recommended number of bins

The third value calculated by Kanban Manager is a new recommended number of bins for the SourceKanban part. This value populates **SourceKanban.RecommendedBins**.

For each SourceKanban record, the recommended number of bins is calculated by multiplying AverageDailyDemand (calculated previously) by lead time for the part source (in Part.PlanningCalendars.TimeUnits), increasing by a SafetyFactor, and dividing by the BinQty. This value is then rounded up to nearest whole number and increased by one. Then the greater of this number or 2 becomes the new recommended number of bins (thus ensuring there will always be a minimum of 2 bins). The expression for this calculation is:

```
Max(RoundUp(AverageDailyDemand * PartSource.LeadTimeDate -
Part.PlanningCalendars.RunDate.FirstDate Part.PlanningCalendars.TimeUnits) *
(SafetyFactor + 1)) / BinQty) + 1, 2)
```

Note: This calculation uses the existing bin quantity, and not the recommended bin quantity. However, it is possible to calculate the recommended number of bins based on the recommended bin quantity using the **SourceKanbanCalcNumBins** macro as discussed in "SourceKanbanCalcNumBins" on page 2247.

Example

Suppose a SourceKanban record with the following values:

- AverageDailyDemand — 125
- Lead time on the part source — 2 days
- SafetyFactor — .2
- BinQty — 120

The recommended bin quantity for the part would be as follows:

```
RecommendedBins = Max(RoundUp((125 * 2 *(.2 + 1)) / 120 ) + 1, 2)
RecommendedBins = Max(RoundUp((250 * 1.2) / 120 ) + 1, 2)
RecommendedBins = Max(RoundUp(300 / 120) + 1, 2)
RecommendedBins = Max(RoundUp(2.5) + 1, 2)
RecommendedBins = Max(3 + 1, 2)
RecommendedBins = Max(4, 2)
RecommendedBins = 4
```

Configuring Kanban Manager calculations

All Kanban Manager calculations are controlled through a series of profile variables and macros included with RapidResponse. Profile variables are used to set planning policies that determine the types of records to include in Kanban Manager calculations, and to set the Kanban Manager part threshold. Macros hold the actual expressions that are calculated during a kanban analysis, and can be edited depending on your company's needs.

Specifying the record types to exclude from kanban calculations

Using profile variables, you can exclude specific types of records from the calculations performed by Kanban Manager. For example, you might want to exclude specific types of demand (such as, safety stock or allocations) or specific types of supplies (such as, on hand or scheduled receipts on sub-assemblies).

To specify record types to exclude from Kanban Manager calculations

1. From the **View** menu, click **Macros and Profile Variables**.
2. Click the **Profile Variables** tab.
3. Modify the **Data Value** for any of the following variables as required:

KanbanDisableDemand	Indicates the DemandTypes to be excluded from Kanban Manager calculations (by having their DemandType.ProcessingRule set to "Ignore" during kanban processing). All DemandTypes to be excluded should be specified in a comma-delimited list. If all DemandTypes are to be excluded, a value of "All" can be specified. If no DemandTypes are to be excluded, this value can be left blank. By default, this value is blank.
KanbanDisableOnhand	Indicates the OnHandTypes to be excluded from Kanban Manager calculations (by having their OnHandType.ProcessingRule set to "NonNettable" during kanban processing). All OnHandTypes to be excluded should be specified in a comma-delimited list. If all OnHandTypes are to be excluded, a value of "All" can be specified. If no OnHandTypes are to be excluded, this value can be left blank. By default, this value is set to "All".
KanbanDisableSupply	Indicates the SupplyTypes to be ignored by Kanban Manager calculations (by having their SupplyType.ProcessingRule set to "Ignore" during kanban processing). All SupplyTypes to be excluded should be specified in a comma-delimited list. If all SupplyTypes are to be excluded, a value of "All" can be specified. If no SupplyTypes are to be excluded, this value can be left blank. By default, this value is set to "All".
KanbanDisableSafetyStock	Indicates whether safety stock values should be ignored by Kanban Manager calculations. By default, this variable is set to "Yes" (that is, PartType.SafetyStockRule is set to "Ignore" for all parts during kanban processing).

Note: For complete information about working with profile variables, see the *RapidResponse Resource Authoring Guide*.

Setting the Kanban Manager part threshold

When Kanban Manager users run a kanban analysis, they select a filter identifying the parts they want to run the analysis on. Using a profile variable, you can set the Kanban Manager part threshold. This threshold determines an upper limit on the number of kanban parts to be included in an analysis before users receive a warning that processing time may be negatively impacted. By default, this value is set to 50.

To set the Kanban Manager part threshold

1. From the **View** menu, click **Macros and Profile Variables**.
2. Click the **Profile Variables** tab.
3. Locate the profile variable named **KanbanPartThreshold**, and specify a new value in the **Data Value** column.

Note: For complete information on working with profile variables, see the *RapidResponse Resource Authoring Guide*.

Controlling Kanban Manager calculations

Kanban Manager calculations are controlled through a series of macros included with RapidResponse. Kanban Manager macro names typically begin with either “ShopKanban” or “SourceKanban” to indicate whether the macro is used in calculations populating the **ShopKanban** or **SourceKanban** table. These macros can be viewed and edited in the Macros and Profile Variables workbook (accessible from the **View** menu in RapidResponse), and are outlined in the following table.

Table 50.2: Kanban Manager macros

Macro	Description
IsShopKanbanPart	In the context of the **Part** table, determines if the part is a ShopKanban part.
IsSourceKanbanPart	In the context of the **PartSource** table, determines if the part is a SourceKanban part.
ShopKanbanAvgDemandCalc	In the context of the **DependentDemand** table, calculates the average daily demand for the part within the kanban horizon. For more information on this calculation, see "Average daily demand" on page 2240.
ShopKanbanAvgDemandFilter	In the context of the **DependentDemand** table, returns the appropriate DependentDemand records pertaining to the part. These records are used when calculating average daily demand for the part.
ShopKanbanCalcBinQty	In the context of the **ShopKanban** table, calculates the recommended bin quantity for the part at this shop floor location. For more information on this calculation, see "Recommended bin quantity" on page 2241.
ShopKanbanCalcNumBins	In the context of the **ShopKanban** table, calculates the recommended number of bins that should exist for the part at this shop floor location. This macro accepts one parameter (QtyParam) which represents the bin quantity to use when calculating the recommended number of bins. For example, you can specify ShopKanbanCalcBinQty to use the Kanban Manager recommended bin quantity, or BinQty to use the existing bin quantity (as is done in the calculation that populates **ShopKanban.RecommendedBins**). For more information on this calculation, see "Recommended number of bins" on page 2241.
SourceKanbanAvgDemandCalc	In the context of the **PlannedOrder** table, calculates the average daily demand for the part within the kanban horizon. For more information on this macro, "Average daily demand" on page 2242
SourceKanbanAvgDemandFilter	In the context of the **PlannedOrder** table, returns the appropriate PlannedOrder records for the part. These records are used when calculating average daily demand for the part.

Table 50.2: Kanban Manager macros

Macro	Description
SourceKanbanCalcBinQty	In the context of the **SourceKanban** table, calculates the recommended bin quantity for this part and source combination. For information on this calculation, see "Recommended bin quantity" on page 2243.
SourceKanbanCalcNumBins	In the context of the **SourceKanban** table, calculates the recommended number of bins that should exist for this part and source combination. This macro accepts one parameter (QtyParam) which represents the bin quantity to use when calculating the recommended number of bins. For example, you can specify SourceKanbanCalcBinQty to use the Kanban Manager recommended bin quantity, or BinQty to use the existing bin quantity (as is done in the calculation that populates **SourceKanban.RecommendedBins**). For more information on this calculation, see "Recommended number of bins" on page 2243.

Note: All RapidResponse macros can be customized. For Kanban Manager, typically only the ShopKanbanCalcBinQty, ShopKanbanCalcNumBins, SourceKanbanCalcBinQty, and SouceKanbanCalcNumBins are considered for customization. For complete information on working with macros, see the *RapidResponse Resource Authoring Guide*.

CHAPTER 51: Multi-enterprise resources

RapidResponse includes the Multi-Enterprise Inventory and Multi-Enterprise Management workbooks to allow brand owners to view and coordinate supply information from their suppliers across multiple sites.

In this chapter you'll learn about:

Configuring the Multi-Enterprise Inventory workbook 2249
Configuring the Multi-Enterprise Management workbook 2250
Configuring multi-enterprise data 2251

Configuring the Multi-Enterprise Inventory workbook

Multi-enterprise inventory calculations are controlled with macros, which are installed with RapidResponse. By editing these macros, you can change how RapidResponse calculates inventory levels and supply replenishment plans in a multi-enterprise environment. The macros can be viewed and edited in the Macros and Profile Variable workbook (accessible from the **View** menu in RapidResponse), and are outlined in the following table.

Table 51.1: Multi-Enterprise Inventory macros

Macro	Description
RR_MEM_ BrandReplenPlan	In the context of the **ScheduledReceipt** table, determines if the scheduled receipt is part of a brand owner's replenishment plan. A replenishment plan is a set of the brand owner's demands that a supplier's scheduled receipt is expected to fill.
RR_MEM_ BrandSite	In the context of the **Part** table, determines if the part's site belongs to the brand owner.
RR_MEM_ SupplierSite	In the context of the **Part** table, determines if the part's site belongs to a supplier.
RR_MEM_ CMReplenPlan	In the context of the **ScheduledReceipt** table, determines if the scheduled receipt is part of a supplier's replenishment plan.
RR_MEM_ RequirementTarget	In the context of the **Part** table, calculates a demand target that is used to determine if a replenishment plan is insufficient to meet demand requirements.

Note: All RapidResponse macros can be customized. However, for the Multi-Enterprise Inventory workbook, typically only the RR_MEM_RequirementTarget macro would be considered for customization. For complete information about macros, see the *RapidResponse User Guide*.

Configuring the Multi-Enterprise Management workbook

Multi-enterprise management operations are controlled with macros, which are installed with RapidResponse. By editing these macros, you can change how RapidResponse handles multi-enterprise data and calculations. The macros can be viewed and edited in the Macros and Profile Variables workbook (accessible from the **View** menu in RapidResponse), and are outlined in the following table.

Table 51.2: Multi-Enterprise Management macros

Macro	Description
RR_MEM_BrandSite	In the context of the **Part** table, determines if the part's site belongs to the brand owner.
RR_MEM_ HorizonLimit	In the context of the **CTPActivity** table, defines a time horizon for examining supply and demand requirements. This macro accepts one parameter (Part), which represents the part being examined.
RR_MEM_ HubDateCalc	In the context of the **CTPActivity** table, calculates a time offset for transporting supply from a hub (warehouse) site to a brand owner site. Within this offset, the supply is considered to be at the brand owner site. This macro accepts one parameter (BaseDate), which represents the date the supply is expected to leave the hub site.
RR_MEM_HubSite	In the context of the **Part** table, determines if the part's site is a hub or warehouse site.
RR_MEM_ HubIntransit	In the context of the **ScheduledReceipt** table, determines if the order is in transit to a hub site.
RR_MEM_ BrandIntransit	In the context of the **ScheduledReceipt** table, determines if the order is in transit to a brand owner site.
RR_MEM_ HubInventory	In the context of the **CTPActivity** table, determines if an on hand supply is located at a hub site.
RR_MEM_ OnHandDestination	In the context of the **CTPActivity** table, determines the destination of an on hand supply.
RR_MEM_ OnHandSupplier	In the context of the **CTPActivity** table, determines the supplier of an on hand supply.
RR_MEM_ OutsourcedPart	In the context of the **Part** table, determines if a part is made by a contract manufacturer or obtained from a supplier.
RR_MEM_ShipPlan	In the context of the **ScheduledReceipt** table, determines if a part has been committed to a brand owner.
RR_MEM_ SupplierSite	In the context of the **Part** table, determines if the part's site belongs to a supplier.

Note: All RapidResponse macros can be customized. However, for the Multi-Enterprise Management workbook, typically only the RR_MEM_HubDateCalc and RR_MEM_HorizonLimit macros would be considered for customization. For complete information about macros, see the *RapidResponse Resource Authoring Guide*.

Configuring multi-enterprise data

The multi-enterprise workbooks include data from multiple sites and companies. Because not all companies use the same part names, this data could potentially cause confusion by showing a mix of different names for the same part. By enabling reference part functionality, a reference part toggle button is added to all workbooks that include the Part control. This button allows for toggling between showing or not showing your company's name for multi-enterprise parts.

To show or hide reference parts, a global setting can be configured.

To enable or disable the reference part toggle

1. In the **Administration** pane, under **System Settings**, click **Global Settings**.
2. Expand the **Client** category.
3. In the **Allow toggling to reference part** list, select one of the following:
   - **No**—the reference part toggle button is not included in workbooks displaying a Part control.
   - **Yes**—the reference part toggle button is included in workbooks displaying a Part control.
4. Click **OK**.

CHAPTER 52: Data validation resources

RapidResponse includes the Data Integrity - Outsourced Supply and Data Integrity - Distribution workbooks, which report potential problems with the data you have imported from your enterprise data sources. The Data Integrity - Outsourced Supply workbook is used to validation data in an outsourced manufacturing environment, and the Data Integrity - Distribution workbook is used to validate data in a demand management environment. For each type of environment, you can configure the site types that determine whether a part is in transit, at a supplier, or at a site you own.

All data validation site definitions are controlled with macros, which are installed with RapidResponse. By editing these macros, you can change how RapidResponse determines which site types apply to each sites. The macros can be viewed and edited in the Macros and Profile Variable workbook (accessible from the **View** menu in RapidResponse).

In this chapter, you'll learn about:

Configuring outsourced manufacturing validation 2253
Configuring demand management validation 2254

Configuring outsourced manufacturing validation

Site definitions are the most important information you must specify for the Data Integrity - Outsourced Supply workbook. Sites in an outsourced manufacturing environment can belong to one of the following the types:

- **Brand**—Brand sites are owned by the brand owner, and tend to be the destination of supply orders.
- **Hub**—Hub sites are owned by the brand owner, and are used to direct supply from the supplier to one or more brand sites.
- **Supplier**—Supplier sites belong to the supplier, and represent the entry points of supply into the brand owner's supply network.

You can also specify how demands at supplier sites are defined, how a part is defined as being in transit, and how a supply plan is defined at a brand site, among other configurations. The calculations are controlled by the macros outlined in the following table.

Table 52.1: Outsourced manufacturing macros

Macro	Description
RR_MEM_ BrandInTransit	In the context of the **ScheduledReceipt** table, determines if the order is in transit to a brand owner site.
RR_MEM_ BrandSite	In the context of the **Part** table, determines if the part's site belongs to the brand owner.
RR_MEM_ HubInTransit	In the context of the **ScheduledReceipt** table, determines if the order is in transit to a hub site.
RR_MEM_ HubSite	In the context of the **Part** table, determines if the part's site is a hub or warehouse site.
RR_MEM_ SupplierDemand	In the context of the **IndependentDemand** table, determines whether the supplier has demands for the part.
RR_MEM_ SupplierSite	In the context of the **Part** table, determines if the part's site belongs to a supplier.
RR_MEM_ Supplier_ Supply_ Part	In the context of the **Part** table, determines whether a part at a supplier site is the brand owner's part.
RR_MEM_ SupplyPlan	In the context of the **ScheduledReceipt** table, determines if the scheduled receipt is part of a supplier's supply plan. A supply plan is a set of the supplier's scheduled receipts that satisfy demands at the brand sites.

Note: All RapidResponse macros can be customized. For outsourced manufacturing, typically only the RR_MEM_BrandSite, RR_MEM_HubSite, and RR_MEM_SupplierSite macros should be considered for customization. For more information about macros, see the *RapidResponse Resource Authoring Guide*.

Configuring demand management validation

Site definitions are the most important information you must specify for the Data Integrity - Distribution workbook. Sites in a demand management environment can belong to one of the following types:

- **Direct fulfillment**—Direct fulfillment sites are owned by a supplier, and ship supplies to your customers.
- **Distribution centers**—Distribution centers are sites where customer orders are fulfilled. Distribution centers receive supply from suppliers and ship supply to customers.
- **Vendor-managed inventory**—Vendor-managed inventory (VMI) sites are customer sites that you manage. VMI sites receive supply from your distributions centers. Demand at VMI sites is always considered to be satisfied by on hand inventory.

You can also configure how transfer part sources are determined to be transferring in to or out of a site, and how buy part sources are determined to be internal or external purchases. The calculations are controlled by the macros outlined in the following table.

Table 52.2: Demand management validation macros

Macro	Description
RR_Dist_ PSEffective	In the context of the **PartSource** table, determines whether a part source is effective, based on the part's run date.
RR_DistSite	In the context of the **Part** table, defines the part's site's type.
RR_ DistSite_DC	In the context of the **Part** table, determines whether the part's site is a distribution center.
RR_ DistSite_ DCText	Defines the text displayed in site type columns for distribution centers.
RR_ DistSite_ Direct	In the context of the **Part** table, determines whether the part's site is a direct fulfillment site.
RR_ DistSite_ DirectText	Defines the text displayed in site type columns for direct fulfillment sites.
RR_ DistSite_ VMI	In the context of the **Part** table, determines whether the site is a vendor-managed inventory site.
RR_ DistSite_ VMIText	Defines the text displayed in site type columns for vender-managed inventory sites.
RR_ ExternalBuy	In the context of the **PartSource** table, determines whether a Buy part source is purchasing supply from a site external to your company, such as a supplier.
RR_ InternalBuy	In the context of the **PartSource** table, determines whether a Buy part source is purchasing supply from a site inside your company. Internal Buy part sources are treated as Transfer part sources.
RR_Transfer	In the context of the **PartSource** table, determines whether a Transfer part source has different values in the Part and TransferPart fields. This indicates the part has been transferred, and is using different name (for example, changing from the supplier's name to your name) or is present at a different site (for example, changing from the supplier's site to a distribution center).

Note: All RapidResponse macros can be customized. For more information about macros, see the *RapidResponse Resource Authoring Guide*.

CHAPTER 53: Clear to Build resources

RapidResponse includes the Clear to Build workbook to allow manufacturers to view production orders that are blocked (cannot be completed) due to unavailable supply.

In this chapter you'll learn about:

Configuring Clear to Build calculations 2257
Configuring orders in the Clear to Build workbook 2258

Configuring Clear to Build calculations

Clear to Build workbook calculations are controlled with macros, which are included with RapidResponse. By editing these macros, you can change how RapidResponse determines which orders cannot be completed in a production environment. The macros can be viewed and edited in the Macros and Profile Variable workbook (accessible from the **View** menu in RapidResponse), and are outlined in the following table.

Table 53.1: Clear to Build workbook macros

Macro	Description
RR_ ClearToBuild	In the context of the **SupplyDemand** table, determines if the date an order is available to start production comes before the production part's first run date. If true, the order is clear. Otherwise, it is blocked.
RR_CtbExcess	In the context of the **OnHand** table, determines if the nettable on hand quantity for a part is greater than the total demand for the part. This macro accepts one parameter (TargetPart), which defines the part for which excess is calculated.
RR_CtbPrdnSR	In the context of the **ScheduledReceipt** table, determines if a scheduled receipt is required for an order that is in production.
RR_ CtbProduction	In the context of the **SupplyDemand** table, determines if a scheduled receipt is required for an order that is in production. This macro accepts one parameter (SR), which defines the scheduled receipt for which production is determined.
RR_ CurrentCPO	In the context of the **SupplyDemand** table, determines if a planned order is currently ready to begin production. This macro accepts one parameter (CPO), which determines the planned order for which current readiness is determined.
RR_CurrentSR	In the context of the **SupplyDemand** table, determines if a scheduled receipt is required for an order that is currently ready to begin production. This macro accepts one parameter (SR), which determines the scheduled receipt for which current readiness is determined.

Note: All RapidResponse macros can be customized. For complete information about macros, see the *the RapidResponse Resource Authoring Guide*.

Configuring orders in the Clear to Build workbook

Using workbook variables, you can define how orders ready for production are identified in the Clear to Build workbook, and configure how order numbers are assigned to new firm orders that are created from planned orders.

To configure orders in the Clear to Build workbook

1. In the **Workbooks** pane, right-click the **Clear to Build** workbook, and then click**Properties**.
2. Click the **Variables** tab.
3. Select either of the following variables to be modified:
   - **CTBProductionValue—**indicates the order priority used to determine if an order is ready to begin production.
   - **RR_FirmProductionOrder—**indicates the order number assigned to a new firm order created from planned order data.
4. Click **Edit**.
5. In the **Initial value** area, specify a default value for the variable.

Note: For complete information about working with workbook variables, see the *RapidResponse User Guide*.

Part 4: Analytic and data model change summary

- Data model calculations after upgrading
- Analytic and data model changes in 2019
- Analytic and data model changes in 2018
- Analytic and data model changes in 2017
- "Analytic and data model changes in 2016" on page 2389

CHAPTER 54: Data model calculations after upgrading

All new data model analytics or changes to existing analytics are designed to allow backwards compatibility with existing configurations; that is, the changes will not impact the results RapidResponse was previously generating. The default settings for each of the new configurable data model features is set so that the calculated results from RapidResponse after upgrade are the same as the results before upgrade.

A common upgrade practice is to "stage" the upgrade on a backup server comparing the before and after calculation results. Kinaxis has performed this activity on a number of customer data sets as part of our quality assurance efforts. As part of the comparison process you might notice differences in results. These result differences are considered "explainable" and can be attributed to corrected defects in the data model analytic calculations.

When comparing or validating results, you should review three top-level parts to ensure independent demand available dates align.

After you upgrade RapidResponse, all queries, data in custom applications, and inserts continue to be compatible with the new data model; that is, errors are not generated.

The following changes can occur in the data model after an upgrade:

- A field is moved from one table to another.
- A field is deleted from a table.
- A field is added to a table.
- A table is added.

Fields that have been moved or removed in the upgraded data model are still available; however, they are read-only.

Fields that have been added to the data model are given default values to ensure that operations function. The default value for a given field can be viewed in the Data Model dialog box by accessing the field's properties. If necessary, you can specify a different default value to override the initial default. For more information about the Data Model dialog box, see "Data Model dialog box" on page 147.

CHAPTER 55: Analytic and data model changes in 2019

Details on new and changed features in the following 2019 releases are available:

- Service Update 1908
- Service Update 1907
- Service Update 1906
- Service Update 1905
- Service Update 1904
- Service Update 1903
- Service Update 1902
- "Service Update 1901" on page 2281
- Version 2016.2 changes Service Update 30
- "Version 2016.2 changes Service Update 29" on page 2284
- Version 2016.2 changes Service Update 28
- Version 2016.2 changes Service Update 27

Service Update 1908

This section provides a summary of changes made in Service Update 1908 that affect the data model and RapidResponse calculations. The following changes are discussed in this section:

- Analytic and calculation changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Service Update 1908 including:

- Planning time fence enhancement
- Limited availability of MLS infrastructure changes
- Defect fixes and calculation changes

Planning time fence enhancement

Enhancements have been made to how scheduled receipt and supply allocation due dates can impact the planning time fence calculation.

Previously, the last due scheduled receipt or supply allocation associated with a given part source could be used in that part source's **PTFDate** calculation. That is, when **OrderPolicy.PTFRule** was set to "LastDueLead", the **PTFDate** was based on the latter of lead time or the last due scheduled receipt or supply allocation associated with the part source. Similarly, when the **PTFRule** was "LastDueFence", the **PTFDate** was based on the **PlanningTimeFence** field or the last due scheduled receipt or supply allocation associated with the part source (if later).

This service update introduces the following optional enhancements:

- Scheduled receipts and supply allocations across of all a part's sources can now be considered when calculating the **PTFDate** for a specific part source. Two new settings, "LastDueLeadAll" and "LastDueFenceAll", are added to the **OrderPolicy.PTFRule** field. These are identical to the existing "LastDueLead" and "LastDueFence" settings other than that they use the last due scheduled receipt or supply allocation found on any part source that provides the part. For example, this can help ensure that the **PTFDate** is synchronized across all of a part's sources.
- Specific types of scheduled receipts and supply allocations can now be excluded from the **PTFDate** calculations when using any of the "LastDue" options. This is configured through a new "PTFLastDueRule" field that has been added to the **SupplyType**, **SupplyStatus**, and **SupplyAllocationType** tables. For example, depending on what certain **SupplyType** and **SupplyStatus** records represent in your system, this can help ensure that only due dates on appropriate scheduled receipts will influence the planing time fence calculation.

For more information, see "Determining the PTFDate" on page 1344.

Limited availability of MLS infrastructure changes

A future service update, expected to ship in early 2020, will introduce a significant re-architecture of the underlying infrastructure supporting multi-level search logic. This change will mostly impact the

expected performance and stability of the multi-level search analytic, while also supporting future scalability and enhancements. Additionally, these changes might result in some solution output differences under specific conditions.

As a result, this service update includes the option to enable the new multi-level search infrastructure in any given scenario(s). This gives the opportunity to evaluate any potential performance or solution differences that might be seen when the new infrastructure replaces the existing and report any related issues as necessary.

For more information about enabling this new MLS infrastructure and the expected improvements or differences that might be seen with its implementation, see "MLS infrastructure changes (optional and limited release)" on page 1929.

Note: In order to enable the new MLS infrastructure, the latest version of the **MLS Configuration** workbook is required. The new **MLS Performance Manager** dashboard is also available to help evaluate the impact of the new infrastructure.

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Control table changes

New fields

This section outlines new fields added to Control tables in Service Update 1908.

Table 55.1: New Control table fields

Table	Field/name change
SupplyAllocationType	PTFLastDueRule
SupplyStatus	PTFLastDueRule
SupplyType	PTFLastDueRule

Modifed fields

This section outlines Control tables that were modified in Service Update 1908.

Table 55.2: Modify Control table fields

Table	Field/name change
OrderPolicy	PTFRule Two new options, LastDueLeadAll and LastDueFenceAll, are added.

Service Update 1907

RapidResponse Service Update 1907 contains no data model changes nor any enhancements to RapidResponse analytic calculations.

However, this Service Update includes analytic and data model defect fixes, some of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Service Update 1906

This section provides a summary of changes made in Service Update 1906 that affect the data model and RapidResponse calculations. The following changes are discussed in this section:

- Analytic and calculation changes
- "Calculated table changes" on page 2268
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Service Update 1906 including:

- Backload constraint scheduling
- Defect fixes and calculation changes

Backload constraint scheduling

Normally during constraint scheduling, an initial backward phase is performed that plans prioritized supplies and loads them onto available constraints. This is then followed by a forward phase that adjusts the initial scheduling results with the aim of ensuring that supplies use constraint in continuous chunks where possible. This can reduce fixed constraint usage and can minimize costly constraint switch overs (for example, it might take time to prepare a machine to process a new part)

In this Service Update, a new option (**ConstraintType.SchedulingRule**) is added that allows for the forward phase of constraint scheduling to be disabled (thus, constraints are back loaded only). This is intended for certain cases where normal constraint scheduling might produce sub-optimal results. Typically, this occurs in cases where a part requires multiple constraints and the forward scheduling phase sometimes has to look several periods out to find sufficient parallel availability of all required constraints. This can result in unnecessarily late demands and, when partial planning logic is enabled, might result in demands being satisfied late and beyond tolerance.

For more information, see "SchedulingRule" on page 1667.

Data change tracking

Data changes tracked in the Scenario properties worksheets (Pending Commit, Inherited Changes, and Pending Updates) are now formatted for display by the process that collects the changes. This replaces the previous behavior of collecting changes as a single string value and processing the record into its identifying and modified fields as part of displaying the changes in the scenario properties window. The data change records are now stored in the format the worksheet expects, and the processing step is no longer required when a user views the data changes.

Storing the data change records as formatted records improves performance by not processing the records every time they are viewed, however, because the results are processed as they are saved, they are now subject to calculation memory limits. For more information about setting calculation memory limits, see the *RapidResponse Administration Guide*

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Calculated table changes

New tables

This section outlines new calculated tables in Service Update 1906.

Table	Description
"DataChange_ Historical_Split table" on page 1011	Similar to the DataChange_Historical table, but contains separate records for each identifying and modified field in the DataChange record. This table contains DataChange records formatted for display in change tracking resources.
"DataChange_ Local_Split table" on page 1016	Similar to the DataChange_Local table, but contains separate records for each identifying and modified field in the DataChange record. This table contains DataChange records formatted for display in change tracking resources.
"DataChange_ PendingUpdate_ Split table" on page 1022	Similar to the DataChange_PendingUpdate table, but contains separate records for each identifying and modified field in the DataChange record. This table contains DataChange records formatted for display in change tracking resources.

Control table changes

New fields

This section outlines new fields added to Control tables in Service Update 1906.

Table 55.3: New Control table fields

Table	Field/name change
ConstraintType	SchedulingRule

Service Update 1905

This section provides a summary of changes made in Service Update 1905 that affect the data model and RapidResponse calculations. The following changes are discussed in this section:

- Analytic and calculation changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Service Update 1905 including:

- "Safety stock priority adjustments (demand transformations)" on page 2269
- Defect fixes and calculation changes

Safety stock priority adjustments (demand transformations)

An enhancement has been made to how priority is used when planning supply to meet safety stock requirements. Previously, safety stock requirements were always seen as lower priority demands which were then satisfied by supplies that were assigned the part's default priority level (typically, a low priority). If there was a drop in the required safety stock level, then the released safety stock supply became eligible to satisfy real demands. However, this could result in higher priority demands being satisfied by low priority safety stock supplies and in some cases made later than their high priority dictated they should be.

In this service update, a new control setting, **DemandTransformationRule**, is added to the **SafetyStockPolicy** table. The default setting of "Ignore" provides backward compatibility with previous versions and can also potentially lead to the issue described above.

Instead if the **DemandTransformationRule** is set to "FirstInFirstOut", then new logic is introduced whereby supply is always assumed to eventually satisfy a real demand and the supply's priority is then taken from that real demand. This ensures that higher priority demands still have their priority preserved and propagated throughout the product structure even when satisfied by supply that was initially planned because of a safety stock requirement.

For more information, see "Safety stock and priority adjustments" on page 1773.

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Control table changes

New fields

This section outlines new fields added to Control tables in Service Update 1905.

Table 55.4: New Control table fields

Table	Field/name change
SafetyStockPolicy	DemandTransformationRule

Service Update 1904

RapidResponse Service Update 1904 contains no data model changes nor any enhancements to RapidResponse analytic calculations.

However, this Service Update includes analytic and data model defect fixes, some of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Service Update 1903

This section provides a summary of changes made in Service Update 1903 that affect the data model and RapidResponse calculations. The following changes are discussed in this section:

- Analytic and calculation changes
- Input table changes
- Calculated table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Service Update 1903 including:

- "Multi-level search (MLS) enhancements" on page 2272
- Best Fit enhancement
- Self-healing yield
- Defect fixes and calculation changes

Multi-level search (MLS) enhancements

The following changes have been made to multi-level search logic:

- **Improved inventory pooling support-** Improvements have been made and certain limitations removed that existed MLS was combined with inventory pooling logic. The main change is that multi-level search logic will now compare and evaluate whether to provide supply from the demand pool and common pool (when applicable). This supports better use of MLS for parts that use a **MUEPoolNettingType.PoolRule** setting of "Net" and when the demand pool is different from the part's common pool (in particular, cases where the **MUEPoolNettingType.PoolFlowToCommon** setting is "NetCommon" or "NetOriginal"). Additionally a new part source selection criterion is added that supports the use of MLS with inventory pooling logic as described below.
- **Selection criteria (substitution/pooling-** when using either global part substitution and/or inventory pooling logic along with MLS, a new scheduled receipt selection criterion is added and evaluated after supply level and supply type:
  - When using MLS with global part substitution and a **PartType.SubstitutionPreference** of "ScheduledSupply", if multiple sources can satisfy a demand equally well and are considered equivalent in priority and depth/level, then a source that can provide supply from an allocated or unallocated scheduled receipt at a given level in the product structure is preferred over one that requires a new planned order.
  - When using MLS with using inventory pooling logic, if multiple sources can satisfy a demand equally well and are considered equivalent in priority and depth/level, then a source that can utilize an allocated or unallocated scheduled receipt in one pool (for example, the common pool) is preferred over a sources that requires a new planned order to be created in another pool (for example, the demand pool).

  Note that previously an unallocated scheduled receipt was considered equivalent to a new planned order in multi-level selection criteria. With this change, when using MLS along with global

part substitution or inventory pooling logic, the need for new planned orders on certain parts might be reduced or delayed in cases where there are unallocated scheduled receipts that can be allocated and used up first. For more information, see "Multi-level part source selection criteria" on page 1877.

Best Fit enhancement

A new setting allows for model set parameters on the BestProfileDetail table, which are used for fine-tuning best fit forecasting results, to be overridden by equivalent values on the ForecastItemParameters record.

Self-healing yield

Calculations changes and many new data model tables and fields were added for the Self-Healing Supply Chain capability which now supports yield correction (previously, only lead-time correction was supported).

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Input table changes

New tables

This section outlines new input tables added in Service Update 1903.

Table	Description
"HistoricalReceiptHeader table" on page 345	New header table referenced by the new HistoricalReceipt table. Used by the Self-Healing Supply Chain
"HistoricalReceipt table" on page 345	Contains historical receipt information relating to yields used by the Self-Healing Supply Chain.
"ProjectedYield table" on page 506	Contains yield values projected by the Machine Learning server for a set of ItemSource records. The projected yields are processed based on the AutomatedYieldType settings for the ItemSource. Used by the Self-Healing Supply Chain.

New fields

This section outlines fields added to Input tables in Service Update 1903.

Table 55.5: Input table fields

Table name	Field name/change
"ForecastItemParameters table" on page 296	IgnoreBestFitProfileDetailParameters Ignores the model parameter values that are associated with the Best Fit profile that are stored in **BestFitProfileDetails** and applies the model parameter values from the **ForecastItemParameters** record to the models in the Best Fit profile instead.
"Id" on page 379	Id Used by the Self-Healing Supply Chain. An extra key allowing item sources with the same Part and Source to have various BOMAlternate and Routing combinations.
"BOMAlternate" on page 379	BOMAlternate Used by the Self-Healing Supply Chain. A reference to the alternate BOM for the ItemSource.
"Routing" on page 380	Routing Used by the Self-Healing Supply Chain. A reference to the alternate Routing for the ItemSource.
"YieldCluster" on page 381	YieldCluster Used by the Self-Healing Supply Chain. A reference to the Cluster associated with the ItemSource for Yields.
"AutomatedYieldType" on page 379	AutomatedYieldType Used by the Self-Healing Supply Chain. Defines how projected yields for a given ItemSource are processed by the Self-Healing Supply Chain.
"AcceptYieldChanges" on page 379	AcceptYieldChanges Used by the Self-Healing Supply Chain. Used to manually accept projected yields for a given ItemSource.
"YieldCorrectionStatus" on page 381	YieldCorrectionStatus Used by the Self-Healing Supply Chain. Specifies whether or not the given ItemSource has been reviewed for yield correction or not. Valid values are **Reviewed** and **NotReviewed**.
"CalcHistoricalYieldError" on page 383	CalcHistoricalYieldError Calculated field used by the Self-Healing Supply Chain. A measure of the error between modeled yields (from PartSource.Yield and TimePhaseYield.Yield) and observed yields (from HistoricalReceipt,StartQuantity and HistoricalReceipt.ReceivedQuantity).
"CalcProjectedYieldCOV" on page 384	CalcProjectedYieldCOV Calculated field used by the Self-Healing Supply Chain. Coefficient of variation of ProjectedYields.Yield for this ItemSource.
"CalcYieldZone" on page 385	CalcYieldZone Calculated field used by the Self-Healing Supply Chain. The zone (inside, near or outside tolerance) into which this ItemSource falls.

Table 55.5: Input table fields

Table name	Field name/change
"CalcUseYieldChanges" on page 385	CalcUseYieldChanges Calculated field used by the Self-Healing Supply Chain. Whether the values recommended in the ProjectedYield table should be applied, either because they should be accepted automatically, or because the user has manually accepted them.
"CalcYieldUsageRule" on page 385	CalcYieldUsageRule Calculated field used by the Self-Healing Supply Chain. The yield usage rule applicable to the ItemSource.
"SelfHealingParameter" on page 243	SelfHealingParameter Used by the Self-Healing Supply Chain. Allows the Cluster table to differentiate between Lead Time and Yields for Self-Healing Supply Chain analysis.

Control table changes

New tables

This section outlines new Control tables added in Service Update 1903.

Table 55.6: New Control tables

Table	Description
"AutomatedYieldType table" on page 683	

Calculated table changes

New tables

This section outlines new calculated tables added in Service Update 1903.

Table	Description
"ItemSourcePartSourceHistoricalReceipt table" on page 1091	

New fields

This section outlines calculated fields that were added in Service Update 1903.

Table 55.7: New Calculated table fields

Table name	Field name/change
"BOMAlternate" on page 1088	BOMAlternate Used by the Self-Healing Supply Chain. A reference to the BOMAlternate for the ItemSourceCandidate.
"Routing" on page 1088	Routing Used by the Self-Healing Supply Chain. A reference to the Routing for the ItemSourceCandidate.

Deleted fields

This section outlines calculated fields that were removed in Service Update 1903.

Table 55.8: Deprecated Calculated table fields

Table name	Field name/change
"HistoricalSupplyCategory" on page 1088	HistoricalSupplyCategory Previously used by the Self-Healing Supply Chain. Field removed from the ItemSourceCandidate table.

Service Update 1902

This section provides a summary of changes made in Service Update 1902 that affect the data model and RapidResponse calculations. The following changes are discussed in this section:

- Analytic and calculation changes
- Input table changes
- Calculated table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Service Update 1902 including:

- Safety lead time calendar
- Component need dates under independent demands
- "Production Planning calculations" on page 2278
- Defect fixes and calculation changes

Safety lead time calendar

In previous versions, part and part source safety lead time values were always assumed to be expressed in terms of the **TimeUnits** calendar (the part's working calendar). The new **PartType.SafetyLeadTimeCalendarRule** setting now gives greater control over the calendar in which a part's safety lead time is defined. This control field allows toggling between use of the part's referenced **PlanningCalendars.TimeUnits** calendar or the "Everyday" (Gregorian) calendar.

For example, the dynamic safety stock methods of "TimePhasedSafetyDaysTotal" and "TimePhasedSafetyDaysMaximum" use the "Everyday" calendar for demand collection (by default). If using these safety stock methods, it might be preferable to express safety lead time in the same "Everyday" calendar as is used in the safety stock logic.

For more information, see "Safety lead time calendar" on page 1322.

Component need dates under independent demands

The new **FlatDemand** table reports the date on which each component would be needed for on time fulfillment of a given independent demand. This table determines need dates by exploding through the bill of material under the end-item assembly and offsetting for lead time at each level of the structure. This is done even if some higher level parts in the structure might be satisfied by on-hand inventory or existing supply.

For example, this table might be used to help fairly compare independent demands at a constrained level without biasing the distribution of supply due to relative lead times.

For more information, see FlatDemand table.

Production Planning calculations

New production planning logic allows you Production Planning calculations allow you to use constraint to model the production of parts on production wheels. Setup and cleanup activities are accounted for between groups of parts and product sequences can be used to predefine the order of manufacture to improve capacity utilization. In addition, you can apply segmentation strategies to production and use production recurrence to align production frequency.

For more information, see "Production Planning calculations" on page 1691.

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Input table changes

New tables

This section outlines new input tables added in Service Update 1902.

Table	Description
PartSequence table	Stores the properties for part sequencing used in production wheels.
ProductionGroup	Stores the properties for a production group including the constraint utilized in changeovers.
ProductionRecurrence	Stores the definitions that determine how often a part can be produced in a production wheel.
ProductionWheel	Stores the parameters that control production wheel cycles.
ProductionWheelConstraint	Stores the constraint parameters that affect production wheels.
Segment	Stores the information on the segments a production wheel is divided into.

New fields

This section outlines fields added to Input tables in Service Update 1902.

Table 55.9: Input table fields

Table name	Field name/change
PartSource	ProductionRecurrence A reference to the production recurrence associated with the part source.
PartSource	ProductionGroup A reference to the production group associated with the part source.
PartSource	ProductionWheel A reference to the production wheel associated with the part source.

Table 55.9: Input table fields

Table name	Field name/change
Site	ProductionGroups The set of ProductionGroup records associated with a site.
Site	ProductionWheels The set of ProductionWheel records associated with a site.
Site	Segments The set of Segment records associated with a site.

Control table changes

New tables

This section outlines new Control tables added in Service Update 1902.

Table 55.10: New Control tables

Table	Description
DDMRPRule	This table contains DDMRP related configuration settings. It is used to identify the demand-processing behavior settings used to turn DDMRP calculations on or off for parts using this rule. This table supports the DDMRP application. This control table turns on Decoupled Lead Time calculations for each part.
ProductionWheelType	This table defines the rules and values used to manage production wheels in RapidResponse.

New fields

This section outlines new fields added to control tables in Service Update 1902.

Table 55.11: New Control table fields

Table name	Field name
PartType	SafetyLeadTimeCalendarRule Allows for safety lead time to be expressed in terms of either the part's TimeUnits calendar or the Everyday calendar. By default, the part's TimeUnits calendar is used (as was always used in previous versions).

Calculated table changes

New tables

This section outlines new calculated tables added in Service Update 1902.

Table	Description
FlatDemand	This table reports the dates that all of a part's components would be needed to satisfy a given IndependentDemand (assuming that higher-level demands are not satisfied by existing supplies/inventory).
ProductionCycle	Reports on the cycles of production wheels in the system including any changes in constraint availability.

New fields

This section outlines calculated fields that were added in Service Update 1902.

Table 55.12: New Calculated table fields

Table name	Field name/change
ConstraintUsed	ProductionCycleNumber The cycle number constraint is used in.
ConstraintUsed	ProductionWheel A reference to the production wheel associated with the ConstraintUsed record.
ConstraintUsed	Type The type of constraint used during constraint scheduling.

Service Update 1901

This service update does not contain any data model changes or enhancements to RapidResponse analytic calculations. However, the following changes are discussed in this section:

- Version numbering
- Defect fixes and calculation changes

Version numbering

As of this version of RapidResponse, the version numbering now reflects which release number of the year the version is. This supports the continuous delivery of RapidResponse, and allows you to correlate the version number of your RapidResponse installation with the order of releases for that year.

For example, RapidResponse 1901 is the first release of 2019 using this version numbering.

If a letter designation appears in the version number (for example, G) this indicates the major platform version.

Defect fixes and calculation changes

This service update includes fixes to reported analytic issues, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Version 2016.2 changes Service Update 30

This section provides a summary of changes made in RapidResponse 2016.2 Service Update 30 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 30 including:

- Defect fixes and calculation changes

Defect fixes and calculation changes

Service Update 30 includes analytic and data model defect fixes, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Input table changes

New fields

This section outlines fields added to Input tables in Version 2016.2, Service Update 30.

Table 55.13: Input table fields

Table name	Field name/change
KPI	EnableUOM A boolean indicating if the KPI has a unit of measure and can be converted (intended to support writing conditional expressions and is not used by any analytic).

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 30.

Table 55.14: Modified Input table fields

Table name	Field name/change
HistoricalPartKPI	Quantity This field now has a Unit of Measure path (enables unit conversions).

Calculated table changes

New fields

This section outlines calculated fields that were added in Version 2016.2, Service Update 30

Table 55.15: New Calculated table fields

Table name	Field name/change
Forecast	ConsumedQuantity The amount of the forecast that has been consumed by sales actuals.
Forecast	EffectiveDemandQuantity The amount of the forecast for which supply should be planned/assigned (it represents unconsumed forecast outside the demand time fence).

Hidden fields

The section outlines calculated fields that were modified in Version 2016.2, Service Update 30.

Table 55.16: Hidden Calculated table fields

Table name	Field name/change
Forecast	UnconsumedQty This field reported the portion of a forecast that was unconsumed by actual demand, unless that unconsumed forecast was within the demand time fence in which case a value of 0 was reported. The **UnconsumedQty** field is now obsolete and has been hidden. Note that any existing resources that use this field will still function, however its continued usage is discouraged as the field will be removed completely in a future version of RapidResponse. For reporting consumption details in the **Forecast** table now, the new **ConsumedQuantity** and **EffectiveDemandQuantity** fields should be used: The **ConsumedQuantity** represents the amount of the original forecast **Quantity** that has been consumed by sales actuals. Thus, the difference between these two fields represents the unconsumed forecast quantity. The new **EffectiveDemandQuantity** field then reports the amount of the forecast that should be planned and will have supply assigned to it. Thus, within the demand time fence this field typically reports 0, but beyond the demand time fence it is the unconsumed forecast quantity.

Version 2016.2 changes Service Update 29

This section provides a summary of changes made in RapidResponse 2016.2 Service Update 29 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 29 including:

- Control BOM explosion for "NoOrders" parts in FlatBillDown reports
- Defect fixes and calculation changes

Control BOM explosion for "NoOrders" parts in FlatBillDown reports

You can now choose whether **FlatBillDown** calculations should explode through and report components under parts from sources where **OrderPolicy.OrderGenerationRule** is set to "NoOrders". The "NoOrders" setting is used to prevent new planned orders from being generated from a given part source. By default, parts from sources with this setting are reported in the **FlatBillDown** table but any components under them are not.

However, in some cases you might want to explode BOMs under "NoOrders" parts and include their components in **FlatBillDown** reports. For example, this might be done if there is a need to reports components used in archived finished goods or if reporting rolled up costs on historical BOMs.

To enable explosion through "NoOrders" part sources, the **RR_Analytics_FlatBill_ExplodeNoOrders** analytic parameter has been added. This parameter is enabled by adding it as a workbook variable to a workbook containing one or more worksheets based on the **FlatBillDown** table. The **RR_Analytics_FlatBill_ExplodeNoOrders** parameter accepts two values; "True" (BOM explosion continues through "NoOrders" parts) and "No" (BOM explosion stops at "NoOrders" parts).

Note also that depending on which version of RapidResponse you are upgrading from, the FlatBillDown results you are used to seeing might differ as follows:

- If upgrading from Version 2016.2 Service Update 18 or later, then **FlatBillDown** calculations did not explode through and report components under "NoOrders" parts. No changes are required to maintain this behavior (though the new parameter can be added if you want to start exploding through "NoOrders" parts).
- If upgrading from Version 2016.2 Service Update 17 or earlier, then **FlatBillDown** calculations did explode through and report components under "NoOrders" parts. To maintain this behavior, add and enable the **RR_Analytics_FlatBill_ExplodeNoOrders** variable in the appropriate workbooks and worksheets.

For more information, see "Explode BOMs for "NoOrders" parts in FlatBillDown calculations" on page 2221.

Statistical forecasting enhancements

When configuring forecast item parameters used in generating a statistical forecast, it is now possible to show confidence intervals using the Double exponential smoothing, Exponential smoothing, and Holt-Winters (multiplicative and additive) statistical forecasting models.

Defect fixes and calculation changes

Service Update 29 includes analytic and data model defect fixes, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Input table changes

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 29.

Table 55.17: Modified Input table fields

Table name	Field name/change
ForecastItemParameters	ConfidenceLevel This field can now be used by the Double exponential smoothing, Exponential smoothing, and Holt-Winters (multiplicative and additive) models.

New fields

The section outlines control fields that were added in Version 2016.2, Service Update 29.

Table 55.18: New Control table fields

Table name	Field name/change
PartType	OnTimeAllocationRule This field can be used to specify that maximizing the on-time quantity on some late demands is more important than the overall available dates on other late demands.

Calculated table changes

Modified fields

The section outlines calculated fields that were modified in Version 2016.2, Service Update 29.

Table 55.19: Modified Calculated table fields

Table name	Field name/change
StatisticalForecast	PredictionIntervalLower This field can now be used by the Double exponential smoothing, Exponential smoothing, and Holt-Winters (multiplicative and additive) models.
StatisticalForecast	PredictionIntervalUpper This field can now be used by the Double exponential smoothing, Exponential smoothing, and Holt-Winters (multiplicative and additive) models.

Version 2016.2 changes Service Update 28

RapidResponse 2016.2 Service Update 28 contains no data model changes nor any enhancements to RapidResponse analytic calculations.

However, this Service Update includes analytic and data model defect fixes, some of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Version 2016.2 changes Service Update 27

This section provides a summary of changes made in RapidResponse 2016.2 Service Update 27 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- "Control table changes" on page 2291
- "Control table changes" on page 2291

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 27 including:

- Reallocation of late supplies to improve on-time quantity
- Statistical forecasting enhancements
- Multi-level search now supports material expiry
- Defect fixes and calculation changes

Reallocation of late supplies to improve on-time quantity

A new option can be used to try and improve the on-time quantity associated with late demands. Normally when there is not enough supply to satisfy all demand on time, the demands that are of the highest priorities and/or have the earliest due dates receive supply first. The goal of this normal supply allotment process then is to make such demands on-time or at least to minimize their lateness. Instead, if improving the total on-time quantity on late demands is more important than the final available date on late demands, some of the normal supply allotments can be reallocated. That is, supplies that are late to the demand can potentially be reallocated to try and improve the on-time quantity on other late demands.

This is controlled through the new **PartType.OnTimeReallocationRule** setting. For more information, see "Improve on-time quantity by reallocating late supply" on page 1471.

Statistical forecasting enhancements

RapidResponse now includes multiple linear regression (MLR) forecasting models to generate statistical forecasts. MLR is a statistical forecasting method that examines multiple regressors to predict the future demand of a forecast item. The following multiple linear regressions models are available: ARIMAX, Elastic Net, and Multiple Linear Regression. New data model tables and fields were included to support the use of multiple linear regression models. The MLR forecasting models use the R programming language to perform calculations. In addition, the MLR forecasting models are available only if R has been enabled.

For more information, see "Multiple Linear Regression forecasting models" on page 2042.

Multi-level search now supports material expiry

Previously the material expiry analytic was not compatible with multi-level search logic. Thus when expiry was used in multi-source configurations, it would optimize part source selections in consideration of factors such as supply expiry risk. In this service update, material expiry support is added to multi-level search logic and enables expiry risk to factor into sourcing decisions as discussed in "Multi-level part source selection criteria" on page 1877. Note also that certain setting/feature limitations still exist when using multi-level search along with material expiry as discussed in "Material expiry" on page 1929.

Defect fixes and calculation changes

Service Update 27 includes analytic and data model defect fixes, many of which might lead to differences in calculated output. For more information about these fixes, see the Release Summary Help.

Input table changes

New tables

This section outlines new input tables added in Version 2016.2, Service Update 27.

Table	Description
RegressionModel	Stores the names of regression models that are defined for use with MLR statistical forecasting models.
RegressionModelTerm	Stores information about the model terms that are associated with regressors used with MLR statistical forecasting models.
Regressor	Stores records for regressors that are used with MLR statistical forecasting models.
RegressorCategory	Stores regressor categories that are used with MLR statistical forecasting models.
RegressorValue	Stores individual values for a regressor on a given date. This is used with MLR statistical forecasting models.

New fields

This section outlines fields added to Input tables in Version 2016.2, Service Update 27.

Table 55.20: Input table fields

Table name	Field name/change
BestFitProfileDetail	RegularizationParameter Use this field to set a parameter that controls the strength of the penalty term for coefficients that is used in Elastic Net regression.
Part	DDMRPRule Reference to DDMRPRule control table for each part.
Part	DecoupledLeadTime Use this field to view the calculation for the multi-site cumulative lead time between buffer points. The calculation will follow the same logic as multi-site cumulative lead time, but the calculation will stop at any decouple points.

Table 55.20: Input table fields

Table name	Field name/change
ForecastItemParameters	ElasticNetParameter Use this field to set a parameter that controls how much penalization of the L1 and L2 norms is applied when using the Elastic Net forecast model. Elastic Net combines both the LASSO and ridge regression techniques to shrink coefficients.
ForecastItemParameters	RegressionModel This field is used to reference the regression model that is used by the forecast item. This field applies only to forecast items that are using multiple linear regression to generate the statistical forecast.
ForecastItemParameters	RegularizationParameter Use this field to set a parameter that controls the strength of the penalty term for coefficients that is used in Elastic Net regression.
ForecastItemParameters	ResidualAutocorrelationConfidenceLevel Use this field to specify the value used to calculate confidence levels for residual autocorrelation when using statistical forecasts that are generated using a multiple linear regression forecasting model.
ForecastItemParameters	WeightsDecayFactor Use this field to specify the factor to use to decrease the influence of regressors over time. When WeightsDecayFactor is set, the regressor values in more recent time periods are given more importance than the values in earlier time periods in the statistical forecast calculation. It is an optional value that is used to improve model fit when using a multiple linear regression forecasting model.

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 27.

Table 55.21: Modified Input table fields

Table name	Field name/change
BestFitProfileDetail	DifferenceLevel This field now supports differencing operations for forecast items using the ARIMAX forecast model.
BestFitProfileDetail	ForecastModel This field now supports the ARIMAX, Elastic Net, and Multiple Linear Regression forecast models.
BestFitProfileDetail	SeasonalIntervalCount This field now lets you specify the duration of a season for forecast items using the Elastic Net and Multiple Linear Regression forecast models.
BestFitProfileDetail	TrendDecayFactor This field now lets you specify a decay factor for forecast items using the Elastic Net and Multiple Linear Regression forecast models.

Table 55.21: Modified Input table fields

Table name	Field name/change
ForecastItemFitOutput	ForecastModel This field now supports the ARIMAX, Elastic Net, and Multiple Linear Regression forecast models.
ForecastItemParameters	ConfidenceLevel This field now lets you set a confidence level to calculate prediction intervals in statistical forecasts that use the ARIMAX and Multiple Linear Regression forecast models.
ForecastItemParameters	Constant This field now indicates whether a constant value, also known as the Y intercept, is included in a forecast calculation for forecast items using the ARIMAX forecast model.
ForecastItemParameters	DifferenceLevel This field now supports differencing operations for forecast using the ARIMAX forecast model.
ForecastItemParameters	SeasonalIntervalCount This field now lets you specify the duration of a season for forecast items using the Elastic Net and Multiple Linear Regression forecast models.
ForecastItemParameters	TrendDecayFactor This field now lets you specify a decay factor for forecast items using the Elastic Net and Multiple Linear Regression forecast models.
ForecastItemParametersFitOutput	ForecastModel This field now supports the ARIMAX, Elastic Net, and Multiple Linear Regression forecast models.
Part	UnsatisfiedDemandTolerance This field now supports newly introduced on-time reallocation logic (was previously only used by partial planning logic).

Control table changes

Modified fields

The section outlines control fields that were modified in Version 2016.2, Service Update 27.

Table 55.22: Modified Control table fields

Table name	Field name/change
ForecastItemParametersType	ModelConstantUsage For forecast items that are using Multiple Linear Regression or the Elastic Net forecast model to generate a statistical forecast, this field now determines whether constants are calculated or the values are taken from the ForecastItemParametersRegressionCoefficient table.

Calculated table changes

New tables

This section outlines new calculated tables added in Version 2016.2, Service Update 27.

Table	Description
ForecastItemParametersRegressionWeight	Reports the regression weights for forecast items that are using an MLR forecast model.
ForecastItemParametersRegressor	Contains the values for regressors that are associated with each **RegressionModelTerm**. This is used with MLR statistical forecasting models.
StatisticalForecastResidualAutocorrelation	Reports the autocorrelation function of the residuals for forecast items that use an MLR forecast model.
StatisticalForecastResidualCount	Reports the number of observations that have residuals with quantities that are within a particular range, or bin. This is used with MLR statistical forecasting models.

Modified fields

The section outlines calculated fields that were modified in Version 2016.2, Service Update 27.

Table 55.23: Modified Calculated table fields

Table name	Field name/change
StatisticalForecastFitOuput	ForecastModel This field now supports the ARIMAX, Elastic Net, and Multiple Linear Regression forecast models.

CHAPTER 56: Analytic and data model changes in 2018

In 2018, many new and changed features were included in service updates to Version 2016.2.

Details on the following are available:

- Version 2016.2 changes Service Update 26
- Version 2016.2 changes Service Update 25
- Version 2016.2 changes Service Update 24
- Version 2016.2 changes Service Update 23
- Version 2016.2 changes Service Update 22
- Version 2016.2 changes Service Update 21
- Version 2016.2 changes Service Update 20
- Version 2016.2 changes Service Update 19
- Version 2016.2 changes Service Update 18
- Version 2016.2 changes Service Update 17
- Version 2016.2 changes Service Update 16
- Version 2016.2 changes Service Update 15

Version 2016.2 changes Service Update 26

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 25 to RapidResponse 2016.2 Service Update 26 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 26 including:

- Defect fixes and calculation changes

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 26

Table 56.1: Defect fixes and calculation changes in Version 2016.2, Service Update 26

Fix Area / Change	Description
Multi-level search (MLS)	The following issues where fixed: • An unexpected increase in new purchase orders was observed in certain situations in SU24 as compared to SU19.1, and was related to an issue in the logic where MLS tries to minimize new purchase costs. • PartSource.SafetyLeadTime was not applied correctly if MLS was used along with constraints. • Part.SafetyLeadTime was not applied correctly if MLS was used along with constraints and there were transfer part sources. • More scheduled receipts than necessary were pulled in in some situations thereby leading to excess balance.
Pooling	An internal server error would sometimes occur when doing forecast consumption by pool.

Table 56.1: Defect fixes and calculation changes in Version 2016.2, Service Update 26

Fix Area / Change	Description
Full-level pegging	The following issues were fixed: • When a Part had no PartSource defined, but the Part had an IndependentDemand record and there was OnHand record for the Part, a WhereConsumedForDemand record would be generated for the IndependentDemand, showing the OnHand as being allocated to it (but this activity was correctly ignored in other reporting tables). Now, the part is effectively ignored and no WhereConsumedForDemand record generated. • When there are interchangeable parts, and there is a ScheduledReceipt on the alternate part which will be used for demand on the primary, and that ScheduledReceipt explodes into component as dependent demand, records for the primary part in the WhereConsumedForSupply table did not report the component level supplies that came from the explosion under the alternate. Component level supplies from exploding a ScheduledReceipt under an alternate part will now be reported for the primary part.

Input table changes

New table

This section outlines new Input tables added in Version 2016.2, Service Update 26.

- **ItemSourceFitOutput**- Contains settings related to forecast accuracy in the Self-Healing Supply Chain application.
- **ItemSourceFitOutputClass**-Contains settings used to customize the class (for example, ErrorMeasure) used by the Self-Healing Supply Chain.
- **ItemSourceFitOutputForecastModel**-Contains settings used to customize the forecast model used by the Self-Healing Supply Chain.
- **ItemSourceFitOutputValueName**- Contains settings used to customize the metric or characteristic being reported by forecast accuracy in the Self-Healing Supply Chain (for example, MAPE).

New fields

This section outlines fields added to Input tables in Version 2016.2, Service Update 26.

Table 56.2: Input table fields

Table name	Field name/change
HistoricalSupplyActual	LineDueDate
HistoricalSupplyActual	StatisticalDate

Version 2016.2 changes Service Update 25

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 24 to RapidResponse 2016.2 Service Update 25 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes

This Service Update also introduces BOM charts, which you can use to gain a better understanding of the bill of material.

A BOM chart provides you with an up-to-date view of the bill of material, which you can easily filter to focus only on the components or sites (or other criteria) that are relevant to you. The new BOM Visualization worksheet in the BOM Analysis workbook includes a BOM chart. For more information, see the *RapidResponse User Guide*.

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 25 including:

- Pushed supply actions for all transfers
- Defect fixes and calculation changes

Pushed supply actions for all transfers

Push planning logic applies to parts that have the **DistributionPlanningRule.ProcessingRule** set to "Enable" and also supports the Distribution Requirements Planning application. This logic has been enhanced to allow for pushed supply actions (such as, **CTPActivity** records with a **Type** set to "PushTransferOut") to potentially be generated for all transfers of supply from the source site to the destination site.

Previously, pushed supply actions were only generated for supplies at a sourcing site that were early to meet future demand (assuming the source part's maximum inventory level had been exceeded). In this service update, pushed supply actions can now be generated for all transfer supplies as long as their demand for them at destination site. This includes late supplies that are otherwise seen as being pulled by demand.

For more information, see "Pushed supply activity" on page 1719.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 25

Fix Area / Change	Description
Multi-level search	The following issues related to either multi-level search logic, or the use of multi-level search logic with other analytics, were fixed: • An issue where some lower-level component parts had a negative ending balance and some top-level assembly demands were left with an available date of "Future". • An issue related to use of multi-level search on parts with safety lead time. In some cases, the part's safety lead time would sometimes be ignored. This resulted in demands with equivalent DemandDate and DueDate values, when they should have been different in respect of safety lead time. • An issue related to use of multi-level search on parts that had reschedulable scheduled receipts. In some cases, more of these reschedulable receipts were being pulled in (rescheduled) than was needed to meet corresponding demand. This issue was seen to lead to excess supply and has been resolved (the ReschedDate should not be updated for receipts that don't ultimately need to be pulled in to meet demand). • An issue related to the use of multi-level search logic with BOM-level part substitution. In somce cases, the last planned order in the horizon didn't always provide sufficient supply which thereby led to a negative ending balance. • An issue related to how BOM rounding, as configured through the BOMType.RoundingRule setting, was applied when multi-level search logic was enabled.
Days of supply	An issue was fixed where when a DaysSupplyRule of "ByPeriodEnd" or "ByPeriod" was used and the PeriodSupplyDueCalendar was neither Workday nor equivalent to the PeriodSupplyInterval calendar, then planned orders might end up covering more demands outside of the expected interval. As a result of fixing this issue, some orders that were previously bucketed together into one planned order will no longer be and related differences in due and available dates might be seen.
Attributes	An issue was fixed where an internal server error was generated when using attribute-based planning along with time-phased safety stock and material expiry. Although this specific combination of analytics is not supported and may result in unexpected results, their combined usage should no longer lead to a server error or crash

Input table changes

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 25.

Table 56.3: Modified Input table fields

Table name	Field name/change
ForecastItemParameters	OutlierThreshold When setting a threshold for detecting outliers within historical data for a forecast item, there is no longer a limit to the value that you can set using this field.
SafetyStockItem	DemandOutlierThreshold When setting a threshold for detecting demand outliers within historical data for safety stock items, there is no longer a limit to the value that you can set using this field.

Control table changes

New table

This section outlines new Control tables added in Version 2016.2, Service Update 25.

- **SHSCConfiguration**- This table contains configuration settings related to the self-healing supply chain application. It replaces a series of profile variables that were previously used to configure the self-healing supply chain and which were referenced by various workbooks and resources in the application. For more information, see "SHSCConfiguration table" on page 889.

New fields

This section outlines fields added to Control tables in Version 2016.2, Service Update 25.

Table 56.4: Control table fields

Table name	Field name/change
DistributionPlaningRule	SupplyPushRule This field can be used to control whether pushed supply actions are generated for all transfer supplies allocated to demand, rather than just those that are early to meet future demand.

Version 2016.2 changes Service Update 24

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 23 to RapidResponse 2016.2 Service Update 24 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 24 including:

- Defect fixes and calculation changes

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 24

Table 56.5: Defect fixes and calculation changes in Version 2016.2, Service Update 24

Fix Area / Change	Description
Multi-level search with BOM-level substitution and interchangeable parts	An issue was fixed where if the primary component in BOM-level substitute relationship an interchangeable part but orders could not be created on either, and the secondary component was avaiable to satisfy the demand, multi-level search logic would not select the secondary component as expected and instead end up leaving the demand unsatisfied.
Co/by-products	Fixed an issue where a sustained negative inventory balance was occurring when using co and by-product logic along with parts that use a PartType.CommitLevel of "High".
UOM conversions	Fixed an issue where an error was generated when inserting a record into the ScheduledReceipt table with Unit of Measure conversion enabled. This was caused by an issue that occurred when the Unit of Measure path was determined by a calculated field (for example, PartSource.SupplierUOM on the ScheduledReceipt table) and has been resolved.
RecommendOnly scheduled receipts	Fixed an issue where a RecommendOnly scheduled receipt was not expatiated when Soft Safety Stock Date is before the real demand
Constrained planning	Fixed an issue where the AvailableStartDate for a scheduled receipt is earlier than MRP date when the receipt overloads constraint on its start date.

Version 2016.2 changes Service Update 23

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 22 to RapidResponse 2016.2 Service Update 23 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- "Input table changes" on page 2307
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 23 including:

- Cost considerations in multi-level search logic
- Defect fixes and calculation changes

Cost considerations in multi-level search logic

A new option allows for the rolled-up material cost from purchased components to be included in the sourcing criteria used by multi-level search logic. This is intended to allow giving preference to sources that require less costly purchases when satisfying a given requirement. This is controlled through the new **MultiLevelSearchRule.NewPurchasesCost** control setting, and if enabled is considered just before targets in the source selection criteria.

For more information, see "Including material cost as a selection criteria" on page 1869.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 23

Table 56.6: Defect fixes and calculation changes in Version 2016.2, Service Update 23

Fix Area / Change	Description
Distribution requirements planning	The following was fixed: • An issue where more supply was being pushed out than was suggested by the TimePhasedMaximumInventory value. This was caused by pure excess on the part contributing to the inventory level on the RunDate when it should have been contributing to inventory on the AvailableDate. • An issue where "PushTransferOut" activity records were not always being generated as expected when there were demand constraints. This occurred because early low priority demands that got a late supply would contribute to the inventory calculation. In such cases now, the early demand should not contribute to the inventory level calculation.
Days of supply	An issue was fixed where when the PartType.DaysSupplyRule setting of "ByPeriodWithPriority" was used with a PartType.CommitLevel of "Low", the priority of planned supply would be set based on the demands rather than using the default priority. This issue with the "ByPeriodWithPriority" setting has been fixed and the default priority should now be applied to planned supply when PartType.CommitLevel is set to "Low". When using the "ByPeriodWithPriority" setting, only a CommitLevel of "High" will propagate priority from demand to supply when using "ByPeriodWithPriority setting.

Table 56.6: Defect fixes and calculation changes in Version 2016.2, Service Update 23

Fix Area / Change	Description
BOM substitution (and interactions with multi-level search, co-products, and constraint)	The following was fixed: • An issue that occurred when BOM-level part substitution was configured along with multi-level search logic and the part used a SourceRule.AllotmentRule setting of "Proportional". With this configuration, the proportional splits of a given requirement were not always as expected (for example, if two other equivalent sources each had a Target of "1", the requirement wasn't being evenly split). • An issue that occurred when BOM-level part substitution was used along with multi-level search logic, co-products, and constraints. With this configuration, if the BillOfMaterial.Target values were equal for all components in SubstituteGroup, the dependent demand was being split proportionally between them and thereby leading to excess supply. • An issue where BillOfMaterial.Target values were not always respected when distributing dependent demands in response to changes in available supply.
Partial planning (and interactions with constrained planning and expiry)	The following was fixed: • An issue where when partial planning was used in constrained planning environments and there was a shared constraint and AbsolutePrePlanLimit was set, then demand portions that could not be satisfied within tolerance were not left unsatisfied as expected. • An issue where when partial planning was used with material expiry logic and some demands were left unsatisfied because they couldn't be satisfied within tolerance, then subsequent results reported by CTP logic were not always as expected.

Table 56.6: Defect fixes and calculation changes in Version 2016.2, Service Update 23

Fix Area / Change	Description
Co-products (and interactions with multi-level search and constraints)	The following was fixed: • An issue where co/by-product supply can have a different due date from the supply of the primary product if multi-level search logic is enabled along with constrained planning and a planned order has to be moved due to upper level constraints. • An issue where co/by-product supply is not fully considered in part source selection when the planning time fence (PTFDate) is set for a part such that it cannot generate any planned orders on time. • An issue where multi-level search logic could causes some demands to have reported available dates that are later or worse than they could be when there is a multi-level constraint with a co-product in the middle of the structure.
Multi-level search (and interactions with pooling, alternate BOM, constraints, and global substitution)	The following was fixed: • An issue where when multi-level search was used with pooling and the MUEPoolNettingType.PoolFlowToCommon setting of "NetCommon" then planned orders were sometimes created despite sufficient supply on hand to meet a given requirement. • An issue where when multi-level search was used with constrained planning then with more constraint preplanning than expected was seen. • An issue where sourcing splits were not as expected with the SourceRule.AllotmentRule setting of "Ongoing" for sources of the same BOMAlternate (same structure). • An issue with how global substitute parts were used when multi-level search logic was enabled. This issue started occurring in Service Update 9.
Priority	An issue was fixed related to the PartType.CommitLevel setting of "Medium" where pool transfer supply priority values did not match demand priority values.

Table 56.6: Defect fixes and calculation changes in Version 2016.2, Service Update 23

Fix Area / Change	Description
Constrained planning	The following was fixed: • An issue where a record in the FLPConstraint table referenced a ConstraintUsed.Date that was later than the referenced WhereConsumed.SplitSupplyAvailable date value. • An issue that occurred when PartSourceType.ConstraintDateRule was set to "ShipExtend"whereby calendar rounding could result in incorrect planned order start dates. • An issue that occurred whereby calendar rounding could cause ConstraintLoad.Date to be set well before PlannedOrder. DueDate (issue was seen when pre-planning was enabled).
Capable-to-promise	An issue was fixed where a planned order was seen as having availability prior to the required materials being ready (that is, CTPPlannedOrder.AvailableDate was earlier than CTPPlannedOrder.MaterialStartDate).
ATP	An issue was fixed where an Index out of range error was reported in certain workbooks when comparing new demands against ATP.

Input table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 23.

- **ItemSourceHistoricalSupplyCategory table**—This table is used to defines a set of **HistoricalSupplyCategory** records that are associated with an **ItemSource**, and supports the self-healing supply chain capability. For more information, see "ItemSourceHistoricalSupplyCategory table" on page 389.

Deprecated fields

This section outlines fields removed in Version 2016.2, Service Update 23.

- **ItemSource field**—This field was used in the **HistoricalSupplyCategory** table to represent item sources associated with a historical supply category. This field was used to support the Self-Healing Supply Chain application, and was replaced by the **ItemSourceHistoricalSupplyCategory table**. For more information, see "ItemSourceHistoricalSupplyCategory table" on page 389.

Control table changes

New fields

The section outlines new control fields added in Version 2016.2, Service Update 23.

Table 56.7: Control table fields

Table name	Field name/change
MultiLevelSearchRule	NewPurchasesCost This field can be used to include or exclude material purchase cost from being a consideration in multi-level sourcing logic.

Version 2016.2 changes Service Update 22

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 21 to RapidResponse 2016.2 Service Update 22 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- "Input table changes" on page 2311

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 22 including:

- FlatBillDown date parameter
- Defect fixes and calculation changes

FlatBillDown date parameter

To give greater refinement and control over how many **FlatBillDown** records are generated, a new analytic parameter named **RR_Analytics_FlatBill_ParameterDate** is added. This parameter can be enabled as a workbook variable and then used by **FlatBillDown** worksheets in a given workbook. This parameter allows specifying an effective date that determines which records are used in generating **FlatBillDown** output. For example, only **BillOfMaterial** and/or **PartSource** records that are effective for an assembly on the date defined by the parameter would be considered when performing **FlatBillDown** calculations. Similarly, if an assembly **PartSource** had multiple time-phased lead time values, only those effective on the date defined by the parameter would be used in **FlatBillDown** calculations.

If this new parameter is not enabled as a variable in a given workbook, then any **FlatBillDown** reports in the workbook will perform as they did previously. That is, all possible **PartSource** and **BillOfMaterial** paths throughout the planning horizon can be considered (dependent on other parameters that might be used) along with any associated time-phased lead time settings.

For more information, see "Apply a parameter date to FlatBillDown calculations" on page 2222.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 22

Table 56.8: Defect fixes and calculation changes in Version 2016.2, Service Update 22

Fix Area / Change	Description
Co-product allotments with multi-level search	A change was made to the logic used by the capable-to-promise analytic when allotting co-product supplies with multi-level search logic enabled. Previously, because the final available date on these supplies is not always known when allotting them to demands, co-products were sorted and allotted in date sequence based on the supply's **MaterialStartDate** plus lead time. However, the resulting date was sometimes significantly different from the actual available date on the co-product supply. This could sometimes lead to supply allotments where late co-product supplies were allotted to earlier demands while earlier co-products were not allotted to any demand. As of this Service Update, when co-products are used along with multi-level search, the date that Netting/multi-level search logic expects the supply to be available (the "planned available date") is instead used as the date for sorting and allotting co-product supplies. This change in behavior might typically lead to differences in calculated output when using co-products along with multi-level search logic. However, these differences will generally be improvements in the allotment of co-product supplies and hence better demand fulfillment.
Multi-level search issues	The following issues related to multi-level search logic were fixed: • An issue related to use of BOM-level part substitution with multi-level search logic where incorrect bom-level part substitution decisions (thereby leading to excess inventory). For example, dependent demands and planned orders were placed on a less preferred substitute even though there was sufficient firm supply on the primary component. Note that this issue only occurred on parts that used, or were associated with parts that used, an **OrderPolicy.OrderGenerationRule** setting of "NoOrders".. • An issue where if multi-level search logic was enabled for parts that referenced a **SourceRule.AllotmentRule** setting of "Proportional", then planned requirements were not always being properly split between otherwise equivalent sources (typically, one part source was being over used).
Constrained planning	An issue was fixed where in some cases the calculated **ConstraintUsed** and **ConstraintLoad** tables weren't being populated as expected (but the **ConstraintSpreadLoad** table was). This issue was typically seen to occur when there were parts that used an **AvailableRule.ProcessingRule** setting of "Ignore"(along with fractional quantity per values and multi-level search logic).

Table 56.8: Defect fixes and calculation changes in Version 2016.2, Service Update 22

Fix Area / Change	Description
Allotment overrides	The following issues related to allotment override logic were fixed: • An issue that could occur if an allotment override was made for a part that had a subsequent/later safety stock drop (reduction in the safety stock level, such as from a TimePhasedSafety records). In these cases, the safety stock drop could be used for the override even if doing so would make the demand late when a new planned order would allow it to be satisfied on time. In such cases now, the planned order will be generated to satisfy the demand on time and the safety stock drop not used for the override (potentially leading to short-term excess that would not have been present with the previous behavior). • An issue that could occur when allotment overrides were used in environments with scheduled/non-reschedulable scheduled receipts along with limited constraint. In this case, some scheduled receipt whose due date would allow them to effectively cover the override were not used for it and new planned orders generated. This was seen to contribute to excess supply and has been resolved.
Order point planning	An issue where order point logic was found to be incompatible with certain inventory pooling logic. For order point parts that also used a MUEPoolNettingType.FlowToCommon setting of "NetCommon", planned orders were being generated in the demand pool when they should have been generated in the common pool. In such cases, planned orders should now be properly created in the common pool Note however that in some other cases the combination of order point and pooling logic might produce unexpected results, particularly when used with part source pools and time-phased safety stock.
Time-phased yield	An issue related to analytic performance when using time-phased yield logic (performance was seen to degrade when moving from Service Update 20 to Service Update 21).

Input table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 22.

The following tables are added to Solutions namespace and support Live Lens:

- **SupplierMetric table**—This table identifies the combination for each supplier and metric definition generated for a specific as of date that are used in live lens and comparison scorecards. For more information, see "SupplierMetric table" on page 615.

- **SupplierMetricValues table**—This table stores the supplier calculations for metrics used in live lens and comparison scorecards. For more information, see "SupplierMetricValues table" on page 616.

Version 2016.2 changes Service Update 21

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 20 to RapidResponse 2016.2 Service Update 21 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Input table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 21 including:

- Self-Healing Lead Time
- Defect fixes and calculation changes

Self-Healing Lead Time

RapidResponse provides the optional Self-Healing Lead Time capability that is part of the Self-Healing Supply Chain. The Self-Healing Lead Time capability uses machine learning to automatically detect and correct deviations in lead times used in the supply planning data for Buy part source types.

Note: The Self-Healing Lead Time capability is only available if it has been enabled, and is only available with the On-Demand RapidResponse service. In these cases, the Self-Healing Lead Time data model and calculations will be available. For more information, see "Self-Healing Supply Chain" on page 2135.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 21

Table 56.9: Defect fixes and calculation changes in Version 2016.2, Service Update 21

Fix Area / Change	Description
Multi-level search issues	The following issues related to multi-level search logic were fixed: • An issue an where when using multi-level search logic a small number of planned orders were generated beyond the part source's effectivity (later than PartSource.EffectiveOutDate). • An issue where in certain cases when multi-level search logic was enabled for a part, the Netting analytic would not complete and an error would display indicating the AnalyticMemoryLimitMB had been be exceeded. This issue would typically occur when an assembly with multi-level search enabled had its Type.AvailableRule set to "Normal" and a component had its Type.AvailableRule set to "Ignore" (especially in cases of extremely small QtyPer values). • An issue where when multi-level search logic was enabled for an assembly involved in a bill of material relationship where BOMType.RoundingRule set to "QtyPerOrder", then Netting analytic would in some cases not complete and in other cases would generate incorrect results (such as, generating too many dependent demands). • An issue related to use of multi-level search logic along with constrained constraints whereby limited component material was not always fully utilized which would result in a positive ending balance for those components even while the higher level parts that require them would end up with low fulfillment rates.
Constrained planning	The following issues related to constrained planning were fixed: • An issue related to the use of constrained planning where the CTP analytic would not complete and an error would display indicating the AnalyticMemoryLimitMB had been exceeded. This issue would typically occur when using a constraint calendar that did not contain sufficient dates to cover the demand/supply planning horizon. • An issue with constrained planning and constrained constraints (those with Type.ProcessingRule set to "Constraint") where in some cases a planned order would be generated even though there was an available scheduled receipt which could thereby lead to excess supply. This was related to an issue with calendar rounding and would typically occur when the scheduled receipts were due on non-work days.

Table 56.9: Defect fixes and calculation changes in Version 2016.2, Service Update 21

Fix Area / Change	Description
Distribution planning	The following issues related to interchangeable part logic were fixed: • An issue where an Internal Server Error (exception) was sometimes generated in worksheets when trying to access data from the SupplyDemand table. The issue was occurring in cases where push planning logic (parts where DistributionPlanningRule.ProcessingRule is set to "Enable") was used along with interchangeable parts on the parent level of push planning. • An issue where when material expiration was configured for parts that also have push planning enabled (DistributionPlanningRule.ProcessingRule set to "Enable"), supply allotment logic would not always handle expiring on hand supply correctly. This could result in unnecessary expiring quantities on on hand supply and thereby an increased potential for shortages.
Attributes	An issue was fixed where some parts would have a slightly different number of CTPPlannedOrder records generated in different scenarios or at different times (unstable results) even when there were no changes/differences in the underlying input data. This issue was seen to occur with forecast demands and when there were attributes enabled.
Co-products	An issue was fixed where in some cases when supply was generated for the primary part in a co-product relationship, the corresponding supplies were not generated on its co-products. This Issue was related to how primary/co-product parts were processed when they had different low level codes.
S&OP	An issue was resolved related to how the lack of an expected SOPAnalyticsConfiguration record was handled.
Forecast	An issue was fixed where some parts would have a slightly different number of Forecast records generated in different scenarios or at different times (unstable results) even when there were no changes/differences in the underlying input data. This issue typically occurred when BOM-level substitution logic was used.

Input table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 21.

The following tables are added for the Self-Healing Lead Time capability:

- **Cluster table**—This table stores groups of **ItemSource** records with similar lead time characteristics, as reported by the Machine Learning service. For more information, see "Cluster table" on page 243.

- **ClusterProperty table**—This table stores the key properties and values for a particular **Cluster**, as reported by the Machine Learning service. For more information, see "ClusterProperty table" on page 244.
- **ItemSource table**—This table defines multiple sources of supply for parts that have similar lead time deviations. For more information, see "ItemSource table" on page 379.
- **Job table**—This table logs the status of the machine learning jobs used to calculate the **ProjectedLeadTime**, the **Cluster** and the **ClusterProperty** for the **ItemSource**. This table is in the MLService namespace and is unversioned. For more information, see "Job table" on page 389.
- **ProjectedLeadTime table**—This table contains lead time information projected by the Machine Learning service for a set of **ItemSource** records. The accepted lead time changes are then copied to the **TimePhasedPartSourceLeadTime** table. For more information, see "ProjectedLeadTime table" on page 505.

Calculated table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 21.

The following tables are added for the Self-Healing Lead Time capability:

- **ItemSourceCandidate table**—This table reports combinations of **Part**, **Source**, and **HistoricalSupplyCategory** records that are **ItemSource** candidates. For more information, see "ItemSourceCandidate table" on page 1087.
- **ItemSourceHistoricalLeadTime table**—This table matches **HistoricalSupplyActual** records with **ItemSource** records.
- **ItemSourceIntervalEffectivePartSource table**—This table reports the set of effective **PartSource** records for a given **ItemSourcePartSourceEffectivityInterval**. For more information, see "ItemSourceIntervalEffectivePartSource table" on page 1088.
- **ItemSourcePartSourceEffectivityInterval table**—This table reports **PartSource** "effectivity intervals", which determine the intervals in which a set of effective **PartSource** records are constant for a given **ItemSource**. For more information, see "ItemSourcePartSourceEffectivityInterval table" on page 1089.
- **ItemSourcePartSourceHistoricalLeadTime table**—This table matches **HistoricalSupplyActual** records for **ItemSources** with lead times from effective **PartSource** records. For more information, see "ItemSourcePartSourceHistoricalLeadTime table" on page 1089.

Control table changes

New tables

This section outlines fields added to Control tables in Version 2016.2, Service Update 21.

The following tables are added for the Self-Healing Lead Time capability:

- **AutomatedLeadTimeType table**—This table contains the parameters used in defining how the lead times projected by the Machine Learning service are processed for **ItemSource** records of a particular type. For more information, see "AutomatedLeadTimeType table" on page 678.

Version 2016.2 changes Service Update 20

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 19 to RapidResponse 2016.2 Service Update 20 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 20 including:

- "Global part substitutes in FlatBillDown calculations" on page 2318
- Defect fixes and calculation changes

Global part substitutes in FlatBillDown calculations

A new analytics parameter, **RR_Analytics_FlatBill_AlternatePartsExpode**, is available and can be added as a workbook variable for use in worksheets based on the **FlatBillDown** table. This parameter can be used to include or exclude any of an assembly's global substitutes, along with components of those substitutes, from being included in FlatBillDown reports. By default, the **FlatBillDown** table does not generate records for an assembly's global substitutes or the components under those substitutes (but this parameter can be used to toggle their inclusion or exclusion)

However, in **FlatBillDown** worksheets that made use of the existing **RR_Analytics_FlatBill_ MultiSourceExplode** parameter, all of a part's global substitutes and the components under those substitutes will have been reported when the parameter was set to "True" (including from non-primary part sources). For this case, the new **RR_Analytics_FlatBill_AlternatePartsExplode** parameter can be used to disable explosion of those global part substitutes and thereby prevent reporting of components under them in the **FlatBillDown** table. Because including components under substitute parts can significantly increase the number of records generated in the **FlatBillDown** table, use of this variable can potentially improve worksheet performance.

For more information about using this new parameter, see "Include global substitutes and their components in FlatBillDown calculations" on page 2219"Pools and part sources" on page 1750

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 20

Table 56.10: Defect fixes and calculation changes in Version 2016.2, Service Update 20

Fix Area / Change	Description
Multi-level search issues	The following issues related to multi-level search logic were fixed: • An issue related to constraint utilization with the multi-level search analytic. In some cases during the multi-level search trial phase, even if a given source was not used to supply a demand, subsequent trials could potentially have a limited or reduced view of constraint available. This issue was reflected in cases where some sites/sources within a constrained environment were not fully utilized even though they had adequate constraint available. • An issue where in certain cases multi-level search smoothing logic could increase planned orders (and hence short-term excess) rather than reduce them. • An issue where the value in PartSource.SecondaryPriority was being ignored within multi-level search logic's part source selection criteria (in cases where the value in this field was expected to make a difference). For example, if using "ongoing" part source selection criteria and choosing between two (otherwise equivalent) constrained make sources with equal BOM alternates, the value in SecondaryPriority was not considered in the selection process. • An issue with a potential server crash when running the multi-level search analytic. • An issue related to the occurrence of a worksheet error indicating "AnalyticMemoryLimitMB limit of 32000 is exceeded". This was caused by reclaim logic and an endless loop that occurred when muiti-level search logic was used together with dynamic or time-phased safety stock requirements close to RunDate. • Several issues related to use of multi-level search logic with time-phased safety stock requirements (such as, with a a SafetyStockQuantityRule setting of "TimePhasedQuantity") were not enough planned orders were created to cover demand in some situations. This could result in an IndependentDemand.AvailableDate of "Future" or unmet safety stock requirements, even when there was the capacity to provide sufficient supply, and was typically seen when with prioritized safety stock, safety stock drops, and constraints.

Table 56.10: Defect fixes and calculation changes in Version 2016.2, Service Update 20

Fix Area / Change	Description
Constrained planning	The following issues related to constrained planning were fixed: • An issue where some where some planned orders in constrained environments had an "Undefined" available date. This issue has been fixed and CTPPlannedOrder.AvailableDate should never return "Undefined" but always return either a valid date or else a value of "Future" (if the supply cannot be built). • An issue related to planning with constrained constraints on parts that use an AvailableRule.ProcessingRule of "Ignore". With this combination, planned orders and/or scheduled receipts erroneously could be reported late in cases where constraint availability was not such as to make the supply late.
Interchangeable parts	The following issues related to interchageable part logic were fixed: • An issue where wrong supply allotments were sometimes pegged/reported incorrectly for interchangeable parts having phantom components. This issue was seen to contribute to issues such as forecasts having available dates set to "Future" even when there was supply available to satisfy them. • An issue where when interchangeable parts were used with attributes and there were forecast demands, capable-to-promise was logic was unable to peg supplies to some forecast demands even though appropriate supply had been planned. This resulted in those forecast demands being given an available date of "Future" even though there was supply available to satisfy them.
Global part substitution	An issue was fixed related to global part substitution in constrained environments whereby large numbers of small substitute demands and supplies, many for the exact same quantity, were erroneously being created.
Allocations	An issue was fixed where an Allocation was erroneously treated as an instance of recursion, and hence ignored, in cases where a given part had allocations to multiple scheduled receipts for its parent assembly and those scheduled receipts were a mix of "explode only" and normal/reschedulable receipts.
Statistical forecasting (R)	Fixed an issue to Improvevaidation of analytic calls to the (optional) R Server.

Version 2016.2 changes Service Update 19

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 18 to RapidResponse 2016.2 Service Update 19 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 19 including:

- "Pooling enhancements" on page 2321
- Defect fixes and calculation changes

Pooling enhancements

Several enhancements have been made to inventory pooling logic and the use of pools including the following:

- Pools can now be specified directly on **IndependentDemand** and **ScheduledReceipt** records. If populated, new **InputPool** fields on these tables override the corresponding **Pool** reference on the **DemandOrder** or **SupplyOrder** . For example, pools might be specified at the line item level when pools represent areas such as a plant or storage location, but specified at the order level.
- Part source selection logic has been extended to allow for choosing part sources based on pool. This can be configured using a new **Pool** reference added to the **PartSource** table and the logic is further dependent on the settings in the **PoolRule** and **PoolFlowToCommon** fields on the **MUEPoolNettingType** table.
- Pool information can now be introduced or injected at the dependent demand level and not necessarily driven down all levels from the end-item demand. This requires **MUEPoolNettingType.PoolRule** to be set to "Net" or "Optional" on the component whose dependent demands the pool should first appear on (and potentially be propagated down from), and set to a value other than "Net" or "Optional" on the parts above the component where the pool information should be injected.

For more information, see "Pools and part sources" on page 1750

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 19.

Table 56.11: Defect fixes and calculation changes in Version 2016.2, Service Update 19

Fix Area / Change	Description
Multi-level search issues	The following issues related to multi-level search logic were fixed: • An issue where a required planned order to satisfy dependent demand on a bottom-level component (that had no substitutes, components, and only one part source) was not being generated. However, the planned order was generated when multilevel search logic was disabled. This has been fixed and the required supply should now be generated, regardless. • An issue related to multi-level search smoothing logic where the priority on planned orders created to satisfy safety stock was not always set to the safety stock priority. • An issue that occurred when the MultiLevelSearchRule.PartSourceLimit field was used to place a limit on the number of effective part sources that could be evaluated when planning new supply to satisfy demand. When the SourceRule.AllotmentRule of "Proportional" was used, the number of part sources considered could be one greater than the specified limit. • An issue related to fair-share logic (OrderPriority.MLSAllocationRule set to "FairShare") where a demand might not be fully processed which could thereby led to an incorrect negative ending balance and the potential for the demand to not be fully satisfied when it could be (based on actual supply availability). • An issue in that could occur when committing planned orders generated during the trial phase. In cases, where the order quantity being committed was different from the quantity on the initial trial(s), planned orders would be scaled. However, the scaling was erroneously based on the inventory quantity rather than the source quantity. This could result in planned order quantities (along with any related co-products) being smaller than they should have been and thereby resulting in sub-optimal fulfillment and/or negative ending balances. • An issue related to the processing of non-reschedulable scheduled receipts which could result in sub-optimal part source selection, thereby leading to increased planned order quantities along with excess supply and/or late supply availability.

Table 56.11: Defect fixes and calculation changes in Version 2016.2, Service Update 19

Fix Area / Change	Description
Co-products and AvailableDate issues	The following issues related to co-product logic were fixed: • An issue where the AvailableDate for some co-product and by-product supplies was earlier than the AvailableDate on the primary part's supply when it should not have been. The AvailableDate for co-products/by-products should now always return the AvailableDate from the primary part supply (offset by the optional BillOfMaterial.LeadTimeOffset value, if applicable). • An issue where an undefined(blank) AvailableDate was erroneously reported on some co-product/by-product supplies even though the corresponding primary part supply had a valid AvailableDate populated. This would typically occur when the supply was going to be late due to a constraint on the primary part.
Global part substitution and negative balance	Fixed an issue where the process to remove any excess supply built up from the use of global substitution logic could, in some cases, remove too much supply and result in a very slight negative balance.
Material expiry	Fixed an issue where an allotment that had already been reclaimed was erroneously being used in expiry calculations.
Two-date planning and incorrect CalcStartRequestDate	Fixed an issue where some independent demands were not being pulled in to meet their request date even though supply availability dictated that they could be. This was related to incorrect CalcStartRequestDate values calculated on late supplies for bottom-level components.

Input table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 19.

All of these tables are available for use in live lens or comparison scorecards.

ConstraintMetric table—This table identifies the combination for each constraint and metric definition generated for a specific as of date.

ConstraintMetricValues table—This table stores the constraint calculations for metrics.

Metric table—This table stores metric definitions for metrics.

MetricAsOf table—This table stores the date that the metric calculations are generated on for the PartMetric, SiteMetric, and ConstraintMetric tables.

PartMetric table—This table identifies the combination for each part and metric definition generated for a specific as of date.

PartMetricValues table—This table stores the part calculations for metrics

SiteMetric table—This table identifies the combination for each site and metric definition generated for a specific as of date.

SiteMetricValues table—This table stores the site calculations for metrics

New fields

This section outlines fields added to Input tables in Version 2016.2, Service Update 19.

Table 56.12: Input table fields

Table name	Field name/change
IndependentDemand	InputPool This field can be used to assign a pool to an individual line item, which then overrides the pool specified at the demand order level.
PartSource	Pool This field can be used to assign a pool to a part source, which can then be used in part source selection logic to ensure a part source with a matching pool is used.
ScheduledReceipt	InputPool This field can be used to assign a pool to an individual line item, which then overrides the pool specified at the supply order level.

Calculated table changes

New fields

This section outlines fields added to Calculated tables in Version 2016.2, Service Update 19.

Table 56.13: Calculated table fields

Table name	Field name/change
NewTransferAllocation	MappedPool This field reports the pool used in netting the demand.
PlannedTransferAllocation	MappedPool This field reports the pool used in netting the demand

Version 2016.2 changes Service Update 18

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 17 to RapidResponse 2016.2 Service Update 18 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 18 including:

- Time-phased lead time
- Prioritized safety stock
- Defect fixes and calculation changes

Time-phased lead time

Certain lead times defined at the **PartSource** and **Source** level can now be time-phased. Time-phased fixed and dock-stock lead times can now be defined in the new **TimePhasedPartSourceLeadTime** table, while time-phased transit and pre-ship lead times can now be defined in the new **TimePhasedSourceLeadTime**table. T

This is useful for cases where lead time values are expected to increase or decrease over time. Previously, the only way to have lead times change over time, was to create additional **PartSource** or **Source** records to model the different lead times in different effective periods. However, doing so can result in unnecessary data maintenance and in some cases performance issues. The use of the new tables is recommended instead.

If lead times are not expected to change over time, then the stationary lead-time fields on the **PartSource** and **Source** tables can continue to be used instead.

For more information, see "Lead time elements" on page 1308

Prioritized safety stock

In capable-to-promise logic, satisfying safety stock requirements can now be prioritized over actual demands. When there is limited supply to satisfy all requirements on time, allotments of supply to demand can now be temporarily held (not released to the demand) in order to ensure that safety stock levels are maintained instead.

This functionality is controlled through the new **CTPHoldRule** field on the **SafetyStockPolicy** table. When this field set to "Use", allotments of supply to demand can be held if doing so protects a higher priority safety stock (ensures the safety stock level is not violated or is only minimally violated). However, actual demands are still given preference over safety stock of an equivalent or lower priority. When the **CTPHoldRule** field is set to "Ignore", then actual demands are always given preference over safety stock regardless of priorities. This setting corresponds to how safety stock was always treated in previous versions.

To allow explicitly specifying different priorities on safety stock requirements, an **OrderPriority** reference has been added to the **TimePhasedSafety** table. For example, a part can have a high priority portion of safety stock that is given preference over actual demands, and a low priority portion that is not. When this reference is left Null, and for safety stock rules that don't make use of the TimePhasedSafety table, the priority on a part's safety stock comes from **PartType.DefaultPriority** (as was always done previously).

For more information, see "Priority and safety stock holds in CTP" on page 1780.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 18.

Table 56.14: Defect fixes and calculation changes in Version 2016.2, Service Update 18

Fix Area / Change	Description
FlatBillDown explosion with "NoOrders" setting	Fixed an issue where the FlatBillDown table was erroneously exploding and reporting lower-level component details for make and transfer parts that had OrderPolicy.OrderGenerationRule set to "NoOrders". These parts should no longer be exploded for the purposes of reporting in FlatBillDown (the part itself should still show).
Consensus forecast override behavior	Fixed an issue where the ConsensusForecast.OverrideQuantity value was erroneously reduced to 0 (zero) anytime an override with type/rule of "RebalancingOverride" was provided in ForecastDetail. The original OverrideQuantity should now always be preserved in this field regardless of other override values provided.
Constrained planning fixes	The following issues were fixed: • Fixed an issue where some scheduled receipts that used a SupplyStatus.TargetAccum setting of "Ignore" were consuming constraint during Netting. Such constraints should not consume constraint. • Fixed an issue where the ConstraintUsed in a particular bucket was greater than the ConstraintAvailable (rate) for that bucket. This would typically occur when planning close to RunDate and when short planning fence allowed for planned orders to be started before RunDate.

Table 56.14: Defect fixes and calculation changes in Version 2016.2, Service Update 18

Fix Area / Change	Description
Multi-level search issues	This service update includes several changes made to enhance general stability and performance of the multi-level search analytic. The following issues were also addressed: • Fixed an issue in where excess supply could be generated in cases where co-products were generated when substitute supplies were used. • Fixed an issue where an AvailableDate of "Future" was erroneously reported on an independent demand. The issue was related to the use of multi-level search logic along with global part substitution, co-products, and constraints. • Fixed an issue that could occur when multi-level search smoothing logic was used with global part substitution. In some cases, this combination would lead to more substitute supply being planned than was needed to cover demand. This has been resolved. • Fixed an issue where an endless loop was encountered when using multi-level search smoothing logic which resulted in Netting and other analytics from being unable to complete for certain parts. This was typically occurring for parts that had very small fractional quantities on demands (including safety stock) and supplies due how rounding was applied. • Fixed an issue where unexpected part source selection choices were sometimes made when multi-level search logic was used together with attribute-based planning. Previously, the part source selection logic was sometimes too restrictive in what could be used. • Fixed an issue related to multi-level search logic with bom-level part substitution where a new planned order was generated even though there was sufficient supply on a scheduled receipt to cover demand (thus, leading to excess supply). The issue was related to processing of a sibling part that had OrderPolicy.OrderGenerationRule set to "NoOrders" and has been resolved. • Fixed an issue that could occur when multi-level search logic with a MultiLevelSearch.ShortTermExcess setting of "Reduce" was used with constraints. With this combination, if there were scheduled receipts that used a SupplyStatus.RescheduleRule of "Scheduled", then such receipts could potentially consume constraint in excess of capacity.

Table 56.14: Defect fixes and calculation changes in Version 2016.2, Service Update 18

Fix Area / Change	Description
Global part substitution fixes	The following issues were fixed: • Fixed an issue in Netting logic whereby all parts in a global substitution relationship should be pegged to the same level in a Netting family but some were not. This is expected to improve performance of Netting calculations when substitution and multi-level search are used. • Fixed an issue where certain analytic calculation (such as, WhereConsumedForSupply) were generating internal server errors in cases where bi-directional substitute relationships were defined. Although bi-direction (2-way) substitutes are not supported, they should not generate internal errors. • Fixed an issue where the use of global part substitution logic could lead tom some excess supply being planned. This was due to substitute supplies not being properly considered in the stage of the Netting analytic that aims to reduce planning excess.
BOM Substitution excess (with other analytics)	Fixed an issue related to BOM-level part substution when used together with attribute-based planning, material expiry, and time-phased safety stock. Previously with this combination, in cases where supply was freed up because of reduction in the safety stock level of an bom-level substitute part, this additional supply was not used to satisfy subsequent demands it was eligible to satisfy. As a result, new planned orders were generated instead resulting in excess supply being available . This has been fixed and excess supply should no longer be generated in this case.
Order point increase	Fixed an issue with order point logic where if temporary increases in the part's order point threshold are being calculated, and the associated OrderPointCalendar does not include the RunDate, then Netting calculations could get stuck in an endless loop.
Partial planning and scheduled supply	Fixed an issue with partial planning logic where a demand could be ignored in some cases when there was a scheduled receipt that could be used for the demand and that receipt was late but still within tolerance.
Co-products with multi-level constraint	Fixed an issue related to available date calculations when multi-level constraints and co-products were involved. Previously, if a supply was late due to constraint and that supply also generated co-product supplies which were late, and if the parent supply of the co-product supply was also in the family, then that parent supply's available date was note updated. This issue has been fixed and parent supply available date should be updated in this case.

Table 56.14: Defect fixes and calculation changes in Version 2016.2, Service Update 18

Fix Area / Change	Description
CTP and LowLevelCode	Fixed an issue where the CTPFamilyLLC (used to indicate the level in the family where a part exisits) used in reporting differed from that used in CTP analytic calculations.
Expiry and ABP (with small fractional quantities)	Fixed an issue with a RapidResponse Server crash occurred related to use of material expiry logic with attribute-based planning. The issue was related to incorrect processing of some extremely small fractional quantities in expiry logic and has been resolved.

Input table changes

New tables

This section outlines new tables added in Version 2016.2, Service Update 18.

- **TimePhasedPartSourceLeadTime table** —This table can be used to define time-phased fixed and dock-to-stock lead time values for part sources (to use in place of the stationary values defined in the **PartSource** table). For more information, see "TimePhasedPartSourceLeadTime table" on page 654.
- **TimePhasedSourceLeadTime table** —This table can be used to define time-phased transit and pre-ship lead time values for sources (to use in place of the stationary values defined in the **Source** table). For more information, see "TimePhasedSourceLeadTime table" on page 660.

New fields

This section outlines fields added to Input tables in Version 2016.2, Service Update 18.

Table 56.15: Input table fields

Table name	Field name/change
TimePhasedSafety	OrderPriority This field can be used to assign a priority to safety stock, and potentially protect that safety stock against real demands in CTP.

Control table changes

New fields

This section outlines fields added to Control tables in Version 2016.2, Service Update 18.

Table 56.16: Control table fields

Table name	Field name/change
PartSourceType	UseTimePhasedPartSourceLeadTime This field can be used to enable or disable the planning use of any time-phased lead times defined in the **TimePhasedPartSourceLeadTime** table.
PartSourceType	UseTimePhasedSourceLeadTime This field can be used to enable or disable the planning use of any time-phased lead times defined in the **TimePhasedSourceLeadTime** table.
SafetyStockPolicy	CTPHoldRule This field can be used to specify how priority on safety stock is handled, and whether supply allotments can be held to protect safety stock requirements.

Calculated table changes

New tables

This section outlines new calculated tables added in Version 2016.2, Service Update 18.

- **CTPSafetyStockHoldings table** —This table reports details of all supply allotments that are available to maintain the safety stock level for a part. This combines allotments of supply to demand have been held in order to protect minimum safety stock requirements, pushed allotments, and allotments specifically made to requirements from safety stock. For more information, see "CTPSafetyStockHoldings table" on page 1002.

New fields

This section outlines fields added to Calculated tables in Version 2016.2, Service Update 18.

Table 56.17: Calculated table fields

Table name	Field name/change
CTPActivity	HoldEndDate This field reports the date as of which a held supply can be used without violating the minimum safety stock level.
CTPActivity	HoldStartDate This field reports the date from which a supply needed to be held in order to respect the minimum safety stock level.
SupplyDemand	HoldEndDate This field reports the date as of which a supply allotment can be used (released to the demand) without violating the minimum safety stock level.
SupplyDemand	HoldStartDate This field reports the date from which a supply allotment needed to be held from a demand in order to respect the minimum safety stock level.

Version 2016.2 changes Service Update 17

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 16 to RapidResponse 2016.2 Service Update 17 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 17 including:

- Limiting part sources evaluated in multi-level search logic
- Defect fixes and calculation changes

Limiting part sources evaluated in multi-level search logic

A new control field, MultiLevelSearchRule.PrePlanLimit was added. This field is used to place a limit on the number of part sources that multi-level search logic will evaluate for a given part when satisfying a demand. For parts that have very large numbers of low yielding part sources, this limit can improve performance without significant impact on solution quality.
For more information ,see "Limiting the number of part sources evaluated" on page 1870.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 17.

Table 56.18: Defect fixes and calculation changes in Version 2016.2, Service Update 17

Fix Area / Change	Description
Multi-level search	This service update includes several changes made to enhance general stability and performance of the multi-level search analytic. The following issues were also addressed: • To improve analytic performance, the PrePlanLimit field was added to the MultiLevelSearchRule table as described in the previous section. Because the default setting for this field is 100, in some cases if a given part has more than 100 effective part sources at a given point, then multi-level search logic might generate different results than it did previously when there wasn't a practical limit placed on the number of part sources evaluated when satisfying a demand. Note however that this setting is configurable. • Fixed an issue in multi-level search smoothing logic that was causing "internal server errors" (for example, as seen when viewing data in certain worksheets). • Fixed an issue where an "Internal server error" was generated on opening some worksheets that returned data from the SupplyDemand table. This issue was typically seen to occur when using multi-level search logic and where MultiLevelSearchRule.ShortTermExcessRule was set to "Reduce". • Fixed an issue where an "Internal server error" was generated when reporting SupplyDemand results for parts that used global substitution and had PartType.SubstitutionPreference set to "Primary". Note that this issue only occurred in cases where global part substitution was used along with bom-level substituion and multil-level search on the component level (but not on the part level). However, it has been resolved and the internal server error (exception) should no longer be generated in these cases. • Fixed an issue where when multi-level search logic was used with bom-level substitution logic, the amount of dependent demand being planned for was sometimes less than required. • Fixed an issue where a negative ending balance was incorrectly reported for parts that used multi-level search enabled, along with a SafetyStockQuantityRule of "TimePhasedQuantity", and had in-transit scheduled receipts where Order.Type.ProcessingRule was set to "InTransitExact". Erroneous negative ending balances should no longer be reported when multi-level search is enabled and "intransit exact" scheduled receipts are used.

Table 56.18: Defect fixes and calculation changes in Version 2016.2, Service Update 17

Fix Area / Change	Description
FlatBillDown	Fixed an issue related to how global part substitutes (alternate parts) were reported in the FlatBillDown table. Previously, if a component part had global substitutes, then those alternate parts were shown in a FlatBillDown report. However, when the assembly part itself had global substitutes, then those alternate parts were not shown. For example, assume part B1 was used as a component in assembly A and that part B1 had global part substitutes (alternates) B2 and B3. When reporting under Part A, parts B2 and B3 were shown as flat components. However, when reporting under Part B1, parts B2 and B3 were not being shown as flat components. This issue has been fixed and alternate parts should now be reported as expected in the FlatBillDown table.
Co-products	Fixed an issue where incorrect available dates were sometimes reported on coproduct supplies. This would typically occur when the coproduct and primary part were at the top of a multi-level constraint family. In these cases, portions of the coproduct supplies were not having their available dates updated when supply splits occurred during the allotment of supply to demand.
Multi-level constraints	Fixed an issue related to the use of multi-level constraints along with incremental availability and lot-sizing logic. With this combination of analytics, supply was sometimes reported on time even though a component had an available date of Future. This was seen to occur in cases where the first portion of a component supply was available on time but was not large enough to satisfy the effective minimum or multiple quantity and a second portion of the component supply had to then be used as a result. In such cases, the available date of that second portion (which could be late) wasn't impacting availability of the assembly.
Incremental availability	Fixed an issue where when incremental availability logic was enabled, inappropriate calendar rounding could result in a supply's AvailableStartDate being calculated on a date earlier than the MaterialStartDate for the supply. This was caused by the calculated MaterialStartDate not always being a date on the PartSource.Source.LeadTimeUnit calendar. Therefore, the MaterialStartDate will now always be rounded up to the Source.LeadTimeUnit calendar (if not already on it) before doing AvailableDate and AvailableStartDate calulations on each portion of incremental supply.
Time-phased yield	Fixed a memory leak associated with the use of time-phased yield logic. Note that this memory leak was not present in Service Update 15 and earlier.
Expiry	Fixed an issue related to the use of material expiration logic with dynamic safety stock Previously when OnHand inventory expired and certain dynamic safety stock logic was used (such as, when SafetyStockQuantityRule was set to "DynamicRangeOfCoverage" or "TimePhasedSafetyDays"), new supply would sometimes planned too late to satisfy safety stock requirements and hence risked potential stock outs.

Control table changes

New fields

This section outlines fields added to Control tables in Version 2016.2, Service Update 17.

Table 56.19: Control table fields

Table name	Field name/change
MultiLevelSearchRule	PartSourceLimit This field can be used to limit the number of (effective) part sources that multi-level search logic will consider when planning supply to satisfy a given demand for a part.

Version 2016.2 changes Service Update 16

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 15 to RapidResponse 2016.2 Service Update 16 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 16 including:

- Statistical forecasting enhancements
- Forecast consumption with request date change
- "Constraint pre-planning enhancement" on page 2335
- Defect fixes and calculation changes

Forecast consumption with request date change

A change was made to the forecast consumption logic that consumes relative to request date rather than due date (that is, where **ForecastConsumptionDateRule** is set to "RequestDate"). Previously when consuming forecast using the "RequestDate" setting, in cases where the request date was later than the due date, then consumption would be relative to the due date instead (but request date used otherwise). This has been changed and when using the "RequestDate" setting, forecast consumption is now always be performed relative to the request date (as long as it is a valid date, and regardless of when the due date is)..

Constraint pre-planning enhancement

An enhancement was made to constrained planning logic to optionally allow for any pre-planned quantities to be pre-planned as late as possible. This is done by filling constraint buckets from latest to earliest by splitting orders in smaller quantities sized to fit into the available capacity in the latest bucket. In previous version, if pre-planning was required then after having identified the earliest bucket needed for pre-planning, bucket then capacity was filled from earliest bucket to latest.

To enable this new changed functionality, a new **PrePlanRule** field was added to the **PartSouceType** table with options of "Normal" (existing behavior of pre-planning from earliest to latest bucket) and "JustInTime" (new behavior of pre-planning from latest to earliest).

Statistical forecasting enhancements

When configuring forecast item parameters used in generating a statistical forecast, it is now possible to show confidence intervals using the ARIMA and Step-wise ARIMA statistical forecasting models.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 16.

Table 56.20: Defect fixes and calculation changes in Version 2016.2, Service Update 16

Fix Area / Change	Description
Expiry	Fixed an issue where safety stock supplies for parts that use both expiry and safety stock logic could sometimes expire prematurely (when the originating supply does not expire). In cases where a supply that does not expire was used to satisfy a safety stock demand, then no expiry-related demand was created. Thus, any corresponding safety stock drop would not have its expiry date updated and so would stay at the initial value (the date of the drop) and thereby could not be used to satisfy demand. Such supplies should no longer expire prematurely and thereby might be eligible to satisfy demands (thus, preventing generation of additional planned orders).
Statistical forecasting	Fixed an issue related to use of the R forecast model for statistical forecasting where, following an upgrade to the version of R Server used by RapidReponse, errors were returned when attempting to generate a forecast using the "Rforecast" model.
FlatBillDown	Fixed an issue where the FlatBillDown table was incorrectly reporting details of the globally substitutable primary part of a given component (unexpected components reported in the table). For example, if Assembly A used component B2, and component B2 was a global substitute to part B1 (but B1 was not a substitute for B2), then B1 would incorrectly be shown under A. This issue has been fixed and FlatBillDown table should now only report components that can potentially be used to satisfy demand for the assembly/part.
WhereConsumed	Fixed an issue where the WhereConsumed table were incorrectly reporting NaN (not a number) values for certain supply quantity fields (SupplyQuantity, NewProcessCost, NewPurchaseCost) when a value of 0 (zero) was expected. This was typically seen when coproduct logic was used. The issue has been fixed and expected zero values should now be returned rather than NaN.
BOM rounding logic	Fixed an issue where the PlannedOrder.OnTimeQuantity field was reporting incorrect values in cases where the related BillOfMaterial record used a BOMType.RoundingRule setting of "QtyPerOrder". In these cases, the OnTimeQuantity reported typically aligned to the value in BillOfMaterial.QuantityPer (note that this error did not affect the actual available date reported on the order). This issue has been fixed and the correct OnTimeQuantity should now be reported regardless of RoundingRule setting.

Table 56.20: Defect fixes and calculation changes in Version 2016.2, Service Update 16

Fix Area / Change	Description
Attributes, Expiry, BOM Subsitution	Fixed an issue where when BOM-level part substitution, attribute-based planning, and material expiry logic were all enabled for a given part, and there was a safety stock drop for that part (reduction in its safety stock level), any supply freed up as a result would not be seen as available to satisfy demand which could then result in additional being planned orders generated (excess supply).
Multi-level search	Performance related improvements to the multi-level search analytic.

Input table changes

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 16.

Table 56.21: Modified Input table fields

Table name	Field name/change
ForecastItemParameters	ConfidenceLevel This field can now be used by the ARIMA and Step-wise ARIMA models.

Control table changes

New fields

This section outlines fields added to Control tables in Version 2016.2, Service Update 16.

Table 56.22: Control table fields

Table name	Field name/change
PartSourceType	PrePlanRule Provides an option to do any constraint pre-planning as late as possible. This field has options of "Normal" (existing behavior of pre-planning from earliest bucket required to latest) and "JustInTime" (new behavior of pre-planning from latest bucket possible to earliest)..

Calculated table changes

Modified fields

This section outlines fields in Calculated tables that were modified in Version 2016.2, Service Update 16.

Table 56.23: Modified Calculated table fields

Table name	Field name/change
StatisticalForecast	PredictionIntervalLower This field can now be used by the ARIMA and Step-wise ARIMA models.
StatisticalForecast	PredictionIntervalUpper This field can now be used by the ARIMA and Step-wise ARIMA models.

Version 2016.2 changes Service Update 15

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 15 to RapidResponse 2016.2 Service Update 15 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 15 including:

- Statistical forecasting enhancements
- Defect fixes and calculation changes

Statistical forecasting enhancements

The following changes were made to statistical forecasting:

- When configuring forecast item parameters used in generating a statistical forecast, it is now possible to show confidence intervals using the Linear statistical forecasting model.
- When using the Croston's method, you can now verify individual values for the final estimated non-zero demand at every InterarrivalValue and the final estimated arrival time using the **StatisticalForecastFitOutput** table. In the **ValueName** field, the **SmoothedValue** model output reports the final estimated non-zero demand at every InterarrivalValue and the **InterarrivalValue** model output reports the final estimated arrival time.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 15.

Table 56.24: Defect fixes and calculation changes in Version 2016.2, Service Update 15

Fix Area / Change	Description
Multi-level search analytic fixes and calculation changes	The following changes might be seen if using the multi-level search epic: • A change was made in how demands are processed when multi-level fair share logic is used on parts where **CommitLevel** is set to "MediumExplodePriority". Previously demands at this CommitLevel were treated the same as "High" and splits across demands where only made with the same/original priority. Now at the CommitLevel of "MediumExplodePriority", demands are processed by due date and splits are potentially performed across more demands (which might have performance impacts in some cases). • A single demand within an MLS fair share window will now split if the family has co-by products. • Improved solution output with co-products through better use of co-product supplies. • When a demand is outside the effectivity range of any part source, then multi-level search will now evaluate all part sources in determining which should satisfy the demand. • Addressed an issue where an endless loop could potentially be encountered when using fair share capabilities in multi-level search logic. • Fixed an issue where multi-level search logic was committing some paths with very low yields. • Fixed an issue with the planned order smoothing split logic which could result in a **PlannedOrder.CalcRequestStartDate** that was calculated too early relative to **PlannedOrder.DueDate** (that is, the smoothing logic was applied correctly to the due date but not as expected to CalcRequestStartDate). This issue could lead to too much supply being built ahead. • General performance and memory usage improvements
Material expiry and safety stock	Changes were made to how safety stock was handled on parts that used material expiry logic in order to mitigate against potential negative balance and therefore insufficient stock. Additionally an an issue related to the use of material expiry logic together with safety stock requirements and attribute-based planning where attributes are defined on the **TimePhasedSafety** table. With this combination of settings, internal server errors (exceptions) were being generated. Note also that this combination of settings is not officially compatible in RapidResponse, however their use should not generate an exception.
Push planning	Fixed an issue where expected push planning events did not occur if **PartSource.MultipleQty** was set to "0" or **OrderPolicy.MultipleQtyUsage** was set to "Ignore". In these cases now, the logic should behave as if **MutipleQty** is set to "1" and the push planning will occur as expected.

Table 56.24: Defect fixes and calculation changes in Version 2016.2, Service Update 15

Fix Area / Change	Description
Constrained planning	Fixed an issue where certain independent demands were reported as (avoidably) late when **OrderPriority.SchedulePriority** was set and the part used a **PartType.CommitLevel** of "MediumExplodePriority". With this combination of settings in prior versions, some opportunities to preplan in order to avoid lateness could be missed thus resulting in late availability on demands.
CRP	Fixed an issue where changing the work center referenced on a **CRPOperation** record did not trigger the recalculation of daily and weekly load tables (for example, the **LoadDailyCurrent** and **LoadWeeklyPlanned** tables).
DisaggregationRateByPartCustomer calculation	If the **DisaggregationQuantityRule** field on the **HistoricalDemandCategoryType** table is set to "ConsensusForecast", the adjustments in **ForecastDisaggregationOverride** table and the **DisaggregationInnerCalendar** and **DisaggregationOuterCalendar** fields on the **SOPAnalyticsConfiguration** table are not used.

Input table changes

Modified fields

This section outlines fields in Input tables that were modified in Version 2016.2, Service Update 15.

Table 56.25: Modified Input table fields

Table name	Field name/change
ForecastItemParameters	ConfidenceLevel This field can now be used by the Linear model.
ForecastItemParametersFitOutput	ValueName This field now includes the **InterarrivalValue** model output for the Croston's method. In addition, for the Croston's method, **SmoothedValue** is now the estimated non-zero demand at every **InterarrivalValue**.

Calculated table changes

New fields

This section outlines fields added to Calculated tables in Version 2016.2, Service Update 15.

Table 56.26: New Calculated table fields

Table name	Field name/change
DisaggregationRateByPartCustomer	QuantityWithoutOverride

Modified fields

This section outlines fields in Calculated tables that were modified in Version 2016.2, Service Update 15.

Table 56.27: Modified Calculated table fields

Table name	Field name/change
StatisticalForecast	PredictionIntervalLower This field can now be used by the Linear model.
StatisticalForecast	PredictionIntervalUpper This field can now be used by the Linear model.
StatisticalForecastFitOutput	ValueName This field now includes the **InterarrivalValue** model output for the Croston's method. In addition, for the Croston's method, **SmoothedValue** is now the estimated non-zero demand at every **InterarrivalValue**.

CHAPTER 57: Analytic and data model changes in 2017

In 2017, many new and changed features were included in service updates to Version 2016.2.

Details on the following are available:

- Version 2016.2 changes Service Update 14
- Version 2016.2 changes Service Update 13
- Version 2016.2 changes Service Update 12
- Version 2016.2 changes Service Update 11
- Version 2016.2 changes Service Update 10
- Version 2016.2 changes Service Update 9
- Version 2016.2 changes Service Update 8
- Version 2016.2 changes Service Update 7
- Version 2016.2 changes Service Update 6
- Version 2016.2 changes Service Update 5

Version 2016.2 changes Service Update 14

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 13 to RapidResponse 2016.2 Service Update 14 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 14 including:

- Dynamic safety stock enhancement
- Defect fixes and calculation changes

Dynamic safety stock enhancement

When using dynamic safety stock logic (for example, a **SafetyStockQuantityRule** of "TimePhasedSafetyDaysTotal" or "FractionOfDemand"), a new option allows for specific demand types to be excluded from use in the calculation of safety stock levels. For example, certain types of irregular demand might need to be excluded from use in determining the required safety stock levels.

This is enabled using the new "SafetyStockDemandUsage" setting on the **DemandType** table. In previous releases, if a part used a dynamic safety stock rule then all of its demand types would always be included in the safety stock calculation.

Additionally, for dependent demands on parts using a **SafetyStockQuantityRule** of either "TImePhasedSafetyDaysTotal" or "TimePhasedSafetyDaysMaximum", this new option is used along with the **SafetyStockPolicy.SafetyDaysDependentDemandUsage** setting to determine whether a particular type of dependent demand is used in safety stock logic.

For more information, see "Demand usage in dynamic safety stock calculations" on page 1415.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 14.

Table 57.1: Defect fixes and calculation changes in Version 2016.2, Service Update 14

Fix Area / Change	Description
AttributeMap.Weight calculation	Previously when the **AttributeMap.Weight** field was left at the default value of 0, which indicates that the Weight value should be calculated by RapidResponse, the calculated Weight was not always consistent. Differences might be seen in the calculated Weight which is meant to ensure a higher weight is assigned to records that are likely to have fewer attributes.
UOM conversion on HistoricalSupplyActual	The definition of the unit of measure (conversion) path on the **HistoricalSupplyActual.Quantity** field has been changed from **Header.PartSupplier.SupplierUOM** to **Header.PartSupplier.Part.UnitOfMeasure**
ForecastItemParamtersType.Value default	The default value for the **ForecastItemParametersType.Value** field has been changed from "Default" (which might not always be a defined value in the system) to "*" (this allows the system to pick the value, typically it will be the first alphabetically).
Reschedulable scheduled receipts with safety stock and expiry parts	Previously when reschedulable scheduled receipts used to cover safety stock on expiry parts using a **MUEPoolNettingType.AttributeRule** of "Ignore" were not being properly counted towards the safety stock level. As a result, planned orders could be created before they were needed and the **EarliestExpiryDate** on some supplies was set incorrectly (note that this problem did not occur when attribute-based planning was enabled). This issue has been fixed with **EarliestExpiryDate** values and planned order **DueDate** now set as expected attribute-based planning is disabled and scheduled receipts are rescheduled to cover safety stock on expiry parts.
Part.TotalDemand calculation with BOM substitution	The logic used to populate the **Part.TotalDemand** field has been updated to fix an issue where non-zero/positive values could be reported in cases where there was no demand on the part. This was seen to occur in cases where BOM-level part substitution logic was used and there were extremely small values defined in the **QuantityPer** field for some **BillOfMaterial** records.
Multi-level search performance	Fixed an issue with multi-level search logic where the Netting analytic would hang while performing trials with scheduled receipts on certain parts. This was addressed by a change in multi-level search logic related to how trials look for available input supply.
TimePhasedSafetyDaysMaximum with global part substitution	Fixed an issue related to the use of global part substitution together with parts that use a **SafetyStockQuantityRule** of "TimePhasedSafetyDaysMaximum". Previously with this combination of settings, If a safety stock supply was not associated with a safety stock demand that was satisfied, it could still be used as a "least late input supply" for a substitute supply, but would not be consumed. The logic should no longer use these as substitute supplies.

Control table changes

New fields

This section outlines fields added to Control tables in Version 2016.2, Service Update 14.

Table 57.2: New Control table fields

Table name	Field name/change
DemandType	SafetyStockDemandUsage

Version 2016.2 changes Service Update 13

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 12 to RapidResponse 2016.2 Service Update 13 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 13 including:

- Statistical forecasting enhancements
- Defect fixes and calculation changes

Statistical forecasting enhancements

The following changes have been made to statistical forecasting:

- The Mean Absolute Scaled Error (seasonal) error measure was added for use in fitting a forecast, such as when using the "BestFit" or "Rforecast" forecasting method. It is now available as a setting in the **ForecastItemParametersType.FitMeasure** control field.
- When defining mappings between forecast items, which allows the historical demand from one item to be used in calculating the statistical forecast of another, you can now specify a lag value. Lag represents the **ForecastItemParametersType.IntervalsCalendar** periods after the **HistoryStartDate** at which to begin using the historical demand of the referenced forecast item in statistical forecast calculations. It is now available on the **ForecastItemParametersMap** table.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 13.

Table 57.3: Defect fixes and calculation changes in Version 2016.2, Service Update 13

Fix Area / Change	Description
Yield rounding	Fixed an issue where, in some cases, yield rounding logic would lead to demand available dates that were erroneously set to "Future" . When using yield and yield rounding, and there where there was either a non-zero minimum quantity or a non-zero multiple quantity, the combination of rounding up and down could, in some cases, cause an insufficient planned order quantity would be generated if the desired quantity contained a fraction. In these cases, an additional planned order was being generated to make up for the rounded off quantity but this additional quantity was sometimes lost during rounding when merging the planned orders together.
Multi-level search	Fixed an issue where exceptions were generated in the **SupplyDemand** table if using multi-level search logic along with **IndependentDemand** records that had their **DueDate** set to "Future"
Full-level pegging	Fixed an issue in where parent split supply available dates (that is, **WhereConsumedForDemand.SplitSupplyAvailable**) could sometimes be reported as earlier than the available dates on its late components.
BOM substitution	Fixed an issue related to BOM substitution logic where very small non-zero values would sometimes be reported in **Part.TotalDemand** for a part that had no actual demand reported elsewhere.

Input table changes

This section outlines new and modified Input tables in Version 2016.2, Service Update 13.

New fields

This section outlines new fields added to Input tables in Version 2016.2, Service Update 13.0.

Table 57.4: New Input table fields

Table name	Field name
ForecastItemParametersMap table	Lag

Modified fields

This section outlines fields modified in Input tables in Version 2016.2, Service Update 13.0.

Table 57.5: Modified Input table fields

Table name	Field name/change
ForecastItemParameters table	SeasonalIntervalCount This field can now be used by the Rforecast model.
ForecastItemParametersFitOutput table	ValueName The MeanAbsoluteScaledError (seasonal) option was added.

Control table changes

Modified fields

This section outlines fields modified in Input tables in Version 2016.2, Service Update 13.0.

Table 57.6: Modified Input table fields

Table name	Field name/change
ForecastItemParametersType table	FitMeasure The MeanAbsoluteScaledError (seasonal) option was added.

Version 2016.2 changes Service Update 12

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 11 to RapidResponse 2016.2 Service Update 12 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 12 including:

- Statistical forecasting enhancements
- Statistical forecasting enhancements
- Time-phased yield calculations
- Defect fixes and calculation changes

Statistical forecasting enhancements

The following change was made to statistical forecasting:

- When configuring the statistical forecast, it is now possible to set a unit of measure for forecast items by using the UnitofMeasure field on the ForecastItemParameters table. For parts that are associated with forecast items, a part-specific unit of measure conversion factor can be set using the PartUOMConversion table. The conversion factor is used to convert the part unit of measure to the forecast item unit of measure when calculating the statistical forecast.

Consensus forecast enhancements

When configuring the consensus forecast, it is now possible to specify the weight of the forecast ratios for forecast categories across defined horizons so they can be used to calculate the consensus demand plan. For example, when setting forecast ratios for the consensus forecast, the sales forecast ratio could be set to 100% for the first three months of the horizon, the sales and marketing forecast ratios could be set to 50% in the following three months, and then the marketing forecast ratio could be set to 100% for the remainder of the horizon.

This can be achieved by using the following tables:

- ConsensusForecastRollingHorizon table
- HistoricalDemandHeaderRollingWeight table
- HistoricalDemandHeaderRollingWeight table
- ConsensusForecastRollingHorizonWeight table

Time-phased yield calculations

Yield factors on product and co-products can now be time-phased allowing for these values to change over time. For example, a new product might have one yield factor when it is introduced and then an increasing/improving yield factor over time as processes are improved.

To support this capability, the **TimePhasedYield** table has been added which includes a reference to the **PartSource** table and allows for (optionally) defining multiple time-phased yield values per part source (previously, only a single yield value per source was supported as defined in the existing **PartSource.Yield** field).

For more information about yield, see "Yield" on page 1391, and more information about coproduct yield, see "Co-product and by-product basics" on page 1581.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 12.

Table 57.7: Defect fixes and calculation changes in Version 2016.2, Service Update 12

Fix Area / Change	Description
Capacity resource planning (CRP)	Fixed the following issues: • In cases where no time is required to complete a given operation (that is, all operation times such as setup, run, and queu are all set to "0"), then the **OperationEndDate** on such supplies could be reported as earlier than the **OperationStartDate** (the **OperationEndDate** would be reported as the end of the date before the **OperationStartDate**). • In some cases when CRP variable lead time (**OrderPolicy.UseVarLeadTime** = "CRP") was used, the calculated CRP lead time was sometimes a negative value. As a result, available dates were sometimes reported earlier than intended and in some cases an assembly might have erroneously been reported as available before all of its components were available. This issue was typically seen in cases where all lead times, such as "CRP lead time", were set to "0" (but might also be seen in some cases of non-zero lead times).
FlatBillUp and FlatBillDown (performance issue)	Resolved an issue related to use of expensive queries against the **FlatBillUp** and **FlatBillDown** tables. In some cases, such as when queries on these tables were run against all parts in the database, worksheets which were based on or referenced those tables would generate errors and not open. This issue was caused by RapidResponse running out of memory as the memory pool consumed by this process didn't have any definable limit. To address this issue, the **AlgorithmCacheMemoryLimitMB** setting has been added with a default limit of 2000 (2000MB/2 GB). This setting is expected to be suitable for most environments and when best practices are generally followed. For example, with this default setting, the **FlatBillDown** and **FlatBillUp** tables should be avoided for filtering or in queries that will run against all parts. If needed, however, the default value for AlgorithmCacheMemoryLimitMB can be changed as outlined in the RapidResponse Administration Guide (but it is strongly recommended to only change this value in consultation with Kinaxis Customer Support).

Table 57.7: Defect fixes and calculation changes in Version 2016.2, Service Update 12

Fix Area / Change	Description
Global part substitution	Fixed an issue in global part substitution where if P**artType.SubstitutionPreference** was set to "Primary" and the **Part.PrimarySubstitutionSequence** value (on the primary part) was greater than the **AlternatePart.Sequence** value (on the substitute part), then excess on time planned orders on substitute parts were not always being used before trying to create a new planned order on the primary part. Now when **SubstitutionPreference** is "Primary", the logic performs as expected and on-time excess planned orders from substitute parts should be tried before creating a new planned order on the primary.
Part customer disaggregation	Fixed an issue where the **DissagregationRateByPartCustomer** table did not always report correct disaggregation rates in cases where the **DisaggregationQuantityRule** (on the **HistoricalDemandCategoryType** table) was set to "ConsensusForecast". The correct disaggregation rates should now be returned regardless of whether the "ConsensusForecast" or "Actuals" setting is used.
Multi-level search	Several issues were addressed with regards to multi-level search logic including the following: • Fixed an issue where planned order merging could sometimes results in a small negative ending balance. • Fixed an issue where certain data conditions were resulting in an endless loop within the analytic. • Fixed an issue with planned order splitting, when MultiLevelSearchRule.ShortTermExcess was set to "Reduce", when used in cases where yield is applicable and there is a significant amount of constraint available. With this combination of factor, some independent demand available dates were erroneously being set to "Future". • Fixed a performance issue where multi-level search logic would sometimes taking a long time to complete for parts in cases where multi-level search logging was enabled and the part had a relatively large safety lead time value. • Fixed an issue where when multi-level search logic is used with two-way co/by-products that are globally substitutable for one another, a negative ending balance (supply shortage) would sometimes be erroneously reported for those co/by-products.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 12.

New tables

The following tables are added:

- **ConsensusForecastRollingHorizon table** —This table stores records for defining the order and the duration of the horizon when generating the consensus forecast. For more information, see "ConsensusForecastRollingHorizon table " on page 246.
- **HistoricalDemandCategoryRollingWeight table** —This table stores forecast weight records for forecast categories which are used when generating the consensus forecast within the sales and operations planning process in RapidResponse. Each record in this table applies to a forecast category and indicates the weight given to that forecast category for a specific horizon. For more information, see "HistoricalDemandCategoryRollingWeight table " on page 333.

- **HistoricalDemandHeaderRollingWeight table** —This table stores rolling weights for each header which is used when generating the consensus forecast within the sales and operations planning process in RapidResponse. Each record in this table applies to a specific header and also indicates the forecast weight that is given for a specific duration of the horizon. For more information, see "HistoricalDemandHeaderRollingWeight table " on page 336.
- **TimePhasedYield**— This table can be used to store time-phased yield and coproduct yield values for a given part source. In previous releases, each part source could have only stationary yield and coproduct yield values as set in the **Yield** and **CoProduct** fields on the **PartSource** table (these fields are still used for any part source whose yield values are not defined in the **TimePhasedYield** table). For more information, see TimePhasedYield table.
- **UnitOfMeasureCategory**— This table holds categories used to group related types of unit of measures, and supports unit of measure conversion logic in RapidResponse. For more information, see "UnitOfMeasureCategory table" on page 664.

New fields

This section outlines new fields added to Input tables in Version 2016.2 Service Update 12.0.

Table 57.8: New Input table fields

Table name	Field name
CurveParameters	UnitOfMeasure
ForecastItemParameters	UnitofMeasure
PartSupplier	SupplierUOM
ScheduledReceipt	ParameterDate

Modified tables

This section outlines modified in Input tables in Version 2016.2 Service Update 12.0.

Table 57.9: Modified Input table

Table name	Field name/change
PartUOMConversion table	Now includes the capability to use the **PartUOMConversion** table to define a part-specific unit of measure conversion factor for calculating of the statistical forecast. Prior to this update, the table was only intended to store part-specific unit of measure conversion rates for reporting purposes.

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 12.

New fields

This section outlines fields added to Control tables in Version 2016.2 Service Update 12.0.

Table 57.10: Modified Calculated table fields

Table name	Field name/change
ConstraintUnitOfMeasure	UnitOfMeasure
CRPUnitOfMeasure	UnitOfMeasure
UnitOfMeasure	Category

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2 Service Update 12.

New tables

The following table was added:

- **ConsensusForecastRollingHorizonWeight**-This table reports the weight that is used to calculate the consensus forecast. For more information, see "ConsensusForecastRollingHorizonWeight table" on page 971

New fields

This section outlines fields added to Calculated tables in Version 2016.2 Service Update 12.0.

Table 57.11: Modified Calculated table fields

Table name	Field name/change
Activity	SupplyQtyUnitOfMeasure
CTPActivity	SupplyQtyUnitOfMeasure
CostOfGoodsSold	SupplyQtyUnitOfMeasure
InProcess	QuantityUnitOfMeasure
LateSupply	QuantityUnitOfMeasure
PlannedOrder	ParameterDate
Supply	QuantityUnitOfMeasure
SupplyDemand	SupplyQuantityUnitOfMeasure
WhereConsumed	SupplyQuantiyUnitOfMeasure
WhereConsumedForDemand	SupplyQuantityUnitOfMeasure
WhereConsumedForForecast	SupplyQuantityUnitOfMeasure

Version 2016.2 changes Service Update 11

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 10 to RapidResponse 2016.2 Service Update 11 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 11 including:

- CRP variable lead time with incremental availability
- Safety stock with partial planning
- Defect fixes and calculation changes

CRP variable lead time with incremental availability

It is now possible to use incremental availability logic in CTP calculations together with CRP variable lead time (lead time calculated based on the amount of time needed to run through all required work center operations). Previously, when a part source's **OrderPolicy.VarLeadTime** setting was "CRP", incremental availability would be automatically disabled on the part.

Note that although incremental availability is now compatible with CRP variable lead time, the variable lead time value used is still always the one determined by the Netting analytic. In other word, CRP variable lead time is not recalculated based on reduced and incrementally available supply quantities determined during CTP calculations.

Safety stock with partial planning

Improvements have been made to the logic that uses order splitting to try and reduce the short-term excess that can result when safety stock requirements are used along with partial planning logic (introduced in Service Update 10). For example, this logic has been extended to give enhanced compatibility and results in cases where partial planning with safety stock requirements is used alongside co-product and/or BOM substitution logic.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 11.

Table 57.12: Defect fixes and calculation changes in Version 2016.2, Service Update 11

Fix Area / Change	Description
Part source effectivity	Fixed an issue where a **PartSource** that had its **EffectiveInDate** later than its **EffectiveOutDate** could potentially (and incorrectly) be assigned to a scheduled receipt (but was not used by Netting planning logic). This has been changed so that an **EffectiveInDate** later than **EffectiveOutDate** indicates the part source is never effective and it will never be assigned to scheduled receipts.
Distribution planning	Fixed the following: • Issue where when distribution planning logic is enabled and the **DistributionPlanningRule.AllotmentRule** setting is "Normal", some planned orders would have an undefined available date. • Issue where when there is enough on-time or early supply to cover safety stock requirements, the push planning signal was associated with the wrong demand and the safety stock level ended up not being maintained correctly by CTP calculations. • Issue where the calculated maximum inventory level was not fully respecting inventory pooling logic (for example, the maximum inventory level might be applied to the blank pool rather than the common level thereby causing the sourcing site to push excess inventory out quicker then desired).
Full-level pegging	Fixed an issue with unstable pegging results when on hand inventory is pegged to different allocations having the same due date and quantity.
Partial planning	Fixed an issue where when partial planning logic is enabled along with certain other advanced analytics the **SupplyDemand** table might in some cases incorrectly report a demand as being satisfied whereas e other tables correctly reported the demand as not being fully satisfied.
Multi-level search	Fixed the following: • Issue where an exception was being generated within multi-level search logic due to the wrong demand being pushed into allotments. • Issue where the **CalcRequestStartDate** field was not calculated correctly when multi-level search logic was enabled (the earliest request date was not used in cases where a single planned order supplies multiple independent demands).
Constrained planning	Fixed the following: • Issue where when there were multiple demands over some number of periods whose sum exceeds the constraint available in those periods thereby causing late planned availability on planned orders, and time-phased safety stock levels are changing in these periods, then unnecessary short-term excess was being generated from planned orders. • Issue where the **AbsolutePrePlanLimit** was not applied correctly in constraint scheduling if this capability was used together with multi-level search smoothing logic. In cases where smoothing logic moved planned order dates later, pre-planning would still happen but the resulting dates were not accurate, and in cases where smoothing logic moved planned order dates earlier, pre-planning would be disabled. • Issue on the **SourceConstraint** table where the **EffectiveInDate** field was added as a key field in a previous version, however the corresponding IdentificationFields list for this table was not updated (used in data change tracking).

Table 57.12: Defect fixes and calculation changes in Version 2016.2, Service Update 11

Fix Area / Change	Description
Capable-to-promise	Fixed the following: • Issue where if an assembly's available date, which was determined from component availability, was not defined in the applicable calendar than the available date was moved to the next earliest date in the calendar and potentially earlier than some component's available date (assembly available date will now not be set earlier than component available date in such cases). • Issue where the analytic memory limit was being hit during CTP calculations in some cases where demand quantities were extremely large and scheduled receipt quantities were relatively very small.
Statistical forecasting	Fixed the following: • Issue with the characters "%s" being inappropriately appended to the end of an error message related to the PeriodsForwardCoverage function. • Issue where the **MeanAbsolutePercentageError** function would fail if the trend sample were less than 3 (trend samples of any value greater than or equal to 1 should now be accepted).

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 11.

New fields

This section outlines new fields added to Input tables in Version 2016.2 Service Update 11.0.

Table 57.13: New Input table fields

Table name	Field name
PartValidation	CommentsForDemand
PartValidation	CommentsForOnHand
PartValidation	CommentsForSupply
PartValidation	KnownIssueForDemand
PartValidation	KnownIssueForOnHand
PartValidation	KnownIssueForSupply

Modified fields

This section outlines fields modified in Input tables in Version 2016.2 Service Update 11.0.

Table 57.14: Modified Input table fields

Table name	Field name/change
PartValidation	Comments is renamed to CommentsByPart
PartValidation	KnownIssue is renamed to KnowIssueByPart

Table 57.14: Modified Input table fields

Table name	Field name/change
SAO_MD04	MNG01_RecordedRequiredQuantity has its data type changed from Integer to Quantity.

Version 2016.2 changes Service Update 10

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 9 to RapidResponse 2016.2 Service Update 10 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes

- Analytic and calculation changes
- Input table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 10 including:

- Multi-level search with constraint/material fair share
- Multi-level search logging enhancement
- Partial planning with safety stock enhancement
- Statistical forecasting enhancements
- Defect fixes and calculation changes

Multi-level search with constraint/material fair share

Multi-level search logic now supports the fair-sharing of material and/or constraints at a given driver level as well as multiple levels in the structure below (existing fair forms of fair-share logic were limited or disabled when used with multi-level search). When multi-level search is used with partial planning logic, this can provide a fair way of leaving demands unsatisfied across customers, sites, or so on. This functionality is enabled at the order priority level and several supporting fields are added to the **OrderPriority** table (**MLSAllocationRule**, **MLSFairshareCalendar,** and **MLSFairShareSliceFactor**).

For more information about fair-sharing in multi-level search logic, see "Fairshare of material and constraints" on page 1868

Multi-level search logging enhancement

When logging details of demands left unsatisfied due to partial planning and multi-level search logic, a new option allows for the consumption of unused capacity when calculating the additional capacity/material needed to fulfill an unsatisfied demand (on a demand by demand basis). This is supported through a new setting added to the **MultilLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** control field, "FullQuantityPrimaryPathAfterNetting", which can be used to provide a consistent picture of what is needed resolve a particular unsatisfied demand.

Partial planning with safety stock enhancement

When using partial planning logic along with safety stock requirements, changes have been made to help reduce the short-term excess that can result from safety stock drops. This is implemented through the existing "Reduce" setting in the **MultiLevelSearchRule.ShortTermExcess** control field. The logic

implemented by this setting has been extended to allow splitting of existing planned orders based on the demand requirements.

Statistical forecasting enhancements

The following changes have been made to statistical forecasting logic:

- Two new fit/error measures are added for use in fitting a forecast, such as when using the "BestFit" forecasting method. The new supported measures are "Symmetric Mean Absolute Percentage Error" and "Mean Absolute Scaled Error". These are both now available as settings in the **ForecastItemParametersType.FitMeasure** control field.
- When processing historical data for use in generating the statistical forecast, it is now possible to begin with the first non-zero data point in the specified range (previously, calculations would begin with the data point determined by ForecastItemParameters configuration, even if that included a zero data point). This option is controlled through the new **ForecastItemParameters.HistorySkipLeadingZeros** field.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 10.

Table 57.15: Defect fixes and calculation changes in Version 2016.2, Service Update 10

Fix Area / Change	Description
Constraint fair sharing	Fixed an issue where after Netting applied fair sharing as expected, CTP logic gave the impression fair sharing had not taken place (due to the backward scheduling phase consuming constraint before AbsolutePrePlanLimit).
Lot size on incremental splits with constraint	Fixed an issue where part source order minimums were not respected in incremental availability splits if the ConstraintMinimumRule/ConstraintMultipleRule fields on the PartSourceType table were set to "MultiplePeriods".
Available calculation in PeriodConstraintLoad	Fixed an issue where PeriodConstraintLoad.Available field could return a negative value in cases where the sum of all ConstraintAvailable records in a period returned a negative value, (the field should have returned zero rather than a negative number in these cases).
FLPConstraint pegging	Fixed an issue in the FLPConstraint table, where if ConstraintUsed.Load was less than the value in WhereConsumed.SupplyQuantity, the constraint wasn't pegged to the WhereConsumed record that actually used it (but rather spread amongst all WhereConsumed records).
ConstraintUsed.Date calculation	Fixed an issue where, with incremental availability turned on and there are multi-level constraint families, it was possible for ConstraintUsed.Date to be earlier than corresponding PlannedOrder.AvailableStartDate.
StepWiseARIMA forecast model	Fixed the following issues related to use of StepWiseARIMA model in statistical forecasting: • An issue where the StepWiseARIMA forecast model would not generate a statistical forecast for an item if all of its HistoricalDemandActual records had a uniform quantity over the entire historical period (instead of generating statistical forecast records with the same quantity). • An issue where StatisticalForecastFitOutput calculations could report the same ARIMA model more than once.
Phantom part explosion	Fixed an issue where independent demands would not explode properly through a phantom part.

Table 57.15: Defect fixes and calculation changes in Version 2016.2, Service Update 10

Fix Area / Change	Description
Spread forecast with partial planning	Fixed an issue where if forecast spreading was used with partial planning logic, then an on-time forecast demand could sometimes show as having its AvailableDate at "Future".
Demand.Quantity negative values	Fixed an issue where Demand.Quantity could return a negative value if transfer dependent demands reference the "Partial" setting in PartSource.Type.PlannedOrderSupplyType.AllocationDemandType.PlanningRule (due to subtracting the already subtracted IgnoredQuantity).
Supply allotment logic leads to endless loop	Fixed an issue were allotment logic could sometimes end up in an endless loop if the remaining quantity to be satisfied for the demand being processed was relatively small (less than 1/1000 of the original demand quantity).
Full-level pegging missing parent order ID	Fixed an issue where a top level planned order, driven directly by an independent demand, couldn't be fulfilled by any supply along the BOM path (could occur in cases where yields are different in the product structure and constraint scheduling creates a mismatch between the allotment start of the parent part and current part).
AlternatePart recursion	Fixed an issue where RapidResponse Server would crash when running certain queries, such as against the FlatBillUp table, if recursive records existed in the AlternatePart table.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 10.0.

New fields

This section outlines new fields added to Input tables in Version 2016.2 Service Update 10.0.

Table 57.16: New Input table fields

Table name	Field name
ForecastItemParameters	HistorySkipLeadingZeroes
PartSolution	SafetyStockItemCreate

Modified fields

This section outlines fields modified in Input tables in Version 2016.2 Service Update 10.0.

Table 57.17: Modified Input table fields

Table name	Field name
PartValidation	Part This field is set as the key on the table (was previously keyless)
SAP_MD04	MATNR_Part This field is renamed/replaced by MD04_Ma

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 10.0.

New tables

The following tables were added:

- **IPOConfiguration**—Contains settings supporting the Inventory Planning and Optimization application.

New fields

This section outlines new fields added to Control tables in Version 2016.2 Service Update 10.0.

Table 57.18: New control table fields

Table name	Field name
OrderPriority	MLSAllocationRule
OrderPriority	MLSFairShareCalendar
OrderPriority	MLSFairShareSliceFactor

Modified fields

This section outlines fields modified in Control tables in Version 2016.2 Service Update 10.0.

Table 57.19: Modified control table fields

Table name	Field name
ForecastItemParametersType	FitMeasure Two new options, MeanAbsoluteScaledError and SymmetricMeanAbsolutePercentageError are added.
MultiLevelSearchRule	UnsatisfiedDemandLoggedQuantityRule The FullQuantityPrimaryPathAfterNetting option is added, and the previously existing option FullQuantityPrimaryPathLoadOnly is removed.

Version 2016.2 changes Service Update 9

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 8 to RapidResponse 2016.2 Service Update 9 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Namespace changes
- Input table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 8 including:

- "Warning for negative forecast replaced by zero" on page 2363
- Defect fixes and calculation changes

Warning for negative forecast replaced by zero

Statistical forecasting calculations will now generate warning in cases where a negative forecast is replaced by zeros. Previously, if negative forecast values were present they were replaced by zeros without warning which could give the impression that no forecast was generated

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 9.

Table 57.20: Defect fixes and calculation changes in Version 2016.2, Service Update 9

Fix Area / Change	Description
Full-level pegging	The following fixes were made related to full-level pegging logic: • Fixed an issue with memory comparison that could lead to unstable WhereConsumedForSupply.NeedQuantity results • Fixed an issue with memory handling that could lead to unstable results with respect to the number of records generated in the WhereConsumedForSupply table. • Fixed an issue with sorting conditions for excess supply that could lead to unstable WhereConsumedForDemand.NeedQuanttiy results.
Multi-level search	The following fixes were made related to multi-level search logic: • Fixed an issue in multi-level search logic where exceptions could be generated in cases when, within a product structure, a component is using multi-level search logic but the assembly is not. • A performance improvement was made in cases where there are multiple late part sources that are made the same way. If there are two make part sources using the same BOMAlternate and neither uses constraint, then they can be considered equivalent (that is, MLS logic can avoid evaluating the second part source because it will not provide a better solution). Performance improvements from this are most likely to be seen in cases of large numbers of equivalent make part sources, where they all provide supply late due to lower level material constraints.
Co/by-products	Fixed an issue related to co/byproduct logic, where planned orders would sometimes be reduced too much and the corresponding CoByProductSupply is similarly reduced resulting in shortages on the part the CoByProductSupply is for.
Partial planning	The following fixes were made related to partial planning logic: • Fixed an issue where when partial planning was used with allotment overrides, constraint, and reschedulable scheduled receipts, late demands outside of tolerance would not always be left unsatisfied. • Fixed an issue where a (forecast) demand that was partially on-time and partially left unsatisfied was reported as available at "Future" but should have been reported as available at its due date.

Namespace changes

The following namespaces were added:

- **Validation**—Contains tables intended for validating data brought in to RapidResponse from an SAP data source.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 9.

New tables

The following tables were added:

- **LogisticsLocation**—Identifies locations used for cross-docking. Supports new mapping capabilities in RapidResponse. For more information, see "LogisticsLocation table" on page 392.
- **PartValidation**—Contains fields that are used for recording information about parts used during input validation. For more information, see "PartValidation table" on page 479.
- **SAP_MD04**—Contains fields that represent stock requirement data extracted from an SAP data source. This table is intended to be compared to data in other RapidResponse tables to verify the data was extracted correctly. For more information, see "SAP_MD04 table " on page 567.
- **SiteManager**—Identifies individuals (site managers) who have responsibility for a given site. Supports new mapping capabilities in RapidResponse.
- **SiteManagerType**—Used to categorize site managers. Supports new mapping capabilities in RapidResponse.

New fields

This section outlines new fields added to Input tables in Version 2016.2 Service Update 9.0.

Table 57.21: New Input table fields

Table name	Field name
Carrier	LogisticsLocation
Shipment	Airport
Shipment	Customs
Shipment	Port
Site	Address
Source	EstimatedCost

Version 2016.2 changes Service Update 8

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 7 to RapidResponse 2016.2 Service Update 8 that affect the data model and RapidResponse calculations.

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 8 including:

- Report unplanned safety stock within lead time
- Ignore demands outside part source effectivity
- Modified MLS part source selection criteria
- Enhanced logging for MLS with partial planning
- Defect fixes and calculation changes

Report unplanned safety stock within lead time

It is now possible to calculate safety stock from **RunDate**, including within a part's lead time, but to only plan those safety stock levels that are outside of the part's lead time (the safety stock levels calculated within lead time are used for reporting purposes only). This can be achieved using the new "LeadTimeReportUnplanned" setting on the **SafetyStockDateRule** field of either the **SafetyStockPolicy** or **PartType** tables (this option is similar to the existing "LeadTime" setting except for the reporting of safety stock within lead time).

This might be useful for cases where safety stock is not meant to be planned within lead time, but where the calculation of that safety stock level is used for other reporting purposes (for example, in safety stock metrics). Details of safety stock levels calculated but not planned within lead time due to this setting are reported in **Activity** table records with a **Type** value of "UnplannedSafety".

For more information, see "SafetyStockDateRule" on page 880.

Ignore demands outside part source effectivity

It is now possible to suppress the planning of a new planned order for a demand if that demand's due date (less applicable safety lead time) falls outside of part source date effectivity. Previously, if a demand due date was outside of the effectivity period for all potential part sources, the demand would then be moved forward or backward into the effective period for a part source.

To support this capability, a new control field, **SourceRule.DateEffectivityRule** has been added. With the default setting of "Normal", the behavior is as in previous versions (demands can be pushed or pulled into the effective period for a part source when needed). Instead, if using the "AdjustedDemandDate" option, when a demand's due date (less applicable safety lead time) falls outside of the part source's effectivity period, no new supply can be planned for the demand (that is, the demand goes unsatisfied).

For more information about part source selection, see "Part source selection process" on page 1378.

Modified MLS part source selection criteria

Enhancements have made to the part source selection criteria used in multi-level search logic to give greater control over how part sources are chosen to source a supply. This includes the following changes:

- On hand supply and scheduled supply can now be treated as equal at the point in selection criteria where otherwise equivalent part sources are being compared to determine the supply type associated with the lowest level of component supply to be used by each. Previously, sources using on hand component supply were preferred to those using scheduled supply. This is now configurable through the new **MultiLevelSearchRule.ScheduledSupplyRule** field (regardless of the setting in this field, both on hand and scheduled supply are still preferred to new planned supply).
- It is now possible to sequence or prioritize between part sources of a given Priority level (for example, to prioritize amongst secondary part sources). This can be done using the new **PartSource.MLSPreTargetSequence field** which is added to the multi-level source selection criteria and used to distinguish between part sources of the same priority that can provide the supply on-time/least late and with the same level/type of component supply. In other words, it is used directly before **PartSource.Target** in the multi-level selection criteria.
- An additional (secondary) level of priority is also added after PartSource.Target in the multi-level selection criteria, and used to prioritize when part sources that are otherwise equivalent up to and including their Target value. This is controlled through use of the PartSource.SecondaryPriority field which was added in a previous 2016.2 Service Update (this field is also used in standard source selection logic for certain rules as well as when assigning part sources to scheduled receipts).

For more information about multi-level search source selection, see "Multi-level part source selection criteria" on page 1877.

Enhanced logging for MLS with partial planning

Enhancements have made to the logging of unsatisfied demands from partial planning and multi-level search logic (in MLS logging tables lsuch as **MLSActivity**, **TrialPlannedOrder**, and **TrialConstraintShortage**).

Previously, if the unsatisfied demand quantity would be satisfied by multiple supply portions, then only details of the first portion of the demand to be left unsatisfied and the supply amounts which would have been assigned to it are reported in the MLS logging table. That is, just the initial portion that caused multi-level search logic to stop planning the demand and leave some or all of it unsatisfied is reported. In this service update, it is now possible to report and log details of the entire unsatisfied demand quantity and all late supply portions that would potentially be required to satisfy it on time (or within tolerance).

The **MultiLevelSearchRule.UnsatisfiedDemandLoggedQuantityRule** setting has been added to provide control over the level of logging performed on unsatisfied demands from multi-level search logic with partial planning. For more information, see "Logging unsatisfied demands from partial planning" on page 1866.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 8.

Table 57.22: Defect fixes and calculation changes in Version 2016.2, Service Update 8

Fix Area / Change	Description
MLSTrialLimitPerDemand across all levels in structure	The AnalyticsConfiguration.MLSTrialLimitPerDemand field usage was modified in order to address potential performance issues on parts that use multi-level search logic. This trial limit is now applied across all levels in a data structure, whereas previously it was applied per demand and therefore only checked at the top level. Now once the limit is reached, across all levels, if a viable solution has reached the analytic will stop trying other paths. This might typically lead to a measurable MLS performance improvement with minimal to no impact on solution quality (in some cases, small differences will be noticed). Note that performance improvements will be most noticeable for parts that have product structures that are very deep (many levels) and/or very broad (many PartSource records per part).
Attribute reporting on IndependentDemand records	Fixed an issue where attributes were not reported correctly in the IndependentDemand if planned order merging logic was applied to constrained planned orders (in some cases, when attribute-based planning logic was used and part sources were constrained, incorrect unsatisfied demand portions could be reported).
Multi-level search logic with constrained part sources	Several issues related to multi-level search logic when used with constrains, were resolved including: • Fixed an issue where, when multi-level search logic was used with partial planning and constrained part sources, unnecessary late supply could be generated leading to availability on demands being pushed late. This could occur in cases where a part using multi-level search logic and partial planning had multiple constraints, and one of the constraints (the first to be processed by the analytic) had availability while the other didn't. With this change, some improvements in multi-level search logic results might be seen on parts using multi-level search on parts having multiple constraints per part. • Fixed a performance issue related to the multi-level search analytic when used together with large numbers of part sources when there are many constraints active on each of those part sources • Fixed an issue related to how planned orders are merged together in multi-level search logic when lot-sizing and constraints are applicable.
Partial planning and past due demands	Fixed an issue where in some cases with using partial planning logic, the available date on an independent demand due before RunDate (and which could not be fully satisfied but was partially satisfied by on hand inventory) was reported as "Past" where it should have been report at the RunDate.
PartSource.PTFDate calculation	Fixed an issue where the PartSource.PTFDate was reported incorrectly in cases where PTFRule was set to "LastDueLead" and there was a supply allocation added and assigned to the part source on a date later than lead time . Note in this case, the planning results always respected the "LastDueLead" setting, even though the PTDDate could be reported incorrectly prior to this fix.
FLPConstraint table missing records	Fixed an issue where, in some cases, the FLPConstraint table was missing expected records related to activity that was reported on the ConstraintUsed table.
ScheduledReceipt records with input dates beyond calendar markers	Fixed an issue where when scheduled receipt input dates (for example, due or start date) are set later than the last CalendarDate markers, analytic calculations can end up in an endless loop due to constraint dates being set to Future.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 8.

New fields

The following table lists fields added to Input tables in Version 2016.2 Service Update 8.

Table 57.23: New input table fields

Table name	Field name
PartSource	MLSPreTargetSequence

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 8.

New fields

The following table lists fields added to Control tables in Version 2016.2 Service Update 8.

Table 57.24: New control table fields

Table name	Field name
MultiLevelSearchRule	ScheduledSupplyRule
MultiLevelSearchRule	UnsatisfiedDemandLoggedQuantityRule
SourceRule	DateEffectivityRule

Modified fields

The following table lists fields modified in Control tables in Version 2016.2 Service Update 8.

Table 57.25: Modified control table fields

Table name	Field name
PartType	SafetyStockDateRule. A new option, LeadTimeReportUnplanned, was added.
SafetyStockPolicy	SafetyStockDateRule. A new option, LeadTimeReportUnplanned, was added.

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2 Service Update 8.

Modified fields

The following table lists fields modified in Calculated tables in Version 2016.2 Service Update 8.

Table 57.26: Modified calculated table fields

Table name	Field name
Activity	Type A new potential enum value, UnplannedSafety, was added.
DemandException	Type A new potential enum value, SourceEffectivityWindow, was added.

Version 2016.2 changes Service Update 7

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 6 to RapidResponse 2016.2 Service Update 7 that affect the data model and RapidResponse calculations.

- Analytic and calculation changes
- Input table changes
- Control table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 7 including:

- Days of supply changes (single and multi-echelon inventory optimization)
- Defect fixes and calculation changes

Days of supply changes (single and multi-echelon inventory optimization)

Enhancements have been made to days of supply safety stock logic in both single-echelon inventory planning and multi-echelon inventory optimization logic (items where **SafetyStockItemType.TimePhasedProcessingRule** is "DaysOfSupplyBackward" or "DaysOfSupplyForward").

This includes the following:

- A new control field (**SafetyStockItemType.ExcludePastDueDemands**) allows for demands due before the Interval containing RunDate to be excluded from the safety stock calculation. This option allows you to retain records of past due demands to give visibility to your priorities without affecting safety stock recommendations.
- A new input table (**SafetyStockRangeOfCoverage**) gives greater control for specifying the range of demands that are used in the safety stock calculation.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 7.

Table 57.27: Defect fixes and calculation changes in Version 2016.2, Service Update 7

Fix Area / Change	Description
Full-level pegging when BOMType.RoundingRule is set to "Up"	An issue was fixed in full-level pegging calculations where, in some cases, when the associated BOMType.RoundingRule was set to "Up", only a portion of a supply would be reported and pegged to demand in the WhereConsumed table (but expected single-level results were generated in SupplyDemand table). Such supplies should now be appropriately reported and fully pegged to demand in the WhereConsumed table.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 7.

New tables

This section outlines new Input tables in Version 2016.2 Service Update 7.

- **SafetyStockRangeOfCoverage**—provides additional options for specifying the range of demands that are involved in time-phased safety stock calculations using days of supply with single-echelon inventory planning or multi-echelon inventory optimization. For example, the length of the range over which average demand is calculated can be specified as a number of "IntervalsCalendars" intervals, and the start date of the this range can now be optionally offset relative to the date for which safety stock is being calculated.

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 7.

New fields

The following table lists fields added to Control tables in Version 2016.2 Service Update 7.

Table 57.28: New Control table fields

Table name	Field name
SafetyStockItemType	ExcludePastDueDemands

Version 2016.2 changes Service Update 6

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 5 to RapidResponse 2016.2 Service Update 6 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service, Update 6 including:

- Additional multi-sourcing options
- Modified expiry logic for DependentStart
- Safety lead time applied at request date
- Multi-echelon inventory optimization enhancements
- Defect fixes and calculation changes

Additional multi-sourcing options

The logic to determine how new supply requirements are sourced or split between eligible part sources has been enhanced with two control settings added to the **SourceRule.AllotmentRule** field ("OngoingToDate" and "ProportionalLotSize").

The **OnGoingToDate** option will source each supply requirement from the single eligible part source that is furthest away from its quota rating based on all supply sourced to date (up to the current requirement). This is similar to the existing "OnGoing" option with the main difference being that this new option does not consider the size of the current requirement or the impact on quota/expected target if a given source is not chosen to supply the current requirement.

The **ProportionalLotSize** option will split sourcing of supply between eligible part sources using their specified Target values similar to the existing "Proportional" option. However with this new option, supply is sourced starting with the highest Target part source, lot-sizing is applied to that generated supply, and the remaining quantity to be split between the other sources adjusted for this lot-sized supply quantity. Thus, this option can reduce excess from lot-sizing that might be seen with the "Proportional" option however the actual supply splits might not end up matching the target percentage splits.

For more information about multi-sourcing, see "Part source selection" on page 1377.

Modified expiry logic for DependentStart

Expiry date logic has been modified so that the calculated expiry date on assemblies having an **ExpiryRule** of "DependentStart" is less likely to be impacted by components having an **ExpiryType** of "Normal". Additionally, input **ExpiryStartDate** fields have been added have been added to the OnHand and ScheduledReceipt tables, and these fields are now used in setting the calculated **ExpiryStartDate**

for these supplies (based on configuration settings, they are also potentially used in expiry date calculations). For more information, see "DependentStart" on page 1619.

Safety lead time applied at request date

Safety lead time can now, optionally, be applied to the demand request date in addition to the demand due date. Additionally safety lead time can now be applied to a demand only in cases where doing so doesn't make other demands worse off than they would be if that demand were planned strictly to its due date. This functionality is enabled through two new settings that have been added **OrderPriority.TwoDatePlanningRule** field ("UseWithSoftSafetyLeadTime" and "UseWithHardSafetyLeadTime", each of these also have effect of enabling two-date planning for a given priority level).

With the **UseWithHardSafetyLeadTime** setting ,Netting logic plans to cover the demand by its due date less safety lead time, while CTP logic first attempts to satisfy the demand by that same due date less safety lead time and then attempts to satisfy the demand by its request date less safety lead time (if this can be done without making other demands worse off).

With the **UseWithSoftSafetyLeadTime** setting, Netting logic initially plans to cover the demand by its due date (ignoring any safety lead time), and CTP logic will first try to satisfy that same demand due date and then attempts to move it closer to its request date less safety lead time (if this can be done without making other demands worse off).

For more information, see "Apply safety lead time at request date (optional)" on page 1807.

Multi-echelon inventory optimization enhancements

Several enhancements have been made to multi-echelon inventory optimization including the following:

- Inventory and solution costs for all the different iterations that were performed can now be reported using new tables such as **MEIOFamilyIterationResult** and **MEIOStageIterationResult.** This gives insight into how analytic results changed or improved from iteration to iteration (previously, only the lowest cost iteration was reported). To enable this type of reporting for a given safety stock item, the **SafetyStockItem.ReportIterationResults** field should be set to "Y".
- Approximations of the expected impact on service and inventory levels from implementing MEIO recommended safety stock settings can now be reported in the new **MEIOStageSimulation** table. These are based on demand and supply simulations, and can be enabled via the **MEIOSimulationSteps** and **MEIOSimulationTrials** fields on the **AnalyticsConfiguration** table.
- Calculations that rely on time-series computations, such as are reported in the **SafetStockHistoricalDemand** and **SafetyStockHistoricalForecast** tables, can now be configured to begin with the first non-zero data occurrence in the period for which historical demand and forecast data are collected (remove all bucketed values up to the first non-zero bucket). This is configured by setting the new **SafetyStockItem.HistoricalSkipLeadZeros** field to "Y". Consider using this feature if, for example, a product that is configured as a safety stock item was introduced in June, but it has **HistoricalDemandActual** records with zero quantity starting in January.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 6.

Table 57.29: Defect fixes and calculation changes in Version 2016.2, Service Update 6

Fix Area / Change	Description
Co-products and material expiry	The following issues related to the use of co/by-product supplies with material expiration were addressed: • Previously, the expiry date on a co-product supply could be calculated without consideration for the applicable value in **BOM.ITEOffset** (if the co-product used an **ExpiryRule** of "Dependent" or "DependentLimit". This has been fixed and the **CoByProductSupply.ExpiryDate** field now accounts for any interval to expiry offset regardless of the **ExpiryRule** setting. • Fixed an issue where the expiry date on a co-product supply could be impacted if a value was specified in the **BillOfMaterial.LeadTimeOffset**. Such values will now be ignored in expiry date calculations (expiry on co-product should be determined by the primary supply's expiry date and, if used, **BillOfMaterial.IntervalToExpiryOffset)**
ExpiryStartDate was erroneously set to "Past"	Previously, when using material expiry logic a supply's **ExpiryStartDate** would sometimes be set to "Past" instead ts actual **AvailableStartDate** under certain data conditions. This was occurring for parts having an **ExpiryRule** of "Self", where **ExpiryType.ProcessingRule** was set to "LimitingComponent", and **PartSourceType.ExpiryDateRule** was set to "StartDate". Expiry start dates are now populated correctly in these cases.
DemandException records for MLSSourceEffectivity	Previously, **DemandException** records with a **Type** value of "MLSSourceEffectivity" could be generated in some cases where the supply due date and expected component availability were still within the part source's effectivity range. This would occur in cases where the component had a constraint rate of zero. In such cases now, the DemandException record should not be generated and the supply should be planned at "Future" rather than going unplanned.
DependentStart expiry calculations	Calculated expiry date and expiry start date calculations might show differences for parts that have an **ExpiryRule** of "DependentStart" and have components using an **ExpiryType** of "Normal" (such components should not influence expiry of the assembly).
Input allocations with constraint	Under certain data conditions in prior releases, an internal server error was generated by the Netting analytic when processing input supply allocations that required constraint. This issue has been fixed and input allocations requiring constraint should now be processed as expected.
ReschedDate with constraint pre-planning	Previously when a reschedulable scheduled receipt was pre-planned to earlier periods for constraint usage (when **ConstraintType.FillSchedule** was set to "Y"), the receipt's **ReschedDate** could be left at the receipt's **DueDate**. In such cases now, the **ReschedDate** will be set to the last date of the bucket where constraint was consumed (this is consistent with how planned orders are treated).

Table 57.29: Defect fixes and calculation changes in Version 2016.2, Service Update 6

Fix Area / Change	Description
Partial planning logic issues	Several issues were fixed relating to partial planning logic (DemandType.**PlanningRule** is "Partial") including: • In some cases, demands configured for partial planning were being planned outside of the part's unsatisfied demand tolerance and satisfied by late supply (when they should have been ignored and left wholly or partially unsatisfied). • Inconsistent supply selection with partial planning could lead to a customer demand that was (correctly) left unsatisfied not being reported correctly in the **IndependentDemand.UnsatisfiedQuantity** field nor logged in the **MLSActivity** table. • When partial planning was used with multi-level search, in some cases, a later demand could incorrectly be assigned on-hand supply and an earlier demand assigned planned supply. • When partial planning was used with multi-level search logic and co-products, incorrect planning results could be seen in cases where late co-product supply was available to a demand.
Days of supply and multiple demands before PTFDate (with multiple sources)	Previously, when days of supply logic was used with multiple part sources and there were multiple demands due before the calculated **PTFDate**, demands were not always grouped and split properly resulted in more planned orders than required. This issue has been fixed and the expected supply splits with days of supply planning should now be seen at PTFDate.
Alternate part relationships in FlatBillUp/FlatBillDown reporting	Previously, records in the **FlatBillDown** and **FlatBillUp** tables would always report alternate relationships regardless of their effectivity settings. These tables will now ignore and not report relationships in either of the following cases: • **AlternatePartType.ProcessingRule** is set to "Ignore" • if **AlternatePartType.ProcessingRule** is set to "Substitute" and **AlternatePartType.EffectivityRule** is set to "Never".
Multi-level search logic with two-date planning and safety lead time	An issue was fixed where when multi-level search logic was used with two-date planning and safety lead time, in some cases, planned orders were being unnecessarily split (too many created).
Request date propagation under allocations	Previously planned orders generated to satisfy demand from **Allocation**, **NewAllocation**, or **NewTransferAllocation** records would not inherit request date details from the parent scheduled receipt (CalcRequestStartDate was left as undefined on the planned orders and could lead to unexpected results when using "soft" safety lead time). Now, request date information should be propagated from demands to allocations through the scheduled receipts they are associated with and down to the corresponding planned orders.
Multi-level constraints and constraint usage reporting	An issue was fixed where, in some cases, if multi-level constraints where present constraint usage was not being reported (even though planning logic was correctly respecting the constraints).
TrialConstraintShortage and top-level parts	An issue was fixed where the **TrialConstraintShortage** table would not report details of constraint shortages if they occurred on the top-level part. Such cases are now reported in this table.

Table 57.29: Defect fixes and calculation changes in Version 2016.2, Service Update 6

Fix Area / Change	Description
SafetyStockDemandParameterSet table with invalid calendar setting	Previously, the **SafetyStockDemandParameterSet** table would never be populated for a safety stock item if either of the **SafetyStockItemType.CycleCalendar** or **SafetyStockItemType.PeriodCalendar** fields did not reference a valid calendar. However, these fields are not relevant to items that don't use time-phased safety stock calculations, so an invalid calendar will now only prevent the **SafetyStockDemandParameterSet** table from being populated in cases where the **SafetyStockItem.TimePhasedProcessingRule** field is set to a value other than "Ignore".
PushTransferId field in CTPActivity	An issue was fixed in distribution planning logic where the calculated **PushTransferId** values on the **CTPActivity** table were sometimes rearranged.
ExpiryStartDate field for demands in CTPAcitivity	An issue was fixed where demands could report a value of "Future" in their **ExpiryStartDate** field on the **CTPActivity** table (this field should now report a value of "Undefined" for demands)
SupplyAvailableDate field for inventory transfers in WhereConsumed	An issue was fixed where **WhereConsumed** records pertaining to inventory transfers would report a **SupplyAvailableDate** of "Past". Instead for inventory transfers (where incremental availability is turned off), the **WhereConsumed.SupplyAvailableDate** should now equal the **WhereConsumed.SplitAvailableDate**.
Available field for "CumMax" constraints in PeriodConstraintLoad	An issue was fixed where the **Available** field in the PeriodConstraintLoad table always returned a value of 0 (zero) in cases where **ConstraintType.ProcessingRule** was set to "CumMax". Now, the **Available** will report the CumlativeMax amount less any decrease in availability due to constraint consumption.
CalcAverageLeadTime in SafetyStockItem	An issue was fixed where a single-echelon lead time outlier correction didn't affect the results reported for **SafetyStockItem.CalcAverageLeadTime** and **SafetyStockItem.CalcStandardDeviationLeadTime**.
CalcStartDate and CalcFinishDate on in Project	Previously, in cases where a project had no tasks defined, the **CalcStartDate** and **CalcFinishDate** fields on the **Project** table would be left undefined. In such cases now, they should be set to the values in **StartDate** and **FinishDate** fields on the **Project** table.

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 6.

New fields

This section outlines new fields added to Input tables in Version 2016.2 Service Update 6.

Table 57.30: New Input table fields

Table name	Field name
OnHand	ExpiryStartDate
PartSource	SecondaryPriority
SafetyStockItem	HistoricalSkipLeadingZeros

Table 57.30: New Input table fields

Table name	Field name
SafetyStockItem	ReportIterationResults
ScheduledReceipt	ExpiryStartDate

Modified fields

The following table lists fields modified in Input tables in Version 2016.2 Service Update 6

Table 57.31: Modified Input fields

Table name	Field name
ScheduledReceipt	The calculated ExpiryStartDate field has been renamed to CalcExpiryStartDate

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 6.

New fields

The following table lists fields added to Control tables in Version 2016.2 Service Update 6.

Table 57.32: New Control table fields

Table name	Field name
AnalyticsConfiguration	MEIOSimulationSteps
AnalyticsConfiguration	MEIOSimulationTrials

Modified fields

The following table lists fields modified in Control tables in Version 2016.2 Service Update 6.

Table 57.33: Modified Control table fields

Table name	Field name
SourceRule	AllotmentRule Two new options, OngoingToDate and ProportionaLotSize are added.

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2 Service Update 6.

New tables

The following tables were added:

- **MEIOFamilyIterationResults**—Reports solution costs from each analytic iteration per MEIO family.
- **MEIOStageIterationResult**—Reports incoming/outgoing service times from each analytic iteration per MEIO stage.
- **MEIOStageLinkIterationResult**—Links MEIO stage iteration results together.
- **MEIOStageSimulation**—Reports simulated impact of MEIO recommended safety stock settings on service and inventory.

Modified fields

The following table lists fields modified in Calculated tables in Version 2016.2.

Table name	Field name
MEIOFamilyResult	RunTime. Data type was changed from Integer to Quantity.

Version 2016.2 changes Service Update 5

This section provides a summary of changes made from RapidResponse Service Update 4 to RapidResponse Service Update 5 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service Update 5 including:

- Days of supply and period planning by dock date
- Identify reasons for unsatisfied demand from multi-level search logic
- Reduce pre-planning in multi-level search logic
- Specify intervals to expiry values specific to CTP
- Import externally calculated statistical forecast
- "Enhanced safety stock item outlier detection and correction " on page 2382
- Exclude calendars from bucketing selections
- Defect fixes and calculation changes

Days of supply and period planning by dock date

When a part uses days of supply logic with any of the "ByPeriod" rules, groups of demands are collected over a specified number of intervals and satisfied by a single planned order. Previously, the **PeriodSupplyIntervalCalendar** would determine the calendar interval over which demands are collected based on due date, and the **PeriodSupplyDueCalendar** would determine the calendar on whose intervals the planned order due dates are set.

In this version, new configuration options allow for the specified period calendars to be aligned to dock dates instead of due dates. This can be useful in cases where parts have different dock-to-stock lead times and there is a need to align and co-ordinate the arrival of supply at the destination site and/or shipment of supply from the sourcing site. This is configured using new two control settings on the **DaysSupplyPolicy** table (that is, **PeriodSupplyDateRule** and **LongTermPeriodSupplyDateRule**).

For more information, see "Dates used in period-based days of supply" on page 1369.

Identify reasons for unsatisfied demand from multi-level search logic

When an end-item is configured for multi-level search logic (**MultiLevelSearchRule .ProcessingRule** is "Use") and a demand for that part is configured for partial planning (**DemandType.PlanningRule** is "Partial"), the demand might be left wholly or partially unsatisfied if multi-level search logic determines the demand will be late and outside of a specified tolerance for lateness.

In this version, details of those unsatisfied demands, and the supplies that caused multi-level search logic to give up on satisfying them, are reported in a series of new calculated tables (for example, the **MLSActivity** table is added). These tables can be useful in helping to assess the reasons why a given demand was left unsatisfied so that appropriate action can be taken. Use of these new tables for logging unsatisfied demand details is enabled through the new **MultiLevelSearchRule.LoggingRule** field.

Reduce pre-planning in multi-level search logic

A new option on the **MultiLevelSearchRule** table, **ShortTermExcessRule**, is available and can be used to reduce the amount of short-term excess supply that can result when using multi-level search logic. This is accomplished by delaying or moving supplies earlier in order to reduce the amount of supply pre-planning that can be caused when multi-level search logic is used with priority planning, dynamic safety stock, lot-sizing and/or constraint scheduling logic.

The **MultiLevelSearchRule** table also has a new **FutureAvailabilityRule** which can be used to prevent supplies from being planned when multi-level search logic determines a "Future" availability, and thus prevents dependent requirements from being driven down the network (potentially consuming material and constraint).

Specify intervals to expiry values specific to CTP

When using material expiration logic, the **IntervalsToExpiry** value on each **PartSource** is used to specify the length of time that supply from the part source should last before it expires. In previous releases, the **IntervalsToExpiry** value could be used in both Netting calculations (to determine an **EstimatedExpiryDate**) and in CTP calculations (to determine the actual **ExpiryDate**). However, based on configuration settings, these two versions of the expiry date were calculated differently (from different starting points) even though both relied on the same **IntervalsToExpiry** value.

Therefore the estimated expiry date produced by Netting would not always be a good estimation of the actual expiry date calculated later by CTP. To address this situation, a new **CTPIntervalsToExpiry** field has been added to the **PartSource** table. This field, optionally, allows providing an intervals to expiry value that is specific to CTP when calculating the **ExpiryDate** while the existing **IntervalsToExpiry** value is still used by Netting when calculating the **EstimatedExpiryDate**.

Note that if the new **CTPIntervalsToExpiry** field is not populated, then the **IntervalsToExpiry** field continues to be used in both Netting and CTP calculations.

For more information, see "Defining expiry intervals on part sources" on page 1604.

Import externally calculated statistical forecast

It is now possible to import statistical forecasts calculated using external tools (instead of having the forecast calculated by RapidResponse). Once imported against a given forecast stream, these statistical forecasts can then make use of other RapidResponse forecasting calculations such as disaggregation and NPI/EOL logic.

Statistical forecast values can be provided in the new **ImportedStatisticalForecast** table which identifies the relevant **ForecastItemParameters** record along with a forecast date and quantity to import. To make use of values in this input table, the **ForecasttemParametersType.ForecastModel** field should be set to the new option of "ForecastInput".

For more information, see "ForecastImport" on page 2058.

Enhanced safety stock item outlier detection and correction

Enhancements have been made to the processing of outliers in the historical data used to calculate an item's safety stock recommendations. Previously, outlier detection and correction was applicable to historical actual and forecast demands in the context of safety stock items. This was largely controlled through two settings on the **SafetyStockItemType** table; **DemandOutlierProcessingRule** and **ForecastErrorOutlierProcessingRule** (these fields are now hidden and not recommended, though existing resources that use them will continue to function).

In this release, outlier detection and correction is applicable to historical actual and forecast demands, as well as to historical lead times on supplies in the context of both single-echelon and multi-echelon stages. These options are configured through several new input fields on the **SafetyStockItem** and **MEIOLeadTime** tables, each of which now includes a reference to the existing **OutlierType** table.

The **SafetyStockItemOutlier** and **MEIOLeadTimeOutlier** input tables have been added to report outlier adjustments for safety stock items. The **SafetyStockItemOutlier** table references the **SafetyStockItem** table and reports outlier adjustments in historical actual and historical forecast demand data for both single and multi-echelon safety stock items, as well as in historical lead time data for single-echelon safety stock items only. The **MEIOLeadTimeOutlier** table references the **MEIOLeadTime** table and reports outlier adjustments in historical lead time data for multi-echelon safety stock items only.

Additionally, four new calculated tables have been added to report summary and detailed outlier information generated as part of the inventory planning and optimization process: **SafetyStockItemOutlierResult**, **SafetyStockItemOutlierResultSummary**, **MEIOLeadTimeOutlierResult**, and **MEIOLeadTimeOutlierResultSummary**.

For more information about enhancements related to the detection and correction of outliers for safety stock items, see "Safety stock item outlier processing" on page 1983.

Exclude calendars from bucketing selections

Typically, calendar values defined in the **Calendar** table can be used for planning and/or bucketing purpose. For example, a Month calendar might be used as an organization's forecast calendar and also selected by user's when bucketing data for presentation in a given worksheet. However, in some cases an organization might have a large number of calendars which are intended for specific or short-term planning purposes only and are not likely to be used for worksheet bucketing.

In this version, a new **Calendar.PlanningOnly** field can be used to designate a calendar as being for planning purposes only. This will exclude the calendar from showing up in dialog boxes that users use to select calendars for worksheet bucketing (and thus ensures a more manageable and relevant list of calendar selections in those dialogs).

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 5.

Table 57.34: Defect fixes and calculation changes in Version 2016.2, Service Update 5

Fix Area / Change	Description
Issues related to use of attribute-based planning logic	The following issues related to attribute-based planning logic were resolved: • Attributes are now checked when pruning unsatisfied demands. This addresses a problem where certain demands were incorrectly reported as being satisfied on time. • Fixed an issue where Netting would sometimes end up in an endless loop when attribute-based planning was used along with safety stock planning (and expiry). • Fixed an issue related to order merging when attribute-based planning is used along with safety stock, expiry, and days of supply logic. Now, planned orders created without an attribute are bucketed properly and potentially able to merge with other planned orders created without an attribute. • Fixed an issue where expiry calculations reported in the CTPActivity and CTPPlannedOrder tables could give incorrect results in cases where attribute-based planning was turned on for the part.
Safety lead time and partial planning logic	Safety lead time is now applied and considered in the process of determining whether demands should be left unsatisfied when partial planning logic is enabled.
Multi-level search and CommitLevel of High	When using multi-level search, resolved an issue where demands could be incorrectly left unsatisfied when there was limited constraint and the part's CommitLevel was set to High.
Issues with flow-to-common logic	The following issues related to flow-to-common logic were resolved • Fixed an issue where exceptions might be generated when flow-to-common pool logic was used along with partial planning logic. • Fixed an issues where planned orders were only ever generated in the common pool when using flow-to-common pool logic (specifically the NetCommon setting in the MUEPoolNettingType.PoolFlowToCommon field) along with partial planning logic. In these cases, it is not possible for planned orders to be generated outside the common pool where appropriate.
Dynamic safety stock generation with multi-level search logic	Fixed an issue where if multi-level search logic is used along with dynamic safety methods, planned orders were not always generated for safety stock in cases where that dynamic safety stock level was lower than the total demand quantity.
NaN results in MEIO with invalid calendars	Resolved an issue in time-phased multi-ecehlon inventory logic where an invalid calendar could lead to a NaN (not a number) result.
Rounding of service levels less than .5	Resolved an issue with unstable results from time-phased multi-echelon inventory optimization calculations when the service level provided was less than 50% (to fix this, service levels less than 0.5 are now rounded up to 0.500001).
Negative ending balance when certain analytics were used with CommitLevel of "High"	Resolved an issue where if a part used a CommitLevel of "High", along with global substitution, co-product configuration, and had demands of different priorities, than a negative ending balance would sometimes be reported.
NeedQty calculation in WhereConsumed tables (inventory transfers)	Fixed an issue in the WhereConsumed tables populated by FLP logic where the NeedQty was sometimes incorrectly calculated when inventory transfers were involved.

Table 57.34: Defect fixes and calculation changes in Version 2016.2, Service Update 5

Fix Area / Change	Description
AvailableDate potentially earlier than CalcStartDate on scheduled receipts	Fixed an issue where a scheduled receipt's AvailableDate could sometimes be reported as earlier than its CalcStartDate (in certain cases where constraints use Workday calendars and CalcStartDate falls on a weekend and needs to be rounded).
Safety lead time use in calculating scheduled receipt dates	Fixed an issue where a scheduled receipt's AvailableDate could sometimes be reported as earlier than its CalcStartDate (in certain cases where constraints use Workday calendars and CalcStartDate falls on a weekend and needs to be rounded).
Issues with order trimming and merging when multi-level search logic enabled	Resolved an issue with multi-level search where excess trimming and order merging logic could sometimes result in slightly less supply being planned on components than there was demand for.
Global part substitution and co-product compatibility	The following issues were fixed that previously could arise when using global part substitution logic along with co-products: • Fixed an issue where excess or unnecessary planned orders were being generated on substitute parts when global part substitution logic is used with co-products. • Fixed an issue where part substitution was not occurring when global part substitution was used with co-products and multi-level search logic was enabled for those parts.
With partial planning logic, demands were being satisfied outside of tolerance in some cases.	When partial planning logic was enabled on a demand, and multi-level search logic and constrained planning were both used, constraint scheduling logic could sometimes lead to demands being planned outside of lateness tolerance. This combination of settings should no longer allow demands to be planned outside of tolerance (they should be left unsatisfied instead).

Input table changes

This section outlines new and modified Input tables in Version 2016.2 Service Update 5.

New tables

The following tables are added:

- **HistoricalShipment**—This table stores historical shipments, and is used for reporting historical shipping data to support the Distribution Requirements Planning application. For more information, see "HistoricalShipment table" on page 346.
- **HistoricalSupplyOrder**—This table stores historical supply orders, and is used for reporting historical supply to support the Distribution Requirements Planning application. For more information, see "HistoricalSupplyOrder table" on page 337.
- **ImportedStatisticalForecast**—This table allows for a statistical forecast generated by an external tool or application to be imported into RapidResponse (rather than calculated). The imported forecast can then be used in subsequent forecasting calculations (for example, NPI/EOL and disaggregation).

- **MEIOLeadTimeOutlier**—This table holds outlier adjustments for historical lead times associated with an MEIOStage.
- **SafetyLeadTimeProfile**—This table holds named profiles that can be used to enable time-phased safety-lead time values for a part. If a Part has a Null reference to this table, then a single safety lead time value as defined on the Part record is used. For more information, see "SafetyLeadTimeProfile table" on page 538.
- **SafetyLeadTimeProfileDetail**- This table holds details associated with records in the SafetyLeadTimeProfile table (for example, the date on which a safety lead time value becomes effective for a part). For more information, see "SafetyLeadTimeProfileDetail table" on page 539.
- **SafetyStockItemOutlier**—The table is used to store outlier adjustments for historical demand or lead times associated with a SafetyStockItem.
- **SafetyStockTimePhasedBounds**—This table contains minimum and maximum safety stock bounds for parts, and the dates on which the bounds rules become effective. For more information, see "SafetyStockTimePhasedBounds table" on page 565.
- **StorageType**—This table is used to define storage types for storage locations to support the Distribution Requirements Planning application. For more information, see "StorageType table " on page 610.
- **TimePhasedLocationCapacity**—This table is used to define the time-phased storage capacity of locations in a distribution network to support the Distribution Requirements Planning application. For more information, see "TimePhasedLocationCapacity table" on page 653.
- **TransferDestination**—This table is used to model and store routes for transferring inventory to support the Distribution Requirements Planning application. For more information, see "TransferDestination table" on page 662.

New fields

This section outlines new and modified Input tables in Version 2016.2 Service Update 5.

Table 57.35: New Input table fields

Table name	Field name
ForecastItemParameters	OutlierSmoothingAfterIntervalCount
ForecastItemParameters	OutlierSmoothingBeforeIntervalCount
HistoricalSupplyActual	ActualDockDate
HistoricalSupplyActual	ActualShipDate
HistoricalSupplyActual	HandlingCost
HistoricalSupplyActual	ShippingCost
HistoricalSupplyActual	ReceiptQuantity
HistoricalSupplyActual	Order
HistoricalSupplyActual	Shipment
InventoryTransfer	ActualDockDate
InventoryTransfer	Source
Location	StorageType
MEIOLeadTime	LeadTimeOutlierMovingAverageWindow

Table 57.35: New Input table fields

Table name	Field name
MEIOLeadTime	LeadTimeOutlierThreshold
MEIOLeadTime	LeadTimeOutlierType
MEIOLeadTime	LeadTimeSmoothingAfterIntervalCount
MEIOLeadTime	LeadTimeSmoothingBeforeIntervalCount
PartSolution	EffectiveDRPFrozenWindow
PartSolution	EffectiveDRPHorizon
PartSolution	EffectiveDRPPlanningHorizon
PartSolution	DistributionPlannerCode
PartSolution	DRPFrozenWindow
PartSolution	DRPHorizon
PartSolution	DRPPlanningHorizon
PartSource	CTPIntervalsToExpiry
PartSource	ShippingCost
SafetyStockItem	CalcAverageLeadTime
SafetyStockItem	DemandOutlierMovingAverageWindow
SafetyStockItem	DemandOutlierThreshold
SafetyStockItem	DemandOutlierType
SafetyStockItem	LeadTimeOutlierMovingAverageWindow
SafetyStockItem	LeadTimeOutlierThreshold
SafetyStockItem	LeadTimeOutlierType
SafetyStockItem	LeadTimeSmoothingAfterIntervalCount
SafetyStockItem	LeadTimeSmoothingBeforeIntervalCount
SafetyStockItem	RemoveDemandOutliers
ScheduledReceipt	ActualDockDate
ScheduledReceipt	ActualShipDate
Source	Carrier
Source	TransportationMode

Obsolete fields

The **ForecastErrorOutlierThreshold** field on the **SafetyStockItem** table is declared obsolete and hidden (new tables and fields have been added to control all historical outlier detection for multi-echelon calculations). Note that any resources based on this field will continue to function.

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 5.

New fields

The following table lists fields added to Control tables in Version 2016.2 Service Update 5.

Table 57.36: New Control table fields

Table name	Field name
MultiLevelSearchRule	FutureAvailability
MutiLevelSearchRule	LoggingRule
MultiLevelSearchRule	ShortTermExcess

Modified fields

The following table lists fields modified in Control tables in Version 2016.2

Table 57.37: Modified Control table fields

Table name	Field name
ForecastItemParametersType	ForecastModel. A new option, ForecastImport, is added.
OrderPriority	TwoDatePlanningRule. Two new options, UseWithHardSafetyLead Time and UseWithSoftSafetyLeadTime, are added.
OutlierType	AboveThresholdRule. Three new options (SmoothKeepExcess, SmoothRemoveExcess, and Ignore are added).
OutlierType	BelowThresholdRule. Three new options (SmoothKeepExcess, SmoothRemoveExcess, and Ignore are added).

Obsolete fields

The **DemandOutlierProcessingRule** & **ForecastErrorOutlierProcessingRule** fields on the **SafetyStockItemType** table have been declared obsolete and hidden (new tables and fields have been added to control historical outlier detection for multi-echelon calculations). Note that any resources based on these fields will continue to function.

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2 Service Update 5.

New tables

The following tables were added:

- **MEIOLeadTimeOutlierResult**—Reports details related to outliers in historical lead time for MEIO stages used in inventory planning and optimization calculations.
- **MEIOLeadTimeOutlierResultSummary—**Reports summary information related to outlier detection in historical lead times for MEIO stages.
- **MLSActivity**—Reports details of end-item demands that were left unsatisfied because multi-level search determined they could not be planned within a specified lateness tolerance, and also identifies the supplies (at all levels) that couldn't be planned in time to satisfy those demands. For more information, see "MLSActivity table" on page 1131.
- **SafetyStockItemOutlierResult**—Reports details related to outliers in historical demands or lead time for safety stock items used in inventory planning and optimization calculations.
- **SafetyStockItemOutlierResultSummary—**Reports summary information related to outlier detection in historical demand or lead times for safety stock items.
- **TrialCoByProductSupply**—Reports details of co-product or by-product supplies created under a scheduled receipt or trial planned order that couldn't be planned in time to satisfy the driver demands they were matched to by multi-level search logic. For more information, see "TrialCoByProductSupply table " on page 1216.
- **TrialConstraintShortage**—Reports details of constraint shortages that prevent supplies from being planned in time to allow multi-level search logic to satisfy driver demands within a specified lateness tolerance. For more information, see "TrialConstraintShortage table" on page 1218.
- **TrialConsumption**—Reports details of dates with constraint consumption on scheduled receipts and trial planned orders, but where not enough constraint was available when needed to allow multi-level search to satisfy end-item demands within a specified lateness tolerance. For more information, see "TrialConsumption table" on page 1219.
- **TrialPlannedOrder**—Reports details of trial planned orders created by multi-level search logic, but where the orders could not be planned in time to allow their end-item driver demands to ultimately be satisfied within a specified lateness tolerance. For more information, see "TrialPlannedOrder table" on page 1220.
- **TrialSubstituteSupply**—Reports details of trial substitute supply created by multi-level search logic, under a primary part, but where the supply could not be planned in time to allow the primary parts driver demands to be satisfied within a specified lateness tolerance. For more information, see "TrialSubstituteSupply table" on page 1222.

New fields

The following table lists fields added to Calculated tables in Version 2016.2 Service Update 5.

Table 57.38: New calculated table fields

Table name	Field name
MEIOHistoricalSupply	IsLeadTimeOutlier
MEIOHistoricalSupply	OriginalLeadTime
SafetyStockHistoricalSupply	IsLeadTimeOutlier
SafetyStockHistoricalSupply	OriginalLeadTime

CHAPTER 58: Analytic and data model changes in 2016

Many new and changed features were included in the initial release of RapidResponse Version 2016.2. Additional features have been added in service updates.

Details on the following are available:

- Version 2016.2 changes Service Update 4
- Version 2016.2 changes Service Update 3
- Version 2016.2 changes — Initial release

Note: Features that were added in versions of RapidResponse prior to 2016.2 are not included in this list of changes.

Version 2016.2 changes Service Update 4

This section provides a summary of changes made from RapidResponse 2016.2 Service Update 3 to RapidResponse 2016.2 Service Update 4 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service Update 4:

- Full-level pegging reporting enhancements
- Controlling source effectivity in multi-level search
- Defect fixes and calculation changes

Full-level pegging reporting enhancements

Several of the **WhereConsumed** tables have had fields added reporting the amount of the driver supply satisfied by a pegged supply.

Controlling source effectivity in multi-level search

A new option allows for source selection using multi-level search logic to only consider part sources where the rolled up availability of components is within the part source's effectivity (as opposed to determining effectivity based on order due date alone). This is configured using the new **SourceEffectivity** setting on the **MultiLevelSearchRule** table. Note that use of this new optional logic can result in demands being unsatisfied which otherwise wouldn't be (such demands are logged in the **DemandException** table with a **Type** of "MLSSourceEffectivity").

For more information, see "Multi-level part source effectivity" on page 1865.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse Service Update 4 that might lead to slight differences being seen in calculated output.

Table 58.1: Defect fixes and calculation changes in Version 2016.2, Service Update 4

Fix Area / Change	Description
Constraint consumption on canceled scheduled receipts	If a scheduled receipt is canceled, it will no longer consume constraint in Netting calculations (previously a scheduled receipt would not consume constraint in Capable-to-Promise calculations, but would consume constraint in Netting).
"Partial" planning rule now works on part's using multi-level search with late scheduled receipts outside of tolerance	If a demand uses a DemandType.PlanningRule of "Partial" and the part has a direct scheduled receipt on it that is late, available outside of unsatisfied demand tolerance, and beyond the planning time fence, the demand will now be ignored. Previously, such demands for parts that have multi-level search enabled would be planned late (treated as if the PlanningRule were set to "Full".)
Available date correction on co/by-products and split supplies	Previously, in some cases when co/by-product logic was used and supplies were split to reflect incremental availability, a driver part's available date could be reported earlier than a component supply's available date. This issue has been fixed.
Supplier restriction on MEIO lead-time variability calculation removed	If calculating lead time variability for multi-echelon inventory optimization, a non-Null reference is required in the HistoricalSupplyActual.Source field (to ensure the correct source is chosen). Previously, HistoricalSupplyActual records were ignored for lead-time variability if the Source.Supplier did not match the Header.PartSupplier.Supplier. This restriction has been removed (and for some data configurations will result in more records being generated in the MEIOHistoricalSupply table than were previously.
Pool-based netting with flow-to-common and days of supply logic	Previously, when days of supply logic was enabled for a part that also used pool-based netting and the flow-to-common option, a part could end up with a planned order in both the common pool and a custom pool in the same days of supply window (whereas they should have been merged into one planned order). This issue has been fixed and might result in fewer planned orders for parts that use days of supply alongside pool-based netting with flow-to-common logic.
Demand date on co/by-product supplies no longer includes safety lead time	Previously, the calculated CoByProductSupply.DemandDate field erroneously included consideration for any applicable safety lead time values for the part. This issue has been fixed and might result in later DemandDate values that were seen previously on co/by-product supplies for parts that use safety lead time.
ForecastEndOffset now used in statistical forecast disaggregation	Previously, the SOPAnalyticsConfiguration.ForecastEndOffset value was ignored when determining forecast disaggregration rates. This issue has been fixed and, in cases where a positive ForecastEndOffset value is specified, can result in fewer StatisticalForecastDetail records than would have been generated previously.
ScheduledReceipt and PegPart now populated for all push transfer demands	Previously, if a CTPActivity record represented a quantity of demand to be pushed or transferred to a destination site (had Type of "PushTransferOut") and the dependent demand was a new transfer allocation, then the ScheduledReceipt and PegPart references on the "PushTransferOut" record were not populated. This issue has been fixed and the aforementioned references should now be populated correctly for push transfer records.
Slight changes in allotment override processing	Corrections have been made in how allotment overrides are handled in capable-to-promise logic. When pegging supply to a demand with an allotment override, logic will now look backwards from AllotmentOverride.Date for available supply either not tagged to an allotment override or tagged to the same allotment override as the demand (previously, only supply not tagged to an allotment override was considered). If no supply is found, the logic then continues from the first non-zero supply at/before AllotmentOverride.Date (previously, a supply of 0 might have been considered by the logic). In some cases, this change will result in demands being pegged to earlier supplies than they would have when allotment override logic are used.

Table 58.1: Defect fixes and calculation changes in Version 2016.2, Service Update 4

Fix Area / Change	Description
Certain EffectiveUnitPrice fields will no longer return negative values	The EffectiveUnitPrice field calculated on various Sales & Operations Planning tables, such as ConsensusForecast and ForecastDetail, is no longer allowed to return a negative value. Previously, a negative value could be returned if both the CustomerPrice.UnitPirce and Part.AverageSellingPrice input fields were negative (in these cases now, a value of 0 will be returned).
Forecast.EffectiveUnitPrice now respects override from ForecastDetail.UnitPrice (for consensus forecast)	Previously, if a UnitPrice override value was specified on the ForecastDetail table, the corresponding records in the Forecast table would not reflect that override in its EffectiveUnitPrice field (for example, that price would be derived from the values on the Part or CustomerPrice tables). This issue has been fixed.

Input table changes

This section outlines new and modified Input tables in Version 2016.2, Service Update 4.

Modified fields

The following table lists fields modified in Input tables in Version 2016.2, Service Update 4.

Table 58.2: Modified Input fields

Table name	Field name
SafetyStockAverageDemandProfile	Multiplier. This field's data type was changed from Integer to Quantity.

Control table changes

This section outlines new and modified Control tables in Version 2016.2, Service Update 4.

New fields

The following table lists fields added to Control tables in Version 2016.2 Service Update 4.

Table 58.3: New Control table fields

Table name	Field name
MultiLevelSearchRule	SourceEffectivityRule

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2, Service Update 4.

New tables

The following tables were added:

- **MEIOStageDailyDemandParameterSet**—Reports daily historical average and standard deviation of demand, as well as future average demand calculated for customer-facing stages in time-phased MEIO calculations.

New fields

The following table lists fields added to Calculated tables in Version 2016.2, Service Update 4.

Table 58.4: New calculated table fields

Table name	Field name
DemandException	Type
LateSupply	DriverNeedQuantity
WhereConsumed	DriverNeedQuantity
WhereConsumedForDemand	DriverNeedQuantity
WhereConsumedForForecast	DriverNeedQuantity
WhereConsumedForSupply	DriverNeedQuantity

Hidden fields

The **IndependentDemandByFeature.Feature** field was hidden. Note that existing resources that use this field will continue to function now, however this field will be removed completely in a future release.

Version 2016.2 changes Service Update 3

This section provides a summary of changes made from RapidResponse 2016.2 Initial Release to RapidResponse 2016.2 Service Update 3 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 Service Update 3 including:

- Events and forecast/price adjustments
- Priority-based forecast consumption
- Controlling priority on planned orders satisfying multiple demands
- Time-phased constraint factors
- Full-level pegging reporting enhancements
- Defect fixes and calculation changes

Events and forecast/price adjustments

Events can now be modeled in RapidResponse and used to adjust forecast quantities and unit prices within the demand planning process. For example, a sales promotion can be modeled as an event and used to adjust calculated forecast quantities within a specified time horizon. Events are defined in the new **Event** and **EventPhase** tables and can then be applied to the forecast associated with one or more **HistoricalDemandHeader** records using the new **EventPhaseHeader** table. A series of new calculated tables are also available to report details of event-adjusted forecasts.

For more information, see "Event Management" on page 2109.

Priority-based forecast consumption

Forecast consumption can now be configured so that higher priority actual demands consume forecast before lower priority demands. Previously, sales actuals would consume forecast in due date order regardless of priority. However, this new optional logic allows for order priority to take precedence over order due date in determining which actuals consume forecast first. Priority-based forecast consumption is implemented through the new **SalesActualConsumptionOrder** setting on the **PartType** table.

Controlling priority on planned orders satisfying multiple demands

A new configuration option enables control over how priority is set on a planned order satisfying multiple demands (as might occur when lot-sizing policies are used). Previously, assuming a **CommitLevel** of "High" or "MediumExplodPriority" for the part, each planned order would inherit the

priority of the triggering demand. Using the existing **PrioritySource** setting on the **SupplyStatus** table, a planned order can be configured to optionally take the highest priority of all demands it satisfies or an average of the priority from the demands it satisfies.

Time-phased constraint factors

Constraint factors can now be modeled to vary over time. For example, a manufacturing process might improve over time or a machine might degrade over time resulting in a need to define different constraint factors. To enable time-phasing of constraint factors, a new key field, **EffectiveInDate**, is added to the **SourceConstraint** table. This allows the possibility of multiple **SourceContraint** records for a given **Constraint** and **PartSource** combination, with different effective dates and constraint factors for each.

Full-level pegging reporting enhancements

Several of the **WhereConsumed** tables have had reference fields added that allow reporting of the incremental **SupplyDemand** details for the immediate parent order.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 Service Update 3 that, in some cases, might lead to slight differences being seen in calculated output. .

Table 58.5: Defect fixes and calculation changes in Version 2016.2, Service Update 3

Fix Area / Change	Description
When used with multi-level search logic, safety stock drops followed by increases are now handled properly	Previously, for parts using multi-level search logic and having a CommitLevel of "High", if a safety stock drop was followed by a safety stock increase, the amount of the original drop was not considered in bringing the safety stock level up to the new amount (this could result in higher than needed levels of inventory). This issue has been fixed.

Input table changes

This section outlines new and modified Input tables in Version 2016.2, Service Update 3.

New tables

The following tables are added:

- **CurveParameters**—Holds parameters that define curves thatc an be used by event phases when adjustments beyond simple quantity or percentage changes are required against forecast or unit prices.
- **Event**—Used to logically group event phases that can are used to apply adjustments to forecast quantities or unit prices.
- **EventType**—Used to categorize events by type. For example, a trade promotion or consumer promotion.

- **EventPhase**—Holds single items that can be used to adjust (increase or decrease) forecast quantities or unit prices. Can be grouped together as events, and applied to forecast through event phase headers.
- **EventPhaseHeader**—Used to associate event phases with historical demand headers, so that the event phase can then be used to adjust the forecast under the historical header.

New fields

This section outlines new and modified Input tables in Version 2016 Service Update 3.

Table 58.6: New Input table fields

Table name	Field name
SourceConstraint	EffectiveInDate

Control table changes

This section outlines new and modified Control tables in Version 2016.2 Service Update 3.

New fields

The following table lists fields added to Control tables in Version 2016.2, Service Update 3.

Table 58.7: New Control table fields

Table name	Field name
HistoricalDemandCategoryType	EventAllowNegative
PartType	SalesActualConsumptionOrder

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2, Service Update 3.

New tables

The following tables were added:

- **DirectAssembly**—Identifies direct assemblies of a part, and thus identifies all immediate parent parts that create dependent demand on a given component.
  For more information, see "DirectAssembly table" on page 1036.
- **EventConsensusForecastDetail**—Reports details of EventForecastDetail
  Adjustment records that are used in generating the consensus forecast value for part and customer combinations. For more information, see "EventConsensusForecastDetail table" on page 1039.

- **EventDisaggregationRate**—Reports time-phased, disaggregated forecast adjustment rates for use in calculating forecast adjustments. For more information, see "EventDisaggregationRate table" on page 1041.
- **EventForecastDetailAdjustment**—Reports event-based adjustments made to forecast quantity values from the **ForecastDetail** table. For more information, see "EventForecastDetailAdjustment table" on page 1041.
- **EventStatisticalDisaggregationRate**—Reports time-phased, disaggregated forecast adjustment rates for use in calculating statistical forecast adjustments. For more information, see "EventStatisticalDisaggregationRate table" on page 1043.
- **EventStatisticalForecastDetail**—Contains statistical forecast details from the **ForecastDetail** table against which adjustments can be applied. For more information, see "EventStatisticalForecastDetail table" on page 1044.
- **EventStatisticalForecastDetailAdjustment**—Reports event-based adjustments made to statistical forecast quantity values from the **ForecastDetail** table. For more information, see "EventStatisticalForecastDetailAdjustment table" on page 1044.
- **FlatAssembly**—Identifies direct as well as indirect assemblies of a part, and thus identifies all parts/levels that create dependent demand on a given component.
  For more information, see "FlatAssembly table" on page 1046.

New fields

The following table lists fields added to Calculated tables in Version 2016.2. Service Update 3.

Table 58.8: New calculated table fields

Table name	Field name
WhereConsumed	PegSupplyDemand
WhereConsumedForDemand	PegSupplyDemand
WhereConsumedForForecast	ParentSupplyDemand
WhereConsumedForSupply	ParentSupplyDemand

Version 2016.2 changes – Initial release

This section provides a summary of changes made from RapidResponse 2015.3 to 2016.2 that affect the data model and RapidResponse calculations. In this section, you'll learn about the following changes:

- Analytic and calculation changes
- Input table changes
- Control table changes
- Calculated table changes

Analytic and calculation changes

This section describes major changes made to RapidResponse analytics and calculations in Version 2016.2 including:

- Safety stock bounds
- Distribution planning
- Forward-bucketed fair share
- Time-phased safety stock enhancements
- Time-phased safety lead time
- Time-phased days of supply
- Days of supply and end of period orders
- Transit time calendar
- Adjusted due date from calculated start date
- Safety stock with expiring supply enhancement
- Capacity and load only constraints
- Filtering FlatBillDown and FlatBillUp calculations by model
- MultiLevelSearch now respects PlanningRule of Partial
- Unsatisfied demand tolerance for PlanningRule of Partial
- Lot sizing in attribute-based planning
- Forecast Error Pooling
- Support for order BOMs
- Defect fixes and calculation changes

Safety stock bounds

You can now place upper and lower bounds on safety stock levels for individual parts, which are taken into account when performing calculations for single-echelon and multi-echelon inventory optimization. It might be useful to place bounds on safety stock if, for example, a site has a limited capacity to store parts or your company has a contractual obligation to maintain a minimum safety stock level. RapidResponse tries to find a safety stock solution that satisfies the service level you have specified without violating any bounds that have been placed on safety stock.

For more information, see "Safety stock bounds" on page 2398.

Distribution planning

RapidResponse now includes analytics to support the automatic distribution, or pushing, of pre-built supply from a sourcing site to destination sites. The pre-built supply that will be pushed is determined by the maximum inventory level that you set for a part. When the inventory for a part exceeds the set maximum inventory level, the pre-built supply will be automatically pushed out. For example, if you are pre-building to meet future demand and your manufacturing facility cannot store the finished goods, you can now automatically push them into your distribution network once the maximum inventory level is exceeded.

For more information, see "Distribution planning" on page 1717.

Forward-bucketed fair share

RapidResponse now includes additional forward-bucketed fair share capabilities, which are used by the capable-to-promise (CTP) analytic. This fair share enhancement now provides a method for satisfying stockouts at distribution centers that are caused by customer orders or forecast as fairly and efficiently as possible. Once the stockouts are resolved, the analytic also ensures the replenishment of distribution centers to their safety stock levels, in a fair and efficient manner. The fair share distribution is set on a part-by-part basis. In addition, it moves from earliest to latest buckets and also shares on-time supply, instead of only late supply.

The new fair share distribution capabilities are supported by the introduction of the "AllotmentRule" field that was added to the new **DistributionPlanningRule** control table. If distribution planning is enabled, fair share ensures that the pre-built supply gets pushed to distribution sites in a fair and equitable manner.

For more information, see "Distribution planning" on page 1717.

Time-phased safety stock enhancements

Additional methods for calculating time-phased safety stock quantities have been added.

Previously, time-phased safety stock quantities could be specified as a series of date-effective **Quantity** values in the **TimePhasedSafety** table. Safety stock for a given effective date can now be set to either the maximum of that time-phased quantity and some number of forward-looking periods of demand, or as the sum of that time-phased quantity and some number of forward-looking periods of demand.

These capabilities are supported through the use of two new control settings, "TimePhasedSafetyDaysMaximum**"** and "TimePhasedSafetyDaysTotal", which have been added to the **SafetyStockQuantityRule** field in the **SafetyStockPolicy** table (note that these settings have not been added to the equivalent field on the **PartType** table). When using either of these settings, the **Days** field in the **TimePhasedSafety** table is used to specify the number of forward-looking periods of demand to use (with the calendar referenced by **SafetyStockPolicy.SafetyDaysCalendar**>determining the calendar interval in which the **Days** value is expressed).

For more information, see "Time-phased safety days" on page 1407.

Time-phased safety lead time

Safety lead time values, used when setting planned orders due dates, can now be time-phased for a part. For example, one safety lead time value might be applied when satisfying demands as of a specified date early in the planning horizon and lesser safety lead time values might be applied against demands as of different effective dates later in the horizon. In previous versions, only a single stationary safety lead time value was allowed per part (as set by the value in **Part.SafetyLeadTime**).

This capability is supported through the use of safety-lead time profiles defined in the new **SafetyLeadTimeProfile** and **SafetyLeadTimeDetail** tables. If a part does not make use of these tables (has a Null reference in **Part.SafetyLeadTimeProfile**), then the safety lead time value defined in the **Part** table is used as was always the case in previous versions.

For more information, see "Lead time elements" on page 1308 and "SafetyLeadTimeProfile table" on page 538.

Time-phased days of supply

When using days of supply planning to group demands over a number of periods and then satisfy those grouped demands with a planned order, the days of supply periods can now be time-phased. For example, early in the planning horizon each planned order for a part might be created to satisfy some number of weeks of demand, while later in the planning horizon each planned order for the part might be created to satisfy some number of months of demand.

This capability is supported by the new **DaysSupplyPolicy** table which allows defining days of supply settings in two configurable time-phased horizons (representing short-term and long-term requirements). Note that if a part does not make use of this table (the **Part.DaysSupplyPolicy** reference is Null), then a single days of supply setting is assumed which relies on values in the **Part**, **PartType**, and **PlanningCalendars** tables as was always the case in previous versions.

For more information, see "Days of Supply calculations" on page 1362 and "DaysSupplyPolicy table" on page 705.

Days of supply and end of period orders

When using days of supply planning to group demands over a number of periods and then satisfy those grouped demands with a planned order, it is now possible for the resulting planned orders to be generated towards the end of the days of supply period rather than the start.

This capability is supported by the introduction of two new settings, “ByPeriodEnd” and “ByPeriodEndWithPriority”, that have been added to the **DaysSupplyRule** field on the **PartType** table. These new settings are conceptually similar to the existing “ByPeriod” and “ByPeriodWithPriority” settings, except that they allow planned orders due dates to be set towards the end of the days of supply period (that is, on the last **PeriodSupplyDueCalendar** interval in the period) rather than at the start of the period.

For more information, see "ByPeriodEnd and ByPeriodEndWithPriority" on page 1368.

Transit time calendar

Transit time values can now be specified in terms of an explicitly set **TransitCalendar** in the **Source** table. This allows lead time calculations to properly account for those days on which an item that has been shipped is not actually being transported. For example, if the vehicles used to ship a given item are not allowed to travel on weekends, then this calendar might be set to a standard Workday calendar.

Note if the transit calendar is not specified on the **Source** table (**TransitCalendar** reference is Null), then an Everyday calendar is used instead as was always done in previous versions.

For more information, see "Lead time calendars" on page 1317.

Adjusted due date from calculated start date

A supply's dock date, ship date, and start date are always calculated backwards from the due date. Because the dock, ship, and start dates each have their own separately referenced calendar, calendar rounding might push these dates further back then they would otherwise be, thus resulting in a potentially earlier calculated start date.

In this version, a new forward-looking calculation is applied to determine new (and potentially earlier) ship, dock, and due dates based on the calculated start date. The results of these calculations are stored in a new series of fields (**ShipDateFromStart**, **DockDateFromStart**, and **DueDateFromStart**) added to the **PlannedOrder** and **ScheduleReceipt** tables.

For more information, see "Adjusted dates after calendar rounding" on page 1319.

Safety stock with expiring supply enhancement

When safety stock is maintained on a part subject to expiry, a new setting provides control over when supply to replenish expiring safety stock is planned. Previously, a replenishment order would be planned for the expiry date of the supply held as safety stock. However, this meant that safety stock could be held for some period of time where, although it had not yet expired, it also could not be used to satisfy any demands due to the part's shelf-life requirements. Therefore, replenishment orders can now be planned for the date when the expiring safety stock can no longer be used to satisfy demand (the supply's expiry date less the part's minimum shelf life).

This new functionality is configured through the new **SafetyStockPolicy.ExpiryRule** field which has options of "AtPartMSL" and "ExpiryDate" (the default setting).

Note that in some Version 2014.4 configurations, this functionality had been made available and implemented using an internal ProcessingRule/Configuration record. If upgrading from such a configuration, ensure the following to allow safety stock expiry to continue to account for a given part's minimum shelf life requirements:

- The part's safety stock settings must be defined using the **SafetyStockPolicy** table. If safety stock control settings are defined using legacy fields on the **PartType** and **PlanningCalendars** tables, then **SafetyStockPolicy** records should be created with equivalent settings to what was defined in those fields and the part's **SafetyStockPolicy** reference populated to point to an appropriate record. For more information, see "Safety stock policy configuration" on page 1401.
- The **SafetyStockPolicy** record referenced by the part must have its **ExpiryRule** field set to "AtPartMSL". For more information, see "Specify when expiring safety stock is replenished" on page 1606.

Capacity and load only constraints

Load only constraints can now be used with part's that have CRP variable lead time enabled. This means that load only constraint reporting is supported on parts that have the variable portion of their lead time set to the time needed to process a supply across all work center operations in its routing.

Note that all constraints where the **ConstraintType.ProcessingRule** is set to a value other than "LoadOnly" continue to be ignored on parts that have CRP variable lead time enabled (in previous versions, this limitation also applied when the **ProcessingRule** was set to "LoadOnly").

Filtering FlatBillDown and FlatBillUp calculations by model

FlatBillDown and **FlatBillUp** calculations now ensure that only records with a specified model are computed, which decreases the memory requirement for these calculations.

For more information, see "Limiting FlatBillDown and FlatBillUp calculations by model" on page 2224.

MultiLevelSearch now respects PlanningRule of Partial

If a top-level demand with a **DemandType.PlanningRule** of "Partial" is partially available in a future forecast period due to limited supply on a component-level part, now the demand portion that can be satisfied in the forecast period or earlier will be satisfied, and the other portion of demand will expire.

For more information, see "Ignoring unsatisfied demands" on page 1420.

Unsatisfied demand tolerance for PlanningRule of Partial

With a **PlanningRule** of "Partial", you can now place a limit on how many periods the demand can be late before it is ignored.

For more information, see "Ignoring unsatisfied demands" on page 1420.

Lot sizing in attribute-based planning

When there are demands with many different groups of attributes, one planned order is created for every attribute combination at every level, so order multiples can result in excess inventory. Attribute-based planning logic has now been enhanced to prevent the creation of excess as a result of lot-sizing.

For more information, see "Attribute-based planning" on page 1817.

Forecast Error Pooling

Forecast error pooling is a different way to calculate standard deviation at non-customer facing stages. Instead of propagating standard deviations from customer-facing stages, we propagate the actual forecast errors. We then calculate each stage's standard deviation of forecast error directly from the forecast error for that stage. This will more accurately reflect demand variability at lower level stages, which leads to a more accurate safety stock recommendation.

For more information, see "Use forecast error pooling for safety stock (optional)" on page 1963.

Support for order BOMs

Assemblies can be exploded using either an order BOM (which is specific to the sales order) or a standard material BOM. The order BOM can be defined at multiple levels in the product structure. The same assembly can use an order BOM for one order and a material BOM for another order. Once an assembly uses a material BOM instead of an order BOM, then all multi-level components below that assembly also use the material BOM. Existing supplies at multiple levels might be intended for a particular order as well. These supplies should be used to satisfy only the associated order's component demands.

When an order transitions from the order BOM to the material BOM, supplies in the same assembly can now be exploded using either an order-specific BOM or a standard BOM. Demand can now be satisfied using order-specific components or standard components depending on whether the demand is order-specific of standard. Excess on a standard supply created to satisfy one demand can be used to satisfy another demand that should not be treated as order-specific.

For more information, see "Explosion and order BOMs" on page 1342.

Defect fixes and calculation changes

The following table describes analytic changes related to either reported defects fixed or new logic introduced in Version 2016.2 RapidResponse (including Service Updates) that, in some cases, might lead to slight differences being seen in calculated output. Note that this table does not include discussion of most defect fixes made in the initial release of Version 2016.2 and which were also released in service updates to previous versions (details of those fixes are available with the service updates in which they were released).

Table 58.9: Defect fixes and calculation changes in Version 2016.2

Fix Area / Change	Description
Use of artificial stages when generating safety stock recommendations (MEIO)	In previous versions, when the multi-echelon inventory optimization analytic generated recommendations for safety stock levels, some of the recommended safety stock could potentially be placed on artificial stages (for example, a stage that was injected to model multiple "buy" sources for a part). In this version, to support the new safety stock bounds capability, safety stock will not, by default, be recommended on artificial stages. In some cases, this can lead to slight increases in the overall recommended safety stock quantities as compared to previous versions. However, if safety stock calculations should match the results produced in previous versions, a new setting in the AllowSafetyStockOnArtificialStages field on the AnalyticsConfiguration table should be set to "Y" (by default, this field is set to "N" to prevent safety stock on artificial stages). Note also that, regardless of this setting, safety stock is not allowed on artificial stages if there are records in the new SafetyStockTimePhasedBounds table.
Lot-sized quantities at end of horizon might differ when used with multi-level search or a CommitLevel of "High".	If a part source is configured to generate lot-sized planned orders that use either order minimums or multiples, and the part either has PartType.CommitLevel set to "High" or makes use of multi-level search logic, then differences in planned order quantities might be noticed at the end of the part's planning horizon. The differing planned order quantities will be of a smaller quantity than was seen previously, and therefore will result in less excess supply.
Planned orders in Netting (excess)	A change was made to how Netting logic internally handled or processed planned orders specifically as it relates to excess supply. This change was meant to improve ease of analytic maintenance going forward, and might also improve analytic performance and potentially lead to reductions in excess supply. Due this change, a difference in planned order quantities at the end of the planning horizon as compared to previous versions. Typically, it might be that the last planned order now has a smaller EffectiveQuantity and NeedQuantity.

Input table changes

This section outlines new and modified Input tables in Version 2016.2

New tables

The following tables are added:

- **IndependentDemandFeatureList**— This table contains string values used to specify the names of features associated with demand records in the IndependentDemand table (supports Feature BOM capabilities). For more information, see

"IndependentDemandFeatureList table" on page 369.

- **SafetyLeadTimeProfile**—This table holds named profiles that can be used to enable time-phased safety-lead time values for a part. If a Part has a Null reference to this table, then a single safety lead time value as defined on the Part record is used. For more information, see "SafetyLeadTimeProfile table" on page 538.
- **SafetyLeadTimeProfileDetail**- This table holds details associated with records in the SafetyLeadTimeProfile table (for example, the date on which a safety lead time value becomes effective for a part). For more information, see "SafetyLeadTimeProfileDetail table" on page 539.
- **SafetyStockTimePhasedBounds**—This table contains minimum and maximum safety stock bounds for parts, and the dates on which the bounds rules become effective. For more information, see "SafetyStockTimePhasedBounds table" on page 565.
- **TimePhasedMaximumInventory**—This table identifies the maximum inventory quantity that is set for a part that has the **DistributionPlanningRule** enabled. For more information, see "TimePhasedSafety table" on page 655.

New fields

This section outlines new and modified Input tables in Version 2016.2.

Table 58.10: New Input table fields

Table name	Field name
Allocation	Model
InventoryTransfer	TransitCalendar
Part	DaysSupplyFenceDate
Part	DaysSupplyPolicy
Part	DistributionPlanningRule
Part	MEIOFamily
Part	SafetyLeadTimeProfile
Part	UnsatisfiedDemandTolerance
SafetyStockOverride	AverageDemand
SafetyStockOverride	CalculatedServiceLevel
SafetyStockOverride	MaximumDaysOfSupply
SafetyStockOverride	MaximumSafetyStock
SafetyStockOverride	MinimumDaysOfSupply
SafetyStockOverride	MinimumSafetyStock
SafetyStockOverride	ServiceLevel
SafetyStockOverride	StandardDeviationDemand
SafetyStockOverride	UnboundedQuantity
ScheduledReceipt	DockDateFromStart
ScheduledReceipt	DueDateFromStart
ScheduledReceipt	ShipDateFromStart
Source	TransitCalendar

Modified fields

The following table lists fields modified in Input tables in Version 2016.2

Table 58.11: Modified Input fields

Table name	Field name
IndependentDemand	FeatureList. This field was previously a String field used to store a delimited list of features on a configurable end-item demand. It has been changed to a Set field that returns a vector set of strings stored in the new IndependentDemandFeatureList table. Note that during a RapidResponse upgrade, all existing strings in the FeatureList field are automatically converted to a vector set of strings on the IndependentDemandFeatureList table.

Obsolete fields

The **ForecastErrorOutlierThreshold** field on the **SafetyStockItem** table is declared obsolete and hidden (new tables and fields have been added to control all historical outlier detection for multi-echelon calculations). Note that any resources based on this field will continue to function.

Control table changes

This section outlines new and modified Control tables in Version 2016.2.

New tables

The following tables are added:

- **DaysSupplyPolicy**—This table can be used to store days-of-supply configuration settings in two time-phased zones (representing short-term and long-term planning horizons). In previous versions, only a single (non-time phased) days-of supply policy for a Part was supported based on values in the **Part**, **PartType**, and **PlanningCalendars** table (this method is still used if a Part has a Null reference to the **DaysSupplyPolicy** table). For more information, see "DaysSupplyPolicy table" on page 705.
- **DistributionPlanningRule**—This table contains the rule used for enabling the automatic distribution, or pushing, of excess supply from a sourcing site to destination sites and the rule for enabling fair share distribution to resolve stockouts at distribution sites. For more information, see "DistributionPlanningRule table" on page 722.
- **DRPConfiguration**—This table contains settings to configure planning windows to support the DRP application in RapidResponse. For more information, see "DRPConfiguration table" on page 724.

New fields

The following table lists fields added to Control tables in Version 2016.2

Table 58.12: New Control table fields

Table name	Field name
AnalyticsConfiguration	MEIOAllowSafetyStockOnArtificialStages
MUEPoolNettingType	AttributeExcessRule
SafetyStockItemType	BoundsRule
SafetyStockPolicy	SafetyDaysCalendar
SafetyStockPolicy	SafetyDaysDependentDemandUsage
SafetyStockItemType	MEIODemandHistoricalsPropagationRule

Modified fields

The following table lists fields modified in Control tables in Version 2016.2.

Table 58.13: Modified Control table fields

Table name	Field name
PartType	DaysSupplyRule. Three new options, ByPeriodEnd, ByPeriodEndWithPriority, and Ignore are added.
SafetyStockPolicy	SafetyStockQuantityRule. Two new options, TimePhasedSafetyDaysTotal and TimePhasedDaysMaximum, are added. NOTE: These options are not added to the corresponding SafetyStockQuantityRule field on the PartType table.

Calculated table changes

This section outlines new and modified Calculated tables in Version 2016.2.

New tables

The following tables were added:

- **DataChange_Historical**—This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes that previously occurred in a parent of this scenario, and was inherited by this scenario when it was derived from its parent. It is also faster and less memory intensive than the DataChange table because it only reports one type of source of change (Historical). For more information, see "DataChange_Historical table" on page 1008.
- **Data Change_Local**—This table is used to track user-initiated data changes between a scenario and its parent. It indicates a change that occurred only in this scenario (or was committed into this scenario). It is also faster and less memory intensive than the DataChange table because it only reports one type of source of change (Local). For more information, see "DataChange_PendingUpdate table" on page 1019.
- **DataChange_PendingUpdate**—This table is used to track user-initiated data changes between a scenario and its parent. It indicates only those changes made in the parent scenario, that have not yet been updated into this scenario. It is also faster and less memory intensive than the DataChange table because it only reports one type of source of change (PendingUpdate). For more information, see "DataChange_PendingUpdate table" on page 1019.

- **MaximumInventoryResult**—This table reports on parts that have the **DistributionPlanningRule** enabled. It also indicates the date, quantity, and if the source of the maximum inventory level on a specific date was determined from maximum inventory or from safety stock. For more information, see "MaximumInventoryResult table" on page 1115.
- **MEIOStageHistoricalDemand**—This table is only relevant for users that want to enable forecast error pooling. It contains demand records only for MEIO stages that are part of a family for which forecast error pooling is enabled. For more information, see "MEIOStageHistoricalDemand table" on page 1127.
- **MEIOStageHistoricalForecast**—This table is only relevant for users that want to enable forecast error pooling. It contains forecast records only for MEIO stages that are part of a family for which forecast error pooling is enabled. For more information, see "MEIOStageHistoricalForecast table" on page 1127.

New fields

The following table lists fields added to Calculated tables in Version 2016.2.

Table 58.14: New calculated table fields

Table name	Field name
Activity	AdjustedDemandDate
ConsensusForecastDetail	Date
CTPActivity	AdjustedDemandDate
CTPActivity	PushTransferId
MEIOFamilyResult	InventoryCost
MEIOFamilyResult	SafetyStockBoundsViolation
MEIOStageTimePhasedResult	AverageDemand
MEIOStageTimePhasedResult	ServiceLevel
MEIOStageTimePhasedResult	StandardDeviationDemand
PlannedOrder	DockDateFromStart
PlannedOrder	DueDateFromStart
PlannedOrder	ShipDateFromStart
SafetyStockResult	CalculatedServiceLevel
SafetyStockResult	EarlyArrivalStock
SafetyStockResult	IncomingServiceTime
SafetyStockResult	MaximumDaysOfSupply
SafetyStockResult	MaximumSafetyStock
SafetyStockResult	MinimumDaysOfSupply
SafetyStockResult	MinimumSafetyStock
SafetyStockResult	OutgoingServiceTime
SafetyStockResult	UnboundedSafetyStock
SafetyStockTimePhasedResult	AverageDemand

Table 58.14: New calculated table fields

Table name	Field name
SafetyStockTimePhasedResult	CalculatedServiceLevel
SafetyStockTimePhasedResult	MaximumDaysOfSupply
SafetyStockTimePhasedResult	MaximumSafetyStock
SafetyStockTimePhasedResult	MinimumDaysOfSupply
SafetyStockTimePhasedResult	MinimumSafetyStock
SafetyStockTimePhasedResult	ServiceLevel
SafetyStockTimePhasedResult	StandardDeviationDemand
SafetyStockTimePhasedResult	UnboundedSafetyStock
SupplyDemand	PushDate

Modified fields

The following table lists fields modified in Calculated tables in Version 2016.2

Table name	Field name
CTPActivity	Type field PushTransferOut value was added
CTPActivity	ExtendedType field PushTransfer value was added
MEIOFamilyResult	RunTime Data type was changed from Integer to Quantity.

CHAPTER 59: Legacy upgrade: SafetyStockPolicy table

Moving safety stock settings to the SafetyStockPolicy table

The **SafetyStockPolicy** table added in Version 2014.4 holds safety stock settings that control behavior such as how safety stock levels are determined in RapidResponse and the conditions under which those levels are maintained. This table is referenced by **Part.SafetyStockPolicy** and can be used to set safety stock policies by part.

In versions prior to 2014.4, all settings now found on the **SafetyStockPolicy** table were configured using equivalent fields on either the **PartType** or **PlanningCalendars** tables as shown below.

Table 59.1: SafetyStockPolicy fields and equivalents

SafetyStockPolicy fields	Pre-2014.4 equivalent
OrderPointDateRule	PartType.OrderPointDateRule
OrderPointIncrease	PartType.OrderPointIncrease
PercentSafetyInterval	PlanningCalendars.PercentSafetyInterval
RangeOfCoverage	PlanningCalendars.RangeOfCoverage
SafetyStockDateRule	PartType.SafetyStockDateRule
SafetyStockQuantityRule	PartType.SafetyStockQuantityRule
SafetyStockRule	PartType.SafetyStockRule

Note that all **PartType** and **PlanningCalendars** fields shown in the preceding table are still available and applicable to all parts whose **SafetyStockPolicy** reference is Null. Otherwise, if the **SafetyStockPolicy** reference is populated then the part takes its safety stock settings from the **SafetyStockPolicy** table. Use of this new table is recommended as it supports easier maintenance of safety stock settings than is possible using the **PartType** and **PlanningCalendars** tables each of which contain many other planning policies. For example, modelling many different combinations of safety stock settings in what-if scenarios might be needed when working with the Inventory Planning and Optimization solutions included with RapidResponse.

To support moving safety stock settings to the new table, RapidResponse includes a process to automatically create **SafetyStockPolicy** records based on a part's existing **PartType** and **PlanningCalendars** values. The **Part.SafetyStockPolicy** reference on each **Part** record is then updated to point to the appropriate value in the **SafetyStockPolicy** table. This process requires use of the **Create**

Safety Stock Policy workbook, which is available for download from the Kinaxis Knowledge Network at https://knowledge.kinaxis.com/s/article/CreateSafetyStockPolicyworkbook50812. Kinaxis consultants can also assist with use of this workbook and setting up records in the **SafetyStockPolicy** table.

The **Create Safety Stock Policy** workbook contains two workbook commands, one of which creates basic **SafetyStockPolicy** records using only the equivalent settings from the **PartType** table and the other which creates complete **SafetyStockPolicy** records including calendar references. Each of these commands subsequently updates the **SafetyStockPolicy** reference on **Part** table entries to point to one of the newly created records as appropriate (note that if basic **SafetyStockPolicy** records are created, the **RangeOfCoverage** and **PercentSafetyInterval** references are left Null and each part's **PlanningCalendars** reference is still used to provide those values).

To create basic SafetyStockPolicy records

1. Import the **Create Safety Stock Policy** workbook and then open the workbook.
2. Create a private scenario based on the **Enterprise Data** scenario, and then select the newly created scenario.
3. Select the **Safety Stock Policy Basic** worksheet.
4. Click **Run Command** on the workbook toolbar, and then click **Create Safety Stock Policy - Basic**.
5. Click **Yes**, and then **OK**.
6. Ensure records have been added to the **Safety Stock Policy Basic** worksheet.

   There should be one record added for each **PartType** record with the **Value** field and all other settings inherited from the **PartType** (all fields other than **Value** can subsequently be changed if necessary). Additional values can also be added to this table as required.
7. Click the **Part** worksheet, and ensure each part's **SafetyStockPolicy** reference has been updated (the **Value** field should be populated). The referenced policy values can also be changed if necessary.
8. Commit the private scenario created in step 2 to the **Enterprise Data** scenario, and ensure other scenarios based on **Enterprise Data** are subsequently updated to include the **SafetyStockPolicy** changes.

Caution: Before the next data update, ensure that any **Part** records being provided have their **SafetyStockPolicy** fields populated to reference the appropriate **Value** (with data mappings updated as required).

To create SafetyStockPolicy records with Calendar references

1. Import the **Create Safety Stock Policy** workbook and then open the workbook.
2. Create a private scenario based on the **Enterprise Data** scenario, and then select the newly created scenario.
3. Select the **Safety Stock Policy Basic** worksheet.
4. Click **Run Command** on the workbook toolbar, and then click **Create Safety Stock Policy - With Calendars**.
5. Click **Yes**, and then **OK**.
6. Ensure records have been added to the **Safety Stock Policy Basic** worksheet.

   There should be one record added for each **PartType** record with the **Value** field and all other settings inherited from the **PartType** and **PlanningCalendars** values (all fields other than **Value,**

which concatenates Values from the appropriate **PartType** and **PlanningCalendars** records, can subsequently be changed if necessary). Additional values can also be added to this table as required.

7. Click the **Part** worksheet, and ensure each part's **SafetyStockPolicy** reference has been updated (the **Value** field should be populated). The referenced policy values can also be changed if necessary.
8. Commit the private scenario created in step 2 to the **Enterprise Data** scenario, and ensure other scenarios based on **Enterprise Data** are subsequently updated to include the **SafetyStockPolicy** changes.

Caution: Before the next data update, ensure that any **Part** records being provided have their **SafetyStockPolicy** fields populated to reference the appropriate **Value** (with data mappings updated as required).

CHAPTER 60: Legacy upgrade: Statistical forecasting

Transitioning to analytics for forecasting and disaggregation

RapidResponse Version 2014.4 introduced improved S&OP processes that include statistical forecasting and disaggregation calculations performed by analytics. If you are upgrading from RapidResponse 2014.2 (or earlier) to RapidResponse 2014.4 (or later), you will need to complete the following procedures so that you can take advantage of the new statistical forecasting and disaggregation capabilities of RapidResponse:

- "Set S&OP parameters using the SOPAnalyticsConfiguration table" on page 2413
- "Move outlier adjustments to a new table" on page 2415

Set S&OP parameters using the SOPAnalyticsConfiguration table

In RapidResponse 2014.4, S&OP parameters for forecasting and disaggregation calculations are defined in the new **SOPAnalyticsConfiguration** table, which holds only one record. This table is used primarily to define the default configuration of forecast disaggregation, but it also includes fields used to define the forecast, such as **ForecastEndDate** and **HistoricalEndDate**. This table must be populated before you can use analytics to calculate the forecast.

In RapidResponse 2014.2 (and earlier), S&OP settings were controlled by the Demand Planning profile variables that are specified in the **S&OP** worksheet in the **RapidResponse Administration** workbook. With the exception of five fields, all fields on the **SOPAnalyticsConfiguration** table replace profile variables, as shown in the following table.

Table 60.1: SOPAnalyticsConfiguration fields and corresponding profile variables

SOPAnalyticsConfiguration field	Replaces Profile variable
CycleCalendar	DP_CycleForecastCalendar
DisaggregationActualsCategory	DP_DefaultDisaggregationActuals
DisaggregationCalendar	DP_BaseCalendar
DisaggregationHistoricalIntervalCount	DP_HistoricalIntervalCount
DisaggregationInnerCalendar	DP_InnerCalendar

Table 60.1: SOPAnalyticsConfiguration fields and corresponding profile variables

SOPAnalyticsConfiguration field	Replaces Profile variable
DisaggregationOuterCalendar	DP_OuterCalendar
CalcForecastStartDate	DP_ForecastStartDate
CalcHistoricalEndDate	DP_HistoricalEndDate
CalcForecastEndDate	Not applicable
ForecastDisaggregationOverrideCategory	Not applicable
ForecastEndOffset	Not applicable
ForecastStartOffset	Not applicable
RunDate	Not applicable

To keep using the S&OP settings specified by profile variables, administrators can view the profile variables to determine the values that should be used to populate the table. For example, if the value of the profile variable **DP_HistoricalIntervalCount** is "12", the **DisaggregationHistoricalIntervalCount** field should be set to "12".

To view profile variables

1. In RapidResponse 2014.4 (or later), in the **Administration** pane, in the **System Settings** list, select **Application Settings**.
2. Select the **Application Variables** worksheet.

To show or hide profile variables in the Application Variables worksheet

1. In RapidResponse 2014.4 (or later), in the **Administration** pane, in the **System Settings** list, select **Application Settings**.
2. Select the **Application Variables** worksheet.
3. On the workbook toolbar, click **Data Settings**.
4. In the **Data Settings** dialog box, in the **Application** list box, select an option from the list.
5. In the **S&OP Analytics** list box, select one of the following:
   - **Configured**
   - **Not configured**
6. Click **OK**.

The **SOPAnalyticsConfiguration** table is populated by adding values to the **SOP Analytics Configuration** worksheet in the **Control Tables** workbook. The field values of **SOPAnalyticsConfiguration** are modifiable on a per-scenario basis (each scenario includes a version of the table's record).

To open the SOP Analytics Configuration worksheet

1. From RapidResponse 2014.4 (or later), in the **Explorer**, select the **Control Tables** workbook.
2. Select the **SOP Analytics Configuration** worksheet tab.

Note: The **SOPAnalyticsConfiguration** table references the **Calendar** table to define key dates for the forecast. Ensure that the calendars are defined in the **Calendar** table so the correct dates can be referenced.

When the **SOPAnalyticsConfiguration** table has been populated, the replaced profile variables are no longer required for demand planning. An administrator can hide the unnecessary profile variables from the S&OP worksheet.

Move outlier adjustments to a new table

Before calculating the statistical forecast, historical demand data must be cleansed of errors and anomalies. RapidResponse allows you to adjust the demand data for data errors, causals, and outliers. In RapidResponse 2014.2 (and earlier), data error, causal, and outlier adjustments were grouped together as causal factors, and all adjustment data was stored in the **CausalFactorDetail** table for a part, customer, and category combination.

In RapidResponse 2014.4, outlier adjustments are defined at the forecast item parameters level and stored in the new **ForecastItemParametersOutlier** table. This change offers you more flexibility when determining if adjusted historical demand quantities should be included when generating forecasts, as items that belong to more than one forecast category in **ForecastItemParameters** can use a different set of outlier adjustments for each category.

RapidResponse includes a process to move outlier adjustment data from the **CausalFactorDetail** table to the **ForecastItemParametersOutlier** table. This process requires the S&OP Outliers Upgrade workbook, which is available for download from the Kinaxis Supply Chain community at https://community.kinaxis.com/community/supply_chain_management_kinaxis_rapidresponse/upgrade_center.

The S&OP Outliers Upgrade workbook includes two worksheets: Outliers Source and Outliers Insert. The Outliers Source worksheet is used to retrieve outlier data from the **CausalFactorDetail** table, and the Outliers Insert worksheet is used to hold the outlier data that will populate the **ForecastItemParametersOutlier** table.

The Outliers Insert worksheet requires you to select one of two commands. The first command is called Copy & Delete Outliers. It copies outlier records from the Outliers Source worksheet to the Outliers Insert worksheet and then deletes the outlier records from the **CausalFactorDetail** table. Use this command if outlier adjustments should be made only at the forecast item parameters level using the new **ForecastItemParametersOutlier** table.

The second command is called Copy Outliers and Set Processing Rule to Safety Stock. It copies outlier records from the Outliers Source worksheet to the Outliers Insert worksheet and then changes the **ProcessingRule** field on **CausalFactorCategory** to "SafetyStock". This means that outlier adjustments stored in **CausalFactorDetails** are not deleted but used only for inventory planning and optimization (safety stock) calculations. Use this command if part-level outlier adjustments should be used for safety stock calculations.

To delete outliers from CausalFactorDetails

1. Import the **S&OP Outliers Upgrade** workbook and then open the workbook.
2. Create a private scenario based on the **Enterprise Date** scenario, and then select the newly created scenario.
3. In the **Outliers Source** worksheet, select **Outlier** as the Causal Category.

4. In the **Outliers Insert** worksheet, click **Run Command** on the workbook toolbar , and then click **Copy & Delete**.
5. Click **Yes**, and then **OK**.
6. Commit the private scenario created in step 2 to the **Enterprise Data** scenario, and ensure other scenarios based on **Enterprise Data** are subsequently updated to include the changes.

To use CausalFactorDetail outliers for safety stock calculations

1. Import the **S&OP Outliers Upgrade** workbook and then open the workbook.
2. Create a private scenario based on the **Enterprise Date** scenario, and then select the newly created scenario.
3. In the **Outliers Source** worksheet, select **Outlier** as the Causal Category.
4. In the **Outliers Insert** worksheet, click **Run Command** on the workbook toolbar , and then click **Copy Outliers and Set Processing Rule to Safety Stock**.
5. Click **Yes**, and then **OK**.
6. Commit the private scenario created in step 2 to the **Enterprise Data** scenario, and ensure other scenarios based on **Enterprise Data** are subsequently updated to include the changes.

Index

A

about
  analytic modifiers 1287
  analytics 1285
  available to promise 1449
  calculated tables 937
  capable-to-promise 1453
  constraints 1629
  control tables 669
  data groups 150
  dates 1296
  effectivity 1737
  explosion 1337
  full-level pegging 1493
  global settings 2203
  inclusive lead time 1323
  input tables 171
  interchangeable parts 1545
  lead time 1307
  master production scheduling 1425
  multi-level constraints 1633
  multi-sourcing 1378
  netting 1331
  notes tables 1269
  order prioritization 1765
  part sources 1629
  pools 1745
  safety stock 1400
  site calculations 1731
  tables 149
accessing
  help and documentation 115
Acrobat 116
Activity records 941, 2203
Activity table vs CTPActivity table 985
adjusted R-squared 2062
Adobe Reader 116
Akaike information criterion 2062
allocated scheduled receipts 1503
allocations 1424
  adjustments 1399
  and full-level pegging[allocations
    full-level pegging] 1499
  BlowThroughNewAllocation 1391
  BlowThroughPlannedAllocation 1391
  constraints 1637
  data generation 1399
  interchangeable parts 1549
  kit 1499
  minimum 1637
  overriding 1817, 1845, 1853, 1935
  scheduled receipts 1503
  substitute 1576
Allotment Override
  analytic 1290
allotment overrides 1817, 1845, 1853, 1935
  data model 1853
  processing rules 1854
  setting up 1854
alternate bill of material 1759
alternate BOM 1759
Alternate BOM
  analytic 1289
alternate parts 1545, 1551
AlwaysOn Analytics
  Allotment Override 1290
  Alternate BOM 1289
  Available-to-Promise 1286
  Bill of Material 1286
  BOM substitution 1288
  Capable-to-Promise 1286
  Capacity Requirements Planning 1286
  Co/By Product 1290
  Constraint analysis 1289
  Cost of Goods Sold 1287
  Cost Roll-Up 1287
  Days of supply 1290
  Explosion 1286
  Fair/equal share 1289
  Financial Analysis 1287
  Forecast Consumption 1286
  Full-Level Pegging 1286
  Global part substitution 1288
  Incremental availability 1289
  Interchangeable parts 1288
  Inventory Analysis 1286
  Inventory pooling 1288
  Material expiry 1290
  Model effectivity 1288
  MRP 1286
  Multi-site 1288
  Multi-sourcing 1288
  Netting 1286
  Priority 1289
  Unit effectivity 1288
analytic compatibility
  attribute-based planning 1828
analytic limitations 1291
analytic modifiers
  about 1287
  alternate BOM 1759
  constraint analysis 1629
  inventory pooling 1745
  model effectivity 1737
  multi-sourcing 1378
  priority 1765
  site calculations 1731
  unit effectivity 1738

analytics
about 1285
affected by interchangeable parts 1549
capable-to-promise 1453
capacity requirements planning 1515
impacts by multi-level constraints 1644
inventory analysis 2205
inventory turns 1451
master production scheduling 1425
material requirements planning 1331
projected cost of goods sold 1503
See also analytic modifiers; calculations[analytics aaa] 1285
anchor date logic 1341
ARIMAX forecast model 2042
assembly explosion 1337
ATP. See available to promise[ATP] 1449
attribute-based planning 1817, 1832
attribute rules
creating 1825
attributes
creating 1820-1821
data types 1821
attrition
yield and scrap controls 694
yield and scrap quantities 1391
Auto-Regressive Integrated Moving Average 2034
Available-to-Promise
analytic 1286
available to promise
interchangeable parts 1549
available to promise, about 1449
AvailableDate 1300
AvailableDate calculations using lead time 1397
AvailableStartDate 1297

B

Bass Diffusion Model 2118
best fit model 2053
with holdout period 2053
bill of material
effectivity 1341
explosion 1338
explosion effectivity 1739
negative quantities 1338
netting alternates 1761
planning bill 1431
using named alternates 1759
Bill of Material
analytic 1286
bill of materials
with interchangeable parts 1549
BOM substitution
analytic 1288
BOMs
order 1342
Boolean
data type 143
buy parts, explosion 1340

C

CalcStartDate 1298
Calculated Data Model poster 118
calculated fields in input tables 171
calculated table owners 937
calculated tables 133
about 937
See also Table Index[calculated tables zzz] 937
calculations
available date 1397
available to promise 1449
capable-to-promise and priority 1771
capacity load 1742
capacity requirements planning 1515
configuring 1707, 1717, 2161, 2203, 2209, 2253
constraint analysis 1629
cost roll-up 1511
dates 1295
dynamic safety stock 1402
explosion 1337
forecast consumption 1431
forecast spreading 1426
full-level pegging 1493
gating constraint 1458
gating part 1457
Inventory Analysis 2205
inventory turns 1451
late supply 1458
lead time overrides 1328
material requirements planning 1331
netting 1331
order priority 1765
order spreading 1376
projected cost of goods sold 1503
safety stock 1400
shortages 1375
spread profile 1429
start date for planned orders 1376
transaction sequence 1491
calendars
display values 1306
with interchangeable parts 1297
Campaign Planning 166
capable-to-promise
about 1453
effect of constraint 1491
gating constraint 1458
gating part 1457
impacts by multi-level constraints 1645
late supply calculations 1458
matching supply to demand 1488
order priority 1771
scenarios 1488
tables 1454
transaction sequence 1490
Capable-to-Promise
analytic 1286
capable to promise
interchangeable parts 1456
capacity load calculations 1742
Capacity Requirements Planning
analytic 1286
category
forecast 2073
chained interchangeable part groups 1547
co-products, by-products 1581

Co/By Product
analytic 1290
CommitLevel 1790
components, explosion 1337
configuration records. See global settings 2203
configuring
Inventory Analysis calculations 2205
master production scheduling 1442
multi-level search 1873
planned order limit 2204
RapidResponse 1707, 1717, 2161, 2203, 2209, 2253
time unit records 2203
Conflict table 2173
conflicts
project management 2173
consensus demand planning
calculations 2072
data model 2070
Consensus demand planning 2069
consensus forecast 2069
calculations 2072
category weights 2073
data model 2070
with Event Management 2120
Consensus forecast adjustments and overrides 2079
ConsensusForecast table 2070
constrained planning
constraint rate 1640
creating planned orders 1638
CumulativeMax 1640
load 1637
MaterialStartDate 1641
minimum allocation 1637
multi-level constraints 1633
multiple constraints 1643
OnTimeQuantity 1638
PrePlanLimit 1638
rescheduling scheduled receipts 1636
scheduled receipts with tasks 1637
transfer orders 1638
Constraint analysis
analytic 1289
Constraint Manager 166
constraints
about 1629
allocating 1637
CumulativeMax 1640
effect on capable-to-promise 1491
families 1633
gating 1458
interchangeable parts 1634
load 1637
multiple 1629
netting 1334
project management 2172
rate 1640
setting up 1631
Contents tab 118
control character 145
control sets 161
control tables 133
about 669
configuration not in 1707, 1717, 2161, 2203, 2209, 2253
See also Table Index[control tables zzz] 669
cost of goods sold
interchangeable parts 1550
Cost of Goods Sold
analytic 1287
cost roll-up
FlatNetQuantityPer 1512
labor 1512
material 1511
overhead 1513
Cost Roll-Up
analytic 1287
costs
interchangeable parts 1550
creating interchangeable parts 1545
Croston's method 2034
CTP 1454
CTPActivity records 2203
CTPActivity table vs Activity table 985
cumulative lead time
with model and unit effectivity[cumulative lead time model and unit effectivity] 1741
customer orders
demands 1397
gating constraint 1458
gating part 1457

D

dashboards
help 122
data groups, about 150
Data Model for Import poster 117
data model posters 116
data scenarios
capable-to-promise 1488
full-level pegging 1496
data type
enum 145
data types 143
Date
data type 143
date constants
Future 1303
MRPDate, date constants 1303
Past 1303
Undefined 1303
dates
about[dates aaabout] 1296
anchor[dates aaanchor] 1341
AvailableDate 1300
AvailableStartDate 1297
CalcStartDate 1298
calculation rules[dates aacalculation rules] 1304
calculations[dates aacalculations] 1295
constants[dates aaconstants] 1303
DeliveryDateTime 1299
DemandDate 1302
display values 1306

DockDate 1299
DueDate 1301
EffDockDate 1299
EffStartDate 1298
FirstAvailableDate 1301
material requirements planning[dates
aamaterial requirements planning] 1303
MaterialAvailableDate 1300
MaterialStartDate 1298, 1641
NeedDate 1301
PTFDate 1302
RecommendedDate 1302
RecommendedDockDate 1300
ReschedDate 1302
ReschedDockDate 1300
rescheduled by explosion[dates
aarescheduled by explosion] 1340
ShipDate 1299
ShipDateTime 1299
start for planned orders[dates
aastart for planned orders] 1376
StartDate 1298
SupplyAvailableDate 1301
SupplyStartDate 1298
DateTime
data type 143
days of supply
interchangeable parts 1549
Days of supply
analytic 1290
Days of Supply 1362
deadlines 2191
DeliveryDateTime 1299
demand expiry 1607
DemandDate 1302
demands
controls 1398
customer orders 1397
ignoring unsatisfied 1420
negative 1338
netting 1332
records 1398
tables 1397
time fence 1441
display values for calendar dates 1306
distribution planning 1718
DockDate 1299
documentation
feedback 119
documentation conventions 118
documentation, accessing 115-116, 118
double-precision
data type 144
double exponential smoothing 2036
DueDate 1301

E

earliest expiry date 1611
EffDockDate 1299
effectivity
about 1737
bill 1341, 1739
model 1737
testing for 1739
unit 1738
EffectivityRule test 1742
EffStartDate 1298
enum data type 145
error measures
statistical forecasts 2060
Event Management 2109
calculated tables 2120
date bucketing 2125
disaggregation 2128
event phases 2112
input tables 2113
negative adjustments 2133
setup 2114
statistical forecast 2123
event phases
action types 2110, 2116
Events 2109
execution rules. See processing rules[execution
rules] 1398
expediting scheduled receipts 1654
expiration 1601
expiring quantities 1624
expiry data
on demands 1607
on supplies 1602
setting up 1601
expiry date
calculating on scheduled receipts 1617
estimated on scheduled receipts 1617
expiry dates
calculated on planned orders 1618
providing 1605
scheduled receipts 1603
expiry dates estimated on planned orders 1618
expiry intervals 1603
explosion
about 1337
alternate bill of material 1761
assembly and components 1337
bill effectivity 1739
bill of material 1338
low level code 1341
material 1341
negative values 1338
part types 1340
planned orders 1741
product structure information 1337
rescheduled dates 1340
with interchangeable parts 1549
Explosion
analytic 1286
exponential smoothing 2037, 2043

F

fair share 1868
Fair/equal share
analytic 1289
families of constraints 1633
feedback, providing 119
field types
vector set 141
filtering
table owner 937

financial planning
allocated scheduled receipts 1503
projected cost of goods sold 1503
unallocated scheduled receipts 1505
FirstAvailableDate 1301
FixedDuration setting 2177
FixedUnits setting 2178
FixedWorkRate sertting 2179
forecast
consumption 1431
spreading 1426
forecast adjustments
by attribute 1831
forecast category 2069
weights 2073
forecast consumption
by attribute 1830
interchangeable parts 1549
Forecast Consumption
analytic 1286
forecast disaggregation
calendars 2090
configuration 2088
data model 2083
statistical forecast 2103
forecast disaggregation rates 2087
forecast error 2063
forecasts
consumption with interchangeable parts 1549
foreign key 141
forms
help 123
frequently used tables 148
full-level pegging
about 1493
examples 1496
interchangeable parts 1499
through allocations[full-level pegging allocations] 1499
Full-Level Pegging
analytic 1286
fully interchangeable parts. See interchangeable parts [fully interchangeable parts] 1545
Future, date constants 1303

G

gating constraint 1458
gating part 1457
interchangeable parts 1457
generating allocation data 1399
Global part substitution
analytic 1288
global settings 1707, 1717, 2161, 2203, 2209
about 2203

H

Hannan-Quinn criterion 2063
help
accessing 115
historical data
safety stock 1947
statistical forecasts 2021
Historical Demand poster 118
Historical Supply poster 118
Holt-Winters models 2037

I

ignoring unsatisfied demands 1420
importing
statistical forecasts 2058
in-transit scheduled receipts 1734
interchangeable parts 1549
inclusive lead time
about 1323
Incremental availability
analytic 1289
Index tab 118
input tables
about 171
calculated fields 171
reference fields 171
See also Table Index[input tables zzz] 171
table types
input 133
Integer
data type 144
Integrated Project Management poster 118
interchangeable parts
about 1545
calendars 1297
capable to promise 1456
chained 1547
constraints 1634
effect on analytics 1549
full-level pegging 1499
gating part 1457
matching supply to demand 1334
model-unit effectivity 1745
netting 1334
pool 1745
recursive 1548
setting up 1545
Interchangeable parts
analytic 1288
InTransit rule 1734
InTransitExact rule 1734
Invalid character 145
Invalid characters 145
inventory
levels for planned orders 1358
pools 1745
turns 1451
using available 1358
Inventory Analysis
analytic 1286
Inventory Analysis, configuring 2205
inventory optimization
multi-echelon 1936
inventory planning
single-echelon 1936
inventory planning and optimization
configuration 1947
Inventory Planning and Optimization poster 118
Inventory pooling
analytic 1288
inventory sites 596

IsRecursive 1423
IsShopKanbanPart 2246
IsSourceKanbanPart 2246
itemsource
  partsource 2135

K

Kanban analysis
  interchangeable parts 1550
Kanban calculations 2239
KanbanDisableDemand 2244
KanbanDisableOnhand 2244
KanbanDisableSafetyStock 2244
KanbanDisableSupply 2244
key fields
  foreign 141
  primary 141
Kinaxis
  about 114
  contacting 114
  providing feedback to 119
Kinaxis Customer Support 126
Kinaxis Knowledge Network 126

L

labor costs
  projected 1508
  roll-up calculations 1512
lag percentage error 2063
lead time 2135
  about 1307
  calculations with overrides 1328
  cumulative 1741
  inclusive 1323
  overrides 1328
  transfer part sources 1732
  usage in AvailableDate calculations 1397
leading part. See primary part [leading part] 1545
LevelResources setting 2186
licensing
  sites 596
limiting planned orders 1376
linear forecast model 2039
links between tables 150
log-likelihood 2064
logical sites 596
lot sizing 1358, 1361
low level code 1341

M

Mac OS 118
make parts, explosion 1340
MASE 2065
master production scheduling
  about 1425
  available to promise 1449
  configuration 1442
  forecast consumption 1431
  forecast spreading 1426
  spread profile calculations 1429
matching supply to demand 1488
material cost 1869
material costs
  projected 1505
  roll-up calculations 1511
material expiration 1601
Material expiry
  analytic 1290
material explosion 1341
material requirements planning
  calculations 1331
  run date 1303
MaterialAvailableDate 1300
MaterialStartDate 1298, 1641
maximum order quantity 1359
mean absolute error 2064
mean absolute error by mean 2065
mean absolute percentage error 2064
mean absolute percentage error by forecast 2064
mean absolute scaled error 2065
mean error 2066
mean percentage error 2066
mean percentage error by forecast 2066
mean square errors 2066
MEIO 1936
  control settings 1953
  creating families 1990
  data model 1942
  forecast error pooling 1963
milestones 2191
minimum order quantity 1359
minimum shelf life 1607
MLR
  forecast accuracy 2043
  models 2042
  parameters 2043
MLS 1869-1870
  part source effectivity 1865
  short-term excess 1870
MLS Configuration workbook 1873
model
  effectivity 1737
  netting 1745
model-unit effectivity
  interchangeable parts 1745
Model-Unit Effectivity 167
Model and unit effectivity
  analytic 1288
Model effectivity
  analytic 1288
model, unit, and pool netting
  interchangeable parts 1550
modifiers. See analytic modifiers 1378
Money
  data type 144
moving average 2039, 2043
MPN-MRP set. See interchangeable parts [MPN-MRP set] 1545
MPS. See master production scheduling[MPS] 1425
MRP
  analytic 1286
MRP. See material requirements planning[MRP] 1331
MRPDate, date constants 1303
MUERule test 1743
multi-echelon inventory optimization
  control settings 1953

creating families 1990
data model 1942
lead time configuration 1960
multi-level constraints
about 1633
impact on analytics 1644
multi-level search
configuration workbook 1873
material cost 1869
part source selection 1877
part substitution 1909
short-term excess 1870
Multi-level search
part source effectivity 1865
multi-level search;part source selection 1870
Multi-site
analytic 1288
multi-site. See site calculations[multi-site] 1731
Multi-sourcing
analytic 1288
multi-sourcing, about 1378
multiple
constraints 1629
order quantity 1359
multiple linear regression
forecast accuracy 2043
models 2042
parameters 2043

N

namespaces 131
NeedDate 1301
negative bill of material 1338
negative dependent demands 1338
netting
about 1331
Activity records 2203
alternate bill of material 1761
impacts by multi-level constraints 1645
interchangeable parts 1334
maintaining commitments 1490
matching supply to demand 1488
model 1745
model effectivity 1738
model, unit, and pool factors 1746
model, unit, and pool with interchangeable parts 1550
order priority 1490
pool 1745
scheduled receipts 1396
start date calculations 1376
start unit values 1334
supply 1396
transaction sequence 1490
types 1332
unit 1746
Netting
analytic 1286
new features 2347, 2350, 2355, 2359, 2363, 2366, 2370, 2373, 2380, 2390, 2394, 2398, 2409, 2413
non-reschedulable supplies 1335
Not Reschedulable rule 1335
Note
data type 144
note fields in custom tables 1269
notes tables 133
about 1269
See also Table Index[notes tables zzz] 1269
null reference 142

O

operation loads, scheduling 1537
operations
phases 1523
order BOMs 1342
order point 1351
order point increase 1352
order prioritization
about 1765
effective priority 1767
effective transaction sequence 1769
in netting[order prioritization netting] 1490
planned orders 1790
scheduled receipts 1788
See also order priority scenarios[order prioritization aaa] 1765
sequencing 1771
tables, fields, values 1765
with capable-to-promise calculations[order prioritization capable-to-promise calculations] 1771
with dependent demands[order prioritization dependent demands] 1770
order priority scenarios
basic prioritization 1795
maintaining commitments 1796
orders have same due date 1797
orders not prioritized 1795
overriding protected orders 1800
protecting released orders 1800
scheduling production 1801
setting default priorities 1802
order quantities 1359
orders
customers 1397
lot sizing 1358
policies 1358
prioritization 1765
quantity 1358
spreading with Takt Time 1376
start date calculations 1376
transfer 1638
outliers
in statistical forecasts 2022
overhead costs
projected 1508
roll-up calculations 1513
owner
reference field 937

P

part-substitution
MLS 1909
part sources
about 1629

and lot sizing[part sources lot sizing] 1361
for planned orders[part sources planned orders] 1333
for scheduled receipts 1389
multiple 1377
replenishing supply 1629
part substitution 1551
BOM level 1565-1566
customer-specific 1560
global 1551
limiting to excess supply 1558, 1569
planned order generation 1571
parts
buy 1340
explosion 1340
interchangeable 1545
make 1340
making interchangeable 1545
phantom 1390
primary 1545
See also part sources[parts aaa] 1333
substitute 1545
vendor managed 1341
partsource
itemsource 2135
Past, date constants 1303
PDF 116
periods of coverage
calculating 1144
calculations 1145
calendar for 2206
permissions 115
phantom processing 1390
physical sites 596
planned orders
creating 1638
CumulativeMax 1640
interchangeable parts 1549
inventory levels 1358
limiting 1376, 2204
order point 1351
part sources 1333
preplan 1638
priority 1790
quantity 1361
recommending new 1333
rescheduling 1343
sizing 1358
start date calculations 1376
Takt Time and interchangeable parts 1549
with part substitution 1571
planning time fence 1344
pool
interchangeable parts 1745
pools
about 1745
mapping 1748
netting 1745
PrePlanLimit 1638
primary key 141
primary part 1545
printing
help 115, 121
Priority
analytic 1289
processing rules
ActivityBeat 2203
allotment overrides 1854
bill effectivity 1739
demand 1398
demand type 1399
for supply[processing rules supply] 1397
MaximumPlannedOrdersPerPeriod 1344
processing rules. See global settings 2203
product structure, explosion 1337
project management
costs 2188
levelingresources 2186
resource availability 2183
task conflicts 2173
tasks 2167, 2176
project planning
deadlines 2191
milestones 2191
penalties 2191
projected cost of goods sold
calculations 1503
labor 1508
material 1505
overhead 1508
PTF 1344
PTFDate 1302
purchased parts, explosion 1340

Q

Quantity
data type 144
QuantitySingle
data type 144

R

R-squared 2067
R integration 168
RapidResponse Calculated Data Model poster 118
RapidResponse configuration 1707, 1717, 2161, 2203, 2209, 2253
RapidResponse Data Model for Import poster 117
RapidResponse Historical Demand poster 118
RapidResponse Historical Supply poster 118
RapidResponse Integrated Project Management poster 118
RapidResponse Inventory Planning and Optimization poster 118
RapidResponse Sales and Operations Planning poster 118
ratios
forecast category 2073
real number
data type 144
Recommend Only rule 1335
RecommendedDate 1302
RecommendedDockDate 1300
record uniqueness 141

recursion 1423
recursive data 1423
recursive interchangeable part groups 1548
Reference
  data type 144
reference field
  owner 937
reference fields
  nullable 142
reference fields in input tables 171
regression sum of squares 2067
relationships between tables 150
reporting horizons 2227
ReschedDate 1302
ReschedDockDate 1300
reschedulable supplies 1335
rescheduling
  planned orders 1343
  scheduled receipts 1636
  supply 1335
reserving components 1817, 1845, 1853, 1935
residual sum of squares 2067
Rforecast 2039
rolled-up costs for interchangeable parts 1550
rolling lag percentage error 2067
rolling weights
  forecast category 2073
root mean square errors 2068
RR_ClearToBuild 2257
RR_CtbExcess 2257
RR_CtbPrdnSR 2257
RR_CtbProduction 2257
RR_CurrentCPO 2257
RR_CurrentSR 2257
RR_Dist_PSEffective 2255
RR_DistSite 2255
RR_DistSite_DC 2255
RR_DistSite_DCText 2255
RR_DistSite_Direct 2255
RR_DistSite_DirectText 2255
RR_DistSite_VMI 2255
RR_DistSite_VMIText 2255
RR_InternalBuy 2255
RR_MEM_BrandIntransit 2250
RR_MEM_BrandInTransit 2254
RR_MEM_BrandReplenPlan 2249
RR_MEM_BrandSite 2249-2250, 2254
RR_MEM_CMReplenPlan 2249
RR_MEM_HorizonLimit 2250
RR_MEM_HubDateCalc 2250
RR_MEM_HubIntransit 2250
RR_MEM_HubInTransit 2254
RR_MEM_HubInventory 2250
RR_MEM_HubSite 2250, 2254
RR_MEM_OnHandDestination 2250
RR_MEM_OnHandSupplier 2250
RR_MEM_OutsourcedPart 2250
RR_MEM_RequirementTarget 2249
RR_MEM_ShipPlan 2250
RR_MEM_Supplier_Supply_Part 2254
RR_MEM_SupplierDemand 2254
RR_MEM_SupplierSite 2249-2250, 2254
RR_MEM_SupplyPlan 2254
RR_Transfer 2255

S

safety stock
  about 1400
  bounds 1959, 1983, 2010
  calculations 1402
  control settings 1948
  data model 1937
  days of supply 1981
  disabling 1749
  fixed lead time 1968
  for interchangeable parts 1549
  forecast error pooling 1963
  historical data 1947
  item mapping 1962
  order priority 1418
  policy 1401
  recommendations 2005
  rules 1402
  time-phased 1977
  variable lead time 1974
Safety stock
  order priority 1780
safety stock items
  configuring 1954
Sales and Operations Planning poster 118
SBC 2068
scheduled receipts
  allocated 1503
  expediting 1654
  in-transit 1734
  in-transit with interchangeable parts 1549
  lead time overrides 1328
  part sources 1389
  priority 1788
  rescheduling 1636
  unallocated 1505
  usage in netting 1396
  with tasks[scheduled receipts
    tasks] 1637
scheduling
  tasks 2167
scheduling operation loads 1537
Schwarz Bayesian information criterion 2068
scrap 1391
SEIP 1936, 1968
  average demand profiles 1962
  data model 1938
self-allocations 1424
self-healing supply chain 2135
Self-Healing Supply Chain 169
  configuring 2153
  data requirements 2141
  error measure calculation 2150
  itemsource candidacy 2146
  machine learning 2141
  PartSource effectivity 2147
  processing rules 2153
  tables 2136
setting up constraint data 1631
ShipDate 1299
ShipDateTime 1299
ShopKanbanAvgDemandCalc 2246
ShopKanbanAvgDemandFilter 2246

ShopKanbanCalcBinQty 2246
ShopKanbanCalcNumBins 2246
shortage calculations 1375
single-echelon inventory planning 1968
  average demand profiles 1962
  data model 1938
single-precision
  data type 144
single use kanban, planned additional requirements 1352
site calculations
  about 1731
  tables 1731
sites
  inventory 596
  licensing 596
  logical 596
  physical 596
SourceKanbanAvgDemandCalc 2246
SourceKanbanAvgDemandFilter 2246
SourceKanbanCalcBinQty 2247
SourceKanbanCalcNumBins 2247
spreading 1426
standard deviation 2024
start date calculations 1376
StartDate 1298
statistical disaggregation
  data model 2098
statistical forecast
  with event management 2123
statistical forecast disaggregation 2103
statistical forecasting
  univariate models 2032
statistical forecasts
  configuration 2020
  disaggregation rates 2102
  error 2060
  historical data 2021
  importing 356, 2058
  outliers 2022
  validation 2030
Step-wise ARIMA 2039
String
  data type 144
substitutable allocations 1576
substitutable components 1567
substitute part 1545
substitute parts 1545, 1551
supply
  controls 1397
  netting 1332
  rescheduling 1335
  usage 1396
supply allotment
  by attribute 1827
supply expiry 1602
SupplyAvailableDate 1301
SupplyStartDate 1298
symmetric mean absolute percentage error 2069
system tables 133

T

table
  owner 937
table owners 937
table types
  calculated 133
  control 133
  notes 133
  system 133
  vector 138
tables
  about 149
  calculated 937
  control 669
  for demands[tables
    demands] 1397
  input 171
  notes 1269
  relationships 150
  See also Table Index[tables
    aaa] 25
  site calculations 1731
Takt Time 1376
  interchangeable parts 1549
tasks
  constraints 2172
  demand orders 2171
  duration 2176
  material requirements 2171
  project management 2167
  supply orders 2171
threshold
  order point 1351
Time
  data type 144
time buckets 2203
TimeUnit records 941, 2203
transaction sequence
  calculations 1491
  effective 1769
  example 1491
  in netting[transaction sequence
    netting] 1490
  in order prioritization[transaction sequence
    order prioritization] 1769
  updating 1769
transfer orders 1638
transfer part sources 1731
  lead time 1732
  UOM 1733
two-date planning 1806
  priority level 1805
  safety lead time 1807

U

unallocated scheduled receipts 1505
unconstrained quantity 1637
Undefined, date constants 1303
unit
  effectivity 1738
  netting 1334, 1746
  StartUnit number 1750
unit of measure
  in statistical forecasts 2027
  transfer part sources 1733
unit of measure conversions 933, 2028
unsatisfied demands 1420

UOM
transfer part sources 1733

V

variance error 2069
Vector Set
data type 144
vector set field 141
Vector tables 138
vendor managed parts, explosion 1341

W

weights
forecast category 2073
WhereConsumed table 2227
widgets
text 122
viewing help 122
work centers 1534
workbook help 120
show 120
workbooks
MLS Configuration 1873
workbooks, procedures with
accessing help 115
worksheet help
show 120
worksheet Help
show 120

Y

yeild
ignoring 1395
rounding 1395
yield 1391, 2135

World Headquarters

700 Silver Seven Road
Ottawa, Ontario
Canada K2V 1C3

Tel. +1 613.592.5780
Toll free. +1 866.236.3249
Support. +1 866.463.7877

Email. info@kinaxis.com

Want knowledge about RapidResponse from Kinaxis-vetted experts? Join the Kinaxis Knowledge Network!

https://knowledge.kinaxis.com